La Bibbia del Game Design

Lorenzo Vergata

November 12, 2026

Contents

I Seminari per il Game Designer **55**

0.1La cultura del feedback **57**

0.1.1 Il workshop e il relatore . . . 58
0.1.1.1 Formato e regole di partecipazione . . . 58
0.1.1.2 Il percorso professionale . . . 58
0.1.1.3 Struttura non vuol dire burocrazia . . . 59
0.1.2 Perché il lavoro è sempre lavoro di squadra . . . 59
0.1.2.1 Non si va in ufficio per fare amicizia . . . 59
0.1.2.2 Lo *useless genius* e il costo delle persone tossiche . . . 60
0.1.2.3 Il fit culturale . . . 60
0.1.3 Che cos'è il feedback . . . 61
0.1.3.1 Le prime risposte della classe . . . 61
0.1.3.2 Da che cosa dipende il modo di dare feedback . . . 61
0.1.3.3 Con chi si lavora meglio . . . 62
0.1.3.4 Il feedback come somma di piccoli miglioramenti . . . 62
0.1.4 Le cinque fasi di vita di un team . . . 62
0.1.4.1 Perché conoscerle . . . 62
0.1.4.2 Forming, storming, norming, performing, adjourning . . . 63
0.1.4.3 La fase che tutti saltano . . . 63
0.1.5 Domande sulle fasi del team . . . 64
0.1.5.1 Lo storming che ritorna . . . 64
0.1.5.2 Dove si passa la maggior parte del tempo . . . 64
0.1.5.3 Analizzare anche i successi . . . 64
0.1.5.4 L'autoanalisi personale . . . 65
0.1.5.5 Quando il team cambia di continuo . . . 65
0.1.5.6 Obiettivi chiari nei piccoli gruppi . . . 66
0.1.5.7 L'ambiente determina la qualità del prodotto . . . 66
0.1.6 Confrontarsi con chi è più bravo . . . 67
0.1.7 Domande sul lavoro nell'industria . . . 68
0.1.7.1 Presenza, remoto e trasferimenti . . . 68
0.1.7.2 Differenze culturali nel team . . . 69
0.1.7.3 Negoziare, discriminazione e stipendi . . . 69
0.1.7.4 Freelance e *work for hire* . . . 70
0.1.8 Il circolo virtuoso: feedback, miglioramento, fiducia . . . 70
0.1.8.1 Il triangolo . . . 70
0.1.8.2 Accettare un feedback non vuol dire fare come dice l'altro . . . 71
0.1.8.3 Fiducia e scadenze . . . 71
0.1.8.4 Domande sul dare e ricevere feedback . . . 71
0.1.8.5 Non portare la soluzione . . . 72
0.1.8.6 Amici, parenti e parole proibite . . . 72
0.1.8.7 Ego, muri e il peggior modo di scegliere un leader . . . 73
0.1.9 Perché la cultura conta: retention, costi e fiducia . . . 73

0.1.9.1 L'ambiente di lavoro e la retention . . . 73
0.1.9.2 Il costo di chi se ne va . . . 74
0.1.9.3 La cultura è forte quando si lavora per gli altri . . . 74
0.1.9.4 Il lavoro è valutato come opera d'insieme . . . 75
0.1.9.5 Il coraggio di chiedere aiuto . . . 75
0.1.9.6 Perché si dà un feedback . . . 75
0.1.10 Among Us: la caccia al traditore . . . 76
0.1.11 Il principio del default to trust . . . 77
0.1.11.1 La definizione . . . 77
0.1.11.2 L'esempio del ritardo al daily stand-up . . . 77
0.1.11.3 Condividere i successi, schermare i fallimenti . . . 78
0.1.12 Traiettorie di carriera e valutazione delle performance . . . 79
0.1.12.1 Quale traiettoria vorreste? . . . 79
0.1.12.2 La carriera non è una linea retta . . . 79
0.1.12.3 Lying, hiding, faking . . . 80
0.1.12.4 "Non ho capito" e la conferma delle istruzioni . . . 80
0.1.12.5 Il feedback più utile che abbiate ricevuto . . . 81
0.1.13 Come dare un feedback . . . 81
0.1.13.1 Concentrarsi sull'azione e sui suoi effetti . . . 81
0.1.13.2 Il pericolo degli ambienti solo positivi . . . 82
0.1.13.3 Che cosa fare . . . 82
0.1.13.4 Che cosa non fare . . . 83
0.1.13.5 Il primo feedback e il debriefing . . . 83
0.1.14 Come ricevere un feedback . . . 84
0.1.14.1 Il feedback positivo . . . 84
0.1.14.2 Il feedback accidentale . . . 84
0.1.15 Quando, quanto e con chi . . . 84
0.1.15.1 Frequentemente e recentemente . . . 84
0.1.15.2 Quanto dura un feedback . . . 85
0.1.15.3 I manager peggiori: "a che numero sto pensando?" . . . 85
0.1.15.4 Dare feedback a un superiore egocentrico . . . 85
0.1.16 Il feedback come regalo . . . 86
0.1.17 Domande e risposte finali . . . 87
0.1.18 Anticipazioni sulla seconda giornata . . . 89
Punti chiave del capitolo . . . 89

0.2 Tempo, idee e leadership **91**
0.2.1 Il programma della giornata . . . 92
0.2.2 Colloqui di lavoro: capire che cosa cerca l'azienda . . . 92
0.2.2.1 L'annuncio come prima fonte di informazione . . . 92
0.2.2.2 Quanti colloqui e con chi . . . 93
0.2.2.3 Le domande da fare al selezionatore . . . 93
0.2.2.4 Le domande che dicono molto sull'azienda . . . 94
0.2.2.5 La cifra minima mentale . . . 94
0.2.2.6 La scrematura iniziale e il portfolio . . . 94
0.2.2.7 Test pratici, da casa e con qualsiasi strumento . . . 95
0.2.2.8 Junior, mid e senior . . . 95
0.2.2.9 Gli strumenti di intelligenza artificiale . . . 96
0.2.3 Time management: pianificare la giornata . . . 96
0.2.3.1 Time management o prioritizzazione? . . . 96
0.2.3.2 La *must do list* . . . 96
0.2.3.3 To-do list e checklist . . . 97
0.2.3.4 La checklist: l'esempio dell'aviazione . . . 97
0.2.3.5 *First contact resolution* . . . 98

0.2.3.6 Non iniziare senza le informazioni . . . 99
0.2.3.7 Il costo delle interruzioni . . . 99
0.2.4 Prioritizzare: la matrice di Eisenhower . . . 99
0.2.5 Proteggere il proprio tempo . . . 100
0.2.5.1 Evitare gli sprechi: un limite che nessuno può insegnare . . . 100
0.2.5.2 Distrazioni volontarie e involontarie . . . 100
0.2.5.3 Il peggior nemico siamo noi . . . 101
0.2.5.4 Spezzettare i task grandi . . . 101
0.2.5.5 Accettare di dire di no . . . 101
0.2.5.6 Dire di no ai meeting . . . 102
0.2.5.7 Il multitasking non esiste . . . 103
0.2.5.8 La fine della giornata . . . 103
0.2.6 Domande sul time management . . . 104
0.2.7 Una parentesi: tempo libero, ferie e benessere . . . 105
0.2.7.1 Le tre condizioni del relatore . . . 105
0.2.8 Brainstorming: obiettivo e problemi tipici . . . 106
0.2.8.1 Perché il brainstorming riguarda tutti . . . 106
0.2.8.2 Generare idee, non decidere . . . 106
0.2.8.3 “Non esistono cattive idee”? . . . 106
0.2.8.4 I problemi più comuni . . . 107
0.2.9 Un metodo di brainstorming . . . 107
0.2.9.1 Preparazione e ambiente inclusivo . . . 107
0.2.9.2 Separare generazione e discussione: *brainwriting* e *rapid ideation* . . . 108
0.2.9.3 *Dot voting* . . . 108
0.2.9.4 Quanti voti e quali varianti . . . 109
0.2.9.5 Il moderatore e gli invitati . . . 109
0.2.10 Dalla votazione alla decisione: la matrice impatto–sforzo . . . 110
0.2.10.1 Massimo impatto, minimo sforzo . . . 110
0.2.10.2 Posizionare le idee, insieme e con precisione . . . 110
0.2.10.3 Leggere la matrice in diagonale . . . 110
0.2.10.4 Il ruolo del moderatore e dell’esperto . . . 111
0.2.10.5 Chi decide . . . 111
0.2.11 Consigli pratici e domande sul brainstorming . . . 112
0.2.11.1 Musica e cambio di scenario . . . 112
0.2.11.2 Domande . . . 112
0.2.12 Disagree and commit . . . 113
0.2.12.1 Quando la scelta non ci convince . . . 113
0.2.12.2 La demotivazione fa parte del lavoro . . . 113
0.2.12.3 Chi arriva a lavori iniziati . . . 114
0.2.13 Leading from within: perché parlare di leadership . . . 114
0.2.13.1 La ripresa del pomeriggio . . . 114
0.2.13.2 Il leader non è il più bravo . . . 114
0.2.13.3 Le carriere divise: manager o *subject matter expert* . . . 115
0.2.14 Il “momento Piero Angela”: la biologia del gruppo . . . 116
0.2.14.1 Perché una digressione scientifica . . . 116
0.2.14.2 Endorfine . . . 116
0.2.14.3 Dopamina . . . 116
0.2.14.4 Il villaggio di diecimila anni fa . . . 116
0.2.14.5 Serotonina . . . 117
0.2.14.6 Ossitocina . . . 117
0.2.14.7 Cortisolo . . . 118
0.2.15 Domande sulla biologia del gruppo . . . 118
0.2.16 La piramide della fiducia . . . 119
0.2.16.1 La leadership non è un titolo . . . 119

0.2.16.2 Alla base: l'integrità morale . . . 119
0.2.16.3 Al centro: l'affidabilità . . . 120
0.2.16.4 In cima: la leadership, cioè dare l'esempio . . . 120
0.2.17 Prendersi cura del gruppo . . . 121
0.2.17.1 La soluzione: più leader possibile . . . 121
0.2.17.2 Il villaggio di oggi . . . 122
0.2.17.3 Smettere di dire "ci penserà qualcun altro" . . . 122
0.2.17.4 Domande sul "me ne occupo io" . . . 122
0.2.18 Se si diventa manager . . . 123
0.2.18.1 Meriti, colpe e *big picture* . . . 123
0.2.18.2 Il ruolo è più grande della persona . . . 123
0.2.18.3 Il tetto della bravura del capo . . . 123
0.2.18.4 Celebrare i successi e i fallimenti . . . 124
0.2.18.5 Quanto ci vuole? . . . 124
0.2.18.6 L'ultima slide . . . 125
0.2.19 Domande e risposte finali . . . 125
0.2.19.1 Leadership e organizzazione del lavoro . . . 125
0.2.19.2 Condividere fallimenti e successi in pratica . . . 126
0.2.19.3 Cultura aziendale e formazione . . . 127
0.2.19.4 Performance e persone in difficoltà . . . 127
0.2.19.5 Carriera, colloqui e candidature . . . 129
0.2.20 Chiusura del workshop . . . 132
Punti chiave del capitolo . . . 132

0.3 Agile e Scrum nella pratica **135**
0.3.1 Apertura e presentazione degli ospiti . . . 136
0.3.1.1 Una registrazione andata persa . . . 136
0.3.1.2 Chi sono gli ospiti e perché sono qui . . . 136
0.3.2 Simone: il punto di vista del producer . . . 137
0.3.2.1 Chi è il relatore . . . 137
0.3.2.2 Una lezione partecipativa . . . 137
0.3.2.3 Un piccolo sondaggio . . . 137
0.3.3 Il mindset Agile . . . 138
0.3.3.1 Agile non è un framework . . . 138
0.3.3.2 I quattro valori, e perché non vanno presi alla lettera . . . 138
0.3.3.3 Altri mindset e i framework . . . 138
0.3.4 Le sprint e le riunioni . . . 139
0.3.4.1 Il libro di Scrum . . . 139
0.3.4.2 Che cos'è una sprint . . . 139
0.3.4.3 La regola d'oro: meno riunioni possibili . . . 139
0.3.4.4 Una riunione deve essere rapida . . . 139
0.3.4.5 La sprint come unità di tempo . . . 140
0.3.5 Le cerimonie e la regola della consistency . . . 140
0.3.5.1 Le cerimonie di Scrum secondo Simone . . . 140
0.3.5.2 Consistency: una cerimonia saltata fa cadere le altre . . . 140
0.3.5.3 Perché il daily scrum non funziona più . . . 141
0.3.5.4 Perché il planning poker rallenta tutto . . . 141
0.3.5.5 Il punto chiave: crearsi il proprio framework . . . 141
0.3.6 Il backlog . . . 142
0.3.6.1 Una "rottura di scatole" inevitabile . . . 142
0.3.6.2 Il backlog a scaglioni . . . 142
0.3.6.3 Le epic . . . 142
0.3.6.4 Ogni task deve avere un obiettivo chiaro . . . 142
0.3.7 La velocity . . . 143

0.3.7.1 Il peso dei task . . . 143
0.3.7.2 Mai le ore . . . 143
0.3.7.3 La sequenza di Fibonacci . . . 143
0.3.7.4 Che cos'è la velocity e perché serve . . . 143
0.3.7.5 Come calcolarla in pratica . . . 144
0.3.7.6 Come assegnare i punteggi . . . 144
0.3.7.7 Che cosa blocca che cosa . . . 144
0.3.8 Ricapitolando: come nasce una sprint . . . 144
0.3.8.1 Ascoltare il team . . . 145
0.3.8.2 Nessun framework va bene al 100% . . . 145
0.3.8.3 La nota "smielata": la fiducia . . . 145
0.3.9 Domande a Simone . . . 146
0.3.9.1 La dimensione del team allunga le riunioni? . . . 146
0.3.9.2 Quanto deve imporsi lo scrum master? Il timekeeper . . . 146
0.3.9.3 E se il problema blocca tutti? . . . 146
0.3.9.4 E se mancano delle persone? . . . 146
0.3.9.5 Quando è la persona a non adattarsi . . . 147
0.3.9.6 Come si calcola la velocity, in pratica . . . 147
0.3.9.7 I problemi più gravi incontrati con Agile . . . 148
0.3.9.8 Che cosa rispondere a chi dice che le riunioni sono inutili . . . 148
0.3.9.9 I saluti a Simone . . . 148
0.3.10 Cristian: il punto di vista del game designer e project manager . . . 149
0.3.10.1 Chi è il relatore . . . 149
0.3.10.2 Come è strutturata la sua parte . . . 149
0.3.10.3 Le premesse . . . 149
0.3.11 Il caso di studio: *L'Inf* . . . 150
0.3.11.1 Le caratteristiche del gioco . . . 150
0.3.11.2 Un "trauma" di produzione: il cambio di team . . . 151
0.3.11.3 Perché raccontare tutto questo . . . 151
0.3.12 Il loop di Scrum . . . 151
0.3.13 La stesura del backlog . . . 152
0.3.13.1 Un backlog per milestone . . . 152
0.3.13.2 Da dove vengono i task . . . 152
0.3.13.3 Come organizzare i task . . . 152
0.3.13.4 Come scrivere i task e dove . . . 153
0.3.13.5 Non prevedere tutto: la flessibilità . . . 153
0.3.13.6 Quanto dettaglio? . . . 154
0.3.14 La selezione dei task . . . 154
0.3.14.1 Quali task . . . 154
0.3.14.2 Quanti task . . . 155
0.3.14.3 Definire bene il goal . . . 155
0.3.14.4 Bonus: progettare il ritmo dello sviluppo . . . 155
0.3.15 L'implementazione: controlli, blocker e feedback . . . 156
0.3.15.1 Il check dei progressi . . . 156
0.3.15.2 Agire subito sui blocker . . . 156
0.3.15.3 Analizzare i blocker prima di risolverli . . . 156
0.3.15.4 Bonus: i feedback costanti . . . 157
0.3.16 Il calcolo della velocity . . . 157
0.3.16.1 Confrontare il fatto con lo stimato . . . 157
0.3.16.2 Più di un numero . . . 158
0.3.16.3 Bonus: mai tornare indietro . . . 159
0.3.17 Il tuning della produzione . . . 159
0.3.17.1 Che cosa e quando . . . 159
0.3.17.2 Su che cosa basarsi . . . 160

0.3.17.3 Flessibilità anche nella struttura dei documenti . . . 160
0.3.17.4 Evitare il perfezionismo . . . 160
0.3.18 Domanda: come comunicare un taglio al team . . . 161
0.3.19 Il team management . . . 161
0.3.19.1 Senza team non esiste Scrum . . . 161
0.3.19.2 Una produzione senza stress . . . 161
0.3.19.3 Mai la velocity sui singoli . . . 162
0.3.19.4 Primo aspetto: adattare Scrum al team . . . 162
0.3.19.5 Secondo aspetto: tutti devono conoscere il mindset . . . 162
0.3.19.6 Il post-mortem . . . 163
0.3.19.7 La sincerità . . . 163
0.3.20 Chiusura della lezione . . . 164
Punti chiave del capitolo . . . 164

0.4 Publisher e producer **167**
0.4.1 Prima dell'inizio: il percorso degli studenti . . . 168
0.4.2 Chi è il relatore . . . 169
0.4.2.1 La domanda della zia a Natale . . . 169
0.4.3 Che cosa fa un publisher . . . 169
0.4.3.1 Una casa editrice per videogiochi . . . 169
0.4.3.2 Come si cercano i giochi . . . 170
0.4.3.3 Che cosa offre il publisher . . . 170
0.4.3.4 L'obiettivo: il *magic* . . . 170
0.4.4 Come funzionano i soldi: i tre malloppi . . . 171
0.4.4.1 Development: lo stipendio del team . . . 171
0.4.4.2 Service: tutto il lavoro che serve al gioco . . . 171
0.4.4.3 Marketing: promuovere il gioco . . . 171
0.4.4.4 Per saperne di più: il contratto pubblico . . . 172
0.4.5 Che cosa fa un producer . . . 172
0.4.5.1 Un asterisco e una domanda alla classe . . . 172
0.4.5.2 La definizione di Michele . . . 172
0.4.5.3 L'anello tra due catene . . . 172
0.4.5.4 Le quattro caratteristiche del producer . . . 173
0.4.6 Il producer in Raw Fury . . . 173
0.4.6.1 Fuori dal team . . . 173
0.4.6.2 I *production duo* . . . 173
0.4.6.3 Più progetti, a stadi diversi . . . 174
0.4.6.4 Come si dividono i compiti . . . 174
0.4.7 Demo ed eventi . . . 174
0.4.7.1 Ogni evento ha il suo pubblico . . . 174
0.4.7.2 Che cosa mettere nella demo . . . 175
0.4.7.3 Le restrizioni degli eventi . . . 175
0.4.7.4 Valutazione ed esserci . . . 175
0.4.8 Quality assurance . . . 175
0.4.9 Console porting . . . 176
0.4.10 Una domanda: fermare un reparto e il debito tecnico . . . 176
0.4.11 Localizzazione . . . 176
0.4.11.1 Il riepilogo prima della pausa . . . 177
0.4.12 Dopo la pausa: cinque consigli del producer . . . 177
0.4.12.1 1. Fate riunioni e parlate dei problemi . . . 177
0.4.12.2 2. Le stime sono sempre sbagliate, ma meglio di niente . . . 177
0.4.12.3 3. Mettetevi del margine: il consiglio del salumiere . . . 178
0.4.12.4 4. Non aspettate l'ultimo momento per fare le build . . . 178
0.4.12.5 5. Tagliare feature non è una vergogna . . . 178

0.4.13 Domande: compiti, bug e decisioni . . . 179
0.4.13.1 Il producer assegna dei compiti? . . . 179
0.4.13.2 Come si decide il budget per i bug? . . . 179
0.4.13.3 Il budget si ridiscute dopo il successo? . . . 179
0.4.13.4 Chi prende queste decisioni? . . . 179
0.4.13.5 Le lingue accorpate . . . 180
0.4.14 Domande: soldi, contratti e potere decisionale . . . 180
0.4.14.1 Qual è il malloppo più grande? . . . 180
0.4.14.2 Che cosa succede se il rapporto si interrompe? . . . 180
0.4.14.3 Soci al 50% . . . 180
0.4.14.4 Quanto può decidere il publisher sulle feature? . . . 181
0.4.15 Domande: progetti che si fermano, VR e DLC . . . 181
0.4.15.1 Progetti che non vale la pena continuare . . . 181
0.4.15.2 E se il gioco cambia strada? . . . 181
0.4.15.3 I giochi in VR . . . 182
0.4.15.4 Ridiscutere il budget dopo l'uscita . . . 182
0.4.15.5 Conviene presentarsi con i piani per il dopo? . . . 182
0.4.16 Domande: come si sceglie un progetto . . . 182
0.4.16.1 Primo: quando esce . . . 182
0.4.16.2 Secondo: il gioco . . . 182
0.4.16.3 La *due diligence* sulle persone . . . 183
0.4.17 Domande: finanziamento, calendario e lingue . . . 183
0.4.17.1 Da dove vengono i soldi del publisher . . . 183
0.4.17.2 Se il calendario è pieno, quando ripresentarsi? . . . 183
0.4.17.3 Il publisher cerca giochi di un certo tipo? . . . 184
0.4.17.4 Un gioco legato a una lingua . . . 184
0.4.17.5 Un'offerta più bassa del budget . . . 184
0.4.17.6 Perché niente *early access*? . . . 184
0.4.18 Chiusura . . . 185
Punti chiave del capitolo . . . 185

0.5 Il game producer **187**
0.5.1 L'avvio del collegamento . . . 188
0.5.2 Chi è il relatore . . . 188
0.5.2.1 Il ruolo e il taglio del seminario . . . 189
0.5.3 I ruoli in un team di sviluppo . . . 189
0.5.3.1 Che cosa vedremo . . . 190
0.5.4 Miti e aspettative sul game producer . . . 190
0.5.4.1 "È un lavoro meraviglioso" . . . 190
0.5.4.2 Il game producer crea i videogiochi? . . . 190
0.5.4.3 Il game producer è un game designer? . . . 191
0.5.5 Come è cambiato il ruolo nel tempo . . . 191
0.5.6 Altri miti da sfatare . . . 192
0.5.6.1 Non abbiate paura . . . 192
0.5.6.2 L'inglese . . . 192
0.5.6.3 Cominciare dal mobile . . . 192
0.5.7 Che cosa cercano gli investitori e il mercato . . . 193
0.5.7.1 Poche idee radicali, IP "per sempre" . . . 193
0.5.7.2 I contenuti a pagamento: l'esempio di Street Fighter . . . 193
0.5.7.3 Il *game as a service* . . . 193
0.5.7.4 La personalizzazione al posto dello stile . . . 194
0.5.8 Prime domande dalla chat . . . 194
0.5.8.1 Il producer fa anche project management? . . . 194
0.5.8.2 Chi ha l'ultima parola? . . . 194

0.5.8.3 Sony, il Game Pass e le scelte audaci . . . 195
0.5.9 Investitori, abbonamenti e offerta appiattita . . . 195
0.5.9.1 Una domanda più varia dell'offerta . . . 195
0.5.9.2 Il dibattito sui giocatori casual . . . 196
0.5.10 I producer delle nicchie . . . 196
0.5.11 Le testimonianze degli studenti . . . 196
0.5.11.1 Uri: producer in uno studio di quindici persone . . . 196
0.5.11.2 Giuseppe: un team piccolo e la passione per i classici . . . 197
0.5.11.3 Commodore, l'Italia e il networking . . . 198
0.5.11.4 Luca: il fulcro di un progetto da studenti . . . 198
0.5.12 Il game producer è un product manager? . . . 199
0.5.12.1 Il pitch e l'arte di andare oltre il pitch . . . 199
0.5.13 Il game producer è un project manager? . . . 199
0.5.13.1 L'esempio delle piattaforme . . . 200
0.5.14 Il game producer è un client manager . . . 200
0.5.14.1 Rapporti sbilanciati . . . 200
0.5.14.2 Costruire un rapporto paritario . . . 201
0.5.14.3 Il modello familiare italiano . . . 201
0.5.14.4 Trasformare tutto in un vantaggio . . . 201
0.5.15 Agile dal punto di vista del producer . . . 201
0.5.15.1 Product owner e project manager . . . 202
0.5.15.2 Quanto vale questo prodotto? . . . 202
0.5.15.3 Agile non è una Bibbia . . . 202
0.5.15.4 Gli sprint . . . 203
0.5.15.5 Product owner e producer: un binomio . . . 203
0.5.15.6 Il product manager secondo Agile . . . 204
0.5.15.7 Il team Agile: i vantaggi . . . 204
0.5.15.8 L'esperienza di Luca con gli sprint . . . 205
0.5.15.9 Altri vantaggi . . . 206
0.5.15.10 Gli svantaggi . . . 206
0.5.16 Il Kanban del producer . . . 207
0.5.16.1 Le colonne e il livello di dettaglio . . . 207
0.5.16.2 Dove va la colonna del testing? . . . 207
0.5.16.3 Colli di bottiglia e senso di avanzamento . . . 208
0.5.17 Il framework Scrum . . . 208
0.5.17.1 Le caratteristiche dello Scrum . . . 208
0.5.17.2 Lo schema 3-3-5-5 . . . 209
0.5.17.3 Quando gli sprint si infittiscono . . . 209
0.5.17.4 I meeting e la regola dei venti minuti . . . 210
0.5.17.5 I cinque valori . . . 210
0.5.18 Producer famosi . . . 211
0.5.19 La visione . . . 212
0.5.19.1 La visione dello studio . . . 212
0.5.19.2 Imparate le metodologie, poi abbandonatele . . . 212
0.5.19.3 Rimanere videogiocatori . . . 213
0.5.19.4 Costruitevi il vostro Kanban . . . 213
0.5.20 Creare opportunità: publisher e interlocutori . . . 213
0.5.20.1 Scegliere il publisher . . . 214
0.5.20.2 Che cosa è compito del producer . . . 214
0.5.20.3 La legge di Murphy . . . 214
0.5.21 Trovare il developer giusto . . . 215
0.5.22 L'attitudine del game producer . . . 215
0.5.22.1 La guida del team . . . 215
0.5.22.2 Fare pratica davanti alla telecamera . . . 216

0.5.22.3 Dedizione, comunicazione, umiltà . . . 216
0.5.23 B2B e B2C . . . 216
0.5.24 La gamification . . . 217
0.5.25 Il panorama internazionale . . . 217
0.5.25.1 Un buon momento, ma non facile . . . 218
0.5.26 Il "problema italiano" . . . 218
0.5.26.1 Normativa, burocrazia, conduzione familiare . . . 218
0.5.26.2 Situazioni tipiche . . . 219
0.5.27 Ultimi consigli . . . 220
0.5.28 Chiusura . . . 220
Punti chiave del capitolo . . . 221

0.6 Il mindset del game designer 223
0.6.1 L'avvio e le presentazioni . . . 224
0.6.1.1 Chi partecipa . . . 224
0.6.2 Chi è il relatore . . . 225
0.6.2.1 Perché questo workshop . . . 226
0.6.3 La struttura del workshop . . . 226
0.6.3.1 La bibliografia . . . 227
0.6.4 Scrittori, artisti e game designer: i problemi . . . 227
0.6.4.1 Il caos non è una strategia . . . 228
0.6.4.2 Out of scope e out of the box . . . 228
0.6.4.3 I problemi di oratoria . . . 228
0.6.4.4 Un mindset che fa da argine . . . 228
0.6.5 Perché serve un mindset: i bias . . . 229
0.6.5.1 I pregiudizi sul game designer . . . 229
0.6.5.2 I pregiudizi sul game design . . . 230
0.6.6 Il game design come metodologia . . . 230
0.6.6.1 Le competenze "tangenziali" . . . 230
0.6.6.2 Che cos'è il mindset . . . 230
0.6.7 Che cosa vuole un game designer: l'esperienza . . . 231
0.6.7.1 La lente dell'esperienza . . . 231
0.6.7.2 Strumenti, tecnologie, ricerca e feedback . . . 231
0.6.8 Idea e visione . . . 232
0.6.8.1 L'obiettivo superordinato . . . 232
0.6.8.2 Non tutti i giochi sono per tutti . . . 233
0.6.9 Il gioco come comunicazione differita . . . 233
0.6.9.1 La definizione di gioco del relatore . . . 233
0.6.10 Design principle, pilastri e iterazione . . . 234
0.6.10.1 I pilastri . . . 234
0.6.10.2 Iterare . . . 234
0.6.10.3 Che cosa portarsi a casa . . . 235
0.6.11 L'MDA e l'ADMDA . . . 235
0.6.11.1 Le definizioni del relatore . . . 235
0.6.11.2 Le due direzioni . . . 235
0.6.11.3 Il caso di The Last of Us . . . 236
0.6.11.4 L'ADMDA . . . 236
0.6.12 Gli strumenti della prima fase . . . 237
0.6.12.1 Gli strumenti di brainstorming e di visualizzazione . . . 237
0.6.13 La pipeline e il micromanagement . . . 238
0.6.13.1 Che cos'è il micromanagement . . . 238
0.6.14 Il giocatore e i player type . . . 239
0.6.15 L'High Vision Document e il one page design . . . 239
0.6.15.1 Il one page design . . . 240

0.6.16 Domande durante la pausa . . . 240
0.6.16.1 Da dove si comincia un High Vision Document? . . . 240
0.6.16.2 Termini chiari per tutto il team . . . 240
0.6.16.3 Must have e nice to have . . . 240
0.6.16.4 Concretamente, che cos'è una meccanica? . . . 241
0.6.17 Le meccaniche . . . 241
0.6.17.1 Che cosa può fare Mario . . . 241
0.6.17.2 Vincoli e rinforzi . . . 242
0.6.17.3 Meccaniche filtrate dall'estetica . . . 242
0.6.17.4 Verbi e regole . . . 242
0.6.18 Loop, dinamiche e sistemi . . . 243
0.6.18.1 Il loop e il core loop . . . 243
0.6.18.2 Il core loop di Ticket to Ride . . . 243
0.6.18.3 Le dinamiche emergenti . . . 243
0.6.18.4 Le reazioni e l'action feedback . . . 244
0.6.18.5 I sistemi . . . 244
0.6.18.6 La manipolazione . . . 245
0.6.19 Noia, frustrazione, flow e feedback loop . . . 245
0.6.19.1 Il pacing . . . 245
0.6.19.2 Il flow . . . 245
0.6.19.3 I feedback loop . . . 246
0.6.19.4 Gli habit loop . . . 246
0.6.20 Un mindset di caos e tre idee da portare a casa . . . 246
0.6.21 Le estetiche: le superfici che guidano . . . 247
0.6.21.1 Journey . . . 247
0.6.21.2 Glossari . . . 247
0.6.21.3 Le regole e le house rule . . . 247
0.6.21.4 Tutorial e dialoghi . . . 248
0.6.22 Icone e idiomi . . . 248
0.6.22.1 Icone nei tutorial e ancore visive . . . 249
0.6.23 Le emozioni e i piaceri . . . 249
0.6.23.1 Il framework 6-11 . . . 249
0.6.23.2 L'esempio di Super Mario . . . 250
0.6.23.3 Gli otto piaceri di LeBlanc . . . 250
0.6.23.4 Una discussione sull'avidità . . . 250
0.6.24 Comunicare con le immagini: ambienti e forme . . . 251
0.6.24.1 Cerchio, quadrato, triangolo . . . 251
0.6.24.2 E la musica . . . 251
0.6.25 Il workshop della seconda giornata . . . 251
0.6.25.1 Che cosa bisogna fare . . . 251
0.6.26 Il riepilogo del mindset . . . 252
0.6.27 Le domande finali . . . 252
0.6.27.1 Un piccolo studio e la visione condivisa . . . 252
0.6.27.2 Eventi, fiere e playtest . . . 253
0.6.27.3 I grandi team e il crunch . . . 253
0.6.27.4 Quanta documentazione? . . . 253
0.6.27.5 Allargare il team e le soft skill . . . 254
0.6.27.6 Un publisher cinese e i pattern culturali . . . 254
0.6.27.7 Le domande di Luca: gli otto piaceri, l'MDA e il flusso di lavoro . . . 255
0.6.27.8 One page e High Vision Document sono la stessa cosa? . . . 255
0.6.27.9 I compiti e la partenza di Michele . . . 256
0.6.27.10 Il game designer durante lo sviluppo . . . 256
0.6.27.11 Gruppi e aspettative . . . 256
0.6.28 Chiusura . . . 256

Punti chiave del capitolo . . . 257

0.7 Hackerare un gioco **259**
0.7.1 L'attesa e gli strumenti di lavoro . . . 260
0.7.1.1 Servono strumenti particolari? . . . 260
0.7.1.2 Miro e le altre lavagne digitali . . . 260
0.7.1.3 Le slide di riepilogo . . . 261
0.7.2 Il programma della giornata . . . 261
0.7.2.1 I passi . . . 261
0.7.2.2 Che cosa ci si aspetta . . . 262
0.7.3 I tre giochi da vicino . . . 262
0.7.3.1 Lasers & Feelings . . . 262
0.7.3.2 Ticket to Ride . . . 263
0.7.3.3 Super Mario Bros. . . . 264
0.7.3.4 Domande prima di partire: i punti dell'analisi e la USP . . . 264
0.7.4 I gruppi . . . 265
0.7.5 Primo passo: l'esempio di Daniele e Michele . . . 265
0.7.5.1 Le azioni . . . 265
0.7.5.2 Il core loop . . . 266
0.7.5.3 I power up . . . 266
0.7.5.4 Le emozioni: una correzione a Dillon . . . 266
0.7.6 Primo passo: le analisi dei gruppi . . . 267
0.7.6.1 Sessione 1: Super Mario . . . 267
0.7.6.2 L'orgoglio è un'emozione emergente . . . 267
0.7.6.3 Miro, ancora . . . 268
0.7.6.4 Avidità o raccolta? . . . 268
0.7.6.5 Sessione 2: Lasers & Feelings . . . 268
0.7.6.6 Partire da una board vuota . . . 268
0.7.6.7 Sessione 3: Super Mario . . . 269
0.7.6.8 Sessione 4: Ticket to Ride . . . 269
0.7.6.9 Pescare è un'azione sola . . . 269
0.7.6.10 Dove sta l'avidità in Ticket to Ride . . . 269
0.7.6.11 I colori dei vagoni e i binari . . . 270
0.7.7 Secondo passo: visione, esperienza desiderata e pubblico . . . 270
0.7.7.1 Come si conosce il pubblico . . . 270
0.7.7.2 Un esempio sopra le righe . . . 271
0.7.8 Secondo passo: il Bromance Mario . . . 271
0.7.8.1 La visione . . . 271
0.7.8.2 L'esperienza voluta . . . 272
0.7.8.3 L'audience e i riferimenti . . . 272
0.7.8.4 Il player type . . . 272
0.7.8.5 Genere, gimmick e mood . . . 273
0.7.8.6 La presentazione . . . 273
0.7.9 Secondo passo: le visioni dei gruppi . . . 274
0.7.9.1 Sessione 1: Creepy Mario . . . 274
0.7.9.2 I falsi amici di Bartle . . . 274
0.7.9.3 Sessione 2: Thunder and Prophecies . . . 274
0.7.9.4 Sessione 3: Gloomy Mario . . . 275
0.7.9.5 Sessione 4: un Ticket to Ride con i saccheggiatori . . . 275
0.7.10 Terzo passo: le meccaniche . . . 276
0.7.10.1 I glossari . . . 276
0.7.10.2 Dal D6 al D12: dadi e probabilità . . . 276
0.7.10.3 Che cosa consegnare . . . 277
0.7.10.4 Icone e idiomi nel glossario . . . 277

0.7.10.5 Dove ci si ferma . . . 277
0.7.11 Terzo passo: le meccaniche del Bromance Mario . . . 277
0.7.11.1 Camminare e colpire, invece di correre e saltare . . . 278
0.7.11.2 Colpire o lanciare . . . 278
0.7.11.3 Il pastrano e il duck . . . 278
0.7.11.4 Le tracce degli animali . . . 279
0.7.11.5 I richiami degli animali . . . 279
0.7.11.6 Le regole . . . 279
0.7.11.7 Le tre avvertenze . . . 279
0.7.11.8 Nella stanza principale . . . 280
0.7.12 Le presentazioni finali . . . 280
0.7.12.1 Il Bromance Mario . . . 280
0.7.12.2 Creepy Mario: la barra del panico . . . 280
0.7.12.3 Gloomy Mario: la torcia, la pala e Bowser . . . 281
0.7.12.4 La zona di scambio e le convenzioni di genere . . . 282
0.7.13 I consigli conclusivi . . . 282
0.7.13.1 Non perdersi . . . 282
0.7.14 Il portfolio . . . 283
0.7.14.1 La licenza . . . 283
0.7.15 Chiusura . . . 283
Punti chiave del capitolo . . . 283

0.8 Esplorare il metaverso **285**
0.8.1 L'avvio del collegamento . . . 286
0.8.2 L'evoluzione del web . . . 286
0.8.2.1 Il web 1.0: un giornale online, in sola lettura . . . 286
0.8.2.2 Il web 2.0: la comunicazione diventa bidirezionale . . . 287
0.8.2.3 Il web 3.0: una realtà non ancora cristallizzata . . . 287
0.8.2.4 Che cos'è, allora, il web 3.0 . . . 287
0.8.3 Il metaverso e le esigenze nuove . . . 288
0.8.3.1 Il browser dimenticato . . . 288
0.8.4 Metaverso e realtà virtuale . . . 288
0.8.4.1 Il nuovo casco di Sony . . . 288
0.8.4.2 Una domanda per e-mail: un metaverso aperto è un'utopia? . . . 289
0.8.4.3 Gli albori di Internet: siamo ancora in sperimentazione . . . 289
0.8.5 La realtà virtuale oltre il videogioco . . . 290
0.8.5.1 L'educazione a distanza: una classe come piccolo metaverso . . . 290
0.8.5.2 La medicina: operazioni a distanza . . . 290
0.8.6 Meta e la sua visione del metaverso . . . 290
0.8.6.1 Un metaverso per una nuova economia . . . 291
0.8.6.2 Il limite logico: parlare del paradiso . . . 291
0.8.7 Le domande ai partecipanti . . . 291
0.8.8 Che cos'è il metaverso per i partecipanti . . . 292
0.8.8.1 Giada: un luogo sociale . . . 292
0.8.8.2 Enrico: una definizione tecnica . . . 292
0.8.8.3 Marcello: una sorta di Matrix . . . 292
0.8.8.4 Luca: la realtà aumentata . . . 293
0.8.8.5 Dario: un mondo parallelo . . . 293
0.8.8.6 Francesco: il nostro io digitale . . . 293
0.8.8.7 Leandro: una rete di mondi virtuali in 3D . . . 294
0.8.8.8 Roberto: servizi creati dagli utenti, VRChat e Ready Player One . . . 294
0.8.8.9 Matteo: il pessimista . . . 294
0.8.8.10 Amici artificiali . . . 295
0.8.8.11 Un luogo dove imparare in sicurezza e il lavoro meno tossico . . . 295

0.8.8.12 Gioele: perché preferire una vita nel metaverso? . . . 295
0.8.9 Rischi e speranze dalla chat . . . 296
0.8.9.1 Controllo e profilazione . . . 296
0.8.9.2 Ancora Roberto: la scena del ballo . . . 296
0.8.9.3 Giada: che si integri senza troppi danni . . . 296
0.8.9.4 Leandro: la regolamentazione . . . 296
0.8.9.5 Fortnite: un danno all'industria . . . 296
0.8.9.6 Il metaverso di Meta: nessuno ci crede . . . 296
0.8.10 Web 3.0, metaverso e token . . . 297
0.8.10.1 Il token come abbecedario del metaverso . . . 297
0.8.10.2 Gli outfit per l'avatar . . . 297
0.8.10.3 Un avatar unico per tutti i giochi . . . 297
0.8.10.4 Il prezzo dell'alter ego . . . 298
0.8.10.5 L'educazione alla customizzazione . . . 298
0.8.11 Dalla customizzazione al "pay to exist" . . . 298
0.8.11.1 L'alfabetizzazione al metaverso . . . 298
0.8.11.2 L'allarme . . . 298
0.8.11.3 Dal pay to win al pay to exist . . . 299
0.8.12 La Human Computer Interface . . . 299
0.8.12.1 Dall'elaboratore in attesa all'entità semisenziente . . . 299
0.8.12.2 Breve storia dell'interazione . . . 299
0.8.12.3 Verso un'interazione più umana . . . 300
0.8.12.4 Lo spatial computing . . . 300
0.8.13 Snow Crash e la nascita del metaverso . . . 300
0.8.13.1 La corporazione e il real estate . . . 301
0.8.14 Comprare immobili nel metaverso . . . 301
0.8.15 Oltre la realtà . . . 302
0.8.15.1 Le esperienze che puoi fare solo lì . . . 302
0.8.15.2 Il passo esaltante di Snow Crash . . . 302
0.8.15.3 Il mondo non è al tuo servizio . . . 302
0.8.15.4 Il tempo sottratto alla vita . . . 302
0.8.16 Votare nel metaverso? . . . 302
0.8.16.1 Un intervento a voce . . . 303
0.8.16.2 Metaversi come pianeti alieni . . . 303
0.8.17 I tre modelli del metaverso . . . 303
0.8.17.1 Il gaming engine . . . 303
0.8.17.2 I virtual environment . . . 304
0.8.17.3 Le virtual economy . . . 304
0.8.18 Gli NFT . . . 305
0.8.18.1 Chi vuole davvero gli NFT? . . . 305
0.8.18.2 A che cosa possono servire . . . 305
0.8.19 I quattro layer del metaverso . . . 306
0.8.19.1 La blockchain e le parti tralasciate . . . 306
0.8.20 Perché le aziende investono . . . 306
0.8.21 Buon viaggio nel metaverso . . . 307
Punti chiave del capitolo . . . 307

0.9 Il game audio **309**
0.9.1 L'avvio e le presentazioni . . . 310
0.9.1.1 Aspettando i ritardatari . . . 310
0.9.1.2 Perché un giro di presentazioni . . . 310
0.9.1.3 I partecipanti . . . 311
0.9.2 Il relatore e il suo studio . . . 312
0.9.2.1 DP Studios: uno studio verticale . . . 312

0.9.2.2 Dall'ingegneria a Ubisoft . . . 312
0.9.2.3 Gli anni in Ubisoft . . . 312
0.9.2.4 Primo consiglio: la cura del proprio brand . . . 313
0.9.2.5 Secondo spunto: otto progetti, tutti italiani . . . 313
0.9.3 Una parentesi: la contrazione del mercato . . . 313
0.9.4 Più di metà dell'esperienza: la citazione di Spielberg . . . 314
0.9.5 I tre elementi di importanza dell'audio . . . 315
0.9.6 Il potere evocativo . . . 315
0.9.6.1 Il trailer di *Gears of War* . . . 315
0.9.6.2 Un'idea mai realizzata . . . 316
0.9.6.3 *Assassin's Creed*: la musica che si apre . . . 316
0.9.6.4 *Manhunt*: l'oppressione senza musica . . . 316
0.9.6.5 Altri esempi: *Alien: Isolation*, *Silent Hill*, *Scanner Sombre* . . . 317
0.9.7 La caratterizzazione . . . 317
0.9.7.1 Una simulazione, non una sonorizzazione . . . 318
0.9.7.2 Una città africana: superfici, riverberi, oggetti . . . 318
0.9.7.3 *Assassin's Creed II*: la Venezia di Ezio . . . 318
0.9.8 Il supporto al gameplay . . . 319
0.9.8.1 I passi della guardia . . . 319
0.9.8.2 La telecamera di sorveglianza e il microfono sulla camera . . . 319
0.9.8.3 La musica che dice se sei in combattimento . . . 319
0.9.8.4 Che cosa sa il motore di gioco? . . . 319
0.9.8.5 Il giocatore che sfrutta la musica . . . 320
0.9.9 Breve storia dell'audio nei videogiochi . . . 320
0.9.9.1 Perché la storia . . . 321
0.9.9.2 Tre periodi . . . 321
0.9.10 Gli anni Ottanta: chip, sintesi e linguaggio macchina . . . 321
0.9.10.1 Nasce la produzione audio . . . 321
0.9.10.2 Team piccoli e kit costosissimi . . . 321
0.9.10.3 Registri e forme d'onda . . . 322
0.9.10.4 Il *voice stealing* . . . 322
0.9.10.5 Il *voice stealing* oggi . . . 322
0.9.10.6 La pausa . . . 322
0.9.10.7 Il programmatore-compositore . . . 322
0.9.10.8 Rob Hubbard e le macro in linguaggio macchina . . . 323
0.9.10.9 Il *tracker* . . . 323
0.9.10.10 Gli editor di suono: un'onda quadra su Game Boy Advance . . . 324
0.9.11 Gli anni Novanta: campioni, MIDI e CD . . . 324
0.9.11.1 La wavetable . . . 324
0.9.11.2 Il MIDI e la separazione dei ruoli . . . 324
0.9.11.3 Il CD della PlayStation . . . 325
0.9.11.4 Un periodo ibrido . . . 325
0.9.11.5 Dal linguaggio macchina al C e agli script . . . 325
0.9.12 Dal 2000 a oggi: il livello del cinema . . . 325
0.9.13 Il team audio . . . 326
0.9.13.1 Perché conoscere i ruoli . . . 326
0.9.13.2 Il team dei sogni e il team reale . . . 326
0.9.14 L'audio director . . . 326
0.9.15 Il sound designer . . . 327
0.9.15.1 Le limitazioni: qualità e lunghezza . . . 327
0.9.15.2 Le code dei suoni e le campane . . . 328
0.9.15.3 Suoni asciutti e riverbero in tempo reale . . . 328
0.9.15.4 Zero silenzio in testa . . . 328
0.9.15.5 Lineare e interattivo . . . 328

0.9.16 Il compositore . . . 328
0.9.16.1 La musica è al 90% loop . . . 328
0.9.16.2 La musica è interattiva . . . 329
0.9.16.3 Molti stili, una sola persona . . . 329
0.9.16.4 Il compositore di *Rayman M* . . . 329
0.9.17 Il doppiaggio e il dialogue director . . . 329
0.9.17.1 Una domanda sull'intelligenza artificiale . . . 330
0.9.17.2 Il contratto e la frase "troppo moscia" . . . 330
0.9.18 L'audio programmer . . . 331
0.9.18.1 Gli *stem* che non c'erano . . . 331
0.9.19 Metà del lavoro è l'implementazione . . . 331
0.9.20 L'audio nel ciclo di produzione . . . 332
0.9.20.1 La pre-produzione: il team di sviluppo . . . 332
0.9.20.2 La pre-produzione: l'audio . . . 332
0.9.20.3 La produzione: contenuti e integrazione . . . 333
0.9.20.4 L'armatura in tre layer . . . 333
0.9.20.5 La post-produzione: dalla beta alla gold master . . . 334
0.9.21 Le quattro categorie di suoni . . . 335
0.9.21.1 La musica: source, lineare, non lineare . . . 335
0.9.21.2 Action e mood . . . 335
0.9.21.3 Il mod di *Far Cry*: la colonna sonora che si scrive giocando . . . 335
0.9.21.4 Interattività orizzontale e verticale . . . 336
0.9.21.5 Gli effetti sonori . . . 337
0.9.21.6 Gli ambienti . . . 337
0.9.21.7 I dialoghi . . . 337
0.9.22 Le domande finali . . . 338
0.9.22.1 Il labiale nelle altre lingue . . . 338
0.9.22.2 Come fa il gioco a sapere che hai sparato alla televisione? . . . 338
0.9.23 Contatti ed eventi di settore . . . 338
Punti chiave del capitolo . . . 339

II Fondamenti di Unreal Engine 341

1.1 Introduzione a Unreal Engine 343
1.1.1 Presentazioni: chi c'è, e chi ha già aperto un engine . . . 344
1.1.2 Software, game engine e scripting . . . 345
1.1.3 Il rendering 3D: dai poligoni ai pixel . . . 345
1.1.4 Il *real time rendering* e i *frame* . . . 346
1.1.5 Unreal Engine: perché è così usato . . . 347
1.1.6 Gli engine di una volta, e Unreal fuori dai videogiochi . . . 348
1.1.7 Il *game loop* . . . 348
1.1.8 La produzione: idee su carta, indie, AA, AAA . . . 349
1.1.9 Chi usa l'engine, e per che cosa . . . 350
1.1.10 Attività in classe: da che cosa è formato un gioco? . . . 351
1.1.11 Le domande prima della pausa . . . 354
1.1.12 Scaricare Unreal Engine 5.5 . . . 356
1.1.13 Dopo la pausa: dove cercare aiuto . . . 356
1.1.14 Un po' di teoria: il *Play in Editor* e il tempo di gioco . . . 357
1.1.15 Il primo progetto: il Project Browser e i template . . . 358
1.1.16 L'interfaccia dell'editor . . . 359
1.1.17 Il primo cubo e il *transform* . . . 360
1.1.18 Muoversi, duplicare, raggruppare . . . 362
1.1.19 Lo *snapping* e la velocità della camera . . . 362

1.1.20 L'Outliner e l'ordine della scena . . . 363
1.1.21 Esercitazione: il ristorante dei tre tavoli . . . 363
1.1.22 Salvare la mappa, e la chiusura . . . 365
Punti chiave del capitolo . . . 366

1.2 Programmare con i Blueprint **367**
1.2.1 La ripresa: il ristorante, e il programma di oggi . . . 368
1.2.2 Actor, transform e componenti . . . 368
1.2.3 Riaprire la mappa e filtrare il Content Browser . . . 369
1.2.4 Le classi Blueprint: Actor, Pawn, Character . . . 370
1.2.5 L'editor dei Blueprint e gli eventi . . . 372
1.2.6 Il primo nodo: Print String . . . 372
1.2.7 Le variabili: GET e SET, colori e calcoli . . . 374
1.2.8 Primo esercizio: Time To Print . . . 374
1.2.9 I dettagli di una variabile, e il *tuning* . . . 375
1.2.10 I tipi di variabile . . . 376
1.2.11 Le booleane e il Branch . . . 378
1.2.12 Gli operatori logici: AND, OR, NOT . . . 379
1.2.13 Secondo esercizio: due Branch, OR e AND . . . 380
1.2.14 Il Level Blueprint e la piattaforma . . . 381
1.2.15 La mobilità: Static, Stationary, Movable . . . 382
1.2.16 Ruotare di 5 gradi a ogni pressione: Split Struct Pin . . . 383
1.2.17 Il problema di Giulia, e la pausa . . . 383
1.2.18 Dopo la pausa: Add Actor World Rotation e i tasti W A S D . . . 384
1.2.19 La pallina, la fisica e il Teleport . . . 385
1.2.20 L'Event Tick e i *delta seconds* . . . 386
1.2.21 La piattaforma comandata dal mouse . . . 387
1.2.22 La vaschetta e il soffitto invisibile . . . 389
1.2.23 I compiti, e la chiusura . . . 390
Punti chiave del capitolo . . . 392

1.3 Texture, materiali e mesh **393**
1.3.1 L'attesa, le presentazioni e il programma . . . 394
1.3.2 Il materiale didattico . . . 395
1.3.3 Che cos'è una texture . . . 395
1.3.4 I tipi di mappa . . . 396
1.3.5 I canali RGB e le mappe impacchettate . . . 398
1.3.6 Il materiale: la vernice . . . 398
1.3.7 Material Domain, Blend Mode e Shading Model . . . 399
1.3.8 I nodi: le Material Expression . . . 401
1.3.9 Il problema della mappa vuota . . . 401
1.3.10 Dentro una texture e il Texture Sample . . . 402
1.3.11 Le costanti, le operazioni e le scorciatoie . . . 402
1.3.12 Il colore che brilla: Emissive, Multiply e Glow Intensity . . . 403
1.3.13 Il PBR: Metallic, Specular, Roughness e Normal . . . 404
1.3.14 Il Texture Coordinate e il tiling . . . 405
1.3.15 Le Material Instance . . . 406
1.3.16 Le Material Function . . . 408
1.3.17 Il Lerp: interpolare . . . 408
1.3.18 Il Panner: far muovere le texture . . . 409
1.3.19 La pausa: il progetto che non si trova, e Git . . . 410
1.3.20 L'import e la torretta . . . 410
1.3.21 Che cos'è una mesh . . . 411
1.3.22 Dimensioni, unità di misura e formati . . . 412

1.3.23 Il pivot del cannone e il Reimport . . . 413
1.3.24 Mettere ordine: le cartelle . . . 414
1.3.25 Lo Static Mesh Editor e le collisioni . . . 414
1.3.26 Raggruppare e fondere: Merge Actors . . . 415
1.3.27 I materiali della torretta . . . 416
1.3.28 Le domande: il Reimport e il merge . . . 417
1.3.29 Lo stretch, lo z-fighting e i materiali World Aligned . . . 418
1.3.30 L'esercizio della lava, il nodo Noise e la chiusura . . . 420
Punti chiave del capitolo . . . 423

1.4 Vettori, rotazioni e la torretta 425
1.4.1 L'attesa: Quixel Megascans e la lava . . . 426
1.4.2 Le domande e il trucco del CCD . . . 427
1.4.3 Il programma: il mulino e la torretta . . . 427
1.4.4 I vettori . . . 428
1.4.5 Il prodotto scalare e i vettori normalizzati . . . 430
1.4.6 I rotator, il gimbal lock e i quaternioni . . . 431
1.4.7 Il Rotating Movement Component . . . 432
1.4.8 Il Blueprint del mulino . . . 433
1.4.9 Far girare una pala: Event Tick e Combine Rotators . . . 434
1.4.10 La gerarchia: la seconda pala e il cubetto . . . 436
1.4.11 Il mulino che guarda il player: Find Look at Rotation . . . 437
1.4.12 Il Clamp, il debug con Print String e la pausa . . . 439
1.4.13 Dopo la pausa: la torretta in tre pezzi . . . 440
1.4.14 Il raggio d'azione: la Sphere Collision . . . 440
1.4.15 Gli eventi dei componenti e l'overlap . . . 441
1.4.16 Is Valid: esiste davvero? . . . 442
1.4.17 I tag e Actor Has Tag . . . 443
1.4.18 La variabile del bersaglio e i Custom Event . . . 443
1.4.19 Il timer: Set Timer by Event . . . 444
1.4.20 Move Turret: la torretta che segue il player . . . 445
1.4.21 Il tuning e i problemi degli studenti . . . 447
1.4.22 La chiusura e la prossima lezione . . . 448
Punti chiave del capitolo . . . 448

1.5 Spawn, trace e proiettili 451
1.5.1 L'attesa: i materiali spariti e il salvataggio . . . 452
1.5.2 Le sperimentazioni e il programma . . . 453
1.5.3 Che cos'è lo spawn . . . 453
1.5.4 Spawn Actor from Class e Destroy Actor . . . 455
1.5.5 I cicli e il For Loop . . . 455
1.5.6 Il fix della torretta: Find Relative Look at Rotation . . . 456
1.5.7 I trace: linee immaginarie . . . 457
1.5.8 L'evento Shoot e i vettori del trace . . . 458
1.5.9 I trace channel e le risposte di collisione . . . 460
1.5.10 Il proiettile: BP_Projectile e il Projectile Movement . . . 461
1.5.11 Il proiettile che si distrugge . . . 462
1.5.12 Visibility, Camera e il canale ProjectileTrace . . . 463
1.5.13 La pausa e i problemi degli studenti . . . 464
1.5.14 Lo spawn dei proiettili: le due bocche . . . 465
1.5.15 Il fire rate: Do Once, Sequence e Delay . . . 466
1.5.16 Gli errori della ripresa e il Multi Sphere Trace . . . 466
1.5.17 La torretta si ferma: On Component End Overlap . . . 468
1.5.18 L'esercizio della matrice di cubi: il For Loop . . . 469

1.5.19 La torretta che mira ai piedi, e la chiusura . . . 471
Punti chiave del capitolo . . . 471

1.6 Collisioni, timeline e audio **475**
1.6.1 L'attesa e il programma . . . 476
1.6.2 Il ripasso: il For Loop . . . 477
1.6.3 Le collisioni: Ignore, Overlap e Block . . . 477
1.6.4 Una domanda sui trace: quanti canali, e quando . . . 479
1.6.5 La dimostrazione: la palla e il cubo . . . 480
1.6.6 La matrice di cubi: dai trace ai cubi . . . 481
1.6.7 La seconda riga: Sequence e un secondo For Loop . . . 482
1.6.8 I problemi degli studenti con la matrice . . . 484
1.6.9 La piattaforma: componenti e controllo sul player . . . 484
1.6.10 Le timeline . . . 486
1.6.11 L'interpolazione lineare e l'alfa . . . 488
1.6.12 Il primo metodo: un Lerp sulla Z . . . 489
1.6.13 La pausa e il Nintendo Direct . . . 490
1.6.14 La velocità della timeline: Set Play Rate e Duration . . . 490
1.6.15 Il secondo metodo: il Billboard e le posizioni salvate . . . 491
1.6.16 La timeline sbagliata . . . 493
1.6.17 L'audio: la Sound Cue . . . 493
1.6.18 Far suonare la piattaforma: Spawn Sound, Do Once e Stop . . . 494
1.6.19 Il pitch che cambia con la direzione . . . 495
1.6.20 La chiusura . . . 496
Punti chiave del capitolo . . . 496

1.7 Pulsante, ereditarietà ed event dispatcher **501**
1.7.1 Prima della lezione: i problemi della piattaforma . . . 502
1.7.2 Il programma della giornata . . . 504
1.7.3 Gli array: gli scaffali delle variabili . . . 504
1.7.4 L'ereditarietà: parent e child . . . 505
1.7.5 La dimostrazione: il pulsante finito . . . 506
1.7.6 Il pulsante: due mesh e una box . . . 506
1.7.7 Le collisioni della Static Mesh: simple e complex . . . 507
1.7.8 La timeline del pulsante . . . 509
1.7.9 Le curve: linear e auto . . . 510
1.7.10 I problemi degli studenti con il pulsante . . . 512
1.7.11 Play o Play from Start? . . . 512
1.7.12 Gli attivabili: la classe BP_Activable . . . 513
1.7.13 Il bottone e la lista degli attivabili . . . 514
1.7.14 Il figlio: BP_ActivableLight e i due Begin Play . . . 516
1.7.15 La luce: la Point Light . . . 516
1.7.16 Salvare l'intensità e sovrascrivere gli eventi . . . 517
1.7.17 La pausa e il suono della piattaforma . . . 519
1.7.18 La correzione del pitch . . . 519
1.7.19 Un altro metodo: gli event dispatcher . . . 519
1.7.20 La luce si mette in ascolto: Get Actor Of Class e Bind Event . . . 520
1.7.21 Il bottone chiama: Call . . . 520
1.7.22 Override, Event e la scatolina . . . 521
1.7.23 Chiamare anche il parent . . . 523
1.7.24 L'esercizio . . . 523
1.7.25 La soluzione di uno studente . . . 524
1.7.26 Il compito per casa e la chiusura . . . 525
Punti chiave del capitolo . . . 525

1.8 Game Instance, livelli e animazioni **529**
1.8.1 Prima della lezione: l'Audio Component, Mixamo e le Game Jam . . . 530
1.8.2 Il programma della giornata e le variabili globali . . . 531
1.8.3 La dimostrazione: collezionabili e porte fra i livelli . . . 532
1.8.4 Il Gameplay Framework di Unreal . . . 533
1.8.5 Un fix per l'ascensore: la booleana Activated . . . 536
1.8.6 La Game Instance: GI_Platform . . . 537
1.8.7 Il collezionabile: BP_Collectible . . . 538
1.8.8 Il cast: entrare nella scocca . . . 539
1.8.9 Il tasto E: leggere il conteggio dal personaggio . . . 541
1.8.10 Il volume che azzera: BP_VolumeShop . . . 541
1.8.11 Il volume che carica un livello: BP_VolumeLoadLevel . . . 542
1.8.12 La prova fra i due livelli . . . 544
1.8.13 Mixamo: scegliere il personaggio . . . 544
1.8.14 La pausa: i problemi degli studenti . . . 544
1.8.15 L'import del personaggio . . . 545
1.8.16 Lo scheletro: ossa e giunture . . . 546
1.8.17 Le animazioni di Mixamo . . . 547
1.8.18 Importare le animazioni sullo scheletro . . . 549
1.8.19 L'editor dell'animazione . . . 550
1.8.20 L'Animation Blueprint . . . 550
1.8.21 Il nuovo Character: BP_Knight1 . . . 551
1.8.22 Initialize: il cast e le referenze . . . 552
1.8.23 Update: la velocità e la caduta . . . 552
1.8.24 L'AnimGraph, un'anteprima del Blend Space e il personaggio finito . . . 554
1.8.25 I materiali mancanti e la chiusura . . . 555
Punti chiave del capitolo . . . 558

1.9 State Machine, Blend Space e Montage **559**
1.9.1 Prima della lezione: i corsi degli studenti e la lezione del 25 . . . 560
1.9.2 Il ripasso: l'Animation Blueprint del cavaliere . . . 561
1.9.3 La State Machine della locomozione . . . 562
1.9.4 Il Blend Space 1D . . . 562
1.9.5 Una parentesi: "questo asset non ha schede ancorate" . . . 565
1.9.6 Il cavaliere in gioco: la Default Pawn Class . . . 565
1.9.7 La camera: Camera e Spring Arm . . . 565
1.9.8 I grafi di movimento, presi in prestito . . . 567
1.9.9 Tre spunte sulla rotazione . . . 567
1.9.10 Il retargeting: il salto del manichino sul cavaliere . . . 568
1.9.11 Le regole di transizione . . . 569
1.9.12 Lo sprint con l'Enhanced Input . . . 570
1.9.13 Un tentativo: FInterp To . . . 573
1.9.14 Prima della pausa: le animazioni da scaricare . . . 574
1.9.15 La pausa e i feedback . . . 574
1.9.16 La direzione: Calculate Direction . . . 575
1.9.17 Il Blend Space 2D . . . 577
1.9.18 Gli Animation Montage . . . 579
1.9.19 Dentro il Montage: Notify, sezioni e Slot . . . 580
1.9.20 Lo Slot nell'AnimGraph: Save Cached Pose . . . 581
1.9.21 Far partire l'attacco: Play Montage . . . 581
1.9.22 Il compito e la chiusura . . . 583
Punti chiave del capitolo . . . 583

1.10 Lean, State Machine e Physical Material **587**
1.10.1 Prima della lezione: il movimento da riparare . . . 588
1.10.2 Gli annunci e le domande . . . 590
1.10.3 Il bot di Federico . . . 591
1.10.4 Il risultato finale, in anteprima . . . 592
1.10.5 Il nuovo Animation Blueprint e il nuovo Character . . . 592
1.10.6 L'Initialize del bot e il Validated Get . . . 593
1.10.7 Ordine nel grafo: commenti e Reroute . . . 595
1.10.8 Il Blend Space 2D del bot . . . 595
1.10.9 Il calcolo del Lean . . . 597
1.10.10 La State Machine a sette stati . . . 599
1.10.11 Le regole del salto e la variabile dell'input . . . 600
1.10.12 Dopo la pausa: gli atterraggi . . . 601
1.10.13 I bot in gioco . . . 603
1.10.14 Il debug in diretta . . . 603
1.10.15 I passi: l'Anim Notify . . . 605
1.10.16 Physical Surface e lo Switch . . . 606
1.10.17 I suoni: Sound Wave e Sound Cue . . . 607
1.10.18 Notify nelle animazioni: Sync Marker e Notify Track . . . 608
1.10.19 Physical Material e le pedane . . . 609
1.10.20 Gli ultimi problemi e la chiusura . . . 610
Punti chiave del capitolo . . . 611

1.11 UI: vita, danno e stamina **615**
1.11.1 Prima della lezione: il Lean e il bot che non si muove . . . 616
1.11.1.1 Il Lean di Filippo . . . 617
1.11.1.2 Il bot di Lucrezia . . . 617
1.11.2 Che cos'è la UI, e il risultato di oggi . . . 619
1.11.3 La torretta e il personaggio nuovo . . . 619
1.11.4 Il danno: Apply Damage ed Event AnyDamage . . . 620
1.11.5 La vita: variabili, Clamp, Event Dispatcher e il livello che riparte . . . 621
1.11.6 Il Widget Blueprint . . . 623
1.11.7 La Progress Bar e le ancore . . . 624
1.11.8 L'HUD sullo schermo: Initialize, Create Widget e Add to Viewport . . . 625
1.11.9 Il collegamento: Bind Event e Set Percent . . . 626
1.11.10 La seconda barra e la corsa . . . 628
1.11.11 La pausa: le torrette e i bot degli studenti . . . 629
1.11.11.1 La torretta di Tommaso . . . 630
1.11.11.2 Il bot di Lucrezia, risolto . . . 630
1.11.12 Dopo la pausa: ancora la torretta, e gli annunci . . . 632
1.11.13 I due timer della stamina . . . 632
1.11.13.1 Il timer che consuma . . . 633
1.11.13.2 Il timer che ricarica . . . 633
1.11.13.3 L'evento StopRunning . . . 634
1.11.14 La stamina sulla barra . . . 636
1.11.15 Il compito e la chiusura . . . 637
Punti chiave del capitolo . . . 638

1.12 Menu e sistema di interazione **643**
1.12.1 Prima della lezione: recuperare, l'esame, il designer . . . 644
1.12.2 Gli annunci, l'ordine della giornata e la dimostrazione . . . 645
1.12.3 Il menu: la mappa, il widget e i bottoni . . . 646
1.12.4 Il Level Blueprint del menu: il widget, la Game Mode e la camera . . . 648
1.12.5 Il Graph del menu: cursore, input e bottoni . . . 649

1.12.6 Il diorama, e i problemi degli studenti . . . 652
1.12.7 Interfacce e Blueprint Component . . . 653
1.12.8 Il componente BPC_RayInteract: la Sphere Trace . . . 654
1.12.9 Il Trace Channel e la Blueprint Interface . . . 656
1.12.10 La pausa: la stamina di Tommaso . . . 657
1.12.11 La logica: la variabile Interactable . . . 657
1.12.12 L'oggetto di prova: BP_InteractableTest . . . 660
1.12.13 L'input: IA_Interact e il tasto E . . . 661
1.12.14 L'interazione nel personaggio . . . 661
1.12.15 Il widget "premi E" . . . 664
1.12.16 Un esempio: la luce che si accende . . . 665
1.12.17 La chiusura del corso . . . 666
Punti chiave del capitolo . . . 666

1.13 Gym, block out e luci **671**
1.13.1 L'apertura e il programma delle ultime quattro lezioni . . . 672
1.13.2 Da dove si parte: la realtà o un altro gioco? . . . 673
1.13.3 Il level design document e il layout . . . 674
1.13.4 La gym: il livello di prova . . . 676
1.13.5 Il block out e il caso di God of War . . . 677
1.13.6 Un engine: la gym, la griglia e la distanza di sprint . . . 679
1.13.7 I salti e il feeling del personaggio . . . 680
1.13.8 I parametri del Character Movement . . . 681
1.13.9 Le geometry brush: che cosa sono e come si usano . . . 683
1.13.10 Le altre forme e il Brush Editing . . . 685
1.13.11 La pausa e la scelta del formato . . . 686
1.13.12 Le luci: i tipi e la mobility . . . 687
1.13.13 La directional light e la sky light . . . 689
1.13.14 Point light e spot light . . . 690
1.13.15 La rect light . . . 692
1.13.16 Il build delle luci e i riflessi . . . 692
1.13.17 I compiti per martedì e la chiusura . . . 693
Punti chiave del capitolo . . . 694

1.14 Il Landscape: scultura e materiale **699**
1.14.1 L'apertura: le "meccaniche divertenti" e il programma . . . 700
1.14.2 Il riepilogo: layout, block out e gym . . . 701
1.14.3 La gym di Federico . . . 703
1.14.4 La forgiveness e i salti al limite . . . 705
1.14.5 La domanda sul pivot all'angolo . . . 706
1.14.6 Il livello di Filippo e il mindset di chi impara da solo . . . 706
1.14.7 Le modalità dell'editor e che cos'è il Landscape . . . 708
1.14.8 Creare un landscape: componenti, sezioni e risoluzione . . . 709
1.14.9 Il pennello: falloff, strength, Alpha e Pattern . . . 711
1.14.10 La domanda sulle caverne . . . 712
1.14.11 Gli strumenti di scultura: Sculpt, Erase, Smooth, Flatten . . . 714
1.14.12 Ramp e i LOD . . . 715
1.14.13 Erosion, Hydro, Noise, Mirror e Copy/Paste . . . 717
1.14.14 La pausa e la domanda su The Sims . . . 718
1.14.15 Paint e il materiale del landscape di StackOBot . . . 719
1.14.16 L'erba che spawna e le Virtual Texture . . . 721
1.14.17 Le Layer Info e il Landscape Grass Type . . . 722
1.14.18 Un materiale del landscape fatto a mano . . . 723
1.14.19 La Visibility Mask: i buchi nel terreno . . . 724

1.14.20 L'automaterial, i compiti e la chiusura . . . 725
Punti chiave del capitolo . . . 729

1.15 Automaterial, foliage e post process **733**
1.15.1 L'apertura: recuperi, Niagara e i corsi dell'accademia . . . 734
1.15.2 L'automaterial: la scena di prova . . . 735
1.15.3 La prima versione: la pendenza con il World Aligned Blend . . . 736
1.15.4 La Material Instance e i suoi parametri . . . 739
1.15.5 La seconda versione: la pendenza e l'altezza . . . 739
1.15.6 Unire e separare gli attributi . . . 742
1.15.7 La terza versione: i layer e l'erba . . . 742
1.15.8 Due correzioni sulle luci: Lumen e i light shaft . . . 745
1.15.9 La Camera Actor e i coni di luce dell'engine . . . 746
1.15.10 La pausa e il riepilogo per Edoardo . . . 747
1.15.11 Il Landscape Grass Type, rivisto . . . 747
1.15.12 Foliage Mode: che cos'è . . . 748
1.15.13 Gli strumenti del Foliage Mode . . . 749
1.15.14 I parametri di un foliage type . . . 750
1.15.15 Densità e raggio: chi vince . . . 751
1.15.16 Le altre impostazioni: pendenza, altezza, mobilità, collisioni . . . 751
1.15.17 Gli alberi e le collisioni . . . 753
1.15.18 Selezionare, rifinire e il foliage non naturale . . . 755
1.15.19 La domanda sui mondi procedurali . . . 756
1.15.20 Niagara e il Post Process Volume . . . 757
1.15.21 I compiti, la vetrata colorata e la chiusura . . . 758
Punti chiave del capitolo . . . 760

1.16 Post process, packaging, spline e decal **767**
1.16.1 L'apertura: una lezione improvvisata . . . 768
1.16.2 Il riepilogo della lezione precedente . . . 769
1.16.3 La post produzione nel ciclo di sviluppo . . . 770
1.16.4 Il Post Process Volume e la Camera Actor . . . 770
1.16.5 Le transizioni e l'Infinite Extent . . . 771
1.16.6 Il Bloom . . . 772
1.16.7 L'Exposure . . . 773
1.16.8 La Chromatic Aberration . . . 774
1.16.9 Il Lens Flare . . . 775
1.16.10 La Depth of Field . . . 775
1.16.11 Il Color Grading . . . 776
1.16.12 La domanda sull'effetto in due toni . . . 777
1.16.13 Il packaging: che cos'è e dove si avvia . . . 778
1.16.14 Le impostazioni del progetto . . . 778
1.16.15 Le configurazioni di build . . . 779
1.16.16 Le mappe e le cartelle da cucinare . . . 780
1.16.17 L'avvio del packaging e l'Output Log . . . 781
1.16.18 La pausa e Mallory . . . 782
1.16.19 Le spline: che cosa sono . . . 783
1.16.20 A che cosa servono le spline . . . 784
1.16.21 Il Blueprint per punto . . . 784
1.16.22 Il Blueprint per lunghezza . . . 786
1.16.23 Le domande: ottimizzazione e amico grafico . . . 788
1.16.24 Le decal: che cosa sono . . . 789
1.16.25 L'area di proiezione e Receives Decals . . . 790
1.16.26 Il materiale di una decal . . . 790

1.16.27 I nodi delle decal e il fade . . . 791
1.16.28 Il progetto estivo e i saluti . . . 792
Punti chiave del capitolo . . . 794

III Analog Game Design 799

2.1 Introduzione all'Analog Game Design 801
2.1.1 L'avvio della lezione . . . 802
2.1.2 I docenti . . . 802
2.1.2.1 Daniele: narrativa, fiction e gioco di ruolo . . . 803
2.1.2.2 Ruggero: game design e ricerca e sviluppo . . . 803
2.1.2.3 Beatrice: i giochi pervasivi . . . 803
2.1.2.4 I docenti ospiti . . . 804
2.1.3 Le presentazioni dei partecipanti . . . 804
2.1.3.1 Tiziana . . . 804
2.1.3.2 Nicola . . . 805
2.1.3.3 Rita . . . 805
2.1.3.4 Tancredi . . . 805
2.1.3.5 Lorenzo Vergata . . . 806
2.1.3.6 L'altro Lorenzo . . . 806
2.1.3.7 Angelo . . . 806
2.1.3.8 Gianmarco (dopo le slide) . . . 806
2.1.4 I materiali e gli strumenti del corso . . . 807
2.1.5 Lo scopo e il metodo del corso . . . 807
2.1.5.1 Diventare autori analogici . . . 808
2.1.5.2 Un corso intersezionale . . . 808
2.1.5.3 Un corso cross-generazionale . . . 808
2.1.5.4 Un corso pratico e adattabile . . . 809
2.1.5.5 Inquiry-based learning: domandate . . . 809
2.1.5.6 Feedback, non valutazione . . . 809
2.1.5.7 Le soft skill e i criteri di valutazione . . . 809
2.1.6 Le regole di comportamento e il carico di lavoro . . . 810
2.1.7 Il project work e il calendario . . . 810
2.1.7.1 Come funziona il project work . . . 810
2.1.7.2 Il calendario delle lezioni . . . 811
2.1.7.3 Il calendario del project work . . . 811
2.1.8 L'intelligenza artificiale nel corso . . . 812
2.1.8.1 La slide di Daniele . . . 812
2.1.8.2 La domanda: che cosa vuol dire usare l'IA in un gioco analogico? . . . 813
2.1.9 Le figure professionali dell'industria analogica . . . 814
2.1.9.1 Una decina di etichette . . . 815
2.1.9.2 Game designer . . . 815
2.1.9.3 Game developer . . . 815
2.1.9.4 Editor . . . 816
2.1.9.5 Publisher . . . 816
2.1.9.6 Sales e marketing . . . 816
2.1.9.7 Graphic designer e illustratore . . . 816
2.1.9.8 Sensitivity reader . . . 817
2.1.9.9 Producer . . . 817
2.1.9.10 Perché conoscere queste figure . . . 817
2.1.10 Marketing e sales . . . 817
2.1.10.1 Beatrice: i giochi trattati come prodotti . . . 817
2.1.10.2 Daniele: che cos'è (e che cosa non è) il marketing . . . 818

2.1.11 Game design e game development . . . 818
2.1.11.1 Il red flag dei designer . . . 818
2.1.11.2 Daniele: una distinzione che manca al videogioco . . . 819
2.1.12 Il mercato: Italia ed estero . . . 819
2.1.12.1 L'Italia . . . 819
2.1.12.2 L'estero . . . 820
2.1.13 Perché serve una terminologia comune . . . 820
2.1.14 Game e play . . . 821
2.1.14.1 Giochi e giocattoli . . . 821
2.1.15 Il framework MDA (e il DPE) . . . 822
2.1.15.1 Meccaniche, dinamiche, estetiche . . . 822
2.1.15.2 Il falso amico: "estetica" . . . 822
2.1.15.3 Il DPE: design, play, experience . . . 822
2.1.16 Le parole vietate . . . 823
2.1.16.1 La regola . . . 823
2.1.16.2 Perché . . . 823
2.1.17 Chi crea, chi gioca, chi facilita . . . 824
2.1.18 Playtest e iterazione . . . 824
2.1.18.1 Nessun gioco nasce perfetto . . . 824
2.1.18.2 L'aiuto dopo il corso . . . 825
2.1.18.3 Il dizionario comune e il production dictionary . . . 825
2.1.19 Tabletop Simulator . . . 826
2.1.19.1 Il software di settore . . . 826
2.1.19.2 Una dimostrazione . . . 826
2.1.19.3 Il primo task . . . 827
2.1.19.4 Tabletopia e altri vantaggi . . . 827
2.1.20 Le domande a Daniele: il tema del project work . . . 827
2.1.21 Le domande finali con Ruggero . . . 828
2.1.21.1 Le regole delle domande . . . 828
2.1.21.2 Come approcciarsi a Steam e Tabletop Simulator . . . 828
2.1.21.3 Il bilanciamento . . . 828
2.1.21.4 Si può giocare con i docenti? . . . 829
2.1.21.5 Dove si trovano i giochi su Tabletop Simulator . . . 829
2.1.21.6 I ruoli nei team e le dinamiche tra le persone . . . 829
2.1.21.7 Fare da developer di sé stessi? . . . 829
2.1.21.8 Il ruolo indispensabile e la pubblicazione . . . 830
2.1.21.9 Il gioco di ruolo in Italia . . . 831
2.1.21.10 Che cosa bisogna sapere già . . . 831
2.1.21.11 Game e play: i quattro quadranti . . . 831
2.1.21.12 House rule: il caso di Monopoly . . . 832
2.1.21.13 Il fine del game è il play? . . . 833
2.1.21.14 I giochi "comunicati male" . . . 833
2.1.21.15 I giochi di carte collezionabili e i saluti . . . 834
Punti chiave del capitolo . . . 834

2.2 I casi studio dell'Analog Game Design **837**
2.2.1 L'avvio della lezione . . . 838
2.2.2 Due libri per cominciare (e uno da rimandare) . . . 839
2.2.2.1 Un volume sull'edu-larp . . . 839
2.2.2.2 La progettazione dei giochi da tavolo di Walter Nuccio . . . 839
2.2.3 Perché analizzare i casi studio . . . 840
2.2.4 Il gioco analogico e le sue quattro macrocategorie . . . 840
2.2.4.1 Un termine ombrello . . . 840
2.2.4.2 Le quattro macrocategorie . . . 840

2.2.4.3 I giochi pervasivi: una prima definizione . . . 841
2.2.4.4 Ibridazione e sperimentazione . . . 842
2.2.5 Non chiudersi nel proprio genere . . . 842
2.2.6 I giochi pervasivi: la premessa di Beatrice . . . 843
2.2.7 Pervasivi con un'app: Pokémon Go, geocaching, GuruShot . . . 844
2.2.7.1 Pokémon Go . . . 844
2.2.7.2 Il geocaching . . . 845
2.2.7.3 GuruShot . . . 845
2.2.8 La città come campo di gioco: Journey, SF0 e Critical City . . . 845
2.2.8.1 Journey to the End of the Night . . . 845
2.2.8.2 SF0 . . . 846
2.2.8.3 Critical City Upload . . . 846
2.2.9 Teatro, ARG e marketing: da Sleep No More a The Institute . . . 846
2.2.9.1 Sleep No More . . . 846
2.2.9.2 Gli alternate reality game e Why So Serious . . . 846
2.2.9.3 Un gioco da tavolo che è anche un ARG . . . 847
2.2.9.4 Collettivo Cinetico: Cinetico 4.4 e Amleto . . . 847
2.2.9.5 Lettere anonime per un camminatore . . . 848
2.2.9.6 MP3 Experiment . . . 848
2.2.9.7 The Institute e Dispatches from Elsewhere . . . 848
2.2.9.8 L'ARG di Stranger Things a Milano . . . 849
2.2.10 Cacce al tesoro, corse e indagini . . . 849
2.2.10.1 Le guide-caccia al tesoro su carta . . . 849
2.2.10.2 PacManhattan . . . 849
2.2.10.3 Zombies, Run! . . . 849
2.2.10.4 Reptilarium . . . 850
2.2.11 Pervasivi per la formazione: Echi resistenti . . . 850
2.2.12 I giochi di ruolo dal vivo: le macrocategorie . . . 850
2.2.12.1 Il boffer LARP . . . 852
2.2.12.2 Il Mind's Eye Theatre . . . 852
2.2.12.3 Chamber LARP e parlor LARP . . . 852
2.2.12.4 L'American freeform . . . 852
2.2.12.5 I murder party . . . 853
2.2.13 Il Nordic LARP . . . 853
2.2.13.1 Perché è così importante . . . 853
2.2.13.2 Come si riconosce . . . 853
2.2.13.3 Knutpunkt e i Knutepunkt books . . . 854
2.2.13.4 Dogma 99 . . . 854
2.2.14 Deepform: il dramma quotidiano . . . 854
2.2.14.1 Il gioco della coppia perfetta . . . 854
2.2.14.2 Fat Man Down . . . 854
2.2.14.3 Le meccaniche di sicurezza . . . 855
2.2.15 Il 360° e l'alta risoluzione . . . 855
2.2.16 Tape LARP e black box . . . 856
2.2.16.1 Il tape LARP . . . 856
2.2.16.2 Il black box LARP . . . 856
2.2.17 Chi osserva? Il LARP e il teatro dell'oppresso . . . 857
2.2.18 I giochi di ruolo da tavolo: quattro parole per orientarsi . . . 858
2.2.18.1 Il tribalismo e le regole tramandate a voce . . . 858
2.2.18.2 Conflitto, azione, risoluzione, effetto . . . 858
2.2.19 Le origini: da Dungeons & Dragons agli anni Novanta . . . 859
2.2.19.1 Dungeons & Dragons e il gioco di guerra . . . 859
2.2.19.2 I generi e la struttura del gioco di ruolo "early" . . . 859
2.2.19.3 Gli anni Ottanta e Novanta: adattare un regolamento a tutto . . . 860

2.2.20 La New Wave e The Forge . . . 861
2.2.20.1 Il sistema conta . . . 861
2.2.20.2 I giochi della New Wave . . . 861
2.2.20.3 The Forge e la scena nordica . . . 862
2.2.20.4 Fiasco . . . 862
2.2.21 Il gioco di ruolo contemporaneo . . . 862
2.2.21.1 Prima del COVID: i giochi focalizzati . . . 862
2.2.21.2 Dopo il COVID: il ritorno all'origine . . . 863
2.2.21.3 Focalizzati, non semplificati . . . 863
2.2.21.4 Not the End . . . 863
2.2.21.5 Che cos'è l'OSR, e com'è nato davvero il gioco di ruolo . . . 864
2.2.22 Story game e giochi di ruolo sperimentali . . . 864
2.2.22.1 Gli story game . . . 864
2.2.22.2 The Quiet Year e i map drawing game . . . 865
2.2.22.3 Dialect . . . 865
2.2.22.4 Servono davvero le categorie? Il problema editoriale . . . 865
2.2.23 Giochi di ruolo per due, in solitario e micro . . . 865
2.2.24 I giochi da tavolo: i generi storici . . . 866
2.2.24.1 Roll and move . . . 866
2.2.24.2 Astratti . . . 866
2.2.24.3 Party game . . . 867
2.2.24.4 Giochi di parole . . . 867
2.2.24.5 Giochi di carte . . . 867
2.2.24.6 German game ed Eurogame . . . 868
2.2.24.7 American . . . 868
2.2.24.8 Strategici e wargame . . . 868
2.2.24.9 Movimento nascosto . . . 869
2.2.24.10 Destrezza . . . 869
2.2.24.11 18XX . . . 869
2.2.24.12 Tempo reale . . . 869
2.2.24.13 Legacy . . . 869
2.2.25 Due libri sul gioco da tavolo . . . 870
2.2.26 Libri-gioco, megagame e altri ibridi . . . 870
2.2.27 La chiusura: niente interrogazioni, e i materiali . . . 871
2.2.28 Le domande finali . . . 872
2.2.28.1 Che cosa fa scegliere una forma? . . . 872
2.2.28.2 Che cosa va per la maggiore in Italia? . . . 872
2.2.28.3 Avvisi . . . 873
2.2.29 Due cimiteri: i giochi di carte collezionabili e i "fix" . . . 873
2.2.29.1 Non fate Magic: The Gathering 2 . . . 873
2.2.29.2 Non fate un gioco per "aggiustarne" un altro . . . 874
Punti chiave del capitolo . . . 874

2.3 I framework del game design **877**
2.3.1 L'attesa: link, tutoraggi e gruppi . . . 878
2.3.2 Da giocatore e facilitatore a designer . . . 879
2.3.2.1 Il design è l'opposto della tradizione . . . 879
2.3.3 Il gioco è un'esperienza creata da regole . . . 880
2.3.3.1 Il gioco per tutti non esiste . . . 881
2.3.4 Concetti di base . . . 881
2.3.4.1 "Buon game design" vuol dire tutto e niente . . . 881
2.3.4.2 Le lenti di Schell . . . 881
2.3.4.3 I giochi sono un medium . . . 882
2.3.5 Design, playtest, iterazione e feedback . . . 882

2.3.5.1 Il playtest non si salta . . . 883
2.3.5.2 L'iterazione . . . 884
2.3.5.3 Il feedback: il sentimento, non la soluzione . . . 884
2.3.6 MDA e DPE, di nuovo . . . 885
2.3.6.1 L'MDA: su che cosa abbiamo controllo? . . . 885
2.3.6.2 Progettato bene, giocato volentieri . . . 886
2.3.6.3 Il DPE . . . 887
2.3.7 Giordano si presenta . . . 887
2.3.8 Il Player Involvement Model . . . 888
2.3.8.1 Il libro di Calleja . . . 888
2.3.8.2 Che cos'è un gioco immersivo? . . . 888
2.3.8.3 Sei forme di coinvolgimento . . . 889
2.3.8.4 Il coinvolgimento cinestetico . . . 890
2.3.8.5 Il coinvolgimento spaziale . . . 891
2.3.8.6 Il coinvolgimento condiviso . . . 891
2.3.8.7 Il coinvolgimento narrativo . . . 891
2.3.8.8 Il coinvolgimento affettivo . . . 892
2.3.8.9 Il coinvolgimento ludico . . . 893
2.3.9 Le domande sul coinvolgimento . . . 893
2.3.10 Il Mixing Desk of Larp . . . 895
2.3.10.1 Da dove viene . . . 895
2.3.10.2 Un doppio uso . . . 896
2.3.10.3 Il mixer . . . 896
2.3.11 I cursori del Mixing Desk . . . 897
2.3.11.1 Comunicazione: verbale o fisica . . . 897
2.3.11.2 Apertura tematica: trasparenza o segreto . . . 897
2.3.11.3 Diegesi ambientale: 360° o indipendenza materiale . . . 898
2.3.11.4 Creazione dei personaggi: chi gioca o chi progetta . . . 899
2.3.11.5 Gestione del gioco: attiva o passiva . . . 899
2.3.11.6 Universo narrativo: giocabile o plausibile . . . 900
2.3.11.7 Rappresentazione tematica: storia o azioni . . . 902
2.3.11.8 Meccaniche: intrusive o discrete . . . 903
2.3.12 La modularità e gli altri cursori . . . 905
2.3.13 Un'analisi dal vivo: Exit, Hidden Games e Profiler . . . 905
2.3.14 Perché proprio questi due framework . . . 908
2.3.15 Le domande finali . . . 909
2.3.15.1 La responsabilità di chi progetta . . . 910
2.3.15.2 La chiusura . . . 910
Punti chiave del capitolo . . . 911

2.4 Personaggi e ruoli **913**
2.4.1 L'attesa: componenti, tema e giochi astratti . . . 914
2.4.1.1 Componenti e aspettative: *Nemesis* e *The Gallerist* . . . 914
2.4.1.2 Tabletop Simulator e un canale per la classe . . . 915
2.4.1.3 Meccaniche perfette, tema assente . . . 916
2.4.2 Player, character e role . . . 917
2.4.2.1 Il ruolo non basta . . . 918
2.4.3 Che cosa rende giocabile un personaggio . . . 918
2.4.3.1 I quattro pilastri . . . 919
2.4.4 La chiarezza: sette schede per Joey Doe . . . 919
2.4.4.1 Scheda 1: solo il ruolo . . . 920
2.4.4.2 Scheda 2: un elenco di caratteristiche . . . 920
2.4.4.3 Scheda 3: una voce esterna . . . 921
2.4.4.4 Scheda 4: la sua voce . . . 921

2.4.4.5 Scheda 5: una fotografia . . . 921
2.4.4.6 Scheda 6: gli oggetti . . . 922
2.4.4.7 Scheda 7: le istruzioni di comportamento . . . 922
2.4.4.8 Chiarezza non vuol dire lunghezza . . . 923
2.4.5 La connettività: le tre cerchie . . . 923
2.4.5.1 Le relazioni esistono solo se le giochi . . . 925
2.4.6 Spinte e attività . . . 925
2.4.6.1 Pilastri e coinvolgimento . . . 926
2.4.7 Chi decide chi siamo: estetica, fiction e ruolo . . . 927
2.4.7.1 Lo slider dell'estetica . . . 927
2.4.7.2 Lo slider della fiction . . . 927
2.4.7.3 Lo slider del ruolo . . . 928
2.4.7.4 Pro e contro . . . 929
2.4.8 Spettatori, componenti, NPC e arbitri . . . 929
2.4.9 Lo spazio dei personaggi . . . 930
2.4.9.1 Personaggi non umani . . . 931
2.4.10 Il primo task: una scheda personaggio . . . 932
2.4.11 Le domande sul task . . . 935
2.4.12 Le domande finali . . . 937
2.4.12.1 Framework, dinamiche e playtest . . . 937
2.4.12.2 *Vast*, Leder Games e le ristampe . . . 938
2.4.12.3 La chiusura . . . 939
Punti chiave del capitolo . . . 939

2.5 Progettare emozioni **941**
2.5.1 L'attesa: che cosa resta di una game jam . . . 942
2.5.2 Le schede personaggio del primo task . . . 943
2.5.2.1 Tiziana: Lin Mezzacorona, la giornalista mezzelfa . . . 943
2.5.2.2 Rita: Ambra, la eco fashion designer . . . 944
2.5.2.3 Giordano: Frank, il ricettatore . . . 944
2.5.2.4 Nicola: Lucio Paoli, il giornalista senza scrupoli . . . 945
2.5.2.5 Angelo: Ezio Nava, il truffatore . . . 946
2.5.2.6 Giovanni Palmieri, l'alchimista . . . 946
2.5.2.7 Lorenzo: Carletto Magnamani, il critico che mangia tutto . . . 947
2.5.2.8 Tancredi: Ernesto Lesto, il ladro gentiluomo . . . 947
2.5.2.9 Gianmarco: Maria Bianca Giannini, la stilista . . . 948
2.5.2.10 L'età dei personaggi . . . 949
2.5.3 Che cosa prova chi gioca . . . 949
2.5.4 Il problema della parola "divertimento" . . . 950
2.5.4.1 Che cosa vuol dire "mi sto divertendo"? . . . 950
2.5.4.2 Il divertimento non è nel gioco, è nel giocare . . . 951
2.5.5 Giochi su temi difficili . . . 951
2.5.5.1 Si può fare, ma non da soli . . . 953
2.5.6 Il nanogioco del paracadute . . . 953
2.5.7 Il bleed . . . 955
2.5.8 La sicurezza nei giochi . . . 956
2.5.8.1 Le origini: dalla sicurezza fisica a quella emotiva . . . 956
2.5.9 Prima del gioco . . . 957
2.5.10 Durante il gioco: le meccaniche di sicurezza . . . 958
2.5.10.1 Le domande su linee e veli . . . 960
2.5.10.2 Le metatecniche diegetiche . . . 960
2.5.11 Dopo il gioco . . . 961
2.5.12 Il 200 Word RPG Challenge . . . 962
2.5.13 Il secondo task: un gioco di ruolo in 200 parole . . . 963

2.5.14 Le domande finali . . . 965
2.5.14.1 Workshop e debriefing in concreto . . . 965
2.5.14.2 Il debriefing nei giochi da tavolo . . . 966
2.5.14.3 Le persone prima dei giochi . . . 966
2.5.15 La dispensa: dalla ricerca al design . . . 967
2.5.16 Design thinking, Agile e design sprint . . . 967
2.5.16.1 Il Double Diamond: divergenza e convergenza . . . 969
2.5.17 Dati e insight . . . 970
2.5.18 Fattibilità, stakeholder e mercato . . . 971
2.5.18.1 L'analisi di fattibilità . . . 971
2.5.18.2 L'analisi degli stakeholder . . . 972
2.5.18.3 La value curve . . . 972
2.5.18.4 Le mappe di posizionamento . . . 973
2.5.18.5 Lavorare sulle proprie idee: la domanda latente . . . 974
2.5.18.6 *Games User Research* e il design sprint . . . 974
2.5.19 Il brainstorming strutturato . . . 975
2.5.19.1 Le Liberating Structures . . . 975
2.5.19.2 I canvas . . . 976
2.5.19.3 SCAMPER . . . 976
2.5.19.4 Il Crazy Eight . . . 977
2.5.19.5 Il parking lot . . . 978
2.5.19.6 Gli strumenti ibridi: le votazioni . . . 978
2.5.20 Gli strumenti di convergenza e il pitch deck . . . 978
2.5.20.1 Il one-liner . . . 979
2.5.20.2 I riflettori . . . 979
2.5.20.3 Lo high vision document . . . 980
2.5.20.4 Il pitch deck come validazione . . . 981
Punti chiave del capitolo . . . 982

2.6 Ricerca e sviluppo **985**
2.6.1 L'attesa: un'emozione, troppe idee, lo stato dell'arte . . . 986
2.6.2 Come si legge un regolamento . . . 988
2.6.3 I giochi di ruolo in 200 parole . . . 989
2.6.3.1 Tiziana: *Parlatene* . . . 989
2.6.3.2 Giordano: *Yamanobe no Michi* . . . 990
2.6.3.3 Gianmarco: *Beniamino va all'università* . . . 991
2.6.3.4 Nicola: *Finché resta cenere* . . . 991
2.6.3.5 L'altro Lorenzo: *Out There* . . . 992
2.6.3.6 Tancredi: *Qualcosa di più grande* . . . 993
2.6.3.7 Angelo: *Una brutta serata* . . . 994
2.6.3.8 Lorenzo Vergata: *What Remains of the Stories Behind Us* . . . 995
2.6.3.9 Rita: il tabellone dei futuri possibili . . . 996
2.6.4 Che cosa insegnano i nove giochi . . . 997
2.6.5 Perché si fa ricerca . . . 999
2.6.6 Per che cosa si fa ricerca . . . 1000
2.6.7 Divergenza e convergenza . . . 1000
2.6.8 Che cos'è la creatività: *Everything is a Remix* . . . 1001
2.6.9 Il plagio e le ispirazioni dichiarate . . . 1002
2.6.10 Dall'idea al gioco validato . . . 1003
2.6.11 A chi sto parlando: il target . . . 1005
2.6.12 Dall'etichetta alle persone: la mappa dell'empatia . . . 1006
2.6.13 Come si raccolgono le informazioni . . . 1007
2.6.14 Il target troppo largo: partire da chi si esclude . . . 1008
2.6.15 Le risorse e gli stakeholder . . . 1008

2.6.16 Gli strumenti e i siti per la ricerca . . . 1010
2.6.17 Le domande finali . . . 1011
2.6.17.1 Esiste un catalogo di meccaniche ed emozioni? . . . 1011
2.6.17.2 Le escape room . . . 1014
2.6.17.3 I saluti . . . 1014
Punti chiave del capitolo . . . 1015

2.7 Obiettivi di design e design sprint **1017**
2.7.1 L'attesa . . . 1018
2.7.2 Il project work: il calendario . . . 1019
2.7.3 Che cosa si porta al pitch del 12 ottobre . . . 1019
2.7.4 Il mandate . . . 1020
2.7.5 I gruppi . . . 1020
2.7.6 Le regole per lavorare in gruppo . . . 1021
2.7.7 Perché facciamo game design: due obiettivi . . . 1021
2.7.8 Le competenze che ogni obiettivo richiede . . . 1022
2.7.9 Il design sprint . . . 1025
2.7.9.1 Le fasi del file . . . 1025
2.7.9.2 La chiacchierata iniziale . . . 1026
2.7.10 La lavagna di Canva . . . 1027
2.7.11 Prima fase: che obiettivo vorrei dare a questo gioco? . . . 1027
2.7.12 Il racconto delle idee . . . 1028
2.7.12.1 Rita . . . 1028
2.7.12.2 Nicola arriva . . . 1029
2.7.12.3 Lorenzo Vergata . . . 1029
2.7.12.4 Tancredi . . . 1030
2.7.12.5 Tiziana . . . 1030
2.7.12.6 Angelo . . . 1031
2.7.12.7 Gianmarco . . . 1031
2.7.12.8 Nicola . . . 1032
2.7.12.9 L'altro Lorenzo . . . 1032
2.7.12.10 Giordano . . . 1032
2.7.13 La votazione a pallini . . . 1033
2.7.14 Dopo la pausa: il problema dei tempi . . . 1034
2.7.15 La ricerca e sviluppo sul tema . . . 1034
2.7.16 Le fasi successive del file . . . 1035
2.7.17 La mappa impatto-sforzo . . . 1035
2.7.18 Il percorso dell'utente e "state sul semplice" . . . 1037
2.7.19 La ricerca condivisa . . . 1038
2.7.19.1 La ricerca di Lorenzo Vergata . . . 1038
2.7.19.2 Ascoltare senza giudicare . . . 1039
2.7.20 La chiusura della lezione . . . 1040
2.7.21 Il primo tutoraggio con Ruggero . . . 1040
2.7.21.1 Ripassare i passi del design sprint . . . 1040
2.7.21.2 Le emozioni prima delle meccaniche . . . 1040
2.7.21.3 Il gioco del traditore . . . 1042
2.7.21.4 Il Go con i funghi . . . 1042
2.7.21.5 Tabletop Simulator: prepararsi alla lezione di domani . . . 1043
2.7.21.6 Le idee "a caso" e le emozioni . . . 1044
2.7.21.7 Che cos'è il pitch del 12 ottobre . . . 1045
2.7.21.8 "Io vi chiederò perché" . . . 1045
2.7.21.9 Il Mixing Desk: paletti, non validazione . . . 1046
2.7.21.10 Il gioco di gruppo è un compromesso . . . 1046
Punti chiave del capitolo . . . 1047

2.8 Tabletop Simulator **1049**
2.8.1 L'attesa . . . 1050
2.8.2 Il 12 ottobre: un pitch che non è un pitch . . . 1051
2.8.3 Il programma: le mani nel motore . . . 1051
2.8.4 Avviare il software e creare una stanza . . . 1052
2.8.5 "Un gioco da tavolo è un gioco su un tavolo" . . . 1052
2.8.6 Il tavolo e la table presence . . . 1053
2.8.7 Caricare un'immagine: file locale e cloud . . . 1054
2.8.8 Lo sfondo cambia la luce . . . 1055
2.8.9 I componenti: importare o creare? . . . 1055
2.8.9.1 Blocchi e cubetti . . . 1055
2.8.9.2 Le dimensioni reali . . . 1056
2.8.10 Le plance . . . 1056
2.8.11 Le carte standard . . . 1057
2.8.12 Dama, scacchi, dadi e figurine . . . 1057
2.8.13 Go, biglie, miscellanea . . . 1057
2.8.14 I sacchetti . . . 1059
2.8.15 I componenti custom . . . 1059
2.8.16 Il PDF del regolamento e la luce . . . 1060
2.8.17 Token e tile . . . 1061
2.8.17.1 Il Custom Token . . . 1061
2.8.17.2 Il Custom Tile . . . 1062
2.8.18 Il dado custom . . . 1062
2.8.19 Il mazzo custom . . . 1063
2.8.19.1 Le opzioni di importazione . . . 1063
2.8.19.2 Il Deck Editor . . . 1064
2.8.19.3 Le domande di Gianmarco . . . 1065
2.8.19.4 Carte con retri diversi . . . 1066
2.8.19.5 La carta singola e le carte bucate . . . 1066
2.8.20 Il tabellone senza cornice . . . 1066
2.8.21 Penna, testo e fogli di carta . . . 1066
2.8.22 Salvare e pubblicare il modulo . . . 1067
2.8.22.1 Upload e Update . . . 1067
2.8.22.2 La visibilità: unlisted . . . 1067
2.8.23 Prendere i componenti degli altri . . . 1068
2.8.23.1 Quando la sorgente sparisce . . . 1068
2.8.24 I moduli di Ruggero . . . 1069
2.8.24.1 Un gioco di carte . . . 1069
2.8.24.2 La demo di un gioco grande . . . 1069
2.8.24.3 Gli ancoraggi . . . 1070
2.8.24.4 Un gioco di ruolo con lo scripting . . . 1070
2.8.25 Perché proprio Tabletop Simulator . . . 1070
2.8.26 Prototipi in scala . . . 1071
2.8.27 Aggiornare un mazzo e tenere in ordine il cloud . . . 1072
2.8.28 Un software di settore suo malgrado . . . 1072
2.8.29 Niente script . . . 1073
2.8.30 Dopo la pausa: scorciatoie, fisica e destrezza . . . 1073
2.8.31 Un consiglio: giocate The Box . . . 1074
2.8.32 Saturi . . . 1074
2.8.33 Il playtest e il calendario del corso . . . 1074
2.8.34 Domande e risposte, alla rovescia . . . 1075
2.8.35 Le risorse da seguire . . . 1075
2.8.36 Giochi invendibili e l'economia del gioco . . . 1076
2.8.37 I giochi lunghi . . . 1077

Punti chiave del capitolo . . . 1077

2.9 Organizzare la progettazione e giochi formativi 1079
2.9.1 L'attesa: come state? . . . 1080
2.9.2 Il programma della serata . . . 1081
2.9.3 Organizzare la progettazione . . . 1081
2.9.3.1 Le risorse . . . 1082
2.9.3.2 Stimare, monitorare, aggiustare il tiro . . . 1083
2.9.4 Il progetto come una game jam . . . 1083
2.9.5 Il documento "Project management di un gioco" . . . 1084
2.9.6 Obiettivo e target: un recap . . . 1084
2.9.7 I vincoli già presenti . . . 1085
2.9.8 Quanto tempo abbiamo . . . 1086
2.9.9 Le fasi e le ore . . . 1086
2.9.9.1 L'enorme regola: non allocate tutte le ore . . . 1087
2.9.10 Emozioni, meccaniche e flusso: esplicitare . . . 1088
2.9.11 Il percorso dell'utente . . . 1088
2.9.12 Il primo prototipo: fisso, provvisorio, non ancora . . . 1089
2.9.13 Playtest e iterazione . . . 1090
2.9.14 Documentare sempre . . . 1091
2.9.15 Monitorare: che tipo di stimatori siamo . . . 1092
2.9.16 La stima: il 20% in più e la stima di pancia . . . 1092
2.9.17 Semplice è meglio . . . 1093
2.9.18 Il commento della classe . . . 1094
2.9.19 Esperienze ludiche 2: due strumenti invece di una carrellata . . . 1094
2.9.20 Il Game Design Document . . . 1094
2.9.21 L'Experience Canvas . . . 1095
2.9.22 Le tre grandi domande . . . 1096
2.9.23 Giochi formativi: gamification e game based learning . . . 1096
2.9.23.1 La gamification . . . 1097
2.9.23.2 Motivazione intrinseca ed estrinseca . . . 1097
2.9.23.3 Il game based learning . . . 1097
2.9.24 La volontarietà: quando un gioco formativo non è un gioco . . . 1097
2.9.24.1 La domanda di Tiziana: che cosa vuol dire scegliere . . . 1098
2.9.25 Sugata Mitra e l'educazione minimamente invasiva . . . 1100
2.9.26 Le escape room formative . . . 1100
2.9.27 Tre strutture di enigmi . . . 1101
2.9.27.1 La struttura lineare . . . 1101
2.9.27.2 La struttura ad albero . . . 1101
2.9.27.3 La struttura a raccolta . . . 1102
2.9.28 Dove si trovano gli enigmi . . . 1102
2.9.29 Le indicazioni del paper . . . 1104
2.9.30 Dopo la pausa: la parte burocratica . . . 1106
2.9.31 Ricerca e sviluppo o mazzo di carte? . . . 1108
2.9.32 L'Urban Game Deck . . . 1108
2.9.33 Come state? Il feedback sul corso . . . 1110
Punti chiave del capitolo . . . 1111

2.10 UX design e paragame 1113
2.10.1 L'attesa: il 12 "sta nella pietra" . . . 1114
2.10.2 Due domande su Tabletop Simulator . . . 1115
2.10.3 La UX nel gioco: uno strato tra la persona e le regole . . . 1115
2.10.3.1 Dove sta la UX . . . 1115
2.10.3.2 I quattro pilastri . . . 1116

2.10.4 Immediatezza . . . 1116
2.10.4.1 Un tiro di dadi . . . 1117
2.10.4.2 Quattro risorse . . . 1117
2.10.4.3 La corona d'alloro . . . 1117
2.10.4.4 Le icone che fanno il contrario . . . 1118
2.10.5 Efficienza . . . 1118
2.10.6 Memorabilità . . . 1119
2.10.7 Soddisfazione . . . 1120
2.10.8 Quante informazioni porta una carta? . . . 1120
2.10.8.1 L'icona capovolta . . . 1121
2.10.8.2 Una carta prima e dopo: il wireframe cambia . . . 1121
2.10.9 Accessibilità . . . 1122
2.10.10 Dopo la pausa: il libro che avete consigliato . . . 1122
2.10.11 Che cos'è il paragame . . . 1123
2.10.11.1 I bordi sono zone grigie . . . 1124
2.10.12 Le aree del paragame . . . 1124
2.10.13 Materialità . . . 1126
2.10.13.1 Il digitale come materiale . . . 1126
2.10.13.2 Le carte non sono tutte uguali . . . 1126
2.10.13.3 Clessidra o timer? . . . 1127
2.10.14 Formato . . . 1127
2.10.15 Regolamento . . . 1128
2.10.16 La scatola . . . 1129
2.10.17 Grafica e illustrazioni . . . 1129
2.10.17.1 Due edizioni, due destini . . . 1130
2.10.17.2 I font, e quante icone . . . 1130
2.10.18 Interfaccia analogica . . . 1130
2.10.18.1 La scatola in piedi: *Dobro* . . . 1131
2.10.18.2 Le opere sulla plancia di *The Gallerist* . . . 1131
2.10.18.3 Dove stanno le informazioni quando le carte sono in mano . . . 1131
2.10.19 Ambiente . . . 1131
2.10.20 Audio . . . 1132
2.10.21 Contesto . . . 1132
2.10.22 Lingua . . . 1133
2.10.23 La città, la stanza, il telefono . . . 1133
2.10.24 Casi studio . . . 1133
2.10.24.1 *Cuba Libre*: sei brochure . . . 1135
2.10.24.2 *293 Days*: solo un mazzo di carte . . . 1135
2.10.24.3 Un gioco a forma di metropolitana . . . 1136
2.10.24.4 *Shenzhen I/O*: un videogioco che chiede un manuale . . . 1136
2.10.24.5 Un flipper in scatola . . . 1136
2.10.24.6 *Lift It*: diventare la gru . . . 1136
2.10.24.7 *Coffee Rush*: le tazzine . . . 1137
2.10.24.8 *Ironwood*: legno contro metallo . . . 1137
2.10.24.9 *Nyctophobia*: giocare bendati . . . 1137
2.10.24.10 *Keep Talking and Nobody Explodes*: la bomba e il manuale . . . 1137
2.10.24.11 *City of Six Moons*: un gioco senza regole . . . 1138
2.10.24.12 *Scandal in the Deep*: il fiammifero . . . 1138
2.10.24.13 *Metal*: metalli diversi . . . 1138
2.10.24.14 *Meltdown*: tessere di ghiaccio . . . 1138
2.10.25 La discussione aperta . . . 1139
2.10.25.1 Il link a *World of Yoho* . . . 1139
2.10.25.2 I font di *Al Rashid* . . . 1139
2.10.25.3 Il paragame può agire sulle dinamiche? . . . 1139

2.10.25.4 Graphic design o illustrazione? . . . 1140
2.10.25.5 *Mountaineers*, gli inserti e il *setup* . . . 1140
2.10.25.6 Le domande "stupide" . . . 1141
2.10.25.7 Chi ci pensa, al paragame? *Rhino Hero* e *The Cat and the Tower* . . . 1142
2.10.26 Chiusura: ci si vede il 12 . . . 1142
Punti chiave del capitolo . . . 1142

2.11 Il pitch dei progetti di gruppo 1145
2.11.1 L'attesa: una "quotina di ansia" . . . 1146
2.11.2 Le regole del *pitch* . . . 1147
2.11.3 Primo *pitch*: *Murder Memory* . . . 1148
2.11.3.1 Il *pitch* . . . 1148
2.11.3.2 Le domande . . . 1149
2.11.4 Secondo *pitch*: *La Peste* . . . 1151
2.11.4.1 Il *pitch* . . . 1151
2.11.4.2 Le domande . . . 1152
2.11.5 Terzo *pitch*: un *roguelike* di *journaling* in solitario . . . 1152
2.11.5.1 Il *pitch* . . . 1153
2.11.5.2 Le domande . . . 1153
2.11.6 Prima dei feedback: indicazioni, non consigli . . . 1154
2.11.7 Che cos'è un regolamento (anche in una *escape room*) . . . 1156
2.11.8 Feedback a *Murder Memory*: convergere . . . 1158
2.11.9 Feedback a *La Peste*: obiettivo e *agency* . . . 1160
2.11.10 Feedback al *roguelike*: focalizzare . . . 1163
2.11.11 I casi studio della serata . . . 1163
2.11.12 Domande aperte: scrivere il gioco per chi lo gioca senza di voi . . . 1163
2.11.13 Il *pitch* visto da dentro: la vergogna delle idee . . . 1165
2.11.14 *Roguelike*, RNG e narrativa modulare . . . 1167
2.11.14.1 *Dead Cells* o *Slay the Spire*? . . . 1167
2.11.14.2 Da dove si comincia? Non dai *perk* . . . 1167
2.11.14.3 Le meccaniche sono metafore . . . 1168
2.11.14.4 Giusto o sbagliato? Due livelli . . . 1169
2.11.15 Chiusura: la nona ora . . . 1170
Punti chiave del capitolo . . . 1171

2.12 Prototipi, regolamenti e raccolta dei feedback 1173
2.12.1 L'attesa: flusso di gioco, tutoraggi, scadenze . . . 1174
2.12.2 Quanto può cambiare un gioco dopo il *pitch*? . . . 1175
2.12.3 Che cos'è un prototipo, e perché lo facciamo . . . 1176
2.12.4 Il triangolo dei prototipi: ruolo, *look and feel*, implementazione . . . 1177
2.12.4.1 Il test di ruolo . . . 1177
2.12.4.2 Il test di *look and feel* . . . 1178
2.12.4.3 Il test di implementazione . . . 1178
2.12.4.4 Un test alla volta, e nell'ordine giusto . . . 1178
2.12.5 Il regolamento: come si insegna un gioco? . . . 1179
2.12.5.1 Pre-gioco, post-gioco, fiction, intenti . . . 1180
2.12.5.2 La scrittura è lineare, il gioco no . . . 1181
2.12.5.3 Indici e blocchi: tre esempi . . . 1182
2.12.6 Che cosa c'è in un regolamento . . . 1183
2.12.6.1 L'esperimento su *Diamant* . . . 1184
2.12.7 Raccogliere i feedback: che cosa succede durante la partita . . . 1185
2.12.7.1 L'osservazione in presenza . . . 1185
2.12.7.2 La registrazione audiovisiva . . . 1186
2.12.7.3 Nessuna osservazione: il test a porte chiuse . . . 1186

2.12.8 Raccogliere i feedback: che cosa succede dopo la partita . . . 1187
2.12.8.1 Il *debriefing* orale di gruppo . . . 1187
2.12.8.2 Il *debriefing* orale uno a uno . . . 1188
2.12.8.3 Il form chiuso . . . 1188
2.12.8.4 Il report libero . . . 1189
2.12.8.5 Combinare i metodi con il triangolo . . . 1189
2.12.9 Il sentimento dietro la soluzione . . . 1191
2.12.10 Chi testa, dove, quanto, con chi . . . 1191
2.12.11 Domande sulle slide . . . 1192
2.12.12 Dopo la pausa: il *playtest* nel corso . . . 1193
2.12.13 Il gruppo di Rita: fermare la ricerca, scrivere il regolamento . . . 1193
2.12.14 *Murder Memory*: la storia non deve bloccare il gioco . . . 1195
2.12.14.1 Il flusso di gioco non è il regolamento . . . 1196
2.12.14.2 Il vostro gioco è meglio come storia o come gioco?" . . . 1197
2.12.14.3 Gli enigmi: nodi, catene, parallelo . . . 1197
2.12.15 *Automa*: le carte sono il gioco . . . 1199
2.12.15.1 Dove si trovano i *tester* . . . 1200
2.12.15.2 Progettare le carte . . . 1201
2.12.16 Chiusura: la materialità del gioco, i tutoraggi, i contatti . . . 1202
Punti chiave del capitolo . . . 1203

IV Alcune Professioni nel Mondo Videoludico 1205

V Game Design 1207

1 Il mestiere del game designer 1209
1.1 Il contesto: un corso come simulazione di lavoro . . . 1209
1.1.1 Il principio fondamentale . . . 1209
1.1.2 Perché l'impegno conta più che mai . . . 1209
1.1.3 Sbagliare qui per non sbagliare fuori . . . 1210
1.1.4 Il valore del fallimento . . . 1210
1.2 Organizzazione del corso e gerarchia . . . 1210
1.2.1 La struttura gerarchica . . . 1210
1.2.2 I canali di comunicazione . . . 1211
1.2.3 I tre moduli e la settimana tipo . . . 1211
1.2.4 Le due anime delle lezioni di game design . . . 1211
1.2.5 Altre componenti del corso . . . 1212
1.3 Il metodo: l'inquiry-based learning . . . 1212
1.3.1 Che cos'è . . . 1212
1.3.2 Non sempre esiste una risposta giusta . . . 1213
1.3.3 Sensibilità, non memoria . . . 1213
1.3.4 Le lezioni come corsi di aggiornamento . . . 1213
1.3.5 La gerarchia della competenza . . . 1213
1.3.6 I livelli dell'apprendimento . . . 1214
1.4 Task, feedback e regole del corso . . . 1214
1.4.1 I task come opportunità . . . 1214
1.4.2 Quanto lavorare: un calcolo indicativo . . . 1214
1.4.3 Niente voti, solo feedback . . . 1215
1.4.4 La condivisione dei task . . . 1215
1.4.5 Non copiare . . . 1215
1.4.6 Il "non leggo" . . . 1215
1.4.7 Requisiti per superare l'anno . . . 1216

1.4.8 Scadenze . . . 1216
1.4.9 Accessibilità e DSA . . . 1216
1.5 Le competenze trasversali . . . 1216
1.5.1 I giochi si fanno in gruppo . . . 1216
1.5.2 Essere persone con cui si vuole lavorare . . . 1216
1.5.3 Indipendenza e proattività . . . 1217
1.5.4 Fare domande . . . 1217
1.6 Strumenti: engine, Unreal e materiali . . . 1217
1.6.1 Unreal Engine e gli altri motori . . . 1217
1.6.2 Programmazione: cosa ci si aspetta da un designer . . . 1218
1.6.3 Versione e preparazione . . . 1218
1.7 Letture e risorse di riferimento . . . 1218
1.7.1 I libri . . . 1218
1.7.2 Canali e conferenze . . . 1219
1.7.3 Associazioni e premi: come leggerli . . . 1219
1.8 Che cosa fa un game designer . . . 1219
1.8.1 Progettare meccaniche e struttura del gioco . . . 1219
1.8.2 Le 3C: camera, character, control . . . 1220
1.8.3 Il personaggio . . . 1220
1.8.4 Documentare le informazioni per gli altri dipartimenti . . . 1220
1.8.5 Mediatore e leader “al servizio” . . . 1220
1.8.6 Gestire l’avanzamento del prodotto . . . 1220
1.8.7 Presentare le idee . . . 1221
1.8.8 Bozze e prototipi . . . 1221
1.8.9 Fare ricerca per l’idea, e poi smontarla . . . 1221
1.8.10 Analizzare i competitor . . . 1221
1.8.11 Lavorare in engine e fare tuning . . . 1222
1.8.12 Bilanciare . . . 1222
1.8.13 Analizzare il mercato . . . 1222
1.8.14 Sintesi . . . 1222
1.9 I limiti del design . . . 1222
1.9.1 Non siete “gli uomini delle idee” . . . 1223
1.10 Ruoli e specializzazioni . . . 1223
1.10.1 La scala gerarchica . . . 1223
1.10.2 UX e UI . . . 1224
1.10.3 Le specializzazioni del game design . . . 1224
1.10.4 Le specializzazioni del level design . . . 1224
1.10.5 Discipline vicine: suono, UI, produzione . . . 1224
1.11 Approfondimenti emersi in lezione . . . 1225
1.11.1 Roguelike e roguelite . . . 1225
1.11.2 Applicazioni fuori dall’industria videoludica . . . 1225
1.12 Il primo task . . . 1225
Punti chiave del capitolo . . . 1225

2 Che cos’è un gioco 1227
2.1 Una premessa: il direttore del corso . . . 1227
2.1.1 Chi è . . . 1227
2.1.2 Due consigli dal mestiere . . . 1228
2.2 L’esercizio: scrivere la propria definizione di gioco . . . 1229
2.3 Le definizioni della classe . . . 1229
2.3.1 Un prodotto con giocatori, regole, spazio, tempo e scopo . . . 1229
2.3.2 Un’attività tra persone con regole concordate . . . 1229
2.3.3 Un’attività o un oggetto che stimola emozioni e pensiero . . . 1230
2.3.4 Un prodotto che regala stimoli ed emozioni . . . 1230

2.3.5 Un'esperienza di immedesimazione e di fuga dalla realtà . . . 1230
2.3.6 Emozioni positive e negative . . . 1231
2.3.7 Un'esperienza individuale o di gruppo, con un obiettivo . . . 1231
2.3.8 Un'attività intrattenente basata sull'interattività . . . 1231
2.3.9 Vivere una storia in prima persona . . . 1232
2.3.10 Un'avventura da vivere come un alter ego . . . 1232
2.3.11 L'interazione con un medium, con l'ambiente o con la propria immaginazione . . . 1232
2.3.12 Un insieme di regole e sistemi . . . 1233
2.3.13 Un insieme di regole da rispettare . . . 1233
2.3.14 Un'attività interessante e non obbligatoria . . . 1233
2.3.15 Un regolamento creato da un designer . . . 1234
2.3.16 Interazioni che influenzano il gioco, sfida e progressione narrativa . . . 1234
2.3.17 Un'esperienza attraverso uno schermo, fatta di personaggi . . . 1234
2.3.18 Un'opera d'arte . . . 1234
2.3.19 Divertire, insegnare, far riflettere, meravigliare . . . 1235
2.3.20 Un'attività per la persona giocante . . . 1235
2.3.21 Un'attività per una o più persone . . . 1235
2.3.22 Una promessa e un punto di vista diverso dal proprio . . . 1235
2.3.23 Creare una specifica esperienza . . . 1236
2.3.24 Scenari, meccaniche e personaggi ideati dal designer . . . 1236
2.3.25 Un'esperienza sempre diversa . . . 1236
2.3.26 Un viaggio o una sfida . . . 1236
2.4 Le definizioni "autorevoli" . . . 1237
2.5 Il gioco non è definibile . . . 1237
2.5.1 Il gioco come stato mentale . . . 1237
2.5.2 Le definizioni dei docenti . . . 1237
2.5.3 La definizione come bussola . . . 1238
2.6 Gli ingredienti della zuppa . . . 1239
2.6.1 La metafora della zuppa . . . 1239
2.6.2 Raph Koster: artefatti ludici e sistemi ludici . . . 1239
2.6.3 Game e play sono la stessa cosa . . . 1240
2.7 Il framework di Jesse Schell: la tetrade elementare . . . 1240
2.7.1 Che cosa sono i framework . . . 1240
2.7.2 Meccaniche . . . 1240
2.7.3 Narrativa . . . 1241
2.7.4 Un principio fondamentale sui framework . . . 1241
2.7.5 Estetiche . . . 1241
2.7.6 Tecnologia . . . 1242
2.8 Il framework di Brenda Romero e Ian Schreiber . . . 1243
2.8.1 Perché più punti di vista . . . 1243
2.8.2 Gli elementi . . . 1243
2.8.3 Confronto con Schell . . . 1243
2.9 Il framework di Stéphane Bura . . . 1244
2.9.1 Una tabella a incroci . . . 1244
2.9.2 La colonna Freedom . . . 1244
2.9.3 La colonna Mastery . . . 1245
2.9.4 La colonna Data . . . 1246
2.10 Il framework del direttore . . . 1246
2.11 Le lenti: non esiste un metodo unico . . . 1247
2.11.1 Il principio più importante . . . 1247
2.11.2 La migliore difesa contro l'intelligenza artificiale . . . 1247
2.11.3 The right tool for the right job . . . 1247
2.11.4 Il Deck of Lenses . . . 1247
2.12 Il vocabolario del designer . . . 1248

2.12.1 La metafora del medico . . . 1248
2.12.2 La parola proibita . . . 1248
2.12.3 Non solo “divertente”: anche “bello”, “valido”, “noioso” . . . 1249
2.12.4 Il nuovo vocabolario: una prima lista di emozioni . . . 1249
2.13 La forma mentis del designer secondo Chris Bateman . . . 1251
2.14 Il task della settimana . . . 1251
Punti chiave del capitolo . . . 1252

3 Iterare e documentare 1253
3.1 Domande e risposte sulla lezione precedente . . . 1253
3.1.1 Schell: game piece, verbi e regole . . . 1253
3.1.2 Romero e Schreiber: i player sono il target audience . . . 1254
3.1.3 Perché due framework di core elements . . . 1254
3.1.4 Come si usa la matrice di Bura . . . 1255
3.1.5 Una critica personale al framework di Bura . . . 1256
3.1.6 Scrivere da designer, non da recensori . . . 1257
3.2 Il Drive del corso e le regole dei task . . . 1257
3.2.1 La struttura del Drive . . . 1257
3.2.2 Come si consegna un task . . . 1258
3.3 Il ciclo di produzione . . . 1259
3.3.1 La lezione zero . . . 1259
3.3.2 Preproduzione, produzione, post-produzione . . . 1259
3.3.3 DevOps e LiveOps . . . 1259
3.3.4 Qual è la parte più importante? . . . 1260
3.3.5 Tutto quello che facciamo è brutto . . . 1260
3.3.6 Tanti cicli, una sola forma . . . 1261
3.3.7 Il termine chiave: iterativo . . . 1262
3.4 Iterazione e realtà della produzione: le domande . . . 1262
3.4.1 Se una meccanica strutturale non funziona . . . 1262
3.4.2 Se l’ordine arriva dall’alto . . . 1263
3.4.3 Publisher, licenze e stakeholder . . . 1263
3.5 I principi dell’iterazione . . . 1264
3.5.1 Target audience . . . 1265
3.5.2 Involvement . . . 1265
3.5.3 Iteration . . . 1266
3.5.4 Holistic . . . 1266
3.5.5 Adaptive . . . 1267
3.5.6 I segreti per iterare . . . 1267
3.6 Perché la documentazione è importante . . . 1268
3.6.1 Una pausa e un promemoria . . . 1268
3.6.2 La prima vera lezione di documentazione . . . 1269
3.6.3 A che cosa serve la documentazione . . . 1269
3.6.4 I documenti evolvono . . . 1270
3.6.5 La vostra firma e la vostra arte . . . 1270
3.7 Le golden rules della documentazione . . . 1270
3.7.1 Mostra, non scrivere . . . 1270
3.7.2 Essenziale . . . 1271
3.7.3 The right tool for the right job . . . 1271
3.7.4 Coerenza e sequenza . . . 1272
3.7.5 Mind the language . . . 1272
3.7.6 Niente decorazioni inutili . . . 1272
3.7.7 Grafici con intento . . . 1272
3.7.8 Colori per separare, aggregare, far emergere . . . 1272
3.7.9 Conosci il lettore . . . 1273

3.7.10 Sempre aggiornati . . . 1273
3.7.11 Iterate . . . 1273
3.8 Scegliere il documento giusto . . . 1273
3.8.1 Less is more . . . 1273
3.8.2 Strumenti aggiuntivi . . . 1275
3.9 Le regole formali del corso . . . 1275
3.10 Il task della settimana . . . 1276
Punti chiave del capitolo . . . 1277

4 Problema e soluzione 1279
4.1 L'atomo di gioco di Raph Koster . . . 1279
4.1.1 Il punto rimasto in sospeso . . . 1279
4.1.2 Gli elementi dell'atomo . . . 1280
4.1.3 Il modello mentale del giocatore . . . 1281
4.1.4 A che cosa serve nel design . . . 1282
4.2 Un ripasso della documentazione . . . 1282
4.3 Il production dictionary . . . 1283
4.3.1 Un documento comune, con molti nomi . . . 1283
4.3.2 A che cosa serve . . . 1283
4.3.3 Che cosa contiene . . . 1284
4.3.4 Esercizio in aula: una carta di Hearthstone . . . 1284
4.4 Due principi sulla documentazione . . . 1285
4.4.1 La documentazione non sostituisce il gioco . . . 1285
4.4.2 Non esistono i template . . . 1285
4.5 Problema e soluzione: il design come compromesso . . . 1286
4.5.1 La preistoria della produzione . . . 1286
4.5.2 Il miglior compromesso possibile . . . 1286
4.5.3 Cinquanta per cento scienza, cinquanta per cento arte . . . 1287
4.6 Senza dati si è solo un'altra persona con un'opinione . . . 1287
4.6.1 Quanti dati, e quando . . . 1288
4.6.2 Il design thinking in cinque passi . . . 1288
4.6.3 Partire da un problema o da una necessità . . . 1289
4.6.4 Esercizio in aula: che cosa non va nei soulslike? . . . 1289
4.6.5 Correlare, chiudere la visione, prototipare . . . 1290
4.7 La mappa della ricerca in pre-produzione . . . 1290
4.8 Competitor diretti e indiretti . . . 1291
4.8.1 Il competitor diretto . . . 1291
4.8.2 Il competitor indiretto . . . 1291
4.8.3 Gli esempi della classe . . . 1292
4.8.4 Come individuarli: i punti insindacabili . . . 1292
4.8.5 L'opposizione . . . 1293
4.9 Analisi degli utenti, del mercato e del gameplay . . . 1293
4.9.1 Analisi degli utenti . . . 1293
4.9.2 Analisi di mercato . . . 1294
4.9.3 Analisi di gameplay e visione . . . 1295
4.9.4 Dopo la pausa: game qualities e generi . . . 1295
4.10 Client pitch direction analysis . . . 1295
4.10.1 Fate domande . . . 1295
4.10.2 Le cinque W . . . 1295
4.10.3 I nove perché e la root cause analysis . . . 1296
4.10.4 Osservare, intervistare, filmare . . . 1296
4.10.5 Priorità e visione del cliente . . . 1296
4.10.6 Must have e nice to have . . . 1296
4.10.7 Esempi concreti e situazioni estreme . . . 1297

4.11 Gli stakeholder . . . 1297
4.11.1 Chi sono . . . 1297
4.11.2 Classificarli . . . 1298
4.12 Problema contro necessità . . . 1299
4.12.1 Due definizioni . . . 1299
4.12.2 Strumenti per metterlo a terra . . . 1300
4.13 Analisi dei competitor . . . 1301
4.13.1 La SWOT e le domande . . . 1301
4.13.2 Il design dominante . . . 1301
4.13.3 Gli unique selling point . . . 1302
4.13.4 Che cosa resta da vedere . . . 1304
4.14 Il task della settimana . . . 1304
Punti chiave del capitolo . . . 1304

5 Dalla ricerca alle idee **1307**
5.1 In apertura: il production dictionary, ripreso . . . 1307
5.1.1 Prima di cominciare . . . 1307
5.1.2 Nessun limite di pagine . . . 1308
5.1.3 File a sé o paragrafo d'apertura . . . 1308
5.1.4 Ibridare i documenti . . . 1309
5.1.5 Che cosa ci va, e chi lo decide . . . 1309
5.2 Documenti di visione e documenti di produzione . . . 1309
5.2.1 Una categoria personale . . . 1309
5.2.2 Dalla Bibbia alla wiki, e oltre . . . 1309
5.2.3 Chi tiene in ordine il "giardinetto" . . . 1310
5.3 Dove eravamo rimasti . . . 1310
5.3.1 Il mantra del compromesso . . . 1310
5.3.2 Una parentesi su Deming . . . 1310
5.3.3 La mappa della ricerca, a che punto siamo . . . 1311
5.4 Analisi dei competitor: framework ed elicitazione triadica . . . 1311
5.4.1 I framework servono anche all'analisi . . . 1311
5.4.2 L'elicitazione triadica . . . 1312
5.4.3 Parlare con l'utenza . . . 1312
5.5 La perceptual mapping . . . 1312
5.5.1 Che cos'è . . . 1312
5.5.2 La perceptual mapping nel design: i nemici . . . 1313
5.5.3 Esercizio in aula: la mappa dei metroidvania . . . 1314
5.6 Altre fonti di dati . . . 1316
5.7 L'analisi degli utenti . . . 1316
5.7.1 Uno dei punti più difficili . . . 1316
5.7.2 Che cosa interessa sapere . . . 1316
5.7.3 Il caso Riot e League of Legends . . . 1317
5.7.4 Chi gioca, chi giocherà . . . 1318
5.7.5 Il feedback online e l'ombra digitale . . . 1318
5.7.6 Interviste, focus group e survey . . . 1318
5.7.7 Archetipi e osservazione . . . 1320
5.7.8 La mappa dell'empatia . . . 1320
5.8 L'analisi di mercato . . . 1321
5.8.1 C'è spazio per il mio gioco? . . . 1321
5.8.2 Seguire i trend . . . 1321
5.8.3 Lo stato emotivo del mercato . . . 1322
5.8.4 Barriere, tasse e logistica . . . 1322
5.9 L'analisi di gameplay: le motivazioni di Quantic Foundry . . . 1322
5.9.1 Un modo, non l'unico . . . 1322

5.9.2 Quantic Foundry . . . 1323
5.9.3 Sei gruppi, dodici motivazioni . . . 1323
5.9.4 Community . . . 1323
5.9.5 Competition . . . 1324
5.9.6 Excitement . . . 1325
5.9.7 Destruction . . . 1326
5.9.8 Completion . . . 1326
5.9.9 Power . . . 1326
5.9.10 Strategy . . . 1327
5.9.11 Challenge . . . 1327
5.9.12 Fantasy . . . 1328
5.9.13 Story . . . 1328
5.9.14 Discovery . . . 1328
5.9.15 Design . . . 1329
5.9.16 A che cosa serve . . . 1329
5.10 Correlare: pattern, coincidenze e insight . . . 1329
5.10.1 Chiedersi perché . . . 1329
5.10.2 Il cervello che vede pattern . . . 1329
5.10.3 Dall'insight all'empatia . . . 1330
5.11 La fine della pre-produzione . . . 1330
5.12 Idee e brainstorming: i principi . . . 1331
5.12.1 Le idee sono inutili . . . 1331
5.12.2 Generare tante idee, scartare le prime . . . 1331
5.12.3 Le regole del brainstorming . . . 1332
5.13 Il design elegante . . . 1332
5.13.1 Che cosa vuol dire . . . 1332
5.13.2 Elegante non vuol dire migliore . . . 1333
5.14 Essere creativi . . . 1333
5.15 Le tecniche per generare idee . . . 1334
5.15.1 Una rassegna, non un obbligo . . . 1334
5.15.2 Le categorie sensoriali . . . 1334
5.15.3 Opposite direction . . . 1334
5.15.4 Break the theme . . . 1335
5.15.5 Le regole euristiche . . . 1335
5.15.6 What if . . . 1335
5.15.7 Wicked questions . . . 1336
5.15.8 SIT e TRIZ . . . 1336
5.15.9 Limitation . . . 1336
5.15.10 Drawing e storyboard . . . 1336
5.15.11 Sharing . . . 1336
5.15.12 Ad absurdum . . . 1337
5.15.13 Che cosa resta da vedere . . . 1337
5.16 Il task della settimana . . . 1337
Punti chiave del capitolo . . . 1338

6 Dal brainstorming ai flowchart 1339
6.1 In apertura: ancora il production dictionary . . . 1339
6.1.1 Prima di cominciare . . . 1339
6.1.2 Che cosa si intende per nomenclatura . . . 1340
6.1.3 Priorità: un limite di una pagina . . . 1340
6.1.4 Raggruppare o ordinare alfabeticamente? . . . 1341
6.2 Dove eravamo rimasti . . . 1341
6.2.1 La forbice che si allarga . . . 1341
6.3 Come si conduce un brainstorming . . . 1342

6.3.1 Le condizioni . . . 1342
6.3.2 Il facilitatore . . . 1342
6.3.3 Pause e cibo . . . 1343
6.4 Altre tecniche per generare idee . . . 1343
6.4.1 Una lista, non un programma . . . 1343
6.4.2 Circle . . . 1343
6.4.3 Acting brainstorming . . . 1343
6.4.4 I sei cappelli . . . 1344
6.4.5 L'oggetto dal cappello . . . 1344
6.4.6 Storyboard . . . 1344
6.4.7 Le domande . . . 1344
6.4.8 Papà e mamma: peggiore è, meglio è . . . 1344
6.4.9 Associative e mind mapping . . . 1345
6.4.10 TRIZ . . . 1345
6.5 Le regole di Kawasaki e lo SCAMPER . . . 1346
6.5.1 Le regole dell'innovazione di Guy Kawasaki . . . 1346
6.5.2 SCAMPER . . . 1347
6.6 Dalle cento alle dieci idee . . . 1347
6.6.1 La brainstorming matrix . . . 1347
6.6.2 Almeno dieci idee diverse . . . 1348
6.6.3 Distruggere le proprie idee . . . 1348
6.7 Prototipi, movimento e routine da spezzare . . . 1349
6.7.1 KISS e il cuore del gioco . . . 1349
6.7.2 Muoversi, usare i sensi . . . 1349
6.7.3 Rituali che spezzano la routine . . . 1350
6.7.4 Chiusura della parte su problema e idea . . . 1350
6.8 Flowchart: che cosa sono . . . 1350
6.8.1 Matematica? Poca, e di base . . . 1350
6.8.2 Una definizione . . . 1351
6.8.3 Dove nascono . . . 1351
6.9 A che cosa servono a un designer . . . 1351
6.10 Le regole di lettura . . . 1352
6.10.1 Prima le si impara, poi le si piega . . . 1352
6.10.2 Dall'alto in basso, da sinistra a destra . . . 1352
6.10.3 Livelli e frecce . . . 1352
6.11 I blocchi . . . 1353
6.11.1 Inizio e fine . . . 1353
6.11.2 Il rettangolo: un'azione . . . 1353
6.11.3 Il rombo: una decisione . . . 1353
6.11.4 Il sotto-processo . . . 1353
6.11.5 Il parallelogramma: input e output . . . 1354
6.11.6 Il documento . . . 1355
6.12 Il flusso deve chiudersi . . . 1356
6.12.1 Che cosa c'è di sbagliato? . . . 1356
6.12.2 Gli stessi blocchi, in tante salse . . . 1356
6.12.3 Riepilogo e una domanda . . . 1357
6.13 Flowchart nella pratica: esempi che piegano le regole . . . 1358
6.13.1 Oxenfree: misurare il peso della narrativa . . . 1358
6.13.2 Il gioco di carte: un flowchart per sé . . . 1359
6.13.3 Machinations: un flowchart che si gioca . . . 1359
6.13.4 Il behavior tree: il caso più estremo . . . 1360
6.14 Il task della settimana: il flowchart di Pac-Man . . . 1360
6.14.1 La consegna . . . 1360
6.14.2 Le regole di consegna . . . 1361

6.14.3 Informazioni di servizio . . . 1362
6.15 Grafici: linea o gradini? . . . 1362
6.15.1 La domanda . . . 1362
6.15.2 Le risposte della classe . . . 1362
6.15.3 La risposta del docente . . . 1362
6.16 I grafici di base . . . 1363
6.16.1 Barre, colonne e istogrammi . . . 1363
6.16.2 La torta e la ciambella . . . 1364
6.16.3 Linee e aree . . . 1364
6.16.4 Il clustering . . . 1364
6.17 Oltre i grafici di base . . . 1364
6.17.1 Treemap e sunburst: torte travestite . . . 1364
6.17.2 Il grafico a radar . . . 1365
6.17.3 Il grafico a corde . . . 1365
6.18 La campagna di Russia in un grafico . . . 1366
6.18.1 Due avvertenze . . . 1366
6.18.2 Che cosa mostra? . . . 1366
6.18.3 La risposta . . . 1366
6.18.4 Dalle infografiche ai documenti . . . 1367
6.19 Principi per grafici migliori . . . 1367
6.20 La matematica di base . . . 1368
6.20.1 Quattro cose, da quinta elementare . . . 1368
6.20.2 La media . . . 1368
6.20.3 La media pesata . . . 1368
6.20.4 Le proporzioni . . . 1369
6.20.5 Che cosa verrà dopo . . . 1369
6.21 Chiusura e prossimi appuntamenti . . . 1369
Punti chiave del capitolo . . . 1370

7 Eventi, nodi e variabili 1371
7.1 Prima di cominciare: chi c'è, e chi ha già programmato . . . 1372
7.1.1 Le versioni dell'engine . . . 1372
7.1.2 Un giro di presentazioni . . . 1372
7.2 Due modi di programmare, tre concetti da imparare . . . 1372
7.2.1 C++ e Blueprint . . . 1372
7.2.2 Eventi, nodi, variabili . . . 1373
7.3 Il progetto in terza persona . . . 1373
7.3.1 Creare il progetto . . . 1373
7.3.2 L'editor, in due parole . . . 1373
7.3.3 Il personaggio: viewport ed Event Graph . . . 1374
7.4 Il primo Blueprint: un cubo . . . 1374
7.4.1 Blueprint Class, Actor . . . 1374
7.4.2 Un oggetto visibile, e più volte in scena . . . 1374
7.5 Gli eventi e il primo nodo . . . 1374
7.5.1 Begin Play e Tick . . . 1374
7.5.2 Print String . . . 1375
7.6 Le variabili . . . 1375
7.6.1 Che cos'è una variabile . . . 1375
7.6.2 I tipi, e i loro colori . . . 1375
7.6.3 Il valore di default . . . 1376
7.7 GET e SET . . . 1376
7.7.1 Leggere e scrivere . . . 1376
7.7.2 Un numero che sale: il primo punteggio . . . 1377
7.8 Il personaggio: tasti, lanci, scala e teletrasporto . . . 1377

7.8.1 Un tasto come evento . . . 1377
7.8.2 Launch Character . . . 1378
7.8.3 La scala: la prima abilità . . . 1378
7.8.4 Set Actor Location: il teletrasporto . . . 1378
7.8.5 Cinque minuti per esplorare . . . 1378
7.9 Il Character Movement . . . 1378
7.9.1 Dove si trova il movimento . . . 1378
7.9.2 Premuto e rilasciato: lo sprint . . . 1379
7.9.3 Delay e Branch . . . 1379
7.10 Le interazioni tra oggetti: la trigger box . . . 1380
7.10.1 Un'area che si accorge di chi entra . . . 1380
7.10.2 Controllare chi è entrato: il cast . . . 1380
7.10.3 L'indovinello del *target* . . . 1380
7.10.4 Tutto ciò che si poteva fare là, si può fare qua . . . 1381
7.11 Generare, distruggere, cadere . . . 1381
7.11.1 Spawn Actor e Destroy Actor . . . 1381
7.11.2 Simulate Physics . . . 1381
7.11.3 Ogni componente è una miniera di variabili . . . 1381
7.12 Le monete . . . 1382
7.12.1 Costruire la moneta . . . 1382
7.12.2 Raccoglierla . . . 1382
7.12.3 Contarle: la variabile sta sul personaggio . . . 1382
7.12.4 Il debug in diretta . . . 1383
7.13 L'editor in inglese, e il salvataggio . . . 1383
7.14 Altri nodi per stimolarvi . . . 1383
7.14.1 Confronti e Branch . . . 1384
7.14.2 Flip Flop e Sequence . . . 1384
7.15 Il task in classe: due livelli . . . 1384
7.15.1 La consegna . . . 1384
7.15.2 Requisiti minimi, e tutto il resto . . . 1385
7.15.3 Creare il secondo livello . . . 1385
7.16 Le monete che ruotano . . . 1385
7.17 L'Event Tick e i *delta seconds* . . . 1386
7.17.1 Un oggetto che si muove . . . 1386
7.17.2 Il problema grave . . . 1386
7.17.3 La soluzione: moltiplicare per i delta seconds . . . 1386
7.18 Ripasso: la trigger box, passo per passo . . . 1387
7.18.1 Perché non la Trigger Box già pronta? . . . 1387
7.19 Un sistema di vita . . . 1387
7.19.1 La proposta . . . 1387
7.19.2 Ripartire: Open Level sullo stesso livello . . . 1388
7.19.3 Le variabili non sopravvivono al cambio di livello . . . 1388
7.20 Actor, Pawn, Character; e il Line Trace . . . 1388
7.20.1 Le tre classi . . . 1388
7.20.2 Il Line Trace By Channel . . . 1389
7.21 Le regole dei collegamenti . . . 1389
7.22 Le domande di chi parte da zero . . . 1390
7.22.1 "Che valore ha la freccina bianca?" . . . 1390
7.22.2 La corrente che fa il giro: il Delay . . . 1390
7.22.3 L'interfaccia, ridotta all'osso . . . 1390
7.22.4 Interrompere è un favore a tutti . . . 1390
7.22.5 Il task, riassunto . . . 1391
7.23 I progetti, e la chiusura . . . 1391
Punti chiave del capitolo . . . 1391

8 Introduzione al level design 1393
8.1 Prima di cominciare: gli annunci pratici . . . 1394
8.1.1 Cinque minuti di margine . . . 1394
8.1.2 In aula o da remoto . . . 1394
8.1.3 Pausa e appello . . . 1394
8.2 Chi insegna . . . 1394
8.3 Il giro di presentazioni . . . 1395
8.3.1 Le sei domande . . . 1395
8.3.2 L'esempio del docente: la Cattedrale del Sangue Sacro . . . 1395
8.3.3 Valerio: Subnautica . . . 1396
8.3.4 Alberto: Nightreign, e la differenza tra scelta ed errore . . . 1396
8.3.5 Mattia: Monster Hunter, il sound design e la Città Infame . . . 1397
8.3.6 Francesco: i JRPG, Fabula Ultima e il salto finale . . . 1398
8.3.7 Federica: i giochi intimi e la mappa di Pentiment . . . 1399
8.3.8 Daniele: l'esplorazione e gli spazi vuoti . . . 1400
8.3.9 Francesca: i giochi psicologici e un labirinto di piani . . . 1400
8.3.10 Antonino: i roguelike e la tempesta di Frostpunk 2 . . . 1400
8.4 Perché presentarsi: parlare in pubblico . . . 1401
8.5 Che cos'è il level design . . . 1401
8.6 Che cosa fa un level designer . . . 1402
8.6.1 La risposta breve . . . 1402
8.6.2 La risposta lunga: un termine ombrello . . . 1402
8.7 Lo strumento del level designer . . . 1403
8.7.1 Il braccio armato dello sviluppo . . . 1403
8.7.2 La parola magica: "dipende" . . . 1404
8.7.3 La risposta vera: il level designer . . . 1404
8.8 Come si impara il level design . . . 1405
8.8.1 La teoria . . . 1405
8.8.2 La pratica . . . 1405
8.8.3 L'expertise . . . 1405
8.8.4 L'esperienza sul campo . . . 1405
8.8.5 Il gruppo . . . 1405
8.8.6 La parola che si ripete . . . 1406
8.9 Due curve da conoscere in anticipo . . . 1406
8.9.1 Il picco della fiducia . . . 1406
8.9.2 L'effetto Dunning-Kruger . . . 1406
8.10 Tre corollari . . . 1407
8.10.1 Primo: non esistono regole fisse . . . 1407
8.10.2 Secondo: il design è una disciplina olistica . . . 1407
8.10.3 Terzo: il design non è arte, è artigianato . . . 1408
8.10.4 Sporcarsi le mani . . . 1408
8.11 La struttura del corso . . . 1408
8.12 Il progetto in Unreal Engine . . . 1409
8.12.1 Lanciare l'engine in anticipo . . . 1409
8.12.2 Creare il progetto . . . 1409
8.13 L'editor: muoversi e piazzare oggetti . . . 1409
8.13.1 La viewport . . . 1410
8.13.2 Place Actors e le forme di base . . . 1410
8.13.3 Il gizmo . . . 1410
8.14 L'Outliner: tenere in ordine il livello . . . 1410
8.15 Il Content Browser: asset e actor . . . 1411
8.15.1 Istanze e stampi . . . 1412
8.16 Il transform e la finestra Details . . . 1412
8.16.1 Copiare, incollare, annullare . . . 1412

8.16.2 Il transform . . . 1413
8.16.3 La finestra Details . . . 1413
8.16.4 W, E, R (e Q) . . . 1413
8.16.5 I gruppi . . . 1414
8.16.6 Le operazioni nei campi numerici . . . 1414
8.17 Leggere la documentazione . . . 1414
8.18 I livelli e il salvataggio . . . 1414
8.18.1 Le scorciatoie utili . . . 1415
8.19 Il task: il blockout della propria stanza . . . 1415
8.20 Per chi vuole approfondire . . . 1416
8.21 La chiusura . . . 1416
Punti chiave del capitolo . . . 1417

9 Agency, comportamento e visione 1419
9.1 In apertura: impaginazione, riconsegne, errori . . . 1419
9.1.1 Prima di cominciare . . . 1419
9.1.2 Un problema di impostazione pagina . . . 1420
9.1.3 Che cosa succede a un task non letto . . . 1420
9.1.4 Come si riconsegna . . . 1420
9.1.5 Il limite dei cinque errori e la dislessia . . . 1421
9.2 La libertà non esiste: esiste l'agency . . . 1421
9.2.1 Un concetto che ritorna da Bura . . . 1421
9.2.2 Ciò che le persone cercano davvero . . . 1421
9.2.3 Esempi di agency . . . 1422
9.2.4 E il contrario: togliere agency . . . 1422
9.3 Troppe scelte: il carretto, i cereali, gli skill tree . . . 1422
9.3.1 Il carretto dei dolci . . . 1422
9.3.2 I cereali . . . 1423
9.3.3 Gli skill tree . . . 1423
9.3.4 Il cervello non è fatto per i grandi numeri . . . 1423
9.3.5 La paura di scegliere, e il rimorso . . . 1423
9.3.6 Rimborsi e scelte permanenti . . . 1424
9.4 Il paradosso della scelta . . . 1424
9.4.1 Più scelte, più infelicità . . . 1424
9.4.2 Che cosa ce ne facciamo come designer . . . 1424
9.5 Come decidono i giocatori: la Theory of Planned Behavior . . . 1426
9.5.1 Un framework per leggere i comportamenti . . . 1426
9.5.2 La struttura . . . 1426
9.5.3 Intenzione e comportamento: l'orfanotrofio dei gattini . . . 1426
9.5.4 L'attitudine . . . 1427
9.5.5 Le norme soggettive . . . 1427
9.5.6 Il perceived behavioral control . . . 1428
9.5.7 Le barriere . . . 1429
9.6 Mettere a terra la TPB: due modi di usarla . . . 1429
9.6.1 Ad alto livello: riempire lo schema . . . 1429
9.6.2 Con i numeri: la formula semplificata . . . 1429
9.7 Un esempio reale: il meta di Valorant . . . 1431
9.7.1 La scelta degli agenti . . . 1431
9.7.2 Il survey, e una lezione sulle risposte aperte . . . 1431
9.8 Il feedback, e da dove viene la TPB . . . 1431
9.8.1 Senza feedback non c'è comportamento ripetuto . . . 1431
9.8.2 La situational awareness . . . 1432
9.9 Il compendium, o design bible . . . 1432
9.9.1 Due documenti di visione . . . 1432

9.9.2 Dalla Bibbia alla wiki, al compendium . . . 1433
9.9.3 Che cosa contiene . . . 1433
9.10 I pilastri: l'esempio di God of War . . . 1434
9.10.1 La visione va documentata . . . 1434
9.10.2 I tre pilastri di God of War . . . 1434
9.11 Il documento di vision . . . 1435
9.11.1 Che cos'è . . . 1435
9.11.2 Pilastri e prime emotions . . . 1436
9.11.3 Obiettivo, meaning e morale . . . 1436
9.11.4 Must do, must not, nice to have, USP . . . 1436
9.11.5 In coda al documento . . . 1437
9.11.6 Le regole del documento . . . 1437
9.12 La team vision . . . 1438
9.12.1 Un documento per il team, non per il gioco . . . 1438
9.12.2 Siate onesti . . . 1438
9.12.3 Che cosa contiene . . . 1438
9.13 La fine della pre-produzione, ripresa . . . 1439
9.14 Template e linee guida: sperimentate adesso . . . 1439
9.14.1 Linee guida, non template . . . 1439
9.14.2 Il campo di prova ideale . . . 1440
9.15 L'esercizio: i core elements di Frogger, dal vivo . . . 1440
9.15.1 La scelta del gioco e del volontario . . . 1440
9.15.2 La lista dei core elements, a memoria . . . 1440
9.15.3 Le indicazioni del docente durante l'esercizio . . . 1440
9.16 La riformattazione del docente . . . 1442
9.16.1 Un modo, non il modo . . . 1442
9.16.2 Il titolo . . . 1442
9.16.3 Gli stili e le intestazioni . . . 1442
9.16.4 Le game qualities: dalla lista alla tabella . . . 1442
9.16.5 Il tema in un box . . . 1443
9.16.6 Gli obiettivi in due box . . . 1443
9.16.7 Le regole: mostrare invece di scrivere . . . 1443
9.16.8 Una o due pagine . . . 1444
9.17 La chiusura . . . 1445
Punti chiave del capitolo . . . 1445

10 Dare, raccogliere e tracciare il feedback 1447
10.1 In apertura: il task su compendium e vision . . . 1447
10.1.1 Documentare, finché non avrete di meglio da fare . . . 1447
10.1.2 Ricordatemi i task . . . 1448
10.1.3 Il compendium di Metroid Dread . . . 1448
10.1.4 La vision di Metroid Dread . . . 1448
10.1.5 Gli altri due punti . . . 1449
10.2 Serve una legenda della formattazione? . . . 1449
10.3 Il disclaimer sui task: il design va descritto nel dettaglio . . . 1449
10.3.1 Un design amputato . . . 1449
10.3.2 L'esempio del battle pass . . . 1450
10.3.3 Nuove UI e nuove carte . . . 1450
10.3.4 Fare design significa trattare tutti i dettagli . . . 1450
10.4 Una premessa sulla severità . . . 1451
10.4.1 Niente di personale . . . 1451
10.4.2 Il mondo là fuori . . . 1451
10.4.3 Che cosa si vedrà . . . 1451
10.5 Gli assunti di base . . . 1451

10.5.1 Dall'altra parte c'è un umano . . . 1451
10.5.2 Le soft skill . . . 1452
10.5.3 I feedback sono costruttivi . . . 1452
10.5.4 Feedback 101 . . . 1453
10.6 Come si compone un feedback . . . 1453
10.6.1 Costruttivo . . . 1454
10.6.2 Neutrale . . . 1454
10.6.3 Sincero . . . 1454
10.6.4 Risolutivo . . . 1454
10.6.5 Senza giudizio . . . 1455
10.6.6 La tecnica del sandwich non funziona . . . 1455
10.7 Chiari, precisi, analitici . . . 1456
10.7.1 Chiari . . . 1456
10.7.2 Precisi . . . 1456
10.7.3 Analitici . . . 1456
10.8 Alcune pillole aggiuntive . . . 1457
10.8.1 Siate veritieri: non state zitti . . . 1457
10.8.2 Teamwork: un team, non una famiglia . . . 1457
10.8.3 Le nomenclature: una truffa che funziona . . . 1457
10.8.4 Il linguaggio del corpo . . . 1458
10.9 Il task: siete il lead di un gioco di zombie . . . 1459
10.9.1 La consegna . . . 1459
10.9.2 Il fiero . . . 1459
10.9.3 Che cosa consegnare . . . 1460
10.10 Raccogliere i feedback . . . 1461
10.10.1 Testare: prima possibile . . . 1461
10.10.2 Chiedere: chi sono i vostri tester? . . . 1462
10.10.3 Collezionare: più dati possibili . . . 1462
10.11 I survey . . . 1462
10.11.1 Gli strumenti . . . 1462
10.11.2 Le regole del survey . . . 1463
10.11.3 Sia lodata la scala di Likert . . . 1463
10.11.4 Ricompense: meglio le gemme che gli euro . . . 1464
10.12 Spacchettare i feedback: l'esempio di God of War . . . 1465
10.12.1 Due tester . . . 1465
10.12.2 Primo: identificare il target audience . . . 1465
10.12.3 Secondo: la rilevanza statistica . . . 1465
10.12.4 Terzo: tradurre il linguaggio . . . 1466
10.12.5 Quarto: le risposte emotive . . . 1466
10.13 I feedback dei competitor . . . 1466
10.14 La curva dell'utenza e i consigli finali . . . 1467
10.14.1 Evangelisti e odiatori . . . 1467
10.14.2 Problemi, mai soluzioni . . . 1468
10.14.3 Difendete la vostra visione . . . 1468
10.14.4 Data informed, non data driven . . . 1469
10.14.5 Non eliminate subito ciò che l'utenza odia . . . 1469
10.14.6 Prima e dopo il lancio . . . 1469
10.15 Una lezione jolly . . . 1470
10.16 Il versioning tracker . . . 1471
10.16.1 Il libretto di istruzioni dell'IKEA . . . 1471
10.16.2 Le regole d'oro . . . 1471
10.16.3 Che cosa contiene . . . 1471
10.16.4 L'esempio: la camminata . . . 1472
10.17 Il documento di playtest . . . 1473

10.17.1 Obiettivo: ottenere dati . . . 1473
10.17.2 I punti fermi . . . 1473
10.17.3 Il documento non basta: bisogna testare . . . 1474
10.18 La chiusura: lavorare entro dei limiti . . . 1475
10.18.1 Che cosa è arrivato della documentazione pratica? . . . 1475
10.18.2 La soluzione sta nel problema . . . 1475
10.18.3 I blocchetti di Lego . . . 1475
10.18.4 I prossimi appuntamenti . . . 1476
Punti chiave del capitolo . . . 1476

11 Blockout, coordinate e luci 1479
11.1 La struttura delle lezioni del primo anno . . . 1480
11.1.1 Cinque minuti per chi arriva . . . 1480
11.1.2 Prima i task, tutti insieme . . . 1480
11.2 La revisione dei blockout della stanza . . . 1480
11.2.1 Francesca: cubi, piani e un CD sotto la sedia . . . 1480
11.2.2 Antonino: una PlayStation 5, e la cartella condivisa . . . 1481
11.2.3 Federica: abbiamo appena traslocato . . . 1481
11.2.4 Mattia: le foto allo schermo, e uno spadone . . . 1481
11.2.5 Francesco: la stanza senza soffitto, e il verosimile . . . 1482
11.2.6 Alberto: lo showroom dell'IKEA . . . 1482
11.2.7 Valerio: super preciso, e il problema della scala . . . 1482
11.3 La lezione di oggi, e lo Starter Content che non c'è . . . 1482
11.4 Lavoro artistico e lavoro di gameplay . . . 1483
11.4.1 Il termine ombrello, di nuovo . . . 1483
11.4.2 La specializzazione . . . 1483
11.5 Il blockout . . . 1484
11.5.1 Un concetto, molti nomi . . . 1484
11.5.2 Perché non partire dall'arte finale? . . . 1484
11.5.3 Testare idee e rispondere a domande . . . 1485
11.6 Due esempi: Call of Duty e Uncharted . . . 1485
11.6.1 Una mappa multiplayer di Call of Duty . . . 1485
11.6.2 Il level designer dà ordini agli artisti? . . . 1487
11.6.3 Uncharted: dichiaro che è una cascata . . . 1487
11.7 I cinque strumenti del blockout . . . 1487
11.7.1 I primitivi . . . 1488
11.7.2 I kit modulari . . . 1488
11.7.3 Brush e modellazione . . . 1490
11.7.4 La scultura del terreno . . . 1490
11.7.5 Le spline . . . 1491
11.8 Che cosa si farà nel primo anno . . . 1492
11.9 Asset e actor, di nuovo . . . 1492
11.10 Coordinate globali e coordinate locali . . . 1492
11.10.1 L'origine del mondo . . . 1492
11.10.2 Coordinate locali: la spada e la cassa . . . 1493
11.10.3 La pausa . . . 1494
11.11 Lo spazio delle coordinate nell'editor . . . 1494
11.12 Genitori, figli e gruppi . . . 1494
11.12.1 Parent e child . . . 1494
11.12.2 I gruppi . . . 1496
11.13 Ctrl+Z: anche cliccare è un'azione . . . 1496
11.14 Lo snapping . . . 1496
11.14.1 Che cos'è . . . 1496
11.14.2 La barra degli strumenti della viewport . . . 1496

11.14.3 Lo snapping della rotazione . . . 1497
11.14.4 Ruotare attorno a un asse . . . 1497
11.15 Il Content Browser e la lingua dell'editor . . . 1498
11.16 Il pivot . . . 1498
11.17 Mesh e collisioni . . . 1499
11.17.1 Che cos'è una mesh . . . 1499
11.17.2 Le collisioni sono un'altra cosa . . . 1499
11.18 Le luci . . . 1500
11.18.1 La point light . . . 1500
11.18.2 La spot light . . . 1501
11.18.3 La rect light . . . 1501
11.18.4 La directional light: il sole . . . 1501
11.19 Lo Starter Content, a mano . . . 1502
11.19.1 Un nuovo progetto, e un engine cambiato . . . 1502
11.19.2 Perché l'importazione normale non funziona . . . 1502
11.19.3 La procedura . . . 1502
11.19.4 Che cosa c'è dentro . . . 1503
11.20 Un livello da zero . . . 1503
11.20.1 Il livello vuoto . . . 1503
11.20.2 Un piano, una luce, un cubo . . . 1504
11.20.3 Una parentesi su Lumen . . . 1504
11.20.4 Perché il lato in ombra è nero . . . 1504
11.20.5 La cartella Lighting: il modo pratico . . . 1505
11.21 Materiali e texture . . . 1505
11.21.1 Assegnare un materiale . . . 1505
11.21.2 Il tiling e lo stretch . . . 1506
11.21.3 Texture e materiale non sono la stessa cosa . . . 1506
11.22 Le modalità di visualizzazione . . . 1506
11.23 Il post process volume . . . 1507
11.24 Il task: una stanza, qualunque stanza . . . 1508
11.25 Le domande e la chiusura . . . 1509
Punti chiave del capitolo . . . 1509

VI Revisioni dei Compiti 1511

1 Gli errori nei task 1513
1.1 Come usare questo capitolo . . . 1513
1.1.1 Struttura . . . 1513
1.1.2 Legenda . . . 1514
1.1.3 I tre livelli di gravità . . . 1514
1.2 Il registro degli avvertimenti . . . 1514
1.3 La checklist prima della consegna . . . 1517
1.3.1 Forma e regole formali . . . 1517
1.3.2 Linguaggio . . . 1517
1.3.3 Contenuto e framework . . . 1518
1.3.4 Dati, analisi e design . . . 1518
1.3.5 Flowchart . . . 1519
1.4 Task 1 — Arcade core elements . . . 1519
1.4.1 La consegna . . . 1519
1.4.2 Le consegne in sintesi . . . 1519
1.4.3 Errori di forma e di impaginazione . . . 1521
1.4.4 Errori di scrittura e di linguaggio . . . 1523
1.4.5 La definizione di gioco arcade . . . 1526

1.4.6 Dichiarare e usare il framework . . . 1527
1.4.7 Descrivere le meccaniche: il dettaglio richiesto . . . 1528
1.4.8 Narrativa, estetiche e tecnologia . . . 1530
1.4.9 Mostrare invece di scrivere . . . 1531
1.4.10 Che cosa è stato apprezzato . . . 1532
1.4.11 Le schede delle singole consegne . . . 1532
1.4.12 Punti chiave del task T1 . . . 1545
1.5 Task 2 — Production dictionary di un Pokémon . . . 1545
1.5.1 La consegna . . . 1545
1.5.2 Le consegne in sintesi . . . 1546
1.5.3 I documenti non letti . . . 1547
1.5.4 Errori di forma e di impaginazione . . . 1548
1.5.5 Errori di scrittura e di linguaggio . . . 1550
1.5.6 La voce del dizionario: che cosa deve contenere . . . 1551
1.5.7 Priorità: che cosa entra in una pagina . . . 1553
1.5.8 Che cosa è stato apprezzato . . . 1555
1.5.9 Dal task 1 al task 2: confronto e pattern ricorrenti . . . 1555
1.5.10 Le schede delle singole consegne . . . 1558
1.5.11 Punti chiave del task T2 . . . 1564
1.6 Task 3 — TCG pain points (Marvel Snap e Yu-Gi-Oh! Master Duel) . . . 1564
1.6.1 La consegna . . . 1564
1.6.2 Le consegne in sintesi . . . 1565
1.6.3 Errori di forma e di impaginazione . . . 1566
1.6.4 Dati e fonti: mostrare il feedback . . . 1567
1.6.5 L'analisi: trovare il problema vero . . . 1567
1.6.6 Il design delle soluzioni: dall'idea alla proposta . . . 1568
1.6.7 Che cosa è stato apprezzato . . . 1570
1.6.8 Dai task 1 e 2 al task 3: confronto e pattern ricorrenti . . . 1570
1.6.9 Le schede delle singole consegne . . . 1572
1.6.10 Punti chiave del task T3 . . . 1575
1.7 Task 4 — Il flowchart di Pac-Man . . . 1576
1.7.1 La consegna . . . 1576
1.7.2 Le consegne in sintesi . . . 1576
1.7.3 Le regole del flowchart . . . 1576
1.7.4 La logica del gioco: implementabile o rotta? . . . 1580
1.7.5 Leggibilità e forma . . . 1582
1.7.6 Completare il task: profondità, tempo e compromessi . . . 1583
1.7.7 Che cosa è stato apprezzato . . . 1584
1.7.8 Dai task precedenti al task 4: confronto e pattern ricorrenti . . . 1585
1.7.9 Le schede delle singole consegne . . . 1585
1.7.10 Punti chiave del task T4 . . . 1589

Part I

Seminari per il Game Designer

Chapter 0.1

Lavorare in squadra: la cultura del feedback

Questo capitolo apre le *lezioni zero*: incontri che precedono l'inizio vero e proprio del corso di Game Design e che non riguardano una tecnica di design in senso stretto, ma **la figura professionale** di chi lavora nell'industria dei videogiochi. Prima di imparare a progettare un gioco conviene infatti capire in che tipo di ambiente lo si progetterà: un ambiente fatto di team, di gerarchie, di scadenze e, soprattutto, di persone che devono riuscire a lavorare insieme.

La lezione è la prima giornata di un **workshop di due giorni** (circa dieci ore in totale, pause pranzo comprese) tenuto da Stefano Cozzi[1], un professionista con circa quindici anni di esperienza nel settore, tra Blizzard Entertainment, Riot Games e, oggi, Stormind Games. Il workshop è nato da un confronto con i coordinatori dell'accademia[2]: serviva un momento dedicato a *come ci si comporta in un ambiente professionale*, cosa che riguarda sia il lavoro futuro sia i progetti di gruppo svolti durante il percorso di studi, dove ci si deve già abituare a comportarsi "come se si fosse in un'azienda".

La prima giornata ha un unico grande tema, sezionato in più parti: **la cultura del feedback**. La seconda giornata (che sarà oggetto della prossima lezione zero) sarà più pratica: *time management*, *brainstorming* e il "guidare da dentro" un team anche senza esserne il leader (sezione 0.1.18).

Il percorso del capitolo segue quello della lezione:

1. il formato del workshop e il percorso professionale del relatore (sezione 0.1.1);
2. perché le aziende tengono tanto al lavoro di squadra e al *fit* culturale (sezione 0.1.2);
3. che cos'è il feedback e da che cosa dipende il modo di darlo (sezione 0.1.3);
4. le cinque fasi di vita di un team e il valore del *post-mortem* (sezioni 0.1.4 e 0.1.5);
5. il confronto con i colleghi e alcune domande sul lavoro nei grandi studi (sezioni 0.1.6 e 0.1.7);
6. il circolo virtuoso tra feedback, miglioramento e fiducia, e i suoi effetti sulla *retention* e sulla cultura aziendale (sezioni 0.1.8 e 0.1.9);
7. il "gioco del traditore" e il principio del **default to trust** (sezioni 0.1.10 e 0.1.11);
8. le traiettorie di carriera e la valutazione delle performance (sezione 0.1.12);
9. come si dà, come si riceve e quando si dà un feedback (sezioni 0.1.13, 0.1.14 e 0.1.15);

[1] Il cognome è riportato come compare nella trascrizione automatica, che in più punti storpia i nomi propri; andrebbe verificato. Nella stessa trascrizione lo studio per cui lavora appare come "Stormine", "Stormline" e "Stormind": si tratta di Stormind Games, studio con sede a Catania, coerentemente con quanto detto in lezione.

[2] Nella trascrizione l'accademia è resa in modi diversi ("Hive", "a dive", "D.I.V."); i coordinatori citati sono Enrico e Andrea, insieme al direttore Diego Ricchiuti. Nel resto del capitolo si usa semplicemente "l'accademia", come negli altri capitoli.

10. il feedback come regalo, le domande finali e le anticipazioni della seconda giornata (sezioni 0.1.16, 0.1.17 e 0.1.18).

Una premessa di metodo: il relatore ha alternato continuamente teoria, domande alla classe e racconti della propria esperienza. I racconti non sono digressioni: sono gli esempi concreti con cui i principi vengono resi tangibili, e per questo sono riportati con cura.

0.1.1 Il workshop e il relatore

0.1.1.1 Formato e regole di partecipazione

Il relatore ha cercato di "riassumere quindici anni di esperienza in dieci ore". Ha materiale per circa il doppio delle ore disponibili: le cose strettamente necessarie vengono trattate per prime, il resto è tenuto "in riserva" come *nice to have*. Le regole del workshop:

- le domande si fanno **alla fine di ogni slide**, così da chiudere prima il discorso; non c'è un'unica sessione finale di domande e risposte (anche se ci sarà), perché il relatore preferisce fermarsi spesso e non fare "uno spiegone di dieci ore";
- le domande non devono per forza riguardare la slide appena vista: se una slide dà lo spunto per altro, se ne può parlare; al massimo sarà il relatore a rispondere in breve o a rimandare a una slide successiva;
- si può chiedere anche di esperienze passate, opinioni e sensazioni; se per NDA o per serietà professionale non può rispondere, lo dirà esplicitamente ("non lasciate che sia un *blocker*");
- su Zoom si usa l'alzata di mano o la chat, che il relatore tiene aperta sul secondo monitor;
- non ci sono pause ufficiali: la lezione è registrata e ognuno è libero di assentarsi quando serve, cercando però di non disturbare il flusso della lezione;
- le slide verranno fornite integralmente tramite la segreteria dell'accademia (Vanessa), insieme alle registrazioni: le slide senza il parlato servono poco, ma sono un buon promemoria.

0.1.1.2 Il percorso professionale

Il relatore si presenta (scherzosamente) come la "guida escursionistica" che accompagna la classe nella "foresta incantata" della cultura del feedback, perché per molti è un territorio sconosciuto. Il suo percorso:

- in Italia ha lavorato per aziende che fornivano servizi a terzi, tra cui il lavoro su *Warhammer Online*, e per un canale televisivo sul digitale terrestre dedicato ai videogiochi[3];
- a **Blizzard Entertainment** (circa sei anni, a Parigi) ha fatto il *community manager* per l'Italia (*StarCraft II*, *World of Warcraft* da *Mists of Pandaria* in poi, *Hearthstone*, *Diablo III*, le espansioni di *StarCraft*, il lancio di Battle.net), per poi diventare il manager dei community manager europei;
- a **Riot Games** (circa cinque anni, a Dublino) è stato brevemente community manager italiano e poi *publishing manager* di un team di circa trenta persone per l'Europa. Il *publishing* non è solo community: è l'unione di **community, marketing e PR**;
- a **PG Esports** si è occupato di eventi dal vivo e online (ad esempio la Milano Games Week e tornei di *Valorant* e *League of Legends*), restando in contatto con Riot e Blizzard;
- da circa un anno è tornato nello sviluppo, a **Stormind Games**, dove oggi si occupa di risorse umane e organizzazione del lavoro; lo studio verrà presentato più ampiamente al *Career Day* dell'accademia insieme alla sua HR manager;

[3] Nella trascrizione: "Digital Game Network Channel". Il nome esatto del canale andrebbe verificato.

- fa inoltre consulenze aziendali anche al di fuori dei videogiochi e insegna in master universitari (IULM di Milano).

Il suo ruolo ricorrente, sia a Blizzard sia a Riot, è stato quello di *Mr. Wolf* ("per chi ha visto *Pulp Fiction*"): quello che risolve i problemi. Veniva inserito nei team più in difficoltà, ad esempio dal punto di vista delle performance, per sistemarli e passare al successivo. Era presente in alcuni dei momenti più critici, come il lancio di *Diablo III* o il lancio di espansioni di *World of Warcraft*[4], a fare *damage control*.

La scoperta fondamentale di questi anni è che i problemi da risolvere **quasi mai riguardavano il prodotto** (non era certo lui a sistemare i server di *Diablo III*), ma i team che c'erano dietro: persone che litigavano, non collaboravano, non riuscivano a stare nella stessa stanza senza urlarsi contro. E nel 90% dei casi tutto questo si riduce a **problemi di comunicazione**: è raro che perfino le incompatibilità caratteriali non siano risolvibili parlando.

0.1.1.3 Struttura non vuol dire burocrazia

A Stormind il relatore sta accompagnando un team passato da 30 a 95 persone in meno di un anno, attraverso tutti i cambiamenti di cui parla il workshop. Darsi una struttura viene spesso percepito male, come "più processi, più procedure", ma bisogna distinguere:

- un processo come "timbrare il cartellino" può essere davvero invasivo e noioso;
- definire *come funziona un documento di game design, chi lo approva, chi lo visiona e chi no* è invece una struttura necessaria.

Uno studio di 30 persone nella stessa sede può farne a meno: quando c'è un dubbio ci si alza e si va alla scrivania del collega. Uno studio di 95 persone sparse per l'Italia (e in parte per l'Europa) non può più farlo: servono una struttura diversa, un modo di lavorare diverso e una pianificazione del futuro diversa.

0.1.2 Perché il lavoro è sempre lavoro di squadra

0.1.2.1 Non si va in ufficio per fare amicizia

Il primo messaggio è netto: **non si va in ufficio per fare amici** o per trovare compagni di vita. Si va in ufficio perché c'è un lavoro da fare, e lo scopo è riuscire a farlo *anche con chi sta antipatico*. Con le persone simpatiche è facile: diventeranno amici, e non sarà mai un problema. È con chi sta antipatico che le cose si complicano, ed è lì che emergono i **veri professionisti**: quelli che riescono a lavorare con chiunque senza che l'antipatia pesi sul modo in cui lavorano e collaborano, mettendo il prodotto e il team davanti al proprio ego e alle proprie preferenze personali.

La frase che fa da filo conduttore a tutto il workshop (e che il relatore promette di riproporre "fino alla nausea") è:

> *La capacità di un gruppo di persone di fare cose straordinarie dipende dal modo in cui queste persone operano insieme.*

Le aziende raramente pensano che l'innesto di una singola persona possa cambiare il destino di un progetto. Questo è tanto più vero quanto più grande è il team, ma vale anche per i piccoli studi, anche per un gruppo di cinque persone. Come negli sport di squadra: l'individuo conta fino a un certo punto, e anche se si ha Dončić in squadra, se si gioca uno contro cinque si perde comunque[5].

[4] La trascrizione riporta "il lancio con i Gary's" e "World of Trenor": si tratta verosimilmente di *Mists of Pandaria* e di *Warlords of Draenor*.

[5] Il riferimento è alle finali NBA in corso nei giorni del workshop tra Dallas Mavericks e Boston Celtics (giugno 2024), il che permette di datare approssimativamente la lezione.

0.1.2.2 Lo *useless genius* e il costo delle persone tossiche

Molti dipartimenti HR usano la definizione di **useless genius**[6]: si può essere bravissimi, a volte i migliori nel proprio lavoro, ma se non ci si sa comportare, relazionare e operare in un gruppo si è fondamentalmente inutili ai fini del gruppo. Nel migliore dei casi inutili, nel peggiore *dannosi*.

In passato le aziende tendevano a "chiudere un occhio": la persona è fondamentale, la teniamo. Oggi la domanda è diventata: *quanto costa all'azienda, oltre al suo stipendio, tenere una persona che crea un ambiente tossico e peggiora chi le sta intorno?* Ci sono costi sommersi che le aziende cominciano a vedere, e la tendenza è liberarsi di queste persone il prima possibile. Il ragionamento è semplice:

- le **competenze si possono trasmettere** o sostituire: si trova qualcuno che fa lo stesso lavoro magari nel doppio del tempo, oppure più persone che se lo dividono;
- il **carattere delle persone** invece cambia molto raramente.

0.1.2.3 Il fit culturale

Quando qualcuno **non è un *fit* culturale** per un'azienda è meglio separarsi presto. Ma non essere adatti a *quell'*azienda non significa non essere adatti a lavorare: è perfettamente legittimo che una persona inadatta a un'azienda sia adattissima a un'altra.

Il relatore l'ha vissuto in prima persona. Dopo sei anni a Blizzard viene contattato da Riot e affronta un processo di selezione lunghissimo ("quasi undici colloqui"), al termine del quale viene scartato. Scrive allora una mail che gli ha "cambiato la traiettoria di carriera": chiede semplicemente perché non è stato ritenuto adatto. La risposta: in nessun colloquio si era esposto abbastanza da far capire *perché* volesse lasciare Blizzard, e quindi pensavano che in realtà stesse bene dov'era. Il motivo era che, per educazione professionale, non gli piace parlare male del proprio posto di lavoro; portata all'eccesso, però, questa riservatezza era diventata un "è tutto bellissimo, ma vorrei provare Riot", senza un perché. Lo invitano comunque a un incontro dal vivo ("dal vivo è diverso"), e alla fine viene assunto.

Una volta arrivato, i sei mesi di prova (*probation*) sono serviti tutti per smettere di ragionare e lavorare "come a Blizzard". La differenza culturale era enorme, tanto che in Riot esisteva un gruppo di supporto interno tra ex dipendenti Blizzard: un collega gliel'ha descritto come "uscire da una relazione tossica e doversi riabituare a una relazione normale"[7].

Blizzard era *developer first*: il focus era chiaramente sugli sviluppatori, prima di tutti gli altri. Dopo sei anni c'erano ancora meeting a cui non poteva partecipare perché non abbastanza *senior*, informazioni che non poteva sapere, piani che non poteva vedere.

Riot era *player first*: prima i giocatori, e i dipendenti come mezzo per trasferire il feedback dei giocatori dentro il gioco. Il secondo giorno lo portano in una stanza e gli mostrano il piano dei cinque anni successivi. Lui, sorpreso, pensa a un errore ("sono qui da oggi, non vorrei mettervi nei guai"); la risposta è: "sei un dipendente Riot o no? Allora non ci sono livelli di accesso: devi sapere queste cose per fare meglio il tuo lavoro".

Da questa esperienza ha imparato molto sulla **trasparenza**: condividere le informazioni aiuta il lavoro. Non sempre, però: a volte non si condivide qualcosa non per segretezza, ma perché non è ancora pronta per essere presentata, esattamente come non si mostra un gioco quando è ancora solo teoria ("te lo faccio vedere quando ho qualcosa di tangibile").

La lezione per gli studenti è doppia: le aziende valuteranno il *fit* culturale sempre di più, ma capire se si è un *fit* richiede anche **lavoro da parte del candidato**. Bisogna avere la presenza di spirito di dire: "conosco i valori di questa azienda, non mando nemmeno il CV, perché so che durerei un mese e mezzo".

[6] Il concetto è più noto in letteratura manageriale come *brilliant jerk*; in lezione è usata l'espressione *useless genius*.

[7] Il relatore precisa che parla della Blizzard di allora, che non esiste più: oggi è parte di Microsoft e nessuno dei suoi ex colleghi vi lavora ancora, anche a causa dei licenziamenti di massa.

0.1.3 Che cos'è il feedback

0.1.3.1 Le prime risposte della classe

La prima domanda alla classe è: *quando sentite la parola "feedback", a cosa pensate?* Le risposte raccolte, a voce e in chat:

- un **riscontro** su ciò che si fa e su come lo si fa, sul lavoro o sul modo in cui ci si interfaccia con il gruppo; può essere lavorativo oppure comunicativo e comportamentale. Chi ha risposto lo vede come qualcosa che, se l'azienda non dà, va attivamente richiesto, perché è importante sapere come sta andando per migliorarsi;
- né positivo né negativo, ma **costruttivo**: in entrambi i casi serve a migliorarsi;
- uno *scambio di opinioni*, un *riscontro produttivo*;
- un feedback "di reazione", non verbale: si fa una cosa e si osserva come cambia l'ambiente o il comportamento delle persone intorno (anche perché a voce la gente spesso si tiene le cose negative);
- per un designer, anche il **linguaggio del corpo**: osservando un giocatore giocare si può dedurre l'emozione che prova (frustrazione, incomprensione).

Sulla forma (scritto o orale?) le opinioni divergono: c'è chi lo preferisce *mille volte* scritto; chi pensa che dipenda dalle dimensioni dell'azienda (orale nelle piccole, scritto in quelle con team di centinaia di persone); chi pensa che dipenda dall'importanza del feedback (scritto se tecnico o pieno di dati, da tenere sotto gli occhi; orale se è un consiglio veloce).

Il relatore usa due esempi per mostrare che *dipende*:

- un feedback sulla programmazione ha senso darlo scritto, magari commentando direttamente il codice, più che "andando a prendere un caffè";
- ma se il feedback è "ieri, durante il meeting, ho percepito male il fatto che mi interrompessi di continuo", ce lo si aspetta **orale**.

0.1.3.2 Da che cosa dipende il modo di dare feedback

Lo *spoiler* della giornata è che **non esiste una risposta univoca** su come dare feedback. Il workshop non insegna "il" modo, ma fornisce *tutte le opzioni*, perché il modo giusto cambia in base a molti fattori:

- a chi lo si dà: un collega, un superiore, il proprio capo, un proprio *report* (un sottoposto);
- se è personale o professionale;
- se è la prima volta o la terza volta che si dà lo stesso feedback;
- se riguarda un progetto importante o secondario.

Manca però l'elemento fondamentale, che la classe individua subito: **l'altro**. A parità di tutti gli altri fattori, il *carattere della persona che riceve il feedback* dovrebbe far cambiare il modo in cui lo si dà. Per questo, prima della lezione, agli studenti era stato chiesto di fare un test di personalità: non un test di psicologia avanzata, ma uno strumento per rendersi conto di quanta varietà ci sia anche solo in una classe di 53 persone (il che non vuol dire trovare 52 modi diversi di dare feedback, ma lavorarci un po' sì).

Il relatore porta se stesso come esempio. Preferisce:

- i feedback **positivi scritti**, perché è a disagio a ricevere complimenti in pubblico;
- i feedback sul proprio lavoro **diretti**: "vai subito al punto e non girarmi intorno".

È molto orgoglioso del suo lavoro e, quando qualcuno lo critica, la sua primissima reazione istintiva è "come ti permetti". Negli anni ha imparato a farla durare un secondo, ma chi gli dà feedback di persona dice di vederla ancora negli occhi. Per mitigarla, dice esplicitamente a chi gli dà feedback: *"ignora il modo in cui reagisco nei primi cinque secondi, vai avanti, poi mi passa"*. È considerato da tutti molto aperto al feedback e non ha problemi a dire "hai ragione, facciamo diversamente": ma ci sono reazioni istintive e caratteriali che è difficile controllare al cento per cento.

0.1.3.3 Con chi si lavora meglio

Guardando le persone con cui si lavora abitualmente (anche nei progetti dell'accademia), si noterà che si lavora meglio con due profili:

1. chi è **esattamente come noi**, ma qui è un 50 e 50: alcuni caratteri vanno d'accordo con i propri simili, altri non vogliono nessuno con lo stesso carattere nel gruppo (e si potrebbe rientrare in questa categoria);

2. chi è **affine**: ha qualcosa in comune con noi, ma non tutto, e proprio per questo può dare punti di vista differenti.

Il lavoro della giornata è capire come valorizzare il secondo caso: ottenere feedback da persone con caratteri, visioni e background diversi, per completare una visione del proprio lavoro che è necessariamente limitata. È come esplorare una mappa con la nebbia di guerra: un'altra persona, al di là del muro, vede cose che noi non vediamo.

0.1.3.4 Il feedback come somma di piccoli miglioramenti

Per il relatore il feedback è questo: **applicare tanti piccoli miglioramenti periodici**, che si sommano nel tempo. Migliorare dello 0,1% ogni giorno porta a fine anno a un miglioramento enorme, e questo vale per i prodotti, per i giochi, per le persone e per i team.

L'esempio del ciclismo britannico. La squadra di ciclismo britannica, fino a una quindicina d'anni fa, non aveva vinto quasi nulla. Cambiato il team manager, tutti si aspettavano grandi cambiamenti sportivi (convocazioni, strategie di gara). Arrivarono invece *minuscoli cambiamenti non sportivi*: un altro chef in trasferta, materassi migliori nei camper per riposare meglio, un sellino più comodo a fine tappa. Cambiamenti marginali che da soli non avrebbero cambiato niente; eppure, dopo due o tre anni di questo lavoro quotidiano, la squadra ha cominciato a vincere a ripetizione, Olimpiadi comprese, cambiando il modo in cui il ciclismo è visto in Gran Bretagna[8]. Certo, si è investito anche sulle persone giuste, ma tutto è partito dai piccoli miglioramenti periodici.

Per gli studenti vale lo stesso: chiarire un conflitto con una persona con cui si ha ancora poco a che fare, o una situazione che ha lasciato "l'amaro in bocca", sembra inutile oggi, ma darà i suoi frutti tra tre, sei, nove mesi. Sono *investimenti sul futuro*.

Peggiorare ogni giorno. Esiste anche il percorso inverso: lasciar correre, "buttare sotto il tappeto" le cose perché "tanto è inutile parlarne". Queste cose tornano sempre fuori dopo otto mesi, come il *libro dei rancori* dei nani di Warhammer: "ti ricordi quando, un anno e mezzo fa, all'inizio del progetto, mi hai detto quella cosa? Mi sono offeso". A quel punto è quasi impossibile risolvere, perché nessuno ricorda più con precisione cosa sia successo ("ma quale cena?", "forse era in pizzeria", "no, eravamo da un'altra parte"). **Tenersi le cose nel cassetto per tirarle fuori quando servono è sempre sconsigliato**: non servono ed è impossibile usarle in modo costruttivo.

0.1.4 Le cinque fasi di vita di un team

0.1.4.1 Perché conoscerle

Quasi tutti gli studenti lavorano già in gruppo (al primo anno su un gioco da tavolo, negli anni successivi su un videogioco) e si saranno accorti che un team attraversa delle fasi: il modo di lavorare insieme cambia nel tempo. Spesso ci si fa prendere di sorpresa: "eravamo così affiatati, cosa è successo?".

Un esempio da una consulenza fuori dai videogiochi: un'azienda farmaceutica gli affida un team commerciale (rappresentanti che presentano medicinali a ospedali, medici e privati). Il problema: "prima andava tutto bene; ora che siamo in diciotto dovremmo lavorare meno e meglio, e invece va peggio". Va peggio

[8] È la nota strategia delle *marginal gains* associata a Dave Brailsford (British Cycling e Team Sky); il nome non è citato in lezione.

perché le persone nuove sono appena arrivate, mentre il management si aspettava già di vedere i risultati dell'investimento.

Fino a pochi anni fa le fasi di un team venivano disegnate come una **linea dritta verso l'alto**: le fasi erano le stesse, ma viste come un miglioramento costante. Non si vedevano le parti *fisiologiche* del lavoro in team, cioè anche i peggioramenti. Accettarle è il primo passo per accettare i fallimenti: nessun percorso personale o di team è lineare.

Il relatore ammette che il suo modo di lavorare è nato per il 70% dal proprio impegno a imparare, fare bene e rispettare le persone, e per il 30% dall'aver lavorato con "incapaci colossali" che gli hanno insegnato come *non* fare le cose. A Blizzard un manager gli fece capire che, pur non sapendo ancora che tipo di *people manager* sarebbe diventato, sapeva benissimo quale tipo di manager *non* voleva essere.

0.1.4.2 Forming, storming, norming, performing, adjourning

Le fasi sono cinque, un po' come le fasi del lutto. Molte aziende ne considerano solo quattro ("e sono quelle che solitamente chiudono"); le aziende al passo con i tempi ne considerano cinque[9].

Forming Si viene messi insieme e si comincia a lavorare. Non si sa nulla degli altri; spesso non si conoscono nemmeno i nomi, ed è normale.

Storming Si cominciano a "dare le testate come le capre in montagna per determinare il territorio": questa cosa la faccio io, questa la vorrei fare così, chi decide? Si stabilisce chi fa cosa e come si divide il lavoro. Nel grafico è una **fase di peggioramento**: che il team litighi o discuta civilmente non cambia nulla, il team deve investire parte di tempo ed energie per risolvere queste questioni. È normale e **non è evitabile**.

Norming Si comincia a capire come lavorano gli altri, cosa sanno fare e come far funzionare il gruppo; iniziano i miglioramenti. È una forma di *accettazione* degli altri, che deriva dall'aver capito cosa devo fare io e cosa devono fare loro.

Performing C'è uno sforzo **proattivo**: "questa cosa toccherebbe a me, ma so che Alessio la fa molto meglio, quindi gli chiedo aiuto". Si sfruttano i punti di forza degli altri, perché li si conosce e perché si sa che passare il testimone non avrà effetti negativi (il capo non dirà "perché non l'hai fatta tu?").

Adjourning Il progetto finisce, si chiude o viene riorganizzato, il team viene sciolto. Quello che andrebbe fatto è analizzare **cosa ha funzionato, cosa no e cosa migliorare** per il progetto successivo.

0.1.4.3 La fase che tutti saltano

"Spoiler incredibile": l'*adjourning* è la fase che molte aziende saltano, **perché c'è sempre il progetto dopo**.

A PG Esports, per la Milano Games Week (come per qualsiasi evento dal vivo a cadenza annuale: Lucca Comics, Etna Comics, Comicon...), finita l'edizione di un anno si cominciava dal giorno dopo a lavorare a quella successiva, perché un anno sembra tanto ma non basta mai. Nei videogiochi è lo stesso: finito un gioco, a parte le patch di mantenimento e del *day one*, si passa subito al successivo.

Il cambiamento introdotto dal relatore (in parte già praticato, ma da lui strutturato) è stato: finita la Games Week, **tutti in ferie per una settimana**; poi ci si ritrova per definire cosa ha funzionato e cosa no, con un **post-mortem** (o *debrief*); solo dopo si inizia a lavorare all'edizione successiva. Senza questa fase manca la parte di *learning*, cioè di ciò che si è imparato dal progetto.

Il consiglio vale per gli studenti: quando si finisce il gioco da tavolo, e poi il gioco digitale, bisogna trovare il tempo per il post-mortem, altrimenti si butta via **il 90% dell'utilità del progetto**. Una struttura semplice per farlo è descritta nella sezione 0.1.17.

[9] Il modello è quello di Bruce Tuckman (1965), a cui la quinta fase, *adjourning*, fu aggiunta nel 1977. Il nome di Tuckman non è citato in lezione; la trascrizione riporta la quinta fase come "journing".

0.1.5 Domande sulle fasi del team

0.1.5.1 Lo storming che ritorna

Capita che lo storming si ritagli spazio anche nelle altre fasi? Lo *storming* è una fase precisa, che però molti manager, coordinatori e team cercano di **saltare**: "se io sono bravo come manager e loro sono bravi come team, passeremo da forming a norming senza problemi". È un errore, perché senza una fase in cui si discute davvero non si sviluppa una *skill* fondamentale: **saper discutere**. Tutti conoscono persone non abituate a farlo, per cui ogni minimo disaccordo diventa un affronto personale ("avevamo deciso la pizza, perché andiamo al sushi? Perché mi fate questo affronto?"): sono persone che non sono abituate ad avere conflitti costruttivi.

Se si pensa di essere in *norming* e continuano a emergere conflitti, il punto è che **non si era davvero in norming**: lo storming non "si trascina" nelle fasi successive, semplicemente non se n'è mai usciti. Allo stesso modo, a chi chiede se ogni mini-fase di storming durante la vita del gruppo sia frutto di uno storming fatto male o solo di carattere, la risposta è già contenuta qui.

Un partecipante osserva che spesso, ancor prima di finire un progetto, si è già messi su quello successivo: è il modo migliore per essere in costante *damage control*. Era l'esperienza del relatore a Blizzard: veniva spostato fisicamente da un team all'altro, e finito il lancio di *Diablo III* era già su *World of Warcraft*, senza vivere il "dopo". Si è dovuto ritagliare da solo lo spazio per un debrief personale, ed è lì che ha deciso che tipo di manager non voleva essere. Il consiglio: **se non lo fa il team, fatelo voi**.

0.1.5.2 Dove si passa la maggior parte del tempo

In quale fase si passa più tempo nella propria carriera? Gli studenti rispondono *storming* e *norming*. Il relatore, in quindici anni, calcola di aver passato in *performing* circa **dieci mesi**. Non è pessimismo: arrivare a performing deve essere sempre l'obiettivo, sia da manager sia da membro del team; ma è difficile arrivarci, ed è normale che non sempre ci si arrivi.

Lo storming può durare a lungo, ma con team molto esperti dura pochissimo, perché c'è chi lo sa gestire. In realtà **la maggior parte del tempo si passa in forming**, perché ogni equilibrio raggiunto vale per *quelle* persone, *quel* progetto, *quelle* tempistiche. Basta che cambi un qualsiasi fattore (una deadline spostata, una persona nuova, qualcuno che si dimette, un progetto cancellato) e si ricomincia, perché i bilanciamenti precedenti non valgono più.

Esempio: un team di cinque persone affiatatissimo, in pieno performing. La dirigenza, entusiasta, "raddoppia le risorse" e lo porta a dieci. O si lascia tutto com'è (e i cinque nuovi non fanno niente), o bisogna ridiscutere chi fa cosa e come i nuovi possono aiutare: che piaccia o no, si ricomincia da forming.

Quindi la bravura di un manager, ma anche di un membro del team, **non sta nel passare tanto tempo in performing, ma nell'accelerare il passaggio da una fase all'altra**, e nel fare bene la fase di *adjourning*.

0.1.5.3 Analizzare anche i successi

C'è la tendenza a fare post-mortem solo dei progetti andati male: "questa cosa non ha funzionato, vediamo perché". Va benissimo. Ma quando un progetto va bene si dà per scontato che sia andato bene "perché era buono". Bisogna invece chiedersi *perché* è andato bene: era il prodotto? La tempistica? Il modo in cui abbiamo fatto una cosa piuttosto che un'altra? **Se non si capisce davvero il motivo del proprio successo, si tenderanno a replicare le cose sbagliate.**

Blockbuster. Quando arrivò la notizia che Netflix avrebbe lanciato un servizio di streaming, Blockbuster (la catena di noleggio di videocassette e DVD) aveva tutto il tempo per preparare un'offerta concorrente. In una riunione del management raccontata in un libro (di cui il relatore non ricorda il titolo), la direzione sembrava orientata a investire nello streaming, finché un dirigente fece notare che il 12% del fatturato derivava dalle *multe per riconsegna tardiva*, impossibili con lo streaming, mentre secondo le stime lo streaming avrebbe prodotto il 10% del fatturato. "Non possiamo rinunciare a questo 2%": niente streaming. Oggi non c'è più

un Blockbuster "manco a cercarlo col lanternino". Non avevano capito qual era il vero motivo del loro successo, cioè il loro modello di business[10].

The Founder. Nel film sulla nascita di McDonald's c'è una scena in cui il futuro socio del protagonista gli dice: "tu pensi di essere nella *food industry*, e invece sei nella *real estate industry*". Non dovrebbe investire sugli hamburger, ma sui terreni su cui costruisce i ristoranti. Anche per un team vale lo stesso: bisogna capire che cosa l'ha reso davvero di successo, oppure no.

0.1.5.4 L'autoanalisi personale

E quando si cambia azienda? Uno studente chiede se il debrief sia possibile anche quando si lascia un'azienda. Non si può tornare in ufficio a discutere della propria esperienza, ma lo si può fare **per se stessi**. Il relatore l'ha fatto, anche se all'epoca in modo poco consapevole, ed è così che è passato dal *community management* al *people management*. Analizzando cosa aveva fatto bene a Blizzard si era reso conto che:

- aveva costruito un ottimo rapporto di fiducia con i giocatori (con alcuni proprietari di fansite parla ancora oggi);
- pur senza la seniority, era diventato un punto di riferimento per il team, visto come imparziale ed equilibrato anche su temi che non lo riguardavano direttamente;
- sapeva **risolvere problemi**: durante il lancio di *Diablo III* era il più tranquillo di tutti, non perché sminuisse il problema, ma perché sapeva che si sarebbe risolto con i suoi tempi; il lavoro non era spegnere l'incendio ma "far sfollare la gente" e aspettare che tutto tornasse in ordine.

E aveva capito cosa sapeva fare meno bene, ad esempio **gestire il proprio manager**. Avere un manager non significa che il manager gestisca te: il manager gestisce il tuo lavoro, ma tu devi saper gestire i tuoi *stakeholder*. Spesso lo stakeholder è il proprio manager; da manager, gli stakeholder saranno CEO, CFO e così via. Gestirli significa conoscerli, sapere come portare loro le informazioni, come far presente che qualcosa non va. Fa parte del lavoro, anche se a lui non piace, e ci ha lavorato.

Nel mese e mezzo di pausa prima di entrare in Riot si è concentrato su questo: "cosa porto come mio punto di forza?". L'autoanalisi è culminata nel chiedere espressamente alla persona che l'aveva assunto: "prima mi avete detto di no, ora mi avete preso: perché?".

Come si capisce se un progetto è andato bene? Dipende dall'**obiettivo**. Se l'obiettivo era vendere milioni di copie e il gioco le ha vendute, bisogna capire perché. Se, più ragionevolmente, l'obiettivo era un gioco da tavolo che permettesse di mettere in pratica le conoscenze del corso, va valutato per quello: ha permesso di farlo? Se sì, perché? Se no, perché no? L'obiettivo va fissato **all'inizio del progetto, non alla fine**, proprio per poterlo valutare dopo.

0.1.5.5 Quando il team cambia di continuo

Tutti questi cambiamenti non sono dannosi per il progetto? Assolutamente sì, ma quasi mai sono volontari. I progetti andati male sono spesso progetti grandi.

Il relatore racconta di *Cyberpunk 2077*. Conosceva persone in CD Projekt Red ("il mondo è piccolissimo, le cose si sanno"). Alla Gamescom va con Diego Ricchiuti, direttore dell'accademia, a vedere la presentazione del primo gameplay, quella "in cui regalavano il giubbottino da samurai". Finito il video, tra gli applausi, si guardano e capiscono entrambi che **il gioco non è pronto**, a soli quattro mesi dall'uscita prevista: Diego aveva notato il combattimento e la guida. Gli amici di CD Projekt Red (uno aveva lavorato con lui a Blizzard per sei anni) gli confermano tra le righe che "ce la faremo, ma ci sono ancora cose da sistemare". Il gioco fu poi rimandato più volte[11].

[10] Le percentuali sono quelle riferite in lezione e non sono state verificate.

[11] In lezione il relatore dice "due, forse tre volte"; l'uscita di *Cyberpunk 2077* è stata effettivamente rinviata tre volte prima del dicembre 2020.

Per deformazione professionale, chi ha sviluppato l'occhio critico non riesce più a godersi un trailer: in un trailer di *The Day Before* Diego gli aveva fatto notare che l'effetto della fiammata di un fucile era stato chiaramente aggiunto in postproduzione e non faceva parte del gameplay. "È il triste epilogo di questa storia": non riescono più a godersi un trailer.

Il punto è che più il team è grande, più queste variabili vengono imposte, e più è facile che il fallimento **non dipenda dal fatto che il prodotto sia brutto** (*Cyberpunk* è un bellissimo gioco sotto molti punti di vista, e il relatore lo sta rigiocando ora che è stato corretto ed espanso). Nei grandi gruppi qualcuno arriva, qualcuno si dimette, cambia il budget, arriva il rinvio; nel caso di *Anthem*, il team ha fatto così tante volte forming–storming–forming–storming che ne è uscito "un prodotto che dovevamo espellere: l'abbiamo espulso, buttato fuori dal nostro organismo pur di liberarcene".

Tra i cambiamenti c'è anche la crescita verticale: se il capo viene promosso perché lavora benissimo, bisogna scegliere un nuovo capo, ed è sempre un caos (chi ci resta male, chi pensava di meritarlo, chi voleva il suo amico, chi non sopporta il prescelto). Per questo il relatore è **contrario alle promozioni durante i progetti**: si rischia di rovinare tutto il lavoro di storming.

0.1.5.6 Obiettivi chiari nei piccoli gruppi

Nei gruppi piccoli, come quelli degli studenti, il problema deriva più spesso dal **non avere chiaro che cosa si vuole ottenere**: un prodotto puramente educativo e formativo, o un prodotto richiesto dall'accademia che, se viene bene, si potrebbe anche vendere? Un gioco commerciale o un gioco artistico? Va chiarito subito, perché ci saranno sempre momenti in cui scegliere tra A e B, e a guidare la scelta sarà la domanda: *quale delle due opzioni ci porta più vicino all'obiettivo?* Se l'obiettivo non è chiaro, non si sa cosa scegliere.

Questo porta molte aziende a fare **scelte di design che in realtà sono scelte commerciali**. Le microtransazioni sono scelte commerciali: nessuno ha mai pensato "sai cosa piacerebbe tanto ai giocatori? Pagare le bustine". Prima le bustine si guadagnavano giocando; ora la tendenza è "potresti giocare, ma se paghi è meglio". Si pensi alle polemiche su Ubisoft per l'opzione di "saltare il gioco" pagando, o ai famosi *boost* di livello di *World of Warcraft*: "devo pagarti per saltare il gioco?".

Una studentessa (Silvia) cita *Final Fantasy XIV*, che a un certo punto fu "resettato" perché non funzionava: un esempio positivo. Il relatore aggiunge quello negativo di *Age of Conan*, che non è mai riuscito a fare quel reset iniziale ("è una schifezza, ve lo ripresentiamo quando è pronto"). Ci vuole consapevolezza, ma anche i mezzi: un conto è Square Enix, che può permetterselo, un altro è un'azienda che non può chiudere tutto per un anno continuando a pagare gli stipendi.

Riassumendo: nei piccoli gruppi i problemi sono più di **comunicazione e chiarezza** (obiettivi non chiari o che si spostano); nelle grandi aziende di più di **persone che vanno e vengono, obiettivi e tempistiche che cambiano**. In accademia le scadenze sono fisse (il gioco da tavolo si consegna a fine anno scolastico, a giugno, e giugno resta); in azienda può succedere che una feature prevista per la *vertical slice* di luglio venga anticipata a maggio per portarla alla Gamescom, e un sistema perfetto per consegnare a giugno può non esserlo più per consegnare a maggio.

0.1.5.7 L'ambiente determina la qualità del prodotto

È vero che chi passa per Electronic Arts finisce per fare brutti giochi? Uno studente cita BioWare, che faceva ottimi giochi finché era autonoma. Il problema non è dei developer, che non sono "improvvisamente diventati incapaci": è evidente che il problema è altrove, e che **l'ambiente in cui si fa un gioco determina molto spesso la qualità del prodotto**.

Un esempio locale positivo: Tommaso Bonanni, conosciuto dal relatore insieme a Diego e già docente dell'accademia, ha fondato a Roma Caracal Games Studio, applicando fin dall'inizio questi concetti di cultura aziendale. L'azienda è rimasta piccola, ma ha molto chiaro cosa vuole fare e come. Quando si predispone fin dall'inizio l'azienda e il team a lavorare in un certo modo è fattibilissimo; è molto più difficile quando si è EA, si acquisisce un'azienda e le si dice "da oggi si fa così".

Il relatore ha vissuto la transizione da Blizzard ad Activision Blizzard (uno dei motivi per cui se n'è andato, volontariamente). Ha visto la cultura interna cambiare per gradi:

1. "facciamo giochi fighissimi, perché i giochi devono essere fighissimi";

2. "facciamo giochi fighi così vendono di più";

3. "vendiamo di più facendo giochi fighi" (sembra uguale, ma è molto diverso);

4. "facciamo più soldi".

Lo si capiva dalle scelte di design, osservando quale fosse il peso più grande sulla bilancia. Lo stesso varrà per gli studenti, nei loro giochi e nei colloqui di lavoro: capire subito che cosa vuole l'altra parte.

Leader e visione. Nei gruppi dell'accademia, dal secondo anno, c'è una persona che assume il ruolo di *lead* o *scrum master*. È diverso dall'essere un *game director* ("il gioco si fa come dico io, aiutatemi a farlo con la mia visione"), come avviene nei grandi studi: da lì nascono i nomi nei titoli, come *Sid Meier's Civilization*, che indicano la persona che ha fatto quel gioco, oppure un puro branding, come *Tom Clancy's Rainbow Six*.

Inclusion come jolly. A margine, uno studente osserva che in molte aziende "la risposta giusta è sempre *inclusion*", usata come un jolly. È comunissimo, conferma il relatore: *diversity and inclusion* viene spesso usata per dire "così includiamo più clienti", ma non è quello il suo punto ("altro discorso").

0.1.6 Confrontarsi con chi è più bravo

Arrivato a Riot dopo sei anni a Blizzard, il relatore pensava di avere un buon bagaglio di esperienza. Si è trovato invece "come un fante buttato in mezzo ai Navy SEAL": persone che avevano superato 11, 12, 13 colloqui, con un livello molto alto.

Individua subito un collega francese che, nella sua testa, diventa "la versione migliore di me": stesso ruolo (community manager), esperienza simile (arrivava da Ubisoft, come quasi tutti i francesi di Riot), ma molto più spigliato, sempre con la battuta pronta, "l'anima della festa". Il relatore invece è introverso: un workshop come questo per lui è *emotivamente drenante*, non ama parlare in pubblico pur amando condividere le proprie esperienze, e non riesce a sostenere conversazioni come "cosa hai fatto questo weekend?". Lavorando raramente insieme, aveva sviluppato una rivalità "da allenatore di Pokémon" (lui era Ash, il relatore l'antagonista).

Dopo sei mesi, a un evento di *team building* aziendale (a Tenerife), il suo capo, molto attento a queste dinamiche, li mette apposta nello stesso gruppo. A fine serata, complice l'open bar, il relatore gli confessa di averlo mal sopportato dal primo giorno per invidia della sua estroversione. Il collega ride e risponde che pochi giorni prima aveva detto al capo la stessa cosa al contrario: invidiava a Stefano il suo essere sempre organizzato, sul pezzo, presentarsi nella sala riunioni giusta (mentre lui finiva in quella sbagliata), arrivare con gli appunti, con i dati a supporto e le analisi di mercato, mentre lui spesso non aveva nemmeno letto la mail del meeting. Un'invidia in due direzioni, che li ha portati a lavorare molto insieme e a migliorarsi a vicenda.

Le lezioni:

- non bisogna demoralizzarsi vedendo qualcuno che fa lo stesso lavoro ed è più avanti sotto alcuni aspetti: spesso c'è un bilanciamento. I "punti caratteristica" sono limitati: se li ha messi in carisma, non li avrà messi in saggezza;

- conviene **chiedere a queste persone come mai** sono così brave in qualcosa, come si fa con un progetto andato bene: magari non lo si farà mai bene quanto loro, ma si comincerà ad avere il *mindset* giusto; e viceversa, loro saranno spinti a chiedere a noi;

- spesso **le proprie doti le scopriamo dagli altri**. Il relatore non si era mai accorto di essere "quello che resta calmo durante le emergenze" (il lancio di *Diablo III*, quello di *Draenor*, i problemi dal vivo alla Games Week) finché non gliel'hanno fatto notare tutti: per lui era la normalità. A chi pensa "non sono capace, forse non è il mio percorso", può darsi che semplicemente nessuno gli abbia ancora detto "questa cosa la sai fare davvero bene, concentrati su questo".

0.1.7 Domande sul lavoro nell'industria

Diverse domande degli studenti hanno riguardato le condizioni di lavoro nei grandi studi internazionali. Le risposte sono raccolte qui per argomento.

0.1.7.1 Presenza, remoto e trasferimenti

Grandi studi: quasi solo in presenza. Sia a Blizzard sia a Riot si lavorava **solo in presenza**, senza smart working: sei anni a Parigi e cinque a Dublino. Si parlava solo inglese: anche nei meeting tra soli italiani si tendeva a comunicare e a scrivere in inglese, per correttezza verso chi si fosse aggiunto e per lasciare una traccia comprensibile a tutti. Il francese serviva per fare la spesa e sopravvivere, non in ufficio. L'ambiente era estremamente eterogeneo (francesi, polacchi, spagnoli e molti altri): durante il colloquio dal vivo a Dublino, mentre si preoccupava di essersi vestito in modo adeguato, vide passare dietro la parete a vetro un collega svedese con i dreadlock, a piedi scalzi, con una pianta sotto un braccio e un aquilone sotto l'altro, e capì di essere "a posto così". Un'esposizione a molte culture e modi di pensare che il relatore considera bellissima.

Anche oggi, tranne alcuni studi particolari, la maggior parte del lavoro si svolge di persona: amici a Paradox lavorano a Stoccolma, i pochi rimasti a Blizzard si sono dovuti trasferire a Irvine. Per posizioni *junior* o *entry level* i grandi studi raramente concedono il lavoro da casa; per profili senior può esserci qualche eccezione (ad esempio nove mesi da remoto e tre in sede).

Relocation. Spesso tocca spostarsi, ma il relatore non ha mai pagato un trasloco di tasca propria: queste aziende offrono la *relocation* (aiuto a trovare casa, ad aprire il conto in banca e supporto pratico), perché è un investimento sulla persona.

Colloqui mentre si lavora altrove. Per i colloqui in presenza prendeva ferie, senza dire dove andava. Il consiglio generale: per fare colloqui **prendete ferie o permessi**, non datevi malati, "che non è un bel modo di comportarsi".

Remoto "doppio". Un collega lavora per l'azienda coreana che gestisce *PUBG*, con ufficio ad Amsterdam, ma lavora da Milano: un "doppio remoto". Si trova, ma in genere un ambiente anglofono con sedi in Europa chiederà di trasferirsi in Europa.

Famiglia e trasferimenti. Il relatore ha avuto la fortuna di sposare una persona che fa lo stesso lavoro (conosciuta lavorando su *Warhammer Online*, poi con lui a Blizzard e a Riot, nella localizzazione e nel doppiaggio, ad esempio di *World of Warcraft* in italiano): avendo competenze rare, spesso venivano assunti entrambi, al massimo a pochi mesi di distanza. Per i developer la soluzione frequente è che la famiglia segua la persona assunta, perché lo stipendio basta per tutti. Ma non sempre c'è equilibrio, e a volte uno dei due deve sacrificarsi: sua moglie ha lasciato Blizzard per seguirlo a Dublino (per poi essere assunta da Riot); lui ha lasciato Riot per tornare in Italia quando lei ha trovato lavoro qui. Ha dovuto rinunciare più volte a lavorare in Islanda per CCP (*EVE Online*), perché non era una scelta solo sua. Si fanno compromessi, ma si riesce a far quadrare.

Ci si sposta per ogni progetto? No: si sono spostati esclusivamente per cambiare azienda. Cambiando progetto nella stessa azienda è raro, quasi mai ("non l'ho mai visto succedere"), che venga chiesto di trasferirsi. In Francia stavano benissimo (vivevano fuori Parigi; "i problemi con i francesi sono in realtà problemi con i parigini"), parlavano francese ed erano vicini alla doppia cittadinanza: se ne sono andati perché non vedevano un futuro a Blizzard, per il modo in cui l'azienda trattava i colleghi.

Il mercato oggi. Non esiste un *silver bullet* per trovare lavoro domani dove si vuole. Con tutti i *layoff* recenti ci sono migliaia di esperti del settore che cercano lavoro: serve pazienza, ma è fattibile.

Studi indie e remoto. Negli studi più piccoli il remoto è più diffuso: a Stormind circa 60 persone su 95 lavorano da remoto, ma lo studio si è strutturato fin dall'inizio per farlo. Serve consapevolezza. Bending Spoons (che sviluppa app per mobile, non giochi, ed è l'azienda dell'app Immuni), pur essendo grandissima e con sede a Milano, lavora da remoto e lascia l'ufficio a chi vuole: lo smart working è posizionato quasi come un *benefit*, ma permette anche di avere un ufficio da 300 persone invece che da 700. Le sedi fisiche costano decine di milioni all'anno (Blizzard ha chiuso le sedi di Amsterdam e di Parigi). Tuttavia, per lavori molto verticali e specifici gli studi preferiscono avere le persone dal vivo; i ruoli più spesso remoti sono quelli che non fanno sviluppo (customer service, PR...).

Anche gli artisti lavorano spesso in presenza: l'accademia può dare solo una *simulazione* di quanto stretto sarà il contatto tra programmazione, design e art. "Sarà ancora di più."

La comunicazione da remoto. È più difficile. Pur essendo estremamente introverso, il relatore **obbliga tutti a usare la webcam** nei meeting in cui conta: non in un aggiornamento con novanta persone collegate, ma certamente in un colloquio a due sulla performance dell'ultimo anno. La componente fisica è una parte enorme della comunicazione[12]: anche se si è timidi, accendere la webcam è l'approssimazione più vicina all'essere dal vivo. Negli ambienti dove si lavorava senza webcam la situazione era "esponenzialmente peggiore".

0.1.7.2 Differenze culturali nel team

Gestire sensibilità diverse. A Riot i team erano divisi per *lingua*, non per nazione: il team di lingua spagnola includeva anche un messicano, quello di lingua russa includeva russi e ucraini, proprio nel periodo della guerra in Crimea. Tra i colleghi non c'era alcun problema (erano amici), ma ci si è posti la domanda: *come comunichiamo con una community che sarà divisa su questo tema?* Nei giochi online la divisione era evidente: in *Rocket League* c'era chi, con un compagno di squadra con la bandiera avversaria, si faceva otto autogol pur di farlo perdere. Allo stesso modo, con molti americani mandati a Dublino a coprire posizioni scoperte, c'erano le elezioni (Clinton–Trump, Biden–Trump) e varie guerre: serviva sempre una certa sensibilità.

Imparare a navigare queste situazioni lo ha reso un professionista migliore: gli ha insegnato a fermarsi e chiedersi "io la percepisco così; gli altri dieci, con background, storie e lingue diverse, come la penseranno?", a ragionare più *long term* e più *big picture*, con più empatia.

Stili di comunicazione. Un partecipante che ha lavorato nel Regno Unito chiede dello stile comunicativo italiano, a volte frainteso. Uno dei più grandi complimenti ricevuti dal relatore è stato "non sembri italiano" ("gesticoli poco per essere italiano", "parli bene inglese per essere italiano"). Ha imparato ad **adattare la comunicazione alla persona**: con spagnoli e francesi la battuta sulla nazione funziona, perché c'è un "odio–amore" condiviso; con russi e giapponesi no. Doveva adattare anche l'umorismo. Un collega metà russo e metà messicano diceva di aver preso "il lato peggiore di entrambe le nazionalità", ed è stato un ottimo maestro per imparare a leggere culture diverse.

Non bisogna generalizzare, ma gli stereotipi possono essere un punto di partenza ragionevole, *sapendo che troverete sempre le eccezioni*. E vanno letti correttamente: gli americani tendono a essere più estroversi e ad avere sempre qualcosa da dire non perché "sono fatti così", ma perché vengono educati così (all'università tra i corsi c'è il *debate*, saper argomentare in pubblico); un collega giapponese può essere più introverso per ragioni culturali. Nel dubbio si parte da lì, poi si conosce la persona.

0.1.7.3 Negoziare, discriminazione e stipendi

La famiglia come leva. Avere un partner riduce le opzioni, ma le aziende lo sanno: uno spostamento è per due. Si può usare come leva per negoziare (un compenso diverso, un supporto per il trasloco): se un'azienda ha deciso di volere quella persona, 2, 3, 4 o 10.000 euro in più all'anno, per un'azienda che fattura centinaia di milioni o miliardi, non cambiano nulla.

[12] In lezione si parla di "quasi l'ottanta per cento"; il dato va preso come indicativo, poiché le percentuali di questo tipo circolano in forme diverse e sono spesso citate fuori contesto.

Essere sposati è un motivo per non essere assunti? No. In alcuni paesi è proprio illegale discriminare su questo: in Francia era illegale chiedere CV con data di nascita, foto, sesso o religione, e chiedere se si vogliono figli. Purtroppo, dai racconti dei suoi studenti di master (in settori come *beauty and wellness* e *food and wine*), in piccole realtà italiane queste domande ("hai 25 anni, hai intenzione di sposarti?") vengono ancora fatte, e servono a filtrare i candidati. Nelle grandi aziende ci sono sistemi per evitarlo; anzi, molte danno *più* benefit a chi ha famiglia ("porta pure la famiglia, è tutto pagato"), perché le persone competenti hanno spesso una certa età e quindi, ragionevolmente, una famiglia.

Gli stipendi. Dipendono molto da dove si vive. Lo stipendio a Riot non era paragonabile a quello di Blizzard perché in Francia era tassato al 52% e in Irlanda al 17%[13]: nessuno stipendio competitivo può compensare il 35% di differenza di tasse. Negli Stati Uniti, con pochissima previdenza pubblica, gli stipendi sono molto più alti perché poi ci si paga tutto ("se hai un raffreddore paghi tu"). Bisogna valutare il potere d'acquisto di quei soldi nel paese in cui si andrà a vivere.

0.1.7.4 Freelance e *work for hire*

All'inizio della sessione pomeridiana uno studente che lavora nell'audio chiede quali siano le condizioni per chi lavora come *freelance*.

- Dipende molto dal progetto: spesso il contratto è a progetto, per portare a termine una parte o un intero progetto.
- Non tutte le aziende usano freelance: Riot e Blizzard lo facevano pochissimo e preferivano assumere (magari trattando poi la persona "come un freelance", lasciandola a casa a fine progetto).
- Spesso i lavori offerti come freelance sono in realtà **sostitutivi di un full time** ("freelance, ma sei ore al giorno").
- Si può fare, ma bisogna essere molto chiari nel definire l'impegno (quante ore, come vengono tracciate): non tutti specificano cosa significa davvero "freelance". Lavorando così per due o tre aziende si può arrivare a un tempo pieno.
- È più comune il *work for hire*: aziende che forniscono personale ad altre aziende; si è assunti da X ma si lavora "embeddati" presso Y (una sorta di subappalto). Anche lo studio del relatore lo fa.
- Il puro freelance di quattro mesi esiste, ma di solito per quei quattro mesi diventa un lavoro a tempo pieno; è raro "quattro mesi, due ore al giorno". Nelle grandi aziende è improbabile lavorare nello stesso mese per Ubisoft, Riot e Blizzard tre ore ciascuna; è più realistico lavorare per più startup indie italiane, che non possono permettersi un'assunzione ma possono pagare due ore al giorno.
- Molto passa dai contatti: trovare le aziende giuste e rendere esplicite le condizioni.

0.1.8 Il circolo virtuoso: feedback, miglioramento, fiducia

0.1.8.1 Il triangolo

L'obiettivo a cui il workshop vuole arrivare è un **triangolo che si autoalimenta**:

1. si dà un **feedback** con l'obiettivo di migliorare le performance (inclusi elementi caratteriali);
2. la persona **migliora** e vede di averne beneficiato;
3. questo genera **fiducia**, che rende più facile dare e ricevere il feedback successivo.

[13] Aliquote come riportate in lezione, riferite alla situazione personale del relatore.

Togliendo uno qualsiasi dei tre elementi, il circolo non è altrettanto forte.

In qualsiasi fase si parta, il rispetto per i colleghi è inizialmente dettato dall'*educazione*, non dalla fiducia. La fiducia arriva quando i colleghi cominciano a dire cose giuste e sensate e a **mantenere la parola**. Dare e ricevere feedback è il modo in cui, in un certo senso, "inganniamo il cervello" per creare una *case history* di fiducia: si ha motivo di fidarsi di più, di ascoltare di più, di essere più ricettivi. E si diventa sempre più bravi ad accettare il feedback e ad agire di conseguenza.

0.1.8.2 Accettare un feedback non vuol dire fare come dice l'altro

Dieci anni fa il relatore non era ostile ai feedback, ma molto *difensivo*: "sì, sì, va bene, ora ci guardo"; oppure, a chi gli diceva "avresti dovuto farla così", rispondeva spiegando perché l'aveva fatta in quel modo. Oggi ha imparato a dire "ok, grazie, ora ci ragiono": non deve difendere la propria posizione da chi viene solo a dire "hai pensato a questa cosa?". Prima lo viveva come un attacco ("mi viene a dire che non so fare il mio lavoro"), ora come un "ottimo punto di vista, lo prendo in considerazione".

Con una postilla fondamentale: **accettare il feedback non significa fare come dice l'altro**. "Mi hai detto che preferisci questo mob verde. Ho pensato anche a farlo verde? Sì. Penso comunque che giallo sia meglio? Sì, lo faccio giallo." Il relatore accetta *sempre* il feedback, valido o no: garantisce di tenerlo a mente quando lavora. Che poi faccia come gli viene detto è un altro discorso.

0.1.8.3 Fiducia e scadenze

La fiducia funziona allo stesso modo con gli impegni. Se si chiede a qualcuno una cosa per venerdì e la consegna venerdì, la volta dopo gliela si chiederà per venerdì. Se invece arriva il giovedì della settimana dopo, la volta dopo gli si darà una **deadline anticipata**, come con chi arriva sempre in ritardo agli appuntamenti ("se ti dico alle 7 arrivi alle 7 e mezza; la prossima volta ti dico alle 6 e mezza").

Al lavoro questo è dannosissimo, perché crea **deadline fittizie** che non sono mai quelle vere, e nascono discussioni del tipo "perché devo farlo in due giorni se si fa in cinque?" ("sì, se tu lo facessi in cinque"). La conversazione giusta è un'altra: *"che problemi hai avuto nel consegnare venerdì?" "Questo, questo e questo." "Cosa posso fare per evitare che succeda?"*

Il relatore non vuole mai sentirsi dire "non sono riuscito a farlo". Preferisce di gran lunga sentirsi dire, tre giorni prima, **"potrei non riuscire a farlo, per questi motivi"**. Chi ha la visione d'insieme può dare più budget, più persone, spostare la deadline, cambiare lo scopo del progetto: può fare qualsiasi cosa per aiutare. Ma se lo si scopre il venerdì, cinque minuti prima della deadline, non può fare più niente. Il messaggio è: **"aiutami a poterti aiutare"**. Non penserà mai male di chi dice "non ce la faccio per questi motivi", perché non sta dicendo "non ho voglia": sta indicando i problemi, e lavorare su quei problemi è compito del manager. Non può invece fidarsi di chi fino al giorno prima dice "tranquillo, ce la faccio" e il giorno dopo "non ce l'ho fatta".

Questo è fondamentale per chi lavora in programmazione, design o art, perché ci saranno sempre deadline: bisogna sempre **lavorare sulle aspettative**, dire al proprio manager (o al proprio report) cosa è lecito aspettarsi. Non bisogna fare come i pinguini di *Madagascar* ("sorridete e salutate, va tutto bene"): se c'è un problema, va detto, altrimenti si peggiora solo la propria situazione.

Da italiani si tende a non voler mai alzare la mano per chiedere aiuto ("no, no, ce la facciamo, lo faccio io"). A volte non ce la si può fare, non perché non si è capaci, ma perché vari fattori lo impediscono. Accettarlo è importante: **è molto meglio avvisare e chiedere una mano che dire "non ce l'ho fatta"**.

0.1.8.4 Domande sul dare e ricevere feedback

"Ne parliamo a fare, se poi fai come vuoi?" Uno studente (Cristian) riceve feedback che, dopo averli analizzati, non ritiene congrui; quando non li segue gli viene detto "ne parliamo a fare?". C'è un problema di **aspettative**: l'idea che accettare un feedback voglia dire fare come dice l'altro. Accettarlo significa prendere in considerazione ciò che è stato detto. Chi decide diversamente ha però un **onere**: argomentare perché si fa comunque a modo proprio ("mi hai detto verde, la facciamo gialla perché…"). Poi bisogna solo aspettare i risultati per vedere chi aveva ragione.

La decisione spetta sempre **a chi ha la responsabilità di quella decisione**. Se il game director deve scegliere tra A e B, anche se il feedback dice B, la scelta è sua perché la responsabilità è sua. Non sarebbe corretto essere ritenuti responsabili di una scelta delegata a chi dà feedback. La risposta può essere semplicemente: "l'ho tenuto in considerazione, ma visto che ne sono responsabile io e le conseguenze sono mie, ho scelto di fare X".

E se l'altro fa ripicca? Un partecipante chiede di chi, vedendo il proprio feedback non accolto, si vendica ignorando a sua volta feedback buoni. Per il relatore è un problema caratteriale: chi si comporta così "è già sulla strada per uscire". Non c'è spazio per l'ego in team di questo tipo. Il fenomeno nasce dall'idea che *il feedback negativo faccia fare brutta figura*, che il fatto che si potesse fare meglio tolga qualcosa alle proprie capacità.

Non è così: **il feedback non è un giudizio**. Si può passare un anno a Riot ricevendo solo feedback negativi e avere una valutazione annuale stellare, perché ricevere molti feedback negativi significa essere una persona approcciabile, che agisce sul feedback e invita gli altri a darlo. Il relatore ha visto valutazioni molto peggiori per chi riceveva solo feedback positivi, perché la gente aveva paura di dargliene di negativi: ma solo "pacche sulle spalle" non fanno migliorare. Il problema va affrontato parlando con la persona; a volte le conversazioni iniziano con **"ti do un feedback sul feedback"**, cioè sul modo in cui l'altro ha dato il suo.

Esempi dalla sua esperienza:

- lui chiede spesso di **non ricevere feedback senza esempi tangibili**, perché fa fatica ad astrarre: "sei stato maleducato" non significa nulla, "ieri al meeting mi hai risposto male su questa cosa" sì;
- ad altri serve l'opposto: ha ricevuto il feedback "quando devi dirmi che ho sbagliato, puoi girarci un po' intorno e darmi del contesto?", perché il suo stile diretto ("questa cosa non ha funzionato, dovremmo fare così") feriva. Ha adattato il suo modo di dare feedback: con alcuni è diretto, con altri gira intorno. L'importante è saperlo.

0.1.8.5 Non portare la soluzione

Un consiglio centrale: **quando si dà un feedback, non si dà una soluzione**. Non si dice "io farei così", ma "secondo me questa cosa si potrebbe fare diversamente, per questo e questo motivo: come potremmo farla diversamente?". È come chi smette di fumare perché vuole e chi smette perché glielo dicono moglie e figli: se non si è convinti, spesso non si smette.

Quando il relatore diceva "c'è questo problema, io farei così", il meeting successivo iniziava inevitabilmente con "ci ho provato come hai detto tu, con me non ha funzionato". Sempre. Perché la soluzione era sua. La responsabilità della soluzione deve essere invece **congiunta**: ogni feedback deve creare un piccolo team che ha come obiettivo risolvere quel problema. Questo toglie l'antagonismo (non "ti dico che non sai fare il tuo lavoro", ma "ti voglio aiutare a farlo meglio, lavoriamoci insieme") e sposta il focus: il problema non sei tu, è ciò che sta accadendo.

E se sono i piani alti a tenere all'oscuro dei problemi? È una questione di cultura aziendale. Il relatore fa sempre presente al proprio capo il concetto di **service level agreement** (che verrà ripreso nella seconda giornata): "se mi dai questi input, ti garantisco questo output in questi tempi". Se il capo chiede delle cose senza dire budget, tempi e perché, non può aspettarsi lo stesso risultato. È una comunicazione a due vie: si dice al manager cosa serve per fare il lavoro, e lui deve fornirlo o accettare che il risultato non sarà quello voluto.

0.1.8.6 Amici, parenti e parole proibite

"Non si lavora con amici e parenti"? Per sua definizione personale il relatore ha pochissimi amici ("stanno sulle dita di una mano") e una soglia molto alta per considerare amico un collega. Allo stesso tempo, in quindici anni e con centinaia di colleghi, ha avuto solo **due** esperienze in cui non è riuscito a lavorare con qualcuno: non serve essere amici per far funzionare le cose. Con gli amici, anzi, gli viene più facile: se dice a un amico che una cosa "fa schifo", l'amico non pensa "come si permette", ma "se perfino Stefano, che è mio amico, me lo dice, allora è proprio così". Diventa un rafforzativo, grazie alla fiducia

costruita prima. Il problema del lavorare con amici e parenti nasce dal *non voler* dire all'amico che sta sbagliando; se invece la confidenza permette di dirlo, si può saltare la "confezione" del messaggio e passare direttamente al contenuto.

Le frasi peggiori. Alla domanda se esistano parole peggiori di "ottimo lavoro", il relatore indica la sua classifica: "ottimo lavoro" (detto in modo vuoto)[14]; poi la frase che lo "fa uscire di testa", **"abbiamo sempre fatto così"** ("e infatti avete chiuso", come Blockbuster); infine **"siamo una famiglia"**, riferito al posto di lavoro. Un'azienda non è mai una famiglia, perché non ci si può aspettare lo stesso impegno da chi la possiede e da chi ci lavora: non c'è un corrispettivo proporzionale tra impegno e ritorno. Per quanto inclusiva, flessibile e razionale, non sarà mai una famiglia.

0.1.8.7 Ego, muri e il peggior modo di scegliere un leader

Chi mette davanti l'ego e alza muri. Esistono, ma di solito durano pochissimo in azienda. Non si tratta di avere un ego forte, ma di **non saper ricevere feedback**. Spesso sono persone bravissime nel loro lavoro che poi si chiedono perché tutti intorno diventino più bravi di loro: perché gli altri accettano feedback e hanno più fonti da cui attingere, mentre loro si sono rintanati nella loro torre.

Molti manager tendono ad **assumere persone un filo peggiori di loro**, per non fare brutta figura. Il relatore assume solo persone più brave di lui, "perché non è una gara": ha lavorato con molti community manager più bravi di lui come community manager, ma lui era più bravo a fare il manager. Non viene valutato su quanto sa fare il community manager, ma su quanto il suo team sa farlo. Un partecipante chiede se la paura di avere qualcuno di migliore esista anche in ambito accademico. Assolutamente sì: al relatore è capitato di non essere preso come docente in un corso di formazione sui videogiochi perché il responsabile aveva percepito che ne sapesse più di lui[15]. "Ma perché te ne importa? Tu sei quello che ha portato chi sa fare il suo lavoro: ben venga." O si ha questa cultura, o si ha paura per il proprio posto.

Promuovere il migliore. Quando bisogna scegliere un leader per un team (programmazione, design, art), molto spesso si sceglie **il migliore tecnico**: il miglior programmatore, il miglior designer. È spesso la scelta sbagliata: è come prendere un idraulico per fare l'elettricista. Fare il *people manager* non è come fare il designer o il programmatore; si può essere un ottimo programmatore e un pessimo manager, e viceversa. Le aziende l'hanno capito e lo fanno meno di prima. A Blizzard c'era un grave problema di *middle management*: molte persone in ruoli importanti erano state promosse cercando le skill sbagliate, erano frustrate dal non saper fare il proprio lavoro e schiacciavano chi stava sotto per paura di fare brutta figura.

Un partecipante cita una teoria simile: una persona viene promossa finché non raggiunge *il proprio livello di incompetenza*, dove si blocca[16]. Un altro cita Michael Scott di *The Office*, su cui il relatore tornerà (sezione 0.1.15).

0.1.9 Perché la cultura conta: retention, costi e fiducia

0.1.9.1 L'ambiente di lavoro e la retention

Perché tutto questo è così importante? Perché l'ambiente di lavoro conta moltissimo per la **retention**, cioè per trattenere le persone e farle lavorare in modo efficiente. Se contasse solo lo stipendio, molti non lascerebbero mai Riot o Blizzard; e più si va avanti con l'età, più si impara a dare valore anche al resto.

Inoltre, secondo gli studi citati nelle slide, chi vive il proprio lavoro in modo negativo è **meno efficiente nel problem solving**: l'energia mentale va verso "come faccio a non farmi dare la colpa" o "come faccio a fare bella figura", non verso la soluzione del problema. In ambienti negativi, dove ci si deve preoccupare della propria "sopravvivenza", si lavora peggio.

[14] Il passaggio della trascrizione è frammentario: si intende verosimilmente il complimento generico e di circostanza, che non dice nulla di utile.

[15] La trascrizione dice "che non sapevo più di lui", ma il senso del passaggio richiede il contrario.

[16] È il *principio di Peter* (Laurence J. Peter, 1969), citato in lezione come "la teoria di Peter".

0.1.9.2 Il costo di chi se ne va

Ogni volta che qualcuno si dimette o viene licenziato, il costo di trovare una nuova persona, fare i colloqui, formarla e farle attraversare forming, storming, norming... è circa **un terzo del suo stipendio**. Per questo le aziende tendono a fare più *retention* che *acquisition*, almeno con le figure considerate utili alla produzione.

Allo stesso tempo si vedono sempre più spesso *layoff* e *hiring wave*: si licenziano 3.000 persone e sei mesi dopo se ne riassumono 2.500 o 3.500, ma tutte più junior. Il relatore ne ha scritto in un articolo su Eurogamer.it, in cui spiega anche alcuni "trucchi del mestiere": perché i licenziamenti avvengono sempre in certi periodi e sempre "a gruppi" (prima Bethesda, poi Blizzard, poi un'altra...).

L'offerta di Riot. A Riot esisteva una pratica (di cui il relatore non ricorda il nome) per cui, firmato il contratto, si aveva **un anno di tempo per dire "non mi piace, non me la sento, me ne vado"** e ricevere subito come buonuscita un anno di stipendio. Avevano stimato che fosse molto meglio pagare subito un anno a qualcuno e lasciarlo andare che tenerlo due, tre, quattro anni come *underperformer*: la responsabilità della scelta di assumere era dell'azienda[17]. L'hanno usata pochissimi, ma il solo fatto che esistesse faceva lavorare meglio anche lui da manager, perché sapeva che chi non si trovava bene poteva andarsene con un anno di tempo per trovare un altro lavoro.

Il ricircolo. C'è un forte ricircolo nell'industria: molti ex colleghi passano da Ubisoft a Paradox, da Paradox a Funcom, da Funcom a Blizzard. Il relatore ha scelto di uscirne lavorando solo per aziende **indipendenti**: Stormind non è un indie di cinque persone, ma è indipendente, e non deve accontentare azionisti. Quando vede un'azienda con utili miliardari licenziare migliaia di persone si chiede "dov'è la linea" (*when enough is enough*): con azionisti che vogliono un ritorno, se quello è l'obiettivo stanno facendo la cosa giusta, ma lui non condivide l'obiettivo[18].

L'open space vuoto. A Riot, durante un'ondata di licenziamenti, del suo team di trenta persone è rimasto solo lui: "teniamo te come coordinatore, poi ricostruirai il team". Ha rifiutato di ricostruire un team cancellato "con un colpo di spugna" e ha deciso che non avrebbe più lavorato per una grande azienda. Nessuno aveva pensato di spostarlo, e ha lavorato per tre mesi in un open space di cento metri quadri con la sua scrivania e altre ventinove vuote. Su una lavagna aveva scritto i nomi dei suoi ragazzi e li cancellava man mano che, anche grazie ai suoi *referral*, trovavano lavoro. Cancellato l'ultimo nome, si è dimesso: per lui il lavoro era finito.

Team che si dissolvono. Uno studente cita un articolo su Larian Studios, il cui management è contrario a ricostruire ogni volta il team a fine progetto. È lo stesso problema: si pensa ai "giochi Blizzard" o ai "giochi BioWare", ma le persone che li hanno fatti non ci sono più; è rimasto solo il brand, e non ha senso avere le stesse aspettative da un gioco fatto da persone diverse. E anche chi sopravvive ai layoff vive pensando che il prossimo potrebbe essere lui: alcune persone, licenziate nell'ultima ondata a Blizzard, gli hanno detto "erano due anni che lo sapevo". Vivevano a Cork, "un paesotto irlandese dove o vai per lavorare a Blizzard o non ci vai", avevano comprato casa: non ci si può aspettare chissà che morale.

0.1.9.3 La cultura è forte quando si lavora per gli altri

> *La cultura è forte quando le persone lavorano per gli altri e a favore degli altri, e non per se stesse.*

Se si dà feedback per far migliorare l'altro e non per imporre il proprio modo, se si aiuta per il bene del progetto e non per fare bella figura, la cultura si rafforza. È dai **piccoli gesti** che emerge la cultura aziendale: ad esempio, dal non dover chiedere "perché mi stai aiutando?".

[17] La pratica, nota come "The Offer", è descritta in fonti pubbliche con condizioni diverse (un periodo più breve e una somma pari a una frazione dello stipendio annuo). Si riportano le condizioni come raccontate in lezione; andrebbero verificate.

[18] In lezione si parla di "6,5 miliardi di utile" e di "3.000 persone licenziate"; le cifre sono riportate come dette e non sono state verificate.

L'incognita. L'incognita è spesso dannosa quanto le cattive intenzioni. In un esperimento ("mi pare a Stoccolma, cinque o sei anni fa") si distribuivano per strada banconote da 500 euro, e la maggior parte delle persone le rifiutava chiedendosi "dov'è la fregatura?" [19]. Lo stesso accadeva al relatore, arrivato a PG e a Stormind da Riot: a "vedo che sei in difficoltà, posso aiutarti?" la risposta era sempre "perché?". Alla fine rispondeva: "perché quando sarò in difficoltà io vorrei che tu mi aiutassi".

Quando si comincia a lavorare per gli altri e si smette di pensare "così avrò la promozione, così avrò l'aumento", promozioni, aumenti e rispetto arrivano da soli. A Blizzard aveva fallito tre volte i colloqui per essere promosso a manager (inteso come coordinatore del team); quando ha deciso di smettere di pensarci e di fare solo il bene del proprio lavoro, la volta dopo è stato promosso. Funziona così: chi aiuta i colleghi e fa andare bene il progetto viene notato ("questa persona fa migliorare cinque persone, non solo se stessa: vale di più").

0.1.9.4 Il lavoro è valutato come opera d'insieme

Il relatore riconosce che buona parte del suo successo è venuto dal fatto che il suo team lavorava bene, non dalla sua bravura personale. Anche i progetti degli studenti, presentati a Diego, a Enrico, ad Andrea, verranno valutati **come opera d'insieme**. Nessuno dice "il *combat system* di questo gioco l'ha fatto X, che bravo": si dice "BioWare ha fatto un bel gioco", "Blizzard ha fatto un bel gioco".

Lo ha replicato anche lato HR. In molte aziende ci sono **bonus personali** legati a obiettivi (se il gioco vende X, se la feature è pronta in tempo). Il relatore li ha dimezzati: metà restano personali, metà diventano **bonus di team**, che tutti ricevono se il team raggiunge gli obiettivi. Un incentivo concreto a collaborare: non solo si dice che il lavoro di gruppo è importante, lo si dimostra dandogli più valore economico. Aveva proposto il 100% di bonus di team, ma gli è stato concesso il 50 e 50; ha funzionato comunque, anche se sarebbe stato ancora più potente. Ovviamente, alla domanda "perché mi aiuti?" si può anche rispondere "perché voglio il bonus": un aiuto dato "per il motivo sbagliato" era un compromesso che era disposto ad accettare.

0.1.9.5 Il coraggio di chiedere aiuto

Un partecipante osserva che aiutarsi dovrebbe essere la norma. Il modello diffuso, invece, è "devi essere forte e indipendente, non chiedere mai una mano", e vale in tutte le direzioni (dal maschilismo alla "donna forte e indipendente che non ha bisogno di nessuno"): **non ammettere di avere bisogno di aiuto è sempre uno dei motivi per cui le aziende vanno male**, che si tratti di uomini, donne o persone non binarie. Il problema è non avere la sicurezza di poter alzare la mano.

Il relatore ha cercato di togliere ogni conseguenza dall'aver alzato la mano: se si *depenalizza* qualcosa, le persone non sono più disincentivate a farla. Ha detto espressamente che non avrebbe mai valutato nessuno per il numero di volte in cui chiedeva aiuto o per il numero di feedback negativi ricevuti, e questo ha funzionato.

0.1.9.6 Perché si dà un feedback

Perché avete dato il vostro ultimo feedback? Che cosa volevate ottenere? Le risposte della classe:

- per **riconoscere il valore** di un collega, anche verso il manager, perché qualcuno che non aveva visibilità vedesse quanto quella persona fosse efficace;
- per **spingere una persona a superarsi**;
- per aiutare un collega a rendere il prodotto all'altezza di ciò che voleva rappresentare;
- per **genuinità**: c'era qualcosa che poteva essere fatto meglio.

Il relatore aggiunge che la forma cambia col tempo: all'inizio era "devo darti un feedback, hai dieci minuti?"; ora è un messaggio informale su Discord, perché ha costruito un rapporto di fiducia e sa che non verrà letto

[19] L'esperimento è riportato in modo vago in lezione e non è stato possibile identificarlo con certezza.

come "l'HR manager mi scrive, sono nei guai?", ma come "Stefano è così, mi dà feedback, resterà tra noi e lo risolveremo insieme".

Un altro motivo è **risolvere incompatibilità caratteriali**: le conversazioni più difficili, del tipo "io e te non stiamo funzionando, cosa possiamo fare diversamente?". Sono importanti perché, se due responsabili non vanno d'accordo, il team se ne accorge, comincia a raccontarsi storie ("si odiano, ci saranno problemi") e, per difendere il proprio manager, comincia a giudicare male il lavoro dell'altro gruppo. È la dinamica da fermare: **valutare gli altri solo per difendere la propria fazione**.

Dare e ricevere feedback sono **due muscoli da allenare** (il relatore è sempre stato più bravo a darlo che a riceverlo), e si impara col tempo. È come fare colloqui: fare colloqui è una skill a parte, che non indica quanto si è bravi nel proprio lavoro. Essere scartati non significa non saper fare quel lavoro: spesso significa aver fatto male il colloquio o che è stato preferito qualcun altro.

Feedback pubblici e privati. Uno studente racconta di aver dato un feedback negativo a un superiore, senza un motivo particolare, per dimostrare a sé e ai colleghi che non si viene licenziati per questo. Il relatore coglie l'occasione per precisare che **i feedback negativi si danno individualmente**: lui dà moltissimi feedback di cui il resto dell'azienda non sa nulla. Reagirebbe molto male se, in un meeting, qualcuno alzasse la mano per dire "comunque questa cosa non l'hai fatta bene": quello non è feedback, è discussione, accettabile solo se è un brainstorming e se l'opinione è stata chiesta. Alcune aziende hanno piattaforme pubbliche di feedback, ma spesso servono solo ad applaudire; lo stesso accade nelle aziende che *obbligano* a lasciare un certo numero di feedback a settimana, che diventano "lui è bravo, lei è competente, anche questa settimana l'abbiamo fatto". Bisognerebbe chiedere a chi le adotta cosa pensa di ottenere (forse trasparenza), ma difficilmente una persona "attaccata" pubblicamente reagisce bene.

0.1.10 Among Us: la caccia al traditore

Prima della pausa pranzo il relatore mostra tre film di fantascienza di qualche decennio fa e chiede che cosa li accomuni[20]. In ognuno c'è un gruppo di persone con un obiettivo e la presenza, reale o percepita, di qualcuno che rema contro: **l'energia del gruppo viene spesa per trovare il responsabile**.

L'esperimento degli alieni. Nei master in presenza il relatore propone un esercizio. Distribuisce segretamente cartellini plastificati con scritto "essere umano" o "alieno"; le carte alieno contengono istruzioni di comportamento con cui gli alieni, che cercano di fingersi umani, si tradirebbero (dare sempre ragione a chi ha parlato prima, finire ogni frase con una domanda, parlare in un certo modo). Lo scopo: in dieci minuti formare gruppi, e **vince il gruppo più numeroso senza alieni**; basta un alieno per perdere. Si formano sempre gruppi piccoli (due, cinque, sei persone), perché la tendenza è "mi fido al massimo di voi due".

Ma vincono i gruppi più grandi, per un motivo semplice: il relatore **non distribuisce nessuna carta alieno**. Mostra a tutti entrambe le carte, ma consegna solo carte "umano". Vuole simulare ciò che accade in ufficio: la tendenza a percepire sempre qualcuno che rema contro (i manager, l'altro team, quel collega), quando in realtà non c'è nessuno, "è tutto nella nostra testa". Quando chiede le motivazioni, emergono sospetti *fabbricati*: "faceva sempre domande", "parlava solo all'infinito" ("no, parla così e basta"). Ci si fabbrica da soli i motivi per non fidarsi degli altri, ed è comunissimo.

Among Us. L'autosabotaggio è, per tornare ad *Among Us*, **concentrarsi sul cercare cadaveri e traditori invece di fare le task**: non si vince perché non si sono completate le task, mentre si cercava il colpevole. Succede tantissimo negli uffici, ed è il motivo per cui l'80% dei problemi che il relatore risolveva erano di comunicazione. Entrando nei team sentiva immancabilmente: "facciamo il possibile, ma questa lotta continua con l'altro gruppo è estenuante". "Perché c'è questo problema?" "Non lo so, c'era già quando sono arrivato." Un paradosso: c'è un problema, nessuno sa dire quale sia, ma c'è sempre stato e quindi lo si accetta.

[20] I titoli non sono leggibili nella trascrizione: viene citato *Event Horizon* da uno studente, e il film "in mezzo" è indicato come "l'originale", verosimilmente *La cosa* (*The Thing*).

L'esperimento delle scimmie. È l'esempio, "finto o reale", delle scimmie in gabbia[21]. In una gabbia con alcune scimmie pende una banana elettrificata: chi la tocca fa prendere la scossa a tutte. Le scimmie imparano e, quando una si avvicina alla banana, le altre la picchiano. Poi si sostituisce una scimmia alla volta con una nuova: la nuova va verso la banana, le altre la picchiano, e smette. Si continua finché nella gabbia ci sono solo scimmie nuove, poi si toglie la corrente. Nessuna scimmia tocca la banana: nessuna ha mai preso la scossa, ma tutte hanno imparato che chi tocca la banana prende le botte.

Nei team è lo stesso: il motivo per cui si "odia" l'altro team si trasforma, ma un vero motivo non c'è. Ne derivano la sfiducia, il non alzare mai la mano per chiedere aiuto, il non dire mai "bel lavoro" agli altri perché non sia mai che qualcuno ti veda fare un complimento "a quelli". Risolvere queste dinamiche è lungo e tedioso, e sempre pieno di "non mi ricordo", "io non c'ero" e "abbiamo sempre fatto così", la cosa più frustrante per chi cerca di risolverle.

0.1.11 Il principio del default to trust

0.1.11.1 La definizione

Il relatore indica questa come la slide più importante della presentazione. **Default to trust** è uno dei valori aziendali di Riot Games, che lui ha tradotto, adattato e "esportato":

> *Approccio ogni situazione di conflitto partendo dal presupposto che una persona abbia agito in un certo modo perché, con le informazioni a sua disposizione in quel momento, ha pensato che fosse la soluzione migliore, e non perché sia un'incapace o volesse danneggiare il progetto intenzionalmente.*

In altre parole: **di default, mi fido**.

0.1.11.2 L'esempio del ritardo al daily stand-up

A Riot il relatore aveva un *daily stand-up* dal vivo alle 15 con il suo team di venti persone. Una ragazza arrivava regolarmente alle 15:15, per una, due, tre settimane. Alla quarta ci ha parlato. *Come iniziereste la conversazione?* Le proposte della classe:

- sottolineare l'importanza di esserci dall'inizio, perché potrebbe perdersi qualcosa che le interessa, e per rispetto dei colleghi presenti;
- "devi arrivare in orario", spiegando perché è importante;
- **chiedere perché**.

Il relatore osserva, anche dalle sue consulenze fuori dai videogiochi, che **più si sale con la seniority**, più la risposta è "devi smettere di arrivare in ritardo", spesso con un *or else* finale ("altrimenti succede quest'altra cosa"). Più si scende con l'età, e quindi con meno abitudini consolidate, più la prima domanda diventa "dimmi perché arrivi in ritardo". Già spiegare perché è importante è "miglia avanti" rispetto al semplice "smetti".

Partire dal perché gli ha permesso di scoprire la causa: il meeting precedente della ragazza, in un'altra zona del campus a un quarto d'ora di distanza, finiva sempre tardi, e lei doveva attraversare il campus di corsa. Non aveva la confidenza per dire all'altro manager, che insisteva perché si restasse fino alla fine, che doveva andare; sacrificava quindi il meeting del relatore, sapendo che era il più comprensivo dei due.

Il relatore si è offerto di **mediare**: ha detto all'altro manager che o anticipava l'inizio del suo meeting, così da finire comunque in tempo, oppure lasciava uscire la ragazza, perché quel meeting gli serviva. C'è anche una questione di **economia di scala**, spesso sottovalutata: se venti persone aspettano qualcuno per dieci minuti, si sono sprecati dieci minuti per venti. Per lo stesso motivo il workshop è iniziato alle 10:05: iniziare alle 10:10 avrebbe significato far aspettare cinquanta persone cinque minuti in più. A Riot c'era una

[21] È il noto racconto delle "cinque scimmie", spesso presentato come esperimento scientifico ma probabilmente apocrifo (lo stesso relatore lo definisce "finto barra reale"); nella trascrizione il numero delle scimmie varia.

scrivania vuota con un pupazzo gonfiabile, **Mr. Late**: sommando tutti i minuti di ritardo a meeting, call e conversazioni, in un anno veniva fuori praticamente *una persona a tempo pieno*. Serviva a ricordare che anche cinque minuti contano, e che bisogna rispettare gli altri arrivando in orario o almeno avvisando.

Gli effetti. La conversazione era avvenuta nel primissimo periodo a Riot, quando il team non aveva ancora motivo di fidarsi di lui. Risolvendo il problema ha eliminato anche tutto lo strato di micro-dinamiche spiacevoli che un manager impara a vedere (due colleghi che si guardano con la risatina quando lei entra in ritardo). La ragazza ha cominciato a entrare tranquilla, con una certa leggerezza: non le faceva piacere arrivare in ritardo ("ci sono anche persone che lo fanno apposta, e quello è un problema da affrontare diversamente"). Soprattutto, con i colleghi non parlava di lui come del capo che minacciava, ma come di chi le aveva risolto un problema: era diventato un **alleato**. La settimana dopo altre tre persone sono andate da lui con un problema da risolvere. Ripetuto cinque, sei, sette, dodici volte, in un paio di mesi poteva dire quasi qualsiasi cosa nei meeting sapendo che richieste e critiche sarebbero state percepite come "me lo chiede perché vuole darmi una mano", non per fare bella figura o per presentare i successi del team ai suoi superiori.

0.1.11.3 Condividere i successi, schermare i fallimenti

Un altro elemento fondamentale (ripreso nella seconda giornata): il manager **condivide i successi e protegge il team dai fallimenti**. "Se va male qualcosa è colpa mia; se va bene qualcosa, voglio che ne parliate voi." Il lavoro del manager non dovrebbe essere valutato su ciò che presenta del lavoro del team: tutti sanno comunque chi è il manager. Tanto vale lasciar presentare il team, che potrà dire con orgoglio di aver presentato a Mike Morhaime[22] la propria campagna europea per *StarCraft II*. Ci sono manager che dicono "come mi prendo le colpe, mi prendo i meriti", ma i meriti li hanno già: è inutile ostentarli. Con i fallimenti invece il manager è la **rete di sicurezza** del team. Il relatore ne ha fatto una questione di principio: **non scaricare mai la colpa su qualcun altro**.

Colpevole o causa. Un esempio che il relatore attribuisce a Pirate Software, creatore di contenuti ed ex dipendente Blizzard[23]. Contattando il servizio clienti Blizzard per motivi personali, aveva scoperto che i colleghi gli avevano scritto nei ticket informazioni confidenziali che non avrebbero dovuto condividere in quel modo, nemmeno con un collega. Non ha segnalato le persone da punire: ha detto "c'è un problema, evidentemente le policy non sono chiare". Quando gli hanno chiesto i nomi, ha rifiutato: "se vi dico chi è, punite quelli, e gli altri cinquecento continueranno a farlo; non avremo cambiato la causa alla fonte".

> *È inutile trovare il colpevole. Voglio trovare la causa. Perché il colpevole cambia.*

Il colpevole a volte è vittima della situazione: per questo la definizione dice *"con le informazioni a sua disposizione in quel momento"*. Se il relatore avesse rimproverato la ragazza in ritardo, non solo non avrebbe risolto il problema, ma avrebbe creato un'**ingiustizia**: lei, che attraversava il campus di corsa per una colpa non sua e di cui non poteva parlare, sarebbe stata anche rimproverata.

Nessuno fa un brutto lavoro apposta: nessuno è contento quando sbaglia o perde. Perché al lavoro diamo per scontato che gli altri sbaglino intenzionalmente? Se qualcuno ha fatto una cosa male, è perché in quel momento credeva fosse la scelta migliore. La domanda giusta non è "perché hai fatto questa cosa che è stata un fallimento?", ma:

> *Che cosa ti ha fatto pensare che questa fosse la scelta giusta?*

Magari erano limiti di budget, di tempo, di scopo: si agisce su quelli, non sulla persona. Se si cambia la persona ma le si danno gli stessi fattori, è molto probabile che arrivi alle stesse conclusioni, e ci si troverà a dover "disciplinare" anche lei.

[22] Cofondatore ed ex presidente di Blizzard; nella trascrizione "Mike Morayme".

[23] Il relatore aggiunge che si tratterebbe del figlio dello sviluppatore a cui si sarebbe ispirato il personaggio del giocatore "senza vita" nell'episodio di *South Park* dedicato a *World of Warcraft*. L'informazione è riportata come detta in lezione e non è verificata.

La reputazione costruita sui rumor. Durante il giro di presentazione in un ufficio (non in nessuna delle aziende citate), al relatore è stato detto: "questo è il dipartimento di design; se ti serve qualcosa chiedi a lei, non a lui, perché lui consegna sempre in ritardo". Oltre a trovarlo poco professionale, una settimana dopo ha chiesto: "cosa ti ha fatto questa persona?" "A me niente, però ho sentito che...". Si torna alle scimmie: magari quella persona non consegna in ritardo da mesi, ma ormai lo dicono tutti. È un circolo vizioso da fermare subito.

Niente escalation inutili. Lo stesso vale per il feedback: quando qualcuno va dal relatore dicendo "ho un problema con questa persona", lui chiede: **"perché ne stai parlando a me e non a lei?"**. "Perché sei il mio capo" non basta. Passare da lui significa dare per scontato che l'altra persona non abbia interesse a risolvere, mentre magari basta parlarle. È la strada più facile ("lo dico a Stefano, ci pensa Stefano"), e proprio per questo bisogna mettere dei paletti: il manager interviene quando serve, ma le cose che si possono gestire da soli vanno gestite da soli. (Quando invece il confronto diretto fallisce, l'escalation è legittima: si veda la sezione 0.1.17.)

0.1.12 Traiettorie di carriera e valutazione delle performance

0.1.12.1 Quale traiettoria vorreste?

Il relatore mostra un grafico con quattro curve colorate, contrassegnate anche da lettere (A, B, C, D) per eventuali persone daltoniche ("a me è capitato che qualcuno scoprisse di esserlo durante una lezione"). Le curve rappresentano la performance lavorativa nel tempo. Due domande: *quale vorreste essere? E quale pensate realisticamente di essere?* Un partecipante aspira a B e pensa di essere C; quasi tutti scelgono B.

Perché non A, che sale più rapidamente? Un partecipante risponde che una crescita così repentina può essere più dannosa che utile, come una "rush" di studio molto faticosa che poi non si riesce a sostenere. Il relatore propone una lettura suggerita da un amico psicologo, che spiega la forma del grafico: di B, C e D **vediamo dove finiscono**; di A no. Potrebbe continuare a salire oltre la scala del grafico, o crollare sotto D. Non saperlo ci spinge a preferire B. (A chi obietta che anche le altre proseguono oltre il bordo, il relatore precisa: il bordo destro è "oggi", e di A non sappiamo come sia arrivata fin lì.)

0.1.12.2 La carriera non è una linea retta

Tutti immaginano la carriera come una linea: si parte da X e, attraverso stage e posizioni, si arriva a Y. In realtà carriera e performance avranno un andamento irregolare, con alti e bassi. Come non si accetta che un team possa temporaneamente *underperformare*, tanto meno lo si accetta da se stessi. Bisogna accettare che ci saranno momenti bassi.

Per questo il relatore **detesta le valutazioni annuali delle performance**: prendono un punto casuale della traiettoria, che se si è fortunati è il picco più alto e se si è sfortunati il più basso, e lo usano per valutare un intero anno. Insegna ai manager a fare più punti di controllo e a guardare altre cose:

1. **quanto tempo passa tra un picco e l'altro**, cioè quanto *cooldown* serve a una persona. È il motivo per cui, dopo la Games Week, tutti staccavano una o due settimane. Il relatore stesso, che si impegna moltissimo e tende al burnout, non potrebbe rifare lo stesso workshop il lunedì e il martedì successivi con la stessa qualità;

2. **la differenza tra la miglior performance e la peggiore**: che cosa ha portato a lavorare così bene, e così male? Con chi si lavorava, su cosa ci si concentrava, quanto tempo e budget c'erano? L'obiettivo è capire come *replicare* le condizioni che fanno lavorare bene, invece del solito "hai lavorato male, è colpa tua". Il relatore ha avuto molti manager per cui quando lavorava male era colpa sua e quando lavorava bene "che bravo", ma sapeva che raramente era l'unico fattore della propria performance. È la fallacia del "se sai fare 100, perché non fai sempre 100?", come nella scena di *Schindler's List* sull'operaio che produce una vite in pochi secondi: non sempre un ritmo è sostenibile;

3. **il fallimento di oggi confrontato con quello di sei mesi fa**. Il relatore non è contento della presentazione di oggi (l'avrebbe migliorata per altri due anni), ma rispetto a quelle di cinque anni fa

vede un grande miglioramento di chiarezza: il suo peggior fallimento di oggi equivale al suo primo successo di inizio carriera.

Quello che conta è la **traiettoria** dall'inizio dell'anno a oggi, non i singoli punti. E se si riesce a capire insieme quando e perché si ha successo e quando e perché si fallisce, si può lavorare per replicare solo le situazioni che fanno lavorare bene.

Lo stesso vale per i giochi. È il motivo principale per cui i giochi delle grandi case vanno bene o male. Quando si riescono a replicare le condizioni che hanno prodotto *Warcraft II*, esce *Warcraft III*; quando non ci si riesce, esce "il nuovo *Dragon Age*": la gente non è arrabbiata perché il gioco non è bello, ma perché non è come gli altri *Dragon Age*; non ne replica il *feeling*, anche se potrebbe essere un gioco favoloso. L'eccezione positiva è il passaggio da *Dune* a *Dune II*: il primo era di fatto un'avventura grafica, il secondo ha generato il genere degli strategici in tempo reale. Gli sviluppatori avevano capito che il successo non stava nel tipo di gioco, ma nell'ambientazione che attirava le persone: un bel gioco in quell'ambientazione sarebbe piaciuto comunque. Il relatore comprò *Dune II* aspettandosi un seguito dell'avventura e, pur sorpreso, lo finì.

0.1.12.3 Lying, hiding, faking

Quando si è valutati su un singolo punto nel tempo, e da quel punto dipendono bonus, stipendio e il rischio di licenziamento, si comincia a fare **lying, hiding e faking**:

Lying dire che è sempre andato tutto bene, che si finirà sempre in orario, che non c'è nessun problema;

Hiding nascondere: "non era un mio task", "non c'ero", "non ho visto niente";

Faking fingere, anche in cose apparentemente innocue come "sì, sì, ho capito tutto" quando si ha solo paura di dire "me lo rispieghi? Non ho capito niente".

0.1.12.4 "Non ho capito" e la conferma delle istruzioni

Dire "non ho capito, ripetimelo" è ancora visto come una figura da sciocco. Il relatore non si arrabbierà mai se qualcuno gli chiede di rispiegare un'intera slide: le cose ovvie per lui possono non esserlo per altri. Più si conosce qualcosa, **più è difficile spiegarla**, perché si danno per scontate molte cose. Ne parlava proprio con Diego, che gli diceva che fare l'insegnante diventa sempre più difficile man mano che si diventa esperti. È come spiegare *EVE Online*, uno dei giochi più complessi in circolazione: chi lo conosce bene dà per scontato che "ovviamente non puoi fare questa cosa", mentre per un nuovo giocatore non è affatto ovvio. A scuola chi dice "non ho capito" viene visto come lo "scemo"; nessuno si chiede "non è che l'ho spiegato male?".

Il racconto più "traumatico" dei ragazzi che fanno stage nelle grandi aziende è sempre lo stesso: "Cinzia, devi fare questa cosa." Cinzia la fa. "Non te l'avevo chiesta così." Il problema è che nessuno ha **confermato che Cinzia avesse capito**. Il consiglio, valido sempre:

- quando si *ricevono* istruzioni, ripeterle: "ho capito bene? Vuoi che faccia questa cosa in questo modo?";
- quando si *danno* istruzioni, chiedere a chi le riceve di ripeterle, non perché lo si ritenga incapace, ma per capire che cosa ha capito dalla propria spiegazione ("quindi vuoi un quadrato?" "No, ho detto un triangolo..." "Però mi hai detto una forma con quattro angoli." "Hai ragione: voglio un quadrato").

Frasi come "non te lo ripeto due volte" o "stammi dietro" portano solo a fare le cose come si pensa che vadano fatte e a sentirsi dire a fine progetto "non la volevo così": che si tratti di una feature, di un design o di un modello 3D. (Un partecipante osserva che lo stesso accade insegnando le lingue: chi insegna dà per scontato, ad esempio, che si sappia dell'esistenza del genere neutro in tedesco.)

Come convincere il team a chiedere? Il relatore mette sempre come premessa che **l'obiettivo non è trovare la persona più intelligente nella stanza, ma fare un buon lavoro**. Che Gianni capisca alla prima e un altro alla quinta non gli interessa, se alla fine entrambi lavorano bene: ripetere cinque volte è comunque meno lavoro che far rifare le cose sette volte. È un investimento a lungo termine, e non dissimula

alcun fastidio: davvero non gli dà fastidio. Ma ripetere le stesse parole non aiuta: "non ho sentito" si risolve ripetendo; "non ho capito" si risolve cambiando spiegazione. Meglio ancora: "dimmi che cosa hai capito tu, e vediamo cosa non torna". Se intravedesse nel team uno sguardo del tipo "è la quinta volta che lo chiede", interverrebbe: spesso chi giudica è proprio chi dice "ho capito" e poi sbaglia. Preferisce di gran lunga chi chiede cinque volte.

0.1.12.5 Il feedback più utile che abbiate ricevuto

Qual è il feedback più utile che abbiate mai ricevuto? Il relatore ne ha almeno dieci che gli hanno cambiato il modo di lavorare. Le risposte:

- un semplice **"grazie"** per un lavoro fatto, più utile di un "questa cosa fa schifo";
- un feedback sulla propria **impulsività** nel rispondere: fermarsi un attimo e capire di cosa si sta parlando prima di intervenire;
- **essere se stessi**, anche al lavoro, e non scusarsi continuamente per ogni cosa.

I feedback che toccano di più sono quelli **veri**. Il "grazie" significa "ho visto quello che hai fatto e l'ho apprezzato". Quelli negativi che si ricordano sono quelli che hanno fatto realizzare "questa cosa che pensavo di saper fare, non la so fare", oppure "la so fare, ma questa volta non mi è venuta" (non è che se una cosa una volta non riesce non la si sa più fare: può capitare di sbagliare una porta). Arrivano sempre da un **punto cieco**: qualcosa che non si sapeva fosse percepito dagli altri. Conta più ciò che le persone ti dicono di ciò che pensi che pensino di te; e ciò che il relatore pensa che gli altri pensino di lui è sempre terribile, mentre il loro giudizio si rivela molto più positivo del previsto.

Il denominatore comune siete voi. Si potrebbe pensare che questi concetti non siano applicabili da chi arriva in un'azienda senza poterne cambiare la cultura. Ma c'è sempre un denominatore comune a cui applicarli: *se stessi*. La cultura aziendale può non spingere a dare feedback, può vedere male chi chiede aiuto; ma il relatore ha visto che quando lui lo fa, le persone intorno tendono a farlo, e un gruppo che lo fa lavora meglio. Perfino i CEO più ostici a un certo punto gli chiedevano: "perché il tuo team ora performa meglio degli altri?". La differenza è il default to trust: una pratica che col tempo fa operare il team *come un team* e non come un insieme di individui. Un videogioco è un progetto corale: c'è chi fa giochi da solo, ma non si fa da soli *Red Dead Redemption 2*; e se il gruppo non è affiatato e allineato non esce *Red Dead Redemption 2*, esce *Anthem* ("oggi la mia fissazione è *Anthem*, domani sarà un'altra").

0.1.13 Come dare un feedback

0.1.13.1 Concentrarsi sull'azione e sui suoi effetti

Il primo principio: **il feedback non riguarda mai la persona**. Non si dice "tu hai fatto questa cosa che non va bene", ma "questa cosa non va bene: come facciamo a non farla più?". Che sia un comportamento, un'azione, una decisione o una strategia, ci si concentra su quello, e si **esplicita l'effetto** che ha. Nell'esempio del ritardo l'effetto era "quindici minuti di attesa moltiplicati per venti persone". In un progetto può essere una catena:

> se non rispetti le deadline, l'animazione che viene dopo resta ferma; se loro sono in ritardo, la *vertical slice* è in ritardo; se la vertical slice non è pronta per la Gamescom, non andiamo alla Gamescom e non troviamo un publisher; se non troviamo un publisher, chiudiamo.

Più la conseguenza è grave, più dovrebbe sembrare piccolo il gesto di dire "datemi una mano": è il *male minore*. Poi, a cose fatte, ci si dà feedback: cosa ha funzionato, cosa no, cosa fare di più, di meno, cosa smettere di fare.

0.1.13.2 Il pericolo degli ambienti solo positivi

Gli ambienti dove ci si fa sempre grandi complimenti e nessuno parla dei problemi creano difficoltà vere alle aziende. Il problema non è mai la mancanza di complimenti, ma un **eccesso di ottimismo**: si comincia a pensare che le cose andranno come si spera, perché tutti intorno danno solo feedback positivi.

A Blizzard i feedback determinavano gli **aumenti annuali**: ognuno doveva indicare cinque persone che avrebbero dato feedback su di lui, e ovviamente tutti indicavano gli amici, o colleghi di progetti andati bene. Un sistema nato con le migliori intenzioni diventava uno *scambio di voti* ("se mi metti follow ti metto follow"), senza alcun miglioramento.

Al contrario, **le conversazioni più brutte sono state le più utili**: affrontare i licenziamenti a Riot, o la cancellazione di *Project Titan* a Blizzard, il progetto che doveva essere il nuovo MMO dopo *World of Warcraft* e da cui poi è nato *Overwatch*[24]. Andando a vedere perché lo si cancellava, c'erano solo motivazioni razionali e pragmatiche, di business, chiare a tutti, non un "Mike Morhaime ci odia". Sono anche le conversazioni in cui ha lavorato su se stesso: è molto **avverso ai conflitti**, e questo a volte gli ha procurato conseguenze molto più gravi di quanto sarebbe costato un piccolo disagio. Dieci anni fa faceva fatica ad accettare questo difetto; ha dovuto subire le conseguenze di conflitti evitati e poi cresciuti.

0.1.13.3 Che cosa fare

Le linee guida della slide, che il relatore legge rapidamente perché saranno disponibili nel materiale:

Attenzione al linguaggio del corpo dare un feedback a braccia incrociate o guardando altrove è diverso dal guardare l'altro negli occhi.

Sii specifico con esempi tangibili.

Esplicita gli effetti come visto sopra.

Discuti il prima possibile non aspettare un mese per parlare di qualcosa successo ieri, altrimenti nessuno ricorderà più bene cosa sia accaduto.

Concentrati sull'azione e sul problema, non sulla persona il nemico è l'azione, non la persona che si ha davanti. Non bisogna far cambiare l'altro: bisogna evitare che il problema si ripresenti.

Gli insulti sono rivolti al ruolo. Un esempio solo apparentemente scollegato. Il relatore ha lavorato su *League of Legends*, "non esattamente noto per avere una community da orsetti del cuore", in varie *task force* sulla tossicità. A chi smetteva di giocare veniva inviato un sondaggio sul perché; il 33% indicava la tossicità della community. Dopo varie campagne di sensibilizzazione e cambiamenti si è scesi al 30%: sembra solo un 3% in meno, ma valeva circa **120 milioni di revenue in più all'anno**, perché quei giocatori restavano esposti a skin e contenuti e continuavano a comprare. Anche qui, piccoli guadagni che si sommano.

Lavorando con degli psicologi, aveva notato che gli insulti **non sono quasi mai rivolti alla persona, ma al ruolo**: "il mio jungler fa schifo", "il mio DPS non è capace", "il nostro portiere è una pippa". Chiamare qualcuno per ruolo lo *depersonalizza* e rende più facile insultarlo. Lo stesso nell'esempio del giro dell'ufficio: non "non chiedere a Giulio", ma "non chiedere a quel grafico, consegna sempre in ritardo". Concentrarsi sull'azione, senza attaccare la persona, fa sì che il feedback non sia (e non venga percepito come) un attacco personale.

Considera il carattere. Qui tornano utili i test di personalità. Servono più agli studenti che al relatore (che sa già che usciranno 53 profili diversi, senza un pattern): bisogna conoscere il proprio profilo e **condividerlo con chi lavora con noi**, chiedendo anche agli altri di fare lo stesso. In ufficio il relatore ha usato una versione più avanzata, a pagamento, con un report di circa trenta pagine: il **profilo DISC**[25]. Ha mappato tutta l'azienda e ha fatto stampare il profilo di ciascuno sulla sua scrivania, così che tutti sapessero con chi stavano parlando. Capire come ragionano gli altri ("è introverso, non gliene parlo davanti a tutti") è una delle cose che gli sono tornate più utili: *adattare ciò che si vuole dire a chi si ha davanti.*

[24] Nella trascrizione si legge anche "doveva essere Warcraft 2", probabilmente un lapsus o un errore di trascrizione.
[25] Nella trascrizione "disk". Si tratta del modello DISC (Dominance, Influence, Steadiness, Conscientiousness).

Assicurati di non disturbare. Sotto il profilo di ognuno c'era un piccolo **semaforo**: verde, "parlami pure, interrompimi"; giallo, "interrompimi solo se è urgente"; rosso, "mi sto concentrando, non mi parlare".

Lascia finire il lavoro. Durante gli eventi dal vivo il relatore si era reso conto di quanto fosse grave dare feedback mentre qualcuno sta ancora lavorando. Se una cosa sta andando male, dirlo in quel momento non aiuta: tra la pressione di finire e quella del feedback, peggiorano entrambe le cose, e anche il rapporto. Meglio lasciarla finire, anche male, e parlarne dopo.

0.1.13.4 Che cosa non fare

Non basarti su supposizioni "mi hanno detto che ieri in un meeting hai fatto...": o c'eri e l'hai visto, o non esiste.

Non giudicare la persona.

Non fornire feedback vago.

Non accettare e non fornire feedback di terze parti Se qualcuno vi dicesse "mi hanno detto che hai fatto un brutto lavoro", il primo pensiero non sarebbe "cosa ho fatto male?", ma *"chi te l'ha detto? Chi è la vipera che ho in seno? Chi è l'impostore?"*. Siamo fatti così. Ogni conversazione che inizia con "mi hanno detto che" è inutile: l'altro non ascolterà, penserà solo a chi dare la caccia, e spesso penserà alla persona sbagliata, creando antipatie tra persone che non c'entravano nulla. Quando al relatore dicono "mi hanno detto che lui ha fatto...", risponde "vai a parlarne con lui"; se gli dicono "mi hanno detto che tu hai fatto...", risponde "mandami la persona che te l'ha detto, altrimenti non ne parliamo". Non per cattiveria: se chiedesse "che cosa ho detto di preciso a Marco?" e la risposta fosse "non lo so, non c'ero", di cosa si starebbe parlando? **Parlate con la fonte**, altrimenti fate come se quel feedback non esistesse.

Non dare consigli non richiesti come visto, non portare la soluzione: la soluzione deve venire da chi riceve il feedback.

"Non mi piace" non è un feedback "Penso che questo design non funzioni perché, visto che il nostro obiettivo è..." ha argomenti; "non mi piace il combat system" è un'opinione. Anche "io l'avrei fatto diversamente" non è un feedback.

Accettare non vuol dire fare come proponi deve essere chiaro a chi dà il feedback: l'aspettativa è essere ascoltati e presi in considerazione, ma se la responsabilità è dell'altro, deciderà l'altro.

Non offenderti se il feedback non viene seguito. Restarci male significa pensare di saperne più dell'altro, cioè *venire meno al default to trust*. "Decidi tu" significa "mi fido di te, perché sei lo specialista, e mi fido che mi ascolterai e farai la cosa migliore per tutti", che potrebbe anche non essere ciò che ho detto io. Se voglio che l'altro faccia sempre come dico io, mi sto fidando solo di me stesso.

0.1.13.5 Il primo feedback e il debriefing

La cosa più difficile è dare il **primo** feedback: è un muscolo da allenare. Chi non l'ha mai fatto (soprattutto con questa struttura) dovrebbe provarci, anche su minuzie ("abbiamo concordato di usare WhatsApp nel gruppo: non potremmo usare Telegram?"), anche solo per vedere come va. Altre indicazioni:

- feedback **solo uno a uno**, mai davanti ad altri, soprattutto se negativo;
- **prepararsi** prima;
- fare il **debriefing del feedback**: dopo, fermarsi a pensare a com'è andato. Il relatore, quando media un feedback tra due persone, si sente spesso rispondere "benissimo" da una e "malissimo" dall'altra: la percezione è molto diversa. Non si fa problemi a tornare dalla stessa persona: "ieri ti ho dato un feedback: com'è andata? Sei soddisfatto? Come poteva andare meglio?". Con le persone con cui si ha meno a che fare, questo aiuta molto la volta successiva.

0.1.14 Come ricevere un feedback

Per il relatore è la parte più difficile. Si riassume in quattro passi:

1. **Ascolta**: si parte sempre ascoltando.
2. **Chiedi dettagli**, se serve, ma per capire meglio, non per difenderti.
3. **Accetta, a prescindere**: non si sta decidendo chi ha ragione; si sta ascoltando un'opinione. Non c'è nulla da cui difendersi.
4. **Agisci**: fatti un piano. Può essere banale ("mi hanno detto che interrompo sempre: cerco di non interrompere"), oppure "devo migliorare questa cosa: come?". Pensa anche a *chi* potrebbe aiutarti e a *che supporto* ti serve: il piano non deve essere per forza "devo risolvere tutto da solo".

Una domanda che molti sottovalutano: *"sono junior in un'azienda: sarò mica il primo ad avere questo problema?"*. Probabilmente qualcun altro l'ha già avuto: basta chiedere "tu come hai fatto?".

0.1.14.1 Il feedback positivo

Tutto questo vale per i feedback costruttivi negativi, a cui si pensa di default; ma esistono anche i **feedback positivi**, e vanno usati. Quando si riceve un "grazie" o un "bravo", la domanda è: *come faccio a ricreare le stesse condizioni?*

Il relatore ha lavorato anche in aziende dove il feedback era solo negativo. Il rischio? Si immagini l'impiegato ideale: vuole sempre migliorare, è sempre pronto a dire "magari sbaglio io". Se nessuno gli dice cosa sta facendo *bene*, spinto dal desiderio di migliorare rischia di cambiare cose che erano già buone, e di farle peggio. Il rinforzo positivo **non è solo una gentilezza**: è un atto pratico, che evita che le persone peggiorino ciò che sanno già fare bene.

0.1.14.2 Il feedback accidentale

Esiste infine il **feedback accidentale**, molto frequente negli uffici:

- il "chiedi a lei, non a quel grafico" del giro dell'ufficio è un feedback negativo su una persona, dato accidentalmente, né richiesto né voluto;
- a Riot li chiamavano **drive-by**, come nelle scene dei film in cui passa un'auto, si abbassa il finestrino, si spara e si riparte. In molte aziende con una "cultura del feedback" in cui tutti devono dire la loro, si riceve una valanga di commenti del tipo "comunque il sito a me non piace, ciao". Non si sa perché, non si sa cosa fare di diverso, non si sa che rischi ci siano: *a che cosa è servita questa conversazione?* A niente.

0.1.15 Quando, quanto e con chi

0.1.15.1 Frequentemente e recentemente

Quando avete ricevuto il vostro ultimo feedback? Per gli studenti, qualche giorno fa (nel corso) o qualche mese fa (al lavoro, dopo una consegna). Come aveva osservato Silvia, il feedback andrebbe dato spesso.

Il relatore usa l'esempio di una nave da crociera ("la Costa Concordia") che deve andare da A a B. La rotta ideale (azzurra) è quasi impossibile: nessun progetto va esattamente come previsto. C'è la rotta "panoramica" (verde), che gira intorno all'isola d'Elba; ci sono poi rotte (arancione e rosa) che a un certo punto si allontanano molto e poi correggono bruscamente: qualcuno si è accorto che si stava sbagliando, "ma non è quello che avevo chiesto", "non è dove dovevamo andare". Sono i famosi **sanity check** o *checkpoint*.

L'ideale è avere **tanti piccoli check per fare tante micro-correzioni**: correggere la rotta di un paio di gradi è molto più rapido che farlo dopo essersi allontanati. Bastano trenta secondi di "ehi, vado bene?". Spesso non lo si fa per paura di sembrare insicuri o poco autonomi, ma per il relatore è l'opposto: quando qualcuno gli chiede "vado bene così?" e lui risponde "sì", si alleggerisce anche del proprio carico mentale.

Un capo che assegna un compito e lo vede solo alla fine vive sempre col dubbio ("avranno capito? Lo staranno facendo giusto? Arriverà in tempo?"); con un check di dieci secondi al giorno ("tutto ok, Stefano") è tranquillo, e la persona va avanti sapendo che quanto fatto finora va bene. Non serve fare ogni giorno il giro del dipartimento: anche ogni uno o due giorni basta a evitare la rotta verde.

Per il feedback è lo stesso: **va dato subito**. Dire oggi "ieri hai fatto questa cosa" è molto diverso da dire "negli ultimi sei mesi hai fatto questa cosa". Il relatore si è sentito dire più volte, alla valutazione di dicembre, "da maggio hai fatto questa cosa che non andava". La sua risposta: "e aspetti dicembre per dirmelo? Mi hai lasciato sbagliare per sei mesi?". È una mancanza di rispetto, e la conversazione non può più essere costruttiva. Per questo non ama le valutazioni annuali: i manager crescono abituandosi a prendere appunti delle cose negative per tirarle fuori alla valutazione ("a marzo hai fatto questa cosa" "sì, però poi ho smesso" "sì, però a marzo l'hai fatta").

In sintesi:

Frequentemente riguarda l'intervallo tra un check e l'altro.

Recentemente significa non lasciar passare un anno prima di dare il feedback, altrimenti non serve più a niente.

0.1.15.2 Quanto dura un feedback

Un feedback dura **da trenta secondi a quindici minuti**; raramente di più. Chi non ne ha mai dato uno dovrebbe sforzarsi di darne almeno uno, magari positivo per cominciare. Ogni feedback dato o ricevuto è un **deposito nel proprio "fondo fiduciario"** (*trust fund*), che costruisce la reputazione futura. Il relatore si è costruito la reputazione di chi dà feedback il più possibile oggettivi e analitici: giusti o sbagliati che siano, costringono a pensare. Non impone mai il suo punto di vista ("si fa come dico io"), ma chiede "tu come faresti? Come posso aiutarti?". Il feedback, per lui, serve a *far crescere qualcun altro*.

0.1.15.3 I manager peggiori: "a che numero sto pensando?"

I peggiori manager che il relatore abbia avuto erano quelli che volevano che si facesse come volevano loro, anche quando chiedevano "tu come la faresti?". In realtà la domanda era "*a che numero sto pensando?*": se dicevi 5, "no, dovremmo fare 9"; se dicevi 9, "no, 5". Lo stress costante di dover indovinare cosa vuole il capo nasce dalla mancanza di fiducia ("se mi chiedi come fare, fammelo fare").

Lo stesso gli succede con i CEO che lo chiamano per "sistemare la cultura aziendale" e poi dicono "non sono sicuro, io farei diversamente". La sua risposta: "lo so che faresti diversamente, per questo mi hai chiamato". Se si chiama un esperto, bisogna lasciargli fare il suo lavoro: non si va dal meccanico a cambiare le gomme per poi dirgli "non dovresti girare un po' di più quel bullone?". Chiedere un parere a un esperto significa dargli anche il potere di scegliere. Il problema è il *micromanagement*, o in generale il non accettare punti di vista diversi dal proprio (magari perché si è il fondatore e si pensa di essere l'unico a saper fare le cose).

0.1.15.4 Dare feedback a un superiore egocentrico

Come si dà un feedback negativo a un superiore con un grande ego? Il trucco è **non puntare sull'ego, ma sull'azione e sui suoi effetti**. Non "tu fai così e io mi sento così": anzi, si alimenta il suo ego rendendolo il *main character* dell'avventura, l'eroe che risolve il problema ("tu, grande manager, aiutaci a risolvere questo problema che affligge noi che facciamo l'operatività"). Non è l'ideale ("io cambierei manager"), ma pochissimi possono scegliersi i colleghi: il relatore stesso ha quasi sempre ricevuto team preassemblati ("ecco le chiavi, questa è la tua macchina"). Bisogna giocare con le carte che si hanno in mano.

Un esempio pratico. Con un manager che fa micromanagement e mette sempre "il becco" nel proprio lavoro, si può dire: "se ti fidi di me per questo lavoro, con le tue capacità potresti dedicarti a questa cosa più strategica e di lungo termine: pensa quanto sarebbe più utile per l'azienda". In sostanza: "guarda cosa potresti raggiungere se mi lasciassi fare il mio lavoro". Non come un "tu fai il tuo e io faccio il mio", ma

come un "ti sollevo da questa parte del carico, che so che ritieni importante: lascia che ci pensi io, così puoi dedicarti ad altro", che magari gli dà anche più visibilità.

Si gioca su ciò che quella persona pensa di saper fare, e che probabilmente sa anche fare. Anche Michael Scott di *The Office*, per quanto preso in giro, è raccontato come un venditore bravissimo, che ha chiuso i contratti più importanti per l'azienda, e rispetto ai manager che gli succedono è forse "uno dei meno peggiori". Tutti, anche i manager peggiori, hanno punti di forza: **giocate su quelli**.

0.1.16 Il feedback come regalo

Per chiudere, il relatore risponde alla domanda d'apertura, che aveva lasciato apposta in sospeso: che cos'è il feedback per lui?

> *Il feedback è un regalo che mi viene fatto da qualcuno che so che vuole il mio interesse.*

Ha costruito un team di persone che si fidano di lui e che non hanno motivo di sabotarlo o di ferirlo: per questo interpreta anche le cose peggiori con il default to trust. È un regalo, e cerca di prenderlo in considerazione il più possibile.

Ci è arrivato partendo da se stesso. Oltre a essere avverso ai conflitti, si definisce "avverso alla comunicazione umana": paradossalmente, una persona introversa e timida finita a fare comunicazione, marketing e ora HR ("succede spesso"). Prima di ordinare una pizza al telefono ha dieci minuti di preparazione mentale, e se l'ordine delle domande cambia va in confusione. Per lui un feedback di dieci minuti sono spesso **tre ore di preparazione**: costruisce un **albero dei dialoghi**, come in un'avventura grafica personale, con tutte le opzioni. Poi spesso finisce con "hai fatto questa cosa, andrebbe migliorata" "hai ragione, grazie", e basta.

Così ha pensato: chissà quanti si fanno gli stessi problemi con lui, e passano ore a prepararsi per dirgli "questa cosa l'hai fatta male", sapendo che riceveranno comunque quell'occhiata fugace del "come ti permetti". Eppure vengono a dirglielo. Non può far altro che rispondere: "grazie, hai ragione, cercherò di fare meglio". È così che ha metabolizzato il ricevere feedback: capendo che non c'è nessuna cattiva intenzione e che, se non proprio felice ("ci ho messo una pietra sopra"), può almeno essere *grato* che ci sia ancora qualcuno che ha così a cuore il suo interesse da dargli feedback.

Il silenzio fa più paura. Non bisogna spaventarsi dei feedback negativi che emergono dai sondaggi aziendali: bisogna spaventarsi del **silenzio**, quando nessuno dice più nulla. Significa che nessuno si fida più che si sappiano risolvere i problemi.

La sindrome dell'impostore. Il suo motivatore personale, all'inizio, era soprattutto "non voglio fallire": per questo cercava di migliorare, e migliorare è diventato accettare feedback, perché più feedback riceve, più migliora. Il feedback è un **acceleratore**, e lui cerca di incentivare gli altri a darglielo. Resta però sempre la vocina del "e se si accorgono che non sei capace?". Con la *impostor syndrome* convive da anni e continuerà a conviverci; se ce l'avete, conviveteci anche voi. Avrete sempre la sensazione che "se ne accorgeranno": forse sì, ma sapete fare anche altre cose. Non concentratevi solo su ciò che non sapete fare. Gli errori capitano: la perfezione non è di questo mondo.

Esercitarsi in accademia. Per fare pratica con il feedback, l'ambiente migliore è quello dei colleghi di corso: persone nella stessa situazione, con esperienza simile e lo stesso obiettivo, in un contesto in cui non si può essere licenziati e l'unico effetto a lungo termine è "ho parlato con questa persona, ora le cose vanno meglio". E bisogna avere il coraggio di parlare anche con i docenti ("dire a Diego: secondo me questa lezione potevi farla in questo modo"; "so che è una persona che lo accetta"), senza farsi bloccare dalla gerarchia. Nell'azienda giusta (come quella in cui lavora ora) il relatore va sempre dal CEO a dare feedback, sapendo che non verrà licenziato: il CEO gli ha detto "ti ho preso per darmi feedback".

La frase di chiusura:

> *Per superare le vostre difficoltà, tutto quello che vi viene chiesto è il coraggio di chiedere aiuto.*

Chiedere aiuto non è una debolezza: è un segno di consapevolezza di sé (*self-awareness*) e di forza personale. Se tutti lo facessero, i progetti andrebbero molto meglio; spesso invece ci si fa frenare dal "non voglio fare brutta figura", "non voglio passare per quello scemo", e sono proprio queste paure a rallentarci e a impedirci di raggiungere gli obiettivi.

0.1.17 Domande e risposte finali

L'ultima mezz'ora è dedicata a domande libere.

E se una persona peggiora per uno o due anni di fila? Se nessuno gliel'ha mai detto, c'è un grosso problema, non suo ma dell'azienda. Se invece c'è consapevolezza, il percorso è: "cosa vuoi fare per risolvere?"; "ho provato questo, questo e questo"; "visto che non ha funzionato, vuoi un mio consiglio? Proviamo con quello?"; "che supporto ti serve?". Se, con tutto il supporto, il tempo e le soluzioni concordate per mesi, il miglioramento non arriva, per il relatore la soluzione è: **"dimmi un prezzo per andare via"**. Preferisce pagare una buonuscita (*severance package*) che avviare la strada dei richiami formali sulla performance fino al licenziamento per giusta causa. A un certo punto andare avanti non è un favore per nessuno.

È vero che nei videogiochi si viene sfruttati? Meglio un piccolo studio italiano? Per il relatore si è quasi più a rischio a Blizzard che in un piccolo studio di Roma o Torino. Nel piccolo studio si sa dove ci si va a cacciare: non ci sarà il team building a Santo Domingo, ma ci si concentra su un gioco alla volta e, se va bene, si fa il successivo. A Blizzard si può lavorare su *Overwatch*, che vende milioni, e perdere comunque il posto, in modo totalmente scollegato dal proprio lavoro. Gli stipendi più alti all'estero compensano anche questo rischio, che spesso non dipende da come va il gioco (anzi, "il gioco non ha fatto come volevamo" è spesso una scusa per licenziamenti che non avevano motivo).

Cambiare settore a trent'anni, da web developer full remote? Lo stipendio indicato dallo studente non è fuori dalla portata di vari studi italiani, anzi è superabile. Non va esclusa la possibilità di lavorare da remoto per aziende piccole ma estere. Alcuni amici del relatore, pagati con stipendi italiani, si sono trasferiti a Bali o in paesi con un costo della vita più basso; il collega di *PUBG* percepisce uno stipendio olandese vivendo a Milano, che per quanto cara non costa come i Paesi Bassi.

Come si diventa manager? Dipende dall'azienda: alcune promuovono i migliori (processo tendenzialmente sbagliato), altre propongono un percorso di *people management*. Diventare manager significa **abbandonare ciò che si sa fare per imparare a fare altro**. Il relatore ha dovuto accettare che non sarebbe più stato "il miglior community manager italiano del mondo" ("eravamo in pochi, quindi era facile"): dedicandosi alle persone non poteva più fare il suo lavoro bene come prima. È stata una negoziazione con se stesso: fare il manager significa non essere più l'esperto di qualcosa, ma saper gestire e far crescere persone. Significa anche **cedere il controllo della propria carriera**: se il team decidesse di non fare più nulla, il manager ne pagherebbe le conseguenze. Il suo lavoro diventa scegliere le persone giuste e farle lavorare bene. Lui lo esplicita sempre: sa di affidare al team parte della propria carriera, e il team sa che lui combatterà per loro, anche prendendosi le colpe.

L'inglese. Va assolutamente imparato, anche lavorando in Italia. Non saperlo preclude circa il 95% del mercato (tutto quello fuori dall'Italia) e soprattutto il *networking* con persone che potrebbero dare occasioni future: l'inglese è spesso l'unica lingua in comune con chi lavora all'estero ed è oggi la lingua universale dei videogiochi.

- *Che livello serve?* Non importa B2 o C2: bisogna saper sostenere una conversazione, guardare almeno Netflix con sottotitoli in inglese, comunicare senza dodici pause e senza "come si dice...". Il certificato non verrà mai chiesto: verrà chiesto se si sa parlare inglese. Il relatore ha sentito persone con un C2 che non sapevano parlarlo.

- *Come allenarlo senza occasioni di conversazione?* Netflix, Prime Video, Now TV, Udemy, Coursera, YouTube, Duolingo, siti di scambio linguistico con madrelingua, Discord: non sono mai esistiti così tanti mezzi gratuiti. Il relatore ha imparato l'inglese giocando (con una pronuncia sbagliata, perché leggeva senza sentire, su cui ha poi dovuto lavorare); guardare un film in inglese con i sottotitoli in italiano insegna anche la traduzione.
- Oltre all'ascolto passivo è fondamentale **parlare**: in una flotta di *EVE Online* con 150 persone quasi tutte madrelingua inglese si è esposti a decine di accenti e modi di dire (mille modi di dire "come stai" oltre a *how are you*); lavorando all'estero si impara a distinguere gli accenti (Manchester, Londra) e ad apprezzarli anche nei media.

Quanto deve sapere un junior? Dipende da quanto è tecnico il ruolo. Un game designer junior che non sa qualcosa può essere affiancato da un senior; un programmatore che non sa programmare, in quel momento, non è utile all'azienda.

Annunci "junior con cinque anni di esperienza". "Cercasi ventenne con trent'anni di esperienza." Non è l'annuncio per un junior: è un errore dell'azienda, che in realtà cerca un *mid*. Riceverà profili junior, si stupirà che non abbiano cinque anni di esperienza, rifarà l'annuncio e avrà perso sei mesi. Il consiglio: tenere d'occhio quella pagina, perché dopo qualche mese l'annuncio verrà corretto.

Una struttura per il post-mortem. A chi chiede un formato di base, il relatore anticipa quanto vedrà nella seconda giornata a proposito di brainstorming: una lavagna **divisa in quattro aree**, su cui ognuno attacca tutti i post-it che vuole:

Smettere di fare ciò che non ha funzionato	**Iniziare a fare** ciò che non abbiamo fatto e andava fatto
Fare di meno ciò che ha fatto perdere tempo rispetto all'impatto ottenuto	**Fare di più** ciò che ha funzionato ma, fatto con troppa leggerezza, non ha dato i frutti sperati

Con i post-it ognuno ha voce e tutto risulta catalogato; per il report basta una foto della lavagna (o uno screenshot, se la lavagna è digitale, come Miro).

Come sbloccare la paralisi del feedback verso il management? Uno studente ha visto manager dire "non abbiate paura, la mia porta è sempre aperta" senza che cambiasse nulla. C'è una dissonanza tra ciò che dicono e ciò che fanno: è "la mamma che dice vieni qua che non ti faccio niente". Qualcosa deve aver fatto pensare alle persone che non è sicuro. Prima va sistemato quello; poi si può provare con i **feedback anonimi**. L'obiettivo del relatore resta il feedback aperto e firmato, ma a Stormind ha iniziato con feedback anonimi verso il management per verificare se l'anonimato fosse un fattore. Non lo era (evidentemente non c'erano problemi a parlare con il CEO), e la volta successiva intende abbandonarli. Ma è un buon *check*: se senza anonimato non arriva nulla e con l'anonimato arrivano cento feedback, il problema è chiaro.

Quando è giusto rivolgersi a un superiore? La strada giusta resta parlare direttamente con l'interessato. Ma se si prova e non si trova una soluzione, la cosa più ovvia è andare dal proprio manager insieme: "ci aiuti?". Quando il relatore parlava di mediare i dialoghi, era proprio questo: fare da moderatore, aiutare a sbloccare i punti fermi o, se non si riesce a scegliere tra A e B, scegliere lui. L'escalation, quando necessaria, va fatta.

Il tempo libero veniva rispettato? No, e se l'è dovuto imporre lui. Il lavoro gli piaceva così tanto che continuare a casa non gli sembrava lavorare. Gli si è ritorto contro con il lancio di *Diablo III*, fatto in orario americano (le due o l'una di notte in Europa), con l'ufficio chiuso perché a nessuno era stato chiesto di restare. Il lancio va malissimo, i giocatori protestano, e lui, sveglio, risponde sui forum fino alle tre, quattro, cinque del mattino: "non posso risolvere il problema, ma resto qui ad aggiornarvi con quello che riesco a

sapere dal team di sviluppo". Il giorno dopo trova sulla scrivania un **richiamo formale dell'HR**: aver lavorato fuori orario senza autorizzazione, condotta non professionale. Per lui è stato un interruttore: "ora me ne vado". Due settimane dopo faceva colloqui con Riot. Ancora oggi è contrario a quel richiamo, ma gli ha fatto capire come ragiona un'azienda: "hai un contratto 9–18, non mi interessa se l'hai fatto con le migliori intenzioni". Eppure per anni ha goduto di un'ottima reputazione in quelle community come "quello che è rimasto con noi fino alle cinque del mattino", e anche l'azienda ne ha beneficiato.

Ci sono poi aziende in cui, soprattutto per i developer, viene richiesto il famigerato **crunch**: lavoro straordinario continuativo, senza pause. Dipende molto dall'azienda; sta venendo ("grazie al cielo") deprecato, perché molte aziende capiscono che non è sano ed è visto male anche dal pubblico, ma esiste ancora.

E se il tuo capo demotiva qualcuno del tuo team? Qui "diventa personale": il team del relatore si fida di lui perché sa che in questi casi "diventa una bestia". "Te la puoi prendere con me quanto vuoi, ma il mio team non lo tocchi." Lo gestisce direttamente con l'altro manager, per quanto brutta possa diventare la conversazione. Lo sforzo più grande è **non andarne poi a parlare con il team** ("ragazzi, ho fatto questa cosa"), perché minerebbe la fiducia verso l'alto: non vuole un rapporto ottimo fino a lui e pessimo oltre. L'obiettivo è che l'altro manager "si faccia perdonare" cambiando il proprio comportamento, e che il team percepisca da solo il cambiamento.

Chiusura della giornata. Il relatore chiede un feedback collettivo in chat ("se domani ci dai *more of the same* siamo contenti, oppure ditemi cosa limare"), si scusa per i ripetuti crash della connessione e ringrazia per la proattività nelle domande, che gli servono anche come *check*: se nessuno chiede nulla su un argomento, significa che può andare avanti. Racconta di aver passato una settimana a decidere che cosa mettere nelle slide e di averle poi preparate in poco tempo (rifacendone alcune fino alle tre di notte). Ricorda infine di usare il test di personalità per sé e per il proprio gruppo, facendolo fare a chi non l'ha ancora fatto: tornerà molto utile per cooperare meglio.

0.1.18 Anticipazioni sulla seconda giornata

La seconda giornata del workshop sarà più pratica: "oggi vi ho spiegato perché, domani vi insegno come lo fanno queste aziende". Gli argomenti previsti:

Time management come organizzarsi una giornata professionale in azienda: come strutturare una *task list*, cosa fare e cosa non fare, come e cosa prioritizzare.

Brainstorming i gruppi dovranno fare moltissimi meeting (come facciamo il gioco, come lo chiamiamo, che scopo gli diamo). Se non si sanno fare bene, i brainstorming sono **dannosi ed eterni**: un brainstorming dovrebbe durare **al massimo due ore**. Verranno mostrati metodi che accorciano la durata, massimizzano l'effetto e, soprattutto, **fanno partecipare tutti**: non c'è niente di peggio di un brainstorming in cui parlano in due per il 90% del tempo. Il classico "ragazzi, non esistono idee sbagliate, sparate" è il modo peggiore di farlo; non è colpa di nessuno, ma si può fare diversamente.

Guidare da dentro come essere leader nel proprio team anche senza esserne il leader: sviluppare responsabilità personale e verso gli altri. Non essere il manager di qualcuno non significa non esserne responsabili: la persona accanto è responsabilità di tutti, perché si è un team e non un insieme di individui. "Se qualcuno cade, tutti lo facciamo rialzare: non c'è l'addetto al rialzamento." È la versione *advanced* della giornata: come comportarsi da manager senza averne il titolo.

Torneranno inoltre il *service level agreement* e il tema della condivisione dei successi e della protezione del team dai fallimenti.

Punti chiave del capitolo

- "La capacità di un gruppo di persone di fare cose straordinarie dipende dal modo in cui queste persone operano insieme." Il 90% dei problemi dei team sono **problemi di comunicazione**.

- Le aziende si liberano sempre più presto dello **useless genius**: le competenze si trasmettono, il carattere no. Il **fit culturale** va valutato anche dal candidato.
- Non esiste un unico modo di dare feedback: dipende dal contesto e soprattutto da **chi lo riceve**. Il feedback è una somma di piccoli miglioramenti periodici; tenersi le cose nel cassetto non serve mai.
- I team attraversano **forming, storming, norming, performing e adjourning**. Lo storming non si salta; si passa la maggior parte del tempo in forming; la bravura sta nell'accelerare i passaggi e nel fare bene il **post-mortem**, anche dei successi.
- Gli obiettivi di un progetto vanno fissati **all'inizio**: solo così si può valutare se è andato bene e scegliere tra A e B.
- Il circolo virtuoso è **feedback** $\rightarrow$ **miglioramento** $\rightarrow$ **fiducia**. Accettare un feedback non significa fare come dice l'altro: decide chi ha la responsabilità, ma deve argomentare.
- "Aiutami a poterti aiutare": meglio dire in anticipo "potrei non farcela" che "non ce l'ho fatta". Chiedere aiuto non è una debolezza.
- Quando si dà feedback **non si porta la soluzione**: la si costruisce insieme a chi lo riceve.
- La cultura è forte quando si lavora **per gli altri**; il lavoro è valutato come opera d'insieme (anche con bonus di team).
- Come in *Among Us*, i team sprecano energie a cercare il traditore invece di fare le task. **Default to trust**: chi ha sbagliato ha agito con le informazioni che aveva. Cercare **la causa, non il colpevole**: "che cosa ti ha fatto pensare che fosse la scelta giusta?".
- Carriere e performance non sono lineari: vanno valutate le **traiettorie**, non un punto dell'anno. Le valutazioni puntuali generano *lying, hiding, faking.*
- Confermare sempre le istruzioni facendosele ripetere; "non ho capito" non è una colpa.
- Dare feedback: sull'**azione** e sui suoi **effetti**, specifico, presto, uno a uno, adattato al carattere, senza terze parti, senza "non mi piace", senza offendersi se non viene seguito; poi fare il debriefing.
- Ricevere feedback: **ascolta, chiedi dettagli, accetta, agisci**. Il feedback positivo serve a non peggiorare ciò che si fa bene; il feedback accidentale e il *drive-by* non servono a nulla.
- Feedback **frequenti e recenti**: tanti piccoli *sanity check* costano meno di una grande correzione di rotta.
- Il feedback è **un regalo** di chi ha a cuore il nostro interesse; fa paura il silenzio, non le critiche.

Chapter 0.2

Gestire il tempo, generare idee, guidare da dentro

Questo capitolo raccoglie la **seconda giornata** del workshop di due giorni con cui si aprono le lezioni zero, tenuto dallo stesso relatore del capitolo 0.1, Stefano Cozzi[1]. Se la prima giornata spiegava *perché* le aziende tengono tanto alla cultura del feedback, la seconda è dichiaratamente più pratica: "ieri vi ho spiegato perché, oggi vi insegno come lo fanno queste aziende". I concetti della prima giornata (il *default to trust*, il "potrei non farcela" detto in anticipo, la condivisione dei successi, il feedback frequente) ritornano di continuo, applicati alla gestione del tempo, alle riunioni creative e alla leadership.

La giornata si è svolta di domenica, a chiusura di un fine settimana di workshop[2], con il relatore alle prese con la voce un po' calata dal giorno prima ("la sfida principale di oggi sarà contro la mia voce").

Il percorso del capitolo segue quello della lezione:

1. il programma della giornata (sezione 0.2.1);
2. le domande sulla prima giornata, che si trasformano in una lunga sessione su **colloqui e selezione** (sezione 0.2.2);
3. il **time management**: pianificare la giornata, le liste, le checklist, le interruzioni (sezione 0.2.3);
4. la **matrice di Eisenhower** per prioritizzare (sezione 0.2.4);
5. come proteggere il proprio tempo: sprechi, distrazioni, il saper dire di no (anche ai meeting), il mito del multitasking e la chiusura della giornata (sezione 0.2.5);
6. le domande sul time management e una parentesi su tempo libero, ferie e benessere in azienda (sezioni 0.2.6 e 0.2.7);
7. il **brainstorming**: obiettivi, problemi tipici, un metodo completo (*rapid ideation* e *dot voting*) e la **matrice impatto–sforzo** per decidere (sezioni 0.2.8, 0.2.9 e 0.2.10);
8. i consigli pratici, le domande sul brainstorming e il principio del **disagree and commit** (sezioni 0.2.11 e 0.2.12);
9. il **leading from within**: perché la leadership non è un titolo, le carriere divise tra manager ed esperti (sezione 0.2.13);

[1] Per il cognome e per il nome dello studio (Stormind Games) valgono le avvertenze della nota all'inizio del capitolo 0.1. In questa giornata la trascrizione riporta soltanto il nome di battesimo, "Stefano", con cui il relatore si cita più volte nei suoi esempi.

[2] Il relatore dice "oggi chiaramente è domenica" e saluta la classe con "buon resto del weekend"; la prima giornata si era quindi svolta di sabato. Più avanti parla di "dodici ore" di workshop in totale, mentre nel capitolo 0.1 si parlava di circa dieci ore: la differenza dipende probabilmente dal conteggio delle pause.

10. il "momento Piero Angela" sulla biologia del gruppo e le domande che ne sono nate (sezioni 0.2.14 e 0.2.15);

11. la **piramide della fiducia**, il prendersi cura del gruppo e che cosa cambia quando si diventa manager (sezioni 0.2.16, 0.2.17 e 0.2.18);

12. le domande finali e la chiusura del workshop (sezioni 0.2.19 e 0.2.20).

Come per il capitolo precedente, una premessa di metodo: buona parte della giornata è fatta di domande della classe e di racconti dell'esperienza del relatore. Sono riportati con cura perché sono il modo in cui i principi vengono resi concreti, e perché spesso contengono le indicazioni più direttamente utilizzabili (soprattutto sui colloqui di lavoro).

0.2.1 Il programma della giornata

Come annunciato alla fine della prima giornata (sezione 0.1.18), gli argomenti sono due:

- un "doppio topic", **time management e brainstorming**;
- il **leading from within**, che il relatore definisce "la versione avanzata di ieri": come guidare un team stando dentro al team, verso i colleghi e verso il gruppo con cui si lavora.

Poiché l'ordine è indifferente, il relatore lascia scegliere alla classe: dalle poche preferenze espresse, la leadership va al pomeriggio, così che le domande finali si concentrino su quella, mentre la mattina è dedicata a time management e brainstorming. Il materiale c'è, ma il relatore avverte che la giornata potrebbe finire prima delle quattro: non vuole trattenere nessuno se non c'è niente di specifico da fare. Per l'eventuale tempo avanzato prima della pausa pranzo propone un **argomento bonus**: i colloqui di lavoro, selezione e *recruitment*, visto che il relatore ha fatto e fa tuttora selezione del personale e che per molti studenti sarà la cosa più immediata da affrontare finito il percorso di studi (domande comuni, come rispondere, eccetera).

Prima di cominciare, però, il relatore apre a **domande sulla giornata precedente**: la classe ha avuto modo di "metabolizzare" i contenuti e potrebbero essere emersi dubbi a posteriori. È proprio da una di queste domande che nasce, in anticipo, la parte sui colloqui.

0.2.2 Colloqui di lavoro: capire che cosa cerca l'azienda

0.2.2.1 L'annuncio come prima fonte di informazione

La domanda: il giorno prima si era detto che è importante capire che cosa l'azienda cerca davvero; ma se il primo colloquio è con le risorse umane, cioè con qualcuno che magari "di tecnico non sa", come si fa a capirlo? Il relatore risponde subito, nel dubbio che più tardi manchi il tempo.

La prima fonte è **l'annuncio**. Non viene scritto a caso: è approvato sia dall'HR sia dall'*hiring manager*[3], cioè la persona che seguirà il candidato una volta assunto, e spesso dall'intero dipartimento HR. È calcolato e scritto in un certo modo per far capire che cosa serve. Molti annunci moderni, oltre alla parte su *che cosa si verrà a fare*, contengono una parte su *che cosa ci si aspetta da questa persona*, cioè il comportamento e il carattere che servono in quella posizione: ed è quella che dice molto sulla **cultura aziendale**.

L'esempio è quello di due annunci per la stessa posizione (programmatore senior) descritta in modo diverso:

1. "cerchiamo un programmatore esperto che possa prendersi cura dei nostri programmatori junior, aiutarli a crescere, sviluppare un ambiente di lavoro inclusivo e stimolante, in modo che i nostri progetti futuri siano all'avanguardia";

2. "cerchiamo un programmatore esperto che sappia rispettare le deadline, seguire le indicazioni dei dati, sia indipendente nel suo lavoro e sappia relazionarsi con gli *stakeholder* per capire e proporre nuove direzioni per il progetto".

[3] Nella trascrizione "Airing Manager" e "Iron Manager"; allo stesso modo "e-char" e "char" stanno per HR.

Nel primo caso ci si aspetta un team di junior, probabilmente con un'età media più bassa, con cui ci si relazionerà soprattutto; per un senior è l'occasione di sviluppare le doti da *people manager*, e al colloquio arriveranno domande come "hai mai lavorato con dei team? Con che età media? Ti sei mai trovato a guidare un gruppo?". Nel secondo caso il senior deve eseguire un piano deciso o proporre qualcosa, e si relazionerà soprattutto *verso l'alto*; al colloquio arriverà piuttosto "fammi questo test di programmazione". Sono, di fatto, due colloqui completamente diversi.

0.2.2.2 Quanti colloqui e con chi

Ormai è molto raro essere assunti con un solo colloquio: ne servono almeno due, realisticamente **tre**, che è diventata la media. L'iter tipico:

1. il primo con l'HR;
2. il secondo con l'HR o con il futuro manager;
3. il terzo sicuramente con il futuro manager, perché l'azienda vuole far conoscere al candidato la persona con cui lavorerà, e viceversa.

Già le domande diverse dei vari interlocutori aiutano a capire che cosa cercano. Ma c'è una parte fondamentale, che secondo il relatore molti sottovalutano: il momento finale delle **"hai domande?"**.

0.2.2.3 Le domande da fare al selezionatore

Un principio di base: **i colloqui non sono "devo farmi selezionare dall'azienda", sono "dobbiamo selezionarci a vicenda"**. Le domande vanno quindi preparate: ci sarà sempre qualcosa che, dalla *job description* o dal colloquio, non è chiaro al cento per cento. Nel dubbio, il relatore suggerisce tre domande "che funzionano sempre":

"Se iniziassi domani, quale sarebbe il mio *day to day*?" Ottiene due effetti. Il primo è pratico: descrive la giornata tipo e fa capire se piacerebbe o no. Il secondo è più psicologico, ma il relatore sa che funziona: costringe l'interlocutore a pensare al candidato *già dentro l'azienda*, cioè a che cosa potrebbe fare da subito e a quale impatto potrebbe avere con le sue competenze.

"Se aveste una bacchetta magica, che cosa cambiereste subito dell'azienda?" Sposta la conversazione su di loro e va bene sia con l'HR sia con l'hiring manager. Li obbliga a indicare una sfida interna che vorrebbero risolvere, ed è sempre un'informazione utile: se entrambi rispondono che cambierebbero il modo in cui si prendono le decisioni, viene da pensare che non ci sia un percorso chiaro e trasparente da parte di chi sta in alto.

Lo scambio: "vi do un motivo per non prendermi, voi datemene uno per non venire". È più avanzata e il relatore non la usa sempre, ma la trova particolarmente interessante. Permette al candidato di esprimere i propri dubbi in modo costruttivo ("mi dite che lavorerei a stretto contatto con un team di junior: un motivo per non prendermi è che non ho anni di esperienza nella gestione di giovani sviluppatori, sarebbe la mia prima volta; però penso di poterlo fare perché ho questo interesse e queste competenze"). Dall'altra parte, il selezionatore dirà che cosa secondo lui sarebbe più difficile per il candidato: si scopre così che cosa sta già pensando, e l'informazione torna utile anche per i colloqui successivi.

E se il selezionatore si innervosisce? "Se un recruiter si innervosisce per le domande del candidato, ha sbagliato lavoro." Al massimo il relatore si innervosisce se un senior gli chiede che cosa faccia di preciso un programmatore senior ("lo fai da dieci anni, lo saprai tu"); ma se un junior chiede che cosa dovrebbe fare ogni giorno, lo si spiega volentieri. E in realtà vale anche per i senior, perché **in ogni azienda lo stesso ruolo fa cose diverse**: conviene sempre farsi spiegare che cosa intendono per quel titolo. L'esempio viene dal mondo delle community: *community manager* a Riot indicava la persona a capo del team di *community representative* di una nazionalità ("community manager italiano"); a Blizzard indicava la persona sopra

l'intero dipartimento che gestiva quei team. Candidarsi per lo stesso titolo non garantisce di aver capito per che cosa ci si sta candidando, e i recruiter lo sanno.

Se poi un recruiter si innervosisce comunque, sta dando più risposte di quelle che pensa: vuol dire che l'azienda non vuole domande, o non vuole interazioni del tipo "ma perché facciamo questa cosa?". Che abbia formato i suoi recruiter a innervosirsi, o che accetti che lo facciano, è informativo.

0.2.2.4 Le domande che dicono molto sull'azienda

Allo stesso modo, come detto il giorno prima, è informativo se un recruiter chiede a una donna se vuole avere figli: "ah, quindi a voi interessa sapere se voglio avere figli: questo già mi dice che probabilmente non voglio lavorare qui".

Un esempio *gender neutral*: la richiesta di vedere la **busta paga precedente**. "Quanto è la tua RAL? Mandami una busta paga." La risposta del relatore è no: "ti ho detto qual è la mia RAL; se vogliamo instaurare un rapporto in cui non ti fidi di quello che dico, abbiamo un altro problema". È di nuovo *default to trust* (sezione 0.1.11). Il relatore è quasi sicuro che legalmente non si possa chiedere la busta paga di un candidato[4]: a quel punto diventa lui antagonista del recruiter, e visto che la conversazione è finita in una "zona grigia", fa lui le domande ("come mai è legale questa cosa che mi chiedi?"). Ha già deciso che probabilmente non lavorerà lì, ma così magari evita che i candidati successivi debbano subire le stesse richieste. Cita l'esempio perché sa che in Italia è abbastanza comune, mentre all'estero non gli è mai successo, anche se avrebbe potuto "sparare qualsiasi cifra".

Il motivo è che **non c'è alcun onere di dimostrare quanto si guadagna**: chi spara troppo alto si autoesclude. Recruiter e manager non arrivano al colloquio con carta bianca né con l'indicazione di risparmiare il più possibile, ma con un *range*: "per questa posizione possiamo spendere dai 30 ai 40 mila euro l'anno; paga questa persona a seconda della sua seniority". Se un candidato che guadagna 30 dice di prenderne 55 per "provare a fare i soldi" e il budget arriva al massimo a 40, il relatore non chiederebbe nessuna prova: direbbe "grazie, siamo in un range diverso, possiamo salutarci". La domanda sulla busta paga è quindi *formulata male*, perché è poco furba. Quella giusta è: **"hai già una RAL in mente per questa posizione?"**. Si ottengono le stesse informazioni senza passare per la zona grigia.

0.2.2.5 La cifra minima mentale

E se la domanda la fanno a un junior che non ha mai lavorato? Il consiglio vale per tutti: **preparatevi sempre la vostra cifra minima mentale**, per qualsiasi offerta di lavoro, prima ancora di fare il colloquio. Non serve conoscere la media italiana per quella posizione o quanto paga quell'azienda: serve sapere *quanto si è disposti ad accettare senza andare in negativo*, tenendo conto dei propri costi (la benzina per andare in ufficio ogni giorno, un eventuale trasferimento, e così via). Quella cifra va sempre avuta in testa.

0.2.2.6 La scrematura iniziale e il portfolio

Uno studente chiede come funzioni la risposta a una candidatura: arriva un rifiuto diretto, oppure c'è una scrematura iniziale e poi i colloqui? E vale per tutti i ruoli? Sì: c'è una **scrematura iniziale** fatta dall'HR o, in alcune aziende, direttamente dal software dell'HR. Si scarta chi è chiaramente fuori da ciò che si cerca:

- si cerca un *graphic designer* e arriva il CV di un programmatore ("faccio questi esempi perché purtroppo succede");
- si cerca un *animator* tecnico bravo con MotionBuilder, lo si scrive nell'annuncio, e il candidato risponde "so fare tutto tranne MotionBuilder": c'è un motivo tecnico per non poterlo prendere;
- problemi di seniority: si cerca un senior e risponde un junior con un anno di esperienza.

Se la figura non è già chiaramente fuori target, si fa il primo colloquio.

[4] È l'opinione del relatore, espressa come tale ("sono quasi sicuro"); non si tratta di un parere legale e non è stata verificata.

Gli anni di esperienza negli annunci junior. Spesso gli annunci junior chiedono come minimo tre anni di esperienza. **Se l'annuncio dice "junior", ignorate i requisiti di esperienza**: o sono lì per sbaglio, o sono lì per scremare. Il secondo caso è più *borderline*, ma il relatore l'ha visto fare in alcune aziende americane: si mettono requisiti molto alti, chiaramente irraggiungibili, per vedere chi ha la "faccia tosta" di mandare comunque il CV ("non me ne frega se chiedete un *bachelor's degree*, non ce l'ho, mando lo stesso"). Così però si filtra una grossa fetta di popolazione e si seleziona un certo tipo di carattere: si stanno cercando "i Top Gun". Non è quindi necessario iniziare con uno stage: si può iniziare direttamente da junior.

Chi guarda il portfolio. Il relatore non ha mai avuto un portfolio, ma per le figure che lo richiedono parte comunque dal CV (per escludere, ad esempio, il designer che si candida da programmatore) e guarda il portfolio solo a parità con altri candidati: da HR può aprirlo e dire "bello", ma non può dare un giudizio tecnico. Il portfolio lo apre chi sarà l'hiring manager, che è sempre molto più tecnico. L'ordine quindi è: l'HR guarda il CV e si salva il portfolio; se il candidato passa la prima scrematura, CV e portfolio vanno a un tecnico che conferma ("sì, sa fare queste cose") o boccia ("il portfolio indica chiaramente che non sarebbe in grado, digli pure che non facciamo il colloquio").

0.2.2.7 Test pratici, da casa e con qualsiasi strumento

Si può chiedere un test pratico al posto del portfolio? Uno studente chiede se, con un portfolio che non rispecchia le proprie capacità, si possa chiedere direttamente un test. No: se l'azienda ha bisogno di verificare una competenza ha già pronto un test e lo fa fare a prescindere. Il relatore, per esempio, fa fare un test ai programmatori e lo fa correggere dal suo senior developer; per gli artisti si chiedono animazioni di prova, chiaramente veloci. Non chiede mai test troppo lunghi, perché non vuole far lavorare nessuno gratis: non chiede "fammi un pitch di progetto", chiede uno spunto per capire come si ragiona e come si lavora. Per il resto, **se il portfolio non è indicativo, va aggiornato il portfolio**.

Test dal vivo o a casa? Il relatore li fa fare sempre a casa. Sa benissimo che un programmatore a casa lavorerà con due monitor, il codice da una parte e Google o ChatGPT dall'altra. Ne ha discusso con i suoi senior programmer: interessa che qualcuno sappia le cose a memoria, o che sappia trovarle e fare quel lavoro a prescindere? A lui interessa che il lavoro **venga fatto**: che il candidato abbia imparato il codice a memoria o che in venti minuti trovi le informazioni, le metta insieme, le faccia elaborare a ChatGPT, le corregga e consegni il codice che serve, non ha preferenze. Quando si lavora le indicazioni saranno "fai questa cosa il prima possibile": se la si sa fare anche con strumenti aggiuntivi, ben venga, "anzi, spiegalo anche agli altri". Farla a memoria va benissimo, "ma perché mettere l'*hard mode*?". E comunque ci sono cose che non si possono delegare a uno strumento: se arriva un codice che non funziona e va fatto il *debug*, bisogna saperlo fare e saper progettare i test. La figura più completa, per il relatore, è anche quella che sa trovarsi le informazioni e gli aiuti.

Le prove dal vivo sono soprattutto domande di *problem solving*: il relatore vuole sentire come si ragiona e chiede espressamente di **pensare ad alta voce**, per seguire il percorso mentale che porta alla conclusione. Sono più domande impegnative che veri test. I test veri e propri si fanno con i propri tempi e i propri strumenti: "scegli il momento, ti mandiamo il test, hai due ore per rimandarlo indietro". Questo è l'accordo; in quelle due ore si può fare quello che si vuole.

0.2.2.8 Junior, mid e senior

Perché ci sono così pochi annunci per figure mid? Uno studente ha notato su LinkedIn e sulle piattaforme di recruiting un divario notevole tra le richieste di junior e quelle di mid. Il motivo è che per molte aziende, Stormind inclusa, **la differenza tra junior e mid è sparita**: o si è senior o non lo si è. Ci si è resi conto che la differenza tra un junior e un mid è che il mid si è abituato a lavorare in quell'azienda e il junior non ancora; ma un mid che arriva con tre anni di esperienza in un altro studio (ad esempio Milestone[5]) va comunque "ripreso come junior", perché si lavora in modo diverso e deve reimparare. Un senior, invece, ha molte più competenze e le sa applicare in un certo modo, e quella distinzione resta netta.

[5]Nella trascrizione "se ha fatto tre anni in milestone": si tratta verosimilmente di Milestone, studio di sviluppo milanese.

Ci si può candidare anche se il livello non coincide? Un'altra studentessa dava per scontato che, se un annuncio non specifica "junior", sia per mid. Se non è specificato "senior", secondo il relatore si può mandare il CV chiedendo di valutare. Certo, se è specificato junior e si è junior le possibilità di essere contattati sono maggiori. Ma vale lo stesso discorso del livello di inglese fatto il giorno prima: si cerca qualcuno che *sappia fare quelle cose*, non una certificazione. Se serve un programmatore che sappia usare Unreal Engine 5 e sistemare la cosa X o Y, che sia mid da due anni o junior da uno non è un problema.

0.2.2.9 Gli strumenti di intelligenza artificiale

Una domanda in chat riguarda l'uso quotidiano dell'intelligenza artificiale in azienda. Il relatore sta facendo fare corsi di IA al suo team "perché voglio che la gente lavori meno": vuole che si sfrutti qualsiasi strumento disponibile, perché gli altri lo stanno facendo e non vuole restare indietro. Se un lavoro si può fare in tre ore di meno, non c'è alcun vantaggio a farlo in modo lungo e tortuoso. Il rischio esiste: se si fa fare tutto a ChatGPT e non ci si tiene al passo con il proprio *know-how*, quando ci sarà da risolvere qualcosa ci si potrebbe "impallare". Ma sta al singolo restare aggiornato e saper fare il proprio lavoro: **gli strumenti devono essere strumenti, non un sostituto delle competenze**. "Non è una partita di calcio, con regole su come si devono fare le cose o un *dress code*": il lavoro va fatto nel modo più efficiente possibile.

A uno studente a cui era saltata la connessione proprio in questo momento il relatore risponde che la lezione è registrata e si può recuperare dal video. Poi chiude la parentesi: "la parte difficile era finire ieri in tempo, ce l'abbiamo fatta".

0.2.3 Time management: pianificare la giornata

0.2.3.1 Time management o prioritizzazione?

La domanda di partenza alla classe è: *chi di voi, in questo momento, fa time management su se stesso*, cioè si organizza la giornata all'inizio e organizza il proprio tempo? Qualcuno risponde di sì, qualcuno ha uno *schedule* settimanale con orari da rispettare, qualcuno "ci prova" ("hai fatto bene a specificare *ci provo*: magari non la rispetto"). Il relatore lo chiede perché è uno di quegli argomenti in cui molti, dopo aver visto la spiegazione, si accorgono di non fare davvero time management: fanno piuttosto **prioritizzazione**, ad esempio una lista di cose da fare, che è un po' diverso.

Nessuna delle tecniche è complicatissima, ma è utile sapersi muovere. Il relatore colloca volutamente questo modulo *dopo* quello sul feedback: saper gestire il proprio tempo permette anche di dare feedback agli altri sulla loro gestione del tempo.

0.2.3.2 La *must do list*

Il punto di partenza è il **planning iniziale**: non improvvisare la giornata. Improvvisando si è in balia di una grande limitazione umana: ci si dimentica le cose, oppure ci si lascia trascinare dal task più urgente o dall'ultimo arrivato. Esistono modi di dire come "se arriva una mail o rispondi subito o diventa un task per domani", perché il "rispondo dopo" non arriva mai; più avanti si vedranno modi più raffinati di gestire le zone grigie, ma la pianificazione resta fondamentale.

La prima cosa da fare *non* è la lista delle cose da fare oggi, ma la lista delle cose che **assolutamente** vanno fatte oggi. Le due liste non devono coincidere: la seconda richiede un livello di ragionamento in più. Sono le cose per cui, se alle sei si stacca e non sono fatte, si resta finché non sono fatte, perché sono imprescindibili.

Il primo momento per dire "potrei non farcela". La *must do list* è anche il primo indicatore della giornata. Se è troppo corposa (dipende naturalmente dalla complessità dei task), quello è il momento, "alle 9:01 se iniziate alle 9", per dire al proprio manager *oggi sarò in difficoltà*. Alle 9:01 il manager ha l'intera giornata per aiutare; se invece si pecca di ottimismo o di presunzione ("ce la posso fare") e glielo si dice alle 17:59, non può fare nulla. È lo stesso discorso del giorno prima (sezione 0.1.8): anche facendo la to-do list si applicano i concetti della cultura del feedback, ed è il motivo per cui questo modulo viene dopo. Ci saranno vari momenti di questo tipo nella giornata, ma questo è il primo.

E se ci si accorge a metà giornata di non farcela? Uno studente chiede che cosa fare se, convinto di farcela, si accorge a metà giornata di non poter completare bene la *must do list*. Il relatore lo concederebbe *una volta*: se si vogliono testare i propri limiti, va bene, ma comunque si dice al capo "oggi potrei essere tirato, ci provo", e sarà lui a valutare se il rischio è accettabile. Dipende dal rischio: se il *must do* è consegnare una milestone a un cliente, "nel dubbio facciamolo fare a qualcun altro, non possiamo correre quel rischio"; se il rischio è che domani un animator riceva il codice con tre ore di ritardo, lo si può correre. "È un po' questo il lavoro."

Cinque minuti e una stima. Tutto questo va fatto in circa cinque minuti. Se non è chiaro che cosa si deve fare in una giornata, **si chiede**: ogni giorno dovrebbe essere chiaro. Poi si cerca di **stimare il tempo** di ogni cosa. Si diventa più bravi col tempo: alla quinta volta che si fa il report mensile si sa che ci vuole un'ora e ventidue minuti; la prima volta si può chiedere a un collega quanto ci mette lui, sapendo che per noi potrebbe essere diverso.

0.2.3.3 To-do list e checklist

Esistono molti strumenti per tenere to-do list e checklist; il relatore non ne consiglia uno in particolare e cita come esempi Any.do, i promemoria di Windows o Samsung Notes[6]: "potete usare davvero la qualunque". L'importante è tenere la lista **visibile** da qualche parte sul computer. Alcuni accorgimenti:

- i **must do** vanno sempre **in cima** alla lista;
- quando durante la giornata arriva un task nuovo, la cosa da fare subito non è *eseguirlo*, ma **aggiungerlo alla lista**: "se arriva una mail, aggiungete subito alla lista *rispondi alla mail*", decidendo se è diventato un *must do* o se è comunque un task per oggi;
- non limitarsi alle cose operative: il relatore ha iniziato ad aggiungere anche task come "parla con Tizio" o "parla con Caio", perché è da un po' che non chiede loro un aggiornamento; se non li mette in lista si accorge di non farli;
- spuntare le voci completate: per quanto minimo, rilascia **dopamina** (se ne parlerà nel pomeriggio, sezione 0.2.14), aiuta a motivarsi e ad andare avanti. Sono piccole cose, ma si sommano.

Priorità contro recenza. Che cosa succede se si esegue sempre l'ultimo task arrivato? Lo chiede il relatore alla classe. Si interrompe il flusso di lavoro e si cambia continuamente attività, certo; ma soprattutto si stanno **mescolando priorità e recenza**: si sta dicendo "non importa come ho prioritizzato i task, la mia priorità è sempre l'ultimo arrivato", che non vuol dire fare le cose più importanti. È il motivo per cui tanti dipartimenti IT, quando si chiede loro qualcosa, rispondono "mi apri un ticket?". È diventata quasi una battuta, ma il concetto è chiarissimo: devono poter prioritizzare quella richiesta rispetto alle altre 112 arrivate in giornata, e senza ticket non possono confrontarla con le altre. Il relatore è totalmente a favore: "certo che ti apro un ticket, altrimenti tu come fai a tracciare il tuo lavoro?".

0.2.3.4 La checklist: l'esempio dell'aviazione

La **checklist** non è la task list. La task list contiene le cose da fare; la checklist contiene **i passi da seguire quando si esegue un task specifico**. L'esempio è quello delle **checklist pre-volo**, introdotte nell'aviazione (secondo il relatore tra gli anni Sessanta e Settanta): chi vola spesso nota che i piloti seguono sempre un ordine simile nei controlli, tanto che il relatore capisce quanto manca al decollo da quando controllano i flap, i motori, quando accendono e spengono le luci. L'introduzione delle checklist avrebbe ridotto di circa il 75% gli incidenti in fase di decollo[7], e soprattutto per i **piloti più senior**. Perché? Perché chi ha fatto un decollo centomila volte non va a controllare l'alettone destro: ha funzionato centocinquantamila

[6] Nella trascrizione "AnyDo, Windows Memo". Any.do è un'app di liste e promemoria; "Windows Memo" indica probabilmente le note o i promemoria integrati in Windows (Sticky Notes, Microsoft To Do).

[7] Date e percentuale sono quelle riferite in lezione e non sono state verificate. L'origine della checklist pre-volo viene di solito fatta risalire al 1935, dopo l'incidente del prototipo del Boeing B-17, causato da un controllo dimenticato da un equipaggio molto esperto; la diffusione sistematica delle checklist e dei protocolli di cabina è invece successiva.

volte, funzionerà anche stavolta. Quando ci si abitua a fare qualcosa, si sbaglia proprio quando qualcosa non va. La checklist ha obbligato a fare quei controlli ogni volta e ha ridotto in modo drastico gli incidenti.

Il relatore usa checklist anche per cose banali, ad esempio le **email**: essendo una persona abbastanza ansiosa su queste cose, se prepara una mail alla leggera finisce per passare troppo tempo a ricontrollare l'oggetto, il destinatario, l'allegato, e comunque si dimentica qualcosa. Seguire una checklist sembra più complesso, ma **fa risparmiare tempo**: se la checklist è completa, si sa di aver fatto tutto e si può andare avanti senza continuare a pensare "ho chiuso il gas? ho spento la luce?". È come la lista della spesa: se tutto è segnato e preso, punto. È un investimento: trenta secondi in più per fare la cosa, invece di cinque minuti a sperare di averla fatta giusta.

E se ci si dimentica qualcosa nella checklist? Succede. A quel punto *preparare la checklist* è un task in sé: un giorno in cui si ha tempo si fa quel task pensando a quali passi si seguono e a come assicurarsi che funzioni. Nessuna checklist è perfetta: ci saranno sempre compromessi tra "devo davvero controllarlo ogni volta?" e "posso farne a meno?". Però, in generale, **se qualcosa anche solo una volta ha dato problemi, va in checklist**, perché vuol dire che potrebbe darne di nuovo. Nessun manager rimprovererà "come hai fatto a non prevedere che questa cosa poteva andare male?": si diventa esperti anche nelle checklist di un task facendo quel task, ed è un processo iterativo. Se qualcosa è stato dimenticato e ha creato problemi, lo si aggiunge per la volta dopo. Sbagliare una o due volte non è un problema; l'importante è che la situazione migliori la volta successiva.

L'inizio di un junior e l'*onboarding*. A uno studente che osserva che l'inizio di un junior è quindi inevitabilmente un po' travagliato, il relatore risponde che è giusto così: "se volessi il dipendente perfetto da subito, cercherei un senior, non un junior". Allo stesso modo non capisce le aziende che prendono un junior e non lo fanno seguire da nessuno: che cosa ci si aspetta, che assorba passivamente tutto quello che ha intorno e impari "per osmosi"? Può succedere, ma imparerà a fare le cose come vuole lui, perché nessuno gli sta dicendo "meglio farle così". Moltissime aziende, oltre al manager (che magari ha quindici *report* e non ha il tempo materiale di seguire un junior come andrebbe seguito), assegnano un **referente**, di solito un senior, con il compito di prendersi cura della persona nuova e insegnarle a fare le cose. Altrimenti l'**onboarding**, cioè il tempo necessario per arrivare al cento per cento nel proprio lavoro, dura un anno. Più risorse l'azienda dà all'inizio, prima la persona diventa operativa: è nell'interesse dell'azienda dare il massimo supporto a chi arriva.

Il relatore diffida delle aziende che "vogliono vedere se uno se la cava". È il discorso dei cento metri e della corsa a ostacoli[8]: non si può far correre uno sui cento metri e l'altro con gli ostacoli e aspettarsi tempi simili, perché chi ha gli ostacoli ci metterà per forza di più. Il supporto ai junior è un investimento sensato.

0.2.3.5 *First contact resolution*

Un concetto che viene dai call center: **quante richieste vengono risolte al primo contatto**. È importante quando un task richiede iterazioni o comunicazione con altri: bisogna organizzarsi come se si dovesse risolvere la cosa *in una sola passata*. Se viene chiesto un modello 3D, lo si fa giusto la prima volta, al meglio delle proprie possibilità. Se si deve parlare con qualcuno, si raccolgono tutte le richieste in una.

L'esempio è quello dei ticket che il relatore mandava ai colleghi del supporto di Blizzard: "ciao, ho un problema, non riesco a loggarmi" — "mi dici lo username?" — "è questo" — "mi dici l'email?" — "è questa" — e così via, con un paio d'ore tra un messaggio e l'altro perché il ticket deve fare il suo giro. Tutto poteva essere riassunto in "ciao, mi servono username, email e questo e quest'altro". Unificare le richieste su una singola cosa aiuta a risolverla il prima possibile; bisogna avere chiaro *che cosa serve prima di fare una richiesta*, in modo che ce ne sia una sola. In molti sistemi di supporto, se si apre un secondo ticket per aggiungere informazioni al primo, il primo, anche se era quasi in lavorazione, viene rimandato in fondo alla coda: serve a disincentivare chi manda continuamente ticket del tipo "mi sono ricordato questa cosa". O si manda tutto subito, o ci vorrà tempo.

[8] Il relatore lo introduce come un discorso già fatto ("si torna al discorso: vuoi fare i cento metri o la corsa a ostacoli?"), ma il passaggio non compare nella trascrizione della prima giornata riportata nel capitolo 0.1.

0.2.3.6 Non iniziare senza le informazioni

Corollario: **se non si ha tutto quello che serve per completare un task, non lo si inizia**. Prima ci si procura quello che manca. Al relatore capita con la pianificazione dei due anni di lavoro successivi: se non ha tutte le informazioni, prima lavora per averle ("mi dai questa informazione, mi dai quell'altra"), poi pianifica; altrimenti pianificherebbe qualcosa di cui non ha completa visibilità.

0.2.3.7 Il costo delle interruzioni

Alternare continuamente l'attenzione tra i task e, soprattutto, reagire di continuo agli input esterni (la mail, il messaggio, il collega che parla) fa **sparire il tempo dalla giornata**. Se mentre si prepara un report in Excel si viene interrotti (dal gatto, da una mail, da un collega, da una telefonata) e poi si torna al foglio, servono in media **un quarto d'ora** per tornare mentalmente al cento per cento; chi è particolarmente bravo ci mette dieci minuti, ma si parla comunque di dieci-quindici minuti[9]. Se si viene interrotti di continuo, si passa più tempo a cercare di tornare al cento per cento che a lavorare. È una partita persa in partenza.

Per questo il giorno prima si era parlato dei **semafori** sulla scrivania (sezione 0.1.13): il semaforo può essere anche semplicemente il proprio stato su Slack, su Discord o su qualunque strumento si usi con i colleghi. Se si è occupati, si è occupati: è una propria valutazione che, accettando di essere disturbati di continuo, quella cosa non si riuscirà a farla, o almeno non in tempo.

0.2.4 Prioritizzare: la matrice di Eisenhower

Ora che si hanno task list, checklist, *must do* e *nice to do*, resta la domanda di prima: se arriva un task nuovo e si decide di non fare per prime le cose nuove, *come si capisce se è davvero urgente*? Serve un metodo, ed è la **matrice di Eisenhower**[10] (la classe prova a indovinarne il nome proponendo altre matrici, come una matrice dei rischi, ma è un'altra cosa). Ogni task che arriva va posizionato su questa tabella:

	Urgente	**Non urgente**
Importante	**Fallo** (*do*): diventa un task di oggi.	**Pianificalo** (*schedule*): va fatto, ma non subito.
Non importante	**Delegalo** (*delegate*): va fatto, ma non serve che lo faccia tu.	**Eliminalo** (*delete*): non lo fai.

Urgente e importante: lo fai. Sono di solito task con una **deadline molto chiara** (entro oggi, entro un'ora, entro X) e con **conseguenze** se non vengono fatti: da qui l'"importante".

Importante ma non urgente: lo pianifichi. Va fatto, ma non subito: o lo si mette tra le cose di oggi "ma dopo", o anche alla settimana successiva. L'esempio: un collaboratore scrive "il mio rinnovo è tra tre mesi, ne possiamo parlare?". È importante, perché il rinnovo di una persona è importante; non è urgente, se ne può parlare anche domani, così da potersi preparare e fare altre cose nel frattempo.

Urgente ma non importante: lo deleghi. Qui "non importante" **non** vuol dire che il task non sia importante, ma che **non è importante che lo faccia tu**, che è molto diverso. Se arriva "mi serve questo cambio a questa animazione 3D" e ciò che conta è il cambio, non chi lo fa, si può dire "sono d'accordo che è urgente, ma non serve che lo faccia io" e chiedere a qualcun altro. Sono task che vanno fatti ma non richiedono la propria particolare *expertise*.

Il relatore insiste molto su questo punto con manager e senior: quando arriva un task urgente, i senior tendono a dire "lo faccio io, perché è urgente". Ma se un junior può farlo, lo si fa fare al junior e si investono le competenze più alte del senior in qualcos'altro: il senior può fare il lavoro del junior, ma il junior non può

[9] Ordini di grandezza come riportati in lezione. Le ricerche più citate sul tema (ad esempio quelle di Gloria Mark sulle interruzioni nel lavoro d'ufficio) indicano tempi medi di ripresa anche superiori, intorno ai venti minuti.

[10] Nella trascrizione anche "Jason Howard". Dal nome del presidente statunitense Dwight D. Eisenhower, a cui si attribuisce la distinzione tra urgente e importante.

fare il lavoro del senior mentre il senior fa quello del junior. Le persone non sono intercambiabili. Si usa il **minimo sforzo necessario**: se una cosa la può fare un mid, la fa il mid, non il senior.

Né urgente né importante: lo elimini. È il quadrante che quasi nessuno usa: tutti fanno, delegano o pianificano, ma non pensano mai a *delete*. Invece è fondamentale. A volte è corretto dire: "questa cosa non dovrei farla io, non è urgente, non la faccio; sappi che non la faccio, e se c'è un problema ne parli con il manager". Non si sta delegando a qualcun altro: si sta dicendo che quella cosa così com'è **non andrebbe fatta**, perché non ne sono chiari l'importanza e il perché. Nella propria lista di oggi non ha spazio, e non è abbastanza importante per essere fatta da sé nemmeno nei giorni successivi.

Un esempio di *delete* che non è un vero *delete*: arriva una richiesta di meeting di tre ore sul futuro del progetto, con invitati tutti gli art director, il game director e così via. Il relatore spesso risponde "io non vengo": per lui è *delete*, ma il meeting si fa lo stesso. Uno studente osserva che a volte si tratta di una richiesta mal formulata o poco chiara: è esattamente così. "Nella sua forma per me è no; se poi mi spieghi meglio perché dovrei farlo io o perché è urgente, allora cambia."

0.2.5 Proteggere il proprio tempo

0.2.5.1 Evitare gli sprechi: un limite che nessuno può insegnare

Evitare gli sprechi di tempo è difficile, soprattutto da remoto: è una sfida costante con se stessi. Ha molto a che fare con il carattere, il *self-control* e ("passatemi anche questa") la professionalità. Il relatore può dare tutti i consigli del mondo, ma se dentro di sé non si percepisce l'urgenza di fare il proprio lavoro quando lo si deve fare, non lo si può insegnare. Se si è a casa e invece di lavorare si gioca a qualcosa su Steam, nessuno deve spiegare che non si dovrebbe. Ci sono limiti oltre i quali non si può andare.

Quello che si può fare è sapere che **ogni interruzione ha un costo sommerso**: ogni messaggio, domanda, telefonata, ogni cambio di focus. Sono i quindici minuti di prima, che possono arrivare a trenta a seconda del tipo di task.

È più difficile tornare su un task logico o creativo? Il relatore lo chiede alla classe: fa più fatica un artista a tornare sul suo modello 3D o un *data analyst* a tornare sul suo foglio Excel? Le risposte si dividono ("creativo", "entrambi"), ed è proprio il punto: dipende a chi lo si chiede. I creativi rispondono "creativo", gli analisti "analitico", perché ognuno ha una chiara comprensione di che cosa serve per essere davvero concentrati su ciò che fa. Il creativo sa che per essere pienamente creativo gli servono l'ambiente, la concentrazione, gli spunti, gli esempi da tenere chiari in testa; l'analista deve avere in testa i conti, i fogli di dati e così via. Sono **ugualmente difficili**, ma solo chi fa quella cosa ne apprezza fino in fondo la difficoltà. Si può avere un assaggio dell'altra parte, ma la piena comprensione arriva solo facendo: è come guardare uno sport in tv e pensare "potrei farlo anch'io", finché non ci si trova lì.

0.2.5.2 Distrazioni volontarie e involontarie

Uno studente, che sta leggendo manuali di *design thinking*, chiede come conciliare tutto questo con l'idea che distrarsi, cioè non restare focalizzati, aiuti a trovare un nuovo punto di vista per risolvere un problema. Qual è il giusto equilibrio? Il relatore è **totalmente d'accordo** con quel concetto: sta parlando di **interruzioni non volontarie**. Sa benissimo che dopo cinque ore su un foglio Excel "esplode la testa"; ma se si pianifica un quarto d'ora di pausa e lo si rispetta, si sta investendo sulla propria creatività ed efficienza. Non è un investimento dover rispondere a una chiamata ogni tre minuti.

Anzi, il consiglio è **crearsi interruzioni pianificate**, sotto il proprio controllo: fare pause, fare altro, soprattutto nel *problem solving*. Il relatore stesso, se non trova una soluzione, si ferma, fa un altro task e poi ci torna; ma è una sua scelta. La discriminante è che la pausa sia *scelta* e non imposta: anche se si fanno le pause giuste, se si viene interrotti proprio nel momento di massima concentrazione (raggiunta magari grazie alla pausa) si butta via tutto. Uno studente cita la **tecnica del pomodoro**: è esattamente quello, piccole pause pianificate che aiutano a essere creativi, purché restino pianificate. Se i quindici minuti diventano "vabbè, facciamo mezz'ora" o "lo faccio nel pomeriggio", non è più la tecnica del pomodoro: diventano interruzioni, o un pretesto per non lavorare.

0.2.5.3 Il peggior nemico siamo noi

E chi sono i peggiori nemici in questo? "Noi stessi": siamo noi ad autoconvincerci che venti minuti vanno bene quanto quindici. Le **distrazioni autoinflitte** sono spesso le peggiori. Il relatore porta l'esempio di una sua giornata, che ha annotato perché lo aveva fatto ridere quanto tempo era riuscito a buttare via senza che nessuno glielo imponesse:

- gli piace avere qualcosa in sottofondo mentre lavora ("ha a che fare con l'ADHD sicuramente"[11]), e in genere lo aiuta. Il problema è quando il sottofondo è troppo interessante: la musica va bene, ma se nella playlist di Spotify parte un podcast di Alessandro Barbero sulla guerra civile americana, finisce per ascoltare quello più del lavoro;
- un'abitudine che ha poi perso: se faceva straordinari alle due di notte, da solo, per qualcosa di urgentissimo, apriva comunque Slack a controllare i messaggi, e la mail. "Come se alle due di notte qualcuno mi scrivesse qualcosa di urgente";
- la pausa con un po' di TikTok, dopo la quale si accorgeva di aver dimenticato su quale slide della presentazione stava lavorando.

La regola: **inserire i break è meglio che subirli passivamente**. Un conto è dire "faccio una pausa intenzionalmente", un conto è "mi sono distratto e ho perso due ore di lavoro". Chi ne è particolarmente soggetto non deve demoralizzarsi: è anche un allenamento.

0.2.5.4 Spezzettare i task grandi

Un altro concetto utile: se si fa molta fatica a iniziare i task grossi, perché sembrano infiniti ("più tardi inizio, più tardi inizia questa sofferenza"), **vanno spezzettati**. Un task grosso non è quasi mai un task grosso: è spesso un insieme di task piccoli. Si comincia a farne dei pezzi e si va avanti piano piano, spuntandoli. Aiuta tantissimo anche nel *problem solving*: un problema gigante va scomposto in problemi più piccoli, da risolvere per primi, per poi ricomporre il puzzle.

0.2.5.5 Accettare di dire di no

Tornando alla matrice di Eisenhower: bisogna **accettare di dover dire di no**, perché dire sempre sì porta a problemi ben peggiori di quanto si pensi.

Il collega che diceva sempre sì. A Riot il relatore aveva un collega estremamente disponibile, amichevole e collaborativo: qualsiasi cosa gli si chiedesse, rispondeva "certo, la faccio io" ("mi porti fuori il cane?" — "certo"). Il problema è che era umano: a un certo punto si supera la quantità di cose umanamente possibili in una giornata. In più tendeva a offrirsi volontario per far vedere che era intraprendente e aveva iniziativa, e nei progetti che coordinava il relatore doveva spesso rifiutare la sua candidatura perché era sicuro che non ce l'avrebbe fatta. Il suo dire sempre sì, a un certo punto, lo ha danneggiato: era quello che dice sempre che lo fa lui, ma non è detto che lo faccia. "Ce la fai?" — "Sì, sì, assolutamente" — e poi "scusa, non ce l'ho fatta". "Ok, però ora hai messo nei guai me e il resto del team." Quella disponibilità era più dannosa di un "no, non posso". È lo stesso discorso del giorno prima: dire all'ultimo momento che non ce la si fa è peggio che dirlo a inizio settimana.

Uno studente lo paragona alla *sindrome dello yes man*. Non proprio: lo *yes man* dà sempre ragione all'altro perché è avverso al conflitto e non vuole la discussione del "spiegami perché sei in disaccordo". Il collega, invece, pensava davvero di poter fare molto più di quanto potesse, e rifiutava l'idea di non riuscirci.

"Ci hai pensato bene?" Vale per i progetti e per le deadline. Con i junior soprattutto, quando a "vuoi fare questa cosa?" arriva un sì troppo convinto, il relatore chiede sempre: "ci hai pensato bene? Me lo stai dicendo per farmi contento? Sappi che se mi dici di no non penso peggio di te; penso peggio di te se mi dici di sì e poi non ce la fai. Vuoi rivalutare la risposta, e magari dirmi *non lo so* invece di un sì categorico?".

[11] Nella trascrizione "IDACD". Più avanti il relatore dice di avere l'ADHD "come dicevo ieri"; il capitolo 0.1 non lo riporta, ma riferisce il suo racconto sull'"overthinking" anche nell'ordinare una pizza (sezione 0.1.16).

Che cos'è una sconfitta. Uno studente chiede se davanti alle "sconfitte" non si dovrebbe rimediare. Il relatore chiarisce che per lui la sconfitta non è il task non fatto: quella è una sconfitta *a breve termine*, si troverà un modo di farlo, è un danno "ma chi se ne frega". La vera sconfitta, un danno *a lungo termine*, è aver pensato di riuscirci e non esserci riusciti: se non si risolve il motivo per cui si è detto sì quando era no, lo si ripeterà altre cinque, dieci, quindici volte. Nella testa del collega di Riot c'era sempre una scusante ("c'erano questi imprevisti che mi hanno bloccato"), come quella con cui ci si giustifica le interruzioni. Il relatore gli rispondeva: "se ogni volta c'è un imprevisto, forse c'è un *pattern*: il problema non è che non riesci a fare le cose, è che non prevedi mai gli imprevisti". Dopo sette ritardi su dieci dovrebbe scattare l'ingranaggio mentale ("forse sbaglio io a dire sempre di sì"), ma non è sempre facile farlo scattare, e a volte serve una spinta. È una delle situazioni più frequenti. E la cosa più difficile è che, quando glielo si fa notare, queste persone lo prendono come un "non sei capace". Il messaggio è invece: **"sei bravo, ma sei umano anche tu"**. Non si può fare un numero infinito di cose in un'ora: bisogna accettare i propri limiti "dati dalla carne" e non pensare di essere un cyborg.

Il cliente non ha sempre ragione. Uno studente osserva che vale anche per i project manager e per il principio "il cliente ha sempre ragione". Anzi: il relatore ha lavorato con molti clienti e a volte **il cliente non sa che cosa vuole**, che è molto diverso dall'avere sempre ragione. Soprattutto con i clienti che chiedevano pitch creativi, la sua squadra presentava sì un'opzione con la loro richiesta, ma anche le proprie ("se vuoi raggiungere questo, sarebbe meglio fare così"), e molto spesso il cliente sceglieva quella proposta dal team. Spesso il cliente ha in mente *una soluzione*, ma bisogna capire *quale obiettivo* ha in mente e poi dargli una soluzione: non vuole la soluzione, vuole il *goal*, e se c'è un modo migliore per arrivarci, forse è meglio un'altra soluzione.

0.2.5.6 Dire di no ai meeting

La prima cosa a cui il relatore ha imparato a dire di no sono i **meeting**. Ha passato anni in ambienti in cui il suo calendario era letteralmente solo meeting dalla mattina alla sera: estenuante per lui e inutile per il team. Tra i fattori c'è il **FOMO**: tanti manager vogliono essere in tutti i meeting per non perdersi niente ed essere sempre informati su quello che fa il team. In realtà vanno in *burnout*, passano i pomeriggi nei meeting e finiscono per essere i più distratti, quelli che non ascoltano e che fanno domande inutili solo per far sentire che fanno domande, senza aiutare davvero.

Prima di accettare un invito il relatore si fa una serie di domande, **come una checklist** (e le fa anche agli altri):

1. **Sono davvero necessario?** Servo davvero io, o se mancassi non succederebbe niente? Ai meeting "di cortesia" non va: se pensa di essere necessario lo dice lui, e il fatto che si faccia un meeting importante e urgente senza di lui non danneggia il suo ego. "Non aggiungermi solo perché pensi che serva che io venga aggiunto."

2. **Uno di noi potrebbe partecipare e informare il resto del team?** C'è già qualcuno del suo team? Se sì, non può andare solo lui? Per un meeting informativo (una nuova policy, una nuova procedura da spiegare a tutti i dipartimenti), andarci tutti significa investire tre ore a testa; se ci va uno solo e poi spiega agli altri, si risparmia tempo.

3. **Il riassunto sarà reso disponibile?** Per il relatore è obbligatorio. Per lo stesso motivo per cui la lezione viene registrata: se ci si perde un pezzo lo si può rivedere, se qualcosa non è chiaro lo si può risentire. Ma c'è anche un motivo di *office politics*: se in un meeting si chiede "mi fai questa cosa per venerdì?" e la risposta è sì, non si vuole sentire venerdì "intendevo venerdì prossimo" o "non ero sicuro". Nelle note del meeting c'è scritto che hai detto sì. "Lo so che è un po' paraculo, ma in realtà è così che si lavora": bisogna dare più peso alla parola data, e quindi tenerne traccia. Finché tutto resta orale, niente rimane scritto e tutto è interpretabile. Oggi è anche facile: Zoom, con l'assistente IA, produce in automatico trascrizione e sottotitoli, senza bisogno di qualcuno addetto alle *meeting minutes*. In ogni caso, se in un meeting viene presa una decisione, il relatore vuole che sia **ripetuta per iscritto**: non deve ipotizzare, deve poter andare a rileggere.

4. **Il meeting aggiunge valore rispetto al documento?** Se arriva un invito con allegato un documento di venti pagine, il relatore chiede: "il meeting consiste nel leggermi il documento? Perché altrimenti non vengo". Se non c'è un valore aggiunto, se lo legge da solo e manda conferma di averlo letto. È lo stesso motivo per cui si sconsigliano le presentazioni in cui si leggono le slide; e infatti, osserva, in quel momento lui stesso stava leggendo una slide, ma aggiungendo informazioni. "Se tutta la lezione fosse io che vi leggo le slide, non servirei io: vi manderei le slide."

5. **Per che cos'altro potrei usare questo tempo?** È forse la più importante. Anche ammesso che serva, che serva proprio lui e che il meeting sia importante, in quel momento dovrebbe fare qualcosa di più importante? Se sì, fa quella. Non bisogna avere paura di dire "no, non vengo" anche se si serve, oppure "ho un'altra cosa importante, spostiamo il meeting". Il relatore si è sforzato di essere più **assertivo**: "capisco la richiesta, ma ho qualcosa di più urgente da fare in questo momento".

E le richieste a voce? Uno studente chiede se una richiesta fatta a voce da un senior ("devi fare questo") vada accettata così o fatta mandare per mail o per messaggio. Accettarla a voce ci può stare. Ma se la richiesta è "devi farla in questo modo particolare e preciso", è meglio farsi mandare il modo preciso **per iscritto**: non si deve lasciare a interpretazione *come* fare quella cosa, per non ricadere nel "ah, ma io non la volevo così". Oppure si conferma: "sì, la faccio; va bene se la faccio così, così e così?". Se dice sì, si è a posto.

Non dovrebbero pensarci gli organizzatori? Dalla chat: queste domande non dovrebbero farsele già gli organizzatori dei meeting? Sì, sarebbe meglio che tutti lo facessero, e il relatore stesso se le fa quando invita qualcuno: "è davvero necessario Marco? È davvero necessario Andrea?", e solo se sì li invita. Ma lavora in aziende anche di cinquecento dipendenti, dove non è detto che tutti conoscano o applichino questa logica; per questo la applica per sé, sperando che la applichino tutti. Ha sempre chiarito con tutti che da lui non arriveranno mai inviti di cortesia o dettati dalla gerarchia ("deve esserci anche il mio capo"): "devi esserci tu; se vuoi portarti il capo, portatelo, ma non lo invito solo perché qui si vuole sempre il capo presente: sono sicuro che avrà di meglio da fare".

E i task? Dire di no vale naturalmente anche per i task: si applica la matrice di Eisenhower (sezione 0.2.4) e si decide se fare, delegare, pianificare o cancellare. Non c'è molto da aggiungere.

0.2.5.7 Il multitasking non esiste

"Questo potrà urtare tanti ego", ma **il multitasking non esiste**. Si può essere più bravi o più portati di altri a fare più cose contemporaneamente; la realtà è che, concentrandosi su più cose, **la somma non farà mai cento**. Se si lavora su due cose non si lavora al 50% su una e al 50% sull'altra, ma al 40% e al 40%, e il 20% va perso in "costo di manutenzione", cioè nella capacità di seguirle entrambe. Il consiglio è dedicarsi al cento per cento a un task, poi passare al successivo, poi al successivo, invece di farne cinque insieme: si può avere la percezione di andare più veloci, ma non è così.

Uno studente cita una lettura di neuroscienze secondo cui il cervello è in grado di fare una sola operazione alla volta. Il relatore conferma: smettiamola di parlarne come di un pregio o di una *skill*. È un modo che alcuni cervelli trovano per svagarsi, l'equivalente di accendere un podcast: fare due cose insieme vuol dire cercare di distrarsi alternandole, e farle entrambe peggio di quanto si farebbero una alla volta. A chi preferisce fare tante cose insieme suggerisce di provare a farne una alla volta: le farà più velocemente. Anche il semplice passaggio da una cosa all'altra non è istantaneo: ci sono sempre una decina di secondi in cui il cervello si chiede "che cosa stavo facendo?"; per cinquanta cambi, già questo spiega la perdita di velocità. È una questione di **bandwidth mentale**, un concetto che tornerà nel pomeriggio: anche lo *switch* occupa banda.

0.2.5.8 La fine della giornata

A fine giornata si **rimette a fuoco la giornata**: si riguardano la task list e la *must do list* e si conferma di aver fatto tutto. È letteralmente il controllo prima di uscire di casa: chiavi, porta chiusa, cellulare, portafoglio. Al relatore è capitato di dimenticare un *must do* rimasto in lista per tutto il giorno, ed entra

in gioco quella che chiama la **sindrome del tirannosauro**[12]: se una cosa è lì, non è detto che la si veda finché non si muove. Averla in lista tutto il giorno non vuol dire averla percepita come da fare.

Può anche capitare di non farcela perché quel giorno sono arrivate così tante cose urgenti e importanti da rendere inutili le stime: la giornata no capita e non c'è nessun problema. Ma se era davvero un *must do*, **va fatto comunque**: è lì che scattano la richiesta di straordinario o il "devo farlo comunque". Attenzione: se si chiude la giornata con un *must do* ancora aperto e si decide "vabbè, lo faccio domani", può starci, ma allora **non doveva essere in quella lista**, perché era rimandabile.

Al relatore questo controllo serve anche "per quieto vivere": se chiude ogni giornata con la *must do list* completa, sa che magari non ha fatto tutto quello che c'era nella to-do list, ma i *must do* sì. "Il mio, oggi, l'ho fatto": può stare tranquillo senza continuare a chiedersi se ha fatto quello che doveva.

0.2.6 Domande sul time management

C'è un numero medio "sano" di *must do* quotidiani? No, perché un conto è avere dieci *must do* come "rispondere a dieci mail", un conto è averne uno solo come "inventarmi un nuovo gioco". Dipende dalla dimensione: se ne gestiscono di più se sono minuscoli, di meno se sono giganteschi. Ci sono lavori che per definizione hanno tantissimi *must do* quotidiani e lavori che ne hanno uno al giorno. Si imparerà da sé qual è il numero giusto.

C'è un modo per stimare quanto si riesce a fare in una giornata? "Lo so che non è una bella risposta", ma l'unico modo è la propria **storia**: sapere quanto si riesce a fare nel quotidiano. È normale e fisiologico che le prime stime siano sbagliate, e probabilmente eccessivamente ottimistiche; l'importante è rendersene conto. È letteralmente la *seniority*: abituarsi a fare qualcosa insegna quanto ci si mette. Va benissimo metterci di più all'inizio, ed è normale che il tempo cambi man mano che si migliora: ciò che oggi si stima in un'ora, tra sei mesi lo si stimerà in mezz'ora. Chi ha lavorato con Scrum lo sa: su un progetto nuovo tutti sbagliano la dimensione delle *stories* all'inizio, perché nessuno sa quanto ci vorrà, e al terzo *sprint* consecutivo è più chiaro. Le stime sono stime, altrimenti si chiamerebbero "inserire quanto tempo ci metterai".

Se però si dà una stima e poi il lavoro si rivela il triplo, il problema non è la stima: è che **bisognava chiedere più informazioni** prima di darla. La risposta giusta non è la stima, ma "per darti una stima accurata mi serve sapere questo, questo e questo". Il problema non è sbagliare la stima, è non darsi i mezzi per azzeccarla.

Come si fa a far entrare questi metodi nella routine? È solo determinazione? È un misto. Il relatore risponde da persona con ADHD, con la tendenza a distrarsi e a fare un mucchio di *overthinking* ("come dicevo ieri, anche ordinare la pizza è overthinking"). È un po' disciplina, ma soprattutto far diventare qualcosa un'**abitudine**: lo sforzo per fare le checklist era molto più alto un anno prima, oggi è un'abitudine. Di solito servono **circa tre mesi** perché qualcosa diventi un'abitudine[13]: la teoria dice che chi inizia oggi ad andare in palestra ogni giorno, dopo tre mesi non penserà più "oggi devo andare in palestra", ma la palestra farà parte della sua giornata. Non è più uno sforzo, ma "è così che lavoro".

Chi fa davvero time management? Prima di passare al brainstorming il relatore torna alla domanda iniziale: qualcuno, ora che sa che cosa intendeva, cambierebbe risposta? Uno studente risponde che lo fa, ma senza checklist, perché non ne sente il bisogno (non gli capita spesso di dimenticare le cose); la *must do list* invece la segue sempre, e le cose meno importanti capitano nelle giornate in cui ne salta qualcuna. Va benissimo: non esiste un unico modo, ognuno troverà il proprio. Il relatore porta il suo esempio perché sa che è servito a molte persone con cui ha lavorato; con chi faceva fatica ad applicare un metodo (come chiedeva prima uno studente) ne hanno trovati altri, perché ciò che funziona per uno non è detto che funzioni per un altro. L'importante è che **ognuno trovi il proprio metodo**, perché la soluzione non è non fare time management: ci si sentirà più "liberi" improvvisando la giornata, ma non la si sta sfruttando a dovere.

[12] Il riferimento è alla celebre scena di *Jurassic Park*, in cui si dice che il tirannosauro vede la preda solo se si muove.

[13] Stima come riportata in lezione. Gli studi più citati sulla formazione delle abitudini indicano tempi molto variabili da persona a persona, con una mediana intorno ai due mesi.

E non si parla dei video sul *mindset* in cui ci si alza alle cinque, si fa la checklist e si fa colazione leggendo le mail: organizzarsi meglio la giornata **dà più tempo libero**. E per il relatore il tempo libero delle persone è fondamentale, perché è ciò che permette loro di lavorare meglio.

0.2.7 Una parentesi: tempo libero, ferie e benessere

0.2.7.1 Le tre condizioni del relatore

Il relatore è un sostenitore della **settimana lavorativa di quattro giorni**, che ha applicato in varie aziende; dove non ci è riuscito, ha introdotto almeno **venerdì liberi**, due volte al mese o una volta al mese, cioè giornate in cui non si fa niente. Come le pause brevi sono importanti per essere creativi, anche le pause lunghe, cioè giornate intere, lo sono per essere creativi ed efficienti. È lo stesso motivo per cui, dopo la Games Week, dava una settimana libera alle persone (sezione 0.1.4): ha bisogno che siano riposate, e quella settimana "persa" è in realtà un investimento per le settimane successive. In chat qualcuno lo paragona al "metodo finlandese": è un misto tra quello e il metodo del buonsenso.

Il relatore viene spesso chiamato a **cambiare le culture aziendali**, ed è un po' il suo focus. Come un allenatore che accetta una squadra solo se può portare il suo staff (il preparatore, il tecnico), lui chiede tre elementi:

1. possibilmente **quattro giorni lavorativi**, o altrimenti giornate libere per dare spazio alle persone;
2. **ferie illimitate**: nessun limite massimo, ma un **limite minimo garantito**. Ferie illimitate non devono voler dire poter scegliere di non prenderle: "devi prendere almeno X giorni, fino a infinito";
3. la **feedback culture**.

Senza questi tre elementi non garantisce il risultato: è un modo di gestire le aspettative fin dall'inizio, come si diceva il giorno prima.

La feedback culture. Il perché è stato spiegato nella prima giornata, ma il relatore lo sottolinea: un ambiente in cui ci si può fidare del team vale moltissimo. Quando va a fare lezione allo IULM, anche in settimana, non deve preoccuparsi di che cosa stia facendo il team mentre lui non lavora ("oggi è domenica e non ho questa preoccupazione a prescindere, ma anche se fosse mercoledì non l'avrei"): sa che stanno facendo del loro meglio, e non gli interessa sapere altro.

Ferie e permessi illimitati. Nell'esperienza del relatore, sia a Riot sia a Blizzard, avere ferie illimitate significava spesso *prenderne meno* di quelle obbligatorie: c'è una vocina che dice "ti serve davvero un giorno di ferie oggi? Tanto sono illimitate, te le prenderai quando ti serviranno", e sapere di averne illimitate spingeva a lavorare di più. Per lui era altrettanto dannoso, e per questo ha introdotto un **minimo obbligatorio di 24 giorni** di ferie.

C'è stato però un effetto non previsto, e molto desiderato: in tutte le aziende in cui ha introdotto ferie e permessi illimitati, **i giorni di malattia sono calati del 70%**[14]. La gente si ammala di meno? O piuttosto il giorno in cui ci si sveglia e si pensa "oggi non ne ho voglia, mi do malato" diventa il giorno in cui si prende un permesso? Si smette di usare come malattia un giorno che non era di malattia.

L'*Hangover Day*. Sia a Riot sia a PG Esports il relatore aveva introdotto quello che a Riot, "essendo Riot a Dublino", si chiamava **Hangover Day**[15]: una giornata all'anno in cui si aveva il diritto di telefonare alle nove del mattino e dire "oggi non vengo perché non ne ho voglia" (tradotto: perché ieri sera ho bevuto troppo). Altrimenti quella giornata sarebbe diventata un giorno di malattia o di ferie. Il fatto è che **quasi nessuno lo usava**: il relatore stesso a volte ci ha pensato, ma poi la mattina si chiedeva "voglio davvero giocarmelo oggi? Tutto sommato potrei anche andare", e ci andava. Ma anche solo sapere di averlo lo faceva stare tranquillo. Allo stesso modo, con le ferie illimitate, chi voleva prendersi un mese lo faceva ("buon per loro"), chi non lo faceva andava bene lo stesso: l'importante era che staccassero. C'era un **benessere più diffuso** nel team, perché sapevano che, volendo, avrebbero potuto.

[14] Dato riferito dal relatore sulla base della propria esperienza, non verificabile.
[15] Nella trascrizione "Angover Day".

Il *Neko Day*. Il primo punto, i giorni liberi, era chiamato **Neko Day**, da *neko*, "gatto" in giapponese: il giorno in cui si può fare il gatto, rannicchiati davanti al camino a non fare niente. Perché introdurlo? Chi lavora dal lunedì al venerdì passa di solito il sabato o la domenica a fare le commissioni che non può fare nei giorni lavorativi: la spesa, le bollette, il bollo dell'auto. In questo modo sfrutta meno il weekend. Con un venerdì libero al mese (o due, o tre) le persone facevano quel giorno le commissioni del weekend, e sabato e domenica si riposavano davvero ("anche non fare niente andava bene, chiaramente"). Il lunedì arrivavano molto più cariche e riposate. Questo ha aiutato tantissimo a mantenere un team sano, fisicamente e mentalmente.

Il supporto psicologico. A PG Esports il relatore ha introdotto anche il **supporto psicoterapeutico offerto dall'azienda**: sei sedute da un'ora con uno psicoterapeuta, online. Era il periodo in cui si parlava del bonus psicologo statale, "che poi non è mai arrivato"[16], e lui ha pensato: l'idea è buona, perché non introdurla a livello aziendale? Erano gli anni dei lockdown: l'azienda stava perdendo molte persone che andavano in depressione o non avevano più voglia di lavorare, perché avevano difficoltà a vivere quel periodo ("io stesso non è che l'abbia vissuto benissimo"). Qualcuno ne ha usufruito e gli ha detto: "mi stavo per dimettere, poi ho risolto le cose e ho preferito restare".

Secondo il relatore oggi c'è una consapevolezza maggiore da parte delle aziende sulla **salute mentale**: vuol dire dare più ferie, dare questo tipo di supporto, dare più *benefit* (anche i buoni pasto). Il benessere non è solo economico, è anche mentale, ed è una cosa positiva da sapere.

0.2.8 Brainstorming: obiettivo e problemi tipici

0.2.8.1 Perché il brainstorming riguarda tutti

Secondo il relatore questa è la parte che servirà di più, sia nei gruppi e nei progetti del percorso di studi (il gioco da tavolo, il gioco digitale) sia nelle aziende. Rispetto al time management ha una pertinenza più immediata: tutti hanno esempi di **brainstorming andati male**, da cui si è usciti esausti, con la sensazione di non aver ottenuto niente, o di non aver parlato abbastanza ("ha parlato solo quello"), o di non essere sicuri della scelta. Il brainstorming lascia una sensazione molto chiara, a fine meeting, che qualcosa non torna; il time management dà piuttosto la tristezza del "non ce l'ho fatta a fare quello che dovevo oggi, come mai?".

0.2.8.2 Generare idee, non decidere

Il primo chiarimento riguarda **l'obiettivo**, che il relatore vede spesso travisato. L'obiettivo del brainstorming è **generare idee**; invece diventa spesso decisionale, con lo scopo di scegliere che cosa fare. Ma allora non sarebbe un brainstorming, sarebbe un *decision making meeting*. Generare molte idee e scegliere la migliore nello stesso meeting non funziona né per generare idee né per scegliere la migliore, perché sono **due metodi che si contrappongono**. Il brainstorming va fatto quando si devono generare idee *per poi* scegliere che cosa fare: deve venire **prima** del *decision making*, come primo passo della fase di esplorazione di un nuovo progetto.

0.2.8.3 "Non esistono cattive idee"?

Bisogna evitare che i membri del team **autofiltrino** le proprie idee per paura di essere rifiutati o presi alla leggera. Da qui nasce la frase "non esistono cattive idee, buttate fuori tutto", che è vera solo fino a un certo punto. Le cattive idee esistono, oggettivamente; quello che non si deve fare è *colpevolizzarle*. Né si deve per forza perseguire ogni opzione: parlare di opzioni che sono effettivamente brutte è uno spreco di tempo, e il brainstorming ha anche un problema di gestione del tempo. Il relatore vede spesso brainstorming gonfiati a ore e ore in cui si parla anche di cose che non meritano nemmeno dieci minuti, come i feedback buoni ma inutili del giorno prima. **Trattare tutte le idee allo stesso modo è sbagliato**, perché non hanno la stessa qualità. Bisogna trovare un modo di valutarle senza che chi le ha proposte si senta preso di mira, bullizzato o sminuito.

[16] Così in lezione. In realtà il bonus psicologo è stato istituito in Italia nel 2022, ma con fondi limitati rispetto alle richieste e con forti ritardi nell'erogazione; il relatore si riferisce probabilmente a questo.

Per questo a ogni brainstorming dovrebbe seguire un **follow-up**, il momento in cui si decide che cosa fare: le due parti vanno separate, altrimenti "si crea solo casino". Il vero lavoro arriva quando le idee vengono messe a terra: c'è la parte di brainstorming e di *theorycrafting*, e la parte esecutiva viene dopo.

0.2.8.4 I problemi più comuni

Le prime idee monopolizzano la sessione Le prime due idee "sparate a caso" sono quelle su cui ci si concentra per la maggior parte del tempo, sviscerandone i dettagli ("intendevi così o così?"), per poi chiudere con "altre idee? Questa, questa e questa? Sì, no, ok, andiamo avanti". Si parla troppo delle prime idee e poco delle ultime.

Poche persone parlano per la maggior parte del tempo È un effetto collegato al precedente.

Anchoring Ci si lega alle prime idee, e spesso si votano o si scelgono non perché siano le migliori, ma perché sembra di conoscerle meglio di quelle menzionate in trenta secondi alla fine. Non c'è nessun motivo per pensare che un'idea detta alla fine sia peggiore: era solo arrivata dopo. C'è un problema di *timing*. Anzi, un'idea venuta istantaneamente è forse quella meno ragionata, perché ci si è pensato meno tempo.

Silenzi imbarazzanti Spesso le persone arrivano al meeting senza nemmeno sapere che è un brainstorming. Quando il relatore convoca un brainstorming, comunica prima a tutti il *topic*, il tipo di idee cercate, i paletti e le restrizioni, così che anche chi è meno portato all'improvvisazione arrivi con qualche idea pronta. Se la gente arriva impreparata rischia di non dire niente per un'ora, e allora perché è stata invitata? Non è un difetto loro: tanti manager dicevano al relatore "ci sono persone inadatte ai brainstorming perché non sanno improvvisare". No: si sta facendo brainstorming con un metodo che premia solo chi sa improvvisare ed è abbastanza estroverso da parlare sul momento. Non significa che chi non parla non avrebbe potuto contribuire, ma che non gli è stato dato modo di farlo.

Estroversi e pensatori rapidi dominano Le personalità estroverse e chi pensa velocemente tendono a dominare la conversazione, più o meno intenzionalmente, senza lasciare agli altri il tempo di contribuire. Non c'è uno spazio in cui un introverso possa dire "veramente io volevo...": "no, no, stiamo parlando di questa bellissima idea, facciamo questa".

0.2.9 Un metodo di brainstorming

Il relatore propone una soluzione principale, declinata in vari modi, che tocca tutti questi punti.

0.2.9.1 Preparazione e ambiente inclusivo

Arrivare con un obiettivo chiaro. Bisogna arrivare al brainstorming con chiaro e preciso che cosa si sta cercando, cioè **che cosa si sta cercando di risolvere**, **perché** e soprattutto **con quali vincoli**. Si stanno solo raccogliendo idee per un cliente? Si cercano idee fattibili, o idee innovative? Che cosa si vuole produrre da questo meeting? Deve essere chiaro a tutti *prima* del meeting, non spiegato durante, perché altrimenti si ricade nel problema delle persone non pronte a intervenire.

Non giudicare le idee degli altri. Al relatore è capitato spessissimo di sentire un'idea terribile da una persona e, mezz'ora dopo, l'idea perfetta dalla stessa persona. Ma se all'idea terribile tutti rispondono "che stupidaggine, per favore stai zitto", quella persona non dirà la successiva, perché non vuole rivivere l'esperienza. Giudicando le idee degli altri, di fatto, li si zittisce. Le idee migliori nascono quando si propone quella che si pensa sia l'idea perfetta, magari non è buona e non viene scelta ("fa niente"), si ascoltano anche le altre e poi la si **riformula** integrando i pareri sentiti nel frattempo. Sono queste **idee combinate** a essere spesso le soluzioni migliori.

0.2.9.2 Separare generazione e discussione: *brainwriting* e *rapid ideation*

Con il classico "non ci sono cattive idee, sparate", succede spesso che uno prenda la parola, spieghi la sua idea e il suo concetto, e gli altri comincino subito a fargli domande ("ma quindi la vedi così?"): si sta unendo la persona che *genera* l'idea con la persona che la *discute*, nello stesso momento. Invece bisogna **dividere la generazione dalla discussione**: un momento in cui si generano idee e un momento in cui si discutono. È il principio del **brainwriting**. Funziona molto quando nel team c'è una figura con una posizione gerarchica più alta: se quella persona esprime un'idea e i due momenti sono uniti, tutti tenderanno ad allinearsi. E si vuole che *tutti* possano avere idee.

Lo strumento è la **rapid ideation**. Il relatore di solito dice: "per i primi cinque minuti non parliamo; scrivete su dei post-it il pitch delle vostre idee". Se ne possono scrivere quanti se ne vuole, dieci o uno, e si mettono da parte. Ma il tempo è di **cinque minuti**: non dieci, non due, e soprattutto non mezz'ora. Il **limite di tempo breve** è importantissimo, perché costringe, consapevolmente o meno, a filtrare le idee mentalmente e a scrivere solo quelle che si ritengono migliori: magari non è chiaro perché si preferisca un'idea all'altra, ma il cervello sta facendo il lavoro in background e fa venire in mente le cose più importanti o più giuste. È ottimo anche per i team che tendono a distrarsi, perché sono solo cinque minuti.

0.2.9.3 *Dot voting*

Poi si passa al **dot voting**:

1. si attaccano **tutte** le idee su una lavagna fisica, un muro o una lavagna digitale (come Miro[17]);

2. si **uniscono le idee che hanno lo stesso concetto di base**. Se si sta pensando a un evento dal vivo e la proposta del relatore è un parco di dinosauri, mentre quella di un collega è un parco di dinosauri "del paleolitico" con i tirannosauri, le si raggruppa sotto "parco di dinosauri": non sono due idee separate. Un altro gruppo potrebbe essere, ad esempio, un parco a tema Avengers;

3. ognuno ha **da tre a cinque voti** e va alla lavagna a mettere i propri pallini sulle idee che gli piacciono e di cui vorrebbe sapere di più.

Il risultato tipico: un'idea con molti voti, una con un paio, altre con un numero discreto.

Che cosa si vede subito? Il relatore lo chiede alla classe. Si vede una **preferenza** per un tema o per un'idea. Non c'è **polarizzazione** su una o due idee iniziali, perché c'è varietà. Il voto è **anonimo**: non ci sono nomi. E soprattutto manca un'altra cosa importantissima: **il giudizio negativo espresso**. Se c'è un'idea con sette voti, una con cinque e una con tre, è ragionevole parlare prima di quelle, perché tutti hanno espresso interesse e vogliono saperne di più; ma nessuno sta dicendo "non voglio assolutamente parlare di questa perché mi fa schifo". È semplicemente oggettivo che quell'idea non ha ricevuto voti.

Autovotarsi. Il sistema permette anche di votare la propria idea, e non c'è nessun problema: se la propria idea ha un pallino e gli altri ne mettono sette su un'altra, il proprio voto è ininfluente. Il relatore vedeva spesso che all'inizio la gente tendeva ad autovotarsi, poi capiva che era inutile, perché non sarebbe mai stato il voto decisivo, e tanto valeva votare le idee che interessavano davvero. La tendenza spariva con il tempo.

Che cosa si è ottenuto. Tre ore di meeting sono state filtrate in una discussione concentrata solo sulle idee che tutti ritengono interessanti, con un *agreement* di fondo: non si parla dell'idea numero uno perché è uscita per prima, non è una questione di *timing* ma di qualità. Il relatore ha vissuto tanti meeting con la frustrazione interna del "stiamo sprecando tempo su questa idea, quando vorrei dire la mia che secondo me è meglio". Qui non si può sprecare tempo, perché si parla di qualcosa che tutti vogliono sentire.

[17] Nella trascrizione "mirror board": si tratta della lavagna digitale Miro, già citata nella prima giornata.

E se si formano sottogruppi che si votano a vicenda? Uno studente chiede se non possano esistere "cordate". Anche se esistessero, significherebbe che il gruppo di quel meeting vuole parlare di quella cosa, e va bene: l'importante è non sprecare tempo su idee che non interessano a nessuno. Se si investono due ore, quarantacinque minuti su ciascuna di tre idee vanno benissimo; dieci minuti per ogni idea no. E comunque il voto è anonimo: non si può sapere che un'idea è di Gianni per dare supporto a Gianni, quindi un eventuale sostegno reciproco sarebbe involontario.

E chi è lento a elaborare? Uno studente osserva che il problema degli introversi potrebbe ripresentarsi con i post-it: con un tempo così ristretto, chi è più lento potrebbe andare ancora più in panico. Per questo bisogna **arrivare preparati**: si conosce il topic in anticipo, quindi almeno un paio di idee già formalizzate si hanno in mente. Magari non se ne scriveranno quindici, ma un paio sì.

0.2.9.4 Quanti voti e quali varianti

Dare più pallini a testa sarebbe controproducente? Il relatore indicava da tre a cinque voti. Chiede alla classe quanti siano i gruppi di lavoro: dai quattro agli otto, alcuni gruppi del secondo anno sono nove. La sua prassi:

- **dai cinque partecipanti in su**, **tre voti** a testa, più che sufficienti per popolare molto la lavagna;
- **fino a cinque partecipanti**, **cinque voti** a testa, che permettono anche di variare un po'.

Esistono varianti: ad esempio tre voti di colori diversi, il blu per l'idea più innovativa, il verde per quella più bella, il giallo per quella più facile, così da qualificare *perché* un'idea piace. Se l'obiettivo è proporre a un cliente tre pitch (uno facile, uno medio, uno avanzato), la votazione è già fatta. All'inizio, però, meglio partire da tre o cinque voti generici; con il tempo si imparerà ad accelerare.

0.2.9.5 Il moderatore e gli invitati

Serve un moderatore? Può essere utile. Il relatore fa spessissimo il moderatore: partecipa ai brainstorming senza fare altro che moderare, "perché so che le persone con le idee fighe sono loro e non io". Ci si può anche autogestire. Nei team piccoli non ha senso "perdere" una persona per la moderazione; nei team più grandi (dai cinque in su, sette-otto persone), se qualcuno dice "non penso di avere idee particolari su questo, vi modero così vi aiuto", è un ottimo modo di contribuire.

Chi invitare? Dipende da che cosa si cerca. Per inventare un nuovo gioco il relatore invita i game director, gli art director e magari il *publishing manager* per un input commerciale; per decidere il *combat system* invita magari il senior game designer e il game director. La domanda che si fa chi convoca il meeting è: dato l'obiettivo, chi è più ragionevole che possa contribuire?

Un 3D artist della classe chiede se allora gli artisti 3D non vengano inclusi. Dipende: se l'obiettivo è definire la linea creativa dello stile dei personaggi, il relatore invita anche il 3D artist, e non solo i *concept artist*. Il 3D artist può dare un input diverso dal concept: vota un'idea perché per lui è *più fattibile*. Un concept può essere bellissimo ma "un inferno" da modellare, e chi sa di doverlo fare entro giugno non vota un'idea che sa già infattibile.

In realtà si può invitare chiunque: per un pitch su un'area dedicata al basket alla Games Week, il relatore ha invitato un tecnico di regia che gioca a basket, per avere anche la prospettiva di chi ha familiarità con il tema. A uno studente che porta l'esempio di un elemento di interfaccia sul personaggio, il relatore risponde che, volendo, si trova un motivo per invitare chiunque: l'importante è avere chiaro **chi è necessario**. Il gruppo di lavoro può essere di otto e il brainstorming di dodici, con input esterni a chi poi farà il lavoro. Basta non esagerare: se la maggior parte dei voti arriva da chi non farà quel lavoro, si vota su qualcosa che poi non faranno loro, e non ha più senso.

0.2.10 Dalla votazione alla decisione: la matrice impatto–sforzo

0.2.10.1 Massimo impatto, minimo sforzo

Fatto questo, non si è finito: ora si sa quali idee interessano di più, ma non si è ancora deciso che cosa fare. Serve la parte decisionale. L'obiettivo di qualsiasi azienda (ed è il motivo per cui al relatore non importa se il programmatore fa il test da casa cercando su Google) è **massimizzare l'impatto e minimizzare lo sforzo**. Vale per i progetti, i task, il recruitment, per qualsiasi cosa: "non perché sono pigro, ma perché sarebbe stupido fare diversamente".

Le idee selezionate si mettono quindi in un'altra matrice, con due assi:

- sull'asse verticale il **valore**, cioè quanto **impatto** avrebbe l'idea (in alto l'impatto alto);
- sull'asse orizzontale lo **sforzo** (*effort*), da basso (a sinistra) ad alto (a destra).

In alto a sinistra ci sono quindi le idee ad alto impatto e basso sforzo, in alto a destra quelle ad alto impatto e alto sforzo, e così via.

L'esempio Adidas. Se il cliente è Adidas, con la linea di abbigliamento per il basket, un'idea a basso valore potrebbe essere una campagna su TikTok in cui gli utenti condividono i loro momenti con un pallone da basket; un'idea ad alto valore potrebbe essere fondare un parco divertimenti dove i giocatori NBA vengono a sfidarti uno contro uno. Questo non dice che una sia meglio dell'altra: dice che l'impatto è diverso. Allo stesso modo si valuta lo sforzo: la campagna social è a basso sforzo, il parco divertimenti ad alto sforzo per molti motivi (i permessi di costruzione, i giocatori NBA).

0.2.10.2 Posizionare le idee, insieme e con precisione

Il gruppo che a quel punto è **il gruppo decisionale**, cioè chi farà il lavoro, deve posizionare sulla matrice le idee ritenute migliori, e deve essere **d'accordo tutto il gruppo**: "siamo tutti d'accordo che il parco dei dinosauri di Stefano e Andrea è ad alto valore ma anche ad alto sforzo? Sì? Allora lo mettiamo qui". E bisogna indicare **un punto preciso**, non solo il quadrante: piazzare un'idea in un punto non è come piazzarla poco più in là. Il risultato è una sorta di **mosaico** delle idee sul grafico.

0.2.10.3 Leggere la matrice in diagonale

Davanti a una situazione tipica, con varie idee sparse, il relatore chiede alla classe: se foste i project manager, se doveste fare voi queste cose, *quali* fareste? Al plurale, perché spesso da un brainstorming si esce con più idee realizzabili e si deve scegliere su quali concentrarsi. Le risposte variano: c'è chi sceglie le due con più impatto, chi aggiunge che dipende da tempi, budget e risorse, chi preferisce quella più innovativa ma anche più rischiosa.

Il metodo che secondo il relatore va applicato è **procedere in diagonale**, dalle idee a basso sforzo e alto impatto verso quelle ad alto sforzo e basso impatto[18]:

1. un'idea a **basso sforzo e alto impatto** (nella slide, quella rossa) va fatta **per prima, sempre**: "non dovrebbe essere nemmeno in discussione";
2. poi, a seconda delle risorse e del tempo, si possono fare **contemporaneamente** le idee che stanno sulla stessa diagonale successiva, che hanno cioè un rapporto simile tra impatto e sforzo;
3. poi le successive;
4. le idee ad **alto sforzo e basso impatto** (nella slide, la gialla) **non vanno nemmeno considerate**. Nessuno, infatti, l'aveva scelta.

[18] Nella slide le idee erano rappresentate da post-it colorati (rossa, rosa o salmone, azzurra, verde acqua, viola, blu, gialla), e le indicazioni del relatore si riferiscono a quei colori. Dalla sola trascrizione non è possibile ricostruire con certezza la posizione di ciascun post-it, e alcune indicazioni sui colori risultano contraddittorie; qui è riportato il principio generale, che il relatore chiarisce nelle risposte alle domande.

Per le scelte intermedie valgono due criteri, chiariti rispondendo a uno studente che non capiva perché alcune idee fossero preferibili ad altre: **a parità di sforzo si sceglie l'idea con più impatto**; **a parità di impatto si sceglie quella con meno sforzo**. Con risorse infinite si farebbero anche le idee più costose; ma di solito le risorse sono limitate, e allora tra due idee con lo stesso sforzo e un impatto molto diverso la scelta è evidente. Se ci piacerebbe fare un'idea più ambiziosa ma abbiamo risorse solo per una meno costosa sulla stessa diagonale, va bene lo stesso: abbiamo comunque stabilito che è meglio delle idee sulle diagonali successive. Quanto fare dipende dalla **bandwidth** a disposizione.

"La gialla era la mia idea: devo offendermi?" No. Qualcuno ha detto che l'autore fa schifo? No. Si è concordato che la sua idea, per quanto votata ("qualcuno ha votato per la mia idea"), è piuttosto difficile da realizzare rispetto al suo impatto. Non si sta dicendo che non sa più fare il suo lavoro. Anzi, se lui stesso dovesse scegliere tra la propria idea gialla e la rossa di un altro, sceglierebbe la rossa: non è nemmeno una questione di discussione. La matrice rende **visivamente evidente** che cosa fare, e permette di aggirare molte discussioni che non sono più soggettive: oggettivamente, dovremmo fare così.

0.2.10.4 Il ruolo del moderatore e dell'esperto

È qui che lavora il moderatore: "non è più una discussione; visto il posizionamento, dobbiamo fare queste cose". Non si decide a maggioranza dove mettere le idee: se uno posiziona un'idea e un altro non è d'accordo, difficilmente dirà "la rossa dovrebbe stare al posto della gialla"; dirà "secondo me è un po' più a destra, un po' più in basso", e si fanno gli aggiustamenti. Se invece nascono discussioni del tipo "la rossa dovrebbe stare al posto della gialla", vuol dire che non si è definita bene l'idea e non è chiaro quanto sia davvero difficile da realizzare.

Oppure interviene **l'esperto**: è l'esempio del concept artist e del 3D artist. Il concept artist dice che un'idea è a basso sforzo e alto impatto; il 3D artist risponde "ne condivido l'impatto, ma dovendola fare io ti assicuro che lo sforzo non è basso, è molto alto". Allora è un altro discorso, perché l'esperto sta dicendo che lo sforzo è diverso da quello stimato. Ma alla fine tutto deve essere concordato.

0.2.10.5 Chi decide

Il gruppo decisionale può essere diverso da quello del brainstorming? Sì: a questo punto escono quelli invitati come "consulenti", che hanno dato idee ma non devono poi realizzarle. Farli votare anche qui significherebbe perdere il collegamento tra chi deve fare le cose e che cosa si decide di fare. In quel momento non hanno responsabilità: hanno dato un input, e se decidessero che cosa devono fare gli altri dovrebbero esserne i manager. Non dovrebbero poter dire agli altri che cosa fare se non ne hanno la responsabilità. Vale per tutti i team, piccoli e grandi; secondo il relatore chi decide dovrebbe essere **al massimo una decina di persone**. In un team di dieci che fa il brainstorming tutto insieme, va benissimo che chi vota coincida con chi ha partecipato: l'importante è che **chi vota sia chi farà il lavoro**.

Lo sforzo è il costo o la fatica? Uno studente chiede se l'alto impatto includa anche il costo. No, il costo sta nello sforzo, che per il relatore va **ragionato in risorse**: denaro, tempo e così via. Un'idea può avere un basso costo monetario ma un alto costo in tempo, e allora va bilanciata e si troverà a metà. È lì che si spostano le idee e che avviene la conversazione: i tecnici dicono che lo sforzo tecnico è un certo, poi arriva l'*accountant* che dice che a livello economico è un altro, e la si sposta.

E il piacere di fare il progetto? In un gruppo di due o tre persone, osserva uno studente, si finisce per votare anche per preferenza personale: quando si fa un gioco bisogna avere anche il piacere di farlo. Ma **anche quello è impatto**: l'impatto sulla motivazione. Come lo sforzo sono le risorse, l'impatto è l'impatto *su di te, sul team, sul mercato*, su ciò che si vuole. Il relatore lo ha tenuto volutamente generico: può essere quanti soldi si fanno o quanto è utile per il proprio CV, e l'impatto cambia tra un progetto accademico e uno professionale.

0.2.11 Consigli pratici e domande sul brainstorming

0.2.11.1 Musica e cambio di scenario

"Sembrano tutte stupidaggini, ma vi assicuro che aiutano."

Musica di sottofondo. È dimostrato che aiuta[19]. Meglio una musica con **tempo e volume costanti**: niente *bass drop*, niente variazioni di tema o di ritmo. Va bene la musica fatta di rumore ambientale a basso volume; è stato rilevato anche che le canzoni in **chiave maggiore** aiutano la parte creativa a produrre idee più originali. Meglio la **musica strumentale**, senza testo: per quanto ci si impegni a non ascoltarlo, il cervello cercherà di capire le parole. È un effetto marginale e non succede niente se capita la canzone con il testo, ma si mette in conto che "l'uno per cento del cervello" sarà impegnato a capirlo, come quando in treno si ascolta, che lo si voglia o no, la telefonata di qualcun altro. È *min-maxing*.

Cambiare scenario. A Riot c'era una stanza dedicata al brainstorming, pensata per essere creativa: le finestre sul fiume, le porte a vetri colorati, tutto fatto per ispirare creatività. Ma fare brainstorming sempre nella stessa stanza generava sempre lo stesso tipo di idee: anche solo cambiare ambiente aiutava ad averne di diverse. Il relatore spesso spostava il meeting, ad esempio nello Starbucks sotto l'ufficio. Lo spazio fisico gioca un ruolo importante, ma anche quello virtuale: se lo si fa sempre su Discord, si può provare con ognuno uno sfondo diverso e simpatico. Oppure: ha sempre moderato la stessa persona? Si prova con un altro moderatore. Si cambiano i fattori perché i risultati non siano sempre gli stessi.

Uno studente chiede se il cambio di scenario possa essere anche qualcosa come passarsi una palla per avere la parola, o un altro espediente per tenere viva l'attenzione. Con questo modello, che non richiede di prendere la parola, l'esempio della palla perde senso, ma come esempio va benissimo: anche il comportamento fisico e il *modus operandi* sono un ottimo cambio di scenario. Come lo sono cambiare il moderatore o **cambiare l'obiettivo**: se il cliente chiede X, gli si darà un'idea X, ma si può provare anche a votare per un'idea Y, facendo più votazioni con vari criteri di valutazione.

0.2.11.2 Domande

L'idea scelta deve essere approvata da qualcun altro? Se un manager dice "fate questo brainstorming senza di me", sta **delegando la scelta**: "mi fido di quello che sceglierete". Avendo preparato prima limiti, paletti e *constraint*, si aspetta che la soluzione portata sia in linea con essi; ma se non partecipa deve fidarsi della scelta. Se poi la scelta debba passare anche da un livello superiore dipende dall'azienda e dagli accordi precedenti.

Si può fare un brainstorming ibrido, tra presenza e remoto? Uno studente fa l'esempio di un gruppo in cui uno è a New York e gli altri a Milano. Sì, senza problemi: si fa una call, il portatile punta verso il muro con le idee, o le si mostra fisicamente con il cellulare. Chi è da remoto scrive i suoi post-it, ad esempio mandandoli su WhatsApp al moderatore o a qualcuno presente che li scrive per lui, e vota dicendo "mettimi un pallino qui, qui e qui" (in quel caso il voto non è anonimo, ma si può fare tranquillamente); se deve parlare, parla.

E se i colleghi non riconoscono o non condividono il metodo? Ci deve essere un **agreement**: non si può arrivare e dire "facciamo così" se gli altri non vogliono. Questo metodo funziona solo se tutti sono d'accordo a usarlo. Il relatore ne ha proposto uno tra i cento metodi di brainstorming esistenti, un buon compromesso tra semplicità ed efficienza (ce ne sono di più semplici e di molto più efficienti ma più complessi). Se il gruppo non ne vede il valore, al massimo insisterebbe per **fare almeno una prova**, altrimenti diventa chiusura mentale; se poi per quel team non funziona, se ne cercano altri.

[19] Le affermazioni sugli effetti della musica (chiave maggiore, rumore ambientale) sono riportate come dette in lezione e non sono state verificate.

E nei gruppi molto piccoli, dove diventa un duello? Uno studente ha lavorato in gruppi di tre o anche di due persone, dove il brainstorming diventa quasi un duello in cui ognuno difende la propria idea. Bisogna **aspettare la discussione**: invece di "scegliamo tra la mia idea e la tua", si cerca di concordare quale idea abbia l'impatto maggiore e quella lo sforzo minore. E se non si è d'accordo, bisogna **portare esempi**: se uno dice "questa idea costa di più", l'altro chiede "in che senso? Che cosa costa di più?". Se non sa rispondere, non si può fare qualcosa senza sapere come va a finire: meglio rimandare la discussione a quando si avranno le informazioni. A volte la domanda giusta è *perché non abbiamo queste informazioni?*: forse è **prematuro** decidere, perché non si hanno ancora tutti i fattori.

Nei team piccoli non dovrebbe esserci un "si fa come dici tu o come dico io", una scelta a pelle: si dovrebbero conoscere così bene le proprie proposte da poterle posizionare con assoluta certezza sulla matrice, e avere anche la presenza di spirito di dire "la tua è chiaramente migliore, si fa la tua". Si fa la cosa migliore per entrambi, che dovrebbe essere oggettiva e non soggettiva. Non è facile arrivarci, ma deve essere l'obiettivo del team.

0.2.12 Disagree and commit

0.2.12.1 Quando la scelta non ci convince

Uno studente teme che chi vede scartata la propria idea perda motivazione, in un gruppo piccolo come in uno grande. Non è un deragliamento: è una conseguenza potenziale di qualsiasi scelta. Il relatore ha visto gente demotivata perfino dalla settimana di quattro giorni: ci sarà sempre qualcuno più o meno motivato dalle scelte. Ma qual era l'alternativa? Un brainstorming classico in cui tutti parlano, chi più chi meno, e si sceglie un'idea che tutti pensano sia la migliore senza esserne davvero sicuri, perché non si è votato, né espresso una preferenza, né discusso davvero le idee migliori.

Qui entra il concetto di teamwork del **disagree and commit**: "non sono d'accordo con questa decisione, non mi piace, non penso sia la migliore, ma la faccio". Per due motivi:

1. perché **è il mio lavoro**, e dovrebbe essere il punto principale;
2. perché **voi ritenete che sia la scelta migliore**. Se si è la voce fuori dal coro e tutti hanno scelto un'altra strada, non si rema contro: è *default to trust*. Se gli altri vi hanno visto più potenzialità, e (grande premessa) **hanno ascoltato** il proprio disaccordo e i suoi motivi, e hanno comunque scelto così, ci si fida di loro.

Con una postilla: se le cose vanno male, **non si dà la colpa agli altri** e non si dice "ve l'avevo detto", come se non si facesse parte del team. Disagree and commit vuol dire "mi fido del team e facciamo questa cosa"; se poi si rivela sbagliata, *il team* ha sbagliato, e la volta dopo si cambierà il modo di scegliere. Altrimenti non è disagree and commit, è "*disagree and ostracize*": disaccordo più ostilità. Non bisogna cercare il colpevole, perché si è deciso tutti insieme (sezione 0.1.11).

Se si pensa davvero che si stia sbagliando clamorosamente il posizionamento di un'idea, si dice piuttosto "non andiamo avanti finché non definiamo bene lo sforzo di questa cosa"; ma bisogna saperlo giustificare, altrimenti è chi dissente a dover fare un passo indietro.

0.2.12.2 La demotivazione fa parte del lavoro

La demotivazione ci sarà sempre, perché non si sceglierà mai solo ciò che piace: per quanto piaccia il proprio lavoro, ci saranno task sgradevoli e scelte che non spettano a noi. E ci saranno scelte ("ve lo dico per esperienza personale") che si è convinti siano sbagliate, al punto di pensare "piuttosto lascio il team", ma che si finisce per accettare, per poi rendersi conto che gli altri avevano ragione. Come c'è un *bias* verso l'amico e il nemico, c'è anche un **bias verso noi stessi**: crediamo di conoscere bene una cosa, e se altre dieci persone dicono "secondo me è fattibile", bisognerebbe aprirsi alla possibilità che ne sappiano più di noi.

"Non lo accetto ma lo faccio" dovrebbe essere alla base del lavoro insieme, perché è *default to trust*: rifiutarsi ("non mi fido di voi, non lo faccio") significa non fidarsi del giudizio del proprio team. E dire ad alta voce "non sono d'accordo, ma ti darò comunque il massimo" è la cosa che **fa guadagnare di più il rispetto degli altri**. Non c'è niente di meglio che vedere qualcuno che aveva un'opinione diversa impegnarsi

per far vedere di avere torto, facendo andare bene la cosa. Per il relatore dimostra solo che è una persona matura e importante; ha avuto persone che, passate da questa situazione, gli hanno fatto pensare "è molto più matura di quanto credessi", e che ha valorizzato molto di più in seguito, perché sa di potersi fidare di chi fa disagree and commit: è stato aperto nella comunicazione, ha detto che cosa secondo lui non andava, ma non ha fatto niente per remare contro, anzi ha dato il suo supporto. "Non posso chiedere di più a una persona."

0.2.12.3 Chi arriva a lavori iniziati

Se il team cambia in corso d'opera, chi subentra come si rapporta alle decisioni già prese? (per dimissioni, promozioni e così via). Deve **accettare lo status quo**. Al relatore capita spesso di entrare in team con progetti già in corso: si fa dire che cosa è stato scelto finora e va **in commit di default**: "non ho preso parte a questa decisione, quindi non sono in *agreement*, ma ora faccio parte del team, quindi vado in commit".

E se l'esperienza dice che una scelta ha uno sforzo molto diverso? Ad esempio che un'idea "rossa" ha in realtà lo sforzo di una "viola". Lo si fa presente, ma dipende dal **tempismo**: se la scelta è appena stata fatta e c'è ancora modo di cambiarla, la si segnala; se si è al 90% del progetto, è controproducente dire "fermi tutti, passo indietro", e a un certo punto bisogna rispettare la scelta del team ("avete scelto così, andiamo avanti e vediamo come va"). E se si sa che, calcolando ferie e periodo estivo, un'animazione prevista per giugno arriverà a settembre? È molto *case by case*: a seconda dell'apporto si può modificare il risultato o no. È come riaprire un processo se emergono nuove prove: se dimostrano che l'imputato non è l'assassino, è giusto riaprirlo; se dimostrano che l'assassino ha sparato con la sinistra invece che con la destra, conta fino a un certo punto. È giusto segnalarlo, ma non sempre quello che si porta giustifica uno "*scratch that*, rifacciamo tutto": diventa **un feedback, non una risoluzione** ("state facendo così, questo è il mio feedback, poi vediamo come va").

Con cinque minuti di anticipo si va in pausa pranzo, con l'accordo per il pomeriggio: circa un'ora di presentazione più "standard" sul leading from within, poi domande e risposte, "e quando finiscono le domande abbiamo finito".

0.2.13 Leading from within: perché parlare di leadership

0.2.13.1 La ripresa del pomeriggio

Alla ripresa, mentre si aspettano gli ultimi arrivati, il relatore chiede se durante la pausa siano venute in mente domande. L'unica è logistica: le slide verranno date a fine workshop? Sì: il relatore le passerà all'accademia (a Vanessa, Andrea ed Enrico[20]), che le farà avere alla classe. Gli appunti vanno comunque presi, meglio su quello che dice che su quello che mostra.

L'ultima parte, dice, dovrebbe essere abbastanza veloce da presentare, ma vale la pena fermarsi a parlarne, perché è un discorso più avanzato. È la logica chiusura di quanto detto tra la prima giornata e la mattina: tirare le fila e mostrare come tutto faccia parte di un unico filone di ragionamento e progettualità, con un unico scopo, un unico obiettivo e un unico modo di ragionare.

La voce sembra a posto, ma il relatore è stanco dopo due giorni, come immagina lo sia la classe, nello *stretch* finale di due giornate molto piene. Sono cose che in condizioni normali spiega al suo team nell'arco di settimane, con il tempo di metterle in pratica, discuterle e approfondirle. Qui è tutto concentrato: servirà tempo per metterlo in atto e fare le proprie prove, quindi non bisogna spaventarsi se non verrà tutto giusto la prima volta.

0.2.13.2 Il leader non è il più bravo

Perché parlare di leadership in un workshop non rivolto solo ai leader? La risposta completa arriverà più avanti, ma il punto di partenza è che il relatore **non crede nei leader di team o di progetto che**

[20] Nella trascrizione l'accademia è chiamata "Ive" e "Hive", come nella prima giornata; per i nomi dei referenti vedi la nota all'inizio del capitolo 0.1.

detengono da soli il sapere e il modo di fare le cose. Come detto il giorno prima, essere un leader, un manager o un capo non dovrebbe voler dire essere il migliore in quel lavoro, ma essere **la persona più adatta a ricoprire quel ruolo**: guidare un gruppo significa **occuparsi delle persone** al suo interno.

L'obiettivo che il relatore si dà, e che secondo lui ogni manager dovrebbe avere in mente, è che quando si torna dal lavoro ci si senta **ispirati, sicuri e realizzati**, e che questo sia **un diritto naturale di tutti**, non un lusso riservato a chi ricopre certe posizioni, a pochi fortunati o "a qualcuno sì e a qualcuno no". Ha notato, specialmente in Italia, la tendenza a vedere tutto ciò che viene prima del diventare manager come la *gavetta* necessaria per arrivarci. Invece chiunque faccia parte di un'organizzazione, che sia un'azienda di videogiochi o altro, dovrebbe sentirsi soddisfatto, ispirato e sicuro in quello che fa; e non è corretto che un'azienda tratti bene le persone "da manager in su". Anche quando non lo dicono, a volte i fatti sono quelli: da manager si viene trattati in un certo modo e si ha accesso a certe cose, altrimenti no. Non si tratta di **appiattire la gerarchia**: si tratta di assicurarsi che, pur dando cose diverse ai diversi livelli, a tutti i livelli venga riconosciuto uno strato comune di **rispetto, fiducia e professionalità**.

0.2.13.3 Le carriere divise: manager o *subject matter expert*

Uno studente chiede conferma: quindi un *lead artist* non è necessariamente più bravo di un senior a fare l'artista? Esatto. E qui il relatore apre una parentesi. In molte aziende, e soprattutto in quelle in cui tende a lavorare lui, la motivazione principale per diventare manager è avere una promozione e quindi un aumento, più che il desiderio di prendersi cura del team e delle persone. Come si ovvia al problema? **Dividendo le carriere**. Arrivati al livello senior, il relatore fa con ognuno dei suoi senior **"the Talk"**, per esporre le due opzioni che hanno davanti:

Busta A: diventare manager È il percorso standard: si valuta se si è adatti a fare il leader e a guidare un team, e si danno gli strumenti per farlo, perché "nessuno nasce leader imparato": prima di provarlo sulla propria pelle nessuno sa che cosa voglia dire. Per quanto si possa essere carismatici, fare il leader è un'altra cosa, e non vuol dire solo essere carismatici. Ci sono formazione, strumenti e un percorso di crescita.

Busta B: diventare *subject matter expert* Una persona che va oltre il senior nella stessa disciplina: diventa **l'esperto dell'intera azienda** su quella disciplina, perché la sua priorità non è aiutare gli altri ma essere il migliore. "Non c'è assolutamente nulla di male." Il suo ruolo è essere il più specializzato in quel campo, il punto di riferimento da cui tutti vanno a farsi spiegare le cose.

Il concetto è abbastanza nuovo, e il punto chiave è che l'azienda **paga il *subject matter expert* e il leader esattamente allo stesso modo**: non c'è un guadagno economico nello scegliere di fare il manager invece dello specialista, e il relatore valuta quel tipo di conoscenza quanto il saper gestire persone. Così chi vuole essere il migliore, o anche solo un aumento, può continuare a fare ciò che sa fare e avere l'aumento; chi vuole l'aumento *e* prendersi cura delle persone sceglie l'altra strada. Così si fa una scrematura sul *motivo* per cui si vuole fare il manager. Sembra una scelta piccola, ma è molto importante capire se ci si vuole occupare di se stessi o degli altri. Non è un giudizio ("non sto dicendo che sia meglio l'uno o l'altro"): le due cose **non convivono**, e bisogna sapere che cosa si vuole fare, sapendo che l'azienda tratterà allo stesso modo le due scelte.

Si può essere entrambi? Si può scegliere ad esempio il percorso da lead artist e voler comunque continuare a migliorare le proprie capacità. Ma, come nelle simulazioni di conversazione del giorno prima, il relatore chiederebbe: "se avessi le risorse per fare solo una delle due cose, quale sceglieresti?". Perché non le si avrà: non ci sarà il tempo e il modo di fare bene entrambe. Il suo "cinismo" arriva a dire che bisogna scegliere, perché tenersi buone entrambe le strade porta a farle male tutte e due. Preferisce un ottimo *people manager* o un ottimo specialista; le figure intermedie, in questo campo, rischiano di diventare pericolose per sé e per il team.

Specializzazioni verticali. La strada dello specialista permette specializzazioni molto verticali: in chat qualcuno cita il *mecha designer*, ma può esserci anche lo specialista di animazioni delle espressioni facciali. Naturalmente l'azienda deve avere bisogno di quella competenza (non ci si specializza in cose che non

servono), e non è detto che ne serva uno solo. È un percorso di carriera parallelo e alternativo a quello del manager "standard".

0.2.14 Il "momento Piero Angela": la biologia del gruppo

0.2.14.1 Perché una digressione scientifica

Il relatore annuncia un "piccolo excursus biologico", fondamentale per capire dove vuole arrivare, e prova a mettere un sottofondo musicale adatto: la condivisione dell'audio non funziona, quindi manda il link in chat perché ognuno lo faccia partire per conto suo. Un studente lo battezza subito "momento Piero Angela". Il relatore cerca di far combaciare le motivazioni aziendali del suo metodo di leadership e di progettualità con **basi scientifiche**: in passato ha collaborato con varie università che si occupavano di neuromarketing e neurobiologia, per avere una conferma o una smentita del fatto che ciò che ipotizzava "non fosse una cavolata colossale". Semplificando moltissimo, si guardano solo alcune sostanze che si traducono direttamente nel modo in cui si vive l'essere in un gruppo, in un team, in un'azienda[21].

0.2.14.2 Endorfine

Le **endorfine** sono rilasciate per **mascherare il dolore**. Lo sa chi corre, chi fa pesi o ginnastica, ma anche chi ha riso così tanto per una battuta da avere male ai fianchi: è quella sensazione di dolore in cui però si è contenti di provarlo. Il *runner's high* è l'esempio più calzante: essere distrutti dopo una corsa ma soddisfatti. È la risposta biologica allo sforzarsi in cose che sono per il nostro bene, un modo con cui il corpo ci "convince" a farle.

0.2.14.3 Dopamina

La **dopamina** è più familiare: è la **ricompensa per un'azione**. Dalle cose più banali (si mangia e si riceve dopamina) a quelle viste in mattinata: anche completare una checklist rilascia dopamina (sezione 0.2.3). Viene rilasciata quando si vede l'obiettivo, come il traguardo durante una corsa; è la stessa cosa che accadeva ai cacciatori del villaggio in tempi antichi, che rilasciavano dopamina quando vedevano le tracce e venivano così incentivati a seguirle. Sono concetti fondamentalmente "animali", ma si applicano tuttora anche in azienda.

Una nota di colore: la dipendenza. La dopamina **crea dipendenza**: molti casi di ossessione per alcol, nicotina, gioco d'azzardo e social media sono problemi legati alla dipendenza da dopamina. Secondo il relatore non è un caso che le diagnosi di ADD e ADHD siano aumentate del 60% negli ultimi dieci anni[22], perché buona parte di quegli stimoli è diventata ampiamente disponibile proprio in quel periodo. Anche la dipendenza dal lavoro, a volte, ha la stessa origine: si vuole il rispetto che deriva dal lavoro, il *rush* che deriva dall'averlo.

0.2.14.4 Il villaggio di diecimila anni fa

Si fa poi un salto di diecimila anni, in un villaggio. Gli esseri umani, per quanto più o meno introversi o estroversi, sono **animali sociali** che tendono a vivere esperienze di gruppo; le aziende nascono da un gruppo di persone che si danno un obiettivo condiviso, come in antichità un gruppo di persone che voleva sopravvivere insieme. Quello che non è cambiato è che **fuori dal gruppo ci sono dei pericoli**: tigri, lupi, draghi ("do per scontato che nell'antichità ci fossero i draghi, perché D&D mi insegna così"). Per evitarli, i gruppi tendono a riunirsi e a creare difese intorno a sé: da qui le prime difese dei primi villaggi, per isolarsi dai pericoli esterni. "Vi giuro che avrà tutto senso."

[21] L'impianto di questa parte (endorfine, dopamina, serotonina, ossitocina e cortisolo, il villaggio protetto dai pericoli esterni, il "cerchio di sicurezza") ricalca da vicino quello del libro di Simon Sinek *Leaders Eat Last* (2014), che il relatore cita poco più avanti. Le descrizioni biologiche sono molto semplificate, come avverte lo stesso relatore, e vanno lette come tali.

[22] Nella trascrizione "EDD e HD". Il dato e il nesso causale con la maggiore disponibilità di alcol, nicotina, gioco d'azzardo e social media sono riportati come detti in lezione e non sono stati verificati; l'aumento delle diagnosi viene in genere attribuito anche ad altri fattori, come la maggiore consapevolezza e il cambiamento dei criteri diagnostici.

0.2.14.5 Serotonina

La **serotonina** viene rilasciata anche *indirettamente*: non deve per forza riguardare noi. Quando si assiste a una cerimonia o celebrazione pubblica che include una persona cara, si rilascia serotonina come se si fosse noi l'oggetto del riconoscimento. È uno dei motivi per cui si fanno diplomi e lauree "all'americana", con la consegna sul palco davanti a tutti: gli studi avrebbero mostrato che anche i genitori in platea rilasciano serotonina, come chi riceve il premio. Lo stesso vale per parenti, amici, matrimoni: sono esperienze condivise, e si tende a condividere in pubblico riconoscimenti e successi perché in tutti i partecipanti viene rilasciata serotonina.

Purtroppo ha a che fare anche con i beni materiali: la tendenza all'*hoarding*, cioè a collezionare eccessivamente qualcosa, nasce dal fatto che quell'attività rilascia serotonina, ingannando la mente nel credere di aver ottenuto un successo.

È chiamata anche **la sostanza della leadership**, perché aiuta a riconoscere in qualcun altro chi ha valore per il gruppo. Nel villaggio, quando arriva un pericolo, c'è sempre un gruppo di persone che dice "ce ne occupiamo noi": il gruppo sa che si stanno prendendo un rischio, e la serotonina è il modo per riconoscere che meritano rispetto. Il relatore consiglia il libro di Simon Sinek *Leaders Eat Last*[23]: i leader mangiano per ultimi, all'opposto di quanto accadeva in antichità, quando al ritorno dalla caccia mangiavano per primi i cacciatori, che dovevano essere in forze per la caccia successiva, poi donne e bambini, poi gli altri. C'è un **riconoscimento sociale**: tu fai qualcosa per proteggere il gruppo, noi ti riconosciamo qualcos'altro in cambio.

Le aziende sono strutturate in modo molto diverso? È una domanda retorica: quando si dice che serve un capo, un manager, un rappresentante, si sta dicendo che quella persona dovrebbe avere l'interesse di proteggere il gruppo e di prendersi cura di tutti. Per questo sapere che il proprio capo guadagna di più non fa particolarmente arrabbiare: lo si vede come quasi naturale, perché più responsabilità si ha verso il gruppo più si guadagna, dato che quello dovrebbe essere il suo scopo. Ed è anche il motivo per cui il relatore si arrabbia molto quando legge di CEO o *executive* che, a fronte di licenziamenti di migliaia di persone, si assegnano dei bonus. È una rabbia non del tutto razionale, che sente da dentro: è come se si stesse violando un **contratto sociale**. Il contratto è "tu ti prendi dei rischi per me e mi difendi, io ti riconosco più cibo, più soldi"; quando si abusa di quella posizione per un beneficio personale a scapito di chi si dovrebbe proteggere, tutti lo vedono come un'ingiustizia, perché si viene meno al contratto stabilito diecimila anni fa con la nostra biologia.

0.2.14.6 Ossitocina

L'**ossitocina**[24] è la **sensazione di sicurezza** che si genera quando si è vicini a qualcuno. Non è un caso che tra i legami di affinità e collaborazione più forti ci siano quelli dei team che si uniscono contro un **nemico comune**: i team che superano insieme avversità e difficoltà ne escono molto legati. Tra le persone che il relatore sente più spesso in questo periodo ci sono quelle con cui ha condiviso momenti difficili, un cattivo manager, una brutta esperienza lavorativa: sa che erano lì con lui. Vale anche per i veterani di guerra o per i sopravvissuti a una stessa tragedia, che sviluppano un rapporto molto forte (se ne è parlato molto negli Stati Uniti a proposito dei veterani), perché hanno rilasciato immense dosi di ossitocina. È anche ciò che fa accettare subito il figlio alla madre quando lo prende in braccio dopo il parto, ed è il motivo per cui, a migliaia di anni di distanza, si stringono ancora accordi con una stretta di mano o un abbraccio: c'è sempre qualcosa di fisico, generato dal bisogno di ossitocina.

Viene generata anche quando si compiono o si **assiste ad atti di generosità** senza aspettarsi nulla in cambio. Se il relatore dicesse di aver donato cinquanta euro a un'associazione di beneficenza, la reazione sarebbe "ok, bene"; se dicesse che ogni sabato pomeriggio fa volontariato, lo si vedrebbe come qualcosa di molto più prezioso, e la cifra potrebbe crescere parecchio prima che qualcuno la consideri più preziosa dei suoi pomeriggi: tutti sanno che la vera donazione è **il tempo** investito, che vale più del bene materiale. E basta assistere: vedere qualcuno che aiuta un'anziana ad attraversare la strada fa stare meglio anche noi. È percepita a livello di gruppo. Infine, l'ossitocina rafforza il **sistema immunitario**: sentirsi sicuri e tranquilli fa stare meglio e, in un certo senso, ammalare di meno.

[23] Nella trascrizione "Leadership Lust". Il titolo è *Leaders Eat Last: Why Some Teams Pull Together and Others Don't* (2014).

[24] Nella trascrizione "ossidocina".

0.2.14.7 Cortisolo

L'opposto è il **cortisolo**: la sensazione di stress, paranoia e paura, la risposta *fight or flight* (o scappo o combatto), con battito accelerato, muscoli in tensione e così via. Tra i suoi effetti:

- **blocca il sistema immunitario**: se si sta per morire, al corpo non importa di prendersi un raffreddore. È il motivo per cui le persone stressate si ammalano di più;
- **blocca l'ossitocina**: non ci si può sentire tranquilli quando si è in ansia;
- soprattutto, e questo è il punto più importante per le aziende, è **contagioso**. In qualsiasi documentario, quando una gazzella che bruca alza la testa e percepisce un movimento nella savana, scappano tutte, non solo lei. Succede, non sorprendentemente, anche in ufficio: quando qualcuno ha paura perché potrebbero esserci licenziamenti, o perché il manager è molto aggressivo, la paura si diffonde nel gruppo ed è molto dannosa.

È anche l'effetto alla base della **sindrome dell'impostore**: sentirsi costantemente inadeguati perché si teme che succeda qualcosa significa vivere in uno stato di stress. Ed è il motivo per cui chi è insoddisfatto del lavoro tende a stare peggio anche fisicamente e ad ammalarsi di più.

Sostanza	Che cosa fa (in sintesi)	Esempi dalla lezione
Endorfine	mascherano il dolore di uno sforzo che ci fa bene	*runner's high*, ridere fino ad avere male ai fianchi
Dopamina	ricompensa un'azione, si attiva vedendo l'obiettivo; crea dipendenza	spuntare una checklist, le tracce per il cacciatore, gioco d'azzardo, social
Serotonina	riconoscimento sociale, anche indiretto; "sostanza della leadership"	lauree e premi in pubblico, chi difende il villaggio, *hoarding*
Ossitocina	sicurezza e legame; rafforza il sistema immunitario	il nemico comune, i veterani, la stretta di mano, il volontariato
Cortisolo	stress, *fight or flight*; blocca difese immunitarie e ossitocina; è contagioso	la gazzella che scappa, la paura dei licenziamenti, la sindrome dell'impostore

0.2.15 Domande sulla biologia del gruppo

Il gioco d'azzardo include i gacha e le loot box? Assolutamente sì, anzi ne sono il **driver principale**: i gacha, i FIFA Ultimate Team, lo "sbustare" da *Magic* a *Hearthstone*, e in generale il "vediamo se stavolta mi va bene" sono tutti gioco d'azzardo. Lo studente aggiunge che molti giochi mobile sono costruiti su questi meccanismi e che c'è chi gioca più a quelli che ai giochi "normali". È il motivo per cui alcuni stati hanno dichiarato illegali certi giochi[25].

Come si genera l'ossitocina nei rapporti da remoto? "Bellissima domanda." È il motivo per cui, come detto il giorno prima, il relatore insiste perché i meeting si facciano almeno in **webcam**, perché i riconoscimenti siano dati e i successi celebrati **pubblicamente** (quando non mette troppo a disagio la persona), e in generale cerca di creare **situazioni condivise**. Anche lavorando da remoto, insiste per avere il *team building* dal vivo e almeno **un incontro all'anno** di persona in ufficio, perché crea situazioni di rapporto diretto.

Come si contrasta il cortisolo? In chat qualcuno suggerisce la meditazione: sì, assolutamente. Per il lavoro, la risposta arriva subito dopo nella presentazione.

[25] Il caso più noto è quello del Belgio, che dal 2018 considera gioco d'azzardo le *loot box* a pagamento; anche i Paesi Bassi le hanno contestate.

Il nemico comune è una strategia che un manager può usare? Spesso **il manager è il nemico comune**, senza bisogno di sforzarsi. Ma il relatore chiede di capire bene il punto: alcuni manager applicano consapevolmente il concetto "mi ergo io a nemico del team, così sono tutti uniti contro di me e lavorano bene", e ne fanno uno stile di management. Decidono di essere il *bad cop*, il *villain*, ed è così che ottengono risultati. È una scelta lecita e funziona; il relatore preferisce fare diversamente, ma non per questo dice che non funzioni. Molte volte si sottovaluta quanto sia consapevole: "non si rende conto di quanto sia antipatico a tutti"; forse se ne rende perfettamente conto e lo fa apposta per essere il bersaglio. Il relatore l'ha visto fare spesso come soluzione disperata, quando il team era troppo impegnato a litigare al suo interno: "piuttosto che vederli litigare tra loro, gli do io un motivo per odiare me, così si uniscono contro di me". Non lo consiglia assolutamente, ma l'ha visto funzionare.

Uno studente racconta che nella sua azienda il manager fa vedere *il lavoro stesso* come il nemico ("prima finiamo, meglio è"). È un modello più basilare ma, come dice lo studente stesso, funziona.

Quanto è etico puntare sulla dipendenza del giocatore per fare soldi? "Non è assolutamente etico, ed è assolutamente profittevole, altrimenti le aziende non lo farebbero." Il relatore racconta una cosa che lo ha sconvolto quando l'ha scoperta, a Blizzard. Lavorava su *Hearthstone*[26] e si stava discutendo di come monetizzare il gioco; venne preso come consulente un esperto, israeliano se ricorda bene, che lavorava a Malta per società di poker online e di *gambling*. Raccontò che molte aziende che monetizzano sulla dipendenza dal gioco d'azzardo avevano sviluppato metodi più o meno etici ("più spesso meno"). L'esempio culminante: analizzavano gli utenti occasionali, quelli che giocavano magari una settimana al mese, per capire *quando arrivava loro lo stipendio* (di solito si deposita lo stipendio e poi si gioca quello che resta), per colpirli il giorno dopo con email di offerte per tornare a giocare. Li si stimolava proprio nel momento in cui avevano disponibilità, sapendo che erano sensibili, per aumentare le probabilità di riaverli come giocatori. "È orribile. È terrificante e, come tutte le cose terrificanti, è anche altamente funzionale."

Gli ha fatto riflettere molto su dove ci si possa spingere per fare soldi. La versione etica, per lui, è ad esempio far uscire un gioco sotto Natale, quando si è più disposti a regalarlo ai figli: si stanno ottimizzando le proprie possibilità, e lo accetta. Si scade nel **predatorio** quando si sfrutta la debolezza di qualcuno in un momento preciso per estorcergli soldi che non può permettersi di perdere. Poi ognuno tragga le proprie conclusioni: "non è un workshop sull'etica del lavoro". Non pensa di dover imporre la propria opinione, né si arroga il diritto di dire che chi lo fa è una bestia senza cuore, visto che è il loro lavoro: "chiunque riesca poi a dormire in pace con se stesso, buon percorso".

0.2.16 La piramide della fiducia

0.2.16.1 La leadership non è un titolo

Chiuso il "momento Piero Angela", il relatore arriva al punto. La frase della prima giornata ("la capacità di un gruppo di persone di fare cose straordinarie dipende dal modo in cui queste persone operano insieme") porta a capire che **la leadership non è un titolo lavorativo**. Essere manager da contratto, o avere "manager" nel titolo, rende un manager, non un leader. Il relatore non ha mai visto un team rispettare qualcuno solo perché da contratto doveva farlo: si può avere paura o timore del proprio manager, ma non lo si deve rispettare perché è il manager. **Il rispetto si guadagna**, e si possono influenzare le persone intorno solo attraverso il rispetto guadagnato, quando si sono accumulati abbastanza esempi e *case history* da giustificare rispetto e fiducia. Altrimenti, dice, sarebbe il primo a chiedere "perché vi fidate di me, se non vi ho ancora dato motivo di farlo?": non sarebbe razionale.

Da qui la **piramide della fiducia**, che si costruisce partendo dalla base.

0.2.16.2 Alla base: l'integrità morale

Per essere un buon leader o un buon manager bisogna **voler genuinamente aiutare gli altri membri del team**. È il motivo del discorso sulle carriere divise: è a questo livello che si screma chi vuole la promozione per i soldi. Chi vuole essere leader non deve cercare il guadagno personale, ma quello del gruppo. Come

[26] Nella trascrizione "Arsdol".

detto il giorno prima, il relatore è pronto a litigare per il suo team e a prendersi le colpe per il suo team, perché vuole il bene del suo team e sa che poi loro si prenderanno cura di lui. Chi ha in mente questo sta in questa categoria. Poi:

- **essere onesti**. Il relatore può affermare di non aver mai mentito intenzionalmente a un collega, a un cliente o a un giocatore. Gli è capitato di dare informazioni errate perché in quel momento sapeva quelle: ad esempio annunciare che un'espansione sarebbe uscita in un certo mese, e poi l'uscita è slittata[27]. Ha detto una cosa sbagliata, ma tutti hanno capito che non stava mentendo: in quel momento pensava che fosse vera;
- soprattutto, **accettare la responsabilità dei fallimenti**: quando si sbaglia, saper dire "ho sbagliato". Il relatore lo ha messo perché "non avete idea di quanti facciano fatica" ad ammettere di poter sbagliare: c'è sempre una scusante, una mitigazione, un "sì, ma". Lui vuole sentire se è colpa tua o no;
- **chiedere scusa** e **ammettere di non sapere**: anche dire "non lo so" viene spesso evitato, come accettare un fallimento.

"Raccontami il tuo più grande fallimento". Per lo stesso motivo, nei colloqui il relatore chiede spesso, più che "parlami di un tuo grande successo" o di un progetto andato bene, **"mi racconti il tuo più grande fallimento?"**, e osserva come lo raccontano. Chi non si aspetta la domanda spesso racconta di quella volta in cui ha provato in tutti i modi a fare in un certo modo, ma il capo gli ha detto di fare diversamente e il progetto è andato male. "Non mi hai risposto: mi hai raccontato un progetto andato male, ma il tuo messaggio è che volevi fare diversamente e qualcuno ti ha detto di fare così. Quindi non è un tuo fallimento." È come rispondere a "qual è la tua debolezza?" con "sono così attento ai dettagli che a volte...": si fa passare un punto di forza per una debolezza. Il relatore vuole un esempio in cui *tu*, senza altri fattori, hai sbagliato, e come ti sei comportato. Gli dice molto su come quella persona reagirà ai suoi feedback: se ha davanti "un muro" che dirà sempre "non è colpa mia", non è adatta a un ambiente in cui il feedback è alla base di ogni comunicazione. Chi non ammette di aver mai sbagliato in vita sua, lo scarta.

È possibile colpevolizzarsi troppo per un fallimento? Sì, assolutamente. Rientra molto nella sindrome dell'impostore, o nell'aver sempre vissuto rapporti di lavoro tossici: ci si abitua ad accettare che ci vengano urlate contro le cose e che sia sempre colpa propria, e ci si autoconvince che lo sia.

0.2.16.3 Al centro: l'affidabilità

Superato questo livello si arriva all'**affidabilità**. Per essere un buon leader non bisogna essere il migliore, ma affidabili: basta essere riconosciuti come bravi in qualcosa, non necessariamente i migliori, e quell'affidabilità ce la si porterà anche nella leadership. Significa:

- **rispettare la parola data**: se si promette "parleremo di un aumento al prossimo slot disponibile", se ne parlerà. Non ci si dimentica, e non si dice poi "in realtà non si poteva fare, ma ho voluto provarci fino all'ultimo". Una promessa va mantenuta;
- **impegnarsi per il tempo necessario** a raggiungere gli obiettivi;
- **saper rifiutare un lavoro** quando necessario: è il saper dire di no della mattina, ancora la matrice di Eisenhower (sezione 0.2.4). "Sono sempre gli stessi concetti."

0.2.16.4 In cima: la leadership, cioè dare l'esempio

L'ultimo gradino è la **leadership**[28], cioè **dare l'esempio**. Non bisogna essere i più bravi; bisogna saper **offrirsi volontari non solo quando il progetto è un successo sicuro, ma anche quando c'è il rischio di un fallimento**. Sono tutti bravi a offrirsi per il gioco nuovo che andrà sicuramente bene.

[27] Nella trascrizione l'esempio non è coerente: prima si parla di un'uscita a novembre rimandata di una settimana, poi di "pensavo fosse settembre". Il senso resta chiaro.

[28] Nella trascrizione "dealership".

Il rappresentante dei licenziati. Al relatore è capitato di offrirsi per rappresentare il team durante i *layoff*, con il rischio di essere ostracizzato dall'azienda; con quelle persone è ancora in contatto. Quando Blizzard fece i licenziamenti nella sede francese, vicino a Versailles[29], che coinvolsero quasi tutto l'ufficio, lui non era tra i licenziati, ma venne **eletto rappresentante sindacale** di chi lo era. Per lui è stato molto importante: pur non essendo parte in causa, lo ritennero degno della loro fiducia. Per quanto ne sapevano, avrebbe potuto negoziare e accettare tutto quello che proponeva Blizzard, perché non ne era toccato: che si negoziasse un mese di liquidazione in più o in meno non gli portava un soldo. Avrebbe persino potuto usare la situazione per negoziare un aumento per sé ("gli facciamo accettare tre mesi invece di sette e a te diamo un aumento"): non era fuori discussione. Se si sono fidati è solo perché c'era una piramide di questo tipo e lui aveva dato loro abbastanza motivi per farlo.

Autorevolezza, non autorità. A uno studente che lo riassume così, il relatore risponde che è proprio questo: un carisma diverso dall'essere simpatici, l'essere un punto di riferimento, **più un pilastro di supporto che il motore** che fa andare avanti le cose. Non il motore davanti a tutti, ma il **meccanico** che, quando si rompe qualcosa, sai che non ti frega: ti sistema le cose senza "farti la cresta" sulle pastiglie dei freni.

Non abbandonare nessuno. E soprattutto **non abbandonare chi è in difficoltà**. Sembra una banalità, ma è alla base del motto dei Marines statunitensi, *no one left behind*: sapere che, se si viene feriti, verranno a prenderti anche sotto i missili è un forte elemento motivazionale per correre quel rischio. In azienda è il motivo per cui il relatore ribadisce sempre che, se qualcuno è in difficoltà, la sua prima opzione non è licenziarlo ma **dargli una mano**, e lo ha fatto a volte anche all'eccesso, per far vedere agli altri che quando saranno in difficoltà non li lascerà in mezzo a una strada. A volte ce l'ha fatta: ha esempi di persone "a tanto così" dal licenziamento recuperate per i capelli e diventate ottimi *performer*. Altre volte la persona non era recuperabile e alla fine è stata licenziata. Ma chi attraversa un periodo difficile (il *burnout*, la depressione, una relazione finita) sa che la prima risposta non sarà "ci sei servito finora, ora puoi andartene", ma "che cosa possiamo fare?". E questo fa lavorare tutti tranquilli.

0.2.17 Prendersi cura del gruppo

0.2.17.1 La soluzione: più leader possibile

In un caso del genere manager e leader coincidono, ed è l'ideale: un manager dovrebbe essere anche un leader. Ma la vera soluzione è **avere più leader possibile**. Se l'obiettivo del leader è prendersi cura degli altri, e il manager dovrebbe essere "il primo tra i leader", ancora meglio è che **tutti siano leader del proprio gruppo**, perché la leadership non deriva da un titolo né dall'essere il manager. Il relatore ha visto tanti gruppi andare avanti perché qualcuno al loro interno era il leader, non perché lo era il manager.

Vuol dire mettere in pratica tutto il discorso della prima giornata su reciprocità, gentilezza, aiuto e collaborazione. Si sa che, nel peggiore dei casi, c'è il manager, ma in condizioni normali ci sono i colleghi: e allora si va da chi è in difficoltà a dire "posso darti una mano?". Così, quando saremo noi in difficoltà, magari senza nemmeno rendercene conto, sarà qualcun altro a dirci "ti vedo in difficoltà, come posso aiutarti?".

L'esempio è contagioso, nel bene e nel male. Quando si ha una posizione anche solo di rispetto agli occhi degli altri, tutto ciò che si fa ha effetto sugli altri. In negativo vale la **slippery slope**[30]: se si accetta un cartone di pizza abbandonato per strada, domani se ne accetteranno due, dopodomani tre. Ma vale anche in positivo: chi vede qualcuno fare qualcosa di positivo pensa "lo voglio fare anch'io". "Come fai una cosa, fai tutto", dice un proverbio che il relatore attribuisce alla tradizione cinese[31].

[29] Nella trascrizione "i layoff di Versailles". La sede europea di Blizzard si trovava a Vélizy-Villacoublay, nei pressi di Versailles; il relatore non precisa l'anno.

[30] L'esempio del cartone abbandonato ricorda la cosiddetta "teoria delle finestre rotte" (*broken windows theory*), secondo cui i piccoli segni di degrado tollerati favoriscono comportamenti peggiori.

[31] La massima ("how you do anything is how you do everything") è attribuita a fonti diverse, spesso alla tradizione zen; l'attribuzione è incerta.

0.2.17.2 Il villaggio di oggi

Si torna al villaggio del "momento Piero Angela", con i difensori e le bestie (reali o no) intorno. Oggi fuori dal gruppo non ci sono più tigri, ma le **deadline**, l'**overwork**, i **layoff**, il manager che urla, le aspettative, i **KPI**. Avere manager che si prendono cura del gruppo è un buon inizio, ma è ancora meglio avere **il gruppo che si prende cura del gruppo**. Non bisogna più limitarsi a pensare che sia responsabilità del manager prendersi cura di noi: è responsabilità nostra prenderci cura del gruppo di cui facciamo parte.

Che cosa serve? La piramide vista prima. Serve essere percepiti come **autentici**: il relatore ormai riconosce abbastanza facilmente chi si vuole ergere a leader perché gli piace *sentirsi* leader, non perché vuole *fare* il leader, e quella mancanza di genuinità non aiuta. In generale bisogna **generare sicurezza** nel resto del gruppo.

0.2.17.3 Smettere di dire "ci penserà qualcun altro"

Un esempio concreto è smettere di dire **"ci penserà qualcun altro"**. Nello sviluppo è evidente: se si trova un bug che non riguarda il proprio lavoro, tutti sono d'accordo che vada comunque segnalato, perché altrimenti potrebbe arrivare in *live*, nella *release*.

Le lasagne di PG Esports. Circa un anno e mezzo prima, quando lavorava ancora a PG Esports (che organizza anche eventi dal vivo), l'azienda ospitava in ufficio dei potenziali sponsor venuti a conoscere il team per decidere se sponsorizzare la Milano Games Week. C'era un catering, con i classici buffet. In mezzo alla confusione (una ventina di persone di PG Esports e una trentina tra manager, sponsor ed *executive*), il relatore vede per terra, in mezzo alla stanza, una mezza porzione di lasagne abbandonata. Senza pensarci la raccoglie per portarla via: non è sua, ma è un piatto in mezzo alla stanza. Nel tragitto fino al bancone del buffet, almeno altri sei tra executive e sponsor che non lo conoscevano gli consegnano il loro piatto non finito, come a un cameriere: nella loro testa l'unico motivo per cui qualcuno potesse raccogliere un piatto da terra era che fosse il suo lavoro. Sul momento non ci fa caso; arrivato al bancone si gira e vede tutto il suo team che ride della scena.

Torna da loro e ne nasce una discussione: "avete visto quanto è stato educativo?". Avevano visto esattamente come ragionano i gruppi che non sono abituati a pensare "adesso ci penso io", ma "ci penserà qualcun altro". In quelle persone non c'era malizia: sicuramente era un cameriere, perché altrimenti farlo? "Non è il mio lavoro, lo avrà fatto perché è il suo." E diventa un problema quando la stessa tendenza ("non c'entra con il mio lavoro, ci penserà qualcun altro") si applica a qualsiasi altra cosa. Il relatore si aspetta che **tutti raccolgano quel piattino**: che chi vede un bug, se non può correggerlo lo segnali, e se può correggerlo lo corregga anche se non è il suo lavoro, "perché è un piattino in mezzo alla stanza".

Da reattivi a proattivi. Significa passare da un atteggiamento **reattivo**, cioè di fatto passivo, a uno **proattivo**: prevedere i problemi e cercarli intorno a sé per risolverli, invece di aspettare che esplodano ("finché non esplode non è un problema mio"). Ragionare nell'ottica del "e se succedesse, e io me ne fossi accorto senza fare niente?". Uno studente lo chiama *mindset da tester*: il relatore si aspetta che, se si vede qualcosa, ci si assicuri di non essere gli unici ad averlo visto. A volte basta farlo presente: il relatore non potrebbe mai correggere un bug, quindi se lo trova lo segnala, e va bene così. Ma non bisogna dare per scontato che "è un bug così evidente che qualcuno l'avrà già segnalato". È il paradosso dell'incidente stradale: **più l'incidente è grosso, più è probabile che nessuno chiami l'ambulanza**, perché tutti pensano che qualcuno l'abbia già fatto[32].

0.2.17.4 Domande sul "me ne occupo io"

Se correggo un bug, devo avvisare il collega? Dopo una breve pausa per problemi audio ("facciamo *double testing*"), uno studente chiede se, correggendo il lavoro di un collega, il dovere sia poi andare a dargli il feedback. A prescindere bisogna informare qualcuno; e la conversazione dovrebbe essere: "ho trovato questa cosa, vuoi che te la sistemi io?". Correggere senza dirlo a nessuno potrebbe essere un danno: magari era *working as intended*, o qualcuno ci stava già lavorando.

[32] È il cosiddetto *effetto spettatore* (*bystander effect*), studiato in psicologia sociale a partire dagli anni Sessanta.

E se divento "quello che risolve quel problema"? Uno studente porta la sua esperienza: in un'azienda in cui ha lavorato tre anni, risolvendo un problema facile che non era di sua competenza è diventato, di fatto, l'incaricato di quel problema, distolto dalle sue mansioni. È un'arma a doppio taglio. Per il relatore lì il problema è del **management**. In alcune aziende si applica il criterio "chi trova il problema lo risolve" ("se te ne sei accorto tu, pensaci tu"), ed è un altro modo di scaricare la responsabilità sugli altri. Non è ciò che dice lui: deve essere chi trova il problema a valutare se è giusto e positivo che lo risolva lui. Il cartone di pizza lo raccoglie perché ci mette dieci secondi; ma se ha trovato un bug nel sistema di pagamento che richiede tre settimane di lavoro, e chi l'ha introdotto fa finta di niente, non deve risolverlo lui: deve far presente che c'è un problema, che è stato ignorato e che va risolto ("pensateci voi, ma io vi dico che c'è"). Va fatto caso per caso: se è immediato, facile e nel proprio interesse, lo si risolve. Ma il relatore sconsiglia alle aziende la regola "se lo trovi lo risolvi", perché **disincentiva a trovare i problemi**: qualcuno potrebbe non segnalarli per non avere altro lavoro e aspettare che li trovi qualcun altro, ed è negativo quanto la situazione descritta. Per chiudere, ne discuterebbe con il management: "possiamo rivedere questa policy? Incentiva l'azione sbagliata".

E se ci dicono di non mettere mano? Uno studente che lavora come tester racconta che spesso potrebbero correggere certe cose, ma viene loro detto di non toccarle. Esatto: si comunica. "Ho trovato questa cosa, ne siete consapevoli? Sì? Ci state già lavorando? No? Posso farlo io? Sì? Allora lo faccio." Non bisogna esagerare con l'iniziativa: anche qui vale il *default to trust*. Si parte dal presupposto che qualcuno potrebbe averlo già visto e starci già lavorando, quindi si chiede e si offre il proprio supporto.

0.2.18 Se si diventa manager

0.2.18.1 Meriti, colpe e *big picture*

Una breve digressione per chi, un giorno, diventerà manager: che cosa cambierebbe? Oltre a tutto quanto detto, ci sono il discorso su **meriti e colpe** e quello su **progetto e task**: **delegare i meriti e prendersi le colpe**, e pensare al progetto più che ai singoli task, cioè alla *big picture*, invece che al "voglio tornare a fare le cose che sapevo fare".

È un altro problema dei manager promossi dal gruppo di chi sapeva fare bene un certo lavoro: quando hanno l'occasione di tornare a farlo, lo fanno molto volentieri. "C'è un task da fare, lo faccio io", magari con la scusa di dare l'esempio. No: si sta andando nella propria **comfort zone**. Il relatore lo capisce (anche lui tornerebbe volentieri a fare ciò che faceva prima), ma bisogna accettare che **quelle cose non le si deve più fare**.

0.2.18.2 Il ruolo è più grande della persona

Anche da manager, il ruolo che si ricopre per il gruppo è **più importante di sé**. Non importa chi sia il community manager di Blizzard, di Riot o di un'altra azienda: conta ciò che rappresenta per i giocatori in quanto community manager, non il fatto che sia lui piuttosto che un altro.

Prendersi cura del gruppo vuol dire applicare concetti molto semplici:

Invece di dire...	**... dire**
"Qui non si accettano errori."	"Spiegami perché abbiamo sbagliato": fammi capire perché è successo (sezione 0.1.11).
"Qui si fa come dico io."	"Tu come faresti?"

0.2.18.3 Il tetto della bravura del capo

Il secondo punto è cruciale soprattutto quando si promuove il più bravo in un lavoro: se il criterio noto è che si promuove il più bravo, il promosso lo sa, e la prima cosa che penserà è "faccio lavorare tutti come me, visto che sono il più bravo". Il relatore chiede alla classe: che cosa si nega l'azienda con questa dinamica? Anche ammesso che sia davvero il più bravo, se tutti fanno come lui diventeranno tutti più bravi, no?

Il problema è che **si fissa come tetto massimo la bravura di quella persona**: se tutti fanno solo come dice lui, si arriverà al massimo al suo livello, mentre ognuno ha il proprio metodo. Da qui quanto detto il giorno prima sulla tendenza dei manager ad assumere solo persone peggiori di sé: se il criterio di promozione è la bravura, si prenderanno sempre persone un po' meno brave per evitare che diventino il proprio capo. Se invece le carriere sono divise ("se sei bravo fai questo, se vuoi fare il *people manager* fai quest'altro"), il manager è incentivato a prendere persone **migliori di sé**; e se, come detto il giorno prima, il bonus dipende anche dalle performance del gruppo, è nell'interesse di tutti che il gruppo sia pieno di superstar, un ulteriore incentivo ad assumere i più bravi.

L'opzione C. C'è poi un'altra cosa, forse minima, ma per il relatore molto importante: si perdono le **occasioni di innovare**. Nelle aziende precedenti era visto come il punto di riferimento per il *problem solving*: in un'emergenza, se c'era da scegliere tra A e B, "decide Stefano". Andava benissimo, ma la tendenza era che a ogni emergenza venivano da lui. Così ci si negava **l'opzione C**: quella a cui lui non avrebbe pensato e qualcun altro sì. Magari la prima volta sbaglia, la seconda anche, e la terza ha un'idea migliore della sua, proprio perché lui non è più l'esperto di quel settore. Fissare tutto come "al massimo decide Stefano" significa non accettare altre idee: un limite molto grosso all'innovazione, all'aggiornamento e al fare le cose in modo più moderno e flessibile.

0.2.18.4 Celebrare i successi e i fallimenti

Celebrare i successi del team. Il relatore vuole assicurarsi che tutti capiscano quando si è ottenuta una vittoria o un successo: come detto il giorno prima, se non si sa che cosa si fa bene si rischia di smettere di farlo. "Questa cosa l'abbiamo fatta bene, complimenti, bravi tutti."

Celebrare i fallimenti. Sui fallimenti si era spinto ben oltre il semplice accettarli: aveva indetto **meeting in cui si celebravano i fallimenti**, con le persone che raccontavano i propri errori per distribuire quella conoscenza al resto del team. "Non mi fa arrabbiare l'errore fatto, mi fa arrabbiare lo stesso errore fatto quindici volte." Condividere un fallimento con il gruppo non significa "guardate questa persona sciocca che ha fatto questo errore", ma "questa persona ci sta permettendo di evitare lo stesso errore in quindici": sta salvando l'azienda dal ripeterlo quindici volte, perché la gente ha paura di dire agli altri che ha sbagliato. Mostrare dove si è sbagliato diventava quindi un incentivo, un servizio all'azienda. E si torna al punto di prima: se il proprio ruolo per il gruppo è più importante di sé, non importa dover alzare la mano e dire "ho sbagliato, ho fatto così", perché gli altri penseranno "grazie, così io non lo faccio". Si dovrebbe essere lieti che qualcuno abbia fatto quell'errore prima di noi, e andare a ringraziarlo per averlo condiviso.

Condividere i successi. Infine bisogna **dare il merito a chi lo ha**. Al relatore è capitato spesso di ricevere ringraziamenti come "grazie a Stefano e al suo team per aver fatto questa cosa", e la sua risposta è stata "grazie, ma ha fatto tutto Gianni". Non bisogna avere paura di dire "grazie del feedback, ma è stato lui": è giusto ringraziare lui e il team (il relatore si sente parte del team, chiaramente). Non si ottiene niente dal dire "grazie, riferirò i complimenti al team": il relatore non ha mai creduto nel "riferire" che il management ha detto "ottimo lavoro". Chiede al management di dirlo **direttamente** al team, perché sentirlo dire dal management ha un impatto diverso che sentirlo riferire da lui. In questo senso, osserva uno studente, ogni gruppo è il gruppo di sostegno di se stesso: "assolutamente sì".

0.2.18.5 Quanto ci vuole?

Non c'è una scadenza. Il rispetto non si ottiene pagando per fare prima, "con il *boost* a livello 60". Servono tempo ed energia, ed è difficilmente misurabile: il relatore non può dire quanto il team si fidi di lui in questo momento. Lo percepisce da come lavorano, ma lo si vedrà davvero solo quando ci sarà un problema grosso; fino ad allora è tutta una sua percezione. Il team lo sente ripetere queste cose ogni giorno e lo vede metterle in pratica ogni giorno, e lui può solo sperare che il messaggio passi; quando ci saranno davvero "gatte da pelare" capirà come è messo, perché non c'è momento migliore di una difficoltà per vedere come reagisce un team.

Essere leader è prendersi un impegno a lungo termine verso i propri colleghi: non ci sono modi per velocizzarlo. Bisogna fare il proprio lavoro ogni giorno "a testa bassa", aiutando gli altri, facendolo vedere ed **esplicitando perché lo si fa**. Il relatore lo fa quotidianamente: "ti sto aiutando; nel caso avessi un dubbio, lo faccio perché il mio lavoro è aiutarti e io ne traggo vantaggio dal fatto che tu lavori meglio. Sono qui per te, ma lo faccio anche per me: non pensare che sia solo gentilezza. È il mio lavoro, ma sappi che lo faccio spontaneamente".

0.2.18.6 L'ultima slide

L'obiettivo dovrebbe sempre essere lo stesso: tutti dovremmo poter finire la giornata, staccare e tornare a casa sentendoci

- **ispirati**, perché abbiamo fatto qualcosa che ha un effetto tangibile;
- **sicuri**, perché sappiamo di avere colleghi, un leader e dei manager che si prendono cura di noi;
- **realizzati**, per quello che facciamo e perché è ciò che ci piace fare.

E questo dovrebbe essere **un diritto naturale di tutti**, non un "sì, ma solo il manager", "sì, ma solo se sei un esperto", "sì, ma solo se sei un game director". Ognuno ha le proprie capacità: bisogna solo trovare ciò che piace e che si vuole fare.

0.2.19 Domande e risposte finali

Finita la presentazione, si passa alle domande: "chiedete tutto quello che volete, perché è l'ultima possibilità". Le domande sono riportate per temi.

0.2.19.1 Leadership e organizzazione del lavoro

La leadership riguarda tutti o solo chi vuole diventare leader? Il discorso sui manager vale solo per i manager; ma il consiglio è **essere tutti leader del proprio gruppo, da dentro**. Essere un leader significa sapere che gli altri possono fare affidamento su di sé, e di queste figure in un team ce ne possono essere infinite: non è una gara a chi è il leader. "Siate tutti una di quelle figure a cui gli altri guardano quando sono in difficoltà."

Da manager non si hanno più orari e si fanno straordinari di continuo? No: il relatore ha notato di essere molto più efficiente come manager quando si dà **paletti molto precisi**. Se stacca alle sei e alle sei e cinque arriva un messaggio, risponde "se non mi dici che è urgentissimo, non lo guardo nemmeno, lo leggo domani"; se non è urgente, lo guarda il giorno dopo. Ma gestisce le aspettative: "se è urgente ci sono e ci sarò sempre, ma non aspettatevi che faccia orari eccessivi solo perché sono manager, così come non me lo aspetto da voi". Si aspetta che ognuno faccia ciò che deve; se è urgente tutti faranno di più, ma se non lo è, no.

L'orario davvero flessibile. Qui il relatore aggiunge una cosa che si era dimenticato di dire parlando di ferie e permessi illimitati: introduce anche l'**orario davvero flessibile**. Non il flessibile di molte aziende, in cui si può iniziare alle 9 e finire alle 18 oppure iniziare alle 10 e finire alle 19: se oggi si vuole iniziare alle 10 e finire alle 15, e si è fatto tutto quello che si doveva, va bene così. Ci si prendono le ore libere in più per altro, per staccare, per riposarsi, sapendo che se la settimana dopo bisognerà lavorare dalle 9 alle 21, lo si farà (o si dirà "ieri ho fatto dalle 9 alle 21, domani faccio dalle 10 alle 15"). L'obiettivo dato è un lavoro e un progetto, non un orario: si lavora per completare i propri task e rispettare le deadline. L'orario indicato è solo indicativo, ed è quello dei meeting: **nessun meeting prima delle 10 o dopo le 17**. Se poi qualcuno vuole lavorare dalle 10 alle 11 e poi di notte, perché quel giorno preferisce lavorare di notte, più rilassato, faccia pure: l'importante è che trovi il tempo per riposare e staccare. Ci si autogestisce senza timbrare cartellini; l'unico accordo è "hai un task, va consegnato entro il giorno X e mi garantisci che venga fatto; come ci arrivi non mi interessa".

Le è mai capitato di essere aiutato da qualcuno, in un momento di difficoltà? Sì: come detto, questo comportamento è **contagioso in senso positivo**. Se ogni volta che qualcuno è in difficoltà qualcun altro lo aiuta, prima o poi capita anche a chi aiuta. Il relatore non lo dice "tanto per dire": ha avuto momenti in cui faceva davvero fatica ad alzarsi dal letto per lavorare, perché esausto o demotivato per altri motivi, e il team gli ha detto "stai a letto, ci pensiamo noi oggi". Questo lo aiuta ad aiutare, perché sa che la cosa è bidirezionale.

Tra le soft skill del workshop, da quale partire? Il feedback: saperlo dare e ricevere. È la più difficile, molto più del "voglio aiutarti", ma è anche quella su cui si costruisce tutto il resto: attraverso il feedback si costruisce la **fiducia**, attraverso la fiducia il **rispetto**, e il rispetto diventa la base portante di tutto il resto.

E se il leader se ne va? A una domanda su che cosa fare quando viene meno la figura di riferimento del gruppo[33], il relatore risponde che la domanda da farsi è un'altra: **è cambiato l'obiettivo?** La direzione è sempre la stessa? Se sì, chi è la persona più adatta a portarci lì? Non conta tanto chi guida, ma l'obiettivo che ci accomunava, di solito la visione di un leader in cui tutti si allineano: ci crediamo ancora tutti, o è cambiato qualcosa? È il momento di fare autoanalisi: "ero qui per quella persona o per la visione?". Si dice, ed è vero, che **le persone non si dimettono dai posti di lavoro, ma da capi che non rispettano più**, perché spesso l'unica interfaccia con l'azienda è il proprio capo: le dimissioni non sono un messaggio per l'azienda, ma per il capo ("non voglio più lavorare con te"). Allo stesso modo, se non c'è più il leader della visione, bisogna prima capire se si condivide ancora qualcosa.

Lavorare così è più faticoso? Sì e no. "Non voglio demotivarvi, perché non è il mio lavoro", ma non è tutto rose e fiori: lavorare in questo modo è anche più stancante **emotivamente**, perché non si può più pensare "faccio il mio, alle cinque ho finito e chi si è visto si è visto". Si sente un senso di responsabilità verso gli altri. Succede che si finisca il proprio lavoro del pomeriggio, ci si giri e si veda un collega che finirà alle nove di sera, e l'istinto sarà "posso farti un pezzo del lavoro?": così si finisce entrambi alle sei. In un certo senso ci si accolla più lavoro, con la consapevolezza che quando si sarà in difficoltà ci sarà qualcuno che si girerà a dire "ti vedo stressato, posso darti una mano?". È bellissimo, ma non dà solo benefici diretti: richiede anche un investimento personale che non sempre le persone vogliono fare. Pro e contro: è il modo che sceglierebbe il relatore, ma non è per tutti. Conosce persone che vogliono poter staccare appena finito, e va benissimo: si cerca un'azienda in cui si possa lavorare così.

0.2.19.2 Condividere fallimenti e successi in pratica

Come si celebrano i fallimenti e si condividono i successi? Condividere i successi sembra più semplice ("bravo Mario che ha fatto questa cosa"). Il relatore dà uno spunto ancora migliore: **"bravo Mario, e ora Mario ci racconta come l'ha fatto e com'è andata"**. Sono sottigliezze, ma contano: raccontare in prima persona ciò che si è fatto dà un moto d'orgoglio che non dà sentirlo raccontare da un altro.

Per i fallimenti, che sembrano dover partire dalla persona stessa, l'obiettivo era che tutti li condividessero spontaneamente; ma all'inizio era difficile, perché non viene naturale alzare la mano e dire "ho sbagliato". Per questo il relatore ha creato artificialmente dei **"festival del fallimento"**, dal tono chiaramente ironico: durante una pausa pranzo, mentre tutti mangiavano insieme, a turno si saliva "sul palco" a raccontare la stupidaggine fatta in settimana. Molto spesso iniziava lui, raccontando i propri errori ("ho sbagliato questa cosa, sappiate che non si fa così"). Dopo aver forzato l'interazione per una, due, tre settimane, alla quarta lo facevano da soli: era diventato organico dire a tutti "ragazzi, attenzione, ho fatto questa sciocchezza, non fatela anche voi", e c'era perfino un **tag apposito su Slack**. È stato il team a decidere come condividere i fallimenti: come sempre, invece di imporre il modo, il relatore ha lasciato scegliere a loro, e funziona. È impegnativo tanto quanto lo si vuole rendere tale.

[33] La domanda non è leggibile nella trascrizione; dalla risposta si capisce che riguardava il caso in cui il leader che incarnava la visione del gruppo se ne va.

Che tipo di fallimenti si condividono? Anche quelli personali? Di solito si parte da quelli **professionali**, cioè da ciò che si riconosce come utile anche agli altri: qualcosa che ha rallentato, fatto perdere tempo o che rischia di ripetersi ("sappiate che, per evitare che lo facciate anche voi..."). Il resto è più da conversazione privata, *one to one*, con la persona con cui ci si vuole sfogare, o con gli amici: se non ha un potenziale impatto sugli altri, non c'è un incentivo a dirlo. E un errore che si è fatto per il proprio tipo di personalità? Qui tornano i test di personalità della prima giornata: conoscendo il proprio profilo e quello dei colleghi, si sa che qualcuno ha lo stesso carattere, e quindi lo si aiuta.

0.2.19.3 Cultura aziendale e formazione

Come si arriva a poter fare queste cose in azienda? Conta molto la cultura aziendale. Il relatore sa di lavorare bene perché oggi ha il *background* e i successi alle spalle per dimostrare che il suo metodo funziona, e quindi glielo lasciano fare; ma se dieci anni prima avesse proposto le stesse cose, "mi avrebbero mandato a quel paese". C'è un momento e un posto per fare le cose, e non è sempre possibile fare ciò che si ha in testa; ma ci si può lavorare. Per lui la domanda è diventata: "sono convinto che così sarebbe meglio; come faccio ad avere carta bianca?". **Portando risultati**: si è concentrato su quello, e ci è arrivato progressivamente.

Come funziona la formazione con workshop come questo, in azienda? Il relatore li fa in modo un po' più rilassato ed esteso, con molte più interazioni ed esempi, perché in azienda si possono usare esempi che si hanno davanti ogni giorno, e parlare di problemi e discussioni diventa molto più reale. Ma in generale è sempre **formazione pratica**, ripetuta finché il messaggio non è recepito. Il segno che qualcosa è stato recepito è quando arriva una persona nuova e **non è lui a spiegarle le cose**: è il team stesso a spiegarle che cos'è la feedback culture, e lui non deve parlare. È la conferma che il concetto viene perpetuato dal team.

Quanti manager dell'industria condividono questi principi? Il relatore ha fatto molti interventi all'IGDA di Roma e ne farà uno a Milano la settimana successiva ("il 24, mi pare"); interviene spesso a workshop ed eventi, e il riscontro è sempre "che bello, vorremmo farlo anche noi in Italia", e ci stanno lavorando. La moglie lavora in un'azienda che non ha niente a che fare con i videogiochi e lo stanno già applicando. Magari in Italia è tra i primi a parlarne, perché il paese è ancora un po' indietro, ma non è assolutamente l'unico: è più diffuso di quanto si pensi, non è una cosa di nicchia.

Questi spunti si applicano anche all'IT e alla consulenza? "Praticamente tutti." Si applicano facilmente quando c'è un gruppo di persone su cui applicarli; se si cambia continuamente cliente è più difficile portare questa mentalità da loro, ma questo non impedisce a chi lavora con noi di applicarla direttamente con noi.

Ci si può tenere in contatto? Su LinkedIn: il relatore accetta sempre tutti e risponde alle domande quando riesce (ne riceve tante, ma cerca di stare al passo almeno su base settimanale). E, "a costo di sembrare quello che si fa pubblicità", chi lavora in un'azienda che fa fatica ad applicare questi concetti o non sa che esistono può metterla in contatto con lui: è spesso così che ha trovato nuovi modi di applicarli. Vale anche per chi tra sei mesi lavorerà altrove. I suoi profili sono pubblici e non c'è nessun problema a contattarlo.

0.2.19.4 Performance e persone in difficoltà

Se un'azienda è a corto di personale, conviene comunque eliminare chi performa poco? Il giorno prima si era detto che conviene separarsi presto da chi non performa (sezione 0.1.2); ma non si finisce per caricare di lavoro tutti gli altri? Per il relatore **eliminare non vuol dire non sostituire**: vuol dire cambiare la persona con qualcuno magari più junior, ma più adatto come *fit* culturale. Il vecchio discorso del "fare lo stesso lavoro con meno gente" non è sostenibile: lo si può fare fino a un certo punto, in caso di necessità, ma non è una soluzione. Non è ottimizzare, è far andare in *burnout* chi resta.

Ha più senso tenere una persona poco performante e non in sintonia con il team, piuttosto che licenziarla senza poterla sostituire? No: se una persona è poco in sintonia, performa poco ed è in generale tossica e negativa per il team, **prima ci si separa, meglio è**. Questo non vuol dire non provare a recuperarla, ma bisogna andare dritti al punto: "abbiamo un problema: ti vedo poco performante, non ti vedo allineato con il team, vedo che il team non si fida di te. Che cosa vogliamo fare?". Se la risposta è "non lo so", la strada porta dritta all'uscita. Bisogna **recidere i rami morti** il prima possibile, facendo però attenzione a essere sicuri che si tratti di un ramo morto e non di un ramo recuperabile: tagliare un ramo recuperabile mette negli altri la paura di essere i prossimi, "alla prima foglia gialla". Quello che si vuole è che, quando un ramo è morto, la pianta smetta di alimentarlo, e ne crescerà un altro. Entrambi i messaggi devono essere chiari: **non si subisce passivamente la situazione**, e allo stesso tempo **non si abbandona nessuno** (se qualcuno è in difficoltà, la prima soluzione non è tagliare il ramo). È un compromesso tra le due cose. (A questo punto arriva, puntuale, il crash della connessione: "non era ancora successo oggi".)

Che cosa fare se qualcuno nel team sembra soffrire di depressione? È un discorso complesso, perché il relatore **non è uno psicologo**. Può dare la sua disponibilità a concedere permessi, coprire il lavoro della persona e in generale darle modo di seguire un percorso con uno specialista: non può risolvere la depressione, ma può alleviarne la parte lavorativa e permettere di ricevere supporto da chi è qualificato. È lo stesso motivo per cui, a Blizzard e a Riot, quando arrivavano ticket con minacce di suicidio ("mi avete bannato, quindi mi ammazzo", ticket purtroppo abbastanza comuni), il *customer service* era istruito a non intervenire direttamente: non dovevano essere loro a fermare la persona, perché non qualificati, ma indirizzarla a qualcuno di qualificato, prendendo tempo. Una persona non qualificata che prova a farlo rischia di peggiorare la situazione; come davanti a un incidente si chiama l'ambulanza e non si fa una tracheotomia. Per il relatore salute mentale e salute fisica sono la stessa cosa: come non si azzarderebbe a operare qualcuno di appendicite, non si azzarda a fargli da terapeuta.

Come si danno feedback adatti a ogni tipo di personalità? Bisognerebbe leggere i risultati del profilo della persona sullo stesso sito su cui si è fatto il test di personalità della prima giornata, che spiega anche come interagire con quel profilo. In generale: con chi è più **analitico**, portare dati a supporto; con chi è più **emotivo**, metterlo in una condizione di comfort; con chi è molto **pratico**, andare dritti al punto senza girarci intorno. Poi fare pratica. Oppure il modo "perfetto" per il relatore: provarci come viene e poi **chiedere** "andava bene così o preferisci in un altro modo? Potevo fare meglio?". Bisogna creare anche quel rapporto: fidarsi di discutere anche di queste cose.

Se nel team fossimo quasi tutti leader, non ci sarebbero "troppi galli"? No, perché per leader si intende chi si prende cura degli altri, e magari lo fossero tutti. Non significa che tutti scelgano che cosa fare: spesso lo sceglierà comunque il manager, o guiderà la scelta. Ma avere tutte persone con iniziativa e capaci di scegliere quando serve è qualcosa per cui il relatore "metterebbe la firma". Non ha mai corso il rischio dei "troppi galli": se ci sono quei caratteri, il team è stato assemblato male, perché non tutti sono compatibili con questo modo di lavorare e magari non si hanno le persone giuste per applicarlo.

Se la fiducia richiede tempo ed esperienza, quanto è difficile per un junior? Meno di quanto si pensi: proprio perché su un junior non ci sono aspettative, un junior che chiede "posso darti una mano?" dà un segno chiaro che tiene più al team che a se stesso. Si può essere leader anche non dal punto di vista professionale, ma come chi va a dare supporto agli altri: "il medico del team, l'*healer*", e lo si può fare anche da junior. È così che ha iniziato il relatore: andare ad aiutare a risolvere problemi non suoi gli ha fatto accelerare molto il percorso per ottenere rispetto e stima, e lo ha esposto a cose a cui altrimenti non sarebbe mai arrivato. È una questione di carattere: se ci si tiene agli altri, lo si fa vedere, e il resto arriva come risultato. "È quasi un reddito passivo": un investimento nel far vedere che ci si è, e il resto arriva ("Stefano si è preso cura di me, quindi ora mi prendo cura di lui").

Aiutare gli altri solo dopo aver finito il proprio lavoro? Dipende dall'urgenza. Il relatore spesso, se qualcosa è davvero urgente, mette in pausa ciò che sta facendo, risolve e poi torna al suo lavoro; ma è

troppo *case by case*. Se due colleghi litigano su chi debba avere le ferie di Natale, lo si può risolvere anche domani; se litigano perché si è perso il *repository* e domani c'è la consegna, lo si risolve subito.

Come mantenere motivato un team su progetti non retribuiti, come quelli accademici? Essendo il più **trasparenti** possibile nei processi decisionali: se si deve decidere se fare un action o un RPG, tutti devono sapere perché si fa l'uno invece dell'altro (da qui il metodo di brainstorming della mattina). Deve essere sempre chiaro a tutti perché si stanno facendo le cose in quel modo, perché **la demotivazione nasce dal disallineamento** rispetto all'obiettivo del gruppo, e l'unico modo di essere allineati è essere sempre consapevoli e partecipi del processo che decide dove sta andando il gruppo. E soprattutto: **non chiudere mai un meeting senza aver concordato che cosa si vuole fare** e senza essere tutti d'accordo. Se non lo si è, bisogna continuare a parlarne, per quanto scomodo, lungo e noioso: si deve sempre uscire da un meeting sapendo che si è tutti d'accordo, o al massimo in *disagree and commit* (sezione 0.2.12). "Quello è il massimo accettabile."

0.2.19.5 Carriera, colloqui e candidature

Come ha iniziato a lavorare all'estero? Negli annunci online si trovano spesso requisiti come un titolo di studio conseguito nel paese o l'abilitazione a lavorarci. Questo vale per i paesi fuori dall'Unione europea: per lavorare negli Stati Uniti o in Sud America serve un visto o una *green card*. In molti casi, però, è proprio ottenere il lavoro a permettere di lavorare lì: quando il relatore doveva andare a lavorare in California (a Irvine, a Santa Monica, dove capitava), era l'azienda a occuparsi dei permessi necessari, perché "per venire qui devi avere un lavoro, e il lavoro te lo diamo noi".

Il suo inizio è stato abbastanza semplice. In Italia lavorava per Digital Bros, occupandosi dei giochi online gestiti dall'azienda e di quello che era il canale Game Network sul digitale terrestre[34]. Vide un annuncio a Dublino per *Warhammer Online*, mandò la candidatura, fece i colloqui e gli dissero "ti paghiamo la *relocation*, vieni qui". Come detto il giorno prima, sono le aziende a occuparsi del trasferimento: se devono creare a Dublino un'unità che segue dodici lingue europee, sanno di dover far arrivare persone da fuori, ed è già a budget. Non è nemmeno una richiesta da fare: è prevista. Per il relatore, da questo punto di vista, è indifferente lavorare per un'azienda italiana o estera: se si è adatti, si è adatti. Per questo insisteva sull'inglese: se lo si parla, è indifferente di che cosa ci si occupi e quale sia la propria lingua madre, a meno che la posizione non sia esplicitamente "community manager italiano". Un programmatore polacco o italiano non cambia niente. Lavorare all'estero non è più difficile: c'è più concorrenza solo se l'azienda cerca attivamente persone da tutto il mondo e non si pone problemi di *location*. Cambiano l'azienda, il fatto di lavorare da remoto o dal vivo, l'eventuale trasferimento; per il resto non cambia molto.

Gli anni passati nell'IT contano se si vuole passare ai videogiochi? Dipende da che cosa si vuole fare: se da responsabile IT si vuole diventare programmatore, bisogna saper dimostrare di saper programmare. Vale anche per i passaggi interni: se un game designer vuole diventare art director, al relatore va bene, deve solo dimostrare di saper fare l'art director. Anzi, incentiva le **promozioni interne**, perché si conosce già il candidato; ma gli spostamenti non sono liberi "a sentimento": bisogna saper fare quella cosa. I recruiter valuteranno non tanto gli anni quanto ciò che si sa fare; gli anni servono comunque, perché dimostrano di saper lavorare in un ambiente professionale, che è un altro insieme di competenze. Ma se si sa fare ciò che chiedono, aver lavorato nell'azienda X o Y non dovrebbe cambiare molto.

Quanto vengono considerate le candidature spontanee? Non tantissimo. Soprattutto per le aziende non grandi, se non stanno cercando qualcuno è perché non se lo possono permettere: una candidatura spontanea non le convincerà ad assumere. Più l'azienda è piccola, più conviene aspettare l'annuncio, perché se non c'è vuol dire che non c'è budget, e potrebbero rispondere "ti vorremmo, ma non possiamo prenderti". C'è anche il rischio che finisca nel vuoto (l'HR magari non controlla nemmeno la mail, perché non sta assumendo) o, peggio, che rispondano "non ci servi" e una settimana dopo esca la posizione, con il candidato già nel cestino dei rifiutati. Si rischia di **bruciarsi** la possibilità.

[34] Questo passaggio chiarisce il dubbio segnalato nel capitolo 0.1 sul "Digital Game Network Channel": si tratta di Game Network, canale televisivo dedicato ai videogiochi legato al gruppo Digital Bros.

Come ci si accorge di essere riconosciuti come leader, e come lo si "vende" a un colloquio? Uno studente osserva che la leadership, così descritta, non la si riconosce da sé ma la riconoscono gli altri. Internamente il relatore non ha mai dovuto "venderla", perché parlavano i risultati. Finché non lo si vive è difficile capirlo, ma quando si arriva alla posizione in cui il team rispetta e vede qualcuno come punto di riferimento, non ci sono grandi dubbi: è evidente che vengono da lui. Si può riflettere su **quanto spesso gli altri vengono da noi**, cercano di emulare il nostro modo di fare, vanno a rivedere le nostre cose per capire dove hanno sbagliato, chiedono un parere, ci parlano di un problema: la somma di tutto questo significa molto spesso che si fidano. Per "venderla" all'esterno si può dire di essersi occupati di un team in cui, attraverso il lavoro e il rispetto degli altri, si è arrivati a essere visti come punto di riferimento; ma il suggerimento è **portare esempi tangibili** ("c'era questo progetto, non sapevamo come risolverlo, il team mi ha chiesto una mano e l'ho risolto così"), invece di "incensarsi" dicendo "sono il punto di riferimento del team". Se il relatore racconta di essere stato eletto rappresentante dei licenziati senza essere tra loro, un'azienda capisce che si fidavano di lui, punto: non ci sono altre conclusioni da trarre.

Quali sono gli errori più gravi dei junior ai colloqui, a livello di soft skill? In ordine di priorità:

1. **non arrivare preparati**, cioè **senza esempi**. Il relatore ha sempre un esempio pronto per ogni cosa di cui vuole parlare: per la fiducia, per gli *achievement*, per i risultati. Servono sempre esempi tangibili per far capire che cosa si intende;

2. **non avere domande alla fine**, perché dimostra di non avere chiaro che cosa si andrebbe a fare. Anche solo chiedere che cosa si farebbe, che cosa si può migliorare, o "che impressioni hai avuto finora dal mio colloquio?": si possono fare domande molto aperte al recruiter per capire come sta andando;

3. **non leggere bene la *job description***: presentarsi con vaghezza, senza aver capito che cosa cerca l'azienda. Lo si impara anche con la pratica; ma il designer che si candida per una posizione da programmatore, senza nulla nel CV che faccia pensare che sappia anche programmare, il relatore non solo lo scarta: lo mette in una **blacklist** di persone che forse non considererà mai. Se non si riesce a interpretare un testo come una *job description*, è un problema troppo grosso da superare in una collaborazione;

4. **errori di battitura e CV impaginati male**, che molti sottovalutano. Il relatore ha letto letteralmente, in una lettera di presentazione per una posizione da *copywriter*, la frase "sono una persona molto attenta ai dettaggi". Come si può considerare quella persona per quel ruolo? Non ha nemmeno incollato la lettera in Word per il controllo automatico: non è un errore sfuggito al correttore perché la parola esiste con un altro senso, "dettaggi" viene segnato in rosso in qualsiasi lingua. Come affidarle la comunicazione aziendale? Vale per tutti gli errori: ciò che si manda è **il primo biglietto da visita** con l'azienda, e non si manda un biglietto da visita sgrammaticato. Un conto è non andare ai colloqui in giacca e cravatta ("non l'ho mai fatto e probabilmente non lo farò mai"), un altro è presentarsi con errori;

5. un altro esempio comune: **l'email sbagliata nel CV**. Se la mail torna indietro, il relatore non telefona per farsi dare quella corretta: se non si è in grado di scrivere giusta la propria email, non vale la pena. Più l'errore è banale e facilmente evitabile, più lo fa arrabbiare ("scusate se mi sono inalberato"): evidentemente il candidato non teneva abbastanza a quel posto.

Quanto è importante la lettera di presentazione? Sempre di più: sempre più aziende la richiedono come obbligatoria, quindi conviene prepararne una, sapendo di doverla adattare al destinatario. Deve essere **meno lunga di una pagina**, in **tre paragrafi**:

1. **chi siete**: un minimo di *backstory*, che cosa avete fatto, che cosa volete fare, da dove arrivate;

2. **perché avete mandato il CV**: che cosa vi ha interessato di questa azienda;

3. **perché dovrebbero chiamare voi**: che cosa dovrebbe spingere l'azienda a scegliere proprio voi.

Uno studente dice di riuscire bene con i primi due, ma di trovare difficile il terzo, che è poi la domanda che tutte le aziende fanno. In realtà basta **collegarsi alla *job description***: se cercano un 3D artist in grado di fare X, Y e Z, si conferma di averlo letto e si spiega che con la propria esperienza si è imparato a fare proprio quelle cose. Non vogliono sentire niente di particolare, "non devi cambiargli il mondo".

Che cosa guarda un recruiter in un portfolio? Il relatore li guarda poco: fa un *sanity check* generale, per vedere che non sia evidente perfino a lui, "a digiuno di tante cose tecniche", che c'è un problema (se se ne accorge perfino lui, il problema c'è). Per il resto guarda che il portfolio sia **coerente con il CV**: se arriva un CV da senior con dodici progetti completati e anni di lavoro, e il portfolio sono quattro immagini, non è coerente. Vuole una concordanza tra ciò che si racconta e ciò che si vede.

Che strumenti per creare un CV? **Canva**, che ha un catalogo di template gratuiti e a pagamento amplissimo e personalizzabile: anche se non piace, lo si prende e lo si adatta, ma fa già il 90% del lavoro. Insieme a **ChatGPT**, perché "è inutile far finta che non esista": ci si fa scrivere la lettera di presentazione e poi la si corregge e la si adatta, e lo stesso per il CV. L'importante è renderli adatti e coerenti, non lasciarli solo frutto di ChatGPT; ma per fare il grosso del lavoro è un ottimo strumento.

Conviene lasciare nel CV esperienze non del settore? Uno studente ha alle spalle il liceo, vari stage e lavori che non c'entrano niente con la modellazione 3D. Si possono mettere finché c'è spazio: il relatore consiglia **al massimo due pagine**, e quando ha cominciato a sforare nella terza ha iniziato a togliere ciò che non era più rilevante. Ormai nel suo CV la formazione è un paragrafo: ha così tante esperienze lavorative che non interessa a nessuno che abbia fatto il liceo scientifico o classico, e non lo mette nemmeno più. Si filtra in base a ciò che è più rilevante per lo spazio disponibile.

Che cosa fare con le aziende che dicono "ti faremo sapere" e poi spariscono? Non aspettare uno o due anni per scrivere. Il suggerimento: tra le domande finali del colloquio, l'ultima del relatore è sempre **"tra quanto mi fate sapere?"**. Se rispondono dieci giorni, all'undicesimo scrive: "mi avete detto dieci". Non si sente in difetto a mettere pressione, perché sono stati loro a dare la scadenza. E se viene ignorato del tutto, quell'azienda per lui è "morta": non si ricandiderà mai, perché non vede perché il rispetto debba andare in una sola direzione.

Quanto essere trasparenti, nel CV e nella lettera, su problemi personali come l'ADHD? Uno studente con ADHD[35] dice di tendere ad avere performance altalenanti. Il relatore lo invita a **non usare l'ADHD come scusa** per essere altalenante: anche lui ha l'ADHD, ma non è altalenante. Esserlo è senz'altro un *malus*, ma esistono i mezzi per non avere problemi di questo tipo: lui ha trovato i suoi metodi (la to-do list, la *must do list*, il delegare il lavoro al team, perché occuparsi delle cose per gli altri lo mantiene molto motivato, sveglio, all'erta, attento a come vanno le cose). Sta anche a ciascuno trovare un lavoro in cui compensare ciò che viene richiesto: se domani lo rimettessero a fare il programmatore, come all'inizio, probabilmente impazzirebbe, perché non riusciva a restare concentrato sul codice oltre un certo tempo. Lì ha capito che non poteva fare il programmatore, ma ha trovato qualcos'altro che gli permette di essere com'è e fare ciò che fa. Bisogna capire i propri punti di forza e di debolezza e giocare su quelli: si possono avere tutti i difetti del mondo e voler comunque fare una cosa, ma bisogna accettare che sarà più difficile.

Quanto alla trasparenza, il consiglio è **non mentire mai nel CV**, "perché tanto vi beccano". E per il discorso della *bandwidth*: tutte le energie spese a mentire, e a tenere viva la bugia, sono energie tolte allo spiegare bene ciò che si sa fare. È un investimento nel posto sbagliato. Bisogna concentrarsi su ciò che si sa fare, con esempi tangibili, e dire "non lo so fare" quando non lo si sa fare. Il peggio non è essere scoperti, ma **non esserlo**: se si finge di sapere il C++, si viene assunti e messi a lavorare in C++, che cosa si fa il giorno dopo? Si rischia davvero di rovinarsi la carriera. Se non si è un *fit*, si sarà il *fit* per la prossima. L'incubo del relatore è essere assunto in un posto dove pensano che sappia fare qualcosa che non sa fare; anzi, tende a esplicitarlo: "cercate questa cosa? Io non la so fare, va bene lo stesso?" — "Sì, te la insegniamo noi" — "Voglio solo che sappiate che non la so fare". Meglio questo che vivere con l'ansia che prima o poi si accorgano che non sa programmare.

[35] Nella trascrizione "essendo 10D"; nel resto della risposta "di" e "dei 10D" stanno per ADHD.

Gli capita spesso con chi mente sull'inglese. Al primo colloquio chiede "sai parlare inglese? A che livello?" — "C1, *fluent*, quasi madrelingua" — "Perfetto", e lascia cadere l'argomento. All'ultimo colloquio: "ti va se lo facciamo in inglese?" — "No, ma io intendevo che..." — "No, mi avevi detto *fluent*: ora facciamo il colloquio in inglese". Prima o poi le bugie raggiungono chi le dice.

0.2.20 Chiusura del workshop

Il relatore chiede alla classe di far sapere com'è andata a Vanessa, il punto di contatto con l'accademia per il workshop: sarà lei a riferirgli eventuali lamentele o suggerimenti, che gli permetteranno di fare meglio i prossimi. Ringrazia per aver "retto per dodici ore" e spera che sia stato utile, consapevole che tutti sono stanchi quanto lui.

Gli studenti rispondono che è stato chiarificatore su molte cose che non sapevano, soprattutto su come interfacciarsi nei team, e che sarà materiale da rivedere per fissare bene i concetti, vista la mole in due giorni; qualcuno si aspettava che sarebbe stato molto più pesante e l'ha trovato invece scorrevole. Il relatore ribadisce che **niente sostituirà l'esperienza diretta**: il workshop dà un'infarinatura e un punto di vista, ma i concetti si metteranno in pratica solo nel *day to day*. Bisogna avere **pazienza con se stessi**, sapendo che non verrà tutto bene dal primo giorno. Confessa di essere stato un po' preoccupato per la durata, ma il materiale c'era, e farlo in un giorno avrebbe rischiato di trasformarlo in una corsa: "rischiava di essere uno *sprint*, e invece è una maratona".

Infine ricorda che si può aggiungerlo su LinkedIn, che registrazione e slide (in forma integrale) verranno fatte avere tramite Vanessa ("dite che vi ho autorizzato io a chiederle") e che, per qualsiasi dubbio, resta a disposizione. "Buon resto del weekend."

Punti chiave del capitolo

- I colloqui sono una **selezione reciproca**. L'annuncio dice molto sulla cultura aziendale; alla fine del colloquio vanno sempre fatte domande (il *day to day*, la "bacchetta magica", "tra quanto mi fate sapere?"). Si arriva con una **cifra minima mentale** e con **esempi tangibili**; non si mente mai.

- Time management non è una lista di cose da fare: si parte dalla **must do list**, che è anche il primo momento per dire "oggi potrei non farcela"; si stimano i tempi e si chiede quando qualcosa non è chiaro.

- Ogni nuovo task va **prima aggiunto alla lista**, non eseguito: priorità non è recenza. Le **checklist** fanno risparmiare tempo, anche (e soprattutto) ai più esperti.

- **First contact resolution**: raccogliere tutto ciò che serve in una sola richiesta, e non iniziare un task senza le informazioni necessarie. Ogni interruzione costa dieci-quindici minuti di concentrazione.

- **Matrice di Eisenhower**: fai, pianifica, delega, elimina. "Non importante" significa che non è importante che lo faccia tu; il quadrante dimenticato è *delete*.

- Le pause vanno **scelte, non subite**; i task grossi vanno spezzettati; bisogna **saper dire di no**, anche ai meeting (sono necessario? può andarci uno solo? ci sarà un resoconto scritto? per che cos'altro potrei usare questo tempo?). **Il multitasking non esiste**.

- A fine giornata si verifica la *must do list*: un *must do* rimandabile non era un *must do*.

- Ferie illimitate con un minimo, giornate libere e feedback culture sono, per il relatore, le condizioni per un team sano; il benessere è anche mentale.

- Il **brainstorming genera idee, non decide**. Va preparato (topic, obiettivo, vincoli), separa la **generazione** dalla **discussione** (*rapid ideation* di cinque minuti, *dot voting* anonimo) e combatte l'*anchoring* e il dominio degli estroversi.

- La decisione si prende con la **matrice impatto–sforzo**: prima le idee a basso sforzo e alto impatto, poi in diagonale. Lo sforzo sono le risorse, l'impatto comprende anche la motivazione; decide chi farà il lavoro.
- **Disagree and commit**: si dichiara il disaccordo, poi ci si impegna al massimo. Se va male ha sbagliato il team, non "ve l'avevo detto".
- **La leadership non è un titolo**: il rispetto si guadagna. Le carriere vanno divise tra **manager** e **subject matter expert**, pagati allo stesso modo, così che diventi manager solo chi vuole prendersi cura degli altri.
- Dopamina, serotonina e ossitocina legano il gruppo; il **cortisolo** è contagioso e lo disgrega. Il "contratto sociale" del villaggio vale ancora: chi protegge il gruppo riceve di più, chi ne abusa tradisce.
- La **piramide della fiducia**: integrità (aiutare davvero, onestà, ammettere gli errori), affidabilità (mantenere la parola, saper rifiutare), leadership (dare l'esempio, offrirsi anche quando si rischia, non lasciare indietro nessuno).
- Il gruppo deve prendersi cura del gruppo: **raccogliere il piattino**, smettere di dire "ci penserà qualcun altro", segnalare e chiedere prima di intervenire.
- Da manager: delegare i meriti e prendersi le colpe, lasciare la comfort zone, chiedere "tu come faresti?", celebrare successi e **fallimenti**, dare il merito a chi lo ha. Essere leader è un **impegno a lungo termine**.
- L'obiettivo: tornare a casa **ispirati, sicuri e realizzati**, come diritto di tutti e non privilegio di pochi.

Chapter 0.3

Agile e Scrum nella pratica: l'esperienza di chi ci è passato

Le prime due lezioni zero (capitoli 0.1 e 0.2) guardavano al lavoro in team dal punto di vista di un professionista con quindici anni di industria alle spalle. Questa terza lezione cambia prospettiva: a parlare non è un veterano, ma **due persone che hanno fatto lo stesso percorso di studi** di chi ascolta e che, meglio di chiunque altro fino a quel momento, sono riuscite a *internalizzare* Agile e Scrum nei progetti dell'accademia.

La lezione è introdotta e chiusa da Diego Ricchiuti[1], che ha già spiegato alla classe la teoria di Agile e Scrum e che interviene di tanto in tanto per sottolineare o completare un passaggio. Gli ospiti sono due:

- **Simone Cicchetti**, ex studente dell'accademia, oggi level designer e producer, che parla per circa un'ora e mezza della teoria "vista dalla vita reale" e degli errori più comuni;
- **Cristian Liberatore**[2], studente del terzo anno, game designer e project manager di due progetti nati come progetti d'anno, che segue passo per passo il ciclo di Scrum con gli esempi del suo gioco.

Diego li ringrazia per il tempo dedicato: Simone lavora, quindi sta sacrificando tempo personale, e Cristian è uno studente con lo stesso carico di lavoro della classe. Sono stati scelti perché, tra gli studenti avuti fino a quel momento, sono quelli che hanno davvero **internalizzato** Scrum e Agile, ottenendo grandi risultati produttivi nel progetto scolastico e, nel caso di Simone, anche dopo l'accademia[3].

Perché insistere sulla parola *internalizzare*? Perché, dice Diego, "utilizzare Scrum e Agile sembra molto facile, ma poi in realtà quando li provi ad applicare veramente ti rendi conto che facili come sembravano quando facciamo lezione non lo sono". Se invece li si internalizza davvero, si ottengono grandi risultati. L'idea della lezione è che possa essere utile vedere l'esperienza di qualcuno *molto più vicino* agli studenti di quanto lo sia il docente, che ormai usa questi strumenti da anni. Il formato annunciato è di circa un'ora e mezza per ciascun ospite, domande comprese.

Due avvertenze di lettura. La prima: i due interventi sono fatti di teoria richiamata in breve e di moltissimi esempi concreti, spesso con numeri, errori e ripensamenti; sono riportati con cura perché sono proprio la parte che la teoria non dà. La seconda: i due relatori usano in modo intercambiabile alcuni termini (*framework* e metodologia, *post-mortem* e revisione della sprint, *score*, peso e punteggio dei task), e il capitolo segue il loro uso, segnalando in nota dove si discosta dalla terminologia ufficiale di Scrum.

Il percorso del capitolo segue quello della lezione:

[1] Nella trascrizione compare soltanto il nome di battesimo, "Diego", con cui i due ospiti si rivolgono più volte a lui; il cognome è quello indicato per il direttore dell'accademia nel capitolo 0.1. Dal contesto è anche il docente che ha già spiegato Scrum e Agile alla classe nelle lezioni precedenti ("Diego vi avrà già ampiamente indirizzato", "come avete fatto con Diego nelle lezioni scorse").

[2] Nella trascrizione il nome compare sia come "Cristian" sia come "Christian"; qui si usa la prima forma, quella con cui viene presentato all'inizio della lezione. Anche i nomi dell'accademia sono resi in modi diversi ("Hive", "a dive", "IVE Online", "Live Online"): come negli altri capitoli si usa semplicemente "l'accademia"; "IVE Online" sembra indicare il corso a distanza dell'accademia.

[3] La trascrizione dice "nel caso di Christian post-accademia", ma è Simone l'ex studente che oggi lavora, mentre Cristian è ancora iscritto: il riferimento al dopo-accademia è stato quindi attribuito a Simone.

1. un'apertura organizzativa e la presentazione degli ospiti (sezione 0.3.1);
2. **Simone**: chi è, un sondaggio sulla classe e le prime esperienze di Scrum degli studenti (sezione 0.3.2);
3. il **mindset Agile**: perché non è un framework e perché va adattato (sezione 0.3.3);
4. le **sprint** e le riunioni di pianificazione e *post-mortem* (sezione 0.3.4);
5. le **cerimonie** e la regola della *consistency*, con le critiche al *daily scrum* e al *planning poker* (sezione 0.3.5);
6. il **backlog** e gli obiettivi dei task (sezione 0.3.6);
7. la **velocity**: perché non usare le ore, la sequenza di Fibonacci e come la velocity permette di "prevedere il futuro" (sezione 0.3.7);
8. il riepilogo della creazione di una sprint e la conclusione sulla **fiducia** (sezione 0.3.8);
9. le domande a Simone, con la costruzione in diretta di un foglio per il calcolo della velocity (sezione 0.3.9);
10. **Cristian**: chi è e come imposta la sua parte (sezione 0.3.10);
11. il caso di studio, il gioco *L'Inf*, e il "trauma" del cambio di team (sezione 0.3.11);
12. il **loop di Scrum** in cinque fasi (sezione 0.3.12): la stesura del backlog (sezione 0.3.13), la selezione dei task (sezione 0.3.14), l'implementazione con i suoi controlli, i *blocker* e i feedback (sezione 0.3.15), il calcolo della velocity (sezione 0.3.16) e il *tuning* della produzione, con il problema del perfezionismo (sezione 0.3.17);
13. una domanda su come comunicare un taglio al team (sezione 0.3.18);
14. il **team management**: il team prima di Scrum, la produzione senza stress, il mindset condiviso, il *post-mortem* e la **sincerità** (sezione 0.3.19);
15. la chiusura della lezione (sezione 0.3.20).

0.3.1 Apertura e presentazione degli ospiti

0.3.1.1 Una registrazione andata persa

La lezione si apre con una comunicazione organizzativa[4]. Il programma usato per registrare una lezione precedente registra in una modalità che va convertita alla fine; durante la conversione il computer di Diego, probabilmente per colpa degli aggiornamenti di Windows, si è bloccato e la conversione è "andata a pallino". Poiché si tratta di una lezione importante, Diego si rende disponibile a **rifarla interamente** per registrarla di nuovo oppure, come la classe preferisce, a fornire un audio, un video o qualunque altro formato sia più comodo, che poi caricherà. Chiede agli studenti di pensarci e di fargli sapere.

0.3.1.2 Chi sono gli ospiti e perché sono qui

Poi Diego presenta gli ospiti speciali della giornata: Cristian Liberatore, studente del terzo anno, e Simone Cicchetti, ex studente dell'accademia (che in un primo momento "sparisce" dalla griglia della videochiamata, finché Diego non lo ritrova in alto a destra). Le ragioni della scelta, il valore della parola *internalizzare* e il formato (un'ora e mezza ciascuno, con domande) sono quelli già riassunti all'inizio del capitolo. Detto questo, Diego si silenzia e lascia la parola a Simone.

[4]La trascrizione comincia a frase già iniziata ("in una modalità non video fin tanto che ce la chiamate poi la converte alla fine"): l'inizio della spiegazione non è stato registrato e non è chiaro a quale lezione precedente si riferisca.

0.3.2 Simone: il punto di vista del producer

0.3.2.1 Chi è il relatore

Simone si presenta mentre condivide lo schermo ("adesso tutto è imploso, perfetto"). È uno degli ex studenti del terzo anno e oggi lavora come **level designer**; collabora con Diego su alcuni progetti legati alla gestione dei team, al *game design*, al *framework design* e ad altre "cose molto fighe" che alcuni studenti forse conoscono[5]. Sta inoltre continuando a lavorare al progetto del suo terzo anno (il *Biggest One*[6]), un gioco chiamato *ISO*[7], per il quale è level designer, game designer, social media manager e soprattutto **producer**: si occupa di gestire il lavoro delle persone, di far rispettare le cerimonie di Scrum e, "punto centrale della questione", di "rompere le scatole alla gente".

0.3.2.2 Una lezione partecipativa

Il tempo a disposizione è molto, quindi Simone vuole una lezione il più possibile **partecipativa e cooperativa**: a differenza del solito, le domande vanno fatte in qualsiasi momento, scrivendo in chat per attirare la sua attenzione o alzando la "manina" con le emoji, che controlla su un secondo monitor con la griglia dei partecipanti. Al massimo aspetterà la fine della slide per non perdere il filo.

L'obiettivo dichiarato non è spiegare la teoria, che la classe ha già fatto con Diego, ma darne un breve *recap* e poi spiegare **a che cosa serve nel pratico e com'è nella vita reale**: "come spesso accade, la teoria è una cosa, la vita reale è un'altra".

0.3.2.3 Un piccolo sondaggio

Prima di partire, Simone chiede chi abbia "un amore incommensurabile e infinito" per il producing, lo *scrum mastering* e il management in generale: nessuno, e uno studente, Gavi[8], risponde spammando il "no". Poi la domanda opposta: chi lo odia così tanto da preferire "affrontare un alligatore in una lotta greco-romana"? Ancora nessuno. La classe è quindi, in generale, "tutta intermedia".

Chi si è già dovuto occupare di queste cose nei progetti d'anno o nei lavori di gruppo? Rispondono due studenti, Mattia e Lorenzo[9], ognuno per il proprio team. Simone chiede solo i punti salienti: che cosa è andato bene, che cosa no e perché.

L'esperienza di Mattia. Il suo gruppo usa Scrum da pochissimo, circa una settimana. È lui a gestire la *Scrum board*, con l'aiuto degli altri due membri, ma il metodo non è ancora applicato a pieno. Mattia è interessato a Scrum, ma la **responsabilità di guidare altre persone** oltre a se stesso gli pesa parecchio, e usare Scrum in modo serrato tutti i giorni gli fa "una bella paura": se qualche meccanismo si rompe, bisognerebbe correre ai ripari. Che cosa intende per "rompere un meccanismo"? Che se una cerimonia non viene rispettata, tornare allo *status quo* non sarebbe semplice, anche per la sua mancanza di esperienza nell'organizzare i team. Simone riassume: la preoccupazione è più sul piano umano che su quello delle ore di lavoro necessarie. Mattia riconosce che Scrum avrà le sue "quasi magiche proprietà", ma non avendolo ancora visto all'opera nella vita reale non può dirlo con certezza.

L'esperienza di Lorenzo. Niente di diverso: anche il suo gruppo ha iniziato da pochissimo e proprio il giorno prima ha fatto la prima riunione di Scrum, decidendo insieme i pesi dei task. Alla domanda se il ruolo sia arrivato per voto popolare, Lorenzo risponde che si è offerto lui: conoscere un *framework* di lavoro, e soprattutto capire come applicarlo, gli interessa parecchio.

[5] Il passaggio è confuso nella trascrizione ("lavoro con diego alcune alcuni progetti diciamo sul gestione di pro dei team game design framework design"): la ricostruzione è approssimativa. Più avanti Simone dice anche di lavorare per una società australiana.

[6] Dalla lezione si capisce che i progetti d'anno dell'accademia si chiamano *Big One* (primo anno), *Bigger One* (secondo anno) e *Biggest One* (terzo anno); la trascrizione li scrive in vari modi ("biggest one", "bigger one", "BiggerOne").

[7] Il nome del gioco compare sempre come "ISO"; non è possibile stabilire dalla sola trascrizione se sia la grafia corretta (potrebbe essere, ad esempio, *Iso* o una sigla).

[8] Così nella trascrizione; potrebbe trattarsi di un soprannome o di un nome storpiato.

[9] Nella trascrizione "da noi Mattia, Podini" e "da noi Lorenzoaldasso": i cognomi non sono ricostruibili con sicurezza ("Podini" potrebbe essere il cognome di Mattia o il nome di un altro studente).

Simone trova molto interessante quanto detto da Mattia sulle cerimonie e sul doverle rispettare, e promette di tornarci più avanti (sezione 0.3.5).

0.3.3 Il mindset Agile

0.3.3.1 Agile non è un framework

La prima slide parte da una frase che tutti conoscono: **Agile non è un framework, è un mindset** (Simone dice di aver "percepito la furia negli occhi di Diego" quando la classe esitava a rispondere). La frase sembra una di quelle che servono "per fare i fighi con i colleghi", ma ha un significato preciso: un *framework* presuppone **regole da seguire**, mentre Agile **non ha regole**. È una serie di **linee guida da internalizzare**: la parola chiave è che devono diventare *interne*, al punto da non dover più pensare attivamente a quali siano le regole, i punti o la mentalità di Agile. Bisogna capire la logica che c'è sotto.

L'analogia è quella della **lingua straniera**: all'inizio si ragiona in italiano, si costruisce la frase in italiano, la si traduce e la si dice; man mano questi passaggi si annullano e si comincia a ragionare direttamente nella nuova lingua. Con Agile funziona allo stesso modo.

0.3.3.2 I quattro valori, e perché non vanno presi alla lettera

Simone rilegge rapidamente i quattro valori del manifesto Agile:

1. gli **individui e le interazioni** sono più importanti dei processi e degli strumenti;
2. un **prodotto funzionante** è più importante di una documentazione esaustiva[10];
3. la **collaborazione con il cliente** è più importante della negoziazione del contratto;
4. **rispondere al cambiamento** è più importante di seguire un piano.

Come è scritto sotto al manifesto stesso, questo non significa che le cose a destra, quelle "meno importanti", non contino o non vadano seguite. Ma Simone va oltre: **non significa nemmeno che queste regole debbano essere seguite davvero**. Non è detto che individui e interazioni debbano essere per forza più importanti di processi e strumenti. Il punto chiave di Agile è *sentire* quale sia il procedimento e la mentalità giusti e **adattare Agile al proprio team**, perché ogni team e ogni progetto è diverso.

Se un team e un progetto, per vari motivi, hanno bisogno di pagine e pagine di documentazione, allora per loro la documentazione è più importante del prodotto funzionante. Gli esempi sono infiniti; uno è il **gioco di carte**: lì il prodotto funzionante è relativamente meno importante della documentazione, perché le logiche si possono scrivere in qualsiasi momento, ma senza una documentazione solida a supporto "cadrà tutto quanto immediatamente". Quindi non bisogna seguire "a menadito" i punti del manifesto, ma capirne la logica di fondo, capire **a che cosa serve** e capire **se serve**: non è detto che Agile serva.

0.3.3.3 Altri mindset e i framework

Esistono molti altri *mindset*. Di recente va molto di moda la mentalità **lean**, che Simone definisce "già il 2.0"[11]. L'importante, aggiunge con Diego, è **non usare Waterfall**, "perché Waterfall nel gaming... non dirò nient'altro".

Sulla base di Agile sono state create delle **metodologie**, che Simone si corregge chiamando *framework*: e queste sì che hanno delle regole. L'esempio è Scrum, ma ce ne sono infiniti, come Kanban. Si basano tutti sul principio Agile dell'**inspect and adapt** (ispeziona e adattati, analizza e adattati), che si può riassumere in una parola chiave: **iterazioni**. Scrum è completamente basato sulle iterazioni, e anche lo sviluppo dei videogiochi lo è, "molto più dello sviluppo software".

[10] Nella lezione "documentazione comprensibile"; nella formulazione corrente del manifesto Agile il termine è *comprehensive documentation*, cioè documentazione esaustiva.

[11] Così nella lezione. Storicamente il *lean* nasce nella produzione industriale (il sistema Toyota) e precede Agile; nello sviluppo software il *lean software development* si è diffuso negli anni Duemila. Il "2.0" va letto come una battuta sul fatto che oggi è di moda, non come una derivazione da Agile.

Il resto della sua parte è dedicato ai punti di Scrum, il *framework* da cui presumibilmente partono tutti gli studenti, e agli **errori classici** in cui si incappa spesso. L'ideale sarebbe trasmettere la logica alla base di tutto, ma Simone avverte che sarà difficile, perché "sono cose che si iniziano a capire mettendole in pratica". Per questo invita di nuovo a fare domande sulle proprie esperienze.

0.3.4 Le sprint e le riunioni

0.3.4.1 Il libro di Scrum

Simone mostra il libro in cui viene raccontata la genesi del *framework* Scrum, scritto da Jeff Sutherland[12]: un "libricino" di circa trecento pagine. Avverte che userà le parole "metodologia" e "framework" in modo intercambiabile. Lo consiglia assolutamente a chi è interessato al tema, perché è scorrevole e contiene *insight* molto interessanti; chi non è particolarmente interessato può evitarlo del tutto, perché non avrà niente da offrire che non venga detto in questa lezione.

0.3.4.2 Che cos'è una sprint

Scrum, a differenza di Agile, è un *framework* a tutti gli effetti: ci sono regole da seguire in modo preciso e da rispettare. Il modo in cui abbraccia il principio dell'*inspect and adapt* sono le **sprint**: periodi di tempo tipicamente di **una o due settimane**. Possono essere anche di quattro, come si studia, ma secondo Simone non ha senso farle così lunghe a meno di non lavorare a un grande progetto "con duecento miliardi di persone che ci stanno lavorando costantemente"[13].

Prima di ogni sprint c'è una riunione di **sprint planning**, dopo ogni sprint una riunione di **post-mortem**[14].

0.3.4.3 La regola d'oro: meno riunioni possibili

"Le riunioni non piacciono a nessuno": sono, dice Simone, una "rottura di scatole infinita", e chiunque, soprattutto chi non è designer, non aspetta altro che poter dire "questa riunione è inutile, dobbiamo andarcene tutti a lavorare perché abbiamo cose più importanti da fare". In chat qualcuno risponde con il meme di Bernie Sanders a braccia incrociate; Simone avrebbe voluto mettere quello del cane che urla sulla scrivania perché "tutta la riunione poteva essere una mail", e promette che dalla prossima volta ogni slide avrà un meme alla fine.

Da qui la sua **regola d'oro**: **fare meno riunioni possibile**. Sottolinea però che è una regola "per quanto mi riguarda": magari un team non vede l'ora di fare riunioni, magari si portano le pizzette a ogni incontro ("non so se con le pizzette le riunioni siano effettivamente efficienti, però se funziona, perché no?").

Una mossa molto semplice per ridurle è **unire sprint planning e post-mortem** in un'unica riunione: si fa letteralmente la metà delle riunioni. Ciò che si fa in quella riunione è:

- **pianificare** la prossima sprint (*sprint planning*);
- **analizzare** la sprint precedente (*post-mortem*), indicando in particolare i problemi incontrati e le cose rimaste bloccate.

0.3.4.4 Una riunione deve essere rapida

La riunione unica deve essere **rapida**. Durante l'ultimo anno, per il *Biggest One*, il team di Simone contava circa quindici persone, e alle riunioni di planning e post-mortem ne partecipavano in media una dozzina.

[12] Si tratta verosimilmente di J. Sutherland, *Scrum: The Art of Doing Twice the Work in Half the Time* (2014), edito in italiano come *Fare il doppio in metà tempo*. Il titolo non viene pronunciato: Simone mostra la copertina e più avanti cita "Scrum, di Jeff Sutherland". La lunghezza indicata ("circa 300 pagine") è approssimativa.

[13] Nella trascrizione "a meno che non state facendo un sesto plà con 200 miliardi di persone": "sesto plà" è incomprensibile e potrebbe essere una storpiatura di "tripla A"; il senso, in ogni caso, è quello di una produzione enorme.

[14] Nella guida ufficiale di Scrum la riunione di fine sprint è divisa in *sprint review* (che cosa è stato prodotto) e *sprint retrospective* (come ha lavorato il team); qui, come nel resto della lezione, entrambe confluiscono nel termine *post-mortem*, che in senso stretto indica l'analisi a progetto concluso (sezione 0.1.5).

Quando sono "diventati bravi", le riunioni duravano **venticinque minuti, con dodici persone che parlavano tutte**. Quello è l'obiettivo, "anzi, il vostro obiettivo è fare anche meno".

Il motivo per cui si sfora è una **tentazione** costante: quella di **risolvere i problemi**. Quando qualcuno descrive un problema, un paio di persone cominciano a discuterne ("hai provato a fare questo?", "no, dovresti fare quest'altro"), e in un attimo sono passati dieci minuti in cui hanno parlato solo in due senza che la riunione sia andata avanti di un passo. In queste riunioni non si fa brainstorming e non si ragiona su come risolvere i problemi: "bloccatelo sul nascere, picchiate quelle persone, fate qualsiasi cosa per impedirglielo".

Quando c'è un problema, si **prende un appuntamento per risolverlo**. Simone mostra come dovrebbe suonare un report:

> "La settimana scorsa mi sono occupato della logica di movimento del personaggio, ho fatto un po' di tuning e di testing e ho preparato il block out del primo livello. L'unico problema è che il blueprint che faceva volare via gli uccelli non mi ha funzionato bene: visto che tu, Cristian, l'hai utilizzato in un altro livello, più tardi ci sentiamo su Discord quindici minuti e mi spieghi come l'hai settato. Questa settimana mi occuperò di completare il block out e di risolvere il problema del blueprint."

Fine del report: **circa un minuto**, e c'è tutto. Tutti devono fare così: trovare una persona con cui risolvere il problema, identificare *come* risolverlo e andare oltre. "C'è tempo e luogo per ogni cosa", ma la riunione di sprint planning non è il momento giusto.

0.3.4.5 La sprint come unità di tempo

Ultimo punto: le sprint sono **l'unità di tempo** fondamentale per ogni cosa. Le *milestone* non si misurano in mesi, giorni o settimane, ma **in sprint**. Bisogna prendere l'abitudine di dire "completeremo la preproduzione in otto sprint" e non "completeremo la preproduzione tra due mesi". È importantissimo per il calcolo della velocity (sezione 0.3.7).

0.3.5 Le cerimonie e la regola della consistency

0.3.5.1 Le cerimonie di Scrum secondo Simone

Le regole di Scrum si chiamano **cerimonie**. Simone le divide in due gruppi[15]:

Cerimonie principali	Altre cerimonie
planning; post-mortem; calcolo della velocity; creazione del backlog	*daily scrum*; *poker time estimation* (*planning poker*)

Le quattro principali sono quelle che "non potete evitare di fare, altrimenti non state facendo Scrum": il che, precisa, non è necessariamente un problema (ci torna alla fine, sezione 0.3.8). Le altre due le tiene separate perché a lui non piacciono granché, e prima di spiegare perché beve un sorso d'acqua "per lasciarvi la suspense".

0.3.5.2 Consistency: una cerimonia saltata fa cadere le altre

In Scrum c'è una parola chiave da ricordare: **consistency**. Bisogna evitare a tutti i costi di creare **precedenti di *inconsistency***, cioè di incostanza: con Scrum bisogna rimanere costanti. **Se si salta una sola cerimonia si crea un precedente che, a effetto domino, butta giù tutto il resto.** Questo è il punto già sollevato da Mattia. Ma, avverte Simone, questo *non* vuol dire che ci si debba forzare: ed è questo il punto chiave.

[15] La classificazione è quella proposta da Simone. Nella guida ufficiale di Scrum gli "eventi" sono la sprint, lo *sprint planning*, il *daily scrum*, la *sprint review* e la *sprint retrospective*; il backlog è un "artefatto" e il calcolo della velocity e il *planning poker* sono pratiche diffuse ma non obbligatorie.

0.3.5.3 Perché il daily scrum non funziona più

Il *daily scrum* era pensato "per il millennio scorso". Scrum è nato molto tempo fa[16], per team che si incontravano tutti i giorni in ufficio alla stessa ora, con un orario d'ufficio standard: team di sviluppo software, quindi per lo più programmatori (anche i più "artistici", come gli UX designer, erano alla fine programmatori), con una mentalità piuttosto simile e, soprattutto, con lo stesso orario.

Oggi, Covid a parte, questo non è più né fattibile né sensato. Gli studenti non hanno nemmeno un ufficio: si incontravano a volte in sede quando si poteva, ma per lo più lavorano online, e **non tutti lavorano allo stesso orario**. "E va bene così": c'è chi lavora di notte (la salute potrebbe risentirne, ma se si sentono bene così, va bene), e quelle persone non scriveranno il proprio *daily scrum* all'orario in cui lo fanno tutti. Senza arrivare a casi del genere, Simone lavora per una **società australiana**, dal lato opposto del globo: quando lui si sveglia, loro stanno finendo di lavorare, e il suo *daily scrum* cadrebbe nella loro notte.

In queste condizioni è difficile tenere traccia del *daily scrum* e rispettarlo, e prima o poi qualcuno lo salterà. Quello è un **precedente di incostanza**: qualcuno comincia a dire "ok, non lo faccio neanche io", poi "ma visto che non faccio più gli scrum, a che servono le riunioni? Questa volta non vengo", "e io neanche": "e da lì è il caos".

La conclusione è netta: **quando qualcosa non funziona, tagliatelo**. Il senso di "individui e interazioni sono più importanti di processi e strumenti" è proprio questo: se un processo o uno strumento non funziona **non è colpa vostra, è il team che non ne ha bisogno**. Non bisogna fissarsi su ciò che si vuole fare, ma sentire di che cosa ha bisogno il team e adattarsi a quel bisogno.

0.3.5.4 Perché il planning poker rallenta tutto

Il *planning poker* (che Simone chiama *poker time estimation*) non gli piace perché lo trova **estremamente lungo e tedioso**: rallenta un processo già lento di suo. Creare il backlog e valutare i task è già lunghissimo ("ci vuole almeno un'intera sprint di riunioni per fare un backlog"); aggiungerci il planning poker significa moltiplicare il tempo. Si informa con Diego su come sia stato spiegato alla classe: con il metodo di Fibonacci, "*power-uppato*"[17], il che farà risparmiare un po' di spiegazioni più avanti.

Simone capisce la necessità di sentire le opinioni di tutti ("*empowering the team*", eccetera), ma l'effetto è rendere il processo ancora più lento. E poiché le riunioni non piacciono a nessuno, se si quintuplica il tempo necessario "si parte subito male", perché il team si annoia già. Anche questo, quindi, lui personalmente non lo usa.

0.3.5.5 Il punto chiave: crearsi il proprio framework

La cosa più importante è che **non bisogna eseguire tutto a menadito**. I *framework* sono stati pensati per essere generici, ma anche per essere **adattati**. La procedura che Simone suggerisce:

1. si parte **rispettando il framework al 100%**;
2. quando qualcosa non funziona, si capisce *perché* non funziona;
3. si smette di farlo, si risolve, si fa altro;
4. si capisce qual è il framework più adatto e **ci si crea il proprio**.

"Questo è il punto chiave di tutto questo."

[16] Simone non ricorda la data esatta ("fine degli anni Novanta, forse fine degli anni Ottanta"). Il termine *scrum* applicato allo sviluppo di prodotti risale a un articolo di Takeuchi e Nonaka del 1986; il *framework* Scrum fu presentato da Jeff Sutherland e Ken Schwaber nel 1995.

[17] Nella trascrizione "è rimasta nelle lezioni di trial" e "è stata power upata": il senso sembra essere che il planning poker è stato spiegato nelle lezioni precedenti in una versione potenziata con la sequenza di Fibonacci. "Trial" potrebbe essere una storpiatura.

0.3.6 Il backlog

0.3.6.1 Una "rottura di scatole" inevitabile

Sulla creazione del backlog Simone spende "giusto due parole", perché la parte è molto tecnica e i consigli sono pochi. Creare il backlog è faticoso, "non ci sono mezze misure per dirlo": richiede moltissimo tempo e lavoro ed è intenso a livello mentale, perché bisogna ragionare tantissimo.

0.3.6.2 Il backlog a scaglioni

In teoria i backlog si creano all'inizio dello sviluppo: in un mondo perfetto si conoscerebbero fin da subito tutti i task necessari per completare il progetto. Nel mondo reale questo non accade. Ciò che accadrà, più probabilmente, è che si andrà avanti **per milestone**. È ciò che Simone ha cominciato a fare con *ISO* e con cui si trova bene:

- ha fatto il backlog della **preproduzione**, con tutto ciò che serviva per la preproduzione;
- finita la preproduzione, ha fatto quello della **produzione**;
- finita la produzione farà quello della fase successiva ("espansione, build finale, non lo so"), a seconda di come si svilupperanno le cose.

Fare i backlog **a scaglioni** li rende più semplici e il processo meno lungo; e tanto, comunque, bisognerà aggiungere cose in corso d'opera: "non si scappa".

0.3.6.3 Le epic

L'organizzazione in **epic** è molto utile, soprattutto per la stesura: si comincia ragionando per *epic* e poi si scende ai task. Un esempio di epic è "creazione del personaggio". Che cosa contiene il personaggio? Movimento, tuning, modello, animazioni, materiale, il design delle feature che dovrà avere, la creazione di tutte queste feature, e così via. Il processo diventa più *streamlined*, meno pesante e più semplice da seguire.

0.3.6.4 Ogni task deve avere un obiettivo chiaro

Fondamentale: **i task devono avere un obiettivo chiaro**. Un consiglio comune è scriverli come **user story**: "come utente voglio avere un personaggio che si senta potente, che io riesca a percepire come potente". Aiuta moltissimo nello sviluppo software e parecchio nei videogiochi per alcune cose, come appunto il personaggio. In generale è buona pratica **spiegare l'obiettivo e non la soluzione**: dire che il personaggio deve muoversi, saltare e fare le capriole, invece di dire che "quando premo il tasto W deve subire una forza che lo spinge in avanti". Però non è fondamentale.

Ciò che è davvero importante è che il task abbia un **obiettivo verificabile**: qualcosa di cui si possa dire "è completo" o "non è completo". Altrimenti le persone **non lo completeranno**, o lo completeranno in modo diverso da come lo si immaginava. Un esempio di task senza obiettivo chiaro è **"completare il livello 5"**:

- per Simone vuol dire aver creato il block out con qualche elemento estetico per sapere dove vanno le cose e come funzionano con i moduli già disponibili, e basta;
- per qualcun altro potrebbe voler dire inserire illuminazione, *post-processing*, *particles*, tutti gli shader fatti bene, ottimizzazione e *level streaming*;
- per altri ancora potrebbe voler dire arrivare all'illuminazione, lasciando *level streaming* e ottimizzazione a una seconda fase.

È la cosa più importante da ricordare quando si crea il backlog e si scrivono i task.

0.3.7 La velocity

"Bellissimo, bellissimo": il calcolo della velocity è per Simone "la cosa più figa e allo stesso tempo più odiata".

0.3.7.1 Il peso dei task

Come la classe sa, i task sono valutati in base al loro **peso** (o punteggio, o *score*). Il peso dipende da quanto un task è difficile e/o da quanto tempo richiede: è un po' una zona grigia, e Simone consiglia di valutarlo come **combinazione delle due cose**.

0.3.7.2 Mai le ore

Il consiglio fondamentale, che Simone non capisce perché si continui a contraddire, è **non usare le ore per valutare i task**, "per niente, neanche alla lontana". Nel libro di Sutherland c'è un paragrafo intero che spiega perché. Diego commenta che chi lo consiglia di solito fa "Agile Waterfall" e non ha nemmeno letto i libri su Agile.

Il motivo: **il cervello umano è completamente incapace di stimare le ore**, cioè di prevedere il tempo necessario per fare qualcosa: "non siamo pensati per questo". Siamo invece **molto bravi con le misure relative**: fissato un task che vale 10, sappiamo dire bene quale vale 5, quale 7, quale 2. Sappiamo dire che un task è molto più grande di un altro.

0.3.7.3 La sequenza di Fibonacci

Per aiutarsi ulteriormente, Simone usa la **sequenza di Fibonacci** (1, 2, 3, 5, 8, 13, 21, 34, ...), anche questo un consiglio di Sutherland che "nessuno segue". Sutherland racconta addirittura di un team che usava le **taglie dei cani** ("questo task è un pinscher, questo è un golden retriever")[18]; a Simone sembra eccessivo, e "i buoni vecchi numeri" bastano. Fibonacci ha due scopi molto utili:

1. **Gli intervalli crescenti assorbono l'errore di stima.** Man mano che un task diventa più grande e complesso, cresce anche l'errore associato. È facile dire che aprire la mail vale 2 e rispondere a tutti sul server Slack vale 3: che cosa può andare storto? La mail non si apre e bisogna reinserire la password; è esploso qualcosa e ci sono cinquanta messaggi invece dei soliti venti. Niente che cambi granché. È molto più difficile, invece, valutare la differenza tra creare il *behavior* di movimento del personaggio e costruire un intero livello: possono succedere infinite cose, soprattutto quando c'è di mezzo la programmazione. Il behavior del personaggio è un 20 o un 21? "È un 21, punto": non è un 34, perché non è così difficile, e non è un 13, perché sarebbe troppo poco. Magari la cosa più corretta sarebbe stata 23, ma non importa: lo scopo è **appianare** i risultati.

2. **Meno scelta, meno discussione.** I valori sono "infinitamente arbitrari", "completamente, al 100%", e nessun altro al mondo li condividerà; ma Fibonacci li rende un po' più condivisibili, perché con meno possibilità di scelta ci sono meno occasioni di discutere.

Che i valori siano arbitrari non è un problema: finché **è una persona sola a darli**, sono tutti arbitrari allo stesso modo, quindi **consistenti**; e se sono consistenti hanno una logica e si possono usare per fare calcoli.

0.3.7.4 Che cos'è la velocity e perché serve

Sommando il peso dei task completati in una sprint si ottiene la **velocity**. Ed è "fighissima" perché permette, in sostanza, di **prevedere il futuro**.

Simone mostra il grafico della velocity della preproduzione di *ISO* durante l'anno scolastico. La consegna era a maggio ("10 maggio, o poco dopo"). Già a inizio marzo, guardando l'andamento, aveva capito che

[18] Nel libro di Sutherland le stime per razze canine sono citate come esempio di scala relativa alternativa ai numeri; nella trascrizione "pincher".

non sarebbero riusciti a finire in tempo. Il ragionamento era semplice: mancano dodici sprint; quanto facciamo ogni sprint? Circa 89; quanto manca nel backlog? Circa 2000. "Non ce la facciamo"[19].

La velocity aiuta a prevedere il futuro e quindi a **prendere provvedimenti prima**. A marzo Simone ha cambiato i piani: prepariamo meno cose, smettiamo di perdere tempo su ciò che ci blocca, consegniamo qualcosa di diverso. In più, la velocity del team è **aumentata** sempre di più, e alla fine sono arrivati in fondo alla preproduzione **senza crunch**: "zero, completamente zero crunch". Nessuno ha dovuto lavorare di più; l'ultima settimana non hanno nemmeno toccato il progetto: hanno fatto la build, l'hanno spedita e per una settimana non ci hanno lavorato. E sono riusciti a consegnare anche di più del previsto, perché Simone ha **analizzato i trend precedenti**: perché in un certo punto c'era un picco? Che cosa li bloccava e faceva diminuire la produttività? L'ha scoperto, l'ha capito e hanno agito.

0.3.7.5 Come calcolarla in pratica

Il consiglio è di **trovare un modo per fare il calcolo**. Simone usava Monday, che non aveva uno strumento costruito apposta, ma ci è riuscito lo stesso; si può fare tranquillamente con **Google Sheets**, "col sorriso". Per farlo davvero bene bisognerebbe scrivere un po' di script dentro il foglio, "niente di improponibile". Quando comincia a girare "funziona da dio": "assolutamente fatelo, non posso dirlo abbastanza". (Un esempio costruito in diretta è nella sezione 0.3.9.)

0.3.7.6 Come assegnare i punteggi

Per assegnare gli *score* conviene partire dagli **estremi**: quali task sono un 34? Quali sono un 1? Da lì si mettono i valori intermedi. Gli estremi fanno da **ancore** e facilitano il processo di valutazione *relativa* descritto sopra.

Lo stesso vale per le **priorità**, se si usa un sistema di priorità: regole simili, ma con **intervalli più piccoli** e senza Fibonacci, che lì non serve. Simone usa una scala da 1 a 5; ma non la considera così fondamentale.

0.3.7.7 Che cosa blocca che cosa

Durante il lavoro sul backlog bisogna capire **che cosa blocca che cosa**. Ad esempio: ci sono 29 task, per un valore complessivo di 400, tutti relativi alla creazione dei livelli. Ma se non c'è il movimento del personaggio, non ha senso fare i livelli, perché vanno costruiti attorno a quel movimento: la creazione del personaggio **blocca** i livelli e quindi ha priorità più alta. Da qui si comincia anche a capire quale reparto si occupa di che cosa: probabilmente all'inizio i programmatori faranno i *behavior* fondamentali (come il personaggio), mentre i designer lavoreranno su documentazione e ricerca. Ma dipende dal gioco: alcuni non costruiscono i livelli attorno al personaggio, bensì il personaggio attorno ai livelli, quindi anche l'esempio "è opinabile".

0.3.8 Ricapitolando: come nasce una sprint

Prima di passare alle domande, perché "quello che avevo da dirvi di pratico è quasi finito", Simone ricorda che il grosso non sono le regole, ma **capire la logica alla base e integrarla nel proprio team e nel proprio progetto**. Poi riassume la creazione di una sprint in quattro passi.

1. **Post-mortem della sprint precedente.** Si vede che cosa ha fatto ciascuno, si capisce che cosa è bloccato e si *programma un incontro* per risolvere i problemi.

2. **Pianificazione.** Dal backlog si guarda che cosa blocca che cosa, quali sono le priorità e che cosa si può fare in questa sprint, e si prendono i task.

3. **Analizzare i problemi e agire.** Si guarda la velocity: è in aumento? In calo? Si è in un picco? L'andamento a dente di sega, "su, giù, su, giù", è **assolutamente normale**, ed è ciò che ci si deve aspettare; e man mano che il progetto va avanti la velocity **tenderà a diminuire**. Le domande da

[19] I numeri sono quelli detti a voce: 12 sprint per circa 89 punti danno poco più di 1000 punti, circa la metà dei 2000 rimasti nel backlog.

farsi: come sta andando il progetto? Chi sta facendo di più e chi di meno, e perché? Qualcuno ha avuto un calo di rendimento? C'è un problema che si ripresenta spesso, un collo di bottiglia? E come risolverlo, soprattutto **ascoltando il resto del team**.

4. **Rimanere consistenti**, in particolare con Scrum.

0.3.8.1 Ascoltare il team

Come scrum master o producer, il lavoro più importante è e rimane **ascoltare il team**. Tutto il resto non conta se poi, quando il team dice una cosa, non lo si ascolta. È lo stesso principio dell'adattare il *framework* al team:

- se il team odia il *daily scrum*, non si fa: non serve a niente;
- se il team fa fatica a prendere i task dal backlog, non glielo si fa fare: si assegnano i task in modo diverso, "non fategli manco vedere il backlog";
- se il team vuole sprint di due settimane invece che di una, si fanno di due settimane.

Magari le persone si organizzano personalmente in modo diverso: Simone, per esempio, organizza tutto il suo lavoro, su tutti i fronti, **su base settimanale**, e una sprint di due settimane gli sconvolgerebbe i piani. Se il team lavora per lo più così, si fa così. "Adattatevi" non vuol dire solo adattarsi ai bisogni del software, ma **soprattutto ai bisogni del team**.

0.3.8.2 Nessun framework va bene al 100%

Esistono infiniti *framework*, e **nessuno andrà bene al 100% per un team**. Bisogna provarli, il più possibile, e adattarli: cambiare qualche regola, togliere qualche cerimonia. Il *framework* più usato oggi, ad esempio, è **Kanban**, che non ha nemmeno le sprint[20]: ha soltanto un gigantesco backlog. "Ogni team è diverso, ogni team ha bisogni diversi, quindi adattatevi" (una frase che Simone calcola di aver ripetuto "almeno 49 volte").

0.3.8.3 La nota "smielata": la fiducia

La conclusione: alla fine di tutto, **Agile non si basa sulle iterazioni**, né sull'efficienza, né sulla creazione del miglior videogioco o software del mondo, né sul controllo. L'unica cosa su cui si basa davvero è **la fiducia**:

- la fiducia che il team ripone in chi lo guida, e che chi lo guida ripone nel team: "queste persone lavoreranno nel miglior modo possibile, sempre e comunque";
- la fiducia del team nel dire "se qualcosa va male, so che lo scrum master ha un piano di riserva che ci risolverà il problema, e il mio lavoro non sarà sprecato";
- la fiducia di chi guida nel dire "non posso controllarvi costantemente, non posso assicurarmi che facciate sempre tutto il lavoro o che seguiate le cerimonie come dovreste, però mi fido": mi fido che lo farete, che se ci saranno problemi ve lo direte, che farete sempre del vostro meglio.

Bisogna **costruire fiducia nel team e trasmettere fiducia al team**. È lo stesso principio del *default to trust* della prima lezione zero (sezione 0.1.11).

Simone ha finito con mezz'ora d'anticipo e apre alle domande ("go crazy"). Diego apprezza molto il finale, "smielato ma in realtà estremamente veritiero": lui è più orientato all'ambito professionale, ma crede che in Agile ci debba essere davvero questa fiducia, e che sia anche una delle sue forze. Simone pensava che non l'avrebbe apprezzato, tanto era smielato ("mentre lo scrivevo ho percepito il diabete che aumentava"); Diego gli suggerisce, la prossima volta, di aggiungere immagini con gli occhi a cuore e qualche gattino.

[20] Simone dice prima "Kanban neanche ce l'ha il backlog" e poi si corregge: "scusatemi, neanche ce l'ha le sprint, ha soltanto un gigantesco backlog".

0.3.9 Domande a Simone

0.3.9.1 La dimensione del team allunga le riunioni?

Uno studente, che lavora in un team di tre persone ("essendo in tre le cose vengono un po' più facili"), chiede quanto la **grandezza del team** influisca sui tempi delle riunioni e delle cerimonie, e se ci sia un modo per mantenerle rapide comunque.

Influisce assolutamente, "per una mera questione matematica". Ma in genere è possibile far sì che un team di qualsiasi dimensione completi una riunione in **una trentina di minuti al massimo**: quello è l'obiettivo, "anche a costo di dire meno cose, anche a costo di essere meno precisi". A meno di avere un team di una cinquantina di persone, la riunione durerà sempre trenta minuti; e con cinquanta persone non si fa comunque una riunione tutti insieme, perché è impossibile: si fanno le **riunioni di reparto**, e poi i **capi reparto** fanno la loro riunione di sprint planning. I numeri tornano piccoli: a una riunione ci saranno al massimo una quindicina di persone, venti "se proprio i pianeti si sono allineati", e venti persone possono stare in trenta minuti.

0.3.9.2 Quanto deve imporsi lo scrum master? Il timekeeper

Lo stesso studente chiede quanto lo scrum master debba **imporsi** sugli altri, sul piano umano, per far rispettare i tempi. Moltissimo, risponde Simone, ma **non deve per forza essere lo scrum master a farlo**. Nel team australiano in cui lavora, all'inizio di ogni riunione si sceglie, in modo pseudo-casuale, tra chi è disponibile o lanciando un dado, un **timekeeper**: la persona che ha "lo scettro del comando" e tiene in mano un cronometro, perché nessuno dedichi ai propri report più tempo del dovuto. È **comunemente accettato da tutti** che quella persona abbia il diritto di "rompere le scatole": è socialmente accettato.

Se le riunioni si dilungano troppo, si può proporre di introdurre la regola del timekeeper, scelto ogni volta a caso o sempre lo stesso: ha il diritto di richiamare gli altri perché, oggettivamente, il tempo scorre.

0.3.9.3 E se il problema blocca tutti?

Ultima domanda dello stesso studente: si è detto che i problemi si risolvono dopo la riunione; ma se un problema è molto grave, se il fatto di non averlo risolto blocca la produzione di più persone ("praticamente un disastro"), vale la pena fermarsi e parlarne tutti insieme, o si dedica comunque quel tempo solo ai report e poi si fa una riunione in più?

La risposta semplice è **no**: non si risolve nella riunione di sprint planning e post-mortem. "L'unica cosa che mi viene in mente che possa essere così tanto grave è se la stanza in cui state facendo il meeting va a fuoco." La riunione non è pensata per quello. Subito dopo, piuttosto, si avvia un'altra riunione con tutte le persone necessarie ("che durerà 25 ore") per risolvere il problema. Ma finché si fa planning e post-mortem, si fa **quello e basta**, anche se il problema riguarda tutti.

0.3.9.4 E se mancano delle persone?

Si fa comunque la riunione? La si rimanda? Simone la fa **sempre**, a meno di essere da solo, per **rispetto delle persone che sono venute**. Le assenze possono avere cause diverse. Chi non è venuto deve mandargli un messaggio su Telegram, anche vocale, in cui fa di fatto il proprio report.

Se una persona manca spesso, Simone ci parla e cerca di capire perché. Magari l'orario della riunione non va bene perché ha altro da fare: nel gruppo di *ISO* c'è un ragazzo che lavora a settimane alterne di mattina e di pomeriggio, e quando lavora di pomeriggio, tornando a casa, trova spesso traffico; quindi a settimane alterne non può venire, o arriva molto in ritardo. "Lo so, me lo posso aspettare, non è un problema." Se invece qualcuno manca in modo pseudo-casuale, bisogna capire il motivo: non gli va bene l'orario? Ne preferirebbe un altro? Pensa che le riunioni non servano? Non ha voglia di farle? Da lì si capisce il problema e si cerca di risolverlo, spiegando che **le riunioni sono importanti** e che è importante esserci tutti, e chiedendo "dimmi come possiamo aiutarti a rendertele meno pesanti".

0.3.9.5 Quando è la persona a non adattarsi

Se però, nonostante tutto, la persona "continua a fare la *drama queen*", a un certo punto bisogna ammettere che esistono anche persone non adatte a lavorare in un certo contesto, e si può valutare se lo sia.

Diego interviene per completare: in Agile tutti vogliono ottenere la cosa migliore per il team e per il metodo di lavoro di ciascuno. Ma se un individuo non si adatta al team, o il team non riesce ad adattarsi a lui, che sia o meno una *drama queen*, **la presenza di quella persona nel team va valutata**. "Nessuno, a parte casi eccezionali, tipo Miyamoto, vale il lavoro degli altri." Le *soft skill* restano alla base. E aggiunge che non è solo questione di soft skill: ci sarà la *drama queen*, quello che non si adatta, "lo stronzo o la stronza" (il genere, precisa, è irrilevante), quello che non ha voglia[21]; ma c'è anche la persona che ha voglia di fare ma non si adatta al team. Se non ci riesce, o il team non riesce ad adattarsi a lei, potenzialmente deve andare via.

Simone conferma che è un punto molto importante, che rientra nel discorso dell'**ascoltare di che cosa ha bisogno il team**: se il team non riesce a lavorare con un individuo, "non c'è niente di male, è così e basta". Per fortuna non è troppo comune: a un certo livello professionale è difficile che succeda, soprattutto in questa industria, dove le persone, se non stessero lavorando insieme, potrebbero diventare amiche, perché hanno un interesse in comune e sono molto appassionate. Ma può succedere.

0.3.9.6 Come si calcola la velocity, in pratica

Diego invita gli studenti ad approfittare di una persona "che ha fatto il vostro percorso". Una domanda sul calcolo: la velocity è semplicemente la **somma dei pesi dei task completati** nel tempo trascorso, cioè nella sprint? Esatto. Si fa la sommatoria dei task **taggati come *done*** che hanno la stessa sprint (o la stessa data di consegna, a seconda di come la si indica): si prendono tutti i task con "sprint 5", di questi quelli in stato *done*, e se ne sommano i pesi. Allo stesso modo si possono sommare i task non completati, quelli rimandati, e così via.

Uno sguardo ai fogli personali. Simone vorrebbe mostrare il foglio di *ISO*, ma è difficile da leggere per la quantità di dati; si chiede se mostrare i propri fogli rompa qualche NDA, poi nota che sta già condividendo lo schermo, "quindi gli NDA sono già stati rotti". Mostra allora uno dei suoi fogli personali: organizza il proprio lavoro **settimana per settimana**, con il nome della settimana, l'elenco dei task, la somma di ciò che era da fare, quanto è stato effettivamente fatto, quanto è stato tagliato e quanto rimandato per cause diverse. Il risultato è un grafico con l'andamento settimanale di questi valori.

La "speedrun velocity". Poi decide di fare "una cosa più figa" e di costruire un foglio in diretta, che chiama *speedrun velocity*. La struttura minima[22]:

Task name	Score	Sprint	Status
Task 1	5	Sprint 1	done
Task 2	13	Sprint 1	WIP
Task 3	3	Sprint 2	cut
...	...	...	...

Nella colonna *Status* inserisce una convalida dei dati con i tre valori *cut*, *WIP* e *done*, "per semplificarsi la vita". Poi scrive una serie di task con punteggi a caso (copiati da un altro foglio) e assegna le sprint, avvertendo che "ovviamente non dovrebbe essere fatto esattamente così, ma *for the sake of showcase*"; aggiunge qualche colore, perché "sono pazzo", anche se non è la cosa fondamentale.

Il calcolo è una somma condizionale su più criteri (`SUMIFS`, in italiano `SOMMA.PIÙ.SE`): si somma la colonna dei punteggi per le righe in cui la sprint è quella cercata *e* lo stato è *done*. Ad esempio:

```
=SUMIFS(B:B; C:C; "Sprint 1"; D:D; "done")
```

[21] Nella trascrizione "stronzo stronza,umbaquin": la parte finale è incomprensibile.

[22] I riferimenti alle celle detti a voce durante la costruzione ("B2B", "G1", "F1", "C2, F2") sono confusi nella trascrizione; la tabella e la formula sono una ricostruzione dello schema, non una copia esatta del foglio mostrato.

Lo stesso si può fare con *cut* per sapere quanto è stato tagliato. Si ripete per la sprint 2, e così via, e con i risultati si inserisce un **grafico a linee** (Sheets propone il tipo di grafico sbagliato, e Simone lo corregge). Con più sprint si ottiene un andamento come quello dei suoi fogli personali o di *ISO*. Chi vuole fare le cose per bene può aggiungere altre analisi: chi ha fatto più task, chi ha più peso e carico di lavoro, lo stato generale del progetto ("magari il backlog fosse così vuoto").

Era, dice, per essere sicuro di riuscirci; è disponibile a rifare qualche passaggio, ma alla classe è tutto chiaro. Uno studente chiede se il foglio si possa avere: Simone lo mette in una cartella e sarà Diego a farlo avere alla classe.

0.3.9.7 I problemi più gravi incontrati con Agile

Diego osserva che Simone ha capito come applicare i principi di Agile con una velocità sorprendente, più di quanto sia successo a lui quando gli fu spiegato, e che all'inizio si fa spesso "Waterfall Agile", cioè Waterfall con le sprint, credendo di fare Agile[23]. Poi la domanda: quali sono stati i problemi più gravi?

I task "tenuti in caldo". Simone si è accorto che le persone non chiudevano i task al momento giusto. Se diceva che un task richiedeva, per esempio, quattro ore, qualcuno lo finiva in un'ora e poi, invece di segnarlo come completato, aspettava ("me lo rivedo più tardi, tanto ho ancora altre tre ore"). Non funziona così: **quando un task è completato, è completato**, punto. Non c'è "me lo rivedo tra un po"' né "devo farlo in queste quattro ore, quindi ci metto quattro ore". Questo falsava i risultati, e Simone ha cambiato subito approccio[24].

Le riunioni che nessuno voleva fare. Durante il secondo anno le persone non volevano fare le riunioni settimanali. All'inizio le facevano solo tra designer; poi le hanno estese a tutti, perché non aveva senso limitarle, ma la risposta era "questa riunione è inutile, non ha senso". In quel caso Simone non è riuscito a risolvere il problema: non sapeva come farlo. C'erano problemi anche sul piano umano, non tra i designer ma con membri di altre classi: problemi che, col senno di poi, si sarebbero potuti risolvere, ma all'epoca non ci sono riusciti. La soluzione adottata è stata non fare le riunioni con tutto il team: gli altri membri mandavano un **recap settimanale per messaggio**. Non era un metodo del tutto fallimentare, ma si perdono molti benefici delle riunioni tutti insieme: il *team building*, il confronto, il vedersi, il parlare. È stato, dice, "un grosso problema e anche un grosso fallimento".

Per il resto, niente di così grosso da valere la pena di raccontarlo: valutazioni di task sbagliate, sprint fatte male, e un periodo di "Waterfall Agile" anche per lui, finché non ha capito di sbagliare.

0.3.9.8 Che cosa rispondere a chi dice che le riunioni sono inutili

E oggi, se qualcuno dicesse "per me queste riunioni sono inutili", che cosa gli risponderebbe? **"Perché?"**. E da lì si capisce qual è il problema: non vuole dedicarci tempo? Durano troppo? Gli costa fatica? Ha problemi con le riunioni perché soffre di qualche forma d'ansia e non gli piace stare con tutti i membri del team, anche fuori dalle riunioni ("tipo ci becchiamo a giocare online a qualcosa")? Bisogna capire il perché e **tirarlo fuori dalle persone**, perché spesso le persone stesse non sanno il perché delle cose che dicono e sentono.

0.3.9.9 I saluti a Simone

Diego ringrazia Simone e spera che gli studenti abbiano appreso il più possibile: un'esperienza più diretta di quella di Simone e Cristian non si può avere, perché hanno fatto letteralmente lo stesso percorso (la lezione di Diego è "fondamentalmente la stessa" da un anno all'altro, con qualche aggiunta). Ricorda che Simone

[23] Il passaggio è frammentato e non è del tutto chiaro chi abbia fatto "Waterfall Agile" all'inizio: dalla risposta di Simone ("ho fatto anch'io waterfall agile per un po"') sembra trattarsi di un errore comune, che Diego attribuisce anche a se stesso o agli studenti in generale.

[24] L'esempio parla di stime in ore, in apparente contrasto con il consiglio di non usarle (sezione 0.3.7); è plausibile che si riferisca proprio a una fase precedente in cui le ore erano ancora usate, e che il "cambiamento immediato" sia stato anche questo.

ha concesso anche il foglio della *speedrun velocity*, che Diego passerà alla classe, e lo saluta per non rubargli altro tempo.

0.3.10 Cristian: il punto di vista del game designer e project manager

0.3.10.1 Chi è il relatore

Il secondo intervento è di Cristian, che attiva la webcam ("mi vedete super zoomato, spero non sia una brutta vista") e condivide le slide, mentre Diego cerca di farle andare in modalità presentazione perché vengano meglio nella registrazione. Cristian ringrazia Diego per la presentazione e si presenta:

- è uno **studente del terzo anno** di Game Design, quindi ha fatto lo stesso percorso della classe;
- è anche **tutor** del corso di Game Design per gli studenti dell'accademia online;
- è **game designer e project manager** di due progetti:
  - *Gamers in Time*[25], un **gioco da tavolo**, vincitore del *Big One* del primo anno;
  - *L'Inf*[26], un **videogioco** che ha vinto, "tra virgolette", il *Bigger One* del secondo anno e che il team ha avuto l'opportunità di continuare come *Biggest One* del terzo anno, opportunità che ha accettato.

In entrambi si è occupato di tutta la parte di design, insieme ad altri colleghi, ed è per lo più anche **project manager, scrum master, producer**, "come volete definirmi".

0.3.10.2 Come è strutturata la sua parte

Oltre all'esperienza personale, la lezione ha due parti principali:

1. la **produzione** effettiva tramite Scrum;
2. la **gestione del team** in un ambiente di lavoro Scrum.

Possono sembrare la stessa cosa, ma "il progetto tendenzialmente è una cosa e il team è un'altra": ci sono ovvi rapporti tra le due parti, ma per ciascuna ci sono precauzioni specifiche.

0.3.10.3 Le premesse

- Quello che descrive è **un** percorso, un modo di applicare Scrum, **non l'unico**: se non si è d'accordo, o se il team non "sopporta" questo metodo, si possono fare tutti i cambiamenti che si vuole (e qualche consiglio lo darà strada facendo).
- Non è una lezione descrittiva come quelle di Diego (che cos'è Scrum, le definizioni, le cerimonie, come si calcolano la velocity e il peso dei task), ma un **confronto** sull'esperienza dei due progetti, più su *L'Inf* che su *Gamers in Time*, perché Scrum l'ha applicato al videogioco e non al gioco da tavolo.
- **Tutto è in continua evoluzione**. Per questo vuole le domande della classe: il primo obiettivo è chiarire i dubbi e far uscire tutti dalla lezione avendo capito un po' di più la mentalità agile, "con tutti voi stessi"; il secondo è ricevere **feedback** per migliorare non solo le lezioni, ma lo stesso modo in cui applica Scrum.
- Ha un po' di mal di gola e si fermerà spesso a bere: si scusa in anticipo.

[25] Così nella trascrizione; il titolo non è stato verificato.

[26] Il titolo compare in molte forme ("l'Inf", "Linf", "Leanf", "Linfa"); qui si usa *L'Inf*, la più frequente. Non è chiaro se la grafia corretta sia *Linf*, *L'Inf* o *Linfa*.

0.3.11 Il caso di studio: *L'Inf*

0.3.11.1 Le caratteristiche del gioco

Qualcuno in classe forse conosce Cristian o ha provato il gioco a Level Up, l'evento in cui è stato presentato. Le caratteristiche principali:

- è **single player** e offline: niente co-op, niente multiplayer, niente server;
- è un'**avventura fantasy**: un gioco di **durata fissa**, che tendenzialmente si gioca per una *run*; chi l'ha apprezzato di più e vuole vedere tutti gli aspetti del design e delle meccaniche, o esplorarne i segreti, lo rigiocherà;
- il giocatore ha delle **magie colorate** con cui affronta **orde di nemici colorati**: spara le magie e, a seconda di quanti nemici "scoppiano" con un solo colpo, ottiene vari bonus, come succede in *Candy Crush*.

Dati e passione. Diego interrompe perché ritiene questa parte molto importante: com'è nato l'elemento davvero innovativo di *L'Inf*? Dallo **studio dei dati**, ma anche dalla **passione del team**. I dati danno una direzione: le ricerche di Cristian indicavano il successo delle meccaniche *match-3*[27]; e Cristian è anche, principalmente, un giocatore mobile. La combinazione tra passione e ricerca ha prodotto un progetto molto valido sul piano della fattibilità (single player, fantasy, espandibile).

Cristian aggiunge un'altra ricerca fondamentale, da cui è stato poi delineato il **documento di visione**[28] del gioco: quella su che cosa chiedesse Epic Games per finanziare i progetti con l'**Epic MegaGrant**. Il team ha individuato tre elementi fondamentali[29]:

1. **innovazione**;
2. **temi ambientali**: attraverso i colori e l'ambientazione, il gioco vuole criticare l'inquinamento globale;
3. un **elemento narrativo**.

Questi tre elementi, le ricerche specifiche sui vari tipi di gioco e la passione (di Cristian in primis, come lead, ma anche degli altri membri, incuriositi dalla proposta) hanno portato a un progetto che al momento sembra funzionare: "stiamo andando nella direzione giusta, almeno secondo quanto previsto; non si può mai prevedere al 100% ciò che accadrà".

Il contenuto. Oltre alla meccanica *match-3*, il gioco prevede:

- **quattro modalità di sparo**;
- **cinque livelli**, con esplorazione e fasi di combattimento contro le orde;
- **puzzle**;
- **cinematic** per lo più ambientali: niente attori, doppiatori o mimica facciale, ma scene di presentazione dell'ambiente di gioco;
- una **boss fight** finale.

Il perché di questo elenco sarà chiaro tra poco.

[27] Nella trascrizione "match free" e "match tree": si tratta delle meccaniche *match-3* (allineare tre o più elementi dello stesso colore), tipiche di *Candy Crush*.

[28] Nella trascrizione "iVision" e "l'iVision": si tratta quasi certamente della *vision*, il documento di visione (sezione 3.6).

[29] È la lettura dei requisiti fatta dal team, come riferita da Cristian; non è stata verificata sui criteri ufficiali del programma.

0.3.11.2 Un "trauma" di produzione: il cambio di team

La produzione si è sviluppata in due sezioni, e *L'Inf*, "per fortuna o purtroppo", ha avuto un grave problema: è la risposta, anticipata, alla domanda fatta a Simone sui problemi più gravi. Come tutte le produzioni di videogiochi, ha subito dei "traumi". Il più grande: **il 100% del team originario è stato sostituito**. Non per scelta, ma perché i membri non si sono iscritti al terzo anno: avevano altri obiettivi; c'è chi cerca lavoro e chi ha aperto un'azienda. Cristian, volendo continuare il terzo anno e soprattutto *L'Inf*, un progetto in cui crede e che ha dimostrato il suo potenziale a Level Up e con i feedback costanti dei giocatori, ha dovuto fare dei **colloqui** per formare un nuovo team che si adattasse meglio al tipo di sviluppo del gioco[30].

La composizione è cambiata così:

Team	Composizione
originario	2 game designer, 2 level designer, 1 technical designer
attuale	3 game e level designer, 2 technical designer

Il motivo: la preproduzione, cioè la parte più importante di game design, era finita, e servivano **più tecnici**. Il gioco si basa molto sulle meccaniche: un sistema di combattimento innovativo che richiede di scriptare le meccaniche, diversi tipi di sparo, puzzle, una boss fight. È molto più basato sulle meccaniche che su narrativa o ambientazione. Allo stesso tempo servivano level designer, perché il gioco doveva prendere forma e i livelli andavano costruiti.

0.3.11.3 Perché raccontare tutto questo

Non per vendere il gioco ("anche se magari è un effetto collaterale ben voluto"), ma per mostrare che **ogni progetto, a seconda di come è composto** sul piano del design, della produzione e soprattutto del team, **ha un modo diverso di applicare Scrum**. Per questo Cristian è sceso nel dettaglio delle caratteristiche: ognuna comporta ambiti di produzione diversi.

- I livelli richiedono level designer.
- Il *match-3* e i diversi tipi di combattimento richiedono tecnici.
- Il single player è più un vantaggio che un limite: un gioco multiplayer non è il doppio, ma "tipo dieci volte più difficile" da gestire a livello produttivo.
- Il genere *fantasy adventure* deriva soprattutto dalle ricerche.

Quindi **ogni progetto ha il suo framework di Scrum**. E il fatto di aver cambiato team a metà produzione, finita la *vertical slice*, con persone completamente nuove (ex compagni di studio, che però non avevano mai visto come funzionava il gioco), ha avuto diverse conseguenze, che Cristian rimanda alle domande di fine lezione.

0.3.12 Il loop di Scrum

L'applicazione di Scrum è un **loop** molto semplice, simile a quello mostrato da Simone:

1. **creazione del backlog**;
2. **selezione dei task** da svolgere, a ogni sprint;
3. **implementazione dei task** durante la sprint, con controlli e *scrum* (con la frequenza che si decide);
4. **calcolo della velocity**, per vedere com'è andata e come migliorare la produzione nelle sprint successive;

[30] La frase è in parte persa nella trascrizione ("ho dovuto fare dei colloqui Grazie a tutti. Cioè, servisse una parola brutta, ma semplicemente si adattasse di più al tipo di sviluppo"): il senso ricostruito è che "colloqui" è una parola forte, ma che si trattava di scegliere persone adatte a questa fase.

5. **interpretazione** della velocity per fare il *tuning* della produzione; e si ricomincia.

Cristian le affronta una per una, con esempi della sua esperienza e consigli. È contento che la classe stia cominciando proprio adesso, perché questa parte potrebbe tornare utile già dalla settimana successiva, quando gli studenti applicheranno il *framework*.

0.3.13 La stesura del backlog

0.3.13.1 Un backlog per milestone

Anche Cristian, all'inizio, cercava di prevedere il più possibile tutto il gioco, dal prototipo fino al rilascio. Oggi invece lo struttura **per milestone**: il backlog dell'*alpha*, poi quello della *beta*, poi quello della *gold*. Questo non impedisce di cominciare a prevedere che cosa ci sarà in beta e in gold, ma dà un'idea molto chiara di dove si sta andando, soprattutto per capire che cosa tagliare o aggiungere quando ci si accorge che qualcosa serve prima o dopo, e adattarsi di conseguenza: "la cosa più importante del mindset agile è adattarsi ed essere flessibili".

0.3.13.2 Da dove vengono i task

Il gioco deve pur nascere da qualche parte: i task vengono dai **documenti chiave** della produzione, già visti nel corso (capitolo 3):

- il **documento di visione**;
- i **GDD** delle varie feature, che dettagliano la visione;
- il **versioning** delle feature, a cui si dovrebbe pensare già mentre si scrive il GDD;
- gli **LDD** (*level design document*) e tutti gli altri.

L'ordine dipende dal gioco: in un gioco fortemente basato sulle meccaniche è probabile che gli LDD arrivino dopo lo sviluppo del *core*; in un gioco basato sull'esplorazione non succede proprio l'inverso, perché le meccaniche servono comunque, ma le due cose avvengono quasi in contemporanea.

Una volta che si ha la documentazione con tutte le descrizioni e le caratteristiche delle feature, **dalla versione 0 alla versione X**, si ha già una serie di implementazioni da realizzare. Un esempio:

- **versione 1**: il giocatore deve camminare; si usa il manichino base di Unreal;
- **versione 2**: si implementa la *skeletal mesh* fatta dal 3D artist e si adatta la camera al movimento, con gli effetti di vibrazione a seconda che il personaggio corra, cammini o sia abbassato;
- **versioni successive**: quando il personaggio si muove su terreni diversi solleva tipi diversi di polvere, di più su un terreno molto polveroso. Non è fondamentale e potrebbe essere una delle ultime versioni, verso la fine.

Il consiglio è che, quando si fa il *versioning*, si inseriscano **tutti gli ambiti necessari** per quella feature: programmazione, arte (se serve un asset), animazione, UI, VFX, suoni. Se tutti i *field* compaiono già nella documentazione, la creazione del backlog sarà più chiara e più spedita.

0.3.13.3 Come organizzare i task

I giochi hanno moltissimi task. Come organizzarli perché siano leggibili non solo da chi li ha scritti (scrum master o project manager), ma anche da chi deve navigarli per capirci qualcosa? Cristian usa tre tipi principali di organizzazione.

1. Per feature (o epic). Si suddividono i task per feature, collegandosi alle *epic* di Simone. Ad esempio, l'epic è il movimento del personaggio e i task sono camminata, corsa, salto, volo, "dipende dal gioco".

Tutto in un solo posto. La suddivisione non va fatta per forza in documenti diversi, anzi Cristian lo sconsiglia: se tutto è nello stesso strumento (Google Sheets, Monday, ClickUp, Trello) si usano i **filtri** e si trova tutto lì, senza navigare tra mille siti o mille documenti.

Per ambito, con un color code. Quando si suddivide per ambito (programmazione, arte, UI, eccetera), conviene usare un **codice colore**. Cristian lo usa "per qualsiasi cosa": uno per le milestone, uno per gli ambiti di produzione. Diego gli chiede di mostrare il suo:

Ambito o milestone	Colore	Perché
Suoni	viola	—
VFX	rosso	sta imparando a usare Niagara
Level design	verde	"erano finiti i colori"
Game design	arancione	è il suo colore preferito
Alpha	azzurro	—
Beta	viola	—
Gold	giallo scuro	il colore richiama il nome

Come si vede, è un misto di funzionalità e preferenze: la cosa più importante è che **tutto il team sia d'accordo**.

2. Per milestone, con le sotto-milestone. Un secondo metodo è suddividere il backlog per milestone. Ma se il backlog dell'alpha è separato da quelli di beta e gold, come si fa a dividere in alpha, beta e gold un backlog in cui tutto è alpha? Cristian suddivide le milestone in **sotto-milestone**: l'alpha dell'alpha, la beta dell'alpha, la gold dell'alpha. Serve ad avere idee più chiare, a tenere sempre in mente lo *scope* senza perderlo e soprattutto a capire meglio **se il gioco è fattibile**. Una suddivisione più fine permette anche di calcolare con più precisione (una precisione che "lascia il tempo che trova") quante sprint servono per ogni sotto-milestone. La dissonanza tra "dividete tutto in alpha, beta e gold" e "dentro l'alpha trovate alpha, beta e gold" è quindi voluta: si parla delle sotto-versioni.

3. Per priorità. Chi non vuole usare le sotto-milestone può usare il sistema di **priorità** di cui parlava Simone: una scala numerica, da 1 a 5 o da 1 a 10, a seconda di come si riesce meglio a distinguere le priorità. Serve soprattutto a capire quali task selezionare (sezione 0.3.14). In realtà è più o meno la stessa cosa con un approccio diverso: indicare alpha, beta e gold dell'alpha crea già di per sé delle priorità. "Dipende tutto da voi."

0.3.13.4 Come scrivere i task e dove

Il consiglio è scriverli **dal punto di vista dell'utente**. Ma, come diceva Simone, non sempre "il gioco vale la candela". Lo è soprattutto in un caso: se il programmatore legge un task dell'artista 3D scritto in modo troppo tecnico e non lo capisce, si crea dissonanza tra i reparti e possono nascere attriti. Quindi è sempre consigliabile, ma non obbligatorio: spesso basta una chiamata ("prima semplicemente ti giravi verso il tuo compagno e gli chiedevi che cosa significa"). "Però è giusto che vi dica le cose come dovrebbero essere fatte."

Dove scriverli? In qualsiasi strumento: Trello, Monday, ClickUp, gratuiti o a pagamento, oppure un foglio Google costruito da sé. Dall'esperienza di Cristian emerge che **Google Sheets sta antipatico alle persone**: già il nome "foglio di calcolo" spaventa. Strumenti come Monday o ClickUp hanno più colori e soprattutto **mandano le notifiche**, quindi sono più accessibili a tutti. "Decidete voi."

0.3.13.5 Non prevedere tutto: la flessibilità

È fondamentale capire che **prevedere tutti i task passo per passo**, in un ordine preciso ("dopo la camminata faremo il salto, dopo il salto il sistema di sparo, e così via all'infinito"), va contro i principi di Agile, in particolare contro il **quarto valore**: rispondere ai cambiamenti più che seguire un piano.

Da qui il consiglio centrale: la **flessibilità** deve essere il cuore della propria filosofia di lavoro e del proprio modo di applicare Scrum. Bisogna essere sempre pronti a **tagliare** nel modo più agile possibile: se si è capita davvero la mentalità agile, si riesce a tagliare una feature **senza bloccare la produzione delle altre**.

L'esempio più eclatante viene dall'inizio di *L'Inf*: Cristian faceva il backlog di tutto il gioco, arrivando a circa **400 task** di ogni ambito, nel dettaglio: per la camminata arrivava a indicare come si muoveva la camera quando il personaggio girava a destra, e come cambiava quando saltava o cadeva da un dirupo. "Vi viene subito da pensare: ma come fai a prevedere tutto?" Infatti la soluzione è **non prevedere tutto**: capire a grandi linee ciò che si vuole, qual è l'obiettivo della feature, senza scendere troppo nel dettaglio, perché il dettaglio crea di per sé un **lavoro Waterfall**, che è ciò che bisogna assolutamente evitare. E scrivere un backlog così dettagliato richiede tantissimo tempo, che è limitato: la produzione ne risente in tutti i modi possibili.

Bonus: gli elementi innovativi. Se il gioco (il proprio *Bigger One*) ha elementi innovativi, la flessibilità va portata **ai massimi livelli**. *L'Inf* era partito con una meccanica di **assorbimento e sparo**: si assorbe un nemico colorato, ad esempio giallo, e lo si spara contro altri nemici gialli, prendendo ispirazione da altri giochi. Nel loro caso, però, creava più problemi che altro: per due o tre ore di gioco, "assorbire, sparare, assorbire, sparare" risultava **troppo ripetitivo**, e hanno deciso di tagliare la meccanica. Il problema è che l'hanno tagliata dopo che Cristian aveva già scritto **600 task nel backlog e 12 GDD**: tempo perso. Per questo la flessibilità, negli elementi innovativi ma in generale, "è la prima cosa".

0.3.13.6 Quanto dettaglio?

Quando un backlog è abbastanza dettagliato e quando lo è troppo poco? "La risposta purtroppo non c'è." Viene da due fattori:

1. l'**esperienza**: man mano che si scrivono GDD e si va avanti nello sviluppo, ci si "tuna";
2. il **trial and error**: si sbaglia e si torna indietro, il che riporta, di nuovo, all'esperienza.

0.3.14 La selezione dei task

Con un bellissimo backlog ordinato per colori, ambiti e priorità, quali task si selezionano? "Da qualche parte bisogna pur cominciare." Le domande principali sono due: **quali** selezionare e **quanti**.

0.3.14.1 Quali task

La priorità. Il primo criterio è la priorità: i task da fare prima sono le scelte più urgenti, e quindi si scelgono quelli[31].

Le capacità attuali degli sviluppatori. Il secondo criterio sono le **abilità dei developer**. Cristian precisa che non intende in alcun modo giudicarne la professionalità: intende letteralmente **che cosa sa fare lo sviluppatore in quel momento**. Se un programmatore non ha mai usato Unreal, non è il momento di dargli task di implementazione in Unreal: bisogna prevedere anche il **periodo di apprendimento**, eventualmente come task vero e proprio ("hai questa sprint per imparare a implementare l'IA").

È successo due mesi prima con uno dei suoi technical designer: gli ha dato **due settimane** dicendogli "in queste due settimane non ti preoccupare di niente, il resto lo produciamo noi". Doveva essere d'accordo, e lo era, anzi era contento di approfondire l'argomento: studiare come funzionano le IA (anche grazie a vari link passati da Diego) e soprattutto come implementarle nel motore. Per due settimane ha dovuto "solamente" pensare all'apprendimento. Il consiglio: **prevedere questi "tempi morti"**, perché ne ha bisogno il team, e ne ha bisogno chi pianifica per non falsare la previsione dello *scope*. Se in questo momento ("sottolineo, in questo momento") un programmatore non sa fare una cosa, la si rimanda e si mette altro; prima o poi imparerà, e a quel punto si torna a selezionare per priorità.

[31] L'inizio di questo passaggio manca nella trascrizione ("quanti selezionare. cui scegliere prima, sarà la scelta più urgente da fare"): il primo criterio è stato ricostruito dal contesto, anche perché poco dopo Cristian dice che, risolto il problema delle competenze, "si ritorna a selezionarli per priorità".

0.3.14.2 Quanti task

Il primo criterio sulla quantità è la **velocity**, ma Cristian l'ha messo nella slide solo per completezza del loop: se non si ha ancora una velocity, non ha senso. Per ora la si trascura e si fa finta che sia la **prima sprint**, la prima applicazione di Scrum.

Nelle prime sprint la quantità si basa sulle **capacità** e sulla **disponibilità** degli sviluppatori:

- **disponibilità**: per quanta parte della sprint il technical designer (per riprendere l'esempio) è disponibile? Non in termini di ore, perché Cristian concorda con Simone che le ore sono sempre molto relative;
- **performance**: gli studenti si conoscono ormai da quasi un anno e mezzo. Se uno sviluppatore non ha mai fatto un livello in Unreal (per motivi suoi, magari problemi al PC) e dice "voglio fare un livello e ci metto due settimane", sta a chi pianifica cominciare a capire **chi può fare che cosa**. Tutti possono fare tutto, se si prevede il tempo di apprendimento; ma se Cristian arrivasse a dire "in due settimane vi faccio l'IA del vostro gioco implementandola in Unreal", "vi sto prendendo in giro".

Queste sono le analisi preliminari quando non si ha una velocity. Se invece non si conosce affatto il team, l'unico punto di partenza è la **disponibilità** degli sviluppatori, da cui si stima quanto si riesce a fare.

Tanto, **nelle prime sprint si sbaglieranno completamente tutte le previsioni**, "ed è giusto così, e ben venga": se non si sbaglia, probabilmente si sta applicando Waterfall, o si sta applicando male Scrum. All'inizio si tratta solo di buttarsi: "questa settimana abbiamo previsto 15 task; ne abbiamo fatti 40. Bene, la prossima ne proviamo 40"; poi magari se ne fanno 25, e così via, **tarando** man mano la quantità.

0.3.14.3 Definire bene il goal

Per ogni sprint, e anche per ogni singolo task, bisogna **definire bene l'obiettivo**. Serve a mantenere concentrati gli sviluppatori: se so che entro questa settimana devo finire il block out del livello, so dove sto andando, lavoro per quel risultato e soprattutto **non perdo tempo**, come succede spessissimo. Se durante il block out qualcuno si mette a inserire effetti visivi ambientali o il *post-process*, viene il dubbio che non stia lavorando per l'obiettivo: è un **campanello d'allarme**, perché potrebbe far mancare l'obiettivo per cui la sprint era stata pensata.

0.3.14.4 Bonus: progettare il ritmo dello sviluppo

Un consiglio che Cristian ha sentito raramente, ma che ha deciso di sperimentare: poiché gli studenti sono designer, e tendenzialmente anche lo scrum master del loro progetto lo sarà, possono **progettare il *pace* dello sviluppo**.

Di solito si pensa (e potrebbe anche essere giusto) che l'obiettivo di Scrum sia trovare una *velocity target* e mantenerla costante per tutta la produzione. Può essere una soluzione; ma con entrambi i suoi team Cristian ha notato che, per come sono fatti gli esseri umani, ci sono **picchi di attenzione e di produzione**. Con l'esperienza si impara a percepirli in anticipo e a progettare di conseguenza; oppure sono gli sviluppatori stessi a dirlo: se la settimana prima hanno implementato 12 nemici, 15 modalità di sparo e 13 cutscene, quella dopo diranno "andiamoci piano". Di solito non succede, perché le sprint sono programmate apposta per adattarsi alle capacità degli sviluppatori, ma è un metodo possibile.

Che cosa intende, in concreto? Se per un periodo si sono implementate molte feature o sistemi che però, visivamente, **non hanno portato un aumento di qualità percepibile nel gioco**, gli sviluppatori non si sentono riconosciuti e ripagati dello sforzo. Allora, la settimana dopo, ci si concentra su qualcosa che dà **soddisfazione**. Questo lo permettono le sprint, lo permette Scrum e non Waterfall: è una differenza fondamentale.

L'esempio di *L'Inf.* Per più di otto mesi il gioco è rimasto con **quattro tipi di nemici**. Nonostante l'impegno (quattro nuove modalità di sparo, la boss fight, tantissima programmazione "di background", cioè non visibile al giocatore ma necessaria al funzionamento del gioco), il team cominciava a percepire che **non c'era corrispondenza tra impegno e qualità percepita**. Cristian ha allora deciso di limitare per un periodo la produzione degli elementi di background (programmazione come l'IA, tutto ciò che faceva proseguire il gioco "sotto"), di inserire altri tipi di nemici, che tutti non vedevano l'ora di avere ("sapevo

che era uno di quei jolly da sfruttare per far risalire il morale del team"), e di dedicare tempo a effetti visivi e sonori, a migliorare tutto ciò che era estetico. Diego suggerisce il termine: la **juiciness**. Ciò che vedeva il giocatore lo vedevano anche gli sviluppatori, che si sentivano più appagati e sentivano i propri sforzi ricompensati. "Adesso loro non lo sanno, ma li ributterò sotto un treno di lavoro" ("in realtà lo sanno, però non me lo dicono").

Per Cristian è un punto fondamentale, che gli è costato "mesi di sofferenze" di esperienza per capirlo. Diego invita la classe a fare domande su un *insight* ottenuto per tentativi ed errori.

0.3.15 L'implementazione: controlli, blocker e feedback

0.3.15.1 Il check dei progressi

Selezionati i task, viene la parte "più tranquilla" dal punto di vista di Scrum (i problemi di implementazione, ovviamente, sono altri): parte la settimana di lavoro, ognuno ha i suoi task, i *task owner* hanno scelto che cosa fare, e questo prosegue per tutta la durata della sprint.

Durante l'implementazione si fa un **check dei progressi**. Non serve a controllare che cosa stanno facendo le persone: di base bisogna **fidarsi** che, se il programmatore dice "farò questo", lo farà entro i limiti della sprint. Cristian parla qui di *daily scrum* perché è ciò che si dovrebbe fare in generale, anche se il suo team non li fa ogni giorno (sezione 0.3.19). Oltre alle solite domande (che cosa hai fatto ieri, che cosa farai oggi), **la cosa più importante è trovare i blocker**.

0.3.15.2 Agire subito sui blocker

Se il programmatore dice "non ho potuto implementare il personaggio perché mi manca l'asset, la *skeletal mesh* del protagonista", bisogna **agire subito**, in due modi:

1. **dare altri task** al programmatore, perché altrimenti resta lì ad aspettare, e questo non lo si vuole assolutamente;
2. **capire che cosa è successo** alla skeletal mesh, parlando con la 3D artist. Se è solo questione di tempo, non è davvero un problema: è normalissimo aver assegnato l'implementazione prima che l'asset fosse pronto ("è normale sbagliare, l'importante è rimediare"). Se invece anche lì c'è stato un blocco, bisogna **risalire a cascata** fino a trovarlo.

Magari si scopre che il problema sono proprio i game designer: quando hanno fatto la *mood board* non sono stati abbastanza chiari, e la 3D artist ha dovuto spendere una settimana a cercare reference per conto suo, mentre il designer pensava che quelle fornite bastassero e non si è posto il problema della fase di ricerca delle reference da parte di chi fa la grafica o il 3D (Cristian parla al femminile perché la 3D artist del suo team è una donna, ma il discorso è generale). Anche questo, come l'apprendimento, è uno di quei task che non bisogna trascurare.

0.3.15.3 Analizzare i blocker prima di risolverli

Il consiglio è **analizzare i blocker**, non risolverli appena si presentano, perché si rischia di trascurare il problema alla radice. Cristian richiama una delle prime lezioni del primo anno, quella su **bisogni e problemi**: bisogna capire se il blocker è un problema o un bisogno, e agire di conseguenza. Il guaio nasce quando lo si trascura o, peggio, quando lo si risolve in fretta senza averlo considerato: "poi vi si ritorcerà contro".

L'esempio dei muri invisibili. *L'Inf* è un gioco di esplorazione ambientale, per lo più all'aperto: su cinque livelli, circa l'80% del gioco è all'esterno. Per dare *meaning* al livello finale, quello della boss fight e della rivelazione narrativa su dove siano finiti tutti gli esseri umani del pianeta, il team ha deciso di sottolinearne la diversità rendendolo **al chiuso**.

La camera è in terza persona e abbastanza lontana dal giocatore, non classica. Dare le collisioni alla camera sembrava fastidioso, perché a ogni rotazione "sbatte" contro i muri. La prima idea: **rendere**

invisibili i muri quando la camera ci passa dietro. Non si potevano far sparire tutti insieme quando il giocatore era solo dietro il muro destro, quindi ogni muro aveva il suo *trigger*, che stabiliva quante e quali pareti dovessero sparire. Il livello dura mezz'ora, con una buona decina di minuti di esplorazione al chiuso, completamente circondata da pareti: **un mese di lavoro** solo per assegnare i trigger, cambiare i materiali e gestire il tutto. "Era l'idea geniale che ci era venuta in quel momento."

Quando hanno consegnato la beta e l'hanno fatta provare ai professori e ad altri giocatori, tutti hanno detto più o meno la stessa cosa: i muri che spariscono **non sono eleganti e rompono l'immersione**. Hanno tagliato tutto e rimesso le collisioni alla camera. Se avessero analizzato bene il problema prima, avrebbero capito che bastava **gestire meglio gli spazi**: non fare curve strette ma allargarle, così che anche ruotando la camera il giocatore tendenzialmente non sbatta contro il muro. Era "assolutamente una questione di riallargare più o meno tutti gli spazi"; ed è ciò che hanno fatto alla fine, insieme alle collisioni. "Ad averlo fatto prima avremmo risparmiato due settimane di tempo"[32]. Ecco perché è importante analizzare i blocker.

0.3.15.4 Bonus: i feedback costanti

Cristian nota che, soprattutto ultimamente, in una fase di *lockdown*[33] in cui tutti sono costantemente in contatto su Discord, "anche più di quanto lo saremmo nella vita vera", c'è un **bisogno costante di dare feedback** sul lavoro.

Lui è pro feedback in qualsiasi momento, anzi "ossessionato": all'inizio ha chiesto alla classe di criticarlo e di dargli consigli sulle slide, perché è l'unico modo per migliorare. Il problema è che **i feedback costanti producono un calo di produttività enorme**. L'esempio è personale: Cristian è anche il VFX artist del team, e sugli elementi estetici c'è sempre modo di migliorare, quindi riceve di continuo feedback ("io lo farei più grande, rosso, verde, giallo, a forma di triangolo, a forma di quadrato"). Sono feedback correttissimi: chi li dà, soprattutto in produzione, non lo fa perché pensa che tu sia scarso, ma per il bene del gioco, che vive e vuole ben prodotto. I migliori mandano anche video di riferimento ("ho visto questo VFX in questo gioco, sapresti rifarlo?"), e questo ben venga. Il problema è che siano **costanti**.

Il consiglio: **limitare la richiesta e l'ascolto dei feedback a fasi specifiche** (alla fine della sprint, oppure in un controllo intermedio). Dipende anche da che cosa si sta facendo: con i VFX molte cose dette oggi sono sistemate il giorno dopo; ma se chi costruisce un livello, in block out o in *environment*, riceve mille feedback per ogni metro quadro ("un po' più a destra, un po' più a sinistra"), il lavoro si ferma lì. Sia perché chi lo fa si **demoralizza**, sentendosi costantemente criticato, sia per una questione di fiducia, tornando al discorso di Simone: "fidati di me come io mi fido di te. Fammi completare il task; poi, quando ti chiedo il feedback, o più avanti, è ben accetto, anzi ben venga".

Cristian si scusa per l'apparente presunzione di definire "fondamentali" i propri consigli: lo fa perché, se li avesse avuti prima, si sarebbe risparmiato tantissimi problemi. Diego scherza che andranno nelle slide "con la mia faccia super inquietante"; Cristian dice che può passare anche le sue slide.

0.3.16 Il calcolo della velocity

0.3.16.1 Confrontare il fatto con lo stimato

Sulla parte pratica Simone è stato più esaustivo, quindi Cristian si concentra sull'interpretazione[34]. La velocity consiste nel **confrontare ciò che è stato fatto con ciò che era stato stimato**, in numero di task o in pesi, a seconda di come la si gestisce. Da lì si capisce che cosa è successo e che cosa si può prevedere:

- **fatto > stimato**: si può prevedere di più; nella sprint successiva si aumentano i pesi o il numero di task;

[32] Poco prima Cristian parla di "un mese buono" di lavoro sui trigger (correggendosi da "due mesi"); le "due settimane" sono forse il risparmio netto stimato, tenendo conto del lavoro di allargamento degli spazi che è stato comunque necessario. La trascrizione non permette di chiarirlo.

[33] Il riferimento al lockdown colloca la lezione nel periodo della pandemia di Covid-19, coerentemente con l'accenno di Simone al Covid e alle sedi chiuse ("quando si poteva andare").

[34] Un breve tratto della spiegazione iniziale è perso nella trascrizione, che riprende a metà frase ("nel comparare quello che è stato fatto da quello che è stato stimato").

- **fatto < stimato**: o si è esagerato con la stima (succede sempre, così come succede sempre il caso opposto), oppure ci sono stati problemi gravi o *blocker* nella produzione, ed è fondamentale capire quale delle due cose sia successa;
- **fatto = stimato**: succede quasi mai, ma è l'obiettivo. Le possibilità sono due: si è trovato il proprio **bilanciamento**, e si può continuare così; oppure **non si sta applicando bene Scrum**, ad esempio perché si calcola la velocity su ciò che si è fatto invece che sulla stima.

0.3.16.2 Più di un numero

L'obiettivo non è arrivare a una velocity pari a 1, intesa come rapporto tra fatto e stimato (100 punti stimati, 100 fatti). La velocity è **molto più di un numero**: dà almeno tre informazioni.

1. La bontà degli strumenti di stima. Planning poker, Fibonacci e tutti gli strumenti di stima sono utilissimi, ma **vanno testati**: la velocity mostra se ci sono metodi sbagliati di stima dei pesi o del numero dei task.

2. La raggiungibilità dello scope. Se per tutta la produzione la velocity resta, ad esempio, a 0,8, sempre al di sotto di quanto dovrebbe essere (in teoria non dovrebbe succedere, perché ci si dovrebbe tarare), e si sono previsti dieci livelli, si capisce già da subito che **non ci si arriverà mai**, a meno di aggiungere uno o due anni di sviluppo.

L'esempio degli otto livelli. Finita la *vertical slice*, il team ha ricontrollato tutta la preproduzione e impostato il gioco come un gioco completo, non come la creazione di una parte: c'è tutta la storia, ci sono tutti i livelli, tutto ciò che fa il giocatore dall'inizio alla fine, chi incontra e quando; tutto scritto, ma tarato e lasciato **flessibile**, "altrimenti ricadiamo nella trappola del Waterfall".

A settembre, all'inizio dell'anno ("per farvi capire quanto il processo è rapido"), avevano previsto **otto livelli**. Sembrava una stima credibile: il vecchio team, con due level designer che volevano fare esattamente quel lavoro, produceva un livello in **due o tre settimane**, a una qualità abbastanza elevata: magari mancavano l'illuminazione, il *foliage* e alcuni dettagli, ma il livello era **giocabile**, che è la parte fondamentale. Cristian, unico rimasto del vecchio team, si è basato su quella stima: "da qualche parte dovevo cominciare".

Ma i level designer del nuovo team **non sono level designer puri**: sono soprattutto game designer, interessati al game design, che hanno deciso di sperimentare con il level design per necessità (il game design serve sempre, ma se manca un level designer, visto che il percorso di studi insegna sia level sia game design, ci si adatta). C'è chi si è adattato a fare il level designer, e chi ha deciso proprio quell'anno di specializzarsi nel level design: "perché avrei dovuto dirgli no, tu non sei un level designer? Anzi, ben venga". L'unica cosa è stata prevedere che avesse più bisogno di apprendimento: magari sbaglierà di più, dovrà informarsi su come si fanno i livelli, avrà bisogno di più feedback. L'importante è capirlo all'inizio.

Guardando l'andamento del primo livello, Cristian vedeva che **in un mese avevano a malapena raggiunto il block out**. Ha convocato una **riunione di emergenza**, lunghissima (non una revisione della sprint da venticinque minuti come quelle di Simone, ma un altro tipo di riunione). Il suo obiettivo non era tanto convincerli che lo *scope* era troppo alto (ci è riuscito), quanto **assicurarsi che non la prendessero come una sconfitta personale**. È stato difficile ma interessantissimo: non se la sono presa né con se stessi, per non riuscire a performare, né con lui, che stava di fatto dicendo "non ce la farete". Anzi, nonostante il taglio, sono stati **loro stessi a proporre la soluzione**: "ok, abbiamo capito; facciamo cinque livelli, e se vediamo che arriviamo (speriamo a fine marzo) con tutti i livelli, poi possiamo decidere di aumentarli o di cambiare parti del gioco". Cristian l'aveva già in mente, ma il fatto che l'abbiano proposta loro l'ha reso "veramente un successo".

Tutto questo grazie a un numero. Se Cristian fosse arrivato tra un mese e mezzo, dopo che avevano già fatto quattro livelli "con l'acqua alla gola", dicendo "ne dobbiamo fare altri quattro entro marzo", e poi a fine marzo "basta, non ce la facciamo", sarebbe stato molto più grave[35]. "Ringraziamo in questo caso la

[35] Il passaggio è confuso nella trascrizione ("mi ha portato ad affrontare queste situazioni che affrontate a fine marzo", "non potne quattro"): il senso ricostruito è che la velocity ha permesso di anticipare a inizio produzione una decisione che, presa a fine marzo, sarebbe stata molto più dolorosa.

velocity."

3. Lo stato del team. La velocity permette di capire lo **status del team e le sue performance**, perché si basa su una stima fatta a partire dalla settimana precedente. Si ricollega al "progettare il *pace* dello sviluppo" (sezione 0.3.14): ci si può basare proprio sulla velocity per capire qual è il ritmo attuale e prevedere i prossimi. "Tutto torna."

0.3.16.3 Bonus: mai tornare indietro

Un bonus che "più che un bonus è un'altra lezione": **prevedete sempre task che portano a un incremento del prodotto**. La produzione può rallentare, perché possono esserci difficoltà, ma **mai, mai, mai andare indietro**.

È un'altra lezione insegnata da *L'Inf*, legata alla stesura del backlog. Quando Cristian aveva scritto tutto il backlog ("400 task: come vedete ogni tanto dico 400, 600, 300, giusto per dire che erano veramente tanti") e si accorgeva di aver sbagliato, invece di adattarsi e appoggiarsi alla produzione in corso **fermava la produzione** ("ditemi pure che ero folle: lo ero") e riscriveva da capo tutti i task, arrivando magari a mille o riducendoli.

Lo stesso vale per i **documenti**. "La documentazione è bellissima": Cristian era un grande fan della documentazione (Diego può confermarlo), ma lavorando con Scrum, per ragioni di produttività, ha dovuto **scendere a compromessi**: darsi un limite su quanto un documento è bello e ricco di dettagli. Il motivo è sempre l'esempio della prima meccanica, l'assorbimento e sparo: tutti i GDD del gioco si basavano su di essa, con tutte le interazioni, anche secondarie (che cosa si poteva assorbire? Nemici, oggetti, chiavi, cose per recuperare vita, bonus, malus, qualsiasi cosa). Tagliata la feature, tutti i GDD sono stati "letteralmente presi e buttati", e Cristian ha deciso di rifarli tutti da capo.

Questo approccio potrebbe funzionare in Waterfall ("finiti i documenti, iniziamo la produzione"), ma l'obiettivo era **consegnare un videogioco, non fare documenti**; e soprattutto il gioco era innovativo. È successo altre due volte: in tutto Cristian ha riscritto **backlog e GDD da capo più volte**[36], riportando ogni volta la produzione all'inizio ("ok, possiamo iniziare il prototipo; no, è sbagliato, ricominciamo"). Lo racconta per far capire quanto ci ha messo a entrare nel mindset agile, e per tranquillizzare la classe: non indovinarci subito è normale, "succederà il 90% delle volte".

Quando poi hanno fatto il prototipo, si sono accorti che **tutta la documentazione era sbagliata**: con un gioco innovativo non si poteva sapere che cosa avrebbe funzionato senza prima vederlo, perché non c'erano reference: nessuno aveva fatto un gioco d'avventura *match-3*. "Il *match-3* esiste, i giochi d'avventura esistono, mischiarli insieme è una follia", e lo dice "per criticarmi, non per vantarmi, perché non lo farò più".

0.3.17 Il tuning della produzione

0.3.17.1 Che cosa e quando

Il *tuning* della produzione si basa sull'analisi fatta: dove ci si trova, qual è lo stato delle cose, che cosa si potrebbe fare meglio e stimare meglio[37]; per poi tornare all'inizio del loop con più dati su cui basare una stima corretta.

Si può fare **a ogni sprint**, a seconda della durata, oppure **ogni serie di sprint**: "vedetelo voi". Cristian suggerisce la seconda opzione perché ritoccare continuamente il backlog per tagliare o aggiungere feature va contro il principio del lavorare sempre per incrementare il prodotto. Va da sé che, se si tratta di un taglio molto importante, **prima lo si prende e meglio è**: "ragionateci bene".

[36] Nella trascrizione: "è successo altre due volte quindi per ben quattro volte io ho riscritto tutto il backlog e tutti i GDD". Il conteggio non torna del tutto (una volta più altre due fa tre); forse Cristian conta anche la prima stesura.

[37] Anche qui l'inizio della spiegazione è parzialmente perso nella trascrizione ("che in parte si basa su ci si trovano, qual è il loro status").

0.3.17.2 Su che cosa basarsi

Sulle performance, cioè sulla velocity. Se la velocity è alta, si può dire: "a questo punto possiamo reintrodurre qualche feature che era nei *nice to have*[38] o che era addirittura stata tagliata". Se è bassa, si sistemano già milestone e backlog in base al risultato.

Sui feedback. A Level Up *L'Inf* ha ricevuto tantissimi feedback, per lo più da giocatori. Trattandosi di una *vertical slice*, spesso dicevano cose che il team già sapeva e che avrebbe implementato nella sprint successiva (ma, per questioni di tempo, da qualche parte bisogna tagliare). Altri, invece, hanno parlato di **accessibilità**.

Il cambio di camera. A questi si sono aggiunti i feedback costanti di Diego sulla camera: più che un "dovete cambiarla", un "perché non provate a cambiare camera?". Diego interviene: "c'è voluto del tempo a convincere Cristian: otto mesi". Alla fine l'ha fatto e sta funzionando, anche se ha pesato molto sul team. Il motivo: il **twin-stick shooter**[39] richiede una grande **dimestichezza con i controlli** (muoversi e mirare contemporaneamente con le due levette) e di per sé esclude gran parte del pubblico casual. I giocatori più casual dicevano proprio questo: "non riesco a camminare e mirare contemporaneamente". Il team se l'è segnato, ne ha discusso e alla fine ha accettato il cambio di camera, con tutte le conseguenze.

0.3.17.3 Flessibilità anche nella struttura dei documenti

Un promemoria sulla **flessibilità**: più si è stati flessibili all'inizio, più si riesce a rimettere mano a backlog e produzione in seguito. E questo vale anche, banalmente, per **come è strutturato un documento**: se ogni casella fa riferimento a un'altra casella, che fa riferimento a un'altra due righe sopra, modificare quel documento (o un'implementazione costruita allo stesso modo) richiederà molto più tempo. Flessibilità in tutto: nella scrittura dei documenti e nella produzione ("non lo ripeto, altrimenti ricomincio").

0.3.17.4 Evitare il perfezionismo

"A proposito di perfetto": il consiglio sul *tuning* è **evitare in tutti i modi il perfezionismo**. Cristian ci sta ancora lavorando "tantissimo" ("anche qui Diego lo sa meglio di me"): è migliorato molto e nei progetti ci riesce, ma in altre cose pretende ancora troppo. Il perfezionismo **va contro la flessibilità e crea inevitabilmente dei blocker**. Lo sta vivendo in prima persona, non per sé ma con altri membri del team.

Il block out con il foliage. Il task è "fai il block out di questa parte del livello" (dare l'intero block out di livelli da mezz'ora richiede comunque tempo, quindi il task riguarda la parte iniziale). E nel block out Cristian si ritrova il *foliage*, il *landscape* scolpito bene, le rocce. "Va benissimo perché ti piace il lavoro... anzi no, sto proprio dicendo che non va bene." L'approccio di voler consegnare il task nel miglior modo possibile è comprensibile; il problema è che questo perfezionismo **blocca tutti gli altri**, per moltissimi motivi. Se si passano due ore a mettere il foliage e poi, quando si va in gold a rifinire il livello, si trova un foliage migliore per il *mood* estetico del gioco, quel lavoro va buttato. Il perfezionismo crea **buchi di produzione** che alla lunga diventano letteralmente mesi di lavoro buttato o da buttare: un peccato per chi lo fa e per chi lavora con lui.

La pressione sugli altri. Un'altra conseguenza: il perfezionismo **mette pressione a tutti gli altri**. Chi lavora alla seconda parte del block out, con il task di renderla giocabile, vede che la prima parte ha foliage, rocce e tutti i props (non l'illuminazione, che negli ambienti aperti è tendenzialmente globale, salvo situazioni specifiche) e si sente in dovere di raggiungere quel livello di rifinitura[40], trascurando la parte di gameplay, che invece nella prima parte era stata fatta. Il perfezionismo di uno ha generato **ansia** nell'altro, ed è da evitare in ogni modo.

[38] Nella trascrizione "nice to the epoch": quasi certamente *nice to have*.
[39] Nella trascrizione "twin steep shooter".
[40] Nella trascrizione "si sentirà in dovere di raggiliere la pozione della vita": passaggio incomprensibile, il senso è ricostruito dal contesto.

I criteri di accettazione. È un altro motivo per cui sprint e task devono avere un **goal** e un **criterio di accettabilità**, per usare i termini di Scrum: il task si considera fatto quando è *done*, e va bene così, perché lo si è valutato prima. Tutto ciò che si fa oltre il *done* è **tempo perso**. Può andare bene che quello che fai adesso non dovrai farlo dopo, ma è sbagliato come approccio: se il lavoro è stato fatto bene prima, si può andare avanti tranquilli, e al passo successivo ci si penserà dopo.

0.3.18 Domanda: come comunicare un taglio al team

Prima della seconda parte, che Cristian promette più breve, una domanda: sul piano relazionale ed emotivo, come ha gestito il dover convincere, o comunque spiegare, al team la necessità del taglio da otto a cinque livelli?

Ricordare come funzionano i feedback. "Ottima domanda." Emotivamente è stato pesante, perché non tutti reagiscono ai feedback allo stesso modo. Per questo, a ogni riunione, Cristian ricorda i principi dei feedback ("so che sembra banale, so che non serve"), in particolare una clausola che tutti dimenticano: **il feedback si può accettare o rifiutare**. Se non si è d'accordo, lo si può ignorare; invece tutti partono dal presupposto che un feedback vada seguito al 100%[41].

Partire dai dati. Sembra brutto affrontare una questione emotiva partendo dai dati, ma l'obiettivo era **far arrivare loro alla soluzione**, non dargliela. Ed è andata così: sono stati loro a dire "per adesso facciamo cinque livelli, e poi, **se ci riusciamo**, implementiamo gli altri". Quel "se ci riusciamo" era ciò che Cristian voleva sentire: se fosse rimasto solo "ne facciamo cinque", l'avrebbero vissuto come un limite su se stessi ("non ho fatto in tempo a farne otto"). Il "se ci riusciamo" gli ha comunicato che avevano capito che **non era una critica al loro modo di lavorare**, né un modo per dire "non siete capaci": lo aveva sottolineato più volte, magari non con parole così dirette, ma "un po' *soft-skillato*, come diciamo noi designer".

Hanno colto che non c'erano alternative ("otto non li facciamo, punto"), ma anche le possibilità: facciamo cinque livelli, abbiamo altro tempo, perfezioniamo ciò che abbiamo fatto; vogliamo aggiungere una nuova modalità di gioco? Benissimo. Altri due livelli? Vediamo, in base al tempo. Esserci riuscito lo ha reso "super fiero".

0.3.19 Il team management

0.3.19.1 Senza team non esiste Scrum

Cristian accelera, per stare nei tempi. Il punto da capire è che **Scrum si adatta al team**: quindi il team è la cosa più importante di tutte, più di Scrum stesso. "Se io mi adatto al team, il team deve esserci. Senza team non esiste Scrum."

0.3.19.2 Una produzione senza stress

Di conseguenza, una produzione in Scrum **dovrebbe rendere il team rilassato**. Come il team di Simone, anche il primo team di Cristian (quello fino alla *vertical slice*) **non ha mai fatto crunch**. Il nuovo team sì, per i grossi cambiamenti già raccontati: la camera è cambiata completamente, e "cambiare la camera in un gioco è fare un altro gioco". Cristian aveva già spostato la scadenza ("invece di finire a marzo finiamo ad aprile"), ma il team si è sentito comunque in dovere di risolvere prima lo scompenso creato da quei cambiamenti.

In generale ci si dovrebbe sentire meno stressati per un motivo semplice: chi guida sta dando i task che lo sviluppatore ha chiesto, verbalmente o dimostrando di saperli fare. "So che in due settimane devo fare cinque cose, faccio queste cinque cose": non c'è la pressione di dover fare il più possibile. Se poi uno è

[41] Nella prima lezione zero il relatore invitava ad accettare il feedback "a prescindere" (sezione 0.1.14); lì però "accettare" significava ascoltarlo senza difendersi, non necessariamente applicarlo. Le due indicazioni sono quindi compatibili: si ascolta tutto, poi si decide che cosa farne.

perfezionista e passa da cinque cose a quindici perché "ha tempo", il problema ("sembra brutto dirlo") è suo, nasce nel singolo individuo; e chi guida deve basarsi sul team, sulle performance del team.

0.3.19.3 Mai la velocity sui singoli

Da qui un consiglio sulla velocity: **non calcolate mai, mai, mai la velocity delle singole persone**, e nemmeno i task fatti dai singoli. Lo si può fare per sé, ma tenendolo "chiuso in un'agenda scritta a penna": se un membro del team scopre che si stanno valutando le sue prestazioni individuali, **si sente giudicato** e soprattutto non sa più come dire che non vuole fare una cosa.

A Cristian è successo: "mi hanno sgamato". Si è dovuto scusare, anche se non erano scuse: ha detto la verità, cioè che lo faceva più per capire lui la produzione che per valutare le loro performance. Ma spiegarlo è complicato, quindi è meglio non farlo: tanto la velocity si basa sulle performance del team, e i dati individuali non servono. Se li si vuole per sé, per capire a chi dare un task piuttosto che a un altro (se una persona in un'ora fa dieci implementazioni e un'altra una, ci si regola quando si selezionano i task), non bisogna assolutamente farli ricadere sulle persone.

0.3.19.4 Primo aspetto: adattare Scrum al team

Per una produzione senza stress servono due aspetti chiave. Il primo è che **Scrum va adattato al team**: si prende la base di Scrum (quella di Diego, dei libri, di Simone e di Cristian), la si **testa**, si **analizzano** la velocity e tutto il resto, la si **tara** ("*daily scrum*? No, facciamolo ogni tre giorni, ogni settimana, o non facciamolo"), la si **riprova**, e si ripete il ciclo per tutta la produzione. Alcuni esempi da *L'Inf*:

- i **daily scrum** li fanno **ogni tre giorni**, "se va bene", e dipende anche dal tipo di produzione;
- la **lunghezza delle sprint** è flessibile (una, due o tre settimane) e dipende:
  - dalla **disponibilità del team**: se tutto il team non c'è, è inutile fare una sprint di una settimana;
  - soprattutto dalla **fase di produzione**: quando si implementano script e in una settimana si riescono a implementare tantissime meccaniche, la sprint dura una settimana, al termine della quale si controlla ciò che è stato fatto e si sceglie che cos'altro implementare; quando si passa al level design, con lavori molto dispendiosi in termini di tempo, le sprint si diluiscono, e infatti in quel momento sono a sprint di due settimane, concentrati sul completamento dei livelli[42];
- la velocity si calcola **sui task e sul loro peso, non sulle ore lavorative**, come diceva Simone. Soprattutto per degli studenti, che non hanno ore fisse. In un ambiente di lavoro in cui tutti sono disponibili dalle 9 alle 18 potrebbe cominciare ad avere senso: "secondo me no, ma ve lo dico per correttezza".

0.3.19.5 Secondo aspetto: tutti devono conoscere il mindset

Il secondo aspetto è che **tutti i membri del team devono conoscere il mindset di Scrum**[43]. Per due motivi:

- se tutti lo conoscono, il rischio di cadere nel Waterfall o nel perfezionismo si riduce, perché ognuno sa che cosa è giusto e che cosa è sbagliato;
- tutto il team ha **lo stesso goal**: se la sprint chiede il block out, tutti sanno che, secondo la mentalità agile, va bene farlo ma non deve essere perfetto. Bisogna fare i *done*, non i *perfect*.

Come far capire a tutti la mentalità di Scrum? Cristian usa due modi.

1. **Parlarne costantemente**, a ogni riunione, come sta facendo con la classe: sia per migliorare lui stesso e farsi entrare il mindset, sia per trasmetterlo agli altri.

[42] Nella trascrizione "sull'ultima mento, sull'ultima azione, non mi ricordo, scusate. Per completare il livello": forse "l'ultima milestone".

[43] A questo punto Cristian chiede a Diego cinque minuti in più per chiudere bene, e Diego acconsente.

2. Il più efficace: **lo shipping**. Il team riesce a consegnare quasi costantemente. Ci sono momenti in cui un problema cade all'ultimo, come in quella settimana, con alcuni strumenti particolari che li hanno rallentati, ma si risolve tutto. Dimostrando di riuscire a **mantenere le milestone**, il team ha acquisito fiducia in lui e ha capito quanto è potente Agile. Nella *vertical slice* consegnavano le build addirittura prima di quanto richiesto; e la consegna prima di Natale, nota da settembre, è stata rispettata nonostante il cambio di camera e tutto il resto. Il messaggio: **non è importante che ciò che si consegna sia perfetto, ma che rispetti le condizioni di accettabilità** che ci si è dati.

Sono i due metodi che funzionano meglio finora; se ne troverà altri, li condividerà: "tutto è in continua evoluzione".

0.3.19.6 Il post-mortem

Il *post-mortem* è **la garanzia di capire se ciò che si sta facendo è corretto**: serve a vedere come stanno andando il team e la produzione, se il mindset di Scrum è stato compreso o no, e ad agire di conseguenza.

Quando farlo lo si decide liberamente: a ogni sprint, a ogni gruppo di sprint, a ogni milestone.

Ciò che Cristian sottolinea è che **nel post-mortem devono parlare tutti**, anche lo sviluppatore che non vuole mai parlare: bisogna dargli il tempo per farlo. Un modo è **organizzarlo in anticipo**: se è tra una settimana, lo si comunica la settimana prima. Cristian prepara sempre un **documento della riunione** prima e lo aggiorna dopo, con una sorta di elenco: sembra burocratico, ma aiuta a mantenere il focus. Bisogna far capire a ciascuno che tra una settimana parlerà, e non può sottrarsi (se si assenta ci perde lui). E bisogna dirgli di **prepararsi** qualcosa da dire, magari scrivendolo o facendo una piccola revisione: se ti viene in mente una cosa da dire la settimana dopo e non la scrivi, finirai per pensare "sai che c'è, non la dico"; se l'hai scritta, non puoi dimenticartela, e hai qualcosa di cui parlare. È fondamentale perché bisogna tenere conto di **tutti i tipi di personalità**: non solo i giocatori hanno personalità diverse, anche gli sviluppatori hanno personalità con bisogni diversi.

0.3.19.7 La sincerità

Cristian propone di tagliare la parte "sdolcinata", ma Diego vuole sentirla per intero ("non si sfugge al miele"; "poi mi manderete gli IBAN e vi pagherò il tempo perso"). Come Diego sa, Cristian cerca sempre di metterci "qualcosa di filosofico". Per **sincerità** intende che, in qualsiasi momento della produzione, **tutti i membri del team devono sentirsi protetti in ciò che dicono** e liberi di esprimere opinioni e feedback. È importantissima sia nella produzione sia nel team.

Nella produzione. Senza sincerità la produzione va a picco; si può comunque consegnare, non è obbligatoria, ma con la sincerità si realizza un prodotto migliore. Se a inizio produzione si pensa che una meccanica "faccia schifo" (prende un *player type* completamente fuori target e richiede troppo lavoro per essere implementata), **è quello il momento di dirlo**. Altrimenti "il karma vuole che tu stesso che pensi questa cosa sarai colui che la andrà a implementare", e la farai male, perché non vuoi farla. Se non l'hai detto, o l'hai detto troppo tardi, "ti mangerai le mani".

E il **"ve l'avevo detto"** va risparmiato: anche se il giorno dopo arriva la conferma di ciò che si era detto, anche se è umano, va evitato, perché crea attriti nel team che non si risolvono più (Cristian parla per esperienza diretta). È lo stesso spirito del *disagree and commit* (sezione 0.2.12).

In Scrum. Se uno sviluppatore dice "questa settimana so fare 15 task" e ne fa 7, la prima volta passa. La settimana dopo dice di nuovo "ne faccio 15, tranquillo", magari inventando scuse ("quel problema non ce l'ho più"). Una volta, due volte; **alla terza bisogna chiedere sincerità**: qual è il problema? Non hai tempo?

Perché sul tempo, "ragazzi", **è importante dire quanto tempo si ha, non garantirlo**. Se il team (cinque persone, nel caso di Cristian) dice di avere due ore al giorno, lui pianifica una produzione da dieci ore al giorno; se il team dice di avere otto ore al giorno, pianifica di conseguenza ("80 ore"[44]), e se poi in realtà le ore sono dieci, gli diranno "ci dai troppi task".

[44] Così nella trascrizione; con cinque persone a otto ore al giorno il totale sarebbe 40 ore, oppure 80 su due giorni. Il senso dell'esempio non cambia.

Bisogna essere sinceri **prima di tutto con se stessi**: se un task richiede due giorni, si dice due giorni; magari tre, perché un "tempo cuscinetto" ben venga, ma lo si dice per com'è. E non bisogna proteggersi dal timore che "se dico la verità nessuno si fiderà più di me" o "mi daranno più task": è compito dello scrum master, ma di tutto il team, far capire che **si è tutti sulla stessa barca**.

Nel team. Gli attriti generati dalla mancanza di sincerità si trasformano in **relazioni migliori** quando c'è sincerità. Le incomprensioni vanno **risolte subito**, affrontandole; magari porteranno a un litigio, se proprio quello è il finale, ma tra persone mature si dovrebbe riuscire a risolvere in modo pacifico. Per questo, sulla slide, la sincerità è definita **"rischiosa ma necessaria"**.

0.3.20 Chiusura della lezione

Cristian salta la slide di riepilogo (la lezione si potrà rivedere online) e lascia i suoi contatti: **Telegram**, dove molti studenti lo trovano già nei gruppi (in particolare quelli dell'accademia online), oppure l'**email**, che Diego può fornire. Si rende disponibile per qualsiasi cosa, sulla lezione o su un'idea da sottoporgli ("ho pensato così, secondo te può funzionare?"), precisando di non volersi sostituire in alcun modo a Diego. Si scusa per i dieci minuti finali "di corsa".

Diego lo ringrazia per la disponibilità, ricordando che anche lui, da studente, ha un carico di lavoro notevole. L'obiettivo della lezione era far vivere alla classe un'esperienza e dare informazioni simili a quelle di chi ha fatto lo stesso percorso, ma ha **internalizzato Agile** meglio di altri: tutti hanno avuto le stesse lezioni, ma Cristian e Simone sono quelli che le hanno capite meglio. Ci sono molti *insight* utili in entrambi gli interventi, e i punti chiave di Agile sono stati ripetuti più volte, "perché alla fine Agile è comprendere i punti chiave". "Grazie ancora, e buon pranzo."[45]

Punti chiave del capitolo

- **Agile è un mindset, non un framework**: non ha regole, ha una logica da **internalizzare**, come una lingua che a un certo punto si smette di tradurre. I quattro valori non vanno seguiti alla lettera: vanno adattati al team e al progetto, e non è detto che Agile serva.
- I *framework* (Scrum, Kanban, ...) hanno regole e si basano sull'**inspect and adapt**, cioè sulle iterazioni. Si parte rispettandoli al 100%, si capisce che cosa non funziona e perché, e **ci si costruisce il proprio**.
- **Meno riunioni possibili**: unire planning e post-mortem, stare in 25–30 minuti, report da un minuto. **Nelle riunioni non si risolvono i problemi**: si fissa un appuntamento con chi può aiutare. Un **timekeeper** può far rispettare i tempi.
- Le **sprint** (una o due settimane) sono l'unità di tempo: le milestone si misurano in sprint, non in mesi.
- **Consistency**: una cerimonia saltata fa crollare le altre a effetto domino. Per questo ciò che non funziona per il team (spesso il *daily scrum* o il *planning poker*) va **tagliato**, non trascinato.
- Il **backlog** si fa **a scaglioni**, per milestone, partendo dalle *epic* e dai documenti (visione, GDD, *versioning* con tutti gli ambiti, LDD). Si organizza per feature, per ambito con un **color code** condiviso, per milestone e sotto-milestone o per priorità, tutto in un solo strumento.
- Ogni task (e ogni sprint) ha un **obiettivo verificabile** e un **criterio di accettazione**: "completare il livello 5" non lo è. Le *user story* aiutano, ma non sono obbligatorie.
- **Mai stimare in ore**: si stima in modo **relativo**, con la sequenza di **Fibonacci**, partendo dagli estremi. I valori sono arbitrari ma **consistenti**, e quindi utilizzabili.

[45] La trascrizione si chiude con la parola "grazie" ripetuta diverse centinaia di volte: è un artefatto tipico della trascrizione automatica su un tratto di audio silenzioso o di saluti sovrapposti, non un contenuto della lezione.

- La **velocity** (somma dei pesi dei task *done* per sprint, confrontata con lo stimato) **prevede il futuro**: dice se gli strumenti di stima funzionano, se lo *scope* è raggiungibile e come sta il team. Permette di tagliare per tempo e di arrivare in fondo **senza crunch**. L'andamento a dente di sega è normale.
- Le prime sprint **si sbagliano tutte**, ed è giusto: la quantità si tara sulle disponibilità e sulle capacità attuali, prevedendo i **tempi di apprendimento**.
- Si può **progettare il ritmo dello sviluppo**: dopo periodi di lavoro invisibile, sprint che danno soddisfazione (nuovi nemici, VFX, *juiciness*) per il morale del team.
- I **blocker** vanno trovati subito e **analizzati alla radice** prima di essere risolti: la soluzione più rapida (i muri invisibili) può costare settimane.
- I **feedback costanti** abbattono la produttività: vanno raccolti in momenti specifici. Il feedback si può accettare o rifiutare.
- **Flessibilità** in tutto, soprattutto negli elementi innovativi: non prevedere tutto nel dettaglio (è Waterfall), non fermare mai la produzione per riscrivere backlog e documenti, lavorare sempre per **incrementare il prodotto**.
- Il **perfezionismo** crea blocker, lavoro buttato e ansia negli altri: si fanno i *done*, non i *perfect*.
- **Il team viene prima di Scrum**. La velocity non si calcola mai sui singoli. Tutti devono conoscere il mindset: parlarne a ogni riunione e, soprattutto, **consegnare** rispettando le milestone.
- Nel **post-mortem** parlano tutti: va annunciato prima, con un documento, chiedendo a ciascuno di prepararsi.
- Alla base di tutto ci sono la **fiducia** reciproca e la **sincerità**: dire subito ciò che non va, dire quanto tempo si ha (non garantirlo), evitare il "ve l'avevo detto". La sincerità è **rischiosa ma necessaria**.

Chapter 0.4

Publisher e producer: come si porta un gioco sul mercato

Le prime tre lezioni zero hanno guardato al lavoro in team *dall'interno*: come ci si dà feedback (capitolo 0.1), come si gestiscono il tempo, le idee e la leadership (capitolo 0.2), come si applicano davvero Agile e Scrum (capitolo 0.3). Questa quarta lezione sposta lo sguardo **fuori dal team di sviluppo**, verso due figure con cui ogni studio indipendente prima o poi si confronta: il **publisher**, cioè l'azienda che finanzia, promuove e aiuta a pubblicare un gioco, e il **producer**, cioè la persona che tiene insieme tempi, budget e comunicazione.

A parlarne è Michele[1], "Miki" per gli amici, che vive e lavora da remoto a Torino come **producer per Raw Fury**, un publisher indie con sede a Stoccolma[2]. Non è un docente dell'accademia ma un ospite, invitato da Diego Ricchiuti[3], che segue la lezione collegato online e interviene un paio di volte. La lezione è un **workshop di circa due ore**, in presenza, con una parte della classe collegata da casa; Michele avverte subito che parla "abbastanza veloce" e che alla fine ci sarà tutto il tempo per domande, consigli "o quello che volete voi. Anche una birra, potenzialmente, se siete qui".

Tre avvertenze di lettura. La prima: Michele insiste più volte che ciò che racconta vale per **Raw Fury, oggi, e per la sua esperienza**; altri publisher e altri producer lavorano diversamente, e il capitolo conserva queste cautele. La seconda: su alcuni temi (marketing, PR, *early access*, VR) il relatore dichiara apertamente di non essere esperto e di andare "per supposizione"; anche queste incertezze sono riportate. La terza: la trascrizione è automatica e, oltre a storpiare molti nomi propri, mescola le voci del relatore e dei partecipanti; dove l'attribuzione di una frase o di una domanda non è sicura lo si segnala in nota. I riferimenti temporali ("il 2024", "il 2025") vanno letti rispetto al momento della lezione, che dal contesto si colloca tra la fine del 2022 e l'inizio del 2023[4].

Il percorso del capitolo segue quello della lezione:

1. l'attesa prima dell'inizio, con le domande di Michele sul percorso degli studenti (sezione 0.4.1);
2. chi è il relatore e da dove viene (sezione 0.4.2);
3. che cos'è e che cosa fa un **publisher** (sezione 0.4.3);
4. come funzionano i soldi: i **tre "malloppi"** di development, service e marketing (sezione 0.4.4);

[1] Nella trascrizione il relatore si presenta solo con il nome di battesimo ("Sono Michele [...] Michele o Miki, se volete"); il cognome non viene mai pronunciato. Nel capitolo lo si chiama semplicemente Michele.

[2] Nella trascrizione il nome è reso in molti modi diversi ("Raw Fury", "Rofury", "Rofiuri", "Rufiuri", "Roffiuri", "Arofiori"): si tratta sempre di Raw Fury, publisher svedese di giochi indipendenti. La sede a Stoccolma è indicata dal relatore stesso più avanti nella lezione.

[3] Nella trascrizione compare soltanto il nome di battesimo, "Diego": Michele lo chiama per verificare l'audio della videochiamata e lo ringrazia alla fine "per l'occasione". Il cognome è quello indicato per il direttore dell'accademia nel capitolo 0.1.

[4] Il relatore dice di aver lasciato il suo impiego precedente "quest'agosto" per entrare in Raw Fury a settembre, cita come uscito "a settembre" *Dome Keeper* (pubblicato da Raw Fury nel settembre 2022), parla di una cena di Natale avvenuta "recentemente" e di un calendario di uscite già completo per il 2023 e il 2024. La datazione è quindi una deduzione, non un'informazione esplicita della lezione.

5. che cos'è un **producer**, secondo Michele (sezione 0.4.5);
6. come lavora un producer in Raw Fury: fuori dal team, in coppia, su più progetti (sezione 0.4.6);
7. le sue quattro aree: **demo ed eventi**, **QA**, **console porting** e **localizzazione**, con una domanda sul debito tecnico (sezioni da 0.4.7 a 0.4.11);
8. dopo la pausa, **cinque consigli** per i progetti degli studenti (sezione 0.4.12);
9. la lunga sessione di **domande**: compiti, bug, budget, contratti, potere decisionale, selezione dei progetti, finanziamento, lingue (sezioni da 0.4.13 a 0.4.17);
10. la chiusura (sezione 0.4.18).

0.4.1 Prima dell'inizio: il percorso degli studenti

La registrazione comincia una decina di minuti prima della lezione, mentre Michele prepara la condivisione dello schermo e aspetta che i partecipanti online si colleghino (per un po' non vede "nessuno connesso"; si sposta in modo da essere inquadrato anche quando si alza). Ne approfitta per farsi raccontare dagli studenti in aula come funziona il corso. La conversazione è frammentaria nella trascrizione, ma se ne ricavano diverse informazioni utili sul percorso[5]:

- tutti gli studenti presenti seguono lo stesso corso;
- il **primo anno** il progetto è un **gioco da tavolo**;
- il **secondo anno** è un **videogioco**, come progetto unico: si può riprendere il progetto dell'anno precedente oppure partire da uno nuovo;
- alla fine del **terzo anno** c'è la possibilità di **pubblicare su Steam** con la scuola (Michele chiede se anche su console: la risposta, per quanto si capisce, è no);
- per il *project management* i docenti spiegano gli strumenti, in particolare **Asana**, ma lasciano libertà su quali usare; quest'anno le scadenze sono più ravvicinate (sprint di due settimane) rispetto all'anno scorso, quando erano mensili[6];
- per il lavoro dopo il diploma, i più meritevoli hanno una giornata in cui incontrano alcune aziende con cui la scuola ha contatti; l'anno precedente l'ultimo giorno del secondo anno coincideva con quello in cui il terzo anno incontrava un'azienda.

Michele racconta che anche lui ha frequentato una scuola di game design, non la stessa ma un'altra scuola di Torino, tra il 2015 e il 2017[7]: due anni di game design, dopo i quali è "andato alla deriva" verso ruoli diversi dal game design. Gli studenti presenti vengono tutti dal Nord Italia (Milano soprattutto, ma anche Parma, Brescia, la Romagna; uno è originario di Bassano del Grappa, in Veneto). Prima ancora, confessa, aveva fatto il liceo classico e, non sapendo "che cavolo fare nella vita", si era iscritto a Economia perché il padre fa il commercialista: "quattro esami" e poi ha cambiato strada[8]. Alcuni studenti vengono da Lettere moderne e da Comunicazione.

Prima di cominciare, Michele chiede alla classe che cosa sappia di Raw Fury e del *publishing*, e quali siano le aspettative verso il workshop. La risposta è chiara: saperne di più sul **mondo indie** e sulla

[5] In questo tratto le voci si sovrappongono e molte frasi sono incomplete; la ricostruzione che segue riporta solo ciò che si capisce con ragionevole sicurezza. "Un gioco da Taos" è con ogni probabilità una storpiatura di "un gioco da tavolo".

[6] Il passaggio è molto confuso ("di più quest'anno che noi siamo conto di che lo scorso avevamo delle mensili con due settimane"): l'interpretazione è approssimativa.

[7] Nella trascrizione: "anch'io ho fatto la scuola, non è Hive, ho fatto Event Horizon a Torino". "Hive" è una delle grafie con cui la trascrizione rende il nome dell'accademia (si veda la nota nel capitolo 0.1); "Event Horizon" è riportato così come compare e non è stato possibile verificarlo. Non è del tutto certo che la battuta sia di Michele e non di un altro adulto presente, ma il seguito ("Anch'io game design e poi sono andato alla deriva non facendo game design") è coerente con il percorso che racconta poco dopo.

[8] Il seguito della frase ("E poi ho trovato a soccorso a Vitoraio") non è comprensibile.

pubblicazione, perché almeno uno dei progetti di quest'anno ha come obiettivo proprio la ricerca di un publisher. Poi Michele verifica l'audio con Diego ("Diego, batti un colpo se mi senti"), che risponde di sentire e vedere tutto: si conoscono "da un po', mettiamola così". Si presentano alcuni partecipanti collegati, tra cui Luca e Marcello, e la lezione comincia.

0.4.2 Chi è il relatore

Michele lavora nell'industria dei videogiochi da circa **sei anni e mezzo**. Il suo percorso è un buon esempio di come si possa entrare dal basso e crescere "restando lì":

- **QA tester a 34BigThings**, a Torino[9], dove ha lavorato a *Redout*. All'epoca lo studio non aveva tester: lui si è messo lì dicendo "tanto io rimango qua, non vi libererete di me", finché lo hanno assunto. Poco dopo serviva qualcuno che si occupasse della pubblicazione di *Redout* su PlayStation 4 e Xbox One, cosa che lo studio "non aveva mai visto", e gliel'hanno affidata. Ci è rimasto due anni e mezzo.

- **Senior QA tester a Rebellion**, in Inghilterra, lo studio di *Sniper Elite* e *Zombie Army*. Il lavoro era simile, ma su una scala molto più grande: se a 34BigThings era l'unico tester, con l'aiuto di un paio di volontari, a Rebellion il team di QA contava circa **25 persone**, più circa **200 tester esterni** (dall'India, dal Canada e da altri paesi). Promosso **lead tester**, ha lavorato a *Sniper Elite 4*, *Strange Brigade* e al porting di *Zombie Army 4* su Stadia ("che riposi in pace Stadia"), e poi come lead a *Sniper Elite 5*.

- Intorno a settembre 2021 Rebellion chiede ai dipendenti di tornare a lavorare in Inghilterra; lui, che lavorava da remoto dall'Italia, rifiuta ("non mi rivedrete in Inghilterra") e da novembre 2021 torna a **34BigThings**, dove resta fino alle dimissioni dell'agosto successivo.

- Da settembre lavora come **producer per Raw Fury**.

In tutto: tre paesi (Italia, Inghilterra, Svezia) e **diciassette giochi pubblicati**, "ma onestamente ho dovuto far fatica per ricordarmeli". Tra le slide compare anche *MXGP*, a cui ha lavorato come **playtester per Milestone**, prendendo il treno da Torino[10]. Fa il producer, in totale, da circa due anni.

0.4.2.1 La domanda della zia a Natale

Perché è qui? Per rispondere a una situazione che tutti conoscono: la cena di Natale in cui la zia chiede "ma come si chiama il lavoro che fai?". Di solito la risposta è "faccio i computer"; agli studenti, invece, vuole dare una risposta più completa ed esaustiva. Le cose da scoprire nel workshop sono due: **che cosa fa un publisher** e **che cosa fa un producer**.

0.4.3 Che cosa fa un publisher

0.4.3.1 Una casa editrice per videogiochi

La definizione che Michele usa con chi non è del settore è semplice: **pensate a una casa editrice di libri**. Il publisher di videogiochi fa la stessa cosa, con delle ovvie differenze: **cerca un prodotto da pubblicare e lo aiuta nella pubblicazione**, ricevendo in cambio una percentuale, una quota dei ricavi. "Alla fine non è qualcosa di così incredibilmente astruso."

[9] Nella trascrizione lo studio è reso come "3 for big teams", "3 for Big Things" e "Free Four Big Things": si tratta di 34BigThings, studio torinese noto per *Redout*, il gioco citato subito dopo ("Redout", "readout").

[10] Qui la trascrizione riporta anche un breve scambio con un altro presente che ha fatto il playtester a Milestone ("Io ho fatto playtester a Milestone da gennaio. No, raccordi 8, erano i primi minuto comodi"): il senso non è ricostruibile.

0.4.3.2 Come si cercano i giochi

Raw Fury cerca giochi in due modi:

- in modo **attivo**: si va nei luoghi dove si sa che esistono comunità di sviluppatori. Di recente il team è stato in tutto il Sud America: Messico, Uruguay, Paraguay, Cile, Perù e altri paesi[11];
- in modo **passivo**: c'è un'email pubblica a cui chiunque può mandare il proprio *pitch* (più avanti, nelle domande, si parlerà di che cosa significa "pitchare" a Raw Fury, sezione 0.4.16). Ogni singola cosa che arriva viene guardata e giocata, sperando di dire "mi sono accorto che sono dieci minuti che ci sto giocando e ci sono rimasto attaccato".

0.4.3.3 Che cosa offre il publisher

Una volta deciso che un gioco piace e che si vuole andare avanti, il publisher aiuta lo sviluppatore su due fronti: lo **sviluppo** e il **marketing**.

- **Paga lo sviluppo**: "ti paghiamo per il tempo in cui sviluppi il videogioco". Se lo sviluppo dura 12 o 24 mesi, per quel periodo lo sviluppatore e il suo team ricevono un salario. Si cerca di capire quanto costa un team di un certo livello in una certa area del mondo: un team che lavora da Barcellona costerà meno dello stesso team a San Francisco, e questi costi vanno ponderati.
- **Paga il marketing**: comunicati stampa, trailer e tutte le altre attività di promozione ("non è il mio campo, quindi perdonatemi se non vi so indicare tutte le attività di marketing").
- **Porta il gioco agli eventi**, fisici e virtuali: GDC, Gamescom, BitSummit in Giappone, Tokyo Game Show, PAX; e poi eventi come lo Steam Next Fest o, di lì a poco, un evento a tema *base building* su Steam. L'obiettivo è aumentare la **riconoscibilità** del gioco, fare in modo che la gente inizi a dire "ma questo gioco continuo a vederlo".
- **Aumenta le opportunità di vendita attraverso i partner**: Xbox Game Pass, PlayStation Indies, Nintendo. Ci sono momenti in cui il publisher presenta i propri giochi ai rappresentanti, ad esempio, di Microsoft per capire se c'è un interesse comune.

L'ultimo punto merita una precisazione: quando un gioco viene scelto da un partner come Game Pass o PlayStation Indies, gli sviluppatori ricevono un **bonus finanziario**. Per questo essere scelti è molto appetibile: significa un **pagamento immediato** all'uscita del gioco, indipendentemente da quante copie si vendono ("se porto il gioco su Game Pass, so che Microsoft mi darà una certa quantità di soldi").

0.4.3.4 L'obiettivo: il *magic*

Qual è l'obiettivo? Come per ogni publisher, pubblicare giochi che abbiano quel ***magic***: sul sito di Raw Fury c'è una pagina dedicata a questo concetto. Il *magic* è quella cosa per cui resti attaccato a un gioco senza sapere perché: "perché sto giocando questo? Non lo so, c'è quella roba".

Accanto a questo, gli obiettivi dichiarati sono tre:

1. che i developer abbiano una **buona esperienza** del processo;
2. che il loro prodotto **venda**, che recuperino (il *recoup*) i soldi investiti insieme al publisher e che, possibilmente, **guadagnino anche di più**. Michele cita *Dome Keeper*[12], il gioco a cui ha lavorato e uscito a settembre: il suo sviluppatore "per ora non deve più preoccuparsi, sta a posto per un bel po' di tempo". "È qualcosa che ci rende orgogliosi";
3. infine, che **l'investimento ritorni**: che il "malloppo" di soldi investito sullo sviluppatore torni indietro con qualcosa in più, perché quel qualcosa in più è anche ciò che va allo sviluppatore.

[11] La lista nella trascrizione finisce con "e tutto compagno cantante", probabilmente una storpiatura di "e compagnia cantante".

[12] Nella trascrizione "Don't Keeper" e "Domeomekeeper": si tratta di *Dome Keeper*, pubblicato da Raw Fury nel settembre 2022.

0.4.4 Come funzionano i soldi: i tre malloppi

Come lavora, in teoria, il publisher? Michele chiede di immaginare **tre "malloppi"**, illustrati nella slide come tre casse di soldi: **development**, **service** e **marketing**. All'inizio della collaborazione si decide di che tipo di gioco si parla (un gioco mobile, un gioco per Switch, un gioco che deve uscire su tutte le piattaforme, ...) e, a seconda del tipo di gioco, si decide la **dimensione** di ciascun malloppo.

0.4.4.1 Development: lo stipendio del team

Il *development* è il **salario che il developer riceve**. Per calcolarlo si può fare così:

1. contate le persone del team (ad esempio sei);
2. stabilite lo stipendio mensile di ciascuno (semplificando, mille euro a testa: "non a Milano");
3. moltiplicate: $6 \times 1\,000 = 6\,000$ euro al mese, per il numero di mesi in cui volete sviluppare il gioco;
4. aggiungete **un paio di mesi** di margine.

Quei mesi in più servono a coprire il ritardo con cui arrivano i primi incassi: quando il gioco inizia a vendere, i soldi di Steam non arrivano subito, e il publisher aggiunge quindi un paio di mesi di *lag*. Il risultato sono i soldi che il team riceverà. Un esempio numerico più realistico emerge nelle domande (sezione 0.4.14).

0.4.4.2 Service: tutto il lavoro che serve al gioco

Il secondo malloppo, il *service*, copre **tutto il lavoro che deve essere fatto sul gioco** oltre allo sviluppo vero e proprio:

- il **QA**: se serve pagare un team che testi il gioco, i soldi vengono dal service;
- la **localizzazione** e il **voice acting**: ogni gioco è diverso, uno ha bisogno di doppiaggio e un altro no, ed è impossibile dire in astratto quanto costi ("dipende da tante cose");
- il **console porting**;
- i **freelancer**: se il team di sei persone scopre a metà progetto di aver bisogno di un sound designer che all'inizio non aveva previsto, il suo compenso viene preso dal service.

Per il porting e per altri servizi Raw Fury ha anche uno studio proprio, **Fury Studios**[13], con programmatori, artisti e tester, che fornisce servizi al publisher: portare un gioco su Xbox o PlayStation, fare QA, oppure realizzare le **grafiche per gli store**, un servizio "assolutamente necessario".

0.4.4.3 Marketing: promuovere il gioco

Il terzo malloppo sono i soldi spesi per **promuovere il gioco**:

- **eventi**: se si decide che una persona del team di sei vada alla GDC, di solito la accompagna una persona del team di produzione del publisher. Sono due persone a cui servono aereo, hotel e pasti per una settimana: "non è *cheap*";
- **trailer**: come per lo sviluppo, anche per il marketing si assumono consulenti esterni; se il team media interno è occupato, si prende un freelance per fare il trailer ("vai su Fiverr e assumi un freelance");
- **PR**: articoli su siti come IGN, su blog, spazi come Xbox Wire o il PlayStation Blog. Michele crede che siano a pagamento, ma precisa che non è il suo ambito;
- **social e community**: Discord, Twitch e gli *streamer*, che vengono pagati perché giochino un certo numero di ore al gioco o facciano un certo numero di dirette.

[13] Così nella trascrizione. Più avanti, parlando del controllo tecnico sui progetti, Michele cita anche "il nostro studio in Croazia" (sezione 0.4.15): dal solo testo non si capisce se sia lo stesso studio di servizi o un'altra struttura.

0.4.4.4 Per saperne di più: il contratto pubblico

Questa, avverte, è un'infarinatura "molto, molto a volo d'uccello". Chi vuole approfondire può trovare online il **contratto di publishing di Raw Fury**, che è pubblico: cercando "Raw Fury Publishing Agreement" si trova tutto ciò che riguarda il flusso dei soldi, in una direzione e nell'altra. Su YouTube c'è inoltre una serie di video (più di uno) in cui un avvocato di professione, che cura un canale a tema legale, discute il contratto paragrafo per paragrafo: "quindi non solo ce l'avete da leggere per conto vostro, ma avete anche una persona che fa quel mestiere"[14].

0.4.5 Che cosa fa un producer

0.4.5.1 Un asterisco e una domanda alla classe

Prima di passare alla *production*, Michele mette un **asterisco**: quando dice *production* o *producer* si riferisce a Raw Fury, "in questo momento del tempo". Altri producer in altre aziende possono lavorare in modo diverso, e non pretende che questo sia *il* modo di lavorare.

Poi chiede alla classe se, nei propri team, ci sia una persona che si occupa della produzione. Risponde Gianluca[15], collegato da casa: è il **project manager** del suo team e, insieme a una persona che gli dà supporto, organizza il *workflow*, la struttura e la gerarchia del team. Cerca però di lasciare **più autonomia possibile**: il team è su base volontaria, quindi non controlla ogni singola cosa né dice esattamente che cosa fare a ogni sprint, ma lascia che le persone si autovalutino, in modo che siano più coinvolte e "ci mettano la testa anche loro" nel pensare a che cosa sia meglio fare. Michele risponde che va tutto benissimo.

0.4.5.2 La definizione di Michele

Anche qui Michele dà la propria definizione, "senza ambizioni", come "umile parere": **il producer è il *master* della programmazione e del budget**, cioè dei tempi e dei soldi[16].

Tutto ciò che riguarda il gioco in sé **non dovrebbe essere di competenza del producer**. A lui non interessa minimamente se il personaggio lo si vuole blu, rosso o viola, né "se salta, spara o spara bestemmie": la scelta giusta è quella che fa stare il progetto **nel tempo e nel budget prefissati**. La *vision* del gioco la mantiene il **lead designer**[17]; il compito del producer è fare in modo che **il gioco esca quando si pensa che esca, senza far fare crunch a nessuno e restando nel budget**.

0.4.5.3 L'anello tra due catene

Più di metà del tempo, nella giornata di un producer, se ne va a **passare informazioni da una persona all'altra**: il producer è **l'anello fra due catene**. Le persone del team hanno le loro specialità (design, programmazione, brand management, trailer, ...) e sono così concentrate sul proprio lavoro da non poter andare a vedere, in ogni momento, come sta lavorando l'altro membro del team. Il producer le mette in comunicazione, facendo sapere a ciascuno **ciò che è importante sapere per il proprio lavoro**.

Per questo, a chi fosse interessato a questo ruolo, serve essere **a proprio agio nel parlare con le persone**. E, aggiunge, forse il requisito fondamentale per lavorare nei videogiochi in generale è **essere una persona a modo**: se lo si è, non si farà fatica a fare questo lavoro[18].

[14] Il nome del canale non viene detto. Michele si offre di mandare il link a chi gli lascia un indirizzo email.

[15] Nella trascrizione il nome compare più avanti come "Gianluca Sutrini" ("Gianluca Sutrini alzato la mano"): la grafia del cognome non è verificabile. Il primo intervento non è introdotto da un nome; lo si attribuisce a Gianluca perché la sua domanda successiva, nella sezione 0.4.17.5, riprende lo stesso ruolo di project manager.

[16] "Programmazione" va inteso qui come pianificazione temporale (*scheduling*), non come scrittura di codice: lo chiarisce il seguito del discorso, in cui il producer non si occupa di nulla che riguardi il gioco in sé.

[17] Nella trascrizione "vado dall'e-design": con ogni probabilità "dal lead design[er]".

[18] Michele annuncia "gli unici due requisiti per lavorare nei videogiochi", ma nella trascrizione ne enuncia esplicitamente uno solo ("essere una persona a modo"); il secondo potrebbe essere l'essere un buon comunicatore, che è la voce della slide che segue subito dopo.

0.4.5.4 Le quattro caratteristiche del producer

La slide elenca quattro caratteristiche.

Comunicatore. Spesso le persone si isolano nel lavorare alla propria parte e hanno bisogno di qualcuno che le informi su ciò che devono sapere: "guarda, questa cosa che stai facendo non serve più perché abbiamo preso quest'altra decisione", oppure "stai lavorando su questo, ma in questo momento non è la cosa più importante: ci serve quest'altra". L'obiettivo è far comunicare le persone perché lavorino nel modo **più efficiente possibile**.

Problem spotter. Un producer deve saper capire che cosa, **a distanza di tempo**, potrebbe rivelarsi un problema: "perché facciamo così? Perché questa cosa, fra sei mesi o fra un anno, quando andiamo a pubblicare, potrebbe essere un problema". Questa caratteristica **migliora con l'esperienza** e va allenata nel tempo: chi fa il producer da sei mesi, al primo lavoro, non può sapere quali sono tutti i problemi con le **certificazioni delle console**. Secondo Michele è proprio ciò che **separa i buoni producer da quelli mediocri**.

Un esempio dalla sua esperienza recente: in Raw Fury c'è un gioco che ha bisogno di **voice acting**, e lui non ne ha mai avuto esperienza. Sa come funziona la pubblicazione su console, perché l'ha fatta più volte, ma non sa dire se un certo passaggio del doppiaggio debba avvenire tre o sei mesi prima dell'uscita. "Al prossimo gioco magari ne saprò un po' di più."

Quando ha preparato la slide aveva scritto *problem solver*, poi ci ha ripensato: **il producer non risolve i problemi, li individua e fa in modo che vengano risolti**. Alcuni li può risolvere lui, altri no[19].

Tutto il resto. L'ultima voce è **tutto ciò che non è coperto da altre professioni**. Se il team lavora per una scadenza fissata per dopodomani e tutti stanno facendo crunch ("succede anche che si fa crunch"), il producer si chiede che cosa possa fare perché **le ore in più siano ore che contano**: rendere le persone più comode, procurare le sedie che servono, ordinare e pagare il pranzo. Tutto ciò che non è lavoro, non è comunicazione e non è *problem spotting*, ma aumenta l'efficienza del team: "e incluso ordinare la pizza".

Michele chiude la definizione ricordando che è frutto di una piccola esperienza, perché fa il producer da due anni: è un *work in progress*, e tra un paio d'anni potrebbe dire cose diverse.

0.4.6 Il producer in Raw Fury

0.4.6.1 Fuori dal team

Tutto quanto detto finora si applica molto bene a un **producer interno**, che sta dentro il team di sviluppo. Il producer di Raw Fury invece **sta fuori dal team**. Non fa le *schedule* degli sviluppatori e non decide che il primo mese si lavora al livello uno e il secondo al livello due: come lavorare lo decide il team, che è "padrone e *king*" del proprio progetto. Al publisher interessa solo sapere che il team **lavora in modo efficiente**; se il team pensa di non riuscirci o di aver bisogno di una mano, allora il producer subentra. Normalmente, però, il team ha **mano libera su tutto, tranne le scadenze**: ad esempio, sapere che fra sei mesi c'è la consegna di una demo. Quella è la scadenza che il publisher vede; come lavorare all'interno di quei sei mesi è scelta del team.

0.4.6.2 I *production duo*

In Raw Fury i producer lavorano in **coppie**, i *production duo*. Michele lavora con Loke[20], un producer più senior di lui, con circa **quindici anni** di esperienza, che ha lavorato con DICE, Avalanche e Paradox. La coppia gli è stata presentata come "Mario e Luigi" ("non sono colpevole di questo").

[19] Durante questa spiegazione Gianluca alza la mano; Michele gli chiede di tenere la domanda per un momento e la riprende poco dopo (sezione 0.4.10).

[20] Nella trascrizione il nome è reso come "Loche", "Loke", "Loki" e "all'occhio"; si usa "Loke", grafia plausibile per un nome svedese. La battuta su Mario e Luigi si regge sulle iniziali: M (Michele) e L (Loke).

I due hanno **seniority, responsabilità e campi d'azione diversi**, ma cercano di **sovrapporsi**: Loke è più esperto, ad esempio, di voice acting, Michele dei processi di pubblicazione. La sovrapposizione serve a far sì che, se uno dei due va in vacanza, "il progetto non vada a fuoco e fiamme dopo una settimana".

0.4.6.3 Più progetti, a stadi diversi

La coppia lavora su **più progetti contemporaneamente**, a stadi di sviluppo diversi: in quel momento quattro o cinque, con uscite previste tra il 2023 e il 2025. A seconda delle necessità di ciascun progetto lavora con persone diverse, sia dentro Raw Fury (ad esempio un *brand manager* per il progetto uno, un altro per il progetto due) sia, ovviamente, con developer diversi. L'orizzonte temporale della loro giornata, mostrato in una slide, è più o meno questo:

- un progetto vicino all'uscita, in fase di *release* entro pochi mesi;
- un progetto per cui si sta lavorando alle **demo** per eventi, promozione e trailer;
- progetti in fase iniziale, su cui si sta ancora capendo "quali sono le meccaniche, qual è l'engine, qual è la durata o la *scale* del gioco: deve durare dieci ore o quindici?".

0.4.6.4 Come si dividono i compiti

Loke si occupa delle cose più **manageriali e burocratiche**:

- mantiene i rapporti con la **leadership** di Raw Fury. Se un developer è in ritardo e si fa fatica a ricevere le build, si potrebbe arrivare a pensare "questa cosa non sta funzionando, dovremmo interrompere il contratto": è Loke a occuparsi di questo genere di decisioni a livello manageriale. Il motivo è concreto: se un gioco è previsto per giugno 2024, è previsto che in giugno 2024 entri una certa somma di denaro; se per qualunque motivo non succede, tutta la dirigenza di Raw Fury deve esserne informata;
- fa da **ambasciatore del progetto**, dentro Raw Fury (per motivi come questi) e fuori, ad esempio agli eventi;
- segue il **voice acting**;
- gestisce **contratti e soldi**: se un developer deve assumere un freelancer, Loke fa il colloquio, valuta se è la persona giusta per il team, eventualmente lo assume e controlla che contratti, pagamenti e tutto il resto siano regolari.

Michele si occupa principalmente di quattro aree: **demo ed eventi**, **QA**, **console porting** e **localizzazione**. Le sezioni che seguono le illustrano una per una.

0.4.7 Demo ed eventi

0.4.7.1 Ogni evento ha il suo pubblico

"Andare a un evento" vuol dire cose molto diverse a seconda dell'evento. L'esempio è il confronto tra GDC e Gamescom:

	GDC	**Gamescom**
Pubblico	industria e stampa di settore	pubblico generale
Situazione	la persona viene nel nostro spazio, su un divanetto, con mezz'ora di tempo	coda, zaino in spalla, gente stanca, annoiata, che deve andare in bagno
Gioco adatto	anche un gioco narrativo o meno immediato, da spiegare	un gioco accattivante dal punto di vista visivo
Durata della demo	lunga, con tempo per parlare	cinque, sei, al massimo dieci minuti

A Gamescom un gioco narrativo "non funzionerebbe"; alla GDC, dove si presentano i giochi a *press*, *influencer* e stampa, la persona ha invece tutto il tempo per godersi un gioco che magari non è quello più accattivante a colpo d'occhio. La prima cosa da fare è quindi **capire quale gioco va a quale evento** e come promuoverlo al meglio.

0.4.7.2 Che cosa mettere nella demo

La seconda è decidere **che cosa mettere nella demo**, con lo stesso ragionamento. Michele chiede alla classe: se andate alla Games Week, qual è la vostra resistenza a un gioco? Si fa mezz'ora o un'ora di coda, si gioca cinque, sei, "forse dieci minuti, a essere lunghi", e poi l'esperienza deve finire, perché un'altra persona deve provare il gioco. Una demo di mezz'ora non ha senso in un evento per il pubblico; ha senso invece alla GDC, dove si ha più tempo per parlare con chi si ha di fronte e illustrargli il gioco.

0.4.7.3 Le restrizioni degli eventi

Ci sono poi cose da sapere sulle **restrizioni** di ciascun evento. Alcuni, come Gamescom, hanno un **age rating**: se nel gioco c'è un'uccisione o un contenuto che potrebbe andare contro quel limite, bisogna adottare soluzioni che mettano al sicuro, per evitare di arrivare all'evento e vedersi chiudere lo stand. A Gamescom, per quanto ricorda Michele, il limite è sotto i 18 anni[21]: se si mostra qualcosa che va oltre il PEGI 18, le soluzioni sono due:

- fare in modo che lo **stand sia coperto e chiuso**, accessibile solo a chi ha l'età giusta;
- oppure **eliminare quel contenuto** dalla demo.

0.4.7.4 Valutazione ed esserci

Dopo l'evento si fa una ***post-event evaluation***: si verifica se ciò che si pensava ha funzionato o no. Anche qui ogni gioco è diverso: si stabilisce prima quale obiettivo si vuole raggiungere e si fa poi un *assessment* per capire se lo si è raggiunto.

Infine, la ***public presence***: essere lì, presentare il gioco, mostrare il meglio del gioco e il meglio del publisher, e costruire una buona reputazione per Raw Fury, per il gioco e per tutte le persone coinvolte.

0.4.8 Quality assurance

La seconda area è il **QA**: fare in modo che il gioco sia testato nel modo più efficiente possibile, trovando i bug e creando le situazioni in cui il gioco viene testato. Nel malloppo del service una parte del budget è **riservata al QA**, che può essere affidato a Fury Studios o a partner esterni.

Non tutti i bug possono essere eliminati per il *day one*, "come penso immaginate". Per il giorno dell'uscita tutti i **fatal** e tutti i **must fix** devono essere sistemati; poi si migliora il gioco nel tempo con **patch e update**. Michele se ne occupa per *Dome Keeper*: dopo l'uscita a settembre sono stati rilasciati **nove update**, praticamente uno a settimana. Il lavoro consiste nel tenere sotto controllo:

- quali bug sono presenti e se sono stati sistemati;
- quali feature sono presenti e se sono state testate;

e, a seconda del lavoro fatto e di quello ancora da fare, coordinarsi per rilasciare update e patch quando servono.

L'esempio di *Dome Keeper* vale per un gioco che ha avuto un ottimo successo. Se invece ci si accorge che un gioco non vende, si cerca di **chiudere ciò che c'è**: ciò che è veramente in sospeso si sistema sempre, ma non si penserà a fare DLC per un gioco che non ha avuto successo[22].

[21] Michele stesso lo dice con incertezza ("c'è un age rating credo sotto 18"). La trascrizione rende "stand" come "boot", cioè *booth*.

[22] La trascrizione si interrompe a metà frase ("Ma non andremo a pensare di fare DLC per un gioco") e riprende direttamente con il console porting: una parte del discorso, probabilmente il resto di questa frase e l'inizio della sezione successiva, non è stata registrata.

0.4.9 Console porting

La terza area è fare in modo che il lavoro necessario all'eventuale **porting** su console sia fatto **per tempo** e in modo corretto. Anche qui ogni gioco è diverso:

- un gioco prevede il porting, un altro no;
- un gioco fatto da sviluppatori alla prima esperienza può avere bisogno di **moltissima ottimizzazione**, mentre sviluppatori esperti sanno già strutturare il lavoro in vista del porting.

Il producer fa in modo che il lavoro di ottimizzazione, se serve, sia fatto e organizzato: capisce quando va fatto e quanto tempo richiede, e allinea tutto con l'uscita del gioco oppure con il periodo successivo.

Un esempio di vincolo esterno: con **Game Pass** il gioco deve uscire su **tutte le console nello stesso momento**. Se esce su Steam il 10 giugno, deve uscire il 10 giugno su tutte le console Xbox, **compresa Xbox One**.

0.4.10 Una domanda: fermare un reparto e il debito tecnico

A questo punto Michele recupera la domanda rimasta in sospeso. Gianluca racconta che parlando ha trovato la risposta da solo, ma la pone comunque. Gli capita di dover dire a un reparto, ad esempio alla programmazione, di **mollare quello che sta facendo** su una parte del gioco per dedicarsi subito a una cosa specifica: una consegna o una scadenza[23] inattese, un'emergenza, la presentazione del gioco a un publisher. La difficoltà è l'*expertise* del programmatore, che risponde: "se mi fermo adesso, lascio tutte queste cose a metà e più in là sarà molto difficile riprenderle; stiamo guadagnando tempo adesso, ma lo pagheremo in settimane dopo per risistemare". Oppure si prendono decisioni del tipo "facciamola funzionare adesso e la sistemiamo dopo". Come si gestisce?

La risposta che Gianluca ha trovato ascoltando la definizione di producer è questa: il producer non trova la soluzione, ma la trova **il reparto stesso**. Si comunica al programmatore che c'è questa cosa da fare e che bisogna trovare un modo, e il programmatore trova da solo la soluzione, perché ha l'*expertise* per dire esattamente come farlo.

Michele è perfettamente d'accordo: "questo è il punto alla fine". Lui non ha nessuna idea di programmazione: il programmatore è l'esperto del campo, quindi il producer **comunica il problema e il bisogno**, e sta al programmatore decidere come operare perché le cose restino il più corrette possibile nel tempo.

Gianluca aggiunge una domanda più generale: guadagnare tempo adesso sapendo di perderlo dopo è una cosa da evitare assolutamente, oppure è comune? Michele fa fatica a capire la domanda e gli propone di pensarci e di riformularla più tardi, nella sessione finale[24]. Chiede poi se ci siano altre domande da casa, per coinvolgere anche chi è collegato: da casa e dallo studio è tutto chiaro.

0.4.11 Localizzazione

L'ultima area è la **localizzazione**: capire in quali lingue va tradotto il gioco. È una decisione presa soprattutto da chi si occupa di *sales* e marketing, sulla base dei dati: ad esempio, con la demo su Steam si riesce a capire in quali territori il gioco va meglio.

Ci sono generi che funzionano meglio in certi territori. Andando "per sommi capi":

- gli **strategici** vanno meglio nei territori del Nord e del Centro-Est europeo: Germania, Svezia, Russia, Polonia;
- i giochi **carini**, *cute*, hanno più mercato in **Asia**, in Giappone: si pensi a Pokémon o a *Stardew Valley*.

[23] Nella trascrizione "una headline": quasi certamente "una deadline".

[24] Nella sessione di domande Gianluca pone però un'altra domanda (sezione 0.4.17.5), e quella sul debito tecnico non viene più ripresa: la lezione non dà quindi una risposta esplicita. Per il tema, si veda ciò che Simone e Cristian dicono sui *blocker* risolti in fretta nel capitolo 0.3, sezione 0.3.15.

Si valuta poi se c'è il **budget** per tradurre, perché a volte non c'è: "abbiamo un gioco di centomila parole, vogliamo tradurlo in venti lingue? Mi sa di no, anche no", perché è una cosa veramente costosa. L'obiettivo è che ogni giocatore **nei mercati di punta** abbia la migliore esperienza possibile. È anche il motivo per cui tante volte un gioco non viene tradotto in italiano: **l'Italia non è un grande mercato** per i videogiochi.

Uno studente chiede se Michele abbia un esempio particolare in mente. Lui racconta l'esperienza di 34BigThings: il gioco veniva sviluppato in inglese, tutti i testi venivano scritti prima in inglese e poi mandati a un traduttore[25]. Anche quello è un costo, e dipende dall'azienda: si può fare lo step in più ("siamo italiani, spendiamo questi soldi perché lo vogliamo avere") oppure no ("siamo corti, non ci sta; tanto in Italia vendiamo mille copie, quindi pace").

0.4.11.1 Il riepilogo prima della pausa

Michele chiude la prima parte ricordando l'obiettivo di tutto: che **l'investimento ritorni**, cioè che i soldi spesi per il developer tornino indietro e, potenzialmente, "facciano i developer mega ricchi". Poiché ha "buttato tante informazioni" verso la classe e oltre i 45 minuti "il cervello inizia a colare fuori dalle orecchie", propone una pausa caffè di dieci minuti; le domande vengono tenute per dopo, in modo da coinvolgere anche chi è collegato da casa.

0.4.12 Dopo la pausa: cinque consigli del producer

Al rientro ("Ci siamo, carichi") Michele dedica alcuni minuti a qualche **consiglio per i progetti degli studenti**: consigli che "possono suonare banali", ma a volte le cose banali sono proprio quelle che si trascurano.

0.4.12.1 1. Fate riunioni e parlate dei problemi

Quando si sente che qualcosa non funziona come si pensava, o che qualcosa richiede più tempo del dovuto, **significa che qualcosa non è chiaro**. E allora bisogna parlarsi, e soprattutto **parlare dei problemi**, "perché quella è la parte fondamentale del discorso".

Nello **stand-up** si parla di tre cose:

1. quello che sto facendo oggi;
2. quello che cercherò di fare domani;
3. **quello che mi blocca**.

Con uno dei suoi team Michele fa uno stand-up di **15 minuti ogni giorno**, che serve a valutare l'evoluzione del lavoro e dei problemi, se ci sono. Il consiglio è prendersi quei quindici minuti, veloci, e fare in modo che, se qualcuno ha un problema, **gli altri lo sappiano**. Non è detto che si trovi una soluzione, ma già il fatto che un'altra persona del gruppo sappia che si è bloccati su qualcosa è un ottimo aiuto. Sulla slide c'è la foto di una persona in terapia, "che fitta molto bene"[26].

0.4.12.2 2. Le stime sono sempre sbagliate, ma meglio di niente

Quando si cerca di capire quanto tempo richiederà una feature, **le stime sono sempre sbagliate**: "lo so io, lo sapete voi, se chiedo alla signora al supermercato lo sa anche lei". **Però sono meglio di niente.** Quando ci si siede al tavolo e si cerca di elencare tutte le cose da fare e il tempo che richiedono, si ottiene **una data** in cui tutto sarà fatto: fra sei mesi, fra un anno. Di solito poi ci si accorge che la feature stimata in una settimana ne richiede tre, e succede; ma è meglio avere quella settimana messa in conto che procedere a tentoni, **senza alcuna stima né orizzonte temporale**.

[25] La trascrizione dice "il gioco viene sviluppato in italiano e in inglese scusami quindi tutti i testi sono fatti in inglese prima e vengono mandati a un produttore": la correzione ("scusami") e il seguito fanno pensare che i testi nascessero in inglese e venissero poi tradotti, e che "produttore" stia per "traduttore".

[26] Sul *daily scrum* e sulle riunioni brevi si veda anche quanto detto da Simone nel capitolo 0.3 (sezione 0.3.5), che pure mette in guardia dal risolvere i problemi durante la riunione stessa.

Il consiglio è quindi stimare ogni funzione, ogni feature, ogni singola cosa del progetto: quanto tempo prende, quando inizia, quando finisce. Si avrà così un orizzonte temporale per il progetto[27].

0.4.12.3 3. Mettetevi del margine: il consiglio del salumiere

Nella slide successiva (in parte coperta) c'è la foto di un salumiere, perché bisogna ragionare come lui quando chiede "signora, che faccio, lascio?" e ne mette un po' di più[28]. Quando una feature è difficile, o non si è sicuri di come farla, bisogna **mettersi del margine**. Esempio: per il prototipo bisogna sviluppare il personaggio in prima persona, e la stima è di due settimane; ma, conoscendo il programmatore, si sa che è qualcosa di molto difficile da fare bene. Allora perché non dire quattro? "Così al massimo è pronto in due settimane; però se abbiamo quattro settimane siamo tutti più tranquilli, siamo tutti più sani", soprattutto quando il lavoro va avanti. Meglio finire in anticipo.

La frase che riassume il concetto (e che, con l'aiuto di Diego, Michele recupera dopo qualche tentativo: "queste cose improvvisate sono pessime") è:

> *Better to under-promise and over-deliver than to over-promise and under-deliver.*

cioè: meglio promettere meno e consegnare di più che promettere di più e consegnare di meno. Potrebbe essere, dice, il titolo del ruolo di producer.

0.4.12.4 4. Non aspettate l'ultimo momento per fare le build

"Vedo sorrisi, vedo gente che si sgomita." Succede sempre, e dalle facce degli studenti succede anche a loro: si arriva a una settimana dalla consegna finale, si deve fare la build e **la build non parte**. "Sono contento di aver azzeccato qualche consiglio che va nella giusta direzione."

Il punto è che, quando si consegna, **non si viene valutati sul progetto, ma sulla build consegnata**. Il consiglio:

- capire qual è l'orizzonte temporale (un anno, sei mesi, ...) e stabilire di fare **una build ogni tot tempo**;
- **più ci si avvicina alla scadenza, più spesso si fanno le build**. Con sei mesi a disposizione, nel primo mese si può evitare di fare una build a settimana, ma negli ultimi due mesi la build va fatta **ogni settimana**, ad esempio il venerdì;
- la build del venerdì **si testa**: il solito gruppo di sei persone ci gioca per almeno un'ora e annota i problemi che trova.

Così, almeno la parte del "fare la build" è già fatta, funziona ed è testata quando si arriva alla consegna, e non ci si trova a scoprire all'ultimo che c'è un plugin che rompe tutto o che bisogna aggiornare Visual Studio ("perché lo so, è successo").

0.4.12.5 5. Tagliare feature non è una vergogna

L'ultimo consiglio, "il consiglio finale del producer": **non è una vergogna tagliare feature**. **È meglio tagliare feature che fare crunch.** Se tagliare porta un beneficio al progetto, "allora tagliate, tagliate, tagliate". Nessun gioco vale la propria **sanità mentale** e il proprio tempo. A volte succede comunque di fare crunch quando si sarebbe dovuto tagliare; il consiglio è capire quali parti **non sono essenziali** al gioco e quali si possono aggiungere dopo, per restare nell'orizzonte temporale previsto **senza crunch**.

"Questo è quello che spero vi portiate a casa da queste due ore con me." Chiude la presentazione con una GIF: gli hanno promesso che, se la GIF lascia più imbarazzati della presentazione, allora ha fatto un buon lavoro. Poi si aprono le domande, anche da casa.

[27] Michele ragiona qui in settimane di lavoro, cioè con stime *assolute*. Nel capitolo 0.3 (sezione 0.3.7) Simone sconsiglia invece di stimare in ore e propone stime *relative* con la sequenza di Fibonacci. Le due posizioni non si escludono: entrambi insistono sul fatto che una stima imperfetta, purché fatta e poi corretta con l'esperienza, è meglio di nessuna stima.

[28] Il senso della battuta, resa in modo confuso nella trascrizione ("dovete ragionare con signora che faccio l'ascio"), è quello del margine in più: come il salumiere che taglia un po' più del richiesto, così nelle stime conviene abbondare.

0.4.13 Domande: compiti, bug e decisioni

Il primo a prenotarsi è Marcello[29], collegato da casa.

0.4.13.1 Il producer assegna dei compiti?

La prima domanda di Marcello non è stata registrata: dalla risposta si capisce che chiedeva se il producer assegni compiti ad altri[30]. Michele risponde che non ha un "galoppino" a cui chiedere "ti va di fare questa cosa?": non funziona così. Una situazione in cui assegna un task è, ad esempio, la creazione della **pagina Steam** di un gioco: servono tutte le immagini dello store (la *capsule*, l'immagine in alto e così via), ciascuna con dimensioni e risoluzione precise, e quel task va all'**artista**, perché lui non si occupa di arte. Ma non ha un assistente con le sue stesse mansioni a cui passare il proprio lavoro: di solito, se ci sono delle cose da fare, le fa lui; se è totalmente sommerso, chiede a Loke, il suo "*partner in crime*", se può aiutarlo.

0.4.13.2 Come si decide il budget per i bug?

Marcello chiede poi come si decide l'allocazione del budget per i bug: c'è una lista di importanza, oppure si guarda se un bug è molto frequente, molto pericoloso, molto fastidioso?

In ogni progetto c'è una parte di **bug management**, e i bug sono ordinati per **severità**, dalla più alta alla più bassa ("anche qui, pessimo italiano, scusatemi"), con tutte le sfumature di mezzo. Di solito i livelli sono tre o quattro:

Severità	Che cosa significa
Fatal	pochi bug che vanno sistemati "o morire, o morire provandoci"
Major	bug che sarebbe meglio sistemare il prima possibile
Minor	di solito più numerosi: si sistemano quando si riesce
tutto il resto	bug per cui "se il gioco viene supportato per sei anni, si fixano"

Ma quando si è in quella situazione non si parla di budget: **del budget si parla prima**. Si decide "possiamo spendere tot soldi per la sessione di testing di questo gioco", e poi, con quei soldi, si sistemano i bug che emergono secondo la loro severità. Non è che si hanno dei bug e allora si fa il budget: **viene prima il budget e poi il bug**[31]. Marcello riassume: si decide all'inizio il budget da allocare, e i bug che escono e che si riescono ad aggiustare vengono aggiustati. Esatto.

0.4.13.3 Il budget si ridiscute dopo il successo?

E se il gioco ha successo? **Assolutamente sì.** Se si vendono, ad esempio, 100 000 copie a settimana e ci sono 100 000 persone che stanno giocando, non solo si prova a sistemare tutti i bug, ma si cerca anche di capire che cosa si può cambiare in termini di ***quality of life***. Ci sono cose che, con la scadenza vicina, erano state "sistemate a calci": ora che ci sono persone che comprano il gioco si può rivalutare la situazione e farle decentemente. Si può fare ottimizzazione, oppure una sequenza di **FTUE** (*first time user experience*), perché chi gioca per la prima volta abbia un'esperienza migliore di quella che c'era all'inizio.

0.4.13.4 Chi prende queste decisioni?

Queste decisioni vengono prese da una persona o da un team specifico? Da **non troppe persone**, "una risposta molto democristiana": dieci persone che decidono insieme non ha senso, una sola persona che decide non ha senso, ma **le due o tre persone più importanti del progetto** che decidono insieme sono, secondo Michele, il giusto equilibrio.

[29] Più avanti la trascrizione lo chiama "Marcello di Staso": la grafia del cognome non è verificabile.

[30] La trascrizione riprende direttamente dalla risposta ("Cioè non ho una persona, non ho un galoppino a cui chiedo..."); la formulazione della domanda è una ricostruzione.

[31] Nella trascrizione Michele dice "Viene prima il budget e poi il budget": un evidente lapsus, dato che subito prima ha detto "Non viene prima il bug e poi il budget".

0.4.13.5 Le lingue accorpate

L'ultima domanda di Marcello riguarda la slide sulla localizzazione: alcune lingue vi apparivano accorpate. Marcello vive in Portogallo e ha notato la differenza tra il portoghese europeo e il portoghese brasiliano. Michele chiede quante persone parlino portoghese brasiliano rispetto a quello europeo: circa **dieci a uno**. Per questo si cerca di fare in modo che certi giochi abbiano entrambe le versioni, ma a volte non è possibile, ed è ovvio che chi vive in Portogallo e gioca in portoghese brasiliano noti delle differenze.

0.4.14 Domande: soldi, contratti e potere decisionale

Poi tocca a Filippo, in aula, che ha quattro domande.

0.4.14.1 Qual è il malloppo più grande?

Dei tre malloppi ("non quotarmi su questo"), qual è di solito la parte più grande, pur sapendo che dipende dal tipo e dal genere di gioco? "Dipende. Tante volte la risposta a qualsiasi domanda è sempre *dipende*." L'esempio sono due sviluppatori a San Francisco, dove lo stipendio medio di uno sviluppatore è, a quanto crede Michele, di circa **5 000 dollari al mese**:

Voce	**Importo**
2 sviluppatori × 5 000 $	10 000 $ al mese
× 12 mesi di sviluppo	120 000 $
+ 2 mesi di margine	140 000 $

(Michele sbaglia il conto al primo colpo, "che merda che sono con la matematica", e la classe lo corregge: 140 000[32].) Con quella cifra, osserva, si potrebbero pagare **cinque persone che lavorano a Napoli per due anni**. Quindi dipende. Filippo pensava soprattutto al marketing, ma riconosce che dipende davvero.

0.4.14.2 Che cosa succede se il rapporto si interrompe?

Michele aveva accennato alla possibilità di interrompere il contratto quando le scadenze non vengono rispettate (sezione 0.4.6). Che cosa succede, in concreto, se publisher e studio si separano a metà sviluppo?

Michele non può dirlo per esperienza diretta, perché **non gli è mai successo**. Invita a leggere il contratto, dove c'è la sezione sulla *contract termination* con le clausole e i motivi per cui un contratto può essere interrotto. Ciò che crede di poter dire è che **il publisher acquisisce i diritti di pubblicazione del gioco**: se il publisher paga lo sviluppo del gioco X, ne acquisisce i diritti per pubblicarlo. Se i rapporti si interrompono e lo sviluppatore vuole riavere i diritti di pubblicazione, **deve ricomprarli**. C'è stato un altro gioco di Raw Fury con cui i rapporti si sono interrotti, e Michele è "quasi sicuro" che gli sviluppatori, per arrivare alla pubblicazione con un altro publisher, abbiano dovuto restituire dei soldi a Raw Fury; che fossero soldi del development, del service o di altro, "di nuovo non quotatemi su questo"[33].

0.4.14.3 Soci al 50%

A margine, Michele racconta una cosa che lo ha convinto a candidarsi in Raw Fury: sul sito si legge che publisher e developer sono **soci al 50%**, "nella buona e nella cattiva sorte, come un matrimonio". Sono soci con **pari diritti e pari doveri**: il dovere del developer è sviluppare il gioco fino alla *gold master*[34], quello del publisher è aiutare il developer nel miglior modo possibile.

In concreto, i team sono molto diversi (Michele lavora sia con un team di dodici persone a Barcellona, sia con un ragazzo che lavora da solo a New York), ma il rapporto parte sempre da:

[32] Nella trascrizione la cifra è indicata a un certo punto come "120.000 euro dollari": la valuta dell'esempio è il dollaro.

[33] La trascrizione di questo passaggio è frammentaria ("potrebbero o non potrebbero essere dei soldi del development, del service, del all'interno del team di sviluppo"); anche il nome del gioco non viene detto.

[34] Nella trascrizione "la goalmaster master per PC": con ogni probabilità la *gold master* della versione PC, cioè la versione definitiva consegnata per la pubblicazione.

- una **call a settimana** ("come va, come non va"), l'equivalente dello stand-up di cui ha parlato prima, ma con cadenza settimanale;

- quando c'è bisogno di discutere, **incontri di persona** a Stoccolma, dove ha sede Raw Fury: due giorni insieme, birre, cene, "ci ubriachiamo insieme e parliamo di quello che c'è bisogno di parlare". "Cerchiamo di essere brave persone a tutto tondo."

Anche gli incontri vanno però pesati sui costi. Se il team di dodici persone di Barcellona ha bisogno di un *reset* del progetto, portarle tutte a Stoccolma (e far arrivare Michele da Torino) significa pagare dodici voli e dodici hotel, più il suo. Allora si cerca il modo migliore: magari tre persone fondamentali del team vanno a Stoccolma, non ci si vede tutti, ma si **risparmiano soldi che restano nel service**.

0.4.14.4 Quanto può decidere il publisher sulle feature?

L'ultima domanda di Filippo: quanto potere decisionale ha il publisher sulle feature da tenere nel gioco finale, soprattutto se il gioco è ancora in pre-produzione e può svilupparsi in modi diversi? **Il developer ha il 100% del potere decisionale.** Il publisher dà **suggerimenti**, in modo che poi si possa dire: "ricordi che sei mesi fa ti avevo detto X? È successo X: la prossima volta cerchiamo di lavorare meglio".

L'esempio è un gioco a cui Michele sta lavorando, che prevede un **login con password** all'avvio. Lui ha fortemente consigliato allo sviluppatore di eliminarlo, perché il 90% dei giocatori che vede un'interfaccia del genere appena avviato il gioco chiede il rimborso[35]. Lo sviluppatore ha risposto che voleva farlo, perché era la sua visione artistica. Il gioco è stato mandato in *review* a Steam ed è stato **bocciato**: la persona di Valve che prova e autorizza i giochi per il rilascio non riusciva a capire come avviarlo e giocarlo. Ma, di nuovo, il gioco è dello sviluppatore, e la visione artistica è sua: al publisher spetta essere lì, aiutare e dare consigli.

0.4.15 Domande: progetti che si fermano, VR e DLC

Segue una serie di domande da un partecipante in aula[36].

0.4.15.1 Progetti che non vale la pena continuare

È mai capitato di trovare progetti che non valeva la pena continuare? Michele ha detto che a lui non è mai successo di chiudere un progetto a metà, ma **può succedere**, per diversi motivi, e quasi sempre dipende dallo sviluppatore: quando Raw Fury firma, si impegna per due anni a portare a termine il lavoro. Può succedere, ad esempio, che lo sviluppatore dica: "ragazzi, ho un problema, non voglio più lavorare a questo gioco, sono completamente fuso, voglio andare in terapia, non voglio più vederlo". È successo. "Ok, grazie lo stesso": si chiude il discorso e ci si separa.

0.4.15.2 E se il gioco cambia strada?

E se il gioco presentato è il gioco A, ma durante lo sviluppo la visione cambia e A diventa pian piano B? Al momento della firma c'è un ***vision statement***: un documento che dice "il gioco è questo, questo è ciò che finanziamo". Non può succedere che un FPS finanziato diventi un gioco di corse.

Prima della firma si fanno anche altre verifiche, che rientrano nella ***due diligence***, termine che racchiude "tutto ciò che riguarda l'essere accorti". Ad esempio, ci si assicura di avere un'idea dello **stato tecnico** del progetto: allo studio di Raw Fury in Croazia viene dato accesso al codice, per controllare se gli sviluppatori hanno l'esperienza che dicono di avere o se sono alla prima esperienza ("che è normale").

[35] Nella trascrizione "fa il R-Funk": quasi certamente "fa il *refund*", cioè chiede il rimborso su Steam. La percentuale è quella detta dal relatore, non un dato verificato.

[36] La trascrizione introduce questa parte con "A questo punto viene una domanda a me" e non dice chi parla; dalle domande successive sembra trattarsi della stessa persona.

0.4.15.3 I giochi in VR

Che cosa si pensa quando arriva la proposta di un gioco in **VR**? Raw Fury ha giochi VR in catalogo, ma sono tutti giochi **pubblicati in precedenza** in versione non VR e poi adattati. Storicamente Raw Fury **non è un publisher affine al VR**; altri potrebbero esserlo di più. Il modo in cui ci ha lavorato è, "e qui rientra nell'ambito della supposizione da parte mia", questo: il publisher detiene i diritti di un gioco del proprio portfolio e si chiede come farlo rendere di più, decidendo di spendere una certa cifra per farne la versione VR. Non crede che Raw Fury debba essere il primo partner a cui pensare per un gioco VR, "però provaci".

0.4.15.4 Ridiscutere il budget dopo l'uscita

Se, dopo l'uscita, si decide di mettere nuovo budget per migliorare il gioco o per altri contenuti, la cosa deve essere prevista nel contratto iniziale o si scrive un nuovo contratto? **Si fa un aggiornamento.** Il futuro non lo può prevedere nessuno: sarebbe stupido scrivere in un contratto di due anni prima "questo gioco non avrà DLC" ("lasciati le porte aperte, innanzitutto"). Se succede, si scrive un nuovo contratto in cui si dice, ad esempio, che per il 2024 il budget assegnato al progetto è di 100 000 euro, con cui si faranno X, Y e Z.

0.4.15.5 Conviene presentarsi con i piani per il dopo?

E conviene, al momento del *pitch*, mostrare che ci sono già piani per contenuti successivi[37]? Secondo Michele **non troppo**. Quando un publisher finanzia un gioco, il focus è **il gioco**, non ciò che viene dopo: pensare al dopo significa dare per scontato che il gioco andrà bene e tutta una serie di altre cose. Il discorso diventa persino pericoloso, perché porta a chiedersi "perché pensi a quello che succederà fra tre anni quando non sai quello che succederà fra un anno?". Il consiglio: **mettere il focus sull'obiettivo più vicino**, cioè il gioco, la sua uscita, come lo si svilupperà e perché avrà successo. Del DLC si parla dopo, quanto si vuole, ma non ha senso "mettere il carro davanti ai buoi".

0.4.16 Domande: come si sceglie un progetto

Un altro partecipante ("sono tremendo, scusate") chiede come funziona il **processo di selezione**: che cosa si aspetta Raw Fury da un progetto per capire se potrebbe essere un buon partner? "Domanda super difficile, c'è un universo dietro questa domanda." La risposta segue gli stadi della valutazione.

0.4.16.1 Primo: quando esce

La primissima cosa che si guarda è **quando si pensa di far uscire il gioco**. Se la risposta è il 2024, la risposta di Raw Fury è no, perché i giochi che pubblicherà nel 2024 li conosce già. Un publisher, di solito, **ragiona con due anni di anticipo**: per il 2023 e il 2024 i giochi a cui si sta lavorando sono già decisi.

Questo dipende anche dal fatto che Raw Fury **lavora dall'inizio del progetto**. Altri publisher lavorano a sei mesi dalla fine: a loro si può dire "ho un gioco quasi finito, gli manca questa feature, fra sei mesi sono pronto", e va benissimo; ma Raw Fury non è quel publisher. Lavora da quando c'è il primo prototipo e porta avanti la relazione per un anno, un anno e mezzo, due anni, **non oltre due anni**, perché poi diventa troppo tempo ("però in due anni puoi fare parecchie cose").

0.4.16.2 Secondo: il gioco

Non c'è un genere che venga escluso a priori: i giochi **si provano tutti**. Forse l'unico genere mai affrontato da Raw Fury è il **racing**; per il resto ci sono tanti generi e tante meccaniche diverse. La valutazione procede per stadi:

1. un gruppo di persone **prova i prototipi** e scrive le proprie impressioni;

[37] La domanda è quasi incomprensibile nella trascrizione ("di mettere a per fuori un numero che ci sono già dei piani per fare un'inversità"): la ricostruzione si basa sulla risposta.

2. se il gioco supera questo primo stadio ("piace a tutte e cinque le persone che provano i giochi all'inizio"), lo si fa provare a **dieci persone** per vedere se l'impressione si conferma;

3. nello stadio successivo si comincia a ragionare sul **mercato**: che pubblico ha questo gioco? È super di nicchia o appartiene a un genere con un pubblico ampio?

4. si valuta il **developer**: quanta esperienza ha? Alle prime armi va bene, con esperienza è ancora meglio. E poi: è una buona persona con cui lavorare, oppure "un pezzo di merda" con cui si odierebbe lavorare per due anni?

0.4.16.3 La *due diligence* sulle persone

Anche quest'ultimo aspetto fa parte della *due diligence*: si va a vedere **che tipo di persona** è quella con cui si andrà a lavorare, banalmente spulciando i social, Twitter, per capire se è una persona **divisiva**. Generalmente si cerca di evitare questo tipo di persone, "come è normale": si vuole lavorare con persone con cui ci si trova d'accordo e con cui si ha piacere di lavorare. Le verifiche prima della firma sono quindi tre:

Due diligence	Che cosa verifica
finanziaria	costi del team, mercato, ritorno atteso
tecnica	stato del codice ed esperienza reale del team (il controllo dello studio in Croazia)
personale	che persona è lo sviluppatore, anche attraverso i social

Se tutto va nella direzione giusta, se il gioco piace a molte persone del publisher e se c'è un mercato, allora si invita il team a Stoccolma "e firmiamo il firmabile"[38].

0.4.17 Domande: finanziamento, calendario e lingue

0.4.17.1 Da dove vengono i soldi del publisher

Michele chiede se ci siano altre domande da casa e invita Diego a farne una; Diego dà la precedenza a Luca. Luca chiede come si finanzia Raw Fury: chiede investimenti a qualcuno, oppure reinveste ciò che guadagna con i giochi pubblicati? La risposta è "molto parziale": Raw Fury è **posseduta da un gruppo di investimento**, Altor[39]. Oltre ai soldi di Altor, i giochi devono naturalmente **generare profitto**. "Diciamo che abbiamo le spalle coperte." "I piccioli", commenta.

0.4.17.2 Se il calendario è pieno, quando ripresentarsi?

Diego chiede: se un progetto viene inviato quando la *line-up* è già completa, quando e come ci si può ripresentare? È difficile, risponde Michele, perché da esterni **non si può sapere** quando è il momento giusto: lo sta dicendo ora solo perché è lui, ma il fatto che Raw Fury sia pieno fino al 2024 non è un'informazione pubblica. A volte capita un buco nel calendario in cui entra proprio un progetto piaciuto a un po' di persone, ma il più delle volte non succede.

L'esempio viene dal **Tech & Tonic**, un evento di Torino in cui si tengono lezioni brevi, di circa quindici minuti, "e poi andiamo a bere". Lì lo hanno contattato i ragazzi di *Universe for Sale*[40], che erano stati lasciati dal loro publisher, non per altri motivi ma perché il publisher aveva chiuso. Gli hanno chiesto se potessero ripresentare il gioco a Raw Fury: mancavano sei mesi, forse un anno, uscita forse nel 2024. Lui ha detto loro ciò che dice alla classe: il 2024 è pieno. Allora: potevano aspettare e ripresentarlo nel 2025? La sua risposta: "avete un progetto che l'anno prossimo è pronto, e lo tenete in supporto vitale per due anni con la speranza di lavorare con Raw Fury?". Il suggerimento è stato: **pubblicate**, fate esperienza, magari

[38] I contenuti della colonna "Che cosa verifica" sono ricavati da quanto detto nel corso della lezione (costi del team e mercato, controllo del codice, social): Michele elenca le tre *due diligence* senza descriverle una per una.

[39] Così nella trascrizione ("si chiama Altor"); si tratta con ogni probabilità del fondo di investimento svedese Altor. Il dettaglio dell'assetto societario non è stato verificato.

[40] Così nella trascrizione, che parla di "ragazzi che hanno fatto un progetto" a Torino; il titolo è riportato come compare.

guadagnate due soldi, e poi **ripresentatevi con il progetto successivo**, con un prototipo per il 2025, "perché avrete un progetto in più sulle spalle e sarete più forti".

0.4.17.3 Il publisher cerca giochi di un certo tipo?

Un'altra domanda, sempre di Diego[41]: visto il discorso della partnership al 50%, è mai capitato che Raw Fury suggerisse agli sviluppatori, o cercasse, un tipo specifico di prodotto? Michele non ha mai sentito di situazioni simili: non gli risulta che si dica "cerchiamo di pubblicare uno strategico". "Secondo me non facciamo così."

0.4.17.4 Un gioco legato a una lingua

Marcello torna con una domanda: se un gioco ha un'identità molto legata a una lingua e, per budget, si decide di tradurlo in poche lingue che non la comprendono, vale la pena rifare tutto da capo nell'altra lingua, magari con il progetto già avanzato e molto materiale pronto, o conviene mantenere la lingua originale?

"Guarda la mia faccia: rifare da capo?" Questo è un discorso che va fatto **quando si pensa al mercato del prodotto**, cioè al momento della valutazione finanziaria e di marketing: se un prodotto parla più forte a un certo tipo di pubblico, significa che è ristretto a un pubblico ristretto.

Marcello precisa che pensa all'**italianità**, che vende sia in Italia sia all'estero; ma all'estero si nota che vende un'italianità "esterizzata": italiana, sì, ma con traduzioni curate, in cui solo certi termini restano veramente italiani. Michele concorda: è il modo per rendere il prodotto **accessibile a più persone possibile**. "Posso anche decidere di fare un gioco in siciliano, però lo capiscono solo i siciliani. Se faccio il gioco in italiano, lo capiscono tutti gli italiani. Se faccio un gioco in italiano inglesizzato, internazionalizzato, parla a più persone."

0.4.17.5 Un'offerta più bassa del budget

Gianluca porta un caso concreto del suo team: parlando con alcuni publisher, hanno ricevuto offerte **più basse del budget** previsto, e non sanno bene come muoversi, perché non possono abbassare troppo il budget. La loro idea era proporre al publisher qualcosa come: "questi soldi non bastano a fare tutto; ci finanzi fino a un certo punto e poi si decide se continuare o aggiornare il contratto". Non l'hanno ancora proposto perché non sanno se abbia senso per un publisher[42].

La risposta di Michele, "così di getto": se si chiede a un publisher un budget per due anni di lavoro e il publisher offre i soldi per un anno e mezzo, allora bisogna dirgli **"ok, invece di questo gioco ti consegno questo gioco"**, dove il secondo è un gioco **con meno contenuti** del primo. In altre parole: se il publisher propone meno soldi di quelli che servono, **bisogna avvisarlo che il gioco sarà meno di quello che gli è stato presentato**. Michele precisa che non sta parlando di *early access*, su cui non è esperto: Raw Fury non fa giochi in early access, "proprio li evitiamo". Gianluca conferma che proporranno quanto suggerito.

0.4.17.6 Perché niente *early access*?

Un partecipante in aula chiede allora perché Raw Fury eviti l'early access. Il motivo, risponde Michele, andrebbe chiesto a chi si occupa di *sales*: semplicemente **non sono esperti di quel tipo di giochi**. È lo stesso discorso del VR: con metodologie di lavoro che non si conoscono ci si va più cauti, o non ci si lavora per nulla.

[41] L'attribuzione è probabile ma non certa: la trascrizione introduce la domanda con "A questo punto l'opera un'altra domanda", subito dopo la risposta a Diego, e Michele risponde ringraziando prima di passare a Marcello.

[42] La formulazione dell'idea è confusa nella trascrizione ("offrirgli di dire ok questi soldi non sono quanti ci servono per farlo tutto e invece che appoggi se continuare a svilupparlo o aggiornare il contratto come dicevi prima"): la ricostruzione è approssimativa. In seguito Gianluca precisa che più che offrire meno soldi, i publisher dicevano di essere molto interessati al gioco ma di non poter arrivare oltre una certa cifra.

0.4.18 Chiusura

Non essendoci altre domande, Michele conclude "in gloria", circa due ore dopo l'inizio. Ringrazia chi è rimasto online e Diego per l'occasione ("quando mi rivuoi fammi sapere"), e invita tutti al **Tech & Tonic** di Torino, che si tiene di solito una volta al mese: lì lo si trova ad assistere alle conferenze, "ma più che altro a bere". Alla battuta sulla professionalità risponde che quella l'ha "lasciata fuori dalla porta"; qualcuno commenta che anche quello è *networking*. Ricorda che per qualunque domanda lo si può contattare ai recapiti mostrati in sovrimpressione durante la lezione.

A registrazione quasi chiusa, uno studente torna sulla *due diligence* personale: che cosa si fa se la persona con cui si dovrebbe lavorare non ha alcun social network, a parte LinkedIn? La si **invita a Stoccolma**: si passano due giorni insieme, si beve una birra e si cerca di capire che persona è, conoscendola di persona. "Le birre sono fondamentali."

Punti chiave del capitolo

- Un **publisher** è come una casa editrice: cerca un gioco da pubblicare, **lo finanzia e lo promuove**, e riceve in cambio una quota dei ricavi. Cerca giochi in modo attivo (comunità di sviluppatori nel mondo) e passivo (*pitch* via email), e li gioca tutti.
- I soldi di un accordo si dividono in **tre malloppi**: **development** (stipendi del team per i mesi di sviluppo, più un paio di mesi di margine per il ritardo degli incassi), **service** (QA, localizzazione, voice acting, porting, freelancer, grafiche per gli store) e **marketing** (eventi, trailer, PR, community, streamer). La loro dimensione dipende dal tipo di gioco e da dove vive il team.
- I partner di piattaforma (Game Pass, PlayStation Indies, Nintendo) portano un **bonus immediato**, ma impongono vincoli: con Game Pass il gioco esce lo stesso giorno su tutte le Xbox, Xbox One compresa.
- L'obiettivo è il *magic*, una buona esperienza per il developer, il **recoup** dell'investimento e un guadagno per entrambi. Il contratto di Raw Fury è pubblico: leggerlo è il modo migliore per capire il flusso dei soldi.
- Il **producer** è il *master* di **tempi e budget**, non del gioco: la *vision* è del lead designer. Il suo compito è far uscire il gioco quando previsto, **senza crunch** e nel budget.
- Il producer è **l'anello fra due catene**: un **comunicatore**, un *problem spotter* (individua i problemi futuri e li fa risolvere, non li risolve per forza lui; la capacità cresce con l'esperienza) e chi si occupa di **tutto il resto** che aumenta l'efficienza del team, pizza compresa. Requisito di base: **essere una persona a modo**.
- Davanti a un'emergenza tecnica il producer **comunica il problema e il bisogno**; **come** risolverlo lo decide l'esperto del reparto.
- In Raw Fury il producer è **esterno al team**: il developer è padrone del proprio progetto, il publisher guarda le scadenze. I producer lavorano **in coppia**, con competenze diverse ma sovrapposte, su più progetti a stadi diversi.
- **Eventi**: ogni evento ha il suo pubblico (GDC per l'industria, Gamescom per il pubblico), quindi giochi e demo diversi (demo di pochi minuti per il pubblico), restrizioni da rispettare (age rating) e una valutazione dopo l'evento.
- **QA**: il budget si decide **prima**, poi si sistemano i bug per severità (*fatal*, *major*, *minor*, resto). Al day one i *fatal* e i *must fix* devono essere chiusi; se il gioco ha successo si investe in patch, *quality of life* e FTUE.
- **Localizzazione**: le lingue si scelgono sui **mercati** (dati della demo, generi che funzionano in certi territori) e sul budget. L'Italia non è un grande mercato; un gioco **internazionalizzato** parla a più persone.

- Cinque consigli per i progetti: **parlate dei problemi** (stand-up breve: cosa faccio oggi, cosa farò domani, cosa mi blocca); **stimate sempre**, anche se le stime sono sbagliate; **mettete margine** (*under-promise, over-deliver*); **fate build regolari** e sempre più frequenti verso la scadenza, perché si viene valutati sulla build; **tagliate feature piuttosto che fare crunch**.

- Il **developer decide al 100%** sul proprio gioco; il publisher consiglia. Il *vision statement* firmato fissa che cosa si finanzia.

- Per essere scelti conta prima di tutto il **calendario**: un publisher ragiona con **due anni di anticipo**. Poi il gioco (provato da sempre più persone), il mercato e il developer, con una **due diligence** finanziaria, tecnica e personale.

- Se il calendario di un publisher è pieno, meglio **pubblicare**, fare esperienza e ripresentarsi con il progetto successivo. Se l'offerta è inferiore al budget, bisogna dire chiaramente che il gioco **sarà più piccolo**. Nel pitch, il focus è il gioco, non i DLC.

Chapter 0.5

Il game producer: un mestiere da costruirsi

Questa quinta lezione zero è dedicata interamente a una figura: il **game producer**. Non è una lezione "da manuale": il relatore avverte fin dall'inizio che quello del producer è un profilo **articolato e ancora in divenire**, per il quale non esiste una guida che dica che cosa fare "regola per regola, giorno per giorno". È un ruolo che **ci si costruisce con l'esperienza**, che copre mansioni e aspetti molto diversi all'interno di un team, e per il quale bisogna capire presto se si è portati e se piace, perché è "sicuramente un ruolo abbastanza stressante".

A parlarne è Walter[1], che si descrive come **game producer e game director** insieme. Il seminario si svolge **interamente online**, in videochiamata, con il relatore che condivide le proprie slide e i partecipanti che intervengono a voce o scrivendo in chat; è organizzato dall'accademia[2] e aperto anche a qualche partecipante esterno. Occupa un intero pomeriggio, con quattro brevi pause, e si chiude "all'ora di cena"[3]. È annunciata come la **prima di due parti**: alcuni partecipanti ritroveranno il relatore il giovedì successivo.

Tre avvertenze di lettura. La prima: Walter dichiara più volte che le slide sono solo una **traccia** e che molto di ciò che dice è **opinione personale**, frutto della sua esperienza, non necessariamente "quella giusta"; il capitolo conserva queste cautele. La seconda: una parte consistente della lezione rilegge **Agile e Scrum** dal punto di vista del producer, spesso in modo critico; le definizioni della metodologia sono riportate così come il relatore le presenta, con qualche nota dove la trascrizione le rende in modo impreciso. La terza: la trascrizione è automatica, storpia nomi propri e termini inglesi, perde pezzi di frase in vari punti (soprattutto quando il relatore legge le slide) e a volte mescola le voci; dove la ricostruzione non è sicura lo si segnala in nota. I riferimenti temporali vanno letti rispetto al momento della lezione, che si colloca nella **seconda metà di settembre 2022**[4].

Il percorso del capitolo segue quello della lezione:

1. l'avvio del collegamento e la presentazione del relatore (sezioni 0.5.1 e 0.5.2);
2. i **ruoli** in un team di sviluppo e quelli indispensabili in uno studio appena nato (sezione 0.5.3);
3. i **miti** sul game producer: un lavoro meraviglioso? un creatore di videogiochi? un game designer? (sezione 0.5.4);

[1] Nella trascrizione il relatore non si presenta mai per nome: è chi organizza il collegamento ad accoglierlo all'inizio ("Ciao Walter, benvenuto") e a chiamarlo per nome alla fine ("ci penso io Walter, non ti preoccupare"). Il cognome non viene mai pronunciato; nel capitolo lo si chiama semplicemente Walter.

[2] Nella trascrizione l'accademia è resa in molti modi ("AIV", "AIB", "AIW", "Hive", "Ivy", "iV"): si tratta sempre di AIV, l'Accademia Italiana Videogiochi. La persona che gestisce il collegamento per l'accademia non viene mai chiamata per nome.

[3] La durata esatta non è chiara. A metà lezione il relatore dice scherzando che "è da quattro ore" che spiega che cosa fa il game producer; alla fine parla di contenuti "concentrati così in sei ore". Sono probabilmente iperboli, ma in ogni caso il seminario è stato molto lungo.

[4] Il relatore dice esplicitamente "siamo nel 2022" e "oggi, nel 2022"; parla del ritorno di Monkey Island come di un fatto attuale (*Return to Monkey Island* è uscito il 19 settembre 2022); uno studente prepara una demo per l'11 ottobre; la seconda parte del seminario è fissata per "il 29, quindi giovedì", e il 29 settembre 2022 era appunto un giovedì. La datazione è una deduzione, non un'informazione esplicita.

4. l'**evoluzione storica** del ruolo dagli anni Ottanta a oggi (sezione 0.5.5);
5. altri miti da sfatare: la paura, l'inglese, il mobile (sezione 0.5.6);
6. che cosa cercano gli **investitori** e il mercato, con le prime domande dalla chat (sezioni da 0.5.7 a 0.5.10);
7. le **testimonianze** di tre studenti che fanno già, in qualche forma, i producer (sezione 0.5.11);
8. il producer come **product manager**, **project manager** e **client manager** (sezioni da 0.5.12 a 0.5.14);
9. **Agile e Scrum** riletti dal producer: product owner, product manager, team, Kanban, framework Scrum, meeting e valori (sezioni da 0.5.15 a 0.5.17.5);
10. alcuni **producer famosi** (sezione 0.5.18);
11. la **visione**, gli strumenti personali e il rapporto con publisher e developer (sezioni da 0.5.19 a 0.5.21);
12. l'**attitudine** del producer, il B2B e il B2C, la **gamification** (sezioni da 0.5.22 a 0.5.24);
13. il **panorama internazionale** e il **"problema italiano"** (sezioni 0.5.25 e 0.5.26);
14. gli ultimi consigli e la chiusura (sezioni 0.5.27 e 0.5.28).

0.5.1 L'avvio del collegamento

La registrazione comincia mentre il relatore cerca "l'angolazione giusta della telecamera" e prova la condivisione dello schermo, che funziona al primo colpo ("è una rarità questa"). Chi organizza per l'accademia conferma di sentirlo e vedere le slide e propone di attendere qualche minuto chi deve ancora collegarsi: all'inizio mancano cinque o sei persone, poi "giusto una persona". Vengono salutati i primi partecipanti online (Valerio, Luca, Diana, Michele[5]), ai quali si ricorda che potranno intervenire a voce o, in alternativa, scrivere le domande in chat.

L'organizzatore ringrazia il relatore per il tempo e la competenza che mette a disposizione. Walter ricambia: ha conosciuto l'accademia mesi prima, la considera "una struttura abbastanza unica nel panorama italiano" e spera di collaborare "in modo continuativo". È curioso di sapere l'età degli studenti e se qualcuno lavora già nel settore. Gli viene detto che i partecipanti sono **otto**: **cinque studenti dell'accademia**, del primo e del secondo anno, e **tre esterni**. Il relatore si scusa ironicamente di non essere "particolarmente fotogenico" e, in attesa dell'ultimo partecipante, inizia una presentazione preliminare, "anche perché online è sempre più complicato rispetto a stare in una classe di persona".

0.5.2 Chi è il relatore

Walter è al momento sia **game producer** sia **game director**. Lavora per **Commodore**, dove si occupa della **pubblicazione di videogiochi**, e ha "svariati anni di esperienza" nel gaming. Il gioco più noto su cui ha lavorato in questo ruolo è **Simon the Sorcerer**[6], di cui "si parla abbastanza" perché dovrebbe uscire ad aprile dell'anno successivo, in un periodo in cui è tornato anche Monkey Island: "le due cose vanno abbastanza a braccetto".

La sua esperienza però è "a 360 gradi": viene in origine dall'**ingegneria informatica** e ha lavorato nelle telecomunicazioni e nell'IT, con aziende come Motorola ed Ericsson. Più avanti nella lezione aggiungerà due dettagli: possiede la **certificazione PMI** da *project manager* e **produce nuovi giochi per la vecchia console Intellivision**, nell'ambito del *retrogaming*. Insegna anche il metodo Agile.

[5] I nomi sono quelli che si leggono nella trascrizione; uno dei partecipanti risulta collegato con un audio "un po' a scatti".

[6] Il relatore parla di un nuovo *Simon the Sorcerer*, "previsto per il lancio ad aprile dell'anno prossimo", cioè, alla data della lezione, per l'aprile 2023. Non specifica il titolo esatto né il suo ruolo preciso nel progetto.

0.5.2.1 Il ruolo e il taglio del seminario

Il tema dell'incontro è il ruolo del game producer, che il relatore inquadra subito così:

- è un profilo **articolato e in divenire**, senza una guida precisa che lo definisca in modo semplice;
- va **costruito con l'esperienza** e copre diverse mansioni e diversi aspetti all'interno di un team;
- è **stressante**: bisogna capire se si è portati e se piace;
- è **aperto a tutti**, indipendentemente dal ruolo già ricoperto nei videogiochi; è avvantaggiato chi ha avuto ruoli di **project management** o di **business**, mentre è "un po' più complicato" per chi ha un retroterra principalmente tecnico, come un programmatore.

Il relatore vorrebbe sapere se qualcuno ha già esperienze lavorative, per confrontarle con quelle di chi fa davvero questo mestiere e capire meglio le prospettive (più tardi lo chiederà esplicitamente: si veda la sezione 0.5.11). Precisa anche il metodo: seguirà le slide che ha preparato, ma **non è un workshop nozionistico** in cui si leggono slide da imparare. Le slide sono una **traccia**, servono a dare spunti per approfondire, raccontare esperienze e soprattutto rispondere alle domande.

0.5.3 I ruoli in un team di sviluppo

Il primo argomento è un *excursus* sulla quantità e sulla varietà dei ruoli presenti in un team. Serve a capire un punto: il game producer **sarà sempre presente** in un team di sviluppo, ma può essere una persona che ricopre o ha ricoperto **anche altri ruoli**.

La slide mostra un lunghissimo elenco di ruoli, che il relatore dice di non aver scritto lui ma di aver preso da internet[7]. I ruoli possono essere "veramente tantissimi" perché i team tendono con il tempo a **ingrandirsi**; quanto, dipende dal budget, dalle possibilità dell'azienda, da come è strutturata e dal fatto che produca i giochi internamente o collabori con altri studi, magari in *co-development*.

Il primo concetto da comprendere è che gli studi possono avere **le dimensioni più disparate**:

- il relatore ha lavorato anche con studi di **due sole persone**, che sono riusciti a realizzare piccoli prodotti, magari per cellulare. I team molto piccoli sono "più avventurosi", spinti dalla passione, e difficilmente trovano poi uno sbocco;
- questo però non significa che uno studio debba essere grande: le grandi *major* (Ubisoft, Electronic Arts, "quelle che volete voi") arrivano a studi che superano le **50 persone**[8], e crescere "a volte può essere anche controproducente".

Per questo uno studio, **soprattutto appena lanciato**, dovrebbe essere molto attento nella creazione del team e nella scelta dei profili: alcuni sono **indispensabili**, altri porteranno benefici ma possono essere introdotti **in un secondo momento**, oppure non far parte dello studio ed essere **consulenti esterni**. Dall'ampia lista, il relatore indica che cosa conta di più per la sua esperienza:

Programmatori. Ne serviranno sempre almeno un paio.

Game design. In uno studio piccolo appena nato tende a coincidere con il programmatore, "perché in fondo lui sa che cosa deve realizzare nel gioco" e sa certamente creare i livelli. Ma è un lavoro importante di per sé: sarebbe necessario individuare la persona più adatta e **fargli fare solo quello**.

Arte. La grafica del gioco, ma anche i disegni che accompagnano il lancio, quindi il materiale di marketing. È spesso affidata a una persona esterna, ma per esperienza del relatore è **molto importante averla internamente**: è un budget da considerare.

Animazione. Va "più o meno di pari passo" con l'arte.

[7] L'elenco della slide non viene letto e non è riportato nella trascrizione: si sa solo che è molto ampio.

[8] Così nella trascrizione. La cifra è probabilmente da intendersi come ordine di grandezza minimo: gli studi delle grandi major superano spesso di molto le cinquanta persone.

Audio. È una risorsa che di solito **si acquista esternamente**.

Si potrebbe passare "tutta la giornata" ad approfondire che cosa fanno tutte queste persone[9]; il seminario invece si concentra sulla *game production*.

0.5.3.1 Che cosa vedremo

Prima di entrare nel merito, Walter elenca ciò che si propone di fare:

- dare spunti e idee basilari su che cosa sia un game producer, di cui presume che tutti abbiano già un'idea (eventuali commenti vanno scritti in chat per confrontarli);
- vedere "ad ampio respiro" i **metodi** che un producer usa per portare avanti il suo lavoro e gli **strumenti** tipici, magari gli stessi degli altri membri del team: negli ultimi tre o quattro anni si è molto diffusa la tendenza ad adottare **certificazioni Agile**;
- fare esempi, possibilmente tratti da esperienze reali;
- soprattutto, lasciare "una serie di piccole cose" che restino nella memoria e siano riutilizzabili come **best practice** nella vita lavorativa. In questo senso è molto importante vedere anche **gli errori commessi**.

Dopo un controllo dell'audio ("si sente assolutamente"), il relatore invita a scrivere le domande in chat, così da riprenderle.

0.5.4 Miti e aspettative sul game producer

0.5.4.1 "È un lavoro meraviglioso"

La prima slide contiene una domanda che il relatore ama fare al pubblico: perché una persona vuole fare il game producer? Intorno al ruolo ci sono "un po' di miti" e aspettative non sempre legate alla realtà. Per provocazione la slide dice: **è un lavoro meraviglioso**.

Molti si aspettano che il producer sia la persona **in prima linea**: quella che va agli *expo*, rilascia le interviste, è spesso in contatto diretto con il top management e quindi, in un certo senso, cura l'immagine di tutto lo studio o di uno specifico gioco. Il ruolo appare:

- **dinamico**, e questo "è senz'altro vero";
- **molto creativo**, e questo dipende dai contesti e dalle dimensioni del team, cioè da quanta **carta bianca** viene data: più il team cresce, meno carta bianca si ha, e più aumentano eventualmente i producer al suo interno, che non sono necessariamente uno solo.

Sembra quindi un lavoro divertente e meraviglioso, lo sbocco a cui chiunque lavori nell'industria aspira più di altri, "l'immagine del bello" di questo settore. Non è necessariamente così: chi si avvicina al mestiere, soprattutto da studente, deve essere **cauto** e approfondire.

0.5.4.2 Il game producer crea i videogiochi?

"Diciamo di sì, però non è il suo ruolo all'interno dello studio." Il producer:

- è certamente coinvolto nel momento creativo, nello studio del concetto del gioco, e ha voce in capitolo, soprattutto se è una risorsa esperta: può indicare che cosa cambiare nella caratterizzazione dei personaggi e suggerire al team scelte per indirizzare meglio il gioco verso la **nicchia di mercato** a cui si rivolge;

[9] In questo punto la trascrizione salta un pezzo: dopo l'audio si passa direttamente a "tutta la giornata solo per approfondire cosa fanno tutte queste persone".

- in questo senso "crea" il gioco perché lo **indirizza lungo tutto il ciclo di vita**, e può partecipare attivamente alla definizione del *concept* iniziale. Questo vale soprattutto nei team molto piccoli ed è una parte bella del lavoro.

Ma non è necessariamente ciò che l'azienda chiede. Se si viene assunti in uno studio già formato, in un secondo momento, perché serve un nuovo producer, è possibile che quella partecipazione creativa **non solo non sia richiesta, ma non sia voluta**.

Lo **scopo principale** del producer è **creare il business**:

- capire molto bene le **potenzialità del titolo** e il tipo di persone a cui si rivolge (fasce di età, genere e così via);
- **tramutare il prodotto in sviluppo in qualcosa di vendibile**;
- essere pronto anche a **fermare il lavoro del team** se ritiene che il prodotto sia troppo di nicchia o arrivi in un momento poco florido per quel tipo di gioco;
- raccogliere un "mondo" di strategie e di conoscenze di marketing e di mercato, comprese quelle dei *social manager*, e saperle tradurre in qualcosa di utile per il team.

Creatore di videogiochi, quindi, no; sì, se lo si intende come chi **indirizza lo stile dello sviluppo**.

0.5.4.3 Il game producer è un game designer?

"Direi di no, anche qui." Nei team molto piccoli succede che uno degli sviluppatori, con più attitudine o conoscenze degli altri per i social o con esperienze di partecipazione agli *expo*, assuma il ruolo di producer: è un tecnico che "fa anche quello". Ma così non funziona bene, perché sia il lavoro del game designer sia quello del producer sono **molto impegnativi** e, per ragioni di tempo e di esigenze, **non dovrebbero sovrapporsi**. Nelle startup succede abitualmente, il relatore l'ha visto succedere, ma poi bisogna **evolvere in fretta** verso una situazione in cui il producer fa solo ciò che un producer deve fare.

0.5.5 Come è cambiato il ruolo nel tempo

Segue uno spunto storico, "sempre molto curioso per gli studenti": il ruolo è cambiato molto nelle varie generazioni di videogiochi, e chi ha studiato da autodidatta qualche libro di storia del videogioco può essersene fatto un'idea "un po' nebulosa".

Anni Ottanta. Durante "una delle principali *golden age*" il producer non era definito e coincideva sostanzialmente con lo **sviluppatore**: spesso i giochi erano realizzati da una sola persona che faceva tutto, perché il lavoro era circoscritto, c'era poca memoria disponibile e un gioco poteva essere sviluppato anche in un mese. Lo sviluppatore era quasi un **imprenditore**: doveva lavorare intorno alla sua idea, realizzarla tecnicamente, inventarsi qualcosa di originale e accattivante e trovare il modo di presentarlo al pubblico, cioè di venderlo. Faceva già il producer, "anche se non sapeva neppure che esistesse questa definizione".

Anni Novanta. L'industria è molto più complessa, con studi già di una certa importanza. Il **marketing** diventa un dipartimento a sé, con budget per produrre materiali per le riviste, rilasciare interviste e cominciare a partecipare a presentazioni ed *expo*, che "esploderanno" negli anni successivi. Avendo alle spalle anni di titoli anche importanti, nasce la **gestione delle licenze**: quelle possedute dallo studio, da riutilizzare, rivendere o concedere per un periodo ad altri studi. Qualcuno deve occuparsene, ed è una mansione vicina al profilo del producer. Marketing e licenze si aggiungono così alle esigenze dei manager del team.

Dopo il 2000. Tutto diventa "in scala", ancora più grande e complicato, anche perché certe attività si moltiplicano per il numero di partner o di titoli: il producer può dover **rappresentare tutte le IP** sviluppate dallo studio.

Il lavoro è quindi molto complesso e richiede **capacità di adattamento**: chi si orienta verso questa professione deve chiedersi se si trova a suo agio in una moltitudine di mansioni[10].

0.5.6 Altri miti da sfatare

Dopo aver controllato che non ci siano domande, il relatore propone "qualche altro mito da sfatare", anche per alleggerire il discorso.

0.5.6.1 Non abbiate paura

Il producer è un ruolo estremamente importante, che può influire profondamente sul successo del lancio di un gioco; chi è alle prime armi può esserne spaventato, temere di non sentirsi all'altezza o di dover fare corsi per impratichirsi in certe competenze. Il messaggio è **non avere paura**. È normale sentirsi tesi in un'industria così competitiva, con attori enormi e in gran parte internazionali, ma lavorando si scopre che alcune cose sono "più malleabili e più semplici del previsto".

0.5.6.2 L'inglese

La prima riguarda l'**inglese**. Lavorando negli studi, anche in Italia, è molto probabile avere a che fare con partner, collaboratori o aziende stranieri[11]. Ma i timori sono infondati: gli interlocutori tipici di questa industria sono **molto casual**. Ci sono molti programmatori diventati rappresentanti o dirigenti d'azienda ("se vogliamo usare il termine *nerd*, utilizziamolo"), persone *easy going*:

- alle riunioni o in videochiamata si trovano molto difficilmente persone in cravatta; se capita, "probabilmente non sono gli interlocutori giusti", non sono quelli con più potere decisionale;
- è un ambiente di persone creative che badano alla **praticità**: hanno degli obiettivi e, se parlano con voi, è perché vogliono raggiungerli insieme a voi; quindi non si perde tempo nei formalismi;
- per esperienza del relatore, con aziende piccole e grandi, se in una comunicazione compaiono **errori di inglese**, **nessuno farà mai problemi**.

0.5.6.3 Cominciare dal mobile

Un altro mito, "creduto da tutti almeno fino a un paio di anni fa", è che un producer alle prime armi debba lanciarsi sul **mercato dei giochi per cellulare**, perché è il più grande in assoluto, "sterminato": si fa molta esperienza e quasi tutti ottengono quel minimo successo che permette di fare i primi passi. In realtà il mercato mobile (Android, iOS) è **molto controverso** perché **saturo da tempo**: era un approccio tipico a inizio carriera, ma oggi il relatore **non lo consiglia**.

I mercati mobile e delle piattaforme sono molto diversi, così come lo sono (anche se stanno "piano piano confluendo") quelli del PC e delle console casalinghe. Le strategie di mercato per un prodotto per cellulare sono molto diverse da quelle per un prodotto per PC, anche se molti titoli escono su entrambe le piattaforme: le grandi aziende cercano di massimizzare il reddito convertendo i propri giochi per ogni piattaforma. Lo spunto è quindi di chiedersi **come utenti** dove ci si trova più a proprio agio: se si gioca sempre su tablet e cellulare o se si è giocatori "pro" con desktop e schede grafiche all'ultima moda. Se si ha un'affinità con un tipo di fruizione, conviene **pensare il proprio futuro ruolo di producer focalizzato su quell'ambiente**: il producer per il mobile è diverso dal producer per le altre piattaforme.

Il relatore chiede se qualcuno si è già "spaventato" o se sono tutti ancora decisi a fare il producer; dalla classe arriva un "a posto".

[10] La trascrizione dice "in una moltitudine di nazioni"; dal contesto è quasi certamente "di mansioni".

[11] Qui la trascrizione perde un pezzo di frase: si passa da "delle aziende che" a "in effetti questo non è assolutamente così". Dal seguito si capisce che il timore da sfatare è quello di dover padroneggiare un inglese impeccabile e formale.

0.5.7 Che cosa cercano gli investitori e il mercato

La slide successiva serve a far capire più nel concreto il **lavoro quotidiano** del producer, che dovrà interfacciarsi con una serie di entità che parlano con lo studio e lavorare con loro per **creare aspettative di business**. Una delle più importanti sono gli **investitori**.

0.5.7.1 Poche idee radicali, IP "per sempre"

Oggi gli investitori sono un argomento delicato: hanno una **posizione piuttosto rigida** nei confronti degli studi. È complicato proporre concetti molto originali e restarvi coerenti fino alla fine: quando si ha una demo bisogna trovare uno sbocco sul mercato, rivolgendosi a un publisher o a un investitore, e il mercato è "non vorrei dire ostile" ma certo **poco disposto ad accogliere idee radicali**. Si cercano sostanzialmente due cose:

1. **IP utilizzabili idealmente per sempre**;
2. IP valide per **la più ampia quantità di utenti**, che includano tipologie di gioco e opzioni (la personalizzazione degli avatar, il multiplayer e così via) interessanti per tanti tipi diversi di acquirenti. In questo senso gli investitori cercano titoli "tra virgolette un po' più generici".

Qui il lavoro del producer diventa difficile e molto importante: presentare la visione e il concept dello studio facendo capire che il gioco è **originale**, perché introduce qualcosa di nuovo capace di attirare recensori e pubblico, e insieme **totalmente in linea con ciò che vende di più**. Non è una cosa buona per gli studi, soprattutto per le startup, il cui potenziale più grande è la creatività: i videogiochi sono ormai "un numero infinito" e chi lancia una startup da giovane vuole provare nuove idee, ma non è l'approccio che vogliono gli investitori.

0.5.7.2 I contenuti a pagamento: l'esempio di Street Fighter

Oltre a frenare la creatività, gli investitori cercheranno di imporre **meccaniche di contenuti a pagamento** dentro i giochi; oggi quasi tutti i generi, anche quelli storici, ne sono "intrappolati". L'esempio è **Street Fighter**, che ha attraversato tutte le generazioni e in quella attuale è diventato una sorta di **mercato di contenuti**: si paga per aggiungere personaggi, per i costumi (magari fatti apposta per Natale o Halloween), per nuovi fondali, perfino per partecipare a certi tornei, "più o meno per tutto". È un esempio molto riuscito dal punto di vista dell'investitore e del business, e "molto poco appagante" dal punto di vista dello sviluppatore, che deve occuparsi solo di **iterare all'infinito** questo meccanismo. Gli investitori guardano con attenzione a questi aspetti: il producer deve imparare a **gestirli, descriverli e venderli**.

0.5.7.3 Il *game as a service*

Altro tratto della generazione attuale è il **game as a service**. Un business sempre più grande (basti pensare al fenomeno Netflix, non solo ai videogiochi) ha bisogno di **sicurezza**, la sicurezza viene da un **flusso continuo di denaro**, e questo è garantito dagli **abbonamenti** più che dagli acquisti singoli. Investitori e management delle major hanno quindi spinto, e stanno riuscendo, a far migrare l'industria verso il **modello del servizio**:

- abbonarsi e usufruire di un certo numero di giochi senza sceglierli necessariamente;
- oppure essere appassionati di un singolo gioco come **Fortnite**, apparentemente gratuito ma concepito come un servizio: una porta in cui i giocatori entrano, costantemente aggiornata, a cui si torna per la curiosità di vedere i nuovi contenuti, i balletti, le armi, e non per la meccanica di gioco in sé. L'obiettivo non è più sconfiggere il gioco o fare record, ma **sentirsi parte del mondo** di quel gioco.

Il relatore dice di non essere "per niente un amante" di questa deriva del mercato, ma riconosce che gli studenti, che si sono affacciati da poco a questo mondo, sono già abituati a giocare così. Il producer deve interpretare questo specchio del mercato attuale e, secondo lui, avere anche **competenze storiche** per capire come ci si è arrivati.

0.5.7.4 La personalizzazione al posto dello stile

Un effetto di questa tendenza[12] è che in giochi come Fortnite, "ma tantissimi altri", anziché individuare uno **stile specifico** e curarlo in modo maniacale fino a farne il simbolo del gioco, si dà all'utente **libertà nella personalizzazione dei personaggi**. Il relatore "inorridisce" davanti a certi trailer di lancio che mostrano per gran parte del tempo come si costruisce il personaggio, come si cambiano capelli e occhi, "come se quello fosse il gioco": è un'opzione ormai presente ovunque, che dovrebbe essere superflua e poco interessante. Le aziende la mettono, e gli investitori la chiedono, perché così si includono tutti i tipi di utenti, anche quelli "super casual" che il gioco non lo comprenderanno davvero ma entreranno per costruirsi il personaggio e vagare.

Sono, per il relatore, **evoluzioni un po' aberranti** del concetto storico di videogioco e poco attinenti all'aspetto artistico, ma sono **il vocabolario del business**. Il producer vi avrà a che fare ogni giorno: dovrà guardare al gioco del proprio studio come a **uno strumento**, una "scusa" per creare dinamiche di affezionamento degli utenti e un servizio che garantisca un flusso di denaro. È certamente una persona innamorata dei videogiochi, ma deve diventare **"molto asettico"** e vedere il gioco come il presupposto per generare *cash*. Il relatore spera che questo non contrasti troppo con una visione più romantica che qualcuno può avere.

0.5.8 Prime domande dalla chat

A questo punto ci sono alcune domande in chat, e il relatore si interrompe per commentarle subito.

0.5.8.1 Il producer fa anche project management?

Prima domanda: rientrano tra i compiti del producer anche quelli di **project management**, come lo *Scrum Master* in Agile, o si tende a lasciarli a una persona separata? Il tema sarà ripreso più avanti (si vedano le sezioni 0.5.13 e 0.5.15); il project management è "molto attinente" al ruolo, quindi la risposta semplice sarebbe sì, ma "dipende".

0.5.8.2 Chi ha l'ultima parola?

Seconda domanda, di Giuseppe: chi ha l'ultima parola nello sviluppo, il game producer o il game director? E nei team molto grandi questi due ruoli devono coincidere, come nel caso di Ron Gilbert e di altri? Il relatore nota che è una domanda tipica: se gli viene fatta di nuovo, vuol dire che a chi vuole fare il producer questo aspetto sta molto a cuore.

- L'ultima parola, cioè il **potere decisionale maggiore**, ce l'ha il **game director**. È il ruolo che si avvicina più di tutti a quello del *project manager* ed è **responsabile della qualità del prodotto**; il producer è responsabile di **tradurre un prodotto di qualità in un prodotto che vende**.

- Accade però molto spesso che **producer e director coincidano**: producer di grandissimo successo, come quelli citati dallo studente, hanno la possibilità di assumere anche il ruolo di director e vogliono farlo per avere un forte **controllo creativo**.

- Le grandi aziende **non amano** questa sovrapposizione e tendono a separare i ruoli; a volte è proprio questo a far nascere problemi in uno studio.

Non ha un consiglio preciso. Tendenzialmente pensa che **entrambi i ruoli siano necessari** e che l'ideale non sia farli coincidere nella stessa persona; ma ammette, "in sintonia" con Giuseppe, che anche lui, come producer, vorrebbe essere sempre identificato anche come il director.

[12] La trascrizione qui è corrotta ("il discorso della rete astrashica"); dal seguito si capisce che il tema è l'abbandono di uno stile estetico curato a favore della personalizzazione.

0.5.8.3 Sony, il Game Pass e le scelte audaci

Giuseppe scrive anche della scelta di **Sony** di rifiutare il modello **Game Pass**, ritenuto dannoso per il suo processo produttivo e poco incentivante per la ricerca della qualità, e chiede se il producer debba assumersi la responsabilità di **scelte audaci**, contrarie alla volontà del mercato. Sono due domande diverse, su cui si può avere solo un'opinione personale, "non è detto che sia quella giusta".

- Il relatore "la pensa come Sony". Sony è in questo momento quasi **l'ultimo baluardo** tra le grandi aziende che investe (e si spera continui a investire) su **titoli in esclusiva**. Per averli deve farne dei campioni di vendite, delle *killer application*; ora sta un po' virando in altre direzioni e perdendo un po' dello spirito iniziale. In generale si esprime così perché deve avere una posizione diversa da quella di Microsoft e valorizzarla; non è detto che in futuro non faccia le stesse scelte, adottando un modello simile al Game Pass, ma oggi ha un'immagine legata alle esclusive ed è giusto che la difenda.
- Il producer **può** assumersi scelte audaci, ma bisogna vedere se gli viene permesso: sono scelte sempre molto discusse internamente. Per il relatore, ancora in linea con Giuseppe, **dovrebbe** prendersi la responsabilità di andare contro il mercato e introdurre qualcosa di nuovo, "ma purtroppo è difficile che questo accada".

Si fa quindi una **prima pausa** di cinque-dieci minuti, "c'è ancora parecchio da dire sugli investitori".

0.5.9 Investitori, abbonamenti e offerta appiattita

Alla ripresa il relatore torna sugli investitori: il tema è "perfettamente attinente" al ruolo, perché è il producer a dover tradurre in fatti concreti le loro esigenze. Per "investitori" si possono intendere, in senso generico, anche coloro che **all'interno dello stesso studio** detengono il budget. Più ci si avvicina alle major e alle aziende di grandi dimensioni, come è ormai diventata l'industria, più gli investitori vogliono **il controllo, le garanzie e un rischio più basso**.

La domanda di Giuseppe andava controcorrente: il producer è un ruolo intrinsecamente creativo che vuole sperimentare, ma questo è in contrasto con i concetti base del business delle grandi aziende, che vogliono un producer **al loro servizio**, che le aiuti a pilotare le scelte degli utenti e a ridurle il più possibile. Un modello come il **Game Pass** è simile a quello di Netflix, magari di qualità superiore, ma basato sul fatto che l'utente va **convinto a non abbandonare l'abbonamento**. Non c'è alla base la proposta di qualcosa di artisticamente valido: non è proibito, ben vengano prodotti nuovi di grande qualità, ma l'aspetto più importante è **riproporre ciò che vende da tempo**, in modo continuativo.

0.5.9.1 Una domanda più varia dell'offerta

Eppure la domanda degli utenti è **più diversificata** dell'offerta attuale. Due esempi:

La *pixel art*. È abbastanza autoriale, si ispira spesso ai *platform* classici ed è più vicina al mondo indie. Oggi c'è moltissima offerta in pixel art.

I collezionisti e il *retrogaming*. C'è chi vuole ancora un prodotto con il **packaging** e contenuti artistici aggiuntivi che non siano "l'ennesimo vestito" o personaggio selezionabile. In questa nicchia lavorano per esempio **Limited Run**, "che adesso è stata acquisita", e tutto il retroterra del retrogaming, che il relatore conosce bene perché produce **nuovi giochi per Intellivision**. Il retrogaming è molto attivo: nell'ultimo anno è "letteralmente esploso" il mercato del **Mega Drive**, con tantissimi titoli di qualità, versioni di classici prima non disponibili e idee nuove, grazie a tool avanzati che facilitano lo sviluppo. A turno tutte le vecchie console vengono coinvolte. È un mercato "tutt'altro che indifferente", in crescita, che durerà a lungo, almeno finché riguarda le console precedenti agli anni 2000, indicativamente fino alla prima PlayStation.

Nonostante questa domanda, i grandi studi **non collaborano**: non vogliono investire in produzioni chieste dai collezionisti perché, anche se disposti a spendere di più, sono pochi. Il concetto "Netflix-Game Pass" tende così ad **appiattire l'offerta** su pochi generi che funzionano sicuramente, come l'FPS. Per il relatore

è un **meccanismo educativo** su cui queste aziende investono in marketing e immagine (partecipando per esempio agli eventi di cosplay) per abituare il pubblico all'idea che l'offerta sia quella e che debba andargli bene: "l'addormentamento del grande mercato".

0.5.9.2 Il dibattito sui giocatori casual

È un tema di discussione molto acceso e non tutti condividono questa "fotografia del momento". Il relatore cita una discussione nata la settimana prima su *The Game Machine*, dove un articolo accusava apertamente i **giocatori casual** di aver "ucciso il mercato" e suggeriva loro di smettere di giocare perché il mercato si risvegliasse. Una parte del pubblico, di solito quella più veterana, condivide questa posizione; molta parte del pubblico più giovane difende invece l'idea, propagandata da anni da Netflix, che gli abbonamenti garantiscano un flusso che l'azienda reinveste per produrre sempre di più:

- se raddoppia il numero di giochi, anche se molti non sono originali né curati, aumentano anche i pochi titoli belli;
- se prima uscivano 5-6 titoli nuovi al mese, di cui magari uno di grande qualità, ora ne escono 50: guardando gli streamer che parlano delle uscite, "praticamente quasi ogni giorno esce un nuovo gioco".

Il problema, per il relatore, è che in questo modello il titolo originale o di qualità **non è trainante** nell'offerta del publisher: pur usando parte del budget, non sarà mai quello su cui si spinge di più. È affidato alla **fortuna** di trovare un creatore geniale, il cui gioco venga visto dalla major come così bello da poter diventare a sua volta "riproponibile all'infinito". È il punto di vista dei *business manager* che oggi sono ai vertici dell'industria: chi ha comprato la maggior parte delle quote di un'azienda[13] "non parlerà con voi di videogiochi", ma di come "accalappiare il pubblico e lasciarlo attaccato al vostro servizio". Un bravo producer deve avere il **coraggio di limitare questa tendenza** e introdurre dinamiche di qualità superiore: se ci riesce lo studio avrà successo, ma è un lavoro molto difficile.

0.5.10 I producer delle nicchie

Che prospettive hanno allora i producer che lavorano nelle nicchie? Quando si parla di retrogaming o pixel art si parla quasi sempre di **studi indie**, dove i ruoli si sovrappongono e il producer ha probabilmente "la fortuna" di essere anche il director e di prendere decisioni rischiose. Per amore del lavoro e per l'entusiasmo che si spera di avere ogni mattina, il relatore consiglia di fare, all'inizio o in qualche fase della carriera, l'esperienza di **lavorare in uno studio indie**: lì le dinamiche del business sono più diluite, c'è più libertà d'azione e si può sperimentare. Qualcuno ci resta, ma si parla comunque di mercati di nicchia. Se tra le opportunità che si presenteranno c'è quella di diventare il producer di un piccolo studio indie, è "un'esperienza formativa molto bella", che una volta fatta difficilmente capiterà di ripetere.

0.5.11 Le testimonianze degli studenti

Il relatore chiede allora a uno o due partecipanti, possibilmente con opinioni diverse, di prendere la parola: perché volete fare il producer, e in una grande azienda come Ubisoft o in un piccolo studio, magari aperto con degli amici?

0.5.11.1 Uri: producer in uno studio di quindici persone

Il primo a parlare è Uri[14], che più che voler fare il producer **lo fa già da un anno**, in uno studio che sta fondando con dei colleghi:

[13] La trascrizione perde qui un pezzo di frase ("di carattere cultur[...] ha comprato la maggior parte delle quote"); il senso è che gli investitori che controllano un'azienda ragionano in questi termini.

[14] Il nome è quello della trascrizione; potrebbe essere una resa imprecisa di un nome diverso.

- sono in **15**, quasi tutti di Roma, e lavorano alla loro prima IP; hanno vinto anche dei premi, tra cui un *Grand Prix* a Istanbul in occasione di una conferenza[15]. Lo studio si chiama "Weight Pointer" e il gioco "Dino Path Trail";
- si è ritrovato nel ruolo **per necessità**, insieme a un collega con cui lo divide; hanno messo a frutto ciò che hanno imparato in accademia e hanno studiato molto da autodidatti, migliorando con l'esperienza;
- il lavoro comprende chiedere consulenze, stilare gli stipendi di tutti, fare i **budget** da presentare ai publisher, cercare i publisher e interagire con loro, scrivere il **business plan**: "tutte cose che altre persone studiano un'intera università";
- il ruolo di **game director** lo ha il collega, che ha l'ultima parola sulla maggior parte delle decisioni; business plan e costi li ha stilati Uri. Riescono a convivere in questi ruoli "quasi condivisi" un po' per fortuna, un po' perché si conoscono da tanto.

Il relatore si complimenta: uno studio di 15 persone è già di **media dimensione**; la sovrapposizione dei ruoli è delicata, ma se funziona va benissimo, e farlo in due va bene perché è un lavoro complesso. Promette di contattarlo su LinkedIn. Alla domanda se consiglierebbe il ruolo agli altri, Uri risponde che, non conoscendo i loro percorsi, non si sente di consigliarlo né sconsigliarlo: se hanno predisposizione a **gestire altre persone**, ad **adattarsi alle situazioni** e sono disposti a **imparare cose che non vengono necessariamente insegnate** in accademia, dà le sue soddisfazioni e permette di prendersi alcune libertà; altrimenti, meglio prima un corso dedicato.

Walter riprende: lo scopo dei corsi che insegnano questo ruolo è innanzitutto far capire a che cosa si va incontro, cioè che si rinuncerà forse a certi aspetti belli del lavoro in uno studio per concentrarsi profondamente sul business. Una volta presa quella strada è difficile tornare indietro, e lui è il tipo di docente che "sfida molto gli studenti, perché devono essere convinti". Non è come programmare: se cambiano i tool (da Unity a Unreal o CryEngine) le esperienze di un programmatore sono riutilizzabili; il lavoro del producer invece è **in evoluzione continua** e sempre più complesso, soprattutto quando, come per lo studio di Uri, bisogna interfacciarsi con il **mercato estero**, andare a *expo* internazionali e parlare con realtà molto più grandi, con ritmi e dimensioni dei team diversi dai nostri. Aprire uno studio in Italia richiede "molto coraggio", e quel coraggio deve prenderlo "a piene mani" il producer, che in un certo senso è **in trincea**. In Italia ci sono "una quarantina di studi più o meno di svariate dimensioni", ma c'è ancora tantissimo da fare.

Sulla piattaforma, Uri racconta che lanceranno **intorno a giugno 2023 su Steam** e, a seconda del publisher con cui riusciranno ad accordarsi, anche su **console**, dove secondo le loro ricerche di marketing e di *target audience* il gioco può funzionare molto bene; ma dipende dal budget concesso, perché richiede sforzi economici. Si appoggiano quindi a un **publisher esterno**. Il relatore osserva che qui tornano i temi di prima: quanto potere decisionale viene sottratto dalla presenza di un publisher e quanto il producer sa **negoziare** questa libertà d'azione, o perfino il budget, nel corso dello sviluppo, quando ci sono milestone da rispettare e certe cose si possono rinegoziare. Aver iniziato così in una startup piccola, "con la terapia d'urto", può essere una fortuna.

0.5.11.2 Giuseppe: un team piccolo e la passione per i classici

Il relatore chiede se qualcuno ha un'esperienza diversa, per esempio in uno studio già strutturato; in mancanza, gli basta sentire un altro studente che spieghi perché vuole fare il producer. Interviene Giuseppe[16], che segue il seminario per capire meglio la figura del director. Nel suo piccolo team ricopre più mansioni: è **game producer, game director, narrative director** e fa anche *artwork*. Ciò che lo appassiona è "avere in mano un progetto e dargli un senso, dargli unicità", in linea con la *mission* del team: riproporre un contesto classico di gaming anni Ottanta-Novanta "proiettato verso il futuro". È "della vecchia scuola" e preferisce creare un prodotto **estremamente di nicchia** che faccia parlare di sé per dieci o vent'anni, piuttosto che un successo immediato "diluito" dalle richieste del mercato. È l'idea che lega il team e gli

[15] Nella trascrizione: "il Grand Prix ad Istanbul alla One Conference"; il nome dell'evento è probabilmente storpiato. Anche il nome dello studio ("Weight Pointer") e quello del gioco ("Dino Path Trail") sono riportati come si leggono nella trascrizione e potrebbero essere imprecisi.

[16] In questo tratto l'audio di Giuseppe è disturbato e la trascrizione è frammentaria (il relatore gli chiede di avvicinare il microfono); la ricostruzione riporta ciò che si capisce con ragionevole sicurezza.

dà la forza di andare avanti, anche se i membri sono sparsi per l'Italia e devono trovare il modo di riunirsi fisicamente. La parte più appagante dell'essere in controllo del progetto è interfacciarsi con ragazzi nuovi e scoprirne le ambizioni, e anche "spulciare" il mercato, i dati e le cifre pubblicate per capire se la direzione è giusta, se cambiare rotta o se dare il tutto per tutto, anche andando in *overscoping*.

Il relatore commenta che Giuseppe ha colto "la parte bella" del mestiere, anche se le indagini di mercato non danno sempre le risposte che si vorrebbero. Il team è in **fase iniziale**, quindi libero, senza milestone; di recente ha contattato **Shuhei Yoshida**[17] per avere indicazioni su una possibile pubblicazione su PlayStation. Sono molto cauti: per una futura uscita pensano di non affidarsi a Steam ma a **GOG**, perché sono "per la libertà da DRM". Walter osserva che GOG è una piattaforma "di persona più complessa da approcciare" e che servirebbe almeno un'altra alternativa, perché quelle esistenti oggi sono deboli. Sarà il ruolo di Giuseppe capire se, in funzione del concept, lanciarsi su **Nintendo Switch**, su PlayStation o su entrambe; oggi lo Switch "sta funzionando molto", e per un concept ispirato ai classici il primo passo è quello. "Però questa decisione devi farla tu, è il ruolo del game producer." Il nome provvisorio dello studio è **Bitpick Games**[18].

0.5.11.3 Commodore, l'Italia e il networking

Il relatore chiede il nome dello studio anche perché la realtà in cui lavora, **Commodore**, guarda "esclusivamente, diciamo" alla **produzione italiana**: tramite collaborazioni con le accademie vuole individuare persone competenti e appassionate necessarie per far crescere il mercato italiano, che "si muove sempre abbastanza tardi" ma ora si sta muovendo. Il consiglio, nel frattempo[19], è di contattare anche le **realtà estere**: non sperando che vi acquisiscano o diventino partner, ma per creare almeno un piccolo **network** di persone che fanno un lavoro affine, o di altri producer che lavorano all'estero, e non necessariamente in Giappone o in America, perché ci sono paesi dove l'industria è molto incentivata e gli studi crescono velocemente. È un'attività che il producer "sulla carta" non è tenuto a fare, ma a cui dovrebbe dedicare tempo: questi contatti servono a scambiarsi opinioni sull'**efficacia commerciale** di un'idea, perché per quanto si sia di nicchia o nostalgici esiste sicuramente altrove qualche piccolo studio con la stessa visione. "Fate network, così diventate più consapevoli e più forti."

0.5.11.4 Luca: il fulcro di un progetto da studenti

Il relatore ammette che chiedere interventi è "una specie di test": chi vuole fare il producer sarà **costantemente in call** e deve avere l'attitudine a parlare e anche a contraddirlo. Prende la parola Luca, tra il secondo e il terzo anno, che ha in preproduzione un progetto da presentare all'inizio del terzo anno. L'idea è portare una demo nella build finale e poi parlare con i professori e la direzione dell'accademia per capire se il progetto possa avere un futuro ed essere pubblicato. Luca vuole capire quanto debba essere "cattiva", cioè **aggressiva**, questa figura nel "bussare a tutte le porte".

"Sicuramente lo deve essere." In un contesto accademico **non bisogna avere nessuno scrupolo**: è auspicabile che i professori lavorino nel settore e abbiano quindi un interesse personale a essere coinvolti in un prodotto interessante. Poi dipende un po' dalla fortuna creare dinamiche tra studenti; la cosa più ovvia è che alcuni di quelli che lavorano insieme provino ad aprire uno studio, almeno per un anno. Proporre il titolo ai professori per capire se è vendibile "è già parte di questo lavoro".

La demo giocabile è prevista per l'**11 ottobre**, data del pitch; se autorizzati, la faranno avere al relatore. Il team conta **tre studenti di game design** (tra technical, level e game design "puro"), alcuni concept artist esterni con cui collaborano, studenti di 3D Art e di sound design: in totale **12-15 persone**. Non c'è un professore che faccia da director: fanno tutto gli studenti, senza un game director, e Luca si è preso "la briga" di gestire tutti i team, diventandone il fulcro. "Stai impazzendo?" "Sì, parecchio." Il difficile è gestire tutti contemporaneamente, le tempistiche di produzione e le milestone; "è tosta, ma ce la stiamo facendo". Il relatore osserva che le competenze acquisite in accademia, in particolare sull'**Agile**, sono importanti: non sono proprie del producer, ma meglio le si conosce, meglio è.

[17] Nella trascrizione "Shuei Yoshida": è con ogni probabilità Shuhei Yoshida, all'epoca responsabile delle iniziative indie di Sony PlayStation.

[18] Così nella trascrizione; il nome è dichiarato temporaneo.

[19] La trascrizione salta un pezzo ("nel frattemale, di andare poi a contattare le realtà estere"); il senso del consiglio è comunque chiaro.

0.5.12 Il game producer è un product manager?

Dopo le testimonianze, che hanno mostrato quante cose diverse i partecipanti stiano già facendo insieme, il relatore propone un confronto "uno a uno" per capire fino in fondo che cosa sia il producer. Si può imparare lavorando, ma poi bisogna individuare **l'obiettivo principale** del proprio ruolo, ciò che lo definisce meglio di altri, e identificarsi in quello, anche in prospettiva futura.

La slide riassume le identificazioni che sente più spesso dagli studenti:

- tantissimi, forse la maggior parte, identificano il producer con il **project manager**, che invece, come detto, va più nella direzione del game director;
- alcuni con il **product manager**;
- pochi con il **client manager**, forse perché è un concetto molto di business e non chiarissimo a tutti.

"È tutto vero", ma con pesi diversi.

0.5.12.1 Il pitch e l'arte di andare oltre il pitch

Il producer fa il **product manager**. Che lavori direttamente al concept (come alcuni partecipanti stanno facendo) o no, **sarà lui a presentare il pitch**, e tendenzialmente a realizzarlo, anche se non necessariamente da solo. L'interlocutore cambia di volta in volta, ma il producer deve avere **chiarissimo il concept**: deve partecipare alla fase di design del concept anche se lo definisce qualcun altro, essere presente alle riunioni e alle discussioni, leggere tutti i documenti di design e l'evoluzione dei GDD, avere una **visione cristallina**. Così capirà i **punti di forza** ("augurandosi che ce ne siano") e su quelli lavorerà per convincere gli interlocutori.

Il pitch è un documento di **importanza capitale**, da cui inizia il dialogo. Ma se si passa agli step successivi, la situazione tipica è questa:

- la riunione con il potenziale investitore o publisher avviene magari **dieci giorni dopo** la presentazione del pitch, con una persona che **l'ha dimenticato**, perché ne ha visti altri o ha ben altro da fare: le era piaciuto, ma non se lo ricorda più;
- nelle riunioni successive ci saranno **altri interlocutori**, con potere decisionale superiore al *product manager* contattato all'inizio. Questi il pitch dovrebbero averlo visto, visto che si è già al secondo step della negoziazione: "vi posso garantire che l'hanno ricevuto, il fatto che l'abbiano studiato è molto meno probabile".

Il producer deve quindi saper **andare oltre il pitch**. È un errore frequente in ambito accademico considerare il pitch lo strumento più potente fornito dalla carriera universitaria; è solo uno strumento. L'abilità sta nell'individuare nel pitch **uno o due punti molto sensibili** che risvegliano l'attenzione dell'interlocutore. Un pitch magnifico per grafica e contenuti può descrivere un prodotto banale, "senza nulla di memorabile": si è arrivati alla discussione grazie alla qualità del pitch, non alla potenza del prodotto. Bisogna **arricchire la narrazione** e portare sul tavolo una visione capace di trasformare un interlocutore interessato **in un finanziatore**. E la quantità del finanziamento è spesso **negoziabile**, anche se all'inizio non sembra: bisogna fare in modo che lo sia.

Questo è "il vestito del product manager": ha in mano un prodotto, che può essere una demo ma anche solo il concept, un *mock-up* e il pitch, e deve saperlo **vendere, abbellire e farne crescere il potenziale**. Il producer è quindi un product manager, e per il relatore questa attitudine è **la più importante**. Bisogna farsi un esame di coscienza: presentare un prodotto, essere sempre rimessi in discussione, riassumere idee in un certo numero di slide è stressante e molto orientato al business; le tattiche sono più o meno le stesse sia che si venda il pitch di un videogioco, sia quello "di una palestra avveniristica".

0.5.13 Il game producer è un project manager?

"Sicuramente sì", ma questo aspetto ha un'importanza **inferiore** a quello di business: è più organizzativo e, pur non essendo indispensabile, è **molto utile**. Se il producer ha attitudini da project manager ed è

coinvolto nel ciclo di vita del prodotto, ha un'idea più precisa dei **tempi di sviluppo**. Questo significa **non fare promesse sbagliate**, "tendenziose", e non negoziare milestone che torneranno indietro "come un boomerang" sul lavoro del team, creando dinamiche difficili da gestire, che dovrà gestire lui, "perché le avete create voi".

0.5.13.1 L'esempio delle piattaforme

Lo stesso vale per la **scelta delle piattaforme**, che può avere motivazioni precise legate al lavoro del team:

- un team può decidere di puntare inizialmente sullo **Switch** per ragioni valide; il producer che va a vendere il pitch si sentirà "quasi sicuramente" dire: "lo vogliamo su tutte le piattaforme, compreso Android". Può essere sbagliato, ma bisogna saperlo **giustificare**;

- conoscere le dinamiche del progetto e la quantità e qualità delle risorse del team dà le informazioni per **argomentare e negoziare**: se ha senso andare su più piattaforme, quale sia la prima, se iniziare come d'abitudine da **Steam** e poi valutare Xbox e PlayStation;

- un caso tipico: un progetto nasce per PC e poi console, ma a lavori in corso decide di **rinunciare alle console** e restare solo su PC, magari perché il riscontro di mercato è inferiore alle aspettative. L'investitore dirà: "a maggior ragione, si è venduto poco su PC, tanto a voi non costa molto convertirlo, magari un mese di lavoro, facciamolo su tutte le altre piattaforme e cerchiamo di recuperare".

Chi mette il budget ha la tendenza innata a **semplificare troppo**: ciò che per lui è semplice diventa per il team una richiesta molto impattante, che magari blocca risorse già al lavoro sui DLC o sul concept successivo e ritarda tutto. Di queste cose si occupa di solito il *product owner*, "il vero project manager"; ma se il producer sa interpretare il flusso di lavoro e leggere i Gantt è **molto più forte contrattualmente** quando discute con i partner tempi e direzioni del prodotto. Il producer, quindi, non è davvero un project manager, ma **sarebbe auspicabile che lo fosse**.

Il terzo punto viene rinviato dopo una **seconda pausa** di una decina di minuti.

0.5.14 Il game producer è un client manager

Il terzo concetto è "molto interessante", complesso da comprendere e spesso trascurato nell'insegnamento universitario: di solito lo si impara portando avanti la carriera, non alle prime armi, con il proseguire dei rapporti con i publisher. La risposta è subito sì: **il bravo game producer è un client manager**.

0.5.14.1 Rapporti sbilanciati

Quando si formalizza un rapporto con un partner, tipicamente un publisher, nascono dinamiche collaborative fortemente **vincolate dai termini del contratto iniziale**. Chi ha in mano il budget tende a posizionarsi in modo **predominante** e, dove possibile, a forzare il rapporto:

- distribuire i pagamenti come preferisce, di solito **pagando poco all'inizio** e le cifre più alte verso la consegna;

- **ritardare i pagamenti** "accampando mille scuse", sapendo che, nonostante il contratto sia vincolante, lo studio non ne farà mai una questione legale[20].

Le condizioni del **primo contratto** con un publisher non saranno ideali: saranno **sbilanciate verso il publisher**. Sarebbe logico migliorarle nel lavoro successivo, ma gli ampliamenti e i prolungamenti dei contratti sono di solito legati a un **porting** del titolo già previsto o alla realizzazione di **DLC**, cioè, almeno per un po', al primo prodotto negoziato: sarà molto difficile giustificare un cambio di condizioni.

[20] La trascrizione qui perde un pezzo ("voi non andrete mai a farne una questione che finisce in funzionare nel modo giusto"); il senso più probabile è che lo studio difficilmente arriverà a una causa.

0.5.14.2 Costruire un rapporto paritario

Il futuro economico dello studio dipende allora dalla capacità di instaurare un **rapporto di fiducia il più possibile paritario**: una collaborazione che non vada in una sola direzione, dal committente che fornisce il budget allo studio che consegna rispettando le milestone. Bisogna **coinvolgere anche il committente** su alcune responsabilità:

- spingerlo a fare **reportistica** sulle sue mansioni, tipicamente il marketing;
- chiedere conto di **come, dove e per quanto tempo** il prodotto viene pubblicizzato, e perché è presente a certi *expo* e non ad altri;
- diventare una **parte attiva** anche nel lavoro del publisher.

Il rapporto diventa più complesso ma più paritario, e lo studio ottiene un certo **potere contrattuale**. È un livello molto avanzato, a cui i producer arrivano raramente, anche perché gli studi sono spesso al primo lavoro.

0.5.14.3 Il modello familiare italiano

In Italia[21] il problema è aggravato dal fatto che l'industria ha un **modello molto familiare**: il budget dipende di solito da **un'unica persona**, quella che ha investito di più e vuole avere voce in capitolo, "che fa la voce grossa". Con aziende a conduzione pseudo-familiare tutto è più complicato: il lavoro del producer in Italia è **ancora più difficile** che altrove.

Per chi ha la forza di portarlo avanti, però, è un **vantaggio**: la trincea è più formativa. Dopo un paio di prodotti realizzati con interlocutori italiani, se al terzo ci si interfaccia con un **publisher internazionale**, si scoprirà che "tutto sarà molto più facile": i contratti vengono rispettati di più, le tempistiche sono più precise e soprattutto c'è un **flusso di comunicazione continuo**, mentre in Italia tutto tende a essere "sotto banco", in confidenza, "parlato invece che scritto".

Il monito è doppio: sapere che queste difficoltà fanno crescere più in fretta, ma **non adottare** quegli atteggiamenti poco professionali. Bisogna riconoscerli subito, capire chi si ha di fronte, essere molto **diplomatici**, saper **prevenire** i problemi, sapere che un interlocutore ritarderà le comunicazioni ed elaborare strategie per farle riprendere. La mancanza di comunicazione si traduce in **ritardi**, di pagamenti e di milestone.

0.5.14.4 Trasformare tutto in un vantaggio

Essere client manager significa dare un **valore aggiunto** che deriva da tutte le competenze viste: il coinvolgimento nel lavoro del team, il sapere in anticipo quando sarà pronta una milestone, se arriverà prima del previsto o se ci saranno ritardi, e trasformare tutto questo in strumenti a proprio vantaggio. "Persino il vostro ritardo può diventare un vantaggio", perché può darsi che in quel momento il publisher sia contento di ritardare anche il pagamento. Lo si capisce solo con doti di **diplomazia e comunicazione**, sapendo che dall'altra parte potrebbero mancare: è "lo step finale, il livello più alto" del producer. È utile acquisire competenze **psicologiche e linguistiche** ed esperienze in **ambiti internazionali**, dove le abitudini lavorative sono molto diverse, per poi riportarle in Italia e "forzare un pochettino" i propri interlocutori (publisher, investitori) a adottarle, suggerendole "in maniera nascosta" nello scambio di comunicazioni.

Non ci sono manuali che dicano "vi arriva questo messaggio e dovete rispondere così": è una parte che si sviluppa nel tempo, ed è la prova che il mestiere evolverà molto negli anni, crescendo di complessità.

0.5.15 Agile dal punto di vista del producer

Si passa poi agli aspetti **metodologici**, descritti da vari manuali e forse già visti in altri corsi, con l'obiettivo di tradurli nella vita quotidiana di un producer: quanto sono utili, quanto vanno conosciuti a fondo e se sono sempre giusti, perché lavorando "non tutto è quello che uno si aspetta" e bisogna farsi una propria opinione.

[21] Anche qui la trascrizione salta qualche parola ("e questo è un problemaese"); dal seguito è chiaro che si parla dell'Italia.

Da tre o quattro anni, nell'industria dei videogiochi, la metodologia **Agile** è molto in auge: molti studi formano i dipendenti o la chiedono già negli annunci di lavoro, volendo candidati certificati o con esperienza in ambienti Agile. Sembra fondamentale, utile, potente; ma dal punto di vista del producer ci sono **due lati della medaglia**, e la realtà lavorativa non è quella descritta dalla metodologia.

0.5.15.1 Product owner e project manager

L'aspetto di Agile che più si sposa con il lavoro del producer è il **product owner**, che deve "dettare i ritmi del lavoro". Qualcuno lo identifica con il **project manager**, ma non è esattamente così:

Il project manager è quello che elaborava, ed elabora ancora, i **Gantt**; un tempo era certificato PMI (il relatore ha questa certificazione). È una persona capace di **pianificare tutto il progetto** dall'inizio alla consegna: deve aver già immaginato i tempi, individuato i punti nevralgici dove introdurre un po' di **ridondanza** per allungare le fasi più complesse, saper stimare quante risorse servono e allocarle.

Il product owner fa in parte questo, ma lavora con ritmi **molto più ristretti e dinamici** e non è abituato ai Gantt: come evolverà il progetto fra sei mesi non gli interessa "e non può neanche saperlo". Deve **risintonizzare continuamente** tutte le attività, con riunioni e metodi Agile che permettono di capire se il team lavora in linea con le sue capacità produttive. Altrimenti può accelerare i ritmi, non solo ridistribuendo i task, ma anche intervenendo sulla **struttura**: individuare una carenza da eliminare, per esempio assumendo una nuova risorsa, discuterne con il game director e valutare se c'è il budget per ridimensionare il team.

Non è tenuto a sapere tutto[22], ma deve saper rispondere a domande come: "abbiamo una milestone fra un mese, mi consegnerai una demo adeguata oppure mi proponi qualcosa di diverso?". Deve sapere **dove andrà a parare il progetto** e, magari su richiesta del producer, saperne **cambiare la direzione**, anche radicalmente, per esempio introducendo un supporto al multiplayer non previsto. E deve farlo non come un project manager che studia a tavolino chi fa cosa, ma **confrontandosi con il team** e lasciandogli libertà d'azione perché sia il team stesso a proporre una soluzione. Il relatore lo chiama "una specie di **direttore d'orchestra**"[23].

Nella metodologia, che è "più su carta", bisogna considerare che Agile è nato in ambito **manifatturiero e industriale** e ha pensato il product owner come mansione negli ambienti IT, molto più statici delle necessità di un team di videogiochi. La differenza è sottile ma importante: la **capacità di adattamento** del product owner è un valore aggiunto per la produttività del team.

0.5.15.2 Quanto vale questo prodotto?

Il product owner dovrebbe dialogare con il producer e con il director per **quantificare il valore** del prodotto che il team sta realizzando: è in linea con le aspettative o ha un potenziale **superiore**? Se sì, su che cosa lavorare per **massimizzarlo**? E, ancora più importante in prospettiva, quali caratteristiche lo rendono appetibile anche in un altro modo[24]? Un bravo product owner sa **far evolvere il progetto verso nuove direzioni**.

0.5.15.3 Agile non è una Bibbia

In questo senso alcune pratiche tipiche di Agile diventano **restrittive**, troppo meccaniche e ripetitive. Un product owner può rispettare alla lettera i dettami che fissano riunioni in certi modi, in certi giorni della settimana e con una certa durata, "tutte cose statisticamente valide" ma poco applicabili agli studi di videogiochi. Il relatore insegna Agile e ne condivide l'uso, perché è valido per organizzare il lavoro, ma è **molto cauto** nel considerarlo "una Bibbia". Ci sono correnti di pensiero per cui Agile farà fare il salto di qualità all'azienda, è inconcepibile non adottarlo e tutti i dipendenti devono essere certificati. Per lui è giusto che qualcuno lo sia, non tutti: un **musicista**, anche se dipendente interno, può non volere vincoli

[22] La trascrizione perde qui un pezzo di frase ("e le quasi, una cosa del genere. Non è tenuto lui a saperlo").
[23] Nella trascrizione "registra d'orchestra": è quasi certamente "direttore d'orchestra".
[24] La frase nella trascrizione è incompleta ("e una cosa ancora più importante in prospettiva successiva. che ha questo gioco che lo rendono appetibile anche in una maniera diversa").

né partecipare a riunioni in cui non ha argomenti, e voler essere libero di lavorare nei giorni e nelle ore che preferisce. Ricondurre queste dinamiche in uno schema, per quanto valido, finisce per ingabbiarle e **ridurre la produttività** di ogni singola persona.

Il relatore ama considerare un team di sviluppo **un team di appassionati di videogiochi**: chi fa questo mestiere di solito ama giocare, e quindi dovrebbe avere piacere a giocare al gioco che sta realizzando. Perché sia così, il team deve poter gestire un po' da solo i propri ritmi:

- prendendosi pause in cui tutti i programmatori fanno i **tester** e provano il risultato;
- **scambiandosi le posizioni** anche solo per un giorno, per mettere una parte del team nelle condizioni dell'altra.

Agile prevede in realtà questo flusso continuo di comunicazione, ma non può essere schematico: **deve essere spontaneo**. Il product owner ha quindi la grande responsabilità di **motivare il team**, di accorgersi quando rende poco perché è diventato troppo meccanico e di rompere gli schemi, anche con attività che non riguardano la consegna, come mezza giornata di **team building**, purché sensata e gradita a tutti.

0.5.15.4 Gli sprint

Lo **sprint** è "un po' lo specchio" del lavoro del product owner. Il relatore lo presenta come una **riunione in cui si fa il punto sull'avanzamento**[25], tipicamente a fine o inizio settimana, il venerdì o il lunedì: "di solito si tende a farle il venerdì ed è assolutamente sbagliato"[26], ma sono scelte che si faranno nel proprio team. Il metodo indica la cadenza in modo "quasi matematico":

- ogni **quattro settimane** all'inizio del progetto;
- ogni **quindici giorni** durante lo svolgimento;
- **settimanali** vicino alla consegna o quando si constata un ritardo.

La realtà, avverte, è molto diversa[27]. Conoscere i concetti di Agile è un bene, ma il team deve poi fare il passo successivo: **capire da solo come preferisce lavorare** e gestire gli sprint. Il product owner non può fare il **project manager "dittatore"**, che ha in mano il Gantt e quindi sa che cosa è meglio: deve essere **uno dei membri del team** che ascolta i consigli e le esigenze di tutti.

0.5.15.5 Product owner e producer: un binomio

Il product owner **può** essere il producer, e negli studi appena nati succede, ma non dovrebbe: dovrebbe essere il suo **braccio destro**, pur restando del tutto indipendente, un alleato che aiuta a individuare le **potenzialità latenti** del progetto o quelle che funzionano bene, per accelerare su di esse e dire al producer: "poni l'accento su questi aspetti".

Product owner	**Game producer**
è responsabile dell'**evoluzione del prodotto**, di quanto bene verrà	è responsabile della **roadmap**: che cosa ci facciamo con questo prodotto?
aumenta le potenzialità del prodotto, "le fa esplodere"	fa sue queste potenzialità come **argomenti comunicativi** per portare il prodotto sul mercato

Sono due ruoli distinti ma **perfettamente in sintonia**. Per il relatore l'arma più potente di uno studio è avere **un bravo product owner e un bravo producer con grande affinità**, magari che si conoscevano da prima, magari ex compagni di studi che hanno deciso di lavorare insieme in queste due posizioni: un binomio dal potenziale enorme, anche se non sempre sviluppato nel modo giusto. È auspicabile che il

[25] È la definizione del relatore: nella terminologia Scrum lo sprint è propriamente il ciclo di lavoro di durata fissa, e le riunioni che lo scandiscono sono lo *sprint planning*, la *sprint review* e la *retrospective*, che il relatore riprende più avanti.

[26] Così nella trascrizione, che subito dopo perde un pezzo ("ma che ci sono scelte che farete"); il relatore non spiega perché, e non è del tutto certo che il giudizio si riferisca al venerdì.

[27] Il relatore inizia a raccontare un'esperienza ("io ho lavorato in team che..."), ma la trascrizione la perde.

producer **assimili il lavoro del product owner** più del contrario, come se gli fosse sempre accanto nell'ottimizzazione del team.

Nelle startup i ruoli si confondono e una sola persona fa tutto: non c'è nulla di male in un team di 5-6 persone, in cui chi ha più attitudini manageriali se ne occupa. Ma se il team cresce servono **due persone forti in queste due posizioni**; se la stessa persona continua a fare product owner e producer si crea una **debolezza**, perché manca lo stimolo reciproco e si sarà carenti da un lato o dall'altro. Il product owner è quindi un concetto di Agile fondamentale anche negli studi di videogiochi, purché lo si interpreti con la massima **dinamicità**, che il metodo originario, forte proprio dell'organizzazione preventiva, non prevedeva.

0.5.15.6 Il product manager secondo Agile

Il **product manager** è, per Agile, un **esperto di mercato**: deve sapere che tipo di prodotti vendono e soprattutto in che direzione vogliono andare i **gestori delle piattaforme**, cioè gli store, che condizionano molto, per esempio il livello dei **prezzi medi**: lo studio non decide quanto costerà il gioco, deve allinearsi al mercato. Inoltre:

- deve fare un minimo di strategia sulle **finestre di lancio**, anche se oggi escono giochi ogni giorno. Un tempo il grosso del business era a **Natale**, poi sono nate due o tre fasce più o meno importanti; oggi è tutto più diluito, ma un'azienda strutturata che lavora su due progetti terrà il prodotto su cui punta di più per i momenti strategici (ancora Natale, eventualmente) e l'altro per un periodo in cui, in quel genere, c'è poca concorrenza;
- **negozia le revenue**, verso i licenziatari (se il prodotto è su licenza) o verso il publisher. Si punterà sui guadagni delle vendite, e la quota negoziabile è ancora ampia: difficilmente supera il **50%**, più probabilmente sta **fra il 30 e il 50%** nei confronti di un finanziatore o di un publisher[28]. Bisogna tentare "il gioco del rilancio", ma i publisher sono molto potenti.

La definizione non è rivoluzionaria. La differenza con il producer è **concettuale**: per Agile il product manager è un ruolo **finanziario**, una persona che porta numeri e dà valore al prodotto con un numero. Potrebbe non sapere nulla del gioco: gli si danno titolo e *box art*, sa più o meno il genere e lo confronta con le tendenze per fare proiezioni. Anche il producer dà un valore numerico, ma quel numero deve esprimere **la potenzialità del concept**: conoscendo a fondo design e concept, capisce se c'è un potenziale e come giustificarlo all'interlocutore.

L'idea latente di tutti questi confronti è che il producer sia **sempre qualcosa di più** di questi ruoli: usa gli strumenti del product manager ma è più competente sull'argomento; sa fare il project manager, non in modo meccanico ma come **motivatore**; dialoga con il product owner per introdurre la creatività giusta. Qui si scontra con l'idea di fondo di Agile, nata in ambito industriale per descrivere con precisione il lavoro settimanale del team: nei videogiochi il lavoro cambia di settimana in settimana e le priorità devono essere dettate **da chi lavora**. L'esperto di grafica che riceve dal product owner la richiesta di un asset entro una settimana può rispondere che non è possibile, o che sarebbe meglio modificarlo e proporre qualcosa di diverso. Se c'è apertura mentale e collaborazione, **si va oltre Agile**: diventa un insieme di precetti utili a cui aggiungere valore. È importante capirlo perché molti studi credono che certificarsi Agile sostituisca automaticamente il lavoro che devono fare, in particolare parte di quello del producer, "mentre in realtà non è così".

0.5.15.7 Il team Agile: i vantaggi

Prima della pausa successiva, il relatore elenca i vantaggi del concetto di **team** in Agile, aspetti "molto validi" da far propri:

Parte dalle specifiche, ma è creativo. Sa esattamente che cosa dovrà fare il prodotto ma, se in corso d'opera si accorge che qualcosa non funziona o potrebbe funzionare meglio, agisce **contro le specifiche** iniziali e propone correzioni, o nei casi estremi corregge direttamente e lo riferisce allo sprint. Si sposa perfettamente con uno studio di videogiochi.

[28] Dalla trascrizione non è chiaro se la percentuale indichi la quota che resta allo studio o quella del finanziatore: "potrete negoziare una revenue che è ancora abbastanza ampia [...] è difficile che superi il 50% [...] fra il 30 e il 50% nei confronti di un finanziatore". La lettura più naturale è che sia la quota dello sviluppatore.

Modifica le scadenze d'accordo con il cliente. Qui interviene il producer, ed è fattibile anche con interlocutori grandi: se lo si fa **molto in anticipo** e nei termini giusti, di solito si riesce. "Tanto alla fine questo comporterà sempre che avrete pagati più in là", quindi è un argomento che funziona.

Modifica i requisiti discutendoli con il cliente. Più difficile: serve un canale sempre aperto. Per esempio, lo studio cambia idea o ritiene che abbia più potenziale un **personaggio femminile**, e Agile prevede che questo si rinegozi[29]. Anche l'interlocutore dovrebbe essere certificato Agile, ma più in generale è questione di buonsenso e di quanti rapporti si sono già avuti: al primo progetto è difficile. Se ci si riesce è un vantaggio, perché team e product owner capiscono meglio del committente il valore di una modifica.

Riutilizza i *minimum viable product*. In ambito industriale, parti dei meccanismi di un prodotto possono servire per un altro, senza riprogettarle da zero[30]. Si sposa perfettamente con il lavoro del programmatore: si cerca di riutilizzare perfino **un intero engine**; dopo un'avventura *platform* 3D se ne fa un'altra di ambientazione fantascientifica, sempre con personaggi 3D che camminano e saltano, riusando il lavoro e le soluzioni tecniche già trovate. "Appare addirittura abbastanza ovvio."

Il relatore chiede se qualcuno sta già usando Agile in un progetto; arriva una risposta affermativa in chat ("a noi sta capitando nel nostro progetto")[31], e si fa una **terza pausa**.

0.5.15.8 L'esperienza di Luca con gli sprint

Alla ripresa il relatore si scusa per le pause brevi e per una parte "un po' più noiosa", ma insiste: il valore di un seminario con chi lavora nel settore non è dire "esiste Agile, lo usano tutti, certificatevi", ma far capire che Agile ha **concetti validi** da conoscere perché danno spunti, da cui poi elaborare strategie proprie[32].

Luca ha scritto che il suo team ha dovuto **riformulare tutta la pianificazione** per andare incontro alle necessità dei **sound designer**: un buon esempio, perché una parte del team che di solito ha poca voce in capitolo ha espresso un'esigenza e gli altri si sono adeguati. Invitato a raccontare, Luca spiega che:

- lavorano a **sprint di due settimane**, "la scelta classica", e usano Agile da **due anni**, da quando glielo hanno presentato in accademia, trovandosi molto bene;
- un suo collega fa proprio lo **Scrum Master** e ha sviluppato una documentazione "veramente degna di nota", premiata dai compagni e dai professori, gestendo tutto in modo molto dinamico;
- quando i sound designer hanno detto che un certo tipo di task andava gestito diversamente, per esigenze diverse da quelle del game design, hanno fatto una riunione con le figure di riferimento e deciso di sviluppare quei task in modo diverso, sia per dare loro il tempo necessario sia per avere il metodo giusto per visualizzarli e implementarli;
- non hanno un vero product owner, "non siamo così avanti", ma il collega in pratica ne fa le veci, insieme ad altre mansioni;
- la documentazione non sono dei pitch, ma **fogli Excel** con cui lo Scrum Master gestisce tutti i team e gli sprint e controlla la **velocity**, sia a livello di progetto sia per singolo team (sound design, game design, 3D art).

Il relatore commenta che i suoi consigli vanno sempre presi come tali: con l'evolversi del progetto e l'eventuale crescita del team, bisogna puntare **il prima possibile** ad avere una persona che faccia **solo il product owner**. Nei piccoli team si tende a dire che tutti devono fare qualcosa "di oggettivamente riscontrabile" e a considerare il product owner trascurabile, "come un DPU"[33]: invece è un ruolo **molto importante**. Il fatto

[29] La trascrizione salta un pezzo prima dell'esempio ("bisogna avere un canale aperto costantemente con l'altro ma poi il team che lo sviluppa, c'è lo studio, cambia idea").

[30] Così il relatore descrive il concetto, avvicinandolo al riuso di componenti; nella letteratura Agile il *minimum viable product* indica piuttosto la versione minima di un prodotto sufficiente a raccogliere riscontri dagli utenti.

[31] Nella trascrizione: "A noi sta capitando il nostro oggetto". Dal seguito si capisce che a scrivere è Luca.

[32] A un certo punto il relatore si interrompe perché il telefono si è attivato da solo ("Come ti posso aiutare?").

[33] Così nella trascrizione; la sigla non è chiara. Il senso è che il ruolo viene percepito come accessorio, non produttivo.

che i team siano ben distinti richiama poi il concetto che i singoli team possono essere considerati **entità riutilizzabili** nel progetto successivo o in più progetti contemporaneamente: il valore di un buon Scrum Master emerge proprio quando si lavora su **due titoli insieme**. "State usando l'Agile come si deve."

0.5.15.9 Altri vantaggi

Il relatore riprende i vantaggi, più velocemente:

Impara dal cliente. Le commesse possono essere strane, originali o diverse dal solito. Un'azienda con una certa *expertise* tende a dire al cliente qual è il modo migliore di fare una cosa; un team Agile deve essere abbastanza flessibile da capire che una richiesta strana può farlo lavorare in modo più originale e insegnargli qualcosa da riusare.

Delega il project management. Velocizza le attività invece di accentrare tutto nelle mani del product owner, e il relatore lo riscontra ogni giorno. Se un team di grafica è di due o tre persone, una più senior, è giusto che questa, ricevute le attività pianificate dal product owner, si senta **in diritto di riorganizzarle e riallocarle**, perché sa chi è più bravo e veloce in una certa cosa. L'importante è che allo sprint successivo sia tutto **esplicitato**, il product owner lo sappia e si valuti insieme se le scelte sono state giuste.

Chi finisce prima non si ferma. Se un team è in anticipo non deve restare fermo, ma essere autorizzato a riorganizzare il proprio lavoro, anticipandolo, e portare allo sprint un vantaggio[34]. Sembra ovvio, ma nella realtà ci sono molti blocchi: chi è in anticipo spesso si ferma e non lo comunica, per esempio per **paura** che il product owner, visto che è più bravo del previsto, gli assegni più lavoro. Non deve succedere: il team non deve aver paura di mostrare di essere bravo, ma assumersi più **autonomia**, dettandosi da solo il ritmo. Dare più lavoro solo perché si è più bravi sono "concetti vecchi, antiquati, tipicamente italiani mi viene da dire, industriali"; le startup di videogiochi devono essere aziende moderne che valorizzano il contributo di ciascuno.

Non si aspetta lo sprint. Né per l'anticipo né per il **ritardo**: appena si è consapevoli di non riuscire a rispettare i tempi si coinvolge il product owner e si rinegozia l'attività. Ogni team è autosufficiente e deve essere **produttivo**; i team di videogiochi non sempre lo sono, ma devono esserlo.

0.5.15.10 Gli svantaggi

Tornando al concetto ribadito più volte, Agile non è "la Bibbia che ha già previsto tutto", come diranno ripetutamente quelli che vendono corsi e certificazioni. Alcuni aspetti critici:

È nato per il software. Cioè per i programmatori; ma un team di videogiochi è fatto anche di **artisti**, con esigenze, ritmi e abitudini molto diversi.

È nato per l'ufficio. Con orari fissi uguali per tutti: in America il tipico *nine to five*, in Italia un po' di più, ma comunque circoscritto. Un team di videogiochi ha contrattualmente orari precisi, ma non è nato per lavorare così: ci sono persone produttive la sera o perfino nel weekend[35]. L'obiettivo è il lavoro produttivo di un **team creativo**.

Fatica a includere lo smart working. Il metodo è "abbastanza cristallizzato". In Italia ci sono ancora aziende convinte che sia meglio tenere i team sotto controllo, o che un team tutto in una stanza "con un rumore assordante" sia sinonimo di creatività. Per il relatore lo **smart working ha solo vantaggi**: è una strategia di **aumento drastico della produttività**, da cucire su misura per ogni dipendente[36]. C'è chi preferisce l'ufficio (la casa è troppo buia, i gatti danno fastidio, qualunque

[34] La trascrizione qui è incompleta ("Quindi, il dialog prima del previsto"): probabilmente la voce della slide riguardava il lavoro completato prima del previsto.

[35] La trascrizione qui è lacunosa ("Nel caso in cui ab negli anni 60, a cui si fa riferimento l'Agile"). Il riferimento agli anni Sessanta riguarda probabilmente i modelli industriali e d'ufficio da cui Agile eredita alcune idee; il Manifesto Agile vero e proprio è del 2001.

[36] Nella trascrizione: "che dovete vestire uno per uno ai vostri dipendenti".

motivo) e va benissimo, ma altri lavorano meglio da casa. È fondamentale quando si usano **risorse esterne o straniere**: i consulenti dall'estero lavoreranno per forza in smart working, quindi conviene abituarsi subito.

L'autonomia può sfuggire di mano. Il team Agile si assume dei **rischi**, in modo ponderato, perché di solito conviene per aumentare i ritmi. Per questo il product owner è importante: valuta i rischi e interviene quando si perde il controllo sull'avanzamento o un team diventa **autoreferenziale** e non comunica più con gli altri responsabili. Anche la delega del project management è un vantaggio ma anche uno svantaggio: bisogna imparare in fretta il ruolo per gestire queste cose con delicatezza, soprattutto in smart working, dove non c'è contatto visivo e si valuta il lavoro delle persone dai feedback su WhatsApp o dalle consegne.

0.5.16 Il Kanban del producer

Più che una metodologia, il **Kanban** è un vero strumento, il più conosciuto, "il più ovvio" e storicamente la prima cosa introdotta da Agile[37]. Funziona davvero in tutti gli ambiti e, in casi estremi (il relatore l'ha verificato di persona), può perfino **sostituire il Gantt**: il Gantt dà molte più informazioni e punti di controllo con cui il project manager ottimizza, mentre il Kanban riassume tutto in poche colonne. Semplifica molto, ma è una **fotografia immediatamente visibile e sempre consultabile dello stato di salute del progetto**, ed è ciò di cui ha bisogno il producer.

Il producer non dovrebbe essere ignaro di ciò che si sta facendo, e dovrebbe lavorare sempre con il product owner; ma se è sempre in viaggio, non ha tempo di incontrare il team o lavora da remoto in un'altra città, il Kanban è "**la salvezza imprescindibile**": la fotografia del progetto rispetto alle milestone che dovrà giustificare con gli interlocutori.

0.5.16.1 Le colonne e il livello di dettaglio

Le attività sono di norma **da fare**, **in corso** e **concluse**. Nel Kanban del producer **non ci sono i dettagli** di che cosa fanno i team di animazione, modellazione 3D, level design: si lavora **per concetti**. Per esempio: "il personaggio principale è utilizzabile?"; "abbiamo almeno un livello completamente giocabile da proporre in riunione a un interlocutore?". Il producer deve individuare le **chiavi fondamentali** che descrivono il progetto e riportarle nella tabella. Alla fine i Kanban sono quindi **due**: quello di progetto vero e proprio e quello che il producer deve costruirsi.

0.5.16.2 Dove va la colonna del testing?

Nei videogiochi c'è quasi sempre una quarta colonna, quella della **QA**, del testing. Il relatore lancia una sfida: in che posizione la mettereste? Il primo a rispondere, Daniele, scrive: **fra il *doing* e il *done***. È la risposta tipica: si pensa che, realizzato un prodotto, ci sia una fase di test conclusa la quale si passa ad altro[38].

Per il relatore è **sbagliato**: la QA viene **dopo il *done***, come quarta colonna. Dal punto di vista del programmatore era giusto metterla fra *doing* e *done*; dal punto di vista del producer la QA è un'attività che **non ha mai fine** e va proiettata a medio termine, pensando al prodotto come qualcosa che viene rilasciato e poi ulteriormente corretto in base al **feedback degli utenti**. È diventata purtroppo la prassi: si rilasciano prodotti con dei bug, aspettando di vedere chi se ne accorge, e appena arrivano le segnalazioni si correggono; oppure gli utenti evidenziano problemi non individuati nella fase interna. Si usa **l'utenza per fare ulteriore QA**, e questo avviene dopo il rilascio, dopo il *done*. Per il producer questo feedback è il più importante: va documentato e tenuto presente, perché deve conoscere le reazioni degli acquirenti e capire se il prodotto si sta trascinando in ulteriori *fixing*. La QA resta quindi un'attività latente non solo durante lo sviluppo, ma **anche dopo**.

[37] Così il relatore. Storicamente il Kanban nasce nella produzione industriale giapponese (Toyota) e solo in seguito è stato adottato nello sviluppo software accanto ai metodi Agile.

[38] In chat arriva anche la proposta di considerare il testing come parte del *doing*; il relatore osserva che così si torna ad avere tre colonne.

To do	Doing	Done	QA
da fare	in corso	concluso	continua anche dopo il rilascio

0.5.16.3 Colli di bottiglia e senso di avanzamento

Il Kanban ha avuto successo soprattutto perché **evidenzia i colli di bottiglia**: in origine era una tabella appesa al muro con tanti post-it, magari di colori diversi, e un'enorme quantità di post-it nella zona centrale (tanto lavoro in corso che non si sblocca) segnalava un collo di bottiglia. Anche qui il producer ragiona diversamente: i blocchi tecnici[39] non lo interessano; gli interessa capire se i concetti della visione del prodotto sono **verificabili dal possibile acquirente tramite la demo**, anche se dietro c'è un bug che fa bloccare il gioco. Non è un problema se, davanti a un publisher, la demo va in *crash*[40]: "il publisher [...] sa benissimo che le demo crashano". Un collo di bottiglia che, a livello di programmazione, farebbe dire al product owner che la demo non è pronta, per il producer va interpretato in un altro modo.

Un altro vantaggio del Kanban, nell'ottica dell'ufficio con il team nella stessa stanza, è che, appeso al muro, mostra quanto lavoro è concluso e quindi **appaga il team**, che vede di avvicinarsi all'obiettivo. Anche questo è lontano dall'idea di lavoro del producer, per il quale **il lavoro non è mai finito**: la build lanciata sul mercato non è un progetto concluso, ma il primo o l'ennesimo step di un **rapporto di business** che può evolvere nel prodotto successivo, in DLC, in versioni ampliate o per una nuova piattaforma. Il Kanban standard è quindi "un po' troppo circoscritto, un po' troppo semplice" per il producer; il relatore ne ha parlato perché è troppo importante e utile nel team perché il producer non lo usi.

0.5.17 Il framework Scrum

Il relatore passa più velocemente, "per non risultare tedioso", allo **Scrum framework**, di cui bisogna avere presenti tre concetti: l'idea di miglioramento continuo, lo schema "3-3-5-5" e la **project vision**.

0.5.17.1 Le caratteristiche dello Scrum

Miglioramento continuo. Il relatore lo chiama "*view and tune*"[41]. Il prodotto è sviluppato con l'idea di poterlo **migliorare continuamente**: non si trova un metodo e poi si va "con gli stampini per sempre", come si faceva nell'industria. È stato uno degli aspetti rivoluzionari: la produzione parte, ma settimane dopo il prodotto esce dalla catena di montaggio con qualche piccola miglioria. Lo Scrum si applica a ogni ambito, dal marketing al software.

Incrementale. Evolve alla stessa velocità con cui cresce la complessità del sistema che descrive.

Trasparente. "Molto interessante per noi." Le descrizioni dei processi di ogni attività che porta a realizzare una componente (pensata in origine per un prodotto fisico; nei videogiochi, per esempio, una singola routine) dovrebbero essere comprese da **tutti i membri del team**, anche da chi lavora in un'altra parte del ciclo e riceve quei componenti già pronti: capire come sono fatti permette di avere un'opinione su come si potevano fare diversamente. Oggi si traduce nella **documentazione** e negli strumenti per condividerla, che non devono essere "un malloppo di GDD" che descrive dettagliatamente ogni fase e che alla fine nessuno ha voglia di leggere: i documenti devono essere **snelli e dinamici**, con una **terminologia condivisa**, non leggibili solo dai programmatori di grafica perché contengono termini che conoscono solo loro. Se quei termini servono, tutti devono studiarne la definizione: **nessuna nomenclatura deve essere incomprensibile** per qualcuno del team.

Ispettivo. Il product owner verifica continuamente che tutti gli artefatti prodotti siano coerenti con l'**obiettivo finale**, senza ridondanza né dispersione. È giusto che tutti dicano la loro ("si poteva aggiungere questo

[39] La trascrizione perde qui un pezzo ("l'attività non puòatori in realtà non interessano il producer"); il senso è che i blocchi interni allo sviluppo non sono il centro dell'attenzione del producer.

[40] Nella trascrizione "cresce", evidente storpiatura di "crasha".

[41] Così nella trascrizione ("View & Tune"); con ogni probabilità si tratta del principio di ispezione e adattamento (*inspect and adapt*) o comunque del miglioramento continuo, che è il senso della spiegazione.

o quello"), ma bisogna **scremare**, distinguere una vera miglioria da un semplice "di più": altrimenti si trova sempre qualcosa da aggiungere e si allungano i tempi. Il product owner deve saper "vivisezionare" i contributi creativi e andare dritto all'obiettivo.

Adattabile. Il product owner sa cambiare metodologie di lavoro, perché i progetti evolvono e soprattutto evolvono gli engine e gli strumenti: può abbandonarne alcuni e adottarne altri, anche se comporta un periodo di studio per capire il nuovo tool.

0.5.17.2 Lo schema 3-3-5-5

Il "3-3-5-5" riassume gli elementi di Scrum: **tre ruoli**, **tre artefatti**, **cinque eventi** e **cinque valori**[42].

I tre ruoli. **Product owner**, **team** e **Scrum Master**. Lo Scrum Master è per il relatore un ruolo "abbastanza **mitologico**", soprattutto nei team di videogiochi: non è sempre chiaro a tutti che cosa faccia e se debba esistere. Di solito fa riferimento al product owner ed è una sorta di **predicatore**: chi più di tutti ha compreso la **visione del progetto** e la ricorda sempre a tutti. Nella definizione "gestisce e prende il backlog", "ma qui sorvoliamo".

I tre artefatti. **Product backlog**, **sprint backlog** e **incremento**, cioè ciò che è stato effettivamente realizzato, con la reportistica e il confronto nei meeting: quanto, in concreto, "se volessimo semplificare" in percentuale, il progetto è andato avanti.

- Il **product backlog** è di solito responsabilità del product owner e potrebbe essere identificato con le **milestone**: una milestone potrebbe essere descritta da un backlog ("è stato fatto questo, questo e questo: la milestone è conclusa"). È semplicistico, perché il backlog **evolve**: dopo un mese la milestone è sempre lì, ma il backlog è cambiato molto, quindi andrebbe ridefinita anche la milestone, "cosa che ovviamente è sempre meglio non fare": la milestone deve restare ben fissa. Per il producer il backlog è una **fotografia** di quanto il progetto è in linea con le milestone o sta prendendo un'altra direzione, nel bene o nel male, e serve a **rinegoziare il business** con gli interlocutori. Tutto questo andrebbe però **filtrato dal product owner**.
- Lo **sprint** di solito non coinvolge il producer, almeno in un team ben dimensionato e coordinato. Il relatore **sconsiglia** al producer di partecipare agli sprint, che sono molto numerosi: se ne occupa il product owner, che deve poi fornire il feedback come strumento utilizzabile. In pratica producer, director e product owner finiscono per sovrapporsi in certe fasi: al relatore capita ancora di condurre degli sprint, "però non dovrei farlo io". Se capita, non vuol dire che non lo si debba fare, ma bisogna essere consapevoli che **qualcosa non funziona** nella struttura del team.
- L'**incremento**, un insieme di elementi testabili, per il producer è soprattutto **l'output della QA**.

I cinque eventi. Sprint, *sprint planning*, *daily scrum*, *sprint review* e *retrospective*. L'elemento essenziale, per il relatore, è che tutte queste entità sono **sovrabbondanti** rispetto alle necessità di un producer. Se ci sono e sono gestite bene, ben vengano ("meglio più documentazione che meno"), ma il producer ha talmente tante attività che deve saper **scremare**, semplificare e accedere a **meno documentazione**, anche se disponibile. Vanno usati **con parsimonia**, trasferendo nel proprio lavoro ciò che si scopre davvero utile.

0.5.17.3 Quando gli sprint si infittiscono

In particolare per gli sprint: ogni due settimane è il metodo solito. Se diventano **settimanali** vuol dire essenzialmente due cose:

1. ci sono dei **problemi** e serve un monitoraggio più stretto; ma allora dopo qualche settimana lo sprint dovrebbe tornare quindicinale, altrimenti il team non riesce a riadattarsi;

[42] Nella trascrizione "3355". I cinque valori sono trattati nella sezione 0.5.17.5.

2. più spesso, purtroppo, c'è un **eccesso di volontà di controllo** da parte di qualcuno, tipicamente il game director.

Se chi può chiederlo aumenta sprint, riunioni, reportistica e feedback solo per avere il controllo, senza un vantaggio reale per il team, in sostanza lo **rallenta**. Succede molto spesso; il messaggio è: **non siate voi quella persona**. Il producer, che vorrà spesso più informazioni possibili, deve essere cauto per non intralciare il team, saper **filtrare** ed essere essenziale; il monito sarebbe addirittura di **non partecipare affatto agli sprint**. Ma sono cose che "cozzano con la realtà": nella quotidianità spesso Scrum e Agile non si riescono ad applicare correttamente. Non è una tragedia, si possono comunque raggiungere gli obiettivi; l'importante è **sapere** quando una cosa non è fatta nel modo ottimale, che c'è una debolezza nel team, così che quando arriverà l'occasione per porvi rimedio si potrà consigliare come fare.

0.5.17.4 I meeting e la regola dei venti minuti

Il relatore scorre velocemente le durate dei meeting previste da Scrum (lo *sprint planning* "solitamente limitato a otto ore", la *sprint review* "al massimo 4 ore", la *retrospective* anch'essa definita in ore)[43], "cose troppo meccaniche che poi andrete a cambiare a modo vostro". Ne ricava una nozione che torna utile in qualunque contesto: idealmente **il miglior meeting possibile dura 20 minuti**. Se un meeting, organizzato da voi o da altri, "buca" i venti minuti, e succederà spesso, **non è efficace** come dovrebbe. È una regola da approfondire, ma va presa come "insegnamento che cade dall'alto".

Il criterio generale per il producer: essere presente non ai meeting in cui si ascolta come stanno andando le cose o come sono stati risolti i problemi, ma a quelli in cui si è nella posizione di **suggerire o pilotare un cambiamento**, non di come si lavora, ma di **come dovrebbe risultare il prodotto finale**. In sintesi:

Evento	**Il producer...**
Sprint	può pilotarlo, se serve un feedback più continuo a seconda della fase di avanzamento, delle negoziazioni con il publisher ecc.
Sprint planning	gli è utile, ma è meglio non partecipare: ridurre la propria presenza
Daily scrum	ammesso che il team lo faccia, non lo deve coinvolgere
Sprint review	è **fondamentale**: serve per fare il punto della situazione
Retrospective	può tranquillamente non partecipare

Il senso di tutto il discorso è che Agile è una metodologia ben definita, "ma forse troppo definita": per il producer deve essere solo **un fiume su cui navigare** alla velocità che conviene, senza fermarsi a tutte le tappe previste, viaggiando a modo proprio.

0.5.17.5 I cinque valori

Il relatore chiude lo Scrum con i **cinque valori**, che idealmente ogni membro del team dovrebbe far propri[44]:

Apertura, cioè la disponibilità ad adattarsi a ogni lavoro.

Coraggio di lavorare per risolvere le difficoltà invece di nascondersi.

Rispetto. Per esperienza del relatore è quello che ha più valenza, e che "non spesso" gli capita di riscontrare. È il rispetto tra i membri del team e verso il concetto stesso di team, e qui il producer ha molta voce in capitolo. Si riflette sulla comunicazione: qualunque mancanza di rispetto "prima o poi trapela, oggi che siamo nei social", e il lavoro del producer è fortemente legato al feedback che si diffonde sui social. Non si può cambiare l'indole di nessuno, ma il rispetto, per un producer, va coltivato a fondo: se si è manchevoli, "vi si ritorcerà contro pesantemente".

Concentrazione (focus) e **impegno**: "non c'è bisogno di approfondire".

[43] Il relatore legge le slide molto in fretta e la trascrizione è lacunosa; accenna anche a una certificazione come "ottimizzatore dei meeting", ma la frase è incompleta.

[44] Nella guida ufficiale di Scrum i cinque valori sono impegno, focus, apertura, rispetto e coraggio. Il relatore dice che compongono "la parola dello scrum", ma l'affermazione non trova riscontro nella parola stessa: probabilmente si riferiva a una slide in cui i valori erano disposti intorno al nome.

Segue un'altra pausa[45]; il relatore chiede se qualcuno ha "finito le energie" ("Ci siamo, ci siamo") e promette, dopo questa parte "un po' troppo descrittiva", immagini e cose "più veloci, più divertenti".

0.5.18 Producer famosi

Alla ripresa il relatore cambia "regime": si immagina che, dopo ore di "il producer fa questo, quest'altro è meglio che non lo faccia", qualcuno stia pensando che il mestiere sia difficile e noioso. Ma al producer è spesso associata anche l'idea del **divo**, del personaggio finito su tutte le riviste. Per coinvolgere i partecipanti mostra alcune fotografie e chiede chi le riconosce.

John Romero. Riconosciuto subito da un paio di persone. È famoso per *Wolfenstein 3D*, *Doom* e *Quake*; ha un'aria "un po' più anni Settanta", con i capelli lunghi, ed è un divo perché ha **definito il genere degli FPS**, che soprattutto con *Doom* è diventato il genere più prolifico in assoluto e ancora oggi traina.

Hideo Kojima. "Questo forse era troppo facile." Celebre soprattutto per *Metal Gear Solid*, anche se il relatore ha citato un titolo che gli piace di più, *Zone of the Enders*. È un producer, ma soprattutto un **narratore** e un rivoluzionario, che ha definito un proprio modello di producer, più simile a uno **scrittore**, un ideatore di storie e personaggi che sono usciti dal singolo gioco per diventare **IP essi stessi**, "IP all'interno delle IP".

Hironobu Sakaguchi. Lo riconosce Giuseppe ("sei molto informato"). Non si concede molto e il suo volto è poco noto, a differenza della sua creazione, *Final Fantasy*. Il relatore non lo definirebbe neppure un producer: ha sempre pensato alla sua creazione come un **regista**, partendo da un contesto e da un ambiente per creare un mondo che vive di storie a sé stanti, all'infinito. La curiosità: chiamò il gioco *Final* perché non era affatto sicuro di proseguire il mestiere; se il publisher non avesse accettato il gioco avrebbe rinunciato e sarebbe tornato a fare **ingegneria**, come voleva suo padre[46].

Un autore di strategici. La foto successiva mette in difficoltà i partecipanti; il relatore aiuta citando *Pirates!*, *Civilization* e anche *XCOM*, titoli ancora oggi riproposti e che continuano a vendere[47]. All'uscita erano "qualcosa di più di una *killer application*": *Civilization* andava provato a prescindere, anche se il genere non piaceva o non lo si capiva; era "un evento", uno di quei punti di passaggio dell'industria.

Shinji Mikami. Riconosciuto dopo un tentativo scherzoso[48]: *Resident Evil* e *Devil May Cry*. Il *survival horror* esisteva già in qualche forma, ma è con *Resident Evil* che si è affermato. Il relatore lo ricorda bene: lo provò su PlayStation senza averne letto nulla, perché gli arrivò la versione giapponese molto prima dell'uscita europea, e "rimase sconvolto come un po' tutti quanti". Oggi, tornando ai discorsi di prima, *Resident Evil* è passato dalle mani del producer a quelle del **marketing**, che lo ripropone in ogni salsa (film, cartoni animati), "fino probabilmente a farcelo odiare a un certo punto".

Riccardo Zacconi. Nessuno lo riconosce, neppure Giuseppe: è un **italiano**, il creatore di *Candy Crush*, che ancora oggi supera i **40 milioni di utenti attivi al mese**[49]. Non vive più in Italia, risulta essere in Inghilterra e occuparsi d'altro. È un esempio "benaugurante" di producer italiano che ha avuto un successo sterminato con un singolo titolo, "quindi si è arricchito, in poche parole". Un caso estremo, ma italiano; ce ne sono altri.

Ancora oggi può capitare che il producer sia una persona famosa, intervistata, perché ha proposto qualcosa di innovativo che ha dato una sterzata al mercato. Il relatore considera l'Italia **molto creativa**: i

[45] La trascrizione registra qui una quarta pausa; nell'insieme il seminario ne ha quindi almeno quattro, tutte brevi.

[46] È l'aneddoto come lo racconta il relatore. La versione più diffusa dice che Sakaguchi pensava di lasciare i videogiochi e tornare all'università se il gioco non avesse avuto successo; il dettaglio sul padre e sull'ingegneria non è verificabile dalla lezione.

[47] Il nome del personaggio non compare nella trascrizione. *Pirates!* e *Civilization* indicano con ogni probabilità Sid Meier; *XCOM*, pur legato a MicroProse e poi a Firaxis, non è propriamente un suo titolo.

[48] Nella trascrizione si legge "Il pilota di Formula 1": è probabilmente una risposta scherzosa di un partecipante prima di quella giusta.

[49] La cifra è quella data dal relatore. Zacconi è più precisamente cofondatore ed ex amministratore delegato di King, la società che ha pubblicato *Candy Crush Saga*.

programmatori italiani hanno sempre avuto, per i pochi prodotti fatti anche nelle generazioni passate, una visione particolare, diversa da quella di altri paesi (come accade anche per Francia o Inghilterra), e quei punti di vista andrebbero valorizzati. È un lavoro "bello e appagante per quanto è difficile e a volte un po' sommerso".

0.5.19 La visione

Per concludere l'*excursus* su Scrum il relatore torna sulla **vision**, molto legata al ruolo del producer, cercando di non essere "troppo laconico"[50]. La **visione di tutto il progetto** è principalmente **compito del producer**; in Scrum si esplicita con il *project vision meeting*, ma "non parliamo più di definizioni".

Al di là di inventarsi un'idea rivoluzionaria come i personaggi appena visti, uno degli aspetti fondamentali del producer è **ragionare in termini di business**, sia per una IP nuova da lanciare, sia per ciò che lo studio ha già: IP in sviluppo o già sviluppate che possono essere riutilizzate. Il producer deve:

- saper creare dei **business case**;
- sposarli con la visione sia della singola IP sia di **tutto lo studio**;
- sviluppare questa visione con gli elementi fondamentali del team, come il product owner, e quelli "più o meno fondamentali" come lo Scrum Master;
- saperla **descrivere in modo chiaro e veloce** in qualsiasi circostanza.

0.5.19.1 La visione dello studio

Il relatore la identifica soprattutto con la **visione dello studio**. Giuseppe ha parlato di giochi ispirati ai classici: è una visione che riguarda tutto lo studio. Una startup può lanciarsi nell'avventura, vedere come va, adattarsi al mercato e cambiare rotta, ma sarebbe importante che avesse **una sua visione fin dall'inizio** e sapesse lavorarci intorno per produrre qualcosa di vendibile, con il contributo del producer. Se una piccola azienda si specializza in una tipologia di gioco, ha uno **stile grafico ben definito** che tutti i suoi giochi rispecchiano ("cartoonesco, *cel shading*, quello che volete"[51]) o vuole specializzarsi nello *storytelling* piuttosto che in altri aspetti della giocabilità, qualunque sia l'elemento distintivo, il lavoro del producer che dovrà vendere i prodotti sarà **più facile**.

In prospettiva ci possono essere sbocchi importanti: una startup spera "innegabilmente" di ricevere entro due o tre anni un'**offerta di acquisizione**, che magari poi non accetta, se ha avuto riscontri sufficienti per restare autonoma. Ma è molto complicato, perché all'estero l'industria è fatta di team con budget molto grandi, grazie ad **aiuti fiscali**, **business angel** e investitori interessati che aiutano le startup, cosa che in Italia tendenzialmente non funziona (il relatore ne parlerà più nel dettaglio in un successivo workshop su come strutturare uno studio). Proprio per la difficoltà di emergere, avere **elementi distintivi** e lavorare sulla visione dello studio è fondamentale, e se ne deve far carico il producer. Se si è capaci di produrre tante idee, concept e visioni per il proprio studio, probabilmente è il lavoro giusto.

0.5.19.2 Imparate le metodologie, poi abbandonatele

Dopo la slide su *business goal* e fattori di rischio ("siamo ancora troppo prolissi"), la conclusione su Scrum e Agile è questa: **imparate queste metodologie**, studiate Agile, capite quali elementi sono in linea con le vostre idee e vi sono davvero utili. "Ma poi io vi incentivo ad **abbandonare queste metodologie** e a cercare un vostro approccio", fatto di **best practice**, di **piccoli documenti creati da voi** da riutilizzare in certe situazioni, di un lavoro autonomo in cui si decide da soli che cosa fare in ogni circostanza.

Tutti i framework come Scrum sono **orientati al team**, al lavoro di gruppo, cercano di essere "all inclusive". Ma il producer non deve pensare al lavoro di gruppo, bensì **al concept**: è l'elemento del team che lavora concettualmente "al di sopra di tutti gli altri", disegnando la visione intorno al lavoro del team. Le

[50] Così nella trascrizione; dal contesto il relatore intende probabilmente "troppo prolisso", termine che usa poco dopo.
[51] Nella trascrizione "side shading": quasi certamente *cel shading*.

metodologie non devono ingabbiarlo; deve essere libero di essere creativo, anche andando contro metodologie e ritmi del team, talvolta interrompendone il lavoro o contraddicendolo.

E non deve pensare solo ai numeri: "voi non siete *sales manager*[52], non siete responsabili del personale, non siete il commercialista del team". I numeri devono riflettere l'obiettivo del gioco, il **messaggio** che vuole dare al mercato, e per questo il producer non deve mai abbandonare l'idea di **essere un videogiocatore**.

0.5.19.3 Rimanere videogiocatori

Il producer, se riesce a ritagliarsi il tempo, deve **continuare a giocare**: tanti tipi di giochi, i giochi della concorrenza, le nuove piattaforme quando arrivano. Con gli anni e lo stress il tempo per giocare si riduce "al lumicino" (il relatore ammette di non riuscire a trovarlo in questo momento), ma bisogna sforzarsi. Più di chiunque altro, il producer deve **rimanere un videogiocatore**, anche a 40 o 50 anni.

Vale anche adesso: chi si avvicina a questo mestiere deve già essere un videogiocatore. Chi non ama i videogiochi o non ha quell'esperienza (per esempio quella che permette di riconoscere i personaggi delle foto) deve farsi un esame di coscienza: al di là delle competenze metodologiche e da business manager, il lavoro parlerà di videogiochi e deve trapelare che il producer ama l'argomento. "Se vendete videogiochi e non siete un videogiocatore, questa cosa si percepirà." Per il relatore è una caratteristica **fondamentale**.

0.5.19.4 Costruitevi il vostro Kanban

Le tabelle Kanban sono orientate soprattutto al product owner. Quindi: **createvi le vostre tabelle**. Il relatore ha sviluppato un paio di concetti che evolvono il Kanban: l'idea di base è valida, ma va sviluppata, non applicata solo perché la si trova in corsi, certificazioni e libri e "tutti la usano". Il producer è la parte creativa del team, e ogni strumento che usa deve essergli utile, quindi va **rielaborato**. È un esercizio molto interessante costruirsi una tabella Kanban personale, differenziata da quella standard:

- è **molto concettuale**: parte dal **concetto del gioco** che si vuole vendere e conclude il suo percorso con **l'esperienza che il cliente vivrà** giocandolo;
- tutto ciò che descrive il prodotto ma non la visione e il concept è, per il producer, **superfluo**, e gli fa perdere attinenza con il suo ruolo.

Una best practice ricavata dall'esperienza personale: una volta costruito il proprio Kanban, **imponetevi dei momenti in cui lo rivoluzionate**, lo rimettete in discussione, anche se funziona: riscriverlo, aggiungere colonne, revisionarlo. "Aggiungete creatività al vostro flusso creativo", per non incanalarsi in attività ripetitive che, poco a poco, disperdono il valore aggiunto dato al team.

Il producer dovrebbe essere un ruolo **non sostituibile** da un altro producer preso a caso: deve **identificarsi con lo studio**, diventarne il personaggio più rappresentativo e, agli occhi del publisher e degli interlocutori, in un certo senso il più importante. Per questo deve mettersi sempre in discussione ed essere consapevole se, di settimana in settimana, ha introdotto qualcosa di creativo nell'attività del team.

0.5.20 Creare opportunità: publisher e interlocutori

In definitiva, che cosa fa il producer? **Crea opportunità di business**: usa il prodotto realizzato come "uno strumento da portarsi in valigetta" per parlare con potenziali partner e creare possibilità. Deve saper spiegare l'opportunità e **far innamorare gli investitori** del concetto. Per questo la visione deve essere una sorta di **missione**: bisogna essere **predicatori della visione** del proprio studio[53]. Il **coraggio**, che è anche un valore di Agile, va benissimo: lanciatevi "senza aver preso la laurea da game producer", perché una cosa del genere "non esiste in tutto e per tutto". È un mestiere che si impara facendolo, dandogli la propria impronta.

[52] Nella trascrizione "size manager"; dal contesto è probabilmente *sales manager*.

[53] La trascrizione perde qui una parte del discorso ("dovete essere predicatori della visione del vost pratica, quando avrete fatto i vostri primi tentativi"); il senso generale è che molte cose si impareranno nella pratica.

0.5.20.1 Scegliere il publisher

Poi bisogna saper **trovare e selezionare i publisher**, se si lavora già in uno studio che cerca chi pubblicherà i suoi giochi. Le alternative non sono sempre tante (magari alla fine un solo publisher è interessato), ma idealmente bisognerebbe averne e saper valutare l'**esperienza** del publisher, le sue **capacità** e soprattutto la sua **serietà**.

In Italia, "torniamo a mettere i puntini sulle i", la **professionalità** nell'industria è ancora "tutta da costruire". I publisher sono pochi e non abituati a lavorare con studi internazionali che hanno ritmi e aspettative di puntualità molto alti[54]. È il panorama di tanti settori, e quello dei videogiochi non fa eccezione. Il producer deve quindi saper cogliere il **tono degli interlocutori**: quanto sono seri, quando diffidare di promesse e "parlantina eccessiva" e soprattutto **distinguere quelli che parlano da quelli che scrivono**. "Tutto ciò che non risulta scritto è destinato a cambiare nel corso del tempo." Va benissimo anche un'email.

0.5.20.2 Che cosa è compito del producer

Non è compito del producer (potrebbe esserlo, ma non dovrebbe) dettagliare i **contratti** e definirne gli elementi legali; può esserlo definire le **milestone** al loro interno. È invece certamente suo compito **capire l'affidabilità dell'interlocutore**: un elemento difficile e fondamentale per il destino dello studio. Piccole startup di 5, 6, 10 studenti, che per un breve periodo si affidano ai fondi dei familiari o sperano in piccoli finanziamenti pubblici, hanno una prospettiva che non va oltre **un anno, un anno e mezzo**: devono trovare in fretta partner che abbiano possibilmente **liquidità**. Al di là delle dimensioni, esistono **partner seri** che hanno voglia di aiutare. Bisogna essere perspicaci nel capire il tono delle comunicazioni e bravi a descrivere la visione, perché l'interlocutore la capisca, la condivida e voglia lo studio con sé: lo studio diventerà uno degli elementi distintivi di quel publisher, che avrà piacere di parlarne agli eventi e magari "produrrà un poster un po' più grosso" per quel gioco, "perché è il primo che gli viene in mente". Sembrano discorsi banali, ma nella pratica sono complicati.

0.5.20.3 La legge di Murphy

Se c'è una cosa del profilo del project manager che il producer deve assimilare, è che i project manager sono "assolutamente ligi alla **legge di Murphy**": se qualcosa può andare male, andrà male. Ciò che non viene insegnato adeguatamente è che **l'insieme delle cose che possono andare male è sempre sottovalutato**: andranno male quelle previste, ma anche molte altre non identificate, a sorpresa. "Questo non è assolutamente pessimismo, è professionalità", perché i problemi ci saranno sempre. Il producer non è padrone dello studio, del publisher, dell'andamento del mercato, né decide se il pubblico comprerà il gioco: è una delle tante persone dello scenario, quella che cerca di valorizzare il più possibile il potenziale del prodotto. Se anche solo metà degli attori va nella direzione opposta, si è "in un mare di guai". Succede quasi sistematicamente, quindi:

- non tutti i progetti andranno a buon fine;
- servono dei **piani B**;
- serve una **strategia di sviluppo**, non a lungo termine, ma almeno a medio termine: una visione che si spinga a **2-3 anni**, un tempo già molto lungo in questa industria. Fra tre anni magari esce già la PlayStation 6; ci saranno evoluzioni della **realtà virtuale**, cose che oggi si immagina esisteranno ma non si sa ancora come interpretare.

Lo stesso vale per il **metaverso**, "fortemente spinto da Facebook", che spiega come sarà la nostra vita "senza però sapere se a qualcuno di noi interessa vivere in quella maniera lì"[55]. Investire oggi in videogiochi nel metaverso ha senso se si viene **finanziati profumatamente** da un publisher che vuole sperimentare in quella direzione; se invece si devono anticipare risorse strutturali proprie, "pensateci molto bene". Sono commenti molto pratici, "fatti da un game producer verso dei game producer": il relatore dice di parlare ai partecipanti considerandoli tali.

[54] Anche qui la trascrizione perde qualche parola ("In Italia si lavora un po' più. Iniziamo e poi").

[55] Prima del metaverso la trascrizione perde un pezzo ("non sembrano proprio più l'argomento del giorno staremo a vedere"), forse riferito ad altre tecnologie di moda.

0.5.21 Trovare il developer giusto

Tornando a una frase della slide, "trovare il giusto developer": il producer non è necessariamente legato a un'azienda in cui è nato e in cui resterà per sempre, o alla nascita di una startup. Può essere un **professionista** che ha già fatto questo lavoro in altri progetti e altre aziende e viene assunto da un developer che ha bisogno di un producer. In quel caso si parla di un producer già definito, non di un junior, che ha capito che continuerà a fare questo lavoro; e allora deve saper **valutare l'interlocutore dall'altro lato della barricata**: il developer che gli propone di collaborare è adatto?

- Il suo **livello di professionalità** è alto o basso? È da quantificare secondo la propria sintonia.
- Il suo **livello di esperienza**? Non è detto che debba essere esperto: uno studio alle prime armi, nuovo e molto piccolo, può essere un vantaggio, perché il producer assume una posizione più preponderante e gli si affideranno moltissimo, dandogli ascolto. C'è naturalmente un rischio grande.
- La cosa più importante: **usano delle metodologie**? Non necessariamente Agile, ma hanno una **struttura interna**? E dentro quella struttura sono **rispettosi del lavoro dei dipendenti**, c'è **comunicazione**?

Per un ruolo in cui comunicazione e creatività sono il pane quotidiano, ritrovarsi in un team dove la gente ha **paura di parlare**, è terrorizzata dal capo, non osa dare un'opinione e fatica a comunicare un ritardo, magari preferendo lavorare di nascosto qualche ora in più a casa per paura di essere sgridata ("parlo sempre di esperienza vissuta"), è un pessimo segnale. Non è facile capirlo, ma si può **visitare la sede**: vedere come sono fatti gli uffici, che cosa c'è appeso ai muri, se le persone che si incontrano sono palesemente stressate o si stanno un po' divertendo. Bisogna restare vigili: se un developer cerca un producer perché evidentemente qualcosa non funziona, "è disperato e ha bisogno di qualcuno che gli risolva i problemi", non è detto che ci si debba accollare tutti i problemi di un team che non funziona. Se invece l'ambiente è accattivante, inclusivo, con persone motivate, probabilmente ci si troverà bene. È un consiglio per chi è già più senior, e il relatore augura a tutti di trovarsi in quella posizione.

0.5.22 L'attitudine del game producer

Tutto questo descrive l'**attitudine** che il producer dovrebbe avere. È un lavoro molto stressante, difficile, con orari non sempre ben definiti, e quindi deve includere una componente, "volevo dire divertimento, ma diciamo più di **stimolo e di sfida**". Forse, dice il relatore, la cosa più importante che i partecipanti porteranno via dal seminario è **il dubbio** se vogliono o no fare il producer: se decidono di no, benissimo; se decidono di sì, ancora meglio. Gli esempi servono a farsi un quadro di sé stessi dentro un team in cui c'è una persona che fa il producer e che, giorno per giorno, deve un po' **inventarsi il mestiere**: cercare opportunità, parlare con tanta gente, al telefono e su WhatsApp, fare cene dopo il lavoro, presentare l'azienda nelle accademie, partecipare agli *expo*. Attività che non si svolgono "andando alle 9 del mattino sedendosi lì", cosa che pure si farà, ma non tutti i giorni. Il mestiere è ancora in evoluzione, e i futuri producer saranno tra quelli che lo descrivono e lo modificano.

0.5.22.1 La guida del team

Agli occhi di chi lavora con lui, il producer deve essere **la guida del team verso il risultato**. Quando un gioco è concluso, pubblicato su Steam e si cominciano a guardare vendite e commenti nelle community, si va "un po' allo sbaraglio" e si resta ansiosi. Il punto di riferimento è il producer: deve **dare fiducia al team** ed eventualmente rispondere se il gioco non vende. Non è colpa sua, ma forse qualcosa poteva essere fatto diversamente e qualche correttivo si può ancora fare; in quel momento, più che in altre fasi, diventa cruciale la sua **comunicazione**[56]: magari andavano fatte attività parallele alla pubblicità del gioco per far conoscere di più lo studio.

"Non siete la persona più importante del team, questo è un concetto sbagliato." Il producer è una delle parti del team, ma quella che più di ogni altra sta **sul fronte del palco**:

[56] La frase nella trascrizione è spezzata ("diventa cruciale la attimino la comunicazione nei vostri confronti").

- quando si cerca un partner, è lui a dover convincere che il team è valido e vincente;
- quando si va agli *expo* a mostrare il gioco, è lui a dover far credere al pubblico che il gioco ha qualcosa in più degli altri;
- probabilmente rilascerà interviste su canali YouTube: non è obbligato, ma il producer ha una certa autonomia e sarà lui a **investire sulla propria immagine**.

0.5.22.2 Fare pratica davanti alla telecamera

Per questo è utile fare esperienza: qualcuno magari è già uno streamer, abituato alla telecamera. Altrimenti conviene **partecipare a qualche piccola community** o a un canale YouTube di nicchia sui videogiochi, dedicato a qualcosa che interessa e di cui si sa parlare, e trovare il modo di partecipare a qualche **live**. Sono competenze "solo relativamente di marketing", più di *social management*, e più pratica si ha, più il lavoro sarà facile.

Ci sono tantissimi aspetti da coltivare, con alcuni si avrà più affinità, con altri meno: si possono tranquillamente **investire più sforzi in ciò che è più in sintonia** con i propri gusti. Se non piace apparire in video si eviterà di farlo, ma qualcuno dovrà farlo al proprio posto: si può **coinvolgere un programmatore** e mandarlo a parlare dello sviluppo del gioco. Si possono realizzare documenti interni al team in forma di **brochure o fanzine**, che riassumano la vita del team e diventino un appuntamento motivante. E bisogna imparare che cos'è la **gamification** (si veda la sezione 0.5.24).

0.5.22.3 Dedizione, comunicazione, umiltà

"Adesso mi sembra proprio di essere sul pulpito": ognuno troverà la propria strada, perché, diversamente da tanti altri lavori, questo è abbastanza **personale**. Il relatore insiste però su alcuni punti:

Dedizione. Il lavoro può essere appagante ma è pesante. Anche andare agli *expo*, esperienza molto divertente, con gli anni diventa ripetitivo ed è faticoso per l'impegno logistico.

Creatività. "L'abbiamo detta abbondantemente."

Comunicazione professionale. È **assolutamente cruciale**. Non servono corsi di dizione, ma non bisogna mai rinunciare a comunicare. La comunicazione deve essere **principalmente scritta** e va benissimo che sia **abbondante**. Bisogna **diffidare dei partner che non si fanno sentire per più di due o tre giorni**, anche solo con una riga: la comunicazione deve essere continua, perché è una parte preponderante del lavoro.

Consistenza. Va bene vendere e abbellire il prodotto, perché si vende la visione, ma **non bisogna mai parlare di cose che non esistono** né far credere cose diverse dalla realtà, altrimenti si perde in fretta credibilità. Il producer deve essere la persona su cui un publisher vuole scommettere: attraverso di lui scommette sulla startup, e se non lo vede completamente affidabile ne valuterà altre.

Umiltà. Non bisogna cadere nell'errore di credersi l'azienda con il prodotto eccezionale che qualunque publisher vorrebbe, per cui "se un publisher non lo vuole non ha capito niente". Ci sono tanti prodotti e tante aziende, e i publisher ragionano in termini diversi da quelli dell'appassionato, **solo per il business**: bisogna capire e interpretare ciò che vogliono. "Quindi niente supponenza, umiltà, ma entusiasmo", consapevoli di avere qualcosa di valido da mostrare.

0.5.23 B2B e B2C

Un "velocissimo accenno" alla differenza tra mercati **B2B** e **B2C**.

B2B. L'attività del producer si svolge solo da azienda ad azienda, per esempio quando lo studio è ingaggiato da un publisher per realizzare **prodotti su licenza**. Sarà il publisher a dire che cosa va fatto e in quali tempi, senza accettare molto contributo creativo. I problemi sono parecchi: probabilmente c'è una **escalation di autorità**, quindi una comunicazione più rigida, e pochissima elasticità sulle milestone.

È uno scenario che riguarderà facilmente le nuove startup: un conto è ideare un gioco nuovo da lanciare da sé, un altro è farsi riconoscere come sviluppatore che contribuisce al lavoro altrui, in *development* o semplicemente **su commessa**, perché un'azienda che lavora a un concept non ha abbastanza personale. Sarebbe ottimo riuscire a lavorare in **co-development**. Ma qui il ruolo del producer è "un pochettino sminuito": poca libertà d'azione, conta soprattutto la **diplomazia** e mantenere la comunicazione nei giusti binari.

B2C. Il producer è più vicino al concetto tradizionale del B2C, l'azienda che studia il lancio di un prodotto di largo consumo, anche se oggi funziona così solo relativamente: si mette il gioco sugli store e si aspetta che il pubblico lo compri, ma con i modelli in abbonamento l'industria va in una direzione in cui **B2B e B2C confluiscono**, creando "questo nuovo essere che è il mercato del videogioco", da saper interpretare.

Anche questo è un messaggio importante: il lavoro del producer **nel 2022** è abbastanza diverso da quello di tre o quattro anni prima, e quasi certamente sarà molto diverso fra altri tre o quattro anni.

0.5.24 La gamification

La **gamification** è fondamentale sia perché **incentiva il lavoro dei team**, sia perché è perfettamente affine alla personalità del producer: "game producer è il nome, lo dice in sé stesso", deve essere una persona affine al gioco. È semplicemente il concetto di **osservare le attività quotidiane, soprattutto quelle lavorative, con gli occhi di un giocatore**. Chi si sente debole o poco esperto in questo ambito deve approfondire: procurarsi subito un testo di gamification, fare delle prove.

Il relatore chiede a chi è già in un piccolo studio se usa strategie di gamification. Risponde Luca Cassone[57], che ha studiato informatica e nell'ultimo anno ne ha visto qualcosa, "ma non tantissimo". Per il relatore la gamification è un altro di quegli argomenti diventati famosissimi, di cui "parlano tutti": da due o tre anni molte aziende fanno corsi interni "senza sapere minimamente alla fine di che cosa si tratta". Nel nostro ambito è molto utile; il problema è **chi se ne prende la responsabilità**. Forse andrebbe ideato un nuovo profilo, una persona aggiuntiva che crei **routine di gamification**. Siccome non è quasi per niente formalizzata negli studi, potrebbe farsene carico il producer, con un vantaggio in termini di **fidelizzazione del team**: riversare verso i colleghi il lavoro che fa verso i publisher, ricordando con qualche iniziativa di gamification **il valore del lavoro** che stanno facendo, perché vale la pena farlo, e rendendolo più divertente. Il producer non deve essere troppo tecnico, ma deve cogliere gli aspetti divertenti e di valore del progetto; se riesce a ricordarli ai colleghi, gliene saranno tutti grati.

Si aggiunge così alla comunicazione e alla partecipazione a qualche canale YouTube: chi vuole fare il producer dovrebbe impratichirsi **il prima possibile** con la gamification, ma **senza nozionismo**: "state lontani dai testi che cercano di classificare le cose". La gamification deve essere un'**attitudine**; qualcuno ce l'ha già senza saperlo, perché è il suo modo normale di affrontare la vita, chi non ce l'ha farebbe bene ad approfondirla.

0.5.25 Il panorama internazionale

L'ultima parte riguarda il **panorama italiano**, perché i partecipanti sono italiani e, salvo chi partirà "per la Svizzera", lavoreranno presumibilmente in aziende italiane, con partner e publisher italiani. Il relatore premette che il "qualunquismo" sull'Italia sempre indietro non gli interessa: parla dell'industria dei videogiochi. Ci sono differenze con quella internazionale? Sicuramente sì. Per capire quanto profonde, divide il mercato internazionale in tre grandi zone, in ordine di importanza "storica, non più tanto attuale":

Giappone. Non tanto l'Asia in generale, con le sue limitazioni legislative, ma il Giappone, che ha sempre prodotto **capolavori** sia hardware sia software ed è stato trainante. L'Europa viveva dell'importazione dei prodotti giapponesi (Sega, Nintendo).

[57] Il cognome è quello che si legge nella trascrizione. La sua risposta è in parte persa.

America. Ha sempre contrapposto una **produzione interna** (Atari, ma anche publisher come MicroProse), senza però raggiungere i picchi di qualità giapponesi.

Europa. "Pullula" di studi piccoli e medi. La maggior parte di quelli storici degli anni Novanta è scomparsa dopo varie acquisizioni, ma altri più recenti funzionano molto bene: per la **grande qualità del lavoro**, secondo il relatore superiore a quella americana ("tanti produttori europei sono veramente di altissimo livello"), e perché molte nazioni hanno capito l'importanza di questa industria e hanno investito in **aiuti alle aziende**, **finanziamenti** e soprattutto in una **rielaborazione della burocrazia**, creando infrastrutture nuove per facilitarne la crescita. Funziona bene in **Polonia**, abbastanza bene in **Spagna** e **Olanda**, ancora bene in **Irlanda**.

Oggi l'industria, ingigantita, è trainata da aziende che storicamente non hanno questa visione, in particolare **Microsoft**: a dettare le direzioni sono **produttori di servizi**, per i quali il videogioco è uno strumento, e che sono anche i più potenti finanziariamente. Basti pensare che **Netflix** produce videogiochi "a fondo perduto", pagando team di **200 persone** per fare prodotti per cellulare "assolutamente sconosciuti", perché può permettersi di non rientrare dei costi. Le dimensioni sono gigantesche: ogni tanto si sente ripetere, anche nei telegiornali, che l'industria del videogioco ha superato quella discografica e cinematografica, ma è successo **già dieci anni fa**, e "ha già doppiato" gli altri settori. Oggi è molto più orientata verso l'America, e il Giappone è molto meno forte.

0.5.25.1 Un buon momento, ma non facile

Per chi vuole lavorare nell'industria, anche come producer, lavorare in Europa è in generale **un buon momento**: c'è richiesta, ci sono opportunità, il mercato è molto grande. Secondo la stima che il relatore riporta ci sono indicativamente **3 miliardi di clienti potenziali** nel mondo, e ci sono stime che indicano il doppio[58]; bisogna poi capire che cosa cercano, se la maggior parte gioca su cellulare e così via, ma non è il tema del seminario.

Fiorente non significa che si apre uno studio e dopo un anno si sono risolti tutti i problemi finanziari. Anzi, nonostante la quantità di lavoro, è **più difficile oggi aprire uno studio di successo** che negli anni Novanta[59]. Nelle generazioni passate la qualità era più omogenea, mediamente più alta, e la risposta del pubblico di conseguenza: si facevano meno prodotti e quasi tutti avevano abbastanza successo, con le dovute eccezioni. **Oggi realizzare un buon prodotto non basta**, e il ruolo del producer diventa fondamentale.

0.5.26 Il "problema italiano"

Quando si parla dell'Italia si aggiungono problemi ulteriori: un conto è essere adatti al lavoro e capaci, un altro sono le persone, le aziende e le realtà con cui ci si confronta. Qualcuno potrebbe avere un successo immediato all'estero ed essere scoraggiato in Italia; è bene saperlo. "Non è questione di pessimismo, è questione di sapersi rimboccare le maniche", senza immaginare, come trent'anni fa con ingegneria, che un titolo di studio sistemi a vita.

0.5.26.1 Normativa, burocrazia, conduzione familiare

Normativa. Le cose si muovono lentamente, ma almeno c'è l'intenzione. Lo studio di videogiochi non è ancora identificato come realtà a sé stante rispetto ad altre realtà industriali: non ci sono **normative ad hoc** che agevolino l'iscrizione ai registri, le agevolazioni e così via.

Burocrazia. Probabilmente servirà un **commercialista** per stare dietro a una burocrazia "abbastanza avvilente", che non ha ragione di esistere: dovrebbe indirizzare nel lanciare l'azienda nel modo migliore, invece crea difficoltà.

[58] Nella trascrizione la fonte è indicata come "la SAE"; potrebbe trattarsi dell'ESA (Entertainment Software Association) o di un'altra associazione di settore. La cifra di circa tre miliardi di giocatori nel mondo è in linea con le stime correnti all'epoca.

[59] Prima di questa frase la trascrizione perde un pezzo ("continua ad assumere nuove persone. gigantesca").

Conduzione familiare. Questo modello poco evoluto fa sì che la grande maggioranza delle startup italiane già esistenti (non quelle dei partecipanti, "fresche, appena nate", con un approccio diverso) abbia il modello italiano della **conduzione familiare**: abbastanza improvvisate, restie ad aprirsi a contributi esterni; e l'accesso al lavoro in Italia tende a dipendere più dall'essere **persone di fiducia** di qualcuno che dall'essere molto competenti[60].

Che cosa significa? Lavorare molto sulla **propria rete**. **LinkedIn** è uno strumento molto utile, non l'unico, ma va usato. Bisogna cominciare a "lanciare relazioni", essere sempre **trasparenti**, attenti alle relazioni tra le persone che si conoscono, vigili per capire chi può aiutare e come si può aiutare chi già lavora nel settore.

In queste startup a impronta imprenditoriale italiana il **CEO** vuole spesso un controllo assoluto e, almeno all'inizio, **pretende di essere lui il game producer**. Di solito, "dopo un paio d'anni di disastri", capisce che bisogna assumere un producer che faccia il producer, "e lì magari interverrete voi". Bisogna saper valutare le situazioni, non per scappare (magari si prende subito la prima offerta), ma per farlo **in modo consapevole**: molte startup italiane sono problematiche e porranno in condizioni molto stressanti.

Diverso è sfruttare le opportunità delle istituzioni come l'accademia, dove chi vuole fare questo mestiere si incontra e certi legami si creano molto presto: la rete che si ha all'interno dell'accademia è **molto preziosa**, e non bisogna farsi scrupoli a chiedere a tutti[61].

0.5.26.2 Situazioni tipiche

Il relatore propone alcune situazioni che vive quotidianamente e che chiama "il problema italiano". Non dice che all'estero vada tutto liscio, "ci mancherebbe altro", ma ha lavorato anche con aziende europee e la differenza è evidente.

Scadenze non rispettate. È **sistematico**: in Italia si fanno le milestone e "non succede mai che vengano rispettate". Il producer non può far rispettare magicamente una milestone, ma sarà lui a gestire l'*impasse* in termini di negoziazione e comunicazione, con le difficoltà che nascono all'avvicinarsi della scadenza.

Pagamenti in ritardo. All'inverso: il publisher che finanzia lo studio deve pagare le milestone, e in Italia sembra normale pagare **due o tre mesi dopo** la scadenza. Per un piccolo studio alle prime armi può creare grossi problemi. Il producer può fare poco, se non un lavoro "certosino" di **comunicazione continua**, rinnovata **ogni due o tre giorni**, che crei un certo legame emotivo e porti il publisher a essere più responsabile.

Sparizioni prima delle scadenze. Un **segnale d'allarme**: il flusso di comunicazione tende a scomparire vicino a certe scadenze, come quelle dei pagamenti. Se quando è il momento di accettare e pagare una milestone l'interlocutore non risponde e, dopo due o tre tentativi, si scusa dicendo che era all'estero, "non va bene": l'azienda intende pagare in ritardo per sua convenienza.

Comunicazioni non sincere. "Questo è opinabile."

Comunicazioni verbali. Un problema molto grosso, secondo il relatore "addirittura solo italiano": interlocutori che, contattati, dicono "ci sentiamo al telefono", vogliono parlare a voce. "**Non si lavora così.**" Chi prende decisioni discutendo al telefono non è professionale, e all'estero è inconcepibile, perché ciò che è detto a voce non ha rilevanza. L'esempio: la persona con cui si è parlato si licenzia la settimana dopo; chi prende le consegne chiede "su che mail avete preso le decisioni, così mi allineo?"; "l'abbiamo detto a voce"; "arrivederci". Se gli altri si comportano così, bisogna **scrivere una mail**, forzare un po' la situazione e **insegnare agli altri a lavorare in modo professionale**. "Siamo nel 2022 e il telefono non lo usa quasi più nessuno": deve restare tutto scritto in modo chiaro.

[60] La frase nella trascrizione è spezzata ("tendito in Italia sia legata al fatto che voi siete persone di fiducia di qualcuno"); la ricostruzione è quella più probabile.

[61] Qui la trascrizione salta un pezzo ("Non fatevi scrupoli di chiedere a tutti. a cercare lavoro nelle realtà attuali che sono ancora un po' troppo lente"): probabilmente il relatore contrapponeva la rete dell'accademia al cercare lavoro soltanto nelle realtà esistenti, "ancora un po' troppo lente, troppo legate, troppo arretrate".

Risposte lente. C'è una regola non scritta, nata tradizionalmente in **Germania**: a una richiesta scritta è maleducazione attendere più di **24 ore** per rispondere (ci può essere un motivo, una vacanza), ed è **assolutamente intollerabile** non rispondere entro **48 ore**. Il relatore lo ha verificato con aziende tedesche, ma vale quasi ovunque all'estero: nella peggiore delle ipotesi ti rispondono che non hanno l'informazione e te la faranno avere, ma rispondono entro 48 ore. In Italia questo "tende a non succedere mai": si è spesso costretti a **riscrivere due o tre volte** la stessa richiesta.

Può sembrare poco grave o perfino normale, ma l'industria del videogioco "non l'abbiamo inventata noi": è internazionale. Se questi atteggiamenti si adottano verso un interlocutore estero, "non lavorate più", perché sono intollerabili. Non bisogna mai essere lenti a rispondere, e non deve mai succedere che un publisher estero chieda **due volte**, a distanza di due o tre giorni, la stessa cosa: se succede, **scusatevi immediatamente**. Imparare a lavorare come si lavora all'estero, anche restando in Italia, porta solo vantaggi: si viene individuati come persone particolarmente professionali, e molti diranno "meno male che ho a che fare con un producer che finalmente mi risponde".

Il motivo per cui molti non rispondono è che **non vogliono prendersi responsabilità**: un altro indizio che l'azienda partner non è ben strutturata o, peggio, ha una struttura tipicamente familiare. Sono critiche specifiche all'Italia: in Francia, dice il relatore, non le avrebbe citate, "magari qualche cos'altro avrei detto, ma molto differente". Siccome si è in Italia, vanno segnate, perché servono a diventare professionisti.

0.5.27 Ultimi consigli

Verso la fine, il relatore lascia alcune "piccole scintille" a cui pensare:

Si è giudicati sui risultati. Il producer è sempre giudicato in base ai risultati.

Ogni progetto poteva essere migliore. Anche concluso e di successo, probabilmente poteva essere migliorato: è utile ripensare proprio ai progetti che hanno funzionato e scoprire che si poteva fare qualcosa di meglio. È il **continuous learning**, non quello dei corsi di aggiornamento: **sui propri progetti già realizzati**.

Flessibilità nei due sensi. Il lavoro è stressante e gli orari sono molto flessibili, ma anche a proprio favore: è tipico considerare il producer **sempre disponibile**, quindi ci si può prendere la libertà di entrare a orari diversi in certi giorni o, se possibile, lavorare da casa per motivi propri. La flessibilità è congenita al ruolo: bisogna ottenerla **in entrambe le direzioni**.

Nessuno rende meglio sotto stress. Il relatore avrebbe ancora delle slide, ma chiude con una frase sentita spesso nella sua carriera, soprattutto ai colloqui: "rendo molto bene sotto stress, do il meglio di me". **Non è fisiologicamente possibile**. Ci sono persone che resistono meglio allo stress e lo gestiscono meglio, "ma nessuno dà il meglio di sé sotto stress, tutti rendono di meno". Siccome il lavoro del producer è stressante, bisogna saper **gestire i propri tempi** e prendersi delle pause per lavorare meglio[62].

0.5.28 Chiusura

Il relatore chiede se ci sono domande finali dopo "questo lungo *excursus*", consapevole che tutto, concentrato in poche ore, è una grande mole di informazioni; spera che alcuni spunti siano stati utili. "Forse vi ho tramortiti, non c'è più nessuno." "No, ci siamo, ci siamo." Ci si avvicina all'ora di cena, "bisogna riprendersi un pochettino".

L'organizzatore si occupa della chiusura del collegamento ("ci penso io, Walter, non ti preoccupare") e ricorda che alcuni partecipanti ritroveranno il relatore **giovedì 29** per la **seconda parte**: "c'è molto da dire". Walter ribadisce che non è semplice introdurre certi argomenti in modo così condensato, ma l'importante è che tutti siano motivati a fare un lavoro che "è un bel lavoro", ringrazia i partecipanti, in particolare

[62] La trascrizione perde qualche parola ("prendete poss a lavorare meglio"); la ricostruzione è la più probabile.

quelli che hanno fatto domande rendendo il tutto più interattivo, e saluta: "salute a tutti e buona serata". L'organizzatore lo ringrazia per il tempo e le ore passate con la classe[63].

Punti chiave del capitolo

- Il **game producer** è un ruolo **articolato e in divenire**, senza una guida che lo definisca: **ci si costruisce con l'esperienza**. È stressante, aperto a tutti, più facile per chi viene dal project management o dal business.
- In uno studio appena nato si scelgono con cura i profili: servono almeno un paio di **programmatori**, il game design andrebbe affidato a una persona dedicata, l'**arte** conviene averla internamente, l'audio di solito si acquista fuori. Crescere troppo può essere controproducente.
- Il producer **non è un game designer** e non "crea" il gioco: lo **indirizza** lungo il ciclo di vita. Il suo scopo principale è **creare il business**: capire potenzialità e pubblico del titolo, renderlo vendibile, fermare il lavoro se il prodotto è troppo di nicchia o fuori tempo.
- Il ruolo è nato con lo sviluppatore-imprenditore degli anni Ottanta, si è arricchito di marketing e gestione delle licenze negli anni Novanta e dopo il 2000 può comprendere tutte le IP dello studio.
- Miti da sfatare: **non abbiate paura**; l'inglese imperfetto non è un problema in un'industria casual e pratica; il **mobile**, saturo, non è più il punto di partenza consigliato. Scegliete l'ambiente in cui vi riconoscete come giocatori.
- Gli **investitori** vogliono controllo, garanzie e rischi bassi: IP "per sempre", per il pubblico più ampio, contenuti a pagamento, *game as a service*. Il producer deve presentare il gioco come originale e insieme in linea con il mercato, e avere il coraggio di frenare queste tendenze.
- L'**ultima parola** spetta al **game director**, responsabile della qualità; il producer traduce la qualità in vendite. Spesso i due ruoli coincidono, ma sarebbero entrambi necessari.
- Il producer è soprattutto un **product manager**: prepara e presenta il **pitch**, ma deve saper **andare oltre il pitch**, che gli interlocutori dimenticano o non studiano, puntando su uno o due punti sensibili. Il finanziamento è spesso negoziabile.
- Essere anche un po' **project manager** serve a non fare promesse sbagliate e a negoziare tempi e piattaforme con cognizione di causa.
- Il bravo producer è un **client manager**: costruisce con il publisher un rapporto di fiducia il più possibile paritario, gli chiede conto del marketing e trasforma in vantaggio perfino i ritardi.
- **Agile** è utile ma **non è una Bibbia**: nato per software e ufficio, fatica con artisti, orari creativi e smart working. Il **product owner** è il braccio destro del producer: il primo cura l'evoluzione del prodotto, il secondo la **roadmap**.
- Nel **Kanban** del producer si ragiona per concetti, e la **QA** è una quarta colonna **dopo il *done***, perché continua dopo il rilascio con il feedback degli utenti. Le demo crashano, e i publisher lo sanno.
- Dello **Scrum** (3 ruoli, 3 artefatti, 5 eventi, 5 valori) il producer usa poco e con parsimonia: niente sprint né daily, sì alla **sprint review**. Il miglior meeting dura **20 minuti**. Tra i valori, il più importante è il **rispetto**.
- La **visione** del progetto e dello studio è compito del producer; elementi distintivi chiari rendono più facile vendere. Imparate le metodologie, poi **costruitevi i vostri strumenti** (anche un Kanban personale, da rivoluzionare periodicamente).

[63] Nei saluti finali un partecipante dice anche "ci vediamo domenica": è probabilmente una svista o un errore di trascrizione, visto che l'appuntamento indicato è per giovedì.

- Il producer deve **restare un videogiocatore**, a qualunque età: se vende videogiochi senza amarli, si percepisce.
- Con i partner: distinguere **chi parla da chi scrive**, valutarne la serietà, rispettare la **legge di Murphy** (le cose che possono andare male sono sempre più di quelle previste), avere piani B e una visione a **2-3 anni**.
- Il producer è **la guida del team**, non la persona più importante: sta sul fronte del palco. Servono dedizione, comunicazione **scritta, continua e abbondante**, consistenza (mai parlare di cose che non esistono) e **umiltà con entusiasmo**.
- Nel **B2B** (licenze, commesse, co-development) il producer ha poca libertà; nel **B2C** ne ha di più, ma i due mondi stanno convergendo. La **gamification** è un'attitudine che il producer può portare nel team.
- In Europa è un buon momento, ma **un buon prodotto non basta**. In Italia pesano normativa, burocrazia e **conduzione familiare**; la **rete** (LinkedIn, accademia) è essenziale.
- Il "problema italiano": milestone non rispettate, pagamenti in ritardo, sparizioni vicino alle scadenze, **decisioni prese a voce**, risposte lente. All'estero si risponde **entro 48 ore** e non si fa chiedere due volte la stessa cosa: lavorate così anche in Italia.
- Si è giudicati sui risultati; fate **continuous learning** sui vostri progetti; pretendete flessibilità in entrambe le direzioni; **nessuno rende meglio sotto stress**.

Chapter 0.6

Il mindset del game designer: un workshop per scrittori e artisti

Questa sesta lezione zero non è una lezione del corso in senso stretto, ma la **prima giornata di un workshop** rivolto soprattutto a chi **non** fa il game designer: **scrittori, artisti e world builder**, e, con qualche adattamento, anche musicisti e programmatori. Il suo scopo non è insegnare il game design in poche ore ("per quello abbiamo dei corsi di tre anni"), ma trasmettere il **mindset del game designer**: il modo in cui un game designer ragiona, le parole che usa, gli strumenti minimi con cui lavora, così che persone con professionalità diverse riescano a **capirsi e collaborare in un team**. Il tema portante, dichiara il relatore, è la **comunicazione tra ruoli differenti**, che viene dalla sua esperienza personale in "parecchi progetti" dove la comunicazione interna a volte si inceppava[1].

A condurlo è Daniele[2], **game narrative designer** e **world builder**, affiancato da Michele, studente del terzo anno di Game Design che lo ha aiutato a progettare il workshop. Il workshop si svolge **interamente online, su Zoom**, ed è diviso in **due giornate di tre ore** ciascuna: la prima, quella di questo capitolo, è la parte "più discorsiva", in cui parla soprattutto il relatore; la seconda è la parte **pratica**, in cui i partecipanti modificheranno un gioco esistente (si veda la sezione 0.6.25). La giornata si svolge di mattina, con una pausa a metà e una lunga coda di domande, e si chiude all'ora di pranzo[3].

Tre avvertenze di lettura. La prima: il relatore dichiara apertamente che molte definizioni sono **sue**, pensate per spiegare il game design a chi non è un tecnico, e che il workshop è volutamente **denso e un po' confusionario**: "l'obiettivo era confondervi", perché il mindset del game designer è "un mindset di caos"; il capitolo conserva queste cautele e segnala quando una definizione è personale. La seconda: molte slide sono immagini, GIF o link che la trascrizione non riporta; dove il relatore si riferisce a ciò che mostra senza descriverlo, lo si dice. La terza: la trascrizione è automatica, storpia moltissimo i nomi propri e i termini inglesi (l'accademia, per esempio, diventa "Hive", "Ive", "Ivo", "Dive", "Ivy", "Aive", "AIVA"[4]), a volte ripete in loop la stessa parola e perde pezzi di frase; dove la ricostruzione non è sicura lo si segnala in nota. I riferimenti temporali vanno letti rispetto al momento della lezione, che si colloca **a novembre 2022**[5].

Il percorso del capitolo segue quello della lezione:

1. l'avvio, le presentazioni dei partecipanti e quella del relatore (sezioni 0.6.1 e 0.6.2);
2. la struttura del workshop e la bibliografia (sezione 0.6.3);

[1] La trascrizione dice che la comunicazione "sfregola e fa tritone": il senso, cioè che la comunicazione stride e non funziona, è chiaro; le parole esatte no.

[2] Il relatore non pronuncia mai il proprio nome per intero, ma viene chiamato "Daniele" all'inizio ("ok grazie Daniele") e si cita da solo immaginando l'obiezione di uno studente ("Daniele, ma che cosa stai dicendo?"). Il cognome non compare mai; nel capitolo lo si chiama semplicemente Daniele.

[3] La pausa è fissata "alle 45", cioè alle 11.45; alla fine il relatore saluta perché ha "la pasta in piatto".

[4] Si tratta sempre di AIV, l'Accademia Italiana Videogiochi. Analogamente "a Dai vi è vietata" va letto "in AIV è vietata".

[5] Il relatore dice "è brutto da dire nel 2022" e ricorda "tutto il mio 2021"; sta giocando *A Plague Tale: Requiem* (uscito il 18 ottobre 2022), di cui è a metà, e un partecipante sta giocando *God of War Ragnarök*, uscito il 9 novembre 2022; il corso online di narrative design dell'accademia è stato lanciato "quest'anno a settembre". Anche l'allusione a chi acquista un social network "perché è stato punto sul vivo" e alle "recenti notizie" di licenziamenti di massa rimanda all'autunno 2022. La datazione è una deduzione, non un'informazione esplicita.

3. i **problemi** che nascono quando scrittori, artisti e game designer lavorano insieme (sezione 0.6.4);
4. perché serve un **mindset**: i **bias** sul game designer e sul game design (sezione 0.6.5);
5. il game design come **metodologia** (sezione 0.6.6);
6. che cosa vuole un game designer: l'**esperienza** per un pubblico (sezione 0.6.7);
7. **idea** e **visione**, il gioco come **comunicazione differita**, i principi di design e l'iterazione (sezioni da 0.6.8 a 0.6.10);
8. l'**MDA** e l'ADMDA, il framework personale del relatore (sezione 0.6.11);
9. gli strumenti della prima fase: brainstorming, visualizzazione, pipeline, player type, **High Vision Document** e **one page design** (sezioni da 0.6.12 a 0.6.15);
10. le domande durante la pausa (sezione 0.6.16);
11. le **meccaniche**, le **dinamiche**, i **sistemi** e i **loop** (sezioni da 0.6.17 a 0.6.19);
12. le tre idee da portare a casa (sezione 0.6.20);
13. le **estetiche**: superfici, glossari, regole, icone e idiomi, emozioni e forme (sezioni da 0.6.21 a 0.6.24);
14. il workshop della seconda giornata, il riepilogo e le domande finali (sezioni da 0.6.25 a 0.6.28).

0.6.1 L'avvio e le presentazioni

La registrazione comincia mentre i partecipanti entrano "piano piano" nella videochiamata. Il relatore saluta, chiude "tutte le cose che potrebbero disturbare" (Telegram, per esempio) e fa una "sorta di check": conta undici persone collegate, ma ne mancano ancora cinque o sei. Nell'attesa chiede a ciascuno di presentarsi, dicendo il proprio nome, che cosa fa nella vita e se è **interno** all'accademia o **esterno**, e in particolare se tra i presenti ci sono dei game designer "imboccati per caso"[6]: il workshop infatti è "pensato per scrittori, artisti e world builder".

0.6.1.1 Chi partecipa

Le presentazioni danno un quadro molto vario[7]:

- **Andrea, Matteo e Luca**, di uno studio italiano di videogiochi indipendente[8]. Andrea è "l'imputato" (cioè l'addetto) alla scrittura, al world building, ai dialoghi e ai personaggi; i due colleghi lo seguono nel workshop per migliorare la qualità delle loro riunioni "in tema di storytelling". Più avanti si scoprirà che sono amici e parenti (si veda la sezione 0.6.27);
- **Flavio**, studente di game design dell'accademia, a Roma;
- **Alice**, esterna, che studia fondamenti di *visual development* in un'altra accademia[9]: aspirante artista, ma con una laurea in ingegneria informatica; nella vita si occupa di tutt'altro, ma è appassionata e vorrebbe entrare nel settore;
- **Jacopo**, studente del primo anno di game design a Roma;
- **Francesco Celata**, studente di game design dell'accademia a Roma ("sentivo un'altra voce molto profonda");

[6] Così nella trascrizione; dal contesto il senso è "infiltrati", parola che il relatore userà più volte scherzando sui game designer presenti.

[7] I nomi sono quelli che si leggono nella trascrizione, che li storpia spesso; dove compaiono più versioni si è scelta la più plausibile. Alcuni partecipanti scrivono solo in chat e vengono presentati dal relatore.

[8] Il nome dello studio è reso in tre modi diversi ("Bindai Studio", "BitNine", "3bit9"); non è possibile stabilire quello corretto.

[9] Nella trascrizione "IDE Academy": potrebbe trattarsi dello IED o di un'altra scuola; il nome non è certo.

- **Giulia Del Vecchio**, esterna all'accademia ma "interna al game development": lavora come **concept artist** per un'azienda italiana di videogiochi e si è occupata anche della scrittura del gioco; vuole approfondire il game design per entrare di più nel ruolo di *narrative designer*;
- **Claudio Dallai**, studente del primo anno di programmazione dell'accademia: è lì perché nei prossimi anni ci saranno collaborazioni tra i team e vuole un'infarinatura generale. Il relatore lo avverte che il discorso non è molto centrato sui programmatori, ma che alcune parti possono interessarlo;
- un secondo **Francesco**, interno all'accademia, che studia musica ma è molto interessato alla scrittura e alla narrazione nei videogiochi. Il relatore spiega che il workshop è "molto tarato sull'arte e sulla scrittura", ma "con i dovuti accomodamenti" si può seguire anche venendo da un altro ambito: è mirato a chi non fa game design;
- **Giovanni**, esterno, che scrive in chat: è *designer* e illustratore, tutor didattico all'ISIA di Roma, dove si occupa di design di sistemi ("caspita, complimenti");
- **Luca Brunetti**, studente di game design dell'accademia a Roma, a cui si sono appena rotte le cuffie;
- **Marco**, esterno: aveva provato a iscriversi al corso triennale di game design ma all'ultimo non c'era più posto; la sua ragazza, che studia grafica all'accademia, gli ha regalato il pacchetto di corsi ("che bel regalo");
- **Michele**, studente del terzo anno di Game Design, che in realtà è "un infiltrato": ha aiutato a progettare il workshop e farà da collaboratore per entrambe le giornate;
- **Filippo**[10], studente del primo anno di game design a Roma. Qui il relatore fa un breve inciso, non del tutto chiaro nella trascrizione, sul fatto che a Milano è il primo anno di un indirizzo legato alla narrativa[11].

In tutto i partecipanti sono "metà interni, più o meno, metà esterni". Il relatore chiede se lo sentono tutti bene e, dato che nessun altro deve presentarsi, comincia.

0.6.2 Chi è il relatore

Daniele spiega subito perché tiene questo workshop: **viene da un altro ambiente**. "Nasce scrittore" e oggi è **game narrative designer** e **world builder**. Lavora molto sul *tabletop*, cioè sul **gioco di ruolo** e sul **gioco da tavolo**: ha all'attivo un paio di pubblicazioni e un paio di campagne Kickstarter nel mondo del gioco di ruolo da tavolo (GDR), non solo di grandi dimensioni ma anche "GDR più piccolini"[12]. È un mondo "un po' di nicchia, un po' nascosto in Italia, ma molto molto florido, molto molto attivo".

Il suo percorso, ricostruito lungo la mattinata:

- ha studiato **lettere moderne**, indirizzo cinema, all'Università di Pavia, e si è poi specializzato con un **master in scrittura e marketing**, in *storytelling* "puro": come comunicare un marchio attraverso le storie, l'*employer branding* interno ed esterno all'azienda;
- per anni è stato **impaginatore di riviste** e **grafico** ("tuttora a volte impagino"), ha fatto il *layout artist* e varie attività di grafica, anche online;
- è stato a lungo **blogger** e, negli ultimi anni, anche **streamer** su Twitch;
- negli ultimi tre anni ha lavorato soprattutto come **freelance**, come *corporate storyteller*, collaborando con diverse aziende italiane. Ora è in un **periodo sabbatico**, "perché dopo tre anni a rincorrere i clienti" deve "tirare un po' il fiato": da adesso si occupa solo dei progetti ludici dell'accademia;

[10] Nella trascrizione "Filippo Gagliavi"; il cognome è probabilmente storpiato.

[11] La frase è corrotta ("Mi chiedo se ero a Milano perché è il primo anno a Milano in segno, quindi narrativa di design"): forse il relatore chiedeva se Filippo fosse di Milano, dove è appena partito un percorso di narrative design.

[12] La trascrizione dice "non solo da necessario": forse "non solo di grande formato"; il senso è che ha lavorato anche su giochi di ruolo di piccole dimensioni.

- insegna in accademia, in particolare nel corso di scrittura multimediale, e a settembre è stato lanciato il primo anno di un **nuovo corso online di narrative design**, con "un ottimo riscontro da parte degli studenti". Tiene la lezione da cui è nato il workshop anche come ospite in associazioni della zona di Pavia e di Milano.

Nelle slide si trovano i suoi contatti e quelli di Michele. Michele si presenta a sua volta: da quest'anno lavora come **game designer** presso una startup americana che si occupa di *e-learning*, cioè di far studiare i ragazzi attraverso i videogiochi[13]; spera di finire l'ultimo anno e poi trovare lavoro in uno studio. Il relatore sottolinea che, come si vede, già dal secondo o terzo anno gli studenti iniziano le prime collaborazioni.

0.6.2.1 Perché questo workshop

Daniele insiste su un punto: non è nato game designer né creatore di giochi di ruolo; anzi, fino a qualche anno fa pensava che vivere creando giochi fosse "una grande illusione". Quando ha iniziato a studiare il game design partiva come scrittore e come grafico. Nel suo lavoro di *corporate storyteller* si tenta di fare *branding* e operazioni di comunicazione del marchio, ma "molto spesso le aziende italiane sono un po' indietro": manca ancora il sito, manca il marketing, mancano le basi necessarie per fare davvero *corporate branding*. Per questo si è scontrato spesso con la realtà tipica di chi fa un mestiere creativo (artisti, musicisti, scrittori) e ha sentito più volte l'adagio "tanto mio cugino lo fa meglio".

Scontrandosi con questo mondo ha dovuto **imparare sul campo**, anche in un ambiente, quello del gioco di ruolo e del gioco da tavolo, che pensava fosse più professionalizzato. Ancora quest'anno si è reso conto che ci sono "delle enormi aree" di quei settori ancora **poco professionalizzate** ("non mi riferisco a nessuno in particolare, però l'esperienza ha voluto che lo scoprissi sulla mia pelle"). Da qui l'idea che serva qualcosa in cui un narrative designer dovrebbe eccellere: **migliorare la comunicazione**. In accademia si parla molto di *soft skill*, comunicazione e lavoro in team, "e quanto sono vere quelle parole".

0.6.3 La struttura del workshop

In sei ore, ammette il relatore, non può "aprire il cervello di un game designer" ("non posso prendere Michele, aprire il suo cervello e spiegarvi il cervello di Michele"), ma può spiegare un po' il suo **mindset**. Il workshop è diviso in **tre parti**:

1. una sorta di *talk* sul **mindset**, mirata "esplicitamente ed esclusivamente" ad artisti e scrittori, ma adattata anche a musicisti, programmatori e a chiunque non faccia il game designer, con quello che il relatore ha appreso sul campo lavorando con i game designer;

2. l'**esplorazione di alcuni concetti del game design**, presentati nel modo "più visivo e visuale possibile", con parti più esplicative e parti più tecniche;

3. il **workshop** vero e proprio.

Chi è già game designer non deve preoccuparsi: il workshop è pensato per chi non lo è, ma nasce proprio dall'esperienza del relatore con chi lo è. Alcuni argomenti saranno già noti o verranno rivisti a lezione, ma, "lo dico da insegnante", è sempre bello e utile **sentire gli stessi argomenti spiegati da più persone**, cosa dimostrata anche da chi studia l'apprendimento. Il consiglio è di portarsi a casa il modo in cui lui spiega le cose, anche quando "può non suonarvi", farlo proprio e prenderlo come spunto di riflessione; più avanti si parlerà anche di *bias* e di provocazione.

La prima giornata, "le due ore e mezza che ci rimangono", sarà dedicata alla spiegazione; la seconda al workshop. Il workshop consiste in un **hack**, cioè una **modifica**: "hackerare il regolamento" di un gioco esistente partendo da un *concept* proprio. Si potrà scegliere fra tre giochi:

- **Lasers & Feelings**[14], un gioco di ruolo *one page*, che "sta in una pagina";
- **Ticket to Ride**, un gioco da tavolo "con i trenini";

[13] Nella trascrizione "Alter Learning"; il nome della startup non è verificabile.
[14] Nella trascrizione "Laser and Feelings": è il gioco di ruolo *one page* di John Harper.

- **Super Mario Bros.**, "uno dei videogame per eccellenza e anche uno dei più semplici".

Non è detto che tutti gli argomenti previsti vengano coperti: le slide saranno inviate a tutti, e ciò che non si farà in tempo a vedere si potrà leggere lì. Molte slide sono "molto verbose": il relatore cercherà di saltarle e di dirle a voce.

0.6.3.1 La bibliografia

La slide successiva mostra una bibliografia selezionata[15]. Per chi viene da ambiti diversi dal game design il relatore ha pochi libri da consigliare, ma tre in particolare:

- per gli **artisti**, *Interactive Stories and Video Game Art* di **Chris Solarski**, che si occupa moltissimo del rapporto tra game design e arte;
- per gli **scrittori**, *Dramatic Storytelling and Narrative Design*[16], una guida ai videogiochi e al *transmedia* per scrittori;
- per chi è interessato al **world building**, *Worldbuilding for Fantasy Fans and Authors*[17], "un libro che può sorprendere" ("non dico niente, è un libro sorprendente"). Il relatore pensa che il world builder del gruppo sia Luca; è invece Andrea.

0.6.4 Scrittori, artisti e game designer: i problemi

La *talk* parte dal rapporto tra **scrittori, artisti e game designer**: "una bella gatta da pelare". Il relatore ha lavorato in team composti da tutte e tre le figure, e lo scopo del workshop è fornire il mindset per collaborare. Non si aspetta che i partecipanti ricordino tutto ("mi raccomando, non è questo l'obiettivo"), ma che entrino loro in testa alcuni termini che potrebbero sentire o usare, e qualche piccolo strumento di design per fare l'*hack* e capire come "mischiare" la propria arte nel processo. Il tentativo è tradurre in termini **metaforici e generalisti** concetti complicati o *tricky* del game design.

Si immagini, dice, di avviare un nuovo progetto. Si ha un'idea; di solito le idee nascono un po' all'improvviso[18], e si può avere un team alle spalle. La prima cosa da capire è che **il game design di solito si fa in team**: un team che "prende armi e bagagli" e parte verso delle idee. Chi non è game designer può trovarsi circondato da game designer senza capire di che cosa parlano ("trust me, ci sono passato"); chi è game designer può parlare di cose che per lui sono normalissime e vedere nella stanza, o nella stanza virtuale, le facce "molto enigmatiche" di artisti e scrittori. I problemi tipici sono questi.

Un linguaggio che va tradotto. I game designer a volte parlano una lingua "molto strana", che richiede traduzione. Ma vale per tutti: anche il game designer, sentendo parlare un artista o uno scrittore, si chiede "ma di cosa stanno realmente parlando?"[19].

Un *brief* nebuloso. Chi collabora ai contenuti del game design riceve dal game designer un *brief*, cioè l'insieme delle informazioni e dei compiti (*task*) da svolgere, che può sembrare nebuloso. Non è necessariamente colpa del game designer, che magari è tecnicissimo, preciso e puntuale: usa però un **gergo tecnico** che agli altri manca, oppure **termini identici che significano cose diverse**. L'esempio principale è la parola **conflitto**: nel game design significa una cosa, nel narrative design un'altra, "motivo per cui di solito in narrative design preferiamo utilizzare *tensione drammatica*".

Documenti che nessuno usa. Si possono produrre documenti bellissimi, "cesellati", che poi sviluppatori e game designer non usano né guardano. Viene da chiedersi: dovevo farli? Ce n'era bisogno? Bisogna capire dove sta il problema: nella documentazione creata, o nel fatto che non era utile e non andava bene.

[15] La slide contiene più titoli di quelli citati a voce; la trascrizione riporta solo quelli che il relatore commenta.
[16] Probabilmente *Dramatic Storytelling & Narrative Design: A Writer's Guide to Video Games and Transmedia* di Ross Berger; l'autore non viene nominato.
[17] Nella trascrizione "Warbuilding for Fantasy Fans and Author"; l'autore non viene nominato.
[18] La trascrizione dice "di solito le idee nascono come fratello", espressione non ricostruibile con certezza.
[19] Qui il relatore apre una parentesi sulle slide: in ogni slide che cita un gioco il nome del gioco è nell'angolo; se un'immagine non riporta un nome, non viene da un gioco.

Un piano che si dimentica. Non dovrebbe accadere, e di solito non accade nelle grandi aziende, nelle società e nei team veterani ("quando lavoro con delle aziende importanti di solito non ho questo problema"), ma capita, soprattutto con persone alle prime armi: si stabilisce un piano d'azione e poi lo si dimentica completamente, e ognuno va nella direzione che vuole.

0.6.4.1 Il caos non è una strategia

Al relatore viene spesso obiettato che dal caos, dall'uscire dal seminato, nascono i prodotti migliori: ci sono giochi fantastici, come **BioShock**, nati in situazioni in cui i piani non sono stati del tutto seguiti. Vero, risponde, ma **con grandi sacrifici**: sacrificando gran parte dell'idea originale e "anche magari un po' di sanità mentale". Chi legge la storia della creazione di BioShock trova grossi problemi interpersonali, *crunch*, litigi. E soprattutto, **se non si segue un piano, a volte si fallisce**: il prodotto arriva alla fine e non funziona, o non è quello che si pensava, e tutto ciò che è stato prodotto non ha l'effetto sperato.

Questo dipende anche dalla natura del game design: si lavora con **concetti** e con **sistemi mentali** che servono a controllare che cosa il giocatore o la giocatrice sente e prova, le emozioni che ha. Quando manca, o è lacunosa, la parte di **visualizzazione**, cioè la creazione di qualcosa di visivo che permetta di capire dove si sta andando e se si è sui binari, spesso si va **out of scope**[20]: si insegue qualcosa per la semplice volontà di inseguirlo, e il progetto deraglia.

0.6.4.2 Out of scope e out of the box

Un po' di *out of scope* ha senso nelle **prime fasi** di un progetto, quando si raccolgono le idee: nel *brainstorming* è del tutto normale, per sviluppatori, game designer, artisti visivi e scrittori, andare fuori tema e pensare *out of the box*, anche nel *problem solving*. Ma le due cose vanno distinte con cura, e il relatore dichiara che la distinzione è **sua** ("è tutta mia, lo dico"):

Out of scope. È qualcosa di **completamente distante dalla propria visione e dal proprio obiettivo**, "di solito abbastanza grande". È una soluzione che **non va bene**.

Out of the box. È quando, durante il design, si ha un problema e non si trova una soluzione perché si è troppo focalizzati su di esso: allora bisogna cercare la soluzione **fuori dai propri processi mentali**. È lì che ha senso introdurre creatività nel team, sfruttando i membri "più esterni" alla visione canonica del game design. Vale anche nella scrittura: quando il relatore è bloccato, studia cose molto lontane dalla sua esperienza. È una soluzione **perfetta**.

Più tardi, rispondendo a una domanda (si veda la sezione 0.6.27), ne darà un esempio: se il personaggio non riesce a interagire bene con l'ambiente o a sparare bene, e viene un'idea geniale che sistema tutto, quella è *out of the box*; se l'idea per sistemare tutto è "ragazzi, cambiamo, facciamo il gioco in un altro mondo narrativo", quella è *out of scope*, e se ogni problema si risolve così "il gioco non lo fai più".

0.6.4.3 I problemi di oratoria

C'è poi un problema "molto retorico, molto di oratoria", che il relatore sperimenta anche a lezione: si dice una cosa nel team e si ottiene una reazione che non corrisponde, perché retoricamente si tende a mettere nelle parole di una persona **qualcosa in più di quello che ha detto**. A lui succede ogni volta che afferma che una narrazione ha successo perché "una grande narrazione è una storia giusta per il pubblico giusto, non una grande storia fatta benissimo": tutti gli rispondono che sta parlando di *fanservice* e che ne è un "estremo difensore". Sono cose completamente diverse; c'è uno spettro, e questi problemi di oratoria creano problemi a cascata.

0.6.4.4 Un mindset che fa da argine

Per arrivare all'idea con un team di più persone, con più sensibilità e "più idee sulle idee", bisogna considerare che siamo molto complessi: il relatore cita Norman Holland, studioso di neurocognizione, secondo cui gli

[20] La trascrizione scrive quasi sempre "autoscope" o "autoscopio"; più avanti il relatore stesso corregge: "out of scope, fuori dallo scope, fuori dall'obiettivo, quindi non *autoscope*, ma *out* scritto *out*".

esseri umani sono gli unici capaci di avere **idee di stati mentali**[21]. Per non perdersi sul cammino ("vedete che qua si può andare ovunque") e avere la certezza che le idee raggiunte siano quelle di partenza serve un **forte mindset**: un insieme di strumenti e di tecniche che **mantengano la rotta senza irrigidire**. "Attenzione, e questo è il punto più sensibile del discorso": non è un mindset che irrigidisce i processi e spunta la creatività, ma servono **degli argini**.

È la ragione per cui il relatore è affezionato a una definizione di **Diego Ricchiuti**, direttore del corso di game design: il game design è **50% arte e 50% scienza**. C'è un elemento artistico e, "uso una parola brutta", **soggettivo**, perché narrando e creando non si può evitare di mettere sé stessi nell'opera. Ma se ci si basa esclusivamente sulla propria sensibilità, senza processi, pattern mentali, un mindset, non si arriva all'idea. Con il mindset si può dire: anche se non il 90%, ma l'80% o il 70% delle idee sviluppate erano quelle originali, è comunque una buona percentuale, e soprattutto sono idee che hanno avuto sul pubblico **l'effetto sperato**.

0.6.5 Perché serve un mindset: i bias

I game designer hanno bisogno di un mindset; ma perché ne hanno bisogno anche gli altri, e perché il workshop è incentrato su questo? Per svariate ragioni. La prima è che il mindset permette di **superare i bias**. Molti dei problemi elencati prima "non sono la realtà dei fatti, sono bias", e i bias tendono a diventare **ciclici**, a diventare **spirali**: comincio a sospettare, ad avere un pregiudizio verso il team e verso i game designer, e invece di scioglierlo comunicando, non comunico, e il pregiudizio peggiora.

0.6.5.1 I pregiudizi sul game designer

I pregiudizi sui game designer sono tantissimi, e molti accomunano gli artisti in generale:

- **"Sì, ma di lavoro che cosa fai?"**. Al relatore è capitato spessissimo, come scrittore ma anche come game designer, in fiera: "Creo giochi da tavolo." "Ah, ok, no, ma di lavoro esattamente cosa fai?" "Quello, è quello il mio lavoro." Succede molto anche a scrittori e artisti.

- **"Crei giochi, quindi giochi"**. Per creare giochi bisogna certo giocare, fare esperienza ed essere aggiornati ("e che fatica: io sto ancora finendo *A Plague Tale: Requiem* e non so quando riuscirò a giocare altro"), ma non è solo quello: come un regista va a vedere i film, ma non diventa regista o esperto di sceneggiatura guardando film.

- **"Sei un designer, quindi disegni"**. Questo "mi fa sempre ridere, ma esiste, Michele lo sa bene". Saper disegnare è una capacità utilissima per chi deve visualizzare delle idee (un'idea che non si riesce a visualizzare è difficile da implementare), ma **non è essenziale** essere ottimi disegnatori.

- **"Non stai facendo niente"**. I capi pensano spesso che non si stia lavorando, perché il game designer, come gli scrittori e gli artisti ("io ho un passato da grafico"), a volte deve fissare un'immagine per trenta minuti: non perde tempo, cerca il bandolo della matassa, perché ragiona con **sistemi veramente complessi**.

- **Il game designer fa "miliardi di cose"**. Il relatore cita Giulia, concept artist finita anche a scrivere il gioco: qualcosa di simile succede ai game designer. Fino a poco tempo fa, quando la professione del narrative designer non era strutturata come "negli ultimi mesi, non sto parlando degli ultimi anni, sto parlando degli ultimi mesi", i dialoghi, in assenza di un *game writer*, venivano affidati al game designer che all'università aveva sostenuto un esame di scrittura.

[21] Così nella trascrizione ("gli uomini sono gli unici a poter avere idee di stati mentali"). Norman N. Holland è stato un critico letterario che ha applicato psicoanalisi e scienze cognitive alla lettura; il concetto evocato è vicino a quello di *teoria della mente*. L'attribuzione è riportata come la dà il relatore.

0.6.5.2 I pregiudizi sul game design

I bias non riguardano solo i game designer, ma anche **il game design in sé**. Il relatore, che ha collaborato con molte aziende e progetti, ha scritto un proprio gioco di ruolo e ha tradotto molto, vede questo pregiudizio soprattutto nel gioco di ruolo da tavolo e, "purtroppo mi sento di dire", anche nel gioco da tavolo: l'idea che il game design **non sia una disciplina utile**. Chi studia game design in accademia penserà "Daniele, ma che cosa stai dicendo?", eppure è così. Molte persone che iniziano in modo amatoriale non entrano nell'ottica di studiare. Fino a qualche anno fa, in fiera, vedeva tantissime persone arrivare con il proprio gioco, di solito da tavolo; consigliava loro di studiare un po' di game design e la risposta era: "Ma tanto io gioco, a cosa mi serve studiare game design?". C'è il bias che il game design sia qualcosa di inutile, fatto di processi mentali che complicano la questione.

Il relatore ha lavorato molto con artisti, scrittori e musicisti. Per esempio **Le Notti di Nibiru** è uno dei pochi prodotti GDR ad avere una propria *soundtrack*, composta da un'azienda italiana[22] e disponibile su Spotify: la si è voluta perché molti nel team erano musicisti o amavano la musica. Collaborare è sempre molto difficile, e anche qui capire il mindset, il modo in cui le persone riflettono e pensano, è fondamentale.

0.6.6 Il game design come metodologia

Prima di proseguire, una breve definizione di che cos'è il game design. Senza voler essere "petulante, pedante, paternalistico", il relatore prende un insieme di definizioni dal libro **21st Century Game Design** di **Bateman**[23]. A lui è particolarmente cara la definizione di game design come **metodologia**. In italiano la parola ha una distinzione importante da *metodo*: la metodologia è **la parte di una disciplina che permette di scegliere il metodo con cui si opera**. E il game design "non è nient'altro che una metodologia di sviluppo": una delle cose principali che un game designer deve fare è **scegliere il metodo e gli strumenti con cui sviluppare un'idea**. Dei quattro punti della slide è questo "la parte che mi porto più a casa".

È importante anche perché qui entrano l'aspetto artistico e quello della scrittura. Per sviluppare giochi servono competenze di game design; ma essendo il game design una metodologia che usa spesso **metodi multidisciplinari**, non è raro che un gioco nasca prendendo, "rubando", traendo ispirazione da metodi che non hanno nulla a che fare con il game design. L'esempio è **Gris**, che ha preso moltissimo dagli studi sul **dolore** e sul processo che una persona attraversa di fronte al lutto, gli **stadi del dolore** (*stages of grief*). Gli sviluppatori hanno fatto esattamente ciò che dice la parola metodologia: hanno scelto un metodo che, accanto ai metodi del game design, prevedeva lo studio della psicologia, e hanno usato gli stadi del dolore come *pattern*, come *design principle* ("direbbe Truby, poi lo vedremo"), come colonna portante, spina dorsale del gioco.

0.6.6.1 Le competenze "tangenziali"

Qui entra in gioco la capacità di ciascuno, che venga dalla programmazione, dalla musica o dall'arte. Il relatore e Michele hanno un collega game designer che ha fatto **tre anni di geologia**: nel world building, e non solo, qualche competenza di geologia può tornare utile. "Ovviamente non vi sto dicendo: andate fuori e studiate geologia, attenzione". Ma, da buon *storyteller* e da *coach* di storytelling, il consiglio è di **non sottovalutare le piccole competenze e i piccoli studi** che sembrano tangenziali e inutili per lo sviluppo di un gioco: possono tornare utilissimi. Quante persone sono state spinte a diventare game designer e a creare il proprio prodotto da un'esperienza vissuta, magari anche molto tragica.

0.6.6.2 Che cos'è il mindset

Il **mindset** che il relatore cercherà di trasmettere è l'insieme di tre cose:

Pensiero critico. Cioè **analisi e scelta del metodo di sviluppo**.

[22] Il nome dell'autore della colonna sonora è reso "l'Obsus Freudiano" e il relatore stesso dice di non ricordarlo bene; potrebbe essere qualcosa come "Lapsus Freudiano", ma non è verificabile.

[23] *21st Century Game Design* di Chris Bateman e Richard Boon (2006). La slide mostra quattro punti, che la trascrizione non riporta: il relatore commenta solo quello sulla metodologia.

Capacità espressiva. La parte d'arte: non "il 50% di arte", ma la capacità di **esprimere delle idee** e arrivare a un'esperienza di gioco.

Attitudine al *problem solving*. Si ha una visione e bisogna arrivarci: tra il punto di partenza e la visione resa vera, prodotta, implementata nel gioco vero e proprio, **ci saranno dei problemi**, e bisognerà affrontarli tenendo ben chiara la visione e senza perdersi "nel labirinto del design".

Il mindset è ciò che permette di **stare al di sopra della metodologia**. In poche ore il relatore non può dare procedure, metodologie e metodi ("per quello abbiamo dei corsi di tre anni"), ma può partire dal mindset e arrivare fino ad alcune **tecniche**.

0.6.7 Che cosa vuole un game designer: l'esperienza

Che cosa vuole un game designer da un gioco? Qual è l'obiettivo che vuole ottenere alla fine di uno sviluppo? Alcuni dicono una parola "che molti di voi sapranno che in AIV è vietata": **divertimento**, una parola "un po' strana, ambigua". Altri dicono i soldi, il pubblico, i fan, la fama, oppure "siamo masochisti, vogliamo farci del male. A volte anche quello". In realtà ciò che si vuole è **un'esperienza per un pubblico**.

C'è anche l'elemento dei **soldi**, perché si sta parlando di **sviluppo ludico editoriale**, non di un prodotto fatto per la propria volontà[24]. Anche dietro molti sviluppatori indipendenti di videogiochi e giochi da tavolo c'è un aspetto economico e di business, e qui il relatore vede "un altro bias enorme": essere schizzinosi e titubanti sulla questione monetaria[25]. No: **servono soldi per fare i giochi**, serve un team, servono investimenti. Ogni gioco ha il suo prezzo ed è giusto che sia così, come c'è un prezzo sulla copertina di un libro e come gli *artwork* dovrebbero essere acquistati, soprattutto se usati per produrre altri prodotti.

L'obiettivo resta **dare a un pubblico un'esperienza** tale da creare emozioni, da avere un effetto nella vita del giocatore o della giocatrice.

0.6.7.1 La lente dell'esperienza

L'esperienza è il cardine, "la prima delle lenti" di **Jesse Schell**[26], "uno dei più grandi e famosi teorici del game design", ma anche game designer: ha fondato una propria azienda di sviluppo e ha un passato in Disney[27]. Il suo **The Art of Game Design** è ritenuto la "Bibbia" del game design: un **libro di lenti**, dove una lente è la capacità del designer di analizzare un gioco attraverso alcune sue caratteristiche, guardandolo per esempio attraverso il concetto di comunità o di giocatore, cioè mettendosi nei panni delle persone che ruotano attorno a un prodotto. La prima di queste lenti, "o comunque una delle prime", è proprio l'**esperienza**[28].

L'esperienza ludica è ciò che fa **appassionare** i giocatori a una storia, a dei personaggi, a un prodotto, a una IP, a un marchio, e li fa **tornare** più volte a giocare, avendo un'esperienza costante in quel mondo narrativo. È, in qualche modo, la ricompensa "più morale, più intellettuale, più emotiva" del game designer: vedere che il proprio gioco fa passare ai giocatori due o tre ore "di divertimento, direbbero alcuni". Il relatore non ama questa parola, perché l'esperienza ludica può essere molte cose: "divertimento" è una **spia** che indica uno degli infiniti piaceri che si possono provare giocando. Non è sbagliato dire che un giocatore si diverte; è solo che è un termine **molto povero**.

0.6.7.2 Strumenti, tecnologie, ricerca e feedback

L'esperienza ludica non può esistere senza due cose:

[24] La trascrizione dice "un prodotto de core": forse "di cuore", cioè un progetto personale fatto per passione.
[25] Nella trascrizione "essere un po' picchi"; il senso è quello di un certo pudore o snobismo verso il denaro.
[26] Nella trascrizione "Jesse Shell" e "Gershyshell".
[27] Il relatore dice "in Pixar, in Disney". Schell ha lavorato alla Disney (in particolare per i parchi a tema e i progetti di realtà virtuale) prima di fondare Schell Games; non risulta un suo passato in Pixar.
[28] Nel libro di Schell la lente numero 1 è in effetti la *Lens of Emotion* nella terza edizione e la *Lens of Essential Experience* nelle precedenti; la cautela del relatore ("o comunque una delle prime") è giustificata.

Strumenti. Non in senso tecnologico: ciò che nella scrittura è l'**arco di trasformazione** del personaggio o il *plot*; in programmazione non il codice, ma **il concetto di codice**; nella musica non il software o lo strumento, ma **l'idea della musica**, la melodia.

Tecnologie. Senza tecnologia non c'è esperienza ludica, soprattutto nei videogiochi, ma anche nei giochi da tavolo e di ruolo: senza il manuale di *Dungeons & Dragons*, senza la scatola e le componenti del gioco da tavolo, non si gioca.

L'esperienza nasce da un'idea, da una visione, ma non arriva al giocatore "semplicemente perché mi sveglio un giorno con un'idea": servono **ricerca e sperimentazione** sull'idea. Non si può prendere la prima idea, darla ai giocatori e vedere che cosa succede. Nel game design c'è un forte elemento di **feedback**: è una di quelle arti ("io le chiamo arti, ovviamente, come la scrittura") in cui il feedback conta molto più che nelle altre. Anche nel cinema esistono gli *screen test*, proiezioni fatte molto prima dell'uscita in sala per vedere come reagisce il pubblico. Ma prima del feedback dei giocatori vengono ricerca e sperimentazione, che tutti conoscono nelle proprie discipline: anche scrittori e musicisti le fanno, magari "in maniera un po' più inconscia"[29], senza mettersi a scrivere documenti in modo scientifico e preciso.

A questo punto a Daniele si chiude improvvisamente Zoom ("mi sentite? perdonatemi"); rientrato, riprende: bisogna trovare "l'incastro corretto" per creare l'esperienza, e il mindset aiuta a farlo. Tolta la tecnologia, che in poche ore non si può insegnare, il mindset permette di capire alcune tecniche di design di base e le informazioni basilari per progettare bene un prodotto. "Non mi aspetto che voi diventiate game designer dopo queste sei ore"[30], ma che capiate un po' come i game designer lavorano, pensano e riflettono.

0.6.8 Idea e visione

Si torna alla "pagina bianca": un'idea per un nuovo gioco, un team, delle idee e, insieme all'idea, una **visione**. Il relatore fa un esempio: un videogioco ambientato in un mondo **completamente sotterraneo**, perché la superficie del pianeta è stata distrutta; si interpreta un personaggio "alla Super Mario" che, invece di andare dall'alto verso il basso, deve trovare il modo di **ripopolare** il pianeta, una sorta di **terraformazione** partendo dal basso ("così c'è anche il messaggio ecologico"). È un modo di ribaltare il concetto di *Super Mario Bros.* Questa è un'idea, ma è anche una visione:

L'idea è ciò che sta **alla base** del prodotto: "quella che tu hai".

La visione è **dove vogliamo andare**: "quella che tu vuoi raggiungere, vuoi ottenere".

Le due cose possono non coincidere al cento per cento: la visione può discostarsi lievemente dall'idea, e durante lo sviluppo ci si può spostare un po'. Ma è la visione che deve guidare. Di solito la visione corrisponde a un'**esperienza desiderata**, quella che si spera che il giocatore o la giocatrice abbiano quando prendono in mano il gioco e lo provano. Se quell'esperienza fallisce, il prodotto non funziona: l'idea e la visione non sono state confermate dall'esperienza. Questo, secondo il relatore, non è lontano da ciò che fanno tradizionalmente scrittori, registi, artisti, programmatori e musicisti.

0.6.8.1 L'obiettivo superordinato

Al relatore piace descrivere la visione con un termine del business, preso dal **modello delle 7S** di **Waterman**[31]. In un'azienda ci sono la **strategia**, la **struttura**, le **abilità**, lo **staff**, il **sistema** con cui la struttura si muove, lo **stile**; ma tutto è volto a raggiungere un **obiettivo superordinato**: un obiettivo a cui tutti tendono, che non è il *goal*, il KPI, l'obiettivo del trimestre o dei prossimi quattro mesi, e nemmeno necessariamente un obiettivo di business. Può essere anche **moralmente ed eticamente sconnesso** dal business, positivo o negativo: per esempio acquisire un social network "semplicemente perché sei stato un pochettino punto sul vivo" è comunque una visione, un obiettivo superordinato[32]. L'idea di acquisire quel

[29] Nella trascrizione "in coscia"; probabilmente "inconscia" o "incosciente".
[30] La trascrizione dice "non mi aspetto che voi poi dimentiate game designer": dal contesto, "diventiate".
[31] Il modello delle 7S (*strategy*, *structure*, *systems*, *shared values* o *superordinate goals*, *style*, *staff*, *skills*) è stato elaborato alla fine degli anni Settanta da Robert Waterman e Tom Peters, con altri, presso McKinsey.
[32] L'allusione, senza nomi, è con ogni probabilità all'acquisto di Twitter da parte di Elon Musk, concluso a fine ottobre 2022.

social network per ribaltarlo e renderlo migliore è invece un po' diversa dalla visione: "c'è un po' un distinguo da fare".

0.6.8.2 Non tutti i giochi sono per tutti

Se si fa tutto correttamente, anche se il gioco cambia, anche se c'è un elemento artistico e creativo e tra le idee e la visione iniziale c'è una minima divergenza ("ripeto, 50% scienza, 50% arte"), il game design come metodologia fa da argine e tiene in una certa direzione: si arriva a un'esperienza *rewarding*, appagante, per il giocatore o la giocatrice (qui, nella slide, "un giocatore un po' particolare: un piccolo gattino"), che è esattamente ciò che si voleva ottenere.

Ci sono però casi in cui questo non accade: due giocatori di fronte allo stesso prodotto lo vivono in modo completamente diverso. Qui entra un altro elemento "molto pregiudizievole", soprattutto nel mondo dei giochi da tavolo, dove solo quest'anno il relatore comincia a vedere un cambiamento: il fatto che **non tutti i giochi sono per tutti i giocatori**. Un prodotto ha probabilmente là fuori un **pubblico ideale**. Non è lontano dall'esperienza dello scrittore: in narratologia c'è il concetto di **lettore ideale**. E **Umberto Eco** ricordava che tutto comunica, e quindi tutto può significare tutto per tutti[33]. Per questo una delle attività principali di un artista, e il designer non fa eccezione, è cercare di **evitare la decodifica aberrante**: non si può impedire che alcuni fruiscano il prodotto in modo diverso da altri, ma si può **ridurre** il numero di chi lo decodifica in modo completamente distante dalla propria visione. L'ideale è che chi non è il pubblico giusto, alla fine, dica: "Ok, non è il mio tipo di prodotto, ma tecnicamente è ben fatto, è perfetto, semplicemente non fa per me". Questo, per il relatore, è il concetto di *player type* applicato alla comunicazione[34].

0.6.9 Il gioco come comunicazione differita

Il gioco, infatti, **è una comunicazione**: produrre un gioco è una comunicazione tra un team di game design e un giocatore. Ed è una comunicazione **differita**[35]: chi ha creato il gioco **non è nella stanza** con il giocatore o la giocatrice. È un "grosso, grosso, enorme problema". In un romanzo, in un film, in una serie, non serve essere nella stanza perché **non c'è interazione**: c'è una possibile decodifica aberrante ("uno può vedere un film e capire l'uomo aperto"[36]), il pubblico è attivo, partecipa alle vicende, ma non ha molta libertà. Nel gioco la struttura tipica della comunicazione (un messaggio che cerca di arrivare a un destinatario, disturbato dal **rumore**, cioè dalle distrazioni) si complica perché c'è anche l'**interazione**.

0.6.9.1 La definizione di gioco del relatore

Ne deriva la definizione personale del relatore. Il gioco è una **comunicazione differita** tra un designer, o un team di designer, e un pubblico di giocatori. E che cos'è un gioco?

> La **ricerca volontaria di piaceri ed emozioni**, agendo con le **possibilità e le restrizioni** di un **sistema arbitrario dotato di significato**.

La slide riporta altre cinque definizioni "molto interessanti", alcune già viste in accademia[37]. Ce ne sono di più precise e ridotte, come quella di **Sid Meier**: "un buon gioco è una serie di scelte interessanti", forse la più stretta e corretta che si possa trovare. Ma, come dice Schell, nessuna definizione è volta a definire esattamente che cos'è un gioco: ognuna è **una lente** per osservarne un aspetto. Nella sua, il relatore ha voluto inserire la comunicazione differita: è uno **sforzo comunicativo**, perché si vuole comunicare un'esperienza ludica, portare una visione ai giocatori e alle giocatrici, esprimere delle idee.

Il problema è che non si è lì con loro: bisogna veicolare messaggi e soprattutto **gestire il modo in cui interagiscono** con il gioco senza esserci, agendo attraverso il prodotto. Se non si è attenti, ciò che si

[33] La trascrizione è incerta ("diceva che nulla può, tutto comunica"). Il riferimento è alla semiotica di Eco e al concetto, citato subito dopo, di *decodifica aberrante*, elaborato da Eco negli anni Sessanta.

[34] Nella trascrizione "il concetto di player tag, di comunicazione"; dal seguito si tratta probabilmente di *player type*.

[35] La trascrizione scrive sempre "indifferita"; dal contesto ("una comunicazione in cui noi non siamo lì nella stanza con il giocatore") il termine è "differita". Più avanti compare infatti anche "una comunicazione differita".

[36] Frase corrotta nella trascrizione; il senso è che uno spettatore può fraintendere un film.

[37] La trascrizione dice "alcuni sterredive, alcuni terredive": forse "alcune le vedrete in AIV". Le altre definizioni della slide non sono lette.

aveva come visione può arrivare alla fine del processo "non esattamente come l'avevamo ordinato". La slide mostra due giochi con un'idea simile, ambientati sott'acqua e con creature "un po' umanoidi": **Subnautica** e **Seaman**[38], "due prodotti completamente differenti". L'obiettivo è creare un prodotto vincente con la giusta esperienza, e per questo servono **principi di design**: strumenti, teorie e tecniche che permettano di arrivare dove si vuole.

0.6.10 Design principle, pilastri e iterazione

Il concetto è molto simile al **design principle** di **John Truby**, scrittore ed esperto di sceneggiatura, autore di *The Anatomy of Story*[39]. Il *design principle* è il principio che sta alla base di molte storie. Per esempio, nelle storie legate al concetto di **isola** (*Jurassic Park*, *Shutter Island*, *Lost*, ma anche isole "ipotetiche" come quella di *Shining*) ci sono tre possibilità:

1. si va sull'isola, si vivono le vicende e se ne esce **cambiati**, verso un altro luogo, un'altra dimensione, un altro status;
2. si va sull'isola, si vive un'esperienza e si **torna a casa** cambiati, ma nella dimensione che si conosceva;
3. si va sull'isola e si **rimane bloccati** lì.

Il *design principle* di solito non è uno solo: è un'idea, un insieme di idee, una serie di strumenti che aiutano lo scrittore a scrivere una storia. Truby fa anche l'esempio del **Padrino**, il cui *design principle* prende la storia classica, "un po' delle fiabe", del personaggio agli inizi, un po' innocente, e mostra come venga travolto da un mondo che non conosceva.

0.6.10.1 I pilastri

Può sembrare lontano dai videogiochi, ma anche i videogiochi hanno principi simili, che alcuni chiamano **pilastri** (*pillars*)[40]. Un esempio famosissimo, ricordato più volte dal team di sviluppo, è la recente **dilogia di God of War** ("che poi avrà sicuramente dei sequel"), che ha tre pilastri: **combattimento**, **rapporto padre e figlio** ed **esplorazione**. Sotto la visione stanno principi, pilastri, modi per non perderla. E la visione qual è? Un **Kratos** non più adirato, pieno magari di rimorsi e rimpianti, più avanti negli anni, **più umanizzato**: per questo il pilastro padre-figlio è importante per comunicare quelle emozioni. Ecco come il discorso si ricompone: **esperienza**, **emozioni** che il pubblico vuole provare, **pilastri** e principi che aiutano a raggiungerle.

0.6.10.2 Iterare

La parte delle tecnologie resta fuori dal workshop, ma il relatore promette gli strumenti minimi per fare un po' di design, soprattutto strumenti che permettano di fare **molte iterazioni**: partire da un'idea "un po' Seaman" e lentamente arrivare a un'idea "un po' Subnautica", un gioco che ha avuto un enorme successo di *community*. Come? Iterando, con tante modifiche e implementazioni, "altro punto su cui ci sono tantissimi bias".

La slide mostra visivamente un processo di sviluppo iterativo: un progetto partito da un'idea vaghissima, fare "un Killzone un po' diverso", perché **Guerrilla** veniva dall'esperienza della saga di *Killzone*. All'inizio (nell'angolo in alto a destra) l'idea era un Killzone con nemici **dinosauri robotici**; poi si è sviluppata fino a diventare **Horizon**. Prototipi, *concept* e lavoro iterativo hanno permesso di creare un titolo "assolutamente eccezionale", pur con i suoi difetti: sia *Horizon Zero Dawn* sia *Forbidden West* non ne sono esenti, ma hanno il loro pubblico e la loro esperienza arriva. In alcuni punti l'illusione, la **sospensione dell'incredulità** direbbe uno scrittore, è rotta da problemi tecnici: in *Forbidden West*, se ci si perde in una missione, la protagonista continua a ripetere "dobbiamo andare di là, dobbiamo andare di là". A parte questi dettagli, è un prodotto di successo.

[38] Nella trascrizione "simen": quasi certamente *Seaman* (Sega, 1999), il gioco per Dreamcast in cui si alleva una creatura acquatica dal volto umano.
[39] Nella trascrizione "The Natural View Story".
[40] Nella trascrizione "Pindars".

0.6.10.3 Che cosa portarsi a casa

Si vedranno tantissime cose: tecniche, strumenti, *pipeline*. Il relatore non si aspetta che i partecipanti le portino tutte a casa, ma che ne prendano **terminologia, punti di vista, visione**: "rubate quello che vi sembra più interessante". Si rende conto che ci sono molti più game designer di quanto si aspettasse ("infiltrati, eh, dite la verità: siete venuti a vedere come lavorano gli scrittori e gli artisti"). L'invito è ad avvicinare il workshop alla propria esperienza e a mettere al servizio dei game designer i propri principi di design, i propri **pattern mentali** di artisti, scrittori, programmatori, musicisti. Il game design è una metodologia, a volte molto multidisciplinare: quanti giochi sono basati su mitologie, su idee musicali, perfino sulla programmazione (ci sono interi *serious game* per imparare a programmare).

Il relatore chiede se ci sono domande ("vi ho confuso per bene?"); nessuno si lamenta. In chat si può scrivere o alzare il pollice con le icone di Zoom.

0.6.11 L'MDA e l'ADMDA

Per raccontare il game design in sei ore anche il relatore ha bisogno di un *design principle* che regga l'intero discorso: un *pattern*, un paradigma, come le storie di isole hanno il paradigma della freccia, della spirale e della freccia che esce, o il Padrino quello delle fiabe e dell'*underdog*. Parlando con Michele nei giorni precedenti, si sono accorti che il paradigma migliore è l'**MDA**[41].

L'MDA è un **framework di design**, "il più famoso", che descrive il diverso modo in cui **giocatori** e **game designer** si avvicinano al prodotto ludico, dividendolo in tre componenti un po' *tricky* da definire: **meccaniche**, **dinamiche** ed **estetiche**. È stato formalizzato da tre esperti del settore, **Hunicke, LeBlanc e Zubek**[42]; LeBlanc ha elaborato anche la teoria degli **otto tipi di *fun***, di divertimento ("andatevele a leggere"). L'MDA è la raffigurazione mentale di un processo; il relatore ne dà **definizioni proprie**, pensate per chi non è tecnico. Altri parlano di emozioni e divertimento per le estetiche, di *behavior* e processi per le dinamiche, di *code* e *rules* per le meccaniche; ma anche qui le parole sono ambigue: che cosa si intende davvero per divertimento, comportamento, processo, codice, regola?

0.6.11.1 Le definizioni del relatore

Estetiche. Il giocatore parte dalle **superfici di gioco**. Il termine "superficie" viene dal postmoderno, e "superficie non vuol dire superficiale, attenzione": è il quadro, il testo, la superficie astratta dove qualcuno interagisce con qualcosa ("potete chiamarla anche interfaccia, se volete, non mi offendo"). Le estetiche sono **l'insieme delle emozioni e dei piaceri evocati dalle sensazioni che un giocatore prova interagendo con le superfici di gioco**, non solo con l'interfaccia.

Dinamiche. Sono la seconda cosa a cui pensa il giocatore. Sono **scelte**: il giocatore si trova di solito di fronte a un bivio[43], e sono scelte "non da fare a cuor leggero", che **cambiano lo stato del gioco**.

Meccaniche. Solo per ultime il giocatore le analizza, e a volte non ne ha nemmeno percezione. Il relatore le descrive come **vincoli e rinforzi**: cose che **obbligano, vietano o consentono** al giocatore l'interazione con lo stato di gioco.

0.6.11.2 Le due direzioni

Il **giocatore** va da destra verso sinistra (estetiche, dinamiche, meccaniche); il **designer**, di solito, da sinistra verso destra. Ma, attenzione: il designer non parte dalla cosa più fredda, le meccaniche, nel vuoto. Ha una visione, delle idee, un'**ipotesi di estetica**, cioè l'esperienza voluta e desiderata, non ancora l'estetica finale prodotta. Dopo aver stabilito la visione e avuto delle idee, cerca le **meccaniche** che supportano l'esperienza

[41] Prima di introdurlo, il relatore torna indietro per chiarire la distinzione tra *out of scope* e *out of the box*, che aveva dato per scontata: la si è riportata nella sezione 0.6.4.2. Chiude la parentesi con una frase spezzata dalla trascrizione: "se avete domande che non hanno risposte, la risposta è sempre fatta e ric[erca]".

[42] Nella trascrizione "Hannichel, Leblanc e Zubek": Robin Hunicke, Marc LeBlanc e Robert Zubek, autori dell'articolo *MDA: A Formal Approach to Game Design and Game Research* (2004).

[43] Nella trascrizione "un bibio".

voluta; queste creano **dinamiche**, a volte **indipendenti dal designer**, perché creare meccaniche ha ricadute non sempre prevedibili; infine meccaniche e dinamiche hanno bisogno di essere **comunicate** attraverso uno stile, le estetiche.

Il framework è stato espanso in vari modi (la slide ne riassume alcuni, "non è necessario per oggi"), anche includendo la **narrazione**[44]. Al relatore piace, ma, pur essendo narratore e narrative designer, sostiene una cosa che fa venire "un infarto" a molti narratori: **gameplay is king**. In un gioco il gameplay è la cosa più importante: **play** conta più di *tell*, più di *show*, più del raccontare. Narrazione, tecnologia e arte non è che non esistano: sono **sotto traccia**, il che non vuol dire che siano meno importanti, ma che sostengono i processi di design. A volte sono protagoniste, soprattutto nei giochi *story driven* e *plot driven*, ma non sono le principali protagoniste.

0.6.11.3 Il caso di The Last of Us

Luca obietta in chat che *The Last of Us* sembra fare l'opposto. Il relatore chiarisce che intendeva *story driven*, e che è un bias: anche in *The Last of Us* il gameplay è importantissimo, perché senza gameplay "sarebbe un film". Che il gameplay stia allo stesso livello della storia e che la storia sia "sotterranea" non significa che sia inferiore: è lo stesso bias che molti hanno sulla parola *subplot*. Il *subplot* non è meno importante del *plot*: è sotterraneo, non si vede. In *The Last of Us* la storia è molto visibile, portata alla ribalta; ma senza gameplay *The Last of Us* non sarebbe *The Last of Us*.

0.6.11.4 L'ADMDA

Prima delle meccaniche, si è detto, ci sono una visione e delle analisi da fare. Per questo il relatore ha sviluppato un **suo MDA**, che parte "un po' da prima" e che chiama **ADMDA**. Dal punto di vista del game designer, dice, un gioco è una **funzione**, un'equazione, un **algoritmo**, "artistico, non scientifico", che non parte dalle meccaniche ma dall'analisi e dall'estetica:

A, analisi più estetica. La ricerca e la codifica delle emozioni, delle intenzioni e delle sensazioni che evocano la visione che si vuole dare allo sviluppo.

D, sviluppo (*development*). La parte delle dinamiche di game design, il secondo passo. È **solo del game designer**: il giocatore non la fruirà mai. Come il giocatore fa scelte quando gioca, lo sviluppatore deve fare scelte quando sviluppa, per portare una determinata esperienza a una **target audience**. "Non si può parlare a tutti, è molto difficile parlare a tutti": bisogna parlare a giocatori che capiranno quell'esperienza e ne avranno piacere.

MDA. Solo dopo si arriva a tutta la struttura dell'MDA.

Il framework stabilisce anche un'altra cosa di cui molti non si rendono conto: **play** e **game** sono due cose diverse. Il *play* è la **sessione di gioco**, il giocatore che effettivamente gioca; il *game* è il **prodotto gioco**. Entrambi sono "totalmente inferiori", cioè più piccoli, rispetto al *development*: il mondo sviluppato nel *world building* è più grande del mondo fruito; l'aspetto narrativo, che nel narrative design non si ferma al *plot* e ai personaggi, è molto più grande di ciò che si vede nel gioco e nel *play*. Lo stesso vale per il game design: come giocatori, a volte non ci si accorge nemmeno delle meccaniche e delle dinamiche.

Come in una formula, nell'ADMDA la **moltiplicazione** di analisi più estetica e sviluppo produce la **documentazione**, "croce e delizia del game designer"[45]. La documentazione deve essere un **supporto concreto, visivo e compatto** dell'esperienza di gioco: "non un supporto visivo tipo 300, 2.300 pagine". Una parte della documentazione deve certo essere corposa, soprattutto quella sui **contenuti di gioco**; ma non bisogna cadere nel tranello: i documenti sui contenuti non sono la documentazione di game design, né la documentazione di design in generale. Sono documentazione di contenuti. Quando si parla di documentazione, si intendono i documenti che **danno concretezza alla visione**: team, idee, visioni che a volte sono difficili da raggiungere hanno bisogno di **supporti visibili**, percepibili, sempre presenti, anche nella fase di *brainstorming*.

[44] Il relatore cita un framework "MTDA più N", forse una variante dell'MDA che aggiunge la tecnologia e la narrazione (in letteratura esistono per esempio estensioni chiamate MDA+N o DPE, *Design, Play, Experience*); il nome esatto non è ricostruibile.

[45] Nella trascrizione "croce edilizia", evidente storpiatura di "croce e delizia".

0.6.12 Gli strumenti della prima fase

Da qui il relatore percorre i passi dell'ADMDA. Il primo, l'analisi delle estetiche, consiste nell'**analisi, elaborazione e sviluppo di documenti che permettono di visualizzare i processi di design**. Servono:

- **strumenti di brainstorming**, per non andare *out of scope*;
- **strumenti di visualizzazione**, per vedere dove si sta andando e fare "prove mentali" del game design;
- un **documento che mantenga la visione generale** del prodotto, il cosiddetto **High Vision Document**[46] (si veda la sezione 0.6.15);
- il **one page design**, per essere il più compatti e concentrati possibile. Attenzione: non è un design ridotto a una singola pagina, ma **un design costruito attorno a singole pagine**, più pagine, in ciascuna delle quali tutte le informazioni su un aspetto sono contenute. Ogni pagina è un aspetto, un *layer* del prodotto. Nei prodotti molto piccoli, come quelli che si modificheranno nel workshop, il *one page* è davvero una pagina sola.

0.6.12.1 Gli strumenti di brainstorming e di visualizzazione

Il relatore non li definisce tutti ("andiamo un po' veloci"): qui entrano l'esperienza e la capacità artistica di ciascuno, che può portare i propri strumenti di brainstorming, di coerenza, di orientamento. Ne cita diversi:

Canvas. Modelli preimpostati che aiutano a fare brainstorming in modo corretto, come il canvas indicato nella slide con un link[47].

Mind mapping. Le mappe mentali.

Crazy eight. Uno strumento del *design thinking*, tra quelli che il relatore usa di più. Quando si vogliono produrre molte idee, si dà a ogni membro del team un foglio diviso in otto, e ognuno deve produrre **otto idee, disegnandole** ("attenzione, perché questa è una delle problematiche del *crazy eight*", cioè il fatto che bisogna disegnare). Se servirà nel workshop, lo spiegherà meglio.

Plus* e *delta. Uno strumento per gestire le persone: alla fine di una riunione si attaccano post-it nella parte *plus* (ciò che è andato benissimo) e nella parte *delta*, il triangolo (ciò che bisogna cambiare). Non solo nel prodotto, ma "soprattutto nel modo di sviluppare, nel modo di fare idee".

Fibonacci vote. Carte con i numeri di Fibonacci (il relatore usa di solito quelle dal 3 al 13: 3, 5, 8, 13), che servono a far **spiegare le persone**. Uno dei problemi maggiori in un team è che due persone parlano di un argomento con la sensazione di parlare della stessa cosa, mentre stanno parlando di cose completamente diverse. Quando si valuta quanto è pesante un'attività, o quanto è importante una componente del gioco, votare con la scala di Fibonacci invece che da 1 a 5 **fa emergere le divergenze** e permette di essere più granulari. Se si chiede quanto sarà pesante il *character design* del nuovo gioco e una persona dice 3 e un'altra 13, bisogna chiedersi perché: "è un modo per far parlare le persone".

Flowchart. Grandissimo uso dei diagrammi di flusso e delle mappe mentali, ma soprattutto dei *flowchart*, che vengono dalla programmazione. Mettere subito un'idea di gioco in un *flowchart* **rompe l'idea stessa**: si vedono persone che dicono "nella mia mente era così chiaro", e quando devono tradurlo in decisioni che l'utente deve compiere scoprono che è più difficile.

Lavagne e raccolte visive. Ci sono "miliardi" di strumenti: Milanote, Miro ("uso tantissimo Miro"), Pinterest.

[46] Il nome del documento è reso in moltissimi modi ("Ike Vision Document", "iGVision", "iVision", "Light Vision", "Like Vision", "AIG Vision", "One Vision"); la forma più chiara, che compare più volte verso la fine, è "High Vision Document". Lo si chiama così in tutto il capitolo.

[47] Nella trascrizione "il JGC canvas"; il nome esatto non è ricostruibile.

Stylescape. Una tecnica avanzata, per chi è più legato all'arte, di **The Futur**, la società di **Chris Do**[48].

Strumenti dal marketing. Si possono portare nel team strumenti di altri ambiti, come la **matrice di posizionamento**, che viene dal marketing e dal *design thinking*.

"Non abbiate timore", conclude, a portare il proprio modo di organizzare le idee nel team di sviluppo.

0.6.13 La pipeline e il micromanagement

Il brainstorming porterà a chiedersi quali sono le **scelte** e i **passi** dello sviluppo: che cosa viene prima e che cosa dopo, se si sviluppa prima il personaggio o il *level design*. Non è così immediato. Da **Assassin's Creed Origins** in poi, Ubisoft ha cominciato a sviluppare prima l'**intero world building**, l'intera struttura geografica dell'area esplorabile, e solo dopo i *quest designer* hanno disegnato le missioni: così potevano trarre ispirazione dai luoghi più importanti della storia dell'epoca scelta. A volte si pensa che la *pipeline* sia di un certo tipo, e poi si scopre che **ogni progetto ha le sue necessità**. Ancora una volta il game design è una metodologia: alcune cose resteranno standard, "come si è sempre fatto", ma bisogna non cadere nella trappola del "come si è sempre fatto". Se il progetto ha bisogno di pensare *out of the box*, è lì che bisogna "tirare fuori le idee dal cappello".

Le scelte e le ricadute del design si guardano nella **pipeline**, che si può riassumere in alcune domande da farsi:

- Che cosa viene prima? Che cosa viene dopo?
- Chi fa che cosa?
- Ciò che sto facendo io quale altro processo va a intaccare?
- Se io ritardo, chi altro ritarda?

Molto noto è il caso di **Cyberpunk 2077**: CD Projekt Red ha annunciato pubblicamente, con un video "più pensato per gli investitori che per i fan della saga", che i prossimi giochi passeranno dalla metodologia **waterfall** (a cascata, in cui ogni attività può partire solo quando è conclusa quella da cui dipende) a una metodologia più **agile**. Per il relatore l'aspetto migliore dell'*agile* è che **evita il micromanagement**: "ragazzi, il micromanagement è un incubo". Evitare il micromanagement è "uno dei mantra" del workshop.

Nel mindset dei game designer si noterà quanto si ragiona su che cosa viene prima e dopo e su quali sono le **priorità**: si fa uno **scrum meeting** per stabilirle, e nel team dovrebbe esserci uno **scrum master**, una persona deputata a capire come rendere agile lo sviluppo e adeguarlo alle necessità del team, "altrimenti non si va da nessuna parte". La slide riporta il link a uno degli articoli più completi e precisi sulla *pipeline* di sviluppo; nel workshop però si lavorerà su giochi minimali, senza una vera *pipeline*: ciò che il relatore si aspetta è che i partecipanti **ragionino su quelle domande**.

0.6.13.1 Che cos'è il micromanagement

Il relatore chiede chi abbia avuto esperienza di micromanagement aziendale ("alzi la mano, metta un commento in chat, così facciamo una piccola pausa"). Giulia chiede se si intenda la gestione di materiali o di inventario. "No, no, no": è una cosa **più psicologica**. Non va confuso con il *management* nel senso di produzione: è quando una persona **gestisce produzioni e processi nel minimo dettaglio**, al punto da **non lasciare libertà di gestione** alle persone del team, focalizzandosi sul controllo di ogni singolo aspetto, di ogni persona, di ogni attività. Di solito sono i capi, i *lead*, a lasciare poco spazio, creando cortocircuiti in cui ci si "spacca il capello in due" su piccole cose che diventano enormi. Nasce un ambiente in cui **non si può essere proattivi**, e nello sviluppo dei giochi essere proattivi è importantissimo.

Sul termine, il relatore precisa: si dice anche *micromanaging people*; il verbo è *to micromanage*, il gerundio *micromanaging*, il sostantivo *micromanagement*. Si rende conto che c'è una parola di business che significa

[48] Nella trascrizione "The Future" e "Chris Du". Il relatore scherza sul cognome: "mai cognome fu più azzeccato per un designer, Chris Do, Chris fa cose".

un'altra cosa[49], ma si può usare il termine che si preferisce. Il micromanagement **danneggia molto anche la produttività**: lui ha lavorato su progetti in cui c'era, "ed era veramente un incubo". "Chiaro, Giulia?" "Chiarissimo."

0.6.14 Il giocatore e i player type

Fin qui si sono visti il team e la visione; manca **il giocatore**. Il team procede verso la visione, ma a un certo punto "spunta fuori come un NPC" un giocatore che dice: "Ehi, ci sono anch'io, tenetemi in considerazione". Lo sviluppo non può prescindere dalla comprensione del **player type**, il tipo di giocatore a cui ci si rivolge. Ogni visione, ogni idea, ha là fuori dei lettori tipo, dei giocatori tipo, persone a cui quel prodotto dice molto.

Di solito i game designer dividono i giocatori nelle quattro categorie della **tassonomia di Bartle**[50]:

Killer. Legati al concetto di **agire** sugli altri giocatori; "non pensate che siano assassini in senso esplicito". Sono legati alle emozioni più forti, all'eccitazione[51].

Achiever. Più legati al **mondo**, al **completamento** delle cose.

Explorer. Legati al mondo, ma in modo **interattivo**, esplorativo.

Socializer. Legati agli **altri giocatori** e all'interazione.

Al relatore non interessa che si imparino i player type, anche perché ci sono "tantissimi modi" di farli. **Diego Ricchiuti** ha creato un modello chiamato **Prismatic Model**, non ancora pubblico ("ma fra un po' sarà pubblico, fortunatamente"), che riassume molti di questi modelli: il modello di **Quantic Foundry**, il **Temperament Sorter di Keirsey**, i tipi di gioco di **Caillois** (che in realtà analizzano i giochi, ma soprattutto il comportamento che suscitano nei giocatori), quello di **Lazzaro**[52]. Ce ne sono tantissimi.

Ciò che conta è che, nella parte operativa, ci si chieda **chi è il giocatore**: chi gioca questo tipo di giochi, com'è fatto. Anche qui ciascuno può portare i propri strumenti, purché simili a quelli mostrati. Nel libro *Worldbuilding for Fantasy Fans and Authors*, che il relatore ha con sé, una delle appendici propone una versione dei **reader type**, cioè dei tipi di lettore, basata su Bartle: pensata per i lettori, secondo lui funziona anche per analizzare i giocatori ("poi eventualmente ve la faccio vedere a video", perché nella slide l'immagine è poco leggibile).

0.6.15 L'High Vision Document e il one page design

A questo punto la visione del team può diventare *blurry*, sfocata: per questo è importante l'**High Vision Document**. È un **documento informativo** che deve essere **visibile da tutti in qualsiasi momento** e che contiene tutti gli **elementi core** del gioco. Sono i membri del team a stabilire quali siano; il relatore suggerisce:

- **tema** o **pilastri**;
- l'**obiettivo del gioco**, "che, attenzione, è una delle cose che maggiormente ci si dimentica di fare": ci si perde in un bicchier d'acqua, e l'obiettivo del gioco non c'è;
- **personaggi**, **regole**, **setting**, **narrativa**: tutto ciò che deve sostenere lo sviluppo e la visione.

Può contenere anche informazioni sul **pubblico**, sulle **emozioni** che si vogliono generare, sullo **stile** e l'**atmosfera**, elementi di marketing come l'**unique selling proposition** o *unique selling point* (l'elemento che rende il gioco talmente unico che i giocatori non lo associano ad altri: vedendo un'immagine pensano subito a quel gioco), elementi di design come le cose che **bisogna avere** e quelle che **si potrebbero avere** e piacciono (si veda la sezione 0.6.16), il setting, gli extra narrativi, le estetiche. L'indicazione è di mettere tutto in un **unico documento**.

[49] Il relatore non dice quale; chiede anzi di essere corretto "se sbaglio, chi ne sa qualcosa nell'ambito", temendo un "lapsus freudiano". Il significato che dà è comunque quello corrente del termine.

[50] Nella trascrizione "Bartol".

[51] La trascrizione dice "alle emozioni più al trailer": il senso è probabilmente quello di emozioni forti, da *thriller*.

[52] Nella trascrizione "Quantifoundry", "Imperament Sorter di Kersey", "Callois", "Lazaro". Per Lazzaro si tratta presumibilmente dei *Four Keys to Fun* di Nicole Lazzaro.

0.6.15.1 Il one page design

L'idea, "non l'ho inventata io", viene da una "splendida" *talk* che si trova nella GDC Vault, tenuta da **Stone Librande**[53], "famosissimo e bravissimo" game designer, oggi in Riot Games: il **one page design**, cioè il tentativo di **riassumere tutte le informazioni basilari di un layer del gioco in un'unica pagina**. Per giochi piccoli una pagina basta; per giochi grossi si fanno più pagine, ma **non sequenziali**: sono **correlate come le pagine di un wiki** e hanno senso anche da sole. Si può fare il *one page* del personaggio, quello del *level design*, quello delle meccaniche, "il one page di X, il one page di Y e il one page di Z".

Arrivati qui si fa la pausa: "ci sono tante informazioni, vado velocissimo"; alla fine della giornata ci sarà tempo per le domande, e il relatore cerca di tenere oggi tutta la parte discorsiva e domani quella operativa. Dieci minuti di pausa, ripresa alle 11.45; ne prende un po' meno per sé, così chi vuole fare domande può restare collegato dopo che lui avrà bevuto qualcosa e si sarà sgranchito le gambe.

0.6.16 Domande durante la pausa

0.6.16.1 Da dove si comincia un High Vision Document?

Un partecipante[54] sta scrivendo un High Vision Document per un suo gioco e chiede da dove partire. Il relatore riprende l'inizio: qual è l'obiettivo di un designer? Raggiungere una certa **esperienza di gioco**. Quindi bisogna fare una grande riflessione sull'esperienza che si vuole ottenere e tradurla in termini utili a ricordarsela: temi, emozioni, a volte anche un **concept narrativo**. Uno molto semplice ed efficace è la terna **conflitto, azione, obiettivo**:

Conflitto. Il mondo e che cosa succede nel mondo.

Azione. Il personaggio e che cosa può fare.

Obiettivo. Dove bisogna arrivare.

Alcuni aggiungono dei **giochi di riferimento**. L'importante è che il contenuto sia **stringato, diretto, in una pagina**, e riguardi l'esperienza desiderata; anche l'obiettivo del gioco va bene. A seconda del tipo di gioco (da tavolo o no) e del genere, le parti del documento cambiano.

Il suo è un gioco **horror in realtà virtuale**, "più esperienza visiva interattiva che gioco vero e proprio". Allora le domande sono: quanto ci si spinge con la narrativa? Quanto con l'interattività? E come si vuole usare la tecnologia VR? Soprattutto, essendo horror, il consiglio da "grande amante del genere" è di **partire dalle emozioni**, "che non sbagli mai": il tipo di emozione, il tipo di esperienza anche visiva, l'atmosfera, il *mood*.

0.6.16.2 Termini chiari per tutto il team

Attenzione però a usare **termini chiari per tutto il team**, soprattutto per il *mood*. Dire che il *mood* di un gioco è *gloom*: per chi chiede *gloom* è "alla Silent Hill" ("bravissimo"); per qualcun altro un gioco *gloom* è un gioco notturno, oscuro, pieno di nebbia. Se la differenza non è chiara a tutti, meglio non usare quel termine: si cerca un sinonimo, un termine che tutti comprendono, "perché altrimenti diventa un disastro".

Per le emozioni il relatore segnala il framework **6-11**, che riassume le possibili reazioni di un giocatore in **sei emozioni e undici istinti** (lo si vedrà nella parte sulle estetiche: sezione 0.6.23.1), oppure la **ruota delle emozioni di Plutchik**[55], di cui esistono varie versioni. Si parte dall'esperienza voluta e ci si chiede: come arrivo a quell'esperienza? Che cosa penso che possa permettermelo? Mi fa davvero arrivare lì?

0.6.16.3 Must have e nice to have

L'High Vision Document resta un **documento di visione**. Ci si possono mettere i **must have** e i **nice to have** di design: per esempio, must have "un sistema di dialogo", che significa che i personaggi parlano,

[53] Nella trascrizione "Stony Brand" e "Stonley Brande". Stone Librande ha tenuto alla GDC 2010 la *talk One-Page Designs*.
[54] Non viene chiamato per nome, e dalla trascrizione non è possibile stabilire chi sia; sta lavorando a un gioco horror in VR.
[55] Nella trascrizione "Plutchlin" e più avanti "Robert Plachkin": Robert Plutchik.

non che c'è interazione; nice to have "la possibilità di scegliere fra più battute". Nella visione questo non significa che alla fine debba esserci, ma che è un punto importante all'inizio dello sviluppo. Alla fine si potrebbe decidere che il dialogo ci deve essere, "perché se no è difficile andare avanti"[56], oppure, in VR, evitarlo del tutto: dipende.

Il partecipante riassume: nel documento scrive "voglio fare un gioco horror in VR che fa provare queste emozioni con questo gameplay", e poi ragiona su come arrivarci. Sì, conferma il relatore. Narrativamente, lo si può impostare così: **in un mondo** (descritto brevemente) **succede qualcosa**, l'inizio della storia, il *catalyst* (o il prologo), **tale per cui un personaggio agisce** con determinati poteri **per raggiungere un obiettivo ed evitare di perdere una posta in gioco**. L'esempio: in un mondo futuristico popolato di dinosauri robotici nasce una bambina senza genitori; scopre di avere una forte connessione con la nascita dei dinosauri e sfrutta la sua capacità di sopravvivenza per **scoprire la verità** (l'obiettivo) e **sopravvivere**: se non raggiunge l'obiettivo in tempo, perde la posta in gioco, cioè la vita[57].

Si può anche fare un **one-liner comparison**: "il mio gioco è un'esperienza VR simile a X, con un *mood* come quel gioco là, con una meccanica di questo tipo". L'importante è farne uno o due, mettere le emozioni, i pilastri: cose **utili a mantenere la direzione**. Le idee sono quelle, la visione è quella. L'esperienza reale finale sarà un po' diversa per problemi di produzione sul lungo termine, ma la visione originale deve essere qualcosa che **tutti hanno in mente**. Altrimenti, ogni volta che ci si ritrova, qualcuno dirà: "Ho visto *The Haunting of Bly Manor* su Netflix, bella quella cosa, mi piace". Può essere *out of scope*: come si fa a stabilirlo? Lo si stabilisce **proprio grazie all'High Vision Document**, che permette di distinguere ciò che è *out of the box* (una soluzione perfetta) da ciò che è *out of scope* (una soluzione che non va bene).

0.6.16.4 Concretamente, che cos'è una meccanica?

Un altro partecipante chiede di avere in mente in modo più concreto l'MDA: in un videogioco, concretamente, che cos'è una meccanica? "Perfetto": le slide successive trattano proprio quello.

0.6.17 Le meccaniche

"*Let's design a game*": da dove si parte? L'MDA è meccaniche, dinamiche ed estetiche; per il relatore le meccaniche sono **vincoli e rinforzi**, le dinamiche **comportamenti e scelte**, le estetiche **superfici**. Ma concretamente che cosa sono? Per capirlo propone un "giochino" su **Super Mario Bros.**, che tutti conoscono.

0.6.17.1 Che cosa può fare Mario

Qual è una delle cose più basilari che il gioco permette di fare al giocatore? Le risposte dei partecipanti, discusse una per una:

- **muoversi** ("carina come risposta") e soprattutto **saltare**, una delle cose più basilari che il personaggio può fare;
- **saltare sui nemici**;
- **rompere i blocchi** "con la testa": in realtà con il pugno, "solo che non tutti sanno che quando Mario salta non è la testa, ma è il pugno che va";
- **raccogliere oggetti**.

E ci sono cose che Mario **non** può fare?

- **tornare indietro**: può muoversi avanti e indietro nell'inquadratura, ma una volta che lo schermo è scorso non può tornare indietro nel livello (salvo rari casi nei Super Mario più recenti, che hanno "giocato con questa regola");

[56] Nella trascrizione "è difficile dargli via"; ricostruzione incerta.
[57] È, appena travestita, la premessa di *Horizon Zero Dawn*, già citato poco prima.

- **attaccare i nemici direttamente** senza il fiore di fuoco: può solo saltare loro addosso o usare il fiore.

Tutte queste sono **azioni** che il personaggio può compiere o **limiti** che possiede: sono **le meccaniche**. Un partecipante propone: "il modo in cui interagisci con l'ambiente". Sì, risponde il relatore, a parte che nelle dinamiche c'è qualcosa di più complesso: le meccaniche sono tutte le cose basilari che rappresentano **le basi del gameplay**, le scelte e le azioni di base. Quando le scelte diventano **strategie** non si è più nelle meccaniche basilari. Quando un gioco dà una scelta, quella è una meccanica; quando si può compiere un'azione (premere un tasto per dire una battuta, personalizzare il proprio personaggio[58]), quella è una meccanica.

0.6.17.2 Vincoli e rinforzi

Perché il relatore è affezionato a queste due parole? Quando si crea un personaggio, per esempio in **Biomutant**, molti dei numeri che si ottengono sono **vincoli o rinforzi** alle idee sul personaggio e a ciò che può fare. Avere 28 in forza **rinforza** l'idea che la forza sia una delle caratteristiche più alte, e che quindi "c'entri" con il personaggio. Un valore basso, per esempio 10 in fortuna[59], è un **vincolo**. Azioni, limiti, vincoli e rinforzi appartengono alle meccaniche.

La definizione completa: le meccaniche sono l'**insieme di vincoli e rinforzi che consentono, obbligano o vietano l'interazione con lo stato di gioco**. Con lo **stato di gioco**, attenzione, non con il *gameplay*: quando si arriva al gameplay si entra nelle dinamiche. Le meccaniche sono legate al concetto di **regola**, e un insieme di regole è un **riferimento normativo all'agire**: tutto ciò che in un gioco limita e norma l'agire del protagonista e del giocatore è una meccanica.

0.6.17.3 Meccaniche filtrate dall'estetica

Nel **Prince of Persia** originale le azioni erano saltare, andare avanti e indietro e combattere con affondi di spada, più alcune micro-azioni (un salto, un saltino, un passo, un passo lungo); e c'erano vincoli e rinforzi nella scena, come pezzi del livello che potevano crollare o su cui ci si poteva aggrappare. Nella slide si nota per esempio un pixel in più nei punti a cui ci si può aggrappare[60]. Questo il giocatore lo vede come estetica, ma l'idea dietro quell'estetica è un vincolo: da una certa parte "non ci puoi andare". Molte delle regole che si vedono a schermo sono **filtrate dall'estetica**, dalla superficie di gioco. Il designer le vede così solo a valle; a monte, all'inizio del design, ha delle idee, e poi serve un po' di *user experience* e di *user interface*.

Quante cose il giocatore può già dedurre! Molti giocatori dei **Dark Souls**, di **Elden Ring**, dei *soulslike* in generale, notano che avere la fiaschetta, la pozione che cura, sempre lì e magari in numero elevato porta a pensare "posso usarla"; avere la barra della vita piena mentre quella del nemico è già bassa dice "dai, magari lo vado a combattere". Così le meccaniche portano spesso, in modo molto naturale, alle **dinamiche**: scelte che il giocatore compie, possibili o impossibili, permesse o proibite.

0.6.17.4 Verbi e regole

Ogni volta che si può fare un'azione, quella è una meccanica. Le meccaniche si possono definire come **azioni, cioè verbi, e regole**:

Azioni e verbi. Ciò che il personaggio può fare. Nei giochi di ruolo e nei vecchi videogiochi, come le avventure *punta e clicca*, era naturale: si avevano direttamente i nomi delle azioni possibili. Anche negli RPG moderni: in *Final Fantasy VII Remake* si possono usare le abilità, le magie, gli oggetti. Di solito, non sempre, sono verbi: l'elenco dei verbi che rappresentano ciò che il giocatore può fare attraverso il personaggio è già un elenco di meccaniche. Mario, di base, può **run, jump, beat**.

Regole. Si possono identificare come **limiti**, se si fa questa distinzione: "regole, preclusioni e quant'altro", come dice la slide che il relatore aveva saltato.

[58] La trascrizione dice "io posso cliccare rotondo per dire nero": probabilmente "premere il tasto cerchio per dire una battuta", ma la frase è corrotta.

[59] Nella trascrizione "Pensiamo all'H10": dal seguito ("non sono molto fortunato") si tratta di un valore basso di fortuna.

[60] Nella trascrizione "che si potevano cliccare" e "ciò che noi possiamo cliccare"; in *Prince of Persia* non si clicca: dal contesto si tratta dei bordi a cui il personaggio può aggrapparsi.

0.6.18 Loop, dinamiche e sistemi

L'insieme delle meccaniche porta a dei **comportamenti**. Che cos'è *Super Mario*, se non un lungo elenco delle stesse azioni, una ripetizione costante di "run, jump, run, jump, run, jump..." con, ogni tanto, un "beat"?[61]

0.6.18.1 Il loop e il core loop

Questo in termini tecnici si chiama **loop**. Molte meccaniche vengono messe in relazione per creare un **ciclo di azioni** che il giocatore compie per arrivare all'obiettivo, e quel ciclo deve **supportare l'esperienza di gioco**. Il *run, jump, beat* di Mario, per esempio, supporta il piacere di correre: soprattutto nei Super Mario successivi al primo, a un certo punto della corsa Mario fa una mossa particolare, perché gli sviluppatori si sono accorti che uno dei piaceri del gioco era proprio **vedere Mario correre**.

Il loop di un gioco è l'insieme delle operazioni che il giocatore compirà più spesso. Si parla di **core loop**: il loop di base, delle azioni principali, che rappresenta anche **la cosa più "additiva"** dei giochi[62]. Può essere un'operazione piccola, *micro*, o intermedia: nei giochi *match-3* come *Homescapes* e *Gardenscapes*[63], il tutorial mostra esplicitamente il loop.

0.6.18.2 Il core loop di Ticket to Ride

Il relatore spiega il gioco da tavolo **Ticket to Ride**, uno dei tre giochi del workshop. L'obiettivo è **accumulare punti vittoria**, segnati sul bordo del tabellone:

- posizionando **trenini colorati** sulle tratte (per esempio da Salt Lake City a Denver, da Denver a Omaha, da Santa Fe a El Paso[64]);
- completando con successo un percorso tra due città indicate sui **biglietti di destinazione**, che assegnano punti a chi completa quella tratta;
- completando il **percorso più lungo**.

A ogni turno si può **pescare** da cinque carte scoperte, che tutti vedono: se ne pesca una per due volte, quindi fino a due carte. Le carte dello stesso colore possono poi, in un altro turno, essere scartate per **piazzare** i trenini: per la tratta da Denver a Kansas City servono quattro carte nere o quattro arancioni, e potendone prendere due a turno ci vorranno probabilmente due turni. Durante il turno **bisogna scegliere**: o si pescano carte, cioè si accumulano risorse, o si piazzano trenini, cioè si fanno punti. Non entrambe le cose. Questo è il **core loop**.

Ci sono però delle complicazioni. Il fatto che le carte siano scoperte apre due strategie: pescare per avvantaggiarsi nel costruire la propria tratta, oppure, vedendo che Francesco o Filippo stanno raccogliendo molte carte nere, o arancioni per una tratta lunga come quella da sei, prendere l'ultima carta arancione **non per sé, ma per impedire loro di costruire**. Questa è una **dinamica**: l'insieme delle scelte possibili, date dalle meccaniche, che attraverso di esse **fanno evolvere lo stato di gioco**. Anzi, già la scelta tra pescare e piazzare è una dinamica: pescare e piazzare sono meccaniche, ma la scelta nel turno è una dinamica, perché cambia lo stato di gioco. Ogni volta che si tolgono carte scoperte, altre vengono pescate dal mazzo e messe al loro posto; piazzare un trenino impedisce agli altri di farlo nello stesso punto, soprattutto dove due città sono collegate da una sola tratta.

0.6.18.3 Le dinamiche emergenti

Anche la strategia "dietro le quinte" è una dinamica, che a volte il game designer **non può controllare del tutto**: si parla di **dinamiche emergenti**, strategie a cui il creatore del gioco non aveva pensato ma

[61] La trascrizione ripete "run, jump" oltre un centinaio di volte: è probabilmente il relatore che legge la slide in modo volutamente ossessivo, amplificato da un errore di trascrizione.
[62] Così nella trascrizione; quasi certamente "additiva" sta per "addictive", cioè ciò che crea dipendenza.
[63] Il relatore li chiama "giochi di matchmaking, di element making": si tratta dei giochi di abbinamento di elementi.
[64] Nella trascrizione "Solday City" e "Okama".

che vengono costruite dai giocatori. L'esempio che il relatore fa a lezione ("anche se non è esattamente tale, quindi perdonatemi") sono le **aperture degli scacchi**: non sono regole del gioco, ma dinamiche, un insieme di scelte ormai talmente comprese da essere state codificate quasi come meccaniche[65]. Gli scacchi hanno poi dinamiche vere e proprie, con scelte enormi, perché a ogni turno si può muovere un solo pezzo. Una delle più famose è la nozione di **tempo**: una mossa che, in una strategia, permette di risparmiare uno dei passi necessari per arrivare all'obiettivo. Il concetto di tempo si applica anche a tutti i giochi da tavolo e ad alcuni videogiochi.

Riassumendo: le meccaniche sono azioni (*pescare* è un verbo, un'azione, una meccanica; *piazzare* pure); "nel dubbio, pensate a verbi e a limiti". Decidere se piazzare o pescare è una **scelta**, una dinamica.

0.6.18.4 Le reazioni e l'action feedback

Tutte le scelte hanno delle **reazioni**, da parte del gioco o degli altri giocatori. Se non ci sono, "questo non è un gioco"[66]: si ha la sensazione che il gioco non sia davvero lì. Se il gioco non dà la sensazione di colpire davvero i nemici, come invece accade in **Path of Exile**, dove i nemici bruciano e la loro barra crolla lentamente, si può pensare che le proprie azioni non abbiano successo. Per questo i giochi, soprattutto i videogiochi ma "attenzione, anche i giochi da tavolo", hanno risposte visive agli input: gli **action feedback**[67], reazioni visive alle azioni, alle meccaniche attivate.

Gli *action feedback* possono **generare dinamiche** e strategie. In **Super Mario Maker** molti *action feedback* molto tecnici e interni alle meccaniche vengono usati come *cheat* o per *speedrun* velocissime; già nel vecchio Super Mario si potevano sfruttare singoli pixel per saltare dove in realtà non si sarebbe potuto. Nella GIF della slide non si usano nemmeno le tartarughe per salire in alto, ma pixel precisi.

0.6.18.5 I sistemi

L'insieme di meccaniche e dinamiche, di scelte, di automatismi del gioco, di tutto ciò che compone azioni, regole, scelte e gameplay, crea un **sistema di gioco**: un insieme di automatismi, regole, azioni e scelte che genera una porzione del gioco o perfino l'intero gioco.

L'esempio è un'infografica ("non è esattamente così che funziona") sui dialoghi di **Hades**[68], un *roguelite* in cui si interpreta Zagreus, figlio di Ade, e che ha dialoghi "procedurali", anzi più precisamente **randomici**. Ogni volta che Zagreus interagisce con un personaggio, per esempio Nyx, il sistema:

1. crea un elenco di battute possibili sulla base di alcuni **trigger**, cioè di elementi del gioco;
2. ne sceglie una a caso;
3. se quella battuta è già stata scelta, la toglie: non potrà più uscire finché non saranno uscite tutte le altre.

Il gioco ha un numero enorme di battute ("sono dei pazzi")[69], e questo dà la sensazione di un gioco che **non si ripete mai due volte**. È un sistema; nella slide c'è il video che lo spiega, per i curiosi.

Un sistema, dunque, è un **insieme di meccaniche e dinamiche** che crea un gruppo di elementi interconnessi che lavorano per lo **stesso obiettivo, esperienza o tema**, "vi ricordate l'esperienza, importantissima", spesso **potenziandosi o depotenziandosi** fra loro. Più un sistema entra in gioco, più potenzia o depotenzia un certo comportamento del giocatore, che ne viene influenzato e **manipolato**.

[65] La frase nella trascrizione è corrotta ("sono state tradotte da un po' di dinamica un po' di cd da delle dinamiche delle meccaniche vere e proprie"); la ricostruzione è la più probabile.

[66] Qui la slide mostra una GIF che "sembra un po' Monkey Island, ma in realtà non c'entra niente con Monkey Island, stranamente".

[67] Il relatore avverte di non confonderli con "un'altra cosa che vedremo dopo", cioè i *feedback loop* (sezione 0.6.19).

[68] Nella trascrizione "AIDS" ("so che ha una pronuncia differente a seconda delle zone in cui si abita"): si tratta di *Hades* di Supergiant Games.

[69] Il relatore dice "più di 20 milioni di battute", aggiungendo "o cose del genere". È un'evidente iperbole o un lapsus: le battute di dialogo doppiate di *Hades* sono nell'ordine delle ventimila.

0.6.18.6 La manipolazione

Nel game design si sentirà spesso la parola **manipolazione**, da non intendere in senso negativo come in italiano ("oddio, sto manipolando il giocatore"). Certo, ci sono tantissimi giochi basati su un *trick*, un colpo di scena che porta a credere che il protagonista sia nel giusto quando non lo è ("senza fare spoiler"). Ma per manipolazione si intende **tutto ciò che permette di controllare il comportamento dei giocatori** e soprattutto di **spingerli a compiere delle azioni, anche per il loro bene**.

Si immagini un gioco senza ostacoli: **Pac-Man senza fantasmi**; un **Prince of Persia** in cui arrivare in fondo è semplicissimo perché ogni volta che si cade si riparte esattamente da lì, dove la morte esiste "tra virgolette" e i puzzle sono facilissimi: è il *Prince of Persia* del 2008, "ve lo dico per esperienza, l'ho provata sulla mia pelle". Quando un gioco ha dei feedback ma non dei feedback che creino una sensazione di **ostacolo**, la manipolazione positiva **manca**: manca ciò che spinge l'utente verso l'esperienza voluta. Il loop diventa così potente che il giocatore vince "dal vuoto", senza sforzo[70]: se in Super Mario non ci fossero nemici, alla lunga il loop porterebbe alla vittoria senza alcuna difficoltà.

Quindi la manipolazione non serve a ingannare il giocatore: serve a **controllare informazioni, azioni, regole e meccaniche**, e, per quanto possibile, le dinamiche (anche se spesso il game designer ne perde il controllo), per **portare il giocatore verso l'esperienza di gioco** e sostenerlo nella sua ricerca. Per questo a volte si inseriscono scelte importanti, anche morali (ma non solo): non per ingannare, ma per dare quel guizzo, quella sensazione di **tensione**, che qualcosa debba o possa andare male.

0.6.19 Noia, frustrazione, flow e feedback loop

Qualcosa deve **bilanciare il loop**, "altrimenti non siamo altro che il dinosaurino di Google Chrome": un passatempo carino quando manca internet, ma dopo due o tre volte che si saltano i cactus l'esperienza finisce, perché è **noiosa**.

Ma la noia non è l'unico problema che un giocatore percepisce se non lo si "manipola". C'è anche la **frustrazione**. La slide mostra il tutorial di **Driver**, "tristemente famoso": il relatore lo ha giocato da adolescente e conferma che è più complesso di tutto il resto del gioco. Bisogna compiere un elenco di manovre con l'auto entro un tempo dato, senza che il tutorial venga spiegato bene, ed è così stretto nei tempi che "è molto più difficile fare il tutorial che l'intero gioco". Un partecipante conferma: "non gioco nemmeno ai giochi di macchine, ma appena vedo quel foglio di carta mi sale il sangue alla testa"[71].

0.6.19.1 Il pacing

Per questo bisogna essere molto attenti al **pacing**: il **ritmo** con cui il giocatore incontra ostacoli e momenti di tensione. Ma il pacing da solo non sistema il loop: anche facendo incontrare ostacoli o limiti, cioè regole che limitano spesso il personaggio, si crea frustrazione ma non si bilancia il loop, che continuerà a portare il giocatore alla vittoria. Serve qualcosa di più della tensione: un **attrito** che controbilanci il loop. Si pensi a Super Mario con pochissimi nemici, in cui si prendono monete e "un fracco di vite"; o al classico livello "da farm" di **Crash Bandicoot 3**, così semplice e così ben imparato che lo si ripete mille volte per accumulare centinaia di vite.

0.6.19.2 Il flow

Senza questo attrito non si costruisce il **flow**, il flusso: una teoria psicologica, "attenzione, non di game design: ancora una volta il game design prende da altri ambiti", di **Mihály Csíkszentmihályi**[72], grande teorico della psicologia, ungherese, morto nel 2021. Il flow è il momento in cui si è "nel *groove*": un momento di **grande sfida**, ma, "attenzione, non basta", anche di **grandi capacità** di affrontarla. Una persona in flow fa le cose in modo elegantissimo e precisissimo, perché ha capito, è molto reattiva. Non è super eccitata né

[70] Nella trascrizione "vinceranno mai dal voto" e "senza spromattuto": la ricostruzione è congetturale, ma il senso è chiaro dal contesto.

[71] Ricostruzione libera di una frase molto corrotta nella trascrizione ("sento il vermino e il buon sangue").

[72] Nella trascrizione "Michael Kitzes-Miajli" ("credo di averlo pronunciato bene, è un ungherese"). Csíkszentmihályi è morto nell'ottobre 2021.

completamente in controllo: è in quella **forbice** che permette di essere talmente concentrati da dimenticarsi del resto.

0.6.19.3 I feedback loop

Come si crea il flow? Se il *core loop* è positivo, cioè porta il giocatore verso la conclusione del gioco, serve un loop che **gli impedisca di arrivarci** troppo facilmente: il **feedback loop negativo**. I loop infatti non sono solo i *core loop*, ma si dividono in:

Loop a feedback positivo. Ogni volta che si attivano **amplificano**, **destabilizzano** il gioco verso l'obiettivo: riducono la lunghezza della partita, avvicinano l'*endgame* e la condizione di vittoria, potenziano il *setup* della fase iniziale, creano **escalation**. Di solito sono i loop che portano il giocatore a vincere.

Loop a feedback negativo. Vanno usati per **bilanciare** i primi: **stabilizzano** il gioco, allungano un po' la partita, ma soprattutto creano pacing, un po' di **stallo**, quel punto in cui non si è né in completa eccitazione né in completo controllo. Il punto del flow.

Su questo punto il relatore si contraddice a voce[73], ma il senso è chiaro: il loop negativo controbilancia il positivo e permette di allungare un po' il gioco creando tensione.

0.6.19.4 Gli habit loop

Questi loop possono diventare talmente abitudinari da trasformarsi in **habit loop**, loop di abitudine, molto presenti nei giochi mobile ("chi di voi conosce Clash of Clans?"[74]). Anche questa è una teoria psicologica: copre tutti i casi in cui una **ricompensa positiva** crea la voglia di riattivare il loop, crea una **routine**, fino a un ciclo in cui ci si dimentica che si tratta di una dinamica o di una meccanica. Come diceva Michele qualche giorno prima: vedere nel videogioco quel pallino, quella notifica che si accende, e cominciare automaticamente a fare delle operazioni senza accorgersene, perché da consce sono diventate **inconsce**.

0.6.20 Un mindset di caos e tre idee da portare a casa

"Domande fin qua? So che è un casino." Il relatore chiede ai game designer presenti se "tengono botta" o se ha detto cose che li hanno fatti cadere dalla sedia. Uno risponde che, a parte lo "spoiler" dell'MDA, "tutto *on point*"; ma "tanta roba". "Lo so, ragazzi, c'è tanta roba": il workshop è molto denso, e il giorno dopo non si userà tutto. La confusione è **voluta**: una delle cose che il relatore vuole far capire è che il mindset del game designer è **un mindset di caos**. Ci sono così tante cose da considerare che un corso di tre anni "ha un po' senso". Vedersi slide come queste o leggere libri è utile, ma richiede tempo e manca di una guida, perché il game design è una **disciplina molto giovane**: sembra anziana, ma nasce più o meno **40-50 anni fa**, giovanissima rispetto ad altre. Per questo "azienda che vai, teorie che trovi; azienda che vai, metodologie che trovi", ed è utile definire il game design come **metodologia**: la scelta di un metodo ideale per raggiungere un'esperienza e una visione volute.

Le cose che il relatore si aspetta che si capiscano non sono tutte quelle viste, ma **tre**:

1. **Il game design è una grande cassetta degli attrezzi**: *lo strumento corretto per il giusto lavoro.* Chi non è game designer è invitato a trovare nella propria esperienza di artista, designer, musicista o programmatore i **propri strumenti** e metterli al servizio del design, sotto forma di principi di design. Non conta sapere a memoria tutti gli strumenti, "è impossibile": le slide verranno date proprio per usarle nel workshop. Un amico del relatore che aiuta le risorse umane in un'azienda di software racconta che ai colloqui, quando si assegna un compito di programmazione, molti si bloccano perché non ricordano una formula. E lui dice: "Cercala. Anch'io non la so. Quello che voglio è che tu mi faccia

[73] Dopo aver detto che il loop positivo "destabilizza" e quello negativo "stabilizza", che è la terminologia corrente, aggiunge: "non fatemi ingannare, non è che il negative destabilizza, il positive stabilizza, è il contrario, il positive stabilizza perché porta verso la vita[ttoria]". È probabilmente un lapsus, o un modo di dire che il loop positivo "porta a compimento" la partita; nel testo si è seguita la terminologia standard, coerente con la prima formulazione.

[74] Nella trascrizione "Clash of Clown".

capire il mindset": che, dato un problema, si abbia il **pensiero critico** per analizzarlo e scegliere il metodo giusto per risolverlo.

2. **Le idee non sono nulla senza implementazione**. Se si perde la strada, se si hanno tante idee ma non si produce niente, è perché non si sta implementando la giusta documentazione: invece di fare l'High Vision Document o il design document e poi sviluppare il prodotto, con meccaniche, dinamiche ed estetiche, si rifà continuamente il design della visione, "mille volte". Ci si perde in un bicchier d'acqua. La visione va fatta, e fatta bene; può cambiare, ma "50% scienza, 50% arte": evitare le idee *out of scope*, dare il benvenuto alle idee *out of the box*.

3. **Anche il giocatore può risultare confuso**. Ed è qui che entrano in gioco scrittori, artisti, programmatori, musicisti, perché molto spesso queste professionalità vengono impiegate per creare **le estetiche** del gioco.

0.6.21 Le estetiche: le superfici che guidano

Si immagini di trovarsi davanti a un gioco come **Path of Exile**, la cui **mappa delle abilità** è sterminata: è ciò che il giocatore si trova davanti quando comincia. O a un gioco complesso come **StarCraft II**: il relatore lo ha disinstallato quando nei server europei sono arrivati i giocatori coreani (la slide mostra una GIF dalle finali), capaci di micro-variazioni di comportamento così veloci e di entrare nelle basi altrui in modo così strategico che giocare era impossibile.

Per aiutare il giocatore servono le **estetiche**; per comunicare il gioco servono le estetiche; per avere il gioco servono le tecnologie e le estetiche. Le estetiche, si ricorda, sono **l'insieme di emozioni, piaceri e sensazioni evocati dalle superfici di gioco**: grafica, intreccio, trama, testo.

0.6.21.1 Journey

La slide mostra il **design plot** di **Journey**[75]: il modo in cui gli sviluppatori hanno disegnato l'andamento del gioco, molto grafico e visivo. Journey si affida a una grande grafica. Qual è il suo obiettivo? "Arrivare lì", alla **montagna**, e lo comunica l'immagine stessa. Luca aggiunge: "il viaggio" ("bravissimo Luca"). L'obiettivo del viaggio è la montagna; l'esperienza vera del gioco è il viaggio in sé, ciò che sta tra l'inizio e la fine, e questo lo comunica il titolo. Le superfici hanno un **compito di guida**: **guidano, aiutano, semplificano e automatizzano** le informazioni per il giocatore, ma anche per il game designer.

0.6.21.2 Glossari

Nei giochi da tavolo ci sono molti esempi: il tutorial passo per passo, l'elenco delle icone di **Gloomhaven**, che il giocatore torna a consultare. Lo stesso vale nei videogiochi, con il *codex* nel menu, l'elenco dei luoghi e dei comandi. E vale per i game designer, che hanno bisogno di **glossari**:

Glossario di produzione (*production dictionary*). Il dizionario dei termini che servono nella produzione, compresi quelli che i membri del team interpretano in modo strano: "meglio spiegarlo".

Glossario di mercato. Come il pubblico chiama certe cose, in quale genere colloca il gioco: "sto facendo un gioco con determinate caratteristiche: come lo identifica il pubblico?".

Glossario di gioco e glossario visivo. Che si vedranno più avanti.

0.6.21.3 Le regole e le house rule

Ci sono anche teorie su come scrivere regole e glossari[76]. **Ticket to Ride** ha regole specifiche, che si potranno vedere anche in alcuni video. La **scrittura** e la **comunicazione delle regole** sono molto importanti, perché

[75] La slide contiene un link, come molte altre ("dove c'è questa icona c'è sempre un link").

[76] Il relatore cita uno studio "del 2008" di un autore che la trascrizione chiama "Pavlet"; non è stato possibile identificarlo. Lo definisce "un po' più tecnico".

se nei videogiochi di oggi c'è il tutorial che accompagna passo passo, nel gioco da tavolo no. Molti giochi da tavolo hanno il problema che, essendo le regole complesse, vengono spiegate da altri giocatori, che non usano il regolamento: nascono così le **house rule**, le regole della casa. Gli esempi:

- nel **Monopoly**, quando si arriva per primi su una casella (per esempio Piazza Vittoria), nell'uso italiano la si può comprare; nella versione originale americana parte un'**asta**, anche se si è i primi ad arrivarci. Qualcuno precisa che nella versione italiana l'asta è rimasta solo se chi ci capita sopra decide di non acquistare: "esatto", nell'originale l'asta c'era e basta, in Italia è diventato "puoi comprarlo, se non lo compri c'è l'asta"[77];
- nel **Risiko** italiano, al regolamento originario è stato aggiunto un **dado in difesa**, e questo ha creato partite molto più lunghe del normale[78] ("eh sì, Giulia, queste cose poi si scoprono lentamente");
- l'antenato del Monopoly, il **Landlord's Game**, aveva regole ancora diverse, pensate per far capire alle persone i rischi "del capitalismo, tra virgolette", cioè delle rendite fondiarie e della nuova imposta sugli affitti introdotta in America. L'autrice era una donna, **Lizzie Magie**[79].

0.6.21.4 Tutorial e dialoghi

Di nuovo, i **tutorial** sono importantissimi, anche quelli **visivi**: non servono troppe parole per fare un tutorial. A volte usano dei personaggi come **veicoli**, come in *Mario & Luigi: Dream Team*. E i **dialoghi** sono una parte importante delle estetiche: ci sono teorie sul dialogo, che vengono dal cinema.

Il relatore usa una teoria che chiama **Ex-Dialog theory**[80], nata per scrivere dialoghi ma utilissima anche per scrivere **regolamenti**: individua le **nove funzioni** del testo, di una comunicazione, di un dialogo[81]. Siccome il gioco è una comunicazione differita, valgono anche per le regole. Inoltre molti dialoghi devono essere **evidenziati**: al giocatore non si danno solo informazioni, ma un **ordine di informazioni**, con punti focali, come i grassetti in un manuale, che danno al lettore il colpo d'occhio e l'ordine d'importanza. La slide mostra per esempio la parola *look* che salta all'occhio[82]. Lo stesso vale per i testi, per i dialoghi e per i **bark**, le battute pronunciate dagli NPC che servono a indirizzare il giocatore: a volte servono **codici colore**, a volte termini precisi inseriti nel dialogo.

0.6.22 Icone e idiomi

Da qui nasce una teoria "molto semplice da utilizzare, che useremo domani", una delle teorie del relatore ("così vi do cose di altri, ma anche cose mie"). Poi ha scoperto, "giustamente, perché io non sono nessuno, ragazzi", che è molto simile a una teoria di **Roland Barthes**, contenuta nel saggio *Introduzione all'analisi strutturale dei racconti*[83]: il concetto di **icone e idiomi**.

Molti giocatori **ragionano per parole chiave**. Icone e idiomi sono termini, parole che per il giocatore, e anche per chi progetta, sono **ancore di senso**. Un glossario di solito è formato da icone e idiomi; le azioni del personaggio, i verbi, sono icone e vanno nel glossario. La differenza:

Icone. Sono **snodi visivi**: parole, immagini o simboli che richiamano una parte del mondo, ma sono **generici** e **comuni a più giochi**. Si applicano anche alle immagini: un codice colore è un'icona. In

[77] È così anche nel regolamento ufficiale attuale: l'asta scatta se il giocatore non acquista. L'uso "casalingo" più diffuso è ignorare del tutto l'asta.

[78] Nel *Risiko!* italiano il difensore può tirare fino a tre dadi, contro i due del *Risk* originale.

[79] Nella trascrizione "la Mage". Il relatore dice che lo ha inventato "nell'800"; il brevetto del *Landlord's Game* è del 1904, e il gioco era ispirato alle teorie di Henry George sulla tassa unica sulla terra.

[80] Il nome è quello della trascrizione; potrebbe essere storpiato e non è stato possibile identificare la fonte.

[81] Le nove funzioni non vengono elencate a voce.

[82] La trascrizione aggiunge: "che questo è un work in progress, poi alla fine l'hanno messo R3, anziché guarda in alto": sembra un riferimento a una versione provvisoria di un tutorial in cui l'indicazione testuale è stata poi sostituita dal nome del tasto. Il gioco non è nominato.

[83] Nella trascrizione "In produzione all'analisi strutturale narrativa". Il saggio è uscito nel 1966 sul numero 8 della rivista *Communications*, che il relatore definisce "americana": è in realtà una rivista francese. Il relatore mostra anche l'antologia *Narratology*, una raccolta di saggi di narratologia che "costa un casino" e che contiene il saggio di Barthes; "fatto da un'egalanda" rimanda probabilmente a José Ángel García Landa, che l'ha curata con Susana Onega (1996). Barthes, in quel saggio, distingue tra *indizi* e *informanti*; la corrispondenza con "icone e idiomi" è quella proposta dal relatore.

Path of Exile la sfera rossa che scende quando si subisce danno è un'icona; in molti giochi il rosso e la pozione rossa sono associati alla vita, come le fiaschette Estus dei Dark Souls, di aspetto rossastro.

Idiomi. Sono **specifici del gioco** che si sta facendo.

Gli esempi, a coppie:

Icona	Idioma	Da dove viene l'idioma
guerriero	witcher	*The Witcher*
ascia bipenne[84]	Stormbreaker	il Thor del Marvel Cinematic Universe
ramingo	Dúnedain	*Il Signore degli Anelli*
demone	Mangiamorte	*Harry Potter*
oracolo	Precog	*Minority Report*
centurione	Templari Neri	*Warhammer 40.000*[85]
demone	Malebranche	la *Divina Commedia*
innesto	Squid	*Strange Days*[86]

Tutti questi termini aiutano il giocatore a capire il mondo del gioco e il gioco stesso. Quando si mettono parole nel proprio gioco bisogna stare attenti a **icone e idiomi**, e si possono anche fare studi specifici su di essi.

0.6.22.1 Icone nei tutorial e ancore visive

Le icone sono importantissime nei tutorial. Nel tutorial di **The Witcher 3**, **Vesemir** è evidenziato in rosso perché bisogna combatterlo: il gioco mostra l'icona del nemico e insegna che, quando la si vede, si ha davanti un nemico. Mostra anche i comandi ("io li avrei messi un po' più grandi"): *left shift* tra parentesi quadre, in arancione, è un'icona "visiva e terminologica".

Il giocatore può però saltare il tutorial, ed è qui che entrano le **ancore visive**, quelle che nel *level design* si chiamano **landmark** o punti di interesse. Si possono comunicare informazioni al giocatore con una mera estetica, e anche attraverso le **emozioni** che prova il personaggio principale: si vuole creare un'esperienza fatta di emozioni, e quindi è qualcosa da comunicare, sempre come icona o idioma, oppure come dialogo, oltre che visivamente.

0.6.23 Le emozioni e i piaceri

Qui entra il concetto di **emozione**. Le estetiche sono per lo più emozioni e sensazioni: si può usare la **ruota delle emozioni di Robert Plutchik** oppure altri framework.

0.6.23.1 Il framework 6-11

Uno dei più interessanti è il **6-11** di **Dillon**, autore anche del framework **AGE**: azioni, *gameplay*, esperienze ("azioni, regole, gameplay, esperienze")[87]. Il 6-11 divide le reazioni di un giocatore in:

Sei emozioni. Le emozioni primarie: **paura**, **rabbia**, **orgoglio**, **tristezza**, **gioia** ed **eccitazione**. La rabbia è un'**emozione spia**: di solito nasconde altre emozioni.

Undici istinti. Tra questi l'istinto di **sopravvivenza**, di **identificazione**, di **raccolta**, l'**avidità**, l'**aggressività**, la **competizione**, la **protezione**, la **curiosità**, l'**apprezzamento estetico**[88].

[84] Nella trascrizione "sciabipenne": "l'ascia bipenne", generica, contrapposta a Stormbreaker, che "ce l'ha soltanto il Thor della MCU".

[85] Il relatore dice solo "Centurioni e Templari Neri"; l'attribuzione dei Templari Neri a *Warhammer 40.000* è una deduzione.

[86] Nella trascrizione "Stranger Days" ("per chi è fan dei film cyberpunk"): è *Strange Days* (1995), in cui lo SQUID è il dispositivo che registra e riproduce le esperienze. Il relatore spiega anche che i Precog sono quelli che vedono nel futuro "e sono solo lì", e cita l'Oracolo di Delfi come esempio di oracolo con un nome proprio.

[87] Roberto Dillon, autore di *On the Way to Fun* (2010), dove presenta sia il framework AGE (*Actions, Gameplay, Experience*) sia il 6-11.

[88] Qui la trascrizione si blocca, ripetendo "la competizione" alcune centinaia di volte, e riprende direttamente sull'esempio

0.6.23.2 L'esempio di Super Mario

La slide applica AGE e 6-11 a Super Mario:

Azioni. Correre e saltare, quelle basilari.

Gameplay. Acquisire monete e *power up*, combattere o evitare i nemici, correre fino alla fine, scoprire aree nascoste.

Esperienza. Curiosità; **avidità** (*greed*), anche se il relatore non è molto d'accordo: ci sono le monete, ma più che avidità è **raccolta**; **eccitazione**, per aver preso la moneta.

Curiosità, eccitazione e raccolta danno un'**incitazione** che crea il cosiddetto divertimento, "che appunto non è mai il mio divertimento". Viene anche dalla **gioia**, che per il relatore, "con tutto rispetto" per l'analisi della slide, non deriva dall'orgoglio di aver finito il livello o dalla protezione, ma dall'**atto del correre**, dall'arrivare fino in fondo correndo: è un **apprezzamento estetico**, più che altro. Più che divertimento, è meglio dire **soddisfazione**; anzi, "è addirittura meglio dire **piacere**". Emozioni e piaceri.

0.6.23.3 Gli otto piaceri di LeBlanc

I tipi di *fun* di **LeBlanc** sono infatti **otto piaceri**: **sensazione**, **fantasia**, **narrativa**, **sfida**, **amicizia**, **scoperta**, **espressione** ed **evasione**[89]. In Mario c'è il piacere della **fantasia**, quello della **sensazione**, poco piacere della narrativa, e soprattutto, nel correre, quello dell'**evasione**.

0.6.23.4 Una discussione sull'avidità

Un partecipante chiede quale meccanica potrebbe essere collegata all'avidità: per esempio **collezionare tutte le auto** in Cyberpunk o in giochi simili? Non solo, risponde il relatore: quella è **raccolta**. L'avidità è addirittura **raccolta più protezione**. Dillon stesso dice che il suo è una **prima versione** di un possibile framework: alcuni istinti non sono "paritetici a livello di paradigma". L'avidità sembra l'estremo della raccolta; l'apprezzamento estetico, accanto alla sopravvivenza, sembra messo lì e "c'è qualcosa che non quadra". Bisognerebbe studiare meglio quali siano gli istinti, magari partendo dalla **piramide dei bisogni di Maslow**[90].

Dalla chat arriva un suggerimento: l'avidità è forse più il *loot* di **Borderlands**. Sì: tutti i giochi in cui si ha un bottino che **si deve difendere** stimolano l'avidità. Un altro propone "quel gioco Ubisoft di qualche anno fa" in cui si aveva una torre con il tesoro da proteggere con le trappole[91]: "è possibile". Ma l'avidità è anche **correre molti rischi per ottenere un premio**. In **A Plague Tale: Requiem**, che il relatore sta giocando, le casse più importanti sono spesso in zone piene di nemici, raggiungerle è molto rischioso e bisogna essere precisissimi; il gioco **fa vedere la cassa** per stimolare l'avidità (c'è un punto in riva al mare in cui si tira una corda e la cassa scivola in mezzo a uno sciame di ratti). Ritornano **manipolazione**, **esperienza** e **avidità**. E perché il rischio ha senso in *Requiem*? Perché c'entra con il **tema** del gioco, molto legato al rischiare, all'andare oltre i limiti, al perdere il controllo: inserire quel tipo di esperienza è tematicamente sensato, non è *out of scope*. "Iniziate a capire il mindset?"

Un partecipante obietta che a lui sembrava più un'idea di **protezione** verso il fratellino. Nel primo capitolo, *A Plague Tale: Innocence*, "assolutamente sì"; *Requiem* comincia con un senso di protezione, ma poi arriva un punto, un dialogo, in cui si capisce che "non è *Innocence*".

---

di Mario: l'elenco completo non viene pronunciato (o non è stato trascritto). Gli istinti nominati qui e nel seguito sono quelli riportati; "l'istinto di abilità" della trascrizione è con ogni probabilità "avidità" (*greed*). Nel libro di Dillon gli undici istinti sono: sopravvivenza, identificazione, raccolta, avidità, protezione/cura, aggressività, vendetta, competizione, comunicazione, esplorazione/curiosità, apprezzamento estetico (*color appreciation*).

[89] Nell'originale inglese: *sensation*, *fantasy*, *narrative*, *challenge*, *fellowship*, *discovery*, *expression*, *submission*. "Evasione" è la resa del relatore di *submission*, il gioco come passatempo in cui abbandonarsi.

[90] Nella trascrizione "Marlow".

[91] Il titolo non viene nominato e non è stato possibile identificarlo con certezza.

0.6.24 Comunicare con le immagini: ambienti e forme

Si può comunicare non solo con i termini, ma con le **immagini**. Per esempio il sistema di **tracce** da seguire in **The Witcher 3**, o l'**environmental storytelling**, la narrazione ambientale: in **Cyberpunk 2077** il Trauma Team è dotato di elementi che lo rendono molto minaccioso, "non è il classico *trauma team*", e da tutta una serie di elementi ambientali si capisce di essere in un mondo cyberpunk.

Visivamente lo si può fare con una teoria che deriva dall'**enneagramma**, quella dei **tre centri emotivi**: **cervello, cuore e fegato**, un po' ripresa in *Horizon Zero Dawn* e *Forbidden West*[92], che Chris Solarski, legandola al *level design*, collega alla **teoria delle forme** (*shape*).

0.6.24.1 Cerchio, quadrato, triangolo

La teoria delle forme è "l'ultima teoria che vedremo oggi" e una delle più semplici per fare **estetiche visive** anche se non si è disegnatori, "una delle tecniche più basilari". Si parte dalle tre forme base, **cerchio**, **quadrato** e **triangolo**, e le si associa a **comunicazioni implicite**[93]. Nintendo stessa ha ammesso che in **The Legend of Zelda: Breath of the Wild** il *world building* è stato costruito dando un peso visivo molto maggiore ai **triangoli**. Perché? Perché il triangolo rimanda implicitamente all'idea dell'**obiettivo**. Nel *level design* il lavoro di *shaping* è importantissimo, e lo è anche nell'arte.

0.6.24.2 E la musica

Non va sottovalutata la musica: nel libro *Interactive Stories and Video Game Art* ("eccolo qua", lo mostra) Solarski fa esempi anche musicali, per esempio sui **temi musicali dei personaggi** di Super Mario, quali temi ci sono e quali no. Molta della riflessione odierna sulla musica nei giochi deriva proprio dai temi legati ai personaggi; si pensi a giochi come **Florence**, o ad altri che creano molta atmosfera con la musica.

Con questo si chiude la carrellata su meccaniche, dinamiche ed estetiche: "ovviamente siete confusi, ma ripeto, l'obiettivo era confondervi".

0.6.25 Il workshop della seconda giornata

Il relatore anticipa che cosa si farà il giorno dopo (in parte si potrebbe cominciare già oggi, ma si andrà più in dettaglio domani). Ognuno, da solo o in coppia ("vediamo come fare"), sceglierà **uno dei tre giochi**:

Lasers & Feelings. Un gioco di ruolo "tipo Dungeons & Dragons", ma *one page*: sta tutto in una pagina ed è pensato per avventure "alla Star Trek". Le regole sono tutte lì: creazione del personaggio, creazione della nave, meccanica di base, regole per il *game master*, come condurre il gioco.

Ticket to Ride. Il gioco da tavolo già visto (sezione 0.6.18.2).

Super Mario Bros. Ancora lui (nella slide doveva esserci una GIF che non si carica).

0.6.25.1 Che cosa bisogna fare

Ci si aspetta un'**analisi** del prodotto e l'**idea di un hack**, cioè una versione alternativa del gioco. Gli esempi del relatore:

- **Lasers & Feelings** è su Star Trek: farne una versione ispirata alla **mafia**[94], togliendo l'elemento fantascientifico e cambiando parole e concetti;
- **Ticket to Ride** ha i trenini, ma le stesse meccaniche si possono usare per qualsiasi cosa legata al **viaggio**: degli eremiti, un gioco storico sul **pellegrinaggio**, **Marco Polo** che deve andare in Cina;

[92] Così nella trascrizione; il relatore non spiega in che modo la teoria sia ripresa nei due giochi.

[93] La slide mostra le associazioni, che il relatore non legge. Nella formulazione corrente (anche in Solarski) il cerchio comunica morbidezza, innocenza e amicizia, il quadrato stabilità e solidità, il triangolo dinamismo, pericolo o direzione.

[94] Nella trascrizione "ispirato a Maffi" e, poco dopo, "togliere l'elemento sci-fi, mettere dentro baffi": la lettura "mafia" è la più plausibile, ma non è certa.

- **Super Mario**: riprendere l'idea del **mondo sotterraneo da terraformare** (sezione 0.6.8). Si vive nei sotterranei dopo anni di avanzamento tecnologico, si hanno i poteri per tornare in superficie e terraformare un mondo "un po' cartoonesco": un Super Mario Bros. al contrario, con liane e altri elementi sotterranei.

Non si deve fare il gioco intero, "attenzione": si deve fare un **High Vision Document**, cioè:

1. stabilire la **visione**;
2. stabilire il **pubblico**;
3. fare un po' di **ricerca**;
4. creare alcune **idee per modificare le meccaniche**. Per farlo bisogna analizzare per forza il **core loop**: capire quali sono i **verbi**, quali le meccaniche. Ci si ferma lì: **non si arriva a dinamiche ed estetiche**, ci si ferma ai verbi;
5. fare un **one page** che dica: "la nostra idea è prendere Lasers & Feelings e hackerarlo in questo modo". Per Lasers & Feelings si può indicare quali parti del foglio cambiare; per Ticket to Ride fare un'**ipotesi visiva del tabellone**; per Super Mario prendere degli *screenshot* e modificarli un po', creando una **mappa mentale** di come diventerebbe il gioco.

Quindi non il gioco in sé, ma un High Vision Document di come il gioco dovrebbe essere.

0.6.26 Il riepilogo del mindset

Il relatore resta a disposizione "una ventina di minuti, mezz'oretta" per le domande e manda le slide da usare il giorno dopo. Ripete le tre idee: **lo strumento corretto per il giusto lavoro**; **le idee non sono nulla senza implementazione**; **anche il giocatore può risultare confuso**, "e anche voi potete risultare confusi, quindi ricordatevi il mindset". Riepiloga il mindset (sezione 0.6.6.2) in chiave operativa:

Pensiero critico. Analizzate il gioco, capite quale metodo usare per svilupparlo.

Attitudine al *problem solving*. Create una visione, cercate di capire se ci sono problemi, definitela, rifinitela, iteratela. "Fate tre o quattro visioni e poi **fermatevi**".

Capacità espressiva. Fate capire a lui e ai colleghi le **emozioni** e le **esperienze di gioco** che volete generare.

Il risultato si presenterà come *one page* o come elenco, anche in un Google Doc: "non mi aspetto una roba trascendentale, attenzione". Durante la presentazione, gli altri partecipanti, che non fanno parte del gruppo, **agiranno come un team più allargato**: daranno feedback e faranno vedere in concreto il problema di capirsi in un team, quello del mindset e dello sviluppo.

0.6.27 Le domande finali

"Chiusa la parte discorsiva, tiro il fiato e bevo un po' d'acqua." Il relatore interrompe la condivisione dello schermo per preparare la copia delle slide e invita a fare domande, considerazioni, "anche critiche".

0.6.27.1 Un piccolo studio e la visione condivisa

Andrea, visto che nessuno parla, racconta l'impressione del suo gruppo, che viene da un'azienda che di videogiochi ne ha già fatti. Il loro percorso è particolare: non venivano dal mondo dei videogiochi, hanno fatto un gioco mettendo insieme le proprie competenze, partendo "dal lato consumer" e diventando sviluppatori. Molte delle nozioni viste le avevano **in modo istintivo**, e prenderne coscienza è molto importante. Il loro primo gioco l'hanno fatto **senza alcun High Vision Document**, forse perché fra loro tre la visione era molto chiara. Si conoscevano già? Sì: sono amici e **parenti** ("io sono loro cugino, quindi è partito tutto in

famiglia"; e un altro scherza: "come fratello non ho bisogno di un game design document, perché ci capiamo al volo").

Il relatore consiglia comunque, quando si passa al professionale, di usarlo. Lo vede molto nel gioco di ruolo, che spesso nasce da un *master* che coinvolge il proprio gruppo di giocatori nella creazione di un prodotto nuovo: "lì si fa una fatica immane", e l'High Vision Document è importantissimo, perché senza un concetto di visione ci si perde in un bicchier d'acqua. Certo, ci sono sviluppi che nascono lo stesso perché le persone hanno sensibilità molto affini, come il gioco che il creatore di *Monument Valley* ha fatto dopo *Monument Valley*, di cui non ricorda il nome[95].

Andrea riconosce che il documento potrà aiutarli: si presentano nuove opportunità, e durante lo sviluppo ci sono state "più volte" situazioni in cui tutti e tre avevano un'idea chiarissima e poi, confrontandosi, emergevano **discrepanze anche grosse** su ciò che intendevano con la stessa identica frase. "È naturale, normale." Il relatore aggiunge che artisti e scrittori gli chiedono spesso se la visione "tarpi le ali alla creatività": è proprio per questo che ha introdotto la distinzione tra *out of scope* e *out of the box*, con l'esempio riportato nella sezione 0.6.4.2. Che un progetto cambi per esigenze interne, di marketing o altro, soprattutto all'inizio, è normale; ma se ogni problema si risolve cambiando tutto, il gioco non si fa più.

0.6.27.2 Eventi, fiere e playtest

Andrea conferma di averlo vissuto in prima persona. Ci sono però modi per regolarsi a vicenda: loro sono stati "bravi e fortunati" perché hanno portato il gioco a molti **eventi**, cinque o sei l'anno, quasi tutti in Italia e una volta a Londra. Il primo è stato Svilupparti, a Bologna, a un mese dall'inizio dei lavori, e poi ogni due o tre mesi. Questo li ha aiutati moltissimo.

Il relatore ha avuto la stessa esperienza con il suo gioco di ruolo: **le fiere sono una manna**. Danno il **feedback diretto** degli utenti, fanno capire che cosa vogliono, insegnano a **vendere** la propria idea e a **spiegarla**: spiegandola agli altri ci si accorge che alcune cose non sono chiare nemmeno a sé stessi. Andrea concorda: davano per scontate tante cose, perché erano venute in mente a loro, e poi scoprivano che "non era chiaro per niente".

Alla fine, dice il relatore, si vuole che l'utente finale abbia un certo tipo di esperienza. Nel gioco da tavolo un **playtest** si può fare anche in un fine settimana con un prototipo; se va male lo si nota subito. A volte il gioco funziona, ma si torna a casa e ci si accorge che l'esperienza che si voleva **non è quella che i giocatori hanno avuto**. Allora bisogna chiedersi: ha senso portare avanti il gioco? Cambiare completamente l'esperienza voluta? O cambiare alcune cose per avvicinarsi all'esperienza voluta? Viene in mente **No Man's Sky**, partito con una visione molto chiara che però non piaceva alle persone; "e le promesse sono state diverse", osserva un partecipante, "eccome se lo sono state".

0.6.27.3 I grandi team e il crunch

Quando si sviluppa in tanti, con molta posta in gioco, come nelle storie di **BioShock** o di **Red Dead Redemption**[96], deve intervenire davvero qualcuno che non faccia solo lo scrum master, ma che si occupi della **gestione del team**: gli faccia capire come lavorare a livello di business e di collaborazione interna. Andrea commenta che è complesso già in pochi, figurarsi con centinaia di persone. Nel suo team il *project manager* lo faceva lui, nominalmente: è ingegnere informatico, non è il suo mestiere, ma ha seguito corsi per conto suo e ha studiato la metodologia **agile**, molto diffusa nell'informatica. Ora stanno iniziando un nuovo progetto, e sono proprio nel momento di cominciare a fare i documenti che per il primo non avevano fatto.

0.6.27.4 Quanta documentazione?

"Documentazione, croce e delizia": il relatore ne ha fatto molta esperienza quest'anno nel world building, e il problema è capirsi su quanta farne. Sempre di più adora il **one page design**: un **macro concetto per pagina**, per lavagna, a colpo d'occhio. Già arrivare a dire "ho questo enorme progetto: quanti *layer*, quanti

[95] Probabilmente *Florence* (2018), di Ken Wong, *lead designer* di *Monument Valley*, già citato poco prima a proposito della musica.

[96] Il relatore si corregge: intendeva il primo *Red Dead Redemption*, aggiungendo che "anche il 2 ha avuto problemi" ed è stato "veramente tossico a livello di *crunch*".

strati, quanti **sottosistemi** ha?" è importante, perché **ogni sottosistema deve essere sviluppato** e probabilmente deve avere un **responsabile**. Se lo si sa, vuol dire che si è già fatto un buon lavoro di visione, di sviluppo e di ricerca; e a quel punto si ha chiaro anche il **tempo** e l'**investimento economico** richiesti. "È difficile, però si può fare."

0.6.27.5 Allargare il team e le soft skill

Il gruppo di Andrea pensa di **allargare il team** con persone esterne, per la prima volta: finora erano persone che si conoscevano da sempre. Aggiungere esterni, dice il relatore, rischia a un certo punto di far sì che non ci si capisca: è importante avere il **vocabolario** e gli **strumenti** per capirsi con altri professionisti. E le **soft skill**: "è brutto da dire nel 2022", ma tutto il suo 2021 è stato all'insegna di "ci vorrebbero un po' più di soft skill". Quindi:

- **pazienza** e **garbo**;
- **porre domande**, non dare risposte;
- considerare sempre che l'altro magari è in dubbio, dare sempre il **beneficio del dubbio**.

Poi ci sono situazioni tossiche da cui è meglio uscire, "non credo che sia il vostro caso". Oggi, come mostrano anche le notizie recenti (non solo le aziende che lasciano a casa i dipendenti, ma i dipendenti che se ne vanno[97]), le persone sono molto più restie a sacrificare parte della propria salute mentale e della propria vita per lavorare in ambienti tossici.

0.6.27.6 Un publisher cinese e i pattern culturali

Un altro membro del gruppo aggiunge che stanno lavorando con un **publisher**, lo stesso del primo gioco, **cinese**: "puoi immaginare la differenza di cultura, di fuso orario, di lingua". Il relatore manda in chat un link "soprattutto per voi che parlate con la Cina": "non è un link di scienza, è più oroscopo, tra virgolette, però è interessante, vi farà un po' ridere".

Si tratta dei **pattern di comunicazione culturali**[98]: schemi visivi di come le diverse culture gestiscono la comunicazione e il business (come comunicano teoricamente gli americani, e così via). Ne esistono versioni più o meno complesse, alcune fatte solo di linee; tra le più divertenti, "e non so se sono veri", quelle turca e polacca, mentre quella americana è "abbastanza verosimile". È una teoria molto usata anche nelle aziende italiane: al padre del relatore, che lavorava in Poste, hanno fatto un workshop solo su questo. Il relatore invia anche una versione più semplice e un articolo di Business Insider; non ritrova invece un post che la spiegava molto bene ("ho più di 2000 link salvati su Google Chrome").

La versione più semplice e conosciuta, inglese, distingue cinque pattern:

- **inglese**: una linea retta;
- **semitico**: tanti livelli;
- **orientale**: una spirale;
- **romanzo**, "che siamo anche noi": una linea a zigzag;
- **russo**: una linea a zigzag ma anche tratteggiata.

Per i giapponesi, in un'altra versione, è una linea dentro una scacchiera. Fu elaborata, "se non mi sbaglio", negli anni Settanta da Kaplan[99]. Una partecipante chiede su quale struttura antropologica e culturale si basino queste teorie. Quelle di Kaplan su una base antropologica e un po' di business; i *conversational*

[97] La frase nella trascrizione è corrotta ("aziende che lasciano a casa di penne di mani di penne di chi se ne vanno"): la ricostruzione più probabile è quella del testo.

[98] Il relatore li chiama "Global Communication Pattern" e "Communication Patterns Theory" e suggerisce di cercare "communication patterns culture".

[99] Il relatore dice "Alan Kaplan", con molte esitazioni. Si tratta quasi certamente di Robert B. Kaplan, il linguista che nel 1966 pubblicò *Cultural Thought Patterns in Intercultural Education*, da cui vengono gli schemi della linea retta, delle linee parallele, della spirale e dello zigzag.

pattern, invece, derivano interamente da business e finanza, "quelle cose da venditore porta a porta": sono **empiriche** ("esatto, empirica, brava"). Per questo sono "un po' un oroscopo": a volte ci azzeccano, a volte sembrano un meme. Ma è sempre utile conoscerle.

0.6.27.7 Le domande di Luca: gli otto piaceri, l'MDA e il flusso di lavoro

Luca Brunetti, che ha avuto problemi con le cuffie, fa due domande. Gli otto tipi di divertimento di LeBlanc: sono nelle slide? Sì, alla numero **160**; molte slide sono immagini. Poi chiede conferma: nell'MDA i giocatori partono dalle estetiche per arrivare alle meccaniche, mentre i game designer fanno l'opposto? "Teoricamente sì", ma è una **teoria in evoluzione**. Dopo aver fatto la visione e stabilito tutto, la prima cosa che il designer vuole stabilire, per ottenere una certa esperienza, sono i **verbi**: le azioni del personaggio, le regole, che cosa può e non può fare, il *core loop*. Poi ragiona su quali dinamiche si creano e se dinamiche e meccaniche portano all'esperienza voluta.

Di recente, aggiunge, l'MDA viene riletto **in chiave ciclica**: è come se attorno a ciascuno ci fosse una sfera, uno spettro che passa dall'estetica alle dinamiche alle meccaniche; il giocatore è **immerso** nella sfera e la vede dall'interno verso l'esterno, il game designer la vede **dall'esterno**, al contrario. Perché a volte le estetiche finiscono nelle meccaniche, le meccaniche nelle dinamiche. Quindi la regola vale "fino a un certo punto": si trovano cose per cui ci si chiede se siano meccaniche o dinamiche, e bisogna saperle riconoscere, anche perché meccaniche e dinamiche si legano molto ai **player type**. A chi piace completare i giochi, prendere gli *easter egg* e tutti gli oggetti, piaceranno probabilmente certe meccaniche e dinamiche più di altre.

Luca chiede infine di rivedere il **flusso di lavoro** del giorno dopo, e il relatore ("dammi pure del tu, che mi fai sentire vecchio") indica le slide **43** e, in versione più semplice e visiva, **32**, e lo ripete per tutti:

1. prendere uno dei tre giochi;
2. cercare un'idea, una visione per cambiarne una componente, una parte: il cosiddetto *hack*[100];
3. fare un minimo di **ricerca** sul tipo di giocatore e su come la propria idea, implementata nel gioco, cambierà il *player type*;
4. fare un documento compatto di **analisi del gioco di partenza**, anche *one page*;
5. creare, probabilmente nelle due ore finali ("in un'ora, secondo me, avere delle idee, fare un minimo di ricerca e due considerazioni si può"), il **one page High Vision Document**, che dica tutto ciò che serve, secondo il gruppo, per raggiungere l'esperienza ludica desiderata, usando gli strumenti visti.

Che cosa metterci è una scelta: il *player type* sì o no; le emozioni ("ve lo consiglio"), magari non quelle del 6-11 ma i piaceri di LeBlanc, che il relatore usa di solito perché li trova più completi; dei pilastri; delle immagini, una sorta di *mood board* ("perfettissimo"). L'importante è esserne soddisfatti e considerarlo un High Vision Document che, se si dovesse davvero implementare il gioco, **funziona**.

0.6.27.8 One page e High Vision Document sono la stessa cosa?

Luca chiede ancora se il *one page* e l'High Vision Document siano la stessa cosa. "Non sempre." L'High Vision Document si tende a tenerlo il più stretto e ridotto possibile, e il consiglio è di farlo in forma di *one page*, ma non sono identici. Il *one page design* è una **metodologia di sviluppo di documenti**, spiegata da Stone Librande nella sua *talk*: prendere ogni parte di un gioco (lui fa anche l'esempio della catena alimentare di alcuni nemici di un videogioco) e farne una pagina, un grafico visivo con riferimenti visivi e poco testo, che stimoli la visione e la visualizzazione del progetto. "*One page* non significa quattro, eh": significa che sta tutto **in una singola tavola di lavoro visibile a colpo d'occhio**. Se lo si fa su Miro e per leggerlo bisogna zoomare, non è un *one page*[101]. Qualcuno scherza: "abbiamo una consegna per lunedì di questa cosa, quindi aiuto". "State usando questo workshop per lavorare", ride il relatore.

[100] Nella trascrizione "color hack, o hack che si voglia dire": forse "hack" detto in due pronunce diverse.

[101] La trascrizione aggiunge "no se il vostro one page è una tre va benissimo", frase non ricostruibile: forse "se il vostro one page è una [tavola] A3 va benissimo".

0.6.27.9 I compiti e la partenza di Michele

Una domanda "molto stupida e veloce": l'analisi e l'*hack* sono da fare per il giorno dopo o si fanno durante il workshop? "No, no, no, no": si fanno domani, durante il workshop. Chi vuole usare le slide per fare qualcosa oggi è liberissimo, "ma io oggi come oggi mi riposo". Michele saluta ("grazie mille, a domani"), e così via via altri partecipanti.

0.6.27.10 Il game designer durante lo sviluppo

Marco[102] premette che è la sua primissima esperienza vicina al mondo lavorativo dei videogiochi, di cui non sa nulla. Si è visto come si lavora **a monte** di un progetto: il game designer lavora anche **durante** la produzione, perfino verso la fine, quando il gioco viene programmato e si ha una risposta visiva di ciò che si è progettato?

"Assolutamente sì." All'inizio si ha solo la visione e un'idea di quali potrebbero essere meccaniche, dinamiche ed estetiche, ma poi vanno **sviluppate**: non basta pensarle. Il lavoro si affida ai programmatori, e i game designer si **specializzano**: c'è il game designer che fa un po' lo scrum master, quello che cura la documentazione, quello che si occupa del gameplay vero e proprio, il **gameplay designer**.

Marco racconta che è proprio ciò che lo attira di più, il **level design** e il gameplay: sta giocando *God of War Ragnarök*, "un capolavoro", con linee guida chiare, "tutto perfetto"; non può dire lo stesso di *A Plague Tale: Requiem*, dove in alcuni punti il *level design* non è abbastanza "*affordabile*", cioè non dà abbastanza informazioni per capire dove andare e che cosa fare, cosa che stanno notando in molti. Il relatore conferma di essersi spaesato anche lui in un ambiente: "gameplay designer e level designer, soprattutto level designer". È ciò che, giocando, si vede di più.

Durante lo sviluppo, aggiunge, anche chi fa **narrative design** o **world building** non sta con le mani in mano: ogni volta che c'è un **prototipo** bisogna capire se ciò che è stato implementato raggiunge ancora l'obiettivo. Chi implementa ha spesso la competenza tecnica per dire se funziona: "io dico: **fidati degli esperti**". Se il programmatore dice che due secondi in più di reazione del nemico non risolvono il problema, fidati. Ma è comunque un **lavoro di team**: si vuole arrivare a un obiettivo, e chi ha chiaro l'obiettivo è di solito il game designer. È un **lavoro strategico**: una strategia per comunicare al giocatore in modo differito e **manipolarlo**, nel senso di farlo andare dove lui vuole fare esperienze e dove tu vuoi che vada.

0.6.27.11 Gruppi e aspettative

Sui gruppi del giorno dopo[103]: se qualcuno resta in più, "lo prendiamo io e Michele e ci facciamo un gruppo a parte". Non tutti i gruppi devono scegliere lo stesso gioco. Il relatore e Michele replicheranno l'esercizio insieme ai partecipanti, pronti a rispondere alle domande, "con le slide in fronte". Di solito ha visto fare questo esercizio soprattutto con **Lasers & Feelings**, un gioco di ruolo che, contro l'idea dei manuali enormi, sta in un'unica pagina ("bellissima, geniale"): spesso i team riescono a farne un *hack* **completo**, un prodotto finale, "fattibilissimo in tre ore". Ma ricorda anche tre ragazzi che hanno fatto un *hack* di **Super Mario** "fantastico": non sa se l'abbiano portato avanti, ma il documento che gli avevano mostrato era molto interessante.

0.6.28 Chiusura

Prima di salutare, un'ultima curiosità personale "che non c'entra": venendo "dal lutto di *The Last of Us*", *A Plague Tale* è consigliato come seguito? Il relatore non ha ancora finito *Requiem* (chi l'ha giocato gli dice che è a metà). Il primo gli era piaciuto moltissimo e gli aveva dato tanto a livello di impatto emotivo: "una bella esperienza". Il secondo anche, ma con alcuni punti in cui la **sospensione dell'incredulità** viene rotta, perché alcune cose sembrano inserite apposta per **rispondere alle critiche** fatte al primo. "Quando questo

[102] Nella trascrizione "Marco Ronchi"; è probabilmente lo stesso Marco esterno presentatosi all'inizio, che infatti dice che questa è la sua "primissima esperienza" nel mondo lavorativo dei videogiochi.

[103] La trascrizione qui è molto confusa ("e gli eviti di un po' è un po' è un po' è un qualcuno che è extra"): il senso è che chi resta senza gruppo lavorerà con il relatore e Michele.

succede, si rompe": invece **bisogna andare per la propria strada**. È, in fondo, un'ultima applicazione del discorso sulla visione.

"Ragazzi, scappo, che ho la pasta in piatto." "Buon appetito." Il relatore ringrazia tutti, augura buona giornata e dà appuntamento al giorno dopo per la parte pratica.

Punti chiave del capitolo

- Il workshop non insegna il game design, ma il **mindset del game designer**, per far **comunicare e collaborare** ruoli diversi: scrittori, artisti, world builder, musicisti, programmatori.
- I problemi tipici in team: **linguaggi diversi** ("conflitto" nel game design non è "tensione drammatica"), *brief* nebulosi, **documenti che nessuno usa**, **piani dimenticati**. Dal caos possono nascere grandi giochi, ma con grandi sacrifici, e spesso si fallisce.
- Distinguere **out of scope** (lontano dalla visione: da evitare) da **out of the box** (soluzione creativa a un problema: benvenuta).
- Il game design è **50% arte e 50% scienza** (Diego Ricchiuti): il mindset fa da **argine** senza irrigidire.
- I **bias** sul game designer ("ma di lavoro che fai?", "giochi tutto il giorno", "disegni", "non fai niente") e sul game design ("non serve studiarlo") diventano spirali se non si comunica.
- Il game design è una **metodologia**: scegliere il metodo e gli strumenti, anche presi da altre discipline (Gris e gli stadi del dolore). Le competenze "tangenziali" contano.
- Il mindset è **pensiero critico**, **capacità espressiva** e attitudine al **problem solving**.
- Il game designer vuole **un'esperienza per un pubblico**; "divertimento" è una parola povera, meglio **piacere**. Servono strumenti, tecnologie, ricerca, sperimentazione e feedback; e servono soldi, senza pudori.
- L'**idea** è ciò che si ha, la **visione** è dove si vuole arrivare (l'obiettivo superordinato). Non tutti i giochi sono per tutti: bisogna ridurre la **decodifica aberrante**.
- Il gioco è una **comunicazione differita** con interazione: "la ricerca volontaria di piaceri ed emozioni, agendo con le possibilità e le restrizioni di un sistema arbitrario dotato di significato".
- **Design principle** e **pilastri** (God of War: combattimento, padre e figlio, esplorazione) tengono la visione; l'**iterazione** trasforma l'idea (da Killzone con i dinosauri a Horizon).
- **MDA**: il giocatore va da estetiche a meccaniche, il designer al contrario, ma partendo da un'ipotesi di estetica. **Gameplay is king**. L'**ADMDA** aggiunge analisi ed estetica, sviluppo e target; *play* e *game* sono più piccoli del *development*.
- La **documentazione** (croce e delizia) deve essere concreta, visiva e compatta: strumenti di brainstorming (*crazy eight*, *plus* e *delta*, *Fibonacci vote*, *flowchart*) e di visualizzazione.
- La **pipeline** si riassume in domande (prima/dopo, chi fa che cosa, chi ritarda se ritardo). Evitare il **micromanagement**.
- Chiedersi sempre **chi è il giocatore** (Bartle, Prismatic Model e altri modelli).
- L'**High Vision Document** contiene gli elementi core (temi, pilastri, **obiettivo del gioco**, personaggi, regole, setting, pubblico, emozioni, USP, must e nice to have) ed è sempre visibile; il **one page design** (Stone Librande) dedica una tavola, leggibile a colpo d'occhio, a ogni layer del gioco. Usare **termini chiari per tutti**.
- Le **meccaniche** sono vincoli e rinforzi, verbi e regole; le **dinamiche** sono scelte che cambiano lo stato di gioco, anche emergenti; i **loop** (*core loop*) e i **sistemi** (Hades) generano il gameplay.

- La **manipolazione** è positiva: porta il giocatore verso l'esperienza. Contro noia e frustrazione servono **pacing**, attrito e **feedback loop negativi** che bilancino quelli positivi, per arrivare al **flow**.
- Tre idee: **lo strumento corretto per il giusto lavoro**, **le idee non sono nulla senza implementazione**, **anche il giocatore può risultare confuso**.
- Le **estetiche** guidano: superfici, glossari, regole scritte bene (attenti alle *house rule*), tutorial, dialoghi, **icone** (generiche) e **idiomi** (specifici del gioco), emozioni (Plutchik, 6-11, piaceri di LeBlanc), ambienti e **forme** (cerchio, quadrato, triangolo).
- Nel workshop si fa un **hack** di Lasers & Feelings, Ticket to Ride o Super Mario: visione, pubblico, ricerca, analisi dei verbi del *core loop*, **one page High Vision Document**.
- Fiere e playtest danno feedback diretto; nei team che crescono servono vocabolario comune, gestione del team e **soft skill**: pazienza, garbo, domande, beneficio del dubbio. Il game designer lavora anche durante lo sviluppo, e si fida degli esperti.

Chapter 0.7

Hackerare un gioco: la giornata pratica del workshop

Questa settima lezione zero è la **seconda giornata** del workshop sul mindset del game designer cominciato nel capitolo 0.6. Se la prima era la parte "discorsiva", questa è la parte **pratica**: "tutta la parte teorica ce la siamo tolta ieri". I partecipanti, divisi in **quattro gruppi da tre**, scelgono uno dei tre giochi annunciati il giorno prima (*Lasers & Feelings*, *Ticket to Ride* e *Super Mario Bros.*), ne analizzano le **azioni** e il **core loop**, immaginano un **hack**, cioè una versione modificata del gioco, ne definiscono la **visione**, l'**esperienza desiderata** e il **pubblico**, e infine provano a cambiarne le **meccaniche**, fermandosi prima delle dinamiche e delle estetiche. Il tutto deve confluire in un **one page High Vision Document**, secondo il programma esposto alla fine della prima giornata (sezione 0.6.25).

Conducono ancora **Daniele**, game narrative designer e world builder, e **Michele**, studente del terzo anno di Game Design, che stavolta non fa solo da assistente: mentre i gruppi lavorano nelle loro stanze, i due **svolgono l'esercizio in diretta**, nella stanza principale, costruendo su Miro un loro *hack* di Super Mario, il "Bromance Mario", che fa da esempio per tutti. Il workshop si svolge di nuovo **online, su Zoom**, di mattina, e si chiude all'ora di pranzo, come il giorno prima; lo scenario temporale è quindi lo stesso, **novembre 2022**[1].

Valgono le stesse avvertenze di lettura del capitolo precedente. La trascrizione è automatica: storpia i nomi propri e i termini inglesi (il *core loop* diventa "corlupo" o "corru", l'High Vision Document "iVision" o "revision document", Bartle "Bartol", Ticket to Ride "Stick it to Ride" o "Ticatoride"), ripete in loop alcune frasi[2] e a volte mescola le voci di persone diverse. In più, questa giornata ha una difficoltà propria: **il lavoro nelle stanze separate non è registrato**[3]. Di ciascun gruppo si conosce quindi solo ciò che riferisce in plenaria, mentre il ragionamento di Daniele e Michele è riportato quasi per intero, compresi i passaggi in cui le due voci si alternano rapidamente; dove non è chiaro chi parli, si attribuisce l'idea alla coppia. Molti nomi dei partecipanti sono quelli già incontrati nella sezione 0.6.1.1.

Il percorso del capitolo segue quello della giornata:

1. l'attesa iniziale e gli **strumenti di lavoro**, Miro e le altre lavagne digitali (sezione 0.7.1);
2. il **programma** della giornata e che cosa ci si aspetta dai gruppi (sezione 0.7.2);
3. i **tre giochi** presentati da vicino, con le loro regole (sezione 0.7.3);
4. la formazione dei **gruppi** (sezione 0.7.4);
5. il primo passo, l'**analisi di azioni e core loop**: l'esempio di Daniele e Michele e le presentazioni dei gruppi (sezioni da 0.7.5 a 0.7.6);

[1] Si veda la nota sulla datazione nel capitolo 0.6. Il congedo finale della trascrizione è "ciao a tutti, ciao, domenica": se va letto "buona domenica", la seconda giornata cadeva di domenica e la prima di sabato. È un'ipotesi.

[2] Per esempio, alla riapertura delle stanze, "Grazie a tutti" ripetuto una ventina di volte.

[3] Lo dice lo stesso relatore verso la fine: la registrazione riguarda la stanza principale; le stanze secondarie "credo che siano teoricamente private".

6. il secondo passo, **visione, esperienza desiderata e pubblico**: come fare ricerca, l'esempio del Bromance Mario e le visioni dei gruppi (sezioni da 0.7.7 a 0.7.9);

7. il terzo passo, le **meccaniche**: glossari, icone e idiomi, dadi e probabilità, e le meccaniche del Bromance Mario (sezioni da 0.7.10 a 0.7.11);

8. le **presentazioni finali** (sezione 0.7.12);

9. i consigli conclusivi, il **portfolio** e i saluti (sezioni da 0.7.13 a 0.7.15).

0.7.1 L'attesa e gli strumenti di lavoro

La registrazione comincia mentre i partecipanti rientrano nella videochiamata, tra saluti e caffè. Una partecipante racconta di aver dovuto uscire in anticipo il giorno prima per un imprevisto, e di essere stata buttata fuori da Zoom almeno una volta ("Zoom ogni tanto mi butta fuori anche a lezione"). Il relatore fa il punto sulla prima giornata: è andata secondo i piani, con una ventina di minuti di domande, "più o meno un quarto d'ora", e qualche minuto in più verso l'una. Oggi, conferma, c'è "la parte pratica". Chi non ha ancora le slide le trova in chat, e la segreteria le invierà comunque a tutti[4].

Nell'attesa invita a fare domande sulla parte del giorno prima o a chiedere un breve riepilogo di ciò che si deve fare, e rassicura: la parte di oggi si vedrà ***step by step***, "non vi faccio entrare nella parte pratica senza un minimo di orientamento".

0.7.1.1 Servono strumenti particolari?

Un partecipante chiede se si debba installare qualcosa. No: il relatore si definisce "una persona molto *easy* sugli strumenti". Basta l'accesso a **Google Drive**, magari per aprire qualche **Google Document** e lavorare in modo sincronizzato; per il resto ognuno usi gli strumenti con cui è più comodo. Questa parte non richiede tecnologie specifiche: i workshop dell'accademia che ne hanno bisogno le fanno installare molto prima. Chi vuole può registrarsi su **Miro** o su un altro strumento di visualizzazione; ma "potete usare anche un Google Document, lo ripeto, anche carta e penna: io giro sempre con i quaderni, la tecnologia migliore del mondo".

0.7.1.2 Miro e le altre lavagne digitali

Il relatore mostra comunque Miro a chi non lo conosce. Lui ha un account privato con il piano da **freelance**, a **10 euro al mese**, che usa anche per lavoro; ma esiste un piano **gratuito**, con due limiti:

- tutte le *board* sono **condivise con tutti i membri del team**: se si hanno altri utenti, tutti vedono le proprie lavagne;

- si possono creare *board* all'infinito, ma solo **le tre più recenti sono modificabili**: creando la quarta, la prima diventa di sola visualizzazione[5].

È uno strumento di visualizzazione, e ne esistono altri simili:

- la piattaforma **Atlassian**, quella di **Jira**, "uno dei software più conosciuti per fare *project management* e *product management*, uno dei tool più conosciuti nell'industria", che ha "se non mi sbaglio" un suo strumento di lavagna[6];

- la lavagna digitale di **Microsoft**, **Whiteboard**, che gli hanno detto essere stata potenziata di recente ed è molto simile;

[4] Il relatore condivide lo schermo dal profilo dell'accademia ("me l'ha aperto con il profilo di Hive", cioè di AIV, "ma va bene") e promette di mostrare più tardi anche alcune sue *board*.

[5] Sono le condizioni del piano gratuito di Miro descritte dal relatore nel 2022; potrebbero essere cambiate.

[6] La frase nella trascrizione è corrotta ("Se mi sbarro, la piattaforma Atlassian è uno"). Atlassian offre in effetti lavagne digitali all'interno di Confluence.

- la **Google Jamboard**, che invece non consiglia, "a meno che non l'abbiano cambiata recentemente"[7]: è carina ma **molto limitata**. Si può disegnare, si possono mettere post-it, ma bisogna prima creare il post-it e poi inserirlo, "che è un po' problematico quando si fa brainstorming"; si possono caricare file, fare forme e disegni che però "rimangono solo per un tot", scrivere testo.

Strumenti come Miro sono secondo lui molto più utili, anche perché hanno la tecnica del **frame**: una volta fatta una lavagna, la si può esportare come CSV ("una cosa nuova") ma soprattutto **come immagine**. Si può quindi fare direttamente il *one page* lì, lavorandoci in sincronia con il team, e una volta finito lo si esporta con un clic e lo si mette in qualsiasi strumento di lavoro: Slack ("adoro Slack"), Discord, "qualsiasi cosa".

0.7.1.3 Le slide di riepilogo

Il relatore si è preparato alcune **slide di riepilogo**, che servono a lui per spiegare la giornata: sono una parte delle slide del giorno prima, estratta e rimessa insieme. Ciò che servirà di più ai partecipanti è avere aperte le **slide di ieri**. Anche lui chiude le sue e apre la copia per gli studenti, condivisa su Google Slides, così vede dove sono tutti: se qualcuno ha problemi su una slide, gli basterà cliccare sul suo avatar e "teoricamente Google dovrebbe portarmi lì da voi. Non sempre funziona".

0.7.2 Il programma della giornata

La prima slide di riepilogo ricorda che cos'è il game design: una **metodologia di sviluppo** (sezione 0.6.6). Ciò che il relatore vorrebbe vedere, quindi, è che i partecipanti **cerchino metodi** per sviluppare idee e arrivare a un'esperienza partendo dalla propria visione, usando gli strumenti visti il giorno prima ma anche la propria competenza, la propria *expertise*, la propria esperienza "multidisciplinare, tra virgolette": "potete prendere e rubare a qualsiasi disciplina, io lo faccio sempre".

Due raccomandazioni di metodo:

- si fa design **per giocatori reali**, "non per giocatori fittizi, non per noi": bisogna capire per chi lo si sta facendo;
- non per forza partendo dal pubblico, ma **partendo dalla visione dell'esperienza** che si vuole. Una volta che si ha la visione dell'esperienza, si capisce qual è il tipo di giocatore.

E soprattutto: bisogna **applicare il mindset**, non solo i *tool*. Uno degli obiettivi del workshop è proprio **lo strumento corretto per il giusto lavoro** (sezione 0.6.20): non serve usare tutti gli strumenti visti. Si possono usare i player type di Bartle, o non usarli.

0.7.2.1 I passi

Il lavoro procede per passi, e a ogni passo i gruppi tornano in plenaria a presentare:

1. **scelta e analisi del gioco**. Di ciascun gioco si cercano di capire "banalmente due cose, solo due":
   - le **azioni**, ed eventualmente, come *nice to have*, le **regole**. Le azioni, ricorda il relatore, sono traducibili in **verbi** e sono ciò che il personaggio, e il giocatore, può fare; le regole sono tutto ciò che il giocatore teoricamente non può fare: **vincoli e rinforzi**. È la parte delle meccaniche dell'MDA (sezione 0.6.17);
   - il **core loop**, che non deve essere super dettagliato: basta una riflessione su quale sia, secondo il gruppo, il *core loop* del gioco.

   Non serve costruire nulla: si può prendere uno *screenshot* di Super Mario, o uno dei tre giochi, e analizzarlo direttamente sopra;

[7] Google ha poi annunciato la chiusura di Jamboard, avvenuta alla fine del 2024.

2. creare una **visione per un hack** di quel gioco, cambiando **un singolo elemento** del prodotto: il genere, qualcosa di meccanico, di regolistico, di dinamico, di estetico, "qualsiasi cosa di importante", purché se ne abbia subito una visione;

3. fare "un pelino" di **ricerca e sperimentazione**, cercando i player type (il relatore darà "qualche trucco in più" su che cosa cercare e dove guardare);

4. capire qual è l'**esperienza ludica** che si vuole ottenere, data la visione e l'idea;

5. cominciare a sviluppare un **High Vision Document**.

In termini di framework, l'obiettivo è andare **dalla A dell'ADMDA fino, al massimo, alla M**[8]: **non si arriverà alle dinamiche e alle estetiche**.

0.7.2.2 Che cosa ci si aspetta

Alcuni studenti, con *Lasers & Feelings*, "un gioco molto molto molto compatto", riescono ad arrivare fino all'estetica e quindi a fare un **intero gioco**: è molto semplice da usare, è un PDF che si apre, oppure se ne copia il testo in un documento Google, ed è un attimo cambiare due o tre riferimenti, capire qual è la meccanica del *core loop*, cambiare un paio di meccaniche. Alcuni ragazzi **non hanno nemmeno cambiato le meccaniche**: hanno cambiato solo i nomi, cioè **le icone, gli idiomi** (sezione 0.6.22), i nomi importanti del gioco, un paio di dettagli, l'immagine di Star Trek in fondo alla pagina e il colore blu del gioco, "e me l'hanno fatto in tre ore". Ma è un caso speciale, "non è la regola, è l'eccezione"[9].

Con *Ticket to Ride* e *Super Mario Bros.* il massimo a cui si può aspirare è un High Vision Document che contenga, "attenzione", non tutta la ricerca e tutta l'analisi, ma **alcuni dati importanti**:

- che cosa ha dato la **ricerca**;
- qual è l'**analisi**, soprattutto quella sul prodotto: quali **azioni** e quale **core loop** si sono trovati;
- qual è la **visione**, con un paio di **strumenti di visualizzazione**;
- il **development**: quali **scelte di design** si vogliono applicare per raggiungere l'esperienza voluta, e qual è la **target audience**;
- facoltativa, un'ipotesi di **pipeline**: se si dovesse davvero sviluppare il prodotto, dividendolo in grossi livelli di sviluppo, che cosa verrebbe prima e che cosa dopo? Prima la parte narrativa o prima il *level design*? "Così, ipoteticamente" (sezione 0.6.13);
- una **riflessione su come i cambiamenti e l'esperienza voluta porterebbero a modificare le meccaniche e le azioni**: si lavorerà "molto a stretto giro" su meccaniche e *core loop*.

"Io mi fermerò lì." Chi ha la voglia, il tempo e la dedizione di andare oltre nelle tre ore è libero di farlo; ma il relatore non si aspetta più di un High Vision Document con la visione, il concetto del gioco, ciò che si vuole ottenere, magari l'analisi del *core loop* del gioco di partenza, le modifiche che si vogliono portare per arrivare all'esperienza voluta, un paio di riferimenti a come farlo, "magari una bella immagine, una visualizzazione di tutto, fine".

0.7.3 I tre giochi da vicino

0.7.3.1 Lasers & Feelings

Lasers & Feelings è un gioco di ruolo *one page*, che si trova online **tradotto in italiano dal relatore** ("perdonatemi") insieme alla sua "cricca" di giocatori di ruolo, sul sito **Storia del Ruolo**, il suo vecchio blog; ma lui passa direttamente il link al PDF su Drive, così non serve aprire il sito. "È tutto qui, il gioco":

[8] "Arrivare dalla A di A-D-M-D-A fino al massimo alla M di A-D-M-D-A": cioè dall'analisi, attraverso lo sviluppo, fino alle meccaniche (si veda la sezione 0.6.11.4).

[9] La trascrizione dice "non è la regola, è l'ec, è la regola": evidentemente "è l'eccezione".

- una **parte introduttiva** che spiega che cos'è narrativamente il gioco: si è l'equipaggio di un'astronave da esplorazione interstellare "molto simile all'*Enterprise*"[10];

- la **creazione del personaggio**, che avviene scegliendo uno **stile**, un **ruolo** e un **numero da 2 a 5**. Un numero alto significa che il personaggio è bravo con i **laser**: tecnologia, scienza, fredda razionalità, calma, azioni meticolose. Un numero basso significa che è migliore nei **sentimenti** (*feelings*): intuizione, diplomazia, seduzione, azioni sfrenate e passionali. Poi si dà un nome al personaggio, si dice che cosa possiede e si fissa l'**obiettivo**. Qui il relatore si ferma: l'obiettivo è "importantissimo", "uno dei punti più dimenticati della costruzione di un gioco da tavolo, ma di un gioco in generale"; a volte apre i documenti degli studenti[11] e dice: "ragazzi, ma non avete detto qual è l'obiettivo del gioco per il giocatore". Interessante, nota, che in *Lasers & Feelings* **l'obiettivo del giocatore e quello del personaggio sono separati**: sono due cose diverse;

- la **creazione della nave**;

- la **meccanica di base**. Si usano **dadi a sei facce**: si lancia un D6 per vedere come va, e si aggiunge un D6 extra se si è preparati e uno se si è esperti. Se si usano i **laser**, cioè scienza e ragione, bisogna tirare **sotto** il proprio numero: più alto è il numero, più si è avvantaggiati. Con un personaggio molto scientifico, con 5, si può fallire una prova di laser solo con un 6. Se si usano i **sentimenti**, bisogna tirare **sopra** il proprio numero. Si confronta ogni dado e si contano i successi: più successi si fanno, più vantaggio si ottiene, e quindi è sempre possibile fallire ed è sempre possibile avere successo. Caso speciale: se un dado fa **esattamente il proprio numero** (si è scelto 4 ed esce un 4), quel dado è un **"laser feelings"**, una sorta di risultato speciale[12];

- come **creare un'avventura** in poco tempo, tirando alcuni D6 e confrontando i risultati con le tabelle[13], e "due o tre indicazioni per il master". "Finisce qua."

0.7.3.2 Ticket to Ride

Per *Ticket to Ride*, cliccando sull'immagine della slide 162 si trova il **regolamento in PDF di Asmodee**, quello del gioco base. Sono **quattro pagine**: "fantastica sta cosa, eh? Avere un gioco che si riassume in quattro pagine di regolamento, di cui una è la copertina, sfido". L'unica critica che il relatore fa al PDF è che non hanno messo le pagine affiancate, per una lettura migliore; "ma va bene, li perdoniamo". Il contenuto della scatola:

- una mappa della **rete ferroviaria del Nord America**;

- **240 vagoni** colorati, 45 per ogni colore[14];

- **144 carte illustrate**: le carte vagone divise in colori (violetto, bianco, blu, giallo, arancio, nero, rosso, verde) più le **locomotive**, che fanno da jolly; una **carta riassuntiva**; i **biglietti destinazione**, che sono gli obiettivi; una **carta bonus** di 10 punti per il percorso più lungo;

- i segnapunti in legno, per tenere il conto dei **punti vittoria**, segnati attorno al tabellone.

Ogni biglietto destinazione indica **due città che devono essere connesse**, "in qualsiasi modo": non dice quale strada fare, "dovete connetterle, punto". A fine partita si girano i biglietti e si prendono i punti. Un biglietto collega raramente due città vicine: di solito due città separate almeno da una terza, e ci sono biglietti che vanno, per dire, da Seattle fino a El Paso o a New York, e "valgono un botto di punti".

L'**obiettivo** è accumulare il maggior numero di punti, in tre modi:

[10] Nella trascrizione "l'equipaggio di un incognito restellare": nell'originale l'astronave è l'*interstellar scout ship* Raptor.
[11] Nella trascrizione "apro delle tasche di studenti": forse "le task" o "le cartelle".
[12] Nel regolamento originale un risultato "Laser Feelings" conta come successo e dà al personaggio un'intuizione: si può fare al GM una domanda a cui deve rispondere sinceramente.
[13] Il relatore dice "si tirano 4 di 6"; nell'originale i dadi si tirano, uno per tabella, sulle tabelle della minaccia, di ciò che vuole e del suo piano.
[14] Così nel regolamento: cinque colori da 45 vagoni, più alcuni vagoni di ricambio.

- **controllando una tratta** tra due città adiacenti: si spendono carte dello stesso colore (per esempio sei carte blu) e si posizionano altrettanti vagoni; una tabella riassuntiva sul tabellone dice quanti punti vale ogni tratta (una tratta da sei vagoni ne vale 15, e si avanza di 15);
- completando con successo un percorso tra le due città dei propri **biglietti destinazione**;
- completando il **percorso più lungo**, che vale altri 10 punti.

Il **turno** del giocatore, ammette il relatore, prevede **tre azioni, non due**, e se ne deve eseguire **una soltanto**: pescare carte vagone, controllare una tratta (cioè posizionare i vagoni), pescare biglietti destinazione. La terza azione non era nella slide del giorno prima (sezione 0.6.18.2) perché è **molto rara**: tutti i giocatori cominciano con delle destinazioni[15], e chi le ha finite può pescarne altre, ma chi pesca destinazioni nel suo turno non può fare nient'altro: è un'azione che si fa due o tre volte in una partita. Il regolamento spiega poi come pescare, come controllare una tratta e come posizionare i vagoni: si spendono, per esempio, tre carte blu e si mettono i propri vagoni verdi sulla tratta blu; sulle tratte **grigie** si possono spendere carte di qualsiasi colore, purché tutte dello stesso ("io avevo tre carte verdi e le ho spese"). Seguono la tabella dei punteggi, la fine della partita e il calcolo dei punti: "nulla di eccezionale".

0.7.3.3 Super Mario Bros.

Per *Super Mario Bros.* il consiglio è di cercare online delle *walkthrough*[16], guardare video e "quant'altro". Michele, come si vedrà, lo gioca direttamente su un emulatore.

0.7.3.4 Domande prima di partire: i punti dell'analisi e la USP

Un partecipante chiede di ripetere i punti da dichiarare nell'analisi: uno è il *target*? Teoricamente il relatore si aspetta:

- la **visione**;
- l'**esperienza desiderata**, in termini di piaceri, emozioni, "quello che volete";
- il **target**, se si vuole, descritto brevemente;
- il **core loop** originario e le **meccaniche originarie** del gioco, e quelle che si vogliono cambiare e come, se se ne vogliono cambiare;
- tutto ciò che può essere utile.

"Vi sto buttando in una sorta di oceano pieno di squali": in realtà la scelta dovrebbe essere dei partecipanti. Alla **slide 75**, quella sull'High Vision Document, si legge che contiene temi, pilastri, obiettivi, personaggi, regole, setting e narrativa di base (secondo il relatore "il minimo possibile"), e sotto che può contenere anche pubblico, emozioni, stile, USP, design, setting, narrativa, estetiche: tutto ciò che può servire allo sviluppo (sezione 0.6.15). Analizzato il gioco, sarà più o meno chiaro che cosa serve e che cosa no. E, "essendo stringati di tempo", i partecipanti proveranno la sensazione che hanno tutti i game designer quando sviluppano: **dovranno fare delle scelte**. Non si può mettere tutto: si sceglie ciò che può essere utile.

Un altro chiede che cosa fosse la **USP**. È l'**unique selling point**: in soldoni, l'elemento che, secondo chi lo progetta, rende il prodotto **unico al mondo**, o almeno per il pubblico di riferimento. Un prodotto può contenere tantissime cose che ricordano tanti giochi: un tripla A "alla God of War", con un barbaro vichingo come protagonista e un figlio, magari con il concetto degli Assassini e dei Templari ("vedi come sto mettendo dentro tantissime cose che sembrano tanti giochi")[17]. Ma ci sarà sempre una cosa che lo rende unico: per esempio, il protagonista è stato **maledetto dagli dèi**, nel senso che è morto e ritorna in vita per salvare il figlio, e questo lo distingue, teoricamente, da God of War. Più l'elemento è unico nel suo genere, più il prodotto si distingue. Trovarlo è difficile, ma si pensi alla prima analisi: cercando online "*Lasers &*

[15] Il relatore dice "due destinazioni"; nel regolamento se ne ricevono tre e se ne tengono almeno due.
[16] Nella trascrizione "ball through".
[17] In mezzo all'esempio il relatore si interrompe: "bravo Michele per l'emulatore di Super Mario, stavo per dirtelo".

Feelings hack" si trova "un botto" di giochi fatti su *Lasers & Feelings*; magari nessuno si è preso la briga di farne uno ispirato a **Tomb Raider**. La USP sarebbe allora: "il mio è l'unico hack di *Lasers & Feelings* ispirato a Tomb Raider".

0.7.4 I gruppi

Mentre cerca come aprire le stanze separate di Zoom ("ogni volta non mi ricordo mai dov'è il *tick* da mettere"), il relatore conta i presenti: tolto l'organizzatore sono **quattordici**, quindi **quattro gruppi da tre**, più lui e Michele, che resteranno nella stanza principale a disposizione per aiutare. Se le stanze non funzionassero, i gruppi possono usare altri canali, un Discord o qualunque luogo dove scambiarsi messaggi[18]. Le stanze, spiega, si chiamano "sessioni secondarie": lui le assegna manualmente, i partecipanti le trovano cliccando sui tre puntini o sulla voce "sessioni secondarie" nella barra di Zoom e scorrendo fino a "sessione 1", "sessione 2" e così via; chi non ci riesce viene inserito da lui.

I gruppi sono composti cercando di **"pareggiare le esperienze"** ("perché sennò è un delirio"):

Sessione 1. **Andrea, Matteo e Luca**, i tre dello studio indipendente, che qui confermano il nome: **Bit9 Studio**[19].

Sessione 2. **Filippo, Luca Brunetti e Jacopo**, studenti del primo anno di Roma. Filippo, "da quello che ho capito", non può parlare molto, quindi lo si mette con i colleghi che conosce. Si offrivano in quattro, ma i gruppi sono da tre, e uno di quelli che possono parlare viene spostato in un altro gruppo[20].

Sessione 3. **Flavio**, anche lui dell'accademia, **Claudio**, dal corso di programmazione, e **Francesco**.

Sessione 4. **Giulia, Alice e Giovanni**, i tre esterni.

Prima di entrare, il relatore ricorda che chi ha problemi può uscire dalla stanza e chiamare lui o Michele: "siamo in due, quindi ci possiamo sfruttare". **Primo compito** della giornata: scegliere il gioco, analizzarne le azioni e il *core loop*. "Non abbiamo fretta." Su richiesta, lo ripete: scelta del prodotto, analisi di meccaniche e *core loop*, "brevemente"; dieci minuti o un quarto d'ora, "non deve essere un'analisi incredibile". Dopo, il secondo passo sarà creare la visione. Ci si rivede "alle 10.45, 10.50".

0.7.5 Primo passo: l'esempio di Daniele e Michele

Rimasti soli nella stanza principale, i due scelgono anche loro un gioco. Michele sta già giocando a Super Mario ("ah, che aveva il cranio, anche bisogna abbassarsi": si può infatti **abbassare** Mario). Si scelgono quindi un **hack di Super Mario**, "gioco storico", e aprono una *board* su Miro intitolata "Super Mario Hack".

0.7.5.1 Le azioni

Le azioni sono "abbastanza *easy*": **jump** e **run**. Poi c'è un'azione speciale, abbassarsi. Il nome dà da discutere: *crunch*? ("mi ricordi Tomb Raider 3?"), *sink*, "piegarsi, accorciarsi"... si scrive **crouch**, poi, cercando la lista ufficiale dei comandi, si scopre che in Super Mario Bros. si chiama **duck**. Il *duck* si può fare **solo se si è cresciuti** ("*only grow*"): "non in tutti i titoli", ma nel primo Mario ci si può abbassare solo da grandi[21]. Michele aggiunge il **dash**: quando si corre e si lascia il tasto dello scatto cambiando direzione, Mario scivola. Ma **run** e **sprint** vanno messi insieme, "perché è proprio l'azione di base".

[18] Qualcuno, forse Andrea, commenta: "noi di Bit9 non usiamo di solito Discord". In attesa, un collega di Filippo non riesce ad accedere a Zoom, e il relatore manda a Filippo il link della stanza in privato perché glielo passi. Si legge anche l'avviso, probabilmente scritto in chat da Filippo, "sono un bel po' stordito causa malattia, quindi non riesco a lavorare molto bene"; il relatore risponde che non c'è problema e lo invita a mettersi con i colleghi del primo anno di Roma.

[19] Nella sezione 0.6.1.1 il nome era rimasto incerto. Qui il relatore chiede "i ragazzi di Bit9, detto giusto?" e la risposta è "sì, giusto, Bit9 Studio". Più avanti il relatore, ripetendo i gruppi, dice "Andrea, Mattia e Luca": è con ogni probabilità un lapsus per Matteo.

[20] Nella trascrizione, tra i nomi, compare anche "Jacopo, Alessandro": forse il collega che non riusciva a entrare, o un errore di trascrizione. Il gruppo definitivo è comunque quello indicato.

[21] Michele gioca su emulatore e ricorda che si usa "shift" per lo scatto.

0.7.5.2 Il core loop

Il *core loop* è: si corre, ogni tanto si salta. Nella slide del workshop (sezione 0.6.18) c'era *run, jump, beat*, ma ora il *beat* sembra ai due "poco efficace": in realtà colpire i nemici è già dentro il salto, "il *beat* è una conseguenza del *jump*". Il loop è quindi molto semplice: **run, jump**. Ciò che cambia è **il motivo per cui si salta**, che può essere diverso:

- **hit brick**, colpire il mattone;
- **hit enemy**, colpire il nemico[22];
- **overcome obstacle**, superare l'ostacolo;
- **acquire item**, prendere un oggetto, aggiunto più tardi: di solito si salta anche per acquisire gli oggetti.

Anche il *crouch*, o *duck*, si fa "un po' di meno", perché si può fare solo da grandi: lo mettono in **viola**, con le frecce, perché lo si può fare solo in certi momenti. E si può fare per più di un motivo: per **evitare un ostacolo** e basta nel primo Mario; nel secondo forse anche per passare in certi punti. Di base, comunque, per evitare un ostacolo, e quindi lo collegano a *overcome obstacle*.

0.7.5.3 I power up

Michele propone di aggiungere lo **sparo**, con il fiore di fuoco: "mi sembra che già nel primo livello c'è". Ma per il momento i due **separano le azioni vere e proprie da quelle che vengono con i power up**. Cercano la lista dei *power up* del primo Super Mario Bros. ("proprio il primo primo primo"): il **fungo** (*Magic Mushroom*), il **fiore di fuoco** (*Fire Flower*), lo **Starman** e il **fungo 1-Up**, che dà una vita in più[23]. Lo Starman rende invincibili: con lui si superano gli ostacoli e si colpiscono i nemici semplicemente correndo, "ma questo non lo mettiamo". Mettono invece **shoot fire**, che fa *hit enemy* con la palla di fuoco e si sblocca **solo ed esclusivamente** con il fiore di fuoco. Il resto lo tengono come *core loop* di base, "molto molto normale". Le azioni che si fanno con i comandi di base sono in **verde**.

0.7.5.4 Le emozioni: una correzione a Dillon

Il relatore riapre l'analisi di Super Mario con il framework 6-11 vista il giorno prima (sezione 0.6.23.1), che "non gli torna molto". Sul *gameplay* è d'accordo: combattere i nemici, correre fino alla fine, scoprire aree nascoste; e acquisire *power up* e monete, cioè **acquire item**, che è un'altra possibilità del salto. Ma non descriverebbe alcune cose con gli istinti scelti da Dillon:

- non **greed**, l'avidità, ma **raccolta** (*collection*): la gestione delle risorse, raccogliere le risorse. Michele concorda: è anche proprio il termine, *greed* è uno dei sette peccati capitali, detto di Super Mario, "un gioco quasi innocente", è troppo forte, anche se *greed* può avere vari livelli;
- **curiosity**, d'accordo; **excitement**, "d'accordissimo"; **joy**, d'accordo;
- ma non **pride**. Qui il 6-11 manca di una cosa che c'è nei piaceri di LeBlanc, l'**evasione**: la gioia viene dall'evasione, "l'istinto di evasione e di libertà";
- e più che **protection**, "probabilmente il raggiungimento di qualcosa, ipotizzo".

Sulla *board* le etichette diventano commenti attaccati alle azioni ("volevo metterle come *label*, ma non me lo fa fare"):

[22] Nella trascrizione "eat enemy" e "eat brick": è chiaramente *hit*.

[23] Michele ricorda anche uno "scoiattolino" che "ti mettono in cima quando continui a morire", ma lo colloca in titoli successivi: si tratta verosimilmente del potenziamento di aiuto (la foglia o la tuta da Tanooki bianca) che i Mario più recenti offrono a chi muore più volte nello stesso livello.

Azione	Emozione o istinto
run e sprint	*evasion*, *joy*
hit brick	*curiosity*: c'è un punto interrogativo, "chissà cosa viene fuori"
hit enemy	*survival*
overcome obstacle	*curiosity*
acquire item	*collection*

Su *hit enemy* i due discutono: non lo assocerebbero a *pride* o all'aggressività, perché Mario non è "uno di quei giochi dove tu spari perché vuoi sparare", ma perché vuoi sopravvivere: **survival**. Forse c'è un po' di *excitement* quando si colpisce bene un nemico, ma per ora lasciano solo *survival*.

Poi il relatore chiude le stanze: "mi sembra brutale chiudere tutte le sessioni di botto, ma chiuderò tutte le sessioni di botto". Zoom dà ai gruppi sessanta secondi prima di riportarli nella stanza principale ("siamo stati risucchiati alla realtà").

0.7.6 Primo passo: le analisi dei gruppi

Le scelte: **Super Mario** per le sessioni 1 e 3, **Lasers & Feelings** per la sessione 2, **Ticket to Ride** per la sessione 4[24]. Il relatore mostra la *board* sua e di Michele come "supporto visivo e immediato", ma avverte: "io non mi aspetto questo". Basta aver identificato, anche solo a voce o per appunti, le azioni e magari il *core loop*.

0.7.6.1 Sessione 1: Super Mario

Il gruppo di Bit9 ha fatto una lista delle azioni possibili e di alcune regole che le limitano e le inquadrano. Le azioni di base: **saltare**, **correre**, **distruggere blocchi**, **schiacciare i nemici** per eliminarli, **acquisire power up**. In più hanno messo **cercare percorsi nascosti**, perché lo ritengono fondamentale per la *replayability*, e **ottenere un punteggio finale**. Gli altri ricordano che hanno dimenticato il *crouch*: "alla fine è un'azione secondaria che prendi solo quando sei grande".

Il relatore si ritrova in tutto, tranne nel punteggio: più che un'azione del personaggio è un'azione del giocatore, "una sorta di **dinamica**", perché mette insieme più meccaniche. E fa notare come **sensibilità diverse creino analisi diverse** del *core loop*, e che questo è "un valore aggiunto, non è negativo": quando si analizzano meccaniche, azioni e *core loop*, l'obiettivo non è solo ottenere la descrizione migliore, ma vedere come sensibilità diverse diano ipotesi di azioni differenti. Cercare le aree nascoste, per esempio, è qualcosa che si fa comunque con *run*, *sprint*, *jump* e *crouch*, ed è una cosa che lui e Michele non avevano messo. Loro avevano distinto i **due comandi di base** (*run/sprint* e *jump*) dalle **quattro azioni che con quei comandi si compiono** (*hit brick*, *hit enemy*, *overcome obstacle*, *acquire item*), che non hanno un comando dedicato.

0.7.6.2 L'orgoglio è un'emozione emergente

Il gruppo era partito elencando emozioni e istinti, perché gli era rimasto in mente lo schema del 6-11 ("giustissimo"), e chiede: metterebbe l'**orgoglio** tra le emozioni desiderate? Secondo loro la raccolta di punti e la competizione con gli altri giocatori sono importanti nel Super Mario originale. Il relatore non metterebbe né *greed* (preferisce *collection*) né, in prima battuta, *pride*. Certo, se si gioca con altre persone, uno contro uno, il *pride* ci sta: ricorda che da giovanissimo si facevano le gare a Super Mario. Ma è un orgoglio **acquisito**, non insito: non è diretto, nasce dal fatto che, al di fuori del gioco, si può fare un'operazione di *pride*. È, come suggerisce qualcuno, **emergente**: "bravo, è emergente, sono d'accordo". "Ottima domanda, bellissima domanda".

[24] Prima delle presentazioni, saputo che la sessione 2 ha scelto *Lasers & Feelings*, qualcuno chiede al relatore, "a titolo emotivo", il suo capitano di Star Trek preferito. Risposta: Janeway, "per motivi di cuore, perché sono cresciuto con *Voyager*", che deve riportare tutti a casa da una galassia lontana; anche se la protagonista di *Discovery* gli piace molto.

0.7.6.3 Miro, ancora

Un partecipante chiede che cosa sia "il programmino simpatico" sullo schermo. È **Miro**, gratuito con i limiti già detti (sezione 0.7.1.2): quando si crea la quarta *board*, la prima non è più modificabile, e la si può lasciare così, oppure si passa alla versione per freelance. Per il relatore è uno strumento di lavoro vero: lo usa con i clienti, e lo aiuta tantissimo a visualizzare e a "fargli vedere qual è il problema logico nel processo mentale che hanno". Ce ne sono tantissimi altri.

0.7.6.4 Avidità o raccolta?

Un altro partecipante avrebbe messo l'**avidità** in Super Mario, visto che ci sono le monete: il relatore preferisce raccolta perché il gioco non è multiplayer? Non proprio. È vero che la moneta spinge a giocare: la si vede, si dice "ne voglio sempre di più", ci si va e si muore "brutalmente" ("sono scarsissimo"), perché a un certo punto con le monete si guadagna una vita, o comunque si alza il punteggio. Ma *greed* è una parola **molto forte**: il relatore non la userebbe per una raccolta di oggetti che si può fare o non fare, ma per **qualcosa che il gioco spinge effettivamente a cercare di acquisire**. "Questa è la mia distinzione: come vedete, le sensibilità."

0.7.6.5 Sessione 2: Lasers & Feelings

Il gruppo ha fatto un Drive condiviso e, non conoscendo il gioco e volendo "farlo tutto", ha cominciato analizzandone la struttura. Le azioni principali: **creare i personaggi**, **selezionare le statistiche**, con le varie specifiche, **personalizzare la nave**, **lanciare il dado**. Il gruppo aveva un dubbio sul terzo dado: "questo è il limite dei *one page*". Il relatore chiarisce: un D6 se si è **preparati**, cioè se si possiede qualcosa che può aiutare, e un D6 se si è **esperti**, ed è il **GM** a dire quanti dadi si tirano, perché è lui a considerare esperto il personaggio.

Due correzioni:

Due azioni, non una. Lanciare il dado è giusto come meccanica del giocatore, ma ha due varianti: si lancia per essere *laser* o si lancia per essere *feelings*. Il relatore metterebbe proprio due azioni distinte, **lanciare il dado laser** e **lanciare il dado feelings**, perché sono meccanicamente diverse: richiedono di dichiarare quale dei due poli si vuole usare.

Il GM è un giocatore. Il gruppo pensava che una meccanica importante fosse il GM che narra, "un *core element*". Il relatore ribalta la prospettiva: "prima di tutto, ricordatevi sempre, quando fate analisi di giochi di ruolo, il GM **è un giocatore**". Molti non se ne rendono conto: dicono "ci sono i giocatori, poi c'è il GM". È quello che viene chiamato ***over player***, un giocatore con più poteri degli altri e, di solito, con **discrezionalità**. Ma è un giocatore, e quindi ha delle azioni: **creare un'avventura** e **condurre il gioco**, da tenere separate e distinte da quelle degli altri giocatori.

Il *core loop*, per il gruppo, è **lancia e narra**: descrivere le proprie azioni, lanciare i dadi, il GM narra la reazione, si descrive, si lanciano i dadi. "Sì, il core loop alla fine è quello."

0.7.6.6 Partire da una board vuota

Nel frattempo[25] un partecipante chiede quale modello di Miro abbia usato il relatore. Nessuno: di solito parte **dal vuoto**. Se si vuole un modello, il consiglio, "per esperienza", è il **flowchart**, anche quando si deve fare una mappa concettuale: la *mind map* e la *concept map* sono "un po' *tricky*" da usare e servono più ad altro; "il *flowchart* è il migliore" (si veda anche la sezione 0.6.12.1). Per partire dal vuoto, Miro lo chiede alla prima *board*, con un pulsante *empty*; altrimenti si seleziona tutto il modello e lo si cancella. Lo strumento **frame**, in basso a sinistra, permette di creare anche un *frame* personalizzato: "il frame è proprio una lavagna su cui tu puoi creare cose".

[25] Nel mezzo, Michele configura qualcosa "da mandare ai ragazzi" ("vai, vai, non ti preoccupare").

0.7.6.7 Sessione 3: Super Mario

Il gruppo di Flavio, Claudio e Francesco ha distinto:

Azioni. **Correre**, **saltare**, **colpire**, **raccogliere potenziamenti** e **scendere nei tubi**.

Core loop. **Correre, saltare e colpire**: "senza di quello non si procede".

Limitazioni. Non si può andare a sinistra dell'inquadratura: l'unico spostamento è verso destra. L'unico modo per affrontare i nemici è saltarci sopra: non li si può toccare, altrimenti si prende danno. I livelli hanno un **limite di tempo**.

Sensazioni. La **curiosità** nel trovare percorsi secondari o nascosti e nel colpire i blocchi; l'**istinto di sopravvivenza** nel platforming e nell'affrontare correttamente i nemici ("bravi sul *survival*, bravi sul *survival*"); la **gioia** durante lo *sprint* e il salto, soprattutto quando riescono *trick* abbastanza complessi; la **sfida** (*challenge*), cioè fare il percorso nel minor tempo possibile e comunque concludere il livello, "che non è scontato"; la **raccolta** delle monete sparse nel livello.

"Molto bravi, molto molto bravi." Hanno tutto in un *one page*.

0.7.6.8 Sessione 4: Ticket to Ride

Per il gruppo di Giulia, Alice e Giovanni parla un membro del gruppo, "visto che tu hai il Pod"[26]. Il gruppo si è soffermato anche sul **numero di giocatori**, per capire se il gioco sia cooperativo o competitivo: *Ticket to Ride* è ovviamente competitivo, ma l'informazione potrà tornare utile per un *hack*. Lo scopo è l'accumulo di punti, che si ottengono in tre modi: in base alla lunghezza del percorso, al numero di vagoni del singolo tragitto, e agli obiettivi raggiunti.

Il *core loop* scandisce anche la suddivisione dei turni, perché in un turno si devono per forza fare delle scelte, e le scelte "sono due e mezzo". La prima è **pescare carte** per accumulare risorse: se si pesca la **locomotiva** jolly scoperta se ne può pescare una sola, altrimenti due ("la carta locomotiva pescarla costa due", conferma il relatore). Sempre pescando, si possono prendere **nuovi obiettivi**. L'altra opzione, conseguente all'accumulo di risorse nei turni precedenti, è **posizionare i vagoni**, che fa prendere i primi punti durante la partita.

0.7.6.9 Pescare è un'azione sola

"Bravi, bravi su questa cosa": il gruppo ha colto che **pescare è un'azione unica**. "Molti non ci arrivano." *Ticket to Ride* non ha due azioni distinte: pescare carte vagone e pescare biglietti destinazione è sempre pescare, **cambia solo il mazzo**. Al contrario di *Lasers & Feelings*, dove il dado laser e il dado feelings sono stati divisi perché sono **meccanicamente diversi** (uno si tira sotto il numero, l'altro sopra[27]), qui pescare non cambia nulla a livello tecnico. Forse cambia qualcosa per la locomotiva, a seconda che la si peschi dal mazzo o tra le carte scoperte: "sì, ok".

0.7.6.10 Dove sta l'avidità in Ticket to Ride

Chi parla ha una domanda sull'**avidità**, "non come sentimento della vita reale, ma nell'ambito ludico". Nel loro caso le sembra presente nella scelta tra pescare e mettere giù i primi vagoni: non avendo ancora interiorizzato la meccanica, sente il rischio di accumulare senza mai posare. Il relatore non metterebbe l'avidità sulle azioni, sulle meccaniche, sul *core loop* di base: l'avidità in *Ticket to Ride* è un istinto che scatta quando si fanno **strategie** con le tratte, cioè nelle **dinamiche**, quando si devono fare scelte. Pescare il jolly o due carte? Finiti i propri biglietti, se ne pesca un altro o si punta al percorso più lungo? **È avidità**, perché si vogliono accumulare molti più punti; se si pesca il quarto biglietto, è avidità, anche perché può

[26] Così nella trascrizione ("che tu hai il pod"): il senso non è chiaro, forse un riferimento a chi ha il documento aperto o a un microfono migliore.

[27] Il relatore si confonde a voce ("il laser va verso l'alto e il feeling va verso il basso, cioè il contrario, scusate, il laser va verso l'alto"); la regola è quella esposta nella sezione 0.7.3.1.

capitare un obiettivo lunghissimo e delicatissimo. "Pensate agli istinti umani e non sbagliate mai": si può andare anche oltre gli undici elencati da Dillon.

Un'altra partecipante aggiunge la **sopravvivenza**: giocando il giorno prima aveva l'angoscia che qualcuno occupasse la sua tratta più in fretta e le tagliasse la strada. "Brava": quando si nota che altri vanno verso le stesse città, gli stessi obiettivi, si comincia a pensare che quella tratta è una sola, non doppia, e che se la occupano bisognerà fare il giro, magari già occupato anche quello, "oddio, devo fare un giro enorme". Il relatore aggiunge l'aspetto **visivo**: il tabellone ha un ritorno emotivo perché c'è tanto colore. Uno degli istinti è l'apprezzamento estetico (*color appreciation*), o il concetto di *appealing*: il gioco è molto colorato e invoglia a giocare; "non sarebbe la stessa cosa se fosse pieno di grigi, di cose brutte".

0.7.6.11 I colori dei vagoni e i binari

Un dubbio, "non avendoci giocato" ("siamo qui per i dubbi"): chi parla non condivide la scelta di dare ai **giocatori gli stessi colori di alcune tratte**. Vedendolo, aveva pensato: "io sono il giallo e devo metterli sui gialli". Invece no. Se ne sono accorti anche gli autori, risponde il relatore: gli esempi del regolamento sono fatti apposta **invertendo i colori**, per far capire che i vagoni blu non c'entrano con le carte che si giocano. È d'accordo che sia una **dissonanza**[28], che però in questo gioco non ha creato grossi problemi: i vagoni erano semplicemente un modo per indicare il possesso delle tratte, e il lavoro sui colori delle carte e dei vagoni è comunque buono. Aggiunge che il gioco è più giocabile da chi ha un'ottima percezione delle **sfumature di colore**[29]: il blu dei vagoni e quello delle carte sono uguali, forse anche il bianco, ma il giallo e il nero non sono esattamente identici. Anche lui avrebbe fatto diversamente: per esempio, avrebbe dato ai giocatori **colori metallici**.

Chi parla proponeva di mettere, al posto dei vagoni, dei **binari**. Il relatore rivela un retroscena "che non puoi sapere": i binari erano previsti nella prima versione, ma **li hanno tolti**. In alcune espansioni e giochi successivi della serie[30] ci sono tratte senza binari, che si devono costruire prima di metterci sopra il treno. Per un motivo di **scalabilità del progetto**, una decisione che di solito si prende nel brainstorming, nel gioco base hanno deciso di non metterli. Ma si può fare una variante in cui si è costruttori di binari anziché di treni.

0.7.7 Secondo passo: visione, esperienza desiderata e pubblico

Fatta l'analisi, si uniscono **ricerca** e **development**: un quarto d'ora, venti minuti. Il compito:

1. ipotizzare uno, due o tre *hack* e sceglierne uno, "del tipo: ah, ma perché non facciamo un Super Mario dove il protagonista non è un idraulico ma un **boscaiolo**?";

2. scrivere la **visione** come si crede meglio: può essere un *pitch* narrativo, una descrizione di meccaniche, "non importa";

3. ipotizzare le **esperienze**, cioè l'insieme di istinti ed emozioni che si vogliono ottenere, rapidamente, "perché poi cambierà";

4. ma soprattutto **conoscere i giocatori**, la **target audience**.

0.7.7.1 Come si conosce il pubblico

La cosa migliore che il relatore può dire è che conoscere il pubblico **passa per la ricerca di dati online**. Cercando "*player type* Super Mario" o "analisi *player type* Ticket to Ride" si trova sicuramente qualcosa. Ma il punto non è usare i player type di Bartle: se lui dicesse che Super Mario è un gioco per *achiever* e *killer*, i partecipanti gli direbbero "ma Daniele, che cosa stai dicendo? Non è così". Bisogna **trovare modi**

[28] La trascrizione dice "una dissonanza ludica, non ludica, ma solo ludica": forse il relatore stava per dire "dissonanza ludonarrativa" e si è corretto.

[29] Il relatore dice "lo dico proprio a livello di genetica"; il riferimento implicito è al daltonismo.

[30] Il relatore cita, "se non mi sbaglio", "Europa 1800"; non è stato possibile identificare con certezza il titolo a cui si riferisce.

per descrivere il pubblico che si capiscano all'interno del team. A volte basta descrivere l'utente: chi gioca questo gioco? Persone di una data età, con una data esperienza, a cui piacciono giochi di tipo X.

Già le ***wanted experience*** danno un'idea del player type. In un gioco **horror**, tra le esperienze volute ci sarà probabilmente lo *scare*, ci sarà il **thrill**, la sensazione di quando si hanno tante emozioni addosso, come sulle montagne russe, e c'è anche l'***excitement***, un po' nascosto, da distinguere dalla *joy*: "non è gioia, è eccitazione dovuta al fatto che sei stato spaventato". È logico che chi non ama essere spaventato e non ama il *thrill* difficilmente giocherà un gioco horror. Basta quindi la descrizione delle esperienze volute per fare un po' di analisi del player type.

Ma si può fare di meglio:

- **leggere i commenti dei giocatori** sui forum, su **BoardGameGeek** e "quant'altro";
- **non sottovalutare il potere di Google**. Molti dicono: cerco "Ticket to Ride player type analysis" e magari non esce niente, o escono cose inutili. Ma poi si vedono i commenti su BoardGameGeek, una recensione "*four-sided*" di Ticket to Ride[31], perfino uno studio di *intelligenza artificiale* per costruire mappe di Ticket to Ride[32];
- **non sottovalutare le schede Notizie e Libri** della ricerca.

Ovviamente non bisogna fermarsi alla prima ricerca, ma andare a fondo: "voi non sapete quello che potete trovare con una semplice ricerca Google", e ogni tanto si trovano cose molto particolari.

Ci si può anche **ispirare ai propri amici**: chi ha giocato a questi giochi con delle persone può descriverle, cambiando il nome ("avete giocato con Laura, potete mettere Luca"; cambiarlo davvero, "non tenere solo l'iniziale"), e descrivere il tipo di persona che di solito gioca quel prodotto.

0.7.7.2 Un esempio sopra le righe

Un partecipante chiede conferma: in sostanza bisogna creare la visione di un *hack*, trovare il tipo di esperienza che si vuole e il prototipo delle persone che vorranno giocarci? "Assolutamente sì." L'esempio: la visione è **Super Mario Bros. con un boscaiolo** al posto dell'idraulico. Il boscaiolo avrà ostacoli e azioni diverse: magari **comincia con l'ascia**, non ha il salto, che prende dopo, ma ha subito il lancio dell'ascia. Qual è la *wanted experience*? Cambia rispetto a Super Mario? E qual è l'audience? Se il gioco va molto sopra le righe, diventa "un po' truzzo": un Super Mario con il boscaiolo che incrocia *Fast and Furious*, dove a fine livello il boscaiolo si strappa la camicia e mostra i peli. Si capisce subito che quell'esperienza ha un **player type molto preciso**: un pubblico nerd, con senso dell'umorismo, a cui piacciono le cose "un po' trash" e probabilmente *Fast and Furious*. E queste cose "mi possono dare un fracco di idee".

Un altro chiede se nell'*hack* si debba cambiare una sola meccanica o più di una. Per il momento **le meccaniche non si toccano**: si va *step by step*, visione, *wanted experience* e audience. Le meccaniche sono il passo successivo. Se però viene in mente una possibile meccanica, come il lancio dell'ascia, la si annota e la si lascia lì: tornerà utile dopo. "È molto utile per andare un po' più sul focus e non avere troppe idee."

0.7.8 Secondo passo: il Bromance Mario

Riaperte le stanze, Daniele e Michele prendono proprio il boscaiolo.

0.7.8.1 La visione

Michele teme che salvare il mondo, o il bosco, sia un tema troppo grande. Allora il boscaiolo è un **amante degli animali**, "di quei boscaioli un po' druidi", che non caccia ma si nutre di ciò che raccoglie; e qualcuno **gli ruba gli animali**, i suoi compagni, che lui deve salvare. Il relatore ci vede subito **John Wick** ("com'è che si chiama quel film con Keanu Reeves?"): "un John Wick fiabesco, dove in ogni mondo c'è qualcuno che ha sottratto un compagno animale al boscaiolo". Ma chi? Un John Wick boscaiolo sarà probabilmente

[31] Probabilmente una recensione del sito *The Opinionated Gamers*, che pubblica recensioni a più voci, o di un sito analogo; il titolo letto è "Ticket to Ride: A Four-Sided Game Review".

[32] Il titolo letto è "Artificial Intelligence Approach to Build Ticket to Ride Maps".

un **ex guerriero**: Michele propone un ex operaio dell'azienda che andava a disboscare; meglio, uno che era bravissimo a uccidere, tanto che qualcuno, per farlo tornare in azione, "dalla sua parte", gli porta via qualcosa. Risultato:

> Un **ex mercenario boscaiolo**, in un mondo **trash fiabesco**, deve recuperare i suoi **compagni animali**, sottrattigli da un ***trust* di villain** per farlo tornare a lavorare per loro.

0.7.8.2 L'esperienza voluta

Si parte dalla tabella delle emozioni di Super Mario (evasione, gioia, curiosità, sopravvivenza, raccolta) e la si cambia, separando gli istinti dalle emozioni:

Emozioni. Non c'è più l'evasione, la gioia della corsa: c'è il fatto che si deve sopravvivere e soprattutto "spaccare roba". Quindi **rabbia** (*anger*)? Meglio una sua declinazione: il **rancore**, come quello del Kratos della nuova saga di God of War, "non quello che è arrabbiato". Poi un po' di **tristezza** (*sadness*), perché gli hanno sottratto gli amici animali, e la **curiosità**: magari ha lasciato il mercenariato perché voleva davvero andare in giro per il mondo a trovare la pace.

Istinti. Sopravvivenza, sicuramente; **raccolta**, che ci può stare (si trovano *power up*); **avidità** "sicuramente no". Scorrendo la lista di Dillon: **aggressività** sì, il *fight or flight* di fronte ai nemici; competizione no; **vendetta** sì, forse più del *fight or flight*; e, con la vendetta, la **protezione**, perché una volta recuperati alcuni animali li si vuole proteggere; **curiosità**; e l'**apprezzamento estetico**, perché c'è molto il tema della pace, dell'andare a trovare luoghi belli e fantastici.

0.7.8.3 L'audience e i riferimenti

Per descrivere l'audience si parte dai giochi che quel pubblico gioca, anche se non simili al proprio:

- **God of War**, sicuramente, se il protagonista sarà "un bel barbuto con un'ascia";
- "paradossalmente" **The Binding of Isaac**, per il tema della sopravvivenza più che della vendetta, e perché è un prodotto molto sicuro della propria estetica, che la porta fino in fondo[33];
- **Alto's Adventure** e **Alto's Odyssey**, che Michele presenta: si è uno sciatore o uno *snowboarder* che deve fare *trick*, prendere punti e arrivare fino in fondo. È un gioco con pochissimi feedback loop negativi (sezione 0.6.19), basato sull'apprezzamento estetico, sulle combo, con una grafica 2D molto minimale ma molto curata;
- il gioco del **taglialegna** in cui si devono schivare i rami che cadono[34].

Qui Michele osserva che quel gioco, pur parlando di un taglialegna, è **amichevole**, "anche per la forma che ha", ed è l'occasione per ripassare la **teoria delle forme** (sezione 0.6.24): il **tondo** è amichevole, gioioso; il **triangolo** è pericoloso, ostile; il **quadrato** di solito è molto schematico, "robotico", "è il robot". Si possono fare anche gli **incroci**: un tondo con gli occhi quadrati e la bocca triangolare crea una **dissonanza** che genera ambiguità, utile per personaggi complessi. E si possono mischiare le forme: un quadrato con gli angoli tondi, un esagono, un poligono.

Poi, ricordando che il gioco è "un po' più trash", l'audience si allarga ai riferimenti non videoludici: **John Wick**, **Fast and Furious**, le cose "sopra le righe" ("tipo Fast and Furious dove saltano con la macchina dentro l'aereo"), e tra i videogiochi l'esagerazione massima, **Devil May Cry**.

0.7.8.4 Il player type

"E visto che siamo tecnici": che player type è? Date le *reference*, il **killer**, per l'*excitement* e per l'unicità del protagonista, "quasi una forza della natura". Non *achiever*. C'è però l'elemento del recuperare gli animali,

[33] I due ricordano anche che il gioco è molto focalizzato sul tema della religione.
[34] Nella trascrizione "Lumberjack" e poi "Timberman": si tratta di *Timberman*.

la **relazione con l'animale**, che lo avvicina al *socializer*; e un'esplorazione implicita. Il risultato, con i sottotipi della slide sui player type avanzati[35]:

- tipo principale: **killer**;
- secondari: **hacker**, cioè un **esploratore implicito**, con una comprensione intuitiva del mondo, "non razionale"; e **friend**, un socializzatore che gode della compagnia degli altri.

"Molto bene, molto molto bene." C'è infatti molto il tema della **bromance**, come in *Fast and Furious* e in John Wick. Da qui il nome: non più *Super Mario Bros.*, ma "**Super Bro Mario**", o **Bromance Mario**, "proprio per essere trash di brutto"[36].

0.7.8.5 Genere, gimmick e mood

Resta da capire la **gimmick**, l'elemento che lo rende unico ("il problema delle gimmick è poi capirle"). Il genere è un **platformer 2D a scorrimento**: non per forza in *pixel art*, ma oggi fare un gioco in *pixel art* è un po' più semplice, perché tante persone vendono anche *reference* e risorse su itch.io. Il mondo è quello dei mercenari, quindi uno **spionaggio sopra le righe**. Il **mood** è quel tipo di ironia che però si prende molto sul serio, come in **One Piece** (Michele lo conosce, ma "lo odio": non l'ha mai visto). Per le *gimmick* il relatore trova un video, "*Every unique level gimmick in Super Mario*", da guardare come ispirazione.

Poi chiude le sessioni, stavolta senza conto alla rovescia: "quante volte io spavento i miei studenti". Chi rientra dice di non aver nemmeno toccato il compito; "non è un problema", si recupererà.

0.7.8.6 La presentazione

Il relatore presenta a tutti l'idea sua e di Michele: la visione dell'ex mercenario boscaiolo. "Avrete notato tutti che è **John Wick che incontra Super Mario**." Quando un gioco si riassume così, "questo gioco che incontra quest'altro gioco", si parla di **high concept**[37]. È un platformer 2D a scorrimento, di spionaggio sopra le righe, con un *mood* alla One Piece: molta comicità, ma anche molta "serietà", e molta *bromance*.

Dall'esperienza di Super Mario hanno tolto l'*excitement* e la *joy* e hanno messo il **rancore**, "che non è la rabbia: è una declinazione del concetto di rabbia molto simile al rancore che prova Kratos", la **tristezza** per gli amici animali sottratti e la **curiosità**, perché il mondo è esplorativo e naturale. Gli istinti: sopravvivenza, raccolta (i *power up*), vendetta, protezione e percezione estetica. Il mercenario si è ritirato non per una vita di *retirement* come John Wick, ma **per viaggiare**: andare in posti molto belli, fare il nomade digitale, fare *hiking*; lo si immagina tornare da quei luoghi per andare a combattere.

L'audience l'hanno descritta partendo "dalla cosa più visiva" che potevano trovare: che prodotti giocano e guardano i giocatori di questo ipotetico Bromance Mario? God of War, The Binding of Isaac (prodotti molto sicuri della loro estetica), Alto's Adventure ("una saga molto legata all'estetica": nei sequel hanno trovato modi nuovi per inserire "cose molto fighe"; l'estetica sarà uno dei principali elementi distintivi del gioco), John Wick, Fast and Furious, Devil May Cry. E, conoscendo Bartle, hanno preso da lì i player type: un giocatore **killer**. "Attenzione a non pensare che è un assassino": il killer è una categoria un po' problematica. I *killer* sono quelli che vogliono **forti emozioni**, molto legati al protagonista: nella mappa della **slide 72** il killer è legato al *player* e all'*acting*, cioè all'agire e al concetto di *main character*[38]. Come sottotipo esploratore, l'esploratore istintivo, l'*hacker*; come socializer, non il *networker* ma l'**amico**, che gode della compagnia dell'altro. Per questo nel primo capitolo della saga ("perché sarà una saga, ovviamente") ci sarà probabilmente uno **Yoshi**: un animale, uno dei primi che si recuperano, che si può cavalcare, "probabilmente un cervo con le corna cromate".

[35] Sono i sottotipi del modello a otto tipi che Bartle ha proposto dopo la tassonomia originale, distinguendo per ogni tipo un comportamento implicito e uno esplicito: tra gli *explorer* lo *hacker* (implicito) e lo *scientist* (esplicito), tra i *socializer* il *friend* e il *networker*, tra i *killer* il *griefer* e il *politician*, tra gli *achiever* l'*opportunist* e il *planner*.

[36] I due scherzano anche sui *meme* di *Fast and Furious* sulla famiglia ("l'ho fatto per la mia famiglia"). Il passaggio è molto disturbato nella trascrizione.

[37] Nella trascrizione "Ike Concept". Il relatore aggiunge: "è un'idea molto *mainstream*", e, sembra, "lo so anch'io".

[38] Nella trascrizione "il killer è molto legato a player e al ducking": nella mappa di Bartle i *killer* sono collocati sugli assi *acting* e *players*.

0.7.9 Secondo passo: le visioni dei gruppi

0.7.9.1 Sessione 1: Creepy Mario

Il gruppo di Bit9 propone **Creepy Mario**: "un Mario platformer dal gusto **horror**, inquietante e angosciante, che **obblighi il giocatore ad avanzare** nonostante le difficoltà". Meccaniche e dinamiche non le hanno ancora discusse, ma immaginano di sostituire la libertà estrema di percorso e di movimento di Mario con l'**incertezza** di non sapere che cosa c'è davanti, dovendo però avanzare perché **qualcosa arriva da dietro**. Hanno sostituito la gioia con la **paura**; le emozioni desiderate sono quindi paura, **eccitazione** e **orgoglio**, inteso come punteggio e competizione tra utenti su chi sopravvive di più. Tra gli istinti, la **sopravvivenza** passa in primo piano; restano la **curiosità** e la **competizione**; stavano discutendo di aggiungere la **protezione**, con persone da salvare lungo la strada, che altrimenti "finiranno molto male": salvandole si sacrifica qualcosa, si rallenta. "Interessante."

Qualcuno ricorda la *creepypasta* sul gioco di Godzilla[39] ("bravo Filippo"): il gruppo ha delle *reference*. Il **player type**: un pubblico di **adolescenti**, appassionati di horror *creepy cute*, come i Pokémon inquietanti, **Five Nights at Freddy's**, **Poppy Playtime**; un pubblico ***streamer friendly***. Con Bartle, il gruppo propone gli **achiever**, perché il gioco può attirare chi vuole fare un *high score*. Il relatore aggiunge di **non sottovalutare il killer**: con molto *thrill*, molto horror, "il killer per forza".

0.7.9.2 I falsi amici di Bartle

Il killer, insiste il relatore, è un **falso amico**: "Bartle ha il problema che molti di questi sono falsi amici". Guardando l'asse *acting* e *players*, il killer non è l'idea che il nome suggerisce: infatti in versioni successive del modello il killer comprende anche il concetto di **artigiano**, di **tattico**, di chi **cerca sensazioni forti**. Per questo lui i player type li usa "fino a un certo punto". Il gruppo ammette di non avere molta dimestichezza ("non ci spingiamo troppo"), e va bene così: "senz'altro qualcuno che vuole un'emozione forte io lo metterei nella lista". Qualcuno vede anche un aspetto da **socializer**: il piacere di guardare l'amico che "sbrocca" mentre è inseguito. "Può essere, assolutamente."

Il relatore ricorda i sottotipi: i *socializer* si dividono in **networker** e **amico**; i *killer* nel **politico**, che grazie alla sua buona reputazione sa trascinare le persone ("il killer dovrebbe essere più il leader, la prima donna"), e nel **griefer**, che invece ha una cattiva reputazione. Alla **slide 73** ci sono i player type avanzati, modificati, "e questa è una delle cose più...": ogni volta che fa lezione sui player type, "i nomi traggono in inganno". "Bravi, ottimo, non vedo l'ora di vedere quello che farete nella prossima fase."

0.7.9.3 Sessione 2: Thunder and Prophecies

Il gruppo di *Lasers & Feelings* è "leggermente indietro, ma ce la faremo". L'idea è rifare il gioco con il nome di **Thunder and Prophecies**[40], riprendendo il tema degli **dèi greci in stile pop**, "alla Percy Jackson", in chiave leggermente moderna. L'obiettivo è creare una **competizione tra gli dèi**. Le prime ipotesi:

- nel caso di un solo giocatore, il GM ("va bene GM come denominazione?", "va benissimo") userebbe un avversario specifico[41] da contrapporre al personaggio principale; con più giocatori, i personaggi si scontrano tra loro;
- meccaniche "aleatorie": ogni dio ha un **valore predefinito** e, con l'aggiunta di due dadi, si creano azioni specifiche, ancora da definire;
- l'**obiettivo è segreto** per ciascun personaggio.

L'ultima idea ricorda al relatore **Lupus in Tabula** e **Werewolf**, quei giochi sociali in cui nessuno sa chi è il lupo (e nelle varianti ci sono anche la strega e altri ruoli): qui nessuno sa l'obiettivo degli altri e ognuno "tira l'acqua al proprio mulino". Quindi c'è competizione di base.

[39] Nella trascrizione "la creepypasta su quel gioco di Godzilla che fece Frows" e poi "Plaggemon". La *creepypasta* più nota su un gioco di Godzilla è la *NES Godzilla Creepypasta*; "Plaggemon" potrebbe essere una storpiatura di un'altra *creepypasta* o di un titolo legato ai Pokémon, a cui il gruppo fa riferimento subito dopo. Il riferimento resta incerto.

[40] Più avanti il relatore lo chiama anche "Thunder and Prophecy".

[41] Nella trascrizione "un ghost specifico": forse un personaggio ombra, o un *boss*, contrapposto al protagonista.

Ma il relatore avverte: "questa è la classica cosa che succede quando si prendono persone molto esperte di videogiochi e gli si fa fare un *hack* di un gioco di ruolo *one page*". Il rischio di andare **fuori scope** è altissimo[42]: quante volte vede gli studenti del corso di Milano fare *hack* di giochi di ruolo e mettere "ottomila cose". Carina l'idea di **scegliere il numero in base alla divinità**, "non male"; bello il nome, "mi piace molto, devo dire la verità". Ma due cose:

- in un gioco di ruolo **non esiste il single player**: c'è un gruppo di persone, c'è un GM, e il GM narra;
- i giocatori possono essere in competizione: esistono giochi di ruolo **non cooperativi**, in cui ognuno è contro gli altri; sono più difficili da costruire, ma si possono fare. La competizione ha senso: ogni scena giocata, suggerisce, potrebbe essere **una delle prove che gli dèi danno**. "Vi sto dando anche dei suggerimenti per come costruire il prodotto dopo."

Dalla regia arriva una precisazione: **Dungeon Master** è un marchio registrato di Wizards of the Coast, mentre **Game Master** si può usare; per questo si tende a evitare il primo. GM va bene.

Le emozioni e gli istinti del gruppo: la **gioia** dell'avventura, l'**eccitazione** dei dadi e della competizione, l'**identificazione** (perché c'è la creazione del personaggio, con la scelta di una serie di caratteristiche), la **competizione**, la **comunicazione**, l'**orgoglio**. Il relatore azzarda la **vendetta**: "pensateci, non è necessariamente tale, dipende"; è un tema molto legato al mondo delle divinità, anche per via di Kratos. Gli archetipi di giocatore non li hanno ancora fatti: "non è un problema".

0.7.9.4 Sessione 3: Gloomy Mario

"Incredibilmente" anche il gruppo 3 fa una versione tenebrosa di Mario. L'idea di partenza è un'**inversione**, cioè non Mario che va a destra e insegue, ma **Mario che va verso sinistra ed è inseguito** ("molto bene"). La seconda idea viene da **Darkest Dungeon**, sia per gli elementi lovecraftiani sia per la **meccanica della torcia**. Il gioco è ambientato in una **caverna**: Mario è un grande esploratore, "tipo Indiana Jones", che esplora una caverna maledetta e, trovando un artefatto, risveglia **Bowser**, un'entità malevola che comincia a inseguirlo. Nella grotta la torcia si scarica, e man mano che la barra si svuota il **campo visivo** diminuisce; la si ricarica trovando cariche nei blocchi o tra gli oggetti lasciati dai nemici.

La *wanted experience* è una versione **molto più dark** di Mario, più **claustrofobica**, con la sensazione di pericolo data dalla maggiore presenza e aggressività dei nemici: **paura** e *thrill* ("*fear*, *thrill*, certo"), la **sfida** anche nello scappare, la **sopravvivenza** nel fuggire da Bowser verso sinistra, la **curiosità**, perché restano le meccaniche di esplorazione di Super Mario, e la **competizione** per il punteggio più alto nel percorrere il livello. La *target audience* è più adulta di quella del Super Mario classico, dai **16 anni** in su, *millennial*; e anche quella a cui piacciono le *creepypasta*, "i giochetti vecchi modificati con quel sapore horror". Come player type, anche loro il **killer**, chi cerca sensazioni forti, e come sottotipo l'**achiever**, per la ricerca dei segreti nel gioco.

Il relatore sottolinea il confronto con Creepy Mario: "io lo dico sempre in narrativa: **due concept identici portano**, date le sensibilità, **a cose completamente diverse**". *Idea is nothing, implementation is everything* (sezione 0.6.20).

0.7.9.5 Sessione 4: un Ticket to Ride con i saccheggiatori

Per il gruppo di *Ticket to Ride* parla, con ogni probabilità, Giovanni[43]. Il gruppo è partito dall'"ansia da *survival*" emersa prima, in un gioco che è invece "un po' gioioso", di viaggio. Facendo un po' di ricerca, non tanto sul tipo di giocatore quanto sugli **stili di gioco** che i giocatori assumono, ha trovato il **thrasher**: quello che vuole mettere i bastoni tra le ruote agli altri. L'idea di partenza è quindi **far entrare nel regolamento**, negli obiettivi e nelle future meccaniche, **quel comportamento**. E, visto questo nuovo tipo di obiettivo, aggiungere alla competizione, con cui *Ticket to Ride* nasce, anche una **cooperazione**, con una divisione in fazioni.

[42] Il relatore dice "overscope" ("andare in overscope"): è l'*out of scope* della sezione 0.6.4.2, con una sfumatura di "fare troppo".

[43] Il nome non è pronunciato; chi parla alla fine chiede aiuto "ad Alice e Giulia", le altre due componenti del gruppo, e quindi è verosimilmente Giovanni.

Il relatore approva: un gioco in cui il *thrasher* è "prima donna" rischia di piacere a nessuno, e il modo per renderlo accettabile è probabilmente renderlo anche competitivo, come nel gioco da tavolo di **Game of Thrones**, dove tutti sono contro tutti ma ci sono anche i Bruti. Gli viene in mente anche **Nemesis**, "fatto un botto su Kickstarter", dove ognuno ha il suo obiettivo, gli umani devono sopravvivere all'obiettivo principale, ma ci si può anche lasciar morire qualcuno alle spalle per riuscire a fuggire.

Le emozioni: sopravvivenza, competizione e cooperazione con divisione; la **paura** di perdere, di essere sostituiti; un po' di *excitement*. "Ci sta, per il momento va benissimo." Il gruppo ragiona anche sul fatto che il gioco parla di viaggio: che cosa diventa se, invece del viaggio, si introduce **la fuga**? Magari con un **tempo** che non è scandito solo dai turni, e quindi dai vagoni che finiscono. Il player type: da un lato la fazione dei **fuggitivi**, che deve avere una forte impronta **strategica** per raggiungere il prima possibile la meta della fuga; dall'altro i **thrasher**, che godono nell'ostacolare l'altra fazione. "Quindi *killer* e giocatori strategici." Un nome non c'è ancora: per un refuso era uscito "**Ticket to Busan**", con un'allusione agli zombi di *Train to Busan*. "Amate molto tutti quanti l'horror. Io sono molto contento."

0.7.10 Terzo passo: le meccaniche

Per l'ultima ora, il passo successivo. Arrivati qui i partecipanti hanno già idee di meccaniche e di dinamiche, e magari hanno dovuto trattenersi dal buttarle giù. Ora si comincia: si parte dalle **meccaniche di base**, cioè da come spostare i piccoli mattoncini analizzati, facendo piccoli cambiamenti, per **raggiungere e sostenere la *wanted experience***.

0.7.10.1 I glossari

Costruendo l'elenco delle azioni, uno schema, un *core loop*, si farà un'altra cosa fondamentale, che il relatore "non ripeterà mai abbastanza": i **glossari**, i **lessici** (sezione 0.6.21). Non serve un glossario vero e proprio: già avere parole che identificano le cose, e chiedersi se il comando *run* resta *run* o cambia nome, se abbassarsi si dice *crouch* o *duck*, che danno ipotesi e **sensazioni diverse**, significa creare glossari. È importantissimo avere **glossari di produzione**, e anche quelli minimi andrebbero nel *one page*. La *board* del Bromance Mario è già un *one page*: c'è l'analisi del gioco di base, c'è la visione, la *wanted experience*, l'audience, e lì si metterà l'ipotesi del *core loop* successivo.

Per chi ha scelto giochi molto diretti, come *Lasers & Feelings*, è semplicissimo prendere il testo e cambiarne delle parti. Il consiglio è di **partire proprio dal glossario**, come il gruppo ha già fatto con il titolo: non è *Lasers & Feelings*, è *Thunder and Prophecy*. Che cos'è *Thunder*? Che cos'è *Prophecy*? Non è per forza ciò che è scritto tra parentesi nell'originale. Che cosa si possiede? Non la nave, probabilmente qualcos'altro. Si ottiene davvero un D6 perché si è preparati ed esperti, o lo si ottiene perché l'azione si svolge **nell'area di influenza della propria divinità**? Un campione di Ade avrà aree di influenza diverse da un campione di Ares. Già ragionando su questo testo si vede come **cambiare il testo direttamente aiuti a costruire l'*hack***. Per Super Mario e Ticket to Ride è un po' più difficile.

0.7.10.2 Dal D6 al D12: dadi e probabilità

Il gruppo di *Thunder and Prophecy* pensa di passare a un **D12**, ma teme per il *testing*: cambiando il dado, il gioco resta equilibrato? Il relatore spiega che cambiare un dado da sei a dodici facce non lo rende solo **più granulare**: la probabilità cambia nei momenti in cui si tirano **più dadi**. Consiglia a tutti **AnyDice**, "un sito bellissimo" per calcolare le probabilità dei dadi:

- con un **solo dado**, D6 o D20 che sia, non c'è una campana: ogni faccia ha la stessa probabilità di uscire. Per questo si dice che *Dungeons & Dragons* è "troppo legato al dado": con il D20 ogni faccia è identica; il bonus sposta la probabilità verso l'alto (se esce 1, con il bonus si fa 7), ma resta piatta;

- con **due dadi** sommati, per esempio 2D6, si ottiene una **campana**: i risultati intermedi sono molto più probabili;

- AnyDice permette anche di **tenere il dado più alto** tra due D20, di fare collezioni di dadi e altro; la sua documentazione spiega come fare[44].

Ma "non perdetevici": strumenti come questi possono far andare *out of scope*, "è talmente carino fare sta roba che...". "Sì, è molto simpatico." Ed è comunque molto utile.

Per il caso concreto: se si passa **dal D6 al D12 con un dado solo non cambia niente**, la probabilità resta uniforme[45]. Il consiglio è di **non mettere molti dadi**: il D12 va benissimo. Attenzione però a una cosa: con un meccanismo "sopra o sotto il proprio numero", i valori estremi, **1 e 12**, non si possono usare come numero del personaggio, "perché se no non puoi fallire". Il gruppo vuole comunque lasciare simili alcune meccaniche di base: "cambiate il resto", e il tutto deve **stare in una pagina**.

0.7.10.3 Che cosa consegnare

Bisogna rifare il PDF di *Lasers & Feelings*? No: si possono incollare alcune parti del PDF in un documento Google semplicissimo, in verticale, **senza replicare la grafica**. Da consegnare entro un'ora, "quello che riuscite". Ma ciò che il relatore vorrebbe da tutti i gruppi è che riescano a fare **almeno le meccaniche e il *core loop***: le meccaniche, come ha fatto lui sulla *board*, con un po' di **regole** e i **power up**, cioè le risorse.

Le **risorse**, precisa, non sono proprio meccaniche: sono qualcosa che fa da **vincolo o rinforzo** delle meccaniche (sezione 0.6.17). Il *power up*, se ci si pensa, è un rinforzo al fatto che Mario diventa più grande, quindi più potente. Se si riesce a fare un copia e incolla di quella struttura e a usarla "un po' come se fosse il vostro **design principle**" (sezione 0.6.10), il lavoro è impostato. Lui, nel Bromance Mario, farà così: cambierà i nomi, ragionerà sul glossario, sui verbi, sulle parole.

0.7.10.4 Icone e idiomi nel glossario

Nel glossario si può fare anche il ragionamento su **icone e idiomi** (sezione 0.6.22). L'ascia dell'ex mercenario probabilmente non si chiamerà "ascia", ma avrà un **nome specifico**, perché è un **idioma**: si vuole creare **identificazione** nel prodotto. Ma se si rende tutto un idioma, se si cambia il nome a ogni comando (*run* e *sprint* diventano "foot", *jump* diventa "leg"), il giocatore **non capirà niente**. "Mi raccomando, state attenti a questi elementi."

0.7.10.5 Dove ci si ferma

C'è un'ora, tutto il resto della sessione. Chi finisce le meccaniche, magari con le dinamiche, nel *one page* e le visualizza bene, può andare oltre e porsi dubbi e domande sulle **scelte** dei giocatori e sulle procedure. Ed è qui, dice il relatore usando "una parola vietata, ma bella", la parte più **divertente**: ragionando sulle meccaniche di base ("non mettete troppe meccaniche extra, state sulle meccaniche base e fate qualche cambiamento"), guardando il *core loop*, si noteranno sempre più ***behavior***, comportamenti, **scelte implicite** che, una volta esplicitate al team di sviluppo, richiedono magari di trovare modi per limitarle. E lì comincia il ragionamento sui **feedback loop**, positivi e negativi (sezione 0.6.19): che cosa fare perché l'esperienza vada nella direzione giusta. "Io mi voglio fermare lì."

0.7.11 Terzo passo: le meccaniche del Bromance Mario

Riaperte le stanze, Daniele e Michele si chiedono come raggiungere la loro esperienza.

[44] Con due D20 sommati la distribuzione è triangolare; tenendo il più alto dei due (il "vantaggio" della quinta edizione di *Dungeons & Dragons*) non si ottiene una campana, ma una distribuzione sbilanciata verso i valori alti. Il relatore unisce i due casi in una sola frase ("se tu prendi due di 20 ti viene fuori una campana; tra l'altro non è due di 20, è output 2 di 20, tieni il più alto"); la sintassi di AnyDice per il secondo è `output [highest 1 of 2d20]`.

[45] Uniforme sì, ma su dodici valori: cambia la granularità, cioè quanto pesa ogni punto del numero del personaggio (con il D12 un punto vale 1/12 di probabilità invece di 1/6).

0.7.11.1 Camminare e colpire, invece di correre e saltare

Michele si chiede se il colpo d'ascia, che non è più il fuoco del fiore ma un'ascia, debba diventare una meccanica **quasi principale**: non c'è più il senso di evasione, c'è un senso di attacco "da killer", di rancore, quindi di rabbia. Il relatore immagina un boscaiolo che **non corre**, ma fa **passi pesanti**: quindi **walk**, non *run*; "però *walk* spedito, *walk* veloce". E, anziché saltare, il **colpo d'ascia**: da vicino, "tu alzi l'ascia e fai pam".

Con il colpo d'ascia si potrebbero far crollare i muri e creare punti da cui salire; ma il gioco "non è molto *puzzling*", è più per *explorer* e *socializer*, e l'idea non regge. Ha senso invece **colpire gli alberi per creare passaggi**: si butta giù l'albero e si crea un **ponte**. Al posto dei mattoni, le **rocce** a terra, che si spaccano con i colpi. Più che *overcome obstacle*, quindi, si **trasforma un ostacolo nel suo contrario**: l'albero da ostacolo diventa... "Qual è il contrario di ostacolo?" "Non lo so, sinceramente." "**Asset**: una parola molto mercenaria." Il relatore nota che **trasformare gli ostacoli in asset** ricorda i Pokémon con la mossa **Taglio**, che abbatte gli alberi, "e questo mi fa molto piacere": ognuno porta i suoi strumenti, la sua disciplinarietà[46].

0.7.11.2 Colpire o lanciare

Hit enemy: con l'ascia, "senza ombra di dubbio". *Acquire item*: rompendo le rocce? No: nel primo Mario si salta anche per prendere gli oggetti (il fungo esce dal blocco, va a destra o a sinistra, lo si insegue e ci si salta sopra), e l'ascia da vicino non permette di prenderli. Forse il salto bisogna rimetterlo? Michele propone un'altra strada: visto che si colpisce, si potrebbe colpire l'oggetto **guardando verso l'alto**. Il relatore la sviluppa: se si preme "su" e si attacca, invece di colpire in orizzontale **si lancia l'ascia in alto**. Nasce così la distinzione:

Axe hit. Il colpo da vicino, che fa tutto (colpire nemici, rocce, alberi) tranne *acquire item*.

Axe throw. Il lancio dell'ascia, verso l'alto ma anche in orizzontale, con cui si acquisiscono gli oggetti[47].

Michele teme che il lancio cambi molto il gameplay, se i nemici muoiono con un colpo solo come in Mario. Allora si diversificano i nemici: bisogna capire **se colpirli da vicino o da lontano**, e "ci può stare". Gli oggetti non stanno più in blocchi o rocce, che "non sono proprio un *brick*", ma in **elementi naturali**: un frutto in cima a un albero da far cadere, "magari una volta un alveare" ("bravo, bravo, bravissimo").

0.7.11.3 Il pastrano e il duck

Serve un potere come il *duck*, che si usa solo in certi casi. Al posto del crescere con il fungo: che cos'hanno i boscaioli? La cuffia, il cappotto pesante. Michele ha un'idea legata a vendetta, raccolta, sopravvivenza e rancore: la possibilità di **chinarsi** solo in certi casi, per esempio vicino a un nemico ucciso, il cui cadavere resta lì, per **trovare un oggetto**. Stimola il rancore, perché fa vedere il nemico ucciso, la curiosità, e la tristezza, "perché hai ucciso comunque una persona". Il relatore pensa a una **dinamica emergente** dei giochi multiplayer, il ***teabagging***[48]: abbassarsi ripetutamente sul cadavere dell'avversario in segno di scherno. "Comunque è killer."

La forma definitiva: il boscaiolo indossa un **pastrano** pesante, che lo distingue dal Mario con il fungo. Finché ha il pastrano, cammina e uccide i nemici; la prima volta che viene colpito **perde il pastrano**, "quindi è come se tu avessi una doppia vita". Ma, una volta perso il pastrano, **può fare *duck***: chinarsi sui nemici, schivare. Solo in quel momento, per bilanciare il fatto di aver perso una vita: si è molto più deboli, perché al primo colpo si è fuori, ma si ha il vantaggio di trasformare i cadaveri dei nemici in oggetti. Michele aggiunge che chinandosi si può cercare di recuperare dal nemico proprio un **cappotto**: il pastrano diventa un'**armatura** che si può trovare anche sui nemici. "Ci sta, mette più profondità al gameplay."

[46] Qui il relatore aggiunge che non si aspetta che tutti imparino i player type subito, "assolutamente no": si potrebbe fare una lezione di tre ore solo su quelli, ed è giusto far capire che chi fa il corso di game design ci mette tempo a ragionare su queste cose.

[47] In mezzo, Michele nota che il *jump* di Mario potrebbe chiamarsi *jump/fist*, perché quando Mario salta colpisce con il pugno (sezione 0.6.17).

[48] Nella trascrizione "T-Bag".

0.7.11.4 Le tracce degli animali

Le **monete** danno troppo l'idea dell'avidità, e l'esperienza voluta è tristezza, rancore, curiosità. Al loro posto Michele propone **ciuffi di pelo degli animali**, il relatore le **tracce** degli animali rapiti. Rompendo le rocce a terra si possono trovare tracce, ma l'importante è che siano legate agli animali e restino nell'ambito naturalistico. Più tracce si trovano, più si è sulla strada giusta per arrivare agli animali, e le tracce, "che ne so", danno punti.

0.7.11.5 I richiami degli animali

E il fiore di fuoco, o lo Starman? Lo Starman potrebbe essere non la birra, ma una **bevanda proteica**, "la bevanda delle dodici uova"[49]. Michele ricorda un'arma nascosta di **Battlefield**: la mano che diventa una pistola, con il personaggio che fa i versi dello sparo. Da qui l'idea: il boscaiolo usa i **versi degli animali** per colpire i nemici. "Mi piace, i versi degli animali." Non troppi, tre o quattro.

Michele racconta un aneddoto dal lavoro per la sua startup americana: dovendo creare gli effetti sonori degli animali marini, lui e una collega cercavano il verso dello **squalo** e non trovavano niente; dopo un po' di ricerca hanno scoperto che **lo squalo non fa nessun rumore**. Da allora, in fase di ricerca, prima di cercare il suono di un animale si verifica che l'animale faccia un suono. "Perfetta questa modalità."

I richiami (**animal call**) diventano una categoria della *board*: il **ruggito dell'orso** (*bear's roar*), lo **strido dell'aquila** (*eagle screech*), l'**ululato del lupo**; il **cervo**, che più che un verso suggerisce l'incornata, diventa invece un **compagno animale da cavalcare**, come Yoshi, che magari carica i nemici. Lo squittio dello scoiattolo non va bene. Che cosa fanno? Non bisogna complicare troppo un gioco "che alla fine è Mario" (Michele ricorda un gioco in cui l'urlo abbassava la resistenza dei nemici, e lo scarta). La regola: sono ***one shot***. Quando si trova il richiamo, lo si attiva una volta e fa il suo effetto, ma su tutto:

- il **ruggito dell'orso** uccide tutti i nemici **in mischia**, cioè vicini;
- l'**ululato del lupo** uccide tutti i nemici **a distanza**, in un certo raggio[50];
- lo **strido dell'aquila** rompe tutti gli oggetti sopra il boscaiolo, anche quelli nascosti, che però non cadono: restano lì, e bisogna trovare il modo di raggiungerli.

Uno per tipo, "per dinamica di gameplay"; in arancione, "un arancio più pesante", sulla *board*[51]. Michele propone anche di far ricominciare il livello a chi sbaglia, come in certi *runner* per telefono e in *Geometry Dash*; "no, così no".

0.7.11.6 Le regole

Come regole restano quelle di Mario: **non si può andare a sinistra**, **non si può saltare** e c'è il **limite di tempo** del livello, perché si devono salvare gli amici animali prima che il tempo finisca. "Così rimaniamo fedelissimi a Mario, togliendogli comunque una *core mechanic*." Anzi, Michele nota che **Diego** "sarebbe molto orgoglioso" di un gioco **senza la meccanica del salto**: era una sua sfida[52]. Restano anche tutte le regole implicite, "le basi basi basi" di Mario, per esempio che un nemico uscito dallo schermo è uscito.

0.7.11.7 Le tre avvertenze

Il relatore si annota due cose da dire alla fine. La prima: l'esperienza di oggi è stata **semplificata**, perché si è partiti dall'analisi di un gioco molto simile a quello che si produrrà; quando si deve creare un gioco nuovo **da zero**, è molto più difficile. La seconda: di solito **un hack espande un po' rispetto all'originale**,

[49] Nella trascrizione segue "Eggnog", proposto forse come nome; l'idea viene poi scartata.

[50] All'inizio il relatore pensava che l'ululato rivelasse tutte le tracce sullo schermo, per legarlo all'olfatto del lupo; poi la distinzione mischia/distanza "taglia la testa al toro".

[51] Riordinando la *board* il relatore scherza sul proprio bisogno di ordine ("per la mia OCD non va bene") e risolve cambiando il colore di un elemento in blu.

[52] Si tratta con ogni probabilità di Diego Ricchiuti, il direttore del corso di game design citato nel capitolo 0.6. La trascrizione dice "era la sfida dell'Interball": il nome della sfida, o dell'occasione in cui è stata lanciata, non è ricostruibile. "Un incubo", commentano i due.

"infatti noi stiamo molto espandendo". Prima di passare alle dinamiche, il relatore va a fare un giro nelle stanze dei gruppi per vedere come procede.

0.7.11.8 Nella stanza principale

Al ritorno dei partecipanti[53], Michele parla con due di loro ("Michele Bosio e Marco Ronchi")[54]: racconta che hanno tolto il salto e messo il lancio dell'ascia ("hai rimosso il salto?", "ammazzato"). Marco, arrivato tardi, si è perso il lavoro sul campo; quanto alla registrazione, il relatore conferma che Zoom mostrava "*recording in progress*", ma le singole stanze non sono registrate; si vedrà comunque il lavoro suo e di Michele.

0.7.12 Le presentazioni finali

0.7.12.1 Il Bromance Mario

Il relatore riassume l'*hack*, che a lui e a Michele "è piaciuto molto fare". Il protagonista **non salta**: cammina e può **lanciare l'ascia** oppure **colpire i nemici vicini**; alcuni nemici si possono colpire solo da vicino, altri solo da lontano. Può **accucciarsi** solo quando è a petto nudo: comincia con un cappotto che perde la prima volta che viene colpito, come una vita extra, e da quel momento può accucciarsi per raccogliere dai cadaveri dei nemici, magari, un cappotto di riserva. Ha **tre poteri speciali**, i **richiami degli animali**, che a differenza di quelli di Mario si usano direttamente una volta presi: bisogna essere bravi a usarli nel momento migliore. Il richiamo del lupo uccide tutti i nemici a distanza, in un certo raggio, ma non quelli vicini; il ruggito dell'orso, al contrario, uccide tutti i nemici vicini; l'urlo dell'aquila distrugge tutti gli oggetti sopra il boscaiolo e apre "un fracco" di oggetti nascosti, che poi si possono prendere.

E c'è già una **dinamica**: si può decidere di **farsi colpire apposta** per perdere il cappotto e potersi accucciare all'occorrenza. "Quella è una strategia, quella è una dinamica." Il *one page design* li soddisfa molto; da Miro lo si può esportare come immagine, o come PDF con la versione a pagamento.

Non c'è tempo per far presentare tutti i progetti: il relatore dice che tutti quelli che ha visto nelle stanze sono stati "veramente eccezionali". Chiede quindi solo ai due gruppi di Super Mario di spiegare brevemente la loro idea, perché ne ha parlato poco con loro, e poi chiuderà; chi deve andare può andare.

0.7.12.2 Creepy Mario: la barra del panico

Il gruppo 1 ha introdotto una **barra del panico** come meccanica principale. La barra **sale**:

- ogni volta che si colpisce un blocco con la testa, perché ci si fa male e cresce la confusione;
- ogni volta che si tocca per sbaglio un mostro.

Più la barra sale, più **aumenta la capacità di correre e saltare**; ma se la barra arriva al massimo, **si muore**. Quando invece non si è in panico si è più lenti e goffi, ma al sicuro dal *game over*. La barra **scende** schiacciando i mostri, che restituiscono una piccola parte della barra, e soprattutto **salvando altri sopravvissuti**, che prendono il posto dei *power up*: bisogna prima distruggere la gabbia e poi prenderli, e restituiscono una grossa parte della barra. È così che il giocatore si bilancia.

Il relatore apprezza molto: è un **bilanciamento di feedback loop**. Da un lato il panico avvicina al *game over*; dall'altro, "e qua è la bellezza", il fatto che renda più veloci non è un *positive feedback loop* ma un **negative feedback loop**: è un potere positivo per il personaggio, ma un feedback negativo, perché alla lunga stabilizza il gioco, dato che si è più veloci e quindi più capaci di schivare i nemici; però ogni volta che, da quel momento in poi, non si schiva un nemico, si è un passo più vicini alla morte, e la destabilizzazione è maggiore[55]. "Ragazzi, perfetto, bellissimo."

[53] La trascrizione registra qui una ventina di "grazie a tutti" ripetuti, durante la riapertura delle stanze.

[54] Così nella trascrizione; "Michele Bosio" potrebbe essere il cognome di Michele o di un altro partecipante.

[55] La lettura del relatore è sottile: lo stesso stato (il panico) ha un effetto stabilizzante (più velocità, quindi più capacità di evitare danni) e uno destabilizzante (ogni errore avvicina al *game over* più che nello stato calmo). Si è riportata la sua spiegazione così come la dà; per la terminologia si veda la sezione 0.6.19.

Sarebbe bello, aggiunge, vedere anche le dinamiche e le scelte di gioco. Lì entrano i **nemici**: non fanno solo parte delle meccaniche, ma **spingono il giocatore a compiere determinate scelte**, determinati *behavior*. Ogni volta che si entra nell'ambito dei nemici e degli ostacoli si entra nelle **dinamiche**.

Il gruppo cita un'ultima idea: il comportamento dei nemici. Il classico percorso fisso dei nemici di Mario, intervallato da **azioni casuali**: uscire dal percorso, fare uno scatto verso il personaggio, per ricreare il **jumpscare** o comunque la situazione inaspettata, in cui conviene avere un controllo reattivo e quindi essere più vicini al *game over*. Il relatore lo collega all'**action feedback** (sezione 0.6.18), di cui non si era parlato direttamente: questo effetto casuale dovrebbe avere effetti speciali "molto molto molto presenti, visibili". Mettere tanta reattività e tanto feedback visivo è il concetto di **juice**[56]: rendere una meccanica, una dinamica, molto visibile. Lui si immagina un **fantasma** bianco che, quando scatta la cosa casuale, diventa improvvisamente trasparente e orrorifico, "tipo pum", e deve essere fatto molto bene; qualcuno aggiunge "un *flash*, un'immagine a tutto schermo". "Bravissimo."

0.7.12.3 Gloomy Mario: la torcia, la pala e Bowser

Il gruppo 3 riprende il progetto: l'obiettivo è **risalire dalla grotta**. Il *core loop* è leggermente modificato, con l'aggiunta di una meccanica di **raccolta e colpo** (*gather and hit*). La meccanica fondamentale è **Bowser**, il mostro che **insegue il giocatore**, prima dalla parte destra e poi, hanno pensato, dalla sinistra, per dare l'idea di una risalita a zig-zag; non può essere danneggiato e **sta sempre al limite della luce**.

La **torcia** è il nuovo oggetto, e anche la meccanica fondamentale: illumina lo schermo in base alla carica, con stadi al 100%, 75%, 50% e 25%; a 0% è *game over*. Bowser si comporta in base a quanto illumina la torcia:

- al 100% e al 75% si limita a stare al limite della luce e a inseguire;
- al 50% attira verso di sé i funghetti e potenzia la velocità dei Koopa e la velocità di sparo delle piante piranha;
- al 25% fa comparire degli **ostacoli** davanti a Mario, come blocchi, con cui si può interagire e che si possono rompere[57].

Gli altri nemici:

Dark Koopa. Le tartarughe camminano su un percorso predefinito, ma quando vedono il giocatore cominciano a correre verso di lui e ogni tanto si fermano. Se le si uccide **saltandoci sopra** lasciano il guscio, che è una **carica della pala**; se le si uccide con la pala non lasciano nulla.

Funghetti. In versione più oscura, camminano sul percorso prestabilito, a meno che Bowser non li attiri a sé.

Piante piranha. Sempre davanti al giocatore, sul terreno: si affrontano con la pala, scavandole da sotto, oppure si evitano del tutto con il platforming.

La **pala** piace al relatore: se richiede tempo per essere usata, cioè se è molto forte (per esempio distrugge gli oggetti) ma lenta, diventa interessante strategicamente, perché mentre si scava non si può reagire, e se ci si deve salvare bisogna ricominciare da capo. C'è un elemento di **rischio**: scavo e aspetto, o non scavo? Il gruppo ha pensato anche alle cariche: la pala parte da **zero cariche**; la prima volta che la si trova, in un blocco, ne dà **cinque**, che non si ricaricano col tempo: si ricaricano solo uccidendo i Koopa **nel modo corretto**, saltandoci sopra. A un certo punto bisogna quindi scegliere se usare una carica come via di fuga, per rompere il blocco davanti dopo aver sbagliato il platform, o per uccidere un nemico.

Nella costruzione delle mappe c'è il **blocco crepato**: ha una *texture* simile a quella normale, ma colpito con la pala, usando una carica, rivela passaggi segreti, scorciatoie, blocchi. Anche loro avevano pensato all'**albero che crolla** e fa da ponte, o all'albero che si taglia per entrarci dentro. Gli oggetti, sempre nei blocchi, che si rompono con il salto o con la pala:

[56] Nella trascrizione "il concetto di juiz": il *juice*, o *game feel*, è l'insieme di effetti visivi e sonori che rendono un'azione ricca di risposta.

[57] Nella trascrizione "a 0% Wipe, mi sono dimenticato il 25": lo 0% è il *game over*, e la correzione riguarda il 25%.

- la **stella**, che blocca per un certo tempo la diminuzione della torcia;
- il **fungo**, che dà una vita extra;
- il **fiore di fuoco**, che ricarica del 25% la torcia, "la fonte di luce", e sarà molto presente perché è la meccanica di base per andare avanti;
- le **monete**, raccoglibili come nel Super Mario classico.

Le monete servono perché tra un livello e l'altro c'è una **zona di scambio**, una zona di riposo con le casette del funghetto, dove le si scambiano con potenziamenti *una tantum* ("avevamo pensato a un cappellino").

0.7.12.4 La zona di scambio e le convenzioni di genere

Sulla zona di scambio il relatore invita a ragionare. Da un lato è vicina agli istinti e alle priorità del gruppo, alla sfida; ma **abbassa molto la sensazione di sopravvivenza e di paura**: "sono in una zona sicura, posso fare degli scambi". In alternativa, propone un **PNG**, un mostro che **scappa**: se lo si acchiappa, blocca un attimo lo schermo e permette di fare scambi, ma con un **timer**, quindi con poco tempo a disposizione.

È un caso tipico. Quando si gioca un gioco e c'è qualcosa che "non convince troppo", di solito è perché gli sviluppatori hanno messo qualcosa di molto legato al **genere** e allo **stile di riferimento**, pensando di doverlo inserire per necessità: "tutti gli RPG hanno quella roba lì, quindi la devo mettere". Ma, se si ragiona bene, **non c'entra molto con il player type**[58].

0.7.13 I consigli conclusivi

Un partecipante chiede il contatto Telegram del relatore, che lo dà volentieri[59]. Poi chiude, "così se qualcuno deve andare a pranzo facciamo libera tutti".

0.7.13.1 Non perdersi

Bravi anche i ragazzi di *Ticket to Ride*, che hanno avuto "un'ottima idea": **carovane e carrozze nell'Ottocento, con i banditi**. Erano un gruppo senza esperienza di game design, e hanno avuto la **sensazione di perdizione**: "tenetevela", perché è una sensazione che hanno anche i game designer. "Essere parte di un team significa anche **aiutare le persone a non perdersi**."

I punti da ricordare:

- **visione**, **esperienza**, **sensazioni**;
- il ***budget design***: un *hack* è il modo più veloce per fare un prodotto, e allora bisogna **modificare il meno possibile** il prodotto originale. In un gioco da tavolo o di ruolo, **modificare direttamente il testo**, partendo dalle icone e dagli idiomi, cioè dai nomi delle cose;
- **se si fissano dei punti, fissarli e non cambiarli**, a meno che non si abbia un'idea *out of the box*: se, invece di tenere punti fermi e cardini, si comincia improvvisamente a fare solo brainstorming, ci si perde.

Anche *Thunder and Prophecy* è un'ottima idea. Il gruppo aveva un po' meno esperienza nella scrittura di giochi di ruolo, ed è normale: ogni volta che il relatore vede game designer di videogiochi fare un design per un gioco di ruolo da tavolo, il problema è che il gioco di ruolo è **molto più minimale** di un videogioco, e bisogna ragionare molto di più sul **player type**, su **l'esperienza che i giocatori di ruolo si aspettano**. "Anche voi avete fatto un ottimo lavoro."

[58] Nella trascrizione "con il player tag"; come nel capitolo precedente, si intende il *player type*.

[59] Il relatore detta il proprio nome utente Telegram e, più tardi, il proprio indirizzo di posta elettronica dell'accademia; non vengono riportati qui. I contatti suoi e di Michele sono comunque all'inizio delle slide del primo giorno.

0.7.14 Il portfolio

Qualche "domanda secca". Come si presenta il lavoro nel **portfolio**? Lo schema fatto oggi, "sistemato, messo nome e cognome, la data e quant'altro, messo un paio di immagini", può diventare automaticamente un **one page design document** da mettere in portfolio.

Per chi lavora su *Lasers & Feelings*, il relatore consiglia **itch.io**: un sito dove ci sono giochi indipendenti e "un fracco" di *hack* di videogiochi e di giochi di ruolo. Cercando "*Lasers & Feelings hack*" se ne trovano moltissimi: un *hack* ispirato a *Squid Game*, e altri dai titoli come *Schemes & Teams*, *Gun & Elf*, *Diablo & Seraph*[60]. È "molto *easy* da usare". Mostra anche la sua pagina itch.io, con alcuni suoi giochi: giochini piccoli fatti per le *jam*, che fa ogni tanto, tranne uno "un po' più grossino"; e promette di mandare il link.

La cosa migliore, dice, è **pubblicare il gioco su itch.io**, con una bella pagina di descrizione, e poi metterlo nel portfolio, che "per chi non lo sapesse **è la cosa più importante nel mondo del game design**". Nel portfolio si possono mettere il *one page*, la pagina itch.io, l'High Vision Document, e far vedere **come si è passati da questo al prodotto vero e proprio**. Va bene anche mettere il PDF originale di *Lasers & Feelings* accanto alla versione modificata, per mostrare la struttura: "uno più l'altro, sì, assolutamente".

0.7.14.1 La licenza

Ultima raccomandazione: **mettere la licenza**. *Lasers & Feelings* è un gioco gratuito di cui si possono fare *hack*, **purché si citi l'originale**[61]. Bisogna quindi scrivere che il proprio gioco, *Thunder and Prophecy*, è un *hack* di *Lasers & Feelings*, gioco di tale autore, con tale licenza: "basta scrivere l'ultima parte alla fine".

0.7.15 Chiusura

"Ragazzi, è stato un piacere: credo che sia stato uno dei workshop migliori, **con più *outcome***, che io abbia mai fatto." Il relatore spera che sia stato utile ("a me è stato utilissimo"), di ricontattare tutti e di vedere che cosa faranno in futuro. Lo si può aggiungere su **LinkedIn**, come qualcuno ha già fatto; Facebook lo usa poco, usa soprattutto Telegram e LinkedIn. Giovanni chiede l'indirizzo e-mail, e il relatore lo dà, ricordando che tutti i suoi contatti, e quelli di Michele, sono all'inizio delle slide del giorno prima. Poi, "un minutino ancora" per le ultime domande, e i saluti: deve scappare a pranzo.

Punti chiave del capitolo

- La seconda giornata del workshop è pratica: quattro gruppi da tre fanno l'**hack** di un gioco esistente (*Lasers & Feelings*, *Ticket to Ride* o *Super Mario Bros.*), andando, nell'ADMDA, **dall'analisi fino alle meccaniche**, senza arrivare a dinamiche ed estetiche. Il risultato è un **one page High Vision Document**.

- Gli strumenti contano poco: Google Docs, Miro (o un'altra lavagna digitale, meglio se con i *frame* e l'esportazione in immagine), "anche carta e penna". Conta **lo strumento corretto per il giusto lavoro**.

- I passi: **analisi** di azioni (verbi) e regole (vincoli e rinforzi) e del **core loop**; **visione** dell'hack, cambiando un elemento; **esperienza desiderata** (emozioni e istinti); **pubblico**; **meccaniche**. Si fa design per giocatori reali, partendo dalla visione dell'esperienza.

- Nell'analisi **sensibilità diverse producono analisi diverse**, ed è un valore. Distinguere i **comandi di base** dalle azioni che con essi si compiono; distinguere le azioni dalle **dinamiche** (il punteggio, le strategie).

[60] I titoli sono letti a schermo e riportati come compaiono nella trascrizione; potrebbero essere imprecisi.

[61] *Lasers & Feelings* di John Harper è distribuito con una licenza Creative Commons che consente le opere derivate con attribuzione; i dettagli della licenza vanno verificati sulla versione che si usa.

- Due azioni sono distinte se sono **meccanicamente diverse** (il dado laser e il dado feelings), non se cambia solo l'oggetto (in Ticket to Ride **pescare è un'azione sola**, cambia il mazzo).
- Nei giochi di ruolo **il GM è un giocatore**, un *over player* con più poteri e discrezionalità, con azioni proprie: creare l'avventura, condurre il gioco. Non esiste il single player.
- Le emozioni vanno scelte con cura: in Mario **raccolta** più che avidità, **evasione** più che orgoglio; l'**orgoglio** del punteggio è **emergente**; l'avidità nasce nelle **dinamiche**, quando il gioco spinge a rischiare per avere di più. "Pensate agli istinti umani e non sbagliate mai."
- Il pubblico si conosce con la **ricerca**: le esperienze volute già suggeriscono il player type; poi Google (anche Notizie e Libri), BoardGameGeek, i commenti dei giocatori, gli amici. I player type di Bartle sono **falsi amici** (il killer non è un assassino, ma chi cerca emozioni forti): vanno descritti in termini che il team capisca.
- "X incontra Y" è un **high concept**. La **USP** è ciò che rende il prodotto unico per il suo pubblico.
- Due concept identici (Creepy Mario e Gloomy Mario) portano a giochi diversi: **l'idea è nulla, l'implementazione è tutto**.
- Per cambiare le meccaniche si parte dai **glossari**: i nomi danno sensazioni diverse; gli oggetti chiave possono diventare **idiomi**, ma se tutto è idioma il giocatore non capisce niente.
- Le **risorse** e i *power up* sono vincoli e rinforzi delle meccaniche. Togliere una *core mechanic* (il salto) e sostituirla (l'ascia) cambia l'esperienza; un vantaggio può avere un costo (il duck solo senza pastrano), e da lì nascono le dinamiche (farsi colpire apposta).
- I dadi: un dado solo dà probabilità uniforme, più dadi danno una **campana**; AnyDice aiuta, ma non bisogna perdercisi.
- Le scelte implicite dei giocatori portano ai **feedback loop**: la barra del panico di Creepy Mario è un esempio di bilanciamento. I nemici portano nelle dinamiche; gli eventi improvvisi richiedono **juice**, un forte *action feedback*.
- Attenzione alle **convenzioni di genere** inserite per abitudine (la zona di scambio sicura in un gioco di sopravvivenza): se non c'entrano con il player type, stonano.
- In un hack: **budget design**, modificare il meno possibile, partire dal testo, **fissare dei punti e non cambiarli**; un team serve anche ad aiutare a non perdersi. Il **portfolio** è la cosa più importante: one page, pagina itch.io, il percorso fino al prodotto, e sempre **la licenza** dell'originale.

Chapter 0.8

Esplorare il metaverso: dal web 1.0 alla società digitale

Questa ottava lezione zero è un **workshop sul metaverso**, che il relatore presenta come un'**"esplorazione del metaverso"** e, insieme, una riflessione sulla **"digital society"**: il metaverso è infatti una forma di condivisione di esperienze, e quindi di società digitale, ma è "anche molto più di questo e anche qualcosa di diverso da questo". Fin dalla prima frase il relatore mette una premessa: è un argomento di grande interesse in questo periodo e proprio per questo **ancora in evoluzione**, "cangiante, non semplice da definire e da padroneggiare". Una definizione del metaverso, bene o male, esiste, o almeno è stata tentata; ma prima che passi da un livello puramente teorico a qualcosa di concreto, riconoscibile anche visivamente sulle piattaforme tecniche che si usano, "c'è ancora" molta strada da fare.

Il relatore[1] parla a un pubblico ampio: il workshop si svolge **interamente online**, su Zoom, con il relatore che condivide alcune slide "come spunto" e i partecipanti che intervengono soprattutto **scrivendo in chat**, oltre che con qualche domanda inviata **per e-mail** e, verso la fine, con un intervento a voce. Le slide, dichiara, non sono il centro dell'incontro: sono uno spunto per "iniziare questa discussione, questo racconto, chiamiamolo come preferite". Nella seconda metà il relatore sacrificherà volentieri le parti più nozionistiche per lasciare spazio alle opinioni dei partecipanti, che trova "molto più interessante".

Tre avvertenze di lettura. La prima: il relatore dichiara più volte di non avere risposte definitive e di essere personalmente **scettico**; molte affermazioni sono **opinioni**, e il capitolo le conserva come tali. La seconda: il workshop è fatto in larga parte di **commenti alla chat**; i nomi dei partecipanti sono quelli che si leggono nella trascrizione, spesso senza garanzie sull'esattezza. La terza: la trascrizione è automatica, storpia nomi propri e termini inglesi ("Now Crash" per *Snow Crash*, "The Central Land" per Decentraland, "Nescape" per Netscape), perde pezzi di frase soprattutto quando il relatore legge le slide e in alcuni punti salta da un argomento all'altro; dove la ricostruzione non è sicura lo si segnala in nota. I riferimenti temporali vanno letti rispetto al momento del workshop, che si colloca con ogni probabilità **alla fine di febbraio 2023**[2].

Il percorso del capitolo segue quello del workshop:

1. l'avvio del collegamento (sezione 0.8.1);
2. l'**evoluzione del web**: 1.0, 2.0, 3.0 e oltre (sezione 0.8.2);
3. il metaverso come risposta a esigenze nuove e il destino del browser (sezione 0.8.3);

[1] Nella trascrizione il relatore non viene mai chiamato per nome, né si presenta. Di sé racconta alcuni dettagli che il capitolo riporta dove capitano: ha vissuto gli albori di Internet, quando lavorava in ambito universitario con Solaris; è stato a lungo un utente Amiga; ha "tantissimi anni di esperienza lavorativa" in ambienti italiani e internazionali; lavora con aziende che sviluppano mondi 3D con i motori di gioco; soffre di *motion sickness*; vive fuori dalla regione in cui ha la residenza.

[2] La data non è mai detta. La si deduce da tre indizi: il nuovo casco per la realtà virtuale di Sony "è appena stato lanciato" e il relatore sta guardando i primi *unboxing* (PlayStation VR2 è uscito il 22 febbraio 2023); le sfide via chat all'intelligenza artificiale sono "di moda da un mesetto" (ChatGPT è stato reso pubblico alla fine di novembre 2022); Microsoft ha "ultimamente" licenziato dipendenti chiudendo l'area dedicata al metaverso (i tagli sono di gennaio-febbraio 2023). La datazione è una deduzione, non un'informazione esplicita.

4. il confronto con la **realtà virtuale**: il nuovo casco di Sony, la motivazione all'uso, educazione e medicina, con una prima domanda arrivata per e-mail (sezioni da 0.8.4 a 0.8.5);
5. **Meta** e la sua visione del metaverso (sezione 0.8.6);
6. le domande ai partecipanti e le loro **definizioni del metaverso** (sezioni 0.8.7 e 0.8.8);
7. i rischi, le speranze e il "metaverso di Meta" visti dalla chat (sezione 0.8.9);
8. web 3.0, **token** e **avatar**, e un allarme: il "pay to exist" (sezioni da 0.8.10 a 0.8.11);
9. la **Human Computer Interface**, dalle schede perforate allo *spatial computing* (sezione 0.8.12);
10. **Snow Crash**, la nascita del concetto e il **real estate** virtuale (sezioni 0.8.13 e 0.8.14);
11. le esperienze impossibili, la vita sottratta e il **voto** nel metaverso (sezioni 0.8.15 e 0.8.16);
12. i tre modelli: **gaming engine**, **ambienti virtuali**, **economie virtuali** (sezione 0.8.17);
13. gli **NFT** e i loro usi (sezione 0.8.18);
14. i **quattro layer** del metaverso e la chiusura (sezioni da 0.8.19 a 0.8.21).

0.8.1 L'avvio del collegamento

La registrazione comincia mentre il relatore sistema Zoom, che dice di non usare abitualmente. Con la persona del supporto tecnico che lo assiste si accorda su un dettaglio: alcuni partecipanti hanno già mandato delle **domande**, che gli verranno inoltrate **via e-mail**, "l'unico modo"; il relatore le leggerà "mano a mano". I partecipanti cominciano ad arrivare ("ci sono 10 persone, 15"), c'è qualche "inghippo tecnico" con l'audio, poi il relatore riprende dal benvenuto.

Per il momento[3] i partecipanti sono **ventidue**, e altri continueranno ad arrivare per tutto il workshop (il relatore li ammette e li saluta più volte). Essendo così tanti, avverte, l'interazione non sarà semplicissima, "complicato da remoto": chiede quindi di usare la **chat** per domande al volo, interventi e commenti, e anticipa che la userà anche lui "per pungolarvi" quando gli interesserà sentire un parere. Terrà d'occhio anche le e-mail, per integrare nella discussione le domande arrivate da lì. Poi passa dalla propria immagine alla **condivisione delle slide**.

0.8.2 L'evoluzione del web

Il metaverso, spiega il relatore, è **l'ultimo passo di un'evoluzione teorica** cominciata molti anni fa, che riguarda la descrizione del web e di tutte le funzioni che mette a disposizione. Nel corso dei decenni il web si è evoluto da una prima versione 1.0 alle successive, ed è utile partire dalle origini con un piccolo riassunto.

0.8.2.1 Il web 1.0: un giornale online, in sola lettura

Agli albori il web è inteso come una **nuova forma di comunicazione**, qualcosa che andrà a prendere il posto dei mezzi editoriali tradizionali: a quel tempo le riviste, anche in forma digitale, ma distribuite comunque su supporti fisici come i CD. Internet introduce la possibilità di comunicare molto più velocemente e di aggiornarsi in modo praticamente costante, anche più volte nella stessa giornata. È un *improvement* tecnologico, ma al servizio di una comunicazione intesa **in maniera tradizionale**: qualcuno crea contenuti, notizie, approfondimenti, e li mette a disposizione di chiunque o degli abbonati a un servizio.

Le fonti sono quelle considerate attendibili, e hanno un'**autorità praticamente assoluta** nel definire il contenuto[4], che poi viene diffuso. Il web è così una **sorta di giornale online** su cui l'interazione degli utenti è minima: le notizie seguono gli orari decisi dalla fonte e, anche se fruite attraverso un computer, la fruizione è di tipo ***read only***.

[3] La trascrizione dice "prima di momento", con ogni probabilità "per il momento".

[4] Qui la trascrizione perde l'inizio della frase ("Che cosa significa? essere considerate attendibili..."); il senso è che i contenuti vengono prodotti da fonti riconosciute come attendibili.

0.8.2.2 Il web 2.0: la comunicazione diventa bidirezionale

Abbastanza velocemente si passa a una nuova idea di interazione, quella del **web 2.0**. Ci si arriva sperimentando tantissime soluzioni tecnologiche, spinti da una necessità evidente: rendere la comunicazione **bidirezionale**, fare in modo che gli utenti diventino **proattivi**. Prima commentando le notizie; poi, cosa ancora più importante, **contribuendo alle notizie stesse**, arricchendole, o diventando spontaneamente **nuove fonti** di informazione, per esempio creando club o siti personali in cui parlare dei propri argomenti di interesse.

La sperimentazione porta piano piano alle **piattaforme social**, che all'inizio "non erano così scontate ed evidenti": il concetto stesso di interazione social si sviluppa nei primi anni. Il punto importante è che i contenuti sono **sempre più generati dagli utenti**. Si perde un po' di controllo sulla qualità, ma si ottiene un vantaggio enorme: una copertura di argomenti di ogni genere, in più lingue, a livello mondiale. Internet diventa quella che in quegli anni veniva concepita come una sorta di **enciclopedia**, e l'esempio più semplice è **Wikipedia**, che nasce proprio con il web 2.0. Nonostante le minori garanzie di attendibilità, il meccanismo funziona e spinge il web sempre di più. I siti personali si moltiplicano a migliaia, tanto che alcune verifiche sull'uso dei siti ipotizzavano, in certi momenti, che **solo un quarto** dei siti mondiali fosse davvero utilizzato e che gli altri fossero "dormienti o morti".

Il grande successo del web 2.0 è dunque questo: il canale non è più *read only*, ma ***read and write***.

0.8.2.3 Il web 3.0: una realtà non ancora cristallizzata

Si arriva così al **web 3.0**, che "in modo molto semplicistico" sarebbe il web che tutti usiamo oggi, o almeno da qualche anno. Semplicistico perché questa realtà **non è ancora ben cristallizzata**: dal punto di vista della descrizione e dei servizi tipici non è una cosa conclusa e matura. Molti siti ne parlano in modo un po' confuso, e c'è chi introduce già, in maniera molto preliminare, il concetto di **web 4.0**: al relatore è capitato di vedere fornitori di servizi che definivano "web 4.0" strumenti applicativi ancora in sviluppo, o addirittura solo ipotizzati come concept. Cercando in rete si trovano molti spunti sul web 4.0, ma è tutto **molto prematuro**, dato che lo stesso web 3.0 è ancora in evoluzione.

Uno dei grandi aspetti introdotti dal web 3.0 è proprio il **metaverso**[5], che sta cercando di affermarsi e che, per certi versi, cerca di **sostituirsi al concetto stesso di web 3.0**. Mentre alcuni parlano dei servizi della successiva evoluzione, il 4.0, altri vogliono dimostrare che **il metaverso è in realtà il web 3.0**, perché è ciò che sta facendo qualcosa di veramente rivoluzionario rispetto al 2.0. Anche solo sul piano della ricostruzione storica e nozionistica, quindi, il quadro non è chiaro: il metaverso non è ben compreso "neppure dagli stessi teorici", né, aggiunge il relatore, da lui o dai docenti che cercano di insegnarlo, ed è una cosa poco chiara anche per chi ci lavora a livello professionale[6].

0.8.2.4 Che cos'è, allora, il web 3.0

Il web 3.0[7] va inteso come un'**interfaccia** che va molto oltre l'interazione *read and write* e rende possibili **nuove forme di interazione**, pensate e ideate magari dagli utenti stessi. Agli occhi dei server che gestiscono il flusso del web 3.0, l'utente non è più un input che arriva dalla tastiera di un computer collegato da qualche parte: comincia ad avere una sua **fisionomia virtuale**, cioè a essere un'**entità** con potenzialità e idee che non sono predefinite dal sistema, ma che possono evolvere introducendo nuove interazioni e nuovi modi di usare il web.

Internet diventa una sorta di **tavolozza** su cui sono gli utenti, o almeno dovrebbero essere gli utenti, a **dettare le regole di utilizzo**. Il flusso dell'informazione non va più soltanto dall'utente verso il servizio: è il servizio stesso, il web 3.0, a permettere agli utenti di gestire flussi **verso altri utenti**.

[5] Questo passaggio è molto rovinato nella trascrizione ("lo stesso web 3.0 è in evoletti uno delle grandi aspetti, delle grandi realtà introdotte da questo 3.0 che sta cercando di affermarsi e che per certi versi [...] cerca di sostituirsi allo stesso concetto di web 3.0"). La ricostruzione più plausibile, coerente con quanto segue, è che il soggetto sia il metaverso.

[6] Anche qui la trascrizione salta parole ("di riassunto dal punto di vista nozionist è molto compreso neppure dagli stessi teorici"); il senso, ripreso più volte nel workshop, è che il metaverso non è ancora ben compreso da nessuno.

[7] La trascrizione ha "Cos'ico.", probabilmente "Che cos'è, quindi" o simile, all'inizio della definizione.

0.8.3 Il metaverso e le esigenze nuove

Oggi si guarda Internet in modo molto diverso da anni fa. Il relatore pensa che la maggior parte dei collegati, come lui, abbia vissuto gli albori: quando c'era **Netscape**[8] come primo browser, o addirittura prima, quando ci si collegava con il modem. All'inizio una vera "concezione di Internet" non c'era: c'era una tecnologia che faceva capire che si poteva comunicare **per iscritto** e in modo **asincrono**, a qualunque ora, superando i problemi dei fusi orari che rendevano difficile comunicare con mezzi tradizionali come il telefono, e con una velocità di connessione con gli altri molto maggiore rispetto, per esempio, a un fax. Era una rivoluzione **principalmente tecnica**.

Per i giovani che iniziano a usare Internet oggi tutto questo è scontato: Internet fa parte della vita comune, e la sfida è **capire come usarlo in modi nuovi**. Il metaverso è una risposta a questa esigenza, ma probabilmente **non sarà l'unica**, perché questo tipo di esplorazione è ancora molto giovane.

0.8.3.1 Il browser dimenticato

Il relatore azzarda una previsione. Quando si arriverà davvero alla versione 4.0 di Internet, il metaverso sarà visto dalla generazione successiva come un semplice **supporto tecnico**, qualcosa di scontato. Non si penserà più al browser, non si discuterà più di quale sia il migliore per privacy o velocità di aggiornamento delle pagine: il browser sarà dimenticato "un po' nel sottofondo". Si penserà al metaverso, non perché diventerà il nuovo browser, ma perché sarà il **concetto** in cui gli utenti della prossima generazione andranno a interagire: per usufruire di certi servizi sapranno di dover **accedere al metaverso**, magari con le tradizionali schermate a video, con i caschi virtuali "o quello che sarà". È tutto da vedere, ed è molto interessante perché saranno proprio i partecipanti, in parte, a produrre soluzioni per questa fruizione, e in parte semplicemente a viverle.

0.8.4 Metaverso e realtà virtuale

Il confronto tra metaverso e **realtà virtuale** porta alle offerte tecnologiche del momento. Come in passato sono diventati nuovi strumenti il mouse o la registrazione di messaggi in MP3, oggi ci sono strumenti che cominciano a essere **abbastanza economici da diffondersi rapidamente** e che il metaverso userà subito. In che modo, è tutto da vedere: la realtà virtuale, pur esistendo da diversi anni, è rimasta **molto di nicchia** e, fino a oggi, anche a livello di prestazioni "non è stata neppure molto convincente".

0.8.4.1 Il nuovo casco di Sony

Chi è appassionato di realtà virtuale sa che è appena stato lanciato il **nuovo casco di Sony**. Il relatore sta guardando i primi *unboxing* e i primi test su YouTube: i giudizi sono piuttosto positivi e le prestazioni sono sicuramente elevate. Ma il punto non è questo: devono essere i **contenuti** a diventare il motivo per cui si compra una tecnologia del genere.

Usare la realtà virtuale per fruire di giochi già esistenti, di IP ben conosciute e lungamente replicate, a cui si aggiunge soltanto la possibilità di esplorare l'ambiente muovendo la testa invece della levetta del joypad, è "molto limitativo" e "non ha letteralmente senso": diventa solo un gioco per ricchi che vogliono provare una nuova apparecchiatura. La **svolta** arriverà quando ci saranno **designer capaci di pensare applicativi nati per la realtà virtuale**, che si possono sfruttare solo con la realtà virtuale e che, ridotti alle piattaforme tradizionali, non sarebbero efficaci o non sarebbero proprio utilizzabili. Lo stesso discorso vale per il metaverso: oggi si vede qualcosa che viene chiamato metaverso, ma "ci attende molto di più". Per diventare ciò che si spera, il metaverso dovrà introdurre **esperienze interattive nuove**, che creino **sensazioni prima non provate**.

Il relatore ha atteso con interesse il casco di Sony, ma non perché sia appassionato di realtà virtuale: in realtà **non la usa per niente**, perché ha grossi problemi di ***motion sickness***. Lo attendeva perché il gigantesco mercato della PlayStation permetterà di **diffondere molto questa tecnologia**. Sony ha fatto studi di mercato piuttosto ottimistici, secondo i quali potrebbe vendere **11 milioni di caschi** lungo il ciclo di vita[9] del prodotto: da vedere quanto sarà lungo, indicativamente sei anni, ma potrebbe essere molto di

[8] Nella trascrizione "Nescape".

[9] La cifra è riportata così come la dice il relatore, senza fonte.

più o non avere un termine preciso. Se arriveranno **IP abbastanza forti** da rendere il casco l'elemento più usato, trasformando la console nel semplice supporto tecnico che permette di vivere l'esperienza di realtà virtuale[10], allora ci sarà un futuro "a lunghissimo termine" per gli applicativi del casco.

Perché è importante, anche se per ora riguarda solo i giochi? Perché servirà a **sdoganare** questa tecnologia, e tutti questi sono passi necessari per entrare nel metaverso, o meglio per fare in modo che il metaverso diventi qualcosa di veramente utile e non soltanto un argomento di curiosità. Oggi, per il relatore, il metaverso **non esiste ancora davvero**[11]: non è importante, tanto meno indispensabile, nelle nostre vite; non ha sostituito i metodi attuali per fruire di determinati servizi; sta cercando di tradurli nei propri ambienti virtuali, ma non ancora in modo da rendere **preferibile** usare il metaverso.

0.8.4.2 Una domanda per e-mail: un metaverso aperto è un'utopia?

A questo punto il relatore controlla le e-mail arrivate, anche se riguardano argomenti successivi, "altrimenti poi mi rimangono arretrate". Enrico parte dal crescente interesse per la **standardizzazione dei protocolli** del metaverso e cita alcune iniziative: lo *standard forum*, l'*Open Metaverse Interoperability Group*, l'*Open Metaverse Foundation*, *OMA3*[12]. La sua domanda: **quando potremo avere un metaverso realmente interoperabile, open source, basato su standard e protocolli aperti e gratuiti?** È un'utopia o un progetto possibile?

La domanda anticipa cose che il relatore dirà più avanti, ed è difficile, perché una parte di essa va subito **contro le tendenze attuali**. Un metaverso interoperabile presuppone una **condivisione di intenti** da parte delle grandi aziende. Creare forum in cui si stabiliscono i protocolli si fa già, e avverrà in modo abbastanza spontaneo anche l'integrazione delle proposte in qualcosa di omogeneo, non si sa con quale velocità. Ma, pur essendo probabilmente nell'ordine naturale delle cose, **non è ciò che sperano le grandi aziende**: il metaverso è spinto da aziende che hanno interessi di mercato, e lavorare sull'open source non è proibito per un'azienda, ma non è la soluzione imprenditoriale migliore. Si dovrà trovare un **punto d'incontro**, che oggi non è chiaro; le supposizioni si basano "semplicemente sulla speranza". Per il relatore **non è un'utopia**, è possibile, ma ci vorrà **più tempo di quanto pensiamo** e sarà complicato arrivarci.

0.8.4.3 Gli albori di Internet: siamo ancora in sperimentazione

In chat qualcuno osserva che è la stessa situazione dei primordi di Internet: "nessuno si ricorda di AppleTalk?"[13]. Il relatore non lo usava: agli albori di Internet lavorava in ambito universitario con **Solaris**, sulla piattaforma della Sun ("che si ricorderà ancora meno gente"), poi è passato a tanti ambienti operativi ed è stato a lungo un utente **Amiga**, "per chi si ricordasse la Commodore". AppleTalk è un esempio di quella sperimentazione. Erano anni "molto belli": gli strumenti che venivano creati oggi sembrano semplicissimi, ma erano "l'uovo di Colombo", risolvevano necessità e cambiavano davvero la vita.

Esempi se ne possono fare per ogni periodo. Oggi c'è **WhatsApp**, che almeno in Europa è installato praticamente su ogni cellulare (in America un po' meno): il relatore lo usa di continuo e lo trova perfino uno strumento di lavoro migliore di tanti altri, forse quello che usa più spesso. Il discorso non è nostalgico: serve a far capire che **siamo ancora in fase di sperimentazione**. Fra dieci anni le cose saranno molto cambiate, **più nei servizi che nelle tecnologie**. Oggi stiamo subendo una continua corsa all'aumento dei frame rate e delle prestazioni delle schede grafiche, al 4K che diventerà 8K, ma sono cose "relativamente superflue" rispetto al **contenuto** di ciò che viene offerto. Siamo davanti a una **slot machine** che continua a girare, ma da cui non è ancora uscito nulla di veramente rivoluzionario, né per il metaverso né per la realtà virtuale.

[10] La trascrizione dice "trasformare la PlayStation 6 nel supporto tecnico". Il casco appena uscito si usa con la PlayStation 5: può trattarsi di un errore di trascrizione o di una proiezione sulla generazione successiva di console.

[11] La frase è spezzata nella trascrizione ("Oggi a me sicuramente il metaverso realmente esista"); dal seguito si capisce che il relatore intende dire che oggi non gli sembra che il metaverso esista davvero, o almeno che non sia ancora importante nelle nostre vite.

[12] I nomi sono resi come si leggono nella trascrizione. Corrispondono presumibilmente al Metaverse Standards Forum, all'Open Metaverse Interoperability Group (OMI), alla Open Metaverse Foundation e a OMA3 (Open Metaverse Alliance for Web3).

[13] Nella trascrizione "Apple Talk".

0.8.5 La realtà virtuale oltre il videogioco

Sulla slide il relatore ha scritto che ciò che **latita** è la **motivazione all'uso**: la nascita di nuove applicazioni per le quali il casco non sia un espediente, qualcosa che possa esistere **solo in funzione della realtà virtuale**. Allora si cambierà marcia e ci si sposterà "verso nuove autostrade", in una direzione tutta da scoprire. Si capisce perché il metaverso sia così discusso: dà prospettive interessanti, ma è ancora abbastanza indefinito.

La realtà virtuale, esistendo da diversi anni, ha portato avanti sperimentazioni in **nicchie** piuttosto diverse tra loro. Del videogioco si è già parlato; molto è stato fatto, anche se meno discusso e meno conosciuto, nell'**educazione** e in **medicina**.

0.8.5.1 L'educazione a distanza: una classe come piccolo metaverso

Per educazione il relatore intende i **corsi a distanza**, in cui si può entrare in un'aula e assistere a una lezione. Sembra superfluo, ma significa poter **interagire**: per esempio con le lavagne, scrivendo di persona i propri interventi, come lo studente che invece di stare seduto ad ascoltare (cosa che può fare anche da remoto) interagisce con il professore e con gli altri studenti per creare attività.

Una classe universitaria o scolastica in realtà virtuale si può già concepire come un **piccolo metaverso**: molto limitato, con regole ben precise e uno o due strumenti soltanto, come la lavagna su cui scrivere con la penna. Ma è un esempio che forse funziona **meglio di quello dei videogiochi**: il videogioco ha comunque un suo scopo (anche se oggi vanno di moda gli open world), un inizio, una fine, degli obiettivi. Una sala di studio virtuale invece **vive in funzione degli studenti** che ci sono dentro: è qualcosa di più esplorativo, più libero, più vicino al concetto che sta dietro al metaverso.

0.8.5.2 La medicina: operazioni a distanza

Un'altra nicchia che funziona abbastanza bene, o che almeno ha un grande interesse anche economico, è quella delle **operazioni a distanza**, e ancora meglio **automatizzate**, robotizzate. Il relatore premette di non essere un medico[14] e di non volersi sbilanciare. Queste operazioni esistono, sono state fatte in America e in Giappone; lui non è in grado di giudicarne i risultati, ma è una strada destinata a proseguire, perché in certe situazioni lo **stress**, la **fatica** e la **disattenzione** dei medici possono essere compensati dall'automazione e dall'intelligenza artificiale. Parlarne è però "pericolosissimo", perché le capacità dell'intelligenza artificiale sono ancora agli albori.

La realtà virtuale permette anche di operare a distanza pazienti che, per le loro condizioni, non possono muoversi, o che più semplicemente, per motivi logistici ed economici, non possono spostarsi da un continente all'altro. Il relatore ha visto fare cose molto interessanti in questo ambito.

È metaverso? Direi di no, risponde. In medicina si tratta di qualcosa di super specifico, che non si avvicina molto al concetto di metaverso; può però essere il **primo tassello** su cui crescono altre idee, come un vero e proprio **ospedale all'interno di un metaverso**: un ospedale che fornisce soluzioni vere, non che cura l'avatar virtuale.

Videogioco, educazione a distanza e interventi medici sono dunque tre esempi, "avveniristici", in cui la realtà virtuale può avere uno **scopo vero, oggettivo**. Dal punto di vista commerciale, è in medicina che ci sono i costi e gli interessi più grandi.

0.8.6 Meta e la sua visione del metaverso

Non si può non citare "il nostro caro" **Zuckerberg** ("non so se sia amico di tutti"). Facebook è l'esempio, già accennato con la domanda di Enrico, di un'azienda che si è inserita con forza nel processo di standardizzazione e di discussione delle caratteristiche del metaverso, non solo nei forum, ma proponendosi come **pilota della sua creazione**. In modo un po' tendenzioso, dice il relatore, si potrebbe affermare che Facebook ha una

[14] La trascrizione dice "io sono un medico, quindi non vorrei sbilanciarmi troppo": è quasi certamente "io non sono un medico", come confermano il "non vorrei sbilanciarmi" e il successivo "non sono in grado di giudicarne i risultati".

sua visione del metaverso e non ha molta voglia di discuterla, modificarla o adattarla a quelle degli altri: la vuole **imporre**. Si possono usare termini più blandi, ma Facebook, che diventerà **Meta**, ha motivi validi per voler rendere dominante la propria visione.

È un aspetto delicato. Rendere dominante il proprio metaverso difficilmente passa dalla dimostrazione che la propria soluzione è la migliore: è un processo molto più lungo, probabilmente destinato a fallire, perché arriveranno sempre punti di vista e ottimizzazioni dall'esterno a dimostrare che la soluzione di una singola azienda, Facebook o qualsiasi altra, non può essere la migliore al 100% sotto ogni punto di vista. Il modo di imporre il proprio metaverso passa quindi **attraverso altri meccanismi**.

0.8.6.1 Un metaverso per una nuova economia

Che cosa vuole creare Meta? Uno **spazio virtuale** in cui interagiscono gli utenti, ma con una finalità **sostanzialmente economica**: il metaverso come l'ambiente in cui prolifera **una nuova economia**. Per farlo bisogna spingere alcuni strumenti, che il relatore vedrà più avanti, anche con slide "un po' nozionistiche" e "leggermente più noiose", ma necessarie. Dal punto di vista aziendale significa **appoggiarsi a una blockchain**: pensare il metaverso come un nuovo mondo con un'economia "universale", ma universale all'interno di quell'universo, basata non più sugli scambi finanziari tra banche e sul forex, come nel mondo reale, ma sulla **blockchain**, con vantaggi e svantaggi. Ciò che si discute qui è l'**opportunità** di concepire il metaverso in questo modo.

0.8.6.2 Il limite logico: parlare del paradiso

Il principale problema di Meta, secondo la slide, è un **limite logico**: chi si definisce creatore, e quindi gestore, del metaverso parla necessariamente del **suo** metaverso. Proprio perché il metaverso è un concetto ancora così astratto, è un po' come **parlare del paradiso**: chissà com'è fatto. È un muro contro cui Meta si sta scontrando, perché è ovvio che altre aziende, meno potenti e meno diffuse, avranno una loro visione diversa di questo paradiso.

Dal punto di vista economico e imprenditoriale è un rischio enorme: il **proliferare di vari metaversi**, magari con le stesse tecnologie di base ma con servizi e utenze diverse. In futuro potrebbero esistere più metaversi, in ciascuno dei quali l'utente si iscrive e ha un'**identità diversa e separata**. Questo **frammenta il mercato**, rende necessario far comunicare i metaversi e quindi trovare protocolli d'intesa, **rallenta l'evoluzione** e soprattutto **rallenta gli investimenti**: gli investitori devono valutare in quale metaverso conviene operare, e diventa "un terno al lotto".

Sulla slide, citando Facebook e Meta, il relatore definisce il metaverso "futuribile e ineluttabile", anche dal punto di vista delle aziende: almeno per ora saranno loro a dettare i primi passi, **non gli utenti**. È un momento molto delicato per la **democrazia digitale**, perché una rivoluzione così grande rischia di non essere portata avanti dai suoi stessi fruitori.

0.8.7 Le domande ai partecipanti

Prima di approfondire, e prima di una pausa, il relatore prova a coinvolgere di più i partecipanti. Ognuno avrà un'idea più o meno precisa di che cosa sia il metaverso, e sarebbe "sorprendente" metterle a confronto: in aula lo si sarebbe già fatto, ma si può tentare in chat, scoprendo che **non tutti hanno la stessa idea**. Lancia due domande:

1. **che cos'è per voi il metaverso**, in una sola frase;
2. **che cosa vi aspettate di trovarci** rispetto a ciò che già usate. Tutti usiamo Internet per una quantità di servizi: transazioni bancarie, operazioni in posta, il sito dell'INPS, community per i videogiochi. Oltre a chiedersi che cosa sia il metaverso, bisogna cominciare a chiedersi **che cosa vorremmo trovarci** per capire se lo si vuole davvero usare.

Con queste domande comincia una **pausa di dieci minuti**. Al ritorno il relatore attende un po' più a lungo, perché vede arrivare molti commenti, e nel frattempo, per "pungolare ulteriormente" l'immaginazione, aggiunge altre domande:

3. ciò che **Meta** sta cercando di descrivere e proporre è la **giusta visione** del metaverso? È il metaverso in cui vorreste ritrovarvi, quello teorizzato in precedenza, o qualcosa di diverso?

4. dal punto di vista **imprenditoriale**, visto che molti vogliono lavorare nell'industria del videogioco o più in generale dell'intrattenimento: per come è oggi il metaverso, o per come sta diventando, **ci investireste**? Aprireste una **startup** che produce servizi per il metaverso, o addirittura una startup **all'interno** del metaverso?

Sono "provocazioni interessanti", ma intanto il relatore passa a leggere gli interventi, che sono parecchi e contengono "spunti molto validi".

0.8.8 Che cos'è il metaverso per i partecipanti

0.8.8.1 Giada: un luogo sociale

Per Giada il metaverso è un **luogo sociale**. La definizione è breve ma "azzeccata" e importante, perché la fruizione del metaverso deve partire da qui: deve essere un ambiente **molto più socializzante** di quanto sia stato finora Internet, dove chi interagisce è, o vorrebbe essere, totalmente **anonimo**. Nel metaverso l'utente deve essere un agente molto più attivo verso gli altri, ricreando una sorta di **socialità** tutta da indagare, che potrebbe avere regole diverse da quelle a cui siamo abituati negli uffici o nei luoghi comuni. Giada aggiunge che in questo ambiente si potrebbero fare azioni che vanno **dal divertimento all'acquisto alla creazione di nuovi posti di lavoro**, anticipando così la domanda sulle startup. Per il relatore comincia ad avere "una visione abbastanza veritiera" di che cosa potrebbe essere il metaverso.

0.8.8.2 Enrico: una definizione tecnica

Enrico manda una descrizione in inglese, che per come è scritta fa venire al relatore il sospetto che sia stata prodotta con ChatGPT[15]: descrive il metaverso come un **ambiente virtuale generato dal computer** in cui sia possibile, in modo **persistente**, inserire, modificare e cancellare oggetti e creare interazioni, un ambiente **programmabile con un linguaggio di scripting**. È una definizione molto tecnica, "come probabilmente la vedrebbe il computer", un po' fredda. Enrico prosegue in italiano precisando: un ambiente condiviso con più utenti su Internet. È valida, ma "spersonalizzata": bisogna fare il passo mentale in più per capire quali sono le proprie **aspettative personali**. Perché vorresti usare un metaverso?

Non tutti lo useranno. Convivono più generazioni, e molte, compresa quella del relatore, potrebbero non avere mai a che fare con il metaverso. C'è il rischio di creare un'ulteriore **incomunicabilità tra generazioni**, già oggi indotta fortemente da **TikTok**, che con il metaverso potrebbe diventare "qualcosa di esplosivo".

0.8.8.3 Marcello: una sorta di Matrix

Per Marcello il metaverso è un luogo virtuale in cui è possibile **immergersi con tutti i sensi** e muoversi come nel mondo reale, una sorta di **Matrix**: quindi un avanzamento più **hardware** che software. È la visione degli albori, quella con cui i media, i libri e i film hanno definito il metaverso: "romantica", possibile, non si sa se auspicabile, ma giusta. Passa da un'evoluzione soprattutto hardware, e il software deve essere al servizio dei nuovi strumenti.

Il relatore pensa sempre, e segue personalmente da tempo, all'arrivo dei **guanti**. Il casco è un po' limitante; ulteriori *add-on* potrebbero permettere non solo di essere dentro l'ambiente virtuale, ma di **viverlo con le sensazioni**. Esistono già guanti che, oltre alla sensazione tattile, fanno avvertire il **calore** (caldo e freddo) e perfino la sensazione dell'**aria**. Sono "pseudo commerciali": si possono comprare, ma costano **sopra i 4-5 mila euro** e non hanno quasi applicativi, perché è tutto sperimentale. Come i caschi di circa quindici anni fa, però, i guanti sono il prossimo passo hardware "veramente fondamentale". Con questi piccoli passi si comincerà a parlare sempre più di un metaverso simile a quello di Matrix. Pensando al film

[15] La trascrizione dice "mi fa venire sospetto che non sia stata fatta con chat GPT": è il costrutto colloquiale italiano con il "non" pleonastico, e il senso è il sospetto che sia stata scritta proprio con ChatGPT, come conferma il commento successivo ("come probabilmente secondo me vedrebbe il computer").

ci sono aspetti inquietanti, ma è una soluzione possibile nel tempo: quanto tempo, non si sa, "ipotizziamo una ventina d'anni".

0.8.8.4 Luca: la realtà aumentata

Per Luca il metaverso è una **sovrapposizione della realtà con la realtà digitale**, cioè la **realtà aumentata**: restare consci della realtà, come con i **Google Glass**. Per il relatore è qualcosa di diverso dal metaverso: una **compenetrazione delle due realtà**, probabilmente oggi la più auspicabile, che significa uscire dall'ambiente chiuso di Internet e del web 3.0 e portare servizi tipicamente web **dentro la propria vita**. Con i Google Glass si può guardare il cielo e vedere subito previsioni, temperatura, probabilità di pioggia; guardare un prodotto in vetrina e, siccome si compra solo online, vederne il prezzo su Amazon e comprarlo all'istante; osservare un edificio e leggere informazioni turistiche. Le interazioni possibili sono moltissime.

È metaverso? Probabilmente no, almeno non come lo vuole Facebook. Potrebbe col tempo essere inglobato nel concetto, perché i servizi della realtà aumentata potrebbero essere gli stessi condivisi nel metaverso, ma per ora sono **due strade che non si sono congiunte**. Il relatore aveva molte aspettative, perché questo ambito[16] è veramente utile, ma la tecnologia si muove **molto lentamente**, per buoni motivi:

- la **privacy**: gli occhiali oggi disponibili, a quanto sa, devono per legge accendere una luce per segnalare che si sta registrando un video (anche se "tutto si può hackerare");
- il **digital divide**: i servizi arrivano dal cloud, e magari entrando in un negozio non ci si riesce più a connettere;
- la **batteria**, la "grande spada di Damocle" di tutto ciò che è mobile, a partire dai cellulari.

Ci sono migliorie ancora da fare, e lo studio di soluzioni e applicazioni, anche concettuale, sta andando stranamente a rilento. Le informazioni che si potrebbero raccogliere con gli occhiali sono enormi[17], se non altro per monitorare la posizione o registrare ciò che si guarda più spesso per capire i gusti, ma sono molto difficili da tradurre in **informazioni catalogabili**: sono filmati, che vanno tutti analizzati, il che è probabilmente molto dispendioso. Per questo l'industria non spinge molto in quella direzione. Il relatore ringrazia per lo spunto, che gli interessa particolarmente.

0.8.8.5 Dario: un mondo parallelo

Dario vede il metaverso come un **mondo parallelo**, quasi all'opposto: due realtà differenti, quella reale e una totalmente virtuale, con servizi di cui non capisce bene l'utilità perché, nella sua visione, **replicherebbero servizi che esistono già** nel mondo reale. È una domanda giusta: ha senso che esista nel metaverso uno **sportello postale**, quando oggi ci si collega già dal cellulare? Lo stanno facendo davvero: ci sono aziende e sportelli telematici, non necessariamente di Poste Italiane, che stanno replicando i loro sportelli nel metaverso e investendo in questo. Probabilmente perché hanno **paura di rimanere indietro**: sentendo parlare di metaverso, non vogliono sembrare quelle che non offriranno i loro servizi agli utenti che amano starci. Ma, come dice Dario, **non è la strada giusta**: ha poco senso replicare servizi che già esistono. Se fosse tutto qui sarebbe deludente, ma di sicuro non è tutto qui: il problema è che **dobbiamo ancora capire quali siano le esperienze nuove**.

0.8.8.6 Francesco: il nostro io digitale

Per Francesco il metaverso rappresenta la **trasposizione del nostro io digitale** in uno spazio in cui interagire con infrastrutture e persone in modo completamente nuovo, un'esperienza che coinvolge direttamente l'utente anche attraverso i **sensi**, con i pro e i contro della situazione. È la descrizione teorica del passo di cui si parlava: coinvolgere l'utente in esperienze diverse. Fino a che punto coinvolgeranno tutti i sensi lo si vedrà,

[16] La trascrizione dice "perché il Tango X è veramente utile". Non è chiaro a che cosa si riferisca: potrebbe trattarsi di un errore per "il tema" o di un riferimento a Project Tango, la piattaforma di realtà aumentata sperimentata da Google.

[17] La trascrizione perde qui l'inizio della frase ("che teoricamente con gli occhiali sono enormi come flusso di informazioni"); il soggetto sottinteso sono i dati raccolti dagli occhiali.

perché la cosa può anche **spaventare**. Immaginiamo di entrare nel metaverso per **scalare l'Everest**, una cosa che la gente comune, se non è fatta di professionisti, può fare solo lì: se l'esperienza diventa troppo realistica può avere dei contro, può essere spaventosa, non piacevole. Per il relatore ogni esperienza, nell'ambito dell'intrattenimento, deve in linea di principio essere **positiva**; ma a seconda di come verrà implementato, il metaverso potrà essere "tutt'altro che positivo". Sarà quindi necessario introdurre delle **etiche**: argomenti molto delicati che richiederanno moltissimo lavoro negli anni a venire.

0.8.8.7 Leandro: una rete di mondi virtuali in 3D

Per Leandro il metaverso è una **rete di mondi virtuali in 3D**, ambienti condivisi a cui le persone accedono attraverso Internet. Il fatto che vi si acceda attraverso Internet, osserva il relatore, oggi è vero, ma potrebbe non esserlo più: il concetto si sposterà verso il **cloud** e potrebbe legarsi soltanto all'uso di caschi o occhiali, senza passare più per i computer e smettendo di usare i browser. L'idea di "andare su Internet" potrebbe perdersi col tempo, sostituita da quella di "andare un attimo a sbirciare che cosa c'è nel metaverso", qualunque sia la piattaforma con cui si entra.

È importante l'idea di una **rete di mondi**: le tante aziende che investiranno proporranno i loro mondi, che, siccome lì la fantasia non è limitata, potranno essere molto diversi. Si immagini un mondo in cui si usa un avatar che ci rappresenta e, accanto, un altro mondo di un'altra azienda o community in cui tutti gli avatar sono **cammelli**: per passare di là, ci si trasforma in cammello. Questa quantità di soluzioni possibili può generare grandissima **confusione**, e andrà in qualche modo teorizzata.

0.8.8.8 Roberto: servizi creati dagli utenti, VRChat e Ready Player One

Roberto scrive che i servizi di interesse saranno di natura diversa e **creabili anche dall'utenza** ("un altro spunto interessante"), sotto la **stessa piattaforma multimediale**, che faciliti la navigazione da un servizio all'altro e il collegamento tra i servizi. Per farla semplice, ciò che pensa è **VRChat** e il film **Ready Player One**.

Per il relatore *Ready Player One* non è poi tanto diverso: è una versione più verosimile del concetto di Matrix. Sei te stesso in un mondo dove fai azioni non possibili nel mondo reale: guidi un'auto che salta da un ponte e si mette a volare. È una visione futuristica, "magari ci arriveremo", ma oggi non del tutto verosimile. Il relatore, che non ama molto il film, lo riconduce al modo tipico dei **videogiocatori** di concepire il metaverso: un ambiente con **upgrade** che danno nuove capacità e permettono di divertirsi e **competere** con gli altri, un enorme **open world**, una visione molto legata al videogioco. *Ready Player One* è pioniere nel descrivere questa realtà, ma forse ne descrive **solo una versione possibile**.

0.8.8.9 Matteo: il pessimista

Matteo si dichiara pessimista: secondo lui il metaverso **sostituirà completamente la vita dei nostri figli**, prendendo il posto del mondo reale. Fisicamente non è possibile, risponde il relatore, ma se si pensa a quante ore la gente passa oggi sul cellulare, qualcosa di simile accadrà necessariamente con il metaverso. Il cellulare è facilissimo da usare, lo si ha sempre con sé, si connette perfino in metropolitana, e vince facilmente nell'attirare la nostra attenzione; il metaverso dovrà "faticare" di più per diventare una consuetudine del genere.

La visione di Matteo[18] è legata all'idea che i giovani stiano molto in casa e si **isolino**. È un pericolo teorizzato, ma il relatore ricorda quanto si diceva ai suoi tempi delle **console**, che avrebbero isolato i ragazzini impedendo loro di fare amicizia; col tempo si è scoperto che forse era il contrario: gli amici di scuola venivano più spesso a casa perché c'era un videogioco a cui giocavano tutti. E ragazzi che hanno difficoltà a socializzare negli ambienti comuni, su Internet, grazie alla **protezione dell'avatar** e alla possibilità di disconnettersi nelle situazioni di disagio, si dimostrano più coraggiosi e finiscono per avere moltissimi contatti. Se considerarli amici in senso tradizionale è da discutere, ed è proprio questa una sfida del metaverso.

[18] Il relatore parla prima di "figli" e poi di "nipoti"; probabilmente il riferimento è sempre alle generazioni future.

0.8.8.10 Amici artificiali

Molte delle interazioni nel metaverso, infatti, avverranno con altri utenti che saranno **un misto di popolazione reale e virtuale**[19]: un ulteriore passo probabilmente ineluttabile. Avere **amici irreali guidati da un'intelligenza artificiale** sarà disdicevole? Il relatore ammette di non avere una posizione.

Da circa un mese va di moda sfidare l'intelligenza artificiale via chat: è sorprendente e interessante, ma nulla di veramente efficace rispetto a ciò che diventerà. Se si importasse nel metaverso il livello di intelligenza artificiale di queste chat, per animare avatar virtuali, si avrebbero **finte persone** facilmente riconoscibili, perché l'intelligenza artificiale non è ancora in grado di creare una propria personalità né di superare il **test di Turing**. Si vede però come il metaverso diventi **terreno fertile** per convogliare tutte queste sperimentazioni, dietro le quali ci sono aziende con i loro interessi: "che piaccia o meno il metaverso, piano piano ce lo vorranno far digerire".

0.8.8.11 Un luogo dove imparare in sicurezza e il lavoro meno tossico

Un altro partecipante[20] scrive che, oltre al luogo sociale, manca un **luogo dove imparare**, più sicuro rispetto a determinate situazioni di vita e di lavoro. Il relatore è d'accordo: vale per i ragazzi con problemi di socializzazione, ma anche per il **lavoro**. Nel metaverso si possono creare ambienti in cui certe **dinamiche estremamente negative** degli uffici, indotte dalla personalità delle persone, siano letteralmente **bandite**: a beneficio dell'azienda, con una produttività molto superiore, e dei lavoratori, con un ambiente **meno tossico**.

È un tema che si trova su Internet e su LinkedIn, ma che si discute troppo poco, "lo posso garantire": il relatore ha tantissimi anni di esperienza in molti ambienti di lavoro, italiani e internazionali, e l'Italia, purtroppo, è uno dei paesi dove questi problemi sono particolarmente accesi. La **tossicità degli ambienti di lavoro**, dei capi[21] ma anche dei collaboratori, è una costante ed è uno dei motivi per cui da tempo si teorizza la necessità del **welfare**: non quello "finto, politico, di propaganda", ma quello per cui il lavoro è un posto dove la mattina ti va di andare, dove magari ti alzi prima perché vuoi arrivare in ufficio il prima possibile. Quasi non succede mai, ma andrebbe a beneficio anche delle aziende. Quando certi tipi di lavoro saranno trasferiti nel metaverso il tema tornerà attuale: il metaverso potrebbe essere **un ambiente dove è più facile lavorare**.

0.8.8.12 Gioele: perché preferire una vita nel metaverso?

Gioele chiede quali siano, a parte il gaming e l'informazione, i **reali vantaggi** di questa tecnologia (forse riferendosi più alla realtà virtuale) e perché si dovrebbe **preferire una vita nel metaverso** a quella reale. È una domanda senza una risposta univoca[22]. Ci sono tante persone **per niente soddisfatte della propria vita reale**, condizionata da regole e routine molto stressanti, con una qualità della vita bassissima, soprattutto nelle grandi città: è ampiamente documentato, basta guardare le classifiche annuali del *Sole 24 Ore* sui posti migliori dove vivere. Nelle grandi città lo stress è diventato insopportabile, e il metaverso diventa **evasione** in un posto dove si sta meglio. Può diventare una forma di assuefazione[23]; in teoria il relatore stesso non sarebbe d'accordo, ma capirà sicuramente le tante persone che preferiranno passare ore nel metaverso, dove si sentono più libere, più accettate e vivono in modo meno stressante.

[19] La frase è incompleta nella trascrizione ("tante interazioni che farete all'interno del metaverso verso altri utenti quindi un misto di popolazione reale e virtuale"); il senso, ripreso più avanti nel workshop, è che una parte degli interlocutori sarà guidata da intelligenze artificiali.

[20] Il nome è incerto: la trascrizione ha "Enrico dice che è un gruppatore, Erosa scrive". Potrebbe trattarsi di una partecipante di nome Rosa o di un commento di Enrico.

[21] Nella trascrizione "dei bosco a cui si lavora": quasi certamente "dei boss", cioè dei capi.

[22] Nella trascrizione "una risposta inautabile", probabilmente "univoca" o "inattaccabile".

[23] La trascrizione dice "può diventare a sue fazioni", quasi certamente "assuefazione".

0.8.9 Rischi e speranze dalla chat

0.8.9.1 Controllo e profilazione

Un commento avverte che, con l'informatica in mano a pochi[24], il metaverso rischia di diventare uno **strumento di controllo** e di alienazione dell'utente, trasformandolo in un semplice **produttore di dati** da profilare. "Ma questo succede già oggi", risponde il relatore: non siate così ingenui da pensare che esistano grandi aziende ("nomi non ne facciamo") interessate alla vostra qualità di vita. **Per le aziende voi siete un produttore di dati**: è già così da almeno cinque o sei anni, e si spinge sempre di più in quella direzione.

0.8.9.2 Ancora Roberto: la scena del ballo

Roberto precisa che *Ready Player One* è esagerato e si concentra sul gaming; lui si riferiva piuttosto al concetto di **mondi virtuali diversi che condividono la stessa piattaforma** e le stesse informazioni: nel film c'è una scena in cui, invece di giocare, si va a **ballare**.

0.8.9.3 Giada: che si integri senza troppi danni

Giada spera che il metaverso riesca a integrarsi **senza troppi danni con la vita reale**: usato in modo intelligente apre molte possibilità, usato male potrà creare molti danni; si augura che se ne colga il buono. Dipende, dice il relatore, da due fattori. Le **aziende**, che avendo un interesse solo economico seguiranno più o meno gli spunti della grande utenza; e gli **utenti**, da ciò che vorranno fare con il metaverso. Purtroppo la grande utenza, tendenzialmente, non ha mai fini molto etici o creativi nell'uso delle tecnologie. Basta pensare a **Fortnite**: un successo straordinario, un ambiente immersivo che è "un altro piccolo metaverso in sé", dove però non si fa altro che uccidere avversari per poi sbeffeggiarli con dei balletti. Il relatore non è ottimista; ma il metaverso non sarà **la causa** di esperienze negative, sarà semplicemente in linea con quanto già succede su altre piattaforme.

0.8.9.4 Leandro: la regolamentazione

Leandro ricorda che le aspettative sono molte, ma che in alcuni casi la **regolamentazione** di queste attività non è ancora definita. Infatti non lo è, ed è difficile capire quando comincerà a esserlo[25]. In chat qualcuno ricorda anche che è stato scritto *Ready Player Two*, che però non ha avuto molto successo; il relatore non l'ha letto.

0.8.9.5 Fortnite: un danno all'industria

Giada risponde che il danno di Fortnite la preoccupa poco[26]. Il relatore, pur senza voler allungare un discorso fuori tema, è "un po' più preoccupato". Fortnite è stato uno dei principali motori che hanno spostato tutti gli investimenti dell'industria dei videogiochi verso **modelli ripetitivi**, estremamente negativi[27], a uso e consumo delle **microtransazioni**: nulla di ciò che si può dire intorno a Fortnite è davvero positivo dal punto di vista delle abitudini che ha introdotto nell'utenza.

0.8.9.6 Il metaverso di Meta: nessuno ci crede

Riprendendo le slide, il relatore nota che **nessuno** ha affermato di vedere in Meta la realizzazione del metaverso come se lo aspettava. La sua risposta alla sua stessa domanda è che **neanche Meta sa esattamente che cos'è**: per ora è semplicemente quella che **ha alzato la voce più degli altri**. Si vedrà che cosa succederà.

[24] Il passaggio è rovinato ("Con l'informatica tutta la cia"); l'autore del commento non è indicato.
[25] La trascrizione qui si interrompe a metà frase ("anche perché proprio i paesi...") e salta a un altro commento della chat.
[26] La trascrizione ha "secondo Giada, il Dan e Fortnite mi preoccupa poco": la lettura "il danno di Fortnite" è una ricostruzione.
[27] Nella trascrizione "assamente negativi", presumibilmente "estremamente" o "massimamente".

0.8.10 Web 3.0, metaverso e token

Il relatore chiude il discorso sull'evoluzione del web verso il 3.0 con una slide di sintesi:

- il passaggio al web 3.0 è inteso come **infrastruttura**, ed è "tutto un rincorrersi": da due o tre anni molti siti si dichiarano 3.0, ma ognuno ne dà la propria definizione, proprio perché non c'è ancora[28] un punto fermo; l'elemento caratterizzante è l'**adozione della blockchain** e, di conseguenza, la creazione degli **NFT**;
- la **tecnologia della realtà virtuale** è invece **necessaria**: è l'aspetto hardware, il canale con cui oggi si pensa di entrare nel metaverso. In futuro potrà evolvere, ma oggi la tecnologia[29] e il metaverso vanno di pari passo;
- **metaverso e web 3.0 non sono la stessa cosa**, ma hanno **ampie aree di sovrapposizione** e si giustificano a vicenda.

Alla fine non è così fondamentale stabilire se il web 3.0 esista e sia compiuto, e quindi si comincino a fare passi verso il 4.0, o se sia ancora in evoluzione e destinato a coincidere con il metaverso stesso: "rischia di diventare filosofia". Discutendone, però, emergono elementi fondamentali, e uno di questi sono i **token** e gli **NFT**.

0.8.10.1 Il token come abbecedario del metaverso

Il token è, almeno dal punto di vista delle aziende, il **vocabolario**, o più semplicemente l'**abbecedario**, con cui si creeranno le esperienze nel metaverso. L'enorme quantità di informazioni che l'utente fornirà entrando nel metaverso, moltiplicata rispetto a oggi, le aziende che lo costruiscono la vorranno usare in qualche modo, se non altro per capire quali nuove esperienze creare.

0.8.10.2 Gli outfit per l'avatar

Un esempio che presuppone l'uso dei token sono gli **outfit esclusivi per gli avatar**. Succede già nei videogiochi, dove si comprano contenuti aggiuntivi per cambiare il costume del personaggio di **Street Fighter**, ma qui si va molto più in là: la **Nike** crea delle scarpe per il tuo avatar, tu le compri e le paghi **200 euro**, come scarpe vere, ma le usa soltanto il tuo avatar. È materiale per professionisti e artisti che potranno decidere di lavorare solo all'interno del metaverso[30]. Il concetto può sembrare quasi risibile, e c'è chi pensa che la gente disposta a spendere soldi così sia pazza; in realtà "non si può definire nessuno pazzo", si può solo prendere atto che prendono piede **modi nuovi**.

E funzionerà, almeno nel metaverso: quando i metaversi comunicheranno tra loro e avrete una vostra **identità** in questa nuova realtà, gli outfit comprati si potranno riutilizzare anche in un altro metaverso. Saranno **trasversali** e vi daranno un'identità magari migliore di quella reale, permettendovi di azzardare acconciature od oggetti che nella vita reale non avreste il coraggio di usare. L'avatar dovrebbe diventare **unico**, identificare l'utente ed essere usato perfino nei videogiochi.

0.8.10.3 Un avatar unico per tutti i giochi

È una cosa che il relatore non capisce perché le aziende non abbiano spinto in modo più aggressivo. Rispetto ai **Mii di Nintendo**, "abbastanza ridicoli", un **avatar realistico**, costruito dalle fotografie del proprio volto scattate con il cellulare e utilizzabile in tutti i videogiochi giocati nel metaverso che supportano quel formato, permetterebbe di avere automaticamente se stessi in ogni nuovo gioco, senza passare dagli "infiniti menu di creazione e customizzazione" che vanno tanto di moda e fanno perdere **30 minuti**. Il risultato finale è bello, ma sono estremamente limitanti, e non capisce perché siano ancora lo standard. Meglio personalizzare **una**

[28] La frase è tagliata nella trascrizione ("proprio perché non c'adozione della blockchain"). Il senso più probabile è che l'elemento caratterizzante del 3.0 sia l'adozione della blockchain, ancora non generalizzata.

[29] La trascrizione dice "web technology": dal contesto è più probabile "VR technology", la tecnologia della realtà virtuale.

[30] La trascrizione perde qui qualche parola ("potranno eventualmente decidere di lavorare solo all' concetto del genere quasi irrisibile"); il senso è che il concetto può sembrare quasi risibile.

volta sola, sul cellulare, e ritrovare il proprio avatar in tutti i giochi, Xbox o PlayStation che sia. Si aspetta che succeda, ed è quasi ovvio immaginandolo nel metaverso.

0.8.10.4 Il prezzo dell'alter ego

Il problema è **spendere soldi** per queste cose. È giusto e divertente avere un alter ego super personalizzato, ma le aziende che creano il metaverso in cui vive vogliono che paghiate **tutti gli outfit** e tutte le personalizzazioni, in un modo o nell'altro, direttamente o indirettamente. Qui dovrebbe insinuarsi un dubbio nella testa degli utenti: questa esperienza creativa, "divagante", in un mondo alternativo più bello della vita reale ha un **prezzo tangibile** che si **somma** a quello della vita reale. E non sarà certo una frazione di quanto costa vivere nella vita reale: semmai ci sarà una **spesa senza limiti** da saper controllare, "figuriamoci" se il problema si riflette sui figli, che hanno una visione del denaro che "piove dal cielo". Sono problematiche morali che partono fin dalla creazione del metaverso.

0.8.10.5 L'educazione alla customizzazione

I videogiochi, o almeno tutti gli open world, spendono molta tecnologia nell'affinare i menu di customizzazione dell'avatar per **educare**, e soprattutto educare i giovani, all'idea che un mezzo artistico come il videogioco vada fruito così: devono abituarvi a non voler fruire di un videogioco se non attraverso un avatar super customizzato, e più la customizzazione si spinge avanti, più diventa **a pagamento**. È uno scopo "proprio educativo, proprio voluto" dell'industria dei videogiochi di oggi.

La gente non ci pensa, perché vive a ritmi insostenibili e fruisce di tutto velocemente, ma pochissimi si rendono conto che i giochi oggi sono **molto più facili da completare**: non puntano sulla **sfida**, ma su un'esperienza spinta verso un mondo con **libertà d'azione**, dove non ci si cura minimamente della **storia**, perché "la storia non è più importante".

0.8.11 Dalla customizzazione al "pay to exist"

Un commento in chat osserva che le esigenze degli utenti "non sono molto lusinghiere": allora perché seguirle? Le aziende, risponde il relatore, seguono le esigenze degli utenti finali là dove questi **compreranno il più possibile** materiale per customizzare la loro esperienza. Questo funziona solo con una **centralizzazione** di tutto, e di metaversi, pensandoci, ce ne sono **cinque** attualmente in sviluppo, e probabilmente altri arriveranno. Enrico interviene: se si usa una blockchain, questa potrebbe essere la **piattaforma di interscambio sicura** tra i diversi mondi virtuali e le relative identità.

0.8.11.1 L'alfabetizzazione al metaverso

L'educazione che oggi si fa ai videogiocatori, Meta la vuole replicare su **tutti gli utenti generici**. Attraverso Meta e gli NFT stiamo assistendo all'**alfabetizzazione al metaverso** dei suoi utenti potenziali. Il rischio c'è, e i partecipanti lo evidenziano in chat: **non esiste un ente internazionale** che se ne occupi. Il relatore si chiede se esisterà mai. E se nascerà un ente che non regolamenti la blockchain, ma cerchi proprio di frenare queste tecniche di marketing e certe tipologie di esperienze, nascerà probabilmente **dentro il metaverso stesso**: bisogna considerare che il metaverso crei **leggi proprie**, non necessariamente simili a quelle della vita reale. Uccidere l'avatar di un altro utente, per esempio, potrebbe non essere così fortemente biasimato, ma tollerato in modo ben diverso.

0.8.11.2 L'allarme

Il relatore lancia un **allarme**, da tenere a mente e da ripetere "in ogni occasione". Il metaverso, entità giovanissima, "neonata", sta poggiando le proprie basi, proprio perché pilotato dalle grandi aziende, su un **ecosistema economico**. Non lo si sta pensando come un mondo di potenzialità che ispiri i creatori di contenuti a fare qualcosa di veramente nuovo e migliore, ma si stanno facendo tantissimi ragionamenti su

come creare l'**infrastruttura economica**. Così non diventerà migliore della realtà[31]: sarà soltanto "uno specchio un po' più colorato" della realtà come la conosciamo.

Sulla slide ha scritto: questo progetto di ampio respiro, di qualunque livello di eticità, nei prossimi anni dovrà essere concettualizzato intorno a un'**infrastruttura blockchain e NFT**, che purtroppo non sarà ideata da designer con concetti di **gamification** o da teorici di una società ideale, ma è pensata attualmente da **speculatori**. In altre parole, il metaverso rischia di diventare fin da subito una **copia virtuale** (o un'alternativa virtuale, se vogliamo) dei meccanismi negativi della realtà stessa.

0.8.11.3 Dal pay to win al pay to exist

Il relatore dà un nome a tutto questo, prendendo spunto dai videogiochi. Oggi il videogioco è diventato un **servizio**, e nel videogioco pensato come *service* l'utente è un **fornitore di denaro** che paga per vincere: vive un'esperienza ***pay to win***. Nel metaverso, allargando il discorso alle tante esperienze possibili, potrebbe diventare un'esperienza ***pay to exist***: avrai modo di crearti un'esperienza di vita sempre più realistica, ma **pagando**. Entrerai nel metaverso per pagare il **diritto di avere una vita alternativa**.

Sono disquisizioni ancora teoriche, ammette, e le slide successive saranno "ahimè" molto più tecniche; ma le questioni teoriche sono **più importanti**, perché sono la base di come si userà il metaverso e di chi, almeno in parte, lo userà. I figli dei partecipanti saranno sicuramente **la generazione del metaverso**, e sarebbe auspicabile che i genitori ne abbiano una visione precisa fin dal principio. È un'opportunità: capire che cosa sta succedendo, che cosa il metaverso vuole introdurre nelle nostre abitudini, **sfrondarlo** della visione romantica o divertente di *Ready Player One*, averne una visione **più realistica** per riuscire a usarlo nel modo giusto, "e non nel modo in cui le aziende vogliono che voi lo utilizziate".

0.8.12 La Human Computer Interface

Si passa a elementi più storici e nozionistici. Il relatore non ama il nozionismo, "ma necessariamente in un corso questo spunta a spallate fuori prima o poi". Il punto di partenza è la teorizzazione del modo in cui gli utenti devono **parlare con i computer** ("computer" in senso semplicistico; per essere attuali, si potrebbe dire intelligenza artificiale). È l'oggetto della **Human Computer Interface**, che in alcune università è materia di corsi o master: descrive come gli umani hanno storicamente interagito con gli elaboratori e come questa interazione evolve.

0.8.12.1 Dall'elaboratore in attesa all'entità semisenziente

Uno spunto riguarda proprio l'intelligenza artificiale e **ChatGPT**. Storicamente si è sempre considerato ineluttabile che l'elaboratore, l'interfaccia con cui interagiamo attraverso una UI, fosse un elemento **in attesa di istruzioni**, "al tuo servizio". Oggi, in un'epoca anche qui pionieristica, le intelligenze artificiali cominciano a sostituire questo concetto con quello di **entità semisenzienti**: l'elaboratore è in grado di simulare una vera interazione, non quella di una macchina in ascolto di istruzioni. In futuro le domande arriveranno **in senso contrario**[32]: sarà l'intelligenza artificiale a farvi domande, a proporvi attività, a chiedervi che cosa state facendo, a permettersi di avere **opinioni sulle vostre scelte di vita**. È un aspetto altrettanto interessante quanto il metaverso, e i due dovranno necessariamente confluire: che ruolo avrà l'intelligenza artificiale nel metaverso? Non ChatGPT, che oggi è "un po' semplicemente un divertissement", ma le nuove intelligenze artificiali dei prossimi anni.

0.8.12.2 Breve storia dell'interazione

Il relatore fa un *excursus* veloce, con una fotografia sulla slide:

[31] Qui la trascrizione fonde due frasi ("su come riuscire a creare l'infrastruttura del metaenterà migliore della realtà reale"); la ricostruzione segue il senso del discorso.

[32] La trascrizione perde l'inizio della frase ("arrivano in senso contrario, cioè sarà l'intelligenza artificiale a farvi delle domande"); la ricostruzione segue il senso.

Le schede perforate. Le *punch card* sono state, per il relatore, la prima occasione di interagire con gli elaboratori "in real time": prima si programmava un enorme computer introducendo un algoritmo e ci si sedeva ad aspettare l'output, anche per tempi molto lunghi; con le schede perforate l'operazione diventa molto più veloce, l'output è continuativo e la scheda permette di introdurre modifiche[33].

La linea di comando. Piano piano subentra la riga di comando: si pensi al **DOS**.

Il puntatore. Poi il puntatore, cioè il mouse, con le prime sperimentazioni dello **Xerox Star**.

Interfacce grafiche e multitasking. Introdotti più o meno contemporaneamente da **Amiga**, **Apple** e, ai suoi tempi, **Atari ST**.

È un'evoluzione continua che raggiunge il suo apice nella **GUI**: l'interazione non si basa più sulla programmazione vera e propria, ma è **visiva**, fatta di **icone**: si danno istruzioni con un clic su un'icona o trascinandola in un'altra finestra. Per chi l'ha vissuto è "estremamente affascinante"; oggi è scontato, ma è importante conoscere almeno le basi della Human Computer Interface, perché certe dinamiche si **riaffacceranno con il metaverso**: come si interagirà con un metaverso concepito come un mondo "semisenziente"? Si possono prendere esempi dalla storia e replicare ciò che ha funzionato, o bisognerà inventare **interazioni completamente diverse**?

0.8.12.3 Verso un'interazione più umana

Lo spunto dato in chat sulla realtà aumentata porta alla **mixed reality** e all'**extended reality**: soluzioni **ibride** che sono probabilmente il modo giusto di interagire con il metaverso, anche se non è ancora chiaro quale sia la migliore. È chiaro invece che, col tempo, le ore passate a scrivere su una **tastiera**, sul totale delle ore passate a interagire con un servizio o un elaboratore, **diminuiranno** sempre di più, verso un'interazione più umana. Si pensi a quanto erano lontane dalla comunicazione umana le schede perforate rispetto a oggi, quando si usano strumenti che cercano di riconoscere ciò che si dice: banalmente le **smart TV**, a cui si danno istruzioni con il telecomando non ripetendo un messaggio pre-registrato, ma perché alcune TV ("lo fa la mia, il mio LG") capiscono più o meno dalle parole chiave di che cosa si sta parlando e propongono soluzioni.

0.8.12.4 Lo spatial computing

Trasferita in un ambiente 3D, questa evoluzione prende il nome di ***spatial computing***: la prossima conquista, già in corso, dopo le schede perforate, il puntatore e il multitasking. Si tratta di **abbandonare il piano bidimensionale** a cui siamo ancora abituati, con strumenti sul monitor, sostituendolo con applicativi pensati per un **ambiente 3D**.

Il metaverso corrisponde, a livello di definizioni, a ciò che la Human Computer Interface ha ipotizzato come spatial computing? "Su due piedi" il relatore direbbe di no, ma ragionandoci è una domanda speculativa a cui non è semplice rispondere.

0.8.13 Snow Crash e la nascita del metaverso

Chi ha cercato di rispondere in modo più approfondito è **Neal Stephenson**, con ***Snow Crash***[34]: lo spunto da cui sono nati poi *Ready Player One* e il resto. Probabilmente quasi tutti lo conoscono, ma non si può non ricordarlo come uno dei momenti in cui è nato il concetto di metaverso.

La parola **metaverse** combina **meta**, che nel senso dato dal relatore significa "virtuale" ("e qua capite perché Facebook utilizza questa parola"), e **verse**, abbreviazione di *universe*: **universo virtuale**. Il termine è stato introdotto nel libro di Stephenson, che aveva ipotizzato uno spazio a cui ci si collegava interagendo con gli elaboratori e in cui gli umani erano riprodotti da un **avatar**. L'ambiente aveva limitazioni specifiche: era **solo urbano**, costruito intorno a una *Street*, con spazi di movimento limitati e regole da seguire.

[33] La ricostruzione storica è quella del relatore. Di solito l'elaborazione a schede perforate viene descritta come elaborazione a lotti (*batch*), e l'interazione in tempo reale viene associata piuttosto ai terminali e alla linea di comando che vennero dopo.

[34] Nella trascrizione "Neil Stephenson" e "Now Crash". Il romanzo è del 1992.

0.8.13.1 La corporazione e il real estate

Senza addentrarsi nei contenuti del libro, il relatore sottolinea un'idea "avveniristica" dell'autore: l'esistenza di una **corporazione**, reale e non virtuale, che sfrutta il metaverso come una grande landa per il ***real estate***. Spinge la creazione di nuovi contenuti perché si creino **necessità continuative** per gli avatar, cioè per gli utenti, che li portino a tornare sempre più spesso nel metaverso, fino ad avere una vera e propria **vita secondaria** lì dentro, che presuppone anche una **casa**. La corporazione rende il metaverso sempre più sofisticato perché vuole **vendere edifici** al suo interno.

È l'idea che l'**economia immobiliare possa essere virtuale**. È un'altra di quelle cose che i coetanei del relatore, non abituati a questi temi, considerano una follia: spendere soldi veri per una casa, una proprietà o un terreno edificabile che non esistono. Ma è stato profetizzato dal libro e **già succede**: è una delle primissime speculazioni introdotte dal metaverso.

0.8.14 Comprare immobili nel metaverso

Che senso ha tutto questo? "Ognuno deve trovare il suo." Il relatore azzarda una domanda: qualcuno tra i partecipanti ha **comprato edifici nel metaverso**? Sarebbe curioso sentire perché: per speculazione, o semplicemente per comprarsi una casa, tenersela e lasciarla in **eredità** ai figli. I vostri figli che erediteranno edifici virtuali: è follia? No, perché esiste già. La domanda è **se sia giusto** e come porsi di fronte a questa realtà.

Le risposte in chat[35]:

Esther non ha comprato nulla, ma pensa che chi l'ha fatto l'abbia fatto **per investimento**, perché il valore sale. Ma se il valore sale, osserva il relatore, è perché c'è sempre più gente interessata: non tutti possono comprare per speculazione, alla lunga non può durare. Bisogna capire che cosa pensano di farne, in futuro, quelli che hanno comprato, e perché quelle proprietà siano così appetibili.

Enrico aveva un'**isola in Second Life**. Second Life, a quanto risulta al relatore, è di nuovo in auge: siccome assomiglia in tutto a un metaverso, sta tornando di moda. L'idea è proprio la stessa.

Dario scrive che, "senza scomodare Totò" e la Fontana di Trevi, c'è gente che ha pagato **pezzi di luna**. Il relatore è uno di loro: ha un certificato di acquisto di un terreno sotto il cratere **Aristoteles** sulla Luna[36], comprato "un po' romanticamente" per circa **30 dollari**, se ricorda bene. Qui invece si parla di case con valori quasi paragonabili a quelli dell'immobiliare reale, quindi molto alti.

Esther aggiunge che poi qualcuno **ci costruirà sopra**. Bisogna capire che cosa: probabilmente negozi, edifici per concerti, cose che il relatore ha messo anche nelle slide successive, "ammesso che riusciamo ad arrivarci".

Michele, che precisa di non essere uno studente, ha comprato un pezzo di terra in **Decentraland**[37] per pura **speculazione** e per **diversificare** il proprio investimento: "un vero e proprio speculatore". Il relatore è curioso di sapere come andrà. Michele pensa che quella terra sarà usata per l'architettura, per ambienti di simulazione, per ingegneria e design.

Francesca [38] ha comprato su **Earth 2**, principalmente per speculazione. Al momento "tutto tace", il progetto non va avanti, e nel frattempo i developer hanno raccolto un bel po' di soldi grazie alla **FOMO**.

Almeno un paio di partecipanti, dunque, l'investimento l'ha fatto. È un argomento valido per attirare gente nel metaverso: farlo sembrare, e magari è vero, un luogo con un **valore speculativo**. È difficile prevedere che cosa succederà; il relatore spera che **non si crei una bolla**. È ancora molto presto, e del resto gli speculatori devono arrivare il prima possibile per sperare di fare buoni affari.

[35] Prima delle risposte compare anche un commento incomprensibile ("Il bibliotecario ha un chat GPT anche l'Itnam dice Enrico"), che il relatore dice di dover "andare a indagare".

[36] Nella trascrizione "sotto il vulcano Aristoteles": Aristoteles è un cratere lunare.

[37] Nella trascrizione "The Central Land".

[38] Il nome è incerto: la trascrizione dice prima "Francesca dice che io ho comprato su Earth 2" e poco dopo "Francesco ci scrive, no, l'avevamo già letto, The Earth 2".

0.8.15 Oltre la realtà

0.8.15.1 Le esperienze che puoi fare solo lì

Nel metaverso si possono vivere esperienze che si possono fare **solo lì**[39]: perché nella realtà si vive lontano dal luogo dell'esperienza; perché non se ne ha la capacità fisica, come il **free climbing**, per ragioni fisiche o perché si soffre di vertigini; o perché, semplicemente, non ce la si può permettere, come **guidare una Ferrari**. Nel metaverso, almeno per ora, una Ferrari forse costa poco: sarebbe curioso scoprire quando e se Ferrari aprirà i propri **showroom nel metaverso** e quanto farà pagare delle Ferrari da guidare con il casco. Sono cose ormai imminenti.

0.8.15.2 Il passo esaltante di Snow Crash

Il passo esaltante introdotto da *Snow Crash*, pensato da Stephenson a fini narrativi ma capace di anticipare tutti questi discorsi, è l'idea di vivere esperienze che non solo non si riescono a fare nella realtà, ma che **vanno oltre la realtà stessa**. Per esempio **volare**: cose che non si vedono granché in *Ready Player One*, ma che sono ipotizzate nel libro. O fare cose impossibili: nella realtà sei una donna, ma il tuo avatar è un uomo. "Con la fantasia potete spingervi ancora più in là." Questo fa crescere molto l'*appeal* del metaverso, e si capisce perché oggi se ne parli tanto: è un **mondo di possibilità**, che permette di immaginare una realtà con regole diverse, di cambiare la propria identità diventando più giovani, più belli, di un'altra etnia "o quello che sia".

0.8.15.3 Il mondo non è al tuo servizio

Bisogna però **vivere** in quel mondo, e il mondo **non sarà al tuo servizio**: il tuo avatar non è il padrone del metaverso. Quando accetterai di vivere una vita alternativa lì dentro interagirai con altre entità, che saranno intelligenze artificiali o altri umani, sicuramente un mix dei due. Sarai in grado di distinguerle? Ti potrai **fidare delle intelligenze artificiali**? Sono aspetti a cui la gente, secondo il relatore, non sta pensando, ma che andranno presi in considerazione.

0.8.15.4 Il tempo sottratto alla vita

Poi, **tutto il tempo passato nel metaverso è sottratto alla propria vita**. Estremizzando, se si arriverà a passare più tempo nel metaverso che nella vita reale, diventerà necessario trasferire lì molte **attività obbligatorie**, come **andare a votare** o altre cose non divertenti ma necessarie che oggi si fanno nella vita reale. La domanda non è se potranno essere fatte anche lì (sarà sì), ma se **dovranno** essere fatte lì e non più nella vita reale: "ci spingeremo fino a questo punto".

Enrico chiede se non sia meglio **farsi il proprio mondo virtuale**. Se intende crearsi un metaverso personale, risponde il relatore, un metaverso in cui ci sei solo tu perde uno dei suoi scopi principali, la **socializzazione**.

0.8.16 Votare nel metaverso?

Un partecipante scrive in chat: votare no, "e lo dico da esperto di sicurezza", per gli stessi motivi per cui non si fanno i voti elettronici. Non farebbe votare nel metaverso. Purtroppo, osserva il relatore, in Germania sono già state fatte sperimentazioni in questo senso, e sicuramente succederà.

Lui ci vede anche **implicazioni positive**, a partire dalla sua esperienza. Vive **fuori sede**, in una regione che non è la sua; spostandosi molto negli anni, ha mantenuto la residenza nella regione d'origine (altrimenti avrebbe dovuto cambiarla ogni anno). Per questo ha difficoltà a votare, perché in Italia, "un paese di cui si può dire tutto tranne che sia all'avanguardia", anche per il digital divide, **non si può votare da remoto**, "che è una cosa ridicola". Se il metaverso risolvesse questo problema sarebbe utile a lui, e quindi a migliaia di altre persone.

[39] L'inizio del ragionamento si perde nella trascrizione, che riprende da "fare solo lì, perché nella realtà...".

0.8.16.1 Un intervento a voce

A questo punto un partecipante chiede di intervenire a voce[40]. Saluta, si complimenta ("è veramente molto interessante") e, chiedendo il permesso di dare del tu, precisa che il problema non è di semplice "aggiustamento". Cita l'**RSA**, uno degli algoritmi più importanti della crittografia[41], e afferma che gli esperti di sicurezza hanno convenuto che **né la blockchain né qualunque altro sistema di voto elettronico è sicuro**: un dato tecnico molto importante, con ripercussioni oggettive. Sul piano politico si può dialogare, "ma quello è un altro paio di maniche"; entrambi concordano di non voler "sprofondare nel politichese".

Il relatore osserva che, se degli enti sono giustamente intervenuti su questo aspetto, lo hanno fatto **nel mondo reale**: non sa se avranno autorità dentro un metaverso. L'interlocutore insiste: non è possibile, perché trasferito in un ambito virtuale, o elettronico, il voto perderebbe le sue **caratteristiche essenziali**, quelle che a livello nazionale o internazionale sono fondamentali, come la **segretezza** e le altre[42].

Il relatore rilancia: perché si vota? Per eleggere dei rappresentanti. La stessa situazione potrebbe presentarsi **nel metaverso**, quando esisteranno enti e organizzazioni politiche che vorranno introdurre regolamentazioni **solo nel metaverso**: per farla breve, a un certo punto esisteranno **città virtuali con i loro comuni**, in cui magari si dovrà eleggere un **sindaco**, anche lui virtuale. Quelle realtà avranno probabilmente regole che diventeranno autonome, e quelle votazioni saranno comunque introdotte nel metaverso.

L'interlocutore capisce l'esigenza, ma chiude dicendo apertamente che, da cittadino, **non si fiderebbe**. "Probabilmente neanche io", ammette il relatore, per il quale oggi non è nemmeno un'esigenza, "però lo sarà". L'interlocutore ringrazia e si scusa del disturbo; per il relatore l'argomento è talmente interessante che potrebbe occupare un'altra ora di discussione.

0.8.16.2 Metaversi come pianeti alieni

Sono tutti scenari in divenire, che forse non sarà facile regolamentare. Bisognerà introdurre **elementi basilari** che identifichino il metaverso, il più possibile **democratici** e accettati dalle comunità; altrimenti sarà difficile che i numerosi metaversi che potrebbero nascere confluiscano in un tutt'uno. Per assurdo, si potrebbe arrivare ad avere **due o tre pianeti Terra** abitati da avatar che ai nostri occhi sono **alieni**, perché hanno ideologie completamente diverse, e che non interagiranno più con altri utenti[43].

Al momento lo guardiamo un po' da **spettatori**, perché è difficile avere una posizione netta: oggi è difficile dire che cosa sia giusto e che cosa sbagliato. Il relatore ha però una grande paura, quella introdotta parlando di Facebook: questi mondi sono costruiti con **ottiche aziendali**, "quindi tutt'altro che etiche". Da tutto ciò che si è detto, anche con Enrico, ricava che questo workshop ha l'aria di essere **il primo di altri appuntamenti**, in cui chi è interessato potrà seguire come evolve l'argomento: alcune cose dette oggi potrebbero cambiare nei mesi a venire, e ci saranno spunti nuovi da aggiungere.

0.8.17 I tre modelli del metaverso

Il relatore si "forza" a diventare un po' più nozionistico. La formalizzazione del metaverso non è del tutto assente: sono stati teorizzati **tre modelli**, più o meno accettati da tutti ma ancora in evoluzione, descritti dal punto di vista della produzione come **gaming engines**, **virtual environments** e **virtual economies**.

0.8.17.1 I gaming engine

Sono gli strumenti usati dagli sviluppatori, soprattutto di videogiochi, che tutti conoscono: **Unity**, **Unreal**, e il relatore aggiunge **CryEngine**. Sono gli stessi strumenti che, almeno all'inizio, useranno i creatori dei metaversi, perché il metaverso è un **mondo 3D**.

[40] Chi interviene non si presenta. Dal contesto è con ogni probabilità l'autore del commento "da esperto di sicurezza", e il relatore, subito dopo, parla di ciò che "abbiamo detto insieme ad Enrico": potrebbe quindi trattarsi di Enrico, ma la trascrizione non lo conferma.

[41] La trascrizione, qui molto frammentata, non riporta che cosa l'interlocutore dica esattamente dell'RSA; ne riporta solo la conclusione.

[42] La trascrizione elenca "Ripicabilità, segretezza, eccetera": il primo termine è incomprensibile, forse "verificabilità" o "irripetibilità".

[43] Qui la trascrizione salta qualche parola ("non interagiranno più con altri ut, al momento lo vediamo un po' come spettatori").

La confusione delle aziende. Qui il relatore porta la sua esperienza professionale. C'è molta confusione[44]: ultimamente molte aziende contattano i **system integrator** che hanno esperienza, o personale capace di lavorare con questi gaming engine, e chiedono che venga fatto **un metaverso per la loro azienda**. Perché confusione? Perché chi fa queste richieste **non sa davvero che cosa vuole**: chiede un metaverso perché tutti ne parlano e vuole essere al passo con i tempi. In pratica, chiede **ambienti interattivi**, una sorta di **videogiochi**.

Un amico del relatore, "abbastanza famoso" nell'ambito della gamification, "un vero guru", **Fabio Viola**, ne ha dato su Facebook, durante una discussione, una bellissima definizione: **il metaverso oggi è un videogioco realizzato male**. È così: le aziende e le startup nate ultimamente che hanno i metaversi nel portfolio, e dicono di saperli creare attirando clienti in quel senso, non fanno altro che usare Unity per creare **ambienti customizzabili**, di solito indirizzati a uno **store in 3D**. L'avatar è un personaggio che gioca in un ambiente "interattivo, per modo di dire", con interazioni più o meno simpatiche che dovrebbero però convincerlo all'**acquisto** di un prodotto. Non sono più gli store tradizionali noiosi, ma ambienti 3D: in sostanza videogiochi, e per questo fatti con Unity. Nella sua esperienza, se non il 100% delle aziende, sicuramente una grossa percentuale oggi confonde la vendita di questi prodotti con la realizzazione di un metaverso[45], ed è chiaramente sbagliato.

Serviranno **nuovi strumenti pensati per il metaverso**. Più probabilmente nasceranno **espansioni di Unity e Unreal**, perché oggi sono aziende troppo grandi e potenti per pensare che nascano concorrenti capaci di superarle: espansioni di questi gaming engine pensate più specificamente per il metaverso.

0.8.17.2 I virtual environment

Gli **ambienti virtuali** sono luoghi artificiali in cui **scambiare informazioni**. Esempi tradizionali sono **Twitter** e **Discord**: in Discord nascono community con un pubblico molto diversificato. Questi servizi sono accomunati dalla necessità di una **comunicazione molto veloce**, senza attese, senza dover prendere appuntamenti sul *calendar*. Il calendario e i concetti collegati non sono vecchi, avranno ragione di esistere ancora a lungo, ma non sono tipici del metaverso, che deve essere **molto più veloce**: più simile a Twitter, che punta sulla propagazione immediata di brevi notizie, e a Discord, che, creando la community, permette a tutti di sapere che il messaggio indirizzato ad altri utenti è stato **recepito subito**, senza mandare un'e-mail e aspettare notizie. Si parte da questi concetti di comunicazione moderna e li si trasferisce negli ambienti virtuali.

A livello di definizione si distinguono **due tipi** di ambienti virtuali:

- il **real world**, cioè la **realtà aumentata**;
- il **purely digital environment**: quando si pensa a *Ready Player One* si pensa a questo.

La tecnologia è diversa nei due casi, come si era visto anche con la chat. La fotocamera del cellulare usata in **Pokémon Go** è un esempio "non più moderno" e un po' adattato; i **Google Glass** sono un esempio più moderno di come far viaggiare questo flusso di comunicazioni negli ambienti virtuali.

0.8.17.3 Le virtual economy

Un partecipante aveva parlato dell'isola comprata in Second Life: **Second Life** è un esempio calzante, perché è stato uno dei primi ambienti interattivi, anzi sicuramente uno dei primi mondi virtuali o, se vogliamo, uno dei primi metaversi, "tra virgolette". Un altro esempio è **Decentraland**. Sono **economie virtuali** nel senso che funzionano con una **moneta propria**. La circolazione di moneta virtuale in Second Life ha dimensioni enormi: si stima che abbia superato il flusso di scambi in moneta reale di **piccole nazioni** del mondo reale. Second Life mostra come un'economia virtuale sia **necessaria**, debba nascere e nasca **ineluttabilmente**.

[44] L'inizio di questo passaggio si perde nella trascrizione ("Qualcos'altro aggiungo. un'azienda che viene a interloquire..."); dal seguito ("perché parlo di confusione?") si capisce che il tema è la confusione delle aziende.

[45] La frase è spezzata nella trascrizione ("una grossa percentuale dell'azienda oggiere la vendita di pure la realizzazione di un metaverso"); il senso più probabile è quello riportato.

0.8.18 Gli NFT

Si arriva così, puntuale, agli **NFT**, anche perché il tempo "si è dilungato". Gli NFT sono **token crittografici** basati su un modello di **blockchain** e **non possono essere replicati**. Perché sono così importanti? Perché permettono di creare un'economia basata su **manufatti che non esistono**, cioè virtuali: qualunque articolo, qualunque creazione artistica, anche solo un'immagine, può diventare un'entità virtuale e far parte di questa economia. Il discorso potrebbe essere molto più lungo, ma non c'è tempo.

Alcuni esempi di transazioni NFT, presi circa un mese prima e quindi probabilmente già non più attuali, mostrano quanto velocemente siano cresciute le dimensioni[46]:

Opera	Valore della transazione
The Merge	oltre 90 milioni di dollari
Everydays	69 milioni di dollari
Human One	28 milioni di dollari

Sono compravendite di oggetti rese possibili dalla tecnologia NFT e dai **token**, cioè da una moneta proprietaria virtuale, che però riconduce a **spese reali** fatte con soldi veri.

0.8.18.1 Chi vuole davvero gli NFT?

Anche questo va meditato. Ogni metaverso, ogni servizio nel metaverso può avere **un proprio token**: oggi le monete virtuali sono "decine e decine", e prima che si arrivi a poche ne passerà. Ma al di là della singola moneta, il meccanismo degli NFT che permette di creare questa economia: **chi lo vuole**? Chi lo sta spingendo, chi lo ha reso subito indispensabile per creare le esperienze e i servizi del metaverso? Le **aziende** che creano i metaversi, le major? Gli **speculatori** che sperano di guadagnare trasformando in NFT le fotografie di un loro disegno? O gli **utenti**, la gran parte delle persone che useranno il metaverso? I partecipanti sono un misto: speculatori e utenti insieme. Che cosa vorreste diventare nel metaverso? Vi fidate di un'economia costruita così?

0.8.18.2 A che cosa possono servire

Sulla slide ci sono alcuni esempi di usi degli NFT che non siano la semplice vendita di un'immagine:

- un **accesso VIP** a dei festival: per esempio un festival musicale importante (il relatore pensa a Sanremo, "beh quello non accadrà facilmente") che si svolge in tempo reale ma **solo in uno stadio nel metaverso**; per seguirlo e sentire le canzoni bisogna entrare nel metaverso e comprare il biglietto tramite NFT;
- **store** creati appositamente nel metaverso: gli store esisteranno anche nella realtà, ma Nike o Reebok potrebbero avere nel loro metaverso un negozio con prodotti **esclusivi**. L'ultima versione delle Nike Air, visto che vanno a ruba, si potrebbe comprare **solo nel metaverso**, non nei negozi: un prodotto reale, spedito a casa, ma acquistabile solo lì;
- gli **airdrop** e tattiche simili, applicabili un po' a tutto.

Sono **ponti tra il mondo reale e il metaverso**, ed è qui che l'NFT comincia ad avere più senso: non limitato all'ambiente virtuale, dove si spende per cose di cui fruisce solo l'avatar, ma una via di mezzo. Per esempio, con un **solo acquisto**, il tuo avatar compra un paio di scarpe che userà nel metaverso, e quelle stesse scarpe arrivano anche a te, **in forma fisica**, nel mondo reale. Per gli utenti sono cose interessanti, con una parte ludica e un valore aggiunto che può avere senso; per le aziende lo sono ancora di più, perché ampliano il business e spingono ad **aumentare i prezzi** dei prodotti con questo espediente.

[46] I titoli sono resi come nella trascrizione ("The Merge", "in Everdays", "in Human1"). Corrispondono presumibilmente a *The Merge* di Pak, a *Everydays: The First 5000 Days* e a *Human One* di Beeple; le cifre sono quelle dette dal relatore, arrotondate.

0.8.19 I quattro layer del metaverso

Un modello[47] descrive il metaverso attraverso **quattro layer**. Non c'è più molto tempo, quindi il relatore li riassume:

Foundation layer. Il livello **tecnologico**: tutto ciò che permette il funzionamento e la creazione del mondo virtuale, per esempio i **server**, la tecnologia anche hardware.

Infrastructure layer. Il livello dei **servizi**, cioè che cosa realizzare tramite la tecnologia: per esempio i **data center**.

Content layer. Il livello delle **esperienze**: non solo i videogiochi, ma le esperienze sociali, culturali e così via.

True metaverse. Il vertice, quello di *Ready Player One*: l'unica cosa che interessa all'**utente finale**, non a chi ci lavora ma a chi fruisce soltanto delle esperienze. È l'evoluzione finale, in cui gli utenti vivono in un **mondo coerente**, in teoria e idealmente, in futuro, **indistinguibile dalla vita reale**: magari completamente diverso dalla vita reale, ma vissuto come un'esperienza reale.

Al relatore viene in mente il commento fatto "ieri" da uno youtuber che ha provato uno dei giochi del nuovo casco di Sony: dopo un'ora, togliendoselo, ha vissuto alcuni minuti di **disorientamento**, perché non riusciva più a capire se la realtà fosse quella appena vissuta con il casco o la stanza in cui si trovava. Ammesso che fosse sincero, è il concetto teorico a cui punta il *true metaverse*: permettere agli utenti di **vivere**, e di conseguenza **lavorare** e **socializzare**, in un altro mondo.

0.8.19.1 La blockchain e le parti tralasciate

In tutto questo la **blockchain**, con le sue componenti di sicurezza, i suoi benefici e le sue controindicazioni, diventa una parte **preponderante**. Nella parte "noiosa" delle definizioni il relatore avrebbe parlato di **scalabilità**, **centralizzazione** e **sicurezza**, e di un "teorema"[48] che fa capire quanto sia complesso strutturare un metaverso anche quando si ha già a disposizione una blockchain su cui costruirlo. Esistono poi i **layer della blockchain**, descritti "in maniera universitaria". Pazienza se non si è riusciti ad affrontarli: erano elementi molto più nozionistici, ed è stato "molto più interessante discutere assieme delle vostre opinioni".

0.8.20 Perché le aziende investono

Delle ultime slide il relatore riprende solo alcuni esempi, tra realtà **mista** ed **estesa** e la **vendita al dettaglio**. Dal punto di vista delle aziende il metaverso è visto come un **grosso mercato** potenziale, magari grande quanto il mondo stesso; ma anche in una versione più ridotta è **più facile da gestire**. Nel metaverso scompaiono, o si gestiscono in modo molto più facile ed economico, molti problemi che incidono sulle revenue:

- le spese **logistiche**;
- le **dogane**;
- le **truffe**;
- i **pagamenti** non andati a buon fine;
- la **restituzione** degli oggetti.

[47] La trascrizione perde l'inizio della frase ("che descrive il metaverso attraverso quattro layer"): non si sa a chi o a che cosa il relatore attribuisca il modello.

[48] Nella trascrizione "del teorema K". La terna scalabilità, (de)centralizzazione e sicurezza corrisponde al cosiddetto *trilemma della blockchain*, secondo cui è difficile ottenere tutte e tre le proprietà insieme; è probabile che il relatore si riferisca a questo, e che "centralizzazione" stia per "decentralizzazione".

Questo spiega perché tantissime aziende stiano indagando e investendo, con risultati diversi. In controtendenza, il relatore cita **Microsoft**, che ultimamente ha cominciato a licenziare dipendenti **chiudendo l'area metaverso**, perché ha visto che per ora gli investimenti non valevano la pena: un messaggio di ulteriore confusione per gli utenti. Ma non c'è dubbio che in futuro Microsoft tornerà a investirci: per il momento non ha smesso del tutto, semplicemente era **ancora troppo presto** dal suo punto di vista.

0.8.21 Buon viaggio nel metaverso

Il relatore sorvola sugli altri esempi ("i concetti li abbiamo visti") e chiude augurando **buon viaggio nel metaverso**: a chi lo vivrà da **utente**, a chi (alcuni già lo sono) vorrà entrarci da **speculatore** e a chi ci **lavorerà**, tra gli ingegneri, i programmatori e gli artisti che lo costruiranno. Gli interessano soprattutto questi ultimi, "perché c'è veramente tanto da fare". Il suo augurio, la sua speranza per le generazioni successive alla sua, è che **sia fatto bene**: che il metaverso diventi davvero **un'esperienza migliore** di quella che può essere, certe volte, la vita reale. È scettico, come si è visto lungo tutto il percorso, "ma tutto è possibile"; passa la palla ai partecipanti e augura loro di contribuire alla nascita del metaverso nel modo migliore possibile.

Ringrazia per la partecipazione e per le tantissime risposte in chat, disattiva l'audio e saluta: "buona serata".

Punti chiave del capitolo

- Il **metaverso** è un argomento **ancora in evoluzione**: una definizione è stata tentata, ma non è chiaro neppure a teorici e professionisti. È l'ultimo passo dell'evoluzione del web.
- Il **web 1.0** era un giornale online *read only*, con fonti autorevoli e interazione minima; il **web 2.0** è *read and write*, con contenuti generati dagli utenti, social e Wikipedia; il **web 3.0** non è ancora cristallizzato, e c'è chi sostiene che **il metaverso sia il web 3.0**. Nel 3.0 l'utente acquista una **fisionomia virtuale** e Internet diventa una tavolozza in cui gli utenti dettano le regole.
- In futuro il **browser** sarà dimenticato: si "entrerà nel metaverso" per usufruire dei servizi, qualunque sia il dispositivo.
- La **realtà virtuale** esiste da anni ma è rimasta di nicchia. Il nuovo casco di Sony può sdoganarla, ma servono **contenuti nati per la VR**, che non abbiano senso altrove: oggi manca la **motivazione all'uso**. La corsa a frame rate e risoluzioni è secondaria rispetto ai contenuti.
- Educazione a distanza (una classe virtuale è un **piccolo metaverso**, più vicino al concetto del videogioco perché vive in funzione dei partecipanti) e **medicina** (operazioni a distanza e robotizzate) sono usi della VR con uno scopo reale, ma non sono metaverso.
- Un metaverso **interoperabile e open source** non è un'utopia, ma va contro gli interessi delle grandi aziende e richiederà più tempo di quanto pensiamo.
- **Meta** vuole imporre la propria visione: un metaverso come ambiente per una **nuova economia** basata sulla blockchain. Il suo limite logico è che ognuno parla del *suo* metaverso, come del paradiso; il rischio è la **frammentazione** in più metaversi con identità separate, che rallenta evoluzione e investimenti. Per ora Meta è solo "quella che ha alzato la voce più degli altri".
- Le definizioni dei partecipanti mostrano visioni diverse: luogo **sociale**, ambiente persistente e programmabile, **Matrix** con tutti i sensi (servono i **guanti** aptici), **realtà aumentata** (frenata da privacy, digital divide e batteria), mondo parallelo che replica servizi esistenti (strada sbagliata), io digitale, **rete di mondi 3D**, *Ready Player One*.
- Il metaverso potrà essere evasione per chi vive male la vita reale, un luogo di lavoro **meno tossico**, ma anche fonte di isolamento, incomunicabilità tra generazioni e rapporti con **amici artificiali**. Servono **etiche** e regolamentazione, che oggi mancano.

- Per le aziende gli utenti sono già oggi **produttori di dati**; i **token** sono l'abbecedario delle esperienze del metaverso.
- **Outfit** e personalizzazioni dell'avatar a pagamento, magari trasversali tra metaversi: un **avatar unico** per tutti i giochi sarebbe utile, ma la spesa rischia di essere senza limiti. I videogiochi di oggi **educano** alla customizzazione a pagamento e trascurano sfida e storia.
- L'**allarme**: il metaverso nasce su un **ecosistema economico** pensato da speculatori, non da designer o teorici di una società ideale, e rischia di essere uno specchio più colorato della realtà. Dal *pay to win* si può passare al ***pay to exist***.
- La **Human Computer Interface** va dalle schede perforate alla linea di comando, al mouse, alle GUI; l'IA diventa un'entità **semisenziente** che fa domande; la tastiera perde peso e il passo successivo è lo ***spatial computing***.
- Il termine **metaverse** (meta + universe) nasce con *Snow Crash* di Neal Stephenson, che anticipa anche il **real estate virtuale**, oggi già realtà (Second Life, Decentraland, Earth 2), tra speculazione e rischio di bolla.
- Il metaverso promette esperienze **impossibili o oltre la realtà**, ma il mondo non è al servizio dell'avatar, il tempo passato lì è sottratto alla vita, e attività come il **voto** potrebbero spostarsi lì, nonostante i dubbi sulla sicurezza del voto elettronico.
- Tre modelli: **gaming engine** (Unity, Unreal, CryEngine; oggi il metaverso è spesso "un videogioco realizzato male"), **virtual environment** (realtà aumentata o puramente digitale; comunicazione immediata alla Twitter e Discord), **virtual economy** (monete proprie, come in Second Life).
- Gli **NFT** sono token non replicabili che creano un'economia di manufatti virtuali; hanno più senso come **ponte** con il mondo reale (accessi VIP, store esclusivi, acquisti fisici e virtuali insieme).
- Quattro layer: **foundation**, **infrastructure**, **content** e **true metaverse**, idealmente indistinguibile dalla vita reale.
- Le aziende investono perché nel metaverso spariscono logistica, dogane, truffe e resi; ma è presto, come mostra la ritirata di Microsoft. L'augurio finale: che il metaverso **sia fatto bene**.

Chapter 0.9

Il game audio: ruoli, storia e produzione del sonoro nei videogiochi

Questa nona lezione zero è una **masterclass sul game audio**, cioè sul sonoro dei videogiochi: musica, effetti sonori, ambienti e dialoghi, e soprattutto il modo in cui tutto questo viene progettato, prodotto e **integrato** in un gioco. Il relatore, **Davide Pensato**, ne chiarisce subito la natura: è una masterclass, "un incontro in cui vi andrò a raccontare un po' cos'è, come funziona, come si lavora", e **non un workshop**: non si faranno esercizi. Se i conti con il tempo tornano, però, si darà almeno "uno sguardo" a qualche strumento di lavoro e a qualcosa di già fatto, perché anche questo "può essere sicuramente utile". E in effetti, dopo la pausa, il relatore aprirà dal vivo un *tracker* e un emulatore di Game Boy Advance.

L'incontro si svolge **online** e dura circa quattro ore, con una pausa di una ventina di minuti a metà. Gli iscritti sono quattordici, ma il relatore sa che non tutti seguiranno in diretta: all'inizio i collegati sono sei, poi ne arrivano altri due o tre. Quasi tutti accendono la **webcam**, su invito del relatore ("a me fa piacere, così ci vediamo"), che commenta soddisfatto: "tutti in webcam, benissimo, perfetto. Una situazione perfetta". Il numero ridotto permette uno scambio vero: il relatore fa domande, chiede pareri dopo ogni video, e i partecipanti intervengono a voce.

Tre avvertenze di lettura. La prima: gran parte della prima metà dell'incontro è costruita su **video di gioco** che il relatore mostra e commenta; la trascrizione ne registra solo il commento (e, talvolta, frammenti dei dialoghi dei video, riportati in modo confuso). Il capitolo ricostruisce di volta in volta che cosa si vede, sulla base di quello che il relatore dice, e segnala in nota quando il titolo del gioco non è detto o è una deduzione. La seconda: il relatore avverte che i suoi video sono "molto vecchi", ed è una scelta voluta (se ne parla nella sezione 0.9.6); gli esempi vanno letti come esempi di *tecniche*, non come rassegna dei giochi più recenti. La terza: la trascrizione è automatica e storpia moltissimi nomi propri e termini tecnici ("ESO Wars" per *Gears of War*, "WISE" per Wwise, "milchi tracker" per MilkyTracker, "Karsten Obansky" per Karsten Obarski, "Vengeokazu" per *Banjo-Kazooie*); dove la ricostruzione è sicura il capitolo usa la forma corretta, altrimenti lo segnala in nota. Durante la pausa, inoltre, la trascrizione registra alcune frasi senza senso che non hanno nulla a che fare con la lezione, e che il capitolo ignora.

Quanto alla data, la masterclass si colloca con ogni probabilità **tra la fine della primavera e l'inizio dell'estate del 2026**[1]: i riferimenti a "oggi", "adesso" e "ultimamente" vanno letti rispetto a quel momento.

Il percorso del capitolo segue quello dell'incontro:

1. le presentazioni dei partecipanti (sezione 0.9.1);
2. il percorso del relatore, il suo studio e due consigli di esperienza (sezione 0.9.2);
3. una parentesi sulla **contrazione** del mercato dei videogiochi e sui giochi *Triple-I* (sezione 0.9.3);
4. la citazione di Spielberg: l'audio comunica "più della metà" dell'esperienza (sezione 0.9.4);

[1] La data non è mai detta. La si deduce da alcuni indizi: un partecipante dice "adesso nel 2026"; il relatore dice che nel 2027 il suo studio compirà vent'anni di attività; racconta che il First Playable di Firenze si è svolto "la settimana scorsa", "giovedì e venerdì"; dice che lo Svilupparty di Bologna, che si tiene "di solito" a metà maggio, "c'è stato". È una deduzione, non un'informazione esplicita.

5. i tre elementi di importanza dell'audio: **potere evocativo**, **caratterizzazione** e **supporto al gameplay**, ciascuno con i suoi video (sezioni da 0.9.5 a 0.9.8);

6. una **breve storia dell'audio nei videogiochi**: gli anni Ottanta, i *tracker*, gli anni Novanta e il presente (sezioni da 0.9.9 a 0.9.12);

7. il **team audio**: audio director, sound designer, compositore, dialoghi e doppiaggio, audio programmer (sezioni da 0.9.13 a 0.9.18);

8. la seconda citazione, di Alexander Brandon, e il posto dell'audio nel **ciclo di produzione** (sezioni 0.9.19 e 0.9.20);

9. le **quattro categorie di suoni**: musica, effetti sonori, ambienti e dialoghi (sezione 0.9.21);

10. le domande finali e i consigli sugli eventi di settore (sezioni 0.9.22 e 0.9.23).

0.9.1 L'avvio e le presentazioni

0.9.1.1 Aspettando i ritardatari

L'incontro comincia con qualche minuto di attesa. Il relatore ha quattordici iscritti e sei collegati; sa che non tutti seguiranno dal vivo, e propone di aspettare "5-6 minuti". Poi, visto che "non c'è un flusso di persone che si aggiungono per cui ha senso stare ad aspettare", decide di partire: chi arriverà più tardi, verso le 14:15, sarà "accodato" alle presentazioni. Durante il giro arrivano infatti Alessio e Giovanni, e dopo la pausa il relatore saluta anche un Luca, che non ha la videocamera accesa ("fammi un cenno se ci sei"; "silenzio assenso, andiamo avanti").

0.9.1.2 Perché un giro di presentazioni

Il relatore comincia sempre, anche nelle lezioni in presenza, con un giro di presentazioni, soprattutto quando i partecipanti non sono tantissimi. Gli serve per tre cose: **conoscersi**, **far parlare** i partecipanti e cominciare a **focalizzare** gli argomenti. È vero che si tratta di una lezione singola, ammette, "però è sempre utile comunque confrontarci"; e in una lezione online lo è "a maggior ragione", per avere "almeno un minimo di scambio" e non "quattro ore in cui voi siete lì fermi e zitti e io parlo, parlo, parlo".

Ha preparato una slide con le domande, e chiede a ognuno di rispondere in modo rapido, "un po' scherzando, all'anonima alcolisti":

- **che cosa siete oggi**: che studi avete fatto, qual è il vostro percorso;
- quali sono i vostri **obiettivi**, presumibilmente nella game industry e magari nel game audio, ma è "sempre interessante scoprire gli obiettivi personali di ciascuno";
- se avete un'**attitudine** più per la musica o più per il sound design;
- se avete già qualche **esperienza**: un progetto, un aiuto ad amici, una jam, un lavoro;
- e, "come curiosità finale", il vostro **gioco preferito per l'audio**.

Sull'ultima domanda il relatore fa un'osservazione che ripete sempre. Di solito riceve **tre tipi di risposta**: chi ha davvero un gioco preferito per l'audio ("ha l'audio bellissimo, ha la colonna sonora"); chi risponde "boh, non lo so", e allora gli si chiede semplicemente il gioco preferito; e chi risponde "boh, io non gioco, non mi piacciono i giochi". Anche questa, dice, "è una risposta". In una masterclass così focalizzata, immagina, sarà più facile che tutti ne abbiano uno.

0.9.1.3 I partecipanti

Il relatore procede seguendo l'elenco testuale dei partecipanti.

Alessio è al quarto anno di un istituto tecnico a indirizzo informatico, dove con degli esperti esterni che vengono dall'accademia[2] lavora con Unreal Engine. Parallelamente cerca di inserirsi nel mondo del **doppiaggio**, molto presente nei videogiochi "soprattutto negli ultimi anni": alla domanda del relatore, precisa di voler fare il doppiatore, cioè l'attore, non il tecnico. Non ha un gioco preferito, ma ama le colonne sonore di *Assassin's Creed*.

Cindy ha frequentato il liceo musicale; si è iscritta per orientarsi verso il **sound design**, che l'ha sempre incuriosita. Gioca a tutto, "che sia un *Dark Souls*, che sia un *Hollow Knight*", e il suo gioco preferito per il sonoro è proprio ***Hollow Knight***: un indie 2D "molto leggero", che "può girare dappertutto", ma con colonne sonore e suoni "pazzeschi", di "una semplicità e di una bellezza" che l'hanno colpita dall'inizio alla fine. Ricorda anche le **voci** del gioco, che "tornavano in continuazione" e parlano "un linguaggio tutto loro", e i motivetti che "ti rimangono in testa"; tutto "si amalgama benissimo".

Fabrizio ha fatto un istituto tecnico, poi ha "virato totalmente" verso lingue straniere all'università. Condivide con Alessio la passione per il doppiaggio, da fan ("seguo doppiatori") e un tempo anche con il desiderio di provarci. Gli interessa la pipeline dello sviluppo, perché pensa di mettersi in proprio o di collaborare con amici usciti dall'accademia o da accademie simili. Tra musica e sound design non saprebbe scegliere; ama gli MMO, i JRPG e gli open world, e cita *Kingdom Come: Deliverance II*, in cui "anche solo andare a camminare nel bosco" è "un'esperienza fantastica" ("non ho idea di come ci siano riusciti"). Non ha esperienza di audio: è un "osservatore terzo". Il gioco che lo ha segnato di più sul piano musicale è **Pokémon**, fino al periodo del Nintendo DS, per la musica a 8 bit[3]: "potrebbe essere una risposta molto comune, però secondo me ci azzeccano quasi sempre".

Giacomo ha 22 anni ed è perito capotecnico in automazione. Oggi **programma videogiochi** con un suo team, come programmatore e game designer. Ha però alle spalle **otto anni** come sound designer, compositore e autore di testi, smessi l'anno prima: si occupa di mixaggio, mastering, composizione e scrittura "da dove tira il vento", e vende testi e basi. Il suo gioco preferito per il suono, nel senso degli effetti e delle voci, cioè della "tecnica dell'audio", è ***Spider-Man*** (il primo, il migliore della serie su questo piano); per la musica, ***Zelda***.

Giovanni ha 30 anni, è di Reggio Emilia e si è diplomato al conservatorio di Bologna in **musica per film e pubblicità**. Nasce come pianista; ha lavorato tre anni nel mondo discografico italiano in uno studio di Modena, imparando a suonare più strumenti, e poi ha scelto i videogiochi, affascinato dal "lato tecnico" di tutto quello che ci sta dietro. Lavora a due giochi, forse a un terzo, sia come compositore sia come sound designer, e sta imparando a programmare e a **implementare l'audio negli engine**. Ultimamente fa soprattutto il sound designer, "che mi sta appassionando". L'obiettivo è fare del game audio il suo lavoro "a 360 gradi", perché oggi fa anche altro in ambito musicale.

Nicole si è diplomata in economia e aveva cominciato da autodidatta a creare videogiochi "senza nessuna base"; dopo il diploma si è iscritta al **primo anno di Game Design** dell'accademia. Non ha ancora chiaro che cosa vorrà fare, continua a studiare anche da autodidatta, e non ha un particolare legame con la musica: si è iscritta "per avere una conoscenza in più". Il suo gioco preferito è ***Elden Ring***[4], il primo *soulslike*[5] che ha giocato, che trova "molto elaborato" sotto molti aspetti.

Alla fine del giro il relatore chiede, "per totale curiosità", quanti siano gli studenti dell'accademia: sono tre. Gli altri partecipanti vengono da fuori.

[2] Nella trascrizione l'accademia è resa in modi diversi ("IBU", "IBO", "Hive", "AVE"). Come negli altri capitoli, si usa semplicemente "l'accademia".

[3] Nella trascrizione "la musica in beat": quasi certamente "a 8 bit".

[4] Nella trascrizione "Ender Ring".

[5] Nella trascrizione "source deck".

0.9.2 Il relatore e il suo studio

"Vi siete un po' raccontati, adesso vi racconto un po' qualcosa di me", dice il relatore, anche per "togliervi il dubbio" che sia lì "per caso" e per giustificare il fatto di essere lì "a raccontarvi delle cose".

0.9.2.1 DP Studios: uno studio verticale

Il suo studio si chiama **DP Studios**, "tutto minuscolo, tutto attaccato": una "lotta" che il relatore fa ogni giorno e che pensa "di non vincere mai". Si occupa da molti anni di **game audio** ed è uno studio **verticale**: lavora soltanto sull'audio per i videogiochi. Il relatore stesso e i suoi collaboratori hanno anche altre competenze (c'è chi ha lavorato a cortometraggi, chi in pubblicità, chi fa audio "lineare", cioè per media non interattivi), ma è stata una sua scelta precisa fin dall'inizio: DP Studios fa solo videogiochi, e se qualcuno del team fa un lavoro di altro tipo, quel lavoro non entra nel portfolio dello studio. Finisce nel portfolio personale di chi l'ha fatto, oppure, in futuro, in un altro "contenitore" che il relatore ha in mente da tempo.

La ragione è una convinzione che il relatore esprime senza mezzi termini: non gli sono mai piaciuti gli studi che dicono "faccio sound per cinema, installazioni, videogiochi". Sono "cose talmente eterogenee" che "fare tutto bene è difficile"; preferisce piuttosto un'**associazione di più studi**, di più professionisti specializzati.

Lo studio ha quasi vent'anni ("nel 2027 abbiamo 20 anni di attività"), un sito (`dpstudios.it`), un team di professionisti che il relatore riesce in genere a tenere tutti occupati, "a seconda dei periodi, poi c'è chi più chi meno", e un portfolio "abbastanza vasto", di cui mostrerà un estratto.

0.9.2.2 Dall'ingegneria a Ubisoft

Il relatore nasce come **ingegnere delle telecomunicazioni**, con un percorso "tecnico-scientifico": liceo scientifico, poi ingegneria. In realtà comincia con **ingegneria gestionale**, seguendo "quella stupidata che ognuno di noi si è sentito dire almeno una volta nella vita: fai quello perché c'è lavoro sicuro". Durante il corso si accorge che non gli piace. Avrebbe potuto mollare; ma dalla fine del liceo aveva cominciato a suonare la **chitarra** e a **cantare**, senza mai frequentare un conservatorio ma prendendo lezioni private e seguendo percorsi formativi, e la passione per la musica si era allargata a una passione "proprio per il suono". Chiede consiglio, e gli dicono di passare a **telecomunicazioni**: è lì che si studia "tutto l'aspetto del suono, il segnale, lavorazione, digitalizzazione, trasmissione". Erano gli anni in cui il digitale c'era "da non così tantissimo tempo" e "si stavano ancora muovendo tante cose". Da allora il relatore ha sempre avuto un **dualismo**: "l'anima ingegneristica e tecnica" unita allo "spirito creativo del cantante, del chitarrista".

Finiti gli studi, cerca tutti i posti di lavoro che possano avere a che fare con l'audio, anche in ruoli tecnici. Scrive a tutte le aziende possibili (ricorda che c'era ancora qualche costruttore con una sede in Italia, "non so se era la Roland o la Edirol"), aiutandosi con un libro che in fondo aveva una lista di aziende. Un giorno va di persona al **laboratorio di informatica musicale** dell'Università degli Studi di Milano, a portare il curriculum. Il laboratorio non ha i fondi per prendere una persona, anche se gli servirebbe un profilo come il suo; ma il direttore gli racconta che sono passate delle persone di **Ubisoft**, che hanno lasciato un volantino: cercano un **audio designer**.

Qui il relatore apre una piccola riflessione sul **caso**: a volte la fortuna arriva, "ma non si sa se è veramente fortuna" o se è qualcosa che ci si è cercati, perché lui in quel laboratorio ci era andato di persona. "Ai tempi si portava ancora a mano il curriculum": prendevi la metropolitana e lo consegnavi in busta. E così diventa **audio designer negli studi Ubisoft di Milano**, assunto a tempo indeterminato, in un mondo "ahimè, diverso, molto diverso da quello che esiste oggi". Era la fine degli anni Novanta ("proprio '99", dirà più avanti), "un'esperienza abbastanza lontana da oggi".

0.9.2.3 Gli anni in Ubisoft

Negli anni in Ubisoft il relatore ha lavorato a **otto progetti**, che una slide mostra in miniatura. Il primo è ***Rayman*** per **Game Boy Color**[6], che gli ha permesso di lavorare "veramente con l'8 bit" (la cartuccia

[6] Nella trascrizione "Ryman".

è ancora in una bacheca del suo studio). Poi arriva il **Game Boy Advance**, dove "inizia a esserci un po' di digitale, poco poco"; poi i giochi per PC, Xbox, PlayStation, PlayStation 2, fino a **Xbox 360**: l'ultimo è ***Splinter Cell: Double Agent***. Il relatore ha quindi vissuto "da interno" tutto il periodo fino all'Xbox 360. Più avanti citerà anche ***Rayman M***, sviluppato metà a Milano e metà a Parigi (sezione 0.9.16).

Un partecipante lo interrompe: da piccolo ha giocato a un *Tomb Raider* di quella serie, "The Prophet"[7], e ce l'ha ancora a casa. Il relatore spera che gli sia piaciuto; il partecipante conferma, e il relatore aggiunge che, per quanto ne sa, il gioco "era andato bene".

Dopo circa **otto anni e mezzo** il relatore lascia Ubisoft, diventa libero professionista e fonda **DP Studios**.

0.9.2.4 Primo consiglio: la cura del proprio brand

Su questa slide il relatore si sofferma sempre su due cose, perché per lui è importante, oltre a "raccontare le cose che devo raccontare", anche "portare un po' di esperienza".

La prima: fin da subito ha scelto di presentarsi con un **brand da studio**. Ha scelto il nome immediatamente, con un logo che è "rimasto praticamente sempre lui" (lo stile vecchio, che si trova ancora in rete, era "fondamentalmente identico", solo meno moderno, con colori leggermente diversi), e sotto un *claim* "bello chiaro". Anche se era da solo, ha creato il nome di uno studio.

Il consiglio che ne ricava è questo: **date importanza alla brandizzazione**. Che decidiate di avere un nome di brand o di usare il vostro nome e cognome ("sono scelte lecitissime"), pensate da subito alla vostra **presentazione**:

- un **sito**, o comunque una pagina di riferimento con il vostro portfolio;
- almeno una **galleria dei lavori ben organizzata**;
- un bel **profilo LinkedIn** da poter girare;
- va bene anche un **Linktree**, ma "è bello che il Linktree abbia il loghettino" e una scelta di colori;
- un piccolo **logo** e dei **colori** riconoscibili.

Sono dettagli, ma in ambito professionale contano. Il relatore lo vede dal suo punto di vista quando riceve un curriculum: "una persona che ha due link, clicco, si apre la pagina, bella ordinata, si capisce tutto, comunque fa un effetto migliore". E lo vede quando incontra team di sviluppo, per esempio al **First Playable** di Firenze della settimana precedente: quando poi deve fare il *follow up*, senza un sito "è dura", perché "ad oggi un sito che spiega quello che fai" è importante.

0.9.2.5 Secondo spunto: otto progetti, tutti italiani

La seconda cosa riguarda le miniature. La slide nasce con **otto** miniature perché otto erano i progetti Ubisoft, e per coerenza il relatore ha continuato a usarne otto anche per lo studio: il portfolio sul sito è "un muro di progetti", e ogni tanto la slide va aggiornata ("già adesso dovrei aggiornarlo e non riesco mai a farlo"), togliendo un progetto e mettendone un altro.

Ebbene, da diversi anni, "forse più di 5-6 anni", quando il relatore aggiunge un progetto, quello nuovo è **sempre di uno studio italiano**. Ormai gli otto progetti in vetrina sono tutti italiani, mentre prima c'era sempre qualche cliente estero, perché i progetti "degni di essere mostrati" erano misti. Per il relatore è "un bel messaggio, un bel segnale": la **scena italiana sta crescendo** e "c'è lavoro, tendenzialmente c'è lavoro per tutti".

0.9.3 Una parentesi: la contrazione del mercato

Il relatore si rende conto che le sue parole possono sembrare "un po' di propaganda", "una campagna elettorale", in un periodo in cui il settore dei videogiochi è **in contrazione**, "anche forte", a livello mondiale.

[7] Con ogni probabilità *Tomb Raider: The Prophecy* (2002) per Game Boy Advance, sviluppato da Ubisoft Milano.

Ma insiste: rispetto a com'era la scena vent'anni fa, "la contrazione di oggi è oro". Oggi ci sono tante aziende, mediamente più lavoro, più opportunità, tanti team italiani che sviluppano. Apre allora una parentesi, "velocissima", sulle cause della contrazione, che spera finisca presto ("anche se c'è chi dice no, durerà anni, vedremo").

Prima di tutto, chiarisce, la contrazione **non è dovuta all'intelligenza artificiale** nel senso in cui se ne parla per tutti i settori, cioè l'IA che sostituisce il lavoro: "no, non è così". Le cause, per lui, sono tre.

1. **Il boom del Covid e i licenziamenti.** Durante la pandemia il videogioco ha avuto un boom; sviluppatori e publisher hanno investito tantissimo, e i team sono cresciuti "troppo", pensando erroneamente che la situazione sarebbe durata per sempre. L'esempio del relatore: hai un team di 20 persone, arriva il Covid, "figata, allora facciamo tre progetti in parallelo", e prendi 60 persone. Quando la domanda si normalizza, ti ritrovi con 60 persone e licenzi. Succede in tutte le aziende: anche Epic Games, ricorda, ha licenziato mille persone "due o tre mesi fa". I licenziamenti creano panico e **sfiducia**; e gli investitori, "che poi decidono un po' tutto", vedendo tagli e chiusure si allontanano.

2. **L'intelligenza artificiale, ma da un altro punto di vista.** Gli investitori che prima mettevano i soldi nelle aziende di videogiochi oggi li mettono nelle aziende che sviluppano intelligenza artificiale: "investono in Anthropic, investono in OpenAI". Il focus "di dove sono i soldi" si è spostato, e nel gaming ce ne sono meno.

3. **Il modello di business.** La tipologia di giochi che il pubblico acquista è diventata "un po' più piccola": oggi sviluppare un grosso titolo, anche solo un doppia A, è molto rischioso per uno studio magari semi-indie, ed è molto più semplice sviluppare giochi più piccoli.

A questo proposito il relatore segnala una nuova etichetta: i giochi ***Triple-I*** ("AAAI" nella trascrizione), una "classificazione nuova". Sono i giochi indie "un po' più grossini", "un po' più fatti bene": una via di mezzo. Non sono doppia A, che richiedono ancora team di 30, 40, 50 persone, ma titoli indie con team magari di dieci persone, "fatti bene, curati", che hanno successo. "Scusate un attimo per la divagazione, ho colto la palla": l'argomento sarebbe venuto fuori comunque, e il relatore preferisce "smarcarlo subito".

Il programma della giornata, annuncia poi, prevede una **panoramica sul game audio**, i **ruoli e le competenze** e, verso la fine, la **produzione**: il processo, la pipeline e l'integrazione dell'audio nel gioco.

0.9.4 Più di metà dell'esperienza: la citazione di Spielberg

Il relatore comincia sempre il suo racconto con una **citazione di Steven Spielberg**: "il punto di partenza di tutto". Spielberg non si occupa di videogiochi, ma la frase lo ha colpito, sia perché mostra la sua "attenzione e cura" per l'audio, la stessa che il relatore ha in ambito gaming, sia perché rappresenta bene che cosa sia l'audio per il videogioco. Il senso, come il relatore la riporta, è che **il suono e la musica comunicano più della metà dell'esperienza**, "ancor più di quello che vedi"[8].

Spielberg parla di cinema, un'esperienza lineare, ma riportata sul videogioco la frase vale "praticamente lo stesso". Anzi, la seconda parte, "ancor più di quello che vedi", è quella che "va a centrare l'obiettivo". Anche in un film si può ricreare un ambiente che non si vede, o anticipare la tensione con una musica anticipatoria; ma in un medium **interattivo** questo diventa ancora più forte. Per chi fa audio, dice il relatore, il videogioco è "veramente la quintessenza", "una leva molto importante": perché quello che il giocatore **non vede**, se lo **sente**, gli permette di **giocare e comportarsi in modo diverso**.

Gli esempi sono tre.

- **Il corridoio.** Stai percorrendo un corridoio e senti certi suoni: capisci che in fondo c'è qualcosa. In un film la scena successiva arriva comunque; in un gioco puoi **tornare indietro**, evitare quel passaggio se possibile, andare a curarti o a rinforzarti, lasciarlo per ultimo perché sarai più forte.

[8] La slide non viene letta per intero. Nella forma che circola più spesso la frase attribuita a Spielberg dice che "il suono e la musica sono il 50% dell'intrattenimento in un film"; il relatore la riassume a voce come "più della metà" dell'esperienza, "ancor più di quello che vedi".

- **La gara.** In un gioco di corse, alle ultime curve, sei testa a testa con un avversario. Invece di premere il tasto che mostra lo specchietto, rischiando di sbagliare nella foga e finire fuori strada, semplicemente **ascolti il motore**, a destra o a sinistra, e chiudi le traiettorie. Al relatore capita "spessissimo": senti l'avversario che aumenta sulla sinistra, "sta passando di lì", e lo chiudi.
- **La giungla e il dungeon.** In una giungla, graficamente, è difficile far muovere e vedere tutti gli elementi, "perché sennò il processore non ce la fa": ambienti di per sé molto ricchi hanno spesso poco che si muove a schermo. Ma con l'audio ci puoi mettere fruscii, foglie, vento, animali. In un dungeon puoi far sentire i topi che squittiscono, gli insetti, i pipistrelli, "ma non ci sono". Con l'audio si aggiunge "tantissimo di quello che non si vede".

0.9.5 I tre elementi di importanza dell'audio

Quando gli chiedono quali siano gli elementi di importanza dell'audio in un videogioco, il relatore risponde con tre parole, che presenta come "il mio contributo, la mia idea":

1. il **potere evocativo**;
2. la **caratterizzazione**;
3. il **supporto al gameplay**.

Il terzo è forse "la cosa più specifica proprio dell'interattività". In un film il potere evocativo c'è, la caratterizzazione c'è; al posto del supporto al gameplay si potrebbe parlare di *supporto alla narrazione*, "però in realtà non è la stessa cosa". Il supporto al gameplay è ciò che permette di **giocare in modo diverso** a seconda di quello che si ascolta, o che **guida** il giocatore.

Per ciascuno dei tre il relatore mostra alcuni video. Avverte subito che sono **video vecchi**, di giochi e situazioni datate, ed è in parte voluto: ha costruito tutto il racconto su questi video, ci è "molto affezionato", ha provato negli anni ad aggiornarli e ha delle idee per farlo, ma il racconto funziona talmente bene che ha "sempre paura" che, cambiandoli, "si rompe tutto". "Se vedete dei videogiochi e dite: caspita, videogioco di 50 anni fa, sappiate che è in un certo senso un po' voluto". Dopo ogni video, chi vuole può intervenire.

0.9.6 Il potere evocativo

Il **potere evocativo** è la capacità di **provocare reazioni emotive**: porta l'emozione, l'emotività. Si usa per **rafforzare** situazioni di gioco e anche per **anticipare** situazioni e snodi di gameplay; si usa moltissimo nelle *cutscene*[9], quando c'è una situazione d'impatto. Qui "la fa da padrona" la **musica**, ma non è l'unico strumento: anche il **sound design** ha un forte potere evocativo, come mostreranno i video.

0.9.6.1 Il trailer di *Gears of War*

Il primo video è "la quintessenza del potere evocativo", anche se non è una scena di gioco: serve a far capire "dove si può arrivare". È il **primo trailer di *Gears of War***[10], trasmesso in televisione quando il relatore lavorava ancora in Ubisoft. Il giorno dopo "un po' tutti ne parlavamo", in particolare con un collega programmatore che era anche musicista professionista, con un gruppo che girava l'Italia: si discuteva di come la **scelta audio** avesse reso "surreale" quel video.

La scena è una **scena di guerra**: fuochi, fumo, esplosioni, pioggia, un soldato che si muove e poi corre a combattere. Il relatore propone un esperimento mentale, "una cosa che prima o poi farò": se desse questo video da sonorizzare, probabilmente il **95%** delle persone metterebbe prima di tutto del **sound design**, perché la scena "chiama sound design" ("cioè addirittura piove"); e come musica, qualcosa di **tensione**,

[9] Nella trascrizione il termine compare in forme storpiate ("cazzi", "cuts in", "cazzin"): è sempre *cutscene*, le scene di intermezzo non interattive.

[10] Nella trascrizione "ESO Wars"; più avanti il relatore lo chiama "il trailer di Gears of War". Il brano che si sente è *Mad World*, nella versione lenta di Gary Jules: la trascrizione ne riporta alcuni versi ("All around me are familiar faces, worn out places, worn out faces...").

sospeso, poi una crescita, un *rise*, fino al climax finale con una musica d'azione adrenalinica, "perché lui va lì a combattere".

Il trailer fa "totalmente l'opposto": su un video di guerra c'è **solo musica**, una musica **malinconica e introspettiva**. E questa musica **sposta l'emotività**. C'è anche un "trucchettino" più sottile: mentre il testo dice *worn out faces*, l'inquadratura gira proprio sul volto del soldato. "Sono stati molto bravi". Guardando il video non si pensa alla guerra, ma a lui: è triste, "poverino", magari ha una famiglia, una casa; si pensa all'**ineluttabilità della guerra**, a concetti "più interiori". Anche quando il soldato parte e combatte, non si ha la percezione di una scena d'azione. È uno spostamento "grosso", da adrenalina e combattimento a malinconia: per questo il relatore lo mostra sempre, pur non essendo gameplay.

Cindy interviene: verso la fine il video le ha dato "proprio una sensazione di tristezza"; si è immedesimata moltissimo nella scena del soldato che corre, vista dall'alto, "come se stesse andando a morire". Ha avuto come una **premonizione** che quella sarebbe stata la sua fine. "Esatto, è proprio quello", conferma il relatore: l'obiettivo "è proprio stato centrato".

0.9.6.2 Un'idea mai realizzata

Il relatore fa una premessa al video successivo. Durante il **gameplay**, ottenere uno spostamento emotivo così forte è più difficile: un trailer "si presta di più". Per quanto ne sa, nessuno ha ancora fatto la scelta estrema di un gioco in cui la musica "non c'entri proprio nulla" a livello emotivo con quello che succede: "non so, un videogioco di guerra tutto con musica country". Ha anche cercato qualcosa del genere, senza trovarlo. "Potrebbe essere un'idea".

0.9.6.3 *Assassin's Creed*: la musica che si apre

Il secondo video è di un ***Assassin's Creed*** "bello vecchio". Si parte da una **cinematica** di tensione, "tensione estrema"; a un certo punto il controllo passa al giocatore (lo si capisce dal "classico movimento di camera" che segnala il passaggio) e la musica **cambia**, "si apre", come se si liberasse della tensione e dicesse al giocatore: *ok, adesso vai, perché tocca a te.* "Quasi ti andasse a dire: vai e purifica", è la lettura del relatore.

Il relatore usa questi primi video anche per dare qualche spunto tecnico, e qui individua **tre livelli**.

1. La **cinematica**, il caso più semplice: è un pezzo lineare, e la musica la segue.

2. Il **passaggio di controllo**: la musica si apre. Se il giocatore restasse fermo, o si muovesse in quella piccola zona, la musica andrebbe avanti probabilmente in **loop**.

3. Il **trigger**: a un certo punto il personaggio raggiunge un punto dove, presume il relatore, c'è un **trigger**. Un trigger è "praticamente un volume, un box invisibile" che, quando il personaggio lo tocca, comunica al gioco che si sta entrando in una zona d'azione: lo stato di tensione passa, per esempio, "da 1 a 2", e il sistema **alza il volume di una traccia**. Si sente entrare **in fade in** un *layer* con una parte ritmica, e la musica cresce, diventa "più sostenuta" man mano che c'è più azione.

In questo caso la musica **non sposta** l'emotività, come nel trailer, ma la **supporta**. E tutto avviene in modo "abbastanza trasparente": con gli strumenti di oggi queste transizioni si possono fare senza che il giocatore se ne accorga. Qui, nota il relatore, è anche relativamente facile, perché la sequenza è lineare e "sai più o meno quando è il punto di passaggio".

0.9.6.4 *Manhunt*: l'oppressione senza musica

Il terzo video mostra come si possa ottenere un effetto emotivo **anche senza musica**. Il gioco è ***Manhunt***, sviluppato da **Rockstar Games**. Il relatore lo descrive come "un po' il precursore di *Hunger Games*": il protagonista è un condannato a morte che viene "graziato", ma la grazia consiste nel buttarlo in una città fantasma dove deve uccidere chi incontra, altrimenti lo uccidono. Il gioco, sottolinea, è arrivato molto prima di *Hunger Games*.

Il relatore lo conosce bene per una ragione personale: lead designer del gioco era **Christian Cantamessa**, un ex collega di Ubisoft passato a Rockstar. Tra colleghi ed ex colleghi c'erano le "classiche prese in giro"

("adesso sto facendo giochi seri"), e quando il gioco uscì il team di Milano lo provò, un po' anche per poterlo prendere in giro, visto quanto il collega se n'era vantato. In pausa pranzo, nella zona relax, dopo cinque o dieci minuti il collega che stava giocando **passò il pad** a quello di fianco: "ho l'ansia, ho l'ansia, gioca tu". Il gioco aveva davvero questo effetto. Nei giorni successivi il relatore, che nel frattempo lo aveva giocato anche da solo e aveva notato che l'audio era "un po' particolare", contattò Cantamessa. Lui gli confermò che l'indicazione data al dipartimento audio era proprio quella: tutto doveva essere il più **opprimente** possibile, oltre alla storia e all'ambientazione anche l'audio. L'input era "oppressione, ansiogeno", perché quello era il tipo di gioco.

Nel video, e in tutto il gioco, il relatore fa notare diversi elementi.

- Tutto ciò che è **tutorial** e spiegazione arriva con una **voce filtrata**, con un rumore radio, come se venisse dalla telecamera: già questo dà un senso di **alienazione**.
- Durante il gameplay **non c'è musica**. La musica c'è solo nelle cutscene, ed è una musica "abbastanza ruvida", di tipo punk rock, se il relatore ricorda bene.
- L'**ambiente** è metà suono di città (ci sono anche finestre accese, e non si capisce se la città sia davvero fantasma) e metà **elementi tonali** che non si capisce se siano musica o parte dell'ambiente: "un soundscape un po' ibrido".
- Il **bonus** non fa "bling" come in Super Mario, ma un suono cupo.
- Avvicinandosi alla vittima si sente il **battito cardiaco**, fino alla fase finale, l'esecuzione, dove tutto è "più semplice" perché legato a un'azione violenta.

Senza musica, solo con il sound design, il gioco riesce a trasmettere "qualcosa di cupo, di scuro, di oppressivo". Il potere evocativo, conclude il relatore, si esercita con la musica "nella stragrande maggioranza dei casi", ma "molto spesso anche con il sound design".

0.9.6.5 Altri esempi: *Alien: Isolation*, *Silent Hill*, *Scanner Sombre*

Un gioco che il relatore aveva valutato di usare al posto di *Manhunt* ("a me questo video piace molto, difficilmente troverò qualcosa per sostituirlo") è ***Alien: Isolation***: camminando si sentivano ogni tanto dei suoni intorno, come se l'alieno si muovesse nei condotti proprio accanto a te. Poi non c'era. Non erano *jump scare*, cose che ti fanno sobbalzare: erano suoni che ti facevano paura *mentre camminavi*.

Una partecipante[11] si aggancia proprio a questo punto, al suono dell'ambiente. Negli horror di oggi vanno moltissimo di moda i *jump scare*, ed è "la cosa che trovo meno spaventosa di tutto": ti prendono alla sprovvista, ma trova molto più cupo e ansiogeno un horror in cui magari non c'è nemmeno un mostro, "non c'è effettivamente niente", ma il suono è fatto talmente bene (il battito cardiaco, "tutte queste cose") da **rievocare** un momento di tensione o di paura vissuto davvero. La paura, dice, è molto più una cosa **ansiogena** che un "oh mio Dio, c'è qualcosa che mi segue", e con un buon sound design la si percepisce molto meglio.

Il relatore è d'accordo e aggiunge un suo esempio: il suono che lo "terrorizzava tantissimo" era quello di **Pyramid Head** in ***Silent Hill***. Le prime volte sentivi il suono e poi lo vedevi passare; dopo, ogni volta che sentivi quel suono, *non* lo vedevi, e scattava subito il panico: "oddio, me ne devo andare". Ogni volta che lo sentiva cercava di andare via. "Ti metteva proprio l'ansia, solo sentire il suono".

Fabrizio, che nel frattempo cercava il nome, ne aggiunge un altro: ***Scanner Sombre***, visto su YouTube. Il giocatore ha un apparecchio che scansiona digitalmente la stanza, e si sentono fondamentalmente solo **rumori digitali di scansione**; più qualcosa si avvicina, più il rumore diventa distorto o si accentuano certi elementi. Anche questo, per lui, è molto più pauroso, "magari anche senza incontrare mai un mostro", di tanti horror che puntano sull'effetto sorpresa.

0.9.7 La caratterizzazione

Il secondo elemento è la **caratterizzazione**, e anche qui il videogioco è diverso dal film: la caratterizzazione è fatta **in tempo reale**.

[11] La trascrizione non dice chi parla; dal contesto è con ogni probabilità Cindy.

0.9.7.1 Una simulazione, non una sonorizzazione

Bisogna caratterizzare prima di tutto l'**ambiente**. "Ormai al giorno d'oggi" non si fa più un unico loop audio che gira sotto la scena: si **lanciano suoni** che si sovrappongono, e si possono variare man mano che il giocatore si muove, cambiando il set di suoni. "Diventa tutta una cosa **generativa**" (se ne riparlerà nella sezione 0.9.21.6).

Poi ci sono gli **oggetti** che riempiono il mondo. Un condizionatore appeso alla finestra emette un suono; se gli spari e lo danneggi, si sentono le scintille e poi il suono di un motore danneggiato. È una caratterizzazione che non funziona come al cinema, dove "vedo la scena e attacco i suoni in base a quello che succede": "la parola giusta è proprio **una simulazione**". Gli oggetti hanno degli **stati**, e il sound designer studia che cosa può fare ciascun oggetto. Una sedia da scrivania si può trascinare piano o velocissimamente, si può lanciare, si può distruggere: **ogni stato** che un oggetto interagibile può avere deve avere il suo suono. "E quindi è un bel lavoro".

0.9.7.2 Una città africana: superfici, riverberi, oggetti

Il primo video di caratterizzazione mostra una missione in una città africana, "se non ricordo male Kinshasa"[12]. Il relatore commenta mentre il video scorre.

- Guardandosi intorno **non si muove niente**: non ci sono macchine, non ci sono voci. Ma **si sentono**: dei *clack*, delle vocine, che danno l'idea di un villaggio africano. L'ambiente è caratterizzato dal suono.
- L'**equipaggiamento** del personaggio e i **passi**, che cambiano con ogni superficie: dal marciapiede all'asfalto, forse una ghiaietta, poi il legno quando si entra in casa.
- Entrando cambia l'**ambiente**: non si sente più l'esterno e si sente un **riverbero** diverso.
- I **colpi** sono tutti caratterizzati: un oggetto di legno, una ringhiera.
- Una **televisione** accesa trasmette una musichetta; appena la si colpisce, la musichetta si interrompe e si sente il rumore bianco della televisione distrutta.

Questo tipo di gioco, dice il relatore, è particolarmente adatto alla caratterizzazione, perché non c'è musica in gioco ma c'è "tanto, tanto, tanto" altro: le armi, la ricarica, quanta roba il personaggio ha addosso, se l'armatura è pesante o leggera, i passi su tutte le superfici, l'apertura delle porte, l'ambiente, il riverbero. E tutto è in tempo reale: se si torna fuori, il passo cambia di nuovo e si torna a sentire il clacson.

0.9.7.3 *Assassin's Creed II*: la Venezia di Ezio

Il secondo video viene dalla stessa serie del primo esempio di potere evocativo, con una caratterizzazione "super estrema". Si sente un dialogo in inglese ("Ezio, Leonardo says that the machine is ready..."): è ***Assassin's Creed II***, a Venezia. Il relatore anticipa un dettaglio: la musica che si sente è **diegetica**, cioè suonata da un'orchestra presente nella scena, e quindi caratterizza anche l'ambiente e l'epoca.

Oltre ai passi e a tutti i suoni del personaggio che si arrampica, la peculiarità del gioco erano le **voci**: gli **emettitori** erano attaccati alle coppie di persone, e camminando tra la folla ogni tanto partiva un piccolo dialogo. Tutti questi dialoghi che partivano casualmente davano la sensazione di una città viva. Quando Ezio sale, la musica e l'ambiente **scendono**; si sentono i piccioni; quando fa cadere una guardia, si sente il suono della caduta; poi i fuochi d'artificio. "È proprio un lavoro minuzioso e complesso", con una cura di tutti i dettagli, dai passi agli appigli, ai fuochi, alle voci.

L'obiettivo di questo lavoro, e in generale del sound design, è **contestualizzare** e **far sentire immersi**; in questo caso vale anche per la musica, che è diegetica. Il relatore riassume il cammino fatto: "abbiamo l'emotività, l'immersività, se vogliamo, e arriviamo poi al supporto al gameplay".

[12] Il titolo non è detto. Una missione a Kinshasa c'è in *Splinter Cell: Double Agent*, a cui il relatore ha lavorato; il riferimento è una deduzione.

Un confronto con i libri. L'idea che il suono sia ciò che rende credibile un luogo è anche in *The Art of Game Design* di Jesse Schell, che nell'elenco delle competenze utili a un designer scrive del sound design: "il suono è ciò che convince davvero la mente di trovarsi in un luogo; in altre parole, *sentire è credere*". Parlando di realtà virtuale, Schell nota anche che il suono realistico è molto **contestuale** (lo stesso oggetto suona diversamente in una stanza piccola o in una grande, su un tavolo di legno o di vetro) e che una moneta che cade senza fare rumore basta a ricordare al giocatore che il mondo è finto: è la stessa logica della simulazione per stati descritta dal relatore. (Il confronto non è stato fatto a lezione.)

0.9.8 Il supporto al gameplay

Il **supporto al gameplay** significa che l'audio **fornisce al giocatore elementi utili**, spesso **non visibili**, per migliorare l'esperienza di gioco. (Il relatore si chiede se esista una traduzione di *gameplay*: probabilmente "esperienza di gioco", "giocabilità", con accezioni leggermente diverse.) In un'esperienza interattiva l'audio è fondamentale per **guidare** e **aiutare**, come negli esempi della citazione di Spielberg.

0.9.8.1 I passi della guardia

Il primo video è di un gioco **stealth**: l'audio fa capire che c'è qualcuno, e quindi si gioca in modo diverso. Il giocatore è entrato da una finestra, ha sentito dei passi e si ferma ad ascoltare: **prima sente** i passi, **poi vede** la guardia. In questi giochi si ascolta il più possibile quando i passi si fermano e quando ripartono, per capire il percorso della guardia; dopo averla vista passare due o tre volte, si capisce quando ci si può muovere. È la caratteristica tipica dei giochi stealth.

0.9.8.2 La telecamera di sorveglianza e il microfono sulla camera

Il secondo video è simile, ma riguarda un oggetto: il giocatore sente che c'è una **telecamera di sorveglianza**. E fa una cosa particolare: muove la **camera di gioco**, perché **il microfono è sulla camera di gioco**. Ruotandola, cerca il punto in cui il volume del ronzio della telecamera si alza: "ah, ok, si sta alzando il volume, quindi probabilmente è dietro l'angolo". Il giocatore **sfrutta l'audio** per localizzarla, e poi la neutralizza. (Il relatore chiede se i partecipanti sentano il video in stereo, perché lui non ha impostato la condivisione stereo.)

Sono due esempi "forse un po' più classici": l'audio che fa capire se arriva un nemico, se c'è una telecamera, un laser o qualcos'altro che può danneggiarti.

0.9.8.3 La musica che dice se sei in combattimento

Anche la **musica** può aiutare. Il video successivo è il seguito di uno dei video precedenti (quello di *Assassin's Creed*): la musica fa capire, in modo molto semplice, se si è **in combattimento** oppure no. È importante soprattutto nei giochi **a orde**, in cui compaiono più nemici: si comincia a combattere, c'è "il cattivone lì che parla", la musica va; quando la musica finisce, il giocatore sa che non arriveranno altri nemici, e può andare a uccidere il cattivo senza guardarsi intorno. Infatti nel video non si guarda neanche in giro: "è guidato dalla musica".

Il relatore potrebbe sostituire questo video con uno di ***Solstice***[13], uno dei giochi a cui ha lavorato di recente, dove la situazione è ancora più estrema: in una classica *combat arena* continuano ad arrivare nemici, anche lontani, anche dopo che hai ucciso quelli vicini. Se la musica non si ferma, capisci che sei ancora in combattimento e vai a cercarli.

0.9.8.4 Che cosa sa il motore di gioco?

Su questo video il relatore fa da qualche anno una domanda ai partecipanti. Ferma il video in un momento preciso, durante l'ultimo combattimento: la musica sta andando. *Che cosa sappiamo, in questo preciso*

[13] Così nella trascrizione ("In Solstice"): uno dei giochi a cui lo studio del relatore ha lavorato di recente. Il titolo esatto non è ricostruibile con certezza.

istante? Non dal punto di vista del giocatore, precisa, ma **dal punto di vista del motore**: che cosa sa il gioco, "a livello di statistica, di dati, di informazioni"?

Un partecipante risponde che è il colpo finale. Nicole, che fatica ad accendere il microfono, pensa che, essendo il colpo finale, il gioco dovrebbe "piano piano far finire la musica dinamica". Il relatore insiste: c'è un'informazione che il motore di gioco può già passare al **motore audio**, perché prepari la sua transizione e cambi la musica in modo pulito. Qualcuno ci arriva: **il nemico è già morto**.

È proprio così. Quella che si vede è una **mossa *finishing***, la mossa finale: in quel momento il nemico è già morto, il personaggio sta solo eseguendo l'animazione conclusiva. Il motore di gioco può quindi dire al *sound engine* "morto", e il sound engine, sulla base di come il compositore ha costruito la musica, probabilmente finisce la battuta, va in un piccolo loop di transizione e arriva sul colpo finale. Da quel momento, "anche se non sembra", la musica si sta già portando verso la conclusione. Il relatore conta: "uno, due, tre, quattro, cinque". **Cinque secondi**: "è tantissimo tempo".

La **musica interattiva** si basa tutta su cose di questo tipo: sfruttare il fatto che c'è un'animazione di *finishing*, una piccola cinematica, uno dei tanti momenti in cui il gioco **ha già l'informazione**, per avere il tempo di chiudere la musica con una piccola transizione e finire "perfettamente a tempo".

0.9.8.5 Il giocatore che sfrutta la musica

L'ultimo video è "una piccola chicca", perché il supporto al gameplay qui funziona al contrario: non è la musica strutturata *per aiutare* il giocatore, è il giocatore che **capisce come è strutturata la musica** e la sfrutta.

Il gioco è uno *Splinter Cell*[14]. Il giocatore guarda attraverso la serratura con la fibra ottica e vede un corridoio lunghissimo, con una guardia. La soluzione "normale" sarebbe aspettare che la guardia vada in fondo, scassinare la serratura, aprire la porta piano piano, entrare, mettersi in un punto dove non lo vede: "è un quarto d'ora per fare 'sto corridoio". Invece **apre la porta di proposito** e si nasconde. Parte la musica di tensione: il sistema è in **allerta**. La guardia lo cerca; lui la **immobilizza** e la porta via, fuori dalla zona d'allerta, tornando da dove era venuto. A un certo punto **la musica si ferma**.

Che cosa ha scoperto? Che **non ci sono altre guardie allertate**: quella era l'unica. In questi giochi, normalmente, quando una guardia dà l'allarme ce n'è magari un'altra seduta nel buio, o in un'altra stanza, che dice "ah, cosa succede?", e cominciano a girare in tre. Facendo scattare l'allerta e aspettando che la musica cali, il giocatore ha verificato che quello era l'unico trigger di allerta: ora il corridoio lo può fare "di corsa, saltellando e fischiettando".

Come il trailer di *Gears of War*, è un esempio **estremo**: un giocatore ha scoperto questa caratteristica e la usa. Per quanto ricorda il relatore, non era stata pensata: "non c'era una volontà di dire: con la musica tu puoi giocare diversamente".

Un confronto con i libri. Chris Bateman, in *21st Century Game Design*, analizzando i modi in cui i giochi stealth informano il giocatore, nota che *Thief* e *Splinter Cell* gli mostrano un indicatore di luce sul proprio stato di visibilità, mentre le informazioni sui nemici arrivano "altrimenti da indizi visivi e sonori": è esattamente il ruolo dell'audio nei due video sui passi e sulla telecamera. (Il confronto non è stato fatto a lezione.)

0.9.9 Breve storia dell'audio nei videogiochi

Prima della pausa il relatore propone di fare il punto sui tempi ("vabbè, un po' lungo come sempre") e di cominciare un racconto di **storia dell'audio nei videogiochi**. Due pause da dieci minuti o una sola da venti? I partecipanti scelgono di arrivare fino alle "meno un quarto" e fare una pausa unica di una ventina di minuti, "così se uno vuole mangiare, caffè, relax".

[14] Il relatore non dice il titolo esatto, ma precisa di aver "un po' lavoricchiato alla serie di Splinter Cell".

0.9.9.1 Perché la storia

L'obiettivo del racconto è dare un po' di **contesto**, anche culturale, "che non fa mai male"; ma soprattutto mostrare che molte **tecniche** che si usano oggi sono nate in quell'epoca. Oggi ci sono "super middleware, super strumenti", memoria e processore in abbondanza, "non dico infinita"; eppure le **limitazioni** sono sempre le stesse, e bisogna stare attenti più o meno alle stesse cose. "È un attimo, ancora oggi, arrivare in situazioni in cui spariscono suoni perché abbiamo esaurito la polifonia della scheda audio", per esempio della PlayStation.

0.9.9.2 Tre periodi

Il relatore divide la storia in **tre periodi**:

- le **origini**: gli anni Ottanta;
- il **boom**: gli anni Novanta, "veramente un boom in generale, non solo per l'audio";
- il **presente**, che per lui va dal 2000 in poi. Non sono più dieci anni come gli altri periodi, "sono più del doppio", ma per l'audio non c'è più stato un salto tecnologico, generazionale, tale da giustificare un periodo in più.

I videogiochi, ovviamente, esistevano già negli anni Settanta: *Pong* risale a quel decennio, e "se non sbaglio" anche *Space Invaders*; il relatore ha letto che già alla fine degli anni Sessanta c'era "embrionalmente" qualche gioco sulle grosse workstation "a muro". Ma è negli anni Ottanta che il videogioco diventa una **forma di intrattenimento di massa**.

0.9.10 Gli anni Ottanta: chip, sintesi e linguaggio macchina

0.9.10.1 Nasce la produzione audio

Negli anni Ottanta il videogioco si diffonde: nascono i **publisher**, nascono i **generi** e i sottogeneri (l'RPG, l'adventure, il platform, il racing). Il settore matura e diventa un'**industria**. Ed è in questi anni che nasce la **produzione audio**: prima i suoni erano dei *beep*, che in gergo tecnico si chiamano ***chirp***, emessi da piccoli altoparlanti; ora c'è bisogno di un vero sound.

Due cose, unite, fanno nascere la produzione audio, cioè musica ed effetti sonori.

- La **possibilità**: le console e i personal computer dell'epoca hanno a bordo dei **chip audio**, piccoli sintetizzatori, "processorini audio" con per esempio **quattro voci**. Bisogna immaginare un sintetizzatore in grado di far suonare quattro strumenti alla volta.
- L'**esigenza**: ora c'è l'RPG, e quindi il combattimento, la morte, la vittoria. "Non era più ping, ping, ping, ping".

0.9.10.2 Team piccoli e kit costosissimi

Il relatore negli anni Ottanta non lavorava nel settore (ci è arrivato nel '99, anche se giocava, "avevo il Commodore 64"); il suo racconto viene da un paio di collaboratori e amici a cui ha chiesto. I **team erano piccolissimi**, di massimo due o tre persone, anche perché i **kit di sviluppo** costavano moltissimo: "tipo 20 milioni di lire", una cifra enorme per l'epoca, come se oggi un kit costasse 30.000 euro. "Io sono un team indie, devo sviluppare per PlayStation e devo acquistare il kit a 30.000 euro": non cominci nemmeno.

Il tipico team "indie" degli anni Ottanta era fatto di un programmatore e di una persona che faceva il game designer. Il **programmatore** era "un po' il tuttofare": spesso faceva l'audio, se era anche musicista; a volte la grafica.

0.9.10.3 Registri e forme d'onda

Tutto quello che accadeva veniva programmato in **assembly** o addirittura in **linguaggio macchina**: si dovevano scrivere i **registri** della CPU o dei vari chip. Musiche e suoni erano fatti in **sintesi**. Il chip audio tipico, almeno nella configurazione iniziale (poi "si è un po' espanso"), aveva:

- **due onde quadre**;
- un'**onda custom**, in cui si poteva caricare una forma d'onda a scelta: normalmente una sinusoide o un dente di sega;
- una **quarta voce di rumore bianco**.

Per far suonare qualcosa bisognava scrivere dei valori nel chip: "scrivere e leggere, scrivere e leggere", tutto in linguaggio macchina.

0.9.10.4 Il *voice stealing*

Con un video di un gioco dell'epoca[15] il relatore mostra una tecnica che esiste ancora oggi, "anche se ovviamente un po' mitigata": il ***voice stealing***, letteralmente "rubare una voce".

La musica usa tutte e quattro le voci: il basso, la linea melodica, la batteria. La batteria si faceva con la sinusoide per la cassa e con il **rumore bianco** per il rullante e il charleston. Ma quando il personaggio distrugge un blocco, anche l'effetto sonoro usa il rumore bianco: per suonarlo, il gioco **ruba la voce** alla musica, e il charleston sparisce per un istante, per poi tornare. Verso la fine del video, quando il personaggio continua a distruggere, viene rubata addirittura la **linea melodica**: la musica sotto continua, ma la melodia sparisce. Era "proprio classico"; ma l'orecchio veniva ingannato e il giocatore non se ne accorgeva.

0.9.10.5 Il *voice stealing* oggi

"Ancora oggi si usa". Di recente, su un gioco, al relatore è capitato un suono, il motore di un veicolo, che continuava a sparire. Investigando, ha scoperto che venivano chiamate **troppe voci** contemporaneamente. Oggi, dice, è "ancora peggio", perché "ci sentiamo supereroi": si lanciano suoni ovunque, si crea un evento con dentro cinque suoni "perché così è più realistico", poi se ne lancia uno, due, tre, e si fa presto ad arrivare a **64 voci contemporanee**. A quel punto qualcosa sparisce.

Per questo si usa ancora il *voice stealing*, in modo controllato: si decide, per esempio, che di **passi** se ne vogliono sentire al massimo tre contemporaneamente; se il gioco deve lanciarne cinque, si zittiscono i primi lanciati per far suonare i nuovi. Anche qui, "sempre col discorso della **psicoacustica**", si inganna l'orecchio.

La lezione è generale: la **gestione del numero di istanze** dei suoni arriva dagli anni Ottanta ma è attuale. Non vale il ragionamento "tanto memoria, tanto processore, faccio quello che voglio": ci sono sempre i **vincoli di RAM** (non si possono creare suoni di lunghezza infinita, perché la RAM serve anche ad altri), di **processore** (anche lui condiviso) e di **polifonia**. Negli anni Ottanta le limitazioni erano più strette, oggi meno; ma il lavoro "rispecchia ancora quello che succedeva ai tempi".

0.9.10.6 La pausa

Il relatore vorrebbe chiudere gli anni Ottanta prima della pausa, ma c'è ancora una parte con qualcosa "di concreto che suona". Chiede ai partecipanti se preferiscono vederla dopo la pausa, con calma, o se deve tagliare: qualcuno risponde "io vorrei vederlo". Si fa quindi la pausa di una ventina di minuti, per riprendere "con calma, freschi, senza andare di corsa" e vedere il *tracker*.

0.9.10.7 Il programmatore-compositore

Alla ripresa, il relatore riparte dalla figura "un po' mitologica" degli anni Ottanta: il **programmatore-compositore**. In tantissimi team era il programmatore a comporre la musica. Il motivo non è certo, ma il relatore lo presume: molte persone suonano uno strumento o hanno un po' di formazione musicale, e tra

[15] Il titolo non è detto: è un gioco in cui il protagonista distrugge dei blocchi.

i programmatori era statisticamente probabile trovarne uno che sapesse fare un po' di musica. "Vabbè, io suono la chitarra, allora mi occupo anche della parte audio, visto che è comunque tutta sintesi e codice, lo faccio io".

In Italia il relatore conosce, per esempio, **Ivan Del Duca**, oggi technical director di 505 Games, con una carriera da programmatore (è stato anche in Milestone), "persona di rilievo in ambito tecnico" ma famosa anche perché su alcuni giochi degli anni Ottanta e Novanta, pur essendo programmatore, aveva composto le musiche. E "così ci sono tanti nomi".

0.9.10.8 Rob Hubbard e le macro in linguaggio macchina

Uno di questi è **Rob Hubbard**, compositore famoso dell'epoca ma anche programmatore, perché si programmava lui stesso le **macro** in assembly. Il relatore apre un articolo trovato in rete qualche anno prima, che racconta il suo lavoro[16], e ne mostra tre parti:

- le **macro**, scritte in istruzioni di linguaggio macchina ("LDA", "tutte istruzioni"): la prima si chiama *init music*, poi *music off*, e così via. Una volta scritte, si potevano riutilizzare in più giochi; ma bisognava comunque scriverle, e quindi conoscere il linguaggio macchina;
- i **dati** che le macro andavano a leggere: la **partitura** della canzone, scritta in un formato numerico ("track 1, track 2, track 3");
- i **pattern** e gli **strumenti**, cioè le sonorità: per ottenere una certa voce bisognava scrivere nei registri del chip la forma d'onda desiderata, il timbro di ogni singola voce.

"Era tutto un delirio". A un certo punto qualcuno inventò degli **editor** per scrivere questi dati in modo più semplice.

0.9.10.9 Il *tracker*

Poi arriva il "game changer": **Karsten Obarski**[17], su **Commodore Amiga**, crea **Ultimate Soundtracker**. Non è più un editor testuale dove scrivere byte ("03, 04A") da trasformare poi in partitura, ma un **tracker**: un software con delle **tracce a scorrimento verticale**, un po' come un software di programmazione, in cui si poteva scrivere la musica in modo abbastanza semplice; poi il tracker esportava i dati.

Per farlo vedere, il relatore apre **MilkyTracker**[18], un tracker gratuito che si può scaricare ("avete tra le mani un tracker come quelli che si usavano negli anni Ottanta"), e carica una canzone. Si lavora così:

- la musica è divisa in **pattern** di un certo numero di righe ("tipo 64, 32, 128 righe", a seconda del tipo di pattern), un po' come le battute musicali; si creano tanti pattern;
- in ogni riga si scrive la **nota**, l'**ottava** e lo **strumento**;
- poi ci sono dei **modificatori**: il *pitch bend*, il vibrato, il volume (per esempio un "03").

Mandando in play, la barra scorre e suona le note che incontra ("nulla di speciale"). Quando si salvava la canzone, il tracker salvava la partitura in un formato che poi si inseriva nel programma, e suonava.

Per curiosità, il relatore aggiunge che i tracker sono "i papà dei **sequencer**". Il primo Cubase era "semplicemente un sequencer", e i sequencer erano "i tracker messi in orizzontale", più simili a un foglio di musica: si scrivevano le note, si faceva il *piano roll*, si pilotavano campionatori e sintetizzatori e si faceva musica elettronica. I sequencer si sono poi evoluti nelle **digital audio workstation**, strumenti "per fare musica a 360 gradi". Questo, però, è un capitolo della storia della musica elettronica; per i videogiochi interessa che il tracker sia nato **proprio in ambito gaming**, creato su Amiga per aggirare la complessità di scrivere tutto a mano.

[16] Nella trascrizione l'autore è "Anthony Maxwell". Con ogni probabilità si tratta dell'articolo di Anthony McSweeney che smonta e commenta il codice di un brano di Rob Hubbard per Commodore 64; il riferimento non è verificabile dalla trascrizione.

[17] Nella trascrizione "Karsten Obansky" e "Karsten O'Barsti". *Ultimate Soundtracker* è del 1987.

[18] Nella trascrizione "milchi tracker".

0.9.10.10 Gli editor di suono: un'onda quadra su Game Boy Advance

Il tracker serviva a scrivere la partitura; ma poi bisognava *suonare*, e si usavano le **voci del dispositivo**: del Game Boy, dell'Amiga, e così via. Bisognava scegliere la forma d'onda, e c'erano degli **editor** per modificarne i parametri e provarli in tempo reale.

Quando lavorava a *Rayman* per Game Boy Color, il relatore aveva un **kit di sviluppo** e un'applicazione simile a quella che mostra ora. Prima ne fa vedere uno screenshot, poi la apre dal vivo: lancia **no$gba**[19], un emulatore di Game Boy Advance, e carica una ROM con un editor di suoni ("per esempio la voce 1"). Su Game Boy c'era qualcosa di simile, "ovviamente con una grafica più Nintendo", e lo si controllava con la croce direzionale e il tasto A per il play; nell'emulatore il relatore usa la barra spaziatrice.

Sente la **voce a onda quadra** e ci gioca: "voglio che la *sweep direction* sia all'insù, ma voglio che ci metta 5 step; no, troppo lungo; 4, 3. Ok, questo è il suono che mi piace per quando prendo il bonus". Mentre cambia i parametri, in alto cambiano i numeri del **registro** ("lì c'è scritto *reg sound*"), e il suono cambia un po'.

"Nel lavoro era tutto così". Con queste applicazioni si scrivevano direttamente i registri, si provava, si sceglieva una sonorità per la linea melodica o si creava un effetto sonoro, e poi si salvavano dei **preset**, cioè i valori, da richiamare nel momento in cui serviva quel suono. Era un lavoro molto **tecnico**, che implicava conoscere i registri, saper lavorare con il codice e saper testare: per questo in molti team lo faceva il programmatore. Gli strumenti tipici erano quindi due: un **editor per creare i suoni**, vincolato strettamente alla console (e allora si sentiva direttamente la console che suonava, non un emulatore), e il **tracker**.

Il relatore si offre di passare questi strumenti, tutti gratuiti, a chi voglia "spippolarci" per curiosità o per usarne i suoni.

0.9.11 Gli anni Novanta: campioni, MIDI e CD

Negli anni Novanta arriva "il primo salto tecnologico grosso", quello che nell'arco di un decennio porta allo standard che abbiamo ancora oggi. "Succede tanto".

0.9.11.1 La wavetable

Arrivano le **schede audio con wavetable**. La wavetable è uno spazio di **RAM** in cui si possono **memorizzare dei suoni**, cosa che prima non esisteva: prima era tutto sintesi. All'inizio lo spazio era piccolissimo. Il relatore aveva una Sound Blaster AWE64[20] e la ricorda con "64k, neanche mega": bisognava mettere campioni molto ridotti, magari a 22 kHz e a 8 bit ("mi è capitato di fare l'8 bit"). Poi le memorie sono cresciute, 128, un mega, "poi pian pianino".

Poter memorizzare dei suoni significava poter avere lo sparo, la voce, ma anche i **campioni di strumenti**: un campione di chitarra, uno di batteria, da far suonare con il **MIDI**.

0.9.11.2 Il MIDI e la separazione dei ruoli

Il **MIDI** era già dagli anni Ottanta lo standard della produzione di *computer music*: un linguaggio che permette di far comunicare strumenti diversi (una tastiera Roland e un sintetizzatore Korg, con il loro cavo) e che si era poi esteso al PC, con i sequencer collegati a moduli esterni o virtuali. Negli anni Novanta il gaming **importa questo standard di produzione**.

La conseguenza è enorme. Non si va più dal programmatore a chiedergli di simulare il suono della batteria con le voci del chip: si va da uno **studio esterno** e si chiede un brano. Lo studio lo crea con il suo Cubase e consegna il **MIDI** e i **campioni** ("per le chitarre abbiamo usato questi due o tre campioni, ti do i suoni di batteria"), cioè tutti i sorgenti. L'esempio classico è il **Nintendo 64**: le musiche di *Zelda* e di *Banjo-Kazooie*[21] sono tutte MIDI che suonano dei campioni, con quel suono "un pelo finto" tipico degli anni Novanta, ma molto più reale delle sonorità precedenti. Lo stesso vale per gli effetti, ora digitali.

[19] Nella trascrizione "NoCash", "scritto NoDollaroGBA": è l'emulatore *no$gba*.
[20] Nella trascrizione "Sound Blaster Avdi 64".
[21] Nella trascrizione "Vengeokazu".

Con uno standard di produzione audio importato nel videogioco si può lavorare con uno **studio di produzione**, come faceva già il cinema: si sceglie uno studio perché è bravo, gli si fa fare la musica, registrare gli effetti. I **ruoli si separano**: il programmatore si occupa della parte tecnica, e ci si appoggia a **creativi** veri e propri. Anche negli anni Ottanta c'erano compositori che non programmavano; ma per implementare la loro musica dovevano stare "spalla a spalla" con un programmatore che convertiva tutto, "un bel delirio". Ora il compositore fa il MIDI, lo si porta nel gioco, c'è un player MIDI, "e via".

Nasce così una figura chiamata **Game Audio Engineer**: la persona che sta in mezzo tra i creativi e i programmatori. Parla con lo studio, gli dà le specifiche e la direzione creativa; poi prende i contenuti e si siede con il programmatore: "ok, questo è il MIDI, questi sono i sample, implementiamoli".

0.9.11.3 Il CD della PlayStation

Nel 1995, "proprio a metà del decennio", arriva la **PlayStation**, che fa "un po' il terremoto" perché ha il **CD**, e quindi tanto spazio. Sul CD si possono memorizzare **tracce audio in formato CD**: lanciando il gioco non si ha più il MIDI con i campioni "un po' fintarelli", ma la registrazione stereo dell'Orchestra Sinfonica di Praga; nel menu di un altro gioco, gli Iron Maiden. "È stato un bel salto tecnologico".

0.9.11.4 Un periodo ibrido

Negli anni Novanta si arriva così, man mano, a un audio che **non è più di sintesi**. Ma la sintesi non sparisce del tutto: molte console avevano ancora la parte chip, usata per alcune sonorità, magari i suoni dei menu o qualche effetto specifico. Era una situazione **ibrida**: anche sul Game Boy Advance[22] c'erano le voci di sintesi più una **voce digitale**, e si poteva decidere di suonare le cose più importanti con i suoni digitali.

Il relatore lo fa sentire con un video di un *Final Fantasy*: la **cinematica** ha già un audio digitale, con riverbero e archi; quando si passa **in gioco** si sente che c'è ancora qualcosa di simile alla sintesi, "anche il basso è un po' finto". La direzione, però, era quella del digitale.

0.9.11.5 Dal linguaggio macchina al C e agli script

C'è un'ultima cosa sugli anni Novanta: non c'è più il linguaggio macchina, c'è il **C**. Nascono i primi **script**, e quindi la possibilità di programmare in modo più semplice: anche un sound designer che sapesse un po' programmare poteva scriversi i suoi. In Ubisoft il relatore aveva un editor con "una sorta di JavaScript" e funzioni come *play music*, *play sound*, *stop sound*, con cui si scrivevano piccole routine: se succede questo, lancia questa musica; se succede quest'altro, lancia quest'altra. "Un po' quello che è il C# di Unity oggi, più o meno".

Il processo era però ancora molto **iterativo** e lento: produrre, mettere il materiale nel gioco, aspettare la **build** ("andavi a prendere un caffè, anche due caffè"), tornare, testare, segnare quello che non andava, correggere, reintegrare gli asset corretti, "altro caffè e così via". Con "grandi bruciori di stomaco", dopo dieci caffè al giorno: al relatore è capitato di esagerare con il caffè della macchinetta, e da allora non lo prende più.

0.9.12 Dal 2000 a oggi: il livello del cinema

Il grosso salto avviene intorno al **2000–2005**: tutto l'audio diventa **digitale**. La RAM costa meno, le console escono con più RAM e con **processori dedicati all'audio**; più memoria e più potenza di calcolo significano più qualità. Diventano possibili:

- audio campionato a **44 kHz e 16 bit**;
- l'**audio spaziale**, cioè i suoni 3D;
- gli **effetti in tempo reale**: riverberi, equalizzazione, distorsione, "tutti gli effetti che possiamo pensare" applicati in tempo reale nel gioco;

[22] Nella trascrizione "Pandora Advance".

- l'**audio adattivo**: far entrare tracce, spostarsi all'interno di un brano, **randomizzare**, cioè mandare in play suoni scelti a caso.

Tutto questo ha portato il game audio, pian piano, **al livello del cinema**. Come il videogioco dagli anni Duemila a oggi è diventato graficamente "quasi un film", anche l'audio ha raggiunto quella qualità, nella colonna sonora, nel sound design e nel doppiaggio. Sono cresciuti i budget, e il **team** che lavora all'audio è diventato più grande e complesso.

Un confronto con i libri. Un esempio classico di audio degli anni d'oro dei cabinati è citato da Jesse Schell, in *The Art of Game Design*, a proposito di *Space Invaders*: gli invasori in marcia producevano una specie di **battito cardiaco** che accelerava insieme a loro, con un effetto "molto viscerale" sul giocatore. Il battito cardiaco tornerà, venticinque anni dopo, nel *Manhunt* mostrato dal relatore. (Il confronto non è stato fatto a lezione.)

0.9.13 Il team audio

Il relatore guarda l'orologio ("sono le quattro e mezza") e constata di essere in ritardo di "una mezz'oretta", "ma poco male": la recupererà mostrando un paio di video in meno. Apre la seconda parte, dedicata al **team audio** e ai suoi **ruoli**.

0.9.13.1 Perché conoscere i ruoli

Mettere ordine nei ruoli è importante. All'inizio della sua carriera al relatore non era "chiarissimo": era stato assunto in Ubisoft come sound designer, ma si rendeva conto che, oltre a fare sound design, aveva un ruolo di **integrazione** e di **gestione**. "Di fatto ho scoperto che poi in realtà io facevo l'audio director".

I ruoli che presenta sono **macro-ruoli**. Alcuni sono univoci ("l'audio director è l'audio director"), altri hanno declinazioni più o meno tecniche: accanto al sound designer esiste il **technical sound designer**, un sound designer più tecnico che "mette più le mani nell'engine". Ognuno può specializzarsi in quello che gli piace di più: c'è il sound designer o il compositore che "non ne vuole sapere di aprire una linea di codice" e chi invece dice di sì. Il compositore dello studio del relatore, per esempio, quando compone apre **FMOD**, uno strumento di lavoro, e crea direttamente un **evento FMOD** con le sue tracce: non consegna una traccia audio con una cattura video che mostra come funzionerebbe la musica interattiva, ma **la musica interattiva vera e propria**. Chi non sa o non vuole farlo si concentra sulla parte creativa.

0.9.13.2 Il team dei sogni e il team reale

Una slide mostra i **titoli di coda** dell'audio di *Manhunt*, presi all'inizio dei credits, che dividono il lavoro in quattro aree: **sound, music, dialogue, programming**, per cinque persone in tutto. Il relatore nota un dettaglio: in quel team, che non era un tripla A ma un doppia A, nel sound design c'è **una persona sola**, che probabilmente faceva sia l'audio director sia il sound designer.

Succede spesso. Quello della slide è il "team dei sogni"; nei titoli **indie** c'è molto spesso una persona che compone e una che fa sound design, o addirittura **una persona che fa tutto**. È capitato anche al relatore di dover fare sia le musiche sia il sound design: sono lavori diversi e "secondo me è giusto separarli", ma capisce che in certe situazioni il team è piccolo, il budget è piccolo, e c'è una persona sola. "Dipende dal progetto".

0.9.14 L'audio director

L'**audio director** è una persona di riferimento: è un *director*, come l'art director o il creative director, e quindi **dirige**; è un po' il responsabile del team audio.

Il percorso tipico: l'audio director **viene quasi sempre dal sound design**. È un sound designer che magari diventa technical sound designer e poi si specializza fino a diventare audio director. È più facile che

sia un sound designer che un compositore: il compositore "tendenzialmente è più affezionato" al suo ruolo e ci resta.

I suoi compiti:

- **definisce il design**: decide che il gioco deve avere un certo stile, un certo approccio creativo e anche tecnico; definisce la **visione** dell'audio del gioco;
- lavora con i **game designer**, con i **programmatori** e con tutte le altre figure del team;
- si occupa degli **aspetti tecnici**, il "supporto per l'audio API" della slide, cioè tutta la parte tecnica: quali strumenti di lavoro si usano, e così via. Nei team più grossi, pur sapendo mettere le mani, spesso fa anche parte dell'integrazione;
- **sceglie le persone**: il compositore e magari il sound designer. Molti team lavorano più o meno sempre con gli stessi; in altri, a seconda del progetto, si sceglie un compositore diverso ("mi serve musica orchestrale, e secondo me questo studio di composizione, questo artista è adatto a questo progetto"). "L'audio director è quello che decide un po' tutto".

0.9.15 Il sound designer

Il **sound designer** è la persona che crea **tutti gli effetti sonori**, compresi gli **ambienti**. Le tecniche di lavoro sono quelle che i partecipanti probabilmente conoscono già:

- il **field recording**, la registrazione in esterna;
- il **foley**, la registrazione in studio dei suoni;
- le **librerie audio**, usatissime: dipende sempre dal progetto e dal budget, e molti progetti indie hanno budget talmente piccoli da non potersi permettere di registrare, e lavorano con le librerie.

Due immagini mostrano l'attrezzatura del field recording e del foley, "più una curiosità".

0.9.15.1 Le limitazioni: qualità e lunghezza

Quello che interessa al relatore è un'altra cosa: il sound designer per videogiochi deve **sempre tenere presente le limitazioni**. Non si può sempre avere la massima qualità, suoni a 48 kHz e 24 bit, o addirittura a 96 kHz: spesso bisogna **stringere**. E non si possono avere suoni troppo **lunghi**.

Al relatore capita di chiedere a un sound designer il loop di un macchinario e di riceverne uno di **25 secondi**: "no, attento, questo loop me lo devi fare massimo di 7 secondi". Non è semplice, ma 25 secondi di suono, rispetto a 7, significano allocare molta più RAM e chiedere all'engine uno sforzo molto più grande. E il ragionamento va fatto su tutti i suoni. Per questo si cerca di avere uno **standard**:

- tendenzialmente i loop non superano i **10 secondi**;
- un loop semplice, come il ronzio di un trasformatore elettrico, può durare anche **2 secondi**;
- un macchinario **5–6 secondi**;
- un loop con uno sviluppo più complesso si cerca di farlo stare in **10 secondi**.

C'è anche una ragione tecnica in più: in **FMOD**, racconta il relatore, i suoni sopra i 5 secondi vengono messi in automatico in **streaming**. Lo streaming si può disattivare, ma avere tantissimi suoni in streaming rischia di **saturare il sistema di streaming**. Attenzione quindi alla lunghezza dei loop, parlando di effetti sonori.

0.9.15.2 Le code dei suoni e le campane

Lo stesso vale per le **code** dei suoni. Se serve il colpo di martello di un fabbro, non servono sei secondi di coda della risonanza: "tagliamela, quella coda", tanto nel mix è abbastanza nascosta.

Il relatore racconta un episodio di Ubisoft, su **PlayStation 2**. Il gioco a un certo punto **non partiva più**: la memoria non bastava. Si scoprì che uno stagista, che nel frattempo aveva finito lo stage ed era andato via, aveva preparato i **rintocchi delle campane** di un villaggio: ogni rintocco durava **27 secondi**, perché conteneva tutta la coda della campana. Il relatore lo tagliò a 6 secondi. E non solo: lo stagista aveva fatto lo stesso su tantissimi campioni, molti dei quali avevano anche del **silenzio in coda**. Il relatore passò una giornata a "tagliare, tagliare, tagliare", recuperò "tipo 50–60 mega", e il gioco partì. "C'è proprio la differenza" tra il gioco che non va, con "la Francia" che ti chiama perché sei il responsabile, e il gioco che va: tagliare le code dei suoni.

0.9.15.3 Suoni asciutti e riverbero in tempo reale

Una partecipante[23] fa una domanda: per evitare che, tagliando le code, si tagli anche l'ambiente, ha senso inserire una **reverb zone** in tutti gli ambienti che suonano, per alleggerire il carico?

"Sì, assolutamente", risponde il relatore: è un'osservazione corretta, e porta a un'altra regola. Tutti i suoni di un videogioco si fanno tendenzialmente **asciutti**. Non serve fare il passo riverberato, perché il riverbero lo applica il motore **in tempo reale**: quando sei in una chiesa applica il riverbero della chiesa, quando sei in una caverna quello della caverna. Quindi suoni asciutti e senza troppa coda.

0.9.15.4 Zero silenzio in testa

Un'altra regola importante: **zero silenzio in testa**. Un suono di passo deve suonare nel momento in cui il piede appoggia: in quel momento il motore dice *play*. Se il file ha del silenzio all'inizio, il suono arriva in ritardo rispetto all'immagine. Tutti i suoni devono quindi cominciare **sul transiente**, "appena prima, proprio sulla parte iniziale del transiente", il più possibile.

Se poi si vuole che qualche suono parta leggermente dopo, gli strumenti come FMOD o Wwise[24] permettono di impostare un **ritardo casuale**: anche se il play arriva al tempo zero, non tutti i suoni partono a zero. "A livello tecnico abbiamo tutta la possibilità del mondo", ma il ritardo si aggiunge nel motore, non nel file.

0.9.15.5 Lineare e interattivo

Il sound designer che lavora per il gaming, conclude il relatore, deve tenere presenti molte cose che nel **lineare** non cura. Nel lineare si mette tutto su una timeline, si sincronizza, si sposta, se un suono è lunghissimo lo si accorcia, e alla fine si fa un bell'export: una traccia stereo, mixata, renderizzata. Nel gioco, invece, **tutti gli asset sono separati**: vengono caricati in memoria e suonano insieme, e bisogna tener conto di tutto.

0.9.16 Il compositore

Il **compositore** si occupa della musica: compone, scrive, dirige, con tutte le competenze classiche. Il relatore si concentra sulle **peculiarità** del lavoro per i videogiochi, che richiede un processo creativo specifico.

0.9.16.1 La musica è al 90% loop

La prima peculiarità il relatore non l'ha nemmeno scritta, perché la dà per scontata: nei videogiochi **la musica è al 90% loop**. Parte e gira su se stessa. E anche qui il brano **non è lunghissimo**: spesso un minuto, un minuto e mezzo, fino a due minuti; difficilmente tre, perché è "pesante", anche banalmente come spazio occupato.

[23] Il relatore le risponde "brava"; la trascrizione non dice il nome.
[24] Nella trascrizione sempre "WISE".

Ne nasce un difficile **bilanciamento**: una musica in loop, di durata non eccessiva, deve **variare** abbastanza da non far capire che sta girando, ma restare abbastanza **omogenea** da non disturbare. Il relatore, che da tempo non compone più (si è spostato sul sound design e "in quest'ultima parte della carriera" fa soprattutto l'audio director e l'integrazione), ricorda le difficoltà che aveva a creare un loop "abbastanza vario, abbastanza omogeneo allo stesso tempo, per non annoiare".

Oggi ci sono delle tecniche per aiutarsi. Si possono creare dei **micro loop** che a ogni giro vengono **selezionati in modo casuale**. Per esempio, se una linea di basso deve coprire una musica di un minuto e mezzo, si scrivono alcune linee diverse da 30 secondi e ogni 30 secondi se ne sceglie una a caso: la musica continua a variare, e con poca musica si copre più tempo.

0.9.16.2 La musica è interattiva

La seconda peculiarità: la musica è **interattiva**. Spesso non basta comporre un brano che va in loop: bisogna comporre un brano che **si adatti a diverse situazioni**. È come prendere un brano e arrangiarlo per situazioni diverse, potendo **passare dall'una all'altra**: bisogna quindi creare anche le **transizioni**. "È un lavoro complesso".

0.9.16.3 Molti stili, una sola persona

La terza peculiarità è "un po' trascurata": la **varietà di stili**. Quando un film ha bisogno di musiche molto diverse, di solito sono brani di musicisti diversi, o tracce famose di gruppi e artisti diversi: nei titoli di coda si leggono le tracce del gruppo A, del gruppo B, dell'artista C. Nei videogiochi succede solo in alcune serie che usano *playlist* di artisti diversi, come *Grand Theft Auto* o *FIFA*. Normalmente, invece, **si viene contrattualizzati per creare la musica di un intero gioco**, che magari ha ambientazioni diversissime. Un gioco d'avventura può avere una parte nella foresta amazzonica, poi le piramidi, poi i ghiacciai: e bisogna comporre in **stili diversi**, in tempi abbastanza brevi ("non hai tre anni o due anni di produzione"), magari anche solo riarrangiando lo stesso brano in stili diversi. Bisogna essere bravi, conoscere più stili, essere **duttili e flessibili**. "Questa è una difficoltà che non è banale".

0.9.16.4 Il compositore di *Rayman M*

Il relatore racconta un aneddoto. ***Rayman M***, a cui ha lavorato, fu sviluppato metà a Milano e metà a Parigi, e il compositore era francese, "molto bravo" e abbastanza conosciuto in Francia (il relatore non ne ricorda il nome). Durante una visita, il compositore raccontò che quando doveva comporre la musica per un livello si **immergeva totalmente** in quel genere: lo ascoltava in qualsiasi situazione, in macchina, la sera. E scherzosamente aggiunse che, per la mappa *Cemetery*, aveva abbassato le tapparelle e acceso candele a forma di zucca, trasformando la casa in una specie di Halloween per entrare nel *mood*; e che per la musica piratesca si metteva il cappello da pirata.

Il relatore non sa quanto scherzasse; ma l'aneddoto fa capire una pratica reale. Sa che lo fanno diversi compositori, anche per il cinema: ricorda un'intervista (non sa più di chi) in cui un compositore raccontava di masterizzarsi dei CD con il genere su cui stava lavorando e di tenerli in macchina e ovunque, anche solo per capire "quali potevano essere gli schemi, i pattern".

0.9.17 Il doppiaggio e il dialogue director

Nel **doppiaggio** c'è una figura, presente soprattutto negli studi più grossi o negli studi esterni specializzati: il **dialogue director**, la persona che **supervisiona** e gestisce tutto il doppiaggio. Il relatore fa questo lavoro quando il suo studio si occupa di doppiaggi, e lo descrive così:

- si occupa degli **script** e dei testi: non li scrive, ma li prende e li **organizza**, per esempio in un bel file Excel, separati per lingua;
- crea il **brief per i doppiatori**: per ogni personaggio, una descrizione, l'età, lo stile di voce, e poi i testi per ogni lingua;

- riceve i **contenuti** registrati e li organizza.

È un lavoro complesso che si appoggia a un team: il doppiaggio è "quasi un dipartimento a sé". Ci sono gli autori dei testi, i doppiatori, il direttore del doppiaggio, i fonici, chi fa l'**editing** (pulizia e taglio dei file). I file doppiati, puliti e tagliati vengono poi **post-prodotti**, cioè livellati, e consegnati al dialogue director, che li **implementa** nell'engine.

0.9.17.1 Una domanda sull'intelligenza artificiale

A questo punto un partecipante[25] chiede la sua opinione sull'**intelligenza artificiale** nel doppiaggio. "Adesso, nel 2026", ha avuto modo di sperimentare software di doppiaggio e anche di musica, e vede un "buon rischio" di sostituzione, soprattutto per i giochi indie a budget praticamente zero.

Il relatore conferma che **il rischio c'è**. L'anno prima gli sono stati chiesti dei test di doppiaggio con l'IA su un progetto: il risultato è "buono, molto buono", ma "sull'aspetto recitativo manca ancora qualcosa". Il partecipante è d'accordo. Per un team a budget zero, che deve scegliere tra non avere doppiaggio, avere doppiatori non professionisti e un risultato che non funziona, o una voce sintetica non perfetta, l'IA può avere senso.

0.9.17.2 Il contratto e la frase "troppo moscia"

C'è poi un punto delicato dal lato dei **doppiatori**. Molti dei doppiatori con cui il relatore lavora vogliono che nel **contratto** sia scritto che la loro voce non potrà essere usata per addestrare sistemi[26] né modificata. Uno in particolare si è "proprio acceso" quando un cliente chiedeva il contrario: "pensare che poi loro possono avere la mia voce per fare tutto non è giusto".

Il relatore invita a pensarci. Se, finito il doppiaggio, ci si accorge che una frase è venuta male, che manca una "frasettina corta", o che il creative director vuole un'intonazione diversa ("questa frase è troppo moscia"), e il doppiatore è impegnato o difficile da raggiungere, perché non generarla? Il relatore lo ha vissuto: aveva tutto il doppiaggio di una doppiatrice e gli serviva una sola frase detta in modo diverso. "Ho i contenuti, cioè io ho tutto il tuo doppiaggio, lo prendo, lo butto dentro ElevenLabs, la clono, ti firmo che non lo userò, però prendo quella singola frase, gliela faccio generare 50 volte" e scelgo quella che mi piace. "Questo non è una cosa così brutta".

Il partecipante è d'accordo, con una precisazione: per lui "l'idea deve partire dall'uomo", ma l'IA ha dei lati positivi poco visti "per una paura di utilizzare troppo l'intelligenza artificiale". Se fosse doppiatore, dice, **clonerebbe la propria voce** e offrirebbe **due tariffe**: la voce originale, con qualità al 100%, registrata di persona; oppure la voce clonata, a un costo molto più basso, "ma almeno così ottieni del lavoro".

Il relatore aggiunge, "detto tra noi, lo dico e non lo dico", che una frase breve generata dal doppiaggio esistente probabilmente non la nota nessuno, forse nemmeno il doppiatore. Lo paragona a quando, in un brano musicale da tagliare in un punto, si ricrea con strumenti virtuali una coda finale che non c'era: si aggiunge qualcosa che non viene dal compositore, ma "un ritocco secondo me non è sicuramente un uso" improprio. Dipende da chi si ha dall'altra parte: chi dice "la mia arte non deve essere toccata dall'intelligenza artificiale" rischia, per il partecipante, di diventare "quasi troppo vincolante".

Il relatore però **capisce la paura** dei doppiatori: che una fetta di clienti, tutti gli indie (anche del cinema), cominci a usare voci clonate, e che il doppiatore si ritrovi con metà del lavoro. O ti adatti, cloni la tua voce e la metti in vendita, o non hai lavoro: "è vedere il lavoro in una maniera diversa". E in effetti, racconta, su **ElevenLabs**[27], "il sistema più diffuso", ha riconosciuto le voci di un paio di doppiatori italiani che aveva inserito nel suo casting, con lo stesso nome: hanno fatto proprio quello che suggeriva il partecipante. "Per un doppiatore che resta purista e nega questa cosa, ce ne sono cento che danno la voce da clonare": ci si deve adattare, a meno di essere "lo Hans Zimmer dei doppiatori".

[25] La trascrizione non dice chi parla.
[26] Nella trascrizione "per fare trading": quasi certamente "per fare training".
[27] Nella trascrizione "Eleven Labs", "Level Labs" e, più avanti, "level up".

0.9.18 L'audio programmer

L'ultimo ruolo è l'**audio programmer**, con compiti tutti tecnici (il relatore va "un pelo veloce"):

- fornisce gli **elementi e le limitazioni tecniche**: dice ai sound designer che cosa si può fare su una specifica piattaforma;
- realizza il **sound engine**, oppure implementa un **middleware** esterno;
- crea dei **tool**; in generale si occupa di tutta la parte tecnica.

È un programmatore, ma deve avere **competenze audio**. E deve ricordare alcune cose.

- **Comunica con un team audio** che spesso non esprime le esigenze in termini tecnici. Al relatore è capitato di avere a che fare con programmatori "un po' rigidi" che non capivano le sue richieste perché non usava termini tecnici. L'audio programmer deve conoscere lui stesso i termini audio.
- Deve saper fare dei **test in autonomia**. L'audio programmer di Ubisoft, che lavorava a un sistema per mandare in play dei loop con regole particolari, si creava da solo i suoi loop di prova.
- Ogni **soluzione tecnica** o limitazione ha **conseguenze sulla produzione audio**. Per esempio, se si deve cambiare traccia in una musica interattiva, spesso bisogna aspettare la **fine della battuta**: non la si può troncare.
- Le **specifiche di consegna** vanno date in modo chiaro e **da subito**. Altrimenti il compositore consegna le musiche nel formato sbagliato, poi passa a un altro progetto, non lo si trova più, e "ti devi tenere quello che ti è arrivato".

0.9.18.1 Gli *stem* che non c'erano

Il relatore racconta un caso. Un team lavorava con un compositore abbastanza conosciuto, con un contratto "a minuto di musica": tanti soldi per tanti minuti. Il compositore consegnò le tracce. Poi subentrò lo studio del relatore, per l'integrazione in FMOD, e a un certo punto il team chiese: "vogliamo che quando dall'*exploration* parte il *combat*, la musica cambi". "Ok", rispose lo studio, "se ci date gli **stem**, riusciamo a farla cambiare". Il team, che non era di sound designer, chiese che cosa fossero gli stem[28]; andarono dal compositore, che consegnò semplicemente degli **arrangiamenti diversi** della stessa musica. Gli stem non arrivarono mai, e lo studio dovette arrangiarsi: il compositore dello studio creò degli stem e si "simulò" la transizione. Il problema era a monte: nessuno aveva detto chiaramente al compositore che doveva consegnare gli stem. Per questo è "importantissimo" che chi gestisce la parte tecnica dia le **specifiche corrette**.

0.9.19 Metà del lavoro è l'implementazione: Alexander Brandon

"Cambiamo argomento leggermente" con la seconda e ultima citazione, di **Alexander Brandon**, audio director "abbastanza storico, ancora più storico di me": quando il relatore cominciava, Brandon lavorava già da un po'. Il senso: **la produzione**, cioè le musiche, gli effetti sonori, il doppiaggio, i contenuti, **è solo metà del lavoro**; l'altra metà è **come li implementi**.

Si possono avere contenuti prodotti benissimo, doppiaggi perfetti, il miglior doppiatore, il miglior compositore; ma se poi si **implementa male** e si **mixa male** in tempo reale nel gioco, l'audio è brutto, nonostante la qualità dei contenuti. E questo dipende da **come si lavora** durante lo sviluppo: per questo è importante capire come l'audio interviene nelle varie fasi di un videogioco.

[28] Gli *stem* sono le tracce separate di un brano (per esempio percussioni, archi, basso), esportate in modo da poterle far suonare insieme o separatamente.

0.9.20 L'audio nel ciclo di produzione

Il relatore racconta il ciclo di produzione in tre slide, una per fase: **pre-produzione**, "una fase molto importante per il gaming", **produzione** e **post-produzione**. In ogni slide, in alto c'è il **team di sviluppo** (con l'icona del pad), in basso il **team audio** (con l'icona dell'altoparlante). La figura 0.9.1 riassume le tre slide.

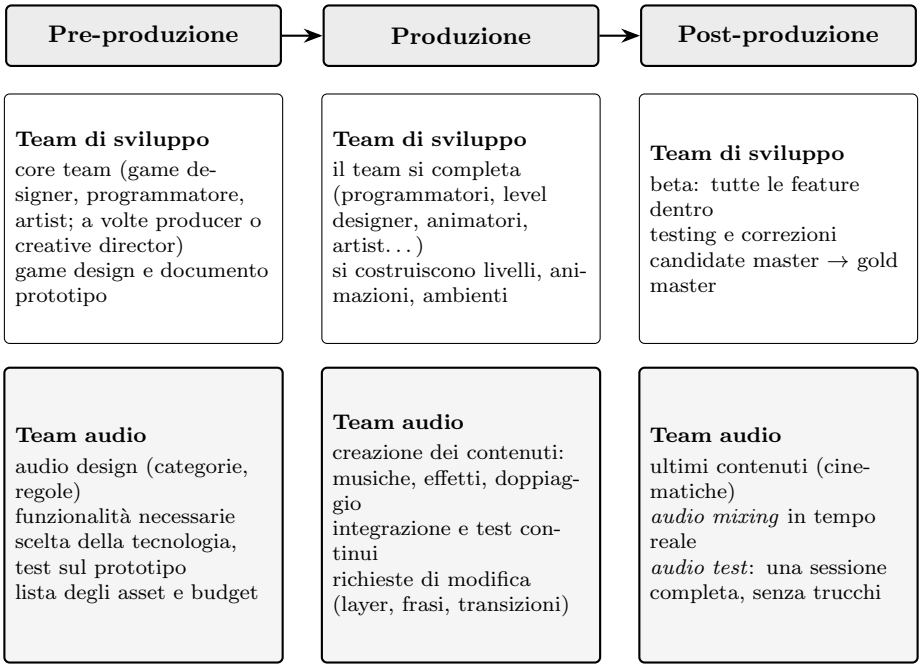


Figure 0.9.1: Le tre fasi del ciclo di produzione, con il lavoro del team di sviluppo (in alto) e del team audio (in basso). Ricostruzione schematica delle tre slide del relatore.

0.9.20.1 La pre-produzione: il team di sviluppo

In pre-produzione il team di sviluppo si forma: nasce un **core team**, tipicamente un **game designer**, un **programmatore** e un **artist**, a volte un producer o un creative director. Sono tre o quattro persone che poi diventeranno i *lead* (il programmatore diventerà lead programmer quando arriveranno altri programmatori). Il core team crea il **game design**: si definiscono le regole, si scrive il documento di game design, e da lì si costruisce un **prototipo**, per capire se il gioco "è bello, funziona, può vendere".

Lo si fa sempre, nelle grandi multinazionali come negli indie. Le multinazionali come Ubisoft[29] chiedono spesso alle varie filiali dei prototipi (anche al team del relatore ne furono chiesti un paio): magari ne fanno fare 6, 8, 10, con una pre-produzione di sei mesi; ne scartano sei, agli altri quattro danno altri sei mesi; poi ne scartano tre "e tirano fuori l'*Assassin's Creed*", che ripaga tutti i soldi spesi negli altri prototipi. Per i **team indie** vale la stessa logica, ma il prototipo, invece di farlo valutare dalla sede centrale, lo si fa valutare **dal mercato**: si va agli eventi, si cerca un publisher a cui piaccia e che metta i soldi. L'obiettivo è sempre quello: capire con un prototipo se partire davvero.

0.9.20.2 La pre-produzione: l'audio

Sotto, nella parte audio, la frase che il relatore sottolinea è che **l'audio deve esserci già in pre-produzione**. "Credetemi, non è scontata". Gli è capitato proprio al First Playable dei giorni precedenti: chiede un meeting a un team per capire se si può collaborare, e la risposta è "ma noi siamo ancora in una fase iniziale,

[29] La trascrizione cita anche "Tony Karts", un nome non ricostruibile con certezza.

non stiamo ancora pensando all'audio". "Male, male": è proprio all'inizio che bisogna pensare all'audio e coinvolgere una figura dedicata. In pre-produzione si fanno quattro cose.

1. **L'audio design.** Se si sta facendo il game design, si possono già definire le regole dell'audio. Per esempio: ci sono 20 personaggi diversi; li si **categorizza** (grossi, medi, piccoli), e a livello sonoro si decidono **tre tipologie di passo**: pesante, medio, leggero. Su quali **materiali**? Dov'è ambientato il gioco, quali superfici ci sono: neve, erba, ghiaia? Le **armi**: che sonorità hanno? Se queste regole non si definiscono subito, in produzione non c'è il tempo di farlo.

2. **Le funzionalità.** Fatto l'audio design, si decide che cosa serve: musica interattiva? Un effetto specifico di riverbero in tempo reale? Il multistreaming? Bisogna sapere che cosa serve.

3. **La tecnologia.** Da lì si decide l'implementazione: usare il motore audio di **Unreal Engine** oppure no, scegliere **Wwise** "perché è uno strumento più completo e mi permette di fare determinate cose". Sono scelte da fare ora, e da **provare sul prototipo**: si può chiedere al programmatore due cartelle diverse, una con Wwise e una con FMOD, per fare dei test, dei *benchmark*, e capire bene quale tecnologia si vuole.

4. **La lista degli asset.** Alla fine della pre-produzione si comincia a definire "la lista della spesa": 50 passi su 10 superfici diverse, lo *swoosh* della spada, dell'ascia, il colpo della mazza, cinque musiche, la musica del menu, quella dei credits. Serve per il **budget**: la pre-produzione può durare un mese o tre, mentre il gioco magari due anni, e alla fine bisogna andare dal publisher a dire "ho bisogno di 800 mila euro". Il publisher vorrà sapere quante musiche servono, se c'è del doppiaggio: sono costi.

Un confronto con i libri. Lo stesso allarme si trova in *The Art of Game Design* di Jesse Schell, nel paragrafo *Use Audio*: uno degli errori più gravi degli sviluppatori è aggiungere musica e suoni solo alla fine. Schell suggerisce anzi di **scegliere la musica all'inizio**, "forse prima ancora di sapere che gioco sarà", perché un brano che dà la sensazione giusta prende già molte decisioni sull'atmosfera; e se una parte del gioco contrasta con quella musica, è un buon segnale che sia quella parte a dover cambiare. Schell cita anche uno studio in cui due gruppi di giocatori valutavano *solo la grafica* dello stesso gioco: il gruppo con l'audio di qualità più alta giudicò migliore anche la grafica. (Il confronto non è stato fatto a lezione.)

0.9.20.3 La produzione: contenuti e integrazione

Se il prototipo va a buon fine, si passa in **produzione**. Sopra succede qualcosa di prevedibile: il team si completa con programmatori, designer, level designer, texture artist, animatori, environment artist ("bla bla bla bla"), e man mano si costruiscono livelli, animazioni, ambienti, "si compone tutto il gioco".

Se la pre-produzione è importantissima, la produzione è la fase "più concreta", in cui si mette in atto tutto. Il compositore crea le musiche, il sound designer gli effetti, si realizza il doppiaggio: si creano i **contenuti**. Ma soprattutto **i contenuti si integrano** nel gioco: si mettono dentro, si provano, e si scopre che qualcosa non funziona. Serve un pezzo di musica in più per collegare due ambienti, una zona di transizione a cui non si era pensato, e si torna dal compositore a chiedere una modifica. Un suono fatto in un certo modo non funziona, va spezzato in elementi separati per mixarli in tempo reale, e si torna dal sound designer.

0.9.20.4 L'armatura in tre layer

L'esempio pratico del relatore è "un caso reale" di un progetto. Il sound designer aveva consegnato i suoni del movimento di un'**armatura**. Ma in gioco si scopre che il personaggio si muove in modi diversi: piano, a velocità media, bruscamente. Si torna allora dal sound designer e gli si chiede di consegnare lo stesso suono in **tre layer**:

- il **metallo** che si muove;
- le **catene**, "tutto ciò che è catenella";
- la **pelle**, "tutto ciò che è tessuto".

Suonati insieme danno lo stesso identico suono; ma nel motore di gioco sono tre livelli diversi. Se il personaggio si muove piano si sente soprattutto la pelle e un po' di catenella; a velocità media pelle, catenella e un po' di metallo; se si muove bruscamente, soprattutto il metallo. Questo lo si scopre **solo in produzione**: "produci, integri e dici: caspita, suona finta questa roba!". Allora si separa, suona meglio, e magari si scopre che manca un quarto layer a cui non si era pensato. "È tutto questo processo dove si fa la differenza".

Lo stesso vale per i **dialoghi**: si implementano e si scopre che mancano delle frasi, o che vanno mixati meglio, o che serve un **sidechain**, cioè abbassare un po' la musica quando parte un dialogo perché altrimenti si sente male. "Tutte queste cose le scopri mentre integri". Quello che si è deciso in pre-produzione aiuta, perché dà gli strumenti per cambiare direzione; ma le scelte finali su come si producono e integrano le cose si fanno qui. Si crea, si compone, si integra, si testa, fino alla chiusura della produzione.

0.9.20.5 La post-produzione: dalla beta alla gold master

Chiusa la produzione si arriva alla **beta**. Le scadenze del videogioco sono quelle dello sviluppo software:

- l'**alpha**, una versione intermedia;
- la **beta**, che segna la fine della produzione: il gioco con **tutte le feature implementate**. Possono anche funzionare male: c'è una "regola non scritta" per cui a volte la beta deve avere tutto dentro e funzionante, a volte si sa consapevolmente che qualcosa è ancora vago, "però deve essere tutto dentro";
- poi un periodo di **testing**, di uno o due mesi, in cui si sistema tutto;
- la **candidate master**, se si è su console: la versione *candidata* a essere quella che esce. Si chiama così perché la si consegna a Sony, Nintendo o Microsoft, che la provano e possono respingerla perché qualcosa non è conforme alle loro specifiche. A volte sono cose gravi, come un **tempo di caricamento** troppo lungo (su PlayStation, ricorda il relatore, non può superare "i 6 secondi, i 10 secondi"); a volte "cavolate", come aver chiamato il controller "Gamepad". Si corregge e si consegna una nuova candidate master;
- la **gold master**, quando la piattaforma dà l'ok: la versione che si può rilasciare.

In questa fase **l'audio non smette di lavorare**, e fa due cose.

L'audio mixing. Anche l'audio, in beta, dovrebbe avere tutto dentro. "Non è mai così". Il relatore ha "sempre questa lotta" con il producer: le **cinematiche**, per esempio, sono pronte due giorni prima della beta ("e noi come facciamo a mettere dentro i suoni? Avete 20 cinematiche finite"). È "un gioco delle parti": spesso si concede di chiudere l'audio qualche giorno dopo la beta; e il team audio cerca comunque di avere tutto dentro, magari con suoni *placeholder*, in modo che la cinematica "suoni qualcosa", per poi rifinire.

Poi si deve **mixare**, perché in produzione si è messo dentro tutto senza bilanciarlo. Oggi il mixaggio si fa **in tempo reale**: si collega lo strumento (FMOD o Wwise) al gioco, si gioca su un monitor tenendo aperta la sessione sull'altro, e si regolano i volumi *mentre si gioca*. Può durare due giorni, una settimana o di più, a seconda della grandezza del gioco, e coinvolgere più persone, ognuna su parti o aspetti diversi.

L'audio test. L'ultima cosa è il **test audio**. Anche se c'è un team di test esterno, spesso i tester **non trovano i problemi audio**: se non si chiede espressamente un giorno di test audio, capita che giochino con la loro musica in cuffia, perché "poveretti" devono giocare otto ore al giorno, sbattere contro i muri per cercare i bug, fare sempre lo stesso salto, e non vogliono sentirlo quaranta volte. Si possono chiedere test audio dedicati, ma essendo molto specifici spesso il team audio **se li fa da sé**.

Si prende il gioco e ci si mette in un ambiente tranquillo: in sede magari dalle sei di sera, quando la gente comincia ad andare via, fino alle otto; in remoto, "meglio ancora". E si gioca **senza cheat**, senza codici per saltare da una parte all'altra (quelli si usano quando si mixa): si **simula una sessione reale**. Si entra nel menu, si girano tutte le voci, si provano tutti i tasti, si cambiano le opzioni; si comincia, si guarda la cinematica iniziale senza saltarla, e si fa un *walkthrough* il più completo possibile, verificando che tutto l'audio funzioni.

"È tutto chiaro? Non siete ancora abbastanza stanchi da non capire?".

0.9.21 Le quattro categorie di suoni

L'ultimo argomento sono le **categorie di suoni** in un videogioco. Sono fondamentalmente **quattro**, ciascuna con una funzione principale (tabella 0.9.1).

Categoria	Funzione principale	Chi la produce
Musica	emozionale	compositore
Effetti sonori (SFX)	descrittiva	sound designer
Ambienti	descrittiva	sound designer
Voce (dialoghi)	narrativa	doppiaggio

Table 0.9.1: Le quattro categorie di suoni di un videogioco e la loro funzione principale, secondo la slide del relatore.

Perché quattro e non tre? Perché **effetti sonori** e **ambienti**, pur essendo entrambi fatti dal sound designer, sono categorie diverse. Il suono di uno sparo e quello di un gabbiano sembrano la stessa cosa (prendi il microfono e registri), ma sono diversi **sia per come vengono prodotti sia per come vengono implementati**.

Visto il tempo, il relatore mostra un solo video di questa sezione, "l'unico che ha senso vedere a quest'ora".

0.9.21.1 La musica: source, lineare, non lineare

La musica ha **tre tipologie**.

Source music. È la musica prodotta da **sorgenti nel gioco**: la si è già vista nel video di *Assassin's Creed II*, con l'orchestrina a Venezia. C'è qualcosa nel gioco che emette musica: è musica **diegetica**. Va composta e prodotta, magari ha un loop, ma viene **integrata e trattata come un effetto sonoro**: nel video, quando Ezio saliva in alto, il suo volume si abbassava. Non è una *soundtrack* che si sente ovunque si vada nel gioco: ha uno scopo diverso e si integra in modo diverso.

Musica lineare. "Guarda un po', nel videogioco, che è interattivo, c'è anche la musica lineare". È quella dei **menu**, dei **credits** e delle **cutscene**. Nel menu si può comporre una **canzone**: non deve essere interattiva né per forza un loop, può avere un'introduzione e una fine e poi ricominciare (spesso nei menu c'è una musica che finisce e riparte). Nelle cutscene si hanno 40 secondi, o 10 secondi, di scena, e si compone come se si sonorizzasse un piccolo cortometraggio.

Musica non lineare. È quella che "la fa da padrona": composta in modo da **adattarsi a quello che succede nel gioco**, cambiando nel modo più fluido possibile. Si chiama anche **musica adattiva** o **interattiva**.

0.9.21.2 Action e mood

La musica non lineare reagisce tendenzialmente a **due cose**.

- A un'**azione**: è l'*action music*. Parte il combattimento, parte la musica; finisce il combattimento, la musica si ferma. È il caso già visto nel video della mossa *finishing*, "quindi non serve guardare nessun video".
- A un **mood**, cioè a uno **stato del gioco**: è la musica *mood driven*, che reagisce alla vita del giocatore, alla tensione, alla vicinanza all'obiettivo. Sono valori dinamici, continui oppure a gradini (0, 1, 2, 3), e la musica reagisce.

0.9.21.3 Il mod di *Far Cry*: la colonna sonora che si scrive giocando

Il video è "un bellissimo esempio di musica non lineare basata sul mood". Il relatore lo ha trovato qualche anno prima: è il lavoro di una persona che probabilmente voleva usarlo come *showcase* (non sa se fosse

un technical sound designer, un audio programmer o un compositore). È un **mod di *Far Cry***: il gioco permette di creare mod, e quindi di accedere ad alcuni parametri di codice. Prima di avviarlo, il relatore lo spiega.

L'autore ha raccolto vari **parametri di gioco**: se il personaggio è nell'erba e quindi coperto, la distanza dei nemici, se i nemici guardano nella sua direzione, probabilmente quanto rumore fa. Li ha condensati in un unico **cursore bianco**, che va da un minimo a un massimo: la **tensione**, lo stato di minaccia. Se il giocatore corresse come un pazzo verso cinque nemici che si girano verso di lui, il cursore schizzerebbe al massimo.

Poi ha fissato delle **soglie**: sopra il livello 2 lo stato diventa *suspense*, poi suspense crescente, e sopra il livello 5 *minaccia* (*threat*). E ha composto delle **clip** per ogni stato. Quando la timeline arriva a metà di una clip, il sistema **sceglie la clip successiva** in base a dove si trova il cursore in quel momento (nel video la clip scelta lampeggia).

Nel video si vede il giocatore uscire allo scoperto all'inizio, con i nemici ancora lontani: il cursore sale nel giallo e viene scelta una clip gialla, di suspense. Poi si sposta a sinistra dietro un albero e si copre "appena in tempo" prima della scelta della clip successiva (altrimenti avrebbe dovuto aspettare un altro giro): il cursore scende e si torna al verde. Poi si avvicina ai nemici, sempre di più, la tensione sale, finché lo vedono e si va alla tensione massima. Muovendosi, **il giocatore si crea la colonna sonora**: un'altra partita darebbe una soundtrack diversa, fatta degli stessi "mattoncini" concatenati in modo diverso.

Il compositore, ipotizza il relatore, avrà composto un certo numero di mattoncini per lo stato neutro, per la suspense e per la minaccia, più delle **zone di transizione**: dal verde si passa a una transizione, che può risolvere tornando al verde o salire allo stato superiore. "Non è banale". La prima fase creativa è abbastanza standard: si pensa e si compone il brano, probabilmente in modo lineare; poi bisogna cominciare a **tagliare** e a pensare per **blocchi**.

0.9.21.4 Interattività orizzontale e verticale

Questo è un esempio di **interattività orizzontale**: si suona un loop, si passa a un altro, ci si muove in orizzontale nella sequenza dei blocchi. L'**interattività verticale** è invece quella in cui si ha una traccia e, sotto, altre tracce che suonano mute: quando serve se ne **alza il volume**, "come se tu alzassi proprio i volumi di alcune tracce". È il caso del layer ritmico che entrava in *fade in* nel video di *Assassin's Creed* (sezione 0.9.6). La figura 0.9.2 schematizza le due tecniche.

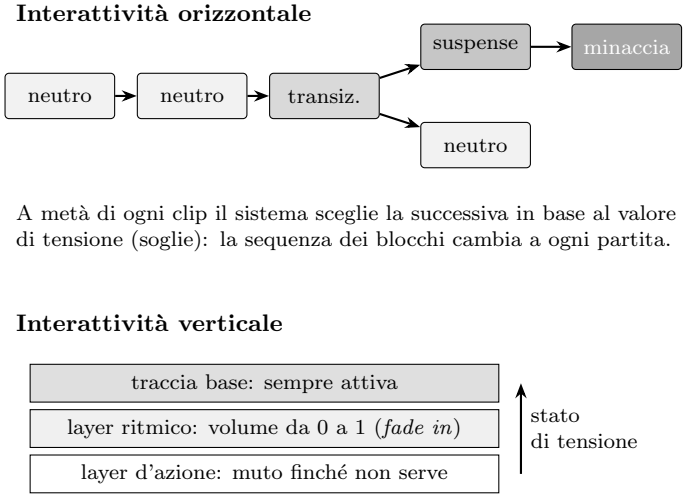


Figure 0.9.2: Le due forme di interattività della musica non lineare. In alto, quella orizzontale del mod di *Far Cry*; in basso, quella verticale del layer che entra in *fade in*. Ricostruzione schematica, non tratta dalle slide.

0.9.21.5 Gli effetti sonori

Gli **effetti sonori** sono i suoni di **tutto ciò che succede** nel gioco. Sono associati a quattro tipi di cose, e la slide dice anche come vengono implementati (i metodi di produzione, invece, si sono già visti):

- le **azioni dei personaggi**: si implementano spesso sulle **animazioni**, agganciando il suono a un frame dell'animazione;
- gli **oggetti di gioco**: hanno un codice, degli **stati**, degli script, e il suono si aggancia lì (oggetto distrutto, acceso, spento, che esplode);
- gli **eventi di gioco**: spesso si lanciano con dei **trigger** piazzati nella scena (passi da un trigger e succede qualcosa);
- l'**interfaccia**: i suoni sono collegati agli **input**.

0.9.21.6 Gli ambienti

I **suoni d'ambiente** sono anch'essi effetti sonori, ma sono i suoni di **tutto ciò che non succede**: il contesto che si ha intorno. Se si entra in una scena e si sta fermi, si sente l'ambiente. Spesso si registrano in **field recording**.

Un tempo gli ambienti erano **loop molto lunghi**: un minuto di traffico, un minuto di giungla, un minuto di rumore di una stanza piena di computer. Oggi l'approccio è cambiato:

- si crea un **loop molto omogeneo**, quasi un sottofondo, che può durare poco, anche **10–15 secondi**: il rumore generico della città, il fondo della giungla;
- sopra, con gli strumenti di oggi (FMOD, Wwise, ma anche i MetaSounds di Unreal[30] o uno script in Unity), si lanciano continuamente dei **suoni casuali**: per la città si producono 5–6 clacson, 6–7 frenate, 10 accelerazioni e altri suoni urbani, e il sistema continua a lanciarli. Si chiamano ***one shot random***.

Il risultato è un **ambiente dinamico**: anche restando fermi per due minuti si sentono suoni lanciati casualmente, magari con volumi casuali, "un po' più basso, un po' più alto", tutto intorno. È questa, oggi, la metodologia con cui si creano gli ambienti.

Un confronto con i libri. Tra le lenti di Jesse Schell c'è la **lente dell'atmosfera** (*Lens of Atmosphere*): l'atmosfera è "invisibile e intangibile", ma ci avvolge e ci fa sentire parte del mondo, e una delle domande della lente è proprio come usare i contenuti artistici, **visivi e sonori**, per renderla più profonda. (Il confronto non è stato fatto a lezione.)

0.9.21.7 I dialoghi

Anche per i **dialoghi** ci sono due tipi.

- I **dialoghi lineari**, come per la musica: nelle cutscene, o quando c'è un narratore. C'è un testo da leggere e lo si registra.
- I **dialoghi non lineari**: si pensi alle **telecronache**, o alle situazioni di botta e risposta, in cui una **macchina a stati** sceglie la frase successiva. Il dialogo si costruisce in base alle scelte: domandi, rispondi una cosa o un'altra, scegli che cosa dire; il tuo personaggio dice quella frase e l'altro risponde con un'altra.

I dialoghi hanno un paio di **criticità**. La prima è la **localizzazione**: oggi con i middleware la si gestisce abbastanza in automatico, ma in produzione bisogna comunque gestire le varie lingue, e i **file sono tantissimi**.

[30] Nella trascrizione "MetaSounds di ARRIA".

La seconda è che spesso bisogna usare dei **placeholder**. Non si fanno registrare subito gli attori: se cambiano i testi o si aggiungono frasi, bisogna richiamare l'attore per tre frasi, e o non è disponibile o fa pagare un'intera sessione di lavoro. Si registrano quindi dei **file temporanei**. Un tempo il relatore faceva registrare amici e colleghi, "registrazioni grezze" per sentire come funzionavano i dialoghi; oggi si può usare l'**intelligenza artificiale**: si generano tutti i dialoghi con ElevenLabs, li si mette in gioco, e alla fine del progetto, quando tutto è **congelato** e non ci saranno nuove frasi, si fanno registrare i doppiatori veri e si sostituiscono i file.

La registrazione vera si fa quindi **il più tardi possibile**, "però non troppo in là, perché sennò poi non ci sei dentro": si individua il momento in cui le frasi devono essere definitive.

0.9.22 Le domande finali

0.9.22.1 Il labiale nelle altre lingue

Un partecipante[31] pone una domanda nata dagli studi di animazione all'accademia. Nei giochi con un pubblico molto vario (Asia, America, Europa) ci sono spesso diversi doppiaggi, ma le **animazioni facciali** non vengono necessariamente rifatte per ogni lingua: ci si ritrova con il "famigerato" *ciao* su un labiale completamente diverso. Ha visto che a volte, con delle *blend shape*, si rende il labiale volutamente vago, abbastanza da dare l'idea senza combaciare con precisione con la lingua originale, così che "sfugge all'occhio". Esistono altri metodi?

Il relatore risponde su due piani.

- Per l'**apertura della bocca**, il piano "fonosintattico" di cui parla il partecipante, esistono **strumenti che leggono l'audio e modificano l'apertura in automatico**. Il relatore ricorda che su un progetto (non ricorda quale) li stavano valutando, e sa che esistono "già da forse più di dieci anni".
- Per la **durata**, il problema si riversa normalmente sul **doppiaggio**. I doppiatori studiano diverse tecniche, e una di queste si chiama ***time constraint***: se la battuta in inglese dura 3 secondi, anche quella in cinese deve durare 3 secondi. "Non sempre facile, non sempre probabilmente si riesce", ma la tecnica esiste, ed evita di avere la voce che continua quando l'animazione è già finita.

0.9.22.2 Come fa il gioco a sapere che hai sparato alla televisione?

Un altro partecipante non ha capito bene il video della televisione colpita: il suono è costruito "sul momento" oppure è "un codice premeditato"? Come fa il gioco a sapere che si è sparato proprio lì?

"È tutto in tempo reale", risponde il relatore, e semplifica. La televisione è lì e sta trasmettendo. Immagina che dall'arma esca un **raggio invisibile** che intercetta gli oggetti del mondo: puntando la pistola, il raggio tocca la televisione. Quando premi il grilletto, il codice dice "ho sparato, e stavo toccando quell'oggetto": il punto d'impatto è sulla televisione, che entra in uno **stato** di codice, "mi distruggo"; e allo stato corrispondono il suono della musichetta che si interrompe e il rumore bianco. Se invece stessi puntando un faretto, si distruggerebbe il faretto. "Non è esattamente così, però il concetto è questo".

Lo stesso vale per i **passi**. Dal personaggio parte un raggio invisibile verso il basso, lanciato ogni tanto mentre cammina, che tocca la superficie; e **ogni superficie ha un parametro**. Il raggio dice "sto toccando erba, erba, erba" e il suono è il passo sull'erba; se la superficie cambia e il raggio intercetta il *cemento*, il suono diventa il passo sul cemento. È tutto codice che intercetta delle **collisioni**, anche invisibili, e restituisce che cosa fare.

0.9.23 Contatti ed eventi di settore

"Direi che ce l'abbiamo fatta": il relatore è riuscito a mostrare quasi tutto, recuperando "proprio il pelo" del ritardo. Per restare in contatto, la cosa migliore è cercarlo su **LinkedIn**, come Davide Pensato, e chiedere il collegamento; accetta tutte le richieste (un partecipante gliel'ha già mandata) e si può scrivergli per qualsiasi cosa.

[31] La trascrizione non dice chi parla: è uno studente dell'accademia che si occupa anche di animazione.

Si era segnato anche di consigliare alcuni **eventi di settore**, utili da frequentare se si è nelle vicinanze:

- il **First Playable** di Firenze, un evento *business*, a pagamento perché vi si incontrano le aziende;
- lo **Svilupparty** di Bologna, gratuito, di solito a metà maggio ("c'è stato");
- la **Milan Games Week**, a Milano, con la sua zona indie, dove il relatore e il suo team vanno di solito;
- sempre in zona Milano, un evento annuale "che è cresciuto molto ed è diventato abbastanza interessante"[32];
- gli incontri dei capitoli locali dell'**IGDA**, l'International Game Development Association, che si stanno diffondendo in tutta Italia (il relatore cita Torino, la Sardegna, la Campania, Bologna; quello di Milano è fermo da un po');
- i **Milano Indies**, un evento nato a Milano che si tiene ogni due mesi a Cologno Monzese, di fronte agli studi Mediaset, e che ha dato origine anche ai **Firenze Indies** e ai **Bologna Indies**: ci sono tanti team indie, ma spesso anche professionisti che lavorano in aziende più grandi.

Il relatore ringrazia per l'invito e saluta: "ci vediamo a qualche evento, o magari a un prossimo incontro". Si era parlato, infatti, di organizzare un **workshop** a seguito della masterclass.

Punti chiave del capitolo

- Il relatore, **Davide Pensato**, ingegnere delle telecomunicazioni e musicista, è stato audio designer (di fatto audio director) in **Ubisoft Milano** per circa otto anni e mezzo e ha poi fondato **DP Studios**, uno studio **verticale** che fa solo game audio.
- Consiglio di carriera: curare da subito la propria **brandizzazione** (sito, portfolio ordinato, LinkedIn, Linktree con logo e colori). Segnale positivo: da anni i nuovi progetti dello studio sono **tutti italiani**.
- La **contrazione** del mercato non è dovuta all'IA che sostituisce il lavoro, ma al ridimensionamento dopo il boom del Covid, allo spostamento degli investimenti verso le aziende di IA e a un mercato che premia giochi più piccoli, come i ***Triple-I***.
- Per Spielberg suono e musica comunicano **più della metà** dell'esperienza; nel videogioco, interattivo, quello che si sente e non si vede cambia il modo di giocare.
- Tre elementi di importanza dell'audio: **potere evocativo** (spostare o rafforzare l'emotività, con la musica ma anche con il solo sound design, come in *Manhunt*), **caratterizzazione** (una **simulazione** in tempo reale, con un suono per ogni stato di ogni oggetto) e **supporto al gameplay** (informazioni non visibili: passi, telecamere, musica di combattimento).
- La musica interattiva sfrutta i momenti in cui il gioco **sa già** che cosa succederà (la mossa *finishing*: cinque secondi per una transizione pulita). A volte è il giocatore a sfruttare la struttura della musica.
- Storia: negli **anni Ottanta** chip a quattro voci, sintesi, linguaggio macchina, il **programmatore-compositore**, il ***voice stealing*** (ancora attuale) e la nascita del **tracker** su Amiga; negli **anni Novanta** wavetable, **MIDI**, studi esterni, il **Game Audio Engineer**, il CD della PlayStation e i primi script; **dal 2000** audio digitale, spaziale, effetti in tempo reale, audio adattivo, qualità cinematografica.
- Ruoli: **audio director** (visione, tecnica, scelta delle persone; viene quasi sempre dal sound design), **sound designer** (field recording, foley, librerie; loop brevi, code tagliate, suoni asciutti, zero silenzio in testa), **compositore** (musica in loop, interattiva, in molti stili), **dialogue director** e doppiaggio, **audio programmer** (limiti, engine, tool, specifiche chiare: il caso degli *stem* mancati).

[32] Nella trascrizione "Cotoletta 8 Beat"; il nome esatto non è ricostruibile con certezza.

- Sull'IA nel doppiaggio: il rischio di sostituzione esiste, soprattutto per i progetti a budget zero; generare una singola frase da un doppiaggio esistente può essere un ritocco legittimo; molti doppiatori stanno già mettendo in vendita la propria voce clonata.
- Per Alexander Brandon la produzione dei contenuti è **solo metà del lavoro**: l'altra metà è l'**implementazione**.
- Nel ciclo di produzione l'audio deve esserci **già in pre-produzione** (audio design, funzionalità, tecnologia, lista degli asset e budget); in produzione si creano e soprattutto si **integrano** i contenuti (l'armatura in tre layer); in post-produzione, tra beta e gold master, si fanno il **mixing in tempo reale** e l'**audio test**.
- Quattro categorie di suoni: **musica** (source, lineare, non lineare; *action* o *mood*; interattività **orizzontale** o **verticale**), **effetti sonori** (agganciati ad animazioni, stati, trigger, input), **ambienti** (loop breve e *one shot random*) e **dialoghi** (lineari e non lineari, localizzazione, placeholder, registrazione finale il più tardi possibile).

Part II

Fondamenti di Unreal Engine

Chapter 1.1

Introduzione a Unreal Engine: che cos'è un engine, chi lo usa, e il primo contatto con l'editor

Questa è la **prima lezione** del corso breve di **introduzione a Unreal Engine**, un corso che ha il suo centro nei **Blueprint**, il sistema di programmazione visuale "a nodi" dell'engine di Epic Games. La lezione ha due metà molto diverse. La prima è teorica e larga: che cos'è un *software*, che cos'è un *game engine*, come un modello fatto di poligoni diventa un'immagine sullo schermo, perché Unreal è oggi l'engine più diffuso, quali figure di una produzione lo usano e per che cosa, e infine, con un'attività in classe, da che cosa è formato un gioco. La seconda metà è pratica: si scarica Unreal Engine 5.5, si guardano le risorse che Epic mette a disposizione (la documentazione, i corsi gratuiti, il negozio di *asset* Fab), si crea un primo progetto vuoto e si impara a muoversi nell'editor: piazzare un cubo, spostarlo, ruotarlo, scalarlo, duplicarlo, raggrupparlo, ordinarlo in cartelle. La lezione si chiude con un'esercitazione: arredare un piccolo ristorante con tre tavoli e le loro sedie.

Il docente è **Igor**[1], che si presenta come ex alunno della stessa scuola degli studenti, "precisamente per quanto riguarda game design"[2]. Ha lavorato a ***Steel Seed***, "principalmente come *level designer*"; ma "essendo un ambiente indie" faceva "un po' di tutto", anche programmazione e *scripting*, e quindi ha fatto "uso intensivo" di Unreal Engine "e di tutte le sue feature". Il corso è diviso tra due docenti: con Igor la parte di programmazione e di "piccole feature"; con **Davide**, a partire dal 26 del mese, una parte "un po' più artistica". Le lezioni si alternano: il giovedì successivo gli studenti saranno di nuovo con Igor, il martedì dopo con Davide.

La lezione è **online**, su Zoom, con un server Discord del corso per i materiali e le domande; inizia poco dopo le 17 e finisce alle 19, con una pausa di una decina di minuti verso le 17 e mezza[3]. Gli studenti sono tutti **nuovi iscritti al primo anno**, di corsi diversi: game design (la maggioranza), narrative design, character animation, 3D art e programmazione. Il corso breve, in altre parole, è comune a figure che in una produzione useranno Unreal in modi molto diversi, ed è proprio da questa varietà che Igor parte.

Tre avvertenze di lettura. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal diventa "Arreal", "Arial Engine", "ArrayLange", "ArrayLab", "Arrival Engine", "Arrial", "Ariane Engine", "Riala" e "a Real Engine"; *Clair Obscur: Expedition 33* è "Expedition" e "Spedisce 33"; Sandfall Interactive è "Send Follow Interactive"; *shader* è anche "scender". Dove la ricostruzione è sicura il capitolo usa la forma corretta e, la prima volta, lo segnala in nota; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: in più punti la registrazione contiene lunghi blocchi di testo senza senso e ripetuto ("qual è la risposta che ci può essere?", "a parte il plus time…"): sono artefatti del riconoscimento vocale durante la

[1] Il docente si presenta come "Igor Ripanti"; il cognome è quello della trascrizione automatica, che altrove storpia quasi tutti i nomi propri, e il capitolo non è in grado di verificarlo.

[2] La trascrizione dice "sono stato un alunno di Live": il nome della scuola è con ogni probabilità storpiato.

[3] Igor, credendo per errore che la lezione finisse alle 18 ("sono da un altro corso che finiva alle 18"), dopo la pausa annuncia che "manca mezz'ora alla fine"; gli studenti lo correggono ("nella mail c'era scritto" alle 19), e il docente ha "un'altra ora" per la parte pratica.

pausa e i momenti di lavoro individuale, e il capitolo li ignora. La terza: la seconda metà della lezione è una dimostrazione dal vivo nell'editor; il capitolo ricostruisce i passaggi da ciò che Igor dice mentre li fa, e li completa, dove serve, con i nomi esatti dei comandi.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono gli schemi spiegati a voce o sulle slide (la pipeline di rendering, il *game loop*, gli elementi di un gioco, l'interfaccia dell'editor, il *transform*); le tabelle riassumono i ruoli della produzione e le scorciatoie da tastiera della seconda parte. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro *Blueprints Visual Scripting for Unreal Engine 5* di Marcos Romero e Brenden Sewell, che sarà il testo di riferimento per questa parte del libro, e a qualche nozione tecnica generale; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. le presentazioni e le esperienze degli studenti (sezione 1.1.1);
2. il **software**, il **game engine** e lo **scripting**, con i Blueprint (sezione 1.1.2);
3. il **rendering 3D**: poligoni, *vertex shading*, rasterizzazione, *fragment shading*; poi il *real time rendering* e i *frame* (sezioni 1.1.3 e 1.1.4);
4. **Unreal Engine**: perché è così usato, la prototipazione, i giochi, gli engine di una volta, il cinema, il **game loop** (sezioni da 1.1.5 a 1.1.7);
5. la **produzione** e i **ruoli** che toccano l'engine (sezioni 1.1.8 e 1.1.9);
6. l'attività in classe: **da che cosa è formato un gioco**, e le domande prima della pausa (sezioni 1.1.10 e 1.1.11);
7. il **download** della 5.5, le **risorse** (documentazione, *learning*, Fab) e un po' di teoria sul *Play in Editor* e sul *delta time* (sezioni da 1.1.12 a 1.1.14);
8. il primo contatto con l'**editor**: il progetto, l'interfaccia, il **transform**, la navigazione, l'organizzazione (sezioni da 1.1.15 a 1.1.20);
9. l'**esercitazione dei tre tavoli**, il salvataggio e la chiusura (sezioni 1.1.21 e 1.1.22).

1.1.1 Presentazioni: chi c'è, e chi ha già aperto un engine

La lezione comincia con qualche minuto di attesa ("aspettiamo cinque, dieci minuti"), un appello e una richiesta: accendere la webcam, "così almeno ci vediamo un po' in faccia". A chi non ha una webcam Igor suggerisce le applicazioni che trasformano il telefono in una webcam, "come se ti facessi una videochiamata": lui usa di solito *Iriun*[4]. I lavori in corso nell'edificio del docente producono "rumori molesti" per tutta la lezione, e Igor se ne scusa.

Poi il giro di presentazioni: nome, corso, "un piccolo background". Tutti sono iscritti al primo anno; i corsi sono game design, character animation, narrative design (più di uno), 3D art e programmazione. "C'è una bella quantità di nuovi compagni di design", commenta Igor, "quindi avremo pane per i nostri denti", oltre ovviamente a 3D art e programmazione.

Chi ha già usato un engine? La domanda successiva è se qualcuno abbia "mai aperto un software di *develop* per i videogiochi": Unity, Godot, "in generale". Qualcuno ha provato Godot; qualcuno un po' di Unity, seguendo un corso online "molto veloce" e facendo "un paio di giochini", un *platform* e un clone di *Flappy Bird*, "di 3D niente"; qualcuno è "digiuno", e qualcun altro ha lavorato "con roba presa da Fab". Igor accoglie tutte le risposte allo stesso modo: "ci sta assolutamente". Fab, aggiunge, sarà uno dei temi di oggi, perché è "una risorsa incredibile", soprattutto per le cose gratis, quelle in sconto e i contenuti gratuiti del mese.

[4]Nella trascrizione "Ingur" e "Iruin"; si tratta con ogni probabilità di *Iriun Webcam*, un'applicazione gratuita che fa esattamente questo.

La mentalità prima degli strumenti. A chi parte da zero Igor dà subito una rassicurazione che vale per tutto il corso: "l'importante è avere la mentalità". In programmazione, "molto spesso" si tratta "semplicemente" di imparare a usare gli strumenti, mentre "la logica fa da padrone". Passare da un engine all'altro, o da nessun engine al primo, è soprattutto "riabituarsi a un nuovo campo di gioco".

1.1.2 Software, game engine e scripting

Che cos'è un software. Igor parte dalla definizione più semplice possibile: il **software** è "quello che non è hardware, fondamentalmente". È un insieme di istruzioni, di dati e di programmi con funzionalità precise, che ci permettono di creare cose: strumenti di 3D art come **Blender** e **Maya**, oppure *engine* come **Unreal Engine** e **Unity**.

L'engine come calderone. Il **game engine** è "il calderone, chiamiamolo un po' così", che riunisce tutte le facoltà di chi sviluppa (3D art, programmazione, design) e le mette "in un ambiente comune", l'ambiente di sviluppo dell'engine. Gli engine più noti sono **Unreal**, **Unity**, **Godot** e **GameMaker**, e ognuno contiene al suo interno molte funzionalità: i **motori fisici**, che gestiscono le **collisioni**; la gestione dei **suoni**; le **animazioni**; il **networking**; l'**intelligenza artificiale** dei personaggi. Nessuna di queste cose "si fa da sola": ci mettono le mani gli sviluppatori, "tramite lo *scripting*".

Lo scripting, e i Blueprint. Lo **scripting** è "la forma di programmazione che ci permette di entrare in interazione e creare cose all'interno dell'engine". Esistono molti linguaggi: Igor cita il **C**, il **C#** (quello di Unity) e i **Blueprint**, che sono il tipo di *scripting* di Unreal Engine e l'oggetto di questo corso. Più avanti nella lezione (sezione 1.1.9) Igor tornerà sul punto: in Unreal si può programmare anche in codice (in C++), "ma noi non lo useremo", perché i Blueprint bastano "tranquillamente", sono usati anche dai programmatori, e "tanti giochi nel comparto indie" sono fatti "solo con Blueprint, senza mai toccare una riga di codice". I Blueprint sono un **linguaggio a nodi**: "prendiamo proprio e colleghiamo tutti i nodini" per dare un senso alla programmazione.

Approfondimento: le due accezioni di "Blueprint". Romero e Sewell, nel primo capitolo di *Blueprints Visual Scripting for Unreal Engine 5*, fanno notare che in Unreal la parola *Blueprint* ha due significati. È il nome del **linguaggio di scripting visuale** creato da Epic Games; ed è anche il nome di un **tipo di oggetto di gioco** creato con quel linguaggio. I Blueprint di questo secondo tipo sono di due specie: il **Level Blueprint**, uno per ogni livello del gioco (non se ne possono creare altri separati), e le **Blueprint Class**, che servono a creare oggetti interattivi riutilizzabili in qualunque livello. Un progetto di Unreal Engine 5, aggiungono gli autori, si può sviluppare in Blueprint, in C++ o in una combinazione dei due: si può aggiungere codice C++ a un progetto Blueprint e viceversa. La scelta di partire dai Blueprint, quindi, non chiude nessuna porta.

1.1.3 Il rendering 3D: dai poligoni ai pixel

Un'applicazione grafica come un gioco 3D ha una particolarità: il **3D rendering**, cioè "la fase finale dello sviluppo di un'immagine". È il processo che prende un ***3D asset*** ("per il linguaggio un po' più tecnico") e ci presenta davanti la scena vera e propria, con gli oggetti creati dai 3D artist "che hanno faticato tanto per poterlo fare". Quegli oggetti sono formati da **poligoni**.

Quanti poligoni ha questo oggetto? Igor chiede se qualcuno ha mai sentito parlare del numero di poligoni di un modello. Uno studente sì: gli sono capitati "dei video di ottimizzazione" su come diminuire i poligoni "per cercare di diminuire il *lag*". "Esatto, bravissimo." La regola è semplice, e Igor la dice con una battuta: "se mai doveste avere il caso fortuito di avere un cubo con due milioni di poligoni, sappiate che il vostro gioco esploderà, probabilmente". Nella **viewport** ("un elemento che poi vedremo": la finestra in cui l'engine, ma anche Blender mentre si modella, mostra la scena) vale sempre lo stesso compromesso: **più poligoni ci sono, più l'immagine è precisa, ma più è pesante da renderizzare**.

Le tre fasi. Il rendering di un'immagine passa per molti passaggi; "noi oggi" ne guardiamo tre principali (figura 1.1.1).

1. ***Vertex shading.*** Le immagini e i modelli sono formati principalmente da **triangoli**, e i triangoli sono formati da **vertici**. I vertici definiscono la forma: in questa fase si identificano e si posizionano i tre punti di ogni triangolo.

2. **Rasterizzazione.** Il triangolo così definito viene *rasterizzato*: si stabilisce quali **pixel** dello schermo cadono dentro quel triangolino. Igor la spiega a partire dalla grafica 2D ("qualcuno ha fatto grafica?", e una studentessa laureata in grafica conferma). Un'immagine **raster**, come quelle di Photoshop, è fatta di pixel; un'immagine **vettoriale** è fatta di vettori. La differenza: se si ingrandisce ("strecciate") un'immagine raster, diventa "sgranatissima"; un'immagine vettoriale la si può ingrandire "quanto volete" e resta uguale. Gli schermi, però, sono fatti di pixel: per questo i triangoli vanno rasterizzati.

3. ***Fragment shading.*** È la fase finale, quella "vicina alla visualizzazione". Ai **frammenti** ("poligoni, triangoli, chiamateli come vi pare") si applicano i **colori**, la **luce**, le **ombreggiature** e i **riflessi**, tenendo conto della posizione della camera. È anche "una parte molto artistica", perché Unreal lavora molto sulla resa della luce: si possono piazzare luci, gestire come la luce entra in una stanza, e così via.

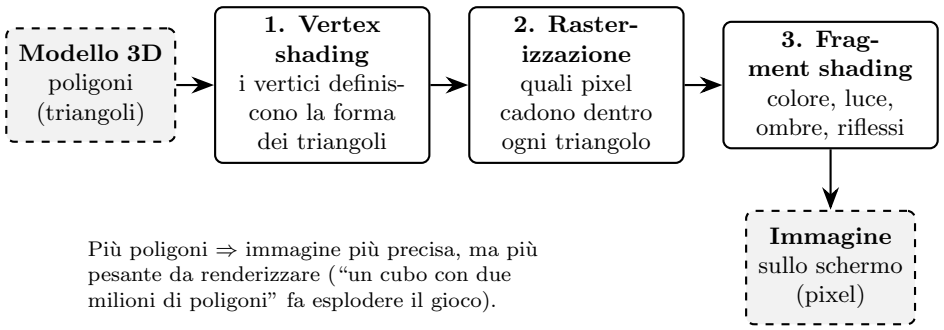


Figure 1.1.1: Le tre fasi principali del rendering di cui parla Igor. Il processo reale ha molti passaggi intermedi; la lezione si ferma a questi tre.

Per vertici o per frammenti. Sulla slide, in basso a destra, ci sono due versioni della stessa sfera illuminata. Nella prima la luce è calcolata **per vertici**: la luce "quasi" lascia vedere i triangoli. Nella seconda è calcolata **per frammenti**, cioè dopo il *fragment shading*: la luce è "molto più diramata", morbida, "e fa sì che funzioni per bene".

Le UV. Igor accenna anche alle **UV**, di cui si parlerà più avanti nel corso: sono "delle mappe che i 3D artist creano" e che permettono di dire come il materiale si comporta sulla superficie del modello, come gestire e rifrangere la luce, "questo materiale così", e così via.

Approfondimento: che cosa sono le UV. Per completezza (la lezione ci torna solo di sfuggita): le **coordinate UV** sono il modo in cui la superficie di un modello 3D viene "srotolata" su un piano a due dimensioni, come si apre la scatola di cartone di un cubo. Le lettere U e V indicano i due assi di quel piano, per non confonderli con gli X, Y e Z dello spazio 3D. Su quel piano si dipingono le *texture* (il colore, la rugosità, i dettagli in rilievo), che poi l'engine "riavvolge" sul modello durante il rendering. Un buon *UV mapping* è il motivo per cui una texture appare senza deformazioni o cuciture visibili.

1.1.4 Il *real time rendering* e i *frame*

In Unreal Engine il rendering è **in tempo reale**. Igor lo collega a un'esperienza che molti studenti faranno: lo *showreel*, soprattutto per chi diventerà 3D artist, sarà "molto accompagnato" dal rendering di Unreal,

con camere ed effetti "fatti per bene, scriptati", che permettono di vedere gli oggetti "alla massima qualità possibile". E come sempre: più alta è la qualità, più tempo ci vuole per renderizzare. Il *real time rendering* è il rendering che produce le immagini a una velocità compresa, più o meno, tra i **20 e i 120 fotogrammi al secondo**.

Che cos'è un frame. Un **frame** (fotogramma) è una singola immagine; la **visualizzazione per frame** è un susseguirsi di immagini correlate l'una all'altra che danno l'**illusione del movimento** sullo schermo. È lo stesso principio dell'animazione: il cinema lavora a **24 *frame* al secondo** ("quando andate a vedere un film"), il gaming di solito a **30 o 60 *fps***, e c'è chi gioca a 120–144 *fps* ("incredibili", commenta Igor, "l'*fps* che se la tira tanto").

Approfondimento: il budget di un frame. Un modo utile per leggere questi numeri è rovesciarli. A 30 *fps* l'engine ha circa $1000/30 \approx 33$ millisecondi per produrre ogni immagine; a 60 *fps* ne ha circa 16,7; a 120 circa 8,3. In quel tempo deve leggere gli input, aggiornare la logica di gioco (il *game loop* della sezione 1.1.7) e renderizzare la scena. È per questo che l'ottimizzazione (meno poligoni, meno luci dinamiche, nebbia più leggera) non è un dettaglio tecnico ma una condizione per avere un gioco giocabile: tutto ciò che non entra nel *budget* fa scendere il *frame rate*.

1.1.5 Unreal Engine: perché è così usato

Unreal Engine è "il nostro bellissimo" game engine, quello che si userà per tutto il corso. È sviluppato da **Epic Games**, e per usarlo bisogna scaricare l'**Epic Games Launcher** dal sito di Epic (lo si farà durante la lezione, sezione 1.1.12).

Il più usato. È, dice Igor, "proprio il più usato al momento": "gran parte dei giochi moderni", AA e AAA, tendono a usarlo. Una delle ragioni è che, essendo un motore comune, Epic lo aggiorna continuamente, con nuove funzionalità come il **Control Rig**, che permette per esempio di fare le animazioni in modo procedurale. Nel launcher si vede: ogni volta propone una versione diversa da scaricare (5.5, 5.6, 5.7...), e il canale YouTube ufficiale di Unreal Engine pubblica a ogni versione un video riassuntivo delle novità.

Un linguaggio comune. La seconda ragione è che Unreal è un **linguaggio comune**: permette ai professionisti di parlarsi e di produrre insieme. I designer, per esempio, lo usano "molto spesso per prototipare". La **prototipazione** è una fase "molto importante" della creazione di un gioco: "tiro fuori l'idea dal cappello" e la si mette "direttamente in engine", usando lo *scripting* per vedere se "questa idea che ho in testa funziona". È "il modo più veloce e più giusto per capire subito se qualcosa funziona". Poi ci sono molte altre figure che usano l'engine nelle sue diverse caratteristiche (sezione 1.1.9).

Alcuni giochi fatti in Unreal. Sulla slide: *A Way Out*, *Ark*, *Crash Bandicoot 4* ("per chi ha giocato, bellissimo") e *Fortnite*. Per *Fortnite* esiste anche una versione dell'engine, **Unreal Editor for Fortnite** (UEFN), che permette di prendere le funzionalità del gioco e "metterle insieme". Igor lo paragona a **Roblox**, che sta facendo "un simile effetto": creare un **mega *game template***. Un *game template* è "il *pattern* di regole", tutto ciò che forma il gioco, messo a disposizione delle persone perché ci facciano "quello che vogliono".

Il caso *Expedition 33*. Igor chiede quale gioco fatto in Unreal sia rimasto in testa agli studenti, e la risposta arriva subito: ***Clair Obscur: Expedition 33***[5]. "Benissimo, vedo che ci intendiamo." Il gioco, racconta Igor, è nato da persone che, "per quanto dicano loro", hanno detto: "non sapevamo fare un gioco, e ci siamo messi a guardare YouTube". Igor mostra il video del **prototipo** di *Expedition 33* fatto in Unreal Engine (dopo la solita pubblicità): è lo stadio che si affronta "molto spesso all'inizio di una produzione", quello in cui "non ci interessa se è bello o brutto, è importante che funzioni". Unreal permette proprio questo: mettere subito in campo le cose più usate del gaming.

[5] Nella trascrizione quasi sempre solo "Expedition" o "Expedition 33", una volta "Spedisce 33".

Le cose comuni. Ci sono infatti tante cose "comuni all'interno del nostro panorama videoludico". Il **salto**, per esempio: "quante volte in un gioco si salta?". C'è chi non lo usa, ma i giochi con il salto sono "a bizzeffe". Unreal, creando una sorta di *template* comune, facilita molto i primi passi.

1.1.6 Gli engine di una volta, e Unreal fuori dai videogiochi

Quando ogni gioco aveva il suo engine. "Non siamo lì, ragazzi, nel 2004." Nei vecchi giochi, ricorda Igor, si creavano engine **apposta**: un programmatore doveva passare dalle linee di codice alla visualizzazione in 3D, "in tutto un processo che andava dai due ai tre anni". E quegli engine erano poco riutilizzabili, perché contenevano "*pattern* di regole molto fissati". Si trovavano comunque modi interessanti di riusarli: Igor cita, come "piccola curiosità", un gioco di Shrek per PlayStation 2 ("Shrek 2", si corregge) realizzato modificando il motore di un gioco del *Signore degli Anelli*[6].

Gli engine proprietari. Esistono poi gli engine **proprietari**, come **Source** e **Source 2** di Valve (*Half-Life*): sono "loro", di Valve, ma la casa mette a disposizione gratuitamente gli strumenti per creare mappe, con l'editor **Hammer**. Unreal, invece, mette a disposizione "tante cose diverse" per "mettere subito la carne sul fuoco" e facilitare il progetto.

Unreal nel cinema. Oltre ai videogiochi, Unreal è "molto usato" nel ***filmmaking*** e nel *videomaking*. Igor mostra uno spot di motociclette ("della Indian"): le rocce intorno al motociclista "non esistono", sono renderizzate in Unreal. È "la classica storia del *green screen*", con dietro il lavoro di chi è esperto di Unreal. Uno studente ricorda che anche *The Batman* fu girato in parte in questo modo, e Igor conferma che "se ne parlò": oggi è "molto in voga", perché "nessuno ha voglia di andare di nuovo in Islanda" per avere delle rocce molto belle; ci si fa le rocce pre-renderizzate.

Ottimizzazione contro dettaglio. I processi, però, sono "molto differenziati". Nei videogiochi si parla molto di **ottimizzazione**: modelli ***low poly***, non pesantissimi e non precisissimi, "belli da vedere ma ottimizzati". Nel *filmmaking* l'ottimizzazione esiste, ma conta il **dettaglio**, perché il risultato è "solo una raffigurazione". Un videogioco, invece, è **un programma in esecuzione**: continui *frame*, continue interazioni, un continuo *game loop*.

Le rocce che si vedono ovunque. Una battuta tra sviluppatori: esistono rocce che "sono le stesse, perché sono bellissime". Sono fatte in 4K e **scannerizzate**: qualcuno si è messo letteralmente a fotografare le rocce, le ha digitalizzate e le ha messe gratis sull'*asset store*. "E le vedete ovunque": "ormai sono riconoscibili", fatte molto bene ma molto comuni.

Approfondimento: *virtual production* e *scan*. Le due cose di cui parla Igor hanno un nome. La prima è la ***virtual production***: invece di aggiungere lo sfondo in post-produzione su un *green screen*, si proietta in tempo reale l'ambiente renderizzato da Unreal su grandi pareti di LED intorno agli attori, così che luci e riflessi siano già giusti in ripresa (la tecnica è stata resa celebre da *The Mandalorian*). La seconda è la **fotogrammetria**: si fotografa un oggetto reale da decine o centinaia di angolazioni e un software ne ricostruisce il modello 3D con le texture. Le rocce "che si vedono ovunque" vengono quasi certamente dalla libreria **Quixel Megascans**, che Epic ha acquisito e reso gratuita per Unreal, e che oggi è distribuita attraverso Fab.

1.1.7 Il *game loop*

Che cosa succede, allora, "all'interno di un *frame*" di un gioco? Il ***game loop*** (figura 1.1.2) è il ciclo che distingue un videogioco da un video:

[6] L'aneddoto è riportato come lo racconta Igor, che lo introduce con "io mi ricordo"; il capitolo non ha verificato quale gioco del *Signore degli Anelli* e quale versione di *Shrek* siano coinvolti.

1. si **inizializza** il gioco;
2. il programma è **in attesa di un input** e lo legge: "io premo X, A, o quello che è";
3. dopo l'input c'è un ***update***: a livello di programmazione il personaggio salta, si ha il movimento;
4. si **renderizza** l'immagine;
5. se si esce dal gioco il processo finisce; altrimenti si ricomincia dal punto 2, "in questo iter continuo".

La differenza sostanziale tra il *videomaking* e "un vero videogioco" è proprio la presenza del *game loop*.

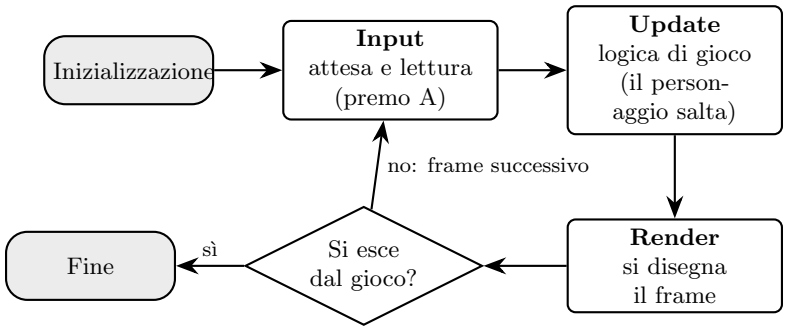


Figure 1.1.2: Il *game loop* come lo descrive Igor. Ogni giro del ciclo produce un *frame*: a 60 *fps*, sessanta giri al secondo.

Approfondimento: l'evento Tick. Nei Blueprint il *game loop* si vede in modo molto concreto. L'evento **Event Tick** viene eseguito **una volta per ogni *frame***, e porta con sé un valore, **Delta Seconds**, che è il tempo trascorso dal *frame* precedente. È il punto in cui, nei Blueprint, si mette la logica che deve aggiornarsi di continuo (un oggetto che ruota, una barra che si riempie); ed è anche il motivo per cui Igor, più avanti nella lezione (sezione 1.1.14), insiste sul rendere i movimenti "relativi al tempo di gioco".

1.1.8 La produzione: idee su carta, indie, AA, AAA

Una produzione ha molte figure ("ognuno di voi è programmazione, 3D artist, game design"), e tutte si incontrano in un punto comune: Unreal Engine. Prima dello sviluppo, però, c'è la **pre-produzione**, in cui si gestisce l'idea.

Lo storyboard. Igor chiede chi sappia che cos'è uno **storyboard**: una studentessa che ha studiato cinema e televisione, e una che ha lavorato in animazione, ne hanno "un minimo di base". In pre-produzione bisogna capire che cosa si vuole fare, perché se arriva qualcuno a dire "io ho l'idea del secolo", la risposta è: "sì, ma me la devi far capire". **Ogni buona idea non esiste finché non sta su carta** e non viene compresa da tante altre persone.

La documentazione. Per i designer, la carta è la **documentazione**: "noi designer facciamo documenti", spieghiamo le meccaniche "anche nel minimo dettaglio", soprattutto per le figure più in alto nella gerarchia.

Indie, AA, AAA. La differenza tra le produzioni **indie**, **AA** ("doppia A") e **AAA** ("tripla A") è soprattutto una differenza di soldi: quelli che i giochi producono (lo suggerisce uno studente) e soprattutto quelli "che ci si mette dietro", e quindi le possibilità.

- Nell'**indie** si lavora in team piccoli: "al massimo" venti persone, venti–trenta, e "trenta già siamo abbastanza". Le figure sono poco separate: "mi puoi fare la programmazione, ma io sono un designer, e vabbè", è stata "molto la mia esperienza".

- Il **AA** è "una media impresa": ci sono soldi, ma non come nel AAA, e le figure "aleggiano" spesso tra l'indie e l'AA, perché "quando c'è necessità si fa virtù". Anche *Expedition 33* è un AA, "che comunque c'ha tanti soldi".
- Nel **AAA**, più si sale, **più preciso è il ruolo**. "Ci sono artisti che fanno solo foglie": i *leaf artist*, che studiano "tutte le particolarità di una foglia", come cade, come il vento agisce su di essa, le sue nervature, come si differenzia nell'ambiente, come è ottimizzata.

L'*outsourcing*. Sandfall Interactive[7], lo studio di *Expedition 33*, ha usato l'***outsourcing*** per le animazioni: secondo Igor sono state fatte in Corea. Uno studente definisce l'*outsourcing*, correttamente: è "affidare alcune parti della produzione ad altri studi"; "esatto, bravissimo". Un altro studente cita il remake di *Warcraft III*, fatto da uno studio esterno, "se non sbaglio coreano". Igor accetta l'esempio, ma aggiunge una precisazione importante: quando c'è la *ownership* del prodotto, cioè quando lo studio esterno ha la proprietà di ciò che fa, "quello non è *outsourcing*". L'*outsourcing* è quando "io sono l'azienda X, vado dall'azienda Y e le chiedo: senti, mi puoi fare le animazioni? Mi puoi fare il *level design*?".

1.1.9 Chi usa l'engine, e per che cosa

I ruoli si distinguono per anzianità ("il classico" **junior**, **mid** e **senior**) e per area. Sulla slide le aree sono sei: **produzione**, **audio**, **programmazione**, **design**, **visual art** e **business** ("*miscellaneous*": ci finiscono anche, per esempio, i recensori e le testate giornalistiche come IGN). Per ognuna Igor racconta come si interfaccia con Unreal (tabella 1.1.1).

Design. Tre esempi.

- Il **game designer**, per esempio un *combat designer*, crea **situazioni prototipate** di combattimento usando dei *pattern* creati in Unreal dal programmatore. Lavora su richiesta: "mi serve che questo parametro sia esposto, così lo posso muovere". Quanto danno fa, per esempio.
- Il **level designer** lavora con il ***blockout*** (o *grey box*, *white box*). "Il *blockout* è la forma primordiale di un livello": non serve che il livello sia "fatto, finito e compiuto, tutto bello, bellissimo", ma che le forme riportino la loro **funzionalità**, che poi il 3D artist renderà belle. Nel *blockout* si definiscono gli spazi e le **metriche**: quanta distanza c'è, se è abbastanza per sparare, se si vede bene il nemico. Lo si fa in Unreal, anche con il **Modeling Tool** (la modalità di modellazione dell'editor), meno preciso di Blender ma comodo per fare forme "un po' più carine". Sulla slide, un *blockout* di *Uncharted* di Emilia Schatz: "queste sono le prime immagini che i level designer danno agli artisti".
- Il **narrative designer** inserisce dialoghi e **momenti scriptati**: il personaggio che parla tra sé, il punto in cui si può giocare, quello in cui il personaggio discute, l'NPC con cui si interagisce. Il programmatore, "molto spesso in relazione con il narrative designer", crea sistemi prototipati e porta tutto a finalizzazione; se si è un *technical* (narrative designer), lo si fa in parte da sé. Le figure di design sono molto collegate, perché "i livelli devono avere un contesto": "ambientato in una fabbrica" e "una fattoria" sono livelli totalmente diversi.

Visual art. Qui l'engine entra in molti modi.

- I **materiali**: nelle *texture* si può "mettere delle logiche" e farle muovere. La lava che scorre "molto spesso è un'immagine ferma" che si muove grazie ai materiali di Unreal; o si usano effetti di **tassellazione** che fanno "molleggiare" la superficie.
- L'**animazione**, con il **Sequencer**: permette di fare animazioni in tempo reale dentro il gioco, momenti scriptati come una *cutscene* "in live", scatole che cadono, un'astronave che passa davanti. Igor mostra un video di *Star Wars Jedi: Fallen Order*: oltre alle *cutscene* principali ci sono eventi "totalmente scriptati", come una piccola navicella che passa mentre il personaggio parla, o le navicelle tutto intorno.

[7] Nella trascrizione "Send Follow Interactive".

Sono animazioni "in modo cinematografico", fatte molto spesso con il Sequencer, e spesso volute dal narrative design.

- Il **concept art** non ha un uso molto particolare dentro l'engine, ma può servire, soprattutto se si parla di 2D, perché anche in Unreal si possono fare giochi 2D.
- Il **VFX artist**, che lavora con i sistemi di particelle (Niagara, di cui si parlerà più avanti).
- Il **technical artist**, la figura che sta tra programmazione e parte artistica e "riesce a mettere in relazione le due cose".
- L'**animatore**, di cui si è già detto.

Programmazione. "Io già do un consiglio ai designer: **trattate bene i vostri programmatori**, sempre." La finalizzazione di tutte le idee che vengono in testa ai designer "è loro". Anche qui, più si sale nell'industria, più i ruoli sono sezionati:

- il **combat programmer** e il **3C programmer**, dove le **3C** sono *camera*, *controller* e *character*;
- il **rendering engineer** (o *engine programmer*), che lavora su come l'immagine viene renderizzata. Unreal è un engine a **sorgente aperto**: lo si può aprire e dire "ti cambio la visualizzazione di come renderizzi una certa cosa". L'esempio è il remake di *Silent Hill 2*: la **nebbia** è "la parte più importante", e "pesa tantissimo"; per avere un gioco pieno di nebbia senza che vada "a scatti" serve un rendering della nebbia ottimale, ed è il lavoro di questa figura;
- l'**AI programmer**;
- chi si occupa del **networking**, per il multiplayer (client e server, "un po' complesse");
- e figure più esterne, per database e servizi.

Il linguaggio di codice di Unreal è il C++[8], "ma noi non lo useremo": per Unreal basta il **Blueprint**, che "è usato dai programmatori" ("quindi, programmatori, non scappate"). Il C++ si usa per esempio per ottimizzare certi processi, "però l'interfaccia principale è sempre Blueprint". E "tanti giochi nel comparto indie" sono fatti solo con i Blueprint.

Audio. Nessuno studente è del corso di audio, ma l'audio è gestito da Unreal anche a livello di programmazione: si possono **randomizzare** i suoni, o **modularli**, con un modulatore presente nell'engine.

Produzione. Il *playtesting*, la segnalazione dei *bug*, la gestione del progetto. Il *producer* lavora soprattutto su altre piattaforme; il **game tester** usa di più l'engine, per esempio con **Unreal Insights**, uno strumento che risponde a domande come "quante cose sono visualizzate a schermo qua sopra, e quanto è pesante?".

Business. La parte "più esterna" usa l'engine poco, ma lo usa: per l'**eseguibile**, e per le ***build* di *debug*** che vengono presentate, per esempio, ai giornalisti. Sono *build* con la facoltà di "mettere i *cheat*", per saltare parti del gioco o andare direttamente a una certa mappa, "come i codici di una volta" ("non sono così anziano, però...").

1.1.10 Attività in classe: da che cosa è formato un gioco?

La pausa, propone Igor, si può fare subito o più tardi; la classe preferisce "verso le 5", e si va avanti ancora un quarto d'ora con un'attività: "per voi, da che cosa è formato un gioco? Quali sono gli elementi che formano un gioco?".

[8] Nella lezione Igor dice "C"; il linguaggio di Unreal Engine è il C++.

Area	Figure citate	Come usano Unreal
Design	game (combat) designer, level designer, narrative designer	prototipi di combattimento con parametri esposti; *blockout* e metriche dei livelli, Modeling Tool; dialoghi e momenti scriptati
Visual art	materiali, animatori, concept artist, VFX artist, technical artist, leaf artist	materiali con logica (lava, tassellazione); Sequencer per eventi scriptati; 2D; particelle; ponte tra arte e codice
Programmazione	combat e 3C programmer, rendering/engine programmer, AI programmer, network	Blueprint (e C++ per ottimizzare); modifica del rendering (la nebbia di *Silent Hill 2*); IA; client e server
Audio	sound designer	randomizzazione e modulazione dei suoni
Produzione	producer, game tester (QA)	*playtesting*, *bug*, gestione; Unreal Insights per misurare il peso della scena
Business	stampa, marketing	eseguibile; *build* di *debug* con *cheat* per i giornalisti

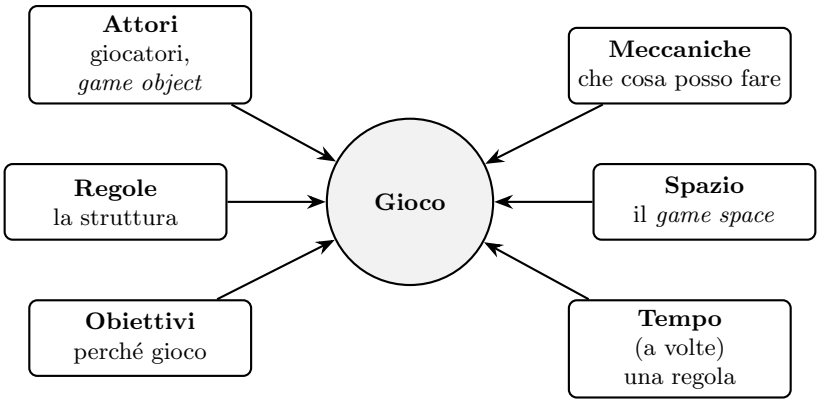
Table 1.1.1: Le aree di una produzione e il loro rapporto con l'engine, come le presenta Igor. Dentro ogni area i ruoli si distinguono anche per anzianità: junior, mid, senior.

Prima risposta, e una correzione di rotta. La prima risposta è da *game designer*: l'idea, il prototipo su cui si basa la struttura, le meccaniche di gameplay, la parte artistica, il concept art, il sonoro. Igor la accoglie ("ok, ok, ok"), ma la riorienta: non i componenti di una produzione, ma gli elementi di un gioco qualunque. "Facciamo acchiapparello" (o "ce l'hai", come lo chiama qualcuno): perché si chiama gioco? "Proprio una parola, ragazzi."

Le risposte della classe. Arrivano una per volta.

- Le **regole**: "è formato da una serie di regole". "Bravo, regole." Quindi una struttura.
- Il **divertimento**. Igor lo ridimensiona: il divertimento è "una conseguenza".
- L'**interazione**, un modo di interagire col gioco; e, detto "in un altro modo, un po' filosofico", i **giocatori**: "se non c'è nessuno che gioca…". "Bravissimo!" I giocatori sono gli **attori**: è il termine che si ritroverà in Unreal (gli *actor*), dove gli attori sono "gli oggetti di gioco". Altri li chiamano *game object*, "ognuno li chiama a modo suo". Ci sono anche gli elementi sociali: la **socialità** "è comunque gameplay", nei giochi da festa come negli MMO (*World of Warcraft*), che si basano "tantissimo" sulla componente sociale.
- Dalla chat: *Homo Ludens* ("bello", commenta Igor), e l'osservazione che la definizione è complessa, perché "molti giochi fanno anche a meno" di certe componenti[9]. "Giustissimo": sono definizioni su cui anche nel panorama dello sviluppo "c'è sempre un'opinione un po' diversa".
- Un **obiettivo**: "bravissimo". È il movente, "perché giochiamo, alla fine", ed è una cosa "molto psicologica".
- Le **meccaniche**, cioè i modi di fare: "io sparo o salto". Con le meccaniche definiamo la tipologia di gioco: *souls-like*, *roguelike* sono "macro termini" che raffigurano che cosa si può fare in un gioco, e anche le sue regole.
- Lo **spazio**. Igor lo cerca con una domanda ("quello che abbiamo intorno a noi"; "l'ambiente di gioco?"; "in che senso?"), finché arriva la parola: lo spazio, il ***game space***. Acchiapparello in un parchetto o in spiaggia non è lo stesso gioco; nel calcio "c'è il fuorigioco perché c'è lo spazio che lo definisce". Lo spazio di gioco è la mappa, il corridoio che si percorre.
- Il **tempo**, "volendo": anche il tempo può essere una regola ("devi fare questa cosa entro un tot, devi scappare").

[9] Lo studente cita un esempio che la trascrizione rende con "Three Monkeys"; Igor ammette di non conoscerlo.



Il **divertimento** non è un elemento ma una *conseguenza*; le **scelte** che questi elementi permettono creano il gameplay.

Figure 1.1.3: Gli elementi di un gioco emersi dall'attività in classe. Il termine "attori" anticipa gli *actor* di Unreal.

L'*escape room*. Igor ha lavorato in una *escape room* ("ragazzi, io ci ho lavorato"), e la propone come caso: anche l'*escape room* è un gioco, perché ha tutti questi elementi. "Definitemi attori, meccaniche, spazi, obiettivi e regole; uno per volta, se no ci sovrapponiamo." Risponde una studentessa che ha esperienza da *game master*:

- bisogna intanto **stabilire i confini**, "una cosa molto importante": "molto spesso le persone arrivano senza regole e distruggono cose, rovinano la scenografia". Igor lo chiama il ***behavior***: "non fare la persona brutta";
- bisogna stabilire una **chimica di gruppo** e dei **ruoli**: c'è sempre quello "più scafato", che cerca in giro ed è veloce, e quello che ci mette tanto "ma poi arriva alla fine dell'enigma". I ruoli fanno la forza del gruppo, perché ognuno sa che cosa deve fare;
- bisogna **non perdere tempo**, concentrarsi sulle cose giuste, "ma senza farsi prendere dall'ansia", "perché è pur sempre un gioco".

La scelta crea il gameplay. Igor sottolinea l'ultimo punto: "bisogna gestire l'ansia, bisogna capire che cosa è importante". Il gioco "sta anche lì", nella **capacità di scelta** del giocatore. Ad acchiapparello si può giocare nascondendosi e aspettando che qualcuno arrivi per fargli "lo sgambettino", oppure inseguendo tutti "a prescindere", facendo "un *randomizer*", andando "a cavolo". Questo insieme di cose crea scelta, e quindi crea **gameplay**. Il gameplay è spesso personalizzabile negli spazi che lo sviluppatore concede.

Lo spazio oltre lo schermo. Lo spazio di gioco è lo schermo, la mappa; ma lo si può anche allungare, rompendo la quarta parete. Gli **ARG** (*alternate reality game*[10]) sono giochi che escono dal gioco: per aprire una porta bisogna andare a cercare la risposta su un sito internet. È uno spazio di gioco che "si è esternato".

***Expedition 33*: perché si gioca?** Poi un videogioco. Igor propone *Expedition 33* ("ve la butto lì"; "eh vabbè, non vale, è il mio preferito"), abbastanza fresco per tutti, insieme a un secondo titolo scelto da una studentessa[11]. La domanda è: *perché* si gioca? La studentessa risponde per *Expedition 33*: l'ha iniziato dopo i trailer, colpita dalla qualità della *motion capture* ("alcune volte mi sembra di vedere un film"), e perché ha "un debole per i giochi strazianti" ("quando ho capito che avrei sofferto, ho detto: ecco il gioco per me"). Ma è rimasta stupita dal combattimento: ha giocato tantissimo a *Final Fantasy X*, che ama per la storia,

[10] Nella trascrizione "Artero Diperiale di Game".

[11] La trascrizione lo rende con "Barbarschettere" e "Barbara Street 3": potrebbe trattarsi di *Baldur's Gate 3*, ma la ricostruzione non è sicura.

ma non è un'amante dei giochi a turni, e trova invecchiato il combattimento a turni di quel *Final Fantasy*. *Expedition 33* l'ha trovato "molto fresco": ha rinnovato una cosa "invecchiata male", con un gameplay "che non è perfetto" ma che ha portato a un gioco a turni anche chi li odiava. In sintesi: si gioca per la bellezza del gameplay, la ricchezza dell'arte e la trama.

Un gioco per tutti non è un gioco per nessuno. Igor generalizza: ognuno porta nel gioco "una sua ricerca personale", dentro il proprio "spettro psicologico". Sulla slide c'è una frase attribuita ad Arrowhead Game Studios con cui Igor è "molto d'accordo": **un gioco per tutti non è un gioco per nessuno**. Se si volesse accontentare tutti si finirebbe a gestire "tutte queste cose", come nella vita vera. Si gioca per la trama, per una relazione sociale, "per sentirmi figo" ("gioco a LoL e dico: tu fai schifo perché non hai fatto la *kill*"), per vedere il punteggio che sale e ritrovarsi in classifica.

Obiettivo e piccoli obiettivi. Quando si parla di obiettivi, chiarisce una studentessa, non si intende lo scopo del gioco ma "ciò che porta il giocatore a iniziare e continuare". Igor è d'accordo e lo articola: c'è l'**obiettivo effettivo** (in *Dark Souls*, ravvivare la fiamma) e c'è il percorso, fatto di **piccoli obiettivi** che possono intrecciarsi con cose personali: "adesso vai alla campana"; e come ci vai? *Build* forza, *build* destrezza, *build* mago. "Hai un obiettivo; il modo in cui lo fai è tuo, giocatore."

Meccanica, dinamica, estetica. Quel "modo" ha un nome: la **dinamica**. Un gioco, "molto riassunto", è formato da **meccaniche, dinamiche ed estetiche**.

- La **meccanica** è "quello che posso fare": dare spadate, attaccare.
- La **dinamica** è come lo faccio: "aspetto o attacco adesso? Aspetto oppure evito?".
- L'**estetica** è "come mi fa sentire fare questa cosa". Si gioca il mago perché è "intelligente": si sta a distanza, un gameplay *ranged*, la situazione sotto controllo, il nemico che non può avvicinarsi. Si gioca forza (Igor "va proprio di forza bruta") perché si vuole sentire l'impatto dei colpi, il martellone "di due metri e mezzo" che schiaccia un nemichetto, e il **numerone** del danno ("bravissimo, il numerone!"). E il numerone, a sua volta, è qualcosa da mostrare agli amici: "costruisco una componente sociale".

È "un iter continuo": la creazione di spazi personali, dettati però da obiettivi precisi.

Approfondimento: il modello MDA. La terna di Igor è il modello **MDA** (*Mechanics, Dynamics, Aesthetics*) di Hunicke, LeBlanc e Zubek (2004). Nel modello originale le tre parti sono viste da due lati opposti: il designer costruisce le *meccaniche* (regole e componenti), dalle quali, quando il gioco è in funzione, emergono le *dinamiche* (il comportamento del sistema e dei giocatori), che a loro volta producono le *estetiche* (le risposte emotive di chi gioca). Il giocatore fa il percorso inverso: incontra prima le emozioni, poi capisce le dinamiche, e solo alla fine le regole. È per questo che, come dice Igor, il designer controlla direttamente solo "quello che posso fare", e l'emozione la può solo progettare indirettamente.

1.1.11 Le domande prima della pausa

Alle cinque Igor chiede due cose: scaricare l'Epic Games Launcher, se non lo si è già fatto ("per chi già smanetta un po', è la 5.5 che ci serve"), e se ci sono domande. La classe preferisce farle subito.

Il tempo è davvero un elemento del gioco? La prima domanda era rimasta in chat (Filippo): il tempo non fa *obbligatoriamente* parte del gioco; ci sono momenti in cui bisogna fare un'azione entro un certo tempo, ma ci sono anche giochi che non pongono limiti. "Sì", risponde Igor: il tempo **può** essere una regola, e dipende dal designer, di solito il *senior* o il *lead* che definisce il *pattern* di regole. Gli esempi di giochi in cui il tempo è centrale si moltiplicano, tra Igor e gli studenti:

- *The Legend of Zelda: Majora's Mask*: se non si fa una cosa entro un certo tempo, la luna si abbatte sul villaggio;

- *Lightning Returns: Final Fantasy XIII*, lo *spin-off* in cui Lightning deve fare tutto entro un lasso di tempo, altrimenti il gioco ricomincia;

- *Metal Gear Solid 3*, in cui Kojima[12] ha previsto che si possa battere un *boss* (The End) facendo avanzare di una settimana il calendario;

- *The Blood of Dawnwalker*[13], citato da uno studente: un gioco in uscita, di ex sviluppatori di *The Witcher*, a tema vampiresco, con il ciclo giorno–notte. Le *quest* fanno avanzare il tempo, e a seconda di come lo si fa avanzare cambia l'interazione con il livello: di giorno si è umani, di notte si usano i poteri vampirici. E c'è un limite: la premessa è finire la trama in trenta giorni, come in *Majora's Mask*;

- *Persona 5*, con il calendario: se non si completa un palazzo entro un certo tempo, "ciao".

Il tempo può anche creare dinamiche **sociali** ("un amico diceva: io ho 4000 ore su questo gioco"). Igor riprende poi una formula di uno studente: **gestione del tempo**. Il tempo c'è "perché lo dobbiamo gestire"; è sempre un'interazione. Lo si può **seguire** (i giochi ritmici, come *Hi-Fi Rush*), **controllare** o **sfruttare**: Tracer, in *Overwatch*, torna indietro nel tempo ("relativo al gioco, ovviamente, e a lei"); *Life is Strange* fa compiere un'azione, mostra l'esito, e poi chiede di scegliere. "Il gameplay sta sulla scelta": tornare indietro nel tempo serve sempre al *game loop* centrale, che è scegliere; e "la bellezza è che le scelte sono grigie".

Chi si occupa del bilanciamento? Seconda domanda dalla chat. Il bilanciamento è di solito una **stretta collaborazione tra designer e QA tester**. Il QA tester "è proprio giocare al gioco, però non è solo giocare al gioco": è giocarlo in modo molto preciso. Igor ha fatto il QA tester, e racconta di una collega che scoprì come saltare tutti i livelli: dando una spadata in avanti si generava una forza in diagonale che permetteva di arrampicarsi su qualunque superficie. Un buon feedback di QA è un feedback da cui si possono prendere informazioni: "questo combattimento mi sembra molto facile, perché mi sono avvicinato molto facilmente a un nemico *ranged*". Quale informazione si collega? La capacità del nemico di identificare il giocatore e sparargli. E così via, "una continua iterazione". Nelle produzioni più piccole è il designer stesso a giocare il gioco; ma la cosa migliore, "molto spesso", è far giocare chi non l'ha mai visto, "perché è la cosa più pura" ("ovviamente dicono: non mi piace").

Come si fa un gioco in stop motion? Una studentessa chiede di *The Midnight Walk*, un indie "molto artistico", fatto in stop motion e "molto artigianale". Igor ammette di non sapere precisamente come sia stato fatto ("devo dire la verità"), ma spiega il principio. Unreal permette di sperimentare. Si possono **scannerizzare** i modellini e le animazioni: si muove il modellino, lo si scannerizza fotogramma per fotogramma, si compone l'animazione e la si mette nell'engine. Cita anche *Harold Halibut*, un'avventura grafica fatta tutta in stop motion, e *Cuphead*[14], fatto in Unity, in cui i disegni a mano sono stati scansionati, digitalizzati e composti in animazioni. Il punto è che il game engine **unisce** componenti fatte altrove (con Blender, Maya, Cinema 4D, Marmoset...) e ci costruisce sopra il *pattern* di regole ("adesso abbiamo un'animazione, muoviti da qui a qui"). Nella stop motion la qualità non dipende dalla tecnica dell'engine ma da quella dell'animazione: sono "due compartimenti differenti", uno che compone e raggruppa, l'altro che "crafta" tutte le "cosette belline". La studentessa è rimasta male che il gioco sia stato candidato solo come migliore esperienza VR[15]: secondo lei meritava anche per la colonna sonora.

Il tempismo delle uscite. "Quando esce un gioco", commenta Igor, il momento è un dato importantissimo. Un gioco può uscire in concomitanza, per esempio, con il remaster di *Oblivion* di Bethesda, "droppato" senza preavviso, e questo pesa sulle vendite e sul successo. Anche *Expedition 33* è arrivato al momento giusto: un combattimento a turni di quel livello "non c'era da un po"'. Una studentessa obietta che l'anno prima c'era stato *Sea of Stars*, e Igor le dà ragione.

[12] Nella trascrizione "Cugino".
[13] Nella trascrizione "The Blood of the Walker".
[14] Nella trascrizione "KPED"; il gioco disegnato a mano e sviluppato in Unity è *Cuphead*.
[15] La trascrizione dice "migliore esperienza in War".

1.1.12 Scaricare Unreal Engine 5.5

Prima della pausa, il download. Dal launcher di Epic si va nella sezione Unreal Engine, nella scheda **Libreria**, e accanto a **Versioni del motore** si clicca sul + per aggiungere una versione; dal menu a tendina si sceglie la versione. Qualcuno vede di default la 5.6, la 5.7 o la 5.8 *preview*: per il corso serve la **5.5**.

5.5 o 5.5.4? "L'ultimo numero non è importante": è solo l'aggiornamento. Conviene prendere la **5.5.4**, che è l'ultima, ma ciò che conta è il **5.5**. Un progetto fatto in 5.5.4 si può aprire anche in una 5.5.2; passare invece da 5.5 a 5.3 è "molto differente", e lì si possono vedere degli errori. Il punto è la **compatibilità**.

Approfondimento: come si leggono le versioni. Il numero di versione di Unreal segue lo schema *major.minor.hotfix*. Il primo numero (5) è la generazione dell'engine; il secondo (5.5, 5.6...) è una versione con nuove funzionalità e cambiamenti nel formato dei file; il terzo (5.5.4) è una correzione di errori che non cambia il formato. Per questo un progetto si sposta senza problemi tra 5.5.2 e 5.5.4, mentre un progetto salvato in una versione *minor* più recente in genere non si apre in una più vecchia. Il launcher permette di tenere installate più versioni nello stesso computer e di scegliere quale avviare.

1.1.13 Dopo la pausa: dove cercare aiuto

Dopo la pausa (e dopo il chiarimento sull'orario: la lezione finisce alle 19, non alle 18), la seconda metà comincia dalle **risorse**: "quando vi mettete le mani nei capelli e dite: non so più cosa fare". Oltre all'"amatissimo" YouTube, dove si scrive il proprio problema e "di solito" si trovano risposte buone, c'è la fonte "più centrale".

La documentazione. La **documentazione** di Unreal è "un palo a cui aggrapparci": "molto, molto esaustiva" e "molto spesso ottima". Igor fa una ricerca di esempio, **Branch**: il nodo che corrisponde all'*if* della programmazione, "il nostro bellissimo *if/else*, quindi vero o falso". Ci sono poi i **forum**, dove si fanno domande e si ricevono risposte, e i problemi "triti e ritriti": "il problema che un tizio di Reddit ha avuto cinque anni fa, adesso lo avete voi", e la sua soluzione risolve anche il vostro.

I requisiti minimi. Una studentessa chiede di mandare sul Discord una foto dei requisiti minimi, e Igor lo fa nella chat generale. Un portatile o un PC "abbastanza medio", "di questi tempi", dovrebbe reggere Unreal, anche se è "leggermente sotto i requisiti minimi": "provate". Ha visto PC "cavalcare tranquillamente" e altri che "hanno crashato e basta": "sempre un terno al lotto". Più avanti, mentre l'editor si apre, aggiungerà che la componente da tenere d'occhio è la **scheda grafica**, su cui Unreal pesa "piuttosto" come i giochi, e che a volte l'*upgrade* è "un po' obbligatorio".

Il *learning*. Nella sezione di apprendimento del sito di Epic ci sono "tante, tante cose" per il mondo artistico, la programmazione, gli effetti visivi ("molto, molto carino"): tutti **corsi gratuiti**, piccole lezioni come *scatter on mesh* o *environment basics* (più per chi fa 3D art), o **Niagara**, il sistema di particelle. Niagara nel corso "forse" si vedrà: le basi di programmazione "sicuramente sì". Ci sono poi la guida introduttiva, i tutorial, l'assistenza agli sviluppatori. "Però siamo qua per questo, quindi non vi preoccupate."

Fab, il magazzino dei desideri. **Fab** è il negozio di **asset** di Epic, "il nostro magazzino dei desideri". Un *asset* è, in generale, qualunque cosa possa essere inserita in un game engine: modelli 3D, animazioni, *texture*, decalcomanie (*decal*), pennelli, e addirittura interi sistemi di gioco.

- **Gli *asset* gratuiti del mese.** "Una cosa molto importante, veramente importante": all'inizio di ogni mese, nella sezione delle offerte, Fab regala dei *pack*. "Prendeteli sempre": si aggiungono alla propria libreria e "sono vostri per sempre". Quel mese c'erano, per esempio, uno *scan* (un oggetto reale scannerizzato e digitalizzato), *asset* modulari e un *pack* di vetro temperato e rotto.
- **Più engine.** Su Fab ci sono *asset* per vari engine, anche per Unity; a noi interessano quelli per Unreal Engine e, "se mai doveste toccarlo", per Unreal Editor for Fortnite.

- **MetaHuman.** Una sezione "molto interessante" per chi fa *character design*: MetaHuman è lo strumento di Unreal per creare volti umani realistici, e su Fab ci sono *asset* per personaggi, vestiti, persone.
- **Tutto il resto:** materiali, *texture*, *sprite*, *flipbook* ("libri animati"), animazioni, audio, effetti. "Di tutto, di più."
- **Il filtro "gratis".** Oltre alle offerte del mese, filtrando per prezzo ci sono "tanti, tanti" *asset* gratuiti da usare nei propri piccoli progetti.

Prima la funzione, poi l'*asset*. Un consiglio importante su Fab. Non bisogna fare come al *Black Friday*: "ficca nel carrello e via", e poi "metti nel progetto". "Meglio prima creare la funzione di qualcosa, rispetto a mettere subito l'*asset* bello, carino e carinissimo": altrimenti ci si ritrova con un progetto da 70 GB, di cui se ne usano tre. Si prende ciò che serve.

Il materiale didattico. Agli studenti arriverà (via Discord, con un link a una cartella su Drive) il **materiale didattico** del corso: una raccolta di *asset* e un progetto Unreal da cui si partirà nelle lezioni successive. Igor manda il link con l'accesso già abilitato, per evitare le richieste di autorizzazione ("come se bussasse alla posta").

I giochi gratuiti. "Piccola cosa": nel launcher ci sono anche i giochi gratuiti del mese, "se li volete accaparrare".

1.1.14 Un po' di teoria: il *Play in Editor* e il tempo di gioco

Prima di aprire l'editor, "una piccola cosa più teorica", per "instaurarla in testa".

Che cosa succede quando si preme Play. Quando si avvia il gioco dentro l'editor (il ***Play in Editor***, PIE) succedono tante cose una dopo l'altra. Gli **attori** (*actor*), i "nostri *game object*", vengono inizializzati nello spazio di gioco, e vengono **duplicati**: si crea una copia degli attori del livello per la sessione di gioco. Si caricano le mappe, si caricano tutti gli attori oltre al personaggio del giocatore, si inizializzano, e parte il ciclo delle interazioni. Gli attori sono di diversi tipi: Igor nomina gli **actor** "normali", i **pawn** e i **character**, che si vedranno più avanti.

Il *delta seconds*. Poi la regola che Igor chiede di tenere "bella in testa", anche se "ancora non abbiamo qualcosa di oggettivo davanti": quando si crea qualcosa che produce un **movimento** nel gioco, deve essere **relativo al tempo di gioco**, che in Unreal si chiama ***delta seconds***. Nell'editor "tutto quanto ha un suo tempo", definito soprattutto dagli ***fps***: c'è il nostro tempo e c'è il tempo dell'editor. Se un movimento che dovrebbe durare quattro secondi è legato ai *frame* e non al tempo, con *fps* più bassi durerà di più: il tempo, in quel caso, non è relativo "a queste cose". È un concetto generale che il corso riprenderà.

Approfondimento: perché si moltiplica per il *delta time*. Romero e Sewell lo spiegano nel quinto capitolo del libro, a proposito di un bersaglio che si muove: il ***delta time*** è la differenza di tempo tra il *frame* precedente e quello attuale, e serve perché il tempo tra un *frame* e l'altro può variare. Moltiplicando la velocità per il *delta time* si ottiene che l'oggetto si muova alla stessa velocità qualunque sia il *frame rate*. Un esempio numerico chiarisce il problema di cui parla Igor. Se un cubo si sposta di 5 cm a ogni *frame*, a 60 *fps* percorre 300 cm al secondo, a 30 *fps* solo 150: sullo stesso percorso, su un computer più lento, ci metterà il doppio del tempo. Se invece lo spostamento per *frame* è

$$\Delta s = v \cdot \Delta t, \qquad v = 300 \text{ cm/s},$$

a 60 *fps* ($\Delta t \approx 0{,}0167$ s) il cubo si sposta di 5 cm per *frame*, a 30 *fps* ($\Delta t \approx 0{,}033$ s) di 10 cm: in un secondo, in entrambi i casi, percorre 300 cm. Nei Blueprint il valore si ottiene dal pin Delta Seconds dell'Event Tick, o dal nodo **Get World Delta Seconds**.

1.1.15 Il primo progetto: il Project Browser e i template

Con l'installazione completata (chi è ancora al 50 o al 90% "per adesso segue"), si apre Unreal Engine. Oggi non si inizializza ancora il progetto del corso, che arriverà con il materiale didattico: si crea un progetto di prova solo per vedere le funzioni di base.

Le categorie. La prima finestra è il **Project Browser**, con una colonna a sinistra: i **progetti recenti** e le categorie di progetto, che dipendono da ciò che si vuole fare: **Games**, **Film/Video & Live Events** e così via. Si sceglie **Games**.

I template. Dentro **Games** ci sono vari tipi di progetto: **Blank**, **First Person**, **Third Person**, **Top Down**, **Vehicle**, **Handheld AR**[16] e **Virtual Reality**. Sono ***preset***, **template** che riprendono i modi generici in cui un gioco può essere fatto, e sono "un'ottima base da cui partire". Si distinguono soprattutto per le **3C**, "un concetto di design però molto importante": ***camera***, ***controller*** **e** ***character***. In prima persona, in terza, dall'alto, su un veicolo, in realtà virtuale: cambia la camera, e con la camera cambiano i controlli. **Blank** è vuoto: "non c'è niente".

Le impostazioni del progetto. Si seleziona **Blank** e si compilano le opzioni:

- **Project Location**: la cartella in cui salvare il progetto; meglio crearsene una apposta ("Unreal Engine Projects", "come volete");
- **Project Name**: per esempio "Prova_1". Attenzione agli **spazi**: Unreal non li accetta nei nomi, e la convenzione, anche per le cose interne al progetto, è separare le parole con il trattino basso (*underscore*);
- **Blueprint** oppure **C++**: si sceglie **Blueprint**;
- **Target Platform**: **Desktop**, non Mobile;
- **Quality Preset**: **Maximum** "va benissimo";
- **Starter Content**: attivo. Lo *Starter Content* aggiunge al progetto degli *asset* "proprio basici": forme geometriche, cose molto semplici.

Poi **Create**. Chi oggi può solo guardare (uno studente lo scrive in chat) recupererà dalla registrazione: "adesso parliamo solamente di navigazione dentro l'engine".

Approfondimento: i template e le opzioni secondo il libro. Romero e Sewell descrivono i template come "contenitori di base con alcuni file chiave e un livello", utili per prototipare in fretta o per imparare le meccaniche di base di un tipo di progetto; il loro uso è **facoltativo**, perché ogni template si può ricostruire partendo da Blank. Il *First Person* ha un personaggio rappresentato da due braccia con un'arma che spara un proiettile sferico; il *Third Person* un personaggio con una camera che lo segue da dietro e un po' più in alto, con animazioni di camminata, corsa e salto; il *Top Down* un personaggio guidato col mouse e una camera molto in alto, la vista tipica dei giochi di ruolo d'azione; il *Vehicle* un veicolo semplice e uno con sospensioni su un piccolo circuito; il *Virtual Reality* il teletrasporto, oggetti da afferrare e una camera per lo spettatore; l'*Handheld AR* la logica per attivare la realtà aumentata su Android e iOS. Tutte le opzioni del progetto (Blueprint o C++, piattaforma, qualità, Starter Content) si possono **cambiare anche dopo**; e un progetto Blueprint può ricevere codice C++ in qualunque momento. Il libro, per i suoi esempi, sceglie la qualità *Scalable* anziché *Maximum*: disattiva alcune funzioni complesse ed è più leggera, il che può essere utile su un computer al limite dei requisiti. Quanto ai nomi: il Project Browser accetta per il nome del progetto solo lettere, numeri e trattino basso.

[16] Nella trascrizione "Underhanded" e "AR".

La compilazione degli *shader*. Al primo avvio Unreal compila gli ***shader***: "abituatevi". Uno studente ricorda la **pre-compilazione degli shader** che fanno molti giochi al primo avvio; Igor conferma che "ci mettono un casino di tempo", perché gli *shader* sono ciò che dà forma alla visualizzazione, "la cosa più pesante". Una studentessa racconta di aver passato "un brutto quarto d'ora" aprendo *Final Fantasy XVI*, quindici minuti per gli *shader*, e Igor scherza: "se qualcuno inventerà un metodo per renderli più leggeri, probabilmente diventerà ricco". Mentre le barre avanzano ("42% di compilazione"; un'installazione che resta ferma allo 0% e poi "ha deciso di andare su"), la conversazione scivola su *Final Fantasy VII* (originale e *remake*) e sull'attesa di *Kingdom Hearts 4*, e sul fatto che gli autori si riconoscono anche dal modo in cui gestiscono storia e meccaniche.

Approfondimento: che cos'è uno *shader*. Uno ***shader*** è un piccolo programma che gira sulla scheda grafica e realizza le fasi della figura 1.1.1: il *vertex shader* trasforma i vertici, il *pixel* (o *fragment*) *shader* calcola il colore di ogni pixel. In Unreal ogni materiale diventa uno o più *shader*, e ognuno va tradotto ("compilato") nel linguaggio della scheda grafica in uso, in molte varianti diverse a seconda delle situazioni in cui il materiale viene usato. È per questo che la prima apertura di un progetto, o la prima volta che si avvia un gioco su un nuovo computer, richiede tanto tempo; le volte successive l'engine riusa i risultati salvati.

1.1.16 L'interfaccia dell'editor

L'editor, appena aperto, fa l'effetto della cabina di un aereo: "oh mio Dio, che cosa devo fare, non voglio morire". Ci sono tante finestrelle; ma Unreal **è fatto a finestre**, e ognuna ha il suo nome e il suo compito (figura 1.1.4).

Le finestre si spostano. Le finestre si possono prendere e spostare come si vuole: si possono mettere una dentro l'altra (Igor trascina **Place Actors** in un altro pannello e passa dall'una all'altra con le schede), stringere, allargare, "e vedere un po' come mi piace di più".

La viewport. Al centro c'è la **viewport**: "quando parleremo di viewport, è questa qua", cioè la visualizzazione dello spazio in 3D. Tutto ciò che le sta attorno sono schede con un nome: **Details**, **World Settings**, **Outliner**, che si vedranno una per una.

Il Content Browser. La parte più importante per mettere e creare cose è il **Content Browser**, il sistema di navigazione dei file interni del progetto. Igor ce l'ha in basso.

Il menu Window. Se una finestra manca, la si recupera dal menu **Window**, che elenca tutte le schermate disponibili: un secondo Content Browser, i Details, l'Outliner, la viewport ("potete mettere anche due viewport"). Se per errore si chiude una finestra, per esempio i Details ("oh, porca miseria"), la si riapre da lì: "*don't panic*".

Le cartelle, e i loro colori. Nel Content Browser, con il **tasto destro**, si possono creare vari tipi di *asset*; per cominciare, una cartella: **New Folder**, chiamata "prova". Si possono creare tutte le cartelle che si vogliono, "in modo da avere ordine". Una "piccola chicchetta": con il tasto destro su una cartella, **Set Color**, la si può **colorare** (Igor la fa blu), un feedback visivo che rende la navigazione molto più veloce. Il Content Browser mostra le cartelle in due modi: a destra come in un *explorer*, a sinistra come un albero "a cascata". Se si mette una cartella dentro un'altra, a sinistra compare una piccola freccia che apre la cartella sottostante: cartelle e sottocartelle.

L'ordine, per sé e per gli altri. "È molto importante l'organizzazione." Le cartelle si riempiranno di *asset*, e bisognerà cercarne uno solo senza sapere dove andare. L'ordine serve "non solo per la salvezza vostra, ma anche dei vostri possibili collaboratori": "vi assicuro che quando siete ordinati, la gente vi ringrazia".

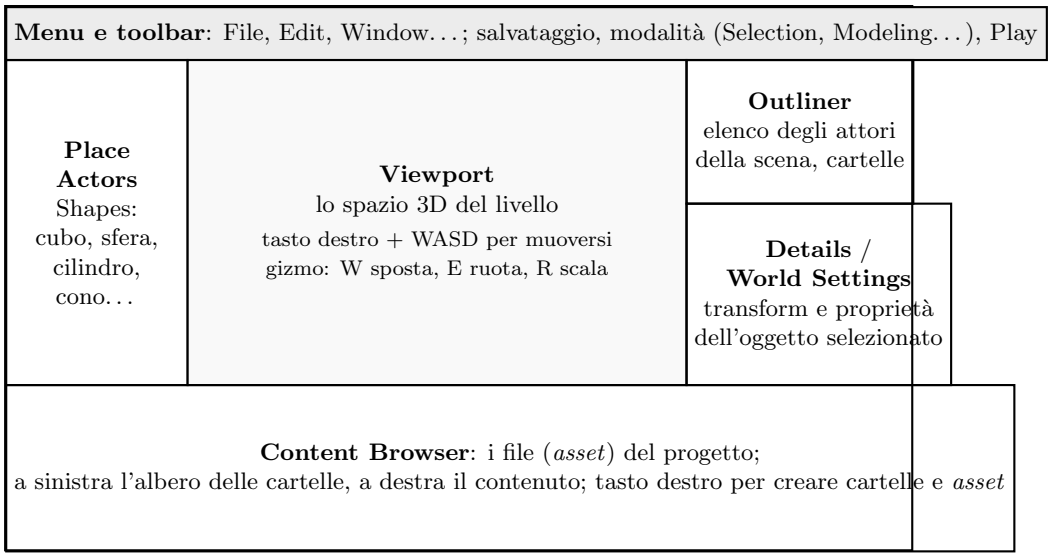


Figure 1.1.4: Una disposizione tipica dell'editor, come quella mostrata da Igor. Ogni pannello si può spostare, agganciare ad altri o chiudere; il menu Window li riapre.

Approfondimento: il Content Drawer e la definizione di *asset*. Nella disposizione predefinita di Unreal Engine 5 il Content Browser non è sempre aperto: lo si richiama dal pulsante **Content Drawer** in basso a sinistra (o con Ctrl+Spazio), e lo si può agganciare stabilmente all'interfaccia ("Dock in Layout"), come ha fatto Igor. Romero e Sewell definiscono l'*asset* come "un pezzo di contenuto di un progetto Unreal": materiali, *static mesh* e Blueprint sono tutti *asset*. Quando si trascina un *asset* dal Content Browser nel livello, l'editor ne crea una **copia** da piazzare nella scena: l'*asset* resta uno, le sue istanze nel livello possono essere molte. Gli stessi autori descrivono l'Outliner come l'elenco degli oggetti presenti nel livello, e i Details come il pannello che mostra le proprietà modificabili dell'oggetto selezionato.

1.1.17 Il primo cubo e il *transform*

Place Actors. Gli *actor* sono gli elementi di gioco. Dal pannello **Place Actors** (se manca: Window, Place Actors), nella sezione **Shapes**, si prende un **cubo** e lo si trascina nella viewport. "Ecco a voi un cubetto renderizzato": il primo cubo in Unreal, "e ne avrete tanti".

I tre assi. Sul cubo compaiono tre frecce: sono i tre assi principali, **X**, **Y** e **Z**, lungo i quali lo si può muovere. Ogni asse ha il suo colore: **X rosso**, **Y verde**, **Z blu**.

Il *transform*. Di ogni oggetto si possono modificare **locazione**, **rotazione** e **scala**. L'insieme dei tre è il ***transform*** ("quando parleremo di *transform*, ricordatevelo"), e lo si rivedrà "a oltranza". Lo si modifica in due modi: con le frecce nella viewport, o con i numeri nel pannello **Details**.

I numeri nei Details. Nei Details la locazione del cubo è, per esempio, 480, 430, 120: è la **raffigurazione numerica**, in unità di Unreal, del punto in cui si trova l'oggetto. Se si scrive 0 al posto del primo valore e si preme Invio, "poplà": la coordinata X del cubo adesso è 0. Se si trascina una freccia guardando i numeri, si vede il valore crescere o diminuire, andare in positivo o in negativo: è il verso dell'asse.

Unreal ragiona in centimetri. "Molto importante": **l'unità di Unreal è il centimetro**. Se un oggetto va spostato di tre metri, lo si sposta di 300 unità. Nella griglia del pavimento ogni riquadro, dice Igor, è un metro, e un cubo, di default, è un metro.

W, E, R. Il **pivot** (o *gizmo*) è "questo cosettino degli assi", e permette tre movimenti: con **W** si sposta (locazione), con **E** si ruota, con **R** si scala. Gli stessi tre strumenti sono anche in alto a destra nella viewport, come icone.

Assi locali e assi globali. Esistono due riferimenti: gli assi **relativi** all'oggetto stesso (**locali**) e quelli del **mondo** (**globali**, la sua *absolute location*). Finché il cubo non è ruotato la differenza non si vede; ma se lo si ruota fino a farlo diventare "una specie di rombo" e si infila nel terreno, alzarlo lungo i suoi assi locali richiede "un doppio movimento, così e così", perché gli assi locali ruotano insieme a lui ("avremo il nostro bel pivot ruotato di 40 gradi"). Con gli assi globali, invece, l'alto è sempre l'alto. Il passaggio dall'uno all'altro si fa con l'icona in alto nella viewport: **il cubetto con tre assi** indica il riferimento **locale**, **il mondo** indica quello **globale**. La scelta vale per locazione e rotazione.

La scala è solo locale. Non vale per la scala: **un oggetto si può scalare solo nei suoi assi locali**. Se si scalasse un oggetto ruotato lungo gli assi del mondo, "verrebbe totalmente stretchato malissimo": l'oggetto è girato in un modo e l'asse in un altro, e allungarlo così lo deformerebbe. Unreal "ce lo impedisce".

Le unità di misura del *transform*. La locazione è in centimetri, la **rotazione in gradi** ("io l'ho ruotato e c'è un 40"), la **scala in unità**: 1 è la dimensione originale, 2 il doppio.

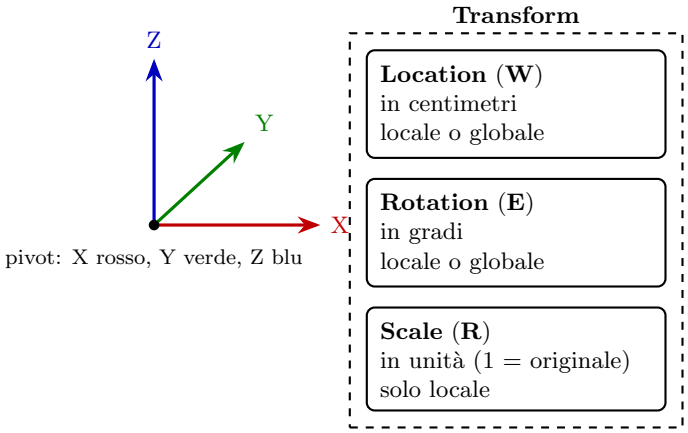


Figure 1.1.5: Gli assi e il *transform*. Lo *snapping* (sezione 1.1.19) si imposta separatamente per ognuna delle tre componenti.

Il tasto Fine. Un'altra "piccola chicchetta": se si piazza un cubo in aria e si preme il tasto **Fine** (*End*, quello "sopra le frecce"), il cubo cade sulla prima superficie che trova sotto di sé. "Molto utile", perché evita di stare lì ad abbassarlo a occhio.

I Details seguono la selezione. Ogni volta che si clicca un oggetto, i Details mostrano *quell'*oggetto: due cubi hanno due locazioni, due rotazioni, due scale. **Per modificare i parametri di un oggetto, bisogna selezionarlo.**

Lavorare con i numeri. I numeri dei Details sono lo strumento preciso, e sono molto usati, per esempio, dai level designer: "dobbiamo allargare il corridoio di tre metri". Igor allunga un cubo di tre metri sull'asse Y. Nei campi numerici si può anche scrivere, oppure **trascinare**: si mette il mouse sul campo e lo si fa scorrere a destra e a sinistra ("slidando") per cambiare il valore, su qualunque asse, per locazione, rotazione e scala.

Reset. Accanto a ogni gruppo di valori c'è una **freccetta**: cliccandola, il valore torna a quello originale (la scala torna a 1, la locazione al punto di partenza).

1.1.18 Muoversi, duplicare, raggruppare

Forme e pivot. Si possono piazzare tanti *actor* diversi: cilindri, coni, sfere. Il pivot è al **centro** di ogni figura, e per questo un cilindro, appena piazzato, è "mezzo interrato". Per **scalare su tutti gli assi insieme** si prende il centro del gizmo di scala e si trascina verso l'esterno (allarga) o verso l'interno (stringe); trascinando una sola maniglia si scala su un solo asse.

Duplicare con Alt. Igor costruisce un tavolino con un cilindro schiacciato e uno allungato. Serve un altro pezzo uguale: bisogna prendere un altro cilindro e rifare tutto? No: **si tiene premuto Alt e si trascina** (o si ruota) l'oggetto, e se ne crea una copia esatta. "Lo userete a manetta." Funzionano anche **Ctrl+C** e **Ctrl+V**. Una studentessa, abituata a Blender, si confonde: premeva Alt e lo lasciava. Alt va **tenuto premuto mentre si trascina**, "non è solo *tap* Alt".

Muoversi nella viewport. "Mi sono dimenticato di dirvelo, perdonatemi": per muoversi nella viewport si tiene premuto il **tasto destro** del mouse e si usano **W, A, S, D**, come in un gioco in prima persona; muovendo il mouse si ruota la camera. Uno studente scopre da solo che **E** sale e **Q** scende: "bravissimo, esatto". Bisogna cliccare dentro la viewport: è la camera dell'editor, non quella del gioco.

Seguire un oggetto. Se si tiene premuto **Shift** mentre si trascina un oggetto, la camera lo **segue** nel movimento: utile per piazzamenti particolari.

Selezionare più oggetti. Per selezionare più oggetti si tiene premuto **Shift** (o Ctrl) e si cliccano uno dopo l'altro. Con il palo e il piano del tavolo selezionati, Alt e trascinamento: "pac!", un secondo tavolino.

I gruppi. Selezionare ogni volta tutti i pezzi è scomodo, perché se si clicca il piano del tavolo si seleziona solo il piano. Si possono allora **raggruppare**: si selezionano gli oggetti, tasto destro, **Group**, oppure **Ctrl+G**. Intorno al gruppo compare "una specie di reticolo verde", e cliccando un pezzo qualunque si seleziona il tutto. Si possono anche raggruppare dei gruppi, per farne uno più grande. Per **sciogliere** un gruppo: **Shift+G**, oppure tasto destro, Groups, **Ungroup**.

Ctrl+Z. "Qua funziona": **Ctrl+Z** annulla. "Sappiatelo, ogni errore può essere cancellato, o almeno per poco."

I gruppi non sono un oggetto unico. Igor fa vedere un limite: se si **scala** un gruppo, Unreal non lo scala come un oggetto unico; ogni pezzo si scala rispettando i propri assi relativi. Per avere un oggetto davvero unico bisognerà creare una **static mesh**, "che vedremo in seguito".

Mesh e static mesh. Una **mesh** è un oggetto solido formato da poligoni: quando qualcuno dice "mi serve la mesh del tavolo" intende il **modello 3D** del tavolo. La **static mesh** è un tipo di mesh; esistono anche le **dynamic mesh**, "un discorso che vedremo più avanti".

Approfondimento: *transform* relativi nei Blueprint. Il comportamento dei gruppi anticipa un tema che nei Blueprint tornerà spesso. Romero e Sewell, nel capitolo sui nodi matematici, distinguono il ***transform* del mondo** di un attore (la sua posizione assoluta nel livello) dal ***transform* relativo** dei suoi componenti, che è relativo al componente "genitore". Il loro esempio è un tavolo con sopra una statua: se si sposta il tavolo, la statua lo segue; se si cambia il *transform* relativo della statua, il tavolo resta dov'è. È il modo in cui, dentro una Blueprint Class, si costruiscono oggetti composti che si muovono come un tutt'uno.

1.1.19 Lo *snapping* e la velocità della camera

Lo *snapping*. Ruotando un oggetto, l'editor mostra un numero che sale a scatti: 20, 30, 40, 50, 60, 70 gradi. È lo ***snap***, che si controlla in alto a destra nella viewport: ci sono tre pulsanti, per locazione, rotazione e scala, ognuno con un valore. **Se il pulsante è azzurro, lo *snap* è attivo.**

- Senza *snap* di rotazione l'oggetto ruota liberamente, con i decimali; con lo *snap* a 10, ruota di dieci gradi alla volta. Cliccando sul numero si sceglie il passo: 5, 10, 15, 30, 45, 60, 90, 120 gradi. Con 120, "pam!", diretto da 0 a 120; con 5, una rotazione più fine.
- Lo stesso vale per la **locazione**: con lo *snap* l'oggetto si muove "a scattini" (di 1, 5, 10, 50 unità...), senza si muove di qualunque valore, decimali compresi.
- E per la **scala**: senza *snap* si scala "quanto mi pare", con lo *snap* a passi anche molto piccoli, come 0,25.

Se lo *snapping* impedisce di mettere un pezzo dove si vuole, "basta disattivarlo".

La velocità della camera. Per raggiungere "quella montagna là" si può **velocizzare la camera**: mentre ci si muove (tasto destro premuto), si gira la **rotellina del mouse** in avanti per andare più veloci e indietro per andare più lenti, quando bisogna osservare un oggetto da vicino con precisione (Igor scende fino a una velocità di 0,03 per guardare sotto il tavolino).

1.1.20 L'Outliner e l'ordine della scena

L'elenco della scena. L'**Outliner** (se manca: Window, Outliner) è una raffigurazione a lista di **tutti gli oggetti in scena**. Se si duplica un cono, nell'Outliner compaiono Cone, Cone2, Cone3, Cone4... Un oggetto si può selezionare sia nella viewport sia nell'Outliner, ed è utile quando "avete un bel *pack* di roba" da gestire. Per eliminare un oggetto: **Canc** (*Delete*).

Le cartelle dell'Outliner. Anche nell'Outliner si possono creare cartelle, e a Igor "aiuta molto" nella pianificazione e nel piazzamento. Tasto destro nell'Outliner, **Create Folder**, e si crea una cartella "Coni". Poi si selezionano i coni: visto che sono elencati per nome, si clicca il primo, si tiene premuto **Shift** e si clicca l'ultimo, e si seleziona tutta la lista; si trascinano dentro la cartella. Da quel momento, invece di cliccarli uno per uno, basta il tasto destro sulla cartella e **Select All Descendants**: "eccoli qua".

Perché farlo subito. "Immaginatevi una scena intera, gigantesca." Senza ordine si passano "tre quarti d'ora" a cliccare un oggetto dopo l'altro; con le cartelle, un tasto e si è fatto. "Vi risparmiate molto tempo."

Il tasto G. Un ultimo tasto, che Igor spiega durante la dimostrazione: se il pivot "scompare", bisogna premere **G**. G passa dalla visualizzazione dell'editor, con gli aiuti visivi, a quella "normale" del mondo di gioco, in cui gli aiuti sono nascosti.

Approfondimento: la Game View. Il comando si chiama **Game View**: con G la viewport mostra la scena come la vedrebbe il giocatore, nascondendo icone, *gizmo* e oggetti che nel gioco sono invisibili (le icone delle luci, i volumi, i punti di partenza del giocatore). È comodo per giudicare l'aspetto finale di una scena; ma se lo si attiva per sbaglio sembra che gli strumenti siano spariti, ed è per questo che Igor lo nomina proprio quando il pivot "scompare".

1.1.21 Esercitazione: il ristorante dei tre tavoli

Con quello che si è imparato, "una piccola esercitazione". Bisogna costruire, con le forme di base, **tre tavoli con le loro sedie**:

1. un tavolo con **tre posti**: "vengono tre ospiti a cena";
2. un **tavolo quadrato** per **cinque ospiti**;
3. una **cena romantica**: un tavolo per **due**, "e se volete metteteci pure la candela".

Non serve una sedia precisa: "l'importante è che applicate tutto quello che abbiamo fatto". Chi non riesce oggi può farlo per la volta successiva ("assolutamente"), e riguardare la registrazione.

Comando	**Che cosa fa**
Tasto destro + W A S D	muoversi nella viewport; con il mouse si ruota la camera
Tasto destro + E / Q	salire / scendere
Rotellina (durante il movimento)	velocità della camera: avanti più veloce, indietro più lenta
W / E / R	gizmo di locazione / rotazione / scala
Icona cubetto / mondo	assi locali / globali (per locazione e rotazione)
Pulsanti azzurri dello *snap*	attivano lo *snapping* e ne scelgono il passo
Fine (*End*)	fa cadere l'oggetto sulla superficie sottostante
Alt + trascinamento	duplica l'oggetto (Alt tenuto premuto)
Ctrl+C / Ctrl+V	copia e incolla
Shift + clic	selezione multipla; nell'Outliner, selezione di un intervallo
Shift + trascinamento	la camera segue l'oggetto
Ctrl+G / Shift+G	raggruppa / separa (anche: tasto destro, Groups)
Ctrl+Z	annulla
Canc (*Delete*)	elimina l'oggetto selezionato
G	Game View: mostra o nasconde gli aiuti dell'editor
Freccetta accanto ai valori	riporta il valore all'originale
Ctrl+S	salva

Table 1.1.2: I comandi della prima lezione. Tutti valgono nel Level Editor di Unreal Engine 5.5 con le impostazioni predefinite.

Partire da una reference. Un consiglio, "soprattutto per voi artisti": per fare qualcosa di artistico, prendere una **reference**. Ma non una sedia troppo complessa: "non ci interessa fare una cosa bella e bellissima". Igor sceglie una sedia "molto semplice", la "sedia della nonna che vi fa il ragù".

La dimostrazione. Igor costruisce la sua sedia passo per passo, ed è un buon riassunto di tutta la seconda parte:

1. un cubo, scalato con lo *snap* a 0,25, per la seduta; la freccetta di reset per riportarlo alle dimensioni di partenza quando serve ("le sedie sono alte più o meno un metro");
2. la prima gamba, posizionata ("se avete problemi con lo *snapping*, basta disattivarlo");
3. le altre gambe: Shift per selezionare, Alt per duplicare. Ne viene "questo bellissimo sgabello", con le gambe "non proprio precisissime";
4. lo schienale, duplicando altri pezzi per il dorso;
5. tutti i pezzi selezionati e **Ctrl+G**, "perché sennò dobbiamo fare ogni volta uno, due, tre, quattro";
6. la sedia accostata al tavolo ("è un po' spesso questo tavolo, ci saranno delle ginocchiate incredibili, ma non ci importa").

Si può allineare un oggetto a un altro? Uno studente, che oggi può solo guardare, chiede se c'è un modo di allineare le gambe della sedia a un oggetto, visto che a mano non è semplice. Igor risponde che non è una cosa che in Unreal si fa "professionalmente": strumenti di allineamento precisi ci sono nei programmi di modellazione, come Blender e Maya. In Unreal ci sono il tasto **Fine**, lo *snapping*, il lavoro "un po' a mano", i *plugin*, oppure la **Modeling Mode**, la modalità di modellazione dell'editor, con strumenti più precisi, che si vedrà in seguito.

Gli strumenti si somigliano. Lo stesso studente nota che, in attesa del download, ha usato **Inkarnate**, un programma per disegnare mappe, e che i principi visti finora sono "molto simili". "Sì, assolutamente": sono principi condivisi dai vari game engine e dai programmi grafici. In Unity "magari qualche bottone è un po' diverso", ma le funzioni sono simili; "si impara a muoversi da una parte e si impara a muoversi

dall'altra". Igor mostra anche **Dungeon Scrawl**[17], uno strumento per disegnare mappe per le partite di D&D con gli amici, usato anche per il level design.

I risultati. Dopo una ventina di minuti di lavoro individuale, i "ristoratori in erba" mostrano i loro tavoli.

- Uno studente ha aggiunto una **luce** per la candela, ma la candela fa ombra. Igor gli mostra come risolvere: si seleziona la luce e nei Details, in basso, si toglie la spunta a **Cast Shadows**.
- Qualcuno ha fatto un tavolo rettangolare invece che quadrato: "va benissimo, tranquilli".
- Uno studente, che ha appena finito il download, ha fatto solo la base del tavolo e una gamba della sedia, sottilissima. Igor ricorda che Unreal lavora in centimetri, e quella gamba è di pochi centimetri: "non ci farei sedere nessuno, potrebbe crollare". Ma "è soggettivo".

"A parte l'ironia", la conclusione è che gli oggetti in sé contano poco: conta capire come ci si muove in Unreal, duplicando, scalando, muovendo, ruotando. "Questa è la cosa più importante che oggi impariamo."

Una cartella per ogni tavolo. Un'ultima richiesta: mettere ogni gruppo in una cartella dell'Outliner. Create Folder, "Tavolo da due", si selezionano i pezzi e li si trascina dentro; poi "Tavolo da tre", e così via. Con Select All Descendants si riprende il tavolo intero in un clic.

1.1.22 Salvare la mappa, e la chiusura

Salvare. Per salvare il lavoro c'è il classico **Ctrl+S**. Siccome la mappa è stata creata "da nulla", la prima volta Unreal chiede dove salvarla: le si dà un nome ("MappaProva") e la si salva dentro la cartella "prova" del Content Browser. In basso a destra, poi, l'editor tiene il conto delle modifiche non salvate: se si sposta anche solo una sedia, compare l'indicazione degli oggetti ***unsaved***, "tutti gli oggetti che devo salvare".

Che cos'è il secondo file? Uno studente nota che il salvataggio crea anche un secondo file, un **HLOD Layer**[18]. Igor spiega che dipende dalla mappa: quella su cui hanno lavorato contiene un **landscape**, le montagne che si vedono in fondo. Il landscape è un piano che si può modellare con uno strumento apposito per formare, per esempio, il paesaggio di sfondo; il file salva come quel piano è composto. La volta successiva si userà una mappa davvero vuota, senza landscape.

Approfondimento: LOD e HLOD. **LOD** sta per *level of detail*: lo stesso oggetto esiste in più versioni, con un numero decrescente di poligoni, e l'engine usa quelle più semplici quando l'oggetto è lontano dalla camera, dove i dettagli non si vedrebbero comunque. È la risposta pratica al compromesso della sezione 1.1.3 tra precisione e peso. Gli **HLOD** (*hierarchical* LOD) fanno lo stesso a livello di gruppi: interi blocchi lontani della mappa vengono sostituiti da un'unica versione semplificata. Le mappe "a mondo aperto" di Unreal Engine 5, con il landscape, li usano per impostazione predefinita, ed è per questo che il salvataggio crea un file di *HLOD Layer*.

Le ultime domande. Come si separano gli oggetti raggruppati? "Shift+G, oppure tasto destro, Ungroup: se lo cerchi c'è proprio scritto." Quando arriva la registrazione? Sul sito, "credo il giorno dopo". Le foto dei lavori si possono mandare a Igor su Discord, "tranquillamente" ("quando ci sono, vi rispondo"), anzi in un *thread* o un canale apposito, "così almeno è più ordinato e non mi riempite la casella privata".

Il calendario. Giovedì si rivede Igor; il martedì successivo c'è Davide, con la parte artistica; poi le lezioni si alternano, "settimana a settimana". Sono le 19: "per chi tra poco cena, auguro buona cena a tutti". "Se avete domande, fatele adesso o tacete fino alla prossima volta." "Ciao, ragazzi."

[17] Igor ci arriva per tentativi: "Dungeon Crawler", "Dungeon Designer", "Dungeon Scroll".
[18] Nella trascrizione "Halodle" e "Halod Layer".

Punti chiave del capitolo

- Il **game engine** è "il calderone" che riunisce 3D art, programmazione e design in un unico ambiente, con fisica e collisioni, suono, animazione, networking e IA. Lo si governa con lo **scripting**; in Unreal, con i **Blueprint**, un linguaggio a nodi usato anche dai programmatori (il C++ resta per ottimizzare). Conta la **logica**, più dello strumento.
- Il **rendering** trasforma poligoni in pixel: ***vertex shading*** (i vertici dei triangoli), **rasterizzazione** (quali pixel), ***fragment shading*** (colore, luce, ombre, riflessi). Più poligoni = più precisione, ma più peso: "un cubo con due milioni di poligoni" fa esplodere il gioco.
- Il ***real time rendering*** lavora tra 20 e 120 *fps*: il cinema è a 24, il gaming a 30–60 e oltre. Un **frame** è un'immagine; la loro successione dà l'illusione del movimento.
- Unreal è il più usato perché è un **linguaggio comune**, aggiornato di continuo, e perché permette di **prototipare** subito: "non ci interessa se è bello o brutto, è importante che funzioni" (il prototipo di *Expedition 33*). È usato anche nel cinema, dove conta il dettaglio più dell'ottimizzazione.
- Ciò che distingue un videogioco da un video è il **game loop**: input, *update*, render, e di nuovo, a ogni *frame*.
- **Ogni buona idea non esiste finché non sta su carta.** Indie, AA e AAA si distinguono per i soldi e per la specializzazione dei ruoli (fino ai *leaf artist*). L'***outsourcing*** è chiedere a un altro studio una parte del lavoro, senza cedergli la proprietà.
- Tutti i ruoli toccano l'engine: *blockout* e metriche per il level designer, momenti scriptati per il narrative designer, parametri esposti per il combat designer, materiali e Sequencer per gli artisti, rendering, IA e rete per i programmatori, Unreal Insights per il QA. **Trattate bene i vostri programmatori.**
- Un gioco è fatto di **attori, meccaniche, spazio, obiettivi, regole** (e a volte **tempo**); il divertimento è una **conseguenza**, e il gameplay nasce dalle **scelte**. Meccanica = che cosa posso fare; dinamica = come lo faccio; estetica = come mi fa sentire. "Un gioco per tutti non è un gioco per nessuno."
- Il bilanciamento è un lavoro tra **designer e QA**; il feedback utile è quello da cui si ricava un'informazione. Far giocare chi non ha mai visto il gioco "è la cosa più pura".
- Per il corso serve Unreal **5.5** (la 5.5.4): l'ultimo numero è solo l'aggiornamento, il secondo decide la compatibilità.
- Le risorse: **documentazione**, forum, corsi gratuiti di *learning*, **Fab** con i *pack* gratuiti del mese ("prendeteli sempre"). Ma **prima la funzione, poi l'*asset***, o il progetto pesa 70 GB per usarne tre.
- Quando si preme Play gli attori vengono inizializzati e duplicati. Ogni movimento va reso **relativo al tempo di gioco** (*delta seconds*), altrimenti dipende dagli *fps*.
- Il progetto di prova: Games, **Blank**, Blueprint, Desktop, Starter Content, nome **senza spazi**. I template si distinguono per le **3C**: camera, controller, character.
- L'editor è fatto di **finestre** spostabili (Window le riapre): viewport, Details, Outliner, World Settings, **Content Browser**. Cartelle colorate e ordine: "quando siete ordinati, la gente vi ringrazia".
- Il **transform** è locazione (**centimetri**), rotazione (**gradi**) e scala (**unità**); **W, E, R**; X rosso, Y verde, Z blu; assi locali o globali, ma **la scala è solo locale**.
- **Alt + trascinamento** duplica, **Ctrl+G** raggruppa, **Fine** appoggia a terra, lo ***snap*** azzurro fa muovere a scatti, la **rotellina** cambia la velocità della camera, **Select All Descendants** riprende un'intera cartella dell'Outliner.

Chapter 1.2

Programmare con i Blueprint: actor, variabili, logica e il gioco della pallina

Questa è la **seconda lezione** del corso breve di **introduzione a Unreal Engine**, e la prima in cui si programma davvero. Se la prima lezione aveva insegnato a muoversi nell'editor (piazzare, spostare, ruotare, scalare e ordinare gli oggetti), questa insegna a dare agli oggetti un **comportamento**, con i **Blueprint**, il linguaggio visuale "a blocchi" di Unreal Engine. Il docente, Igor, lo annuncia così: "oggi avremo l'onore di andare avanti e andare a vedere un po' di programmazione vera e propria". Quindi gli *actor*, "le particolarità che possiamo farci", "piccole e semplici operazioni" e "qualche esercizio qua e là".

La lezione ha una struttura chiara. Nella prima parte si costruiscono i concetti di base: che cos'è un **actor** e da che cosa è fatto (i **componenti**); che cos'è una **classe** e quali classi di Blueprint si possono creare (**Actor**, **Pawn**, **Character** e le altre); come è fatto l'editor di un Blueprint e che cos'è un **evento**; che cosa sono le **variabili**, i loro **tipi** e i loro colori; come si fanno i **calcoli** e come si prendono le **decisioni**, con il nodo **Branch** e gli operatori logici **AND**, **OR** e **NOT**. Ogni concetto è accompagnato da un piccolo esercizio fatto insieme. Nella seconda parte i concetti diventano un piccolo gioco: una **piattaforma** che si inclina con i tasti, e poi con il mouse, sulla quale rotola una **pallina** con la fisica attiva, dentro una "vaschetta" di muri. Lungo la strada si incontrano il **Level Blueprint**, la **mobilità** degli oggetti, l'**Event Tick** con i ***delta seconds*** e il **Clamp**, che impone a un valore un minimo e un massimo.

La lezione è **online**, su Zoom, come la prima. La registrazione comincia poco prima delle 16 ("le quindici e cinquanta"), Igor aspetta che entrino tutti ("aspettiamo almeno sedici e zero cinque, sedici e dieci, in modo tale che facciamo entrare tutti quanti") e propone una pausa "verso 17 e mezza, diciamo proprio per dividere la lezione in due"; chiede però agli studenti di segnalargli quando l'attenzione cala ("lo capisco assolutamente"), perché la pausa "la facciamo tranquillamente". La pausa, di una ventina di minuti, arriva in effetti a metà lezione, e la lezione finisce alle 19 e 3 minuti. Igor alterna di continuo le **slide** e l'**engine**, "in modo tale da un po' fare in *real time*". Ci sono due presenze nuove rispetto alla prima volta ("due nuovi presenti oggi", dal corso di grafica, "da quello che so"), e uno studente arriva in ritardo ("scusate, sono arrivato un ciclone").

Tre avvertenze di lettura, le stesse della prima lezione. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal diventa "ArrayLab", "Arreal", "a Real", "Arry" e "Rea"; l'**Event Graph** è "EvenGraph" o "even graph"; la **Skeletal Mesh** è "scheda mesh" e "scheda dal mesh"; il tipo *bool* è "Bull" e "bullo"; il *float* una volta è "flotto" e una "flop"; "sponato" sta per *spawnato*. Dove la ricostruzione è sicura il capitolo usa la forma corretta e, la prima volta, lo segnala in nota; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione, durante la pausa e in qualche momento di lavoro individuale il riconoscimento vocale produce lunghissimi blocchi di testo senza senso e ripetuto (discorsi su "la migliore dal 2005", "la struttura del giardino", "la Corte Suprema di Internet", "qual è la risposta che ci può essere?"): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è una dimostrazione dal vivo nell'editor dei Blueprint. Il capitolo ricostruisce i grafi di nodi da ciò che Igor dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei comandi di Unreal Engine 5.5.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono gli schemi spiegati a voce, sulle slide o nell'editor: la gerarchia delle classi, l'editor dei Blueprint, i pin e i loro colori, i grafi degli esercizi, il grafo finale della piattaforma comandata dal mouse. Le tabelle riassumono le classi, i tipi di variabile, gli operatori logici e le scorciatoie dell'editor dei Blueprint. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro *Blueprints Visual Scripting for Unreal Engine 5* di Marcos Romero e Brenden Sewell, il testo di riferimento per questa parte del libro, e a qualche nozione tecnica generale; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. la ripresa e il ripasso: **actor**, **transform**, **componenti**, editor e *runtime* (sezioni 1.2.1 e 1.2.2);
2. riaprire la mappa e **filtrare il Content Browser** (sezione 1.2.3);
3. le **classi Blueprint**: Actor, Pawn, Character e le altre (sezione 1.2.4);
4. l'**editor dei Blueprint**, l'**Event Graph**, gli **eventi** e il primo nodo, **Print String** (sezioni 1.2.5 e 1.2.6);
5. le **variabili**: GET e SET, i colori, i calcoli, il primo esercizio, i dettagli e i **tipi** (sezioni da 1.2.7 a 1.2.10);
6. le **booleane**, il **Branch** e gli **operatori logici**, con il secondo esercizio (sezioni da 1.2.11 a 1.2.13);
7. il **Level Blueprint** e la piattaforma che si inclina: input, referenze, rotazione, **mobilità**; il problema prima della pausa (sezioni da 1.2.14 a 1.2.17);
8. dopo la pausa: **Add Actor World Rotation** e i tasti W A S D, la **fisica** della pallina e il **Teleport** (sezioni 1.2.18 e 1.2.19);
9. l'**Event Tick** e i ***delta seconds*** (sezione 1.2.20);
10. la piattaforma comandata dal **mouse**, con il **Clamp** (sezione 1.2.21);
11. la **vaschetta**, il soffitto invisibile, i compiti e la chiusura (sezioni 1.2.22 e 1.2.23).

1.2.1 La ripresa: il ristorante, e il programma di oggi

"Allora, ragazzi, com'è andata con il vostro ristorante personale?" L'esercitazione della volta precedente (arredare un ristorante con tre tavoli e le loro sedie) è andata bene: "sono venuti, hanno assaggiato, sono venuti i clienti", scherza Igor. Poi annuncia il programma: gli *actor*, che cosa ci si può fare, qualche operazione e qualche esercizio.

Prima di cominciare chiede se ci sono domande sulla lezione scorsa, dubbi, difficoltà con l'esercizio. Nessuno ne ha: "vi vedo abbastanza sicuri di voi. Buono, siatelo sempre."

1.2.2 Actor, transform e componenti

Che cos'è un actor, di nuovo. Igor parte da un ripasso. Gli **actor** (attori) sono "fondamentalmente gli elementi di gioco": il nome con cui Unreal chiama tutti gli elementi che possono essere messi nella scena (nella *viewport*) e che hanno un **transform** in 3D. E il *transform*, "ve lo ripeterò a manetta, quindi sarà un buon *meme*", è **locazione, rotazione e scala**: i tre fattori che si sottintendono ogni volta che si parla di *transform*.

I componenti. Un actor può essere composto da **componenti** (*component*). Un esempio di componente è la sua **mesh**. Igor chiede se qualcuno ricorda che cos'è la *mesh*, e una studentessa risponde: "un oggetto solido formato da poligoni". "Esatto": la *mesh* è fondamentalmente il modello 3D, e può essere un componente di un actor, una cosa che l'actor ha al suo interno. Oltre ai componenti, un actor ha una **logica**, che si programma con i **Blueprint**: il metodo di programmazione di Unreal, "un linguaggio visuale a blocchi".

Chi ha già programmato? Igor chiede chi abbia esperienza con i linguaggi di programmazione, o almeno abbia sentito parlare di programmazione a blocchi. Qualcuno sì; qualcuno è "proprio programmazione zero, proprio al di là del muro". "Perfetto": oggi si prendono "proprio le basi della programmazione". E la rassicurazione è la stessa della prima lezione. Programmare non è quello che si vede nei film di *hacker*, "con tutte quelle linee verdi che vanno a manetta e voi non ci capite niente": sono "cose molto più semplici", e anzi "a volte fanno molto più ridere", perché ci si trova davanti a un foglio pieno di informazioni e si sbatte la testa. "Di base conta tanto la logica": "proprio logica, punto A, punto B".

Editor e runtime. Poi il processo di Unreal "spiegato molto velocemente". Quando si è nell'**editor** si ha una **preview**: si agisce, si modificano le cose, si muove questo e quell'altro. Quando si preme **Play**, invece, c'è l'**eseguibile**, quello che si chiama ***runtime***. "Una cosa è muoviamo le cose, e l'altra è premere Play." Quando si preme Play tutti gli attori messi nel "nostro bellissimo giochino" **prendono vita**: vengono **duplicati**, vengono **inizializzati**, e con loro vengono **pre-inizializzati e inizializzati** anche tutti i loro componenti. Vale sia per gli attori già posizionati nel livello, sia per quelli ***spawnati***[1], cioè creati durante il gioco.

I tipi di componenti. Sulla slide ci sono alcuni esempi di **Actor Component**: **audio**, ***particle effect***, **Point Light** ("le luci, fondamentalmente; sono una tipologia di luce"), **Scene**, **Skeletal Mesh**[2] e **Static Mesh**. La **Skeletal Mesh** è una *mesh* che ha dentro di sé uno **scheletro**: tipicamente un umanoide. Per chi fa animazione, parlare di scheletro porta a parlare di ***rigging***, "che però è meno piacevole perché è un processo molto lungo". Lo scheletro regola la rotazione delle **ossa**: come le ossa umane ruotano e compiono i movimenti, così si compie un'animazione su un corpo umanoide. I componenti possono essere di vario tipo e possono avere anche una loro **gerarchia**: uno può essere interno a un altro, e quindi "**imparentato**". "Adesso capiamo poi meglio che cosa si intende."

Approfondimento: Actor Component e Scene Component. La distinzione che la slide accenna è precisa, e Romero e Sewell la fanno nel terzo capitolo del libro, a proposito della gerarchia delle classi: **Actor Component** e **Scene Component** non sono actor, ma classi da cui nascono i componenti che si aggiungono agli actor. Un **Actor Component** è un comportamento "puro", senza posizione nello spazio (per esempio un componente che gestisce i punti vita, o il Rotating Movement che il libro usa nel primo capitolo per far girare una sedia). Uno **Scene Component** ha in più un *transform*, e quindi può essere posizionato e soprattutto **agganciato** ad altri componenti: è qui che nasce la gerarchia di cui parla Igor. Static Mesh, Skeletal Mesh, luci, camere e collisioni sono tutti Scene Component (più precisamente, quelli visibili sono *Primitive Component*, figli dello Scene Component). Un componente agganciato a un altro ne eredita il *transform*: se si sposta il "genitore", il "figlio" lo segue.

1.2.3 Riaprire la mappa e filtrare il Content Browser

"Prima di tutto vi chiedo: aprite Unreal, aprite il progetto dell'altra volta", "e incominciamo comunque a mettere qualche cosa sul fuoco". Igor aspetta che tutti confermino ("io ci sono", "anch'io"), poi dà un'indicazione importante: se in alto nell'editor c'è scritto **Untitled**, la mappa aperta è quella di default di Unreal, non quella della lezione scorsa. Bisogna riaprire la mappa "con i tavolini", salvata con un nome (quella di Igor era **MappaProva**). La si trova cliccandoci sopra nel Content Browser, se ci si ricorda dov'è, oppure con un piccolo trucco: i **filtri**.

I filtri del Content Browser. Il **Content Browser**, ricorda Igor, è la schermata che permette di navigare tra gli elementi (gli *asset*) che si possono usare in Unreal. Cliccando l'icona con le **tre linette**, accanto alla barra di ricerca, si può scegliere la **tipologia di *asset*** da mostrare; Igor ha già impostato i **livelli** (**Level**), "perché molto spesso mi serve". Il filtro compare come un'etichetta che si può **accendere e spegnere**

[1] Nella trascrizione "esponati" e, più avanti, "sponato". *Spawnare* (dall'inglese *to spawn*) significa far comparire un actor nel mondo durante il gioco, invece di piazzarlo prima nell'editor.

[2] Nella trascrizione "scheda mesh".

("cerca sì, cerca no") con un clic, e se ne possono aggiungere altri, per esempio **Blueprint Class**: "così ve le cercate, è molto più facile".

Attenzione al percorso. "Mi raccomando, attenzione, perché è sempre relativo al percorso." Il filtro mostra gli *asset* del tipo scelto dentro la cartella in cui ci si trova, e nelle sue sottocartelle. Se si è in **All > Content** si vedono tutti i livelli del progetto; se si entra in **All > Content > prova** si vedono solo quelli di quella cartella, "come l'indirizzo classico" dell'Esplora file di Windows. Di solito conviene mettersi sulla cartella **Content**, dove la mappa dovrebbe essere stata salvata. Con il filtro acceso la mappa salta fuori subito: "ah ok, è la mappa prova, eccola qua".

1.2.4 Le classi Blueprint: Actor, Pawn, Character

"Abbiamo parlato tanto di sti actor, ma come si creano?"

Prima, una cartella. "Prima di tutto facciamo una bella cartella, perché l'ordine è sempre buona cosa." Nel Content Browser: tasto destro, **New Folder**, nome **Blueprint**, e un colore ("vediamo un celestino"). Si entra nella cartella e, con il tasto destro, si trova un menu con le cose più comuni che si possono creare. La prima è proprio la **Blueprint Class**.

Che cos'è una classe. Igor chiede se gli studenti abbiano mai sentito il concetto di **classe**: "non classe come le medie o le elementari". Una classe è una **tipologia di oggetti**: "c'è la classe palle, la classe cubi, la classe coni". È un termine generico, che non indica proprio i cubi o le palle, ma un **gruppo di elementi che hanno delle similitudini tra di loro**. L'immagine che Igor sceglie è quella del **supermercato**: c'è il reparto dei surgelati, il reparto degli insaccati, e ognuno è una classe. Cliccare su Blueprint Class è "come andare nel reparto degli insaccati" e scegliere se si vuole la mortadella, il prosciutto crudo o il prosciutto cotto.

La scelta della classe genitore. La finestra **Pick Parent Class** propone infatti le **classi comuni**: **Actor**, **Pawn**, **Character**, **Player Controller**, **Game Mode Base**[3], **Actor Component** e **Scene Component**[4]. Sono tutti Blueprint, ma "differenziano tra di essi": hanno delle differenze piccole, ma "abbastanza nette". (Qui Igor si ferma un attimo per un problema di audio di uno studente, Adri, che poi si sente meglio: "a posto".) Le più comuni sono Actor, Pawn e Character, e Igor le crea tutte e tre per mostrarne le differenze (tabella 1.2.1).

La nomenclatura. "Di solito la nomenclatura per i Blueprint è BP, *underscore*, nome della cosa che vogliamo fare." I tre Blueprint di prova si chiamano quindi **BP_ActorProva**, **BP_PawnProva** e **BP_CharacterProva**[5]. Già nel Content Browser hanno icone diverse.

L'Actor: un centro e basta. Se si apre il Blueprint di tipo Actor (doppio clic) "non c'è assolutamente nulla": c'è solo un'icona che ne indica il **centro**, per poterlo vedere e selezionare nella *viewport*. Nel gioco non sarà visibile.

Il Pawn: un actor che si può possedere. Il **Pawn**, aperto, sembra identico. La differenza la dice la descrizione che Unreal mostra quando lo si crea: un Pawn è "*an actor that can be possessed and receive input from a controller*", un actor che può essere **posseduto** e ricevere input da un **controller**. Quindi "qualsiasi cosa che sia impossessabile dal player è un Pawn": il giocatore ne può prendere possesso e muoverlo. "Potrebbe anche essere un cubo."

[3] Nella trascrizione "Game Base".
[4] Nella trascrizione "Sing Component".
[5] La trascrizione dice "andremo a fare actor. Prova", "bp pawn, prova" e poi "andiamo a creare anche un character"; i nomi esatti sono una ricostruzione coerente con la convenzione appena enunciata.

Il Character: un Pawn con un corpo. Il **Character** si distingue "in modo abbastanza più netto". Se si passa dall'uno all'altro, in alto a sinistra (nel pannello **Components**) ci sono già cose diverse: "questi qua sono i *component*" che "scandiscono" che cos'è un Character. Prima di tutto una **Skeletal Mesh**, che è vuota, perché va obbligatoriamente impostata: dentro si mette la *mesh* di un personaggio, e si ottiene un Character "definitivo". Il Pawn è possedibile ma non ha una Skeletal Mesh; si potrebbe aggiungergliela con il pulsante **Add** del pannello Components ("però non fatelo, ragazzi": era solo per mostrare la differenza). E naturalmente un Character è anch'esso un Pawn: "qualsiasi cosa di cui il player può prendere possesso prende la caratteristica di essere un Pawn". Per oggi, comunque, serve solo il primo, l'Actor.

Classe	Che cos'è
Actor	un oggetto che si può piazzare o *spawnare* nel livello; vuoto, ha solo un centro e un *transform*
Pawn	un Actor che può essere **posseduto** da un Controller e ricevere input; il "corpo" controllato dal giocatore o dall'IA
Character	un Pawn pronto per un personaggio che cammina, corre, salta, nuota, vola: capsula di collisione, freccia, Skeletal Mesh, Character Movement
Player Controller	il "cervello" del giocatore umano: riceve gli input e possiede un Pawn
Game Mode Base	definisce le regole della partita (quale Pawn e quale Controller usare, per esempio)
Actor Component	un componente riutilizzabile, senza *transform*, da aggiungere agli actor
Scene Component	un componente con un *transform*, che si può agganciare ad altri

Table 1.2.1: Le classi comuni della finestra Pick Parent Class. Igor mostra le prime tre; le descrizioni delle altre sono un'integrazione dal libro di Romero e Sewell.

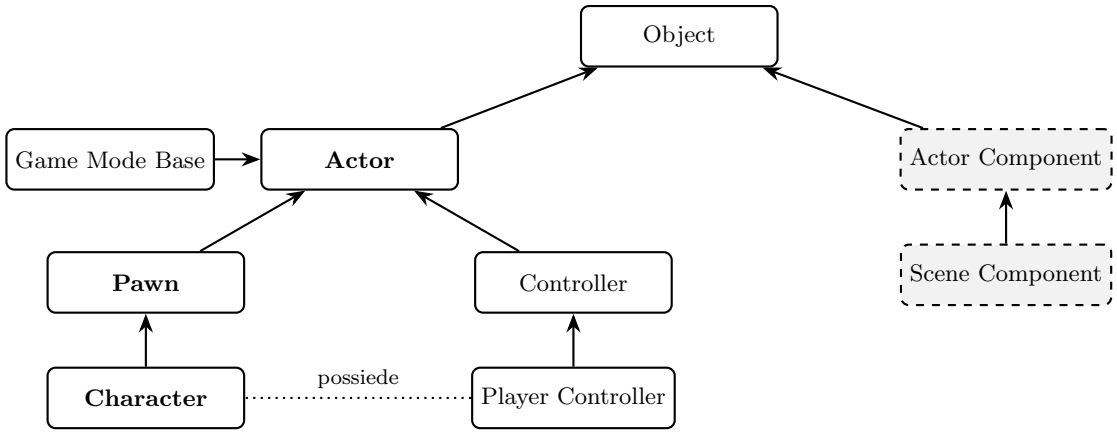


Figure 1.2.1: La gerarchia delle classi comuni (ricostruzione dal terzo capitolo del libro di Romero e Sewell; Game Mode Base deriva in realtà da Actor attraverso una classe intermedia, Info). Le frecce vanno dal figlio al genitore: un Character è anche un Pawn e un Actor. I riquadri tratteggiati sono componenti, non actor.

Approfondimento: la gerarchia delle classi. Il libro di Romero e Sewell dedica il terzo capitolo alla programmazione a oggetti e al *Gameplay Framework*, e rende esplicito quello che Igor dice a voce (figura 1.2.1). In Unreal c'è una classe madre di tutto, **Object**; ogni classe **eredita** le caratteristiche della classe sopra di lei, la sua *classe genitore*. Per questo un Character "è" contemporaneamente un Character, un Pawn e un Actor. Il libro usa un'immagine utile: il **Pawn** è il **corpo** fisico del personaggio, il **Controller** che lo possiede è il suo **cervello**. Il Controller ha due figli principali, il **Player Controller**, usato dai giocatori umani, e l'**AI Controller**, che controlla il Pawn con l'intelligenza artificiale. Pawn e Character ricevono gli eventi di input solo se un Player Controller li possiede. Il Character, infine, eredita quattro componenti: la **Capsule Component** per le collisioni, l'**Arrow Component** che indica la direzione, la

Mesh (la Skeletal Mesh di cui parla Igor, animata da un *Animation Blueprint*) e il **Character Movement**, che gestisce camminata, corsa, salto, nuoto e volo, e anche la replica in rete nel multigiocatore.

1.2.5 L'editor dei Blueprint e gli eventi

"Visto che abbiamo parlato di metterci un po' le mani", Igor apre BP_ActorProva e presenta la schermata "principalmente dei Blueprint", l'**editor dei Blueprint** (figura 1.2.2). Se una schermata non si trova, sono tutte raggiungibili dalle schede in alto.

La viewport del Blueprint. La prima cosa che si vede è la **viewport**: "letteralmente la visualizzazione grafica", in 3D, del **singolo oggetto**. È separata dalla *viewport* del livello: è uno spazio tutto suo, in cui si vede effettivamente quello che si modifica a livello grafico.

Construction Script. C'è poi il **Construction Script**, che "lo vedremo un'altra volta".

L'Event Graph, il cuore del Blueprint. Quello che interessa "veramente tantissimo" è l'**Event Graph**[6]. È "il cuore stesso di un Blueprint": la programmazione dell'oggetto sta lì, è "il nucleo, il motore centrale". Igor lo paragona a un motore di cui si ha solo la **scocca**, e dentro il quale si mettono tutti i pezzi: "fai quello, fai quell'altro, mettiamo il bullone", e così via.

I tre nodi grigi. In un Blueprint nuovo l'Event Graph contiene già tre "blocchetti", **deopacizzati** (semi-trasparenti): sono grigi perché non sono collegati a niente[7].

Che cos'è un evento. "Una cosa molto importante da capire è che all'interno del Blueprint si parla di **eventi**." Un evento è "un momento con delle specifiche regole": ha una **condizione** precisa che lo fa entrare in esecuzione. Igor chiede: dal nome, quando avviene **Event BeginPlay**? "All'avvio del programma", risponde uno studente. "Non esattamente è giusto dirlo." Il BeginPlay scatta quando **un attore comincia a esistere**: quindi all'avvio, per gli attori già nel livello, ma anche quando un attore viene *spawnato* più tardi. Ogni attore ha il suo: "ci sta l'attore 1, l'attore 2, l'attore 3, l'attore 4". Se tutti i BeginPlay partissero "in una botta" all'avvio, sarebbe "un sovraccarico di informazioni" di cui non si ha un uso effettivo; il BeginPlay di un actor, invece, fa la sua azione appena quell'actor entra in gioco, "semplicemente esiste".

Approfondimento: i pannelli dell'editor. Romero e Sewell, nel primo capitolo, elencano i pannelli principali del *Blueprint Class Editor*: la **Toolbar**, i **Components** (i componenti già pronti che si aggiungono con Add: *mesh* statiche, luci, suoni, collisioni, particelle, camere), **My Blueprint** (dove si creano variabili, macro, funzioni e grafi, con i pulsanti +), i **Details** (le proprietà dell'elemento selezionato, con una barra di ricerca in alto), la **Viewport** e l'**Event Graph**. Nell'Event Graph, notano gli autori, un **evento** è un nodo **rosso**, e le altre azioni che seguono un evento si collegano con dei fili; ci si sposta nel grafo trascinando con il tasto destro. Di ogni evento se ne può mettere **uno solo** per grafo, ma si possono creare eventi propri (**Add Custom Event**). Oltre a BeginPlay e Tick, il libro cita gli eventi di **collisione** (quando due actor si scontrano o si sovrappongono), gli eventi di **input** (tastiera, mouse, *touch*, *gamepad*) ed **Event End Play**, quando l'actor sta per essere rimosso.

1.2.6 Il primo nodo: Print String

"Andremo a fare una piccola prova." Unreal gestisce tutto con i **nodi**. Se si porta il mouse sul "pinnetto" bianco a forma di freccia dell'Event BeginPlay, lo si clicca e lo si **trascina verso l'esterno**, al rilascio compare un menu con tutti i blocchi che si possono collegare: "possiamo metterci tutto quello che vogliamo, tutti i blocchi che ci serviranno per fare un'azione".

[6] Nella trascrizione "EvenGraph" ed "even graph".

[7] Igor non li nomina tutti; in un Blueprint di tipo Actor appena creato sono **Event BeginPlay**, **Event ActorBeginOverlap** ed **Event Tick**. L'ultimo tornerà più avanti nella lezione (sezione 1.2.20).

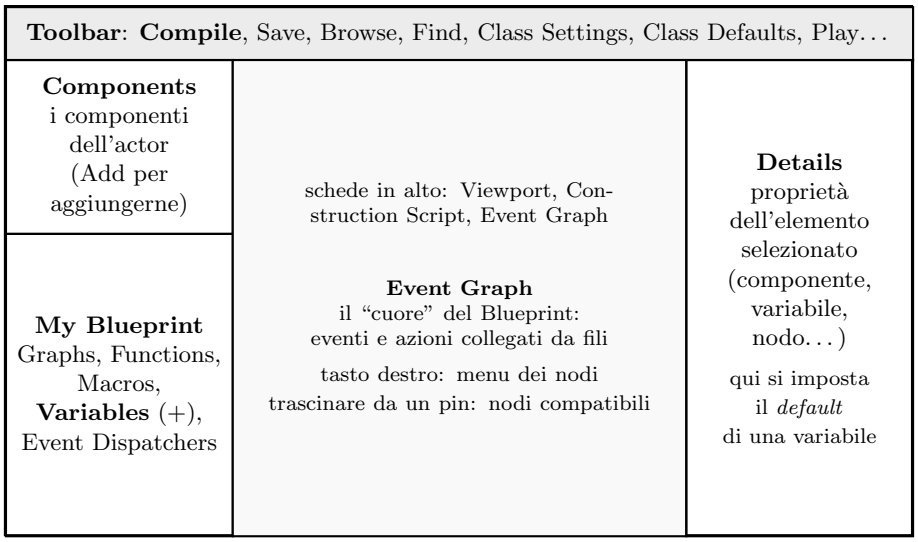


Figure 1.2.2: L'editor dei Blueprint (Blueprint Class Editor), ricostruito dalla lezione e dal primo capitolo del libro di Romero e Sewell. La disposizione dei pannelli si può cambiare, come nel Level Editor.

Print String. L'azione più semplice è **Print String**, che "letteralmente ci stampa qualcosa a schermo". Il testo predefinito è "Hello".

La barra "Development Only". Igor fa notare un dettaglio. A differenza di altri nodi, come **Set Actor Location** o **Enable Collision**, il Print String ha una **barra in basso**. Vuol dire che quel blocco ha effetto **solo in modalità di sviluppo**: quando si sviluppa e si va in Play nell'editor funziona, ma se si facesse un **eseguibile** da mandare a qualcun altro "questa cosa non avverrà mai". Gli altri nodi, invece, sì.

Compilare. "Molto importante": quando si fanno dei cambiamenti importanti bisogna **compilare** (**Compile**, in alto a sinistra). La compilazione "legge quello che abbiamo fatto" e aggiorna il programma; in molti casi Unreal lo fa anche in automatico, ma conviene farlo sempre. E **salvare**, "ovviamente, sempre buona cosa".

La prova. Si abbassa la finestra del Blueprint, si **mette l'actor nella scena** ("ovviamente, se no non succede niente": il Blueprint è solo una classe, finché non se ne mette una copia nel livello non esiste) e si preme Play. In alto a sinistra compare "Hello", ma per pochissimo. Igor cambia allora il parametro **Duration** del nodo a 100 secondi, e la scritta resta: "è piccolo", ma c'è. L'actor è entrato in funzione e ha fatto la sua azione appena ha cominciato a esistere, cioè subito, perché era già nella scena.

Una finestra di gioco separata. "Se vi è più comodo", si può far partire il gioco in una finestra esterna: dai **tre puntini** accanto al pulsante Play si sceglie **New Editor Window (PIE)**. "Io lo trovo molto più comodo": invece di abbassare e rialzare la finestra del Blueprint, il gioco gira in una finestra a parte da cui si controlla tutto. Igor lo scrive anche in chat.

Approfondimento: *development* e *shipping*. La barra del Print String rimanda a una distinzione che Romero e Sewell spiegano nel capitolo sul *packaging*: le *build* di **sviluppo** (*Development*) contengono informazioni che servono al *debug*, quelle di **distribuzione** (*Shipping*) sono più "pulite", senza informazioni di *debug*, e sono quelle da pubblicare. I nodi marcati *Development Only*, come il Print String, vengono semplicemente ignorati in una *build* Shipping: si possono quindi lasciare nel grafo mentre si lavora senza che il giocatore finale veda mai le scritte di prova.

1.2.7 Le variabili: GET e SET, colori e calcoli

Che cos'è una variabile. Igor chiede chi abbia mai sentito parlare di variabili *bool*, *float*, *int*; qualcuno sì. Una **variabile** è "uno slot di una tipologia di valore" in cui si può mettere un valore: "è come una **scatola**". Si creano delle piccole scatoline in cui mettere dei valori, ma ogni scatola è fatta per un certo contenuto: "c'è la scatola per i gioielli, c'è la scatola per le praline" ("mi invento un po' di cose"). Il tipo della variabile decide che cosa ci si può mettere dentro.

Creare una variabile. L'esempio: rendere "esterna" la durata del Print String, cioè metterla in una variabile. Nel pannello My Blueprint, alla voce **Variables**, si clicca +: si crea subito una variabile, di tipo booleano oppure dello stesso tipo dell'ultima variabile creata. Igor la chiama **duration time text**[8].

GET e SET. Se si trascina la variabile nel grafo, Unreal chiede se si vuole un **GET** o un **SET**. GET è **prendere** il valore che c'è dentro; SET è **impostare** il valore dentro. Per dare il valore alla durata del Print String serve un GET.

I colori devono combaciare. "Però attenzione, che succede? Questo è rosso e questo è verde." Il pin della variabile appena creata è **rosso** (è una booleana), quello della Duration è **verde**: "non sono della stessa tipologia". Due pin rossi si collegano; un rosso e un verde no. Il verde è il **float**. "Cosa sono i numeri float?" "I numeri con la virgola." "Ottimo": i numeri con i **decimali**. Si cambia quindi il tipo della variabile in Float, il pin diventa verde, e "guardate un po', si collega".

Please compile the Blueprint. Appena collegata, la variabile mostra l'avviso *please compile the blueprint*: è stata appena creata, e il Blueprint "non sa ancora che c'è una variabile nuova". Dopo la compilazione, nei Details compare il suo **valore predefinito** (*Default Value*): "guarda, questo è il tuo valore. Quanto lo vuoi?" Igor mette 10. In Play la scritta dura 10 secondi ("un po' tanti") e poi scompare: "non è più 100". La lezione: si possono creare variabili, impostarle e darle ai blocchi di esecuzione.

Nodi di esecuzione e nodi di calcolo. I **blocchi di esecuzione** si riconoscono perché hanno l'intestazione **blu** e il pin di esecuzione, "la freccettina" bianca. Ci sono però anche nodi **verdi** senza quel pin: sono nodi che "**calcolano solamente**". Igor ne fa vedere uno, **Find Look at Rotation**: chiede due variabili di un tipo e ne restituisce un'altra, dello stesso tipo o di un altro. "Oppure, esempio classico": gli **operatori** aritmetici, il +, il − (*Subtraction*), il **per**, che si cerca con l'**asterisco** (*Multiply*), e il **diviso** (*Divide*). Si possono invocare tutti gli operatori che servono (figura 1.2.3).

Sottrarre due secondi. L'esempio: "indipendentemente da quanto tempo noi mettiamo qua dentro", togliere sempre 2 secondi. Si prende il nodo meno, si collega la variabile al primo ingresso (il nodo "è diventato verde", perché ha riconosciuto che si tratta di un float, e "ci chiede: te lo sottraggo di un altro float"), si scrive 2 nel secondo e si collega il risultato alla Duration.

Da numero a testo. Si può anche **stampare** il valore: collegando il float all'ingresso di testo del Print String, Unreal inserisce in automatico un **convertitore**. I pin **rosa** sono le **stringhe**, cioè testo. In Play, il Print String non stampa più "Hello" ma **8**, "che è 10 meno 2", e la scritta dura 8 secondi. (Ogni volta che Igor apre Unreal, Zoom va a scatti: "a quanto pare mi lagga Zoom benissimo".)

1.2.8 Primo esercizio: Time To Print

"Vorrei fare un piccolo esercizio con voi. Molto velocissimo."

[8] Così nella trascrizione (una volta "duration time next"); il nome esatto digitato non si sente, ma non ha importanza per l'esempio.

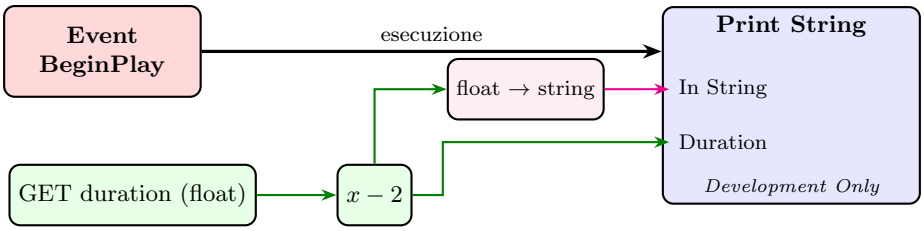


Figure 1.2.3: Il primo grafo della lezione, nella sua versione finale: all'inizio del gioco l'actor stampa la durata meno 2 (cioè 8) per 8 secondi. La freccia nera è il filo di **esecuzione** (l'ordine delle azioni, da sinistra a destra); le frecce colorate sono fili di **dati**, del colore del loro tipo (verde il float, rosa la stringa). Il nodo blu, con il pin di esecuzione, è un'**azione**; i nodi verdi, senza, sono nodi di **calcolo**, che si eseguono quando un'azione chiede il loro valore. Il convertitore è inserito da Unreal in automatico. Schema ricostruito, non copia della schermata.

Staccare un filo. Prima, un comando: per **staccare** un collegamento si tiene premuto **Alt** e si clicca sul pin.

La consegna. Creare un'altra variabile, **time to print**, di tipo **float**; staccare la variabile della durata dall'ingresso del testo e collegarci invece time to print; dare a time to print il valore **5**, che dovrà essere stampato a schermo. Anche la Duration va messa a 5. "Senza che andiamo a vedere poi ognuno": l'importante è che ognuno dica "stampa 5, sì".

Un esercizio guidato. Uno studente arriva in ritardo ("scusate, non riuscivo ad entrare"). A una studentessa il Print String stampa **0**, e Igor la guida passo per passo in condivisione dello schermo: fermare il gioco (lo **Stop**, o il tasto **Esc**); creare la variabile dal + di Variables nel pannello a sinistra; chiamarla time to print; cambiarle il tipo per farla **verde**, cioè float ("bravissima"); prenderne il GET; staccare il vecchio collegamento con Alt e attaccare time to print; compilare ("in alto a sinistra"); poi, con la variabile selezionata, guardare nei **Details** a destra: "vedi, adesso è apparsa. In fondo". In fondo ai Details c'è il **valore predefinito**, ed è lì che si mette il 5. Lo 0 dipendeva proprio da questo: il valore predefinito di un float appena creato è zero.

1.2.9 I dettagli di una variabile, e il *tuning*

Tutte le caratteristiche di una variabile si vedono nei **Details**, sia cliccando la variabile nel grafo, sia cliccandola nella lista Variables. Igor consiglia di impostare l'engine **in inglese** ("io sono più abituato all'inglese"), ma se si trova più facile l'italiano non c'è nessun problema.

Il nome, il tipo, la descrizione. Nei Details si possono cambiare il nome, il tipo e una **descrizione**. La descrizione non la vedrà il giocatore, ma "i nostri amici programmatori, o comunque colleghi": ci si può scrivere un piccolo commento, come "variabile da stampare nel Print". È un'informazione "per lo più organizzativa", "non un qualcosa che troverete spesso".

Pubblica e privata: Instance Editable. L'opzione **Instance Editable** rende una variabile **pubblica**. Gli studenti non conoscono la distinzione "in questo contesto". Una variabile **privata** esiste solo all'interno dell'actor; una **pubblica** si può vedere e modificare direttamente nei **Details** dell'actor piazzato nel livello. Uno studente chiede se sia come "globale e locale". "Non esattamente, perché globale è anche un'altra cosa che andremo a vedere": le variabili davvero globali si fanno con la **Game Instance**, un oggetto che **esiste finché esiste il gioco** e che può contenere variabili da "dare in pasto" a tutti gli actor che si vuole.

L'occhietto. Lo stesso risultato si ottiene con l'icona dell'**occhio** accanto alla variabile nella lista. Igor lo mostra: con l'occhio aperto, selezionando l'actor nel livello, nei Details compare la sezione **Default** con *time to print*; con l'occhio chiuso, dopo aver ricompilato, la voce scompare.

Il *tuning*. A che cosa serve? "Il design è sempre con me." Come designer, a Igor è capitato spesso di fare ***tuning***: la pratica di **aggiustare i valori**, "quanto danno fa, quanto fa di contrattacco", in generale i valori modificabili. Diceva al programmatore: "lasciami quei valori, che li devo controllare", cioè testare. E poi andava nei Details dell'actor e cambiava "questo, quell'altro, questo, quell'altro". Le variabili pubbliche servono proprio a questo: permettono di **interagire con l'editor stesso** senza toccare il grafo. Le cose private, invece, restano nel campo dell'actor.

Approfondimento: gli attributi di una variabile. Romero e Sewell, nel secondo capitolo, elencano tutti gli attributi che i Details mostrano per una variabile.

- **Variable Name** e **Variable Type**: il nome e il tipo. I Blueprint sono un linguaggio **fortemente tipizzato**: il tipo si sceglie alla creazione e non cambia durante l'esecuzione.
- **Instance Editable**: se attivo, **ogni copia** del Blueprint piazzata nel livello può avere un valore diverso; altrimenti tutte le copie (le *istanze*) condividono lo stesso valore iniziale. È la precisazione tecnica del "pubblico" di Igor.
- **Blueprint Read Only**: se attivo, i nodi non possono modificare la variabile.
- **Tooltip**: il testo che compare passando il mouse sulla variabile; è la "descrizione" di cui parla Igor.
- **Expose on Spawn**: la variabile si può impostare nel momento in cui l'actor viene *spawnato*.
- **Private**: se attivo, i Blueprint figli non possono modificarla.
- **Expose to Cinematics**: la rende visibile al Sequencer.
- **Category**: per raggruppare le variabili.
- **Slider Range** e **Value Range**: il minimo e il massimo, per lo *slider* e per i valori ammessi. Per il *tuning* sono preziosi: chi aggiusta i valori non può uscire dai limiti sensati.
- **Replication**: per i giochi in rete.
- **Default Value**: il valore iniziale; si può impostare solo dopo aver compilato, come si è visto nell'esercizio.

1.2.10 I tipi di variabile

Igor cancella tutto (seleziona e preme **Canc**, *Delete* sulla tastiera inglese) e passa a una slide sui tipi di variabile (tabella 1.2.2). "Immaginiamoci sempre delle scatolette, con dentro dei valori."

Booleana. "Qualcuno ha mai sentito parlare di booleane?" Qualche sì, qualche no. Una variabile **booleana** ha due valori possibili: **vero** o **falso**. "Dentro o c'è vero, o c'è falso, una delle due. Basta." Con un esempio dall'elettrotecnica, è "letteralmente lo *switch* della luce": spento è falso, acceso è vero; a livello di circuito, "è aperto o è chiuso".

Byte, Integer, Integer64. Il **byte** è un valore numerico intero compreso tra 0 e 255. L'**integer**[9], o intero, è un numero **senza la virgola**: "1, 2, 3, 4, 5, 6, 7, 8, senza virgola, senza niente", compreso tra -2^{31} e 2^{31} ("possiamo anche non capire qual è il numero preciso, perché tanto è difficile che ci arriviamo"). L'**integer64** arriva a 64 bit, tra -2^{63} e 2^{63}.

[9] Nella trascrizione "gli integrali, i numeri integrali".

Float. Il **float** è il valore numerico **con i decimali**.

Name, String, Text. Il **name** è un testo usato principalmente per gli **identificatori**, per identificare degli elementi in Unreal. La **string** è un gruppo di caratteri **alfanumerici**, "numeri e lettere". Uno studente chiede la differenza: anche il name è una stringa? La stringa, risponde Igor, si usa per lo più nei **testi**; i name sono **nomenclature**: "il nome di un livello, il nome di un *asset*". "Tipo un'etichetta", propone lo studente: "bravissimo, è come un'etichetta", "quasi come un **tag**". Il **text**, infine, è un testo usato principalmente nella **UI**, la *user interface*: tutto ciò che è 2D e con cui l'utente interagisce, un termine usato "in tanti campi, anche coi siti".

Vector, Rotator, Transform. I **vettori** sono "molto importanti", essendo Unreal un ambiente tridimensionale. Un vettore può essere la **locazione**: "ci ricordiamo che quando abbiamo spostato i nostri cubetti e coni abbiamo visto quei tre valori cambiare, la X, la Y e la Z?" Un vettore è "la composizione" di questi tre: una variabile che contiene **tre valori**, X, Y e Z. Il **rotator** contiene gli X, Y e Z della **rotazione** ("poi faremo un po' di discorsi, perché esistono i quaternioni, eccetera"). Il **transform** contiene location, rotation e scala.

Object. Infine il tipo **object**: "volendo, noi potremmo creare una variabile di tipologia di un actor che abbiamo creato". È una variabile che contiene un **riferimento** a un actor (si vedrà con il Level Blueprint, sezione 1.2.14).

Tipo	**Colore**	**Che cosa contiene**
Boolean	rosso	vero o falso (*true*/*false*)
Byte	verde scuro	intero da 0 a 255 (8 bit)
Integer	verde acqua	intero da -2^{31} a $2^{31}-1$ (32 bit)
Integer64	verde acqua scuro	intero da -2^{63} a $2^{63}-1$ (64 bit)
Float	verde	numero con decimali; in Unreal Engine 5 i Blueprint lo trattano in precisione singola o doppia (*Double*)
Name	lilla	identificatore, "etichetta" (nome di un livello, di un *asset*)
String	rosa	gruppo di caratteri alfanumerici
Text	rosa scuro	testo per la UI, **localizzabile** (traducibile)
Vector	giallo	tre float X, Y, Z (per esempio una locazione)
Rotator	viola-lilla	tre float della rotazione: X (*roll*), Y (*pitch*), Z (*yaw*)
Transform	arancione	locazione, rotazione e scala
Object	blu	riferimento a un oggetto (per esempio un actor nel livello)

Table 1.2.2: I tipi di variabile della slide di Igor, con i colori dei loro pin nell'editor (i nomi dei colori sono approssimativi) e le integrazioni del libro di Romero e Sewell (i limiti esatti degli interi, *roll*, *pitch* e *yaw*, la localizzazione del Text).

Approfondimento: Double, Text e i tre angoli. Tre precisazioni dal secondo capitolo del libro. Primo: oltre al Float a 32 bit (sette cifre decimali di precisione) esiste il **Double** a 64 bit (sedici cifre); in Unreal Engine 5 l'editor mostra entrambi come "Float" e sceglie la precisione da solo. Secondo: il **Text** serve ai testi che saranno **localizzati**, cioè tradotti in altre lingue: è per questo che è il tipo della UI, mentre String è il testo "di servizio" e Name l'identificatore. Terzo: i tre valori del **Rotator** hanno un nome. X è il ***roll*** (il rollio, la rotazione attorno all'asse X, quello "in avanti"), Y il ***pitch*** (il beccheggio, attorno all'asse Y, alzare o abbassare il muso), Z lo ***yaw*** (l'imbardata, attorno all'asse Z verticale, girare a destra e a sinistra). Sono i nomi che compariranno nei nodi della piattaforma (sezione 1.2.21). I **quaternioni** a cui accenna Igor sono un altro modo, matematicamente più robusto, di rappresentare una rotazione: evitano il problema del *gimbal lock*, in cui due dei tre assi finiscono per coincidere e si perde un grado di libertà. Unreal li usa al suo interno, ma nei Blueprint si lavora quasi sempre con i Rotator.

1.2.11 Le booleane e il Branch

"Adesso vi faccio vedere." Igor crea una variabile, la chiama "letteralmente" **bool**, e le dà il tipo booleano.

GET e SET in un gesto. "Piccola cosetta": invece di scegliere dal menu, si può fare un GET "velocissimamente" trascinando la variabile nel grafo con **Ctrl** premuto, e un SET trascinandola con **Alt**. Il **GET** è il pin che dice "questa è la scatoletta che c'è dentro qualcosa, prendi il valore da qui". Il **SET** è diverso: ha un **nodo di esecuzione**, a differenza del GET, e ha una **casella di spunta**[10]: cliccandola si imposta il valore. "Col SET sto letteralmente impostando il suo valore": spunta, vero; niente spunta, falso.

Stampare una booleana. Igor rifà un Print String con Duration 10 ("sennò è troppo poco"), collega il pin della booleana al testo e va in Play: in alto a sinistra compare **false**. Mette la spunta, rifà Play: **true**. Il Print String riceve, tramite la stringa, il valore della variabile.

A che cosa serve sapere se è vero o falso? "La domanda magari è: ok, ma che ci serve?" Serve perché si possono usare delle **condizioni**. "Qualcuno ha mai sentito parlare di *if*, *else*?" Edoardo sì: "ho studiato informatica alle superiori". "Grande, perfetto." L'**if-else** è una condizione, e in Unreal non si chiama così: tenendo premuto **B** e cliccando nel grafo compare il **Branch**, "cioè due percorsi". È "un **bivio**": "se questa condizione che abbiamo qua è vera, vai per questo percorso; se questa condizione è falsa, vammi per un altro".

Il primo Branch. Igor collega il Branch dopo il BeginPlay e porta la variabile bool nel pin rosso **Condition**. Sull'uscita **True** mette un Print String con "la variabile è vera"; per l'uscita **False** copia il nodo ("per copiare qualcosa, Ctrl+C, Ctrl+V, classico") e scrive "la variabile è falsa". Con la variabile falsa, in Play esce "la variabile è falsa"; con la spunta, "la variabile è vera" (figura 1.2.4).

I binari del treno. "Letteralmente sta prendendo un bivio: o vado di qua, o vado di là." Igor lo paragona ai **binari di un treno**: "se la leva è a destra, vado a destra; se la leva è a sinistra, vado a sinistra. E la nostra leva è la nostra variabile". Il bool è molto importante perché permette di avere delle condizioni, e quindi gli attori "**pensano**": hanno la "capacità di scelte", "ah, ok, allora prendo questa strada perché ho questa variabile di questo tipo".

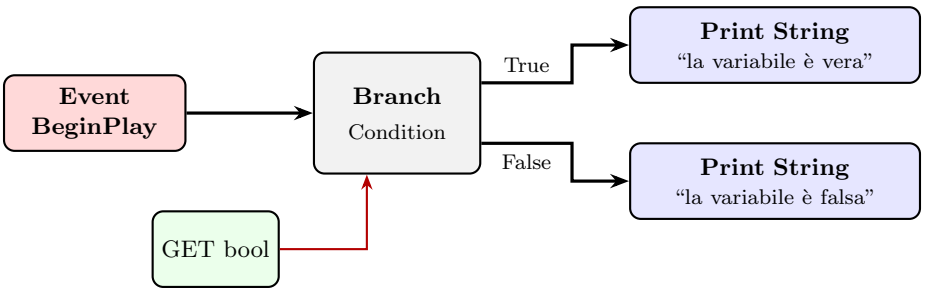


Figure 1.2.4: Il Branch è un bivio: segue l'uscita True o l'uscita False a seconda del valore della condizione. "La nostra leva è la nostra variabile."

Si può accedere al codice? Uno studente chiede se sia possibile accedere anche al codice, "immagino che sia C". Sì, è possibile, "però non lo faremo". Con il C++ si possono definire anche i blocchi stessi, creare blocchi di tipo diverso; ma "di base, su Blueprint possiamo farlo allo stesso modo anche noi". Chi vuole può andare a vederlo; e c'è chi va "proprio nel profondo della programmazione", come gli *engine programmer*.

[10] Nella trascrizione "un TIL": è il *tick*, la spunta della casella.

Allineare i nodi. "Una piccola cosa, se vi piace l'ordine": si selezionano due nodi e si preme **Q**, e Unreal li allinea.

Approfondimento: Branch e operatori relazionali. Romero e Sewell presentano il Branch nel secondo capitolo, parlando del **percorso di esecuzione**: i pin bianchi sono i **pin di esecuzione**, gli altri, colorati, i **pin di dati**; l'esecuzione parte da un evento rosso e segue il filo bianco da sinistra a destra fino all'ultimo nodo. Alcuni nodi, come il Branch, **controllano il flusso**: decidono quale strada prendere. Il loro esempio è un personaggio con una booleana *Has Shield*: se è vera, il Branch imposta *Shield Value* a 100; se è falsa, a 0. Nella lezione la condizione è sempre una variabile booleana; nella pratica, spesso, è il risultato di un **operatore relazionale**: i nodi $<$, $\leq$, $>$, $\geq$, $=$, $\neq$ confrontano due valori e restituiscono una booleana. L'esempio del libro: quando un oggetto subisce danni, un Branch controlla se *Health* ≤ 0, e in quel caso l'oggetto viene distrutto; se no, non succede niente, perché all'uscita False non è collegato nulla. Quanto alla scorciatoia: nel menu che si apre con il tasto destro su una selezione di nodi c'è la voce **Alignment**, con allineamenti in alto, al centro, in basso, a sinistra, a destra, e **Straighten Connection(s)**, che raddrizza i fili; il tasto **Q** è la scorciatoia di quest'ultima. Il libro ricorda anche le altre scorciatoie "tasto + clic": **B** per il Branch, **S** per il nodo Sequence, **D** per il Delay, **F** per il For Each Loop (tabella 1.2.4).

1.2.12 Gli operatori logici: AND, OR, NOT

Igor cancella i nodi di prova ("tanto non ci sono problemi") e passa agli **operatori logici**. "Qualcuno ha mai sentito parlare di AND, OR, NOT?" Sì.

Il gioco dei tubi. A che cosa servono? Una variabile booleana ha i suoi valori, vero e falso; "però mettiamo conto che ne abbiamo altre", e bisogna metterle in **comparazione**. L'esempio è "il gioco dei tubi", l'enigma in cui bisogna girare i tubi nel verso giusto per collegarli. Se ci sono due tubi, per risolvere l'enigma un tubo deve essere girato nel modo giusto **e** anche l'altro: sono **due condizioni** da prendere insieme. "Come facciamo a metterle insieme?"

I nodi. Igor crea una seconda booleana, **bool 2**. Gli operatori si cercano scrivendo **OR Boolean**, **AND Boolean** e **NOT Boolean**. "Questi sono i tre più usati"; ce ne sono altri, come il NOR, "ma sono un attimo da vedere poi più avanti".

OR. L'OR dice: "se almeno un valore è vero, allora la condizione finale è vera". Con bool vera e bool 2 falsa, collegato alla condizione del Branch, esce "la variabile è vera". È "un modo per sommare le variabili tra *true* e *false*", "una specie di somma, chiamiamola così": "letteralmente dice: o questa, o questa".

AND. L'AND, con gli stessi valori, vero e falso, dà "la variabile è falsa", "perché AND vuol dire *questa e questa*": tutte le condizioni devono essere vere perché il risultato sia vero. Igor mette vera anche la seconda e... non succede niente: "scusate, questo non l'ho collegato, sono io". Collegato, esce "la variabile è vera". Ricapitolando: se tutte e due sono vere, è vero; se una sola è falsa, "tutto quanto è falso"; se tutte e due sono false, è falso ("doppio falso").

NOT. Il NOT ha un solo ingresso e "letteralmente **inverte**": se la variabile è vera diventa falsa, se è falsa diventa vera. Collegato a una variabile vera, dà falso. "Non lo useremo tanto." La tabella 1.2.3 riassume i tre operatori.

Approfondimento: NOR, NAND, XOR, e perché l'OR è una "somma". Il libro completa la lista degli operatori logici dei Blueprint con tre combinazioni. Il **NOR** (NOT + OR) è vero solo se entrambi gli ingressi sono falsi; il **NAND** (NOT + AND) è falso solo se entrambi sono veri; lo **XOR**, l'OR esclusivo, è vero se i due ingressi sono **diversi**. L'esempio del libro per l'AND: un Print String che si esegue solo se *Health* > 70 **e** *Shield Value* > 50, con due operatori relazionali che alimentano un AND. Quanto alla "somma" di Igor, è un'intuizione che ha un nome: nell'**algebra di Boole**, da cui il tipo *boolean* prende il nome, l'OR si

A	B	A OR B	A AND B	NOT A	A NOR B	A NAND B	A XOR B
falso	falso	falso	falso	vero	vero	vero	falso
falso	vero	vero	falso	vero	falso	vero	vero
vero	falso	vero	falso	falso	falso	vero	vero
vero	vero	vero	vero	falso	falso	falso	falso

Table 1.2.3: Le tabelle di verità degli operatori logici. Le prime tre colonne di risultati sono gli operatori della lezione; le ultime tre sono un'integrazione dal libro di Romero e Sewell.

scrive come una somma ($A + B$) e l'AND come un prodotto ($A \cdot B$). Se si leggono vero e falso come 1 e 0, l'AND si comporta esattamente come una moltiplicazione (basta uno 0 per fare 0), e l'OR come una somma in cui "1 + 1" resta 1.

1.2.13 Secondo esercizio: due Branch, OR e AND

"Allora facciamo un piccolo esercizio."

La consegna. All'Event BeginPlay si collega un **SET** della variabile bool (quello che si ottiene trascinandola con Alt), e il suo valore va alla condizione di un **primo Branch**. Su ciascuna uscita del primo Branch va un **secondo Branch**: su True un Branch la cui condizione è **bool OR bool 2**; su False un Branch la cui condizione è **bool AND bool 2**. Solo se il secondo Branch è vero si stampa, e invece di "la variabile è vera" si stampa **il valore finale** del calcolo, collegando l'uscita dell'operatore all'In String (figura 1.2.5). "Ce la fate?" Cinque minuti.

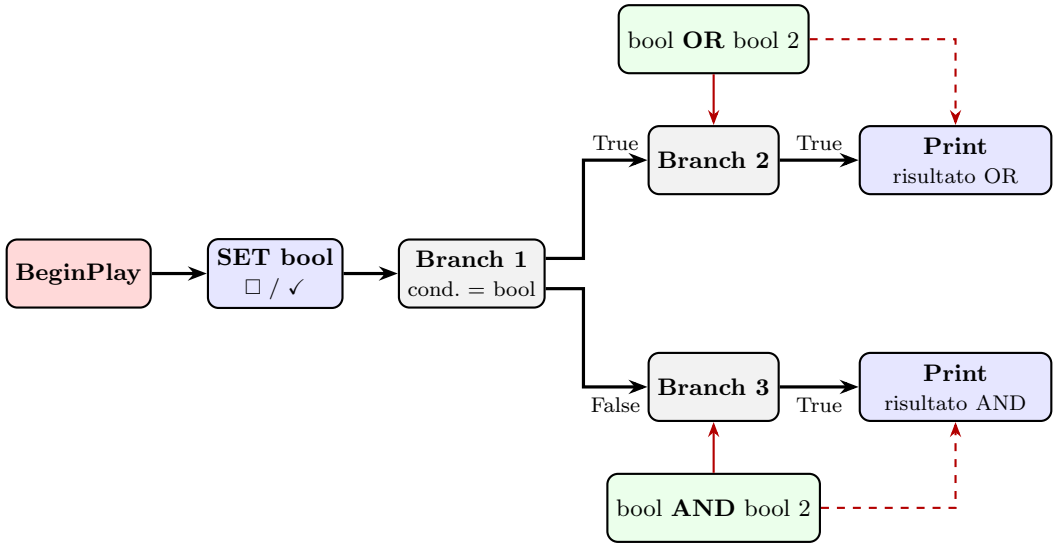


Figure 1.2.5: Il secondo esercizio. Le frecce nere sono l'esecuzione, quelle rosse i valori booleani (tratteggiate: il risultato dell'operatore, convertito in testo e stampato). Le uscite False dei Branch 2 e 3 non sono collegate: in quei casi non si stampa niente.

Le domande durante l'esercizio. Lo studente arrivato in ritardo chiede come si crea il Print String: tasto destro, Print String. Un altro si è perso gli operatori: si scrivono proprio "AND Boolean", "OR Boolean", "NOT Boolean". Una studentessa non ricorda come si prende "l'oggetto con il pallino rosso" (la variabile): "mettilo qua dentro, ti appare in automatico". Trascinando la variabile direttamente sul pin, Unreal crea da solo il GET.

La spunta che non c'entra. Un'altra studentessa ha un problema: sul primo Branch, nella condizione, ha una **spunta** che Igor non ha. Il pin Condition non è collegato all'**uscita** del SET. "Devi collegare questo pallino a questo pallino." Igor spiega che cosa è successo, con una dimostrazione. Il nodo SET ha a destra un pin di uscita: "è il valore che io gli ho impostato", che esce dal nodo "come un collegamento". Il Branch ha a sinistra "un posto in cui posso mettere quel valore". Se il pin non è collegato, la casella di spunta del Branch è una **condizione interna al Branch**, che non ha niente a che vedere con la variabile: "la porta si aprirà secondo bool, non secondo questa condizione qua". Igor mette la variabile a vero e la spunta del Branch a falso: il Print String collegato al True "non mi stampa niente", "perché questa qua non è relativa a questa". Collegando il pin, stampa. ("Stavo impazzendo perché non mi stampava niente.")

I "what if". Poi Igor cerca "un candidato" per una piccola verifica: "se io adesso vi cambio un po' i parametri, dove andiamo a finire con tutte le linee?" Si offre uno studente.

- **bool vero, bool 2 falso.** "Siccome è *true*, finisce sopra, sul primo Branch" (il Branch 2 della figura). "Controlla se tra bool e bool 2 uno dei due è vero. In questo caso lo è, perché hai settato bool come vero. Il secondo non è importante." Quindi va su True e stampa il valore. "Perfetto. Netto, liscio."
- **bool falso.** "False scende giù e controlla se entrambi, bool e bool 2, sono veri. In questo caso bool è falso, quindi anche qui il secondo valore non è importante, perché uno dei due è falso. Il risultato dell'AND è falso e non stampa niente." "Esatto, perfetto."

La morale: "indipendentemente dalle condizioni, possiamo cambiare i binari di un percorso di programmazione". È "una delle cose più principali", e serve molto spesso per **diversificare i comportamenti** dei Blueprint. Igor chiede ancora se ci sono domande, e decide di non cancellare il grafo ma di lasciarlo lì, scollegato. "Sono stato chiaro, dai."

Approfondimento: la valutazione "pigra". Lo studente coglie, senza nominarlo, un principio importante: in un OR, se il primo ingresso è vero il secondo "non è importante"; in un AND, se il primo è falso il secondo non cambia il risultato. In molti linguaggi di programmazione questo si chiama *short-circuit evaluation* (valutazione a corto circuito): il secondo operando non viene nemmeno calcolato. Nei nodi Blueprint, attenzione, i due ingressi vengono comunque calcolati entrambi; ma il ragionamento sul *risultato* è lo stesso, ed è quello che serve per prevedere dove andrà l'esecuzione.

1.2.14 Il Level Blueprint e la piattaforma

"Adesso andremo a fare una piccola cosa direttamente nel nostro bellissimo Blueprint." L'obiettivo è una **piattaforma** che **ruota**, cioè si inclina, "a seconda di quanto premiamo" dei tasti (D, A, e così via).

La piattaforma. Si prende un **cubo** dal pannello Place Actors, lo si mette in scena e lo si allarga "quanto volete", fino a farne una piattaforma.

Il Level Blueprint. "Vi farò conoscere il bellissimo mondo del **Level Blueprint**." In Unreal esiste anche un Blueprint **interno al livello**: quando si apre quel livello, al suo interno ci sono delle regole ("spawna qualcosa", eccetera). Lo si apre dall'icona dei Blueprint nella barra in alto: **Open Level Blueprint**. Una studentessa si è persa ("scusa, come lo apriamo?"), e Igor le mostra l'icona.

Un evento di input. Si è parlato di eventi con le loro condizioni; uno dei più semplici è l'**input**. Tasto destro nel grafo, si scrive **W** e si sceglie la voce con "l'immaginetta della tastierina" e la W: è un evento che scatta **alla pressione del tasto W**.

La referenza. Ma come si dice al grafo che deve agire su *quella* piattaforma? Si crea una **referenza**. Con il cubo selezionato nel livello, nel Level Blueprint si fa tasto destro e si sceglie **Create a Reference to Cube94** (il cubo di Igor si chiama Cube94, e lo si ritrova con quel nome nell'Outliner). "Questo può essere fatto solo nel Level Blueprint, ve lo dico."

Set Actor Rotation e i suoi pin. Dalla referenza si trascina un filo e si sceglie **Set Actor Rotation**. Il nodo ha tre pin di dati:

- un pin **blu**, il **Target**: un *Actor Object Reference*, cioè una variabile che si riferisce a un actor, a uno qualunque degli elementi della *viewport*, "che abbiano programmazione o meno". È l'actor su cui il nodo agisce;
- un pin **lilla**, **New Rotation**: un **rotator**, la composizione degli assi X, Y e Z della rotazione;
- un pin **rosso**, un **bool**: **Teleport** (tornerà utile con la fisica, sezione 1.2.19).

Il Player Start. Per vedere la piattaforma serve un punto da cui partire. "Prendete il **Player Start**": lo si trova nel Place Actors, o cercandolo, ed è "un controllone" (l'icona di un *joystick*). Lo si mette davanti alla piattaforma e lo si gira verso di essa.

Consume Input. Selezionando l'evento W, nei Details c'è una casella interessante, **Consume Input**. Se è spuntata, il tasto W "sarà dedicato solo a questo evento": il giocatore non potrà più usarlo per muoversi in avanti. Senza la spunta si muoverebbe "avanti e indietro"; "a noi interessa stare fermi", quindi la spunta resta.

Quale asse, e in che verso. La piattaforma va ruotata attorno all'asse **X**. In positivo o in negativo? Igor lo verifica nella *viewport* ruotando il cubo: in un verso la rotazione è negativa, nell'altro positiva. "Per come mi regolo", W manda la piattaforma in avanti, quindi si piega in un verso, e indietro nell'altro. Igor scrive 5 nella X del New Rotation e prova.

1.2.15 La mobilità: Static, Stationary, Movable

Non funziona, "giustamente", e Unreal mostra un avviso: "ricordate che questo che mi è appena uscito è un problema che Unreal ci darà sempre". L'actor non ha la capacità di essere **Movable**: non si può muovere nel mondo.

Dove si trova. Nel *transform* dell'actor, sotto location, rotation e scala, c'è la **Mobility**, con tre valori: **Static**, **Stationary** e **Movable**. È "la tipologia di oggetto" rispetto a come reagisce al mondo, e riguarda "soprattutto il rendering".

- **Static.** L'oggetto sa che non si muoverà. Quando si calcolano le luci, la sua ombra viene calcolata una volta e poi è come **dipinta**: "è dipinta". "Per alleggerirsi la cosa", l'engine dice: è un oggetto statico, "basta che mi calcolo una volta l'ombra e basta".
- **Stationary.** Una via di mezzo: le ombre vengono messe in *cache*[11], l'oggetto può essere cambiato, ma non può essere mosso.
- **Movable.** Totalmente **dinamico**.

La piattaforma va messa su **Movable**, e allora ruota di 5 gradi. Igor prova con valori più grandi ("facciamo tipo 25") per vederlo meglio.

Approfondimento: la mobilità secondo il libro. Romero e Sewell incontrano lo stesso problema nel quinto capitolo, quando un bersaglio cilindrico deve cominciare a muoversi: bisogna prima portare la sua Mobility da Static a **Movable**, nei Details sotto il TRANSFORM. E aggiungono una nota importante: gli actor di base piazzati nel mondo sono **Static per impostazione predefinita**, e gli oggetti statici sono "significativamente meno costosi" da renderizzare; per questo Static dovrebbe essere la scelta predefinita per tutto ciò che non è interattivo, per tenere alto il *frame rate*. Stationary ha senso soprattutto per le **luci**: una luce Stationary non si sposta, ma se ne possono cambiare colore e intensità durante il gioco, e parte della sua illuminazione è precalcolata. Movable va riservato agli oggetti che devono davvero muoversi.

[11] Nella trascrizione "vengono chesciate": con ogni probabilità "cachate", dall'inglese *cached*.

1.2.16 Ruotare di 5 gradi a ogni pressione: Split Struct Pin

Funziona una volta sola. "Però che succede? Guardate un po'." Se si preme di nuovo W, la piattaforma non fa niente. Il problema è che la si sta impostando a un **valore preciso**: Set Actor Rotation dice "mettiti a 25 gradi", e la seconda volta la piattaforma risponde "ci sto già, a 25". Serve invece una rotazione che **parta da quella attuale**.

Prendere i valori dell'actor. Igor mostra che dalla referenza di un actor si possono prendere **tutti** i suoi valori: la location, la rotation, la scala, la *mesh*, i materiali ("che poi vedremo"). "È andare nella scatola di quell'*asset*": prendo quel valore, e poi lo modifico. Qui serve **Get Actor Rotation**, la rotazione attuale, "che si aggiorna ogni volta".

Split Struct Pin. La rotazione è data da tre valori, X, Y e Z, e se ne vuole modificare uno solo. Con il tasto destro sul pin del Rotator si sceglie **Split Struct Pin**: il pin si divide nei suoi tre valori, "e guardate un po', la cosa bella è che sono cambiati di colore". Sono diventati **verdi**, "perché sono tre valori float". Uno studente chiede come si fa a trovare i tre valori; Igor rifà il gesto all'indietro (**Recombine Struct Pin**) e poi di nuovo avanti: tasto destro, Split Struct Pin. Si fa lo stesso sul pin New Rotation del Set Actor Rotation, che altrimenti, lilla, non accetterebbe un float.

Il grafo. Si prende la X di Get Actor Rotation, le si somma 5 con un nodo +, e il risultato va nella X del Set Actor Rotation (figura 1.2.6, a sinistra). Ora la piattaforma si inclina di 5 gradi a ogni pressione di W: "posso aggiungere quanto me ne pare", a parte che a un certo punto la piattaforma **compenetra** il pavimento. "Prima non andava bene e adesso invece sì", perché ogni volta si parte dalla rotazione attuale. Con un Print String collegato alla X, a ogni pressione compaiono i valori **0, 5, 10, 15, 20, 25, 30, 35**: "si aumenta sempre".

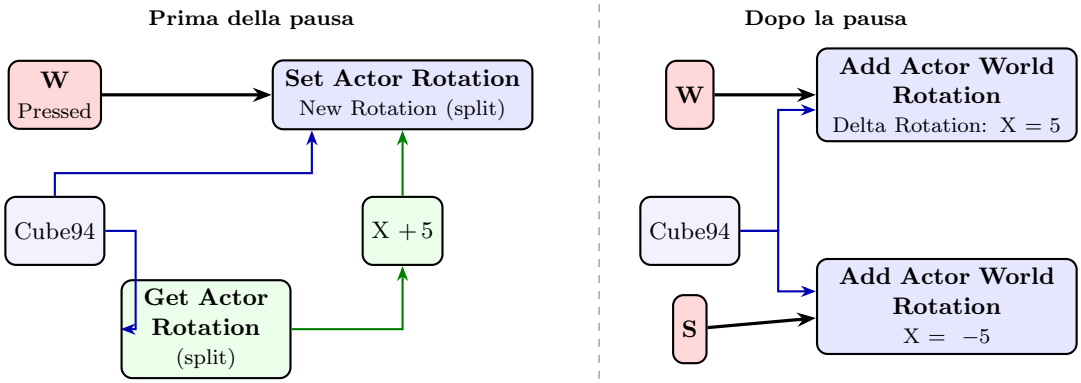


Figure 1.2.6: La piattaforma comandata da tastiera, nel Level Blueprint. A sinistra la prima versione: si legge la rotazione attuale, si aggiungono 5 gradi alla X e la si reimposta. A destra la versione adottata dopo la pausa: Add Actor World Rotation aggiunge direttamente 5 gradi (o −5) alla rotazione, nello spazio del mondo. A e D funzionano allo stesso modo sull'asse Y.

1.2.17 Il problema di Giulia, e la pausa

"Un attimo, un attimo." A Giulia la piattaforma si muove **una volta sola**. Igor segue il problema in condivisione dello schermo, e il *debug* dal vivo è istruttivo anche perché non arriva a una risposta.

- La piattaforma è **Movable**? Sì.
- Il grafo è uguale a quello di Igor? Sembra di sì: Cube71 (il suo cubo), Set Actor Rotation, il più, il Get Actor Rotation. Ruotata a mano con E nella *viewport*, la piattaforma ruota normalmente.

- Forse un problema di **collisioni** con il pavimento? Igor fa alzare la piattaforma: niente.
- Igor fa rifare alcuni collegamenti, ricostruire il nodo, splittarlo di nuovo, eliminare un nodo superfluo ("questo get actor... l'actor location sopra?", "è uguale, lo elimino"), ricollegare il Target al Cube71. Niente.
- La **camera**: Giulia vede "la piattaforma sbilenca e basta". Igor fa girare il Player Start di 90 gradi e mettere la piattaforma davanti. Niente.
- Ha cliccato nella finestra di gioco, perché riceva l'input? Sì, "solo una volta". Il Consume Input c'è.
- Un controllo più fine: Igor chiede di selezionare il nodo Set Actor Rotation e premere **F9**, che vi mette un *breakpoint*. In Play il nodo viene effettivamente raggiunto: "funziona a meraviglia". Eppure la piattaforma non ruota oltre la prima volta.
- Infine Giulia si accorge che nel livello **manca il Player Start** ("mancano delle robe che ho cercato di mettere, ma ho perso"). Igor fa rimettere il Player Start dal Place Actors.

A quel punto è ora della pausa: "facciamo pausa, io intanto cerco di capire un po' il problema che c'è, Giulia". Venti minuti.

La soluzione, durante la pausa. Durante la pausa Igor prova una cosa diversa: al posto di Set Actor Rotation, con il Get e il più, usa **Add Actor World Rotation**, con la referenza come Target e 5 nella X ("senza il più, senza niente, solo questi due"; -5 per la S, che Giulia non ha ancora). "Sì, adesso lo fa. Perfetto." "Ma era un errore mio?" "No, no, tranquillo." La spiegazione di Igor è che il problema dipende dagli assi: con Set Actor Rotation si stava ruotando "sulla sua relativa", ed è meglio ruotare sull'asse **globale**, quello del mondo. Ma su un punto Igor è onesto: "il fatto che non prenda l'input a ogni W, quello non te lo so dire, perché a tutti quanti lo prendeva". "Non so se magari è un *flash* o un qualcosa, non ne ho idea. Ti devo dire la verità." Per capirlo dovrebbe guardare con calma le impostazioni dell'editor di Giulia; intanto si semplifica. "Visto che ho risolto la cosa, vado a farmi il caffè", dice Giulia. "Ottima idea. Pure io."

Approfondimento: Set, Add Local, Add World, e il *breakpoint*. Unreal ha più nodi per ruotare un actor, e la differenza è esattamente quella tra assi locali e globali vista nella prima lezione.

- **Set Actor Rotation** imposta una rotazione **assoluta**: "mettiti così". Per ottenere un incremento bisogna leggere la rotazione attuale e sommarci qualcosa, come nella prima versione.
- **Add Actor Local Rotation** aggiunge una rotazione attorno agli assi **locali** dell'actor, quelli che ruotano con lui.
- **Add Actor World Rotation** aggiunge una rotazione attorno agli assi **del mondo**, che restano fissi: premere W inclina sempre la piattaforma "in avanti" rispetto alla scena, comunque sia già inclinata.

Perché la prima versione non funzionasse sul computer di Giulia la lezione non lo stabilisce, e il capitolo non è in grado di dirlo; il passaggio sul nodo "get actor location" da eliminare fa pensare a un collegamento sbagliato (la *location*, gialla, al posto della *rotation*, lilla), ma è solo un'ipotesi. Resta istruttivo il metodo. Il ***breakpoint*** (F9 su un nodo selezionato) ferma l'esecuzione quando il gioco arriva a quel nodo e apre l'editor del Blueprint lì, mostrando i valori che passano nei fili: è il modo più diretto per sapere se un nodo viene davvero eseguito e con quali dati. Lo stesso F9 lo toglie. Il Print String, che Igor usa per tutta la lezione, è l'altro strumento: stampa i valori a schermo senza fermare il gioco.

1.2.18 Dopo la pausa: Add Actor World Rotation e i tasti W A S D

"Ora ci siamo, ragazzi?" Visto il problema di Giulia, Igor semplifica l'esercizio per tutti, con il blocco che ha usato durante la pausa: **Add Actor World Rotation**, "nel senso che andiamo semplicemente a mettere

una rotazione nel mondo, non dipendente dalla sua relatività". Prima, spiega, si impostava la rotazione relativa all'asse dell'actor; doveva essere relativa al mondo. La consegna: tenere solo i due blocchi (l'evento e Add Actor World Rotation), levare tutto il resto, **splittare** il pin della rotazione e scrivere 5 nella X (figura 1.2.6, a destra). "Praticamente qui si dovrebbe tranquillamente muovere, molto più semplice."

S, A e D. Igor ha aggiunto anche la **S**: "se una è 5, l'altra è -5". E per destra e sinistra, A e D? "Su che asse ci dobbiamo muovere?" Lo si vede nella *viewport*: cliccando la piattaforma e guardando il gizmo, se W e S ruotano attorno alla X, A e D devono ruotare attorno alla **Y**. "E vorrei questo piccolo esercizio: lo fate voi." A e D, "non è Donkey Kong, questo con la tastiera". "Mi raccomando, ricordatevi sempre che il **Target** è importante": senza collegarlo, il nodo non si riferisce a niente.

Allineare in verticale? Uno studente chiede se esista un tasto per allineare i blocchi in verticale. "Non lo conosco, devo dire la verità." (L'allineamento c'è, nel menu Alignment del tasto destro: si veda l'approfondimento della sezione 1.2.11.)

Qual è il verso di D? Un altro studente non capisce che valore mettere sulla D. Igor lo guarda insieme a lui sulla piattaforma: destra, nella sua vista, è "di là", opposto a lui. Ruotando a mano la piattaforma in quel verso, il valore che compare sulla Y è negativo: "siccome siamo in asse globale, devi andare in negativo sulla Y". La soluzione, "velocissimamente, perché se no non abbiamo tempo": gli stessi due blocchi, il Target collegato, la rotazione sulla Y, $+5$ per un tasto e -5 per l'altro. "E possiamo, wow, destra e sinistra."

1.2.19 La pallina, la fisica e il Teleport

"Qua sopra, una bella sferetta." In Unreal gli oggetti possono essere **fisici**. Si prende una **sfera** dal Place Actors e la si mette sulla piattaforma. In Play, però, "quella rimane lì, sta lì ferma": "non gliene frega assolutamente nulla". La sua fisica non è attiva.

Simulate Physics. Nei Details della sfera c'è la casella **Simulate Physics**. Spuntata, in Play la sfera **cade** e rotola: "abbiamo la nostra sferetta che si muove". Uno studente chiede dove si attiva: basta cliccare la sfera e cercare la voce. Un altro condivide lo schermo: "una cosa che va, finalmente".

La pallina scagliata. C'è però una cosa particolare: muovendo la piattaforma, la sfera viene **scagliata in alto**, oltre a uscire dai bordi. Che esca si risolve "semplicemente mettendoci un muro, un muro invisibile" (lo si farà alla fine, sezione 1.2.22). Che venga lanciata, invece, dipende dal modo in cui la piattaforma si muove.

La freccia sotto il nodo, e il Teleport. Nel Level Blueprint, sotto il nodo Add Actor World Rotation c'è una **freccia**: cliccandola si aprono altri parametri. "Molto spesso blocchi del genere hanno queste tipologie di parametri interni, che sono nascosti un po'." Uno è **Teleport**. La spiegazione di Igor: quando c'è uno spostamento nello spazio, l'engine lo fa avvenire sui *tick*, sui *frame*, e "pum!": lo spostamento porta con sé il **fattore fisico**, "abbiamo proprio là il colpo fisico", che spinge via la pallina. Se invece lo si **teletrasporta**, "è come fare un *pop-up*" dello spostamento della piattaforma: l'effetto fisico si annulla, e la pallina non viene scaricata via. Igor abbassa un po' la pallina, spunta Teleport su tutti i nodi e va in Play: "la nostra palletta non verrà influenzata dalla particolarità fisica della nostra piattaforma".

Approfondimento: che cosa fa davvero Teleport. Quando Unreal sposta o ruota un oggetto che partecipa alla simulazione fisica, deve decidere come trattare quel movimento. Con **Teleport** disattivato, lo spostamento è considerato un movimento "vero": il motore fisico ricava dalla differenza tra la posizione vecchia e quella nuova una **velocità**, e la trasmette agli oggetti a contatto. Una piattaforma che ruota di 5 gradi in un solo *frame* ha, per il motore fisico, una velocità enorme, e la pallina riceve un colpo proporzionato. Con **Teleport** attivo, lo spostamento è un salto istantaneo, senza velocità associata: la pallina si ritrova sulla piattaforma nella nuova posizione e scivola per gravità, senza essere colpita. Accanto a Teleport, nella

stessa sezione nascosta, c'è spesso **Sweep**: se attivo, l'oggetto si ferma quando nel tragitto incontra qualcosa che lo blocca, invece di attraversarlo.

I bug della fisica. Mentre gli studenti provano, uno trova "un *bug*": se si porta la pallina in un angolo e si premono insieme W e A, cioè le direzioni opposte all'angolo, la pallina **finisce sotto** la piattaforma. (Un altro nota che E porta il giocatore in alto e Q in basso: anche durante il gioco ci si muove come nell'editor.) Igor risponde che "possono esserci dei *bug* fisici", perché la **fisica è emergente**: spostare le cose con la fisica dipende da come la si gestisce, e "probabilmente ne potresti trovare non pochi". Ci sono engine proprietari, "come quello di Valve", che gestiscono fisica e gravità "in un modo assurdo", "con molto matematichese", e sono ottimizzati per le azioni che quei giochi fanno. Unreal, "essendo più generico come sistema fisico", può dare problemi fisici "molto più evidenti, molto più spesso".

"Non va niente". Uno studente ha messo anche la S, ma premendo W, A, S, D non succede niente. Igor gli fa cliccare la piattaforma: non è **Movable**. "È la terza" delle tre opzioni. Rimessa su Movable, funziona; manca solo il Teleport.

Approfondimento: il motore fisico di Unreal. Unreal Engine 5 usa un motore fisico proprio, **Chaos**, che ha sostituito PhysX della quarta versione. È pensato per essere generale: corpi rigidi, distruzione, tessuti, veicoli. Il caso della pallina che finisce sotto la piattaforma ha un nome, ***tunneling***: se in un *frame* un oggetto si sposta più dello spessore di ciò che dovrebbe fermarlo, il motore fisico non vede mai il contatto e l'oggetto "passa attraverso". Con il Teleport attivo la piattaforma salta da una rotazione all'altra senza che la fisica la segua, e il rischio aumenta. Le difese tipiche sono oggetti più spessi, spostamenti più piccoli a ogni *frame* e la **CCD** (*Continuous Collision Detection*), un'opzione dei componenti fisici che controlla le collisioni lungo tutto il tragitto e non solo nelle posizioni finali.

1.2.20 L'Event Tick e i *delta seconds*

"Abbiamo il nostro bellissimo gioco della palla incredibile." Il passo successivo è comandare la piattaforma con il **mouse** invece che con W A S D. Prima, però, un evento nuovo.

Che cos'è un tick. "Sapete cos'è un *tick*?" È, semplicemente, "**un per *frame***". L'**Event Tick** è un blocco che esegue tutto ciò che c'è dopo di lui **a ogni *frame***: "ogni *tick*, ogni *tick*, ogni *frame*".

La piattaforma che gira da sola. Igor copia un Add Actor World Rotation, collega la referenza come sempre e lo attacca all'Event Tick: 5 gradi a ogni *frame*. "Non sto premendo nulla", e la piattaforma gira da sola, velocissima. Quanti sono gli *fps*? "In questo caso saranno tipo 100": la piattaforma ruota di 5 gradi cento volte al secondo.

I *delta seconds*. L'Event Tick ha un'uscita, **Delta Seconds**: "l'intervallo tra un *frame* e l'altro". Perché la velocità sia la stessa "indipendentemente dal numero di *frame*" (30, 60, 70, 100), bisogna **moltiplicare** il valore per i *delta seconds*. Si tira fuori un **Multiply**, si collega Delta Seconds e si scrive 5. "Vedete, è molto diverso da prima": la piattaforma gira molto più piano. Prima si aggiungevano 5 gradi a ogni *frame*; ora si moltiplica il tempo tra i *frame* per 5. E se il gioco girasse a 30, 60, 70, 100, 120 o 144 *fps*, la velocità sarebbe la stessa; senza la moltiplicazione, con un limite a 30 *fps* sarebbe più lenta che a 100.

"Un tick ogni 5 frame?" Uno studente prova a riformulare: "quindi in realtà non fa partire il programma ogni 5 *tick*, ma fa in modo che un *tick* sia ogni 5 *frame*?" "Non 5 *frame*: 5 **unità**", le unità in cui si muove la piattaforma; "è il float, fondamentalmente". Il *tick* resta uno per *frame*; cambia quanto si muove l'oggetto a ogni *tick*.

La regola. "Questa è una cosa da tenere tanto a mente, soprattutto con oggetti che si muovono." Se in un progetto si fa un oggetto in movimento senza *delta seconds*, "vi si ritorcerà contro": su una macchina con certe prestazioni andrà più lento, su un'altra più veloce. "Moltiplicatevi i *delta seconds* se muovete le cose, soprattutto nel tempo." Per il gioco della pallina, però, la rotazione continua non serve, "perché se no non ha *gameplay*": "sarebbe quasi un mulino".

Approfondimento: i conti. Con i numeri di Igor: senza *delta seconds*, 5 gradi per *frame* a 100 *fps* fanno $5 \times 100 = 500$ gradi al secondo, quasi un giro e mezzo; a 30 *fps* ne fanno 150. Con i *delta seconds*, a 100 *fps* ($\Delta t = 0{,}01$ s) ogni *frame* aggiunge $5 \times 0{,}01 = 0{,}05$ gradi, a 30 *fps* ($\Delta t \approx 0{,}033$ s) circa 0,167: in un secondo, in entrambi i casi,

$$\sum_{\text{frame in 1 s}} 5 \cdot \Delta t = 5 \cdot 1 \text{ s} = 5 \text{ gradi}.$$

Il 5, insomma, non è più "gradi per *frame*" ma **gradi al secondo**: è la "unità" di cui parla Igor rispondendo allo studente. Romero e Sewell, nel secondo capitolo, descrivono l'Event Tick proprio così (a 60 *frame* al secondo viene chiamato 60 volte al secondo) e, nel quinto, usano lo stesso prodotto velocità × *delta time* per muovere un bersaglio. Più avanti, presentando i **timer**, il libro li indica come alternativa all'Event Tick: tutto ciò che si collega al Tick viene eseguito a ogni *frame*, e ciò che non ha bisogno di aggiornarsi così spesso può essere eseguito a intervalli più lunghi.

1.2.21 La piattaforma comandata dal mouse

"Adesso sto staccando tutto, stacco tutto quanto. Voglio dare il movimento al mio mouse."

Evento o valore. Per il mouse ci sono due cose diverse con lo stesso nome. C'è l'**evento Mouse X**, che scatta "quando ho un input dal mouse X", e c'è il **valore Get Mouse X**, "proprio il valore del mouse X". Mouse X è il movimento del mouse a destra e a sinistra (Mouse Y, in alto e in basso). Invece di usare un evento, Igor fa "molto più semplice": resta sull'Event Tick, collega **Get Mouse X** al **Roll** (la X della rotazione) e **Get Mouse Y** al **Pitch** (la Y) dell'Add Actor World Rotation. La piattaforma si muove con il mouse; "è molto incontrollabile", ma si muove.

Assi scambiati, e invertiti. I due valori risultano scambiati, "perché dipende dalla *world location*", da come è orientata la piattaforma rispetto al mondo: si invertono X e Y, "ed è molto meglio". Poi c'è il verso. A Igor non piace che, portando il mouse in giù, la piattaforma si pieghi in quel modo: "lo vogliamo fare al contrario". "Qualcuno ha qualche idea per invertire un valore? In matematica, come si inverte un valore in negativo?" "Moltiplichi per meno 1." "Bravissimo." Si prende Get Mouse X, si aggiunge un **Multiply** per -1, "in modo tale che farà l'inverso". (Più tardi Igor aggiunge che chi preferisce i controlli invertiti, "se vi piace di più", può tranquillamente togliere il -1.)

Che cosa si è visto finora. "Quindi oggi abbiamo visto praticamente come si fanno" le operazioni: crearsi dei valori, moltiplicarli, metterli e poi assegnarli, in questo caso alla rotazione; volendo, si potrebbero assegnare anche nuove locazioni. Igor pensa per un momento di far muovere la pallina con un **Add Torque** ("no, quella è sempre rotazione"), e decide invece di migliorare la piattaforma: per ora la si può ruotare "quanto mi pare", anche capovolgerla.

Il Clamp. "Voglio spiegarvi uno degli operatori che ci permettono di controllare i nostri valori": il **Clamp** ("non Clamp Axis, scusate"; il nodo giusto è **Clamp (Float)**). Igor prova a disegnare su Zoom con uno strumento di annotazione, "ma a quanto pare non funziona" ("sarà per la prossima volta"), e lo spiega a voce: **clampare** un valore vuol dire dargli **un minimo e un massimo**, in modo che "viaggi su queste due linee" e non le superi mai. Igor mette un Clamp su entrambi gli assi, tra -1 e 1: il movimento diventa "più preciso".

"Perché valori così piccoli?" Una studentessa chiede perché proprio -1 e 1: lei aveva messo -45 e 45, "pensando ai gradi della rotazione". "Ah, i gradi! No." Si possono modificare a piacere, ma 45 "è un po' troppo". In quel punto del grafo, infatti, il Clamp non limita l'inclinazione: limita **quanto** la piattaforma può ruotare a ogni *frame*.

La versione con il limite d'inclinazione. "Allora, ragazzi, adesso abbiamo questo, ma non abbiamo finito." L'obiettivo vero è che la piattaforma non si inclini oltre un certo angolo. Il grafo (figura 1.2.7) diventa:

1. dalla referenza del cubo, **Get Actor Rotation**, la rotazione attuale;
2. invece dello Split Struct Pin, "una cosa carina che magari vi può servire": un nodo **Break Rotator**, "la stessa cosa di fare lo split", che divide il Rotator nei suoi tre valori;
3. si **somma** il valore del mouse all'asse corrispondente della rotazione attuale (la Y, per Get Mouse X): "lo stesso discorso di prima: prendere la sua rotazione e sommargliela con quella del mouse, in modo che si aggiorni continuamente";
4. la somma passa per un **Clamp** tra -20 e 20: "abbiamo clampato il valore, ma vogliamo che vada massimo a 20 o a -20";
5. un **Make Rotator** ricompone una rotazione nuova con i valori clampati;
6. un **Set Actor Rotation**, con il Target collegato e **Teleport Physics** attivo, "perché è lo stesso discorso" della pallina. Qui si torna a Set Actor Rotation, invece di Add Actor World Rotation, perché la rotazione nuova è già calcolata per intero.

La costruzione dal vivo. Costruire questo grafo in diretta non è lineare, e la trascrizione lo mostra. Igor a un certo punto pensa di moltiplicare anche qui per i *delta seconds*[12], poi toglie il Clamp tra -1 e 1, rompe i collegamenti, ne rifà altri, si accorge di un problema di inversione ("ah, c'è proprio un problema sull'inversione", "infatti, sospettavo"), e per capirlo stampa i valori. Alla fine la soluzione è la stessa di prima: **invertire X e Y**. "Perdonate, colpa mia." Con due Print String, uno per asse, si vede in alto a sinistra che il valore arriva a -20 "e rimane lì, fermo"; lo stesso a 20, su entrambi gli assi. La piattaforma "si piega solamente così".

La ricapitolazione. "Dopo tutto sto baradam di roba", che cosa si è fatto? Si prende Get Mouse Y (e Get Mouse X) e lo si **aggiunge continuamente alla rotazione attuale**: "adesso è 5, adesso è 7, adesso è 8, adesso è 9, adesso è 10", e si aggiunge sempre qualcosa, finché la somma non arriva a 20 o a -20. Lì il Clamp la ferma. "Stessa identica cosa vale per tutte e due le assi."

I problemi degli studenti. Uno studente ha bisogno di rivedere il grafo, e Igor lo ripercorre: Break Rotator della rotazione attuale; Get Mouse X sommato alla sua Y; un Clamp tra -20 e 20; un nuovo Rotator "formato dalla X e dalla Y". Lo studente ha un problema: il valore restituito dal suo nodo è **giallo**. Ha preso **Get Actor Location** invece di Get Actor Rotation: "*rotation*, non *location*". Il giallo è il colore dei vettori (la locazione), il lilla quello dei rotator.

Approfondimento: due usi del Clamp. La confusione sui "45 gradi" nasce da un punto sottile: lo stesso nodo Clamp fa due cose diverse a seconda di *dove* sta nel grafo. Nella prima versione il Clamp è applicato al **movimento del mouse** in un *frame*, che poi viene *aggiunto* alla rotazione: limita la **velocità** con cui la piattaforma si inclina (al massimo 1 grado per *frame*), ma non l'inclinazione totale, che può crescere senza fine. Nella seconda versione il Clamp è applicato alla **rotazione risultante**: limita l'**angolo**

[12] Il passaggio è quasi incomprensibile nella trascrizione ("facciamo così, che mi crea... i secondi e la moltiplichiamo per") e Igor toglie subito dopo il Clamp precedente; dalla ricapitolazione finale risulta che nel grafo definitivo i *delta seconds* non ci sono.

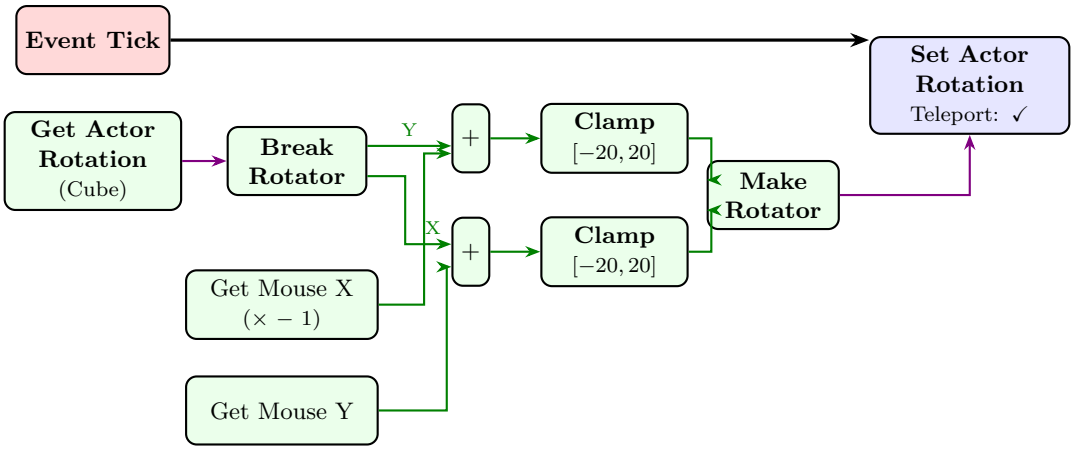


Figure 1.2.7: Il grafo finale della piattaforma comandata dal mouse (ricostruzione dalla ricapitolazione di Igor e di uno studente). A ogni *frame* si legge la rotazione attuale, le si somma il movimento del mouse, si limita il risultato tra -20 e 20 gradi e lo si reimposta. Quale movimento del mouse vada su quale asse, e con quale segno, dipende dall'orientamento della piattaforma rispetto al mondo: a lezione X e Y vanno scambiati. La referenza del cubo va collegata anche al Target del Set Actor Rotation (omesso nello schema).

(mai oltre 20 gradi), ed è lì che ha senso ragionare in gradi. In formule, se θ_t è l'inclinazione al *frame* t e m_t il movimento del mouse,

$$\text{prima versione: } \theta_{t+1} = \theta_t + \text{clamp}(m_t, -1, 1), \qquad \text{seconda: } \theta_{t+1} = \text{clamp}(\theta_t + m_t, -20, 20).$$

Una nota: Get Mouse X e Get Mouse Y restituiscono lo spostamento del mouse *nell'ultimo frame*, che è già una quantità "per *frame*"; per questo nel grafo finale non serve moltiplicare per i *delta seconds*. Make Rotator e Break Rotator, infine, fanno con dei nodi quello che Split e Recombine Struct Pin fanno sul pin: comodi quando lo stesso Rotator va letto o composto in più punti.

1.2.22 La vaschetta e il soffitto invisibile

"Adesso, se vogliamo fare un giochettino un po' più carino", e come esercizio per la prossima volta: allargare un po' la piattaforma e farci dei **piccoli percorsi** per la palla.

I muretti, e Attach To. Si costruiscono dei **muretti** con dei cubi scalati, prima sui bordi esterni, perché la pallina non esca, e volendo anche interni. Ma i muri devono muoversi con la piattaforma. Si selezionano i muri ("1, 2 e 3"), tasto destro, **Attach To**: si possono **attaccare** i cubi a un altro cubo, che diventa il loro *genitore*. In Play, "la nostra bellissima vaschetta è stata creata e segue i suoi movimenti". È la gerarchia di cui si parlava a inizio lezione: un oggetto agganciato a un altro ne eredita il *transform*.

La camera. "Ovviamente c'è da fissare un pochino la camera": il Player Start va alzato e inclinato verso il basso, finché la piattaforma non si vede bene. "Eccolo fatto. E abbiamo il gioco della palletta." Con Davide, la volta successiva, si vedranno i **materiali**, "così almeno potremo dare un tocco di colore a queste piattaformine".

I problemi, uno per uno. Gli ultimi minuti sono dedicati ai problemi degli studenti, risolti in condivisione dello schermo.

- **La piattaforma fa una capriola e torna in piano.** A due studenti, appena parte il gioco, la piattaforma fa "uno scatto strano" e poi non si muove più; oppure si muove, ma appena si smette di muovere il mouse "torna" in piano. Nel grafo c'è **Set Actor World Rotation** al posto di **Set Actor**

Rotation, "quello blu". Igor fa sostituire il nodo, ricollegare il Target e rimettere **Teleport Physics**. Funziona, e la pallina, per festeggiare, "se n'è scappata via".

- **I bordi non si attaccano.** "Si muove solo la base." Si selezionano i muri, tasto destro, **Attach To**, e poi si sceglie il genitore dalla lista oppure con il **contagocce** (*Pick Parent Actor*), cliccando la piattaforma nella *viewport*. Se un muro è già attaccato, Unreal avvisa che l'operazione non è compatibile; "non c'è bisogno" di cercare il numero del cubo: basta il contagocce.
- **Si muove in una sola direzione.** Nel grafo c'è un **Multiply** dove ci vuole una **somma**: "hai moltiplicato, devi sommare sopra". "Ah, è vero, non me n'ero accorto."
- **Non si muove più.** Il **Target** del Set Actor Rotation non è collegato: si trascina la referenza (Cube103) sul Target, si compila, e va.

Il primo percorso. Uno studente ha già fatto "un percorsino semplice" e lo condivide, ma si vede solo la finestra del Blueprint ("bel cursore, comunque"). Il problema è che il gioco è aperto in un'altra finestra: dai tre puntini accanto a Play si sceglie di giocare nella *viewport* selezionata (**Selected Viewport**), "la prima". La pallina parte dall'alto e la camera la inquadra male: meglio una **visuale dall'alto**, ruotando il Player Start. Intanto "la pallina è volata fuori".

Il soffitto invisibile. Sono le 19 e 3, e Igor chiude con "un piccolo consiglino". Prende un cubo, lo mette **sopra** la vaschetta, lo allarga al massimo in modo da coprirla tutta, "abbastanza preciso", e lo attacca alla base (o a uno dei muri: "va bene comunque, basta che segua il movimento"). Poi, nei Details, spunta **Hidden in Game**. In Play il soffitto **non si vede**, "ma la sua collisione sì": la pallina può provare a uscire, "ma ci sarà una collisione che la bloccherà". Al primo tentativo il soffitto non è abbastanza grande e la pallina esce da un lato; poi lo si allarga. E la pallina, "giustamente", a un certo punto compenetra lo stesso: "un classico", "cose derivanti dalla fisica" di Unreal.

Approfondimento: nascosto, ma solido. Hidden in Game agisce solo sulla **visibilità**: il componente non viene disegnato, ma conserva la sua **collisione**. Il contrario si ottiene nella sezione Collision dei Details, cambiando il *preset* di collisione (per esempio NoCollision): un oggetto visibile che non blocca niente. Romero e Sewell usano la stessa proprietà nel terzo capitolo, per nascondere un effetto di fuoco finché non serve (con il nodo **Set Hidden in Game** lo si può nascondere e mostrare durante il gioco), e di nuovo più avanti per un componente che deve esistere senza essere visto. Per i muri invisibili Unreal offre anche un actor apposito, il **Blocking Volume**, che esiste solo come collisione e non ha nemmeno una *mesh* da nascondere.

1.2.23 I compiti, e la chiusura

"Qualche altra domanda, ragazzi? Se no, andiamo a cena tutti quanti." Tutto chiaro.

I compiti. Per la prossima volta, "se vi va": un piccolo **percorso** in cui giocare con la pallina, "un piccolo labirinto". "Stesso discorso": si prende un muro, lo si copia, e si costruisce.

Dove mandarli. Igor ricorda il canale **Intro UE** del Discord, dove gli studenti hanno già mandato "tutti i tavoli" della prima lezione ("bravi, ragazzi"). Lì si possono mandare una foto o un piccolo video del "giochino", tenendo conto del limite di dimensione dei file di Discord ("se vi prende oltre 10 megabyte...").

La prossima lezione. La prossima volta, il martedì, ci sarà **Davide**[13], che introdurrà "il mondo un po' più grafico dei **materiali**" e farà importare quello che servirà dal **materiale didattico**. Una richiesta precisa: **scaricare il materiale didattico**, in modo da usarlo insieme a Davide e poi continuare insieme a Igor. "Allora alla prossima. Ciao, ciao."

[13] La trascrizione dice "allora a giovedì prossimo, martedì, avrete Davide": la correzione è di Igor stesso, e coincide con il calendario annunciato nella prima lezione (lezioni alternate, il martedì con Davide).

Comando	**Che cosa fa**
Trascinare da un pin	apre il menu dei nodi compatibili con quel pin
Tasto destro nel grafo	apre il menu di tutti i nodi (con ricerca)
Compile	compila il Blueprint; necessario dopo ogni modifica importante e prima di impostare il *default* di una variabile nuova
+ (Variables)	crea una variabile
Ctrl + trascinamento di una variabile	crea un GET
Alt + trascinamento di una variabile	crea un SET
Variabile trascinata su un pin	crea il GET (o il SET) già collegato
Alt + clic su un pin	stacca i collegamenti del pin
B + clic	crea un Branch
Q	raddrizza i collegamenti dei nodi selezionati (Straighten Connections)
Ctrl+C / Ctrl+V	copia e incolla i nodi
Canc (*Delete*)	elimina i nodi selezionati
Tasto destro su un pin struttura	Split Struct Pin / Recombine Struct Pin
F9	mette o toglie un *breakpoint* sul nodo selezionato
Freccia in basso in un nodo	mostra i parametri avanzati (per esempio Teleport)
Icona dell'occhio (Variables)	rende la variabile Instance Editable, cioè visibile e modificabile nei Details del livello
Tre puntini accanto a Play	sceglie dove giocare: New Editor Window (PIE), Selected Viewport...
Esc	esce dal Play
Tasto destro su un actor, Attach To	aggancia l'actor a un genitore (anche con il contagocce)

Table 1.2.4: I comandi della seconda lezione nell'editor dei Blueprint e nel Level Editor di Unreal Engine 5.5. Le scorciatoie S + clic (Sequence), D + clic (Delay) e F + clic (For Each Loop), non usate a lezione, sono citate nel libro di Romero e Sewell.

Punti chiave del capitolo

- Un **actor** è un elemento di gioco con un ***transform*** (locazione, rotazione, scala), fatto di **componenti** (*mesh*, luci, audio, particelle...) che possono essere **imparentati**, e di una **logica** scritta in Blueprint. In editor si ha una *preview*; con Play, a *runtime*, gli attori vengono duplicati e inizializzati.
- Una **classe** è una tipologia di oggetti, "come un reparto del supermercato". **Actor**: vuoto, un centro. **Pawn**: un actor che si può **possedere**. **Character**: un Pawn con Skeletal Mesh e movimento. I Blueprint si chiamano **BP_Nome**.
- L'**Event Graph** è il cuore del Blueprint. Un **evento** è un momento con una condizione: **BeginPlay** scatta quando *quell'actor* comincia a esistere, non "all'avvio del programma".
- **Print String** stampa a schermo, solo in sviluppo. Dopo ogni modifica: **Compile**. L'actor va messo nella scena, se no non succede niente.
- Una **variabile** è una scatola di un certo tipo: **GET** prende il valore, **SET** lo imposta (e ha un pin di esecuzione). I pin si collegano solo se i **colori** (i tipi) combaciano: rosso bool, verde float, rosa stringa, giallo vettore, lilla rotator, blu riferimento a un oggetto.
- I nodi blu **eseguono**, quelli verdi **calcolano**. Gli operatori aritmetici fanno i conti; un float collegato a una stringa ottiene un convertitore automatico.
- **Instance Editable** (l'occhietto) rende una variabile modificabile dai Details del livello: è lo strumento del ***tuning***, "lasciami quei valori, che li devo testare". Le variabili globali vivono nella Game Instance.
- Il **Branch** (B + clic) è un bivio, "i binari del treno": la leva è una booleana. **OR**: basta che una sia vera. **AND**: devono esserlo tutte. **NOT**: inverte. Una casella spuntata in un pin non collegato è un valore fisso, non la variabile.
- Nel **Level Blueprint** si possono creare **referenze** agli actor del livello e usare gli **eventi di input** (W, A, S, D, il mouse); **Consume Input** riserva il tasto a quell'evento.
- Un oggetto che deve muoversi va messo su **Movable**: Static è "dipinto", Stationary è una via di mezzo.
- **Set** imposta una rotazione assoluta ("ci sto già, a 25"); per incrementarla si parte dal **Get** attuale, con **Split Struct Pin** o **Break/Make Rotator**; **Add Actor World Rotation** aggiunge direttamente, sugli assi del mondo.
- **Simulate Physics** dà la fisica a un oggetto; **Teleport** evita che il movimento della piattaforma "colpisca" la pallina. La fisica è emergente: i *bug* fisici sono normali.
- L'**Event Tick** esegue a ogni *frame*. Tutto ciò che si muove nel tempo va moltiplicato per i ***delta seconds***, o andrà a velocità diverse su macchine diverse.
- Per invertire un valore si **moltiplica per** -1. Il **Clamp** dà a un valore un minimo e un massimo: sulla rotazione risultante, limita l'inclinazione a ± 20 gradi.
- **Attach To** aggancia i muri alla piattaforma; **Hidden in Game** nasconde un oggetto ma ne lascia la **collisione**: il soffitto invisibile.

Chapter 1.3

Texture, materiali e mesh: la grafica dentro Unreal e la torretta

Questa è la **terza lezione** del corso breve di **introduzione a Unreal Engine**, e cambia sia docente sia argomento. Al posto di Igor, che "aveva un impiccio", c'è **Davide**, che si presenta così: "io sono Davide e in questa lezione, nello specifico, sostituisco Igor". Davide tornerà più avanti: "le ultime lezioni del corso le fate con me", e saranno lezioni "un po' più basate su **level design** e la creazione di mappe", con qualche esercitazione insieme. Dopo due lezioni sull'editor e sulla logica (piazzare oggetti, i *transform*, i Blueprint, la pallina sulla piattaforma), questa lezione guarda a ciò che "fa il grafico all'interno di Unreal": le **mesh**, le **texture**, i **materiali**, e "qualche nodo, qualche cosa carina che potete fare" combinando mesh e materiali.

La lezione ha due metà separate da una pausa. La prima è sui **materiali**: che cos'è una texture e come si avvolge su un oggetto attraverso le **UV**; quali tipi di mappa esistono (**Diffuse**, **Normal**, **Specular**, la mappa di altezza per il *displacement*) e perché in queste mappe il bianco vale 1 e il nero 0; che cos'è un **materiale** e come si costruisce nell'**editor dei materiali**, "nello stesso modo dell'editor dei Blueprint"; le impostazioni principali (**Material Domain**, **Blend Mode**, **Shading Model**); i nodi di base (**Texture Sample**, le costanti, le operazioni, il **Texture Coordinate**, il **Lerp**, il **Panner**); il rendering **PBR** con Metallic, Specular e Roughness; e le **Material Instance**, il modo "bon ton" di applicare i materiali. La seconda metà è sulle **mesh**: si importa una **torretta** in tre pezzi dal materiale didattico, si guarda la finestra di **import**, si corregge il **pivot** del cannone con un **Reimport**, si esplorano lo Static Mesh Editor e le **collisioni**, si fondono i pezzi con **Merge Actors**, si creano a mano i materiali della torretta, e si chiude con i materiali **World Aligned**, che non si deformano quando si stira l'oggetto. In fondo, un esercizio libero: un materiale di **lava**.

La lezione è **online**, su Zoom, come le precedenti, e finisce poco prima delle 19 ("si sono fatte le 7"). La pausa, una decina di minuti, arriva dopo "un'oretta e mezza". Davide, alla sua prima volta con questa classe, combatte per tutta la lezione con la condivisione dello schermo ("io sto facendo a cazzotti da tutta la lezione con la finestra di Zoom", "quanto non sono abituato a streamare") e con un PC di emergenza: "il fisso mi è esploso letteralmente tre giorni fa", e a metà della seconda parte il computer "inizia ad accusare la stream", tanto che Davide gira il ventilatore verso di lui. Chiede spesso conferma che gli studenti vedano quello che mostra: "mi impanicherò ogni tanto a caso e vi chiederò conferma delle robe".

Tre avvertenze di lettura, le stesse delle prime due lezioni. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal diventa "ArrayLink", "ArrayL4", "Arial Engine" e "a Real"; la **Material Instance** è quasi sempre "materialistance", "Material Distance" e una volta "mie dire distanze"; il **Panner** è "panel", "planner" e "banner"; il **Lerp** è "l'ERP"; *World Aligned* è "word aligned" e "word.allocation"; la texture **T_Bark** (una corteccia) è "Arc", "barca", "Barc" e "park"; "strecciare" e "si straccia" stanno per lo *stretch*, lo stiramento; "traccare" sta per *tracciare* un bersaglio (*to track*); la **mappa di altezza** (*height map*) è quasi sempre "light map" o "lightmap". Dove la ricostruzione è sicura il capitolo usa la forma corretta e, la prima volta, lo segnala in nota; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: durante la pausa e verso la fine il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("qual è la risposta che ci può essere?", discorsi sulla "vita data per scontata" e su "vittime"): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è

una dimostrazione dal vivo nell'editor dei materiali e nello Static Mesh Editor. Il capitolo ricostruisce i grafi di nodi da ciò che Davide dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei comandi di Unreal Engine 5.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono gli schemi spiegati a voce, sulle slide o nell'editor: le mappe della torretta, i canali di una texture, il rapporto tra texture, materiale e mesh, i grafi dei materiali costruiti a lezione, le Material Instance, la struttura di una mesh, il pivot del cannone. Le tabelle riassumono i tipi di mappa, le impostazioni del materiale, le opzioni di import, le collisioni e le scorciatoie dell'editor dei materiali. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro *Blueprints Visual Scripting for Unreal Engine 5* di Marcos Romero e Brenden Sewell, il testo di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'attesa, le presentazioni e il programma; che cosa fa un grafico dentro Unreal; il materiale didattico (sezioni 1.3.1 e 1.3.2);
2. le **texture**: la carta regalo, le **UV**, i tipi di mappa, il bianco e il nero, i **canali RGB** (sezioni da 1.3.3 a 1.3.5);
3. i **materiali**: la vernice, l'editor, Material Domain, Blend Mode e Shading Model, i nodi (sezioni da 1.3.6 a 1.3.8);
4. in Engine: la mappa vuota, dentro una texture, il **Texture Sample**, le costanti e le scorciatoie, il colore e l'**emissive** (sezioni da 1.3.9 a 1.3.12);
5. il **PBR** e la Normal (sezione 1.3.13);
6. il **Texture Coordinate**, il tiling e il pavimento (sezione 1.3.14);
7. le **Material Instance** e le Material Function (sezioni 1.3.15 e 1.3.16);
8. il **Lerp** e il **Panner**, poi la pausa (sezioni da 1.3.17 a 1.3.19);
9. l'**import** e la torretta; che cos'è una **mesh**; dimensioni e formati (sezioni da 1.3.20 a 1.3.22);
10. il **pivot** del cannone e il **Reimport** (sezione 1.3.23);
11. lo **Static Mesh Editor** e le **collisioni**; raggruppare e **Merge Actors** (sezioni 1.3.25 e 1.3.26);
12. i **materiali della torretta** e le domande sul Reimport e sul merge (sezioni 1.3.27 e 1.3.28);
13. lo stretch, lo **z-fighting** e i materiali **World Aligned** (sezione 1.3.29);
14. l'esercizio della **lava**, il nodo **Noise** e la chiusura (sezione 1.3.30).

1.3.1 L'attesa, le presentazioni e il programma

"Ok, buonasera." La registrazione parte subito ("sta registrando? Sì, sta registrando, perfetto"), e Davide aspetta "qualche minutino" che entrino tutti. Uno studente avvisa che dovrà staccarsi venti minuti prima per andare a lavorare: "nessun problema", tanto la lezione resta registrata, e anzi a Davide piacerebbe finire "con voi che fate qualcosina", quindi andare via un po' in anticipo "non è neanche un problema". Davide chiede quante persone c'erano alle lezioni precedenti ("una dozzina, credo"), ringrazia chi accende la telecamera ("almeno so che non parlo da solo") e controlla che non si senta il ventilatore in sottofondo ("inizia a far caldo, inizia a essere complicato").

Che cosa avete fatto con Igor? Prima di cominciare, Davide vuole sapere a che punto è la classe, per non ripetere cose già viste. Dal Discord ha capito che si è "ancora all'inizio": le schede, le finestre, i *transform* sugli oggetti, i tavolini, "qualcosa con gli input, con la pallina che si muove". "Se vediamo qualcosa che avete già visto, ditemelo."

Il programma. La lezione di oggi guarda a quello che fa "il grafico all'interno di Unreal": "un po' di nozioni base per quanto riguarda le mesh, un po' di nozioni base per quanto riguarda i materiali, le texture", e "già qualche nodo, qualche cosa carina che potete fare" usando mesh e materiali.

Chi c'è. Davide chiede a che cosa sono iscritti gli studenti, "per capire chi ho davanti". Le risposte: **game design** (più d'uno), **3D art** (tre persone), **character animation** (due), **narrative design**. "Beh, in realtà siamo abbastanza assortiti."

Che cosa fa un grafico dentro Unreal. Ai futuri grafici 3D Davide mette "le mani avanti": la maggior parte del lavoro di un grafico **non** si svolge dentro Unreal, ma in altri strumenti, che verranno spiegati nei rispettivi corsi. Quello che si vede oggi è ciò che un grafico fa "nel momento in cui va a lavorare con altre persone" dentro Unreal Engine: **importare** gli oggetti e creare gli **shader** dentro l'engine.

Come si struttura la lezione. Davide chiede come lavorava Igor: tutta teoria e poi tutta pratica, o un po' e un po'? "Andava prima con la presentazione e poi con Unreal." Davide prova a replicare: "ci facciamo prima una chiacchiera sulla parte un po' più teorica e poi passiamo al progetto". Nella pratica, come si vedrà, teoria ed engine si alterneranno di continuo.

1.3.2 Il materiale didattico

Sul Discord c'è stato "un po' di casino" per il **materiale didattico**. "Non è molta roba": è un piccolo progetto di Unreal, "un progetto che Unreal usa per spiegare alcune cose", e probabilmente quello che Igor farà usare nelle lezioni successive ogni volta che si implementa qualcosa. Gli altri file della cartella sono **asset** da importare nel progetto poco alla volta: oggi la **torretta**; quando si farà l'audio, l'audio; "poi vediamo se aggiungere altre cose".

Dove si trova, Davide lo spiega più avanti a una studentessa: sul Discord, "il 19 maggio, se non ricordo male", c'è un link a una pagina di Drive con un file che si chiama, più o meno, "materiale didattico Unreal Engine Intro"[1]. Lo si scarica, lo si **decomprime**, e dentro c'è "un progettino con un *character*": sono soprattutto cose per le prossime lezioni con Igor, e per oggi serve poco.

1.3.3 Che cos'è una texture

La carta regalo. "Che cos'è una texture? La texture fondamentalmente è il *wrap*, è la **carta regalo** in cui è avvolto il nostro oggetto." Sulla slide c'è la torretta del materiale didattico e, accanto, le sue texture: un'immagine "tutta scomposta e segmentata e divisa in varie parti". Perché divisa in parti? Perché quelle parti poi "si avvolgeranno sopra la mesh". Le texture sono "una serie di mappe, una serie di informazioni che vanno a dare delle qualità specifiche all'oggetto". Con un'altra immagine: la texture "non è altro che una **decalcomania**" dell'oggetto, della *mesh* che si mette in scena.

Le UV. Perché una texture funzioni bene su un oggetto, deve essere pensata "con in mente le **UV** dei modelli su cui si lavora". Le UV "sono le coordinate", tutto quello che riguarda "la struttura dell'oggetto": la **mappa delle UV** è la **prima parte** del processo di creazione di una texture. Dal modello 3D si estraggono informazioni che poi servono a creare "tutti i vari tipi di texture che esistono": sulla slide le texture della torretta sono diverse, e ognuna porta informazioni diverse, "ma sono tutte texture pensate per la stessa torretta".

[1] La trascrizione dice "IVE, materiale didattico, a Real Engine Intro" e, poco prima, "Hive Unreal Engine 5 intro": la parola iniziale non è ricostruibile con certezza.

L'UV mapping. L'**UV mapping** è la tecnica che serve a **mappare le texture 2D su modelli 3D**: si assegnano le coordinate dette UV "ai vertici e agli *edge* del modello", e tutte le texture applicate dopo usano quelle informazioni "per precisione". È un lavoro che si fa **prima** di entrare in Unreal, in un altro software "come Blender, come Maya". Se si usa una texture che non è stata pensata per le UV di quell'oggetto, "la texture verrà strecciata" (*stretched*, stirata) o comunque non darà il risultato voluto.

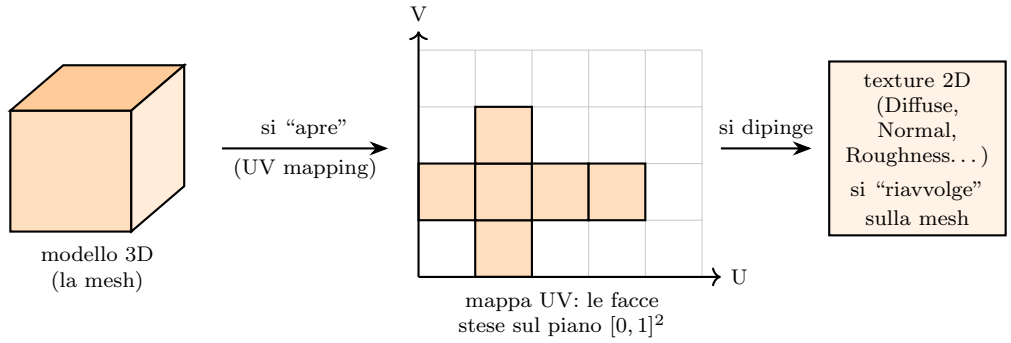


Figure 1.3.1: La texture come "carta regalo": le facce del modello vengono stese su un piano (la mappa UV, fatta in Blender o in Maya), le texture si dipingono su quel piano, e poi si "riavvolgono" sull'oggetto. È per questo che la texture della torretta, sulla slide, appare "tutta scomposta e segmentata". Schema ricostruito; le lettere U e V indicano i due assi della texture, per distinguerli da X, Y e Z del mondo.

Approfondimento: perché U e V. Le lettere U e V sono scelte solo per non confondere gli assi della texture con gli assi X, Y e Z dello spazio 3D. Una coordinata UV è una coppia di numeri, di solito tra 0 e 1: $(0, 0)$ è un angolo della texture e $(1, 1)$ l'angolo opposto. Ogni vertice della mesh riceve una coppia (u, v), e per i punti interni a una faccia il motore interpola le coordinate dei vertici. Se una mesh non ha UV (o ne ha di sbagliate), il motore non sa "quale pezzo di carta regalo" va su quale faccia: di qui le texture stirate di cui parla Davide.

1.3.4 I tipi di mappa

Davide passa in rassegna, sulla slide della torretta, le texture una per una, "piano piano", con le informazioni che portano.

La Diffuse, o Albedo. La prima è la **Diffuse**, "in alcuni casi detta anche **Albedo**". Nei file la si riconosce dalla **nomenclatura**: "T maiuscolo, *underscore*, nome dell'oggetto, *underscore*, A", o D: A per *albedo*, D per *diffuse* (per esempio `T_Turret_D`). La Diffuse è il **colore**: "tutti i dettagli estetici colorati". Sulla slide è quella in alto a destra, con l'arancione: è la texture che dà alla torretta l'arancione sulla base, il nero negli altri dettagli e il bianco in alcune zone. Più avanti Davide ne completa la descrizione: la Diffuse applica "i colori e i dettagli di base" sulle superfici 3D, e dà l'informazione del colore, del tipo di materiale, dei dettagli ("queste linee che vedete qua") e di quanto un oggetto sia ruvido o levigato, se per esempio si applica la texture di una roccia o di un metallo.

La Normal. Poi c'è la **Normal**, la **Normal Map**, che "serve a simulare una tridimensionalità" dentro un'informazione che è 2D. Si riconosce subito: è **blu**, "sempre mappe blu con delle sfumature di rosso e di verde". L'esempio classico è un **muro di mattoni**: in gioco il muro è **piatto**, ma quando gli si applica la Normal il calcolo delle luci e delle ombre "prende in considerazione la normal map" e proietta le ombre in base a lei, dando l'illusione che i mattoni siano davvero tridimensionali. La Normal dà informazioni sulla tridimensionalità "senza appesantire particolarmente il calcolo", perché non deforma davvero l'oggetto come fa una mappa di altezza.

Più avanti Davide ci torna con un esempio sulla torretta. Su una delle gambe della base servono dei **graffi**. Si potrebbe cambiare il modello aggiungendo i graffi, ma questo aggiungerebbe **poligoni**, e quindi "potenza di calcolo non necessaria". Si può invece aggiungere il dettaglio "direttamente in 2D", con la Normal: protuberanze, graffi, rughe. "Per un singolo oggetto non è un appesantimento così importante. Ma considerate quanti oggetti ci sono all'interno di una mappa quando andate a creare un livello": se ogni oggetto non è creato con la giusta accortezza, "si fa molto velocemente ad appesantire il progetto più di quanto dovrebbe essere pesante".

La Specular. Lo **Specular mapping** riguarda "la brillantezza e la riflettività dell'oggetto": come l'oggetto gestisce la luce, come la luce "rimbalza" su di lui. Un oggetto può avere parti metalliche, parti in pietra e altre ancora, e "a seconda del tipo di materiale di cui l'oggetto è fatto, rifrangerà la luce in una maniera diversa". Queste mappe sono quelle "in bianco e in nero". La texture determina come si calcolano i riflessi "sui singoli pixel proprio dell'oggetto", quanta luce viene riflessa; tornerà utile quando si vedranno le luci in Unreal, perché "cambiando un materiale la luce ha un effetto completamente diverso".

La mappa di altezza. Poi c'è la mappa che serve a creare **displacement**[2]: crea "una *tessellation* nell'oggetto", rialza alcune zone, ne abbassa altre, "modifica leggermente l'oggetto". Davide la chiama "una normal sotto steroidi", perché fa davvero *displacement*: aggiunge profondità e dettagli tridimensionali alle superfici piane, e usa le scale di grigio per creare scanalature e protuberanze. "Nelle parti in cui vedete il bianco è sempre il sì", la protuberanza; nel nero, l'effetto opposto.

Il texturing procedurale. Infine il **procedural texturing**: il processo di creare texture, materiali o **shader** "attraverso degli algoritmi". "Andando ad utilizzare dei calcoli matematici, noi andiamo a modificare delle informazioni per avere un risultato." È esattamente quello che fa l'editor dei materiali di Unreal.

Mappa	**Aspetto**	**Che cosa porta**
Diffuse (Albedo)	a colori	il colore e i dettagli estetici; suffisso _D o _A
Normal	blu, con sfumature rosse e verdi	l'illusione del rilievo: la luce e le ombre si calcolano come se la superficie avesse graffi, mattoni, rughe, senza aggiungere poligoni
Specular	bianco e nero	quanto e come la superficie riflette la luce
Roughness	bianco e nero	quanto la superficie è ruvida (1) o liscia (0)
Metallic	bianco e nero	dove la superficie è metallo (1) e dove no (0)
Ambient Occlusion	bianco e nero	le zone in ombra perché "chiuse" (pieghe, angoli)
Height (displacement)	bianco e nero	deforma davvero la superficie: il bianco rialza, il nero abbassa

Table 1.3.1: I tipi di mappa nominati a lezione. Diffuse, Normal, Specular e la mappa di altezza sono quelle della slide; Roughness, Metallic e Ambient Occlusion compaiono nella parte sulle mappe impacchettate e nei materiali della torretta.

Il bianco è 1, il nero è 0. La cosa importante da capire, insiste Davide, è che nelle mappe in bianco e nero "dovete sempre considerare il **bianco come un 1**, il **nero come uno 0** e il **grigio come dei valori nel mezzo** fra lo 0 e l'1". Che cosa vuol dire? Che in Unreal, per esempio, il valore dello Specular va soltanto tra 0 e 1: a 1 riflette la luce, a 0 no. Una mappa di Specular, quindi, va letta come una griglia di numeri: "ci sono degli 1 in queste parti e degli 0 nel resto delle parti". È la chiave per capire quasi tutto il resto della lezione.

[2]Nella trascrizione è sempre "light map" o "lightmap", ma la descrizione (rialza e abbassa zone dell'oggetto, crea *tessellation*, è "una normal sotto steroidi") è quella della **height map**, la mappa di altezza o di *displacement*. Le *lightmap* vere sono un'altra cosa: texture in cui il motore salva l'illuminazione precalcolata.

Approfondimento: la mappa di altezza, la Normal e Unreal Engine 5. La differenza tra Normal e mappa di altezza è la differenza tra *sembrare* e *essere*. La Normal Map memorizza, per ogni pixel, la direzione in cui "guarda" la superficie (la sua *normale*): i tre numeri X, Y e Z della direzione sono salvati nei canali rosso, verde e blu, e siccome la maggior parte dei pixel guarda "dritto fuori" dalla superficie (Z alto) la mappa appare blu. La geometria non cambia: di profilo, un muro con Normal Map resta piatto. La mappa di altezza invece sposta davvero i vertici, e per farlo servono vertici da spostare: di qui la *tessellation*, che suddivide la superficie in molti triangoli. In Unreal Engine 5 il *displacement* vero si fa soprattutto con le mesh **Nanite**, ed è più costoso della Normal: per questo, per graffi e piccoli dettagli, la Normal resta la scelta di default, come dice Davide.

1.3.5 I canali RGB e le mappe impacchettate

"Chiaramente, queste texture utilizzano l'**RGB**": non hanno un solo canale, ma **tre canali** su cui passano le informazioni. Sulla slide c'è una Diffuse scomposta in rosso, verde e blu: ognuno porta informazioni separate, che combinate danno il colore che serve.

Ma c'è un altro modo di vedere la cosa. **Singolarmente**, ogni canale appare come un'immagine in **bianco, nero e grigio**. E bianchi, neri e grigi "sono combinazioni di 0 e di 1": quindi ogni canale si può usare per trasmettere **un'informazione diversa**. È il **multichannel**. Una pratica comune, nell'esportare e importare texture, è avere una singola mappa "che spesso e volentieri viene siglata come **AORM**", per **Ambient Occlusion, Roughness e Metallic**: tre mappe in bianco e nero, ognuna in un canale. "Magari noi vogliamo utilizzare questa mappa qui sopra per l'ambient occlusion, questa mappa qui sotto per le metallic", e così via. Una sola mappa con più informazioni invece di tante mappe diverse: "va ad alleggerire il progetto, va a rendere più veloce, più modulare" l'export delle texture.

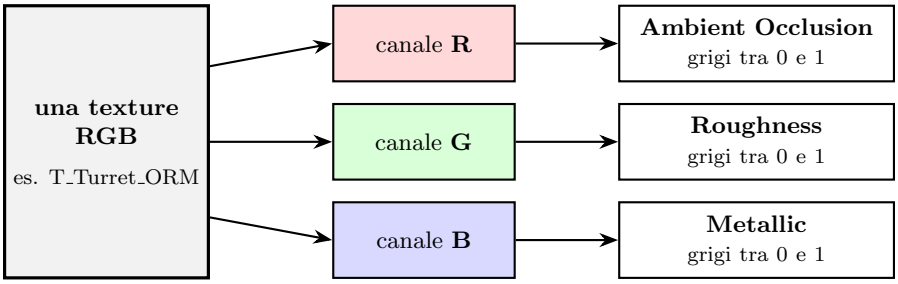


Figure 1.3.2: Una mappa impacchettata (*channel packing*). La lezione la chiama AORM; nella pratica di Unreal la convenzione più diffusa è ORM, con l'Ambient Occlusion nel rosso, la Roughness nel verde e il Metallic nel blu, ma l'ordine dipende da chi esporta le texture.

Approfondimento: sRGB e compressione. Quando si importa in Unreal una texture impacchettata (o qualunque mappa "di dati", come Roughness o Metallic), conviene aprirla e **togliere la spunta a sRGB**: l'opzione sRGB dice al motore che l'immagine è un *colore* da correggere per la percezione dell'occhio, mentre una mappa di dati contiene numeri da leggere così come sono. Nelle stesse impostazioni, il campo *Compression Settings* si mette su **Masks** per le mappe impacchettate e su **Normalmap** per le Normal (Unreal di solito lo riconosce da solo per le texture con suffisso _N). Se ci si dimentica, il materiale funziona lo stesso, ma i grigi vengono letti un po' sbagliati e la superficie risulta più lucida o più opaca del previsto.

1.3.6 Il materiale: la vernice

Le texture non si modificano. "Quello che possiamo fare con le texture in Unreal è relativamente poco, perché non possiamo andare a modificare effettivamente le texture all'interno dell'engine." E allora come si

danno qualità visive alle mesh? "Lo facciamo con i **materiali**."

Il materiale è uno shader. I materiali "sono degli **shader**". E l'**editor dei materiali**, quello che si vedrà "principalmente" oggi, "funziona praticamente nello stesso modo dell'editor dei Blueprint": una serie di **nodi** collegati, che trasportano informazioni fino a un **nodo risultante**.

Vernice, non carta regalo. La distinzione è netta: "il materiale è effettivamente la **vernice** che viene applicata sulla mesh, non la texture". Sugli oggetti si applicano **materiali**. Le texture servono a **costruire** i materiali: si combinano le texture in un unico materiale, che poi si applica sull'oggetto finale. Il materiale dice all'oggetto "come dovrebbe interagire con la luce, che tipo di informazioni visive dovrebbe portare", dove ci sono tridimensionalità e così via: "tutto quello che abbiamo visto dalle varie texture precedentemente viene tutto convogliato all'interno dei materiali". Questo, precisa Davide, se si parla di **oggetti fisici**. I materiali hanno anche altre funzioni: servono per esempio a fare **effetti speciali**, e un materiale per un effetto speciale "dovrà essere costruito in maniera diversa" da quello che riveste una torretta.

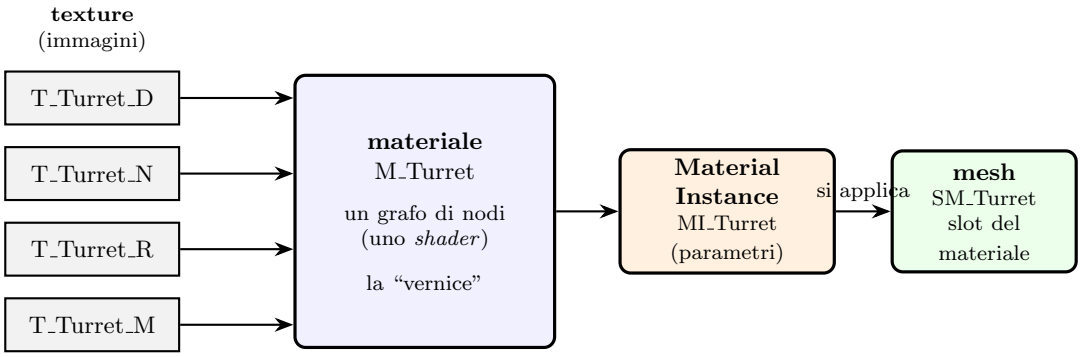


Figure 1.3.3: Texture, materiale, istanza e mesh. Le texture non si applicano direttamente agli oggetti: entrano in un materiale, che è la "vernice"; dal materiale si crea un'istanza, ed è l'istanza che si applica allo slot del materiale della mesh (sezione 1.3.15). I nomi seguono la nomenclatura suggerita a lezione (T_, M_, MI_, SM_).

Approfondimento: il materiale nel libro di Romero e Sewell. Il quinto capitolo del libro ("Object Interaction with Blueprints") usa la stessa immagine di Davide: un materiale è un *asset* che si aggiunge alla mesh di un actor per crearne l'aspetto, "come uno strato di vernice". Il libro crea un materiale rosso, `M_TargetRed`, in una cartella Materials ("non è necessario, ma è buona pratica tenere ordinata la gerarchia del progetto"), e nota che l'editor dei materiali "condivide molte caratteristiche e convenzioni con i Blueprint": al centro c'è il grafo; il nodo grande con il nome del materiale è il **result node**; con il tasto destro si apre un menu di nodi con una ricerca sensibile al contesto; i fili si tirano trascinando dai pin, gli ingressi a sinistra e le uscite a destra. Il colore lo dà un **VectorParameter** collegato a Base Color; due **ScalarParameter** a 0,1 danno un po' di Metallic e di Roughness. Poi, in un Blueprint, il nodo **Set Material** cambia il materiale di un bersaglio quando viene colpito: un primo esempio di come la logica (le lezioni di Igor) e la grafica (questa lezione) si incontrino.

1.3.7 Material Domain, Blend Mode e Shading Model

Come gli oggetti nella *viewport*, anche il materiale ha i suoi **Details**. Dentro l'editor si possono selezionare i singoli nodi, per i dettagli del nodo, oppure vedere i **dettagli generali del materiale**. Tra questi ce ne sono alcuni, evidenziati sulla slide, che "cambiano proprio effettivamente come funziona il materiale" e ne definiscono la struttura e le possibilità.

Material Domain. Il **Material Domain** dice **per che cosa** verrà usato il materiale: una **superficie**, l'**interfaccia** (la UI), il **post process**, un effetto.

Blend Mode. La **Blend Mode** è il modo in cui il materiale "*blenda*" (si mescola) con i pixel calcolati nel resto della scena. La modalità **Opaque**, quella di default, **occlude completamente** ciò che c'è dietro. Per un effetto speciale, per esempio un **fuoco**, serve invece la Blend Mode **Translucent**, che permette di aggiungere parametri per l'**opacità** del materiale.

Shading Model. Lo **Shading Model**[3] definisce come il materiale **reagisce alla luce**. **Default Lit** vuol dire che segue le regole di base dell'illuminazione: il materiale viene illuminato. Per un effetto speciale, in cui non si vuole che la luce influenzi la resa, lo si mette invece su **Unlit**[4]. Ci sono "diverse tipologie" di Shading Model, a seconda di cosa si vuole realizzare.

I pin cambiano. Queste impostazioni si applicano sul nodo risultante, e cambiarle cambia i **pin** del nodo. L'**Opacity** non si può modificare in Blend Mode Opaque, perché un materiale opaco "dà per scontato che l'oggetto sia sempre opaco": non c'è bisogno di dire dove e quando sarà trasparente. Viceversa, la Blend Mode Translucent toglie la possibilità di modificare Metallic, Specular, Roughness e così via: se il materiale deve essere potenzialmente trasparente, "non ha senso andare a calcolare come rifletterà la luce".

Impostazione	Valori citati	Che cosa decide
Material Domain	Surface, User Interface, Post Process...	per che cosa si usa il materiale: una superficie, la UI, il post process, un effetto
Blend Mode	**Opaque** (default), Masked, Translucent	come il materiale si mescola con ciò che ha dietro: Opaque nasconde tutto; Masked ritaglia con una maschera; Translucent sfuma con l'Opacity (fuoco, fili d'erba)
Shading Model	**Default Lit** (default), Unlit...	come reagisce alla luce: Default Lit viene illuminato, Unlit no (effetti speciali)

Table 1.3.2: Le tre impostazioni "più importanti" dei dettagli di un materiale. Cambiarle accende e spegne i pin del nodo risultante.

La prova in Engine. Per far vedere la cosa, Davide apre il progetto del materiale didattico e invita chi vuole a seguirlo: nel **Content Browser** si crea un nuovo materiale ("va bene anche chiamarlo New Material"), lo si apre con un **doppio clic** e si entra nell'editor. Le impostazioni di cui si parlava sono nei dettagli, in basso a sinistra. Se la Blend Mode passa da Opaque a **Translucent**, il nodo risultante cambia: spariscono Metallic, Specular, Roughness, Anisotropy, e compare **Opacity**. È utile, per esempio, per una **fiamma** che verso l'alto diventa trasparente (le informazioni grafiche alla base, e in alto "piano piano va a dissolversi"), o per un **filo d'erba** che parte con il suo colore e poi va in trasparenza. Con la Blend Mode **Masked**, "mascherata", si crea invece una **maschera**, un *layer* che funziona in modo diverso dal resto della superficie. Per ora si resta su **Opaque**, perché i materiali di oggi andranno su oggetti 3D. A metà spiegazione arriva uno studente in ritardo: "Chat, buon pomeriggio, Lorenzo. Noi qui abbiamo già cominciato, ma in realtà abbiamo parlato di cose principalmente teoriche."

Approfondimento: Masked o Translucent? Le due modalità sembrano simili ma lavorano in modo opposto. **Masked** decide pixel per pixel se il materiale *c'è o non c'è*: l'ingresso Opacity Mask viene confrontato con una soglia (di default 0,333), e sotto la soglia il pixel non viene disegnato affatto. È la modalità delle foglie, delle reti e dell'erba "ritagliata": economica, compatibile con la luce e con le ombre, ma senza sfumature. **Translucent** invece *mescola* il colore del materiale con ciò che sta dietro, secondo un valore di Opacity tra 0 e 1: permette le sfumature (fumo, vetro, fiamme), ma costa di più, perché ogni strato

[3] Nella trascrizione una volta "scaling model".
[4] Nella trascrizione "lo metterò un lit".

traslucido va sommato a quelli dietro, e gestisce peggio luce e ordinamento. Regola pratica: Masked quando basta un "sì o no", Translucent solo quando serve davvero la sfumatura.

1.3.8 I nodi: le Material Expression

"Come per i Blueprint, anche l'editor dei materiali ha dei nodi, delle **espressioni**." I **Material Expression Node** sono "dei piccoli set di istruzioni preprogrammate che eseguono delle cose nello specifico, esattamente come i nodi dei Blueprint", o, con un'altra formula, "pezzi di codice statico".

Il primo grafo. Sulla slide c'è un piccolo grafo (figura 1.3.4), e Davide lo legge nodo per nodo:

- il **Texture Sample** comunica l'informazione della texture: "sta dicendo che la texture che c'è in questo nodo è la texture che determina il **base color**"; permette di usare un'immagine e farla diventare la texture del materiale;
- il **Texture Coordinate**, il nodo **rosso**, restituisce le coordinate U e V del materiale, e serve a decidere "quanto è grande il **tiling** della mia texture, quindi quante volte si ripete";
- il **Multiply** è programmato per moltiplicare due valori: "moltiplica A per B e dà una risultante";
- il quadratino è una **costante**, "un singolo *float*", una variabile singola.

Il grafo dice: "moltiplica le coordinate per uno e comunica queste cose alle UV della texture". Così com'è, quindi, "è come se non ci fosse": moltiplicare per 1 lascia le coordinate di default. Ma è così che si costruiscono i grafi: combinando nodi si ottengono risultati molto diversi. Si può moltiplicare, si può fare un'**interpolazione**, si possono far **muovere** le texture per suggerire un movimento nel materiale, "eccetera, eccetera, eccetera".

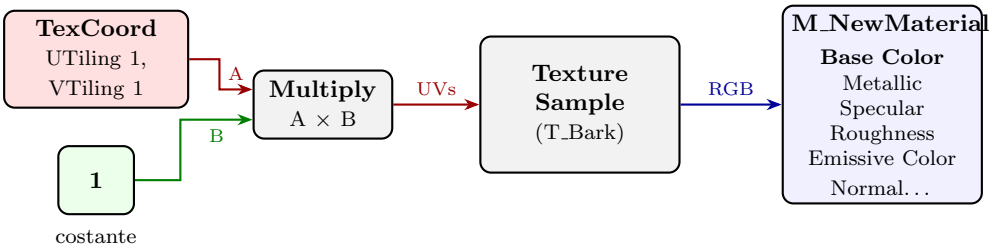


Figure 1.3.4: Il primo grafo della slide: le coordinate UV, moltiplicate per una costante, entrano nel Texture Sample, il cui colore va nel Base Color del nodo risultante. Con la costante a 1 il Multiply "è come se non ci fosse"; cambiando la costante cambia il tiling (sezione 1.3.14). Schema ricostruito.

I parametri. Non solo le costanti, ma i nodi in generale "possono essere utilizzati come **parametri**". Per rendere un nodo un parametro si fa **tasto destro** sul nodo e **Convert to Parameter**, gli si dà un nome, e questo permette di modificare in seguito il valore "senza dover ricalcolare da capo il materiale". I parametri torneranno utili con le **Material Instance** (sezione 1.3.15).

Come si chiama un nodo. "Tasto destro, scriviamo il nome", per esempio *texture sample*, ed ecco il nodo. C'è poi "tutta una serie di *shortcut*" per i nodi più usati, che si vedranno tra poco.

1.3.9 Il problema della mappa vuota

Una studentessa interrompe: non riesce più ad aprire la mappa, "me la apre totalmente vuota, senza titolo". Davide le chiede di condividere lo schermo, ma lei segue da due computer diversi. Prima ipotesi: ha creato

un **nuovo livello**, e un *New Level* appena caricato è tutto nero, perché non c'è niente in scena ("potremmo andare a popolarla mettendoci cose"). Ma il problema è un altro: ha aperto il progetto del materiale didattico, cliccato sulla mappa del personaggio in terza persona (*Third Person Map*), e non succede niente. Davide sospetta che il progetto sia ancora in una **cartella zippata** ("di solito questa cosa succede quando il progetto è una cartella che è ancora zippata"), e le consiglia di **riscaricarlo**. Poco dopo il problema è risolto: "sono riuscita a risolvere adesso". "Meno male, meno male."

Approfondimento: aprire un progetto da una cartella. Un progetto di Unreal è una cartella che contiene un file `.uproject` e le sottocartelle `Content`, `Config` e altre. Unreal non può aprire un progetto che sta ancora dentro un archivio `.zip`: Windows mostra il contenuto dello zip come se fosse una cartella, ma l'editor non ci può scrivere, e il risultato tipico è proprio una mappa vuota o mancante. La soluzione è estrarre tutto ("Estrai tutto") e aprire il `.uproject` estratto con un doppio clic. Se il progetto non compare nell'elenco dell'Epic Games Launcher o nel Project Browser, basta aprirlo dal `.uproject` la prima volta: da lì in poi compare tra i progetti recenti.

1.3.10 Dentro una texture e il Texture Sample

"Facciamo le cose in ordine, se no io mi confondo e mi impiccio." Davide torna al punto saltato: com'è fatta **dentro** una texture.

La finestra della texture. Con un doppio clic su una texture nel Content Browser (Davide ne prende una a caso, una corteccia, **T_Bark**) si apre una finestra in cui "non possiamo fare grandi modifiche", ma si possono vedere i **canali**: il blu, il verde, il rosso, ciascuno fatto a modo suo. "Questa texture non è particolarmente interessante", ma si rivedrà meglio con la torretta.

Il Texture Sample. Nel grafo del materiale si mette un **Texture Sample**, si seleziona il nodo, e nei dettagli, in basso, si sceglie quale texture metterci. Cercando T_Bark e mettendola nel nodo, la *preview* mostra la corteccia: "il nostro materiale sarà così, perché è effettivamente la texture che ci abbiamo messo".

Le uscite separate. Il Texture Sample non ha una sola uscita: oltre all'**RGB** ci sono **R**, **G** e **B** separati (e l'alfa, A). Si può quindi usare, per esempio, solo il canale **rosso** ("non quella completa, ma questa qui") per trasportare un'informazione: collegandolo allo **Specular** si ottiene un materiale che in alcune parti riflette la luce più che in altre. "Con questo materiale in realtà è un po' complicato farlo vedere", ma il principio è quello dei canali visto sulle slide.

Come si mette una texture nel nodo. Una studentessa non ha capito come si aggiunge la texture "al quadratino nero" del Texture Sample. Davide rifà il percorso: si clicca il nodo, e nei suoi dettagli c'è il campo **Texture**, con il quadratino e una lista. Due modi:

- se una texture è già **selezionata nel Content Browser**, la freccetta **Use Selected Asset from Content Browser** la inserisce automaticamente nel nodo;
- più spesso, si clicca sul **nome** della texture nel campo e si fa un *browse*: si cerca il nome ("T underscore Bark") e lo si sceglie dalla lista di tutte le texture disponibili.

"Voi lo vedete questo pop-up che esce?" Sì: perfetto.

1.3.11 Le costanti, le operazioni e le scorciatoie

"Chiaramente ne vediamo un po', non li vediamo tutti, perché sono tanti, sono veramente tanti." Ci sono "tantissime possibilità, tante cose che vanno studiate", ma per ora servono le basi.

La regola delle scorciatoie. Il Texture Sample si può chiamare cercandolo con il tasto destro, oppure **tenendo premuto il tasto T e facendo clic sinistro** nel grafo. "Questa struttura dello shortcut funziona per ogni singolo nodo che ha uno shortcut": un tasto, tenuto premuto, più clic sinistro. Il tasto è "molto spesso l'iniziale del nome del nodo".

Le costanti. Cercando *vector* compare, per esempio, il Vector Noise; cercando *constant* compaiono le costanti: **Constant**, **Constant2Vector**, **Constant3Vector** (e Constant4Vector). Le scorciatoie sono i numeri: **1 + clic** per la costante singola, **2 + clic** per quella a due valori, **3 + clic** per quella a tre. La costante a due "la utilizzerete relativamente poco". Le altre due moltissimo:

- la **costante singola** è un **valore numerico**, che si usa "per andare a fare qualsiasi tipo di calcolo, qualsiasi tipo di costruzione";
- la **costante a tre** è un **colore**: l'engine usa i tre valori come **R**, **G** e **B**. Si sceglie un colore (Davide prende un arancione) e l'engine modifica X, Y e Z per ottenerlo. "Qualcuno di voi l'aveva anche già fatto nel test con la pallina."

Collegando l'arancione al **Base Color** si ottiene un materiale arancione, "perché effettivamente è l'unica cosa che gli sto passando". Ma "molto raramente" si usa un colore piatto come Base Color: lo si usa piuttosto per fare **operazioni**. Più avanti una studentessa chiederà di nuovo come si crea il colore: "è come la costante singola, che è uno clic sinistro, ma è a tre, quindi è tre clic sinistro".

Staccare i fili. Per rompere un collegamento: **Alt + clic sinistro** sul pin, "e si rompono".

Le operazioni. Le operazioni utili sono "le classiche": addizione, moltiplicazione, divisione, sottrazione. Le scorciatoie: **A + clic** per **Add**, **M + clic** per **Multiply**, **D + clic** per **Divide**. La sottrazione (**Subtract**) non ha scorciatoia: si cerca. Il motivo lo spiega Davide più avanti: **S + clic** crea automaticamente un **parametro** (uno Scalar Parameter), "quindi questo potrebbe tornarvi utile".

Quali si usano davvero. In chiusura di lezione Davide aggiunge un consiglio d'esperienza: "l'operazione che utilizzerete di più in assoluto è la **moltiplicazione**". Moltiplicazione e addizione sono le più usate; divisione e sottrazione servono soprattutto per cose più complesse, "tipo sugli effetti speciali".

1.3.12 Il colore che brilla: Emissive, Multiply e Glow Intensity

"A cosa ci servono" le operazioni? Per esempio ad **aumentare il valore** di qualcosa. Il nodo risultante ha diversi pin, "diverse cose su cui andare a collegare dei valori": il **Base Color**, e più in basso l'**Emissive Color**, che controlla "quali parti del materiale emetteranno una piccola luce", un ***glow***.

L'arancione nell'Emissive. Davide stacca l'arancione dal Base Color e lo collega all'Emissive: il materiale "sarà leggermente luminoso". "Adesso è molto poco luminoso, ma è comunque un emissive."

Moltiplicare. Come si ottiene un'emissione maggiore? "Moltiplichiamo." L'arancione entra nella A di un **Multiply**, il Multiply va nell'Emissive. Il secondo valore si può scrivere direttamente nel nodo Multiply ("invece di 1 mettiamo 100"): "verrà decisamente più luminoso". Ma così "è meno controllabile". Meglio attaccare alla B una **costante** separata, darle il valore 100 e convertirla in **parametro**, con un nome come **Glow Intensity**: quando più avanti ci sarà una Material Instance, il valore si potrà cambiare "direttamente da lì". "Ci andiamo a estrapolare un parametro."

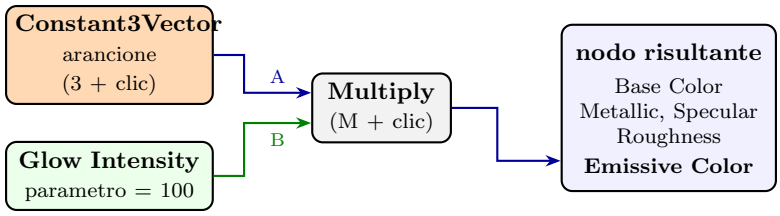


Figure 1.3.5: Un materiale che brilla. Il colore, collegato all'Emissive Color, emette "una piccola luce"; moltiplicato per 100 diventa molto più luminoso. Il valore 100 sta in una costante convertita in parametro (Glow Intensity), così che si possa cambiare da una Material Instance senza toccare il grafo. Ricostruzione dalla dimostrazione.

Approfondimento: perché 100 funziona. I colori di un materiale non sono limitati tra 0 e 1. Nel Base Color valori oltre 1 non hanno senso fisico (nessuna superficie riflette più luce di quella che riceve), ma nell'Emissive sì: un'emissione di $(1; 0{,}5; 0)$ moltiplicata per 100 diventa $(100; 50; 0)$, una sorgente molto intensa che il *post process* di Unreal trasforma in un alone luminoso (il *bloom*). Il Multiply tra un colore e un numero moltiplica ciascun canale per quel numero, quindi la tinta resta la stessa e cresce solo l'intensità. Con Lumen, il sistema di illuminazione globale di Unreal Engine 5, un materiale emissivo può anche illuminare un po' le superfici vicine.

1.3.13 Il PBR: Metallic, Specular, Roughness e Normal

"Mi rendo conto che sto andando un po' a destra e a sinistra", ammette Davide, e torna ai pin del nodo risultante. Base Color ed Emissive si sono visti. Restano **Metallic**, **Specular** e **Roughness**, "degli input abbastanza interessanti", perché sono quelli che **determinano la superficie** del materiale.

Il PBR. Il rendering dei materiali in Unreal è **PBR**, *Physically Based Rendering*: per decidere come si comporta una superficie, il motore mette insieme una serie di informazioni fisiche. Tutti questi parametri vanno **da 0 a 1**.

Roughness. La **Roughness** dice quanto una superficie è **liscia** o **ruvida**. Una **roccia** vuole una Roughness molto alta, cioè 1: il materiale sarà ruvido "e non rifletterà la luce in nessun modo". Uno **specchio** vuole una Roughness a 0: il più liscio possibile, il meno ruvido, il più riflettente.

Specular e Metallic. "Per aiutare in questa situazione" ci sono Specular e Metallic, che "si alternano a seconda del tipo di materiale". Lo **Specular**, come si è visto con le mappe, determina quanta luce viene riflessa e l'angolazione a cui viene riflessa. Per creare uno **specchio** serve un materiale molto, molto liscio e molto, molto riflettente: Davide abbassa la Roughness e alza la riflessione, e la *preview* comincia ad assomigliare "a una parete riflettente"; con altre cose che si vedranno in seguito "iniziamo ad avere un riflesso".

Emissive e Normal. L'**Emissive Color** restituisce un colore che è "effettivamente una luce". La **Normal** è l'ultimo parametro utile, e richiede la texture blu vista prima: si usa sempre un Texture Sample, ma con **T_Bark_N**, la Normal della corteccia, invece della Diffuse. Davide toglie tutto il resto e mette a confronto il materiale con e senza Normal: "vedete la differenza?" La Normal aggiunge un dettaglio tridimensionale "molto semplice da vedere", eppure "la mappa è 2D, non c'è tridimensionalità": crea solo l'illusione.

Approfondimento: Metallic e Specular nella pratica. Nel modello PBR di Unreal i due parametri non hanno lo stesso peso. **Metallic** è quasi sempre *binario*: 1 per i metalli, 0 per tutto il resto (legno, pietra, plastica, pelle); i valori intermedi servono soprattutto per i bordi tra metallo e vernice o per lo sporco. Lo **Specular** ha un default di 0,5, che va bene per la grande maggioranza dei materiali non metallici, e la documentazione di Unreal consiglia di toccarlo poco: per i metalli viene ignorato, perché il loro riflesso

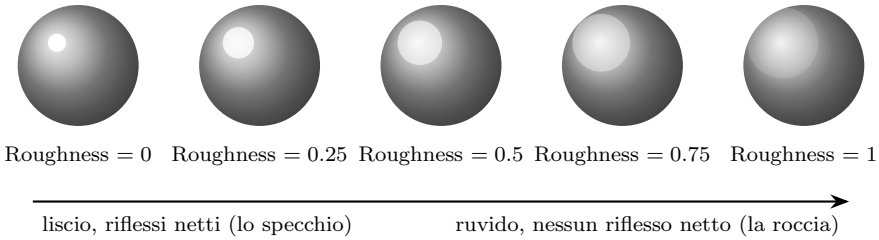


Figure 1.3.6: La Roughness, da 0 a 1, in uno schema qualitativo: più la superficie è liscia, più il riflesso della luce è piccolo e netto; più è ruvida, più la luce si sparpaglia fino a sparire come riflesso. Disegno indicativo, non un rendering di Unreal.

dipende dal Base Color. Lo specchio della lezione, in termini PBR, è quindi Metallic = 1, Roughness = 0, Base Color chiaro. È il senso dell'"alternarsi" di cui parla Davide: nei materiali metallici si lavora con il Metallic, negli altri, se proprio serve, con lo Specular, e quasi sempre con la Roughness, che è il parametro che cambia di più l'aspetto di una superficie.

1.3.14 Il Texture Coordinate e il tiling

"Chiaramente ogni nodo funziona a sé." Il **Texture Coordinate**, il nodo rosso della prima slide, ha i suoi parametri. La scorciatoia è "un po' *tricky* da ricordare", perché T + clic è già preso dal Texture Sample: è **U + clic**, "perché il Texture Coordinate lavora sulle UV".

UTiling e VTiling. Selezionando il nodo, nei dettagli compaiono **UTiling** e **VTiling**: i parametri "molto comodi" con cui si aumenta o si diminuisce il tiling di un oggetto. Davide collega il Texture Coordinate alle UV della Normal e della Diffuse, e chiede: se metto UTiling a 2, che cosa succede? "Vediamo due volte la texture applicata all'oggetto", risponde uno studente. "Sì, però c'è un *catch*, perché è soltanto sulle U": la texture si ripete due volte sull'**asse orizzontale** e una sola volta su quello verticale. Con anche VTiling a 2, la texture si ripete **quattro** volte: "ce n'è una qua, una qua, una qui e una qua".

Lo zoom. Si può fare anche il contrario, per "zoomare" sulla texture: con 0,5 la si vede "la metà delle volte. Anzi, non la metà delle volte, scusate: un quarto" (0,5 su entrambi gli assi). E questo non vuol dire che una parte della texture "viene mangiata": viene semplicemente "applicata in uno spazio differente".

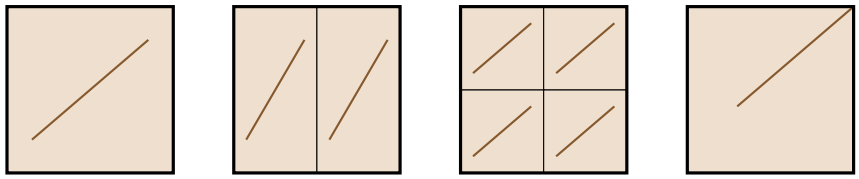


Figure 1.3.7: Il tiling sulla stessa faccia. Con UTiling a 2 la texture si ripete solo in orizzontale; con entrambi a 2 si ripete quattro volte; con entrambi a 0,5 si vede un quarto della texture, ingrandito: non è "mangiata", è applicata in uno spazio più grande. Schema ricostruito dalla dimostrazione.

Il pavimento. "Il tiling, però, va visto." Nella mappa c'è un **pavimento**, e Davide ne cambia il materiale: lo seleziona, e nei **Details**, sotto **Materials**, al posto della *material instance* della griglia grigia di prototipazione[5] trascina il **master material** appena creato. Il risultato: la corteccia è "molto, molto grande"

[5]Nella trascrizione "Materialistance Prototype Grid Gray": è l'istanza della griglia grigia dei progetti modello di Unreal.

("io non so neanche qual è l'utilizzo di questa texture che sto usando, ci ho preso Bark a caso, ma va bene così"). Davide porta il tiling a 3, poi a 10: "iniziamo a vederla più spesso, questa corteccia. Sembrano anche delle dune, vagamente." Il pavimento è probabilmente un cubo allargato, con UV "molto, molto piccole" rispetto alle sue dimensioni: per questo nella *preview* sferica del materiale la texture si ripete "diecimila volte", mentre sul pavimento "abbiamo un pavimento che è più dimensionato".

Una domanda che apre il tema successivo. Uno studente chiede: si possono modificare U e V in base all'oggetto su cui è applicata la texture, per esempio 10 su un pavimento e 0,5 su un altro oggetto? "Sì, è possibile, non facendolo dallo stesso materiale. Questo è il momento in cui entrano in gioco le **Material Instance**."

Approfondimento: il tiling in una formula. Il Texture Coordinate restituisce per ogni punto la sua coppia (u, v); il Multiply la scala: $(u', v') = (k_u\, u,\ k_v\, v)$. Una texture è definita sull'intervallo $[0, 1]$ e si ripete fuori da lì (è il comportamento di default, *wrap*): quando u' va da 0 a 2, la texture viene letta due volte. Con $k < 1$ la faccia "vede" solo un pezzo dell'intervallo, e quel pezzo viene ingrandito. Per questo il tiling dipende dalle UV della mesh: se le UV di un pavimento enorme occupano comunque tutto il quadrato $[0, 1]^2$, una texture con tiling 1 si stende una volta sola su tutta la superficie, ed è "molto, molto grande". Moltiplicare per una costante e usare UTiling e VTiling nel nodo sono due modi di fare la stessa cosa; il primo ha il vantaggio che la costante può diventare un parametro.

1.3.15 Le Material Instance

Davide torna alle slide ("passiamo un attimo alle Material Instance, visto che ne stavamo parlando"). Quello costruito finora è un **master material**: tutto ciò che sta nel grafo. Se si modifica il grafo, si modifica il materiale. Per avere lo **stesso materiale con una piccola differenza** si possono creare due **istanze** e cambiare in ciascuna il parametro collegato al Texture Coordinate.

Che cos'è un'istanza. Una **Material Instance** è "un'istanza di un materiale che noi possiamo utilizzare per modificare quel materiale **senza modificare la sorgente**". Ha diversi vantaggi, "oltre a essere *bon ton*", cioè il modo in cui si dovrebbero applicare i materiali in generale: "non dovreste prendere il master material e applicarlo agli oggetti in scena. È sempre consigliabile applicare un'istanza."

Perché è più leggera: i tavoli. L'istanza è "molto più leggera". Davide lo spiega con i **tavolini** della prima lezione. Tre tavoli in scena con lo stesso materiale del legno: per ogni tavolo l'engine carica la mesh e carica il materiale, quindi "tavolo, tavolo, tavolo" e "materiale, materiale, materiale". Se invece sui tavoli ci sono tre Material Instance, l'engine carica le tre mesh, ma trova tre istanze che puntano **tutte allo stesso master material**, e "va a renderizzare un solo materiale". "Chiaramente, su tre tavoli non cambia niente. Ora immaginate che invece di tre tavoli ci siano diecimila oggetti in scena" e che mille condividano lo stesso materiale: si capisce perché "torna comodo avere un'istanza piuttosto che avere un materiale generale".

Come si crea. "Come lo facciamo su Unreal?" Si fa **tasto destro** sul materiale nel Content Browser e **Create Material Instance**, "qui in alto". L'istanza appare **vuota**: "non c'è un grafo per modificarlo, perché chiaramente non posso modificare nessun grafo quando sono qui dentro". Per avere qualcosa da cambiare, bisogna estrarre dei **parametri** dal master material. Davide torna nel grafo, fa tasto destro sul Texture Sample del colore, **Convert to Parameter**, e lo chiama più o meno *Diffuse Color*[6]; lo stesso per l'altro Texture Sample, chiamato **Normal Map**. Salvato il materiale, nell'istanza compaiono **due parametri** selezionabili.

La nomenclatura. "Normalmente dovreste chiamarlo" non *New Material* ma **M_NewMaterial**, e l'istanza **MI_NewMaterial**: "per nomenclatura, insomma, per ritrovare facilmente tutto quello che ci serve".

[6] Nella trascrizione "discolor": il nome esatto non si ricostruisce, ma è il parametro della texture del colore.

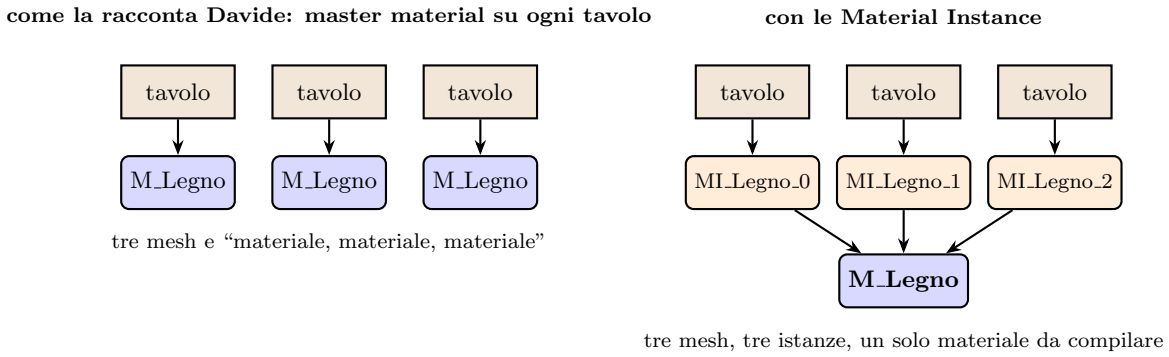


Figure 1.3.8: L’esempio dei tavoli. A sinistra, il master material applicato direttamente; a destra, tre istanze (che possono avere parametri diversi: un legno più chiaro, un tiling più fitto) che rimandano tutte allo stesso master material. Schema ricostruito dalla spiegazione; sul costo reale si veda l’approfondimento.

Cambiare la texture dall’istanza. Davide applica MI_NewMaterial al pavimento: è identico a prima. Ma se nell’istanza si cambia il parametro della texture, da Bark a un’altra, il materiale cambia, senza toccare il grafo. Si potrebbe avere un master material senza texture, “delle texture assenti o base”, e tre o quattro istanze in cui si cambiano le texture: un muro arancione, uno blu, uno rosso e uno verde sono quattro istanze. “Quando io renderizzo la scena, non andrò a chiamare cinque materiali diversi, chiamerò sempre M_NewMaterial, perché sono tutte istanze dello stesso materiale.” “Non vorrei stare complicando troppo la cosa, però insomma, ditemi voi se è chiaro il discorso.”

Il Tiling Multiplier. Ed ecco la risposta alla domanda sul tiling diverso per ogni oggetto. Nel master material Davide prende un **Multiply**, ci collega il Texture Coordinate, collega il Multiply alle UV dei due Texture Sample e attacca alla B una **costante**, che converte in parametro con il nome **Tiling Multiplier**. Nell’istanza, il Tiling Multiplier portato a 10 dà “lo stesso risultato di prima”; ma ora è un parametro. Una copia dell’istanza con il Tiling Multiplier a 5 darebbe un tiling diverso con lo stesso materiale. “È chiaro il discorso?” “Perfetto.”

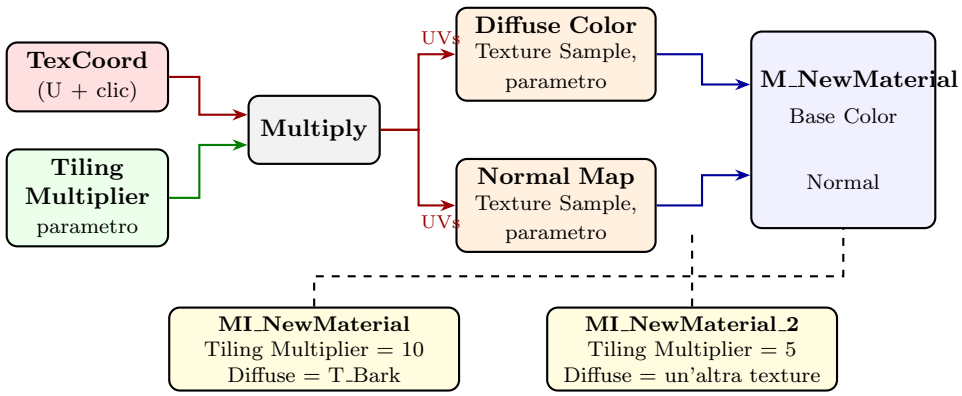


Figure 1.3.9: Il master material con tre parametri (le due texture e il Tiling Multiplier) e due istanze che li impostano in modo diverso. Le istanze non hanno un grafo: hanno solo le caselle dei parametri. Ricostruzione dalla dimostrazione.

Il vantaggio principale. Ricapitolando dalle slide: le Material Instance permettono di **personalizzare velocemente** le superfici “senza andare a modificare il grafico che avete fatto”, e le modifiche si vedono **immediatamente**, “senza dover aspettare la ricompilazione del materiale”.

Approfondimento: che cosa si risparmia davvero. La spiegazione dei tavoli va presa come un'immagine. Il costo principale di un materiale non è "caricarlo" una volta per oggetto, ma **compilarlo**: ogni master material diventa uno o più *shader*, programmi per la scheda video che Unreal deve compilare (e che fanno comparire, nella prima apertura di un progetto, il famoso contatore "Compiling Shaders"). Un'istanza **non** genera un nuovo shader: riusa quello del genitore e cambia solo i valori dei parametri. Cento istanze dello stesso master material sono quindi un solo shader con cento "schede" di valori; cento materiali diversi sono cento shader. Per questo cambiare un parametro in un'istanza è istantaneo, mentre cambiare un nodo nel master material richiede la ricompilazione, come dice la slide. Le istanze del Content Browser si chiamano **Material Instance Constant**; esistono anche le **Material Instance Dynamic**, create durante il gioco da un Blueprint (nodo *Create Dynamic Material Instance*) per cambiare i parametri a *runtime*, per esempio con *Set Scalar Parameter Value*: un materiale che si accende quando la torretta di Igor ha un bersaglio nel mirino.

1.3.16 Le Material Function

"Per chiudere il discorso materiali", l'ultima voce delle slide: le **Material Function**, che oggi non si user-anno. Sono funzioni che raccolgono "una serie di calcoli un po' più complessi": invece di rifare ogni volta dei calcoli che si fanno spesso, li si mette dentro una funzione e si usa la funzione "a vostra discrezione". "È come se fossero delle **parentesi**: voi prendete queste parentesi, le spostate in giro. È un pacchetto che potete portarvi in giro." Servono a velocizzare lo sviluppo quando ci sono cose da ripetere, e possono essere usate e condivise "fra membri del team, fra materiali diversi".

Approfondimento: Material Function e Material Instance. Le due cose risolvono problemi diversi. L'istanza riusa *un intero materiale* cambiandone i valori; la funzione riusa *un pezzo di grafo* dentro materiali diversi. Unreal stesso è pieno di Material Function già pronte (si trovano nel menu dei nodi con l'icona della funzione), e i nodi World Aligned Texture e World Aligned Normal che compaiono alla fine della lezione (sezione 1.3.29) sono, appunto, Material Function del motore.

1.3.17 Il Lerp: interpolare

Restano "un paio di nodi che possono essere carini". Davide ricapitola quelli visti (la costante singola, il colore, il Texture Sample, le operazioni, il Texture Coordinate) e ne aggiunge due: il **Lerp** e il **Panner**. "Se abbiamo capito la regola degli shortcut, questi saranno quale lettera più clic sinistro?" "L." "Esatto": **L + clic** per il Lerp, **P + clic** per il Panner. "Volevo essere sicuro che era arrivata la cosa."

Due colori. Il **Lerp** si occupa dell'**interpolazione lineare** (*linear interpolation*). Per farlo vedere Davide usa due colori, "una roba che non farete mai, non con due colori perlomeno": un **blu** nella **A** e un **rosso** nella **B**. Il Lerp, collegato al Base Color, dà il **viola**. Perché è inutile farlo con i colori? "Perché potevo cercarmi il viola direttamente dal colore." Ma mostra come funziona l'interpolazione. Il Lerp si può fare anche con i numeri, "per fare operazioni matematiche", e anche con le texture.

L'alfa. L'operazione è "gestita interamente dall'**alfa**", il terzo ingresso. L'alfa va **da 0 a 1** e decide quale colore prevale, con A che vale 0 e B che vale 1: con l'alfa a 0 si ha il blu, a 1 il rosso, e in mezzo "una percentuale di quel colore", per esempio 0,7. Modificando l'alfa si va sempre più verso il rosso, o sempre più verso il blu.

Una texture come alfa. Ma l'alfa non deve essere un numero fisso. Davide prende "il nostro fidato Bark" e collega **un suo canale** all'alfa: "vi ricordate il discorso dei canali sulle texture?" Ora quella sequenza di 0 e di 1, di bianco e di nero, "è l'**arbitro** nel decidere dove ci sarà più A e dove ci sarà più B": nelle zone più nere il Lerp mette più blu, in quelle più bianche più rosso. Questa texture è "estremamente bianca", quindi il risultato tende al rosso; con altre texture, o lavorando su questa, si otterrebbero risultati diversi. E

collegando il Multiply delle coordinate anche al Texture Sample dell'alfa, si ottiene un tiling anche dell'alfa, cioè di dove va il blu e dove il rosso. "Chiaro questo processo?" "Sì."

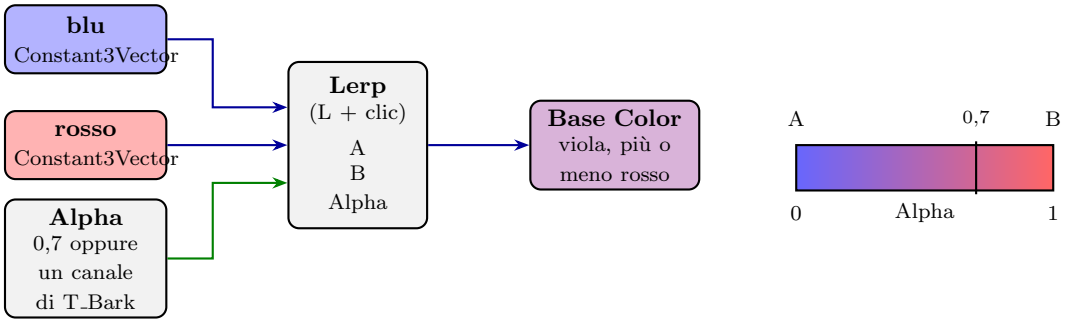


Figure 1.3.10: Il Lerp della dimostrazione: con l'alfa a 0 esce A (il blu), a 1 esce B (il rosso), in mezzo una miscela. Se l'alfa è un canale di una texture, ogni pixel ha la sua miscela: blu dove la texture è nera, rosso dove è bianca.

L'esercizio della pausa. "Provateci voi." Durante la pausa, chi vuole può prendere **due texture**, collegarle a un Lerp e, giocando con l'alfa, vedere quale parte delle due texture viene presa di più e quale di meno.

Approfondimento: la formula del Lerp. L'interpolazione lineare tra A e B con peso α è

$$\text{lerp}(A, B, \alpha) = A\,(1 - \alpha) + B\,\alpha.$$

Con colori, il calcolo si fa canale per canale. Blu $= (0, 0, 1)$ e rosso $= (1, 0, 0)$ con $\alpha = 0{,}5$ danno (0,5; 0; 0,5), il viola della lezione; con $\alpha = 0{,}7$ danno (0,7; 0; 0,3), un viola più rosso. Quando l'alfa è una texture, la formula si applica pixel per pixel, ed è così che si mescolano due materiali su una stessa superficie: erba e terra su un terreno, vernice e ruggine su un metallo, con una maschera in bianco e nero a fare da "arbitro".

1.3.18 Il Panner: far muovere le texture

Il **Panner**[7] "si occupa di far muovere le texture". Davide riprende il Base Color, stacca il resto, e collega il Panner alle **UV** del Texture Sample, "perché lui lavora sulle UV della texture". Selezionando il nodo (o direttamente sul nodo) si vedono **Speed X** e **Speed Y**. Con Speed X a 1 la *preview* comincia a muoversi sull'asse X, "anche un po' troppo veloce per i miei gusti"; con 0,3 in X e 0,1 in Y si ha un Panner che si muove su entrambi gli assi.

A che cosa serve. "Che materiale potremmo volerci fare con questo nodo?" "Dell'acqua." "Dell'acqua. Qualsiasi fluido, qualsiasi cosa che si muove." Il Panner ha il parametro di velocità e l'ingresso delle **coordinate**: per combinarlo con il tiling si collega il Texture Coordinate (o il Multiply) all'ingresso Coordinate. C'è poi l'ingresso **Time**, con cui si può decidere "quando e come deve muoversi"; altrimenti, di default, usa il *game time*, "cioè è sempre in movimento".

Il master material cambia, l'istanza segue. Mentre il Panner lavora, il pavimento nella scena **si muove**: sul pavimento c'è la Material Instance, e "nonostante stiamo facendo delle modifiche al materiale originale e non alla Material Instance, la Material Instance continua a modificarsi". È il rovescio del rapporto tra master e istanze: quello che cambia nel genitore cambia in tutte le istanze.

[7] Nella trascrizione "panel", "planner" e "banner".

Approfondimento: il Panner in una formula. Il Panner non fa altro che sommare alle coordinate un tempo moltiplicato per una velocità:

$$(u', v') = (u + s_x\, t,\ v + s_y\, t),$$

dove t è il tempo in secondi e s_x, s_y le velocità in "texture al secondo": con Speed X a 1 la texture scorre di una larghezza intera ogni secondo, per questo era troppo veloce. Siccome la texture si ripete, lo scorrimento è infinito e senza salti, purché la texture sia *tileable*, cioè con i bordi che combaciano. Due Panner con velocità e tiling diversi sommati tra loro sono il trucco classico per l'acqua e per la lava: il movimento non si ripete mai uguale.

1.3.19 La pausa: il progetto che non si trova, e Git

"Visto che comunque ci siamo fatti un'oretta e mezza", Davide chiude per ora il discorso materiali e propone una pausa di una decina di minuti. Al ritorno: l'import, le mesh, e "una cosina con la torretta che c'è nei file che vi abbiamo passato", così che Igor, quando torna, potrà farci "un piccolo sistema di puntamento".

Il progetto che Unreal non vede. Al rientro, in attesa degli altri, una studentessa racconta il suo problema: ha scaricato e decompresso la cartella del Drive, ma aprendo Unreal il progetto non compare, e deve aprirlo "a mano" da un'altra parte. Davide spiega che dipende probabilmente dal percorso in cui è salvata la cartella: se la si mette dove sono salvati gli altri progetti, Unreal dovrebbe trovarla; in ogni caso, aprire il progetto dal file eseguibile nella cartella funziona. "Se non è un problema che ti impedisce di accedere al progetto, fa niente."

Come si lavora insieme a distanza? Uno studente, programmatore, fa una domanda "fuori dall'argomento": i programmatori usano **Git**, ma i file di un progetto di Unreal sembrano troppo pesanti per lavorarci in contemporanea da lontano. Davide conferma che **Git**, "o diciamo repository di sorta", è "la cosa principale che utilizzerai". Con **Unity** ha sempre avuto meno problemi, perché Unreal "va gestito in maniera un po' più delicata", ma entrambi supportano senza problemi repository e *source control*. Esistono anche altri metodi, che Davide non ha mai provato: per Unreal, "una cosa che io sento da una vita", un modo per lavorare **contemporaneamente** sulla stessa cosa, tramite **VPN** o in **LAN** e con dei *plugin* specifici, in modo che due colleghi modifichino attivamente lo stesso livello. Con Git invece "ognuno lavora sul suo e poi si *merge* tutto insieme". "Git è quello che ho utilizzato sempre, è il più comodo, quindi dovresti trovarti abbastanza a casa."

Approfondimento: Git LFS, Perforce e Multi-User Editing. I timori dello studente sono fondati. Git nasce per file di testo, e gli *asset* di Unreal (`.uasset`, `.umap`) sono file **binari**, spesso grandi, che Git non sa fondere: se due persone modificano lo stesso livello, una delle due modifiche va persa. Per questo con Unreal si usa **Git LFS** (*Large File Storage*), che tiene i file pesanti fuori dalla cronologia normale e permette di **bloccare** (*lock*) un file mentre lo si modifica. Nell'industria, per i progetti Unreal, è molto diffuso **Perforce** (Helix Core), il sistema che Epic stessa usa e che l'editor supporta direttamente, proprio per la gestione dei file binari e dei blocchi. Il sistema "sentito da una vita" di cui parla Davide esiste: è il plugin **Multi-User Editing**, con cui più editor connessi in rete (in LAN o via VPN) condividono una stessa sessione e vedono le modifiche degli altri in tempo reale; nato per la produzione virtuale nel cinema, si può usare anche nello sviluppo di giochi, ma non sostituisce il *source control*, a cui le modifiche vanno comunque consegnate alla fine.

1.3.20 L'import e la torretta

Passato "il nostro quarto d'ora", si riparte: si importano le prime mesh, la **torretta**, "le tre parti della torretta, perché l'abbiamo divisa in tre parti".

La finestra di import, sulla slide. Quando si importa una mesh compare una finestra ("che può essere più larga, può essere più stretta") piena di parametri. Le cose principali che si possono fare da lì:

- **generare automaticamente le collisioni**, se non sono presenti nel file FBX, perché magari gli oggetti non sono stati creati con le collisioni;
- **modificare il transform** dell'oggetto importato "proprio di base": ruotarlo sul suo **pivot**, traslarlo. "Avete già parlato di cos'è il pivot con Igor, giusto? Il fulcro che vi permette di traslare e ruotare gli oggetti all'interno della scena." Con la funzione di **reimport**, si vedrà, questa possibilità diventa molto utile;
- adattare le **unità** e gli **assi** tra il software di origine e Unreal: **convertire la scena** dalle coordinate FBX a quelle di Unreal, **convertire le unità** (Unreal lavora in centimetri), oppure usare **Force Front X Axis**, che cambia l'asse che Unreal considera come "fronte" dell'oggetto. Davide l'ha usato poche volte: una volta il modello di un *character*, importato normalmente, "veniva rivoltato, come se il dentro fosse fuori", e cambiare l'asse frontale gli ha restituito la mesh com'era stata esportata ("mi pare fosse Blender");
- decidere se importare anche i **materiali** e le **texture** collegati alla mesh, e **dove** cercarli: in locale, in un'altra cartella, e così via.

Il pulsante Import. Prima di importare, Davide spegne il Panner ("facciamo che smetto di far muovere il pavimento") e il pavimento torna normale. Nel Content Browser c'è "questo pulsantone con **Import**", che apre l'esplora risorse del computer per cercare i file. Nella cartella del materiale didattico c'è una cartella **torretta**, con tre **static mesh**: **SM_Turret_Base**, **SM_Turret_Cannon** e **SM_Turret_Pivot**[8].

La finestra di import, dal vivo. Importandole si apre la finestra ("nel mio caso è così enorme, ma perché io tendo a usarla così"), e si ritrovano le opzioni della slide: la possibilità di modificare traslazione e rotazione, "anche singolarmente sui vari assi" ("questo è qualcosa che faremo più tardi"); i **LOD** ("decidere il gruppo dei LOD"); le collisioni, se e come importarle. Se si importa una **Skeletal Mesh**[9], come un personaggio, la finestra permette di collegarla a uno **scheletro** esistente, di generarne uno, oppure di importare solo delle **animazioni**: "faremo una lezione specifica sulle animazioni, quindi per il momento glissiamo". In alto ci sono anche le sezioni per andare più veloci, tra cui **Import Materials** e **Import Textures**, con la **Search Location** in cui cercarli.

Niente materiali e texture, per ora. La consegna di Davide: **togliere la spunta** a Import Materials e Import Textures, e fare **Import**. Unreal importa i tre oggetti (Davide elimina quelli della prova precedente e salva). Perché togliere le spunte? Se si fossero importati gli oggetti con materiali e texture, Unreal avrebbe importato automaticamente tutte le texture, creato i materiali adatti e li avrebbe anche **applicati**, "in modo da averli già precisi sulla torretta". Ed è proprio quello che si farà, ma **a mano**, "così vediamo effettivamente anche il processo". "Ci sono stati problemi? Tutto a posto con la torretta?" "Sì, nessun problema."

1.3.21 Che cos'è una mesh

Mentre la torretta è importata, Davide torna sulle slide per "le ultime cose sulle mesh". Sulla slide c'è un **vaso** in *wireframe* e, sotto, una semplificazione. Una mesh è "un insieme di **vertici**, **spigoli** e **facce** che vanno a creare un oggetto in tre dimensioni".

- I **vertici** sono i **punti** che costituiscono gli angoli dell'oggetto, punti nello spazio.
- Gli **spigoli** (*edge*) sono i **segmenti** che collegano due vertici.
- Le **facce** sono le **superfici piatte** definite da più segmenti: sulla slide, "tutti i triangoli".

[8] Nella trascrizione "Turret Base, Turret Cannon e Turret Pivo"; il prefisso SM_ è detto a voce.
[9] Nella trascrizione "scheda dal Mesh" e "Scheletal Mesh", come nella seconda lezione.

Opzione	A che cosa serve
Generate Missing Collision	genera le collisioni se il file FBX non le ha
Import Translation / Rotation / Uniform Scale	sposta, ruota, scala l'oggetto "alla radice": cambia dove sta il pivot rispetto alla geometria
LOD Group	sceglie un gruppo di livelli di dettaglio (*Level Of Detail*)
Convert Scene, Convert Scene Unit	adattano assi e unità di misura del file FBX a quelli di Unreal (centimetri)
Force Front X Axis	cambia l'asse considerato come "fronte"
Skeleton / Import Animations	per le Skeletal Mesh: scheletro esistente o nuovo, solo animazioni
Import Materials, Import Textures	importano e applicano materiali e texture collegati
Search Location	dove cercare materiali e texture da importare

Table 1.3.3: Le opzioni della finestra di import citate a lezione, con i nomi della finestra di Unreal Engine 5 (FBX Import Options; il raggruppamento può variare tra le versioni). Per la torretta: Import Materials e Import Textures senza spunta.

Il **triangolo** è "la forma più comoda che esiste a livello di calcolo", e tendenzialmente tutti gli oggetti saranno composti da "un insieme di triangolini". Più triangoli insieme, più facce, formano dei **poligoni**; i poligoni combinati formano delle **superfici**; e la superficie determina la mesh. Tutto questo si può vedere anche nell'engine, con la visualizzazione **wireframe**: l'oggetto non "come appare", ma "come griglia, come insieme di tutti i poligoni, vertici e spigoli".

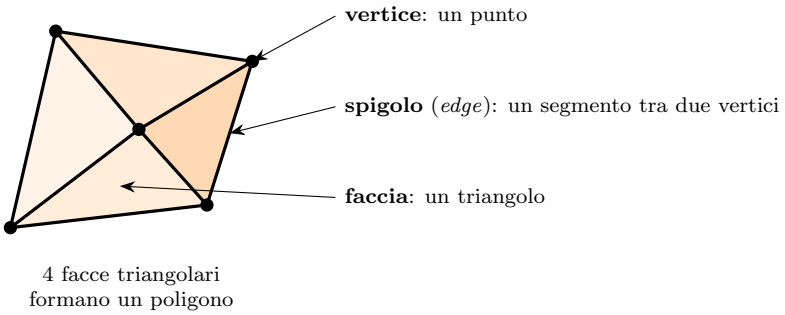


Figure 1.3.11: Vertici, spigoli e facce, come nella semplificazione della slide. Il triangolo è l'unità di calcolo: anche le facce a quattro lati dei programmi di modellazione, quando arrivano in Unreal, vengono divise in triangoli.

1.3.22 Dimensioni, unità di misura e formati

Quando si porta una mesh in Unreal, mentre la si produce o mentre la si cerca, bisogna rispettare alcune cose.

Le dimensioni. La cosa più comune: si decide di fare "una mappa molto carina" su Unreal, si cercano mesh su internet o su **Fab** ("su Fab questo problema c'è meno, però utilizzando siti esterni può capitare"), e la mesh importata è **gigantesca** o **incredibilmente piccola**, perché chi ha fatto l'export, o chi ha fatto l'import, non ha rispettato "le misure di adattamento e transizione da un software all'altro". Unreal usa come unità di misura i **centimetri**; Blender, per esempio, "usa altra roba". Nel passaggio da Blender a Unreal bisogna controllare che la conversione delle dimensioni sia corretta.

FBX. "Altra cosa molto importante": Unreal "tendenzialmente legge soltanto **FBX**". Sono supportati anche altri formati, come gli **OBJ**, "ma non assicurano un corretto funzionamento all'interno dell'engine". Il consiglio: "importate sempre unicamente FBX".

I formati delle texture. Per le texture i formati supportati sono molti (sulla slide c'è un elenco), ma nella pratica si usano soprattutto **JPEG** e **PNG**, "i più comuni, a meno che non state facendo qualcosa di molto specifico".

Approfondimento: Blender, i metri e il fattore 100. Blender lavora di default in metri, con una "unità" uguale a un metro; Unreal lavora in centimetri. Un cubo di lato 2 in Blender è quindi un cubo di 2 m, cioè 200 unità di Unreal: se l'export non converte, l'oggetto arriva cento volte troppo piccolo (o, con altre impostazioni, cento volte troppo grande). La soluzione standard è nell'esportatore FBX di Blender (l'opzione *Apply Scalings* su *FBX Units Scale*) o, lato Unreal, nella scala uniforme e nella conversione delle unità della finestra di import. Un controllo rapido: un personaggio umano in Unreal è alto circa 180 unità, come il manichino dei progetti modello; se la mesh appena importata è molto diversa, c'è un problema di scala. Oltre a FBX, le versioni recenti di Unreal Engine 5 importano anche **glTF** e **USD**, ma FBX resta il formato più collaudato, come dice Davide.

1.3.23 Il pivot del cannone e il Reimport

Davide apre le tre mesh (la base, la torretta e il pivot) e le piazza in scena. Sono state esportate con il **pivot nello stesso punto**: per sovrapporle senza spostarle a mano basta **copiare la X e la Y** di un pezzo negli altri. Ed ecco la torretta, "a parte il fatto che non c'ha il materiale".

I problemi della torretta. Ma "sta torretta ha dei problemi: quali sono?" Il primo si vede: per qualche motivo **non è allineata**, ed è "scomodamente" ruotata: trovare la rotazione giusta sarà scomodo. Il secondo è più sottile. Si supponga di mettere i tre pezzi insieme in un **Blueprint**. Che cosa deve fare una torretta? Deve **tracciare dei bersagli**: ruotare su sé stessa, ma anche andare su e giù, se il bersaglio è in alto o in basso. E qui c'è il problema: il **pivot del cannone** non è in una posizione comoda per la rotazione verticale, perché sta **in basso**, in fondo. Ruotare sul piano orizzontale non dà problemi, ma per alzare e abbassare la canna il pivot dovrebbe essere "più o meno all'altezza di questo **bullone**", il fulcro, e da lì il cannone dovrebbe essere manipolato e ruotato (figura 1.3.12).

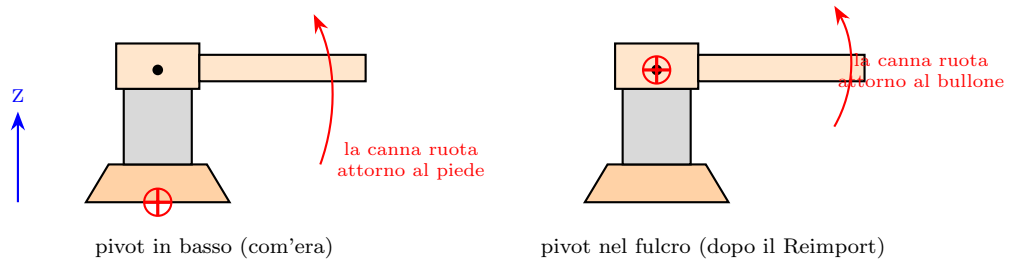


Figure 1.3.12: Il problema del pivot del cannone, visto di lato (schema non in scala). Con il pivot (il mirino rosso) in fondo, alzare la canna la farebbe girare attorno al piede, staccandola dal supporto; con il pivot nel fulcro, all'altezza del bullone, la canna si alza e si abbassa come una vera torretta. La rotazione orizzontale, attorno all'asse Z, funziona invece in entrambi i casi.

Come si sposta un pivot. "In realtà possiamo farlo in tutta una miriade di modi." Con la **Modeling Mode** si potrebbe rimodellare l'oggetto. Ma lo si può fare anche **nell'import**: si fa tasto destro sulla mesh e **Reimport Mesh With Dialog**, che riapre la finestra di import vista prima. Unreal importa lo stesso oggetto e lo **sovrascrive**, ma permette di cambiare come verrà posizionato: l'**offset della rotazione** e l'**offset della traslazione**.

I numeri. Davide se li era scritti, ma non li ricorda, e li ricava. Per sapere quale asse toccare, guarda il *gizmo*: il cannone deve essere allineato con il resto della torretta (e lo stesso il pivot), e il pivot del cannone deve stare non "qui giù in fondo", ma allineato con il fulcro, sull'**asse Z**, "perché è questo blu". Per questa torretta, "quindi anche per voi", i parametri sono:

- per il **cannone**: traslazione Z = −135, che reimporta il cannone "più attaccato al pivot", e rotazione = 113, che lo riallinea[10];
- per il **pivot**: "questa volta non serve modificare la traslazione", ma solo la rotazione, sempre 113.

Dopo il reimport il cannone "sta sotto" (la sua origine ora è nel fulcro, quindi nella posizione di prima risulta più basso) e va rimesso a posto; ma ora, ruotandolo, si alza e si abbassa nel modo giusto. "E chiaramente poi questo farete più avanti, con dei Blueprint." Si salva tutto.

Approfondimento: perché il pivot conta così tanto. In Unreal ogni rotazione e ogni scala di un componente avvengono **attorno alla sua origine**, cioè al pivot della mesh. Un Blueprint di torretta, di solito, ha la base come componente radice, il supporto come figlio che ruota attorno a Z (il *yaw*, per seguire il bersaglio a destra e a sinistra) e il cannone come figlio del supporto che ruota attorno a Y (il *pitch*, su e giù). Per questo ogni pezzo deve avere l'origine esattamente nel punto in cui gira nella realtà: se l'origine è sbagliata, nessun nodo di rotazione potrà correggerla senza calcoli aggiuntivi. Il "piccolo sistema di puntamento" che Igor farà con questa torretta (con nodi come *Find Look at Rotation*, che calcola la rotazione necessaria per guardare un punto) si appoggia interamente a questo lavoro preparatorio.

1.3.24 Mettere ordine: le cartelle

Prima di importare le **texture**, "per dare un po' di ordine", Davide crea nel Content Browser una nuova cartella, **Torretta**, ci sposta dentro tutto ("muoviamo, così è un po' più pulito"), e dentro crea un'ulteriore cartella, **Texture**, in cui importare le texture della torretta. "Questo non è obbligatorio, è solo una questione di *best practice*, cioè è come il discorso delle Material Instance. Non vi vengo a trovare a casa se non lo fate, però vi risulta tutto più comodo."

1.3.25 Lo Static Mesh Editor e le collisioni

Con un doppio clic su una mesh si apre lo **Static Mesh Editor**: "andiamoci a vedere effettivamente un po' qui dentro cosa possiamo vedere delle mesh".

Gli slot dei materiali. A destra, nei **Material Slots**, queste mesh hanno **due slot**: "non è la normalità". (Una delle tre ne ha uno solo.) Il modo in cui la torretta è stata suddivisa fa sì che, per rendere bene i materiali, servano due slot: la base e il cannone condividono in parte il materiale del **pivot**. Nei due slot si inseriranno i materiali più avanti. Altre impostazioni (Davide accenna a quelle sui LOD e sulla *Nanite*[11]) non si vedono oggi.

Le modalità di visualizzazione. Dal menu della *viewport* (la prospettiva "non ci interessa") si cambia la modalità: **Lit**, quella normale; **Unlit**, "com'è il mio oggetto se non è colpito dalla luce?"; e **Wireframe**: "vedete quello che dicevamo prima?" Vertici, facce ed *edge* della mesh, che si potrebbero modificare con la **Modeling Mode** ("se avanzerà il tempo con le lezioni, sarebbe carino vederla più avanti"). Ci sono anche **Lighting Only**, le **Reflections** e altre ancora. Dal menu **Show** si possono mostrare i **vertici**: "dove sono i vertici di questo oggetto?"

[10] La trascrizione dice prima "113 per la traslazione" e poi si corregge: "e 113 sulla rotazione". L'asse della rotazione non è detto; trattandosi di riallineare la torretta nel piano orizzontale, è con ogni probabilità la rotazione attorno a Z (il *yaw*).

[11] La trascrizione dice "per quanto riguarda la night": la ricostruzione come Nanite, la tecnologia di geometria virtualizzata di Unreal Engine 5 che ha una sua sezione nei dettagli della mesh, è probabile ma non certa.

Simple e complex. Un'altra cosa utile "da un punto di vista di gameplay" sono le **collisioni**. Dal menu Show si attiva la **Simple Collision**: "queste sono le collisioni del mio oggetto". In scena, il *character* interagirà con queste: ci salterà sopra, o lo dovrà scavalcare. Ma oltre alle **simple collision** ci sono le **complex collision**. La differenza:

- le **simple collision** sono forme che si possono posizionare a mano, anche da qui, e rifare da capo;
- le **complex collision** usano la struttura in vertici della mesh stessa come collisione. "È qualcosa che è molto più pesante rispetto a una collisione semplice, e torna utile solo in alcuni casi specifici."

Modificare le collisioni. La collisione semplice si può selezionare, spostare, anche eliminare; e se ne possono **aggiungere** altre: una **sfera**, di cui si decide la grandezza; una **capsula**, che "è tendenzialmente la collisione che usano i vostri *character*": il personaggio di solito non ha una collisione super precisa, "è un capsulone", a meno che non si voglia davvero il contrario; una **box**. E una box, per questa mesh, "è più ottimizzata rispetto alla collisione che questa aveva di base": all'atto pratico fa la stessa cosa "con molti meno vertici". Su una singola mesh cambia poco; su molte mesh aiuta le prestazioni. Ci sono anche collisioni più complesse, ma "idealmente, il modo migliore per fare delle simple collision è letteralmente prendere delle **forme base**", magari più d'una, "piuttosto che avere tanti vertici sulla stessa collisione".

Collisione	**Com'è fatta**	**Quando**
Box	un parallelepipedo	oggetti squadrati; spesso basta anche per oggetti più complessi, ed è la più leggera
Sphere	una sfera	oggetti tondi, palle
Capsule	un cilindro con due semisfere	i *character*: "un capsulone"
Convex (più forme)	poliedri convessi, magari più d'uno	quando una forma sola non basta; meglio poche forme base che tanti vertici
Complex	i triangoli della mesh stessa	casi specifici: precisione massima, costo molto più alto

Table 1.3.4: Le collisioni di una static mesh viste nello Static Mesh Editor. Le prime quattro sono *simple collision*, che si aggiungono e si modificano dal menu Collision dell'editor; l'ultima è la *complex collision*.

Approfondimento: chi usa quale collisione. Unreal tiene le due collisioni separate perché le usa per cose diverse. La **simple** è quella della **fisica**: la pallina della seconda lezione che rotola sulla piattaforma, il *character* che cammina e salta, gli oggetti che cadono; i calcoli fisici su forme semplici sono molto più veloci. La **complex** si usa soprattutto per le **interrogazioni** precise, come un *line trace* (un "raggio" lanciato per sapere che cosa colpisce un proiettile o che cosa sta guardando il giocatore), quando la precisione al triangolo serve davvero. Nei dettagli della mesh, il campo *Collision Complexity* decide come combinarle; l'opzione *Use Complex Collision As Simple* usa i triangoli anche per la fisica, ed è comoda per oggetti statici di forma strana, ma va usata con giudizio, per il costo di cui parla Davide.

1.3.26 Raggruppare e fondere: Merge Actors

"Se ad esempio vogliamo avere tutti e tre gli oggetti assieme", o semplicemente spostarli insieme, ci sono due strade.

Raggruppare. Si selezionano tutti e tre e li si **raggruppa** con **Ctrl+G**; si spostano insieme; con **Shift+G** si toglie il gruppo.

Fondere. Oppure li si **fonde** (*merge*). Si evidenziano i tre pezzi in scena, si va nel menu **Actor** in alto e si sceglie **Merge Actors**. Unreal chiede dove salvare il nuovo oggetto e con che nome (per esempio **SM_Turret**[12]), e crea una **nuova mesh**, "che è semplicemente la fusione delle tre mesh precedenti": una somma, un *merge*, appunto.

Il merge, di nuovo. Più tardi uno studente dice di essersi perso come si fa il merge, e Davide lo rifà con calma. I tre oggetti (il cannone, la base e il pivot) sono in scena. Con il **tasto destro** su di essi ci sono delle opzioni, "però queste sono opzioni limitate". Per averle tutte si usa il menu **Actor** in alto: mostra gran parte delle stesse opzioni, ma in basso, nella sezione degli strumenti di Unreal Engine, c'è **Merge Actors**. Al merge Unreal chiede dove salvare e con che nome, e "ti fonde i tre oggetti". Nello stesso menu c'è anche **Chaos**, che "serve per gli oggetti distruttibili": cose più avanzate, che probabilmente non si vedranno nel corso, ma da approfondire "se vi capiterà di voler mettere distruttibilità all'interno del vostro gioco".

Approfondimento: gruppo, merge o Blueprint? Le tre soluzioni non sono equivalenti. Il **gruppo** è solo una comodità dell'editor: gli actor restano separati, e in gioco non cambia nulla. Il **merge** crea una mesh nuova, una sola: meno oggetti da disegnare (utile per le prestazioni, per esempio con tanti pezzi di arredo statici), ma i pezzi **non si possono più muovere uno rispetto all'altro**. Per la torretta di Igor, che deve ruotare il supporto e alzare la canna, il merge è quindi la scelta sbagliata: serve un **Blueprint** con i tre pezzi come **componenti** separati, imparentati tra loro, come si è visto nella seconda lezione. Il merge della lezione è una dimostrazione dello strumento, e serve anche a rispondere alla domanda sui materiali (sezione 1.3.28).

1.3.27 I materiali della torretta

"Ora abbiamo importato le texture. Creiamoci i materiali di queste torrette." Tre materiali, uno per pezzo: **M_Turret_Base**, **M_Turret_Pivot** e **M_Turret_Cannon**. Si potrebbero fare molto elaborati, "ma partiamo semplicemente andando a utilizzare le texture che ci sono state date".

Quattro Texture Sample. Nel materiale della base Davide prende **quattro Texture Sample** e ci mette le texture della base: **Base Color**, **Metallic**, **Roughness** e **Normal** (Turret Base Color, Turret Base Metallic, Turret Base Roughness, Turret Base Normal). Poi le collega ai pin corrispondenti del nodo risultante: il colore al Base Color, la Metallic al Metallic, la Roughness alla Roughness, la Normal alla Normal (figura 1.3.13).

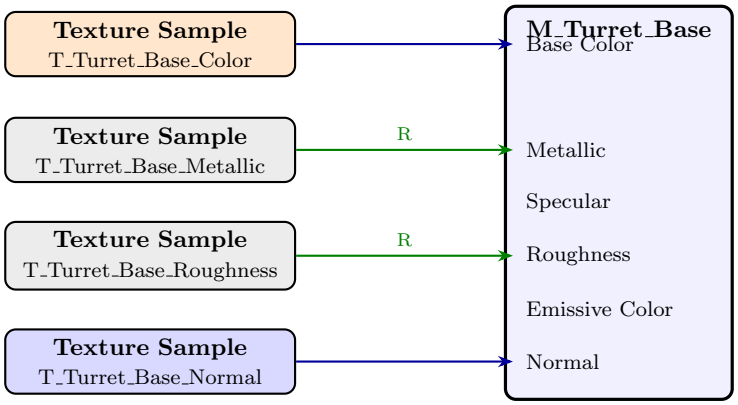


Figure 1.3.13: Il materiale della base della torretta, fatto a mano: quattro Texture Sample, quattro pin. Metallic e Roughness sono mappe in bianco e nero, e basta un canale (per esempio R). M_Turret_Pivot e M_Turret_Cannon sono identici, con le loro texture. Ricostruzione dalla dimostrazione; i nomi esatti dei file sono indicativi.

[12] Nella trascrizione "smTourette".

Le preview. "Mi ero dimenticato di farvi vedere una cosa." Nella *preview* del materiale si può scegliere la forma: il **cilindro**, la **sfera**, il **piano**, il **cubo** e il **custom**. Con il custom, se si seleziona una mesh nel Content Browser e si clicca il pulsante, la preview mostra **quella mesh** con il materiale applicato. La prima volta non la mostra: "sta compilando". Poi compare (Davide aveva collegato una texture "dalla parte sbagliata", e la sistema).

Gli slot. Il materiale è salvato e compilato. Sulla mesh della torretta ci sono i **due slot**: in uno c'è M_Turret_Base, ma "per come è stata strutturata la torretta" nell'altro va il materiale del **pivot**. "Sta rallentando veramente tanto il PC."

Un grafico, un engine. Davide si ferma su che cosa si è fatto: "qui non abbiamo fatto altro che utilizzare le mesh che ci sono state fornite da un grafico". Idealmente un grafico, in un software a parte, ha realizzato la mesh e le texture, e ce le ha passate; noi le abbiamo inserite. Si potrebbe lavorare ancora sull'estetica della torretta modificando i materiali, "tuttavia, in questo caso non è necessario".

Il pivot e il cannone, più furbi. Per il materiale del pivot Davide è "un po' più furbo": cerca la texture *Turret Pivot Color*, e le altre le copia da lì, perché nella lista "stanno direttamente dopo". Collega Metallic, Roughness e Normal, salva, "e abbiamo anche il materiale del nostro pivot". Poi lo stesso per il cannone: la texture *Cannon Color*, **Ctrl+C**, **Ctrl+V**. Lì Davide ricorda le scorciatoie classiche: "tutti i classici shortcut di Control", Ctrl+C, Ctrl+V, Ctrl+Z, Ctrl+Y, Ctrl+A, "sono applicabili anche a Unreal nella maggior parte dei casi".

Salvate spesso. Uno studente chiede perché il suo non si è applicato: "Salvalo." E Davide ne fa una regola: "puntualmente mi capita di vedere gente che sta lavorando a qualcosa, non salva per qualche motivo, Unreal crasha e perde tipo tutto il lavoro che stava facendo. E quindi salvate spesso."

La torretta finita. Nei due slot vanno il pivot e il cannone. "Oh, eccoci qua. Bello lucido." Salvato tutto, la torretta ha i suoi materiali.

Ritoccare il colore con un Lerp. "Se volete che la torretta sia meno lucida, o che sia meno nitido l'arancione", si può intervenire sui singoli materiali. Davide improvvisa sul cannone: non è sicuro di volere il colore "così netto", quindi prende un **nero** e un **Lerp**, collega il nero e la texture del colore alle due entrate e mette nell'alfa un parametro con cui decidere "quanto si deve vedere", per esempio 0,6. Si può convertire in parametro, creare una **Material Instance**, e regolarlo da lì. Al posto del nero si può mettere un altro colore, un verde: "viene un po' più verdastra, è troppo verde", e con l'alfa a 0,8 viene "questo verde mezzo, insomma, un po' scuro, un po' cromato". "Qui chiaramente sto improvvisando cose, ma cose anche stupide. Ma nel momento in cui voi avete un'idea un po' più chiara, un po' più specifica di quello che volete, potete andare a fare un po' più di cose."

Approfondimento: lo stesso lavoro con un master material. I tre materiali della torretta hanno lo stesso identico grafo: cambiano solo le texture. È il caso da manuale per le Material Instance della prima parte della lezione: un solo master material (per esempio `M_Turret`) con quattro Texture Parameter (colore, metallic, roughness, normal) e magari un Lerp per la tinta, e tre istanze (`MI_Turret_Base`, `MI_Turret_Pivot`, `MI_Turret_Cannon`) che impostano le texture di ciascun pezzo. Un solo shader da compilare, e il ritocco verdastro del cannone diventa una casella da regolare nell'istanza. Se le texture fossero state importate insieme alle mesh (Import Materials e Import Textures), Unreal avrebbe creato da solo i materiali, uno per slot, con un grafo simile a quello della figura 1.3.13.

1.3.28 Le domande: il Reimport e il merge

"Ora tutto chiaro il discorso dell'import e dei materiali?" Due domande.

Perché il Reimport con la Z? Una studentessa non ha capito il reimport modificando la Z. Davide riapre il cannone. Il reimport è servito a **modificare la mesh** alla radice. L'oggetto importato, modellato da qualcun altro, era fatto in un certo modo, e non è un problema in sé: lo si può abbassare e girare a mano, in scena. "Però poi, tutte le volte che io prendo e rimetto in scena l'oggetto, l'oggetto sarà sempre posizionato in quel modo e ruotato in quel modo." Il reimport invece "modifica proprio alla radice il modo in cui funziona l'oggetto". E il problema principale non era la rotazione ("so che l'angolo è 113, allora lo rimodifico"), ma il **pivot**, "in una posizione scomoda": il reimport è un modo per spostarlo.

La Modeling Mode. "Non è l'unico modo." Davide apre la **Modeling Mode**, cerca lo strumento del **pivot**, e mostra che lo si può mettere al centro, a sinistra, a destra, oppure "nel punto specifico che dico io: lo prendo e me lo sposto". Ma questa è "una soluzione che hanno aggiunto con Unreal Engine 5": prima la Modeling Mode non c'era, e in Unreal Engine 4 reimportare era il modo comodo per farlo "senza avere bisogno di rimodellare l'oggetto a posteriori". "Fai la stessa cosa, semplicemente non hai fatto un'operazione extra."

Una prova. Davide stesso è curioso: se si reimporta di nuovo con -135 e 113? Il reimport **sovrascrive** quello che c'era prima, ma il cannone in scena conserva la rotazione che gli era stata data a mano. Davide lo rimette dove stava ("io sto facendo a cazzotti da tutta la lezione con la finestra di Zoom, se non si fosse capito").

Che cosa succede ai materiali con il merge? Era la domanda arrivata in chat durante la costruzione dei materiali: se faccio il merge e i pezzi della torretta hanno texture diverse? Davide prova: seleziona i tre oggetti, **Actor**, **Merge Actors**, nome *Turret*, salva. Il risultato è un oggetto con **tre slot** per i materiali: lo slot 0 è tutta la parte del cannone, lo slot 1 il pivot "e tutta questa zona qui", e così via. Se in uno slot si mette un materiale vuoto, "diventa tutto nero": "vedete, tutta questa zona adesso viene influenzata da quel materiale lì". Risposta: con il merge l'actor viene fuso, e i materiali vengono **riposizionati negli slot** della nuova mesh.

1.3.29 Lo stretch, lo z-fighting e i materiali World Aligned

"Mi è venuto in mente un'ultima cosa che vi potrei far vedere per quanto riguarda i materiali, e poi vi vorrei far fare una cosina." Manca una mezz'oretta, "visto che Igor vi ha fatto fare delle cose, ho visto che siete molto partecipi. Quindi vorrei fomentare questa cosa."

M_Test. Davide ricrea il materiale della corteccia, cancellato per sbaglio: **M_Test**, con due Texture Sample (Bark e la sua Normal) e un Texture Coordinate, "proprio la roba più facile del mondo", con il tiling a 10.

Ingrandire e stirare. "Ora vediamo se abbiamo capito come funziona tutto quanto." I materiali sono collegati alle **UV** di un oggetto. Quindi se l'oggetto si **ingrandisce**, al materiale "non dovrebbe succedere granché": si ingrandisce anche lui ("la risoluzione non ne giova particolarmente"). E se l'oggetto si **stira** (*stretch*), tirandolo verso destra? "Credo si stira anche lui." Esatto: il materiale si allunga. "In alcuni casi non è neanche particolarmente brutto", ma la corteccia "non sembra più corteccia"; e tirando di più, "così è brutto".

Il pavimento grande. Se si vuole fare un pavimento per una zona molto ampia, dover pensare a questo "può essere una noia": per non stirare il materiale si dovrebbero fare tante copie dello stesso pezzo, una attaccata all'altra, "che tendenzialmente non è quello che voglio", perché è scomodo.

Una digressione: lo z-fighting. "Visto che è una domanda che arriverà, anticipo": se due piani si trovano **nella stessa posizione**, "clippano": la superficie sfarfalla a chiazze, perché l'engine "non sa se utilizzare il materiale dell'oggetto a destra o dell'oggetto a sinistra". Le soluzioni: separarli davvero, oppure spostarne uno di pochissimo in **Z**, per esempio di 0,1: "è impercettibile, non si nota, però uno è leggermente più sopra rispetto all'altro", e il problema sparisce.

Allineato al mondo. "C'è un modo per far sì che il mio materiale non si stiri quando stiro l'oggetto? Chiaramente sì, perché se no non avrei fatto questa domanda." Bisogna fare un materiale che non è allineato con le coordinate dell'oggetto, ma con quelle del **mondo**. Davide lo spiega con il *gizmo* di spostamento: un oggetto ruotato si può muovere lungo gli assi del mondo o lungo i suoi assi locali, e il pulsante in alto passa dall'uno all'altro. "Più o meno il discorso è lo stesso": nel caso del materiale non si parla di assi, ma di UV.

I nodi World Aligned. Davide copia il materiale (**M_Test1**) e usa due nodi, **World Aligned Texture** e **World Aligned Normal**. Non hanno scorciatoie: si cercano per nome. Hanno "un sacco di parametri", ma per ora ne servono due: **Texture Object** e **Texture Size**. World Aligned Texture va nel **Base Color**, World Aligned Normal nella **Normal**, usando l'uscita **XYZ**, "perché almeno prendo tutti gli assi": il materiale è allineato al mondo su tutti e tre.

Texture Size e Texture Object. Il **Texture Size**, un vettore a tre valori, "fa la stessa cosa del Texture Coordinate": regola la grandezza della ripetizione. Perché allora non tenere il Texture Coordinate sulle UV del Texture Sample? Perché i nodi World Aligned **non prendono** un Texture Sample: vogliono un **Texture Object**. Si converte: tasto destro sul Texture Sample, **Convert to Texture Object**. Il Texture Object "è un oggetto molto più semplice": non ha ingressi, ha una sola uscita, niente RGB; "è molto più limitato in quello che può fare", ma è il "messaggio" che serve ai nodi World Aligned.

La prova. Davide prova diversi valori di Texture Size (1, 2, 10) per avvicinarsi al risultato dell'altro materiale, con qualche esitazione su quale sia "troppo" e quale "troppo poco" ("è proprio piccola la texture, allora"), e applica il nuovo materiale a un secondo oggetto. Poi **stira** entrambi: il primo si stira; il secondo "semplicemente genera nuove texture", rimane orientato con il mondo, "rimane sempre della stessa proporzione".

Dov'è la fregatura. "Se questo rimane sempre proporzionato anche quando stiriamo le cose, perché non dovremmo avere tutti i materiali World Aligned? Dov'è la fregatura?" Eccola: essendo allineato al mondo, se l'oggetto si **muove** o **ruota**, il materiale **resta fermo**. Una **porta** con un materiale normale si apre e il materiale resta attaccato alla porta; una porta con un materiale World Aligned si apre e "il materiale sembra scorrere sulla porta", "come se il materiale fosse animato". "Ci siamo? Ho rotto le palle? Datemi un feedback."

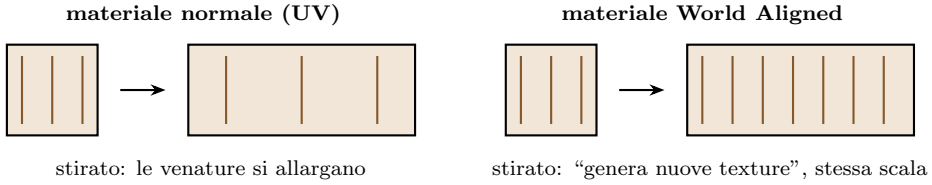


Figure 1.3.14: Materiale legato alle UV e materiale World Aligned a confronto. Il primo segue l'oggetto e si deforma con lui; il secondo non si deforma, ma non segue l'oggetto quando si muove. Va bene per pavimenti, muri e rocce statici; non per porte, veicoli o la torretta.

Le linee nere. Una studentessa vede delle **linee nere** al confine tra una ripetizione della texture e l'altra, sul pavimento. Davide guarda il suo grafo: stessa texture, Bark, ma ha collegato l'uscita **XY** invece di **XYZ**. "In quel modo lui aveva due assi che erano allineati col mondo, e l'altro no", e il risultato era diverso. "Ok, grazie."

Approfondimento: come funziona un World Aligned. Il nodo World Aligned Texture, una Material Function del motore, non usa le UV della mesh: per ogni pixel prende la sua **posizione nel mondo** e la usa come coordinata della texture. Lo fa tre volte, proiettando la texture lungo i tre assi (dall'alto, di lato, di fronte), e mescola le tre proiezioni secondo la direzione della superficie: è la tecnica detta **triplanar mapping**. L'uscita XY usa una sola proiezione (dall'alto, buona per i pavimenti), l'uscita XYZ le mescola tutte; per questo, su un oggetto con facce in tutte le direzioni, con XY le pareti laterali ricevono una texture "stirata" a strisce, che può dare proprio le linee scure viste dalla studentessa. Il **Texture Size** è espresso in **unità del mondo**, cioè in centimetri: è la grandezza di una ripetizione della texture. Funziona quindi al contrario del tiling: un tiling più alto fa ripetere di più la texture, un Texture Size più alto la fa ripetere di meno (una texture di 100 si ripete ogni metro, una di 10 ogni dieci centimetri). È la ragione delle esitazioni di Davide su quale valore fosse "troppo". Il costo: tre letture della texture invece di una, quindi un materiale un po' più pesante.

1.3.30 L'esercizio della lava, il nodo Noise e la chiusura

"Allora, ci siamo visti un po' di cose, fatto un miscuglione di cose per quanto riguarda materiali, texture e import delle mesh." Nessuna domanda: "sembra tutto a posto".

La consegna. Per esercitarsi con i materiali, e magari scoprire qualche nodo non usato a lezione (il nodo **Noise**, qualche **maschera**), Davide propone una prova di una decina di minuti, da continuare poi a casa come le cose di Igor: **un materiale di lava**. Di che componenti avrà bisogno? Deve **muoversi** un po', quindi bisognerà gestire la velocità del **Panner**, "a meno che non vi cercate proprio la texture della lava". Ma la richiesta è di improvvisare: "provate a vedere cosa riuscite a combinare andando a fare delle operazioni fra colori e fra altre texture", con un po' di movimento, un **Base Color**, "ma anche un **Emissive**, perché la lava comunque deve emettere luce".

Il ripasso dei nodi. Una studentessa non ricorda il nome della funzione "che abbiamo usato per collegare rosso e blu": il Lerp. Davide ricondivide lo schermo e fa il riepilogo dei nodi della giornata: la **costante**, il **colore**, la **texture**, il **nodo risultante**, il **Texture Coordinate**, i **World Aligned**, le **operazioni** (con il consiglio sulla moltiplicazione visto nella sezione 1.3.11), il **Panner**, il **Lerp**. Divisione e sottrazione servono soprattutto per gli effetti speciali, e Davide vorrebbe fare una lezione su **Niagara**, "il tool che usa Unreal per fare gli effetti speciali", se ci sarà tempo.

Le prossime lezioni con Davide. Davide annuncia come saranno le sue lezioni finali: ci sarà "una lezione che sarà un'esercitazione intera" in cui si proverà a fare dei **livelli**, e chiederà agli studenti di condividere lo schermo e di *streamare* a turno ("se per voi è un problema, magari lo facciamo solo con le persone per cui non è un problema", e lo dirà in anticipo). Per esempio, "un livello **platform**", con le interazioni che si faranno nelle lezioni di Igor e quello che si vedrà insieme per il *level design*.

Il nodo Noise. Un nodo "un po' più complicato", per chi vuole provare: il **Noise**, che "serve per simulare un *noise*": il *Voronoi*, il *White Noise*, un'alternanza di bianco e nero "che si usa molto spesso per fare delle distorsioni". Ha molti parametri: la scala del *noise*, e la possibilità di filtrarlo con dei vettori o con delle mappe. "Potete smanettarci un pochino."

Lo stream della studentessa. "C'è qualcuno che, nel mentre sta provando a fare qualcosa, vuole streamare in questi ultimi cinque minuti?" Una studentessa ha provato, ma non riesce a trasformare la texture in World Aligned: è perché usa un Texture Sample, e serviva un Texture Object. "Prova a non usare il World Aligned, e farlo così, diciamo, semplice." Senza World Aligned le viene "questa cosa", sempre con il Bark. Davide la guida:

- nella lava servono "delle zone più scure e delle zone più luminose": quindi un **Lerp** tra un **colore** e un **nero**, gestito da un'alfa che sia una texture con dei bianchi e dei neri "posizionati come vuoi", da mandare nell'**Emissive Color**;

- l'alfa va cercata tra le texture del progetto, "qualcosa per dare un effetto tipo di distorsione"; le **Normal**, invece, "lasciale perdere": per un'alfa si usano le Diffuse o le mappe come Specular e Roughness;
- se una texture fa da alfa, nelle entrate del Lerp ci vogliono **due colori**, "uno che sia molto scuro, uno che sia più chiaro"; ricordando che l'alfa "prende i bianchi e i neri e dice: nei bianchi vai più verso B, nei neri vai più verso A";
- il risultato va "forse più nell'Emissive che non nel Base Color", perché la lava la si vuole "infuocata, accesa, luccicante"; il Base Color si può anche non usare, o usarlo con una texture adatta ("il Bark, per come è fatto, forse non è proprio la texture più adatta per fare una lava");
- nell'Emissive si può **moltiplicare** il risultato, in modo che le zone accese siano molto luminose.

La studentessa nota che il nero, nell'Emissive, quasi non si vede: "esatto", il nero non emette niente. E usandolo come colore "non si vede molto", perché il Bark come alfa "è molto bianco", e finisce per scurire un po' il tutto. "È comunque una cosa complicata, quindi vedete effettivamente se riuscite a tirar fuori qualcosa."

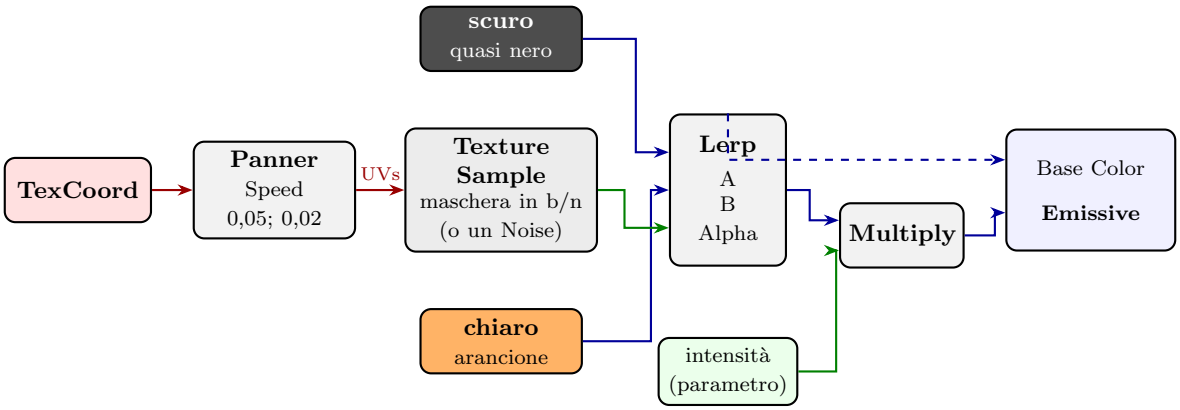


Figure 1.3.15: Una possibile soluzione dell'esercizio della lava, costruita con i soli nodi della lezione e con i consigli dati alla studentessa: una maschera in bianco e nero che scorre con il Panner fa da alfa a un Lerp tra uno scuro e un arancione; il risultato, moltiplicato, va nell'Emissive (e, tratteggiato, eventualmente anche nel Base Color). Non è un grafo mostrato a lezione: i valori sono indicativi.

La chiusura. "Vabbè, io in realtà adesso ho gettato questa bomba, e penso che vi lascerò fare. Prendetevi tutto il tempo che volete, nessuna pressione." Davide rivedrà la classe "a luglio, probabilmente", per le ultime quattro o cinque lezioni del corso, sul **level design** e sulla **creazione di livelli**. "Si sono fatte le 7." "Direi che è stato un piacere, mi sembrate tutti abbastanza interessati, sicuramente belli, belli, belli attivi." E forse ci si rivedrà prima, "se Igor ha bisogno di un'altra sostituzione". "Ciao a tutti, arrivederci."

Approfondimento: una lava più convincente. Partendo dallo schema della figura 1.3.15, tre accorgimenti, tutti con nodi già visti, rendono la lava molto più viva. Primo: **due Panner** con velocità e tiling diversi sulla stessa maschera, sommati o moltiplicati, così che il movimento non si ripeta mai uguale (lo stesso trucco dell'acqua). Secondo: un Multiply sull'Emissive con valori ben oltre 1 (10, 20), perché solo così il *bloom* fa "bruciare" le zone calde, come nel materiale con il Glow Intensity. Terzo: usare la **stessa** maschera per la **Roughness**, con un Lerp tra 0,3 (la lava liquida, lucida) e 1 (la crosta raffreddata, opaca): il materiale cambia così non solo colore ma anche superficie, che è il senso del PBR. Il nodo Noise di Davide può sostituire la texture della maschera, ma è calcolato a ogni pixel e costa più di una texture: per un oggetto grande in scena, una texture di *noise* è di solito la scelta più leggera.

Comando	**Che cosa fa**
Tasto destro nel grafo	menu dei nodi, con ricerca
T + clic	Texture Sample
1 + clic / 2 + clic / 3 + clic	Constant, Constant2Vector, Constant3Vector (il colore)
A / M / D + clic	Add, Multiply, Divide
S + clic	Scalar Parameter (per questo Subtract non ha scorciatoia)
U + clic	Texture Coordinate ("lavora sulle UV")
L + clic	Lerp (Linear Interpolate)
P + clic	Panner
Alt + clic su un pin	stacca i collegamenti
Tasto destro su un nodo, Convert to Parameter	trasforma il nodo in un parametro, regolabile dalle Material Instance
Tasto destro su un Texture Sample, Convert to Texture Object	per i nodi World Aligned
Use Selected Asset from Content Browser	mette nel nodo la texture selezionata nel Content Browser
Tasto destro su un materiale, Create Material Instance	crea un'istanza
Reimport Mesh With Dialog	reimporta una mesh riaprendo la finestra di import
Ctrl+G / Shift+G	raggruppa e separa gli actor selezionati
Actor, Merge Actors	fonde gli actor selezionati in una nuova mesh
Ctrl+C, Ctrl+V, Ctrl+Z, Ctrl+Y, Ctrl+A	le scorciatoie classiche, valide quasi ovunque
Ctrl+S	salvare: "salvate spesso"

Table 1.3.5: I comandi della terza lezione nell'editor dei materiali, nel Content Browser e nel Level Editor di Unreal Engine 5. Le scorciatoie con il clic si usano tenendo premuto il tasto e cliccando nel grafo; World Aligned Texture e World Aligned Normal non ne hanno.

Punti chiave del capitolo

- Il grafico lavora soprattutto fuori da Unreal (Blender, Maya); dentro Unreal **importa** gli oggetti e costruisce gli **shader**.
- Una **texture** è la "carta regalo" dell'oggetto: si avvolge sulla mesh attraverso le **UV**, che si creano prima, nel software di modellazione. Una texture non pensata per quelle UV si stira.
- Le mappe principali: **Diffuse/Albedo** (il colore, suffisso _D o _A), **Normal** (blu: l'illusione del rilievo senza poligoni in più), **Specular**, **Roughness**, **Metallic**, la mappa di **altezza** (*displacement* vero, "una normal sotto steroidi").
- Nelle mappe in bianco e nero il **bianco è 1**, il **nero è 0**, il grigio sta in mezzo. Ogni canale RGB è una mappa a sé: una texture **AORM**/ORM impacchetta tre mappe in un file.
- Il **materiale** è la "vernice", uno **shader**: si costruisce con nodi, come un Blueprint, e combina le texture. Sugli oggetti si applicano materiali, non texture.
- **Material Domain** (per che cosa), **Blend Mode** (Opaque, Masked, Translucent) e **Shading Model** (Default Lit, Unlit) cambiano i pin del nodo risultante.
- Nodi di base: **Texture Sample** (T), costanti (1, 2, 3), **Add**, **Multiply**, **Divide** (A, M, D), Scalar Parameter (S), **Texture Coordinate** (U), **Lerp** (L), **Panner** (P). La moltiplicazione è l'operazione più usata.
- Il rendering è **PBR**: Roughness 1 è la roccia, 0 lo specchio; Metallic e Specular decidono come la superficie riflette; l'**Emissive** moltiplicato fa brillare il materiale.
- **UTiling** e **VTiling** (o un Multiply sulle coordinate) decidono quante volte si ripete la texture: 2 e 2 sono quattro ripetizioni, 0,5 e 0,5 un quarto della texture.
- Un nodo diventa un **parametro** con Convert to Parameter. Le **Material Instance** (MI_) cambiano i parametri senza toccare il grafo del **master material** (M_), si aggiornano subito e sono più leggere: sugli oggetti si applicano sempre le istanze.
- Le **Material Function** sono "parentesi" di calcoli riusabili tra materiali e tra membri del team.
- Il **Lerp** interpola tra A e B secondo un'**alfa** da 0 a 1, che può essere una texture ("l'arbitro"); il **Panner** fa scorrere la texture (acqua, fluidi, lava).
- Importare **FBX**, controllare le **unità** (Unreal usa i centimetri), texture in PNG o JPEG. La finestra di import genera collisioni, corregge il transform, importa materiali e texture.
- Una mesh è fatta di **vertici**, **spigoli** e **facce** (triangoli). Il **pivot** deve stare dove il pezzo ruota: con **Reimport Mesh With Dialog** (cannone: Z −135, rotazione 113) o con la **Modeling Mode** di Unreal Engine 5.
- Le **simple collision** (box, sfera, capsula: il "capsulone" dei *character*) sono leggere; le **complex** usano i triangoli della mesh e costano molto di più.
- **Ctrl+G** raggruppa; **Merge Actors** fonde in una nuova mesh, con tanti slot quanti i materiali dei pezzi.
- Due piani nella stessa posizione sfarfallano (**z-fighting**): basta alzarne uno di 0,1. I materiali **World Aligned** (Texture Object, Texture Size, uscita XYZ) non si stirano, ma restano fermi se l'oggetto si muove.
- **Salvate spesso.** E per esercizio: un materiale di **lava**, con Panner, Lerp ed Emissive.

Chapter 1.4

Vettori, rotazioni e la torretta: il mulino che gira e la torretta che ti segue

Questa è la **quarta lezione** del corso breve di **introduzione a Unreal Engine**, e segna il ritorno del docente titolare dopo la lezione di Davide sulla grafica. Se la terza lezione guardava a ciò che "fa il grafico all'interno di Unreal" (texture, materiali, l'import della torretta), questa torna alla **logica**, cioè ai **Blueprint**, e lo fa con un obiettivo preciso, mostrato in apertura: un **mulino** le cui pale girano, e la **torretta** importata con Davide che "adesso ci segue con lo sguardo". Per arrivarci serve un po' di matematica dello spazio 3D: i **vettori**, le loro operazioni, i **rotator**.

La lezione ha due metà separate da una pausa di venti minuti, presa alle 17:28. La prima metà parte dalla **teoria**: che cos'è un **vettore** ("una freccia con una direzione, una magnitudine e un verso"), le quattro operazioni (**addizione**, **sottrazione**, **moltiplicazione** e **prodotto scalare**, il *dot product*), i **vettori normalizzati**, i **rotator** e il problema del **gimbal lock**. Poi si passa all'engine con un mulino: prima il modo più semplice di far ruotare un oggetto, il **Rotating Movement Component**, poi un Blueprint vero, **BP_Windmill**, in cui una pala gira grazie all'**Event Tick**, ai *delta seconds* e ai nodi **Make Rotator**, **Combine Rotators** e **Set Relative Rotation**; la seconda pala gira "gratis" grazie alla **gerarchia** dei componenti. Infine il mulino diventa "magico" e si gira verso il giocatore con **Find Look at Rotation** e **RInterp To**, con un **Clamp** per limitarne la rotazione. La seconda metà costruisce la **torretta**: un Blueprint in tre pezzi (**Base**, **Pivot**, **Cannon**), una **Sphere Collision** come raggio d'azione, l'evento **On Component Begin Overlap**, i controlli **Is Valid** e **Actor Has Tag**, una variabile per ricordare il bersaglio, un **Custom Event** e un **timer** che, a ogni *frame*, fa ruotare il pivot verso il player.

La lezione è **online**, su Zoom, come le precedenti. Il docente alterna spiegazione e piccole esercitazioni "botta e risposta", da cinque minuti l'una, e passa gran parte della seconda metà a risolvere i problemi degli studenti in condivisione dello schermo. Finisce verso le 19, "andiamo a cena tutti", con l'appuntamento a **martedì**.

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "a Real", "Arreal", "Arry"; l'**Event Tick** è "Leventic", "l'eventic", "eventHIC" e "l'event kick"; i **vettori** sono una volta "vittori"; i *rotator* sono "rotatempi" e "rotatori"; **Find Look at Rotation** è "findLookAtRotation", "find all the rotation" e "Find, look, rotation"; **RInterp To** è "Er inter tu", "R Inter2" e "r interp tu"; **Get Player Character** è "getLayerCharacter"; **Get Actor Location** è "Get. Hector. Location" e "get active location"; il **Rotating Movement** è a un certo punto "vetro in movement"; **Is Valid** è "Svalid" e "isvalid"; **Other Actor** è "OderActor" e "Oder Actor"; la **gun** della torretta è "Ganna" e "GAN"; il **Pivot** è spesso "pivo"; il *Third Person Character* è "ter person character" e "Peter per personacarte"; **Quixel Megascans** è "QuickSell" e "Quixel megaschanico". Dove la ricostruzione è sicura il capitolo usa la forma corretta e, la prima volta, lo segnala in nota; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione, in un punto della prima esercitazione e durante tutta la pausa, il riconoscimento vocale produce lunghi blocchi di testo senza senso

e ripetuto ("qual è la risposta che ci può essere?", "grazie a tutti per l'attenzione", discorsi sul "mercato del lavoro", sul "governo di Dalai Lama" e su "la migliore dal 2005"): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è una dimostrazione dal vivo nell'editor dei Blueprint, spesso costruita a voce ("tirate fuori", "collegate", "splittate"). Il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei pin di Unreal Engine 5.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono gli schemi spiegati a voce o alla lavagna ("perdonate le mie doti molto brutte di disegno"): le operazioni tra vettori, i casi del prodotto scalare, gli assi di rotazione, la gerarchia del mulino, i grafi della pala, del mulino che guarda il player e della torretta, i casi dell'*overlap*. Le tabelle riassumono le operazioni sui vettori, i nodi della lezione e gli errori più comuni incontrati dagli studenti. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'attesa: **Quixel Megascans** e il materiale della lava; il trucco del **CCD**; il programma della giornata (sezioni da 1.4.1 a 1.4.3);
2. la teoria: i **vettori** e le loro operazioni, il **prodotto scalare**, la **normalizzazione**, i **rotator** e il **gimbal lock** (sezioni da 1.4.4 a 1.4.6);
3. il **Rotating Movement Component** e l'esercizio dei tre cubi (sezione 1.4.7);
4. il Blueprint del **mulino**: i componenti, la pala che gira con l'Event Tick, la **gerarchia** (sezioni da 1.4.8 a 1.4.10);
5. il mulino che guarda il player: **Find Look at Rotation**, **RInterp To**, il **Clamp** e il debug (sezioni 1.4.11 e 1.4.12);
6. dopo la pausa, la **torretta**: i tre pezzi, la **Sphere Collision**, l'**overlap** (sezioni da 1.4.13 a 1.4.15);
7. **Is Valid**, i **tag**, la variabile del bersaglio e i **Custom Event** (sezioni da 1.4.16 a 1.4.18);
8. il **timer** e l'evento **Move Turret**: la torretta che segue il player (sezioni 1.4.19 e 1.4.20);
9. il *tuning*, i problemi degli studenti e la chiusura (sezioni 1.4.21 e 1.4.22).

1.4.1 L'attesa: Quixel Megascans e la lava

La registrazione parte mentre il docente fa entrare gli studenti nella stanza di Zoom ("ho aperto io la prima a far entrare un po' tutti") e chiacchiera con il primo arrivato. "Come va con Unreal? Hai sperimentato un po', l'hai toccato un pochetto?" Lo studente ci ha giocato con un collega, che gli ha insegnato le texture, ma non ha trovato online "alcuna fonte per prendere cose utili, file utili": per l'esercizio della lava assegnato da Davide nella lezione precedente, un materiale di lava animata, ha usato quello che c'era nella cartella del Drive.

Quixel Megascans. La risposta del docente è **Quixel Megascans**[1], di cui passa il link in chat. Quixel "ti permette di avere proprio dei materiali": "una marea di materiali di qualsiasi tipo", ma anche rocce, *mesh* e texture di alta qualità. Il docente fatica un po' a ritrovare la pagina giusta, "perché hanno cambiato il sito", e ricorda che serve accedere con l'account di Unreal (cioè di Epic Games). "Ormai fanno la cooperazione tra di loro": c'è un collegamento diretto tra Quixel e Unreal, "proprio tipo *face to face*". Cercando *lava* escono diverse risorse; una, *lava story* o qualcosa del genere, "sembra carina".

[1] La trascrizione dice "QuickSell", "Quixel megaschanico" e "Materials pixel mega scan". Il nome corretto è Quixel Megascans: la libreria di asset scansionati di Quixel, società acquisita da Epic Games, gratuita per l'uso con Unreal Engine.

Come sono fatte. Le texture di Megascans "sono proprio *scan*": c'è "gente che si mette là con queste macchine che costano una vita umana" a scansionare le rocce. "Quixel è un grande, grande calderone di risorse." Da lì si può partire anche per la lava senza trovare la texture perfetta: si prende una **roccia** e, "come avete visto con Davide, con la matematica", cioè con un **Multiply** e con il **Lerp** pilotato da un'**alfa**, si inserisce "il rosso della lava all'interno di queste rocce". Un'altra strada sono i video su YouTube: alcuni tutorial lasciano in descrizione le texture usate (altri le fanno pagare, "il maledetto"), e "ti interessano le texture, il materiale poi lo fai tu".

La lava degli studenti. Arrivano gli altri ("buongiorno, Filippo, Marco, Patrick"), e il docente chiede chi ha provato il materiale della lava "che vi è stato dato da Davide"[2]. Uno studente l'ha mandato sul Discord, e il docente lo commenta: "bello il movimento che hai fatto". L'hai messo su un piano, in engine, per vedere come reagiva? No. "Quello vedilo un po' tu", perché a volte il risultato in *viewport* cambia a seconda di come è costruito il materiale ("che so, hai messo il Multiply sbagliato"): il consiglio è di provarlo sempre in *viewport*, su un oggetto vero. E "sicuramente una texture differente potrebbe essere più carina": torna utile Megascans.

L'appello. Il docente aspetta "2–3 minuti" che entrino gli altri. Uno studente avvisa che anche oggi dovrà staccarsi venti minuti prima per andare a lavorare: "assolutamente nessun problema".

1.4.2 Le domande e il trucco del CCD

Prima di cominciare, il docente apre lo spazio delle domande: "se qualcuno di voi ha avuto qualche particolare cosa di cui vuole parlare, domandare già prima che magari iniziamo, così facciamo il Q&A già da subito". Nessuno: "mi sembrate tutti molto tranquilli".

La pallina che entra nel piano. Allora è il docente a recuperare "un piccolo trucchetto che mi sono dimenticato di dirvi" per il **gioco della pallina** della seconda lezione, quello della piattaforma inclinabile: a volte la pallina, con la fisica attiva, **penetra** nel piano, "come abbiamo visto". Nei dettagli del componente, nella sezione della fisica (vicino a **Simulate Physics**), c'è un'opzione da spuntare: il **CCD**, *Continuous Collision Detection*[3]. "Fondamentalmente, è un *tick* che, se lo mettiamo su questo particolare *asset*, lui in modo continuo cerca di vedere le collisioni": le rende "più precise", ed evita che la pallina attraversi il terreno. "Se fate questo, vi risolve tranquillamente il problema." Chi vuole, può sistemare il gioco per la prossima volta.

Approfondimento: perché la pallina attraversa il piano. La fisica di un gioco non è continua: il motore fisico controlla le collisioni a passi discreti, una volta per *frame* (o per *substep*). Se in un passo la pallina si sposta più dello spessore del piano, può trovarsi "prima" sopra e "dopo" già dentro o sotto, senza che nessun controllo l'abbia mai vista toccare la superficie: è il fenomeno del *tunneling*, tanto più probabile quanto più l'oggetto è piccolo e veloce e il piano sottile. Il CCD (in Unreal la casella *Use CCD* nella sezione *Collision* del componente, nelle impostazioni avanzate) fa sì che il motore fisico consideri anche il percorso tra le due posizioni, la "scia" dell'oggetto, e non solo le posizioni finali. Costa un po' di più, ed è per questo che non è attivo di default: lo si accende solo sugli oggetti che ne hanno bisogno, come proiettili e palline veloci. Un'alternativa complementare è dare al piano uno spessore maggiore.

1.4.3 Il programma: il mulino e la torretta

"Oggi vi dico già da subito cosa faremo." Gli studenti hanno scaricato il **materiale didattico**: dentro il file estratto c'è il progetto *UE5 Intro Start*[4], con il suo file di progetto. Lo si apre, "così almeno gli *shader* si cominciano a buildare", si fa "una piccola spiegazione teorica" e poi "partiamo direttamente con la pratica".

[2] La trascrizione dice "da Dario", ma subito dopo e più avanti il nome è Davide, il docente della terza lezione.
[3] La trascrizione dice "Trucco Collision Collision Detection".
[4] La trascrizione dice "Hive UE5 intro start", la stessa parola iniziale incerta già incontrata nella terza lezione.

I due obiettivi. Il docente sposta il *Player Start* e mostra il livello. A sinistra c'è "questo bellissimo **mulino**", fatto "facendo ruotare le pale". Poi c'è la **torretta** che gli studenti hanno costruito la volta prima con Davide ("avete fatto il *merge*, giusto?"), che "adesso ci segue con lo sguardo". O meglio, dovrebbe: nella dimostrazione non lo fa. "Abbiamo scoperto un piccolo bug. Poi adesso lo aggiustiamo. E giustamente i bug ogni volta appaiono praticamente subito." Il programma della giornata è questo: le pale del mulino, e la torretta che segue il giocatore.

1.4.4 I vettori

"Avete sentito mai parlare di vettori? Matematica, i grafici cartesiani?" Sì.

Che cos'è un vettore. "In generale, un vettore può essere semplificato, in uno spazio, come letteralmente una **freccia**" che ha tre caratteristiche: una **direzione**, una **magnitudine** ("la sua ampiezza", cioè la lunghezza) e un **verso**. In Unreal i vettori si sono già visti senza nominarli: nella **location** e nella **rotation** di ogni oggetto, "in particolare molto di più sulla location". La location è fatta di tre coordinate, **X**, **Y** e **Z**, e "queste particolari coordinate possono formare tra di loro, nello spazio 3D, un vettore". Il docente parte dalla parola "velocità di movimento" e si corregge: "più che una velocità, è più una direzione", con la sua ampiezza e il suo verso. Con i vettori si possono "fare tante cose in Unreal", per esempio muovere degli oggetti; le pale del mulino girano in un verso, e anche se in quel caso si parla di rotazione, e quindi di *rotator*, "sono sempre formate da delle coordinate".

Un punto non è ancora un vettore. Più avanti, alla lavagna, il docente aggiunge una precisazione importante: "ricordatevi che quello che vediamo è un **punto** nello spazio, non è una freccia, quindi non è già un vettore". Un punto, per esempio $(5, 4, 1)$, è solo una posizione; il vettore è la freccia che collega due punti, per esempio dall'origine $(0, 0, 0)$ a $(5, 4, 1)$[5].

Le operazioni. Con i vettori "possiamo fare varie operazioni matematiche, che sono praticamente quelle classiche": **addizione**, **sottrazione**, **moltiplicazione** e **prodotto scalare** (*dot product*), "che è un'altra cosa, che poi spiegheremo". La slide le mostra con due vettori, A e B (figura 1.4.1).

- **Addizione.** "Se andiamo a prendere due vettori e li addizioniamo, avremo un vettore unico." Il vettore A più il vettore B "dà come risultato il vettore C, che è $(5, 4)$": si mette la coda di B sulla punta di A, e C va dall'inizio di A alla fine di B. "Come fare l'ipotenusa, se vi ricordate i triangoli."
- **Sottrazione.** Dà "la **differenza** nello spazio tra due punti". Il risultato dipende dall'ordine: A − B è il vettore che va da B ad A, B − A quello che va da A a B. La sua lunghezza è la distanza tra i due punti.
- **Moltiplicazione.** "La useremo tantissimo": si moltiplica tutto ciò che fa parte del vettore, X, Y e Z, e quindi il vettore "viene allungato nella sua direzione".
- **Prodotto scalare.** "Statemi un attimo a sentire: è l'unica operazione che ci dà un valore **scalare**", cioè "non ci ridà un vettore indietro, ma un valore in *float*". ("Chi si ricorda che è il float?" "I numeri decimali." "Perfetto.") Lo si vede nella prossima sezione.

Approfondimento: i vettori nel libro di Romero e Sewell. Il quattordicesimo capitolo di *Blueprints Visual Scripting for Unreal Engine 5*, dedicato ai nodi matematici e alle *trace*, presenta i vettori con un esempio molto vicino a quello della lezione. In Unreal la struttura **Vector** contiene tre *float*, X, Y e Z, e può rappresentare sia un **punto** nello spazio (una *location*) sia uno **spostamento** o una velocità. Un personaggio sta in $(50, 0, 20)$, un divano in $(450, 0, 20)$, e un'unità di Unreal vale un centimetro. Per arrivare al divano il

[5] A lezione il docente dice che "un vettore è formato dalla somma di due punti". La relazione corretta è la *differenza*: il vettore che va da A a B è B − A (si veda la sottrazione, poco più sotto). Unreal, peraltro, usa la stessa struttura *Vector* sia per i punti sia per gli spostamenti.

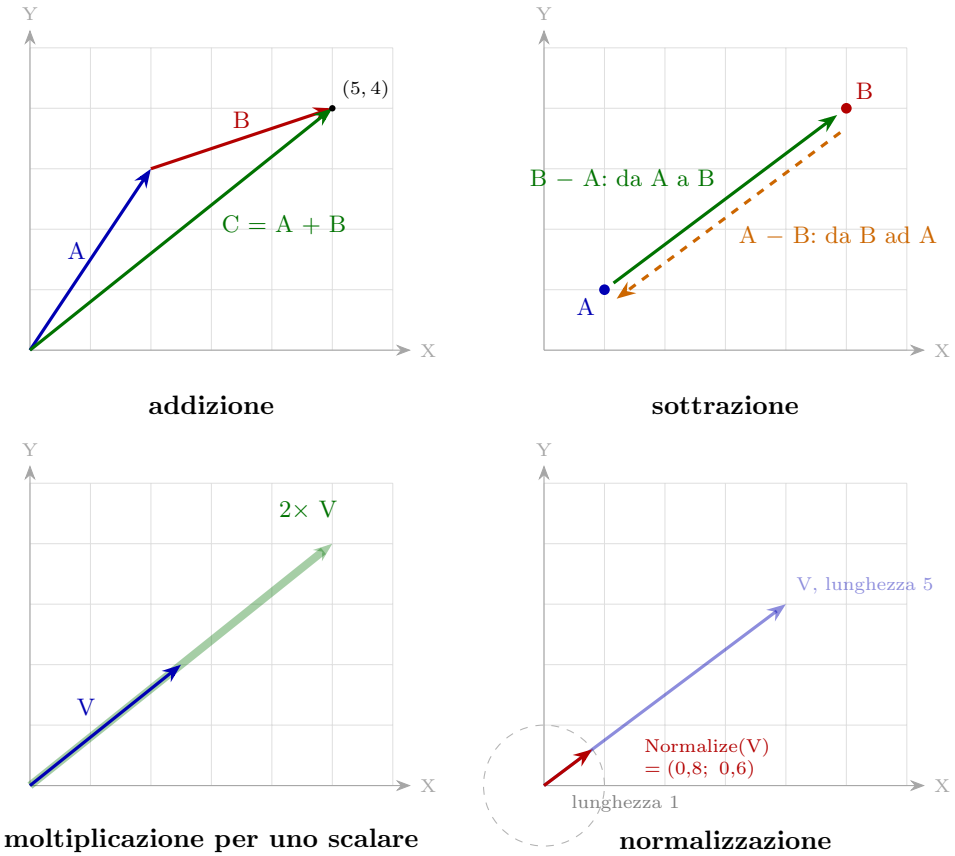


Figure 1.4.1: Le operazioni sui vettori viste a lezione, nel solo piano XY per semplicità. In alto a sinistra la somma della slide: A seguito da B porta al punto $(5, 4)$. In alto a destra la sottrazione, il cui verso dipende dall'ordine dei fattori. In basso, la moltiplicazione per un numero, che allunga il vettore senza cambiarne la direzione, e la normalizzazione, che lo riporta a lunghezza 1 (sezione 1.4.5). I valori di A e B della slide non si leggono nella trascrizione e sono scelti per l'esempio; il risultato $(5, 4)$ è quello detto a lezione.

personaggio deve spostarsi di $(400, 0, 0)$: 400 centimetri lungo X. Le due regole che ne derivano sono quelle della lezione, scritte in formula:

$$\text{destinazione} = \text{partenza} + \text{spostamento}, \qquad \text{spostamento} = \text{destinazione} - \text{partenza}.$$

Ogni operazione si fa componente per componente: $(3, 5, 0)+(5, 2, 0) = (8, 7, 0)$ e $(6, 8, 0)-(1, 4, 0) = (5, 4, 0)$. Il libro aggiunge due esempi pratici: se V1 è la posizione nel mondo di un personaggio e V2 la posizione *relativa* della sua arma, V1 + V2 è la posizione nel mondo dell'arma; e la **lunghezza** della differenza tra due posizioni (il nodo *Vector Length*) è la loro distanza. Moltiplicare per un numero negativo inverte il verso: moltiplicando per -1 il vettore "in avanti" di un attore si ottiene quello "all'indietro", ed è lo stesso trucco usato nel quinto capitolo per far cambiare direzione a un bersaglio mobile. Il nodo **Multiply**, avvertono gli autori, è un nodo *wildcard*: se gli si collega un vettore diventa un *Vector* × *Vector*, e per moltiplicare un vettore per un *float* bisogna fare tasto destro sul secondo pin e scegliere *Convert Pin...* e poi *Float*.

Operazione	**In formula**	**Risultato**	**A che cosa serve in un gioco**
Addizione	$(a_x+b_x,\ a_y+b_y,\ a_z+b_z)$	vettore	spostare un punto: posizione + spostamento = nuova posizione
Sottrazione	$(b_x-a_x,\ b_y-a_y,\ b_z-a_z)$	vettore	la direzione e la distanza da A a B ("dove sta il player rispetto a me?")
Moltiplicazione per uno scalare	$(k\,a_x,\ k\,a_y,\ k\,a_z)$	vettore	allungare o accorciare; con -1 invertire il verso; velocità × *delta seconds*
Lunghezza (*Vector Length*)	$\sqrt{a_x^2 + a_y^2 + a_z^2}$	*float*	la magnitudine; della differenza tra due punti, la distanza
Normalizzazione (*Normalize*)	$\vec{a}\,/\,\lvert\vec{a}\rvert$	vettore lungo 1	tenere solo la direzione, da moltiplicare poi per una velocità
Prodotto scalare (*Dot Product*)	$a_xb_x + a_yb_y + a_zb_z$	*float*	capire se due direzioni sono concordi, perpendicolari od opposte

Table 1.4.1: Le operazioni sui vettori della lezione, con il nome del nodo Blueprint corrispondente dove serve e un uso tipico in un gioco.

1.4.5 Il prodotto scalare e i vettori normalizzati

Il **prodotto scalare** è l'operazione che restituisce un *float* invece di un vettore. "Che cosa ci dà?" Ci dice come sono orientati due vettori l'uno rispetto all'altro. Con i vettori A e B della slide (figura 1.4.2):

- se il prodotto scalare è **maggiore di 0**, i due vettori sono "direzionati nella stessa direzione", grosso modo concordi; se è **1**, "sono proprio uguali", cioè puntano esattamente nello stesso verso;
- se è **uguale a 0**, sono "perfettamente **perpendicolari**", formano un angolo retto;
- se è **minore di 0**, formano un angolo ottuso, "al verso opposto": per due vettori come quelli della slide "magari $-0{,}4$, più o meno"; "se proprio sono così", uno contro l'altro, è -1.

Questi valori "puliti", tra -1 e 1, valgono per i **vettori normalizzati**, ed è il motivo per cui la slide successiva li introduce.

I vettori normalizzati. "Qualcuno di voi sa cos'è un vettore normalizzato?" Uno studente: "un vettore tra 0 e 1". "Esatto": normalizzando un vettore, "indipendentemente dalla sua lunghezza, lo ridurremo a 1". Un vettore "che è lungo 5", normalizzato, "lo portiamo a 1"[6]. La direzione resta la stessa, cambia solo la lunghezza.

[6]Più precisamente: un vettore normalizzato ha lunghezza *esattamente* 1; sono le sue componenti, prese una a una, a stare tra -1 e 1. Il vettore $(4, 3, 0)$, lungo 5, normalizzato diventa (0,8; 0,6; 0), come nella figura 1.4.1.

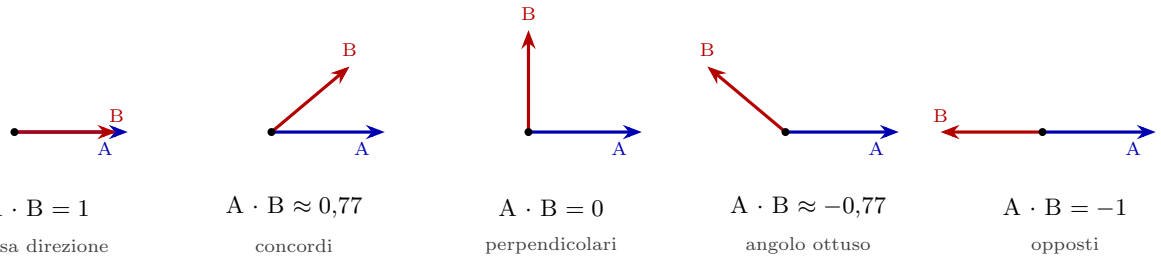


Figure 1.4.2: Il prodotto scalare tra due vettori normalizzati A e B, al variare dell'angolo tra di loro: 1 se puntano nello stesso verso, positivo se sono concordi, 0 se sono perpendicolari, negativo se formano un angolo ottuso, -1 se sono opposti. I valori intermedi sono il coseno dell'angolo ($\cos 40° \approx 0{,}77$).

Approfondimento: il prodotto scalare è un coseno. Per due vettori qualsiasi vale

$$\vec{a} \cdot \vec{b} \;=\; a_x b_x + a_y b_y + a_z b_z \;=\; |\vec{a}|\,|\vec{b}|\cos\theta,$$

dove θ è l'angolo tra i due. Se i vettori sono normalizzati, le lunghezze valgono 1 e il prodotto scalare *è* il coseno dell'angolo: per questo varia tra -1 e 1, come dicono Romero e Sewell nel quattordicesimo capitolo ("il prodotto scalare di due vettori normalizzati è uguale al coseno dell'angolo che formano"), e per questo i casi della slide sono 1, 0 e -1. Con vettori non normalizzati il *segno* resta affidabile (positivo, zero, negativo) ma il valore non è più confrontabile. Nei giochi il prodotto scalare risponde a domande come "il player è davanti o dietro al nemico?": si normalizza la differenza tra la posizione del player e quella del nemico e la si confronta con il *Forward Vector* del nemico, che Unreal restituisce già normalizzato (nodi *Get Actor Forward Vector*, *Right Vector*, *Up Vector*). Un risultato maggiore di $\cos 45° \approx 0{,}71$, per esempio, significa "il player è dentro un cono di visione di 90 gradi". Il quinto capitolo del libro usa invece la normalizzazione per il movimento: si normalizza una direzione, la si moltiplica per una velocità e per il *delta time* (con il nodo **Get World Delta Seconds**, che tornerà nella torretta) e la si somma alla posizione a ogni *tick* con *Add Actor World Offset*. "Normalizzare", scrivono, "è una procedura comune che assicura che il vettore abbia lunghezza 1, il che lo rende compatibile con il resto dei calcoli".

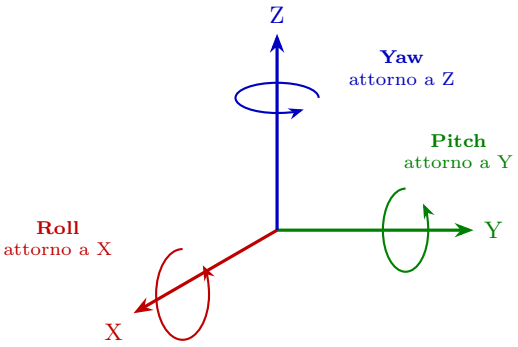
1.4.6 I rotator, il gimbal lock e i quaternioni

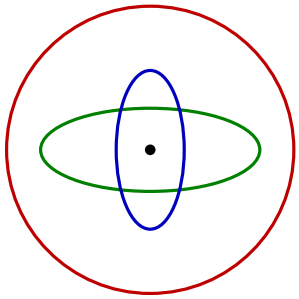
"Andiamo avanti: poi abbiamo i **rotator**." Per spiegarli il docente usa l'immagine del **giroscopio**: tre "anelli che girano tra di loro e formano proprio quella che è la rotazione". Ogni anello corrisponde a un asse, con i colori di sempre: la X in rosso, la Y in verde, la Z in blu (figura 1.4.3).

Il gimbal lock. Con le "regole classiche di Eulero", cioè descrivendo la rotazione come tre angoli applicati uno dopo l'altro, c'è "un possibile problema": il **gimbal lock**. "Non vi sto a spiegare troppo, perché fondamentalmente andremo a parlare di quattro dimensioni invece di tre, ed è tutta una cosa molto, molto complessa." In breve, "potrebbe succedere che due assi si blocchino tra di loro" e comincino a ruotare "sempre dipendenti l'uno dall'altro". Si risolve con i **quaternioni**, che usano quattro valori invece di tre, ma "parliamo di matematica molto complessa, e non andiamo molto avanti su questa cosa". Quello che conta, per ora: per descrivere una rotazione nello spazio servono sempre la X, la Y e la Z, "perché abbiamo tre giroscopi che girano di per sé". Nell'editor, su un qualsiasi *asset*, si vedono proprio questi tre valori.

L'obiettivo. "Cosa vogliamo fare oggi? Vogliamo creare qualcosa che giri": qualcosa che usi il *rotator* per far girare le pale del mulino.

Approfondimento: roll, pitch, yaw e il gimbal lock in pratica. Nella seconda edizione di Romero e Sewell, l'elenco dei tipi di variabile descrive il **Rotator** così: contiene i *float* X (**Roll**), Y (**Pitch**) e Z (**Yaw**), espressi in gradi. I tre nomi vengono dall'aviazione: il *roll* è il rollio (l'aereo che inclina le ali), il





il giroscopio: tre anelli annidati, ognuno ruota attorno al proprio asse

Figure 1.4.3: I tre assi di Unreal con i loro colori e i nomi delle tre rotazioni di un *rotator*: *roll* attorno a X, *pitch* attorno a Y, *yaw* attorno a Z. A destra, l'immagine del giroscopio usata a lezione. Schema ricostruito.

pitch il beccheggio (il muso che sale e scende), lo *yaw* l'imbardata (il muso che gira a destra e a sinistra). Una torretta che insegue un bersaglio usa lo *yaw* per girarsi e il *pitch* per alzare la canna; le pale del mulino, come si vedrà, girano in *pitch*. Il **gimbal lock** succede quando l'anello di mezzo ruota di 90 gradi: il primo e il terzo anello si ritrovano allineati, e ruotarli produce lo stesso movimento, così un grado di libertà va perso. Con gli angoli di Unreal il caso tipico è il *pitch* a ± 90: con il muso dritto verso l'alto, *yaw* e *roll* fanno la stessa cosa. I **quaternioni** (in Unreal la struttura *Quat*) rappresentano invece ogni rotazione come una rotazione unica attorno a un asse qualsiasi, con quattro numeri, e non hanno questo problema. Unreal li usa internamente: un *rotator* è il formato comodo per le persone, il quaternione quello sicuro per i calcoli, e nodi come *Combine Rotators* convertono i *rotator* in quaternioni, li combinano e li riconvertono. Un effetto collaterale, che conviene conoscere: dato che il *pitch* di un *rotator* viene sempre riportato tra -90 e 90 gradi, una pala che ruota in *pitch* oltre i 90 gradi viene descritta "capovolta" (*yaw* e *roll* a 180), e chi stampasse il suo *rotator* vedrebbe numeri che saltano. La rotazione sullo schermo, però, resta corretta.

1.4.7 Il Rotating Movement Component

"Allora, avete aperto l'editor di Unreal? Sì? Perfetto." Prima di costruire il mulino, il docente mostra "quello più semplice" tra i "vari modi per far ruotare un actor in generale"; un modo che però, avverte, "è difficile usarlo su vari componenti".

Un componente aggiunto dai Details. Si trascina in scena un **cubo** (con il tasto **Fine**, *End*, l'oggetto "va diretto sul terreno"[7]). Ora, "qui sui **Details** sulla destra, potete aggiungere dei componenti anche non avendo una schermata di Blueprint": con il pulsante **Add** dei Details si può mettere un componente direttamente sull'actor in scena. Si scrive *rotating movement* e si sceglie il **Rotating Movement Component**. "Trovato? Sì? Perfetto."

I parametri. Il componente ha una serie di dati, tra cui il più importante è il **Rotation Rate**, con i tre assi X, Y e Z: la velocità di rotazione su ciascun asse. "Di default il Rotating Movement, come potete vedere, è messo a **180**" sulla Z: mezzo giro al secondo attorno all'asse verticale.

La regola della mobilità. "Intanto vi dico, ragazzi: come abbiamo visto, se un *asset* si deve muovere all'interno di uno spazio, la sua mobilità dovrà essere **Movable**. Ricordatevelo sempre." Se lo si lascia **Static** e si va in Play, "prima di tutto non si muove", e poi l'engine restituisce un errore, che il docente legge: *Static Mesh Component has to be movable if you'd like to move it with Rotating Movement Component.* Con la mobilità corretta, in Play il cubo gira su sé stesso: "abbiamo fatto le munizioni di *Metal Gear Solid*", scherza

[7] La trascrizione dice "il tasto AND": è il tasto *End* (Fine), che in Unreal appoggia l'oggetto selezionato sulla superficie sottostante.

il docente, pensando agli oggetti da raccogliere che ruotano a mezz'aria. "Questo è un modo per far ruotare": Unreal "ci dà la possibilità di poter ruotare direttamente un nostro bellissimo actor tramite un componente che loro hanno già creato". "Un oggetto ruotante che ruota. Perché 'ruotante' è brutto, sembra."

Esercitazione: i tre cubi. "Piccola esercitazione velocissima, faccio queste botte e risposte con le esercitazioni." Si copia il cubo ("tanto vi ricordate come si copiano") e si fa in modo che **un cubo ruoti sull'asse X** e **un altro sull'asse Y**, alla velocità che si preferisce. "Ricordatevi di metterli Movable. Vi do cinque minuti." La correzione: sul secondo cubo si azzera la Z e si mette 180 sulla X; sul terzo, 180 sulla Y. Si controlla che siano tutti e tre Movable, si va in Play, "ed ecco le nostre tre rotazioni". "Possiamo farlo con qualsiasi *asset*."

Approfondimento: la sedia che gira. Il Rotating Movement Component è il primo componente che Romero e Sewell fanno usare nel libro, già nel primo capitolo: un Blueprint *BP_RotatingChair* con una *Static Mesh* (la sedia dello *Starter Content*) e un Rotating Movement. "Di default", scrivono, "questo componente ruoterà il Blueprint attorno all'asse Z, quindi non serve cambiarne le proprietà": si compila, si trascina la sedia nel livello e la si guarda girare in Play. Il componente fa parte della famiglia dei *movement component*, componenti che non hanno una forma propria ma muovono un altro componente, quello indicato come *Updated Component*, che di default è la **radice** dell'actor. È qui la ragione del limite che il docente scoprirà tra poco con il mulino: il Rotating Movement fa girare la radice, e con essa tutto ciò che vi è attaccato. Il componente ha anche un'opzione *Rotation in Local Space* e un *Pivot Translation*, che sposta il centro di rotazione; ma per far girare una parte sola di un actor la strada più chiara resta quella che segue, il Blueprint.

1.4.8 Il Blueprint del mulino

"Proviamo a fare intanto il nostro *windmill*." Il mulino del livello di esempio è formato da più cose, si vede nei Details. "Chi si ricorda come si chiamano queste cose qua, che sono tutte insieme in un singolo *asset*?" "Componenti." "Sono i **componenti**": come si è visto nella seconda lezione, un actor "può essere, scusate la ridondanza, composto da più componenti". In questo caso un **body**, il corpo, "la nostra *mesh* principale, il cubone", e **due pale**.

La classe. Si crea un nuovo **Blueprint Class** di tipo **Actor**, "perché non sarà disponibile al giocatore, ma sarà solamente là come sfondo": non va controllato, non è un *Pawn*. Il docente lo chiama **BP_Windmill**[8].

Il corpo. Nella **Viewport** del Blueprint si aggiunge una **Static Mesh** (Add, in alto a sinistra, *Static Mesh*) e la si chiama **Body**. Nei dettagli, alla voce *Static Mesh*, si sceglie un cubo, per esempio *SM_Cube*. Il primo cubo scelto dal docente "ha un *pivot* particolarmente strano"; ne sceglie un altro, "molto meglio, perché c'ha il *pivot* al centro". Lo solleva di circa 40 sulla Z e gli dà un materiale: il docente mette la *prototype grid*, "voi mettete quello che volete". Poi lo scala per farne il corpo del mulino: "3 metri per 5" di altezza, "di larghezza facciamo 2 metri", cioè una scala di circa 3, 2 e 5 sui tre assi[9].

La viewport che non si trova. Uno studente non ricorda come si apre la viewport del Blueprint. "O ce l'hai qua, o ce l'hai qua, oppure dovresti avere **Window** e dovresti vedere **Viewport**." La voce è già spuntata, ma la finestra non c'è: forse è "nascosta da qualche schermata di Windows" ("prova a fare Alt+Tab"). Si passa alla condivisione dello schermo, e si scopre che il problema è un altro: lo studente ha creato il Blueprint ma non lo ritrova. La soluzione è il **filtro** del Content Browser: le "tre stanghettine" vicino alla ricerca, categoria *Blueprint*, ed ecco il file; doppio clic e, se si apre la versione ridotta, *Open Full Blueprint Editor*. Poi Add, ma nell'editor in italiano la voce non si trova subito (è "Componente Mesh Statica"). "Effettivamente, ragazzi, vi consiglio di mettervi l'editor in **inglese**, perché è molto più semplice.

[8] La trascrizione dice "Windmill, io in questo caso lo chiamerò 1": probabilmente *BP_Windmill1*, per non confonderlo con il mulino già presente nel progetto.

[9] La trascrizione dice "x32 sulla y5 sull'altezza" e "3:25": la lettura più probabile è X 3, Y 2, Z 5. Il cubo di base di Unreal misura un metro di lato, quindi una scala di 3 corrisponde a 3 metri.

Anzi, ve lo consiglio proprio": i nomi dei nodi e dei componenti nei tutorial, nei libri e nella documentazione sono quasi sempre in inglese.

Le pale. "Adesso dobbiamo creare le nostre palette." Si copia e incolla il corpo (Ctrl+C, Ctrl+V): le copie finiscono dentro il corpo, "quindi cliccatele e tiratele fuori, perché sono uguali". Si scalano a forma di pala, una orizzontale e una verticale, si dà loro un materiale diverso (il docente usa *M_Grid*, "questa qua violacea") e le si rinomina **Pala1** e **Pala2**.

Attaccare le pale al corpo. "Importante, ragazzi: queste pale devono essere attaccate al corpo. Come facciamo? Abbiamo visto l'ereditarietà." È la **gerarchia** dei componenti: "il fatto che io posso mettere degli *asset* attaccati l'uno con l'altro". Si prende una pala e la si **trascina sopra** Body, nell'elenco dei componenti: ora è sua figlia. "Scusami, non ho capito come si fa." "Se hai questa pala, la prendi e la trascini sopra Body."

Il Rotating Movement non basta. "Adesso dobbiamo dare una rotazione a queste pale. Potremmo provare con il Rotating Movement." Il docente lo aggiunge al Blueprint, mette il mulino in scena ("molto più grosso") e va in Play. "Guardate che succede": il Rotating Movement è impostato sulla Z, e **ruota tutto il corpo**. "Una particolarità del Rotating Movement è che non può essere attaccato a componenti: o ruota tutto, o non ruota niente." Quindi bisogna programmare il movimento delle pale. Il Rotating Movement va tolto: "se l'avete messo e avete adesso il vostro mulino che ruota su sé stesso, lo dovete eliminare".

1.4.9 Far girare una pala: Event Tick e Combine Rotators

"Domanda. Questo movimento deve essere fatto per ogni *frame*. Quindi deve essere un evento che riprende ogni *frame*. Chi si ricorda qual è l'evento che lo fa? L'abbiamo usato per la piattaforma dell'altra volta." "L'**Event Tick**." "Ottimo." L'Event Tick, "che sarà il nostro protagonista", è già presente nell'Event Graph di un nuovo Blueprint.

I delta seconds, di nuovo. "Chi si ricorda che sono i **Delta Seconds**?" Uno studente: "determina quanti tick al minuto...". Un altro: "i secondi effettivi, e non quelli basati su *frame*". Il docente riprende: è "lo spazio che intercorre tra un *frame* e l'altro", la differenza di tempo tra due *frame*, "in questo caso si parla di millisecondi". Si moltiplica un valore per i *delta seconds* perché la durata di un *frame* cambia (a 30 *frame* al secondo è molto più lunga che a 120), e così gli spostamenti "non diventano più lenti se abbiamo meno *frame*, o più veloci se ne abbiamo di più". (Il docente prova a disegnarlo sullo schermo, ma lo strumento "a quanto pare non mi funziona benissimo".)

La velocità. Si tira fuori un cavo da Delta Seconds e si crea un **Multiply**: o scrivendo *multiply*, o digitando semplicemente l'**asterisco**, "e vi dà subito il Multiply". Il secondo valore è la **velocità**: "più è alto, più è veloce". Per ora, **90**.

Make Rotator. "Adesso vi dico una cosa." Il nodo **Make Rotator** costruisce un *rotator*, "quello che muove a livello di rotazione un *asset*", a partire da tre *float*: X, Y e Z. Quale asse serve? "Prendiamo la nostra bellissima pala e guardiamo subito in che asse la vogliamo ruotare. La X non mi sembra, la Z neanche: è la nostra carissima **Y**", cioè il **pitch**. Si collega il risultato del Multiply al pin Y (Pitch): il *rotator* creato ha X e Z a 0, "perché non si devono muovere", e la Y uguale a velocità per *delta seconds*.

Combine Rotators. "Possiamo farlo in tanti modi, ma oggi ne facciamo tipo tre." Il primo è **Combine Rotators**, che "combina due *rotator* per darne uno in uscita". Il primo dei due è l'**origine**: "a livello concettuale, qual è l'origine che dobbiamo prendere?" "Le coordinate del componente." "Sì: quale componente?" Per ora una sola pala, la Pala1. Siccome serve un componente e non l'intero actor, si trascina la pala dall'elenco dei componenti nel grafo: si ottiene una **referenza** a quel componente, "un *get*" che "si riferisce a quel componente preciso".

Relativa o del mondo? Da quella referenza si chiede la rotazione. Ma quale? "Abbiamo visto la differenza tra **rotazione relativa** e **rotazione del mondo**." La pala è attaccata al corpo, ma deve muoversi da sola, indipendentemente dal corpo. "Relativa al corpo, secondo me", propone uno studente. "Relativa a sé stessa, quindi la sua asse singola. Se prendiamo quella del mondo, girerà in modo strano." Si usa dunque **Get Relative Rotation**, la si collega al primo ingresso di Combine Rotators, e il *rotator* di Make Rotator al secondo.

Set Relative Rotation. "Chi si ricorda qual è il blocco che serve per settare la rotazione? Tanto è un *set*." È **Set Relative Rotation**, "perché, ovviamente, abbiamo preso la relativa". Lo si crea tirando fuori un cavo da Pala1 (così il *target* è già la pala), lo si collega all'Event Tick e gli si dà in ingresso la nuova rotazione, l'uscita di Combine Rotators (figura 1.4.4). "E mi raccomando, deve essere **Movable**." Si mette il mulino in scena, si va in Play: "ed ecco che una pala ruota".

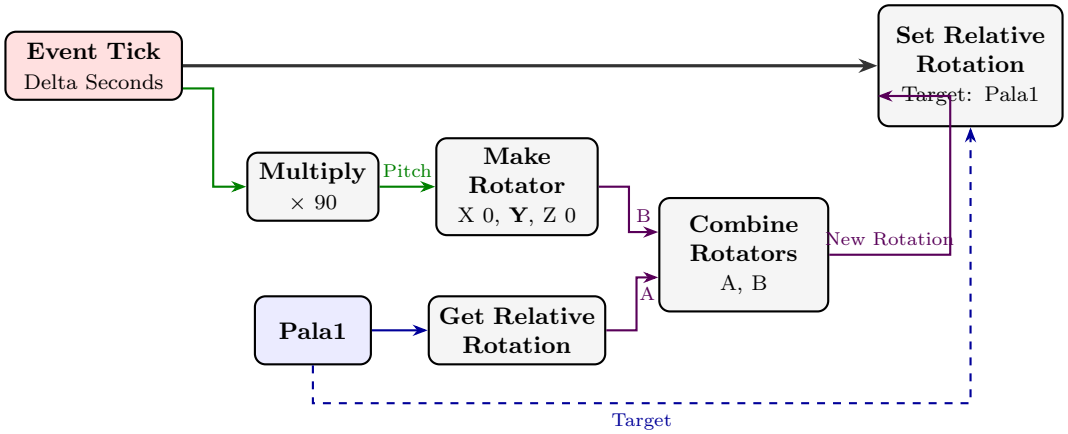


Figure 1.4.4: Il grafo della prima pala. A ogni *frame*, l'Event Tick esegue Set Relative Rotation su Pala1; la nuova rotazione è la rotazione relativa attuale della pala combinata con un piccolo incremento in *pitch*, pari a 90 gradi al secondo moltiplicati per i *delta seconds*. Il cavo nero è l'esecuzione, gli altri sono dati (verde i *float*, viola i *rotator*, blu le referenze). Schema ricostruito.

I problemi degli studenti. "Avete la vostra paletta che ruota?" Non tutti. Una studentessa chiede di rivedere il codice e non trova il blocco Get Relative Rotation: "tira fuori da Pala1 e scrivi *get relative rotation*". Un altro studente vede la pala "staccarsi da tutto l'asset" quando preme Play, e in condivisione si scoprono due errori. Il primo: ha calcolato la rotazione con Combine Rotators ma non l'ha collegata al Set ("hai calcolato la rotazione, ma non gliel'hai assegnata"). Il secondo: anche dopo, la pala gira attorno a un punto sbagliato, perché la *mesh* che ha scelto ha il **pivot** spostato. "Vedi che il pivot sta quasi lì? Cambia la *mesh*": con un cubo che ha il pivot al centro, la pala gira correttamente. (Nel frattempo il docente accenna anche a un **Set World Rotation**, per mostrare la differenza con la rotazione relativa.)

Approfondimento: relativo e mondo nel libro. Il quattordicesimo capitolo di Romero e Sewell spiega la differenza con un esempio molto simile al mulino. Un Blueprint ha come radice un *DefaultSceneRoot*; sotto c'è un componente *Table*, un tavolo, e sotto il tavolo un componente *Statue*, una statua. Il *transform* di Table è **relativo** a DefaultSceneRoot, e quello di Statue è relativo a Table: "se muovete il componente Table nella Viewport, anche il componente Statue si muoverà; ma se cambiate il *transform* relativo di Statue, Table resterà dov'è". Lo stesso vale per il mulino: la rotazione relativa di Pala1 è misurata rispetto a Body. Se il mulino viene ruotato nel livello, la rotazione relativa della pala non cambia, e il grafo continua a farla girare attorno al proprio asse; usando la rotazione del mondo, invece, l'incremento in *pitch* verrebbe applicato attorno all'asse Y del *mondo*, che coincide con quello della pala solo se il mulino non è ruotato: è il "girerà in modo strano" del docente. Nei vettori lo si vede ancora più chiaramente: il punto iniziale di ogni posizione nel mondo è sempre $(0, 0, 0)$, mentre quello di una posizione relativa è il punto in cui sta il genitore.

Approfondimento: una scorciatoia, Add Local Rotation. Il trio Get Relative Rotation, Combine Rotators e Set Relative Rotation è istruttivo perché mostra i tre passaggi di ogni animazione fatta a mano: *leggi* lo stato attuale, *calcola* il nuovo, *scrivi* il risultato. Unreal offre anche nodi che fanno tutto in uno: **Add Local Rotation** (o *Add Relative Rotation*) su un componente aggiunge un *Delta Rotation* alla rotazione attuale, così come *Add Actor World Rotation*, usato nella seconda lezione per la piattaforma, lo fa per un intero actor. Collegando l'Event Tick a un Add Local Rotation su Pala1, con un Delta Rotation fatto con Make Rotator come sopra, si ottiene lo stesso mulino con un nodo in meno. Il risultato è lo stesso; cambia la chiarezza di ciò che succede, ed è per questo che a lezione conviene vedere prima la versione lunga.

1.4.10 La gerarchia: la seconda pala e il cubetto

"Possiamo fare una cosa bella: invece di dover fare la copia esatta per quanto riguarda la nostra prima pala, possiamo vedere quello che la **gerarchia** ci permette di fare in modo molto facile." La Pala2 deve seguire in ogni caso il movimento della Pala1. Invece di programmare anche lei, la si mette "in eredità" ("non quella con Carlo Conti, ragazzi") alla Pala1: la si trascina sopra Pala1, e diventa sua **figlia**. Play: "non abbiamo programmato niente", ma la Pala2 segue la rotazione della Pala1. "È la Pala1 che gira, non la Pala2; la Pala2 segue solamente."

Esercitazione: il cubetto sulla pala. "Se noi volessimo attaccare un cubettino su un estremo della pala... anzi, sapete che c'è? Lo fate voi." La consegna: aggiungere una **Static Mesh** qualsiasi ("un cubo, una *mesh*, quello che vi pare"), piccola, su un'estremità della pala, e attaccarla alla **Pala2**, in modo che segua anche lei il movimento. "Vi do cinque minuti classici. Ricordatevi l'ereditarietà: per spostarlo dovete prenderlo proprio e metterlo sopra."

La domanda sul pivot. Uno studente l'ha fatto, ma è sorpreso: "mi aspettavo di dover spostare il pivot al centro della rotazione. Invece, automaticamente, è come se si agganciasse alla pala." "Esatto. Perché comunque **mantiene la sua posizione relativa**": il cubetto ha il suo pivot al proprio centro, ma la sua posizione è espressa rispetto al genitore, e quando il genitore ruota, il figlio ruota con lui attorno al pivot del genitore. "Se lo dovessimo spostare, andrebbe insieme: manterrebbe la sua origine, il fatto che sta sulla parte destra, sopra, giù, quello che è, ma si muoverebbe relativo all'altro." Play: "eccolo che gira insieme". "Così abbiamo capito come si ereditano piano piano gli oggetti tra di loro" (figura 1.4.5).

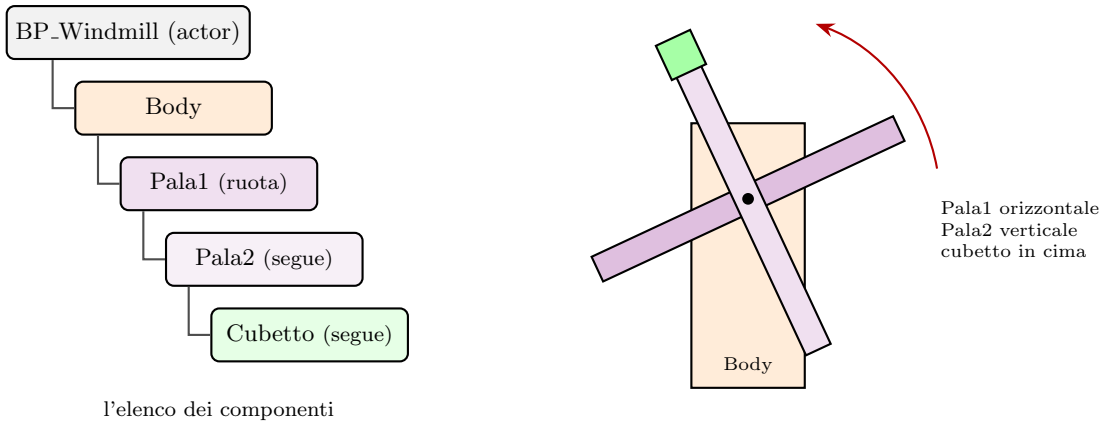


Figure 1.4.5: La gerarchia del mulino. Solo Pala1 è programmata per ruotare; Pala2, sua figlia, e il cubetto, figlio di Pala2, ne seguono la rotazione senza alcun nodo, mantenendo la loro posizione relativa. Schema ricostruito (nel mulino della lezione le pale sono di fronte al corpo).

1.4.11 Il mulino che guarda il player: Find Look at Rotation

"Adesso vi farò vedere un altro metodino" per creare una rotazione, sempre sull'Event Tick, "così vi faccio vedere anche un altro blocchettino interessante, perché sono tanti". Nei Blueprint, come nella programmazione in generale, "non esiste un qualcosa che sia fisso": "essendo che è un mondo di spazi, punti, vettori, ce lo possiamo giocare come ci pare". Il docente ricorda una persona che una volta ha fatto "un materiale con la matematica senza usare texture. Non so come abbia fatto, ma si può fare".

Staccare i cavi. Si stacca l'Event Tick dal grafo precedente. "Ricordatevi: per staccare i cavettini, tenete premuto **Alt** e cliccate sui puntini."

Find Look at Rotation. "Un blocco chiamato **Find Look at Rotation**. Interessante, molto interessante. Questo è un sacco utile, ve lo dico." Il docente legge la descrizione del nodo: *find a rotation for an object at Start location to point at Target location.* Alla lavagna ("scrittura incredibile, mi dispiace") disegna due punti nello spazio: lo **Start**, "da dove parto", e il **Target**, il punto da raggiungere. Il nodo restituisce "la rotazione verso quel punto": la rotazione che un oggetto posto in Start deve avere per guardare Target.

Un falso inizio. Il docente parte con un esempio sbagliato (lo Start e un Target ottenuto sommando 360, poi 359, alla posizione), e si ferma: "scusate, ragazzi, perdonatemi: il Find Look at Rotation lo facciamo per un esercizio che era diverso. Ho sbagliato io, perdonate, perché non ho gli appunti." L'esercizio vero: far ruotare il mulino **in direzione del character**, "un mulino molto magico" che si gira verso il giocatore.

Il grafo. I due punti sono:

- lo **Start**: la posizione del mulino, con **Get Actor Location** (senza *target*, cioè il Blueprint stesso);
- il **Target**: la posizione del giocatore, con **Get Player Character** e, da questo, Get Actor Location.

Il Find Look at Rotation "ci darà il vettore verso la posizione del nostro player", o meglio la rotazione che lo punta.

RInterp To: l'interpolazione. "Possiamo usare anche un altro blocco, che ci darà un'**interpolazione**." Si chiama **RInterp To** (R come *rotator*). "Qualcuno sa cosa vuol dire interpolazione? No." "È letteralmente il passaggio tra due punti": da un valore A a un valore B. I nodi di questo tipo (*Lerp*, *VLerp*, *Interp*) sono ovunque: "avete presente le animazioni? Un'animazione è un passaggio da un punto A a un punto B", e di base è fatta a livello di valori, perché ruotare un braccio è cambiare i valori di mano, braccio, gamba. Il nodo ha quattro ingressi:

- **Current**, la rotazione attuale del mulino (Get Actor Rotation, o la rotazione relativa del corpo);
- **Target**, la rotazione appena trovata da Find Look at Rotation;
- **Delta Time**, che "ci dice in quanto tempo" si fa l'interpolazione;
- **Interp Speed**, "la velocità in cui lo fa".

Solo lo yaw. "Però una cosa importante, ragazzi: non vogliamo che si muova tutto sulla stessa asse." Il mulino deve girare "al massimo così", in orizzontale, non inclinarsi verso l'alto o verso il basso. "Quale asse dovrò influenzare?" "La Z?" "Esatto, perfetto." Si fa **Split Struct Pin** (tasto destro sul pin) sull'uscita di Find Look at Rotation e sull'ingresso Target, e si collega solo la Z, lo *yaw*. Il risultato dell'interpolazione si applica con un nodo di rotazione (nella trascrizione, *Set Relative Rotation*)[10], collegato all'Event Tick (figura 1.4.6).

[10] La trascrizione dice "set relative rotation di pala 1", ma a girare verso il player è tutto il mulino. Con ogni probabilità il nodo è un Set Relative Rotation sul corpo (Body), a cui sono attaccate le pale, o un Set Actor Rotation sull'intero Blueprint: il risultato visibile è lo stesso.

L'offset di 90 gradi. In Play il mulino si gira verso il player, ma di lato. "Possiamo mettere più 90": si somma **90** alla Z prima di collegarla, "in modo tale che sia... ecco". Ora il mulino guarda il giocatore. "Quello che sta facendo è continuare a calcolare, visto che lo fa sull'Event Tick, il vettore che indirizza in un punto, che in questo caso è il player, in modo continuo." "Questo è un blocco che serve moltissimo, ragazzi": serve ogni volta che bisogna fare "delle robe a *target*", e servirà per la torretta. "Vedete che segue continuamente la nostra torretta? Questo è fatto con il Find Look at Rotation."

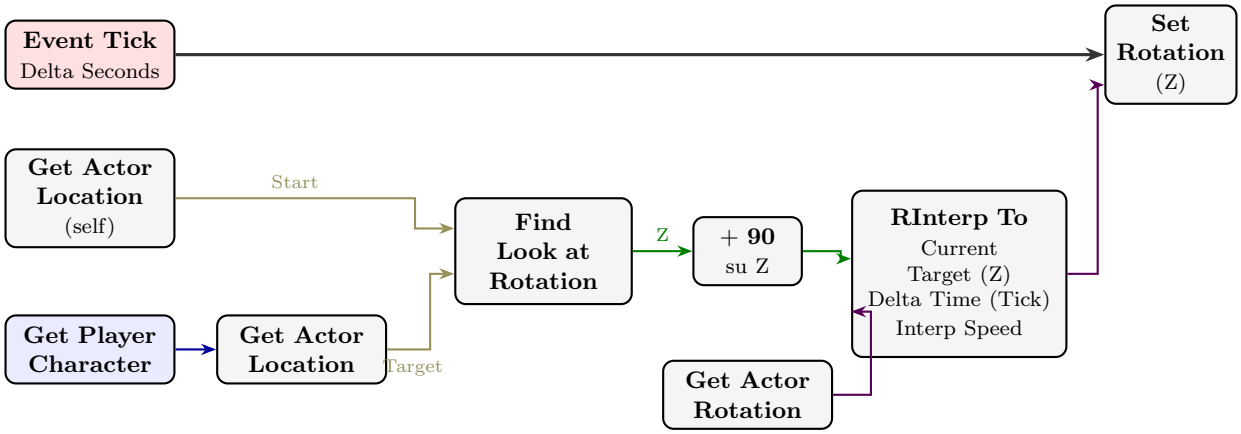


Figure 1.4.6: Il mulino che guarda il player. A ogni *frame*, Find Look at Rotation calcola la rotazione che dal mulino (Start) punta al giocatore (Target); se ne tiene solo lo *yaw* (Z), gli si sommano 90 gradi per compensare l'orientamento della *mesh*, e RInterp To fa avvicinare gradualmente la rotazione attuale a quella cercata. Il Delta Time può venire dall'Event Tick, come qui (un cavo non disegnato per chiarezza), o essere un numero fisso (a lezione, 1); si veda l'approfondimento. Schema ricostruito.

"A me non va." Seguono tre studenti in condivisione.

- Al primo il mulino resta fermo: "ti manca il Delta Time e l'Interp Speed". Li riempie ("metti uno e due"), compila ("non ho compilato, vabbè") e aggiunge l'offset: "tira fuori dalla Z, fai più, metti 90". "Perfetto, adesso ti segue."
- Al secondo manca anche a lui il Delta Time: "metti uno".
- Il terzo, collegando il nodo a Find Look at Rotation, si ritrova un pin diviso che non voleva: ha preso **Get Actor Rotation** invece di **Get Actor Location**. Poi non riesce a collegare l'uscita alla Z: "lo devi splittare: tasto destro sul pinnetto, quello violaceo, quello lilla, Split Struct Pin". E infine l'offset: da Return Value di Find Look at Rotation, la Z, un "più", 90, e al Target.

Approfondimento: che cosa calcola Find Look at Rotation. Il nodo fa una cosa semplice: sottrae la posizione di partenza da quella di arrivo, ottenendo il vettore direzione $\vec{d} = T - S$ (la sottrazione della sezione 1.4.4), e trasforma quella direzione in un *rotator* con *roll* nullo. Per la sola rotazione orizzontale, quella che interessa al mulino e alla torretta, il conto è

$$\text{yaw} \;=\; \operatorname{atan2}(d_y,\; d_x) \quad \text{(in gradi)},$$

cioè l'angolo, misurato nel piano XY a partire dall'asse X, sotto cui il bersaglio è visto. Il risultato presuppone che la "faccia" dell'oggetto guardi lungo il suo asse **X** locale, che in Unreal è la direzione "in avanti" (il *Forward Vector*). Se la *mesh* è stata modellata guardando lungo la Y, come le pale del mulino della lezione, la rotazione calcolata è sfasata di 90 gradi, e il "più 90" del docente non fa che correggere questo sfasamento. Lo stesso problema, come si vedrà, si ripresenta con la torretta.

Approfondimento: come funziona RInterp To. A ogni chiamata, RInterp To calcola quanto manca per arrivare al Target e ne percorre una frazione, pari (semplificando) a

$$\alpha \;=\; \min\big(1,\ \text{Delta Time} \times \text{Interp Speed}\big), \qquad \text{nuova rotazione} = \text{Current} + \alpha \cdot (\text{Target} - \text{Current}),$$

con la differenza presa sempre "dal lato più corto" (da 350 a 10 gradi si passa per 0, non per 180). Ne seguono tre cose utili. Primo: il movimento è veloce quando il bersaglio è lontano e rallenta avvicinandosi, perché a ogni *frame* si percorre una frazione di una distanza sempre più piccola; è l'effetto "morbido" che si vede in Play. Secondo: il nodo è pensato per ricevere come **Delta Time** proprio i *delta seconds* (dall'Event Tick o da *Get World Delta Seconds*) e come **Interp Speed** un numero intorno a 1–10: così la velocità non dipende dal *frame rate*. Con un Delta Time fisso, come l'1 messo a lezione, la frazione percorsa a ogni *frame* è sempre la stessa, e il mulino gira più veloce su un PC che fa più *frame*. Terzo: se il prodotto supera 1, la frazione è 1 e la rotazione **scatta** subito al Target; e se l'Interp Speed è 0 o negativo, Unreal restituisce direttamente il Target. Con Delta Time 1 e Interp Speed 2 il prodotto è 2: il mulino, in pratica, "snappa". Il fenomeno tornerà con la torretta (sezione 1.4.21).

1.4.12 Il Clamp, il debug con Print String e la pausa

"Abbiamo fatto il nostro bellissimo Find Look at Rotation. Potremmo anche **clamparlo**." "Si ricorda il Clamp qualcuno di voi?" "È quello per dare un valore minimo", e massimo. "Esatto, bravissimo." Lo scopo: limitare la rotazione del mulino, in modo che non possa girarsi del tutto.

Il grafo. Si splitta il pin con Split Struct Pin, si inserisce un **Clamp (float)**, "perché è un *float*", sulla Z, e la si ricompone con un **Make Rotator** prima di collegarla. Il primo tentativo è un intervallo stretto, da -25 a 25 gradi, ma qualcosa non torna: i valori in gioco sono molto più grandi ("mi dice: Pala1 è un botto di valori").

Il debug con Print String. "Questo come ho fatto, ragazzi? Questo è un modo per **debuggare** le cose." Non sapendo di quanto ruotasse il mulino e quali limiti dargli, il docente collega un **Print String** al valore e lo stampa a schermo mentre si muove attorno al mulino: "abbiamo stampato letteralmente il valore". I numeri che compaiono vanno più o meno da -160 a 160. Il docente prova quei limiti, poi li allarga: "snappa un pochino. Facciamo duecento massimo. Molto meglio. Non dovrebbe andare oltre"[11].

Approfondimento: Clamp e gli angoli. Il Clamp restituisce il valore stesso se è compreso tra Min e Max, altrimenti il limite più vicino: $\text{clamp}(v, m, M) = \min(\max(v, m), M)$. Con gli angoli c'è una sottigliezza che spiega i valori "strani" della lezione: lo *yaw* restituito da Find Look at Rotation sta tra -180 e 180, e passando "dietro" al mulino salta da 180 a -180; sommandoci 90 gradi di offset, l'intervallo si sposta (da -90 a 270). Un limite come $-25 \ldots 25$, applicato a questi numeri, non corrisponde a "25 gradi a destra e a sinistra della direzione di riposo", ma a una fetta del cerchio che dipende da dove sta il player e da come è orientato il mulino nel livello. Il modo pulito è ragionare sulla differenza tra l'angolo voluto e quello di riposo, riportarla tra -180 e 180 (Unreal ha il nodo *Normalize Axis*, o *Delta (Rotator)* per due *rotator*), e solo allora fare il Clamp. Il metodo del docente, stampare i numeri prima di decidere, resta comunque la prima cosa da fare.

La pausa. "Ci siete? C'è qualche cosa che non avete capito? C'è qualche blocco che vi risulta ancora un pochino nebuloso?" Tutto chiaro. "Sono le 17 e 28. Facciamo pausa, così poi riprendiamo e cominciamo a fare la nostra bellissima torrettina. Ci vediamo tra 20 minuti."

[11] La trascrizione di questo passaggio è molto confusa ("facciamo zero, no, mettere meno duecento... facciamo duecento massimo"), e non permette di ricostruire con certezza i valori finali del Clamp. Quello che conta è il metodo: stampare i valori, osservarli, scegliere i limiti.

1.4.13 Dopo la pausa: la torretta in tre pezzi

"Eccoci qua. Allora, ragazzi, adesso andremo a fare una bellissima torrettina." Il docente mostra la versione finita. Attorno alla torretta c'è una **sfera** disegnata con linee rosse: è il suo raggio d'azione. Quando il giocatore ci entra, la torretta "fa lo stesso processo che abbiamo visto", cioè ruota verso il *character*, e "in questo caso spara dei proiettili". Non ci sono effetti speciali ("non abbiamo Niagara"[12]), ma si costruisce "la logica che ci servirà con la torretta che avete *mergiato*". Si vedono anche delle linee che partono dal cannone: sono un **line trace**, "che permette a noi di avere una corrispondenza dipendente da cosa incontra". Oggi si fa la rotazione; proiettili e *line trace* saranno la prossima volta.

Il Blueprint. "La prima cosa che ovviamente dobbiamo fare è creare un nuovo Blueprint, sempre **Actor**", chiamato **BP_Turret**. Ma come hanno composto la torretta gli studenti, con Davide? "Abbiamo preso i tre pezzi forniti e li abbiamo uniti" con **Merge Actors**: una *mesh* unica. "Allora facciamo una cosa: la facciamo a **tre pezzi**." Per far girare il supporto e il cannone indipendentemente dalla base servono tre componenti distinti, quindi si torna ai tre pezzi importati con Davide: tre **Static Mesh** nel Blueprint, una per la **base**, una per il supporto ("l'*holder*, chiamiamolo", poi rinominato **Pivot**) e una per il **cannone** ("la *gun*", poi **Cannon**).

Chi segue chi. I tre pezzi vanno "uno dentro l'altro". Il docente parte con l'idea che il supporto segua il cannone, poi si corregge: "fate il contrario, che è meglio: fate che la *gun* segua l'*holder*". La gerarchia giusta è quindi: la **base** come radice, il **Pivot** figlio della base, il **Cannon** figlio del Pivot (figura 1.4.8). Così, quando il Pivot ruota orizzontalmente, il cannone gira con lui; e un giorno il cannone potrà alzarsi e abbassarsi da solo, senza trascinarsi dietro il supporto.

Un trucco: Browse to Asset. Per assegnare le *mesh* ai componenti non serve cercarle a mano nel Content Browser "con l'elenchino e tutto quanto". "Andate qua sull'*asset* che avete in scena, cliccate tasto destro e fate **Browse to Asset**": il Content Browser salta direttamente alla posizione del file, e da lì lo si può **trascinare** (*drag and drop*) nello slot *Static Mesh* del componente. In alternativa, come sempre, si clicca sullo slot e si scrive il nome: *turret base*, *turret pivot*, *cannon*. Poi si aggiustano le posizioni ("un buchettino sta un po' più giù") finché i pezzi combaciano.

Un problema di Add. Una studentessa, premendo Add, vede solo la *mesh* intera della torretta (*SM_Torretta*) e nessun'altra opzione. In condivisione si prova a compilare e a riaprire il Blueprint, e alla fine il componente si riesce ad aggiungere: "va benissimo così, abbiamo risolto. A volte Unreal può fare dei giochi simpatici."

1.4.14 Il raggio d'azione: la Sphere Collision

"Adesso dobbiamo creare quello che abbiamo visto qua: vedi questo cerchio rosso intorno alla torretta? Questo è una **Sphere Collision**." È il **range**: entrandoci dentro, la torretta si attiva.

Aggiungerla. In BP_Turret si aggiunge, tra i componenti, una **Sphere Collision**, "che tranquillamente può stare anche esterna alla base", cioè non deve essere figlia di un pezzo che ruota. Appena aggiunta non si vede quasi, "semplicemente perché è molto piccola". Cliccando su *Sphere*, nei dettagli c'è lo **Sphere Radius**: di default è 32; il docente lo porta a **800**. C'è anche la **Line Thickness**, lo spessore delle linee con cui la sfera è disegnata nell'editor, "che serve magari solo per vederla un po' più spessa".

Hidden in Game. In Play, però, la sfera non si vede. "Di base, questa è una **collisione**", una cosa interna agli *asset*, come le collisioni della *mesh* viste con Davide: esiste per la logica, non per essere guardata. Per vederla mentre si prova il gioco ("per avere facilità nel vedere quando entriamo e usciamo") si cerca nei

[12] La trascrizione dice "BevoFX": con ogni probabilità *Niagara VFX*, il sistema di effetti visivi di Unreal Engine, di cui Davide aveva parlato nella lezione precedente.

dettagli **Hidden in Game** e lo si toglie. "Parlando di *rendering*, Hidden in Game dice: so che logicamente esiste all'interno del mondo, ma se non è *hidden in game*, la renderizzo pure." Tolta la spunta, il valore è *false*, "quindi non è nascosto in gioco", e in Play ecco il range. È la stessa opzione usata nella seconda lezione, al contrario, per rendere invisibile il soffitto della vaschetta.

Ripetere i passaggi. Uno studente ha perso questa parte mentre cercava i pezzi della torretta, e chiede di rivedere come si mettono le *mesh* nei tre componenti. Il docente lo rifà: si clicca sul componente, si scrive il nome della *mesh* nello slot Static Mesh (*turret base*, *turret pivot*), si rinominano i componenti in modo che siano coerenti (**Pivot**, **Cannon**, "come a me è chiamata Cannon nella nostra *mesh*").

1.4.15 Gli eventi dei componenti e l'overlap

"Adesso andiamo a vedere un evento in particolare." Oltre agli eventi già visti, che appartengono all'intero actor (l'Event Begin Play, l'Event Actor Begin Overlap, l'Event Tick), "possiamo richiamare degli eventi anche all'interno dei **componenti**". Selezionando *Sphere* e scorrendo in fondo ai dettagli, c'è una sezione **Events** piena di pulsanti: *On Component Begin Overlap*, *On Component End Overlap*, *On Component Wake*, *On Component Sleep*, gli eventi del cursore e altri. "Tutta una serie di eventi" che si possono usare "per far sì che avvenga qualcosa in un preciso momento". Qui la torretta deve partire quando il player entra nella sfera: si clicca sul + di **On Component Begin Overlap**, e nel grafo compare l'evento, già legato alla Sphere.

Che cos'è un overlap. "Sapete cosa si intende per *overlap*?" Uno studente: "tipo appena si passa un confine, qualcosa del genere". "In un certo senso." Alla lavagna ("perdonate le mie doti molto brutte di disegno") il docente disegna la sfera con la torretta al centro: un *overlap* avviene quando il player, "o un qualsiasi actor", **tocca o entra** in quel volume. Un actor fuori non sta sovrapponendosi; uno dentro sì. E uno a cavallo del bordo? "Sta overlappando o non sta overlappando?" "Sì." "Qualcun altro ha un'opinione differente?" Sta sovrapponendosi, "perché anche un qualsiasi poligono da cui è formato è parte integrante di esso": basta un pezzo (figura 1.4.7). Ci sono quindi tre situazioni: non si sovrappone, **comincia** a sovrapporsi (Begin Overlap), **smette** di sovrapporsi (End Overlap, "quando ci esce"). Per ora serve solo la prima: "vogliamo che quando lui entra in questo volume la torretta faccia tipo: ti ho visto, mo' comincio a ruotare verso di te".

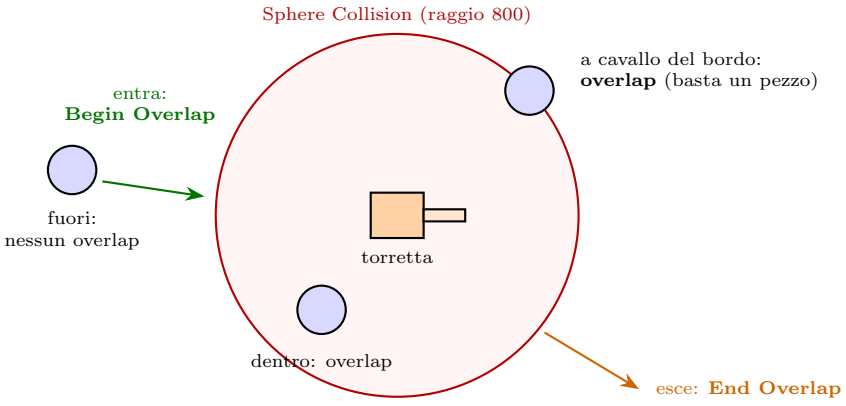


Figure 1.4.7: Le situazioni dell'*overlap* disegnate a lezione. Un actor si sovrappone alla sfera appena una parte qualsiasi della sua forma di collisione ne entra; Begin Overlap scatta una volta, nel momento in cui entra, End Overlap una volta, quando ne esce del tutto. Schema ricostruito.

Other Actor. "Come facciamo a capire quale actor ci dà indietro?" L'evento ha una serie di uscite: *Overlapped Component*, **Other Actor**, *Other Comp*, *Other Body Index*, *From Sweep*, *Sweep Result*. "Sono

tutti input che sono risultanti dall'overlap", "in parole povere" le informazioni su chi si è sovrapposto a che cosa. Per mostrarlo, il docente collega Other Actor a un **Print String** (Unreal inserisce da solo un *Get Display Name*, per trasformare l'actor in testo), va in Play ed entra nella sfera: in alto a sinistra compare *BP_ThirdPersonCharacter_0*[13]. "Mi ridà cosa sta overlappando, che in questo caso è il *character*." Esistono anche gli *overlapped component* e gli *other component*, ma quello che interessa è l'Other Actor, "perché il nostro *character* è un actor".

Approfondimento: perché l'overlap funziona "da solo". Perché Unreal generi un evento di *overlap* servono tre condizioni: entrambi i componenti devono avere attivo *Generate Overlap Events*; le loro risposte di collisione reciproche devono essere *Overlap* (e non *Ignore*); e almeno uno dei due deve muoversi. La Sphere Collision nasce con il profilo di collisione *OverlapAllDynamic*, che si sovrappone a tutto ciò che si muove, e la capsula del *character* è un oggetto dinamico di tipo *Pawn*: per questo, nella lezione, non c'è stato bisogno di toccare nulla. Se un giorno un evento di *overlap* non scatta, è la prima cosa da controllare, nei dettagli di entrambi i componenti, sezione *Collision*. La seconda edizione di Romero e Sewell elenca proprio gli eventi di collisione ("eseguiti quando due actor collidono o si sovrappongono") tra le famiglie principali di eventi, accanto a quelli di input e all'Event Begin Play.

1.4.16 Is Valid: esiste davvero?

"Vogliamo essere sicuri che il nostro actor in questione sia proprio il player. Perché un actor può essere anche un cubo, volendo." Prima, però, un controllo più semplice. Si tira fuori da Other Actor un **Is Valid**: "ce ne avete tanti, ma quello che ci interessa è quello col **punto interrogativo**", la *macro* con due uscite di esecuzione, **Is Valid** e **Is Not Valid**.

Che cosa controlla. "Is Valid, statemi a sentire, ci dà una risultante: è valido o non è valido?" Qui chiede: "hai overlappato qualcosa? È vero, esiste questa cosa?" In generale, Is Valid serve a dire se esiste o no qualcosa, "che può essere anche una **variabile**, quindi una variabile vuota".

L'esempio della variabile. Per farlo vedere il docente costruisce un piccolo esperimento:

1. da Other Actor crea una variabile con **Promote to Variable**;
2. mette un Is Valid su quella variabile, all'inizio dell'evento di *overlap*;
3. sul ramo **Is Not Valid** assegna alla variabile l'actor appena sovrapposto (il *Third Person Character*);
4. sul ramo **Is Valid** stampa con un Print String "valore registrato" e il nome contenuto nella variabile.

Prima entrata nella sfera: non succede nulla di visibile, "è normale". Il Blueprint ha chiesto: "questa variabile ha qualcosa dentro? No, non c'è niente. Quindi non è valida", e allora le ha assegnato l'actor. Si esce e si rientra: "ah, me l'hai fatto prima, quindi questa cosa c'ha qualcosa dentro", e il ramo è Is Valid: compare *valore registrato: BP_ThirdPersonCharacter_0*. "Qualcuno ha qualche dubbio riguardo all'Is Valid? Abbastanza chiaro?" L'esperimento si cancella: "ci serve solamente a capire cosa fa l'Is Valid".

Approfondimento: la regola di Romero e Sewell. Il quarto capitolo di *Blueprints Visual Scripting for Unreal Engine 5*, dedicato alla comunicazione tra Blueprint, mette in evidenza una nota: "usate sempre la macro Is Valid prima di eseguire una funzione usando una variabile che contiene una referenza a un oggetto. La variabile può non essere valida per diversi motivi, ed eseguire una funzione con una variabile non valida causerà un errore durante l'esecuzione". I motivi tipici sono due: la variabile non è mai stata riempita (come nella prima entrata dell'esempio), o l'oggetto a cui puntava è stato distrutto nel frattempo. In entrambi i casi un nodo che prova a leggere la posizione di "nessuno" produce nel registro il classico errore *Accessed None*. Il libro usa lo stesso schema in molti punti: un Is Valid prima di distruggere un actor, prima di applicare un danno a un bersaglio, prima di leggere un salvataggio (se non è valido, lo si crea). La torretta lo userà due volte, nella prossima sezione e nell'evento Move Turret.

[13] La trascrizione dice "BP ha perso un character 0". Il suffisso _0 è il numero che Unreal assegna a ogni istanza di una classe presente nel livello.

1.4.17 I tag e Actor Has Tag

Is Valid dice che *qualcosa* è entrato. Per sapere *chi*, si tira fuori da Other Actor un **Actor Has Tag**, "un blocco che va a esaminare una cosa chiamata **tag**". I tag "sono dei *label*, delle **etichette** che noi possiamo mettere all'interno di vari *asset* del gioco".

Mettere il tag al player. Si apre il **Third Person Character** e, "non dobbiamo fare niente di particolare", si scrive *tag* nella ricerca dei dettagli: compare **Actor** → **Tags**, un elenco vuoto con un **+**. Si aggiunge un elemento e lo si chiama **Player**. "Cosa abbiamo fatto? Abbiamo letteralmente etichettato il player con un tag Player": è come avere un cartellino di riconoscimento sulla schiena, con scritto sopra "tu sei il player".

Il controllo. Tornati nella torretta, Actor Has Tag chiede: il *target*, cioè l'actor sovrapposto (Other Actor), ha il tag **Player**? Il nodo restituisce un valore **booleano**: "sì, questo actor che ho overlappato ha il tag Player", e si va nel **True** di un **Branch**; "se no, invece andrà in False". Il docente lo mostra con due Print String, "Player trovato" sul ramo True e "Player non trovato" sul ramo False: entrando nella sfera, "Player trovato".

Attenzione all'ortografia. "Non lo fate, ragazzi, faccio solo vedere per esempio." Il docente modifica il tag, togliendo la R finale, e rientra: "Player non trovato". "Questo vi fa vedere anche che dovete essere precisi nel dire qual è il tag." Il consiglio: "fate Ctrl+C, Ctrl+V del tag che avete scritto", copiandolo dal *character* nel nodo invece di riscriverlo a mano.

Approfondimento: tag, Cast o confronto? Ci sono almeno tre modi per rispondere alla domanda "l'actor che è entrato è il player?". Il primo è il **tag**, quello della lezione: semplice, e indipendente dalla classe (lo stesso tag *Player* può stare su personaggi diversi, e un tag *Enemy* su nemici di classi diverse); esiste anche *Component Has Tag*, per etichettare singoli componenti. Il secondo è il **Cast To** (per esempio *Cast To BP_ThirdPersonCharacter*), che verifica la classe e, se riesce, restituisce l'actor già "convertito", con accesso alle sue variabili e funzioni. Il terzo è il confronto diretto tra oggetti: nel capitolo sull'intelligenza artificiale della seconda edizione di Romero e Sewell, un nemico che pattuglia usa un *Event Actor Begin Overlap* seguito da un Branch su un nodo **Equal (Object)**, che controlla se l'Other Actor è proprio il punto di pattuglia salvato in una variabile. Una curiosità sui tag: sono di tipo *Name*, e in Unreal il confronto tra *Name* **non distingue maiuscole e minuscole**, per cui *player* e *Player* risultano uguali; quello che conta, come ha mostrato il docente, è che le lettere siano le stesse. Copiare e incollare il tag resta comunque la via più sicura.

1.4.18 La variabile del bersaglio e i Custom Event

Ora che il player è stato trovato, bisogna ricordarselo: "andiamo a crearci una variabile in cui mettiamo il nostro player che è stato trovato", di tipo **Actor**. Il modo più veloce: tirare fuori un cavo da Other Actor e scegliere **Promote to Variable**. Si crea un nodo *Set* di una variabile nuova, che si rinomina **PlayerTarget**. Per rinominare, "o vai qua e fai **F2**, che è più facile, oppure fai tasto destro e Rinomina".

Un Custom Event. "Manca mezz'ora", e il docente si chiede se si riuscirà a finire ("questo lo continueremo la prossima volta"), ma va avanti. "Adesso qua dentro andremo a fare un **Custom Event**." Si scrive *custom event* nel grafo, si sceglie *Add Custom Event* e gli si dà un nome: **TrackTargetTimer**. "Come potete vedere, questo bellissimo evento è un Custom Event: è uguale agli altri", come il Begin Overlap, il Tick o il Begin Play, "ma è un evento che abbiamo creato noi", "che possiamo richiamare quando vogliamo, con delle regole specifiche". Gli eventi di Unreal sono "prefissati": l'*overlap* scatta perché l'editor di Unreal lo controlla già per conto suo. I Custom Event, invece, partono quando li si chiama.

Chiamarlo. Per mostrarlo, il docente collega al Custom Event un Print String che stampa "Hello" e lo chiama dal ramo del Begin Overlap, subito dopo il Set di PlayerTarget: si scrive il nome dell'evento (*TrackTargetTimer*) e compare il nodo che lo invoca. In Play, entrando nella sfera: "Hello". "Ho creato un evento customizzato che richiamo quando entro nella Sphere Collision. Compreso?" "Sappiate che di Custom Event ne potete creare a milioni": basta scrivere *custom event* e dare un nome.

Approfondimento: eventi, funzioni e macro. La seconda edizione di Romero e Sewell ricorda che nell'Event Graph ogni evento di Unreal può comparire una sola volta, mentre i **Custom Event** si possono creare a volontà; e offre una tabella per scegliere tra *macro*, *funzioni* ed *eventi*. In sintesi: tutti e tre accettano parametri in ingresso; solo macro e funzioni restituiscono valori; le funzioni e gli eventi possono essere chiamati da altri Blueprint, le macro no; e solo macro ed eventi possono contenere azioni *latenti*, cioè che aspettano (come un *Delay*), mentre le *Timeline* stanno solo negli eventi. Il capitolo sui vincoli di gioco della stessa edizione usa proprio un Custom Event, *Sprint Drain*, richiamato da un timer, per consumare la stamina del giocatore mentre corre: "un Custom Event permette di attivare i nodi collegati a lui ogni volta che un altro nodo lo chiama; così gruppi di nodi dello stesso Blueprint possono comunicare tra loro anche quando non sono collegati direttamente". È lo schema esatto della torretta.

1.4.19 Il timer: Set Timer by Event

"Cosa vogliamo fare con questo Custom Event?" Durante tutto il tempo, "come un Event Tick", la torretta deve cercare la posizione verso cui ruotare per seguire il player. "Lo potremmo fare tranquillamente nell'Event Tick, volendo, ma possiamo fare una cosa interessante: possiamo fare un **timer**. L'abbiamo chiamato timer apposta."

Il nodo. Dal Custom Event si tira fuori **Set Timer by Event**, un blocco "che ci permette di far partire un timer che ci chiede ogni quanto". I suoi parametri, così come li imposta il docente:

- **Time**: ogni quanti secondi scatta. "Vogliamo che lo faccia allo stesso tempo del Tick." Si potrebbe portare fin qui il cavo dei *delta seconds* dall'Event Tick ("ma è un po' brutto"); c'è invece un nodo apposta, **Get World Delta Seconds**, "un blocchetto semplicemente per comodità", "per non far sì di filare il Tick così, a cavolo";
- **Looping**: spuntato, "vogliamo che sia un timer che *loopi*";
- **Max Once Per Frame**: spuntato anche questo, "massimo una volta per ogni *frame*". "Ci permette almeno di dire: ok, per ogni *frame* si calcola, tanto effettivamente si muoverà per ogni *frame*. Questa è una cosa un po' più nel micro, però noi mettiamo *max*, in modo che sia più ottimizzato."

Quale evento? "Set Timer by Event: quale evento?" Sul nodo c'è un **quadratino rosso**, il pin **Event**, "che guarda caso è uguale a quello del Custom Event": un pin di tipo *delegato*. Vuol dire che lo si può tirare fuori e creare da lì un **altro Custom Event**, quello che il timer chiamerà a ogni giro: lo si chiama **MoveTurret**. È lì che si farà muovere la torretta.

Come ragiona. Per farlo vedere, a MoveTurret si collega il solito Print String, "Hello". La catena completa è: Begin Overlap, Is Valid, Branch su Actor Has Tag, Set PlayerTarget, TrackTargetTimer; e TrackTarget-Timer fa partire un timer, alla velocità dei *delta seconds*, in *loop*, al massimo una volta per *frame*, che chiama MoveTurret, che stampa "Hello". "Domanda: quando mi stamperà Hello?" "Da quando entri in range." "Ottimo, esatto." Appena il player entra, Hello a ogni *frame*. "Esco fuori: si cancellano... ah, no, scusate, non si stanno cancellando, è vero. Perché non stoppiamo mai il timer." Dalla prima entrata in poi, il timer continua all'infinito: "ciò vuol dire che il timer a un certo punto va disattivato; lo vedremo dopo".

Approfondimento: timer o Event Tick? Romero e Sewell presentano i timer, già nel secondo capitolo, come un'alternativa all'Event Tick: tutto ciò che è collegato al Tick viene eseguito a ogni *frame*, e ciò che non ha bisogno di aggiornarsi così spesso può girare a intervalli più lunghi. Nel quinto capitolo un Custom Event *ChangeDirection*, chiamato da un Set Timer by Event con Looping attivo, fa invertire periodicamente la direzione di un bersaglio mobile; nella seconda edizione un timer chiama ripetutamente *Sprint Drain*, e un *Clear Timer by Function Name* lo ferma quando il giocatore smette di correre. Nella torretta della lezione l'intervallo è pari ai *delta seconds*, quindi il timer scatta (circa) una volta per *frame*, come un Tick: il vantaggio non è la frequenza, ma il fatto che il timer **parte solo quando serve** (all'ingresso del player) e **si può fermare**. Tre dettagli utili. Primo: il valore di Get World Delta Seconds viene letto una sola volta, quando il timer viene impostato, e resta fisso anche se poi il *frame rate* cambia. Secondo: se l'intervallo di un timer è più breve di un *frame*, Unreal può eseguirlo più volte nello stesso *frame* per "recuperare"; **Max Once Per Frame**, un'opzione delle versioni recenti di Unreal Engine 5, impedisce proprio questo. Terzo: per fermarlo, Set Timer by Event restituisce un **Timer Handle**, una "maniglia" che conviene salvare in una variabile (Promote to Variable) e passare, quando serve, a **Clear and Invalidate Timer by Handle**. Il posto naturale per farlo è l'evento **On Component End Overlap** della sfera, insieme a *svuotare* la variabile PlayerTarget (un Set senza nulla collegato): altrimenti l'Is Valid di MoveTurret continuerà a trovare un bersaglio valido anche dopo che il player è uscito dal range.

1.4.20 Move Turret: la torretta che segue il player

"Cosa dobbiamo fare? L'abbiamo fatto prima, praticamente": un **Find Look at Rotation**, "perché dobbiamo ruotare il nostro Pivot in direzione del player".

Prima, un Is Valid. "Intanto facciamo un Is Valid. Perché? Perché diciamo: ma ce l'ho un target?" Si trascina nel grafo la variabile **PlayerTarget** e la si collega a un Is Valid, in testa a MoveTurret. "Ce l'ho un target effettivo, o non ce l'ho più, l'ho perso, sono uscito fuori? Se ce l'ho, lo faccio e mi muovo. Se non ce l'ho, non lo faccio": senza questo controllo la torretta "continuerebbe a ruotare a cavolo".

Start e Target. "Del Find Look at Rotation ci servono due punti nello spazio. Quali sono?" "Lo Start è l'*holder*, che tanto la *gun* è collegata all'*holder*, e il Target è il player." "Esatto." Per lo Start si usa **Get Actor Location** della torretta stessa, "la locazione generale dell'actor". Per il Target, il docente comincia con Get Player Character e Get Actor Location, come per il mulino, ma uno studente fa notare che si può usare **PlayerTarget**: "bravo, mi piace che l'hai notato, effettivamente sarebbe stato il secondo metodo". Da PlayerTarget si tira fuori Get Actor Location, "una sua particolarità", e la si collega al Target.

Lo yaw e RInterp To. Il Pivot deve ruotare attorno alla **Z**: "guardate un po' dove si modifica: sulla Z". Si splitta l'uscita di Find Look at Rotation e si fa "quello che abbiamo fatto prima", un **RInterp To**:

- **Current**: la rotazione attuale del Pivot, con **Get Relative Rotation** sul componente Pivot;
- **Target**: splittato, con la sola Z collegata, "perché non ci interessa la X e la Y";
- **Delta Time**: 5;
- **Interp Speed**: **0,04**, "perché, essendo che sta sui *frame*, è molto veloce. Se voi mettete uno, questa *snappa* diretto".

Infine **Set Relative Rotation** sul Pivot ("prendete Pivot, Set Relative Rotation"), con i pin splittati "per sicurezza", collegato al ramo Is Valid; e un Print String per il debug (figura 1.4.8).

L'offset, e il pivot reimportato. In Play la torretta ruota, ma "ha bisogno di un offset, come abbiamo visto con il nostro mulino". Il docente prova con -90, poi con $+90$: "eccolo qua. Non è esattamente precisissimo". Uno studente osserva: "noi questa cosa non la dobbiamo fare, perché l'altra volta abbiamo fatto un **reimport** modificando i valori". Ed è la chiave: senza alcun offset, la torretta del docente "guarda direttamente in faccia", mentre quelle degli studenti no, perché con Davide avevano reimportato il **Pivot**

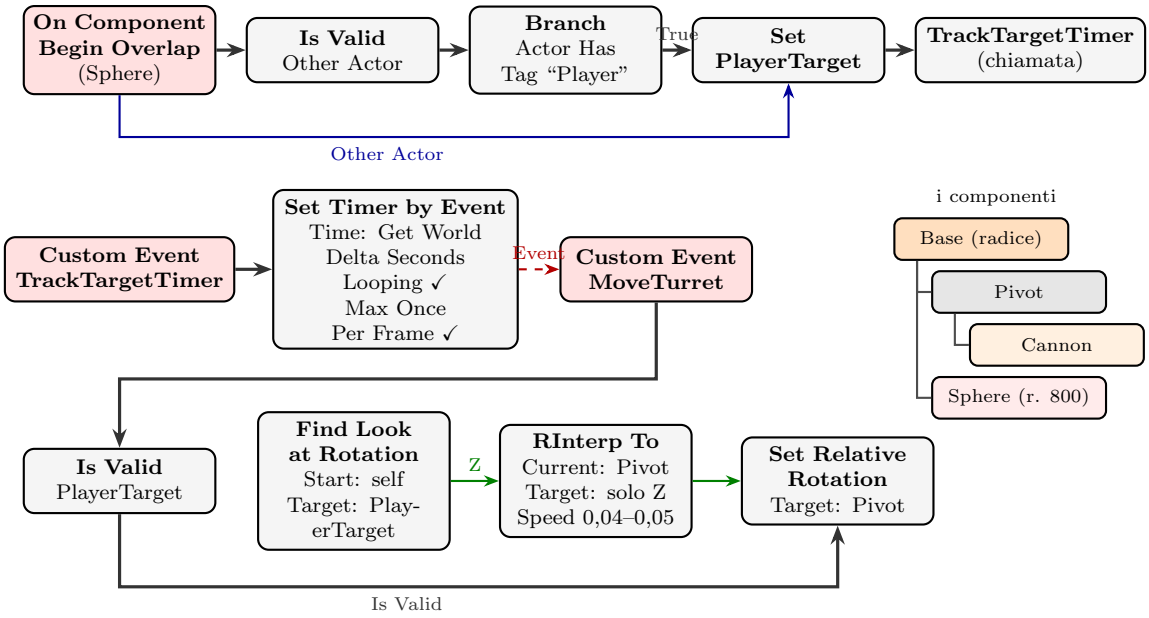


Figure 1.4.8: La torretta della lezione. In alto: quando qualcosa entra nella sfera, se esiste ed è etichettato *Player*, lo si salva in PlayerTarget e si chiama TrackTargetTimer. In mezzo: TrackTargetTimer fa partire un timer che, a ogni *frame*, chiama MoveTurret (il pin rosso tratteggiato è il delegato Event). In basso: MoveTurret, se c'è ancora un bersaglio, calcola la rotazione verso di lui, ne tiene lo *yaw*, la interpola e la applica al Pivot (la Current dell'interpolazione viene da Get Relative Rotation del Pivot; l'eventuale offset è discusso nel testo). A destra la gerarchia dei componenti: il Cannon segue il Pivot, la sfera non ruota. Schema ricostruito.

e il **Cannon** con una **rotazione di 113** gradi sulla Z (la base no). Il docente ricostruisce i numeri con la classe (si parla anche di -67 e di -271, "proprio perfetti"), reimporta prima il pivot, poi il cannone, prova con il solo pivot, e alla fine: "ecco a voi. Adesso la nostra torretta ci segue perfettamente". Per gli studenti la regola pratica, ripetuta più volte nel finale, è: se la torretta ha "un *offsettino*", **reimportare il Pivot mettendo** -113 sulla rotazione Z[14].

Approfondimento: Find Look at Rotation e lo spazio relativo. Il grafo della lezione mescola due spazi: Find Look at Rotation lavora nello spazio del **mondo** (le posizioni di torretta e player sono posizioni nel mondo), mentre Set Relative Rotation imposta la rotazione del Pivot **relativa** alla base. Le due cose coincidono finché la torretta nel livello non è ruotata: se la si piazza girata di 30 gradi, il Pivot punterà 30 gradi a lato del player. Le soluzioni robuste sono due: usare **Set World Rotation** sul Pivot (la rotazione calcolata è già quella del mondo), oppure sottrarre allo *yaw* calcolato quello dell'actor (Get Actor Rotation). Anche la Current dell'interpolazione andrebbe presa nello stesso spazio del Target: Get World Rotation con Set World Rotation, Get Relative Rotation con Set Relative Rotation. Lo Start, infine, è la posizione dell'actor, cioè della base; per una torretta alta conviene usare la posizione del Pivot o del cannone (*Get World Location* sul componente), specie quando si vorrà calcolare anche il *pitch* per alzare la canna.

[14] La trascrizione di questo passaggio è tra le più confuse della lezione: i valori citati sono 113, -113, -67, 118, -270 e -271, e non è chiaro in che ordine siano stati provati. Il senso è comunque chiaro: la rotazione di 113 gradi aggiunta in import nella terza lezione sposta la direzione "in avanti" della *mesh*, e Find Look at Rotation, che presuppone che l'oggetto guardi lungo la sua X, ne risulta sfasato. Le due correzioni possibili sono equivalenti: annullare la rotazione in import, o sommare un offset fisso allo *yaw* nel grafo, come per il mulino.

1.4.21 Il tuning e i problemi degli studenti

La velocità di inseguimento. "Giratele intorno." Per aumentare o diminuire la velocità con cui la torretta "ci mette a cercare il target" si cambia l'**Interp Speed**. A 0,07 è "molto più reattiva"; a 2 "sarà proprio quasi uno *snap*" ("pam, *snappa* proprio"). Il consiglio: "mettetela tipo 0,05". E, volendo, la si può trasformare in variabile (Promote to Variable) "e la potete settare esternamente", nei dettagli dell'istanza in scena, come le variabili *Instance Editable* della seconda lezione.

"A me snappa lo stesso." Uno studente ha Delta Time 5 e Interp Speed 0,05, e la torretta scatta comunque. Si prova 0,02 ("vedo leggerissima differenza"), poi Delta Time 7: "stampa sempre la stessa velocità". In condivisione il docente controlla Get World Delta Seconds, Looping e Max Once Per Frame: tutto giusto. "Prova ad andare di *tweak*, di *tuning*: vedi un po' tu i valori." Abbassando ancora l'Interp Speed, verso lo zero, la torretta rallenta: "ora sì, già un po' più così". (Nel frattempo lo studente che lavora deve andare: "ciao, ciao".)

Approfondimento: perché "snappa". La formula di RInterp To vista per il mulino spiega tutto. Con Delta Time fisso a 5 e Interp Speed 0,04, la frazione di strada percorsa a ogni chiamata è $5 \times 0{,}04 = 0{,}2$: il 20% di quel che manca. Dopo dieci chiamate manca $0{,}8^{10} \approx 11\%$ dell'angolo iniziale; a 100 *frame* al secondo, sono un decimo di secondo. Con 0,05 la frazione è 0,25; con 7 e 0,02 è 0,14; con Interp Speed 2 il prodotto è 10, cioè più di 1, e la torretta arriva in un solo *frame*: lo *snap*. In tutti questi casi, inoltre, la velocità dipende dai *frame*: con il doppio dei *frame* la torretta è due volte più rapida. L'uso previsto del nodo è il contrario di quello della lezione: **Delta Time** collegato a **Get World Delta Seconds** (circa 0,01 a 100 *fps*) e **Interp Speed** nell'ordine delle unità (da 2 a 5 per un inseguimento morbido), così che il prodotto resti piccolo e la velocità sia la stessa su ogni macchina. È lo stesso principio dei *delta seconds* della seconda lezione, applicato a un'interpolazione. E un'avvertenza: un Interp Speed esattamente 0 non ferma la torretta, ma la fa scattare subito sul bersaglio, perché Unreal lo interpreta come "nessuna interpolazione".

La lista degli errori. Il resto della lezione è una carrellata di condivisioni dello schermo, molto istruttiva perché mostra gli errori più frequenti. La tabella 1.4.2 li raccoglie. Il caso più ricco è quello di uno studente a cui "non va perché probabilmente ho mancato delle parti":

- per la Current di RInterp To aveva usato un *Set* invece di **Get Relative Rotation** ("no, no, no, non *set*: *get*");
- l'Is Valid di MoveTurret non era collegato alla variabile **PlayerTarget** ("a sinistra, proprio sulle variabili, non sul grafo");
- MoveTurret non era un evento separato. Per riorganizzare il grafo il docente gli fa selezionare tutto tranne TrackTargetTimer, Set Timer by Event e Get World Delta Seconds ("tieni premuto Ctrl e clicca su quelli che vuoi escludere"), spostare in basso la selezione, tirare fuori dal pin Event del timer un nuovo Custom Event chiamato **MoveTurret** e collegarlo all'Is Valid;
- il timer non era in **Looping**;
- il **Target** di Find Look at Rotation era scollegato: da PlayerTarget, Get Actor Location, al Target;
- il *character* non aveva il **tag**: lo si aggiunge nel Third Person Character, "*Player* esattamente come l'hai scritto, quindi con la P maiuscola";
- la sfera aveva ancora Hidden in Game attivo e raggio **32**: "devi metterla a 800";
- in scena c'era solo la **mesh** della torretta, non il **Blueprint**: "questa è solo la *mesh*, è solo il modello 3D che hai messo in scena"; si cancella e si trascina nel livello BP_Turret;
- il player partiva già dentro il range, e quindi il Begin Overlap non scattava: "il player parte da fuori, mettilo fuori";

- in Play **girava la base**: nella gerarchia, la base era figlia del Pivot. "Devi staccare la base": trascinando il Pivot dentro la base, "perfetto, ce l'hai perfettamente";

- infine, l'*offset* del pivot reimportato: "devi reimportare il pivot a -113".

Un altro studente ha la torretta ferma: "Other Actor devi attaccarlo a PlayerTarget, perché così non gli stai impostando il *target* che è overlappato: non avrà un *target* da seguire". Un terzo non vede la sfera in gioco, solo in modalità *spectator*: aveva saltato il passaggio di Hidden in Game.

Sintomo	**Causa e rimedio**
La pala gira attorno a un punto sbagliato	la *mesh* ha il pivot fuori centro: scegliere un cubo con il pivot al centro (o correggere il pivot)
La pala non gira	il risultato di Combine Rotators non è collegato a Set Relative Rotation; oppure il componente non è Movable
Il mulino o la torretta restano fermi	mancano Delta Time e Interp Speed in RInterp To; oppure non si è compilato
Non si riesce a collegare un valore a una sola componente	fare Split Struct Pin (tasto destro sul pin)
Gira tutto l'actor, non solo una parte	il Rotating Movement fa girare la radice: programmare la rotazione del componente
La torretta non si accorge del player	manca il tag *Player* (o è scritto diversamente); Other Actor non collegato al Set di PlayerTarget; il player parte già dentro la sfera
La sfera non si vede in Play	Hidden in Game ancora attivo; raggio di default 32
Non succede niente in Play	in scena c'è la *mesh*, non il Blueprint
Gira la base	gerarchia sbagliata: la base deve essere la radice, il Pivot suo figlio, il Cannon figlio del Pivot
La torretta guarda di lato	la *mesh* non guarda lungo X (Pivot reimportato a 113): reimportare a -113 o sommare un offset allo *yaw*
La torretta scatta (*snap*)	prodotto Delta Time $\times$ Interp Speed troppo grande: usare Get World Delta Seconds e un Interp Speed di qualche unità
La torretta continua a seguire dopo l'uscita	il timer non viene mai fermato e PlayerTarget resta pieno: End Overlap, Clear and Invalidate Timer, svuotare la variabile

Table 1.4.2: I problemi incontrati durante la lezione, con la causa e il rimedio. Le ultime due righe riguardano punti che il docente ha rimandato alla lezione successiva.

1.4.22 La chiusura e la prossima lezione

"Ok, perfetto, ragazzi." Per questa volta, "nessun particolare compito da casa o dimostrazione". Chi vuole può fare una **torretta di colore diverso**, con un materiale suo, verde o blu, ripassando quello che si è fatto con Davide.

La prossima volta. "La prossima volta faremo la nostra bellissima **torretta che spara**": **spawnerà** dei **proiettili**. Si vedrà "come funziona il sistema di *spawning* di Unreal": come si basa una cosa *spawnata* "con le sue regole", come si fa apparire un oggetto in generale, e come si fa il **line trace**. "Altre domande? Se no, andiamo a cena tutti, ragazzi." "Vi auguro buona serata. E a martedì."

Punti chiave del capitolo

- **Quixel Megascans** è un "calderone" gratuito di materiali, texture e *mesh* scansionati, da usare con l'account Epic. Per la pallina che attraversa il piano: il **CCD** (*Continuous Collision Detection*).

Nodo o componente	**Che cosa fa**
Rotating Movement Component	fa ruotare la radice dell'actor a velocità costante (Rotation Rate, default Z 180)
Event Tick, Delta Seconds	esegue a ogni *frame*; i *delta seconds* sono il tempo tra due *frame*
Get World Delta Seconds	gli stessi *delta seconds*, ovunque nel grafo
Make Rotator	costruisce un *rotator* da tre *float* (Roll X, Pitch Y, Yaw Z)
Combine Rotators	combina due rotazioni in una
Get / Set Relative Rotation	legge e scrive la rotazione di un componente rispetto al genitore
Get Actor Location	la posizione dell'actor nel mondo
Get Player Character	una referenza al personaggio del giocatore
Find Look at Rotation	la rotazione per guardare da Start verso Target
RInterp To	avvicina gradualmente un *rotator* a un altro
Clamp (float)	limita un valore tra un minimo e un massimo
Split Struct Pin	divide un pin di struttura (vettore, *rotator*) nelle sue componenti
Sphere Collision	un volume sferico per rilevare *overlap* (Sphere Radius, Hidden in Game)
On Component Begin Overlap	scatta quando qualcosa entra nel volume del componente; Other Actor è chi è entrato
Is Valid (macro con il "?")	controlla che una referenza punti a qualcosa che esiste
Actor Has Tag	controlla se un actor porta un'etichetta (Actor → Tags)
Promote to Variable	crea una variabile del tipo giusto da un pin
Custom Event	un evento creato da noi, che parte quando lo si chiama
Set Timer by Event	chiama un evento dopo un tempo, una volta o in *loop* (Max Once Per Frame)
Print String	stampa a schermo: lo strumento base del debug

Table 1.4.3: I nodi e i componenti della quarta lezione.

- Un **vettore** ha direzione, magnitudine e verso; in Unreal la struttura Vector (X, Y, Z) rappresenta sia punti sia spostamenti. Somma: si spostano i punti; **sottrazione**: B − A va da A a B; **moltiplicazione**: allunga; **prodotto scalare**: un *float*, 1 se concordi, 0 se perpendicolari, −1 se opposti (per vettori normalizzati).
- Un vettore **normalizzato** ha lunghezza 1: tiene solo la direzione.
- Un **rotator** ha tre angoli: **Roll** (X), **Pitch** (Y), **Yaw** (Z). Gli angoli di Eulero soffrono di **gimbal lock**; i **quaternioni**, usati internamente da Unreal, no.
- Il **Rotating Movement Component** fa girare un actor senza programmare, ma solo per intero. Ogni oggetto che si muove deve essere **Movable**.
- Per far girare un componente: **Event Tick** → velocità × *delta seconds* → **Make Rotator** → **Combine Rotators** con **Get Relative Rotation** → **Set Relative Rotation**. Le rotazioni dei componenti si lavorano in spazio **relativo**.
- La **gerarchia** fa il resto: un componente figlio segue il genitore, mantenendo la propria posizione relativa (la seconda pala, il cubetto).
- **Find Look at Rotation** calcola la rotazione per guardare un punto; dello *yaw* si tiene la Z (**Split Struct Pin**), con un **offset** se la *mesh* non guarda lungo X. **RInterp To** rende il movimento graduale: Delta Time dai *delta seconds*, Interp Speed di qualche unità, altrimenti "snappa".
- Il **Clamp** limita un valore; prima di sceglierne i limiti, si **stampano** i valori con Print String.
- La torretta è un Blueprint in **tre pezzi**: Base (radice), Pivot (figlio, ruota in *yaw*), Cannon (figlio del Pivot). Browse to Asset aiuta a trovare le *mesh*.
- Il range è una **Sphere Collision** (raggio 800, Hidden in Game tolto per vederla). **On Component Begin Overlap** scatta quando qualcosa ci entra; **Other Actor** dice chi.

- **Is Valid** prima di usare una referenza; **Actor Has Tag** con il tag *Player* (scritto identico) per riconoscere il giocatore; **Promote to Variable** per ricordarlo in *PlayerTarget*.
- I **Custom Event** sono eventi nostri, chiamati quando vogliamo; **Set Timer by Event** (Looping, Max Once Per Frame) chiama a ripetizione l'evento **MoveTurret**, che ruota il Pivot verso il bersaglio. Il timer va fermato (End Overlap, Clear and Invalidate Timer): sarà il prossimo passo.
- Prossima lezione: la torretta che **spara**, lo **spawn** dei proiettili e il **line trace**.

Chapter 1.5

Spawn, trace e proiettili: la torretta che spara

Questa è la **quinta lezione** del corso breve di **introduzione a Unreal Engine**, e riprende esattamente da dove si era fermata la quarta: la **torretta** in tre pezzi che, quando il giocatore entra nel suo raggio d'azione, "ci segue con lo sguardo". Il docente lo dice con una domanda retorica: "che cos'è una torretta che non difende? È una torretta che è inutile". L'obiettivo della giornata è quindi farla **sparare**: creare un **proiettile** con un suo comportamento, farlo apparire nel mondo (lo **spawn**), farlo sparire quando colpisce qualcosa (il **destroy**), e decidere *quando* sparare con una "linea immaginaria" che cerca il giocatore, il **trace**.

La lezione ha due metà separate da una pausa. La prima metà parte dalla **teoria**: che cosa vuol dire *spawnare* ("sponare", nel gergo della classe) un oggetto, perché lo spawn costa memoria e va fatto "con parsimonia", che cosa sono il *pooling* e il *garbage collector*, quali nodi lo fanno (**Spawn Actor from Class** e **Destroy Actor**), e che cos'è un **ciclo** (il **For Loop**). Poi un piccolo *fix* della torretta, con un nodo scoperto dal docente: **Find Relative Look at Rotation**. Quindi i **trace**: l'evento **Shoot** lancia uno **Sphere Trace by Channel** dalla bocca del cannone in avanti, costruito con il **forward vector**, una moltiplicazione e una somma di vettori, e un **trace channel** creato apposta, **DetectionTurret**, che vede solo il giocatore. Infine il proiettile, **BP_Projectile**: una sfera, un **Projectile Movement Component** e un trace attorno a sé che, se tocca qualcosa, lo distrugge. La seconda metà monta i pezzi: il proiettile esce a caso da una delle **due bocche** del cannone (**Select** e **Random Bool**), con un **fire rate** regolato da **Do Once**, **Sequence** e **Delay**; quando il giocatore esce dal raggio, **On Component End Overlap** svuota il bersaglio e la torretta si ferma. Negli ultimi venti minuti comincia un esercizio nuovo, da finire la volta dopo: una **matrice di cubi** generata con un **For Loop**.

La lezione è **online**, su Zoom, come le precedenti; il docente è "al momento fuori", in una stanza con "splendide tende rosa" ("la scelta stilistica non è mia"), e lavora su due schermi. Una buona parte del tempo, come nella quarta lezione, va nella risoluzione dei problemi degli studenti in condivisione dello schermo: prima dell'inizio, durante tutta la pausa e dopo ogni passaggio. La pausa, proposta alle 17:28, salta per chi ha problemi; si riprende verso le 18 e un quarto, dopo un caffè del docente, e si chiude verso le 19, "siamo abbastanza cotti tutti quanti".

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "a Real", "Arreal", "Arrial" e "allengine"; lo *spawn* è "spon", "sponare", "sbonare", "spunare", "sfonare" e perfino "lo sfondo"; l'**Event Tick** è "l'event eek", "l'eventic", "il TIC" e "il kick"; il **trace** è "tray" e "trace spawn" diventa "3-spawn", "tray spawn" e "respawn"; il *trace channel* è "3 channel" e "tray channel"; lo **Sphere Trace by Channel** è "Spear 3 Channel" e "spia trace"; **Get Actor Forward Vector** è "getAct4Vector", "get after forward vector" e "getAttorFormulAVector"; **Get World Location** è "getWorldLocationResponse" e "get word location"; **Find Look at Rotation** è "find-on-cut rotation" e "findLookAddRotation"; **RInterp To** è "r inter 2"; **Get World Delta Seconds** è "get word delta seconds"; **Is Valid** è "is va" e "unisvalid"; il **Delta Time** è "delta time" ma anche "interfeed"; la **gun** del cannone è ancora "GAN" e "ganno"; il **Pivot** è "pivo" e "PIGO"; la **sphere** è "sfiera" e "spiga"; **Do Once** è "do1"; *Third Person Character* è "terzo carattere". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della

registrazione, durante la pausa e durante il caffè, il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("qual è la risposta che ci può essere?", "grazie a tutti per l'attenzione", "aiutarvi a aiutarvi", "la struttura del giardino", discorsi su un "governo di Komen", sul "mercato del lavoro" e su "la migliore dal 2005", una sfilza di "Fine. Fine. Fine."): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è una dimostrazione dal vivo nell'editor dei Blueprint, costruita a voce ("tirate fuori", "collegate", "splittate"). Il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei pin di Unreal Engine 5; dove la trascrizione lascia un buco, lo dichiara.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono i nodi della slide sullo spawn, lo schema del *pooling*, i vettori del trace, i grafi della torretta (il *fix*, l'evento Shoot, lo spawn dalle due bocche con il *fire rate*, l'uscita dal raggio), il proiettile e la riga di trace dell'esercizio finale. Le tabelle riassumono le risposte di collisione, i canali creati a lezione, le opzioni di debug, gli errori più comuni incontrati dagli studenti e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'attesa: i **materiali spariti**, il salvataggio, un'idea per i colori della torretta; le sperimentazioni degli studenti (sezioni 1.5.1 e 1.5.2);
2. la teoria: lo **spawn**, il *pooling*, il **destroy**, i nodi **Spawn Actor from Class** e **Destroy Actor**, i **cicli** (sezioni da 1.5.3 a 1.5.5);
3. il *fix* della torretta: **Find Relative Look at Rotation** (sezione 1.5.6);
4. i **trace**: l'evento **Shoot**, i vettori, il canale **DetectionTurret** e le risposte di collisione (sezioni da 1.5.7 a 1.5.9);
5. il **proiettile**: il Projectile Movement, la distruzione, il canale **ProjectileTrace** (sezioni da 1.5.10 a 1.5.12);
6. la pausa e i problemi degli studenti (sezione 1.5.13);
7. lo spawn dalle **due bocche** e il **fire rate** (sezioni 1.5.14 e 1.5.15);
8. gli errori della ripresa, **Multi Sphere Trace**, e la torretta che si ferma con l'**End Overlap** (sezioni 1.5.16 e 1.5.17);
9. l'esercizio della **matrice di cubi** con il **For Loop**, la torretta che mira ai piedi e la chiusura (sezioni 1.5.18 e 1.5.19).

1.5.1 L'attesa: i materiali spariti e il salvataggio

La registrazione comincia, come sempre, con i "soliti 5-10 minuti" di attesa, e con un problema portato da una studentessa: aprendo il progetto, il giorno prima, "mi sono spariti, mi sono cancellati i materiali che avevamo fatto per la torretta". Unreal le aveva mostrato un messaggio d'errore, "dicendo che non era stato salvato", anche se lei era sicura di aver salvato. Risultato: "ieri notte mi sono dovuta mettere a rifare tutto quanto", la *static mesh* e il lavoro dell'ultima lezione. E i materiali rimasti "sono neri e basta": la base, il cannone e il pivot.

Come si salva. La studentessa salva "quando apriamo il Blueprint", poi "sotto, dove c'è scritto non salvato", e poi la mappa. Il docente ammette che "questi sono un po' gli arcani di Unreal": può dipendere da come si è chiuso l'editor, da una referenza rotta, da un file spostato. Il consiglio pratico è uno: usare l'**indicatore in basso a destra** dell'editor, quello che dice quante modifiche non sono salvate. "Quello è tipo l'elenco di tutte le cose salvate. Quindi qualsiasi cosa tocchi, vai là, salvi tutto da lì e non ci sono problemi."

Perché i materiali sono neri. "Se il materiale è presente e sono neri, vuol dire che non ha solamente la texture sopra": il materiale esiste, ma la **texture** a cui puntava non c'è più. La cura è aprire il materiale e riassegnargli la texture; la studentessa, nel frattempo, aveva già reimportato le texture *Turret_01* e *Turret_02*. L'editor, in condivisione, si blocca più volte ("forse con la condivisione a volte un po' peggiora"), e il caso viene rimandato "verso fine lezione".

La torretta "interista". La torretta rifatta in fretta ha colori nero e blu ("una torretta molto interista, oserei dire"). Il docente coglie l'occasione per un'idea: nel materiale c'è un blocco di colore costante (il "blocco 000", cioè un colore nero, $(0, 0, 0)$); facendo doppio clic sul blocchetto nero appare la **ruota dei colori**, e cambiando colore cambia la torretta. Ma attenzione a che cosa fa il nodo che lo usa: "questo non te lo *blenda*, perché fa un **Lerp**; dovrebbe fare un **Multiply**". Il consiglio: duplicare il materiale, assegnare la copia alla *static mesh* della torretta nuova e poi vedere insieme "come moltiplicare due colori assieme, cioè la texture è un colore, fondamentalmente".

Approfondimento: Lerp e Multiply in un materiale. Nel *Material Editor* di Unreal i due nodi fanno cose diverse. **Multiply** moltiplica canale per canale: una texture moltiplicata per un colore costante viene "tinta" di quel colore (il bianco diventa il colore, il nero resta nero), ed è il modo standard per colorare una texture in scala di grigi. **Lerp** (*Linear Interpolate*) invece *sceglie* tra due ingressi A e B in base a un terzo, Alpha: con Alpha 0 dà A, con Alpha 1 dà B, in mezzo una miscela. Usato con una maschera come Alpha, serve a "blendare due colori" in zone diverse della *mesh*; ma se l'Alpha è fisso, il colore costante non tinge la texture, la sostituisce in parte. Da qui la differenza che il docente fa notare.

1.5.2 Le sperimentazioni e il programma

"Domanda: avete fatto qualche cosa? Avete sperimentato? Giocato?" Uno studente ha provato, oltre a far ruotare la torretta in orizzontale, a muovere la sola canna ("la GAN") **in verticale**, "in modo che indichi il personaggio". Funziona verso l'alto; "verso il basso mi mira un po' troppo in alto". Il docente promette una possibile soluzione, che arriva subito con il *fix* di sezione 1.5.6; il problema tornerà, ribaltato (la torretta che mira ai piedi), nel finale della lezione.

Il programma, annunciato prima della teoria: "prima di addentrarci nello **spawning** dei proiettili, che faremo oggi, andremo a fissare una piccola cosa", cioè il Find Look at Rotation della volta scorsa; poi lo spawn, il trace, il proiettile; e un esercizio con il **For Loop**. In collegamento ci sono otto studenti ("siete ocho"), e il docente, per chi seguirà la registrazione, "lascia il messaggio ai posteri".

1.5.3 Che cos'è lo spawn

"Allora, ragazzi, domanda: prima di tutto, qualcuno di voi sa cosa si intende per sponare un qualcosa all'interno di uno spazio di gioco?" Le risposte della classe sono tutte buone, e il docente le raccoglie una per una.

- "Non dovrebbe essere il punto dove si crea quella cosa effettiva?" Più che un punto, "è un'azione effettiva".
- "Forse perché noi su **Minecraft** usiamo lo *spawn* come la zona dove spawniamo": in Minecraft c'è proprio il *punto di spawn*, e nell'editor si danno le coordinate X, Y, Z di dove far apparire qualcosa.

- "Il **falò**" di *Dark Souls*: "dove ti siedi, e poi tu appena muori ritorni lì, perché è il tuo punto di salvataggio".

- "Tutti i **bullet hell**: proprio lo spawn del *bullet*".

La definizione. "Lo spawning è tutto quello che avete detto." In termini tecnici è "l'azione dell'engine di **allocare in memoria** un elemento di gioco", cioè "farlo apparire all'interno del nostro mondo di gioco". Spesso allo spawn si associa un **dove**: il falò è un punto nello spazio, e quando ci si siede il gioco "salva le coordinate di quel falò" e ne fa il nuovo punto di spawn: "se muori, ritornerai a quel punto prefissato nello spazio". L'azione, vista dal programmatore, è "prendo il player e ti rimetto là, in una coordinata precisa"; vista dall'hardware, è "dare uno spazio alla memoria, allocato per quel player".

Lo spawn costa. Proprio perché ogni spawn occupa memoria, i **bullet hell**, che "si basano su spawnare un botto di cose", sono l'esempio perfetto di ciò che succede quando si esagera: "se avete poca memoria, a un certo punto il gioco comincia a *laggare* perché ci sono troppi elementi a schermo" e "vedrete i *frame* calare piano piano". Il docente cita un gioco ("tipo *risk frame*", il titolo nella trascrizione è incerto) che ci gioca apposta: "non ce la faccio più, però mi sto divertendo, perché vedo questa massa informe di pixel davanti a me". La regola: "sponare va fatto con **parsimonia**".

Il pooling. "C'è anche un metodo per sponare in modo più giusto, che è il metodo della **pool**": "creiamo una piscina di elementi sponabili che si ripetono", cioè sono sempre gli stessi; "non sono tipo 100 e oltre, ma sono gli stessi che girano tra quei 100. È un **riciclo**". Lo si vedrà più avanti nel corso; oggi si fanno "cose più semplici" (figura 1.5.1).

Il destroy e il garbage collector. Il contrario dello spawn è il **destroy**, "la distruzione di un oggetto": quando il proiettile andrà a sbattere contro un muro, "andiamo a liberare lo spawn, andiamo ad aggiungerlo al **garbage collector**". "Se lo sentirete mai, è il programma di Unreal che prende proprio gli scarti, i *garbage*, e li colleziona", per rimaneggiare la memoria.

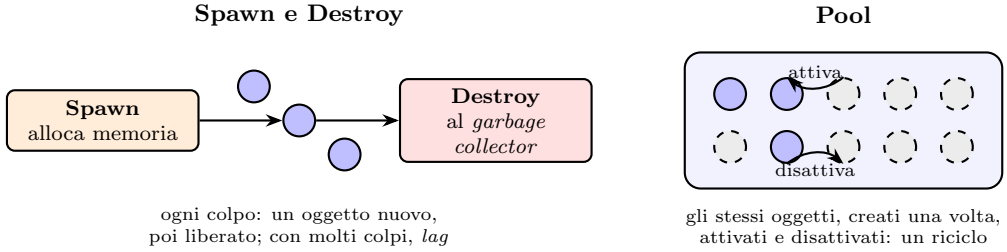


Figure 1.5.1: Due modi di gestire i proiettili. A sinistra, quello della lezione: ogni colpo è uno spawn (memoria allocata) e ogni impatto un destroy (memoria restituita al *garbage collector*). A destra il *pooling* annunciato per le lezioni future: una "piscina" di oggetti creati una sola volta, che vengono attivati (pieni) e disattivati (tratteggiati) a turno. Schema ricostruito.

Approfondimento: perché il pooling conviene. In Unreal lo spawn di un actor non è solo un'allocazione: l'engine costruisce l'oggetto e tutti i suoi componenti, li registra nel mondo, esegue il *Construction Script* e il *Begin Play*. Il destroy, a sua volta, non libera subito la memoria: segna l'actor come "in attesa di distruzione", e il *garbage collector* lo raccoglie più tardi, a intervalli, con un piccolo costo a ogni passata. Per una manciata di proiettili non conta nulla; per centinaia al secondo sì. Il *pooling* evita entrambi i costi: si spawna in anticipo un certo numero di proiettili, li si nasconde e disattiva (visibilità, collisione, movimento), e a ogni colpo se ne prende uno libero, lo si riposiziona e lo si riattiva; all'impatto torna nella piscina invece di essere distrutto.

1.5.4 Spawn Actor from Class e Destroy Actor

Sulla slide ci sono i due nodi della giornata (figura 1.5.2).

Spawn Actor from Class. Il nodo "ci chiederà una **classe**", e "la classe che si intende è la tipologia di oggetto". "Vi ricordate, abbiamo visto il discorso delle classi": c'è la classe del Blueprint che abbiamo creato noi, BP_Turret, e allo stesso modo un BP_Projectile "sarà una classe". Poi chiede uno **Spawn Transform**, che "ricordo è la formazione di *location*, *rotation* e *scale*": dove, con che orientamento e con che scala far apparire l'oggetto. Infine il **Collision Handling Override**, per decidere come gestire le collisioni al momento dello spawn: "queste collisioni le vediamo un po' più avanti, oggi non ci interessa".

Il Return Value. "All'input noi gli diamo tutte queste cose; all'output il nostro spawn ci ridà il **Return Value**, che sarà l'oggetto stesso." "Ricordiamoci: i pin **blu** sono dedicati agli actor", cioè alle referenze a oggetti; è lo stesso colore dell'Other Actor visto la volta scorsa con l'overlap.

Destroy Actor. "È semplicissimo: ci dice letteralmente distruggi quell'attore." Ha un solo ingresso, il **Target**: l'actor da distruggere (di default *self*, l'actor che esegue il grafo).

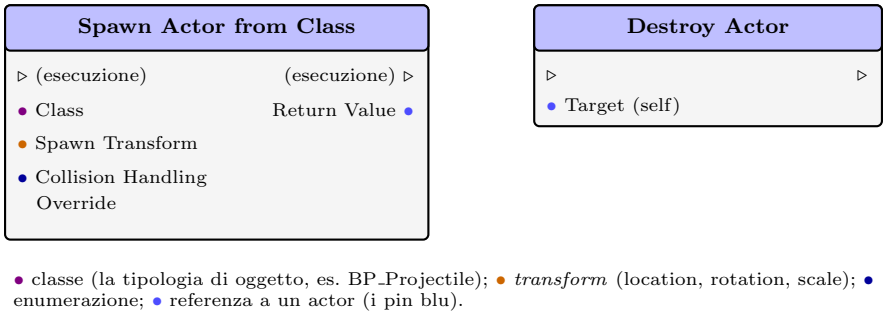


Figure 1.5.2: I due nodi della slide sullo spawn: in ingresso la classe da creare, il *transform* e il modo di gestire le collisioni; in uscita, la referenza all'actor appena creato. Destroy Actor prende solo l'actor da distruggere. I colori dei pin seguono le convenzioni dell'editor. Schema ricostruito.

Approfondimento: Collision Handling Override. Il menù a tendina decide che cosa fare se, nel punto di spawn, l'actor si troverebbe compenetrato con qualcos'altro. Le opzioni sono: *Default* (usa l'impostazione della classe), *Always Spawn, Ignore Collisions* (spawna comunque, anche dentro un muro), *Try To Adjust Location, But Always Spawn* (prova a spostarlo un poco, altrimenti spawna lì), *Try To Adjust Location, Don't Spawn If Still Colliding* (prova a spostarlo, e se non trova posto non lo crea) e *Do Not Spawn* (se c'è collisione, niente spawn). Romero e Sewell (capitolo 3) suggeriscono anche di **promuovere a variabile** il Return Value (tasto destro, *Promote to Variable*) se l'oggetto creato servirà più tardi, per esempio per distruggerlo; prima di usarla, la variabile va controllata con un **Is Valid**, perché l'actor potrebbe essere già stato distrutto.

1.5.5 I cicli e il For Loop

"Anche un'altra cosa che andremo a vedere oggi sarà il **for loop**. Qualcuno ha mai sentito parlare di **cicli** in programmazione?" Uno studente che ha studiato il C ("conosco *while*, *do while*, *for*") dà la definizione: "è ripetere un insieme di linee di codice finché una condizione è soddisfatta". E fa un esempio sulla torretta: "finché il player sta in range della torretta, la torretta si deve muovere; appena esce, quel codice non deve essere più ripetuto". "Sì, assolutamente."

Oggi, spiega il docente, si costruirà un actor che spawna dei cubetti "in linea e in colonne": "fondamentalmente andiamo a fare una **matrice**", "come in Excel, ragazzi: le matrici per righe e colonne". E lo si

farà "non uno per uno, ma tramite un **indice**": il For Loop è "un modo iterativo di eseguire il codice fino a che l'indice, quindi un numero che fa da dire OK, fatto il primo, OK, fatto il secondo, OK, fatto il terzo... Se arrivi a 10, smetti". I cicli, infine, "possono essere innestati: cicli dentro altri cicli", ed è esattamente ciò che servirà per righe e colonne.

Approfondimento: i cicli dei Blueprint. Nei Blueprint i cicli sono nodi di *flow control*. **For Loop** ha due ingressi interi, **First Index** e **Last Index**, ed esegue il pin **Loop Body** una volta per ogni valore dell'indice, dal primo all'ultimo *compresi*, esponendo il valore corrente sul pin **Index**; alla fine esegue **Completed**. **For Each Loop** fa lo stesso scorrendo gli elementi di un array (Romero e Sewell, capitolo 13, lo usano per accendere tutte le luci di una stanza). **While Loop** ripete finché una condizione booleana è vera, come il *while* del C. Una cosa importante, che distingue i cicli dei Blueprint dal tempo del gioco: un For Loop si esegue **tutto nello stesso *frame***, non "piano piano nel tempo". Non può contenere un Delay, e un ciclo troppo lungo (o un While che non finisce mai) blocca il gioco; Unreal lo interrompe con un errore di *infinite loop* dopo un numero massimo di iterazioni. Per ripetere qualcosa nel tempo si usano il Tick o i timer, come la torretta della quarta lezione.

1.5.6 Il fix della torretta: Find Relative Look at Rotation

Prima dei proiettili, "andremo a fissare una piccola cosa". Il problema è quello visto la volta scorsa: la torretta è fatta di tre *static mesh*, e a ruotare è un solo *componente*, il Pivot, che ruota **rispetto al suo genitore**, la base. Find Look at Rotation, invece, "prenderà la rotazione in confronto a quella del **mondo**, cioè di come è posizionato nel mondo". Finché la torretta non è ruotata nel livello le due cose coincidono; ma "io giro la torretta a 90 gradi in engine, e poi mi fa un offset di 90".

La scoperta. "Ho scoperto, ed è una bellissima cosa, oserei dire, che Unreal ci mette a disposizione il **Find Relative Look at Rotation** direttamente." Il nodo "prende il suo *transform* e ci fa a quel punto un Find Look at Rotation in relativa". In pratica, in MoveTurret (il Custom Event chiamato dal timer):

- si fa apparire **Find Relative Look at Rotation** al posto di Find Look at Rotation;
- al **Target** si collega quello che c'era già: la posizione del bersaglio, Get Actor Location di PlayerTarget;
- allo **Start Transform**, che chiede "da che *transform* partiamo", si collega **Get Actor Transform** della torretta;
- l'uscita resta splittata, con la sola **Z** (lo *yaw*) collegata al Target di **RInterp To**: "questo non cambia assolutamente nulla".

"Facendo ciò, da quel punto in poi la nostra torretta la possiamo ruotare come ci pare. Tanto lui prenderà sempre a livello relativo." Il docente gira la torretta nel livello, entra nel raggio: "ecco che ci segue, senza cambiamenti, senza niente" (figura 1.5.3).

La velocità di interpolazione. Il docente aveva trasformato l'Interp Speed in una variabile e "vi dico già che 0,01 era molto basso. Con questo find relative, ho messo **2** e mi va più che bene. È anche più corretto". A uno studente che non sapeva come fosse nata la variabile: "tira fuori dal pin, c'è scritto *Promote to Variable*"; poi si compila e, in basso a destra, nel *Default Value*, si mette 2. E, "intanto", il **Delta Time**: "tira fuori dal Delta Time un pin e fai **Get World Delta Seconds**".

Lo studente con la torretta che non gira. Il primo caso in condivisione è quello di uno studente che ha perso le lezioni precedenti per un problema al computer. Il pivot non era stato reimportato a -113 ("non me l'ha fatto fare", il re-import non partiva). Lo si reimporta trascinando il file del materiale didattico nel *Content Browser* (attenzione a non lasciarlo cadere sopra altre cartelle); poi, in scena, il pivot è "storto": lo si raddrizza con la modalità **Modeling** (**Shift+5**), strumento **XForm**, *Edit Pivot*. Nel Blueprint il pivot è "un po' interrato", e il cannone va staccato e riattaccato. Alla fine ruota, "c'è un piccolo offset, ma è importante che ruoti", e punta "dall'altra parte": il caso è rimandato alla pausa (sezione 1.5.13).

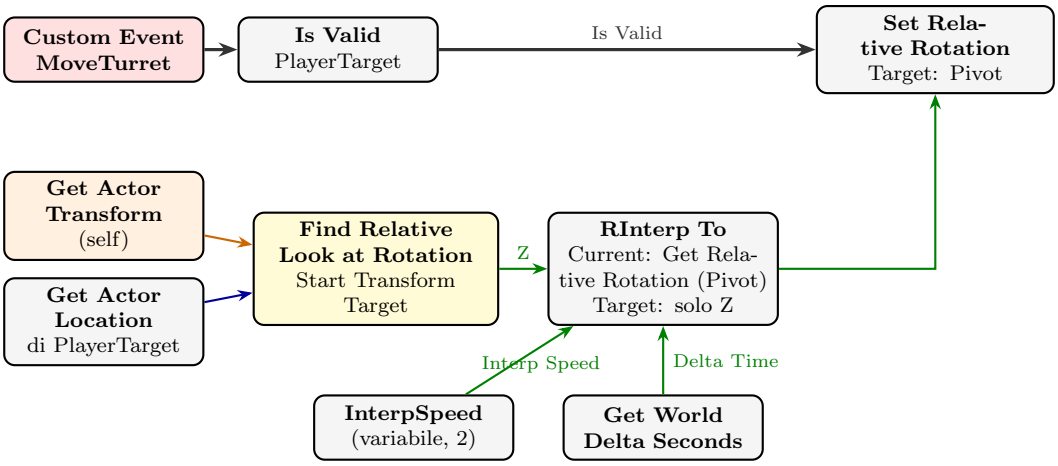


Figure 1.5.3: MoveTurret dopo il *fix*. Il nodo nuovo (in giallo) calcola la rotazione verso il bersaglio nello spazio della torretta, a partire dal suo *transform*: la torretta si può girare nel livello senza offset. Delta Time viene ora da Get World Delta Seconds e l'Interp Speed, promosso a variabile, vale 2. Schema ricostruito.

Approfondimento: perché il nodo nuovo risolve. Find Relative Look at Rotation (*Start Transform*, *Target Location*) trasforma prima il punto bersaglio nello spazio locale del *transform* di partenza, e poi calcola la rotazione per guardarlo da lì. Il risultato è quindi già una rotazione **relativa** all'actor, la stessa grandezza che Set Relative Rotation imposta sul Pivot: i due spazi, che nel grafo della quarta lezione erano mescolati, ora coincidono. Restano due avvertenze. Il punto di partenza è l'origine dell'actor, la base: va bene per lo *yaw*, ma per un *pitch* preciso converrebbe partire dal *transform* del Pivot o del cannone (Get World Transform sul componente). E l'offset dovuto alla *mesh* reimportata a 113 gradi resta: il nodo presuppone che la torretta, a rotazione zero, guardi lungo la propria X. Quanto alla velocità: con Delta Time preso da Get World Delta Seconds e Interp Speed 2, il prodotto è circa 0,02 a 100 *frame* al secondo, e la torretta insegue in modo morbido e uguale su ogni macchina, come anticipato nella lezione precedente.

1.5.7 I trace: linee immaginarie

"Abbiamo parlato di spawn, ma anche una cosa che andremo a vedere è chiamata **line trace**." "Oggi usiamo lo **sphere trace**, però in generale un trace è una **linea immaginaria** che viene fatta dall'engine" per scoprire che cosa c'è lungo un percorso.

La dimostrazione. Il docente lo mostra con la torretta che aveva già preparato ("ecco qua, spawnano la marea di proiettili"). Entrando nel raggio, davanti al cannone compare un **reticolo rosso**: "una linea immaginaria, che non vedremo se dovessimo avere un eseguibile". Ma "vedete che è rosso, rosso, rosso; se *overlappa*, diventa **verde**. Dice: ah, ho visto qualcosa! E quindi comincia a sparare". Poi, per confronto, trasforma al volo lo sphere trace in un **Line Trace by Channel**: "facciamo una linea". All'inizio la linea non trova nulla, "perché è impostata male"; basta cambiarle il canale, e "ecco che ci ha trovato": "fa rosso il puntino e verde. Dice: ah, ok, ti ho beccato".

A che cosa servono. "Sono delle linee immaginarie che servono a noi *developer* per dire: oh, c'è qualcosa che overlappa una linea invisibile? Effettivamente sì, effettivamente no." Il docente racconta di averle usate per un'intelligenza artificiale: "tracciavo una linea tra me e il player; se c'era qualcosa di mezzo, vuol dire che non lo vedeva; se invece non c'era niente, vuol dire che lo vedeva". È "proprio una **linea di vista**". E un avvertimento, ripetuto: "non esagerate con il trace, perché anche esso è molto pesante", tanto più se, come oggi, lo si fa a ogni *frame*.

Approfondimento: le famiglie di trace. Romero e Sewell (capitolo 14) definiscono i trace come test di collisione lungo un segmento. Ne esistono di due tipi di "filtro": **by Channel**, che cercano gli oggetti che bloccano un *trace channel* (Visibility, Camera o un canale creato da noi), e **for Objects**, che cercano gli oggetti di certi *object type* (WorldStatic, WorldDynamic, Pawn, PhysicsBody, Vehicle, Destructible). E di quattro forme: la **linea**, la **sfera**, la **capsula** e il **box**, dove le forme "spesse" sono più costose della linea. Infine, ogni trace ha una versione singola, che restituisce il primo oggetto colpito, e una **Multi**, che li restituisce tutti (sezione 1.5.16). Tutti hanno gli stessi ingressi principali: **Start** ed **End**, il canale o i tipi, **Trace Complex** (testare contro la *mesh* vera o contro le forme di collisione semplificate), **Actors to Ignore**, **Draw Debug Type** e **Ignore Self**. Le uscite sono un **Out Hit** (una struttura *Hit Result* con, tra l'altro, il punto d'impatto, la normale della superficie e l'actor colpito, leggibili con *Break Hit Result*) e un **Return Value** booleano: vero se qualcosa ha bloccato il trace.

1.5.8 L'evento Shoot e i vettori del trace

"La prima cosa è che dobbiamo creare un evento **Shoot**": un **Custom Event** nel grafo di BP_Turret.

Is Valid, per buona maniera. "Intanto la prima cosa che andiamo a fare è vedere se esiste il target": un **Is Valid** ("la macro con il punto interrogativo") su PlayerTarget. "È buona maniera fare questa cosa, di vedere se effettivamente esiste o non esiste, perché ci ottimizza i processi": se non c'è un bersaglio, non si esegue nulla. (Più avanti, nella sezione 1.5.14, questo Is Valid verrà spostato fuori da Shoot.)

Lo Sphere Trace by Channel. Poi lo **Sphere Trace by Channel**: "una linea immaginaria che andremo a creare in modo tale che, se il nostro character entra in questa sfera, vuol dire che io lo sto vedendo ed è in puntamento". Che cosa chiede? "Un punto di **Start** e un punto di **End**", con i pin **gialli**. "Chi si ricorda quali sono gli elementi gialli in Unreal?" "Un vettore." "Bravissimo, è un vettore. In questo caso, un vettore di locazione": una X, una Y e una Z.

Il punto di partenza: TraceSpawn. Il trace deve partire "più o meno dalla canna, da questo punto qua in avanti". Si potrebbe usare la posizione del Pivot, "però vi dico che è più corretto fare una cosa del genere: andiamo a creare proprio un **punto vuoto**" dedicato. Nei componenti si fa *Add* e si scrive **Scene**: un *Scene Component*, "solo un punto nello spazio", senza *mesh* né collisioni. Lo si **attacca al cannone**, lo si posiziona "in mezzo alle due canne", e lo si rinomina ("per rinominare le cose molto più velocemente: cliccate sull'elemento, **F2**, e scrivete") in **TraceSpawn**. "L'abbiamo attaccato al cannone perché il pivot fa ruotare il cannone e il cannone farà ruotare il TraceSpawn: c'è questa **dipendenza a cascata**." Dal componente TraceSpawn si prende **Get World Location**, "dove è posizionato nel mondo", e la si collega allo **Start**.

Il punto d'arrivo: il forward vector. "Ci manca quello del punto di fine. Dove deve arrivare?" Qui torna la teoria dei vettori della quarta lezione. Ogni *mesh*, ogni componente, ogni actor ha dei vettori di direzione "già prefatti":

- **Get Actor Forward Vector**, che "punterà sempre in avanti";
- **Get Actor Right Vector**, "che punterà sempre a destra";
- **Get Actor Up Vector**, "che punterà sempre in alto",

sempre relativi all'oggetto. "Domanda: se io moltiplicassi per -1 il forward vector, dove punterà rispetto alla mesh?" "Dietro." "Bravissimo": moltiplicare per -1 "inverte il risultato"; così si ottengono il *back* vector, la sinistra e il basso.

Lungo 1: moltiplicare. Ma se si collega il forward vector così com'è "non ci darà assolutamente nulla, perché è molto piccolo": "il forward vector che ci dà Unreal è di **lunghezza 1**", un vettore normalizzato. "Buona prassi è prenderlo e moltiplicarlo, per esempio per **800**": "avremo un vettore 800 volte più lungo". (Uno studente non riesce a scrivere 800 nel Multiply: "il pallino è giallo e non verde". "Se moltiplicate un vettore, vi dà direttamente la moltiplicazione per un altro vettore, ma potete cambiare il secondo fattore: tasto destro, *To Float*. Diventerà verde, e ci mettete 800.")

Un punto non è una linea: sommare. "Magari vi verrebbe da dire: ok, ho trovato il punto laggiù, lo vado a mettere nell'End. No, ancora no." Il forward vector moltiplicato è "un punto, non è una linea": è uno spostamento di 800 in avanti, calcolato come se si partisse dall'origine del mondo. "Per farlo diventare una linea dobbiamo letteralmente **sommare** due vettori": la World Location di TraceSpawn più il forward vector per 800. "È un discorso che sembra strano, però effettivamente quello che abbiamo qua è solo un punto nello spazio; con la somma ne traiamo un'effettiva linea" (figura 1.5.4).

Il forward vector giusto. Alla prima prova il trace c'è, ma "non segue il pivot": resta fermo mentre la torretta ruota. Il motivo: Get *Actor* Forward Vector è il vettore dell'actor intero, cioè della **base**, "quindi sarà sempre lì. Non girerà, perché deve girare insieme al cannone". La correzione: "invece di prendere il Get Actor Forward Vector, prendiamo il **Get Forward Vector del TraceSpawn**". ("A me non fa collegare TraceSpawn e Get Forward Vector": bisogna tirare il filo *dal* componente TraceSpawn e cercare *Get Forward Vector*, la versione per i componenti.)

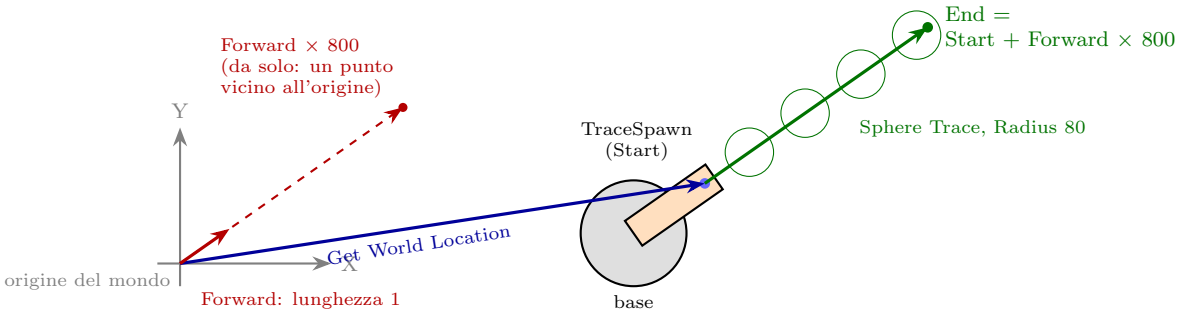


Figure 1.5.4: I vettori del trace, visti dall'alto. Get World Location di TraceSpawn è un vettore dall'origine del mondo al punto di partenza. Il Forward Vector è lungo 1; moltiplicato per 800 diventa uno spostamento, che però, preso da solo, indica un punto vicino all'origine (tratteggio). Sommato alla posizione di partenza, dà la fine giusta: 800 unità davanti alla canna. Lo sphere trace "trascina" una sfera di raggio 80 lungo il segmento. Schema ricostruito, non in scala.

Il raggio e il debug. Due dettagli ancora. Il **Radius** dello sphere trace parte da zero: "il raggio della nostra bellissima sfera sarà zero, quindi non esisterà". Il docente mette **80**. E per vedere il trace in editor, "lo vedremo solo noi *developer*", c'è **Draw Debug Type**, da impostare su **For One Frame**: "perché se adesso la mettiamo *For Duration*, ha una durata, sarà un casino, perché lo fa ogni *frame*" (tabella 1.5.1). Per provarlo, Shoot viene chiamato dall'**Event Tick** ("prendete il Tick, scrivete Shoot, come avete chiamato questo evento"): "giusto per vedere effettivamente che funzioni".

I problemi del momento. Le condivisioni mostrano errori ricorrenti: lo Start e l'End scambiati o collegati senza la somma ("Start è questo, solo questo; lo attacchi così; il più va nell'End"); Draw Debug Type lasciato su *None*; Shoot non chiamato dal Tick ("l'hai messo sul Tick? Ah, no. Metti l'evento Shoot sul Tick"); e la linea che compare ma non ruota con il cannone, per il motivo visto sopra.

Draw Debug Type	Che cosa disegna
None	niente: il trace c'è, ma non si vede (il valore di default)
For One Frame	il trace per un solo *frame*: se il trace è chiamato a ogni *frame*, si vede "in diretta"; è l'opzione usata a lezione
For Duration	il trace resta visibile per un certo tempo (Draw Time, nei dettagli del nodo): chiamato a ogni *frame*, accumula centinaia di disegni sovrapposti
Persistent	il trace resta disegnato per sempre: utile per un trace singolo, da esaminare con calma

Table 1.5.1: Le opzioni di Draw Debug Type dei nodi di trace. I colori del disegno sono il rosso per il tratto libero e il verde dal punto in cui il trace è stato bloccato (Trace Color e Trace Hit Color, modificabili nel nodo).

1.5.9 I trace channel e le risposte di collisione

Ora il trace c'è e ruota, "ma come potete vedere non abbiamo il verdino, non ci dice *ho trovato*": il player ci passa attraverso senza essere visto. Il motivo è il **trace channel**. "Di standard Unreal ci dà due canali, che sono **Visibility** e **Camera**"; quello del docente ("questo qua al testo lo vedete, l'ho creato io") è un canale **custom**, che risponde "in una certa maniera, dipendentemente da certe opzioni". Bisogna crearlo.

Creare il canale. Dal menù **Edit**, **Project Settings**; si scende "giù giù giù giù" fino a **Collision** (nella sezione *Engine*). Lì, **New Trace Channel**: "lo chiamate come volete, lo chiamiamo magari **Detection-Turret**", con **Default Response** su **Ignore**. Che cosa vuol dire? "Guarda questo trace: in risposta a tutti gli altri elementi in cui può collidere, cosa fa di base? Ignora tutto, perché è meglio che ignori tutto, in modo tale che tanto deve trovare solo il player." Se si mettesse *Block*, "ci bloccherebbe tutto quanto", e bisognerebbe togliere il blocco a tutti gli elementi statici e dinamici. "È più facile mettere ignora tutto, e poi mettiamo il *tick* positivo al nostro player."

Il player risponde Block. Si apre il **Third Person Character**, si seleziona la **Mesh** e si cerca *Collision* nei dettagli ("importante che, se non avete compilato, compilate, perché se no non vi apparirà mai il nostro DetectionTurret"). I **Collision Presets** sono grigi, perché è attivo un *preset*: si sceglie **Custom** dalla tendina, e nella riga di DetectionTurret si mette la spunta su **Block**. "In che modo reagisci al trace channel della nostra torretta? Overlap, Block, Ignore. In questo caso andiamo a fare che blocca."

Ignore, Overlap, Block. Il docente ripassa le tre risposte con le mani: "questa è la mia mano, l'oggetto 1, l'oggetto 2". Con l'**Overlap**, "se si toccano non c'è una collisione fisica, quindi volendo si trapassano", ma "c'è una *detection*: l'engine dice, sono loro che collidono, quindi c'è un segnale"; è ciò che si usa "per i blocchi invisibili". Il **Block**, invece, "è proprio un blocco fisico". Il trace non ha un corpo, quindi per lui il Block è "semplicemente un'accortezza": "ehi, ho trovato il player, mi blocco"; ed è grazie al blocco che appare la sfera "parte verde, parte rossa" (tabella 1.5.2).

Risposta	Tra due corpi	Per un trace
Ignore	si attraversano e nessuno se ne accorge	l'oggetto è invisibile per quel trace
Overlap	si attraversano, ma scatta un evento di *overlap* (i "blocchi invisibili", la sfera della torretta)	il trace lo registra ma prosegue: lo vede solo un trace Multi, e il Return Value resta falso
Block	si fermano l'uno contro l'altro: collisione fisica	il trace si ferma lì: è un *hit*, il disegno di debug diventa verde e il Return Value è vero

Table 1.5.2: Le tre risposte di collisione. Ogni componente dichiara una risposta per ogni canale (Visibility, Camera, i canali custom) e per ogni *object type*; quando due oggetti si incontrano, vale la risposta più "debole" delle due.

Scegliere il canale nel trace. Tornati in BP_Turret, nello Sphere Trace by Channel di Shoot si apre la tendina **Trace Channel**: "c'è Visibility, Camera e DetectionTurret". Se compaiono solo Visibility e Camera, "vai con **Compile**, qui in alto a sinistra". In Play: "vedete che è diventato verde. C'è il trace rosso, viene bloccato, e da lì in poi c'è quello spazio verde": "è da lì in poi che il nostro player è presente". Avvicinandosi, il tratto verde si allunga.

Il Return Value. Infine un **Print String** collegato al **Return Value** del trace, "che ci dice letteralmente: hai preso qualcosa? Sì, no". È un **booleano**: falso se non ha trovato niente, vero se ha trovato qualcosa. In Play "spunteranno un botto di true e false, ma è normale, sta sul Tick": "false, false, false... true, true, true".

Approfondimento: Mesh o Capsule? A lezione la risposta Block al canale DetectionTurret viene data alla **Mesh** del personaggio, e funziona. Nel Third Person Character, però, il componente che rappresenta il corpo ai fini delle collisioni è la **Capsule Component**, la radice; la Mesh scheletrica, di default, usa un *preset* (*CharacterMesh*) pensato per non interferire con il movimento. Dare il Block alla Mesh significa che il trace colpisce la forma del personaggio, con le sue parti fisiche; darlo alla capsula significa colpire un cilindro più semplice. Entrambe le scelte sono legittime; l'importante è sapere quale si è fatta, perché l'*Hit Actor* sarà lo stesso, ma l'*Hit Component* e il punto d'impatto no.

1.5.10 Il proiettile: BP_Projectile e il Projectile Movement

"Abbiamo fatto la torretta che gira, abbiamo fatto la torretta che detecta: adesso dobbiamo far sparare." Un Custom Event **SpawnProjectile**, "ma che succede, ragazzi? Non abbiamo un proiettile". Lo si crea: tasto destro nel Content Browser, **Blueprint Class**, **Actor**, e lo si chiama **BP_Projectile**.

Due actor, due compiti. Perché un Blueprint a parte, e non tutto dentro la torretta? "Due actor, uno che fa un compito e uno che fa un altro, è il miglior modo per gestire i comportamenti." "Perché a un certo punto magari vi arriva un *designer* e vi dice: ragazzi, non voglio che spawni più i proiettili, voglio che spawni **banane**. Eh, se l'avete fatto dentro la torretta dovete fare un casino. Invece, cambiare semplicemente il proiettile è molto più facile."

La mesh. Il proiettile è "formato da due cose". La prima è una **Static Mesh**, rinominata **MeshProjectile**: una **sfera** di default ("se volete, potete scaricare un modello, così ci mettete un proiettile al posto della palla"). La si scala a **0,5**, poi a **0,3** ("mi sembra un po' grandicello"), con il **lucchetto** della scala chiuso. "Questo lucchetto l'abbiamo spiegato alla prima lezione": aperto, permette di modificare gli assi uno per uno; chiuso, "se modificate un singolo asse, vi scalerà anche le altre, in modo tale che rimangano uniformi, perché sennò vengono forme stiracchiate". Come materiale, **MI_Solid_Blue** ("mi solid blue").

Il Projectile Movement. La seconda cosa è il movimento. "Unreal ci arriva con un regalo incredibile": come la volta scorsa c'era il Rotating Movement per il mulino, "arriva la possibilità di avere un **Projectile Movement**, per quanti giochi ormai hanno dei proiettili". Si aggiunge il componente, e nei dettagli ci sono "un sacco di dati in cui possiamo giocare": "tanti giochi ai giorni d'oggi hanno cose molto comuni. Quanti FPS ce ne sono? Anche le armi di *Borderlands*, che sono tutte strane, alla base si parla sempre di sparare". I valori della lezione:

- **Initial Speed** 1000 e **Max Speed** 1000 (per guardarlo meglio, a un certo punto l'Initial Speed scende a 100, "la lumaca", e poi torna a 1000, "una velocità da proiettile");
- **Projectile Gravity Scale** a **0**: "questo proiettile al momento è afflitto dalla gravità"; "noi vogliamo che vada dritto per dritto";
- **Velocity** lasciata a $(1, 0, 0)$: "la velocità è la direzione", quindi "è 1 sull'asse X", e il proiettile "andrà dritto così".

E la mesh deve essere **Movable**, "se no vi danno errore". Messo in scena e avviato il Play, "il nostro bellissimo proiettilino se ne va per la sua" strada: attraversa tutto, "overlapperà, e chissà chi lo vedrà più". (A una studentessa la pallina "cade semplicemente": aveva lasciato la gravità a 1.)

Approfondimento: i parametri del Projectile Movement. Romero e Sewell (capitolo 6) modificano il proiettile del modello *First Person* con gli stessi parametri. **Initial Speed** è la velocità con cui il proiettile parte; **Max Speed** è il limite, che conta solo se qualcos'altro lo accelera (un razzo con il motore acceso); per un proiettile d'arma ha senso che le due coincidano. **Velocity** è una direzione nello spazio *locale* del proiettile: se Initial Speed è maggiore di zero, Unreal ne usa solo la direzione e la moltiplica per l'Initial Speed. Per questo è importante che lo spawn dia al proiettile la rotazione giusta: la X locale del proiettile deve guardare fuori dalla canna. **Projectile Gravity Scale** moltiplica la gravità del mondo (0 nessuna, 0,1 "proiettile leggero"). La sezione **Projectile Bounces**, con *Should Bounce*, decide se rimbalza sui muri; e **Rotation Follows Velocity** lo fa girare nella direzione del moto, utile per frecce e razzi.

1.5.11 Il proiettile che si distrugge

"Noi vorremmo che il nostro proiettile sia molto più realistico: trova un muro e scompare." E non è solo estetica: "immaginatevi che adesso spawneremo dei proiettili che andranno nell'infinito e oltre"; l'engine "dirà: guarda, esiste", e "non libereremo mai la memoria".

Un trace attorno al proiettile. La soluzione riusa il trace: nell'**Event Tick** di BP_Projectile, uno **Sphere Trace by Channel** di raggio **30**, "una sfera intorno al nostro proiettile che ci dirà: guarda, hai colpito qualcosa?". Start ed End sono **lo stesso punto**, **Get Actor Location** del proiettile: "vogliamo che l'inizio e la fine siano lo stesso punto". Con Draw Debug Type su For One Frame, "intorno alla nostra pallina si è creato un trace".

Branch e Destroy Actor. Poi: "molto semplicemente, Return Value è vero? Sì? Allora distruggete". Un **Branch** ("tenete premuto B e cliccate") sul Return Value, e sul ramo True un **Destroy Actor** (figura 1.5.5). In Play, alla "velocità della lumaca", il proiettile arriva al muro e sparisce: "si è distrutto, quindi abbiamo il comportamento di base". Il Tick, nota il docente, "inizierà solamente quando l'oggetto esiste": ogni proiettile controlla sé stesso finché vive.

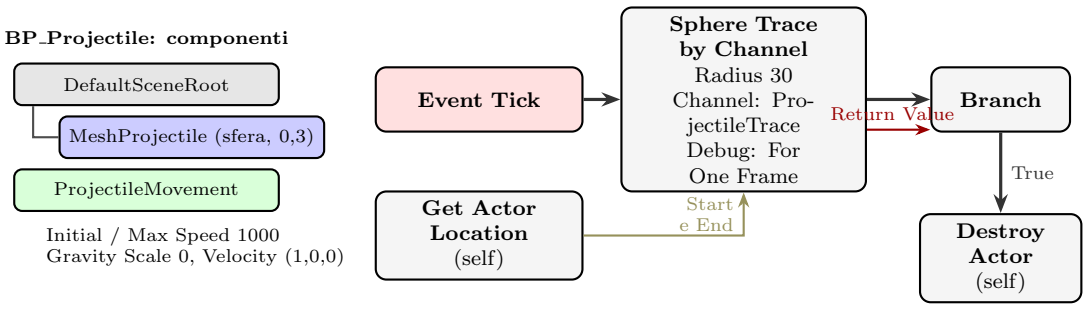


Figure 1.5.5: Il proiettile. A sinistra i componenti: una sfera scalata e il Projectile Movement, che fa tutto il moto. A destra il grafo: a ogni *frame*, una sfera di raggio 30 attorno al proiettile (Start ed End coincidono); se qualcosa la blocca, il proiettile si distrugge. Il canale, all'inizio Visibility e poi Camera, diventa alla fine ProjectileTrace. Schema ricostruito.

Approfondimento: altri modi di distruggere un proiettile. Il trace sul Tick funziona ed è istruttivo, ma non è l'unico modo, né il più economico. Il Projectile Movement ha un proprio evento, **On Projectile Stop**, che scatta quando il proiettile si ferma contro qualcosa che lo blocca; e un componente di collisione (una Sphere Collision come radice del proiettile, con il Projectile Movement che la muove) genera **Event Hit** o **On Component Hit** all'impatto. Romero e Sewell (capitolo 6) collegano proprio all'evento di *hit* il

Destroy Actor, "così che il proiettile venga sempre distrutto quando collide". Infine, per i proiettili che non colpiscono nulla, l'actor ha una proprietà **Initial Life Span** (nei *Class Defaults*): se maggiore di zero, l'actor si distrugge da solo dopo quei secondi. È una rete di sicurezza contro i proiettili che vanno "nell'infinito e oltre".

1.5.12 Visibility, Camera e il canale ProjectileTrace

Il proiettile si distrugge contro il muro, "anche il player, volendo?" No: "mi metto davanti, e infatti non ce lo prende". Con il canale **Visibility** il personaggio non viene visto; cambiando il canale in **Camera**, "così ci prende pure il player".

Perché. Il docente apre le collisioni del Third Person Character: c'è la tabella delle **Collision Responses**, con Ignore, Overlap e Block, e sotto le **Trace Responses**, "cioè come rispondo a questo trace". "Come rispondo al trace Visibility? Vedete che c'è la spunta su Ignore; invece il Camera viene bloccato." "Semplicemente è un concetto di collisioni."

Un canale per il proiettile. "Lo facciamo, ragazzi, così almeno è buona prassi": un secondo canale, **ProjectileTrace**. Qui il ragionamento è opposto a quello di DetectionTurret. Il docente prima pensa a Ignore e a mettere il Block sui quattro muri uno per uno ("però è più pesante"), poi cambia idea: "è molto meglio farlo in **Block**, così facciamo un Block e leviamo semplicemente quelli che non ci interessano". La regola generale: "quante cose ho che devono bloccare, e quante che devono ignorare?". Il proiettile "sbatte praticamente con tutto", quindi Default Response **Block**, e si escludono le poche eccezioni (tabella 1.5.3):

- il *character*: Block ("va benissimo");
- la **Sphere** della torretta, il raggio d'azione: Custom, ProjectileTrace su **Ignore**, altrimenti il proiettile esploderebbe appena nato, dentro la sfera;
- anche **Pivot**, **Base** e **Cannon**: "non vogliamo che rimbalzi contro la stessa torretta che l'ha sparato", quindi Custom e Ignore a tutti e tre.

Nel trace del proiettile si sceglie ProjectileTrace ("sempre compilato, perché se no non vi arriva"), e in Play il proiettile si distrugge sia sul player sia sul muro.

Canale	Default	Eccezioni impostate a lezione
DetectionTurret	Ignore	Mesh del Third Person Character: **Block**. Il trace della torretta vede solo il giocatore.
ProjectileTrace	Block	Sphere, Pivot, Base e Cannon della torretta: **Ignore**; dopo la pausa anche la sfera del proiettile stesso. Character: Block. Il proiettile si ferma su tutto tranne che sulla torretta che lo spara.

Table 1.5.3: I due *trace channel* creati in Project Settings → Engine → Collision. Il principio: se le cose da colpire sono poche, Default Ignore e Block sulle eccezioni; se sono molte, Default Block e Ignore sulle eccezioni.

"A me passa attraverso." Due studenti hanno il proiettile che attraversa player e muri ("il player me lo sposta, e i muri li attraversa"). Si controllano il canale (Block), il nodo (ProjectileTrace), le collisioni della Mesh: tutto giusto. Il colpevole è il **Radius**: "era zero, quindi era invisibile". "Attenzione, ragazzi, perché il radius è la grandezza: se è zero, non esiste." (E il raggio del trace della torretta? "80.") Un'altra studentessa non trova i Collision Presets della Sphere della torretta: bisogna selezionare proprio il componente *Sphere* nella lista dei componenti, e la sezione Collision compare.

Approfondimento: Ignore Self e Actors to Ignore. Tutti i nodi di trace hanno la casella **Ignore Self**, attiva di default: il trace ignora tutti i componenti dell'actor che lo esegue. Il trace del proiettile, quindi, non vede la propria sfera anche senza toccarne le collisioni, e quello della torretta non vede la torretta. Per escludere *altri* actor, senza modificare le loro collisioni, c'è l'ingresso **Actors to Ignore**, un array: per esempio, il proiettile potrebbe ricevere allo spawn una referenza alla torretta che l'ha sparato (i pin *Owner* e *Instigator*, che compaiono espandendo Spawn Actor from Class, servono proprio a passare questo genere di informazione) e ignorarla nel trace. È la soluzione più pulita quando le torrette sono tante e ognuna deve ignorare solo i propri colpi.

1.5.13 La pausa e i problemi degli studenti

Il docente crea il Custom Event SpawnProjectile e tira fuori **Spawn Actor from Class**; poi guarda l'ora: "sono le 17 e 28. Vogliamo fare pausa adesso, per fare lo spawn dopo?" La classe vota per la pausa, "i classici 20 minuti", alle 17:50. Ma chi ha problemi resta, e la pausa diventa un lungo giro di condivisioni, utile soprattutto per le tecniche di *debug* che mostra.

Le collisioni della torretta. Il primo caso è rapido: come si toglie la collisione con il proiettile a Gun, Pivot e Base? "Devi cliccare su ogni asset, Collision Presets, fai Custom, Ignore a ProjectileTrace. A tutti e tre."

Il Tick che non parte. Il secondo caso è il più lungo, e resta irrisolto. Uno studente non vede il trace della torretta in Play. Il docente prova, in ordine:

1. un **breakpoint**: si seleziona il nodo Shoot e si preme **F9**, poi Play; se l'esecuzione arriva lì, l'editor si ferma e mostra il grafo ("quello è un *breakpoint*; poi lo spiego in seguito"). Non si ferma: Shoot non viene mai eseguito;

2. un **Print String** sul ramo *Is Not Valid*, per capire se è il bersaglio a mancare; nel farlo, si scopre che sullo schermo compaiono già delle scritte, e le si insegue fino a un vecchio Print String dimenticato nel mulino;

3. un Print String direttamente sull'**Event Tick**: dovrebbe "spammare a manetta", e invece niente. "Si è rotto il Tick";

4. cancellare l'Event Tick e ricrearlo; chiudere e riaprire il Blueprint; cercare nei dettagli l'opzione di abilitazione del Tick ("scrivi *enable*"); togliere l'actor dalla scena e rimetterlo;

5. un Blueprint nuovo di prova, con solo Event Tick e Print String: quello funziona.

"Diciamo che è un caso abbastanza particolare, ti posso dire la verità. Non riesco a capire effettivamente perché il Tick non funzioni." La soluzione è pratica: il docente comprime il proprio progetto e lo carica su **Classroom**, tra il materiale didattico ("faccio tipo versione del 0406"), e lo studente riparte da quello.

Approfondimento: quando il Tick non parte. Le cause più comuni, da controllare in quest'ordine, sono: l'opzione **Start with Tick Enabled** (nei *Class Defaults*, sezione *Actor Tick*) disattivata; un nodo **Set Actor Tick Enabled** con il valore falso eseguito da qualche parte; un **Tick Interval** diverso da zero, che fa scattare il Tick solo ogni tanto; un *Event Tick* rimasto *scollegato* perché nel grafo ce n'è un altro (Unreal ne permette uno solo, e un duplicato incollato compare come nodo disabilitato); e, nell'editor, l'istanza in scena che non è il Blueprint modificato (una vecchia copia, o un Blueprint diverso con lo stesso aspetto). Il test del Blueprint nuovo, fatto a lezione, è il modo giusto per separare un problema del progetto da un problema dell'actor.

Il cannone che punta dall'altra parte. Il terzo caso è lo studente della sezione 1.5.6: la torretta ruota, ma "quando entra nella durezza" (nel raggio) "il cannone punta dall'altra parte del personaggio". Nel grafo, Is Valid e Find Relative Look at Rotation sono giusti. Il problema è la *mesh* del pivot: "è storto". La reimportazione dà errore, quindi si usa il secondo metodo, la modalità **Modeling** (**Shift+5**): nella colonna di sinistra, **XForm**, poi *Edit Pivot*, con *Box Position* e la rotazione. "Deve essere la **X** dritta": la freccia rossa del pivot deve puntare in avanti, lungo la canna. Attenzione allo spazio del gizmo: "metti il **Local** in alto a destra" (invece di *World*). Dopo un paio di tentativi la torretta segue, ma ruotata di 180 gradi: si corregge ruotando il cannone sull'asse **Z** ("la blu, usa la blu"). Resta il trace che punta storto: TraceSpawn ha una rotazione di 42 gradi rimasta da qualche spostamento; la si azzera ("la freccettina" gialla accanto al valore, che riporta al default), e "è lui".

Il trace che non gira e la sfera. Il quarto caso: TraceSpawn non era figlio del cannone ("probabilmente il TraceSpawn non è collegato al cannone, già lo vedo"): lo si trascina sopra Cannon nella lista dei componenti. Poi la base non era la radice e la Sphere non era figlia della base: si trascina la base sopra la radice e la sfera sopra la base ("è la base, la dipendenza della sfera"). "Se adesso nella viewport sposto la torretta, si sposta anche la sfera?" "Sì." Per rendere la torretta più reattiva: aumentare l'Interp Speed.

1.5.14 Lo spawn dei proiettili: le due bocche

Dopo il caffè, "riprendiamo da dove abbiamo interrotto: dobbiamo spawnare i nostri proiettili in una posizione in particolare".

Due punti di spawn. "Il nostro cannone ha **due bocche**", quindi due *Scene Component*: **ProjectileSpawn1** e, con Ctrl+C e Ctrl+V, **ProjectileSpawn2** ("ne fa il numero in automatico"). Si attaccano al **Cannon**, come TraceSpawn, e si posizionano "all'uscita di ogni bocchetta".

Spawn Actor from Class. Nel Custom Event **SpawnProjectile**, il nodo Spawn Actor from Class:

- **Class**: dalla tendina, BP_Projectile ("o come l'avete chiamato");
- **Spawn Transform**: "la nostra posizione, lì dove lo vogliamo far spawnare". All'inizio il docente fa *Split Struct Pin* ("ci dice Spawn Transform Location, Rotation e Scale") e collega alla Location il **Get World Location** di ProjectileSpawn1; "la rotazione non ci interessa, la scala non ci interessa. Anzi, no, facciamo per sicurezza": ricombina il pin e usa **Get World Transform**, "così ci diamo pure la rotation".

La rotazione, in realtà, è essenziale: è lei a dire al proiettile dove sta "davanti" (la sua X locale, la direzione del Projectile Movement), quindi senza di essa i proiettili partirebbero tutti lungo la X del mondo.

Collegare lo sparo: il trace decide. "Abbiamo la funzione, ma mettiamo: hai trovato qualcosa?" In Shoot, dopo lo Sphere Trace, un **Branch** sul Return Value, e sul True la chiamata a **SpawnProjectile**. E l'Is Valid si sposta: "prendete questo Is Valid e spostiamolo da qui: Ctrl+X", e lo si incolla **sull'Event Tick**, prima della chiamata a Shoot. "Perché è più ottimizzato? Perché la funzione di per sé è a sé stante, ma il Tick parte a fare l'azione Shoot solamente se il player è valido." In Play, i proiettili escono "in un modo velocissimo": uno a ogni *frame*.

Una bocca a caso: Select e Random Bool. "Dobbiamo farli spawnare da due bocche, randomicamente." Si stacca ProjectileSpawn1, si tira fuori anche ProjectileSpawn2, e si usa un **Select**: "secondo un *index*, che può essere un intero, ma noi lo faremo booleano, selezionami l'actor da prendere". Con l'indice falso esce ProjectileSpawn1, con l'indice vero ProjectileSpawn2. All'indice si collega un **Random Bool**: "ogni volta che sparerà, tira fuori un booleano randomico tra vero e falso". Dall'uscita del Select si prende Get World Transform e lo si porta allo Spawn Transform (figura 1.5.6).

Approfondimento: Select e i suoi parenti. Il **Select** è un nodo *puro* (senza pin di esecuzione) che restituisce uno dei suoi ingressi in base a un indice; le opzioni possono essere di qualsiasi tipo (qui referenze a componenti), mentre l'indice può essere un intero, un booleano, un *byte* o un'enumerazione (Romero e Sewell, capitolo 15, lo usano per spawnare un boss diverso a seconda della difficoltà). Le alternative: un **Flip Flop** alternerebbe le due bocche in modo regolare, sinistra, destra, sinistra (Romero e Sewell lo mostrano proprio con due pistole impugnate insieme, capitolo 13); un **Random Integer in Range** con un Select a più opzioni servirebbe per un cannone a tre o quattro canne. Un dettaglio sui nodi puri: vengono rivalutati ogni volta che un nodo li legge, quindi un Random Bool collegato a due nodi diversi può dare due valori diversi nello stesso colpo; se il risultato serve due volte, conviene salvarlo prima in una variabile.

1.5.15 Il fire rate: Do Once, Sequence e Delay

"E dargli un **fire rate**": un proiettile al secondo, non uno per *frame*. La trascrizione di questo passaggio è frammentaria, ma i pezzi sono chiari, e così il risultato mostrato in Play.

Il Delay. Il primo strumento è un **Delay**: "il delay sarà ogni quante volte, cioè un secondo: ogni secondo mi spawno un proiettile". Shoot è chiamato a ogni *frame*, ma un Delay già in corso ignora le nuove chiamate finché non è scaduto: quindi passa al massimo uno sparo per intervallo. "Però vi faccio vedere: se adesso io vado davanti, aspetta un secondo e poi parte. Noi vogliamo che parta **subito**, appena mi vede."

Do Once e Sequence. Il secondo strumento è un **Do Once**, che lascia passare l'esecuzione *una sola volta* finché qualcuno non lo resetta; il terzo una **Sequence**, che esegue in ordine le sue uscite. Il grafo che ne risulta, dopo il Branch del trace:

- **Then 0**: Do Once → SpawnProjectile, il colpo *immediato*, appena la torretta vede il player;
- **Then 1**: Delay → SpawnProjectile, i colpi successivi, uno per intervallo.

Il Do Once va resettato quando il bersaglio sparisce, così il prossimo ingresso nel raggio avrà di nuovo il colpo immediato: "facciamo che si resetta il Do Once quando… Is Valid col player target, quando non è valido. Quindi: mi resetti". Il ramo **Is Not Valid** dell'Is Valid sull'Event Tick va al pin **Reset** del Do Once (figura 1.5.6).

FireRate. Infine la durata del Delay diventa una variabile: *Promote to Variable*, "lo chiamo **FireRate**". "Vogliamo che sia più veloce? 0,5 secondi." "Pum, pum, pum, molto più veloce." "Seguirà con lo sguardo, continuerà a spararci se ci vede. Per adesso la torretta, con questo, è finita." Si potrebbero aggiungere "un sacco di cose, VFX, suoni, tutto quanto, però per quanto riguarda la programmazione è completa". (Più alto è il valore, più lungo è l'intervallo tra un proiettile e l'altro: è una durata, non una frequenza.)

Approfondimento: Delay, Do Once e i timer. Romero e Sewell (capitolo 13) descrivono il **Do Once** con l'esempio di un'arma caricata: alla pressione del tasto spara una volta, e un timer chiama un evento che resetta il Do Once dopo trenta secondi. È la stessa idea, a rovescio, del grafo della lezione. Il **Delay** è un nodo *latente*: sospende quel ramo di esecuzione e lo riprende dopo il tempo indicato; chiamato di nuovo mentre conta, viene ignorato (a differenza del **Retriggerable Delay**, che ricomincia a contare). I nodi latenti funzionano solo nell'Event Graph, non nelle funzioni. L'alternativa più "pulita" a Delay e Do Once è un **timer**, come quello che muove la torretta: all'ingresso del player si chiama subito SpawnProjectile e si avvia un **Set Timer by Event** in *loop* con Time uguale a FireRate; all'uscita lo si ferma con **Clear and Invalidate Timer by Handle**. Così niente lavora a ogni *frame* se non serve.

1.5.16 Gli errori della ripresa e il Multi Sphere Trace

"Adesso vediamo gli errori."

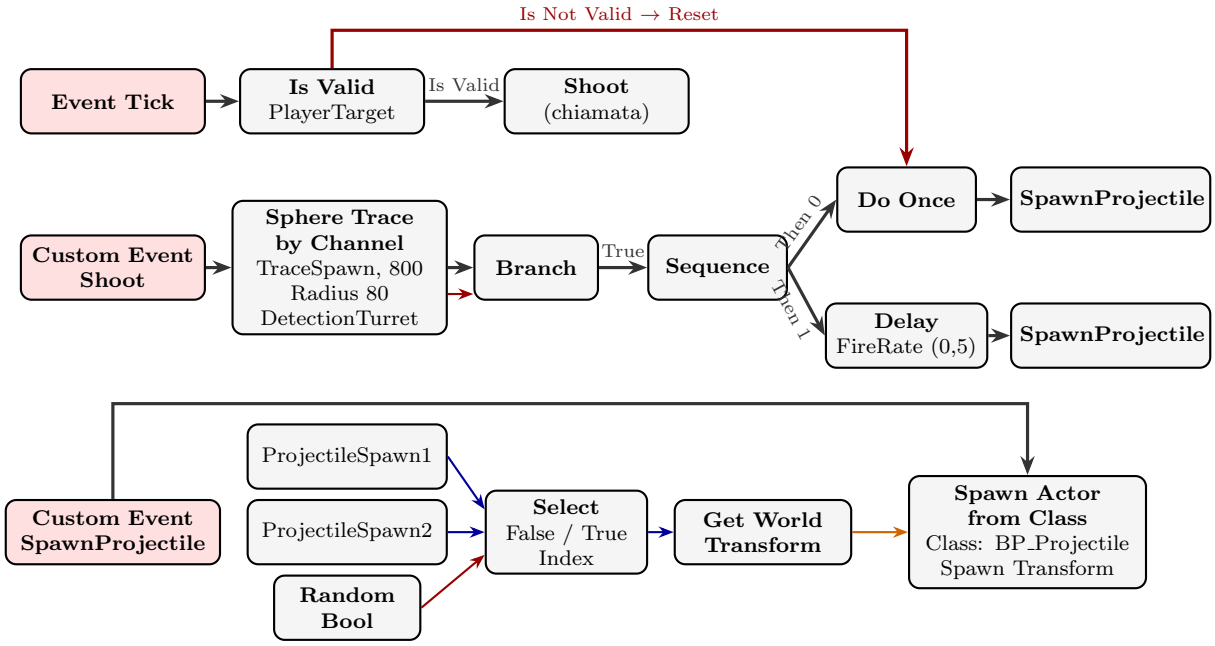


Figure 1.5.6: La torretta che spara. In alto: a ogni *frame*, se c'è un bersaglio, si chiama Shoot; se non c'è, si resetta il Do Once. In mezzo: Shoot lancia il trace; se vede il player, spara subito una volta (Do Once) e poi una volta ogni FireRate secondi (Delay). In basso: SpawnProjectile sceglie a caso una delle due bocche e vi crea un proiettile, con la posizione e la rotazione della bocca. Schema ricostruito: l'ordine esatto dei collegamenti tra Do Once, Sequence e Delay è dedotto dalla trascrizione e dal comportamento mostrato in Play.

Il proiettile che resta lì. Prima della Sequence, due studenti avevano i proiettili che restavano fermi davanti alla bocca, o sparivano appena usciti ("spara il proiettile, però resta lì"; "si distrugge appena esce"). Velocità: 1000, giusta. Collisioni della torretta su Ignore: giuste. Uno dei due, con il Do Once e il Delay, se lo vede risolto da solo: "forse stava spawnando troppi proiettili assieme". "Bravissimo: bisogna togliere la collisione a sé stesso". Siccome ProjectileTrace blocca tutto, i proiettili, uno per *frame*, si colpivano tra loro: si va sulla sfera di BP_Projectile, Collision Presets **Custom**, ProjectileTrace su **Ignore**, "quindi fate ignorare anche loro stessi". All'altra studentessa "c'è la rete, come se sparasse la rete del proiettile, però non c'è il proiettile": si distrugge subito; la causa è un componente della torretta senza Ignore su ProjectileTrace.

Approfondimento: perché i proiettili si distruggevano. Ignore Self, come visto, esclude dal trace i componenti dell'actor stesso, ma non *gli altri* proiettili. Con uno sparo per *frame* e una sfera di trace di raggio 30, ogni proiettile nasce praticamente dentro il precedente, e lo vede: si distruggono a vicenda. Togliere a BP_Projectile la risposta Block su ProjectileTrace li rende invisibili gli uni agli altri, e risolve; rallentare il ritmo di fuoco, che allontana un proiettile dall'altro, risolve anche da solo. Lo stesso accade se il raggio del trace del proiettile (30) è più grande del proiettile stesso (una sfera di 100 unità scalata a 0,3 ha raggio 15) e la bocca è vicina a un componente che blocca: per questo il docente, notando che "i proiettili sono un po' più piccoli rispetto al trace", suggerisce un raggio "di 10, di 20".

Dove si cambia il fire rate. "Nella Sequence, il Delay: io l'ho fatto come variabile, ma di base è la durata del Delay, la *Duration*". A chi manca la variabile FireRate: dal pin Duration, *Promote to Variable*, valore 0,5.

Sphere Trace e Multi Sphere Trace. Uno studente si accorge di aver usato **Multi Sphere Trace by Channel**. "La differenza non ti cambia moltissimo, perché è sempre uno sphere trace. Ma lo Sphere Trace si prende un oggetto; il Multi Sphere Trace te ne prende di più: due, tre, quattro, cinque oggetti insieme", "una selezione a gruppo invece di una selezione singola". "Se metti uno sphere trace e metti due player, ti identificherà solo un player; se fai un multi sphere trace, ti riprende tutti e due." Quindi: lo Sphere Trace "se dobbiamo prendere solamente un bersaglio, come il player"; il Multi "se dovessimo fare un check del tipo: ci devono essere almeno tre player per far sì che la torretta spari". Il riepilogo per chi si era perso: lo Start da TraceSpawn, l'End con la somma, "Radius a 80, Trace Channel DetectionTurret, Draw Debug Type For One Frame".

Approfondimento: che cosa restituisce un Multi. Un trace Multi restituisce un **array** di Hit Result, **Out Hits**, invece di uno solo. Con un canale, il trace attraversa gli oggetti che rispondono *Overlap* e li aggiunge alla lista, e si ferma al primo che risponde *Block*, che è l'ultimo elemento. Il Return Value è vero solo se c'è stato un Block. Quindi, sul canale DetectionTurret dove il player *blocca*, due player in fila danno comunque un solo *hit*, il primo: per contarli tutti, il player dovrebbe rispondere Overlap e si dovrebbe leggere la lunghezza dell'array (*Length*). Romero e Sewell notano che le versioni Multi sono più costose di quelle singole: si usano solo se servono davvero più risultati.

1.5.17 La torretta si ferma: On Component End Overlap

Uno studente fa notare che la torretta lo segue "anche quando esco fuori" dal raggio. "È voluto, poi lo interrompiamo": è il pezzo che manca dalla lezione precedente. "Dobbiamo far sì che, quando il nostro player esce, la variabile del player target si **svuoti**, in modo tale che non abbiamo più un target. La torretta quindi smette di funzionare totalmente e si ferma."

Il contrario del Begin Overlap. Si seleziona la **Sphere**, si scende nei dettagli fino agli eventi e si aggiunge (con il "+") **On Component End Overlap**. È il gemello dell'evento della volta scorsa, che controllava che l'actor entrato fosse valido e avesse il tag, e lo salvava come bersaglio. "Adesso andremo a fare letteralmente il contrario": da **Other Actor**, **Actor Has Tag** *Player* ("l'abbiamo già messo"), un **Branch**, e sul True un **Set** di **PlayerTarget** con il pin di ingresso **vuoto**: "che vuol dire vuota? Vuol dire che non è più valida. Così si setta una variabile vuota: semplicemente non impostando niente" (figura 1.5.7).

L'effetto a cascata. Svuotata la variabile, tutti gli Is Valid del grafo rispondono *Is Not Valid*: MoveTurret non ruota più il Pivot, l'Event Tick non chiama più Shoot, "e quindi il Do Once si resetta, con il problema di prima". In Play: "noi usciamo, il trace scompare. E ricompare", quando si rientra. "Ed ecco che abbiamo finito effettivamente la nostra torretta."

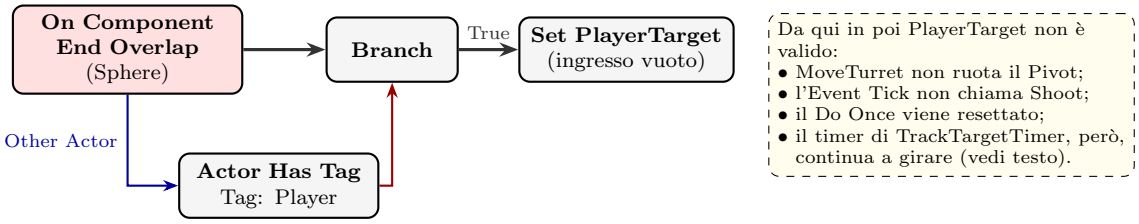


Figure 1.5.7: L'uscita dal raggio: se chi esce dalla sfera è il player, si svuota PlayerTarget. Tutti i controlli Is Valid del resto del grafo fanno il resto. Schema ricostruito.

Approfondimento: svuotare una variabile, e il timer. In un nodo *Set* di una referenza, lasciare il pin scollegato equivale ad assegnare **None**, il valore "nessun oggetto": Is Valid, su None, dà Is Not Valid. Il timer avviato da TrackTargetTimer nella quarta lezione, però, non viene fermato: continua a chiamare MoveTurret a ogni *frame*, e MoveTurret si limita a uscire subito dall'Is Valid. È innocuo, ma è lavoro inutile, e al rientro del player il Begin Overlap ne avvia un altro (Set Timer by Event con lo stesso evento sostituisce il timer precedente, quindi non si accumulano). Il modo completo è salvare il **Return Value** di Set Timer by Event in una variabile *Timer Handle* e, nell'End Overlap, chiamare **Clear and Invalidate Timer by Handle** oltre a svuotare PlayerTarget.

Che ne sarà della torretta. "Sappiate che la torretta la riuserete, perché più in là farete delle lezioni con **Davide**", sui **livelli**, e "avrete l'opportunità di usarla per fare un livellino": "un possibile opponente, un possibile ostacolo".

1.5.18 L'esercizio della matrice di cubi: il For Loop

"Abbiamo 20 minuti. Dobbiamo fare anche un altro esercizio: lo iniziamo oggi, lo finiamo la prossima volta." Il docente mostra il risultato finale, già pronto: una **matrice di cubi**, "perfettamente allineati e spaziati tra di loro", generata da un actor vuoto "usando la matematica e un ciclo *for*".

L'actor e l'evento. Un nuovo Blueprint Class di tipo Actor (il nome, nella trascrizione, è "pp spawn più x", qualcosa come *BP_SpawnX*), con subito un Custom Event **SpawnerRow** e, collegato, un **For Loop** ("non For Each: For Loop"), dalla categoria *Flow Control*. **First Index** 1, **Last Index** 10, "poi magari li cambieremo".

Dall'indice alla posizione. "Andiamo a prendere l'**Index** e lo moltiplichiamo per un tot": per 100. A ogni giro l'indice cresce, "1 per 100 diventa 100, 2 per 100 diventa 200, eccetera". Questo numero si somma alla posizione dell'actor: **Get Actor Location**, splittata, e la sua **X** più l'indice per 100. Il docente lo fa vedere a mano nei dettagli dell'actor: "io adesso sto a 1975; ci aggiungo 100, quindi sarà 2075: vedete che si muove di 100"; con l'indice 2, "1975 più 200 sarà 2175". "Moltiplicando questo valore andremo a creare varie somme che sono sequenziali." Con un **Make Vector** si ricompone il punto: la X nuova, la Y e la Z dell'actor, invariate (figura 1.5.8).

Vedere i punti: un Line Trace. Per ora non si spawna niente: si *visualizzano* le posizioni con un **Line Trace by Channel** ("questa sarà una linea; prima era una sfera"), il cui **Start** è il Make Vector. Per l'**End** serve un vettore che vada *giù*: il docente prova prima a togliere 200 alla Z, poi usa la rotazione dell'actor. **Get Actor Rotation**, da cui **Get Up Vector** ("il vettore che va su"), moltiplicato per −5000: "lo inverti

moltiplicandolo per meno 5000; non per meno 1, perché meno 1 è solo più piccolo". Lo si somma al Make Vector e lo si mette nell'End. Draw Debug Type For One Frame, e SpawnerRow chiamato dall'Event Tick. In Play: "guardate che abbiamo creato dei line trace perfettamente spaziati di 100 con un ciclo for". "Abbiamo già le posizioni dove potremmo spawnare eventualmente i nostri cubetti."

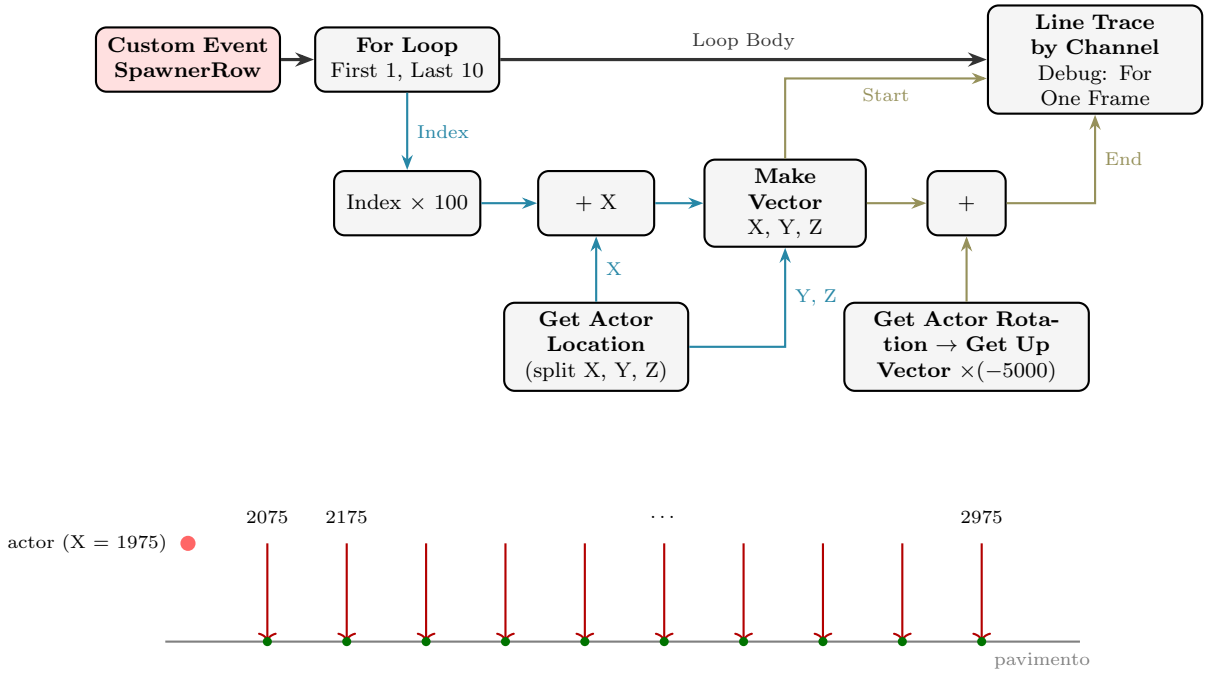


Figure 1.5.8: L'esercizio della matrice, prima parte: una riga. Il For Loop conta da 1 a 10; ogni indice, moltiplicato per 100 e sommato alla X dell'actor, dà un punto; da quel punto parte un line trace verso il basso (l'Up Vector dell'actor invertito e allungato). Sotto, il risultato visto di lato, con le X della lezione: dieci linee distanziate di 100 unità. La prossima volta, al posto dei trace, i cubi, e un secondo ciclo per le colonne. Schema ricostruito.

I problemi. "A me le linee sono oblique": bisogna usare la rotazione dell'actor e il suo Up Vector, non un vettore qualunque. "A me vanno verso l'alto": la studentessa aveva ruotato l'actor; "non lo devi ruotare". "A me non si vedono proprio": l'actor era troppo vicino al pavimento, "spostalo un po' più in alto" e guarda la scena dall'alto; "l'importante è che vedi che i puntini sono spaziati tra di loro".

Perché proprio dieci. "Una cosa che non ho capito: perché ne spawnano specificamente 10?" "Perché abbiamo l'indice a 10." Il For Loop parte dal **First Index**, 1, e "ogni volta che fa questa esecuzione, questo index aumenta di 1: farà 1, 2, 3". "Quando arriva al **Last Index**, 10, si ferma." Il docente mette Last Index a 20 e conta le linee: "sono più di 10, e sono 20". "Lo farà 20 volte, fondamentalmente." La prossima volta: "le righe, e per ogni riga andremo a spawnare di nuovo altri elementi", cioè un ciclo dentro l'altro.

Approfondimento: la matrice completa. Romero e Sewell (capitolo 19) costruiscono esattamente questa griglia nel *Construction Script*, con due For Loop **annidati**: il ciclo esterno conta le righe e calcola la X (indice × spaziatura tra le righe); il ciclo interno, eseguito per intero a ogni giro dell'esterno, conta gli elementi della riga e calcola la Y (indice × spaziatura tra gli elementi). Con r righe e c colonne il corpo interno si esegue $r \times c$ volte, e la posizione dell'elemento (i, j) è

$$P_{ij} = P_0 + (i \cdot d_x,\ j \cdot d_y,\ 0),$$

dove P_0 è la posizione dell'actor e d_x, d_y le spaziature. Due differenze rispetto alla lezione. La prima: il libro non spawna un actor per cubo, ma aggiunge *istanze* a un **Instanced Static Mesh Component** (nodo *Add*

Instance), che disegna centinaia di copie della stessa *mesh* con un costo minimo: è un parente del *pooling* visto a inizio lezione. La seconda: nel Construction Script la griglia si aggiorna già nell'editor, cambiando righe e colonne nei dettagli; chiamare la generazione dall'Event Tick, come per i trace di prova, va bene solo per il debug, perché spawnare cubi a ogni *frame* ne creerebbe migliaia.

1.5.19 La torretta che mira ai piedi, e la chiusura

"Mancano 5 minuti." Una curiosità di uno studente, che aveva fatto muovere la canna anche in alto e in basso: "quando mi allontano, mira i piedi; quando mi avvicino, il torso". Come si mira più in alto?

Due soluzioni. Il docente comincia a mostrarne una: invece di mirare all'actor del player, aggiungere al Third Person Character un *Scene Component* vuoto, "attaccato alla mesh", sulla testa o sul collo, e mirare a quello ("mi sa che ti tocca fare una referenza" del tipo giusto, per poterne leggere i componenti). Ma uno studente propone una soluzione più semplice, che aveva già trovato per lo stesso problema: "Find Relative Look at Rotation utilizza il pivot della **base**, non della GAN". Quindi ha misurato la differenza d'altezza tra il pivot della canna e quello della base, e ha "detto: prendi la location dell'actor e **abbassala**, nel mio caso di 100". "Ottima trovata. Bravo." Con una precisazione sugli assi: "per la rotation è la **Y**" (il *pitch*), "ma per la location è la **Z**" (l'altezza).

Approfondimento: perché la mira cambia con la distanza. L'angolo di *pitch* calcolato da Find (Relative) Look at Rotation è quello della retta che va dal punto di partenza al bersaglio. Se il punto di partenza è la base, più in basso della canna di un'altezza h, e il bersaglio è a distanza orizzontale d, la canna viene inclinata dell'angolo giusto per la base, non per sé stessa: l'errore è circa $\arctan(h/d)$, grande quando il player è vicino e piccolo quando è lontano. Per questo la mira "scivola" con la distanza. Le correzioni sono equivalenti nella sostanza: far partire il calcolo dalla posizione della canna (il *transform* del Pivot o di TraceSpawn), oppure spostare il punto di partenza o il bersaglio della stessa differenza d'altezza, come ha fatto lo studente. Mirare a un punto del corpo (la testa, il busto) invece che all'origine dell'actor risolve il problema opposto: l'origine del Third Person Character è al centro della capsula, a metà altezza.

La chiusura. "Ottimo, perfetto, ragazzi. Ci sentiamo la prossima volta. Qualsiasi domanda in mente, potete farla tranquillamente sul **Discord**." Uno studente chiede di ripartire, la volta dopo, dalla matrice: "mi funziona tutto, ma non ho capito il ragionamento logico dietro certi nodi". "Tanto la prossima volta sicuramente partiremo un attimo dall'inizio, perché abbiamo fatto lo spawn, poi dobbiamo fare subito un altro argomento, e diciamo che a fine lezione la mente fresca viene meglio." L'appuntamento è per **martedì**. "Dai, ragazzi, buona serata."

Punti chiave del capitolo

- Per non perdere il lavoro, salvare tutto dall'**indicatore delle modifiche non salvate** in basso a destra. Un materiale nero ha perso la sua texture. In un materiale, **Multiply** tinge una texture, **Lerp** sceglie tra due ingressi.
- **Spawnare** è allocare in memoria un oggetto e farlo apparire nel mondo, in un punto preciso (il punto di spawn, il falò). Costa: va fatto con parsimonia. Il **pooling** ricicla un insieme fisso di oggetti; il **destroy** li restituisce al **garbage collector**.
- **Spawn Actor from Class** vuole la **classe**, lo **Spawn Transform** e il Collision Handling Override, e restituisce la referenza (pin blu). **Destroy Actor** vuole solo il Target.
- Un **ciclo** ripete un blocco di codice finché una condizione è vera; il **For Loop** conta da First Index a Last Index, e i cicli si possono annidare (righe e colonne).

Sintomo	**Causa e rimedio**
Materiali neri dopo la riapertura	la texture del materiale manca: riassegnarla (e salvare tutto dall'indicatore in basso a destra)
La torretta guarda di lato se ruotata nel livello	Find Look at Rotation lavora nel mondo: usare Find Relative Look at Rotation con Get Actor Transform
La torretta punta dall'altra parte	la *mesh* del pivot non guarda lungo X: reimportarla o correggerne il pivot (Modeling, XForm, Edit Pivot, gizmo in Local)
Il trace non si vede	Draw Debug Type su None; Radius a 0; Shoot non chiamato dall'Event Tick
Il trace non ruota con il cannone	Get Actor Forward Vector (della base) invece del Forward Vector di TraceSpawn; TraceSpawn non figlio del Cannon; rotazione residua su TraceSpawn
Il trace non diventa verde sul player	il player non risponde Block al canale: Collision Presets Custom, DetectionTurret su Block; compilare prima
Il canale custom non compare nel nodo	compilare il Blueprint
Il Multiply non accetta 800	il secondo pin è un vettore: tasto destro, convertirlo in *float*
Il proiettile cade	Projectile Gravity Scale ancora a 1
Il proiettile attraversa tutto	Radius del suo trace a 0; canale sbagliato (Visibility non vede il player)
Il proiettile sparisce appena nato	un componente della torretta (Sphere, Pivot, Base, Cannon) o il proiettile stesso non ignora ProjectileTrace
Spawn di un proiettile per *frame*	manca il *fire rate*: Delay (e Do Once per il primo colpo)
Il primo colpo arriva in ritardo	solo il Delay: aggiungere Do Once con Reset sull'Is Not Valid
La torretta continua dopo l'uscita	manca On Component End Overlap che svuota PlayerTarget
L'Event Tick non scatta	caso irrisolto a lezione: controllare Start with Tick Enabled, Set Actor Tick Enabled, Tick Interval, l'istanza in scena
Le linee della matrice sono oblique o vanno su	usare Get Up Vector della rotazione dell'actor, moltiplicato per un valore negativo; non ruotare l'actor
La canna mira ai piedi da lontano	il calcolo parte dalla base: abbassare la location di partenza (o partire dalla canna)

Table 1.5.4: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo o componente	**Che cosa fa**
Spawn Actor from Class	crea un actor di una classe con un *transform*; restituisce la referenza
Destroy Actor	distrugge un actor (Target)
Find Relative Look at Rotation	la rotazione per guardare un punto, nello spazio di un *transform* di partenza
Get Actor Transform / Get World Transform	il *transform* (location, rotation, scale) dell'actor o di un componente nel mondo
Get World Location	la posizione di un componente nel mondo
Get (Actor) Forward / Right / Up Vector	i vettori di direzione, lunghi 1, dell'actor o del componente; per -1 si invertono
Scene Component	un punto vuoto nella gerarchia: punto di partenza per trace e spawn
Line / Sphere Trace by Channel	test di collisione lungo un segmento (o con una sfera) su un canale; Return Value vero se bloccato
Multi Sphere Trace by Channel	come sopra, ma restituisce tutti gli *hit*
Trace Channel (Project Settings)	un canale di collisione custom, con una risposta di default
Projectile Movement Component	muove un actor come un proiettile (Initial e Max Speed, Gravity Scale, Velocity)
Branch	sceglie un ramo in base a un booleano (scorciatoia: B + clic)
Select	restituisce una delle opzioni in base a un indice (booleano, intero, enumerazione)
Random Bool	un booleano casuale
Do Once (Reset)	lascia passare l'esecuzione una volta, finché non viene resettato
Sequence	esegue in ordine le sue uscite Then 0, Then 1, ...
Delay	aspetta un tempo prima di proseguire (ignora le chiamate mentre conta)
On Component End Overlap	scatta quando qualcosa esce dal volume del componente
For Loop	ripete il Loop Body da First Index a Last Index; Index è il contatore
Make Vector	costruisce un vettore da X, Y, Z
Breakpoint (F9)	ferma l'esecuzione su un nodo, per il debug

Table 1.5.5: I nodi e i componenti della quinta lezione.

- **Find Relative Look at Rotation** (Start Transform: Get Actor Transform) calcola la rotazione nello spazio della torretta: la si può ruotare nel livello senza offset. Delta Time da **Get World Delta Seconds**, Interp Speed 2.
- Un **trace** è una linea (o una sfera) immaginaria che cerca collisioni lungo un segmento: serve a "vedere" (linee di vista, rilevamento), ed è costoso. Draw Debug Type **For One Frame** lo mostra: rosso, e verde dal punto in cui viene bloccato.
- Lo **Start** è la World Location di un **Scene Component** (TraceSpawn) figlio del cannone; l'**End** è Start + **Forward Vector** del componente × 800. Il forward vector è lungo 1, e un vettore moltiplicato è uno spostamento: va **sommato** alla posizione.
- I **trace channel** custom si creano in Project Settings → Collision. **DetectionTurret**: default Ignore, Block sul player. **ProjectileTrace**: default Block, Ignore sulla torretta e sui proiettili. Le risposte sono **Ignore**, **Overlap** (si attraversano, con un segnale) e **Block** (si fermano).
- Il proiettile è un actor a parte (**due actor, due compiti**): una sfera e un **Projectile Movement** (Initial e Max Speed 1000, gravità 0). Uno Sphere Trace sul Tick, con Start ed End coincidenti, lo distrugge quando tocca qualcosa.
- Lo sparo: Event Tick → Is Valid → Shoot → trace → Branch → **Do Once** (primo colpo subito) e **Delay** (**FireRate**, 0,5 s) → SpawnProjectile, che sceglie una delle **due bocche** con **Select** e **Random Bool** e usa il loro **Get World Transform**.
- **Sphere Trace** restituisce un *hit*, **Multi Sphere Trace** tutti; il Multi serve solo se servono più risultati.
- **On Component End Overlap** svuota **PlayerTarget** (un Set senza valore): gli Is Valid fermano rotazione e sparo e resettano il Do Once.
- L'esercizio della **matrice**: For Loop da 1 a 10, Index × 100 sommato alla X dell'actor, **Make Vector**, e un **Line Trace** verso il basso (Up Vector × − 5000) per vedere i punti. La prossima volta: i cubi e le righe.
- Per il *debug*: **Print String** sui rami, **breakpoint** con **F9**, e un Blueprint di prova per isolare il problema.

Chapter 1.6

Collisioni, timeline e audio: la piattaforma che sale

Questa è la **sesta lezione** del corso breve di **introduzione a Unreal Engine**. Ha due compiti. Il primo è chiudere un conto aperto: la **matrice di cubi** cominciata negli ultimi venti minuti della quinta lezione, che si era fermata a una sola riga di *line trace* "perfettamente spaziati di 100". Oggi al posto delle linee arrivano i cubi, e alla riga si aggiunge una seconda dimensione. Il secondo è un esercizio nuovo, "una piccola **piattaformina** che si muove": una piattaforma che, quando il giocatore ci sale sopra, **sale** fino a un punto preciso, e quando il giocatore scende **torna giù**, accompagnata da un piccolo **suono** in loop. Per costruirla servono tre argomenti, che danno il titolo al capitolo: le **collisioni** (in particolare l'*overlap*, il "volume invisibile" che si accorge del giocatore senza fermarlo), le **timeline** ("delle linee del tempo", "molto interessanti e si usano moltissimo") con l'**interpolazione lineare**, e un primo assaggio di **audio** con le *Sound Cue*.

La lezione è **online**, su Zoom, come le precedenti. Si apre con la consueta attesa di "5 minuti" e con una notizia pratica: le registrazioni delle lezioni resteranno disponibili **fino a settembre**. La prima parte (fino alle 17:37 circa) contiene un breve ripasso teorico (il For Loop, le collisioni, le timeline), una dimostrazione delle collisioni, la matrice completata e il primo metodo per muovere la piattaforma; poi una pausa di una ventina di minuti, "la pausa classica"; la seconda parte riprende verso le 18 con il *play rate*, il secondo metodo (quello "giusto"), e l'audio, e si chiude poco prima delle 19: "vi mando a cena". Come sempre, molto tempo va nella risoluzione dei problemi degli studenti, a voce, in condivisione dello schermo o con le foto del codice mandate in chat.

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "a Real", "Arreal", "la Real" e "allengine"; lo *spawn* è "spon", "sponare", "spunare" e "esponare"; l'**Event Tick** è "il TIC" (e una volta "il clic"); **WorldDynamic** è "Word Dynamic" e "wall dynamic"; **Physics Body** è "Physic Body" e "fixy physics body"; **On Component Begin Overlap** è "on component.ito.beghin overlap" e "un beghino per l'app"; **Get Actor Location** è "getActLocation", "gate application" e "chitacture"; lo **Split Struct Pin** è "split strat pin"; il **Break Hit Result** è "break-it result"; il **Lerp** è "LERP", "l'ERP", "Alper" e "ARP", e il **Lerp (Vector)** è "v-lerp" e "BLERP"; la variabile **MaxHeight** è "max height", "Maxite", "Maximaxite" e "map site"; il **grafico** della timeline è "il graffo" (il docente stesso si corregge: "il graffo è il grafico"); la **Sound Cue** è "la queue", "la Q" e "create queue"; i cubi sono "Fubis" e "Qs"; il componente **Billboard** è "il dinosauro" (per via della sua icona). Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione e durante la pausa il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("grazie a tutti per l'attenzione che ci ha spiegato", "la domanda è chi vuole intervenire in questo senso", "qual è la risposta che ci può essere?", "il governo di Dalai Lama", "la situazione in cui sono andata a finire la cena"): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è una dimostrazione dal vivo nell'editor dei Blueprint, costruita a voce ("tirate fuori", "collegate", "promote to variable"). Il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei pin di Unreal Engine 5; dove la trascrizione lascia un buco, lo dichiara.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono le tre risposte di collisione, il grafo della matrice con i due cicli, i componenti della piattaforma, il nodo Timeline e la sua curva, l'alfa dell'interpolazione, i due metodi per muovere la piattaforma, la Sound Cue e il grafo del suono. Le tabelle riassumono come due oggetti decidono la loro collisione, i pin della timeline, gli errori più comuni incontrati dagli studenti e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'attesa, il programma e il ripasso del **For Loop** (sezioni 1.6.1 e 1.6.2);
2. le **collisioni**: Ignore, Overlap, Block, Object Type, gli eventi di overlap, la domanda sui trace e la dimostrazione (sezioni da 1.6.3 a 1.6.5);
3. la **matrice di cubi** completata: lo spawn dei cubi, la seconda riga, il Tick spento, i problemi (sezioni da 1.6.6 a 1.6.8);
4. la **piattaforma**: i componenti e il controllo sul player (sezione 1.6.9);
5. le **timeline** e l'**interpolazione lineare** (sezioni 1.6.10 e 1.6.11);
6. il **primo metodo**, la pausa, il **play rate** (sezioni da 1.6.12 a 1.6.14);
7. il **secondo metodo**, con il Billboard e le posizioni salvate, e il ritorno con Reverse (sezioni 1.6.15 e 1.6.16);
8. l'**audio**: la Sound Cue, lo spawn del suono, lo stop e il pitch che cambia con la direzione; la chiusura (sezioni da 1.6.17 a 1.6.20).

1.6.1 L'attesa e il programma

"Allora, ragazzi, tutto a posto?" La domanda di rito, "la solita domandina", è se qualcuno ha sperimentato per conto suo. Uno studente risponde che, "oltre a farlo mirare anche sulla Y" (la torretta della volta scorsa, con la canna che si muove anche in verticale), non ha fatto altro.

Il pop-up di Epic Games Launcher. Una domanda pratica, prima di cominciare: "ogni volta che apro Epic Games Launcher mi arriva una schermata che mi dice che *project files are not associated with this launcher*, tutte le volte". Il docente ammette di non avere una risposta: "lo sta facendo continuamente anche a me". La soluzione di entrambi è la stessa: cliccare **Fix Now**, e il launcher "rialloca" i file. "Dagli ultimi aggiornamenti sembra che abbia fondamentalmente smosso qualcosa." Ma niente panico: "la tua cartella è al sicuro". Il docente promette di cercare una soluzione online; "se fosse una cosa brevissima, l'avremmo già sistemata".

Approfondimento: che cosa fa il Fix Now. Il launcher tiene un elenco delle installazioni dell'engine e associa i file `.uproject` (le icone, il doppio clic, il menù contestuale) alla versione installata. Quando l'associazione si rompe (dopo un aggiornamento del launcher o dell'engine, o con più versioni installate) il launcher propone di ripararla; il *fix* riscrive l'associazione nel sistema operativo, non tocca il contenuto dei progetti. Per questo il messaggio è fastidioso ma innocuo, come dice il docente.

Il file audio. Su Discord, nel server di Hive, nel canale testuale del materiale didattico del corso di introduzione a Unreal (nella trascrizione "intro ad UE materiale didattico" e "intro 2 materiale didattico"), l'ultimo messaggio contiene un piccolo file audio da scaricare: "oggi andremo a fare un pelino di audio" per l'esercizio della piattaforma. Si scarica dai tre puntini in alto a destra del messaggio. Il docente sottolinea il formato: Unreal accetta file **WAV**, "non MP3, come i brani musicali classici". Il file si chiama, più o meno, *SFX Moving Platform* (la trascrizione lo chiama anche "SFX Platform Moving").

Le registrazioni. "Ho avuto informazione per quanto riguarda le registrazioni delle lezioni", anche per chi segue in differita: resteranno disponibili **fino a settembre**, per ripassare o per recuperare. "Offerta limitata fino a settembre."

Il Nintendo Direct. La lezione cade in contemporanea con un *Nintendo Direct*. "Vi autorizzo a metterlo sul secondo schermo, così poi mi dite mentre faccio lezione se è uscita la bomba." Le notizie arriveranno nella pausa (sezione 1.6.13).

Il programma. "Aprite il progetto di Unreal in background, perché facciamo la solita prima spiegazione e poi passiamo direttamente all'engine." Il piano è: finire l'esercizio della volta scorsa (la matrice), poi cominciare quello nuovo, la piattaforma che si muove, con un argomento nuovo, le **timeline**. La parte teorica, avverte il docente, è breve: "sempre meglio mettere in pratica le cose".

1.6.2 Il ripasso: il For Loop

La prima slide riprende "l'argomento della scorsa volta", perché "siamo arrivati verso la fine, e capisco che mettere due argomenti insieme è un po' stancante". Un **ciclo** è "un'azione, a livello di programmazione, **iterativa**", fatta "un numero tot di volte" secondo una condizione. Il **For Loop** ha due ingressi:

- il **First Index**, "il primo indice da cui devo partire", nell'esempio 1;
- il **Last Index**, "l'ultimo indice", nell'esempio 10.

A ogni giro il ciclo esegue l'azione collegata al **Loop Body** ("stampami delle frasi") e aumenta l'indice di uno: "lo farà 1, 2, 3, 4, 5, 6, 7, 8, 9, fino a 10". L'esempio di gioco: spawnare dei nemici finché il loro numero è inferiore a 5; "controllo ogni volta: sponami fino a che non arrivi a 5. Perfetto, sono arrivato a 5, fermati". E, come già detto la volta scorsa, i cicli si possono **annidare**, "un ciclo dentro un ciclo, ma questi sono casi particolari": sarà proprio il caso della matrice, poche sezioni più avanti.

Approfondimento: For Loop, While Loop e il nemico che "arriva a 5". L'esempio dei nemici, a rigore, descrive un ciclo con una *condizione* più che un ciclo che *conta*. Nei Blueprint (Romero e Sewell, capitolo 13, "Flow Control") il **For Loop** esegue il corpo per ogni intero da First Index a Last Index, estremi *inclusi* (da 1 a 10 sono dieci giri; da 0 a 9, la convenzione dei programmatori, anche); il **While Loop** invece ripete finché una condizione booleana resta vera, e serve quando non si sa in anticipo quante volte ripetere. Tutti i cicli dei Blueprint si eseguono per intero *nello stesso frame*: se la condizione di un While Loop non diventa mai falsa, l'editor interrompe l'esecuzione con un errore di ciclo infinito. Per "spawnare un nemico ogni tanto finché sono meno di 5" si usa invece un evento ripetuto nel tempo (un timer, o un Delay) con un Branch sul conteggio.

1.6.3 Le collisioni: Ignore, Overlap e Block

"Poi invece oggi andremo a vedere le **collisioni**, perché è una cosa molto importante, che in verità abbiamo già usato." Le risposte di collisione sono di **tre tipi** (figura 1.6.1).

- **Ignore**: "non esiste collisione, fondamentalmente; non c'è una collisione né fisica né di *detection*". "L'oggetto viene proprio ignorato sia a livello fisico sia a livello di engine, quindi non succede letteralmente nulla."
- **Overlap**: "non c'è una collisione fisica, ma c'è una detection". Unreal "notifica all'oggetto" che è avvenuto un incontro con un altro oggetto: "è solamente un check, controllami, però a livello fisico i due oggetti si trapassano".
- **Block**: "presenta sia la collisione fisica sia la collisione di controllo"; gli oggetti si fermano l'uno contro l'altro, e l'engine se ne accorge.

A che cosa serve l'overlap. "Vi è mai capitato in un gioco che parte una *cutscene* e dite: oh mio Dio, è partita una cutscene, però qual è la cosa che l'ha attivata?" (la trascrizione dice "cazzin"). "Probabilmente è un **volume invisibile**, che noi non vediamo perché è solamente per i developer." Il volume ha fatto overlap, "perché se no ci avrebbe bloccato: sarebbe stato tipo un muro di gioco, non saremmo potuti andare avanti". Però "ci è servito per dire: parti adesso con la cinematica". I volumi invisibili, conclude il docente, "sono stra-usati, proprio dall'alba dei tempi, e si basano sul concetto di overlapping". È esattamente quello che servirà per la piattaforma: un volume sopra di essa che si accorge del giocatore.

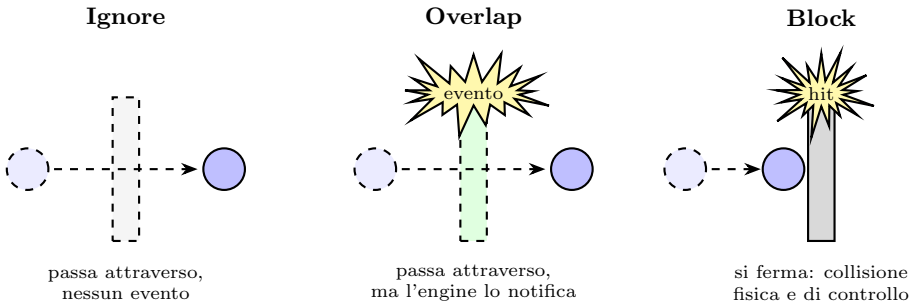


Figure 1.6.1: Le tre risposte di collisione, come nell'immagine della slide (la palla che attraversa o no il muro). Con Ignore la palla passa e non succede nulla; con Overlap passa, ma l'engine segnala l'incontro (gli eventi Begin ed End Overlap); con Block si ferma. Schema ricostruito.

Il pannello delle collisioni. Nei dettagli di ogni componente con una collisione (una *static mesh*, una *box*, una sfera) c'è la sezione **Collision**. Il docente ne elenca le voci principali:

- i **Collision Presets**, cioè configurazioni già pronte, "che possono essere *custom* oppure *default*, o *BlockAll*, *OverlapAll*, eccetera";
- **Collision Enabled**, "fondamentalmente uno switch on-off" delle collisioni;
- l'**Object Type**, che "definisce la tipologia di oggetto";
- le **Object Responses**, cioè la tabella con le tre colonne *Ignore*, *Overlap* e *Block*, una riga per ogni tipo di oggetto: *WorldStatic*, *WorldDynamic*, *Pawn*, *PhysicsBody* e così via (e, sopra, le *Trace Responses*, Visibility e Camera, viste la volta scorsa).

Perché dare un tipo a un oggetto. "A cosa serve dare un Object Type a un oggetto? Serve per capire come gli altri oggetti interagiscono con quella **categoria**." Se voglio che un oggetto ignori tutti gli oggetti di un certo tipo, per esempio tutti i *WorldDynamic*, "metterò il tick su Ignore" nella riga WorldDynamic delle sue Object Responses: "tutti gli oggetti WorldDynamic saranno ignorati a livello di collisioni".

Gli eventi di collisione. L'overlap "viene usato tantissimo per avere degli eventi". Gli eventi sono due coppie.

- A livello di **actor**: **Event ActorBeginOverlap** ed **Event ActorEndOverlap**, che scattano quando un qualunque componente dell'actor inizia o finisce di sovrapporsi a qualcosa.
- A livello di **componente**: **On Component Begin Overlap** e **On Component End Overlap**, "perché solamente nel caso overlappa quel componente": per esempio una *Box Collision* dentro l'actor. Sono quelli già usati per il raggio della torretta.

L'evento di componente restituisce, ricorda il docente, il componente che ha fatto overlap (*Overlapped Component*), l'actor che lo ha provocato (*Other Actor*) e il suo componente (*Other Comp*), più alcuni dati sull'incontro. E "funziona solamente in entrata e in uscita": il Begin scatta una volta quando si entra,

l'End una volta quando si esce, non a ogni frame in cui si resta dentro. Infine, il **Branch** ("l'IF, per chi sa di programmazione", un vero/falso), già noto, e l'argomento "più interessante di oggi", le **timeline**, che il docente mostra in slide solo di sfuggita (sezione 1.6.10) prima di passare all'engine.

Approfondimento: come si decide la collisione fra due oggetti. La tabella delle risposte è *per oggetto*: ogni componente dichiara come risponde a ciascun Object Type (e a ciascun trace channel). Quando due componenti si incontrano, Unreal guarda la risposta di *entrambi* all'Object Type dell'altro e applica la **meno restrittiva** delle due, nell'ordine Ignore < Overlap < Block (tabella 1.6.1). C'è collisione fisica solo se tutti e due rispondono Block; c'è overlap se almeno uno dei due risponde Overlap e nessuno Ignore; se uno dei due risponde Ignore, non succede nulla, qualunque cosa dica l'altro. Due dettagli che fanno perdere ore. Primo: gli eventi di overlap partono solo se *entrambi* i componenti hanno attiva l'opzione **Generate Overlap Events**. Secondo: gli eventi di *hit* (*Event Hit*, quelli che restituiscono il punto d'impatto, usati da Romero e Sewell nel capitolo 6 per il proiettile che colpisce il bersaglio) richiedono una risposta Block e, per un oggetto con la fisica, l'opzione **Simulation Generates Hit Events**. Il campo **Collision Enabled**, che il docente descrive come un on-off, ha in realtà più posizioni: *No Collision*, *Query Only* (solo trace e overlap, senza fisica), *Physics Only* (solo fisica, senza trace né overlap) e *Collision Enabled* (entrambe). Romero e Sewell (capitolo 14) riassumono così: gli Object Type predefiniti sono WorldStatic, WorldDynamic, Pawn, PhysicsBody, Vehicle e Destructible; i trace channel predefiniti sono Visibility e Camera; e altri canali e tipi si creano in *Project Settings* → *Engine* → *Collision*.

Risposta di A al tipo di B	Risposta di B al tipo di A	Risultato
Block	Block	collisione fisica (eventi di *hit*)
Block	Overlap	overlap (eventi di overlap)
Overlap	Overlap	overlap (eventi di overlap)
Ignore	qualunque	niente

Table 1.6.1: Come due componenti decidono la loro collisione: vince la risposta meno restrittiva (Ignore < Overlap < Block). Gli eventi di overlap richiedono inoltre Generate Overlap Events attivo su entrambi. Integrazione, non detta in questa forma a lezione, ma è la regola dietro alla dimostrazione di sezione 1.6.5.

1.6.4 Una domanda sui trace: quanti canali, e quando

Prima di passare all'engine, uno studente torna sui trace della volta scorsa: "i **trace channel**, solitamente in un gioco non tripla A, in un gioco normale, quanti se ne creano? Si cerca di ottimizzare, o se ne fa uno per oggetto?"

Pochi, e per azioni precise. "I trace dipendono sempre da che cosa vuoi fare." Ma una regola c'è: il trace "non dovrebbe essere usato in modo troppo eccessivo in generale, perché è anche abbastanza pesante". "Di solito viene fatto per **azioni molto precise**."

L'esempio del salto. Il docente racconta un suo progetto personale, "un giochino con una palla". "I platformer, ad esempio, cosa fanno?" Quando il personaggio salta e sta tornando giù, "arriva vicino al suolo, ma non attaccato al suolo: magari qualche centimetro, qualche unità in più", e il gioco gli permette già di **risaltare**. "È come se tu stessi anticipando, ancora prima che lui arrivi a terra, la possibilità di saltare di nuovo." In design, dice il docente, questa tolleranza ha un nome (nella trascrizione "Grazia", quasi certamente l'inglese *grace*, cioè un margine di tolleranza). Per farla serve un trace: "ogni volta che salta vado a vedere: ok, adesso puoi saltare, anche se non sei neanche arrivato a terra". Il trace è giustificato perché "è un'azione sul player", e "sul player è molto importante che quelle azioni siano precise".

Il programmatore, il C++ e gli oggetti isolati. "Molto spesso, quando vi riempite di trace, va ad appesantire", e lì interviene il **programmatore**, che in **C++** può controllare più cose e ottimizzare meglio di quanto permettano i Blueprint. Un consiglio di architettura, "importante": gli oggetti di un

gioco "dovrebbero cercare di essere il più **isolati** possibile". Devono comunicare, "dovranno essere due interlocutori", ma non "un pezzo di uno che sta dentro il pezzo di un altro e si mischiano", perché "sennò è uno *spaghetti code*, un casino incredibile". Riassunto del docente: "non esagerare con i trace; dipende dai casi; avere oggetti isolati".

Spegnere il Tick. Un ultimo trucco, che tornerà utile tra poco: "potete disattivare o attivare il Tick di ogni actor". Se un trace gira sull'Event Tick solo per fare un'azione, "come ho finito l'azione, disattivo il Tick, fine: così almeno non fa più il trace".

Approfondimento: *jump buffering* e *coyote time*. Nel game feel dei platformer ci sono due tolleranze classiche, spesso confuse. Il **jump buffering** ("buffer di salto") è quella descritta dal docente: se il giocatore preme salto poco *prima* di toccare terra, il gioco ricorda la pressione e fa saltare il personaggio appena atterra (o, con un trace verso il basso, appena il suolo è entro pochi centimetri). Il **coyote time**, dal coyote dei cartoni che corre oltre il bordo del burrone prima di cadere, è la tolleranza opposta: per qualche frame *dopo* aver lasciato una piattaforma, il giocatore può ancora saltare come se fosse a terra. Entrambe esistono perché il giocatore percepisce il salto come "giusto" anche quando, a rigore, è in ritardo o in anticipo di pochi centesimi di secondo; senza, i controlli sembrano "mangiare" gli input. Il *Character Movement Component* di Unreal include già alcune tolleranze di questo tipo (per esempio il numero massimo di salti e la durata della pressione del tasto), ma il buffer e il coyote time veri e propri si programmano di solito a mano, con un trace o un timer, come fa il docente.

1.6.5 La dimostrazione: la palla e il cubo

"Questo non serve neanche che lo facciate." Nella scena del docente ci sono una **palla** con la fisica attiva ("come abbiamo visto anche nella prima lezione") e un **cubo** sulla sua strada. Nei dettagli, sotto Collision, il cubo è di tipo **WorldDynamic**; la palla, inizialmente scambiata per WorldDynamic anche lei, è in realtà un **PhysicsBody**.

1. **Block.** La palla ha Block nella riga WorldDynamic: rotola contro il cubo e "ovviamente verrà bloccata".
2. **Overlap.** La risposta della palla a WorldDynamic diventa Overlap: la palla "ha completamente ignorato" il cubo, almeno in apparenza. Per far vedere che c'è qualcosa, il docente apre il Blueprint del cubo e aggiunge un evento di Begin Overlap ("ho un beghino per l'app" nella trascrizione) collegato a un **Print String**: "ha overlappato". In Play la palla attraversa il cubo e a schermo compare la scritta. "Come potete vedere, la collisione fisica non è presente assolutamente, ma in questo caso abbiamo un evento."
3. **Ignore.** Con Ignore, "l'oggetto se ne va; non ci dà messaggio, non ci dà niente".

"Queste fondamentalmente sono le collisioni, e come funzionano i tre canali principali."

La domanda sulla combinazione. Uno studente riassume la configurazione: "sulla sfera hai detto che WorldDynamic è Overlap, e sul muro, per Physics Body, metti Block?" "Esatto, va in contrasto." E quale delle due vince? Il docente risponde che ha vantaggio "quello che si muove", e che conviene "creare sempre un **canale a parte**". La risposta, a rigore, è un'altra (vince la risposta meno restrittiva, vedi la tabella 1.6.1): con Overlap da una parte e Block dall'altra il risultato è un overlap, indipendentemente da chi si muove, ed è proprio quello che la dimostrazione mostra.

Un Object Channel custom. Come la volta scorsa per i trace (i canali *DetectionTurret* e *ProjectileTrace*), anche i tipi di oggetto si creano in **Project Settings** → **Collision**, con *New Object Channel*. Il docente crea un tipo **Cube**, poi entra nel Blueprint del cubo ("mi sa che devo entrare dentro il Blueprint"), mette il preset su **Custom** e trova la nuova riga "Cube" nelle Object Responses. Il risultato non è immediato: prima il cubo grande blocca la palla, poi la ignora; il docente riallinea le risposte ("la risposta a se

stesso"), mette Overlap anche dall'altra parte, "così sono equivalenti", e il messaggio di overlap ricompare. "Eccolo. A posto." La trascrizione di questo passaggio è molto confusa: il senso è che, per ottenere un overlap fra due tipi, le risposte dei due oggetti vanno impostate in modo coerente.

Approfondimento: Object Channel o Trace Channel? Nella stessa pagina dei Project Settings ci sono due elenchi. Un **Object Channel** è un nuovo *tipo* di oggetto: compare nel menù Object Type di ogni componente e come riga nelle Object Responses di tutti gli altri. Un **Trace Channel** non è un tipo che un oggetto *è*, ma una domanda che un trace *fa*: compare solo nelle Trace Responses, e ogni oggetto dichiara se lo blocca, lo lascia passare segnalandolo o lo ignora. Per un gioco "non tripla A", la risposta pratica alla domanda dello studente è quindi: pochi trace channel, uno per ogni *domanda* diversa che il gioco fa al mondo (che cosa vede la torretta, che cosa colpisce il proiettile, dove posso spawnare), e pochi Object Channel, uno per ogni *categoria* di oggetti che deve comportarsi in modo diverso dalle altre; non uno per oggetto.

1.6.6 La matrice di cubi: dai trace ai cubi

"Oggi riandiamo a riprendere" l'esercizio della volta scorsa: "c'era un po' di confusione, per via del tempo". Il docente rifà la spiegazione da capo, perché uno studente aveva chiesto proprio questo.

Il codice della volta scorsa. Su un actor vuoto, chiamato dall'Event Tick, c'è un **Custom Event** con un **For Loop** da 1 a 10. "Partimi da uno; ogni volta che hai fatto questa azione, cioè il line trace, aumentami di uno: farà uno, due, tre, fino a dieci." L'**Index** si moltiplica per 100, "quindi 1 per 100 fa 100, 2 per 100 fa 200". Il docente lo spiega con una linea alla lavagna: "siamo al punto 0, dove il nostro actor è presente. Vogliamo muoverci sulla linea di un tot di unità, ma vogliamo che siano uguali tra di loro": segmenti tutti lunghi 100. "Sarebbero 100 centimetri, giusto?" chiede uno studente. "Sì, Unreal usa i centimetri, quindi è un metro." Tra 100 e 200 c'è 100, tra 200 e 300 c'è 100: "abbiamo proprio degli spazi **equidistanti**". L'indice per 100 è uno *spostamento*, e va sommato al punto di partenza: la **X** di **Get Actor Location** (splittata), più l'offset; con **Make Vector** si ricompone il punto (X nuova, Y e Z dell'actor). Da lì parte un **Line Trace by Channel**: Start è il punto, End è il punto più l'**Up Vector** dell'actor "invertito in un numero negativo" (per -5000), "e quindi ci punterà giù". Risultato: dieci linee verticali a 100, 200, ..., 1000 unità dall'actor.

Il Branch. "Vogliamo spawnare qualcosa quando il nostro trace va a collidere con qualcosa", in questo caso il terreno. Dopo il trace, un **Branch** ("B e tasto sinistro") sul **Return Value**: "mi hai preso qualcosa? Sì? Allora spawn".

Il cubo da spawnare: BP_Cube. Lo **Spawn Actor from Class** vuole una classe, "e possiamo sponare solamente classi". Quindi un nuovo Blueprint Class di tipo **Actor**, **BP_Cube** (il docente lo chiama BP_Cube1, perché il suo esiste già), con dentro solo un **Cube**. Le dimensioni consigliate: scala **0.5**, e il cubo alzato sulla Z perché "sarà un po' interrato": un cubo standard è alto 100 unità con l'origine al centro, scalato a 0.5 è alto 50, e la sua metà sono **25** ("25 sulla Z e 0.5 sulla scala"). Una studentessa trascina il cubo *sopra* il *DefaultSceneRoot*, sostituendolo: così il cubo diventa la radice e "ho solo scala", non si riesce più a spostarlo rispetto all'actor. Il consiglio è ricrearlo lasciando il cubo come figlio della radice (o spawnarlo un po' più in alto).

Spawn Actor from Class e il Break Hit Result. Nello spawner ("Spawner Cubes"), lo Spawn Actor from Class riceve la classe BP_Cube. "Quale *transform*?" Del *transform* (location, rotation, scale) serve solo la location, quindi sul pin **Spawn Transform** si fa **Split Struct Pin**, che lo separa in tre. La location viene dall'**Out Hit** del trace, scomposto con un **Break Hit Result**, "come l'altra volta". Il docente ne passa in rassegna i campi: il booleano **Blocking Hit** ("c'è un evento di hit che è avvenuto"), **Initial Overlap** ("è iniziato un overlap"), **Time**, **Distance**, **Location** ("la locazione in cui è avvenuto l'incontro", "l'interruzione del trace", dove la linea di debug passa dal rosso al verde) e **Impact Point**, **Normal** ("un vettore che hanno tutte le superfici, sempre verso l'esterno"). "Quello che ci interessa a noi è la location": va nello Spawn Transform Location.

I cubi che si spawnano all'infinito. In Play qualcosa va storto, e a un certo punto gli spawn si moltiplicano (accompagnati, nell'audio del docente, da un coro di suoni: "papà papà papà"). La causa: il trace gira sull'Event Tick, a ogni frame, e vede il cubo *appena spawnato*; il punto d'impatto si alza, e lo spawn "continuerà a sponarle". Le correzioni sono due.

- Sul nodo di spawn, il **Collision Handling Override** su **Always Spawn, Ignore Collisions**: "possiamo controllare le collisioni nel momento dello spawn", perché "possono succedere queste tipologie di cose".
- Soprattutto, sulla *mesh* del cubo dentro BP_Cube: Collision Presets su **Custom** ("fate Default e poi fate Custom") e, nelle **Trace Responses**, **Visibility** su **Ignore**. Il trace usa il canale Visibility; se il cubo lo ignora, il trace "non trova un'altra cosa" e non "spona, spona, spona". Il docente prova prima con No Collision, poi preferisce questa soluzione, "molto meglio".

"Adesso abbiamo i nostri bei cubetti tutti belli in fila, eccoli qua."

Approfondimento: Collision Handling Override. Il menù a tendina dello spawn decide che cosa fare se, nel punto di spawn, l'actor nuovo si sovrappone a qualcosa. *Default* usa l'impostazione della classe; *Always Spawn, Ignore Collisions* lo crea comunque, dove richiesto; *Try To Adjust Location, But Always Spawn* cerca un punto libero vicino e, se non lo trova, lo crea comunque; *Try To Adjust Location, Don't Spawn If Still Colliding* rinuncia se non trova spazio; *Do Not Spawn* rinuncia subito. Qui l'override non basta da solo perché il problema non è lo spawn in sé, ma il fatto che il *trace* successivo veda il cubo: è la risposta del cubo al canale Visibility a decidere.

1.6.7 La seconda riga: Sequence e un secondo For Loop

"Adesso andiamo un po' più avanti." Una fila non è una matrice: "essendo colonna e riga", serve un secondo ciclo che, invece della X, "influenzi la Y".

Il Sequence. Dopo il ramo True, il docente inserisce un **Sequence**: "un blocco che ci permette di dire: quando viene questa cosa, fai questo e questo e questo"; con *Add pin* si aggiungono tutte le uscite che si vogliono (Then 0, Then 1, Then 2...). Qui ne servono due: **Then 0** spawna il cubo "in colonna", quello del primo ciclo; **Then 1** fa partire il secondo ciclo, che spawna "la riga con i punti".

Il secondo For Loop. "For Each? No, scusate, For Loop; perdonate. For Each è un'altra tipologia di loop." Stessi indici, da 1 a 10. Il trace e lo spawn sono "la stessa identica cosa, ma cambiamo due pin": si copiano (Ctrl+C, Ctrl+V) e si modificano.

- L'Index del secondo ciclo, per 100, va sommato alla **Y** dell'actor, non alla X.
- La **X** invece deve essere quella del *primo* ciclo, cioè la somma "X dell'actor più indice esterno per 100": "la trasciniamo fino a qua". "Perché sarà la X100 alla Y100": ogni punto della colonna diventa l'origine di una riga. "Stiamo facendo una matrice."
- La Z resta quella dell'actor, e l'End è, come prima, il punto più l'Up Vector per -5000.

Il docente lo spiega con le etichette di una tabella: il cubo 1, poi "l'1A, l'1B, l'1C, eccetera"; il cubo 2, poi il 2A, il 2B... "Ogni volta che spawna un cubetto, mi spawna tutti quanti i cubetti in riga" (figura 1.6.2). "Ecco a voi, ragazzi, la nostra bella griglia di cubi." Il docente sposta lo spawner perché la griglia "non vada superando" la piattaforma della scena.

Righe e colonne diverse. Domanda alla classe: "se vogliamo cambiare il numero di cubi solo sulle colonne, come possiamo fare?" "Cambiare l'index." "Benissimo." Con il Last Index del primo ciclo a 5 e quello del secondo a 10 si ottengono "10 sulle righe, ma 5 sulle colonne". E, curiosità: un cubo spawna anche sopra il *player*, perché il trace ha colpito lui.

Spegnere il Tick. "Faccio solo un piccolo appunto per tutti quanti." Il tutto gira sull'Event Tick: a ogni frame si ripetono cento trace e si cerca di spawnare cento cubi. Sull'uscita **Completed** del For Loop ("guarda, sta sotto Index") si collega **Set Actor Tick Enabled** con il booleano a falso: "andiamo a interrompere il Tick, per ragioni di ottimizzazione". Prima il docente "stava laggando"; dopo, "i trace sono scomparsi, ma tutti i cubi sono riapparsi", e il lag è diminuito. "Abbiamo detto: oh, basta, abbiamo sponato tutto."

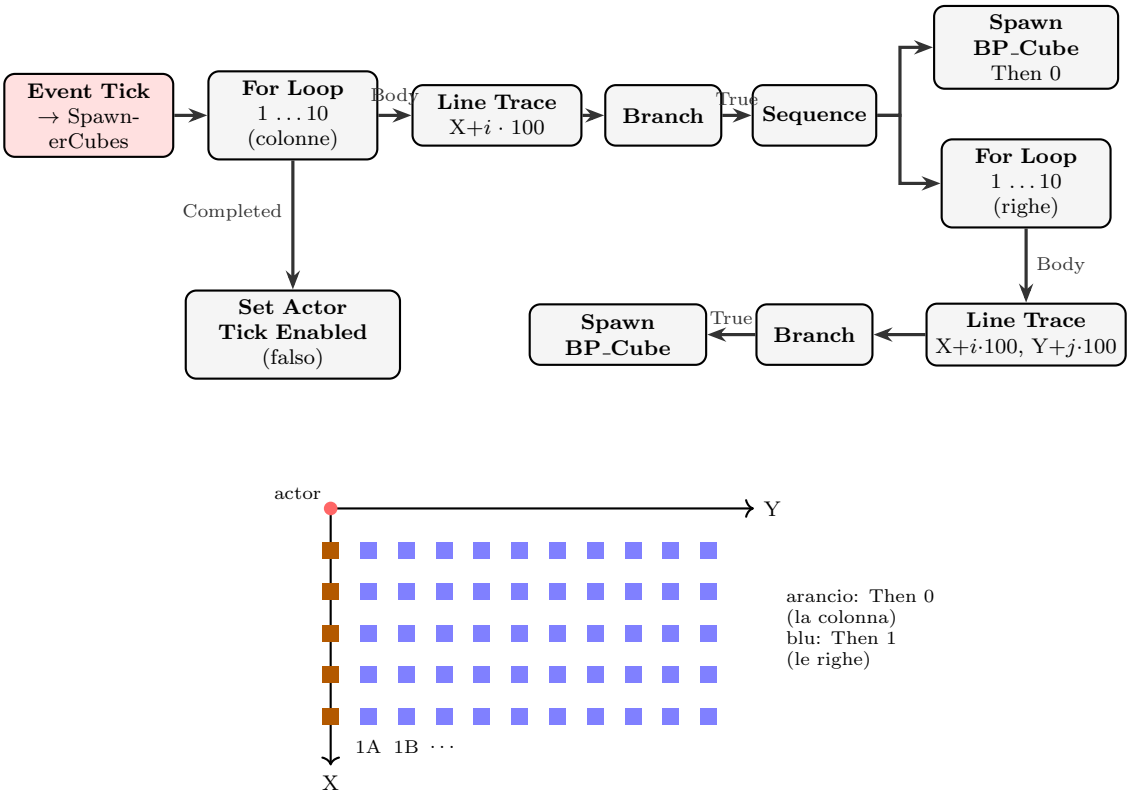


Figure 1.6.2: La matrice di cubi completata. Sopra, il grafo semplificato: il primo For Loop lancia un trace per ogni colonna; se il trace colpisce il terreno, il Sequence spawna il cubo della colonna (Then 0) e fa partire il secondo For Loop (Then 1), che usa la stessa X e una Y crescente. Finito il ciclo esterno, il Tick viene spento. Sotto, il risultato visto dall'alto con il Last Index esterno a 5, come nella prova della lezione: 5 colonne e, per ognuna, 10 cubi in riga (le spaziature sono di 100 unità). Schema ricostruito; i calcoli dei vettori sono quelli della quinta lezione, qui abbreviati.

Approfondimento: quanti giri fa un ciclo annidato. Con il ciclo esterno da 1 a r e quello interno da 1 a c, il corpo interno si esegue $r \times c$ volte: con 10 e 10, cento trace e cento spawn, più i dieci della colonna. La posizione del cubo (i, j) è

$$P_{ij} = P_0 + (i \cdot 100,\ j \cdot 100,\ 0),$$

e il cubo della colonna è il caso $j = 0$. Romero e Sewell (capitolo 19, sulla generazione procedurale) costruiscono la stessa griglia nel **Construction Script**, con un **Instanced Static Mesh Component** al posto degli actor: invece di spawnare cento actor, ciascuno con il suo Tick, i suoi componenti e la sua memoria, si aggiungono cento *istanze* della stessa *mesh* a un solo componente (nodo *Add Instance*), che le disegna con un costo minimo. Nel Construction Script, inoltre, la griglia si vede e si aggiorna già nell'editor. La lezione ha preferito lo spawn per esercitare i nodi visti la volta scorsa; spegnere il Tick dopo il primo passaggio è la toppa che rende accettabile la scelta.

1.6.8 I problemi degli studenti con la matrice

Prima di passare alla piattaforma, una serie di casi in condivisione dello schermo, che il docente risolve uno per uno.

- **"A me esce fuori solo un muro, basso."** Non è un muro: sono i cubi, tutti attaccati, perché sono **troppo grossi** rispetto all'intervallo di 100. "L'offset di distanza è troppo piccolo", o, il che è lo stesso, il cubo è troppo grande: si apre BP_Cube (non lo spawner) e lo si rimpicciolisce, per esempio a 0.2.

- **I cubi si impilano.** Il cubo non ignora il canale Visibility: il trace lo colpisce e lo spawn si ripete (la studentessa aveva cambiato la risposta, ma sullo spawner e non sul cubo spawnato; il caso resta in sospeso finché non si sistemano le dimensioni).

- **Le righe non compaiono.** Sul secondo ciclo era collegata la X di Get Actor Location, non quella del primo ciclo; e mancava la somma sulla Y. "Tu non ti devi più muovere sulla X: ti devi muovere sulla Y, perché devi stampare in riga, non in colonna."

- **Il secondo ciclo non fa niente.** Il pin di esecuzione **Loop Body** non era collegato al trace, e il trace non era collegato allo spawn. "Quello si chiama nodo di esecuzione: se non lo attacchi, ti calcola, ma non esegue."

- **Il trace non si vede.** Mancava l'End: la somma del punto con Get Actor Rotation, Get Up Vector e la moltiplicazione per -5000. Il docente consiglia di copiare tutto il terzetto dal primo blocco (Ctrl+C, Ctrl+V) e collegarlo allo Start e all'End.

- **Un indice collegato per sbaglio.** Il First Index del For Loop era stato collegato alla moltiplicazione: va lasciato al valore fisso, è l'Index (in uscita) a entrare nel calcolo.

- **Mi sono perso degli ultimi passaggi.** Una studentessa entrata tardi riceve un riassunto veloce: "va benissimo".

"Ok, vi spawna a tutti questa bellissima matrice di cubi? Yes." "Abbiamo oggi capito questo meraviglioso mondo del For Loop."

1.6.9 La piattaforma: componenti e controllo sul player

Prima di cominciare, una verifica tecnica: il docente deve condividere anche *l'audio del computer* su Zoom, perché la piattaforma suona ("aguzzate l'orecchio. Sentite qualcosa?"). Poi, "sempre la solita cosa": un nuovo Blueprint Class di tipo Actor, **BP_Platform** (il docente lo chiama BP_Platform1).

Un po' di *reverse engineering*. Il docente mostra la piattaforma già pronta e ne smonta i pezzi: "è formata da una *platform*, da una *box* e da una *billboard*". La billboard servirà dopo (sezione 1.6.15); per ora contano le altre due (figura 1.6.3).

- **Platform**: una **Static Mesh** (il componente "Static Mesh", rinominato *Platform*), con la *mesh* **Modular Platform** presente nel progetto. È la parte visibile e fisica: "la collisione di tutta la mesh".

- **Box**: una **Box Collision**, il volume invisibile "che ci dice: ehi, guarda, è avvenuta la collisione col player, quindi comincia a salire; è finita la collisione col player, quindi ritorna al suo posto". Va resa **figlia** della piattaforma (trascinandola sopra Platform), "perché se non la colleghiamo, la box sta da sola e non segue la piattaforma". Si scala alle dimensioni della piattaforma, "tipo 7 e 7 sulla X e sulla Y", la si stringe un po' e la si **alza**, perché non stia sotto la piattaforma: il volume deve sentire chi ci sta *sopra*.

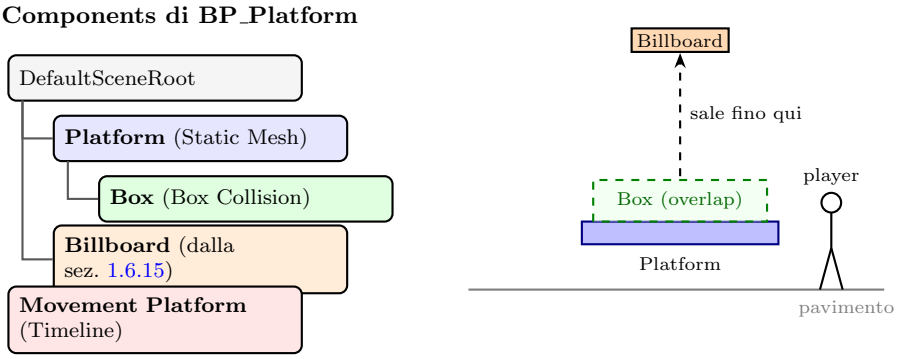


Figure 1.6.3: I componenti della piattaforma. A sinistra la gerarchia: la Box Collision è figlia della Platform, così la segue quando sale; il Billboard, aggiunto con il secondo metodo, segna il punto d'arrivo; la Timeline compare anch'essa tra i componenti, con il suo nome. A destra, la vista laterale: la Box è un volume invisibile appena sopra la piattaforma, che sente il player quando ci sale. Schema ricostruito; nella gerarchia il Billboard è figlio della radice, come richiede il secondo metodo (la lezione non lo dice esplicitamente, vedi la sezione 1.6.15).

Gli eventi della box. Con la Box selezionata, in fondo ai dettagli ci sono gli eventi: **On Component Begin Overlap** (il "più" verde accanto al nome) e, sotto, **On Component End Overlap**. Il docente li crea entrambi.

È proprio il player? "La prima cosa, ragazzi, è controllare se è effettivamente il player quello con cui facciamo overlap." Dall'**Other Actor** dell'evento ("l'attore che ha overlappato la box") si tira fuori un confronto == (*Equal*) con **Get Player Pawn** ("Pawn, ragazzi, è il nostro player"), e il risultato va in un **Branch**. Se è il player, "allora fammi la timeline e tutto il resto".

Uguale uguale. "Una cosa importante, per chi non è programmatore." Il doppio uguale, ==, è un **confronto**: "è uguale a questo?". Il singolo uguale, =, in informatica è un'**assegnazione**: "vuol dire che questo assume il valore dell'altro"; se scrivo che una variabile è uguale a un numero, "la variabile sarà sovrascritta da quel numero". Nei Blueprint l'assegnazione è il nodo *Set*, il confronto è il nodo *Equal*: qui serve il secondo.

Tag o referenza? "L'altra volta avevamo usato **Actor Has Tag** e un Branch": "sono vari modi". Il tag "è più versatile", e il docente lo mostra con il suo progetto personale: una palla che diventa di metallo e prende velocità; "se sta sopra una certa velocità ed è di metallo, e il muro con cui sta collidendo ha un tag *DestructibleWall*, allora spaccamelo". Ma per il giocatore Unreal dà già una referenza diretta, "tipo Get Player Pawn, Get Player Character, Get Player Controller, tutto quello che è player", e il confronto con quella è più semplice.

Approfondimento: Pawn, Character e Controller. I tre nodi restituiscono oggetti diversi. Il **Pawn** è l'actor che il giocatore controlla nel mondo; il **Character** è un tipo particolare di Pawn, con capsula, *mesh* scheletrica e *Character Movement Component* (il Third Person Character del template è un Character, quindi per lui Get Player Pawn e Get Player Character restituiscono lo stesso actor); il **Player Controller** non è nel mondo, è il "cervello" che riceve l'input e *possiede* il Pawn. L'Other Actor di un overlap è un actor fisico, quindi il confronto giusto è con il Pawn o il Character, mai con il Controller: il confronto con il Controller sarebbe sempre falso. Romero e Sewell (capitolo 4, sulla comunicazione tra Blueprint) usano un'alternativa ancora più comune, il **Cast To** della classe del personaggio sull'Other Actor: il ramo di successo scatta solo se l'actor è di quella classe, e restituisce una referenza già del tipo giusto, utile se poi servono le sue variabili.

1.6.10 Le timeline

"Adesso, ragazzi, andiamo a tirar fuori il nostro bellissimo... la **timeline**." Dal ramo True del Branch si aggiunge un nodo *Add Timeline*, e lo si rinomina subito: **Movement Platform**.

Che cos'è. Già in slide, a inizio lezione, il docente l'aveva definita così: le timeline "sono delle linee del tempo". "Vanno in modo **asincrono** rispetto al Tick, quindi non sono sincronizzate": l'Event Tick scatta a ogni frame, "la timeline è a sé stante". Servono "a fare delle animazioni nel tempo": "è proprio una linea del tempo che possiamo usare per **interpolare** vari valori dipendenti dal tempo che gli mettiamo, un secondo, due secondi, tre secondi". "È da capire questo, ragazzi: possiamo con le timeline fare azioni nel tempo che gli diamo noi."

I pin. Il nodo ha sei ingressi di esecuzione (tabella 1.6.2). Il docente li spiega immaginando una timeline che va da 0 a 1.

- **Play**: riparte dal punto in cui si trova, "indipendentemente da dove sta"; se è a 0,5, "continua da 0,5, quindi fa 0,5–1".
- **Play from Start**: "si resetta a 0 e va a 1".
- **Stop**: "si ferma".
- **Reverse**: torna indietro dal punto in cui si trova; "se sta a 0,5, va verso 0".
- **Reverse from End**: "va a 1 e ritorna a 0", cioè parte dalla fine e torna all'inizio.
- **Set New Time**: sposta la timeline a un tempo preciso (il pin *New Time*).

In uscita ci sono **Update** (scatta a ogni aggiornamento, mentre la timeline scorre), **Finished** (scatta quando arriva in fondo, in un senso o nell'altro) e **Direction** ("la direzione", che servirà alla fine, per l'audio), più un pin per ogni *track* creata dentro la timeline.

L'editor della timeline. Doppio clic sul nodo: si apre l'editor, "è nero, è normale". In alto a sinistra, accanto a *Track*, si possono aggiungere quattro tipi di traccia: **Float**, **Vector**, **Event** e **Color**. "A noi ci interessa una **float track**, perché i float, ricordiamo, sono i valori numerici con la virgola": un numero che andrà da 0 a 1. La traccia si chiama **Movement Platform Z** ("la teniamo Z"). Nel grafico ci si muove con il tasto destro e si zooma con Ctrl e la rotella.

La lunghezza. "La parte grigia è la lunghezza della timeline", che di per sé "può essere infinita". Accanto a *Track* c'è **Length**: la si mette a **1**, un secondo, "e guardate che la lunghezza della timeline si accorcia".

I punti. "Come faccio a dire quali punti deve percorrere?" Tenendo premuto **Shift** e cliccando sul grafico ("sul graffo; scusatemi, il graffo è il grafico") appare un punto, una *key*. Il docente l'ha creata a 0,7737 e la corregge a mano nei campi *Time* e *Value*. Attenzione agli assi: "sulle ordinate abbiamo il valore; l'ascissa, quindi l'orizzontale, è il tempo". Domanda alla classe: per passare da 0 a 1 in un secondo, quali sono i due punti? Il primo, (0, 0); il secondo, dopo un'esitazione ("1, 0? 0, 1?"), **(1, 1)**: "vuol dire che a un secondo deve stare a 1". Si potrebbe anche dire "a un secondo devi stare a -1", e la linea va in negativo, o a 0,5, "e in un secondo arrivo alla metà". Qui si resta su 1 (figura 1.6.4).

Il pin nuovo. Tornando all'Event Graph, sul nodo è apparso il pin **Movement Platform Z**, "il pin che ci darà indietro il valore float nel tempo". Il docente lo collega a un Print String sull'Update: in Play i numeri salgono da 0 a 1 in un secondo, e poi "sta a 1, perché quando finisce rimane a 1".

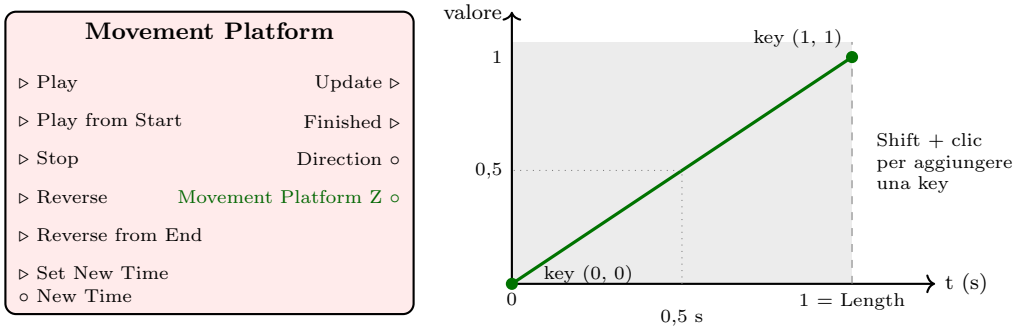


Figure 1.6.4: Il nodo Timeline della piattaforma e la sua float track. A sinistra i sei ingressi di esecuzione e le uscite: Update, Finished, Direction e il pin della traccia. A destra la curva: Length 1 secondo (la zona grigia), due key in (0, 0) e (1, 1); a metà del tempo il valore è a metà. Schema ricostruito.

Pin	**Che cosa fa**
Play	riprende dal tempo attuale, in avanti
Play from Start	torna a 0 e va in avanti
Stop	ferma la timeline dove si trova
Reverse	riprende dal tempo attuale, all'indietro
Reverse from End	salta alla fine e va all'indietro
Set New Time	porta la timeline al tempo New Time
Update	scatta a ogni aggiornamento mentre la timeline scorre
Finished	scatta quando la timeline arriva a un estremo (la fine in avanti, l'inizio all'indietro)
Direction	la direzione attuale: Forward o Backward (un'enumerazione, *ETimelineDirection*)
una uscita per traccia	il valore della traccia al tempo attuale (float, vettore, colore); le Event track hanno invece un'uscita di esecuzione

Table 1.6.2: I pin del nodo Timeline.

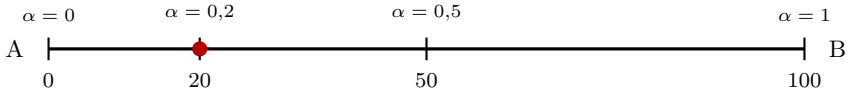
Approfondimento: le timeline secondo Romero e Sewell. Nel capitolo 6 (*Enhancing Player Abilities*) i due autori usano una timeline per lo *zoom* del mirino: una float track *Field of View* da 90 a 45 in 0,3 secondi, collegata dall'Update al nodo *Set Field of View*, con il tasto premuto su Play e il tasto rilasciato su **Reverse**, esattamente lo schema che la lezione userà per la piattaforma che sale e scende. Il libro aggiunge due osservazioni. La prima: le timeline sono perfette "per semplici cambi di valore, come la rotazione di una porta"; per animazioni complesse, di personaggi o cinematiche, Unreal ha il **Sequencer**. La seconda (nella tabella che confronta macro, funzioni ed eventi): i nodi Timeline si possono usare solo nell'**Event Graph** di un actor, non dentro le funzioni né dentro le macro, perché, come i Delay, sono azioni che durano nel tempo. Nelle opzioni in cima all'editor ci sono poi **Auto Play** (parte da sola al Begin Play), **Loop** (ricomincia all'infinito, utile per un'altalena) e **Use Last Keyframe** (fa finire la timeline sull'ultima key invece che alla Length). Le key, con il tasto destro, possono avere interpolazione lineare, costante o *auto* (curva morbida, con le tangenti): una curva a "S" fa partire e arrivare la piattaforma dolcemente, invece che di colpo.

1.6.11 L'interpolazione lineare e l'alfa

"Adesso ci servirebbe fare quello che viene chiamato, in informatica e anche su Unreal, **interpolazione**." C'è una slide: l'**interpolazione lineare** "ci dice: passo da questo punto a questo punto, passandomi per tutti gli intermedi", con un **alfa**.

L'esempio delle animazioni. "L'interpolazione è un concetto che possiamo trovare in tante cose diverse, tra cui le animazioni." Per muovere gli arti di un personaggio si usano le **ossa**, interne alla *mesh*. Il docente muove il braccio davanti alla webcam: "io non è che vedo subito così, pa!, di scatto: passo proprio da questo a questo". "L'interpolazione è il passaggio per tutti i punti intermedi che mi fa arrivare alla posizione finale."

L'alfa. "Seguitemi bene attentamente, ragazzi." Su una linea che va da 0 a 100, l'alfa è "un **puntatore** del punto in cui ci troviamo", espresso da 0 a 1. Se siamo a 0, l'alfa è 0; se siamo a 100, l'alfa è 1; "se l'alfa sta a 0,5, ci troveremo alla sua metà". Domanda: se siamo a 20? Uno studente: "0,2". "Ottimo. Siamo a 0,2: l'alfa sarà a 0,2." "Ci sono varie tipologie di interpolazione, ma l'interpolazione funziona in questo modo" (figura 1.6.5).



$$\text{Lerp}(A, B, \alpha) = A + \alpha\,(B - A)$$
$$\text{con } A = 0,\ B = 100,\ \alpha = 0{,}2\text{: } 0 + 0{,}2 \cdot 100 = 20$$

Figure 1.6.5: L'alfa dell'interpolazione lineare: un numero da 0 a 1 che dice in che punto del segmento da A a B ci troviamo. L'esempio della lezione: a 20 su 100 l'alfa vale 0,2. La formula, sotto, non è stata scritta a lezione ma è quella che il nodo Lerp calcola. Schema ricostruito.

Approfondimento: Lerp, alfa e timeline. Il nodo **Lerp** (per i float) calcola $A + \alpha(B - A)$: con $\alpha = 0$ restituisce A, con $\alpha = 1$ restituisce B, e in mezzo i valori intermedi in proporzione. Esiste per i vettori (**Lerp (Vector)**, che interpola X, Y e Z con lo stesso alfa), per le rotazioni (**Lerp (Rotator)**, con l'opzione *Shortest Path*), per i colori e per i *transform*. L'alfa può anche uscire da $[0, 1]$: con 1,2 il risultato va oltre B (estrapolazione). Il binomio timeline–Lerp è classico perché divide i compiti: la **timeline** decide *come* si scorre nel tempo (quanto dura, con che curva, avanti o indietro), il **Lerp** decide *dove* si va (da A a B). Cambiare la durata o la forma del movimento non tocca A e B, e viceversa. Da non confondere con i nodi *FInterp To* e *RInterp To* (quest'ultimo usato per la torretta): quelli avvicinano ogni frame un valore al bersaglio di una frazione, con una velocità e il Delta Time, senza un alfa né una durata fissa.

1.6.12 Il primo metodo: un Lerp sulla Z

"Abbiamo quindi da tirare fuori il nostro bellissimo **Lerp**." Domanda di partenza, guardando la piattaforma: "quale asse dovremmo influenzare, che si muova nel tempo?" "La Z." "Ottimo, bravo, Federico."

La costruzione. Il docente procede per tentativi, e a un certo punto uno studente chiede: "potresti spiegare un po' tutti i passaggi? Perché a un certo punto mi sono perso". La versione rispiegata, che è quella da seguire, è questa (figura 1.6.6).

1. Si prende il componente **Platform** e si tira fuori **Get Relative Location**. All'inizio il docente aveva usato Get Actor Location, poi la World Location ("scusate, ragazzi, questa è world"), e infine la *relative*, cioè la posizione rispetto all'actor. Si splitta il vettore.
2. La **Z** relativa va, da sola, nel pin **A** del Lerp: il punto di partenza.
3. La stessa Z più una variabile float nuova, **MaxHeight**, va nel pin **B**: il punto d'arrivo. Il valore scelto è **70** ("ci piace 70"; "sono 70 unità sulla Z").
4. L'**Alpha** del Lerp è il pin **Movement Platform Z** della timeline.
5. Un **Make Vector**: X e Y dalla location relativa, invariate; Z dal Lerp.
6. **Set Relative Location** sul componente Platform, con il Make Vector nel pin *New Location*, eseguito dall'**Update** della timeline.

La timeline parte dal Begin Overlap della box, attraverso il Branch sul player, con **Play**.

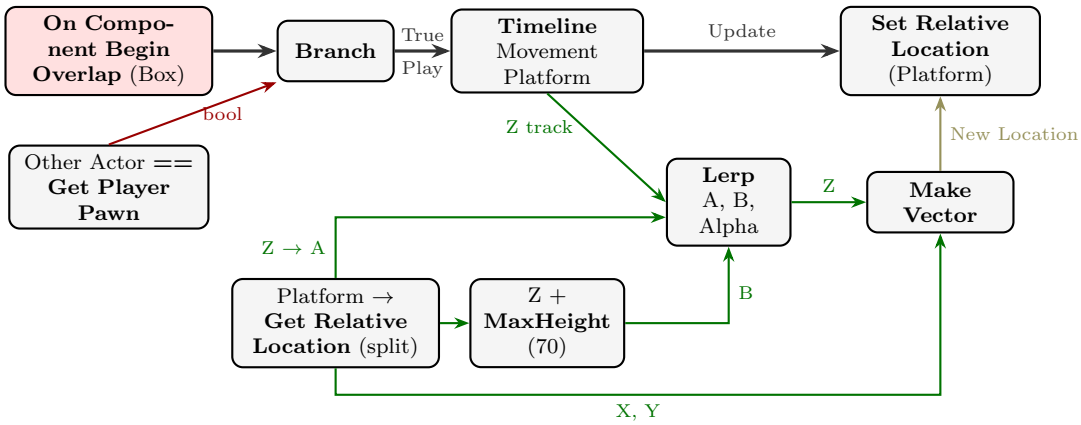


Figure 1.6.6: Il primo metodo. Il Begin Overlap della box, se l'actor è il player, fa partire la timeline; a ogni Update un Lerp tra la Z relativa attuale (A) e la stessa Z più MaxHeight (B), con la traccia della timeline come alfa, diventa la nuova Z della Platform. Funziona, ma ha un difetto: A e B vengono riletti a ogni Update dalla posizione già spostata (vedi l'approfondimento). Schema ricostruito.

"Va altissimo". In Play la piattaforma sale, ma molto più di 70 unità: a uno studente, con 70, "arriva in cielo"; con 30 arriva "poco più alto del muretto" della scena. "Non so perché, però sia così diverso." Il docente promette un metodo "un po' più carino e più malleabile"; la spiegazione arriva, in parte, dopo la pausa: "stiamo sommando" (sezione 1.6.14).

Approfondimento: perché il primo metodo "somma". A ogni Update il grafo legge la Z *attuale* della Platform, che è già stata spostata dall'Update precedente, e la usa come A; B è quella Z più 70. Il risultato è quindi $Z_{\text{nuova}} = Z_{\text{attuale}} + \alpha \cdot 70$: non un punto tra due estremi fissi, ma un *incremento* che si accumula. Se la timeline si aggiorna N volte in un secondo, la piattaforma sale di circa $70 \cdot \sum \alpha_k \approx 70 \cdot N/2$ unità:

con 60 aggiornamenti al secondo, più di duemila, non settanta. Da qui tre conseguenze, tutte osservate a lezione. La salita dipende dal *frame rate* ("è la mia impressione che vada più in alto": su un computer più veloce, più somme). Rallentare la timeline con il play rate fa *più* somme, e la piattaforma sale ancora di più; accelerarla ne fa meno. E con il Reverse il calcolo non torna simmetrico: lo studente che si vedeva la piattaforma "sempre leggermente più in basso fino a scomparire sotto il terreno" stava osservando proprio l'errore che si accumula a ogni salita e discesa. La cura è leggere A e B *una volta sola*, prima che la timeline parta, e tenerli fissi: è il secondo metodo.

1.6.13 La pausa e il Nintendo Direct

"Sono le 17:37. Facciamo la pausa classica di 20 minuti"; si torna "alle 18". Durante la pausa il docente aiuta uno studente con la foto del codice mandata su Discord (il problema è l'altezza della piattaforma con il primo metodo: "70 è giusto", il resto "credo sia tutto giusto"), e poco prima della ripresa arrivano le notizie dal *Nintendo Direct*, riferite da uno studente "referente". Nella trascrizione, molto storpiata, si riconoscono un *remake* di *Ocarina of Time* ("l'Opera of Time Remake"), un nuovo *Xenoblade*, un *Final Fantasy* in 2D-HD "stile *Octopath Traveler*" e del materiale nuovo su *Deltarune*; il tutto è riportato come lo si legge nella trascrizione, senza verifiche. La chiacchierata scivola su *Kingdom Hearts* e sulla trilogia *remake* di *Final Fantasy VII* ("uscirà prima l'intera trilogia remake"), e su Tetsuya Nomura ("le nomurate"). "Comunque, sempre belli questi siparietti", dice il docente; chi segue in differita può partecipare alla conversazione su Discord, "pacificamente".

1.6.14 La velocità della timeline: Set Play Rate e Duration

"Allora riprendiamo la nostra bellissima *elevator*, o piattaforma, quello che vogliamo." Una cosa che si può controllare è il **play rate**: "la velocità in cui essa, sempre ripetendo un secondo, viene detta: vai a due volte la tua velocità, a metà della tua velocità".

La timeline è un componente. "Se avete notato", nel pannello **Components** del Blueprint, accanto a Box, Platform e DefaultSceneRoot, c'è anche **Movement Platform**: "lo stesso nome della timeline". Ogni timeline aggiunta crea un componente con il suo nome, e da quel componente si possono chiamare le sue funzioni. Trascinandolo nel grafo ("Ctrl per il Get"), se ne tira fuori **Set Play Rate**, con il *Target* sulla timeline.

Play rate e lunghezza. Sul **Begin Play**, Set Play Rate a 0,5: "la sua velocità sarà la metà"; a 2, "il doppio"; a 1, "andrà come prima". Il docente insiste: "il play rate **non è la lunghezza**, non è la Length, è diversa". La Length dice quanto è lunga la curva; il play rate dice quanto velocemente la si percorre.

Il primo metodo non regge. Con il play rate cambiato, la piattaforma del primo metodo si comporta in modo strano, e il docente ne dà la spiegazione: "stiamo aumentando di 30 a ogni frame del nostro secondo"; "se riduciamo il play rate, lui sarà più veloce, ma di conseguenza farà meno somme". Detto meglio: con un play rate più alto la timeline finisce prima, fa meno Update, e quindi meno somme; con uno più basso ne fa di più. "Vedete che non si ferma a quell'unità, perché in questo caso stiamo sommando solamente: se finisce prima, vuol dire che finirà prima."

La variabile Duration. Ragionare in velocità relative è scomodo; è più naturale dire "voglio che duri 2 secondi". Una variabile float **Duration** e un piccolo calcolo:

$$\text{Play Rate} = \frac{1}{\text{Duration}} \times \text{Timeline Length},$$

con **Divide** ("lo slash") e **Get Timeline Length** (dal componente della timeline) moltiplicati con **Multiply**, e il risultato nel Set Play Rate del Begin Play. Con Duration 1 e una Length di 1 il play rate è 1, "velocità normale". Con Duration 2 il docente prima dice "il doppio", poi si corregge: "no, scusa, sarà più lunga,

perdonate"; con 0,5 sarà più veloce. È "una variabile d'appoggio" che traduce una durata in secondi nel play rate giusto. Per il resto della lezione la Duration resta a 1, "così almeno siamo tranquilli".

Approfondimento: perché la formula funziona. Una timeline di lunghezza L percorsa con play rate r dura L/r secondi. Volendo che duri D secondi, basta porre $L/r = D$, cioè $r = L/D$: è la formula del docente, scritta come $1/D \times L$. Moltiplicare per la Length rende il calcolo indipendente dalla curva: se un giorno la timeline diventa lunga 3 secondi, la Duration continua a dire la durata vera. Due cautele. Una Duration a 0 produce una divisione per zero, e la timeline non si muove più (il primo sospetto, a lezione, quando la piattaforma di uno studente "ha smesso di muoversi"). E il Set Play Rate va chiamato sulla timeline *giusta*: è l'errore della sezione 1.6.16.

1.6.15 Il secondo metodo: il Billboard e le posizioni salvate

Per avere "due punti specifici da raggiungere, in modo non dipendente da dei valori", il docente rifà il grafo. Il vecchio si può tenere, staccato, e copiare ("Ctrl+C, Ctrl+V") i pezzi che servono; il Set Relative Location della Platform resta: "non lo cancellate, perché è giusto che sia così".

Il Billboard. Si aggiunge (*Add*) un componente **Billboard**: "un oggetto di questo tipo: è questo **dinosauro**". "Fondamentalmente è un punto nello spazio", come lo *Scene Component* già usato per la torretta; "la differenza è che ha un'immagine", visibile solo nell'editor. Il piano: "prendere la sua posizione nello spazio, e dire: *lerpami* da questa posizione della piattaforma iniziale a questa posizione del billboard". Il Billboard si mette a **500** sulla Z ("o di più? 700? Mettete 500, la metà è giusta").

Lerp (Vector). Al posto del Lerp sui float, un **Lerp (Vector)** ("un v-lerp, un lerp tra vettori"): "invece che tra due numeri, due float, tra due vettori". A è la **Get Relative Location** della Platform, B quella del Billboard, l'Alpha la traccia della timeline; l'uscita va direttamente nel Set Relative Location della Platform, senza Make Vector.

"C'è qualcosa di sbagliato?" Così com'è, avverte il docente, il grafo ha ancora il difetto del primo metodo: "non salviamo mai la posizione delle due". Se il Billboard fosse **attaccato** alla Platform, "si muoverà insieme ad essa", e allora "avremo un effetto infinito": la piattaforma insegue un bersaglio che si allontana, "un continuo movimento sempre in alto, sempre in alto, pom pom pom". Il docente lo mostra riattaccando il Billboard: "e si ricomincia". La cura:

1. i due Get Relative Location si spostano sul **Begin Play**;
2. sul pin di uscita di ciascuno, **Promote to Variable**: la Platform diventa **StartLocation**, il Billboard **FinishLocation**;
3. nel Lerp (Vector) entrano i *Get* delle due variabili: StartLocation in A, FinishLocation in B.

"Così facendo abbiamo delle variabili che sono sempre fisse": "sarà sempre la stessa posizione" (figura 1.6.7).

Il bello del Billboard. "La cosa bella, poi, che possiamo vedere con questo metodo": il dinosauro si può spostare nello spazio, nei dettagli della piattaforma, e la piattaforma raggiungerà quel punto. "Dovete fare un livello? L'avete alzato a 8 metri? Prendete il billboard, lo mettete a 800": "devo arrivare qua sopra, quindi arriverò qua. Ecco fatto, sono arrivato preciso". ("Salto, l'ho mancato malissimo.") E non solo in verticale: spostando il Billboard anche sulla X o sulla Y, "andrà in diagonale"; "potremmo pure fare piattaforme in orizzontale, in diagonale, come vi pare". "Invece di fare matematica", si prendono direttamente i due vettori.

Il ritorno: Reverse. Scendendo dalla piattaforma, questa "rimane là". "Vi potrei dire: facciamo una nuova timeline e la facciamo al contrario. Non serve. Perché abbiamo **Reverse**." L'**On Component End Overlap** della box, collegato al pin Reverse della timeline: "esco fuori e lei se ne torna al suo posto", perché "invece di fare da A a B, fa da B ad A. Questo è insito nella timeline".

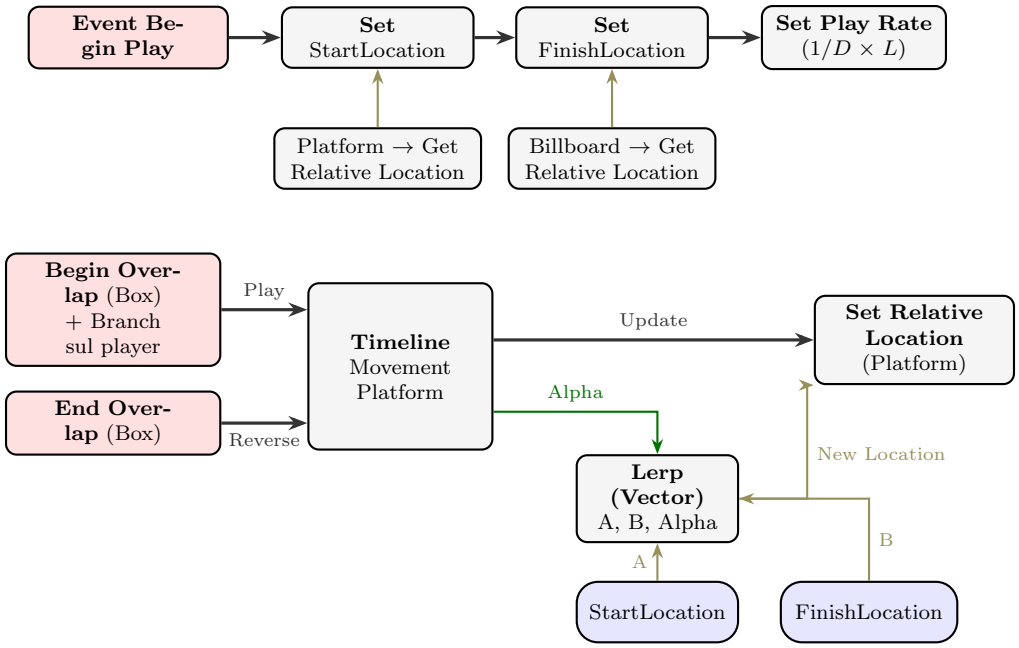


Figure 1.6.7: Il secondo metodo. Sopra: al Begin Play le posizioni relative della Platform e del Billboard vengono salvate una volta per tutte in StartLocation e FinishLocation (con Promote to Variable), e si imposta il play rate dalla Duration. Sotto: il Begin Overlap fa Play, l'End Overlap fa Reverse; a ogni Update un Lerp (Vector) tra i due punti fissi, con la traccia come alfa, diventa la nuova posizione della Platform. Schema ricostruito.

I problemi. "Problemi su questo? Siete a posto?" Tre casi.

- **"A me non si muove."** Il docente ripassa la lista: le due Get Relative Location, le due variabili, la timeline in Play, l'Update, il Lerp tra i vettori, l'Alpha collegato. Lo studente non aveva capito come creare le variabili: tasto destro sul pin, **Promote to Variable**, e il nome. Poi StartLocation in A, FinishLocation in B, e l'Alpha alla traccia della timeline.

- **L'evento diventato Custom Event.** Uno studente scopre da solo il suo errore: copiando e incollando il blocco, l'evento di overlap "mi ha preso come Custom Event generico, e quindi non si muoveva". Un evento di componente incollato in un Blueprint perde il legame con il componente e diventa un evento qualunque, che nessuno chiama. "Errore di distrazione."

- **Ero rimasto indietro.** Uno studente, che aveva ricevuto una telefonata, riceve il riassunto: Billboard, Get Relative Location della Platform in StartLocation e del Billboard in FinishLocation, il Lerp (Vector) con l'alfa, il Set Relative Location, e il Reverse sull'End Overlap.

Approfondimento: Billboard figlio di chi? Il docente fa vedere che, con il Billboard attaccato alla Platform, la piattaforma "sale all'infinito", e che salvando le posizioni al Begin Play il problema scompare. Ma c'è un dettaglio che la lezione non esplicita: se il Billboard resta figlio della Platform, la sua *relative location* è relativa alla Platform, non all'actor; salvarla al Begin Play dà $(0, 0, 500)$ rispetto alla piattaforma, mentre il Set Relative Location della Platform lavora rispetto alla radice dell'actor. I due numeri coincidono solo se la Platform sta nell'origine dell'actor (come nel Blueprint della lezione). La configurazione più robusta è mettere il Billboard come figlio della **DefaultSceneRoot**, accanto alla Platform, così che le due posizioni relative siano nello stesso spazio e il bersaglio non si muova mai con la piattaforma. Un'altra soluzione, equivalente, è salvare e impostare le posizioni nel mondo (Get e Set World Location). Infine: il "dinosauro" è visibile solo nell'editor; nel gioco il Billboard non si vede, come tutti i componenti di servizio.

1.6.16 La timeline sbagliata

Il docente vuole far vedere che la Duration funziona anche con il secondo metodo: "la cambio in 2"; poi, "mettiamo dei valori che posso riconoscere, tipo 10". Ma la velocità non cambia. "Non mi setta il play rate, forse." Anche a uno studente, "l'unica cosa è la duration: non va effettivamente a modificare la velocità". Il docente prova un Print, un play rate a 0,1, a mille: "sempre lo stesso".

La soluzione, da uno studente. "Domanda un po' stupida: non è che forse l'attore deve essere Platform e non Movement Platform?" Non è quello, ma è la pista giusta: "errore mio". Il secondo metodo usa una timeline *nuova*, che si chiama semplicemente **Timeline**; il Set Play Rate del Begin Play aveva come Target ancora **Movement Platform**, "quella dell'esempio di prima". "Stavamo referenziando la timeline dell'altro esempio, e non la playavamo mai."

La lezione dell'errore. "Da questo errore possiamo imparare qualche cosa": "le timeline hanno un **nome**". Ogni timeline è un componente distinto, e le sue funzioni (Set Play Rate, Get Timeline Length, Play, Reverse…) agiscono solo su di lei. Con il Target giusto e una Duration di 3, la piattaforma "adesso è più lenta, perché dura di più". Morale: dare alle timeline nomi chiari, e cancellare quelle che non servono più.

1.6.17 L'audio: la Sound Cue

"Se non ci sono più problemi, passiamo al nostro bellissimo *sound*."

Importare il file. Un consiglio d'ordine: una **cartella audio** nel Content Browser (tasto destro, *New Folder*). Per importare il file scaricato da Discord basta **trascinarlo** nel Content Browser: appare un asset "con la linea", una **Sound Wave**, senza altre domande.

Creare la Cue. Sulla Sound Wave, tasto destro, **Create Cue** ("è il secondo"). La **Sound Cue** ("la queue", "la Q", nella trascrizione) è, dice il docente, "un *sound* programmabile, tra virgolette". Doppio clic: si apre un grafo "come abbiamo visto anche su Unreal molto spesso", con un nodo **Wave Player** (il nostro suono) collegato all'**Output**. A sinistra, i dettagli della Cue: **Volume Multiplier**, **Pitch Multiplier**, *Sound Class* ("class override"), *Attenuation* e così via, "tutti i vari dati che possiamo modificare".

Il pitch. Con il Pitch Multiplier a 2 il suono è più alto; a 0,5 "è pitchato verso il basso": "molto più basso livello di tono, e anche più lento". "Perché il pitch influenza sia la velocità sia la tonalità." Si torna a 1.

Il loop. Con Play il suono parte "solo una volta"; "noi vogliamo che vada in loop" finché la piattaforma si muove, "tipo: pipi, pipi, pipi". Sul nodo **Wave Player** si spunta **Looping**. ("Ovviamente rompe": con la barra spaziatrice si avvia e si interrompe l'anteprima.)

Il Modulator. "Vogliamo che magari il pitch o il volume siano un valore **randomico**, tra un valore e un altro, in modo che non sia sempre lo stesso suono." Si inserisce un nodo **Modulator** tra il Wave Player e l'Output: ha **Pitch Min**, **Pitch Max**, **Volume Min** e **Volume Max**. I valori della lezione: pitch tra **1,7** e **2**, volume tra **1** e **1,2**. A ogni riproduzione il suono è "leggermente più basso, leggermente più alto". Il perché, con il gioco della palla del docente: quando rimbalza fa un *boing*, "e visto che avevo solo un suono", che ripetuto "sembra la stessa cosa", un pitch minimo e massimo hanno dato "un risultato molto più gradevole".

Il Random. "Esiste anche il **Random**": se si hanno più suoni, il nodo Random ne sceglie uno a caso a ogni riproduzione. Il docente lo prova mettendo nel Random due rami, uno con il Modulator e uno senza, e ascoltando quale esce. L'uso tipico: "un proiettile che prende un muro: invece di avere sempre lo stesso suono, mettete tutti e tre i suoni nel Random". Per la piattaforma non serve, e si torna al solo Modulator (figura 1.6.8).

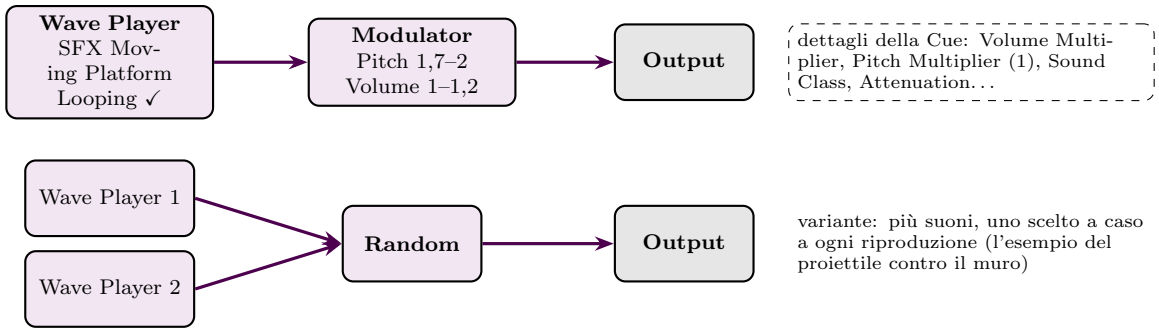


Figure 1.6.8: La Sound Cue della piattaforma: il Wave Player in loop, un Modulator che varia a caso pitch e volume a ogni riproduzione, l'Output. Sotto, la variante con il nodo Random mostrata a lezione e poi scartata. Schema ricostruito.

Approfondimento: Sound Wave, Sound Cue, MetaSound. La **Sound Wave** è il file audio importato, con poche proprietà proprie (fra cui un suo *Looping*). La **Sound Cue** è un piccolo grafo che combina una o più Wave con nodi di elaborazione: Modulator, Random, *Mixer*, *Concatenator*, *Delay*, *Attenuation*...; nei Blueprint si può passare indifferentemente una Wave o una Cue a qualunque nodo che vuole un *Sound*. In Unreal Engine 5 esistono anche i **MetaSound**, un sistema più recente e più potente, che genera e manipola l'audio con un grafo simile a quello dei Blueprint; le Sound Cue restano supportate e sono il punto di partenza più semplice. Il formato **WAV** raccomandato dal docente è quello standard per l'import: non compresso, lo comprime poi l'engine secondo la piattaforma. Sul pitch, l'osservazione del docente è esatta: in una Sound Cue il Pitch Multiplier cambia la velocità di riproduzione, quindi alza o abbassa il tono e accorcia o allunga il suono insieme, come un disco suonato a giri sbagliati.

1.6.18 Far suonare la piattaforma: Spawn Sound, Do Once e Stop

"Abbiamo due modi per piazzare i suoni": i nodi **Play Sound 2D** e **Play Sound at Location**, e i nodi **Spawn Sound 2D** e **Spawn Sound at Location**. "La differenza sostanziale è che uno *playa* il suono solamente; l'altro lo *spawna*, e quindi crea una **referenza**", come lo Spawn Actor from Class "che quando spawna ha un Return Value".

La prova con Play Sound 2D. "Proviamo a mettere Play Sound 2D": la Cue nel pin Sound, l'esecuzione dall'Update della timeline. Il docente sale sulla piattaforma: il suono parte, "e non smette più". Peggio: l'Update scatta a ogni aggiornamento, quindi i suoni si sovrappongono ("come avete sentito, è stato uno stormo di uccelli incredibile"). Due problemi, due soluzioni.

- Il suono è in loop: per fermarlo bisogna sapere *quale* suono fermare, e il Play Sound non lo dice. Serve lo **Spawn Sound at Location**, che restituisce la referenza.
- L'Update scatta molte volte: serve un **Do Once**, "che lo fa una sola volta".

Lo spawn. **Do Once**, poi **Spawn Sound at Location**: Sound la Cue, Location la **Get Actor Location** ("alla locazione dell'attore playerà questo suono"; il docente pensa prima alla piattaforma, poi sceglie l'actor). Dal **Return Value**, **Promote to Variable**: **SoundPlatform**. "Adesso, di questo suono che abbiamo creato, abbiamo la referenza: qualcosa che punta e ci dice: ehi, questo è il suono che fa pipipipi."

Lo stop. "Io voglio che si fermi quando la timeline finisce", cioè quando arriva in cima (a 1) o torna alla base (a 0). Sul **Finished** della timeline: Get di SoundPlatform, **Stop**. "Fino a che non è arrivato in cima, o fino a che non è tornato alla base, il suono playerà, e si stopperà appena finisce la timeline."

Il Reset del Do Once. "Un'ultima cosa molto importante." Il Do Once, una volta passato, resta chiuso; ha però un ingresso **Reset**: "se lo inserisco là dentro, questo Do Once si resetta, si apre, e dà la possibilità di essere rieseguito". "Quando ho stoppato il suono una volta, lo devo riattivare per la volta dopo": dopo lo Stop, il Reset. "E si ferma; scendo, ed eccolo. Benissimo."

1.6.19 Il pitch che cambia con la direzione

"Potremmo anche, ipoteticamente..." L'idea: un suono **più alto quando la piattaforma sale** e **più basso quando scende**. Il componente audio spawnato ha la funzione **Set Pitch Multiplier**, da chiamare sulla referenza SoundPlatform.

Come sapere se sale o scende. Il docente comincia a ragionare con un Branch ("se sta andando da StartLocation a FinishLocation..."), poi si ferma: "possiamo usare una cosa molto interessante". Il pin **Direction** della timeline è un'**enumerazione** ("che vedremo in seguito che cos'è, ma oggi ci serve semplicemente vedere questa cosa"): "una variabile che ha dentro di sé due elementi", *Forward* e *Backward*. Tirandone fuori uno **Switch on ETimelineDirection** si ottengono due uscite di esecuzione: "avanti vuol dire che sta andando 0–1, e indietro è 1–0".

- **Forward**: Set Pitch Multiplier su SoundPlatform a **1**, il pitch normale;
- **Backward**: Set Pitch Multiplier a **0,4**, "per far puh, puh, puh, un po' più bassino: una discesa" (il docente prova anche 0,5 e 0,1, "molto basso"; il valore finale è 0,4).

"Mettiamo tutto nel Do Once" (figura 1.6.9).

L'errore dell'ordine. A una studentessa il grafo, "sulla carta", è giusto, ma il compilatore segnala un errore: il Set Pitch Multiplier dice che il suono "non è mai stato creato". Il motivo è l'ordine: lo Switch, con i due Set Pitch Multiplier, veniva eseguito *prima* che lo Spawn Sound avesse riempito la variabile, quindi su una referenza vuota. "Metti prima il Do Once", e dopo lo spawn e il Set della variabile, lo Switch con i due rami. "Dovrebbe andare." E va (dopo aver condiviso anche l'audio del computer su Zoom, per farlo sentire).

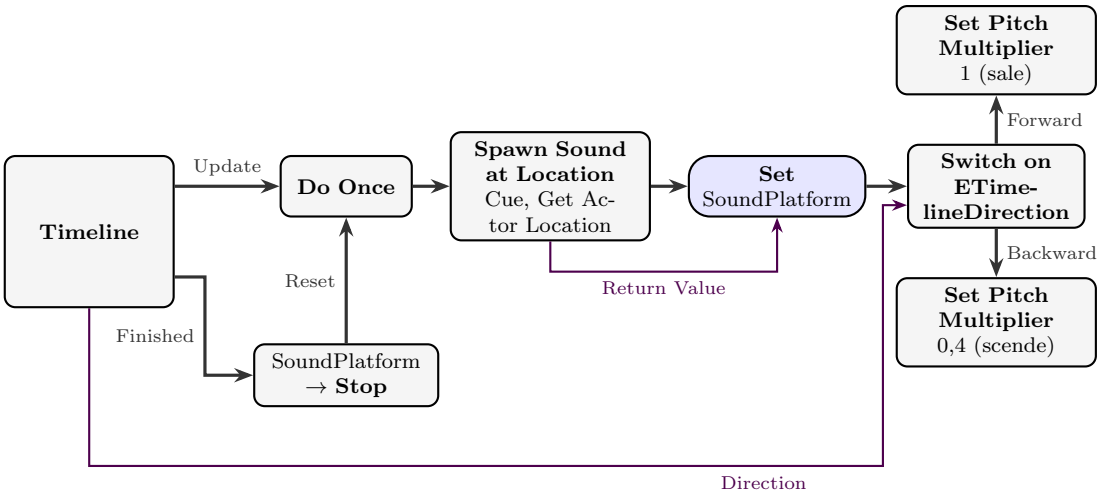


Figure 1.6.9: Il suono della piattaforma. Al primo Update il Do Once lascia passare: lo Spawn Sound at Location crea il suono in loop, la sua referenza viene salvata in SoundPlatform, e lo Switch sulla Direction della timeline sceglie il pitch (1 in salita, 0,4 in discesa). Al Finished, il suono si ferma e il Do Once si riapre per la volta dopo. Schema ricostruito; l'ordine è quello che risolve l'errore della studentessa (prima lo spawn, poi il pitch).

Approfondimento: Play o Spawn, 2D o at Location. Romero e Sewell (capitolo 6) usano **Play Sound at Location** per l'esplosione del bersaglio colpito: un suono breve, "spara e dimentica", che non serve più controllare, con la *Hit Location* dell'Event Hit come posizione. Per un suono in loop, che va fermato o modificato, serve invece uno dei nodi **Spawn Sound**, che creano un **Audio Component** e ne restituiscono la referenza: su di esso si chiamano Stop, Set Pitch Multiplier, Set Volume Multiplier, *Fade Out* e così via. I nodi **2D** suonano "in cuffia", senza posizione né attenuazione con la distanza (adatti a musica e interfaccia); quelli **at Location** suonano in un punto del mondo, e con un'*attenuation* si sentono di meno allontanandosi. Due osservazioni sulla scelta della lezione. La prima: la Get Actor Location è la posizione della *radice* dell'actor, che non si muove, perché a salire è solo la Platform; il suono resta quindi alla base. Per un suono che segue la piattaforma esiste **Spawn Sound Attached**, che lo attacca a un componente (qui, la Platform). La seconda: invece di Do Once e Spawn a ogni salita, si può aggiungere un **Audio Component** direttamente ai componenti del Blueprint, con *Auto Activate* spento, e chiamare *Play* e *Stop* su di esso; la referenza c'è già, e l'errore dell'ordine non può capitare.

1.6.20 La chiusura

"Ci siete voi altri, ragazzi? C'è qualche problema? Funziona tranquillamente?" "Adesso andiamo a cena, siamo a fine lezione." L'ultimo invito: "se ci sono problemi, diteli ora o tacete per sempre; o almeno fino alla prossima lezione". L'appuntamento è per **giovedì**. "Vi ringrazio per l'attenzione. Buonasera a tutti, ragazzi. Ciao, ciao."

Punti chiave del capitolo

- Le risposte di collisione sono tre: **Ignore** (nessuna collisione, nessun evento), **Overlap** (gli oggetti si attraversano, ma l'engine notifica l'incontro) e **Block** (collisione fisica e di controllo). Fra due oggetti vince la risposta meno restrittiva.
- L'**Object Type** è la categoria di un oggetto; le **Object Responses** dicono come l'oggetto risponde a ogni categoria. Nuovi tipi e nuovi canali si creano in *Project Settings* → *Collision*: pochi, uno per ogni categoria o domanda diversa, non uno per oggetto.
- I **volumi invisibili** in overlap sono lo strumento classico per far scattare eventi (una cutscene, una piattaforma): **Begin** ed **End Overlap**, a livello di actor o di componente, scattano una volta in entrata e una in uscita.
- I trace costano: vanno usati per azioni precise (il salto anticipato dei platformer) e spenti quando non servono. Gli oggetti di un gioco devono restare il più possibile **isolati**, e comunicare.
- La **matrice di cubi**: un Branch sul trace, **Spawn Actor from Class** di BP_Cube nella Location del **Break Hit Result**; un **Sequence** che spawna la colonna e fa partire un secondo **For Loop** per la riga (stessa X, Y crescente). Il cubo deve ignorare il canale Visibility, e alla fine **Set Actor Tick Enabled** spegne il Tick.
- La piattaforma: una Static Mesh, una **Box Collision** figlia, un controllo **Other Actor == Get Player Pawn** con un Branch. == confronta, = assegna.
- Una **timeline** è una linea del tempo indipendente dal Tick: una float track con **Length** 1 e due key (0, 0) e (1, 1), messe con Shift e clic. Ingressi Play, Play from Start, Stop, Reverse, Reverse from End, Set New Time; uscite Update, Finished, Direction e il valore della traccia.
- L'**interpolazione lineare** passa da A a B per tutti i punti intermedi; l'**alfa** (da 0 a 1) dice dove ci si trova. La timeline dà l'alfa, il **Lerp** dà la posizione.
- Il primo metodo (Lerp sulla Z con MaxHeight, rileggendo la posizione attuale) **accumula**: sale troppo, dipende dal frame rate e dal play rate, e non torna al punto di partenza.

Sintomo	Causa e rimedio
Pop-up “project files are not associated” a ogni avvio del launcher	associazione dei file di progetto rotta dagli aggiornamenti: Fix Now; i progetti non corrono rischi
Il messaggio di overlap non compare	la risposta di uno dei due oggetti è Ignore, o manca Generate Overlap Events; con Overlap da una parte e Block dall’altra il risultato è un overlap
I cubi si spawnano all’infinito, impilati	il trace sul Tick vede i cubi appena nati: mesh di BP_Cube su Custom, Visibility su Ignore (e Always Spawn, Ignore Collisions)
Il cubo si può solo scalare	è stato trascinato sopra il DefaultSceneRoot, sostituendolo: ricrearlo come figlio della radice
“Un muro basso” invece della griglia	cubi troppo grandi per l’intervallo di 100: rimpicciolire BP_Cube (0.5, 0.2)
Le righe non compaiono	il secondo ciclo usa la X di Get Actor Location invece della X del primo ciclo, o somma l’offset alla X invece che alla Y
Il ciclo calcola ma non esegue	Loop Body non collegato al trace, o il trace non collegato allo spawn
Il trace della riga non si vede	manca l’End (Up Vector $\times -5000$ sommato allo Start)
Lag con la matrice	spawn e trace a ogni frame: Set Actor Tick Enabled (falso) sul Completed del For Loop
La box non segue la piattaforma	la Box Collision non è figlia della Platform
La piattaforma sale troppo (“arriva in cielo”)	primo metodo: A e B riletti dalla posizione già spostata, la salita si accumula; usare posizioni salvate (secondo metodo)
La piattaforma scende ogni volta un po’ di più	stesso errore che si accumula a ogni Play e Reverse: secondo metodo
La piattaforma insegue il Billboard all’infinito	il bersaglio si muove con la Platform: salvare Start e Finish Location al Begin Play (e il Billboard figlio della radice)
La piattaforma resta su quando si scende	End Overlap non collegato a Reverse
La piattaforma non si muove dopo un copia-incolla	l’evento di overlap incollato è diventato un Custom Event generico: ricrearlo dal componente Box
Duration e play rate non hanno effetto	Set Play Rate sulla timeline sbagliata (il Target è la timeline del primo metodo); o Duration a 0
Il suono non smette più	Play Sound non dà una referenza: Spawn Sound at Location, Promote to Variable, Stop sul Finished
Tanti suoni sovrapposti (“uno stormo di uccelli”)	lo spawn sull’Update scatta a ogni aggiornamento: Do Once, con Reset dopo lo Stop
Il suono parte solo la prima volta	manca il Reset del Do Once
Errore su Set Pitch Multiplier (“mai creato”)	il pitch viene impostato prima dello spawn: prima Do Once, Spawn e Set, poi lo Switch

Table 1.6.3: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, componente o impostazione	**Che cosa fa**
Collision Presets / Collision Enabled	configurazioni pronte delle collisioni; attiva o disattiva query (trace e overlap) e fisica
Object Type / Object Responses	la categoria dell'oggetto; come risponde (Ignore, Overlap, Block) a ciascuna categoria
New Object Channel (Project Settings)	crea una nuova categoria di oggetti
Event ActorBegin/EndOverlap	inizio e fine di un overlap di un actor
On Component Begin/End Overlap	inizio e fine di un overlap di un componente (Other Actor, Other Comp)
For Loop (Completed)	ripete il Loop Body da First a Last Index; Completed scatta alla fine
Sequence	esegue in ordine le sue uscite Then 0, Then 1, ...
Break Hit Result	scompone l'Out Hit di un trace (Blocking Hit, Location, Impact Point, Normal, ...)
Split Struct Pin	separa un pin struttura (un *transform*, un vettore) nei suoi campi
Set Actor Tick Enabled	accende o spegne l'Event Tick di un actor
Box Collision	un volume invisibile per overlap e collisioni
Billboard	un punto nello spazio con un'icona, visibile solo nell'editor
== (Equal)	confronta due valori o referenze; restituisce un booleano
Get Player Pawn	la referenza al Pawn controllato dal giocatore
Actor Has Tag	vero se l'actor ha il tag indicato
Timeline	una linea del tempo con tracce (Float, Vector, Event, Color); Play, Reverse, Update, Finished, Direction
Set Play Rate / Get Timeline Length	velocità di una timeline; la sua lunghezza
Lerp / Lerp (Vector)	interpolazione lineare tra A e B con un alfa da 0 a 1
Get / Set Relative Location	posizione di un componente rispetto al suo genitore
Make Vector	costruisce un vettore da X, Y, Z
Promote to Variable	crea una variabile del tipo di un pin e la imposta
Sound Wave / Sound Cue	il file audio importato; il grafo che lo elabora (Wave Player, Looping, Modulator, Random)
Play Sound 2D / at Location	suona un suono e basta, senza referenza
Spawn Sound 2D / at Location	crea un Audio Component e ne restituisce la referenza
Stop / Set Pitch Multiplier	ferma un Audio Component; ne cambia il pitch
Do Once (Reset)	lascia passare l'esecuzione una volta, finché non viene resettato
Switch on ETimelineDirection	due uscite: Forward e Backward

Table 1.6.4: I nodi, i componenti e le impostazioni della sesta lezione.

- Il secondo metodo salva **StartLocation** (la Platform) e **FinishLocation** (un **Billboard**) al Begin Play, con Promote to Variable, e fa un **Lerp (Vector)** fra punti fissi. Spostando il Billboard si ottengono piattaforme verticali, orizzontali o diagonali. L'End Overlap su **Reverse** la riporta indietro.
- **Set Play Rate** cambia la velocità, non la lunghezza; con una variabile **Duration**, play rate = 1/Duration × Timeline Length. Ogni timeline è un componente con un nome: le funzioni vanno chiamate su quella giusta.
- L'audio: il file **WAV** si trascina nel Content Browser (Sound Wave), poi **Create Cue**. Nella **Sound Cue**: **Looping** sul Wave Player, un **Modulator** per variare pitch e volume (1,7–2 e 1–1,2), un **Random** per scegliere fra più suoni. Il pitch cambia tono e velocità.
- **Play Sound** suona e basta; **Spawn Sound** restituisce la referenza (**SoundPlatform**) per fermarlo (**Stop** sul Finished) o modificarlo. Un **Do Once** evita i suoni sovrapposti dall'Update, e va resettato dopo lo Stop.
- La **Direction** della timeline è un'enumerazione: con **Switch on ETimelineDirection**, pitch 1 in salita e 0,4 in discesa, impostato *dopo* lo spawn del suono.

Chapter 1.7

Il pulsante: array, ereditarietà ed event dispatcher

Questa è la **settima lezione** del corso breve di **introduzione a Unreal Engine**. Fino alla sesta lezione ogni Blueprint costruito in classe viveva per conto suo: la torretta mirava, la matrice di cubi si spawnava, la piattaforma saliva quando il giocatore ci metteva i piedi sopra. Oggi, per la prima volta, **due oggetti parlano fra loro**. L'esercizio è "un bellissimo **pulsante**", una pedana a pressione che, quando il giocatore ci sale sopra, si abbassa e **attiva** altri oggetti del livello: una **luce** che si accende, una piattaforma che si muove. "Andremo a vedere come funzionano le **referenze**" e "il fatto di attivare vari oggetti tramite, ad esempio, dei *trigger*".

Per arrivarci servono tre idee nuove, che danno il titolo al capitolo. Gli **array**, "gli scaffali" dove stanno tante variabili dello stesso tipo, perché un pulsante deve poter attivare non un oggetto solo ma "una moltitudine di oggetti". L'**ereditarietà**, il rapporto fra un Blueprint *parent* e i suoi *child*, perché tutti gli oggetti attivabili condividono lo stesso comportamento di base (attivarsi e disattivarsi), ma ognuno lo realizza a modo suo. E gli **event dispatcher**, un secondo modo, "un po' più ordinato", per collegare il pulsante agli oggetti: invece del pulsante che chiama la luce, è la luce che si mette in ascolto del pulsante. Lungo la strada la lezione tocca anche le **collisioni semplici e complesse** delle Static Mesh, la forma delle **curve** nelle timeline, la differenza fra **Play** e **Play from Start**, e una prima occhiata alle **luci**.

La lezione è **online**, su Zoom, come le precedenti. Si apre con una mezz'ora abbondante dedicata ai problemi rimasti dalla piattaforma della sesta lezione, prosegue con una breve parte teorica in slide (array, collisioni, ereditarietà) e con la costruzione del pulsante, della classe degli attivabili e della luce; poi la pausa, annunciata alle "5 e mezza" con ripresa al "solito orario, 17:55, 18"; la seconda parte è dedicata agli event dispatcher e a un esercizio di una ventina di minuti, e si chiude con un compito per casa: "ci sentiamo per martedì prossimo". Come sempre, molto tempo va nella risoluzione dei problemi degli studenti, a voce o in condivisione dello schermo; oggi la condivisione di Zoom fa i capricci più del solito ("mi guardo sempre questo Zoom maledetto").

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Arena Engine", "a Real" e "il Real"; il **Begin Play** è "beg in play", "big in play", "back in play" e "meg in play"; il **Blueprint** è "Drupal" e "DP"; la classe **BP_Activable** è "activable", "attiva ball", "Activa Activa" e "aktiaboletto"; il **Child Blueprint** è "child print"; il **Billboard** è ancora "il dinosauretto"; il **DefaultSceneRoot** è "default in root" e "default string"; l'**End Overlap** è "and overlap" e "hand overlap"; il **Begin Overlap** è "un becchino per la box"; l'**override** è "overrideare"; il **Max Step Height** è "max step height"; il **LOD** è "i load"; il **Do Once** è "Duens"; gli **event dispatcher** sono "limit dispatch", "event spatch" e "eventual spatch"; l'evento **OnCheckStatus** è "on share status" e "check out"; la **Sound Wave** è "wave cell" e "Somewave". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione, durante la pausa e durante l'esercizio il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("grazie a tutti per l'attenzione che ci ha spiegato", "la domanda è chi vuole intervenire in questo senso", "qual è la risposta che ci può essere?", presidenti di repubbliche inesistenti, "la velocità e il ritmo della velocità"): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran

parte della lezione è una dimostrazione dal vivo nell'editor dei Blueprint, costruita a voce ("tirate fuori", "collegate", "promote to variable"), e in un paio di punti il docente improvvisa, prova, torna indietro. Il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei pin di Unreal Engine 5; dove la trascrizione lascia un buco, lo dichiara.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono l'array come scaffale, la gerarchia degli attivabili, i componenti del pulsante, le collisioni semplici, la curva della timeline e le sue interpolazioni, i grafi del pulsante, del ciclo sugli attivabili e della luce, e il meccanismo dell'event dispatcher. Le tabelle riassumono i tipi di contenitore, i tipi di interpolazione delle key, i modi di far comunicare due Blueprint, gli errori incontrati dagli studenti e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. i **problemi della piattaforma** rimasti dalla volta scorsa (sezione 1.7.1);
2. la parte teorica: il programma, gli **array**, l'**ereditarietà** e la dimostrazione del pulsante finito (sezioni da 1.7.2 a 1.7.5);
3. il **pulsante**: le due mesh, la box, le **collisioni** della Static Mesh (sezioni 1.7.6 e 1.7.7);
4. la **timeline** del pulsante, le **curve**, i problemi degli studenti e Play contro Play from Start (sezioni da 1.7.8 a 1.7.11);
5. gli **attivabili**: la classe BP_Activable, l'array di referenze, il **For Each Loop** (sezioni 1.7.12 e 1.7.13);
6. la **luce**: il Child Blueprint, i due Begin Play, la Point Light, l'**override** (sezioni da 1.7.14 a 1.7.16);
7. la pausa, il suono della piattaforma e la correzione del pitch (sezioni 1.7.17 e 1.7.18);
8. gli **event dispatcher**: il dispatcher, il bind, la chiamata, la scatolina, l'unbind e la chiamata al parent (sezioni da 1.7.19 a 1.7.23);
9. l'**esercizio**, la sua soluzione e il compito per casa (sezioni 1.7.24 e 1.7.26).

1.7.1 Prima della lezione: i problemi della piattaforma

La registrazione comincia con gli studenti già collegati e con i dubbi rimasti dalla **piattaforma che sale** della sesta lezione: una piattaforma con una Static Mesh, una Box Collision sopra, un Billboard come punto d'arrivo, una timeline e un suono in loop. Quattro problemi, quattro lezioni di metodo.

Il primo: "accesso nullo" sul suono. Uno studente ha di nuovo problemi "con Arena Engine": il gioco funziona, ma alla fine della prova arriva un messaggio di errore sul suono della piattaforma. In inglese sarebbe qualcosa come "Sound... not found reference"; nell'editor in italiano è un **errore di runtime del Blueprint: accesso nullo** (in inglese *Accessed None*). "Vuol dire molto semplicemente che il suono non esiste, non è valido." È lo stesso errore corretto la volta scorsa con un altro studente: il nodo **Switch on ETimelineDirection**, con i due Set Pitch Multiplier, era collegato subito dopo il Do Once, *prima* dello Spawn Sound. "Prima creiamo SoundPlatform e poi lo modifichiamo": "il problema è che stai modificando una cosa che ancora non esiste". La prima volta che la timeline parte, la variabile è vuota e l'engine protesta; dalla seconda in poi il suono esiste e tutto sembra funzionare, "quindi fa un errore" solo una volta, ed è per questo che passa inosservato. La cura è spostare lo Switch *dopo* lo spawn e il Set della variabile.

Approfondimento: Accessed None e Is Valid. "Accessed None trying to read property…" è forse l'errore più frequente dei Blueprint: una variabile di tipo referenza a oggetto (*object reference*) è usata come *Target* di un nodo mentre non punta a niente. Il gioco non si blocca, il nodo semplicemente non fa nulla, e l'errore compare nel *Message Log* alla fine della sessione di Play, con un link al nodo colpevole. Romero e Sewell (capitolo 4, "Understanding Blueprint Communication") lo trasformano in una regola: *prima di eseguire una funzione su una variabile referenza, controllarla con la macro* ***Is Valid***, quella con il punto interrogativo bianco, perché "la variabile può non essere valida per molte ragioni". Nel caso della piattaforma la cura giusta è l'ordine dei nodi; l'Is Valid è la cintura di sicurezza.

Il secondo: la piattaforma che si sotterra. A un'altra studentessa "funziona tutto, tranne sempre quel problema che si sotterra la piattaforma se ci passi troppe volte sopra". Il docente sospetta il Billboard: è "esterno", attaccato alla piattaforma? Se il punto d'arrivo è figlio della parte che si muove, si sposta con lei, e le posizioni salvate smettono di essere fisse. La studentessa lo stacca e riprova.

Il terzo: la piattaforma che va allo zero del mondo. Il caso più lungo. Uno studente non ha potuto completare l'audio, perché la registrazione della lezione di martedì si interrompe prima di quella parte (il docente promette di spiegarglielo in pausa: sezione 1.7.17); e da quando ha aggiunto *Start Location* e *Finish Location*, "la piattaforma è iniziata ad andare a caso dove voleva lei". In condivisione dello schermo il docente procede per verifiche successive, ed è un piccolo manuale di *debug*.

- **Dove sta il Billboard nel mondo?** Cliccando sull'asset nel livello e poi sul componente Billboard nei Details, lo si vede sopra la pedana, come deve essere.
- **Che cosa vede il Blueprint?** "Fammi un favore: fai un **Print String**" della Finish Location, collegato al pin di esecuzione ("la freccettina"). Il risultato è **0, 0, 500**: "te lo fa come se fosse del mondo, non relativa". La piattaforma, appena ci si sale, "va lì, perché va lo zero del mondo", e poi non torna indietro.
- **La box.** Intanto si scopre che la Box Collision è "piccolissima" e sta sotto la piattaforma: va scalata e alzata, perché "il player deve andare sopra la piattaforma, il box capisce che sta andando sopra e quindi la piattaforma parte". "Se stai mettendo la box sotto la piattaforma, non lo prenderà più."
- **È il Blueprint giusto?** Si apre BP_Platform dall'asset, "giusto per verificare che non fosse un'altra copia diversa".
- **La causa.** Nella lista dei componenti manca il **DefaultSceneRoot**: la Static Mesh della piattaforma è diventata la radice ("non ha il default in root, perché l'hai eliminato praticamente"; "io non ho toccato niente"). Così la piattaforma, che è la radice, si porta dietro tutto, e le posizioni relative non significano più quello che dovrebbero. Il rimedio: aggiungere un componente **Scene** e trascinarlo sopra la Platform, in modo che diventi la nuova radice ("il default è diventato lui"), con la Platform e il resto come figli. Compilazione, prova: la piattaforma sale e scende.

Resta un dettaglio: la piattaforma si riabbassa anche se a uscire dalla box non è il player, perché sull'End Overlap "non c'è una verifica" come quella del Begin Overlap. È la stessa logica, da duplicare.

Approfondimento: perché la radice conta. In un Blueprint di tipo Actor il *Root Component* è il componente la cui trasformazione *è* quella dell'actor: Get Actor Location restituisce la sua posizione, e tutti gli altri componenti sono posizionati *relativamente* a lui. Se la radice è la stessa mesh che la timeline muove, muovere la mesh significa muovere l'actor intero, Billboard compreso: il punto d'arrivo scappa davanti alla piattaforma, e ogni posizione salvata "relativa" va ripensata. Con una radice ferma e invisibile (il DefaultSceneRoot o un componente Scene qualsiasi), invece, la piattaforma si muove *dentro* l'actor, e il Billboard, figlio della radice, resta dov'è. Un componente si promuove a radice trascinandolo sopra quella attuale nel pannello Components: è facile farlo per sbaglio, ed è il motivo per cui lo studente "non ha toccato niente".

Il quarto: il player che si blocca con i piedi. "Posso dire una cosetta anch'io?" Camminando verso la piattaforma, il personaggio ci sbatte contro e si ferma: "l'unico modo per attivarla è saltarci sopra". Non è un problema della box, ma della **collisione della piattaforma**: "il player collide col piede, letteralmente". Due rimedi: **interrarla** un pochino, abbassandola quasi al livello del pavimento, oppure **modificarne la collisione**, cosa che si vedrà proprio oggi, "anche in generale sugli asset" (sezione 1.7.7).

Il quinto: il suono che non cambia. Lo stesso studente ha notato un altro difetto: "se la fai alzare, parte il suono di quando si sta alzando; se poi scendi prima che arrivi al suo punto massimo, continua con quel suono e non parte quell'altro". "Effettivamente questo l'ho notato anch'io": è "un bug che possiamo risolvere", e verrà risolto dopo la pausa (sezione 1.7.18). "Ecco, perfetto. Grazie."

1.7.2 Il programma della giornata

"Allora, oggi ragazzi vi farò vedere molto velocissimamente cosa andremo a fare. Piccola spiegazione teorica, la solita cosa." Le domande, come sempre, si possono fare subito, nella "pausetta" o a fine lezione, "e parliamo di Kingdom Hearts 4". Il programma:

- un **pulsante** su cui "ci possiamo salire sopra e di conseguenza attivare varie azioni"; quindi le **referenze**, cioè il modo in cui un oggetto sa quali altri oggetti deve attivare, e i *trigger*;
- una **sequenza di azioni**, cioè più oggetti attivati insieme;
- gli **array**, insieme al ciclo **for** visto la volta scorsa: "ciclo, fammi questa cosa per ogni asset che sta dentro questa variabile";
- un ripasso delle **collisioni**;
- l'**ereditarietà**.

Il pulsante, avverte subito il docente, funziona come la piattaforma: "il nostro puzzle, per essere attivato, ha sempre il volume sopra di esso che interagisce".

1.7.3 Gli array: gli scaffali delle variabili

"Qualcuno di voi ha mai sentito parlare di array?" Silenzio, a parte un microfono "che ha deciso di impazzire". Il docente apre Paint, "il nostro amico Paint", e disegna.

La metafora. "Se una variabile è una **scatola**, gli array sono gli **scaffali**" dove stanno le scatole ("a parte questo scaffale montato male dell'IKEA"). All'IKEA, ricorda il docente, si prende nota del numero del mobile che si vuole comprare, e allo scaffale con quel numero si trovano tutti i suoi pezzi: "il dietro dell'armadio, la mensola dell'armadio, eccetera". Quel numero è come la **tipologia di variabile**: se lo scaffale è di tipo A (per esempio float), dentro ci saranno "più valori dello stesso tipo". Un array è quindi un **raggruppamento di variabili dello stesso tipo** (figura 1.7.1).

A che cosa serve. "Abbiamo tanti nemici, ragazzi. Spawniamo tanti nemici. Non facciamo una variabile per ogni nemico: facciamo una variabile di tipo array" che li raccoglie tutti, "tutti della stessa tipologia". E con l'array arriva il ciclo: invece di ripetere un'azione dieci volte con un For Loop che conta, si ripete un'azione *per ogni elemento* dello scaffale. È quello che farà il pulsante, che dovrà attivare tutti gli oggetti che gli sono stati affidati.

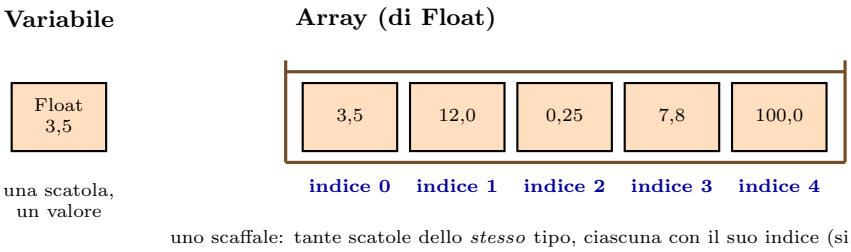


Figure 1.7.1: La metafora del docente. Una variabile è una scatola che contiene un valore; un array è uno scaffale che contiene tante scatole dello stesso tipo. Ogni posto dello scaffale ha un indice, e il primo indice è 0. Schema ricostruito.

Approfondimento: array, set e map. Romero e Sewell (capitolo 13, "Data Structures and Flow Control") chiamano **contenitori** (*containers*) le strutture che raccolgono più valori dello stesso tipo, e nei Blueprint ce ne sono tre. L'**array** è una lista ordinata con accesso per **indice**, "e gli array partono sempre dall'indice 0": con quattro elementi, il primo ha indice 0 e l'ultimo indice 3, cioè *Length* − 1. Il **set** è un insieme senza ordine e senza doppioni, in cui si cerca per valore; la **map** associa a ogni *chiave* un *valore* (per esempio il nome di un'arma e le sue munizioni). Sugli array si lavora con una manciata di nodi: **Get (a copy)** legge l'elemento a un indice, **Set Array Elem** lo modifica, **Add** aggiunge in fondo, **Insert** inserisce a un indice spostando gli altri, **Length** e **Last Index** dicono quanto è lungo, **Random Array Item** pesca un elemento a caso. Gli autori avvertono: mai leggere un indice oltre l'ultimo, perché i risultati sono imprevedibili. La tabella 1.7.2 riassume i quattro modi in cui una variabile può essere costituita.

1.7.4 L'ereditarietà: parent e child

La slide successiva riprende rapidamente le collisioni ("ripasseremo anche un po' le collisioni") e introduce il secondo concetto della giornata. "Qualcuno di voi ha mai sentito parlare dei concetti di **parent** e **child** in programmazione?"

Padre e figlio. "Mettiamo un esempio che possiamo vedere nella vita di tutti i giorni": il rapporto fra genitori e figli. "Io, come figlio di mio padre, ho preso una parte dei geni di mio padre" (e di mia madre), "e però, come essere unico, posso variare nelle mie particolarità: non sono uguale, ma condivido comunque delle cose". Lo stesso succede con gli oggetti: "ci sono degli oggetti che possono essere figli di altri oggetti". Esempio: un cubo *parent* che nel Begin Play stampa un valore, e fa solo quello. "Il child mi stamperà sempre quel valore, ma magari stampa anche un altro valore": "prende la caratteristica del papà, ma volendo può aggiungere, o addirittura **sovrascrivere**, alcune azioni".

Il cubo che diventa cilindro. In Unreal i child si creano direttamente dall'engine. In slide c'è il BP_Cube delle lezioni precedenti: "vogliamo farlo diventare un cilindro? Creiamo un child e cambiamo semplicemente la Static Mesh". "Il parent rimarrà un cubo, invece il figlio ha voluto cambiare forma": per le caratteristiche di Blueprint avranno lo stesso comportamento, "oppure, volendo, lo possiamo anche cambiare". La figura 1.7.2 mostra la gerarchia che si costruirà oggi.

Approfondimento: superclasse, sottoclasse e tipo. Romero e Sewell (capitolo 3, "Object-Oriented Programming and the Gameplay Framework") danno la versione tecnica. Quando si crea un Blueprint, la prima cosa che si sceglie è la sua **classe parent** (la *superclasse*); un Blueprint ha *un solo* parent, ma può avere molti child (*sottoclassi*). Il loro esempio sono le armi: una classe base *Weapon* con tutto ciò che è comune, e le armi concrete come figlie. Il vantaggio è doppio. Da un lato, una funzione definita nel parent (*Fire*) può essere **ridefinita** (*override*) in ogni child (il fucile a energia spara un raggio, il lanciarazzi un razzo), e a runtime viene eseguita la versione della classe effettiva dell'oggetto. Dall'altro, un child **è anche** del tipo del parent: un'istanza del lanciarazzi è un *Weapon*, e una variabile o un parametro di tipo

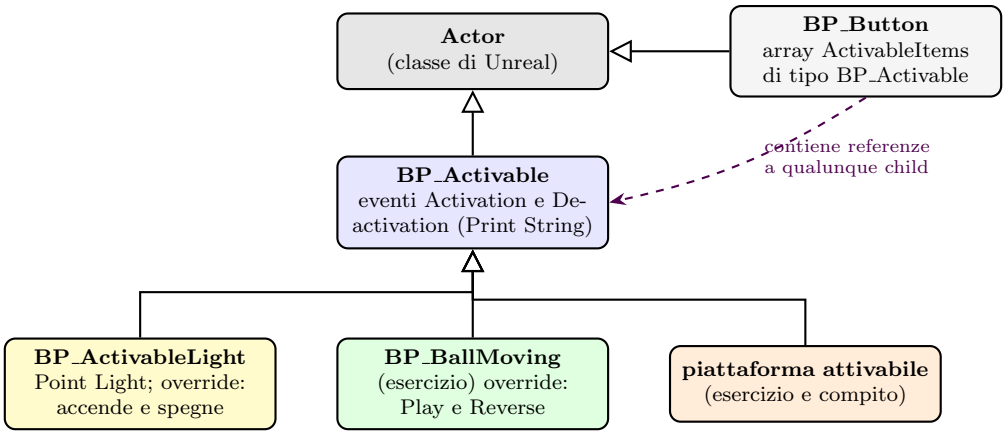


Figure 1.7.2: La gerarchia di classi costruita nella lezione. Le frecce con la punta vuota vanno dal child al parent ("eredita da"). BP_Activable definisce i due eventi comuni; ogni child li ridefinisce a modo suo. BP_Button tiene un array di referenze di tipo BP_Activable, e quindi può contenere qualunque child. I nomi dei Blueprint dell'esercizio sono quelli suggeriti dal docente; schema ricostruito.

Weapon può contenerla. È esattamente il trucco su cui si regge il pulsante di oggi: una variabile di tipo *BP_Activable* potrà contenere una luce, una piattaforma, una palla, qualunque child degli attivabili. Del resto, ogni Blueprint della lezione è già un child: BP_Button ha come parent la classe *Actor*.

1.7.5 La dimostrazione: il pulsante finito

"Andremo a creare un bellissimo pulsantone. Eccolo qua." Nella scena del docente c'è il risultato finale. Il giocatore sale sulla pedana, la pedana si abbassa e **accende una luce** ("vedremo anche un pochino di *lighting*"); se scende, la luce si spegne. Nella stessa scena c'è una piattaforma che reagisce allo stesso pulsante: "ho creato una piattaforma di tipologia **attivabile**", e luce e piattaforma "condividono" qualcosa, "sono della stessa classe, ma comportamenti diversi". Collegata la piattaforma al pulsante, la pressione accende la luce *e* manda giù la piattaforma; "in questo caso ho fatto sì che andasse solo giù": quando la piattaforma è in basso ci si sale sopra, e lei torna su; si torna sul pulsante, e di nuovo giù.

La domanda sull'interazione. "Faremo anche le interazioni, tipo premi E per interagire?" Sì: la prossima lezione sarà sulle **animazioni**, con un personaggio animato, e l'interazione si farà con il **trace**, "infatti il trace è collegato al fatto dell'interazione": ha interagito o non ha interagito.

Approfondimento: overlap o tasto? I due modi di attivare qualcosa hanno usi diversi. L'*overlap* di un volume, quello di oggi, è passivo: scatta perché il giocatore *è* in un posto (pedane a pressione, trappole, cutscene). L'interazione con un tasto è attiva: il giocatore guarda un oggetto e preme. Lo schema tipico, che la lezione annuncia, è un *line trace* dalla telecamera in avanti, eseguito alla pressione del tasto; se colpisce un actor che "sa" interagire, gli si chiama un evento. Per chiamarlo senza sapere in anticipo che classe abbia l'oggetto colpito si usa di solito una **Blueprint Interface** (Romero e Sewell, capitolo 16): una lista di funzioni senza corpo, che ogni classe interessata implementa a modo suo. È una cugina dell'ereditarietà vista oggi, utile quando le classi da accomunare non hanno un parent comune.

1.7.6 Il pulsante: due mesh e una box

"Allora, ragazzi, vi chiedo se siete pronti con il vostro progettino." Prima di cominciare, la raccomandazione di sempre: tenere il progetto ordinato in **cartelle**, "perché se no un giorno succederà che non ritroverete" le cose, "come perdere qualcuno in un centro commerciale il sabato". Nella cartella *Blueprint*, "siamo al solito":

Blueprint Class, Actor. Il docente lo chiama **BP_Button1** ("io farò sempre il mio bellissimo BP_Button1"); nel capitolo è BP_Button.

Due Static Mesh, non una. "Il bottone, per come sono fatte le mesh, sono due Static Mesh": una è il **frame**, la cornice, e l'altra la **piastra** che si abbassa. Il docente le crea con Ctrl+C e Ctrl+V da un componente Static Mesh, e le rinomina **Frame** e **Platform**; nel campo *Static Mesh* assegna alla prima la mesh della cornice della pedana a pressione ("se non sbaglio *pressure plate frame*") e alla seconda quella della piastra ("*pressure plate*... platform"); i nomi esatti degli asset nel progetto del corso non sono sicuri dalla trascrizione. "Essendo che sono fatti all'unisono", i due pezzi sono già allineati.

Un consiglio per i 3D artist. Il motivo della divisione vale per chiunque modelli, o faccia modellare, oggetti interattivi. "Se dovete fare un pulsante, non fate fare un pulsante unico: fate fare un pulsante che è diviso" fra la cornice ("quello che sta attorno") e la *pressure plate*. "Perché se adesso andiamo ad abbassare, ed è un unico asset, tutto il bottone va giù, anche il frame, quindi si va a compenetrare col pavimento." La regola: "se ci sono delle cose interagibili a livello di movimentazione, sappiate che è meglio sempre **splittarle**". Poi, certo, "dipende dalla creatività" e dal designer.

La box. Terzo componente, "la mitica" **Box Collision**, scalata perché sia "leggermente unanime" con il bottone, cioè della sua stessa impronta, e alzata sull'asse Z: "ricordatevi, attenzione a sotto". E soprattutto, come per la piattaforma della volta scorsa, **figlia della Platform**: "la box deve seguire la platform. Importante, in modo tale che se io sposto, si sposta con essa" (figura 1.7.3).

È la box a comandare. Una studentessa chiede conferma: "bisogna fare la box sopra perché è la box che interagisce, giusto?" Sì: il player entra con i piedi nella box, la box interagisce, la timeline parte e il pulsante va giù. "Non è tanto la piattaforma o il pulsante a triggerare l'evento, ma la box sopra." E perché non usare l'overlap della mesh stessa? "Perché le collisioni sono **troppo precise**": sono "a contatto con una superficie", "un controllo sempre più micro", "forse sì, forse no", e "la possibilità di errore è molto più ampia". Un volume, invece, dà una risposta macro: "sì, no".

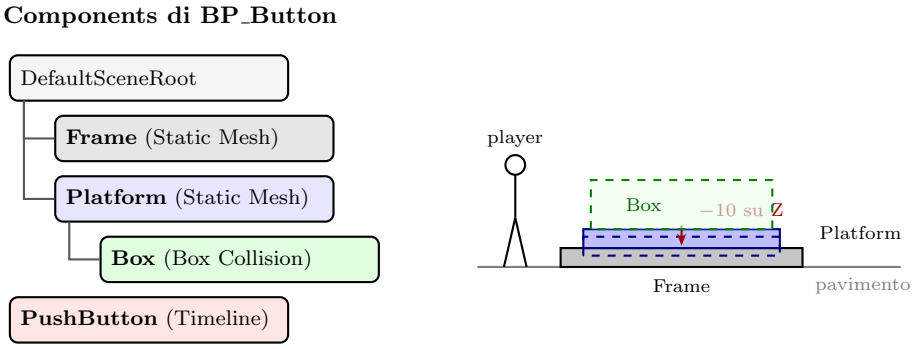


Figure 1.7.3: I componenti del pulsante. A sinistra la gerarchia: Frame e Platform sono due Static Mesh separate; la Box Collision è figlia della Platform, così la segue; la timeline PushButton compare tra i componenti. A destra la vista laterale: la piastra scende di 10 unità dentro la cornice (tratteggio), mentre la cornice resta ferma. Schema ricostruito; le proporzioni non sono quelle dell'asset.

1.7.7 Le collisioni della Static Mesh: simple e complex

"Poi, se non sbaglio, su questo asset c'è un problema." È quello sollevato a inizio lezione: il pulsante, come la piattaforma, potrebbe avere "una collisione un po' tanto alta" per il personaggio.

Il Max Step Height. Nel personaggio del giocatore c'è una variabile, **Max Step Height**, "che non ci permette di andare sopra queste tipologie di asset" se sono più alte di un certo valore. Il docente prova sulla sua scena: a lui va "benissimo", "perché guardate cosa ho fatto io per ovviare a questo problema". A qualcuno degli studenti il personaggio si blocca con i piedi? Pare di no, "bene"; ma "per accortezza", visto che al docente è capitato, si sistema la collisione della mesh.

Approfondimento: Max Step Height e Walkable Floor. I due parametri stanno nel *Character Movement Component* del personaggio. Il **Max Step Height** (predefinito 45 unità, cioè 45 centimetri) è l'altezza massima di uno "scalino" che il personaggio può salire camminando, senza saltare; il **Walkable Floor Angle** (predefinito 44,765 gradi) è la pendenza massima su cui può camminare. Se il bordo della collisione di un oggetto supera lo scalino, la capsula del personaggio ci sbatte contro come contro un muro. Si può alzare lo scalino del personaggio, ma cambia il modo in cui si muove ovunque; di solito conviene sistemare la collisione dell'oggetto, come fa la lezione.

L'editor della Static Mesh. Si seleziona la mesh della piastra e si clicca sulla "finestrella blu", l'icona dell'asset nel campo Static Mesh, per aprire l'**editor della Static Mesh**: "la schermata in cui Unreal fa vedere tutte le particolarità delle collisioni dell'asset stesso", "dati molto più *insight* del modello".

Una parentesi sui LOD. Fra i dati ci sono i **LOD**, i *level of detail*: "quella pratica di abbassare la risoluzione degli asset quando sono distanti, in modo tale da avere ottimizzazione". "Sono tipo a 200 metri da me? Abbassa la qualità, perché pesa." Il corso non li userà, ma è bene sapere che esistono.

Simple e complex. Dal menù **Show** in alto si visualizzano le collisioni dell'asset, e "ci sono due tipologie di collisioni".

- Le **simple**, in **verde**: forme semplici, "usate per avere interazioni di base". Sono quelle che Unreal usa normalmente quando si mette un asset nella viewport, "per tempi e cose di ottimizzazione".
- Le **complex**, "questi lineamenti blu" (azzurri): la forma esatta del modello, triangolo per triangolo. Servono quando "ci serve l'esatto punto" in cui si tocca l'oggetto. Esempio: un **sistema di scalata**, in cui "devo andare a vedere il punto esatto in cui ho messo la mano" per tirare su il personaggio ("poi ci sono anche alternative").

Le due collisioni si preparano "durante la modellazione", e chi segue il corso di 3D art le vedrà. Ma attenzione: "se mettete le complex collision su tutti gli asset, probabilmente potete dire ciao al vostro bellissimo PC", a meno di ottimizzazioni parallele. Qui servono le simple.

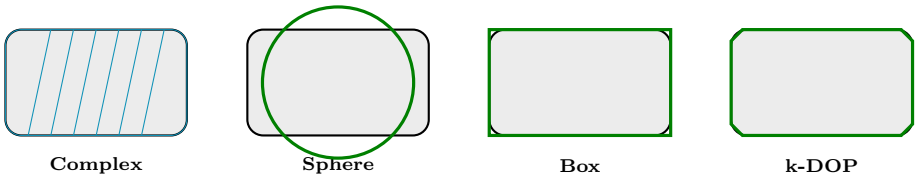
Modificare la collisione. La cosa bella è che le collisioni simple si possono modificare a mano. Cliccando sul "reticoletto verde" si possono muovere, ruotare e scalare come un oggetto qualsiasi. Gli studenti dovrebbero avere una collisione "un po' più arzigogolata", "con tutti i reticolini"; il docente ha già modificato la sua. La procedura:

1. **cancellare** la collisione simple esistente, "quella verde, non quella azzurra";
2. dal menù **Collision** aggiungerne una nuova, scegliendo fra **Sphere**, **Capsule**, **Box** e le varie **k-DOP** (10DOP-X, Y, Z, 18DOP, 26DOP; nella trascrizione "10 di OP meno X");
3. per il pulsante, **Box** ("Box Simplified Collision"): "a noi ci serve una scatoletta, è un quadrato alla fine";
4. selezionare la box e **stringerla** "proprio un'anticchia", abbassandola, dopo aver tolto lo *snap*, "quasi come se fosse una collisione già abbassata".

Ora il pulsante "ha una collisione un po' più affondabile".

Le forme a confronto. Il docente prova le alternative una per una. La **sfera** su una piastra quadrata "non ci serve assolutamente, perché non è coerente alla sua forma". Le **k-DOP** invece seguono il modello con un numero crescente di piani: "sono il livello di dettaglio che la collisione avrà, il livello di vertici"; "più vertici ci sono, più le collisioni sono precise, meno spigolose". Sul cubetto la 10DOP sulla X, sulla Y e sulla Z danno risultati quasi uguali, con "piccole differenze" di taglio; la 18DOP "ha fatto gli angoli più precisi"; la 26DOP, "il massimo", taglia anche gli spigoli (figura 1.7.4).

Un'esperienza di lavoro. "A me è capitato con un lavoro che ho fatto." Nel livello c'era una piccola pista, e alcuni asset avevano collisioni "estremamente complesse": il docente e il gruppo di designer hanno dovuto **cancellare tutte le collisioni e rifarle** a mano, "con i cubettini", per semplificare il lavoro dell'engine. La morale: le collisioni sono parte del lavoro del level designer, non solo del modellatore.



azzurro: la collisione complex (la forma esatta, triangolo per triangolo);
verde: una collisione simple, una forma semplice che approssima il modello

Figure 1.7.4: Collisioni della piastra viste dall'alto, in forma stilizzata. La complex segue ogni triangolo del modello; la Sphere non è coerente con la forma; la Box è la scelta della lezione; le k-DOP aggiungono piani e smussano gli spigoli. Schema ricostruito, non una schermata dell'editor.

Approfondimento: Collision Complexity. Nell'editor della Static Mesh, fra i dettagli dell'asset, c'è l'impostazione **Collision Complexity**. Con il valore predefinito (*Project Default*, in pratica "simple e complex separate") le query fisiche e il movimento dei personaggi usano la collisione simple, mentre i trace che lo chiedono esplicitamente (il flag *Trace Complex*) usano quella complex. Con *Use Complex Collision As Simple* la forma esatta viene usata per tutto: comodo per un pavimento irregolare statico, ma costoso, e incompatibile con la simulazione fisica dell'oggetto. Le k-DOP (*discrete oriented polytope*) sono poliedri convessi che racchiudono la mesh con 10, 18 o 26 piani a orientamento fisso; il menù *Collision* offre anche *Auto Convex Collision*, che scompone la mesh in più pezzi convessi, una via di mezzo fra la precisione della complex e il costo della simple.

1.7.8 La timeline del pulsante

"Fatto questo, ritorniamo sul nostro bellissimo button e andiamo effettivamente a creare quello che sarà il funzionamento di base": il tasto si deve abbassare sulla Z.

Gli eventi e il controllo sul player. "Quello che abbiamo visto l'altra volta": con la Box selezionata, nella sezione degli eventi in fondo ai dettagli, si creano **On Component Begin Overlap** e **On Component End Overlap**, "tutti e due". Poi "la solita cosa": si verifica che sia il player, "con Is Valid o con il tag, ma noi facciamo più semplice". Dall'**Other Actor** si tira fuori un == ("uguale uguale") con **Get Player Pawn**, e il risultato va in un **Branch**.

La timeline. Dal ramo True, "abbiamo già detto che è una timeline quella che useremo": una nuova timeline, rinominata **PushButton**, avviata con **Play from Start** (la differenza con Play si chiarirà nella sezione 1.7.11). Quanto deve scendere il pulsante? Il docente lo misura nella viewport: seleziona la piastra, attiva lo *snap* ("molto meglio"), con passo 10, e la abbassa finché è "totalmente abbassata". Bastano **10 unità**: "quindi abbiamo capito che dovremmo abbassarla di 10".

La curva. Doppio clic sulla timeline. "Il valore da mettere per la lunghezza della timeline è **0,25 secondi**. È molto corto." Poi:

1. una **float track**;
2. un punto (Shift e clic) a **tempo 0, valore 0**;
3. un secondo punto a **tempo 0,25** ("il massimo della nostra timeline") e **valore −10**: "farà un passaggio in 0,25 secondi da 0 a −10";
4. i due pulsantini **Zoom to Fit Horizontal** e **Zoom to Fit Vertical** ("le due freccine"), che adattano il grafico alla lunghezza e all'altezza della curva.

Una domanda dalla chat ("come si chiama il pallino sulla timeline?") riceve la risposta già nota: Shift e clic. Rispetto alla piattaforma della volta scorsa c'è una differenza importante: allora la traccia andava da 0 a 1 ed era un *alfa* da dare a un Lerp; oggi la traccia contiene *direttamente* lo spostamento, in unità del mondo.

Il nome della traccia. "Abbiamo la New Track 0": la si rinomina cliccandoci sopra con **F2**, **PushButtonZ** ("facciamo una Z maiuscola"). Il pin sul nodo prende lo stesso nome; a una studentessa resta scritto *New Track 0* anche dopo la rinomina, e il docente la rassicura: "è solo il nome che gli abbiamo dato", il funzionamento non cambia.

Il movimento. Nell'Event Graph:

- dal pin **PushButtonZ** si crea un **Make Vector**, collegando il valore alla **Z** ("l'asse che ci interessa"); X e Y restano a 0, "tanto la X è zero e rimane zero";
- si trascina la Platform nel grafo e se ne tira fuori **Set Relative Location**, con il vettore in *New Location*. "È relative perché è relativa alla sua transform": lo zero è quello della piastra dentro il Blueprint, non lo zero del mondo;
- il nodo va collegato all'**Update** della timeline.

In Play, sulla pedana, "non esplode, vedete che si abbassa". Ma non si rialza più: "è normale". Manca il ritorno: dall'**End Overlap**, con lo stesso controllo sul player, si collega l'ingresso **Reverse** della timeline. "Premuto, torna su; premuto, torna su." "Eccolo che abbiamo creato il nostro movimento del pulsante, non ancora la sua funzionalità" (figura 1.7.5).

Approfondimento: perché qui niente Lerp. La sesta lezione aveva mostrato il metodo "giusto" per la piattaforma: una traccia da 0 a 1 usata come alfa di un Lerp fra due posizioni salvate. Qui il docente sceglie la strada diretta, e funziona per due ragioni. La prima: il valore della traccia è *assoluto*, non si somma alla posizione corrente, quindi non si accumula (era il difetto del primo metodo della volta scorsa). La seconda: la piastra è figlia di una radice ferma, e la sua posizione relativa di partenza è (0, 0, 0), quindi "−10 rispetto alla radice" vuol dire esattamente "10 più in basso di dove sta". Se la piastra fosse stata spostata nel Blueprint, per esempio a Z 5, il primo Update la teletrasporterebbe a Z 0: in quel caso serve sommare lo spostamento alla posizione iniziale salvata, o tornare al Lerp. Per oggetti che si spostano di poco, una traccia in unità del mondo è la soluzione più leggibile.

1.7.9 Le curve: linear e auto

"Adesso vi faccio vedere anche una particolarità, che potremmo usare anche per la piattaforma": si può **modificare la tipologia dei punti**.

Lineari. I punti creati finora sono **lineari**: "formano una linea", e l'aumento o la diminuzione del valore "è sempre costante" nel tempo: in ogni frazione di secondo il valore cambia sempre della stessa quantità, "tra un punto e l'altro".

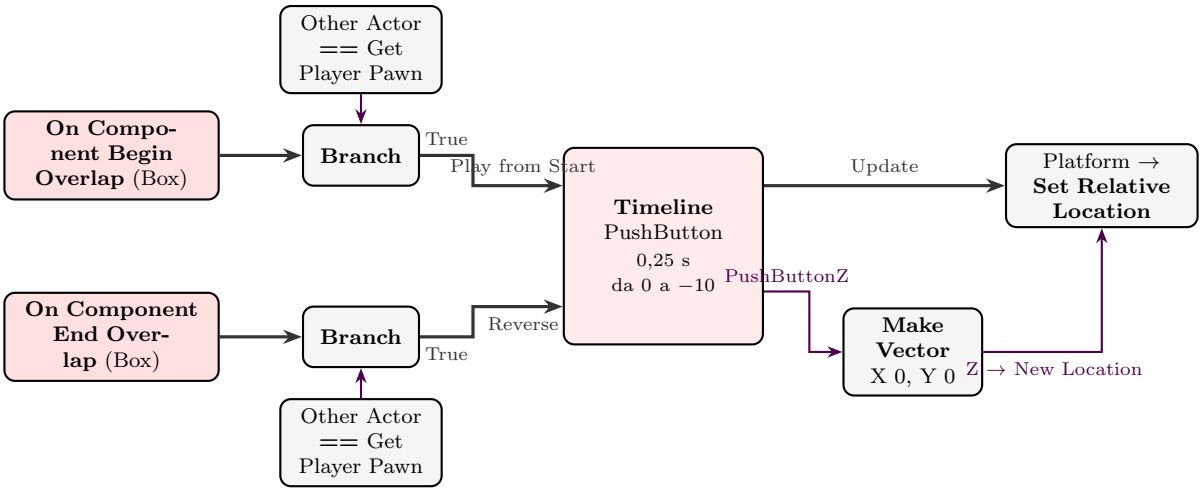


Figure 1.7.5: Il movimento del pulsante. Il Begin Overlap della Box, se l'actor è il player, fa partire la timeline PushButton da capo; l'End Overlap la manda in Reverse. A ogni Update il valore della traccia (da 0 a −10 in 0,25 secondi) diventa la Z di un vettore, e la piastra si sposta con Set Relative Location. Schema ricostruito.

Curve. "Se invece vogliamo una curva un po' più *smooth*", si fa tasto destro su un punto: compare la serie di **Key Interpolation**, "cioè ti dice: quale tipo di punto vuoi che sia?". Il docente sceglie **Auto**, e la linea diventa una curva; lo stesso sull'altro punto. Ora, con i "piccoli braccetti" (le tangenti) di ciascun punto, si può modellare "come nel tempo questa linea va da punto A a punto B" (figura 1.7.6, tabella 1.7.1).

A che cosa serve. Una studentessa chiede che cosa cambi in pratica. Il docente prova: allunga la timeline a 1 secondo e piega la curva in modo che superi il valore finale e poi ci ritorni. Il risultato, con un'escursione di appena 10 unità, sono "micro movimenti": il pulsante "fa boom e ritorna su", "su e giù e si abbassa". "Cambiare la forma della curva ci permette di creare dei piccoli intervalli di vario tipo": il movimento rallenta, accelera, rimbalza. Su un pulsante si vede poco ("è molto piccolo come spazio"); "magari in pausa" il docente propone di mostrarlo con "una piccola sferetta che fa su e giù". Poi rimette la curva com'era.

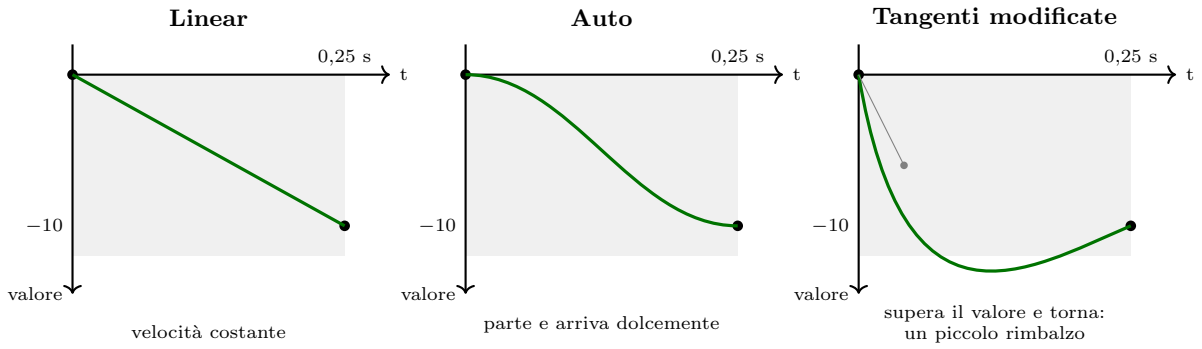


Figure 1.7.6: La stessa traccia con tre interpolazioni. Con le key lineari il pulsante scende a velocità costante; con Auto la curva è morbida agli estremi; trascinando le tangenti (i "braccetti") si ottengono andamenti come il rimbalzo provato a lezione. Schema ricostruito.

Approfondimento: *easing* e *game feel*. Le curve sono lo strumento più economico per dare *peso* a un movimento. In animazione si parla di *ease in* e *ease out* (partire e arrivare lentamente) e di *overshoot* (superare la meta e tornare indietro, come il rimbalzo provato dal docente): una porta che rallenta prima di

Key Interpolation	Andamento fino alla key successiva
Auto	curva morbida, con tangenti calcolate in automatico (la scelta della lezione)
User	curva morbida con tangenti impostate a mano trascinando i "braccetti"
Break	come User, ma le due tangenti del punto si muovono in modo indipendente (spigoli voluti)
Linear	retta: variazione costante nel tempo (il comportamento predefinito dei punti creati con Shift e clic)
Constant	il valore resta fermo e salta alla key successiva al suo arrivo (a gradino)

Table 1.7.1: I tipi di interpolazione di una key, dal menù del tasto destro su un punto della timeline. La lezione usa Linear e Auto; le altre voci sono un'integrazione basata sull'editor di Unreal Engine 5.

chiudersi, un pulsante che affonda di colpo e risale con un piccolo rimbalzo, un ascensore che parte dolcemente. A parità di durata, un movimento lineare appare meccanico, uno con l'*easing* appare fisico. Nelle timeline la curva è anche un *asset* riutilizzabile: dall'editor la traccia si può salvare come **Curve Float** esterna e usare in più timeline, così tutti i pulsanti del gioco si muovono allo stesso modo e si correggono in un punto solo.

1.7.10 I problemi degli studenti con il pulsante

"Il mio bottone non scende." "Neanche a me." Il docente ripassa il grafo (Platform, Set Relative Location sull'Update, il pin della traccia nella Z del Make Vector) e poi chiama le studentesse in condivisione, una alla volta, "Lucrezia per prima". La condivisione dello schermo su Zoom richiede diversi tentativi, fra permessi da dare, barre degli strumenti che spariscono e un'eco "quasi un boomerang"; alla fine funziona.

- **Il volume.** La box è "molto piccola" e il volume "troppo basso": deve stare sopra la piastra.
- **I punti della curva.** Il primo punto non sta a tempo 0: "devi mettere tempo zero, valore zero"; il secondo a 0,25 e -10 "va benissimo". E le due freccine dello Zoom to Fit aiutano a vedere davvero la curva.
- **La lunghezza.** Un'altra studentessa ha messo 0,25 nel tempo del punto ma ha lasciato la **Length** della timeline al valore predefinito: "devi modificare la lunghezza; vedi, c'è scritto *Length* sopra".
- **Stop invece di Reverse.** Sull'End Overlap era collegato *Stop*: il pulsante si ferma dov'è invece di tornare su.
- **L'Update scollegato.** Il caso più istruttivo: il Set Relative Location aveva il pin dei dati collegato, ma non quello di **esecuzione** all'Update. "Qui devi collegare solo l'execution pin." "Ricordati, fai conto che l'Update è quello che fa" il lavoro nel tempo della linea del tempo.

Un consiglio di contorno: "spostati un po' gli asset" di prova delle lezioni precedenti, che potrebbero dare fastidio ai "grubetti" (forse i cubetti della matrice).

Approfondimento: dati senza esecuzione. L'ultimo errore è il più tipico di chi viene da altri strumenti. Nei Blueprint i **pin di dati** (colorati) dicono *con che cosa* lavora un nodo, i **pin di esecuzione** (bianchi) dicono *quando* il nodo viene eseguito. Un nodo con i soli dati collegati non viene mai eseguito: i suoi ingressi verrebbero calcolati se qualcuno lo chiamasse, ma nessuno lo chiama. Fanno eccezione i nodi *pure*, senza pin bianchi (come Make Vector o Get Player Pawn), che vengono calcolati quando un nodo eseguito ne legge l'uscita. Il controllo da fare quando "non succede niente" è quindi sempre lo stesso: seguire il filo bianco dall'evento fino all'ultimo nodo.

1.7.11 Play o Play from Start?

"Posso fare una domanda rapidissima? Perché abbiamo usato Play from Start e non Play?" La risposta del docente, con un esempio di timeline da 0 a 2 secondi.

- Con **Play**, se ci si ferma a 1 secondo (si esce dal pulsante quando la piastra "sta a metà") e poi si rientra, la timeline "continuerà da quel punto in particolare dove ci siamo fermati": da 1 a 2.
- Con **Play from Start**, se si esce e si rientra, la timeline "riparte da 0, non da 1".

Nel caso del pulsante, dice il docente, "non abbiamo particolari interruzioni, perché abbiamo proprio un *in* e *out*".

Approfondimento: che cosa succede davvero con il Reverse. Con un pulsante così breve la differenza si nota poco, ma esiste, ed è utile capirla con la figura 1.7.7. Si entra, la timeline va avanti; si esce a metà, e il Reverse la fa tornare indietro *da dove si trova*. Se si rientra prima che sia tornata a 0:

- con **Play** la timeline inverte di nuovo la direzione dal punto in cui è, e la piastra riprende a scendere senza scatti;
- con **Play from Start** la timeline salta a 0: la piastra torna di colpo in alto e riparte a scendere, un piccolo scatto visibile.

Per oggetti che vanno avanti e indietro seguendo un ingresso e un'uscita (pulsanti, porte, piattaforme) la coppia più naturale è quindi **Play** e **Reverse**; è proprio quella che il docente raccomanderà nell'esercizio finale (sezione 1.7.24) e che correggerà nella soluzione di uno studente ("mettilo su Play, non Play from Start"). Play from Start serve quando ogni attivazione deve ripetere l'animazione da capo, per esempio un colpo o un lampo.

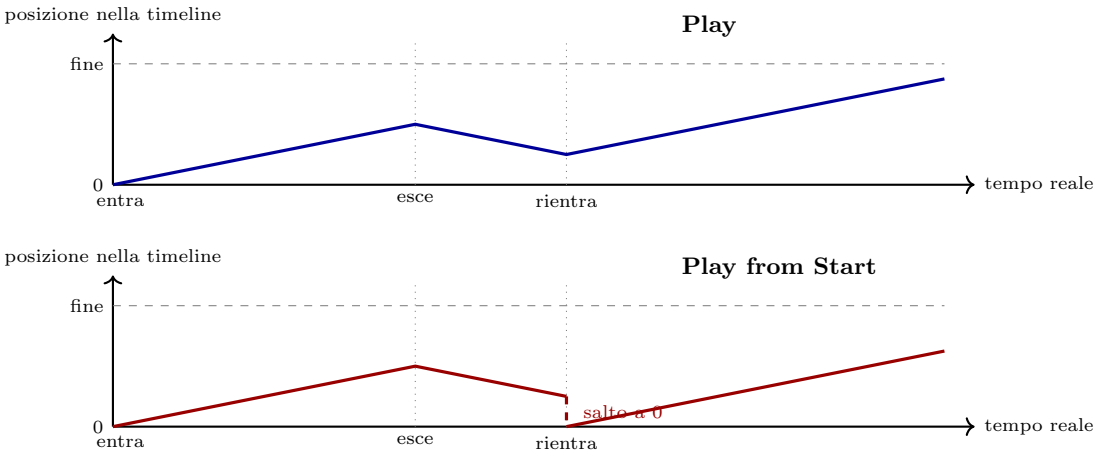


Figure 1.7.7: Entrare, uscire a metà e rientrare prima che la timeline sia tornata a 0. Con Play (in alto) la timeline riprende da dove si trova; con Play from Start (in basso) salta all'inizio, e l'oggetto fa uno scatto. Schema ricostruito come integrazione alla risposta del docente.

1.7.12 Gli attivabili: la classe BP_Activable

"Adesso andiamo a fare una cosa bellina." Il pulsante si muove, ma non fa ancora niente. Prima di creare la luce, "prima, importante", serve una **tipologia di actor**: quella degli **attivabili**, "quelli che per noi saranno gli attivabili".

La classe base. Un nuovo Blueprint Class di tipo Actor: il docente lo chiama BP_Activable1, qui è semplicemente **BP_Activable**. "Che succede qui all'interno? Ci serve solo questo": con la sola creazione "abbiamo intanto definito una tipologia di actor". Dentro, due **Custom Event**:

- **Activation**, collegato a un Print String "attivato";

- **Deactivation**, collegato a un Print String "deattivato" (cioè disattivato).

"Questi due eventi saranno collegati all'attivare qualcosa o disattivare qualcosa." Le due stampe sono provvisorie: servono a vedere che il collegamento funziona, e poi a capire come i figli possono *sovrascrivere* gli eventi del padre (sezione 1.7.16).

La domanda. "Quando è che io lo devo chiamare? E soprattutto, come faccio a fargli dire: ehi, bottone, guarda che devi attivare questo esatto elemento?" La risposta è una variabile nel bottone, ed è il cuore della lezione.

1.7.13 Il bottone e la lista degli attivabili

Una variabile di tipo BP_Activable. Nel Blueprint del bottone si crea una variabile, **ActivableItems**, e se ne cambia il tipo. "Questo è molto interessante, ragazzi: possiamo creare una variabile del tipo di quell'asset che abbiamo appena creato": il tipo è **BP_Activable**, nella forma *Object Reference*. Una variabile così non contiene un numero o un testo, ma un **riferimento** a un attivabile presente nel livello.

Pubblica. "Facciamola **pubblica**, e si fa con questo **occhietto**" accanto alla variabile. Il docente mette nel livello un BP_Activable e seleziona il pulsante: nei Details, in *Default*, "è apparso Activable Items". "Vuol dire che questa variabile è modificabile dai Details del nostro editor", e quindi "ci permette di avere interazioni dirette con la viewport, l'ambiente di gioco": ogni copia del pulsante nel livello può avere i suoi oggetti da attivare.

Un array. "All'inizio di lezione abbiamo parlato degli array": il pulsante non deve attivare un oggetto solo, "ma una moltitudine di oggetti". Si cambia quindi non la *tipologia* della variabile, ma la sua **costituzione** ("cerchiamo di dividerli", precisa il docente): accanto al *Variable Type* c'è un'icona ("questo cilindro, vabbè, un'ellisse") con una freccetta, e il menù offre **Single**, **Array**, **Set** e **Map** (tabella 1.7.2). Si sceglie Array, e l'icona della variabile cambia: "diventa una matrice", una griglia di quadratini. "Adesso abbiamo una variabile array di tipologia BP_Activable."

Costituzione	Che cosa contiene	Esempio
Single	un valore solo (la "scatola")	la referenza a una sola luce
Array	una lista ordinata di valori dello stesso tipo, con indice da 0; ammette doppioni	gli oggetti attivati dal pulsante (ActivableItems)
Set	un insieme di valori unici, senza ordine e senza indice	i nomi degli oggetti già raccolti
Map	coppie chiave–valore, con chiavi uniche	il nome di un'arma e le sue munizioni

Table 1.7.2: Le quattro costituzioni di una variabile, dal menù accanto al Variable Type. La lezione usa Single e Array; le descrizioni di Set e Map seguono Romero e Sewell (capitolo 13).

Quando chiamare gli attivabili. "Vogliamo che quando il nostro bel bottone finisce di fare il suo percorso, e si schiaccia", partano le attivazioni: si usa l'uscita **Finished** della timeline. Ma Finished scatta sia alla fine della discesa sia alla fine della risalita (dopo il Reverse), quindi bisogna sapere "che direzione è". "Vi ricordate che abbiamo usato lo Switch l'altra volta sulla Direction?" Dal pin **Direction** si crea uno **Switch on ETimelineDirection**, collegato al Finished:

- **Forward**: la timeline è andata avanti, il pulsante è schiacciato, quindi **attiva**;
- **Backward**: la timeline è tornata indietro, il pulsante è tornato "alla sua posizione di prima", quindi **disattiva**.

Il For Each Loop. "Abbiamo già visto il For Loop. Questo è il **For Each Loop**." Si prende un *Get* di ActivableItems e se ne tira fuori il ciclo: "questa esecuzione verrà fatta per ogni elemento dell'array che abbiamo inserito qua dentro"; "abbiamo cinque elementi dentro l'array, verrà fatto cinque volte". Il nodo ha due uscite di dati: l'**Array Element**, "il singolo elemento che ha preso in considerazione nel momento preciso del ciclo", e l'**Array Index**. Dall'Array Element, che è di tipo BP_Activable, si può chiamare direttamente l'evento **Activation**, collegato al **Loop Body**. Per il ramo Backward "i primi due li prendete, li copiate, li mettete sotto": stesso Get, stesso For Each Loop, ma con **Deactivation** (figura 1.7.8).

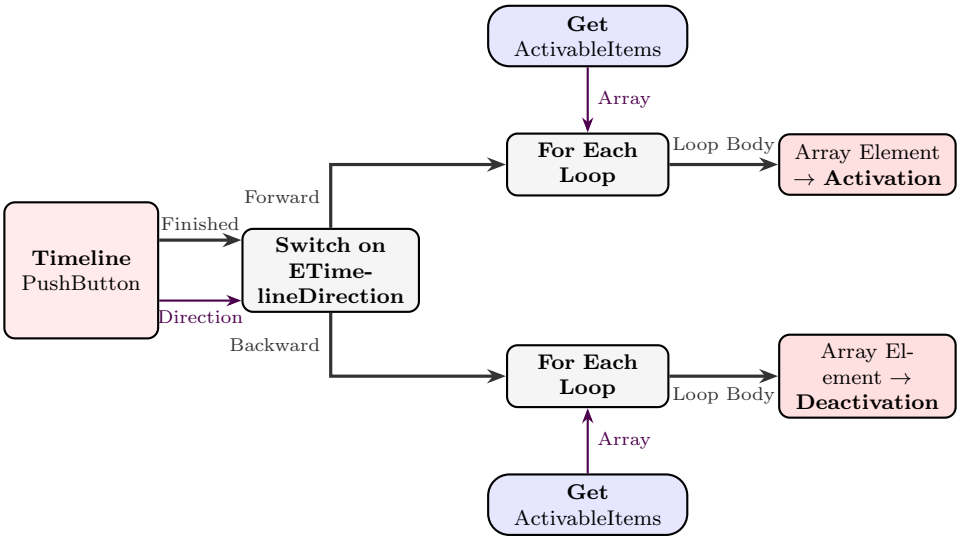


Figure 1.7.8: Il pulsante chiama gli attivabili. Alla fine della timeline, lo Switch sulla Direction sceglie il ramo: in avanti (pulsante schiacciato) un For Each Loop chiama Activation su ogni elemento di ActivableItems; all'indietro (pulsante rilasciato) un secondo ciclo chiama Deactivation. Schema ricostruito.

Riempire l'array. "Ci manca ancora una piccola particolarità": la variabile è pubblica perché gli oggetti da attivare si scelgono *nel livello*. Si clicca sul pulsante nella scena, e in *Default* c'è Activable Items: con il + si aggiungono gli elementi, "tutti quelli che volete". Prima, però, un'avvertenza: "in informatica si parte non da uno, ma da **zero**"; "l'indice 0 è il primo indice". Il primo elemento dell'array è l'elemento 0. Per assegnargli un oggetto ci sono due modi:

- il **menù a tendina**, che elenca "tutti gli oggetti che puoi ficcarci dentro questo array", cioè tutti i BP_Activable presenti nel livello, "perché giustamente è quello";
- il **contagocce** (*pick actor from scene*), "proprio come su Photoshop": si clicca sull'icona e si passa sopra gli oggetti della scena; sugli oggetti di un altro tipo l'editor avvisa che "non è compatibile", sul BP_Activable invece "è compatibile, ci può entrare dentro". Un clic, "ecco che è apparso".

In Play, sul pulsante: "attivato"; si scende: "disattivato". "Vi funziona?"

I problemi. A una studentessa, sul pulsante non compare la voce dell'array: "basta che vai dove sono le variabili: c'è l'occhietto. Hai l'occhietto chiuso?". Si apre l'occhio, si compila ("se no non va"), e la voce compare. Poi bisogna **mettere in scena** l'attivabile, se non c'è, e collegarlo con il contagocce ("c'è la freccettina, c'è questo mouse con dei riquadri, e subito dopo c'è il contagocce"). "Sì, funziona. Ottimo."

"Quindi, ragazzi, adesso abbiamo creato il funzionamento base", cioè "la relazione tra pulsante e oggetto". L'oggetto non ha ancora particolari funzionalità, "se non stampare due frasi", "tuttavia la base ce l'abbiamo".

Approfondimento: la comunicazione diretta. Quello costruito è ciò che Romero e Sewell (capitolo 4) chiamano **Direct Blueprint Communication**: un Blueprint tiene una variabile referenza a un altro

actor, e la usa come *Target* per chiamarne le funzioni e gli eventi. Il loro esempio è sorprendentemente vicino a quello della lezione: un *BP_LightSwitch* con una variabile di tipo Point Light, resa **Instance Editable** (l'"occhietto" del docente: ogni istanza nel livello può avere un valore diverso), che a ogni overlap del giocatore accende o spegne una lampada scelta nei Details dal menù a tendina. La lezione fa un passo in più in due direzioni: la referenza è un *array*, quindi un pulsante comanda molti oggetti; e il tipo è una classe *parent* scritta da noi, quindi il pulsante non sa e non deve sapere se l'oggetto è una luce o una piattaforma. Il libro aggiunge un consiglio che vale anche qui: prima di usare una referenza, controllarla con **Is Valid**, perché un elemento dell'array lasciato vuoto nei Details (il + premuto senza scegliere niente) produrrebbe l'errore "Accessed None" visto a inizio lezione. Lo stesso capitolo 13 mostra il For Each Loop proprio su un array di Point Light, "le lampade di una stanza".

1.7.14 Il figlio: BP_ActivableLight e i due Begin Play

"A questo punto andiamo a evolvere questo attivabile in una luce che si accende e si spegne." Ma, attenzione: "non andiamo a creare un nuovo Blueprint, ma andiamo a creare un **figlio** di questo Blueprint, perché vogliamo che condivida la particolarità di avere le fasi di attivazione e disattivazione".

Create Child Blueprint Class. Tasto destro su BP_Activable nel Content Browser, **Create Child Blueprint Class** ("lo trovate tutto sopra"). Unreal propone un nome con il suffisso *Child*; il docente lo chiama **BP_ActivableLight**. Aprendolo, in alto a destra nell'editor si legge già il **Parent class**: "guarda, questo è il parent, questo è il suo papà; deriva da questo". Una studentessa non trova il comando: va fatto con il tasto destro proprio su BP_Activable, non sulla pedana.

Event BeginPlay e Parent: BeginPlay. Aprendo il figlio, al docente compaiono nel grafo due nodi: **Event BeginPlay** e, collegato a esso, un nodo **Parent: BeginPlay**. Agli studenti no: il docente aveva cancellato il Begin Play dal parent. La prova: nel parent si rimette un **Event BeginPlay** (con un Print String, "così si vede bene"), e il figlio creato dopo li mostra entrambi. Il significato: "io eseguirò il tuo Event BeginPlay assieme all'Event BeginPlay del mio papà". "Va a eseguire in sostanza due Begin Play."

La dimostrazione. Nel figlio il Begin Play non contiene niente, a parte la chiamata al parent; eppure, in Play, la stampa del parent compare. "Questo è il child. Qua dentro non c'è un Print String, non c'è niente. Ma lui andrà a fare il Begin Play del suo papà." Se si **stacca** il nodo Parent: BeginPlay, "non mi stampa più niente". Lo stesso varrebbe per ogni altro evento: "se dovessimo fare un Event Begin Overlap, avremmo la stessa identica cosa", cioè la domanda "vuoi che ti esegua anche quello del tuo papà? Sì o no?". Il nodo si può anche cancellare: "questo era per spiegare la **condivisione degli eventi**".

Approfondimento: estendere o sostituire. Il nodo *Parent: BeginPlay* è la chiave per capire l'override. Quando un child definisce un evento che esiste già nel parent, la sua versione *sostituisce* quella del parent; il nodo *Parent: ...*, collegato al filo di esecuzione, la richiama in un punto preciso. Collegato all'inizio, il child fa prima ciò che fa il parent e poi aggiunge il suo (*estende*); collegato alla fine, aggiunge prima il suo; scollegato o cancellato, il child fa solo il suo (*sostituisce*). Unreal lo crea in automatico quando il parent ha già quell'evento, come gentile promemoria; se manca, si aggiunge con il tasto destro sul nodo dell'evento, **Add Call to Parent Function**. Servirà nella sezione 1.7.23.

1.7.15 La luce: la Point Light

"Andiamo a mettere questa bella lucina." Il docente illumina un po' il suo asset "perché così non si può vedere", e aggiunge al figlio un componente **Point Light**, la luce puntiforme che illumina in tutte le direzioni: "la potete vedere che si è creato un reticolo". I parametri principali, uno alla volta, in una piccola finestra di prova:

- **Intensity**: "l'intensità della luce: aumenta, diminuisce";

- **Attenuation Radius**: il raggio disegnato attorno alla luce, "quanto si espande"; se lo si allarga, il muro vicino viene illuminato;
- **Source Radius**: "il raggio della sua sorgente", il "pallino" che si vede al centro quando si guarda una luce;
- **Soft Source Radius** e **Source Length** ("la lente, volendo, si evolve in verticalità: una specie di lampione tutto strano"), "una cosa un po' più particolare";
- **Light Color**: con la ruota dei colori, il cursore del chiaro e scuro e quello della "tendenza al bianco" (la saturazione). Il docente la fa **blu**, "per differenziarla dall'altra".

L'intensità finale è **20.000** (il docente dice prima "200.000", poi si corregge: "20.000, scusate"). "Questo era semplicemente un piccolo cenno sulle luci."

Approfondimento: unità e mobilità delle luci. In Unreal Engine 5 l'intensità di una Point Light si misura di solito in **candele** (o in lumen), e il valore predefinito di una luce nuova è basso; 20.000 nella vecchia unità adimensionale (*Unitless*) è invece una luce molto forte, e il numero giusto dipende dall'unità impostata nel campo *Intensity Units* e dall'esposizione della camera. C'è poi una condizione per poter cambiare una luce *mentre il gioco gira*: la sua **Mobility** deve essere *Movable* (o *Stationary* per intensità e colore), non *Static*, perché le luci statiche vengono "cotte" nelle lightmap e non esistono più come luci a runtime. Romero e Sewell lo fanno notare nell'esempio del loro interruttore: la lampada va resa Movable. Con Lumen, il sistema di illuminazione dinamica predefinito di Unreal Engine 5, le luci sono in genere già Movable.

1.7.16 Salvare l'intensità e sovrascrivere gli eventi

"Ci serve, per mettere i due stadi di activation e deactivation, prima di tutto" un passaggio nel **Begin Play**. La luce deve partire spenta, e accendersi poi con l'intensità scelta nell'editor (20.000). Ma se all'inizio la si spegne, quel numero andrebbe perso.

IntensityStart. "Vi ricordo che tutte le cose che vedete" nei Details "sono tutte modificabili, sono tutte richiamabili." Dal componente Point Light si tira fuori **Get Intensity** ("stiamo prendendo l'intensità attuale") nel Begin Play, "all'inizio, quando il frame esiste", e con **Promote to Variable** si salva in una nuova variabile, **IntensityStart**: "una variabile di **appoggio**", che serve solo a ricordare il valore da riproporre quando la luce si accenderà. "Non serve il Parent: BeginPlay", aggiunge il docente, "perché tanto nel parent non facciamo niente."

Il problema delle stampe. Se ora si chiamano gli eventi Activation e Deactivation sulla luce, la luce "rimane accesa, è normale", e in più il figlio "condividerà il fatto" di stampare "attivato" e "deattivato": ha ereditato gli eventi del padre così come sono. "Noi non vogliamo questo: noi vogliamo semplicemente che la luce si spenga."

L'override. "Come facciamo a sovrascrivere questo evento?" Nel pannello My Blueprint, accanto a *Functions*, c'è il menù **Override**: elenca gli eventi e le funzioni che si possono ridefinire, quelli di Actor ("Actor-BeginCursorOver", "BeginPlay", "Reset" e così via) e, in fondo, **Activation** e **Deactivation**, ereditati da BP_Activable. Scegliendoli, compaiono nel grafo del figlio. "Che vuol dire override? Vuol dire che stiamo **sovrascrivendo**" gli eventi. Prova: in Play, con i due eventi vuoti, "non mi fa più niente. È giusto così": la versione del figlio ha preso il posto di quella del padre. "Possiamo sovrascrivere vari eventi all'interno dei child, in modo tale da facilitarci la differenziazione dei comportamenti."

Accendere e spegnere.

- Su **Activation**: dalla Point Light, **Set Intensity** ("invece di get, set"), con *New Intensity* collegato a **IntensityStart**.
- Su **Deactivation**: Set Intensity a **0**.

Il Target dei due nodi è la Point Light, "perché stanno qui", nel figlio.

L'ordine nel Begin Play. Resta da spegnere la luce all'avvio: nel Begin Play, "subito dopo il salvarsi la variabile", si chiama **Deactivation**. "Perché non lo facciamo prima? Perché se facessimo Deactivation prima, il valore che si salverebbe è zero" (figura 1.7.9). In Play: la luce, accesa nell'editor, "si spegne perché l'ho disattivata"; si sale sul pulsante, "adesso si accende". "Abbiamo fatto il nostro bottone con una pre-impostazione tra trigger e triggerato."

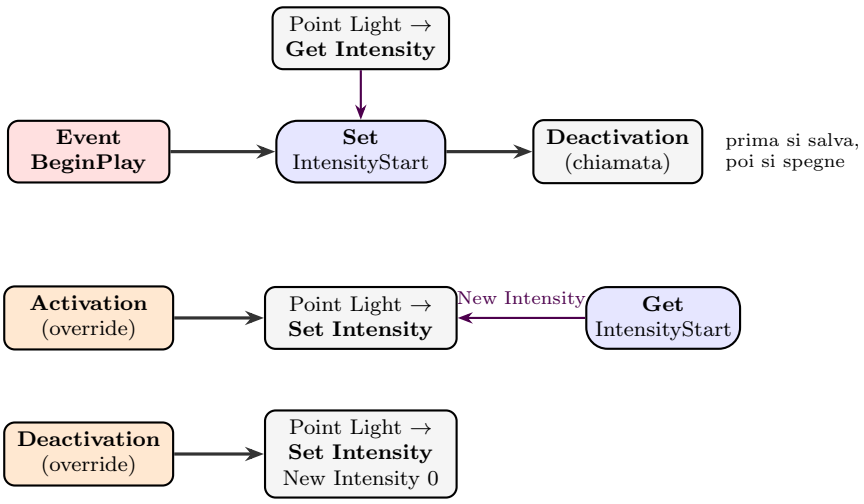


Figure 1.7.9: Il grafo di BP_ActivableLight. Nel Begin Play l'intensità scelta nell'editor viene salvata in IntensityStart e solo dopo la luce viene spenta; i due eventi ereditati da BP_Activable sono sovrascritti (override) e accendono o spengono la Point Light. Schema ricostruito.

I problemi.

- A uno studente la luce resta accesa, e in più compaiono ancora le scritte "attivato" e "deattivato". Il docente ripete la sequenza; lo studente ritrova l'errore da solo: "avevo collegato per sbaglio IntensityStart pure a quello sotto", cioè anche al Set Intensity della Deactivation, che quindi riaccendeva la luce invece di spegnerla.
- A un'altra studentessa, quando preme Play, "non si vede più la luce": è giusto, la luce parte spenta. Ma poi non si accende, e appare solo la stampa. Causa: il pulsante nel livello è ancora collegato al BP_Activable di prova, non alla luce. "Prendi il contagocce e clicca su BP_ActivableLight."

Approfondimento: gli eventi di BP_Activable come contratto. Il risultato di queste sezioni è un piccolo esempio di *polimorfismo*, il concetto che Romero e Sewell illustrano con le armi che ridefiniscono *Fire*. Il pulsante chiama Activation su ogni elemento dell'array senza sapere che cosa ci sia dentro; ogni oggetto risponde con la propria versione dell'evento. Per aggiungere al gioco un nuovo oggetto attivabile (una porta, un ventilatore, una trappola) non serve toccare il pulsante: basta un nuovo child di BP_Activable che sovrascriva i due eventi. Il parent fa da *contratto*: "chi è attivabile sa rispondere ad Activation e Deactivation". È anche il motivo per cui conviene lasciare il parent quasi vuoto: le due stampe di prova sono utili oggi, ma in un progetto vero un evento del parent che stampa qualcosa finirebbe per stampare in ogni figlio che non lo sovrascrive.

1.7.17 La pausa e il suono della piattaforma

"Ragazzi, sono le 5 e mezza." Il programma della seconda parte: la piattaforma che si attiva con il pulsante, "magari un po' come esercizio", e "un altro metodo, che è quello degli **event dispatcher**". "Ci vediamo al solito orario, 17:55, 18."

Durante la pausa il docente mantiene la promessa fatta a inizio lezione, e completa con lo studente rimasto indietro l'**audio della piattaforma** della sesta lezione, in condivisione dello schermo. È un ripasso utile, perché raccoglie in un unico passaggio l'ordine corretto dei nodi.

1. Il file audio è già importato e impostato (il docente lo verifica aprendolo).
2. In fondo al grafo, dopo l'Update della timeline, si toglie il Print String di prova e si aggiunge un **Do Once**.
3. Dopo il Do Once, **Spawn Sound at Location**, con la *Location* presa da **Get Actor Location** ("spazio vuoto, tasto destro"), collegata al pin arancione e non a quello azzurro, e il suono della piattaforma nel campo *Sound*.
4. Dal *Return Value* ("quel pallino blu"), **Promote to Variable**: **SoundPlatform**.
5. Solo ora, dal pin **Direction** della timeline, lo **Switch on ETimelineDirection**, collegato dopo il Set della variabile ("portalo tutto in avanti, oltre quello"): su **Forward**, SoundPlatform → **Set Pitch Multiplier** a 1; su **Backward**, una copia con pitch **0,4**.
6. Sul **Finished**: SoundPlatform → **Stop**, e dallo Stop il **Reset** del Do Once.

"Si sente l'audio. A posto, perfetto." La registrazione della pausa, per il resto, è coperta dai soliti blocchi senza senso della trascrizione automatica.

1.7.18 La correzione del pitch

Alla ripresa, "una piccola cosa per Filippo", che a inizio lezione aveva segnalato il bug del suono. Il problema: lo Switch sulla Direction viene eseguito solo dopo lo spawn del suono, che grazie al Do Once avviene una volta sola, all'inizio del movimento. Se il giocatore scende dalla piattaforma mentre sta ancora salendo, la timeline va in Reverse, ma nessuno rimette il pitch: "partiva e lo faceva solo una volta, quindi non si trova mai il pitch".

La correzione è semplice: "il pitch del suono va settato in automatico sia se va backwards, sia se esce dal volume". Sull'**End Overlap**, prima del **Reverse**, si aggiunge SoundPlatform → **Set Pitch Multiplier** a **0,4**: "nel momento esatto in cui esce dal volume, settiamo il pitch a 0,4", "anche se in mezzo alla piattaforma mi lancio". "E siamo tranquilli."

Approfondimento: lo stesso errore, di nuovo. Il nuovo Set Pitch Multiplier ha lo stesso rischio dell'errore di inizio lezione: se il giocatore entra ed esce dalla box prima che la timeline abbia prodotto il primo Update, SoundPlatform è ancora vuota, e l'End Overlap la usa. In pratica capita di rado, ma una macro **Is Valid** sulla variabile, prima del Set, lo esclude del tutto. Più in generale, un suono il cui pitch deve seguire la direzione del movimento si gestisce meglio leggendo la Direction della timeline a ogni Update (o, ancora meglio, chiamando Set Pitch Multiplier ogni volta che si chiamano Play o Reverse), invece che una volta sola all'inizio.

1.7.19 Un altro metodo: gli event dispatcher

"Allora, ragazzi, eravamo rimasti con il nostro bottoncino che ci dava la possibilità di accendere e spegnere una luce. Possiamo usare questo metodo, però volendo potremmo anche vedere un altro modo": gli **event dispatcher**. Si trovano nel pannello My Blueprint del bottone, sotto *Variables*, nella sezione **Event Dispatchers**.

Che cosa sono. "Gli event dispatcher, a differenza dei custom event, ci permettono di **bindare** vari eventi tra di loro"; "bindare sta per collegare". L'esempio del docente: tre custom event, A, B e C. Con un event dispatcher si possono legare A, B e C tutti insieme a un altro evento, oppure solo A e C: "a un evento di salto succede che si salta; voglio che mi faccia, assieme al salto, A e C". Così si tengono separati, per esempio, un controllo e uno sparo, e li si fa partire insieme quando serve: "guarda che quando salta, bindami questo evento allo sparo o al controllo". "È un modo un po' più ordinato per fare le cose." Lo scopo, oggi, resta lo stesso: accendere e spegnere la luce con il bottone.

Il dispatcher e il suo parametro. Con il + accanto a Event Dispatchers si crea **OnCheckStatus** (nella trascrizione "on share status" e "check status"; il nome esatto non è del tutto sicuro, ma è quello che il docente usa nel resto della spiegazione). Come i custom event, anche un dispatcher può avere *input*: nei Details, con il + della sezione *Inputs*, si aggiunge un *New Parameter*, rinominato **IsActive**. Il tipo proposto di base è proprio il **Boolean**, "e va benissimo così": vero vuol dire che il bottone è attivato, falso che è disattivato.

A parti invertite. "Ma in questo caso andremo a fare una cosa differente. Invece che il bottone chiami la luce, sarà la luce a controllare lo stato del bottone: a parti inverse." Il pulsante non avrà più bisogno di sapere quali oggetti attivare; saranno gli oggetti ad *ascoltare* il pulsante. Il docente, per un attimo, stacca Activation e Deactivation dal resto; poi ci ripensa: "li useremo comunque in ogni caso, perdonate".

1.7.20 La luce si mette in ascolto: Get Actor Of Class e Bind Event

Nel **Begin Play** di BP_ActivableLight (dopo il salvataggio dell'intensità) la luce deve trovare il pulsante e "attaccarsi" al suo dispatcher.

Get Actor Of Class. "Andiamo a ricercare in tutto il gioco, cioè in tutto lo spazio di gioco, un attore in particolare." Il nodo **Get Actor Of Class** riceve una *Actor Class*, qui **BP_Button**, e restituisce un actor di quella classe presente nel livello.

Bind Event. Dal valore restituito si tira fuori **Bind Event to OnCheckStatus**. Il nodo appena creato dà errore: "è normale, ho fatto compilare prima del tempo". Il suo significato, "letteralmente": "attacca un evento". Quale? Dal pin rosso *Event* del nodo si tira fuori un filo e si crea un **Custom Event**, chiamato **CheckStatusEvent**. "Guardate la cosa interessante": il nuovo evento ha *già* un'uscita **IsActive**. Un custom event normale nasce vuoto; questo nasce con la stessa *firma* del dispatcher, "perché comunque sono in comunicazione": "nel check status ci sono questi parametri, che servono alla comunicazione", e il nodo li riporta "nell'evento che tu mi costruisci". "Sono due eventi in comunicazione."

La decisione. Dentro CheckStatusEvent, "letteralmente" un **Branch** sul booleano IsActive: se è vero, **Activation**; se è falso, **Deactivation**. Gli eventi sovrascritti della luce restano quelli di prima; cambia solo chi li fa partire (figura 1.7.10).

1.7.21 Il bottone chiama: Call

"La cosa è che il controllo sta dalla parte della nostra carissima luce. Ma come facciamo a chiamarlo?" Nel bottone, dopo lo Switch on ETimelineDirection ("subito dopo, oppure in sostituzione; in questo caso facciamo subito dopo"), si trascina OnCheckStatus nel grafo. Il menù offre **Call**, **Bind**, **Unbind**, **Unbind All**, **Event** e **Assign**. Si sceglie **Call**: "quando succederà questo evento della timeline, che andrà avanti o indietro, chiamami questo evento e tutti quegli eventi che sono bindati".

- Sul ramo **Forward**, dopo il For Each Loop delle attivazioni: Call OnCheckStatus con **IsActive vero** ("se è forward, vuol dire che è attivo").

- Sul ramo **Backward**: Call OnCheckStatus con **IsActive falso**.

La prova. Per dimostrare che il nuovo metodo funziona da solo, il docente "svuota" il bottone dalle chiamate dirette: stacca i For Each Loop, lascia collegate "solamente le due Call, e basta"; nell'array della luce non serve più nulla, "la referenza non ci serve". In Play la luce si accende e si spegne comunque, "perché sto eseguendo un check status quando il bottone viene premuto, e lui si ricorda che all'inizio gli ho collegato quest'altro evento". "È come se lo tenesse in memoria." "È come dire un **raggruppamento di eventi**." E un'osservazione in risposta a una perplessità: il Bind è nel Begin Play, ma l'evento non scatta solo nel Begin Play; nel Begin Play si *stabilisce il legame*, che poi vale "ogni volta che lo usi".

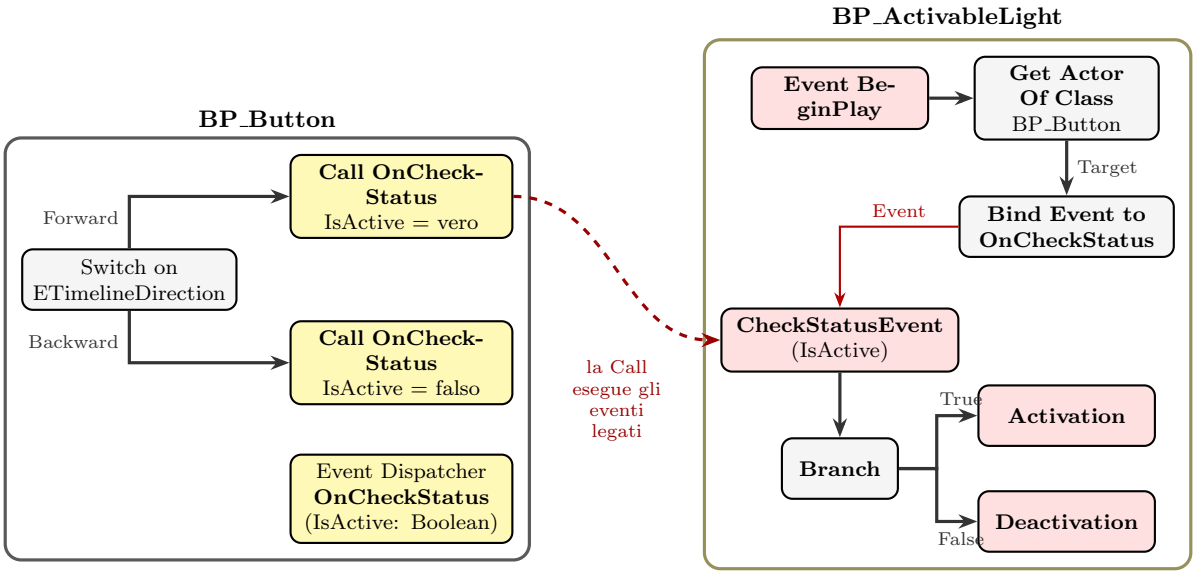


Figure 1.7.10: L'event dispatcher, a parti invertite. Nel suo Begin Play la luce trova il bottone (Get Actor Of Class) e lega al dispatcher OnCheckStatus un proprio evento, CheckStatusEvent, che nasce con lo stesso parametro IsActive. Quando il bottone esegue Call OnCheckStatus, vengono eseguiti tutti gli eventi legati, in qualunque Blueprint si trovino; la luce decide da sola, con un Branch, se attivarsi o disattivarsi. Schema ricostruito.

Approfondimento: dispatcher, ascoltatori e il Level Blueprint. Romero e Sewell (capitolo 4, sezioni "Event Dispatchers" e "Binding Events") descrivono lo stesso meccanismo con un esempio quasi identico: un *BP_Platform* che, quando qualcuno ci passa sopra, esegue *Call* del dispatcher *PlatformPressed* (con un parametro che passa la referenza a sé stesso, *Self*); il **Level Blueprint** si mette in ascolto e fa esplodere qualcosa, e un altro Blueprint, figlio dell'effetto scintille, lega un proprio custom event allo stesso dispatcher con **Bind Event** nel Begin Play, dopo un **Is Valid** sulla referenza. Due differenze rispetto alla lezione. La prima: nel libro la referenza al pulsante è una variabile Instance Editable scelta nei Details, mentre la lezione usa **Get Actor Of Class**, che è comodo ma restituisce *il primo* actor di quella classe che trova: con due pulsanti nel livello, tutte le luci ascolterebbero lo stesso, e quale sia non è garantito. Per più pulsanti servono una variabile referenza esposta (come nel libro) o **Get All Actors Of Class**, che restituisce un array. Inoltre è una ricerca su tutto il livello, da fare una volta (nel Begin Play, come qui), non a ogni frame. La seconda: il vantaggio vero del dispatcher è il **disaccoppiamento**. Chi chiama (il pulsante) non conosce chi ascolta, e chi ascolta può essere qualunque cosa, anche un Blueprint che non è un BP_Activable: un contatore di punteggio, una porta, la musica. La tabella 1.7.3 confronta i due metodi della lezione.

1.7.22 Override, Event e la scatolina

Due domande dagli studenti chiudono la spiegazione.

	Array di referenze (primo metodo)	**Event dispatcher (secondo metodo)**
Chi conosce chi	il pulsante conosce gli oggetti (ActivableItems)	gli oggetti conoscono il pulsante (Get Actor Of Class)
Dove si collega	nei Details del pulsante, con il + e il contagocce	nel Begin Play di ogni oggetto, con Bind Event
Che cosa chiama il pulsante	Activation e Deactivation su ogni elemento, con For Each Loop	una sola Call di OnCheckStatus, con IsActive
Tipo degli oggetti	devono essere child di BP_Activable	qualunque Blueprint che si leghi al dispatcher
Più pulsanti	ognuno ha il suo array: nessun problema	con Get Actor Of Class ascoltano tutti lo stesso pulsante
Si scollega con	togliendo l'elemento dall'array	Unbind Event (o Unbind All)

Table 1.7.3: I due modi della lezione per far comunicare il pulsante con gli oggetti. Le ultime due righe sono un'integrazione, non dette a lezione.

"Perché non compare la stampa del padre?" "Quando facciamo l'Activation della luce, perché non ci compare il print del padre?" "Perché quello che abbiamo fatto prima è stato l'azione di **override**." Il docente la ripete: nel parent, BP_Activable, l'evento Activation stampa "attivato" e Deactivation "disattivato"; nel child, BP_ActivableLight, l'override "va a sovrascrivere quell'evento con un nuovo evento". Prova del contrario: se nel figlio si cancella l'Activation sovrascritta, il pulsante torna a stampare "attivato", e la luce non si accende più.

"E se volessi fare tutti e due?" "Sia quello del padre, sia il suo?" Qui il docente improvvisa, e la trascrizione lo segue a fatica: prova a cercare un nodo *Parent* dal menù contestuale, non lo trova subito ("vediamo un po'"), e alla fine suggerisce una strada alternativa: creare un nuovo custom event ("parent and child activation") che *chiami* Activation "come funzione", cioè con il nodo azzurro di chiamata, e subito dopo esegua ciò che il figlio deve aggiungere; "oppure lo metti alla fine, direttamente lì", dopo il CheckStatusEvent. Il metodo standard di Unreal è più semplice, ed è descritto nella sezione 1.7.23.

"A che cosa serve il check status?" Un'altra studentessa è tornata indietro: "il check status mi serve soltanto per evitare il For Each Loop e tutto?". "Non esattamente. Cerco di spiegarmi un po' meglio." Il Begin Play, ricorda il docente, è "quando l'attore incomincia a esistere nel mondo di gioco". In quel momento la luce "va a cercarsi il bottone nel mondo di gioco e gli dice: ehi, c'è un evento che tu chiami OnCheckStatus; quando viene eseguito, c'è collegato anche questo evento qua". E propone di rinominare l'evento in modo più chiaro, per esempio *CheckIfActive*.

La scatolina. Di nuovo Paint ("bellissima grafia"). "Immaginatevi una **scatolina**: la scatolina è l'evento; è una scatola chiusa per adesso." La prima scatola si chiama *Check* e dentro fa "tutto quello che gli pare". Poi c'è un altro evento, una scatolina più piccola, *Control*, con dentro "varie operazioni matematiche". "Col binding io la sto aprendo e gli sto dicendo: guarda che quando esegui questa cosa, mi esegui anche un'altra cosa, e diventa un **pacchetto completo**." Check potrebbe andare dritto al suo risultato, "ma io gli ho detto: guarda che questo, quando viene eseguito, deve fare anche Control" (figura 1.7.11).

Unbind. Dal dispatcher, il menù offre anche **Unbind**: "rompimi questo legame". La dimostrazione: nella luce, quando si attiva, si esegue **Unbind Event from OnCheckStatus** sulla referenza del bottone (con lo stesso CheckStatusEvent come evento). Risultato: la luce si accende la prima volta, "ma poi non lo fa più, perché gli ho rotto il legame". "All'inizio è collegato, poi lo stacco, e lui andrà direttamente all'output": la scatola Check torna a fare solo il suo contenuto. "Chiaro?" "Sì, chiaro."

Approfondimento: Bind, Unbind, Unbind All, Assign. Le voci del menù di un dispatcher, in sintesi. **Call** esegue tutti gli eventi legati, nell'ordine in cui sono stati legati, passando a ciascuno i parametri. **Bind**

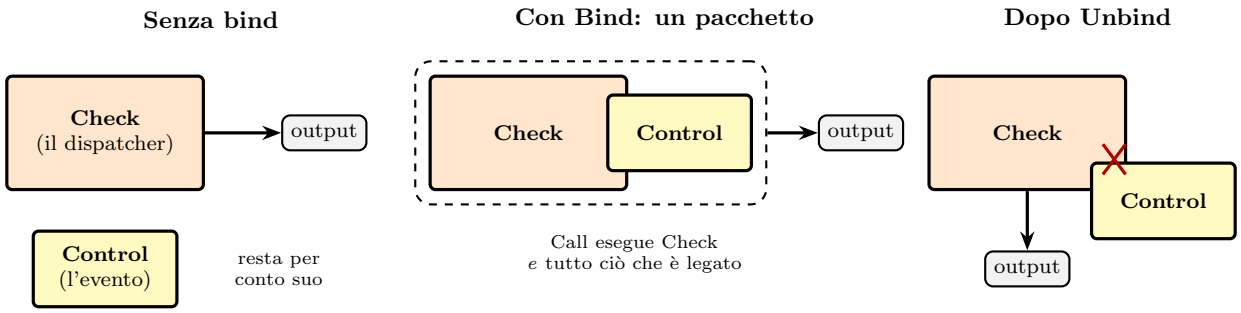


Figure 1.7.11: La "scatolina" disegnata dal docente in Paint. Un evento è una scatola chiusa; il Bind apre la scatola e ci mette dentro un altro evento, così che chiamarne uno esegua il pacchetto completo; l'Unbind rompe il legame, e la scatola torna a fare solo il suo contenuto. Schema ricostruito.

aggiunge un evento alla lista (lo stesso evento legato due volte non viene aggiunto due volte). **Unbind** toglie un evento preciso; **Unbind All** svuota la lista. **Assign** è una scorciatoia che crea in un colpo solo il nodo Bind e un custom event con la firma giusta, quello che il docente fa a mano tirando il filo dal pin *Event*. Infine **Event** crea, nel Blueprint che possiede il dispatcher, un evento già legato. Se l'actor che ascolta viene distrutto, il legame si scioglie da solo; se invece è il pulsante a sparire, la luce non riceve più nulla, senza errori.

1.7.23 Chiamare anche il parent

La domanda rimasta in sospeso nella sezione 1.7.22 ha una risposta diretta, che qui si aggiunge come integrazione alla lezione. Quando un child sovrascrive un evento del parent, nel grafo del child si fa **tasto destro sul nodo dell'evento** (per esempio l'Activation sovrascritta di BP_ActivableLight) e si sceglie **Add Call to Parent Function**. Compare un nodo arancione, **Parent: Activation**, identico nello spirito al *Parent: BeginPlay* visto nella sezione 1.7.14. Collegandolo al filo di esecuzione, prima o dopo il Set Intensity, la luce esegue *sia* la versione del padre (la stampa "attivato") sia la propria (l'accensione). Il metodo del docente, un custom event che chiama Activation e poi fa altro, raggiunge un effetto simile, ma rischia di confondere: se il nuovo evento chiama Activation sulla luce stessa, esegue la versione sovrascritta della luce, non quella del parent.

1.7.24 L'esercizio

"Adesso io vi propongo di fare questa cosa come esercizio", per "farvi un po' sperimentare". Va bene l'uno o l'altro metodo, "usando il check status, l'event dispatcher, o il fatto di chiamare gli eventi con l'array".

La proposta difficile: la piattaforma attivabile. Creare un **figlio** di BP_Activable ("Create Child Blueprint Class") e rifare dentro di esso la piattaforma della sesta lezione, ma comandata dal pulsante: "vorrei che la piattaforma, invece di scendere se io overlappo con essa", si attivi con la pressione del pulsante ("fatela magari in alto che va in basso, oppure se volete che va in alto, tranquillamente"). "L'importante è la pressione di un pulsante." Il docente spiega come trasportare il codice.

- Si copia il grafo dalla vecchia piattaforma e lo si incolla nel nuovo Blueprint: "vi dirà qualche errore, però è normale", perché "queste variabili non esistono".
- Ogni variabile mancante si ricrea con il **tasto destro** sul nodo, **Create Variable**: per esempio Duration. "Vi consiglio di richiamarla allo stesso modo", e il nodo "ritorna blu", cioè valido.
- La timeline va chiamata con lo stesso nome della vecchia (Timeline Platform), perché i nodi Set Play Rate la cercano per nome.

- I componenti vanno riaggiunti a mano: la Static Mesh ("la dovete chiamare allo stesso modo", Platform), il Billboard. Anche la variabile del suono ("Somewave") si ricrea con il tasto destro.
- La box *no*: "non dovete rimettere la box", perché l'evento che fa partire la piattaforma non è più l'overlap, ma l'attivazione da parte del pulsante.

Figlio di due padri? Una studentessa chiede se la nuova piattaforma possa essere "child di entrambi": sia della piattaforma normale, sia dell'attivabile. "No, purtroppo no." Si potrebbe cambiare la classe parent della vecchia piattaforma, ma "sicuramente darebbe errori".

Approfondimento: Reparent Blueprint. Un Blueprint esistente può effettivamente cambiare padre: nell'editor, *File* → **Reparent Blueprint**, o il campo *Parent Class* nelle *Class Settings*. Se il nuovo parent è a sua volta un Actor (come BP_Activable), componenti, variabili e grafi della piattaforma restano; gli errori temuti dal docente nascono quando il nuovo padre non ha qualcosa che il vecchio aveva, o quando nomi di variabili e di eventi entrano in conflitto. Per la piattaforma, il reparent a BP_Activable e l'override di Activation e Deactivation sarebbero una strada percorribile. L'ereditarietà multipla, invece, nei Blueprint non esiste: per dare a una classe più "identità" si usano le **Blueprint Interface**.

La proposta facile: un oggetto che si muove. "Se no, possiamo fare un esercizio un po' più semplice": un Blueprint nuovo, "**BP_BallMoving**, o come vi pare", con un oggetto qualsiasi ("un cubo, un cerchio") che si muove con una timeline e parte solo quando si preme il pulsante. Le regole:

- all'**attivazione**, la timeline parte: **Play**;
- alla **disattivazione**, la timeline torna indietro: **Reverse**;
- sull'Update, si cambia la posizione o la rotazione dell'oggetto, "destra, sinistra, su, giù, come vi pare".

"Quindi è activate, parte la timeline; quando invece finisce, Reverse. Fondamentalmente è fatto."

La piattaforma e il Reverse. "Ma nel caso della piattaforma non bisogna usare Reverse, se no quella risale da sola?" "Esatto." Nella sua versione il docente ha fatto risalire la piattaforma quando il giocatore ci sale sopra, "quindi io ce l'ho messa, la box". Ma l'esercizio, nella sua forma base, è questo: attivazione, la timeline va avanti; disattivazione, la timeline va indietro. La classe vota per provarci da sola, "i più impavidi" con la piattaforma: venti minuti, poi la correzione.

1.7.25 La soluzione di uno studente

"Sarò onesto, non sono convintissimo di come l'ho fatto. Però funziona." Uno studente condivide la sua piattaforma attivabile da un secondo computer. La revisione del docente è un piccolo catalogo di aggiustamenti.

- **Stop sulla Deactivation.** Lo studente ha messo *Stop* sulla disattivazione, così la piattaforma si ferma quando si scende dal pulsante e scende solo finché lo si tiene premuto. "Leva un attimo lo stop sul deactivation."
- **Play, non Play from Start.** il docente chiede di usare Play, non Play from Start: la distinzione della sezione 1.7.11, ora in pratica.
- **La Duration.** Il play rate copiato dalla vecchia piattaforma usa la variabile Duration, che nel nuovo Blueprint vale 0: va messa a **1** (lo studente non ricordava quella parte e voleva cancellarla).
- **Il nome della timeline.** Un nodo non trova la sua timeline: "la devi nominare" come quella originale, con F2. Compilazione, Play: "salici sopra. Ottimo."

- **Start e Finish invertite.** Lo studente ha dovuto scambiare Start Location e Finish Location per far funzionare il Lerp, e il Billboard "l'ho messo 300 sotto". Il docente all'inizio lo trova giusto, poi no; fa ricreare il Billboard, lo fa **alzare** invece di abbassare, e lo fa ricollegare. Il comportamento resta "strano" per qualche tentativo, e la causa precisa non emerge dalla trascrizione.

- **La curva, di nuovo.** Lo studente pensava di dover invertire la curva della timeline, perché la piattaforma parte dall'alto. "No, è sempre 0–1": "tu devi dire che lo 0 è la posizione quando stai in alto e 1 la posizione quando stai in basso. Questo è il passaggio tra i due, non è alto–basso." La timeline dice solo "passi da A a B in un secondo"; chi sia A e chi B lo decidono gli ingressi del Lerp.

Il suono non c'è ancora: "ci penso dopo". Il docente approva, e lascia la regola della giornata: "**sempre ragiona per step**. Fallo funzionare, poi fallo bello." "Bravissimo."

Approfondimento: la timeline come alfa. L'ultima correzione è la più utile da fissare. In una timeline usata come alfa di un Lerp, la curva non descrive *dove* si trova l'oggetto, ma *quanto* del tragitto è stato percorso: 0 è "alla partenza", 1 è "all'arrivo", qualunque siano partenza e arrivo. Per una piattaforma che scende basta mettere in A la posizione alta e in B quella bassa; per una che va di lato, A e B diversi; la curva resta la stessa. Invertire la curva *e* scambiare A e B produce lo stesso movimento di partenza (due inversioni si annullano), ma confonde chiunque legga il grafo, e rende Play e Reverse controintuitivi. La curva va invece modificata quando si vuole cambiare il *ritmo* (sezione 1.7.9).

1.7.26 Il compito per casa e la chiusura

"Mancano tre minuti." Chi non ha finito potrà mostrare il lavoro la volta successiva, o mandarne un video in chat. Il docente chiude due minuti prima, "così questo esercizio ve lo portate a casa, anche chi sta in differita", con il **compito**: **la piattaforma al contrario**.

1. Si preme il pulsante: la piattaforma, che sta in alto, **va giù**.

2. Si scende dal pulsante: sulla Deactivation **non succede niente**, la piattaforma resta giù.

3. Si sale sulla piattaforma: **torna su**.

Si possono usare "tutti e due i mezzi": l'event dispatcher, "oppure la cosa degli eventi" chiamati con il ciclo sull'array. "Va bene, ragazzi. Io vi ringrazio per l'attenzione. Ci sentiamo per martedì prossimo. Ciao a tutti. Buona serata."

Approfondimento: una traccia per il compito. Una possibile struttura, coerente con la lezione: un child di BP_Activable, *BP_ActivablePlatform*, con radice ferma, Platform, Billboard (figlio della radice) e una Box Collision figlia della Platform. Nel Begin Play si salvano Start Location (la Platform) e Finish Location (il Billboard, messo *sotto* la piattaforma). L'override di **Activation** fa **Play** della timeline (alfa da 0 a 1, Lerp da Start a Finish); l'override di **Deactivation** resta vuoto; l'**On Component Begin Overlap** della box, con il controllo sul player, fa **Reverse**. Con il secondo metodo, la piattaforma lega invece un proprio evento all'OnCheckStatus del pulsante, e nel Branch fa Play solo sul ramo vero. Il suono della sesta lezione si riaggiunge alla fine: prima funzionare, poi bello.

Punti chiave del capitolo

- Un **Accessed None** ("accesso nullo") vuol dire che una referenza è vuota quando la si usa: si corregge l'ordine dei nodi e si protegge con **Is Valid**. Un actor ha bisogno di una **radice** ferma (DefaultSceneRoot): se la mesh che si muove diventa la radice, le posizioni relative perdono senso.

- Un **array** è uno scaffale di variabili dello stesso tipo, con indice che parte da **0**. La costituzione di una variabile si sceglie accanto al Variable Type: Single, Array, Set, Map.

Sintomo	Causa e rimedio
“Errore di runtime: accesso nullo” sul suono, a fine prova	il pitch viene impostato prima dello spawn del suono: Switch dopo lo Spawn e il Set di SoundPlatform (ed eventualmente Is Valid)
La piattaforma va allo zero del mondo e non torna	manca il DefaultSceneRoot: la mesh è diventata la radice. Aggiungere un componente Scene e trascinarlo sopra la Platform
La piattaforma si sotterra dopo molti passaggi	il Billboard si muove con la parte che sale: renderlo figlio della radice
La piattaforma si riabbassa anche se esce un altro actor	manca il controllo sul player sull’End Overlap
Il player si blocca con i piedi contro la pedana	la collisione è più alta del Max Step Height: interrare la mesh o rifare la collisione simple (Box, ristretta e abbassata)
Il suono non cambia se si scende a metà salita	il pitch viene impostato solo allo spawn: Set Pitch Multiplier 0,4 sull’End Overlap, prima del Reverse
Il pulsante non scende	box troppo piccola o sotto la piastra; key non a tempo 0; Length non a 0,25; Update non collegato al Set Relative Location
Il pulsante non torna su	End Overlap collegato a Stop invece che a Reverse
Il pin della traccia si chiama ancora New Track 0	solo il nome: non cambia il funzionamento
Activable Items non compare nei Details del pulsante	la variabile non è pubblica: aprire l’“occhietto” e compilare
Il pulsante stampa ma la luce non si accende	nell’array c’è il BP_Activable di prova, non la luce: sceglierla con il contagocce
La luce compare in Play ma non si accende, e stampa ancora “attivato”	IntensityStart collegato anche al Set Intensity della Deactivation, o eventi non sovrascritti
La luce parte con intensità 0 anche quando si accende	Deactivation chiamata prima di salvare IntensityStart
Nel child non compare Parent: BeginPlay	il parent non ha un Event BeginPlay: il nodo compare solo se l’evento esiste nel padre
Il nodo Bind Event dà errore	si è compilato prima di collegare l’evento: creare il custom event dal pin Event e ricompilare
La luce reagisce solo la prima volta	un Unbind Event eseguito sull’attivazione
Errori dopo il copia-incolla del codice della piattaforma	variabili e componenti non esistono nel nuovo Blueprint: tasto destro, Create Variable; ricreare mesh, Billboard e timeline con gli stessi nomi
La piattaforma attivabile scende solo finché si preme	Stop sulla Deactivation: toglierlo (e Play invece di Play from Start)
Il play rate non ha effetto	Duration a 0 nel nuovo Blueprint: metterla a 1

Table 1.7.4: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, componente o impostazione	**Che cosa fa**
DefaultSceneRoot / Scene	la radice dell'actor; ferma e invisibile, fa da riferimento per le posizioni relative
Static Mesh Editor: Show, Collision	visualizza le collisioni simple (verdi) e complex (azzurre); aggiunge Sphere, Capsule, Box, k-DOP
Max Step Height	altezza massima che il personaggio sale camminando (Character Movement)
Timeline: Length, Key Interpolation	durata della timeline; tipo di curva fra le key (Auto, User, Break, Linear, Constant)
Zoom to Fit Horizontal / Vertical	adatta il grafico della timeline alla curva
Play / Play from Start	riprende dal punto attuale; riparte da 0
Make Vector	costruisce un vettore da X, Y, Z
Set Relative Location	posizione di un componente rispetto al suo genitore
Custom Event	un evento definito da noi, che si chiama per nome
Variabile Object Reference	una referenza a un actor di un certo tipo (qui BP_Activable)
Instance Editable (occhietto)	rende la variabile modificabile nei Details di ogni istanza del livello
Single / Array / Set / Map	la costituzione di una variabile
For Each Loop	esegue il Loop Body per ogni elemento di un array (Array Element, Array Index, Completed)
Switch on ETimelineDirection	due uscite: Forward e Backward
Contagocce (pick actor)	sceglie dalla scena l'actor da assegnare a una referenza
Create Child Blueprint Class	crea un Blueprint figlio, che eredita componenti, variabili ed eventi
Parent: BeginPlay / Add Call to Parent Function	esegue la versione del parent di un evento sovrascritto
Override (My Blueprint)	ridefinisce nel child un evento o una funzione del parent
Point Light	luce puntiforme: Intensity, Attenuation Radius, Source Radius, Light Color
Get Intensity / Set Intensity	legge e imposta l'intensità di una luce
Promote to Variable	crea una variabile del tipo di un pin e la imposta
Event Dispatcher (con input)	un evento a cui altri Blueprint possono legarsi
Get Actor Of Class	restituisce un actor di una classe presente nel livello
Bind Event / Unbind Event / Unbind All	lega o slega un evento da un dispatcher
Call (dispatcher)	esegue tutti gli eventi legati, con i parametri
Do Once, Spawn Sound at Location, Set Pitch Multiplier, Stop	il suono della piattaforma, ripassato in pausa

Table 1.7.5: I nodi, i componenti e le impostazioni della settima lezione.

- L'**ereditarietà**: un child eredita componenti, variabili ed eventi del parent, può aggiungerne e può **sovrascriverli** (override). Un child è anche del tipo del parent, e una variabile di tipo parent può contenerlo.
- Gli oggetti interattivi che si muovono vanno modellati in pezzi separati (cornice e piastra). È la **box** a far scattare l'evento, non la mesh: le collisioni della mesh sono troppo precise per un sì o un no.
- Le collisioni di una Static Mesh sono **simple** (verdi, forme semplici, usate di base) e **complex** (azzurre, la forma esatta, costose). Si modificano nell'editor della Static Mesh; per una pedana basta una **Box** ristretta e abbassata, sotto il **Max Step Height** del personaggio.
- La timeline del pulsante dura **0,25 secondi** e va da 0 a -10 sulla Z, con Make Vector e **Set Relative Location** sull'Update; l'End Overlap fa Reverse. Le key possono essere **lineari** o curve (**Auto**, con le tangenti), e la curva decide il ritmo del movimento.
- **Play** riprende dal punto attuale, **Play from Start** riparte da 0: per oggetti che vanno avanti e indietro la coppia naturale è Play e Reverse.
- **BP_Activable** definisce due custom event, Activation e Deactivation. Il pulsante ha una variabile pubblica (**occhietto**) **ActivableItems**, array di BP_Activable, riempita nei Details con il + e il **contagocce**. Sul Finished, lo Switch sulla Direction e un **For Each Loop** chiamano Activation o Deactivation su ogni elemento.
- **BP_ActivableLight** è un **Child Blueprint**: nel Begin Play salva l'intensità della **Point Light** in **IntensityStart** e *poi* si disattiva; Activation e Deactivation sono sovrascritti con **Set Intensity**. Il nodo **Parent:** (o *Add Call to Parent Function*) esegue anche la versione del padre.
- Un **event dispatcher** (OnCheckStatus, con il parametro IsActive) inverte la comunicazione: la luce trova il pulsante con **Get Actor Of Class** e lega un proprio evento con **Bind Event**; il pulsante esegue **Call**, e tutti gli eventi legati partono. **Unbind** rompe il legame.
- Il bug del pitch si corregge impostando il pitch anche sull'**End Overlap**, prima del Reverse.
- In una timeline usata come alfa, **0 e 1** sono partenza e arrivo, non alto e basso: chi sia A e chi B lo decide il Lerp. E la regola della giornata: **ragiona per step, fallo funzionare, poi fallo bello**.

Chapter 1.8

Game Instance, livelli e animazioni: il Gameplay Framework e un nuovo personaggio

Questa è l'**ottava lezione** del corso breve di **introduzione a Unreal Engine**. Nelle sette lezioni precedenti tutto è avvenuto dentro un solo livello e con un solo personaggio, il manichino del modello Third Person: la torretta, i proiettili, la piattaforma che sale, il pulsante che attiva le luci. Oggi la lezione allarga lo sguardo in due direzioni. La prima è **uscire dal livello**: cambiare mappa senza perdere quello che il giocatore ha fatto, cioè **salvare dei valori in modo globale**. La seconda è **cambiare personaggio**: prendere un modello nuovo da **Mixamo**, importarlo con il suo scheletro e le sue animazioni, e cominciare a costruire la logica che decide quale animazione mostrare, l'**Animation Blueprint**.

Il filo che tiene insieme le due parti è il **Gameplay Framework** di Unreal: la "gerarchia" delle entità che l'engine crea e tiene in vita mentre il gioco gira. La **Game Instance**, la prima a nascere e l'ultima a morire, è il posto giusto per i dati che devono sopravvivere al cambio di livello; il **Pawn** e il **Character**, con il loro **Character Movement**, sono la fonte dei dati (velocità, caduta) che l'Animation Blueprint legge per animare lo scheletro. "Andremo sia a vedere quelle che sono le animazioni e come si gestiscono, ma sia a vedere un po' quello che è chiamato il *gameplay framework* di Unreal."

La lezione è **online**, su Zoom, come le precedenti, ma con una particolarità: la webcam del docente si rifiuta di funzionare per tutta la serata ("oggi mi vedrete con uno schermo nero, o almeno il mio schermo è più importante della mia faccia"), e in due occasioni l'editor si chiude da solo per mancanza di memoria video. Si apre con un problema di uno studente rimandato alla pausa, prosegue con una parte teorica in slide sul Gameplay Framework e con la costruzione dei collezionabili, della Game Instance e dei volumi che cambiano livello; la pausa arriva verso le 17:26, con ripresa alle "55, 18, solito orario"; la seconda parte è dedicata all'import del personaggio e delle animazioni e all'Animation Blueprint. Il docente stesso avverte che non si arriverà in fondo: "probabilmente questa lezione la riporteremo nella prossima", in cui si vedranno i Blend Space e il resto delle animazioni. Si chiude con un "ci vediamo giovedì".

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Arena Engine", "Arreal", "a Real", "a RIAL", "Arial" e "Irium" (quest'ultimo, forse, è il programma della webcam); la **Game Instance** è "View Instance", "getViewInstance" e "key instance"; **GI_Platform** è "GI Gaming Instance", "gi i platform", "asg.platform", "giiPatform" e "Di Alto"; il **Pawn** è "Pwn", "la Bone" e "player born"; **Get Player Pawn** è "getLayerOne" e "PlayerOh"; il **Cast To** è "castagna"; l'**On Component Begin Overlap** è "component baking per l'acqua", "onBegin onComponent onBegin overlap" e "On Collision Beginner per l'app"; il **Destroy Actor** è "destroy hardware"; il **Text Render** è "text rendere"; la **Skeletal Mesh** è "scheletal mesh" e "scheda del Mesh"; il **Control Rig** è "control righe"; l'**Idle** è "Idol"; il **root motion** è "root in"; la **T-pose** è "Tico", "tifoso" e "Bix ti pose"; **FBX Binary** è "Fibix binary"; il **Knight** è "Night"; il **Try Get Pawn Owner** è "try get one"; l'**Event Blueprint Update Animation** è "event blueprint a data animation"; **IsFalling** è "isPalling"; il **Diffuse** è "Diffuse", ma l'**eyelash** è "ailash"; **Mixamo** è a volte "Mixamon". Dove la ricostruzione è sicura il capitolo

usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione, durante la pausa e durante i minuti lasciati per scegliere le animazioni il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?", "la situazione è questa, la migliore dal 2005", "i genitori a bordo", la "dottrina sul mercato del lavoro", un Vittorio Prodi chiamato in causa senza motivo, una lunghissima fila di "non, non, non"): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è una dimostrazione dal vivo nell'editor, costruita a voce ("tirate fuori", "castate", "promote to variable"), e in un paio di punti il docente improvvisa o si accorge di aver saltato un passaggio ("ah, giusto, ragazzi, abbiamo scordato una cosa"). Il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, e li completa, dove serve, con i nomi esatti dei nodi e dei pin di Unreal Engine 5; dove la trascrizione lascia un buco, lo dichiara.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono la gerarchia del Gameplay Framework, la vita della Game Instance attraverso i cambi di livello, il cast come "abito giusto", i grafi del collezionabile, del tasto E e dei due volumi, la differenza fra Skeletal Mesh e Skeleton, quella fra animazione in root motion e in place, i due grafi dell'Animation Blueprint e il Blend Space 1D. Le tabelle riassumono le entità del framework, le impostazioni di download di Mixamo, gli asset creati dall'import, gli errori incontrati dagli studenti e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'**inizio della lezione**: un errore sul suono della piattaforma, Mixamo e le Game Jam (sezione 1.8.1);
2. il **programma**, le **variabili globali** e la dimostrazione dei collezionabili che passano da un livello all'altro (sezioni 1.8.2 e 1.8.3);
3. il **Gameplay Framework** di Unreal, dalla Game Instance all'input (sezione 1.8.4);
4. il **fix dell'ascensore** con la booleana Activated (sezione 1.8.5);
5. la **Game Instance**: creazione, variabile, Project Settings (sezione 1.8.6);
6. il **collezionabile**, il **cast** e il tasto E (sezioni da 1.8.7 a 1.8.9);
7. i due **volumi**: lo shop che azzera e il volume che **carica un livello** (sezioni da 1.8.10 a 1.8.12);
8. **Mixamo**, la pausa e i problemi degli studenti (sezioni 1.8.13 e 1.8.14);
9. l'**import** del personaggio, lo **scheletro** e le **animazioni** di Mixamo (sezioni da 1.8.15 a 1.8.19);
10. l'**Animation Blueprint**, il nuovo **Character** e l'AnimGraph (sezioni da 1.8.20 a 1.8.24);
11. i **materiali** mancanti e la chiusura (sezione 1.8.25).

1.8.1 Prima della lezione: l'Audio Component, Mixamo e le Game Jam

La registrazione comincia con i saluti ("buonasera, buonasera, buonasera") e con le scuse del docente: "scusate, ragazzi, se non mi faccio vedere, ma a quanto pare ho dei problemi con la webcam". Il sospetto è un conflitto con OBS, il programma di registrazione ("forse OBS potrebbe fare contrasto"); i tentativi si ripetono per tutta la prima parte, senza successo. "Non vi preoccupate: vi guardo, mi ascoltate, ma non mi vedete."

Le variabili chiare. "Come state, ragazzi? Avete un po' sperimentato?" Uno studente ha recuperato dalla registrazione la lezione del giovedì precedente, e racconta il suo unico problema: "non funzionava perché nella variabile non avevo messo il tempo, e quindi non si muoveva niente". Il docente ne fa una regola: "il fatto di avere delle variabili chiare e concise ti finisce proprio il lavoro", perché altrimenti "stai sempre là a riguardarle" chiedendoti "ma perché non si muove?", finché non ti ricordi "che l'hai creata di nuovo, però sta a zero". Una variabile appena creata ha il suo valore di default (zero, per i numeri), e un tempo, una durata o una velocità lasciati a zero fermano tutto senza dare errori.

Mixamo e le Game Jam. Uno studente chiede se il sito condiviso su Discord sia gratuito: "sì, sì, assolutamente, non ci avete niente da spendere". È **Mixamo**, "un'ottima risorsa" che oggi verrà usata "in modo attivo": essendo uno strumento di Adobe "abbastanza preciso nel replicare le animazioni", e avendo modelli "abbastanza compatibili" con quelli di Unreal, permette di avere personaggi e animazioni senza animarli a mano. "Ci sono animazioni migliori, perché esistono i pack che si pagano fior di quattrini", ma per cominciare Mixamo è "il ben di Dio". Il docente lo usa soprattutto per le **Game Jam**: "uno ha mai fatto una Game Jam?" Gli studenti rispondono da soli: "sono gli eventi dove si programma un videogioco in pochi giorni". "Fondamentalmente si programma il gioco in pochi giorni, che va dai tre giorni a una settimana", "poi ognuno fa un po' come gli pare"; e in quei pochi giorni, avere animazioni pronte è ciò che fa la differenza.

Il primo problema: l'Audio Component che non è valido. Filippo condivide lo schermo. La piattaforma della lezione precedente ("quando ho fatto il bottone, l'altra volta") funziona, il suono si sente, "ma poi mi dà errore", sul **Set Pitch Multiplier**. Il docente insegna prima un'abitudine: nel log degli errori, quando un nome è sottolineato, "puoi direttamente cliccare: dice proprio l'errore e ti ci porta". Il messaggio, in italiano, è un tentativo di accesso a un componente "tramite la proprietà SoundPlatform" che "non è valida": "vuol dire che l'Audio Component non è stato spawnato o non esiste". L'errore compare solo salendo sulla piattaforma, e solo una volta per partita: "lo fa solo una volta, fortunatamente", perché "quelle successive te lo fa". È, a prima vista, lo stesso caso della settima lezione (il pitch impostato prima che il suono esista), già incontrato con un'altra studentessa.

Il docente prova un rimedio rapido: togliere i due *Get* di SoundPlatform ("questi due cilindroni") e collegare i due Set Pitch Multiplier direttamente all'uscita del *Set* della variabile, "così dovremmo averci la sicurezza". Non basta: rileggendo l'errore, il problema "non è SoundPlatform, è l'Audio Component" a non essere valido. "Visto che è una cosa che ci vuole un attimino di fuori programma", il caso viene rimandato alla pausa (sezione 1.8.14): "può essere un errore di logica di base, poi magari sono micro cose".

Approfondimento: quando l'errore compare una volta sola. Un errore di tipo "Accessed None" ("accesso nullo", "non è valida") che compare *solo la prima volta* è quasi sempre un problema di **ordine**: un nodo legge una variabile referenza prima che qualcuno l'abbia riempita. Dalla seconda volta in poi la variabile contiene il suono spawnato la volta precedente, e tutto sembra funzionare. Un errore che compare *solo arrivando in fondo* (come dirà lo studente in pausa) indica invece che il componente esisteva ma è stato distrutto: un Sound spawnato con **Spawn Sound at Location** ha di default l'opzione *Auto Destroy* attiva, cioè l'Audio Component si distrugge da solo quando il suono finisce o viene fermato con **Stop**, e la variabile rimane a puntare a un oggetto non più valido. In entrambi i casi la protezione è la stessa: un **Is Valid** prima di usare la referenza, oppure ricreare il suono quando serve.

1.8.2 Il programma della giornata e le variabili globali

"Oggi andremo a vedere un po' di particolarità." Il programma ha due parti:

- le **animazioni**, "e come si gestiscono";
- il **Gameplay Framework** di Unreal: il fatto che l'engine, "nella sua esecuzione", ha "una gerarchia di quello che tiene, di quello che ha", cioè di entità che "sono sempre presenti in alcuni momenti". Fra queste c'è la **Game Instance**, che oggi permetterà "di salvare dei valori in modo globale, chiamiamolo così".

Variabile globale e variabile locale. "Non so se qualcuno di voi, a livello di programmazione, sa cosa si intende con una variabile globale" ("Federico già lo vedo bello arzillo"). Una **variabile globale** è "visibile in tutte le parti del codice": "indipendentemente dall'ambiente in cui ci troviamo", nel character, in un altro oggetto, nel livello, "è possibile visualizzarla, prenderla, modificarla". L'esempio opposto è a portata di mano: la variabile **ActivableItems** del pulsante della settima lezione. "Questa è pubblica", con l'occhietto aperto, "la possiamo vedere da viewport"; ma è pubblica solo nel senso che la si modifica nei Details dell'istanza. Esiste *dentro* quel pulsante, dentro quel livello: "se noi dovessimo cambiare mappa, questa variabile non la potremmo più vedere", e "se volessimo stampare dei valori, questo qua non esisterebbe più". Pubblica non vuol dire globale.

Approfondimento: pubblica, privata, globale. Nei Blueprint le tre parole indicano cose diverse. Una variabile **Instance Editable** (l'occhietto) è modificabile nei Details di ogni istanza nel livello; una variabile **Private** non è accessibile dagli altri Blueprint, nemmeno dai child; ma in tutti i casi la variabile *vive dentro un oggetto*, e muore con lui. Quando si apre un nuovo livello, Unreal distrugge tutti gli actor del livello precedente, compreso il personaggio del giocatore, e ne crea di nuovi: i valori delle loro variabili tornano ai default. Unreal non ha variabili globali nel senso dei linguaggi testuali; il modo di ottenerle è appoggiarle a un oggetto che *non* viene distrutto al cambio di livello, ed è esattamente ciò che la lezione fa con la Game Instance (sezione 1.8.6).

1.8.3 La dimostrazione: collezionabili e porte fra i livelli

Come sempre, prima di costruire il docente mostra il risultato finito. "Adesso vedrete anche questo nuovo character che andremo a creare da zero, grazie all'aiuto di Mixamo", con "delle animazioni nuove"; e avverte che "cambiare character, visto che tutto quanto va a reagire con il character" (la piattaforma, il pulsante, i controlli sul player), richiederà "un piccolo fix da fare". Per la dimostrazione, quindi, rimette il vecchio **ThirdPersonCharacter**.

I collezionabili. Nel livello ci sono delle palline: "questi sono dei *collectable*, sono delle palline che ho creato". Il personaggio ci passa sopra, "1, 2, 3", e le palline spariscono; premendo il tasto **E**, sullo schermo compare il numero di collezionabili presi: 3.

La porta e il nuovo livello. "Oggi creeremo anche questa, che è la porticina, che ci porterà a un nuovo livello." Il personaggio la attraversa e si trova in un livello quasi vuoto. Premendo E, il conteggio è ancora 3: "interagendo con la Game Instance, che è sempre presente indipendentemente se io cambio livello o meno, mi posso trasportare dei valori". Nel secondo livello ci sono altre palline: "se prendo questa, ne avrò 4", poi 5 e 6; e "se torno al mio hub, ne avrò 6".

Crash Bandicoot. "Non so se qualcuno di voi ha giocato a Crash Bandicoot." I classici della serie tengono il conto dei **cristalli**, dei **frutti Wumpa** ("frutti Pumpa", nella trascrizione) e delle **gemme**: "quelli sono valori che sono sempre salvati quando tu li prendi", che restano tali qualunque livello si attraversi, "quasi totalmente globali". Senza la Game Instance, invece, "si perderebbe il valore", e il conteggio "tornerebbe sempre a zero": "dovresti fare sempre 1, 2, 3, 1, 2, 3".

Che cosa ancora non fa. Tornando nel primo livello, le palline già raccolte sono di nuovo lì: "questo bellissimo collectible è rispawnato". È il comportamento normale (il livello viene ricaricato da capo), ed è anche il prossimo esercizio annunciato: "andremo a fare anche un esercizio del genere, in cui andremo a dire: ehi, guarda che questo l'ho già raccolto, levalo, non me lo far riprendere". Cioè "salvarci globalmente lo stato delle cose". Nella lezione di oggi l'esercizio non viene svolto.

1.8.4 Il Gameplay Framework di Unreal

Che cos'è un framework. "Qualcuno di voi sa che cosa si intende con *framework*? Ha mai usato un framework per qualsiasi cosa, per programmazione, o semplicemente: a volte esistono i framework per la scrittura." Un framework è "uno sviluppo base di processi", una "scheda da seguire". L'esempio del docente è la narrativa: in una storia "c'è l'inizio, c'è lo sviluppo, c'è magari il *cliffhanger*"; un horror può essere psicologico o splatter, un film d'azione "demenziale" o epico, e ognuno di questi generi, "a scelta dell'autore", può seguire uno schema che "lo aiuta a sviluppare un processo di creazione". In Unreal "esiste praticamente questo tipo di processo", fatto però di "tante piccole interfacce", o meglio di **entità**, che "hanno una gerarchia tra di loro". "Questo, ragazzi, è il gameplay framework di Unreal." Il docente lo spiega su una slide con uno schema dall'alto verso il basso, in ordine di nascita e di "priorità" (figura 1.8.1).

La Game Instance. "Quando il progetto viene lanciato, cioè l'engine", "la prima cosa che viene creata, o comunque che ha priorità, è la **Game Instance**." È un'entità "che va a esistere sempre, in ogni caso, quando il gioco è aperto". Subito dopo, nello schema, viene il caricamento del **livello**.

La Game Mode e Gwent. Con il livello nasce la **Game Mode**, "un'altra interfaccia, che in questo caso definisce gli stati di gioco, gli stati del giocatore, oppure le **regole del gioco**". "Questa Game Mode esiste assieme alla mappa", "e di per sé nel mondo di gioco". L'esempio è *The Witcher 3* e il suo gioco di carte, il **Gwent**: "state giocando al gioco normale", parlate con un NPC che vi propone "un round di carte", "e parte il gioco delle carte". "In quel caso la Game Mode cambia": "le regole del gioco, i punteggi, le condizioni di vittoria o sconfitta sono gestite dalla Game Mode".

Game State, Player State, client e server. Scendendo, lo schema mostra il **Game State**, "lo stato del gioco, quindi tutti i dati che sono presenti", e il **Player State**, "lo stato del giocatore". Servono soprattutto quando c'è un "interscambio di dati", come in una cooperativa o in un multigiocatore "con un server di mezzo". Una seconda slide lo mostra: "esiste il **client**" ("avete presente quando aprite quello che si chiama client") e il **server**. "Il client comunica con un server, e va a mandare i dati ad altri giocatori con cui giocate online": è così che, in uno sparatutto, "si capisce che tipo sei morto" e che "quel numero delle death o delle kill aumenta", "perché c'è uno scambio di stati tra i nostri player".

Il Pawn. "Abbiamo parlato di Pawn in precedenza": il canale di collisione *Pawn*, il nodo *Get Player Pawn* usato in ogni overlap. Il docente lo descrive così: "il Pawn non è già il character stesso, ma è l'entità" che può essere controllata, dal giocatore "o addirittura dall'AI", cioè "o da input del giocatore, oppure da un processo di *behavior*, un comportamento già stabilito, come può essere un nemico". Al Pawn "vengono date" le componenti, la mesh, la navigazione, le animazioni, il *view target* (la camera) e, "se ci fossero", i **physical material**, "materiali che hanno delle proprietà fisiche", da vedere più avanti.

Il Character e il Character Movement. "Il character stesso è subito dopo." Il Character ha "le sue particolarità di movimento", raccolte nel **Character Movement Component**, con dati "che possono essere modificati":

- le **movement mode**: "come mi sto muovendo", cioè se sta camminando, se sta volando (*flying*), se sta nuotando, oppure un *custom movement*, "un movimento particolare", come una scivolata;
- le **movement capabilities**: "quanto mi posso muovere, quanto veloce sono", la *braking friction*, "cioè quanto ci metto a frenare";
- le **interazioni fisiche**;
- la **replicazione**, cioè il movimento condiviso in rete.

Skeletal Mesh e Skeleton. Il Character ha anche la sua **mesh**, "la sua particolarità di avere la sua forma". "La Skeletal Mesh e lo scheletro sono due cose completamente diverse, ragazzi." Lo **scheletro** è fatto di ossa, le **bones**, ed è il prodotto del **rigging**, "il compito di un povero ragazzo o ragazza che si mette là a creare le ossa", "un lavoro a sé stante"; poi c'è l'**animatore**, "che ruota queste ossa per compiere un'animazione". La **Skeletal Mesh** "è fondamentalmente una mesh con uno scheletro": "la forma più le ossa", "il corpo e le ossa". Lo scheletro è un asset separato, "gestito in un'altra modalità", in cui si vedono "tutte quante le ossa del nostro bellissimo character" (figura 1.8.7). Le ossa si controllano con l'**Animation Graph**, "spesso chiamato Animation Blueprint", che oggi verrà creato, e con il **Control Rig**, "una cosa che è stata messa da Unreal" per animare direttamente nell'editor, che però oggi "non ci servirà".

L'animation programmer. "Sappiate che ci sono delle figure professionali" che non sono solo l'animatore: c'è l'**animation programmer**. "Fate conto che sono due fratellini": in uno studio dove ha lavorato il docente "andavano avanti e indietro continuamente", "uno fa le animazioni, l'altro le gestisce, con la logica". L'esempio è il **piede sullo scalino**: in molti giochi "i piedi vengono inclinati" quando il personaggio sta su un piano inclinato o su un gradino. "Quella cosa non è fatta da un animatore, assolutamente no": è fatta da un animation programmer, che "prende le ossa della camminata" e dà "un set di regole": "guarda che, se c'è un dislivello di 20 gradi, l'osso va ruotato in questo modo e va messo sopra lo scalino", "la pianta del piede va ruotata e messa in allineamento". Sul character base di Unreal questa particolarità "già c'è", "perché ormai è una cosa strausata".

Il Player Controller. Più in basso nello schema c'è il **Player Controller**. "Se dovessimo andare per una meccanica" in cui modificare "i parametri di controllo", per esempio "switchare tra character" ("controllo uno, controllo l'altro"), si lavora con il controller, che "è un'entità a sé stante". I controller "sono actor non fisici": "non li potete vedere".

L'input. "L'ultima cosa, che veramente è in fondo, è che quasi qua ci siamo noi": l'**input**, "la forma più esterna di interazione tra la macchina e noi", da cui parte "tutto quanto il percorso del gameplay framework". Esiste un sistema moderno, l'**Enhanced Input System**, "molto interessante", "che ci permette di destreggiarci negli input": per ora non verrà usato ("andremo a vedere forme molto più blande"), ma più avanti forse si creerà "un sistema di input interfacciato con Enhanced Input". "Tutto questo ambaradam è come Unreal capisce e comprende" il gioco, e gli dà "una gerarchia". "Visto che vi ho riempito di chiacchiere, andiamo a vedere con la ciccia, con la pratica."

Approfondimento: chi possiede chi. Nella spiegazione a voce i ruoli di Pawn e Controller si confondono un po' ("il Pawn è l'entità che lo possiede"). Romero e Sewell (capitolo 3, "Object-Oriented Programming and the Gameplay Framework") li distinguono con un'immagine netta: il **Pawn** è "il corpo fisico" del personaggio, un actor che può essere *posseduto*; il **Controller** che lo possiede è "una specie di cervello", e rappresenta un giocatore umano (**PlayerController**) o un'intelligenza artificiale (**AIController**). Il nodo **Possess** permette al controller di cambiare corpo, ed è il meccanismo del "controllo uno, controllo l'altro" del docente: nel loro esempio due Character nel livello si alternano premendo 1 o 2, e riceve i comandi solo quello posseduto. Il **Character** è un figlio del Pawn pensato per personaggi "che camminano, corrono, saltano, nuotano e volano", ed eredita quattro componenti: *CapsuleComponent* (le collisioni), *ArrowComponent* (la direzione), *Mesh* (una Skeletal Mesh, animata da un Animation Blueprint) e *CharacterMovement*, che gestisce anche "replicazione e predizione nei giochi multigiocatore". Per ereditarietà, un Character è anche un Pawn e anche un Actor: è per questo che il confronto con *Get Player Pawn* funziona su un Character.

Approfondimento: la Game Mode del livello. Sempre nel capitolo 3, gli autori precisano che la **Game Mode** non definisce solo le regole, ma anche le *classi di default* usate dal gioco: il Pawn che il giocatore riceve (*Default Pawn Class*), il PlayerController, il Game State, l'HUD. Si imposta per tutto il progetto in *Project Settings* → *Maps & Modes* → *Default GameMode*, e si può sostituire livello per livello in *World Settings* → *GameMode Override*. È proprio lì, nei World Settings, che a fine lezione il docente cambierà il personaggio del giocatore (sezione 1.8.24). La **Game Instance**, invece, "non è una delle classi comuni", ma "è importante sapere che esiste": viene creata all'inizio del gioco ed eliminata solo quando il gioco si chiude,

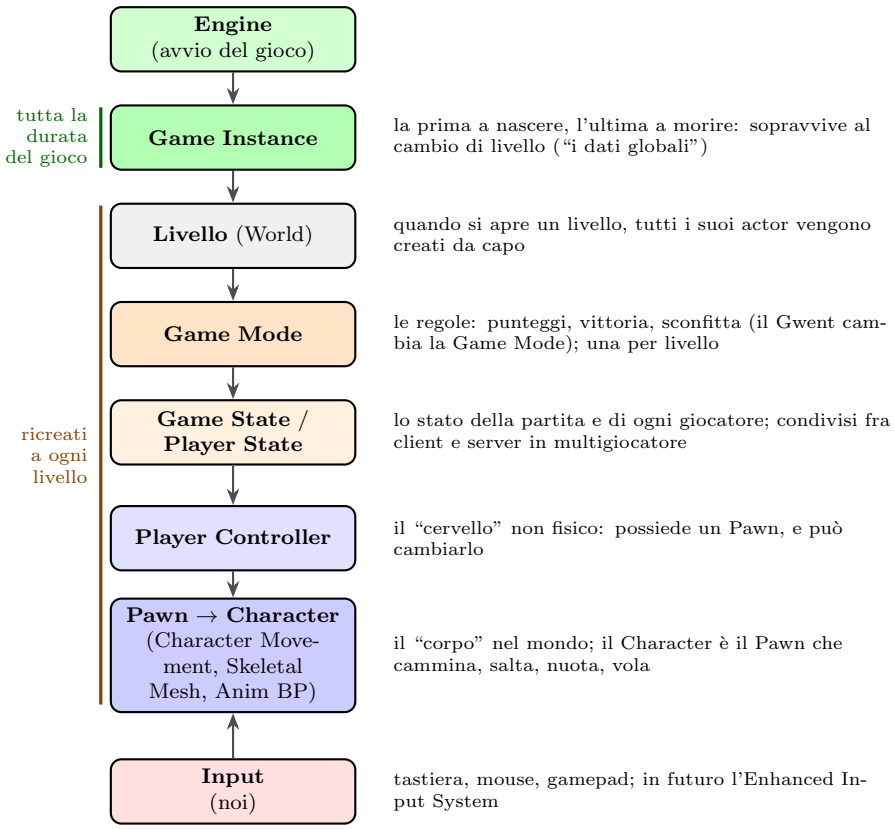


Figure 1.8.1: Il Gameplay Framework come lo presenta il docente: dall'alto verso il basso, in ordine di nascita e di "priorità". La Game Instance esiste per tutta la durata del gioco; tutto quello che sta sotto il livello viene distrutto e ricreato a ogni cambio di mappa. Schema ricostruito; le note a destra riassumono la spiegazione.

mentre “tutti gli actor e gli altri oggetti di un livello vengono distrutti e ricreati ogni volta che un livello viene caricato”. Per questo è “un’opzione da usare se si devono conservare dei valori nella transizione fra livelli”.

Entità	Che cosa fa	Esempio della lezione
Game Instance	esiste dall’avvio alla chiusura del gioco; conserva i dati fra i livelli	il conteggio dei collezionabili; il personaggio scelto
Game Mode	le regole: punteggi, condizioni di vittoria e sconfitta, classi di default; nasce con il livello	il Gwent di *The Witcher 3*
Game State	lo stato della partita, condiviso fra i giocatori	i dati di una cooperativa con un server
Player State	lo stato di un singolo giocatore	le kill e le death in uno sparatutto online
Pawn	l’actor che può essere controllato, dal giocatore o da un’AI	Get Player Pawn nei controlli di overlap
Character	il Pawn con capsula, Skeletal Mesh e Character Movement	il ThirdPersonCharacter, il nuovo BP_Knight1
Character Movement	movement mode, velocità, frenata, fisica, replicazione	la velocità letta dall’Animation Blueprint
Player Controller	entità non fisica che riceve l’input e possiede il Pawn	passare da un personaggio all’altro
Input	tasti, mouse, gamepad; Enhanced Input System	il tasto E che stampa il conteggio

Table 1.8.1: Le entità del Gameplay Framework presentate a lezione. Le precisazioni su Game Mode e Controller seguono Romero e Sewell (capitolo 3).

1.8.5 Un fix per l’ascensore: la booleana Activated

Il docente apre il progetto (“mi sono scordato”) e invita a creare una cartella nuova (“tasto destro, solita cosa”), ma prima di cominciare si ferma: “ah, ragazzi, aspettate, mi sono ricordato di una cosa che volevo farvi fare”. È “un piccolo fix per l’**ascensore** col pulsante”, cioè la piattaforma attivabile del compito della settima lezione: la piattaforma che, comandata dal pulsante, scende, e che risale quando il giocatore ci sale sopra. “Il problema era che si bloccava”: “andava sempre più giù e non si muoveva più”.

La cura. “Basta mettere una booleana.” Nella piattaforma si crea una variabile booleana, **Activated**, e la si usa come un cartello “già fatto”:

- all’inizio dell’evento **Activation** si mette un **Branch** con Activated come condizione; dall’uscita **False** si fa **Set Activated** a *true* e poi si prosegue con la logica che c’era (il Play della timeline). Dall’uscita True non parte niente: se la piattaforma è già attivata, una seconda attivazione viene ignorata;
- sull’**On Component Begin Overlap** della box (il player che sale e fa risalire la piattaforma) si rimette Activated a *false*, in modo che la prossima pressione del pulsante possa di nuovo farla scendere.

“Questo dovrebbe risolvere il fatto che l’ascensore vada continuamente giù” (figura 1.8.2). Nella trascrizione la frase esatta è confusa (“deve essere un branch con il false, mettete activated come condizione principale”); la struttura qui sopra è quella che il docente rifarà passo per passo con uno studente in pausa (sezione 1.8.14).

Approfondimento: la booleana come cancello. Una variabile booleana usata così è un *flag*, o un *gate*: un cancello che lascia passare l’esecuzione una volta e poi resta chiuso finché qualcun altro non lo riapre. È la versione “a mano” del nodo **Do Once**, già usato nella sesta lezione per il suono, che ha un ingresso *Reset*; scritta con un Branch e una variabile ha il vantaggio di essere leggibile da altri punti del Blueprint

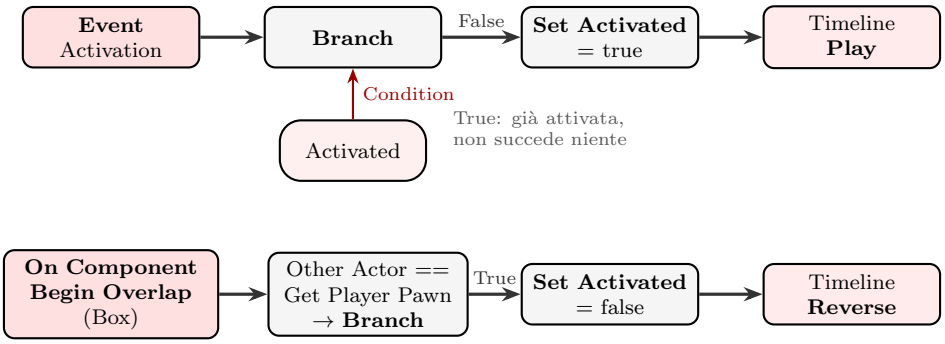


Figure 1.8.2: Il fix dell'ascensore. La booleana Activated impedisce che una seconda attivazione rimandi giù la piattaforma già scesa; quando il player ci sale sopra e la fa risalire, Activated torna a false. Schema ricostruito.

(per esempio da un Print String, mentre si fa debug) e di poter essere riaperta da un evento qualsiasi. Il problema che risolve è comune a tutti gli oggetti comandati da più eventi: un Play chiamato mentre la timeline è già alla fine non fa nulla di visibile, ma la combinazione di Play, Reverse e Set New Time chiamati nell'ordine sbagliato può spostare la posizione di partenza dei calcoli.

Una chiavetta USB. Una studentessa segnala che il progetto non si apre: "mi apre solo UE senza andare dentro". Potrebbe essere "un problema di import"; il consiglio vale per tutti: "attenzione con le chiavette USB, a staccarle" sempre in modo sicuro, "perché se no vi fate i problemi". "Servirebbe un **Git**", cioè un sistema di controllo di versione, "però di base fate attenzione". E comunque "c'è la registrazione".

1.8.6 La Game Instance: GI_Platform

"Allora, questo è solo un piccolo fix, ragazzi. E adesso partiamo con il nostro bellissimo Game Instance." Si crea una cartella **GameInstance**, la si apre, e si crea la Game Instance del progetto: "sarà praticamente il **cervello principale**, che sarà possibile visualizzare da ogni parte, che cambiamo mappa o che non cambiamo mappa".

La creazione. Si fa **Blueprint Class**, "al solito". Nella finestra che chiede la classe genitore, i bottoni in alto sono le classi comuni; sotto "potete abbassare una tendina" (**All Classes**), "e sono presenti tutte le classi di tutti gli attori che potete creare". Si cerca **GameInstance**: "dovreste trovare Object → GameInstance". Select, e il nome: **GI_Platform**, con il prefisso GI per "Game Instance" ("la chiamiamo Platform"). Aperta, "vi si presenterà come un normalissimo Blueprint, che abbiamo già visto tante volte ormai, ma completamente vuoto": "non c'è una viewport, non c'è niente, è solo un'entità non fisica". Dentro si possono creare "delle funzioni, delle macro, delle variabili, degli event dispatcher, come abbiamo visto le altre volte".

La variabile. Si crea una variabile di tipo **Integer**, chiamata **Collectables**. "Da questo momento in poi questa variabile, assieme alla Game Instance, viaggerà assieme a lei": "il suo valore", che lo modifichiamo o no, "quando sta all'interno della Game Instance rimane tale".

I Project Settings. "Adesso dobbiamo dire al gioco quale Game Instance devi usare. Perché lui userà la sua di default, che si trova da qualche parte nei file dell'engine." Da **Edit** → **Project Settings** si cerca "Game Instance": compare, sotto **Project** → **Maps & Modes**, la voce **Game Instance Class**. Dalla tendina si sceglie **GI_Platform**. "Adesso la Game Instance del gioco, fino a che il gioco è in esecuzione, userà questa."

Non appesantirla. "Sappiate che, quando si va in esecuzione, questa Game Instance esiste sempre; quindi è molto importante anche **non appesantirla**." "Questa è perenne finché non chiudete il gioco": "non è fisica, ma si porta dietro la roba". Tutto ciò che vi si mette resta in memoria per l'intera partita.

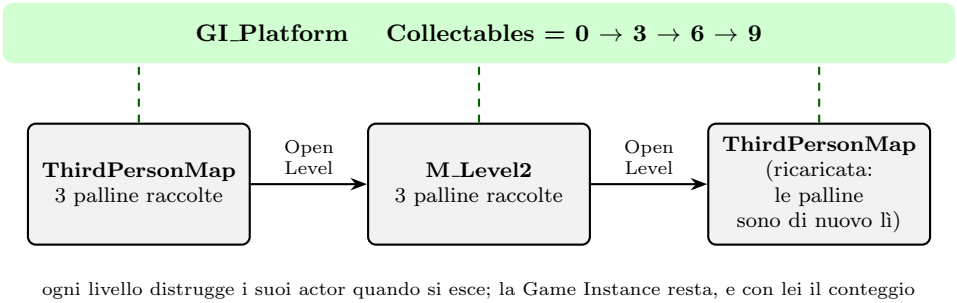


Figure 1.8.3: Perché la Game Instance. I livelli si aprono e si chiudono, e a ogni apertura gli actor (palline comprese) vengono ricreati; la Game Instance esiste sopra di loro per tutta la durata del gioco, e la sua variabile Collectables continua a contare. Schema ricostruito dalla dimostrazione.

Approfondimento: che cosa mettere nella Game Instance. Oltre ai contatori, la Game Instance è il posto tipico per le *impostazioni* scelte nel menù (volume, difficoltà), per il personaggio selezionato prima di entrare in partita (l'esempio che il docente farà a fine prima parte), per lo stato della sessione online e per un riferimento al salvataggio corrente. Ha però due limiti da ricordare. Il primo: vive quanto il *processo* del gioco, non di più; chiuso il gioco, i suoi valori sono persi. Per conservarli su disco, Unreal usa un oggetto diverso, il **SaveGame**, con i nodi *Create Save Game Object*, *Save Game to Slot* e *Load Game from Slot*. Il secondo: nel Play in Editor (PIE) la Game Instance viene creata a ogni pressione di Play, quindi ogni prova ricomincia da zero. Infine, una differenza con la Game Mode: in un gioco in rete la Game Mode esiste solo sul server, mentre ogni client ha la propria Game Instance; per i dati da condividere fra giocatori servono Game State e Player State.

1.8.7 Il collezionabile: BP_Collectible

"Adesso cosa facciamo? Dobbiamo creare quello che sarà il nostro collectable." Sarà "una pallina, come vi pare"; volendo, "possiamo anche creare il materiale". Nella cartella dei Blueprint ("mani con me, quando siete ordinati") si crea una **Blueprint Class** di tipo **Actor** ("non ci serve una roba particolare"), chiamata **BP_Collectible**.

I componenti. Si aggiunge una mesh: "volendo, possiamo pure senza mettere Static Mesh" e aggiungere direttamente il componente **Sphere** delle forme di base, che è già una Static Mesh con la sfera assegnata. La si rinomina **Mesh**. Poi la collisione: **Sphere Collision**, chiamata **Collision**. Le dimensioni:

- la mesh si rimpicciolisce a **scala 0,5** ("facciamo un po' più piccolina"). "Ricordatevi del **lucchetto**": se il lucchetto della scala è aperto "vi cambia solamente un valore"; se è chiuso, la modifica è **uniforme**, "vi cambia tutti e tre i valori";
- la Sphere Collision ha il suo **Sphere Radius**, "il raggio della nostra sfera", "come abbiamo visto anche con la torretta". Un trucco per regolarlo: mettere il mouse sopra il numero e trascinare a destra e a sinistra, "fate un po' da slide". Il docente arriva a circa **52** ("io ce l'ho 52, va benissimo").

Per il materiale, "ci possiamo sbizzarrire"; il docente ne prende uno fra quelli già presenti nel progetto, **Hologrid** ("carino"). Nella trascrizione il nome della cartella da cui lo prende è illeggibile.

La doppia collisione. "Una piccola accortezza che vi consiglio." La pallina ha due collisioni: quella della Sphere Collision e quella della mesh. Il docente lo mostra con un overlap provvisorio che distrugge l'actor: passandoci sopra, "c'è un momento proprio minuscolo in cui il character si ferma", "perché ha fatto **doppia collisione**". Prima di sparire, la pallina blocca il personaggio per un frame con la sua mesh. Per i collezionabili, "cose che voi prendete tranquillamente", è meglio che la mesh non abbia collisioni: sulla mesh, **Collision Presets** → **NoCollision**. Due vantaggi: si alleggerisce l'asset, "e andremo anche a semplificare il gameplay", perché quel micro arresto "il player lo sente", soprattutto "dopo tante" palline sparse in giro.

Il controllo sul player. Selezionata la Collision, si crea l'evento **On Component Begin Overlap**. Poi "quello che ormai abbiamo fatto 30.000 volte": **Other Actor** confrontato con **Get Player Pawn** (il nodo ==), collegato a un **Branch**: "sempre un controllo di norma".

Il problema: dove tenere il conto. "Adesso che cosa dobbiamo fare? Dobbiamo distruggere l'actor." Ma prima bisogna contare la pallina, e il conto non può stare nella pallina, che sparisce, né nel personaggio, che sparisce al cambio di livello: "visto che questa cosa dovrà essere sempre presente, possiamo usare la Game Instance". "Visto che la Game Instance è richiamabile praticamente ovunque", basta un tasto destro e **Get Game Instance**: "ecco a voi che questo blocchetto ci ridà indietro la nostra bella Game Instance". "Se avete delle cose molto importanti, come ad esempio il punteggio, i tempi, i costi, i salvataggi", "la Game Instance è *the way*".

1.8.8 Il cast: entrare nella scocca

"Perché, ad esempio, se noi tiriamo fuori" dall'uscita di Get Game Instance "e facciamo Get Collectables, vedete che non appare niente." Il nodo restituisce una referenza di tipo generico *Game Instance*: "ha preso una referenza, ma non ha preso l'interno". "Sta sulla **scocca**, diciamolo così": "prende le proprietà della Game Instance come asset a sé, ma non quello interno". "Quindi dobbiamo penetrare nella scocca. Come lo facciamo? Grazie a quello che viene chiamato il **cast**": **Cast To GI_Platform**.

Il nodo. Il cast "ci dice: tramite lui, vai a questo oggetto, prova a collegarti" a GI_Platform. Ha:

- l'ingresso **Object**, in cui va il Return Value di Get Game Instance;
- l'uscita di esecuzione principale, quella in alto, che corrisponde al **cast riuscito**: "Cast Successful non c'è scritto, ma questo sopra ci dice che il collegamento tra le due è avvenuto";
- l'uscita **Cast Failed**, "il collegamento non è avvenuto, non è riuscito a collegarsi";
- il pin blu **As GI Platform**, "la referenza dello specifico" oggetto, "come GI Platform fai questa cosa".

La festa di gala. Più tardi uno studente chiede: "non ho capito a cosa servono quei due nodi, il cast". "Serve per mettere in comunicazione la Game Instance con l'asset specifico": "come actor Game Instance, prova a entrare all'interno della GI_Platform". E la metafora: "è come dirti: mettiti un abito e prova a entrare a una **festa di gala**". "Se l'abito non è giusto", non entri; "se invece è quello corretto, vuol dire che puoi entrare e far parte, quindi puoi prendere tutti gli stuzzichini che ti pare". "Indosso l'abito giusto per il contesto giusto" (figura 1.8.4).

Approfondimento: il cast secondo Romero e Sewell. Nel capitolo 4 ("Understanding Blueprint Communication") il nodo **Cast To** "prova a convertire il tipo di una variabile referenza in un nuovo tipo specificato", e si capisce solo ricordando l'ereditarietà: una variabile di tipo *padre* può contenere un oggetto di tipo *figlio*, ma "conosce solo le variabili e le funzioni definite nel padre". L'esempio degli autori è quasi identico a quello della lezione, con la Game Mode al posto della Game Instance: un *BP_GameModeWithScore* con una variabile intera *GameScore* e una funzione *AddGameScore*, e un *BP_Collectable* che, su *Event ActorBeginOverlap*, prima fa un *Cast To ThirdPersonCharacter* sull'Other Actor (per verificare che sia il giocatore: è il secondo uso del cast, "testare in sicurezza se una referenza è del tipo desiderato"), poi *Get*

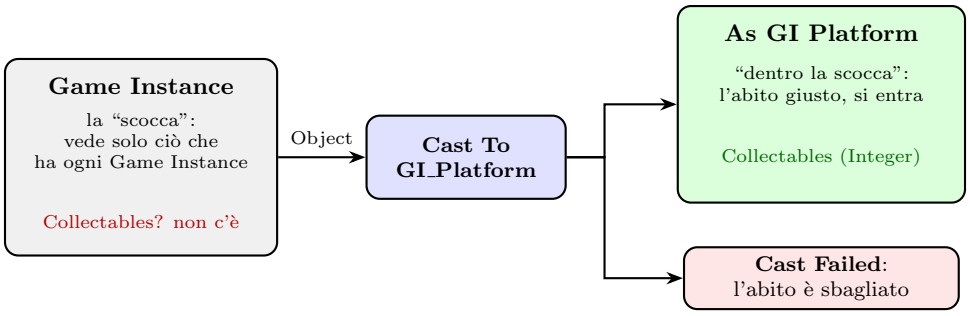


Figure 1.8.4: Il cast come lo spiega il docente. Get Game Instance restituisce una referenza generica, che conosce solo ciò che ogni Game Instance ha; il Cast To GI_Platform prova a "indossare l'abito" della nostra classe: se l'oggetto è davvero una GI_Platform, il pin As GI Platform dà accesso alle sue variabili; se no, si esce da Cast Failed. Schema ricostruito.

Game Mode, *Cast To BP_GameModeWithScore*, *Add Game Score* con 50 punti e *DestroyActor*. Gli autori notano anche che il Collectable usa il preset di collisione *OverlapAllDynamic*: un modo diverso dalla lezione per evitare la doppia collisione, cioè far sì che la mesh stessa generi overlap invece di bloccare. Due note pratiche: un cast fallito non è un errore, è un'informazione; e un cast ha un piccolo costo, per cui conviene farlo una volta e salvare il risultato in una variabile quando il valore serve spesso, come farà il docente nell'Animation Blueprint (sezione 1.8.20).

Un passaggio perso e un evento che non c'è. "Scusami, posso dire una cosa?" A uno studente si è cancellata la sfera, e ha perso gli ultimi passaggi: "dopo il Branch, tasto destro, Get Game Instance, e poi fai Cast To GI_Platform", "o come hai chiamato la tua Game Instance". A un'altra studentessa l'On Component Begin Overlap "non me lo dà, anche se lo provo a creare", né scorrendo gli eventi nei Details. "Ma è proprio la Collision, questa? È una Sphere Collision?" La causa, scoperta dalla studentessa stessa: "probabilmente l'avevo collegato alla mesh". Gli eventi di un componente compaiono nei Details solo quando quel componente è selezionato.

Increment Int. Dal pin As GI Platform si prende **Collectables**. "La possiamo prendere, e possiamo settarla volendo, però vi faccio vedere una cosa molto semplice: con gli int si può fare" il nodo **Increment Int**, quello con il ++: "è un blocco che ci dice: incrementa di 1". Ha come target la variabile, e la modifica direttamente. "Un'altra cosa carina": con −− c'è il **Decrement Int**, che toglie 1. "È una cosa standard che potete usare." Poi un **Print String** del valore, "per vedere quanti ce ne abbiamo", e infine, "l'ultima cosa ma non meno importante", **Destroy Actor** (figura 1.8.5).

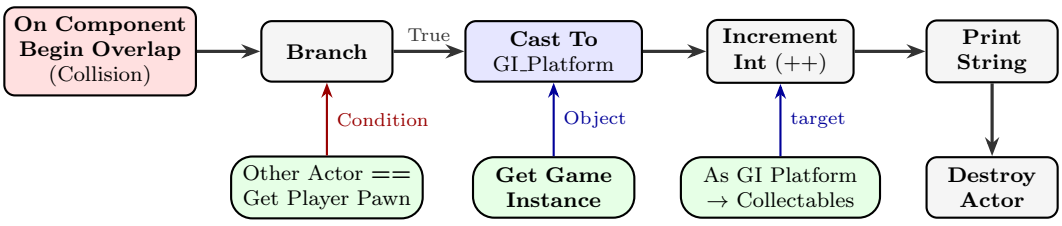


Figure 1.8.5: Il grafo di BP_Collectible. Se a entrare nella Sphere Collision è il player, si prende la Game Instance, la si converte in GI_Platform, si incrementa Collectables, si stampa il valore e si distrugge la pallina. La mesh è in NoCollision, per evitare la doppia collisione. Schema ricostruito.

1.8.9 Il tasto E: leggere il conteggio dal personaggio

Per vedere il conteggio in qualsiasi momento, il docente lo collega a un tasto nel personaggio. Si chiude BP_Collectible e si apre il **BP_ThirdPersonCharacter** ("Content, All Content, scrivete third, dovreste trovarlo"; "trovato, grazie"). "In questo caso non faccio un altro Third Person Character": per ora si lavora su quello del modello.

Gli eventi da tastiera. Si potrebbe usare il sistema di input del progetto, ma "possiamo usare tranquillamente degli input che ci vengono dati direttamente da Unreal": "tutta la tastiera, tutta l'interfaccia, anche il gamepad volendo, è mappato". Unreal "percepisce" ogni tasto, "Q, W, E, R, T, eccetera". Con un tasto destro nell'Event Graph si scrive **key** seguito dal nome del tasto: "in questo caso mi trova bilioni" di risultati (l'apostrofo, il minore, i numeri 0, 1, 2...); scendendo, c'è la **E**. "È un evento legato alla pressione del tasto E sulla tastiera."

Il grafo. Sul **Pressed** dell'evento E: **Get Game Instance**, **Cast To GI_Platform** ("alla pressione lui casta, quindi si prova a collegare"), dal pin As GI Platform **Get Collectables**, e un **Print String**. "Basta. Questo è quello che ci serve." Si mettono tre palline nel livello, "1, 2 e 3", un po' più indietro; in Play, raccolte le tre palline, le stampe dicono 1, 2, 3; premendo E, "ecco a voi che ho 3". "Ho raccolto 3 collectable."

Approfondimento: gli eventi Key e l'Enhanced Input. Gli eventi *Key* (*Keyboard Events* nel menù) sono il modo più rapido di provare qualcosa: si legano a un tasto fisico e hanno le uscite **Pressed** e **Released**. Sono però "cablati" su quel tasto: in un gioco vero si usano le **azioni di input** dell'Enhanced Input System (un asset *Input Action*, per esempio IA_Interact, e un *Input Mapping Context* che le lega a tasti e pulsanti del gamepad), così che i comandi si possano rimappare e funzionino con dispositivi diversi. Il modello Third Person di Unreal Engine 5 usa già l'Enhanced Input per muoversi, saltare e guardarsi intorno; il docente ha promesso di arrivarci più avanti (sezione 1.8.4). Un evento Key funziona solo nel Blueprint che riceve l'input, cioè nel Pawn posseduto o nel Player Controller: messo in un actor qualsiasi del livello, non scatterebbe senza un *Enable Input*.

1.8.10 Il volume che azzera: BP_VolumeShop

"Potremmo anche vedere: esempio, vogliamo fare un volume che ci resetta a zero?" Un'idea di gioco: "andiamo allo shop e usiamo questi collectable per spendere, per prendere una reward". Per ora, il volume si limita ad azzerare. Prima, il docente si assicura che tutti siano al passo ("mi sembrate abbastanza sicuri, perfetto") e risponde alla domanda sul cast (sezione 1.8.8).

Il Blueprint. Un nuovo Actor, **BP_VolumeShop**, con una **Box Collision**. Sull'**On Component Begin Overlap** della box, "sempre il solito discorso": Other Actor == Get Player Pawn, Branch; poi **Get Game Instance**, **Cast To GI_Platform**, e dal pin As GI Platform **Set Collectables** a **0**.

Il Text Render. "Facciamo una cosa per vedere almeno il volume": un componente **Text Render**, "che mette un testino" nel mondo. "Questo è comodo." Nei Details, a destra, alla voce **Text** si scrive il contenuto: il docente scrive qualcosa come "Volume Shop" o "Reset collectable" (la trascrizione dice "Reserva collettivo"). Più tardi aggiunge anche un Print String sull'azzeramento, con un testo che nella trascrizione suona come "Collect Doubles" ("sembra quasi un insulto, però va bene").

Vedere il volume in gioco. "Per far vedere il volume, ragazzi, possiamo fare una cosa molto *easy peasy*": sulla Box, togliere la spunta a **Hidden in Game**. "Hidden in Game è un booleano che ci dice: guarda, se questo è vero, in gioco dev'essere nascosto." Togliendolo, i bordi della box si vedono anche in Play.

Il primo problema: il cast senza Object. A una studentessa, appena salvato, "è comparso un quadrato" nella viewport "con scritto Blueprint", e un errore. Nella trascrizione il dettaglio del quadrato non è chiaro (potrebbe essere il Text Render, che di default mostra la parola "Text", o l'icona dell'actor). L'errore del grafo, invece, sì: il Get Game Instance era stato tirato fuori ma non collegato. "Devi tagliare il Get Game Instance e metterlo dentro Object"; poi dal cast "devi fare Set, così: Set Collectables, va bene lo zero", e collegarlo "alla nuova esecuzione".

Il secondo problema: la classe genitore sbagliata. Compilando, un nuovo errore: GI_Platform "non eredita da istanza di gioco" (nell'editor in italiano la Game Instance è l'*Istanza di gioco*). Prima verifica: i Project Settings. "Scrivi istanza", perché l'editor è in italiano; e infatti nella voce Game Instance Class la classe non c'è. Poi la scoperta: la studentessa l'aveva chiamata Platform Game Instance, e soprattutto, aprendo l'asset, "non è una Game Instance, questa è un'altra tipologia di asset": "non hai come parent class la Game Instance". Rimedio:

1. ricrearla: tasto destro, Blueprint Class, cercare Game Instance (*Istanza di gioco*), chiamarla **GI**; controllare che la *classe genitore* sia *Istanza di gioco*;
2. nei Project Settings, sceglierla in Game Instance Class;
3. ricreare dentro la variabile Collectables;
4. rifare **tutti i cast**: "devi andare su tutti i momenti in cui hai castato", togliere i vecchi Cast To e fare **Cast To GI**, nel collezionabile, nel volume e nel personaggio;
5. ricollegare l'esecuzione *attraverso* il cast: "se no non passi per il cast".

Copiare il grafo da un Blueprint all'altro non basta, perché i vecchi nodi puntano alla vecchia classe. Alla fine, nel personaggio, un *Get* ("non un Set") di Collectables nel Print String: "a posto, ottimo". "Grazie mille."

Approfondimento: perché la classe genitore conta. Il menù di Game Instance Class nei Project Settings mostra solo le classi che *discendono* da GameInstance: una classe creata a partire da un altro genitore (per esempio Object, o Actor) non compare, e un cast verso di essa da Get Game Instance non può mai riuscire. Nei Blueprint il genitore si vede in alto a destra nell'editor (*Parent class*) e si può cambiare da *Class Settings* → *Parent Class*, ma cambiarlo su una classe già piena di logica è rischioso; per una classe quasi vuota, come qui, ricrearla è la via più pulita.

1.8.11 Il volume che carica un livello: BP_VolumeLoadLevel

"Questo, ragazzi, ci serve a vedere come possiamo condividere dei dati", "passarli tranquillamente da un punto all'altro". "In qualsiasi gioco ci sia qualcosa che collezionate", "questo ci permette di appoggiarci alla Game Instance". E il docente collega la Game Instance agli errori visti nelle lezioni precedenti: "molti errori che abbiamo visto sono perché non è valido questo, non è valido quest'altro"; la Game Instance, "essendo perenne", "è onnipotente, potremmo dire", e "ci permette di avere una comunicazione molto diretta con essa". Ora serve il secondo pezzo: "un piccolo volume che ci permette di trasportarci da un livello all'altro".

Il Blueprint. Un nuovo Actor ("tanto è un volume, ragazzi, molto semplice"), **BP_VolumeLoadLevel** ("chiamatelo come volete"), "quasi la stessa logica del volume shop": una **Box Collision**, l'**On Component Begin Overlap**, "il nostro perenne amico", e il controllo Other Actor == Get Player Pawn con il Branch. Sul True, "un blocco singolo, semplicissimo": **Open Level (by Name)**.

Il nome del livello come variabile. "Per semplificarci la vita", dal pin **Level Name** si fa **Promote to Variable**: una variabile **LevelName** di tipo Name, con **l'occhiettino** aperto. Così ogni istanza del volume nel livello può portare a un livello diverso, e il nome si scrive nei Details: "è molto più comodo". Per "una cosa un po' più carina", anche qui un **Text Render**, con scritto **Go to next level**.

La cartella Maps e il nuovo livello. "Ragazzi, se sto andando molto veloce, ditemelo, io rallento." Si crea una cartella **Maps**, per le mappe. Un livello si può creare in due modi:

- con il tasto destro nel Content Browser → **Level**: "vi è di base un **empty level**", "non c'è cielo, non c'è niente";
- da **File** → **New Level**, che fa scegliere un *preset*: il docente consiglia **Basic**, con cielo, luce e pavimento già pronti.

Al cambio di mappa l'editor chiede di salvare le modifiche del livello precedente. Il nuovo livello si chiama *Untitled*: "la prima cosa da fare quando avete una mappa nuova è **Ctrl+S**", e scegliere il nome. Il docente usa il prefisso **M_** per "mappa": **M_Level2**.

Il nome esatto. Nel volume del primo livello (la *ThirdPersonMap*), nei Details, si scrive il Level Name: **M_Level2**. "Deve essere preciso." Il trucco per non sbagliare: cercare il livello nel Content Browser, premere **F2** come per rinominarlo, **Ctrl+C** per copiare il nome, e **Ctrl+V** nel campo del volume. "Sappiate che queste stringhe sono **case sensitive**": "se c'è una L maiuscola e fate la minuscola, vi dà errore". In Play: "ecco a voi che ho un collision overlap, e mi carica il nuovo livello". "Vi funge? Sì. Ottimo." A uno studente che si è perso l'ultimo passaggio il docente rispiega il trucco di F2 e Ctrl+C: "il testo è preciso, è quello che deve essere".

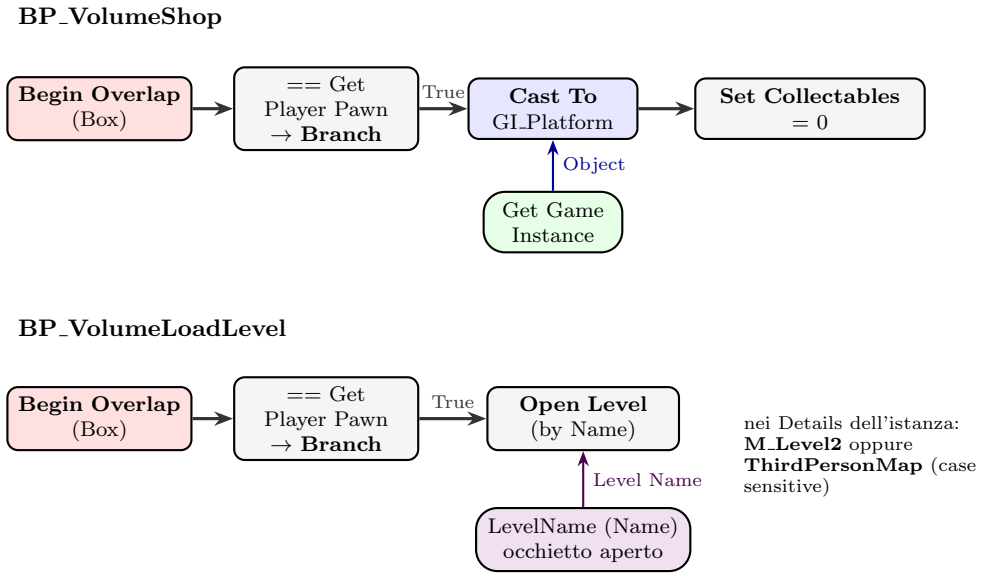


Figure 1.8.6: I due volumi. In alto, lo "shop" che riporta a zero il conteggio nella Game Instance; in basso, la porta verso un altro livello, con il nome del livello promosso a variabile pubblica, così che ogni istanza possa portare altrove. Entrambi hanno un Text Render e la box visibile in gioco (Hidden in Game disattivato). Schema ricostruito.

Approfondimento: Open Level by Name e by Object Reference. Unreal Engine 5 offre due versioni del nodo. **Open Level (by Name)**, quello della lezione, prende un *Name*: è semplice, ma se il livello viene rinominato o spostato il nome non si aggiorna, e l'errore compare solo in gioco. **Open Level (by Object Reference)**, che Romero e Sewell usano nel capitolo 8 per il pulsante Restart del menù di vittoria (ricaricare lo stesso livello "azzera ogni aspetto del livello, compresi bersagli, munizioni raccoglibili e giocatore"), prende invece una referenza all'asset del livello (un *soft reference* a un World), scelta da un menù a tendina: non si può sbagliare a scrivere, e segue il livello se lo si rinomina. In entrambi i casi il livello corrente viene *scaricato*: tutti i suoi actor, il personaggio compreso, vengono distrutti, ed è proprio per questo che il conteggio deve stare nella Game Instance. Per un gioco pubblicato, infine, i livelli da caricare devono essere inclusi nel

pacchetto (*Project Settings* → *Packaging* → *List of maps to include*), altrimenti nella build il caricamento fallisce anche se nell'editor funziona.

1.8.12 La prova fra i due livelli

Si apre M_Level2 e lo si prepara come il primo: alcune palline BP_Collectible ("1, 2 e 3, poi fate come volete"), un po' sollevate da terra, e un secondo BP_VolumeLoadLevel che riporta indietro, con il Level Name **ThirdPersonMap** ("ripeto: scrivete third, F2, Ctrl+C, Ctrl+V"). Attenzione al punto di partenza: per la prova si apre di nuovo la ThirdPersonMap, "se no vi inizia da questa mappa"; e si salva tutto.

Il risultato. In Play: 1, 2, 3 nel primo livello; la porta; 1, 2, 3 nel secondo; E: "quanti collectable ho? 6". Ritorno al primo livello: le palline "mi sono rispawnate", ma il conteggio dice ancora 6. Sul volume shop: 0. Di nuovo 1, 2, 3. "Vi funge tutto?" "Questo, ragazzi, è una forma molto semplice per passarsi dei dati" con la Game Instance, "qualsiasi tipologia di dato che debba essere globale".

La scelta del personaggio. Un ultimo esempio: "avete presente quando, nei vostri giochi, scegliete un personaggio, A, B o C?" Un possibile uso della Game Instance è ricordare la scelta: "prendere la Skeletal Mesh che abbiamo scelto", e dire alla Game Instance: "guarda, ricordati che è il personaggio B quello che ho preso", "per far sì che non si resetti ogni volta" che si cambia livello.

1.8.13 Mixamo: scegliere il personaggio

"Abbiamo voluto finire prima dei 30", cioè prima della pausa delle 17:30, "così magari adesso switchiamo alle nostre bellissime animazioni". Gli studenti hanno già creato l'account su Mixamo, o hanno fatto l'accesso. "Piccola descrizione di Mixamo, ma poi andiamo in pausa, perché manca veramente poco."

Characters e Animations. Mixamo ha due sezioni: **Characters** e **Animations**. "Per adesso quello che ci interessa sono i Characters." Mixamo "in rari casi" non si adatta benissimo, "però, essendo che è stato usato tantissimo per personaggi in Unreal Engine, il **rigging** è piuttosto simile a quello del nostro **Manny**, il nostro manichino principale".

La scelta. "Per questi ultimi 5 minuti sceglietevi un character che vi piace, qualsiasi, veramente, e lo scaricate." Qualche consiglio:

- evitare i personaggi incompleti, "come questo qua che c'ha il braccio mozzo";
- preferire quelli "un po' più umanoidi" (vanno bene "anche i mostri, questi qua", se hanno una forma umana);
- se sul personaggio è applicata un'animazione, "c'è questa X che dovete eliminare", "perché dovete metterlo in **T-pose**", la posa a braccia aperte da cui parte ogni rig.

Il docente ha preso **Paladin J Nordstrom**, un cavaliere. Download, con il formato **FBX Binary** e la posa **T-pose**. "Quando siamo tornati dalla pausa lo importeremo, per far sì che questo diventi il nostro nuovo character."

La pausa. "Sono le 26", le 17:26: "io direi di rivederci tra un po', sempre le solite 20 minuti. Facciamo 55, 18, solito orario."

1.8.14 La pausa: i problemi degli studenti

In pausa il docente resta collegato con chi ha problemi, "dall'altro PC" per la condivisione. Tre studenti, quattro questioni.

Mettere a terra un componente nel Blueprint. "Quando siamo nei Blueprint, come lo metto a terra?" Nella viewport del livello c'è la scorciatoia **End** ("FN e freccia destra" sul portatile dello studente), che fa cadere l'oggetto selezionato sul pavimento; nella viewport del Blueprint no: "non c'è una combinazione di tasti". Si fa "a mano": mostrare i componenti nascosti, selezionare la box, e cambiarne la **Z** nei Details, provando: 40, "tipo 30, 35", infine "75, direi che va bene". "Questa era la prima cosa, ma era una stupidaggine."

L'ascensore che non risale. Lo stesso studente: se salta sulla piattaforma comandata dal pulsante mentre sta scendendo, "succede questo, e poi non risale più". È esattamente il caso del fix di inizio lezione, e il docente lo rifà con lui: una variabile booleana **Activated**; sull'evento di attivazione un **Branch** con Activated, e dal **False** un **Set Activated** a true; sul Begin Overlap, Activated a false ("trascina da Activated, quello a sinistra, quello delle variabili"). Il valore di default di Activated deve essere *false*. Non basta. Si controlla che sia la piattaforma giusta ("sì, è questa"), si scopre che il Billboard "sta sotto" (come doveva, nel compito della piattaforma al contrario), si guarda la timeline ("va benissimo"). Il docente prova un'altra strada: spostare il controllo sul **Finished** della timeline, collegando il ramo True al **Reverse**. "E ovviamente non risale se poi ci va sopra." Ultimo indizio: il **play rate** e la durata ("la durata è 1, va bene"): la piattaforma "fa istantaneo, non si riesce neanche a vedere il movimento". Il caso resta aperto: "mi fai un paio di screen, me lo guardo un po' in separata sede", da mandare "sul canale Materiale Didattico" di Discord.

L'Audio Component, di nuovo. Il problema di Filippo (sezione 1.8.1) ha un sintomo in più: "quell'errore lo fa solo se arrivo fino alla fine; altrimenti non me lo fa". Ed è comparso "da quando ho creato la seconda piattaforma", copiando e incollando la prima. Il docente controlla che SoundPlatform sia il target dei nodi, che il reset del **Do Once** sia collegato; poi fa rifare la variabile da capo: cancellare SoundPlatform, e dal **Return Value** dello Spawn Sound fare **Promote to Variable** ("la chiami Sound"), poi ricollegare il Set, i Set Pitch Multiplier e lo **Stop** ("ne manca uno: Stop"). L'errore resta. "Mi copi questo errore e me lo mandi su Discord, giusto perché voglio capire un attimo."

Approfondimento: un'ipotesi sull'errore di fine corsa. Con i sintomi descritti (il suono funziona, l'errore compare solo quando la piattaforma arriva in fondo, e riguarda l'Audio Component) la causa più probabile è quella anticipata nell'approfondimento di inizio capitolo: lo **Stop** sul Finished della timeline ferma il suono, e il suono spawnato con *Auto Destroy* attivo distrugge il proprio Audio Component. La variabile resta piena ma non valida, e il successivo Set Pitch Multiplier (o il successivo Stop, se il Do Once non viene resettato in tempo) la usa. Le cure possibili: deselezionare *Auto Destroy* nello Spawn Sound at Location (e allora fermare il suono con Stop senza perderlo), oppure controllare la variabile con **Is Valid** prima di ogni uso, oppure resettare il Do Once nello stesso punto in cui si ferma il suono, così che alla volta successiva il suono venga ricreato. La trascrizione non dice come sia finita.

La timeline con il nome sbagliato. Marco: "quando vado in play, anche se non faccio niente", all'uscita compare un errore su **Set Play Rate**. "Clicca su Set Play Rate." La causa: nel Blueprint ci sono *due* referenze a timeline, **Movement Platform**, una delle quali è rimasta da una versione precedente ("c'è un'altra timeline che si chiama Movement Platform"). Il Set Play Rate usa quella vecchia. Rimedio: nel pannello *Components*, "sotto Variabili", eliminare la vecchia ("tanto non ti servirà più"), poi trascinare la timeline giusta nel grafo, **Get**, e collegarla come target "sia lì, sia sotto". "Grazie."

1.8.15 L'import del personaggio

Alla ripresa la webcam continua a non funzionare ("facciamo un'ultima prova; no, proprio no"). "Avete scelto il vostro character? L'avete scaricato?" "Sì." Il docente avverte che la lezione sulle animazioni continuerà la volta successiva: "oggi andremo a vedere un po' come impostare il progetto delle animazioni; vediamo dove arriviamo".

Importare l'FBX. Nell'Esplora file si trova il file scaricato, nel caso del docente *Paladin J Nordstrom.fbx*. Due modi di importarlo: il pulsante **Import** del Content Browser, che chiede di scegliere il file, oppure, "molto più comodo", trascinare il file direttamente nel Content Browser. Compare la finestra delle opzioni di import.

Offset Translation e Offset Rotation. "Sappiate una piccola particolarità": guardate "soprattutto l'**Offset Translation** e l'**Offset Rotation**". Sono "la posizione in cui, sul pivot, viene portato il nostro asset", "un po' come abbiamo importato anche le Static Mesh": "vi ricordate che con la torretta c'era il pivot che era tutto sballato?" "Guardate se sono tutti a zero. Dovrebbero essere tutti a zero." Il docente, importando il cavaliere, l'aveva trovato "a tipo meno 270 sulla rotazione", e l'ha corretto. Il campo **Skeleton** si lascia vuoto: "non ci serve, perché stiamo appunto importando uno scheletro". "Per tutto il resto potete tranquillamente andare": **Import**.

Il crash. A questo punto Unreal si chiude: "scusate ragazzi, ma a quanto pare mi è crashato Unreal". L'errore è di **video memory**, "out of video memory". "Purtroppo questi sono problemi che capitano." Un consiglio generale, per i PC: "cercate di non riempirlo troppo", il disco, "perché Unreal crea anche dei file sull'hard disk", temporanei, "quasi come la RAM"; "cercate di non stare proprio saturi di memoria". Il docente, "per esigenze", ce l'ha "praticamente così pieno". Riaperto l'editor, si reimporta, si verificano di nuovo i valori ("lasciate tutto così: non mi crashare di nuovo"), e l'import va a buon fine.

Gli asset creati. L'import crea diversi asset (tabella 1.8.2):

- un'**animazione**, riconoscibile dal colore **verde scuro** ("verde, però scuretto"; quello chiaro è il materiale): "questo verde scuro sono tutti gli asset che sono animazioni". È la T-pose, "un'animazione ferma", con un frame solo: "potete tranquillamente eliminarla", **Delete**;
- i **materiali** e le **texture**, nel caso del docente quello dell'armatura;
- la **Skeletal Mesh**, il corpo con le ossa;
- lo **Skeleton**;
- il **Physics Asset**.

Aprendone uno, in alto a destra si passa da un tipo di editor all'altro: "questo è il Physics, e questo è lo scheletro".

Asset	Che cos'è	Che cosa se ne fa
Skeletal Mesh	la mesh con lo scheletro: "il corpo e le ossa"	va nel componente Mesh del Character
Skeleton	la gerarchia delle ossa (bones) e delle giunture	le animazioni e l'Animation Blueprint sono legati a lui
Physics Asset	i corpi fisici delle ossa: collisioni, impatti, ragdoll	non visto a lezione ("è un po' un casino")
Animazione (verde scuro)	la T-pose, un solo frame	si cancella
Materiali e texture	l'aspetto del personaggio	a volte vanno creati a mano (sezione 1.8.25)

Table 1.8.2: Gli asset creati dall'import dell'FBX di un personaggio di Mixamo.

1.8.16 Lo scheletro: ossa e giunture

Si apre lo **Skeleton**. "Se cliccate sullo Skeleton, vi si presentano tutte le ossa del corpo"; selezionandole, "queste ossa sono visibili con un reticolo". "Questo è il lavoro del rigger, o comunque dell'animatore che andrà a lavorarci sopra."

Le spine e le giunture. Le ossa, nell'editor, "sembrano appunto delle spine": "le spinette sono le bone". "I **cerchi**, invece, sono i punti di **giuntura**", come quello sulla spalla. "Tramite queste spine, il nostro character è in grado di ruotare il proprio corpo e compiere delle animazioni."

Le ossa si ruotano. "Mi raccomando: mai muovere una spine, perché vi faccio vedere subito cosa succede, non è molto carino." Spostato un osso, il personaggio si allunga in modo innaturale: "avete proprio Slenderman"; e, scherza il docente, "potete fare il gioco dei gonfiabili che stanno davanti all'autonoleggio". "Le ossa non si muovono, ragazzi, le ossa **ruotano**" quando si compie un'animazione. "Questo è un concetto proprio base." Ruotando, invece, "secondo il limite" delle giunture, "si compiono le animazioni": "è tutta una sequenza di rotazioni". Programmi esterni come **Blender** e **Maya** "gestiscono le animazioni sicuramente in miglior modo, tuttavia di base il concetto è rotazione". Nell'editor, dopo la prova, basta non salvare (o annullare con Ctrl+Z) per rimettere l'osso al suo posto.

Il Physics Asset. "Questo non lo vedremo, ma" gestisce "le particolarità fisiche di un osso": "come collide", "come si muovono", "gli impatti fisici", "quindi cose molto più precise e molto più dettagliate". "Non lo andremo a vedere perché è un po' un casino": per ora interessano le ossa.

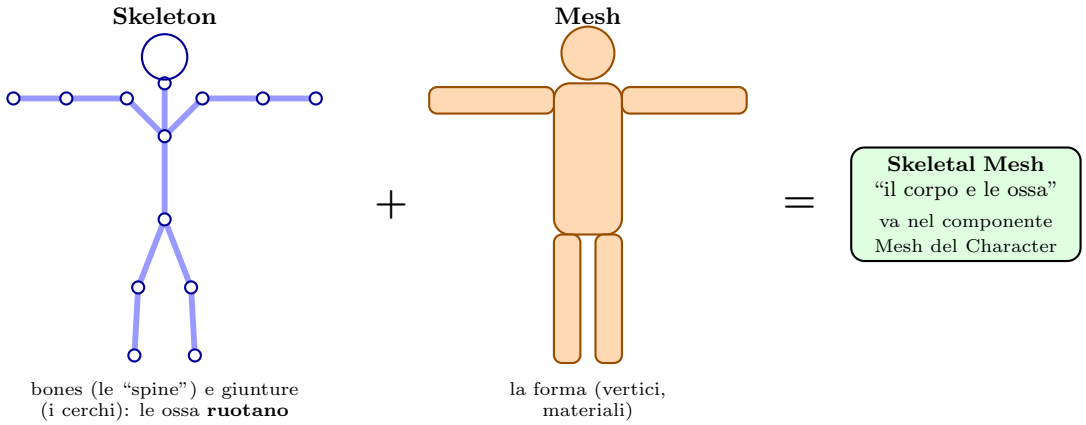


Figure 1.8.7: Skeletal Mesh e Skeleton secondo la lezione: due asset diversi. Lo Skeleton è la gerarchia delle ossa, con le giunture; la Skeletal Mesh è la forma legata a quelle ossa. Le animazioni sono rotazioni delle ossa, e sono legate allo Skeleton. Schema ricostruito, non in scala.

Approfondimento: perché due asset. Romero e Sewell (capitolo 17, "Animation Blueprints") spiegano la separazione così: "uno Skeleton è una gerarchia di ossa interconnesse, usata per animare i vertici poligonali di una Skeletal Mesh"; e "siccome l'animazione è fatta sullo Skeleton, può essere condivisa da molte Skeletal Mesh che usano lo stesso Skeleton". È il motivo per cui un gioco può avere decine di personaggi diversi con un solo set di animazioni, e il motivo per cui, più avanti, le animazioni del manichino di Unreal potranno essere *ritargettate* sul cavaliere di Mixamo, che ha uno scheletro simile ma non identico. Gli autori elencano anche i cinque editor che si aprono dagli asset di animazione, con i bottoni in alto a destra già visti dal docente: *Skeleton Editor*, *Skeletal Mesh Editor*, *Animation Editor*, *Animation Blueprint Editor* e *Physics Asset Editor*.

1.8.17 Le animazioni di Mixamo

"Torniamo su Mixamo." L'importante è che nella viewport di Mixamo ci sia il proprio personaggio, "non un altro"; poi si passa alla categoria **Animations**. Servono **tre animazioni** principali: **idle**, **walk** e **run**. "La idle, per chi non lo sapesse, sono le animazioni di stallo": il personaggio "sta fermo, sta respirando", il *breathing*.

I controlli di Mixamo. Sulla destra, per ogni animazione, ci sono alcuni cursori; il docente li prova su animazioni diverse per farli vedere bene:

- **Overdrive**: la velocità dell'animazione. "Se aumento l'overdrive, questa animazione si velocizza sempre di più; se lo diminuisco, sarà lentissima";
- **Character Arm-Space**: su alcuni modelli, la distanza delle braccia dal corpo, "diventa sempre di più oppure diminuisce". Attenzione al *clipping*, le braccia che entrano nel corpo: "infatti clippa malissimo". Il docente suggerisce valori intorno a 50;
- **Trim**: "tagliare l'animazione", prenderne solo un pezzo. Ma "viene fuori malissimo, perché non c'è il ritorno all'idle": l'animazione tagliata non si chiude su sé stessa;
- **Mirror**: presente solo su alcune animazioni, "semplicemente flippa l'animazione", la *specchia*. Il docente lo mostra con una Capoeira.

Root motion e in place. Il punto più importante. Il docente apre un *Walking*: "vedete che il character sta andando dritto, cioè sta percorrendo dello spazio". "Questa tipologia di animazione viene chiamata **root motion**": le animazioni in root motion "ereditano il movimento già nell'animazione". L'esempio è *Dark Souls 3* ("qualcuno ha giocato Dark Souls 3? Perfetto"): alcune armi hanno abilità speciali, le *weapon art*, e in un video il docente cerca quella della *Holy Sword* ("questa non è in root motion, perché sta ferma"; "eccola, questa qua"): "quando io la compio, mi sposto nello spazio già nell'animazione". "Non sono io che gli do il movimento quando premo il tasto": "metto il piede in avanti, o faccio l'affondo", e il personaggio avanza perché lo fa l'animazione. Le animazioni in root motion "spesso sono usate per abilità" di questo tipo.

"In questo caso, però, su Mixamo non dobbiamo avere un'animazione root motion", perché "erediteremmo un movimento che non ci interessa". "Noi, a livello del nostro gioco su Unreal, vogliamo che l'animazione si compia, ma quella che si muove è la **capsula**": "l'animazione segue la capsula"; "c'è la mesh con l'animazione su quella capsula, e di conseguenza si crea il movimento, o l'illusione del movimento". Quindi si spunta **In Place**: "come potete vedere, adesso l'animazione è sul posto". "Se non trovate una spunta In Place, consiglio di cambiare animazione: ce ne sono tante" (figura 1.8.8).

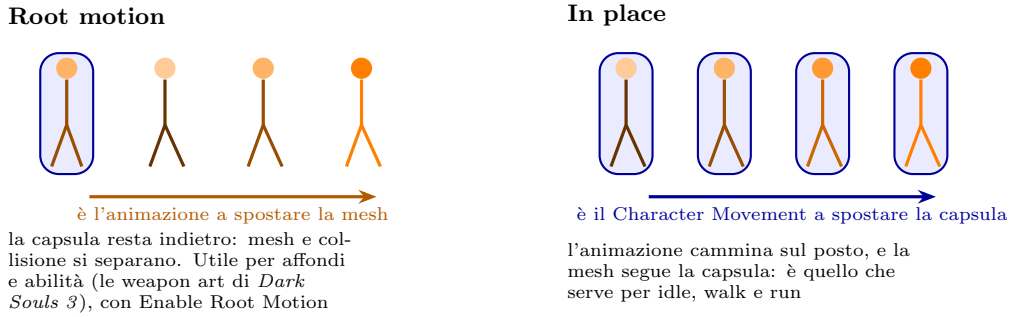


Figure 1.8.8: Root motion e in place. In un'animazione root motion lo spostamento è dentro l'animazione; in un'animazione in place il personaggio cammina sul posto, e a muoversi nel mondo è la capsula del Character. Per la locomozione di base la lezione usa solo animazioni in place. Schema ricostruito.

Le impostazioni di download. Al download Mixamo chiede alcune opzioni (tabella 1.8.3):

- **Format**: **FBX Binary**, "ci va bene così";
- **Frames per Second**: **30**. "Le animazioni devono essere allo stesso tipo di frame": "meglio sempre prenderle tutte quante allo stesso numero di frame", "perché se no avete dei problemi", soprattutto "nel *blending*, cioè nel mixare" due animazioni con frame rate diversi;

- **Skin**: **Without Skin**. "La skin è se volete prendere l'animazione con il modello: non ci interessa, perché il modello ce l'abbiamo già";
- **Keyframe Reduction**: **none**, "non ci interessa" ridurre i keyframe.

Il docente scrive in chat le tre animazioni da cercare, "idle, walk e run/sprint", lasciando "la libertà di scegliere quelle che volete": "importante che abbiate un'animazione che respira, sta fermo, eccetera, una che cammina e una che fa lo sprint". Lui ha preso *Idle*, *Walking* e *Slow Run*. "Vi lascio 5, 10 minuti."

Opzione	Valore	Perché
Personaggio		
Format	FBX Binary	il formato che Unreal importa
Pose	T-Pose	la posa di riferimento del rig; togliere l'animazione applicata (la X)
Animazioni (idle, walk, run)		
Format	FBX Binary	come sopra
Skin	Without Skin	il modello c'è già: serve solo il movimento delle ossa
Frames per Second	30	tutte allo stesso frame rate, per non avere problemi nel blending
Keyframe Reduction	none	nessuna riduzione dei keyframe
In Place	spuntato	niente root motion: a muoversi è la capsula
Overdrive, Arm-Space, Trim, Mirror	a piacere	velocità, distanza delle braccia (attenzione al clipping), taglio, specchio

Table 1.8.3: Le impostazioni di download di Mixamo indicate a lezione.

Approfondimento: Mixamo e Unreal. Mixamo è un servizio gratuito di Adobe, a cui si accede con un account Adobe; personaggi e animazioni si possono usare anche in progetti commerciali. Il suo rig ha una gerarchia simile a quella del manichino di Unreal (Manny e Quinn), ma i nomi delle ossa sono diversi (le ossa di Mixamo hanno il prefisso *mixamorig:*), e manca un osso *root* alla base, sotto il bacino. Per questo le animazioni di Mixamo in root motion non funzionano bene in Unreal senza correzioni: lo spostamento è registrato sull'osso del bacino (*Hips*) invece che su un root. Con le animazioni *In Place*, come quelle della lezione, il problema non si pone. La mancanza dell'osso root e i nomi diversi sono anche il motivo per cui, per usare sul cavaliere le animazioni del manichino (il salto, la caduta), servirà il **retargeting** annunciato dal docente, con gli asset *IK Rig* e *IK Retargeter* di Unreal Engine 5.

1.8.18 Importare le animazioni sullo scheletro

Dopo i minuti di scelta ("ci siete tutti, ragazzi? Sì, perfetto") si importano le animazioni. I file scaricati hanno il nome dell'animazione ("slow run, walking, eccetera"); "l'importante è che ci sia FBX sopra". Al docente una delle due versioni di Walking era rotta, e usa *Walking 1*. "Consiglio: fate meglio una cartella": **Animation**. "Fatela una per una, vi consiglio."

Lo scheletro obbligatorio. La finestra di import è "praticamente la stessa schermata", "ma differenza sostanziale": in basso, **Import** è grigio, non si può premere. "Per l'animazione in particolare gli serve uno scheletro a cui adattare l'animazione." Nella sezione **Common** c'è il campo **Skeleton**: si apre la tendina, in cui compaiono "tutti gli scheletri", compreso quello del manichino "che usiamo per Unreal", e si sceglie *il proprio*, quello creato dall'import del personaggio. "A questo punto, Import vi diventa blu." Nella stessa finestra il docente indica anche la spunta sull'import della sola animazione (nella trascrizione: "fate sì che Import Only Animation sia ok"); la frase che segue è confusa.

Targettare. "Ecco a voi che si è creata un'animazione." "Il file di per sé è a parte, però lui ha **targettato**" ("prendete questo termine") "la nostra animazione su quello scheletro in particolare". Un'animazione appartiene a uno scheletro, e si può usare solo con le mesh di quello scheletro.

Approfondimento: le opzioni di import di un'animazione. Nella finestra FBX di Unreal Engine 5, quando il campo *Skeleton* è impostato e la spunta *Import Mesh* è tolta, Unreal importa *solo* le animazioni contenute nel file, senza creare un'altra Skeletal Mesh e un altro scheletro; con *Import Animations* attivo, ogni animazione diventa un asset **Animation Sequence**. È il caso dei file di Mixamo scaricati *Without Skin*, che contengono solo le ossa animate. Se invece si lascia vuoto il campo Skeleton su un file con la mesh, Unreal crea uno scheletro nuovo a ogni import: è facile ritrovarsi con due scheletri quasi identici ("mi sa che ci sono due ragazzi che hanno importato due" scheletri, osserva il docente sulla sua tendina) e con animazioni non compatibili fra loro.

1.8.19 L'editor dell'animazione

Con un doppio clic sull'animazione si apre l'**Animation Editor**, "che vi permette di visualizzare tutte le particolarità della nostra animazione".

I controlli. In basso c'è il pannello di controllo: andare al primo o all'ultimo frame (*To Front*, *To End*), al frame precedente o successivo ("frame per frame per frame"). "Le animazioni sono costituite da frame": questa ne ha **30**. Con **spazio**, o con il pulsante, *Play* e *Pause*.

Asset Details. Sul lato dell'editor c'è il pannello **Asset Details** ("se non vedete una finestra, la potete trovare su Window → Asset Details", "o Finestra → Dettagli dell'asset, suppongo, in italiano"). Ci sono molti dati "che nessuno andrebbe a vedere"; le cose principali:

- **Loop**: "se questa animazione la vogliamo che vada in loop": "quando finisce, ricomincia";
- **Rate Scale**: a 1 è la velocità normale; "se metto 1,5 l'animazione è più veloce"; a 2 ancora di più; "40, diventa un casino". "È un po' come l'overdrive che abbiamo visto" su Mixamo;
- **Root Motion**: la spunta *Enable Root Motion* non è attiva, "va bene così", perché l'animazione è in place.

Lo stesso per le altre due animazioni: Idle, Slow Run ("vi chiederà sempre lo scheletro"). "Se aprite le vostre animazioni, sono giuste, sono davanti a voi?" Alla fine: "dovreste avere tutte e tre le nostre belle animazioni".

1.8.20 L'Animation Blueprint

"La domanda sorge spontanea: come facciamo a metterle su un character?" Con l'**Animation Blueprint**.

La creazione. Nella cartella del personaggio (il docente l'ha chiamata **Characters**) si crea una cartella **Blueprint**. Poi tasto destro, **Animation → Animation Blueprint**. La finestra chiede "di quale scheletro": si sceglie il proprio (il docente, "il mio paladino"), **Create**, e il nome: **ABP_Knight**, con il prefisso ABP per *Animation Blueprint* ("io ho chiamato Knight, però voi chiamatelo Ciccio Baciccio"). A uno studente che non ha capito, il docente ripete: "Animation, Animation Blueprint; vi chiederà lo scheletro; selezionate lo scheletro vostro". L'Animation Blueprint "sarà la logica, quindi la parte di programmazione" delle animazioni "del nostro bellissimo nuovo character".

L'editor. "L'Animation Blueprint è abbastanza pieno di roba", "ci si può confondere facilmente". Le parti:

- a sinistra, la **preview**, con il cavaliere in T-pose: "e va benissimo, perché è la sua forma base";
- l'**AnimGraph**, con il blocco **Output Pose**: "tutti i calcoli che andremo a fare di animazioni" si collegano a lui, "e questo blocco ci dice: guarda, esce fuori questa roba";

- l'**Event Graph**, "come ce l'abbiamo in un classico Blueprint", dove si possono "creare e prendere variabili, modificare cose".

"Esistono due ambienti particolari": nel primo, l'AnimGraph, "si ragiona per quanto riguarda le animazioni"; nel secondo, l'Event Graph, "a livello di programmazione".

I due dati che servono. Nell'Event Graph si devono ricavare due cose dal personaggio:

1. la **velocità**, "perché vogliamo che la nostra animazione di camminata e di corsa, o di idle, sia relativa alla velocità che abbiamo al momento col character". "C'è una comunicazione tra i dati di gioco, quindi il movimento, e le animazioni": "dato un certo valore di velocità, cambia l'animazione";
2. se il personaggio **sta cadendo** o no, "in modo da dire: guarda, sto saltando, sto iniziando a cadere; fammi un'animazione di salto, e fammi il loop del *falling*".

Le animazioni di salto e caduta "non le abbiamo importate": "le andremo a ritargettare in seguito", perché "Unreal ci mette a disposizione la possibilità di ritargettare delle animazioni", "ma non lo vedremo adesso".

Event Blueprint Initialize Animation. Il primo nodo è **Event Blueprint Initialize Animation**: "dice: quando l'animazione inizia". "Come abbiamo visto, gli eventi come il Begin Play vengono fatti quando viene inizializzato tutto": questo "è il Begin Play, ma per le animazioni". (A questo punto Unreal si blocca di nuovo: "è crashato Unreal; direi che oggi è proprio una gran roba". "Mi sa che mi tocca svuotare un po' di memoria." Liberata la memoria, si riparte.)

Try Get Pawn Owner. Dall'evento si chiama **Try Get Pawn Owner**. "Questo blocco ci permette di comunicare con il nostro Pawn", "il nostro player". "Immaginatevi due ambienti separati": nel Gameplay Framework "il Pawn è quello che controlla il character", mentre l'animazione "è subordinata al Pawn". "Quindi andiamo a dire: ehi, provami a prendere l'owner, cioè il pawn owner." Poi "andiamo a fare un cast: abbiamo detto BP_Knight... Ah, giusto, ragazzi. Abbiamo scordato una cosa. Dobbiamo creare effettivamente il nostro character."

1.8.21 Il nuovo Character: BP_Knight1

"Dobbiamo comunicare con il nostro character, per dire: guarda, mi servono questi dati del movimento." Nella cartella Blueprint del personaggio: **Blueprint Class**, e "attenzione": come genitore si sceglie **Character**. Il nome: **BP_Knight1**.

I componenti del Character. "I character, a differenza degli actor, sono costituiti di base da":

- un **Capsule Component**, la capsula;
- una **freccia** (*Arrow*), "per far capire qual è il forward vector della capsula";
- la **Mesh**, una Skeletal Mesh;
- il **Character Movement**, "il componente che definisce il movimento del character". Nei suoi Details, in *General Settings*, ci sono la **Gravity Scale**, la **Max Acceleration**, la **Braking Friction**. "Come abbiamo detto, dobbiamo interfacciare le animazioni con i valori del movimento", e quei valori stanno qui.

La Skeletal Mesh. Nella Mesh si assegna la Skeletal Mesh del cavaliere ("devo ricordarmi l'indirizzo"; "no, non è questo, è questo qua"). Il personaggio "è apparso in modo abbastanza scorretto", fuori posto rispetto alla capsula e girato male: "basta riposizionarlo". "Consiglio di fare lo **snapping**": si abbassa la mesh in modo che i piedi tocchino il fondo della capsula, e la si ruota "in direzione della capsule", cioè con la faccia nella direzione della freccia (per i personaggi di Mixamo serve di solito una rotazione di -90 gradi sulla Z; il valore esatto la trascrizione non lo dice). "Eccovi che abbiamo creato il nostro primo character. Questa è la T-pose, meraviglioso."

L'Anim Class. "Ma in questo caso non ha ancora animazioni." "Come facciamo a dire: guarda, devi usare proprio questa Animation Blueprint?" Nella **Mesh**, alla voce **Anim Class**, la tendina mostra tutti gli Animation Blueprint compatibili: si sceglie **ABP_Knight1** (il nome dell'asset del docente). "Non è cambiato niente, ma l'abbiamo messo in comunicazione": quando l'Animation Blueprint avrà delle animazioni, "questo qua si aggiornerà". La relazione è a due vie: il Character dice quale Animation Blueprint usare (Anim Class), e l'Animation Blueprint chiede al suo proprietario i dati (Try Get Pawn Owner).

1.8.22 Initialize: il cast e le referenze

Tornati nell'Animation Blueprint, il grafo dell'inizializzazione si completa (figura 1.8.9, in alto):

1. **Event Blueprint Initialize Animation**;
2. **Try Get Pawn Owner → Cast To BP_Knight1** ("castate al vostro character, come l'avete chiamato");
3. dal pin As BP Knight1, **Promote to Variable**: una variabile referenza **BP_Knight1**. "Andiamo a creare una referenza del character, in modo tale che sia più semplice": "ogni volta che dovremmo prendere una cosa del character, dovremmo, se no, fare sempre Cast To BP_Knight", ed "appesantisce castare sempre un qualcosa. Quindi lo facciamo all'inizio";
4. dalla referenza, **Get Character Movement**, "per essere anche un po' più esaustivi", e anche questo **Promote to Variable**: una variabile **MovementComponent**. "Abbiamo creato già due variabili che ci faranno comodo in seguito."

Approfondimento: lo stesso grafo del modello Third Person. Il grafo costruito dal docente ricalca quasi nodo per nodo l'Animation Blueprint del manichino di Unreal Engine 5 (*ABP_Manny*, nel modello Third Person): sull'Initialize, Try Get Pawn Owner, Cast To Character, Set *Character*, Get Character Movement, Set *MovementComponent*; sull'Update, controllo di validità, poi Set *Velocity*, Set *GroundSpeed* (Vector Length XY), Set *ShouldMove* e Set *IsFalling*. La differenza è il cast: il modello fa Cast To *Character*, la classe generica, così che lo stesso Animation Blueprint funzioni con qualsiasi personaggio basato su Character; la lezione fa Cast To BP_Knight1, che dà accesso anche alle variabili aggiunte nel cavaliere (utile, per esempio, per uno sprint), ma lega l'ABP a quella classe.

1.8.23 Update: la velocità e la caduta

Event Blueprint Update Animation. "Questo evento avviene quasi come il Tick, a ogni frame." "Quando un'animazione si aggiorna, qualsiasi micro movimento è un Update": "se abbiamo un idle, comunque è un'animazione; un walk, comunque è un'animazione; e il run, comunque è un'animazione", "quindi sarà sempre attivo". "È per questo che non facciamo il cast qui": "se no farebbe ogni volta, ogni frame, sempre molto pesante".

Il Validated Get. "Dobbiamo vedere se questo character è valido." "Vi ricordate gli Is Valid? Se in una variabile c'è qualcosa dentro o non c'è." C'è "una cosa molto interessante": si trascina nel grafo un **Get** della variabile BP_Knight1, tasto destro, **Convert to Validated Get**. Il Get "normale" prende la variabile e basta; il Validated Get "prende la variabile, e mette anche la particolarità: se lo prendo ed è valido faccio questo, se non è valido faccio quest'altro". Ha un ingresso di esecuzione e due uscite, **Is Valid** e **Is Not Valid**: un Get e un Is Valid in un nodo solo. Sull'uscita Is Valid il docente mette una **Sequence** ("un Is Valid, Sequence") per eseguire in ordine le operazioni che seguono.

GroundSpeed. "Fondamentalmente andiamo a prendere il Movement Component", la variabile creata nell'Initialize, e da lì **Get Velocity**: "la velocità attuale, a ogni frame, del character movement". È un vettore; "ci interessa solamente il movimento" sul piano, "non ci interessa se salta o meno", "quello ci interesserà in altro modo". Quindi il nodo **Vector Length XY**: "la lunghezza del vettore solamente sugli

assi X e Y", "se l'asse X è avanti e dietro e l'asse Y è destra e sinistra". Il risultato è un **float**, e diventa, con **Promote to Variable**, **GroundSpeed**: "una variabile che rappresenta la nostra velocità attuale".

IsFalling. "Molto importante per sapere quando saltiamo o quando stiamo cadendo": dal Movement Component, il nodo **Is Falling**. "Ce ne sono due": uno è quello del *Nav Movement*; quello giusto "deve essere blu all'inizio e rosso alla fine", cioè con l'ingresso target blu (il Character Movement) e l'uscita rossa (una booleana). Il valore va in una variabile, **IsFalling**, aggiornata a ogni frame, "in modo tale da sapere che movimento fargli fare" nel "giro di animazioni".

Velocity. "Per il futuro, per altre cose che ci serviranno", anche il vettore della velocità si promuove a variabile: **Velocity**, "così almeno ci sono delle cose che vogliamo interfacciare con la velocità". "Adesso abbiamo tirato fuori la nostra velocità attuale e il fatto se sta cadendo o meno." A uno studente il Get Velocity non compare: è quello che si ottiene dal Movement Component, con il target giallo e l'uscita gialla (un vettore); cercandolo in fondo alla lista, "trovato".

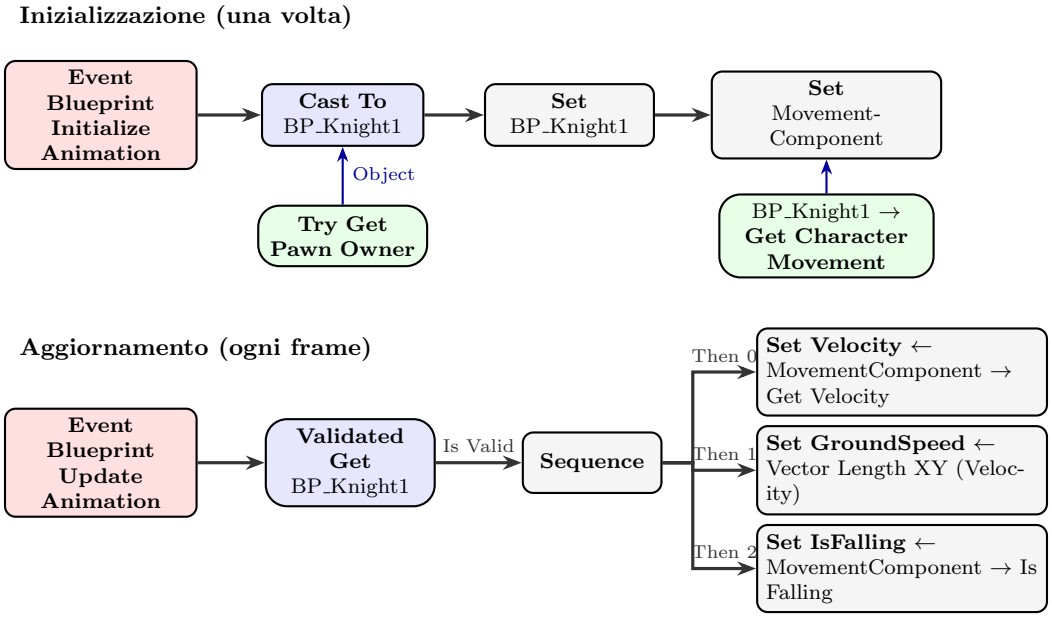


Figure 1.8.9: I due grafi dell'Event Graph di ABP_Knight. In alto, una sola volta: si prende il proprietario, lo si converte nel cavaliere e si salvano due referenze. In basso, a ogni frame: se la referenza è valida, si aggiornano Velocity (vettore), GroundSpeed (la sua lunghezza sul piano XY) e IsFalling (booleana). Accanto a ogni Set, il valore che riceve. Schema ricostruito.

Un registro di dati. "Questo graph funziona letteralmente come" un registro (la trascrizione dice "registri di credito"): serve "principalmente per interfacciarsi con il nostro bellissimo character", e "ci serve per capire valori, vedere cose", "parliamo di numeri per lo più". Le animazioni vere si decidono altrove.

Approfondimento: perché l'AnimGraph non legge da solo. Romero e Sewell (capitolo 17) descrivono la stessa divisione dei compiti: "usiamo l'EventGraph di un Animation Blueprint per prendere i dati dal Pawn o dal Character che sta usando l'istanza dell'Animation Blueprint, e per aggiornare le variabili dell'Animation Blueprint"; e "l'AnimGraph può accedere solo alle variabili dell'Animation Blueprint". Il loro esempio è più semplice di quello della lezione: sull'Update un Is Valid sul Try Get Pawn Owner, Get Velocity dal Pawn, *VectorLength* (la lunghezza su tutti e tre gli assi) e Set di una variabile *Speed*. La scelta del docente, **Vector Length XY**, è più precisa per la locomozione: con la lunghezza completa, un personaggio che cade dritto verso il basso avrebbe una velocità alta e farebbe la corsa a mezz'aria. Gli autori

notano anche che *Event Blueprint Update Animation* ha un parametro *Delta Time X*, il tempo trascorso dall'ultimo frame, utile per interpolare i valori.

1.8.24 L'AnimGraph, un'anteprima del Blend Space e il personaggio finito

"Adesso, ragazzi, vi faccio una spiegazione teorica molto corta, perché capisco che siamo agli ultimi dieci minuti, anzi sette." L'AnimGraph, "differentemente, lavora per esempio per *state machine*".

Un'animazione sull'Output Pose. Il caso più semplice: nell'**Asset Browser** dell'editor ("qua trovate tutte le animazioni che sono compatibili con questo scheletro") si prende un'animazione, per esempio *Walking 1*, la si trascina nell'AnimGraph e si collega "l'omino" del nodo all'omino dell'**Output Pose**. Non è in loop, ma basta spuntare **Loop Animation** nei Details del nodo. Compile: nella preview il cavaliere cammina. "E possiamo vederlo anche nel nostro character", perché l'Anim Class "ci ha permesso di dirgli: guarda, ehi, hai questo in eredità, hai questo sotto di te, l'Animation Blueprint". "Ma, come potete capire, a noi serve molto di più rispetto a mettere queste animazioni" una alla volta.

Il Blend Space 1D. "Sicuramente la prossima volta andremo a vedere quelli che sono chiamati **Blend Space**. Ve lo faccio vedere velocissimamente." Il docente apre un **Blend Space 1D** ("una dimensione"), creato sullo scheletro del cavaliere: un grafico con il personaggio che "ci permette di mettere insieme varie animazioni che *blendano* tra di loro", cioè "che interpolano tra di loro". Nella preview, spostandosi sull'asse: idle, poi walking, poi sprint. "In questo caso ci sarà solamente un asse, che noi diremo che sarà la nostra **GroundSpeed**", la velocità "che definisce il movimento" (figura 1.8.10). "La prossima volta vedremo di costruire questa tipologia di asset e collegare tutto quanto."

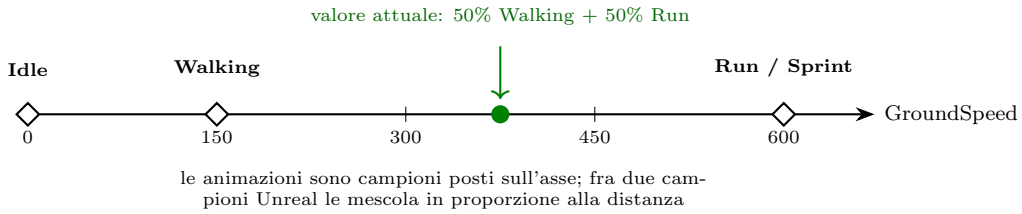


Figure 1.8.10: Un Blend Space 1D come quello mostrato in anteprima: un solo asse, la GroundSpeed calcolata nell'Event Graph, e tre animazioni poste su di esso. I valori sull'asse sono un esempio, non quelli del docente. Schema ricostruito.

Il personaggio finito. Per chiudere, il docente mostra dove arriveranno. "Io posso cambiare il nostro character da qui": nei **World Settings** del livello, nella Game Mode, alla voce **Default Pawn Class**, sceglie BP_Knight. In Play, il cavaliere si muove con idle, camminata e corsa, e "gli ho messo addirittura pure lo sprint". "Il salto, questo è ritargettato": è l'animazione classica di Unreal, portata sullo scheletro di Mixamo. E la vecchia logica funziona con il nuovo personaggio: "possiamo fare tutte le interazioni, e addirittura la nostra torretta ci segue". "Questo lo faremo la prossima volta", "perché ci vuole un pochino di continuità".

Approfondimento: dalle animazioni singole alla State Machine. Il capitolo 17 di Romero e Sewell mostra i passi successivi, quelli annunciati per la prossima lezione. Un **Blend Space** è "un tipo di asset che permette di mescolare animazioni in base ai valori di uno o due parametri": nel loro esempio del modello Third Person (UE4), il Blend Space di locomozione pone *Idle* a 0, *Walk* a 93,75 e *Run* a 375, e con una velocità di circa 234 l'animazione risultante è metà camminata e metà corsa. Trascinato nell'AnimGraph, il Blend Space diventa un nodo *Blendspace Player* con un ingresso per il parametro, a cui si collega la variabile della velocità; nell'*Anim Preview Editor* si può cambiare il valore e vedere il risultato senza andare in Play.

Poi la **State Machine**: un nodo dell'AnimGraph che organizza le animazioni in *stati* (per esempio *Idle* e *Moving*, e più avanti *Jump* e *Fall*), collegati da *transizioni* con regole booleane (per esempio "GroundSpeed > 3" per passare da fermo a in movimento, o "IsFalling" per passare al salto). È lì che la variabile IsFalling di oggi troverà il suo uso.

1.8.25 I materiali mancanti e la chiusura

"Ci sono domande, ragazzi?" Una studentessa: "il mio personaggio non c'ha un materiale, ha solo le texture". "Anche il mio non è importato", aggiunge un altro.

Creare il materiale a mano. La studentessa condivide lo schermo: il suo personaggio di Mixamo è un'arciera (*Erika Archer*, a giudicare dalla trascrizione). "Se non hai un materiale, dovresti crearlo da zero":

1. tasto destro → **Material**, chiamato **M_Archer**;
2. aperto il materiale in una finestra piccola, si trascinano dentro tutte le **texture** ("quelle con il label rosso"), che diventano nodi **Texture Sample**;
3. si legge bene il nome di ognuna: la **Diffuse** va nel **Base Color** ("acciaccala in Base Color"); la **Normal** va nel **Normal**.

Leggendo i nomi, però, si scopre che le texture sono di due gruppi: una Diffuse dei *vestiti* e una della *pelle* (e una delle ciglia, l'*eyelash*). "Ha due materiali diversi, forse." "Quindi mi sa che devi fare due materiali: uno per i vestiti, e l'altro per la pelle", per esempio *M_ArcherClothing*.

Gli slot dei materiali. Per sapere dove va cosa, si apre la Skeletal Mesh: nei Details, la sezione dei materiali mostra gli slot, con il **WorldGridMaterial** (la griglia grigia di default) dove manca un materiale. Gli slot sono *quattro*, non due. Si mette il nuovo materiale nell'**Element 0** e si guarda "dove te lo mette sul corpo": passando sopra uno slot, la parte corrispondente si illumina (*highlight*). Element 0 è il *clothing*; il 2 e il 3 saranno la pelle, con un altro materiale. "Tanto, se non avete i materiali ma avete le texture, ragazzi, potete farlo direttamente a mano."

Le texture che non ci sono. A un altro studente non sono state importate nemmeno le texture. Il docente prova a riprodurre il problema: torna su Mixamo ("Mixamo non mi risponde più, ottimo"), scarica lo stesso personaggio, *The Boss* ("use this character"; "madonna mia"), in T-pose, e lo importa. Nella finestra di import, nella sezione dei materiali, **Import Materials** e **Import Textures** devono essere attivi ("fai sì che i materiali siano importati"); *Import Morph Targets* e simili "non ci interessano". "A me importa tutto." "Se c'hai difficoltà, magari puoi semplicemente riprovare a fare l'import." Intanto Federico ha risolto da solo, assegnando i materiali agli slot con l'highlight: "allora vuol dire che ti sei risolto la vita da solo. Perfetto."

Approfondimento: Diffuse, Normal e gli altri canali. Una **Diffuse** (o *Albedo*, o *Base Color*) è il colore della superficie senza luci né ombre, e va nel pin Base Color; una **Normal map**, riconoscibile dal colore azzurro-violetto, simula i rilievi e va nel pin Normal (Unreal la riconosce all'import e imposta la compressione *Normalmap*). Se ci sono texture di *Specular*, *Roughness* o *Gloss*, vanno nei pin corrispondenti, ricordando che una Gloss è l'inverso di una Roughness (serve un nodo *1-x*, *OneMinus*). Per le ciglia e i capelli, fatti di piani con trasparenze, il materiale va impostato con *Blend Mode Masked* e l'alfa della texture collegata all'*Opacity Mask*: altrimenti le ciglia appaiono come rettangoli pieni. Sono gli stessi nodi visti nella terza lezione, dedicata a texture e materiali.

La chiusura. "Altri problemi? Se no, ci sentiamo la prossima volta." "Se dovete dichiarare qualcosa, non alla dogana, spero, ci sentiamo su Discord": "mandatemi quello che volete, se provate cose". A Filippo: "poi magari ti guardo un po' quelle cosette che abbiamo visto prima". "Allora, ragazzi, buona cena, buona serata. Ci vediamo giovedì. Ciao, ciao."

Sintomo	Causa e rimedio
Una variabile “non si muove”	creata di nuovo e rimasta al default (0): controllare i valori; variabili con nomi chiari
Errore sull’Audio Component “non valido” (Set Pitch Multiplier), solo arrivando in fondo	caso rimasto aperto; ipotesi: lo Stop distrugge il suono (Auto Destroy) e la variabile resta a puntare al vuoto. Is Valid, o Auto Destroy disattivato, o reset del Do Once
L’ascensore va sempre più giù e non risale	attivazioni ripetute: booleana Activated (Branch sul False all’inizio di Activation, false sul Begin Overlap)
Il progetto non si apre (si apre solo Unreal)	probabile chiavetta USB staccata male: rimuoverla in sicurezza; usare un sistema di versione (Git)
Il personaggio si ferma un istante sul collezionabile	doppia collisione: mesh in NoCollision, resta la Sphere Collision
On Component Begin Overlap non compare	è selezionata la mesh e non la Collision
Get Collectables non compare da Get Game Instance	manca il Cast To GI_Platform: la referenza è generica
Il cast non funziona	Get Game Instance non collegato all’Object del cast
“Non eredita da istanza di gioco”; la classe non compare nei Project Settings	la Game Instance è stata creata con un altro genitore: ricrearla da GameInstance, sceglierla in Game Instance Class, rifare variabile e cast
Il collezionabile conta ma l’esecuzione si ferma	il filo di esecuzione non passa per il cast
Open Level non carica il livello	nome scritto male: è case sensitive. Copiarlo con F2, Ctrl+C, Ctrl+V
Le palline raccolte riappaiono	normale: il livello viene ricaricato. Salvare lo stato nella Game Instance (esercizio futuro)
Mettere a terra un componente nel Blueprint	nessuna scorciatoia (End vale solo nel livello): regolare la Z a mano
Errore su Set Play Rate all’uscita dal Play	una vecchia referenza alla timeline con lo stesso nome: eliminarla e usare quella giusta
Unreal si chiude durante l’import	memoria video esaurita: liberare memoria e spazio su disco, reimportare
Il personaggio importato è ruotato	Offset Rotation non a zero: controllare Offset Translation e Rotation all’import
Import grigio per un’animazione	manca lo Skeleton: sceglierlo in Common → Skeleton
L’animazione sposta il personaggio da sola	è in root motion: riscaricarla da Mixamo con In Place
Get Velocity non compare	cercarlo dal Movement Component (target e uscita gialli)
Il personaggio di Mixamo è grigio, senza materiale	creare i materiali a mano (Diffuse in Base Color, Normal in Normal) e assegnarli agli slot; o reimportare con Import Materials e Import Textures

Table 1.8.4: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, asset o impostazione	Che cosa fa
Game Instance (Blueprint Class)	l'oggetto che vive per tutto il gioco; contiene dati globali
Project Settings → Maps & Modes → Game Instance Class	sceglie la Game Instance del progetto
Get Game Instance	restituisce la Game Instance, come referenza generica
Cast To (classe)	converte una referenza nel tipo specifico; Cast Failed se non lo è
Increment Int (++) / Decrement Int (−−)	aggiunge o toglie 1 a una variabile intera
Collision Presets: NoCollision	toglie ogni collisione a un componente
Sphere Collision, Sphere Radius	volume sferico e il suo raggio
Destroy Actor	distrugge l'actor
Evento Key (E)	scatta alla pressione di un tasto (Pressed, Released)
Text Render	un testo nel mondo 3D
Hidden in Game	nasconde un componente in gioco; tolto, la box si vede
Open Level (by Name)	carica un livello dal nome (case sensitive)
File → New Level (Basic)	un livello con cielo, luce e pavimento
Skeletal Mesh / Skeleton / Physics Asset	la mesh con le ossa; la gerarchia delle ossa; i corpi fisici
Offset Translation / Rotation (import)	posizione e rotazione dell'asset rispetto al pivot
Animation Sequence: Loop, Rate Scale, Enable Root Motion	ripetizione, velocità, movimento dentro l'animazione
Animation Blueprint (ABP)	la logica delle animazioni di uno scheletro: Event Graph e AnimGraph
Anim Class (Mesh del Character)	l'Animation Blueprint usato dalla mesh
Character (Capsule, Arrow, Mesh, Character Movement)	il Pawn che cammina, salta, nuota, vola
Event Blueprint Initialize Animation	il "Begin Play" dell'Animation Blueprint
Try Get Pawn Owner	il Pawn che usa l'Animation Blueprint
Event Blueprint Update Animation	scatta a ogni aggiornamento dell'animazione, come il Tick
Convert to Validated Get	un Get con le uscite Is Valid e Is Not Valid
Sequence	esegue più uscite in ordine
Get Velocity / Vector Length XY	la velocità; la sua lunghezza sul piano
Is Falling (Character Movement)	vero se il personaggio è in aria
Output Pose (AnimGraph)	la posa finale applicata alla mesh
Blend Space 1D	mescola animazioni lungo un asse (GroundSpeed)
World Settings → Default Pawn Class	il personaggio del giocatore in quel livello

Table 1.8.5: I nodi, gli asset e le impostazioni dell'ottava lezione.

Punti chiave del capitolo

- Il **Gameplay Framework** è la gerarchia delle entità che Unreal crea mentre il gioco gira: **Game Instance** (per tutta la durata del gioco), livello, **Game Mode** (le regole), **Game State** e **Player State** (lo stato, condiviso in rete), **Player Controller** (il "cervello" non fisico), **Pawn** e **Character** (il "corpo"), e infine l'input.
- **Pubblica non vuol dire globale**: le variabili degli actor muoiono con il livello. Per conservare dati fra i livelli si usa la **Game Instance**, creata come Blueprint Class da GameInstance e scelta in **Project Settings → Maps & Modes → Game Instance Class**. Non va appesantita, perché resta in memoria per tutta la partita.
- **Get Game Instance** restituisce una referenza generica, "la scocca": per leggere le nostre variabili serve il **Cast To** della nostra classe, "l'abito giusto" per entrare alla festa. Cast Failed non è un errore, è una risposta.
- Un **collezionabile**: Sphere Collision, mesh in **NoCollision** (niente doppia collisione), controllo sul player, **Increment Int** sulla variabile della Game Instance, **Destroy Actor**.
- Un volume che cambia livello: **Open Level (by Name)** con il nome promosso a variabile pubblica. I nomi sono **case sensitive**: copiarli con F2 e Ctrl+C. I livelli nuovi da **File → New Level → Basic**, salvati subito con Ctrl+S.
- Una booleana **Activated** con un Branch fa da cancello: l'ascensore ignora le attivazioni ripetute finché il player non lo fa risalire.
- Una **Skeletal Mesh** è "il corpo e le ossa"; lo **Skeleton** è un asset separato, a cui sono legate le animazioni. Le ossa si **ruotano**, non si spostano.
- Da **Mixamo**: personaggio in **T-pose**, **FBX Binary**; animazioni idle, walk e run **In Place** (non root motion: a muoversi è la capsula), **Without Skin**, **30 fps** per tutte, nessuna riduzione dei keyframe. All'import si controllano gli **Offset**; le animazioni si importano scegliendo lo **Skeleton**.
- L'**Animation Blueprint** ha due grafi: l'**Event Graph** raccoglie i dati dal personaggio, l'**AnimGraph** decide la posa (**Output Pose**). Il Character lo usa tramite l'**Anim Class** della Mesh.
- Nell'**Initialize**: Try Get Pawn Owner, Cast To del personaggio, e le referenze salvate in variabili (il cast una volta sola). Nell'**Update**, che gira a ogni frame: **Validated Get**, poi **GroundSpeed** (Vector Length XY della Get Velocity), **IsFalling** e **Velocity**.
- Prossimi passi: il **Blend Space 1D** sulla GroundSpeed, la State Machine, il **retargeting** del salto, e il personaggio impostato come **Default Pawn Class**.

Chapter 1.9

State Machine, Blend Space e Montage: dare vita al nuovo personaggio

Questa è la **nona lezione** del corso breve di **introduzione a Unreal Engine**, e riprende esattamente dove si era fermata l'ottava. Alla fine della lezione precedente il cavaliere importato da **Mixamo** aveva uno scheletro, le sue animazioni e un **Animation Blueprint**, **ABP_Knight**, che a ogni frame leggeva dal personaggio due dati: la velocità sul piano (**GroundSpeed**) e il fatto di essere in aria o meno (**IsFalling**). Mancava tutto il resto: chi decide *quale* animazione mostrare, e quando. "Siccome ci siamo interrotti l'altra volta, andiamo diretti a fare le animazioni: l'altra volta ci siamo interrotti al Blend Space."

La lezione costruisce, un pezzo alla volta, un personaggio giocabile completo. La **State Machine** organizza le animazioni in *stati* (camminare e correre, saltare, cadere) e in *transizioni* con delle condizioni; il **Blend Space 1D** mescola fermo, camminata e corsa in base alla velocità; una **camera** con il suo **Spring Arm**, i grafi di movimento presi in prestito dal modello Third Person e un nuovo input per lo **sprint**, costruito con l'**Enhanced Input**, rendono il cavaliere controllabile; il **retargeting** porta sul suo scheletro le animazioni di salto e di caduta del manichino di Unreal. Dopo la pausa arrivano il **Blend Space 2D**, che aggiunge alla velocità la **direzione** e permette di camminare di lato e all'indietro, e gli **Animation Montage**, le animazioni "una tantum" (un attacco con la spada) che si inseriscono sopra il ciclo di locomozione.

La lezione è **online**, su Zoom, come le precedenti, e la classe è ridottissima: "vediamo un po' quanti siete: siamo in sei"; dopo la pausa "siamo rimasti in cinque", poi "uno, due, tre e quattro"; quando uno studente crasha ("Filippo è andato via"), "siamo rimasti noi tre"; alla fine, "mi sembra che siamo rimasti in due". "Pochi ma buoni." Il ritmo è quello della dimostrazione dal vivo, interrotta più volte dai problemi degli studenti, risolti in condivisione di schermo, e da un editor che "oggi lagga" in modo vistoso ("non capisco perché mi sta laggando oggi"). La pausa arriva dopo la scelta delle animazioni da scaricare ("ci rivediamo alle 18"); si chiude con un compito facoltativo e un "ci sentiamo per martedì prossimo".

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Arreal", "a Real", "ArrayL", "Array Alt" e "a Rial"; il **Pawn Owner** è "Pwn Honor" e "Pwnowner"; il **Blend Space** è "Blaine Space", "Blendspace" e "BlizzBase"; lo stato **WalkRun** è "walk run", "Warcran", "war crunch" e "walker run"; l'**Idle** è "Idol" e "idol"; **BP_Knight** è "Vip Night", "Pippi Night 1" e "BP Night"; **BS_Knight** è "BS Night" ("che dovrebbe essere un po' più British"); il manichino **Manny/Quinn** è "Many Quincy"; il **root motion** è "rut" e "outmotion"; lo **Strafe** è "Strife" e "stair right"; la **Mesh** è "Mesci"; l'**FInterp To** è "finter2"; **Calculate Direction** è "calcolo interaction"; il **Save Cached Pose** è "save cache, pause"; lo **StackOBot** è "StaccoBot" e "stacco bot"; lo **Yaw** è "io", "yo" e "YAW"; l'**Orient Rotation to Movement** è "orientation to movement", "anche rotacion" e "orient rotace tu mente tu"; lo **Use Pawn Control Rotation** è "us control pawn rotation" e "you spawn control rotation". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione e durante la pausa il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("grazie a tutti per l'attenzione

che ci ha spiegato", "qual è la risposta che ci può essere?", "la situazione è questa, la migliore dal 2005", la "dottrina sul mercato del lavoro", la "Corte Suprema di Internet", il governo di Dubai, una lunga fila di "di Sarajevo", la "struttura del giardino", "la blue map", un Vittorio Prodi chiamato in causa senza motivo): sono artefatti, non parole di nessuno, e il capitolo li ignora. La terza: gran parte della lezione è una dimostrazione dal vivo nell'editor, costruita a voce ("tirate fuori", "promote to variable", "ce lo attacchiamo qui"), e in più punti il docente improvvisa, prova, sbaglia e corregge, soprattutto nella parte sulla camera e in quella finale sul Montage. Il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, distingue i tentativi falliti dalla soluzione finale, e completa i nomi esatti dei nodi e dei pin di Unreal Engine 5; dove la trascrizione lascia un buco, lo dichiara.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono la State Machine della locomozione, il Blend Space 1D sulla velocità, la camera con lo Spring Arm, la catena dell'Enhanced Input, il grafo dello sprint, il cerchio della direzione, il Blend Space 2D con le sue otto animazioni, la catena di pose dell'AnimGraph con lo Slot del Montage e il grafo del tasto che fa partire l'attacco. Le tabelle riassumono le transizioni della State Machine, i parametri della camera, i valori e i trigger di una Input Action, la posizione delle animazioni nel Blend Space 2D, gli errori incontrati durante la lezione e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'**inizio della lezione**: i corsi degli studenti e la lezione libera del 25 (sezione 1.9.1);
2. il **ripasso** dell'Animation Blueprint (sezione 1.9.2);
3. la **State Machine** della locomozione (sezione 1.9.3);
4. il **Blend Space 1D**, con una parentesi sui problemi dell'editor (sezioni 1.9.4 e 1.9.5);
5. il personaggio in gioco: **Default Pawn Class**, **camera** e **Spring Arm**, grafi di movimento e rotazioni (sezioni da 1.9.6 a 1.9.9);
6. il **retargeting** del salto e della caduta, e le **regole di transizione** (sezioni 1.9.10 e 1.9.11);
7. lo **sprint** con l'**Enhanced Input** e il tentativo con FInterp To (sezioni da 1.9.12 a 1.9.13);
8. le **animazioni da scaricare**, la pausa e i **feedback** (sezioni 1.9.14 e 1.9.15);
9. il **Blend Space 2D** e la **direzione** (sezioni 1.9.16 e 1.9.17);
10. gli **Animation Montage**: l'asset, i Notify, le sezioni, gli Slot e il tasto che li fa partire (sezioni da 1.9.18 a 1.9.21);
11. il **compito** sullo StackOBot e la chiusura (sezione 1.9.22).

1.9.1 Prima della lezione: i corsi degli studenti e la lezione del 25

La registrazione si apre con qualche minuto di attesa, in cui il riconoscimento vocale produce soprattutto rumore. Poi i saluti ("ragazzi, come state? Buonasera, buonasera"), un controllo rapido ("nessun problema sulla telecamera": stavolta la webcam del docente funziona) e la decisione di aspettare "altri cinque minuti, così vediamo che tutti siate in carreggiata". Uno studente, Tommaso, ha dei problemi di collegamento. Il docente sistema le finestre ("oggi c'è il secondo schermo") e parte: "bando alle ciance".

I corsi di indirizzo. Prima di cominciare, il docente chiede ai presenti quale corso seguiranno "quando inizierete a settembre": il corso di Unreal è infatti un'introduzione comune, a cui seguono gli indirizzi specialistici. Le risposte dei sei presenti:

- **3D art**;
- **game design** (due studenti);
- **narrative design**;
- **character animation** ("perfetto, character animation").

La domanda serve a tarare la lezione: chi farà character animation, in particolare, è il destinatario naturale di quasi tutto quello che si vedrà oggi.

La lezione libera del 25. "Stiamo praticamente arrivando a una lezione che di solito noi lasciamo abbastanza *free*, come argomenti": la lezione del **25** sarà "fondamentalmente la mia ultima lezione di base". Se non ci sono richieste particolari, si faranno "i menu", cioè "la **UI**": "che si intende la UI: menu, bottoni, scritte eccetera". In alternativa, "se ci sono degli argomenti particolari", da concordare fra gli studenti, "possiamo tranquillamente affrontarli", oppure "dividere anche la lezione in due: fare un piccolo menu e poi fare anche la parte in più". "Nel caso, sentitevi tra di voi." La richiesta concreta arriverà durante la pausa (sezione 1.9.15).

1.9.2 Il ripasso: l'Animation Blueprint del cavaliere

"Eravamo rimasti al nostro bellissimo **ABP_Knight**." Il docente ha il cavaliere (*Knight*, il paladino di Mixamo); "altri di voi avevano scelto il proprio personaggio". Nella preview dell'Animation Blueprint il personaggio è ancora "il nostro bel character in T-pose": l'AnimGraph non decide ancora nessuna animazione.

Initialize e Update. L'ultima cosa fatta era l'evento **Event Blueprint Initialize Animation**: "quando l'animazione si inizializza", una volta sola. L'altro evento, **Event Blueprint Update Animation**, scatta invece ogni volta che "si cambia qualsiasi particolare all'interno di un'animazione, la rotazione di un braccio o un minimo particolare": "quello che è fondamentalmente il nostro Tick sulle animazioni". "Quando abbiamo visto Begin Play e Tick: questi praticamente sono i loro corrispettivi."

Il Pawn Owner e le due referenze. Nell'Initialize si prende il **Pawn Owner** con **Try Get Pawn Owner**. "Ricordiamoci: il Pawn Owner è l'entità che controlla il character": il docente distingue il *pawn*, "quello che lo controlla", e il *character* stesso, "che ha la mesh, lo scheletro e tutto quanto". Con il **Cast To** BP_Knight si ottiene una referenza al cavaliere, e si salvano in due variabili il cavaliere e il suo **Character Movement Component**: "queste due variabili ci serviranno per comunicare in modo più diretto con il nostro cavaliere".

Il Character Movement. "Dentro il Blueprint, qualsiasi character lo troverete qui sotto, Character Movement": selezionandolo, il pannello Details a destra "presenta tutte le particolarità di movimento", per esempio la **gravità** (*Gravity Scale*), la **Max Acceleration**, la **Max Step Height** (l'altezza massima di un gradino che il personaggio sale senza saltare). "Vogliamo farlo andare più veloce, meno veloce? Se volessimo farlo sprintare, dovremmo lavorare su questa particolarità"; e infatti "dopo faremo anche uno sprintino molto veloce, per differenziare le animazioni".

GroundSpeed e IsFalling. Nell'Update, dopo una **Sequence**, si prende "la velocity attuale del nostro character", a ogni frame, e se ne salva la lunghezza in **GroundSpeed**. Allo stesso modo, con il nodo **Is Falling** del Character Movement, "andiamo a vedere se effettivamente il nostro character sta cadendo o meno": "questo blocco ci dice letteralmente se sta cadendo, e noi riportiamo con un booleano, true o false". "Di conseguenza, tramite questi dati, andremo a differenziare le animazioni nella nostra State Machine."

1.9.3 La State Machine della locomozione

Creare la State Machine. La State Machine "la possiamo invocare nell'AnimGraph": tasto destro nell'AnimGraph ("ricordatevi che c'è il blocco **Output Pose**"), **State Machine**, e la si chiama **Locomotion**: "questo è il nome classico per la locomozione, appunto il movimento". "Una State Machine, appunto, è una macchina a stati." Con un doppio clic si entra nel suo grafo.

Entry, Conduit, State, Alias. Dentro c'è un solo nodo, **Entry**. "Se noi trasciniamo, abbiamo una freccia che esce fuori"; lasciandola nel vuoto, il menu non è "tutto l'elenco gigantesco" dei nodi dei Blueprint, ma tre sole voci: **Conduit**, **State** e **Alias**. "A noi adesso interessa vedere quelli che sono gli stati." Il primo stato, invece di *Locomotion*, si chiama **WalkRun**: "differenziamo".

Un'altra inception. "Ecco a voi che, ancora sempre, un po' un *inception*": con un doppio clic si entra anche nello stato. Dentro, un nodo **Output Animation Pose**: "in questo stato risiede una tipologia di output", l'animazione che lo stato produce. Il docente vi collega *Walking 1*. La preview però non si aggiorna, "perché non abbiamo collegato la State Machine all'output": bisogna tornare nell'AnimGraph e collegare il nodo Locomotion all'Output Pose. Fatto questo, il cavaliere cammina, ma una volta sola: "l'animazione non è in loop", e basta spuntare il loop nei Details del nodo. "Ecco a voi che cosa fa la State Machine": "dentro Locomotion lui entra, dice ok, inizio, passo e vado direttamente a WalkRun"; in WalkRun "c'è una richiesta di un output, quindi di un'animazione che noi gli diamo".

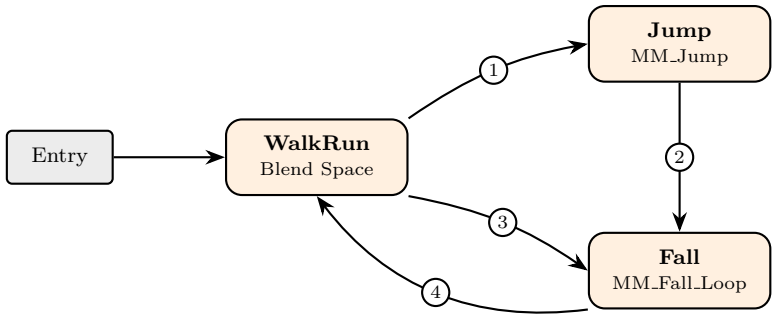
Gli stati Jump e Fall. "Ma la cosa interessante della State Machine" è che si possono creare altri stati: **Jump** e **Fall**. "A livello di logica, la State Machine vuole essere" il racconto di "come entrano e perché entrano" le animazioni nei loro stadi. Una freccia da WalkRun a Jump "vuol dire che dallo stadio WalkRun passerà allo stadio Jump"; una freccia inversa vuol dire che dal Jump "potrà passare di nuovo al WalkRun". Lo schema di oggi ha tre stati (figura 1.9.1): si entra in WalkRun; "se starà cadendo" (la variabile IsFalling diventa vera) si passa a Jump o a Fall; "e far sì che poi, quando lui finisca di cadere, ritorni" a WalkRun. "Quindi fondamentalmente creiamo un loop di animazione secondo delle condizioni." Le animazioni di salto e di caduta non ci sono ancora: "la faremo, la ritargetteremo" (sezione 1.9.10).

Le transizioni. "Come facciamo a mettere queste condizioni? Tramite questi nodini qua": sulle frecce fra due stati compare un'icona tonda, la **transition rule**, che "ci dice il comportamento effettivo che dovrà avere" il personaggio. "Quando è che passo da WalkRun a Jump, a livello di animazione? Quando, ad esempio, stai cadendo. Quando passo da Fall a WalkRun? Se non sta più cadendo. Questa è la logica generale."

Approfondimento: Conduit e Alias. I tre tipi di nodo proposti dal menu della State Machine hanno ruoli diversi. Uno **State** contiene un grafo che produce una posa (un'animazione, un Blend Space, una combinazione). Un **Conduit** non produce pose: è un incrocio, uno snodo da cui partono più transizioni, utile quando molti stati condividono la stessa condizione d'ingresso o d'uscita e non si vuole ripeterla su ogni freccia. Un **State Alias** è un "alias" di uno o più stati: le transizioni che partono dall'alias valgono come se partissero da ciascuno degli stati che rappresenta (per esempio, "da qualunque stato, se il personaggio muore, vai a Death"), e risparmiano un groviglio di frecce. Romero e Sewell (capitolo 17) costruiscono la loro prima State Machine, *Char States*, con due soli stati, *Idle* e *Moving*, e due regole sulla velocità ("Speed maggiore di 5" e "minore di 5"); la scelta della lezione, un solo stato WalkRun che contiene un Blend Space, sposta la distinzione fra fermo e in movimento dentro il Blend Space, e lascia alla State Machine le sole differenze "qualitative" (a terra, in salto, in caduta).

1.9.4 Il Blend Space 1D

"Visto che WalkRun non è solo un'animazione, è un insieme": sono tre animazioni, **Idle**, **Walk** e quella che il docente ha chiamato **Slow Run**. "Dobbiamo mixarle insieme. Come si fa? Perché non possiamo mettere



1 WalkRun → Jump: IsFalling **AND** VelocityZ > 0 (sta salendo)
2 Jump → Fall: *Automatic Rule Based on Sequence Player in State* (finito il salto, la caduta)
3 WalkRun → Fall: IsFalling **AND** VelocityZ < 0 (cade senza aver saltato)
4 Fall → WalkRun: **NOT** IsFalling (è tornato a terra)

Figure 1.9.1: La State Machine Locomotion alla fine della lezione: tre stati e quattro transizioni. Le condizioni con la velocità sulla Z sono quelle aggiunte nella seconda versione (sezione 1.9.11); nella prima, la 1 e la 3 erano soltanto IsFalling. Schema ricostruito.

un'animazione solo così: dobbiamo far sì che sia dipendente dalla nostra velocità." Lo strumento è il **Blend Space**.

Creare il Blend Space 1D. "Fate solamente il Blend Space 1D." Invece di cercarlo nei menu, si può scrivere nella barra di ricerca del menu del tasto destro "blend space", e scegliere **Blend Space 1D**: "già vi dice il percorso, che sta in Animation, Legacy". L'icona ha un colore "rosa pelle". Aperto, si troverebbe il personaggio in T-pose; ma molti l'avevano già creato la volta precedente ("sì, mi sa che già l'avevamo fatto").

Che cos'è un Blend Space. "Un Blend Space è l'azione di *blendare* qualche cosa", di "interpolare o comunque mixare le animazioni": un asset che "ci dà la possibilità di poter traslare da un'animazione all'altra". Si vede subito: dal pannello **Asset Browser** dell'editor ("se non lo trovate, ripeto: Window, Asset Browser") si trascina un'animazione sul grafico; tenendo premuto **Shift** la si aggancia "su dei punti precisi" della griglia. Con un'animazione sola, il personaggio fa solo quella; aggiungendone una seconda, il grafico "crea una linea di passaggio tra un'animazione e l'altra".

Il parametro, come l'alfa. "Ma importante: qual è questo dato?" Il docente lo collega all'**alfa** di un Lerp visto in una lezione precedente: "quel puntatore che punta, dipendente da uno spazio tra due punti, dove sta al momento". Qui il valore dirà "ehi, guarda che sto camminando, sto andando a velocità 300, oppure sto correndo, a velocità 600, oppure sono fermo". Il Blend Space **1D** "ci dà solamente la possibilità di poter usare un asse": nei Details, **Horizontal Axis** (la sezione potrebbe essere chiusa: "basta che la aprite"). L'asse si chiama **GroundSpeed**, "come quella che abbiamo trovato all'interno dell'Animation Blueprint"; poi chiede un **Minimum Axis Value** e un **Maximum Axis Value**.

Le velocità sono relative. "Importante, ragazzi, quando si va a vedere le animazioni: dovete sempre farlo relativo, se si parla di velocità", relativo cioè alla velocità reale del personaggio. Dove si legge? Nel Blueprint del personaggio, BP_Knight, selezionando **Character Movement**: "potete andare a vedere tutti i dati che vi servono". Quello che serve è la **Max Walk Speed**, in alto, nella sezione *Character Movement: Walking*: "è 600 centimetri al secondo". "Unreal ragiona per centimetri": il personaggio va a 6 metri al secondo.

Dividere la velocità a metà. "Visto che noi abbiamo due stadi, uno di walk e uno di run", si vuole che "la walk sia un movimento normale": di default il personaggio cammina, e corre solo se il giocatore lo chiede

con un tasto (sezione 1.9.12). "Quindi la dividiamo a metà": **Max Walk Speed a 300**. Nel Blend Space l'asse va da **0** a **600**, "in modo tale che a metà la nostra walk sia a 300" (figura 1.9.2). A uno studente l'asse arriva solo a 100: "ground speed, devi metterlo a 600, lo trovi qua". E un consiglio: dopo aver cambiato il massimo, "sposta i pallini, perché poi ti switcha tutto"; il valore di ciascun campione si può anche scrivere a mano nei Details.

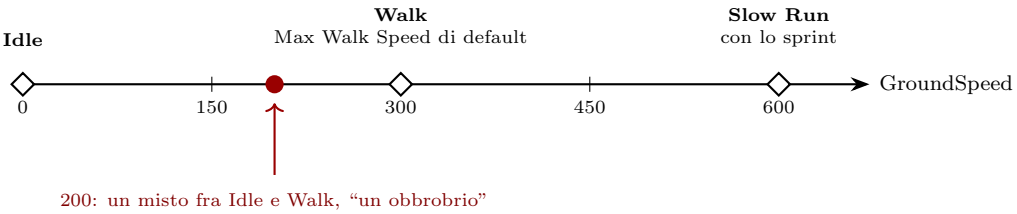


Figure 1.9.2: Il Blend Space 1D BS_Knight: un solo asse, la GroundSpeed, da 0 a 600, con Idle a 0, Walk a 300 (la Max Walk Speed impostata nel Character Movement) e la corsa a 600 (la velocità dello sprint). Un valore intermedio, come 200, mescola due animazioni nate per velocità diverse. Schema ricostruito.

Il Blendspace Player. Tornati in ABP_Knight ("normale, ragazzi, che vi dia dei warning, perché ancora non abbiamo le condizioni": "lui dice, ma io non ho condizioni, quindi faccio tutto in una botta"), si entra nello stato WalkRun. Nell'Asset Browser "è apparso BS_Knight"; trascinato nel grafo, "ci presenta un **Blend Space Player**", diverso dal nodo di un'animazione normale: ha un **valore in entrata**, la GroundSpeed. A quel pin si collega un **Get** della variabile GroundSpeed, calcolata nell'Event Graph (la lunghezza della velocity), e l'uscita del Blend Space Player si collega al **Result** dell'Output Animation Pose, "in modo tale che vada in output".

Get con Ctrl, Set con Alt. Un promemoria: trascinando una variabile nel grafo, "tenete premuto **Ctrl** e tirate fuori direttamente un Get"; "se invece volete fare un Set, tenete premuto **Alt**". Nel grafo dello stato l'Alt non funziona: "non si può settare all'interno" (gli stati dell'AnimGraph leggono le variabili, non le scrivono), "però nei Blueprint, di base, se premete Alt vi setta la variabile".

La prova nella preview. Compilato, "il nostro character nella preview adesso sta in idle, perché giustamente ha velocità 0". Cambiando il valore di GroundSpeed nella preview: a **300** cammina; a **200** "sta interpolando tra l'animazione di idle e l'animazione di walk": "mettere un valore a 200 non è buona cosa", perché "deve essere relativo al vostro movimento, se no vengono degli obbrobri"; a **600** "il nostro carissimo corre" ("guarda come è eroico"). "Quindi adesso andiamo a cambiare le animazioni tramite un valore."

Allineare i nodi con Q. "Se volete fare un po' di ordine": selezionati due o più nodi, il tasto **Q** li allinea (*Straighten Connections*). "Tutto bello ordinato."

Una prova con un solo stato. Prima di andare avanti, un test in gioco: nell'AnimGraph si tolgono le frecce verso Jump e Fall (selezionate e cancellate, o staccate con **Alt**+clic sul pin), e si lascia "solamente WalkRun, che vada in entrata".

Approfondimento: il Blend Space nel libro. Romero e Sewell (capitolo 17) definiscono il Blend Space come "un tipo di asset che permette di mescolare animazioni in base ai valori di uno o due parametri"; nel loro esempio, tratto dal modello Third Person, Idle è a 0, Walk a 93,75 e Run a 375, con una Max Walk Speed di 375. Il principio è lo stesso della lezione: i campioni vanno posti dove la velocità reale del personaggio corrisponde alla velocità "disegnata" nell'animazione. Se un'animazione di camminata è stata animata per una velocità di 150 centimetri al secondo e il personaggio si muove a 300, i piedi "scivolano" sul pavimento (*foot sliding*); è il motivo pratico dietro il "deve essere relativo" del docente. Gli autori ricordano anche che il valore del parametro si può cambiare nell'*Anim Preview Editor*, in basso a destra nell'editor dell'Animation Blueprint, senza andare in Play: è ciò che il docente fa per mostrare i valori 200, 300 e 600.

1.9.5 Una parentesi: "questo asset non ha schede ancorate"

Mentre si sistema la Max Walk Speed, uno studente interviene: "io ho un problema col Blueprint: quando apro la scheda Blueprint mi dice: questo asset non ha ancora schede ancorate". Il docente ha l'editor in inglese ("di solito gli errori me li ricordo in inglese") e chiede di vedere: lo studente condivide prima soltanto una scheda del browser, poi lo schermo intero.

Il messaggio. Aprendo BP_Knight, l'editor del Blueprint è vuoto, senza pannelli. Il testo completo del messaggio, letto ad alta voce, spiega che l'editor dell'asset è stato aperto in un sistema di finestre fluttuanti e nascoste, "non mostra le schede ancorate nell'interfaccia principale", e che "questo comportamento può essere facilmente ripristinato" ripristinando il layout dell'editor o modificando le preferenze perché gli asset si aprano sempre ancorati alla finestra principale (in inglese, *Editor Preferences* → *General* → *Appearance* → *Asset Editor Open Location*). Inoltre, sopra il grafo compare una nota: il Blueprint è *data-only*, e c'è un link **Open Full Blueprint Editor**.

La soluzione. "Mi sa che è meglio ripristinare il layout di default": dalla barra in alto, **Window** (in italiano *Finestra*) → **Load Layout** (*Carica il layout*) → **Default Editor Layout** (*Editor predefinito*). Poi, sulla nota, "apri l'editor Blueprint completo". "Perfetto, eccolo qua."

La lingua dell'editor. "Comunque, per mettere l'editor in inglese" senza riavviare: si cerca *Language* nelle preferenze. Il docente prova prima in **Editor Preferences** (sotto *Edit*), scrivendo "lingua" e poi "language" ("non gli piace che mettiamo due cose insieme"), poi nei **Project Settings**; alla fine la voce *Language* viene trovata con la ricerca ("ah, ecco perché"). Dalla trascrizione non si capisce con certezza in quale delle due finestre. Lo studente ringrazia ("mi hai fatto sentire molto ignorante"; "verrai buttato fuori dalla classe, tranquillo"), e il docente annuncia di volerlo usare in inglese anche lui "per la prossima volta".

Approfondimento: dov'è la lingua. In Unreal Engine 5 la lingua dell'interfaccia è in **Edit** → **Editor Preferences** → **General** → **Region & Language** → **Editor Language**; nei Project Settings c'è invece la lingua del *gioco* (la localizzazione), che è un'altra cosa. Lavorare con l'editor in inglese è una scelta pratica: la documentazione, i tutorial, i libri e i forum usano i nomi inglesi dei nodi e dei pannelli, e la ricerca dei nodi nel menu del tasto destro accetta i nomi inglesi anche quando l'interfaccia è tradotta.

1.9.6 Il cavaliere in gioco: la Default Pawn Class

"Adesso vi faccio vedere come impostiamo il player nuovo." Si aprono i **World Settings** del livello ("qualsiasi cosa che non trovate, c'è sempre Window, e troverete World Settings"); fra le voci della Game Mode c'è **Default Pawn Class**. "Ricordiamo che il Pawn è appunto l'entità che controlla il player." Nel menu a tendina c'è BP_Knight, il personaggio finito che il docente aveva mostrato alla fine della lezione precedente, e c'è BP_Knight1, "quello nuovo", che si può anche cercare scrivendone il nome. Si sceglie BP_Knight1 e si va in Play.

Il primo problema: nessuna camera. "Ecco già un problemino." "Fate attenzione, perché i character non vengono con la camera": un Blueprint creato dalla classe Character ha la capsula, la freccia, la mesh e il Character Movement, ma non una camera. In Play, la visuale finisce in un punto qualsiasi del corpo ("questo meraviglioso…", la frase si perde). Si esce dal Play e si apre BP_Knight1: "come potete vedere, non c'è una camera".

1.9.7 La camera: Camera e Spring Arm

"La camera è impostata su due cose particolari, ragazzi, molto importanti": la **camera** stessa e il suo **braccio**. "Immaginatelo come lo stand": il braccio dice "quanto è lontana" la camera, "quanto offset ha".

Aggiungere i due componenti. Si aggiunge un componente **Camera** ("camera normale"): "eccola qua, che si è già attaccata" al personaggio, ma in una posizione che non va bene ("al suo lato B"). Poi uno **Spring Arm** (il docente lo chiama prima "arm length", poi si corregge: "perdonate, l'arm length è un parametro; lo Spring Arm"). La gerarchia è importante: **lo Spring Arm deve avere la camera dentro di sé**, cioè la Camera va trascinata sopra lo Spring Arm e diventa sua figlia. Selezionando lo Spring Arm, nel viewport compare "un piccolo tracciato rosso": "la lunghezza del nostro braccio", il braccio della camera (figura 1.9.3).

I 3C. "Ricordatevi, ragazzi, molto importante, perché la camera è importantissima: ci sono delle figure che si chiamano **3C**, *Camera, Control e Character*, che lavorano solo su questo." Da qui una regola: "se volete allontanare la camera o cambiarle l'offset, **non cambiate la location della camera**. Non lo fate. Controllatelo sempre da qui, dallo Spring Arm."

Target Arm Length. Il parametro **Target Arm Length** "ci permette di aumentare o diminuire la distanza della camera". Valore scelto: **300**.

Target Offset e Socket Offset. Ci sono due offset, e il docente ha una preferenza netta. Il **Target Offset** "usatelo solamente sulla Z, di poco, veramente di poco, massimo 20"; "e fate tutto con il **Socket Offset**". Il motivo: "se voi cambiate l'offset del target, la camera vi può più facilmente compenetrare con le cose". Con il Socket Offset, invece, "vedrete che la camera va sul muro e torna": "va più vicino", cioè "riduce il suo arm length, fondamentalmente". Dopo la prima prova in Play ("lo testiamo, ragazzi?"), la camera è un po' alta: il Socket Offset sulla Z viene abbassato a **40**.

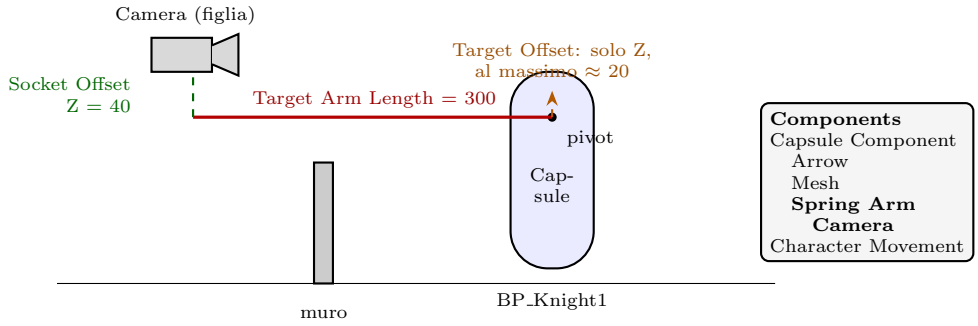


Figure 1.9.3: La camera del cavaliere. La Camera è figlia dello Spring Arm, e la distanza e l'altezza si regolano sul braccio, mai spostando la camera. Se fra il personaggio e la camera si mette un ostacolo (il muro), lo Spring Arm accorcia il braccio e la camera "torna" verso il personaggio; con un Target Offset grande, invece, è più facile che la camera entri negli oggetti. Schema ricostruito, valori della lezione.

Il Field of View. Più avanti, mentre aspetta uno studente, il docente mostra che "nella camera è presente anche un po' di dati con cui potete giocarci", per esempio il **Field of View**. Lo abbassa a **80**: "sappiate che tanti giochi horror usano il field of view a 80. Questo è un piccolo tip."

Approfondimento: perché lo Spring Arm si accorcia. Lo **Spring Arm Component** fa, a ogni frame, una traccia (un *sweep* di una piccola sfera, di raggio *Probe Size*) dal suo punto d'origine verso la posizione desiderata della camera; se incontra un ostacolo nel canale *Probe Channel* (di default *Camera*), accorcia il braccio fino al punto di impatto, e lo riallunga quando l'ostacolo sparisce (la proprietà è *Do Collision Test*). Il *Target Offset* sposta l'*origine* di quella traccia, nello spazio del mondo: spostarla molto può portare l'origine dentro un muro o dietro un oggetto, e allora il test non "vede" più l'ostacolo; il *Socket Offset* sposta invece la *fine* del braccio, nello spazio del braccio stesso, e la traccia resta coerente. È la ragione tecnica del consiglio del docente. Lo Spring Arm ha anche le opzioni *Enable Camera Lag* ed *Enable Camera Rotation Lag*, che fanno "inseguire" la camera con un leggero ritardo e rendono più morbidi i movimenti: il docente le sfiora più tardi, nella sezione del Montage. Quanto ai 3C, nell'industria la sigla indica la disciplina

Parametro	Valore	Effetto e consiglio del docente
Spring Arm → Target Arm Length	300	distanza della camera dal personaggio
Spring Arm → Socket Offset	Z = 40	spostamento della camera alla fine del braccio; da preferire
Spring Arm → Target Offset	Z ≤ 20	spostamento dell'origine del braccio; poco e solo sulla Z, perché fa compenetrare la camera
Spring Arm → Use Pawn Control Rotation	attivo	il braccio ruota con il mouse (il controller); va sullo Spring Arm, non sulla Camera
Camera → Field of View	80	angolo di visuale; 80 è tipico dei giochi horror
Camera → Location	non toccare	la posizione si regola sempre dallo Spring Arm

Table 1.9.1: I parametri della camera toccati durante la lezione.

che cura come il giocatore *vede* (camera), *comanda* (controlli) e *abita* (personaggio) il gioco: tre cose che si progettano insieme, perché ognuna cambia la percezione delle altre.

1.9.8 I grafi di movimento, presi in prestito

In Play, la camera c'è, ma "non ci possiamo muovere": "perché non abbiamo né input né le funzioni di base di movimento". "Questo, ragazzi, non andiamo a farlo direttamente a mano, ma andiamo a fare quella che è una bellissima" scorciatoia (la trascrizione dice "mandragata"): si apre il **BP_ThirdPersonCharacter** classico, "quello che abbiamo usato" finora, che contiene "questi tre principali blocchi che Unreal fa di base": il controllo della **camera** (*Look*), il controllo del **movimento** (con *Add Movement Input*) e il **salto** (*Jump*).

Copiare anche l'Input Mapping. "Mi raccomando, prendete anche questo qua sopra, che è l'**input mapping**", cioè il grafo sull'Event Begin Play che aggiunge l'Input Mapping Context ("andremo a spiegare che cosa si intende"). Tutto con **Ctrl+C**, e poi **Ctrl+V** nell'Event Graph di BP_Knight1: "perché queste sono le funzioni basiche di Unreal".

IA_Move. Un'occhiata all'evento **IA_Move** con l'Add Movement Input, "proprio per fare una lezione sull'input": "questo evento dice che è legato a un tasto, o a dei tasti, che ci riporta due valori, X e Y", che "di conseguenza diventano dei vettori che vanno a creare il movimento sui due assi". Uno è avanti e indietro (*forward*, *backward*), l'altro è destra e sinistra.

La camera che ruota male. Compilato, "adesso la nostra camera si muove", ma in modo strano: si muove "solo così", su un asse. "L'avevo visto un'altra volta": "devo usare il control rotation". La spunta è **Use Pawn Control Rotation**: "se lo attaccate, la camera vi va in *free mode*", "nel senso che usa la rotazione del controller", sia in beccheggio sia in imbardata.

1.9.9 Tre spunte sulla rotazione

Use Pawn Control Rotation, sullo Spring Arm. Uno studente ha la camera che "quando vado in Play posso abbassarla e guardare solo il terreno, non vedo proprio neanche più il personaggio". Il docente pensa prima a una compenetrazione, poi si fa condividere lo schermo: "che camera hai messo? Camera normale, giusto?". Guardando i componenti, il problema salta fuori: "non Use Pawn Control Rotation sulla camera: Use Pawn Control Rotation sullo **Spring Arm**". "Quindi levalo da lì e mettilo sullo Spring Arm." "Anche l'altro?" "No, no, no: solo sullo Spring Arm." La spunta va su *un* componente solo, il braccio: è il braccio a ruotare intorno al personaggio, e la camera, che ne è figlia, lo segue.

Orient Rotation to Movement. In Play, il cavaliere cammina (alla velocità di 300), ma andando a destra "lui cammina dritto, fondamentalmente": si sposta di lato continuando a guardare avanti. "Questo lo possiamo risolvere con" **Orient Rotation to Movement**, sempre nel Character Movement: "dal nome, orienta la rotazione al movimento". Accanto c'è la **Rotation Rate**, la velocità con cui il personaggio si gira ("non è proprio precisissima, però ci piace").

Use Controller Rotation Yaw. Il personaggio però continua a non girarsi bene. "Perché mi ricordo che qua c'era una particolarità che non ricordo": il docente scorre le categorie del Blueprint (*Simulate*, *Rotation*, *Character*, *Orientation*, *Physics*, *Controller*...) e la trova. "Per fissare il nostro character che non si girava, basta che eliminiamo il tick": selezionato il Pawn (la radice del Blueprint, *BP_Knight1 (Self)*), si cerca *yaw* e si toglie **Use Controller Rotation Yaw**, "in modo tale che il character non usi il suo yaw". Senza questa spunta, il personaggio non ruota più insieme alla camera: "andrà dove siamo direzionati", "se io vado a destra, lui si gira completamente verso destra". A uno studente non basta: aveva tolto lo Yaw ma non attivato Orient Rotation to Movement. "Ok, non ce l'avevo." Con tutte e due, "abbiamo il nostro bel character fatto per benissimo".

Approfondimento: chi comanda la rotazione. Le tre spunte della lezione sono le tre "mani" che possono ruotare un personaggio in terza persona, e vanno combinate con criterio. **Use Pawn Control Rotation** (sullo Spring Arm) fa seguire alla *camera* la rotazione del controller, cioè del mouse o della levetta destra. **Use Controller Rotation Pitch/Yaw/Roll** (sul Pawn) fa seguire quella stessa rotazione al *corpo* del personaggio. **Orient Rotation to Movement** (nel Character Movement) fa girare il corpo verso la direzione in cui si muove. La combinazione "da avventura" della prima parte della lezione è: camera libera (Use Pawn Control Rotation sul braccio), corpo indipendente dalla camera (Use Controller Rotation Yaw spento), corpo orientato al movimento (Orient Rotation to Movement acceso): è quella del modello Third Person di Unreal. La combinazione "da combattimento" della seconda parte, con il Blend Space 2D (sezione 1.9.16), è opposta: il corpo guarda sempre dove guarda la camera (Use Controller Rotation Yaw acceso), e le animazioni di strafe mostrano lo spostamento laterale. Se sono accese insieme Use Controller Rotation Yaw e Orient Rotation to Movement, le due "mani" si contendono il personaggio; per questo, nel Character Movement, esiste anche *Use Controller Desired Rotation*, che fa ruotare il corpo verso la direzione della camera in modo graduale, con la Rotation Rate (forse il "desired" che il docente prova a un certo punto, senza esito chiaro nella trascrizione).

1.9.10 Il retargeting: il salto del manichino sul cavaliere

"Adesso, però, guardate una cosa: se ad esempio provo a saltare, non ho nessuna animazione." Il salto funziona (lo fa il Character Movement), ma il cavaliere sale e scende nella posa della camminata. Le animazioni di salto si potrebbero prendere da Mixamo, "anche se potremmo, volendo"; invece il docente usa "l'opzione di Unreal che si chiama **retargeting**". "Diciamo che il retargeting, sappiatelo, può uscire bene, ma può uscire anche abbastanza male: è dipendente dall'animazione. Però andiamo a farlo comunque, per vedere come si fa."

I passaggi.

1. Nel Content Browser si cerca *Jump*: si trova l'animazione del salto del manichino base di Unreal, **MM_Jump**.
2. Tasto destro sull'animazione → **Retarget Animations**.
3. La finestra mostra lo scheletro **sorgente** (*source*), quello da cui viene l'animazione: il manichino di Unreal, **Manny** o **Quinn** ("Many Quincy"). Come **target** si sceglie lo scheletro del cavaliere, il paladino di Mixamo: "eccolo qua, li mette a confronto".
4. Si seleziona **MM_Jump** nella lista delle animazioni: nella preview il paladino salta. "Fortunatamente Mixamo usa degli scheletri che sono abbastanza compatibili con quello di Unreal", e il risultato "è venuto abbastanza bene"; "potete girarci tranquillamente intorno con la viewport".

5. Due pulsanti di esportazione. **Export Retargeted Assets** "non ci serve, ragazzi": "vi crea un asset retargettato", una copia completa degli asset sullo scheletro di destinazione. Basta **Export Animations**: "esportare l'animazione così, come nuova". "Ricordate che c'è il source, e vi esporta il target."

6. Come cartella si sceglie quella del personaggio (*Characters* → *Knight1* → *Animation*) e si esporta.

Lo stesso procedimento si ripete per la caduta: **MM_Fall_Loop**, un'animazione "in loop" di caduta, retargettata sul paladino ed esportata nella stessa cartella.

Il root motion "rotto". Il risultato del salto "non è fantastico": "il problema di Mixamo è che ha il **root motion** un po' rotto". Il docente mostra il *root*, "un punto dello scheletro", l'osso alla base della gerarchia, e osserva che lo spostamento non è quello atteso; "potremmo vederlo", "ma per vedere come funziona il retarget va benissimo". Si potrebbe sistemare in seguito.

Approfondimento: il retargeting in Unreal Engine 5. Il retargeting trasferisce un'animazione da uno scheletro a un altro con proporzioni e nomi delle ossa diversi. In Unreal Engine 5 si basa su due tipi di asset: l'**IK Rig**, che descrive uno scheletro raggruppando le ossa in *catene* (spina dorsale, braccia, gambe, testa) con un osso di *retarget root* (di solito il bacino), e l'**IK Retargeter**, che mette in corrispondenza le catene di due IK Rig e corregge la differenza fra le pose di riferimento (la A-pose del manichino e la T-pose di Mixamo). Nelle versioni più recenti il comando *Retarget Animations* del menu contestuale apre una finestra che crea al volo questi asset quando gli scheletri sono riconosciuti (come quelli di Mixamo e del manichino), ed è ciò che si vede a lezione; per risultati migliori, o con scheletri insoliti, si creano e si regolano a mano. Il problema del root di Mixamo è noto: gli scheletri di Mixamo non hanno un osso di root a terra, e lo spostamento del personaggio è registrato sull'osso dei fianchi (*Hips*); un'animazione retargettata può quindi avere il movimento nel posto "sbagliato" per il sistema di root motion di Unreal.

1.9.11 Le regole di transizione

Con le due animazioni nuove si aggiorna l'Animation Blueprint: "dentro il nostro Jump ci mettiamo l'animazione di jump" ("niente di particolare, ragazzi"), dentro Fall l'animazione di caduta. Si ricollegano le frecce. "Se cammina e corre, può saltare? Sì, certo. Se salta, dovrà per forza almeno loopare in caduta", e "dal Fall dovrà ritornare al nostro WalkRun".

Prima versione. Nelle icone delle transizioni c'è un grafo con un nodo **Result**, che "ci dà un bool": "ci dice quando è che posso entrare in questa animazione".

- **WalkRun** → **Jump**: un *Get IsFalling* collegato al Result. "Potremmo farlo più preciso dopo"; per esempio, "possiamo anche dire: ha iniziato il jump".

- **Jump** → **Fall**: qui il grafo vuoto dà errore, "perché noi vogliamo far sì che questo processo tra il Jump e il Fall sia automatico". Selezionata l'icona della transizione, nei Details si spunta **Automatic Rule Based on Sequence Player in State**: "quando inizia quell'animazione, a seguito ci sarà quest'altra"; in altre parole, la transizione scatta da sola quando l'animazione dello stato Jump finisce. "Lo mettiamo così e non ci darà problemi."

- **Fall** → **WalkRun**: "vuol dire che non sta cadendo": Get IsFalling, poi un **NOT Boolean**. "Ricordo che il NOT Boolean è qualcosa che ci dà il suo contrario": deve essere false perché la caduta finisca.

Uno studente chiede cosa ci sia fra Jump e Fall: "se tu clicchi su questo pallino, ci dice, in Details... vogliamo che questa animazione parta direttamente se effettivamente il jump è finito": *Automatic Rule Based on Sequence Player in State*, "questo tick, lo metti vero".

La prova, e un difetto. Si prova da un punto alto, "la nostra piattaforma". "Non si nota tantissimo, però effettivamente sta facendo giustamente l'azione di saltare." Ma il docente vuole collegare WalkRun anche direttamente a Fall: il personaggio "può cadere anche nel caso in cui lui non abbia spinta, semplicemente cade da qualche parte", per esempio scendendo da un bordo. Con le sole regole su IsFalling, però, le due transizioni in uscita da WalkRun avrebbero la stessa condizione, e Unreal non saprebbe distinguere un salto da una caduta.

Seconda versione: la velocità sulla Z. "Dovremmo andare a differenziare un altro valore: la Z. Se la Z è maggiore di 0, vuol dire che sta saltando; se la Z invece è minore di 0, vuol dire che sta cadendo." Nell'Event Graph dell'ABP, finora, della velocity si prendeva solo la lunghezza. Il docente prova prima la strada sbagliata (la *Jump Z Velocity* del Character Movement, che però è la spinta *iniziale* del salto, un valore fisso), poi prende la **Velocity** già salvata, ne fa lo **Split Struct Pin** ("lo splittiamo") e dalla **Z** fa **Promote to Variable**: una nuova float, **VelocityZ** ("Z va benissimo"). Nelle transizioni:

- **WalkRun → Jump**: IsFalling **AND** VelocityZ maggiore di zero, con un nodo **AND Boolean**. Il docente dice anche "quando la velocità sarà maggiore di 1, più preciso": una soglia piccola invece di zero esatto.

- **WalkRun → Fall**: IsFalling e VelocityZ minore di zero.

Per tenere in ordine i fili, si possono usare i *reroute node* ("volete collegarli con questo? È un po' bruttino"). Il risultato: "se io salto, fa l'animazione"; "se io adesso invece cado solamente, lui va diretto in Fall". Il docente chiude salvando anche il BP_ThirdPersonCharacter che l'editor segnala come modificato ("va bene, grazie").

Transizione	**Regola**	**Significato**
Entry → WalkRun	nessuna	lo stato iniziale
WalkRun → Jump	IsFalling AND VelocityZ > 0	è in aria e sta salendo: ha saltato
Jump → Fall	Automatic Rule Based on Sequence Player in State	finita l'animazione di salto, passa alla caduta
WalkRun → Fall	IsFalling AND VelocityZ < 0	è in aria e sta scendendo: è caduto da un bordo
Fall → WalkRun	NOT IsFalling	è tornato a terra

Table 1.9.2: Le regole di transizione della State Machine Locomotion (versione finale).

Approfondimento: altre regole basate sul tempo. L'opzione automatica della lezione è il modo più rapido per dire "quando l'animazione finisce". Romero e Sewell (capitolo 17), per gli stati *StandToProne* e *ProneToStand*, usano invece una regola esplicita: il nodo **Time Remaining (ratio)** dell'animazione dello stato, seguito da un *Less* con 0,1: la transizione scatta quando manca meno del 10% dell'animazione. Il vantaggio è il controllo: la transizione può partire un po' prima della fine e lasciare spazio al *blend* fra i due stati, che ogni transizione ha (nei Details: *Blend Settings*, con una durata di default di 0,2 secondi). Ricordano anche che le animazioni "di una volta", come il salto, vanno impostate senza *Loop Animation*: è il caso dello stato Jump, mentre Fall, che deve durare quanto la caduta, resta in loop.

1.9.12 Lo sprint con l'Enhanced Input

"Adesso andremo a mettere il nostro sprintino": "un semplicissimo sistemino che passa la Max Walk Speed da un tot a un altro tot". Si torna in BP_Knight1, "a ragionare a livello di Blueprint". "Ma sì, facciamo anche l'input mapping."

L'Input Mapping Context. "Unreal ha introdotto, quasi dalla 5.2", quello che è l'**Input Mapping Context**, "una cosa molto interessante di Unreal che ci permette, ad esempio, di avere degli eventi legati a più input". Finora, in molte lezioni, si sono usati gli eventi dei tasti ("abbiamo visto il tasto E, eccolo qua"): "semplicemente un rilevamento, da parte di Unreal, della tastiera: se è premuto o non è premuto". Adesso, invece, "gli eventi sono legati a più controlli possibili".

Nel Content Browser si cerca **IMC** (*Input Mapping Context*). "È un *contesto*: un contesto di input che vengono dati in un certo momento, e sappiate che sono sovrascrivibili." L'esempio: "io devo cambiare dal gioco di avventura 3D al gioco di guida della macchina"; "i controlli sono completamente diversi, quindi cambio contesto": un Input Mapping Context per la macchina e uno "per il character che cammina normalmente". Aperto **IMC_Default**, ci sono i **Mappings**: **IA_Jump**, **IA_Move**, **IA_Look**.

L'Input Action IA_Run. "Per far sì di avere un'Input Action, dobbiamo creare un asset chiamato **Input Action**." Cercando *IA* nel Content Browser "troverete un sacco di asset, ma a voi vi interessa questo celestino". Con il tasto destro, **Show in Folder View**, "in modo tale che voi troviate direttamente la cartella". Si duplica IA_Jump (Ctrl+C, Ctrl+V, o *Duplicate*) e lo si rinomina **IA_Run**.

Dentro un'Input Action. Aperto IA_Run, "ci sono un po' di valori che possiamo vedere":

- **Description**: "semplice descrizione, quindi quello che fa";
- **Trigger when Paused**: se l'azione deve scattare anche quando il gioco è in pausa, "quindi si blocca il tempo";
- **Value Type**: "la tipologia di valore che ci ritorna indietro. Un input ci dà un valore": **Digital (bool)**, **Axis1D (float)** ("come abbiamo visto, avanti e indietro"), **Axis2D (Vector2D)** ("più assi, due dimensioni") o **Axis3D (Vector)** ("tre tipologie di input che ci vengono dati"). "A noi interessa il Run, che è un bool, quindi premuto o non premuto": Digital (bool) "va benissimo";
- **Triggers**: "le azioni che sono già preinstallate in Unreal", che dicono "ehi, guarda, si triggera se è premuto, o se è rilasciato, o se è sempre premuto". L'elenco letto dal docente: *Chorded Action*, *Combo*, *Down*, *Hold*, *Hold and Release*, *Pressed*, *Pulse*, *Released*, *Tap* (la trascrizione dice anche "Short Deduction" e "repeat", non identificabili con certezza). "Fondamentalmente, questo vuol dire che quando si compie questa azione, questo input viene triggerato." IA_Run eredita da IA_Jump la sua configurazione, "e va benissimo così, perché tanto è un'azione molto simile al jump".

Voce	**Significato**
Digital (bool)	premuto / non premuto: un pulsante (salto, corsa, interazione)
Axis1D (float)	un valore su un asse: un grilletto analogico, avanti/indietro
Axis2D (Vector2D)	due assi: il movimento (X, Y) o la visuale del mouse
Axis3D (Vector)	tre assi: input 3D (per esempio un controller di movimento)
Down	scatta a ogni frame finché il tasto è giù
Pressed	scatta una volta, quando il tasto viene premuto
Released	scatta una volta, quando il tasto viene rilasciato
Hold	scatta dopo che il tasto è stato tenuto premuto per un certo tempo
Hold and Release	scatta al rilascio, se il tasto è stato tenuto abbastanza
Tap	scatta se il tasto viene premuto e rilasciato entro un tempo breve
Pulse	scatta a intervalli regolari mentre il tasto è premuto
Chorded Action	scatta solo se un'altra azione è attiva insieme (una "combinazione")
Combo	scatta se una sequenza di azioni viene eseguita nell'ordine

Table 1.9.3: I tipi di valore (Value Type) e i trigger di un'Input Action, letti a lezione dall'elenco dell'editor. Le descrizioni sono un'integrazione: a lezione il docente legge i nomi senza spiegarli uno per uno.

Il contesto va caricato. Tornati in BP_Knight1: "abbiamo già il contesto caricato", con il grafo copiato dal ThirdPersonCharacter. "Sappiate che se voi non caricate il contesto, non vi prenderà nessuna cosa": sull'Event Begin Play, dal Player Controller si prende l'*Enhanced Input Local Player Subsystem* e si chiama **Add Mapping Context** con IMC_Default (figura 1.9.4).

L'evento IA_Run e i suoi pin. Nel grafo si invoca **IA_Run**; "se lo estendete" (la freccina in basso del nodo), compaiono tutti i pin:

- i pin di esecuzione **Triggered** ("quando è triggerato"), **Started** ("quando inizia a essere triggerato"), **Ongoing**, **Canceled**, **Completed**;
- i valori: l'**Action Value** ("se è true o false, in questo caso"), **Elapsed Seconds** e **Triggered Seconds** ("quanto tempo è stato triggerato"), e l'**Input Action**, "la referenza stessa" all'asset.

Set Max Walk Speed. "Dobbiamo modificare il Character Movement, ragazzi, che dovrà definire quanta velocità avremo": si trascina nel grafo il componente **Character Movement** e se ne tira fuori **Set Max Walk Speed**, due volte. Sul **Triggered**, **600**; sul **Completed**, **300** (figura 1.9.5).

Il mapping dimenticato. In Play non succede niente. "Scusate, ragazzi, mi sono dimenticato di dare l'input": IA_Run esiste, ma non è nel contesto. Si apre IMC_Default, si aggiunge un mapping (il + accanto a *Mappings*), si sceglie IA_Run e gli si assegna il tasto: "un nostro bello **Shift**". "L'Input Mapping Context ci permette di mettere quali tipologie di pulsanti facciano" ciascuna azione.

Lo stesso input, anche con il controller. "Io ad esempio qua, ragazzi, ho un controller": il docente lo collega ("Unreal riesce a prendere il controller"), aggiunge un secondo tasto alla stessa azione, clicca sull'icona della tastiera accanto al campo e preme un pulsante del controller ("mi sono fregato la sorpresa"): "facciamo tipo Dark Souls? Premo cerchio", e l'editor scrive **Gamepad Face Button Right**. "Questa azione, cioè l'azione di correre, è collegata a più input." "Quando faccio progetti in generale, devo fare anche il controller": si corre con Shift sulla tastiera "oppure con il nostro bel controllino, con quello che vogliamo noi", "come è stato designato il gioco". Tutto il resto del movimento sul controller "non ci interessa adesso".

La prova. "Se noi proviamo... rilascio." Con Shift il cavaliere corre, e il Blend Space passa alla corsa; al rilascio torna a camminare. "Ecco a voi che abbiamo creato un nostro bel character nuovo, in cui semplicemente possiamo correre, possiamo modificare la velocità."

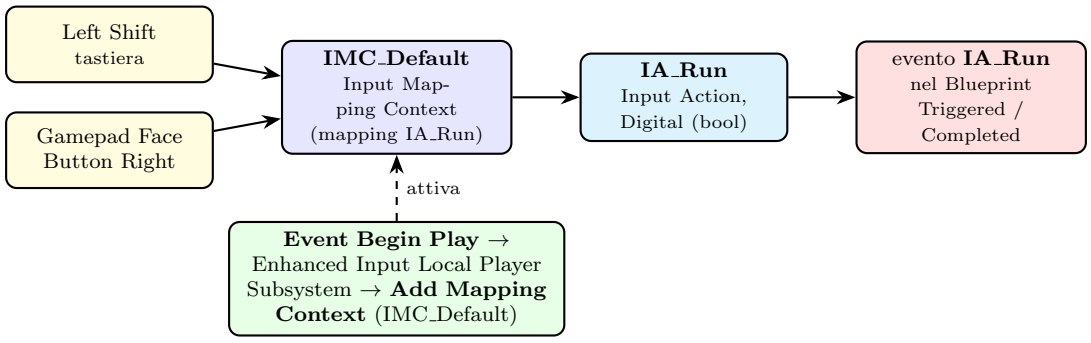


Figure 1.9.4: La catena dell'Enhanced Input. I tasti fisici non sono legati direttamente all'evento: il contesto (IMC) associa più tasti alla stessa azione (IA), e il Blueprint reagisce all'azione. Senza Add Mapping Context, il contesto non è attivo e nessun input arriva. Schema ricostruito.

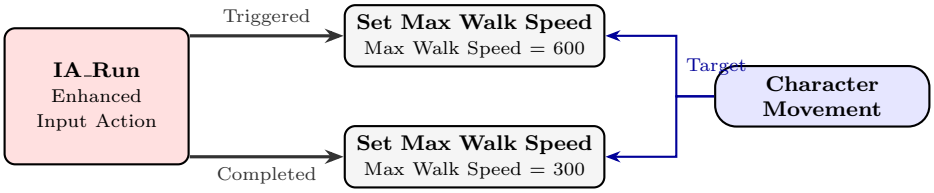


Figure 1.9.5: Lo sprint di BP_Knight1. Premendo Shift (o il cerchio del controller) la velocità massima sale a 600 e il Blend Space mostra la corsa; al rilascio torna a 300, la camminata. Schema ricostruito.

Approfondimento: Action Mapping ed Enhanced Input. Il libro di Romero e Sewell, scritto per le prime versioni di Unreal Engine 5, costruisce lo stesso sprint con il vecchio sistema: un *Action Mapping* chiamato *Sprint* nei Project Settings (sezione *Input*), legato a *Left Shift*, e un evento *InputAction Sprint* con i pin *Pressed* e *Released*, su cui due *Set Max Walk Speed* portano la velocità a 2200 e poi di nuovo ai 600 di default (capitolo 6). Il principio è identico; cambia il sistema. L'**Enhanced Input**, introdotto come plugin in Unreal Engine 5.0 e diventato il sistema predefinito dei modelli dalla 5.1 (il "quasi dalla 5.2" del docente è un ricordo approssimativo), separa tre livelli: l'*azione* (Input Action, che cosa vuole fare il giocatore), il *contesto* (Input Mapping Context, quali tasti attivano quali azioni in una certa situazione) e il *comportamento* (Triggers e *Modifiers*, come per esempio la *zona morta* di una levetta o l'inversione di un asse). I contesti si aggiungono e si tolgono a runtime con *Add Mapping Context* e *Remove Mapping Context*, e hanno una *Priority*: è così che si passa "dal gioco a piedi alla macchina" dell'esempio del docente, e così che si costruisce una schermata di rimappatura dei tasti. Gli autori raccomandano anche di raggruppare i nodi dello sprint in un *commento* (*Create Comment from Selection*), per tenere ordinato il grafo.

1.9.13 Un tentativo: FInterp To

"Se volessimo farlo un po' più carino, ragazzi", il passaggio fra 300 e 600 potrebbe essere graduale invece che istantaneo. Il blocco è **FInterp To** ("finter2"): "fa un'interpolazione di un float". "Avete presente quando abbiamo parlato di interpolazione?" "Fa il passaggio tra due numeri. Lui dice: devi passare da questo valore, cioè **Current**, a questo valore, **Target**, in questo tempo", **Delta Time**, per cui il docente consiglia **Get World Delta Seconds**; e poi c'è l'**Interp Speed**, la velocità dell'interpolazione.

Il grafo. Sul Triggered: **Current** = Character Movement → **Get Max Walk Speed**; **Target** = 600; **Delta Time** = World Delta Seconds; il risultato nel Set Max Walk Speed. Il docente ricorda che il Delta Seconds si può moltiplicare per un valore ("ricordiamo che possiamo modificarlo moltiplicandolo per un valore"), e prova con 10, poi con un decimo. Sul Completed lo stesso, "ma semplicemente invertiamo": il Current parte da 600, il Target è 300. "Ricordiamoci di collegare anche al Character Movement."

Non funziona, e perché. La corsa resta "ancora molto veloce", il passaggio istantaneo. Il docente prova valori diversi (10, 100: "è alla norma del pianeta stare continuamente a vedere se i valori sono giusti") e poi mette dei **Print String**: "ci dà seicento". La diagnosi: "fondamentalmente questa è un'azione a triggered unico, quindi non farà l'interpolazione". FInterp To fa *un passo* dell'interpolazione ogni volta che viene eseguito: se viene eseguito una volta sola, fa un solo passo. "Lo deve fare tramite un **timer**": si rimanda a dopo ("volendo, ragazzi, si può fare con un timer, però facciamolo magari dopo"), e non verrà ripreso nella lezione.

Approfondimento: dove mettere un'interpolazione. Un nodo come FInterp To (o VInterp To, RInterp To per vettori e rotazioni) va eseguito *a ogni frame* finché il valore non arriva al target: il suo risultato è il valore "un po' più vicino" al target, e il Delta Time serve a renderlo indipendente dal frame rate. Ci sono tre modi tipici di farlo girare: nel **Tick**, con una variabile *TargetSpeed* che i due pin dell'input si limitano a impostare (600 o 300); con un **timer** (*Set Timer by Event*, con *Looping*, fermato con *Clear and Invalidate Timer* quando il valore è arrivato), come suggerisce il docente; o con una **Timeline**, vista nella sesta lezione,

che va da 300 a 600 con *Play* e torna indietro con *Reverse*. Con l'Enhanced Input c'è anche una quarta via: il pin *Triggered* scatta a ogni frame se il trigger è *Down* (o se non c'è nessun trigger), e una sola volta se il trigger è *Pressed*. IA_Run, copiata da IA_Jump, ha probabilmente un trigger che la fa scattare una volta sola: è "l'azione a triggered unico" del docente.

1.9.14 Prima della pausa: le animazioni da scaricare

"Adesso andremo a vedere, se non mi esplode prima Unreal, anche come si fanno i **montaggi**"; mancano una ventina di minuti alla pausa. "Dovremmo andare un attimo su Mixamo" ("a quanto pare oggi Mixamo ci mette pure un bel po' a caricare"), per scaricare due gruppi di animazioni.

Un attacco in root motion. "Vorrei che sceglieste un'animazione che sia in **root motion**": "riprendiamo ad esempio una situazione simile a Dark Souls". "Ricordatevi sempre di prendere il vostro character" su Mixamo (il paladino, nel caso del docente), perché le animazioni si adattano al personaggio selezionato. Il docente cerca *attack* e sceglie un attacco con la spada che "di base, come potete vedere, non può essere messo in place". Le impostazioni di download sono quelle della volta precedente: **Without Skin**, **FBX Binary**, **30** fotogrammi al secondo. "Ripeto: non deve essere in place, deve avere il root motion, quindi il movimento ereditato." "Un'azione unica": "ad esempio un attacco con una spada, un qualche cosa", "un'animazione che vi piace di più".

Le animazioni del Blend Space 2D. Dopo la pausa si farà il **Blend Space 2D**, "che ci servirà per fare anche il calcolo della direzione del nostro character". Servono quindi altre animazioni: "walk destra, sinistra, dritto e indietro". "Consiglio spassionato": su Mixamo si chiamano **Strafe** ("provate a cercare Strafe"; "Jog Strafe va benissimo anche"), e queste, al contrario dell'attacco, "dovete metterle **in place**".

Il mirror. "Piccolo consiglio: se volete far sì che siano abbastanza simili tra di loro, potete fare il **mirror**." Si scarica l'animazione verso sinistra non specchiata; poi, con l'opzione *Mirror* di Mixamo, "in automatico andrà a destra". Le due animazioni saranno così perfettamente simmetriche.

Quante animazioni. Su richiesta ("le puoi scrivere per messaggio?"), il docente le elenca anche in chat: "in totale sono otto animazioni". Avanti, indietro, destra, sinistra, e le quattro **diagonali**: avanti a destra, avanti a sinistra, indietro a destra, indietro a sinistra. Uno studente chiede se l'avanti serva, visto che c'è già la camminata: "nel senso, possiamo usare tranquillamente quello di avanti; ma in questo caso abbiamo solamente avanti, dobbiamo trovare anche indietro". E un'altra: le diagonali "soltanto in avanti, o anche indietro?" "Anche indietro": "se io vado tipo così, indietro verso destra, e così, indietro verso sinistra". "Praticamente è la X, letteralmente": le due diagonali incrociate, oltre alla croce degli assi. Il docente nomina anche alcuni pacchetti di Mixamo con animazioni di locomozione ("Locomotion Pack"; il resto dei nomi è incomprensibile nella trascrizione). "Ci rivediamo alle 18."

1.9.15 La pausa e i feedback

Al rientro, "siamo rimasti in cinque", poi in quattro. Il docente apre uno spazio per i commenti: "se avete dei feedback riguardo a questo corso, sparate tranquillamente", "siamo anche molto coetanei, anzi credo io di essere anche forse più piccolo di alcuni di voi". "Se avete cose da chiedere, magari di andare più piano, andare più veloce, oppure un po' come sono strutturate le lezioni". "Capisco pure che può essere effettivamente un po' pesante in alcuni momenti", "perché purtroppo i Blueprint, o Unreal in generale, è divertente quando poi le sai, le cose".

Le funzioni da imparare a memoria. Uno studente: "immagino che sono tantissime funzioni, quindi sono tutte da imparare a memoria". "Quello, eh sì."

Il progetto del docente: la pallina morbida e dura. Per mostrare dove porta la pratica, il docente racconta un suo progetto personale: "una specie di *platformer* in cui una pallina diventa morbida e dura, nel senso che diventa di metallo o di un materiale più morbido". "La cosa è che ho simulato tutta la fisica": "la fisica non è simulata da Unreal, ma sono io che faccio finta, tramite i Blueprint, di far fare delle funzioni fisiche". "Hai messo proprio te le formule fisiche?" "Più o meno." Il meccanismo: si rileva lo **slope**, "il quanto è piegato il piano", perché "la meccanica è il fatto che se sei più pesante con la pallina vai più veloce su uno slope, come nella realtà": "una pallina di metallo e una pallina di gomma: quella di metallo va più veloce, perché è più pesante, comunque più influenzata". "È tutto simulato": "sono io che, tramite la detection dello slope, cambio i valori".

Combinare i blocchi. "E quello è ovviamente tutto custom": "quando hai tutte le nozioni in testa, ti dici: ok, so questo blocco A, B, C, D, E, F, G; quale mi serve? Il blocco C e il blocco A. Li metto insieme e faccio una nuova combinazione." "È sempre il fatto di capire come posso interlacciare le cose." "Qua stiamo facendo cose magari un po' generiche"; poi "dici: ok, voglio fare simile a quello, ma leggermente diverso. Voglio fare il character, però con la spada: che mi cambia?"

Inventari e UI. La richiesta di Federico: "a me interesserebbero la gestione degli inventari e le interfacce". "Sfortunatamente no": "sappi che l'inventario e la UI sono un mondo a parte". "Esistono proprio figure riguardanti questo mondo della UI, sia designer sia programmatori." Si farà "una cosa come un menu", "una cosetta tranquilla", "però per il resto non andremo a fare di più".

Le lezioni con Davide. "Sappiate che il 25 è la mia ultima lezione, e avrete tutte quelle con **Davide**, che invece vi farà la parte più di *level*." Con lui si useranno "tutti gli asset che abbiamo creato: le piattaforme, i pulsanti", per lavorare su concetti come la **gym**, "il concetto della palestra", che "si fa praticamente in via di sviluppo": "è il parco giochi", "letteralmente il parco giochi dove ho tutte le meccaniche, le posso provare tutte quante, vedo se funzionano". Poi si costruiscono situazioni: "voglio fare un livello horror, la bestia che ti insegue: tu entri in una stanza e devi fuggire dalla bestia"; "situazione X, situazione Y, situazione Z", "le costruisci e metti insieme i componenti", "e vedi quei componenti che hai programmato come funzionano a livello anche di esperienza, di gioco". Gli elementi saranno "quelli di un classico platform: le piattaforme, i bottoni", e si vedranno anche i tool di **modeling** dell'editor e "un po' di teoria di level design": "chi fa game design, e a chi interessa tantissimo anche la parte di level: molto, molto interessante".

Dove lasciare i feedback. "Ripeto, ragazzi, anche a chi sta in differita": qualsiasi feedback sulle lezioni va lasciato sul canale **general** della chat, "non su introduzione al materiale, perché sennò si fa casino". Non solo sugli argomenti, "ma feedback anche proprio per le lezioni in generale: andare più lento, andare più veloce, vedere cose più pratiche".

Approfondimento: la gym. Nel level design la **gym** (o *gym level*, *gray box gym*) è un livello di servizio che non finirà nel gioco: una stanza in cui ogni meccanica è disposta in isolamento e misurata. Si mettono salti di lunghezze crescenti per sapere qual è il salto massimo del personaggio, rampe di pendenze diverse per verificare la *Max Walkable Floor Angle*, gradini di varie altezze per la *Max Step Height*, corridoi di larghezze diverse per la camera. Le misure raccolte nella gym diventano le "regole metriche" del level design (quanto può essere largo un burrone, quanto alto un muretto da scavalcare), e i livelli veri si costruiscono rispettandole. È il "parco giochi" del docente; e il problema della camera che compenetra, visto in questa lezione, è uno dei primi che una gym fa emergere.

1.9.16 La direzione: Calculate Direction

"Abbiamo detto prima che oggi dobbiamo fare il Blend Space 2D." Il docente ha importato otto animazioni, "cercando di prenderle tutte similari": *Jog Backward*, *Jog Backward Diagonal*, *Jog Forward*, *Jog Forward Diagonal* e così via, "a parte lo Strafe Left, che devo eliminare perché non andava bene come animazione". Il Blend Space 1D prende solo la velocità: "vai più veloce, l'animazione, benissimo". Il 2D aggiunge un

secondo asse, la **direzione**, "in modo tale da non dover magari direzionare sempre il character", ma vedere "che si muove in diagonale, si muove a sinistra".

Dove si calcola. In ABP_Knight, nell'Event Graph dell'Update, si aggiunge un pin alla **Sequence** (il *+*, *Add pin*): *Then 2*, o il successivo. Questa volta non si parte dal Movement Component, ma dal **character**: "perché la direzione è determinata dalla rotazione del character stesso", "cioè la capsula, come è ruotata". Dalla variabile del cavaliere si tira fuori **Get Actor Rotation**.

Il nodo Calculate Direction. "Potremmo tranquillamente prenderla così, ma noi, per semplificarci anche la vita, andiamo a calcolare proprio la direzione con un blocco chiamato proprio **Calculate Direction**." Il nodo chiede due ingressi: la **Velocity** attuale (dal Movement Component, **Get Velocity**) e la **Base Rotation** (la Get Actor Rotation). Restituisce "un float", che con **Promote to Variable** diventa **Direction**.

Uno studente perso. "Io mi sono perso: Character, Movement Component, da dove vengono questi due?" Il docente ripercorre: "hai fatto questo Event Blueprint Initialize Animation nell'Event Graph, no?"; "dovresti avere qua sotto, sotto le variabili, la variabile del character: la tiri fuori, ne tiri fuori Get Actor Rotation"; dal Movement Component "tiri fuori la velocità"; "da Get Actor Rotation fai Calculate Direction, e questo diventerà il nostro float". Un altro studente nota che la variabile del docente si chiama *Character*, mentre la sua si chiama **As BP Knight**: "va benissimo. Io l'ho solo rinominata." Quando si fa Promote to Variable sull'uscita di un cast, Unreal propone il nome *As* più il nome della classe ("ti dice proprio As BP Knight").

Da −180 a 180. "Molto semplicemente, la direzione sarà tra un range, sempre senza doverla clampare" ("il clamp l'abbiamo fatto quasi all'inizio"): fra **−180** e **180 gradi**. "Si parte da 0; a destra vai verso il positivo; quando arrivi a 180 passa a −179"; "diciamo, non lo fa come un angolo da 0 a 360, ma fa 0, 180, −180 e di nuovo 0" (figura 1.9.6).

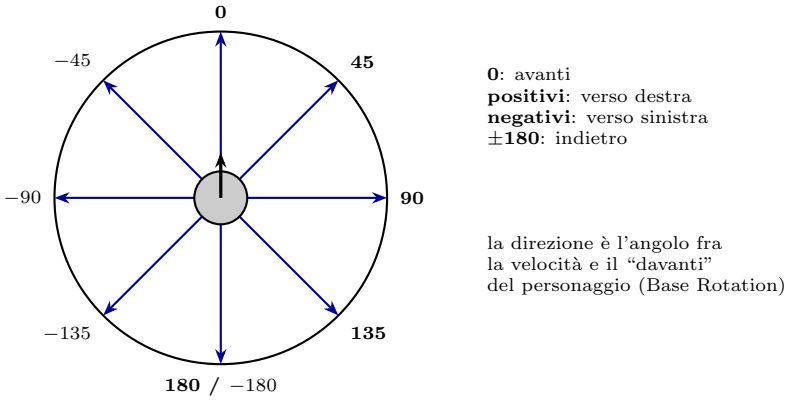


Figure 1.9.6: Il valore di Direction restituito da Calculate Direction. Il personaggio guarda in alto; le frecce sono le otto direzioni delle animazioni del Blend Space 2D. Andare indietro vale sia 180 sia −180, a seconda del lato da cui ci si arriva. Schema ricostruito.

Rimettere Use Controller Rotation Yaw. Per vedere i movimenti laterali, "importante è rimettere lo **Use Controller Rotation Yaw**": "non vogliamo che il character ruoti sempre nella direzione in cui lo stiamo indirizzando". Il corpo deve seguire la camera, e la direzione del movimento rispetto al corpo deve cambiare.

La verifica con Print String. Per sicurezza si stacca un attimo il Blend Space 1D ("staccare un attimo le animazioni") e si stampa Direction con un **Print String**, "che lo farà a ogni frame". "Ci dà direttamente

nella preview questo valore"; in Play, "sarà un po' rotto, ma è normale, ragazzi": "$-70, 70, 160, 0$". "Mamma mia" (il valore dipende dalla camera, "giustamente"). "Come possiamo vedere, questi valori rappresentano la nostra direzione attuale." Il docente nota anche che il cavaliere sembra "leggermente ruotato" rispetto alla capsula: "buono a sapersi".

Approfondimento: che cosa calcola Calculate Direction. Il nodo, della libreria dell'Animation Blueprint (*KismetAnimationLibrary*; nelle versioni più vecchie era una funzione dell'AnimInstance), trasforma la velocità nello spazio locale del personaggio e ne restituisce l'angolo sul piano orizzontale, in gradi, nell'intervallo $[-180, 180]$: 0 se la velocità punta dove guarda il personaggio, 90 se punta alla sua destra, -90 a sinistra, ± 180 dietro. A velocità nulla restituisce 0. Con *Orient Rotation to Movement* acceso il personaggio si gira sempre verso la velocità, e la direzione resta quasi sempre vicina a zero: il Blend Space 2D avrebbe poco da fare. Per questo il docente riattiva Use Controller Rotation Yaw.

1.9.17 Il Blend Space 2D

Creare l'asset. Tasto destro → *Animation* → **Blend Space**: "Blend Space senza niente, di base, è 2D; se viene specificato 1D, è 1D". Unreal chiede lo scheletro: "voi scegliete il vostro scheletro". Il nome: **BS2D_Knight**.

I due assi. "Come potete vedere, a sinistra e giù sono presenti due nomi", per ora *None*. Nei Details:

- **Horizontal Axis**: **Direction**, da -180 a 180;
- **Vertical Axis**: **GroundSpeed** ("più che la velocità, la ground speed"), da **0** a **600**.

Il parametro **Grid Divisions** "ci permette di aumentare il numero di divisioni sulla nostra griglia" (le linee "non si vedono, perché sono veramente molto poco chiare"): sono i punti su cui si agganciano i campioni delle animazioni. Il docente ne mette **8** su entrambi gli assi. Con 8 divisioni, sull'asse orizzontale ogni passo vale 45 gradi, esattamente le otto direzioni; sull'asse verticale vale 75.

Un quiz. "Domanda, ragazzi: se io sto a 600 di velocità e a 90 di direzione, dove sto andando?" "Ricordatevi che positivo è destra, negativo è sinistra." "A destra." "Pienamente a destra, perfetto." "E invece a -135?" "Dietro a sinistra." "Ottimo."

Dove va ogni animazione. Si dispongono i campioni (tabella 1.9.4 e figura 1.9.7). In basso, a velocità zero, l'**Idle**: "mettetene tre, così abbiamo un range per tutto" (a -180, 0 e 180): "siamo fermi, siamo fermi". In alto, a 600, le otto animazioni. L'animazione all'indietro va messa due volte, a -180 e a 180: "stiamo andando indietro sia se abbiamo girato da sinistra, sia da destra". Il docente si corregge una volta ("Backward Diagonal L sarà... no, ho sbagliato": a -135, non a 90) e invita a controllare i valori nei Details del campione.

La preview con Ctrl. "Potete fare la preview delle animazioni con **Ctrl**": tenendolo premuto e trascinando il cursore sul grafico, "potete vedere come l'animazione si muove tra tutti gli step". "Consiglio sempre di mettersi dietro" il personaggio nel viewport, "perché tanto poi, se lo vedete davanti, è difficile" capire destra e sinistra. Il docente controlla tutte le direzioni ("vado a sinistra, vado indietro, vado dritto, vado dritto a destra, vado a destra, giù a destra, indietro. Perfetto"); una delle animazioni "mi sa che non va bene", ma "sì, sto andando a sinistra, è solo molto strana".

Smoothing Time. "Piccola cosa, se volete introdurla": lo **Smoothing Time**, "il tempo di passaggio tra un'animazione e l'altra in generale, per tutte le animazioni". Lo si mette a **0,2** su entrambi gli assi. Il *Damping Ratio* "non ci interessa".

Animazione	**Direction**	**GroundSpeed**	**Movimento**
Idle (tre copie)	−180, 0, 180	0	fermo
Jog Forward	0	600	avanti
Jog Forward Diagonal R	45	600	avanti a destra
Strafe R	90	600	a destra
Jog Backward Diagonal R	135	600	indietro a destra
Jog Backward	180 e −180	600	indietro
Jog Backward Diagonal L	−135	600	indietro a sinistra
Strafe L	−90	600	a sinistra
Jog Forward Diagonal L	−45	600	avanti a sinistra

Table 1.9.4: La posizione dei campioni nel Blend Space 2D BS2D_Knight. I nomi delle animazioni sono quelli di Mixamo citati a lezione, per quanto ricostruibili dalla trascrizione; i valori sono quelli dettati dal docente.

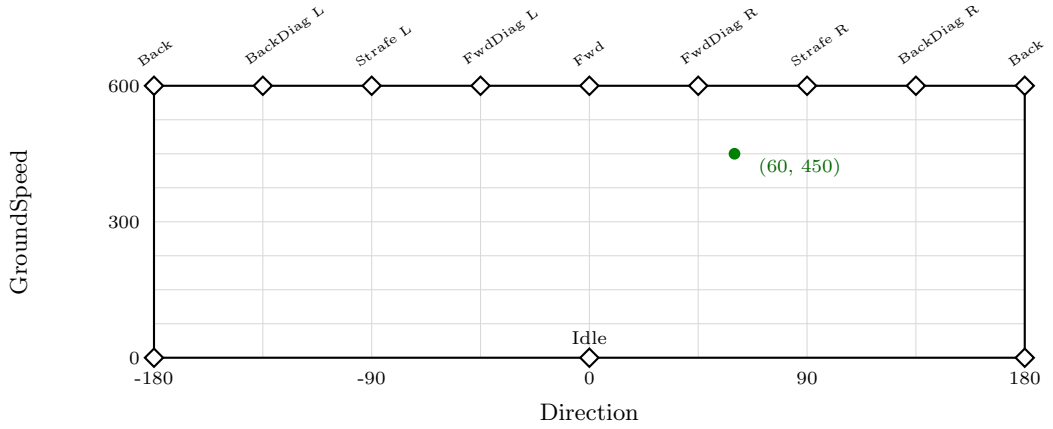


Figure 1.9.7: Il Blend Space 2D del cavaliere: Direction in orizzontale, GroundSpeed in verticale, griglia 8×8. Le tre copie dell'Idle stanno in basso; le otto direzioni in alto, con la corsa all'indietro ripetuta ai due estremi. Il punto verde è un esempio: il personaggio sta andando in avanti a destra, un po' sotto la velocità massima, e Unreal mescola i quattro campioni più vicini. Schema ricostruito.

Nello stato WalkRun. "Facciamo tutto il percorso a ritroso": AnimGraph, Locomotion, WalkRun. Al posto del Blend Space 1D si trascina BS2D_Knight, che chiede "dammi la Direction e la GroundSpeed": si collegano i due Get. Compile: "vediamo che il nostro Blend Space funziona in preview".

Le impostazioni del personaggio. In BP_Knight1: "mi raccomando, l'abbiamo detto prima, ma lo diciamo anche adesso", si riattiva **Use Controller Rotation Yaw**; e nel Character Movement la **Max Walk Speed** va a **600**, "che è quanto abbiamo messo" come massimo dell'asse. "Ecco a voi, ragazzi": "se io mi muovo a destra lui va a destra, se mi muovo a sinistra, anche se l'animazione non è fantastica, va a sinistra; vado indietro; vado in diagonale". Nel frattempo "parte la torretta, che mi spara": "il mio bel cavaliere se ne va a zonzo come gli pare".

Gli scatti. C'è "un piccolo *clip*", uno scatto, in alcuni passaggi. Il docente prova a prendere la rotazione della **Mesh** invece di quella dell'actor ("facciamo che invece di prendere la direzione del character, prendiamo quella della Mesh"; "Get Mesh"), poi ad aumentare lo Smoothing Time della Direction a **0,5** ("un po' più tranquillo"), e infine torna al collegamento originale. La conclusione: "probabilmente sono dei problemi sull'animazione"; "mi sa che devo riscaricare l'animazione, perché snappa un pochino". Ma il risultato c'è: "abbiamo il nostro character che si muove in tutte le direzioni"; "è un po' veloce, devo essere sincero", e la velocità si può abbassare.

Il cerchio, per chi si è perso. Uno studente chiede di rivedere il Blend Space 2D. "Immaginatelo come un cerchio": "se stai a 0 di direction, stai andando dritto; se stai a 180 o -180, stai andando dietro"; "tutti i valori intermedi" sono le diagonali e i lati. "Destra è positivo: 0, 90, 180; sinistra è -90, -180." "Se vai in diagonale a destra sarà 45, se vai pienamente a destra sarà 90", "e 135 a 600 vuol dire che andrai praticamente indietro, ma verso destra". "Una volta capito che una è l'opposto dell'altra", i valori negativi seguono da soli.

Approfondimento: Blend Space 2D e foot sliding. In un Blend Space 2D Unreal divide lo spazio dei campioni in triangoli e, per un valore dato, mescola i campioni ai vertici del triangolo che lo contiene. Ne seguono tre conseguenze pratiche. Primo, l'Idle ripetuto lungo tutta la riga inferiore (le tre copie del docente) evita che a bassa velocità il personaggio "scelga" una direzione a caso. Secondo, la corsa all'indietro va duplicata ai due estremi, perché -180 e 180 sono lo stesso angolo ma due punti diversi della griglia: con un solo campione, passando da 179 a -179 il personaggio attraverserebbe in un istante tutto il grafico. Terzo, lo scatto che il docente vede è spesso proprio quel salto da $+180$ a -180, che lo *Smoothing Time* non può interpolare bene (interpola il valore numerico, non l'angolo); le alternative sono il *Weight Speed* per campione, oppure le soluzioni più avanzate di Unreal Engine 5 come l'*Orientation Warping*, che ruota la parte inferiore del corpo verso la direzione del movimento con quattro sole animazioni cardinali. Infine, una griglia con l'asse verticale fino a 600 e i campioni tutti a 600 significa che anche la camminata a 300 è un misto fra Idle e corsa: per avere una vera camminata laterale servirebbe una seconda riga di campioni a 300, con le animazioni di camminata.

1.9.18 Gli Animation Montage

"Intanto ci abbiamo un'ultima mezz'oretta, ragazzi. Vediamo un attimo i Montage." L'animazione di attacco ("l'avevo scaricata, mi sa che non l'ho messa in Knight Animation"; "scusate, ragazzi") è una **Sword and Shield Attack** di Mixamo: "questa qua sta in root motion": "come potete vedere, eredita il movimento".

Il problema. "Quando vogliamo fare una nostra animazione che di conseguenza sia attivabile durante un ciclo di animazioni", non basta aggiungere uno stato: "dobbiamo creare quello che viene chiamato un **Montage**". (Nel frattempo "Filippo è andato via, è crashato": "siamo rimasti noi tre".)

Che cos'è un Montage, dalla documentazione. Il docente apre la pagina della documentazione di Unreal sugli **Animation Montage**. "Il montaggio di animazioni ci permette sia di montare delle animazioni..." L'esempio della pagina è la ricarica di un fucile: un'animazione formata da tre animazioni diverse, *MM Shotgun Reload Drop*, *Reload Load* e *Reload Lock*. "Mettiamo conto che vogliamo fare un bel gioco shooter. Il fatto di staccare il caricatore su quattro fucili diversi è praticamente uguale"; "non dobbiamo fare quattro animazioni diverse": "l'unica cosa che cambia è il lock, perché magari uno vuole farlo un po' più grosso". Con un Montage si creano **animazioni modulari**: "guarda, il drop è uguale, il load è uguale, il lock invece è diverso". E poi il secondo uso, quello di oggi: "inserire animazioni in contesti di cicli di animazione"; "sono una tantum": "premo un tasto e compio questa animazione, nel mentre che sto camminando". "Magari la prossima volta andiamo a vedere pure un po' più approfondito."

Creare il Montage. Tasto destro sull'animazione di attacco → **Create** → **Create AnimMontage**. Il nuovo asset si chiama come l'animazione con il suffisso *Montage* (*Sword And Shield Attack_Montage*). "La cosa bella del Montage è che ci permette di fare un sacco di cose all'interno dei suoi parametri."

1.9.19 Dentro il Montage: Notify, sezioni e Slot

I Notify. "Una cosa interessante da spiegare sono i **Notify**": "dei punti, dei puntini, sulla linea del tempo dell'animazione", che permettono di aggiungere "un'azione che si triggera": "suoni, un VFX, qualsiasi cosa". "Quelli più generici sono già fatti da Unreal, fortunatamente." Il docente fa clic destro sulla traccia dei Notify, **Add Notify** → **Play Sound**, e come suono sceglie "l'unico sound che abbiamo", quello della piattaforma (un Sound Cue). Le tracce dei Notify possono essere più d'una: con *Add Notify Track* se ne crea un'altra, per esempio una per i VFX e una per i suoni. In preview (tolto il loop, "fammelo fare una volta"): "appena il nostro character arriva a questo punto dell'animazione, compie il suono". "Mettiamo conto: in combattimento c'è un VFX, c'è un suono, c'è un tirare fuori la spada, e quindi si sente la spada che esce fuori dal fodero": con una linea di Notify "questo suono avviene in un preciso momento dell'animazione". "Ovviamente, non andremo a usare il sound effect della piattaforma per il nostro Knight."

Le Montage Section. Nel *Montage group* c'è la sezione **Default**. Con il tasto destro, **New Montage Section**: il docente ne crea una chiamata **Attack**, nel punto in cui "lui sta attaccando". Le sezioni servono "per ordinare il lavoro" e per "dividere l'animazione in più punti diversi, con più animazioni".

Più animazioni nello stesso Montage. La striscia verde sotto le sezioni è l'animazione stessa. "Se vogliamo, possiamo anche introdurre delle animazioni in più." Una dimostrazione volutamente assurda: "io voglio, per qualche motivo arcano, far sì che quando finisce l'animazione lui salti": si trascina **MM_Jump** nella traccia, dopo l'attacco. "È un salto orribile, però farà le due animazioni di conseguenza." Il Montage è "molto importante per azioni consecutive". "Quando si parla con l'animatore, è importante il fatto di dire: questa animazione deve essere consecutiva a questa", per esempio "mi serve che da questo punto in poi continui l'attacco, oppure faccia un'altra animazione che ritorna in place".

L'animazione di ritorno. Finito l'attacco in root motion, il cavaliere resta dove l'attacco lo ha lasciato e non torna in posa: manca "un'animazione che ritorna in place", cioè in idle. Il docente la cerca su Mixamo, provando voci come *Sword and Shield Idle*, *Idle Transition* e *Start Block*; ma "è un po' difficile", perché "bisogna trovare il cavaliere proprio", con la stessa posa di fine attacco, altrimenti "snapperà malissimo, almeno nel mio caso".

Gli Slot. Sotto, c'è il **Default Group** con lo **Slot DefaultSlot**: "questo ci permette di dire: su quale slot vuoi che questa animazione si compia?". DefaultSlot "riprende tutto il corpo". "Potremmo, ad esempio, voler creare uno slot che riprenda solamente alcune zone del nostro character": "vogliamo che faccia un'azione solo nella parte superiore del corpo" (sparare o salutare mentre si corre); "in quel caso creeremo un altro slot, che prenderà solo le ossa" di quella parte. "Ma in questo caso ci serve tutto il corpo."

1.9.20 Lo Slot nell'AnimGraph: Save Cached Pose

Un Montage suona solo se l'AnimGraph ha un nodo **Slot** con lo stesso nome. In ABP_Knight, nell'AnimGraph:

1. dall'uscita della State Machine Locomotion si crea un **New Save Cached Pose**, chiamato **LocomotionCache**: "andiamo a conservarci quello che ha fatto la State Machine", "come se tutto questo procedimento lo mettesse in una memoria";
2. si aggiunge **Use cached pose 'LocomotionCache'**;
3. da lì un nodo **Slot 'DefaultSlot'** (la trascrizione: "default slot");
4. lo Slot va all'**Output Pose**.

"Semplicemente mixa insieme i due": lo slot del Montage e la cache della locomozione (figura 1.9.8). Quando non c'è nessun Montage in corso, lo Slot lascia passare la locomozione; quando un Montage su DefaultSlot parte, la sostituisce. Alla fine della lezione, a uno studente che non legge il nome del nodo ("vedo tutto pixelato"), il docente conferma: "tiri fuori Locomotion Cache", poi "default slot".

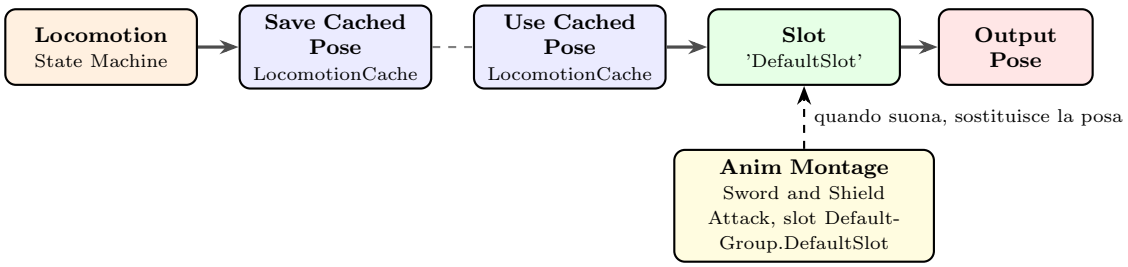


Figure 1.9.8: La catena di pose finale dell'AnimGraph di ABP_Knight. La State Machine produce la locomozione, che viene salvata in una cache e riletta; lo Slot DefaultSlot la lascia passare, a meno che un Montage assegnato a quello slot non sia in esecuzione. Schema ricostruito.

1.9.21 Far partire l'attacco: Play Montage

Il tasto F e il nodo. Per far partire il Montage, il docente non crea un'altra Input Action: usa direttamente un evento di tastiera, **F** ("facciamo con OK... no, facciamo F, F va benissimo"). Nel Blueprint del personaggio, il nodo **Play Montage**:

- **In Skeletal Mesh Component**: "in quale Skeletal Mesh Component farlo", cioè la **Mesh** del personaggio;
- **Montage to Play**: "quello che abbiamo appena fatto" ("ce n'è tanto solo uno");
- **Play Rate**: "possiamo modificare quanto è veloce l'animazione";
- fra le uscite: **On Completed** ("quando sarà finita"), **On Blend Out** ("mentre finisce"), **On Interrupted** ("se fosse interrotta"), **On Notify Begin** e **On Notify End** ("quando un Notify inizia" e "quando finisce").

"Se noi premiamo F, compiamo un Montage."

Fermare il personaggio. "È orribile": a parte la mancanza dell'animazione di ritorno, durante l'attacco "possiamo muovere il character tranquillamente". Quindi, sul tasto F, prima del Play Montage:

1. **Set Max Walk Speed** a **0**: "non dobbiamo farlo muovere";
2. **Get Player Controller** → **Disable Input**: "andiamo a disabilitare gli input, qualsiasi".

Sulle uscite del Play Montage:

- **On Completed**: Get Player Controller → **Enable Input** ("on completed solamente");
- **On Blend Out**: **Set Max Walk Speed** a **600**, "al suo massimo".

La camera che si muove ancora. In Play, però, "la camera si muove effettivamente", e con la camera il personaggio, "quindi dobbiamo far sì che non sia dipendente dalla camera". Segue una serie di tentativi, ragionati ad alta voce:

- togliere *Use Pawn Control Rotation* sullo Spring Arm ("no, no");
- l'*Enable Camera Lag* dello Spring Arm;
- le spunte *Inherit Pitch*, *Inherit Yaw* e *Inherit Roll* dello Spring Arm ("inerit yo e roll"; "no, lo yaw ci deve essere"; "forse sì, solo il roll va bene"), con un *Set* preso dallo Spring Arm ("Set Inherit Yaw", a false, e di nuovo a true alla fine); "completamente... non va";
- *Stop Movement Immediately* e i parametri di input del Player Controller (*movement input*, *look input*).

La soluzione, alla fine, è sulle due spunte già incontrate nella prima parte: "non è lo yaw" dello Spring Arm, "ma è l'orientamento al movimento". Sul tasto F si disattivano entrambe:

- dal **Character Movement**, **Set Orient Rotation to Movement** a **false**;
- sul Pawn stesso, senza collegare nessun target, **Set Use Controller Rotation Yaw** a **false**;

e sull'**On Completed** si rimettono entrambe a **true** (figura 1.9.9). "Ottimo. Adesso riusciamo": il cavaliere attacca fermo, la camera gira senza trascinarlo, e alla fine riprende la locomozione. "Abbiamo fatto il nostro character che attacca e fa la locomotion." (Il docente nota anche, di passaggio, che il cavaliere non interagisce con il pulsante del livello, "giustamente, perché è la Mesh"; dalla trascrizione non si ricostruisce meglio il punto.)

A Federico, che non riesce a sistemare la camera, il docente riassume: "devi giocare con lo yaw e con l'Orient Rotation to Movement: li disattivi tutte e due, e li riattivi sull'On Completed". "Dello Spring Arm, giusto?" "Non dello Spring Arm: questo qua è del Pawn, lo trovi direttamente senza collegare niente. Invece questo è solo del Character Movement."

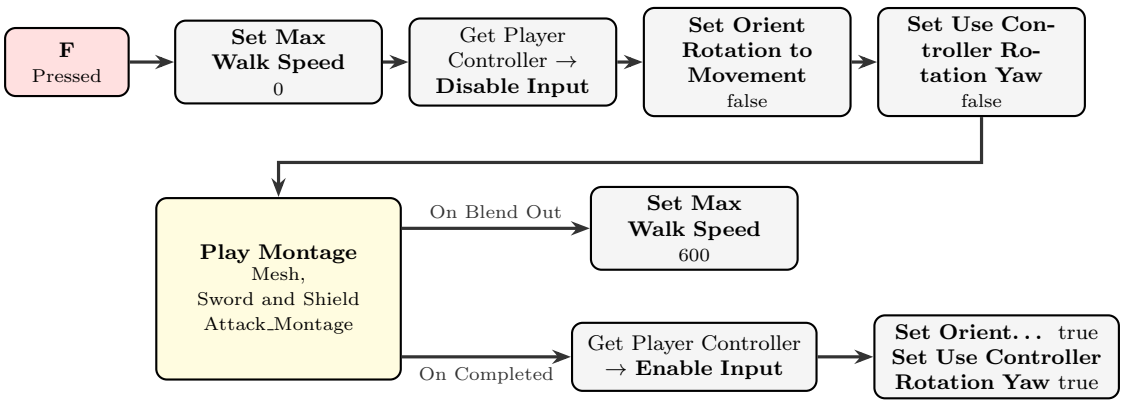


Figure 1.9.9: L'attacco sul tasto F, nella versione finale. Prima del Montage si blocca il personaggio (velocità zero, input disabilitato, nessuna rotazione automatica del corpo); al Blend Out torna la velocità, all'On Completed tornano l'input e le rotazioni. L'ordine esatto dei primi quattro nodi è ricostruito: a lezione vengono aggiunti uno alla volta, durante i tentativi. Schema ricostruito.

Approfondimento: Play Montage, Montage Play e il root motion. Il nodo usato a lezione, **Play Montage**, è un nodo *asincrono* (con l'orologio nell'angolo): le sue uscite scattano più tardi, quando succede l'evento corrispondente, e per questo può gestire il "dopo" del Montage direttamente nel grafo. Esiste anche *Montage Play*, chiamato sull'*Anim Instance* della mesh, che restituisce solo la durata, e *Play Anim Montage*, del Character. Due dettagli completano la lezione. Il primo: un attacco in **root motion** sposta davvero la capsula solo se il Montage ha il root motion attivo e l'Animation Blueprint ha la *Root Motion Mode* impostata su *Root Motion from Montages Only* (il default); se il root dell'animazione di Mixamo è "rotto", come visto nel retargeting, il personaggio avanza nella mesh e poi "torna indietro" di scatto a fine animazione. Il secondo: durante un Montage che fa avanzare il personaggio in root motion non serve azzerare la Max Walk Speed, perché il root motion prende il posto dell'input di movimento; azzerarla, come fa il docente, è però un modo semplice e sicuro di impedire al giocatore di muoversi durante un attacco "in place". Infine, *Disable Input* e *Enable Input* agiscono sull'actor e richiedono un Player Controller: per bloccare solo alcune azioni (il movimento ma non la camera), l'Enhanced Input offre un'alternativa più fine, cioè rimuovere temporaneamente un Input Mapping Context, o aggiungerne uno di priorità più alta.

1.9.22 Il compito e la chiusura

Feedback in chat. Una studentessa, Giulia, ha scritto in chat che è rimasta un po' indietro "causa trasloco" e chiede di poter mandare dei feedback; nel frattempo è già uscita dalla chiamata. "Le risponderò sul Discord, assolutamente." "Controllerò i lavori che mandate sul canale."

Il compito: lo StackOBot. "Una cosa che vorrei darvi come task, se vi va, per la prossima volta": "fare un nuovo character totalmente da zero". Nel progetto c'è già un personaggio, lo **StackOBot** (il robottino dei progetti di esempio di Epic), che "ha queste animazioni qua: vedete che ce le ha già dentro tutte quante". Lo si trova nella cartella *StackOBot* → *Blueprints* → *Character*, con la sua Skeletal Mesh (*SKM_Bot*) e il suo ABP. Il compito:

- creare un **Blueprint Character** per lo StackOBot ("fare la BP");
- creare il suo **Animation Blueprint** da zero, "fare tutto l'Animation Blueprint e collegarci tutte le sue animazioni": il ciclo di locomozione, con State Machine e Blend Space, come quello del cavaliere.

"Se volete farlo, lo potete mandare poi sulla chat", "che tanto magari ci lavoreremo la prossima volta". "Per adesso, poi ne riparliamo."

I saluti. "Se tutto a posto..." Il docente interrompe la condivisione: "vi ringrazio per l'attenzione, e ringrazio da casa chi è in differita". "Ci sentiamo per martedì prossimo. Va bene, ciao, ragazzi. Buona serata."

Punti chiave del capitolo

- Una **State Machine** nell'AnimGraph organizza le animazioni in **stati** (WalkRun, Jump, Fall), ciascuno con la sua **Output Animation Pose**, e in **transizioni** con regole booleane: un "loop di animazione secondo delle condizioni". Va collegata all'Output Pose.
- Un **Blend Space 1D** mescola animazioni lungo un asse: per la locomozione, la **GroundSpeed**. I campioni vanno posti alle velocità reali del personaggio (Max Walk Speed, in centimetri al secondo): Idle a 0, Walk a 300, corsa a 600. Valori intermedi "inventati" producono obbrobri.
- Un **Character** non ha la camera: si aggiungono uno **Spring Arm** e, come suo figlio, una **Camera**. Distanza e altezza si regolano sullo Spring Arm (**Target Arm Length**, **Socket Offset**), mai spostando la camera; il **Target Offset** solo sulla Z e poco. Sono il lavoro dei **3C**: Camera, Control, Character.
- Tre spunte governano la rotazione: **Use Pawn Control Rotation** sullo Spring Arm (la camera segue il mouse), **Use Controller Rotation Yaw** sul Pawn (il corpo segue la camera), **Orient Rotation to Movement** nel Character Movement (il corpo si gira verso il movimento).

Sintomo	**Causa e rimedio**
L'animazione dello stato non si vede nella preview	la State Machine non è collegata all'Output Pose dell'AnimGraph
L'animazione si ferma dopo un ciclo	manca la spunta di loop nel nodo dell'animazione
Warning nella State Machine	transizioni senza condizioni: normale finché non si scrivono le regole
Jump → Fall dà errore	regola vuota: attivare Automatic Rule Based on Sequence Player in State
Il Blend Space 1D arriva solo a 100	Maximum Axis Value da impostare a 600; poi spostare i campioni
Animazioni "mischiate" a metà (a 200)	i campioni non corrispondono alle velocità reali: legarli alla Max Walk Speed
Alt sul nodo non crea il Set	negli stati dell'AnimGraph le variabili si leggono e basta
"Questo asset non ha schede ancorate"	Window → Load Layout → Default Editor Layout, poi Open Full Blueprint Editor
In Play non si vede niente / visuale nel corpo	il Character non ha camera: aggiungere Spring Arm e Camera (figlia)
Non ci si muove	mancano gli input: copiare dal ThirdPersonCharacter i grafi di Look, Move, Jump e l'Add Mapping Context
La camera ruota su un solo asse, o guarda solo il terreno	Use Pawn Control Rotation sulla Camera invece che sullo Spring Arm
Il personaggio va di lato guardando avanti	attivare Orient Rotation to Movement e togliere Use Controller Rotation Yaw (servono tutte e due)
La camera compenetra gli oggetti	Target Offset troppo grande: usare il Socket Offset
Il salto in root motion retargettato non è pulito	il root degli scheletri di Mixamo: accettabile per la prova, da correggere
La State Machine non distingue salto e caduta	entrambe le regole su IsFalling: aggiungere VelocityZ maggiore o minore di zero
Lo Shift non fa correre	IA_Run non aggiunta ai mapping di IMC_Default
FInterp To non interpola	eseguito una volta sola: va eseguito a ogni frame (Tick, timer o Timeline)
Direction quasi sempre a zero	Orient Rotation to Movement acceso, Use Controller Rotation Yaw spento: invertire per il Blend Space 2D
Scatti nel Blend Space 2D	animazioni non omogenee, salto fra 180 e -180: Smoothing Time, o riscaricare le animazioni
Durante l'attacco il personaggio si muove	Set Max Walk Speed a 0 e Disable Input prima del Play Montage
Durante l'attacco la camera trascina il personaggio	Set Orient Rotation to Movement e Set Use Controller Rotation Yaw a false, a true sull'On Completed
Il Montage non suona	manca il nodo Slot DefaultSlot nell'AnimGraph

Table 1.9.5: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, asset o impostazione	Che cosa fa
State Machine (AnimGraph)	organizza le animazioni in stati e transizioni
Entry / State / Conduit / Alias	ingresso; stato con una posa; snodo senza posa; alias di più stati
Output Animation Pose	la posa prodotta da uno stato
Transition Rule (Result)	la condizione booleana per passare da uno stato all'altro
Automatic Rule Based on Sequence Player in State	la transizione scatta a fine animazione
NOT Boolean / AND Boolean	negazione; congiunzione di due condizioni
Split Struct Pin	divide un vettore nei suoi componenti X, Y, Z
Blend Space 1D / Blend Space (2D)	mescola animazioni su uno o due assi
Horizontal / Vertical Axis, Grid Divisions	nome, minimo e massimo degli assi; divisioni della griglia
Smoothing Time	tempo di passaggio fra i campioni
Blend Space Player	il nodo che suona un Blend Space nell'AnimGraph
Calculate Direction	l'angolo, da -180 a 180, fra la velocità e la rotazione
Get Actor Rotation / Get Velocity	la rotazione dell'actor; la sua velocità
Default Pawn Class (World Settings)	il personaggio del giocatore nel livello
Spring Arm: Target Arm Length, Socket Offset, Target Offset	lunghezza del braccio; spostamento in fondo al braccio; spostamento dell'origine
Camera: Field of View	angolo di visuale
Use Pawn Control Rotation (Spring Arm)	la camera segue il controller
Use Controller Rotation Yaw (Pawn)	il corpo segue la rotazione del controller
Orient Rotation to Movement, Rotation Rate (Character Movement)	il corpo si gira verso il movimento, con quella velocità
Max Walk Speed / Set Max Walk Speed	velocità massima a piedi (cm/s)
Retarget Animations, Export Animations	trasferisce un'animazione su un altro scheletro
Input Mapping Context (IMC)	associa i tasti alle azioni in un contesto
Input Action (IA): Value Type, Triggers	l'azione, il tipo di valore, quando scatta
Add Mapping Context	attiva un IMC per il giocatore
Evento IA: Triggered, Started, Ongoing, Canceled, Completed	le fasi di un'azione
FInterp To	un passo di interpolazione fra Current e Target
Get World Delta Seconds	il tempo dell'ultimo frame
Animation Montage	animazione "una tantum", modulare, con sezioni e slot
Notify / Play Sound / Notify Track	eventi sulla linea del tempo; un suono; una traccia di eventi
Montage Section	una parte con nome del Montage
Slot (DefaultGroup.DefaultSlot)	dove il Montage si applica (tutto il corpo, o parti)
Save Cached Pose / Use Cached Pose	salva e rilegge una posa nell'AnimGraph
Play Montage	suona un Montage; On Completed, On Blend Out, On Interrupted, Notify Begin/End
Disable Input / Enable Input	toglie e restituisce l'input all'actor

Table 1.9.6: I nodi, gli asset e le impostazioni della nona lezione.

- Il **retargeting** porta un'animazione su un altro scheletro (Retarget Animations, Export Animations): comodo, ma il risultato dipende dall'animazione, e il root di Mixamo dà problemi.
- Fra salto e caduta: **Automatic Rule Based on Sequence Player in State** per passare a fine animazione, **NOT IsFalling** per tornare a terra, e la **velocità sulla Z** per distinguere un salto ($Z > 0$) da una caduta ($Z < 0$).
- L'**Enhanced Input** separa i tasti dalle azioni: l'**Input Action** (IA_Run, Digital bool, con i suoi Triggers) è associata a uno o più tasti (Shift, il cerchio del controller) in un **Input Mapping Context**, che va attivato con **Add Mapping Context**. Lo sprint: Set Max Walk Speed a 600 sul Triggered, a 300 sul Completed.
- **FInterp To** fa un solo passo per esecuzione: per un passaggio graduale va eseguito a ogni frame (Tick, timer, Timeline).
- Il **Blend Space 2D** aggiunge la **direzione** (**Calculate Direction**, da -180 a 180: positivo a destra, negativo a sinistra, ± 180 indietro): otto animazioni a 600, l'Idle ripetuto a 0, la corsa all'indietro ai due estremi, griglia 8×8, Smoothing Time. Serve Use Controller Rotation Yaw acceso.
- Un **Animation Montage** è un'animazione "una tantum" e modulare, con **Notify** (suoni, VFX in un punto preciso), **sezioni** e **slot**. Per suonarlo, nell'AnimGraph: State Machine, **Save Cached Pose**, Use Cached Pose, **Slot DefaultSlot**, Output Pose. Si fa partire con **Play Montage**, bloccando prima movimento, input e rotazioni e restituendoli su On Blend Out e On Completed.
- Compito facoltativo: un nuovo personaggio da zero con lo **StackOBot**, Blueprint e Animation Blueprint completi.

Chapter 1.10

Il nuovo personaggio: Lean, State Machine e passi con i Physical Material

Questa è la **decima lezione** del corso breve di **introduzione a Unreal Engine**. Alla fine della nona il docente aveva lasciato un compito facoltativo: costruire da zero un personaggio nuovo con lo **StackOBot**, il robottino dei progetti di esempio di Epic, che ha già tutte le sue animazioni. Il compito era "molto facoltativo, perché capisco che è complicato"; in questa lezione lo si fa insieme, dall'inizio alla fine: "faremo proprio un nuovo character, che useremo poi come default".

Il nuovo personaggio riprende la struttura del cavaliere della lezione precedente (Animation Blueprint, State Machine, Blend Space), ma con tre differenze che danno il titolo al capitolo. La prima è il **Lean**: il bot non ha otto animazioni direzionali, ma una corsa e due varianti *inclinate*, verso sinistra e verso destra; il Blend Space 2D non usa più la direzione, ma un valore d'inclinazione calcolato a ogni frame dalla differenza fra la rotazione attuale e quella del frame precedente. La seconda è una **State Machine** più ricca, a sette stati, che distingue il salto da fermo dal salto in corsa e l'atterraggio da fermo dall'atterraggio in movimento, con una nuova variabile che dice se il giocatore sta premendo un tasto di movimento. La terza è il suono dei **passi**: un **Anim Notify** inserito nelle animazioni, nel momento esatto in cui il piede tocca terra, lancia un *line trace* verso il basso, legge il tipo di superficie colpita e suona il passo giusto; le superfici si distinguono con i **Physical Material**, "i materiali che possono determinare uno stato fisico" di un oggetto. Sulla roccia si sente la roccia, sull'acqua l'acqua, altrove il passo normale.

La lezione è **online**, su Zoom, come le precedenti. Si apre con una lunga correzione in diretta del progetto di uno studente, prosegue con la costruzione del bot, interrotta più volte dalle domande e dai problemi dei presenti, risolti in condivisione di schermo, e si chiude sui passi e i Physical Material. La pausa arriva dopo la variabile dell'input ("ci vediamo alle solite 18. Scusate: 17.55"); alla fine, "vi mando a cena". È una lezione di *fine corso*: "mancano tre lezioni alla fine", e il docente lo ricorda per invitare chi segue in differita a entrare in diretta, se vuole correzioni dal vivo.

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Arreal", "ArRIAL", "a Real" e "Arriva"; il **Lean** è "Lin", "Line", "l'in", "LIN" e "Linn" ("l'in è il piegamento della rotazione"); l'**Idle** è "idol" e "Idol"; l'**Event Graph** è "even graph", "Even Graph" e "Vengraph"; l'**AnimGraph** è "Alimgraph", "aningraf" e "Aningraft"; **IA_Move** è "IAMove", "iam move", "Iammov" e "gli ammove"; **IA_Run** è "iaran", "IRRun" e "iarran"; l'**IMC** è "IAMC" e "negli MC"; il **Then 0** della Sequence è "DANZIRO" e "den zero"; i **Reroute Node** sono "re Routenold"; il **Blend Space** è "Benzes" e "Brennese"; il **Validated Get** è "validate get"; l'**FInterp To** è "F inter2" e "fintech 2"; l'**RInterp To** è "R inter to"; l'**Is Falling** è "his falling" e "ease falling"; il **Last Input Vector** è "Lasting Pwn Vector"; lo **Spring Arm** è "Springarma" e "spring guard"; **Use Pawn Control Rotation** è "PoundCtrlRotation", "Pwn rotation" e "use spawn control rotation"; le **World Settings** sono "Word Settings" e la **Default Pawn Class** è "default on class"; l'**Anim Notify** è "an in notify", "Anim Notifaj", "modify", "anni nota" e "anni notifà"; **AN_Footstep** è "a n footstep" e "AN

full step"; il **Physical Material** è "fisic material" e "P FIS materia"; **Get Surface Type** è "Gethsurfest Type" e "get sur festival"; il **Sound Cue** è "queue" e "Q"; il **Draw Debug Type** è "drop the bug type" e "draw the bug type"; il **Break Hit Result** è "break it risult"; **BP_Knight** è "BP Night" e "Night"; il **bot** è a tratti "Boto" e "voto". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione e durante la pausa il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?", "la situazione è questa, la migliore dal 2005", il "rinforzare la democrazia", la "dottrina sul mercato del lavoro", il "dialogo interculturale"): non appartengono alla lezione e sono stati ignorati. La terza: buona parte della lezione è fatta di indicazioni a voce mentre si lavora ("vai su", "clicca qua", "attacca quello sotto"), spesso rivolte allo schermo di uno studente; il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, distingue i tentativi dalle soluzioni, completa i nomi esatti dei nodi e dei pin di Unreal Engine 5, e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono il movimento riparato con l'Enhanced Input, i due eventi dell'Animation Blueprint del bot con il Validated Get, il calcolo del Lean, il Blend Space 2D del bot, la State Machine a sette stati, il grafo dell'Anim Notify dei passi, la catena che porta dalla superficie al suono, il Sound Cue con Random e Modulator, e la linea del tempo dell'animazione di corsa con i Sync Marker e i Notify. Le tabelle riassumono le transizioni della State Machine, i valori dei Sound Cue, gli errori incontrati durante la lezione e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'**inizio della lezione**: il movimento e la camera riparati in diretta (sezione 1.10.1);
2. gli **annunci** e le domande: i passi e i Physical Material, il compito facoltativo, il personaggio nuovo che non interagisce con il livello, la lezione di giovedì, il bot di Federico (sezioni 1.10.2 e 1.10.3);
3. il **risultato finale**, mostrato in anteprima (sezione 1.10.4);
4. il nuovo personaggio: **Animation Blueprint** e **Blueprint Character**, l'Event Graph copiato, l'**Initialize** e il **Validated Get**, commenti e Reroute (sezioni da 1.10.5 a 1.10.7);
5. il **Blend Space 2D** del bot e il calcolo del **Lean** (sezioni 1.10.8 e 1.10.9);
6. la **State Machine** a sette stati, la variabile dell'**input** e le regole di transizione, prima e dopo la pausa (sezioni da 1.10.10 a 1.10.12);
7. il bot **in gioco**, con la camera, e il debug dei progetti degli studenti (sezioni 1.10.13 e 1.10.14);
8. i **passi**: l'**Anim Notify**, il *line trace*, i **Physical Surface**, i **Sound Cue**, i **Notify** nelle animazioni e i **Physical Material** (sezioni da 1.10.15 a 1.10.19);
9. gli ultimi problemi e la **chiusura** (sezione 1.10.20).

1.10.1 Prima della lezione: il movimento da riparare

Dopo i minuti iniziali di rumore della trascrizione, i saluti ("buongiorno, buon pomeriggio") e una prima richiesta. Mentre gli altri "stanno ancora entrando", uno studente racconta il suo problema: ha seguito la lezione precedente in differita ("ho seguito la lezione da quando è stata salvata"), e all'inizio "andava tutto, si muoveva"; poi, a un certo punto, il personaggio non si muoveva più, e "la camera, anche se segnata, non c'è, ed è sparito tutto". Il docente propone di risolverlo subito ("tanto adesso stanno ancora entrando") e la correzione si fa in condivisione di schermo. È una sequenza lunga e caotica, ma istruttiva: mostra come si legge un errore di compilazione e come si ricostruisce un pezzo di Enhanced Input perduto.

Leggere gli errori. "Già ti dico che comunque possiamo già vedere gli errori là sotto": il pannello dei risultati della compilazione (*Compiler Results*) in fondo all'editor del Blueprint elenca i nodi che non trovano più ciò a cui si riferiscono. La prima diagnosi, a colpo d'occhio: "sicuramente ti manca l'*Input Action*". L'errore dice che l'evento di **IA_Run** non esiste più nel nodo, o che il contesto **IMC_Default** non lo contiene: "dice che fondamentalmente non hai IA_Run: l'hai cancellato per sbaglio oppure non l'hai mai creato?". In realtà IA_Run c'è ("il Content Browser, fai IA_Run, ed eccolo là"): il problema è un altro.

IA_Move cancellata. Aprendo l'**IMC_Default** ("scrivi IMC, invece di IAMC"), il docente trova dei mapping che lo studente non ricorda di aver messo ("ma no, non li ho messi io"): li si cancella tutti con il cestino, poi si torna indietro con **Ctrl+Z** ("fai Ctrl+Z a tutti, va benissimo"). Guardando il mapping che dovrebbe essere **IA_Move** si capisce: "hai cancellato IA_Move, fondamentalmente", e al suo posto, nell'Event Graph, l'evento del movimento è stato sostituito da IA_Run ("ah, perché l'hai sostituito"). "E devi stare attento a queste cose."

Ricreare IA_Move. "La cosa che devi fare adesso è praticamente ricreare IA_Move." Il primo istinto è duplicare IA_Run; poi il docente si corregge: "no, no, no: fai un doppione di **IA_Look**". Il motivo è il tipo di valore: IA_Run è un booleano (*Digital*), mentre il movimento, come lo sguardo, è un **Vector 2D** ("è un Vector 2D"): duplicare IA_Look dà un'azione già del tipo giusto. Il doppione si rinomina **IA_Move**. Poi, nell'IMC_Default:

- si rimette IA_Move nel mapping che lo usava prima ("adesso segnali negli MC quello di prima, IA_Move");
- con il **più** in alto si crea un altro mapping, per **IA_Run**: con la freccina sotto il nome si apre la scelta del tasto e, invece di cercarlo nella lista, "basta che clicchi sulla tastiera: clicca Shift sulla tua tastiera". Il campo del tasto, quando è in ascolto, registra il primo tasto premuto.

Una compilazione di prova: "qualche cosa si è levato, benissimo".

L'evento e gli Add Movement Input. Restano gli errori nell'Event Graph del personaggio (il cavaliere, "Knight"): "hai sostituito IA_Move con IA_Run: devi levare questo e mettere IA_Move"; con il tasto destro si ricrea l'evento **IA_Move**. Uno strano dettaglio ("strano che non ti fa vedere") si risolve salvando l'azione: solo dopo il salvataggio l'evento mostra l'**Action Value** come vettore 2D. Si ricostruisce il grafo del movimento, quello copiato la volta precedente dal ThirdPersonCharacter:

1. tasto destro sull'**Action Value** e **Split Struct Pin**, per avere i pin **X** e **Y** separati;
2. la **X** si collega al primo **Add Movement Input**, la **Y** "tutto dritto fino a Scale Value del successivo";
3. il secondo Add Movement Input era stato cancellato ("ah, non c'è"): lo si ricopia dal primo ("copia Add Movement Input");
4. nella **World Direction** si collega "quel forward vector che c'è lì vicino, quello arancione" (il vettore di direzione calcolato dalla rotazione del controller);
5. infine il pin di esecuzione: dal **Triggered** di IA_Move, "attacchi tutto in sequenza".

"E abbiamo riparato il movimento" (figura 1.10.1). Compilato, il personaggio si muove.

Perché "sparisce tutto". Lo studente aveva notato che a volte il personaggio compariva e a volte, dopo una compilazione, "proprio scompariva tutto". "È normale": quando la compilazione di un Blueprint fallisce, Unreal "te lo fa capire in modo molto più palese": l'icona del pulsante *Compile* diventa rossa, con un simbolo d'errore ("un'iconcina"), e un Blueprint che non compila non si comporta come dovrebbe in gioco.

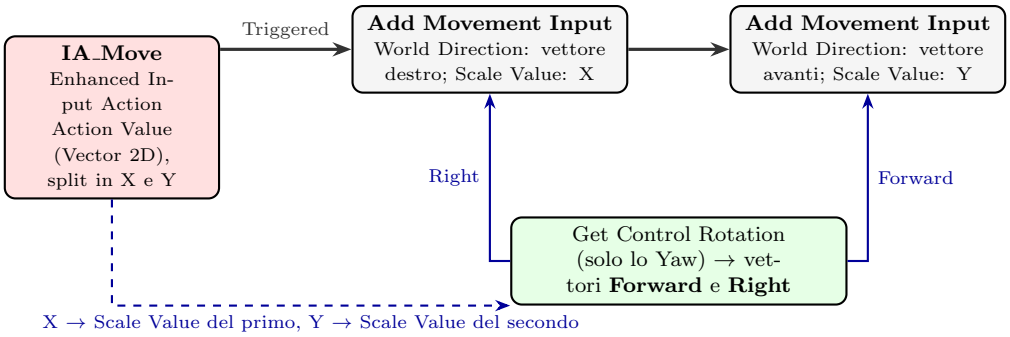


Figure 1.10.1: Il movimento riparato. L'Action Value di IA_Move, un Vector 2D, viene diviso in X e Y: la X muove lungo il vettore destro, la Y lungo quello in avanti, entrambi ricavati dalla rotazione del controller. Se IA_Move viene cancellata o sostituita da un'azione booleana (IA_Run), l'Action Value non è più un vettore e il grafo si rompe. Schema ricostruito dal grafo standard del ThirdPersonCharacter; l'ordine esatto delle due direzioni a lezione non si sente.

La camera storta. In Play il movimento funziona, ma "è un po' strano", e la camera è "molto storta". Tre correzioni, nell'ordine:

1. **Use Pawn Control Rotation**: selezionato lo **Spring Arm**, si cerca "control rotation" e si mette la spunta ("mettilo true, perfetto"): la telecamera segue il controller.

2. **La gerarchia**: la camera era stata agganciata alla *freccia* (l'*Arrow Component* del Character): "attenzione a fare queste cose". Lo Spring Arm va staccato dalla freccia e attaccato al suo posto; la Camera "deve rimanere" figlia dello Spring Arm.

3. **La posizione**: lo Spring Arm era stato spostato con il *pivot* ("l'hai spostato sulla location"): "devi mettere tutto a zero". Accanto ai valori della Location, nei Details, c'è una freccina circolare tutta a destra che li riporta al default.

La regola, ripetuta: lo Spring Arm e la Camera non si muovono dal pivot, "perché se no si rompe tutto". Per spostare la camera si usano solo i valori dello Spring Arm: **Target Arm Length**, **Socket Offset** e **Target Offset**. "Se muovi il pivot, ti cambia la location del braccio, quindi ti fa tutto un effetto strano anche con la camera." Siccome lo studente voleva una visuale più alta: **Target Offset**, Z ("l'ultima a destra") a **60**. "E già la telecamera dovrebbe essersi fissata più o meno": "abbastanza bene".

Le animazioni del giorno prima. Lo studente aveva fatto anche "la roba delle animazioni" ("oddio, è un po' buggato"), ma senza poterla provare, perché il personaggio era fermo. "Tanto oggi la riprenderemo": si rifarà un personaggio "proprio da capo".

1.10.2 Gli annunci e le domande

La cartella Footstep Sounds. "Oggi andremo a vedere anche una cosettina interessante." Il docente l'ha scritto su Discord: nella cartella del materiale didattico, su Drive, c'è una cartella **Footstep Sounds**; va scaricata e decompressa, "lasciatela là", servirà più tardi. Servirà per i **Physical Material**, "cioè praticamente i materiali che possono determinare uno stato fisico, ad esempio di un oggetto": "se un oggetto è di metallo, come deve reagire in certi ambiti". Oggi li si usa per i passi: "quando lui fa superficie acquosa si sentirà il rumore dei passi dell'acqua, se no si sentirà il rumore dei passi della roccia, o si sentirà il rumore dei passi normale: dipende dalla superficie in cui ci troviamo".

Le correzioni in diretta. "Visto che tanto oggi andiamo diretti in editor": chi ha problemi con le lezioni passate e vuole "delle correzioni un po' in live, rispetto a che magari in post", ha ancora "questa lezione, la

prossima e la prossima ancora": "mancano tre lezioni alla fine". L'invito vale anche per chi segue in differita: "entri, se vuole delle correzioni direttamente in live".

Il compito sul bot. Uno studente chiede se il bot andava fatto "per oggi", "perché io non so neanche dove mettere le mani". Il docente ripete che era un compito "per i più coraggiosi", "molto facoltativo", "per chi magari gli piace smanettare": "tanto oggi lo faremo". E, a scanso di equivoci: "non vi metto un voto, state tranquilli".

Il personaggio nuovo che non interagisce. Lo stesso studente ha due problemi con il suo personaggio. Il primo: "corre, ma non ha l'animazione della corsa". "La prima probabilmente sarà un errore che andremo a vedere insieme." Il secondo è più interessante: il personaggio nuovo "non interagisce più con molte delle cose che abbiamo fatto nelle lezioni passate". Il bottone per chiamare l'ascensore funziona, ma salire sull'ascensore no; l'ascensore "quello col suono" sì; la torretta no. Il docente spiega il caso della torretta, ed è un principio generale: quando si cambia personaggio, tutto ciò che nel livello riconosceva il vecchio personaggio va ricontrollato.

- **Il tag**: "la torretta funziona tramite il tag, come gliel'abbiamo fatta". Nelle lezioni precedenti la torretta cercava un actor con il tag **Player**: "quindi, se cambiamo character, dobbiamo rimettergli il tag", cioè aggiungere *Player* ai *Tags* del nuovo Blueprint. "Guarda che questo è il player: segui lui."
- **Le collisioni**: il nuovo personaggio deve rispondere "ai trace che abbiamo fatto" (la torretta *vede* il giocatore con un trace) e alle collisioni "che sono, ad esempio, la sfera" (la sfera che fa scattare la torretta quando il giocatore entra nel suo raggio).

"Intanto lo faremo oggi": "faremo un nuovo character e andremo poi a risistemare un po' tutto".

La lezione di giovedì. "Piccola postilla": i **menu**, che erano rimasti "un po' un buco aperto", si faranno come lezione **opzionale giovedì**; poi ci sarà "un'ultima lezione finale con un'ultima cosa". Ma "se volete proporre anche qualche argomento che vi interessa, fatelo un po' oggi": se gli studenti si mettono d'accordo, "facciamo quello"; "se no, andiamo con i menu tranquillamente".

Approfondimento: tag, collisioni e personaggi intercambiabili. Il problema dello studente è tipico di ogni progetto in cui il personaggio cambia. Le soluzioni viste nelle lezioni precedenti riconoscono il giocatore in tre modi diversi, e ciascuno si rompe in modo diverso: per *tag* (*Actor Has Tag*: basta aggiungere il tag al nuovo Blueprint, nella sezione *Actor* → *Tags* dei Details); per *cast* a una classe precisa (un *Cast To BP_Knight* fallisce con qualunque altro personaggio: è il motivo per cui conviene fare il cast alla classe più generale che serve, *Character* o *Pawn*, o usare un'interfaccia); per *collisione* (il nuovo personaggio ha la sua capsula e i suoi *Collision Presets*: se i canali o gli Object Type non corrispondono a quelli che il trace o l'overlap si aspettano, non succede niente). Un ascensore che porta su il vecchio personaggio ma non il nuovo è quasi sempre un problema del secondo o del terzo tipo. Romero e Sewell (capitolo 14) ricordano che "actor e componenti devono definire come reagiscono a ogni canale di trace e a ogni object type": *Ignore*, *Overlap* o *Block*.

1.10.3 Il bot di Federico

"Ci sono altre domande, ragazzi? Altri problemi, cose che serve fixare?" Federico il bot l'ha già fatto, e si è anche "anticipato sui tempi", con un tasto per interagire ("vedo che ti sei anche anticipato sui tempi, effettivamente, sull'interactable"). Il problema: premendo **F** il personaggio si ferma, si disabilita l'input e dovrebbe partire un **Animation Montage**, ma l'animazione non si vede: "posso solo girarmi".

Il DefaultSlot che manca. La diagnosi è la stessa della lezione precedente. Nell'AnimGraph dell'ABP del bot ("ABP? L'Animation Blueprint"): "devi mettere il **DefaultSlot**", "quel passaggio che serviva praticamente per mixare le due animazioni". Fra la State Machine e l'Output Pose va il nodo **Slot** (*DefaultSlot*): "vai dentro, dentro, e poi fuori". Compilato, l'animazione parte: "perfetto, ottimo".

Il personaggio che gira su se stesso. Durante l'animazione, però, il bot "si gira". Una prima correzione: nel Blueprint del bot, selezionato il componente in cima alla lista ("schiaccia proprio sulle componenti, tutto a sinistra", cioè il Blueprint stesso o il suo Character Movement), si cerca "rotation" e si toglie **Use Controller Desired Rotation** ("fai leva use control desired rotation"). Ma il problema vero è un altro, e il docente lo rimanda: "quello fondamentalmente lo possiamo fare con l'*upper slot*, che poi magari lo possiamo vedere". "È normale che lui giri su se stesso, perché il DefaultSlot prende tutte le ossa del corpo": il Montage sostituisce la posa dell'intero scheletro, gambe comprese, mentre il personaggio continua a muoversi e a ruotare con la camera ("io lo muovo col WASD"; "però si muove comunque con la telecamera"). "Il fatto è che bisogna mixare le due animazioni": la parte alta del corpo dal Montage, quella bassa dalla locomozione. "Poi adesso ti faccio vedere magari come fare le animazioni."

Approfondimento: l'upper slot e il Layered Blend per Bone. Lo slot di cui parla il docente è la tecnica standard per animare solo metà del corpo. Si crea un nuovo slot (per esempio *UpperBody*) nel *Anim Slot Manager* dello scheletro, si assegna il Montage a quello slot invece che al DefaultSlot, e nell'AnimGraph si usa un nodo **Layered blend per bone**: la posa base (la locomozione) entra nel primo ingresso, la posa con lo slot nel secondo, e nei Details del nodo si indica l'osso da cui la seconda posa comincia a valere (di solito la colonna, *spine_01* o equivalente), con un *Blend Depth*. Il risultato è un personaggio che attacca o interagisce con le braccia mentre le gambe continuano a camminare. Il resto del fenomeno, il corpo che ruota con la camera, dipende dalle tre spunte di rotazione già viste nella lezione precedente (Use Pawn Control Rotation, Use Controller Rotation Yaw, Orient Rotation to Movement) e da *Use Controller Desired Rotation*, che fa ruotare il personaggio verso la rotazione del controller con la velocità indicata in *Rotation Rate*.

1.10.4 Il risultato finale, in anteprima

"Allora, ragazzi, oggi andremo a fare questo bellissimo bot." Prima un controllo sull'audio ("non sentite l'audio? Datemi un feedback"): bisogna condividere anche l'audio del computer, in Zoom. Poi la dimostrazione: il bot del docente cammina sul livello e, "se voi fate attenzione, ci sta un piccolo trace sotto, che appare ai piedi del nostro bot". "E praticamente va a identificare, a ogni passo, la tipologia di materiale che lui sta toccando in questo momento." Sul pavimento normale si sente il passo normale; sopra una pedana blu ("questa è l'acqua") a ogni passo si sente l'acqua; "allo stesso tempo anche la roccia". "Questi qua sono fatti con i Physics Material."

Dove si trova il Physical Material. "Se voi cliccate su qualsiasi asset" nel livello, per esempio una Static Mesh, e nella barra di ricerca dei Details scrivete "phys", trovate **Phys Material Override** ("lo accorcia: Phys Materia Override"): lì il docente ha messo un asset che ha creato, **PM_Rock**, "Physic Material Rock". "Possiamo differenziare le tipologie di superfici tramite una cosa di Unreal che ci permette di dire: guarda, puoi creare tutta una varietà di superfici, ognuna ha una sua regola, volendo, su come reagisce a tutto quanto." Il caso di oggi è "la cosa più semplice": "guarda, questa è la roccia: quando stai sulla roccia fammi questo suono; se invece stai sull'acqua, fammi quest'altro suono".

"Come facciamo? Allora, intanto andiamo a fare il nostro character, innanzitutto."

1.10.5 Il nuovo Animation Blueprint e il nuovo Character

La cartella e l'ABP. Nella cartella dei personaggi si crea una nuova cartella per il bot (il docente aggiunge un "2" a tutti i nomi, perché il suo progetto ha già un bot: "glielo chiamerò 2"). Il primo asset è l'**Animation Blueprint**: alla creazione Unreal chiede lo **scheletro**, e "a noi ci interessa" quello del bot, lo scheletro della Skeletal Mesh del bot (*SK_Bot*, nella trascrizione "character SH bot" e "scheda dal mesh del bot"), "come abbiamo visto e come abbiamo creato anche gli altri character". Lo si chiama **ABP_Bot2** e lo si apre.

Copiare l'Event Graph. "Anche senza fare troppe cose, perché tanto l'abbiamo già fatte": si apre **ABP_Knight**, l'Animation Blueprint del cavaliere della lezione precedente (o quello del proprio personaggio), e si va nel suo **Event Graph**. La parte dell'**Update** "può essere tranquillamente uguale. Perché cosa

fa? Questi sono semplicemente calcoli": la GroundSpeed, se sta cadendo, la direzione. Quindi la si seleziona, **Ctrl+C**, e la si incolla con **Ctrl+V** nell'Event Graph di ABP_Bot2. La parte dell'**Initialize**, invece, no: "bisognerà sicuramente fare la stessa cosa, ma non copiamo questo", perché fa il cast a una classe precisa, il cavaliere, e il bot ha bisogno di un Blueprint suo, che ancora non esiste. L'AnimGraph, anche lui, "lo rifaremo per forza, perché sarà totalmente diverso".

BP_Bot2. Nella stessa cartella: **Blueprint Class**, **Character**, nome **BP_Bot2**. Nel Blueprint, selezionato il componente **Mesh**:

- **Skeletal Mesh Asset**: la Skeletal Mesh del bot;
- la mesh si trascina "un po' più in basso", fino ai piedi della capsula, e la si ruota di 90 gradi sulla Z, perché guardi nella direzione della freccia (la posizione esatta, $\mathbf{Z} = -90$, la si fisserà più avanti, quando il personaggio "levita");
- **Anim Class**: **ABP_Bot2**, "in modo tale che lui poi prenda le animazioni, il ragionamento che andrà a fare".

A uno studente che non trova dove si collega l'Animation Blueprint: "se tu clicchi sulla Mesh, devi collegargli l'Animation Blueprint", nel campo *Animation Class* dei Details. "Fatto. Grazie, perfetto."

1.10.6 L'Initialize del bot e il Validated Get

Le variabili da rifare. L'Event Graph incollato "è normale" che dia errori: le sue variabili (le referenze al cavaliere e al suo Character Movement) appartengono a un altro Blueprint. "Dobbiamo semplicemente levare tutte queste variabili qua, perché le dobbiamo rifare tutte. Però intanto ci abbiamo la struttura."

Event Blueprint Initialize Animation. Si rifà l'Initialize come la volta precedente, ma per il bot:

1. **Event Blueprint Initialize Animation**;
2. **Try Get Pawn Owner** e **Cast To BP_Bot2** ("andando a fare un cast tra il Get Pawn Owner e il BP_Bot");
3. dall'uscita del cast, **Promote to Variable**: la referenza al bot, **AsBP_Bot2** ("promuoviamo a variabile la referenza");
4. dalla referenza, **Character Movement**, anche lui promosso a variabile.

"Questa è una prassi, ragazzi": "prendere tutte le referenze, in modo tale che possano comunicare" l'Animation Blueprint e il Blueprint del personaggio.

Il Validated Get. Nell'Update, prima di leggere i dati, la referenza al bot va controllata. Invece del nodo **Is Valid**, "a noi ci interessa farla tranquillamente con questo **Validated Get**": si trascina nel grafo un *Get* della variabile AsBP_Bot2, tasto destro sul nodo e, in fondo al menu, **Convert to Validated Get**. Il Get diventa un nodo con un ingresso di esecuzione e due uscite, **Is Valid** e **Is Not Valid**: "è un get che serve per dire: ehi, guarda, questa variabile all'interno è valida, in automatico, senza fare il blocco Is Valid". Lo si collega a **Event Blueprint Update Animation**, e dall'uscita *Is Valid* parte la **Sequence** incollata. "Perché, se questa referenza non c'è, ci sono errori": "esiste BP_Bot? Se non esiste, non faccio niente", "se non esiste il BP che mi comanda tutto". Il caso tipico è la preview dell'editor dell'Animation Blueprint, dove la mesh non appartiene a nessun personaggio e il cast fallisce.

Ricollegare e ricreare. Poi si rimettono le referenze nei nodi incollati: per la **Velocity** e per l'**Is Falling** il **Character Movement**, "perché sono tutte particolarità del movimento"; dove serve il personaggio, la referenza **AsBP_Bot2**. Le variabili di destinazione (Velocity, GroundSpeed, IsFalling) non esistono ancora in ABP_Bot2, e i loro nodi *Set* sono segnati come errori: "potete tranquillamente ricrearle molto facilmente: cliccandoci sopra, tasto destro, e c'è scritto proprio **Create variable**" (per esempio, *Create variable Velocity*). "Lo fate per tutte quante. Adesso compilate e siamo a posto" (figura 1.10.2).

"Questo, ragazzi, è come si copia e incolla una lezione": "per non rifare le stesse cose che abbiamo fatto volte e volte e volte, continuamente".

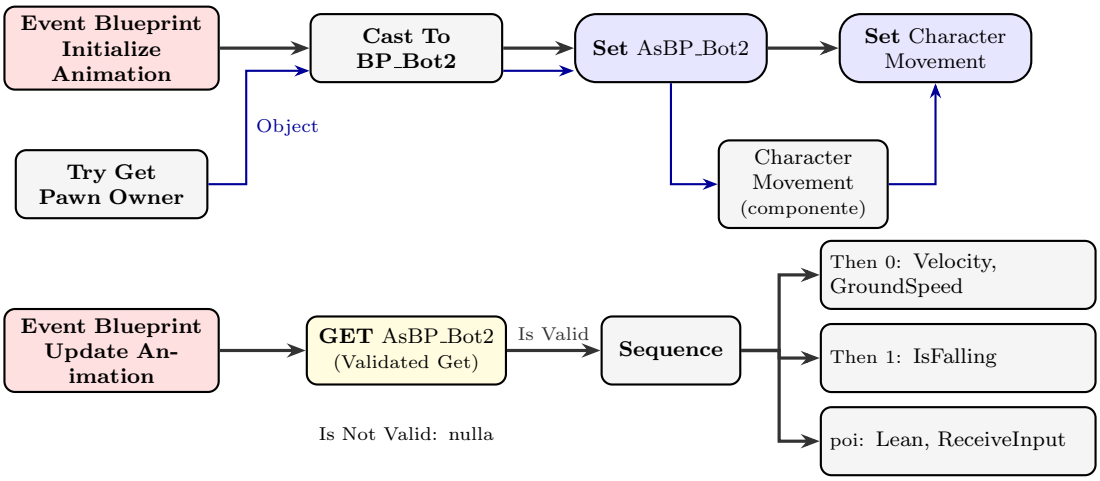


Figure 1.10.2: I due eventi di ABP_Bot2. In alto, l'Initialize, una volta sola: dal Pawn Owner al cast, alle due referenze salvate. In basso, l'Update, a ogni frame: il Validated Get blocca tutto se la referenza al bot non è valida; poi la Sequence esegue i calcoli. La Then 0 e la Then 1 vengono dall'Event Graph di ABP_Knight; il calcolo della direzione, che lì stava su un'altra uscita, viene eliminato e sostituito dal Lean; l'ultima uscita si aggiunge prima della pausa. Schema ricostruito.

Che cosa fa l'Initialize: la spiegazione. Più tardi, durante il lavoro sul Lean, una studentessa chiede che cosa fosse "il blocco di nodi tutto sopra", quello "che abbiamo dovuto fare a mano". La risposta del docente è il riassunto più chiaro della lezione su questo punto. "Quando inizializzi le animazioni, in generale, cosa andiamo a fare? Prova a prendere il Pawn Owner, nel senso il proprietario." Ogni character ha un pawn: "il pawn, ricordiamo, non è il character: è l'entità che controlla il character"; il character "è la mesh con le animazioni". "Quindi dice: vammi a prendere il pawn che ha questa animazione, che ha questo character." Poi il cast: "andiamo a cercare di comunicare direttamente con quel preciso actor, quindi BP_Bot". E le referenze: "visto che ci serviranno un sacco di cose", i valori della velocità e del "sta cadendo", che sono controllati dal Character Movement ("è lui che lavora sulla velocity e comunque la controlla"; "la possiamo prendere solamente se abbiamo un Character Movement a portata di mano"), "andiamo a dire: guarda che mi devi fare una referenza a questa comunicazione". "Come dirti, è un *gancio*: io mi lascio il gancetto attaccato, così almeno io comunico con te." "Crea una via di comunicazione tra l'animazione, cioè l'Animation Blueprint, e in questo caso BP_Bot": "come BP_Bot2, fammi queste azioni; oppure, come Character Movement di BP_Bot2, fammi queste azioni".

Approfondimento: il Validated Get e l'AnimGraph nel libro. Romero e Sewell (capitolo 16, nel modello VR) usano il Validated Get esattamente come la lezione: "quando avete un nodo Get di una referenza a oggetto, potete cliccarci con il tasto destro e scegliere *Convert to Validated Get*"; il nodo diventa "un nodo GET con dei pin di esecuzione che si possono usare per controllare se la referenza è valida". Nel capitolo 4 gli stessi autori raccomandano di usare sempre l'Is Valid prima di eseguire una funzione su una referenza: il Validated Get è la stessa cosa in un nodo solo. Il capitolo 17 spiega poi perché serve tutto questo lavoro di referenze nell'Event Graph: "l'AnimGraph può accedere solo alle variabili dell'Animation Blueprint, quindi

usiamo l'Event Graph per prendere i valori aggiornati dal Pawn". Nel loro esempio il calcolo è il più semplice possibile: *Try Get Pawn Owner*, *Get Velocity*, *VectorLength*, e il risultato nella variabile *Speed*. Una nota tecnica: in Unreal Engine 5 l'Event Graph dell'Animation Blueprint gira sul *game thread*; per personaggi numerosi Epic consiglia di spostare i calcoli nella *Blueprint Thread Safe Update Animation* e di leggere i dati con le *Property Access*, ma per un personaggio solo la tecnica della lezione è del tutto adeguata.

1.10.7 Ordine nel grafo: commenti e Reroute

I commenti. Mentre gli studenti ricopiano, il docente ne approfitta per un po' di pulizia: "potete creare dei commenti premendo **C**". Selezionati dei nodi, il tasto C crea un riquadro intorno a loro, "queste schedine che sono i commenti", con un titolo. Il docente commenta i blocchi dell'Update: **Check Actual Velocity**, **Check Is Falling**, **Check Direction** ("la sua direzione"). "Così viene un codice più ordinato per tutti quanti"; "questo lo sto facendo io per pulizia, ragazzi: se siete indietro, non mi seguite, tranquilli".

Then 0. Uno studente ha "nodi diversi" e chiede che cosa sia il "DANZIRO": è il **Then 0** della **Sequence**, la prima uscita, quella che nel grafo copiato porta al calcolo della velocità: "il Then 0 dice la velocity"; "questo fondamentalmente è il calcolo della velocity".

I Reroute Node. "Come fai a mettere quei puntini bianchi?" Sono i **Reroute Node**: si creano con un **doppio clic sul filo**, e si possono trascinare per piegare il collegamento. "Permettono di avere un po' più di ordine."

Approfondimento: commenti, Reroute e scorciatoie nel libro. Romero e Sewell (capitolo 15, *Blueprints Tips*) descrivono le stesse due tecniche: "per creare un riquadro di commento intorno ad alcuni nodi, selezionate i nodi, poi cliccate con il tasto destro su uno di essi e scegliete *Create Comment from Selection*, oppure premete semplicemente il tasto C"; il commento "aiuta a identificare un blocco di logica, e la sua etichetta resta visibile" anche quando si allontana lo zoom. Nel capitolo 11, durante la costruzione di una macro, notano che "si può fare doppio clic su un filo per aggiungere un Reroute Node e modificare la forma e la posizione del filo". Nello stesso capitolo 15 ricordano altre scorciatoie da tastiera per i nodi più comuni (B più clic per un Branch, S per una Sequence, D per un Delay, F per un For Each Loop), oltre al tasto Q per allineare i nodi già visto nella lezione precedente.

1.10.8 Il Blend Space 2D del bot

Creare il Blend Space. Tasto destro, **Animation**, **Blend Space**: **BS_Bot2**. Uno studente non lo trova ("abbiamo perso il Blend Space"): tasto destro, categoria Animation, Blend Space. Un altro chiede quale dei due: "quello non differenziato è quello 2D, cioè quello in cui non c'è scritto niente; quello 1D c'è sempre scritto 1D". Serve il **2D**.

Le animazioni di Lean. Prima di riempirlo, il docente mostra che cosa ha il bot: "abbiamo delle animazioni che si chiamano *lean*" (le aveva già usate Federico). "Lean è lo spostamento verso sinistra, verso destra": il bot "sta comunque camminando dritto, ma sta, come dire, spostato", inclinato da una parte. "Serve un po' per dare questa cosa di movimento al character": quando il personaggio curva, il corpo si piega verso l'interno della curva, come un corridore.

I due assi. "Nel 2D ci sono due dati che vanno a capire come è il movimento." Nei Details, **Axis Settings**, aprendo la freccina di ciascun asse si trova il nome ("potete dare il nome da qua a sinistra"):

- **Horizontal Axis**: **Lean**, minimo -10, massimo **10**;
- **Vertical Axis**: "la nostra bellissima, conosciutissima" **GroundSpeed**, da **0** alla velocità del personaggio.

La velocità massima del bot, nel Character Movement di BP_Bot2, è la **Max Walk Speed** di default, **600**. "Ma facciamolo un po' più lento, dai, facciamolo più lentino": **500**. "Tenetevi questi due aperti": nel Blueprint, Character Movement, Max Walk Speed a 500; nel Blend Space, **Maximum Axis Value** della GroundSpeed a 500.

Una domanda: legare l'asse alla Max Walk Speed? Uno studente ha un problema che ha incontrato più volte: calibra il Blend Space sui passi (per esempio, la corsa a 300), poi decide che 300 è la velocità massima e la corsa non parte più; oppure, cambiata la velocità, il personaggio fa "l'animazione di corsa anche mentre cammina". Ogni volta deve ritoccare a mano il massimo dell'asse e la posizione delle animazioni: "non c'è un modo per farlo veloce?", cioè per collegare la Max Walk Speed al Maximum Axis Value? "Purtroppo questo no: non so se ci sta un metodo, almeno non lo so, ma spero di sì. Però non l'ho mai visto. Effettivamente sarebbe molto comodo." E un avvertimento: "se tu cambi dei valori, specialmente se lo fai più piccoli, ti fa un casino", e a volte conviene levare tutte le animazioni e rifare il Blend Space da capo. "Sicuramente mi informerò, perché è una cosa che anch'io ambisco tantissimo, devo essere sincero." Allo studente, che aveva usato GroundSpeed e direzione per stabilire il lean, il docente consiglia di invertire gli assi: "sulla verticale metti la speed, sull'orizzontale metti la lean", e di seguire il procedimento della lezione.

I campioni. Le animazioni si trascinano sulla griglia (con **Shift** si agganciano ai punti precisi: "potete praticamente anche snappare i keyframe, tenendo premuto Shift"). La disposizione (figura 1.10.3):

- la **corsa** (*Bot Run*) al centro in alto, Lean 0 e GroundSpeed 500: "la mettiamo centrale, perché ricordiamo che stiamo dividendo";
- il **Lean Left** a -10, sempre a 500 ("500, meno 10"), e il **Lean Right** a 10: "negativo è sinistra, positivo è destra". Più che un'animazione diversa, "sarà praticamente la rotazione del nostro character che determinerà il fatto che lui si muova un po', si inclini a destra e a sinistra";
- in mezzo, "come animazione di appoggio", la **camminata** (*Bot Walk*) a Lean 0 e GroundSpeed **250**: "andiamo a creare questo triangolino".

Uno studente si è perso un valore: la GroundSpeed va da 0 a 500 ("minimum 0, maximum 500"); la walk "è 250". Il nome esatto degli asset di animazione dello StackOBot non si ricostruisce dalla trascrizione; il capitolo usa nomi descrittivi.

E l'Idle? "Scusami, ma dove sta l'Idle? Non la metteremo, ragazzi." Il motivo: il bot ha due atterraggi diversi, *Land Idle* e *Land Run*, "nel senso che lui atterra dopo un salto: se sta fermo, va e ritorna in Idle; oppure, se lui atterra e però sta camminando", continua a camminare. Per poter distinguere i due casi, la differenza fra fermo e in movimento va fatta nella **State Machine**, con uno stato *Idle* separato, e non dentro il Blend Space (sezione 1.10.10). È la scelta opposta a quella della lezione precedente, dove il Blend Space del cavaliere conteneva anche l'Idle a velocità zero.

Approfondimento: quando si cambia la velocità. Alla domanda dello studente Unreal non offre una risposta automatica: gli assi di un Blend Space sono valori dell'asset, non variabili, e non si possono legare a una proprietà del Character Movement. Esistono però due strade pratiche. La prima è *normalizzare* l'asse: invece della GroundSpeed in centimetri al secondo, si passa al Blend Space il rapporto fra la velocità e la Max Walk Speed (da 0 a 1), calcolato nell'Event Graph; i campioni restano dove sono anche se la velocità massima cambia. Il prezzo è che le animazioni possono "scivolare" sul pavimento se la velocità reale si allontana troppo da quella per cui sono state animate. La seconda è usare il *Play Rate* del Blendspace Player, o le opzioni di *Distance Matching* e *Stride Warping* del plugin *Animation Warping* di Unreal Engine 5, che adattano il passo dell'animazione alla velocità reale. Quanto all'avvertimento del docente, cambiare il minimo o il massimo di un asse sposta i campioni già piazzati, che vengono "schiacciati" nel nuovo intervallo: dopo una modifica conviene sempre ricontrollare la posizione di ogni campione, scrivendola a mano nei Details se serve.

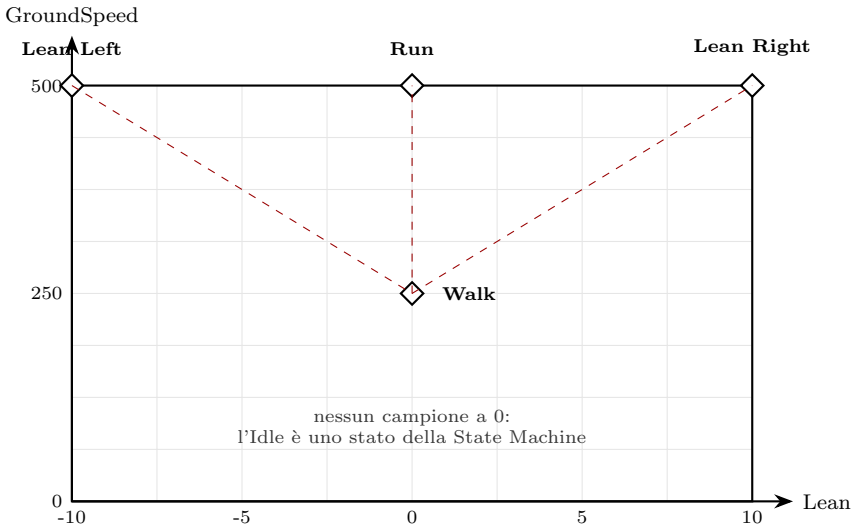


Figure 1.10.3: Il Blend Space 2D BS_Bot2. L'asse orizzontale è il Lean, da -10 (piegato a sinistra) a 10 (piegato a destra); quello verticale la GroundSpeed, da 0 a 500 (la Max Walk Speed del bot). La corsa e le due corse inclinate stanno in alto, la camminata a 250 fa da appoggio ("il triangolino"). In basso non c'è niente: l'Idle è gestito dalla State Machine, per distinguere gli atterraggi. Schema ricostruito.

1.10.9 Il calcolo del Lean

Tornati nell'Animation Blueprint, "una piccola cosa, ragazzi: adesso prendete *Check Direction* e lo eliminiamo". "Non ci serve, perché, a differenza del nostro character che abbiamo usato l'altra volta, che aveva otto animazioni", il bot ne ha una sola di corsa e due di lean. "Non dobbiamo calcolare la direzione del nostro player, ma dobbiamo semplicemente vedere la rotation del nostro character e vedere un po' da dove si sta piegando leggermente."

Che cos'è il Lean. Alla domanda di una studentessa ("io non ho ben capito cosa intende per Lean"), il docente risponde: "il Lean è il piegamento della rotazione; il Lean è *inclinazione*, lo intendiamo proprio". "Se il character si ruota leggermente di 10 gradi a destra, allora l'animazione dovrà diventare questa qua": "questo cammina dritto; questo cammina leggermente verso destra; e questo cammina leggermente verso sinistra". Il dato che lo misura è la variazione della rotazione del personaggio attorno all'asse verticale: "se noi adesso andiamo a vedere il nostro modello, dove ruoterà il nostro character? Sull'asse Z, farà così: destra, sinistra". Quel leggero destra e quel leggero sinistra "saranno dipendenti dall'asse Z", lo **Yaw**.

Il grafo, passo per passo. In una nuova uscita della Sequence (figura 1.10.4):

1. **Get Owning Actor** ("colui che possiede l'animazione, colui che possiede l'Animation Blueprint") → **Get Actor Rotation**;
2. la rotazione si **promuove a variabile**: **CurrentRotation**, la rotazione attuale;
3. si crea una seconda variabile di tipo **Rotator**, **PreviousRotation**, la rotazione del frame precedente;
4. "come si trova la differenza, ragazzi? Si trova con questo blocco, che è **Delta Rotator**" ("il delta è la differenza"); il docente collega PreviousRotation come primo ingresso ("come nostra rotazione iniziale") e CurrentRotation come secondo ("come nostra rotazione finale");
5. "dobbiamo splittare": sull'uscita del Delta Rotator, **Split Struct Pin**, perché interessa un solo valore, lo **Yaw**;

6. il passaggio graduale: il docente prova prima **RInterp To**, poi si corregge, “no, non è vero: dobbiamo usare **FInterp To**”, “perché abbiamo un solo valore”, lo Yaw, un float. Gli ingressi: **Current** è la variabile **Lean** stessa; **Target** è lo Yaw della differenza; **Delta Time** è **Get World Delta Seconds**, “il tempo di gioco, fondamentalmente: il tempo tra un frame e l’altro”; **Interp Speed**: “tipo a 5, così va benissimo” (più tardi, durante il debug, il docente consiglierà 1: “mettetela a 1, ragazzi: meglio”);

7. il risultato va nella variabile **Lean**, un **float** (“una variabile float, è scritto float: Lean”), “il nostro quantitativo di inclinazione”: **Set Lean**;

8. infine, **Set PreviousRotation** con il valore di **CurrentRotation**: “la previous rotation diventa la current rotation”.

Il blocco si commenta **Check Lean**. “Quello che ci interessa poi riportare nella nostra Blend Space è il Lean.”

Il ragionamento. “Spiego in poche parole.” Il calcolo avviene nell’Update, “a ogni frame”: all’inizio di ogni frame si legge la rotazione attuale, la si confronta con quella del frame prima, e la differenza sullo Yaw dice quanto, e in che verso, il personaggio sta girando. “Andiamo a mettere poi come target un quantitativo, ad esempio, tra meno 10 e 10”: “se mi sposto un po’ a sinistra, avrò valore negativo: meno 1, meno 2, meno 3... meno 10, in un tempo di gioco”. L’FInterp To non salta subito al bersaglio, ma ci arriva per gradi, e il risultato si salva nel Lean (“se più 10, più 10: più 1, più 2, più 3”). Poi la rotazione attuale diventa quella precedente, pronta per il frame successivo. “Con questa formula, ragazzi, che succederà? Il nostro character piegherà da una parte, poi ritornerà al suo posto; piegherà da una parte, e ritornerà al suo posto, in un tempo.”

Il primo frame. Una domanda precisa: PreviousRotation e Lean “non vanno dichiarati prima? Non dà errore?”; al primo ciclo l’FInterp To usa un Lean “che ancora non è stato settato”. No: le variabili hanno un valore di default, zero. “Lui parte dal gioco, dirà: aspetta, il Lean è 0, ti interpolo da 0 a 0. Quindi fondamentalmente non succede niente.” “Di base è 0 all’inizio del gioco; lui fa 0, ma nel tempo si aggiorna”, “perché fa ogni frame questa azione”. Al primo frame, in più, la PreviousRotation è zero mentre la CurrentRotation è già quella vera: il primo delta può essere grande, ma l’interpolazione lo assorbe.

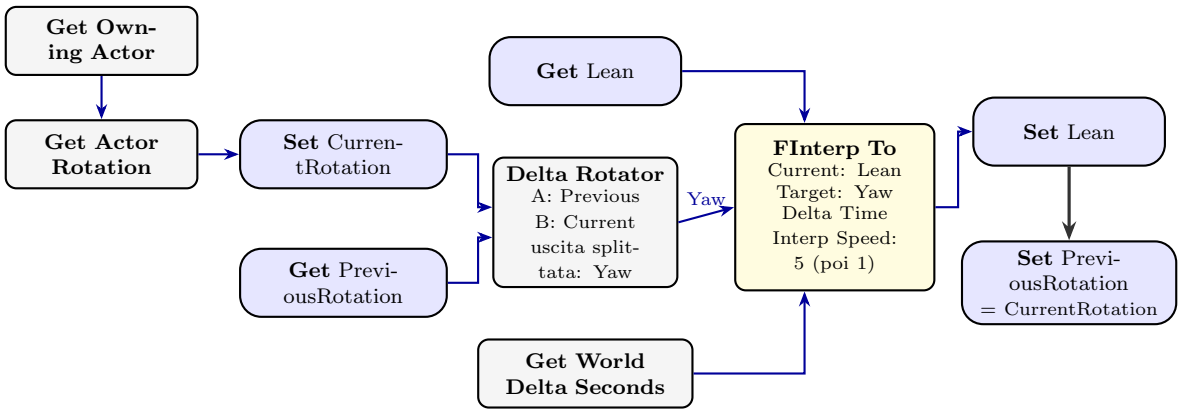


Figure 1.10.4: Il calcolo del Lean nell’Event Graph di ABP_Bot2 (commento *Check Lean*). A ogni frame: la rotazione attuale, la differenza con quella del frame precedente, lo Yaw di quella differenza come bersaglio di un’interpolazione, il risultato nel Lean; poi la rotazione attuale diventa la precedente. L’ordine dei due ingressi del Delta Rotator è quello indicato a lezione. Schema ricostruito.

Approfondimento: il segno, la scala e il limite del Lean. La formula della lezione funziona come idea, ma tre dettagli spiegano perché, nel debug di fine lezione, il Lean di una studentessa “non inclina” (sezione 1.10.14). Il primo è il *segno*: *Delta Rotator* restituisce la differenza normalizzata A − B; con Previous

in A e Current in B, una rotazione verso destra (Yaw che cresce) dà un valore negativo, cioè il contrario della convenzione del Blend Space (positivo a destra). Se il personaggio si piega dalla parte sbagliata, basta invertire i due ingressi. Il secondo è la *scala*: la differenza di Yaw fra due frame consecutivi è minuscola (a 60 frame al secondo, una curva di 90 gradi al secondo dà 1,5 gradi per frame), quindi il Lean resta quasi sempre vicino a zero, lontano dagli estremi ± 10 dove stanno le animazioni inclinate. La soluzione abituale è dividere la differenza per il Delta Time, ottenendo una velocità angolare in gradi al secondo, e poi scalarla (per esempio dividendo per 10 o per 20) prima dell'interpolazione. Il terzo è il *limite*: un *Clamp (float)* fra -10 e 10 tiene il valore dentro l'asse del Blend Space, anche se il Blend Space, di suo, limita già l'ingresso al suo intervallo. Un dettaglio di Unreal completa il quadro: con *Orient Rotation to Movement* il corpo ruota verso il movimento a una velocità massima (la *Rotation Rate*), mentre con *Use Controller Rotation Yaw* ruota istantaneamente con il mouse; il Lean, quindi, si comporta in modo diverso a seconda di quale delle due opzioni è attiva.

1.10.10 La State Machine a sette stati

"Adesso non abbiamo ancora niente, ragazzi: abbiamo semplicemente trovato i dati." Si passa all'**AnimGraph**.

Locomotion, DefaultSlot, Output Pose. Come la volta precedente: una **State Machine** chiamata **Locomotion**; "per precauzione, facciamo anche" il **DefaultSlot**, "così almeno siamo già preparati" (per i Montage); e si collega tutto all'**Output Pose**: Locomotion → Slot DefaultSlot → Output Pose. Dentro Locomotion, per ora, "non c'è niente", e infatti c'è un warning.

Idle e Walk. Il primo stato è **Idle**: dentro, l'animazione di Idle del bot, "con ovviamente loop". A uno studente che si è perso: "state machine: dovete cliccare qua dentro", poi tasto destro, *State* ("tanto lo trovi"), lo si chiama Idle, e dentro ci si mette l'animazione di Idle in loop. Il secondo stato è **Walk**: dentro, il Blend Space **BS_Bot2**, con i suoi due ingressi collegati ai Get delle variabili **Lean** e **GroundSpeed**. Il docente, "giusto per far vedere", mette per un momento dei numeri a mano negli ingressi: 200 ("al 200 non mi va": manca ancora la transizione), 5, "un po' stortino", 10.000. "Con il Lean praticamente controlliamo il piegamento del nostro character." "Voi mettete tutto a zero, ragazzi: questo era solo per far vedere."

Due domande dagli studenti. "A me dà warning": "chi ti dice *Idle to Walk will never be taken*? Sì, normale", perché la regola della transizione non è ancora scritta. "Non vedo lo schemetto dentro Walk", cioè l'anteprima del Blend Space nel nodo: "è perché l'animazione non va; se non va l'animazione, non te lo fa vedere".

Idle ↔ Walk. "Idle dovrà essere bilaterale": da Idle si va a Walk e da Walk si torna a Idle. Sulle due frecce, le *Can Enter Transition*, "un booleano, un vero o falso che ci dice: guarda, può entrare in transizione". Quale variabile? "Possiamo usare la GroundSpeed, perché tanto la GroundSpeed è la velocità in X e Y": si potrebbe usare anche la Velocity, ma allora bisognerebbe rifare il *Vector Length XY*, "perché non vogliamo prendere la Z", che è esattamente quello che la GroundSpeed già fa. Quindi:

- **Idle** → **Walk**: GroundSpeed $>$ **0** ("se è già 1, comincia");
- **Walk** → **Idle**: GroundSpeed $\leq$ **0**. "Io vi dico minore o uguale perché è meglio": Unreal "potrebbe sbagliarsi" e andare su un numero negativo piccolissimo, "tipo meno 0,0111"; "noi, per sicurezza, mettiamo minore o uguale a zero, così non abbiamo errori".

Compilato, il bot nella preview "adesso respira, è vivo, alla sua Idle".

I cinque stati del salto. "Adesso dobbiamo fare un sistema di animazioni basato sugli stati": altri cinque stati ("io faccio Ctrl+C, Ctrl+V", "facciamo più così, così viene più carino", "mettete così, a cascata"): **Jump Idle**, **Jump Run**, **Fall**, **Land Idle** e **Land Run**. La logica (figura 1.10.5): "sono in Idle, salto come in Idle"; vado in *Fall*; da Fall "controllo se sto toccando degli input" (avanti, sinistra, eccetera): "sì? Allora vado in Land Run; se non sto avendo degli input, vado in Land Idle". "Allo stesso modo anche Jump Run": dalla camminata si salta in corsa e si cade nello stesso Fall. "Questo comunque è un bivio": "andiamo

comunque sempre a Fall", perché "l'animazione di caduta è sempre uguale per tutti e due". Da Land Idle si torna a Idle, da Land Run a Walk. "Ovviamente dobbiamo mettere un sacco di regole per ogni contatto."

Le transizioni automatiche. "Qual è quella automatica, cioè quella senza condizioni?" Il passaggio da **Jump** a **Fall**: "se tu salti, in automatico tu cadi". Su entrambe le frecce (Jump Idle → Fall e Jump Run → Fall) si spunta **Automatic Rule Based on Sequence Player in State**, "in modo tale che questi siano in conseguenza": la transizione scatta quando l'animazione del salto finisce.

Le animazioni negli stati. Si riempiono gli stati con le animazioni del bot: il salto da fermo in Jump Idle, il salto in corsa in Jump Run, **Bot Fall** in Fall ("che dovremmo mettere in loop"), **Bot Land Idle** in Land Idle e l'atterraggio in corsa in Land Run. La trascrizione storpia i nomi degli asset ("jump idle, idle jump", "land idle, run jump"); quello che conta è la corrispondenza fra stato e animazione. Gli errori del compilatore, a questo punto, sono normali: mancano le regole delle transizioni, "cioè come si vanno a collegarsi le cose, a transitare".

Una domanda fuori tema: la finestra chiusa. Uno studente ha chiuso per sbaglio, con la X, l'Event Graph o un pannello dell'editor: si riaprono dal menu *Window* o, per i grafi, con un doppio clic nel pannello *My Blueprint*. "Hai tolto entrambe? Perfetto."

1.10.11 Le regole del salto e la variabile dell'input

Idle → Jump Idle: Is Falling. "Secondo quale variabile io dovrò passare da Idle a Jump Idle?" Uno studente propone la velocità sulla Z, "però forse anche Is Falling è giusto". Il docente lo verifica con un **Print String** del valore di IsFalling: per vederlo in gioco servono però il personaggio, la camera e gli input, che il bot non ha ancora ("mi sono scordato: andiamo a mettere lo Spring Arm... tanto poi lo rivedremo dopo"; "adesso dobbiamo spostare anche gli input, perché se no non prende l'input"). La conclusione, comunque: IsFalling "ci darà true appena noi non saremo a terra"; "siamo anche a 1,01 da terra, e già da quel momento il nostro Locomotion" è nello stato di caduta. Si usa quindi **Is Falling**.

Jump Idle o Jump Run? "Attenzione: c'è qualcosa che dobbiamo dire, perché queste condizioni sono uguali." Tutte e due le transizioni partono da IsFalling: "qual è la differenza? Il punto da cui partono": "se stavo in Idle o stavo camminando". Uno studente: "la velocità". "Benissimo, bravo: è la velocità, in questo caso la GroundSpeed." Con un **AND Boolean**:

- **Idle → Jump Idle**: IsFalling **AND** GroundSpeed $\leq$ **0**;
- **Walk → Jump Run**: IsFalling **AND** GroundSpeed $>$ **0**.

Uno studente osserva che è "un doppio controllo", perché Jump Idle si raggiunge comunque solo da Idle: "sì, sì, sì". Il controllo sulla velocità è ridondante rispetto allo stato di partenza, ma rende esplicita la regola.

Cadere senza saltare. Un'altra osservazione, dalla lezione precedente: "mi pare che avevamo un problema con questa struttura": se il personaggio non salta, ma cade da una superficie, la State Machine fa comunque l'animazione di salto. "Perché usavamo, infatti, l'altra volta, la velocity Z, e non IsFalling." "Effettivamente hai ragione": si potrebbe aggiungere la velocità sulla Z (Velocity con Split Struct Pin, Z maggiore di zero). Il docente guarda però le animazioni dello StackOBot, e decide che non serve: l'animazione "sembra fatta anche apposta per quando cadi", e la transizione verso Fall è comunque automatica. "Quindi possiamo tranquillamente usare" la GroundSpeed, "senza usare la Z. Dipende anche dalle animazioni. Proviamo a usare GroundSpeed, poi vediamo, magari possiamo correggerlo."

ReceiveInput: il giocatore sta premendo? Per gli atterraggi serve un dato nuovo: "se sta ricevendo input". Si aggiunge un pin alla **Sequence** dell'Update e:

1. dal **Character Movement** (la variabile; a uno studente che non la trova nella lista, "se lo cerco con la barra di ricerca lo trovo": "va bene, ottimo"), **Get Last Input Vector**;
2. **Vector Length**;
3. $>$ **0**: "se è maggiore di zero, vuol dire che diventerà vero";
4. **Promote to Variable**: **ReceiveInput** (un booleano);
5. il commento: **Check Input Vector**.

"Che cosa controlla questa variabile?" L'*input vector*: "immaginatevi: qualsiasi input che al character viene dato, lui crea un vettore". "Se noi non premiamo niente, il vettore è 0. Se noi premiamo qualche cosa, quel vettore si allunga", da 0 a 1. Un consiglio generale: "se voi ci mettete il mouse sopra", ogni nodo mostra una descrizione di quello che fa ("è un po' informatichese"): questo "ci dà il vettore del Pawn Movement Component" secondo gli input. "Sono due vettori: un vettore è la velocità, l'altro vettore è l'input che gli dà la velocità." "Quindi è positivo quando ci muoviamo?" "Esatto."

La pausa. "Allora, tanto sono le 17 e 28, ragazzi. Direi di per adesso fermarci qui"; dopo la pausa, le ultime regole delle animazioni e i Physics Material, "così andremo a vedere un po' come si fanno i passettini". "Ci vediamo tanto alle solite 18. Scusate: 17.55."

Approfondimento: Last Input Vector, accelerazione e velocità. Il Character Movement offre tre misure diverse del "muoversi", e conviene sapere quale si sta leggendo. La **velocità** (*Get Velocity*) è il movimento reale della capsula: continua per qualche frame dopo che il giocatore ha lasciato il tasto (la frenata) ed è diversa da zero anche quando il personaggio viene spinto o scivola. Il **Last Input Vector** è l'ultimo input di movimento ricevuto dal Pawn, la somma dei vettori passati agli *Add Movement Input*, con lunghezza fra 0 e 1: dice che cosa il giocatore *vuole* fare. La **Current Acceleration** (*Get Current Acceleration*) è l'accelerazione che il Character Movement sta applicando, ed è il dato che usa il modello di Epic, *ABP_Manny*, nella variabile *Should Move* (velocità sopra una soglia *e* accelerazione diversa da zero). Per decidere fra Land Run e Land Idle l'intenzione del giocatore è proprio il dato giusto: se, mentre cade, il giocatore tiene premuto avanti, all'atterraggio deve continuare a correre.

1.10.12 Dopo la pausa: gli atterraggi

"Perfetto, ragazzi. Allora, ricominciamo il nostro lavoretto sulle animazioni." "Eravamo rimasti al" Last Input Vector; "adesso ci mancano le ultime condizioni per far sì che queste animazioni passino tra l'una e l'altra".

Fall → Land Idle e Fall → Land Run. Il Fall, ricorda il docente, "è in automatico": da Jump Idle e da Jump Run si arriva a Fall a fine salto. Da Fall, invece, si esce quando il personaggio tocca terra, e si sceglie l'atterraggio in base all'input:

- **Fall → Land Idle**: "il land sarà quando non sto più cadendo", quindi **NOT** IsFalling ("faremo un NOT in modo tale che sia il suo contrario"), **AND NOT** ReceiveInput: "hai ricevuto un input? No"; "non avendo ricevuto input, andrà a fare la Land Idle";
- **Fall → Land Run**: la stessa regola, copiata e incollata, "levato ovviamente il NOT a ReceiveInput": NOT IsFalling AND ReceiveInput.

Land → Idle e Land → Walk. "Land Idle to Idle è in automatico", e così Land Run → Walk: "prendiamo le due condizioni e gli mettiamo il tick automatico", di nuovo *Automatic Rule Based on Sequence Player in State*. "Abbiamo fatto il nostro ciclo di animazioni" (figura 1.10.5 e tabella 1.10.1).

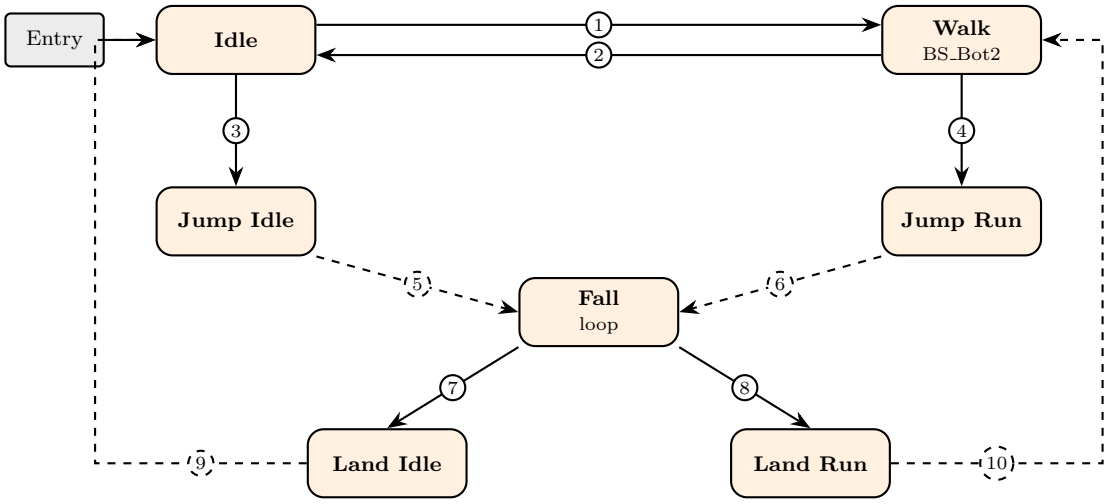


Figure 1.10.5: La State Machine Locomotion di ABP_Bot2: sette stati. Le frecce tratteggiate sono automatiche (*Automatic Rule Based on Sequence Player in State*); le altre hanno una regola (tabella 1.10.1). Fall è un bivio: la stessa caduta per i due salti, due atterraggi diversi a seconda dell'input. Schema ricostruito.

	Transizione	**Regola**	**Significato**
1	Idle → Walk	GroundSpeed > 0	comincia a muoversi
2	Walk → Idle	GroundSpeed ≤ 0	si è fermato (il $\leq$ per sicurezza)
3	Idle → Jump Idle	IsFalling AND GroundSpeed ≤ 0	salta (o cade) da fermo
4	Walk → Jump Run	IsFalling AND GroundSpeed > 0	salta (o cade) in movimento
5, 6	Jump Idle / Jump Run → Fall	automatica	finito il salto, cade
7	Fall → Land Idle	NOT IsFalling AND NOT ReceiveInput	tocca terra, il giocatore non preme niente
8	Fall → Land Run	NOT IsFalling AND ReceiveInput	tocca terra continuando a muoversi
9	Land Idle → Idle	automatica	finito l'atterraggio, fermo
10	Land Run → Walk	automatica	finito l'atterraggio, riprende a camminare

Table 1.10.1: Le dieci transizioni della State Machine del bot.

Approfondimento: atterraggi, priorità e il caso del bordo. Due dettagli della State Machine meritano attenzione. Il primo riguarda il caso discusso prima della pausa: con IsFalling come unica condizione, cadere da un bordo senza saltare fa partire comunque l'animazione di salto. È accettabile se, come nota il docente, l'animazione di salto dello StackOBot "sembra fatta apposta" anche per la caduta; altrimenti la soluzione è la regola della lezione precedente, con la velocità sulla Z: il salto vero comincia con una Z positiva, la caduta da un bordo con una Z negativa, e si può aggiungere una transizione diretta Idle → Fall e Walk → Fall per il secondo caso. Il secondo riguarda le transizioni con regole che possono essere vere insieme: quando da uno stato partono più frecce, Unreal le valuta in base alla *Priority Order* indicata nei Details di ciascuna transizione (il numero più basso vince). Nella State Machine del bot le regole che escono da uno stesso stato sono costruite per escludersi a vicenda (per esempio, ReceiveInput e NOT ReceiveInput), e la priorità non conta; ma è il primo posto da guardare quando una State Machine "sceglie" lo stato sbagliato.

1.10.13 Il bot in gioco

"Per vederlo testato, dobbiamo andare sul nostro bot." In BP_Bot2 si ricrea la camera da capo:

- un **Spring Arm** ("chissà perché me lo crea a 90 gradi; vabbè, lo giriamo, ci frega poco") e, come figlio, una **Camera**, anche lei girata: "questi qua devono essere tutti e due a zero". "Spring Arm e Camera devono essere tutti a zero, ok?";
- "un piccolo offsetino" sul **Target Offset**, perché la camera stia un po' più in alto (più avanti, nel debug, il valore indicato sarà **60**);
- volendo, si ruota la camera di una decina di gradi verso il basso, "in modo tale da avere un pochino di area visibile di più".

Il bot come personaggio del gioco. "Per mettere il nostro character effettivo in game, come facciamo?" **World Settings** ("lo trovate, se no, da Window, World Settings"): nella sezione del Game Mode, **Default Pawn Class** a **BP_Bot2**.

I grafi di input, copiati. In gioco il bot "non vi si muoverà", perché non ha gli input. Il default di Unreal "semplicemente, senza doverlo fare da capo, lo andiamo a copiare e incollare": si apre il vecchio personaggio, **BP_Knight**, e si copiano "solamente le cose commentate" (i blocchi del modello Third Person, già racchiusi nei loro commenti: l'Add Mapping Context sul Begin Play, il movimento, lo sguardo, il salto), **Ctrl+C**, e in BP_Bot2 "ce le spiaccichiamo dentro, perché tanto sono cose ormai default". "E ultima cosa, mi raccomando": **Use Pawn Control Rotation** sullo Spring Arm. "Adesso dovreste praticamente poter vedere il vostro caratterino che si può muovere in giro."

Il bot che levita. "Il mio personaggio levita": basta abbassare la mesh "leggerissimamente" nella viewport del Blueprint. "Guarda, io ce l'ho a meno 90 sulla Z, e sta a terra." Il valore corrisponde, per un personaggio di dimensioni standard, alla metà dell'altezza della capsula: la mesh ha l'origine ai piedi, la capsula al centro.

1.10.14 Il debug in diretta

"A me si muove, ma si muove in T-pose." "Vuol dire che c'è un problema sull'animazione, sicuramente." "Correggiamo gli errori dei nostri peccati": la condivisione di schermo passa allo studente. La sequenza, lunga, è un buon catalogo dei modi in cui un Animation Blueprint copiato e incollato si può rompere.

1. **La Velocity non collegata**: nel primo blocco dell'Update la Velocity "non è collegata"; va collegata quella del Character Movement ("devi collegarci con la velocity sotto"). Il Vector Length, invece, "va benissimo".
2. **L'Interp Speed**: nel calcolo del Lean, "l'Interp Speed mettetela a 1, ragazzi: meglio metterla a 1". Il resto del Lean (FInterp To, Delta Time, PreviousRotation) e il Last Input Vector con il Vector Length "vanno benissimo".

3. **La preview in T-pose**: la State Machine non era collegata. “Vai su AnimGraph, bravissimo, collegalo: non avevi semplicemente l’output.”

4. **Le regole mancanti**: “Jump Idle e Fall: ti mancano delle condizioni”. Le transizioni da Jump a Fall si rendono automatiche (“clicca solo una volta” sulla transizione, non due: con un clic se ne vedono i Details, con il doppio clic si entra nella regola; in alto c’è *Automatic Rule Based...*); anche Land Run → Walk. Poi l’errore residuo: manca la regola da Fall a Land Run, “IsFalling, NOT” e “ReceiveInput, che è quell’altra booleana sotto”, unite da un *AND Boolean*. Un consiglio estetico: “stacca, riattacca quelli sopra, così è un po’ più carino”.

5. **L’Anim Class assente**: il personaggio continua a non animarsi. “Hai messo l’Animation Class?” Nel BP_Bot, selezionata la Mesh: “non hai l’Animation Class”. Si imposta ABP_Bot2 e si compila.

6. **Il grafico del Blend Space che non compare**: nello stato Walk lo studente “non vede il grafico” della GroundSpeed: “vuol dire che non sta entrando in transizione”. Per vedere bene, il pannello *My Blueprint*, “a sinistra, sotto la preview”. Già che ci si è, si aggiunge il **DefaultSlot** fuori da Locomotion (“devi tirarlo fuori da Locomotion”): “questo tanto è solo per dopo”.

7. **Due Character Movement**: la causa vera. Nell’Initialize “c’hai Character Movement 0”: “c’hai due Character Movement”. Incollando l’Event Graph del cavaliere, la vecchia variabile e la nuova convivono, e Unreal ha rinominato la seconda aggiungendo uno 0. L’Update leggeva la velocità da quella sbagliata, mai valorizzata. “Leva Character Movement 1, quello senza lo zero”, e si ricollegano alla variabile buona la Velocity e l’Is Falling (“vedi che c’è quello blu, il pallino blu di Is Falling?”). Si eliminano anche i nodi rimasti della velocità sulla Z (“non ci serve Velocity Z”) e del vecchio *Target* (“devi semplicemente prendere anche l’altro character, collegarci il Character Movement 0”).

8. **Il cavaliere rimasto**: “qua mi ha dato quello del Knight, ma io non l’ho messo”: “elimina lo Knight Character, non ti serve”.

9. **Le variabili doppie**: “facciamo ordine sulle variabili”. Ci sono Velocity 0, 1 e 3, un Lean 0, delle *NewVar* e *NewVar 0* (le variabili create con il più e mai rinominate), una PreviousRotation in più: si eliminano tutte, tenendo una sola variabile per tipo. “Intanto ci abbiamo ordine.”

10. **L’Update non collegato**: “a parte che non hai collegato l’Update”. Dalle variabili a sinistra, AsBP_Bot, Get, tasto destro, *Convert to Validated Get* (“sotto, sotto, sotto”); lo si collega all’*Event Blueprint Update Animation* e all’Is Valid. “Non trovo facile, perché non c’è l’evento collegato, effettivamente”: senza l’evento, nessun calcolo veniva mai eseguito.

“Prova un po’, cammina.” Il bot si anima. Restano tre ritocchi. Il rumore dei passi “è normale che non lo senti, perché lo facciamo adesso”. Il personaggio è “leggermente fluttuante”: si scrive -90 nella Z della Mesh (“scrivici meno 90, così sei preciso”). La camera era stata inclinata di 10 gradi sulla X, cioè con il *Roll*: “devi inclinare di 10 sulla Y”, il *Pitch*, “la verde” (“in basso in alto, sì: 5, ma neanche troppo”), e sullo Spring Arm un **Target Offset** di **60**. Infine, ancora una volta, Use Pawn Control Rotation sullo Spring Arm. “Abbiamo fixato il nostro character, a posto, ottimo.”

Il Lean che non inclina. Una seconda studentessa condivide lo schermo (il docente nel frattempo era rimasto “mutato” senza accorgersene). L’Event Graph è giusto: Interp Speed portata a 1, l’animazione e il Blend Space corretti, i Lean positivi a destra (“sì, allora è giusto”), PreviousRotation e CurrentRotation collegate bene (“hai settato Previous come prima, e Current diventa Previous: va benissimo”). Ma in gioco il corpo “si gira, però non inclina”. I tentativi, sulle spunte di rotazione:

- staccare lo Spring Arm dalla Mesh e riattaccarlo alla radice;
- togliere *Orient Rotation to Movement* dai Details;
- controllare *Use Controller Rotation Yaw* sul BP_Bot (“scrivi Yaw”: è attivo), poi toglierlo;
- cercare dove sia finito l’Orient Rotation to Movement (“mi sa che forse hai messo l’Orient Rotation to Movement su un altro”);

- provare *Use Controller Desired Rotation* ("no, no, no: è orribile").

Il docente prova anche sul proprio bot e si accorge che il comportamento è in parte lo stesso ("il passaggio dietro lo fa pure a me, a quanto pare"). Il Lean dipende da come si ruota il personaggio: "si deve piegare a differenza del mouse, di dove lo stai ruotando": "prova a ruotarlo col mouse mentre cammini". Ultimo tentativo: un **Print String** del Lean nell'ABP, per vedere che valori assume. Ma il tempo stringe: "visto che manca poco alla fine della lezione, finiamo l'argomento, perché lo puoi comunque fare così; poi magari lo fixiamo". La questione resterà aperta fino alla lezione successiva (sezione 1.10.20); le cause più probabili sono quelle descritte nell'approfondimento sul segno e sulla scala del Lean (sezione 1.10.9).

1.10.15 I passi: l'Anim Notify

"Allora, ragazzi, andiamo adesso a fare il nostro bel Physics Material." Il primo pezzo non è il materiale, ma ciò che ascolta i passi: un **Notify**. "Cosa sono i Notify? L'abbiamo visto l'altra volta con le animazioni": nella lezione precedente, dentro il Montage, un Notify faceva partire un suono in un punto preciso dell'attacco.

La classe AN_Footstep. Una cartella **Notify** ("tasto destro, New Folder, chiamate Notify, con la Y finale"); dentro, una nuova **Blueprint Class**: in *All Classes* si cerca **AnimNotify** ("Anim Notify") e la si crea. Il nome che userà il docente è **AN_Footstep**. "Un Anim Notify praticamente sarà un puntatore, chiamiamolo così, che sarà messo da noi all'interno delle animazioni, e farà una determinata cosa al momento di quell'animazione in cui noi lo mettiamo."

Received_Notify. Aperta la classe, nel pannello delle funzioni, **Override** → **Received_Notify** ("il Receive Notify"): "andiamo a sovrascrivere la funzione di ricevimento di Notify", "nel senso che, nel momento esatto in cui l'animazione arriva e punta al Notify, il Notify parte". La funzione riceve tre ingressi (**Mesh Comp**, **Animation**, **Event Reference**) e ha un nodo di ritorno (*Return Node*) con un booleano. A uno studente che non capisce da dove vengano "Mesh Comp, Animation, Event Reference": "hai fatto Override? Ci sta Received_Notify": la funzione sovrascritta "ti dà indietro il Mesh Component e tutte le altre cose".

Il trace verso il basso. L'idea: "andremo a fare un *line trace* che andrà dal nostro character giù, a cercare il terreno, ogni volta che" si fa un passo; "dobbiamo identificare la tipologia di terreno in cui ci troviamo, e di conseguenza quello emetterà un suono". Lo strumento è "il nostro bellissimo e amico" **Line Trace By Channel**. Il grafo (figura 1.10.6):

1. **Mesh Comp** → **Get Owner**: "il proprietario, fondamentalmente, di questo Notify", cioè il personaggio che possiede la mesh che suona l'animazione;
2. **Get Actor Location**: il punto di **inizio**, collegato a **Start**;
3. **Get Actor Up Vector** moltiplicato per **250**: per moltiplicare un vettore per un numero, sul secondo pin del Multiply "tasto destro, *to float*" ("io mi dimentico sempre il multiply con il pallino verde sotto. Come si fa?" "Tasto destro, to float, c'è scritto");
4. la **sottrazione**: Actor Location − (Up Vector × 250), "in modo tale che sia il punto di fine", collegato a **End**. "Sarà praticamente un vettore che si è sottratto, quindi andrà all'indietro, come abbiamo visto anche nella lezione sui vettori": 250 centimetri sotto il personaggio;
5. un **Branch** sul **Return Value** del trace: "ehi, hai colpito qualcosa?";
6. dall'**Out Hit**, **Break Hit Result**; se ha colpito qualcosa, si suonerà un **Play Sound at Location**.

"Ma domanda: come differenziamo le tipologie di superfici?" "Dobbiamo usare il sistema di riconoscimento, comunque di divisione delle superfici, che Unreal ci dà in automatico."

Draw Debug Type. "Per questioni di debug, ragazzi": sul Line Trace By Channel, **Draw Debug Type** a **For Duration** (il docente aveva detto *Persistent*, poi si corregge: "più che Persistent, For Duration, meglio"). La linea del trace resta visibile per qualche secondo dopo ogni passo: è il "piccolo trace" della dimostrazione.

Get Surface Type. "Dalla Out Hit potete prendere anche **Get Surface Type**." "Questo blocco ci permette di dire: ehi, ho colpito questa superficie, mi dici la tipologia di superficie?" "Quindi, tramite questo, andremo a differenziare i suoni."

1.10.16 I Physical Surface e lo Switch

Dare un nome alle superfici. "Per dare dei nomi alle diverse superfici, cosa dobbiamo fare?" **Edit → Project Settings**, sotto la categoria **Engine**, la sezione **Physics**; "scendete giù, giù, giù, giù", fino a **Physical Surface**. Nella prima riga libera si scrive **Rock**, nella seconda **Water**. "Queste due ci vanno benissimo."

Switch on EPhysicalSurface. Tornati nell'AN_Footstep, dall'uscita di Get Surface Type si tira fuori uno **Switch** (*Switch on EPhysicalSurface*): "infatti, guardate che cosa ritorna: Default, Rock e Water". "Default è quello di default; Rock e Water sono quelli che abbiamo inserito." Poi:

- il **True** del Branch entra nello Switch;
- tre **Play Sound at Location** ("andiamo a copiare tre volte il Play Sound at Location"), uno per uscita: Default, Rock, Water;
- la **Location** di tutti e tre è l'**Impact Point**: "se voi andate su Break Hit Result e fate la tendina, ci sta un puntino a modi vettore, che è l'Impact Point", il punto esatto in cui il trace ha colpito il terreno;
- le uscite dei tre suoni vanno, attraverso un **Reroute Node**, al **Return Node**;
- "ultima cosa": il **False** del Branch "direttamente" al Return Node.

"Abbiamo la base del nostro sistema per cercare quale tipo di superficie stiamo camminando sopra. Ci mancano i suoni, però."

Approfondimento: il trace e il Hit Result nel libro. Romero e Sewell (capitolo 14) dedicano una sezione ai *trace*: "servono a verificare se ci sono collisioni lungo un segmento definito"; un trace *by channel* usa un canale, *Visibility* o *Camera* (quello di default, nel nodo della lezione, è Visibility), e restituisce un *Hit Result* con i dati del primo actor colpito. Fra le variabili del Hit Result, gli autori elencano *Blocking Hit* (se il trace ha colpito qualcosa), *Location*, *Impact Normal* e *Hit Actor*; il *Draw Debug Type* può essere *None*, *For One Frame*, *For Duration* ("la linea resta per il tempo indicato nel parametro Draw Time") o *Persistent* ("la linea non scompare"), e la freccina in fondo al nodo mostra i colori e il *Draw Time*. Gli stessi autori ricordano che i vettori *forward*, *right* e *up* di un actor sono normalizzati, di lunghezza 1, e che "per trovare il vettore opposto basta moltiplicarlo per -1": sottrarre l'Up Vector moltiplicato per 250, come a lezione, equivale ad aggiungere il vettore *down* moltiplicato per 250. Il Play Sound at Location, infine, compare nel capitolo 6: "un nodo semplice che prende in ingresso un file audio e una posizione, e suona il suono in quella posizione". Nel Hit Result c'è anche il pin *Phys Mat*, il Physical Material colpito: *Get Surface Type* legge da lì il tipo di superficie. I trace dei Blueprint restituiscono il Physical Material di default, senza bisogno di opzioni aggiuntive. Nelle Project Settings si possono definire fino a 62 tipi di superficie (*SurfaceType1*... *SurfaceType62*); i nomi vengono salvati nel file di configurazione del progetto.

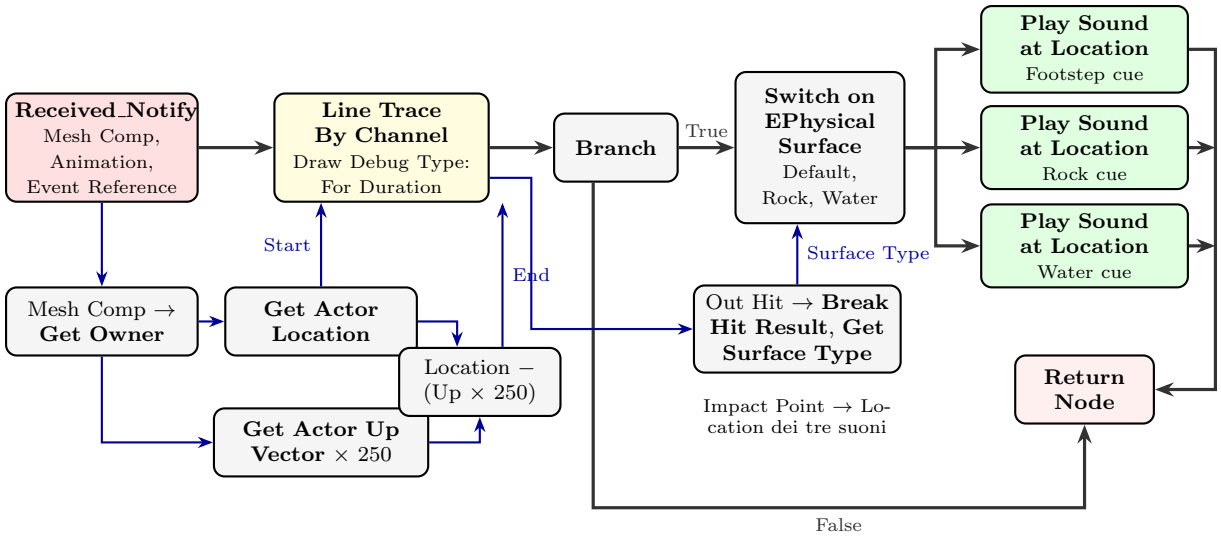


Figure 1.10.6: L'Anim Notify AN_Footstep. Quando l'animazione arriva al Notify, un trace parte dal centro del personaggio e scende di 250 centimetri; se colpisce qualcosa, il tipo di superficie sceglie quale dei tre Sound Cue suonare nel punto d'impatto. Tutte le strade finiscono nel Return Node. Schema ricostruito.

1.10.17 I suoni: Sound Wave e Sound Cue

"Avete scaricato la cartella del materiale didattico, ragazzi?" "Sì." Nella cartella **Footstep Sounds** ci sono i file audio. Si trascinano tutti nella cartella *Audio* del progetto, "già creata" nelle lezioni precedenti ("prendete tutti quanti e schiaffateli qua dentro"). Quelli importati sono **Sound Wave**, "non i file con la cuffietta, ma tutti questi file violacei": il docente, per mostrarlo, cancella i Sound Cue che aveva già.

Perché un Sound Cue. "Dobbiamo creare delle Sound Cue, per far sì che il nostro suono sia un po' modulato in diversi modi." C'è anche un'altra ragione: il passo normale ha un solo file ("footstep Mac 0" nella trascrizione), ma la roccia ne ha due, *Rock 1* e *Rock 2*, e così l'acqua. "Vogliamo metterli e randomizzarli, in modo tale che ogni tanto suoni uno, ogni tanto suoni l'altro."

Create Single Cue. "Una cosa in automatico, ragazzi": si seleziona il primo suono, si tiene premuto **Shift** e si seleziona il secondo; tasto destro, **Create Single Cue**. Il docente rinomina il risultato ("Rock2"; per l'acqua, *Water Cue*). Aperto il Sound Cue: "è presente un blocco chiamato **Random**", "che potete tranquillamente anche evocare, volendo, con il tasto destro". Con questa tecnica il Random fra i due suoni è già fatto: "se adesso io vado a fare un Play Cue qua, randomicamente mi fa suonare uno dei due".

Il Modulator. "Ma vogliamo anche fare un'altra cosa": "modularli in modo tale che il *pitch* sia diverso a ogni passo, in modo tale che sia leggermente un po' più diverso il rumore". Dal Random si tira fuori un **Modulator**, che si collega all'uscita (figura 1.10.7). I valori consigliati, "poi magari anche soggettivo come cosa", sono nella tabella 1.10.2. Per la roccia il docente sceglie un pitch alto, da 2 a 2,1 o 2,2, "perché ho notato che è un suono piuttosto lungo. Se lo fate troppo lento, tende a non essere molto sincronizzato col passo." Per l'acqua parte da 0,95–1,2, poi alza: "forse un periodo un po' più veloce, perché mi sembrano un po' lunghi: 1,3, 1,5, vanno bene".

I suoni nel Notify. "Fatti i suoni, dobbiamo semplicemente tornare al nostro Anim Notify" e assegnare a ciascun Play Sound at Location il suo Sound Cue: a **Default** il passo normale, a **Rock** il Cue della roccia, a **Water** quello dell'acqua.

Sound Cue	Sound Wave	Pitch Min	Pitch Max	Superficie
Footstep	un solo file	0,95	1,3	Default
Rock	Rock 1, Rock 2 (Random)	2,0	2,1–2,2	Rock
Water	Water 1, Water 2 (Random)	1,3	1,5	Water

Table 1.10.2: I tre Sound Cue dei passi, con i valori del Modulator indicati a lezione. Per l'acqua il primo tentativo era 0,95–1,2.

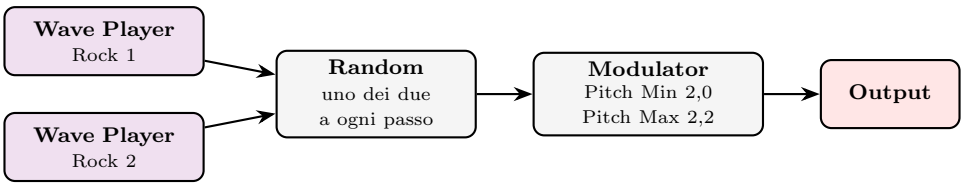


Figure 1.10.7: Il Sound Cue della roccia. *Create Single Cue* su due Sound Wave selezionati crea i due Wave Player e il Random; il Modulator, aggiunto a mano, cambia il pitch a ogni esecuzione. Il Cue del passo normale ha un solo Wave Player e nessun Random. Schema ricostruito.

Approfondimento: Random, Modulator e MetaSounds. Il *Random* di un Sound Cue sceglie a ogni esecuzione uno dei suoi ingressi; nei Details ha dei pesi (*Weights*), per rendere un suono più frequente di un altro, e l'opzione *Randomize Without Replacement*, che evita di ripetere lo stesso suono finché non sono stati suonati tutti. Il *Modulator* sceglie a ogni esecuzione un valore casuale fra un minimo e un massimo, sia per il *pitch* sia per il *volume*: è la tecnica più semplice per evitare l'effetto "mitragliatrice" di un suono identico ripetuto a ogni passo. Il pitch è un moltiplicatore della velocità di riproduzione: a 2 il suono dura la metà ed è un'ottava più acuto. È il motivo pratico della scelta del docente per la roccia (un suono lungo, accorciato perché stia dentro il passo), ma anche il suo prezzo: il timbro cambia molto. In alternativa, si può accorciare il file in un editor audio e usare un pitch vicino a 1. In Unreal Engine 5 i Sound Cue convivono con i **MetaSounds**, il sistema audio procedurale più recente, che fa le stesse cose (scelta casuale, variazioni di pitch e volume) con un grafo simile ai Blueprint. Un ultimo dettaglio: perché un Play Sound at Location si senta davvero "nel punto", cioè più forte da vicino e da una direzione precisa, il suono deve avere delle impostazioni di *Attenuation*; senza, si sente allo stesso volume ovunque.

1.10.18 I Notify nelle animazioni: Sync Marker e Notify Track

"Adesso ancora non abbiamo nessuna tipologia di interazione, perché questo Blueprint è un Animation Notify": "dal nome, deve essere innestato dentro le animazioni". Le animazioni sono le tre della corsa: la **Run**, la **Lean Run Right** e la **Lean Run Left**. "Consiglio": si apre BS_Bot2 ("scrivete BS") e da lì, con un doppio clic su ciascun campione, si apre l'animazione. A uno studente che chiede quale sia "quella finestra con il minutaggio dei passi": "questa qui è l'animazione: vai sul Blend Space e clicca su Bot Run".

Trovare il passo. Nell'editor dell'animazione c'è "tutta la linea del tempo dell'animazione". Il docente elimina i Notify già presenti nell'animazione dello StackOBot ("queste qua le eliminiamo, giusto perché lo rifacciamo"). "Dobbiamo identificare, nei frame dell'animazione, quando il piede del nostro character sta a terra." In tutte e tre le animazioni, "il piede destro sta a terra al frame **3**, invece il piede sinistro sta a terra al frame **13**". "Quindi sarà da mettere l'Animation Notify al 3 e al 13."

I Sync Marker. "Per avere una maggiore sicurezza, per quanto riguarda l'osso del nostro piede", il docente aggiunge dei marcatori di sincronizzazione: sulla traccia dei Notify (quella con scritto *1*), tasto destro, **Add Sync Marker**: al frame 3 un marker **R** ("il piede destro"), al frame 13 un marker **L**. "Questo marker semplicemente cerca di sincronizzare il movimento."

La Notify Track e il Notify. Poi, "qua vicino a quella dell'1, c'è un pulsante chiamato **Track**": **Insert Notify Track**, e la si chiama **Sound**. Sulla linea della traccia Sound, tasto destro, **Add Notify** → **AN_Footstep**, "esattamente in sincronia con il passo"; si può copiare con **Ctrl+C**, **Ctrl+V** e spostare sul secondo passo (figura 1.10.8). "Guardate cosa succede": scorrendo la linea del tempo avanti e indietro, "viene compiuto il passo, il suono almeno": il Notify suona anche nell'anteprima dell'editor. La stessa cosa va fatta per Lean Left e Lean Right, dove bisogna rimettere sia i marker sia il Notify, "perché è diverso": ogni animazione ha la sua linea del tempo, e i Notify non si ereditano.

Il Notify che non si trova. "A me AN_Footstep, se lo cerco, non me lo trova." "Sei sicuro che hai creato la classe Animation Notify?" La classe c'è, ma con un altro nome (nella trascrizione, "da anni notifico"): nel menu *Add Notify* va cercata con il nome che le si è dato. Poi lo studente rifà il procedimento sulle altre due animazioni: "vedi dove è il piede destro, metti il sync, metti il Notify", "sempre 3 e 13: destro al 3, R; sinistro al 13"; "ricordati di creare un'altra traccia", *Insert Notify Track*, "chiami Sound".

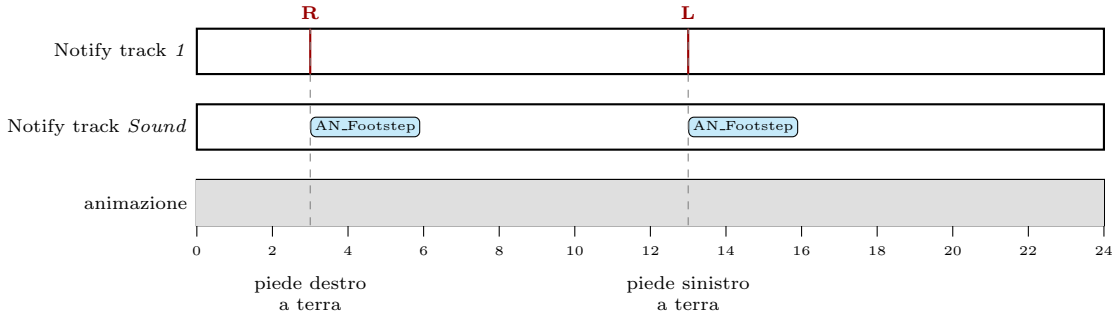


Figure 1.10.8: La linea del tempo della corsa del bot (e delle due corse inclinate). Sulla traccia *1* i Sync Marker R e L ai frame 3 e 13; su una nuova traccia *Sound* l'AN_Footstep negli stessi due punti. La lunghezza della linea del tempo è indicativa: il numero totale di frame dell'animazione non si sente nella trascrizione. Schema ricostruito.

Approfondimento: Sync Marker, Blend Space e Notify doppi. I Sync Marker non sono legati alle ossa, come la trascrizione può far pensare, ma alla linea del tempo: sono etichette con un nome (qui R e L) che segnano le fasi del passo. Servono quando Unreal deve mescolare animazioni di durata diversa, nei Blend Space o con i *Sync Group*: invece di allinearle per tempo normalizzato, le allinea per marker, in modo che i piedi tocchino terra insieme in tutte le animazioni mescolate. Funziona solo se tutte le animazioni hanno gli stessi marker, con gli stessi nomi: è il motivo pratico per cui il docente li aggiunge a tutte e tre le corse. I Notify, invece, pongono una domanda diversa: in un Blend Space che sta mescolando tre animazioni, *di quale* animazione devono scattare i Notify? Nei Details del Blend Space, nella sezione *Animation Notifies*, l'opzione *Notify Trigger Mode* decide: *All Animations* (tutte quelle che hanno un peso, con il rischio di sentire due o tre passi sovrapposti), *Highest Weighted Animation* (solo quella che pesa di più, la scelta più sicura per i passi) o *None*. Da notare infine che, nella lezione, la camminata a 250 non riceve il Notify: a velocità intermedie i passi si sentono solo se il campione dominante è una delle corse; per un risultato completo va aggiunto anche alla camminata, ai suoi frame di appoggio. L'editor delle animazioni offre anche un Notify già pronto, *Play Sound*, che suona un suono senza Blueprint (il suo limite è che non sa distinguere le superfici), e i *Notify State*, che hanno un inizio e una fine invece di un istante.

1.10.19 I Physical Material e le pedane

"Adesso, se noi andiamo in play, andiamo a vedere. Guardate un po'." A ogni passo compare il trace che identifica il terreno, "in questo caso il terreno di default". "Ma come facciamo a dire, guarda, questo è un terreno diverso da quello normale? Dobbiamo usare i Physics Material."

Creare i Physical Material. Una cartella **PhysicsMaterial** ("eccola qua"); dentro, tasto destro, **Physics → Physical Material**. Se ne creano due: **PM_Rock** e **PM_Water**. Poi, "una cosa molto semplice, ragazzi": doppio clic su ciascuno; c'è la proprietà **Surface Type**, che di default vale *Default*: per PM_Water si sceglie **Water**, per PM_Rock **Rock**. I nomi sono quelli scritti nelle Project Settings (figura 1.10.9).

Le pedane. "L'unica cosa che ci manca è prendere una qualsiasi superficie." Un blocco del livello, clonato (**Alt** più trascinamento, o duplicato) e schiacciato: "fate un pianerottolino piccoletto", due pedane. "Adesso non abbiamo i materiali, ad esempio, dell'acqua e della terra, ma ne mettiamo due a caso": un materiale *solid blu* per l'acqua, *Boulder* per la roccia; sono solo un segnale visivo. Poi: selezionata la Static Mesh, "andiamo giù, giù, giù, fino a trovare il **Phys Material Override**", e lì **PM_Water** per l'acqua, **PM_Rock** per la roccia. "Adesso, se noi andiamo sopra con il character, sentiamo che" sulla roccia il passo cambia. "Eccolo qua."

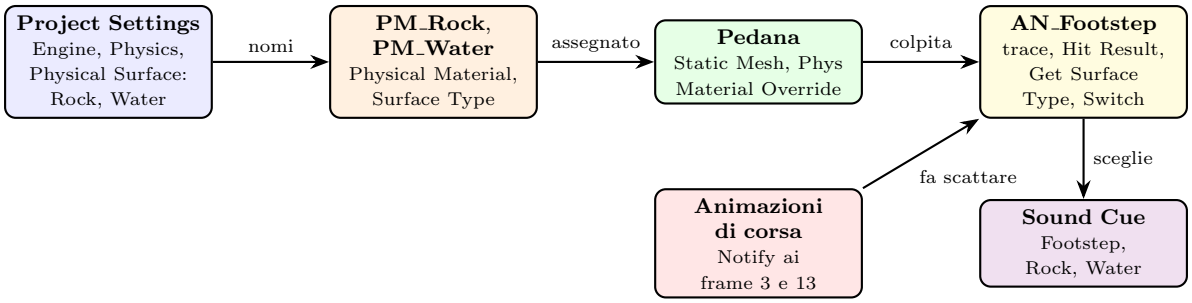


Figure 1.10.9: La catena completa dei passi. I nomi delle superfici nascono nelle Project Settings, vengono assegnati ai Physical Material, che a loro volta si assegnano alle superfici del livello; a ogni passo, il Notify inserito nell'animazione lancia il trace, legge la superficie colpita e sceglie il Sound Cue. Schema ricostruito.

La spiegazione ripetuta. A uno studente che chiede dove trovare "i cosini" (i PM): "quelli li crei, praticamente. Lo fai con tasto destro, Physics, Physical Material, e lo crei. Lo chiami PM_Rock, come vuoi, PM_Rock e PM_Water. Ci clicchi su ognuno di essi e gli cambi il Surface Type: lo metti Rock per il rock e Water per il water. Poi, fatto questo, prendi un qualsiasi asset (vabbè, anche un blocco va benissimo): devi andare su Phys Material Override e gli metti il Physical Material." "Basta che fai due pedanine": "qualsiasi superficie che tu vuoi prendere, ci puoi camminare sopra"; "la scali, fai quello che vuoi, fondamentalmente".

Approfondimento: dove si assegna un Physical Material. Il *Phys Material Override* usato a lezione vale per tutto il componente: comodo per una pedana, poco pratico per un terreno che cambia da un punto all'altro. Unreal offre altri due posti. Il primo è il **materiale**: nei Details di un Material (o di un Material Instance) c'è la proprietà *Phys Material*; tutte le mesh che usano quel materiale "diventano" roccia o acqua, senza toccare il livello, ed è la soluzione abituale nei progetti veri. Il secondo è il **Landscape**: ogni layer dipinto può avere il suo Physical Material, e il trace legge quello del layer dominante nel punto colpito. In più, il Physical Material non serve solo ai suoni: contiene l'attrito (*Friction*), il rimbalzo (*Restitution*) e la densità usati dalla simulazione fisica, ed è la base per gli effetti d'impatto dei proiettili (scintille sul metallo, polvere sulla pietra), con la stessa tecnica del Get Surface Type e dello Switch.

1.10.20 Gli ultimi problemi e la chiusura

Gli ultimi problemi. "Un attimo: è successo un casino? Gli altri? Qualche problema? Funziona?"

- Uno studente si è perso il primo pezzo del grafo dell'AN_Footstep e non sa da dove vengano Mesh Comp, Animation ed Event Reference: sono gli ingressi di Received_Notify, la funzione sovrascritta con *Override*. Poi, di nuovo, la catena: "Get Owner, Get Actor Location, Get Actor Up Vector, e poi sottrai, per far sì che" End sia sotto lo Start, "che è il Get Actor Location".

- Lo stesso studente ha messo le pedane, ma il personaggio non fa nessun rumore, nemmeno sul pavimento normale. I suoni nell'Anim Notify ci sono ("Rock1, Water Cue, Footstep Cue: va benissimo"); l'errore è nel vettore: il Get Actor Up Vector era collegato al pin sbagliato del Multiply. "Mettilo sopra; stacca quello sotto; tasto destro su quello là, to float, 250." Con il Draw Debug Type a For Duration e una ricompilazione, il passo si sente.
- Le pedane, però, non fanno il suono della roccia: "fammi vedere se hai messo la cosa", e infatti il Phys Material Override non c'era. Lo studente non riesce a scendere nei Details fino alla voce: il pannello era filtrato su una categoria (*Generale*); cliccando su *Tutto* e aprendo la parte avanzata della sezione *Physics* ("vai su tutto, clicca su tutto; giù, eccolo là"), si trova *Phys Material Override*. "A posto, perfetto."
- "A me non compaiono le linee": "basta che fai Draw Debug Type e la metti For Duration", sul Line Trace By Channel. "Quello è solo una cosa di debug, tranquilli."

L'inclinazione rimandata. "Io ho sempre quella cosa dell'inclinazione della testa": è la studentessa del Lean che non inclina. "Possiamo farlo la prossima volta?" "Sì, facciamo la prossima volta. Tanto la prossima volta facciamo interazioni, e allora ricordamelo tu, eh?" "Così fixiamo tutto"; "lo guardo poi in futuro, lo guardo poi in post anche".

Il calendario. "Io vi ringrazio per l'attenzione, grazie" anche a chi segue in differita. "Ci vediamo giovedì per la prossima lezione, che faremo in questo caso i menu: andiamo a fare un po' di UI." (Il docente, poco prima, aveva parlato di "interazioni": dalla trascrizione non si capisce se la lezione di giovedì conterrà entrambe le cose.) Uno studente pensava che oggi fosse giovedì; il calendario: "abbiamo questo martedì, giovedì e, se non sbaglio, il 30, che è martedì prossimo: è l'ultima con me. Poi ci avete Davide." "Va bene, ragazzi. Allora ci vediamo. Buona cena. Buona serata."

Punti chiave del capitolo

- Un'**Input Action** cancellata rompe il personaggio in modo silenzioso: si ricrea duplicando un'azione dello stesso tipo (**IA_Look**, Vector 2D, per **IA_Move**), la si rimette nell'**IMC** e nell'evento. Lo **Spring Arm** non si sposta dal pivot: si regolano Target Arm Length, Socket Offset e Target Offset.
- Quando si cambia personaggio, tutto ciò che riconosceva il vecchio va ricontrollato: **tag** (Player per la torretta), **cast** a classi precise, **collisioni** e risposte ai trace.
- Un Animation Blueprint si può costruire **copiando** l'Update di un altro, ma l'**Initialize** si rifà: Try Get Pawn Owner, Cast To, referenze promosse a variabile, "un gancio" fra animazione e personaggio. Le variabili mancanti si ricreano con **Create variable**; il **Validated Get** protegge l'Update. Attenzione alle variabili doppie (Character Movement 0).
- Ordine nel grafo: **commenti** con il tasto C, **Reroute** con un doppio clic sul filo.
- Il **Blend Space 2D** del bot ha il **Lean** in orizzontale (-10, 10) e la **GroundSpeed** in verticale (0–500, la Max Walk Speed): corsa e corse inclinate a 500, camminata a 250, nessun Idle.
- Il **Lean** si calcola a ogni frame: rotazione attuale, **Delta Rotator** con quella precedente, lo **Yaw**, **FInterp To** verso quel valore (Interp Speed 1–5, Get World Delta Seconds), poi la rotazione attuale diventa la precedente. Segno e scala vanno verificati.
- La **State Machine** ha sette stati: Idle e Walk (GroundSpeed > 0, ≤ 0), Jump Idle e Jump Run (IsFalling AND la velocità), Fall (automatica dopo i salti), Land Idle e Land Run (NOT IsFalling AND [NOT] **ReceiveInput**), ritorni automatici. ReceiveInput viene da **Get Last Input Vector**: la lunghezza maggiore di zero dice che il giocatore sta premendo.
- Per giocare con il bot: Spring Arm e Camera a zero, Target Offset, **Default Pawn Class** nelle World Settings, i grafi di input copiati dal vecchio personaggio, Use Pawn Control Rotation, Mesh a -90.

Sintomo	**Causa e rimedio**
Il personaggio non si muove, errori su IA_Run	IA_Move cancellata e sostituita da IA_Run: duplicare IA_Look (Vector 2D), rinominarla IA_Move, rimetterla in IMC_Default e nell'evento
L'evento del movimento non mostra il vettore	salvare l'Input Action; poi Split Struct Pin sull'Action Value
Dopo una compilazione "sparisce tutto"	il Blueprint non compila: leggere i Compiler Results
Camera storta, movimento strano	Spring Arm attaccato alla freccia o spostato dal pivot: riattaccarlo, location a zero, usare Target Arm Length, Socket Offset, Target Offset
La camera non segue il mouse	Use Pawn Control Rotation sullo Spring Arm
La torretta ignora il nuovo personaggio	manca il tag Player; controllare le risposte ai trace e alle collisioni
Il Montage non si vede	manca lo Slot DefaultSlot nell'AnimGraph
Durante il Montage il corpo gira	il DefaultSlot prende tutte le ossa: serve uno slot per la parte alta (Layered blend per bone)
Errori nell'Event Graph incollato	variabili del vecchio ABP: ricrearle con Create variable, rifare le referenze nell'Initialize
Idle to Walk will never be taken	la regola della transizione non è ancora scritta
Lo stato non mostra l'anteprima del Blend Space	lo stato non è mai raggiunto (transizione o dati mancanti)
Il personaggio levita	Mesh a $Z = -90$
Il personaggio si muove in T-pose	State Machine non collegata all'Output Pose, o Mesh senza Anim Class
Errori su Jump e Fall	regole vuote: Automatic Rule Based on Sequence Player in State, e la regola Fall $\rightarrow$ Land Run
GroundSpeed sempre zero	due variabili Character Movement (una mai valorizzata), o Update non collegato al Validated Get
Il Lean non inclina	segno e scala del Delta Rotator, opzioni di rotazione; rimandato alla lezione successiva
La camera inclinata di lato	ruotata sulla X (Roll) invece che sulla Y (Pitch)
AN_Footstep non compare in Add Notify	la classe ha un altro nome: cercarla con quello
Nessun suono di passi	Up Vector sul pin sbagliato del Multiply; Notify non inserito nelle animazioni
Le pedane suonano come il pavimento	manca il Phys Material Override, o il Surface Type del Physical Material
Phys Material Override introvabile	filtro dei Details su Generale: scegliere Tutto e aprire le opzioni avanzate
Le linee del trace non si vedono	Draw Debug Type a For Duration

Table 1.10.3: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, asset o impostazione	Che cosa fa
Input Action, Value Type Vector 2D	azione con un valore a due assi (movimento, sguardo)
Add Movement Input	muove il Pawn lungo una direzione, con una scala
Tags (Actor)	etichette con cui altri Blueprint riconoscono l'actor
Try Get Pawn Owner / Cast To	il Pawn che possiede l'animazione; la sua classe precisa
Promote to Variable	salva un'uscita in una nuova variabile
Convert to Validated Get	Get con controllo di validità (Is Valid / Is Not Valid)
Create variable (tasto destro)	crea la variabile mancante di un nodo incollato
Comment (tasto C) / Reroute Node (doppio clic sul filo)	riquadro con titolo; punto di piegatura di un filo
Blend Space 2D: Lean e Ground-Speed	mescola le corse inclinate e la camminata
Get Owning Actor / Get Actor Rotation	l'actor proprietario; la sua rotazione
Delta Rotator	la differenza normalizzata fra due rotazioni
FInterp To / RInterp To	un passo d'interpolazione fra float; fra rotator
Get World Delta Seconds	il tempo dell'ultimo frame
State Machine, Can Enter Transition	stati e regole booleane di passaggio
Automatic Rule Based on Sequence Player in State	la transizione scatta a fine animazione
AND Boolean / NOT Boolean	congiunzione; negazione
Get Last Input Vector	l'ultimo input di movimento del Pawn (lunghezza 0–1)
Vector Length	la lunghezza di un vettore
Print String	scrive un valore a schermo, per il debug
World Settings: Default Pawn Class	il personaggio del giocatore nel livello
Anim Notify (classe), Received_Notify	evento in un istante dell'animazione; la funzione che lo riceve
Get Owner / Get Actor Location / Get Actor Up Vector	proprietario; posizione; vettore verso l'alto
Line Trace By Channel, Draw Debug Type	trace lungo un segmento; disegno di debug
Break Hit Result: Impact Point	il punto esatto dell'impatto
Get Surface Type	il tipo di superficie colpita (dal Physical Material)
Physical Surface (Project Settings)	i nomi dei tipi di superficie
Switch on EPhysicalSurface	un'uscita per ogni tipo di superficie
Play Sound at Location	suona un suono in un punto
Sound Wave / Sound Cue	file audio; grafo che combina e modula i suoni
Create Single Cue	crea un Cue dai Sound Wave selezionati (con Random)
Random / Modulator	sceglie un ingresso a caso; varia pitch e volume
Add Sync Marker	marcatore con nome sulla linea del tempo (R, L)
Insert Notify Track / Add Notify	nuova traccia di Notify; inserisce un Notify
Physical Material: Surface Type	il tipo di superficie dell'asset
Phys Material Override	assegna un Physical Material a un componente

Table 1.10.4: I nodi, gli asset e le impostazioni della decima lezione.

- I passi: un **Anim Notify** (AN_Footstep, **Received_Notify**) lancia un **Line Trace By Channel** di 250 centimetri verso il basso; dal Hit Result, **Get Surface Type** e **Switch on EPhysicalSurface** scelgono il **Play Sound at Location** sull'**Impact Point**.
- I tipi di superficie si definiscono nelle **Project Settings** (Physics, Physical Surface: Rock, Water); i **Physical Material** (PM_Rock, PM_Water) li assegnano con il Surface Type; il **Phys Material Override** li applica alle mesh del livello.
- I **Sound Cue** variano i passi: **Random** fra più Sound Wave (*Create Single Cue*) e **Modulator** sul pitch (passo 0,95–1,3, roccia 2–2,2, acqua 1,3–1,5).
- Nelle animazioni di corsa: **Sync Marker** R e L e una **Notify Track** *Sound* con l'AN_Footstep ai frame di appoggio (3 e 13), in tutte e tre le animazioni.

Chapter 1.11

L'interfaccia utente: la barra della vita, il danno e la stamina

Questa è l'**undicesima lezione** del corso breve di **introduzione a Unreal Engine**. Nelle due lezioni precedenti il corso ha costruito un personaggio nuovo, il **bot**, con il suo Animation Blueprint, il Blend Space con il Lean e una State Machine a sette stati. Oggi il personaggio impara a *comunicare con il giocatore*: la lezione è dedicata alla **UI**, l'interfaccia utente, e in particolare alle due informazioni più classiche di un *HUD*, la **barra della vita** e la **barra della stamina**. "Oggi vedremo praticamente la UI", "e oggi vedremo le più basiche".

Per avere qualcosa da mostrare sulle barre servono prima due *sistemi* di gioco, cioè due logiche con dei numeri. Il primo è il **danno**: la torretta delle lezioni passate spara al personaggio, ogni proiettile toglie un punto di vita, e a zero il livello si ricarica. Per farlo si usano due pezzi che Unreal mette già a disposizione, **Apply Damage** (chi fa danno) ed **Event AnyDamage** (chi lo riceve). Il secondo è la **stamina**: il personaggio cammina di default, corre tenendo premuto Shift, e la corsa consuma un valore che si ricarica nel tempo, con due **timer** che si accendono e si spengono a vicenda. Fra i numeri e la grafica c'è il ponte già visto con il pulsante e le luci: gli **Event Dispatcher**. Il personaggio "chiama" un evento ogni volta che la vita o la stamina cambiano; il **Widget Blueprint** dell'interfaccia, che si è "agganciato" a quell'evento con un **Bind**, aggiorna la percentuale della sua **Progress Bar**.

La lezione è **online**, su Zoom, come le precedenti, e anche questa volta è fatta in buona parte di correzioni in diretta: si apre con due progetti di studenti rimasti indietro dalla volta prima (il Lean che non inclina, il bot che non si muove), prosegue con la costruzione della barra della vita, interrotta dai problemi dei presenti, e la pausa ("facciamo 17 e 55") è occupata quasi per intero dalle torrette degli studenti, che non riconoscono il personaggio nuovo. Dopo la pausa si costruisce la stamina, con i suoi timer, e la si collega alla seconda barra. Il **menu**, previsto per oggi, è rimandato: "se non ce la facciamo a fare il menu oggi, lo facciamo la prossima volta, e tanto la prossima volta sarà l'ultima lezione che avremo". Si chiude con un compito facoltativo, le **porte** che si aprono, e con un "buon weekend, buon tutto e buona cena".

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Arreal", "Arrial" e "ArrayLa"; il **Lean** è "lin" e "l'in"; l'**HUD** è "HD", "had", "HAD", "AD", "uhd" e "ashD", e il Widget Blueprint **WB_HUD** è "VB ADD" e "vbad"; **Event AnyDamage** è "EVI damage"; **Apply Damage** è "a play damage"; il **Break Hit Result** è "break it result"; la **Multi Sphere Trace** è "molti sphere"; il **Trace Channel** è "tre channel" e "3C Channel"; la **Progress Bar** è "proga e sbarra" e "Programme bar"; il **Set Percent** è, più di una volta, "7%" e "set present"; il **Custom Event** è "Gaston Event", "Castro Event" e "cast event"; il **PlayerRef** è "playerf", "pragref" e "PlayerFlow"; il **Bind Event to** è "bind e update"; la **CurrentHealth** e la **MaxHealth** sono "CurrentElT" e "MaxElT"; la **StaminaDeduction** è "Dessuction"; il **Max Walk Speed** è "max wall speed" e "max work speed"; l'**Is Falling** è "his falling"; l'**IA_Run** è "iaran", "IRRun" e "iran"; l'**IMC** è "gli MC"; la **Detection Turret** è "Tour de Detection" e "Detection Tour"; il **Projectile Trace** è "progetta il trace"; il proiettile è a tratti "proiettilo" e "progetta il"; **Get Player Pawn** è "get player on" e "Get player, pom"; l'**Event Tick** è "Leventick"; le **World Settings** sono "Word Settings"; **Get World Delta Seconds** è "get word delta seconds"; il bot è "Boto" e "botolino". Dove la ricostruzione è sicura il capitolo

usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione e durante la pausa il riconoscimento vocale produce lunghi blocchi di testo senza senso e ripetuto ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?", "la situazione è questa, la migliore dal 2005", il "governo di Dalai Lama", il "governo di rinforzamento", "le donne che vivevano in passato") e, in due punti, lunghissime file di sillabe ("papapapa", "la la la"): non appartengono alla lezione e sono stati ignorati. La terza: buona parte della lezione è fatta di indicazioni a voce mentre si lavora ("vai su", "tira fuori", "attacca quello sotto"), spesso rivolte allo schermo di uno studente; il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, distingue i tentativi dalle soluzioni, completa i nomi esatti dei nodi e dei pin di Unreal Engine 5, e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono il percorso del danno dal proiettile alla barra, il grafo di Event AnyDamage, l'Event Dispatcher come "camion", le ancore e l'allineamento di un widget, i due grafi che creano e collegano l'HUD, i due timer della stamina e il grafo della stamina che scende. Le tabelle riassumono i pin del sistema di danno, le variabili della giornata, gli errori incontrati durante la lezione e i nodi della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di Marcos Romero e Brenden Sewell (*Blueprints Visual Scripting for Unreal Engine 5* e la seconda edizione dello stesso testo, scritta per Unreal Engine 4), i testi di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'**inizio della lezione**: il Lean di Filippo e il bot di Lucrezia che non si muove (sezione 1.11.1);
2. che cos'è la **UI**, e la dimostrazione del risultato finale (sezione 1.11.2);
3. la **torretta** e il personaggio nuovo (sezione 1.11.3);
4. il **danno**: Apply Damage ed Event AnyDamage (sezione 1.11.4);
5. la **vita**: le variabili, il Clamp, l'Event Dispatcher e il livello che si ricarica (sezione 1.11.5);
6. il **Widget Blueprint**: Designer, Graph, Progress Bar e ancore (sezioni 1.11.6 e 1.11.7);
7. l'**HUD** sullo schermo e il collegamento con il personaggio (sezioni 1.11.8 e 1.11.9);
8. la **barra della stamina** e la **corsa** (sezione 1.11.10);
9. la **pausa**, con le torrette e i bot degli studenti, e gli annunci (sezioni 1.11.11 e 1.11.12);
10. i due **timer** della stamina (sezione 1.11.13);
11. la stamina sulla **barra**, gli ultimi problemi (sezione 1.11.14);
12. il **compito** delle porte e la **chiusura** (sezione 1.11.15).

1.11.1 Prima della lezione: il Lean e il bot che non si muove

Dopo i minuti iniziali di rumore della trascrizione, i saluti ("buon pomeriggio, ti senti?"). In collegamento ci sono, per ora, solo il docente, **Filippo** e **Edoardo**; Filippo ne approfitta: "colgo l'occasione che siamo solo noi due e Edoardo per dirti che dobbiamo risolvere quella cosa". (I nomi, qui, vanno presi con cautela: più tardi, durante la pausa, il docente ricorda questa correzione come quella fatta "con Tommaso"; la trascrizione non permette di stabilire chi dei due fosse.) È il problema rimasto aperto alla fine della lezione precedente: il robottino che, girando, non inclina la testa.

1.11.1.1 Il Lean di Filippo

Escludere le cause. Il metodo del docente è quello di sempre: "cerchiamo tutte le variabili possibili" ed escludiamole una alla volta.

1. **Il Blend Space.** Si apre il Blend Space del bot ("scrivi BS", che la trascrizione rende "BBB Blend Space"): l'asse verticale arriva a 500, quello orizzontale va da -10 a 10, "le animazioni sono quelle, perfetto". "Quindi già una cosa la possiamo escludere."
2. **L'Animation Blueprint.** Nel calcolo del Lean ci sono la *Current Rotation* e la *Previous Rotation*, e il valore viene impostato: "va benissimo".
3. **Il valore.** "Facciamo una cosa, giusto perché così abbiamo più dati": un **Print String** collegato al Lean, "in modo tale che sappiamo effettivamente quanto lo calcola". Siccome il calcolo è nell'Update, "te lo fa continuamente". Il risultato: "è molto piccolo, effettivamente", oscilla fra -1 e 0; muovendosi e girando la camera insieme arriva a "3, 3, 3, 4".

La diagnosi. Il personaggio, girando, "sta facendo così": un *micro movimento*. Con un asse da -10 a 10, un Lean che vale al massimo 1 o 2 sposta il punto del Blend Space di pochissimo, e l'animazione inclinata quasi non si vede. La soluzione è adattare l'asse ai valori reali:

1. **prima i keyframe**: "attenzione, sposta prima i keyframe, perché se no c'è un casino". Le animazioni inclinate, che stavano a -10 e 10, si spostano a -1 e 1;
2. **poi l'asse**: nelle impostazioni dell'asse orizzontale, minimo -1 e massimo 1 (la trascrizione dice "minimo meno 1 e massimo meno 1", un lapsus evidente).

La regola, ripetuta: "ricordati sempre: sposta i keyframe, perché se sposti prima l'asse ti fa una marea di errori". Restringere l'asse con i campioni ancora fuori dal nuovo intervallo li fa finire fuori dalla griglia.

Il segno. Provando, il bot "si piega". "Se tu ti pieghi dall'altra parte, allora le animazioni sono opposte": se girando a destra il bot si inclina a sinistra, e viceversa, "puoi invertire le animazioni", cioè scambiare i due campioni. Per Filippo va bene così: "è solo che il Lean era troppo piccolo, quindi di conseguenza la tua animazione faceva così". Il Print String si toglie.

1.11.1.2 Il bot di Lucrezia

Entrano **Lucrezia** e **Guido**. Il docente stabilisce una regola per il tempo: i problemi delle lezioni passate si vedono "magari in pausa", quelli di questa lezione subito; ma Lucrezia ha un problema bloccante, e si comincia da lei ("vai, Lucrezia, puoi condividere te?"). Dopo qualche difficoltà di rete ("stavo avendo problemi con internet"; "dice che la larghezza di banda ha un po' di problemi"), il sintomo: "il personaggio non mi si muove proprio".

Le cause in superficie. Prima le domande banali, "giusto per escludere tutti i problemi più in superficie":

- gli input di base del template sono stati copiati nel nuovo personaggio? Sì;
- quando si preme Play, si è cliccato sulla **viewport**? "Perché se c'è il mouse fuori dall'editor non ti prende i comandi." "Sì, quella cavolata è perfetta."

La mappatura degli input. Il problema vero è nel **Begin Play** del bot: manca il blocco che carica il contesto di input. "Ti manca praticamente: se vai negli altri character, quelli che abbiamo fatto in precedenza, dovresti copiare un altro blocco." È il nodo che aggiunge l'**Input Mapping Context** al giocatore (nel template Third Person, *Add Mapping Context* sul sottosistema dell'Enhanced Input del Player Controller; la trascrizione lo chiama "camera input", forse leggendo il commento del grafo). "Questo praticamente mette tutta la mappatura degli input, la mette disponibile. Se non c'è questa cosa che viene caricata, gli input non saranno mai visti." L'immagine: "è come se dici *vai da quella parte*, ma non hai la mappa". Copiato il blocco, il bot si muove: "intanto ci si muove, buono così".

L'animazione che non parte. Resta un secondo problema: il bot scivola, ma non anima la camminata. Il docente percorre la catena:

1. **l'Anim Class**: cliccando sulla **Mesh** del personaggio, nei Details, l'Animation Blueprint è quello giusto. Un "piccolo trucchetto": accanto al campo *Anim Class* c'è "una cartellina con la lente di ingrandimento" (il pulsante *Browse to Asset*); cliccandola, il Content Browser va direttamente all'asset, "quello illuminato di azzurro". È ABP_Bot, "è proprio lui";

2. nell'Animation Blueprint, un consiglio a margine: l'**Interp Speed** del calcolo del Lean a 1, "perché è una cosa che ho testato: è molto meglio". L'*Initialize* (la trascrizione dice "Analyse") è giusto, il cast è a BP_Bot;

3. nell'**AnimGraph**, la State Machine *Locomotion*: il bot sta in Idle ma non passa in Walk. La regola Idle → Walk è "GroundSpeed maggiore di zero: va benissimo";

4. **la prova dei dati**: si clicca sulla variabile **GroundSpeed** e, nei Details, si cambia il **Default Value** a **300**; nell'anteprima il bot cammina. "Quindi vuol dire che se la velocità va a 300, l'animazione cambia: quindi non è un problema di animazione, è un problema che non ci sono questi dati che passano." Stessa prova con il **Lean** a -10: l'animazione si inclina, "quindi sappiamo che proprio non è un problema di animazioni". Poi si rimette tutto a zero e si **compila**: l'anteprima mostra i valori dell'ultima compilazione ("a ogni aggiornamento che fai della variabile, lui dice: ok, la giornata non la sto leggendo ancora, quindi sono rimasto al risultato di prima").

Il Print String che non stampa. Nell'Event Graph dell'Animation Blueprint, un Print String subito dopo il *Set GroundSpeed*, "dall'execution", con GroundSpeed collegato alla stringa ("non dal pallino": il pin di esecuzione, non quello del dato). Non stampa niente. "Allora potrebbe essere effettivamente che non è il player che tu stai caricando": nelle **World Settings** ("tutto sulla destra, vicino al pannello Details") la **Default Pawn Class** è BP_Bot, e il Player Controller è quello del template. Allora un **breakpoint**: selezionato il nodo *Set GroundSpeed* ("quello verde"), **F9** lo segna; se in Play l'esecuzione passasse di lì, l'editor si fermerebbe e aprirebbe il grafo. Non si ferma: "vuol dire che proprio non è collegato". Ultimo tentativo: in Play, **F8** stacca la camera dal giocatore e permette di **cliccare sul personaggio** nel livello; nei Details dell'istanza in gioco l'Animation Blueprint è ABP_Bot. "Il character è giusto, l'Animation Blueprint è giusto"; "sarà un micro dettaglio". Si rimanda: "facciamolo in pausa, perché vedo effettivamente che magari è un piccolo dettaglietto di cui non mi sto accorgendo", "tanto oggi fortunatamente lavoriamo su altre cose". La soluzione arriverà durante la pausa (sezione 1.11.11).

Approfondimento: gli strumenti di debug dei Blueprint. I passi del docente sono la sequenza tipica del debug in Unreal, che vale la pena fissare (funzionamento generale dell'engine). Il **Print String** scrive un valore a schermo e nel log: dice *quanto* vale un dato e, se non stampa, dice che *quel ramo non viene eseguito*. Il **breakpoint** (F9 su un nodo, o *Toggle Breakpoint* dal menu contestuale) ferma la simulazione quando l'esecuzione raggiunge il nodo, e permette di ispezionare i valori dei pin passandoci sopra con il mouse; anche qui, il fatto che non scatti è un'informazione. **F8** (*Eject*) separa la camera dal Pawn durante il Play in Editor e consente di selezionare gli actor del livello per leggerne le proprietà *nell'istanza viva*, che possono differire da quelle del Blueprint. Infine, modificare il **Default Value** di una variabile nell'Animation Blueprint e guardare l'anteprima separa il problema dei *dati* da quello delle *animazioni*: se con un valore forzato l'animazione cambia, la State Machine e il Blend Space sono a posto, e il guasto è a monte, nell'Event Graph che dovrebbe calcolare quel valore. Romero e Sewell non dedicano un capitolo ai breakpoint; nel capitolo 1, descrivendo la toolbar dell'editor dei Blueprint, citano però il menu *Debug Object*, che "permette di selezionare un oggetto da debuggare" (se non se ne sceglie nessuno, "fa il debug di qualsiasi oggetto creato con la classe Blueprint corrente"): è utile proprio quando nel livello ci sono più istanze dello stesso Blueprint e si vuole seguire quella giusta.

1.11.2 Che cos'è la UI, e il risultato di oggi

"Allora, ragazzi, partiamo direttamente. Oggi vedremo praticamente la UI." Prima una domanda: chi sa che cosa si intende per UI nei videogiochi?

La risposta di uno studente. "In teoria, tutto quello che hai a schermo": "l'HUD, per dire la barra della vita, la valuta di gioco"; "e la UI invece sono i vari menu e i bottoni, giusto?". Il docente allarga: "generalmente la UI, appunto *user interface*, è tutto ciò che nella schermata dà informazioni al player". "La barra della vita, la barra della stamina, quante monete ho": "tutto ciò che tramite una raffigurazione grafica dà informazioni a un player".

Call of Duty e l'informazione senza barra. Un esempio di design: in *Call of Duty*, quando si era bassi di vita, "diveniva quella schermata rossa, proprio sangue", e il giocatore capiva l'informazione: "stavi per morire, quindi dovevi andare a trovare riparo". Non c'è un numero né una barra, ma è comunque UI: "c'è sempre una figura di design". Infatti "ci sono figure UX, *UX designer*, che impiegano proprio la loro mansione nel trasmettere informazioni in un certo modo attraverso forme grafiche".

La dimostrazione. Il docente condivide il risultato finale. Sotto al personaggio ci sono due barre:

- una **barra rossa**, la vita, collegata alla torretta: "se io adesso vengo colpito dalla torretta, io subisco danni"; quando la vita arriva a zero, "il livello viene ricaricato completamente". "Si potrebbe fare un'animazione di morte, volendo": "in quel caso dovremmo prendere un'animazione da Mixamo e far sì che all'evento del danno, quando sono arrivato a zero, faccio anche un'animazione". "In questo caso non ci serve", "poi si può tutto migliorare";

- una **barra verde**, la stamina. Il personaggio ora *cammina* di default ("adesso il character è molto più lento"); premendo **Shift** corre, "la barra verde si consuma e piano piano si ricarica nel tempo". (Un riquadro di testo che si vede a schermo è "vabbè, quello è il debug".)

Il programma della giornata, in una frase: "andremo a creare una logica all'interno del player, e poi far sì che questa logica, questi valori numerici, comunichino in veste grafica con queste barre", "queste *progress bar*, vengono chiamate".

Approfondimento: HUD, UI e UMG nel libro. Romero e Sewell aprono il capitolo 7 (*Creating Screen UI Elements*) con la stessa distinzione. L'editor di Unreal per l'interfaccia si chiama **UMG** (*Unreal Motion Graphics*) ed è "uno strumento visuale per creare la UI", con cui si fanno "i menu e l'HUD". "Un HUD (*Heads-Up Display*) è un display trasparente che fornisce informazioni senza richiedere all'utente di distogliere lo sguardo dalla vista principale. È stato sviluppato inizialmente per l'aviazione militare." Le barre di vita e stamina che compaiono nell'HUD sono, nel loro lessico, *UI meters*. Il loro progetto d'esempio, uno sparatutto in prima persona, costruisce esattamente le stesse due barre (vita rossa, stamina verde) più due contatori di testo (munizioni e bersagli abbattuti). Il rosso dello schermo di Call of Duty è invece un caso di quella che la teoria del game design chiama interfaccia *diegetica* o *spaziale*: informazione resa dentro il mondo o sull'immagine stessa, invece che con un elemento sovrapposto. Questa è una nozione generale di UI design, non della lezione.

1.11.3 La torretta e il personaggio nuovo

Prima di cominciare, una studentessa segnala: "la torretta non mi traccia". È il problema già visto nella lezione precedente: la torretta è stata costruita per il manichino, e il bot è un personaggio nuovo. Il docente sistema la cosa per tutti, in tre passi.

1. Il tag Player. "Per una questione di tag, assolutamente." Nel Blueprint del personaggio ("BP_Bot, che io ho chiamato Boto"), nei Details in alto a destra si cerca **tag**: nella sezione *Actor*, *Advanced*, **Tags**, si preme il più e si scrive **Player**. "Attenzione con la P maiuscola: comunque come l'hai scritto esattamente nella torretta." Il consiglio è aprire la torretta e controllare quale tag cerca ("io ho *Actor Has Tag*: Player"), perché il confronto è esatto.

2. Get Player Pawn al posto di Other Actor. "Piccolo fix, perché può capitare." Nell'*On Component Begin Overlap* della torretta, il bersaglio veniva preso dall'*Other Actor* dell'evento, cioè da *chiunque* entrasse nella sfera: "ho notato che potrebbe prendere altri actor". La correzione: staccare l'Other Actor dal *Set* della variabile **PlayerTarget** e collegarci invece un **Get Player Pawn** (tasto destro, "get player pawn"). "Così siamo sicuri che prendiamo proprio il player pawn."

3. Le collisioni della Mesh. Ora la torretta segue il bot, ma non spara. Il motivo: il *line trace* della torretta non "vede" il bot, perché la sua mesh non risponde ai canali usati. Nel personaggio, selezionata la **Mesh**, nella sezione **Collision**: il preset passa a **Custom**, e i due canali personalizzati creati nelle lezioni della torretta, **Detection Turret** e **Projectile Trace**, vanno entrambi in **Block**. "In modo tale che la torretta, nel suo line trace, veda la nostra mesh, la blocca, e quindi comincia a sparare." (Nello schermo del docente compare anche un terzo canale, "la torretta precedente": gli studenti dovrebbero avere solo questi due.) "E serve anche il tag Player?" "È il tag Player, quello abbiamo detto prima." "Mi funziona adesso."

Il pivot più basso. Un ultimo consiglio: "visto che prima avevamo il character molto più alto", la torretta va abbassata ("prendete il pivot, lo abbassate un pelino"), perché il bot "è un pelino più basso, più piccolo", e i proiettili devono stare "in traiettoria del player".

1.11.4 Il danno: Apply Damage ed Event AnyDamage

Il piano. "Oggi useremo il nostro character e useremo gli Event Dispatcher." Il ripasso: un Event Dispatcher serve a "chiamare l'evento, bindare l'evento", cioè "collegare gli eventi a un certo momento o a un altro actor". Il piano complessivo: ci saranno delle variabili per la vita e per la stamina, il personaggio farà i calcoli, "e poi tutto questo lo riverserà sulla UI". Il docente lavora su un secondo bot pulito, **BP_Bot2** ("così seguiamo direttamente da vuoto"), dopo un po' di ordine nel grafo. "Partiamo con la vita, ragazzi: è molto meglio."

Event AnyDamage. "Ci sta una particolarità in Unreal": il danno è un concetto già previsto dall'engine, "perché in alcuni giochi può succedere che noi subiamo danno". Nel personaggio, tasto destro, **Event AnyDamage**: "un evento che praticamente, a qualsiasi danno provocato, ci riporta il danno, la tipologia, istigato da chi e il *damage causer*". La distinzione fra gli ultimi due: "l'istigato può essere chi spara, e il damage causer è il proiettile", "chi spara il proiettile e il proiettile che ci ha fatto male". Oggi serve solo la quantità: "ci servirà più la quantità di danno" (tabella 1.11.1).

Apply Damage nel proiettile. "Innanzitutto dobbiamo dire a chi fa danno di fare danno." Il primo istinto è aprire la torretta ("anzi no, scusate, perdonate"): il danno lo fa il **proiettile**, *BP_Projectile* ("io ho sempre il Projectile1"), costruito nella lezione della torretta. Il suo Event Tick lancia, a ogni frame, una **Sphere Trace By Channel** lungo il percorso, "appena viene castato cerca il target". Si aggiunge:

1. dall'**Out Hit** della Sphere Trace, **Break Hit Result** ("facciamo un bel break, perché ci servirà"): la struttura del colpo si apre nei suoi campi;
2. un nodo **Apply Damage**. "Apply Damage ci chiede: chi sto danneggiando?" Fra i campi del Break Hit Result ce n'è uno molto importante, **Hit Actor**: "chi ho colpito, che actor ho colpito". L'Hit Actor va nel pin **Damaged Actor**: "questo actor che ho colpito è l'actor che io sto danneggiando";
3. **Base Damage**: "quanto lo vogliamo danneggiare con un proiettile? Andiamo con 1". "E qua, ragazzi, abbiamo finito."

La prova. Un **Print String** collegato al *Damage* di Event AnyDamage nel bot mostra la relazione: "quando mi colpisce, vedete che mi stampa quanto danno il proiettile mi sta facendo; e questo non è fatto nel player, ma è dal nostro proiettile che comunica al player con l'evento Event AnyDamage". (Prima, il docente deve rimettere il tag e le collisioni al suo bot nuovo: "adesso io ho cambiato character, devo mettergli il tag".) Per ora il danno non ha effetti: "adesso noi dobbiamo mettergli tutta la logica".

Il break che non esce. Una studentessa non riesce a tirare fuori il Break Hit Result dalla sua trace. Il motivo, in condivisione: il nodo era una **Multi Sphere Trace By Channel** ("ah, perché c'hai *molti sphere trace*"): "ti sta facendo un array, ti sta facendo più sfere". Una Multi Trace restituisce un *array* di Hit Result (*Out Hits*), che non si può spezzare direttamente; "a noi ce ne serve una". Sostituita con la **Sphere Trace By Channel**, si ricollegano Start ed End, il **Radius** ("ricordati il radius, 20") e il **Trace Channel**, che deve essere **Projectile Trace**, "in modo tale che sia il trace che abbiamo dedicato al proiettile". Poi Break Hit Result e Apply Damage come sopra.

Nodo	Pin	Significato
Apply Damage	Damaged Actor	l'actor che riceve il danno (qui l'Hit Actor)
	Base Damage	la quantità di danno (qui 1)
	Event Instigator	il Controller responsabile (non usato)
	Damage Causer	l'actor che causa il danno (non usato)
	Damage Type Class	la classe del tipo di danno (non usata)
Event AnyDamage	Damage	la quantità ricevuta: l'unico pin usato a lezione
	Damage Type	il tipo di danno
	Instigated By	"chi spara" (un Controller)
	Damage Causer	"il proiettile"

Table 1.11.1: I due capi del sistema di danno di Unreal. Il proiettile chiama Apply Damage sull'actor colpito; l'engine fa scattare Event AnyDamage in quell'actor, con gli stessi dati. I pin non citati a lezione sono riportati secondo la firma standard dei due nodi.

Approfondimento: il sistema di danno nel libro. Romero e Sewell usano la stessa coppia nel capitolo 10, quando il nemico attacca il giocatore: nel task dell'attacco (*BTTask_DoAttack*) "prendiamo la referenza al Target Actor dalla Blackboard e controlliamo che sia valida prima di chiamare la funzione Apply Damage"; il danno, *Damage*, è una variabile con valore 0,25, "il che significa che un attacco toglierà il 25% della vita del giocatore". Poi, nel personaggio: "useremo Event AnyDamage per diminuire il valore della variabile Player Health". Il vantaggio di questo schema, che è anche quello della lezione, è il disaccoppiamento: chi fa danno non deve sapere *come* il bersaglio gestisce la propria vita, né fare un cast alla sua classe; chiama Apply Damage su un actor qualsiasi, e ogni actor decide che cosa fare nel suo Event AnyDamage (o non fa niente, se non lo implementa). Come nozione generale dell'engine: Apply Damage non fa nulla se l'actor ha *Can be Damaged* disattivato o se il danno è zero; oltre all'AnyDamage esistono le varianti *Point Damage* (con il punto e la direzione del colpo) e *Radial Damage* (per le esplosioni), con i rispettivi nodi *Apply Point Damage* e *Apply Radial Damage*.

1.11.5 La vita: variabili, Clamp, Event Dispatcher e il livello che riparte

Il Print String si toglie, "perché tanto era solo per esempio".

Due variabili. "Ci serviranno due variabili": la **MaxHealth**, "la quantità massima di vita", e la **CurrentHealth**, "la vita attuale", "in modo tale che avremo un massimo su cui limitare il nostro diminuire o avanzare della vita". Entrambe **Float**. Dopo una compilazione, i valori di default: **10** e **10**.

Il calcolo. Che cosa succede alla vita quando arriva un colpo? "Viene sottratto il valore del danno alla vita."

1. un *Get* di CurrentHealth ("ricordo: Ctrl", cioè trascinare la variabile tenendo premuto Ctrl, che crea direttamente il Get);

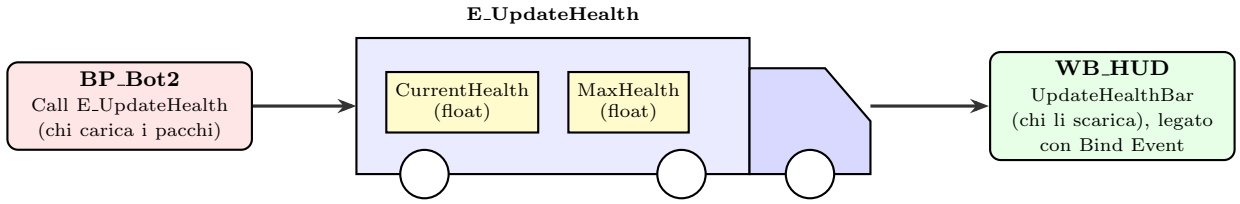
2. un **Subtract** ("meno"), con il **Damage** dell'evento come secondo operando;

3. "per precisione" un **Clamp (float)**: "clampare un valore è fondamentalmente limitarlo tra un minimo e un massimo". Minimo 0, massimo **MaxHealth**, "che sarà sempre 10, quindi sarà una costante";

4. il **Set** di CurrentHealth con il risultato.

Con un Print String di prova: "9, 8, 7, 6, 5, 4, 3, 2, 1, 0. E rimane a 0, perché è clampato a 0". "Questo sarà solamente di debug, perché poi lo andremo a far vedere con la UI."

L'Event Dispatcher E_UpdateHealth. Nel pannello *My Blueprint*, il più accanto a *Event Dispatchers*: **E_UpdateHealth**. Gli servono **due input**. L'immagine del docente: "ricordiamoci che gli Event Dispatcher ci permettono di avere degli input all'interno. Immaginatelo come un *camion*, e quelli sono praticamente i posti per i pacchi" (figura 1.11.1). "Possiamo dare un posto per il pacco variabile CurrentHealth e un posto per il pacco variabile MaxHealth." Cliccato E_UpdateHealth, nei Details, sotto **Inputs**, due volte il più: due **Float**, CurrentHealth e MaxHealth.



Un Event Dispatcher con due input trasporta due valori dal personaggio a tutti gli eventi che si sono legati a lui; un Custom Event creato *dal pin* del Bind ha gli stessi due "posti".

Figure 1.11.1: Il "camion" del docente. L'Event Dispatcher è il mezzo, gli input sono i posti per i pacchi; chi chiama l'evento li riempie con i valori attuali, chi si è legato all'evento li riceve. Schema ricostruito dalla spiegazione a voce.

La morte: minore o uguale. Dopo il Set, un **Branch**: "quando è che moriamo? Moriamo quando la CurrentHealth è minore o uguale a zero". Perché *minore o uguale* e non *uguale*? "Ripeto, perché semplicemente, essendo anche un valore float, noi ci mettiamo minore o uguale per evitare vari errori di calcolo, perché a volte potrebbe succedere: numeri periodici, numeri particolari."

Ricaricare il livello. Sul ramo True: "andiamo a ricaricare il nostro livello". Due nodi:

- **Get Current Level Name**: "andiamo a prendere il nome del livello attuale, perché lo andiamo a ricaricare";

- **Open Level (by Name)**, con il *Return Value* collegato al *Level Name*. "Quindi, di conseguenza, senza doverci andare a vedere come è scritto il livello": "siamo nel livello corrente, prendimi il nome del livello, mi apri il livello tramite il nome, e io glielo passo direttamente".

In Play: dopo dieci colpi "viene ricaricato il livello quando siamo arrivati a zero". "Fino a qui vi funziona, ragazzi?" "Sì."

La chiamata. Infine il dispatcher entra nel grafo: si stacca il collegamento fra il Set e il Branch e, in mezzo, un **Call E_UpdateHealth**, con CurrentHealth e MaxHealth nei suoi due input. "Dobbiamo chiamare il camion per dirgli: ehi, prenditi la CurrentHealth e la MaxHealth attuali." La sequenza completa: "prendo danno, sottraggo il mio danno alla CurrentHealth, chiamo l'evento che, dopo aver aggiornato la mia vita, io vado a comunicare e a trasportare. Adesso, chissà dove" (lo si vedrà fra poco: alla UI). "E se la CurrentHealth è minore o uguale, andiamo a ricaricare il livello" (figura 1.11.2).

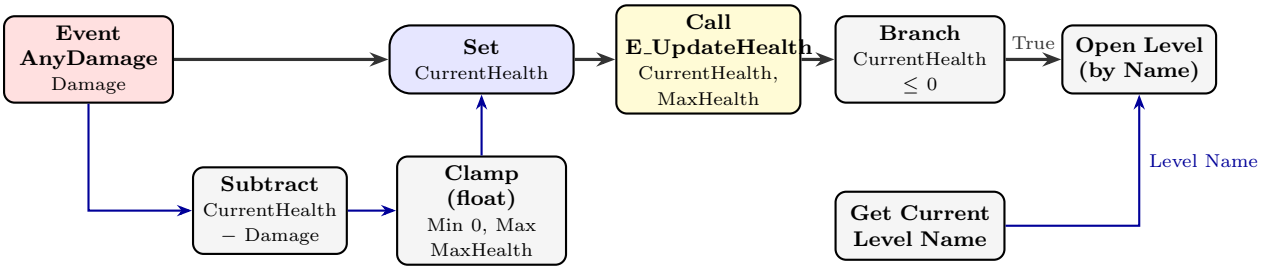


Figure 1.11.2: La vita del bot in BP_Bot2. A ogni Apply Damage ricevuto si sottrae il danno, si limita il risultato fra 0 e MaxHealth e lo si salva; poi si chiama l'Event Dispatcher, che porta i due valori alla UI, e se la vita è finita si riapre il livello corrente. Ricostruzione dalla lezione; i nomi dei pin sono quelli di Unreal Engine 5.

Approfondimento: Clamp, Max e il confronto fra float. Il libro fa lo stesso calcolo con un nodo diverso: nel capitolo 10, "trascinate un filo dal pin di ingresso di SET Player Health e aggiungete un nodo MAX (float). Usiamo questo nodo per assicurarci che Player Health non sia mai minore di 0,0". Il Max taglia solo in basso; il **Clamp** della lezione taglia in basso e in alto, ed è la scelta naturale quando la stessa variabile può anche *salire* (cure, pozioni, ricarica). Per la morte, nel capitolo 11 il libro confronta la vita con lo zero usando **Nearly Equal (float)**, "perché ha una proprietà *Error Tolerance* che serve a confrontare valori con la precisione dei numeri in virgola mobile". È lo stesso problema che il docente risolve con il *minore o uguale*: un float ottenuto da molte sottrazioni (per esempio di 0,1) può fermarsi a un valore come $1{,}49 \cdot 10^{-8}$ invece che a 0, e un confronto di uguaglianza esatta fallirebbe. Con il Clamp a 0 e il $\leq$, la condizione è robusta in entrambi i sensi. Un'osservazione di funzionamento generale: **Open Level** distrugge il mondo corrente e lo ricrea da capo, e con lui il personaggio, la torretta e l'HUD; per questo la vita torna a 10 senza bisogno di "resettarla". Sopravvive solo ciò che vive fuori dal livello, come la Game Instance vista qualche lezione fa.

1.11.6 Il Widget Blueprint

La cartella e l'asset. Nel Content Browser, una cartella **UI** ("io l'ho già creata: tasto destro, ricordo, New Folder"). Poi l'asset: "andiamo a creare un *widget blueprint*. Un widget blueprint praticamente è la controparte di blueprint della UI". Tasto destro, **User Interface**, **Widget Blueprint**; nella finestra che chiede la classe genitore, **User Widget**. Il nome: **WB_HUD** ("io lo metterò 1, perché non posso avere doppioni": WB_HUD1). "E oggi andremo anche a fare il menu con questa stessa logica."

Un mondo a parte. La UI "è un mondo a parte": "addirittura ci sono proprio persone che, anche a livello di programmazione tecnica, fanno UI a manetta". "Se si parla di studi più grandi, c'è sicuramente qualcuno allocato a quel ruolo"; "o in quelli più piccoli ve la prendete voi". La regola generale: "ricordatevi, più piccoli sono gli studi, più roba vi prendete, più dovete uscire dalla vostra comfort zone".

Designer e Graph. Aperto il Widget Blueprint, "avete una schermata che al momento non abbiamo mai visto". In alto a destra due modalità:

- il **Designer**: "uno spazio grafico" in cui si dispongono gli elementi visivi. "Innanzitutto dobbiamo mettere una canvas, *Canvas Panel*, quindi una tela": il contenitore su cui si posizionano liberamente

gli altri elementi. Dalla **Palette** si trascinano dentro "tutte le particolarità grafiche": *Button*, *Image*, *Named Slot*, *Progress Bar*, *Radial Slider*, *Slider*, *Text Box* ("gli slider servono per le opzioni"), e così via. "È uno spazio grafico che vi permette di designare la UI";

- il **Graph**: "ritorniamo un po' nel nostro mondo di Blueprint", quello "delle funzionalità stesse della UI": "cosa fa, che succede quando lo clicco".

Gli eventi di un bottone. Un esempio: trascinato un **Button** nel Designer, nei Details è spuntato **Is Variable**. Nel Graph compare allora come variabile, e in fondo ai suoi Details ci sono "addirittura degli eventi": **On Clicked**, **On Pressed**, **On Released**, **On Hovered**, **On Unhovered**. "Sono tutti eventi collegati alla particolarità di cliccare, premere, rilasciare." Oggi il bottone non serve (lo si toglie); serviranno per il menu.

Approfondimento: il Widget Blueprint nel libro. Romero e Sewell (capitolo 7): "l'editor UMG usa un tipo specializzato di Blueprint chiamato *Widget Blueprint*"; si crea con "User Interface — Widget Blueprint". Il loro HUD è costruito con dei contenitori: un *Horizontal Box* (*Player Stats*) con dentro due *Vertical Box*, uno per le scritte (*Text*) e uno per le barre (*Progress Bar*). "Il grande rettangolo nella vista del grafo rappresenta i confini dello schermo che il giocatore vedrà, ed è chiamato *canvas*: è l'oggetto Canvas Panel al livello più alto della Hierarchy." La lezione usa un solo Canvas Panel con le barre posizionate a mano, che è più semplice; i box del libro diventano utili quando gli elementi sono molti e devono restare allineati fra loro (etichette accanto alle barre, contatori in fila).

1.11.7 La Progress Bar e le ancore

La barra. "Quello che ci servirà adesso è la **Progress Bar**": la si trascina dalla Palette nel Designer e la si allarga "un pelino", afferrandola sui bordi. Per muoversi nella tela, il tasto destro del mouse trascina la vista e la rotellina zooma.

Le ancore. "Questo aggeggino qua" è l'**ancora**. "Le ancore praticamente riguardano la risoluzione dello schermo": "è il pivot della UI, quindi di uno spazio 2D, non di uno spazio 3D". La posizione di un elemento è espressa *rispetto alla sua ancora*; se l'ancora resta in alto a sinistra e la barra sta in basso, "visto che la risoluzione cambia" ("magari avete uno schermo 720 per qualcosa") la barra, "secondo quel pivot, va a quel paese": "si comincia a mixare tutto quanto". Poiché la barra della vita sta sotto lo schermo, l'ancora va **in basso al centro**: cliccata la Progress Bar, nei Details, **Anchors**, "c'è una raffigurazione grafica per tutto quanto", e si sceglie quella in basso al centro.

Il Canvas che non entra. Uno studente non riesce ad aggiungere il Canvas Panel: "me lo trova il pannello, ma non me lo fa mettere, mi dà tipo errore". "Mi dava errore pure a me", risponde un compagno: "se cancelli la Progress Bar e metti prima il pannello, te lo fa mettere". Lo studente aveva trascinato la Progress Bar in un Widget vuoto, e la barra era diventata la *radice* del widget: un elemento come la Progress Bar non può contenere figli, e al livello più alto c'è posto per un solo elemento. Si cancella la barra, si mette il Canvas Panel, poi la barra dentro il Canvas. "Ah, ok. A posto."

Percent e colore. "Quale sarà il valore che dovremo influenzare dalla nostra Progress Bar? È il **Percent**", nella sezione *Progress*. Trascinando il valore "si riempie e si svuota, dipendente se sta a 0 o se sta a 1". Poi "una bella botta creativa": **Fill Color and Opacity**, la ruota dei colori, "di solito la barra della vita è di quel rosso": il docente sceglie "un rosso abbastanza sangue", poi regolabile in chiarezza e saturazione. "Importante: lasciatela a 1, perché tanto ci servirà": la barra parte piena.

Centrare la barra: Position e Alignment. Più tardi, durante la costruzione del grafo, Guido chiede "una cosa più estetica": "c'è un modo nel designer di centrare la barra? L'ho fatto a mano, però ci ho messo un po'." La risposta, provando i valori:

- **Position X** e **Position Y** sono relative all'ancora: si mettono entrambe a **0**, così la barra sta *sull'ancora*;
- **Alignment** è il *pivot* dell'elemento stesso: con 0,0 l'elemento si aggancia all'ancora con il suo angolo in alto a sinistra ("te lo metti in alto a sinistra, come all'angolo"); con **0,5** in X si aggancia con il suo centro orizzontale, "perfettamente al centro"; in Y il docente prova 1,8, poi 2, poi **3**: il punto di aggancio sta tre altezze sotto il bordo superiore della barra, quindi la barra si solleva e il suo bordo inferiore resta due altezze sopra l'ancora.

Guido riassume: "position X e Y sono relative all'anchor, e l'allineamento è il pivot". "Esatto. Bravissimo." Con Position a zero e Alignment 0,5 e 3, la barra è centrata (figura 1.11.3); gli slider dei campi aiutano a essere precisi.

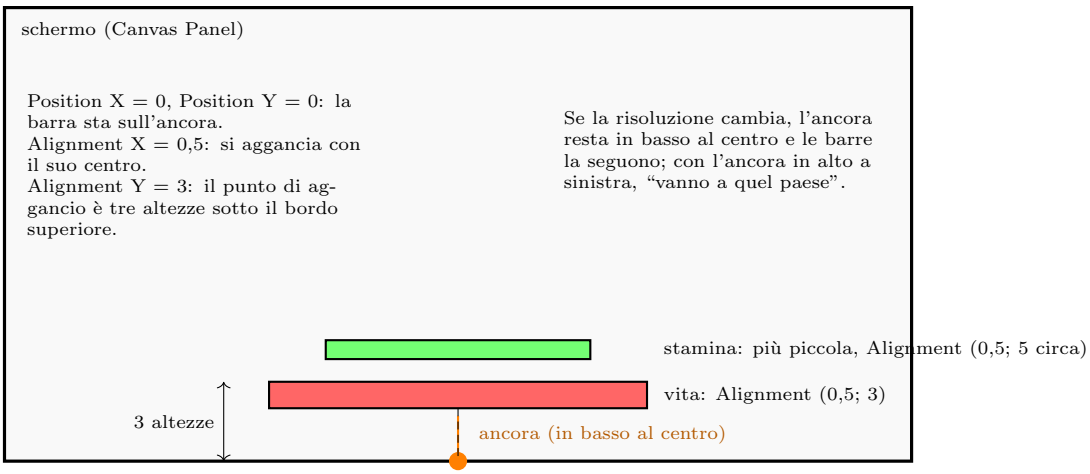


Figure 1.11.3: Ancore, posizione e allineamento di un widget nel Canvas Panel. La posizione si misura dall'ancora, l'allineamento sposta il punto dell'elemento che si aggancia all'ancora, in frazioni della sua dimensione. Schema ricostruito; proporzioni indicative.

Approfondimento: ancore e Percent nel libro. Romero e Sewell (capitolo 7): "poiché le dimensioni e i rapporti degli schermi possono variare, vogliamo assicurarci che gli elementi della UI restino nella stessa posizione relativa sullo schermo. Le ancore servono a definire la posizione desiderata di un widget sul canvas, indipendentemente dalla dimensione dello schermo." Il simbolo a forma di fiore che si vede nel Designer si chiama *Anchor Medallion*, "e rappresenta la posizione dell'ancora dell'elemento selezionato"; la seconda edizione aggiunge che si può anche trascinare direttamente il fiore nel punto voluto, o espandere *Anchors* per i controlli di precisione. Uno dei quesiti di fine capitolo chiede se "i valori X e Y della posizione di un elemento in un canvas sono sempre relativi all'angolo in alto a sinistra dello schermo": la risposta è no, sono relativi all'ancora, che è esattamente ciò che Guido ha scoperto. Sul Percent: "l'intervallo dei valori va da 0,0 (vuota) a 1,0 (piena)". Il libro anticipa anche il motivo per cui serve una divisione: le sue variabili di vita e stamina hanno valore di default 1,0 proprio perché "questo valore è usato dai UI meters, che mostrano il grado di riempimento fra 0,0 e 1,0". La lezione invece tiene la vita in punti (10) e la stamina in centesimi (100), e converte al momento di disegnare.

1.11.8 L'HUD sullo schermo: Initialize, Create Widget e Add to Viewport

L'evento d'inizializzazione. Nel **Graph** del widget ("in alto a destra, Designer e Graph: se clicchi su Graph vai direttamente là"), si tolgono gli eventi predefiniti e si crea un **Custom Event**: **InitializeHealthBar**. Gli serve un input, "la referenza stessa al player": nei Details dell'evento, sotto *Inputs*, il più,

nome **PlayerRef**, tipo la classe del proprio personaggio ("io ho BP_Bot2, ma deve essere il nome del vostro character"), **Object Reference**: "quella azzurra, deve essere azzurra". A che serve? "Chiamerà l'Event Dispatcher": il personaggio dirà "ehi, fammi esistere questa UI, mettimela sullo schermo", e a quel punto bisogna "bindare l'evento al fatto di inizializzare la barra, quindi dargli subito il valore di quant'è la vita". (A chi chiede "com'è che sotto c'è la freccia bianca?": è l'evento con il suo input, che si aggiunge dai Details.)

Il Begin Play del personaggio. "Innanzitutto, buona regola, dobbiamo passargli la PlayerRef." In BP_Bot2, all'**Event BeginPlay**: "quando il gioco inizia, vogliamo che l'HUD, quindi questa schermata, la sua Progress Bar e tutto quanto, vada a esistere nel mondo, nella nostra viewport".

1. **Create Widget** ("scusate, non *Add*: è Create Widget"). Il pin **Class** chiede "la tipologia di asset che andiamo a creare": WB_HUD;
2. il **Return Value** si promuove a variabile, **AsHUD**: "non lo useremo, però è una buona cosa sempre fare una referenza, perché magari volete comunicare altre volte con il nostro HUD e cambiargli dei valori";
3. "questo lo crea, ma non lo mette nella viewport": serve **Add to Viewport**, con l'HUD come target. In Play, "ecco a voi che la nostra barra rossa è apparsa a schermo";
4. infine, dalla variabile AsHUD, la chiamata a **InitializeHealthBar**, con **PlayerRef** collegato a un **Get a Reference to Self** ("fate *self*, vi appare Get a Reference to Self"). "Abbiamo detto: ehi, aggiungi questo widget alla viewport e inizializza."

(Durante questa parte il docente ha qualche problema di connessione: "scusa, ho visto adesso la chat; sto avendo problemi, devo fare un po' di manutenzione".)

1.11.9 Il collegamento: Bind Event e Set Percent

Il Bind. "Ci manca ovviamente la funzione collegata": bisogna fare in modo che l'inizializzazione *agganci* l'aggiornamento della barra all'aggiornamento della vita. Nel Graph del widget, dopo InitializeHealthBar, dal pin PlayerRef: **Bind Event to E_UpdateHealth** (la trascrizione sente "bind e update"). Compilando dà errore, "giustamente": "mi serve un evento: che evento devo collegare al fatto che io aggiorno la mia variabile Health?". L'evento si crea così: dal pin rosso **Event** del Bind si trascina un filo e si sceglie **Add Custom Event**. "Mi raccomando: non fate un Custom Event esterno, tirate fuori proprio da Event e fate Custom Event, perché vi tira fuori e si porta dietro i due valori che gli abbiamo dato di input, cioè CurrentHealth e MaxHealth." Il nome: **UpdateHealthBar**.

Che cosa succede. Una studentessa riassume: "quindi quando viene inizializzato, in automatico fa anche Update Health?". "Esatto." Quando il gioco inizia, l'HUD "binda l'evento di aggiornare la barra al nostro Update Health, cioè al valore numerico che si aggiorna". "Creiamo il collegamento": da quel momento, "quando si aggiorna effettivamente il nostro numero, la nostra variabile, allora avviene anche l'update della barra".

L'evento senza posti. "A me il Bind Event non me l'ha dato": la studentessa aveva creato prima, per sbaglio, un Custom Event separato e l'aveva collegato al Bind; cancellato quello, il Bind risultava "ancora delegato". "Elimina quello che hai adesso; tira fuori da Event, Custom Event." La spiegazione: un Custom Event creato a mano "non ti dà quei valori di input, CurrentHealth e MaxHealth"; "questo qua deve essere creato tramite il collegamento, perché noi gli abbiamo messo degli input". "Non deve essere vuoto: si deve portare dietro i posti per la CurrentHealth e la MaxHealth. Ricordiamoci il camion."

Rinominare la barra. "Per ordine": nel Designer, nella **Hierarchy** ("ci permette di avere tutte le nostre cosine che abbiamo messo nella UI, tutte belle elencate"), si seleziona la Progress Bar e con **F2** la si rinomina **ProgressBar_Health**.

Set Percent. Nel Graph, la variabile ProgressBar_Health si trascina nell'evento UpdateHealthBar e da lei si tira fuori **Set Percent**. "In questo caso però si tratta di una percentuale, quindi è un valore da 0 a 1": il valore è **CurrentHealth diviso MaxHealth**, i due pin dell'evento in un **Divide**. In Play, "la mia barra diminuisce sempre di più, e boom, eccoci qua" (figure 1.11.4 e 1.11.5).

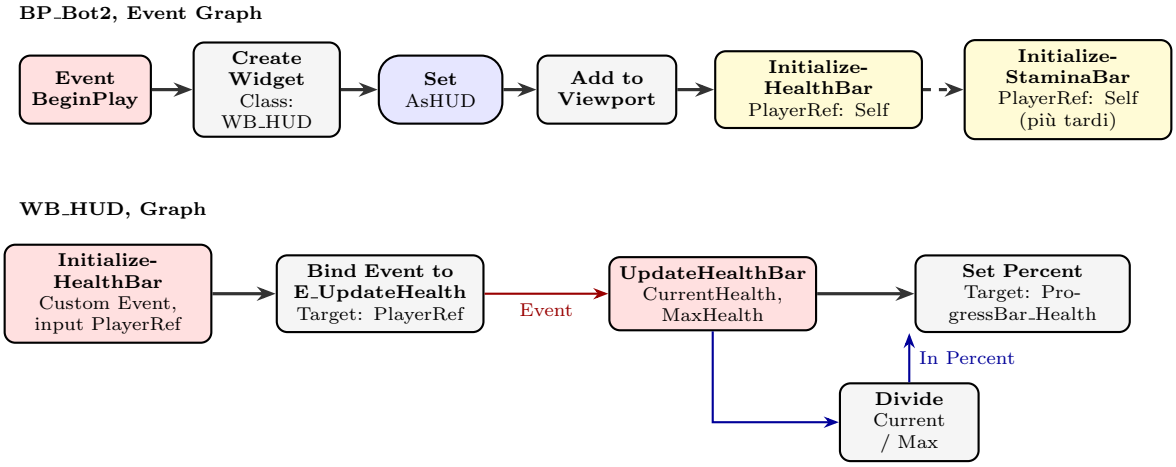


Figure 1.11.4: I due lati dell'HUD. In alto, il personaggio crea il widget, se lo salva, lo aggiunge alla viewport e gli passa una referenza a se stesso. In basso, il widget usa quella referenza per legare il proprio evento UpdateHealthBar all'Event Dispatcher del personaggio: da lì in poi, ogni Call E_UpdateHealth esegue Set Percent. La chiamata tratteggiata si aggiunge con la stamina. Schema ricostruito.

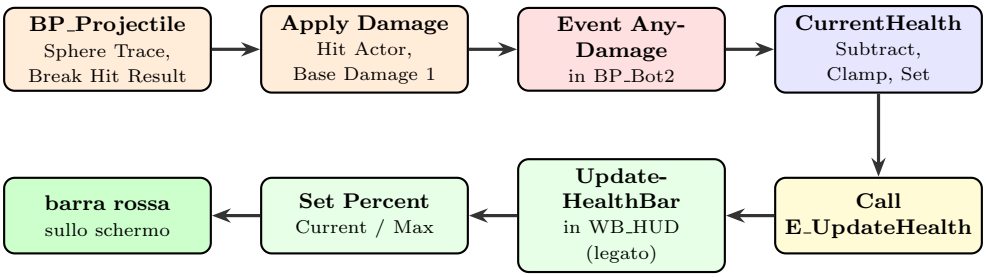


Figure 1.11.5: Il percorso completo di un colpo, dal proiettile alla barra. Il sistema di danno di Unreal collega il proiettile al personaggio; l'Event Dispatcher collega il personaggio all'HUD. Schema riassuntivo.

I problemi dei presenti. "Ci siete, ragazzi?" Tre casi.

- **Set Percent introvabile.** "Non mi dà il Set Percent", e nemmeno i Details per cercarlo. Primo controllo: nel Designer, la Progress Bar deve avere la spunta **Is Variable**, altrimenti nel Graph non esiste come variabile. I Details mancanti si riaprono da **Window**, **Details**. La spunta c'era; il vero errore è un altro: lo studente cercava Set Percent trascinando da un altro nodo. "Ah, ok, lo facevo da qua": Set Percent si trova solo *tirando fuori dalla variabile* ProgressBar_Health, che va presa dalla sezione *Variables* ("se non ce l'hai, c'è la tendina") e trascinata nel grafo. Per la divisione: "tiri fuori dalla CurrentHealth e fai lo slash", oppure si scrive *divide*; poi "questo diviso per quell'altro".

- **La torretta che non spara**, a un'altra studentessa: rimandata alla pausa.

- **La barra che non scende.** "La vita la perdo, perché poi respawno; tuttavia graficamente non scende la barra." Il Bind c'è, il danno è 1 ("l'ho messo come variabile"), la vita è 10. Un Print String dopo il Set Percent, "attaccalo a In String" ("collega quello verde a quello rosa"), non stampa niente. Allora **F9** sull'evento UpdateHealthBar: "dice: sta facendo questo evento?". Il breakpoint non scatta. In BP_Bot2 si scopre perché: nel Begin Play mancava la chiamata a **InitializeHealthBar** dopo Add to Viewport. "Non stai dicendo al gioco che, quando inizia, se io ho dato la vita, mi aggiorni pure la barra." Si aggiunge da AsHUD, con Self nel PlayerRef; si toglie il breakpoint ("se no te lo farà ogni volta"). "Perfetto, grazie."

Il portatile surriscaldato. Un altro studente non riesce a seguire: "sta andando a scatti", "è troppo surriscaldato il computer". Il consiglio, per tutti: "per chi ha un portatile, prendetevi quelle basette con le ventole": "col portatile quello diventa un vulcano a un certo punto". Lo studente ce l'ha già: "ah, ok. Quindi sta proprio andando pesantissimo". "Lascialo riposare il nostro soldato, tranquillo: tanto poi c'è la registrazione."

Approfondimento: Bind Event contro i Binding del libro. Qui la lezione e il libro scelgono due strade diverse, e vale la pena capire perché. Romero e Sewell (capitolo 7) usano i **Binding** delle proprietà: accanto al campo *Percent* della Progress Bar c'è un pulsante *Bind*; scegliendo *Create Binding*, l'editor crea una funzione che restituisce il valore, in cui gli autori mettono *Get Player Character*, *Cast To FirstPersonCharacter* e il Get della vita. "I Binding ci danno la possibilità di legare variabili o funzioni di un Blueprint a un Widget. Ogni volta che la variabile o la funzione è aggiornata, il cambiamento si riflette automaticamente nel Widget." Il prezzo, per il funzionamento generale di UMG, è che una funzione di Binding viene valutata *a ogni frame*, che il valore sia cambiato o no (e, nell'esempio del libro, con un cast a ogni valutazione). Lo schema della lezione è *a eventi*: la barra si aggiorna solo quando il personaggio chiama E_UpdateHealth, cioè quando la vita cambia davvero. Per due barre la differenza di prestazioni è trascurabile; diventa importante negli HUD con molti elementi, e soprattutto rende esplicito *chi* aggiorna *che cosa*. Sul Bind Event, invece, il libro (capitolo 4) dice la stessa cosa del docente: "c'è un nodo Bind Event che lega un evento a un altro evento o a un Event Dispatcher, che può stare in un altro Blueprint. Quando un evento è chiamato, tutti gli altri eventi legati a lui sono chiamati", e il Custom Event si crea trascinando "dal pin rosso del nodo Bind Event" e scegliendo *Add Custom Event*. Un dettaglio generale: un Custom Event creato dal pin riceve automaticamente la *firma* del dispatcher (i suoi input); se in seguito si aggiungono input al dispatcher, gli eventi legati vanno aggiornati, e l'editor lo segnala con un errore di firma alla compilazione.

1.11.10 La seconda barra e la corsa

"Abbiamo un quarto d'oretta. Sai che facciamo? Andiamo a creare la nostra barra della stamina e facciamo insieme la stamina, così dopo magari facciamo qualcos'altro, che è il menu in questo caso."

La barra copiata. "Più che metterne una nuova", nel Designer di WB_HUD si copia la barra della vita con **Ctrl+C** e **Ctrl+V**. La copia ha ancora l'ancora in basso al centro; si azzerano le posizioni, **Alignment** X a **0,5** e Y più alto, perché stia sopra la barra rossa (il docente dice "non 3, ma la mettiamo tipo a 2, e facciamo 5", poi "facciamo 5"), la si fa "un pochino più piccola" (con i campi della dimensione, "1,8, ma sì, 2": "questi rettangolini servono per essere un po' più precisi"), e verde. "Se volete, ragazzi, sbizzarritevi: i colori, proprio pastelli in mano, fate come volete."

InitializeStaminaBar. "Facciamo la stessa identica cosa per l'inizializzazione della barra": nel Graph, un altro Custom Event, **InitializeStaminaBar**, con lo stesso input **PlayerRef** ("fortunatamente, se avete già fatto l'input, vi dice già: ce l'ho, PlayerRef"). "Per adesso lo lasciamo così, perché dobbiamo creare un altro Event Dispatcher, che sarà invece l'update della stamina."

IA_Run. Torniamo nel bot: "dobbiamo far correre il nostro player". Serve un'**Input Action** nuova, **IA_Run**, accanto a IA_Look, IA_Move e IA_Jump; la si era fatta la volta scorsa per il cavaliere, e "for-

tunatamente" basta una volta: "questa è la bellezza del context: la mappatura degli input è separata dal player", e "la posso passare da player a player". Per chi non ce l'ha:

1. da un'azione qualsiasi, tasto destro, **Show in Folder View**; si **duplica** IA_Jump e la si chiama IA_Run. "Se non sbaglio, Jump e Run sono uguali": entrambe hanno un valore booleano (*Digital*);

2. nell'**IMC** (*Input Mapping Context*), il più, IA_Run, e nella tendina la **tastiera**: "cliccate sulla tastiera, premete Shift", "in modo da dire: guarda che questo è l'input collegato alla corsa".

"Io l'ho spiegato molto velocemente, perché so che, se qualcuno nel caso non avesse visto, poi mi dica."

Gli stati dell'evento. Nel bot, l'evento **IA_Run** mostra le sue uscite (*Triggered*, *Started*, *Ongoing*, *Canceled*, *Completed*); "in questo caso ci interesseranno solamente lo **Started** e il **Completed**": l'inizio e la fine della pressione.

Camminare di default. "Per dire già che il player deve camminare di default", si guarda il Blend Space del bot: la camminata è a 250, "facciamo 300, un po' più alto". Nel **Character Movement**, categoria *Character Movement: Walking*, **Max Walk Speed** a **300**: "da adesso in poi il nostro character camminerà". (Se i passi non suonano, "lo fixiamo poi": il docente aveva aggiunto i suoni dei passi anche all'animazione di camminata prima della lezione, "perché prima solamente correvamo".)

Correre. Due **Set Max Walk Speed**, entrambi con il Character Movement come target ("ricordatevi di tutti e due di collegarli"): uno allo **Started**, a **500**, uno al **Completed**, a **300**. (Il docente dice prima 600, poi "500 era? Facciamo 500, va bene", il valore massimo del Blend Space.) In Play: "cammino; Shift, corro; Shift, lascio". "Sicuramente possiamo mettere un po' di smooth all'animazione", con un *blend in* e un *blend out*: "questo è un fix che possiamo fare dopo, tranquillamente".

La stamina di Dark Souls. "Qualcuno di voi ha in mente Dark Souls? La stamina di Dark Souls, o in generale, molto spesso la stamina funziona in questo modo": "viene persa durante un quantitativo di tempo, se si fa una certa azione, in questo caso correre: viene levata gradualmente", e quando l'azione smette ritorna su piano piano. "Visto che parliamo di tempo, useremo due **timer**", come per la torretta, "per far sì che queste due cose vadano a scalare nel tempo". Mancano cinque minuti alla pausa: "facciamo adesso un piccolo timer, oppure vogliamo andare già in pausa?". Si va in pausa: "la prova è 25", "facciamo 17 e 55 e ci rivediamo".

Approfondimento: lo sprint nel libro. Romero e Sewell costruiscono lo sprint nel capitolo 6 con il sistema di input precedente (l'*Action Mapping* "Sprint" sul tasto Shift sinistro, gli eventi *Pressed* e *Released*) e due *Set Max Walk Speed*, esattamente come la lezione con l'Enhanced Input (Started e Completed). Nel capitolo 8 fanno un'osservazione di design che spiega perché la stamina serve: senza un vincolo, "non ci sarebbe niente a scoraggiare il giocatore dal tenere sempre premuto Shift mentre si muove"; "se un'opzione è così attraente che il giocatore si sente costretto a usarla sempre, allora non aumenta il numero di scelte interessanti". "Dal punto di vista del giocatore, il risultato sarebbe lo stesso se aumentassimo semplicemente la velocità di base fino a quella dello sprint." La stamina trasforma lo sprint in una *risorsa*: correre ora significa non poter correre fra poco. È la stessa logica della stamina di Dark Souls, dove la stessa barra è condivisa da corsa, schivata, parata e attacco, e ogni scelta ha un costo.

1.11.11 La pausa: le torrette e i bot degli studenti

La pausa, come spesso in questo corso, diventa uno sportello di correzioni. Il docente resta collegato e guarda i progetti uno alla volta, in condivisione di schermo ("quando condividi, fai *condividi schermo*, non condividere Unreal, perché è più facile: stiamo schermando più grande").

Lo Shift che non corre. Il primo studente: "se mi muovo normalmente cammina, se premo Shift non corre". "Allora c'hai qualche cosa che non va nell'IA_Run." La cosa resta in sospeso: il problema dello stesso tipo tornerà, più grave, alla fine della lezione (sezione 1.11.14).

1.11.11.1 La torretta di Tommaso

"Io entro, mi vede, ma non spara." Il tag Player c'è.

1. **La collisione della mesh del bot**: "non l'avevo messa". Mesh, Collision, Custom, Detection Turret in Block (e Projectile Trace: "va benissimo"). Ora la torretta lo segue, ma non spara.

2. **Il Trace Channel**: nella torretta, il trace di rilevamento usava il canale sbagliato. "Cambia Trace Channel" a **Detection Turret** ("no, no, torna indietro; Ctrl+Z"). "Ok, benissimo."

3. **La funzione dello spawn**: alla torretta di Tommaso manca proprio la funzione che fa nascere il proiettile. Il docente gliela passa con "un trucchettino": "la cosa bella dei Blueprint è che puoi incollarli in un file di testo, copiarli tutti quanti". Seleziona i nodi della sua funzione, Ctrl+C, li incolla in un file *.txt* ("Fix, funzione Spawn") e glielo manda su Discord, in privato. Tommaso scarica il file, ne copia tutto il testo e lo incolla con Ctrl+V nel grafo di Unreal: i nodi ricompaiono.

4. **Sistemare l'incollato**: "adesso c'hai degli errori, eh? Perché devi sistemarlo"; "prima fix, poi ordina". La funzione si rinomina con F2 **Spawn Projectile**. Servono i due punti di uscita: nei Components, Add, due **Scene Component**, **ProjectileSpawn1** e **ProjectileSpawn2** (copiati con Ctrl+C e Ctrl+V), agganciati al **cannone** della torretta e spostati "ai bocchettoni della mesh", "poco fuori la canna".

5. **Lo Spawn Actor rifatto**: il nodo incollato dà un errore sull'*Spawn Actor* ("non ti spawno nessun actor"); l'errore sull'*Instigator* invece "va bene, non ci interessa". "Facciamo prima": si cancella e si rifà **Spawn Actor from Class**; tasto destro sullo *Spawn Transform*, **Split Struct Pin**; si collegano Location e Rotation dei due Scene; la **Class** è il proiettile, BP_Projectile.

6. Una curiosità di Tommaso, nel frattempo: "pure la barra della stamina, io gli ho dato un colore, ma me la dà grigia". "Forse è vuota": il *Percent* è a 0, e una Progress Bar vuota mostra solo lo sfondo. "Faccio 1."

7. **Chiamare la funzione**: la torretta ancora non spara, "perché non spawni": nello *Shoot*, al posto delle due vecchie chiamate, va chiamata la nuova **Spawn Projectile**.

8. **Le collisioni della torretta**: il proiettile non deve fermarsi sulla torretta stessa. Sul cannone, sulla sfera di rilevamento e sul pivot, il canale Projectile Trace va su **Ignore** ("devi ignorare tutto, anche il pivot").

La torretta di Tommaso ancora non spara; "aspetta, adesso facciamo fare Lucrezia, che è un po' che aspetta". Si riprenderà dopo la pausa.

1.11.11.2 Il bot di Lucrezia, risolto

Il Validated Get sulla variabile sbagliata. Con Shift il bot di Lucrezia ora "va più veloce": "quello almeno va benissimo"; quindi il personaggio si muove, "è un problema di animazioni". Il docente esclude un doppio scheletro ("non vorrei che avessi un doppio scheletro e te l'avessi fatto su un altro": ne ha uno solo), poi mette un Print String sull'uscita **Is Not Valid** del Validated Get, all'inizio dell'Update. Stampa: "ecco il nostro problema: la referenza a questo actor non è valida". Il motivo: "hai messo nel Validated Get la variabile del PB_Bot, non del BP_Bot, che sono due diverse": nell'Animation Blueprint c'erano due variabili quasi omonime, e il Validated Get controllava quella che l'Initialize non riempiva mai. Si cancella, si prende la variabile giusta (**AsBP_Bot**), tasto destro, **Convert to Validated Get**, si ricollega. "Dovremmo aver scoperto l'arcano." "Era lui."

La regola di ritorno all'Idle. Ora il bot anima, ma "continua a camminare sul posto" quando si ferma. Nella State Machine, la regola Walk → Idle era *GroundSpeed < 0*: "attenzione, non è minore, è **minore o uguale**", perché una velocità negativa non esiste (la lunghezza di un vettore è sempre ≥ 0), e con il minore stretto la regola non è mai vera ("solo se sarà meno uno la velocità").

Il Lean, di nuovo. Correndo e girando, il bot "non si inclina tantissimo": la stessa correzione fatta a inizio lezione ("come abbiamo fatto anche con Tommaso": la trascrizione attribuisce qui a Tommaso il Lean corretto all'inizio). Nel Blend Space i campioni da ± 10 a ± 1, poi l'asse, nelle *Axis Settings* dei Details, da -1 a 1. "Ecco che si piega leggermente."

La torretta di Lucrezia. Poi la torretta, aperta con *Edit BP_Turret* dal livello ("lo fai diretto"). "Questo è un po' incasinato, eh? Vabbè, tranquillo." In ordine:

- **Begin Overlap**: il fix di inizio lezione, Get Player Pawn al posto dell'Other Actor nel Set di PlayerTarget;
- **Move Turret** (la rotazione verso il bersaglio): il nodo di rotazione usato era un altro, e si sostituisce con **Find Relative Look at Rotation**, che vuole uno *Start Transform* ("non ti serve il Get Actor Rotation, ma devi invocare il **Get Actor Transform**", "pinnetto arancione per pinnetto arancione") e una *Target Location*, il **Get Actor Location** del PlayerTarget. Del risultato, con *Split Struct Pin*, si usa la **Z** (lo Yaw) come *Target* dell'interpolazione; un vecchio "meno" non serve più; nel *Delta Time* va **Get World Delta Seconds**. La trascrizione, qui, è troppo frammentaria per ricostruire il grafo nodo per nodo; il senso è quello della rotazione della torretta costruita nelle lezioni precedenti;
- **Shoot**: Branch, Sequence, i due *Do Once* con lo Spawn Projectile e il Delay, il *Fire Rate*: "va benissimo";
- un secondo *On Component Begin Overlap* rimasto da una prova ("probabilmente una cosa che avevo cancellato", "l'avevo spostata"): si elimina; l'*End Overlap* e il Tick si tengono;
- **il tag**: la torretta ancora non la segue. Il tag Player era stato messo nei *Component Tags* della mesh: "attenzione, non della mesh, ma quello sotto": i **Tags** dell'*actor*, e con la stessa maiuscola.

Il proiettile assassino. Ora la torretta spara ("sei un po' alta"), ma emergono tre problemi del proiettile, uno dopo l'altro.

1. I proiettili urtano la camera del personaggio: sulla *Sphere* del proiettile, Collision, **Camera** su **Ignore**.
2. I proiettili "non si distruggono se il character gli va addosso": la Sphere Trace aveva **Radius 0**, "quindi è grande niente". A **20**.
3. Ora un colpo e il bot "si blocca proprio tutto", anche se la vita non è finita. Nel proiettile, dopo il colpo, c'era un **Destroy Actor** con l'*Hit Actor* come target (un ramo chiamato *Kill Target*): "se colpisco qualcosa, distruggimi quello che ho colpito". Il proiettile cancellava il personaggio dal mondo. Al suo posto: **Apply Damage** (Hit Actor nel Damaged Actor, Base Damage 1), e poi **Destroy Actor** senza target, cioè *self*: "quando collide con qualcosa, distruggi il proiettile: si autodistrugge".

La vita a zero e la NewVar. Ora il bot "muore praticamente subito": la **MaxHealth** aveva default 0, e il Clamp riduceva a zero qualunque vita. MaxHealth a 10. Poi il contrario: il primo colpo toglie vita, "poi adesso è immortale". Nell'Event AnyDamage c'era un Set su una variabile **NewVar**, creata per sbaglio, invece che sulla CurrentHealth: "stai praticamente mettendo una variabile nuova come vita". Si sostituisce con *Set CurrentHealth*, ricollegando il Return Value del Clamp e il valore al Branch del minore o uguale. "Perfetto. Grazie mille."

Approfondimento: i Blueprint come testo. Il "trucchettino" del file di testo funziona perché Unreal copia i nodi negli appunti in un formato testuale (lo stesso formato *T3D* usato per copiare gli actor fra livelli): ogni nodo diventa un blocco `Begin Object ... End Object` con la sua classe, la posizione nel grafo, i valori dei pin e i collegamenti. Incollando quel testo in un altro grafo, anche di un altro progetto, l'editor ricrea i nodi. È il modo più rapido per condividere un pezzo di logica in chat o su un forum, ed esistono siti che lo visualizzano come immagine. Il limite è quello visto a lezione: i riferimenti a variabili, componenti e funzioni del Blueprint d'origine non esistono nel Blueprint di destinazione, e i nodi che li usano arrivano "rotti" e vanno ricollegati o ricreati (come gli Scene Component e lo Spawn Actor di Tommaso). Questa è una nozione generale dell'editor, non trattata nei due libri.

1.11.12 Dopo la pausa: ancora la torretta, e gli annunci

"Eccoci qua, ragazzi, siamo ritornati." Tommaso ha provato a togliere e rimettere la torretta: niente.

L'Event Tick che manca. "Io credo che sia *not valid* il PlayerTarget, probabilmente." Il Begin Overlap e il timer sono giusti; "ah, ti manca il Tick". Nell'Event Graph della torretta non c'è l'**Event Tick** che, a ogni frame, controlla il bersaglio e spara. Si ricrea: Event Tick, **Is Valid** ("punto interrogativo, quello sotto") con il PlayerTarget come oggetto, e sull'uscita *Is Valid* la funzione **Shoot**. La torretta spara.

Fire Rate e velocità. "È un po' lento a sparare": il **Fire Rate**, fra le variabili della torretta, è a 0,2 (la trascrizione dice "02"); il docente suggerisce un altro valore ("facciamolo 0,7"; il suo è 0,5): dalla trascrizione non si capisce se il valore sia un intervallo fra i colpi o un'altra cosa nel grafo di Tommaso, e il capitolo non lo ricostruisce. "Il proiettile è molto lento": nel proiettile, il componente **Projectile Movement**, **Initial Speed** e **Max Speed** a **1000**.

Il danno che non arriva alla barra. "Però non prendo danno." Il proiettile ha l'Apply Damage, "va bene così"; nel BP_Bot, però, l'**Add to Viewport** non era collegato con il pin di esecuzione all'InitializeHealthBar ("vedi che non è stato attaccato"). E non basta: nell'Event AnyDamage il Set era su **MaxHealth** invece che su CurrentHealth ("ha impostato MaxHealth, non CurrentHealth"). Corretti entrambi: "vediamo se muoio". "Ottimo, perfetto: di solito tutto il casino."

Il calendario. Prima di riprendere, gli annunci.

- Il **menu**: "se non ce la facciamo a fare il menu oggi, lo facciamo la prossima volta, e tanto la prossima volta sarà l'ultima lezione che avremo".
- Dopo: "poi avrete Davide, quindi avrete martedì con me e Davide"; "poi inizierete a fare roba di **level design**".
- Manca "un piccolo asset da fare", le **porte**: "possiamo farlo tranquillamente la prossima volta, o ve lo lascio come compito, perché sono sicuro che lo sapete fare". "La timeline l'abbiamo già affrontata" (il compito è descritto nella sezione 1.11.15).

("Non stai condividendo?" "Ah, ok. Grazie mille, Lucrezia.")

1.11.13 I due timer della stamina

"E adesso incominciamo a fare questo bellissimo scattino." Si torna al bot del docente ("il proiettile non ci serve più; ci serve botolino"). Il movimento c'è già (500 e 300 sull'IA_Run); "abbiamo detto che dobbiamo fare due timer ben distinti: uno che la consuma e uno che la riporta al suo massimo, piano piano nel tempo".

1.11.13.1 Il timer che consuma

Set Timer by Event. Dopo il Set Max Walk Speed a 500 (lo Started), un **Set Timer by Event**.

- il pin **Time** si promuove a variabile, **TimeStamina**, "in modo tale da avere questa variabile molto più controllabile"; default **0,05** (impostato poco dopo: "piccola cosa, semplicemente non l'abbiamo impostato"): "questo è il tempo in cui verrà eseguito il timer, ogni 0,05 secondi";
- **Looping** attivo; **Max Once Per Frame** attivo;
- "il timer, come abbiamo visto le scorse volte, è un evento collegato": dal pin rosso **Event** si crea un Custom Event, **StaminaDeduction**.

Le variabili. "Lo stesso discorso vale come per la Health": due Float, **CurrentStamina** e **MaxStamina**, entrambe a **100**.

StaminaDeduction. Il corpo è quasi identico a quello della vita:

1. CurrentStamina **Subtract 1** ("verrà sottratto di 1 ogni 0,05 secondi");
2. **Clamp (float)**, fra 0 e MaxStamina ("il nostro valore oscillerà fra un minimo di 0 e un massimo di MaxStamina, che è 100");
3. **Set CurrentStamina**: "farà 99, 98, 97...".

"Questo è lo stesso discorso che vale per la Health, è quasi praticamente medesimo."

Quando fermarsi: OR con Is Falling. Poi un **Branch** ("tenete proprio B" e cliccate: la scorciatoia del Branch): se la CurrentStamina è **minore o uguale a zero**, si fermerà la corsa ("ancora non abbiamo la funzione, ma la lasciamo così"). E una piccola accortezza nata "durante il testing": "ci sono dei problemi anche nel caso noi stiamo cadendo". "Io tengo premuto il tasto mentre sto cadendo: non voglio che la stamina venga consumata, perché non stiamo correndo effettivamente." Dal Character Movement, **Is Falling** ("ce ne sono due: prendete quello *Nav Movement*", quello con l'ingresso blu e l'uscita rossa), e i due booleani in un **OR Boolean**: "OR è l'operatore che ci permette di dire *o*: ti fermi se sei minore di zero *o* se stai cadendo" (figura 1.11.6).

Il timer che non si ferma. Con un Print String della CurrentStamina: "io cammino, cammino, comincio a sprintare: i valori scendono. Ma se anche smetto di correre, i valori continuano a scendere, perché non stoppiamo mai il nostro timer." La soluzione è tenere un *riferimento* al timer: il **Return Value** di Set Timer by Event (un *Timer Handle*) si promuove a variabile, **TimerDeductionStamina**, "in modo tale che sia più comodo controllarlo da varie parti". Al **Completed** dell'IA_Run, dopo il Set Max Walk Speed a 300: un Get di TimerDeductionStamina e **Clear and Invalidate Timer by Handle**, "andiamo a invalidare e fermare il nostro timer". In Play: "corro, smetto di correre: si è fermato a 75".

1.11.13.2 Il timer che ricarica

Il secondo timer. "Altro problema: se io ricomincio a correre, ripartirà da 75, non torna piano piano verso 100." Quando si smette di correre bisogna "fare l'opposto": subito dopo il Clear, un altro **Set Timer by Event**, "sempre in Looping, sempre in Max Once Per Frame". Per il tempo il docente usa la stessa TimeStamina, "ma volendo si può fare a un valore che volete voi: volete che uno si recuperi più veloce oppure più lento, sta a voi". Il suo evento è un Custom Event, **RegainStamina**: "non c'è neanche tanto da sbizzarrirci: è letteralmente la copia di questo". Si copiano i nodi della deduzione fino al Set e "l'unica cosa che andiamo a cambiare è che invece di sottrarre uno, andiamo ad aggiungere uno": **Add** 1, Clamp, Set CurrentStamina.

Fermare anche lui. Stesso schema al contrario: il Return Value si promuove a **TimerRegainStamina**, e allo **Started** ("quando noi ricominciamo a correre") un **Clear and Invalidate Timer by Handle** su di lui. In Play: "comincio a correre, i valori cominciano a scendere; lascio, i valori piano piano risalgono fino a 100".

Fermarsi a 100. Ma arrivato a 100, il timer della ricarica continua a girare, inutilmente. In RegainStamina, dopo il Set, un **Branch**: se la CurrentStamina è **maggiore o uguale** alla MaxStamina, un Clear and Invalidate Timer by Handle su TimerRegainStamina, "cioè lo stesso timer che stiamo avendo": "se arrivi a 100, smettila". "Non stampo più, si è fermato."

1.11.13.3 L'evento StopRunning

La corsa che non finisce. "C'è una particolarità": correndo fino a zero, "il nostro character non si ferma, e neanche il timer si ferma", e il personaggio continua a correre con la stamina vuota. Serve la funzione di *smettere di correre* anche *dentro* StaminaDeduction, non solo al rilascio del tasto. "Per comodità, invece di fare una funzione a parte": si stacca il **Completed** dalla sua catena (Set Max Walk Speed 300, Clear del timer di deduzione, Set Timer della ricarica), e all'inizio della catena si mette un nuovo Custom Event, **StopRunning**. Poi:

- il **Completed** chiama StopRunning;
- il ramo **True** del Branch di StaminaDeduction (stamina ≤ 0 OR Is Falling) chiama StopRunning.

"Questa cosa ci serve semplicemente per trasportarci il fatto di *fermati*: vai a 300 di velocità e smettila di sottrarmi la stamina, da questo evento che possiamo trasportare un po' dappertutto." In Play: "corro, corro, corro, arrivo fino a zero, mi fermo in automatico", la stamina risale; "e poi devo semplicemente ripremere per poter scattare" (lo *Started* scatta solo alla pressione). "Abbiamo fatto il nostro sistema di stamina" (figura 1.11.7).

Ricapitolare per chi si è perso. Una studentessa ha perso il pezzo di RegainStamina; il docente lo ripete: copiare la catena della deduzione, sostituire il meno con un più, il Branch con il *maggiore o uguale* ("non mi trova il maggiore": "io ho la tastiera più piccola, non ci ho alcuni segni"; "allora scrivi *greater equal*"), MaxStamina e CurrentStamina nei due ingressi, sul True il Clear and Invalidate Timer by Handle con TimerRegainStamina. Poi StopRunning: "stacchi da qui il Completed, ci metti un Custom Event al suo posto, e StopRunning ce lo metti anche al Completed"; la chiamata, "di colore azzurro", "tiri fuori e fai StopRunning", anche dopo il Branch. "Ho recuperato tutto."

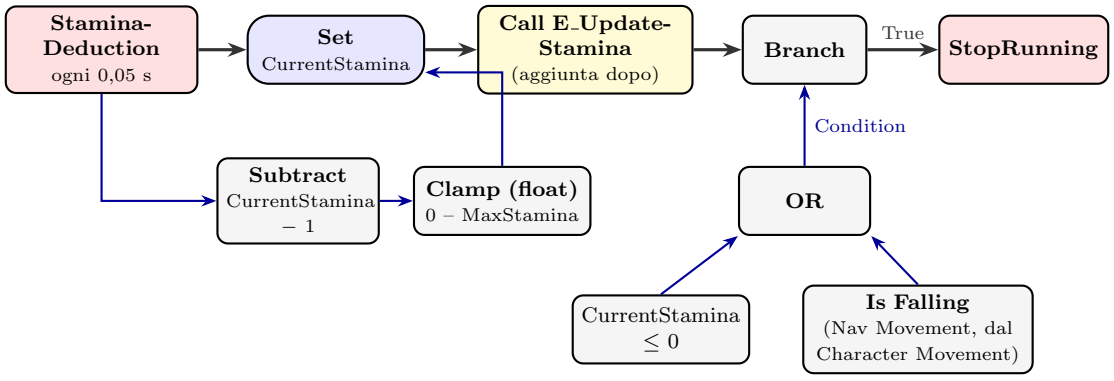


Figure 1.11.6: L'evento del timer che consuma la stamina. Il calcolo è lo stesso della vita, con 1 al posto del danno; la corsa si interrompe se la stamina è finita oppure se il personaggio sta cadendo. La chiamata dell'Event Dispatcher si inserisce dopo, per la barra. Ricostruzione dalla lezione.

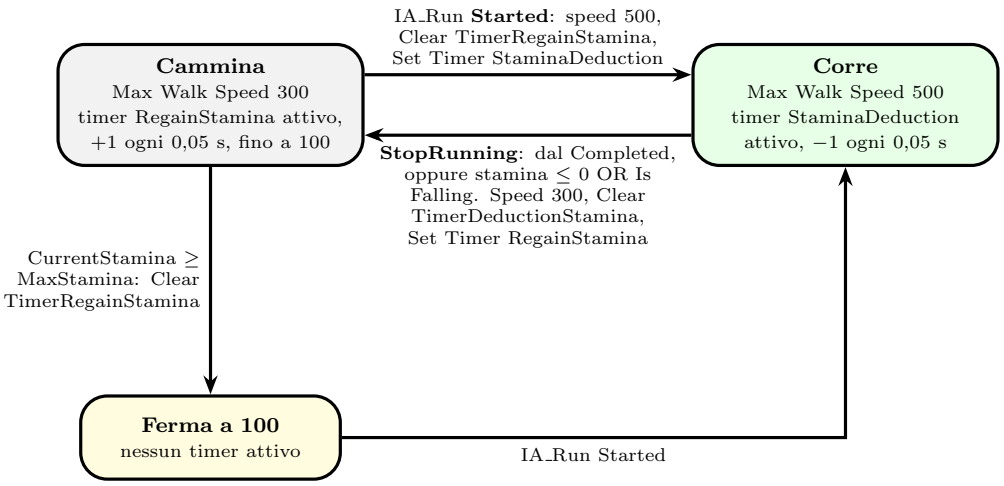


Figure 1.11.7: I due timer della stamina come due stati. Ogni passaggio spegne il timer dello stato che si lascia (con il suo Timer Handle) e accende quello dello stato in cui si entra. A stamina finita la corsa si interrompe da sola; per ricominciare bisogna premere di nuovo Shift. Schema ricostruito.

Approfondimento: la stamina nel libro. Romero e Sewell costruiscono la stessa meccanica nel capitolo 8, *Draining and regenerating stamina*, con scelte diverse che è utile confrontare.

- **Un timer solo.** Invece di due timer, il libro usa un unico evento, *ManageStamina*, chiamato da un timer in loop ogni *Stamina Drain and Recharge Time* = 0,2 secondi ("questo timer chiamerà la funzione cinque volte al secondo"); dentro, a seconda di un booleano *IsSprinting*, l'evento consuma o ricarica. È lo schema dei due stati della figura 1.11.7 con un solo orologio.

- **Per nome, non per handle.** Il timer è creato con *Set Timer by Function Name* e fermato con *Clear Timer by Function Name*; per non sbagliare a scrivere il nome, il libro lo salva in una variabile stringa (*StaminaManagerName*), "per evitare bug dovuti a errori di battitura". Prima di crearlo controlla *Does Timer Exist by Function Name*, per non avviarne due. La lezione usa invece il *Timer Handle* restituito da Set Timer by Event, che identifica il timer senza nomi.

- **Le macro.** La logica è divisa in quattro macro (*StartSprinting*, *StopSprinting*, *ManageStaminaDrain*, *ManageStaminaRecharge*): è la stessa idea dell'evento StopRunning della lezione, un nome per un pezzo di logica da richiamare da più punti.

- **Una condizione in più.** In *ManageStaminaDrain* "ci sono due condizioni perché il giocatore continui a correre e a consumare stamina: deve muoversi, e Player Stamina deve essere maggiore di zero"; il movimento si verifica con *Get Velocity* e *VectorLengthSquared* maggiore di zero ("usiamo VectorLengthSquared invece di VectorLength per evitare una radice quadrata"). Nella lezione, invece, tenere premuto Shift da fermi consuma stamina; la condizione *Is Falling* della lezione copre un altro caso, il salto. Inoltre, alla pressione di Shift il libro controlla con un Branch che la stamina sia almeno pari al costo di un passo di corsa (*Greater* OR *Nearly Equal* rispetto a *Sprint Cost*) prima di avviare lo sprint: nella lezione, a stamina vuota la corsa parte e viene interrotta al primo scatto del timer.

- **I limiti.** Il libro usa *Max (float)* per non scendere sotto 0 e *Min (float)* per non superare 1; la ricarica si ferma quando la stamina è *Nearly Equal* a 1,0. La lezione fa entrambe le cose con il Clamp, e si ferma con il maggiore o uguale.

Con i numeri della lezione: un punto ogni 0,05 secondi sono 20 punti al secondo, quindi la stamina piena (100) dura **5 secondi** di corsa e si ricarica in altri 5 secondi. Nel libro, 0,05 di stamina (su 1,0) ogni 0,2 secondi è 0,25 al secondo, 4 secondi di corsa; la ricarica, 0,01 ogni 0,2 secondi, impiega 20 secondi.

Approfondimento: Timer Handle, Clear and Invalidate, Max Once Per Frame. Alcune nozioni generali sui timer di Unreal, non trattate nei due libri. Il **Timer Handle** è una piccola struttura che identifica un timer attivo nel *Timer Manager* del mondo; **Clear and Invalidate Timer by Handle** ferma quel timer e svuota l'handle, così che un controllo successivo (*Is Timer Active by Handle*, *Is Valid*) dica che non esiste più. Chiamarlo su un handle già vuoto non fa danni: per questo la lezione può "spegnere" il timer della ricarica allo Started anche quando non è mai partito. Se si chiama di nuovo Set Timer by Event sullo stesso evento mentre il timer è attivo, il timer viene reimpostato, non duplicato. Un **Time** minore o uguale a zero non avvia il timer (lo cancella): è il motivo per cui, nella sezione seguente, il timer di una studentessa con il Time a zero "non partirà mai". Infine **Max Once Per Frame**: se l'intervallo del timer è più corto di un frame (0,05 secondi sono 20 volte al secondo; a 15 fotogrammi al secondo un frame dura circa 0,067 secondi), senza questa opzione il timer "recupererebbe" le esecuzioni perse chiamando l'evento più volte nello stesso frame; con l'opzione lo chiama al massimo una volta. La conseguenza pratica è che, a frame rate molto bassi, la stamina della lezione scende un po' più lentamente del previsto, invece di scendere a scatti.

1.11.14 La stamina sulla barra

E_UpdateStamina. "Adesso dobbiamo collegarlo alla nostra UI." Un secondo Event Dispatcher, **E_UpdateStamina**, con due input Float, **CurrentStamina** e **MaxStamina** ("giustamente, ragazzi, non ho dimenticato: bisogna mettere i due input"). Va chiamato "in tutti i momenti in cui noi aggiorniamo CurrentStamina, quindi sia a questo timer sia all'altro": un **Call E_UpdateStamina** dopo il Set di StaminaDeduction e uno dopo il Set di RegainStamina, con i valori collegati.

Il Begin Play. Come per la vita: da AsHUD, dopo InitializeHealthBar, la chiamata a **InitializeStaminaBar** con Self nel PlayerRef (figura 1.11.4). Un problema di passaggio: uno studente aveva chiamato gli input del dispatcher in un altro modo e poi rinominati; il nodo dava errore. "Elimina il blocco e rimettilo": il nodo di chiamata si rigenera con la firma aggiornata.

Nel widget. In WB_HUD, dopo InitializeStaminaBar: dal PlayerRef, **Bind Event to E_UpdateStamina**; dal pin Event, Add Custom Event, **UpdateStaminaBar** (la trascrizione, divertita, sente "update stealth"); "la stessa identica cosa di dividere e fare la percentuale". "Ma attenzione: quale target deve essere?" La seconda barra, che si chiama ancora *ProgressBar_Health1* perché è una copia: la si rinomina **ProgressBar_Stamina**, "per far sì che sia più esplicativa", la si trascina nel Graph e diventa il target del Set Percent. In Play, "corriamo: la barra della stamina andrà giù e ritornerà su".

I problemi dei presenti. Molti, e diversi fra loro.

- **La barra subito grigia.** "Va subito a grigio, però i valori sono quelli." Nelle due Call di E_UpdateStamina i pin non erano collegati: "non gli passi i valori". Si collegano CurrentStamina e MaxStamina, in entrambe.
- **Il Bind che non compare.** "A me non dà il Bind Event per la stamina." InitializeStaminaBar non aveva l'input PlayerRef: senza la referenza al personaggio non c'è un oggetto da cui tirare fuori il Bind. I Details mancanti si aprono da **Finestra**, **Dettagli** (*Window*, *Details*: l'editor della studentessa è in italiano); si aggiunge l'input, e il Bind si trova cercando "stamina".
- **Il timer a zero.** Nel grafo della stessa studentessa, il Set Timer by Event aveva il Time vuoto: "non ti partirà mai, perché il timer è zero". Va collegata la variabile TimeStamina.
- **L'handle mai salvato.** Il Return Value del timer della ricarica non era mai stato salvato: "torna al timer", "prendi Set TimerRegainStamina, imposta, e ce l'attacchi", con anche il pin di esecuzione, "perché sennò quello è il valore, quello là, ha dietro".
- **L'Initialize scollegato e la barra sbagliata.** Nel Begin Play mancava il collegamento dell'InitializeStaminaBar ("la Self a PlayerRef"); nel widget, UpdateStaminaBar va ricreato dal pin, e il Set Percent va fatto sulla barra giusta (che la studentessa ha chiamato *ProgressBar_Endurance*).

- **Il personaggio che non corre più.** Tutto collegato, “mi toglie la barra, ma non mi corre il personaggio”. Il docente ripercorre la catena (Max Walk Speed a 300 nel componente, i due Set a 500 e 300, StaminaDeduction, il minore o uguale, Is Falling, StopRunning) e l’IMC: lo Shift sinistro c’è. Nell’IA_Run, nella sezione *Triggers*, c’erano dei trigger aggiunti (un *Pressed* e un *Released*, a quanto si capisce); la trascrizione è confusa su come vengano sistemati, ma la corsa non riparte. Allora una prova diretta: un **Event Tick** con un Print String di **Get Max Walk Speed** dal Character Movement. “Il problema è che ce n’hai troppi di Print String”: con **Ctrl+F** (*Find Results*) si cercano tutti i Print String del Blueprint, con un doppio clic si va a ciascuno e si tolgono quelli vecchi. Il valore stampato è “300, 300, 300, sempre 300, anche se tengo premuto”. Un’altra ricerca, “Max Walk Speed”, scopre il colpevole: un *secondo* evento IA_Run, rimasto dalla lezione precedente, con i suoi Set. “Questo lo devi levare”, “leva tutto questo ambaradam”. Ora corre.

- **La barra che scende solo al rilascio.** Ultimo sintomo: “a me scende quando mollo il tasto”. Il timer (0,05, Looping, Max Once Per Frame), StaminaDeduction e i suoi nodi sembrano giusti; un F9 sul primo timer non chiarisce. “Guarda, facciamo una cosa: lo facciamo la prossima volta. Ricordamelo tu.”

Approfondimento: Find Results. La ricerca con **Ctrl+F** usata a fine lezione è lo strumento più rapido per un Blueprint grande: cerca il testo in tutti i grafi del Blueprint (Event Graph, funzioni, macro) e mostra i risultati in un pannello, con un doppio clic che porta al nodo; con il binocolo del pannello la ricerca si estende a *tutti* i Blueprint del progetto. È il modo giusto per trovare un evento duplicato, come il secondo IA_Run, o i Print String dimenticati, che altrimenti riempiono lo schermo e nascondono quelli utili (funzionamento generale dell’editor). Il caso della studentessa mostra perché un evento d’input duplicato è insidioso: a lezione il secondo IA_Run, rimasto con i suoi vecchi Set, continuava a riportare il Max Walk Speed a 300, e il Print String nel Tick lo ha reso visibile. Dalla trascrizione non si capisce se l’editor segnalasse il duplicato con un avviso di compilazione; in ogni caso, una Input Action dovrebbe avere un solo evento per Blueprint.

1.11.15 Il compito e la chiusura

Il compito: le porte. “Vabbè, dai, io vi faccio solo vedere un po’ il compito che vi do per la prossima volta, che è sempre molto facoltativo: non c’è voto, non c’è niente.” È l’ultimo asset del pacchetto modulare usato nel corso: “vorrei che faceste questa **porta** qui”, un asset con “un volume” in cui il giocatore entra. Gli elementi sono tre: la cornice, la mesh **SM_Modular_Wall_Door** (“si chiama wall door: se io vado nel Content e scrivo wall, eccolo qua”; all’inizio il docente la chiama “frame”), e due ante **SM_Door**, “la stessa cosa, semplicemente ruotata di 180 gradi”. Il comportamento: “all’avvicinamento e all’entrata in questo volume del character, la porta si apre; viceversa, se il character esce, la porta si deve chiudere”. “In modo tale che abbiamo finito tutti gli asset pack”; “lo fate voi, perché tanto le timeline le abbiamo già affrontate”. Gli strumenti sono quelli della piattaforma che sale: un *Box Collision* con *On Component Begin Overlap* ed *End Overlap*, una *Timeline* con *Play* e *Reverse*, e un *Set Relative Rotation* (o Location) delle due ante. Se ci saranno difficoltà, “la vediamo un attimo” la prossima volta, “e possiamo farla molto velocemente”.

La stamina in caduta. Un’ultima domanda: “anche quando saltiamo o cadiamo, se tengo premuto Shift, continua a consumare la barra”. “Aspetta, questo l’abbiamo visto, c’era il fix.” Nel grafo dello studente, l’Is Falling era il nodo sbagliato (“sei sicuro che questo non è verde?”): va preso quello *Nav Movement*, “che ti dà questo blu e rosso”; e l’OR deve avere un collegamento unico verso il Branch. “Ah, ho trovato l’errore.” Dove si trova l’asset della porta? “Dovrebbe essere già nel vostro progetto”: *wall door* e *SM_Door*.

Il consiglio. Edoardo annuncia che recupererà la lezione prima di martedì. “Vai tranquillo: le lezioni sono registrate fino a settembre, fino al ritorno a scuola.” E un consiglio per tutti: “queste nozioni vi serviranno in generale”: chi farà programmazione “masticherà quasi sempre queste cose”, ma anche i designer, “io ad esempio consiglio di allenarvi un pochino, perché il lavoro del designer, che è quello che poi alla fine faccio io, è prototipare le meccaniche, vedere se funzionano”. “Le idee sono tanto belle e carine nella nostra testa, ma poi bisogna vedere se effettivamente sono concretamente funzionanti all’interno di un ambiente di gioco.”

La prossima volta. "La prossima volta facciamo le **interazioni**", del tipo "premi per interagire" (la trascrizione dice "to interact, roba varia"), oltre al menu rimandato e alle correzioni in sospeso. "Allora, se non ci sono altre domande, io vi lascio. E ci vediamo martedì. Buon weekend, buon tutto e buona cena."

Variabile	Dove	Tipo, valore	A che serve
CurrentHealth	BP_Bot2	Float, 10	la vita attuale
MaxHealth	BP_Bot2	Float, 10	il massimo del Clamp e il divisore della barra
CurrentStamina	BP_Bot2	Float, 100	la stamina attuale
MaxStamina	BP_Bot2	Float, 100	il massimo del Clamp e il divisore della barra
TimeStamina	BP_Bot2	Float, 0,05	l'intervallo dei due timer, in secondi
TimerDeductionStamina	BP_Bot2	Timer Handle	il timer che consuma
TimerRegainStamina	BP_Bot2	Timer Handle	il timer che ricarica
AsHUD	BP_Bot2	WB_HUD (ref.)	la referenza all'HUD creato
E_UpdateHealth	BP_Bot2	Dispatcher	porta CurrentHealth e MaxHealth
E_UpdateStamina	BP_Bot2	Dispatcher	porta CurrentStamina e MaxStamina
PlayerRef	WB_HUD (input)	BP_Bot2 (ref.)	il personaggio a cui legarsi
ProgressBar_Health	WB_HUD	Progress Bar	la barra rossa (Is Variable)
ProgressBar_Stamina	WB_HUD	Progress Bar	la barra verde (Is Variable)
Max Walk Speed	Character Movement	300 / 500	camminata e corsa
Base Damage	BP_Projectile	1	il danno di un proiettile

Table 1.11.2: Le variabili e i valori della giornata, con i nomi usati dal docente.

Punti chiave del capitolo

- Il debug si fa **escludendo le cause**: un Print String dice quanto vale un dato e se un ramo è eseguito; F9 mette un breakpoint; F8 permette di ispezionare l'istanza in gioco; un Default Value forzato separa i dati dalle animazioni.
- Un Blend Space si adatta ai valori reali: se il Lean va da -1 a 1, l'asse va da -1 a 1. **Prima i campioni, poi l'asse.**
- Senza il nodo che aggiunge l'**Input Mapping Context**, gli input non arrivano al personaggio: "non hai la mappa".
- La **UI** è tutto ciò che dà informazioni al giocatore sullo schermo; l'HUD ne è la parte sempre visibile (vita, stamina, monete). Anche lo schermo rosso di Call of Duty è UI.
- Un personaggio nuovo va fatto riconoscere: **tag** Player sull'actor, Get Player Pawn, collisioni **Block** su Detection Turret e Projectile Trace.
- Il **danno** di Unreal ha due capi: **Apply Damage** (nel proiettile, sull'Hit Actor del Break Hit Result, Base Damage 1) ed **Event AnyDamage** (nel personaggio). Il proiettile poi distrugge *se stesso*, non il bersaglio.
- La vita: CurrentHealth − Damage, **Clamp** fra 0 e MaxHealth, Set; morte con il **minore o uguale** a zero (i float non sono esatti); **Get Current Level Name** e **Open Level** per ricominciare.
- Un **Event Dispatcher** con input è "un camion con i posti per i pacchi": il personaggio lo chiama con i valori, l'HUD vi si lega con **Bind Event** e un Custom Event creato *dal pin*, che ne eredita gli input.

Sintomo	Causa e rimedio
Il bot inclina appena	Lean fra -1 e 1, asse del Blend Space da -10 a 10: spostare prima i campioni, poi l'asse a $-1/1$
Il bot non si muove	manca il nodo che aggiunge l'Input Mapping Context nel Begin Play; cliccare sulla viewport in Play
L'animazione non parte, Print String muto	il Validated Get controlla una variabile mai valorizzata (due variabili quasi omonime): usare quella dell'Initialize
Il bot cammina sul posto da fermo	regola di ritorno con GroundSpeed < 0: deve essere ≤ 0
La torretta ignora il bot	tag Player mancante, o messo nei Component Tags invece che nei Tags dell'actor
La torretta prende altri actor	usare Get Player Pawn invece dell'Other Actor
La torretta segue ma non spara	Mesh senza Block su Detection Turret e Projectile Trace; Trace Channel sbagliato; Event Tick con Is Valid e Shoot mancante
Manca la funzione dello spawn	copiarla come testo; ricreare gli Scene Component e lo Spawn Actor from Class
Break Hit Result non disponibile	Multi Sphere Trace (array): usare Sphere Trace By Channel
I proiettili urtano la camera	Camera in Ignore sulla collisione del proiettile
I proiettili attraversano il bot	Radius della Sphere Trace a 0: metterlo a 20
Un colpo e il bot sparisce	Destroy Actor sull'Hit Actor: Apply Damage sull'Hit Actor e Destroy Actor su self
Il bot muore subito	MaxHealth a 0: il Clamp azzera la vita
Il bot diventa immortale	Set su una NewVar (o su MaxHealth) invece che su CurrentHealth
Il Canvas Panel non si aggiunge	la Progress Bar è la radice del widget: cancellarla, mettere prima il Canvas
Set Percent introvabile	va tirato fuori dalla variabile della barra (Is Variable attivo)
Il Bind non accetta l'evento	Custom Event creato a mano, senza input: crearlo dal pin Event del Bind
La barra della vita non scende	Initialize non chiamato, o Add to Viewport non collegato all'Initialize
La barra della stamina è grigia	Percent a 0, o Call di E_UpdateStamina senza valori
Manca il Bind per la stamina	InitializeStaminaBar senza l'input PlayerRef
Il timer non parte	Time a zero: collegare TimeStamina
La ricarica non si ferma	l'handle TimerRegainStamina mai salvato
Errore sulla Call dopo aver rinominato gli input	cancellare e ricreare il nodo
Shift non fa correre (Max Walk Speed fisso)	un secondo evento IA_Run duplicato: trovarlo con Ctrl+F e cancellarlo
La stamina scende solo al rilascio	non risolto a lezione, rimandato
La stamina scende in caduta	Is Falling sbagliato (quello Nav Movement) o OR collegato male
Il portatile rallenta	surriscaldamento: base con ventole

Table 1.11.3: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, asset o impostazione	Che cosa fa
Print String / breakpoint (F9) / F8	scrive un valore; ferma l'esecuzione su un nodo; stacca la camera in Play
Browse to Asset (la lente)	mostra l'asset nel Content Browser
Default Value (variabile)	valore di partenza, utile per provare l'anteprima
Tags (Actor)	etichette con cui la torretta riconosce il player
Get Player Pawn	il Pawn del giocatore
Collision: Custom, Block / Ignore	risposta di un componente a un canale
Apply Damage	infligge un danno a un actor
Event AnyDamage	riceve il danno: Damage, Damage Type, Instigated By, Damage Causer
Sphere / Multi Sphere Trace By Channel	una sfera lungo un segmento; un colpo o tutti i colpi
Break Hit Result: Hit Actor	l'actor colpito
Subtract / Add / Divide	sottrazione; somma; divisione
Clamp (float)	limita un valore fra un minimo e un massimo
Less Equal / Greater Equal ($\leq$, $\geq$)	confronti fra float
OR Boolean	vero se almeno un ingresso è vero
Event Dispatcher (con Inputs)	evento che porta dei valori a chi si è legato
Call / Bind Event to	chiama il dispatcher; lega un evento al dispatcher
Add Custom Event (dal pin Event)	crea un evento con la firma del dispatcher
Get Current Level Name / Open Level (by Name)	il nome del livello; lo apre (o lo ricarica)
Widget Blueprint (User Widget)	l'asset dell'interfaccia: Designer e Graph
Canvas Panel / Palette / Hierarchy	la tela; gli elementi disponibili; l'elenco degli elementi
Button: On Clicked, Pressed, Released, Hovered, Unhovered	gli eventi di un bottone
Progress Bar: Percent, Fill Color and Opacity	riempimento da 0 a 1; colore
Anchors / Position / Alignment	punto di riferimento; scostamento; pivot dell'elemento
Is Variable	rende l'elemento usabile nel Graph
Create Widget / Add to Viewport	crea l'istanza del widget; la mostra
Get a Reference to Self	referenza al Blueprint stesso
Set Percent	imposta il riempimento della barra
Input Action (Digital) / IMC	azione booleana; mappatura dei tasti
IA_Run: Started / Completed	inizio e fine della pressione
Set Max Walk Speed	la velocità massima del Character Movement
Set Timer by Event (Time, Looping, Max Once Per Frame)	chiama un evento a intervalli
Timer Handle / Clear and Invalidate Timer by Handle	identifica il timer; lo ferma
Is Falling (Nav Movement)	vero se il personaggio è in aria
Find Relative Look at Rotation	la rotazione per guardare un punto
Get World Delta Seconds	il tempo dell'ultimo frame
Spawn Actor from Class, Split Struct Pin	fa nascere un actor; separa i campi di un pin
Scene Component	punto vuoto nella gerarchia (bocca del cannone)
Projectile Movement: Initial / Max Speed	la velocità del proiettile
Find Results (Ctrl+F)	cerca un testo nei grafi del Blueprint

Table 1.11.4: I nodi, gli asset e le impostazioni dell'undicesima lezione.

- Il **Widget Blueprint** ha un Designer (Canvas Panel, Palette, Hierarchy) e un Graph. La **Progress Bar** si riempie con **Percent** da 0 a 1: *Current diviso Max*. Va resa **Is Variable** per usarla nel Graph.
- Le **ancore** legano un elemento a un punto dello schermo indipendentemente dalla risoluzione; **Position** è relativa all'ancora, **Alignment** è il pivot dell'elemento (0,5 lo centra).
- L'HUD nasce nel Begin Play: **Create Widget**, referenza promossa a variabile, **Add to Viewport**, poi l'Initialize con **Self** come PlayerRef.
- La corsa: **IA_Run** (copia di IA_Jump, Shift nell'IMC), Started a 500, Completed a 300, Max Walk Speed di default 300.
- La stamina usa due **timer** (Set Timer by Event, 0,05 s, Looping): uno sottrae, uno aggiunge. Ognuno si ferma con il suo **Timer Handle** e **Clear and Invalidate Timer by Handle**; la ricarica si ferma a 100; l'evento **StopRunning** interrompe la corsa al rilascio, a stamina finita o in caduta (**OR** con Is Falling).
- La seconda barra è la stessa cosa: **E_UpdateStamina**, chiamato dopo ogni Set di CurrentStamina, InitializeStaminaBar, UpdateStaminaBar, Set Percent su ProgressBar_Stamina.
- Gli errori tipici sono di *collegamento*: Initialize non chiamato, valori non passati alla Call, Time a zero, handle non salvati, eventi duplicati (da trovare con **Ctrl+F**).
- Il compito facoltativo: una **porta** (SM_Modular_Wall_Door e due SM_Door) che si apre entrando in un volume e si chiude uscendo, con una Timeline. La prossima volta: interazioni e menu.

Chapter 1.12

Il menu principale e il sistema di interazione

Questa è la **dodicesima lezione** del corso breve di **introduzione a Unreal Engine**, ed è l'**ultima** con questo docente: "oggi è l'ultima lezione, almeno mia". Nella lezione precedente il personaggio, il **bot**, ha ricevuto la vita, il danno della torretta, la stamina e le due barre dell'HUD; restavano in sospeso due argomenti annunciati, il **menu** e le **interazioni**. Oggi si fanno entrambi. "Sono argomenti che abbiamo già fatto in precedenza e adesso li uniamo": Widget Blueprint, Level Blueprint, trace, collisioni personalizzate, Montage, timer e attivabili tornano tutti, messi insieme.

Il primo argomento, il più corto, è un **menu principale**: una mappa a parte, con un piccolo *diorama* 3D sullo sfondo, una camera fissa, due bottoni (*gioca* ed *esci*) e il passaggio alla mappa di gioco. Il secondo, "piuttosto lungo", è un **sistema di interazione** del tipo "premi E per interagire", costruito con due strumenti nuovi: un **Blueprint Component**, che si aggiunge a qualunque actor e "traccia" dalla camera gli oggetti interagibili, e una **Blueprint Interface**, il "porto" attraverso cui il personaggio dice a un oggetto *sto iniziando a guardarti*, *non ti guardo più*, *interagisco con te*. L'oggetto di prova mostra un piccolo widget con la lettera E quando è inquadrato; premendo E il personaggio si gira verso di lui e suona un'animazione; alla fine, lo stesso oggetto accende e spegne una luce.

La lezione è **online**, come le precedenti, e si apre con una conversazione più generale: come recuperare le lezioni, che cosa aspetta gli studenti negli anni successivi, e un consiglio di design, "*less is more*". Durante la pausa il docente risolve il problema della barra della stamina rimasto aperto la volta precedente; si chiude con i ringraziamenti e con la raccomandazione che riassume il corso: "ricordatevi di validare sempre molte variabili".

Tre avvertenze di lettura, le stesse delle lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Real" e "Arrial"; il **diorama** è "diuramino"; il Widget Blueprint **WB_Menu** è "VB menu" e "VBIN menu", e **WB_InteractUI** è "VB Interact UI" e "Interacco uai"; il **Begin Play** è "big in play" e "back in play"; la **ThirdPersonMap** è "Terperson map" e "there person"; il **Scene Component** è "sin component" e "sim component"; il componente **BPC_RayInteract** è "BPC Reinteract", "Ray interact one" e "IPC reinteract"; la Blueprint Interface **BPI_InteractionRay** è "VPI interaction", "VPA Interaction Array" e "bibliotecazione:ray"; l'**Hit Actor** è "it actor" e "I Tactor"; il **Validated Get** è "variegate" e "vari da get"; **Does Object Implement Interface** è "Hacker does object implement interface"; il **Draw Debug Type** è "draw the bug type"; il bot è "Boto" e "Boto2", e il suo Blueprint "BiPi Bot"; **Find Look at Rotation** è "findLocatrotation"; l'**Anim Montage** è "un montaggio un po' brasilero"; i *Big One* del corso di laurea sono "biggest one". "Mandragata", invece, non è un errore: è il romanesco *mandrakata*, "trucchetto", e il docente lo usa apposta. Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: durante la pausa il riconoscimento vocale produce un lungo blocco di testo senza senso e ripetuto decine di volte ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?"), seguito da un frammento che sembra l'audio di un video in sottofondo (una "blue map" che mostra "le secret room" senza sprecare bombe, e "un'altra monetina"): non appartengono alla lezione e sono stati ignorati. La terza: buona parte della lezione è fatta di indicazioni a voce mentre si lavora ("tira fuori", "attacca qua", "vai

giù"), spesso rivolte allo schermo di uno studente; il capitolo ricostruisce i grafi da ciò che il docente dice mentre li costruisce, distingue i tentativi dalle soluzioni, completa i nomi esatti dei nodi e dei pin di Unreal Engine 5, e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono la disposizione del menu, i due grafi che lo fanno funzionare (Level Blueprint e Widget Blueprint), il ruolo di componente e interfaccia, la funzione che cerca gli interagibili, il ciclo di vita della variabile Interactable e il grafo dell'interazione nel personaggio. Le tabelle riassumono gli asset e le variabili della giornata, gli errori incontrati durante la lezione e i nodi. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro di Marcos Romero e Brenden Sewell, *Blueprints Visual Scripting for Unreal Engine 5*, il testo di riferimento per questa parte del libro, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. le **domande d'apertura**: recuperare le lezioni, l'esame e i Big One, il lavoro del designer, *less is more* (sezione 1.12.1);
2. gli **annunci**, la scelta dell'ordine e la **dimostrazione** del sistema di interazione (sezione 1.12.2);
3. il **menu**: la mappa, il widget, i bottoni e i loro colori (sezione 1.12.3);
4. il **Level Blueprint** del menu: il widget sullo schermo, la Game Mode e la camera (sezione 1.12.4);
5. il **Graph del menu**: cursore, input e bottoni; il diorama e i problemi degli studenti (sezioni 1.12.5 e 1.12.6);
6. **interfacce** e **Blueprint Component** (sezione 1.12.7);
7. il componente **BPC_RayInteract** e la sua Sphere Trace (sezione 1.12.8);
8. il **Trace Channel** e la **Blueprint Interface** (sezione 1.12.9);
9. la **pausa**: la stamina di Tommaso (sezione 1.12.10);
10. la logica della variabile **Interactable** (sezione 1.12.11);
11. l'oggetto di prova **BP_InteractableTest** (sezione 1.12.12);
12. l'**input** e l'interazione nel personaggio (sezioni 1.12.13 e 1.12.14);
13. il widget **"premi E"** (sezione 1.12.15);
14. la **luce** interattiva e la **chiusura** del corso (sezioni 1.12.16 e 1.12.17).

1.12.1 Prima della lezione: recuperare, l'esame, il designer

Recuperare o seguire? In apertura, prima che arrivino gli altri, uno studente rimasto indietro ("le sto andando pian piano") chiede se gli convenga recuperare le lezioni vecchie o seguire questa. (Il nome, nella trascrizione, è incerto: il docente lo chiama una volta "Dave", ma poco dopo ricorda questa conversazione come quella fatta "con Lorenzo".) La risposta: seguire la lezione in diretta ha un vantaggio, "se c'è un qualche particolare dell'argomento, puoi farmi direttamente una domanda"; poi, siccome oggi si uniscono argomenti già visti, "magari ti rifai anche quelle precedenti in modo tale da stare un po' più al passo". L'unico prerequisito per l'esercizio di oggi è "avere il personaggio che si muove", che "può guardarsi in giro"; "puoi anche usare il personaggio standard, volendo". Le domande si possono fare anche dopo, sul **Discord**: "mandarmi le domande anche quando non ci vediamo più, perché oggi è l'ultima lezione". Lo studente sceglie di seguire e di riguardare la registrazione.

L'esame e i Big One. Seconda domanda: ci sarà un esame, con un progetto da consegnare? "Nel corso no": questo corso "è strutturato che sia un'introduzione per voi". Il docente chiede che corso farà lo studente (game designer: "tanto farete sia game sia level designer") e descrive il percorso, avvertendo che è quello che ha fatto lui ("almeno io feci così, adesso non so come sono cambiate le cose"):

- al **primo anno** il progetto principale è un **gioco da tavolo**: "farete analog game design";
- al **secondo anno** il **Big One** è un progetto su Unreal, "perché Unreal è molto più improntato per i designer";
- al **terzo anno** il progetto più grande (nella trascrizione "biggest one"), che può anche essere la continuazione del secondo, "se il vostro progetto nel secondo anno è andato bene", da portare fino al rilascio su **Steam**, "con l'Accademia assieme".

In questo senso l'esame c'è: "sarà farvi un **pezzo da portfolio**", "un gioco che potete mettere sul vostro portfolio e promuovervi". "Quindi, più sapete, meglio è."

Che cosa fa il designer con Unreal. "Io sono designer": per un designer Unreal "servirà principalmente per prototipare le meccaniche", cioè "concretizzare l'idea che tu hai studiato, che tu hai tirato fuori, concretizzarla in editor e vedere se effettivamente funziona". Il suo strumento sono i **Blueprint**, "quindi non andare troppo in depth" nella programmazione: in ambito lavorativo il designer ha "un programmatore che lo assiste"; "poi, quando si sta da soli, si fa di tutto".

Less is more. Un consiglio "da esperienza": **less is more**. "Più è leggibile un qualcosa anche nella vostra testa a livello di design, più sarà leggibile per un giocatore." L'esempio è "la regola delle parole a schermo": il testo che scorre all'inizio di *Star Wars*. Chi l'ha scritto lo rilegge in venti secondi, perché lo conosce già; lo spettatore, che deve "cominciare a creare le connessioni mentali", ci mette "non 20 secondi, ma 50". Nel gioco vale lo stesso: chi lo ha progettato e testato pensa "vabbè, tanto questo lo so ormai", ma il giocatore "ci metterà il triplo del tempo". "Più roba fate, più tempo il giocatore dovrà metterci per imparare tutto quanto." "Credo che *less is more* ve lo faranno fare", aggiunge, "ve lo diranno"; e per qualunque domanda di design, "possiamo tranquillamente pure parlare".

Approfondimento: la maledizione della conoscenza. Il fenomeno descritto dal docente ha un nome in psicologia, *curse of knowledge*: chi conosce un'informazione fatica a immaginare come la percepisce chi non la conosce. È il motivo per cui il *playtest* con persone esterne al team è insostituibile, e per cui le regole di un gioco andrebbero introdotte una alla volta, ciascuna quando serve (il cosiddetto *onboarding* progressivo). I numeri del docente (20 secondi contro 50, "il triplo") sono un'immagine, non una misura. Questa è una nozione generale di design, non della lezione.

1.12.2 Gli annunci, l'ordine della giornata e la dimostrazione

Che cosa si fa oggi. Prima che arrivino tutti, il docente chiede agli studenti di scegliere: oggi ci sono "sia il sistema di interazioni, che è piuttosto lungo", sia il **menu**, "che l'altra volta non abbiamo finito, praticamente non abbiamo neanche iniziato". Prima il lungo o prima il corto? Gli studenti scelgono "corto e lungo, così facciamo la pausa prima" ("andiamo a un caffè"). Uno studente ricorda un'altra cosa in sospeso: "durante la pausa sistemare il problema della stamina, che se tengo premuto non si vede, ma solo quando mollo il tasto si vede" (è il problema rimasto aperto alla fine della lezione precedente); "ricordamelo appena andiamo in pausa".

Gli annunci. Arrivati gli altri, il docente ripete: "oggi praticamente è la mia ultima lezione". Seguiranno "circa quattro lezioni" ("adesso non mi ricordo quante lezioni avete") con **Davide**, dedicate alla parte di **level design**: "i tool che vi serviranno per fare level design" e alcuni approfondimenti, "credo di materiali". "Ciò non vuol dire che io vi abbandoni":

- il **canale Discord** resta aperto, anche quando inizierà il corso vero e proprio: "fate domande e dovrebbe arrivare qualcuno in vostro aiuto";
- anche i lavori si possono postare per avere un feedback (il docente cita quello di **Federico La Porta**, "che è bravissimo", a cui ha già risposto);
- i due canali vanno tenuti separati: quello del corso ("Introduzione a Unreal") per i compiti, "così almeno non mandate mischiati, perché sennò perdo il filo", e la chat generale "per qualsiasi comunicazione generica";
- le registrazioni restano disponibili "fino a settembre, se non sbaglio", "quindi avete comunque abbastanza tempo per poterle recuperare".

La dimostrazione. "Adesso vi faccio vedere solamente un'introduzione, come nei classici video tutorial." Il personaggio è quello della volta scorsa, "che si muoveva con stamina e adesso perde vita con la torretta". Dalla sua testa parte un **trace** sottile, "tipo un cilindrone": "questo sarà il nostro sistema che identificherà un possibile *interactable* tramite la telecamera". A sinistra, sullo schermo, i messaggi: se la camera inquadra l'oggetto compare *inizio interazione*, se si distoglie lo sguardo *fine interazione*; premendo il tasto, il personaggio "si gira in automatico" verso l'oggetto e interagisce. "Visto che comunque abbiamo creato questo sistema, è un buon stampo modulare": in futuro, pulsanti e altri oggetti potranno avere "l'interazione un po' più precisa", riusando ciò che si è visto con i pulsanti classici: "sempre premi pulsantino e fai cosetta, e animazione".

1.12.3 Il menu: la mappa, il widget e i bottoni

Il Widget Blueprint. Nella cartella **UI** del progetto, tasto destro, *User Interface*, **Widget Blueprint**, "come abbiamo creato per l'HUD". Alla richiesta della classe base, "non ci serve una particolare, ce ne serve una generica": **User Widget**. Il nome: **WB_Menu**.

Una mappa per il menu. "Una cosa importante che io almeno consiglio": creare una **nuova mappa** dedicata al menu, "quindi differenziare il gioco dal menu". "Si possono fare anche volendo insieme, ma a livello di gestione" è più comodo separarli, "ed è molto più rapido". Da **File**, **New Level**, si sceglie **Empty Level**: "se vi diventa tutto nero, è normale, ragazzi: è un empty level". Un livello *Basic*, invece, contiene già "la directional light, la skybox, tante cose che sono già prefatte"; qui si parte da zero. Poi:

- il menu si può fare "sia in 2D, con immagini 2D", sia con elementi 3D, "una specie di diorama"; per cominciare, un **piano** abbastanza grande (dalle forme base: il docente prende prima un cubo, poi si corregge), con un materiale a griglia, "il material base di Unreal";
- si salva la mappa con **Ctrl+S** nella cartella delle mappe, con il nome **M_Menu**.

(Un dettaglio marginale: il docente nota che qualcosa nel livello risulta *Hidden in Game*, poi si accorge che non lo è; la trascrizione non dice di che cosa si tratti.)

Il Canvas Panel. Aperto WB_Menu, "la prima cosa, che in questo caso servirà per adattarlo allo schermo": il **Canvas Panel**, "la classica canvas". La **Hierarchy** si vede "da qua sotto". A uno studente che non trova la **Palette** il docente mostra il pannello a sinistra dell'editor.

Due bottoni. "Adesso andremo a fare due pulsanti molto semplici: uno di *exit* e uno di *open the game*." Nella Palette, la sezione **Common** (accanto alle altre: Audio, Input, Panel, Primitive e così via, che la trascrizione elenca a fatica) contiene il **Button**; se ne trascinano due al centro dello schermo. La disposizione passa per qualche tentativo ad alta voce (posizioni e allineamenti come $-2{,}7$ e -5, presto abbandonati: "ok, mi sono perso"); la versione pulita, quella che il docente detta alla fine, è questa:

1. **Size**: X **300**, Y **80** ("60? No, troppo piccolo");

2. **Anchors** al **centro**, l'icona centrale del menu delle ancore, "in modo tale che questo bottone sia sempre ancorato al centro"; poi si **azzerano** i valori della **Position**;

3. **Alignment 0,5** su X e **1,5** su Y, "perché così è più corretto": il bottone resta centrato orizzontalmente e si alza di una volta e mezza la sua altezza rispetto al centro dello schermo;

4. per il secondo bottone, "copiate e incollate letteralmente" il primo, azzerate la posizione e cambiate solo l'allineamento verticale: dopo un "−1,5? No, troppo" e un "−0,5", il docente si ferma a −0,3 ("va bene"), cioè il secondo bottone sta un po' sotto il centro.

(I valori del secondo bottone, nella trascrizione, sono incerti: il senso è che i due bottoni stanno uno sopra e uno sotto il centro, con un po' di spazio fra loro.) A uno studente il copia e incolla non funziona: "guarda nella Hierarchy: se c'hai due bottoni, vuol dire che ce ne hai due", magari sovrapposti; altrimenti, "metti semplicemente un altro bottone tramite il Common".

I nomi. "Una cosa che è importante da fare": nella Hierarchy si rinominano i due bottoni con **F2**, **Button_StartGame** e **Button_QuitGame** (la trascrizione dice "Button With Game"). "Così abbiamo semplicemente perfettamente nomenclati."

Il testo dentro il bottone. "Una cosa carina che già esiste": un **Text** (dalla sezione Common) trascinato *sopra* un bottone "si va a incastrare per benino", diventa suo figlio e si centra da solo. Nei **Details** del Text, sezione **Content**, il campo **Text**: "cancellate e scrivete *Play Game*, o *Play*, come volete"; nell'altro bottone, *Quit*. Il testo bianco "non si vede tantissimo", ma i colori si cambiano subito dopo.

Lo Style del bottone. Selezionando il bottone ("mi raccomando"), nei Details la sezione **Style** ha tre stati, ciascuno con il suo colore:

- **Normal**: il bottone a riposo;
- **Hovered**: "vuol dire che ci passi sopra il cursore";
- **Pressed**: "se premuto".

(La trascrizione nomina anche i *foreground* e lo stato *Disabled*: nelle versioni recenti dell'engine lo stile del bottone ha anche i colori del primo piano per ciascuno stato.) Cliccando sulla barra del colore si apre la ruota dei colori; il docente mette il Normal **blu**, "e avremo questo cambiamento in diretta".

Copiare un colore esatto. "Se vogliamo prendere perfettamente il colore che ci interessa per spostarlo all'altro bottone": nella finestra del colore, in basso, ci sono i campi **Hex sRGB** e **Hex Linear**. "Potete prendere ambi i due": si copia il codice con Ctrl+C, si apre lo stesso stato nell'altro bottone e lo si incolla *nello stesso campo* ("se avete preso sRGB, sempre su sRGB"). È lo stesso principio dei siti web: "in HTML, per mettere un colore, i colori sono tutti identificati tramite un codice colore".

Un'immagine dietro i bottoni. Si aggiunge un'**Image**, allargata dietro i due bottoni; ma "come potete vedere, mi copre tutto quanto". "Qui ci viene in aiuto la Hierarchy": come i livelli di Photoshop, gli elementi del Canvas sono disegnati nell'ordine dell'elenco, e "quello che è più sotto" nell'elenco, stranamente, è quello "a livello più alto", disegnato sopra gli altri. L'immagine, aggiunta per ultima, è in fondo all'elenco e quindi davanti ai bottoni. La si trascina nella Hierarchy fino a far comparire una **linea blu** fra il Canvas Panel e il primo bottone, e la si lascia lì: ora è disegnata per prima, cioè dietro. Poi la si rende trasparente: nel suo **Color and Opacity** (che il docente cerca nel Brush), l'**alfa** a 0,5, "o forse un po' più basso", **0,2**. "Questa è la alfa di un colore: l'intensità, l'opacità." Ne viene "una specie di schermatina piccoletta", regolata a mano attorno ai bottoni (figura 1.12.1).

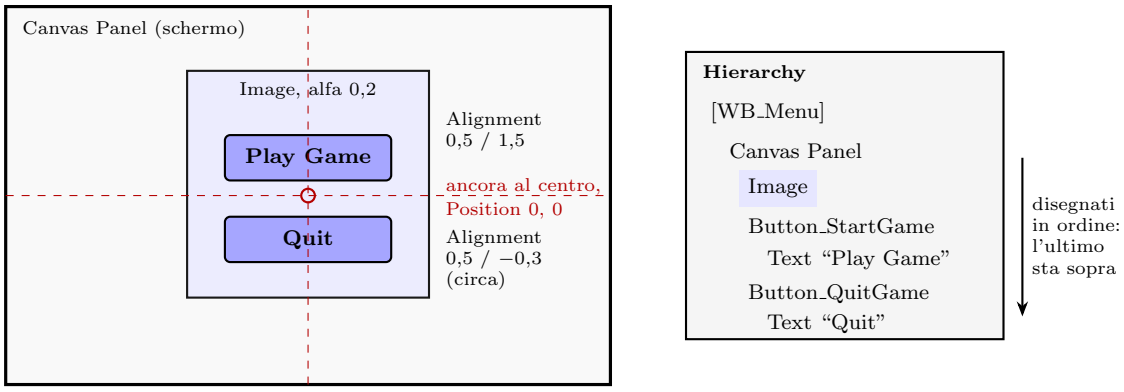


Figure 1.12.1: Il menu nel Designer. I due bottoni sono ancorati al centro dello schermo con la posizione azzerata; è l’Alignment (il pivot del bottone) a spostarli sopra e sotto il centro. L’immagine trasparente è stata trascinata in cima alla Hierarchy, così viene disegnata per prima, dietro ai bottoni. Ricostruzione dalla lezione: i valori del secondo bottone sono approssimativi.

Approfondimento: Alignment, ancore e Size To Content. Con l’ancora al centro e la posizione a zero, l’**Alignment** decide quale punto del bottone coincide con il centro dello schermo: 0 è il bordo sinistro (o superiore), 1 il bordo destro (o inferiore), 0,5 il centro. Valori fuori dall’intervallo da 0 a 1 sono leciti: con Y = 1,5 il punto d’aggancio sta mezzo bottone *sotto* il bordo inferiore, quindi il bottone sale; con Y negativo, scende. È il trucco usato a lezione per impilare due bottoni senza toccare la posizione. La via più ordinata, per menu con molti bottoni, è un **Vertical Box** (sezione Panel della Palette) ancorato al centro, con i bottoni dentro: lo spazio fra gli elementi si regola con il *Padding* di ciascuno e non servono calcoli. Nel libro di Romero e Sewell (capitolo 8, *WinMenu*) i bottoni sono ancorati al centro e spostati con la *Position* (per esempio X −180, Y 150, Size 360 × 100), con l’alignment di default: è la terza via, equivalente. Funzionamento generale dell’UMG, salvo il riferimento al libro.

1.12.4 Il Level Blueprint del menu: il widget, la Game Mode e la camera

Perché il Level Blueprint. “Adesso che ce famo con questa roba?” Il widget va mostrato appena si apre la mappa del menu, e solo lì. Per questo si usa il **Level Blueprint**, “che abbiamo usato nella prima lezione”: ogni livello ha il suo, ed è il posto giusto per la logica che appartiene a *quella* mappa, “perché vogliamo che questa mappa sia dedicata apposta al nostro menu”. Si apre dalla toolbar del Level Editor, dal pulsante dei Blueprint (“c’è il cubetto col più”), **Open Level Blueprint**.

Il widget sullo schermo. “Ci interessa semplicemente usare il Begin Play, perché vogliamo che si crei già da subito”:

1. **Event BeginPlay** → **Create Widget**, con **Class** = WB_Menu, “come abbiamo visto l’altra volta”;
2. **Add to Viewport**, con il **Return Value** del Create Widget nel Target. Il docente pensa per un momento di promuovere il widget a variabile, come per l’HUD, poi rinuncia: “no, beh, non lo facciamo, tanto è il level blueprint”.

Il personaggio di troppo. In Play il menu compare, ma “è molto strano: abbiamo il nostro player dietro”. La mappa usa ancora la Game Mode del progetto, che fa nascere il bot con i suoi controlli. Nel menu, invece, serve “una pawn senza controlli, senza niente”, e Unreal ce l’ha già: nelle **World Settings** della mappa, alla voce **GameMode Override**, si sceglie la **Game Mode base** di Unreal (nella trascrizione “la game mode basica che esiste, la default”), che usa come Pawn la **Default Pawn**. In Play, “già è una cosa diversa: non c’è più il nostro character, sparito, perché in questa mappa ho overridato la modalità

Game Mode classica", quella "proprio base di Unreal, in cui non c'è character in terza persona". "Ma, come potete vedere, non ho controllo" (o meglio, come si vedrà, la Default Pawn si muove ancora: il controllo vero si toglie nel Graph del widget, sezione 1.12.5).

La camera fissa. "Andiamo a vedere anche un'altra cosa": nel livello si aggiunge un **Camera Actor** ("scrivete *camera actor*, eccola qua"), che sarà la vista fissa del menu. Insieme, una **Directional Light** ("e abbiamo messo il sole, ragazzi"), regolabile a piacere, e un muretto, "un po' più carino, così capiamo meglio". Per dire al gioco di guardare da quella camera, di nuovo nel Level Blueprint:

- la **referenza** alla camera: selezionato il Camera Actor nel livello, nel Level Blueprint tasto destro, **Create a Reference to CameraActor**;
- il nodo che cambia la vista. Qui il docente sbaglia strada in diretta: prova da *Get Player Camera Manager*, non trova il nodo giusto e rimanda ("adesso non mi ricordo benissimo, lo vediamo un attimo dopo"). Dopo aver finito il Graph del menu, ritrova la soluzione: "non è che bisogna prendere il camera manager, ma bisogna andare sul controller". Quindi **Get Player Controller** → **Set View Target with Blend**, con la camera nel pin **New View Target**; il *Blend Time* resta al default, "non ci interessa".

"Questo blocco ci permette di blendare una nuova camera": "gli diciamo: guarda, questa è la tua nuova camera, da d'ora in poi" (figura 1.12.2, in alto).

Il Player Start fuori campo. Ultimo ritocco, emerso aiutando gli studenti: la Default Pawn "è una palla", e se nasce davanti alla camera si vede, o si vede il suo riflesso. "Prendete un **Player Start** e mettetelo fuori campo", "così almeno non vi si vede la pallina".

Approfondimento: Game Mode, Default Pawn e Player Start. La **Game Mode** è la classe che stabilisce le regole di una mappa e, fra le altre cose, quale *Pawn* far nascere per il giocatore (*Default Pawn Class*) e con quale *Player Controller*. Il progetto ne ha una predefinita (in *Project Settings*, *Maps & Modes*); il **GameMode Override** delle World Settings la sostituisce *solo per quella mappa*, ed è il modo standard di avere un menu senza personaggio. La classe scelta a lezione, *GameModeBase*, usa come Pawn la **DefaultPawn**: una sfera con un movimento "da spettatore" (WASD e mouse, senza gravità), pensata per fare prove. Il Pawn nasce in un **Player Start**; se la mappa non ne ha, nasce all'origine o nella posizione della camera dell'editor. Un'alternativa ancora più pulita per un menu è una Game Mode con la *Default Pawn Class* impostata su *None*: il Player Controller esiste (e quindi esistono il cursore e gli input dell'interfaccia), ma nessun Pawn nasce. Funzionamento generale dell'engine.

Approfondimento: Set View Target with Blend nel libro. Romero e Sewell descrivono il nodo nel capitolo 15 (*Blueprints Tips*): "è una funzione della classe Player Controller", "usata per passare la vista di gioco fra camere diverse"; "il parametro di input New View Target è l'actor da usare come view target, di solito una camera". Il loro esempio è proprio in un Level Blueprint: quando il giocatore entra nella stanza del tesoro, la vista passa alla camera della stanza. Questo spiega l'errore del docente: il *Player Camera Manager* gestisce la vista, ma la funzione esposta ai Blueprint per cambiarla appartiene al *Player Controller*. Il **Blend Time** a 0 fa uno stacco netto; un valore positivo fa una transizione morbida dalla vista precedente, con una curva scelta in *Blend Func*.

1.12.5 Il Graph del menu: cursore, input e bottoni

Event Construct. Si torna in WB_Menu, scheda **Graph**: "vediamo le funzionalità del nostro menu quando clicchiamo le cose". Dei tre eventi già presenti "non ci serve il Pre Construct, non ci serve il Tick". Serve l'**Event Construct**: "vuol dire quando questo widget è costruito", "è un po' il Begin Play", "viene differenziato dal nome, giusto per non confonderti".

Il cursore e l'input dell'interfaccia. In Play, per ora, "mi posso muovere" (la Default Pawn) e il mouse non si vede. Nell'Event Construct:

1. **Get Player Controller → Set Show Mouse Cursor**, con la spunta attiva: il cursore resta visibile anche cliccando nella finestra di gioco. "Però, ovviamente, posso ancora muovermi";
2. **Set Input Mode UI Only** ("enable input UI", dice prima il docente, poi trova il nodo), con lo stesso Player Controller: "questo ci permette di utilizzare tranquillamente il nostro widget". Il pin *Widget to Focus* resta vuoto: "no, tanto non ci serve".

In Play, ora, "se noi proviamo a muovere lateralmente, non si muove più". "Mi raccomando: questa cosa, quando noi cliccheremo un pulsante, dovrà essere disattivata e rimessa in un'altra modalità."

I colori di hover e pressione. Con il cursore visibile si possono provare gli stati dello Style. Nel Designer il docente cambia il colore di **Hovered**: "voglio che quando io ci passo sopra con il mouse diventi tipo arancione, sì, un bell'arancione" ("lo vediamo in preview, è normale"), e lo copia, con il codice esadecimale, sull'altro bottone. Il risultato, ammette, è "molto Ikea". Il **Pressed** di default diventa grigio, "si vede pochissimo": lo si mette **rosso**, su entrambi i bottoni.

I bottoni. Nel Graph, a sinistra, fra le variabili compaiono **Button_QuitGame** e **Button_StartGame** (sono variabili perché i bottoni, nel Designer, hanno *Is Variable* attivo di default). Ogni bottone ha i suoi eventi, "l'abbiamo visto anche l'altra volta": *On Pressed*, *On Released*, *On Hovered*, *On Unhovered* e **On Clicked**. Si prende l'On Clicked di entrambi.

- **On Clicked (Button_QuitGame) → Quit Game**: "è piuttosto semplice, letteralmente Quit Game è già presente, di standard proprio". ("Adesso non mi ricordo se ti esce proprio dall'editor": si veda l'approfondimento più sotto.)
- **On Clicked (Button_StartGame)**: prima **Get Player Controller → Set Input Mode Game Only**, "molto importante", "in modo tale che ritornino gli input al gioco"; poi **Open Level (by Name)**, con il nome della mappa di gioco. Il docente lo cerca nel Content Browser ("era MLevel1? Ah, questa no, era M level 2?") e trova la mappa del template, **ThirdPersonMap**; prova a copiarne il nome con F2 ("non lo fa, va bene"), e lo scrive a mano.

Tornati nella mappa del menu e salvato tutto, in Play: Play Game "ci carica la nostra bellissima mappina" (figura 1.12.2).

Approfondimento: il menu nel libro. Romero e Sewell costruiscono un menu molto simile nel capitolo 8 (*Creating Constraints and Gameplay Objectives*), il *WinMenu* che compare a fine partita: due bottoni *Btn restart* e *Btn quit* ancorati al centro, ciascuno con un Text dentro ("font size 48"), e gli eventi aggiunti dal pulsante + accanto a *On Clicked*, in fondo ai Details del bottone. Il bottone Restart usa *Open Level (by Object Reference)*, con la mappa scelta da un menu a tendina, seguito da *Remove from Parent*, "che rimuove il widget dalla vista"; il bottone Quit usa *Quit Game*, "così che il giocatore possa chiudere il gioco". Il menu viene mostrato dal personaggio, in un Custom Event *End Game*: Set Game Paused, *SET Show Mouse Cursor* dal Get Player Controller ("per permettere al giocatore di riprendere il controllo del mouse"), Create Widget, Add to Viewport. Nel capitolo 15 il libro elenca anche le tre modalità di input:

- **Set Input Mode Game Only**: "solo il Player Controller riceve gli eventi di input";
- **Set Input Mode UI Only**: "solo la UI riceve gli eventi di input";
- **Set Input Mode Game and UI**: "la UI ha la priorità nel gestire un evento di input, ma se la UI non lo gestisce, lo riceve il Player Controller"; l'esempio è un negozio, in cui si scelgono le opzioni con il mouse ma ci si può allontanare con le frecce.

Una differenza utile da notare: *Open Level (by Object Reference)* non si rompe se la mappa viene rinominata, mentre *Open Level (by Name)*, usato a lezione, richiede il nome esatto (si può scrivere il nome semplice, *ThirdPersonMap*, o il percorso completo, se esistono due mappe con lo stesso nome).

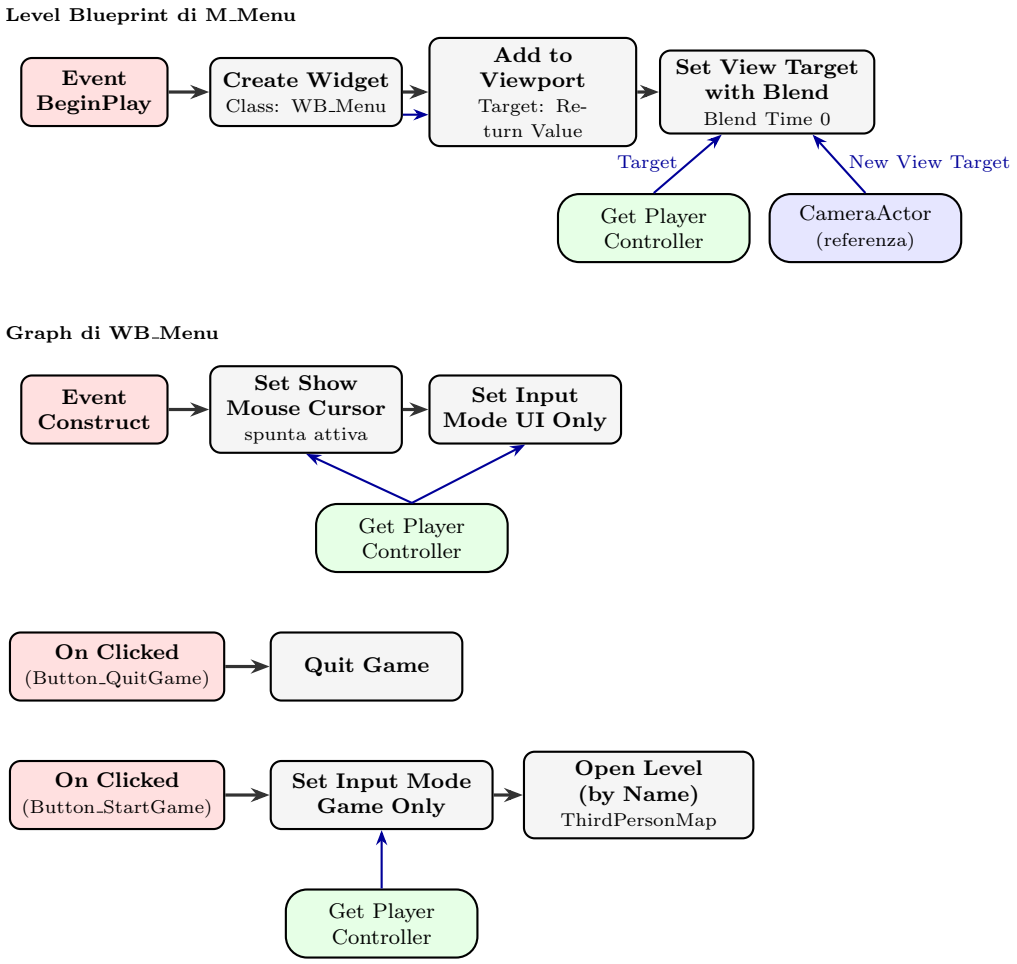


Figure 1.12.2: I grafi del menu. In alto, il Level Blueprint della mappa M_Menu crea il widget, lo mostra e passa la vista alla camera fissa. In basso, il widget prende il controllo del cursore quando nasce, esce dal gioco con Quit e, con Play Game, restituisce l'input al gioco e apre la mappa di gioco. Ricostruzione dalla lezione.

Approfondimento: Construct, Quit Game e il cursore nella mappa di gioco. Alcune nozioni generali, non della lezione. **Event Pre Construct** viene eseguito anche nell'editor, ogni volta che il widget viene ridisegnato nel Designer: serve per anteprime e impostazioni visive; **Event Construct** solo in gioco, quando il widget viene aggiunto allo schermo (e di nuovo se lo si rimuove e lo si aggiunge). **Quit Game**, durante il Play in Editor, *termina la sessione di gioco* e riporta all'editor, senza chiudere l'editor; in un gioco pacchettizzato chiude l'applicazione. Il dubbio del docente si scioglie così. Infine, *Open Level* distrugge il mondo corrente con tutti i suoi actor, widget compresi, e crea un Player Controller nuovo: per questo, nella mappa di gioco, il menu sparisce da solo e il cursore torna nascosto, anche senza un *Remove from Parent* o un *Set Show Mouse Cursor* a falso. Il *Set Input Mode Game Only* prima dell'Open Level, raccomandato dal docente, è comunque una buona abitudine: il caso in cui non è superfluo è quello di un menu che si chiude *senza* cambiare livello, come un menu di pausa (nel libro, capitolo 11, il *Resume* fa Remove from Parent, Set Show Mouse Cursor a falso e Set Game Paused a falso).

1.12.6 Il diorama, e i problemi degli studenti

Il diorama. Con il menu funzionante, "passiamo sopra" all'aspetto. Sullo sfondo, "un piccolo diorama, una piccola box, in cui ci andiamo a mettere tutto quello che vogliamo". "Volendo, potete fare tutto un menu interattivo": gli ostacoli, i Blueprint creati nel corso, i bottoni, la torretta. Il docente si limita ai modelli:

- il **bot**: dal Content Browser, o la sua Skeletal Mesh, "o li metto direttamente l'animazione in Idle": trascinando nel livello un asset di animazione, Unreal crea un actor con la mesh che la riproduce;
- la **porta** modulare del compito della volta scorsa, **SM_Modular_Wall_Door** ("wall door, eccola qua"), con le ante **SM_Door**;
- qualche ritocco alla **camera** e alla **luce**: l'intensità a 10 è "enorme", poi 5, poi valori molto più bassi (la trascrizione, qui, è quasi incomprensibile: si sentono "0,01" e "1").

"Questo caso è più una cosa artistica, ragazzi, quindi potete fare un po' come volete." Le funzioni di base, entrare nel gioco e uscire, ci sono. "Questo è un menu che io di solito faccio quando faccio i miei prototipi", cioè "quando devo semplicemente mostrare qualche cosa a livello di gameplay". Un menu delle *opzioni* "è già un discorso molto più" complesso, "perché bisogna andare a vedere gli slider": non si tratta, "perché tanto poi andiamo a fare qualcosa di più gameplayoso".

I problemi dei presenti. Diversi studenti hanno ancora il personaggio nel menu.

- **La camera.** Ripasso passo per passo: selezionare la camera nel livello ("proprio diventa arancione"), Open Level Blueprint, tasto destro, Create a Reference to CameraActor ("devi stare dentro il Level Blueprint"); Get Player Controller, Set View Target with Blend, la camera nel New View Target.
- **Il personaggio che nasce ancora.** "Devi passare alla Game Mode default": **World Settings**, **GameMode Override**. Uno studente non trova la finestra delle World Settings (si apre dal menu *Window* dell'editor, se è chiusa). Poi il Player Start fuori dalla camera.
- **L'errore su Add to Viewport.** "Mi dà un problema proprio su Add to Viewport": "perché il target di base è Self, quindi non so se devo cambiare il target". Sì: il Target deve essere il **Return Value** del Create Widget. "Probabilmente devi dare return value" al target.
- **Il mouse che non si vede.** Va impostato nell'Event Construct del widget: Get Player Controller, Set Show Mouse Cursor, Set Input Mode UI Only. Lo studente aveva cancellato l'Event Construct ("basta che fai tasto destro" e lo si ricrea) e non aveva messo la **spunta** nel Set Show Mouse Cursor ("ah, però non avevo messo il tick"): il nodo, senza spunta, imposta il cursore a *falso*. "Funziona? Sì."

Chiuso il menu, si torna alla ThirdPersonMap.

1.12.7 Interfacce e Blueprint Component

"Andiamo sul nostro amico fidato": il docente apre il suo bot ("ah no, questo è Boto: dobbiamo andare su Boto2"), cioè BP_Bot2. "Oggi parleremo delle interfacce, ragazzi."

L'interfaccia, un intermediario. "Immaginatevi che un'interfaccia è un intermediario fra due o più Blueprint." Il docente mostra quella già pronta nel suo progetto, nella cartella dei componenti dei Blueprint: un tipo particolare di Blueprint che contiene solo tre **funzioni**, *Interaction Start*, *Interaction End* e *Interaction*. L'immagine: un **volo di linea**, "un'interconnessione tra due o più poli", che "trasporta molto spesso le stesse cose, passeggeri di un tipo, passeggeri di un altro". "Quindi è un qualcosa che va a trasportare varie funzioni da una parte all'altra." Nel caso di oggi l'interfaccia comunica tre messaggi: *sto iniziando un'interazione* (ti sto guardando), *sto finendo un'interazione* (non ti guardo più), *sto interagendo* (ho premuto il tasto).

Il componente, una funzionalità isolata. Il lavoro di *vedere se c'è qualcosa di interagibile* non lo fa il personaggio, ma un **Blueprint Component**: "dei componenti che noi possiamo aggiungere all'interno dei nostri actor per dare funzionalità in modo isolato". Per esempio: "questo actor può tracciare degli interactable"; "questo actor ha la vita". La vita, fatta la volta scorsa *dentro* il personaggio, si potrebbe spostare in un componente, "perché magari abbiamo anche i nemici che hanno la vita", o "degli oggetti che si possono rompere". Lo scopo è "isolare sempre più possibile le funzionalità, e quindi essere anche modulari". Infatti il BP_Bot2 del docente, a parte un input nuovo, "non ha niente di nuovo": si gira verso l'oggetto e suona un Montage, ma "non ha niente per tracciare": il trace è tutto nel componente.

Actor Component o Scene Component. Si crea una **Blueprint Class** e, fra le classi comuni, non un Actor ma un componente. Ce ne sono due:

- l'**Actor Component** "mette semplicemente la funzionalità";
- lo **Scene Component** "è un Actor Component che ha anche una scena", cioè un **Transform**: "non so se vi ricordate la Scene, quando l'abbiamo messa: il punto vuoto". "Almeno si può spostare nello spazio."

Qui serve il secondo, perché il trace è "un raggio che da quel punto parte e in un punto finisce", e il punto di partenza si deve poter spostare. Il docente lo mostra nel suo bot: spostando il componente nel Blueprint del personaggio, in Play il raggio parte dal nuovo punto; lo rimette "al centro della testa, più o meno". Il nuovo Blueprint: **BPC_RayInteract** (BPC sta per *Blueprint Component*; il docente, che ne ha già uno, chiama il suo BPC_RayInteract1).

Il componente aperto. "Come potete vedere, presenta delle similitudini con tutti i Blueprint." "L'unica cosa: non ha una viewport, perché non ha una forma fisica", "tra virgolette, perché è una scena, ma è vuota". "E ricordiamoci che è un componente: quindi sarà qualcosa che deve essere per forza attaccato a qualcos'altro."

Che cos'è una funzione. Prima di scrivere la logica, una domanda: chi ha già sentito parlare di **funzioni** in informatica? Risponde **Federico**, che ha studiato da programmatore: "la funzione è un insieme di istruzioni: chiami la funzione e fai quelle istruzioni". "Esattamente." Il docente indica una catena di nodi nel bot: "gli eventi non possono essere messi come funzioni, ma questo qua può essere una funzione". È "un modo per ordinare anche il lavoro in varie tipologie di funzionalità, che possono essere magari trasportabili da un punto all'altro": "sempre questo senso di modularità". ("Grande Federico, sempre miglior assistente.")

Approfondimento: i componenti nel libro. Romero e Sewell dedicano ai componenti la seconda metà del capitolo 18 (*Creating Blueprint Libraries and Components*). "Quando creiamo un Actor Blueprint aggiungiamo spesso componenti con una funzionalità incapsulata, pronta all'uso, come Projectile Movement, Static Mesh e Collision"; "possiamo anche creare i nostri Actor Component con i Blueprint". Nel pannello

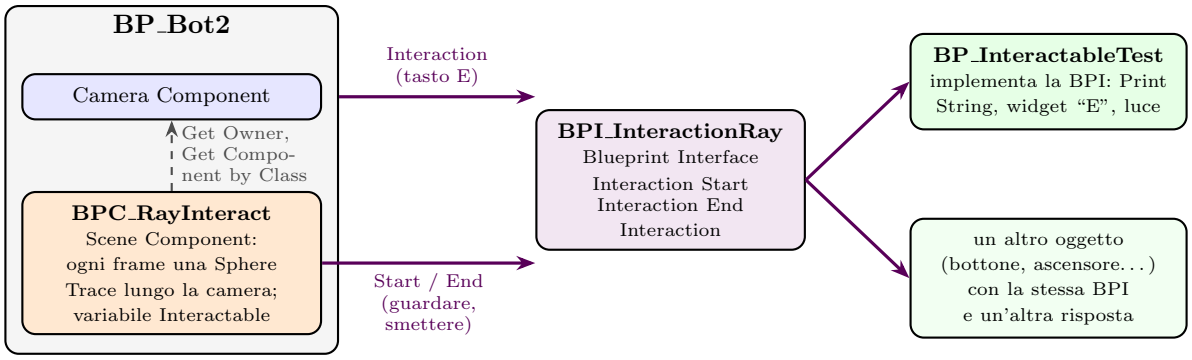


Figure 1.12.3: I ruoli del sistema di interazione. Il componente, attaccato al personaggio, trova la camera del suo proprietario, traccia ciò che il giocatore guarda e invia i messaggi di inizio e fine; il personaggio invia il messaggio Interaction quando si preme E. I messaggi viaggiano attraverso l'interfaccia: chi li riceve decide che cosa fare, e un actor che non la implementa li ignora. Schema ricostruito dalla lezione.

Pick Parent Class le due classi sono fra quelle *comuni*: "lo Scene Component è una classe figlia dell'Actor Component che ha la struttura Transform (location, rotation e scale). Grazie al Transform, uno Scene Component può essere attaccato a un altro Scene Component." E, come a lezione: "quando si scrive la logica di un componente nell'Event Graph, si può ottenere la referenza all'actor che usa il componente con il nodo **Get Owner**". I due esempi del libro mostrano bene la differenza: *BP_ExpLevelComp* è un Actor Component che gestisce punti esperienza e livelli (non ha bisogno di una posizione), mentre *BP_CircularMovComp* è uno Scene Component che ruota attorno all'actor e a cui si attacca una Static Mesh, per simulare uno scudo rotante: "lo Scene Component è usato come posizione di riferimento per altri componenti". Il componente della lezione è dello stesso tipo: la sua posizione è il punto di partenza del raggio.

1.12.8 Il componente BPC_RayInteract: la Sphere Trace

La funzione LookForInteractions. Nel componente si crea una nuova funzione ("eccolo che vi si crea questo pannellino violaceo"), **LookForInteractions**. Dentro, una **Sphere Trace By Channel**. "Seguitemi per bene."

- **Radius**: **5**, "perché se no non vi apparirà mai". Con raggio zero la Sphere Trace degenera in un raggio sottilissimo (e il debug quasi non si vede); 5 unità bastano a renderla visibile e un po' più tollerante.
- **Start**: **Get World Location** del componente stesso, "la posizione nel mondo del nostro Scene Component: dovrà partire da qui".
- **End**: da costruire, ed è la parte con il trucco.

Prima la camera: Get Owner e Get Component by Class. Il raggio deve seguire lo sguardo del giocatore, cioè la *rotazione della camera*. Il componente, però, non sa nulla della camera del personaggio: deve trovarla. Nell'**Event Graph** del componente, all'**Event BeginPlay**:

1. **Get Owner**: "la referenza del proprietario di questo Scene Component";
2. dal proprietario, **Get Component by Class** ("non *components*, *component*, perché se no vi dà un array"), con **Component Class** = **Camera Component**: "sarà il nostro punto principale da cui andremo a fare il nostro trace";
3. il risultato si promuove a variabile, **CameraComponent**, e si collega tutto al Begin Play.

"Perdonate, sono 30.000 componenti: un componente su componente su componente." In sintesi: "in Begin Play io prendo il mio proprietario, prendo la sua camera, mi salvo la referenza alla sua camera, e poi la useremo all'interno del nostro componente".

La camera, se c'è. Il componente deve funzionare anche su un actor *senza* camera: "se il nostro proprietario non ha una camera, allora prendi te stesso come pivot". Quindi un **Select** fra due oggetti:

- l'indice è un booleano, **Is Valid** ("la F verde", la versione *pura* della funzione) sulla variabile CameraComponent: "se il componente camera esiste, quindi il nostro actor ha effettivamente una camera, e quindi questa variabile CameraComponent ha qualcosa dentro";
- su **True**, CameraComponent: "usami la camera come punto di riferimento";
- su **False**, **Self**: "se no, usi te stesso".

Il Select era già comparso con la torretta, per scegliere fra le due bocche di fuoco: lì con un *Random Bool*, qui con l'Is Valid. "So che è un po' complesso."

La "mandragata": il tipo del Select. Il problema è il *tipo* delle opzioni. "Il Select praticamente c'è un piccolo problema": il nodo è jolly (*wildcard*) e assume il tipo della *prima* cosa che gli si collega. Collegando prima la camera, il Select diventa un Select di *Camera Component Object Reference*, e Self (il componente BPC_RayInteract) non entra più; collegando prima Self, il Select diventa un Select di riferimenti a BPC_RayInteract, e la camera non entra. Entrambi però *sono* Scene Component: la camera è uno Scene Component, e lo è anche il nostro componente. Il trucco ("qui bisogna fare una mandragata, ragazzi"):

1. si crea una variabile qualunque di tipo **Scene Component Object Reference** (il docente la chiama *Scene*: "e poi non ci servirà a niente");
2. se ne mette un Get nel Select: ora le opzioni sono di tipo *Scene Component*;
3. si **cancella** il Get e si collegano CameraComponent e Self, che sono entrambi accettati.

"Ecco fatto che abbiamo praticamente fregato Unreal."

La domanda di Filippo. "Non ho capito perché abbiamo creato la variabile Scene prima." Il docente rifà la prova davanti a tutti: un Select nuovo, il cursore sul pin, la scritta "Scene Component Object Reference"; poi Self in un Select vuoto, e il nodo diventa "Self Object Reference", "mi farà fare solamente un select fra due self, non due Scene Component". "Per fregare Unreal" serve "una variabile che appartiene generalmente allo Scene Component". Uno studente prova a trasformare la variabile in Self: "non è che te lo trasforma: lo devi cancellare, e poi tasto destro, scrivi *self*". Filippo insiste: e se avessimo collegato subito la camera? Prova del docente: "Select camera component, camera component object reference, e io, se provo ad attaccare Self, non me lo attacca". "Ok, capito."

Il punto finale. Dal risultato del Select, "quello che abbiamo sempre fatto":

1. **Get Forward Vector** (della camera, o del componente stesso);
2. moltiplicato per una distanza: il secondo pin del Multiply si converte in float ("tasto destro, mi raccomando: *To Float*"), si promuove a variabile, **DistanceTrace**, con valore **300**;
3. il prodotto si **somma** (Add) allo Start, e il risultato va nell'**End**.

Perché lo Start è il componente. Una studentessa chiede perché lo Start sia la World Location: "in relazione a dove attacchiamo il componente?" "Esatto." "Il punto iniziale deve essere sempre il punto, il Transform, del nostro componente"; "il punto che deve diversificare è la camera". Il docente lo mostra nel bot: guardando in basso il raggio va in basso, guardando di lato va di lato, ma parte sempre dal componente, "così magari posso posizionarlo avanti, posizionarlo indietro, posizionarlo qui, posizionarlo là" (figura 1.12.4).

Il componente nel personaggio. Nel BP_Bot2 ("a bot2, in questo caso, è il vostro character"), **Add** e il nome del componente: BPC_RayInteract compare fra i componenti del personaggio. Per vederlo si imposta il **Draw Debug Type** della Sphere Trace ("lo metto a *For One Frame*"). In Play, però, non succede niente: "ah, certo, ecco perché: non facciamo LookForInteractions". La funzione va chiamata nell'**Event Tick** del componente ("ah, nel tick, infatti"), così il controllo avviene a ogni frame. Con il debug persistente (*Persistent*) il raggio lascia una scia "gigantesca"; con *For Duration* resta qualche istante; con *For One Frame* "rimane un po' più carino", e si vede solo il raggio attuale. (La trascrizione, qui, è piena di prove e di parole storpiate: "la duration vado a activate, troppo corto", "Lugano", "Adobe"; anche una confusione momentanea fra il bot e il bot2 del docente.)

Approfondimento: Sphere Trace, Tick e costo. Una **Sphere Trace** è un *line trace* "spesso": fa scorrere una sfera del raggio indicato dallo Start all'End e restituisce il primo oggetto che la sfera tocca nel canale scelto. Rispetto a un raggio sottile perdona di più la mira (con 5 unità di raggio, circa 5 cm, conta anche sfiorare il bordo di un oggetto), ed è il motivo per cui molti sistemi di interazione la preferiscono. Farla nel **Tick**, una volta per frame, costa poco: una singola trace di 300 unità è un'operazione leggera, e il sistema la esegue solo per il personaggio del giocatore. Se gli actor con il componente fossero molti (per esempio dei nemici), conviene ridurre la frequenza con un timer (come quelli della stamina, a 0,1 secondi) o disattivare il Tick del componente quando non serve (*Set Component Tick Enabled*). Una Sphere Trace può stare senza problemi dentro una funzione, perché non ha uno stato interno da ricordare fra una chiamata e l'altra; il Do Once e il Flip Flop, invece, sì (si veda più avanti). Funzionamento generale dell'engine.

1.12.9 Il Trace Channel e la Blueprint Interface

Un canale per le interazioni. "Ha colpito qualcosa? Sì, no." Prima di leggere il risultato serve un canale dedicato, "che ci permette di identificare gli oggetti che vogliamo che siano interagibili": "se no, praticamente, interagirà con tutto". "Come abbiamo visto anche nella nostra torretta": **Project Settings**, *Collision*, **New Trace Channel**, nome **InteractionTrace**, risposta di default **Ignore**. Il docente, che ce l'ha già, non lo rifà. Nella Sphere Trace, il **Trace Channel** diventa InteractionTrace: "mi raccomando, se non vi appare, fate compila, perché ha bisogno un attimo di compilare il nuovo Trace Channel".

Che cosa abbiamo colpito. Dopo la trace, un **Branch** sul suo Return Value ("ha colpito qualcosa?") e un **Break Hit Result** sull'Out Hit, da cui, "vi ricordate, no?", escono l'actor colpito, il Physical Material "e tutto". Il piano: "useremo l'interfaccia, la Blueprint Interface, per andare a comunicare con questo actor": "ha questo porto in cui può accogliere questa tipologia di interazione? Sì? Benissimo, allora fai l'interazione". Dall'**Hit Actor** si chiede **Does Object Implement Interface**: "ehi, questo actor ce l'ha l'interface?" Ma l'interfaccia non esiste ancora: "ci fermiamo qua per adesso, creiamo l'interface, e ci fermiamo". (Il docente parla qui anche di promuovere l'Hit Actor a variabile; la variabile di appoggio vera e propria, *Interactable*, si crea dopo la pausa, sezione 1.12.11.)

La Blueprint Interface. Nella cartella *Blueprints*, *Components*, tasto destro, **Blueprint**, "non Blueprint Class questo, ma Blueprint questo": il sottomenu offre *Blueprint Class*, *Blueprint Function Library* e **Blueprint Interface**. Il nome: **BPI_InteractionRay**. Aperta:

- il grafo è **in sola lettura**: "non possiamo far niente", "cannot create new nodes in read only graph". Un'interfaccia contiene solo *dichiarazioni*;
- si possono aggiungere **funzioni**: la prima c'è già e si rinomina (con F2), **Interaction Start**; con *Add Function* le altre due, **Interaction End** e **Interaction**.

"Abbiamo messo che in questo cargo, questo trasporto, ci stanno Interaction Start, Interaction End e Interaction: quindi queste funzioni possono essere passate tra di loro."

Il collegamento. Nel componente, il pin **Interface** di Does Object Implement Interface diventa BPI_InteractionRay. "Se quel qualcosa che ho colpito ha questa interfaccia, allora andremo a fare tutto il procedimento di interazione." "Ci sono domande? Mi sembra di capire no." Sono le 17:26: pausa fino alle 17:55, "diciotto".

Approfondimento: la Blueprint Interface nel libro. Romero e Sewell presentano le interfacce nel capitolo 16 (*Introduction to VR Development*), nel paragrafo *Blueprint Communication using interfaces*: "una Blueprint Interface è un tipo speciale di Blueprint che contiene solo nomi di funzioni e parametri. Si usa per permettere la comunicazione fra tipi diversi di Blueprint." Si crea dal pulsante *Add* del Content Browser, sottomenu *Blueprints*, *Blueprint Interface*. L'esempio è il template VR: l'interfaccia *VRInteraction BPI* è aggiunta alla pistola dalle *Class Settings*, categoria *Interfaces*, e la pistola implementa la funzione *Trigger Pressed*: "siccome la funzione non ha parametri di output, è implementata come un *evento*". Il Pawn VR chiama Trigger Pressed sull'actor che tiene in mano, "e se l'actor ha implementato l'interfaccia VRInteraction BPI, Trigger Pressed viene eseguito. Non succede niente se l'actor non l'ha implementata." È esattamente lo schema della lezione: il personaggio non sa *che cosa* sta guardando (un cubo, un bottone, un ascensore) e non deve fare un *cast* a nessuna classe; manda il messaggio, e ciascun oggetto risponde a modo suo. Questo è il vantaggio delle interfacce rispetto al cast e all'ereditarietà viste con BP_Activable: classi che non hanno niente in comune (un actor, un personaggio, un oggetto fisico) possono condividere la stessa "porta".

1.12.10 La pausa: la stamina di Tommaso

Il sintomo. In pausa, come promesso, il problema rimasto aperto la volta precedente: **Tommaso** tiene premuto Shift, la barra della stamina non scende, e "appena mollo, si vede". Il docente ripercorre il codice condiviso: l'evento StopRunning, il Clear and Invalidate, il Set Timer by Event e la variabile TimeStamina ("quanto sta? 0,92"; nella trascrizione il valore è questo, ma va preso con cautela: nella lezione precedente il valore indicato era 0,05), RegainStamina, il Clamp con CurrentStamina e MaxStamina, l'InitializeStaminaBar nel Begin Play. "Sembra tutto a posto."

Il debug. Il metodo è quello della volta scorsa: un **Print String** con la CurrentStamina ("attaccalo al viola") dopo l'evento di aggiornamento. I valori scendono correttamente ("500, 500, 300" dice la trascrizione, forse leggendo la velocità), poi risalgono. Ma a schermo ci sono "troppi Print String": si cercano e si tolgono quelli vecchi, uno per volta (quello della torretta, uno della piattaforma, "forse i mulini?", "forse il collectable"; "nessuno è perfetto"). Il comportamento: la barra *ricarica* visibilmente, ma *non scende*.

La causa. "Ah, ecco perché: non chiami l'evento Call Update Stamina." Nella catena di **StaminaDeduction**, dopo il Set di CurrentStamina, mancava la **Call E_UpdateStamina**: il valore calava, ma nessuno lo comunicava alla barra. La Call c'era solo in RegainStamina, che parte al rilascio del tasto; per questo la barra si aggiornava "solo quando mollo". Il rimedio: la Call anche in StaminaDeduction, con CurrentStamina e MaxStamina collegati. "Perfetto. Ottimo."

Il rumore. Il resto della pausa, nella trascrizione, è il blocco di testo ripetuto e il frammento di video già segnalati nell'introduzione. Alla ripresa, uno studente chiede aiuto per BPI_InteractionRay, che dopo un riavvio di Unreal si apre "male": il grafo in sola lettura non si vede. Il docente prova i menu *Window* (Bookmarks, Find Results, Compiler Results...), carica per sbaglio il layout di default dell'editor ("mi sono caricato il bellissimo layout di Unreal"), e alla fine trova la via più semplice: "fai doppio clic sulla funzione" nel pannello *My Blueprint*, e il suo grafo si apre. "Tanto non ti serve": nell'interfaccia non si scrive nulla.

1.12.11 La logica: la variabile Interactable

"Vai, allora finiamo questo bellissimo sistema di interazioni." Si torna in BPC_RayInteract, nella funzione LookForInteractions. Il ramo di partenza è quello in cui la trace ha colpito qualcosa e l'actor colpito

implementa BPI_InteractionRay.

La variabile d'appoggio. Serve una variabile di tipo **Actor** (*Object Reference*), **Interactable**: "sarà la nostra variabile d'appoggio", cioè l'oggetto interagibile che il giocatore sta guardando *in questo momento*, o niente. Il suo scopo è evitare di ripetere il lavoro a ogni frame: "se l'actor che io ho intercettato con il mio trace è lo stesso di prima, allora non voglio che mi rifai alcune sezioni del codice"; "se è lo stesso, non lo svuoto".

È un oggetto nuovo? Un secondo **Branch**, con la condizione **Hit Actor diverso da** Interactable: il nodo *Not Equal* ("diverso si fa con il punto esclamativo e l'uguale"). "Il punto esclamativo in informatica vuol dire che questo è un condizionale che ci dice: guarda, questo è diverso da questo, e ci restituisce un booleano. Se è diverso ci dà True, se sono uguali ci dà False." Sul False non si fa nulla: è lo stesso oggetto di prima.

I messaggi dell'interfaccia. "Adesso, importante": scrivendo *interaction* nel menu dei nodi, compaiono già le funzioni di BPI_InteractionRay, "senza fare niente": "ho già la possibilità di poter comunicare e messaggiare con l'interfaccia". Sono i nodi **(Message)**, con l'icona della busta, che si possono chiamare su qualunque actor. Servono **Interaction End** e **Interaction Start**.

Il ramo True: chiudere il vecchio, aprire il nuovo. Sul True del diverso:

1. un **Validated Get** di Interactable (si mette il Get, tasto destro, *Convert to Validated Get*): "questa variabile Interactable è valida? C'è qualcosa dentro?";
2. **Is Valid**: c'era già un oggetto, quindi prima si chiude la vecchia interazione, **Interaction End** con target il *vecchio* Interactable; poi **Set Interactable** = Hit Actor; poi **Interaction Start** con target il *nuovo* Interactable;
3. **Is Not Valid**: "la variabile è vuota", "non è mai iniziata un'interazione precedente"; si salta l'End e si fa direttamente il Set Interactable = Hit Actor e l'**Interaction Start**.

Il ramo False: non guardo niente. Se la trace non colpisce nulla, *oppure* colpisce un actor senza l'interfaccia ("il muro, ad esempio"), il giocatore ha distolto lo sguardo: "allora ti finisco l'interazione". Entrambi i False (quello del Branch sul Return Value e quello di Does Object Implement Interface) portano a un altro Validated Get di Interactable:

- **Is Valid**: **Interaction End** sul vecchio Interactable, poi **Set Interactable** senza collegare nulla al pin, cioè lo si **svuota**;
- **Is Not Valid**: non c'era nessuna interazione da chiudere; per sicurezza lo si svuota comunque ("semplicemente dirgli: guarda, svuotati").

"Quindi mettere il Set così è come mettere uguale a null?" "È nulla, proprio. La annulliamo: la mettiamo vuota, e quindi diventerà non valida." Il caso del primo ramo, il passaggio diretto da un oggetto all'altro, serve quando due interagibili sono vicini: "lui finisce con quello precedente e inizia con il nuovo"; di solito, invece, gli oggetti sono distanti, la variabile si svuota e poi si riempie di nuovo (figure 1.12.4 e 1.12.5).

Il problema di tipo. Uno studente non riesce a collegare l'Hit Actor al Set Interactable: "mi dice che non sono compatibili, perché uno è un riferimento oggetto e l'altro è un componente di scena". La variabile Interactable era stata creata con il tipo sbagliato (ed era cambiata per un clic involontario in *Geometry*, "che non ho cliccato"). Rimedio: nei Details della variabile, il tipo **Actor**, **Object Reference** ("non quell'Actor: tipo oggetto, riferimento oggetto"), confermare la modifica del tipo, poi staccare e riattaccare i fili rossi dei nodi che usavano la variabile, "era semplicemente sparito".

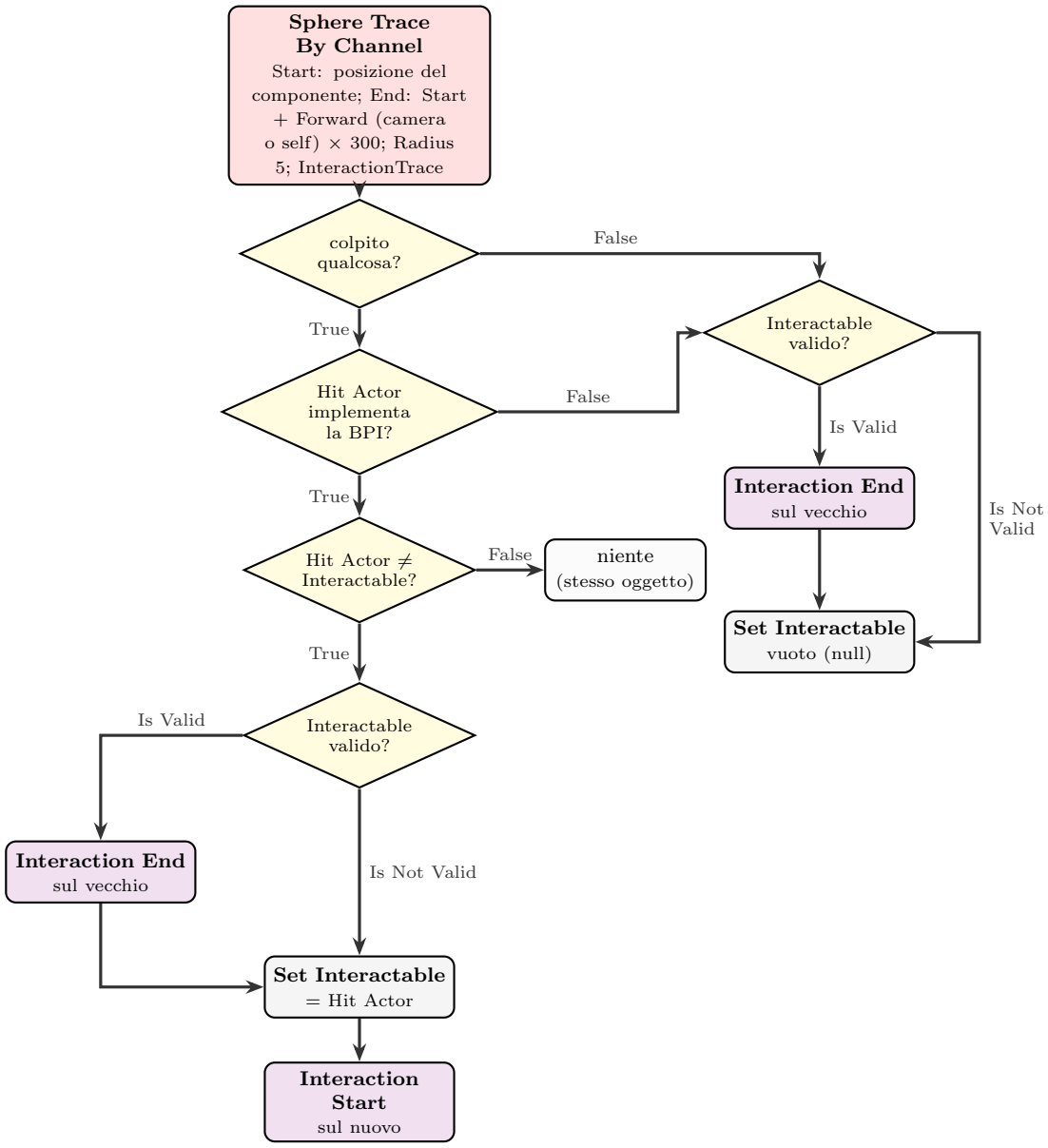


Figure 1.12.4: La funzione LookForInteractions del componente, chiamata nell'Event Tick. Se il giocatore guarda un nuovo interagibile, si chiude l'interazione con il vecchio (se c'era) e si apre con il nuovo; se guarda un oggetto non interagibile, o niente, si chiude l'interazione in corso e si svuota la variabile. I nodi viola sono messaggi dell'interfaccia. Ricostruzione dalla lezione.

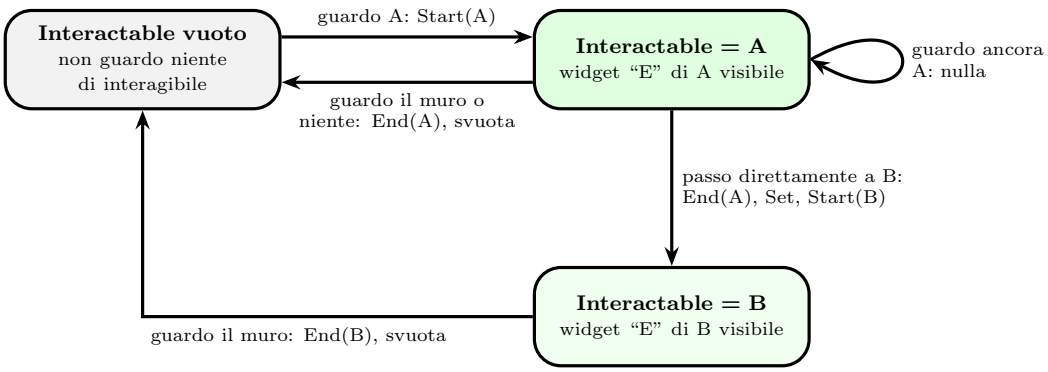


Figure 1.12.5: La variabile Interactable come stato. Ogni Interaction Start è sempre seguito, prima o poi, dal suo Interaction End: l'oggetto può contare sulla coppia per mostrare e nascondere il suggerimento. Schema ricostruito.

Il Tick. "Importante, ragazzi: far sì che questa funzione venga fatta su ogni frame." Nell'Event Graph del componente, **Event Tick** → LookForInteractions (se non lo si è già fatto): "a ogni frame farà il controllo: c'ho qualcosa di interagibile davanti a me, dove sto puntando la camera? Sì o no?".

Approfondimento: messaggi, Does Implement Interface e Validated Get. Tre precisazioni generali sui nodi usati. Primo: un nodo *(Message)* di un'interfaccia si può chiamare su qualunque riferimento a oggetto; se l'oggetto non implementa l'interfaccia, la chiamata semplicemente non fa nulla, senza errori (la frase del libro citata sopra: "non succede niente"). Il controllo **Does Object Implement Interface**, quindi, non serve a evitare un errore, ma a *distinguere* gli interagibili dagli altri oggetti, così da chiudere l'interazione quando si guarda un muro. Secondo: il confronto *Not Equal* fra due riferimenti confronta l'*identità* degli oggetti (è lo stesso actor?), non la loro classe; due cubi identici sono comunque diversi. Terzo: il **Validated Get** è un Get con l'*Is Valid* incorporato, con le due uscite *Is Valid* e *Is Not Valid*; "validare" un riferimento prima di usarlo è la raccomandazione con cui il docente chiude il corso, perché un riferimento vuoto o a un actor distrutto (per esempio un interagibile raccolto o eliminato) produce, se usato, gli errori *Accessed None* a fine Play. Funzionamento generale dell'engine.

1.12.12 L'oggetto di prova: BP_InteractableTest

L'actor. "Adesso creiamo il nostro actor interagibile." Nella cartella dei Blueprint, una Blueprint Class di tipo **Actor**, "veramente semplicissimo": **BP_InteractableTest**. Due componenti: un **Cube** e un **Widget** ("fate widget, che poi faremo"), messo sopra il cubo.

Implementare l'interfaccia. "Non si trova", se si cerca l'interfaccia fra i componenti: va aggiunta in **Class Settings**, nella toolbar. Nei Details compaiono le categorie *Imports* (*Default Namespace*, *Imported Namespaces*) e **Interfaces**, con *Inherited Interfaces* e **Implemented Interfaces**; in quest'ultima, **Add**, BPI_InteractionRay. "Per vedere che l'hai aggiunto": nel pannello My Blueprint compare la sezione **Interfaces**, con Interaction, Interaction End e Interaction Start.

Implementare le funzioni. Trascinare Interaction Start nel grafo non funziona: l'editor avverte che la funzione "was not marked as Blueprint Callable" e non può essere piazzata così. "Dobbiamo overrideare la loro funzione, che di base è nulla": le funzioni dell'interfaccia "sono solo degli appoggi", vuote, e il loro contenuto "sarà da overrideare proprio dal livello di design". Con un doppio clic (o tasto destro, *Implement Event*) si ottengono tre eventi rossi: **Event Interaction Start**, **Event Interaction End** ed **Event Interaction**. Ciascuno con un **Print String**:

- Interaction Start: "inizio interazione", **Text Color** verde;

- Interaction End: "fine interazione", rosso;
- Interaction: "interagendo", giallo-arancione ("ancora non l'abbiamo usato, quindi non lo vedremo, è normale").

Il colore si trova aprendo la tendina del Print String (la freccia in basso del nodo).

La collisione. "È importante però una cosa": il canale InteractionTrace è in Ignore di default, quindi il cubo, così com'è, non viene visto dal raggio. Nella **Collision** del Cube, preset *Custom* e **InteractionTrace** su **Block**. "Su Cube, tanto la mesh è quella."

La prova. Si mette il cubo in scena. In Play: "inizio interazione, fine interazione, inizio interazione, fine interazione". Con la camera in terza persona colpire il cubo è un po' difficile, "ma ci sono delle volte in cui diventa verde". "Vi funziona?" "Sì, funziona." "Quindi adesso abbiamo creato il nostro trace per quanto riguarda le interazioni." Il componente "fa le veci di *vado a controllarti se ci sono delle interazioni, e gestisco tutta la cosa*"; l'interazione vera, il tasto, la fa il personaggio, "in modo classico".

Approfondimento: eventi o funzioni dell'interfaccia. Le tre funzioni di BPI_InteractionRay non hanno *output*: per questo, nell'actor che le implementa, diventano *eventi* nell'Event Graph (è la stessa regola citata dal libro per la Trigger Pressed della pistola VR). Se una funzione dell'interfaccia avesse un valore di ritorno (per esempio un *GetInteractionText* che restituisce "Apri" o "Accendi"), andrebbe implementata come *funzione*, con un doppio clic nella sezione Interfaces, e potrebbe essere chiamata per leggere un dato. Un'interfaccia non ha stato né variabili: ogni classe che la implementa decide da sé che cosa fare, e può anche lasciare vuote le funzioni che non le servono. Funzionamento generale dell'engine.

1.12.13 L'input: IA_Interact e il tasto E

Nel BP_Bot2 serve un nuovo input. "Come si crea un nuovo input? Con l'IMC e con l'Input Action."

1. Nel Content Browser si va alle Input Action del progetto (cliccando su IA_Jump, *Show in Folder View*);
2. "basta che fate una copia di IA_Run": **Duplicate**, nome **IA_Interact**; "basta che qua dentro abbia i **Triggers** *Pressed* e *Released*" (come quelli di IA_Run), da lasciare così;
3. nell'**IMC** di default (*IMC_Default*), il più accanto alle *Mappings* aggiunge una riga per IA_Interact; poi, cliccando sull'icona della tastiera e premendo il tasto, si assegna **E**.

Il bot2 "eredita la stessa cosa": usa lo stesso IMC_Default del template. Le domande dei presenti sono tutte sulla tendina dell'IMC ("non mi fa mettere l'azione di input": "è la tendina") e sui trigger ("il primo è premuto e l'altro qual è?": "premuto e rilascio va benissimo così, lasciato così"). Nel personaggio, cercando *IA_Interact*, compare il nuovo evento d'input.

Approfondimento: Pressed e Released nello stesso trigger. Un'Input Action digitale senza trigger scatta in *Triggered* a ogni frame finché il tasto è premuto; con il trigger *Pressed* scatta una sola volta alla pressione, con *Released* una volta al rilascio. Con *entrambi*, come in IA_Run e quindi in IA_Interact, l'uscita *Triggered* scatta due volte per ogni pressione: quando si preme e quando si rilascia. Nella lezione questo non dà problemi, perché il grafo del personaggio passa da un *Do Once* che si resetta solo alla fine del Montage: il secondo impulso, al rilascio, arriva mentre l'animazione è in corso e viene ignorato. Ma se il Montage fosse molto breve, o il giocatore tenesse premuto a lungo, il rilascio potrebbe far partire una seconda interazione: per un'interazione "a pressione" basta il solo trigger Pressed, o l'uscita *Started*. La trascrizione non dice quale uscita dell'evento IA_Interact usi il docente. Funzionamento generale dell'Enhanced Input.

1.12.14 L'interazione nel personaggio

"Allora adesso partiamo a fare quello che ci servirà per indirizzare il nostro player, cioè farlo ruotare verso il nostro oggetto, e allo stesso modo far sì che l'interazione parta."

Un Do Once. "Vi dico che è un'azione che deve essere fatta su un **Do Once**": l'interazione è una sequenza (girarsi, far partire il Montage, aspettarne la fine) e solo "a fine del montaggio può reiniziare una prossima interazione". Quindi IA_Interact → Do Once.

Chi stiamo guardando. Dal componente **BPC_RayInteract** (trascinato dal pannello dei componenti nel grafo: "prendilo e trascinalo") si prende la sua variabile **Interactable** e la si converte in **Validated Get**: "questo Interactable è presente? Ci sta o non ci sta?". Il personaggio non rifà la trace: legge il risultato del componente.

- **Is Valid**: il messaggio **Interaction** di BPI_InteractionRay ("tira fuori e fai *interaction*, cioè Interaction Message"), con target Interactable: "e questa è proprio la base: così interagiamo".
- **Is Not Valid**: **Reset** del Do Once (aggiunto dopo, quando il docente si accorge che senza il reset, premendo E senza guardare nulla, il Do Once resta chiuso per sempre: "se non è valido, resetta il Do Once comunque, in ogni caso").

Girarsi verso l'oggetto. Dopo il messaggio, **Set Actor Rotation** con target *Self* ("siamo già nel player"). Si splitta il pin *New Rotation*, e si splitta anche l'uscita di un **Get Actor Rotation**: "l'unica asse che ci interessa è la Z", perché il personaggio deve girarsi solo attorno all'asse verticale ("se l'interactable sta qua, deve girare così"). Quindi:

- X (Roll) e Y (Pitch) restano quelle attuali, dal Get Actor Rotation;
- Z (Yaw) viene da un **Find Look at Rotation**, "quello che abbiamo fatto l'altra volta per la torretta", con **Start** = **Get Actor Location** del personaggio (il docente dice prima "rotation", poi si corregge: "location, ragazzi, scusatemi") e **Target** = Get Actor Location dell'Interactable. Il pin di uscita non si lascia splittare ("non me lo splitta"), quindi un **Break Rotator**, e il suo Z va nella Z della nuova rotazione.

La prova provvisoria. Prima del Montage, una prova: la scritta "interagendo" compare, ma il personaggio non si gira; e dopo la prima volta non succede più nulla. "Ah, giustamente, ragazzi": il Do Once non si resetta mai. Il docente collega temporaneamente il Reset alla fine della catena ("lo faccio solamente per farvelo vedere"): "mi giro di qua, e si gira verso di lui". Poi lo stacca, "perché ci servirà più avanti".

Il Montage. Nel Content Browser si cerca *interact*: fra le animazioni del bot c'è **Interact Press Button**. "Vi ricordate come si fa il montaggio?" Tasto destro sull'animazione, **Create**, **Create AnimMontage**: si crea un asset che "è sempre la stessa animazione, ma in questo caso è un montaggio". Nel personaggio, **Play Montage**: nel pin **In Skeletal Mesh Component** la **Mesh** del personaggio ("basta che prendi la Mesh da qui e la metti così"), nel **Montage to Play** il nuovo Montage (*Interact Press Button Montage*).

La variabile Interacting. "Mettiamo, giusto perché in futuro ci possa servire, una variabile" booleana, **Interacting**: "dobbiamo verificare se stiamo interagendo, non stiamo interagendo, eccetera. Sempre buono mettere una variabile che dice: guarda, io sto interagendo". Prima del Play Montage, Set Interacting a **vero**; sull' **On Completed**, "quindi è finita l'animazione", a **falso**.

Fermare il giocatore. Durante l'interazione il giocatore non deve muoversi:

- prima del Montage, **Disable Input**, con il **Get Player Controller** nel pin *Player Controller*: "chi dobbiamo levare? Gli input al controller del nostro player";
- lo **Stop Movement Immediately**, "andiamo a fermare qualsiasi tipologia di movimento". Il target non viene detto: il nodo appartiene al componente di movimento del personaggio, il *Character Movement*;
- sull'On Completed, **Enable Input**, con lo stesso Player Controller;

- e infine il **Reset** del Do Once, "in modo tale che io possa reinteragire di nuovo".

In Play: "io giro di qua, si gira, fa l'animazione, non mi posso muovere"; "e adesso mi riposso muovere appena finito il Montage" (figura 1.12.6). "Ci siete? Vi funziona? Perfetto."

I problemi dei presenti. Uno studente non trova come prendere il componente ("l'hai aggiunto come components? Prendilo e trascinalo"); altri si perdono i due nodi successivi, ripetuti dal docente (Get Interactable, Convert to Validated Get, il messaggio Interaction). Un altro ha il Montage che non parte: manca la Mesh nel pin del Play Montage. Uno studente, a cui prima non funzionava, conferma che ora va, ma ha lo schermo "pieno di *Hello*" (forse i Print String rimasti con il testo di default), "quindi li vedo con la coda dell'occhio".

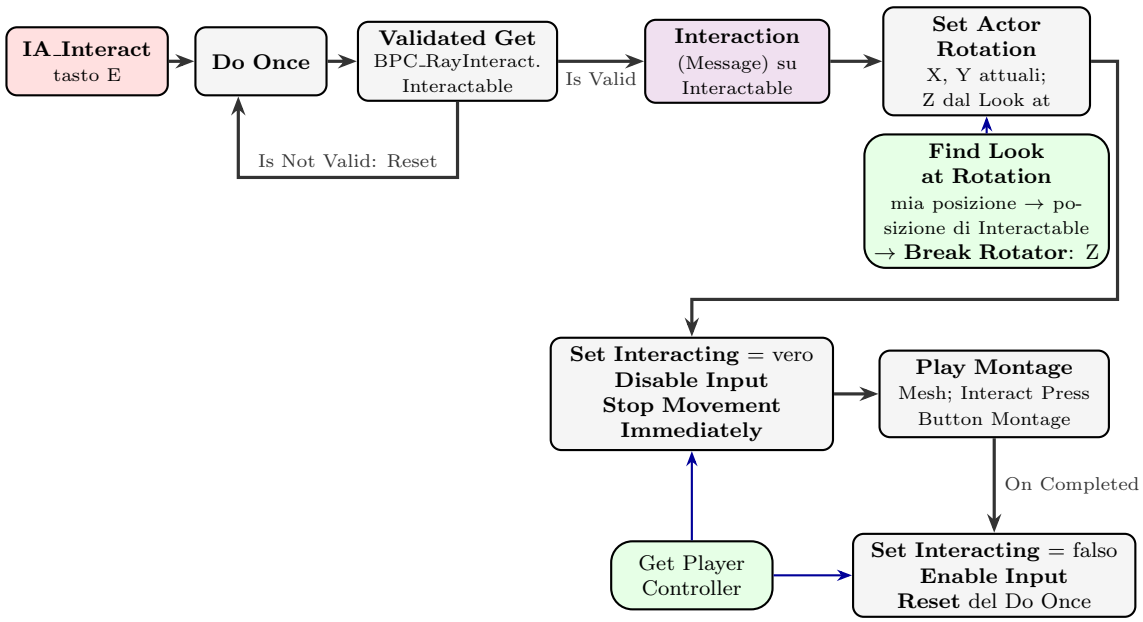


Figure 1.12.6: L'interazione nel personaggio. Il personaggio non rifà la trace: legge la variabile Interactable del suo componente, invia il messaggio, si gira solo sull'asse Z verso l'oggetto, blocca l'input e suona il Montage; a Montage finito restituisce l'input e riapre il Do Once. Ricostruzione dalla lezione; la posizione esatta del Set Interacting e del Disable Input nella catena è dedotta dalle parole del docente.

Approfondimento: Do Once, Disable Input e la rotazione. Romero e Sewell descrivono il **Do Once** nel capitolo 13 (*Data Structures and Flow Control*): "il nodo Do Once esegue il pin di uscita una sola volta. Se viene attivato di nuovo, il pin di uscita non viene eseguito. Il pin di input Reset deve essere attivato per permettere al Do Once di eseguire di nuovo il pin di uscita." Il loro esempio è un'arma caricata, che si ricarica dopo 30 secondi con un timer che resetta il Do Once: lo stesso schema della lezione, con il Montage al posto del timer. Nel capitolo 15 il libro descrive **Enable Input** e **Disable Input**: "definiscono se un actor deve rispondere agli eventi di input", e "hanno bisogno di un riferimento al Player Controller in uso". Due note generali sull'engine. La prima: *Disable Input* chiamato sul personaggio blocca gli eventi d'input di quel Blueprint (movimento, camera, salto), ma non ferma un movimento già in corso, cioè la velocità residua del Character Movement; per questo serve anche *Stop Movement Immediately*, che azzera la velocità sul posto. La seconda: il *Set Actor Rotation* è istantaneo, quindi il personaggio "scatta" verso l'oggetto. Per una rotazione morbida si può usare un *RInterp To* nel Tick o una Timeline, come per le porte; e il template Third Person, che ha l'*Orient Rotation to Movement* attivo, non contrasta la rotazione perché durante l'interazione il personaggio è fermo.

1.12.15 Il widget "premi E"

"Andiamo semplicemente a rendere tutto un po' più carino": un piccolo widget che mostri il tasto da premere sopra l'oggetto.

Gli asset della UI. Il docente aveva messo nel materiale didattico una cartella di immagini per la UI, "la UI asset". Uno studente non l'ha trovata ("pensavo fosse sul Drive"): il docente controlla il Drive condiviso e i canali del Discord, scopre che non era stata caricata e la carica al momento, "tanto pesa pochi kilobyte". "Estraetela e schiaffatela dentro Unreal", nella cartella **UI**. Il trascinamento nel Content Browser a qualcuno dà "fallimento"; il "metodo abbastanza brusco, però va benissimo" è copiare la cartella *estratta* con il gestore dei file direttamente nella cartella *Content/UI* del progetto. "Adesso funziona."

WB_InteractUI. Nella cartella UI, un nuovo Widget Blueprint, classe User Widget: **WB_InteractUI**. Nel Designer:

1. un **Border**, con l'alfa del colore a **0**, ridimensionato a un quadrato di circa **96** × **96** ("94 va bene anche"), e il **Padding** tutto a **zero**;

2. un secondo **Border** *dentro* il primo;

3. le immagini: nel **Brush** di ciascun Border ("vedi che c'è il brush, apri il brush") si sceglie un'immagine del pacchetto, "il panel che volete voi" (ce ne sono diversi, Panel 0, 1, 2, 3, 5...). Dopo una correzione ("questo è il contrario, scusate"), il Border esterno prende **Panel Border 5**, con l'alfa a **1** e la tinta **nera**, e quello interno **Panel 5**;

4. un **Text** con la lettera **E**, con *Horizontal Alignment* e *Vertical Alignment* al **centro**, e la dimensione del **font** a 35, poi 40–45 ("una bella E").

"Piccola cosa, ragazzi, prima non l'ho fatto fare a mano": la dimensione dei Border si può impostare anche con precisione, dall'**Image Size** del Brush (96 × 96).

Il widget che contiene la barra della vita. Una studentessa crea un nuovo widget, ma aprendolo trova le barre della vita e della stamina, e non riesce a modificarle. Rifacendo la creazione insieme, la causa emerge: nella finestra di scelta della classe aveva preso un widget esistente come genitore ("anteprima widget praticamente ti dà un'anteprima dei widget che già ci sono"), cioè il suo HUD; il nuovo widget ne *ereditava* il contenuto, in sola lettura. "È buona prassi fare quello tutto in alto", **User Widget**.

L'immagine che non si vede. Un'altra studentessa ha il bordo esterno che non compare. Il docente controlla tinta, alfa, Image Size, Padding, *Content Opacity*: "è a 1, va bene così". "Perdonami, purtroppo non saprei dire": si cancellano i Border e si rifanno, "basta funzionare". Rifatti, funzionano.

Il widget nell'oggetto. In BP_InteractableTest, il componente **Widget**:

- **Space**: **Screen**, non World, "in modo tale che ci sia una specie di 2D, quindi relativo allo schermo, non al mondo";
- **Draw Size**: **100** × **100**, "un po' più piccolo";
- **Widget Class**: WB_InteractUI.

In Play, il quadratino con la E "scala con lo schermo", ma, se non lo si nasconde, "rimane per sempre".

Mostrare e nascondere. Nel grafo di BP_InteractableTest:

- **Event BeginPlay** → **Set Hidden in Game** sul Widget, con *New Hidden* **attivo**: nascosto da subito;
- **Event Interaction Start** → Set Hidden in Game, **disattivo**: "allora la faccio apparire";
- **Event Interaction End** → Set Hidden in Game, **attivo**.

Il docente prova anche un'idea per l'Event Interaction (nascondere il widget, un **Delay** di un secondo, mostrarlo di nuovo), ma la toglie: "questo potrebbe dare errori, ragazzi", perché il Delay "va in contrasto" con Start ed End (se lo sguardo si sposta durante il secondo d'attesa, il Delay rimostrerebbe un widget che l'End ha appena nascosto). "Lascialo così per adesso."

La spunta al contrario. Una studentessa vede la E anche quando non guarda il cubo: "allora vuol dire che hai sbagliato l'Hidden in Game". "Il tick è positivo sull'End e negativo sullo Start", "e anche positivo sul Begin Play". Dopo la correzione resta un dubbio, poi chiarito: all'inizio della prova la camera stava già inquadrando il cubo da una posizione strana, e il widget era giustamente visibile. "Non lo vedo più. Perfetto, ragazzi."

Approfondimento: il Widget Component. Il **Widget Component** mostra un Widget Blueprint attaccato a un actor. In **World** space il widget è un piano nel mondo 3D: ha prospettiva, può essere coperto da altri oggetti, si rimpicciolisce con la distanza, ed è adatto a schermi diegetici (un terminale, un cartello). In **Screen** space il widget è disegnato sopra l'immagine, sempre rivolto alla camera, con una dimensione costante in pixel, e segue sullo schermo la posizione proiettata del componente: è la scelta tipica per i suggerimenti "premi E" e per le barre della vita sopra i nemici. La **Draw Size** è la risoluzione in pixel a cui il widget viene disegnato. Per nascondere il suggerimento, l'alternativa a *Set Hidden in Game* è *Set Visibility* sul componente; il risultato è lo stesso. Funzionamento generale dell'engine.

1.12.16 Un esempio: la luce che si accende

L'idea. "Sappiate che tutti gli oggetti che create", se implementano l'interfaccia, "avete già un qualcosa che vi identifica se state iniziando un'interazione", se la state finendo, "e la tipologia di interazione". Le funzioni costruite per i bottoni nelle lezioni passate "possiamo traslarle sul nostro interactable": "un diverso modo di interagire e di attivare le cose". Con Davide si vedranno "anche delle piccole cose di modeling": si può modellare un pulsante, dargli BPI_InteractionRay, "e a quel punto magari potete chiamare l'ascensore". "Cosa succede se interagisco? Ci metto quello che voglio: ci metto un timer, ci metto una cosa, accendo una luce."

Velocissimo. "Facciamo velocissimo", in BP_InteractableTest. Dall'Event Interaction si toglie il Print String, e:

1. una variabile del tipo dell'attivabile luce, la classe figlia di BP_Activable delle lezioni sul pulsante ("la Point Light", **BP_ActivableLight**; la trascrizione dice "BP Activables" e "inter light"), con i suoi eventi **Activation** e **Deactivation**;
2. un **Flip Flop**: "questo fa A, poi fa B, poi fa di nuovo A: fondamentalmente gioca su due";
3. su **A**, **Activation** della luce; su **B**, **Deactivation**;
4. la variabile va resa modificabile nell'istanza e collegata alla luce del livello ("però gli dobbiamo dare quale light", "dammi la light, dammi questa").

"Abbiamo acceso la luce e la rispegniamo." "Questo è un esempio, ragazzi": "qualsiasi actor può essere separato dagli altri con la sua specifica interazione", e "quello che la identifica già ce l'abbiamo".

Approfondimento: Flip Flop nel libro. Romero e Sewell (capitolo 13): "ogni volta che un nodo Flip Flop viene eseguito, alterna fra due pin di uscita, A e B"; ha anche un output booleano *Is A*, vero quando sta eseguendo A. Con una nota importante: "il Flip Flop non funziona correttamente dentro una funzione, perché ha una variabile interna che viene cancellata quando la funzione finisce", e quindi dentro una funzione esegue sempre A. Lo stesso vale per il Do Once. È il motivo per cui, nella lezione, il Flip Flop sta nell'Event Graph dell'oggetto e il Do Once nell'Event Graph del personaggio, mentre la funzione LookForInteractions, che non deve ricordare nulla fra un frame e l'altro (lo stato è nella variabile Interactable, che è del componente), può essere una funzione.

1.12.17 La chiusura del corso

"Ci sono domande, ragazzi? Domande su tutto il corso, in generale." Il docente chiede anche un **feedback** sul corso, "adesso o dopo, tranquillamente sul server". La risposta di uno studente, scherzosa: "tutto chiaro, tutto bellissimo, incredibile". "Sono contento."

Per il futuro, anche "per chi guarda in differita": "se avete un problema, state facendo il Big One, eccetera, buttate il vostro problema sul server: sia alunni sia professori provvederanno a rispondervi e darvi una mano". "Se ci sono ultime cose che volete dire, ragazzi, ditele ora, o effettivamente questa volta tacete per sempre." E il congedo: "vi ringrazio per la vostra attenzione, vi ringrazio per aver partecipato al corso. Spero che i vostri progettini possano prendere vita per tutte le cose che poi andrete a fare in Unreal. Per benino, ricordatevi di **validare sempre** molte variabili." "Da questo punto in poi" le lezioni continuano con Davide. "È stato un piacere."

Punti chiave del capitolo

- Il designer usa Unreal per **prototipare** le meccaniche e verificare che funzionino; il progetto d'esame degli anni successivi è un **pezzo da portfolio**. **Less is more**: il giocatore impiega molto più tempo di chi ha progettato il gioco per capirne ogni elemento.

- Il menu vive in una **mappa a parte** (Empty Level), con un **Widget Blueprint** (User Widget) creato e mostrato nel **Level Blueprint** (Create Widget, Add to Viewport con il Return Value).

- Nel Designer: Canvas Panel, bottoni con l'**ancora** al centro e l'**Alignment** per impilarli, un **Text** dentro ogni bottone, i colori di **Normal**, **Hovered** e **Pressed** nello Style (copiabili con il codice Hex), e l'ordine della **Hierarchy** per mettere un'immagine dietro (l'ultimo elemento sta sopra).

- Niente personaggio nel menu: **GameMode Override** con la Game Mode base (Default Pawn) e Player Start fuori campo. La vista fissa: un Camera Actor e **Set View Target with Blend**, dal **Player Controller**.

- Nell'**Event Construct** del widget: **Set Show Mouse Cursor** (con la spunta) e **Set Input Mode UI Only**. Sui bottoni, **On Clicked**: **Quit Game**; **Set Input Mode Game Only** e **Open Level (by Name)**.

- Una **Blueprint Interface** è un intermediario ("un volo di linea"): solo nomi di funzioni, in sola lettura. Chi la implementa (Class Settings, Implemented Interfaces) decide che cosa fare; chi non la implementa ignora i messaggi.

- Un **Blueprint Component** isola una funzionalità da aggiungere a qualunque actor. L'**Actor Component** non ha posizione; lo **Scene Component** ha un Transform. Il componente trova il suo actor con **Get Owner** e la camera con **Get Component by Class**.

- **BPC_RayInteract**: a ogni frame (**Event Tick**) una **Sphere Trace** (Radius 5) dalla posizione del componente, lungo il Forward Vector della camera (o del componente, se la camera non c'è) per 300 unità, sul canale **InteractionTrace** (Ignore di default, Block sugli interagibili).

Asset o variabile	Dove	Tipo, valore	A che serve
M_Menu	mappe	Empty Level	la mappa del menu
WB_Menu	UI	Widget Blueprint	il menu con i due bottoni
Button_StartGame, Button_QuitGame	WB_Menu	Button, 300 × 80	gioca; esci
CameraActor	M_Menu	Camera Actor	la vista fissa del menu
GameMode Override	World Settings di M_Menu	Game Mode base	niente personaggio nel menu
ThirdPersonMap	mappe	livello	la mappa aperta da Play Game
BPC_RayInteract	Blueprints, Components	Scene Component	traccia gli interagibili
CameraComponent	BPC_RayInteract	Camera Component (ref.)	la camera del proprietario
DistanceTrace	BPC_RayInteract	Float, 300	la lunghezza del raggio
Interactable	BPC_RayInteract	Actor (ref.)	l'interagibile guardato ora, o vuoto
Scene	BPC_RayInteract	Scene Component (ref.)	solo per il trucco del Select
InteractionTrace	Project Settings	Trace Channel, Ignore	il canale delle interazioni
BPI_InteractionRay	Blueprints, Components	Blueprint Interface	Start, End, Interaction
BP_InteractableTest	Blueprints	Actor (Cube, Widget)	l'oggetto di prova
IA_Interact	Input	Input Action, Pressed e Released	l'interazione, tasto E
Interacting	BP_Bot2	Boolean	vero durante l'interazione
Interact Press Button Montage	animazioni del bot	Anim Montage	l'animazione dell'interazione
WB_InteractUI	UI	Widget Blueprint, 96 × 96	il suggerimento "E"
Widget (componente)	BP_InteractableTest	Screen, 100 × 100	mostra WB_InteractUI
Light	BP_InteractableTest	BP_ActivableLight (ref.)	la luce accesa e spenta

Table 1.12.1: Gli asset e le variabili della giornata, con i nomi usati dal docente (BPC_RayInteract è, nel progetto del docente, BPC_RayInteract1; il nome della variabile della luce è incerto).

Sintomo	**Causa e rimedio**
La barra della stamina scende solo al rilascio	manca la Call E_UpdateStamina in StaminaDeduction: c'era solo in RegainStamina
Il secondo bottone incollato non compare	è sovrapposto al primo (controllare la Hierarchy); oppure aggiungerne uno nuovo dalla Palette
L'immagine copre i bottoni	è in fondo alla Hierarchy, cioè disegnata sopra: trascinarla prima dei bottoni
Il personaggio compare dietro il menu	GameMode Override nelle World Settings della mappa del menu: Game Mode base
Si vede la pallina (Default Pawn)	spostare il Player Start fuori dalla camera
La camera del menu non si usa	Set View Target with Blend dal Get Player Controller (non dal Camera Manager), con la referenza alla camera
Errore su Add to Viewport	il Target è Self: collegare il Return Value del Create Widget
Il cursore non si vede	Set Show Mouse Cursor nell'Event Construct, con la spunta attiva
Nel menu ci si muove ancora	Set Input Mode UI Only nell'Event Construct
Il Select non accetta Self (o la camera)	il Select prende il tipo del primo collegamento: mettere prima una variabile Scene Component, poi cancellarla
In Play non si vede il raggio	LookForInteractions non chiamata nell'Event Tick; Draw Debug Type a None
InteractionTrace non compare nella trace	compilare dopo aver creato il canale
Il raggio ignora il cubo	la mesh non blocca InteractionTrace (default Ignore): Block
L'evento dell'interfaccia non si trascina	non è Blueprint Callable: implementarlo come evento (doppio clic o Implement Event)
Hit Actor non si collega a Interactable	Interactable di tipo sbagliato: Actor, Object Reference
Si interagisce una sola volta	il Do Once non viene resettato (On Completed e Is Not Valid)
Il Montage non parte	manca la Mesh nell'In Skeletal Mesh Component
Il nuovo widget contiene le barre dell'HUD	creato con l'HUD come genitore: scegliere User Widget
Il bordo del widget non si vede	causa non trovata; rifare i Border
La E si vede quando non si guarda	spunte di Set Hidden in Game invertite: attivo nel Begin Play e nell'End, disattivo nello Start
Trascinare la cartella UI dà "fallimento"	copiare la cartella estratta in Content/UI dal gestore dei file
La BPI si apre senza grafo	doppio clic sulla funzione in My Blueprint

Table 1.12.2: I problemi incontrati durante la lezione, con la causa e il rimedio.

Nodo, asset o impostazione	Che cosa fa
New Level: Empty / Basic	livello vuoto; con luce e cielo già pronti
Widget Blueprint (User Widget)	l'asset dell'interfaccia: Designer e Graph
Button: Style (Normal, Hovered, Pressed)	i colori del bottone nei tre stati
Hex sRGB / Hex Linear	il codice del colore, da copiare
Text dentro un Button	l'etichetta del bottone, figlia del bottone
Image: Color and Opacity (alfa)	la trasparenza dell'immagine
Hierarchy (ordine)	l'ultimo elemento è disegnato sopra
Anchors / Position / Alignment / Size	ancora; scostamento; pivot; dimensioni
Open Level Blueprint	il Blueprint del livello corrente
Create Widget / Add to Viewport	crea il widget; lo mostra
World Settings: GameMode Override	la Game Mode per questa mappa
Default Pawn / Player Start	la sfera della Game Mode base; dove nasce il Pawn
Camera Actor / Directional Light	una camera nel livello; il sole
Create a Reference to (actor)	la referenza a un actor del livello nel Level Blueprint
Set View Target with Blend	cambia la camera del Player Controller
Event Pre Construct / Construct	alla costruzione nel Designer; alla costruzione in gioco
Set Show Mouse Cursor	mostra o nasconde il cursore
Set Input Mode UI Only / Game Only	l'input va solo all'interfaccia; solo al gioco
On Clicked (Button)	l'evento del clic su un bottone
Quit Game	esce dal gioco (in editor, chiude il Play)
Open Level (by Name)	apre una mappa dal nome
Blueprint Interface	solo dichiarazioni di funzioni, in sola lettura
Class Settings: Implemented Interfaces	aggiunge un'interfaccia a un Blueprint
Interaction Start / End / Interaction (Message)	i messaggi dell'interfaccia
Event Interaction Start / End / Interaction	la loro implementazione nell'oggetto
Actor Component / Scene Component	componente senza / con Transform
Get Owner	l'actor che possiede il componente
Get Component by Class	il primo componente di una classe (non l'array)
Sphere Trace By Channel	una sfera lungo un segmento: Radius, Draw Debug Type
Get World Location / Get Forward Vector	posizione nel mondo; direzione davanti
Select (wildcard)	sceglie fra due valori; il tipo è quello del primo collegato
Is Valid (pura) / Validated Get	il riferimento esiste?; Get con le due uscite
Not Equal (!=)	vero se i due valori sono diversi
New Trace Channel (Ignore / Block)	un canale di collisione personalizzato
Break Hit Result: Hit Actor	l'actor colpito
Does Object Implement Interface	l'oggetto implementa l'interfaccia?
Event Tick	a ogni frame
Print String: Text Color	il colore del messaggio
Input Action (Pressed, Released) / IMC	l'azione e il tasto (E)
Do Once (Reset)	esegue una volta, finché non si resetta
Set Actor Rotation / Get Actor Rotation	imposta; legge la rotazione dell'actor
Find Look at Rotation / Break Rotator	rotazione per guardare un punto; i suoi tre assi
Create AnimMontage / Play Montage	un Montage da un'animazione; lo suona (On Completed)
Disable Input / Enable Input	toglie; restituisce l'input all'actor
Stop Movement Immediately	azzera il movimento del personaggio
Border: Brush, Padding, Image Size	il riquadro con immagine; margine interno; dimensione
Widget Component: Space, Draw Size, Widget Class	un widget attaccato a un actor
Set Hidden in Game (New Hidden)	nasconde o mostra un componente
Delay	aspetta prima di proseguire
Flip Flop	alterna fra le uscite A e B

Table 1.12.3: I nodi, gli asset e le impostazioni della dodicesima lezione.

- Il **Select** prende il tipo del primo collegamento: per scegliere fra la camera e Self, prima una variabile *Scene Component*, poi la si cancella (la "mandragata").
- La variabile **Interactable** ricorda l'oggetto guardato: se cambia, **Interaction End** sul vecchio, Set, **Interaction Start** sul nuovo; se non si guarda niente di interagibile, End e svuota. Sempre con il **Validated Get**.
- Nel personaggio: **IA_Interact** (tasto E), **Do Once**, il messaggio **Interaction**, **Set Actor Rotation** con il solo **Yaw** da **Find Look at Rotation**, **Disable Input** e **Stop Movement Immediately**, **Play Montage**, poi Enable Input e **Reset** del Do Once (anche quando non c'è nulla da interagire).
- Il suggerimento "E": un **Widget Component** in **Screen** space, nascosto nel Begin Play e mostrato e nascosto da Interaction Start ed End con **Set Hidden in Game**.
- Ogni oggetto può dare la sua risposta all'interazione: per esempio un **Flip Flop** che chiama Activation e Deactivation di una **BP_ActivableLight**. Flip Flop e Do Once non vanno dentro le funzioni.
- La raccomandazione finale del corso: **validare sempre** le variabili.

Chapter 1.13

Introduzione al level design: la gym, il block out e le luci

Questa è la **tredicesima lezione** del corso breve di **introduzione a Unreal Engine**, e la prima con un **nuovo docente**. Le dodici lezioni precedenti, con **Igor**, hanno costruito un piccolo repertorio di meccaniche (il personaggio **bot** che cammina, corre e salta, la stamina, i collezionabili, la torretta che spara, le porte che si aprono, le pedane che salgono e scendono, la luce che si accende, i materiali fisici dell'acqua e della roccia). Le ultime **quattro lezioni** cambiano prospettiva: non più "come si programma una meccanica", ma **come si costruisce un livello** che quelle meccaniche le metta alla prova. "L'idea delle mie lezioni, di queste mie ultime quattro lezioni, sarà quella di spiegare un po' il *workflow* che c'è alla base della creazione di un livello e iniziare a mettere su dei livellini."

Il docente non dice mai il proprio nome nella trascrizione. Le lezioni precedenti annunciavano che il corso sarebbe proseguito "con Davide" e con il *level design*, e qui il docente parla di Igor come di un collega ("io e Igor proviamo le cose che avete fatto"): il docente è quindi, con ogni probabilità, **Davide**, ma è una deduzione del capitolo, non qualcosa che la lezione dica. Nel resto del capitolo è semplicemente "il docente".

La giornata ha tre parti. La prima è un "filo di teoria", "niente di troppo pesante": da dove si prende l'ispirazione per un livello, il **level design document**, il **layout** (la piantina), la **gym** (il livello di prova in cui si misurano le capacità del personaggio) e il **block out** (il livello rifatto con sole geometrie). La seconda è pratica, in Unreal: si abbozza una gym con il bot, si misurano la corsa e i salti, si toccano i parametri del **Character Movement** e si conoscono le **geometry brush**, lo strumento classico per fare un block out. La terza, dopo la pausa, è una panoramica sulle **luci**: i tipi di luce di Unreal, la loro *mobility*, i parametri principali, il costo delle ombre, il *build* delle luci e le *reflection capture*. In chiusura, il compito per la lezione successiva: una gym propria e un piccolo block out con da tre a cinque collezionabili.

Tre avvertenze di lettura, come per le lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Unreal è "Real", "un reale" e "Arial"; la **gym** è "GIM" e "JIM"; il **block out** è una volta "plot out" e il *blockouting* "lockout"; **Kratos** è "Gratos" e "Grados"; la **Max Sprint Distance** è "Mazda Sprint" e "Max Print Distance"; il Blueprint del bot, **BP_Bot2** ("character bot 2"), è una volta "bottegger"; il **Walkable Floor Angle** è "workable floor angle" e "Walk and roll Flow Rainbow"; la **static mesh** è "state mesh" e "state Mesh"; il **Foliage Mode** è "Folliage Mode"; il **Light Color** è "let color" e "LED color"; il codice **Hex Linear** è "l'ex liner" e "gli ex"; **Affects World** è "FXWord"; l'**ambient occlusion** è "ambiente occlusion"; il **Source Radius** è "la sorsa" e il **Soft Source Radius** "SRADUS soft SRADUS"; la **Source Length** è "la lent"; le **Barn Door** sono "bar and door"; la **Planar Reflection** è "Lanar Reflection Capture"; *baked* ("cotto", precalcolato) è reso con "becata", "abbicata", "baicata" e "vacato"; *Breaking Bad* è "break in bed". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: in alcuni punti il riconoscimento vocale produce testo estraneo alla lezione. In apertura ci sono frasi su "la situazione interna a livello politico" e su "gli egiziani", poco dopo una frase ripetuta ("io personalmente sono a favore di questa cosa"), a metà della parte sulle scale una serie di frasi senza senso ("non preoccupatevi se non mi ricordo bene le mie relazioni") e, durante la pausa, un lungo blocco ripetuto decine di volte ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?"): non appartengono alla lezione e sono stati ignorati. La terza: le slide e

il video mostrati dal docente non sono nella trascrizione; il capitolo ricostruisce le immagini da ciò che il docente dice mentre le indica, completa i nomi esatti dei parametri di Unreal Engine 5, distingue le prove dalle conclusioni e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono la mappa di un metroidvania commentata dal docente, il percorso a quadretti di un platform, la gym costruita in diretta, il "feeling" dei due salti provati, le brush additive e sottrattive, le tre *mobility* delle luci e le forme delle luci locali. Le tabelle riassumono le misure della giornata, i parametri del Character Movement, i parametri delle brush e delle luci, i problemi incontrati e le impostazioni usate. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro di Marcos Romero e Brenden Sewell, *Blueprints Visual Scripting for Unreal Engine 5*, il testo di riferimento per questa parte del libro, al libro di Christopher W. Totten, *An Architectural Approach to Level Design*, che tratta proprio i temi teorici della giornata, e a qualche nozione tecnica generale su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'**apertura** e il programma delle ultime quattro lezioni (sezione 1.13.1);
2. **da dove si parte**: la realtà o un altro gioco (sezione 1.13.2);
3. il **level design document** e il **layout** (sezione 1.13.3);
4. la **gym**, il livello di prova (sezione 1.13.4);
5. il **block out** e il caso di God of War (sezione 1.13.5);
6. in engine: la gym, la griglia e la **distanza di sprint** (sezione 1.13.6);
7. i **salti** e il *feeling* del personaggio (sezione 1.13.7);
8. i parametri del **Character Movement** (sezione 1.13.8);
9. le **geometry brush** (sezioni 1.13.9 e 1.13.10);
10. la **pausa** e la scelta del formato delle lezioni (sezione 1.13.11);
11. le **luci**: tipi e *mobility* (sezione 1.13.12), directional e sky light (sezione 1.13.13), point e spot light (sezione 1.13.14), rect light (sezione 1.13.15), il *build* e i riflessi (sezione 1.13.16);
12. i **compiti** per martedì e la chiusura (sezione 1.13.17).

1.13.1 L'apertura e il programma delle ultime quattro lezioni

L'audio e il caldo. I primi minuti sono di prova tecnica. Il docente ha "cambiato la postazione del PC" e chiede più volte se lo si sente bene, se è "basso", se si sentono "ventole in sottofondo": "se capita che mentre parlo mi si taglia la voce, fatemelo sapere". Il motivo: "lo scorso anno, in questo periodo", facendo lezione per lo stesso corso, "il computer fondeva lavorando su Unreal e streamando contemporaneamente", e ha dovuto comprare "una di quelle pedanine" con le ventole da mettere sotto il portatile, "così almeno non cade la connessione o non crasha Unreal". Il caldo è un tema anche per gli studenti: uno di loro (Edoardo, a giudicare dal seguito) dice che "il caldo non me lo permette particolarmente" e che sta cercando "una soluzione per far funzionare il computer".

Chi c'è. Il docente chiede quanti fossero, di solito, alle lezioni con Igor: "più o meno sette". Si aspetta qualche minuto, fino alle 16:10, "se arrivano altre due o tre persone". Nel frattempo chiede se qualcuno abbia "cominciato a smanettare" per conto proprio con Unreal, "al di là di quello che stava spiegando Igor": Edoardo no, oltre alle lezioni; dagli altri nessuna risposta.

Il programma. "Magari riusciamo a fare qualcosina in più, magari facciamo qualcosina in meno", ma il piano delle quattro lezioni è questo:

- "un filo di teoria", "un'idea alla base", "in modo che quando andremo a creare dei livelli avremo almeno le idee un filo più chiare su cosa stiamo cercando di fare e come dovremmo costruire le cose";
- i **block out** e quindi le **geometry brush** (l'argomento di oggi);
- "se ci riesco", più avanti, il **modeling mode** per le static mesh, "questo magari verrà rimandato";
- alcune delle modalità di Unreal, come il **Landscape Mode** e il **Foliage Mode**, e come possono tornare utili "a seconda che stiamo creando dei livelli in interna o in esterna";
- "sicuramente la gestione delle luci, perlomeno una base di *lighting*", e con il lighting "una base di post produzione";
- "un minimo di **dressing**": si passerà dalle "prime mappine fatte unicamente in geometrie", quelle di oggi, a livelli "esteticamente un pelo più curati".

Il docente chiede se qualcuno sia iscritto al corso di design o di grafica 3D: la parte di oggi copre "principalmente quello che fa un *designer* all'interno della creazione di un livello"; la parte estetica "spesso e volentieri viene rimandata ai grafici 3D, però è importante comunque sapere qualcosina".

L'esercitazione finale. "Cosa più importante": il docente vorrebbe chiudere con una vera **esercitazione** ("non so se farlo nell'ultima lezione o se darvelo per il post"), in cui gli studenti creano qualcosa mentre lui dà dei feedback, e da cui escono degli **eseguibili** da caricare sul Discord, "in modo che a vicenda voi possiate provare i livellini degli altri", e che "io e Igor" proveranno lasciando dei feedback. "Nulla di troppo impegnativo": serve a "iniziare a mettere mano un po' alla creazione di livelli". I livelli si costruiranno con le meccaniche programmate nelle lezioni precedenti. (La trascrizione dice "con i tool che io vi ho fatto, diciamo, fatto fare per foto programmare le scorse lezioni": il senso è, con ogni probabilità, *i tool che Igor vi ha fatto programmare*.)

1.13.2 Da dove si parte: la realtà o un altro gioco?

"Sono le quattro e dieci, sono passate in realtà", e si comincia, con le slide. "Da dove partiamo quando vogliamo andare a creare un livello?" Il grande dubbio è se partire "dall'osservazione della realtà" o da "una mappa, da un videogioco che è stato già creato da qualcun altro". L'esempio: si vuole fare una mappa *open* di **Manhattan**, "come stiamo vedendo qui con *Spider-Man*". A che cosa ci si ispira per la struttura della mappa?

Le risposte degli studenti. Il primo studente risponde: alla realtà, "perché mi sembra più fedele", "è una base che abbiamo sempre, ce l'abbiamo lì pronta", "è una riproduzione più originale". Il docente approva: la realtà è originale, non c'è bisogno di "andare a cercare altri titoli che hanno fatto qualcosa ispirato a quello a cui noi ci siamo ispirati"; esistono anzi "tutta una serie di tool e di siti" che permettono di prendere le **height map** di una città specifica e di ricrearla dentro un engine. Poi chiede se qualcuno voglia difendere l'altra opzione. Un secondo studente: trasformare la realtà in poligoni "richiede tempo", mentre prendendo la mappa di un altro videogioco "si può risparmiare un po' di tempo". "Ah, beh, sì, se possiamo rubare gli asset da un altro videogioco cambia tanto", scherza il docente, ma "è una cosa che lascia un po' il tempo che trova": gli asset che servivano a quel gioco "non sono proprio gli asset che servono a noi".

La risposta: la realtà, perché non è filtrata. "La risposta chiara e diretta è quella per cui siete andati all'inizio": ci si ispira alla **realtà**, perché è **originale** e **non filtrata**. Non filtrata vuol dire che *Spider-Man*, per costruire la sua Manhattan, ha dovuto "estrapolare dalla realtà cose che si incastravano particolarmente bene con il *game design* previsto dal gioco". La componente principale di quel gioco è il *traversal* con le ragnatele, il muoversi fra i palazzi ad alta velocità; per una mappa soddisfacente da quel punto di vista

servono "tanti palazzi alti", e quindi si rimuove "una grossa parte di Manhattan che non ci interessa" perché non si adatta al gameplay. Se nel nostro gioco non si oscilla fra i palazzi, "la mappa che *Spider-Man* seleziona è scarsamente rilevante": conviene tornare all'originale. Magari nel nostro gioco "facciamo delle rapine" e serve "una parte della mappa molto più piccola", in cui entrare negli edifici.

"Ci ispiriamo alla realtà e non alla *fiction*, perché la fiction non è necessariamente quello di cui abbiamo bisogno. Il game design di un altro gioco sarà diverso dal nostro." E da qui il principio che regge tutta la giornata: il **level design** "dovrà essere largamente ispirato da quello che è il nostro game design", dall'idea di gameplay, "dal modo in cui vogliamo, per esempio, far muovere il personaggio".

Assassin's Creed e l'acqua. Secondo esempio, *Assassin's Creed* ambientato a Roma. Il docente esita sul capitolo ("se non ricordo male *Assassin's Creed 2*, no, forse è *Brotherhood*. È *Brotherhood*"): è effettivamente *Brotherhood* il capitolo ambientato a Roma. Il ricordo del docente è che gli sviluppatori avessero voluto inserire un'isola sul fiume (la trascrizione dice "l'isola di Berlina": con ogni probabilità l'**Isola Tiberina**) per sfruttare un'innovazione, il percorso nell'acqua, così da raggiungere "i punti che sono molto coperti dalle guardie". Il docente stesso si corregge a metà ("ah, no, c'era l'acqua nel 2, vabbè, non ricordo"): i dettagli storici sono incerti, ma il punto è chiaro. "Diamo la possibilità al giocatore di sfruttare le nuove meccaniche di gameplay per avere situazioni diverse, situazioni nuove."

E le nostre meccaniche? Il docente ha visto le cose fatte con Igor: "i collezionabili, la torretta che spara, le porte che si aprono, le pedane che si alzano e si abbassano". "In che modo il level design può andare a far leva su quelle componenti lì?" I collezionabili possono essere gli **obiettivi**; attorno a loro ci sono **ostacoli** da capire come superare: "passare davanti a una torretta senza farci ammazzare", "capire come aprire la giusta sequenza di porte per arrivare a prendere il collezionabile", "fare il giusto parkour fra le pedane in movimento". È, in piccolo, il compito che chiude la lezione.

Approfondimento: progettare sulle capacità del personaggio. Totten (*An Architectural Approach to Level Design*, capitolo 2) dice la stessa cosa con l'esempio di *Metal Gear Solid*: "progettare sul gameplay non significa solo misure", ma anche progettare sulle abilità specifiche del personaggio, come i modi di movimento; i livelli di un gioco *stealth* sono costruiti attorno alla possibilità di nascondersi dietro i muri e sbirciare dietro gli angoli. È la versione positiva del discorso di Spider-Man: non si copia la mappa di un altro gioco perché quella mappa era costruita sulle abilità di un altro personaggio.

1.13.3 Il level design document e il layout

"Una volta che abbiamo capito effettivamente cosa vogliamo dal nostro gioco", si scrive un **level design document**. "Non parlerò troppo dei documenti perché non è questa la sede"; la cosa importante è che siano **leggibili da tutti**, **chiari**, e che dicano **come è strutturato il livello**. Il documento ha due vincoli opposti:

- **non troppo generico**: il docente immagina di scrivere la mappa che ha in testa e di passarla a Edoardo, "l'incaricato di fare il block out": Edoardo "deve essere in grado di capire precisamente tutto quello che io voglio da questa mappa dal documento che ho scritto";
- **non troppo costrittivo** (la trascrizione dice "costruttivo"): non deve "limitare troppo Edoardo, che comunque ci deve mettere un po' del suo".

Il layout: la piantina. Una parte fondamentale della creazione di un livello, e del documento, è il **layout**: "fare una mappa, una piantina". Perché? "Perché ci permette di sbagliare senza grossi problemi." Improvvisare direttamente nell'engine va bene per una mappa molto semplice; per una mappa complessa "ci troveremmo spesso a dire: ah no, questa cosa non va bene", e a rifare. "In questa sede non avremo il tempo né l'obiettivo" di fare layout accurati, e si partirà direttamente dai block out; ma è importante sapere che **prima del block out c'è la piantina**.

La piantina numerata. La prima slide mostra ("qui in basso a sinistra") una pianta con dei numeri e una **legenda**. Il docente la legge come esempio:

1. il punto in cui il giocatore *spawna*;
2. il punto in cui trova la prima arma;
3. il punto in cui trova dei nemici;
4. il punto in cui "si passa da piano a piano", cioè da livello a livello.

Ne esce un percorso: "io parto da qui", attraverso una zona in cui "magari non c'è niente, è una roba più *straightforward* che mi permette di familiarizzare con l'ambiente"; poi "scendo giù" nella zona dei nemici, "ma non ho armi": devo passare "senza farmi notare", prendere l'arma, tornare indietro ed eliminare i nemici.

Il platform a quadretti. Lo stesso vale per i giochi in 2D, dove le misure contano ancora di più. Se il personaggio salta "tre quadratini in altezza e tre quadratini in orizzontale", la mappa va disegnata a quadretti "per sapere in anticipo come potrò far saltare il mio personaggio". L'esempio della slide, alla *Super Mario*: all'inizio si fanno prendere "i primi blocchetti", i primi potenziamenti, e si fa saltare il primo ostacolo, "un salto molto semplice"; poi una scaletta, per far capire che si può salire; poi un salto più complesso, su una piattaforma più piccola da cui scendere; e così via, finché si arriva a salti "al massimo dell'altezza e della lunghezza", che devono essere "molto precisi", e ad altri tipi di ostacoli (figura 1.13.1). "Serve per avere le idee chiare" e "poter calcolare bene" il percorso.

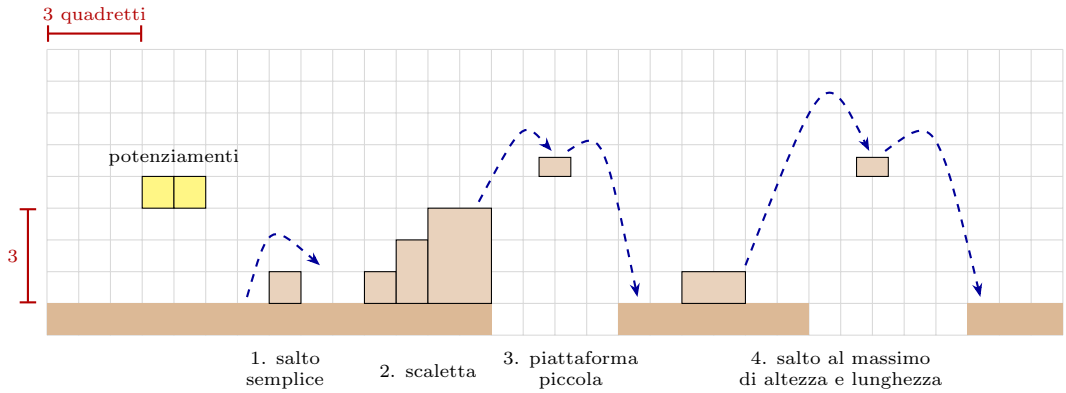


Figure 1.13.1: Il layout di un platform 2D disegnato a quadretti, come nell'esempio della slide: conoscendo il salto del personaggio (qui tre quadretti in altezza e tre in lunghezza, i valori citati a voce dal docente), si dispongono ostacoli via via più difficili, fino ai salti al limite. Ricostruzione schematica: la slide non è nella trascrizione e le posizioni sono indicative.

La mappa di un metroidvania. L'ultima slide del layout è "una roba del genere": da immaginare "come la mappa di un metroidvania". Il docente la percorre ad alta voce, ed è un piccolo trattato di progettazione dei percorsi (figura 1.13.2):

- si parte in alto e si scende fino a una **stanza rossa**, "un'area esplorabile solo con dei power up";
- si continua a scendere: certe direzioni sono chiuse, e c'è uno **shortcut** "che però va aperto nell'altra direzione", quindi si può solo andare giù;
- in fondo si prende il **power up**, "magari un doppio salto, o un salto a parete", con cui si può risalire: "adesso mi si aprono diverse strade";

- da lì si può esplorare, raccogliere un oggetto ("ci sarà sicuramente un oggetto"), salire nell'**area segreta**, o andare al **checkpoint**, la *safe room*;

- più giù ci sono dei **nemici**: battuti quelli, si apre lo shortcut "in modo che poi posso avanzare, se muoio, senza dover ribattere i nemici";

- si prende una **chiave**, che serve per arrivare al **boss**; battuto il boss, si apre lo shortcut e si scende di piano. "Se dovessi morire al piano di sotto, dal checkpoint potrei semplicemente scendere giù, superare lo shortcut che adesso ho già aperto e scendere dalle scale."

"Fondamentalmente, ci facciamo delle mappe per cercare di prevedere e di strutturare quale sarà l'esperienza del giocatore", "cercare di capire come posso costringere il giocatore sui binari che a me interessano".

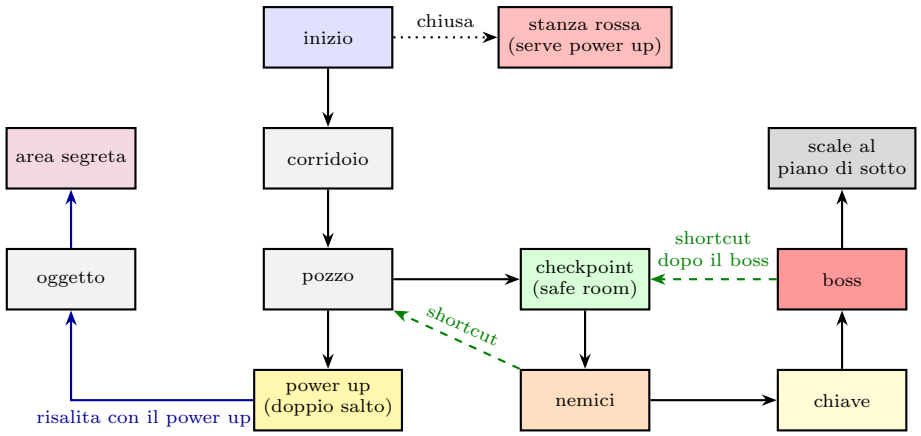


Figure 1.13.2: Uno schema del percorso del metroidvania commentato dal docente: la discesa obbligata fino al power up, le strade che si aprono dopo, gli shortcut che si aprono solo dall'altra parte e il percorso chiave–boss–piano di sotto. È una ricostruzione dei rapporti fra le stanze descritti a voce, non della slide: la disposizione reale delle stanze non è ricostruibile dalla trascrizione.

Approfondimento: la carta a quadretti e le metriche. Totten (capitolo 2) raccomanda proprio i disegni "a quadretti, come quelli degli architetti": stabilendo che un quadretto, o un gruppo di quadretti, vale una certa unità di misura, il designer può pianificare il livello sulle **metriche** del personaggio prima di toccare il computer. "Un errore comune dei designer alle prime armi", scrive, è costruire nel computer livelli che non si adattano al personaggio, "in casi estremi" fino a porte da cui il personaggio non passa. Le metriche sono definite come "misure basate sul gameplay, espresse in unità dell'engine, che descrivono le dimensioni e le proprietà di movimento dei personaggi": la grandezza del personaggio, lo spazio coperto dai salti e dagli attacchi, la distanza percorsa nel tempo camminando o correndo. È il ponte fra la piantina e la gym della sezione seguente.

1.13.4 La gym: il livello di prova

Spostandosi verso l'engine, "sarà quindi molto importante creare una **gym** o un **test level**". "Credo che Igor vi abbia già accennato il concetto di gym, penso, ma comunque nel dubbio ne riparliamo." La gym è "un livello nel quale noi possiamo testare le **capacità del nostro character**". Il bot fatto con Igor "cammina, corre e salta": bisogna sapere "quanto velocemente cammina, quanto velocemente corre, quanto in alto salta", e "se corro e salto contemporaneamente, quanto posso arrivare distante". Si prendono "tutta una serie di misure".

La slide della gym. L'immagine mostrata "riassume un po' il contesto di quello che potrebbe essere una gym, ma poi alla fine la fate sulla base di quello che serve a voi per il vostro livello". Gli elementi che il docente legge:

- **blocchi a parete** di altezze diverse per il salto: "la mia massima altezza di salto è 1,60 m, 1,80 m, 2 m, 2,10": si prova, e se il personaggio salta 1,80 m si decide se va bene "per il tipo di gameplay che io voglio"; altrimenti si cambiano i parametri del character "per far sì che salti l'altezza che a me interessa";
- **porte** di varie misure: 4 m × 4, 2 m × 4, 2 m × 3, 2 m × 2,5, 2 m × 2: "qual è l'esperienza nel passare sotto queste porte? Lo provo, lo testo";
- la **max comfortable wall run distance**: chi ha fatto quella gym aveva nel gioco la corsa sul muro, e ha misurato "il massimo di corsa sul muro che io posso fare prima di arrivare a rischio": 12 metri sono comodi, "perché magari la discesa comincia ai 13, ai 14 metri";
- l'**angolo camminabile**: "su quanto può arrampicarsi il mio personaggio: 45 gradi, 30 gradi, 60 gradi";
- l'**altezza del player**, in piedi e accovacciato, e di conseguenza "quanto devono essere grandi le porte".

Le porte e la camera in terza persona. Un dettaglio che il docente sottolinea: "di solito nei giochi in terza persona si evitano le porte alte due metri", perché i character di default sono alti 1,80 m e "la telecamera spesso e volentieri clippa con le porte così basse". Si fanno porte "falsamente alte": "non so se ci avete mai fatto caso, giocando a un *Uncharted*, *The Last of Us*, qualsiasi gioco in terza persona con telecamera alla spalla: la porta di una casa è molto più grande di quanto dovrebbe essere". Il docente è alto "un metro e 94" e "abbastanza largo": nella realtà certe porte le tocca con la testa saltando, o ci passa "centrando bene con le spalle". Replicarle nel gioco farebbe sbattere sul muro la telecamera, "attaccata qui dietro con un braccio", con "un effetto fastidioso per cui la telecamera si abbassa e si rialza".

Che cosa misurare dipende dal gioco. "Sono tutti valori che io vado a testare e vado a raccogliere per il tipo di gameplay che io voglio avere." In un **platform** questo aspetto va curato particolarmente; in un **puzzle** che non richiede chissà quale movimento si può essere "meno interessati alla gym del salto e della corsa" e testare di più, per esempio, "i range in cui le porte si aprono". "Fondamentalmente è una mappa di test che permette di andare a *tweakare* il personaggio."

Che cosa avete già toccato? Il docente chiede se con Igor si siano visti i parametri del Character Movement. Uno studente risponde che hanno animato un personaggio "da zero prendendo su Mixamo le varie animazioni" e impostato l'animation blueprint, ma "del character movement abbiamo fatto solo la **max speed**". "Va bene, poi lo vediamo quando entriamo in engine."

Approfondimento: spingere le metriche al limite. Totten (capitolo 2) ricorda che "attraversare un livello è comodo quando gli spazi accolgono comodamente le metriche", e che "il dramma del gameplay" nasce da spazi che le spingono al limite: il suo esempio è il mondo 8-1 di *Super Mario Bros.*, dove un burrone è largo 10 blocchi, uno in più del salto in corsa di Mario (9 blocchi), e obbliga a usare l'isoletta larga un blocco nel mezzo; o i corridoi di *Resident Evil*, larghi appena per due personaggi, che rendono la fuga angosciosa. Il caso di *Team Fortress 2* che cita, con i tunnel di una mappa allargati dopo i test perché giocatori e carrello non ci passavano insieme, è un esempio di misura sbagliata scoperta solo provando. Sono le due facce della gym: trovare i valori comodi e sapere dove sta il limite.

1.13.5 Il block out e il caso di God of War

Il **block out** è "la cosa su cui ci concentreremo oggi". Che cos'è? Dopo il layout, si prende la mappa disegnata in 2D, "magari me la importo come PNG all'interno del progetto e la applico come materiale in modo che so dove andare a costruire"; poi si **ricostruisce il livello utilizzando unicamente geometrie**, "cercando di essere quanto più fedele possibile al risultato finale".

A che cosa serve. "Prima di andare a curare la parte estetica dobbiamo poter testare il gameplay del gioco all'interno del progetto." Si costruisce la mappa come la si è immaginata per chiedersi: "è piacevole camminare? L'esperienza va avanti troppo a lungo? È troppo breve? Riusciamo a dire tutto quello che vogliamo dire in questa fase narrativa prima che finisca la camminata? L'arena con i nemici è abbastanza grande?"

Il video: il block out di God of War. Il docente mostra un video, "un *case study*": non il vero block out di *God of War* (2018), ma il lavoro di "una persona che ha cercato di replicare come sarebbe potuto essere il block out" della prima parte del gioco. Le osservazioni, mentre il video scorre:

- già nel block out compaiono i **landmark**, "oggetti che danno identità alle ambientazioni" ("poi ne parleremo"): l'uscita dalla casa di Kratos, con un portale e un segno verso l'alto; un albero crollato; un **doppio albero** che "potrebbe indicare la strada perché fa effetto arco, tu passi sotto l'arco"; la scalinata;
- c'è un primo abbozzo di illuminazione, "in fase super sperimentale, super di prototipazione" (la trascrizione dice "non di plot out delle luci": il senso è che è solo un'idea di come la luce sarà distribuita);
- non c'è **Atreus**, non c'è nulla del combattimento, "nessuna funzione di gameplay effettivo": dove servirebbe un salto, il video "skippa direttamente la parte successiva perché magari noi non abbiamo ancora implementato il salto"; ma sappiamo già che lì servirà. In *God of War* 2018, nota il docente, "non c'è un salto vero e proprio, c'è semplicemente un *trigger*" che porta al prossimo terreno calpestabile; in un platform, dove il salto conta, quella parte si gestirebbe diversamente;
- il tronco da scavalcare; il **forziere**: senza meccanica di apertura né bottino, "ma a livello di design sappiamo già che qui vogliamo piazzare un forziere per **premiare l'esplorazione**": "molto bravo, sei arrivato in questo punto, prendi la reward";
- la **catena**: anche senza la meccanica di discesa, si mette un punto di riferimento "diverso dal resto della mappa", perché il giocatore capisca che ci si può interagire e si chieda "io lì giù come ci arrivo? Dove mi porta quella catena?"; guardando in basso scopre un altro livello e "inizia a pianificare la strada";
- un oggetto da spaccare con l'ascia ("skip, perché non c'è l'ascia"); un albero spaccato, landmark "importante per la resa estetica della scena";
- l'**arena**: "questi sono i nemici, questa è l'arena"; il combattimento si testerà lì appena ce ne sarà una base, "ancora prima di avere la parte estetica"; intanto la zona c'è, e "magari la andiamo ad allargare, magari la andiamo a stringere";
- un cadavere a terra "per far capire che c'è qualcosa da raccogliere";
- le **bandiere**, "punti di riferimento che vanno seguiti", che guidano il passaggio del giocatore: "se io voglio guidare il percorso del giocatore, gli metto degli oggetti che lui può seguire";
- infine un'altra cassa dietro un oggetto da spaccare, e la fine della camminata iniziale, "senza animazioni, senza telecamera che si muove": "non è necessario in questa fase". Si vede che sotto c'è un altro oggetto, "quindi capisco che si può scendere di sotto per esplorare".

"Vado a creare tutta una struttura, tutta una situazione che mi permette di andare effettivamente a prevedere quello che sarà il mio gameplay." Nessuna domanda.

Approfondimento: whiteblocking e landmark nei libri. Quello che il docente chiama block out, Totten (capitolo 2) lo chiama **whiteblocking**: creare un livello "con geometria semplice, il più delle volte blocchi bianchi o con una texture", per verificare che raggiunga gli obiettivi di gameplay prima che arrivi l'arte dell'ambiente. La geometria usata è "la più semplice necessaria per simulare i *collider*" del livello finale. Totten racconta anche il metodo di Valve con l'editor Hammer, le cui primitive si chiamano proprio

brush: si prova lo spazio, si corregge modificando le brush, e "quando i designer si ritrovano a modificare poco degli spazi principali e si concentrano sui dettagli, il livello è pronto per l'arte". È un buon criterio per capire quando un block out è finito. Sui **landmark**, lo stesso libro (nel capitolo sulle strutture dei mondi di gioco) riprende il termine *architectural weenie* di Walt Disney, ripreso a sua volta da Jesse Schell: un elemento riconoscibile e visibile da lontano (il castello di Disneyland, la Cittadella di *Half-Life 2*) che attira il visitatore verso un obiettivo e, allo stesso tempo, gli permette di orientarsi. Il doppio albero e le bandiere del video sono landmark e guide di questo tipo, in scala ridotta.

1.13.6 In engine: la gym, la griglia e la distanza di sprint

Si passa a Unreal ("interrompiamo la condivisione e andiamo a condividere"). Nel progetto c'è già "quello che sembra un abbozzo di gym": la zona in cui sono state sistemate tutte le meccaniche delle lezioni con Igor, "in modo da poterle utilizzare facilmente con il nostro personaggio". Andando in una direzione si vede "la luce che si accende e che si spegne", si collezionano "gli oggettini", si vede come funziona la porta, si provano "l'acqua", "la roccia" e "l'ascensore" (la pedana che sale). Una gym serve anche a decidere su queste meccaniche:

- alla porta "posso voler aggiungere un bottone"; oppure "si apre quando sono troppo vicino, facciamolo aprire con il range un pochino più lontano";
- l'ascensore: "a questo punto è un po' scomodo, è troppo in alto", e si modifica "il massimo livello d'altezza che posso raggiungere";
- "quanti metri riesco a fare con una barra della stamina": per questo serve uno spazio più grande, che si costruisce "un po' improvvisato" in un'altra zona, "un altro piccolo parco giochi in cui possiamo prendere le nostre misure".

Il materiale a griglia. Il materiale del pavimento, "proprio del pacchetto che abbiamo utilizzato", è un materiale con una **griglia**, e non a caso: "la griglia ci permette di avere delle misure". Se l'ascensore deve salire "soltanto di un tot", si modifica il punto d'arrivo nel suo Blueprint "per dire: questo punto deve portarmi fino a x metri"; se si vuole misurare la corsa, si mettono a terra dei "semplici blocchi".

Lo snapping. "Quando andate a lavorare con le misure vi consiglio di utilizzare le **grid**", cioè lo *snapping* della griglia dell'editor: "se dovessi muovere questo oggetto, lo muoverei di 10 cm per volta". (Il docente l'aveva disattivato: "se solo non l'avessi tolto, però ci siamo capiti".) La lettura del pavimento: "ogni quadrato è un metro, quindi ogni quadratino sono 20 cm"; uno spostamento di 10 cm è "la metà di questo quadratino". Per i blocchi di prova il docente li abbassa di un metro, in modo che spuntino dal pavimento della misura voluta.

La misura: Max Sprint Distance. Il procedimento: il personaggio si mette all'inizio della pista, si aspetta che la stamina sia piena ("aspettiamo che la stamina vada") e si corre in linea retta finché la stamina finisce. "Finisce qui." Si contano i metri sul pavimento a griglia, da uno a ventisei: **26 metri**. Il tratto percorso si può evidenziare accorciando il blocco usato come pista, o cambiandogli colore con un altro materiale; e sopra si scrive un testo: "in sprint dritto per dritto, il nostro robottino fa tot metri". Il cartello dice **Max Sprint Distance: 26 m**, "e questo ce lo segniamo". Se il valore non va bene si cambia: con la velocità con cui si consuma la stamina, o con la velocità di movimento del character (figura 1.13.3).

A che cosa serve saperlo? Il docente lo chiede alla classe; la risposta di uno studente, "un inseguimento", è quella giusta. "Se abbiamo una situazione in cui stiamo venendo inseguiti, o in cui stiamo inseguendo qualcuno", dopo quel tot di metri "si crea un problema": o veniamo raggiunti, quindi "dobbiamo metterci in salvo entro tot metri", o dobbiamo raggiungere qualcuno entro tot metri, oppure "ogni tot metri dobbiamo mettere degli oggetti che ricaricano la stamina". Altri usi:

- in un **platform 3D** o in un puzzle in cui si deve correre dal punto A al punto B, si sa che entro i 26 metri, "possibilmente entro i 20, per avere un minimo di **margine d'errore**", potrebbe servire un oggetto che ricarica la stamina;

- in un **horror** le ricariche raccolte al volo non hanno senso; ma "una roba tipo alla *Little Nightmares*": fra l'ostacolo A e l'ostacolo B, che si percorrono correndo, non ci possono essere più di 26 metri, "o più del valore che trovo poi effettivamente come risultato finale".

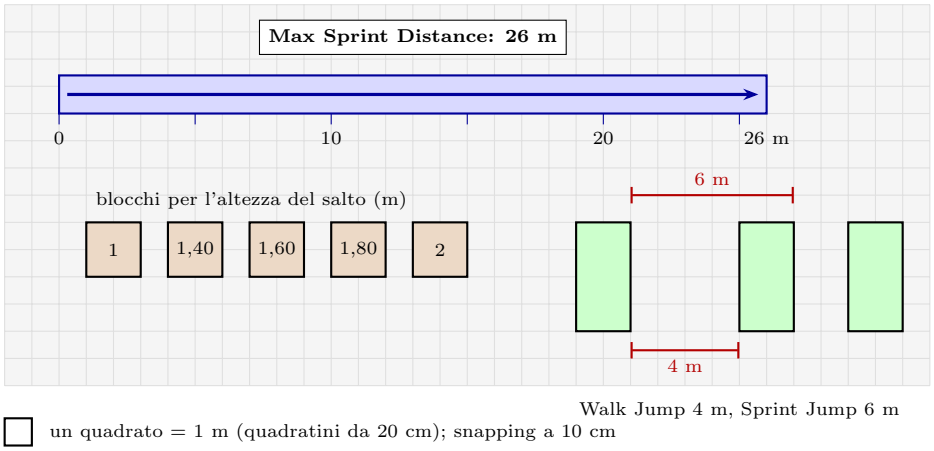


Figure 1.13.3: La gym improvvisata a lezione, vista dall'alto: la pista dello sprint, misurata sul pavimento a griglia fino all'esaurimento della stamina, con il cartello della misura; la fila di blocchi per l'altezza del salto; le piattaforme per la lunghezza dei salti camminando e correndo. I valori sono quelli trovati in diretta dal docente, che li definisce "non calcolati"; la disposizione è schematica.

Approfondimento: il cartello con le misure. Il nodo con cui si scrive il cartello non è nominato a lezione. In Unreal Engine 5 il modo più rapido è l'actor **Text Render** (nel menu Quick Add, sezione dei *Basic* o cercandolo nel Place Actors Panel), il cui testo si scrive nel campo *Text* dei Details e resta visibile anche in gioco; per non vederlo nell'eseguibile si spunta *Hidden in Game*. In alternativa si può usare un widget o, per appunti solo nell'editor, un actor di tipo *Note*. Funzionamento generale dell'engine.

1.13.7 I salti e il feeling del personaggio

"Stessa cosa la possiamo fare con i salti." Si costruisce una fila di blocchi d'altezza, "come erano quelli nella gym che avevamo visto precedentemente". Il docente parte da un blocco "che so essere un metro per un metro" (il primo che aveva fatto "non era esattamente un metro per un metro"), ne fa uno da **1,40 m**, poi da **1,60 m**, **1,80 m** e **2 m**, "eccetera, eccetera". ("Non fate caso al fatto che li ho fatti anche sottoterra, tanto era giusto per testare.")

Due salti, due comunicazioni. In Play il bot supera il cubo da un metro e non quello da 1,40. "Fra questi due salti, quale vi sembra il più corretto? Che feeling vi danno?" Uno studente: quello da 1,40 è "prevedibile"; per quello da un metro "non si capisce neanche perché ce la faccia". "Esatto, e qui entra tutto il discorso del **feeling**" (figura 1.13.4):

- sul blocco da **un metro** il personaggio non atterra sopra: "scivola verso l'alto", perché la sua collisione è una **capsula**, "quindi verso il basso non è quadrata, è tonda": arriva più o meno all'altezza in cui la parte arrotondata può *slidare* sopra lo spigolo. "Questo salto da un metro non è molto soddisfacente"; visto da lontano "sembra quasi più *smooth* che fatto da vicino";

- sul blocco da **1,40** il bordo "non arriva sotto la metà del mio character, arriva ad altezza busto, e io riesco subito a capire che il mio personaggio questo salto non lo farà mai".

"Quindi questa è **buona comunicazione**. Questa è **pessima comunicazione**": un ostacolo deve dire chiaramente al giocatore se si può superare o no. E se il risultato non piace, si cambia "il modo in cui il mio character si comporta a livello spaziale".

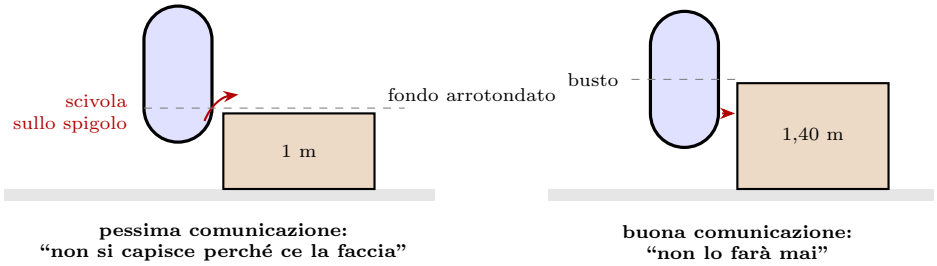


Figure 1.13.4: Il feeling dei due salti provati a lezione. La collisione del personaggio è una capsula: sul blocco da un metro la sua parte inferiore arrotondata raggiunge lo spigolo e ci scivola sopra, e il salto riesce in un modo poco leggibile; il blocco da 1,40 m arriva all'altezza del busto anche al culmine del salto, e il giocatore capisce subito che non si può superare. Ricostruzione schematica.

Approfondimento: perché la capsula "sale" sugli spigoli. Il Character Movement di Unreal muove la capsula con una serie di *sweep*; quando la capsula urta uno spigolo con la sua semisfera inferiore, la normale dell'urto è inclinata e la componente verticale della velocità la fa scorrere verso l'alto. In più, se lo spigolo è più basso della *Max Step Height* rispetto ai piedi, il componente tenta uno *step up*. Per questo l'altezza "limite" di un ostacolo non coincide con l'altezza massima del salto misurata ai piedi: va misurata nella gym, come fa il docente, e gli ostacoli che non si devono superare vanno tenuti con un buon margine sopra quel limite. Funzionamento generale dell'engine; il libro di Romero e Sewell non entra in questo dettaglio.

1.13.8 I parametri del Character Movement

Il **Character Movement** (il componente, selezionato nel pannello Components del Blueprint del personaggio) ha "tutta una serie di parametri". Il docente li scorre dall'alto in basso.

Gravità e massa.

- la **gravità** (*Gravity Scale*), "quanto la gravità è forte per il mio personaggio": "se voglio fare il mio gioco ambientato sulla Luna", la si rende "meno netta", "più lunare, più morbida";
- la **massa** (*Mass*), "quanto pesa": secondo il docente è il valore su cui si calcola il *momentum*, cioè quanta spinta resta quando, correndo, si salta e si smette di dare accelerazione (si veda l'approfondimento alla fine della sezione).

Walking. Più in basso, "tutto quello che riguarda il *walking*":

- **Max Step Height**, "il massimo su cui il mio personaggio può salire" con un passo: con il valore di default di 45 cm, un ostacolo di 45 cm lo si scavalca camminando, uno di 50 blocca il personaggio. Prova in diretta con un blocchetto di circa 40 cm: il bot "ci sale sopra direttamente". "Mi va bene questa cosa? Forse, forse no." Il docente prova poi a portare il valore a un metro, ma il bot non sale sul blocco da un metro (prima dimentica di salvare e compilare; dopo, "lui non scavalca comunque"): "altrimenti ci sono gli altri parametri che vanno modificati anche per fargliela fare". La prova si chiude senza una soluzione;
- **Walkable Floor Angle**, l'inclinazione massima su cui si cammina;

- la **velocità** (*Max Walk Speed*), "che avevate già modificato";
- se c'è il *crouch*, la velocità da accovacciati (*Max Walk Speed Crouched*).

Jumping and Falling. Poi i parametri del salto. "Come funziona il salto? Nel momento in cui gli diamo l'input del salto, lui applica una forza sull'asse Z di **420** centimetri al secondo": è la **Jump Z Velocity**. Aumentandola, "il salto sarà più alto perché riceverà una spinta più forte". Qui c'è un intoppo: il docente modifica il valore ma in Play non cambia nulla ("ma che character? Questo non è quel character lì"): sta modificando un Blueprint diverso da quello che la Game Mode fa nascere. Pensa di provare "col manichino di Unreal, almeno sappiamo che è lui", poi trova il personaggio giusto, **BP_Bot2**. Con la Jump Z Velocity a **700** il bot "salta agevolmente anche sopra i due metri": per un platform "è un salto molto più permissivo", e si arriva anche più lontano. Ma "magari 700 è un po' troppo. Magari faremo 600, 650".

Altri parametri della stessa sezione:

- l'**Air Control**, il "controllo aereo": "una volta che il mio robot è in aria, quanto può muoversi? Quanto può fare destra e sinistra? Quanto controllo ho io del mio movimento?";
- "una serie di altri moltiplicatori" (*Air Control Boost Multiplier* e *Air Control Boost Velocity Threshold*): "se la velocità è inferiore di tot, allora gli applica un moltiplicatore";
- la frizione in caduta (*Falling Lateral Friction*), che il docente descrive come "una decelerazione per cui io cado lentamente" (si veda l'approfondimento);
- più sotto, i parametri del **nuoto** (*Swimming*) e del **volo** (*Flying*), "se poi effettivamente voi avete quelle funzioni all'interno del vostro character".

Le misure del salto. Deciso quanto in alto deve saltare il personaggio, "senza che adesso continuo tutta questa scala per tunare cose", si misura quanto salta *in lungo*. Camminando: "uno, due, tre, quattro metri", **4 m** in orizzontale. Correndo: "uno, due, tre, quattro, cinque, sei", **6 m**. Si scrive sul cartello della gym: **Walk Jump 4 m**, **Sprint Jump 6 m**. Da qui la regola per il livello: se si fa un platform, i salti richiesti dovranno stare all'incirca entro "i due metri e 20, 30 d'altezza e i cinque metri e 80 di lunghezza". "Sto sparando i numeri, sono cose molto rapide, molto non calcolate, però questa è l'idea": "io adesso fittiziamente sto dicendo a me vanno bene questi valori, poi voi decidete il valore che a voi va bene". Si noti che i limiti scelti sono un po' *sotto* i massimi misurati: è lo stesso margine d'errore dei 20 metri contro i 26 dello sprint. (Nella trascrizione la misura "da fermo" e quella "camminando" si confondono: il docente dice "da fermo quanto salto in là?" e poi "salta 4 metri in orizzontale mentre sta camminando"; il cartello dice *Walk Jump*.)

"Poi magari post pausa ci facciamo un po' di esperimenti con questa roba qui": ognuno si farà una gym "per determinare effettivamente come il character volete che funzioni", perché ognuno userà quel character per creare i propri livelli, "dei piccoli livellini in autonomia". (Dopo la pausa, come si vedrà, il docente rinuncia a farlo in diretta e lo lascia come compito.)

Approfondimento: velocità, non forza; massa e frizione. Due precisazioni rispetto alle parole della lezione. La prima: la **Jump Z Velocity** non è una forza ma una *velocità* iniziale (in cm/s): al salto il componente imposta la componente Z della velocità a quel valore, e poi la gravità la riduce. Con la gravità di default (circa 980 cm/s^2) l'altezza teorica del salto è $h = v^2/2g$: con 420 cm/s circa 90 cm, con 700 cm/s circa 2,5 m, cioè "agevolmente sopra i due metri", come visto a lezione; per lo stesso motivo il salto più alto dura di più e porta più lontano. La seconda: nel Character Movement la **Mass** serve soprattutto alle interazioni fisiche (le spinte ricevute con *Add Impulse* o *Add Force* e la forza con cui il personaggio spinge gli oggetti simulati); lo slancio in aria dipende invece da Air Control, *Braking Deceleration Falling* e *Falling Lateral Friction*, che frenano lo spostamento *orizzontale* e non rallentano la caduta (per cadere più piano si agisce sulla Gravity Scale). Infine, la **Max Step Height** non è indipendente dalla capsula: lo *step up* si fa spostando la capsula verso l'alto e in avanti, e un gradino paragonabile all'altezza della capsula stessa non viene salito in modo affidabile (contano anche il Walkable Floor Angle e la superficie su cui si appoggia il passo). È una spiegazione plausibile del fallimento della prova con un metro, che il docente non verifica. Romero e Sewell presentano il componente nel capitolo 3 ("usato per definire vari tipi di movimento del

Parametro (Character Movement)	Default	Che cosa cambia
Gravity Scale	1	moltiplica la gravità: meno di 1 per una gravità "lunare"
Mass	100 (kg)	la massa del personaggio (spinte e interazioni con la fisica)
Max Step Height	45 (cm)	il gradino più alto che si sale camminando
Walkable Floor Angle	44,77°	la pendenza massima camminabile; oltre si scivola
Max Walk Speed	600 (cm/s)	la velocità massima a piedi (nel bot, la corsa cambia questo valore)
Max Walk Speed Crouched	300 (cm/s)	la velocità da accovacciati
Jump Z Velocity	420 (cm/s)	la velocità verticale iniziale del salto: più alta, salto più alto (e più lungo)
Air Control	0,05 (0,35 nei template)	quanto si può sterzare in aria; 1 = come a terra
Air Control Boost Multiplier / Velocity Threshold	2 / 25	più controllo in aria quando la velocità orizzontale è bassa
Falling Lateral Friction	0	attrito orizzontale in aria: frena lo spostamento laterale
Braking Deceleration Falling	0	decelerazione orizzontale in aria senza input
Swimming, Flying	—	i parametri del nuoto e del volo, se il personaggio li usa

Table 1.13.1: I parametri del Character Movement nominati a lezione, con i valori di default del componente in Unreal Engine 5 (approfondimento: i default non sono letti a lezione, tranne 45 per la Max Step Height, 420 per la Jump Z Velocity e "44" per il Walkable Floor Angle; i template di Unreal e i personaggi degli studenti possono avere valori diversi, come la Max Walk Speed a 500 del bot).

personaggio, come camminare, correre, saltare, nuotare e volare") e nel capitolo 6 ne modificano la *Max Walk Speed* (default 600) per lo sprint; i parametri del salto non sono trattati nel libro, e il resto di questo paragrafo si basa sul funzionamento generale dell'engine.

1.13.9 Le geometry brush: che cosa sono e come si usano

Si torna "al volissimo" sulla presentazione, per "un tipo di actor che credo non abbiamo visto assieme": le **geometry brush**. "Correggetemi se sbaglio": non le hanno viste, "per quanto siano una roba molto semplice, molto basilare".

Static mesh o brush. A che cosa servono? "Ci servono per andare a fare **prototipazione**." Un livello non si comincia con gli asset finali; per un block out gli asset principali sono due:

- le **static mesh**, "molto più leggere, ma meno malleabili": "sono dei blocchi", si piazzano, si spostano, si ingrandiscono, si duplicano, "però vi danno meno libertà di manovra, a meno che voi non le modifichiate con il *modeling tool*", che però "nelle precedenti versioni di Unreal non c'era, è solo nella quinta";
- le **geometry brush**, che ci sono anche in Unreal 4: "mettiamo caso che state lavorando con la quarta", perché "c'è ancora chi lavora con la quarta per fare dei progetti, perché è più stabile magari rispetto alla quinta": lì il modeling tool non c'è, le brush sì.

Le brush esistono "in diversi tipi e forme", ognuna con i suoi parametri. "Come per tutte le cose che trattiamo a lezione, se vi serve un approfondimento potete tranquillamente andare sulla **documentazione di Unreal**" (sulla slide c'è un link), che spiega dove trovare l'actor, quali parametri ha, e mostra la differenza fra un block out e il livello finale: "idealmente io voglio costruirlo in questo modo, poi verrà in tutt'altra maniera". "Sono degli strumenti molto semplici da utilizzare, per quanto vadano utilizzati con **parsimonia**" (il perché arriva più avanti).

Dove si trovano. (Un problema con la condivisione blocca per un momento la viewport: "non mi si muoveva più".) Nella barra in alto dell'editor c'è "questo quadratino con la tendina", il menu per aggiungere actor (*Quick Add to the Project*); nella voce **Geometry** ci sono le brush, "come avete gli altri volumi, come avete le shape, come avete le luci". In alternativa si apre il **Place Actors Panel**, "un pannello come il content browser", che si può spostare e agganciare dove si vuole: il docente lo teneva di lato, "ma potete mettervelo anche insieme al content browser". Le forme disponibili: **Box**, **Cone**, **Cylinder**, **Curved Stair**, **Linear Stair**, **Spiral Stair** ("che in realtà è semplicemente una scala curva *with extra steps*") e **Sphere**.

I materiali vanno sulle facce. "Prima cosa molto importante": a differenza delle static mesh, le brush non hanno uno slot di materiale unico o slot personalizzabili, "ma hanno delle **facce**": "ogni faccia avrà un materiale a sé e avrà delle proprietà a sé", e le UV dei materiali si modificano "direttamente dalla faccia". Se si mette un muro di mattoni su una faccia, "il resto non diventerà un mattonato".

Brush Settings, non la scala. Oltre alle facce, nei Details ci sono i **Brush Settings**, diversi per ogni forma. "Altra cosa a cui dobbiamo stare attenti": se si allunga la brush con lo strumento di scala, la si *stretcha*, e con lei i materiali delle facce. "È ideale, quando lavorate con le brush, piuttosto che stretchare, andare a modificare i setting direttamente": cambiando la X nei Brush Settings "vedete che non streccia le facce: le facce rimangono dimensionate e proporzionate come dovrebbero".

La Box. Il docente fa "un box di 600", "il nostro cubone". I parametri:

- **X**, **Y**, **Z**, le dimensioni;
- **Hollow**: il cubo diventa vuoto, "quando noi entriamo ha delle pareti anche all'interno";
- **Wall Thickness**, lo spessore dei muri quando la box è hollow: di default 10, portato a 50: "la stanza all'interno è più piccola di come si vede all'esterno";
- **Tessellated**, che funziona solo se la box *non* è hollow: "vi divide l'oggetto in triangoli in modo che è più facile da editare" (nella modalità di modifica delle brush, più avanti).

Additive e Subtractive. "Una cosa molto importante delle brush, che ci velocizza tantissimo il lavoro", è il **Brush Type**. Di default una brush è **Additive**: "viene aggiunta alla scena, all'ordine delle brush". Ma può essere **Subtractive**, e allora "ci permette di fare dei buchi" nelle brush additive (figura 1.13.5). "Questa cosa vi permette di fare delle casette *very quick*": una box hollow e, dentro, una box sottrattiva (il docente le dà dimensioni come 350) che apre una porta; poi un'altra sottrattiva sul soffitto, "perché vogliamo che nel nostro livello si arrivi in una stanza e si cada dentro un'altra stanza". "Noi adesso abbiamo una casetta con un buco sopra." Si possono fare finestre, strade: "vi dà un modo per andare a modificare le varie strutture, le varie forme".

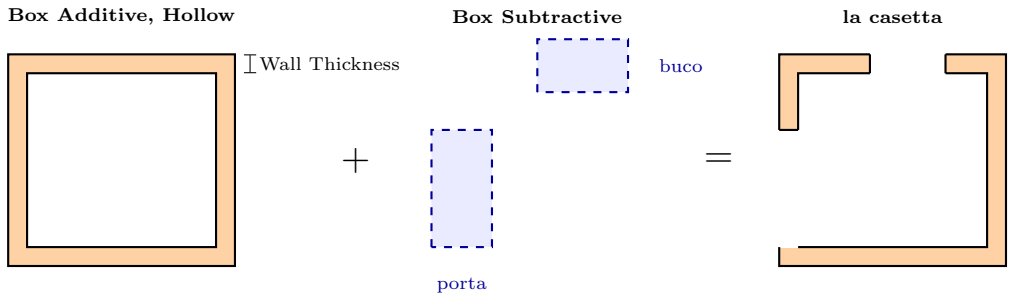


Figure 1.13.5: Additive e Subtractive in sezione: una box additiva vuota (Hollow, con il suo Wall Thickness) e due box sottrattive che la "bucano", una per la porta e una per il buco sul soffitto da cui cadere nella stanza sottostante, come nella casetta costruita a lezione. Ricostruzione schematica; le proporzioni sono indicative.

Il però: le brush sono pesanti. "Ora c'è un però, e il però è che le brush sono **pesanti**." Se si riempie una scena di brush, "perché volete diverse casette e diverse zone dentro cui entrare", "si carica tanto sulla memoria del progetto". Il rimedio: **farla diventare una static mesh**. Nei Details della brush c'è il comando per creare una static mesh dalla geometria (*Create Static Mesh*, nella parte avanzata dei Brush Settings), che chiede dove salvarla. Il docente lo fa con la casetta: nel Content Browser compare "un bloccone", ma solo della brush selezionata ("se avessimo selezionato tutte quante le brush, avremmo avuto un effetto diverso": la mesh le avrebbe comprese tutte). Il docente elimina la brush originale, "non ci serve più": la static mesh che resta "è molto meno pesante. Io posso riempirci la scena di questa roba qua." Bisognerebbe solo sistemarne il **pivot**, "perché ce l'ho salvato col pivot strano". Un consiglio di ordine: invece di "buttarla qui in mezzo al Content", come ha fatto lui, create "una *new folder* che chiamate, che ne so, **Blockout Assets**", e lì dentro mettete tutto quello che create. (Poi le elimina: a lui non servono.)

Approfondimento: perché le brush pesano, e Create Static Mesh. Le geometry brush sono la versione moderna della geometria **BSP** (*binary space partitioning*) delle vecchie versioni di Unreal: le brush additive e sottrattive vengono combinate, nell'ordine in cui sono state create, in un'unica geometria del livello, che l'editor ricalcola (*Build Geometry*) quando le si modifica. Questa geometria non si può istanziare, non ha LOD, ha UV e lightmap generate automaticamente e la sua collisione è meno efficiente di quella di una mesh: per questo, oltre che per la memoria, Epic raccomanda di usarle per il block out e di convertirle (o sostituirle) con static mesh nel livello finale. Il comando **Create Static Mesh** salva la geometria delle brush selezionate come asset *Static Mesh*, con i materiali delle facce come slot; la mesh nasce con il pivot all'origine del mondo o della prima brush, che si corregge con il *Modeling Mode* (strumento *Pivot*) o riposizionandola. Nel libro di Romero e Sewell le brush non compaiono: il libro costruisce i livelli con static mesh duplicate (capitolo 9, il pavimento duplicato con Alt e trascinamento per raddoppiare l'area di gioco). Quanto al modeling tool, il *Modeling Mode* esisteva già come plugin sperimentale nelle ultime versioni di Unreal 4, ma è con Unreal 5 che è diventato uno strumento standard dell'editor: la frase del docente è vera nella sostanza. Funzionamento generale dell'engine, salvo il riferimento al libro.

1.13.10 Le altre forme e il Brush Editing

"Vediamo effettivamente cosa ci permettono di fare le altre brush." (La box sottrattiva il docente se la tiene, "almeno vediamo".)

Cone. Ha sempre le stesse impostazioni per le facce. Nei Brush Settings: un valore **Z**, l'altezza; un **Outer Radius**; un **Inner Radius**, che non si può modificare finché il cono non è **Hollow**: "se io la rendo vuota" si vede com'è dentro, e si crea una struttura "sulla base di quello che a me serve". Il docente ci mette dentro la box sottrattiva: "una capannola mezza favola, una tenda". Si può aumentare il numero di lati (**Sides**): da 3, "una piramide, anzi un D4", fino a 500. A 500 "ha un po' stutterato, e non ci sarà mai un momento in cui ti servirà avere un cono così preciso", ma più facce lo rendono "un po' più smussato".

Cylinder. "Più o meno le stesse cose": l'altezza, l'Outer Radius ("quanto è largo effettivamente"), le facce come per il cono, Hollow o no, e il tipo, additivo o sottrattivo.

Sphere. Il **Radius**, "quanto è ampia", e la **Tessellation**, "quanti blocchi ha, quindi quanto è sferica".

Linear Stair. Tutto si basa sullo scalino, "come anche le altre scale": la lunghezza del singolo scalino (**Step Length**), l'altezza (**Step Height**), la larghezza (**Step Width**) e il numero di scalini (**Num Steps**): "aumentando gli scalini, aumenta l'altezza e aumenta la distanza". Poi **Add to First Step**, "quanto in alto in più deve essere il primo scalino": "vedete, lui alza un tot".

Curved Stair. Altezza e larghezza dello scalino, e in più:

- l'**Inner Radius**, il raggio del cerchio interno: "se è molto largo, le scale vengono più arcuate";

- l'**Angle of Curve**, "quanto deve curvare la scala": di default 90 gradi, "però potreste fare anche 180, 270, anche 360 gradi", se volete "una singola chiocciola";
- Num Steps e Add to First Step, "esattamente allo stesso modo";
- **Counter Clockwise**, per le scale "che vanno nell'altra direzione".

"Con tutte queste, poi potete andare a creare effettivamente delle static mesh."

Spiral Stair. "Gli stessi parametri": Inner Radius, Step Width, Step Height, e in più lo spessore del singolo gradino (**Step Thickness**), "perché è costruita proprio a gradoni, ognuno di questi è un oggetto a sé stante, tutti attaccati l'uno con l'altro". C'è poi il numero di scalini *per 360 gradi* (**Num Steps Per 360**): "se voi dite che devono esserci 33 scalini per fare 360 gradi, poi dovete aggiungere abbastanza scalini per arrivare a fare 360 gradi"; abbassandolo, con lo stesso numero totale di scalini ("50, 60") ne servono meno per fare un giro. Infine si può rendere liscio il soffitto (**Sloped Ceiling**) o il pavimento (**Sloped Floor**), "e quindi avere una sorta di scivolo, una rampa", e Counter Clockwise.

Il Brush Editing. "Prima facciamo un'ultima roba": passare dalla **Selection Mode** al **Brush Editing** (dal menu delle modalità dell'editor). Qui si fanno "piccole operazioni di modellazione": selezionare un **vertice** e spostarlo, una **faccia** e muoverla, un **edge** e muoverlo, "in questo modo ci andiamo a creare esattamente il tipo d'oggetto che serve a noi, anche se la forma che serve a noi non c'era" fra le brush. E lo **Split**: "fa passare una serie di vertici e di edge a tagliare", una sezione dell'oggetto; "si formano nuove facce, e queste nuove facce possono essere prese e modificate". "Non so quanto spesso vi tornerà utile fare un ecomostro del genere, però avete la possibilità di farlo." Un'avvertenza emersa dalla prova: dopo le modifiche, l'oggetto "è ancora una brush", ma "appena l'andiamo a modificare" dai Brush Settings "ritorna come era precedentemente": i parametri ricostruiscono la forma da zero e cancellano le modifiche manuali.

Brush	Parametri (Brush Settings)
tutte	Brush Type: Additive / Subtractive; i materiali e le UV si impostano per faccia
Box	X, Y, Z; Wall Thickness (default 10); Hollow; Tessellated (solo se non hollow)
Cone	Z (altezza); Cap Z; Outer Radius; Inner Radius (solo se hollow); Sides (da 3 a 500); Align to Side; Hollow
Cylinder	Z; Outer Radius; Inner Radius (solo se hollow); Sides; Align to Side; Hollow
Sphere	Radius; Tessellation
Linear Stair	Step Length; Step Height; Step Width; Num Steps; Add to First Step
Curved Stair	Inner Radius; Step Height; Step Width; Angle of Curve (default 90°); Num Steps; Add to First Step; Counter Clockwise
Spiral Stair	Inner Radius; Step Width; Step Height; Step Thickness; Num Steps Per 360; Num Steps; Sloped Ceiling; Sloped Floor; Counter Clockwise

Table 1.13.2: Le geometry brush e i loro parametri. I nomi sono quelli dei Details di Unreal Engine 5; quelli non letti a lezione (Cap Z, Align to Side) sono completati come approfondimento.

"Sono le cinque e mezza." Il docente annuncia che, al ritorno dalla pausa, si potrebbe fare "un piccolo block out utilizzando gli oggetti che avete": le brush, le static mesh e gli oggetti fatti con Igor, "per cui magari vi piazzate dei collectible da raggiungere in giro", "anche molto semplice". Pausa di un quarto d'ora: "facciamo 45".

1.13.11 La pausa e la scelta del formato

(Durante la pausa la trascrizione contiene solo il blocco di testo ripetuto citato nell'introduzione.) Alla ripresa c'è un problema di volume ("stavo leggendo il messaggio che aveva scritto Augusto in chat") e l'appello: Federico c'è, Guido "c'è, ma non gli funziona la telecamera" ("basta che siete responsivi").

La domanda del docente. "Ci stavo pensando adesso che stavamo fermi": come preferiscono strutturare le lezioni?

1. una prima parte di spiegazione e una seconda in cui gli studenti "provano a costruire qualcosa", magari in streaming, "così gli altri vedono anche cosa fate" e il docente può commentare;

2. tutte e tre le ore di spiegazione, con il lavoro pratico fra una lezione e l'altra e i feedback all'inizio della lezione successiva. Il tempo a disposizione cambia: "da oggi, che è giovedì, fino a martedì avrete più tempo; da martedì a giovedì ne avrete meno".

Un primo studente preferisce la prima opzione, "perché almeno abbiamo dei feedback diretti e ci si può soffermare di più sulle varie cose". Un altro la seconda, "perché forse riusciamo a fare più cose", "vediamo un po' più di argomenti". Il docente avverte che le cose che si faranno con lui "sono tendenzialmente non troppo complesse, è più una questione di metterle assieme", e che non vorrebbe la situazione in cui "io vi dico facciamo questa cosa, la prossima volta me lo fate vedere e nessuno fa niente". Edoardo, alle prese con il caldo, dice che fare le cose fra una lezione e l'altra "non è un peso" se trova una soluzione; un altro: "personalmente mi piace la seconda". Il voto ("siete dispari, quindi uscirà fuori una verità o l'altra"; "per chi ci guarda da casa sarà tutto in mano alle persone che erano qui") premia la **seconda**: "Tommaso, indipendentemente dal tuo input, il popolo ha deciso di riempire la lezione con delle cose che magari faccio io".

Conseguenze: la lezione sullo **speed level design** andrà "rimaneggiata", "perché non so quanto ha senso che io mi metta a fare uno speed level per tre ore con voi che mi guardate"; era "una roba che mi sarebbe piaciuto far fare a voi". Per la volta successiva, a chi vuole, il docente chiede "un piccolo livellino" con geometrie e static mesh, "per giocare un po'": "è molto bello sapere come funziona Unreal, però deve esserci del **divertimento** nella creazione. Io mi devo divertire mentre creo il gioco." Le indicazioni precise arrivano a fine lezione (sezione 1.13.17).

Che cosa fare nell'ora rimasta. Personalizzare i parametri del bot in una gym "è relativamente fine a se stesso", "un discorso abbastanza semplice" che gli studenti possono fare da soli. Meglio un altro actor interessante: le **luci**, e "se riesco, un accennino di post process". Le luci si vedranno "in un vuoto", "senza un dressing già finito", e si riprenderanno quando si parlerà di *enrichment*, di dressing e di ambienti, con lo speed level; per quella lezione il docente potrebbe preparare "un ambientino" o passare un pacchetto, "così chi vuole lo fa assieme a me". Le slide non servono: "faremo a braccio". (Di nuovo problemi con Zoom: "ogni volta che parto con la stream non funziona il tasto destro".) Il docente costruisce con una grande box un ambientino chiuso ("così abbiamo un po' d'ombra"), invitando Filippo, che preferiva fare le cose insieme, a sperimentare in parallelo. (Una parte della costruzione, nella trascrizione, è incomprensibile: "me la metà? E questa la facciamo? Giusto per fare razzola".)

1.13.12 Le luci: i tipi e la mobility

"Cos'è che non sta più funzionando in quella zona lì?" Dentro la box: "la luce del sole", risponde uno studente. La luce del sole è un tipo di luce, "la signora **Directional Light**". Le luci di Unreal sono cinque: **Directional**, **Point**, **Spot**, **Rect** e **Sky Light**. Secondo la classificazione del docente:

- directional e sky light sono **luci ambientali**;
- point e spot light si usano "principalmente per l'illuminazione";
- la rect light "principalmente per i riflessi o per delle luci specifiche".

Il costo dei rimbalzi. "Una cosa importante da dire è che il modo in cui Unreal calcola le luci è simulando delle onde" e il modo in cui "queste onde luminose rimbalzano sulle superfici". Ci saranno situazioni in cui a una luce verranno chiesti "troppi rimbalzi, che equivalgono a troppi calcoli, che equivalgono a un peso molto elevato": bisogna stare attenti a come si posizionano le luci. (Sull'idea delle "onde" si veda l'approfondimento alla fine della sezione.)

La Mobility. "Un parametro fondamentale che c'è in tutte le luci" è la **Mobility**: di default le luci sono **Stationary**, ma ci sono anche **Static** e **Movable**. "Non mi aspetto una risposta super completa, voglio vedere se riusciamo ad arrivarci" ("basta che ci passate sopra col mouse e ve lo dice cosa fanno, però state con me"). Le ipotesi degli studenti: movable "sarà tipo la luce di una macchina"; stazionaria "che sta ferma e non si muove", ma statica "mi sembra simile come significato"; un altro propone "statica un lampione, stazionaria il faro di una macchina ferma, che potenzialmente si può muovere"; un altro ancora che la movable "dovrebbe avere delle proprietà fisiche, magari condizionata dalla gravità" ("la gravità non c'entra"); un altro l'angolazione ("no, l'angolazione dipende dal tipo di luce che utilizziamo"). Il docente rinuncia alla prova pratica ("qui c'è un po' di fregatura, perché siamo in preview") e parte dagli usi:

- una luce **movable** è quella di "una torcia a mano", di una torcia elettrica che si punta, dei fari di una macchina, delle luci di un aereo: "qualcosa in movimento";
- una luce **stazionaria** è quella di un lampione, di un soffitto, "di una qualsiasi luce di una casa" *dentro* il livello;
- una luce **statica** serve agli stessi scopi della stazionaria, "ma fuori dal vostro livello".

"Cosa cambia fra statica e stazionaria?" Uno studente: "nella stazionaria ci può interagire un actor come il character". "Esatto." Le proprietà sono due: se si muove la luce, l'ambiente reagisce?; se si passa davanti alla luce, la luce reagisce? (figura 1.13.6):

- la luce **statica** "viene cucinata sulla scena": è *baked* completamente nell'editor e non si può modificare in gioco. Quando si *buildano* le luci o si esporta il livello, "la luce viene bakeata ed è impressa sulla scena". Se si sposta l'actor della luce, la luce "rimane dove è stata bakeata"; se il character ci passa davanti, la luce non reagisce. Si usa "in tutti quei posti in cui so che non avrà interazioni e sarà soltanto luce d'atmosfera, luce d'ambiente". È "il tipo di luce più leggero";
- la luce **stazionaria** "se la muovo si rompe, ma se sta ferma e io ci passo davanti interagisce con me": il character proietta ombre corrette, ma se si sposta la luce dopo il bake, la luce non si aggiorna;
- la luce **movable** "fa entrambe le cose": è completamente dinamica, reagisce agli oggetti e può essere spostata "senza nessun problema".

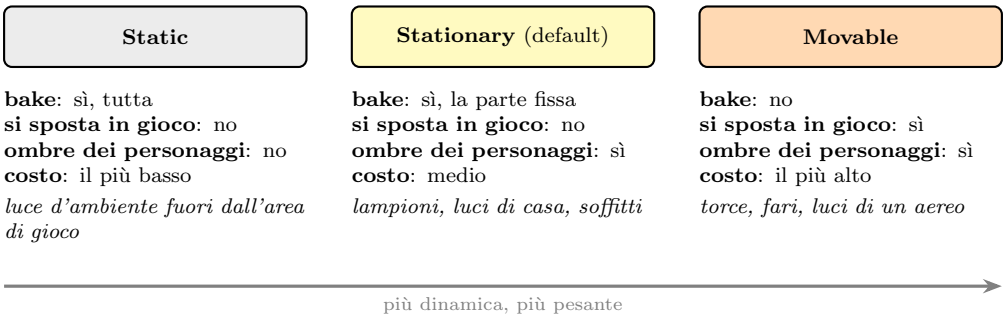


Figure 1.13.6: Le tre Mobility delle luci come le descrive il docente: che cosa viene precalcolato (*baked*), se la luce si può spostare in gioco, se reagisce ai personaggi che le passano davanti, quanto pesa e per che cosa si usa.

Approfondimento: onde, raggi e Lumen. L'immagine delle "onde" è una semplificazione del docente: Unreal non simula la luce come un'onda (niente interferenza o diffrazione), ma come energia che viaggia in linea retta e rimbalza, cioè per *raggi*. Il calcolo dei rimbalzi (l'illuminazione *globale*) avviene in due modi: precalcolato, con **Lightmass**, che scrive il risultato nelle *lightmap* delle superfici al momento del build (luci static e stationary); o in tempo reale, con **Lumen**, il sistema di illuminazione globale dinamica che Unreal Engine 5 attiva di default nei progetti nuovi. Il principio del docente resta valido: più superfici e più luci si

sovrappongono, più calcoli servono, che siano fatti una volta nel build o a ogni frame. Sulla stationary, un dettaglio in più: in gioco se ne possono cambiare colore e intensità (non la posizione), e le ombre che proietta sugli oggetti fermi sono precalcolate, mentre quelle degli oggetti in movimento sono dinamiche. Romero e Sewell (capitolo 4) mettono la *Mobility* di una point light a **Movable** proprio "per poter cambiare le proprietà della luce durante l'esecuzione", quando la accendono e la spengono da Blueprint. Funzionamento generale dell'engine, salvo il riferimento al libro.

1.13.13 La directional light e la sky light

Il docente porta la directional light e la sky light "tutte nello stesso posto". "Spostare queste luci non ha nessun tipo d'effetto." Perché?

La Directional Light. Si può muoverla e scalarla "come voglio e non cambia assolutamente nulla. Ma se la ruoto, la luce reagisce." ("In questo caso reagisce anche" un altro actor del cielo, "perché è collegato": la trascrizione dice "la Schakespeare" e "la Schiapunsky", con ogni probabilità la *Sky Sphere* o la *Sky Atmosphere* del livello; il nome esatto non è ricostruibile.) La ragione: "la directional light è composta in una maniera abbastanza peculiare: è un **fascio di rette parallele** che prende tutta la scena". Spostarla non cambia nulla; ruotarla cambia "il modo in cui viene indirizzata la luminosità". I parametri interessanti:

- l'**Intensity**: aumentandola, "va in questo caso a bruciare un pochino tutta la scena" ("l'intensità della luce calcolata in X": l'unità di misura nella trascrizione è incomprensibile; per la directional è il *lux*);
- il **Light Color**: una ruota dei colori per il colore della luce, la sua saturazione, "quanto sarà scura, magari quanto sarà livida". Se si vuole che più luci abbiano lo stesso colore, si copia il codice **Hex Linear** e lo si incolla nelle altre, "il modo più comodo piuttosto che cercare a mano di replicare la stessa luce": per esempio un corridoio in cui tutte le torce sono "molto calde, molto accese, molto infuocate";
- la **Temperature** (con *Use Temperature*), per una luce "tendenzialmente più calda o più fredda"; "così, se volete fare *Breaking Bad* quando sono in Messico, tutto giallo";
- **Affects World**: "questa luce che sto utilizzando ha effetto sul mondo? È una luce attiva o disabilitata?". Serve, per esempio, a vedere "come contribuisce alla scena" una luce alla volta;
- **Cast Shadows**: la luce illumina comunque, ma proietta le ombre solo se è attivo;
- la **Dynamic Shadow Distance**: secondo il docente, "quanto precisamente viene calcolata l'ombra, quanto è netta". "Non può essere notato adesso perché stiamo utilizzando solo geometrie"; con asset veri un valore basso si nota. Il docente cerca nel progetto una mappa già arredata per mostrarlo ("mi sa che non ne abbiamo, a parte quelle che ha fatto Igor") e rimanda: "la prossima volta mi preparo una scenetta carina a livello estetico". È un parametro che "tende ad appesantire un pochino le scene", ma "l'effetto che fa è talmente meglio rispetto a non averlo": "giocatevi un po' di potenza in più".

Una sola per scena. "Non dovete mai avere **due directional** nella stessa scena": l'engine prova a calcolarle entrambe, "fa molta fatica" e "il risultato è tremendo", con problemi di ombre calcolate in direzioni diverse. Allo stesso modo "evitate di avere due sky sphere", anzi, si corregge, due **sky light**: "skylight e directional light, una soltanto per scena".

La Sky Light. "Funziona in maniera un po' diversa", "un po' più complessa", e "al momento non ci serve troppo indagarla". Che cosa fa: "è la luce che dà il **mood**, la diffusione", "va a colorare l'aria, va a dare colore alla scena". Si vede cambiando il colore della sky light: cambiano di colore le *ombre*. Per una scena al tramonto, "piuttosto che avere solo la directional che è arancione", si media fra directional e sky light, "più tutta una serie di altre cose, perché utilizzeremo anche il post process" (la volta successiva). Fra i parametri, oltre all'intensità, quelli dell'**ambient occlusion**: la distanza massima (*Occlusion Max Distance*), il contrasto (*Contrast*) e "quanto è moltiplicata" (*Occlusion Exponent* e *Min Occlusion*); e anche qui Cast Shadows e Affects World. "Principalmente, mood della scena e ambient occlusion."

Approfondimento: Dynamic Shadow Distance, e due directional. Il nome completo del parametro è *Dynamic Shadow Distance Movable Light* (o *Stationary Light*, a seconda della mobility): è la distanza dalla camera entro cui la directional disegna le ombre dinamiche, divise in *cascate* (*Cascaded Shadow Maps*) di risoluzione decrescente. Non è quindi la "nettezza" in sé, ma lo spazio su cui la risoluzione delle ombre viene distribuita: oltre quella distanza le ombre dinamiche spariscono (restano solo quelle precalcolate), e allungarla, a parità di cascate, rende le ombre vicine meno dettagliate. In Unreal Engine 5, con le *Virtual Shadow Maps* attive di default, questo compromesso pesa molto meno. Sulle due directional, la regola del docente è la pratica comune; a rigore Unreal ne supporta due solo come sole e luna dell'atmosfera (indice *Atmosphere Sun Light Index* 0 e 1), un caso particolare. Funzionamento generale dell'engine.

1.13.14 Point light e spot light

"Passando a delle luci con cui andrete a smanettare molto ma molto di più": le luci "antropiche", quelle che si useranno "per fare quasi tutto", **point light** e **spot light**. "Fra le due, qual è più leggera, più facile da calcolare, indipendentemente dalla mobility?"

La domanda. Uno studente risponde la point light, "perché non si va a calcolare anche la direzione". "È esattamente il contrario." La point light è a 360 gradi: "lui calcola contemporaneamente tutte le direzioni". La **spot light** è più leggera "perché non va a calcolare i rimbalzi della luce nelle altre direzioni": "la discriminante è quella della direzione". In una scena affollata, la spot illumina solo dove la si punta, mentre la point "va posizionata con attenzione perché va a colpire anche gli oggetti che ha alle spalle", "indiscriminatamente dalla direzione".

L'Attenuation Radius. Per la point light, "i due parametri più importanti sono l'intensità e l'**Attenuation Radius**", la sfera che si vede attorno alla luce. "Indipendentemente dal fatto che io ho messo 10.000 miliardi di intensità, se il raggio non tocca nulla la luce non illumina" (o meglio, illumina solo dentro la sfera, "quindi non rimbalza su niente"). È "il raggio massimo entro cui la luce avrà effetto", e gestisce un'attenuazione dalla sorgente fino al bordo: il *gradiente* della luce. Con un'intensità altissima, per un'attenuazione sensata bisogna ingrandire molto il raggio. Ma anche con un'intensità bassa l'attenuazione arriva fino al bordo, e questo è il punto: il raggio è "il parametro che va gestito con attenzione", "perché determina con quanti oggetti la luce andrà a compenetrare", e quindi quante ombre dovrà proiettare. "Anche se l'intensità è molto bassa, nel momento in cui questo muro è all'interno del raggio, l'engine dovrà calcolare" l'ombra e la luce su quella superficie.

La X rossa. "Una cosa che vi capiterà sicuramente": una luce con una **X rossa** sopra. Il docente la riproduce mettendo più luci vicine (al primo tentativo "non abbastanza": i raggi erano piccoli). Vuol dire che "l'engine ha rinunciato a far proiettare ombre a questa luce": la luce illumina comunque, "ma non proietta ombre perché ci sono troppe altre luci con cui questa luce sta compenetrando". L'engine lo segnala come problema, "non grave". I rimedi sono due: dire noi alla luce di non proiettare ombre (Cast Shadows disattivato), così l'engine "non cercherà costantemente di proiettarle"; oppure **ridurre il raggio** "a tal punto da non farlo compenetrare con troppe cose". "Non riempite la vostra scena di luci, piuttosto utilizzate **poche luci molto significative**."

Il corridoio. L'esempio: un corridoio con una fila di luci. Mentre il personaggio passa, le ombre sono corrette "fino a che non arriviamo qui vicino": lì la luce viene calcolata "come se questa non ci fosse", e l'ombra viene proiettata dalle due luci ai lati. "Vedete, qui skippa, qui salta": l'ombra della luce con la X non c'è, c'è solo quella della luce "un po' più lontana", mentre altrove l'ombra è "molto più forte, molto più precisa, molto più netta". "Se replicate questo esperimento per conto vostro, vi rendete conto che è abbastanza semplice da replicare."

La sorgente: Source Radius, Soft Source Radius, Source Length. Come illuminare un corridoio "piuttosto che avere una luce per torcia, una luce per punto"? Si modifica la **sorgente**: ingrandendo il **Source Radius** e allungando la **Source Length**, la luce non parte più da un punto ma "da un cilindro, da

una capsula", ed è "più distribuita". Il raggio può anche essere zero, "se volete una linea e basta". È così che si fanno illuminazioni particolari, "tipo un locale illuminato unicamente dalla luce dei **neon**": mesh con materiali **emissive**, "che quindi già emettono una piccola luce di loro", più luci distribuite "in maniera più omogenea" in questo modo. Il **Soft Source Radius** "serve per creare un'altra lucina interna, come quando puntate una torcia verso il muro": una parte più attenuata all'esterno e una meno attenuata all'interno ("si vede meglio con la spot"; il docente non riesce a mostrarlo bene). Più sotto ci sono di nuovo Affects World e Cast Shadows, e lo *scattering* con la nebbia volumetrica (*Volumetric Scattering Intensity*), che servirà quando si userà una nebbia (la trascrizione dice "una FOD", con ogni probabilità *fog*); poi "molti, molti parametri" per le ombre (risoluzione, comportamento sulle pendenze). "Non tutti vi servono", "a meno che non vogliate fare proprio il *light artist*". ("Non l'ho detto prima": anche la point light ha il Light Color, "per fare le vostre luci da inferno, da nightclub, da *Night City*".)

La spot light. È fatta a **cono**, quindi si ruota. Intensità e Light Color come le altre; l'**Attenuation Radius**, qui, "non dipende da quanto è la sfera, ma è la lunghezza del cono", "quanto in là viene proiettata": "se sta così non colpisce il muro, così illumina questa parte del muro". Poi due angoli (figura 1.13.7):

- l'**Outer Cone Angle**, che gestisce il cono;
- l'**Inner Cone Angle**, il cono interno, "l'effetto torcia": nella parte colpita dall'inner cone "ci sarà una minore attenuazione". Con i due angoli vicini il bordo è netto; "volendo potete avere anche un cono che non attenua mai alla base, che attenua soltanto sulla distanza". Questo cono serve a determinare la *sfumatura*, "ed è anche quello che faceva l'altro nella point, ma con questo si nota meglio".

Anche qui Source Radius, Source Length e Soft Source Radius, e i parametri delle ombre.

La domanda sull'effetto accecante. Uno studente chiede se, guardando direttamente la luce, ci sia "l'effetto accecante". "No, assolutamente no." L'effetto "tipo quando guardi il sole" si crea con il **post process**, che il docente vuole far vedere "quando abbiamo una mappa che non sia soltanto delle geometrie" (con un pacchetto di mesh e mappe già fatte: "sono abbastanza sicuro che questo pacchetto dovesse avere delle mappe"). Con i parametri del post process si aggiungono **bloom** e **lens flare**. Altro esempio: i **god rays** ("god shaft"), "una luce molto forte che entra dalla finestra e c'è questa proiezione luminosa che taglia la scena". "Quella roba lì con la luce non puoi farla. Ti serve la luce, ma devi andare a manipolare il modo in cui la telecamera vede la luce con la post produzione." Per questo "la post produzione va fatta dopo le luci": "prende l'illuminazione della scena e ci aggiunge delle lenti, degli effetti". Fra le "lenti", oltre al bloom: la messa a fuoco, la distorsione, la *chromatic aberration*.

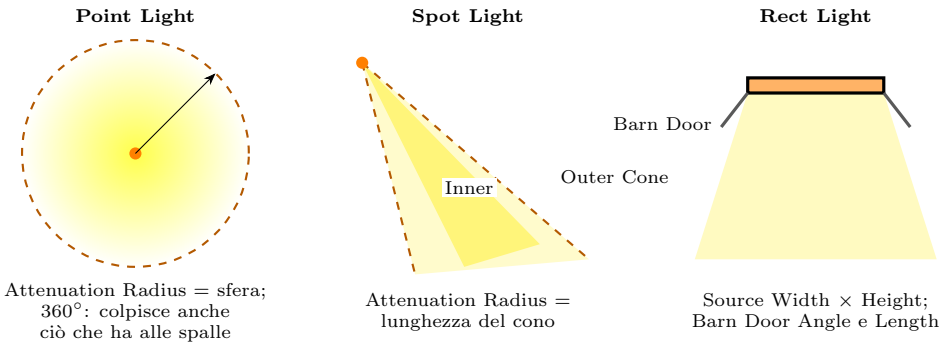


Figure 1.13.7: Le tre luci locali e i parametri che ne determinano la forma: la sfera di attenuazione della point light, i due coni della spot light (nel cono interno l'attenuazione è minore) e la sorgente rettangolare della rect light, con le "porte da stalla" che ne restringono il fascio. Schema indicativo, non in scala.

Approfondimento: la X rossa, il costo delle ombre e i god rays. La X rossa ha una causa precisa: è il limite delle luci **stationary** sovrapposte. Le ombre precalcolate delle stationary vengono scritte in quattro

canali di una texture, quindi in un punto qualsiasi del livello possono sovrapporsi al massimo **quattro** luci stationary con le ombre attive (la directional conta come una); la quinta, segnata con la X rossa, ripiega sulle ombre interamente dinamiche, più costose, o le perde (l'editor la elenca nel *Map Check*). Per questo i rimedi del docente funzionano: ridurre l'Attenuation Radius toglie la sovrapposizione; disattivare Cast Shadows toglie la luce dal conteggio; un'altra via è rendere la luce static o movable. Sul costo: la ragione per cui una spot light è più economica di una point light sta soprattutto nelle ombre, perché una spot proietta le ombre con una sola *shadow map* orientata, mentre una point light ne ha bisogno in tutte le direzioni (una *cube map* di sei facce). Sui **god rays**, la frase del docente va presa nel suo senso: in Unreal l'effetto si ottiene con le opzioni *Light Shaft Occlusion* e *Light Shaft Bloom* della directional light, o con la *Exponential Height Fog* con la nebbia volumetrica attiva; i light shaft sono comunque un effetto calcolato sull'immagine della camera, quindi della stessa natura del post process, come dice il docente. L'abbagliamento guardando il sole viene dal **bloom** e dall'**esposizione automatica** (*Eye Adaptation*), entrambi impostati in un **Post Process Volume**. Romero e Sewell (capitolo 9) ricordano che il *Post Process Volume* e il *Lightmass Importance Volume* del livello "vanno scalati per coprire l'area di gioco" quando la si allarga, "perché servono per l'illuminazione e gli effetti"; e nel capitolo 12 che la qualità del post processing ("motion blur e bloom") e delle ombre ("i dettagli delle ombre hanno spesso un impatto drammatico sulle prestazioni") è fra le impostazioni di *Engine Scalability*. Funzionamento generale dell'engine, salvo i riferimenti al libro.

1.13.15 La rect light

"Ora mi sono scordato le rect light." "Sono un po' strane": sembrano "una spotlight, ma tipo rettangolare", e invece sono diverse. Si usano principalmente per le **illuminazioni industriali**, "le classiche luci quadrate da fabbrica", o "molto più spesso" per i **riflessi**: "il riflesso della televisione su un pavimento". Vanno usate "soltanto su oggetti rettangolari" che proiettano una luce specifica, "come un'insegna al neon, di quelle dei locali". "Se devo farmi calcolare dei riflessi, lo faccio utilizzando la rect light perché dà dei risultati più precisi."

I parametri "non sono troppo diversi": intensità, colore, Attenuation Radius "che ormai abbiamo imparato a conoscere", la larghezza e l'altezza della sorgente (*Source Width*, *Source Height*). Ciò che cambia:

- il **Barn Door Angle**: la sorgente è un rettangolo centrale "più una struttura lungo il rettangolo", che Unreal chiama *barn door* ("come si traduce?", chiede il docente; "la stalla", risponde uno studente: le alette dei proiettori da set, dette appunto *porte da stalla*). Cambiando l'angolo è "come se ci fosse un'ostruzione": le alette si piegano e la luce va solo in avanti; al massimo, la luce arriva fino a terra; nel mezzo, un effetto sfumato;

- la **Barn Door Length**, "quanto è grande questa lente", per "un effetto tipo i riflettori degli stadi", che illuminano "una parte specifica, molto tagliata, molto netta";

- la **Source Texture**: "la texture che noi andiamo a inserire diventa il tipo di luce che viene messo", come un'insegna o un televisore che proietta una luce particolare; "è come se andasse a proiettare la luce attraverso un filtro, per spiegarlo in modo brutto". Il docente cerca una texture riconoscibile nel progetto ("queste non vanno bene") senza trovarla, e l'esempio resta a parole;

- anche qui la Temperature e Affects World.

1.13.16 Il build delle luci e i riflessi

"Stavo per scordare una delle parti fondamentali." La differenza fra static, stationary e movable si vedeva "molto meglio sulla quarta edizione"; ma anche sulla quinta "dovete stare attenti": "vi capiterà di fare dei livelli in cui avete un'illuminazione, poi andate a buildare e la luce che vi ritrovate alla fine non è la luce che pensavate di avere". Questo perché le luci **statiche e stazionarie** hanno bisogno di essere **buildate**.

La qualità del build. Quando si fa il build bisogna scegliere la qualità con cui calcolare le luci (*Lighting Quality*): di default è **Preview**, "la qualità più veloce", in cui "vengono calcolati meno rimbalzi"; quando si pubblica un gioco si usa **Production**, il livello più alto, che viene calcolato al build e "rimane bakeato sul livello". Se non si fa il build, le luci statiche e stazionarie "rimangono delle preview".

La "bestemmia". "Oppure utilizzate questa cosa che sto per dire, sarà una bestemmia: utilizzate **solo luci movable**." Le movable sono pesanti, "quindi non utilizzate solo luci movable, chiaramente"; ma per un livello *molto piccolo*, "in cui avete 2–3 luci massimo", e che serve in fretta, avere luci movable "che sono esattamente come le vedete alla fine" può tornare comodo. "Se dovete illuminare un livello con 20 e passa luci, non mettete solo luci movable": mettete luci static e stationary e fate il build.

Il menu Build. Si possono buildare diverse cose, "pezzo per pezzo": le luci, le **reflection capture** ("ecco che mi mancava, i riflessi"), la visibilità statica (*Precompute Static Visibility*), il **landscape** ("che poi vedremo la prossima volta"), e così via; **Build All Levels** ribuilda tutto. "Con la quinta edizione, tutto sommato, ve la cavate": in Unreal 4, senza il build, ci sarebbero scritte **Preview** sparse per la scena e in alto **Lighting needs to be rebuilt**. "Se mai vi capita di lavorare su una versione precedente di Unreal, sappiate che quella scritta in alto è perché non avete buildato le luci". In ogni caso: "quando avete finito il livello fate sempre un build generale".

Le Reflection Capture. I riflessi "sono abbastanza scollegati dall'illuminazione, ma funzionano in maniera simile": anche loro sono **volumi** che influenzano zone specifiche e vanno buildati. Si trovano fra i volumi: **Box Reflection Capture**, **Planar Reflection** e **Sphere Reflection Capture**. Prendendo la sfera, "semplice da vedere": ha un raggio di influenza (di default 3000), che si ingrandisce e si riduce, e vi si possono assegnare mappe specifiche. Quando si fa il build, "catturano tutto quello che c'è all'interno della sfera e calcolano i riflessi sulle superfici riflettenti".

Quali superfici riflettono? "Perché anche se io buildo qui adesso lui non mi farà vedere niente di diverso?" Il docente rimanda alla lezione sui materiali: uno studente ricorda i parametri "quanto rifletto la luce, l'opacità". "Esatto": i parametri che contano sono **Metallic**, **Specular** e **Roughness**, "soprattutto Specular e Roughness", che determinano quanto il materiale sarà riflettente. I materiali della scena di prova non lo sono; con una roughness "particolarmente bassa" e una specular "particolarmente alta", tutti gli oggetti nell'area di cattura verrebbero riflessi sulle superfici capaci di rifletterli: "le pozzanghere, i pavimenti lucidi, gli specchi". "Magari ci ritorniamo nel momento in cui abbiamo un ambiente più costruito, che non sia solo blocchi."

Approfondimento: il build in Unreal Engine 5. Il motivo per cui "con la quinta ve la cavate" è **Lumen**: nei progetti nuovi di Unreal Engine 5 l'illuminazione globale e i riflessi sono calcolati in tempo reale, e una scena illuminata da luci movable (o da stationary e static non ancora buildate) appare già come apparirà in gioco; i livelli dei template, inoltre, partono spesso senza illuminazione precalcolata. Il build con Lightmass serve quando si sceglie l'illuminazione precalcolata, di solito per ragioni di prestazioni (piattaforme mobili, VR, scene molto statiche), e in quel caso valgono tutte le avvertenze del docente: il build a qualità Production prima di pubblicare, il *Lightmass Importance Volume* attorno all'area di gioco, e il controllo del messaggio "Lighting needs to be rebuilt" (in Unreal 5 segnalato in modo meno vistoso). Anche le *Reflection Capture* perdono importanza con i riflessi di Lumen, ma restano il ripiego quando Lumen è disattivato. La Sphere Reflection Capture ha di default un *Influence Radius* di 3000 unità (30 metri), il valore letto a lezione. Funzionamento generale dell'engine; il libro di Romero e Sewell non tratta il build delle luci.

1.13.17 I compiti per martedì e la chiusura

"Abbiamo una decina di minuti." Le luci, "accennate al volissimo", si rivedranno con il post process e con lo speed level, "se riesco a farvi vedere qualcosina, almeno di che cos'è concettualmente". Uno studente chiede

delle *linee guida* per il lavoro della volta successiva: "delle specie di linee guida". "Molto fumose, molto generiche, però così per darvi qualcosina":

1. una **gym** in cui si sono calcolati i parametri del proprio personaggio "che ritenete importanti per il gameplay". La Max Sprint Distance da 26 m serve solo se nel gioco ci sono cose a tempo: "se voi volete fare soltanto un platform, magari questa cosa non vi serve". Pensate "a seconda del tipo di block out che volete fare" e misurate quanto in alto salta, quanto velocemente si muove, quanto lontano salta, le differenze fra lo sprint e il salto, magari con una gravità modificata; le misure vanno tenute "da una parte, all'interno del livello in cui andrete a fare il block out";

2. **smanettare con i parametri del bot**, BP_Bot2, "in modo da avere un tipo di esperienza che magari funziona meglio per il vostro gioco". Esempio in diretta: il **Walkable Floor Angle**, "44" di default. Il docente fa delle rampe di pendenze diverse: su alcune il bot sale, su una più ripida "lo fa scivolare giù". Così si possono fare "delle parti in cui il vostro robot non può andare, che non sono semplicemente dei muri". O una gravità "meno lunare"; o una camminata più veloce: "io trovo che la camminata di base del robot, per i miei gusti, sia un po' lenta, quindi magari l'andrei a modificare". E, "una cosa carina", il **margine di manovra e di errore**: il bot "salta e non si muove a mezz'aria, ha un pessimo controllo a mezz'aria"; con l'**Air Control** a 1 "si muove come se non fosse in aria". "Quanta libertà di movimento voglio dare al mio giocatore? Per quale motivo?" La gym serve anche a rispondere a domande come: "so che il mio personaggio salta 4 metri in avanti e 2 in alto: posso fare questo salto? E se posso farlo, è comodo farlo?";

3. un **piccolo block out** in cui l'obiettivo è prendere da **tre a cinque collezionabili**, "ognuno ottenuto in una maniera diversa": esplorando, saltellando, usando gli oggetti delle lezioni precedenti, "schivando i colpi della mitraglietta", "quello che volete". Serve a familiarizzare "con l'idea di fare delle piccole sfide che il giocatore deve superare per raggiungere degli obiettivi".

Alla lezione successiva il docente chiederà che cosa è stato modificato e perché: "poi magari ci facciamo una chiacchiera sul perché avete deciso di dare quel tipo di parametro al robottino e cosa volevate farci, che è l'inizio del design: iniziare a pensare a questi numeri, in che modo influenzano l'esperienza del giocatore".

Per chi ha problemi di surriscaldamento: "provateci", "stiamo lavorando principalmente con dei blocchi", "non c'è nulla di pesante da un punto di vista grafico"; "le cose pesanti verranno più avanti, quando cominceremo a utilizzare elementi estetici". "Penso di essermi mangiato le mie parole tipo infinite volte, però di averle dette talmente tante volte che era chiaro il concetto." Nessuna domanda. "Ci becchiamo martedì prossimo": per qualsiasi cosa, il **Discord**, anche per mostrare il proprio lavoro.

Approfondimento: il compito visto dai libri. Il compito mette insieme i temi della giornata nell'ordine in cui Totten li presenta: prima le metriche (la gym), poi il whiteblocking (il block out), poi le sfide costruite sulle metriche e sulle abilità del personaggio. Due criteri dal suo capitolo 2 sono utili come controllo: ogni sfida dovrebbe usare una metrica misurata (un salto vicino al limite, una distanza vicina alla Max Sprint Distance, una pendenza oltre il Walkable Floor Angle) e dovrebbe essere *leggibile*, cioè comunicare al primo sguardo se e come si supera, come il blocco da 1,40 m della lezione. Romero e Sewell (capitolo 13) mostrano come trovare e piazzare rapidamente actor di supporto dal *Place Actors Panel* (lo stesso pannello usato a lezione per le brush), comodo per disseminare il livello di collezionabili e punti di riferimento.

Punti chiave del capitolo

- Le ultime quattro lezioni sono di **level design**: teoria di base, block out e geometry brush, Landscape e Foliage Mode, luci, post process, dressing e, alla fine, piccoli livelli da condividere come eseguibili sul Discord.

- Ci si ispira alla **realtà**, non alla *fiction*: la mappa di un altro gioco è già filtrata dal suo game design. Il **level design** nasce dal **game design** e dal modo in cui si muove il personaggio.

Misura o valore	Valore	Dove e a che serve
Max Sprint Distance	26 m	la corsa con una barra di stamina; limite per inseguimenti e distanze fra ricariche
Margine sullo sprint	circa 20 m	dove mettere una ricarica di stamina
Walk Jump / Sprint Jump	4 m / 6 m	la lunghezza dei salti del bot con Jump Z Velocity modificata
Limiti per i salti di un platform	circa 2,20–2,30 m in alto, 5,80 m in lungo	le sfide del livello, sotto i massimi
Blocchi d'altezza	1; 1,40; 1,60; 1,80; 2 m	il feeling del salto; 1 m superato "male", 1,40 m no
Max Step Height	45 cm (provato con 40 cm e 1 m)	il gradino che si sale camminando
Walkable Floor Angle	"44" (44,77°)	la pendenza oltre cui il bot scivola
Jump Z Velocity	420 → 700 (poi 600–650)	l'altezza del salto: con 700 oltre i 2 m
Air Control	fino a 1	il controllo in aria (1 = come a terra)
Porte nella slide della gym	4×4, 2×4, 2×3, 2×2,5, 2×2 m	l'esperienza di passarci, con la camera in terza persona
Max comfortable wall run	12 m (discesa da 13–14 m)	la slide della gym
Griglia del pavimento	1 m, quadratini da 20 cm	leggere le misure; snapping a 10 cm
Box di prova	600; Wall Thickness 10 → 50	la casetta con le brush
Sides del Cone	da 3 a 500	la precisione del cono
Sphere Reflection Capture	raggio 3000	la zona di cattura dei riflessi

Table 1.13.3: Le misure e i valori della giornata. I valori del salto e della corsa sono quelli trovati in diretta dal docente, che li definisce "molto non calcolati": il compito è trovare i propri.

Sintomo	Causa e rimedio
Il bot supera il blocco da 1 m "scivolando"	la capsula sale sullo spigolo: ostacolo poco leggibile; alzarlo (1,40 m è chiaramente insuperabile) o renderlo chiaramente superabile
Il bot sale su un ostacolo da 40 cm senza saltare	è sotto la Max Step Height (45 cm): abbassarla se non va bene
Max Step Height a 1 m non fa salire il bot	salvare e compilare; poi restano i limiti della capsula e degli altri parametri (non risolto a lezione)
Cambiare la Jump Z Velocity non ha effetto	si sta modificando il Blueprint sbagliato: il personaggio in gioco è BP_Bot2
Il materiale della brush si deforma	la brush è stata scalata: usare X, Y, Z dei Brush Settings
Il materiale va su una faccia sola	le brush hanno un materiale per faccia: assegnarlo a ogni faccia
Inner Radius non modificabile	il cono (o il cilindro) non è Hollow
Tessellated non ha effetto	la box è Hollow
Le modifiche del Brush Editing spariscono	si è cambiato un parametro dei Brush Settings, che ricostruisce la forma
La scena con molte brush è pesante	Create Static Mesh, poi eliminare la brush; mettere le mesh in una cartella Blockout Assets; sistemare il pivot
Spostare la directional non cambia nulla	è un fascio di rette parallele: va ruotata
Ombre sbagliate con due directional (o due sky light)	una sola per scena
Una luce illumina poco nonostante l'intensità	l'Attenuation Radius non raggiunge le superfici
X rossa su una luce, niente ombre	troppe luci (stationary) sovrapposte: ridurre il raggio o disattivare Cast Shadows
Nel corridoio l'ombra "salta"	la luce con la X non proietta ombre: come sopra; o una sola luce allungata (Source Length)
La luce finale non è quella vista nell'editor	luci static e stationary non buildate: Build, qualità Production
Scritte Preview e "Lighting needs to be rebuilt" (Unreal 4)	build delle luci mancante
Le Reflection Capture non mostrano nulla	i materiali non sono riflettenti: Roughness bassa, Specular alta
Il tasto destro non funziona durante la condivisione	problema di Zoom, non risolto

Table 1.13.4: I problemi incontrati durante la lezione, con la causa e il rimedio.

Strumento, actor o impostazione	**Che cosa fa**
Level design document	descrive il livello in modo chiaro, né generico né costrittivo
Layout (piantina)	la mappa 2D del livello, con legenda; si sbaglia senza costi
Gym (test level)	il livello in cui si misurano le metriche del personaggio
Block out	il livello rifatto con sole geometrie, per testare il gameplay
Landmark	oggetto riconoscibile che dà identità e orienta
Snapping della griglia	sposta gli actor a passi fissi (10 cm)
Text Render (approfondimento)	testo nel livello, per i cartelli delle misure
Character Movement	il componente dei parametri di movimento del personaggio
Quick Add (menu), voce Geometry	aggiunge le brush e gli altri actor
Place Actors Panel	pannello agganciabile con tutti gli actor da piazzare
Geometry Brush: Box, Cone, Cylinder, Sphere, Linear, Curved e Spiral Stair	le forme di base per il block out
Brush Type: Additive / Subtractive	aggiunge geometria; la toglie (buchi, porte, finestre)
Hollow, Wall Thickness, Tessellated	vuota; spessore dei muri; divisa in triangoli
Create Static Mesh (Brush Settings)	salva la brush come static mesh, più leggera
Brush Editing (modalità)	sposta vertici, edge e facce; Split per tagliare
Directional Light	il sole: rette parallele, conta solo la rotazione
Sky Light	il colore dell'ambiente, il mood, l'ambient occlusion
Point Light / Spot Light / Rect Light	luce in tutte le direzioni; a cono; da una sorgente rettangolare
Mobility: Static / Stationary / Movable	tutta precalcolata; fissa ma reattiva; dinamica
Intensity, Light Color, Hex Linear	l'intensità; il colore; il codice da copiare
Temperature (Use Temperature)	luce più calda o più fredda
Affects World / Cast Shadows	accende o spegne la luce; ne attiva le ombre
Dynamic Shadow Distance	la distanza entro cui la directional disegna ombre dinamiche
Attenuation Radius	il raggio (o la lunghezza del cono) entro cui la luce agisce
Source Radius / Soft Source Radius / Source Length	la dimensione della sorgente; la sfumatura; la lunghezza (luce a capsula)
Inner / Outer Cone Angle	il cono interno meno attenuato; il cono esterno
Barn Door Angle / Length, Source Texture	le alette della rect light; la texture proiettata
Volumetric Scattering Intensity	l'interazione della luce con la nebbia volumetrica
Build: Lighting Quality (Preview, Production)	la qualità del calcolo delle luci
Build All Levels, Build Reflection Captures	ricalcola tutto; solo i riflessi
Box / Sphere Reflection Capture, Planar Reflection	catturano l'ambiente per i riflessi
Metallic, Specular, Roughness (materiale)	quanto una superficie riflette
Post process: bloom, lens flare, messa a fuoco, distorsione, chromatic aberration	effetti sull'immagine della camera (la prossima lezione)

Table 1.13.5: Gli strumenti, gli actor e le impostazioni della tredicesima lezione.

- Il **level design document** è chiaro e leggibile, né troppo generico né troppo costrittivo. Prima del block out c'è il **layout**: una piantina con legenda (a quadretti per un platform) per prevedere l'esperienza e mettere il giocatore "sui binari".
- La **gym** è il livello in cui si misurano le metriche del personaggio (altezza e lunghezza dei salti, corsa, pendenze, porte, altezza da accovacciati); che cosa misurare dipende dal tipo di gioco. In terza persona le porte si fanno "falsamente alte" per non far sbattere la camera.
- Il **block out** ricostruisce il livello con sole geometrie per testare il gameplay prima dell'estetica: misure, durata, landmark, ricompense, guide come le bandiere, arene da allargare o stringere.
- In engine: pavimento a griglia e snapping per misurare; la **Max Sprint Distance** (26 m a lezione) decide inseguimenti, ricariche e distanze fra ostacoli, sempre con un margine d'errore.
- Il **feeling**: un ostacolo superato "scivolando" con la capsula è pessima comunicazione; uno chiaramente troppo alto è buona comunicazione.
- Il **Character Movement**: Gravity Scale, Mass, Max Step Height (45), Walkable Floor Angle, Max Walk Speed, **Jump Z Velocity** (420 di default, una velocità iniziale), **Air Control**, nuoto e volo. Si misurano i salti risultanti (4 m camminando, 6 m correndo a lezione) e si progettano le sfide un po' sotto i massimi.
- Le **geometry brush** servono a prototipare: materiali per faccia, dimensioni dai **Brush Settings** e non dalla scala, Hollow e Wall Thickness, **Additive** e **Subtractive** per bucare. Sono pesanti: **Create Static Mesh**, in una cartella *Blockout Assets*. Il Brush Editing modifica vertici, edge e facce, ma i Brush Settings ricostruiscono la forma.
- Le luci: **Directional** e **Sky Light** (una per scena), **Point**, **Spot** e **Rect**. La **Mobility**: Static (precalcolata, la più leggera), Stationary (fissa ma reagisce ai personaggi, il default), Movable (dinamica, la più pesante).
- L'**Attenuation Radius** decide con quante superfici la luce interagisce e quindi quanto costa; la **X rossa** segnala troppe luci sovrapposte senza ombre: ridurre il raggio o togliere Cast Shadows. Poche luci molto significative; la spot costa meno della point.
- Static e stationary vanno **buildate** (Production prima di pubblicare); solo movable va bene per livelli minuscoli. Le **Reflection Capture** vanno buildate e si vedono solo su materiali riflettenti (Roughness bassa, Specular alta). Bloom, lens flare e abbagliamento sono **post process**.
- Il compito: una gym con le metriche del proprio bot, parametri modificati e motivati, e un piccolo block out con **da tre a cinque collezionabili**, ognuno raggiunto in modo diverso.

Chapter 1.14

Il Landscape: scolpire il terreno e dipingerlo con il suo materiale

Questa è la **quattordicesima lezione** del corso breve di **introduzione a Unreal Engine**, la seconda delle ultime quattro dedicate al **level design**. La lezione precedente aveva posto le basi: il layout, la **gym** in cui si misurano le capacità del personaggio, il **block out** con le geometry brush, una panoramica sulle luci. Oggi si esce all'aperto. "Oggi vediamo un altro strumentino per creare spazi": il **Landscape Mode**, l'insieme di strumenti con cui Unreal crea e modella il **terreno** degli ambienti in esterna, e il **materiale del landscape**, che permette di *dipingere* sul terreno roccia, sabbia, ghiaia ed erba, e perfino di far crescere da solo il fogliame dove si dipinge l'erba. Il **Foliage Mode**, annunciato in apertura, non entra nel tempo della lezione e viene rimandato alla volta successiva, insieme al *dressing*, alle luci e al post process: "lascia un po' il tempo che trovo se non abbiamo almeno qualcosina in scena che possa essere simpatico da vedere".

Il docente, come nella lezione precedente, non dice mai il proprio nome. In due punti rimanda a cose spiegate "alla prima lezione che avevo fatto" (i canali delle texture) e "la volta scorsa" di "un mese fa" (la Blend Mode dei materiali): sono argomenti della **terza lezione** del corso, quella su texture, materiali e mesh, tenuta da Davide al posto di Igor. Il docente è quindi, con ancora maggiore probabilità, **Davide**; resta una deduzione del capitolo, e nel resto del testo è semplicemente "il docente". (Quando dice "la mia prima è stata la scorsa" intende la prima delle ultime quattro lezioni, non la prima in assoluto.)

La giornata ha tre parti. La prima, prima di aprire Unreal, è dedicata ai **lavori degli studenti**: Federico mostra una gym molto completa, Filippo un livello vero e proprio, e i commenti del docente toccano temi di design (la *forgiveness* dei salti, i primi salti a basso rischio, il *less is more*). La seconda è il **Landscape** vero e proprio: come si crea, com'è fatto, come funziona il pennello e che cosa fanno gli strumenti di scultura. La terza, dopo la pausa, è il **materiale del landscape**: prima quello già pronto del pacchetto StackOBot, smontato pezzo per pezzo, poi uno costruito a mano, e infine la **Visibility Mask** con cui si fanno i buchi nel terreno. In chiusura, il compito: un piccolo paesaggio.

Tre avvertenze di lettura, come per le lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. La **gym** è "Jim" e "GIM"; il **Landscape** è "l'anscape" e "Lens Trip"; lo strumento **Sculpt** è "scalp" e "scout"; **Erase** è "i raise"; il **falloff** è "fall offline"; lo *smoothing* è "smodata"; **Ramp** e il pulsante **Add Ramp** sono "la tramp" e "la tram"; **Flatten** è "flutter"; il *Paste Mode Both* è "BOF"; i **LOD** sono "l'Odd" e "i load"; l'**Outliner** è "Word Outliner" e "Word.liner"; il pacchetto **StackOBot** è "Stacobot" e "Stagopot"; la **Absolute World Position** è "absolute word position"; il nodo **Landscape Layer Blend** è anche "landscape layer lamp"; il **Landscape Layer Sample** è "Landscape Player Sample"; gli *shader* sono "scelter"; **Astroneer** è "Stroneer"; **God of War** è "Eagle of War"; il *less is more* è "l'essis more" e "l'essi smorre"; il *crouch* è "grouciarsi" e "crocciandoti"; il *foliage* è anche "il foglietto" e "il Folge"; l'*Opacity Mask* è "l'opaz di masked" e "lo pass di Mask". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: in alcuni punti il riconoscimento vocale produce testo estraneo alla lezione. In apertura ci sono frasi su "la situazione interna a livello politico" e su "gli egiziani"; durante la prova del pivot c'è un "sì" ripetuto centinaia di volte; nella spiegazione del Landscape una frase su "chat GPT" e "come funziona il nostro programma" ripetuta in loop; nella spiegazione dell'Erosion la frase "Erosion non ci può essere un problema" ripetuta nove volte;

nel Grass Type “questa cosa bisogna fare” ripetuta; e durante la pausa lo stesso lungo blocco già visto nella lezione precedente (“grazie a tutti per l’attenzione che ci ha spiegato”, “qual è la risposta che ci può essere?”). Non appartengono alla lezione e sono stati ignorati. La terza: lo schermo condiviso non è nella trascrizione. Il capitolo ricostruisce i passaggi da ciò che il docente dice mentre clicca, completa i nomi esatti di strumenti, parametri e nodi di Unreal Engine 5, distingue le prove dalle conclusioni e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l’ordine della lezione. Le figure ricostruiscono la gym di Federico, la struttura di un landscape (componenti, sezioni, quad), le forme del falloff, lo stiramento del materiale sui pendii, le modalità del Flatten, lo schema del materiale del landscape con l’erba che spawna e il collegamento della Visibility Mask. Le tabelle riassumono i parametri di creazione del landscape, il pennello, gli strumenti di scultura, i nodi del materiale, i problemi incontrati e gli strumenti della giornata. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro di Marcos Romero e Brenden Sewell, *Blueprints Visual Scripting for Unreal Engine 5*, il testo di riferimento per questa parte del libro, al libro di Christopher W. Totten, *An Architectural Approach to Level Design*, per i temi di design, e a nozioni tecniche generali su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione. Il libro di Romero e Sewell non tratta il Landscape: gli approfondimenti tecnici su terreno e materiale si basano sul funzionamento generale dell’engine, e lo dicono.

Il percorso del capitolo:

1. l’**apertura**, le “meccaniche divertenti” e il programma fino all’ultima lezione (sezione 1.14.1);
2. il **riepilogo** della lezione precedente (sezione 1.14.2);
3. la **gym di Federico** (sezione 1.14.3) e la **forgiveness** (sezione 1.14.4);
4. la domanda sul **pivot** all’angolo (sezione 1.14.5);
5. il **livello di Filippo** e il *mindset* di chi impara da solo (sezione 1.14.6);
6. le **modalità** dell’editor e che cos’è il **Landscape** (sezione 1.14.7);
7. la **creazione** di un landscape e la scheda **Manage** (sezione 1.14.8);
8. il **pennello** con il falloff e i suoi tipi (sezione 1.14.9) e la domanda sulle **caverne** (sezione 1.14.10);
9. gli **strumenti di scultura**: Sculpt, Erase, Smooth, Flatten (sezione 1.14.11), Ramp e i **LOD** (sezione 1.14.12), Erosion, Hydro, Noise, Mirror e Copy/Paste (sezione 1.14.13);
10. la **pausa** e la domanda su The Sims (sezione 1.14.14);
11. il **Paint** e il materiale del landscape di StackOBot (sezione 1.14.15), l’**erba** che spawna e le **Virtual Texture** (sezione 1.14.16), le **Layer Info** e il **Landscape Grass Type** (sezione 1.14.17);
12. il **materiale fatto a mano** (sezione 1.14.18);
13. la **Visibility Mask** e i buchi nel terreno (sezione 1.14.19);
14. l’**automaterial**, i **compiti** e la chiusura (sezione 1.14.20).

1.14.1 L’apertura: le “meccaniche divertenti” e il programma

Federico e le meccaniche. Si aspettano i “soliti 10 minuti”. Il primo a collegarsi è **Federico**: “intanto io ho fatto una gym, non ho fatto un livello”. Il motivo: “mi sembra come se mancassero delle meccaniche divertenti”. Aveva delle idee per migliorare, “per esempio delle zone dove magari prendevi un boost alla velocità o al jump”, ma non ha avuto il tempo di programmarle. Il docente vorrebbe “attaccare col pippone di design”, ma si contiene, e lo riassume così:

- "l'idea di meccanica divertente lascia un po' il tempo che trova": la singola meccanica conta poco. "Già solo il salto: quanti giochi sono stati fatti soltanto col salto?" Ci sono "10.000 modi" per tirare fuori da poche meccaniche un livello "giocabile, interessante";

- è normale che ai primi approcci si voglia "fare questo, questo e quest'altro" e ci si incarti; ma "per chi si occuperà di quell'ambiente lì", cioè del design, è "proprio di buon pomeriggio" (con ogni probabilità *di buon senso*: la frase è storpiata) cercare di tirar fuori qualcosa di interessante, "o perlomeno che possa andare a colpire almeno un tipo di giocatore", "anche con molto poco";

- chi vuole programmare altro lo faccia pure: "una trigger box che quando ci passi attraverso la velocità viene messa a x per y secondi" si fa "tranquillamente";

- però "il **less is more** è una cosa molto valida anche nel design": "piuttosto che aggiungere 10.000 meccaniche, conviene averne un paio che troviamo interessanti e che pensiamo di poter applicare in tanti modi diversi".

Il docente tornerà sul punto commentando il livello di Filippo (sezione 1.14.6), costruito proprio con "una meccanica per livellino".

Il programma. "Oggi vediamo un altro strumentino per creare spazi": un paio di modalità, il **Landscape Mode** e il **Foliage Mode**, "così iniziamo ad usare anche qualche actor estetico in più". Il docente vorrebbe parlare di **dressing** e *enrichment* (la parte estetica), di **luci** e di **post process**, ma senza "qualcosina in scena" da guardare non avrebbe senso: "intanto pensiamo a questa parte, e poi l'altra la bruciamo la prossima volta che ci becchiamo, che tanto è fra due giorni".

La webcam. "Grazie Guido per la webcam": "io già di base odio fare le lezioni così da remoto perché non posso vedervi in faccia". Chi non può accenderla non deve preoccuparsi: basta esserci.

I compiti e il disco pieno. Chi altro ha "smanettato" con il compito della volta scorsa (il block out, la gym, i parametri del Character Movement)? Una studentessa voleva farlo, ma il computer segnalava il **disco C praticamente pieno** ed era lentissimo: deve ancora spostare dei file. Il docente insiste perché questi problemi si risolvano prima della settimana successiva: per l'**ultima lezione** vorrebbe "vedervi fare qualcosa" e commentare mentre si lavora, "che sia la parte più estetica o la parte più funzionale, più di struttura", perché "ci sono tante piccole cose a livello di *mindset*" che gli vengono in mente "soltanto nel momento in cui stiamo con le mani in pasta": "il concetto di *guidance*, l'*affordanza*, come costruirli in scena".

Le date e le registrazioni. Una studentessa, "un po' indietro" per lavoro, chiede quando sia l'ultima lezione e per quanto restino disponibili le registrazioni. Il docente fa "le ultime quattro lezioni": la sua prima "è stata la scorsa", poi oggi, giovedì, e "**martedì prossimo** c'è l'ultima lezione". (La trascrizione dice "oggi facciamo martedì, giovedì": il senso, con la lezione successiva "fra due giorni", è che oggi è martedì.) Sulle registrazioni non sa di preciso, "ma un sacco di tempo"; qualcuno ipotizza fino a **settembre**, "probabilmente fino a quando non cominciano i corsi" ("i corsi di IVE" nella trascrizione: il nome non è ricostruibile con certezza). Le sue lezioni sono "in parte staccate" da quelle di Igor: chi non le ha seguite vedrà "degli asset in meno, degli actor in meno", ma può seguire lo stesso, perché il docente mostrerà "principalmente dei tool, degli strumenti per fare cose". L'obiettivo resta lo stesso della volta scorsa: "una serie di eseguibili linkati sul Discord in modo che tutti quanti voi potete provare le cose che hanno fatto gli altri ragazzi".

1.14.2 Il riepilogo: layout, block out e gym

"Dieci minuti sono passati." Un piccolo riepilogo, "penso solo per **Giulia**, perché la volta scorsa non c'era". Il docente avverte che sarà "un po' sbrigativo".

Dalla realtà al layout. Per costruire un livello "si parte dalla realtà, perché la realtà è una fonte originale, non filtrata da altri sviluppatori". Si passa per dei **documenti** in cui si descrive tutto ciò che si vuole creare, e dentro questi documenti c'è il **layout**: "una piantina dell'ambiente" in cui si segnano i punti di interesse, dove spawnerà il personaggio, "che tipo di challenge dovrà affrontare e dove capitano".

Il block out. Sulla base del layout si costruisce il **block out**: "una mappa costruita unicamente in geometrie che serve per avere modo di testare gli spazi". Il game designer ha creato delle meccaniche, il programmatore (o il designer stesso) le sta prototipando, e intanto il level designer ha bisogno di uno spazio in cui capire se le meccaniche sono interessanti, se vanno "truccate" (cioè ritoccate), se serve più o meno spazio. Le misure del block out si modificano direttamente nell'engine. L'esempio della volta scorsa era il block out del livello iniziale di **God of War** (2018), "la parte in cui Kratos, dopo aver fatto il funerale della moglie, cammina insieme ad Atreus per le stradine vicino casa sua": il block out mette in mostra i **landmark**, i punti di interesse importanti, e i punti interagibili, "come delle catene che possono essere utilizzate per scendere e cambiare di livello, oppure delle chest". Anche se la chest e il loot non esistono ancora, si decide dove stanno, "così da sapere dove andremo a dare dei feedback".

Feedback positivi, hard gate e soft gate. Il docente aggiunge qui una distinzione che la volta scorsa era rimasta implicita:

- se si vuole che il giocatore esplori "tutti gli anfratti possibili", li si riempie di **feedback positivi**: "piccoli oggettini, ricompense, accenni di *lore*", per fargli capire "che è il modo giusto di continuare ad esplorare";
- se non si vuole che vada in certe zone, ci si mettono dei **gate**: gli **hard gate**, "dei muri chiusi, qualcosa che non permetta al giocatore di andare in quella direzione", o i **soft gate**, "magari dei nemici molto difficili da battere", così che il giocatore pensi: "ok, qui i fight cominciano a diventare molto impegnativi, forse è meglio se torno indietro".

"Queste sono tutte componenti di level design che chi di dovere vi spiegherà" nel corso di design.

La gym. "Che cos'è una gym? Un ambiente all'interno del quale vi andate a creare degli **ostacoli fittizi** con cui calcolare le possibilità del vostro personaggio": quanti metri corre prima di finire la stamina, quanto salta in alto e in lungo. Sulla base degli esperimenti si decidono le qualità del personaggio: "voglio un personaggio che faccia dei salti di 5 metri in lunghezza e 2 in altezza? Perché non voglio che vada troppo in alto, ma deve essere un gioco molto veloce in cui il personaggio magari salta in lungo tante volte". Si prova il *feeling*, e poi si costruiscono gli ambienti sulla base di quelle possibilità: "se io so che il mio personaggio salta 5 metri di lunghezza, non andrò mai a fare degli ostacoli che richiedono un salto di 7 metri. A meno che non ci siano dei **power up**", e allora il giocatore pensa: "questo salto è troppo lungo, ci torno più tardi". Per il block out si erano viste le **geometry brush**, da usare insieme alle static mesh "solo per la prototipazione, perché sono pesanti". Il compito era fare una gym, "smanettare" con il Character Movement del robottino (quanto salta, "su quante slope può salire", quanto controllo ha a mezz'aria) e, se possibile, un piccolo livello con le meccaniche delle lezioni di Igor.

Approfondimento: i gate nel libro di Totten. La coppia di *feedback positivi* e *gate* corrisponde a due strumenti che Totten tratta separatamente. Le ricompense disseminate negli anfratti sono una forma di rinforzo nel senso di Skinner, che Totten discute nel capitolo 4 ("Teaching in Levels through Visual Communication"): le azioni corrette vengono rinforzate perché il giocatore impari a ripeterle. I gate sono invece un modo di *controllare la circolazione*: il muro chiuso è un confine fisico, il nemico troppo forte un confine percepito. Le ricompense come oggetti da piazzare nello spazio (ricariche, chiavi, *power up*) sono il tema del capitolo 6 ("Enticing Players with Reward Spaces"). Il caso dei *power up* che rendono raggiungibile una zona prima inaccessibile è nel capitolo 8, con la mappa di *Metroid*: aree "inaccessibili senza abilità specifiche", che si aprono quando il giocatore ottiene, per esempio, "capacità di salto ampliate". È il "ci torno più tardi" del docente.

1.14.3 La gym di Federico

"Se qualcuno vuole cominciare a streamare": Federico condivide lo schermo. Ha fatto "un bel po' di bug fix" sulle meccaniche delle lezioni precedenti, "cose che non mi piacevano, le ho corrette", e poi ha costruito una gym. Il docente la commenta zona per zona (figura 1.14.1).

La torretta e l'ostacolo minimo. La prima zona serve a vedere "l'altezza per cui un ostacolo" protegge dai colpi della **torretta**: "qua non riesce a prendermi, di qua riesce a prendermi a questa distanza"; e a verificare che la torretta funzioni anche quando il bersaglio sale. Il docente la considera la prova più importante: "qual è l'**ostacolo minimo** che io posso utilizzare per non avere problemi con la torretta?" È fondamentale per una sezione **stealth**: "so che passare dietro a quegli ostacoli mi permette di non essere rilevato, o perlomeno di non essere colpito". La torretta del corso spara sempre, ma lo stesso esperimento si può fare con il suo *detect*: "se il muro è troppo alto, magari non mi detecta, non mi spara".

La piattaforma in obliquo. Il codice della pedana mobile permette di muoverla "fino a un endpoint", e Federico ha pensato che possa muoversi "anche in obliquo". Ma era difficile capire dove sarebbe arrivata, così ha aggiunto:

- un **hitbox visibile solo nell'editor** nel punto d'arrivo, da spostare per decidere la destinazione;
- una **funzione attivabile dall'editor** che disegna "una linea dall'inizio alla fine", "per far capire il movimento", perché con più piattaforme "non si capisce quale va a quale".

Il docente collega la cosa a un altro uso della gym: "io quanto in là posso far spostare questa piattaforma facendo sì che sia **intuitivo**, che quindi il giocatore riesca a capire immediatamente come funziona?"

La luce e il pulsante. "Non mi piaceva che la luce era attiva solo quando sto sopra il bottone": Federico ha fatto una copia del Blueprint in cui la luce si attiva alla pressione e resta accesa.

L'altezza del personaggio. Una serie di passaggi bassi serve a vedere "l'hitbox quanto è piacevole": prima il personaggio si bloccava a una certa altezza, ora Federico l'ha abbassata, perché "a me non dà fastidio se le antenne toccano il muro". (Il bot ha due antenne sulla testa: Federico ha accettato che attraversino il soffitto pur di far passare il personaggio in spazi più bassi, cioè ha ridotto l'altezza della capsula.)

Le porte e la telecamera. Un'altra zona riprende la slide della volta scorsa: porte di misure diverse "per vedere la cam". "Qua dà fastidio che fa così": in una porta stretta la telecamera in terza persona sbatte contro gli stipiti. Il docente legge le misure sulla griglia ("ogni quadrato di questi è un metro"): due, tre, quattro metri d'altezza; una porta **alta due metri** è "brutta per la telecamera"; quella da **quattro metri d'altezza e tre di larghezza** "è buona". Si può intervenire anche sul personaggio, "andando a gestire il braccio della telecamera" (lo *Spring Arm*): per esempio con movimenti di camera "più lenti", "più morbidi".

I gradini. Una scala di gradini serve a scegliere il valore del gradino che si sale camminando: "prima l'avevo messo a 60, ma era orribile perché *skippava* tantissimo" (il personaggio scattava verso l'alto a ogni gradino); con un valore più basso "è un po' più piacevole". Il parametro, anche se non viene nominato, è la **Max Step Height** del Character Movement (45 di default).

Le altezze di salto. "Il salto è stato scomodo": non si può impostare in metri, "perché è gestito da una forza" (in realtà una velocità iniziale: si veda l'approfondimento della lezione precedente sulla Jump Z Velocity). Federico ha quindi costruito una fila di blocchi d'altezza e ha trovato il limite. Il docente ne trae la regola per il livello:

- a **1,60 m** il personaggio "salta, ma poi slida verso sopra": ci arriva al limite, scivolando con la capsula sullo spigolo, e "non è piacevole";

- quindi, in un livello, "metterai sempre l'altezza di un salto verticale a **1,40** e non a 1,60"; non serve impedire il 1,60, "ti basta evitare quel tipo di dislivello" nel livello. "È un'informazione per il level design: adesso sai qual è un salto comodo";
- a **2 m** "è abbastanza chiaro che non ci arrivi"; a **1,80 m**, invece, il salto riesce in modo strano ("oh mio dio, questo è stato strano"): "tremendo", "tu non vuoi assolutamente avere dei dislivelli di un metro e ottanta perché questo salto così *snappy* è completamente non comprensibile dal giocatore": lo fa per sbaglio "e poi proverà a saltare su tutti i muri a caso";
- la regola: "vogliamo che un muro su cui lui possa saltare sia **subito comprensibile**": "quel muro è all'altezza del mio salto". Non si vuole che il giocatore provi, sbagli e solo allora capisca. Quindi i dislivelli del livello saranno o **1,40 m** ("ci sale su tranquillamente") o **2 m** ("ci impatta addosso e non può saltare"), e niente in mezzo: "lo rende più leggibile, più comprensibile".

È lo stesso principio della volta scorsa (il blocco da un metro superato "scivolando" era "pessima comunicazione"), ritrovato da Federico con le misure del proprio personaggio.

Pendenze e salti in lungo. Federico ha costruito rampe di diverse pendenze ("45 gradi, 50 gradi"), poi due file di piattaforme per i salti in lungo:

- il **salto in camminata**, "che al massimo riesce a fare tre se sei preciso" (tre quadrati, cioè circa tre metri);
- il **salto in corsa**, "che al massimo riesce a fare sei", "anche questo lo fallisco sempre";
- un **blocco che uccide** ("per l'occasione ho creato un blocco che mi killa"), sotto i salti fatti "al limite".

(Le misure non coincidono con quelle trovate dal docente la volta scorsa, 4 m camminando e 6 m correndo: ognuno ha modificato i parametri del proprio bot, ed è proprio lo scopo della gym.)

Le variabili esposte. "Un'altra cosa che ho fatto, perché mi risultava scomodo, è far uscire delle variabili": nel Blueprint che fa cambiare livello e nel **generatore di blocchi** Federico ha reso modificabili dall'editor alcuni parametri (quale livello caricare; quanti blocchi generare e la distanza fra un blocco e l'altro). "È chiaro: ti sei fatto dei parametri in modo che vai a modificarli senza dover entrare nel Blueprint".

Il giudizio. "Non hai fatto un livello, però è una gym estremamente più completa di quanto io mi aspettavo voi voleste fare": "una gym molto comoda", che "ti permetterà di testare un sacco di cose".

Approfondimento: variabili esposte, Call In Editor e Construction Script. I due accorgimenti di Federico hanno un nome preciso in Unreal. Una variabile che si modifica dal pannello Details di ogni copia del Blueprint piazzata nel livello è una variabile **Instance Editable** (l'icona dell'occhio accanto alla variabile): Romero e Sewell la presentano nel capitolo 2 ("ogni copia di questo Blueprint piazzata nel livello può memorizzare un valore diverso in questa variabile") e la usano nel capitolo 5 proprio per un oggetto che si muove, il bersaglio cilindrico con le variabili Speed, TimeToChange e *Direction* modificabili da istanza a istanza. Una variabile di tipo Vector con l'opzione **Show 3D Widget** mostra inoltre un piccolo gizmo nel viewport, che si trascina per decidere il punto d'arrivo: è il modo più semplice per ottenere il "punto da spostare" di Federico. Le funzioni e gli eventi personalizzati hanno invece l'opzione **Call In Editor**, che aggiunge al pannello Details un pulsante con cui eseguirli senza avviare il gioco (per esempio per disegnare una linea di debug dall'inizio alla fine del percorso). Infine, ciò che deve aggiornarsi da solo ogni volta che si sposta o si modifica l'actor nell'editor va nel **Construction Script**, che Romero e Sewell trattano nel capitolo 3. Un componente che deve essere visibile nell'editor ma non in gioco ha l'opzione *Hidden in Game*. Per Show 3D Widget, Call In Editor e Hidden in Game il capitolo si basa sul funzionamento generale dell'engine.

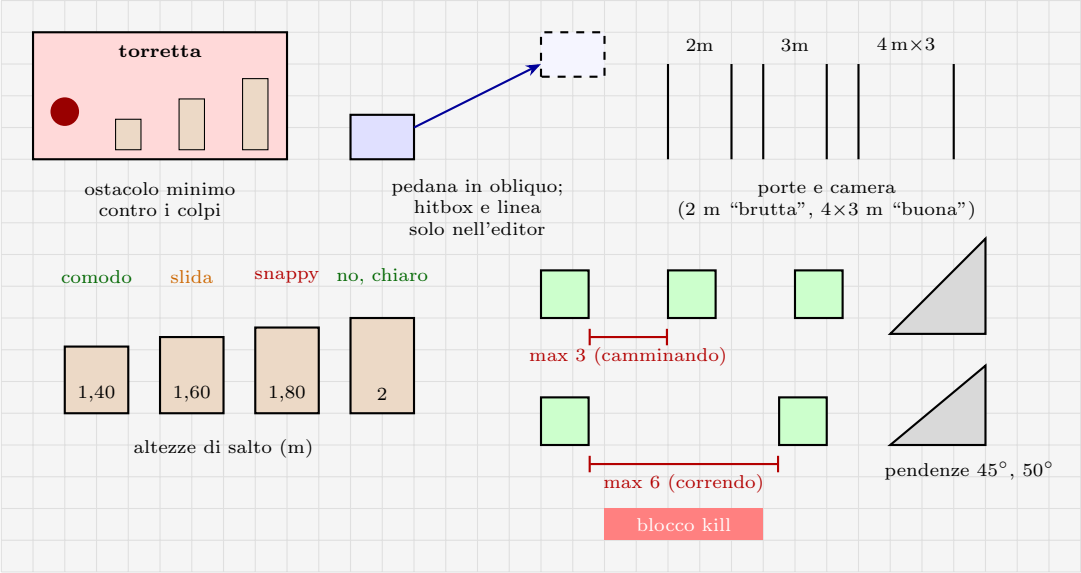


Figure 1.14.1: La gym di Federico, ricostruita per zone: la torretta con gli ostacoli di altezza crescente, la pedana che si muove in obliquo con il punto d'arrivo e la linea visibili solo nell'editor, le porte per la telecamera, i blocchi d'altezza con il giudizio del docente sul *feeling* di ciascuno, i salti in lungo camminando e correndo con il blocco che uccide, le rampe. Disposizione e proporzioni sono schematiche; misure come dette a lezione.

1.14.4 La forgiveness e i salti al limite

Dalle misure dei salti in lungo il docente ricava un concetto di design: la **forgiveness**. "Si basa sul fatto che le persone non sono delle macchine, non sono *tool assisted*": non faranno mai il salto ottimale, "o perlomeno, se lo faranno, lo faranno con molta difficoltà". Quindi, "a seconda dell'esperienza che tu vuoi andare a dare al giocatore", si decide quanto grande fare un salto:

- sulla **corsa** si può dare un po' di forgiveness: "mettendo leggermente meno della lunghezza" massima, il giocatore farà il salto "quasi sempre";
- sulla **camminata** bisogna stare più attenti, "perché la camminata è delicata": è raro che qualcuno faccia "il salto preciso camminando";
- un salto "in cui non sai se stai arrivando oppure no" si calibra così solo se si vuole un **platform hardcore**, "in cui magari prevedi anche un po' di *trial and error*, anche un po' di *rage*": il giocatore cade, torna giù, deve rifare dei pezzi. Il docente cita "il Waze Up e altri giochi del genere, c'è il piede": con ogni probabilità *Only Up!*, il *rage game* di arrampicata ("c'è il piede" non è ricostruibile con sicurezza). Sono giochi che si basano su "vediamo quanto posso tirare il salto prima che diventi un'esperienza estremamente frustrante".

Il parkour di Minecraft. Federico porta l'esempio del **parkour su Minecraft**: "il massimo salto che puoi fare è 4 blocchi, quindi molte volte mettevano il salto da 4 blocchi". Il docente ricorda ("non è tanto che non ci gioco") una tecnica: **accovacciarsi a mezz'aria** per accorciare l'hitbox. "Crea una *forgiveness mechanic* incorporata": se la capsula, saltando, "non arriva ai 5 metri, ma arriva a 4,90", accorciandola a mezz'aria "arriverò anche effettivamente più in là". È "una meccanica di **mastery**" che il giocatore usa "con un pulsante". Lo stesso trucco si fa spesso dal lato dello sviluppo, senza che il giocatore lo sappia: "mentre siamo a mezz'aria, rimpiccioliamo leggermente le collisioni del personaggio in modo che è più semplice arrivare sul bordo". "Queste sono discussioni che si fanno quando si hanno i livelli davanti, per capire che tipo di esperienza si vuole dare." (Il docente ammette: "mi sa che mi sono mangiato un po' di parole".)

Una gym da platform. Per chi segue da casa, e per Giulia, la gym di Federico è "un ottimo modo per strutturarsi una gym per un gioco platform" con pedane mobili, una torretta da evitare e salti precisi: "abbiamo tutta una serie di informazioni adesso che potremmo andare a utilizzare in un livello più avanti".

Approfondimento: forgiveness nella pratica. Il trucco descritto dal docente, rimpicciolire le collisioni in aria, è uno dei tanti accorgimenti di *forgiveness* dei platform. I più diffusi sono il *coyote time* (qualche frazione di secondo in cui si può ancora saltare dopo aver lasciato il bordo), il *jump buffering* (un salto premuto poco prima di toccare terra viene eseguito all'atterraggio) e la *ledge correction* (se la testa urta di poco uno spigolo, il personaggio viene spostato di lato). In Unreal il Character Movement offre già qualcosa del genere: la capsula che "sale" sugli spigoli, vista la volta scorsa, e il parametro *Perch Radius Threshold*, che decide quanto del bordo della capsula può sporgere da una sporgenza prima che il personaggio cada; le altre tecniche vanno programmate. Quanto al salto accovacciato, il caso più celebre è il *crouch jump* dei giochi su engine Source come *Half-Life*, in cui accovacciarsi in aria solleva le gambe e permette di superare bordi più alti; sul comportamento della hitbox in Minecraft la lezione resta generica e il capitolo non aggiunge dettagli. Totten, nel capitolo 2, mostra lo stesso ragionamento sulle metriche con *Super Mario Bros.*: nel livello 8-1 un fosso di 10 blocchi, "un blocco più lungo del salto in corsa di Mario di 9 blocchi", obbliga a usare un isolotto largo un blocco a metà, cioè spinge le metriche al limite di proposito. Funzionamento generale dell'engine per la parte su Unreal.

1.14.5 La domanda sul pivot all'angolo

Prima di cedere lo schermo, Federico fa una domanda pratica. Per costruire la gym ha trovato comodo il **cubo con il pivot all'angolo**, "che in questo caso faceva parte del pacchetto che ci avete dato", mentre i cubi di base di Unreal hanno il pivot "esattamente al centro". Si può avere nel *Place Actors* un cubo con il pivot all'angolo? Federico ha usato **SM_Cube**, ma anche mettendolo nei preferiti non compare dove vorrebbe: "neanche questo escamotage sono riuscito a usare".

Il docente non sa se l'engine lo permetta di default. "Sicuramente una cosa che puoi fare è avere una mesh cubo a cui tu fisicamente cambi il pivot": si prende un cubo e si imposta il pivot su un angolo "con la **modeling mode**", e in quel modo "ce l'hai sempre fisso lì". Prova poi a modificare direttamente il cubo dell'engine, e la risposta è no: "**gli engine asset non possono essere modificati**". Conviene quindi farsi un proprio cubo con il pivot cambiato e tenerlo nel Content Browser, "o appunto negli asset usati di recente".

Approfondimento: cambiare il pivot. Ci sono due modi. Il primo cambia solo l'actor piazzato: con il tasto centrale del mouse sul gizmo (o con il tasto destro sull'actor, *Pivot* → *Set Pivot Offset Here*) si sposta il pivot di quella istanza, ma la mesh non cambia. Il secondo, quello suggerito dal docente, cambia la mesh: nel **Modeling Mode** di Unreal Engine 5 lo strumento *Edit Pivot* (nella categoria *XForm*) permette di spostare il pivot su un angolo, un lato o il centro del *bounding box* e di scriverlo nell'asset. Va fatto su una *copia* del cubo salvata nel proprio progetto, perché il contenuto della cartella *Engine* è condiviso fra tutti i progetti e, nella versione installata dal launcher, non si può salvare. Il cubo di StackOBot, che ha già il pivot all'angolo, è comodo per il block out perché con lo snapping della griglia i suoi spigoli cadono esattamente sui metri. Il libro di Romero e Sewell non tratta il Modeling Mode; funzionamento generale dell'engine.

1.14.6 Il livello di Filippo e il mindset di chi impara da solo

"Io più che altro penso di aver fatto l'opposto di Federico": **Filippo** ha fatto "direttamente un livello e non la gym", un po' perché non aveva capito a che cosa servisse la gym. Il docente lo rassicura: "io l'ho buttata sul più largo possibile", "fate quello che riuscite a fare". La gym "non è necessaria, ma serve per raccogliere informazioni", e con quelle informazioni "riesci a creare le cose in maniera un po' più oculata". Filippo ha lavorato "in modo un po' empirico": se un salto non andava, lo modificava, e pensava di segnarsi i dati finali "su un muro", come suggerito la volta scorsa.

Quando serve una gym. La risposta del docente è una regola di produzione:

- per un livello singolo, "perché ti viene data una task" o "perché vuoi fare qualcosa di molto rapido", il metodo empirico va benissimo: ci si segnano le informazioni direttamente sulla task;
- ma quando si lavora a un gioco grande, con "4, 5, 6 livelli" e "tanti momenti di gameplay separati", avere tutte le informazioni "racchiuse in un'unica gym" è più comodo che "segnarteli passo passo";
- vale anche per un personaggio che si usa spesso: "fatti una gym, settala di tutto quanto, e poi fai un po' di lavoro prima per fare meno lavoro dopo".

Il livello. Il livello di Filippo sfrutta "un po' tutto quello che abbiamo imparato in queste lezioni". Filippo mostra solo alcune parti ("se no ci mettiamo due ore"):

- un hub con una **porta** in cui si consegnano dei **collezionabili**: messi tutti e cinque, il livello finisce. La parte più difficile è stata "il fatto che una volta che ho abbastanza collezionabili non entrassi più" in una zona e che ne servisse "un tot per entrare nell'altra", e soprattutto "mantenere in memoria sia le monete che avevo raccolto che quelle che erano già state messe nella porta";
- una struttura fatta con le **brush** della lezione precedente, con dentro una zona a **gravità ridotta**: "fai tutti i saltini e sopra c'è un altro collezionabile";
- un **platformer** vero e proprio con le **torrette**, orizzontali e verticali; se si cade, si viene **teletrasportati** al punto iniziale, o all'inizio dei salti orizzontali;
- una zona con le **porte** delle vecchie lezioni: "una serie di sequenze per cui se entri in quella giusta va bene, se entri in quella sbagliata ti riporta all'inizio";
- un'ultima zona non finita, in cui si deve "azzeccare la **sequenza di luci**": le luci sono collegate, manca la sequenza. "Un classicone, un buon esercizio";
- il **menu** iniziale fatto con Igor e un nuovo menu per la vittoria.

I commenti di level design. Il docente nota "con piacere" che Filippo è "partito subito avvelenato": "primo salto, secca di sotto, muori". "Normalmente, quando costruisci un livello platform, devi dare prima un po' di tempo al giocatore per **ambientarsi ai salti**": i primi salti devono essere **low stake**, "se salti e sbagli, cadi giù e puoi riarrampicarti sopra per saltare nuovamente". Filippo l'aveva pensato, ma era "una task molto piccola". Il docente riconosce anche un vantaggio del suo teletrasporto: nella sezione di platforming "ti faceva rifare un salto soltanto alla fine", e il ritorno al checkpoint, "poco grazioso", "funziona bene perché non c'è un peso effettivo per il giocatore": "non è che sei morto e devi ricominciare", devi solo rifare il salto. Il giudizio: "è vediamo cosa posso fare con queste robe qui. Ho a disposizione una meccanica per livello, ci faccio delle cose e vedo come viene". Ed è la risposta a Federico: "vedi, il **less is more**: lui letteralmente con una meccanica per livellino comunque ha fatto qualcosa che funziona".

Il mindset. Filippo precisa che molte cose non erano state fatte a lezione: si è informato "tra video, anche il supporto dell'IA", e alcune le ha fatte "da zero", come la memoria della porta. Il docente approva senza riserve: "è il mindset corretto". "Piuttosto che aspettare che qualcuno ti dia l'informazione", è importante mettere le mani in pasta subito, perché "a un certo punto vi troverete contro il muro: ok, so come si fa tutto fino a questo punto, questa parte qui non so come si fa, mi informerò". E la frase che riassume la lezione: "il miglior sviluppatore non è quello che sa fare più cose, ma è quello che **più velocemente riesce ad adattarsi**, ad informarsi, a scoprire cose nuove per fare cose nuove". "Io tante cose non le so fare, ma so esattamente come funziona la struttura di base e so dove andare a toccare per scoprire come farlo." Anche per la lezione di oggi: "vi farò vedere come funziona la landscape mode e il materiale del landscape. È chiaro che ci sono molte altre cose che possono essere fatte; se a voi interessa, andate ad informarvi e approfondite".

Tommaso, interpellato, non aveva nulla da mostrare ("no, io non ho detto niente"). Il docente ringrazia Federico e Filippo, che hanno fatto vedere "le due parti" della volta scorsa, la gym e un block out di livelli funzionanti, e prende lo schermo.

Approfondimento: ricordare i collezionabili e i primi salti. Il problema di Filippo, ricordare che cosa è già stato raccolto e consegnato, ha in Unreal due soluzioni standard. Se l'informazione deve sopravvivere al cambio di livello durante la stessa sessione, la si tiene nella **Game Instance**, l'oggetto che esiste per tutta la durata del gioco (era stato usato nelle lezioni con Igor); se deve sopravvivere alla chiusura del gioco, la si scrive su disco con un oggetto **SaveGame**, che Romero e Sewell costruiscono nel capitolo 11 (una classe figlia di SaveGame con le variabili da conservare, salvata con *Save Game to Slot* e riletta con *Load Game from Slot*). Quanto ai primi salti *low stake*, è la stessa idea che Totten descrive nel capitolo 3 per il castello di Peach in *Super Mario 64*: uno spazio "intimo" in cui il giocatore può raggiungere molte piattaforme "senza mettere Mario a rischio", "senza fossi né nemici", e che fa da spazio di tutorial in cui sperimentare le abilità del personaggio "al proprio ritmo". Nel capitolo 4 Totten aggiunge l'esempio di *Super Meat Boy*, i cui primi livelli scompongono le meccaniche (saltare, correre, saltare sui muri) in compiti singoli ripetuti e rinforzati.

1.14.7 Le modalità dell'editor e che cos'è il Landscape

Dopo qualche problema con la condivisione ("fa di nuovo questa cosa, deve esserci qualcosa che non capisco di Zoom"), il docente invita a interromperlo: "anzi, meglio se mi chiedete cose, io vi rispondo, piuttosto che continuo a parlare di roba per i fatti miei".

Le modalità. Unreal offre diverse **modalità** di lavoro nell'editor (il menu a tendina *Modes* della barra in alto):

- la **Selection Mode**, quella usata finora: cliccare su un cubo, spostarlo, ingrandirlo. È quella con cui "piazzerete le mesh in scena, i prop, i Blueprint", cioè con cui si costruisce effettivamente il livello;
- il **Brush Editing**, visto la volta scorsa per editare le geometry brush;
- il **Landscape Mode**, argomento di oggi;
- il **Foliage Mode**, "se riusciamo"; altrimenti, la prossima volta (e sarà così).

Entrando nel Landscape Mode il docente rimpicciolisce l'Outliner, che non servirà "più di tanto".

Che cos'è il Landscape. "È lo strumento che noi utilizziamo per andare a fare le **scene in esterna**", cioè per creare il **terreno**. Tecnicamente "è un actor che non è altro che un **piano** che noi possiamo andare a modificare manualmente, facendo **terraformazione**". Il paragone: i gestionali "in cui dovete costruire il vostro parco a tema" e modificate il terreno per avere l'ambiente come lo volete: "quella roba lì, soltanto che fatta in forma di editor". È anche l'actor con cui si costruisce un **open world**: "un open world non lo facciamo mettendo tutti i quadratini a terra", ma con uno strumento che modifica la terra "come se avessimo un pennello". (Qui la trascrizione si interrompe in una frase ripetuta senza senso; ne riemerge solo la fine, "landscape secondario o andare a chiarire la zona utilizzando delle brush o delle mesh": con ogni probabilità il docente stava anticipando che le parti che il landscape non sa fare, come le grotte, si completano con un secondo landscape o con delle mesh.)

Approfondimento: un heightfield. Il landscape di Unreal è un *heightfield*: una griglia regolare di vertici, vista dall'alto, in cui ogni vertice ha una sola altezza, memorizzata in una *heightmap* a 16 bit. Da questo derivano quasi tutti i limiti che il docente incontrerà durante la lezione: il terreno non può sovrapporsi a sé stesso (niente archi, ponti o grotte fatti di landscape), le pareti verticali stirano i quad, e i buchi si possono solo "nascondere" con una maschera. In cambio, un landscape è molto più leggero di una mesh di pari estensione: è suddiviso in componenti che si caricano, si semplificano con la distanza (i LOD) e si scartano dal rendering in modo indipendente, e con il *World Partition* di Unreal Engine 5 si divide in proxy caricati a seconda della posizione del giocatore. Il libro di Romero e Sewell non tratta il Landscape; funzionamento generale dell'engine.

1.14.8 Creare un landscape: componenti, sezioni e risoluzione

Entrando nel Landscape Mode compare "questo grosso **lenzuolo verde**": è una **preview** di come sarà il landscape quando si premerà **Create**, in basso. I parametri della scheda di creazione permettono di decidere com'è fatto (figura 1.14.2 e tabella 1.14.1).

Componenti e sezioni. Il landscape è costruito da **chunk**, "che sono i componenti" (i quadrati gialli della preview), e da **sezioni**. Con le **Sections Per Component** a *2x2*, un singolo componente giallo è composto da "due per due sezioni"; riportandole a 1, "il chunk è una sezione" sola.

Grande o piccolo? Qual è la differenza fra un landscape "più grande" e uno "più piccolo"? Quello più grande "è molto più facile da modificare come vogliamo, perché ha molti più poligoni": si ottengono "curve un po' più morbide". "D'altro canto, pesa di più." (Il docente usa "grande" nel senso di *più suddiviso*: a parità di superficie, più poligoni significa più risoluzione; si veda l'approfondimento.)

La Section Size. Si decide anche di quanti quadrati è fatta una sezione: "di default è un **63x63**". Si può scendere a **31x31**, "quindi ogni singolo quadratone è fatto da 31x31 quadratini", o fino a **7x7**. Un landscape 7x7 "sarebbe molto spigoloso nel momento in cui lo andiamo a plasmare", ma può tornare utile per "una piccola sezione di terreno". "Di base, il landscape di default può andare bene."

Gli altri parametri.

- le classiche **transform**: location, rotation e scale, che per ora si lasciano come sono;
- il **numero di componenti** sulle X e sulle Y (*Number of Components*): "voglio un landscape che è molto lungo, ma non è molto largo", o il contrario. Si lascia **8x8**;
- la **risoluzione** complessiva (*Overall Resolution*), che si ricalcola da sola cambiando gli altri valori;
- il **materiale** con cui il landscape verrà creato, rimandato alla seconda parte della lezione;
- **Fill World**, "per riempire il mondo di landscape": crea un landscape gigantesco, "se avete ambienti molto, molto grandi". Non serve.

Premendo **Create** c'è "un attimino di stop", e nell'**Outliner** compare l'oggetto **Landscape**. Tornando in Selection Mode (in Landscape Mode non si evidenzia nulla) si vede che ogni quadrato è "un chunk diverso, un chunk separato".

La scheda Manage. La prima delle schede del Landscape Mode è **Manage**, "la sottofinestra in cui possiamo creare i landscape". Da qui si può:

- crearne uno nuovo (*New*) o **importarne** uno, "per esempio un landscape che avevamo in un altro livello";
- **selezionare** dei componenti (Shift + clic per deselezionarli);
- **aggiungere** componenti, "se vediamo che il landscape che abbiamo creato è troppo piccolo", e **rimuoverne**;
- modificarlo con le **spline**, "che vedremo più avanti": il docente vorrebbe parlarne "un po' nel collettivo", perché sono "uno strumento molto utile per rendere modulare e customizzare" la costruzione degli ambienti;
- con **Brush Size**, nella selezione dei componenti, il pennello diventa "un grosso quadratone" di 1x1, 2x2, 3x3, 4x4 componenti.

Più in basso ci sono i **layer**: "se qualcuno di voi ha un po' di esperienza con qualsiasi strumento tipo Photoshop, i layer funzionano esattamente nello stesso modo". Si può, per esempio, **bloccare** un layer "e far sì che non venga mai modificato da nulla", o modificare solo ciò che sta in un certo layer.

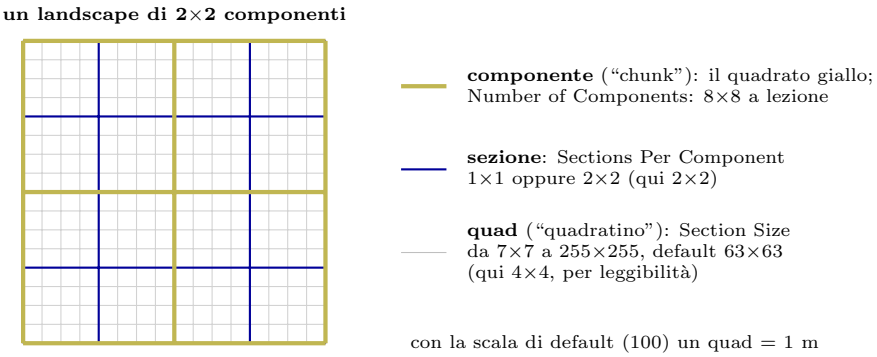


Figure 1.14.2: La struttura di un landscape. I componenti (i *chunk* gialli della preview) sono le unità che il motore gestisce separatamente; ogni componente è diviso in una o quattro sezioni; ogni sezione è una griglia di quad. Più quad significano curve più morbide e più peso. Schema con valori ridotti per leggibilità.

Parametro	**A lezione**	**Che cosa decide**
Section Size	63×63 (default); provati 31×31 e 7×7	quanti quad ha ogni sezione: più quad, terreno più morbido e più pesante
Sections Per Component	2×2, poi 1×1	in quante sezioni è diviso ogni componente
Number of Components	8×8	quanti componenti sulle X e sulle Y (landscape lungo, largo, quadrato)
Overall Resolution	ricalcolata	il numero totale di vertici per lato (deriva dai valori sopra)
Location, Rotation, Scale	invariati	la posizione e la dimensione del landscape; la scala cambia la dimensione dei quad
Material	rimandato	il materiale con cui il landscape nasce
Fill World	non usato	crea un landscape grande quanto il mondo
Edit Layers	visti nella Manage	i livelli di modifica sovrapposti, bloccabili

Table 1.14.1: I parametri di creazione del landscape visti a lezione.

Approfondimento: i numeri del landscape di default. Un quad, con la scala di default (100 su X e Y), misura un metro: è comodo, perché la griglia del terreno coincide con quella del pavimento della gym. La *Overall Resolution* è il numero di vertici per lato: con sezioni di 63 quad, una sezione per componente e 8 componenti per lato si hanno $8 \times 63 + 1 = 505$ vertici, cioè un landscape di 504 metri per lato; con 2×2 sezioni per componente il conto raddoppia ($8 \times 126 + 1 = 1009$). Le dimensioni ammesse sono quelle della forma $2^n - 1$ (7, 15, 31, 63, 127, 255) perché ogni livello di LOD dimezza i quad di una sezione. Il componente è l'unità che il motore disegna, scarta e semplifica: componenti più grandi significano meno oggetti da gestire ma un controllo più grossolano. Epic consiglia di non esagerare con il numero di componenti (qualche migliaio al massimo) e di scegliere le dimensioni fra quelle raccomandate nella documentazione. I *layer* della scheda Manage sono gli **Edit Layers**, attivi di default in Unreal Engine 5: ogni layer conserva le proprie modifiche di scultura e di pittura, si può bloccare, nascondere o cancellare senza toccare gli altri, come in un programma di grafica. Funzionamento generale dell'engine.

1.14.9 Il pennello: falloff, strength, Alpha e Pattern

Lo strumento con cui si modifica il landscape "è un cerchio, un pennello". Cliccando, il terreno si alza nel punto evidenziato: "perché si alza? Perché siamo su **Sculpt** come tool". Prima degli strumenti, però, il docente spiega il pennello, che "lascia anche questa specie di immagine residua quando passa" e ha una serie di parametri comuni a quasi tutti gli strumenti (tabella 1.14.2).

Size, Falloff e Strength.

- la **grandezza** del pennello (*Brush Size*), "quindi effettivamente quanto è largo";
- il **falloff**: "è la distanza che c'è fra il primo cerchio e il secondo cerchio". Il pennello ha due cerchi concentrici: dentro il primo l'effetto è pieno, fra il primo e il secondo sfuma. Con falloff **0** "è tutto pieno", e quando si alza qualcosa "si alza tutto in blocco"; con falloff **1** "si alza solo la punta e il resto è tutto quanto ammorbidito, è tutta sfumatura";
- la **strength** (*Tool Strength*), "la forza del tool": a 0,1 l'effetto si applica lentamente, a 1 molto più in fretta. "È quanto forte viene applicato lo strumento."

I quadratini e lo stretch del materiale. "Cosa stiamo vedendo con questi quadratini a terra?" La griglia disegnata sul landscape mostra "il modo in cui sono strutturati i poligoni" e quindi, "di conseguenza, in che modo verrà **stretchato il materiale** quando ce l'andiamo a mettere sopra". Il docente confronta tre zone (figura 1.14.4): in piano "il massimo della fedeltà, della risoluzione"; su una salita morbida un materiale "stretchato mediamente, ma rimane compatto"; su una parete alzata con falloff zero un materiale che "esplode", perché il terreno non sale "dritto per dritto verticale": c'è un quad che viene tirato in verticale "per far sì che un lato sia sul livello superiore, l'altro lato sul livello inferiore", e lì il materiale "sarà tutto rovinato".

Le forme del falloff. Il *Brush Falloff* ha quattro forme (figura 1.14.3):

- **Smooth**: "il fall off è morbido";
- **Linear** ("sharp linear"): "non è morbido, ma è lineare, è geometrico", una linea "che dritta per dritta parte dalla base e arriva verso l'alto";
- **Spherical**: "un effetto più bombato", "una sorta di collinetta" che poi "spiove verso il basso";
- **Tip**, "il fall off convesso", l'opposto dello spherical: la base è morbida e poi diventa lineare, con un effetto che ricorda "le montagne dell'isola di Drum" di *One Piece* ("Drummond Island" nella trascrizione: sono i *Drum Rockies*, montagne a colonna).

"A seconda della situazione utilizzerete lo strumento più adatto": linee morbide con lo spherical, forme più aguzze con il tip.

I Brush Type. "Di base, il brush è un semplice **cerchio**" (*Circle*). Ma ci sono altri tipi:

- l'**Alpha**: "ora il nostro pennello non è più un cerchio, è una **texture**". Si sceglie la texture dalla lista e "funziona esattamente come avevamo fatto per i materiali: ci sono degli 0 e degli 1": il pennello alza i bianchi e lascia a terra i neri. Con una texture di prova fatta di due quadrati bianchi e due neri si vede bene; il pennello **ruota** seguendo il movimento perché è attivo l'*Auto Rotate*, che si può disattivare. L'uso tipico è con le texture di **noise**, che creano "disturbo": troppo, se esagerato, ma "può esservi utile per andare a creare un effetto montagnoso, zigrinato, che dalla distanza può essere piacevole alla vista". Qui si possono cambiare solo la grandezza e poco altro;
- il **canale** della texture (*Texture Channel*), disponibile sia nell'Alpha sia nel Pattern. Il docente ripassa i canali, visti "alla prima lezione che avevo fatto": una texture ha i canali **RGB**, e i grafici li usano "per mettere più di un'informazione all'interno della stessa texture", come nelle mappe **ORM** (*ambient occlusion* nel rosso, *roughness* nel verde, *metallic* nel blu). Il pennello può usare solo il rosso, solo il verde o solo il blu; con il noise "saranno tutte uguali";
- il **Pattern**: "un portare all'estremo quello che faceva l'alfa". La texture non è più "messa su un quadrato" ma proiettata sotto un pennello circolare, "una fusione delle due precedenti", e si può modificare: la **scala** (più piccola, "molta più differenza nella costruzione"), la **rotazione**, le **U** e le **V**, "quindi il *panning*", cioè lo scorrimento della texture. L'esempio: "la faccia di Marte", la collina che sembra un volto: con una texture a forma di faccia "sembra che il terreno sia costruito in forma di faccia";
- il **Component**, "che io non ho mai usato, per essere onesto": lavora direttamente sui componenti, per esempio per alzare un singolo chunk. Utile per dividere il landscape in due zone, "una zona superiore e una zona inferiore", che il giocatore dovrà collegare "facendo X cose".

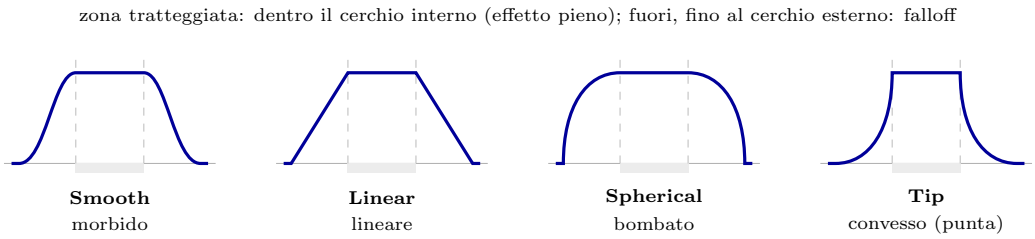


Figure 1.14.3: Le quattro forme del *Brush Falloff*, viste in sezione dopo aver alzato il terreno con lo stesso pennello. Smooth sfuma in modo morbido, Linear con una rampa dritta, Spherical con una collinetta bombata che spiove all'esterno, Tip con una base morbida e una punta ripida. Profili schematici.

Approfondimento: perché la parete "esplode". Il landscape, essendo un *heightfield*, ha i vertici a distanza fissa in pianta: alzare un punto ne cambia solo l'altezza, non la posizione orizzontale. Un dislivello di dieci metri fra due vertici distanti un metro produce un quad lungo più di dieci metri e largo uno, e la texture, mappata di default dall'alto (proiezione planare sul piano XY), vi si stira di dieci volte. I rimedi sono tre: evitare pendenze troppo ripide (lo Smooth, che il docente chiama "il miglior amico dei vostri materiali"), usare nel materiale una proiezione che tenga conto delle pareti (la proiezione *triplanare* o *world aligned*, che il materiale StackOBot usa con la Absolute World Position), oppure coprire le pareti verticali con delle mesh di roccia, come si fa quasi sempre nelle produzioni reali. Funzionamento generale dell'engine.

1.14.10 La domanda sulle caverne

"C'è qualcosa che volevi chiedere per ora? Prima dicevi delle caverne che non si possono fare." Uno studente ha provato a smanettare: è **impossibile fare dei buchi nel landscape** con questi strumenti. Il docente conferma: "per fare un buco avete bisogno dello strumento **Visibility**, della **visibility mask**". Ma la

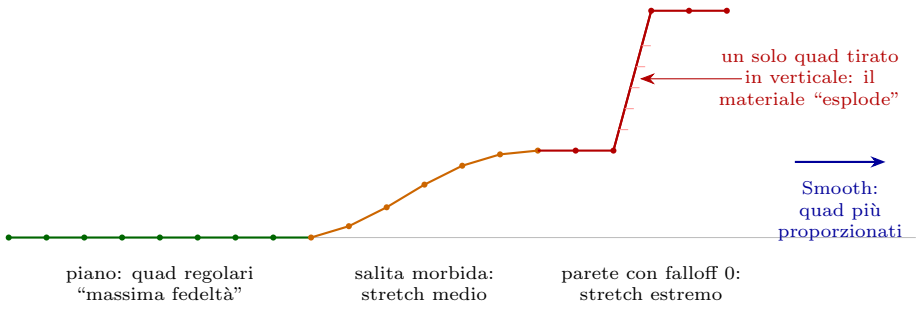


Figure 1.14.4: Lo *stretch* del materiale, visto di lato. I punti sono i vertici della griglia del landscape, a distanza fissa in pianta: in piano i quad sono regolari, su una salita morbida si allungano un po', su una parete alzata in blocco un solo quad copre tutto il dislivello e la texture si stira fino a diventare illeggibile. Lo strumento Smooth riavvicina le proporzioni. Ricostruzione schematica.

Parametro	**Che cosa fa**
Brush Size	la grandezza del pennello
Brush Falloff	la parte sfumata fra il cerchio interno e quello esterno: 0 tutto pieno, 1 tutto sfumato
Tool Strength	quanto in fretta e quanto forte si applica l'effetto (0,1 lento, 1 forte)
Falloff: Smooth, Linear, Spherical, Tip	la forma della sfumatura: morbida, lineare, bombata, convessa a punta
Brush Type: Circle	il pennello circolare di base
Brush Type: Alpha	una texture come pennello: alza i bianchi, lascia i neri; Auto Rotate lo fa ruotare con il movimento
Texture Channel	quale canale della texture usare (R, G o B)
Brush Type: Pattern	una texture proiettata sotto il pennello circolare, con scala, rotazione e panning U/V
Brush Type: Component	agisce su interi componenti

Table 1.14.2: I parametri del pennello del Landscape Mode, comuni a quasi tutti gli strumenti di scultura e di pittura.

visibility mask "si modifica dal materiale del landscape": bisogna aggiungere al materiale dei nodi specifici, e solo allora si potrà avere una parte di landscape attraversabile, "in cui possiamo andare a costruire delle caverne".

Lo studente pensava ad **Astroneer**, in cui scavare il terreno è la meccanica principale. Il docente distingue due logiche: "quando vai a creare un gioco devi far sì che l'editor per il giocatore sia più *user friendly* possibile"; "quando stai creando un editor, forse ti preoccupi di avere più possibilità possibili a disposizione". Il buco "si può fare, non si può fare da qui". In anteprima mostra lo strumento Visibility: il pennello è **rosso** e cliccando non fa nulla, perché in basso Unreal avverte che il *visibility painting* è disabilitato: il componente "non ha il nodo **Landscape Visibility Mask** nel materiale". Lo si vedrà a fine lezione (sezione 1.14.19).

Approfondimento: terreni scavabili. Giochi come Astroneer non usano un heightfield ma un terreno *volumetrico* (voxel), in cui ogni punto dello spazio sa se è pieno o vuoto, e la superficie viene ricostruita a ogni modifica: per questo vi si possono scavare tunnel e caverne in gioco. Il landscape di Unreal non funziona così, e una grotta si fa nascondendo una porzione di terreno con la Visibility e costruendo l'interno con delle mesh. Per terreni scavabili in Unreal esistono plugin di terzi basati sui voxel. Funzionamento generale dell'engine.

1.14.11 Gli strumenti di scultura: Sculpt, Erase, Smooth, Flatten

"Cosa fanno i nostri pennellozzi?" "Ognuno di voi troverà il modo con cui si trova più comodo a fare le cose"; il docente mostra solo che cosa fa ogni strumento della scheda **Sculpt** (tabella 1.14.3).

Sculpt. "Il più semplice delle cose semplici": scolpisce, alza il terreno. Ma, come si è visto "diverse volte per sbaglio", non va solo verso l'alto: **Shift + clic sinistro** "scava", e crea "delle zone interrate", delle depressioni.

Erase. Cancella: "elimina tutte le modifiche che abbiamo fatto, che siano verso l'alto o verso il basso". (Dopo la pausa il docente nota che anche l'Erase rispetta il falloff: cancella "in maniera sfumata" le parti che cadono nella sfumatura.)

Smooth. "Ammorbidisce. Immaginatevelo come uno strumento che crea sfumature." Se il pennello prende sia una parte alta sia una parte bassa, cliccando "crea una discesa più morbida": "fa una media, alza le parti abbassate, abbassa le parti un pochino più alte" e produce una *slope* più leggera. "Lo strumento Smooth è il miglior amico dei vostri materiali": ricordando che quanto sono stirati i quadratini "è anche quanto saranno stretchati i materiali", uno smoothing li rende "sempre più proporzionati". Con un pennello più grande della cima, però, "lui abbassa proprio tutto"; per i "lavori di fino" si rimpicciolisce il pennello e si fanno piccole modifiche "che non vanno a sbragare completamente tutta la parete". Anche qui ci sono falloff, tipi di pennello e strength, e un parametro in più per la **precisione** dello smoothing (il *Detail Smooth*): "più grande è questo valore, più dettagli vengono rimossi", più piccolo, "meno dettagli vengono rimossi". I "dettagli" sono "tutti questi su e giù": con un valore alto e un pennello grande "lui pialla tutto", con un valore piccolo i cambiamenti sono "molto morbidi" e "molto precisi"; "a zero chiaramente non farà mai nulla".

Flatten. "Il mio strumento preferito." Il docente si costruisce "un piccolo parco giochi" di colline per gli esempi. Il **Flatten** "ci permette di selezionare un punto e appiattire tutto quanto a quell'altezza": si clicca dove c'è la croce e tutto ciò che passa sotto il pennello viene portato a quell'altezza, sia in basso sia sulla vetta. Serve a una "terraformazione un po' più controllata":

- **plateau**, altopiani: il "lenzuolo" tirato su si interrompe al bordo, "non riscende giù";
- **scaloni**, cliccando su punti diversi di una slope: "scalone, altro scalone, altro scalone", "delle formazioni un po' più sensate, un po' più normali", da ammorbidire poi con lo Smooth. Per esempio "le cascatine fatte a livelli" o "le **risaie**", con parti piatte e parti rialzate;

- un **canyon** dentro una parte rialzata: un fondo appiattito e i lati rimasti alti.

Le modalità del Flatten. "È abbastanza brutale come strumento, perché prende e appiattisce tutto": si rischia di appiattire le cose sbagliate. Per questo c'è la **Flatten Mode** (figura 1.14.5):

- **Both** (default): "alza i piani che stanno più in basso e abbassa i piani che stanno più in alto";
- **Raise**: alza tutti i punti fino all'altezza scelta, "però non abbassa". Esempio: un rilievo che fa da **bordo della mappa** resta intatto, e si alza solo ciò che c'è sotto;
- **Lower**: abbassa "tutto quello che c'è da abbassare, ma non alza": "se c'è una grotta o un canyon che deve rimanere giù, non lo tira più su".

"Principalmente questi sono i due che vorrete utilizzare, perché sono quelli che vi danno un po' più di controllo." Di un'altra opzione il docente legge a fatica il nome ("height intervals", "specification at intervals"): "questo è uno che effettivamente non ho mai utilizzato". Si tratta, con ogni probabilità, della modalità **Terrace** (si veda l'approfondimento).

Use Slope Flatten. Con la casella **Use Slope Flatten**, invece di appiattire in orizzontale "andate a continuare il piano diagonale preso originariamente": cliccando su un pendio, lo strumento prolunga quella pendenza, "anche scavando verso il basso". Serve per "quei pianori, quegli spiazzi montani" che sono "uno spaccato dritto per dritto, che però va in diagonale". Ultimo uso, personale: "a volte il Flatten tendo a utilizzarlo tipo **gomma**": si punta sul livello base del landscape e si appiattisce.

"Abbiamo visto scolpire, tirare su, tirare giù, ammorbidire, cancellare e appiattire."

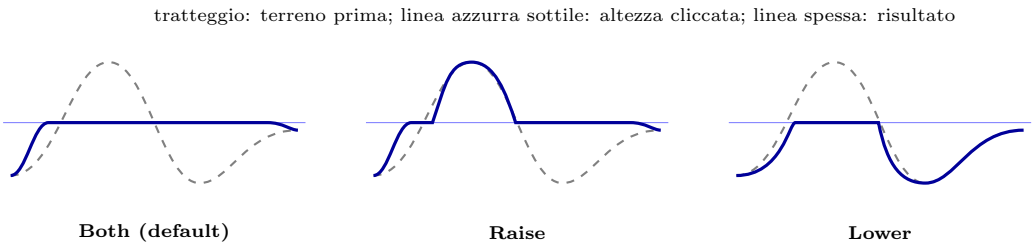


Figure 1.14.5: Le tre modalità principali del Flatten, in sezione. Both porta tutto all'altezza cliccata; Raise alza solo ciò che sta sotto e lascia la cima (per esempio il bordo della mappa); Lower abbassa solo ciò che sta sopra e lascia le valli (per esempio un canyon). Profili schematici.

Approfondimento: Terrace e gli altri parametri del Flatten. Nella versione di Unreal Engine 5 il Flatten ha quattro modalità: oltre a Both, Raise e Lower, la modalità **Terrace**, che appiattisce il terreno a *intervalli* di altezza regolari (*Terrace Interval*), con una sfumatura fra un gradino e l'altro (*Terrace Smooth*): è lo strumento giusto per le risaie e le cascate a livelli che il docente costruisce a mano cliccando su più punti. Altri parametri utili: *Pick Value Per Apply*, che ricampiona l'altezza a ogni pennellata invece di tenere quella del primo clic; *Flatten Target*, che permette di scrivere a mano l'altezza a cui appiattire (comodo per avere esattamente le misure della gym, per esempio un plateau a 1,40 m sopra un altro); e *Show Preview Grid*, che mostra il piano di destinazione. Funzionamento generale dell'engine; la lezione non li tocca.

1.14.12 Ramp e i LOD

"Adesso vediamo uno strumento che funziona in maniera un po' diversa. Non è più un pennello, ma dei clic": lo strumento **Ramp**. Si clicca su due punti, li si sposta come si vuole, e quando si è contenti si preme **Add Ramp**: Unreal aggiunge "un collegamento di landscape da punto A a punto B". Il docente chiede alla classe che cosa succederà con due punti sul fondo di due valli separate da una collina: "ti farà un canyon lì in mezzo" (la rampa porta il terreno "sempre da punto A a punto B", anche scavando).

Gli usi.

- una **rampa** per salire su un punto sopraelevato ("sarà un modo molto brutto, però va");
- le rampe di un gioco di **macchine** o di **skateboard**;
- "una **diga** in un crepaccio", un collegamento pieno da una parte all'altra.

Un **ponte sospeso**, invece, no. Si potrebbe pensare di fare la diga e poi bucarla sotto con la Visibility, "tuttavia ci sarebbe un problema": la parte sotto non collegherebbe più le due metà, "perché il landscape **non può essere sovrapposto**", "non può esserci una parte qui e una parte sotto". "Se volete fare un ponte sospeso, fatelo con le **mesh**: prendete una mesh, ingranditela e mettetela sopra un crepaccio. Non fatelo con il landscape."

Larghezza e falloff. La rampa ha la sua **larghezza** (*Ramp Width*), "se un ponticello, se una grossa salita", e il suo **falloff** (*Side Falloff*): a zero non c'è sfumatura e i lati vengono stirati "a blocco"; tutto sfumatura, e il terreno attorno si raccorda morbido. Ma se si "tira la corda", con molta sfumatura e una rampa molto stretta, "vi si spacca un po' il landscape": "questa cosa è molto rotta, e dalla distanza sarà tremenda", "perché i **LOD** vanno a semplificare il landscape".

I LOD. "Sapete che cos'è un LOD?" **LOD** sta per *Level of Detail*. Un grafico crea una mesh, poniamo da 10.000 poligoni; questa mesh deve avere "altre versioni di sé più grezze", da 5.000, da 2.000 poligoni, perché l'engine, "e non solo in automatico", aggiorna il numero di poligoni "a seconda della distanza da cui lo vedi". L'esempio della statua: da vicino deve essere "super bellissima", ma da lontano "non serve che abbia ancora 500.000 poligoni", perché l'occhio non vedrà il dettaglio, e ridurli alleggerisce le prestazioni. Le distanze si possono impostare ("a 10 metri un tot, a 20 metri un altro tot, a 40 metri un altro tot"), da un Blueprint o dalla mesh stessa. Il landscape funziona allo stesso modo: dalla distanza i suoi poligoni vengono semplificati. Una massa grande non ne soffre, "perché ci sono molti poligoni"; una rampa sottilissima, "costituita di troppi pochi poligoni, viene distrutta dai LOD". "Quindi evitate di fare delle robe in questo modo."

Lo sfogo sui Pokémon. L'esempio, dichiaratamente polemico: i recenti Pokémon open world, *Scarlatto e Violetto* e *Leggende Pokémon: Arceus*. "Potete vedere attivamente la terra che si muove, come se fosse animata": "sono semplicemente i LOD che si attivano troppo presto", il landscape "viene semplificato con troppo anticipo". Il motivo, secondo il docente, è l'ottimizzazione: se in una visione ampia si vedesse tutto con tutti i poligoni "esploderebbe la console". "Non è che sono brutte, ma è che sono troppo ottimizzate, e la troppa ottimizzazione le rende orribili"; e sono così ottimizzate "perché il gioco è programmato in maniera talmente stupida da essere pesante". "Scusate questo piccolo sfogo contro Game Freak." (È un giudizio personale del docente, non un'analisi tecnica verificata; il fenomeno del terreno che "si muove" quando cambia il LOD, il *LOD popping*, è comunque reale e ben noto.)

La regola. "Diamo sempre un po' di spazio al landscape": niente forme troppo sottili. "Questo è anche il motivo per cui vorremmo evitare dei landscape troppo piccoli": un landscape 7x7 "capita molto spesso che si spacchi in questo modo, perché ha pochi poligoni a disposizione". Il docente rifà "una rampina decente".

Approfondimento: i LOD del landscape. Il landscape non ha LOD costruiti a mano come una static mesh: li calcola da solo, dimezzando a ogni livello i quad di ogni sezione (per questo le Section Size sono della forma $2^n - 1$). La transizione è regolata da parametri dell'actor Landscape (in Unreal Engine 5 *LOD 0 Screen Size*, *LOD 0 Distribution Setting* e *LOD Distribution Setting*, più eventuali *LOD Group*), che decidono a quale dimensione sullo schermo un componente passa al livello successivo. Aumentarli rende il terreno lontano più fedele e più costoso. Le forme più sottili di una sezione, come la rampa stretta del docente, sono le prime a sparire perché nel livello semplificato non ci sono più vertici abbastanza vicini per descriverle. Il capitolo 12 di Romero e Sewell, sulla pubblicazione, ricorda che fra le impostazioni di scalabilità del gioco c'è anche la qualità del *foliage*, e che se le prestazioni calano solo in certe zone del livello conviene "ridurre la densità degli oggetti" o la qualità di un modello: il compromesso fra dettaglio e prestazioni di cui parla il docente. Per la parte sui parametri del landscape, funzionamento generale dell'engine.

1.14.13 Erosion, Hydro, Noise, Mirror e Copy/Paste

Erosion e Hydro. "Sono due strumenti con cui io faccio un po' a botte" ("a volte" nella trascrizione, ma il seguito chiarisce il senso): il docente spiega a che cosa servirebbero e perché non gli piacciono. L'idea è "simulare delle deformazioni, del *displacement*". L'**Erosion** è lo strumento con cui "andrei a dettagliare una parete montuosa": "è un po' il contrario dello smooth", crea nel pennello parti rialzate e parti abbassate. (Qui la trascrizione si interrompe in una frase ripetuta; riprende con la domanda di uno studente.)

- il **Threshold** "va a definire quanta differenza fra punto A e punto B deve esserci per andare a fare erosione": con un valore alto l'erosione si applica solo dove il dislivello è grande; abbassandolo ("tipo 17") si erode "molto di più, in molti punti diversi", perché basta una differenza minore;
- **Noise Mode** e **Noise Scale** aggiungono un rumore preimpostato, e come nel Flatten si sceglie se deve alzare e abbassare, solo alzare o solo abbassare.

L'**Hydro** (*Hydro Erosion*) "funziona più o meno nello stesso modo" e vorrebbe creare un *displacement* "stile fondali marini" o "erosione dell'acqua". Il problema è che "abbiamo un po' poco controllo": "io non sono mai riuscito a farlo funzionare bene, proprio non mi trovo".

Noise. "Preferisco piuttosto utilizzare il **Noise**", che "fa rumore" e permette di scegliere se aggiunge, se toglie o se fa entrambe le cose. Con un pennello grande e la scala del noise ridotta, "vi va a fare una deformazione un po' più subdola": "non prende e spacca, ma fa dei leggeri movimenti di alzare e abbassare", il risultato "che al mio occhio è sempre il più piacevole". In modalità solo-aggiunta il Noise alza, "però non come lo Sculpt" (aggiunge rumore, non una collina); lo stesso vale per la sottrazione.

Gli strumenti saltati. "Facciamo al volissimo le ultime cose":

- il **Retopologize** "non ci interessa";
- la **Visibility**, già vista, si riprende più avanti;
- il **Blueprint Brush**, "qualcosa che onestamente non ho mai utilizzato": "presumo sia un brush che può essere programmato" per fare cose specifiche. "Non l'ho mai trovato necessario."

Mirror. Il **Mirror** serve "nel momento in cui avete degli ambienti molto grandi che volete copiare, che volete specchiare o che volete ruotare". Compare un **piano**: premendo **Apply**, tutto ciò che sta da un lato viene copiato sull'altro, "appiattendo" quello che c'era. Il piano si sposta avanti e indietro, e soprattutto dall'**Operation** si sceglie il tipo di operazione:

- **-X to +X**: copia il lato sinistro sul destro; **+X to -X** il contrario; lo stesso per **-Y to +Y** e **+Y to -Y**. Con una forma qualsiasi si ottiene "una macchia di Rorschach";
- le varianti **Rotate** (per esempio *Rotate -X to +X*): stessa direzione del piano, "ma operazione diversa": invece di specchiare, ruota, "come se prendesse questa parte del landscape e la ruotasse nell'altra direzione". Il risultato è uguale, "ma ruotato".

Copy/Paste. "Il copia e incolla, la cosa più semplice del mondo." Prima si seleziona una parte di landscape con lo strumento **Region Select** ("Select"), dipingendo la zona con il pennello; per togliere la selezione serve **Clear Region Selection**, "perché se andate da altre parti questo vi rimane sempre selezionato". Poi, nello strumento **Copy/Paste**, **Ctrl+C** copia la zona in un riquadro (il *gizmo*) che si può spostare, ruotare e scalare dove si vuole, e **Ctrl+V** la incolla, "modificata, ruotata e anche scalata". Ci sono comandi per adattare il riquadro alla selezione e per ripulirlo. La **Paste Mode** ripete la logica del Flatten:

- **Both**: tutto il riquadro viene portato al livello copiato, la zona di destinazione viene abbassata a livello base e la parte copiata rialzata;
- **Raise**: alza soltanto "le parti che erano più basse" e non abbassa le altre: di una collina copiata sopra un'altra resta solo "la punta";

- **Lower**: l'opposto.

"Abbiamo effettivamente un po' tutte le cose viste." Nessuna domanda: pausa, "ci becchiamo alle 55 con il Paint, e quindi i materiali del landscape".

Strumento (Sculpt)	**Che cosa fa, e come lo usa il docente**
Sculpt	alza il terreno; Shift + clic scava
Erase	cancella le modifiche (rispetta il falloff)
Smooth	fa una media delle altezze; Detail Smooth decide quanto dettaglio togliere; "il miglior amico dei materiali"
Flatten	appiattisce all'altezza cliccata; Flatten Mode Both, Raise, Lower (e Terrace); Use Slope Flatten prolunga il pendio; usato anche come "gomma"
Ramp	crea una rampa fra due punti (Add Ramp), con Ramp Width e Side Falloff; non per i ponti sospesi
Erosion	dettaglia le pareti erodendo dove il dislivello supera il Threshold; Noise Mode e Noise Scale; "poco controllo"
Hydro (Hydro Erosion)	erosione "ad acqua"; il docente non lo usa
Noise	aggiunge o toglie rumore; con scala piccola, deformazioni "subdole"
Retopologize	non visto ("non ci interessa")
Visibility	nasconde porzioni di terreno; richiede la Landscape Visibility Mask nel materiale
Blueprint Brush	pennelli programmabili; mai usato dal docente
Mirror	specchia (-X to +X, ...) o ruota (Rotate ...) metà del landscape; Apply
Region Select	seleziona una zona col pennello; Clear Region Selection
Copy/Paste	Ctrl+C e Ctrl+V di una zona, con gizmo spostabile, ruotabile, scalabile; Paste Mode Both, Raise, Lower

Table 1.14.3: Gli strumenti della scheda Sculpt del Landscape Mode, nell'ordine della lezione.

Approfondimento: il terreno come spazio di gioco. Gli strumenti di scultura sono tecnici, ma le forme che producono sono categorie di level design. Totten, nel capitolo 3, osserva che in un open world fatto di paesaggi aperti, come *Skyrim*, i percorsi fra città e dungeon "sono molto meno diretti" e spesso "campi aperti": il giocatore traccia da sé le scorciatoie fra i landmark, ma "rischia di perdersi", e i designer usano "elementi geografici sottili come sentieri sterrati, cartelli o fiumi" per suggerire i percorsi. E aggiunge proprio a proposito degli "editor di terreno integrati nell'engine": tenere a mente queste linee guida serve a creare mondi "non solo esteticamente attraenti, ma anche utilizzabili". Il canyon del Flatten, la diga della Ramp e il bordo di mappa preservato dal Raise sono, in questi termini, un percorso, un collegamento e un *edge*; il plateau è un'area in cui il giocatore guarda lontano (il *prospect space* dello stesso capitolo). Le misure, infine, vengono dalla gym: un canyon che deve essere invalicabile avrà pareti oltre il Walkable Floor Angle del personaggio, un plateau raggiungibile con un salto starà sotto l'altezza comoda misurata.

1.14.14 La pausa e la domanda su The Sims

Alla ripresa ("Guido c'è, Federico c'è, Giulia c'è") il docente chiede se sia tutto chiaro: "è abbastanza semplice, è più una questione di prenderci la mano". Consiglia di usare il landscape "fra questa lezione e la prossima" per familiarizzare e trovare il proprio modo di lavorare: "magari siete dei pazzi completi e lavorate utilizzando soltanto lo Sculpt, ma siete fortissimi. Io vi dico quali sono i tool, poi voi trovate il vostro modo di lavoro".

The Sims. Uno studente chiede se il terreno della serie **The Sims**, "soprattutto del 2", sia fatto allo stesso modo. Il docente non lo sa: "non so con cosa è stato fatto". Dopo un rapido controllo: "a quanto risulta, ogni The Sims ha usato un **engine custom**". È "plausibile" che funzioni in modo simile, e che ci sia stata

"una *cross contamination*, delle influenze da entrambe le parti": magari anche Unreal si è ispirato a come "diversi tycoon o diversi simulatori" gestivano la terraformazione, perché "quel tipo di modifica spaziale è molto comodo farla con un pennello". "Non ti so dire di più": l'ultima volta che ha giocato a The Sims "probabilmente era vent'anni fa". Lo invita a fare una ricerca e a riferire la volta successiva. (Nello scambio, alla domanda "quando è uscito The Sims 2?", la risposta è "nel 2000": in realtà nel 2000 è uscito il primo *The Sims*; *The Sims 2* è del 2004.)

1.14.15 Il Paint e il materiale del landscape di StackOBot

"Abbiamo visto il Manage, abbiamo visto lo Sculpt, rimane soltanto da vedere il **Paint**." La scheda ha "più o meno gli stessi strumenti che c'erano di là": **Paint**, **Smooth**, **Flatten** e **Noise**, con la stessa strength e gli stessi tipi di pennello. Ma in basso Unreal avverte: *non ci sono target layer assegnati a questo landscape; usa i bottoni in alto per aggiungerne di nuovi o popolarli dal materiale correntemente assegnato.* I bottoni sono **Create Layer** e **Create Layers from Assigned Materials**, quest'ultimo, "se non sbaglio, unico di Unreal 5, sulla quarta edizione non c'è".

Un materiale normale non basta. Il landscape può **dipingere il materiale** con il pennello, "il materiale però deve essere fatto *ad hoc*". Prova: un nuovo materiale con dentro "unicamente un colore". Messo sul landscape, il landscape lo tratta "come un oggetto qualsiasi": "tutto quanto diventa disgustosamente verde", e non c'è modo di modificare nulla, "perché il nostro materiale di layer non ne ha".

Il "grosso errore". A questo punto il docente si accorge di "aver fatto un grosso errore: niente di grave, semplicemente una cosa che ci farà perdere un po' di tempo" (la spiegherà più avanti: sono le Virtual Texture, sezione 1.14.16).

Il pacchetto StackOBot. Fra gli asset del pacchetto **StackOBot**, "il pacchettino con il robottino e tutte le cose", nella cartella *Environment* ci sono: *Props* e *Modular* (mesh da vedere alla fine della lezione "o all'inizio della prossima"), *Materials*, *Foliage* (per il Foliage Mode) e **Landscape**, con "tutto quello che riguarda il landscape": delle texture "che non sappiamo bene cosa fanno", una mesh che per ora non serve, dei materiali, fra cui **M_Landscape**, e dei **Landscape Layers** "che non sappiamo bene cosa fanno" e che si rivedranno più tardi.

Il nero. Si assegna M_Landscape al landscape (nel campo *Landscape Material* dei Details): "va tutto **nero**". Anche un materiale pensato per il landscape, appena assegnato, "non ci dà subito quello di cui abbiamo bisogno"; e "il fatto che diventi tutto nero è quello che vogliamo, per noi è positivo". Bisogna stabilire come funzionano i layer: visto che il materiale "è già stato costruito per avere tanti layer diversi", si preme **Create Layers from Assigned Materials**. Unreal crea **quattro layer**, e il terreno resta nero. (Resta nero perché nessun layer è ancora stato dipinto: si veda l'approfondimento.)

Dentro M_Landscape. Il docente apre il materiale, "molto piccolo, vero? Molto comodo" (ironico: il nodo principale è ridotto a un solo pin). Le parti interessanti:

- quattro **material function**, *MF_Rocks*, *MF_Pebbles*, *MF_Sand* e *MF_Grass*: "il funzionamento dei singoli materiali, quindi dei singoli layer". Si creano con tasto destro, *Material Function*: sono "delle piccole funzioni", come quelle dei Blueprint, che descrivono "come funziona l'erba, come funziona la ghiaia, i *pebbles*", la roccia e la sabbia;
- dentro ogni funzione le texture sono collegate alla **Absolute World Position**: "vi ricordate i materiali **world aligned** che avevamo visto la volta scorsa? Questo è il modo per farli, anche per il landscape";
- tutte le funzioni terminano con un nodo **Set Material Attributes**: un nodo "vuoto" in cui, aggiungendo slot all'array, si decide quale informazione passa ogni ingresso (Base Color, Metallic, Specular, Roughness, Normal...). Qui ogni funzione passa **Base Color**, **Roughness** e **Normal** del proprio materiale, presi da dei *Texture Sample* ("vi ricordate, l'abbiamo fatta questa roba");

- le quattro uscite entrano nel nodo che è "alla base del funzionamento del materiale del landscape": il **Landscape Layer Blend**.

I nodi del landscape. Cercando *landscape* nella palette dei nodi, Unreal mostra "tutti i nodi che sono utili per il landscape". Oggi se ne vedono tre (più uno "che sta da un'altra parte", il Landscape Grass Output): il **Landscape Layer Blend**, "il fondamentale", il **Landscape Layer Sample** e la **Landscape Visibility Mask**.

Landscape Layer Blend. È un nodo "vuoto", come il Set Material Attributes, in cui si aggiungono elementi all'array *Layers*. Ogni elemento si apre con delle freccette, e lì si scrive il **nome del layer** (per esempio *Grass*). "Scriveteli bene, perché è importante scrivere bene il nome del layer": se non c'è il nome, "Unreal non lo trova, non lo rileva", anche se ci sono collegate delle cose; e altri nodi, come il Landscape Layer Sample, "hanno bisogno del nome per funzionare". Gli altri campi dell'elemento "sono dei parametri di preview che al momento non ci interessano". Usando questo nodo, gli autori del materiale hanno fatto quattro layer: **roccia**, **sassolini** (ghiaia), **sabbia** ed **erba**, tutti collegati al nodo risultante del materiale.

Use Material Attributes. Il nodo risultante è "così piccolino" perché nei dettagli del materiale è spuntata la casella **Use Material Attributes**, che "trasforma il modo in cui funziona il nodo": invece di avere un pin per Base Color, uno per Roughness, uno per Normal, ha un unico ingresso *Material Attributes*. Senza la casella bisognerebbe collegare il risultato su ogni parametro, cosa comoda se si vogliono modificare le singole cose; ma poiché tutte le operazioni avvengono nelle funzioni, "per lui è comodo" far passare tutto in un colpo solo.

Dipingere. Che cosa permette la suddivisione in layer? "Di avere dei layer da poter dipingere." Selezionato **Rock** e passato il pennello, il terreno "dipinge roccia", "o perlomeno quello che noi abbiamo impostato come valore per il materiale della roccia": se non piace, la texture e le dimensioni si cambiano da dentro la sua material function, "tutto come abbiamo visto la volta scorsa". Poi i **Pebbles** ("dei sassolini, una ghiaietta"), la **Sand** e la **Grass**. Ogni volta Unreal deve "compilare gli shader", e ci vuole un po'.

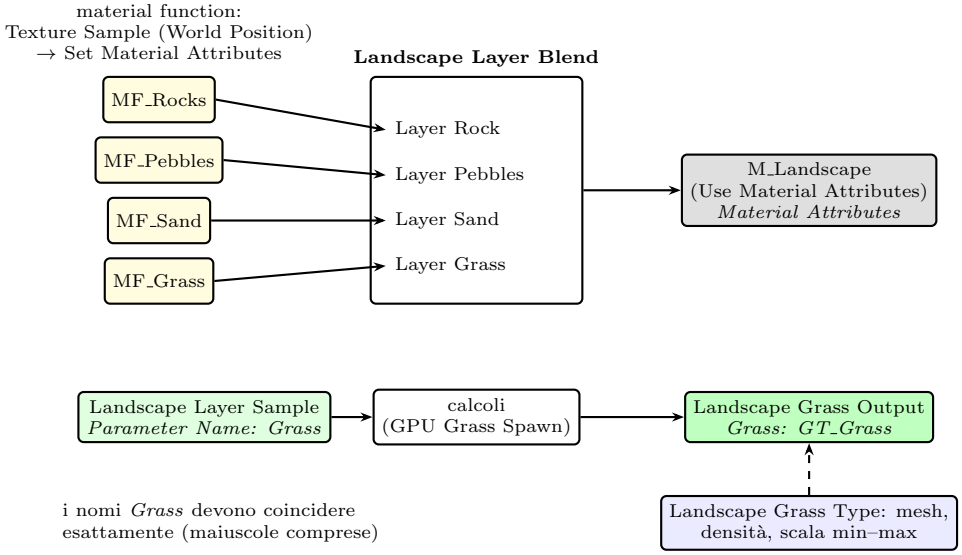


Figure 1.14.6: Lo schema del materiale M_Landscape di StackOBot come descritto a lezione. In alto, la parte che *dipinge*: quattro material function entrano nel Landscape Layer Blend, ognuna nel layer con il proprio nome, e il risultato va al nodo del materiale con Use Material Attributes. In basso, la parte separata che *fa spawnare* l'erba: un Landscape Layer Sample legge dove è stato dipinto il layer Grass e, dopo alcuni calcoli, alimenta il Landscape Grass Output, che usa un Landscape Grass Type. I nodi grigi sono nodi finali. Ricostruzione schematica.

Approfondimento: perché il terreno è nero, e i pesi dei layer. Un materiale del landscape non sceglie da solo dove mettere la roccia o l'erba: per ogni layer il landscape memorizza una *weightmap*, una mappa di pesi da 0 a 1 per ogni vertice, e il Landscape Layer Blend mescola i layer secondo quei pesi. Appena assegnato il materiale, nessun layer ha ancora un peso, quindi la somma è zero e il risultato è nero: per questo il docente lo considera un buon segno. Quando si dipinge il primo layer su un componente, i pesi cominciano a esistere; dipingere un layer riduce gli altri nello stesso punto, perché in un layer *weight-blended* la somma dei pesi resta 1. Il primo layer della lista viene spesso usato come "fondo" e si può riempire tutto il terreno con il comando *Fill Layer* (tasto destro sul layer nella scheda Paint). Ogni layer in più costa: Unreal campiona le texture di tutti i layer presenti su un componente, e per questo conviene non usarne troppi per componente. Il pulsante *Create Layers from Assigned Materials* legge i nomi dei layer dal Landscape Layer Blend del materiale e crea i target layer corrispondenti; se nel progetto esistono già le relative *Layer Info* (come nel pacchetto StackOBot) le collega, altrimenti vanno create. Il libro di Romero e Sewell mostra il Material Editor e i pin Base Color, Metallic e Roughness nel capitolo 5, ma non tratta i materiali del landscape; il resto si basa sul funzionamento generale dell'engine.

1.14.16 L'erba che spawna e le Virtual Texture

La domanda. Con l'erba il docente si ferma: "se mi state seguendo passo passo, noterete una differenza rispetto a quella che ho io". "Vi sembra che quello che ha fatto sia uguale a quello che è successo con gli altri materiali? Domanda a cui mi aspetto una risposta." Gli studenti non stanno seguendo su Unreal ("ah, non siete su Unreal? Va bene"), e il docente spiega da sé.

La differenza. Dipingendo il layer dell'erba non cambia solo il colore del terreno: **spawna del foliage**, dell'erba vera e propria. "Non dovrebbe succedere di default", perché di default si dipinge e basta: "nel materiale è stata fatta una cosa specifica per l'erba che permette di spawnare foliage quando viene utilizzato il layer dell'erba".

Il problema: l'erba nera. Chi ha seguito su Unreal avrà notato che nel proprio progetto **l'erba viene tutta nera**: "si vede scuro, molto forte e molto nero, proprio come se non gli arrivasse la luce". Il motivo è un parametro "che di default è disabilitato". La soluzione, "fatelo adesso, perché sennò poi puntualmente vi dimenticate o più avanti vi dà problemi e non sapete perché":

1. aprire i **Project Settings**;
2. cercare **Virtual Texture**;
3. spuntare **Enable Virtual Texture Support**, che di default è vuota.

Il docente si corregge subito su una prima formulazione ("no, ho detto una cazzata"): non è lo spawn di foliage dal materiale in sé a richiedere le Virtual Texture, ma *quest'erba in particolare*, per il modo in cui è fatta. "Se invece di far spawnare questa erbetta gli facessimo spawnare degli alberi", il problema non ci sarebbe.

Come funziona lo spawn. Nel materiale, sotto la parte già vista, ci sono una parte sulle *Runtime Virtual Texture* ("e cosa influenzano") e una sezione chiamata **GPU Grass Spawn**, con due nodi:

- il **Landscape Layer Sample**: un parametro in cui si scrive il nome di un layer, "scritto esattamente come è scritto il layer", e che restituisce dove quel layer è stato dipinto. Qui il nome è *Grass*: lo spawn avviene "quando il layer grass viene chiamato";
- dopo "tutto un calcolo che lui si fa", il risultato va nel **Landscape Grass Output**, "il nodo che si occupa di far spawnare quell'erba lì".

Il docente mostra che si può aggiungere un secondo elemento al Landscape Grass Output, chiamarlo *Pebbles* e collegarci un Landscape Layer Sample dei Pebbles "senza fare tutti i calcoli, tutte le cose cerimoniose". Ogni elemento richiede però un **Grass Type**.

Approfondimento: Runtime Virtual Texture e l'erba di StackOBot. Una *Runtime Virtual Texture* (RVT) è una texture enorme e virtuale in cui il landscape "stampa" dall'alto i propri attributi (colore, normale, altezza), generata a pezzi sulla GPU solo dove serve. Altri materiali la possono leggere: è il modo tipico con cui l'erba prende *il colore del terreno sottostante*, sfumando alla base nello stesso verde o marrone del suolo. Se il progetto non ha il supporto alle Virtual Texture attivo, il campionamento non restituisce nulla e il materiale dell'erba riceve un colore nullo: da cui l'erba nera. La spiegazione è coerente con le parole del docente ("il modo in cui è fatto questo foliage") ma il capitolo non ha potuto verificare il grafo del materiale dell'erba di StackOBot. L'opzione si trova in *Project Settings* → *Engine* → *Rendering* → *Virtual Textures*, e richiede il riavvio dell'editor; perché il landscape scriva davvero nella RVT serve anche un actor *Runtime Virtual Texture Volume* nel livello, associato alla stessa RVT indicata nei Details del landscape. Funzionamento generale dell'engine.

1.14.17 Le Layer Info e il Landscape Grass Type

Le Layer Info. Che cosa sono gli asset della cartella *Landscape Layers*? "Sono informazioni che Unreal si salva e che utilizza per il materiale del landscape". Per creare un proprio materiale del landscape "bisogna creare questo tipo di informazioni"; qui ci sono già, perché il materiale è preesistente. Aprendo *Grass_LayerInfo* "in realtà nulla di che": "non ci sono informazioni specifiche, non ci sono texture applicate". Ma è "il posto in cui, volendo, potreste andare ad applicare un **Physical Material**", come quelli dell'acqua e della roccia fatti con Igor. L'esempio: se il robottino deve fare suoni diversi su ghiaia, sabbia ed erba, si creano tre physical material e li si collega alle rispettive Layer Info; da quel momento, "ogni volta che io dipingo roccia, quella superficie prende tutti gli attributi fisici del physical material della roccia, quindi ad esempio il suono che fa quando ci passo sopra". (Il docente non lo fa a lezione.)

Il Landscape Grass Type. Il Grass Type è "un altro tipo di oggetto", visto prima "al volo senza sapere cosa facesse". Per crearne uno: nella cartella *Foliage*, tasto destro, *Foliage* → **Landscape Grass Type**, chiamato **GT_Pebbles**. Il GT_Grass del pacchetto ha un elemento con "tutta una serie di parametri modificati *ad hoc*"; il nuovo non ne ha. Il docente aggiunge un elemento, lascia "per ora tutti i parametri al loro valore di default", e nella **Grass Mesh** inserisce una roccia (cercando fra le mesh: *Rock Flats*). Salva, collega GT_Pebbles all'elemento Pebbles del Landscape Grass Output e salva tutto.

Che cosa va storto. "Quello che dovrebbe fare adesso è mettere uno sproposito, probabilmente, perché l'abbiamo fatto senza *tweakare* i numeri, uno sproposito di rocce" nella parte dipinta con i Pebbles. E così è:

- il PC rallenta molto ("credo che sia semplicemente una questione di affaticamento del PC"); per un po' le rocce non compaiono e anche le anteprime dei materiali smettono di funzionare;
- quando compaiono, "ne ha messi talmente tanti che non si vede neanche più quello che c'è sotto". Il docente abbassa la **densità**, prima "minore", poi a 5;
- le rocce non sono "sassetti" come pensava, ma grandi rocce: "non il tipo di asset che io credevo". Imposta una **scala** casuale "da 0,3 a 0,7";
- il materiale delle rocce appare "stranamente riflettente", "disintegrato": "penso sia un problema del materiale dell'oggetto". Prova un'altra roccia (*Boulder*) e ripete lo stesso errore; rimanda ("lo vedrò la prossima volta").

Il punto resta: "vi assicuro che funziona, anche perché è letteralmente quello che hanno fatto loro" con l'erba. Con del fogliame o dell'erba "è una cosa abbastanza rapida" che "ci toglie dall'impiccio di doverlo fare a mano in quella zona specifica e con precisione": "ogni volta che viene attivato il layer grass ci sarà sempre dell'erba sopra". Il foliage si può anche piazzare a mano con la sua modalità (la prossima volta): questo è "una chicca in più per velocizzare e automatizzare alcune cose". Alla fine il docente scollega l'elemento Pebbles: "adesso lui non spawna più niente qua, ma continua a spawnare l'erba".

La domanda sul nodo staccato. Uno studente non capisce come la parte del Landscape Grass "sia collegata al materiale": "visto così sembra separato dall'intero materiale". "È separato dal materiale", risponde il docente: "è quello che succede quando vai a ottimizzare un pochino il grafo". I nodi grigi sono **nodi finali**, "che ricevono l'informazione e fanno qualcosa". Il nodo del materiale dice: ci sono quattro layer con queste informazioni, "e questo è il tuo materiale". Il Landscape Grass Output è "un altro nodo finale", "un nodo che imposta foliage": tramite il Landscape Layer Sample cerca fra tutti i layer "un layer che si chiama Grass". "Se noi qui sbagliassimo e scrivessimo grass con la g minuscola, lui non lo troverebbe più e l'erba non spawnerebbe più." Il nodo "è programmato per dire: spawna foliage controllato, soltanto su questo punto qua", con il Grass Type scelto (qui GT_Grass) e i suoi parametri. "Adesso si chiama tutto grass, quindi è confusionario" (il layer, il parametro, il Grass Type, il nodo). Lo studente: "mi ha confuso tanto, però ora ho capito". Il Landscape Grass Output "è un risultante come il nodo finale del materiale, come il Runtime Virtual Texture Output".

Approfondimento: i parametri del Landscape Grass Type. Ogni elemento (*Grass Variety*) di un Landscape Grass Type ha, fra gli altri: **Grass Mesh**, la static mesh da spargere; **Grass Density**, il numero di istanze per unità di superficie (il valore di default è pensato per fili d'erba piccoli: con una roccia grande produce esattamente lo "sproposito" visto a lezione); **Start Cull Distance** ed **End Cull Distance**, le distanze fra cui le istanze sfumano e spariscono (l'erba procedurale si genera solo attorno alla camera ed è pensata per dettagli piccoli, non per oggetti visibili da lontano); **Scaling** con *Scale X/Y/Z* minima e massima, che il docente imposta fra 0,3 e 0,7; *Random Rotation* e *Align to Surface*; *Use Grid* e *Placement Jitter*, che regolano la distribuzione; e le opzioni di ombra e collisione. Per gli oggetti grandi, come rocce e alberi, Epic consiglia il Foliage Mode o altri sistemi procedurali invece del Grass Type, che non genera collisioni. Quanto ai Physical Material, il collegamento descritto dal docente è corretto: la *Layer Info* di ogni layer ha un campo *Phys Material*, e un *Line Trace* verso il basso sul landscape restituisce nella *Hit Result* il physical material del layer dominante in quel punto, che è esattamente ciò che serve per i suoni dei passi visti con Igor. Funzionamento generale dell'engine.

1.14.18 Un materiale del landscape fatto a mano

"Adesso abbiamo visto queste cose. Come si fanno a creare?" Il docente scollega M_Landscape dal landscape ("questa cosa che ho fatto non gli è piaciuta neanche un po"'). I quattro layer restano, ma "fini a sé stanti"; li elimina tutti, sapendo che le informazioni "rimangono conservate" negli asset delle Layer Info. Poi costruisce un materiale "molto più semplice", "senza funzioni, giusto per avere qualcosa".

I passaggi.

1. Un nuovo materiale, **M_LandscapeTest**, vuoto.
2. "Qual è il nodo necessario?" Uno studente: "quello che ti faceva mettere più materiali, non mi ricordo come si chiama". È il **Landscape Layer Blend**: "se scrivete landscape ve lo dà in automatico, perché è quello che più realisticamente state cercando".
3. Si aggiungono tre elementi all'array: **Grass**, **Sand**, **Rock**. "Se questi non li chiamate, poi non ve li trova. Quindi, occhio."
4. Si copia il Landscape Layer Blend: uno va nel **Base Color**, l'altro nella **Normal**. "Lo faremo molto semplice", solo colore e normale.
5. Per ogni layer servono un colore e una normale. Per l'erba il docente usa la texture *T_Grass* ("quella rossa che era di là") e copia dal materiale originale la piccola catena che la colora, perché "è pensata per essere fatta in maniera specifica": il colore viene scelto fra due tinte in base alla texture ("fra il rosso scuro e il rosso chiaro" nell'originale; "noi, volendo, possiamo farlo fra il rosso scuro e il blu"). L'erba non ha una normal map: il suo ingresso resta vuoto.
6. Per la sabbia e la roccia: un *Texture Sample* con la texture di colore e uno con la normale (per la roccia una texture *tileable*), collegati ai rispettivi ingressi dei due Landscape Layer Blend.

È "un lavoro molto più semplice, spartano e zozzo" di quello del pacchetto, "per farvi vedere che possiamo farlo anche noi e in maniera anche abbastanza semplice". Nel resto del materiale non si modifica nulla. Si assegna M_LandscapeTest al landscape.

Le Layer Info. Con *Create Layers from Assigned Materials* i layer Rock, Sand e Grass ricompaiono già collegati: "non le fa salvare" di nuovo, "perché sono gli stessi identici" layer di prima, e trova le Layer Info esistenti. Per mostrare come se ne crea una, il docente aggiunge un quarto elemento all'array (dopo qualche esitazione sul nome, "*Pebbles*", poi ribattezzato *Test*) e salva. Nella scheda Paint il nuovo layer avverte: *questo layer non ha layer info assegnate; devi crearne o assegnarne una prima di poter dipingere.* Con il + accanto al layer si sceglie **Create Layer Info**; Unreal chiede se **Weight-Blended Layer (normal)** o **Non Weight-Blended Layer**: si sceglie il primo, "normal". Poi chiede dove salvarla: nella stessa cartella delle altre (*StackOBot/Environment/Landscape/Layers*). Compare **Test_LayerInfo**, e il nuovo layer dipinge: dipinge "Pebbles", perché è la texture che gli è stata collegata. Il layer di prova si elimina subito dopo. L'erba ora è diversa da prima, con i colori cambiati: "viola", "sembra un livido" ("è stata una scelta di colori pessima, poi le cambierò").

Il Texture Coordinate. "Cosa importante: vedete che tutte le texture sono molto piccole?" Per ingrandirle serve un nodo "visto anche la volta scorsa", il **Texture Coordinate**, collegato agli ingressi **UV** di tutti i Texture Sample, "in modo che tutti quanti condividano le stesse informazioni". Si cambiano poi i suoi parametri: "invece di 1 e 1", **0,2**, e le texture diventano più grandi. Se si vuole la sabbia più grande della roccia, si usa un Texture Coordinate diverso solo per la sabbia.

Gli strumenti di Paint. "Non serve che vi spiego questa roba come funziona", ma in breve:

- **Paint** dipinge il layer selezionato;
- **Smooth** ammorbidisce "il passaggio che c'è fra una texture e l'altra": "qui è tutta roccia, qui metà roccia e metà sabbia, qui tutta sabbia";
- **Flatten** uniforma tutta la zona al layer del punto cliccato ("ho preso sabbia, ho tirato e ha fatto tutta sabbia");
- **Noise** "crea disturbo", mescolando parti prese da ciò che c'è attorno; ma "idealmente il noise non lo usiamo, facciamo le cose a mano".

Approfondimento: un materiale del landscape essenziale. Il materiale costruito a lezione è il minimo indispensabile, e mostra i tre ingredienti obbligatori: un *Landscape Layer Blend* per ogni ingresso che deve variare con il layer (qui Base Color e Normal; si può fare lo stesso con Roughness), i *nomi* dei layer identici in tutti i Blend, e le *Layer Info* create per ogni layer. Ogni elemento del Blend ha anche un *Blend Type*: *LB Weight Blend* (il normale, quello usato a lezione), *LB Alpha Blend* (un layer che si sovrappone agli altri) e *LB Height Blend* (mescola usando una mappa di altezza, così la sabbia "riempie" prima le fessure fra i sassi); i campi "di preview" (*Preview Weight*) servono solo a vedere il materiale nell'anteprima. Un layer *Non Weight-Blended* non sottrae peso agli altri: si usa per effetti che si sommano, come l'umidità o la neve sopra un altro materiale. Il Texture Coordinate con *UTiling* e *VTiling* a 0,2 ripete la texture una volta ogni cinque unità di UV del landscape, cioè, con la scala di default, una volta ogni cinque metri; la soluzione più pulita, usata da StackOBot, è calcolare le UV dalla *Absolute World Position* divisa per una dimensione in centimetri. Il capitolo 5 di Romero e Sewell introduce i Texture Sample e i parametri nel Material Editor; il resto è funzionamento generale dell'engine.

1.14.19 La Visibility Mask: i buchi nel terreno

"Quello che ci manca è la visibilità che avevamo visto prima." Fra i nodi del landscape c'era la **Landscape Visibility Mask**: "un nodo che non può essere modificato, che fa il suo e basta". Va collegato all'**Opacity Mask** del materiale, ma il pin non c'è. "Vi ricordate cosa dobbiamo fare per far sì che quel pin si veda?" "So che è passato un mese dalla volta scorsa" (la lezione sui materiali): nessuna risposta.

La Blend Mode. Il pin si sblocca dalla **Blend Mode**, nei dettagli del nodo risultante (cliccando sul nodo o sullo sfondo del grafo). Di default è **Opaque**. Con **Translucent** Unreal "elimina tante cose e ne mette altre"; con **Masked**, invece, "è un materiale opaco, normale, che però ha la possibilità di essere influenzato con una maschera di opacità, che quindi decide dove e quanto sarà visibile". Si attiva l'Opacity Mask, ci si collega la Landscape Visibility Mask (figura 1.14.7), e nella scheda Sculpt lo strumento **Visibility** si sblocca.

Il buco. Con il pennello della Visibility "mi scava il buco": "un buco a tutti gli effetti", che "non ha neanche le **collisioni**". In Play il personaggio, camminando sul landscape, arriva al buco e "va giù". (Il tragitto è lungo: "tenere premuto Shift non mantiene il movimento", e "la cosa più intelligente da fare era spostare direttamente lo sfondo lì davanti, ma ci ho pensato" dopo; "lo sfondo" è con ogni probabilità il *PlayerStart*, il punto di partenza del personaggio.) Così si costruiscono le **grotte**: "immaginate che qui invece di esserci questo ci siano delle mesh che scendono verso il basso, un tetto simulato e delle rocce a coprire qua, e abbiamo costruito una grotta". Un consiglio: farlo dove i quadrati non sono stirati, "viene meglio, viene più preciso"; dove sono stirati, anche il pennello si stira e il buco "si espande".

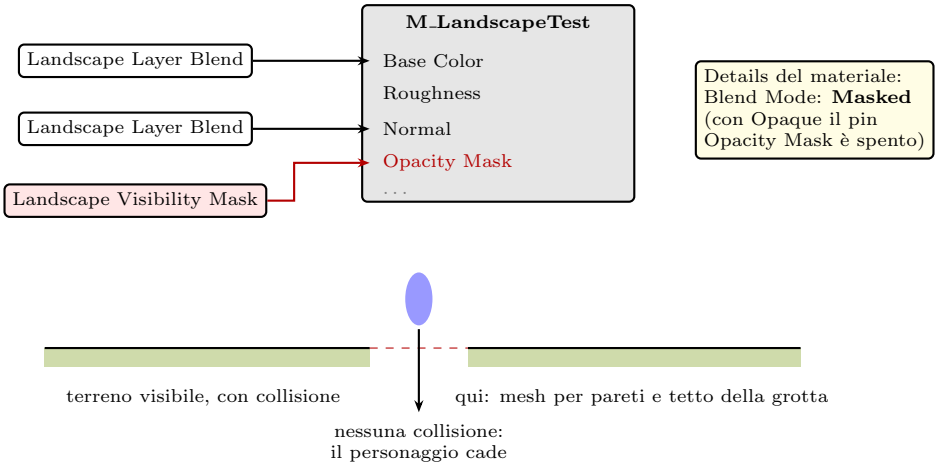


Figure 1.14.7: La Visibility Mask. Nel materiale del landscape la Blend Mode passa da Opaque a Masked, il che rende disponibile il pin Opacity Mask; collegandoci il nodo Landscape Visibility Mask, lo strumento Visibility può dipingere dei buchi. La parte nascosta non ha collisioni. Ricostruzione schematica.

Approfondimento: Masked, collisioni e costi. Due precisazioni. La prima: la Blend Mode *Masked* è la stessa vista nella terza lezione per ritagliare un materiale con una maschera; un materiale Masked costa un po' più di uno Opaque, e su un landscape intero non è gratuito. Per questo, nelle versioni recenti di Unreal Engine 5, esiste un'alternativa: il campo **Landscape Hole Material** nei Details del landscape, che permette di usare un materiale Masked con la Visibility Mask *solo* sui componenti che hanno davvero dei buchi, lasciando il materiale principale Opaque. La seconda: la Visibility toglie sia la parte visibile sia la collisione del landscape (lo strumento lavora per quad interi, quindi la forma del buco segue la griglia), e per questo dove si buca serve sempre qualcosa sotto: le mesh della grotta, un volume che uccide il personaggio o un teletrasporto, come quello del livello di Filippo. Funzionamento generale dell'engine.

1.14.20 L'automaterial, i compiti e la chiusura

"Ci mancano 5 minuti." Ci sarebbero "tante altre cose che si possono fare con il landscape". La più interessante: un materiale "costruito appositamente per modificarsi quando viene modificato il landscape". Esempio: "di default hai la sabbia, ma sulle pareti in verticale hai la roccia"; scolpendo, "la sabbia sarebbe diventata roccia qui ai lati". È l'**automaterial**: "una roba un po' più complicata, ma neanche così tanto", "molto figo quando funziona, perché vi sembra che il landscape reagisca direttamente a voi". (Intanto Giulia deve andare: "ciao, Giulia".) Uno studente, che "già da prima" voleva chiedere "questa cosa dell'automatismo",

conferma che gli farebbe piacere vederla; non vuole "occupare il tempo della lezione" per una sua domanda, ma il docente lo rassicura: si farà "sicuramente la prossima lezione", perché richiede di modificare il materiale "in modo che prenda in considerazione la **pendenza** del landscape". "Voi ricordatemelo." Insieme, probabilmente, al foliage. E cambierà "questo viola che è veramente inguardabile" ("senza il foliage sopra è veramente brutto").

I compiti. "Se volete, continuate a smanettare un pochino con il block out, altrimenti provate a fare qualcosina in landscape, magari un **piccolo paesaggio**, un piccolo ambiente", qualcosa che permetta di usare gli strumenti visti fino ad ora.

La domanda sulla pendenza. Uno studente chiede se ci sia uno strumento che dica l'**inclinazione del landscape**: come si fa a sapere se il personaggio ci può salire? "Non che io sappia". Ma vale "lo stesso parametro delle pendenze fatte in block out": il landscape "funziona come le geometrie". Se il personaggio sale fino a una certa pendenza ("una pendenza dell'80%", "del 50"), sul landscape vale la stessa soglia. Il docente non conosce un modo per leggere la pendenza in un punto preciso dall'editor, "non credo ci sia"; ci sono però dei nodi che permettono di ricavare in un punto la pendenza del landscape, "in modo che su quel punto venga spawnata una determinata cosa", da usare anche "come trigger". "Magari me lo vedo, perché magari lo trovo, ed era una cosa stupida": risponderà la volta successiva. (Il Walkable Floor Angle del Character Movement, visto la volta scorsa, è in *gradi*, non in percentuale: 44,77 gradi corrispondono a una pendenza di circa il 99%, e una pendenza del 50% è di circa 27 gradi. Le percentuali del docente vanno lette come esempi generici.)

La chiusura. "Sono rimaste tre persone." La prossima volta: l'automaterial, "sicuramente qualcosa sul foliage", e magari "creare qualcosa esteticamente un po' più piacevole con le luci e con il post process". "Buonasera a tutti."

Approfondimento: leggere la pendenza e l'automaterial. La pendenza di un punto del terreno è l'angolo fra la sua *normale* e l'asse verticale. In un Blueprint la si ricava con un *Line Trace by Channel* verso il basso: la *Hit Result* contiene l'*Impact Normal*, e l'angolo in gradi è $\arccos(\vec{n} \cdot \hat{z})$ (nodi *Dot Product* e *Acos (Degrees)*); è lo stesso confronto che il Character Movement fa con il Walkable Floor Angle. Romero e Sewell dedicano ai *trace* e alla Hit Result il capitolo 14 ("Math and Trace Nodes"), anche se non applicati al landscape. Per vedere le pendenze nell'editor, il Landscape Mode non ha un misuratore, ma le visualizzazioni del viewport (per esempio la modalità di vista delle normali) aiutano, e un modo pratico è quello del docente: una rampa di prova nella gym, con l'inclinazione nota, accanto alla zona da controllare. L'automaterial usa lo stesso principio dentro il materiale: la normale del terreno nello spazio del mondo (per esempio dal nodo *VertexNormalWS*) moltiplicata scalarmente per l'asse Z dà 1 in piano e 0 sulle pareti verticali; quel valore, passato per una soglia e una sfumatura, diventa il peso con cui un *Lerp* sceglie fra sabbia ed erba (in piano) e roccia (sui pendii). Così il terreno "reagisce" alla scultura senza essere dipinto. Funzionamento generale dell'engine; la lezione successiva lo costruirà in diretta.

Approfondimento: il paesaggio come spazio da leggere. Per il compito, due suggerimenti dal libro di Totten (capitolo 3). Il primo riguarda i passaggi fra un materiale e l'altro, che sono il tema della scheda Paint: le transizioni fra texture di superficie generano dei *confini percepiti* (*edges*); una transizione rapida segna un confine netto, ed è spesso accompagnata da muri o cancelli, "perché il paesaggio raramente cambia di colpo"; una transizione graduale crea attesa per una zona nuova e segnala un passaggio naturale "da pianura a foresta, da deserto a canyon". Lo Smooth della scheda Paint e il numero di quad su cui avviene il passaggio sono, in questo senso, scelte di design. Il secondo riguarda i **landmark**, gli stessi del block out di God of War: in un paesaggio aperto possono essere "oggetti naturali" che contrastano con il resto, come una montagna isolata, e servono a orientarsi e a misurare il proprio progresso, come la Cittadella di *Half-Life 2*, visibile da quasi tutti i livelli.

Nodo o impostazione del materiale	Che cosa fa
Landscape Layer Blend	mescola i layer dipinti; un elemento dell'array per layer, con il nome esatto; uno per ogni ingresso da far variare (Base Color, Normal…)
Landscape Layer Sample	restituisce il peso di un layer (per nome) in ogni punto: dove è stato dipinto
Landscape Grass Output	nodo finale che fa spawnare un Landscape Grass Type dove il suo ingresso è positivo
Landscape Visibility Mask	nodo fisso, da collegare all'Opacity Mask: abilita lo strumento Visibility
Material Function	sotto-grafo riutilizzabile (MF_Rocks, MF_Pebbles, MF_Sand, MF_Grass)
Set Material Attributes	raccoglie più attributi (Base Color, Roughness, Normal) in un solo filo
Use Material Attributes (Details)	riduce il nodo del materiale a un unico ingresso Material Attributes
Absolute World Position	coordinate del mondo, per texture *world aligned*
Texture Sample, Texture Coordinate	la texture; la ripetizione (UTiling, VTiling: 0,2 a lezione)
Blend Mode: Opaque / Masked / Translucent	opaco; opaco con maschera di opacità (serve per la Visibility); trasparente
Runtime Virtual Texture Output	nodo finale che scrive nella virtual texture (nel materiale StackOBot)
Enable Virtual Texture Support (Project Settings)	necessario per l'erba di StackOBot, altrimenti nera

Table 1.14.4: I nodi e le impostazioni del materiale del landscape visti a lezione.

Misura o valore	Valore	Dove e a che serve
Salto verticale comodo (gym di Federico)	1,40 m	dislivelli superabili nel livello
Salto al limite, "slida"	1,60 m	da evitare nel livello
Salto "snappy", incomprensibile	1,80 m	da evitare assolutamente
Dislivello chiaramente insuperabile	2 m	muri che non si devono saltare
Porta per la camera	2 m "brutta"; 4×3 m "buona"	passaggi in terza persona
Gradino (Max Step Height)	60 "orribile", poi più basso	la scala dei gradini
Salto in lungo massimo	3 camminando, 6 correndo	da progettare con un margine (forgiveness)
Parkour di Minecraft	4 blocchi	l'esempio di Federico
Section Size	63×63 (default); 31, 7	quad per sezione
Number of Components	8×8	componenti del landscape di prova
Tool Strength	0,1 – 1	lenta – forte
Falloff	0 – 1	tutto pieno – tutto sfumato
Threshold dell'Erosion	"17"	più basso, più erosione
Densità del GT_Pebbles	ridotta fino a 5	troppe rocce col default
Scala del GT_Pebbles	0,3 – 0,7	dimensione casuale delle rocce
UV del Texture Coordinate	1 → 0,2	texture del terreno più grandi

Table 1.14.5: Le misure e i valori della giornata.

Sintomo	**Causa e rimedio**
Il pivot del cubo dell'engine non si cambia	gli asset dell'engine non sono modificabili: copiare il cubo e cambiare il pivot con il Modeling Mode
Disco C pieno, Unreal lentissimo	liberare spazio prima della lezione (non risolto a lezione)
Lo schermo condiviso "impazzisce"	problema di Zoom, non risolto
Il pennello Visibility è rosso e non fa nulla	manca la Landscape Visibility Mask nel materiale
Il materiale si deforma sui pendii	quad stirati: Smooth, pendenze meno ripide, proiezione world aligned, mesh sulle pareti
La rampa sottile si "spacca" da lontano	i LOD semplificano le forme con pochi poligoni: rampe più larghe, landscape non troppo piccoli
Un ponte sospeso col landscape non funziona	il landscape non si sovrappone: usare una mesh
Il Flatten appiattisce troppo	Flatten Mode Raise o Lower
La regione selezionata resta attiva	Clear Region Selection
Un materiale normale colora tutto e non si dipinge	servono Landscape Layer Blend e layer
M_Landscape rende tutto nero	normale: creare i layer (Create Layers from Assigned Materials) e dipingere
L'erba di StackOBot è nera	attivare Enable Virtual Texture Support nei Project Settings
Un layer non viene trovato, l'erba non spawna	il nome nel nodo non coincide con quello del layer (anche le maiuscole)
"Questo layer non ha layer info"	+ accanto al layer, Create Layer Info, Weight-Blended
Il Grass Type riempie tutto di rocce	densità di default pensata per l'erba: ridurla; scegliere mesh piccole
Le rocce spawnate sono riflettenti	probabile problema del materiale della mesh (non risolto a lezione)
Il PC rallenta dopo il Grass Type	troppe istanze; abbassare la densità
Le texture del terreno sono minuscole	Texture Coordinate con valori minori di 1 nelle UV
Il pin Opacity Mask non c'è	Blend Mode Masked
Il personaggio cade nel buco	la Visibility toglie anche le collisioni: coprire con mesh

Table 1.14.6: I problemi incontrati durante la lezione, con la causa e il rimedio.

Punto	Che cosa dice la lezione, e la precisazione
"La mia prima è stata la scorsa"	è la prima delle ultime quattro; il docente aveva già tenuto la terza lezione del corso
"Oggi facciamo martedì, giovedì"	frase storpiata: oggi è martedì, la prossima lezione giovedì, l'ultima il martedì successivo
Il salto "gestito da una forza"	la Jump Z Velocity è una velocità iniziale, non una forza
Landscape "più grande" ha più poligoni	più poligoni a parità di superficie vuol dire più *risoluzione*, non più estensione
Create Layers from Assigned Materials solo in Unreal 5	detto "se non sbaglio"; non verificato nel capitolo
L'erba nera e le Virtual Texture	spiegazione corretta nella sostanza; il grafo dell'erba di StackOBot non è stato verificato
Opzione del Flatten "height intervals"	con ogni probabilità la modalità Terrace
Pokémon "programmati in maniera stupida"	opinione personale, non analisi tecnica
The Sims 2 "nel 2000"	The Sims è del 2000, The Sims 2 del 2004
Pendenze "dell'80%", "del 50"	il Walkable Floor Angle è in gradi (44,77° ≈ 99%)
"Waze Up", "c'è il piede"	con ogni probabilità *Only Up!*; il resto non è ricostruibile
"Drummond Island"	l'isola di Drum di *One Piece* (i Drum Rockies)
Minecraft e l'accovacciarsi in aria	ricordo dichiaratamente incerto del docente

Table 1.14.7: Le imprecisioni e i punti incerti della lezione, raccolti in un'unica tabella.

Punti chiave del capitolo

- Non servono tante meccaniche: **less is more**. Con un paio di meccaniche applicate in modi diversi si costruiscono livelli interessanti, come mostra il livello di Filippo, "una meccanica per livellino".
- Un block out mette i **feedback positivi** dove si vuole che il giocatore esplori e i **gate** dove non si vuole: **hard** (muri) o **soft** (nemici troppo forti).
- La **gym** di Federico mostra che cosa misurare: l'ostacolo minimo contro la torretta, le porte per la camera, i gradini, le altezze di salto. Nel livello i dislivelli saranno o chiaramente superabili (1,40 m) o chiaramente insuperabili (2 m), mai "al limite": un muro deve essere **subito comprensibile**.
- La **forgiveness**: i giocatori non sono macchine. I salti si progettano sotto il massimo misurato, con più margine in camminata che in corsa; i salti al limite sono per i platform hardcore. I primi salti di un livello devono essere **low stake**.
- Una gym non è obbligatoria per un livello singolo, ma in un progetto con molti livelli fa "un po' di lavoro prima per fare meno lavoro dopo". Il miglior sviluppatore è quello che **si adatta e si informa** più in fretta.
- Gli asset dell'engine non si modificano: per un cubo con il pivot all'angolo se ne fa una copia e si sposta il pivot con il Modeling Mode.
- Il **Landscape** è un piano modellabile per gli esterni e gli open world. Si crea dalla scheda **Manage**: componenti, sezioni, Section Size (63×63 di default), numero di componenti, materiale. Più quad, curve più morbide e più peso.
- Il **pennello** ha Size, **Falloff** (0 pieno, 1 tutto sfumato, con forme Smooth, Linear, Spherical, Tip) e Strength; può essere un cerchio, una texture (**Alpha**, con il canale scelto), un **Pattern** o un Component.
- La griglia sul terreno mostra come verrà **stirato il materiale**: le pareti ripide lo rovinano, lo **Smooth** lo salva.

Strumento, actor o impostazione	**Che cosa fa**
Hard gate / soft gate	muro chiuso; ostacolo percepito (nemico troppo forte)
Forgiveness	margine concesso al giocatore su salti e tempi
Instance Editable, Show 3D Widget (approfondimento)	variabili modificabili per istanza; gizmo nel viewport
Call In Editor (approfondimento)	pulsante nei Details che esegue una funzione nell'editor
Modeling Mode, Edit Pivot (approfondimento)	cambia il pivot di una mesh
Game Instance, SaveGame (approfondimento)	dati fra un livello e l'altro; dati su disco
Selection, Brush Editing, Landscape, Foliage Mode	le modalità dell'editor viste nel corso
Landscape (actor)	il terreno modellabile: un heightfield diviso in componenti
Manage: New, Import, Selection, Add, Delete, Splines	creare e gestire il landscape e i suoi componenti
Edit Layers	layer di modifica sovrapposti e bloccabili
Sculpt: Sculpt, Erase, Smooth, Flatten, Ramp	scolpire, cancellare, ammorbidire, appiattire, rampe
Sculpt: Erosion, Hydro, Noise	deformazioni "naturali" e rumore
Sculpt: Visibility, Mirror, Region Select, Copy/Paste	buchi; specchiare o ruotare; selezionare e copiare zone
Brush Type: Circle, Alpha, Pattern, Component	le forme del pennello
Brush Falloff: Smooth, Linear, Spherical, Tip	le forme della sfumatura
LOD (Level of Detail)	versioni semplificate con la distanza
Paint: Paint, Smooth, Flatten, Noise	dipingere e sfumare i layer del materiale
Create Layer / Create Layers from Assigned Materials	creare i target layer della scheda Paint
Landscape Layer Info (Weight-Blended / Non)	l'asset di ogni layer; contiene il Phys Material
Physical Material su un layer	suoni e attributi fisici diversi per superficie dipinta
Landscape Grass Type	mesh, densità, scala e distanze dell'erba procedurale
Enable Virtual Texture Support	attiva le virtual texture nel progetto
Landscape Visibility Mask + Blend Mode Masked	buchi nel landscape, senza collisioni
Automaterial (la prossima lezione)	materiale che cambia con la pendenza

Table 1.14.8: Gli strumenti, gli actor e le impostazioni della quattordicesima lezione.

- Gli strumenti di scultura: Sculpt (Shift scava), Erase, Smooth, **Flatten** (Both, Raise, Lower, Use Slope Flatten: plateau, scaloni, canyon), Ramp (rampe e dighe, non ponti sospesi), Erosion e Hydro (poco controllabili), Noise, Mirror, Copy/Paste con Paste Mode.
- Il landscape **non si sovrappone**: ponti e grotte si fanno con le mesh. I **LOD** semplificano il terreno con la distanza e distruggono le forme troppo sottili: dare sempre "un po' di spazio al landscape".
- Per dipingere serve un **materiale del landscape**: un **Landscape Layer Blend** con un elemento per layer, nomi scritti esattamente, e una **Layer Info** per ogni layer (Create Layers from Assigned Materials o Create Layer Info, Weight-Blended).
- Il materiale StackOBot usa material function, Set Material Attributes e Use Material Attributes; con un **Landscape Layer Sample** e un **Landscape Grass Output** fa **spawnare l'erba** dove si dipinge il layer Grass, con un **Landscape Grass Type** (mesh, densità, scala). L'erba richiede **Enable Virtual Texture Support**, altrimenti è nera.
- Le Layer Info possono contenere un **Physical Material**: superfici dipinte con suoni e proprietà fisiche diverse.
- Per un materiale fatto a mano bastano pochi nodi: due Layer Blend (Base Color e Normal), i Texture Sample e un **Texture Coordinate** per la scala delle texture.
- I **buchi**: Blend Mode **Masked**, **Landscape Visibility Mask** nell'Opacity Mask, poi lo strumento Visibility. Il buco non ha collisioni: con le mesh diventa una grotta.
- Prossima lezione: l'**automaterial** che cambia con la pendenza, il **foliage**, luci e post process. Compito: un piccolo paesaggio con il landscape.

Chapter 1.15

L'automaterial del landscape, il Foliage Mode e un primo sguardo al post process

Questa è la **quindicesima lezione** del corso breve di **introduzione a Unreal Engine**, la terza delle ultime quattro dedicate al **level design**. La lezione precedente, il martedì, aveva introdotto il **Landscape**: come si crea, come si scolpisce e come si dipinge con un materiale fatto di layer. Si era chiusa con due promesse. La prima era l'**automaterial**, chiesto da Federico: un materiale del landscape che si applica *da solo* mentre si scolpisce, mettendo la roccia sulle pareti ripide e la sabbia o l'erba in piano, senza doverlo dipingere a mano. La seconda era il **Foliage Mode**, la modalità con cui si spargono alberi, cespugli ed erba con un pennello. Oggi si mantengono entrambe le promesse, e in coda si apre appena la porta del **post process**, che sarà ripreso (forse) all'ultima lezione insieme al **packaging**.

Il docente, come nelle due lezioni precedenti, non dice mai il proprio nome. Tutto fa pensare che sia lo stesso: riprende "le cose che abbiamo visto la volta scorsa" sul landscape, corregge "una cosa che non era propriamente corretta" detta "quando vi ho parlato di luci" (la tredicesima lezione) e parla delle "quattro lezioni" che gli sono state affidate. Il docente è quindi, con ogni probabilità, **Davide**, lo stesso che aveva tenuto la terza lezione del corso su texture, materiali e mesh; resta una deduzione del capitolo, e nel resto del testo è semplicemente "il docente".

La giornata ha tre parti. La prima, dopo una breve apertura, è l'**automaterial**: tre materiali costruiti in progressione, dal più semplice (roccia sui pendii, sabbia in piano) al più completo (sabbia in basso, erba in alto, roccia sui pendii, più dei layer da dipingere a mano e l'erba che spawna solo dove ha senso). Seguono due correzioni sulla lezione delle luci (**Lumen** e i **god rays** fatti con la directional light), una parentesi sulla **Camera Actor** e sui coni di luce dell'engine. La seconda parte, dopo la pausa, è il **foliage**: prima i parametri del Landscape Grass Type, poi il **Foliage Mode** vero e proprio, con i suoi strumenti, i parametri di ogni tipo di foliage, gli alberi e il problema delle **collisioni**. La terza, negli ultimi dieci minuti, è un'occhiata al **Niagara** e la base del **Post Process Volume**. In chiusura, il compito: un piccolo landscape da esplorare attorno alla gym.

Tre avvertenze di lettura, come per le lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. **Unreal Engine** è "Ariel Engine" e "Arial 5"; **Lumen** è "la night" e "la nice"; il nodo **World Aligned Blend** è "Word Aligned Blend" e il suo ingresso **World Vector** è "word vector"; il **Lerp** è "l'ERP" e "Lab"; la **Material Instance** è "materia distance", "materialistance" e "l'instance"; **Sand** è "SEND"; il **Landscape Grass Output** è anche "let's skip grass"; **GT_Grass** è "GT Grass"; il **Landscape Mode** è "landscape model"; il **Paint** è "pinto" e il verbo "pintare"; il **foliage** è "fuoleggio", "foglietto", "folliage" e "FolgeMod"; il *Placement Jitter* è "Gintera"; il *culling* è "crawling" e la *Cull Distance* è "tal distance" e "Cal distance"; i **LOD** sono "load"; il *Random Yaw* è "random you"; il *Block All* è "block wall" e "block call"; la *complex collision* è "complex prediction"; le **decal** sono "DEGAL" e "tecla"; i **god rays** sono "Godrace" e "GoldShaft"; la **directional light** è "la diraccia"; il **Niagara** è anche "UFX" (VFX); il *lasso* è "l'azzo"; **Fab** è scritto giusto, ma lo store "ai miei tempi non si chiamava Fab". Dove la ricostruzione

è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: in alcuni punti il riconoscimento vocale produce testo estraneo alla lezione. In apertura ci sono frasi su "la struttura del giardino" ripetuta, sulla "Commissione europea" e su "lo zio"; e durante la pausa torna, ripetuto decine di volte, lo stesso blocco già visto nelle due lezioni precedenti ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?"). Non appartengono alla lezione e sono stati ignorati. La terza: lo schermo condiviso non è nella trascrizione. Il capitolo ricostruisce i passaggi da ciò che il docente dice mentre clicca, completa i nomi esatti di strumenti, parametri e nodi di Unreal Engine 5, distingue le prove dalle conclusioni e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono il principio del World Aligned Blend, la curva della fusione per altezza, lo schema del secondo automaterial, il calcolo che decide dove spawna l'erba nel terzo, il rapporto fra densità e raggio nel Foliage Mode, il Ground Slope Angle, la collisione rifatta dell'albero e i Post Process Volume annidati. Le tabelle riassumono i tre automaterial, i parametri del Grass Type, gli strumenti e i parametri del Foliage Mode, le prove numeriche della giornata, i problemi incontrati, le imprecisioni e gli strumenti visti. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro di Marcos Romero e Brenden Sewell, *Blueprints Visual Scripting for Unreal Engine 5*, il testo di riferimento per questa parte del libro, al libro di Christopher W. Totten, *An Architectural Approach to Level Design*, per i temi di design, e a nozioni tecniche generali su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione. Il libro di Romero e Sewell non tratta i materiali del landscape, il foliage né i parametri del post process: gli approfondimenti tecnici su questi argomenti si basano sul funzionamento generale dell'engine, e lo dicono.

Il percorso del capitolo:

1. l'**apertura**: le lezioni da recuperare, il Niagara, il sondaggio e i corsi dell'accademia (sezione 1.15.1);
2. l'**automaterial**: la scena di prova (sezione 1.15.2), la prima versione con il **World Aligned Blend** (sezione 1.15.3) e la **Material Instance** (sezione 1.15.4), la seconda con la fusione per **altezza** (sezione 1.15.5), gli attributi uniti e separati (sezione 1.15.6) e la terza con i **layer** e l'**erba** (sezione 1.15.7);
3. le correzioni sulle luci: **Lumen** e i **light shaft** (sezione 1.15.8);
4. la **Camera Actor** e i coni di luce dell'engine (sezione 1.15.9);
5. la **pausa** (sezione 1.15.10);
6. il **Landscape Grass Type** rivisto (sezione 1.15.11);
7. il **Foliage Mode**: che cos'è (sezione 1.15.12), gli strumenti (sezione 1.15.13), i parametri (sezione 1.15.14), densità e raggio (sezione 1.15.15) e le altre impostazioni (sezione 1.15.16);
8. gli **alberi** e le **collisioni** (sezione 1.15.17);
9. selezionare e rifinire il foliage, e il foliage non naturale (sezione 1.15.18);
10. la domanda sui mondi **procedurali** (sezione 1.15.19);
11. il **Niagara** e il **Post Process Volume** (sezione 1.15.20);
12. i **compiti**, la vetrata colorata e la chiusura (sezione 1.15.21).

1.15.1 L'apertura: recuperi, Niagara e i corsi dell'accademia

Chi ha guardato cosa. Il docente chiede se qualcuno sia riuscito a "buttare un occhio" sulle cose della volta scorsa, "o il fatto che sia in mezzo alla settimana ha reso le cose più complicate rispetto al fine settimana". "Per me è la seconda, purtroppo", e "pure per me". Uno studente racconta di essersi rimesso "a prescindere" a riguardare qualche lezione: gli mancano due lezioni da recuperare, e le registrazioni "sono disponibili fino a settembre". Il docente ammette che, "avendo quattro lezioni", si era tenuto le cose da fare per il fine settimana: oggi si finisce "quello che stavamo vedendo martedì", cioè il landscape, e si passa al **foliage**.

L'ultima lezione e il Niagara. Due studenti avvisano che forse non potranno seguire l'ultima lezione ("parto, non sono da solo", "se riesco mi collego"); il docente li rassicura: "rimangono registrate comunque". A questo punto, dice, gli conviene fare l'ultima lezione "su un argomento specifico piuttosto che usarla come esercitazione", lasciando magari alla fine "qualcosina da fare" come progetto. Un'idea che gli gira in testa "da un po' di tempo", ma che teme "poco fattibile": una lezione sul **Niagara**, "il tool che Unreal mette a disposizione per la creazione di effetti speciali". Il problema: "è un po' complicato da spiegare, quindi in tre ore non so quanto riesca a fare". Chiede se una lezione sui VFX ("UFX" nella trascrizione) possa interessare.

I superstiti. Si ride sul fatto che l'ultima lezione la faranno "in due o tre persone": "prima eravamo in tanti, poi man mano è sempre di meno". Per il docente è normale: inizia il periodo estivo, la gente va in vacanza, e con corsi del genere si crea sempre la situazione per cui chi salta una lezione pensa: "mi metto a seguire la lezione dopo, non avendo seguito quella prima? A questo punto me le guardo tutte a posteriori". Uno studente racconta di aver sottovalutato all'inizio "quanto fosse pesante" Unreal Engine: non aveva un PC adatto, e le prime tre o quattro lezioni le ha seguite senza poter provare nulla; rimettersi in pari "è stata una fatica". "Ha senso come cosa", conclude il docente: le lezioni sono registrate proprio perché ognuno se le gestisca "a modo suo".

Il mini sondaggio. "Se poteste dire: vorrei proprio vedere come si fa una cosa nello specifico, cosa sarebbe?" Uno studente risponde: la **post produzione**. Ha "modificato l'algoritmo di Instagram" per vedere più contenuti su Unreal, e ci sono tanti VFX artist che fanno "anche solo una scena, anche solo una foto": "tutti gli effetti della luce che ancora non abbiamo visto". Il docente prende nota: è qualcosa che forse si vede già oggi, "a seconda di quanto andiamo rapidi". La lezione di martedì sul landscape gli ha preso più tempo del previsto, "tuttavia sono contento, perché l'abbiamo affrontata bene"; ed è "ancora solo la superficie". Anche l'automaterial di oggi "è una cosa abbastanza superficiale, però è un esempio delle figate che si possono fare" se ci si mette a "smanettare".

Li rivedremo in accademia? Uno studente chiede se gli argomenti del corso si rivedranno "direttamente in Accademia". "Dipende molto dal corso che fate." Quasi tutti sono iscritti a **game design**; uno farà **character animation**.

- Per chi fa design: "sono più o meno tutti argomenti che rivedrete". Il corso di design, almeno "quando lo frequentavo io", era diviso in **tre parti**: una "full game design", una "full level design" e una "miscellanea", con Photoshop, programmazione "e tutta una serie di altre cose". Insieme alla parte teorica, che in questo corso non si fa, le cose viste "in un modo o nell'altro, o facendo programmazione o facendo level design, le rivedrete". Più che lezioni, saranno spesso *task* da risolvere "ingegnandovi": "un po' le rifarete, un po' ci tornerete voi attivamente".
- Per character animation il docente non sa rispondere: "non ho mai lavorato o conosciuto un ragazzo che uscisse fuori da character animation". Sa che dura "un anno e mezzo, se non ricordo male", e che alcune cose di questo corso forse non torneranno utili. "Però più cose sai, meglio."

Il riepilogo. Oggi sono "in 4, cioè in 5". Il docente ricorda la volta scorsa: il landscape, il suo materiale, "tutte le cose che si possono fare per spawnare l'erba" e i diversi layer. "Visto che mi era stato chiesto di andare a vedere un po' gli automaterial, o perlomeno darvi una base di come potremmo andare a fare gli automaterial", si è preparato "due o tre cosine". Condivide lo schermo; "come sempre, per qualche motivo, la prima volta non mi prende la testa del mouse" (il puntatore): "inizio a pensare che sia un problema del mio mouse".

1.15.2 L'automaterial: la scena di prova

Che cosa si vede. La scena è diversa dall'altra volta. C'è un "montarozzo" con un po' d'erba, e il terreno è diviso in automatico: **sabbia** a terra, **roccia** sulle pareti e una transizione a **erba** salendo, con l'erba (il foliage) che spawna andando verso l'alto. "Queste cose non sono cose che ho fatto a mano, ma sono cose che sono state automatizzate." La prova: in uno spazio vuoto il docente prende un pezzo di terreno con lo

Sculpt, lo tira su piano piano, e il terreno "fa una transizione in altri materiali a seconda dell'**altezza** e a seconda della **pendenza**". Anche passando l'Erosion, "il terreno si adatta e cambia".

E il Paint? Nella scheda Paint ci sono ancora i layer **Sand**, **Rock** e **Grass** della volta scorsa, più un layer nuovo, l'**autolayer**: "è quello con cui è stato coperto l'intero piano del landscape". In uno spazio remoto si può comunque ridipingere a mano la roccia, la sabbia e l'erba; dopo un po' ("Unreal e stream contemporaneamente sono un po' too much per il PC") attorno all'erba dipinta spawna anche il foliage. Il materiale, quindi, fa le cose da solo *e* si lascia ritoccare.

Il piano della spiegazione. Nella cartella *Environment/Landscape/Material* ci sono i materiali della volta scorsa, *M_LandscapeTest* e "queste altre due o tre prove". Il materiale assegnato al landscape è al momento "Auto Landscape 3", "il punto d'arrivo". Si parte dall'inizio, cioè dalla prima Material Instance, e si sale di complessità: tre versioni, riassunte nella tabella 1.15.1. ("Qui sotto è incredibilmente *crowded*": il docente lavora su uno schermo piccolo.)

Versione	Che cosa fa	Nodi chiave	Limiti
1 (pendenza)	roccia sui pendii, sabbia in piano	Lerp per attributo, World Aligned Blend, Blend Sharpness/Bias	non si dipinge, niente erba
2 (pendenza + altezza)	sabbia in basso, erba in alto, roccia sui pendii	Use Material Attributes, Blend Material Attributes, Absolute World Position, Subtract, Clamp, Divide	non si dipinge, niente foliage
3 (layer)	la 2 come layer *Auto*, più Grass, Rock, Sand dipingibili; erba che spawna solo dove ha senso	Landscape Layer Blend, Landscape Layer Sample, Add, Subtract, 1-x, Clamp, Landscape Grass Output	la sfumatura lascia un po' d'erba attaccata alle pareti

Table 1.15.1: I tre automaterial costruiti in progressione durante la lezione.

1.15.3 La prima versione: la pendenza con il World Aligned Blend

L'effetto. Con la prima Material Instance il materiale del landscape cambia, e "c'è effettivamente un po' di transizione fra una cosa e l'altra". Ma nella scheda Paint non si può più dipingere: "perché questo materiale che io ho messo è un materiale che non ha i layer del landscape", come il "semplice verde" della volta scorsa. In compenso, scolpendo (allargando una zona, ammorbidendola con lo Smooth), il terreno "in parte cambia" e ha "questo effetto che è un po' roccioso, un po' sabbioso, non pulitissimo". Tirando su con lo Sculpt, "c'è subito la roccia che sulla pendenza va a sostituire la sabbia".

I due parametri. Aprendo la Material Instance ci sono "due parametri che sono stati tirati fuori dal materiale" e che cambiano il modo in cui il terreno è visualizzato:

- il **bias**: "qual è la preferenza, cosa devo preferire: devo preferire la sabbia o devo preferire la roccia?";
- la **sharpness**: "quanto è netto il passaggio da una cosa all'altra".

Sono "due parametri che possiamo andare a utilizzare per fare un po' di *tuning*": cambiandoli, "ci sono degli scalini un po' più netti, un po' più separati".

Dentro il materiale. Il materiale, **M_AutoLandscape**, è "una rappresentazione un po' scorporata per farvi capire cosa sta succedendo": molte parti "in realtà non erano necessarie". Il docente non ha rifatto le mappe delle texture, "perché stanno già qui dentro le funzioni": le **material function** di StackOBot della volta scorsa, che hanno già la World Position, la texture, la normal, e restituiscono *Base Color*, *Roughness* e *Normal*. Chi non ha queste funzioni può fare il lavoro a mano: "vi prendete la texture che volete usare per il colore, la texture che volete usare per la normal, eccetera". (Uno studente chiede come abbia messo *MF_Sand* e *MF_Rock* nel grafo: "dal Content Drawer, prendi e trascini. *Easy peasy.*")

Per mostrare che cosa succede a ogni attributo, l'uscita di ogni funzione passa per un nodo **Break Material Attributes**, che la divide nelle sue parti: Base Color, Roughness e Normal. Le funzioni sono due, **la sabbia e la roccia**, e per ogni attributo si fa "una cosa che avete già visto succedere": un'**interpolazione lineare**.

Il Lerp, ripassato. "Vi ricordate il nodo Lerp come funziona?" Uno studente: "prende due colori e li associa... prende l'alfa di un'immagine e, in base a quanto è bianca e nera, ci mette un colore invece che l'altro". Il docente lo ripulisce:

- il Lerp non funziona "soltanto per le immagini": si può usare per operazioni matematiche, con un valore x, un valore y e il risultato interpolato;
- l'**alfa** "è praticamente il criterio che va a determinare l'interpolazione", "il criterio che determina se sarà prevalente A o prevalente B";
- nelle lezioni sui materiali si usava un'immagine perché un canale di una texture "è una mappa bianca e nera, in cui il nero vale 0, il bianco vale 1 e tutte le scale di grigi sono un valore fra 0 e 1". Con alfa 1 si ha A, con alfa 0 si ha B, e "tutte le scale di grigi vengono sfumature di B e di A".

Il criterio: il World Aligned Blend. Qui il criterio non viene da un'immagine: tutti gli alfa "vanno a confluire in un unico nodo", il **World Aligned Blend**. È un nodo che calcola "una serie di informazioni riguardo alle dimensioni dell'oggetto su cui il materiale è applicato in relazione al mondo". Lo fa con un **vettore che punta verso l'alto**, sull'asse Z: "immaginate che noi abbiamo questo piano; abbiamo il nostro vettore che va verso l'alto, e lui va a vedere in che modo l'oggetto su cui è applicato si relaziona rispetto al mondo per quanto riguarda quell'asse lì". Gli ingressi **Blend Sharpness** e **Blend Bias** sono i due parametri della Material Instance: "quanto è alto il valore dell'alfa rispetto al vettore", cioè quanto valgono numericamente le parti allineate al vettore, e "qual è il range all'interno del quale viene preferita una cosa piuttosto che un'altra".

Come si crea un parametro. Uno studente chiede come abbia fatto i parametri. Tre modi:

1. tenere premuto **S** e fare clic sinistro nel grafo: crea uno *Scalar Parameter*, a cui si dà il nome;
2. tenere premuto **1** e fare clic: crea una costante, che poi si trasforma con tasto destro, **Convert to Parameter** ("stessa cosa");
3. trascinare un filo da un ingresso del nodo e cercare *parameter*.

Il World Vector e il muschio. Di default il vettore del nodo punta verso Z, "però noi volendo lo possiamo modificare" dall'ingresso **World Vector**. Prova: un vettore con 1 sulla Y. Salvato il materiale, il terreno è roccia ovunque, "perché la roccia è il materiale di base da cui parte", e la sabbia compare solo sulle facce che puntano verso Y. Con 1 sulla X e 0 sulla Y, l'alfa si calcola solo verso X: la sabbia "è stretchata, è bruttissima, però ce l'abbiamo". A che serve? A cose più avanzate, come il *painting* sulle mesh o materiali particolari: "abbiamo un albero, il materiale dell'albero è di legno; tuttavia c'è una parte di **muschio**, e il muschio punta sempre verso nord". Con il World Vector si fa in modo che il muschio punti nella direzione voluta. Per il landscape serve invece ciò che è in piano, quindi Z.

Che cosa sta succedendo. "Su tutto ciò che va verso il vettore Z applica un materiale, che in questo caso è la sabbia. Perché è la sabbia? Perché la sabbia sta su A." L'1 è ciò che punta verso l'alto, lo 0 ciò che punta altrove; "se qualcosa punta parzialmente verso l'alto e parzialmente in un'altra direzione, è come se fosse una scala di grigio", e il materiale applica "un po' della sabbia, un po' della roccia". Su una parete a circa 45 gradi "la roccia sta sotto e la sabbia va a sporcare un pochino il resto" (figura 1.15.1). "Chiaramente è un effetto che si potrebbe fare in maniera molto più complicata": questo è "il modo più semplice". Si fa un Lerp "fra due funzioni, fra due texture, fra due quello che volete, anche due colori": con un rosso su A e un viola su B, il rosso andrebbe verso l'alto e il viola sulle altre direzioni. I due parametri cambiano "il range di disponibilità di A rispetto a B", "quanto il bianco viene visto rispetto al nero". Scolpendo, i lati diventano roccia e la cima resta sabbia.

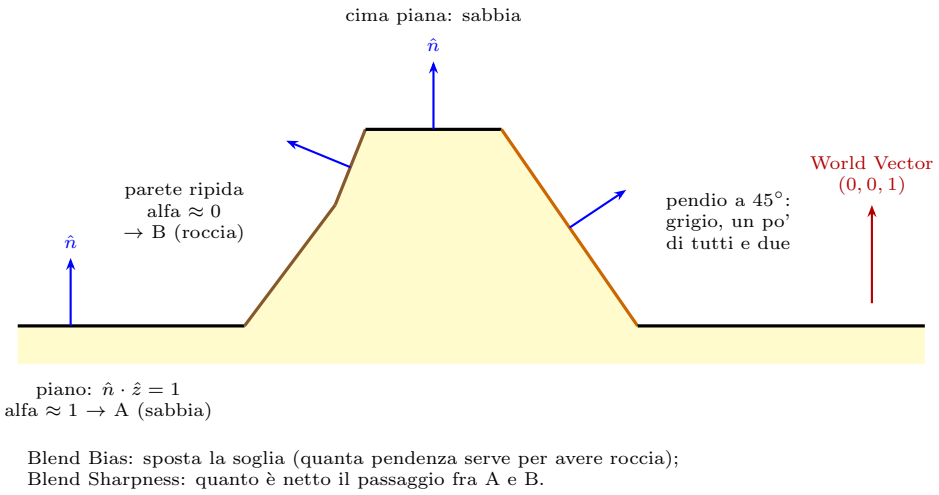


Figure 1.15.1: Il principio del World Aligned Blend nella prima versione dell'automaterial. Il nodo confronta la normale del terreno con il World Vector (di default l'asse Z): dove coincidono l'alfa vale 1 e il Lerp sceglie A (la sabbia), sulle pareti verticali vale 0 e sceglie B (la roccia), nei pendii intermedi mescola. Ricostruzione schematica.

Le uscite del nodo e il problema di Federico. **Federico**, che sta seguendo in Unreal, dice che a lui "per qualche motivo non funziona": vede tutto sabbia. Prima di guardare, il docente spiega le uscite del World Aligned Blend. Passando il mouse sopra, il tooltip dice che l'**Alpha** "è il risultato migliore, ma non funziona con la normal map". "Per usare le normal bisogna usare le altre due":

- **w/ Vertex Normals** (con le normali dei vertici) è "la versione base, quindi funziona in generale";
- **w/ Explicit Normal** richiede una normal map specifica, collegata all'ingresso *Explicit Normal*: il blend si fa sulla base di quella. Se la roccia "deve essere il materiale preponderante perché sta sotto" e la sabbia "deve adattarsi alla roccia", usare l'Explicit Normal con la normal della roccia darebbe "un risultato più bello".

Il problema di Federico, però, è un altro: il docente ha i parametri a 0 e 0 ma ha messo sul landscape la *Material Instance*, e ne modifica i valori lì. Federico invece ha assegnato il materiale: "io ho creato un material, non ho fatto una material instance. Ecco perché".

Approfondimento: il World Aligned Blend e la normale. Il *World Aligned Blend* è una *material function* dell'engine (non un nodo nativo). In sostanza calcola il prodotto scalare fra la normale della superficie, nello spazio del mondo, e il *World Vector*: $d = \hat{n} \cdot \hat{v}$, che vale 1 quando la superficie guarda nella direzione del vettore, 0 quando gli è perpendicolare e -1 quando guarda all'opposto. Al valore viene sommato il *Blend Bias*, il risultato viene moltiplicato per la *Blend Sharpness* e limitato fra 0 e 1: il bias

sposta il punto in cui avviene il passaggio (con un bias negativo serve più "piattezza" per avere A, ed è anche il modo per invertire l'effetto, come il docente farà nella seconda versione), la sharpness rende la sfumatura più o meno stretta. Con entrambi a 0 l'alfa è sempre 0 o quasi, ed è il motivo per cui, con i parametri a zero, si vede un materiale solo. Le tre uscite differiscono per la normale usata: *Alpha* usa la normale per pixel, comprensiva della normal map, ed è la più precisa, ma proprio per questo non può alimentare l'ingresso *Normal* del materiale (si creerebbe una dipendenza circolare: la normale dipenderebbe da se stessa); *w/ Vertex Normals* usa la normale geometrica dei vertici; *w/ Explicit Normal* usa una normal map passata a mano. Funzionamento generale dell'engine; il libro di Romero e Sewell non tratta le material function del landscape.

1.15.4 La Material Instance e i suoi parametri

Creare l'istanza. "È giusto che sia material": il materiale resta, e da lì si fa tasto destro, **Create Material Instance**. Unreal la chiamerebbe *M_AutoLandscape_Inst*, "però togliete l'*instance* alla fine e mettete **MI_**": *MI_AutoLandscape*. Il motivo è pratico: "se dovete cercare, fate direttamente MI e vi trova tutte le material instance".

Perché lavorare sull'istanza. I parametri si potrebbero cambiare anche nel materiale, "però è un po' scomodo, perché tutte le volte dovete salvare, applicare, caricare" (cioè ricompilare il materiale). Dalla Material Instance, invece, si modificano i valori e il landscape si aggiorna subito: il docente affianca la finestra dell'istanza al viewport ("ce la mettiamo qua") e lavora così. Nell'istanza i parametri compaiono "grigi", sotto **Global Scalar Parameter Values**: bisogna spuntare la casella accanto a ognuno per poterlo modificare. "Vedi che a me, anche se questi stessi erano a zero, è tutta sabbia, perché non ha altri riferimenti, non ha dei valori numerici che ti dicono dove deve stare la pendenza e dove no." Modificandoli, "delle zone dovrebbero venirti in pendenza e delle zone orizzontali: le zone orizzontali hanno un colore, le zone in pendenza un altro". Federico: "Sì, eccole. Funziona."

I nomi. Il docente li ha chiamati **Blend Sharpness** e **Blend Bias**, "perché sono letteralmente quello che sono", ma "potete dargli i nomi che volete". Scolpendo, le zone "si adattano di conseguenza".

Il "però": la neve. "Però c'è un però." Potremmo volere una **montagna** con l'erba nelle parti piane e la roccia sulle parti in pendenza; ma la montagna è molto alta, "quindi in altezza vogliamo andare a mettere la **neve**". Per questo non basta la pendenza: serve anche l'**altezza**. È la seconda versione. "Tutto chiaro per tutti il discorso pendenze?" "Sì, sì."

Approfondimento: materiale e Material Instance. Una *Material Instance Constant* è un asset figlio di un materiale: ne condivide lo shader già compilato e può cambiare soltanto i valori dei *parametri* esposti (scalari, vettori, texture, interruttori statici). Per questo modificarla è immediato, mentre ogni modifica al grafo del materiale richiede una nuova compilazione degli shader, che su un landscape è lenta; ed è anche il motivo per cui un materiale senza parametri non ha niente da regolare nell'istanza. Lo stesso materiale può avere molte istanze (un landscape desertico, uno alpino), ognuna con i propri valori. Il prefisso *MI_* è una convenzione di nomi diffusa (con *M_* per i materiali, *MF_* per le material function, *T_* per le texture), la stessa che si ritrova negli asset di StackOBot. Romero e Sewell, nel capitolo 5 ("Object Interaction with Blueprints"), creano nel Material Editor un *VectorParameter* e degli *ScalarParameter* (per esempio *Metallic*) proprio perché i parametri sono ciò che si può modificare senza toccare il grafo; le Material Instance del landscape si basano sul funzionamento generale dell'engine.

1.15.5 La seconda versione: la pendenza e l'altezza

L'effetto. Con la seconda Material Instance succede "esattamente la cosa che succedeva prima": sulle parti verticali c'è la roccia. Ma ora in basso c'è la **sabbia**, in alto l'**erba**, e sulla pendenza una transizione fra

erba, roccia e sabbia. "Poi per carità, sta transizione magari è anche un po' troppo morbida, però possiamo modificarla facilmente."

Il grafo semplificato. Il docente ha tolto la suddivisione per attributi della prima versione: "non ci serve più, perché ve l'ho fatta vedere già". Ora si può "comprimere il materiale risultante" con la casella **Use Material Attributes** nei dettagli del nodo del materiale: "se cliccate qui, abbiamo tutti i material attribute" in un unico pin. E invece del Lerp attributo per attributo si usa un nodo che "fa praticamente tutto questo lavoro con un nodo solo", il **Blend Material Attributes**: "prende tutti gli attributi di un oggetto e dell'altro oggetto e li blenda", dove per attributi si intendono "base color, roughness, metallic, specular, emissive eccetera". Si collega "cosa volete nel punto A, cosa volete nel punto B, e l'alfa", che "è come sempre il criterio". Lo schema completo è nella figura 1.15.3.

La roccia sui pendii. Uno studente riassume l'ultimo nodo: "è quello che prende l'inclinazione e lo sfuma in base al materiale". "Esattamente." Qui A è la **roccia**, B "è il resto", e l'alfa viene dal World Aligned Blend: "dove c'è inclinazione metti la A, dove non c'è inclinazione metti l'altro". È il contrario della prima versione (dove A era la sabbia in piano): "mettendo il parametro in negativo lui fa l'opposto". Il docente ammette: "ho fatto il casino rispetto a quella di prima".

Sabbia ed erba per altezza. Prima di arrivare alla roccia, un altro Blend Material Attributes mescola **sabbia** ed **erba**; il risultato è il B dell'ultimo blend. "Qual è il criterio che decide dove c'è la sabbia, dove c'è la terra e dove c'è un blend di queste due?" Un calcolo in quattro passi:

1. l'**Absolute World Position**: "come ci dice Unreal, va a calcolare la posizione del pixel specifico nella posizione del mondo", quindi la posizione sull'asse X, sull'asse Y e sull'asse Z ("più che l'altezza"). "Si usa per un tot di cose": anche nelle material function di StackOBot, per una sorta di *world align*. Qui serve solo la **Z**: "dobbiamo prendere un punto nello spazio verticale e renderlo una discriminante";

2. una **sottrazione**: alla Z si sottrae un parametro, **Blend Height**, "l'altezza in cui dovrebbe succedere il blend". Con 0 il punto è lo zero; con 1000, "abbiamo 1 in tutto quello che c'è sopra e 0 in tutto quello che c'è sotto": "sopra 1000 abbiamo un oggetto, sotto il 1000 abbiamo un'altra cosa";

3. un **Clamp**: "prende un valore e lo costringe fra due estremi". Il docente lo paragona ai parametri che vanno da 0 a 1: "non vi sembra strano che la roughness, un valore che modifica così tanto, abbia 0 come inizio e 1 come fine? Perché viene clampato il valore". Qui il minimo è 0 e il massimo è un altro parametro, **Blend Width**, "la sfumatura": "sopra il Blend Height deve esserci una cosa, sotto un'altra cosa, e con questo calcolo gli stiamo dicendo quanto deve essere grande la sfumatura". All'inizio era 4000, "poi l'ho modificata perché era troppo alta": 1500;

4. una **divisione** per la stessa larghezza e un secondo **Clamp** fra 0 e 1, "in modo che adesso gli stiamo dando un alfa che è 0–1", "perché lui come alfa legge 0 e 1". Senza la divisione, quell'1 "potrebbe essere 4000, nel nostro caso 1500".

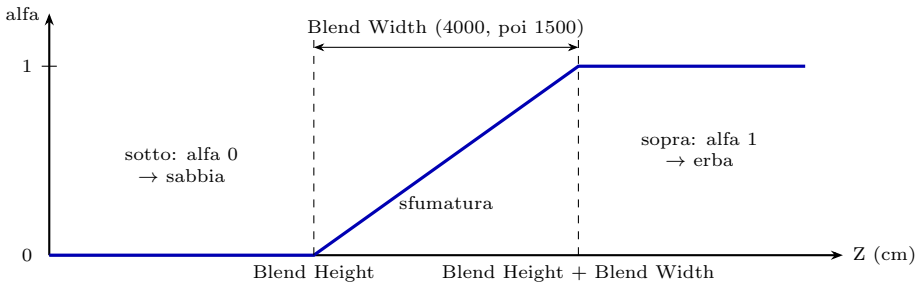
"È chiaro il calcolo? Se potessi disegnare, sarebbe più comodo." Uno studente: "a me sembra chiaro". La curva che ne risulta è nella figura 1.15.2.

L'effetto dei parametri. "Vi faccio vedere l'effetto, così è più chiaro."

- **Rock Blend Bias**: "quanta roccia c'è", "quanto deve essere forte il bias verso la roccia o verso l'altro materiale", che è sabbia ed erba. Per comodità il docente lo lascia a zero ("mi è utile per farvi vedere l'altra roba");

- il **Blend Height** "determina quanto in alto parte la transizione fra sabbia ed erba";

- il **Blend Width** "determina quanto velocemente", cioè "quant'è lo spazio di transizione".



alfa = clamp(clamp(Z − Blend Height, 0, Blend Width) / Blend Width, 0, 1)

Figure 1.15.2: La fusione per altezza della seconda versione. Sotto il Blend Height l'alfa vale 0 (sabbia), sopra il Blend Height più il Blend Width vale 1 (erba), in mezzo cresce linearmente. Il Blend Height sposta il punto in cui comincia la transizione, il Blend Width la allarga o la stringe. Ricostruzione del calcolo descritto a lezione.

Per far vedere la larghezza il docente vorrebbe fare "una roba molto brutta", poi uno studente suggerisce: "oppure una **rampa**". "Sì, effettivamente la rampa è la voce della saggezza." Con lo strumento Ramp fa una rampa molto alta ("chiaramente si spacca tutto, a noi serve guardarlo soltanto da questo punto di vista"). Alzando il Blend Height la transizione parte più in alto; alzando il Blend Width parte sempre "nello stesso punto" ma "piano piano inizia a sfumare", "diventa sempre più rossa" (l'erba del pacchetto è rossa), "sempre meno gialla", e poi è erba del tutto: "quello che sta facendo è letteralmente ampliando la sfumatura". Portandolo sotto 1500 la transizione è molto netta; a "0 virgola qualcosa" è "talmente piccola che non c'è", "però l'altezza è sempre quella". (Il docente chiede anche agli studenti se vedano la finestra dell'istanza che sposta sullo schermo: "mi è venuto il dubbio che prendesse soltanto la prima finestra". La vedono. In un paio di punti la trascrizione riporta valori confusi, "era 5, è di quarta", "meno 5,5", che non si riescono a ricostruire: con ogni probabilità sono i valori del bias della roccia, negativo come detto sopra.)

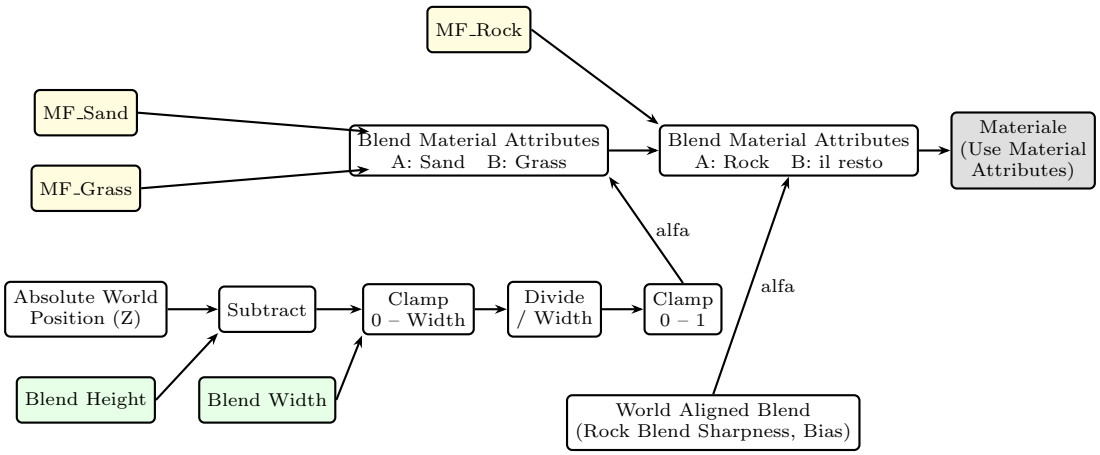


Figure 1.15.3: La seconda versione dell'automaterial. Sabbia ed erba si mescolano in base all'altezza (in basso, il calcolo della figura 1.15.2); il risultato si mescola con la roccia in base alla pendenza (World Aligned Blend). Tutto viaggia come *material attributes* grazie a Use Material Attributes. Quale materiale stia su A e quale su B nel primo blend dipende dal segno dei parametri: la lezione non lo chiarisce, e lo schema segue l'effetto visto (sabbia in basso, erba in alto). Ricostruzione schematica.

Approfondimento: la stessa curva con meno nodi. Il calcolo del docente è una *rampa lineare* fra due quote, e si può scrivere in un solo passaggio: alfa = saturate$\big((Z-H)/W\big)$, dove *Saturate* è il nodo che limita

fra 0 e 1 a costo quasi nullo; il primo Clamp fra 0 e W diventa superfluo. Altre due osservazioni. L'Absolute World Position è in centimetri, quindi un Blend Height di 1000 è a dieci metri sopra lo zero del mondo, *non* sopra il punto più basso del landscape: se si sposta il landscape in verticale, la linea della sabbia resta dov'è. E una sfumatura perfettamente orizzontale si nota come una "linea di marea" innaturale: di solito la si spezza sommando alla Z un po' di rumore (una texture di *noise* campionata in world space), così che il confine fra sabbia ed erba salga e scenda. Il caso della *neve* citato dal docente è identico: un terzo blend, con alfa dato da un'altra quota, sopra tutto il resto. Funzionamento generale dell'engine.

1.15.6 Unire e separare gli attributi

La domanda. Uno studente: "l'unica cosa che mi manca è come hai fatto a fare un output con un singolo nodo". È la casella **Use Material Attributes**, "che ti contrae tutto quanto in modo che non devi andare a mettere singolarmente Base Color, Metallic, Specular, Roughness": "le informazioni di tutti gli attributi vengono storate e confluiscono tutte lì". Dentro le funzioni c'è un **Set Material Attributes**, che "prende discolor, roughness, normal" (Base Color, Roughness, Normal) "e li fa diventare attributi": "unifica la stringa di dati". Poi il Blend Material Attributes "blenda questo e questo, blenda questo con quello che c'era prima", e il risultato arriva al nodo finale senza dover prendere i dati uno per uno.

Quando bisogna separare. Non sempre basta. "Se volessimo mettere, per esempio, la **visibility mask**" della volta scorsa, per cui il materiale deve essere *Masked* e la visibilità va nell'Opacity Mask, "non possiamo fare questa cosa" con il solo filo degli attributi: "abbiamo bisogno di andare a scorporare" gli attributi. Si mette un **Break Material Attributes** dopo l'ultimo blend; "ci abbiamo base color, roughness che va nella roughness, normal che va nella normal", e a parte si prende la *Landscape Visibility Mask* e la si collega dove serve. "Chiaro il discorso di unire, disunire gli attributi?"

L'errore di Federico. Federico ha un errore di compilazione: *the current material has the following compilation errors at feature level SM6.* "Probabilmente è l'alfa dentro una normal": ha usato l'uscita *Alpha* del World Aligned Blend anche per la Normal, invece della *Vertex Normals.* "Sì, giusto. Perfetto, grazie."

Il problema della seconda versione. "Ora però c'è un problema." Con un materiale così non si ha più il controllo di ciò che succede: "non posso dipingerlo io". Non si può dire "qui voglio aggiungere una parte in cui c'è dell'erba, o della roccia". "Per alcuni casi può anche andare bene": "il mio materiale l'ho fatto talmente bene, talmente automatizzato, che va tutto in automatico". Ma manca anche un'altra cosa: senza layer non si può usare il **Landscape Grass Output**, quindi niente foliage, "e infatti l'erba è un piano, non c'è l'erba". Si risolvono entrambi i problemi con la terza versione.

Approfondimento: gli attributi del materiale e SM6. *SM6* è lo *Shader Model 6*, il livello di funzionalità di DirectX 12 usato di default da Unreal Engine 5 su PC: l'errore dice semplicemente che il materiale non compila per quella piattaforma, e il messaggio dettagliato (nella finestra *Stats* del Material Editor) indica il nodo. Il caso tipico è proprio quello di Federico: una normale che dipende dalla normale per pixel. Quanto agli attributi, con Use Material Attributes il materiale ha un solo ingresso, ma la struttura che vi arriva contiene *tutti* gli attributi, compresa l'Opacity Mask: per aggiungere la visibilità basta anche un *Set Material Attributes* finale con un solo pin, *Opacity Mask*, che riscrive quell'attributo e lascia gli altri come sono. La via del Break Material Attributes mostrata a lezione resta la più leggibile quando si vogliono modificare più attributi a valle. La Blend Mode *Masked* va comunque impostata nei Details del materiale. Funzionamento generale dell'engine.

1.15.7 La terza versione: i layer e l'erba

L'effetto. Con il terzo materiale (*Auto Landscape 3*) il terreno è "esattamente come prima", ma ora c'è l'erba sulle parti rosse, che sfuma "dove c'è ancora del rosa verso la sabbia"; la sabbia "diventa tutta sabbia, non c'è più erba". "L'erba non cresce sulla sabbia, non cresce sulla roccia." E si possono di nuovo scegliere

i layer da dipingere: "posso far diventare tutta roccia" una zona, "perché ho bisogno di una parte che sia tutta rocciosa", oppure creare "una piccola **oasi**": una zona tutta erbosa, "con magari un po' di sabbietta nella parte centrale", da scavare verso il basso per metterci "un po' d'acqua".

La struttura. "Stavolta non c'è una material instance, perché non era necessaria. Ed è un po' più complicato." Si isolano prima le cose note:

- tutta la struttura della seconda versione, "letteralmente la stessa": sabbia ed erba che si fondono per altezza, la roccia che si fonde con il resto per pendenza. "Sono cose che potete adesso aggiungere alla cosa che stavate facendo";
- il risultato entra in un **Landscape Layer Blend**, in un layer chiamato **Auto**;
- nello stesso Landscape Layer Blend ci sono gli altri layer, **Grass**, **Rock** e **Sand**, con collegate le rispettive material function.

Il layer Auto è quello steso su tutto il landscape (con il riempimento, *Fill Layer*); gli altri si dipingono sopra dove servono.

I reroute node. "Non so se funziona come con i Blueprint, ma non so se l'avete mai visto": invece di avere un filo "che va così e attraversa tutto quanto ed è brutto", si fa **doppio clic** su un punto del filo, che crea un "nodino" (un *reroute node*); lo si sposta, se ne crea un altro, "in modo da avere un po' più di ordine" quando "tutto rischia di diventare spaghetti". "Spaghetti coding, diciamo."

L'erba: il primo tentativo. "L'unica cosa che ci manca è decidere come spawnerà l'erba." Il Grass Type è sempre **GT_Grass**, quello della volta scorsa. Se si collega direttamente il *Landscape Layer Sample* del layer Grass al Landscape Grass Output, "dove abbiamo dipinto con il layer grass l'erba c'è, ma dove abbiamo il grass nell'automaterial non c'è più". (Il caricamento è lento: "sta caricando", "si è spaccato tutto quanto"; poi compare.) "Perché? Perché gli abbiamo detto: dove c'è il layer grass, metti l'erba. Ma non è quello che ci serve a noi."

Il secondo tentativo. Si potrebbe collegare anche il layer Auto, con un altro Landscape Layer Blend o sommando i due campioni. "Cosa può succedere secondo voi?" Succede che l'erba "spawnerà su tutta la mappa, anche nei punti in cui non vogliamo": sulla sabbia e sulle rocce, "perché gli abbiamo detto: mettila sull'automaterial, mettila sul grass material".

La soluzione. "Quello che vogliamo è che lui prenda come alfa l'erba e i punti in cui non deve esserci la roccia", meno quelli in cui c'è la sabbia:

1. si **somma** il peso del layer **Grass** ("ogni volta che io passo con il pennello e pinto grass") e i punti **senza roccia**, "i punti che non sono in pendenza". Per questi si usa "praticamente lo stesso alfa che abbiamo utilizzato per determinare la pendenza": il World Aligned Blend della roccia, preso dalla parte "non roccia";
2. dalla somma si **sottraggono** i punti con la **sabbia**, usando "la discriminante per altezza", cioè l'alfa del calcolo per altezza;
3. la prima volta il risultato era invertito: "mi ha messo l'erba sulla sabbia e l'ha tolta da qui sopra". Il docente ha quindi inserito un nodo **1-x** (*OneMinus*), "che praticamente inverte il valore": da x fa $1 - x$. Così "mettila in tutte le parti che non sono né in pendenza né in basso";
4. un **Clamp** finale riporta il valore fra 0 e 1, e si collega al Landscape Grass Output, "per dirgli dove andare a metterlo".

Lo schema è nella figura 1.15.4.

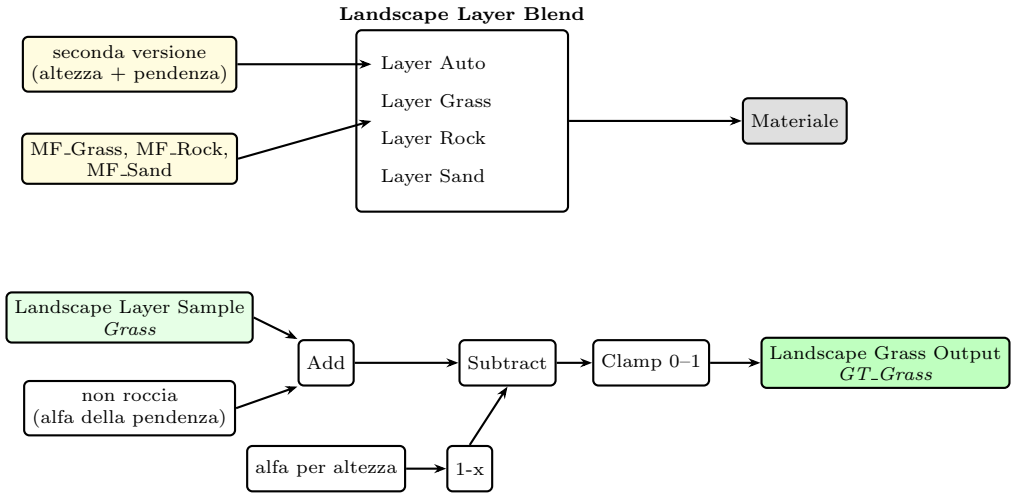


Figure 1.15.4: La terza versione. In alto, la parte che dipinge: la seconda versione diventa il layer *Auto* di un Landscape Layer Blend, accanto ai layer dipingibili. In basso, il calcolo che decide dove spawna l'erba: il layer Grass dipinto più le zone non in pendenza, meno le zone basse della sabbia (con l'inversione 1-x), limitato fra 0 e 1. Ricostruzione schematica: la trascrizione non permette di stabilire con certezza a quale ingresso del Subtract arrivi ogni filo, e lo schema segue la descrizione a parole del docente.

La domanda sulla somma. Uno studente: "la sottrazione credo di averla capita, non ho capito la somma, cosa sta sommando". Il docente rispiega: si somma "l'intero layer grass", perché l'erba deve spawnare sempre dove si dipinge Grass, e "tutti i punti che non sono in pendenza, quindi tutti i punti che non sono rock". "Questo alfa, andando qui, determina dove deve andare la roccia e dove deve andare il resto": si prendono i punti in cui non c'è la roccia, "li sommiamo all'erba e poi ci sottraiamo il resto". Lo studente: "e con la sottrazione togliamo la sabbia". "Perfetto."

La prova. Senza la sottrazione, "mette l'erba anche sulla sabbia, ma non la mette sulla roccia". Con la sottrazione resta solo nella parte erbosa. Un difetto rimane: in alcuni punti l'erba "sta attaccata alle pareti", perché la sfumatura del blend è morbida; "qui poi potremmo andare a usare altri nodi per avere un blend meno morbido, più netto". "Gli altri, oltre a Federico, hanno seguito? Si sono abbioccati a un certo punto? Hanno rinunciato?" "Ci sono."

Che cosa ci si può fare. "Smanettando un pochino con un sistema del genere, potete fare un sacco di cose": decidere dove e in che forme spawni l'erba, e "10.000 cose" che vanno "un po' oltre lo scopo" del corso ("già forse facendo questo automatico siamo andati leggermente oltre lo *scope*"). "Però è una cosa figa e che vi velocizza di molto il lavoro": se il compito fosse "una piccola mappina open" in cui il personaggio esplora e trova dei collezionabili, non servirebbe dipingere nulla, perché "avete già tutto pronto nel momento in cui fate lo sculpting". Il foliage a mano resta utile per personalizzare. Un'ultima nota, non verificata: in altri materiali "presi in giro nel corso del tempo" il docente ha visto che si può **cancellare lo spawn dell'erba** usando la *visibilità*, per avere il colore dell'erba senza il foliage; "dovrei vedere un attimo effettivamente come farlo", e per ora si chiude qui.

L'Erosion e le ombre. Prima di passare oltre, un'osservazione: l'**Erosion** fa un effetto "molto, molto carino", ma "ti lascia ste ombre strane, sti poligoni spezzati". Dopo averla usata "tocca sempre passare un po' col noise o con lo smooth".

Approfondimento: pesare l'erba senza effetti collaterali. Il calcolo del docente lavora su pesi fra 0 e 1, e le operazioni hanno un significato preciso: la somma è un "o" (erba dove c'è il layer Grass *oppure* dove non c'è pendenza), la sottrazione un "ma non" (tranne dove c'è sabbia), il Clamp evita valori sopra 1 o sotto 0, che il Landscape Grass Output interpreterebbe come densità oltre il previsto o nulle. Un'alternativa più robusta è moltiplicare invece di sottrarre, erba = max(Grass, piano · alto), che non può mai scendere sotto zero; e per evitare l'erba attaccata alle pareti si può "stringere" l'alfa della pendenza con una *potenza* o uno *SmoothStep* prima di usarlo. Due attenzioni pratiche. Il layer *Auto* conta nei pesi come gli altri: dipingendo Rock in un punto, il peso di Auto scende e con lui l'erba automatica, che è il comportamento voluto. E ogni layer in più su un componente costa campionamenti di texture: un automaterial ben fatto spesso *riduce* il numero di layer dipinti, e quindi il costo. Il libro di Romero e Sewell non tratta questi nodi; funzionamento generale dell'engine.

Approfondimento: la regola del paesaggio. L'automaterial traduce in nodi una regola "naturale": la roccia affiora dove il pendio è troppo ripido per trattenere il terreno, la sabbia si deposita in basso, l'erba cresce sopra. Totten, nell'intervista a Greg Grimsby alla fine del capitolo 4 ("Teaching in Levels through Visual Communication"), ne fa una "legge del level design": capire l'anatomia del proprio ambiente e costruirla nel mondo; "le foreste hanno alberi, gli alberi muoiono, quindi le foreste hanno alberi morti; l'erba è rada sotto gli alberi, dove la luce non arriva". E se il mondo è inventato, "progettate il vostro insieme di regole e rispettatelo". Un automaterial è esattamente un insieme di regole rispettato in automatico; il layer dipinto a mano (l'oasi del docente) è l'eccezione voluta, che proprio perché rompe la regola diventa un punto di interesse.

1.15.8 Due correzioni sulle luci: Lumen e i light shaft

"Se avete delle domande adesso, me le potete fare. Se no vi faccio vedere una cosina": quando ha parlato di luci, il docente ha detto "una cosa che non era propriamente corretta". Due correzioni.

La prima: Lumen. La volta delle luci (la trascrizione dice "la volta scorsa, parlando di MPC": il riferimento è storpiato, ma l'argomento è la lezione sulle luci) aveva spiegato la differenza fra **static**, **stationary** e **movable**, "le luci che vengono bakeate, tutte queste cose, tutto molto bello". "Ma mi ero dimenticato una cosa fondamentale: queste cose valgono solamente se non stiamo utilizzando **Lumen**." Nei **Project Settings**, nella sezione **Global Illumination** e **Reflections**, "di default c'è Lumen, perché è il default di Unreal 5". Mettendo **None** si torna "all'illuminazione con il building e il baking" di cui aveva parlato; "visto che di default sta su Lumen, teoricamente è una cosa di cui non dovete preoccuparvi". Ma se si sviluppa per piattaforme meno potenti di un PC, "un telefono, una console che non ha chissà quanta potenza, o un PC di fascia bassa", si può voler disabilitare Lumen e tornare "alla vecchia struttura delle luci con il baking".

La seconda: i god rays con la directional light. Aveva parlato dei **god rays** ("god shaft") pensando "soltanto al modo complesso", il post process. "C'è un modo per farli relativamente veloce, soltanto con la directional light": giocare con i parametri **Light Shaft Occlusion** e **Light Shaft Bloom** della directional. Di default "l'occlusion è relativamente bassa, il bloom anche è molto basso"; c'è un valore di *brightness* "al massimo", e il **threshold**, che "è praticamente quanto deve essere alto il valore del colore della scena per creare gli shaft": "una soglia; se superi quella soglia, lui crea lo shaft, se no, non lo fa". Alzando un po' i valori e girando la luce verso la camera, compare "questo effetto stile god ray" quando si guarda verso l'alto, "perché c'è un bloom molto elevato". "Chiaramente il sole vi distrugge gli occhi."

La prova nella stanza. Per un esempio ("brutto, eh?") il docente tira su un pezzo di landscape a fare da stanza e ci entra: "se riesco a beccare la direzione, ecco qua l'impressione del raggio divino che entra dentro, se guardate dal giusto angolo". A questo si può aggiungere "un po' di **fog**". Usi possibili: con l'inclinazione della camera e della luce, illuminare alcune parti nello specifico; o, con valori molto luminosi, "una roba desertica", l'impressione di una forte luminosità esterna "e un po' di distorsione del calore". All'aperto "diventa un po' una schifezza, non è il massimo"; all'interno funziona meglio, e "tanto poi tendenzialmente i god rays li usate principalmente per fare delle schermate, delle scene, delle foto".

Approfondimento: Lumen, la mobilità e i light shaft. La correzione del docente va letta con una sfumatura. Nei progetti di Unreal Engine 5 il metodo di illuminazione globale dinamica (*Project Settings* → *Engine* → *Rendering* → *Global Illumination* → *Dynamic Global Illumination Method*) e quello dei riflessi (*Reflection Method*) sono su **Lumen**: l'illuminazione indiretta è calcolata in tempo reale e il build non serve. La *Mobility* delle luci però non diventa irrilevante: con Lumen attivo le lightmap precalcolate vengono ignorate, le luci *static* (che esistono solo nelle lightmap) non contribuiscono all'illuminazione dinamica, e conviene usare luci *stationary* o *movable*; la mobilità continua a decidere il costo delle ombre e se una luce si può muovere o cambiare in gioco. Con *None* si torna a Lightmass, al build e a tutto ciò che il docente aveva detto nella tredicesima lezione. Quanto ai light shaft: nei Details della directional light, sezione *Light Shafts*, *Light Shaft Occlusion* scurisce la nebbia e l'aria dove la luce è occlusa (i "raggi" scuri fra le fessure), *Light Shaft Bloom* aggiunge un alone luminoso attorno al disco del sole; *Bloom Scale*, *Bloom Threshold* (la soglia di luminosità oltre cui un pixel contribuisce, quella descritta dal docente) e *Bloom Max Brightness* ne regolano l'intensità. È un effetto *in screen space*: funziona solo quando la sorgente è dentro o vicino all'inquadratura, per questo il docente deve "beccare la direzione". L'alternativa più fisica è la nebbia volumetrica (*Exponential Height Fog* con *Volumetric Fog* attiva), in cui la luce diffonde davvero; il libro di Romero e Sewell non tratta né l'una né l'altra. Funzionamento generale dell'engine.

1.15.9 La Camera Actor e i coni di luce dell'engine

Una telecamera per una scena. Per le schermate c'è un procedimento: si prende una **Camera Actor** dal Place Actors, la si piazza dove si vuole, "che inquadra quello che vogliamo", e "ci si costruisce un livello davanti". "È una cosa che si fa con gli *speed level*" (con ogni probabilità lo *speed level design*, l'esercizio di comporre in fretta un ambiente, di solito per una sola inquadratura). Qualche dettaglio mostrato:

- selezionando la camera compare la sua **anteprima**, con l'intensità della luce regolata "in modo che vedo sempre effettivamente come la preview";
- ci si può "switchare" quando si vuole, guardando attraverso la camera (*Pilot*): "se vi muovete da così, spostate anche la camera"; per uscire c'è il pulsante apposito (l'*Eject*, "questo traccetto" nella trascrizione);
- poi ci si costruisce la scena "che preferisco": oggetti, estetica, luci, "fatto in un certo modo, eccetera".

Lo speed level design "si usa principalmente per cose estetiche": immagini (la trascrizione dice "immagini capacitarie", non ricostruibile: probabilmente immagini di copertina o di presentazione), screenshot, "robe particolarmente carine". (Una frase, "si è sommerso il mask, quindi a distanza", è troppo rovinata per essere interpretata.)

I coni di luce dell'engine. "C'è effettivamente un altro modo" per i god rays negli interni: degli asset "dall'engine, quindi delle cose che sono direttamente a disposizione di qualsiasi progetto". Il docente li cerca (il nome nella trascrizione è "pin", non ricostruibile) e trova delle *static mesh* con materiali fatti apposta: "un cono che fa finta di essere un raggio", "le classiche luci dei lampioni": "passate sotto il lampione, vedete la lucina". "Non sono di qualità incredibile", ma si possono modificare: cambiare il materiale o il colore, "se avete un neon, lo potete far diventare viola".

Copiare, non spostare. Invece di usarli direttamente dall'engine, "potete copiarveli" (Ctrl+C, o dal Content Browser) "e incollarli dentro al content del vostro progetto". Il motivo: "se per sbaglio cancellate qualcosa dall'engine potreste creare problemi al modo in cui Unreal funziona, perché magari non trova delle reference". Quindi "non prendete e trasportatele nel content da qui, ma copiatele" in una cartella del progetto. È lo stesso principio del cubo con il pivot della volta scorsa: gli asset dell'engine non si toccano.

Approfondimento: inquadrare una scena. Gli asset dell'engine si vedono nel Content Browser attivando *Show Engine Content* nelle impostazioni di visualizzazione; copiarli nel progetto (*Duplicate* o copia e incolla in una cartella propria) crea asset indipendenti, che si possono modificare senza rischi e che vengono

inclusi nel packaging. I coni "finti" sono mesh con un materiale *translucent* o *additive* che sfuma verso i bordi: costano molto meno della nebbia volumetrica e sono il trucco classico dei giochi meno recenti. Quanto alla composizione, Totten nel capitolo 4 dedica una parte al *framing*: elementi verticali come alberi, colonne o stipiti incorniciano una scena e rendono più drammatici gli arrivi in un luogo; e alla *regola dei terzi*, che divide l'immagine in nove parti e mette i soggetti importanti sulle linee e sulle loro intersezioni. L'intervista a Greg Grimsby nello stesso capitolo aggiunge: usare primo piano, piano intermedio, orizzonte e parallasse per costruire panorami ricchi. Sono le regole con cui si compone l'inquadratura di una Camera Actor; la Camera Actor e il Pilot si basano sul funzionamento generale dell'engine.

Fine della prima parte. "Ci sono domande su queste ultime cose?" Nessuna. Il docente si ferma: "ci rivediamo alle 5:40" per il **Foliage Mode**, "15 minuti di pausa come le altre lezioni".

1.15.10 La pausa e il riepilogo per Edoardo

Durante la pausa la trascrizione contiene soltanto il blocco ripetuto già visto ("grazie a tutti per l'attenzione", "qual è la risposta che ci può essere?"), che non appartiene alla lezione. Alla ripresa il docente si accorge che **Edoardo** è entrato mentre spiegava ("sì, sono entrato verso le 5, avevo un impegno prima"). Gli riassume la prima parte: si è risposto alla domanda di Federico della volta scorsa, "se c'era un modo per far sì che il materiale del landscape si autocompilasse", "che si applicasse da solo senza doverlo andare a dipingere a mano", con diversi esempi e diverse aggiunte. "Chiaramente si può fare anche molto di più": "ve lo andate a spizzare un po' l'argomento, se vi interessa". Edoardo se la riguarderà. All'appello ci sono anche **Guido** e, in arrivo, **Tommaso**; **Filippo** viene nominato. "Ritorniamo in quel di Unreal Engine, che sicuramente di nuovo farà inceppare il mio mouse."

1.15.11 Il Landscape Grass Type, rivisto

"Cominciamo dalla fine": il foliage che il materiale mette in automatico "non è proprio il punto da cui dovremmo partire, ma ci eravamo infognati col discorso landscape, e quindi va bene così". Come la volta scorsa, l'aspetto dell'erba che spawna con il materiale si controlla dal **Landscape Grass Type** (nella cartella dei *Landscape Layers* del pacchetto, *GT_Grass*). Il docente passa in rassegna i parametri.

- **Grass Density**: abbassandola, "c'è meno erba, viene spawnata in maniera meno folta".
- **Use Grid**: le istanze spawnano su una griglia.
- **Placement Jitter** ("Gintera"): "un errore, una randomicità", per cui le cose "vengono piazzate in maniera più o meno uniforme". Abbassandolo l'erba "sembra molto più uniforme", "come se venisse piazzata tutta nella stessa direzione", più compatta; a **1** "c'è un po' più di randomicità": parti con meno erba, parti con più erba, che "simula un po' di più il manto erboso e non è un mantello compatto".
- **Start Cull Distance** ed **End Cull Distance**: "da quale distanza cominciamo a vedere che spariscono e a quale distanza smettono di essere renderizzate". Si può disabilitare il *culling* e avere tutto sempre renderizzato: "chiaramente pesa molto di più".
- **Min LOD**: il livello di dettaglio della mesh. A -1, il default, "va a mettere proprio la static mesh in tutto e per tutto, con tutti i dettagli"; con valori più alti, "versioni meno poligonali".
- La **scala**: qui va da 0,8 a 1,1. Il docente la porta "da 1 fino a 3, cosa molto stupida": alcune piante diventano molto grandi, altre restano piccole, "una distribuzione un po' poco omogenea". "Tre è un'enormità che sovrasta un po' tutto il resto". Il senso: "se il default è 1, quindi se l'oggetto è alto un metro, al minimo ve lo dà 80 cm e al massimo" il valore massimo. (Il docente dice "1,50 m", ma con l'intervallo letto, 0,8–1,1, il massimo sarebbe 1,10 m.) Cambiando lo **Scaling** da uniforme ad altri tipi si sbloccano le scale per asse: "potete farla più alta ma più stretta". "Idealmente non vogliamo mai strecciare gli oggetti che andiamo a piazzare, ma potrebbe essere."

- **Random Rotation**: se le istanze sono ruotate tutte nello stesso verso o a caso.
- **Align to Surface**: se sono allineate alla superficie. Sulla discesa, "vedete quest'erba va in diagonale perché è allineata alla superficie"; togliendo la spunta, cresce dritta. Con l'erba non si vuole quasi mai togliere, "ma magari se state usando questa cosa per piazzare degli alberi" è utile che crescano in verticale, "perché l'albero cresce cercando il sole", "un pino, un abete", per esempio in una tundra.
- **Cast Dynamic Shadow**: se le istanze castano ombre dinamiche. "Un parametro con cui dovete stare molto, molto attenti." Con una luce vicina l'erba reagisce e "dà un effetto più bello", ma "appesantisce di molto la scena".
- **Receives Decals**: "può essere comodo", ma le decal non si sono ancora viste ("potremmo farle l'ultimo giorno", insieme alle **spline**).

Che cosa sono le decal. "Un tipo di texture, un tipo di materiale che può essere applicato a delle superfici come un oggetto." L'immagine: "abbiamo un graffito e abbiamo questo volume, all'interno del quale tutto quello che ci passa gli si applica questo graffito sopra". Esempio: colpisco un nemico, "ci sono degli schizzi di sangue che vengono proiettati sulla parete"; vengono proiettati anche sull'erba vicina oppure no? È ciò che decide *Receives Decals*. Come le ombre dinamiche, sono cose che "appesantiscono più o meno la performance" e danno "un effetto estetico maggiore".

Approfondimento: il Grass Type e i costi. I valori letti a lezione aiutano a capire il sistema. La *Grass Density* è un numero di istanze per unità di superficie, pensato per fili d'erba piccoli: è un valore alto, che con mesh grandi esplode (lo "sproposito" di rocce della lezione precedente). Le istanze del Grass Type non sono salvate nel livello: vengono generate al volo attorno alla camera, a pezzi, entro l'*End Cull Distance*, e distrutte quando ci si allontana; per questo disattivare il culling "pesa molto di più" e per questo il Grass Type non genera collisioni (le istanze non esistono a distanza). *Min LOD* a -1 significa "nessun limite": si usano tutti i LOD della mesh a partire dal più dettagliato. Ombre dinamiche e decal hanno un costo per istanza, moltiplicato per migliaia di istanze: sull'erba di solito si spengono entrambe e si lascia che sia il terreno sotto a portare l'ombra (per esempio con l'ambient occlusion). Il libro di Romero e Sewell, nel capitolo 12 ("Building and Publishing"), elenca fra le impostazioni di scalabilità proprio la qualità del *Foliage*, insieme a *View Distance*, ombre ed effetti: sono i controlli con cui il giocatore abbassa questi costi; il resto è funzionamento generale dell'engine.

1.15.12 Il Foliage Mode: che cos'è

La definizione. "Ritornando all'inizio e parlando di foliage in sé per sé, quindi non collegato direttamente a un materiale." La **Foliage Mode** "è una modalità che ci permette di piazzare e randomizzare mesh, qualsiasi tipo di mesh che voi vogliate, senza doverle piazzare a mano una a una". "Nel momento in cui io devo andare a creare una foresta, non vado a mettere gli alberi a mano uno a uno": uso uno strumento che velocizza le cose.

Gli asset del pacchetto. Nella cartella *Foliage* di StackOBot ("si è scritto male, è scritto *foliage*", una battuta sul nome) ci sono:

- due oggetti che "sono già delle **static mesh foliage**": nel Content Browser "sono in verde, a differenza delle static mesh normali";
- le texture e i materiali, "che fanno delle cose molto carine, tipo fare oscillare le foglie al vento";
- le mesh: "letteralmente un albero, un'erba e un cespuglio".

Il problema da risolvere. Per fare "un contorno di alberi" alla scena senza il Foliage Mode bisognerebbe copiare l'albero, ricopiarlo e piazzarlo a mano: "diventa un'ammazzata", e "sono tutti oggetti singolarmente posizionati" nell'Outliner. Con il foliage, invece, si piazzano gli oggetti "in maniera customizzabile e randomizzata", e "risultano tutti quanti come un unico **foliage actor**".

Static mesh o static mesh foliage? Nel pannello si possono usare entrambi. Trascinando una static mesh normale nel pannello del Foliage Mode, Unreal avverte che sta creando uno **Static Mesh Foliage** (un *foliage type*) e chiede dove salvarlo. L'asset è "letteralmente lo stesso" degli altri; cambia che quelli del pacchetto hanno "già una serie di parametri modificati", mentre quello appena creato "è puro, tutto pulito, tutto vuoto". Il docente cancella il doppione dell'albero e usa quelli del pacchetto; per l'erba, che il pacchetto non ha come foliage type, ne crea uno da zero dalla mesh dell'erba.

Approfondimento: il foliage come istanze. Il *foliage actor* (*InstancedFoliageActor*) contiene, per ogni tipo di foliage, un componente di *Instanced Static Mesh* (più precisamente gerarchico, *HISM*): migliaia di copie della stessa mesh disegnate con pochissime chiamate alla scheda video, invece di un actor per oggetto. Romero e Sewell lo spiegano nel capitolo 19 ("Procedural Generation"): l'*Instanced Static Mesh Component* "è ottimizzato per renderizzare molte copie della stessa mesh", e la variante *Hierarchical Instanced Static Mesh* "è utile quando la mesh ha dei LOD"; nel libro lo si riempie con *Add Instance* da un Construction Script. Il Foliage Mode fa la stessa cosa con un pennello. Il *foliage type* (l'asset verde) memorizza le regole di piazzamento (densità, scala, rotazione, pendenze) e le impostazioni di rendering e collisione delle istanze; cambiare il foliage type aggiorna tutte le istanze già piazzate. Le opzioni del Foliage Mode si basano sul funzionamento generale dell'engine.

1.15.13 Gli strumenti del Foliage Mode

Paint ed Erase. "Letteralmente posso piazzare gli oggetti che voglio con un pennello, esattamente come il brush della Landscape Mode." In alto ci sono gli strumenti; di default si è su **Paint**. Un clic: "mi ha posizionato 1500 mesh d'erbetta". Per cancellare, tenendo premuto **Shift** si passa in automatico a **Erase**.

Quali oggetti dipinge. Nel pannello ci sono albero, cespuglio ed erba, ma il primo clic ha messo solo l'erba: "perché c'è questo check qui sopra che va cliccato". La spunta accanto a ogni foliage type "ci permette di decidere cosa stiamo andando a posizionare". Con tutti spuntati il pennello piazza erba, alberelli e cespugli insieme, "in maniera randomizzata" ("per favore, compilatevi"). La spunta vale anche per l'Erase: per togliere erba e alberi ma lasciare i cespugli, si toglie la spunta ai cespugli e si cancella; "lui ci cancellerà tutto quanto tranne i cespugli, perché non sono interagibili in questo momento".

Brush Size, Paint Density ed Erase Density. Come nel landscape ci sono la **Brush Size** e la **Paint Density**, "la densità con cui il brush dipinge", che è "un fattore che va a sommarsi" alla densità del singolo foliage type. Prove: a 0,5 un clic ha messo 711 oggetti ("penso che sarà più o meno lo stesso"); cancellato e ridipinto, "ci ha messo la metà". Rialzando la densità, "lui ci piazza altre cose, perché gli stiamo dicendo di dipingere con una densità più alta". Lo stesso per l'**Erase Density**: "ho messo troppi oggetti, fammi cancellare, però non voglio cancellare tutto, voglio cancellare la metà". A **1** l'Erase "non elimina niente"; a **0** "elimina tutto".

Un avvertimento pratico. "Una cosa che vi succederà a un certo punto": si dipinge e le cose non compaiono, o non si tolgono. "Probabilmente è perché inavvertitamente avete modificato questi" valori "e vi siete dimenticati di rimodificarli".

Single Instance Mode. Permette di "posizionare gli oggetti una alla volta". Con tre foliage type spuntati, un clic mette "un albero, un cespuglio e un'erbetta tutti nello stesso punto". Serve, dopo il lavoro generale, a "mettere dei dettagli senza tornare nella Selection Mode", facendo in modo che gli asset "rimangano tutti nell'unico foliage actor, che è già un po' più ottimizzato per stare in scena".

Attenzione al PC. "Se avete un oggetto particolarmente piccolo, aumentate la densità di molto e ne spawnate tanti tutti insieme, se non avete una macchina molto performante vi si potrebbe frizzare il computer." Il docente ha visto progetti "in delle scuole" andare in crash "perché le macchine non erano particolarmente performanti": "hanno fatto spawnare due milioni di" oggetti "nello stesso punto, e non ce l'ha fatta".

1.15.14 I parametri di un foliage type

Si passa alle impostazioni del foliage type dell'erba appena creato, "come sono di default quando le andate a creare per la prima volta". In cima:

- la **Mesh**, modificabile ("non so perché, tra l'altro, qui si rifiuta di mostrare l'erbetta, ma va bene");
- la **Component Class**;
- l'**Override Materials**: il materiale dipinto può essere diverso da quello della mesh originale, "per fare diversi tipi d'erba che hanno un colore diverso". Sono foliage type diversi a cui si cambia il materiale.

La densità. "Quante istanze di oggetti vengono piazzate in un'area di mille per mille, fondamentalmente." (Nota: con il pennello non si può togliere il foliage piazzato dal materiale: "non è interagibile, perché è un'altra cosa, viene piazzato in un altro modo e non fa parte del foliage actor".) Le prove, con un clic e un pennello di raggio 0:

- densità **100**, raggio 0: circa 300 oggetti;
- densità **1000**: "3000 e qualcosa"; "c'è talmente tanta roba che non si vede neanche quello che abbiamo piazzato";
- densità **500**: "comunque tanta erba, molto folto, però in alcuni casi potrebbe andare bene".

Il raggio. Per avere "questa densità più o meno, ma un piazzamento un po' più sparso", si aumenta il **Radius**. "Ci torna utile principalmente quando utilizziamo delle mesh un po' più grandi, tipo gli alberi, tipo i cespugli, cose che non vogliamo siano sovrapposte o compenetrate." Il raggio dice: "dalla singola istanza devono passare almeno x unità prima di spawnarne un'altra". Con densità 500 e raggio 100 "è comunque denso, ma c'è un po' di spazio fra ogni singola pianta". Con l'erba non è troppo necessario: "potreste mettere tipo 10 con una densità un po' più bassa", per un manto "bello compatto, un po' variegato, un po' sparso", senza le facce delle mesh che si compenetrano. Tornerà utile con gli alberi.

Lo Scaling. Come nel Grass Type: **Uniform** di default, oppure con un asse bloccato o completamente **Free**. "Nel nostro caso vogliamo uniforme, perché vogliamo mantenere una certa proporzione." Prova: **Scale X** da 0,8 a 1,5, poi fino a 2. Con l'erba "si nota relativamente poco", ma ci sono "delle parti un po' più alte, un po' più incolte", che si vedono meglio con una densità maggiore: "va a creare un manto erboso un po' più randomizzato".

Lo Z Offset e le rocce che levitano. "Un altro parametro molto carino": lo **Z Offset**, "un range minimo e massimo di displacement che viene applicato sull'asse delle Z". A 0 e 0 l'istanza sta sul piano su cui viene dipinta; con **0 e 500**, un clic mette erba in basso ed erba sospesa in alto. "Chiaramente con l'erba questa cosa è completamente inutile." Con dei **sassi**, invece: il docente prende le rocce del pacchetto ("Crocs" nella trascrizione: le mesh *Rocks*), ne fa un foliage type e con uno Z Offset fino a **100** ottiene "una scena con dei sassi che fluttuano". Va usato con giudizio: "magari messo a un lato, in un posto in cui il giocatore non passa, perché poi se ci passa attraverso e non si muovono è un brutto effetto"; o per "una singola immagine, come stavamo vedendo prima con la telecamera".

Il mistero delle rocce nere, risolto. Prendendo le rocce, il docente capisce "per quale motivo l'altro giorno facevano quell'effetto" (le rocce del Grass Type che sembravano "riflettenti" e "disintegrate"): "il materiale delle rocce ha all'interno un calcolo dell'altezza della world position, e quindi se c'è una parte che sta sotto lo zero diventa completamente nera, e invece sopra sono normali". Ci torna più avanti (sezione 1.15.18).

1.15.15 Densità e raggio: chi vince

La domanda. Uno studente: "se si fa il brush, si spawna più volte sullo stesso punto se clicchi, oppure no?" "Mi sembra di no." Il motivo: "abbiamo un raggio che gli dice: la densità è tot, quindi devono esserci un numero x di oggetti massimo, e deve esserci un raggio fra loro di questo tipo". Il docente lo dimostra contando gli oggetti nel pannello.

La prova.

1. Un clic: **253** oggetti. Un secondo clic nello stesso posto: sempre 253.
2. Togliendo il raggio e ricliccando: **260**. "Qualcosa ha messo, perché ha riempito i posti che prima erano stati bloccati dal raggio."
3. Aumentando la densità: **3730**; aumentandola ancora: **13.000**.
4. Ricliccando: niente. "Ha già riempito questa sfera, capito? L'ha già riempita al massimo della densità possibile." Sulla base della densità e del raggio, "lui sa già che deve mettere un tot di oggetti in quel punto".

Il raggio ha la priorità? Lo studente: "il raggio ha la priorità sulla densità? Cioè, se metti mille di densità e duemila di raggio te ne spawna uno, immagino". Il docente prova con valori esagerati ("forse non serviva portare all'estremo così tanto"): un solo oggetto. "Tutto questo casino per piazzare un singolo oggetto." "È vero, però si è calcolato quanti ne doveva mettere": ricliccando nello stesso posto "ha messo sempre uno, perché stiamo comunque a meno di tot dal raggio".

Con numeri "umani". Densità **50**, "in modo che è semplice da vedere":

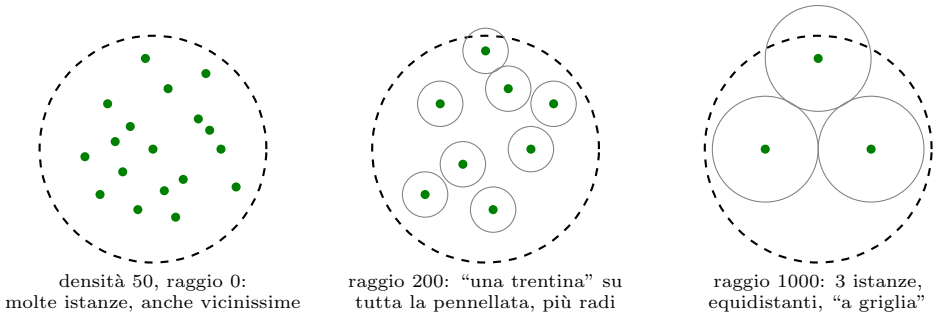
- raggio **0**: un certo numero di oggetti;
- raggio **50**: **64**, "leggermente meno";
- raggio **200**: "una trentina";
- raggio **1000**: **3**. Trascinando il pennello "vedi proprio il raggio: sono tutte equidistanti", "molto preciso". In alcuni punti il raggio non è esattamente lo stesso, "perché dipende da come fai spawnare le cose", ma "deve esserci sempre almeno 1000 di raggio dalla fogliolina più vicina", e "si risolve tutto in una griglia".

Conclusione: "tendenzialmente il raggio ha più potere rispetto alla densità" (figura 1.15.5).

Approfondimento: come piazza il pennello. Il comportamento visto a lezione è coerente con il modo in cui il Foliage Mode lavora. A ogni pennellata calcola quante istanze dovrebbero esserci nell'area del pennello in base a *Density* (istanze per 1000×1000 unità, cioè per 10×10 metri) e alla *Paint Density*; conta quelle già presenti dello stesso tipo e aggiunge solo le mancanti, scartando ogni posizione candidata che cada entro il *Radius* di un'istanza esistente. Da qui le tre cose osservate: ricliccare non aggiunge nulla quando la zona è "piena"; togliendo il raggio si riempiono i buchi che il raggio aveva scartato; con un raggio grande la densità teorica non si raggiunge mai, e le istanze si dispongono a distanza quasi costante. Il raggio è anche il modo più semplice per evitare alberi compenetrati, il problema che Totten chiama *casualità percepita* (si veda l'approfondimento della sezione 1.15.18). Funzionamento generale dell'engine.

1.15.16 Le altre impostazioni: pendenza, altezza, mobilità, collisioni

Align to Normal e Align Max Angle. **Align to Normal** decide se le istanze "sono allineate alla normal su cui sono applicate". **Align Max Angle** è "quanto si inclinano per aggiustarsi alla singola pendenza" (il docente in un primo momento lo confonde con il parametro successivo e si corregge: "no, aspetta, mi sa che...").



La densità fissa *quante* istanze al massimo per area; il raggio impone una distanza minima fra due istanze. Quando le due regole sono in conflitto vince il raggio. Ricliccare su una zona già piena non aggiunge nulla.

Figure 1.15.5: Densità e raggio nel Foliage Mode, con i numeri delle prove della lezione. Il cerchio tratteggiato è il pennello; i cerchi grigi attorno alle istanze rappresentano il raggio minimo. Ricostruzione schematica: le posizioni dei punti sono indicative.

Ground Slope Angle. È "il range in cui il foliage verrà piazzato": l'angolo minimo e massimo della superficie su cui un'istanza può comparire. Il docente dipinge l'erba su un pendio (dopo un momento di confusione: "stavo modificando ancora le cose dell'albero invece era l'erba che andava modificata; ci ho cliccato sopra, pensavo di averla selezionata e invece non era selezionata", "è il tipo di cose che vi capiterà sicuramente ogni tanto"). Con l'intervallo di default, fino a 180, l'erba va "anche sulla parete rocciosa". "Poi sta a voi decidere se volete che il foliage venga piazzato anche lì oppure no." Con un massimo di **45** gradi, l'erba compare "soltanto nelle parti della slope che erano sotto i 47 gradi" (il docente dice 47 avendo impostato 45: con ogni probabilità è un lapsus), e si ammucchia nei punti meno ripidi. Abbassando ancora, "neanche su questa cunetta è stata messa, perché l'angolazione è troppo alta". Un uso: mettere attorno alla base del "montarozzo" dell'erba a mano, "per far vedere che un minimo c'è stata un'espansione erbosa", senza che salga sulle pareti; "anche troppo preciso, devo dire". Il docente lo rimette a 45 (figura 1.15.6).

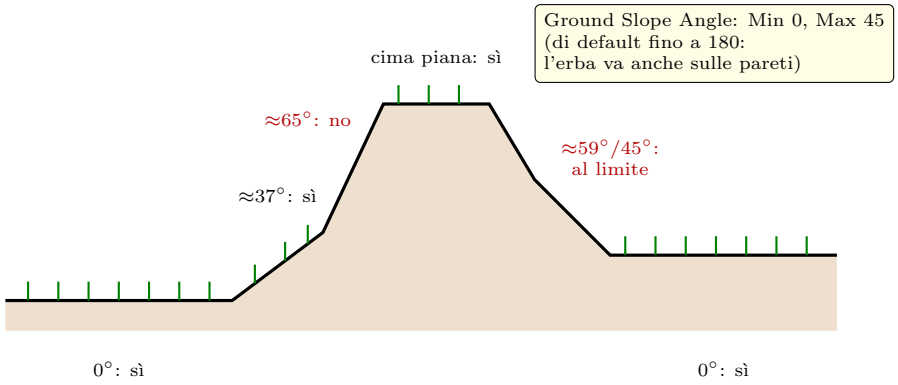


Figure 1.15.6: Il Ground Slope Angle di un foliage type. Con un massimo di 45 gradi il pennello piazza istanze solo dove la superficie è inclinata meno di 45 gradi rispetto all'orizzontale; le pareti restano spoglie. Gli angoli indicati sono esempi. Ricostruzione schematica.

Height. Il range d'altezza in cui il foliage può essere piazzato. Di default va da meno a più infinito ("da meno a più"); con **0 − 10**, "appena andiamo sopra i 10 d'altezza, lui smette di piazzarla".

Mobility. Come per le luci: **Static**, "più veloci da renderizzare", che "funzionano col baking", o **Movable**, "completamente dinamiche", con le luci che "reagiscono in tempo reale", "ma è più lento da gestire".

Cull Distance. "La distanza massima da cui non smettono di essere renderizzate." Allontanandosi, "tutto il resto del foliage è sparito", quello del materiale, "ma questo è rimasto, perché c'è zero di cull distance": "non ha massimo e minimo, esiste e basta, non viene tagliato, è sempre renderizzato". "Gestitelo sulla base della grandezza della vostra scena."

Le ombre e il dressing. *Cast Shadow*, ombre dinamiche e statiche, *Affect Dynamic Lighting*: "tutti i check che potete gestirvi per la resa estetica con la luce". L'erba del pacchetto "è fatta apposta per non castare ombre". Il foliage si può dipingere anche *sopra* le mesh: il docente mette dell'erba sopra una struttura della base, e immagina "un mondo post apocalittico con una pompa di benzina abbandonata" ("non so perché mi ha fatto pensare alla pompa di benzina"), con l'erba che ci cresce sopra e "dei *vine* che scendono giù" (delle liane, dei rampicanti).

Collision Presets. "E poi abbiamo i preset delle collisioni." La frase del docente è ambigua ("di default tutto il foliage è impostato a una collisione, che nel caso dell'erba ci fa anche comodo, perché noi non vogliamo che ci sia l'erba e poi ci inciampiamo sopra"): il senso è che *per l'erba* si vuole che non ci siano collisioni. "Quando andiamo a mettere gli alberi, però, dobbiamo assicurarci che lì sia tutto funzionale, che le collisioni ci siano, perché altrimenti possiamo passarci attraverso."

L'ingresso della base. Prima degli alberi il docente sistema l'erba davanti alla base: "il nostro personaggio uscirà da lì e si trova prima l'erbetta che va a gradiente, poi c'è il folto".

Approfondimento: le impostazioni del foliage type. Qualche precisazione sui parametri visti. *Ground Slope Angle* confronta la normale della superficie con la verticale, come il World Aligned Blend dell'automaterial: le due cose si possono far coincidere, così che il foliage manuale non finisca dove il materiale mette la roccia. *Height* usa la quota assoluta del mondo in centimetri, quindi "0 – 10" è una fascia di dieci centimetri sopra lo zero del mondo, utile per esempio per non piantare canne sopra il livello dell'acqua. *Cull Distance* ha un minimo e un massimo: fra i due le istanze sfumano, e con entrambi a 0 il culling per distanza è disattivato, come visto. I *Collision Presets* del foliage type (*Body Instance*) decidono come le istanze *rispondono* alle collisioni; perché ci sia una collisione, però, la mesh deve averne una (la sezione successiva lo mostra). Romero e Sewell descrivono nel capitolo 14 ("Math and Trace Nodes") la tabella delle *Collision Responses*: ogni oggetto ha un *Object Type* e una risposta (*Ignore*, *Overlap* o *Block*) per ogni canale, con dei preset come *BlockAll*, *OverlapAllDynamic* e *Pawn*, oppure *Custom...*; nel capitolo 5 impostano *BlockAllDynamic* su un oggetto interattivo. Le stesse regole valgono per le istanze del foliage.

1.15.17 Gli alberi e le collisioni

Le impostazioni dell'albero. "Togliamo l'erba, perché sennò dipingiamo anche l'erba insieme agli alberi." Le impostazioni del pacchetto: **Density 1**, "abbastanza sparsi", raggio 0: "ci può stare". Il docente ne mette una fila all'ingresso, "una piccola radura". Gli alberi sono di taglie diverse (**Scale** da 1 a 3: "con l'albero funziona meglio, dà un'idea migliore"; c'è l'albero "di base" e "l'albero grosso") e ruotati in maniera diversa: lo Z Offset è 0, ma c'è il **Random Yaw**, "una rotazione randomica sull'asse verticale". Togliendolo, "gli alberi vanno tutti nella stessa direzione, sono tutti curvi in quella direzione, verso sinistra"; rimettendolo, "iniziano a ruotare". (Un albero "si è rifiutato di essere cancellato".)

Alberi dalle pareti. Lasciando il Ground Slope Angle ampio e Align to Normal attivo, si possono piantare alberi su una parete: "alberi che escono dalle pareti, non so perché dovreste volerlo fare, ma lo potete fare". Per esempio "un percorso di alberi tutto attaccato sul lato della parete, che potete utilizzare come percorso per un platform". Togliendo Align to Normal "ve li spawna dritti". Con gli alberi l'allineamento "non funziona granché bene", "ma per esempio con dei cespugli o dei funghi parassiti" sì.

La collisione strana. Le collisioni dell'albero sono **Block All**. "Col nostro robottino andiamo all'avventura": "siamo alti quanto l'erba" (bisognerà abbassarla). In Play si riesce a passare dentro gli alberi, almeno in parte: "hanno una strana collisione". Prima di guardare, il docente chiarisce "una grossa cosa" sulle collisioni: "questo parametro non determina se l'oggetto avrà collisioni o meno; determina se le collisioni dell'oggetto sono attive o meno, e per cosa sono attive". La prova: Block All sull'**erba**. "Sono positivamente sicuro che potremo comunque camminare dentro l'erba": ed è così ("se dovessimo esplorare un open world del genere, il nostro personaggio dovrebbe muoversi più veloce di così"). Il motivo è nella static mesh di base: nello Static Mesh Editor, con **Show Simple Collision**, "non ci sono simple collision, quindi non c'è nulla da attivare". "Vi capiterà di mettere degli asset in scena, iniziare a giocare e passarci attraverso, perché magari quegli asset sono stati importati o esportati senza le collisioni"; e anche impostando "le collisioni che bloccano tutto, o che bloccano solo i proiettili", "lui comunque non può bloccare, perché non c'è nulla da bloccare".

L'albero, dentro. Aprendo la mesh dell'albero: "ah, geniale. L'hanno messa dalla parte sbagliata." Le forme della collisione semplice ("diversi pezzi") sono ruotate rispetto al tronco, "credo che l'abbiano importato proprio con la collisione sbagliata". Si possono selezionare tutte e ruotare "dalla parte corretta"; "anche se sono proprio strane", sbagliate anche nelle proporzioni. Tre strade:

1. **ruotare** le forme esistenti, come fa il docente ("tanto il nostro giocatore non andrà mai a collidere lassù, a meno che non vogliamo che lo faccia");
2. **eliminare** le simple collision e scegliere **Use Complex Collision As Simple**: la collisione prende "esattamente i poligoni dell'albero". "Chiaramente questa cosa è pesante, però vi risolve il problema di dover fare le collisioni a mano";
3. **rifarle** a mano con forme semplici: "vi basterebbe prendere un cilindro, metterlo in diagonale, e qui sopra metterci un piano circolare". Il docente aggiunge una **capsula**, la fa più piccola e più stretta, la ruota lungo il tronco, e sopra una **box piatta**, "in modo che il giocatore possa salire sulla cima dell'albero e rimanerci sopra" (figura 1.15.7).

Se si vuole che il personaggio salti da un albero all'altro o percorra la parete passando sui tronchi, "non potete fare questa cosa" con la collisione originale: "dovete togliere queste collisioni e rifarle".

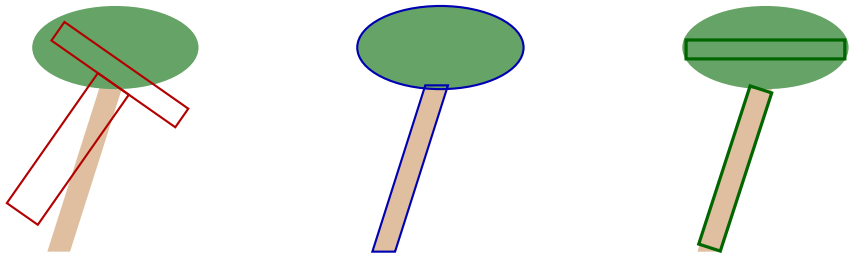


Figure 1.15.7: Tre modi di sistemare la collisione dell'albero di StackOBot visti a lezione: ruotare le forme importate male, usare la geometria della mesh come collisione, oppure rifare la collisione con una capsula (qui disegnata come un rettangolo inclinato) e una box piatta su cui il giocatore può stare in piedi. Ricostruzione schematica.

La domanda sulle collisioni a senso unico. Uno studente ha una domanda "fuori tema": "le collisioni che funzionano da un solo lato?" Gli è venuto in mente un gioco ("PIC" nella trascrizione: con ogni probabilità *PEAK*, il gioco cooperativo di arrampicata, ma il nome non è certo) in cui "puoi salire sugli alberi e puoi attraversare le foglie da sotto, ma non puoi attraversarle da sopra". Il docente non ci ha mai pensato: "probabilmente dipende dal modo in cui è codificato quel tipo di interazione". "Gli dici che se stai andando in una direzione non deve contare quella collisione, se stai andando nell'altra deve contarla. Penso, non lo so, a naso direi che è questo." Non ha giocato il gioco.

No Collision sovrascrive. Rimesso tutto com'era, con le collisioni girate "dalla parte giusta", il docente imposta **No Collision** nel foliage type dell'albero: "lui sovrascrive il fatto che ha delle collisioni". In Play ci si passa attraverso, "perché l'asset del foliage non ha collisioni". Con **Custom** si sceglie che cosa bloccano: "se bloccano la camera, se bloccano i proiettili, se bloccano il pawn". "Di base, il Block All andrà bene."

Approfondimento: collisione semplice, complessa e a senso unico. Una static mesh ha due rappresentazioni di collisione. La *simple collision* è fatta di primitive (box, sfere, capsule, convessi) ed è quella usata dalla fisica e dal movimento del personaggio: è economica, ma va costruita (nello Static Mesh Editor, menu *Collision*: *Add Box*, *Add Capsule*, *Auto Convex Collision*...) o importata dal programma di modellazione. La *complex collision* è la mesh stessa, triangolo per triangolo, usata di solito solo per le tracce precise. *Collision Complexity: Use Complex Collision As Simple* fa usare la seconda anche per la fisica: comoda, ma costosa su mesh dense e, per i corpi simulati, non utilizzabile. Le collisioni "a senso unico" delle piattaforme attraversabili da sotto non sono una proprietà della collisione in Unreal: si ottengono via codice, per esempio con un volume sotto la piattaforma che, quando il personaggio sale, gli fa ignorare il canale della piattaforma (*Ignore*) e lo ripristina quando il personaggio è sopra, oppure controllando la velocità verticale; è proprio la risposta "a naso" del docente. Le risposte *Ignore/Overlap/Block* per canale sono quelle della tabella del capitolo 14 di Romero e Sewell; il resto è funzionamento generale dell'engine.

1.15.18 Selezionare, rifinire e il foliage non naturale

Il cespuglio a mano. "Lo stesso con gli altri oggetti." Il cespuglio del pacchetto spawna poco; è "un tipo di asset che io, piuttosto che metterlo così, farei **single** e andrei a posizionare a mano": "voglio che queste cose stiano molto ammucchiate vicino agli alberi". Le dimensioni sono comunque casuali, "forse troppo varie per i miei gusti". Allo stesso modo si fa "un percorso alberato", con alberi a destra e a sinistra, che "fa comunque parte del foliage" e si integra con il resto.

Gli strumenti di selezione.

- **Select**: si prende "il singolo pezzo di foliage" e lo si sposta ("no, no, quest'albero non deve stare proprio lì, deve stare qui"), "un po' tipo selection mode, ma con tutti gli asset che rimangono all'interno del foliage". Tornando in Selection Mode e cliccando, invece, si seleziona tutto il foliage actor. (Il docente trova nell'Outliner anche delle istanze che non si aspettava, "saranno delle cose che c'erano prima", e non approfondisce.)
- **Select All** e **Deselect**: tutte le istanze dei tipi attivi, da muovere insieme.
- **Select Invalid**: "mi porta automaticamente alle instance che sono invalide, quindi quelle che vanno modificate".
- **Lasso**: "funziona un po' tipo Photoshop": invece di cliccare, si evidenzia una zona. Vale solo per i tipi spuntati: "qui ci sono sempre attivi soltanto gli alberi, quindi se voglio fare questa cosa devo prendere anche questi".
- **Remove**: "questi alberi qua non mi piacciono": elimina tutto ciò che è stato evidenziato.
- un ultimo strumento, "stessa cosa con i raise, però è un discorso un po' diverso, poi modifica piano piano": il nome non è ricostruibile con certezza (con ogni probabilità *Reapply*, che riapplica le impostazioni del foliage type alle istanze già dipinte).

Non deve essere naturale. "È tendenzialmente uno strumento che utilizzate principalmente per motivi estetici." E "non deve essere necessariamente naturale": si può usare "con ogni mesh che voglio". "Se io voglio riempire la scena di robottini, lo posso fare"; una scena in metropolitana con il pavimento pieno di "cartacce, plastiche, cose varie". "È un po' un pennello con cui piazzare rapidamente tutta una serie di oggetti."

Le rocce sotto lo zero. Le rocce dipinte "appena si avvicinano allo zero impazziscono; invece sopra sono perfette". Consiglio: "mettete il piano di base leggermente più alto dello zero". Spostando una roccia, il materiale cambia: "cambia sulla base dell'altezza"; ruotandola, "così diventa una parete rocciosa". "È un po' strano che abbiano scelto di fare un materiale dinamico così per un oggetto standard." Aprendo il materiale ("più complesso di quello che pensavo"), il docente legge nomi come *Virtual Texture*, *World Position* e una "variation" (la trascrizione dice "part time, drop variation", non ricostruibile): "hanno provato a fare una roba carina" per avere un materiale dinamico, "tipo il **vertex painting**", e per "blendare la mesh con l'oggetto del landscape" quando si dipinge il materiale del landscape. Ma "volendo, potete fare anche letteralmente solo una texture di roccia" con le sue mappe, "e avete comunque una roccia".

Approfondimento: la casualità percepita. Il Foliage Mode rende facilissimo spargere oggetti a caso, ed è proprio il rischio che Totten descrive nel capitolo 8 ("Possibility Spaces and Worldbuilding"): in natura alberi, rocce e scogliere si dispongono nel tempo in modi non pianificati, e i designer alle prime armi piazzano alberi, rocce "e perfino edifici" a caso sulla mappa, "ignari dell'importanza della loro posizione"; il risultato è una *casualità percepita* che non dice nulla al giocatore. "Dirigere lo sguardo verso elementi scenici piazzati con cura" è il modo in cui il giocatore capisce il mondo. In pratica: il pennello per la massa (l'erba, il bosco di sfondo), lo *Single Instance Mode* e il *Select* per gli elementi che contano (l'albero isolato che fa da landmark, i cespugli che incorniciano un passaggio), che è esattamente il modo di lavorare del docente con i cespugli. Nel capitolo 3 ("Basic Gamespaces") Totten aggiunge che un cambio nel tipo di vegetazione segnala il passaggio a una zona nuova, come un cambio di texture sul terreno. Sul materiale delle rocce: un materiale che campiona texture in world space (*world aligned*) cambia aspetto quando l'oggetto si sposta o ruota, perché la texture resta ferma nel mondo mentre la mesh si muove; e un calcolo basato sulla quota assoluta (per esempio per sfumare la base nella sabbia o nell'umidità) produce effetti strani sotto lo zero, se chi l'ha fatto ha supposto che il terreno stia sopra. Funzionamento generale dell'engine; il grafo del materiale non è stato verificato nel capitolo.

1.15.19 La domanda sui mondi procedurali

Landscape o mesh? "Ci sono domande?" Uno studente: "i mondi che si generano automaticamente usano il landscape o le mesh?" "Tipo le robe procedurali? Tendenzialmente dipende." "È più comodo farlo con le mesh, perché puoi farti un **array** di oggetti e puoi piazzarli secondo dei criteri che hai scelto tu." Ma "l'ultima volta che ho lavorato con qualcosa di procedurale era un sacco di tempo fa", e magari ci sono "nuovi nodi" o "feature che Unreal ha inserito" che il docente non conosce. Il consiglio: cercare sullo store "qualche pacchetto che parla di asset procedurali, non modulari, proprio procedurali, che ha dei sistemi di proceduralizzazione", e "ti studi un po' loro come le fanno". "Fab, intendi, giusto?" "Sì, su Fab. Ai miei tempi non si chiamava Fab, mi sento vecchio."

Lo strumento che non si trova. Parlando di procedurale, il docente ricorda "un tool che io non ho mai usato, che dovrebbe essere relativamente nuovo": "una specie di box all'interno della quale possono essere impostati dei parametri, e spostando questa box e ingrandendola, lui va a piazzare quel tipo di foliage nel volume che c'è all'interno". Lo cerca fra i tipi di asset del menu *Foliage* (c'è lo *Static Mesh Foliage*, "questo qui è letteralmente il nuovo, ma non è quello che voglio"), non lo trova, e ipotizza che fosse "un plugin". "Me lo devo vedere meglio."

Sprite e flipbook. Nello stesso menu "vi potete vedere tutti i tipi di asset che possono essere creati", anche gli **sprite** e i **flipbook**: "carini se volete fare, che ne so, un menu animato". "Visto che avete fatto la roba col menu, con un flipbook potete andare a fare l'animazione del menu."

Approfondimento: generazione procedurale in Unreal. Lo strumento descritto dal docente corrisponde con ogni probabilità a uno di due sistemi, ed entrambi si attivano da plugin. Il primo è il *Procedural Foliage Spawner* con il suo *Procedural Foliage Volume*: si definisce quali foliage type crescono, con quali regole di competizione e di età, e si fa "simulare" la crescita dentro il volume (è una funzione sperimentale, *Procedural Foliage*, da abilitare nelle impostazioni dell'editor). Il secondo, più recente e oggi il sistema di

riferimento, è il framework *PCG* (*Procedural Content Generation*), con il *PCG Volume* e un grafo a nodi che campiona superfici, filtra i punti per pendenza o altezza e vi genera mesh: il plugin si chiama *Procedural Content Generation Framework*. La risposta del docente sulle mesh resta valida: anche il PCG, alla fine, produce istanze di static mesh. Romero e Sewell, nel capitolo 19 ("Procedural Generation"), definiscono la generazione procedurale come "un metodo per creare il contenuto del livello usando script invece che a mano", utile per evitare compiti ripetitivi; la costruiscono con il *Construction Script*, un *Instanced Static Mesh Component* e dei cicli *For Loop* che dispongono righe di istanze, e poi con le *Blueprint Spline* per piazzare actor lungo un percorso. Gli sprite e i flipbook appartengono a *Paper2D*, il sistema 2D dell'engine: un flipbook è una sequenza di sprite riprodotta a una certa velocità. Fab è lo store di Epic che dal 2024 ha sostituito l'Unreal Engine Marketplace, quello usato nel capitolo 9 del libro per importare l'*Animation Starter Pack*.

1.15.20 Il Niagara e il Post Process Volume

"Come ho detto, ci abbiamo una decina di minuti." In quella decina di minuti si potrebbe "buttare un occhio" sul post process, "visto che ci siamo".

Un'occhiata al Niagara. Prima, "al volo", il Niagara: "è proprio un editor a parte". Creando un **Niagara System** si sceglie fra "tutte le varie forme, modi, colori, possibilità" dei *preset*; il docente prende una **Fountain**, "la base della cosa". Dentro c'è "tutto un sistema così, con la timeline, con degli **emitter**, tutte cose". "È una roba un po' complicata, magari ce la vediamo alla prossima lezione se riusciamo."

Dove si trova il post process. Dal **Place Actors**, nella categoria **Volumes**, "tra le varie cose" ("insieme a tutta una serie di altre cose molto, molto carine") c'è il **Post Process Volume**: "il nostro quadratone".

Che cos'è. "È un volume all'interno del quale voi andate ad applicare delle modifiche alla lente della telecamera, fondamentalmente." "Voi andate a modificare il modo in cui le cose appaiono, non il modo in cui le cose sono." Le stesse "lenti" esistono anche su una **Camera Actor**: nei suoi Details, nella sezione *Post Process*, "abbiamo diverse lenti che possiamo andare a modificare", e queste modifiche "si applicano solo alla singola telecamera". Il volume invece applica gli effetti *al suo interno*.

Le transizioni. "Vi siete mai chiesti come fanno a fare quegli effetti per cui quando entri o esci da una casa il bloom della luce è particolarmente forte e poi si attenua? O com'è possibile che la luce fuori era molto forte, io entro dentro una grotta e piano piano la luce diventa più tenue e cambia colore?" "È perché il post process ha degli effetti di transizione fra l'interno e l'esterno." Si possono piazzare più Post Process Volume con effetti diversi, e passando dall'uno all'altro cambia "il modo in cui la mia telecamera va a gestire le cose".

I volumi annidati. Si può anche mettere un post process dentro un altro, "una roba complicata che vi sconsiglierei di fare": il volume interno "calcola tutto quello che lui deve calcolare e in più ci applica anche quello" dell'esterno. L'esempio: il volume interno rende la scena gialla (*color correction*), quello esterno blu; stando nell'interno "vediamo tutto verde, perché lui somma la color correction dei due post process". "Per cui diventa molto complicato capire cosa vogliamo fare." (Figura 1.15.8.)

Un solo volume per tutto: Infinite Extent. "Se voglio un unico post process che determina il modo in cui la scena viene vista, devo renderlo incredibilmente grande?" No: nei Details del volume ("le stesse lenti che aveva il camera actor, con qualche opzione in più") si cerca *infinite* e si trova **Infinite Extent (Unbound)**: "quando questa casella viene spuntata, il volume copre tutto il mondo, non solo l'area all'interno", e i cambiamenti "vengono visti allo stesso modo in tutta la zona". Per ora la si toglie.

Le lenti, per la prossima volta. "Non so quanto senso abbia iniziare a parlare delle lenti: ci abbiamo 5 minuti, e ogni lente ha un sacco di parametri." Due accenni: il **Bloom**, "abbastanza semplice", "quello che serve per i god shaft" di cui si parlava prima; e la **Chromatic Aberration**: "occhio, che fa male agli occhi". Il resto alla prossima lezione.

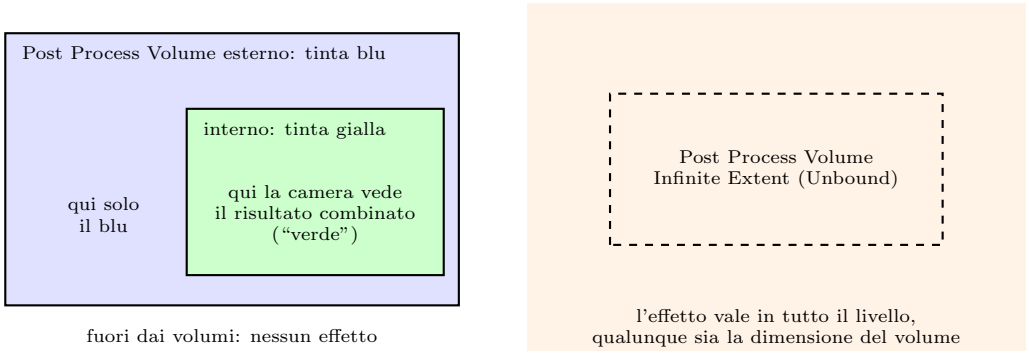


Figure 1.15.8: A sinistra, due Post Process Volume annidati: dentro il volume interno la camera riceve l'effetto di entrambi, come nell'esempio del docente (giallo e blu che danno verde). A destra, un volume con *Infinite Extent (Unbound)*, che vale per tutto il livello. Ricostruzione schematica.

Approfondimento: come si combinano i post process. Il comportamento dei volumi sovrapposti è un po' più ordinato della "somma" descritta a lezione. Ogni Post Process Volume ha una *Priority*, un *Blend Weight* (da 0 a 1) e un *Blend Radius*, la distanza dal bordo entro cui l'effetto sfuma: è il Blend Radius che produce le transizioni graduali fra esterno e grotta. Ogni volume modifica solo le proprietà la cui casella è spuntata; quando la camera è dentro più volumi, Unreal li applica in ordine di priorità, e per ogni proprietà il valore del volume a priorità più alta si *mescola* con quello già calcolato secondo il suo Blend Weight (con peso 1 lo sostituisce). Se due volumi cambiano proprietà diverse, si vedono entrambe le modifiche; se cambiano la stessa con peso minore di 1, si ottiene una via di mezzo: in questo senso le due tinte possono dare un colore combinato. Il consiglio del docente resta buono: pochi volumi, priorità esplicite, e di solito un volume *Unbound* a priorità bassa per l'aspetto generale del livello più volumi locali per le zone speciali. Le impostazioni della Camera Actor (e del *Camera Component* del personaggio) si combinano con quelle dei volumi, con un proprio peso. Romero e Sewell, nel capitolo 9 ("Building Smart Enemies with Artificial Intelligence"), allargando l'area di gioco ricordano che il *PostProcessVolume* e il *LightmassImportanceVolume* "devono coprire la nuova area di gioco", perché "questi volumi sono usati per l'illuminazione e gli effetti"; nel capitolo 12 indicano la qualità del *Post Processing* (filtri come motion blur e bloom applicati allo schermo dopo il rendering della scena) fra le impostazioni da ridurre se le prestazioni sono basse. Il resto è funzionamento generale dell'engine.

1.15.21 I compiti, la vetrata colorata e la chiusura

Il compito. "Visto che le cose principali che abbiamo visto in queste due lezioni sono sicuramente il landscape e sicuramente il foliage", il compito per il fine settimana, "come abbiamo fatto settimana scorsa con la gym e il livello":

- in un altro livello, o nello stesso, "un piccolo spazio": un piccolo landscape con il suo foliage, "che abbia un'estetica abbastanza curata" ("gli oggetti che avete a disposizione sono pochi, quindi non potete fare troppa roba");
- che **circondi la vostra gym**, "il posto in cui avete messo le cose", e in cui il personaggio "possa muoversi un po', girovagare"; conviene aumentare un po' la velocità di movimento del personaggio;
- lo scopo: "un piccolo livellino di **esplorazione** utilizzando il landscape". "Nulla di troppo complesso": magari "delle pedane che vi fanno salire e scendere da una parte all'altra"; come la volta scorsa, "un

paio di **collezionabili** in giro"; "delle alture, degli avvallamenti", e "se ve la sentite particolarmente creativi, anche dei buchi, delle grotticine";

- "potete mettere in pratica il fatto di dover saltare sugli alberi, se volete": sistemate le collisioni e via.

(Nel mostrarlo, il docente alza un pezzo di terreno e "clippa" tutto: "adesso sono sul cane", cioè, con ogni probabilità, sotto o dentro la geometria; "siamo direttamente sulla sabbia".)

Le dimensioni. "Chiaramente non deve essere una mappa così grande." Il docente ragiona ad alta voce: "15 per 15 va piccolino", "**31 per 31** conviene, che avete abbastanza spazio per fare un po' di cose". (La trascrizione non dice a che cosa si riferisca il numero; 31 per 31 è uno dei valori della *Section Size* visti nella lezione precedente, ma potrebbe anche indicare il numero di quad del landscape. In ogni caso il senso è: un landscape piccolo.) Si può usare il materiale costruito insieme e "sbizzarrirsi un pochino", "costruire qualcosina".

Il video. "Se volete potete anche mettere un video di voi che camminate in quell'ambiente su Discord": si registra e lo si carica. Uno studente che non ci sarà all'ultima lezione: "io penso che farò così". "Va benissimo: è comunque una testimonianza del fatto che fate cose. Magari ispirate gli altri." Vale anche per i livelli e le gym della volta scorsa. "Fateli poco pesanti però, perché Discord mi sa che non vi permette di caricare troppa roba"; altrimenti si mettono su un Drive e si condivide il link.

La domanda sul volume. Lo studente che mancherà all'ultima lezione chiede del post process: "la lente funziona nel momento in cui stai dentro l'area, o funziona attraverso, se guardi attraverso?" "Dentro": "è un volume, è come un trigger volume, come un blocking volume; funziona nel momento in cui tu sei all'interno del volume". Se si vuole una stanza di un colore, "devi mettere il post process tutto attorno all'ambiente".

La vetrata colorata. "Ma quindi, per esempio, una finestra colorata che trasforma la luce in luce colorata usa le lenti o qualcos'altro?" "Usa un po' di entrambi, tendenzialmente. Quando ti fai questa domanda, la risposta è sempre: entrambi." L'esempio è la **vetrata di una chiesa**:

1. un oggetto con un **materiale di vetro**, "che permetta di vedere attraverso" e "del giusto colore per dare quell'idea";
2. una **luce forte** proiettata attraverso, con il colore cambiato "in modo che la luce sia di quel colore lì";
3. il **post process** "per accentuare l'effetto di cambio della luce".

E se la directional light deve restare bianca, vista da fuori, ma colorata da dentro? Allora "senza cambiare la luce": il materiale del vetro "che comunque filtra un pochino la luce", e il post process che, mentre sei dentro, cambia il colore percepito. "Ah, è una specie di escamotage." "È tutto un escamotage." "Costruire livelli diventa sempre tipo trovare il **trucchetto del mago** per ingannare la persona." Si potrebbe certo fare in modo che "nel momento in cui la luce attraversa quel determinato trigger il calcolo del colore della luce cambi", "però è sicuramente più complicato e più dispendioso che fare un trucchetto".

La chiusura. Domande e lavori si possono mettere sul Discord. "La prossima lezione, sicuramente, vi parlo al volo del **packaging**": "così, oltre a fare il video, magari possiamo proprio fare un eseguibile che può essere passato e può essere giocato anche dagli altri". "Buon fine settimana, buona serata. E ci vediamo martedì prossimo."

Approfondimento: la luce attraverso il vetro. In Unreal Engine 5 una luce non cambia colore da sola attraversando un materiale traslucido: le ombre dei materiali *translucent* sono di norma solo più o meno scure, non colorate. Per questo il "trucco del mago" del docente è la strada pratica: una *spot light* o una *rect light* colorata dietro la vetrata, eventualmente con una *Light Function* (un materiale che maschera la luce con il disegno della vetrata) per proiettare il motivo sul pavimento, un cono finto come quelli dell'engine per il fascio, e un Post Process Volume nella navata per la tinta generale. Totten, nel capitolo 5 ("Introducing

Emotional Level Design through Survival Instincts"), parla proprio di questa luce: l'*ombra* (*shade*) ottenuta facendo entrare luce diffusa attraverso tende, schermi o vetrate fu un tratto centrale dell'architettura gotica, che cercava un'esperienza eterea chiamata *lux nova*, "luce nuova" o "luce mistica"; e la stessa condizione, con la nebbia, è un caposaldo dell'horror. Nel capitolo 1 ricorda che le vetrate gotiche "filtravano la luce nella chiesa per creare un'atmosfera che facesse sentire i fedeli più vicini a Dio". Nello stesso capitolo 5 Totten riprende Christopher Alexander: la luce posta agli incroci di un percorso *attira* le persone in quella direzione. I god rays della directional light e i coni finti sono, in questo senso, strumenti di guida, non solo di estetica. Sul packaging annunciato, Romero e Sewell (capitolo 12, "Building and Publishing") descrivono il procedimento: *Platforms* → piattaforma → *Package Project*, la scelta della cartella, la finestra del log in caso di errori e le *Packaging Settings*, con la *Build Configuration* e la *Staging Directory*.

Parametro del Grass Type	**Che cosa fa (e che cosa si è provato)**
Grass Density	quante istanze; abbassata, l'erba è meno folta
Use Grid	istanze disposte su una griglia
Placement Jitter	randomicità della posizione; a 1, manto più naturale
Start / End Cull Distance	distanze a cui le istanze sfumano e spariscono; disattivare il culling pesa molto
Min LOD	−1 (default): mesh con tutti i dettagli; più alto, versioni meno poligonali
Scaling, Scale X min–max	0,8–1,1 nel pacchetto; provato 1–3 ("un'enormità"); non uniforme sblocca gli assi
Random Rotation	rotazione casuale o tutte nello stesso verso
Align to Surface	allineate al pendio; tolto, crescono dritte (utile per alberi)
Cast Dynamic Shadow	ombre dinamiche: più belle, molto più pesanti
Receives Decals	se le decal (graffiti, sangue) si proiettano anche sull'erba

Table 1.15.2: I parametri del Landscape Grass Type visti all'inizio della seconda parte.

Strumento o opzione	**Che cosa fa**
Paint	dipinge le istanze dei foliage type spuntati
Erase (Shift)	cancella le istanze dei tipi spuntati
Spunta del foliage type	decide quali tipi il pennello piazza o cancella
Brush Size	grandezza del pennello
Paint Density	fattore di densità del pennello, si somma a quella del tipo
Erase Density	quanto lascia: 1 non cancella nulla, 0 cancella tutto
Single Instance Mode	una istanza per clic, dentro il foliage actor
Select, Select All, Deselect	selezionare e spostare singole istanze o tutte
Select Invalid	seleziona le istanze invalide, da correggere
Lasso	seleziona un'area, come in Photoshop
Remove	elimina le istanze selezionate
Reapply (probabile)	riapplica le impostazioni del tipo alle istanze esistenti

Table 1.15.3: Gli strumenti del Foliage Mode.

Punti chiave del capitolo

- Un **automaterial** sceglie da solo il materiale del terreno in base alla **pendenza** e all'**altezza**: scolpendo, il landscape "reagisce".
- Il **Lerp** interpola fra A e B secondo un **alfa** da 0 a 1: l'alfa è il *criterio*, e non deve per forza venire da una texture.
- Il **World Aligned Blend** dà un alfa che vale 1 dove la superficie guarda verso il **World Vector** (di default Z, cioè in piano) e 0 sulle pareti: è la pendenza. **Blend Bias** sposta la soglia, **Blend**

Parametro del foliage type	**Che cosa fa**
Mesh, Component Class	la mesh da piazzare e il tipo di componente
Override Materials	un materiale diverso da quello della mesh (erbe di colori diversi)
Density	istanze per area di 1000×1000 unità
Radius	distanza minima fra due istanze; vince sulla densità
Scaling (Uniform, Lock, Free), Scale X	proporzioni e intervallo di scala casuale
Z Offset	spostamento verticale casuale (rocce sospese)
Align to Normal, Align Max Angle	allineamento alla superficie e inclinazione massima
Random Yaw	rotazione casuale sull'asse verticale
Ground Slope Angle	pendenza minima e massima su cui piazzare (45 a lezione)
Height	quota minima e massima (0–10 nella prova)
Mobility	Static (veloce, baking) o Movable (dinamica, più lenta)
Cull Distance	distanza di scomparsa; a 0, sempre renderizzato
Cast Shadow e luci	ombre dinamiche e statiche, effetto delle luci
Collision Presets	risposta alle collisioni (Block All, No Collision, Custom); serve una collisione nella mesh

Table 1.15.4: I parametri di un foliage type nel Foliage Mode.

Prova o valore	**Valore**	**Che cosa mostra**
Blend Width dell'altezza	4000, poi 1500	la larghezza della sfumatura sabbia–erba
Blend Height di esempio	0 e 1000	la quota da cui parte la transizione
World Vector di prova	(0,1,0), (1,0,0)	la sabbia sulle facce rivolte a Y o X
Primo clic di Paint	1500 istanze	la densità di partenza dell'erba
Paint Density 0,5	711, poi la metà	il fattore del pennello
Erase Density	1 / 0	non cancella / cancella tutto
Density 100, raggio 0	circa 300	un clic con l'erba
Density 1000	oltre 3000	troppo fitto
Density 500, raggio 100	denso ma distanziato	il raggio separa le piante
Scale dell'erba	0,8–1,5, poi 2	manto più vario
Z Offset	0–500 (erba), 100 (rocce)	oggetti sospesi
Ground Slope Angle	45	niente erba sulle pareti
Height	0–10	niente foliage oltre la quota
Conteggio con raggio	253, poi 260	il raggio blocca dei posti
Densità aumentata	3730, 13.000	ricliccare non aggiunge nulla
Density 50 con raggio 0/50/200/1000	–, 64, ≈30, 3	il raggio vince sulla densità
Albero: Density, Scale	1, da 1 a 3	alberi radi e di taglie diverse
Landscape del compito	31×31	piccolo, ma con spazio

Table 1.15.5: Le prove e i valori numerici della giornata.

Sintomo	**Causa e rimedio**
L'automaterial mostra un solo materiale	parametri a zero, o materiale assegnato al posto della Material Instance: creare l'istanza, spuntare e regolare Blend Sharpness e Bias
Errore di compilazione "at feature level SM6"	l'uscita Alpha del World Aligned Blend usata per la Normal: usare w/ Vertex Normals
Ogni modifica al materiale è lenta	lavorare sui parametri della Material Instance
L'automaterial non si dipinge e non fa erba	manca il Landscape Layer Blend: usare la terza versione con il layer Auto
L'erba spawna solo dove si dipinge Grass	il Grass Output legge solo il layer Grass: sommare le zone non in pendenza
L'erba spawna anche su sabbia e rocce	sottrarre le zone basse (1-x dell'alfa per altezza) e limitare con Clamp
L'erba è invertita (sulla sabbia, non sopra)	alfa al contrario: nodo 1-x
Erba attaccata alle pareti	sfumatura troppo morbida: stringere il blend
Ombre strane dopo l'Erosion	passare Smooth o Noise
Asset dell'engine usati direttamente	rischio di reference rotte: copiarli nel Content
Il pennello del foliage non piazza o non cancella	controllare la spunta dei tipi, Paint Density ed Erase Density
Il PC si blocca dipingendo	troppe istanze di oggetti piccoli: densità più bassa
Il foliage del materiale non si cancella col pennello	non fa parte del foliage actor: si modifica dal materiale
Ricliccare non aggiunge istanze	zona già piena secondo densità e raggio
Erba sulle pareti rocciose	Ground Slope Angle massimo più basso (45)
Si passa dentro gli alberi	collisione della mesh ruotata male: ruotarla, Use Complex Collision As Simple, o capsula + box
Block All sull'erba non blocca	la mesh non ha simple collision
Le rocce diventano nere o cambiano aspetto	materiale world aligned con calcolo d'altezza: tenere il terreno sopra lo zero
Non si trova lo strumento a volume per il foliage	con ogni probabilità è un plugin (Procedural Foliage o PCG)
Video troppo pesante per Discord	caricarlo su un Drive e condividere il link

Table 1.15.6: I problemi incontrati o anticipati durante la lezione, con la causa e il rimedio.

Punto	Che cosa dice la lezione, e la precisazione
Mobility “vale solo senza Lumen”	con Lumen il build non serve, ma la mobilità conta ancora (le luci static non partecipano a Lumen)
“Parlando di MPC”	riferimento storpiato alla lezione sulle luci
Scala 0,8–1,1 “fino a 1,50 m”	con quell’intervallo il massimo è 1,10 m
Ground Slope 45, “sotto i 47 gradi”	con ogni probabilità un lapsus
Collisioni del foliage “impostate a una collisione”	frase ambigua: per l’erba si vuole nessuna collisione
Volumi annidati che “sommano” i colori	le proprietà si mescolano per priorità e Blend Weight
Valori “era 5, è di quarta”, “meno 5,5”	non ricostruibili; probabilmente il bias della roccia
Il primo blend: chi sta su A e chi su B	non chiarito; il capitolo segue l’effetto visto
“Speed level”, “immagini capacitarie”	con ogni probabilità speed level design; immagini di presentazione
Coni di luce “pin”	nome dell’asset non ricostruibile
Il gioco “PIC”	con ogni probabilità PEAK; non certo
Strumento “raise” nel Foliage Mode	con ogni probabilità Reapply
Lo strumento a volume “nuovo”	probabilmente Procedural Foliage Volume o PCG Volume
“31×31” per il compito	non è detto se sezioni o quad: comunque un landscape piccolo
“Adesso sono sul cane”	frase rovinata: il personaggio è finito dentro il terreno
Character animation “un anno e mezzo”	detto con “se non ricordo male”

Table 1.15.7: Le imprecisioni e i punti incerti della lezione, raccolti in un’unica tabella.

Sharpness rende il passaggio più netto; un bias negativo inverte l’effetto. Il World Vector si può cambiare (il muschio verso nord).

- L’uscita **Alpha** è la più precisa ma non si può usare per la Normal (errore SM6): in quel caso **w/ Vertex Normals** o **w/ Explicit Normal**.
- I parametri si regolano su una **Material Instance** (prefisso **MI_**), spuntandoli: niente ricompilazione a ogni modifica.
- La fusione per **altezza**: Absolute World Position (Z), meno il **Blend Height**, limitato e diviso per il **Blend Width**, e limitato fra 0 e 1. Height dice dove parte la transizione, Width quanto è larga.
- **Use Material Attributes** e **Blend Material Attributes** mescolano tutti gli attributi in un colpo; **Break Material Attributes** li separa quando serve collegarne uno a parte (per esempio la Visibility nell’Opacity Mask).
- Un automaterial “puro” non si dipinge e non fa spawnare foliage. La soluzione: metterlo come layer **Auto** di un **Landscape Layer Blend**, accanto a layer dipingibili.
- Per l’erba: layer Grass dipinto **più** le zone non in pendenza, **meno** le zone basse della sabbia (con **1-x**), limitato con **Clamp**, nel Landscape Grass Output. Doppio clic sui fili per i **reroute node**.
- Le correzioni sulle luci: con **Lumen** (default di Unreal 5) il build non serve; con **None** si torna al baking, utile per piattaforme deboli. I **god rays** si fanno anche con **Light Shaft Occlusion** e **Light Shaft Bloom** della directional light.
- Una **Camera Actor** serve a comporre inquadrature per schermate (speed level design). Gli asset dell’engine si **copiano** nel progetto, non si spostano.

Strumento, nodo o impostazione	**Che cosa fa**
Lerp	interpola fra A e B secondo un alfa da 0 a 1
World Aligned Blend (Blend Sharpness, Blend Bias, World Vector)	alfa dalla direzione della superficie rispetto a un vettore: la pendenza
Uscite Alpha, w/ Vertex Normals, w/ Explicit Normal	la più precisa (non per la Normal); la base; con una normal map
Scalar Parameter (S + clic), Convert to Parameter	valori regolabili dall'istanza
Material Instance (MI_), Global Scalar Parameter Values	modificare i parametri senza ricompilare
Break / Set Material Attributes, Use Material Attributes	separare e unire gli attributi del materiale
Blend Material Attributes	mescola due insiemi di attributi con un alfa
Absolute World Position (Z), Subtract, Clamp, Divide	la fusione per altezza (Blend Height, Blend Width)
Landscape Layer Blend con layer Auto	automaterial che si può anche dipingere
Landscape Layer Sample, Add, 1-x, Clamp, Landscape Grass Output	dove spawna l'erba
Reroute node (doppio clic sul filo)	ordinare i fili
Project Settings: Global Illumination / Reflections	Lumen (default) o None (baking)
Light Shaft Occlusion, Light Shaft Bloom, Threshold	god rays con la directional light
Camera Actor, Pilot, Eject	inquadrature per schermate e speed level design
Asset dell'engine (coni di luce)	da copiare nel Content del progetto
Landscape Grass Type	erba procedurale del materiale (tabella 1.15.2)
Decal	materiale proiettato su superfici (graffiti, sangue)
Foliage Mode, Static Mesh Foliage, foliage actor	spargere mesh con un pennello come istanze
Strumenti e parametri del Foliage Mode	tabelle 1.15.3 e 1.15.4
Show Simple Collision, Use Complex Collision As Simple	controllare e sostituire le collisioni di una mesh
Capsule e box di collisione	collisione semplice rifatta a mano
Fab	lo store di Epic (ex Marketplace)
Sprite, Flipbook	asset 2D, per esempio per un menu animato
Niagara System, emitter, preset Fountain	effetti speciali (forse all'ultima lezione)
Post Process Volume, Infinite Extent (Unbound)	lenti della camera in un volume o in tutto il livello
Bloom, Chromatic Aberration	due lenti del post process
Packaging	l'eseguibile, all'ultima lezione

Table 1.15.8: Gli strumenti, i nodi e le impostazioni della quindicesima lezione.

- Il **Landscape Grass Type** controlla l'erba del materiale: densità, Use Grid, Placement Jitter, cull distance, Min LOD, scala, rotazione, allineamento, ombre, decal. Ombre dinamiche e decal costano.
- Il **Foliage Mode** sparge mesh qualsiasi con un pennello, come istanze di un unico **foliage actor**: Paint, Erase (Shift), Single, Select, Lasso, Remove; spunta dei tipi, Paint Density ed Erase Density.
- Ogni **foliage type** ha densità (per 1000×1000 unità), **raggio** (distanza minima, vince sulla densità), scala, Z Offset, allineamento, **Ground Slope Angle**, Height, Mobility, Cull Distance, ombre e collisioni.
- Il **Collision Preset** attiva le collisioni, non le crea: se la mesh non ha simple collision non blocca nulla. Gli alberi con la collisione sbagliata si sistemano ruotandola, con **Use Complex Collision As Simple** (pesante) o con una capsula e una box.
- Il foliage non deve essere naturale (robottini, cartacce), ma non va sparso a caso: il pennello per la massa, le istanze singole per ciò che conta.
- Il **Post Process Volume** cambia come la camera vede la scena, solo **dentro** il volume; con **Infinite Extent (Unbound)** vale ovunque. Volumi annidati si combinano e complicano le cose.
- Fare livelli è trovare il **trucchetto del mago**: una vetrata colorata è vetro, luce colorata e post process insieme.
- Compito: un piccolo landscape esplorabile attorno alla gym, con collezionabili, alture, avvallamenti e magari grotte, da mostrare con un video. Ultima lezione: il **packaging**, forse il Niagara, il post process, decal e spline.

Chapter 1.16

Il post process, il packaging, le spline e le decal

Questa è la **sedicesima** e **ultima lezione** del corso breve di **introduzione a Unreal Engine**, la quarta delle ultime quattro dedicate al **level design**. La lezione precedente si era chiusa sul **Post Process Volume** appena aperto, con due sole lenti nominate, e con una promessa: "la prossima lezione, sicuramente, vi parlo al volo del **packaging**", per poter fare "un eseguibile che può essere passato e può essere giocato anche dagli altri". Oggi si mantiene la promessa e si aggiunge qualcosa: il post process visto lente per lente, il packaging, e due tipi di actor utili a chi costruisce livelli, le **spline** e le **decal**.

Il docente, come nelle tre lezioni precedenti, non dice mai il proprio nome. Tutto fa pensare che sia lo stesso: riparte "da dove ci eravamo lasciati la volta scorsa", riassume il landscape, l'automaterial e il Foliage Mode, cita la gym "che avevamo visto qualche lezione fa con me" e parla di Igor come di un collega ("a me e Igor"). Il docente è quindi, con ogni probabilità, **Davide**, lo stesso che aveva tenuto la terza lezione del corso su texture, materiali e mesh; resta una deduzione del capitolo, e nel resto del testo è semplicemente "il docente".

Per sua stessa ammissione è una lezione "altamente improvvisata". Il piano originale era un'**esercitazione**: gli studenti avrebbero costruito un livello con le meccaniche fatte con Igor, sotto i suoi occhi. Ma alcuni non c'erano, altri avevano problemi con il PC ("surriscaldamento, cose varie"), e il docente non voleva "passare una lezione a fare io un livello per conto mio con nessun altro che lavora in background". Ne esce una lezione "di ripiego" che mette in fila "cose veloci che gli posso far fare senza avere bisogno di altre 15 lezioni": "se il piano non è particolarmente coerente", avverte, è per questo. La giornata ha due metà. Nella prima, il **post process**: dove si colloca nella produzione, come funziona il volume, e le lenti principali (Bloom, Exposure, Chromatic Aberration, Lens Flare, Depth of Field, Color Grading); poi il **packaging**, con le impostazioni che contano davvero. Nella seconda, dopo una pausa anticipata, le **spline** (in due Blueprint, "per punto" e "per lunghezza"), le **decal** e, in chiusura, il **progetto estivo** facoltativo.

Tre avvertenze di lettura, come per le lezioni precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. **Unreal** è "Real", "un Real", "Anria" e "RIA"; il **Place Actors** è "la finestra PlayStation"; il *depth of field* è "la depth of field.nl" e "la TechField"; la **Chromatic Aberration** è "grammatica beration", "grammatic aberration" e "Padma recuperation"; la **Dirt Mask** è "the mask"; il **Bokeh** è "poke"; i *midtones* sono "mittons"; la **Third Person Map** è "Person Map", "terzo map", "terperson map" e "in per person U map"; il **Cook only maps** è "hook only maps"; le **decal** sono "DECALL", "DEGAL", "TEGAL", "regal", "VEGAL", "Deca" e "Tegelblock"; il *Deferred Decal* è "defrade decal" e "Deferr e DeCal"; il **DBuffer** è "debuffer"; il **Niagara** è "UFX, mia gara"; *stretch* è "strecciare", "streccia", "sterciata"; il *gizmo* è anche "ghitmo"; *itch.io* è con ogni probabilità "ICE". Dove la ricostruzione è sicura il capitolo usa la forma corretta; dove non lo è riporta la forma della trascrizione e lo dice. La seconda: in alcuni punti il riconoscimento vocale produce testo estraneo alla lezione. In apertura ci sono frasi su "preprint" e "postprint per l'open access", sulla "VQR" e sulla "situazione in cui la vediamo domani"; poco dopo un "prego, signora Presidente" seguito da decine di "scusatemi"; più avanti frasi su una "valutazione di impatto ambientale" e sui "nostri laboratori", un "LinkedIn mi ha mandato un preprint" ripetuto, un blocco di "raccomandazioni che sono state eseguite", uno sulla "dottrina sul mercato del lavoro"; e durante la pausa torna, ripetuto decine di

volte, il blocco già visto nelle lezioni precedenti ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?"). Non appartengono alla lezione e sono stati ignorati. La terza: lo schermo condiviso non è nella trascrizione. Il capitolo ricostruisce i passaggi da ciò che il docente dice mentre clicca, completa i nomi esatti di strumenti, parametri e nodi di Unreal Engine 5, distingue le prove dalle conclusioni e dichiara i punti in cui la trascrizione lascia un buco.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le figure ricostruiscono le tre fasi della produzione, il Post Process Volume e la transizione fra interno ed esterno, la soglia del Bloom, lo Start Offset della Chromatic Aberration, le quattro zone del Color Grading, il percorso del packaging, i punti e le tangenti di una spline, la differenza fra le due versioni del Blueprint e i loro grafi, e l'area di proiezione di una decal con il suo materiale. Le tabelle riassumono le lenti del post process, le configurazioni di build, le impostazioni del packaging, i nodi delle spline, i parametri delle decal, i problemi incontrati, le imprecisioni e gli strumenti visti. Qua e là, dei paragrafi di **approfondimento** collegano la lezione al libro di Marcos Romero e Brenden Sewell, *Blueprints Visual Scripting for Unreal Engine 5*, il testo di riferimento per questa parte del libro, al libro di Christopher W. Totten, *An Architectural Approach to Level Design*, per i temi di design, e a nozioni tecniche generali su Unreal Engine 5; sono segnalati come tali e non fanno parte della lezione. Il libro di Romero e Sewell tratta il packaging e le spline, ma non i parametri del post process né le decal: gli approfondimenti tecnici su questi due argomenti si basano sul funzionamento generale dell'engine, e lo dicono.

Il percorso del capitolo:

1. l'**apertura**: la lezione improvvisata e i progetti estivi (sezione 1.16.1), e il **riepilogo** (sezione 1.16.2);
2. la **post produzione** nel ciclo di sviluppo (sezione 1.16.3);
3. il **Post Process Volume** e la **Camera Actor** (sezione 1.16.4), le **transizioni** e l'**Infinite Extent** (sezione 1.16.5);
4. le lenti: **Bloom** (sezione 1.16.6), **Exposure** (sezione 1.16.7), **Chromatic Aberration** (sezione 1.16.8), **Lens Flare** (sezione 1.16.9), **Depth of Field** (sezione 1.16.10) e **Color Grading** (sezione 1.16.11);
5. la domanda sull'effetto **in due toni** (sezione 1.16.12);
6. il **packaging**: che cos'è e dove si avvia (sezione 1.16.13), le **impostazioni** (sezione 1.16.14), le **configurazioni di build** (sezione 1.16.15), le mappe e le cartelle (sezione 1.16.16) e l'avvio con l'**Output Log** (sezione 1.16.17);
7. la **pausa** e Mallory (sezione 1.16.18);
8. le **spline**: che cosa sono (sezione 1.16.19), a che cosa servono (sezione 1.16.20), il Blueprint **per punto** (sezione 1.16.21) e quello **per lunghezza** (sezione 1.16.22), le domande sull'**ottimizzazione** e sull'**amico grafico** (sezione 1.16.23);
9. le **decal**: che cosa sono (sezione 1.16.24), dove si proiettano (sezione 1.16.25), il loro **materiale** (sezione 1.16.26) e i loro nodi (sezione 1.16.27);
10. il **progetto estivo** e i saluti (sezione 1.16.28).

1.16.1 L'apertura: una lezione improvvisata

Pochi pochini. "Oggi mi sa che saremo pochini pochini": già la volta scorsa qualcuno aveva avvisato che sarebbe mancato. Poi arrivano **Tommaso** e **Guido**. Il docente chiede se siano riusciti a "smanettare un po"' durante il fine settimana con le cose viste. Uno studente è stato tre giorni a Napoli, da venerdì: voleva portare il portatile, "però il mio portatile non reggeva" l'engine, ed è tornato tardi ("tipo alle 11 a casa"). "Va bene, nessun problema": era solo un'occasione per provare e per raccogliere le domande, "non vi ho neanche dato un esercizio vero e proprio". Si aspettano "i soliti minutini" per chi arriva.

Perché si cambia programma. Come ultima lezione al docente "sarebbe piaciuto" fare una specie di **esercitazione**: ognuno si mette lì e usa le cose viste, "utilizzando il blueprint che avete fatto anche con Igor", per creare un livello. Ma per alcuni era problematico: chi non c'era, chi aveva problemi "di surriscaldamento, cose varie". Per non ritrovarsi a costruire un livello da solo "con nessun altro che lavora in background", ripiega su "alcune cose che possono, secondo me, tornarvi utili":

- un funzionamento base, "in maniera abbastanza generale", del **post process**: il Post Process Volume, la post produzione;
- "sicuramente" come funziona il **packaging** di un progetto;
- "per riempire un pochino il tempo", un paio di tipi di actor "interessanti" per la creazione di livelli: le **spline**, "in maniera molto basilare", e i **decal actor**.

"Cercherò di condensare le cose un po' insieme, quindi buttarvi lì quante più info possibili." Il piano non è "particolarmente coerente" perché è un ripiego: "quali sono delle cose veloci che gli posso far fare senza avere bisogno di altre 15 lezioni? Perché poi effettivamente questa è l'ultima lezione che facciamo".

Perché il packaging conta. Il packaging "è il modo in cui prendiamo il nostro gioco, lo tiriamo fuori dall'editor e lo rendiamo un gioco a sé stante vero e proprio": un **eseguibile** effettivo. Il docente lo ha detto più volte: gli piacerebbe che durante l'estate, "a tempo perso, senza nessuno che vi bacchetta, e se ne avete voglia", gli studenti usassero "tutti gli strumenti" che ormai hanno per fare dei piccoli livelli, impacchettarli, caricarli da qualche parte e mettere il link su **Discord**. Così potrebbero provarli prima di tutto "io e Igor", dando dei feedback a chi li vuole, e poi gli altri: per mostrare che persone partite "con relativamente poche basi, poca esperienza" in un corso come questo possono fare cose "che già sono interessanti, sono già una base".

itch.io o dove si vuole. Una studentessa, **Giulia**, chiede se si possano caricare su "ICE" (con ogni probabilità **itch.io**, la piattaforma per giochi indipendenti). "Potete fare quello che volete. Sono cose vostre, sono produzioni vostre", fatte fuori dal corso, "ancora meglio rispetto a come era l'esercitazione": non c'è "neanche una supervisione mia effettiva". "Io sarei contento se ce le fate provare", ma chi vuole caricarle su itch.io lo faccia tranquillamente; e se per Igor fosse più comodo provarle lì, "non è un problema: basta un link". Poi il docente condivide lo schermo e passa su Unreal, "che sicuramente mi darà problemi con il mouse come l'altra volta"; e infatti: "mi dimentico sempre poi di andare a controllare come mai" succeda.

1.16.2 Il riepilogo della lezione precedente

Il docente riassume ciò che si è visto (la trascrizione, in questo punto, è interrotta da un lungo blocco estraneo):

- il **Landscape Mode**: come si crea un landscape, il *painting*, "eccetera, eccetera";
- un **materiale** che permettesse di dipingere direttamente sul landscape;
- un materiale che si "autogestisce", l'**automaterial**: si modifica "in maniera procedurale sulla base di come andiamo a modificare il landscape", così che a ogni modifica "lui va automaticamente a cambiare il materiale che c'è sopra";
- la modalità, cioè il **Foliage Mode**, che permette di "dipingere effettivamente attivamente" degli asset in scena, riempiendo l'ambiente con gli oggetti voluti "secondo dei criteri che avevamo scelto per i vari oggetti".

"Ci eravamo lasciati su un punto, ovvero iniziare a parlare del post-process, quindi della post-produzione."

1.16.3 La post produzione nel ciclo di sviluppo

Nessun fotografo. "Qualcuno di voi è già confidente? Sa già come funziona la post-produzione, per esempio, di un'immagine? C'è qualche fotografo fra di noi, che ne sa sicuramente più di me?" Nessuno. "Ok, quindi per me meno ansia da prestazione": si fa un discorso generale.

Le tre fasi. Quando si sviluppa un gioco ci sono **pre-produzione**, **produzione** e **post-produzione** (figura 1.16.1):

- nella **pre-produzione** c'è "tutta la fase di documentazione, tutta la fase in cui capiamo cosa vogliamo andare a sviluppare e iniziamo a definire il lavoro";
- nella **produzione** si creano e si implementano "tutte le cose che abbiamo deciso che vogliamo implementare": i livelli, le meccaniche, i modelli 3D;
- nella **post-produzione** si *rifinisce* il progetto. Ci rientrano "tutta una serie di fattori": la **musica** e il **doppiaggio**, la creazione e l'implementazione degli **effetti speciali**, la **juiciness** ("quanto il gioco sia soddisfacente da giocare"), e, "oltre a queste cose", la rifinitura delle scene con il **post process**.

Rifinire con il post process vuol dire "modificare in post" la scena con gli strumenti che Unreal mette a disposizione: cambiare il modo in cui funziona la luce, il modo in cui è gestito il *color coding* dell'ambiente, "eccetera".

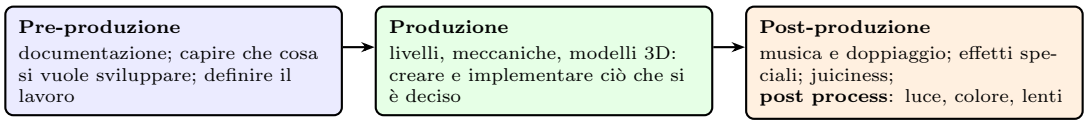


Figure 1.16.1: Le tre fasi dello sviluppo come le riassume il docente. Il post process appartiene alla post-produzione: si applica a scena costruita e illuminata. Ricostruzione schematica.

Approfondimento: post produzione e juiciness. Nel linguaggio del game design la *juiciness* (il "succo") è l'insieme dei riscontri audiovisivi sproporzionati rispetto all'azione che li provoca: scosse della camera, particelle, suoni, rallentamenti di un istante. Il docente la colloca in post-produzione perché, come la musica e il post process, si aggiunge sopra un gioco che già funziona: un buon post process non salva un livello illeggibile, ma può rendere memorabile un livello che funziona. Nella pratica le fasi si sovrappongono: l'illuminazione e il post process si abbozzano già durante il blockout, per capire il *mood*, e si rifiniscono alla fine. In Unreal Engine 5 il post process è un passaggio del rendering che lavora sull'immagine già calcolata: per questo il docente dice che modifica come le cose appaiono, non come sono (funzionamento generale dell'engine).

1.16.4 Il Post Process Volume e la Camera Actor

Dove si trova. Nel **Place Actors** ("la finestra PlayStation", nella trascrizione) c'è un oggetto che si chiama **Post Process Volume**: "lo trovate nella parte di **Visual Effects**, o in generale se cercate *post process*". Lo si trascina in scena. (Nella lezione precedente il docente lo aveva preso dalla categoria *Volumes*: in Unreal Engine 5 compare in entrambe.)

Quello che c'è già. In scena ce n'è già uno, "ma non è un problema": "tutte le scene di default di Unreal vengono create con una luce, un post process, eccetera". Quello esistente "al momento è stato impostato in modo che non influenzi nulla". Perché è importante che non faccia niente? "Perché i post process si **sommano** fra di loro": se si mettessero effetti in entrambi, "quando li andiamo a sovrapporre, poi è confusionario". "Facciamo una cosa alla volta."

Un volume come gli altri. Quello appena messo è un **volume**: "come i blocking volume, come i trigger volume, e quindi come tutti gli altri volumi, ha effetto nel momento in cui qualcosa compenetra con il volume". Gli effetti aggiunti saranno visibili "soltanto nel momento in cui siamo all'interno del volume": se la camera è fuori non si vedono cambiamenti; quando "con la telecamera mi trovo all'interno del volume", cambia il modo in cui la scena viene visualizzata.

La Camera Actor. "Altro modo in cui possiamo andare a vedere e aggiustare la post-produzione è con un **camera actor**", già vista la volta scorsa: un actor che "inquadra sempre determinate zone". Serve per i cambi di inquadratura, oppure per fare uno "speed level" (lo *speed level design* della lezione precedente): "un livello che non è pensato per essere giocato, ma che è soltanto pensato per essere visualizzato", da usare come materiale pubblicitario o come schermata di gioco. Anche la camera permette di fare post produzione, perché "ha la possibilità di andare a modificare le **lenti**". Nei suoi Details ci sono diverse lenti di post process: *Mobile Depth of Field*, *Bloom*, *Exposure*, *Chromatic Aberration*, *Dirt Mask* ("the mask"), *Camera*, *Local Exposure*, *Lens Flare*, "eccetera". "Sono gli stessi parametri che abbiamo all'interno del post process volume."

La differenza. Il Post Process Volume "è un oggetto di scena" che si può mettere nel livello e usare in diversi modi. La telecamera, invece, applica la post produzione "unicamente su quello che la camera inquadra": si potrebbe modificare "il modo in cui la temperatura viene vista all'interno della scena dalla telecamera, ma non dal resto del livello". "Quindi è un po' più limitante, è un po' più preciso", e serve "magari per altre cose", per esempio un effetto "più cinematico". La Camera Actor "tanto alla fine è relativamente poco rilevante per quello che vogliamo fare noi": si passa al volume.

1.16.5 Le transizioni e l'Infinite Extent

Entrare in una casa. Il volume torna più utile perché si può usare per le **transizioni**. Si immagini un edificio: dentro c'è il volume, fuori no. Quando il personaggio entra ed esce, cambia la visualizzazione della scena: entrando in una casa "la luce diventa immediatamente meno forte, il bloom viene ridotto e c'è magari un filtro di colore diverso rispetto all'esterno". Se si vuole entrare "in un posto lugubre", si carica l'ambiente di **toni freddi**, "fra il verde, il blu, il viola", per dare un certo effetto. E si possono anche "concatenare più di un post process per avere una transizione che sia più morbida" (figura 1.16.2).

Rainbow Six Siege. L'esempio del docente: *Rainbow Six Siege* ("Rimbo Six" nella trascrizione), "all'inizio, quando era appena stato pubblicato, aveva un fortissimo effetto di differenza di luce e di ombra". Uscendo dall'edificio della mappa "c'era subito un forte effetto di bloom e un glare molto, molto forte della luce che un po' ti accecava"; entrando, "c'era un forte buio che poi piano piano andava attenuandosi". "Quello è tutta opera di un post process."

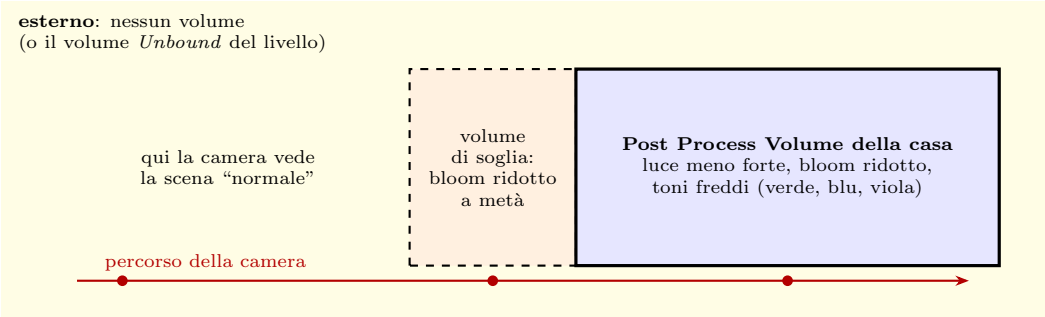


Figure 1.16.2: Una transizione fra esterno e interno. Il volume della casa cambia luce, bloom e colori solo quando la camera vi entra; un secondo volume "di soglia" con effetti intermedi rende il passaggio più morbido, come suggerisce il docente concatenando più post process. Ricostruzione schematica.

Infinite Extent (Unbound). Il funzionamento "non è limitato solo a quello che c'è all'interno del volume". Altrimenti, per applicare un effetto a tutto il mondo di gioco, bisognerebbe rendere il volume "molto molto grande" e coprire tutta la mappa, "che è scomodo e controintuitivo". Nei Details si cerca *infinite* e si trova **Infinite Extent (Unbound)**: "come spiega anche il cartellino che esce fuori" (il tooltip), spuntarla fa sì che il volume copra "tutta l'area del livello". Da quel momento, ovunque sia la camera, gli effetti del post process sono attivi su tutta la mappa. "Per comodità d'uso" il docente la toglie, così da vedere gli effetti entrando e uscendo dal volume.

Approfondimento: Blend Radius, priorità e un volume per livello. Le transizioni morbide che il docente ottiene concatenando volumi si possono ottenere anche con un solo volume: il parametro *Blend Radius* definisce una fascia attorno al bordo in cui l'effetto entra gradualmente, e il *Blend Weight* (da 0 a 1) la sua intensità complessiva. Quando la camera è dentro più volumi, Unreal li applica in ordine di *Priority* e mescola, proprietà per proprietà, solo i valori la cui casella è spuntata: è per questo che il volume della scena di default "non fa nulla" finché non si spunta qualcosa. Lo schema abituale è un volume *Unbound* a priorità bassa per l'aspetto generale del livello e volumi locali a priorità più alta per le stanze speciali. L'effetto di "abbagliamento" all'uscita da un edificio è in gran parte dovuto all'*esposizione automatica* (*eye adaptation*), che si vede più avanti. Romero e Sewell, nel capitolo 9, ricordano che allargando l'area di gioco il *PostProcessVolume* e il *LightmassImportanceVolume* vanno scalati fino a coprire la nuova area, perché "sono usati per l'illuminazione e gli effetti"; il resto è funzionamento generale dell'engine. Totten, nel capitolo 5, dedica ampio spazio al passaggio fra spazi aperti e protetti (*prospect* e *refuge*) e al loro contrasto come fonte di emozione: il cambio di luce all'ingresso di un edificio ne è la traduzione visiva.

1.16.6 Il Bloom

Che cos'è. Si parte da "un effettino di cui abbiamo già parlato la volta scorsa per andare a creare i god shaft": il **Bloom**. "È un effetto che ci permette di aumentare la brillantezza della luce e la forza dei riflessi che la luce ha sugli oggetti metallici, speculari o riflettenti in generale."

Intensity. Spuntati i due parametri, alzare l'**Intensity** ha un effetto "molto forte": "una luminosità quasi afosa" che rimanda subito "a una scena estiva o a un momento della giornata particolarmente illuminato, particolarmente intenso". Il parametro si aggiusta per decidere quanto forte si vuole la cosa. In teoria il bloom dovrebbe anche "sfocare le fonti di luce" e aumentare i riflessi sugli oggetti: il docente cerca un oggetto adatto e lo trova nella **torretta**, il cui riflesso "diventa immediatamente più brillante". "Più il materiale è pensato per questo tipo di effetto, migliore è il risultato."

Threshold. Il parametro sotto, il **Threshold**, "è una soglia": quanto deve essere forte "l'intensità della luce", in questo caso della directional (che il docente ritrova dopo un attimo: "l'avevo spostata la volta scorsa"), per attivare il bloom. La directional ha intensità **3**. L'esempio: in una stanza ci sono tre luci, di intensità 3, 10 e 50, e si vuole il bloom "soltanto per la luce da 50". Il threshold serve proprio a questo. "Di default è a **meno 1**, perché vuol dire che non c'è una soglia." Alzandolo, "smette di esserci l'effetto di bloom non appena ci avviciniamo all'intensità della luce" (figura 1.16.3). È utile per un bloom che deriva soltanto da alcune fonti di luce.

Method. Il **Method** "è un discorso di calcolo". **Standard** "è quello che utilizzeremo la maggior parte delle volte". **Convolution** è "molto, molto pesante": va usato solo se si fa "veramente soltanto qualcosa di cinematografico"; per lo sviluppo di un videogioco è una formula di calcolo troppo costosa.

Approfondimento: che cosa misura la soglia. Nell'engine il Threshold del Bloom non confronta direttamente l'*intensità* delle luci, ma la *luminanza* dei pixel dell'immagine già illuminata, dopo l'esposizione: a superare la soglia sono i punti più brillanti dello schermo, che spesso sono le sorgenti più forti ma anche i loro riflessi su superfici lucide. In pratica la regola del docente funziona, con questa precisazione: cambiando l'esposizione cambia anche ciò che supera la soglia. Il metodo *Convolution* usa una texture (il *kernel*) per simulare la diffusione reale di un obiettivo, con raggi e aloni realistici; costa molto di più del metodo Standard,

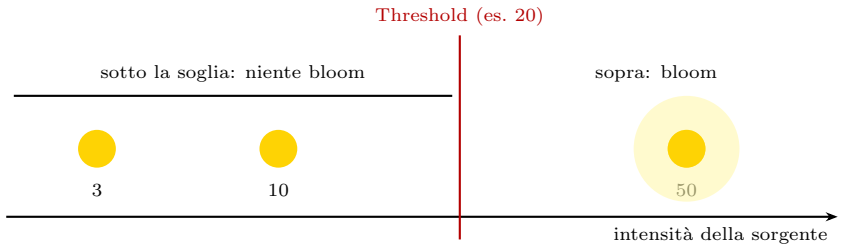


Figure 1.16.3: Il Threshold del Bloom con l'esempio del docente: tre luci di intensità 3, 10 e 50 e una soglia che lascia il bloom solo alla più forte. Il valore 20 è solo illustrativo. Ricostruzione schematica.

che somma alcune versioni sfocate dell'immagine. Funzionamento generale dell'engine; Romero e Sewell, nel capitolo 12, indicano proprio il bloom e il motion blur come esempi dei filtri di *Post Processing* da ridurre nelle *Engine Scalability Settings* se le prestazioni sono basse.

1.16.7 L'Exposure

Che cosa fa. L'**Exposure** "ci permette fondamentalmente di andare a determinare quanta visibilità hanno le ombre, anzi quanta visibilità ha la scena nonostante sia in ombra". Per spiegarlo il docente prepara un esempio con delle immagini ("così almeno senza bisogno di farlo direttamente in scena").

L'hangar. Si immagini un **hangar**, come nell'immagine a sinistra: una zona che, realisticamente, dovrebbe essere molto scura. In una **mappa PvP**, in cui i giocatori "devono spararsi a vicenda", questo eccesso d'ombra crea un problema: i giocatori hanno difficoltà a individuarsi, e gli scontri a fuoco diventano "molto frustranti", perché qualcuno può mettersi "acquattato" ("accecate", nella trascrizione) in un angolo e restare "completamente invisibile". Il parametro permette di simulare "un'adeguazione dell'occhio alle ombre": la zona che dovrebbe essere scura appare "come se fosse mezzo illuminata", con meno contrasto.

La leggibilità prima di tutto. "Questo chiaramente a livello di espressività e a livello di stile crea dei problemi", perché riduce la bellezza estetica della scena. "Tuttavia viene utilizzato principalmente per una questione di **leggibilità**: se il mio giocatore non sa dove si trova o dove si trovano gli avversari o cosa sta succedendo, la bellezza della scena conta fino a un certo punto." La regola: "preferire sempre la leggibilità alla componente di estetica".

I parametri. Modificando i parametri di **compensazione**, i valori **minimi e massimi** e la **velocità**, si determina "quanto viene compensata l'oscurità, quanto velocemente la camera si abitua, e quanto varia la gestione dell'ombra".

Approfondimento: l'esposizione automatica. In Unreal Engine 5 la sezione *Exposure* controlla l'*eye adaptation*, l'adattamento automatico dell'occhio virtuale alla luminosità media della scena. I parametri che il docente nomina sono, con ogni probabilità, *Exposure Compensation* (uno spostamento costante, in stop, verso il chiaro o lo scuro), *Min EV100* e *Max EV100* (i limiti entro cui l'esposizione può adattarsi: se coincidono, l'esposizione è fissa) e *Speed Up* e *Speed Down* (quanto in fretta l'occhio si abitua passando dal buio alla luce e viceversa). L'effetto di Rainbow Six Siege nasce da qui: un intervallo ampio fra minimo e massimo e una velocità bassa danno l'abbagliamento all'uscita e il buio che si attenua all'ingresso. In una mappa competitiva si fa spesso il contrario, restringendo l'intervallo perché nessuna zona diventi troppo scura; esiste anche la *Local Exposure*, che schiarisce le ombre senza bruciare le zone chiare (funzionamento generale dell'engine). Totten, nel capitolo 3, riprende da Kevin Lynch il concetto di *legibility*: la qualità di uno spazio che chi lo attraversa riesce a leggere e a ricordare, e che dovremmo cercare anche negli spazi di gioco. Nel capitolo 4 aggiunge che il *contrasto* (un oggetto chiaro fra oggetti scuri, un colore complementare a

quello dominante) è il modo principale per chiamare l'attenzione del giocatore: un'esposizione che appiattisce tutto toglie anche questo strumento, ed è il prezzo che il docente accetta in nome della leggibilità.

1.16.8 La Chromatic Aberration

Che cosa fa. La **Chromatic Aberration** è "un effetto molto semplice, molto d'impatto, che va gestito in maniera morbida". "Faccio prima a farvelo vedere": è un effetto di sfocatura che "va a suddividere i canali del colore", il blu, il verde e il rosso, che vengono "attivamente suddivisi", soprattutto "nelle parti laterali della scena". Crea "un effetto di confusione, di straniamento".

Con moderazione. "Se è visto dal giocatore per troppo tempo, può dare problemi, può essere fastidioso alla vista" ("occhio, che fa male agli occhi", aveva detto la volta scorsa). Di solito lo si usa "in momenti di acuto": l'intensità viene portata al massimo e poi ridotta di nuovo. Quando usarlo? Gli studenti propongono: "quando il personaggio è vicino alla morte", o in generale quando sta vivendo "qualcosa di scomodo". Il docente precisa: meglio usarlo "principalmente *non* in momenti di gameplay" (non "il mio personaggio ha pochi HP, allora aumenta l'intensità"), ma in **momenti narrativi** in cui bisogna mostrare che il personaggio è in uno stato di *stun* o di confusione. Gli esempi:

- "sono stato colpito da una **flashbang**": insieme alla luce si crea una sfocatura e una distorsione;
- "sto osservando" un'entità antica ("un antico turiano" nella trascrizione, con ogni probabilità un Grande Antico), "una roba **lovecraftiana**, per cui la mia sanità mentale inizia a sfasarsi completamente", e la vista "va a strapparsi in questo modo".

Nella scena di prova è "meno impattante di come potrebbe essere": con un'illuminazione più curata "trasmette subito un messaggio".

Start Offset. L'effetto si compensa con lo **Start Offset**: "va a creare un offset dal centro dello schermo fino ai lati", cioè decide "da quanta distanza dal centro" si applica l'aberrazione. Alzandolo piano piano, "la scena inizia ad essere sempre più chiara al centro" finché l'aberrazione resta solo ai lati e poi sparisce: "a uno, l'offset è tutto lo schermo", e non si vede più nulla. Diminuendolo, si vede l'immagine distorcersi piano piano anche, per esempio, in basso a destra. Si può così avere "il pieno controllo del centro della mia visuale" e l'estremizzazione dei colori solo sui lati (figura 1.16.4). (Nel frattempo il docente si accorge di aver appoggiato qualcosa sul tavolo che interferiva: "mi ero dimenticato che l'avevo messo sul tavolo".)

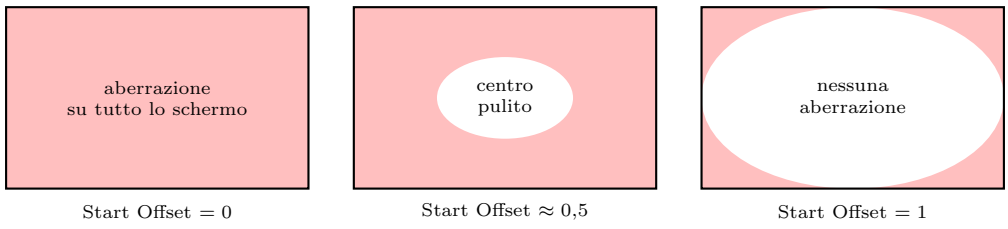


Figure 1.16.4: Lo Start Offset della Chromatic Aberration: dal centro dello schermo fino al valore impostato l'immagine resta pulita, oltre compare la separazione dei canali. A 1 l'effetto sparisce. Ricostruzione schematica.

Approfondimento: aberrazione e accessibilità. L'aberrazione cromatica imita un difetto degli obiettivi reali, che non fanno convergere nello stesso punto le diverse lunghezze d'onda: per questo è più forte ai bordi. Il consiglio del docente coincide con una pratica diffusa dell'accessibilità: effetti come aberrazione, motion blur, vignettatura forte e *film grain* affaticano la vista o danno nausea a una parte dei giocatori, e molti giochi offrono un'opzione per disattivarli. In Unreal si può farlo modificando a runtime le proprietà del post process (per esempio il *Post Process Settings* di una camera o di un volume da Blueprint) oppure,

per un picco di breve durata come la flashbang, animando l'intensità con una *Timeline*, lo strumento visto nelle prime lezioni del corso (funzionamento generale dell'engine). Totten, nel capitolo 5, ricorda che l'horror lavora sull'incertezza percettiva (l'ombra, la nebbia): l'aberrazione dell'esempio lovecraftiano è la versione "sullo schermo" della stessa strategia.

1.16.9 Il Lens Flare

Che cos'è. "Questa è un'altra molto comune, molto vista": "forse è il tipo di post-process con cui siamo più familiari", perché si vede spesso anche nelle immagini delle fotocamere. È la "scomposizione dello spettro della luce": dentro il volume la luce viene scomposta, e si vedono "gli esagoni che si creano". Si può usare "per diecimila motivi": per esempio quando il personaggio è appena uscito all'aperto dopo essere stato al chiuso, e la scomposizione "ci accoglie e ci dà un'idea di quanto effettivamente il sole sia forte", o una luce in generale.

I parametri.

- **Intensity**: quanto è forte. "Se non ricordo male" è influenzato anche dal **bloom**: ed "esatto", aumentando l'intensità del bloom il lens flare diventa più sfumato.
- **Tint**: il colore del flare, "magari prendiamo questo flare violaceo"; si può anche modificare a mano il riferimento dei canali RGB.
- **Bokeh Size** ("poke size": "sono ignorante su come si pronuncia"): la grandezza di "non lo so neanche come chiamarli, dei cristalli, degli esagoni della scomposizione della luce", per un effetto più o meno scomposto (il docente toglie il blu per farlo vedere meglio).
- **Threshold**: "funziona sempre nello stesso modo", la soglia minima di luminosità per avere l'effetto.
- **Bokeh Shape**: "devo essere onesto, non ho mai trovato un'utilità a questa cosa", ma si può cambiare la forma dei cristalli con una **texture**. Quella di default si chiama, nella trascrizione, "Presize default all"; cambiandola si hanno cerchi, pentagoni, "qualsiasi texture abbiamo": l'icona di un bersaglio, "un'immaginina con delle stelline". Le texture troppo dettagliate non rendono ("non si nota effettivamente il dettaglio"), ma per una questione di forma funzionano; la texture porta con sé anche il colore.
- **Tints**: per personalizzare "al 100%" il flare, si può modificare "ogni singolo cristallino": ce n'è uno viola, uno blu, uno verde, e si decide quali colori dello spettro "risultano maggiormente". Sono numerati da 0 a 7 (la trascrizione elenca "0, 1, 2, 3, 4, 5, 7").

"Il lens flare lo teniamo, che insomma è piacevole averlo."

Approfondimento: un difetto che diventa linguaggio. Il lens flare è un riflesso interno fra le lenti di un obiettivo, e i suoi esagoni hanno la forma del diaframma; in Unreal Engine 5 il lens flare del post process è un effetto *image-based*, calcolato a partire dalle zone più brillanti dell'immagine (per questo dipende dal bloom e dalla soglia), e la *Bokeh Shape* è appunto la sagoma del diaframma simulato. Nel cinema e nei giochi il flare comunica "guardi verso una luce forte" e, spesso, "stai guardando attraverso una camera": usato ovunque, indebolisce l'idea di una visione in prima persona; usato al momento giusto, come l'uscita all'aperto del docente, è un segnale di passaggio. Funzionamento generale dell'engine.

1.16.10 La Depth of Field

Mettere a fuoco. "Un'altra cosa molto importante che possiamo utilizzare nel momento in cui andiamo a fare un render di un'immagine, ma anche quando dobbiamo gestire le distanze", è la **Depth of Field** (la profondità di campo). Permette "una serie di modifiche di messa a fuoco": si modifica la **distanza focale**, cioè a quale distanza si mette a fuoco. Nella prova c'è "una leggera sfocatura", e più in là "qualcosa di meno

messo a fuoco, ma comunque non visibilissimo": "mettiamo che il nostro personaggio sia vagamente miope, insomma, vedrà in questo modo qui".

Gli altri parametri. Il docente ricorda altri parametri che permettono di determinare "la distanza a cui vediamo il blur e l'intensità del blur": "quale sia la forza della sfocatura e da quale distanza venga effettivamente attivata". Ma "ricordo funzionasse in maniera diversa": "credo che sulla 4 funzioni in maniera diversa". Promette di controllare durante la pausa e di riferire le eventuali modifiche; dopo la pausa, però, la trascrizione non riporta più nulla sulla depth of field.

Approfondimento: la profondità di campo in Unreal Engine 5. La differenza che il docente ricorda c'è davvero. In Unreal Engine 4 esistevano più metodi (*Gaussian*, *Bokeh*, *Circle*), ciascuno con parametri propri come le regioni di transizione vicina e lontana; in Unreal Engine 5 su PC e console resta di fatto un solo metodo, il *Cinematic* DOF, i cui parametri principali sono la *Focal Distance* (la distanza di messa a fuoco, in centimetri), l'*Aperture* o *f-stop* della camera (più è basso, più la profondità di campo è stretta) e la dimensione del sensore; sono presenti anche *Depth Blur km for 50%* e *Depth Blur Radius*, che aggiungono una sfocatura di distanza indipendente dalla messa a fuoco. Il *Mobile Depth of Field* che il docente vede nella lista della Camera Actor è la versione semplificata per dispositivi mobili, con distanze di transizione esplicite, ed è probabilmente il modello di funzionamento che ricordava. Sulla Camera Actor (*Cine Camera Actor*) la messa a fuoco si regola dalle *Focus Settings*, anche in modo automatico seguendo un actor (funzionamento generale dell'engine). In un gioco la profondità di campo è utile soprattutto per i menu, i dialoghi e le schermate, dove guida lo sguardo verso il soggetto: è la stessa funzione che Totten, nel capitolo 7, attribuisce alla cinematografia, "attirare l'attenzione del giocatore" sugli elementi che il designer vuole fargli vedere.

1.16.11 Il Color Grading

La cosa più famosa. "Forse la cosa più famosa per cui è conosciuta e utilizzata la post-produzione" è la **color correction**: applicare all'immagine "un nuovo stile, un nuovo tipo di colore" con una serie di parametri. Il docente richiude tutte le sezioni per vederle una per una.

Temperature. "Di base, il color grading parte dalla **temperatura**": una scena più calda o più fredda, da un estremo all'altro, con "questa renderizzazione più giallastra" o "completamente sbiancata e bluastra". Poi la **tinta** (*Tint*), per spingere la scena verso il magenta o verso il verde.

Global, Shadows, Midtones, Highlights. Oltre alla temperatura ci sono i parametri "che ci permettono di fare una customizzazione più completa": **Global**, **Shadows**, **Midtones** ("mittons") e **Highlights**. Sono la possibilità di fare color correction su tutta la scena (Global), solo sugli scuri, "nelle parti più buie, nelle ombre" (Shadows), "nei grigi, nei medi" (Midtones) e "nelle luci, nelle parti più illuminate" (Highlights) (figura 1.16.5).

Cinque sottoparametri. "Funzionano tutti allo stesso modo": ognuno ha la stessa lista di **Saturation**, **Contrast**, **Gamma**, **Gain** e **Offset**, da attivare con la spunta ed espandere con la freccia. Con la saturazione si decide quanto la scena sia satura o desaturata, con lo slider sotto. (A questo punto la trascrizione si perde per diverse righe in un blocco ripetuto senza senso; quando riprende, il docente sta parlando del **contrasto**.) Con il contrasto si gioca sulla differenza "fra le luci e le ombre": la si può accentuare o appiattire, "una sorta di maschera di contrasto del colore". "Stessa cosa gamma e gain": moltiplicano la color correction della scena, e si alzano o abbassano per modificare il colore.

Uno strumento avanzato. "Il post-process è un elemento molto molto avanzato": non perché sia difficile da usare, "ma nel senso che è difficile ottenere esattamente il risultato che volete". È un oggetto con cui "dovete sperimentare, mettervi là, fare esperienza" di come funziona e di quali possibilità ha. "Se dovessimo esplorarlo al 100% ci vorrebbe molto di più di una lezione, e probabilmente non sarei neanche la persona adatta." Il docente ha mostrato le lenti "che potreste utilizzare di più" per creare un certo *mood* e un

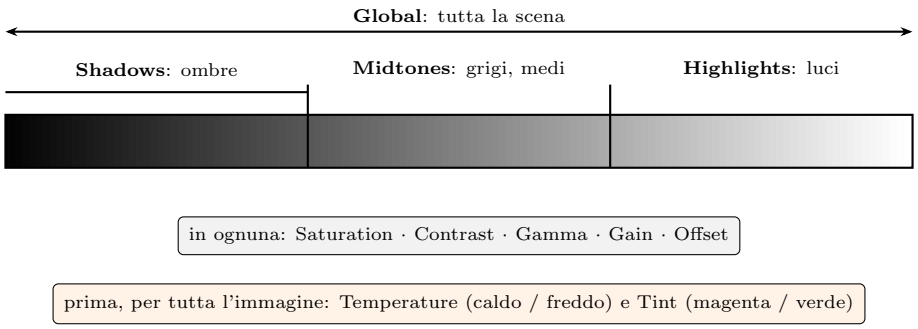


Figure 1.16.5: Le zone del Color Grading. Global agisce su tutta l'immagine, Shadows, Midtones e Highlights sulle zone scure, medie e chiare; ciascuna ha gli stessi cinque sottoparametri. I confini fra le zone sono sfumati e regolabili. Ricostruzione schematica.

certo tipo di scena; si ferma qui per parlare di altre cose "ugualmente importanti", "più basilari forse, meno avanzate". Chi vuole può partire da questa base: mettere in scena il proprio post process e "smontarlo un pochino".

Prima le luci, poi il post process. La post produzione "influisce grandemente sulla luce": a seconda del tipo di luce in scena si avranno effetti diversi. L'ordine giusto: prima si costruisce l'illuminazione, "come abbiamo visto con i light actor qualche lezione fa", e solo quando le luci sono posizionate si fa questo lavoro. "Capisco che è stato molto brutale e molto rapido", ma "più che mostrarvi gli elementi di base e incitarvi a provare a sperimentare" non c'è tempo. "Oggi purtroppo corriamo un po'."

Approfondimento: colore e significato. Il color grading di Unreal segue il modello dei programmi di correzione colore del cinema: Saturation, Contrast, Gamma, Gain e Offset sono i classici controlli di *lift-gamma-gain* applicati per zone, e i confini fra Shadows, Midtones e Highlights si regolano con *Shadows Max* e *Highlights Min*; per un controllo totale si può usare una *Color Grading LUT*, una tabella di colori preparata in un programma esterno (funzionamento generale dell'engine). Totten, nel capitolo 4 ("Basic Color Theory"), fornisce il vocabolario per scegliere: i colori *analoghi* (tre colori vicini sulla ruota, come blu, blu-viola e viola) creano un'atmosfera armonica e, in quel caso, "un senso cupo o freddo"; rosso, rosso-arancio e arancio danno una sensazione calda; i *complementari* (opposti sulla ruota) creano contrasto. È esattamente il "posto lugubre" del docente, caricato di verdi, blu e viola. L'esempio di Totten è *Bioshock Infinite*: una cappella dominata dal blu in cui le uscite sulla strada principale sono segnate da elementi complementari, una luce arancione o un fedele in tunica bianca, mentre i passaggi secondari restano in tinte analoghe. Il color grading regola il colore dominante; il contrasto che guida il giocatore va poi costruito con le luci e gli oggetti.

1.16.12 La domanda sull'effetto in due toni

La domanda. Prima di passare al packaging arriva una domanda in chat: "si possono applicare effetti di luce più estremi, magari su altri programmi?" Per esempio "rendere il livello in due toni", "solo rosso e nero", per ottenere "un effetto tipo **Limbo**". "Sì, si può fare, ma anche già soltanto all'interno di Unreal, utilizzando un post process volume."

La zanzara e il cannone. Il docente sposta però la questione: non è tanto "se Unreal lo può fare oppure no", ma "per fare quella cosa, è davvero necessario utilizzare Unreal?" Se l'idea è creare qualcosa di esteticamente molto semplice, in bianco e nero, si può forse realizzarlo con altri engine. "Teniamo sempre a mente che Unreal è un engine pesante che fa le cose in maniera pesante." Se si vuole rifare Limbo, "che magari è anche in 2D" e quindi più semplice, e lo si fa in Unreal, "stiamo sparando a una zanzara con un cannone": "la zanzara l'ammazziamo, però smontiamo pure quello che c'è dietro". Quanto agli "altri

programmi": se si fa un'immagine in Unreal, uno screenshot di un ambiente, e la si modifica "in post" con un programma di fotoritocco, "sì, lo posso fare".

Mallory. Lo studente cita un gioco, "Mallory" ("Mellori" più avanti; il nome esatto non è ricostruibile dalla trascrizione), un titolo per Switch fatto in Unreal, e condivide dei link. Il docente non lo conosce: "poi me lo guardo durante la pausa". In ogni caso, "se è fatto in Unreal si può fare tranquillamente": l'idea stessa del Post Process Volume è avere un oggetto in scena che permette di passare da un tipo di post produzione a un altro o di controllarla meglio; "una volta che mi impratichisco con lo strumento posso fare un po' quello che mi pare". Serve però anche gestire bene l'illuminazione: ci si costruisce l'illuminazione "come è gestita in quel titolo" e poi la si modifica a posteriori. (Il seguito, dopo la pausa, è nella sezione 1.16.18.)

Approfondimento: scegliere l'engine. La metafora del cannone tocca un tema di produzione: il costo di un engine non è solo la licenza, ma il peso dei progetti, i tempi di compilazione degli shader e del packaging (che il docente sperimenterà di lì a poco) e le macchine richieste ai giocatori, come ha scoperto lo studente con il portatile che "non reggeva". Per un gioco 2D stilizzato un engine più leggero è spesso la scelta più efficiente; Unreal ha comunque strumenti 2D (*Paper2D*, gli sprite e i flipbook citati nella lezione precedente). Un effetto "in due toni" in Unreal si ottiene con un *Post Process Material*: un materiale con dominio *Post Process*, assegnato nei *Rendering Features* del volume (*Post Process Materials*), che legge il colore della scena e lo riduce a due colori secondo una soglia di luminosità (funzionamento generale dell'engine). Totten, nel capitolo 4, usa proprio *Limbo* come esempio di linguaggio visivo: il gioco gioca il suo stile a silhouette contro le convenzioni del giocatore, come nel puzzle della pressa in cui il "pulsante" rialzato non è il vero meccanismo. Lo stile in due toni, lì, non è un filtro sopra il gioco: è parte delle regole con cui il giocatore legge lo spazio.

1.16.13 Il packaging: che cos'è e dove si avvia

Che cos'è. "Il packaging è un processo che viene fatto principalmente da Unreal Engine, in realtà, per cui noi dobbiamo andare a curare soltanto alcune impostazioni." È ciò che "ci consente di andare a creare un **eseguibile** con quello che noi abbiamo all'interno dell'editor".

Platforms. Il packaging si avvia dal pulsante **Platforms** della barra degli strumenti. Scendendo si vedono "tutte le varie piattaforme per cui potremmo sviluppare qualcosa": **Android**, **iOS**, **Linux**, **Mac**, **Windows**. Accanto ad alcune c'è "sempre questo segnale di attenzione": il docente è su Windows, e il progetto "non è gestito o pensato" per funzionare anche sugli altri sistemi. Nella voce **Windows** ci sono il **Package Project**, "il pulsantone che io premo se voglio far partire questo processo", e la scelta della **configurazione**. "Queste sono pochissime impostazioni, e non sono neanche lontanamente tutte quelle che ci servono": le altre sono nei **Project Settings**, la "macro sezione di setting e di impostazioni per tutto il nostro progetto" già vista con Igor (figura 1.16.6).

Il plugin di Fab. Un'altra finestra che può servire è quella dei **Plugins**, che permette di attivare e disattivare i plugin. "Da quello che so, il plugin di **Fab** dà dei problemi con il packaging": quindi, prima di impacchettare, lo si disattiva. "Non so se è stato risolto. Potrebbe essere stato risolto, dovrei informarmi."

1.16.14 Le impostazioni del progetto

Maps & Modes. Nei Project Settings, nella pagina **Maps & Modes**, "oltre alla Game Mode", ci sono le **Default Maps**: **Editor Startup Map** e **Game Default Map**.

- L'**Editor Startup Map** è la mappa con cui si apre l'editor: se la si cambia, "ogni volta che io aprirò questo progetto lui mi partirà da quell'altra mappa", e per tornare al livello di prima bisognerà riaprire a mano la Third Person Map.

- La **Game Default Map** è "allo stesso modo la mappa di partenza che avremo all'interno del nostro eseguibile". Se il gioco ha un menu che porta alla mappa di gioco, non si mette la Third Person Map: "di solito quello che si fa è mettere un **menu principale**", o una schermata che, dopo aver cliccato qualche pulsante, porta alle mappe che si vogliono giocare.

Sono due impostazioni importanti: "se mai vi capiterà di dire: oddio, dove è andata la mia mappa, perché si è aperto così, dove sono finito, cosa è successo"; oppure "per quale motivo ho fatto il packaging e non mi fa partire da dove dico io": è la Game Default Map da modificare. (Il docente scorre altre voci della pagina, "ma questa non è necessaria", senza nominarle.)

Supported Platforms. Un'altra pagina è quella delle **Supported Platforms**. Di default è impostata su **All Platforms**; togliendo la spunta e lasciando solo ciò che interessa si può avere, per esempio, soltanto **Windows** come piattaforma per il packaging, "perché il nostro progetto magari non supporta tutti gli altri sistemi operativi".

La pagina Packaging. "Ultima cosa, ma la più importante": c'è una pagina tutta dedicata al **packaging**. "Sono un sacco di informazioni, un sacco di parametri", ma "la cosa positiva è che la maggior parte di questi parametri sta bene come sta": non servono "modifiche assurde". Alcune voci si vedono subito, altre solo aprendo gli **Advanced**. Le prime: **Build**, "va bene così" (l'opzione "if project has code or running a locally built editor", che lascia decidere all'engine se compilare); poi la **Build Configuration**.

1.16.15 Le configurazioni di build

Cinque configurazioni, tre disponibili. La **Build Configuration** decide "che tipo di configurazione avrà il nostro eseguibile": **Debug**, **DebugGame**, **Development**, **Test** e **Shipping**. "Per la build che abbiamo scelto" ne sono attive soltanto tre (le stesse che si ritrovano poi nel menu Platforms: DebugGame, Development e Shipping). Le differenze (tabella 1.16.1):

- **Debug** e **DebugGame** "sono modalità di debug": un eseguibile in cui c'è ancora una **console di comandi** per inserire comandi, fare aggiustamenti, "vedere le linee, vedere quello che sta succedendo, vedere dei text che vengono printati". "È una versione super di test."

- **Development** è "la configurazione di default di Unreal". Rimuove la maggior parte della struttura di debug, aumenta un po' l'ottimizzazione della build, "ma mantiene comunque alcuni strumenti che possono essere utilizzati per il testing". "Se debug è un eseguibile che può essere di uso a un **programmatore**, development può essere di uso a un **designer**": si può ancora modificare qualcosa e usare degli input, ma serve soprattutto per testare e poi ritoccare in engine.

- **Shipping** è la modalità "per la distribuzione": "il progetto finito, il gioco che le masse giocheranno, si spera le masse". È "la versione più ottimizzata possibile della build", "la build da vendita, praticamente".

Shipping non ottimizza da solo. "Aspettate: questo non vuol dire che lui vi ottimizzi le cose in automatico." L'ottimizzazione "è tutto un discorso a parte, che va fatto separatamente": far sì che non ci siano troppe linee di codice, che non ci siano texture a qualità troppo alta o non utilizzate, eccetera. Shipping "vuol dire semplicemente che lui va a togliere tutta una serie di strumenti che non servono più".

Le lingue. Aprendo gli Advanced si trovano le **lingue** del packaging (*Localizations to Package*): "possiamo andare a selezionare che tipo di lingue dovranno essere utilizzate". "Di default c'è solo l'inglese." Se si è localizzato qualcosa, per esempio perché ci sono già impostazioni e parametri che fanno riferimento a certe lingue, qui si scelgono quelle da includere.

Configurazione	Che cosa contiene (lezione)	Per chi	Nota
Debug	console di comandi, linee, testi stampati: "versione super di test"	programmatore	richiede un progetto con codice C++
DebugGame	come sopra, debug del codice di gioco	programmatore	disponibile
Development	default di Unreal; meno debug, un po' più ottimizzata, alcuni strumenti di test	designer, tester	disponibile; console attiva
Test	vicina a Shipping, con qualche strumento di profilazione	QA	richiede codice C++
Shipping	la versione finale, senza strumenti di sviluppo	i giocatori	disponibile; non ottimizza da sola

Table 1.16.1: Le cinque configurazioni di build. Nel progetto del docente ne sono attive tre; la colonna "Nota" integra con il funzionamento generale dell'engine.

Approfondimento: le configurazioni secondo Romero e Sewell. Romero e Sewell, nel capitolo 12 ("Building and Publishing"), descrivono lo stesso percorso: *Platforms* → piattaforma → *Package Project*, la scelta della cartella, la finestra che avvisa che l'engine sta impacchettando e, in caso di problemi, "i dettagli dell'errore nella finestra dell'output log". Prima, nei *Project Settings*: la pagina *Description* (nome, miniatura, descrizione del progetto), la pagina *Maps & Modes* ("spesso dovrete designare una mappa dedicata al menu principale come prima mappa da caricare", e assicurarsi che sia lei a gestire quale mappa si carica dopo) e la pagina della piattaforma, dove per Windows si cambiano lo *splash screen* e l'*icona* dell'eseguibile. Sulle configurazioni il libro precisa che nei progetti *solo Blueprint* le opzioni principali sono **Development**, che contiene informazioni di debug utili a trovare errori, e **Shipping**, "più pulita", da usare per la versione da distribuire; Debug e Test richiedono un progetto con codice C++ (ed è per questo che nel progetto del docente sono spente). Il libro nomina anche la *Staging Directory*, la cartella in cui viene salvata la build, l'opzione *For Distribution*, richiesta per pubblicare su App Store o Google Play, e lo *Zip Project* del menu File, che comprime i file essenziali del progetto per passarlo a un'altra persona. Una nota importante dello stesso capitolo: Unreal Engine 5 può creare build Windows solo da Windows e build macOS solo da macOS; è il "segnale di attenzione" che il docente vede accanto alle altre piattaforme.

1.16.16 Le mappe e le cartelle da cucinare

Cook only maps. "Fare il packaging di base vuol dire che lui prende tutto il progetto e lo rende pronto per essere un eseguibile", con un certo metodo di compressione. Spuntando **Cook only maps** ("hook only maps" nella trascrizione), invece, il packaging usa "unicamente le mappe": si può dirgli di usare soltanto la Third Person Map, e lui "si vede cosa c'è dentro", mette nel pacchetto "tutto quello che serve per far funzionare" quella mappa "e non mette il resto delle cose".

List of maps to include. Insieme a Cook only maps serve la voce **List of maps to include in a packaged build**. Si aggiunge un elemento, si clicca sui tre puntini e si cerca la mappa: "volete level 1, level 2, menu? Magari vi serve il menu". Nel caso della prova serve soltanto la Third Person Map (il file `.umap`): "non serve andare a inserire anche il build data" (il file `_BuiltData`) "o lo `.uasset`". Si possono aggiungere altre mappe, copiarle, eliminarle.

Directories to never cook. "Un'altra cosa che ci torna utile" è **Directories to never cook**: le cartelle che non devono mai finire nel pacchetto. Si aggiunge un elemento ("un index zero") e si sceglie la cartella. Gli esempi:

- "nulla di *GameApps*" (il nome della cartella, nella trascrizione, è "game apps") "deve finire dentro il packaging";
- una cartella con "tutta una serie di modelli 3D molto fighi" che non si è riusciti a inserire e che magari si vogliono usare in un **DLC** più avanti.

I motivi sono due. Il primo è il **data mining**, "diventato super popolare adesso": i file messi nel gioco "possono essere ritrovati da chi va a scavare". Il secondo: sono comunque cose "che vanno ad essere cucinate e vengono comunque inserite all'interno della build". Togliere le cartelle che non servono "ci fa risparmiare giga": un progetto più piccolo e più semplice da gestire (tabella 1.16.2).

Impostazione	**Dove**	**A che cosa serve**
Package Project, configurazione	Platforms → Windows	avviare il packaging; scegliere DebugGame, Development, Shipping o le impostazioni del progetto
Plugin di Fab	Edit → Plugins	disattivarlo prima del packaging (problemi noti, forse risolti)
Editor Startup Map	Maps & Modes	la mappa con cui si apre l'editor
Game Default Map	Maps & Modes	la prima mappa dell'eseguibile: di solito il menu
Supported Platforms	Project Settings	limitare le piattaforme (per esempio solo Windows)
Build	Packaging	lasciare il default
Build Configuration	Packaging	Debug, DebugGame, Development, Test, Shipping
Localizations to Package	Packaging (Advanced)	le lingue incluse; default inglese
Cook only maps	Packaging (Advanced)	impacchettare solo ciò che serve alle mappe elencate
List of maps to include	Packaging (Advanced)	le mappe (`.umap`) da includere
Directories to never cook	Packaging (Advanced)	cartelle escluse: asset scartati, DLC, spazio
Show Output Log	finestra del packaging	seguire il processo e trovare gli errori in rosso

Table 1.16.2: Le impostazioni del packaging viste a lezione, nell'ordine in cui il docente le mostra.

Approfondimento: che cosa vuol dire "cucinare". Il *cooking* è la fase in cui l'engine converte gli asset dal formato dell'editor a quello ottimizzato per la piattaforma di destinazione: compila gli shader, comprime le texture, elimina i dati che servono solo all'editor. Il packaging vero e proprio raccoglie poi gli asset cucinati in archivi (*.pak*, o *IoStore* in Unreal Engine 5) accanto all'eseguibile. Con Cook only maps disattivato l'engine cucina di norma tutto il contenuto del progetto, compresi gli asset mai usati; con l'opzione attiva segue le *reference* a partire dalle mappe elencate. Un'avvertenza: un asset caricato solo per nome, a runtime (per esempio con un percorso scritto in una stringa), non è una reference visibile al cook e rischia di restare fuori; per questi casi esistono le voci *Additional Asset Directories to Cook*. Anche *Directories to never cook* va usata con attenzione: se un asset escluso è referenziato da una mappa inclusa, il cook segnala un errore o il gioco lo perde (funzionamento generale dell'engine). Totten, nel capitolo 2, parla di *modular design* e di "schedule" dei pezzi di un livello: sapere fin dalla pre-produzione quali pezzi servono rende più facile anche sapere, alla fine, che cosa lasciare fuori.

1.16.17 L'avvio del packaging e l'Output Log

Dove salvare. Per avviare si torna al menu Platforms: lì si vedono le configurazioni ("debug", **Development** e **Shipping**, "perché sono le tre possibili"), oppure *Use Project Setting*, cioè quelle della pagina Packaging. Il docente sceglie **Development**. Con **Package Project** si apre una finestra che chiede dove creare il pacchetto; il docente sceglie il desktop, e parte il processo, accompagnato da un "suonino".

La prima volta è lenta. "Fare un packaging la prima volta è molto lento", anche se è solo di una mappa. "Dopo il primo packaging diventa tutto molto più veloce." Mentre lavora ("parte *Cancel*, chiaramente", per

interromperlo) si può aprire lo **Show Output Log**: lì si vede tutto quello che sta succedendo. Scorrendolo ("con la rotella del mio mouse che è rotta è tremendo") si controlla se va tutto bene, se ci sono errori o "piccoli problemi", o cose che creano problemi al packaging. In quel momento l'engine sta facendo "la compilazione degli shader".

La scritta rossa. "Arriverà un punto in cui magari c'è una scritta rossa." La scritta rossa "vuol dire che c'è un problema che gli impedisce di fare il packaging". Si legge che cosa dice, si risolve quel problema, e si riprova.

Che cosa esce. Se il processo finisce, nella cartella scelta si crea una cartella che, "se non gli viene dato un nome", si chiama **Windows No Editor** (in Unreal Engine 5 è semplicemente *Windows*; *WindowsNoEditor* era il nome di Unreal Engine 4). Dentro c'è l'**eseguibile**: avviandolo parte il progetto, che in questo caso sarebbe soltanto la Third Person Map, "perché è l'unica mappa che gli abbiamo detto di cucinare". Il personaggio spawna sul **Player Start** e può andare in giro: "non c'è un obiettivo, non c'è una fine, non c'è nulla".

I confini della mappa. "Chiaramente, per un packaging, non dovrebbero esserci posti in cui puoi buttarti di sotto nel nulla, come il bordo della mappa"; né posti "in cui io non voglio che i giocatori vadano". Nessuna domanda sul packaging. Siccome ci vorrà "un po' troppo tempo", il docente anticipa la pausa, "in modo che forse durante la pausa lui finisca di calcolare" e si possa vedere che cosa genera. Dopo la pausa: "un paio di altri attori" utili quando si fa un livello, il Blueprint delle **spline**, "che potete utilizzare per un sacco di elementi diversi", e, "se ci avanza un po' di tempo", le **decal**, "un altro strumento di decoro, di arredo" per aggiungere dettagli in giro per la mappa. "Facciamo un 15 minuti di stop. Ci rivediamo alle 5 e mezza."

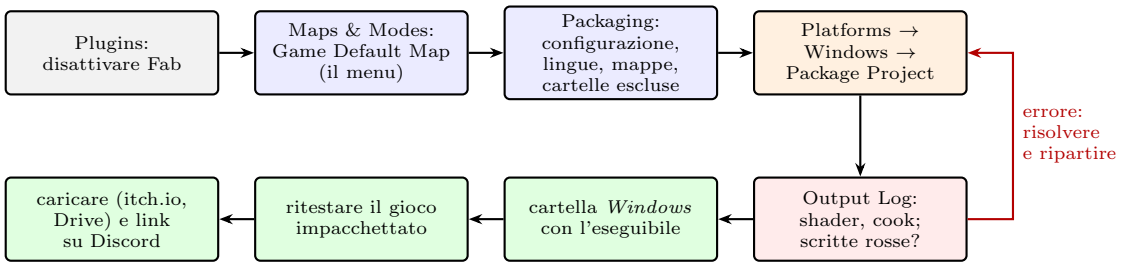


Figure 1.16.6: Il percorso del packaging come emerge dalla lezione, fino al consiglio finale di ritestare il gioco impacchettato e condividerlo. Ricostruzione schematica.

1.16.18 La pausa e Mallory

Il packaging non finisce. Durante la pausa la trascrizione contiene soltanto il blocco ripetuto già visto, che non appartiene alla lezione. Alla ripresa: "purtroppo noi ci siamo, ma Unreal ancora no, ancora non ho finito col packaging". Il docente lo interrompe, "perché sennò mi impedisce di fare altre robe": "non pensavo ci volesse così tanto tempo". Il risultato del packaging, quindi, non viene mostrato.

Mallory, visto. Il docente ha guardato il gioco segnalato prima. "Sembra molto carino": non l'ha potuto giocare, ma dal video "sembra un platform simpatico con un bell'ambiente". Non gli è sembrato che ci sia "un eccessivo utilizzo del post-process". Quello che fanno "sicuramente" è cambiare il modo in cui vengono renderizzate luci e ombre: "ci sono delle parti con le ombre che sono molto rossastre rispetto alla luce, che magari è una luce più neutra", "una piccola vignettatura" e "un effetto di sfocatura messa davanti alla camera". Lo studente ha in mente anche altri titoli; il docente osserva che il genere platform "ora come ora è messo in mano agli indipendenti": "non ci sono più grossi titoloni che fanno quel tipo di roba". È giusto cercare "cose carine fatte da persone in giro", su Twitch o fra i piccoli titoli su Steam. "Purtroppo non è proprio il mio genere", a parte "i vecchi classiconi PS1, PS2".

Come rifare quell'effetto. Le indicazioni del docente:

- una **Dirt Mask**, come effetto di camera: una texture sovrapposta alla lente. Fra le texture del progetto non ce n'è una adatta; una "un po' più fitta" dà già "questa specie di effetto statico, tra virgolette, sulla camera", da gestire poi con i colori corretti (quella provata rende la Dirt Mask rossa: "se riesci a fartela di effetto statico perfetto, potrebbe non essere male");
- negli **Image Effects**, una piccola **vignettatura** (*Vignette Intensity*), che "va a scurire lo schermo" nelle parti esterne e dà "già di base" un mood un po' più scuro;
- il resto "è tutta **color correction**": modificare i colori delle ombre per avvicinarli a quelli delle luci, desaturare un po' la scena ("c'è fondamentalmente una scala di grigio e in più c'è il rosso, barra rosa") e mettere "un accento sul rosso" dove c'è il rosso.

Monument Valley. Mallory ricorda al docente, per la struttura dei livelli "molto geometrici, molto quadratosi", i **Monument Valley**: "non sono dei platform, sono dei puzzle game misto opera estetica". Si basano sull'**illusione ottica**: il personaggio "cammina su un corridoio e poi improvvisamente inizia a camminare sulla parete e poi passa dietro la scena", e bisogna ruotare il livello per capire dove sta andando. "Faccio fatica a descriverli, però cercali, sono molto carini." Poi si torna al programma: "post process, rapidamente e bruttamente, ma lo abbiamo visto".

Approfondimento: accentare un colore. L'"accento sul rosso" in una scena desaturata è un'applicazione diretta del contrasto di Totten (capitolo 4): un elemento di un colore diverso dal dominante attira l'attenzione, e se il colore è sempre lo stesso diventa un segnale che il giocatore impara a leggere. In Unreal non c'è un controllo per "desaturare tutto tranne il rosso" nel color grading standard: lo si ottiene con un Post Process Material o, più semplicemente, desaturando con il post process e dando agli oggetti rossi materiali con un po' di *emissive*, così che restino saturi anche dopo la correzione. La vignettatura si regola con *Vignette Intensity* negli *Image Effects*; la Dirt Mask nella sezione *Lens*, con *Dirt Mask Texture*, *Intensity* e *Tint*, ed è visibile solo dove c'è bloom (funzionamento generale dell'engine). Totten, nel capitolo 4, nota anche come le associazioni dei colori passino nei giochi: rosso e verde per la salute, blu per la magia.

1.16.19 Le spline: che cosa sono

Una linea. Il docente ha preparato due Blueprint "al volo prima di cominciare la lezione". Prima, però, "che cos'è una spline?" È un **component** che si può aggiungere ai Blueprint, ma "in realtà è proprio un tipo di oggetto" con una funzione ben specifica, che si usa anche per modificare il funzionamento del **landscape** (le *landscape spline*). Il docente crea un Blueprint di prova con un **Spline Component**; il tooltip dice che è "una spline shape che può essere usata per altri scopi, tipo animare oggetti". "È una linea, fondamentalmente."

Punti e tangenti. Messa in scena, la spline "non è nient'altro che una linea" che si può allungare, curvare e arricchire di punti (figura 1.16.7):

- all'inizio ha **due punti**, l'inizio e la fine;
- cliccando sul punto finale compaiono altri due punti, la **tangente**; spostandola, la spline si curva;
- per creare **nuovi punti** si fa come quando si copia un oggetto: sulla freccia del gizmo, **Alt** premuto, si trascina, "e ho fatto una copia"; ogni punto nuovo curva la linea, si può alzare, spostare, fino a costruire "una figura, una struttura";
- con il tasto destro, **Delete Spline Point** elimina un punto (la spline si riempie da sola fra i punti rimasti, "lui autofilla") e **Add Spline Point Here** ne aggiunge uno.

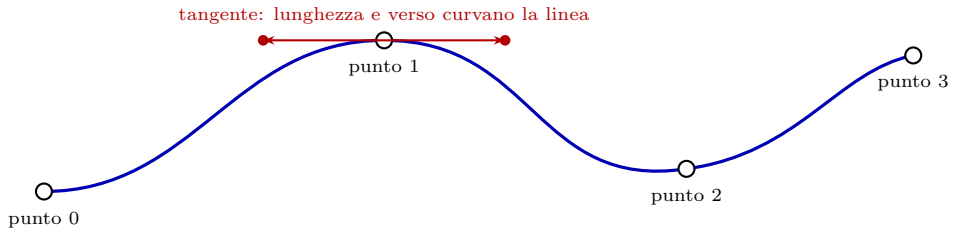


Figure 1.16.7: Una spline con quattro punti. Ogni punto ha una posizione e una tangente; i segmenti sono uno in meno dei punti. Ricostruzione schematica.

1.16.20 A che cosa servono le spline

Il muro di spine. "Immaginate di poter piazzare un oggetto lungo questa linea, o di poter modellare un oggetto lungo questa linea." La storia che il docente racconta gli è successa "quando stavo in Accademia", quando aveva cominciato a "smanettare con questo strumentino". Bisogna creare un **muro di spine**, come nella *Bella addormentata nel bosco*, "tutte le spine attorno al castello". Due strade:

- andare dall'"amico grafico" e chiedergli "un muro di spine fatto così, così e così": una mesh "non particolarmente impegnativa", ma che richiede tempo e che, se va modificata, deve tornare in un software di modellazione come **Blender** o **Maya**;
- "una roba un po' più intelligente": farsi creare soltanto un **cilindro di spine** e *replicarlo* lungo una spline, così che la mesh "viene ricreata e viene modificata sulla base della spline", seguendone l'andamento.

Altri usi. "Al di là di questa cosa", la struttura serve per molte cose: una **pista** per un gioco di macchine ("vi create un blueprint di spline con la strada"); un **muro** più stondato nel blockout o nella gym, per esempio "che sale a spirale verso l'alto", quando "non avete nulla che ve lo fa fare in maniera comoda". Il docente lo mostra "in due forme, in due versioni": una "più semplice e più brutta", l'altra "un po' più funzionale". I Blueprint si chiamano **Spline by Length** e **Spline by Point**.

La mesh del tubo. Per la prova usa un tubo, "un oggetto molto semplice" che è anche nel progetto degli studenti: **SM_PipeSplineBody** ("Sm pipe spline body"). Il docente prima dice che "non è fatto benissimo per essere un oggetto da spline", poi si corregge: "ho detto una cazzata, l'avevo vista in maniera veloce prima". Funziona bene perché è **segmentato**: ha i suoi triangoli e "questa spezzatura in orizzontale", è **modulare** ("cilindro, cilindro, cilindro"). Questo permette al Blueprint di replicarlo "senza andarlo a spezzare", perché la struttura del tubo viene replicata cilindro per cilindro. Nella prima versione, allungando la spline il cilindro si allunga; piegandola, si piega "però si spezza", perché è fatta "in una maniera meno ottimale"; nella seconda "si spacca" ancora, ma "dovrebbe notarsi di meno". La differenza (figura 1.16.8): "il primo blueprint, by point, *stretcha* l'oggetto; questo invece ricrea l'oggetto per lunghezza, quindi è un pochino più fedele".

1.16.21 Il Blueprint per punto

La struttura. Nel **Spline by Point** la viewport contiene la spline e "un oggetto che poi gli abbiamo attaccato con un nodo"; l'Event Graph è vuoto, e tutto è nel **Construction Script**. "È una struttura abbastanza semplice", usabile "tranquillamente" per un lavoro di blockout, "di prototipazione". Alcune cose il docente le fa "un po' più veloce, perché probabilmente le avete già viste anche con Igor". Il grafo (figura 1.16.9):

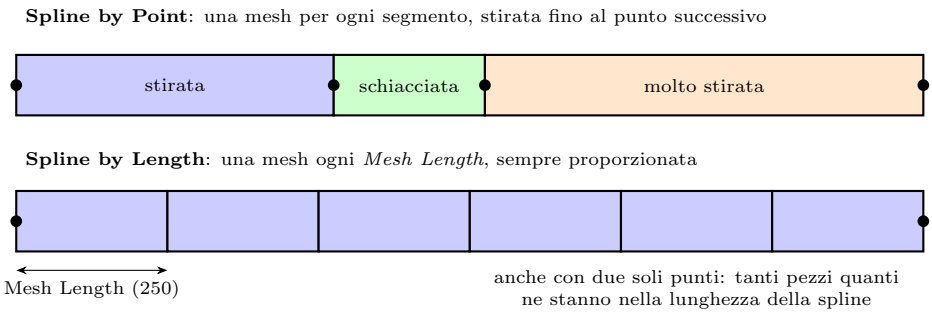


Figure 1.16.8: Le due versioni del Blueprint su una spline dritta. Per punto, il numero di mesh è uno in meno dei punti e ciascuna si deforma fino al punto successivo; per lunghezza, il numero di mesh dipende dalla lunghezza della spline divisa per quella della mesh. Ricostruzione schematica.

1. un **For Loop**, "perché dobbiamo loopare un oggetto". Il *First Index* è **0**, "la base dell'indice". Il *Last Index* viene dal numero di punti della spline, preso trascinando la spline nel grafo e usando **Get Number of Spline Points**; al numero si **sottrae 2**. Il motivo detto a lezione: "sono i due punti che abbiamo già di base, l'inizio e la fine", che "non ci interessano per il calcolo" (vedi l'approfondimento più sotto per la ragione esatta);
2. dal corpo del loop, **Add Spline Mesh Component**, con "tutta una serie di parametri": soprattutto la **Static Mesh** da usare (qui SM_PipeSplineBody), i materiali, *Start Position*, *Start Tangent*, *End Position*, *End Tangent* e il **Forward Axis**;
3. **Set Start and End**, che dà al componente l'inizio e la fine come punti di riferimento;
4. **Attach Component to Component**, che attacca il componente creato al Blueprint, con il **Default Scene Root** come *parent*;
5. due nodi **Get Location and Tangent at Spline Point**, con la spline come target: il primo con l'indice del loop (posizione e tangente di partenza), il secondo con l'indice **più 1** ("il punto successivo"), per la fine. "In questo modo lui, fra punti, va a creare una replica del nostro oggetto."

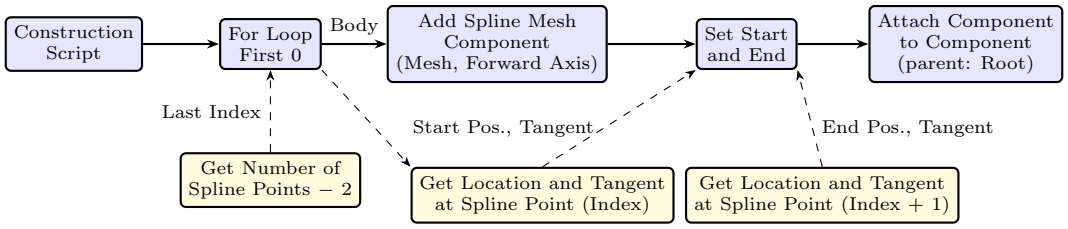


Figure 1.16.9: Il Construction Script del Blueprint *Spline by Point*. Le frecce continue sono il flusso di esecuzione, quelle tratteggiate i dati. Ricostruzione dai nodi nominati a lezione.

Il Forward Axis. Il docente cambia la mesh con un muro modulare (cerca "modulare" nel Content), compila, salva, e il risultato non è quello voluto: "lo sta replicando", ma non come si vorrebbe. Il motivo è il **Forward Axis**: "qual è l'asse frontale dell'oggetto, qual è l'asse allineato con la spline". Per il tubo era rimasto su **X**, ed era giusto: aprendo la mesh, il gizmo mostra che il tubo è allineato con l'asse X ("Z sono verticali, l'asse delle Y va in questa direzione"), e il tubo deve allungarsi proprio lungo X. Per il muro il docente prova **Y** ("presumo"), ottiene un muro da abbassare e si confonde ("aspetta, la stessa Y, sono confuso"); alla fine il muro funziona con l'asse **Z**, "forse per come era stato costruito". "Questo è probabilmente anche il fatto che lui non è fatto per fare questa roba."

Lo stretch. Con il muro si possono fare "le pareti di una stanza", ruotare, modificare; "tuttavia, ogni volta che io lo modifico, il muro si *stretcha*". Il muro "non la sta prendendo benissimo", perché non è pensato per essere modellato così: servono mesh adatte. Il docente prova con un **cubo**: "l'effetto non è gradevolissimo perché viene stretchato", ma permette già "una specie di blockout più o meno preciso". "Chiaro questo discorso?" Nessuna domanda. Il Blueprint costruisce la spline "sulla base del numero di punti" e la si rialza perché i pezzi siano tutti allo stesso livello.

1.16.22 Il Blueprint per lunghezza

La differenza, vista. "Cosa succede se andiamo invece a strutturare questa cosa per lunghezza, per segmenti?" In teoria si ha "un po' più di controllo". Il docente mette le due versioni una accanto all'altra con il muro di prima (asse Z), "soltanto dritte per dritte avanti", senza curve. Nella versione **by point** ogni nuovo punto genera una nuova mesh, ma "questa mesh può anche essere stretchata": allungando, si allunga. Nella versione **by length** "la mesh sarà sempre proporzionata": tirando la spline, "anche se abbiamo soltanto due punti, il muro rimane replicato in maniera corretta". Il muro "si spacca" nelle curve, "perché non è pensato per fare le curve in questo modo".

Attenzione ai poligoni. "State attenti a come utilizzate questa cosa, perché state andando a generare un sacco di mesh": "state aggiungendo un sacco di poligoni".

Il grafo. "La struttura finale è la stessa dello spline by point": For Loop, Add Spline Mesh Component, Set Start and End, Attach Component to Component. Cambia come si calcolano l'indice e i punti (figura 1.16.10):

1. una variabile **float** con la lunghezza su cui replicare l'oggetto: "ogni tot di quella lunghezza, replica l'oggetto". Il docente l'aveva chiamata *SplineLength*, e durante la spiegazione la rinomina **MeshLength**, "che ha più senso". Il suo valore deve essere uguale al lato della mesh che si "stretcha". Era a 186, e il muro risultava "più piccolo di come dovrebbe essere"; la mesh del muro ha una *Approx Size* di **71** $\times$ **250** $\times$ **250**, "quindi, essendo un quadrato, a noi serve 250". Con 250 il muro torna proporzionato. (Il docente toglie anche tre nodi rimasti dalla versione precedente, "che non ci servono più");

2. **Get Spline Length** dalla spline: la lunghezza della linea, x. La si **divide** per MeshLength: "in quante componenti verrebbe suddivisa la spline per mesh". "Sappiamo che la nostra mesh è lunga 10, la nostra spline è lunga 100: ci saranno 10 pezzi." Il risultato va nel *Last Index* del loop, passando per un **Truncate**, "messo come nodo automatico" per convertire da float a integer;

3. al posto di Get Location and Tangent at Spline Point, due coppie di nodi: **Get Location at Distance Along Spline** e **Get Tangent at Distance Along Spline** ("a noi serve la location o la tangente a un punto di distanza lungo la spline"). La **distanza** di partenza è l'indice moltiplicato per MeshLength; quella di arrivo è l'indice **più 1**, moltiplicato per MeshLength;

4. un **Clamp** sulla tangente, con massimo la lunghezza della mesh. La tangente "cresce sulla base della distanza dal punto successivo" e "ruota la struttura": senza clamp, "per questo calcolo cresce all'infinito";

5. tutto si ricollega a Set Start and End: *Start Position*, *Start Tangent*, *End Position*, *End Tangent*. "Questo ci permette di aggiornare e modificare la mesh sulla base della distanza, quindi sulla base di quanto è lunga la spline."

World e Local. "Un'altra cosa": come è messo il gizmo. Tirando un punto "in avanti" se ne crea un altro, ma magari si vorrebbe continuare nella direzione della spline e non si può, "perché siamo allineati col mondo". Passando all'allineamento **locale**, al punto, "posso tranquillamente prendere e tirare dritto". "Potete alternare fra utilizzare l'allineamento al mondo e l'allineamento all'oggetto."

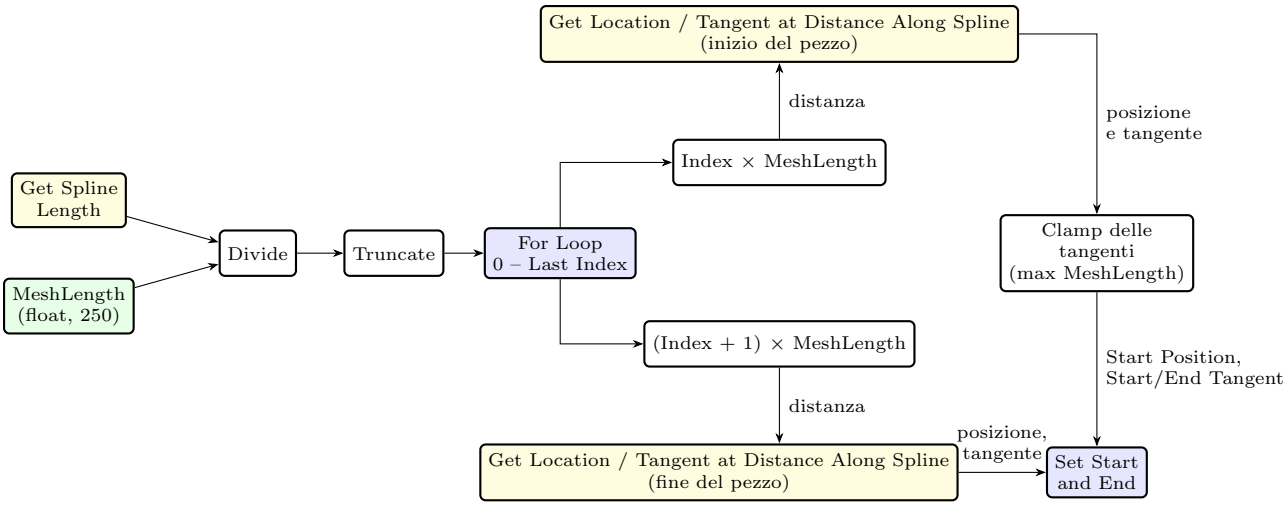


Figure 1.16.10: La parte di calcolo del Blueprint *Spline by Length*. Il numero di ripetizioni è la lunghezza della spline divisa per quella della mesh; ogni pezzo va dalla distanza $i \cdot L$ alla distanza $(i+1) \cdot L$ lungo la spline, con le tangenti (di inizio e di fine) limitate dal Clamp; lo schema semplifica i fili. Add Spline Mesh Component e Attach Component to Component restano come nella figura 1.16.9. Ricostruzione dai nodi nominati a lezione.

Che cosa ci si costruisce. "Un'infinità di cose": qualcosa che va in salita e poi in discesa, una **duna** ("luna" nella trascrizione), delle curve; una **strada**, usando "non un muro ma un piano" come mesh. Torna molto utile per il blockout e, in generale, "come asset per costruire tutta una serie di cose". Eliminando dei punti, la spline si aggiorna in automatico.

Approfondimento: la versione di Romero e Sewell. Romero e Sewell, nel capitolo 19 ("Procedural Generation"), costruiscono un Blueprint quasi identico, *BP_SplinePlacement*, con due differenze istruttive. La prima: non deformano mesh, ma aggiungono istanze di un *Instanced Static Mesh Component* lungo la spline, con *Get Location at Distance Along Spline* e *Get Rotation at Distance Along Spline* (coordinate *Local*); il numero di istanze è calcolato in una macro, *CalculateNumberOfInstances*, dividendo la lunghezza della spline per la variabile *SpaceBetweenInstances* e arrotondando per difetto con *Floor* (per numeri positivi è lo stesso risultato del Truncate del docente). Il libro avverte che una divisione per zero dà errore e suggerisce *Safe Divide*: vale anche per MeshLength, se qualcuno la lascia a 0. La seconda: nella sezione successiva presentano lo *Spline Mesh Component*, che "deforma una Static Mesh lungo una spline di due punti" con *Set Start and End*, e osservano che si potrebbe modificare l'esempio per creare "tubi curvi nel livello", ma che "il calcolo delle tangenti per le istanze della spline richiederebbe concetti matematici che vanno oltre lo scopo del libro". È proprio il passo che il docente fa con il Clamp. Due precisazioni tecniche, dal funzionamento generale dell'engine. Nella versione per punto, il "meno 2" è corretto per una ragione leggermente diversa da quella detta: con n punti ci sono $n-1$ segmenti, e poiché il For Loop include l'ultimo indice e ogni iterazione usa anche il punto $i+1$, l'ultimo indice valido è $n-2$. Nella versione per lunghezza, invece, il risultato del Truncate andrebbe a sua volta diminuito di 1 (o il loop fermato a $\lfloor \text{lunghezza}/L \rfloor - 1$): così com'è, l'ultima iterazione chiede una distanza oltre la fine della spline, che i nodi *at Distance* riportano all'ultimo punto, generando un pezzo schiacciato o di troppo. Infine, la tangente "che cresce" non cresce davvero all'infinito: la tangente di una spline in un punto è proporzionale alla distanza dal punto vicino, e su segmenti lunghi diventa molto più lunga della mesh, che si torce; il Clamp (in pratica un *Clamp Vector Size*) la riporta a una scala coerente con il pezzo.

1.16.23 Le domande: ottimizzazione e amico grafico

Quanto pesa? "Quanto impatta sull'ottimizzazione del gioco fare questa cosa, piuttosto che usare ogni volta la mesh?" "È pesantino, ma dipende da quello che state facendo." Lo si usa perché la mesh da sola "non si curva": la spline la si modella sul momento. Ma "se dovete fare un muro dritto, tanto vale che mettete più volte la mesh", si fa molto prima (e una mesh si può comunque allungare). La differenza sta nelle chiamate: con il muro dritto "abbiamo una singola mesh che viene spawnata una singola volta", o tre o quattro mesh piazzate; con la spline "un grosso numero di mesh che vengono spawnate tutte in un unico actor", un oggetto "molto denso di poligoni". Il consiglio: usarla "soltanto in momenti speciali", e in modo molto comodo per la **prototipazione**, soprattutto con un cubo o un oggetto "che non ha tanti dettagli".

Ciò che pesa davvero: i ricalcoli e i cavi. "Una volta che è piazzato, è pesante, ma fino a un certo punto." Quello che pesa tanto è "il fatto di dover calcolare le modifiche". L'esperienza del docente: le prime volte l'aveva usata per i **cavi**, "molto comodi per guidare il giocatore in una direzione". Si immagini una **pedana** collegata a una **luce** che si accende. Se la luce è davanti, lo si vede subito; se si accende nell'altra stanza, il giocatore non lo sa. "Ma se ho un cavo che da qui è collegato e fa tutto il giro e arriva fino all'altra stanza, io posso seguire il cavo per capire logicamente dove si accenderà la luce." Il cavo "non serve a niente a livello funzionale": serve a dare **guidance** al giocatore. Ma il docente si era "fatto un po' prendere la mano", e al momento di calcolare le luci, con tantissimi punti sulla spline, era diventato problematico. Cinque punti, come nella prova, "non pesano granché"; "se voi ne fate trenta, diventa un problema": "difficile e faticoso per lui andare a calcolare". Con oggetti pensati per la spline, come il muro di spine, "mi è convenuto farlo in quel modo piuttosto che farlo come oggetto fisico": "un piccolo sacrificio di ottimizzazione per una comodità". "Scegliete voi come, quando e dove usarlo."

L'amico grafico. Arriva un altro messaggio, da uno studente che si presenta come "quel famoso amico grafico": conviene fare i rovi sempre con una mesh diversa? "Dipende molto dalle esigenze." Se servono rovi da mettere in giro ogni tanto, anche come foliage, "come cosa principalmente estetica", ci si fanno tre o quattro tipi di rovi e li si piazza: "a te converrà sempre fare le mesh". Le spline "sono più *tips and tricks* da designer che cose da grafico".

Il torrente. Un motivo per cui un grafico potrebbe usare una spline: concentrarsi sulla creazione di **ambienti**, livelli "esteticamente belli". "In questo livello mi servirebbe un fiume, un torrente." Serve un piano e un **materiale d'acqua**; se il torrente deve cambiare intensità più volte, il modo più comodo è una spline. Il docente lo mostra con il muro: un punto in alto, uno un po' più in basso, "immagina che sopra questo muro ci sia un piano d'acqua". Ci si può creare in Unreal un materiale "che si calcola la **pendenza** in tempo reale e cambia la texture" di conseguenza: dove scende l'acqua scorre rapida, dove è in piano scorre placida. "Ti sei creato una struttura modulare" che si riusa per tante cose. Finché si lavora come grafico in Blender, Maya o ZBrush, "fuori dallo sviluppo in engine", si farà sempre tutta la mesh; "questo tipo di modularizzazione diventa utile nel momento in cui si passa in engine": "mi posso fare una regola che funziona un po' per tutto, e la uso dove mi serve, piuttosto che dover andare a correggere tutto quanto manualmente". ("Scusatemi: se mi fate una domanda divento prolisso, però voglio essere chiaro.")

La pista. La pista, al grafico, conviene farla in Blender, "una pista precisa in cui sai già quale sarà il percorso della macchina". Ma poi in engine il designer scopre che una curva non funziona: "ti schianti sempre, non riesci mai ad andare abbastanza veloce"; la si vorrebbe fare in **derapata**, ma è troppo stretta. Se la pista è un pezzo unico, bisogna ritrovare il file, modificarlo, riesportarlo e reimportarlo. Se invece è fatta di *tiling*, di curve e rettilinei specifici, la si piazza e la si modifica in engine con una spline, senza tutto quel processo. "È più una roba da designer, da ottimizzatore, che da grafico." C'è anche un rovescio: con un oggetto fatto così il grafico "inizia ad avere un po' meno potere su come uscirà fuori", perché sta creando uno **strumento** che altri useranno "per creare quello che gli va". Nessun'altra domanda. Le spline servono "per i tubi, per un percorso, per dei muri", e anche nel landscape.

Approfondimento: guidare con le linee, costruire con i moduli. Il cavo che porta alla luce dell'altra stanza è un esempio da manuale di quello che Totten chiama comunicazione visiva. Nel capitolo 3 riprende da Kevin Lynch i *paths*, le linee lungo cui le persone si muovono e che organizzano la mappa mentale di uno spazio; nel capitolo 4 osserva che nei giochi in prima persona si attira lo sguardo del giocatore con elementi visivi come "linee, colori in contrasto o illuminazione", e nel capitolo 7, a proposito della cinematografia, ripete che è fondamentale portare l'attenzione del giocatore sugli elementi che il designer vuole che veda. Un cavo che collega causa ed effetto insegna anche una regola del mondo ("le pedane alimentano qualcosa"), che il giocatore riutilizzerà. Quanto ai moduli: nel capitolo 2 Totten descrive il *modular design* dell'architettura (componenti prefabbricati come finestre, porte, impianti) e il suo equivalente nei livelli, un metodo di costruzione "tipo LEGO" invece di costruire su misura ogni ambiente; l'esempio è il gioco mobile *SWARM!*, i cui livelli non organici erano fatti di tessere modulari elencate in una tabella dei pezzi. La pista "a tessere più spline" del docente è la stessa idea applicata in engine. Infine, il peso: ogni Spline Mesh Component è un componente a sé, con il proprio costo di rendering, e la deformazione si ricalcola nel Construction Script a ogni modifica in editor; con molti punti, o con luci da ricostruire, il costo si sente. Per elementi ripetuti senza deformazione, l'alternativa leggera è quella di Romero e Sewell, le *Instanced Static Mesh* (funzionamento generale dell'engine).

1.16.24 Le decal: che cosa sono

Un actor estetico. "Un'ultima cosa che andiamo a vedere, e torniamo più sulla parte estetica, sono le **decal**." "È un tipo di actor estetico" (il docente all'inizio non lo trova, poi sì): lo si applica sulle cose, e una texture appare sulle superfici che ci entrano in contatto. Gli usi:

- dei **graffiti** in una scena;
- delle **crepe** sui muri;
- delle **impronte** sulla sabbia ("magari invece di essere così, ce l'abbiamo così");
- le **linee a terra**, "quelle di quando ci sono gli omicidi";
- chiamate da un **Blueprint**, per dipingere **fori di proiettile** sui muri: se il *line trace* non colpisce un nemico ma un muro, genera una decal a forma di foro;
- gli **schizzi di sangue** su un muro dopo che un nemico è stato colpito.

"Anche qui, se ci sono domande, in qualsiasi momento fermatevi."

Dove si trova e i parametri. Il **Decal Actor** è nel Place Actors, sotto **Visual Effects**, "giù in fondo". Nei Details (tabella 1.16.3):

- il **Decal Material**;
- un **ordine** (*Sort Order*), perché si possono mettere più decal una sopra l'altra e bisogna decidere quale sta sopra;
- diversi parametri per decidere "se e quando la decal dovrà andare in **fade**": "posiziono qualcosa che piano piano scompare, tipo una macchia di sangue sul muro che piano piano viene lavata";
- la **dimensione** (*Decal Size*), che si può cambiare anche con la scala dell'actor;
- il **Decal Color**, che "dovrebbe permetterci di modificare" il colore, ma qui non fa nulla: "si basa su un nodo presente all'interno del materiale";
- le solite opzioni di visibilità, come *Visible* e *Actor Hidden in Game*.

"Faremo delle decal molto semplici, giusto per vedere un po' come funzionano."

1.16.25 L'area di proiezione e Receives Decals

Il box. "Potreste aver notato un piccolo problema della decal": l'area del suo box "indica tutto ciò su cui questa decal verrà applicata". La decal va proiettata in una direzione, "verso il muro"; il docente vuole applicarla "su questo muro, solo sul muro". Ma, "diciamo che sono una persona disattenta", il box tocca anche le pareti vicine: "così è presente su tutti e tre i muri". "La decal va a proiettare il proprio materiale su tutte le facce che ci sono all'interno della sua area"; se fosse più larga, finirebbe anche sul pavimento (figura 1.16.11). Bisogna stare attenti a dove compenetra. (Nel mostrarlo il docente imposta un parametro a "0:50" e lo descrive come "il parametro per determinare quanto fada a distanza dal centro della decal": il nome non è ricostruibile dalla trascrizione.)

Receives Decals. Il modo per evitarlo: "in **Rendering** ogni mesh ha **Receives Decals**". Si decide così quali oggetti ricevono le decal e quali no: togliendo la spunta a un muro, lì la decal non viene stampata; rimettendola, ritorna.

Il robottino. "A che problema può portare, secondo me? Vediamo se qualcuno ci arriva." Nessuno risponde, e il docente lo mostra: passando dentro il box, anche il **personaggio**, "il nostro robottino", si ritrova la decal addosso. "Scegliete voi a vostra discrezione se volete che il robottino diventi così oppure no. È una cosa tremenda, orribile." Per evitarlo, come per le mesh, si apre il **character** e nelle impostazioni di Rendering si toglie **Receives Decals**. Secondo il docente la cosa "si può modificare anche dai materiali" degli oggetti su cui le decal vengono applicate (la lezione non lo mostra). E un box "un po' più piccolo" limita già di per sé i problemi.

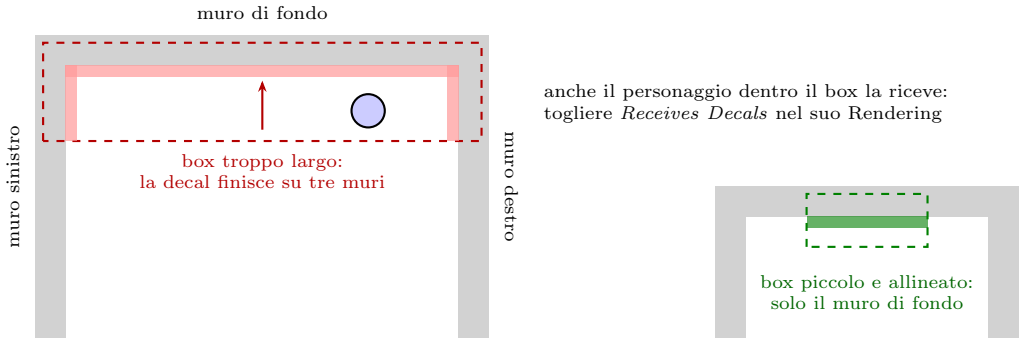


Figure 1.16.11: L'area di proiezione di una decal, vista dall'alto. Tutto ciò che sta dentro il box riceve il materiale, comprese le pareti laterali e il personaggio; un box più piccolo e la casella Receives Decals risolvono il problema. Ricostruzione schematica.

1.16.26 Il materiale di una decal

Un materiale normale non entra. Nella prova c'è "questo libro graffito" (forse un graffito già pronto), ma una decal "non sarà sempre un muro di mattoni": dipende dal materiale. Per creare un materiale da decal serve qualcosa di specifico: prendendo un materiale normale e mettendolo nel Decal Material, "questa cosa non è compatibile". Aprendo il materiale si capisce perché: nel **Material Domain** è impostata una superficie (*Surface*), "ma noi non stiamo creando una superficie, stiamo creando una decal" ("una decalculina").

Deferred Decal e Translucent. Si imposta il dominio a **Deferred Decal**: "gli attributi del materiale descriveranno una deferred decal e saranno mappati sul decal buffer" (il tooltip). Subito il materiale dà un **errore**: con Deferred Decal il **Blend Mode** può essere soltanto **Translucent**, **Alpha Composite** o **Modulate**; e non ci sono più pin "oltre al World Position Offset". Nel Blend Mode si mette uno di quelli: "quello che utilizzeremo sempre sarà il Translucent". L'errore sparisce e tornano i pin.

La texture e l'opacità. Si può collegare un colore, o una **texture**. Il docente prova un paio di texture ("stavolta non gli piace granché"), poi una che funziona: "eccoci qua! Abbiamo il nostro robottino". È una decal a forma di robot, messa "al contrario" e poi raddrizzata; si sistemano le proporzioni (un valore a 1) e la **size**. "Qui non abbiamo fatto niente, abbiamo semplicemente collegato la texture." Domanda agli studenti: "perché abbiamo collegato l'**alfa** all'**opacity**? Cos'è cambiato?" La risposta: l'opacity serve a determinare "dove sono i punti di trasparenza". Un graffito "non sarà mica tutto il quadrato della texture": ha dei punti trasparenti, e l'alfa li indica (figura 1.16.12).

Metallic, Specular, Roughness. Modificando **Metallic**, **Specular** e **Roughness**, il robottino "percepisce la luce" in modo diverso dal resto della parete. Molto metallico, moderatamente speculare e poco rough: "il robottino è diventato uno **specchio** a forma di robot", che riflette tutto ciò che c'è dall'altro lato; conserva un accenno del blu che aveva, ma i parametri hanno reso la trasparenza "molto più forte rispetto al colore". Con la roughness a 1 si ha invece un robottino "più compatto, molto ruvido", che non riflette la luce. Volendo, si può **moltiplicare** il colore per renderlo diverso o più brillante. "Ci possiamo fare un po' quello che vogliamo." Poi si torna al robottino vecchio, "perché non mi dispiace, era simpatico".

La Normal. Aggiunta in coda alla lezione: se si collega una **normal**, la decal "sovrascrive la normal che c'è sull'oggetto" su cui è posizionata. "Se prima era piatto, adesso il robottino diventa un robottino di roccia", che riflette la luce in quel modo perché lo si è reso anche molto metallico. Il docente prova anche a ridurne l'effetto con dei valori ("0,0", "0,444", "0,333"), che la trascrizione non permette di ricostruire.

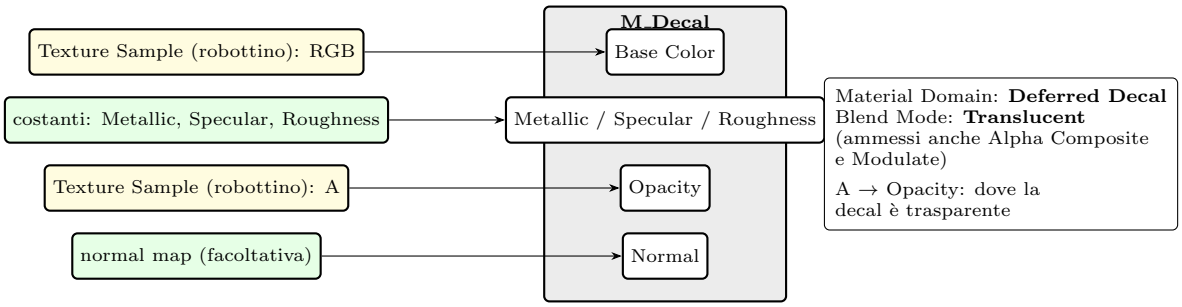


Figure 1.16.12: Il materiale di decal costruito a lezione: dominio Deferred Decal, Blend Mode Translucent, colore della texture nel Base Color e alfa nell'Opacity; Metallic, Specular, Roughness e Normal cambiano come la decal riceve la luce. Ricostruzione schematica.

1.16.27 I nodi delle decal e il fade

I nodi dedicati. "Queste decal avranno dei nodi per loro": cercando *decal* nel materiale compaiono voci come *Decals*, *DBuffer Texture*, *Static Mesh* e funzioni dedicate. Il **DBuffer** "è una roba un po' più avanzata che non vorrei toccare oggi, onestamente": "se vi interessa, la approfondite per conto vostro". I nodi che il docente usa:

- **Decal Color**: "è l'altro nodo che vi dicevo", quello che permette di cambiare il colore della decal dai Details dell'actor. Va collegato da qualche parte nel materiale. Serve anche per **passare informazioni**, per esempio fra le decal e il **Niagara**: colpisco un nemico con la spada, genero un effetto Niagara, uno spruzzo di sangue fatto di particelle; se una particella colpisce un oggetto, su quell'oggetto genero una decal, e voglio "mantenere lo stesso esatto colore del sangue" fra la decal e il VFX;
- **Decal Lifetime Opacity**: permette di gestire "l'opacità *over time*". Dopo qualche tentativo ("aspetta, se non sbaglio era così"), il docente imposta nell'actor una durata ("10") e la decal, "piano piano, svanisce": "abbiamo settato praticamente un counter". Per il robottino serve a poco, ma con questo nodo si gestisce "quanto cambia over time l'opacità dell'oggetto".

Una macchia che si allarga. "Invece di fare queste cose stupide", un'idea più interessante: "vi ricordate il discorso del bianco e del nero, l'uno e lo zero?" Si prende una mappa in bianco e nero, per esempio un **noise**, la si usa come alfa, le si dà un colore ("tipo un sangue") e un fade gestito nel tempo: "in questo modo possiamo dare alla decal un colore, un posto in cui viene applicata e un fade". Con un **parametro** si potrebbe regolare la velocità, e far crescere o ridurre la macchia in base al *lifetime*, "più o meno grande, o più o meno dettagliata". Qui la prova non riesce ("non mi ricordo benissimo come si fa", "la chiudiamo con un fail"), e la trascrizione di questo passaggio è in buona parte illeggibile. Il principio, però, resta: "c'è un modo per cui io posso impostare una texture che funga da alfa e, con un parametro, modificare la grandezza dell'opacità sulla base del lifetime". (Il docente si accorge poi di aver salvato il materiale nel posto sbagliato: "ho sperato fosse un mio fail per questa cosa, era per altro".)

Parametro o nodo	**Che cosa fa**
Decal Material	il materiale proiettato; deve avere dominio Deferred Decal
Sort Order	l'ordine fra decal sovrapposte
Fade (Fade Start Delay, Fade Duration, Fade Screen Size)	scomparsa nel tempo o con la distanza (nomi dall'engine; a lezione "i parametri del fade")
Decal Size, scala	le dimensioni del box di proiezione
Decal Color	colore dall'actor; funziona solo con il nodo Decal Color nel materiale
Visible, Actor Hidden in Game	visibilità in editor e in gioco
Receives Decals (Rendering delle mesh)	se un oggetto riceve le decal; da togliere al personaggio
Material Domain: Deferred Decal	trasforma il materiale in materiale da decal
Blend Mode: Translucent	obbligatorio fra Translucent, Alpha Composite, Modulate
Alfa della texture → Opacity	le parti trasparenti della decal
Metallic, Specular, Roughness, Normal	come la decal riceve la luce; la normal sovrascrive quella della superficie
Decal Lifetime Opacity	opacità che cala durante il fade
DBuffer	tecnica avanzata, non trattata

Table 1.16.3: I parametri del Decal Actor e del materiale da decal visti a lezione.

Approfondimento: decal da Blueprint e narrazione ambientale. I fori di proiettile del docente si fanno con i nodi *Spawn Decal at Location* o *Spawn Decal Attached*: si passa il materiale, la dimensione, la posizione e una rotazione ricavata dalla normale della superficie colpita, e una *Life Span* dopo cui la decal sparisce. Posizione e normale arrivano dalla *Hit Result* di un line trace: Romero e Sewell, nel capitolo 14 ("Math and Trace Nodes"), elencano fra le sue variabili proprio l'*Impact Point* e l'*Impact Normal*, "il vettore normale perpendicolare alla superficie colpita". Le decal *deferred* non si proiettano sulle superfici traslucide, e le *DBuffer decal* (da attivare nei Project Settings) sono la variante che si applica prima dell'illuminazione e funziona meglio con le luci statiche; il colore e la durata si possono passare anche a runtime con un *Dynamic Material Instance* (funzionamento generale dell'engine). Sul piano del design, Totten, nel capitolo 7 ("Storytelling in Gamespace"), descrive gli *embedded narrative spaces*: spazi che raccontano lasciando "tracce dell'uso da parte di personaggi o di eventi accaduti in precedenza". In *Portal* il personaggio del Rat Man lascia contenitori di cibo e graffiti fuori dalle camere di test; in *Left 4 Dead* i rifugi sono coperti dalle scritte dei sopravvissuti passati prima. Graffiti, crepe, impronte, sangue e le linee a terra "degli omicidi" sono esattamente queste tracce: le decal sono lo strumento più economico per scriverle sui muri.

1.16.28 Il progetto estivo e i saluti

Delle linee guida? "Ci sono delle domande?" Il docente propone una "cosina" da fare "quando avete tempo libero, quando volete": servono delle linee guida, o ognuno preferisce sperimentare? In chat: "qualche linea guida aiuterebbe, almeno per me, giusto per essere sicuri di implementare tutto quello che abbiamo fatto nel corso". "Tutto quello che abbiamo fatto nel corso non vi so dire", ma ecco le indicazioni ("io

dico quello che sarebbe figo avere; poi voi fate quello che riuscite", a seconda del tempo e di quanto ci si "intoppa"):

- **un livello**, "più che un gioco": un **puzzle platform**;
- dentro, **gli oggetti fatti con Igor**: le piattaforme, i collezionabili, "magari le torrette come nemici", le porte, "eccetera"; "almeno un oggetto" di ciascuno, e "un unico livello", non diversi livelli ognuno con una meccanica;
- un **menu**, visto che si è visto "un minimo la UI", e il **packaging** che parte dal **main menu**; dal menu si va in **due livelli**: il livello giocabile e la **gym**, quella vista "qualche lezione fa con me", in cui si testa il character e si annota "quanto salta, quanto corre, la velocità di movimento", magari con degli esperimenti di salti;
- il livello vero e proprio può essere in **blockout**, o con "un minimo di customizzazione": l'estetica prende tanto tempo, quindi anche solo un blockout con uno sfondo e un landscape un po' più curato va bene.

Gli asset a disposizione. Chi vuole curare l'estetica ha, nel progetto, la parte **StackOBot** ("Stack-Obot"): **Foliage**, con le mesh viste nella lezione del foliage; **Landscape**, con il landscape abbozzato; **Modular**, **Props** e **Rocks**, con oggetti da piazzare. Con i pezzi modulari si possono costruire "le zone con una muratura che effettivamente abbia un senso". Le **spline** servono per fare in fretta "zone morbide, bombate, non spigolose", con i tubi, i muri, le travi, o per fare percorsi con oggetti come le ventole del pacchetto. "L'estetico è un **plus**": luci, messa in scena e post process sono stati "piccoli accenni". Il consiglio: riguardare le lezioni perse, usare "quello che attira la vostra attenzione" e non per forza tutto; ma gli oggetti fatti con Igor vanno messi, "perché sono componenti di gameplay, che quindi costituiscono il livello".

Prima di condividere. Quando si è soddisfatti: "buildatevi le **luci**, fate il **packaging**, ve lo caricate da qualche parte e poi ci mandate il link sul **Discord**", così che i docenti e gli altri ragazzi possano provarlo. E "provatelo, il packaging": a volte impacchettando cambia qualcosa, "non come funziona il gioco, però ci sono magari delle **collisioni** che prima funzionavano bene e adesso non funzionano granché". Quindi si ritesta il gioco dopo averlo impacchettato. "Non mi aspetto un gioco di un'ora": "un livello di 5 minuti va benissimo", purché "molto carino, molto ispirato, molto simpatico" e che metta in mostra ciò che si è fatto. Per qualsiasi aiuto, Discord.

Una scadenza? "Posso chiedere entro quanto dobbiamo consegnare il progetto?" "Realisticamente parlando, io non ho il potere per dirti consegnamelo entro tot": c'è agosto, ognuno va in vacanza in periodi diversi, una scadenza che va bene a uno non va bene a Giulia. Il suggerimento: farlo "prima che cominci l'anno" in accademia ("con AIP", "di Ive" nella trascrizione; con ogni probabilità la sigla dell'accademia, non ricostruibile con certezza), perché poi bisogna concentrarsi sulle lezioni: "noi siamo un corso estivo". E soprattutto: "**non è un compito**. Io non vi sto chiedendo di fare questa cosa: vi sto dicendo, se volete farlo". Lo scopo è ricevere **feedback**: "dare e ricevere feedback è una delle parti fondamentali di chi lavora in questo ambiente".

I saluti. "Spero sia sicuro, in realtà, che da diverse persone di voi usciranno delle cose carine. Vediamo quante cose carine escono fuori." "Buon proseguimento d'estate e buon inizio di corsi, poi, quando cominceranno. Ciao a tutti. Buona serata."

Approfondimento: la gym, il livello e il ciclo di test. Il progetto proposto ripercorre in piccolo le fasi dell'inizio della lezione: la gym è la pre-produzione (le misure del personaggio, che determinano distanze e altezze del livello), il blockout con le meccaniche di Igor è la produzione, luci e post process sono la post-produzione, e il packaging con il nuovo test è la consegna. Totten, nel capitolo 2, insiste sul *whiteblocking* e sul *playtesting* iterativo: costruire con forme semplici, far giocare, correggere, e solo poi vestire il livello. Già nell'introduzione definisce il playtesting come la valutazione di un livello "facendolo giocare molte volte per verificarne il funzionamento esperienziale e tecnico": il controllo tecnico vale anche

per la build impacchettata, che non è identica alla sessione in editor. Fra le differenze tipiche, oltre alle collisioni citate dal docente: asset caricati per nome e lasciati fuori dal cook, la mappa di partenza sbagliata (la Game Default Map) e l'illuminazione non ricostruita, se il progetto usa luci statiche (funzionamento generale dell'engine). Romero e Sewell chiudono il capitolo 12 con un invito simile a quello del docente: aprire la cartella, lanciare l'eseguibile e giocare al proprio gioco "nella sua forma finale", perché "fare anche giochi semplici non è un'impresa facile".

Lente	**Parametri visti**	**Uso suggerito a lezione**
Bloom	Intensity; Threshold (-1 = nessuna soglia); Method Standard o Convolution (pesante)	luce afosa, estiva; riflessi; bloom solo dalle luci forti
Exposure	compensazione, minimo e massimo, velocità	leggibilità delle zone buie (hangar PvP); abbagliamento entrando e uscendo
Chromatic Aberration	Intensity; Start Offset (1 = nessun effetto)	momenti narrativi: flashbang, follia lovecraftiana; mai a lungo
Lens Flare	Intensity (legata al bloom), Tint, Bokeh Size, Threshold, Bokeh Shape (texture), Tints 0–7	sole forte all'uscita all'aperto
Depth of Field	distanza focale; distanza e intensità del blur	render, schermate, personaggio "miope"
Color Grading	Temperature, Tint; Global, Shadows, Midtones, Highlights con Saturation, Contrast, Gamma, Gain, Offset	stile e mood: toni freddi per un posto lugubre
Dirt Mask, Vignette	texture sulla lente; vignettatura negli Image Effects	effetto "statico" e mood più scuro (Mallory)

Table 1.16.4: Le lenti del post process mostrate a lezione.

Punti chiave del capitolo

- Il **post process** appartiene alla **post-produzione**, con musica, doppiaggio, effetti speciali e juiciness: si applica a una scena già costruita e **già illuminata**. Prima le luci, poi il post process.
- Il **Post Process Volume** cambia come la camera vede la scena solo quando la camera è **dentro**; con **Infinite Extent (Unbound)** vale ovunque. I volumi attivi si combinano: quello della scena di default va lasciato neutro.
- Una **Camera Actor** ha le stesse lenti, ma le applica solo a ciò che inquadra: utile per inquadrature e speed level design.
- I volumi servono per le **transizioni**: entrando in un edificio meno luce, meno bloom, un'altra tinta; più volumi in fila rendono il passaggio morbido (Rainbow Six Siege).
- **Bloom**: Intensity per la luminosità, **Threshold** per limitarlo alle luci più forti (-1 = nessuna soglia), Method **Standard** (Convolution è troppo pesante per un gioco).
- **Exposure**: rende visibili le zone in ombra; costa in estetica, ma la **leggibilità** viene prima della bellezza.
- **Chromatic Aberration**: separa i canali, soprattutto ai bordi; affatica la vista, quindi solo a picchi e nei **momenti narrativi**. Lo **Start Offset** pulisce il centro (1 = nessun effetto).
- **Lens Flare**: la scomposizione della luce, legata al bloom; Tint, Bokeh Size, Threshold, Bokeh Shape con una texture, Tints per ogni elemento.
- **Depth of Field**: distanza di messa a fuoco e sfocatura; in Unreal 5 funziona diversamente da Unreal 4.

Nodo o elemento	Versione	Ruolo
Spline Component	entrambe	la linea: punti, tangenti
For Loop	entrambe	una iterazione per pezzo
Get Number of Spline Points − 2	per punto	ultimo indice del loop
Get Spline Length / MeshLength, Truncate	per lunghezza	quanti pezzi ci stanno
MeshLength (float, 250)	per lunghezza	lato della mesh lungo la spline
Add Spline Mesh Component	entrambe	crea il pezzo; Static Mesh e Forward Axis
Forward Axis	entrambe	X per il tubo, Z per il muro di prova
Get Location and Tangent at Spline Point	per punto	inizio (indice) e fine (indice + 1)
Get Location / Tangent at Distance Along Spline	per lunghezza	inizio a $i \cdot L$, fine a $(i+1) \cdot L$
Clamp della tangente	per lunghezza	impedisce torsioni con tangenti troppo lunghe
Set Start and End	entrambe	deforma il pezzo fra i due estremi
Attach Component to Component	entrambe	attacca il pezzo al Default Scene Root
Alt + trascinamento, Add / Delete Spline Point	editor	aggiungere e togliere punti
Gizmo World / Local	editor	continuare dritto lungo la spline

Table 1.16.5: I nodi e i comandi dei due Blueprint delle spline.

- **Color Grading**: Temperature e Tint, poi Global, Shadows, Midtones e Highlights, ognuno con Saturation, Contrast, Gamma, Gain e Offset. Si impara solo sperimentando.
- Unreal è un engine pesante: per un gioco 2D in due toni si spara "a una zanzara con un cannone". Quel tipo di effetto si fa comunque con il post process.
- Il **packaging** trasforma il progetto in un **eseguibile**: Platforms → Windows → Package Project, dopo aver sistemato i Project Settings e, forse, disattivato il plugin di Fab.
- La **Game Default Map** è la prima mappa del gioco impacchettato (di solito il menu); l'**Editor Startup Map** quella con cui si apre l'editor.
- Configurazioni: **Debug**/**DebugGame** per i programmatori, **Development** (default) per testare, **Shipping** per distribuire. Shipping toglie gli strumenti, **non ottimizza** da sola.
- **Cook only maps** con la **List of maps to include** impacchetta solo ciò che serve; **Directories to never cook** esclude le cartelle inutili, contro il data mining e per risparmiare spazio.
- Il primo packaging è lento; nell'**Output Log** le scritte rosse sono gli errori da risolvere. Poi si ritesta il gioco impacchettato.
- Una **spline** è una linea di punti e tangenti; Alt per aggiungere punti, tasto destro per aggiungerli o toglierli.
- Con **Add Spline Mesh Component**, **Set Start and End** e **Attach Component to Component** si deformano mesh lungo la spline: **per punto** (un pezzo per segmento, stretchato) o **per lunghezza** (un pezzo ogni MeshLength, sempre proporzionato, con la tangente limitata da un Clamp). Il **Forward Axis** dipende dalla mesh.
- Le spline servono per blockout, piste, strade, muri curvi, tubi, cavi che **guidano** il giocatore, torrenti; ma generano molte mesh e pesano con molti punti. Sono strumenti da designer: il grafico, di norma, fa le mesh.
- Una **decal** proietta un materiale su tutto ciò che sta nel suo box: graffiti, crepe, impronte, fori di proiettile, sangue. **Receives Decals** decide chi la riceve (va tolto al personaggio).

Sintomo	**Causa e rimedio**
Effetti del post process confusi	due volumi attivi si sommano: tenere neutro quello della scena di default, una cosa alla volta
Il post process non si vede	la camera è fuori dal volume: entrarci, o Infinite Extent (Unbound)
Bloom ovunque	Threshold a -1: alzare la soglia
Zone troppo buie in una mappa PvP	Exposure: compensazione e limiti; la leggibilità prima dell'estetica
Aberrazione fastidiosa	usarla a picchi, nei momenti narrativi; Start Offset per pulire il centro
Texture della Bokeh Shape senza dettaglio	le texture dettagliate non rendono: usare forme semplici
Depth of Field diversa da Unreal 4	in Unreal 5 il metodo è cambiato (Cinematic DOF)
Il gioco impacchettato parte dalla mappa sbagliata	Game Default Map in Maps & Modes
L'editor si apre su un'altra mappa	Editor Startup Map
Problemi di packaging con il plugin di Fab	disattivarlo prima di impacchettare
Piattaforme con il segnale di attenzione	progetto non pensato per quei sistemi: Supported Platforms
Pacchetto troppo grande, asset scartati inclusi	Cook only maps, List of maps to include, Directories to never cook
Scritta rossa nell'Output Log	errore che blocca il packaging: leggerlo, risolverlo, ripartire
Il primo packaging non finisce	la prima volta è molto lenta (shader): lasciarlo lavorare
Il giocatore cade dal bordo della mappa	chiudere i confini prima di distribuire
Collisioni diverse dopo il packaging	ritestare il gioco impacchettato
Mesh sulla spline stretchata	usare la versione per lunghezza
Mesh orientata male sulla spline	cambiare il Forward Axis (X, Y o Z secondo la mesh)
Pezzi della spline troppo piccoli o grandi	MeshLength diverso dal lato della mesh (186 invece di 250)
Mesh che si torce nelle curve	Clamp della tangente; mesh segmentate e modulari
Non si riesce a tirare dritto un punto	gizmo allineato al mondo: passare al locale
Spline pesante	troppi punti o troppe mesh: usarla per blockout e casi speciali
Decal su più muri	box troppo grande: ridurlo, togliere Receives Decals
Decal sul personaggio	Receives Decals spento nel Rendering del character
Materiale non accettato dalla decal	Material Domain: Deferred Decal
Errore del Blend Mode nella decal	Translucent (o Alpha Composite, Modulate)
Decal quadrata e opaca	collegare l'alfa all'Opacity
Decal Color non ha effetto	manca il nodo Decal Color nel materiale

Table 1.16.6: I problemi incontrati o anticipati durante la lezione, con la causa e il rimedio.

Punto	**Che cosa dice la lezione, e la precisazione**
I post process “si sommano”	si mescolano per priorità e Blend Weight, proprietà per proprietà
Post Process Volume in Visual Effects	a UE15 era in Volumes: in Unreal 5 è in entrambe le categorie
Threshold come “intensità della luce”	confronta la luminosità dei pixel dopo l’esposizione, non l’intensità della sorgente
Exposure “visibilità delle ombre”	è l’esposizione automatica della scena; per le sole ombre c’è la Local Exposure
“Un antico turiano”	frase storpiata; con ogni probabilità un Grande Antico lovecraftiano
“Presize default all”	nome della texture di default della Bokeh Shape, non ricostruibile
Tints “0, 1, 2, 3, 4, 5, 7”	con ogni probabilità da 0 a 7, otto elementi
Depth of Field “diversa sulla 4”	vero: in Unreal 5 resta il Cinematic DOF; il seguito promesso non c’è nella trascrizione
Color Grading, Saturation e Contrast	un tratto della trascrizione è illeggibile
“Mallory”, “Mellori”	titolo non ricostruibile con certezza
Cartella “Windows No Editor”	nome di Unreal 4; in Unreal 5 la cartella è *Windows*
Debug con la console, Development “per designer”	anche Development ha la console; Debug e Test richiedono codice C++
“Nulla di game apps”	nome della cartella non ricostruibile
Plugin di Fab e packaging	detto “da quello che so”: forse risolto
Il packaging finito	interrotto dopo la pausa: il risultato non viene mostrato
“Meno 2” perché inizio e fine “non interessano”	il motivo vero: n punti danno $n - 1$ segmenti, e il loop usa i e $i + 1$
Truncate come ultimo indice	andrebbe diminuito di 1, o genera un pezzo in più
Tangente che “cresce all’infinito”	cresce con la distanza dal punto successivo
Forward Axis del muro: Y, poi Z	dipende da come è modellata la mesh
SplineLength rinominata MeshLength	è la lunghezza della mesh, non della spline
“Luna”	con ogni probabilità “duna”
Parametro della decal a “0:50”	nome non ricostruibile
Receives Decals “anche dai materiali”	non mostrato; il controllo certo è sul componente
“Libro graffito”	con ogni probabilità un graffito già presente nel progetto
La macchia che si allarga	prova non riuscita, passaggio in gran parte illeggibile
“ICE”	con ogni probabilità itch.io
“L’anno con AIP”, “di Ive”	sigla dell’accademia, non ricostruibile con certezza

Table 1.16.7: Le imprecisioni e i punti incerti della lezione, raccolti in un’unica tabella.

Strumento, nodo o impostazione	Che cosa fa
Post Process Volume	lenti della camera applicate dentro il volume
Infinite Extent (Unbound)	il volume vale in tutto il livello
Camera Actor, lenti della camera	post process solo per ciò che la camera inquadra
Bloom, Exposure, Chromatic Aberration, Lens Flare, Depth of Field, Color Grading	le lenti principali (tabella 1.16.4)
Dirt Mask, Vignette	sporco sulla lente, bordi scuri
Platforms, Package Project	avviare il packaging
Plugins	attivare e disattivare plugin (Fab)
Project Settings: Maps & Modes, Supported Platforms, Packaging	le impostazioni del packaging (tabella 1.16.2)
Build Configuration	Debug, DebugGame, Development, Test, Shipping (tabella 1.16.1)
Cook only maps, List of maps to include, Directories to never cook	che cosa finisce nel pacchetto
Show Output Log	seguire il packaging e trovarne gli errori
Spline Component, punti e tangenti	una linea modificabile in editor
Spline by Point, Spline by Length	due Blueprint per deformare o replicare mesh lungo una spline (tabella 1.16.5)
Spline Mesh Component, Set Start and End	mesh deformata fra due estremi
Decal Actor	proietta un materiale sulle superfici del suo box
Receives Decals	se una mesh riceve le decal
Material Domain Deferred Decal, Blend Mode Translucent	il materiale da decal
Decal Color, Decal Lifetime Opacity, DBuffer	i nodi delle decal (tabella 1.16.3)
StackOBot: Foliage, Landscape, Modular, Props, Rocks	gli asset per il progetto estivo

Table 1.16.8: Gli strumenti, i nodi e le impostazioni della sedicesima lezione.

- Il materiale da decal ha dominio **Deferred Decal** e Blend Mode **Translucent**; l'alfa nell'**Opacity**; Metallic, Specular, Roughness e Normal cambiano la luce. **Decal Color** e **Decal Lifetime Opacity** portano colore e fade dall'actor.
- Progetto estivo facoltativo: un livello puzzle platform con gli oggetti fatti con Igor, un menu, la gym, packaging, ritest e link su Discord. "Non è un compito": è un modo per ricevere feedback.

Part III

Analog Game Design

Chapter 2.1

Introduzione all'Analog Game Design: il corso, le figure dell'industria e un primo linguaggio comune

Questa è la **prima lezione** del corso di **Analog Game Design**, cioè di progettazione di giochi *analogici*: giochi da tavolo, giochi di carte, giochi di ruolo da tavolo, giochi di ruolo dal vivo (LARP), giochi *pervasivi* ed esperienze a evento. È un corso **breve**, di circa tre mesi ("da qui a dicembre"), separato dal corso di Game Design, e siamo alla sua **seconda edizione**. Come avvertono subito i docenti, è "una giornata di introduzione": si spiega come funziona il corso, chi sono i docenti, con quali metodi e con quale calendario si lavorerà, e ci si presenta a vicenda. "C'è poco gioco analogico", si scusano. Ma nella seconda parte la lezione comincia comunque a costruire un **linguaggio comune**: le figure professionali dell'industria del gioco analogico, la differenza tra *game design* e *game development*, la situazione del mercato italiano rispetto a quello estero, e un primo glossario di design (*game* e *play*, il framework MDA, le parole "vietate", il playtest e l'iterazione). Si chiude con lo strumento di settore per i prototipi digitali dei giochi da tavolo, **Tabletop Simulator**, e con una lunga sessione di domande.

La lezione si svolge **online**, su Zoom, ed è condotta a più voci dai tre docenti principali del corso: **Daniele**, narrative designer, che guida la parte organizzativa; **Ruggero**, game designer, che guida la parte sui fondamenti e le domande finali; **Beatrice**, progettista di giochi pervasivi, che interviene soprattutto nella prima metà. Gli studenti iscritti sono **nove**; uno è assente e un altro arriva a lezione iniziata. Ci sono una pausa di una decina di minuti (si riprende alle 18:15) e, dopo le 19:15, un'ultima mezz'ora di domande con il solo Ruggero.

Tre avvertenze di lettura. La prima: la trascrizione è automatica e storpia molti nomi propri, compresi quelli dei docenti e dell'accademia ("Dive", "IVE", "Accademia Live", "Ai"); dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante la pausa la trascrizione registra, ripetuta in loop per decine di righe, una frase senza senso che non ha nulla a che fare con la lezione, e che il capitolo ignora. La terza: in molti passaggi più docenti parlano uno dopo l'altro senza che la trascrizione segni il cambio di voce; il capitolo attribuisce le parole a chi le pronuncia quando il contesto lo rende chiaro (chi parla di sé, chi viene chiamato per nome, chi "passa la parola"), e resta generico altrimenti.

Quanto alla data, la lezione si colloca con ogni probabilità **nella prima metà di settembre del 2026**[1]: i riferimenti a "oggi", "quest'estate" e "l'anno scorso" vanno letti rispetto a quel momento.

Il percorso del capitolo segue quello della lezione:

1. l'avvio, i **docenti** e le **presentazioni** dei partecipanti (sezioni da 2.1.1 a 2.1.3);

[1] La data non è mai detta. La si deduce da alcuni indizi: il form iniziale va compilato "entro e non oltre il 15 settembre"; i gruppi verranno annunciati il 25 settembre; le date del calendario (sabato 3 ottobre, lunedì 12 ottobre, sabato 7 e 14 novembre, venerdì 4 dicembre) cadono nei giorni della settimana indicati soltanto nel 2026; un docente parla di un argomento "molto complesso nel 2026"; e fa ancora molto caldo ("le temperature sono ancora indecenti"). È una deduzione, non un'informazione esplicita.

2. i **materiali** e gli strumenti del corso: il form, la cartella condivisa, Discord (sezione 2.1.4);
3. lo **scopo** e il **metodo** del corso, le regole di comportamento e il carico di lavoro (sezioni 2.1.5 e 2.1.6);
4. il **project work** e il **calendario** (sezione 2.1.7);
5. la regola sull'**intelligenza artificiale** e la lunga discussione che ne segue (sezione 2.1.8);
6. le **figure professionali** dell'industria del gioco analogico, con un approfondimento su marketing e sales (sezioni 2.1.9 e 2.1.10);
7. la differenza tra **game design** e **game development** (sezione 2.1.11);
8. il **mercato**: Italia ed estero (sezione 2.1.12);
9. i **fondamenti di design**: perché serve una terminologia comune, *game* e *play*, il framework **MDA** (e il DPE), le **parole vietate**, chi crea, chi gioca e chi facilita, il **playtest** e l'**iterazione** (sezioni da 2.1.13 a 2.1.18);
10. **Tabletop Simulator** e il primo task (sezione 2.1.19);
11. le **domande finali**: il tema del project work, il bilanciamento, i ruoli nei team, la pubblicazione, il gioco di ruolo in Italia, le *house rule* di Monopoly, i giochi "comunicati male" (sezioni 2.1.20 e 2.1.21).

2.1.1 L'avvio della lezione

La lezione comincia con qualche minuto di attesa. "Siamo in nove in classe", e si aspetta che arrivino tutti; uno dei docenti scherza sul fatto che l'unica persona che arriverà "sempre in ritardo di venerdì" è Gianmarco. Mancano Giordano e Tiziana; Tiziana arriva, mentre la segreteria fa sapere poco dopo che Giordano non potrà esserci e vedrà la registrazione. Si aspetta anche Beatrice, la terza docente, che arriva a sua volta dopo pochi minuti.

Nell'attesa, Daniele anticipa il programma della giornata e qualche informazione pratica:

- oggi è una giornata di **introduzione**: come funziona il corso, chi sono i docenti, le presentazioni degli studenti, le **metodologie** e il **calendario**. Gli studenti avranno "un fracco di domande", e alla fine della lezione c'è "proprio uno slot apposta" per le domande;
- le lezioni sono **registrate**: di solito la registrazione viene consegnata dopo circa **12–24 ore** sul profilo personale della piattaforma dell'accademia, che va attivato con la segreteria (Tiziana, già studentessa, probabilmente ce l'ha già);
- i docenti "cuore" del corso sono tre (Daniele, Ruggero e Beatrice), "soprattutto Ruggero"; a loro si aggiungeranno tre docenti ospiti, per meno ore, probabilmente una lezione a testa, "tranne Marta, che avrà due lezioni quest'anno".

E una battuta sul caldo: "spero che voi siate carichi, anche se le temperature sono ancora indecenti". "Carichi moderatamente, per quello che potete".

Poi Daniele condivide lo schermo ("non solo le slide, ma anche un po' di altre cose") e apre ufficialmente il corso: "benvenuti e benvenute nella seconda edizione del corso di Analog Game Design. Questa è la prima lezione". Il percorso è di circa **tre mesi**, fino a dicembre, e durante il corso i docenti cercheranno di far provare agli studenti "cosa significa fare Analog Game Design" con un **project work**, un progetto di gruppo di cui si spiegheranno più avanti la struttura e le richieste.

2.1.2 I docenti

"Anzitutto partiamo da delle presentazioni doverose". Ognuno dei tre docenti principali si presenta con una slide che riporta i contatti e una selezione dei lavori fatti.

2.1.2.1 Daniele: narrativa, fiction e gioco di ruolo

Daniele[2] ha già conosciuto quasi tutti gli studenti, perché ha fatto loro il colloquio di ammissione. È **narrative designer**, o meglio *game narrative designer*; insegna nell'accademia che ospita il corso[3] ma lavora soprattutto come **brand advisor**. Nel corso si occuperà della parte di **gioco di ruolo**, **narrazione** e **fiction**.

I suoi contatti, "in quest'ordine", sono email, Telegram e Discord. Ci tiene a dare alcune indicazioni precise:

- se si vuole avere la certezza di una risposta, bisogna scrivergli su **Telegram**, perché lì ha tutti i suoi clienti;
- Discord lo guarda di meno; l'**email** è per le cose ufficiali;
- non bisogna contattarlo altrove: non su Facebook, non su LinkedIn, non su Instagram, "perché li vedo molto di meno".

La slide mostra anche alcuni giochi, attività e brand su cui ha lavorato negli anni, ma non si dilunga: "nel caso ne parleremo".

2.1.2.2 Ruggero: game design e ricerca e sviluppo

Ruggero (spesso chiamato "Ruger") ha fatto il **game designer** per varie aziende e oggi è game designer e si occupa di **ricerca e sviluppo** per un'azienda del settore[4]. Ha un background "misto": nel suo portfolio ci sono giochi da tavolo, giochi di ruolo, giochi analogici, LARP, videogiochi, giochi in realtà virtuale. Si occupa anche di **divulgazione** sul gioco e di analisi di **game studies** e game design. Più avanti, nelle domande finali, dirà di avere "50 e più giochi prodotti e progettati" e di insegnare da vari anni.

Lo si può contattare liberamente via Gmail e Telegram, e cerca di rispondere **entro 24 ore**, salvo imprevisti o eventi: "sentitevi in libertà di scrivermi se avete dubbi, senza problemi". Quando arriva a parlare dei propri giochi, la slide mostra "un mix di varie cose, dalle più famose alle meno famose", ma Ruggero si imbarazza a presentarli, tra le risate dei colleghi.

2.1.2.3 Beatrice: i giochi pervasivi

Beatrice ("Bea") si occupa di **giochi pervasivi** (*pervasive games*). "Se non li avete mai sentiti nominare, non preoccupatevi": questo corso è "uno dei pochi, forse l'unico in Italia" che li racconta esplicitamente. Li si vedrà nelle prossime lezioni, ma la definizione di base è questa: i giochi pervasivi **utilizzano gli spazi e gli oggetti della vita quotidiana e li trasformano in spazi e oggetti di gioco**.

Beatrice lavora come **freelance** e si occupa di diversi tipi di progettazione: lavora sia con le **aziende** sia con gli **eventi dal vivo**. La sua attività diretta si trova sotto l'etichetta **Minime Differenze**[5], mentre i lavori per le aziende sono "sparpagliati" sui siti delle varie aziende. Molte delle cose mostrate nella sua slide sono attività proposte al pubblico "con un biglietto, con un gioco in formato evento": un tema che si toccherà nel corso, per capire come **comunicare** i propri giochi in modo che siano comprensibili, accessibili per il pubblico e **vendibili**.

Ammette un "grosso limite" con i messaggi: è un po' lenta a rispondere, "ma rispondo sempre a tutto". La si può contattare per email (cognome punto nome) o su Telegram. È "molto curiosa di vedere cosa produrrete".

[2] La trascrizione riporta "Daniel Cusetto"; nel resto della lezione tutti lo chiamano "Daniele". Il cognome non è ricostruibile con certezza.

[3] La trascrizione storpia il nome dell'accademia in molti modi ("Dive", "IVE", "Accademia Live", "Ai", "AIVA"). Con ogni probabilità si tratta dell'**AIV**, l'Accademia Italiana Videogiochi, che ha sede a Roma: lo fanno pensare i corsi triennali di game design e narrative design citati dai docenti e il fatto che una studentessa racconti di essersi trasferita a Roma. È una ricostruzione, non un'informazione esplicita; nel seguito il capitolo parla semplicemente dell'"accademia".

[4] La trascrizione dice "per MS"; il nome dell'azienda non è ricostruibile con certezza. Più avanti Ruggero cita tra i giochi su cui ha lavorato il gioco da tavolo ufficiale di *Cyberpunk*, *Marvel United* e *Status*, un gioco sulla geopolitica.

[5] La trascrizione riporta il nome dell'etichetta come "Minime Differenze" e l'indirizzo del sito come "mininedifferenze.com"; la grafia esatta del sito non è verificabile dalla trascrizione.

2.1.2.4 I docenti ospiti

Daniele presenta poi i tre docenti che il corso avrà "solo per alcune lezioni", tutte verso **novembre**:

- **Diego Ricchiuti**, direttore del corso di Game Design dell'accademia, farà una lezione sull'**RNG** (*random number generation*, la casualità nei giochi): "vi parleremo di RNG quando sarà necessario. Comunque avrete una lezione con lui, sappiatelo";
- **Marta**[6] farà **due lezioni sul development**: quest'anno "abbiamo deciso di fare doppietta", perché l'anno scorso il suo intervento è stato "molto, molto utile";
- **Flavio**[7] farà la lezione di **game production**; è un editore, e di solito partecipa anche alla tavola rotonda con gli editori della giornata finale.

Le slide sono già online e gli studenti avranno accesso alla cartella condivisa: "non vi preoccupate di queste cose, ne parleremo a breve". Ma prima di parlare del corso, "vogliamo conoscere voi".

2.1.3 Le presentazioni dei partecipanti

Le domande per la presentazione sono semplici, da affrontare "a turno, senza fretta, con calma":

- nome e occupazione ("se volete dirla");
- perché si vogliono fare giochi analogici, e a che livello: che cosa ha portato lo studente al corso;
- che tipo di esperienze o conoscenze si hanno già nell'ambito, se si è già fatto qualcosa oppure no.

"Chi vuole partire senza nessun tipo di ritegno e di vergogne?". Per far parlare gli studenti guardandoli, Daniele interrompe la condivisione dello schermo e ricorda lui le domande quando serve. Le presentazioni sono interessanti anche per i commenti dei docenti, che anticipano diversi principi del corso.

2.1.3.1 Tiziana

Tiziana fa l'**infermiera**, ma nella vita ha fatto molti lavori: impiegata, insegnante e "tanti altri lavoretti". Gioca "da sempre": il gioco fa parte anche della sua vita familiare, perché suo fratello fa il programmatore di videogiochi. Ha già **organizzato** giochi: una caccia al tesoro per far conoscere il territorio, due scene con un delitto (giochi con omicidio da risolvere), giochi per intrattenere le persone. Ha riscoperto di recente i giochi da tavolo, arrivando a *Carcassonne* con famiglia e amici; poiché è difficile trovare persone con voglia di giocare nella propria cerchia ristretta, ha scoperto il mondo delle **associazioni** ludiche, "e lì è stato un'esplosione": prima in Veneto (è di Vicenza), poi a Roma, dove si è trasferita.

Ha in mente un progetto ambizioso: far conoscere una certa realtà con la modalità del **gioco di ruolo**, o comunque proporre un gioco suo.

Il commento: gli strumenti, non il gioco dei sogni. Ruggero coglie l'occasione per un'avvertenza che vale per tutti, visto che molti studenti arrivano con idee già pronte. Nel corso si lavorerà a un progetto, ma l'obiettivo dei docenti è dare **strumenti** da applicare anche *fuori* dal progetto, ai giochi che gli studenti vorranno fare un domani: "che non siano esercizi per il corso, ma per voi in primis". Quindi: "non rimaneteci male se non riuscirete a lavorare al vostro gioco dei sogni in questo corso". Anche perché "sarebbe impossibile, in due mesi, fare il gioco dei sogni". Il corso darà invece la possibilità di fare un'esperienza ludica insieme e di imparare a replicarla sulle proprie idee: "le idee non buttatele, anzi, il contrario: tenetele".

[6] La trascrizione riporta il cognome come "Cercasassi"; la grafia non è verificabile.

[7] La trascrizione riporta il cognome come "Mortarino" e, più avanti, "Mortalino", e il nome della sua casa editrice come "Grampibert"; nessuno dei due è ricostruibile con certezza.

2.1.3.2 Nicola

Nicola è uno **studente**. Sei mesi prima ha fatto un colloquio con Daniele perché stava valutando di passare all'accademia, e ha deciso, "nel dubbio", di cominciare da questo corso breve per capire come funziona la formazione; eventualmente si iscriverà al corso di game design vero e proprio. Gli interessa in particolare il lato dell'esperienza del giocatore, su cui è "molto critico" (qui usa la parola "divertimento", e Daniele lo ferma con un "ne parleremo": è una delle parole che il corso vieta, come si vedrà nella sezione 2.1.16).

Ha 21 anni ed è **giocatore di ruolo da circa otto**. Prima ancora di conoscere i giochi di ruolo si era creato un sistema tutto suo, "pessimo, disgustoso, terribile", un sistema a percentuale con il d100 "terrificante"; poi ha scoperto i giochi di ruolo da tavolo, i classici (*Dungeons & Dragons*, *Daggerheart*, *Vampire*[8]). Gioca anche a **card game** e ad altri giochi sociali, pur avendo sempre avuto difficoltà a trovare gruppi, "avendo vissuto in un buco dimenticato dal mondo". Come giocatore è "l'eterno narratore" del gruppo, quello che prepara la storia e si costruisce le regole. Come autore ha fatto soprattutto giochi su carta, "perché sono incapace a programmare, per adesso".

Daniele osserva che il tema del giocatore e del suo rapporto con il gioco è "un argomento importante", anzi "un vaso di Pandora enorme" (espressione che tornerà molte volte nella lezione), e che il corso tratterà anche che cos'è l'analog game design in generale e quanti tipi di prodotti esistono.

2.1.3.3 Rita

Rita[9] ha 30 anni e si collega da una regione del Sud, dove "ancora si muore di caldo". Ha studiato all'**Accademia di Belle Arti di Napoli**, **design della comunicazione visiva**: si occupa dell'aspetto più imprenditoriale della progettazione artistica. Non volendo progettare per un'impresa ("fare loghi, robe del genere"), vorrebbe applicare la propria creatività alla realizzazione di giochi analogici.

Non è una giocatrice di ruolo e non lo è mai stata: è una giocatrice **tradizionale**, cresciuta con Monopoly, la dama e giochi tradizionali cinesi. Ha già **idee, prototipi e progetti** realizzati, e vuole capire come renderli **prodotti per la vendita** e se questo è il campo adatto a lei; magari anche confrontarsi con i colleghi. La sua progettazione è orientata al **sociale** e all'**educativo**: "responsabilizzare un po' le nuove generazioni su come sta andando il mondo e su come potrebbe andare, visto che è tutto nelle nostre mani".

Il commento: siate curiosi. "Pane per i tuoi denti", le risponde Daniele, soprattutto per i LARP e i giochi pervasivi: "vi apriremo un po' la testa da questo punto di vista". E aggiunge un consiglio per tutti: "siate curiosi", perché imparare da altri ambiti dà moltissimo. Ognuno è lì per fare qualcosa di preciso, "però siate aperti anche ad altre esperienze".

2.1.3.4 Tancredi

Tancredi ha 21 anni e non ha ancora grande esperienza lavorativa: qualche "piccolo progettino" che non è andato come voleva. Gli interessa il mondo dei giochi, digitali e non, con un approccio **educativo** e **sociale**. È arrivato soprattutto dai videogiochi, a cui gioca da quando era piccolo; ai giochi da tavolo si è avvicinato solo nell'ultimo anno, grazie a una realtà ludica, perché in famiglia "non siamo molto da giochi". Ha realizzato qualche piccolo videogioco con vari engine: prima quelli senza un vero linguaggio di programmazione, "più a nodi", poi qualcosa di più complicato, e ora sta lavorando a un piccolo prototipo.

Il commento: niente di quello che vedrete è sprecato. "Non butterai via niente di analog game design", gli risponde Daniele, "perché il 99 per cento delle cose che vedremo si ritrovano quasi uno a uno nel mondo dei videogiochi". Tanto che nei corsi triennali dell'accademia (anche in quello di narrative design, dove insegna Ruggero) il primo anno si parte dai **giochi da tavolo e di ruolo**, proprio per spiegare meccaniche più complesse che servono poi per i videogiochi. Ruggero aggiunge che vale anche il contrario: alcuni dei **framework** di game design che si vedranno nelle lezioni tecniche vengono dal videogioco e sono stati riapplicati all'analogico. "Evitare la crossmedialità qui è impossibile". Anzi, tutte le conoscenze "mediatiche adiacenti" (musica, letteratura, cinema) saranno utili.

[8] La trascrizione riporta "DD", "Daggerart" e "Vampire": quasi certamente *Dungeons & Dragons*, *Daggerheart* e *Vampire: The Masquerade*.

[9] Daniele la chiama prima "Nino" (probabilmente un errore della trascrizione) e poi la ringrazia come Rita.

2.1.3.5 Lorenzo Vergata

I Lorenzo in classe sono due; il primo a presentarsi è **Lorenzo Vergata**, 26 anni, **dottorando al Politecnico di Milano**. Gioca ai videogiochi da quando aveva due anni e mezzo, ma ha giocato al suo primo gioco da tavolo "credo nove mesi fa". Gli fa piacere sentire che molti in classe sono interessati ai giochi **sociali** ed **educativi**, un settore che interessa anche a lui e che "può essere esplorato molto", anche con qualche aiuto dell'intelligenza artificiale per esperienze più ricercate.

Di esperienza nell'analogico dice di avere fatto "il minimo sindacale". Ma racconta che diciotto anni prima, da bambino, era "più game designer di adesso": prendeva un gioco e ne cambiava le regole, anche al biliardino ("no, ora introduciamo questa regola"). Gli piace sperimentare, creare regole e vedere che cosa accade in sé e negli altri. Soprattutto vorrebbe "una chiave di lettura per poter rimanipolare la realtà": tre settimane prima è stato a fare la **vendemmia in Francia** e ha visto tantissime dinamiche che potrebbero diventare un gioco da tavolo o un videogioco. Per esempio: finire contro un rovo pieno di semi che si attaccano ai vestiti lo rallentava nella raccolta dell'uva, mentre in un altro filare era più rapido. "Si può fare pure una dinamica", e vorrebbe gli strumenti per farne giochi.

Arriva subito la battuta: una "vendemmia gamificata" da proporre in Oltrepò Pavese ("io sono dell'Oltrepò Pavese, quindi devo per forza bere vino")[10]. Daniele conferma che quest'anno il corso avrà "tante cose sul sociale": "ci piace parecchio".

2.1.3.6 L'altro Lorenzo

Il secondo Lorenzo ("secondo, ma non meno importante") è uno **scrittore di racconti brevi** e sta scrivendo il suo primo romanzo. Gli è sempre piaciuto raccontare storie, e da più grande scriverle; gioca ai videogiochi ed è "un amante delle storie e della cultura pop". Negli ultimi anni è entrato nel mondo del **gioco di ruolo da tavolo** e vorrebbe unire le due passioni, le storie e il gioco di ruolo, magari facendone un lavoro; gli interessa anche lo *strategic writing*. Daniele gli ricorda che ci sarà una lezione proprio sulla **fiction**.

2.1.3.7 Angelo

Angelo si collega senza videocamera dal lavoro: fa il **birraio** (in un piccolo birrificio in Brianza, a Bernareggio, come racconterà dopo la pausa) e, con il caldo, "vi risparmio l'orrore". Per molto tempo era convinto che i giochi da tavolo fossero solo i classici (Monopoly, Risiko) o i party game; poi, quattro o cinque anni fa, ha "sbattuto la faccia" contro giochi davvero appassionanti, con una specie di narrazione dietro che li rendeva ancora più coinvolgenti, e non ha più smesso. È uno che "ha sempre un botto di idee, ma poi non mette in pratica assolutamente nulla": cercava un corso che gli desse le basi e soprattutto lo **motivasse** a provare a fare qualcosa.

Il commento: bisogna buttarsi. Daniele racconta che anche l'anno prima un suo caro amico si era iscritto proprio per questo motivo. Il corso è breve, ma cerca di portare gli studenti "da 0 a 100" e di far realizzare un **prodotto** da presentare alla fine. Il punto è che i giochi "bisogna farli, bisogna buttarsi". E, venendo dalla narrazione, Daniele lo paragona allo scrivere: si scrive e si riscrive, e allo stesso modo si fa **playtest**, si rifà, si aggiorna. "Non è che si fanno così".

2.1.3.8 Gianmarco (dopo le slide)

Gianmarco arriva a lezione iniziata e si presenta più tardi, alla fine della prima parte, dopo che i docenti hanno chiuso le slide organizzative e la discussione sull'intelligenza artificiale ("Gianmarco, ti giuro, ti facciamo presentare"). Lavora nel **team di innovazione** di un'azienda italiana. Vuole fare giochi perché è una sua passione, che lo gratifica, e in passato ci ha provato "in maniera scomposta". Cerca un **metodo** e un confronto con persone più esperte: "studiare solo dei manuali è molto sterile", "stare chiusi nel pertugio non aiuta", ed "essere autodidatta ha estremamente i suoi limiti". Come esperienza è un giocatore e ha letto "un po' di materiale, di blog, di manuali", "niente di rilevante".

Manca solo Giordano, che si presenterà alla lezione successiva.

[10] La trascrizione dice "nell'altro papavese" e non chiarisce chi pronunci la battuta.

Uno sguardo d'insieme. La classe è molto varia: un'infermiera, una designer della comunicazione visiva, un birraio, un dottorando, uno scrittore, un impiegato nell'innovazione, due studenti ventunenni. Alcuni vengono dal gioco di ruolo, altri dai giochi tradizionali, altri dai videogiochi; quasi tutti hanno scoperto i giochi da tavolo moderni da pochi anni o da pochi mesi. E almeno tre studenti dichiarano un interesse per i giochi **sociali** ed **educativi**. Come si vedrà nella sezione 2.1.5, i docenti considerano questa varietà una risorsa.

2.1.4 I materiali e gli strumenti del corso

Prima delle slide sul corso, Daniele sbriga un po' di "clean up casalingo".

Il form iniziale. C'è un **form** da compilare "entro e non oltre il 15 settembre": il QR code è nelle slide, e la segreteria lo ricorderà su Discord. Richiede circa 10–15 minuti; si può fare subito o in giornata, "e ve lo togliete di torno". È un form standardizzato per tutti i corsi ("non siamo io e Ruggero a farvi passare la tortura del test"), diviso in due parti:

- una parte di **conoscenza del candidato**: quanto tempo si ha a disposizione per il corso, come ci si organizza con il lavoro, che cosa piace giocare. È la parte che interessa ai docenti, perché li aiuta a organizzare le lezioni;
- una parte di **test logico**, che i docenti non hanno scritto ("io non sono responsabile del form", "no, no, nemmeno io").

La cartella condivisa. Tutti gli studenti vengono aggiunti alla cartella del corso su **Google Drive**, che si trova tra i file "condivisi con me". Gli studenti ne sono **gestori dei contenuti** e possono caricarci quello che vogliono. Dentro ci sono:

- le **attività in classe**;
- le **dispense**;
- i **materiali didattici**, con le slide (quelle della lezione sono già lì);
- le cartelle dei **project work** futuri;
- i **task**, che "saranno pochi, ma ci saranno";
- il **calendario studenti**, "importantissimo", con due schede: il calendario delle lezioni e quello del project work.

Discord. Il corso ha anche un **server Discord**, a cui tutti sono stati invitati; ci sono i docenti (Daniele, Ruggero, Diego e altri), che si possono taggare. Di solito è lì che la segreteria pubblica le informazioni, per esempio se una lezione "dovesse per caso essere cancellata". Nelle domande finali Ruggero aggiungerà che c'è un canale apposito per il corso, che gli studenti possono usare liberamente per organizzarsi tra loro.

Le reaction di Zoom. Durante le lezioni su Zoom si possono usare le **reaction** (il pollice in alto, il cuoricino, la mano alzata): un modo rapido per far capire ai docenti se si è capito, se si vuole intervenire. Nel frattempo arrivano i primi pollici.

2.1.5 Lo scopo e il metodo del corso

La prima serie di slide (che Daniele, per un lapsus, presenta come "cos'è il corso di pulire il pavimento di casa") spiega che cosa il corso vuole fare e come.

2.1.5.1 Diventare autori analogici

Lo **scopo del corso** è fornire le competenze per **diventare autori analogici** e stimolare la creatività e l'**autorialità** di ciascuno: "voi imparerete a diventare autori di giochi analogici". La differenza tra *game designer* e *game developer* verrà chiarita più avanti (sezione 2.1.11), ma la parola chiave qui è *autorialità*: "vogliamo voi, vogliamo le vostre idee". Magari non si potrà usare l'idea del gioco della propria vita, ma si potranno portare nelle lezioni le proprie idee e le proprie sensibilità.

Gli obiettivi elencati nella slide sono questi:

- dare **maggiore consapevolezza nel design e nello sviluppo**. In Italia, purtroppo, "si fa design a volte in maniera non consapevole". Consapevole vuol dire avere almeno un'introduzione alle basi del design e dello sviluppo, e fare le cose "con metodo, con raziocinio, con consapevolezza";
- fornire gli **strumenti essenziali** per un design consapevole;
- stimolare la **sperimentazione** e la **trasversalità mediatica**. Non si chiede di fare "il *Root* della situazione o il *Carcassonne* della situazione"[11], ma un project work **calibrato sulla durata del corso**, e però **sperimentale** e "assolutamente transmediale";
- portare **numerosi esempi** di tipologie di gioco, dall'analogico al tavolo, al pervasivo "e oltre". Lo si vedrà nella lezione sui **casi studio** del venerdì successivo: "voi non avete idea di quante cose esistono, bellissime, sconosciutissime e favolose". Daniele confessa che preparando e aggiornando le slide i colleghi gli hanno mostrato cose che lui stesso non sapeva nemmeno che esistessero;
- dare gli strumenti per **analizzare** i giochi altrui. Ruggero lo sottolinea: se il corso riesce nel suo intento, gli studenti sapranno non solo fare i propri giochi ma anche analizzare giochi già fatti, che è "una delle skill importanti anche nella parte di ricerca e sviluppo". Si vedranno casi studio e giochi pubblicati, sempre con l'obiettivo di capire **che cosa fanno e perché lo fanno**, per poi fare meglio il proprio lavoro;
- imparare a **fare giochi insieme**. Esistono gli autori che lavorano da soli (i *solo developer*), "ma sappiate che anche l'autore che lavora da solo poi si deve interfacciare con il pubblico, con un editore, magari con un editor, con un publisher". "Non lavorerete mai da soli": bisogna imparare a lavorare con gli altri.

2.1.5.2 Un corso intersezionale

Il corso è **intersezionale**: **non ci sono lezioni dedicate a una singola tipologia di gioco**. Non ci sarà "la lezione sul gioco da tavolo", "la lezione sul gioco di ruolo", "la lezione sul gioco pervasivo". Ci saranno lezioni su temi diversi, ciascuna pensata per essere utile a **tutte e tre le sfaccettature** del corso; alcune saranno più importanti per il gioco da tavolo, altre per il pervasivo, altre per il gioco di ruolo, ma nessuna sarà esclusiva. I docenti vogliono "contaminazioni e applicazioni assolutamente trasversali".

2.1.5.3 Un corso cross-generazionale

È anche un corso **cross-generazionale**: i gradi di esperienza di gioco quest'anno sono abbastanza allineati, ma le esperienze di vita, le età, le estrazioni sociali sono varie. "Ed è un pro, non è un contro. Più diversificata è la classe, più imparate meglio anche voi". La slide ha una piccola nota per fasce d'età:

Under 30 Concentratevi sulla vostra **crescita**. Evitate di confrontarvi con le persone più grandi che magari "ce la fanno meglio" in alcune cose.

Over 30 Concentratevi sul **consolidamento**, ma siate **umili**: i docenti diranno cose che faranno "storcere il naso". "Bisogna un po' reimparare per imparare": bisogna sapersi adattare.

[11] La trascrizione dice "il lumaven della situazione o il carcassone della situazione"; il primo titolo non è ricostruibile con certezza. Il senso è chiaro: non si chiede di rifare un grande classico del gioco da tavolo.

Ruggero contestualizza: lo dicono perché in passato hanno visto studenti che, essendo più adulti o avendo giocato di più, pensavano che l'esperienza *di gioco* si trasformasse automaticamente in competenza *di creazione* di giochi. Non è così. E non bisogna sottovalutare chi gioca meno: proprio perché gioca cose diverse, per risolvere un problema di design specifico "è una ricchezza incredibile avere qualcun altro che riesce a uscire dalla scatola".

2.1.5.4 Un corso pratico e adattabile

Il corso è **pratico**: gli studenti si concentreranno su un progetto di gruppo, il **project work**, lavorando in team. Ed è **adattabile**: il programma è definito, ma "si aggiorna sui vostri stimoli, priorità e necessità". Se durante il corso i docenti vedono che serve più una cosa dell'altra, sono "prontissimi" a dare più spazio a un argomento invece che a un altro.

2.1.5.5 Inquiry-based learning: domandate

Il metodo è quello dell'**inquiry-based learning**, l'apprendimento basato sulle domande. Di solito le persone dicono "ho una domanda, magari sarà stupida". "Non esistono domande stupide. Pensate che ci sia una domanda stupida? Fatela, alzate la mano". Vanno bene anche le **provocazioni**: se qualcosa non torna, è meglio chiedere. La condivisione è fondamentale.

I docenti danno **metodi, dati e conoscenze**, ma lasciano trovare le risposte agli studenti: "vi lasciamo un pochettino annegare nel mare prima di darvi la soluzione, in modo tale che la possiate trovare da soli". Di conseguenza **non ci saranno interrogazioni sulle definizioni**: interessa vedere **come** gli studenti applicano ciò che imparano. Se una definizione non viene in mente, non è un problema; l'importante è saperla usare.

2.1.5.6 Feedback, non valutazione

"Sì al feedback, no alla valutazione". I docenti danno **feedback verbali**, anche sulle attività da fare a casa (i task, che non saranno tanti). Il corso è calibrato per **studenti lavoratori**: c'è un po' da lavorare fuori dalle lezioni, ma in modo calibrato.

Il principio è **fallire oggi per imparare per il futuro**: *fail forward*, *fail to learn*. I docenti lasciano provare e fallire almeno una volta prima di dare la soluzione: "il classico metodo Montessori, quello originale, non quello travisato".

2.1.5.7 Le soft skill e i criteri di valutazione

Grande attenzione va alle **soft skill**: i lavori di gruppo, lo scambio propositivo, l'atteggiamento positivo, la propositività, la proattività. "Lavoro di gruppo" non significa stare ogni giorno in sala studio con gli altri: va benissimo che un gruppo si organizzi dandosi priorità e tempi in base alla disponibilità di ciascuno. L'importante è **collaborare**, essere **professionali**, rispettarsi a vicenda, rispettare le idee proprie e altrui e lasciare spazio agli altri, perché i progetti "non saranno solo vostri, saranno di tutte le persone del vostro gruppo".

La valutazione del project work non riguarda solo la **documentazione** prodotta, ma anche:

- la **presentazione** finale;
- l'**autovalutazione**;
- la **crescita personale**;
- l'**atteggiamento** tenuto nel corso.

Daniele cita: "da ognuno secondo le sue capacità, a ognuno secondo i suoi bisogni". Lavorare in gruppo non vuol dire che, in tre, ciascuno lavori esattamente il 33% del tempo. Ai docenti serve piuttosto capire **chi fa che cosa** nel gruppo, e che il processo interno sia **trasparente**.

Lavorare in cloud. Per lo stesso motivo il consiglio è di **non lavorare in locale**, ma direttamente nelle cartelle condivise del project work e dei task. Si è liberi di fare altrimenti, ma lavorando in cloud si dà ai docenti la possibilità di vedere *come* si lavora, il proprio "processo mentale".

2.1.6 Le regole di comportamento e il carico di lavoro

Alcune slide sono di quelle che si ripetono in ogni corso ("*repetita iuvant*", storpiato per scherzo in "ripetita Juventus").

Comportamento. Prestare attenzione durante le lezioni; interagire; portare rispetto; essere proattivi; alzare la mano e fare domande. **Tolleranza zero** per pregiudizi, maleducazione, mancanza di rispetto, *cheat* e scorciatoie. Daniele ammette che "cheat e scorciatoie" viene da altri corsi dove il tema è più importante, ma Ruggero lo ferma: "non sottovalutate questa cosa", perché implica anche **non far fare a ChatGPT i task**. Daniele ha una slide apposta, "perché è un discorso molto complesso nel 2026" (sezione 2.1.8).

Le lezioni sono intense. Le lezioni possono essere **dense**, "soprattutto le mie, sappiatelo: io sono logorroico", avverte Daniele. Bisogna chiedere spiegazioni se serve, prendere appunti, rivedere le registrazioni. Le lezioni sono "delle scintille": modi per far esplorare i temi oltre le slide. "Andate oltre, se riuscite".

Il carico di lavoro. Le regole sui task sono queste:

- i task **sono per voi, non per noi**;
- se si ha poco tempo, è meglio farli rapidamente piuttosto che non farli: "provateci";
- i docenti cercano di non chiedere più di **un'ora o due a settimana** oltre alle ore di lezione;
- non tutte le lezioni avranno un task, e il **project work** è il più importante di tutti.

I feedback dei docenti. I feedback "si guadagnano" man mano che si reagisce bene: non bisogna aspettarsi "dei papiri" sui task. Se i docenti hanno voglia di dare feedback più ampi lo fanno, soprattutto se vedono che lo studente se li merita. Ad Analog di solito non ci sono task *rejected*; ma se un docente rifiuta un task, vuol dire che si è fatto **meno del livello minimo** richiesto dal corso. I feedback possono non essere immediati: "siate un pochettino impietositi dal fatto che siamo dei docenti". E se un feedback non arriva, o se ne vogliono di più, bisogna scrivere, "perché siamo tre esseri umani e quindi possiamo anche dimenticarci".

2.1.7 Il project work e il calendario

2.1.7.1 Come funziona il project work

Una prima slide con le date del project work è da aggiornare ("perdonatemi il refuso"); Daniele la corregge al volo e rimanda alla slide successiva, più precisa. Il funzionamento è questo:

- i **gruppi li formano i docenti**. Gli studenti sono nove, quindi si faranno **tre gruppi da tre**: una dimensione pensata anche per il fatto che molti sono studenti lavoratori, così da non ritrovarsi "in due, a esplodere";
- ogni gruppo deve prima preparare un **pitch** del proprio gioco, da presentare in anticipo: un'idea, una bozza del gioco;
- poi si lavora al gioco **nelle pause del corso**: da qui a dicembre ci saranno due pause, quindi non ci sarà lezione tutti i lunedì e tutti i venerdì;
- ci saranno **review intermedie** con i **tutoraggi**, per lo più il **sabato mattina**, dalle 11 alle 12, con Ruggero su Google Meet. Servono ad aiutare i gruppi a portare avanti il project work.

2.1.7.2 Il calendario delle lezioni

Il **calendario** è nel calendario studenti della cartella condivisa. Attenzione: le date **possono variare**. "Il 90% rimarranno così", ma può capitare che, per motivi di lavoro, qualcuno stia male o non possa, o che "scompaia l'elettricità da tutte le città d'Italia". E l'ordine delle lezioni può cambiare se i docenti vedono che agli studenti serve prima una lezione invece di un'altra.

Ci sono **due pause importantissime**:

- dal **13 ottobre al 5 novembre**: è il periodo delle grandi fiere (**Essen** e **Lucca**[12]), "un periodo un po' frenetico", e cominciano anche i corsi accademici, quindi la pausa permette ai docenti di respirare. Per gli studenti è la pausa più importante: quasi un mese in cui sviluppare la parte importante del gioco;
- dal **24 novembre all'11 dicembre**, dopo qualche lezione a novembre.

E ci sono **due date da evidenziare**:

- il **25 settembre**: nei primi 5–10 minuti della lezione (che per il resto sarà tenuta da Beatrice) i docenti annunceranno i tre gruppi;
- il **12 ottobre**: il **pitch** dei giochi, con l'idea embrionale. Tra il 25 settembre e il 12 ottobre ogni gruppo prepara un breve pitch in formato slide; quel giorno i docenti lo rivedono e danno feedback: se il gioco va bene, se non va bene, come svilupparlo.

Dal 12 ottobre fino all'inizio di dicembre si sviluppa il gioco, con i tutoraggi: "sfruttate questi momenti di pausa".

2.1.7.3 Il calendario del project work

La slide successiva, "reale e aggiornata", riporta il calendario del project work (tabella 2.1.1 e figura 2.1.1). "Per ora non pensate al project work: ci penserete dal 25 settembre in poi".

Data	Che cosa succede
ven 25 settembre[13]	lezione: annuncio dei gruppi e del *mandate*, cioè di che cosa si dovrà fare
sab 3 ottobre, ore 11	prima review e tutoraggio con Ruggero (dopo una decina di giorni di lavoro)
lun 12 ottobre	pitch del gioco durante la lezione
13 ottobre – 5 novembre	prima pausa: sviluppo del gioco
sab 7 novembre	tutoraggio
sab 14 novembre	tutoraggio
24 novembre – 11 dicembre	seconda pausa: sviluppo del gioco
ven 4 dicembre, entro mezzanotte	consegna del gioco sul Drive (non c'è lezione)
ven 11 dicembre	i giochi vengono visti a lezione
14 dicembre	giornata finale, con la tavola rotonda con gli editori

Table 2.1.1: Il calendario del project work come lo presenta Daniele. Le date possono variare.

Perché la consegna è il 4 dicembre. Il 4 dicembre non c'è lezione: si deve solo consegnare il gioco, entro mezzanotte, sul Drive. La presentazione a lezione è l'11 dicembre, ma i tre docenti hanno bisogno di **una settimana** per rivedere il gioco in privato e valutare anche la **documentazione**; per questo la consegna non coincide con la presentazione. Che cosa voglia dire esattamente "consegnare il gioco" verrà spiegato più avanti.

[12] La trascrizione dice "Essen, Luca, Cose": si tratta con ogni probabilità della fiera Spiel di Essen, in Germania, e di Lucca Comics & Games, che si tengono entrambe in ottobre.

[13] Daniele dice "lunedì 25 settembre", ma nel 2026 il 25 settembre è un venerdì; il giorno della settimana nella tabella è corretto di conseguenza.

La giornata finale. Il 14 dicembre c'è la **giornata finale**, con una **tavola rotonda con gli editori**: di solito viene invitato Flavio, che è anche docente di game production, più qualche altro editore, "vediamo quest'anno chi riusciamo a portarvi".

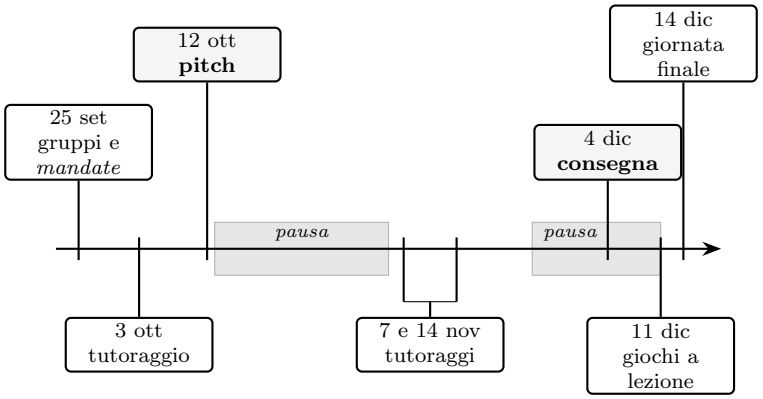


Figure 2.1.1: Una linea del tempo del project work, ricostruita dalle slide (scala approssimativa: dal 25 settembre al 14 dicembre). Le bande grigie sono le due pause senza lezioni, in cui i gruppi sviluppano il gioco.

Finita questa parte, Daniele apre alle domande: "Domande sul calendario? Sulle slide che ho fatto? Considerazioni? Non abbiate timore, io non ci credo". E la domanda arriva, sull'unico tema che aveva lasciato da parte.

2.1.8 L'intelligenza artificiale nel corso

2.1.8.1 La slide di Daniele

Daniele aveva già presentato una slide che definisce "importantissima". "Lo so che è paradossale, me ne rendo conto. Lo so che magari mi darete del luddista, dell'anti-tecnologia: non è così. Uso l'intelligenza artificiale per lavoro. Ma in questo corso, vi prego, **non usate l'intelligenza artificiale**". La richiesta vale per le attività, per i task e **anche per la prototipazione**. Le ragioni:

- "voglio che siate voi a fare il gioco, voglio che siate voi a imparare a farlo". Daniele parla da utilizzatore: prima di ChatGPT, che ha smesso di usare, oggi soprattutto di Claude, di cui "si sottovaluta quanto possa essere incerto, insicuro, impreciso, non all'altezza";

- nella game industry, **i publisher rifiutano automaticamente** i pitch e i prototipi fatti con l'intelligenza artificiale. Daniele lo ha visto di persona in "tantissime pitch review" quell'estate, anche al **First Playable** di Firenze[14]: anche nel mondo dei videogiochi i publisher che vedono pitch o prototipi fatti con l'IA "rigettano automaticamente la build", perché hanno ancora timore dell'IA e delle sue possibili conseguenze;

- "non so come sarà il futuro": non sa se bisognerà abituarsi a usarla tutto il giorno. Non è "totalmente contrario", purché sia usata "con raziocinio"; secondo lui oggi, negli Stati Uniti, non lo è;

- ma in un corso che si chiama *Analog Game Design* gli studenti devono saper fare i giochi da soli. In futuro, se vorranno farsi aiutare dall'IA per essere più efficienti, saranno liberissimi di farlo; durante il corso no, perché i docenti vogliono valutare **le loro competenze e capacità**.

[14] La trascrizione dice "First Playboy a Firenze": si tratta del First Playable, l'evento di business dell'industria videoludica che si tiene a Firenze.

2.1.8.2 La domanda: che cosa vuol dire usare l'IA in un gioco analogico?

La domanda di uno studente è precisa: che cosa si intende per "task fatti con l'IA"? Qual è l'uso dell'IA in un gioco analogico?

Ruggero: l'esperienza di chi valuta i prototipi. Ruggero risponde dalla sua esperienza nel settore, dove riceve prototipi da valutare per un'eventuale pubblicazione. Distingue due usi:

1. le **immagini generate**: illustrazioni, graphic design e così via. Per molti publisher è "un *reject* immediato, proprio immediato";

2. l'IA usata per il **regolamento**, per scrivere le regole e le meccaniche. È più difficile da individuare, ma molti publisher se ne accorgono comunque, perché le IA "non sono ancora brave abbastanza da creare un regolamento coeso meccanicamente e scritto con un senso per il vostro gioco". Quello che fanno è prendere parti di altri regolamenti, elaborare le strutture che vedono nei regolamenti pubblicati e riapplicarle. Ma molti giochi si prestano poco a un regolamento strutturato come quello di altri, perché **ogni gioco è un ecosistema meccanico a sé**. Il risultato sono regolamenti con riferimenti non immediati, e soprattutto con **la stessa meccanica descritta in modo diverso in pagine diverse**: due difetti che fanno perdere tempo all'editore, e in questo caso ai docenti.

Poi aggiunge un punto "personale, soggettivo": i docenti sono interessati a vedere il lavoro, la creatività, "il frutto della vostra creazione". E viceversa gli studenti vogliono i commenti dei docenti: "immaginatevi se voi fate un gioco e noi vi diamo i commenti scritti con ChatGPT". Altrimenti, dice ridendo, gli studenti fanno il gioco con ChatGPT, i docenti lo danno a ChatGPT per i commenti, "e facciamo molto alla svelta".

Daniele: perché gli studenti usano l'IA. Daniele racconta la sua esperienza con gli studenti, che secondo lui ricorrono all'IA in due momenti:

1. quando vanno in **crisi**: in ansia da prestazione o per i tempi. Cercano di risolvere il problema della propria disorganizzazione facendolo risolvere all'IA, senza accorgersi che l'output è "molto, molto scarso";

2. quando hanno l'impressione che l'intelligenza dell'IA sia **migliore** di quello che scrivono loro, o di quello che c'è in giro. Per Daniele è una "mancanza di palato critico": l'output dell'IA sembra buono in superficie, ma quando si va nel dettaglio e ci si fa delle domande "di solito fa un po' di *crumbling*", si sbriciola.

Lui usa Claude ogni giorno e ne conosce i limiti: "non c'è nemmeno una frase che io copio direttamente". Gli fa fare il lavoro più noioso, più ripetitivo e soprattutto **strutturato**, "replicare delle cose come se fosse una macro". Lo usa per il testo e per il codice, mai per le immagini: per motivi di **diritto d'autore**, e perché ha molti amici artisti ed è stufo di vedere il loro lavoro "dilaniato" dall'IA.

Ammette che i docenti non possono controllare gli studenti; per questo "ve lo consiglio". E riprende quella che definisce la cosa più bella detta da Ruggero ("ti ruberò la frase per il corso di narrative"): i docenti i feedback li danno di persona. Daniele a volte manda 30 minuti di audio a uno studente, o fa intere call per rivedere script, pitch e giochi: "è il mio lavoro, mi piace farlo e voglio farlo io, quindi mi aspetto che lo studente lo faccia lui". Come freelance, poi, è "stufo marcio" dei clienti che arrivano con qualcosa fatto con l'IA ("guarda che bello, pubblica questo"), perché il più delle volte il risultato è pessimo: non sanno come usarla. "È anche una questione di competenza".

Il prototipo non deve essere bello. Questo vale anche per il **prototipo**, su cui si tornerà nelle lezioni di prototipazione: in un prototipo non servono "graficoni allucinanti". Le grafiche bellissime fatte con l'IA spesso **peccano di user experience**: sembrano bellissime, ma quando il giocatore deve leggere la carta non capisce qual è il costo, qual è il potere, qual è l'effetto.

Beatrice: si perde l'occasione di imparare. Beatrice aggiunge due motivi che valgono "al di là dei nostri credo". Il primo: chi usa l'IA in sostituzione (per esempio per scrivere un testo, per accelerare i tempi o per trovare strategie) **perde la parte in cui può imparare**. Nel game design bisogna "tenere insieme i pezzi" di un regolamento, ma anche dell'esperienza dell'utente e di come trasmetterla; i passaggi necessari (creare il regolamento, creare le meccaniche, sperimentare, raccogliere i feedback) "si capiscono solo vivendoli". E si sta pagando un corso per imparare. Il secondo, sul piano editoriale: chi presenta a un editore un prodotto che ha come base l'IA rischia da un lato di non avere imparato abbastanza (e di ritrovarsi "a piedi" sul campo senza IA), dall'altro che l'editore se ne accorga, ed è un tema **reputazionale**. Ci sono editori "più illuminati di altri", quindi anche qui c'è un mercato da capire; ma i docenti preferiscono dire che cosa è più sensato per il mercato come lo conoscono.

Ruggero: non abbiate paura del giudizio. Ruggero vuole dare uno "spotlight" a un punto detto da Daniele e rimasto "semi-ignorato": a volte l'uso dell'IA nasce dalla **paura del giudizio**, dal voler consegnare qualcosa di più pulito, più strutturato possibile. "Vi giuro, non ve ne preoccupate". Ruggero detesta i voti e non ragiona per voti; non valuterà nessuno per grafiche fatte a matita. Insegna da anni, ha avuto studenti con dislessia e discalculia: refusi ed errori non gli interessano, "non è un corso di editing o di proofreading". Gli interessa che le **meccaniche siano coerenti con lo scopo di design**. Immagini fatte a matita o con Paint "non sono un problema, anzi, le preferisco" rispetto all'"ennesima immagine fatta con ChatGPT o Midjourney".

Daniele: il pensiero critico, costo e qualità. L'ultimo intervento è di Daniele, sul **pensiero critico**. Ragionando per assurdo ("un pensiero di Belzebù"), immagina che un domani l'IA venga usata ovunque. I suoi limiti, secondo lui, resterebbero due:

Il costo Tutti dicono che sta diminuendo e che diminuirà, ma non se ne può essere sicuri al 100%.

La qualità Cioè la capacità di leggere l'output. Se si dà un regolamento a Claude o a ChatGPT chiedendo quali problemi ha, il modello risponde con i problemi che, in percentuale, è più probabile che gli esseri umani abbiano già individuato nei dati su cui è stato addestrato. Ne derivano due problemi: se il problema del proprio gioco non è mai stato individuato prima, il modello non lo vedrà; e viceversa può segnalare problemi che il gioco non ha.

Fa l'esempio di Grammarly, che usa nella versione premium per rivedere i testi in inglese: spesso non accetta i suoi consigli, perché se vuole scrivere un dialogo di videogioco in **dialetto Cockney** londinese, lo deve scrivere in Cockney. Se Grammarly dice che è sbagliato, "sei tu che non hai capito il motivo per cui voglio scrivere questo testo in Cockney". La conclusione: prima di usare un'IA per un'attività bisogna sapere **che cosa si vuole ottenere**, qual è l'obiettivo, e quali **competenze** servono per valutare in modo critico se l'output di un qualunque LLM è davvero valido. Daniele non vuole dilungarsi oltre: è un argomento "molto caldo" per lui, che è anche scrittore e si sente dire da tutti "ti sostituiranno".

Subito dopo si presenta finalmente Gianmarco (lo si è visto nella sezione 2.1.3), e poi si fa una pausa di una decina di minuti. Daniele avverte che nella terza parte lui non ci sarà (alle sette e un quarto ha un impegno di lavoro), e lo stesso vale per Beatrice: la parte finale di domande e risposte sarà con il solo Ruggero, "in ottime mani".

2.1.9 Le figure professionali dell'industria analogica

Dopo la pausa (in cui si scopre che Angelo lavora in un piccolo birrificio in Brianza e si scherza sul rischio di un *drinking game*, tra la birra e la vendemmia) la lezione riprende con le slide di Ruggero. "Oggi è una lezione introduttiva": gli argomenti si vedranno "molto dall'alto" e verranno approfonditi nelle lezioni successive. Ma servono già ora, per creare **l'inizio di un linguaggio condiviso**: ci sono termini che torneranno spesso, a cominciare da quelli del mercato e dell'industria del gioco analogico, un'industria "molto varia, molto caotica, anche molto mutevole", con tanti tipi diversi di giochi e altrettanti tipi diversi di entità che ci lavorano.

2.1.9.1 Una decina di etichette

I docenti hanno raccolto "una decina di etichette", cioè di **ruoli** che nell'industria sono o possono essere presenti. Due avvertenze:

- nelle aziende più piccole **la stessa persona copre spesso più ruoli**, per forza di cose. Sarà anche il caso degli studenti, che lavoreranno in gruppi di tre e non di venti: è facile che ciascuno finisca a fare più cose. I docenti ne terranno conto, e qui entreranno in gioco anche competenze diverse dal game design (programmi di grafica, illustrazione, impaginazione, layout, *proofreading*...);
- Daniele precisa che **non si chiede agli studenti di diventare** illustratori o altro. Ruggero concorda, ma aggiunge che è utile sapere anche *che cosa non è il proprio lavoro*. Se in un gruppo nessuno sa illustrare, ai docenti va benissimo un prototipo che mostri dove vanno le immagini: **placeholder**, concept, anche semplici box di testo con scritto "qui vorremmo un'immagine di...". "È importante anche sapere che cosa sapete fare e che cosa non sapete fare".

La tabella 2.1.2 riassume le figure; le sottosezioni seguenti le riprendono una per una, con i commenti dei docenti.

Figura	**Di che cosa si occupa**
Game designer	crea le regole alla base dell'esperienza: le meccaniche, come interagiscono tra loro e come chi gioca interagisce con esse
Game developer	fa da ponte tra chi fa game design e il publisher: bilancia il prototipo, cura i modelli matematici, adatta il gioco (e il paragame) alle necessità del publisher
Editor (proofreader)	revisiona e adatta i testi a partire dalla prima stesura del designer (correzione, linguaggio inclusivo...)
Publisher (editore)	pubblica e produce fisicamente il gioco; il suo marchio è sulla scatola o sulla copertina
Sales e marketing	il lato commerciale: vendite (anche B2B) e comunicazione del prodotto
Graphic designer	icone meccaniche, wireframe, UX: il lato funzionale degli elementi visivi
Illustratore	le illustrazioni: il *look and feel*, il world building, il "vibe" del gioco
Sensitivity reader (cultural consultant)	verifica l'accuratezza e la rappresentazione di un tema sensibile che il team non conosce
Producer	coordina il progetto (quasi project management), parla con le tipografie, verifica che i materiali siano sostenibili economicamente

Table 2.1.2: Le figure professionali dell'industria del gioco analogico presentate nella slide di Ruggero, in sintesi.

2.1.9.2 Game designer

Il **game designer** (o la game designer) è chi si occupa, "sembra banale a dirlo", di **creare le regole alla base dell'esperienza** che si vuole far arrivare. Vale per tutti i tipi di giochi, non solo per quelli analogici, e che cosa faccia concretamente in un progetto o in una serie specifica "può mutare tantissimo". Nell'analogico non verrà chiesto di programmare o di prototipare in digitale; verrà chiesto di **creare le meccaniche di gioco**, di capire **come le meccaniche si interfacciano tra loro** e **come le persone che giocano si interfacceranno con le meccaniche**.

2.1.9.3 Game developer

Il **game developer** (o la game developer) sarà l'argomento delle due lezioni di Marta. La parola *development* è nota soprattutto dall'ambito videoludico, ma nell'analogico ha un significato molto specifico "che non c'entra niente con la programmazione": il game developer è **un ponte tra chi fa game design e il**

publisher. Di solito si occupa di **bilanciare il prototipo**, di curare i **modelli matematici**, di adattare il gioco e il **paragame**[15] alle necessità del publisher. La distinzione viene ripresa e approfondita nella sezione 2.1.11.

2.1.9.4 Editor

L'**editor**, a volte chiamato **proofreader** a seconda dell'azienda, si occupa della revisione dei **testi**: non della stesura iniziale, perché di norma, data la scala dell'industria, è chi fa game design a scrivere un **primo draft**, e l'editor ci passa sopra. Non solo per correggere i refusi: spesso il suo lavoro è anche **adattare** i testi, per esempio renderli **gender neutral**. Se ne vedranno casi studio più avanti.

2.1.9.5 Publisher

Il **publisher**, o **editore**, è l'entità che nell'ambito commerciale **pubblica** giochi, giochi di ruolo o giochi da tavolo. È una figura "più solidificata" nel gioco da tavolo e nel gioco di ruolo da tavolo; **nell'ambito pervasivo non esiste proprio in questa forma**, come mostrerà Beatrice parlando dei casi studio. Se si compra un gioco da tavolo o un gioco di ruolo in un negozio, il marchio del publisher è di solito sul retro della copertina o sul fronte della scatola: è l'entità che, di solito, **produce fisicamente** il gioco.

2.1.9.6 Sales e marketing

L'ambito **sales e marketing**, a volte affidato a un'unica persona e a volte separato, riguarda il lato commerciale. In alcune aziende è interno al publisher, e si occupa di parlare con le fabbriche e le tipografie, di "far quadrare i conti". In altre realtà è completamente diverso, specialmente nel **B2B** o nei giochi **didattici** e **formativi**, di cui Daniele e Beatrice "sapranno parlare meglio". Il tema viene ripreso più avanti in dettaglio (sezione 2.1.10).

2.1.9.7 Graphic designer e illustratore

Ruggero presenta insieme **graphic designer** e **illustratore** proprio perché sono ruoli molto diversi che si tende a far coincidere:

- chi fa **graphic design** si occupa delle **icone meccaniche**, del **wireframing**, della **UX**: del lato **meccanico e funzionale** degli elementi visivi, cioè di tutto ciò che serve meccanicamente nel gioco, o anche alla sua "qualità della vita";
- chi fa **illustrazione** si occupa dell'*art*, delle illustrazioni: copre il **look and feel**, la parte di **world building** e il "vibe" che si dà al gioco.

Daniele: che cos'è la user experience. Daniele interviene sulla **user experience**, "importantissima per me": ha un background da **layout artist** (impaginava magazine), un mestiere in cui la UX conta moltissimo. Per user experience non si intende solo l'**interfaccia grafica**, la UI, che anche nei videogiochi è disegnata da un artista. La UX è qualcosa che può fare anche il game designer: **disegnare l'esperienza del giocatore**, cioè "il gesto, l'intenzione, l'intuito" che stanno dietro l'aspetto visivo dell'interfaccia. Per fare UX non serve saper disegnare. L'esempio è quello della **porta**: se si progetta una porta, bisogna indicare all'utente come si apre, come si prende la maniglia; "quella è user experience". Il design della maniglia in sé, invece, è **interfaccia**.

Un aneddoto sulle icone. Ruggero aggiunge che molte persone che fanno **graphic design ben fatto non sanno illustrare**, e spesso vale il viceversa. Lo si nota quando le icone di un gioco sono bellissime ma non funzionali: andando a guardare i crediti, di solito si scopre che le icone le ha fatte la persona che ha fatto le illustrazioni. "Comunque non andiamo adesso in questo, perché è un altro vaso di Pandora".

[15] La trascrizione dice "il Wix, il paragame". Il *paragame* è, in termini generali, tutto ciò che accompagna il gioco senza esserne il sistema di regole (regolamento, scatola, materiali, comunicazione); i docenti lo tratteranno in una lezione dedicata, e qui il termine è solo citato.

2.1.9.8 Sensitivity reader

Il **sensitivity reader**, o **cultural consultant** a seconda del progetto, fa un lavoro di *sensitivity reading* del gioco. È una figura molto usata nell'ambito **formativo e didattico**. L'esempio di Ruggero: se si fa un gioco per un ospedale su una malattia specifica, chi lo progetta non è un medico, ma il gioco deve essere **accurato** e **rappresentativo** delle dinamiche che quella malattia comporta. Serve una persona che conosca quel tema, anche se non sa che cosa sia il game design. Sono figure chiamate di solito **a progetto**, per giochi specifici, che compensano competenze che il team non ha "per forza di cose, perché noi non sappiamo tutto"; e a volte, per lavoro, si devono fare giochi su temi che non si conoscono personalmente.

Daniele: non è un censore. Daniele segnala un problema italiano: alcune persone, anche online, hanno confuso il sensitivity reader con un **censore**, "una persona che fa censura, soprattutto in materia di *woke*". "Assolutamente non è così". La parola *sensitive* non significa che il pubblico sia sensibile a un tema, ma che **l'argomento di cui tratta il gioco è sensibile**. In un gioco sulla medicina, realistico o no, non si possono inserire imprecisioni. Per questo esistono i sensitivity reader "più disparati": da quelli per la cultura indiana o afroamericana fino a quello su "come si allevano gli gnu in Africa". È una persona competente su un argomento che aiuta a essere **veritieri**. Certo, uno degli ambiti principali in cui si usano è quello culturale, ma "non fate l'errore di pensare che siano dei censori". Daniele ha lavorato con "centinaia" di sensitivity reader e nessuno gli ha mai detto "non puoi scrivere questa cosa". Di solito sono come degli **editor**: danno opzioni, mostrano i punti problematici, anche quelli di cui non ci si era nemmeno accorti.

Ruggero conferma con un esempio di quando lavorava in **Danimarca** con l'**OMS**: giochi sul **diabete di tipo 2** nei bambini, per un target di ragazzi delle scuole medie. C'era un team dell'OMS specializzato nella comunicazione sul diabete, per garantire che il gioco fosse accurato e ne desse una rappresentazione adatta al target. Il sensitivity reader aiuta a rendere il gioco più accurato e "più tematicamente coeso".

2.1.9.9 Producer

Il **producer** è una figura a volte interna al publisher. In un progetto grande è la figura di riferimento, "quasi" un project manager. In un progetto piccolo si occupa spesso anche di parlare con le **tipografie** e di fare la **lista dei materiali**, per assicurarsi che i materiali non siano solo funzionali al gioco ma anche **economicamente sostenibili** da produrre: "magari è un gioco bellissimo, ma poi costa più di quanto potete permettervi, e nessuno vi comprerà mai un card game a 200 euro".

2.1.9.10 Perché conoscere queste figure

Ruggero chiude la slide "tornando a ombrello": perché parlarne, se gli studenti sono lì per fare game design? Non solo per saper dare un **nome** alle figure. Non c'è niente di male a creare un gioco solo per sé; ma chi vuole fare giochi a livello lavorativo, perché li giochino altre persone, "magari più persone possibile", **si interfaccerà con queste figure**, e sapere che cosa fanno e come parlarci è parte importante del percorso. Nel corso se ne toccheranno diverse: il development con Marta, la produzione con Flavio, e lezioni sulla UX, sul paragame, sul graphic design. "Abituatevi mentalmente: non siete in solitaria contro il mondo".

2.1.10 Marketing e sales

Arrivato all'angolo della slide con marketing e sales, Ruggero passa la parola ai colleghi: "io è meglio che non mi fate parlare di marketing e sales", anche per esperienze personali.

2.1.10.1 Beatrice: i giochi trattati come prodotti

Beatrice premette che le due voci sono in parte autoesplicative. Nel mondo capitalista in cui viviamo, i giochi sono "strumenti molto potenti per fare n cose", ma sono trattati come **prodotti**, e il corso ragionerà anche in quest'ottica. Nel **marketing** delle industrie ludiche e videoludiche ci sono varie professionalità che gli studenti probabilmente già conoscono: dal *copy* alla grafica, tutto ciò che riguarda il **vendere** il gioco o l'attività di gioco.

Il lato **sales** è forse la distinzione meno nota. Come in tante aziende, è la parte che si occupa delle vendite **B2B**, cioè tra aziende: fare accordi per vendere prodotti ludici a diverse realtà, per esempio ai **distributori**, o a una casa editrice che vende alle librerie, che poi vendono al consumatore finale. Chi si occupa di sales nel mondo ludico capisce come vendere **alle aziende**. I due mondi si parlano spesso, e soprattutto nelle realtà piccole si trovano figure che fanno "un po' sales, un po' marketing".

2.1.10.2 Daniele: che cos'è (e che cosa non è) il marketing

Daniele il marketing si è trovato "purtroppo" a farlo, e lo fa anche per l'accademia. "Purtroppo" perché in Italia è mal compreso: molti lo identificano con l'*advertisement* online. Non è così: il marketing è una **disciplina molto più antica**, e in Italia "abbiamo molto dimenticato" come si fa il marketing tradizionale. Alcune cose da ricordare:

1. **Il marketing non dice che cosa pubblicare, ma come pubblicarlo**. Esistono, nei videogiochi e nei giochi da tavolo, realtà che fanno prodotti **marketing driven**: l'esempio sono gli **hyper-casual game** per smartphone, quelli con il personaggio che va a destra e a sinistra e l'esercito che si allarga. Possono anche guadagnare, ma "sono un po' la morte della creatività". Il marketing ideale **prende un prodotto pensato da altri e gli dà le maggiori chance di essere venduto**: cerca il pubblico ideale per il prodotto che si ha, senza doverlo cambiare. Anzi, il mondo videoludico, e ludico in generale, "è lastricato" di errori di dipartimenti marketing che hanno cambiato completamente non il gioco, ma il modo in cui l'hanno pubblicizzato, e il gioco è fallito.

2. **Usare strumenti del marketing non significa fare marketing**. Game design e game development a volte usano strumenti che vengono dal marketing (gli studenti ne vedranno con Beatrice, con Daniele e con Marta).

3. **Il marketing è un gerundio**: si fa. È un'azione che si compie per comunicare; la comunicazione è attiva, e il marketing è l'insieme delle attività con cui si comunica il proprio gioco, dal *copy* ai social. Può includere il digital marketing, "ma non è solo quello".

4. **Il marketing esiste anche se si crede che non esista**. In Italia molti non vogliono "fare marketing" perché lo associano al lucro. Ma il marketing è **lo studio di come gli esseri umani comprano e vendono**, ed è innato in noi: ci sono logiche di cui siamo vittime, e "anche se sei il bastian contrario, l'anticapitalista", magari si compra comunque su Amazon perché fa comodo. Sono meccanismi che esistono, "affascinanti anche, a volte".

Gli studenti non dovranno fare marketing, ma dovranno ricordare alcuni di questi concetti, soprattutto nelle lezioni con Marta, che tratta argomenti legati al marketing. E, chiude Daniele, come game designer e game developer "per lo più speriamo che il marketing non ci rovini i giochi", soprattutto nel videoludico.

2.1.11 Game design e game development

Ruggero torna sulla distinzione più importante della slide, "proseguendo con questa strada":

Game design è chi **progetta l'esperienza** tramite la **creazione delle meccaniche** di gioco.

Game development è chi fa **mediazione** tra la persona che scrive il game design e la **casa editrice**.

2.1.11.1 I red flag dei designer

Ruggero lo ripete perché lo vede ogni giorno nel suo lavoro: tra giochi pubblicati e prototipi, ne valuta "quasi uno al giorno, se faccio un conto annuale", e solo una minima percentuale arriva alla pubblicazione. Molte volte i **red flag** vengono proprio da designer che non hanno presente questa definizione, e che cominciano a mettere **clausole sulla non modifica del gioco**: il gioco deve restare com'è. "Questa cosa è impossibile con un publisher, spesso". Il publisher, per forza di cose, potrà chiedere di cambiare la grafica, di reimpaginare il regolamento, di cambiare i materiali perché troppo costosi: chiederà dei **compromessi**. E la persona che, all'interno del publisher, fa questo lavoro è il **developer**.

Sta al designer capire **su che cosa del proprio gioco si può trovare un compromesso e su che cosa no**. Publisher diversi lavorano in modo diverso, e il designer ha sempre l'autorità di dire "no, non voglio pubblicare questo gioco se queste sono le modifiche". Ma deve sapere chi fa il lavoro di development, perché se il suo obiettivo è la **pubblicazione tradizionale** si interfaccerà con questa figura.

2.1.11.2 Daniele: una distinzione che manca al videogioco

Daniele sottolinea che questa distinzione è **tipica del mondo analogico** ed è "molto interessante". In Italia la parola *developer* è diventata negli anni quasi un sinonimo di programmatore di videogiochi, o di chiunque lavori nel videoludico. Secondo lui c'entra anche il fatto che qualche anno fa il sito **Gamasutra** ha cambiato nome in **Game Developer**, e la parola è diventata un ombrello per tutte le professioni del videogioco. "Mi fa un po' ribrezzo": il game developer dovrebbe essere un ruolo di design, un designer che, come nell'analogico, fa **mediazione**. Nel videogioco i publisher hanno in effetti i loro developer interni, che adattano il gioco, per esempio alle console e ai loro kit di sviluppo. Ma la distinzione dovrebbe esistere anche nel videogioco, perché è "difficilissimo" avere la mentalità per fare le due cose insieme: chi fa design ha un focus che raramente si può sovrapporre a quello di chi fa mediazione.

A Daniele piace la parola *develop*, **sviluppare**, nel senso di prendere qualcosa che è stato progettato e **renderlo possibile nel mondo reale**. Ci sono progetti che hanno bisogno di uno sviluppo che "polishi" l'esperienza: la semplifichi, la migliori, la renda comprensibile e la concluda, ma soprattutto la renda **stampabile** e **fruibile** per l'utente. La metafora crossmediale è quella della **fotografia**: chi sviluppa le fotografie non è chi scatta la foto, ma chi la rende visibile, anche fisicamente. "È un po' retro come metafora, ma ci capiamo". "È vero, però è giusto", conferma Ruggero.

2.1.12 Il mercato: Italia ed estero

Il corso è in italiano e si svolge in Italia, ma bisogna conoscere anche il **mercato estero**: molti giochi progettati da autori italiani vengono pubblicati all'estero, e conoscere il mercato estero aiuta a capire come il proprio gioco può arrivarci, "o viceversa, come non può arrivarci". La tabella 2.1.3 riassume il confronto della slide.

Italia	**Estero (soprattutto Europa)**
il gioco è ancora visto come un'attività **infantile**, anche se la normalizzazione è in crescita	forte tradizione di gioco anche in un contesto **familiare adulto**, soprattutto "dalla Germania in su"
risposta **conservativa** sul tema ludico	**premi di prestigio** riconosciuti in tutto il mondo (Spiel des Jahres, As d'Or)
forte focus sulla **localizzazione** di giochi stranieri	si cercano soprattutto giochi **originali**
investimento **molto basso** sulla creazione di giochi originali interni; pochissimi designer assunti	**developer interni** pagati con un contratto (Francia, Germania, Danimarca...)

Table 2.1.3: Il mercato del gioco analogico in Italia e all'estero, secondo la slide di Ruggero e i commenti di Daniele.

2.1.12.1 L'Italia

- In Italia il gioco è ancora visto come un'**attività infantile**, anche se la **normalizzazione** è in crescita. C'è ancora una forte risposta **conservativa** sul tema ludico.
- C'è un forte focus sulla **localizzazione** dei giochi stranieri: in una qualunque ludoteca o negozio, molti dei giochi da tavolo e di ruolo sono di publisher o designer esteri, pubblicati in italiano da publisher italiani. (La localizzazione, cioè portare un titolo dall'estero in Italia, esiste come funzione all'interno del publisher; Ruggero non l'ha messa nella slide delle figure perché non è il focus del corso.)

- C'è, "purtroppo, preparatevi psicologicamente", un **investimento molto basso** sulla creazione di giochi originali interni alla casa editrice. Pochissime case editrici italiane hanno designer interni pagati a tempo pieno con contratto a tempo indeterminato. Daniele lo dice senza mezzi termini: "il mercato è molto piccolo, conosciamo praticamente tutto il settore", e le persone che in Italia fanno game design analogico a tempo pieno con un contratto a tempo indeterminato "sono meno delle dita delle mie due mani". Di solito la formula è quella del **freelance**, del contratto, delle **royalty**. Chi va da un publisher aspettandosi di essere assunto a tempo pieno come game designer deve sapere che "non siamo purtroppo ancora in quel tipo di situazione".
- C'è però un contraltare: in Italia, soprattutto nel gioco di ruolo ma anche nel gioco da tavolo, c'è stata molta **imprenditoria**, anche grazie a **Kickstarter**, che ha aperto le porte ad altri sistemi di **crowdfunding**. È un altro argomento, perché richiede un team, un investimento e così via.
- E gli autori non mancano, anzi: le occasioni di incontro tra autori vedono ogni anno un aumento dei possibili autori[16]. Bisogna "sgomitare" tra tanti autori, che magari arrivano da esperienze molto diverse.

Ruggero precisa: l'investimento basso è "dall'alto". Le persone che fanno e vogliono fare giochi sono tantissime e in aumento; scarseggia l'investimento, sia dei publisher disposti a pagare stipendi sia dei **bandi** pubblici. I bandi esistono (Beatrice ne parlerà per i giochi pervasivi, che spesso si muovono in quella direzione, e Ruggero cita qualcuno che sta facendo un card game con un bando italiano, "una cosa assurda, ma lo sta facendo"), ma sono una rarità, soprattutto rispetto all'estero.

2.1.12.2 L'estero

- L'estero (dipende dalla nazione, ma in generale il resto d'Europa) ha una forte **tradizione di gioco** anche in un contesto **familiare adulto**, soprattutto "dalla Germania in su".
- Ha **premi di prestigio** riconosciuti in tutto il mondo, come lo **Spiel des Jahres** tedesco e l'**As d'Or** francese[17].
- Cerca soprattutto **giochi originali**. Lo si vede dal fatto che molti autori italiani di giochi da tavolo pubblicano in Germania o in Francia e **non vengono pubblicati in Italia**, "che è tutto dire".
- Ha **figure di development interne**: non è raro vedere publisher francesi, tedeschi, danesi con developer pagati internamente, a contratto.

"Poi la situazione è ancora più drammatica", ammette Ruggero, "ma per ora vogliamo tenere il morale un po' alto. È l'introduzione. Ci sarà tempo di andare nella tristezza".

Prima di proseguire, Ruggero insiste sulle domande: con lui si può interrompere anche mentre parla, alzando la mano con le reaction o semplicemente togliendo il microfono e chiedendo. "Non mi offenderò mai, anzi, sarò felice di vedervelo fare e mi ricorderò positivamente di voi".

2.1.13 Perché serve una terminologia comune

La serie di slide successiva è "una piccolissima parte di fondamenti di design" ("non spaventatevi, vi vedo che vi spaventate un po' "). Anche qui molti temi verranno ripresi in dettaglio, ma l'idea è arrivare alla prossima lezione con almeno un **glossario**, un **linguaggio condiviso**: se qualcuno dice *game developer*, tutti sanno di che cosa sta parlando.

Ruggero parte da una domanda, e precisa che non è retorica: **perché ci serve una terminologia comune?** Gli studenti rispondono:

- Gianmarco, "molto pragmatico": perché bisogna lavorare in team e capirsi. Vero, nel corso e anche fuori;

[16] La trascrizione dice "i DAG a Parma"; il riferimento è probabilmente a IDEAG, il circuito di incontri tra autori di giochi italiani, ma non è ricostruibile con certezza.

[17] La trascrizione dice "lo Spield AR, la SDOR".

- un'altra studentessa: per la **comunicazione**, tra di noi e con tutti quelli che sono interessati nel settore. Vero anche questo: conoscere i termini comuni permette di parlare con chi fa game design o development nel dettaglio, nel tecnico, nello specifico.

Ruggero aggiunge altre due ragioni.

Saper comunicare il proprio gioco. La prima riguarda la **comunicazione del gioco** verso l'esterno. "E lo dico non tirando acqua al mio mulino, ma anzi togliendola": **non è spesso il buon game design a far diffondere un gioco**. "Vorrei che lo fosse, sarebbe bellissimo, ma purtroppo non è così". Ci sono giochi bellissimi che non sanno comunicare, che non sanno dire che cosa fanno, e che quindi è difficile, da fuori, trovare invitanti; e ci sono giochi mediocri dal punto di vista del design che si comunicano molto bene, e che quindi conoscono in molti. Saper parlare di quello che si fa è utile **a sé stessi**, in primis.

Documentare per sé stessi. La seconda è la **documentazione**: documenti di design, di brainstorming e così via. "Sembrerà una battuta, ma provate a recuperare un documento di brainstorming una settimana dopo, se non avete scritto bene le cose che avete scritto". Chi l'ha fatto sa già di che cosa si parla; "chi non l'ha ancora vissuto, forse lo vivrà".

2.1.14 Game e play

La **prima, importantissima divisione terminologica** è quella tra *game* e *play*. In italiano c'è un problema: **gioco** e **giocare** hanno la stessa radice, mentre in altre lingue, tra cui l'inglese, sono due parole strutturalmente diverse:

Game il **gioco come artefatto**: il gioco come ecosistema di meccaniche, come struttura di regole, come prodotto ("tra virgolette"), come oggetto fisico.

Play l'**atto**, il verbo: le persone che giocano, l'approccio fisico all'attività del gioco.

È importante perché gli studenti faranno **game design**; esistono anche ambiti di *play design*, ma qui "creerete giochi, non giocattoli".

2.1.14.1 Giochi e giocattoli

Qual è la differenza? Uno studente risponde: **la presenza di regole**. Esatto: un gioco ha delle regole, un'attività di *play* non necessariamente. Con un giocattolo si può fare *play*: c'è chi gioca con i Lego, chi gioca a guardia e ladri o a nascondino al parco, e magari alcune di quelle regole sono "più o meno semi-solidificate". Ma il *play* permette spesso un approccio non strettamente legato alle regole.

Viceversa, **un game può esistere anche se nessuno lo sta giocando**. Se si prepara su un tavolo una scacchiera con i pezzi degli scacchi, chi passa e la vede riconosce il gioco degli scacchi, anche se nessuno sta giocando. Se invece si vedono dei bambini correre in un parco giochi, quei bambini *stanno giocando*, ma non stanno giocando *a un gioco* specifico. "È importante separare questa cosa"; ci si tornerà, "ma per ora abituatevi a questa differenza".

Approfondimento: giocattoli, giochi e regole. La stessa distinzione è il punto di partenza del capitolo che Jesse Schell, in *The Art of Game Design*, dedica alla definizione di gioco (*The Experience Rises Out of a Game*). Schell nota che usiamo perfino un linguaggio diverso: "un gioco è qualcosa a cui si gioca" (*a game is something you play*), "un giocattolo è qualcosa con cui si gioca" (*a toy is something you play with*); e arriva a definire il giocattolo come "un oggetto con cui si gioca". Analizzando poi la definizione di gioco di Avedon e Sutton-Smith, ne ricava che "i giochi hanno regole, i giocattoli no", e che le regole sono uno degli aspetti *definitori* del gioco, insieme alla volontarietà, agli obiettivi, al conflitto e alla possibilità di vincere o perdere. Schell osserva anche che molti giochi sono costruiti **sopra** un giocattolo ("una palla è un giocattolo, ma il baseball è un gioco"), e tra i suoi consigli di prototipazione c'è proprio *Build the Toy First*: costruire prima il giocattolo, cioè qualcosa che sia già piacevole da manipolare, e poi il gioco intorno. È una lettura utile della distinzione di Ruggero: il *play* può fare a meno delle regole, il *game* no.

2.1.15 Il framework MDA (e il DPE)

2.1.15.1 Meccaniche, dinamiche, estetiche

Un'altra terminologia fondamentale è quella del framework **MDA** (*Mechanics, Dynamics, Aesthetics*; in italiano a volte "MDE"). Viene da un **paper** nato nell'ambito del videogioco[18], che ha "solidificato" queste terminologie, e divide un gioco in tre aree:

Meccaniche le **regole per come sono scritte**. "Ogni volta che pescate un 2 fate dei punti": quella è una meccanica.

Dinamiche l'**applicazione in tempo reale** delle meccaniche. La dinamica è il giocatore che, pescando una carta, **spera** che esca un 2. Quel processo non è una regola: "il gioco non vi dice che dovete sperare che esca un 2".

Estetiche l'insieme di quello che il gioco vuole comunicare, l'**esperienziale**.

L'esempio di Ruggero mette insieme le tre aree. In un gioco in cui pescare un 2 fa fare punti (meccanica) e in cui quindi si spera che esca un 2 (dinamica), **che estetica si ottiene se nel mazzo di 100 carte c'è un solo 2? E se ce ne sono 50?** L'emozione di aver pescato *l'unico* 2 del mazzo è completamente diversa da quella di pescarne uno sapendo che ce ne sono cinquanta e che è molto probabile trovarlo. Le regole sono quasi identiche; cambia l'esperienza.

2.1.15.2 Il falso amico: "estetica"

Daniele avverte di non sottovalutare la parola **estetica**, che in italiano è un **falso amico**: sembra indicare la parte grafica, e invece non lo è. Si incontreranno designer che hanno studiato MDA e altri che non l'hanno studiato, e lo si capirà proprio da questo: i secondi diranno che le *aesthetics* sono la grafica. Le *aesthetics* sono la parte **esperienziale**: come nell'esempio di Ruggero, quello che lascia al giocatore aver pescato l'unico 2 del mazzo.

2.1.15.3 Il DPE: design, play, experience

Per gli italiani, aggiunge Daniele, è più facile capire l'MDA attraverso un altro paper, quello del **DPE**[19], che di fatto fa un parallelismo assegnando **design, play, experience**:

- le **meccaniche** sono il lato **design**;
- le **dinamiche** sono il lato **play**;
- le **estetiche** sono l'**experience**, l'esperienza che riceve chi gioca.

"Se vivete le *aesthetics* come l'esperienza che vive chi sta giocando, state a postissimo". In questo momento Beatrice lascia la lezione ("ciao, Bea"). I paper di MDA e DPE sono in inglese; chi vuole può dargli una lettura.

Approfondimento: le due direzioni dell'MDA e le otto estetiche. Le frecce della figura 2.1.2 riprendono un'idea centrale del paper originale, che la lezione non esplicita: designer e giocatore guardano il gioco da lati opposti. In *Challenges for Game Designers*, Brenda Brathwaite e Ian Schreiber la riassumono così: i designer si chiedono quale estetica vogliono ottenere, definiscono le dinamiche che porterebbero a quella sensazione, e poi creano le meccaniche che producono quelle dinamiche. Lo stesso libro conferma l'avvertenza di Daniele sul falso amico: "le *aesthetics* non si riferiscono all'aspetto del gioco, ma alla risposta emotiva" che il designer vuole suscitare. E offre esempi di dinamiche nel senso di Ruggero: in uno sparatutto

[18] È il paper di Robin Hunicke, Marc LeBlanc e Robert Zubek, *MDA: A Formal Approach to Game Design and Game Research*, del 2004. Daniele aggiunge il paper alla cartella condivisa durante la lezione.

[19] È il framework *Design, Play, Experience* di Brian Winn, proposto come estensione dell'MDA per i giochi educativi (*serious games*). Anche questo paper viene aggiunto da Daniele alla cartella condivisa.

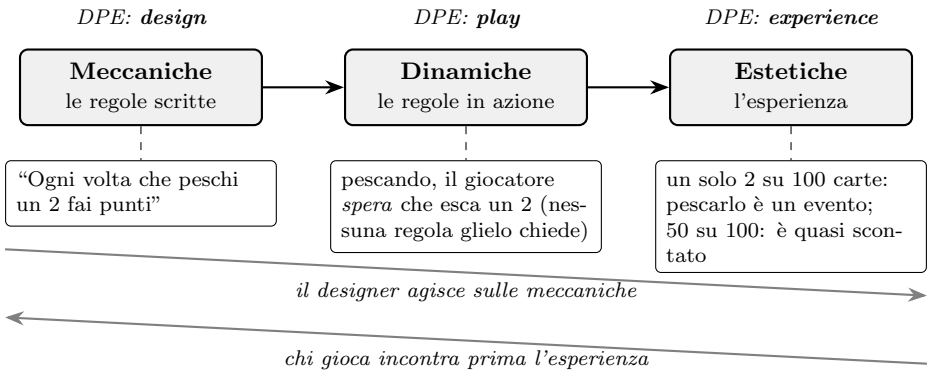


Figure 2.1.2: Il framework MDA con l'esempio del "2" usato da Ruggero, e la corrispondenza con il DPE suggerita da Daniele. Le due frecce in basso riassumono la lettura classica del paper: il designer costruisce le meccaniche per ottenere un'esperienza, mentre il giocatore parte dall'esperienza.

i punti di *spawn* sono una meccanica, mentre appostarsi accanto a uno di essi per colpire chi riappare (lo *spawn camping*) è una dinamica; negli scacchi i movimenti dei pezzi sono meccaniche, mentre le aperture codificate sono dinamiche. Per dare un nome alle estetiche, il paper MDA propone una tassonomia, poi detta degli "otto tipi di *fun*", che Schell riprende in *The Art of Game Design* come *LeBlanc's Taxonomy of Game Pleasures*: **sensazione**, **fantasia**, **narrativa**, **sfida**, **compagnia** (*fellowship*), **scoperta**, **espressione** e **sottomissione** (il piacere di entrare nel "cerchio magico" e lasciarsi andare a un passatempo). È un primo esempio di quello che i docenti chiedono con le parole vietate della prossima sezione: invece di dire che un gioco è "divertente", dire *quale* esperienza offre.

2.1.16 Le parole vietate

"Attenzione, adesso qua triggeriamo delle persone". Ruggero annuncia una regola che "tornerà": nel corso ci sono delle **parole vietate**. "Non perché siamo dei censori"; il perché si vedrà in una lezione apposita, perché spiegarlo adesso richiederebbe un'ora. Ma Daniele vuole spiegarne "un minimo".

2.1.16.1 La regola

In **nessun materiale scritto** consegnato ai docenti, task compresi, sono accettate queste parole, né in inglese né in italiano:

Inglese	**Italiano**
fun	divertimento
immersion	immersione
entertainment	intrattenimento

"Non li voglio neanche vedere". Daniele lo ripete con un paragone: "se mi fate i disegni con Paint, benissimo, top, fantastico, siamo ancora best friend. Se mi scrivete *divertimento*, io sbrocco". È una **regola dell'accademia**: negli stessi corsi triennali di narrative design e game design le parole sono vietate agli studenti per tre anni; "per voi solo per tre mesi". Daniele aggiungerebbe volentieri un quarto termine, **gamification**.

2.1.16.2 Perché

Il motivo approfondito lo darà Ruggero nella lezione apposita. Daniele dà il suo. Ruggero e Diego Ricchiuti sono molto contrari a *fun* e a *immersion*; lui ha aggiunto *intrattenimento*. Sono **parole troppo passepartout e troppo ambigue**:

- **intrattenimento** in Italia è stata usata per anni come un modo per **sminuire**: "ma sì, tanto è un gioco solo di intrattenimento, è un romanzo solo di intrattenimento". Ma anche se un prodotto è "solo di intrattenimento", non vuol dire che non ci si debba lavorare con attenzione. E poi intrattenimento "non significa niente": ci sono tanti modi in cui una cosa può intrattenere;
- lo stesso vale per **fun**, e per **immersion**, che ha un problema in più: è una parola passe-partout per cui si può dire che un gioco è "immersivo" nel senso generale del termine, ma poi non si sa davvero che cosa voglia dire, *immersivo in che senso*.

L'obiettivo didattico è che gli studenti **vadano oltre queste parole**: "un po' come quando alle medie vi dicono smettetela di dire che tutto è bello, perché ci sono vari gradi di bello, ci sono altri modi di dire bello". Bisogna **decostruirle**: quando si sta per usarle, chiedersi **che cosa si intende** con quella parola e cercarne una **più precisa**.

Ruggero chiede, in uno dei "rari casi", di **non fare domande adesso** su questa regola: ogni anno arrivano domande e commenti, e ci saranno lezioni apposite di tre ore, con strumenti per capire il perché. Ma da subito gli studenti devono abituarsi a evitare questi termini, sia perché si spera che abbiano termini più precisi, sia perché queste parole "portano con sé dei problemi agganciati" che si vedranno. "Non spaventatevi, non verrete sparati in un vicolo cieco di notte".

Una piccola gamification. Il divieto va visto come uno **stimolo**, "un po' come se stessimo gamificando la vostra esperienza", una sorta di user experience delle lezioni. Imponendosi di non usare queste parole, si noterà come migliora la capacità di **descrivere i giochi** e le proprie esperienze ludiche, e di essere più precisi. È un invito a "obbligarsi a essere più precisi e ad avere una mentalità più critica". La lezione di Ruggero sull'**involvement**, promette Daniele, sarà "fenomenale" e "vi aprirà la testa".

Approfondimento: anche i libri faticano a definire il "fun". La difficoltà che il divieto vuole far emergere è visibile anche nei testi di game design. Schell, in *The Art of Game Design*, prova a definire il *fun* e propone "piacere con sorprese" (*pleasure with surprises*), ammettendo subito che è strano poter usare una parola per tutta la vita, sapendo con certezza che cosa significa, senza riuscire a esprimerla chiaramente quando qualcuno lo chiede; e invita a mettere alla prova la definizione con controesempi. La sua lente sul *fun* (*The Lens of Fun*) si limita, non a caso, a chiedere quali parti del gioco siano *fun* e *perché*. Anche *immersion* ha molte letture: in *21st Century Game Design*, Bateman e Boon la descrivono, tra l'altro, come la capacità di "perdersi" nel mondo artificiale presentato dal gioco, legata alla sospensione dell'incredulità e ai giochi di mimesi. Sono significati diversi sotto la stessa parola: proprio il motivo per cui, in un documento di design, conviene dire quale si intende.

2.1.17 Chi crea, chi gioca, chi facilita

Un'altra distinzione importante è quella tra **chi crea**, **chi gioca** e **chi facilita**. Spesso, quando si fa un gioco, lo si fa provare di persona a chi gioca. Ma "la norma dei giochi pubblicati non funziona così": quando si pubblica un gioco non si vede chi lo compra e chi lo gioca, e non si sa se qualcuno, a casa propria con i propri amici, segue le regole alla lettera e interpreta correttamente quello che il designer aveva descritto. Quindi già in fase di progettazione bisogna tenere a mente **chi sta facendo game design e chi sta giocando**.

E nei giochi a evento, nei giochi pervasivi, nei giochi di ruolo da tavolo e dal vivo (LARP) c'è un terzo pezzo del puzzle, "importantissimo": **chi facilita**. La domanda da farsi è: **il regolamento è scritto per chi gioca o per chi deve facilitare il gioco?** Bisogna abituarsi a pensare a tutte le persone coinvolte nell'esperienza.

2.1.18 Playtest e iterazione

2.1.18.1 Nessun gioco nasce perfetto

L'ultima voce del glossario è "un'altra cosa importantissima": **playtest** e **iterazione**. Il corso è **pragmatico**: "specialmente da parte mia", dice Ruggero, non si vedranno teorie "messe lì per fare accademia sul

game design nelle nostre torri d'avorio". Ci saranno teorie e framework, ma tutti estremamente pragmatici, perché l'obiettivo non è solo capire i giochi, ma **saperli fare e iterare**.

Playtest e iterazione sono **inevitabili** nel game design, perché **nessun gioco nasce perfetto**, soprattutto perché le persone sono diverse: un gioco perfetto per una persona può essere incomprensibile per un'altra. Chi progetta giochi per altre persone deve capire, dopo averli fatti provare, **che cosa cambiare, che cosa sistemare, che cosa aggiungere, che cosa togliere**. Ci saranno lezioni su playtest, prototipazione, raccolta dei feedback, metodi etnografici. Ma bisogna abituarsi già ora all'idea che **non esiste la consegna definitiva**. A un certo punto i task vanno consegnati per forza di cose, ma, come diceva Daniele all'inizio, alla fine del corso gli studenti avranno un **prototipo**, "che sarà un prototipo", e gli strumenti per portare avanti il lavoro anche dopo.

2.1.18.2 L'aiuto dopo il corso

Non solo gli strumenti: anche l'aiuto dei docenti. Daniele racconta di aver portato a un editore, a Play[20], uno dei progetti realizzati l'anno precedente da due studenti, un gioco di ruolo in solitario ("un *solo game* RPG molto, molto bello"). Se gli studenti vorranno trasformare il proprio lavoro in un prototipo vero e proprio, possono contattarlo: lo fa "per lavoro, ma anche per passione". Gestisce anche una community per aiutare i designer "un po' depressi" a rimettersi a progettare.

Quindi: "non sentitevi persi se durante la fase di iterazione vi sentite persi, è normale".

2.1.18.3 Il dizionario comune e il production dictionary

Daniele riprende un punto detto "benissimo" da Ruggero e che secondo lui non è stato sottolineato abbastanza: il **dizionario comune** serve anche a **non perdersi** mentre si itera. Se da un'iterazione all'altra i documenti usano termini completamente diversi, e non c'è un dizionario di base, ci si perde.

Nel videogioco, una delle primissime cose che un designer fa quando entra in un'azienda è il **production dictionary**: si annota, in un file privato, tutte le parole che usano i colleghi, per allinearsi al dizionario comune dell'azienda, perché lì su un prototipo possono lavorare anche cento persone. Nel gioco da tavolo non si fa molto, perché il mondo dell'analogico è "talmente più piccolo"; ma serve comunque imparare un **dizionario minimo**, dai docenti e dai professionisti. E il corso ne darà una parte, entrando nel dettaglio di alcuni termini e framework.

L'esempio del feedback. Perché serva, lo mostra un esempio. Si testa un gioco e ci si annota come feedback: "questo gioco deve essere più divertente". Che cosa vuol dire? Come lo si migliora? Quella frase non dà elementi utili. Se invece si scrive che deve essere **più adrenalinico**, **più frenetico**, **più rilassato**, **più cozy**, o **più bilanciato matematicamente**, si sa dove andare a lavorare.

Approfondimento: la regola del ciclo. Il principio "nessun gioco nasce perfetto" ha un nome preciso in *The Art of Game Design*: è la **Rule of the Loop**, la regola del ciclo. "Più volte provi e migliori il tuo design, migliore sarà il tuo gioco". Schell precisa che non è una lente, cioè un punto di vista, ma "una verità assoluta", senza eccezioni: ci saranno momenti in cui si cercherà di convincersi che "questa volta il design è così buono che non serve testarlo", e ogni volta se ne pagherà il prezzo. Ed è interessante la ragione per cui la regola pesa di più sul digitale: secondo Schell il tempo e il denaro necessari per provare e correggere un sistema sono molto maggiori nei videogiochi che nei giochi tradizionali. Per un gioco di carte o da tavolo, con il tempo per provarlo e cambiarlo più e più volte, il consiglio è semplicemente di farlo. Il gioco analogico è, da questo punto di vista, la palestra ideale dell'iterazione.

[20] Probabilmente *Play – Festival del Gioco* di Modena; la trascrizione non permette di esserne certi.

2.1.19 Tabletop Simulator

2.1.19.1 Il software di settore

"Last but not least": il corso usa un software, **Tabletop Simulator**, che si compra su **Steam** e costa tra i **10 e i 20 euro** ("aspettate che vada in sconto, va in sconto ogni due per tre"). Daniele spiega perché:

- è **il software di settore** del gioco da tavolo, quello che i publisher usano per i prodotti digitali, interni ed esterni;
- essendo su Steam, permette di fare **playtest** e di giocare con gli amici di Steam. Daniele stesso sta testando lì il proprio gioco;
- funziona anche su Mac: Windows o Mac, nessun problema.

Ci sarà una lezione apposita, di Ruggero, sulla creazione del proprio **modulo** di gioco.

Che cosa non è. Daniele chiarisce subito un "fraintendimento gigantesco", soprattutto per chi ha già esperienza: Tabletop Simulator **non richiede programmazione** e **non è un software per trasformare i giochi da tavolo in videogiochi**. È un software che **simula dei componenti fisici su un tavolo**: serve per giocare i board game in digitale. Simula anche la fisica: c'è perfino il comando per **ribaltare il tavolo** ("non usatelo ora").

La partnership mancata. Da anni Daniele prova a fare una partnership con gli sviluppatori: due anni prima ha contattato l'azienda per acquistare delle chiavi per gli studenti, dato che il software si usa nei corsi. L'azienda ha risposto che non fa accordi commerciali. "Credo sia la prima azienda nella mia vita che dice che non vuole che io le dia dei soldi". Gli studenti dovranno quindi provare ad acquistarlo da sé; se i docenti riusciranno a darglielo, bene.

La domanda di Gianmarco. Gianmarco segue le lezioni da un computer aziendale "super chiuso", su cui non può installare Steam: deve usare Tabletop Simulator durante le lezioni? Ruggero risponde che per la lezione dedicata è meglio poter seguire e rifare in diretta. Ma il software non sarà richiesto necessariamente per il project work: un gruppo potrebbe fare un gioco pervasivo o un gioco di ruolo, non un gioco da tavolo. Essendo un software di settore, è comunque bene conoscerlo. Ruggero aggiunge che le versioni non ufficiali che circolano non permettono di giocare in molti, e quindi non servono per playtestare o giocare insieme.

2.1.19.2 Una dimostrazione

Daniele mostra il software aperto da Steam:

- si può creare una partita **single player** o **multiplayer**; per giocare tra compagni si crea un server multiplayer e ci si aggiunge come amici su Steam;
- si dà al server un nome, un numero di giocatori, eventualmente una password; creato il server, si apre la scelta dei giochi;
- ci sono giochi gratuiti, giochi da acquistare (DLC, non obbligatori) e soprattutto un **Workshop** "ampissimo", con decine di migliaia di giochi caricati dagli utenti: "se c'è un board game che conoscete, quel board game è su Tabletop Simulator, giocabile dal Workshop". L'esempio mostrato è l'open beta ufficiale del gioco da tavolo di *Slay the Spire*[21]: molti publisher, per farsi pubblicità, mettono il proprio gioco in demo o giocabile in digitale;
- c'è un **tutorial** iniziale che spiega come muoversi sul tavolo; con *Esc* e *Help* si trovano i comandi da tastiera.

[21] La trascrizione dice "Slade Spire".

2.1.19.3 Il primo task

Il primo (e per ora unico) task è questo: **entro il 10 ottobre**, giorno della lezione di Ruggero su Tabletop Simulator, giocare **tre o quattro volte in totale**, online, tra compagni di corso o con amici. Non si tratta di imparare a creare moduli, ma di prendere la mano **lato giocatore**: aprire il software, impararne i comandi, giocare. "Per me potete anche andare a prendere domino o mahjong" ("mahjong è bellissimo tra l'altro", aggiunge Daniele, che sta facendo un videogioco sul mahjong), o il pachisi o il go. Gli anni precedenti si chiedeva a ciascuno di proporre ai compagni un gioco da tavolo che apprezzava. È anche un modo per conoscersi. Ruggero spiega il perché: usarlo per giocare è facile e intuitivo, usarlo per creare moduli è più complesso; ma chi arriva al 10 ottobre avendo già aperto il software e giocato, troverà più intuitivo anche creare un modulo. E l'avvertimento: "non riducetevi all'ultimo, non fate una partita il 9 sera".

2.1.19.4 Tabletopia e altri vantaggi

Tancredi racconta di aver provato a caricare un suo gioco su **Tabletopia**. "Questo è un vaso di Pandora", risponde Daniele. Tabletopia è molto usato, ma secondo lui il suo strumento di creazione dei moduli è pessimo, ed è il motivo per cui la maggioranza dei moduli ufficiali dei publisher è su Tabletop Simulator. Tabletopia ha un vantaggio vero, è **gratuito**, ma ha scelte di "qualità della vita" poco felici.

Tabletop Simulator ha invece un vantaggio "fenomenale" che si tende a sottovalutare: essendo su Steam, **il modulo si pubblica direttamente sul Workshop di Steam**; si possono invitare gli amici a iscriversi e giocare con loro, purché abbiano un account Steam. Rispetto ad altri software da imparare, costa "tra i 10 e i 20 euro, ragazzi: si parla proprio di spiccioli". "Io con Tabletop Simulator ci ho playtestato la vita".

Altre caratteristiche utili:

- si possono creare **moduli non elencati** (*unlisted*), perfetti da passare a un publisher prima che il gioco sia pubblicato: non sono pubblici, e nessuno può rubare i materiali;
- il software ha una grafica datata ("*World Art 1998*", scherza uno dei docenti), ma lo stanno portando su **Unity** con molti investimenti e aggiornamenti; ascolta la comunità e introduce funzioni utili anche per chi fa design;
- il suo server Discord è molto frequentato, e c'è un canale YouTube dedicato solo al design di giochi di carte[22] con un server Discord dove si fanno playtest: il 99% è su Tabletop Simulator;
- molti editori, quando si propone loro un gioco da tavolo, **chiedono se c'è un modulo per Tabletop Simulator**.

2.1.20 Le domande a Daniele: il tema del project work

Daniele ha "un minuto" per rispondere alle domande sul corso prima di dover andare. Uno studente chiede, "giusto per curiosità", se il project work avrà un tema o sarà libero.

- C'è un **mandate**, che i docenti propongono ogni anno e aggiornano leggermente; verrà presentato il **25 settembre**. "Non facciamo spoiler". Conterrà quello che si chiede di fare e quello che non si chiede: dei **must**, dei **must not** e dei **nice to have**.
- Verrà chiesto di lavorare su un'**idea originale** che non si è portata al corso: originale nel senso di **creata apposta**, non nel senso di "mai vista prima".
- L'idea va **partorita insieme**. Altri anni alcuni studenti hanno provato a imporre la propria idea al gruppo, "e l'esperienza è stata pessima per tutti". Quindi: **non imponete le vostre idee, fate brainstorming con gli altri**. Daniele, che fa "molto spesso brainstorming", darà strumenti per farlo: nessuno arriverà impreparato a quella lezione.

[22] La trascrizione ne riporta il nome come "Distraction Maker"; non è verificabile.

Ruggero aggiunge che il mandate è **abbastanza flessibile** perché tutti possano fare qualcosa che li interessi: "l'ultima cosa che vogliamo è forzarvi a lavorare su qualcosa che non vi piace". Bisogna lavorare su ciò che piace e per cui si ha uno stimolo personale, anche nella **tipologia** di gioco: nessuno sarà obbligato a fare un gioco di ruolo o un board game, e la lezione sui casi studio mostrerà quante forme diverse di gioco analogico il corso accetta.

L'unica cosa da **non fare**, "perché è contraddittorio per questo corso", sono i **giochi grandi**: "lo chiediamo per voi". Niente progetti mastodontici[23], niente "cose alla *Dungeons & Dragons*". Si può fare un gioco di ruolo, ma con un'esperienza "meno corposa": ci sono tanti giochi di ruolo da 15 o 20 pagine, "bellissimi, fichissimi", che si possono leggere online. Il consiglio è farsi una cultura anche su **itch.io**, "uno dei portali più importanti del gioco di ruolo internazionale", di cui si parlerà anche nella parte di ricerca e sviluppo.

Poi Daniele saluta ("noi ci vediamo venerdì a lezione") e lascia la conduzione a Ruggero.

2.1.21 Le domande finali con Ruggero

2.1.21.1 Le regole delle domande

Ruggero spiega che cosa vuol dire "stare con lui". Alla fine di ogni sua lezione ci saranno **almeno 20–30 minuti di domande** libere, *free form*. Vuole che fare domande sia **normalizzato**: "io gestisco malissimo l'*horror vacui* e divento super cringe molto alla svelta se non ci sono domande in giro". Sono benvenute le domande sulla lezione appena fatta, ma non solo: il corso è crossmediale, e anche le domande che vengono da **divagazioni**, da parti laterali, "fatele". Quello che sembra scontato o banale a uno studente può non esserlo per il docente, e viceversa; e può essere utile anche agli altri. "Se vi sembrano stupide, fatele il doppio". "Magari sembro incazzato, ma vi giuro sono rilassatissimo. Non mordo, magari abbaio un po'".

2.1.21.2 Come approcciarsi a Steam e Tabletop Simulator

Rita non ha mai usato né Steam né Tabletop Simulator e chiede come approcciarsi, e se c'è modo di mettersi d'accordo tra compagni su Discord, magari con un sondaggio sui giochi, per formare gruppi di gioco. Ruggero conferma: sul Discord dell'accademia c'è un **canale apposito per il corso**, ed è degli studenti. Lo possono usare per organizzarsi a giocare, per chiedere a un compagno di prestare un gioco, per domande tecniche (come si scarica, come si installa). "Sarebbe proprio ideale che vi parlaste e vi organizzaste".

Anche perché, aggiunge con un "piccolo spoiler" ("nessuno vi ha detto niente"), se fossero i docenti a organizzare i gruppi di gioco, il task lo starebbero facendo loro. Bisogna **abituarsi a organizzare le sessioni di gioco**, perché arriveranno i task sul **playtest** e su come organizzarlo, e a quel punto si sarà "meno disperati". Sembra scontato, ma saper **organizzarsi per giocare in autonomia** "vi sarà utilissimo come skill in questo settore, ve lo giuro".

2.1.21.3 Il bilanciamento

Angelo confessa "una paranoia grossissima": ci saranno lezioni sul **bilanciamento**? "Ma certo". Sono proprio le lezioni di Marta sul development: il bilanciamento è di solito un tema che tocca **i developer più dei designer**. L'anno precedente c'era una sola lezione con Marta, e i docenti si sono resi conto che serviva andare ancora più a fondo: per questo quest'anno le lezioni sono due. "Non è una paranoia, è una legittima domanda": Ruggero insegna da anni, vede altri corsi, parla con altri docenti e studenti, e "mi incazzo spesso" perché molti corsi di game design **non fanno bilanciamento**. I docenti volevano invece un corso che coprisse tutte le parti del game design: non solo l'idea del gioco, ma il bilanciamento, la produzione, i playtest, l'iterazione, la scrittura dei regolamenti.

[23] La trascrizione dice "non fate delle cose da gnome": probabilmente il nome di un gioco da tavolo molto grande e complesso, non ricostruibile con certezza.

2.1.21.4 Si può giocare con i docenti?

Una studentessa chiede se si può giocare con i docenti. Per motivi etici, durante il corso questo è limitato alle parti di **valutazione** dei giochi e di **tutoraggio**. Dopo il corso, "nessun problema, almeno con me".

2.1.21.5 Dove si trovano i giochi su Tabletop Simulator

Ruggero mostra dove si trovano i giochi: nella propria libreria di Steam, nella pagina di Tabletop Simulator, alla voce **Workshop**, dove ci sono "più o meno 36.000" giochi. Una studentessa scopre intanto che l'app di Steam per telefono è solo una *companion app*: per registrarsi e giocare serve il PC.

Per provare, chiede agli studenti un titolo:

- *Keep the Heroes Out*: non compare, "non tutti i giochi ci sono, perché molti publisher non li mettono"; forse, essendoci già una versione del gioco su Steam, il modulo è stato tolto;
- *Magic: The Gathering*: "almeno 50 moduli", anzi "123 pagine";
- *Bang!*: tre moduli.

Bisogna fare attenzione: alcuni moduli sono in lingue specifiche ("magari il primo è in russo"), alcuni hanno già l'espansione, altri sono solo il gioco base. Conviene sfogliare e non fermarsi al primo risultato. E nel Workshop si trovano anche cose molto particolari: giochi fuori produzione, giochi vecchi, giochi ancora in prototipo.

2.1.21.6 I ruoli nei team e le dinamiche tra le persone

Uno studente torna sulla slide delle figure: nelle realtà piccole molti ruoli hanno "il doppio o triplo cappello". Quali ruoli restano distinti, e come si creano le sinergie tra persone con obiettivi diversi?

Ruggero premette che l'ideale "platonico" è un ambiente di lavoro costruttivo, basato sulla fiducia e "il più *safe* possibile"; ma nell'industria non sempre è così, e le strutture variano moltissimo: alcuni publisher sono molto verticali, altri hanno ruoli già solidificati (se un publisher ha già i suoi developer e non sta crescendo, entrarci come developer è praticamente impossibile). "Se prendiamo due aziende in Italia, vediamo due strutture interne completamente diverse". Non esiste una scorciatoia per farsi assumere da un editore.

Poi prende "l'esempio più comune di tutti", la media dei giochi da tavolo pubblicati (figura 2.1.3):

- il **game designer** è una figura **esterna**, con un **contratto a royalty** con il publisher;
- la sua figura di riferimento è il **publisher** stesso, e la conversazione designer–publisher è "molto calda";
- se il publisher ha dei **developer**, il developer entra in questa conversazione: designer e developer collaborano molto, e internamente developer e publisher collaborano molto. Spesso, nei publisher piccoli, è il publisher a fare le veci del developer;
- **graphic design** e **illustrazione** tendono a parlare molto con il publisher e **molto poco con il designer**. A meno che non si tratti di uno studio indie molto piccolo, è difficile che il designer sappia anche solo chi è l'illustratore prima che il gioco esca.

2.1.21.7 Fare da developer di sé stessi?

Rita chiede in chat se un designer può fare anche da developer del proprio gioco, o se Ruggero lo sconsiglia. In generale **lo sconsiglia**, per lo stesso motivo per cui si sconsiglia in altri media: i **darling**. Il designer ha delle meccaniche a cui è affezionato, e il developer a volte dovrà **tagliarle**; è molto difficile, se si è "gestori di sé stessi", tagliare cose che si vogliono nel gioco. "È un po' come lasciare fare il lavoro di editing allo scrittore che ha già scritto il libro": può rivedersi, certo, ma difficilmente cambierà cose che farebbe bene cambiare. Nelle lezioni con Marta si vedrà perché, e alla fine del corso "non avrete dubbi nemmeno voi" nel separare designer e developer.

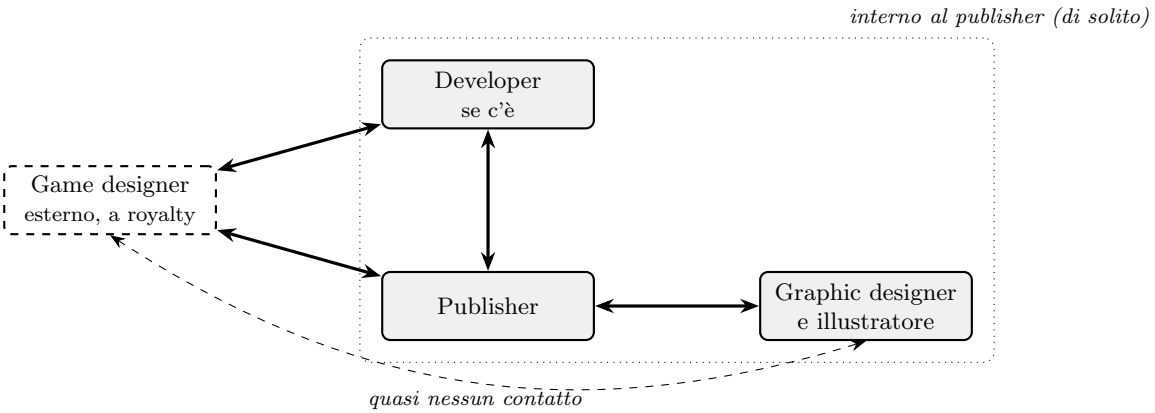


Figure 2.1.3: Le relazioni tipiche intorno a un gioco da tavolo pubblicato, secondo la risposta di Ruggero. Le frecce spesse indicano una collaborazione stretta e frequente; quella tratteggiata, un contatto quasi assente. Nei publisher piccoli il publisher fa spesso anche da developer.

Ma tutto dipende dallo **scopo del progetto**: se si fa un gioco in tre, qualcuno quel lavoro deve farlo per forza, e magari il designer fa anche il developer, compensando con **più playtest**. "Il mondo attuale e il mondo ideale sono due cose molto diverse, e lo vediamo non solo dai giochi ma dalla politica", ma "non entriamo in questo vaso di Pandora".

2.1.21.8 Il ruolo indispensabile e la pubblicazione

Lo studente di prima ne trae una conclusione: senza game designer il gioco non esiste. "Quello sicuramente". Ma Ruggero coglie l'occasione per approfondire un punto che forse non era passato: **fare design di un gioco per pubblicarlo non è l'unica opzione possibile**. Un gioco non è "meno gioco" se non è pubblicato da un editore. È legittimo mettere un gioco su itch.io in **print and play**, fare un gioco a evento senza venderlo, fare un *murder party*, una escape room, un board game per una persona amica: "quell'esperienza lì non è meno gioco". Dipende dall'obiettivo del progetto.

Ruggero ha lavorato molto anche con le **scuole**, dove il publisher non esiste: è un gioco per le classi, e la figura più vicina è l'editoria scolastica, "che però non fa il publisher". Ci sono compromessi diversi sulla materialità e su come il gioco si presenta, che si vedranno nella prototipazione e nei casi studio. Quindi l'unica figura che deve esserci sempre è **chi fa game design**, anche se magari non sa di chiamarsi game designer e non ha mai sentito parlare di game design: "questo non la rende meno game designer".

Un libro consigliato. Ruggero fa anche mentoring e gestisce gruppi di settore, e spesso gli chiedono che cosa leggere per imparare il game design. Non è il tipo che obbliga a leggere libri ("non vi obbligherò a leggere niente"), ma ce n'è uno che consiglia sempre, ed è il primo che ha scelto di leggere: ***Rise of the Videogame Zinesters*** di **Anna Anthropy**[24]. È un libro un po' datato, ma ancora reperibile; parla soprattutto di videogiochi, ma "per l'80%" il contenuto si applica al gioco e al game design in generale. È un libro sulla **democratizzazione del gioco**: il gioco come strumento per parlare di sé e come **medium legittimo**. Come esistono film drammatici, tristi, intensi, possono esistere giochi su tematiche delicate, e se ne vedranno nei casi studio. Molti giochi fatti con questa direttiva non hanno un publisher, che tende a pubblicare ciò che dà un ritorno commerciale.

Commerciale e personale. "Io spero che voi non farete giochi solo per scopo commerciale". Certo, servono un tetto sulla testa, il cibo in tavola, un posto in cui dormire, e così si finisce per fare giochi che non sono il proprio gioco dei sogni. Spesso è un mix: Ruggero ha fatto giochi commerciali "per pagarmi l'affitto" e giochi che voleva fare e su cui ha perso soldi, "sapendolo proprio che sarebbero stati un fallimento". I

[24] La trascrizione dice "Anna Antrofi" e "Rise of the Videogames in Esters".

giochi, come i libri e le canzoni, si fanno per molti motivi, e quelli commerciali di successo non sono più importanti degli altri. Tra i giochi della sua slide c'erano il gioco da tavolo ufficiale di *Cyberpunk*[25] e *Marvel United*, su IP grosse e famose, ma anche giochi fatti di "una candela e una busta su un tavolo". E se gli si chiede quale preferisce in termini di contenuto, non sono quelli sui supereroi.

2.1.21.9 Il gioco di ruolo in Italia

Nicola, che è interessato soprattutto al gioco di ruolo e conosce già persone del settore, ha "una leggera paura": che chi studia game design finisca per fare il **freelance** a vita, "perché non ci sarà mai un'occupazione per bene", o il **copywriter**. Conosce un ragazzo che ha lavorato in quel mondo e che ora fa altro.

Ruggero risponde che la maggioranza dei giochi nel suo portfolio sono giochi di ruolo, quindi il tema lo tocca da vicino, e che in Italia quello che dice Nicola "è semplicemente vero". Molti publisher piccoli hanno una persona di riferimento che copre più ruoli: impagina, fa i playtest, fa un po' di development. È "un settore economicamente completamente disastrato", per motivi culturali che non apre: oggi in Italia è praticamente impossibile **vivere di giochi di ruolo**. Chi ci lavora lo fa nel tempo libero, al di fuori del lavoro principale, oppure coprendo molte altre cose: molti publisher italiani di giochi di ruolo pubblicano anche titoli esteri e localizzano, non fanno solo cose originali. "È un grosso calderone". Poi si ferma: "questo è un *rant*", e "i *rant* tornano sempre", ma non vuole che gli studenti escano dalla prima lezione "con la disperazione nel cuore". Ci sarà spazio per parlarne quando avranno gli strumenti per capirlo, "per non farvi abbattere".

2.1.21.10 Che cosa bisogna sapere già

Uno studente chiede se ci sono nozioni che dovrebbero già avere e che possono recuperare: per esempio, sente parlare di giochi "German" o "American" e non sa che cosa siano. Ruggero spiega l'organizzazione del corso:

- le **prime due lezioni** sono di fondamenta. Questa era "dall'alto": come è strutturato il corso, le tempistiche;
- la **prossima lezione** è di tre ore di soli **casi studio**, "a manetta, a raffica", divisi per categorie e tipologie, con ragionamenti su categorie e generi di gioco: lì si vedranno German, American "e anche altre più utili";
- nel resto del corso le terminologie specifiche sono inserite nelle lezioni dedicate: il **paragame** nelle lezioni sul paragame, il **modello matematico** nelle lezioni di Marta sul development.

Per ora agli studenti si chiede di sapere quello che si è visto oggi: meccaniche, dinamiche ed estetiche; le figure professionali, designer e developer; e soprattutto **game e play**.

2.1.21.11 Game e play: i quattro quadranti

"Specialmente *game* e *play*", insiste Ruggero: è più difficile di quello che sembra, perché l'italiano non abitua a separare le due cose, ed è possibile averle usate per anni come sinonimi. Nella sua esperienza con chi si avvicina al design è "uno dei punti più critici". Spesso i commenti che si fanno sui giochi giocati sono commenti sul *play* applicati al *game*: "il gioco non mi è piaciuto perché, giocando con Tizio, è successo questo"; "il gioco mi sembra sbilanciato perché Tizio al tavolo ha fatto questo".

Separare *game* e *play* serve a capire quali giochi sono progettati bene e quali male, e soprattutto a riconoscere i **quattro possibili incroci** (figura 2.1.4): un gioco progettato bene che piace giocare; uno progettato male che non piace; uno progettato male che piace; uno progettato bene che non piace. "Non è importantissimo in generale nella vita, ci sono cose più importanti", ma per chi fa game design lo è: "a volte vi chiederanno di lavorare su un gioco che non vi piace giocare, e quel gioco deve essere fatto bene lo stesso".

[25] La trascrizione dice "Cyberpunk del 1977": quasi certamente un gioco legato a *Cyberpunk 2077*.

	*mi piace (*play*)*	*non mi piace (*play*)*
*bene (*game*)*	**progettato bene** e **mi piace** giocarlo	**progettato bene** ma **non mi piace** giocarlo
*male (*game*)*	**progettato male** ma **mi piace** giocarlo	**progettato male** e **non mi piace** giocarlo

Figure 2.1.4: I quattro incroci tra la qualità del *game* (come è progettato) e l'esperienza di *play* (se piace giocarlo), descritti da Ruggero. Per un designer i quadranti "scomodi" sono i due fuori diagonale: un gioco può piacere pur essendo progettato male, e va saputo progettare bene anche un gioco che non piace.

La domanda di Tancredi. Tancredi chiede conferma: il *play* non è il "giocare senza regole", ma può essere anche il giocare a un gioco? Sì: il *play* è l'**atto**, e può essere fatto con un *game* o senza. E un *game* può esistere senza che nessuno lo giochi: gli scacchi non sono meno scacchi perché in quel momento nessuno ci gioca. E se, giocando, una persona applica le regole in modo diverso? Allora **non si sta giocando quel gioco**.

2.1.21.12 House rule: il caso di Monopoly

Qui Ruggero fa un po' di *foreshadowing* su una lezione di Daniele: il discorso degli **hack**, delle **house rule**, delle **mod**, cioè di come modificare un gioco esistente. Il punto è che **un gioco modificato da una house rule è un altro gioco**: cambiando le regole cambiano necessariamente anche le **dinamiche** che ne derivano, e quindi l'esperienza. Si immagini *Monopoly* in cui ogni turno si possa, volendo, prendere gratis dei soldi dalla banca: tutto il resto è identico, eppure il gioco cambia.

E questo è talmente interiorizzato che ci sono regole che si applicano "senza neanche rendercene conto". Ruggero chiede quanto duri in media una partita a *Monopoly*. Gianmarco: "quattro ore"; altri: "un pomeriggio"; un altro: "finché uno dei giocatori non aggiunge la regola: ora vince chi…". In realtà, **seguendo le regole scritte**, una partita non dura mai più di **un'ora e venti, un'ora e mezza**. La domanda che fa sempre, e la risposta è sempre la stessa: perché si pensa che *Monopoly* duri tantissimo? Qual è la regola che nessuno rispetta?

Uno studente indovina: **le aste**. Secondo il regolamento, se chi finisce su un terreno non lo compra, il terreno **va all'asta** tra i giocatori, e chiunque può aggiudicarselo. Molti invece giocano come se finire su un terreno desse solo la *facoltà* di comprarlo o di metterlo all'asta[26]. Con la regola applicata, di aste se ne fanno moltissime: all'ottavo turno ci sono già otto terreni assegnati, al ventesimo venti, la gente va in bancarotta velocissimamente e la partita dura un'ora e mezza. Anche le partite competitive di *Monopoly*, nota Ruggero, durano un'ora e venti, un'ora e mezza.

Uno studente obietta che giocava con le aste e durava comunque quattro ore. "Avete dei nemici con l'analisi paralisi", ride Ruggero; e cita un'altra house rule tipica dell'infanzia: genitori che danno soldi per non far andare in bancarotta. Ma **la bancarotta è parte del gioco**: se la si toglie, *Monopoly* non regge. Sul tema che *Monopoly* vuole trattare, e sul suo antenato (*The Landlord's Game*), "non apriamo adesso un altro vaso di Pandora".

Approfondimento: Monopoly, la durata e le house rule nei libri. L'esempio di Ruggero si ritrova quasi identico in *The Art of Game Design*. Parlando del bilanciamento tra partite brevi e lunghe (*Short vs. Long*), Schell scrive che *Monopoly*, giocato secondo le regole ufficiali, "finisce spesso in circa novanta minuti", ma molti giocatori trovano quelle regole troppo dure e le modificano, distribuendo premi in denaro

[26] Più precisamente, nel regolamento ufficiale, se il giocatore che finisce su una proprietà libera decide di non comprarla, la banca la mette all'asta e tutti i giocatori possono fare offerte, compreso chi l'ha rifiutata. L'asta quindi non è una scelta del giocatore, ma la conseguenza obbligata del suo rifiuto.

e allentando i vincoli sull'acquisto delle proprietà: con l'effetto collaterale di far durare la partita molto di più, "tipicamente tre ore, o anche di più". Schell classifica le **house rule** come regole create dai giocatori in risposta a un difetto percepito dopo qualche partita. Anche *Challenges for Game Designers*, nel capitolo sull'aggiungere e togliere meccaniche, parte dallo stesso gioco: il *Parcheggio gratuito*, che per le regole originali non fa succedere nulla, mentre quasi tutti giocano con una house rule che dà un bonus a chi ci finisce sopra. Le due fonti confermano la lezione: una piccola regola cambiata cambia le dinamiche, e le dinamiche cambiano l'esperienza.

2.1.21.13 Il fine del game è il play?

Un altro studente commenta: va bene la distinzione, ma il fine del *game* è il *play*; quello che si vede è sempre l'effetto. Ruggero è d'accordo: di solito, se si fa un gioco, lo si fa per farlo giocare, come si scrive un libro per farlo leggere (anche se esistono, ed "è già stato esplorato", libri scritti per non essere letti e giochi per zero giocatori). Ma la distinzione serve a una cosa precisa: **separare, nel playtest, i feedback sul gioco dai feedback sul gruppo di gioco** e sulle sue dinamiche. Se una persona esce frustratissima da un playtest, non è necessariamente colpa del gioco; se una persona dice "questo è il gioco migliore del mondo", potrebbe non essere merito del gioco ma del gruppo. Capire a che cosa si riferisce un feedback serve a capire **dove iterare** il prototipo.

Lo studente osserva che l'area è sfumata: alcuni giochi si basano proprio sull'avere le persone giuste al tavolo. Un gioco di **negoziazione** con giocatori abituati ai giochi "multi-solitario" diventa ingiocabile; e nei giochi a **economia chiusa** (cita *Container*[27]) un gruppo economicamente conservativo, che spende poco e non fa girare l'economia, fa durare la partita il doppio. Ruggero rilancia: e se il gioco dura il doppio, questa dinamica cambia la **percezione di peso** del gioco, le energie spese, quanto si è stanchi a fine partita. "Tutto è collegato. **I giochi sono ecosistemi di meccaniche**, e cambiandone una, le altre cambiano di conseguenza".

La parola vietata, alla prova. Lo studente aggiunge che ci sono giochi che ha trovato "molto sbagliati, ma che mi divertono". "Attenzione: non si può dire *divertimento*!", lo ferma Ruggero; nelle presentazioni aveva lasciato correre, "ma lì non vi avevo ancora letto la regola". Lo studente ci prova: "giochi che poi voglio **rigiocare**", anche se "sono evidentemente sbagliati, rotti". Ruggero apprezza, ma non vuole anticipare cose che meritano il loro tempo: se ne parlerà nelle lezioni sui framework e soprattutto in quella di **emotion design**, "forse la mia lezione preferita del corso". (Lo studente ammette che "forse alla fine si gioca per quello, per l'emozione".)

2.1.21.14 I giochi "comunicati male"

L'ultima domanda corposa riprende un'affermazione di inizio lezione: ci sono giochi belli ma comunicati male. Qualche esempio? "Mi basta girarmi per vederne 48", risponde Ruggero indicando gli scaffali alle sue spalle. Più che titoli, elenca le **ragioni**:

- **l'Italia localizza lentamente**. Basta guardare i premi del gioco dell'anno in Italia: spesso vanno a giochi usciti due o tre anni prima in Germania, Francia, Stati Uniti o Regno Unito, perché localizzare richiede i suoi tempi. Il mercato italiano è quindi molto dipendente dall'estero, "molto impaurito", e spesso aspetta che un gioco vada bene all'estero prima di portarlo in Italia; e alcuni giochi "perdono il treno";
- **il publisher originale chiude**: come si pubblica un gioco se non c'è più l'azienda con cui fare un contratto? A Ruggero è successo più volte;
- **il dipartimento commerciale italiano non sa comunicare il gioco**: si fa male un'edizione, la stampa vende pochissimo e non viene più ristampata;
- **la scala del mercato**: in Italia un gioco da tavolo di peso medio (un *eurogame*, un *German*) che vende 1.500 copie "è tanto"; all'estero se ne vendono magari 10.000. Con 10.000 copie in inglese ci

[27] La trascrizione cita anche un secondo titolo non ricostruibile ("Tokyo su Vigimarket").

sono i soldi per una campagna di comunicazione: un trailer, i recensori, i tutorial online. In Italia tutto questo è "ridotto all'osso", e la cosa si riflette su stampa e divulgazione: anche i canali per informarsi su un gioco prima di comprarlo sono pochi (se ne parlerà nella lezione su ricerca e sviluppo). Un gioco spesso non arriva al mercato non perché sia brutto, ma perché l'editore non lo sa comunicare, ne fa poche copie e va fuori produzione;

- **il tempismo**: party game progettati nel 2019 e usciti nel 2020, in pieno Covid, quando non si poteva giocare al tavolo, sono falliti tutti, e gli editori ci hanno perso. "Non potevano immaginarselo";
- **i temi caldi**: dopo l'invasione russa dell'Ucraina e la guerra in Palestina, alcuni editori non pubblicano o non localizzano giochi strategici e di guerra. Ruggero ha lavorato a ***Status***, un gioco sulla geopolitica che tocca, tra gli altri temi, l'invasione russa dell'Ucraina e il genocidio in Palestina, e alcuni editori che di solito localizzano i loro giochi (per esempio il partner di riferimento in Russia) non lo hanno localizzato per motivi politici.

Lo studente voleva capire che cosa voglia dire comunicare male: non metterci abbastanza energia, o comunicare una cosa per un'altra, "come il trailer sbagliato, che poi vai al cinema e ti trovi un altro film". "È un mix", e la domanda "era complessa e meritava più spazio".

2.1.21.15 I giochi di carte collezionabili e i saluti

L'ultima domanda ("posso rompere le balle un'ultima volta?") riguarda i **giochi di carte collezionabili**: funzionano con le stesse regole dei giochi da tavolo? Se ne parlerà alla prossima lezione: è proprio il motivo per cui la lezione sui casi studio dura tre ore, perché ogni genere di gioco ha le sue dinamiche e le sue regole, e anche il modo di pubblicare cambia molto tra un TCG, un gioco di ruolo da tavolo, un LARP e un gioco pervasivo.

Ruggero saluta: le email dei tre docenti sono all'inizio delle slide, la registrazione arriverà, "ci si vede questo venerdì".

Punti chiave del capitolo

- Il corso di **Analog Game Design** (seconda edizione) dura circa tre mesi, fino a dicembre, ed è tenuto da tre docenti principali: **Daniele** (narrative design, gioco di ruolo, fiction), **Ruggero** (game design, ricerca e sviluppo) e **Beatrice** (giochi pervasivi), con tre ospiti su RNG, development e game production.
- Lo scopo è formare **autori analogici** consapevoli: strumenti da usare anche fuori dal corso, non il "gioco dei sogni". Il corso è **intersezionale** (nessuna lezione dedicata a un solo tipo di gioco), **cross-generazionale**, **pratico e adattabile**, basato sulle **domande** (nessuna domanda è stupida), sul **feedback** invece che sul voto e sul *fail forward*.
- Il **project work** si fa in tre gruppi da tre, formati dai docenti; pitch il 12 ottobre, sviluppo nelle due pause, tutoraggi il sabato, consegna il 4 dicembre, presentazione l'11 e giornata finale con gli editori il 14. Si valutano documentazione, presentazione, autovalutazione, crescita e atteggiamento.
- **Niente intelligenza artificiale** nel corso: i publisher rifiutano pitch e prototipi con immagini generate, i regolamenti scritti dall'IA descrivono la stessa meccanica in modi diversi, e soprattutto chi la usa perde l'occasione di imparare. Un prototipo può essere fatto a matita.
- Le **figure dell'industria**: game designer, game developer, editor, publisher, sales e marketing, graphic designer, illustratore, sensitivity reader (che non è un censore) e producer. Nelle realtà piccole una persona copre più ruoli.
- Nell'analogico il **game designer** progetta l'esperienza creando le meccaniche; il **game developer** fa da **mediatore** con il publisher (bilanciamento, modelli matematici, adattamenti). Pretendere che il proprio gioco non venga modificato è un *red flag*.

- Il **marketing** non decide che cosa pubblicare ma come; è un'azione, ed esiste anche se lo si ignora.
- In **Italia** il gioco è ancora visto come infantile, il mercato punta sulla localizzazione e investe poco su giochi e designer interni; all'estero (soprattutto dalla Germania in su) ci sono tradizione adulta, premi di prestigio e developer interni.
- **Game** è l'artefatto (le regole, il sistema, l'oggetto); **play** è l'atto di giocare. Si fanno giochi, non giocattoli. Separarli serve a distinguere i feedback sul gioco da quelli sul gruppo e a riconoscere i quattro incroci tra "progettato bene/male" e "mi piace/non mi piace".
- **MDA**: meccaniche (regole scritte), dinamiche (regole in azione), estetiche (esperienza, non grafica); nel **DPE**: design, play, experience.
- **Parole vietate** in ogni materiale scritto: *fun*, *immersion*, *entertainment* (divertimento, immersione, intrattenimento). Bisogna trovare parole più precise.
- Distinguere **chi crea, chi gioca e chi facilita**; nessun gioco nasce perfetto: **playtest e iterazione** sono inevitabili, e un **dizionario comune** aiuta a non perdersi.
- **Tabletop Simulator** è il software di settore per giocare (non creare videogiochi) i giochi da tavolo in digitale. Primo task: giocarci tre o quattro volte entro il 10 ottobre.
- Una **house rule** crea un altro gioco: *Monopoly* con le aste applicate come da regolamento dura circa un'ora e mezza.

Chapter 2.2

I casi studio: giochi pervasivi, giochi di ruolo dal vivo, giochi di ruolo da tavolo e giochi da tavolo

Questa è la **seconda lezione** del corso di **Analog Game Design**, e mantiene la promessa fatta alla fine della prima: è una lezione di tre ore dedicata quasi interamente ai **casi studio** (*case studies*). I docenti la definiscono "un overview", una panoramica "a volo d'aquila": "voleremo in tutti i settori ludici analogici e guarderemo che cosa c'è". Non si scende nei dettagli tecnici dell'analisi (ci saranno lezioni apposite, soprattutto quella di **ricerca e sviluppo**), ma si apre la "diga" degli esempi: decine di giochi, eventi, esperienze, movimenti e manifesti, scelti per mostrare quante cose esistono, quanto sono diverse tra loro e quanto i confini tra i generi siano porosi.

La lezione si svolge **online**, su Zoom, un **venerdì** sera ("che bella la lezione del venerdì! Sembra quasi un aperitivo"), ed è divisa tra i tre docenti principali del corso, ciascuno sul proprio territorio: **Ruggero** introduce l'argomento e presenta i **giochi di ruolo dal vivo** (LARP) e i **giochi da tavolo**; **Beatrice** presenta i **giochi pervasivi**; **Daniele** presenta i **giochi di ruolo da tavolo**. "Ce lo smezziamo un po' tutti", perché l'argomento è diviso "in tante categorie, sottocategorie, sotto-sottocategorie". Dopo i pervasivi c'è una pausa di pochi minuti; gli ultimi dieci minuti sono lasciati alle domande.

Tre avvertenze di lettura, le stesse del capitolo precedente. La prima: la trascrizione è automatica e storpia moltissimi nomi propri, soprattutto i titoli dei giochi, quasi sempre in inglese e pronunciati velocemente ("Blaze in the Dark", "Cobo Bobby", "Sifol", "Jeep Form"...). Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante la pausa la trascrizione registra, ripetute in loop per decine di righe, alcune frasi senza senso (anche all'inizio della registrazione succede qualcosa di simile), che il capitolo ignora. La terza: i docenti si passano spesso la parola senza che la trascrizione segni il cambio di voce; il capitolo attribuisce le parole a chi le pronuncia quando il contesto lo rende chiaro (chi parla di sé, chi viene chiamato per nome, chi "passa la parola"), e resta generico altrimenti.

Quanto alla data, la lezione si colloca con ogni probabilità **venerdì 11 settembre 2026**[1]: i riferimenti a "l'altra volta" e "l'anno scorso" vanno letti rispetto a quel momento.

Come leggere questo capitolo. Più ancora della lezione precedente, questa è un **catalogo**. I docenti lo dicono esplicitamente: non ci sarà nessuna interrogazione sui singoli giochi, e lo scopo non è ricordarli tutti ma sapere che esistono, avere dei punti di riferimento e, soprattutto, abituarsi a guardare oltre il proprio genere preferito. Per questo il capitolo riporta *tutti* i casi citati, con quello che i docenti ne dicono, ma li organizza in tabelle e schemi per renderli consultabili "in fase di ricerca e sviluppo", esattamente come fa

[1] La data non è mai detta. La si deduce da alcuni indizi: la prima lezione si era chiusa con "ci si vede questo venerdì"; uno studente assente ha recuperato "la lezione di lunedì"; a fine lezione si ricorda che il questionario iniziale va compilato "entro il 15" (settembre) e che la lezione successiva, sull'*emotional design*, sarà di lunedì. Nel 2026 il primo venerdì utile prima del 15 settembre, dopo il lunedì della prima lezione, è l'11. È una deduzione, non un'informazione esplicita.

Ruggero con le sue slide ("non sapete quante volte mi sono trovato a riaprire proprio esattamente queste slide"). Qua e là, dei paragrafi di **approfondimento** collegano i casi ai libri di game design di riferimento (*The Art of Game Design* di Jesse Schell, *Challenges for Game Designers* di Brenda Brathwaite e Ian Schreiber, *21st Century Game Design* di Chris Bateman e Richard Boon).

Il percorso del capitolo segue quello della lezione:

1. l'avvio, Discord, i materiali e i **libri consigliati** (e sconsigliati) per cominciare (sezioni 2.2.1 e 2.2.2);

2. **perché** analizzare i casi studio e **che cosa** sono (sezione 2.2.3);

3. le **quattro macrocategorie** del gioco analogico, l'ibridazione e l'invito a non chiudersi nel proprio genere (sezioni 2.2.4 e 2.2.5);

4. i **giochi pervasivi**, con Beatrice: la definizione, il cerchio magico e diciotto casi studio, dalle app agli *alternate reality game*, dal teatro interattivo alle cacce al tesoro (sezioni da 2.2.6 a 2.2.11);

5. i **giochi di ruolo dal vivo**, con Ruggero: boffer, Mind's Eye Theatre, chamber e parlor LARP, American freeform, murder party, il **Nordic LARP** con i suoi manifesti, lo Jeepform, il 360°, i tape e i black box LARP (sezioni da 2.2.12 a 2.2.17);

6. i **giochi di ruolo da tavolo**, con Daniele: le quattro parole (conflitto, azione, risoluzione, effetto), la storia dal primo *Dungeons & Dragons* alla New Wave di The Forge, l'OSR, i giochi focalizzati, gli story game, i giochi per due, in solitario e i micro giochi di ruolo (sezioni da 2.2.18 a 2.2.23);

7. i **giochi da tavolo**, di nuovo con Ruggero: diciassette generi, dal *roll and move* ai Legacy, e gli ibridi (sezioni da 2.2.24 a 2.2.26);

8. la chiusura, i materiali caricati, le **domande finali** e due avvertimenti importanti: non fare un gioco di carte collezionabili, non fare un gioco per "aggiustare" un altro gioco (sezioni da 2.2.27 a 2.2.29).

2.2.1 L'avvio della lezione

La registrazione parte mentre i docenti si stanno ancora collegando ("mamma mia, la burocrazia italiana") e si controlla che il simbolo REC lampeggi. Mentre Daniele prova a condividere le slide ("aspettate, sono un attimo bloccato") e chiede di essere nominato co-organizzatore della riunione, gli studenti entrano uno alla volta: Bea, Lorenzo, Nicola, Rita, Gianmarco, Tiziana, Giordano. L'atmosfera è rilassata. Si scherza sul fatto che la lezione del venerdì "sembra quasi un aperitivo": un docente si mette "bello comodo", un altro ha a casa "le cose per fare l'aperitivo" e chiede il permesso di prepararlo nel frattempo, "purché siate sul pezzo". Daniele scrive anche di dover ancora rispondere a un messaggio di Beatrice; "non preoccuparti", gli risponde lei: settembre è un mese di ripartenza, "in realtà è l'inizio dell'anno reale".

Giordano. Arriva **Giordano**, che era assente alla prima lezione. Ha recuperato la registrazione di lunedì, e sa che gli si chiederà di presentarsi. (Alla fine, per mancanza di tempo, la presentazione verrà rinviata alla lezione successiva: si veda la sezione 2.2.28.) All'inizio mancano ancora Lorenzo e Tancredi; Lorenzo arriva poco dopo, Tancredi dopo la pausa.

Discord. Uno dei docenti nota che gli studenti sono "belli attivi su Discord" e se ne rallegra: "sfruttatelo, perché l'anno scorso lo si sfruttava poco e invece è uno spazio molto importante". I docenti leggono il server ogni tanto, e le domande sono benvenute. L'unica richiesta è **taggarli**: "tra tanti server Discord non riusciamo a leggere" tutto, quindi una domanda rivolta a loro va segnalata con un tag. Lo spazio, ricorda, "è fatto per voi": per confrontarsi, condividere cose e materiali.

Le dispense. Gli studenti hanno visto i materiali caricati: le slide, le dispense "e quant'altro". Qualcuno chiede se le dispense siano in inglese e se siano articoli. Sono i **saggi citati nella prima lezione** (per esempio quelli sull'MDA e sul DPE), e **non è obbligatorio leggerli**: sono lì "per completezza" e per mostrare che dietro gli argomenti delle lezioni "c'è una base teorica". L'importante è che le lezioni facciano capire gli argomenti. Spesso i docenti citeranno **paper accademici** di game design e di *game studies*, e "purtroppo c'è poco da fare": molti non sono mai stati tradotti, o lo sono stati "in maniera un po' approssimativa". Per questo di solito si linka la fonte originale; nulla vieta di cercare traduzioni o di tradurseli per conto proprio. In ogni caso i docenti cercano di scegliere paper **accessibili**: "non vi linkeremo delle tesi di master di 300 pagine, a meno che non ce le chiediate voi".

2.2.2 Due libri per cominciare (e uno da rimandare)

Prima di entrare nel tema, Daniele torna su due libri citati la volta precedente, mostrandoli in video.

2.2.2.1 Un volume sull'edu-larp

A Tiziana, che ha qualche difficoltà con l'inglese, Daniele consiglia un libro italiano in cui si trovano molti degli argomenti del corso: un volume sull'**edu-larp**, il gioco di ruolo dal vivo in ambito educativo, a cura di **Andrea Maragliano** ed edito da **FrancoAngeli**[2]. Con una frecciata affettuosa, Daniele aggiunge che contiene "un'introduzione molto interessante scritta da un certo Ruggero. Non so chi sia". Ruggero protesta ("non fare queste cose, che poi sembra che consiglio i libri in cui ho partecipato"): il libro non è suo, ci hanno scritto "tutti i miei amici", e lui per principio non consiglia i libri su cui scrive ("possiamo anche comprarlo e non leggere l'introduzione", ride un collega).

Daniele però lo consiglia convintamente:

- molti concetti che si trovano nei siti in inglese "qua dentro sono molto spiegati"; lui stesso lo ha usato per studiare;
- c'è una parte, "bellissima", sull'**involvement** (il coinvolgimento), che spiega anche "perché non usiamo la parola immersione" (una delle parole vietate della prima lezione); se ne parlerà anche con Ruggero;
- finalmente si può consigliare "roba genuina, sana, italiana di game design";
- copre l'ambito **didattico e formativo**, cioè il gioco nelle scuole e il gioco come strumento di formazione, che interessa ad alcuni studenti del corso.

Non bisogna leggerlo tutto: è utile "leggerlo pian pianino mentre fate il corso", se si vuole approfondire.

2.2.2.2 La progettazione dei giochi da tavolo di Walter Nuccio

Il secondo libro mostrato la volta precedente è *La progettazione dei giochi da tavolo* di **Walter Nuccio**. Qui Daniele fa un "piccolo cesellamento": "se fossi in voi non lo comprerei per il momento", e lascia la parola a Ruggero, che ne ha una copia alle spalle. Il giudizio di Ruggero è preciso:

- è più utile **dopo** aver interiorizzato alcune basi su come si progettano i giochi; come primo libro è un rischio, "perché sembra che ti dia delle scorciatoie";
- più che un saggio di game design, è **"un grosso archivio di meccaniche di gioco"**, una sorta di dizionario ("che cos'è un malus, per esempio?"), con una parte di *pattern*;
- il pericolo è applicare quelle meccaniche ai propri giochi **senza capire che cosa fanno**: "come se apriamo una cassetta degli attrezzi e non sappiamo che cosa dobbiamo fare. Gli attrezzi sembrano tutti bellissimi, però magari li usiamo male";

[2] La trascrizione riporta il titolo come "EduLARP", "Edu LARP" e "DualArp", e il nome del curatore come "Andrea Maragliano"; il titolo completo del volume non è detto a lezione.

- il consiglio è aspettare a comprarlo, o almeno a leggerlo, fino a dopo la lezione sull'**emotional design**, quando si avrà già un'idea di come partire con il proprio gioco.

Daniele è "assolutamente d'accordo". (Nel frattempo si è spostato in cucina: "benvenuti nella mia cucina, davanti al mio frigorifero, per parlare di user experience".) Del libro di Nuccio, e di un'alternativa in inglese, si riparlerà a proposito dei giochi da tavolo (sezione 2.2.25).

2.2.3 Perché analizzare i casi studio

Ruggero apre l'argomento del giorno, i **casi studio**, con due domande alla classe: *perché* ci teniamo ad analizzarli? E prima ancora: che cosa ci si aspetta che siano? "Non è una domanda provocatoria".

Le risposte degli studenti. Una prima risposta: sono **altri giochi già realizzati**, da analizzare da molti punti di vista, "anche insoliti", non solo da giocatori: dal punto di vista della produzione, o dell'esperienza utente che se ne è avuta, per capire come migliorarli. Una seconda risposta: i casi studio sono delle **milestone**, giochi o situazioni "lapidarie" da cui trarre "figure di merito" da sviscerare per poi generare altre cose. Tiziana chiede invece una cosa "più basica": che cosa si intende esattamente? Modelli, esempi, ma di che cosa?

La definizione. "Ottima domanda". Con **caso studio** si intende un **gioco specifico**, o una **categoria di giochi**, che, analizzato, ci dà delle **informazioni utili**: osservando da vicino che cosa un gioco fa, possiamo estrapolare informazioni da riproporre, o metodologie che ci servono per creare i nostri giochi. L'analogia è con la scrittura: chi vuole scrivere un libro legge tanti libri, e capisce che cosa fanno di positivo quelli che gli piacciono e che cosa non fanno quelli che non gli piacciono, per evitare le cose che non gli piacciono e "iterare sulle cose che ci piacciono". Con i giochi è lo stesso: si guarda che cosa fanno di positivo e di negativo, **tenendo conto anche del contesto** in cui sono nati.

Oggi la panoramica, poi gli strumenti. Gli strumenti più tecnici per analizzare nel dettaglio arriveranno con le lezioni di **ricerca e sviluppo**. Oggi si fa una ricognizione di tutti i settori.

2.2.4 Il gioco analogico e le sue quattro macrocategorie

2.2.4.1 Un termine ombrello

Prima una precisazione. Nel corso, **gioco analogico** è un **termine ombrello** che copre i giochi da tavolo (*board game*), i giochi di carte (*card game*), i giochi di ruolo da tavolo, i giochi di ruolo dal vivo, i giochi pervasivi, le cacce al tesoro, le *escape room*, i *murder party*: tutti quei giochi che "generalmente non sono focalizzati e incentrati sulla programmazione, sull'uso di un terminale, sull'uso di uno schermo"; in breve, che **non sono videogiochi**. Esistono cose ibride, che si vedranno, ma l'area di interesse principale resta il gioco analogico.

2.2.4.2 Le quattro macrocategorie

La lezione parte dal "frame attuale": quattro **macrocategorie**, a loro volta divise in sottocategorie. Ruggero avverte subito che "non sono separate da muri di acciaio non comunicanti": i casi ibridi, difficili da mettere in una sola categoria, sono frequenti. I tre docenti presentano ciascuno la propria.

Gioco da tavolo (Ruggero). La categoria dei *board game*. "Per motivi di industria" vi si includono anche i giochi di carte e i *party game*: tutti i giochi **pubblicati e prodotti all'interno dell'industria del gioco da tavolo**, che abbiano o no un tabellone.

Gioco di ruolo da tavolo (Daniele). GDR in italiano, TTRPG (*TableTop Role-Playing Game*) in inglese. È una categoria "molto particolare", perché anzitutto **non c'è una definizione univoca**: sul mercato si trovano cose molto diverse, perfino giochi che sono quasi "dei LARP da tavolo", vicini ai

chamber LARP. Anche i **formati** sono diversissimi: la slide mostra *Dungeons & Dragons*; *Neverland*, un supplemento[3]; *Dune*, in volumi cartonati "importanti"; ma anche *The Quiet Year*[4], che sta in un sacchettino; e *Lady Blackbird*[5], che è solo digitale: "tu lo prendi, lo stampi in bianco e nero e te lo giochi". L'edizione italiana di *Montsegur 1244*[6] è una sorta di scatola che si apre; *CBR+PNK*[7] "si apre in un modo stranissimo" e contiene dei pieghevoli (*leaflet*); ci sono giochi di ruolo in formato brochure. Daniele li mostrerà meglio nella lezione sulla *fiction*. La conclusione: nei giochi di ruolo **non si può usare il metro del formato**.

Gioco di ruolo dal vivo (Ruggero). Il LARP, *Live Action Role-Playing game*: "anche questa è una categoria gigantesca".

Gioco pervasivo (Beatrice). La categoria meno conosciuta in Italia, e anche per questo inserita nel corso.

Che cosa distingue il gioco di ruolo. Daniele chiede alla classe che cosa distingua i *role-playing game*. La risposta è nel nome: il **role playing**, cioè un **elemento di interpretazione**, che non è detto sia interpretazione di un *personaggio* (in *The Quiet Year*, per esempio, non c'è un personaggio, "abbiamo qualcosa di diverso"). E c'è un elemento "importantissimo": la **fiction**, cioè la narrazione. Non la narrazione intesa come "scrivere una trama e giocarla", ma quella che nasce al tavolo, la **narrazione emergente**: non si è previsto che l'avventura andasse in quel modo, ma mentre si gioca gli altri dicono cose che la portano in una certa direzione. Se ne parlerà nella lezione sulla fiction emergente. Categorizzare i GDR è "veramente molto arduo", e anche il pubblico è particolare: "ognuno ha il suo palato", c'è chi gioca tutto e chi gioca solo un sottogenere. A una domanda dalla classe (il gioco *Lovecraftesque*, citato da qualcuno, rientra tra i giochi di ruolo da tavolo?) la risposta è sì: e di che *tipo* di gioco di ruolo sia si capirà con le sottocategorie che Daniele presenterà più tardi.

Perché il LARP è separato dal GDR. Ruggero spiega che i LARP sono separati dal GDR da tavolo "semplicemente per motivi di settore e di industria". Il gioco di ruolo da tavolo viene venduto con una produzione più vicina al gioco da tavolo: si produce un **oggetto fisico** che poi si vende. Il LARP invece è ancora spesso organizzato **a eventi**: si partecipa **comprando un biglietto**, e si va a giocare in un posto specifico, a un'ora specifica, con persone che sono lì per giocare quel gioco. Da questo punto di vista è più vicino ad alcuni giochi pervasivi. Le sottocategorie sono "una valanga": la più famosa è il **boffer** (il cliché del LARP: persone in costume medievale "che si picchiano con spade di plastazote o di gommapiuma"), ma ce ne sono di molto più astratte e di molto più accessibili, che si possono organizzare anche in casa.

2.2.4.3 I giochi pervasivi: una prima definizione

Beatrice chiede se qualcuno abbia mai sentito parlare di *pervasive games*. Quasi nessuno; una studentessa ne ha sentito parlare solo leggendo la descrizione del corso, e si è incuriosita. "Ci sta tantissimo": in Italia non sono ancora molto conosciuti, ed è uno dei motivi per cui sono nel percorso.

Tiziana chiede se le **cacce al tesoro** organizzate nelle città rientrino negli *urban game*. "Assolutamente sì": gli **urban game** sono una categoria dei giochi pervasivi, e nella slide si vedono proprio come un sottoinsieme del "grande cappello". Beatrice dà due definizioni:

1. la definizione "standard", dai *game studies*: un gioco pervasivo è un gioco che **espande il proprio cerchio magico spazialmente, temporalmente e socialmente**;

2. la definizione più intuitiva, da cui partire per capire poi meglio la prima: un gioco pervasivo è un gioco che **utilizza i luoghi e gli oggetti della vita quotidiana e li trasforma in luoghi e oggetti di gioco**.

[3] Daniele lo chiama "un'espansione per D&D"; più avanti lo descrive come una reinterpretazione di Peter Pan in chiave OSR, compatibile con le regole di D&D.
[4] La trascrizione dice "Quietier" e, più avanti, "The Quiet Ear".
[5] La trascrizione dice "Lady Blackboard". *Lady Blackbird* è un gioco di ruolo distribuito gratuitamente in PDF.
[6] La trascrizione dice "Monsegur".
[7] La trascrizione dice "CBR PNK".

Se la prima non è chiara, "non è un problema, la capiremo con i casi studio". Negli urban game "non giochiamo seduti al tavolo, ma giochiamo in città, e stiamo comunque giocando". La parola *pervasivo* indica la pervasività del gioco, che entra in situazioni in cui non siamo abituati a immaginarlo, anche se magari siamo abituatissimi a viverlo senza pensarci.

Le cene con delitto. Tiziana fa un'altra domanda (Beatrice ricorda che si può sempre interrompere, anche con la mano alzata delle reazioni di Zoom): le **cene con delitto** rientrano tra i pervasivi? E il **teatro interattivo**, visto che in certe cene con delitto chi le propone recita una vera rappresentazione? Per Beatrice, "a mio dire, sì, entrambe". Ma le cene con delitto, o *murder party*, esistono in molte versioni:

- in una, gli **attori** vengono al tavolo con te, magari insospettabili, e tu, come persona giocante, li interroghi: questa versione assomiglia di più al teatro interattivo, che è incluso nei giochi pervasivi;
- in un'altra, **ognuno riceve un ruolo**, con una sorta di copione ("stasera sei la sorella della vittima"): questa va più nella direzione dei **giochi di ruolo dal vivo**.

Sono distinzioni "che facciamo per allenarci a comprendere meglio", ma che spesso si sovrappongono: anche quando si è investigatori e si interrogano degli attori, si è dentro una dinamica di gioco di ruolo, "anche se magari non come la immaginiamo in D&D o in giochi più blasonati". Le definizioni "sono malleabili": si imparano le parole e poi si vede come usarle nel caso specifico.

2.2.4.4 Ibridazione e sperimentazione

Nell'angolo della slide, dietro tutte le categorie, c'è "una piccola cosina": **ibridazione e sperimentazione**. "In realtà le categorie non esistono", dice Ruggero: esistono come divisioni tra industrie, ma escono continuamente giochi che fanno da ponte. La slide mostra alcuni esempi:

- *Fateforge*[8], un gioco da tavolo che usa un'**app** digitale integrata nel gioco: "un po' videogioco, un po' board game";
- un gioco di aste che usa anch'esso un cellulare (il titolo non è detto);
- dei *mock trial* (processi simulati) ibridi, a metà tra il murder party e il roleplay, "un po' roleplay, ma anche un po' giochi pervasivi";
- *Alice is Missing*, un gioco di ruolo da tavolo che usa una **chat** come Telegram tra le meccaniche di gioco.

Le ibridazioni rendono spesso difficile mettere un gioco "al 100%" in uno di questi "cesti".

2.2.5 Non chiudersi nel proprio genere

Prima di aprire la "diga dei casi studio", i docenti insistono su alcuni punti di metodo.

Le categorie servono a ragionare, non a recintare. Ruggero: le categorie non vengono date "per dirvi: le cose sono A, B, C e basta", ma per far ragionare su **quanto sono flessibili i confini**, e per dare esempi di cose **adiacenti** a quelle che interessano, che "possono sbloccarvi delle cose". Ognuno è lì per interessi diversi (i giochi di ruolo, le carte, i giochi formativi e didattici...), ma l'invito è a **non mettersi i paraocchi** sul proprio genere: "è grazie alla **cross-modalità** e al vedere qualcosa di risolto in un altro gioco che vi arriva poi un'idea per un gioco vostro che non c'entra apparentemente niente", magari nemmeno meccanicamente, con l'altro.

[8] La trascrizione dice "FitForge" e, più avanti, "Fateforge".

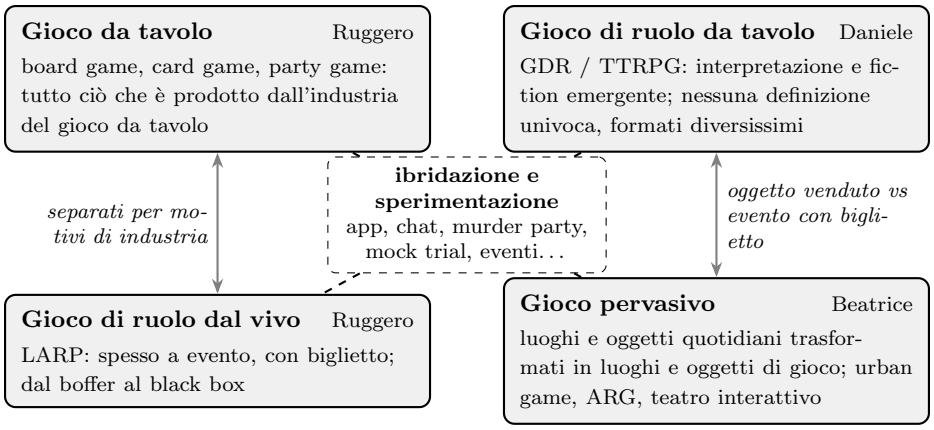


Figure 2.2.1: Le quattro macrocategorie del gioco analogico usate nella lezione, con il docente che le presenta. Al centro, la "piccola cosina" della slide: ibridazione e sperimentazione, che rendono i confini porosi. Le frecce grigie riassumono due osservazioni dei docenti: il GDR da tavolo e il LARP sono separati soprattutto per ragioni di industria (oggetto venduto o evento), e per lo stesso motivo il LARP è vicino a molti giochi pervasivi.

Un corso agnostico di genere. Daniele aggiunge che il corso è **agnostico di genere**: alcune lezioni saranno più importanti per una categoria, ma i **concetti di base del design si applicano a qualsiasi sviluppo ludico**. È fondamentale, perché altrimenti sembra che chi fa un certo tipo di prodotto abbia bisogno di "un game design particolare", e non è così. Anzi: quando è successo, storicamente anche in Italia, si è finito per **ostracizzare e separare le community**, mentre i docenti vogliono **metterle insieme**. "Voi non sapete quante volte i videogiochi vengono ispirati dai giochi da tavolo, e viceversa".

Il project work è libero. C'è anche un motivo pratico. Il progetto di gruppo finale è **libero**: i docenti non assegneranno una categoria di gioco, saranno i gruppi a sceglierla, e può essere anche un'**ibridazione**, una **sperimentazione**, un gioco che non si colloca perfettamente in un genere ("anzi, sarebbe fantastico"). Ma i gruppi saranno formati da persone con interessi diversi, ed è proprio per questo che bisogna restare aperti. L'unica cosa che i docenti chiederanno di non fare è **un gioco enorme**: il progetto deve essere **calibrato**.

"Per me, questo è design". Infine, Ruggero chiede di **non sottovalutare** la lezione: ad alcuni può sembrare "meramente un elenco di cose che vi facciamo vedere per nessun motivo", ma "per me questo è design". Quando deve trovare una soluzione, va a cercare altri giochi; e gli è capitato moltissime volte di riaprire proprio quelle slide per ricordarsi com'era fatto un gioco. E come hanno fatto Tiziana e Rita, bisogna **fare domande**: "fermateci, chiedeteci", i docenti sono lì per approfondire.

2.2.6 I giochi pervasivi: la premessa di Beatrice

Si comincia "in scivolata" con i giochi pervasivi. Beatrice anticipa che diversi casi studio hanno **accenni di digitale**, per tre motivi:

1. alcuni casi sono **molto famosi** e aiutano a capire il concetto di pervasive game, anche se non sono prettamente analogici;

2. un gioco analogico **non esclude completamente la componente digitale**: basti pensare ai giochi da tavolo che usano un'app;

3. l'**ibridazione**: si vedranno casi studio "molto strani".

Aggiunge una considerazione personale e professionale: quello dei pervasivi è un campo in cui c'è ancora **ampio spazio**, perché sono poco conosciuti e la loro **definizione è ancora in costruzione**. C'è spazio "per mettere anche la vostra voce", per esprimersi e sperimentare, proprio perché le definizioni non sono state date in modo rigido come altrove. E anticipa un tema ricorrente del corso: "parleremo molto di **emozioni**, anche di come accompagnarle all'interno di un'esperienza di gioco".

Approfondimento: il cerchio magico. La definizione "da game studies" presuppone il concetto di **cerchio magico**, che a lezione non viene spiegato. In *The Art of Game Design*, Schell lo introduce discutendo le definizioni di gioco: quasi tutte implicano che un gioco sia un sistema *chiuso*, con un confine, e "Johan Huizinga lo ha chiamato il *cerchio magico*", che ha davvero qualcosa di magico: quando siamo mentalmente "dentro il gioco" abbiamo pensieri, sentimenti e valori diversi da quando ne siamo fuori. Schell lo reinterpreta come il nostro **sistema interno di risoluzione dei problemi**: una "microrealtà" semplificata in cui gli elementi acquistano un'importanza che non devono avere fuori dal gioco. Nel capitolo sulle meccaniche, lo stesso Schell identifica il cerchio magico con lo **spazio** del gioco, la prima delle sue sette categorie di meccaniche. Il gioco pervasivo è dunque quello che sposta, allarga o rende sfumato questo confine **nello spazio** (si gioca nella città, non sul tavolo), **nel tempo** (non si sa bene quando la partita inizia e finisce) e **nella società** (vi entrano persone che non sanno di giocare, o che giocano senza chiamarlo gioco)[9]. È esattamente quello che Beatrice dirà di *GuruShot*: "non sappiamo bene dove inizia e dove finisce l'attività di gioco, ed è per questo che viene utilizzata la parola pervasivo".

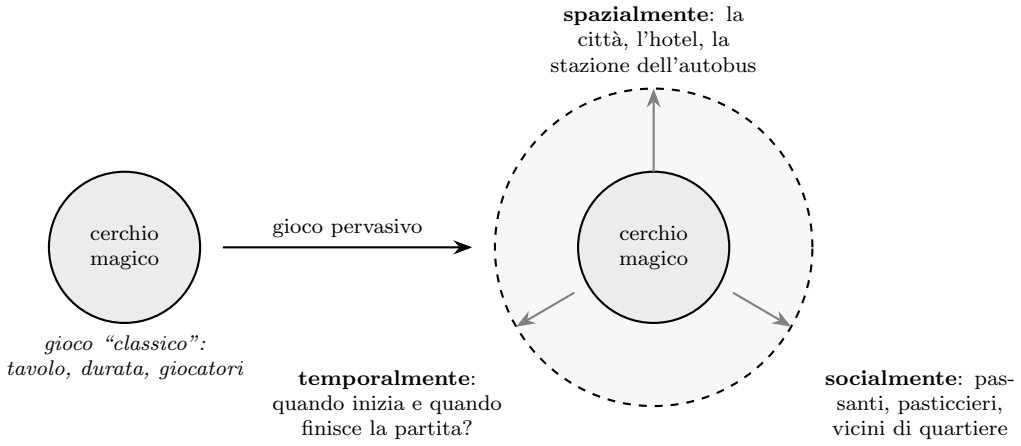


Figure 2.2.2: La definizione "da game studies" citata da Beatrice: il gioco pervasivo espande il proprio cerchio magico nello spazio, nel tempo e nella società. La definizione più intuitiva dice la stessa cosa dal lato degli oggetti: luoghi e oggetti della vita quotidiana diventano luoghi e oggetti di gioco.

2.2.7 Pervasivi con un'app: Pokémon Go, geocaching, GuruShot

2.2.7.1 Pokémon Go

Il primo esempio è scelto "proprio per farvi comprendere": **Pokémon Go**, che conoscono tutti, o di cui almeno tutti hanno sentito il nome. Beatrice lo spiega comunque, "così ci allineiamo" (scusandosi con i grandi fan per la descrizione imprecisa). È un gioco in formato **app**. I Pokémon, "questi animaletti bizzarri", nella narrativa dei Pokémon popolano il loro mondo; in *Pokémon Go* popolano **il nostro mondo reale**. Il gioco consiste nel cercarli, catturarli, andare nelle palestre, e tutto questo si fa **camminando nel mondo**: la stessa via in cui si va a fare colazione al bar o che si percorre per andare in ufficio è la via in cui si può trovare un Pokémon o una palestra. Ed è "tutto costruito dagli utenti".

[9] La formula "espandere il cerchio magico spazialmente, temporalmente e socialmente" è quella proposta da Markus Montola nei suoi studi sui giochi pervasivi, poi ripresa nel volume *Pervasive Games: Theory and Design* di Markus Montola, Jaakko Stenros e Annika Waern (2009). Beatrice non cita la fonte; la nota è un'integrazione.

Su *Pokémon Go* ci sarebbe molto da dire, "anche su come è stato utilizzato per fare altre cose meno belle ed eleganti dell'esperienza di gioco", ma Beatrice non ne parla per ragioni di tempo: ci tiene solo a far notare che "tante cose possono avere tante sfumature", e che i casi più spinosi si possono discutere a parte. Qui serve per una ragione semplice: **è pervasivo perché usa gli stessi spazi della vita quotidiana come spazi di gioco**.

2.2.7.2 Il geocaching

Il **geocaching** è anch'esso abbastanza conosciuto. "Di base è una gigantesca caccia al tesoro", tenuta in piedi da alcuni portali, e nata proprio con il **GPS** e la geolocalizzazione. Il principio:

- le persone giocanti **nascondono** delle scatole, piccole o grandi, con contenuti vari (dei "tesori") e con una sorta di **diario di bordo** (un *log*: fogli o quaderni) su cui chi le trova può scrivere "sono stato qui, ho trovato questa scatoletta";
- la scatola viene nascosta nel posto che l'utente ritiene più idoneo, e sulla piattaforma l'utente dà le **indicazioni** per trovarla;
- la grande regola è che **le scatole non vengono mai tolte**: chi le trova può scambiare gli oggetti al loro interno (piccoli, di qualunque valore, con piena libertà) e segnare la propria presenza.

Non è "un arraffare i tesori", ma un modo di **viverli e dar loro spazio collettivamente**. In Italia, "per la cultura che abbiamo", funziona così così, perché le scatole spesso vengono rubate o spostate. Ma vale la pena guardare online i video dei modi in cui vengono nascoste: alcune sono "piccole opere di ingegneria artistica", quasi piccole escape room, con un gioco di pesi o una chiave "nascosta in bella vista" che si vede solo con grande attenzione; alcune non sono scatole ma **cabine telefoniche** in cui si entra e succedono cose, o intere automobili.

2.2.7.3 GuruShot

GuruShot[10] è di nuovo in formato app, ma Beatrice lo inserisce perché usa uno strumento che lei usa "moltissimo" e che trova "molto potente per fare molte cose": **la fotografia**. È un gioco dedicato a persone molto dentro al tema, fotografi anche di un certo livello di passione e di competenza: una **gara** o maratona fotografica in cui vengono date delle **challenge fotografiche**. Chi vuole giocare apre l'app, sceglie la sfida, la svolge, carica la foto; e poi "succedono cose": valutazioni, momenti di incontro con la community, premi veri e propri offerti dagli **sponsor**. È un'attività **continua**, online da molti anni ("almeno 7–8", corregge Beatrice), con una nicchia internazionale molto nutrita.

Ed è "molto pervasivo": **non si può dire quando una partita è iniziata o finita**, né quando si sta giocando e quando no. Rispetto alla definizione del cerchio magico, non si sa bene dove inizi e dove finisca l'attività di gioco: per questo si parla di pervasività.

2.2.8 La città come campo di gioco: Journey, SF0 e Critical City

2.2.8.1 Journey to the End of the Night

Si torna sotto il cappello dei **giochi urbani**, con un gioco che si gioca in città, sempre **in formato evento**: un'ora di inizio, un'ora di fine e una zona di gioco all'interno della città. È "una sorta di **guardie e ladri gigante**". Lo scopo dei creatori era **riappropriarsi della vita notturna** e viverla in modo ludico, esplorando la città "in maniera *safe* da un lato e un po' *noir* dall'altro". Un piccolo gruppo di giocatori fa i **chaser** (gli inseguitori, le guardie) e gli altri i **runner** (i ladri); ci sono delle **tappe** da raggiungere, e il gioco dura svariate ore. Non è un gioco prettamente sportivo (ci sono altri pervasivi molto più sportivi): ci sono molte dinamiche **strategiche** su come arrivare alla fine del percorso. Beatrice lo inserisce perché, rispetto ai casi precedenti, è **completamente analogico**: non usa nessuna app, nulla. È un evento.

[10] Più avanti la trascrizione lo chiama anche "blue shot".

Il gioco è **Journey to the End of the Night**, nato in America da un collettivo con base a **San Francisco**. È arrivato in Italia nel **2008**, durante un'ondata di giochi pervasivi arrivata "in modo molto forte", e poi in Giappone e in tantissimi altri posti, perché i creatori lo hanno reso **libero**: chi vuole può organizzarlo, purché **non metta un biglietto a pagamento**. Ha avuto per anni un grande seguito: cercando "Journey to the End" Italia si trovano tantissimi eventi, e a **Milano**, tra il 2008 e il 2013, se ne sono fatte svariate edizioni con **2 000–3 000 partecipanti** ciascuna. Beatrice lo racconta "non tanto per le meccaniche", che più o meno si conoscono, ma per **come è arrivato**: un'onda lunga che dura ancora, anche se si sta affievolendo.

2.2.8.2 SF0

Degli stessi creatori è **SF0**[11], che era una sorta di grande **piattaforma online** con lo stesso principio di *GuruShot*: delle **missioni** da svolgere e documentare con video o foto. Ma invece di puntare sull'estetica della fotografia, le missioni puntavano sulle **interazioni cittadine e urbane**.

2.2.8.3 Critical City Upload

SF0 ha avuto ricadute anche in Italia, con "un figlio, chiamiamolo così": **Critical City Upload**, un gioco che è andato molto tra il 2009 e il 2013. Lo scopo è vincere, accumulando punti con le missioni, ma il gioco è molto legato agli **scambi umani e al territorio**. Le missioni andavano da "organizza una cena con una persona del tuo quartiere con cui non hai mai parlato" a una delle preferite di Beatrice: "crea un'opera d'arte interattiva che intrattenga le persone che aspettano l'autobus e posizionala in una delle pensiline del quartiere o della città".

In entrambi i casi **è la community che valuta la community**: non ci sono giudici. E dopo i primi anni, sia SF0 sia Critical City hanno permesso ai partecipanti che avevano raggiunto un certo livello di **creare essi stessi nuove missioni** per i nuovi giocatori: la comunità non si limitava a valutarsi a vicenda, ma creava il gioco per gli altri. Oggi nessuno dei due è più attivo, ma i siti sono ancora online, con molte delle missioni visibili. Quanto a "stranezza" rispetto all'idea comune di gioco analogico, "questo spinge molto forte".

2.2.9 Teatro, ARG e marketing: da Sleep No More a The Institute

2.2.9.1 Sleep No More

Chi conosce il **teatro interattivo** conosce di sicuro **Sleep No More**[12]: un'opera messa in scena all'interno di un **hotel**. È interattiva nel senso che gli attori recitano la loro parte in interazione con il pubblico, e il pubblico, con regole precise sui contatti fisici e su come interagire con gli spazi, è **libero di esplorare l'hotel** come vuole: in ogni zona ci sono attori che fanno cose. Resta un'opera teatrale, con un inizio, una fine e degli eventi nel mezzo; c'è molta documentazione online (interviste, spezzoni). E di nuovo si gioca su **qual è lo spazio di gioco e quale non lo è**.

2.2.9.2 Gli alternate reality game e Why So Serious

Qui Beatrice introduce un nuovo termine: gli **alternate reality game** (ARG), i giochi di realtà alternativa, "giochi in cui c'è un intero mondo alternativo al nostro, in cui succedono cose". Il caso scelto, "uno dei miei preferiti", porta anche nel mondo dei **giochi per il marketing e la comunicazione**: **Why So Serious**, nato come gioco promozionale per *Il cavaliere oscuro* di Christopher Nolan.

- Iniziava con una **mail** ad alcune persone iscritte a mailing list di fan di Batman, contattate dagli organizzatori tramite le loro associazioni.

[11] La trascrizione dice "SF0", "SFZero" e, una volta, "essere al vero".

[12] La trascrizione dice "Slip no more" e "SlipMod". Lo spettacolo è della compagnia britannica Punchdrunk; la versione più nota è quella di New York, allestita in un edificio trasformato nel fittizio McKittrick Hotel. Il nome della compagnia non è detto a lezione.

- La mail era un **invito strano a entrare nella banda del Joker**, con un enigma; da lì partivano attività di socializzazione organizzate attraverso un sito, che era una serie di indirizzi "in cui succedevano cose".
- Il gioco raccontava che **nel nostro mondo il Joker esisteva davvero**, e gli indirizzi erano indirizzi reali.
- Una delle prime cose: andare a questi indirizzi, tutti **pasticcerie**, e dire la parola in codice ricevuta. Si riceveva una **torta** con sopra scritto un numero di telefono; chiamandolo, squillava la torta stessa, perché dentro c'era un **cellulare**: lo strumento con cui il Joker gestiva le comunicazioni con i giocatori.

Molte strutture di questo tipo, molti colpi di scena "anche proprio pratici", che sfruttano la pervasività: cose che nel nostro mondo non hanno motivo di succedere ("non esiste che ci sia un cellulare in una torta") ma che in quel mondo hanno un senso. Gli ARG giocano moltissimo su ciò che **potrebbe essere reale ma non lo è**, e che il gioco dice che lo è: "in quel mondo voi siete gli aiutanti del Joker". *Why So Serious* è opera di un'azienda che si occupa da molti anni di ARG[13]; i dettagli sono nell'estratto che Beatrice caricherà, tratto da *Immersi nelle storie* di **Frank Rose**[14], che racconta proprio l'inizio di questo gioco.

Perché raccontarlo? Perché mostra "come possiamo giocare sulle **emozioni**" e come **connettere le persone alle cose che vogliamo comunicare**, anche costruendo un mondo alternativo. Non è un obbligo, "ma è bene sapere" che esistono questi giochi, "giganteschi".

2.2.9.3 Un gioco da tavolo che è anche un ARG

Tornando alla malleabilità delle definizioni, Beatrice presenta un gioco che è insieme **gioco da tavolo**, **alternate reality game** e gioco con **componenti digitali**, edito da **MS Edizioni**[15]. Si riceve un **fascicolo** di informazioni e poi dei **messaggi**, da cui si capisce di far parte di un gruppo che **sta rapinando una banca**, già nel pieno della rapina. I giocatori sono "l'entità che ha accesso a internet", la voce fuori dall'azione che coordina: parlano con dei personaggi, scambiano informazioni, guardano video, analizzano dati e documenti. È un gioco "con una profondità notevole". Due cose lo rendono interessante:

- mescola cose che **esistono davvero** (la cartina di una città reale, gli scambi della metropolitana di una città reale) con cose del **mondo narrativo** (i personaggi, i fatti, la rapina: "spero che sia chiaro che non state rapinando la banca davvero");
- a differenza degli altri casi, ha un **inizio e una fine ben definiti** ed è **completamente accessibile**: lo si può provare.

Per Beatrice è un caso "unico sul panorama italiano" ("poi mi correggeranno i colleghi a livello internazionale").

2.2.9.4 Collettivo Cinetico: Cinetico 4.4 e Amleto

Cinetico 4.4 è "un altro bell'esperimento": un **gioco da tavolo creato da un collettivo di danzatori e danzatrici**, il **Collettivo Cinetico**, per generare una **performance artistica dal vivo**, teatrale o di danza. Non è un gioco da prendere dallo scaffale di una ludoteca: è pensato per una performance. Chi ci gioca partecipa a un gioco da tavolo che in realtà è un **evento**: si paga un biglietto, si entra, ci si siede, succedono cose. Il collettivo, di cui Beatrice si dichiara "di parte", fa moltissime attività "né danza, né teatro, né gioco, o meglio tutte e tre le cose insieme", difficili da mettere sotto un cappello. E qui Beatrice aggancia un tema delle prossime lezioni: la **transmedialità** e la **ricerca e sviluppo**, cioè attingere anche a cose lontane dal gioco che si vuole creare, perché "più conosciamo, più possiamo aprirci la testa".

Un caso studio in più: l'**Amleto** del Collettivo Cinetico. È l'Amleto di Shakespeare, ma sul palco non ci sono i danzatori del collettivo. Tre giorni prima dello spettacolo fanno una vera e propria **audizione** aperta

[13] Beatrice non fa il nome dell'azienda, che si troverà nell'estratto distribuito. Si tratta di 42 Entertainment (integrazione).
[14] Edizione italiana di *The Art of Immersion* (2011).
[15] La trascrizione chiama il gioco "The Ice"; il titolo non è ricostruibile con certezza (potrebbe trattarsi di un titolo come *The Heist*). MS Edizioni è un editore italiano di giochi; è forse la stessa "MS" per cui, nella prima lezione, Ruggero dice di lavorare, ma la trascrizione non permette di confermarlo.

alla popolazione; tra chi si presenta scelgono **sei potenziali Amleto**, che portano sul palco a danzare e recitare, e creano momenti di **interazione con il pubblico** per scegliere, tra i sei, l'Amleto di quell'edizione. Ci sono giochi veri e propri sia nell'audizione sia nello spettacolo. Loro, da danzatori, lo chiamano spettacolo di danza, "ma se lo vedete c'è tantissimo".

2.2.9.5 Lettere anonime per un camminatore

"Rispetto a quello che vi ho detto, questo è semplice". **Lettere anonime per un camminatore** è del collettivo **Amigdala**, anche loro una compagnia teatrale (Gianmarco, che ha vissuto a Modena, conferma che sono di lì). È una **passeggiata** con piccole interazioni ludiche, tutta basata su un **audio**:

- è un gioco **in solitario**: una persona sola segue un percorso segnato da frecce, per terra o sui muri di una città, guidata da una **voce in cuffia** che le racconta delle cose;
- la struttura del racconto è fissa, ma la **parte centrale** cambia ogni volta: prima di ogni edizione il collettivo **intervista le persone che hanno abitato o abitano il percorso**, e mentre si cammina si ascolta, oltre alla narrazione, il racconto di chi quei luoghi li vive o li ha vissuti;
- la **conclusione** è sempre "col botto": il collettivo sceglie di finire in modo scenografico in luoghi normalmente inaccessibili. Beatrice lo ha giocato a Ferrara, e alla fine del percorso si è ritrovata seduta in poltrona, con un tavolino e un drink, al **centro dello stadio comunale**, con lo stadio vuoto e le cuffie nelle orecchie.

Torna il tema del **percorso emotivo** che si fa fare alla persona giocante. E lo strumento dell'**audio che guida** è interessante anche per il corso: un'app non si riesce a fare nel tempo del project work, "ma per utilizzare l'audio invece possiamo".

2.2.9.6 MP3 Experiment

Un altro esempio basato sull'audio, più simile a un **flash mob**: **MP3 Experiment**, del collettivo newyorkese **Improv Everywhere**[16], che fa moltissimi esperimenti di questo tipo. Ci si ritrova tutti nel giorno, nell'ora e nel posto concordati, ci si collega con le cuffie a una trasmissione in diretta, e la voce dice che cosa fare. Beatrice non svela di più ("guardatevi i video su YouTube, ci sono delle perle"), ma racconta un dettaglio: gli organizzatori **elencano gli oggetti da portare**, e in un'edizione chiesero di portare **due matite**; a un certo punto tutto il gruppo si mise a suonare la batteria con le matite sugli oggetti della città. Quella **scintilla di curiosità** per oggetti di cui non si sa ancora l'uso è uno strumento del percorso emotivo, "non difficile" da usare: "basta solo ragionare di creatività". Come le istruzioni audio.

2.2.9.7 The Institute e Dispatches from Elsewhere

The Institute[17] è, nel modo in cui Beatrice ha imparato questo mondo, uno dei casi più blasonati e famosi di alternate reality game, uno dei primi raccontati come tale. È un'esperienza così complessa che Beatrice preferisce indicare due fonti per approfondirla:

- un **documentario**, intitolato anch'esso *The Institute*, che racconta tutto l'ARG, durato circa un anno e fatto di molti passaggi, con tantissimi colpi di scena del tipo "torta con il cellulare";
- una **serie TV**, *Dispatches from Elsewhere* (Daniele conferma il titolo), che ha preso l'esperienza di The Institute, ci ha aggiunto una storia dei personaggi e ne ha fatto una serie. Tolta la storia dei personaggi ("comunque molto carina"), resta più o meno l'idea dell'ARG.

[16] La trascrizione dice "Improve Everywhere".
[17] Il gioco è noto anche come *The Jejune Institute*, di San Francisco (integrazione).

Con budget zero. Qui Beatrice fa una precisazione importante. Questi esempi sono **difficili da replicare**: "con che soldi compriamo la torta e ci mettiamo il cellulare? Con che soldi affittiamo un intero palazzo come in The Institute?". Serve un intero lato commerciale, di ricerca di sponsor, che non è parte del design del gioco e quindi non è il motivo per cui si è nel corso. Quello che vuole che gli studenti si portino a casa è che **con molto meno** si possono ottenere esperienze "estremamente significative a livello emozionale" e creazioni di game design molto profonde **con budget zero**. I casi grandi servono come **punti di riferimento** e per sapere che esistono cose "molto distanti da quelle che immaginiamo tutti i giorni".

2.2.9.8 L'ARG di Stranger Things a Milano

Nello stesso filone, un ARG che "è stato un'operazione di marketing e basta" ("per me sono molto affascinanti, però era un'operazione di marketing"), ma fatto **a Milano**: quello per una stagione di *Stranger Things* (la terza o la quarta, Beatrice non ricorda). Era una sorta di **caccia al tesoro** in città, con punti di gioco preparati in modo "altamente scenografico" (nella slide si vede un autobus), mescolata con una **telefonata** a un numero in cui uno dei personaggi diceva che il Sottosopra era arrivato a Milano e che serviva aiuto. Il messaggio: anche da noi questi giochi "sono arrivati, arrivano, e credo che col tempo avranno sempre più spazio", e per chi pensa in termini di lavoro "è un tema".

2.2.10 Cacce al tesoro, corse e indagini

2.2.10.1 Le guide-caccia al tesoro su carta

Beatrice presenta poi delle **guide per esplorare la città** in forma di caccia al tesoro[18], uscite negli stessi anni dell'ondata dei pervasivi in Italia (la prima nel 2007–2008), disponibili solo per alcune città e vendute come **libretti** sul loro sito o da rivenditori autorizzati. Tiziana le conosce: aveva comprato quelle di **Venezia** e di **Verona**, ha giocato in parte quella di Venezia, nella versione breve (c'erano due possibilità: un gioco di un paio d'ore o uno da una giornata intera), e ricorda il **numero di cellulare** con cui interagire per avere le indicazioni. Beatrice le aveva giocate con gli SMS, "era un'altra vita"; oggi pare si siano aggiornate a WhatsApp (e chiede aggiornamenti a chi le giocherà).

Sono un altro ibrido di **analogico e digitale**: un'entità parla con i giocatori tramite messaggi, mentre il libretto cartaceo resta in mano a loro. La particolarità è la meccanica per passare da una tappa all'altra: **ogni pagina del libro è divisa in strisce**, e quando si risolve una tappa la guida comunica dei **codici** ("il foglietto 3, il 32C e il 18F") con cui ricomporre la pagina che porta alla tappa successiva.

2.2.10.2 PacManhattan

PacManhattan[19], degli stessi anni, è **Pac-Man per le strade di Manhattan**: una persona fa letteralmente Pac-Man, guidata da un'altra persona ("la mente") tramite telefono o walkie-talkie; altri giocatori fanno i fantasmi. È di nuovo "una sorta di guardie e ladri", ed è uno dei primi urban game che si trovano nei manuali di game studies. Beatrice lo mostra per un'idea precisa: si può **prendere qualcosa che esiste già in una forma e cambiarle forma completamente**. Non è lo stesso gioco di Pac-Man, ma si ispira alla sua narrativa, alle sue meccaniche, alla sua estetica.

2.2.10.3 Zombies, Run!

Zombies, Run![20] è un'**app per fare jogging**: invece di ascoltare musica, o i messaggi vocali degli amici, si ascoltano **gli zombie che stanno per prenderti**, e c'è tutta una narrativa su come si sta scappando mentre si corre. Si trova sugli store.

[18] La trascrizione riporta il nome come "Waiwai"; la grafia non è verificabile.
[19] La trascrizione dice "Pac-Man-Hatton".
[20] La trascrizione dice "Zombie Run".

2.2.10.4 Reptilarium

Si torna al teatro, ma in versione analogico-digitale. **Reptilarium** è un caso studio di cui purtroppo c'è pochissima documentazione. È uscito durante il **COVID**, da una compagnia teatrale di Roma, **Project XX1**[21]. Era a modalità evento (un certo giorno, a una certa ora) e le persone accedevano da **casa propria**, davanti al computer, a un'esperienza con degli **attori** che si mettevano in comunicazione con loro: ricevevano mail e telefonate, potevano vedere video "recuperati da altre persone", potevano **coordinare i movimenti degli attori** e vedere che cosa succedeva mentre li eseguivano. La trama partiva da una **persona scomparsa** da ritrovare, e poi si espandeva molto. Come per le cene con delitto (dove si può interrogare l'attore o essere l'indiziato), anche questo esperimento "può diventare un LARP", un ibrido tra teatro e gioco di ruolo, o tra gioco investigativo e alternate reality game.

Ruggero aggiunge una nota: il **lockdown** ha colpito moltissimo i giochi pervasivi, ma anche altri generi come i LARP, e in quel periodo sono nate esperienze che hanno spostato **online e a distanza** giochi nati per essere giocati di persona. Se ne riparlerà.

2.2.11 Pervasivi per la formazione: Echi resistenti

"Tutte le cose che vedete hanno enorme spazio per essere esplorate in tema **educativo e formativo**", e Beatrice chiude con un accenno a questo ambito. **Pop History** è un'**associazione di promozione sociale**: non si presenta come collettivo teatrale né come gruppo di game designer, ma come associazione ("credo siano anche loro emiliani"), e fa tante piccole sperimentazioni. Una di queste è **Echi resistenti**[22], un gioco sulla **Resistenza e sulla Liberazione**, che ibridava:

- un'**indagine** con dei fascicoli;
- un'**attività teatrale**, con personaggi storici con cui si poteva interagire direttamente;
- una **caccia al tesoro**, in cui si andava a cercare questi personaggi per la città.

Altri casi simili sono sul loro sito.

Approfondimento: giochi con migliaia di giocatori. In *Challenges for Game Designers*, nel capitolo sui giochi multigiocatore, Brathwaite e Schreiber osservano che nei giochi da tavolo il problema dell'interfaccia è evidentissimo ("come si mettono venti persone attorno a un solo tabellone? E mille?"), eppure "esistono giochi non digitali con migliaia e perfino centinaia di migliaia di giocatori": gli **alternate reality game** e i grandi **urban game**, che "portano i giocatori via dai loro computer e dentro il mondo reale". È la stessa scala dei casi di Beatrice: *Journey to the End of the Night* con migliaia di partecipanti a Milano, *Why So Serious* con le sue mailing list di fan, *Pokémon Go* con il mondo intero come mappa.

2.2.12 I giochi di ruolo dal vivo: le macrocategorie

Finita la parte sui pervasivi, si fa una pausa di pochi minuti ("facciamo sette minuti, così andiamo ai 20 tondi"). Al rientro ci sono tutti, compreso Tancredi ("ci sono tutti e due" i Lorenzo), e Ruggero passa ai **giochi di ruolo dal vivo**.

Una struttura diversa. Questa parte è organizzata diversamente da quella sui pervasivi. Il gioco di ruolo dal vivo, come il gioco di ruolo da tavolo e il gioco da tavolo, ha alle spalle **anni di industria solidificata**, ed è quindi più facile identificare **macrocategorie** con nomi e formati specifici. Prima, però, Ruggero chiede chi abbia mai giocato un LARP. Una persona della classe sì: un LARP fantasy, al **festival del gioco di ruolo di Bracciano**, "con le spade di gommapiuma, insomma, quelle cose lì". È il primo impatto di molti.

[21] Grafia come nella trascrizione.
[22] La trascrizione dice "Ecchi Resistenti" e "Eh, che resistenti".

Caso studio	**Formato**	**Perché è nella lezione**
Pokémon Go	app	gli spazi quotidiani diventano spazi di gioco
Geocaching	piattaforma + scatole	caccia al tesoro collettiva: le scatole non si tolgono mai
GuruShot	app	la fotografia come strumento; non si sa quando la partita inizi e finisca
Journey to the End of the Night	evento urbano	guardie e ladri notturno, completamente analogico, libero e non a pagamento
SF0	piattaforma	missioni urbane documentate; la community valuta e crea missioni
Critical City Upload	piattaforma	il "figlio" italiano di SF0 (2009–2013): missioni di relazione sul territorio
Sleep No More	teatro interattivo	il pubblico esplora liberamente un hotel
Why So Serious	ARG di marketing	la torta con il cellulare: il Joker esiste davvero
Gioco di MS Edizioni	gioco da tavolo + ARG	rapina in corso tra documenti reali e finzione; ha inizio e fine
Cinetico 4.4, Amleto	performance	un gioco da tavolo che è uno spettacolo; il pubblico sceglie Amleto
Lettere anonime per un camminatore	passeggiata audio	gioco in solitario, voci degli abitanti, finale scenografico
MP3 Experiment	flash mob audio	la curiosità per gli oggetti da portare
The Institute	ARG	il caso blasonato, con documentario e serie TV
ARG di Stranger Things	ARG di marketing	un ARG fatto a Milano
Guide-caccia al tesoro	libretto + SMS	pagine in strisce da ricomporre con codici
PacManhattan	urban game	un videogioco cambiato completamente di forma
Zombies, Run!	app	narrativa audio durante la corsa
Reptilarium	teatro a distanza	attori guidati da casa durante il COVID
Echi resistenti	gioco formativo	indagine, teatro e caccia al tesoro sulla Resistenza

Table 2.2.1: I casi studio di giochi pervasivi presentati da Beatrice, nell'ordine della lezione, con il motivo per cui sono stati scelti.

2.2.12.1 Il boffer LARP

Chi ha giocato un LARP probabilmente ha giocato un **boffer**, e chi ne ha solo sentito parlare ha in mente quello. È uno dei primi giochi di ruolo dal vivo diffusi, nato all'epoca proprio come **tentativo di adattare *Dungeons & Dragons* dal vivo**, poi evoluto nel tempo. (Daniele mostra una foto e Ruggero protesta: "non evidenziarmi, che già è cringe"; ne ha giocati anche lui.) Il nome viene dalle armi: le armi *boffer* sono armi in plastazote che devono avere caratteristiche precise per non essere pericolose. Sono giochi **incentrati sul combattimento**, su trame epiche e così via. Sono anche quelli che possono essere più grandi: un boffer anche grosso può ospitare **400–500 persone**, "per non dire quelli in Germania, che ne tengono anche migliaia".

2.2.12.2 Il Mind's Eye Theatre

Un altro tipo abbastanza diffuso è il **Mind's Eye Theatre**: se il boffer deriva da *Dungeons & Dragons*, questo deriva da ***Vampire: The Masquerade***. Le trame sono più dark, gotiche, horror, legate alla proprietà intellettuale di Vampire, e chi partecipa di solito interpreta uno dei vampiri delle loro società segrete. Sono giochi **più politici che di combattimento**: l'intrigo, i segreti, il tradirsi a vicenda, "il pugnalare alle spalle qualcuno" sono parte delle dinamiche. Come i boffer, sono giochi **in costume**: il costume è richiesto, e si chiede a chi gioca di vestirsi come il proprio personaggio.

2.2.12.3 Chamber LARP e parlor LARP

Il costume sembra scontato, "ma in realtà non tutti i LARP usano il costume come strumento necessario". I **parlor LARP** o **chamber LARP** sono giochi di ruolo dal vivo in cui il costume **non è richiesto**: si gioca con i propri vestiti di tutti i giorni. Lo studente di prima interviene: a Bracciano, in effetti, giocavano con i propri vestiti ("di solito gli eventi così sono degli ibridi"); e ricorda di aver giocato un altro LARP al **Play**[23], attorno a un tavolo, ognuno con un ruolo e con i propri vestiti: ***A cena col nonno***. "Perfetto, quello è un chamber LARP".

Ruggero precisa un punto sottile: alcuni chamber LARP si giocano **a un tavolo**, ma questo non li rende giochi di ruolo da tavolo, perché si giocano personaggi *che sono seduti a un tavolo*, mentre in *Dungeons & Dragons* il personaggio non è necessariamente seduto a un tavolo in quel momento. Con gli altri LARP hanno in comune la **sovrapposizione tra la fisicità di chi gioca e quella del personaggio**, l'**embodiment**. Rispetto ad altri LARP, i chamber LARP hanno di solito:

- un livello di **accessibilità** più alto, perché spesso non richiedono costumi;
- un **budget ridotto**;
- **non sono a evento**: molti si scaricano in *print and play*, si stampano a casa e si giocano con gli amici;
- per questo, un **numero di giocatori** più ridotto, di solito da 3 a 20 persone.

2.2.12.4 L'American freeform

Una categoria dei chamber LARP è l'**American freeform**: LARP progettati da designer che si occupano soprattutto di giochi di ruolo da tavolo, e infatti è un **genere ibrido**. Visti da fuori sono chamber LARP (gente che fa *embodiment* del personaggio in delle stanze, non necessariamente in costume), ma hanno "un sacco di meccaniche" prese dal gioco da tavolo e dal gioco di ruolo da tavolo: le cosiddette **metatecniche** o metameccaniche, di cui si parlerà in un'altra lezione. Anche questi sono spesso in *print and play*; alcuni sono venduti in un formato più vicino al gioco da tavolo, come mazzi di carte o libretti. Quindi non sono a evento, a differenza dei boffer, del Mind's Eye Theatre e dei murder party.

[23] La trascrizione dice "Play di Bologna"; il festival Play, dedicato al gioco, si tiene a Modena.

2.2.12.5 I murder party

Ruggero riprende l'ottima distinzione fatta da Beatrice. "Murder party", da noi, indica culturalmente due cose:

- le **cene con delitto**, in cui si va al ristorante, degli attori mettono in scena un omicidio e la successiva indagine, e i commensali mangiano e intanto risolvono il caso. **Nessuno dei commensali è l'assassino**: è uno degli attori;
- la categoria "più british", i **murder party all'inglese**, o i loro equivalenti che si stanno diffondendo moltissimo in Asia in questo periodo[24], in cui **si interpretano i personaggi della storia** su cui si indaga: tra i giocatori può esserci l'assassino.

La seconda forma rientra, per il design dell'esperienza e delle meccaniche, sotto l'ombrello dei LARP; gli aspetti ambientali e le decorazioni la avvicinano poi al mondo del pervasivo.

2.2.13 Il Nordic LARP

C'è poi "una grossissima categoria", che si rivedrà più avanti perché "è stata importantissima": il **Nordic LARP**, il LARP progettato in **Scandinavia** (Danimarca, Norvegia, Svezia e Finlandia).

2.2.13.1 Perché è così importante

Ha un'importanza gigantesca nel settore **perché ha documentato tutto**. La maggior parte dei libri accademici di game design e di analisi di questo tipo di giochi, gli eventi e le conferenze, si fanno in Scandinavia, e c'è una lunga storia di analisi e documentazione: la prima conferenza è del **1997**, e da allora è annuale. È "gente che si incontra da parecchi anni a parlare di come si progetta e come si gioca". Ed è una scena molto **cross-mediale**: prende dagli alternate reality game, dai LARP americani, dai parchi a tema, dalle simulazioni economiche, da tante teorie parallele a quelle del gioco, cercando di interiorizzare, iterare e integrare tecniche di altri settori. Anche il Nordic LARP ha le sue sottocategorie: alcune esperienze sono più vicine al chamber LARP, altre al boffer.

2.2.13.2 Come si riconosce

Da fuori, un Nordic LARP si riconosce da **come è progettato il design dell'esperienza**:

- un focus sul **coinvolgimento**, sull'**emozione** e sulla **coesione tematica**;
- pochissime cose **vestigiali**: per esempio, anche LARP molto grandi possono non avere il combattimento, se il combattimento non fa parte del tema;
- una cura del **post-esperienza**: molti giochi hanno una struttura per "che cosa succede dopo il gioco".

Molti concetti nati da questa scena si sono "solidificati nel gergo del game design" e torneranno nella lezione sull'emotional design.

Origini diverse da D&D. Storicamente, il LARP nordico ha un'origine **precedente a *Dungeons & Dragons***. Si tende a pensare che il LARP nasca da "prendiamo D&D e lo facciamo dal vivo", e per alcune nazioni, come l'Inghilterra e l'America, è proprio così; per altre no. Il Nordic LARP prende strategie e strumenti dal **teatro**, dalle avanguardie, dallo **psicodramma**, dalla **Gestalt therapy**; "è ovvio che quello che esce fuori è un gioco molto particolare".

I manifesti. È anche un ambiente "pieno, carico, zeppo di manifesti". La slide ne mostra alcuni, nati negli anni nella scena; non si approfondiscono ora, ma sono tutti pubblici e si possono leggere a parte.

[24] La trascrizione dice "Jubel Shock" e "Juvenile Show". Si tratta quasi certamente dei *jubensha*, i giochi di investigazione con ruoli molto diffusi in Cina.

2.2.13.3 Knutpunkt e i Knutepunkt books

La cosa che interessa di più è la conferenza citata prima, che cambia nome a seconda del paese che la ospita: **Knutpunkt**, **Knudepunkt**, **Knutepunkt** o **Solmukohta**[25]. È il principale punto di ritrovo scandinavo sul gioco di ruolo dal vivo: **quattro giorni di conferenza non stop**, con un programma di centinaia di slot, spesso **otto track in parallelo** di talk, panel e workshop. Si tiene dal 1997, e **ogni anno produce dei libri** che riassumono lo stato dell'arte in forma di articoli: la documentazione è "costantemente in aggiornamento". Ruggero ne ha diversi, "dei bei mattoni": ne mostra uno di **600 pagine** su come si progetta un LARP, uscito da una delle edizioni ("nel 2018, se non ricordo male"). È forse "una delle comunità ludiche che più documenta quello che fa", e soprattutto lo documenta in modo **pragmatico**, lasciando framework e strumenti a chi vuole creare giochi di questo tipo. Molte di queste cose si applicano anche ad altri generi, e torneranno.

2.2.13.4 Dogma 99

In chat qualcuno chiede se c'entri il **Dogma** di Lars von Trier. "Sì e no": il **Dogma 99** è un manifesto di LARP che riprende il manifesto cinematografico del **Dogma 95** per fare un parallelo con il LARP. Il Dogma 95 aveva alcune direttive, per esempio che tutto debba essere **diegetico**, senza nulla di extradiegetico; il Dogma 99 fa lo stesso: in un LARP storico non ci devono essere cose extradiegetiche rispetto alla storia. Una conseguenza: il Dogma 99 **non fa LARP fantasy**, perché la magia non esiste e non si può riprodurre in modo perfetto; si fanno giochi ambientati tendenzialmente ai giorni nostri, che si possono riprodurre facilmente. Da questa catena di ragionamenti sono nati "giochi molto interessanti".

2.2.14 Jeepform: il dramma quotidiano

Tra i manifesti c'era lo **Jeepform**[26], nato all'interno del **freeform**, la zona "a metà tra il LARP e il tabletop". Il suo manifesto, che si trova online, ha tra i punti più importanti questo: **il dramma quotidiano è più interessante del dramma epico**. Tutti i giochi Jeepform sono giochi di ruolo dal vivo su **situazioni quotidiane**, "drammaticamente quotidiane".

Alcuni sono emotivamente molto pesanti, e storicamente importanti per vari motivi; Ruggero ha lasciato fuori i più pesanti (se ne riparlerà nella lezione sull'emotional design) e ne presenta due "più leggeri", che fanno comunque capire in che toni si va.

2.2.14.1 Il gioco della coppia perfetta

Il primo[27] è un gioco per **quattro persone** e **due personaggi**, che per premessa **si amano tantissimo**. Due persone alla volta interpretano i personaggi, in un gioco **diviso in scene**; ogni scena è descritta dalle due persone che non sono in gioco. L'unico obiettivo di chi gioca la coppia è **essere la coppia perfetta**, mostrare quanto si amano. Se la scena si sposta su qualcos'altro, **si rifà**, all'infinito, "come nel giorno della marmotta", finché non parla solo di quanto si amano. Esempio: lei entra in bagno, la porta si rompe e lei rimane chiusa dentro, lui è dall'altra parte; se lui comincia a riparare la porta, si rifà la scena. "Una paradossalità esasperata, quasi".

2.2.14.2 Fat Man Down

Il secondo è introdotto con un *content warning*: **Fat Man Down** "è un gioco molto pesante". Non è il più pesante dei Jeepform, ma è il più pesante di cui si parla nella lezione. È un LARP dello stile Jeepform, quindi un chamber LARP tendenzialmente freeform.

[25] Rispettivamente in svedese, danese, norvegese e finlandese: la conferenza ruota tra i quattro paesi. La trascrizione dice "Knutpunkt o Knutpunkt o Knudpunkt o Solmukott" e, più avanti, "Knut Punk".

[26] La trascrizione dice "G-Form" e "Jeep Form".

[27] La trascrizione riporta il titolo come "Lady 8" e i nomi dei personaggi come "Lady" e "8"; né il titolo né i nomi sono verificabili.

- Il personaggio principale è un uomo di successo, con un buon lavoro e una buona posizione economica, ma **obeso**.
- Tutti i partecipanti **si pesano su una bilancia**, e chi pesa di più interpreta questo personaggio.
- Il gioco è diviso in scene, ognuna con un *framing* che è uno *slice of life*: un'ora in ufficio, la strada da casa al supermercato, dieci minuti di pausa al parco.
- Tutte le altre persone in scena hanno la **direttiva meccanica** di farlo stare malissimo.
- Il protagonista ha una meccanica "importantissima": **tre cioccolatini** in tasca. In qualunque momento può scartarne uno e mangiarlo, e quando lo fa tutti i personaggi in scena devono trattarlo come la persona migliore del mondo.

Il personaggio è messo sotto **costante stress narrativo** finché non è spinto a pensare "devo mangiare per stare meglio". "La roba è molto pesante", e ci si ferma qui.

2.2.14.3 Le meccaniche di sicurezza

Avere un manifesto così noto nella scena nordica, con giochi molto pesanti, molto intimi e soprattutto **senza un filtro fantasy o fantascientifico**, cioè su drammi quotidiani, ha generato e normalizzato cose poi rivelatesi molto utili, come le **meccaniche di sicurezza**. Se qualcuno ne ha sentito parlare, la scena nordica è "la principale responsabile" della normalizzazione, della ricerca e dell'iterazione delle meccaniche di sicurezza nel gioco di ruolo. Se ne parlerà nella lezione sull'emotional design.

2.2.15 Il 360° e l'alta risoluzione

Il Nordic LARP non è solo chamber, freeform e Jeepform. C'è anche "quasi l'opposto": i LARP **360°** o ad **alta risoluzione** (*high resolution*)[28]. Lo scopo è essere **il più fisicamente coinvolgenti possibile**: tutto quello che si ha attorno deve essere **diegetico**. Ma la soluzione non è quella del Dogma 99 (ambientare nel quotidiano, così non serve la magia): "la soluzione è buttare al muro un budget talmente alto che la realtà a quel punto sembra magia". Niente oggetti di scena finti o armi finte: anche la scenografia è curata al punto che sembra di stare davvero nel gioco. I casi studio sono "anche abbastanza storici".

Ground Zero (1998–2001). La premessa è la **Guerra Fredda**. Una cittadina americana riceve l'allarme per una bomba nucleare, e i partecipanti vivono **48 ore di fila**, non stop, chiusi in uno **scantinato** al buio, senza poter uscire. Il gioco si conclude con la bomba che cade: dietro le finte pareti di cartone dello scantinato c'erano dei **subwoofer giganti**, e alla fine i muri cadono addosso ai giocatori con un rumore devastante.

Panopticorp (2003, 2013). Due *run*, anche questo di 48 ore non stop: in teoria si dorme in gioco, "ma in questo caso non si dormiva". È una **distopia corporate** spinta al massimo: si è dipendenti di un'agenzia pubblicitaria sull'orlo della chiusura, che deve licenziare "a manetta" e tenere solo i migliori. Una spinta ultra competitiva e "ultra tossica" per vedere fin dove si è disposti a spingersi eticamente per restare in azienda, dormendo sotto le scrivanie.

Monitor Celestra. Un 360° **fantascientifico** basato su *Battlestar Galactica*: per giocarlo è stato noleggiato un **incrociatore svedese** in disarmo. Una nave, non un'astronave, "ma se non usciamo all'esterno, ci mettiamo un paio di LED, ed ecco qua": per i giocatori è un'astronave. Il gioco durava circa **60 ore**, con cabine allestite in modo che si dormisse in personaggio.

Demeter. Un 360° sul viaggio della bara di **Dracula** dalla Transilvania a Londra, come nel romanzo di Stoker, in cui la nave *Demeter* arriva in porto con tutti morti a bordo e il capitano legato al timone. I giocatori sono i passeggeri, e **da giocatori sanno che moriranno tutti**: per design è impossibile sopravvivere, "altrimenti Dracula non esisterebbe come romanzo". Si gioca in costume su una **nave**

[28] La trascrizione dice "360 o High Resolution" e, più avanti, "IRES 360", "iRes", "Ires".

vera che, naturalmente, non va dalla Transilvania a Londra ma gira nel mare della Germania del nord; narrativamente, però, si gioca quel viaggio.

Bunker 101. Un caso **italiano**, ispirato alla proprietà intellettuale di *Fallout*: si giocava **72 ore** in un bunker. Qui il dormire non era diegetico: nell'ultima run hanno separato la parte di gioco dal sonno. Per tutto il resto, però, valeva il principio 360°: tutto ciò che si vede deve essere diegetico.

2.2.16 Tape LARP e black box

Accanto al Jeepform ("in cui giochiamo in casa con i nostri costumi") e al 360° ("l'ultra-diegesi"), ci sono tipologie scandinave "molto più particolari e quasi astratte".

2.2.16.1 Il tape LARP

Visto che qualcuno conosce Lars von Trier: chi ha visto *Dogville*, il film con lo scotch per terra, sa da dove nascono i **tape LARP**. "Abbiamo visto Dogville, ci ispira un casino": giochi in cui **lo scotch per terra indica gli ambienti**, con tutte le meccaniche che ne derivano (il personaggio vede e sente cose che normalmente non potrebbe sentire; come si agisce e non si agisce da giocatori...). Ne sono stati fatti parecchi, in particolare in Danimarca.

2.2.16.2 Il black box LARP

Ruggero è "un particolare amante" del **black box LARP**, che prende il nome dalle *black box*: per chi ha fatto teatro "dalla Germania in su", spazi teatrali di solito senza spalti né sedie, con **pareti, soffitto e pavimento neri**, molto diffusi in Germania e nel Nord Europa. A **Copenaghen** c'è un festival, il **Black Box LARP Festival**, che usa questi spazi per giochi creati apposta: giochi di ruolo che usano **luci e colori come meccaniche di gioco**. Gli esempi della slide:

Borderline. Un gioco "estremamente astratto". Si gioca **in coppie**, e ogni coppia gioca **un solo personaggio**, con dei conflitti interni: tutta la conversazione è un monologo interiore del personaggio, e i due giocatori, con una candela, lottano per portare ciò che il personaggio fa dalla propria parte o da quella dell'altro.

Waiting for Flight. Ruggero lo ha giocato. Si aspetta un aereo con a bordo persone care, amici, colleghi; **l'aereo precipita**, e muoiono quasi tutti. Si gioca la **sala d'attesa** di chi li aspettava: è un gioco sull'attesa, "sul logorante non avere informazioni" finché vengono centellinate. Si conclude con un momento molto catartico: arrivano le **valigie** delle persone attese, e in alcune (3 su 20) è nascosto un piccolo **cuore di vetro**: se lo trovi, la persona si è salvata; se no, non si è salvata. C'è una fase in cui speri che ci sia, ma potrebbe essere nascosto: "è una cosa lacerante. Abbiamo pianto in 19 su 20".

Stranger. Di una designer danese[29]. È un black box **non verbale**: è vietato parlare. I partecipanti sono divisi in due gruppi, i **blu** e i **verdi**, e prima del gioco ciascun gruppo crea un **linguaggio gestuale condiviso**: come la propria nazione esprime felicità, rabbia, paura, tristezza, amicizia, odio. Il twist: i verdi devono avere gesti squadrati, rigidi, geometrici; i blu gesti curvi, lenti, morbidi, sinuosi. Il gioco è sull'**immigrazione forzata**: ci sono fasi di giorno e di notte, e di notte chi facilita porta alcuni verdi nella nazione blu, dove si risveglieranno, con la regola che il confine non si può riattraversare. Le dinamiche: il primo che si risveglia tra i blu impara facilmente la loro lingua, e i blu lo aiutano con un *onboarding* non verbale; ma quando arrivano molti verdi, tendono a muoversi **in gruppi**, perché a quel punto c'è già chi parla la propria lingua, anche se non è la lingua della nazione. "Un microscopio" di dinamiche della vita reale: emigrazione, immigrazione, inclusione, e quanto è più facile essere inclusi se il sistema ti supporta.

Il gioco di Ruggero. Uno dei giochi della slide è suo, "quindi non ne parliamo", nonostante le proteste dei colleghi.

[29] La trascrizione dice "Nino Ressendrop": con ogni probabilità Nina Runa Essendrop, designer danese nota per i LARP fisici e non verbali. Anche il titolo, "Stranger", è riportato come nella trascrizione.

Un gioco su Cuba. Di una designer che è andata a vivere a **Cuba** e ha partecipato da vicino alla vita di una delle storiche comunità queer cubane[30]. Il gioco racconta che cosa è successo alle comunità **punk, rock e queer**, molto sovrapposte, prima, durante e dopo la caduta del Muro di Berlino: lo Stato, rimasto senza i fondi sovietici, decide chi è cittadino di serie A e di serie B, e le persone queer e punk (la musica occidentale) vengono picchiate e imprigionate. Erano anche gli anni dello scoppio dell'**AIDS**. Una comunità famosa, quella dei *frikis*[31] cubani (di cui sono ancora vive due persone), decise: "tanto ci ammazza la polizia, tanto vale iniettarci sangue infetto da HIV", per andare a morire non per strada ma nei **centri di recupero**, gestiti da medici stranieri, uno dei pochissimi casi in cui a Cuba poteva esserci personale non cubano. Lì si moriva, ma "almeno si poteva ascoltare musica punk", magari con il dottore che arrivava dalla Florida con le cassette rock, e tra simili. "Una roba drammaticissima": anche questo è un gioco devastante.

Tipo di LARP	**Caratteristiche ed esempi citati**
Boffer	nato per adattare D&D dal vivo; costume, armi in plastazote, combattimento e trame epiche; fino a centinaia o migliaia di persone
Mind's Eye Theatre	da *Vampire: The Masquerade*; costume, dark/gotico, intrigo politico più che combattimento
Chamber / parlor LARP	senza costume, accessibile, budget ridotto, spesso *print and play*, 3–20 persone (*A cena col nonno*)
American freeform	chamber LARP con metatecniche prese dal gioco da tavolo e dal GDR; anche in carte o libretti
Murder party	cena con delitto (gli attori indagano, l'assassino è un attore) o murder party "all'inglese" (si interpretano i sospettati)
Nordic LARP	coinvolgimento, emozione, coesione tematica, cura del dopo-gioco; manifesti; Knutpunkt dal 1997; Dogma 99
Jeepform	freeform nordico sul dramma quotidiano (il gioco della coppia perfetta, *Fat Man Down*); meccaniche di sicurezza
360° / alta risoluzione	tutto diegetico grazie a un budget altissimo (*Ground Zero*, *Panopticorp*, *Monitor Celestra*, *Demeter*, *Bunker 101*)
Tape LARP	lo scotch per terra indica gli ambienti, come in *Dogville*
Black box	spazi teatrali neri; luci e colori come meccaniche (*Borderline*, *Waiting for Flight*, *Stranger*)

Table 2.2.2: Le tipologie di gioco di ruolo dal vivo presentate da Ruggero, in sintesi.

2.2.17 Chi osserva? Il LARP e il teatro dell'oppresso

La domanda di Tiziana. Tiziana confessa un'impressione: tutti questi giochi "sembrano cose da studi", come se ci fosse qualcuno che sta sopra e osserva le dinamiche per ricavarne informazioni. Ruggero la rassicura: **non è così**. I designer di questa scena, specialmente quella black box, vengono dall'ambiente ibrido del teatro interattivo, degli *happening*, dello psicodramma, e **non documentano nulla di quel tipo**: non registrano le giocate, non estrapolano informazioni da chi gioca. Spesso sono giochi per poche persone (è difficile che un black box superi le 20), e molti hanno avuto **run singole**, o al massimo due o tre. "Si tratta solo di fare un certo tipo di esperienza". Come in altri media esistono generi con scopi diversissimi (un film e un documentario sono entrambi operazioni visive), anche qui gli scopi sono diversi. E chi non ha mai incontrato giochi del genere si chiederà "perché dovrei giocare una roba del genere?": la risposta arriverà con la lezione sull'emotional design ("lo so, ve l'ho detto quattro volte").

Il teatro dell'oppresso. Prima di passare la parola, uno studente interviene: in queste cose ritrova molto il metodo del **teatro dell'oppresso**[32]. "Ah, certo". Ruggero torna indietro di una slide, a quella sulle

[30] La trascrizione riporta il titolo come "Socialism and Word"; non è verificabile. Anche le date sono incerte ("nel 97, 89 e 91"): il contesto, la caduta del Muro di Berlino, fa pensare al 1987, 1989 e 1991.

[31] La trascrizione dice "Os Fricchios".

[32] Il metodo teatrale ideato dal regista brasiliano Augusto Boal (integrazione).

influenze fondanti del Nordic LARP: ci sono le tecniche di **Moreno** (sociodramma e psicodramma), il *sensitivity training*, il **roleplay come metodo di ricerca**, e tutti quei metodi che usano il teatro e l'interazione come strumento per far emergere il personale e l'intimo. Lo studente precisa che il teatro dell'oppresso nasceva per **trovare soluzioni** a situazioni che apparentemente non ne hanno: a turno il pubblico proponeva una soluzione, si rifaceva sempre la stessa scena, e si studiava come reagivano le persone; quindi, in un certo senso, il comportamento delle persone *si osserva*, ma dall'interno della scena.

Per Ruggero, nella scena LARP questa dimensione "viene un po' meno", perché lo scopo resta **ludico** ("e scopo ludico può voler dire molte cose, come vedremo"): è meno pragmatico del "prendiamo quello che abbiamo giocato e il gioco ci dà già gli strumenti per applicarlo". Alcuni **debriefing**, i momenti strutturati del dopo-gioco, possono dare strumenti a chi ha giocato; ma questi giochi nascono come esperienze che devono essere "ludicamente interessanti per chi gioca", **non come esperimenti scientifici "con dei topi dentro un labirinto"**.

Approfondimento: il LARP in classe e il debriefing. Che il LARP possa essere uno strumento formativo, come nel volume sull'edu-larp citato all'inizio, lo mostra anche *Challenges for Game Designers*. Nel capitolo sui giochi come strumento di insegnamento, Brathwaite e Schreiber propongono una sfida, *Living History*: progettare un LARP che rappresenti un evento storico (una convenzione costituzionale, un protocollo internazionale, una guerra) giocabile da una classe di 20–30 studenti in una o due ore. I passi suggeriti sono: scegliere un evento in cui molte parti, ciascuna con la propria agenda, si incontrano in una negoziazione o in un conflitto; scegliere i giocatori (o le squadre); definire gli **obiettivi** di ciascuno; creare regole semplici e rapide per risolvere i conflitti, perché i giocatori si muovono e parlano di continuo ("alcuni LARP usano un singolo tiro di dado o una partita a sasso-carta-forbici"); e preparare, oltre alle schede dei personaggi, delle **note di debriefing**: che cosa è successo davvero e quali domande discutere confrontando la storia con l'esito della partita. Lo scopo dichiarato è che gli studenti capiscano che "l'evento era complicato" e come l'esito sia *emerso* dalla situazione iniziale. È la versione didattica della cura del dopo-gioco che Ruggero attribuisce alla scena nordica.

2.2.18 I giochi di ruolo da tavolo: quattro parole per orientarsi

"Daniele, ti lascio la parola". Daniele ("mi giri tu le slide quando te le chiedo") avverte che fare una panoramica dei giochi di ruolo in poco tempo è "veramente difficile": ce ne sono tantissimi, e negli anni si sono evoluti e diversificati sempre di più, fino a diventare molto diversi tra loro.

2.2.18.1 Il tribalismo e le regole tramandate a voce

C'è anche un altro problema, che si vedrà nella lezione sulla fiction e che esiste in tutto il gioco analogico: il **tribalismo**, cioè l'**ereditarietà orale delle regole**. Spesso i giochi di ruolo venivano insegnati da altre persone, anche per motivi economici. Daniele ha cominciato così: da bambino, giocando per la prima volta a D&D, l'unico ad avere il manuale era il master; "ergo io non avevo accesso alle regole", e il master poteva inventarsi quello che voleva.

2.2.18.2 Conflitto, azione, risoluzione, effetto

Per capire un po' i giochi di ruolo bisogna ricordare **quattro parole**, che torneranno nella lezione sulla fiction: **conflitto**, **azione**, **risoluzione** ed **effetto**. Nella slide *conflitto* e *risoluzione* hanno un asterisco: per ora basta capirli quanto serve a leggere i casi studio, senza analizzarli troppo.

Conflitto. L'ostacolo, l'incertezza, l'evento che **stimola un'azione o la contrasta**. "Tornerà molto".

Azione. Quello che si descrive di voler fare nei panni del personaggio per risolvere un conflitto.

Risoluzione. Come si risolve l'azione dentro il gameplay, con le regole: che cosa succede?

Effetto. La conseguenza che si applica.

L'esempio: "voglio colpire il goblin con la mia spada". Il goblin ha il suo scudo e la sua armatura. Che azione fai? "Lo becco con la spada". Si tirano i dadi, si risolve, si applica un effetto. Ma non è così semplice, perché in mezzo c'è la **fiction**: nel gioco di ruolo si tende a riscrivere tutto quello che succede nel gameplay *dentro* la narrazione. Se faccio tre danni agli HP del goblin, che cosa vuol dire narrativamente? Come li descrivo? Da qui nasce "una serie di problematiche".

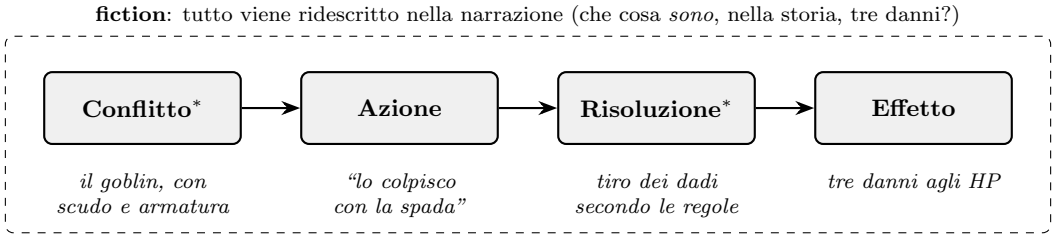


Figure 2.2.3: Le quattro parole di Daniele per leggere i giochi di ruolo da tavolo, con l'esempio del goblin. Gli asterischi sono quelli della slide: conflitto e risoluzione verranno analizzati meglio nella lezione sulla fiction. Nel gioco di ruolo "classico" il conflitto precede l'azione, la risoluzione è un tiro di dado e l'effetto è descritto dal master e dal regolamento; molti giochi successivi cambiano proprio questo schema.

2.2.19 Le origini: da Dungeons & Dragons agli anni Novanta

Le quattro parole servono a Daniele per una breve panoramica, anche storica, dei giochi di ruolo.

2.2.19.1 Dungeons & Dragons e il gioco di guerra

Il primo gioco di ruolo da tavolo "così come viene concepito" è sicuramente ***Dungeons & Dragons***, di **Gygax e Arneson**, nel **1974**. C'è in realtà un precedente, ***Chainmail***[33], e come per tutte le origini ("anche *Cyberpunk* ha lo stesso problema") c'è un po' di ambiguità su che cosa venga prima. Ci sono libri che raccontano la nascita di D&D, c'è la storia della **TSR**, la casa editrice che lo pubblicò per prima, ci sono versioni discordanti e, negli anni, parecchi contrasti tra le famiglie di Gygax, di Arneson e di altri.

D&D viene dal **gioco di guerra**, il gioco di **miniature**. "La leggenda vuole" che Gygax e Arneson, giocando, si siano detti: ho battuto il tuo esercito; che cosa succede se il mio generale entra nel tuo castello e va a trattare con il tuo generale? Abbiamo regole per questo? No: inventiamocele. "Ripeto, una leggenda": non si sa se sia andata così, e in fondo non interessa molto.

2.2.19.2 I generi e la struttura del gioco di ruolo "early"

Quello che interessa è che dopo D&D, nell'**RPG delle origini**, vengono coperti più o meno **tutti i generi letterari**: *Call of Cthulhu* l'orrore lovecraftiano, *Vampire: The Masquerade* l'orrore gotico, *Traveller* la fantascienza, *Cyberpunk* il cyberpunk, e *GURPS*, un **sistema generico** con cui "teoricamente" si può giocare qualunque genere.

Nel gioco di ruolo classico:

- il **conflitto** poteva venire prima o dopo l'**azione**: "voglio colpire quel goblin" (e il goblin ha la sua difesa), oppure "corri, ti sta inseguendo una palla", come in *Indiana Jones*;
- la **risoluzione** era il tiro di dado, e l'**effetto** veniva descritto dal master e dal regolamento;
- in pratica, **tutto tranne l'azione**, che il giocatore gestiva e narrava, era gestito dal **master**.

[33] La trascrizione dice "C'è in Mail" e data *Chainmail* al '72; il gioco è del 1971.

Il master come arbitro. All'inizio il master non era un "maestro" né una figura che insegna: era interpretato soprattutto come un **arbitro**. Nella prima versione di D&D non c'è ancora l'idea, venuta dopo, del master come "dio del tavolo": quando deve decidere, dovrebbe farlo secondo criteri di realismo e oggettività ("puoi essere oggettivo a un tavolo di gioco? La voglio vedere"). Questa idea è importantissima perché verrà ripresa dall'**OSR**, l'*Old School Revival*, il movimento che cerca di riportare nella modernità il modo in cui, a suo dire, si giocava D&D agli inizi.

Il master è un giocatore. Nei giochi di ruolo ci sono quindi **ruoli asimmetrici**. Ma Daniele raccomanda di ricordare una cosa, che tornerà nella lezione sulla fiction: **il master è un giocatore**, magari un giocatore con più regole e più autorità al tavolo, ma "un giocatore tanto quanto i giocatori che interpretano un personaggio". Nei GDR delle origini: si interpreta un personaggio, con una **scheda** e le sue statistiche, dentro una fiction e un **setting** stabilito dal manuale o dal master; quando si dice che cosa si vuole fare, il master reagisce. E lentamente, con gli anni, il master assume anche il ruolo di **regista** e **sceneggiatore**: scrive l'avventura, decide che cosa succede, decide i tagli di scena.

Approfondimento: dal wargame al contratto sociale. La genealogia raccontata da Daniele si ritrova in *21st Century Game Design* di Bateman e Boon. Le radici del gioco di strategia, scrivono, precedono i videogiochi di decenni: alla fine degli anni Cinquanta Charles Roberts fonda Avalon Hill e pubblica i primi **wargame da tavolo** (1958); in un'epoca senza videogiochi, giochi di ruolo e giochi di carte collezionabili, i wargame "fondano l'industria dei *hobby games*", e *Chainmail* (1971), ispirato a quei giochi, è il "trampolino" verso *Dungeons & Dragons*. Gli stessi autori descrivono la forma minima del gioco di ruolo da tavolo come un **contratto sociale** tra l'arbitro (il *games master*) e i giocatori: i giocatori accettano che sia l'arbitro a decidere come le loro azioni sviluppano la storia, e l'arbitro si impegna a garantire intrattenimento o imparzialità, come mediatore e narratore principale. Citano anche un'osservazione attribuita a Gygax: se i giocatori di ruolo si accorgessero di poter fare tutto da soli, i game designer resterebbero senza lavoro. Il fatto, notano, è che la maggior parte dei master ha bisogno di meccaniche per gestire il mondo, e i giocatori si fidano di più dell'arbitro quando una struttura di meccaniche casuali, i dadi, fa da cornice. Anche Schell, in *The Art of Game Design*, cita il master dei giochi di ruolo da tavolo come esempio di stato del gioco *nascosto*: è lui a conoscere privatamente gran parte dello stato del gioco, perché ne è "il meccanismo operativo".

2.2.19.3 Gli anni Ottanta e Novanta: adattare un regolamento a tutto

Qui Daniele fa un "piccolo inciso editoriale". Nella grande espansione degli anni Ottanta e Novanta **non c'era un professionismo** del settore. Gli editori compravano delle proprietà intellettuali, per esempio **007**, prendevano un master appassionato e competente e gli chiedevano di **adattare** il regolamento di D&D, o un altro regolamento esistente[34], a quella IP. A volte nascevano giochi innovativi, a volte giochi "non molto interessanti da giocare": immaginate di giocare con le regole di D&D qualcosa che quelle regole non supportano. Verso la fine degli anni Novanta il problema della qualità "ha iniziato a farsi sentire".

Approfondimento: James Bond 007 e l'esperienza essenziale. Proprio il gioco di ruolo di 007 è il caso con cui Schell illustra, in *The Art of Game Design*, la **lente dell'esperienza essenziale** (*Lens of Essential Experience*), e mostra l'altra faccia della storia di Daniele. Il designer Chris Klug era frustrato dai precedenti giochi di ruolo di spie, come *Top Secret* della TSR, perché "si giocavano troppo come dei wargame": l'essenza di ciò che rende eccitanti i film di spionaggio non c'era. Per il gioco di James Bond, Klug progettò le meccaniche in modo che restituissero in ogni modo possibile la sensazione dei film. L'esempio è l'invenzione degli **Hero Points**: nei GDR tradizionali, davanti a un'azione rischiosa (saltare da una finestra su un elicottero in volo), il master calcola una probabilità, il giocatore tira i dadi "e questo è quanto"; ma se la probabilità è bassa nessuno rischia, e se è alta tutti si comportano da supereroi. I punti eroe risolvono il dilemma in modo coerente con il genere. È esattamente quello che la New Wave rivendicherà: **il sistema conta**, e un regolamento adattato "a nastro" non restituisce l'esperienza di un genere.

[34] La trascrizione dice "Call of Duty 2", che non può essere corretto in questo contesto; il senso è "il regolamento di un altro gioco già esistente".

2.2.20 La New Wave e The Forge

2.2.20.1 Il sistema conta

La slide successiva è sulla **New Wave**: un movimento nato tra la fine degli anni Novanta e l'inizio dei Duemila. Daniele avverte che non vuole fare un'analisi geografica ma "un riassunto della storiografia". La New Wave cerca un **metodo diverso di interpretare il gioco di ruolo**, e la prima cosa che stabilisce, in un forum chiamato **The Forge** (c'è anche un libro in inglese su questo movimento), è che **il sistema dei giochi di ruolo è importante**: le regole, il gameplay, la parte meccanica **non sono un orpello**. Non si può tenere lo stesso sistema, cambiare il setting e sperare che funzioni da sé. Tanto è vero che perfino *GURPS*, il sistema generalista, aveva tantissimi manuali di espansione delle regole per ogni genere: *GURPS* base, *GURPS Fantasy*, *GURPS* fantascienza...

2.2.20.2 I giochi della New Wave

Dentro la New Wave nascono esperienze diversissime, che spostano l'attenzione su due cose: la **scrittura del manuale**, diversa da quella tradizionale, e la **rottura degli schemi**.

- ***Apocalypse World***[35], di **Vincent Baker**, cerca di scardinare il modo classico di strutturare un manuale di gioco di ruolo (Daniele accenna a una nuova edizione recente).
- ***Nobilis***, uno dei primi giochi abbastanza famosi **diceless**, senza dadi: azioni e conflitti si risolvono in un altro modo.
- ***Don't Rest Your Head*** (in italiano *Non cedere al sonno*) e ***Polaris***, che lavorano sul concetto di chi è e chi non è il personaggio. *Non cedere al sonno* è un gioco "surrealista" in cui ci si sveglia a **Mad City**; la parte importante sono i **poteri**. All'epoca *Vampire* aveva stabilito che i poteri fossero una lista di possibilità con statistiche a cui applicare le regole; in *Don't Rest Your Head* ci sono poteri "totalmente assurdi, fenomenali", che non richiedono meccaniche in più ma usano le meccaniche base "in maniera molto innovativa".
- ***Fate***, un sistema anch'esso generalista, "più o meno come *GURPS*", ma con delle sue specifiche: ci sono **procedure per creare i setting** e per stabilire quali regole un setting richiede.
- ***Dread***, uno dei primi giochi a usare per l'incertezza **un elemento che non è un dado**: una **torre Jenga**. È un gioco dell'orrore in cui ogni volta che si fa un'azione si toglie un blocchetto, e se la torre cade il personaggio muore ("non è proprio così, è più complicato, però per semplificare").

Le guerre di religione. La New Wave ha avuto "una serie di problematiche" in giro per il mondo, e soprattutto in Italia ha creato una sorta di **guerra teologico-filosofica** tra i "tradizionalisti" di D&D e i "forgiti" della New Wave. "Per vostra sanità mentale non ci entriamo".

Una reazione a due cose. La New Wave è stata una reazione:

1. ai manuali in cui "c'è un sistema e quello è il sistema", con il setting applicato sopra "a nastro", senza intreccio tra la narrazione e le regole che la raccontano: si voleva **coerenza**;
2. all'assenza di studio: è stato il **primo movimento a voler studiare i giochi di ruolo**, a capire come sono fatti. Da qui viene il **Big Model**, un modello di analisi dei giochi di ruolo di cui si vedranno alcune cose nella lezione sulla fiction.

Ma il Big Model "non è fondamentale", perché per Daniele uno dei problemi della New Wave è aver creato **un dizionario collaterale** a quello del game design del gioco da tavolo e del videogioco: termini diversi che dicono la stessa cosa.

[35] La trascrizione dice "A Polly's Ward" e "Minster Baker".

2.2.20.3 The Forge e la scena nordica

Ruggero fa da ponte: **The Forge ha fatto per il gioco di ruolo da tavolo quello che la scena scandinava ha fatto per il LARP**. Inventavano e scrivevano moltissimo, hanno lasciato framework e strumenti. La scena scandinava è nata un po' prima, The Forge un po' dopo, ma hanno agito in parallelo. La differenza è che The Forge era **meno strutturato**: essendo un forum, era più caotico, con discussioni anche accese, soprattutto sulle interpretazioni. Anche la **preservazione storica** è diversa: della scena nordica ci sono i libri fisici, mentre per il forum bisogna passare dalla Wayback Machine. Ma in termini di normalizzazione di termini, meccaniche e framework, entrambi hanno fatto moltissimo.

2.2.20.4 Fiasco

Daniele riprende: a un certo punto The Forge chiude, e nascono altri giochi dagli stimoli e dalle discussioni che quell'ambiente aveva generato, non necessariamente da persone che ne facevano parte. Per esempio quelli di **Jason Morningstar**, noto anche per dei chamber LARP (il migliore amico di Daniele, Edoardo, "più esperto di me", ne è talmente fan che il suo gatto si chiama Giasone).

Fiasco è per Daniele uno dei primi giochi che **rompe il canone**. Racconta, appunto, un fiasco, come nei film dei **fratelli Coen** o di Tarantino: c'è uno scenario, ci sono dei personaggi, e ogni volta che si fa una scena **il giocatore di turno sceglie se aprirla** (e gli altri la risolvono) **o se lasciare che gli altri la impostino** (e risolverla lui). Già solo questo "micro elemento" (non l'unica meccanica: c'è anche quella, interessantissima, dei dadi) rompe l'idea che in un gioco di ruolo si debba descrivere l'azione, tirare i dadi e vedere che cosa succede, oppure reagire a ciò che descrive il master. Per chi veniva da *AD&D* seconda edizione o da *Vampire* era "assolutamente interessante": più leggero e più legato alla fiction. Anche i **dadi** sono usati in modo nuovo: vengono tirati **prima**, messi sul tavolo, selezionati, usati quasi come **token**: un'innovazione anche nel modo di usare i materiali.

2.2.21 Il gioco di ruolo contemporaneo

2.2.21.1 Prima del COVID: i giochi focalizzati

Daniele salta alla contemporaneità. Prima del COVID c'è un grande aumento di giochi che, mentre le persone si appassionano ai giochi di ruolo, vanno o nella direzione di D&D o in una **direzione più ibrida**:

- ***Blades in the Dark***[36];
- più recenti, il gioco di ruolo di ***Cowboy Bebop***[37] e ***The Clay That Woke*** di **Paul Czege**[38], autore di "uno dei principi di design più interessanti";
- ***Fabula Ultima***, di **Emanuele Galletto**, italiano ("che salutiamo").

Sono prodotti che **non cercano di fare la stessa cosa di D&D**. Hanno un impianto simile (scheda del personaggio, dadi, azioni descritte, tiri), ma si pongono **obiettivi di design più coerenti**: "si chiedono che cosa vogliamo che il giocatore senta". Il sistema di *Cowboy Bebop*, per esempio, usa gli **orologi** (*clock*) che si trovano anche in *Blades in the Dark* e nei giochi *Powered by the Apocalypse*, cioè la stessa meccanica, ma è un sistema completamente diverso, perché deve raccontare una fiction diversa:

- in *Blades in the Dark* si giocano dei **criminali** a **Doskvol**[39], città di un mondo alternativo dove è sempre notte, dove i fantasmi non passano nell'aldilà e vivono con i vivi, e dove si è costretti a restare perché fuori dalle mura i fantasmi uccidono. Si fa parte di una banda che cerca di scalare la gerarchia della città, "tipo *Peaky Blinders*";
- in *Cowboy Bebop* si è "letteralmente una ciurma di persone nello spazio, nel futuro, che fanno cose", perché "l'anime si racconta con i personaggi che fanno cose" (Daniele, "molto fan dell'anime", lo dice come un complimento).

[36] La trascrizione dice "Blaze in the Dark". Daniele lo data "2016–18 più o meno".
[37] La trascrizione dice "Cabobby Bop" e "Cobo Bobby".
[38] La trascrizione dice "Paul Ksege, The Clayed Walk".
[39] La trascrizione dice "Oswald".

2.2.21.2 Dopo il COVID: il ritorno all'origine

Soprattutto dopo il COVID si è notato un "ritorno fortissimo" al **giocare come una volta**. Per Daniele entrambe le tendenze rispondono a una necessità: **semplificare**. *D&D* 3 e 3.5, le nuove edizioni di *Vampire* e altri sono diventati "pieni di regole, pieni di contenuti", e chiedono a master e giocatori di imparare moltissimo, senza contare il tempo necessario per una campagna. Allora, da un lato, si cercano sistemi più semplici "da intavolare" che facciano bene una cosa; dall'altro si cerca la stessa cosa per altri motivi, ricreando le atmosfere del D&D *old school*: **alta mortalità**, parte narrativa e interpretativa meno importante, immediatezza. ***Mörk Borg***[40], per esempio, sono circa **cento pagine**, con poche regole. C'è più attenzione al "realismo" (termine un po' sbagliato, si vedrà nella lezione sulla fiction), e il vantaggio è che questi giochi possono usare sia le proprie regole sia quelle di **D&D quinta edizione**, che è stata importante soprattutto "per un motivo editoriale" (anche questo si vedrà nella lezione sulla fiction).

La domanda di Gianmarco. Gianmarco chiede se questa tendenza a manuali e partite "asciutti" segua un pubblico più adulto, con meno tempo, o sia una tendenza generale del mercato. È soprattutto la prima. Il gioco di ruolo ha avuto parecchie difficoltà ad attirare nuove leve, e i giocatori sono cresciuti. Daniele gioca da molti anni, ma il suo "vero inizio" è stato con un gioco della New Wave[41]: per lui, all'università, avere in un solo manuale da 200 pagine tutte le regole e tutti i personaggi è stato "un toccasana". Chi inizia con D&D, magari alle superiori o all'università, man mano che cresce tende a preferire:

- o giochi più focalizzati sull'aspetto **narrativo**, che hanno comunque scheda, tiro di dado e struttura tradizionale, ma una scheda e un sistema più semplici;
- o un'esperienza simile al D&D canonico, ma con meno "regoline e regolette" e meno **simulazionismo**.

Due differenze pratiche. I manuali **OSR**, che un po' di simulazionismo ce l'hanno, sono pieni di **soluzioni per l'arbitro** ("che non è il master, è l'arbitro") per risolvere i problemi sul momento; e l'OSR è **sandbox**: c'è una mappa, una zona in cui si va dove si vuole, e il master non ha scritto un'avventura. I **giochi narrativi**, dal canto loro, non hanno una campagna scritta ma uno **scenario**. In entrambi i casi il tempo di preparazione del master si riduce "di gran lunga".

2.2.21.3 Focalizzati, non semplificati

Ruggero interviene sulle parole: non vorrebbe che passasse l'idea che questi giochi siano "più semplici" o *dumbed down* rispetto a D&D. Si parla di **giochi focalizzati**: invece di semplificare, **focalizzano**, sono molto più **coesi tematicamente**, hanno "un cuore tematico molto forte" e si giocano per giocare quello. D&D invece, a livello di cuore meccanico, è giocato in modi diversissimi da dieci gruppi diversi.

Daniele fa un esempio: nei manuali di D&D (dalla terza edizione e 3.5 alla quinta) ci sono le regole per il **danno da caduta**. In *Blades in the Dark* non servono: c'è un altro metodo, "non è così generalista, è focalizzato", e la narrazione si risolve in altro modo. Anche nell'OSR: *Shadowdark* ha il danno da caduta, perché magari è importante; *Neverland*, una reinterpretazione di **Peter Pan** in chiave OSR, no, perché lì conta di più che il manuale offra tutti i personaggi del Peter Pan tradizionale in una versione *grimdark*. "Anche l'OSR diventa un po' focalizzato quando ci prova".

2.2.21.4 Not the End

Nicola, per cui questo è "il settore che più adoro del gioco analogico", chiede dove si collochi ***Not the End***, un gioco italiano. La risposta è nella slide successiva. *Not the End* continua nella direzione più narrativa e focalizzata ("semplicistica è un termine non corretto"): ha procedure, meccaniche, un gameplay focalizzato a fare quello che il gioco chiede. Si possono creare i propri setting, ma il gioco resta quello: il personaggio principale è un **eroe**, ed è il giocatore a decidere quando sarà la sua fine, quando giocherà il suo momento di eroismo stoico (da cui il titolo). È un gioco **senza dadi**: c'è un conflitto, si fa e si descrive un'azione, e la **risoluzione** consiste nell'**estrarre dei token** bianchi e neri (o arancioni e viola, per le persone con difficoltà

[40] La trascrizione dice "Morg Borg".

[41] La trascrizione dice "Paura dell'Epoclips": probabilmente *Il mondo dell'apocalisse*, l'edizione italiana di *Apocalypse World*, ma non è certo.

nella visione dei colori); l'effetto viene descritto secondo le procedure del gioco. Il panorama, insomma, si è diviso: c'è chi cerca un'esperienza più simile all'originale, e chi un'esperienza più focalizzata sulla narrazione, magari senza un *world building* preciso.

2.2.21.5 Che cos'è l'OSR, e com'è nato davvero il gioco di ruolo

Un'altra studentessa chiede che cosa significhi OSR: **Old School Renaissance**, "rinascimento della vecchia scuola"[42]. È un movimento "molto, molto particolare": ogni persona che ne fa parte ha il proprio principio di OSR, gioca un OSR diverso dagli altri, e gli appassionati "adorano scontrarsi" su che cosa intendono per OSR. Per questo nella lezione sulla fiction Daniele parlerà di **culture** del GDR, più che di community.

La stessa studentessa (iscritta a un altro corso dell'accademia, di cui sta recuperando le lezioni del primo anno) riporta quello che ha capito dell'origine di D&D: i due autori avrebbero avuto visioni opposte, uno più libera e meno regolamentata, partendo dai giochi di guerra, l'altro, che ha portato il gioco a livello commerciale, più strutturata, dettagliata e pesante. "È giusto, è un riassunto corretto, ma è un po' più complicato di così", e lo si vedrà nella lezione sulla fiction. Ruggero aggiunge che D&D "non è l'unica origine del gioco di ruolo": il gioco di ruolo diventa *famoso* con D&D, ma gli stessi autori si ispirarono alle **simulazioni militari**, molto diffuse nell'America di quegli anni, e al **reenactment storico** ("ci sono due episodi della *Signora in giallo* sul reenactment della guerra civile, un argomento allucinante"). Esistono esempi storici, dell'Ottocento e perfino del Seicento, di **giochi di ruolo** veri e propri, con quest, avventure e personaggi. Il *mainstream* ha normalizzato l'idea che D&D sia il primo gioco di ruolo, "ma non è così": nel manuale originale l'espressione *role-playing game* non compare da nessuna parte, anche perché Gygax odiava il termine. "Un vaso di Pandora che non apriamo".

Daniele però raccoglie l'osservazione. Nei primi anni editoriali di D&D il problema maggiore è stato il **simulazionismo**: interpretare "simulazione" come la necessità di avere una regola per ogni cosa che esiste nel realismo del gioco. Ne è venuto un gioco molto strutturato, in cui "ogni cosa deve avere una regola", perché ciò che non è coperto dalle regole il giocatore può usarlo come *loophole*. E ne è venuta anche l'idea che il master sia il **padrone del tavolo** e che il giocatore debba in qualche modo **vincere contro il master**: un altro "vaso di Pandora enorme".

Non esiste solo D&D. È per questo che Daniele non si concentra solo su D&D: "l'idea con cui dovete uscire da qui è che non esiste solo D&D". Anzi, oggi D&D è **in controtendenza**, non per motivi di design ma per **motivi editoriali**: il problema principale di D&D come proprietà intellettuale "è sempre stato editoriale", e di recente la Wizards of the Coast ha fatto un po' di confusione con il *branding*.

2.2.22 Story game e giochi di ruolo sperimentali

2.2.22.1 Gli story game

La slide successiva è sugli **story game**, che in realtà nascono anche prima di The Forge. "Story game è un termine che io odio, perché vuol dire tutto e non vuol dire niente": sono giochi di ruolo da tavolo che si focalizzano **ancora di più su una storia precisa**.

- ***Alice is Missing***: la storia di Alice, scomparsa, e dei suoi amici che cercano di capire che cosa le è successo. Si gioca con una **chat** sul telefono, con un video a schermo, e con **due ore di tempo**.
- ***Microscope***: "letteralmente *world building* narrativo".
- ***Bleak Spirit***: giocare un *Dark Souls* in cui tutti controllano a turno il viandante protagonista.
- ***The Zone***[43], giocabile anche con un supporto online: si narra, di base, *Annientamento* ("se avete visto il film"). Si entra in una zona in cui una presenza aliena sta cambiando il mondo, e **moriranno tutti tranne una persona**, che non si sa in anticipo chi sarà. È un'influenza del LARP: come in *Demeter*, si sa che il proprio personaggio morirà.

[42] Poco prima Daniele aveva sciolto la sigla come *Old School Revival*; entrambe le forme sono in uso.
[43] La trascrizione attribuisce il gioco a "Ruff D'Amico"; il nome non è verificabile.

- ***Archipelago*** e ***For the Queen***, e altri giochi che **tolgono i dadi** e mettono le carte, o tolgono perfino la **scheda del personaggio**: in *Microscope* ci sono solo dei post-it.

2.2.22.2 The Quiet Year e i map drawing game

"C'è un minimo di suspense": si arriva a esperienze ancora più lontane. In ***The Quiet Year*** non si interpreta nemmeno un personaggio: si è parte di una **comunità post-apocalittica** e si gioca **un anno** della sua vita. Non si può discutere di ciò che si gioca. A ogni turno si gira una **carta** che dice che cosa succede, e durante il proprio turno si può fare una tra due o tre azioni (disegnare, iniziare un progetto...). Tutto si gioca su una **mappa** su cui si disegna ciò che si vuole intendere: c'è chi gioca in modo più rigoroso, senza poter nemmeno scrivere parole sulla mappa, e chi lo permette. È "un gioco fenomenale", perché fa capire che il gioco di ruolo può essere anche altro; e ha normalizzato un genere, quello dei **map drawing role-playing game**, i giochi di ruolo in cui si disegnano mappe.

2.2.22.3 Dialect

E poi ***Dialect***. Qui si ha un personaggio, ma è solo un nome e un ruolo, un archetipo. Si gioca **la morte di una comunità attraverso la morte di una lingua**: una comunità umana isolata dalle altre, che ha creato il proprio linguaggio, e che giocherà la fine di quell'isolamento attraverso la morte del linguaggio. A ogni turno si gioca una carta che è un **concetto della lingua** (per esempio: come si dice "amico" nella nostra lingua?); si stabilisce la parola e si gioca una breve scena in personaggio, senza tiri di dado né altro; quando la parola viene usata in personaggio, chiunque al tavolo può dire stop: "abbiamo usato la parola, va bene così", e si va avanti.

Non è un gioco "non complesso", come diceva Ruggero: *Dialect* è "chiarissimo" da leggere, ma la complessità sta nel **design**, un design elegante che crea una *one shot* con "una profondità" enorme. Daniele con *Dialect* ha pianto; e quando incontra gli amici con cui lo ha giocato anni fa, si parlano ancora nella lingua inventata allora: "ehi tu, oceano", e chi c'era sa che cosa vuol dire. Anche qui si vede come le esperienze nate dal Nordic LARP siano **"pervase"** ("rubo una parola a Beatrice") in altri ambiti.

2.2.22.4 Servono davvero le categorie? Il problema editoriale

Ci sono ancora esperienze più tradizionali; "il mondo è molto variegato", e si parlerà di **culture**: spesso chi si avvicina al gioco di ruolo vuole giocare una certa cosa, una cultura di gioco con titoli che "vanno bene" e titoli che no. L'ultima slide è "una piccola critica": è davvero necessario mettere le categorie ai giochi di ruolo? Da un lato sì, dall'altro no, perché i problemi principali sono due, e il primo è **il mondo editoriale**: i giochi di ruolo sono stampati come **libri**, e l'editore ha "una vita travagliata" con il gioco. Esempi:

- ***In the Name of God***[44], stampato solo in **self-publishing** e assente dal mercato, "ma bellissimo"; la sua meccanica di risoluzione chiede di fare qualcosa fisicamente al tavolo (la trascrizione dice "fare home");
- un'**espansione per qualsiasi gioco di ruolo**[45];
- un **GDR stampato su una maglietta**, di due autori che Daniele saluta;
- un gioco di **cultisti contro investigatori** che prende da esperienze di chamber LARP[46].

2.2.23 Giochi di ruolo per due, in solitario e micro

L'ultima parte sui giochi di ruolo torna a Ruggero, che presenta tre formati.

[44] Titolo e autore ("Alessandro Pirodi") come nella trascrizione; non verificabili.
[45] La trascrizione dice "Mew Wild in the Subway".
[46] La trascrizione lo chiama "SOT".

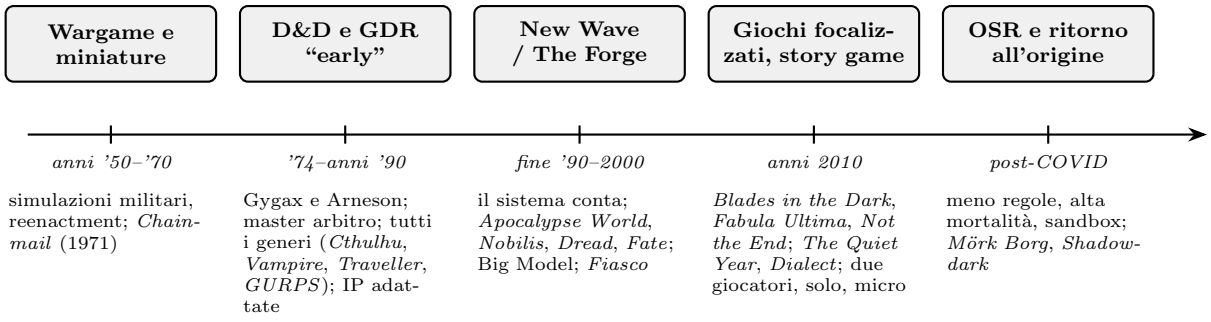


Figure 2.2.4: Una ricostruzione schematica della "piccola storiografia" dei giochi di ruolo da tavolo raccontata da Daniele. Le epoche sono indicative e si sovrappongono (gli story game, per esempio, nascono anche prima di The Forge); la storia completa è rimandata alla lezione sulla fiction.

Two player only. Storicamente i giochi di ruolo nascono in gruppi di gioco ampi, ma c'è stata "una grossissima wave" di giochi di ruolo **per due persone**, e non nel senso che una fa il master e l'altra un personaggio: spesso l'**agency narrativa**, il potere di decidere che cosa succede, viene **passata meccanicamente** al tavolo o divisa in responsabilità diverse. Ce ne sono tantissimi, e per Ruggero sono molto interessanti anche perché, come si vedrà nella lezione sull'emotional design, permettono di fare cose molto **intime**: non serve un gruppo di gioco, si possono giocare con il partner o con un'altra persona.

Solo RPG. Dal 2013 circa si sono normalizzati i **giochi di ruolo in solitario**: si gioca da soli, senza un'altra persona e senza fare da master a sé stessi. Spesso non c'è un master: ci sono **procedure** che guidano il gioco, e il manuale è il sistema di gioco, strutturato "quasi come un board game" in termini di regole. Ma resta tutto ciò che è tipico del gioco di ruolo: l'agency emergente, l'interpretazione di un personaggio, la creazione di una narrazione. Di solito si giocano in formato **diario** (*journaling*) o in formato **audio**, e sono in grande tendenza.

Micro RPG. Infine, i **micro giochi di ruolo**[47]: giochi di ruolo su **biglietti da visita**, o raccolte in cui un libro ne contiene cento. Il **200 Word RPG** (*200 Word RPG Challenge*) era un contest che chiedeva di scrivere un gioco di ruolo in **massimo 200 parole**, e ogni edizione ne raccoglieva migliaia: nel 2017, ricorda Ruggero, circa 900 giochi, su circa mille proposte.

2.2.24 I giochi da tavolo: i generi storici

"Passerei ai giochi da tavolo, correndo velocemente e senza fare pause, se ce la fate". Come per i LARP e i giochi di ruolo, Ruggero parla di **categorie dall'alto**, e per ognuna mostra come il genere si sia **evoluto** nel tempo.

2.2.24.1 Roll and move

I **roll and move** si basano, come dice il nome, sul **tirare un dado e muoversi**. Sono forse i più comuni nella storia dell'umanità, con quello che da noi è il **gioco dell'oca**: un tracciato su cui si tirano i dadi e si avanza. L'**agency** in questi giochi è "praticamente inesistente", e il genere si è presto evoluto integrando altre meccaniche: con **elementi economici** (come *Monopoly*) o con **elementi deduttivi**, come *Cluedo*, che è un roll and move.

2.2.24.2 Astratti

Un altro genere storicamente gigantesco è quello degli **astratti**: scacchi, dama, go, tutti quei giochi la cui componente tematica è ridotta "quasi all'osso". Gli scacchi mantengono ancora qualche forma di tema; il

[47] La trascrizione dice "handheld RPG"; dal contesto si tratta dei *micro RPG*.

go, di fatto, è un rettangolo di legno con delle caselle e delle pietre bianche e nere. Anche qui il genere non si è fermato: ci sono iterazioni moderne come *ZÈRTZ* e i giochi del **progetto GIPF**[48], e altri astratti contemporanei (uno è mostrato a destra nella slide[49]), che mantengono dinamiche "molto asciutte, molto distillate": **informazione aperta**, trasparente, niente carte imprevisti, niente mosse segrete. Diventano quasi una **sfida d'intelletto** con l'avversario.

2.2.24.3 Party game

I **party game**: *Taboo*, *Lupus in Tabula* o *Werewolf* a seconda dell'edizione. Sono pensati per **molte persone** (di solito dalle quattro in su), con regole "molto asciutte, molto leggere", perché si giocano in contesti di festa, in cui non è detto che chi è nella stanza ami giocare o abbia mai giocato. Poche regole, leggerezza, durata breve (spesso non più di mezz'ora): sono **un possibile punto di accesso**. Anche questi si sono evoluti, con giochi come *Hitster*[50] e *Wavelength*, che portano sensibilità più moderne e meccaniche più interessanti del "tirare alla cieca per capire chi tra di noi è un lupo mannaro".

2.2.24.4 Giochi di parole

I **word game**[51]: *Boggle*, *Scarabeo*, giochi in cui la meccanica centrale è **creare parole**, e più la parola è lunga, complessa o usa lettere rare, più punti si fanno. Anche qui le iterazioni sono interessanti: *Cross Clues*[52] trasforma il genere in un gioco **cooperativo**, in cui non si compete per la parola più lunga ma si gioca con le parole per capire "quanta intesa c'è tra di noi"; e *Illiterati* è praticamente un gioco con le *boss fight*, in cui si sconfiggono i boss formando parole. Ibridazioni anche qui, perché chi crea giochi li crea in base a ciò che ha giocato prima.

2.2.24.5 Giochi di carte

I **card game**, che in Occidente si diffondono storicamente con i mazzi **francesi** e, in Italia, con quelli "da briscola" (ma ci sono anche altri casi storici[53]). Le evoluzioni:

- i **micro card game** come *Love Letter*, che in una decina o dozzina di carte racchiude un gioco complesso;
- giochi come *Munchkin*, in una direzione più ampia, con il **take that** ("voglio mettere i bastoni tra le ruote" agli altri);
- la "gigantesca categoria" dei **trading card game**, già menzionata nella lezione precedente: *Magic: The Gathering*, *Yu-Gi-Oh!*, *Pokémon*. Il *trading* non è solo una dinamica tra persone: è nel nome della categoria. Scambiare carte **fuori dalla partita** è parte della dinamica comunitaria;
- i **Living Card Game** (LCG), che provano a risolvere uno dei principali problemi editoriali dei TCG: "se io ho più soldi di te, tu contro di me a Magic non vinci", perché comprerò carte da 800 o 900 euro l'una. Gli LCG **non hanno booster casuali**: ogni espansione ha l'elenco preciso del contenuto. Restano giochi di carte con il **deck building pre-partita**, con mazzi "fortemente identitari" (si possono scegliere le carte anche solo perché sono più simpatiche), ma senza l'aspetto economico della **rarità indotta**;
- i **deck-building game**, in cui la costruzione del mazzo **non è più pre-partita ma avviene durante la partita**: spesso si parte con mazzi identici, e durante il gioco si acquisiscono carte che cambiano **permanentemente** il proprio mazzo, con lo scopo di costruire il mazzo migliore possibile.

[48] La trascrizione dice "Zerz o il Gift Project".
[49] La trascrizione non permette di ricostruirne il titolo ("i borsa astratti, per esempio, di Supernova").
[50] La trascrizione dice "Hit"; la ricostruzione è probabile ma non certa.
[51] La trascrizione dice "war games", ma gli esempi (*Boggle*, *Scarabeo*) chiariscono che si tratta dei giochi di parole.
[52] La trascrizione dice "Cross Cruise".
[53] La trascrizione cita i "Mazda 1"; il riferimento non è ricostruibile.

2.2.24.6 German game ed Eurogame

Altre categorie, che Ruggero mette tra virgolette, sono i **German game**: un genere sviluppato in **Germania** dopo la metà degli anni '50–'60, con giochi a **interazione indiretta**, competitivi ma "senza troppo dar fastidio direttamente all'altra persona": non ci si può attaccare direttamente. Sono giochi poco bellici, anche per **rigetto della Seconda guerra mondiale**, e si è sviluppato moltissimo il filone **comunitario**, di *community building* e di scambio: costruire la propria fattoria, costruire un villaggio, scambiarsi le risorse ("io ho delle risorse, tu altre, scambiamocele"), senza distruggere ciò che l'altro ha costruito.

Il genere si è poi reso più complesso e si è espanso oltre la Germania con gli **Eurogame**, che includono Spagna, Portogallo, Francia e anche l'Italia (Ruggero ne ha fatti parecchi). L'interazione resta indiretta, ma i giochi diventano molto più complessi: spesso durano **due o tre ore** e provano a rendere **una simulazione focalizzata di qualcosa del mondo reale**. Nella slide, a sinistra, un gioco in cui si gestisce una **galleria d'arte**[54], con tutte le dinamiche di compravendita di quadri; e *Endeavor*, che Ruggero ha alle spalle, sul commercio mondiale in epoca moderna. Dinamiche che simulano un tipo di mercanteggiare: "a volte compro e vendo quadri, a volte compro e vendo vasi".

Approfondimento: l'Eurogame secondo Brathwaite e Schreiber. In *Challenges for Game Designers*, nel capitolo sul *targeting* di un mercato, gli Eurogame ("noti anche come *German games* o *designer games*") sono un caso studio di gioco per il mercato di massa: giochi per la famiglia, godibili a tutte le età; *I coloni di Catan* è il primo ad aver reso popolare il genere in Nord America, seguito da *Carcassonne* e *Puerto Rico*. Il libro ne dà una definizione operativa, utile per confrontare le osservazioni di Ruggero:

- **durata tra 20 e 90 minuti** (troppo brevi annoiano gli adulti, troppo lunghi i bambini);
- **setup breve**, perché il setup non contiene decisioni;
- **regole semplici**, curva di apprendimento breve;
- **cooperazione più che scontro**: scambi e negoziazioni invece di meccaniche distruttive in cui un giocatore danneggia direttamente un altro (il che non impedisce una competizione intensa);
- **decisioni strategiche con casualità misurata**;
- **tempi morti minimi**: turni brevi, o azioni anche nei turni altrui.

La "cooperazione invece dello scontro" è l'interazione indiretta del German game; la durata, invece, mostra bene l'evoluzione che racconta Ruggero: gli Eurogame più complessi, da due o tre ore, sono ormai un'altra cosa rispetto al gioco per famiglie descritto dal libro.

2.2.24.7 American

In contrapposizione, oltreoceano c'erano gli **American** (o *Ameritrash*): giochi molto focalizzati sulla **produzione** (le miniature di plastica), con il **dungeon crawling**, tematicamente molto vicini a *Dungeons & Dragons*, quasi versioni da tavolo del combattimento a griglia normalizzato da D&D. Continuano la tradizione di *Chainmail* e dei giochi strategici e tattici: sono spesso giochi di combattimento, a **interazione diretta** (si attacca e ci si difende), e buona parte dei loro regolamenti è dedicata alla simulazione del combattimento.

2.2.24.8 Strategici e wargame

Un altro genere gigantesco è quello dei **giochi di strategia**, usati storicamente anche nella **formazione militare**, "non a caso": la **RAND Corporation** ne ha creati diversi, e ci sono editori specializzati. Lo scopo era ricreare battaglie storiche. Anche qui, evoluzioni:

- i giochi **a zone** o ad **aree** (*area control*) con unità su una mappa, di cui *Risiko* è il più famoso: io ho un esercito, tu ne hai un altro, dobbiamo conquistare territori, e il conflitto diretto è inevitabile perché gli obiettivi si sovrappongono;

[54] La trascrizione dice "The Gallery"; probabilmente *The Gallerist*.

- i **card-driven strategy game**: strategici con mappe e zone di controllo, ma con **mazzi di eventi**. I designer prendono eventi reali di conflitti reali e cercano di renderli coesi tematicamente con le meccaniche, dando a ogni evento una risoluzione precisa: "l'importante non è tanto quante unità ho, ma quali carte evento ho in mano". Sono, se ci si pensa, più vicini ai card game che agli strategici.

Nicola chiede di ***Campaign for North Africa***[55]: è uno **strategico classico**, con esagoni e pedine di cartoncino, "un'esperienza devastante in termini di lunghezza": forse non è il caso di cominciare da lì.

2.2.24.9 Movimento nascosto

Il genere degli **hidden movement**[56]: *Scotland Yard*, *Letters from Whitechapel*, ma anche altri titoli più recenti[57]. Sono giochi **asimmetrici**: di solito una persona si nasconde e scappa, e le altre vincono collettivamente se la catturano. Uno contro tutti.

2.2.24.10 Destrezza

I **dexterity game** richiedono **destrezza fisica**: colpire componenti con le schicchere, impilarli senza farli cadere. Non basta la strategia che si ha in testa: "posso anche avere la strategia migliore, ma se poi le mie mani non ce la fanno a mettere in piedi questi cilindretti, crolla tutto".

2.2.24.11 18XX

Un genere che Ruggero ama particolarmente, e di cui è "abbastanza sicuro" che nessuno abbia mai sentito parlare, anche se conta centinaia di titoli ("più di 400 giochi" sotto l'etichetta): i **18XX**. Il nome viene dal secolo delle **ferrovie**, l'Ottocento: sono tutti giochi ferroviari, ognuno centrato su un anno e un luogo specifici (*1849*, le ferrovie della Sicilia nel 1849; *1889*, quelle dell'isola di Shikoku). Hanno una forte componente di **costruzione delle ferrovie** sulla mappa e una componente **economico-azionaria**: comprare e vendere azioni, speculazione finanziaria. Sono "molto belli, ma anche molto intensi": *1889*, che Ruggero usa di solito come introduttivo, "dura tre ore".

2.2.24.12 Tempo reale

Culturalmente si pensa ai giochi da tavolo come a qualcosa che avviene **a turni**, ma non è sempre così. I giochi **in tempo reale** usano clessidre, timer o altre meccaniche per cui non si ha tutto il tempo per pensare: bisogna reagire in fretta. *Kitchen Rush*, per esempio, è "praticamente *Overcooked* da tavolo": si gestisce un ristorante, e **la clessidra è il lavoratore**, che non si può spostare finché non si è esaurita. Chi prende gli ingredienti, chi li prepara sul fornello, chi va in sala a prendere gli ordini, chi porta i piatti, chi li lava: ne nasce un certo caos nell'intreccio.

2.2.24.13 Legacy

I **Legacy** sono "un po' una rivoluzione nel genere". Di solito un gioco da tavolo resta sempre quello: lo si prepara, lo si gioca, la settimana dopo lo si gioca con un altro gruppo. I giochi Legacy invece chiedono di avere un **impatto fisico sul gioco**, per cui **le partite successive sono influenzate dalle precedenti**:

- *Pandemic Legacy* chiede di **strappare carte**; alcune città vengono distrutte dalla pandemia, coperte con degli **adesivi**, e non esistono più sulla mappa;
- *SeaFall*[58], sulla scoperta di isole in un Ottocento alternativo: le isole hanno il nome vuoto, e chi le scopre le battezza; da lì in poi l'isola porterà quel nome per tutto il resto del gioco.

"So che qualcuno sta già sudando freddo all'idea di strappare carte o scrivere sul tabellone", ma sono "esperienze emotivamente incredibili": "giocatevi dei Legacy".

[55] La trascrizione dice "Campi for Nord Africa".
[56] La trascrizione dice "Eat the Movement".
[57] La trascrizione cita "Sniper Elite, Mind Dame e GMT"; il secondo è probabilmente *Mind MGMT*, GMT è un editore di wargame.
[58] La trascrizione dice "Sifol".

Approfondimento: Risk Legacy e la rottura di un'assunzione. Schell dedica ai Legacy due passaggi di *The Art of Game Design*. Nel capitolo sul **tema**, presenta *Risk Legacy* di **Rob Daviau** come esempio di tema fondato su una verità: Daviau ha fatto "qualcosa di mai tentato prima nel mondo dei giochi da tavolo", creando meccaniche per cui le scelte dei giocatori **cambiano il gioco per sempre**. Le regole impongono di modificare il tabellone con adesivi non rimovibili, di scriverci sopra con un pennarello indelebile per rivendicare i territori, di strappare e scartare carte e perfino di modificare permanentemente il regolamento. Queste meccaniche, nota Schell, sono nuove e intriganti, ma soprattutto **rafforzano il tema centrale**: "la guerra cambia il mondo". Nel capitolo sulle tecniche di brainstorming, Schell racconta come Daviau ci sia arrivato: elencando le **assunzioni** date per scontate sui giochi da tavolo e provando a "farne saltare" una, "una partita non influenza la successiva". È un buon esempio di quello che Ruggero chiedeva all'inizio: guardare un genere che si conosce e chiedersi che cosa succederebbe se una delle sue regole implicite non valesse più.

2.2.25 Due libri sul gioco da tavolo

A proposito di risorse, Ruggero torna sul libro di Nuccio, *La progettazione dei giochi da tavolo*, citato a inizio lezione, e indica un'alternativa in inglese che a suo parere "fa quello che fa Nuccio, meglio": ***Building Blocks of Tabletop Game Design*** di **Geoffrey Engelstein e Isaac Shalev**[59]. È più **contemporaneo** negli esempi e "più funzionale". Per i giochi da tavolo, se si ha già un framework di riferimento, sono entrambi "una risorsa incredibile". (Resta valido l'avvertimento di inizio lezione: sono archivi di meccaniche, da usare dopo aver capito che cosa le meccaniche fanno.)

2.2.26 Libri-gioco, megagame e altri ibridi

"Manca ancora qualcosa?" Mancano diverse cose.

- I **libri-gioco**, nel formato libro tradizionale e nel formato "più contemporaneo" dei giochi da tavolo che sono libri-gioco, come *Sherlock Holmes Consulente Investigativo*.

- I **megagame**: giochi da tavolo **per cento persone**. Non hanno un'organizzazione editoriale come *Monopoly* o gli altri board game: sono organizzati in modo più vicino ai giochi pervasivi e al LARP, spesso **a evento**, spesso con biglietto, anche perché gestire un gioco da tavolo con cento persone in parallelo "può diventare molto intenso".

- I **LARP urban game**, di cui ha parlato Beatrice, con esempi storici anche italiani, come *Pathos*[60]: giochi di ruolo dal vivo che sono anche urban game, e un po' ARG.

- Produzioni ispirate al freeform ma vendute come giochi da tavolo, come ***Inhuman Conditions***[61]: praticamente il **test di *Blade Runner*** per scoprire chi è un androide, per due persone. È un gioco di ruolo, ma prodotto esattamente come un gioco da tavolo (carte, scatola, regolamento) e non si presenta come gioco di ruolo, "ma questo non lo rende meno gioco di ruolo".

- Ibridi con il party game, come ***Sì, Oscuro Signore!***: un party game, ma anche un *role-playing game*, perché si fa il role play del proprio personaggio.

- Ibridi con il videogioco, come *Fateforge* e gli altri giochi che usano un'**app** non solo come segnapunti ma come meccanica durante la partita.

[59] La trascrizione dice "Engelstein e Schaler" e "Building Blocks of Tablet of Game Design".
[60] Grafia come nella trascrizione.
[61] La trascrizione dice "In Human Condition".

Genere	Che cosa lo caratterizza ed evoluzioni citate
Roll and move	tira e muovi (gioco dell'oca); agency quasi nulla; poi elementi economici (*Monopoly*) o deduttivi (*Cluedo*)
Astratti	tema ridotto all'osso, informazione aperta (scacchi, dama, go, *ZÈRTZ*, progetto GIPF)
Party game	molte persone, regole leggere, partite brevi, punto di accesso (*Taboo*, *Lupus in Tabula*, *Wavelength*)
Giochi di parole	creare parole per fare punti (*Boggle*, *Scarabeo*); cooperativi (*Cross Clues*), boss fight (*Illiterati*)
Giochi di carte	mazzi tradizionali; *Love Letter*; take that (*Munchkin*); TCG, LCG e deck-building
German game ed Eurogame	interazione indiretta, costruzione e scambio; simulazioni focalizzate di due o tre ore
American	produzione in plastica, dungeon crawling, combattimento e interazione diretta
Strategici	formazione militare; aree e zone (*Risiko*); card-driven; esagoni (*Campaign for North Africa*)
Movimento nascosto	uno contro tutti, asimmetrici (*Scotland Yard*, *Letters from Whitechapel*)
Destrezza	schicchere, impilare: conta anche la mano
18XX	ferrovie e azioni dell'Ottocento; più di 400 titoli (*1849*, *1889*)
Tempo reale	clessidre e timer (*Kitchen Rush*)
Legacy	le partite cambiano fisicamente il gioco (*Pandemic Legacy*, *SeaFall*)
Libri-gioco, megagame, ibridi	*Sherlock Holmes Consulente Investigativo*; cento giocatori a evento; *Inhuman Conditions*, *Sì, Oscuro Signore!*, giochi con app

Table 2.2.3: I generi di gioco da tavolo presentati da Ruggero, in sintesi.

2.2.27 La chiusura: niente interrogazioni, e i materiali

"E ho corso un po', ma ce l'abbiamo fatta". Mancano gli ultimi dieci minuti, e Ruggero si complimenta con chi è "arrivato fino a qui". Ripete il punto fondamentale: **non ci sarà nessuna interrogazione** su questi giochi ("Tiziana, ti do subito il sollievo"). Non si chiede di sapere che cosa sia ogni caso studio: lo scopo era dare una **panoramica**, degli spunti, e far sapere che "questa roba esiste". Le slide restano, e ci si potrà tornare in fase di ricerca e sviluppo. Con la lezione di **ricerca e sviluppo**, che darà gli strumenti per farlo in autonomia, ognuno troverà **i casi studio che servono al proprio gioco**, "e io ve li chiederò, noi ve li chiederemo". Questa lezione serviva ad **aprire le possibili direzioni**.

I materiali caricati. Mentre Ruggero parla, le slide vengono caricate nella cartella degli studenti. Beatrice aggiunge di aver caricato:

- il paragrafo di **Frank Rose** su *Why So Serious*;
- un estratto sulla **definizione dei pervasive game** tratto da *Game Design*, un libro di **Maresa Bertolo e Ilaria Mariani**[62], "un ottimo libro".

Non sono letture obbligatorie.

Un avvertimento sui libri italiani di gioco di ruolo. Daniele aggiunge una cautela sulla parte dedicata al gioco di ruolo del libro di Bertolo e Mariani: alcuni paragrafi "generalizzano molto verso il mainstream". E poi un *warning* più generale: **la maggior parte della letteratura italiana sul gioco di ruolo è "terribile"**, al punto che alcuni libri i docenti non solo non li consigliano, ma li **sconsigliano attivamente**. Con la premessa che non è nulla di personale contro l'autore, Daniele sconsiglia il manuale *Giochi di ruolo*

[62] La trascrizione dice "Marianne Bertolo" e "Bertolo Mariani". Il libro è con ogni probabilità *Game Design. Gioco e giocare tra teoria e progetto* di Maresa Bertolo e Ilaria Mariani.

di **Mauro Longo**[63], perché è molto soggettivo, molto legato alla visione del gioco di ruolo dell'autore e a un solo tipo di gioco, quello di *Lex Arcana*[64], con teorie proprie che non corrispondono al design generale dei giochi di ruolo. "Va bene se volete fare *Lex Arcana* esattamente come *Lex Arcana*", non se si vuole fare altro.

Meglio il volume sull'edu-larp, e soprattutto **giocare**: "giocate i giochi, perché in Italia abbiamo una creatività sfrenata". Si trovano "robe favolose" online: tantissimi **one page game** di ruolo, il mondo del **print and play** anche nei giochi da tavolo, chamber LARP e freeform da stampare direttamente da internet. "Fatevi una ricerca e vedete quello che vi garba".

2.2.28 Le domande finali

"Siamo stati più veloci dell'anno scorso" (quando si erano accumulati trenta minuti di ritardo), e restano sette minuti per le domande.

2.2.28.1 Che cosa fa scegliere una forma?

Gianmarco, "un po' per rompere il ghiaccio", fa una domanda "molto generica", che sa non avere una risposta semplice: si sono visti giochi dalle forme diversissime, ma i **framework** per crearli sono più o meno gli stessi. Che cosa determina la scelta di una forma rispetto a un'altra? Ruggero rimanda la risposta alla lezione sull'**emotional design**, "che è proprio una risposta a questa cosa": la risposta è così lunga che "abbiamo una lezione di tre ore per rispondere alla tua domanda".

2.2.28.2 Che cosa va per la maggiore in Italia?

Angelo chiede quale genere "vada per la maggiore" in Italia. Ma che cosa significa "andare per la maggiore"? Il successo commerciale non è uguale per tutti i generi, "nemmeno nei videogiochi". Uno dei docenti (con ogni probabilità Daniele) risponde con due domande "provocatorie", che valgono anche per i libri (ne parlava quel giorno con un'amica narrative designer di videogiochi):

1. quante copie deve vendere un gioco per essere un successo?

2. il numero di copie vendute coincide con il successo commerciale ed economico?

La risposta a entrambe è **no**, ma si capirà con la lezione di **Flavio** sull'aspetto produttivo: quando si ragiona in logica commerciale si è "in logica di prodotto", e ci sono varie **metriche** e varie scale di successo. A questo si aggiungono altri livelli:

- **quale mercato**? In Italia **i giochi che non sono in italiano non vendono**, e molti editori italiani sono innanzitutto **editori di localizzazione**: prendono giochi dall'estero, li traducono, e questo è buona parte della loro sopravvivenza, nel gioco da tavolo come nel gioco di ruolo. Un gioco in inglese si vende in Inghilterra, in America e in moltissimi paesi dove l'inglese è la seconda lingua; un gioco in italiano si vende solo in Italia, e un editore italiano non ha facilmente la struttura per arrivare all'estero. Per questo anche autori italiani di giochi da tavolo pubblicano con **editori esteri**;

- **perché si fa il gioco**? Non è detto che lo scopo sia commerciale. Un gioco può essere **formativo o didattico**, fatto con un'azienda o una scuola, o **commissionato** da un cliente per uno scopo specifico: allora non interessa che venda mille copie. Il designer viene pagato, ma il pagamento non corrisponde alle vendite.

Beatrice chiede ad Angelo perché faccia la domanda, perché i **settori** appena visti spesso "non si parlano minimamente", e alcuni non sanno nemmeno dell'esistenza degli altri: la cross-medialità presentata nella lezione non viene presentata spesso così. Se la domanda nasce da prospettive lavorative, come per alcuni nel corso, la risposta è che ci sono "immense possibilità" a seconda della direzione: fare un gioco da tavolo

[63] La trascrizione riporta l'editore come "Dino Dino Editori": probabilmente Dino Audino Editore.
[64] La trascrizione dice "l'ex arcana".

è una cosa, lavorare nella **formazione** un'altra, e c'è un intero ambito non trattato, quello dei **giochi per le aziende**.

Angelo precisa: voleva solo sapere **che cosa giocano di solito gli italiani**. È "un altro volo pindarico": in Italia è difficile trovare dati statistici affidabili, perché i questionari e le survey sono spesso invalidati dal modo in cui sono formulate le domande. Da quello che sa il docente che risponde[65], ci sono sicuramente **più giocatori di giochi da tavolo che di giochi di ruolo**: il gioco da tavolo è più conosciuto e frequentato, anche nelle ludoteche, mentre il gioco di ruolo e il LARP richiedono un investimento economico diverso e sono soprattutto pratiche **comunitarie**, "tribali", che nascono dal basso e si trasmettono **oralmente**. Chi gioca ai giochi da tavolo li compra; chi gioca di ruolo spesso lo fa perché qualcuno lo fa giocare, e non è detto che abbia il gioco in casa.

2.2.28.3 Avvisi

Intanto Daniele ricorda che il **questionario** della prima slide va compilato **entro il 15**, e che Giordano deve ancora presentarsi ("siamo delle pessime persone"). Visto che è ora di cena, Giordano si presenterà come **prima cosa** alla lezione successiva, **lunedì**, sull'emotional design ("ricordacelo, perché se no ci dimentichiamo").

2.2.29 Due cimiteri: i giochi di carte collezionabili e i "fix"

Prima di chiudere, a un docente torna in mente "una cosa importantissima" che voleva dire.

2.2.29.1 Non fate Magic: The Gathering 2

"Vi prego, non cadete nella trappola" di vedere un gioco di successo commerciale e farne uno simile pensando che venderà moltissimo. "Ci possiamo fare un cimitero, una fossa comune" con migliaia di **trading card game** e **collectible card game**. "Non fate un trading card game. Ve lo dico per voi: non fate *Magic: The Gathering 2*". Chi vede *Magic* o *Pokémon* pensa di poter fare un sacco di soldi, ma **non vede la struttura commerciale e produttiva gigantesca** che c'è dietro, quasi impossibile da emulare dal basso. Quasi tutti, anche editori grossi, quelli che hanno provato a competere con *Pokémon*, *Yu-Gi-Oh!* e *Magic* "sono andati a picco". "Se volete perdere soldi, quello è il modo perfetto".

Un collega aggiunge che c'è un **revival dei TCG**, ma ogni titolo dura un anno o due e poi muore. Si sperimentano anche dei **micro TCG** annunciati fin dall'inizio come limitati ("questo gioco avrà tre espansioni e poi chiuderà"), una sorta di ibrido con gli LCG. Ma attenzione: spesso sono fatti da ex designer di LCG, di *Magic* o di *Hearthstone*, persone con contatti, entourage e un'azienda alle spalle; da fuori si vede una persona, ma dietro c'è una struttura che non si può replicare.

Approfondimento: perché Pokémon non si replica. *The Art of Game Design* offre un buon esempio della "struttura gigantesca" di cui parlano i docenti. Nel capitolo sui mondi transmediali, Schell racconta che il gioco di carte collezionabili di *Pokémon* nacque nel 1999 da una collaborazione tra Nintendo e Wizards of the Coast, la casa di *Magic*, e che fu progettato per restare il più possibile fedele alle meccaniche del videogioco per Game Boy. Il gioco di carte era la **terza porta d'ingresso** in un unico mondo, accanto al videogioco e alla serie TV: portatile, sociale, economico, perfetto per lo slogan "acchiappali tutti". Le tre porte si rafforzavano a vicenda (voler vincere spingeva a guardare la serie, e la serie rendeva il gioco più vivido), e Schell ricorda un dirigente dell'intrattenimento che nel 1999 profetizzava la fine della "moda" con l'uscita del film. Non andò così, appunto, perché il successo non stava in un singolo prodotto ma nell'ecosistema. Un gioco di carte collezionabili nato dal basso, senza un ecosistema del genere, compete con questo.

[65] La trascrizione dice "poi Lucio mi correggerà"; il nome non è ricostruibile con certezza.

2.2.29.2 Non fate un gioco per "aggiustarne" un altro

Un altro docente (probabilmente Daniele, che cita "le lezioni di framework con Ruggero") aggiunge un altro cimitero: quello di chi vuole fare un gioco per **"fixare" un gioco esistente**, cioè farne una versione migliore. Come esperimento può essere interessante, ma, come si vedrà nella lezione sull'emotional design e nelle lezioni sui framework con Ruggero, l'intenzione deve essere "**eticamente onesta**". Non si vuole aggiustare D&D per il piacere di aggiustare D&D: bisogna chiedersi sempre **che cosa non mi piace dell'esperienza** di D&D, che cosa voglio aggiustare di quell'esperienza. E non creare "D&D dove tolgo una regola e ne metto un'altra, ed è sistemato", ma **partire dall'esperienza che si vuole raggiungere**, quella che non si è avuta da D&D, e costruirla. Con la lezione di ricerca e sviluppo si vedrà anche come prendere spunto dai giochi esistenti, non per copiarli ma per capire che cosa fanno, perché **prenderne pezzi senza capire che cosa fanno** è proprio l'errore da evitare (lo stesso del libro di Nuccio letto troppo presto).

I saluti. Un docente deve scappare e saluta; gli altri ricordano il questionario, che le loro email sono nelle slide per eventuali domande, e l'appuntamento a lunedì.

Punti chiave del capitolo

- Un **caso studio** è un gioco, o una categoria di giochi, che, analizzato tenendo conto del suo contesto, dà informazioni e metodi utili per i propri giochi: come leggere libri per imparare a scrivere. Questa lezione è una panoramica; gli strumenti di analisi arriveranno con la ricerca e sviluppo. "Per me, questo è design" (Ruggero).

- **Gioco analogico** è un termine ombrello per tutto ciò che non è videogioco. Le quattro **macrocategorie** sono gioco da tavolo, gioco di ruolo da tavolo, gioco di ruolo dal vivo e gioco pervasivo, ma i confini sono porosi: **ibridazione e sperimentazione**. Il corso è **agnostico di genere**, e il project work è libero, ma non deve essere un gioco enorme.

- Il **gioco pervasivo** usa luoghi e oggetti della vita quotidiana come luoghi e oggetti di gioco; in termini di game studies, **espande il cerchio magico** nello spazio, nel tempo e nella società. Casi: *Pokémon Go*, geocaching, *GuruShot*, *Journey to the End of the Night*, SF0, *Critical City Upload*, *Sleep No More*, gli ARG (*Why So Serious*, *The Institute*, *Stranger Things* a Milano), Collettivo Cinetico, Amigdala, *MP3 Experiment*, *PacManhattan*, *Zombies, Run!*, *Reptilarium*, *Echi resistenti*.

- Esperienze emotivamente significative si possono fare **con budget zero**: l'audio che guida, gli oggetti da portare, un finale scenografico.

- Il **LARP** è separato dal GDR per ragioni di industria (evento con biglietto contro oggetto venduto). Tipi: boffer, Mind's Eye Theatre, chamber/parlor, American freeform, murder party, Nordic LARP (Jeepform, 360°, tape, black box).

- Il **Nordic LARP** ha documentato tutto (Knutpunkt dal 1997, libri annuali, manifesti come il Dogma 99), punta su coinvolgimento, emozione, coesione tematica e dopo-gioco, e ha normalizzato le **meccaniche di sicurezza**. Non è un esperimento scientifico: lo scopo resta ludico.

- Nel **GDR da tavolo** contano la fiction emergente e quattro parole: **conflitto, azione, risoluzione, effetto**. Il master è un giocatore. Dalla nascita di D&D (1974) dal wargame, attraverso gli adattamenti a nastro degli anni '80–'90, si arriva alla **New Wave** di The Forge ("il sistema conta"), ai **giochi focalizzati** (non semplificati), all'OSR, agli story game, ai giochi per due, in solitario e micro.

- Nel **gioco da tavolo** ogni genere si è evoluto: roll and move, astratti, party game, giochi di parole, giochi di carte (TCG, LCG, deck-building), German game ed Eurogame, American, strategici, movimento nascosto, destrezza, 18XX, tempo reale, **Legacy**, libri-gioco, megagame, ibridi.

- Libri: il volume sull'**edu-larp** sì; Nuccio e Engelstein–Shalev dopo l'emotional design; Bertolo–Mariani con cautela sulla parte di GDR; gran parte della letteratura italiana sul GDR no. Soprattutto, **giocare**.

- Il **successo** non si misura solo in copie, e dipende dal mercato (in Italia vende solo ciò che è in italiano) e dallo scopo del gioco.
- Due cimiteri: **non fare un gioco di carte collezionabili** e non fare un gioco per **"aggiustarne" un altro** senza partire dall'esperienza che si vuole creare.

Chapter 2.3

I framework: dal giocatore al designer, il Player Involvement Model e il Mixing Desk

Questa è la **terza lezione** del corso di **Analog Game Design**, ed è la prima davvero "tecnica". Dopo l'introduzione al corso e la lunga panoramica dei casi studio, si comincia a mettere in mano agli studenti degli **strumenti**: i **framework**, cioè delle strutture di pensiero che permettono di analizzare un gioco già esistente e, soprattutto, di progettarne uno nuovo "in maniera più focalizzata, in maniera più precisa". La lezione ne presenta quattro. Due li riprende dalla prima lezione e li approfondisce (**MDA** e **DPE**); gli altri due sono "la fetta grossa" della serata, perché sono "impegnativi" ma anche "i framework più utili pragmaticamente": il **Player Involvement Model** di Gordon Calleja e il **Mixing Desk of Larp**, nato nella scena del gioco di ruolo dal vivo nordico. Sono, dice il docente, "gli unici due framework che io personalmente vi chiederò di sapere da questo corso".

La lezione si svolge **online**, su Zoom, ed è tenuta da **un solo docente**: "oggi avete lezione, tra l'altro, solo con me". È **Ruggero**[1], che il corso conosce già: game designer con un portfolio "misto" di giochi da tavolo, giochi di ruolo, LARP e videogiochi, oggi impegnato in **ricerca e sviluppo**. La lezione comincia alle **17:10**, con una pausa a metà e una mezz'ora abbondante di domande, esempi e analisi in fondo.

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi propri, soprattutto quelli stranieri ("Kaleia" e "Callea" per Calleja, "Jesse Shell" per Schell, "Joanna Coleone" per Johanna Koljonen, "Goobs" per GURPS, "Genga" per Jenga…). Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: all'inizio della registrazione e, soprattutto, durante la pausa, la trascrizione registra frasi senza senso ripetute in loop per decine di righe ("il numero di persone è molto più basso…", "qual è la risposta che ci può essere?"), che il capitolo ignora. La terza: gli studenti intervengono spesso a voce o in chat senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro (il docente li chiama per nome, o leggono la chat), e resta generico altrimenti.

Quanto alla data, la lezione si colloca con ogni probabilità **lunedì 14 settembre 2026**[2]. Rispetto a quanto annunciato alla fine della lezione precedente, l'ordine è cambiato: la lezione sull'*emotional design* non è questa, e arriverà "non nella prossima lezione, ma in quella dopo". Il calendario, del resto, era stato presentato fin dall'inizio come modificabile.

[1] Il nome del docente non viene mai pronunciato dalla trascrizione. Che sia Ruggero lo dicono diversi indizi convergenti: è il docente che terrà il tutoraggio del sabato 3 ottobre; cita come propri i playtest di *Marvel United*, del gioco da tavolo di *Cyberpunk* e di *Status*, gli stessi giochi che Ruggero aveva elencato presentandosi; e nella prima lezione Daniele aveva promesso che proprio la lezione di Ruggero "sull'*involvement*" avrebbe spiegato il divieto della parola *immersione*. Alla fine della registrazione la trascrizione gli fa dire "sono Leopoldo": è con ogni probabilità un errore del riconoscimento vocale.

[2] La data non è mai detta. La si deduce da alcuni indizi: la seconda lezione, di venerdì 11 settembre, si era chiusa con l'appuntamento "a lunedì" e con la presentazione di Giordano rinviata alla lezione successiva, e qui Giordano si presenta; Gianmarco arriva in ritardo spiegando che "lunedì" ha un impegno di lavoro fino alle 17:30; la lezione di "venerdì 18" viene citata come futura; i gruppi saranno annunciati "il 25". È una deduzione, non un'informazione esplicita.

Come leggere questo capitolo. Diversamente dal catalogo della lezione precedente, questa è una lezione di **concetti**, e il capitolo la tratta come tale: ogni framework è ricostruito per intero, con le definizioni, gli esempi (volutamente "cross mediali", dal Jenga a *GTA*), le domande degli studenti e le risposte del docente, che spesso sono la parte più utile. Il docente insiste continuamente su un punto che vale per tutto il capitolo: le categorie del primo framework e i cursori del secondo **non sono qualitativi**, cioè non dicono se un gioco è bello o brutto, ma *che cosa* fa e *dove* si può lavorare. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (*The Art of Game Design* di Jesse Schell, *A Theory of Fun for Game Design* di Raph Koster, *Challenges for Game Designers* di Brenda Brathwaite e Ian Schreiber, *21st Century Game Design* di Chris Bateman e Richard Boon); sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa prima dell'inizio: i link in chat, i tutoraggi, i gruppi e i task (sezione 2.3.1);
2. perché si studiano i framework: **da giocatore a designer**, il design come **opposto della tradizione**, il gioco come **esperienza creata da regole** (sezioni 2.3.2 e 2.3.3);
3. alcuni **concetti di base**: le lenti, il gioco come medium mediato ed emergente, la pipeline **design, playtest, iterazione** e il modo giusto di ascoltare il **feedback** (sezioni 2.3.4 e 2.3.5);
4. **MDA** e **DPE**, ripresi e approfonditi (sezione 2.3.6);
5. la presentazione di Giordano (sezione 2.3.7);
6. il **Player Involvement Model**: perché non dire "immersione" e le sei forme di coinvolgimento (sezioni da 2.3.8 a 2.3.9);
7. il **Mixing Desk of Larp**: otto cursori più uno libero, e la sua modularità (sezioni da 2.3.10 a 2.3.12);
8. un'**analisi dal vivo** con il Mixing Desk: *Exit*, gli Hidden Games e *Profiler* (sezione 2.3.13);
9. le **domande finali** e la **responsabilità** di chi progetta giochi (sezioni 2.3.14 e 2.3.15).

2.3.1 L'attesa: link, tutoraggi e gruppi

Il docente si collega in anticipo e "muta" ancora qualche minuto. All'inizio c'è solo uno studente; arriva **Giordano**, che aveva avvisato di poter arrivare più tardi ("c'è stata una riunione convocata e cancellata"): "ti facciamo presentare subito, appena iniziamo". In realtà, "tra una cosa e l'altra", la presentazione slitterà di nuovo, fino alla semi-pausa prima del primo framework grosso (sezione 2.3.7). Mentre si aspetta, il docente invita a fare domande: "se avete qualcosa da chiedere o da dire, sono qui".

Niente task, per ora. Il docente avverte subito: "anche oggi non vi daremo task per la prossima lezione". È un'altra lezione "che non vi darà i compiti a casa". Ma **dalla prossima** cominceranno ad arrivare dei **piccoli task** in preparazione; "poi il grosso task sarà quello di creare il gioco, effettivamente".

I link in chat non funzionano. Uno studente fa una domanda "che non c'entra niente col corso": nella lezione precedente i link scritti nella chat di Zoom non si aprivano. Altri confermano. Il docente fa una prova al volo, incollando in chat il sito dell'accademia[3]: non è cliccabile, e qualcuno non riesce nemmeno a copiarlo con Ctrl+C e Ctrl+V, tanto da doversi riscrivere i link a mano. "Scusateci, non lo sapevamo": è probabilmente il primo anno che succede, forse perché in estate sono cambiate alcune impostazioni della piattaforma Zoom per tutti i corsi. Il docente ne parlerà con la segreteria e con il reparto tecnico; nel frattempo, eviterà di mettere link in chat e li metterà **su Discord**, e chi ha problemi può chiedere i link via email.

[3]La trascrizione dice "il sito dell'IV" e, poco dopo, "i corsi dell'EV" e "il reparto tecnico di IV": è la sigla dell'ente che organizza il corso, non ricostruibile con certezza dalla trascrizione.

I tutoraggi. Un'altra domanda, "tra virgolette pratica": i **tutoraggi** con il docente si fanno di sabato, dalle 11 alle 13, tutti i sabati? No. Le date sono nel calendario mostrato nella prima lezione: **3 ottobre**, **7 novembre** e **14 novembre**. Come aveva detto Daniele, sono date "potenzialmente modificabili" in caso di necessità. Lo studente chiede se sia possibile spostare quella del 3 ottobre: per il docente "non è impossibile", ma per ragioni "burocratiche" non ha l'autorità di cambiare le date, quindi bisogna **scrivere una mail a Daniele**.

I gruppi, il 25. Quando verranno comunicati i gruppi? "Il venticinque". Per formarli, i docenti terranno conto anche dei **task** che gli studenti faranno prima del 25: non quelli di oggi (che non ci sono), ma quelli della prossima lezione e di quella successiva. Servono come base per capire in che gruppo mettere ciascuno, e anche per capire "come vi approcciate al *problem solving* nel game design", "al come creare un gioco". L'obiettivo è fare gruppi "il più coesi" possibile e che lavorino bene insieme.

Lo scaffale. Uno studente nota la libreria di giochi alle spalle del docente: "viene voglia di giocare". Il docente confessa di sentirsi "quasi in colpa": alcuni li ha comprati e non è ancora riuscito a giocarli ("mi guardano e mi dicono: maledetto, hai già speso soldi"). Alcuni sono facili da intavolare, altri più intensi, e non è facile trovare le persone adatte. Ne indica uno, "*2045*"[4]: un **cooperativo** fino a cinque persone, progettato da designer **taiwanesi**, ambientato nel 2045, in cui la Cina invade Taiwan e i giocatori devono difenderla. Solo che dura **tre ore** di gioco strategico, e quando lo propone il gruppo "non è proprio invogliatissimo" di giocare tre ore di geopolitica.

Alle 17:10 si comincia. Il docente chiede di fermarlo se, concentrato sulle slide, non si accorgesse di qualcuno che entra, "perché voglio segnarvi le presenze in modo corretto". Il programma: alcuni framework "a volo d'aquila" già accennati nella prima lezione, poi due framework grossi, con una pausa in mezzo. Il primo potrebbe andare lungo: "vi chiedo di resistere in questo primo blocco", perché sono due framework che "non è facile spezzettare più di così".

2.3.2 Da giocatore e facilitatore a designer

Prima dei framework, una domanda di fondo: **che cosa siamo qui a fare?** Il docente dà per acquisito ("ce l'avete detto anche voi") che il gioco interessi agli studenti come tema e come **medium**, e che prima ancora di essere persone che vogliono creare giochi siano **persone che i giochi li giocano**. È comprensibile: a chi non legge non viene in mente di scrivere un libro, e allo stesso modo è plausibile che gli studenti abbiano giocato giochi significativi, "che vi abbiano lasciato qualcosa dentro", e che si siano detti: *ma perché no? Magari posso crearli anch'io*. "Intanto vi assicuro: potete crearli anche voi". È il primo passo.

Ma il corso, e questa lezione in particolare, servono a imparare a farlo **in maniera più strutturata**. Perché? **Per buttare meno tempo possibile**. Creare giochi, "per fortuna o per sfortuna", non è una cosa recente: l'umanità crea giochi da millenni, e da secoli in maniera più strutturata. Molte persone hanno incontrato problemi nel game design e li hanno risolti prima di noi, identificando i punti più sensibili, problematici e delicati del processo. I **framework** sono il deposito di questo lavoro: strutture su cui basarsi per

- **risparmiare tempo** nel creare giochi;
- creare giochi in maniera **più focalizzata** e **più precisa**;
- assicurarsi che i giochi portino a chi li gioca **quello che si vuole che arrivi emotivamente**. (Sulla questione emotiva in senso stretto si tornerà nella lezione sulle emozioni.)

2.3.2.1 Il design è l'opposto della tradizione

Il docente lancia una citazione di **Johanna Koljonen**[5]:

[4] Il titolo è riportato come nella trascrizione e non è verificabile.

[5] La trascrizione dice "Joanna Coleone" e, più avanti, "Ioanna Coleonen". Il docente la presenta come "una designer svedese"; Johanna Koljonen è una designer e studiosa di giochi di ruolo dal vivo nota nella scena nordica.

> **fare design è l'opposto di seguire la tradizione**.

Se si rimette in un gioco qualcosa **solo perché si è sempre fatto così**, non si sta facendo design: si sta seguendo una tradizione, creando qualcosa nel modo in cui è sempre stato fatto. "Questo ha dei pro e dei contro", ma l'importante è esserne **consapevoli**.

L'esempio del gioco di ruolo. A livello culturale i giochi di ruolo hanno alcune caratteristiche ricorrenti: la figura del **master**, i **dadi**, le **schede personaggio**. Ma sono elementi *formali*, intrinseci al gioco di ruolo, o fanno parte di una *tradizione*? I casi studio lo hanno mostrato: esistono giochi di ruolo **senza master**. Quando sono usciti i primi, la reazione è stata fortissima: "questi non sono giochi di ruolo perché non hanno un master". Con gli anni si è capito che lo sono, semplicemente *masterless* (o *masterful*, a seconda delle definizioni). Lo stesso per i dadi: *Dungeons & Dragons*, *Pathfinder* e GURPS[6] hanno i dadi, ma moltissimi altri giochi di ruolo non ce li hanno: usano le carte da poker, le carte da briscola, oppure non hanno proprio sistemi di generazione della casualità, e hanno altri aspetti che li rendono giochi di ruolo. Della storia e delle forme del gioco di ruolo parlerà più nel dettaglio Daniele, nella lezione dedicata.

La domanda da farsi. Quando si crea un gioco si ha spesso in mente un **genere di riferimento**. Il docente chiede di distinguere:

- gli elementi che si vogliono nel gioco **perché ha senso che ci siano**;
- gli elementi che si stanno mettendo, magari inconsapevolmente, **solo perché il genere li ha sempre avuti**.

In una frase: **chiedetevi che cosa state mettendo nel vostro gioco, e perché**.

2.3.3 Il gioco è un'esperienza creata da regole

La seconda frase che il docente vuole alla base della lezione, e del resto del corso, è la definizione di gioco di **Anna Anthropy**, dal libro già menzionato *Rise of the Videogame Zinesters*[7]:

> **il gioco è un'esperienza creata da regole**.

Chi ha già letto di game design e cercato definizioni la troverà "molto più generica delle altre". Ma **regole** ed **esperienza** sono "le uniche due cose che davvero sono sempre presenti nei giochi", e sono le cose da ricordare "nella maniera più assoluta". Quando si crea un gioco, due sono le cose che interessano:

L'esperienza su cui **non si ha controllo**: non si può obbligare una persona a provare emozioni.

Le regole su cui **si ha controllo**, e che sono l'unico strumento che aiuta a portare l'esperienza che si vuole.

Da qui la definizione del mestiere: **il lavoro del game designer è creare regole che creino esperienze**. "So che vi sembra banale, ma vi giuro": spesso, anche nelle presentazioni, al docente arrivano prototipi da valutare che "meccanicamente hanno anche senso", ma quando l'autore dice *che esperienza* vuole che il gioco dia, si scopre che le regole non danno quell'esperienza "neanche per sbaglio". La raccomandazione è far **combaciare** quello che si vuole far arrivare con le regole che si mettono nel gioco.

Approfondimento: il gioco non è l'esperienza. La stessa distinzione è uno dei pilastri di *The Art of Game Design*. Schell la formula così: "il gioco non è l'esperienza. Il gioco *rende possibile* l'esperienza, ma non è l'esperienza". La illustra con il vecchio indovinello zen dell'albero che cade nella foresta senza che nessuno lo senta: se per "suono" si intendono le molecole d'aria che vibrano, il suono c'è; se si intende l'esperienza di sentirlo, no. Al designer, dice Schell, dell'albero e di come cade importa poco: importa l'esperienza di

[6] La trascrizione dice "Da Genentech hanno i dadi, Pathfinder hai i dadi, Groups hai i dadi": si tratta con ogni probabilità di *D&D*, *Pathfinder* e GURPS, che la trascrizione più avanti chiama anche "Gurps", "Goobs" e "GuardStore".

[7] La trascrizione dice "Anantrophy" e "Rise of Video Games in Esters". Il libro di Anna Anthropy, del 2012, è un manifesto a favore dei giochi fatti da autori indipendenti e non professionisti.

sentirlo cadere, e l'albero è solo un mezzo. Il giocatore e il gioco sono reali, l'esperienza è "immaginaria", ma i designer vengono giudicati proprio sulla qualità di questa cosa immaginaria; e se potessimo creare esperienze direttamente, senza tabelloni, schermi e regole, lo faremmo. Non potendo, si lavora sull'unica cosa su cui si ha presa: il gioco, cioè le regole e i componenti. È esattamente la divisione di Anna Anthropy tra regole (controllabili) ed esperienza (non controllabile).

2.3.3.1 Il gioco per tutti non esiste

Tiziana chiede di intervenire (scusandosi, come sempre, di chiedere il permesso: "è la regola generale delle lezioni con me: potete sempre interrompermi"). Porta la sua esperienza di giocatrice, "perché poi ogni giocatore cerca un'esperienza diversa": ci sono giochi con meccaniche "meravigliose" che lei però percepisce come **astratti**, perché il tema è appiccicato e "potresti metterci qualsiasi tematica e funzionerebbero lo stesso". Lei preferisce i giochi più **ambientati**, che portano dentro un tema e fanno fare l'esperienza legata a quel tema.

Il docente risponde che è legittimo, ed è proprio quello che i casi studio hanno mostrato: le tipologie di giochi, e anche le "psicometrie di approccio", sono molto diverse. A lei un astratto dice poco, "non è un tuo *drive*"; a chi gioca un astratto per le meccaniche, una *visual novel* può far venire voglia di spararsi dopo mezz'ora. Il docente conosce persone che giocano tantissimo ma che, "se devono leggere in un gioco, già le abbiamo perse": una visual novel investigativa, per quanto incredibile, "non è pane per i loro denti". E viceversa, per qualcun altro può diventare uno dei titoli preferiti. È come per i film: lo stesso film può essere odiato da una persona e amato da un'altra.

Da qui una regola da ricordare: **il gioco per tutte le persone non esiste**. Bisogna uscire dal *mindset* per cui il proprio gioco debba essere per tutti.

Accessibilità non vuol dire "per tutti". Il docente distingue subito un concetto vicino, che verrà ripreso più avanti nel corso: l'**accessibilità**. Un gioco deve essere accessibile **per le persone che vogliono giocarlo**; questo non vuol dire che il gioco sia per tutti, ma che deve essere "per tutte le persone per cui quel gioco è". "Un po' intricato", ammette, e fa un esempio: una persona ama leggere e vuole una visual novel investigativa lunga e tosta; quella visual novel esiste, ma è scritta in un font piccolissimo, e la persona ha problemi di vista. Il gioco c'è, ma lei non può farne esperienza se nelle opzioni non ci sono le impostazioni di accessibilità e di qualità della vita. Gli stessi problemi esistono nel gioco da tavolo e nel gioco di ruolo: il docente ha visto **manuali con font non leggibili**, "e non puoi mettere un font non leggibile in un regolamento, che è il modo principale in cui le persone imparano i giochi". Avrebbe degli esempi, "ma vi voglio troppo bene per farveli vedere adesso": li mostrerà quando gli studenti saranno "psicologicamente in grado di sopportare la vista di un manuale della GMT".

2.3.4 Concetti di base

2.3.4.1 "Buon game design" vuol dire tutto e niente

Si parte dai concetti di base, e il primo è quasi una provocazione: **il "buon game design" vuol dire tutto e niente**. Per questo servono strumenti più precisi, e un vocabolario condiviso.

2.3.4.2 Le lenti di Schell

Il docente cita uno dei libri più noti del settore, "non necessariamente perché ve lo consigliamo da leggere, ma perché se finite a fare game design il termine di **lenti** lo incontrerete prima o poi". Il termine viene da *The Art of Game Design: A Book of Lenses* di **Jesse Schell**, che esiste in inglese e in italiano, e in cui l'autore ha catalogato "una cinquantina" di lenti[8]: **metodi di osservare il proprio design**.

[8] Il numero è quello detto a lezione. La terza edizione inglese del libro, quella usata come riferimento in questo volume, ne conta in realtà più di cento: l'indice va dalla *Lens of Emotion* (#1) alla *Lens of the Raven* (#112), più una *Lens of Your Secret Purpose* numerata con il simbolo dell'infinito.

Il concetto è quello, già noto come modo di dire, del **mettersi cappelli diversi**: in questo momento guardo il gioco con la lente della qualità della vita, in quest'altro guardo come sono scritte le regole, in quest'altro guardo l'esperienza di chi gioca. Il giudizio del docente è sfumato:

- "non sono tutte utilissime, queste lenti, se vogliamo proprio essere onesti": alcune aiutano, altre no, e alcune, secondo lui, sono **dannose** (ne parlerà più avanti);
- il corso non userà molto il termine, ma è giusto averlo nel proprio **glossario**, perché molte persone nel settore lo usano;
- "il libro non è neanche brutto, secondo me": si può leggere in autonomia.

2.3.4.3 I giochi sono un medium

Il secondo concetto, già detto nella lezione introduttiva, è che **i giochi sono un medium**. Vuol dire che hanno le stesse problematiche, e le stesse funzionalità, degli altri media: **con i giochi si comunica**. "Non stiamo semplicemente mettendo qualcosa nel vuoto dello spazio e un alieno poi se lo gioca": chi crea giochi è una persona e fa "cose da persone", e chi gioca è a sua volta una persona, con la propria vita "in mezzo" tra sé e il gioco. Ne discendono alcune caratteristiche (figura 2.3.1).

Il feedback è bidirezionale. Il designer comunica con chi gioca, e chi gioca comunica con il sistema: non con il designer direttamente, ma con la sua creazione.

Il gioco è un'esperienza differita. Tranne rari casi (alcuni giochi a evento, alcuni giochi pervasivi o LARP), si crea un regolamento oggi e qualcuno lo gioca domani[9]. "Non arrivate con le scatole dei giochi" a casa di chi li compra ("altrimenti qui c'è gente che mi dovrebbe pagare l'affitto un bel po"'): bisogna fidarsi, incrociare le dita, e sperare che sia tutto chiaro quando qualcuno apre la scatola e legge.

Il gioco è mediato. "Non siamo in grado di comunicare con la telepatia": non si possono instillare pensieri nella mente di chi gioca, quindi la conversazione va **mediata**, e di solito lo strumento di mediazione è il **regolamento**. Con un asterisco: nel gioco di ruolo da tavolo, nel gioco di ruolo dal vivo e nel gioco pervasivo il regolamento scritto potrebbe non esserci, ma esiste comunque una mediazione: procedure, **workshop pregioco**, sistemi di principi, linee guida. "Il gioco è mediato sempre", anche quando lo si spiega a voce.

Il gioco è emergente. È un *foreshadowing*: ci si tornerà parlando di playtest. "A meno che non abbiate l'onniscienza", non si può sapere in anticipo come le persone mischieranno le meccaniche, e spesso chi gioca trova interazioni particolari tra le regole e ne fa uscire **dinamiche** che il designer non aveva minimamente immaginato. A gioco pubblicato non si può dire "no, il gioco non suggeriva così": nessuno ascolterà, il gioco è già nelle case delle persone. Bisogna imparare a **prevedere** il più possibile questi comportamenti emergenti **prima**.

2.3.5 Design, playtest, iterazione e feedback

"E a proposito: il playtest e il design". Nella **pipeline** della creazione di un gioco ci sono passi che **non si possono saltare**: "perlomeno in questo corso non dovete saltare. Poi, fuori da questo corso, potete fare quello che volete" ("non sono un poliziotto, non entrerò in casa vostra a spaccarvi i mobili se non fate i playtest"). Ma nel corso, e in generale nell'industria, la pipeline esiste e va rispettata (figura 2.3.2).

[9] La trascrizione dice "esperienze indifferenti": dal contesto, il senso è quello di esperienze *differite*, in cui chi progetta e chi gioca non condividono lo stesso momento.

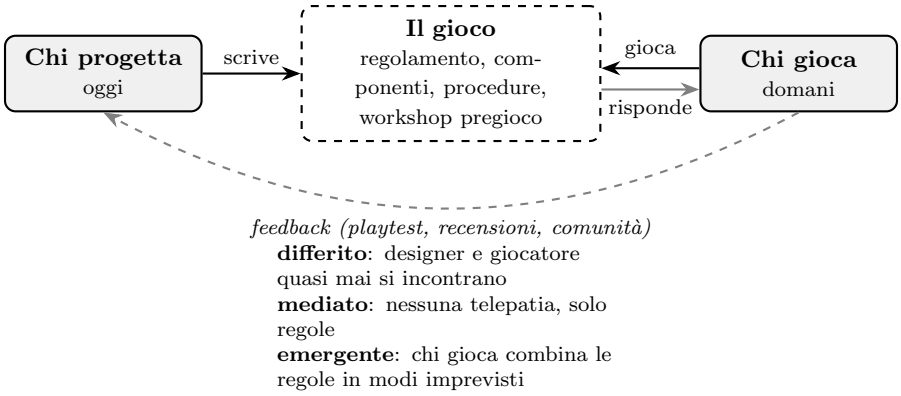


Figure 2.3.1: Il gioco come medium, nella lettura del docente: chi progetta non parla a chi gioca, ma al sistema che ha costruito; chi gioca interagisce con il sistema, e solo indirettamente, attraverso il feedback, con chi lo ha progettato.

2.3.5.1 Il playtest non si salta

Si parte dalla **progettazione**; e quella progettazione, prima di andare in stampa o di arrivare a chi il gioco deve giocarlo alla fine del percorso, **va testata**. "Vi prego, non saltate il playtest". Ci sarà una lezione di tre ore dedicata, quindi per ora non bisogna preoccuparsi di *come* si fa: deve restare che **si deve fare**. E si deve fare in ogni caso:

- se si vuole proporre il gioco a un **editore**, cioè pubblicarlo con uno scopo commerciale;
- se il gioco **non ha uno scopo commerciale**: per sé, per gli amici, per motivi artistici;
- persino se è un gioco che non giocherà nessuno, "ma magari esposto in un museo": "il gioco va playtestato".

La caccia alle lumere. Uno studente porta un'esperienza personale. A fine ottobre dell'anno precedente ha organizzato con altre persone una sorta di **caccia alle lumere** (le lanterne con le zucche) in un bosco, con **sei percorsi** segnati da fili di lana. Sapeva che l'ideale sarebbe stato far provare i percorsi a qualcuno prima, ma non è stato possibile: e infatti sono emerse delle difficoltà. "Tu metti giù, pensi che sia chiaro", risponde il docente, "perché tu conosci il percorso"; poi qualcuno che non lo conosce lo fa, e ci si rende conto che così chiaro non è. Anche nelle cose più semplici.

I giochi pervasivi e il mondo reale. Restando sul gioco pervasivo, il docente ha giocato giochi in città progettati da persone che **quelle città non le vivevano**: su Google Maps due tappe erano vicine, nella realtà erano a **40 minuti** di distanza perché c'era una strada in mezzo che non si poteva attraversare. I giochi pervasivi sono molto sensibili ai cambiamenti del mondo reale: se si organizza un evento in un parco per un certo giorno senza assicurarsi che il parco sia agibile, "vi potete beccare, come è successo, dei bei cantieroni" che restaurano proprio la statua che era "un pezzo importantissimo" dell'enigma ambientale. "Buona fortuna" a chi giocherà senza poter vedere quelle informazioni.

Anche i modelli matematici vanno testati. Nel gioco pervasivo è facile accorgersene, perché il corpo e i suoi limiti fisici impattano con il mondo reale. Ma anche in un gioco da tavolo "astratto" **i modelli matematici non sono meno da testare** del resto: se c'è un componente troppo sbilanciato, troppo forte, una **strategia dominante**, bisogna poterla "beccare" il prima possibile. Gli strumenti per farlo arriveranno con la lezione sul playtest.

2.3.5.2 L'iterazione

Dopo il playtest arriva una cosa "che non piace a nessuno, ma va fatta": l'**iterazione**. Il playtest funziona **come una spia**: "non una spia di un agente segreto tipo 007, una spia della macchina", cioè un **segnale** che qualcosa non funziona o che qualcosa è potenzialmente un problema. Ricevuto il segnale, bisogna mettersi al lavoro e **modificare il prototipo**, cambiando quelle che erano le regole precedenti. L'iterazione può essere:

- banalmente, **riscrivere una regola** perché non sia ambigua;
- oppure **tagliare una meccanica** a cui si è affezionati.

Il secondo caso "succede a chiunque, succede anche a me". Il docente è "un grandissimo amante" dei **giochi da tavolo in tempo reale**, e prova sempre a mettere una clessidra, un timer, qualcosa del genere nei suoi prototipi di board game. E troppe volte succede che il tempo reale non ci sta, e va tolto: "peccato, va bene così, se ne va. Tornerà al prossimo prototipo. Ma per ora se ne deve andare".

Il ping pong. Dopo l'iterazione si torna al playtest, e si fa **ping pong** (iterazione, playtest, iterazione, playtest) finché:

- si va **in stampa**, oppure
- si consegna il prodotto finale all'**editore**, oppure,
- se il gioco non è commerciale, si è **soddisfatti** e il gioco è pronto.

La perfezione ha un costo infinito. "Tra virgolette", perché **la perfezione ha un costo infinito**: si può sempre migliorare un gioco, spendendo tempo, risorse, energie e soldi infiniti. Si può playtestare per anni, e il docente ne è un esempio:

- nei giochi commerciali, come *Marvel United* o il gioco di *Cyberpunk*, ha avuto periodi di playtest lunghi, **uno o due anni**, "perché un editore più di quel periodo non li può reggere";
- per *Status*[10], che ha dietro di sé, ci sono stati **sei mesi** di playtest;
- per i progetti personali non commerciali ha giochi in playtest da **sette, otto anni**: "chi mi ferma? Niente, non è commerciale".

Nelle aziende, negli editori, nei publisher questo è un grosso problema: **nessuno darà tempo infinito di playtest**, e spesso le scadenze sono strette. Gli editori di solito dicono quanti mesi di playtest o di sviluppo possono dedicare a un prodotto, e la maggior parte tende a dare la priorità a progetti già sviluppati. "È un vaso di Pandora che non apriamo adesso".

2.3.5.3 Il feedback: il sentimento, non la soluzione

Ultimo elemento della pipeline, su cui si tornerà nel dettaglio: il **feedback**. Quando un *playtester* indica un problema, **non sta necessariamente indicando la cosa corretta**: quello che serve è **l'emozione, il sentimento** che sta provando. Chi playtesta non ha necessariamente basi di game design, e non saprà dire "questa carta nel modello matematico pesa la metà delle altre". Dirà: "questa carta fa schifo". E magari non è la carta che fa schifo, "magari è un numerino da cambiare su quella carta". Il metodo è:

1. **ignorare la soluzione** proposta dal playtester;
2. **risalire al sentimento**: che cosa sta provando?
3. cercare **la fonte** di quel sentimento, il **problema reale** che lo genera;
4. dare **la propria soluzione**: "non ve la deve dare il playtester, non è chi playtesta che deve risolvere il vostro gioco".

[10] La trascrizione dice "ho quasi come status qua dietro": il docente indica con ogni probabilità la scatola di *Status*, il gioco sulla geopolitica che aveva citato tra i propri lavori nella prima lezione.

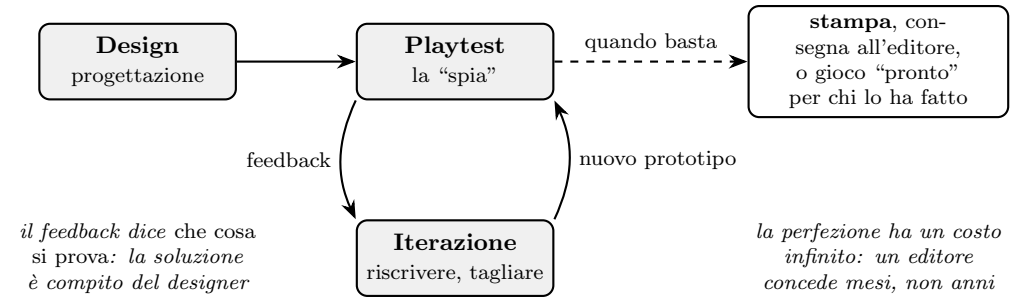


Figure 2.3.2: La pipeline che "va rispettata": si progetta, si playtesta, si itera, e si fa ping pong tra playtest e iterazione finché il gioco va in stampa, viene consegnato o è pronto.

Approfondimento: il ciclo e le domande dopo il playtest. I libri di riferimento descrivono lo stesso anello. *Challenges for Game Designers* lo scompone in quattro passi (prototipo, playtest, revisione, ripetizione) e nota che a volte la revisione è un piccolo cambiamento incrementale, altre volte il designer scopre "dinamiche di gioco inattese" che portano il gioco in una direzione completamente diversa: è l'emergenza di cui parla il docente. Lo stesso libro definisce il playtest come "il test sistematico di gameplay, sistemi, bilanciamento e interfaccia" per trovarne errori e incoerenze. Schell, dal canto suo, dedica un capitolo intero all'iterazione e la riassume in una regola che dichiara senza eccezioni, la *Rule of the Loop*: più volte si testa e si migliora un design, migliore sarà il gioco. Sul feedback, poi, Schell offre un aiuto pratico in linea con il consiglio del docente: per il dopo-sessione suggerisce domande che fanno emergere *che cosa* hanno provato, come "qual è stato il momento più frustrante?", "qual è stato il momento preferito?" e "che cosa stavi facendo?". E avverte che i playtester tendono a non voler ferire i sentimenti di chi ha progettato, motivo in più per non prendere alla lettera i loro giudizi.

2.3.6 MDA e DPE, di nuovo

"E adesso parliamo un attimo di framework principali". Sono quattro; due si vedranno in maniera approfondita, mentre MDA e DPE, già visti nella lezione introduttiva, qui ricevono "un po' più di spazio".

2.3.6.1 L'MDA: su che cosa abbiamo controllo?

"Qualcuno si ricorda dell'MDA?" Il docente fa per chiamare **Tancredi** ("io stavo aspettando voi, in realtà"), e uno studente ricostruisce: le **meccaniche** sono come funziona il gioco; le **dinamiche** come si attuano nella realtà, nel giocare il gioco; l'**estetica** è l'esperienza che fa il giocatore. "Esatto".

Seconda domanda: su quale di questi aspetti ha controllo il game designer?

- **Sulle meccaniche**, sì: "non siamo telepati, non siamo maestri di ipnosi".
- Un po' meno sulle dinamiche, che "in realtà sono semplicemente la conseguenza delle nostre azioni di game design": sono l'applicazione in tempo reale delle meccaniche da parte di una persona, e possono emergere cose che non si erano previste.
- Se si è fatto bene il lavoro, si punta a una specifica **estetica**, e per arrivarci si modificano le meccaniche.

In sintesi, **si ha potere sulle meccaniche**, e **non si possono imporre dinamiche**. Non si può dire a qualcuno "ehi tu, devi provare fretta": gliela si fa provare, "mettendo magari un timer, una clessidra", tornando al tempo reale che magari nei prodotti non entra, "ma entra in questi esempi" (figura 2.3.3).

Il gap tra estetica promessa ed estetica arrivata. Chi gioca non è detto che, leggendo un regolamento, capisca che cosa proverà. Anzi: "vi è successo sicuramente" di leggere un regolamento e non avere la certezza del tipo di esperienza che aspettava; o di leggere un gioco, dire "sembra bello", giocarlo e trovarlo pessimo. Perché? Perché c'è un **gap tra l'estetica promessa e l'estetica che poi arriva**.

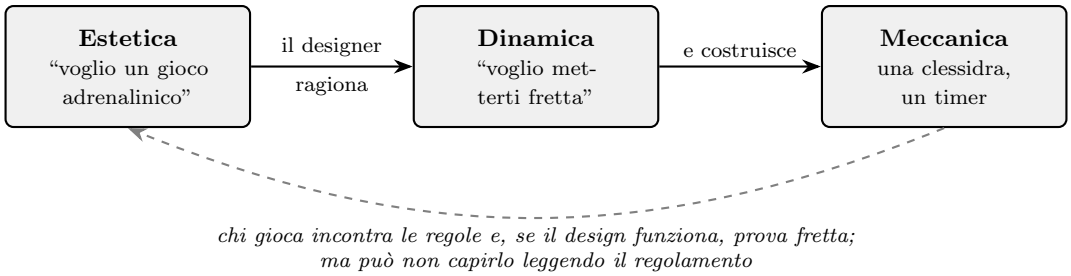


Figure 2.3.3: L’esempio del docente letto nelle due direzioni dell’MDA: il designer parte dall’estetica e arriva alla meccanica, l’unica leva su cui ha davvero controllo; chi gioca fa il percorso inverso.

Il costruttivismo personale. Il docente richiama un concetto già menzionato: il **costruttivismo personale**. Ogni persona ha una propria vita, con esperienze necessariamente diverse da quelle di ogni altra; quello che arriva a una persona in un certo modo non arriva uguale a un’altra. Lo stesso gioco può essere pessimo per una persona e il preferito di un’altra.

2.3.6.2 Progettato bene, giocato volentieri

Per questo, ricorda il docente, nella prima lezione si era separato il *game* dal *play*, il gioco dal giocare. Bisogna **allenarsi** a farlo, e non è facile: è facile pensare a giochi progettati bene che piace giocare, e a giochi progettati male che non piace giocare. Ma sono i due casi intermedi, “quelli più bizzarri”, che bisogna imparare a riconoscere (figura 2.3.4):

- giochi **progettati bene che non piace giocare**;
- giochi **progettati male che piace giocare**.

Danno informazioni su che cosa piace a sé come persone, e danno gli strumenti per fare game design migliore: capire che il buon game design **non è necessariamente quello che porta a giochi che piacciono a sé**. Lavorando nel settore, potrebbe essere chiesto di creare giochi che non piacciono, ma che hanno senso per un publisher. È come uno scrittore che scrive fantasy e a cui l’editor chiede un *romance fantasy*, quando a lui la parte romance non piace per niente.

	mi piace (*play*)	**non mi piace**
design buono	progettato bene, mi piace giocarlo *facile da riconoscere*	progettato bene, **non** mi piace giocarlo *da imparare a riconoscere*
design debole	progettato male, mi piace giocarlo *da imparare a riconoscere*	progettato male, non mi piace giocarlo *facile da riconoscere*

Figure 2.3.4: Separare il gioco dal giocare: i due casi “bizzarri” in grigio scuro sono quelli che insegnano di più, sia su sé stessi come giocatori sia sul game design.

2.3.6.3 Il DPE

L'MDA è stato poi **iterato** nel **DPE**, *Design, Play, Experience*, il cui paper è citato in cima alla slide[11]. Nasce da un approccio **educativo e formativo**: è un'iterazione che punta a parlare dei giochi nell'uso didattico. Ma, secondo il docente, nel farlo riesce a semplificare e a "distillare" il cuore interessante dell'MDA, con tre nomi "più accurati":

Design la parte in cui chi progetta crea le meccaniche;

Play chi gioca, che applica e "esperisce" le dinamiche;

Experience che cosa rimane, a livello esperienziale, a chi ha giocato.

Il DPE dispone queste tre parti su diversi **livelli** (*layer*) tematici. Nel livello del *gameplay*, il design contiene le meccaniche, il play le dinamiche, l'experience la parte affettiva ed esperienziale. Nel livello dell'**apprendimento**, se si parla di gioco formativo, il design contiene il **contenuto pedagogico** (che cosa il gioco deve affrontare e far passare), il play il momento in cui quel contenuto viene insegnato, e l'experience, se il gioco è andato bene, è il fatto che le persone **imparano** quello che il gioco voleva far loro imparare.

È un paper "un po' più tecnico" dell'MDA, tagliato sull'ambito formativo, ma "non così complesso da risultare ostico" a chi si avvicina al tema: il docente lo consiglia molto. Con una precisazione che vale per tutto: "leggerlo non vuol dire prendere tutto per vero". Su MDA e DPE si è dibattuto moltissimo: "non prendete per vero niente, nemmeno quello che vi sto dicendo io". I due paper sono abbastanza noti, soprattutto l'MDA; il DPE spesso è un po' ignorato da chi lavora nel videogioco perché legato all'ambito didattico, ma secondo il docente resta molto interessante, "oltre che una semplificazione e un'iterazione dell'MDA in maniera più digeribile".

Una studentessa chiede di rivedere il titolo del paper. Le slide, come al solito, saranno nel Drive dopo la lezione: "non correte a segnarvi su un post-it il paper". "Non vogliamo nascondervi le fonti"; e per qualsiasi dubbio si può scrivere una mail al docente. Tiziana segnala che sono entrati altri partecipanti: in quel momento ci sono **Lorenzo**, **Giordano**, **Nicola**, **Tancredi** e **Tiziana**.

Approfondimento: i livelli del DPE. Nel paper di Winn i livelli sono più di due. Oltre al *gameplay* e all'apprendimento (*learning*), il framework prevede un livello di *storytelling*, uno di *user experience* e, alla base di tutto, la *technology*, che abilita ma non determina gli altri. Ogni livello ha la propria terna design, play, experience: per esempio, nello storytelling il designer scrive la storia "del designer", il play produce la storia "del giocatore" (quella che emerge dalle sue scelte), e l'experience è il coinvolgimento che ne deriva. È un'idea vicina a quanto il docente sta per dire sul coinvolgimento narrativo, e alla distinzione tra regole ed esperienza da cui la lezione è partita.

2.3.7 Giordano si presenta

"MDA, DPE, e adesso arriviamo alla parte importante". Il docente annuncia il primo framework grosso, il **Player Involvement Model**, con una promessa: "se faccio il mio lavoro in maniera corretta, alla fine di questa lezione vi sarà immediatamente chiaro perché non dovete usare *immersione* come termine nel corso. E se ho fatto il mio lavoro ancora meglio, non lo userete mai più nella vita. Ma questo è un obiettivo più ambizioso".

Arriva **Gianmarco**, in ritardo: il lunedì ha un impegno di lavoro fino alle 17:30. "Non ti preoccupare". Il docente ne approfitta per una **semi-pausa** prima di tuffarsi nel dettaglio, e chiede finalmente a Giordano di presentarsi ("mi stavo dimenticando") con le solite domande, che scrive anche in chat: nome, che cosa fa nella vita, perché vuole fare giochi, che cosa lo ha portato qui, che esperienza ha già.

[11] È il paper di Brian Winn, *The Design, Play, and Experience Framework*, pubblicato alla fine degli anni Duemila nello *Handbook of Research on Effective Electronic Gaming in Education*. La trascrizione riporta il titolo come "Design Play Experience di PA Framework by Win".

La presentazione. Giordano gioca a **Magic: The Gathering** "dall'alba dei tempi". Dopo il COVID si è spostato di più su **giochi da tavolo** e **giochi di ruolo**, soprattutto negli ultimi due anni, anche per lavoro: è quasi il suo **secondo lavoro**. Lavora come **facilitatore e dimostratore** per un editore[12], e collabora con un'**associazione** che fa il giro delle **biblioteche della Brianza** organizzando serate di giochi da tavolo e di ruolo, dove gli capita di fare da master o di aiutare nella promozione dei giochi.

Ha scelto il corso perché è **focalizzato solo sul gioco analogico**, senza la parte di videogioco e di programmazione, e quindi più compatibile con un lavoro a tempo pieno. Lo fa per due ragioni: acquisire una **competenza di base**, "a livello analitico", per coltivare meglio la sua passione; e riuscire a fare qualcosa, "non necessariamente da pubblicare", da proporre negli eventi dell'associazione. "Figata", commenta il docente, che gli chiede se sarà a **Play**[13] come dimostratore. Quest'anno no, perché gli erano saltati gli appoggi per dormire. L'anno prima, invece, era saltato un altro ragazzo, e lui si è "beccato" *Rival Cities*: circa **24 ore** di dimostrazioni dello stesso gioco in due giorni. "Chiaro, chiaro, chiaro".

2.3.8 Il Player Involvement Model

"Ora che Giordano si è presentato" ci si butta nel primo framework, "tuffandoci con cautela ma anche con determinazione". Il **Player Involvement Model** è uno dei due framework che il docente chiederà di **sapere**: "non nel senso che vi interrogo", ma nel senso che, dal suo punto di vista ("non ho la verità in tasca, tutto quello che dico è il mio punto di vista"), è uno strumento **pragmatico** e utile in due direzioni:

- per capire che cosa fa un gioco **già pubblicato**;
- per mettere le basi del **proprio** gioco.

2.3.8.1 Il libro di Calleja

Il framework viene da un libro di **Gordon Calleja**, *In-Game: From Immersion to Incorporation*, pubblicato dalla MIT Press[14]. Secondo il docente se ne trova il PDF gratuito; chi non lo trova può scrivergli e glielo manda. Ma non sarà richiesto leggerlo: "se ho fatto le mie slide correttamente", è già condensato lì tutto quello che serve.

La tesi di partenza: per Calleja **"immersione" è una parola che ormai non vuol dire niente**, "per mille ragioni che non sto qua a dirvi". Da designer è più utile chiedersi **che cosa intende la gente** quando parla di immersione: capendolo, diventa più facile progettarla. "Immersione" resta una parola vietata: la si usa in queste slide, per contesto, "ma poi non la voglio più sentire nominare".

2.3.8.2 Che cos'è un gioco immersivo?

"Se io vi dico: voglio un gioco più immersivo, che cosa vuol dire?" Le risposte degli studenti:

- un'esperienza **più coinvolgente**, che non permette ai giocatori di distrarsi;
- un gioco che dà la sensazione di vivere un'esperienza **il più vicina possibile alla realtà**; il docente nota subito che è soggettivo, perché dipende da persona a persona;
- **Lorenzo**, appena entrato (il docente gli riassume dove si è arrivati), propone: un gioco in cui la **sospensione dell'incredulità** funziona bene.

[12] La trascrizione dice "per la Genos" e, poco dopo, "Ministro Ghienos": si tratta con ogni probabilità dell'editore italiano Ghenos Games.

[13] Con ogni probabilità Play – Festival del Gioco di Modena, la principale fiera italiana del gioco da tavolo.

[14] La trascrizione dice "Gordon Callea", "Kaleia" e "Calleia", e chiama l'editore "Meatpress". Il libro è del 2011; Calleja è un game designer e studioso maltese. Il docente lo chiama sia *Player Involvement Model* sia *Player Involvement Framework*.

Le provocazioni. Il docente risponde con una serie di domande "provocatorie".

Gli scacchi sono un gioco immersivo? "Dal punto di vista cognitivo, sì", risponde qualcuno. Ma gli scacchi non chiedono sospensione dell'incredulità, né di fare un'esperienza simile alla realtà. Uno studente osserva che bisogna capire che cosa si intende, perché negli scacchi si è concentrati, non ci si immedesima ("oh, sono il cavaliere, il pedone").

Il *flow*. Qui il docente introduce un termine che non ha messo nelle slide: il **flow**, o **flusso**. Non è solo un termine di game design ma, più in generale, di psicologia comportamentale: è la zona mentale in cui si trova una persona quando fa un'attività **né noiosa né troppo sfidante**, "tra il *questa cosa mi chiede impegno* e il *non sono frustrato da questo impegno*". In quella zona si sta molto bene, si è molto concentrati e coinvolti anche emotivamente. Gli scacchi possono mandare in flow, come molti altri giochi, e come lo **sport**; ma lo sport non chiede sospensione dell'incredulità.

Otto ore di *Civilization*. Una persona che gioca otto ore di fila a *Civilization IV* ("qualcuno di voi l'ha fatto, sono sicuro, non negatelo"): uno studente confessa quattordici ore consecutive con un amico ("vabbè, ma quattordici ore sono dieci minuti nel tempo del giocatore"). Per lui rientra nell'immersione: si gioca al punto di dimenticarsi di mangiare.

I tornei di picchiaduro. Chi gioca un torneo di *Tekken* o di *Street Fighter* è immerso?

***Dance Dance Revolution*.** E chi balla su *Dance Dance Revolution*? E chi fa un LARP dentro un castello?

La conclusione: è "molto difficile" dare una definizione di immersivo che racchiuda tutto. "Immersione" viene usata come **parola jolly**, "una roba che ti coinvolge tanto"; ma dal punto di vista del game design **non dà nessuno strumento**.

Approfondimento: il flow nei libri. Il concetto che il docente richiama viene dallo psicologo Mihaly Csikszentmihalyi, e Schell gli dedica una lente (la *Lens of Flow*). In *The Art of Game Design* il flow è definito come "una sensazione di concentrazione completa ed energica in un'attività, con un alto livello di piacere e di appagamento", e Schell ne elenca le condizioni: **obiettivi chiari**, **nessuna distrazione**, **feedback diretto** e una **sfida continua**, che resti nello stretto margine tra la noia (troppo facile) e la frustrazione (troppo difficile), il cosiddetto *flow channel*. L'esempio classico, ripreso da Csikszentmihalyi, è proprio uno sport: un ragazzo che impara il tennis è in flow finché la difficoltà è adeguata alle sue capacità, si annoia quando migliora senza che la sfida cresca, e si agita quando incontra un avversario troppo forte. Anche *21st Century Game Design* dedica una sezione all'esperienza del flow, per capire come diversi tipi di giocatori la raggiungano in modi diversi. Il fatto che il flow si ottenga con gli scacchi, il tennis o un torneo di *Tekken*, senza alcuna finzione da "credere", è proprio l'argomento del docente: sotto la parola "immersione" convivono almeno due fenomeni diversi, l'assorbimento nell'attività e la sospensione dell'incredulità.

2.3.8.3 Sei forme di coinvolgimento

Calleja propone di "buttare fuori" dal termine immersione qualcosa di **pragmatico e funzionale**: un **modello del coinvolgimento di chi gioca**. Nello specifico crea **sei categorie**, che **non sono mutualmente esclusive**: in ogni gioco alcune sono più alte e altre più basse, ma la presenza di una non esclude le altre (figura 2.3.5 e tabella 2.3.1):

1. il coinvolgimento **cinestetico**;
2. il coinvolgimento **spaziale**;
3. il coinvolgimento **condiviso**;
4. il coinvolgimento **narrativo**;
5. il coinvolgimento **affettivo**;
6. il coinvolgimento **ludico**.

Gli esempi saranno **volutamente cross mediali**, con anche dei videogiochi, "per farvi vedere fin dove si può arrivare con questi coinvolgimenti" e per far capire che è una cosa trasversale a tutto il game design: "non è che se parliamo di giochi da tavolo, allora alcune di queste cose non ci interessano".

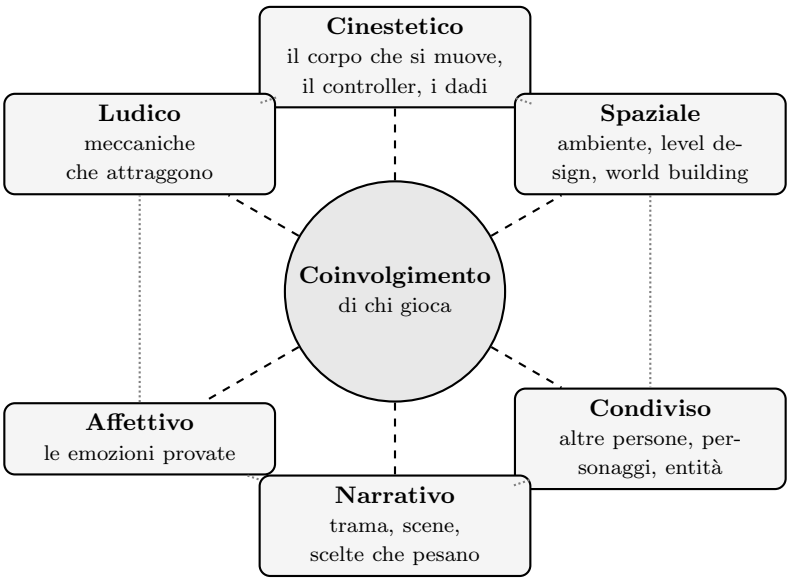


Figure 2.3.5: Le sei dimensioni del Player Involvement Model di Calleja come le presenta il docente. Le linee punteggiate ricordano che le aree "si parlano": non sono stanze stagne, e una stessa scelta di design ne tocca spesso più d'una.

2.3.8.4 Il coinvolgimento cinestetico

È il coinvolgimento che si ha quando un gioco **coinvolge il corpo** e ci si **muove**. *Dance Dance Revolution*, giocare con la Wii, fare sport, entrare nella realtà virtuale: tutto coinvolgimento cinestetico. Lo è anche un **boffer LARP**, quando bisogna combattere con spade imbottite e schivare i colpi. Ma anche cose molto più piccole:

- **usare un controller** è cinestetico;
- **tirare i dadi** è cinestetico;
- **pescare carte** è cinestetico: "se vi piace pescare carte, un gioco che vi fa pescare carte a manetta andrà a titillare il vostro senso di apprezzamento dell'esperienza";
- prendere una pedina e muoverla è cinestetico, anche a *Monopoly*.

Nel gioco da tavolo c'è un intero genere che si basa su questo coinvolgimento: i **giochi di destrezza** (*dexterity*), dalle schicchere ai giochi in cui bisogna impilare cose. Il **Jenga**[15] ha un coinvolgimento cinestetico? Sì.

Cursori, non interruttori. Da qui un'avvertenza che vale per tutte le categorie: **non sono booleane**, "non è che o c'è o non c'è". Vanno immaginate come **cursori** (*slider*): ce ne può essere pochissimo, poco, abbastanza, tantissimo. Il Jenga ha un coinvolgimento cinestetico molto alto; il tennis più alto ancora; ma non per questo il Jenga "non è coinvolgente" cinesteticamente rispetto al tennis.

[15] La trascrizione dice "Genga" e "Gengar".

2.3.8.5 Il coinvolgimento spaziale

È il coinvolgimento che riguarda **l'ambiente**: il **level design**, il **world building** del gioco.

- Un LARP fantasy o medievale in un **castello**: la slide mostra un castello polacco "famoso perché ci hanno fatto centinaia di LARP" [16]. "Non sono nel mio salotto di casa, sono in un castello".
- Il caso studio di *Monitor Celestra*, visto la volta precedente: un LARP di fantascienza dentro un incrociatore, con i neon e le pareti metalliche. "Un certo effetto te lo fa".
- I **videogiochi di guida**: se ci si pensa, sono tutti basati sul **tracciato**, e il tracciato è una strada, cioè spazio che si attraversa. "Il design di quello spazio è il 90% di questi giochi": curve strette o larghe, rettilinei. E si può estendere all'atmosfera, al clima: piove, c'è nebbia, c'è il sole, è notte, ci sono i neon.
- Un **dungeon crawler** in un gioco di ruolo cartaceo: la mappa del dungeon, anche se i giocatori non la vedono perché è dietro lo schermo del master, è coinvolgimento spaziale; e lo è quando il master **descrive** un ambiente, che cosa si sente, i suoni, gli odori, i rumori. "Non è che se non vediamo lo spazio non c'è: ce lo stiamo immaginando, e quell'immaginarcelo ci coinvolge spazialmente".
- Gli **open world**, nei videogiochi ma anche nei giochi da tavolo: generi che si basano su un mondo aperto ed esplorabile, in cui **lo spazio non è una limitazione ma un punto di *agency***. Lo spazio dice: puoi andare dove vuoi, puoi fare quello che vuoi.

2.3.8.6 Il coinvolgimento condiviso

È il coinvolgimento che si ottiene quando si **condivide l'esperienza con altre entità**.

- Di solito sono **amici**: un gioco online, come *World of Warcraft* "all'epoca", giocato con persone amiche. Si è coinvolti in un gioco che da soli magari non si giocherebbe, o che non prenderebbe così tanto; e invece "ogni martedì sera ci si trova a giocare".
- Ma si estende anche ai **personaggi digitali**: un personaggio che mi sta subito simpatico ("non so chi sia, ma…"); oppure esce l'ennesimo titolo di una serie con lo **stesso protagonista**, e "io sono già coinvolto prima ancora di accendere il gioco".
- Il **gioco di ruolo da tavolo** punta tantissimo sul coinvolgimento condiviso: è spesso un'esperienza sociale.

Va tenuto in considerazione soprattutto nei giochi **multiplayer**, perché lì il coinvolgimento condiviso "ce lo mette in mezzo" comunque: **se non lo progettate voi, lo progettano le persone al posto vostro**. E non è sempre un'idea fantastica, perché vuol dire che parte dell'esperienza è fuori dal controllo del designer.

2.3.8.7 Il coinvolgimento narrativo

È il coinvolgimento che arriva dall'**esplorazione narrativa**: trame, plot, twist, sviluppo di quello che succede in scena, scene, momenti, dialoghi. È quello che fa dire: **voglio sapere come finisce questa storia**.

- Un gioco investigativo **in tribunale**[17], in cui si è un avvocato che deve capire che cosa è successo e scagionare il proprio cliente: "ne hanno fatti sei", ed è un successo gigantesco proprio perché il coinvolgimento narrativo è fortissimo. I casi sono intriganti e si vuole sapere la verità dietro il mistero.

[16] Con ogni probabilità il castello di Czocha, in Polonia, noto per i LARP della serie *College of Wizardry*; la trascrizione dice "un castello bassa famoso".

[17] La trascrizione dice "E Saturday". Dalla descrizione (un avvocato in tribunale che deve scagionare il proprio cliente, sei capitoli principali) si tratta con ogni probabilità della serie *Ace Attorney*.

- I giochi di Telltale[18], "di fatto l'equivalente di libri game in formato digitale": scelte ramificate, A o B, che impattano sul mondo di gioco e possono cambiare drasticamente la narrazione. La *agency* ha un peso narrativo: non "mi vesto di blu, mi vesto di rosso", ma "muore Tizio o muore Caio".

- Il **gioco di ruolo**: spesso non si gioca solo per entrare nel dungeon, uccidere mostri, prendere il tesoro e uscire ("alcune persone giocano anche per quello"): gran parte del suo fascino è l'**agency emergente**, la possibilità di impattare la storia con le proprie scelte.

- *Non cedere al sonno*[19]: la slide mostra un pezzo della scheda personaggio. Alla creazione non si chiedono numeri, non statistiche come destrezza, forza, intelligenza: si chiede **che cosa ti tiene sveglio**, che cosa ti è successo, come gli altri ti percepiscono **in superficie** e come sei **dentro**. Si può giocare tantissimo su questa collisione narrativa. È chiaramente un gioco che, **già da design**, fa creare personaggi sfumati.

2.3.8.8 Il coinvolgimento affettivo

"Questo ve lo dico per esperienza: molte persone lo confondono e lo sovrappongono con il narrativo, ma non è così". Il coinvolgimento **affettivo** riguarda **le emozioni provate**. Spesso le emozioni le danno le storie, "perché siamo esseri umani", e quindi un gioco che coinvolge affettivamente spesso coinvolge anche narrativamente. Ma non necessariamente: **anche gli scacchi possono coinvolgere affettivamente**, "e non è che lo storytelling, il world building degli scacchi sia proprio questo capolavoro".

- Ci rientrano molti **giochi focalizzati**, giochi su tematiche delicate, giochi che puntano a provocare specifiche emozioni molto forti, "giochi in cui si piange a manetta".

- Ma il coinvolgimento affettivo può anche essere **positivo**: un gioco che fa ridere tantissimo, come un **party game**.

- I giochi **Jeepform** visti come casi studio (*Fat Man Down*, il gioco della coppia perfetta[20]): narrativamente non hanno quasi niente, "è una riga la premessa del gioco". Non si giocano per il coinvolgimento narrativo: si giocano perché hanno un coinvolgimento affettivo "grosso come un tir che ti viene in faccia".

***Involved*, non *immersed*.** In chat **Lorenzo** chiede se allora "immersione" vada intesa più come *to be involved* che come *to be immersed*. Il docente precisa: qui si sta parlando di un framework specifico, quello di Calleja, e la **policy del corso** è adottare questo framework per parlare di ciò che di solito si chiama immersione. Immersione resta una parola vietata perché **non dà strumenti**: se qualcuno chiede "un gioco più immersivo", non vuol dire niente; se chiede "un gioco con più coinvolgimento affettivo" o "più coinvolgimento narrativo", si hanno già delle **aree operative**, si sa dove andare a lavorare.

E le aree servono a progettare proprio perché non si escludono: un gioco può avere tanto coinvolgimento su più fronti, e magari nessuno su altri che non interessano. Il docente mette a confronto due estremi (figura 2.3.6):

- a un **Jeepform** del coinvolgimento spaziale "non gliene frega niente": lo si gioca vestiti come si è, nel salotto di casa;

- un **LARP *blockbuster*** di due giorni in un castello punta invece tantissimo sul coinvolgimento spaziale.

"Non c'è giusto o sbagliato": il framework **non è qualitativo**. Non dice che se c'è tanto di tutto il gioco è bello per forza, e se c'è poco di tutto è brutto per forza. Serve a dire **che cosa interessa e che cosa serve** al proprio gioco.

[18] La trascrizione dice "Dreadnought Helltail": con ogni probabilità Telltale Games, lo studio noto per le avventure narrative a scelte ramificate.

[19] È l'edizione italiana di *Don't Rest Your Head* di Fred Hicks, già citato nella lezione sui casi studio; la trascrizione dice "Non ci Dare al Sonno".

[20] La trascrizione lo chiama "Lady E8" e "le IDE8"; nella lezione precedente era trascritto "Lady 8". Il titolo resta non verificabile.

2.3.8.9 Il coinvolgimento ludico

L'ultimo tipo è il coinvolgimento **ludico**: quello dato da **meccaniche che piacciono**, che attraggono, che "a vederle vi fanno dire: questo gioco devo provarlo".

- Piace tirare i dadi? In un nuovo gioco si tirano un sacco di dadi: "già magari ci prende bene". Ma per chi odia tirare i dadi, vedere dei dadi in un gioco funziona già da **anti-coinvolgimento**.
- Il docente ama gli **economici azionari ferroviari**: ne vede uno nuovo e "sono già intrigato, voglio già provarlo". Altri amano i giochi di ruolo, le escape room.
- Piace costruire un **mazzo personalizzato** come in *Magic: The Gathering*, *Yu-Gi-Oh!*, *Pokémon*? È un tipo di coinvolgimento che si può progettare, pensando a **per chi** è il gioco.
- Il **piazzamento lavoratori** (*worker placement*), genere che "va molto in Germania, in Europa": il docente conosce almeno tre persone a cui "si illuminano gli occhi" quando gli si propone un piazzamento lavoratori, e altre che invece "se ne vanno".

Il coinvolgimento è soggettivo: **non esiste un passe-partout che coinvolga tutte le persone**. Le aree servono a capire che cosa serve al proprio gioco, che cosa metterci, dove andare a lavorare.

Le scelte di design toccano più aree. Uno studente trova interessante la distinzione, perché pensava che il ludico rientrasse nell'affettivo. Il docente aggiunge una cosa importante: **molte scelte di design coprono più aree**. Un **boffer LARP** dark fantasy con le armi di gommapiuma e un sistema di combattimento è coinvolgimento cinestetico, perché porta a combattere fisicamente; ma il boffer è anche una categoria meccanica, e chi apprezza le meccaniche boffer andrà a giocare boffer: quindi è anche coinvolgimento **ludico**. "Queste non sono sei aree murate con dei muri di acciaio che separano in maniera stagna le aree del game design": si ibridano, si mischiano, e una meccanica che si aggiunge può impattarne molte.

Coinvolgimento	Che cosa coinvolge	Esempi citati a lezione
Cinestetico	il corpo, il movimento, la manipolazione	*Dance Dance Revolution*, Wii, VR, sport, boffer LARP; controller, dadi, pescare carte, muovere pedine; giochi di destrezza, Jenga
Spaziale	ambiente, level design, world building, atmosfera	LARP nel castello, *Monitor Celestra*, tracciati dei giochi di guida, il dungeon descritto dal master, open world
Condiviso	altre persone o entità con cui si condivide l'esperienza	amici online (*World of Warcraft*), personaggi ricorrenti di una serie, il GDR da tavolo come esperienza sociale
Narrativo	trama, plot, twist, scene, scelte che cambiano la storia	la serie in tribunale (*Ace Attorney*), i giochi Telltale, l'agency emergente del GDR, la scheda di *Non cedere al sonno*
Affettivo	le emozioni provate, anche positive	giochi focalizzati, party game, Jeepform (*Fat Man Down*, la coppia perfetta), persino gli scacchi
Ludico	meccaniche che attraggono (o respingono)	tirare dadi, economici ferroviari, deck building, piazzamento lavoratori, boffer, tornei

Table 2.3.1: Le sei forme di coinvolgimento del Player Involvement Model, con gli esempi della lezione.

2.3.9 Le domande sul coinvolgimento

E la competizione? Una domanda: un coinvolgimento legato alla **competizione** rientra nel modello? Il docente rilancia: che cos'è che attrae della competizione? Se è il senso, "più lato sportivo", di **superare un altro essere umano**, o un'altra squadra di esseri umani, allora si è nel coinvolgimento **condiviso**: sapere

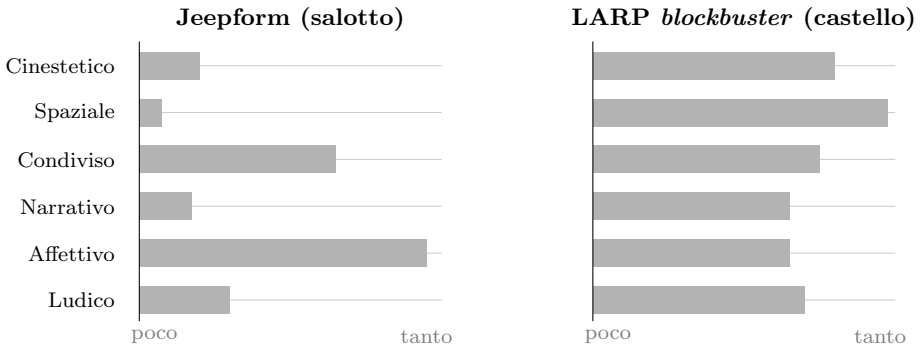


Figure 2.3.6: Due profili di coinvolgimento, a titolo puramente illustrativo. Il docente cita solo il contrasto sullo spaziale (nullo nel Jeepform, altissimo nel LARP in castello) e l'affettivo "grosso come un tir" del Jeepform; gli altri valori sono ipotesi plausibili, utili a ricordare che il modello descrive un profilo, non un punteggio.

che dall'altra parte c'è un altro essere umano da battere. Ma la competizione può anche essere **meccanica**: molte persone vanno a un **torneo** con un approccio da *min-maxer*, perché quel gioco "meccanicamente mi attrae", e allora è coinvolgimento **ludico**. Le due cose non sono sbagliate: si torna sempre a **per chi è il gioco**. Un gioco può essere competitivo senza essere da torneo, senza essere per chi fa min-maxing, senza essere per chi vuole muoversi fisicamente. "Non esiste una combinazione giusta di queste aree": sono uno strumento "sulla carta" per dire su che cosa il gioco spinge e su che cosa non spinge, o non interessa.

Non fate *Magic* 2.0. Anche perché, "purtroppo", spingere su tutto costa tantissimo in termini di budget, risorse, tempo, playtest. Il docente ribadisce: **non fate giochi giganteschi in questo corso**, "lo dico per voi, perché se no poi non li portate a compimento". "Non fate *Magic: The Gathering* 2.0". Si può fare un gioco di carte, ce ne sono tantissimi anche compatti. *Magic* è stato citato da più persone, anche nelle presentazioni ("Giordano, tu sicuramente"); ma esistono giochi di carte compatti, creati anche da persone che hanno lavorato a *Magic*, come *Compile*[21], che hanno le stesse dinamiche interpersonali di competizione di *Magic* con un numero di carte ridottissimo, in una scatola molto compatta: "più nell'ottica di cose che potete fare all'interno di questo corso".

Il *checkpoint* prima della pausa. Prima della pausa il docente si ferma: "da quando facciamo la pausa, io darò per scontato che voi avete capito questo framework". "Non esistono domande stupide", e una domanda che sembra stupida può essere importantissima anche per gli altri. Rimette la slide con le tipologie di coinvolgimento e scrive in chat il nome del libro di Calleja (il nome, non il link, visto il problema dei link).

Perché il coinvolgimento al centro? Una studentessa chiede: se il coinvolgimento è ovviamente qualcosa da perseguire, perché il modello lo mette al centro? Qual è il fine ultimo? Il docente risponde che il modello nasce dalla necessità di **smettere di usare "immersione"**; ma l'immersione non sarebbe una parola così usata e pervasiva se non fosse **qualcosa che chi gioca cerca**. Tutti cercano e apprezzano l'immersione, pur avendone definizioni diverse: "adesso qui siete in sette, e avete sette definizioni di immersione diverse", magari sovrapponibili per alcuni giochi. Ma gli studenti, da qui, sono **game designer**, non solo persone che giocano, e serve loro sapere **su che cosa stanno lavorando**: avere strumenti pratici per **confrontare il proprio lavoro con le intenzioni**. Lo stesso lavoro di decostruzione si farà più avanti con il **divertimento**. E il docente torna alla slide iniziale: **il gioco è un'esperienza creata da regole**; si possono creare quelle regole, e dentro quelle regole si può lavorare su questi tipi di coinvolgimento.

La distanza tra esperienza intesa ed esperienza giocata. Il salto da un'idea a un regolamento scritto, senza perdere cose per strada, "è molto difficile". È un tema a cui il docente tiene moltissimo: la sua **tesi**

[21] La trascrizione cita "Redlands o Compile". Il primo titolo non è ricostruibile con certezza; il docente li presenta come giochi di carte competitivi, compatti, con pochissime carte in una scatola piccola.

di master era proprio sul **gap tra le intenzioni di progettazione** di un game designer e le persone che interpretano il regolamento per poi vivere l'esperienza, misurando **quanta distanza c'è tra esperienza intesa ed esperienza giocata**. **Lorenzo** scrive in chat che un designer può pianificare un prodotto per coinvolgere l'utente su certi aspetti, ma poi è quest'ultimo a decidere che cosa lo coinvolge davvero. "È esattamente quello che vi ho appena detto": la parte difficile del lavoro, e quella che, se fatta bene, fa fare passi avanti, è **distillare in un regolamento non ambiguo** procedure, meccaniche, "un ecosistema di regole" che porti a far giocare un'esperienza specifica a chi gioca, **senza che vi conosca e senza che vi possa chiedere cose**.

È facile se il gioco lo si organizza di persona: lo si spiega, si corregge chi sbaglia. Ma è un caso rarissimo. Nella maggior parte dei giochi pubblicati, il designer non lo si incontrerà mai, e a volte non si sa nemmeno chi sia: "che ne sapete del suo costruttivismo personale?" Che cosa c'è nel gioco perché quella persona ci crede fermamente, ed è parte del suo paradigma, e che cosa c'è **per sbaglio**? "Lo farete anche voi", non per sfiducia, ma perché è inevitabile: non si può essere costantemente attenti a tutto, e a volte si prendono decisioni di design senza pensarci, **per vestigialità, per abitudine**. In quel momento si sta riproponendo un paradigma: si torna a **fare tradizione**, non design.

Le domande di Tiziana. Tiziana fa due domande. La prima: il playtest si fa quando si ha il regolamento? Il docente la ferma subito, "non perché non mi interessa la domanda": ci sarà una lezione apposta, e aprire adesso "il vaso di Pandora di come si struttura un playtest" impedirebbe di arrivare al secondo framework. Ed è una fase delicatissima: se la progettazione e il prototipo sono "la vostra mente e il vostro gioco", il playtest è il momento in cui "questa roba va a schiantarsi con persone esterne che non l'hanno mai giocata".

La seconda: le categorie del modello le sono sembrate molto puntuali. Si era chiesta perché non ci fosse un "coinvolgimento sociale", e si è risposta da sola: "sociale" vuol dire una marea di cose, "condiviso" è più preciso. Esatto, e non solo: **condiviso** permette di includere entità **non necessariamente umane**. Il personaggio a cui si è affezionati in altri giochi della serie coinvolge non per la meccanica né per la storia: in *Mario Kart*, "che non è un gioco con altissima narrazione", rivedere un personaggio già incontrato può far dire "scelgo questo perché mi sta simpatico". Di nuovo: le aree non sono mutualmente esclusive, "si parlano".

La pausa. "Facciamo pausa". Il docente la fissa fino alle "e trenta", cioè fino alle **18:30**[22].

Approfondimento: oltre le sei dimensioni. La lezione si ferma alle sei dimensioni, ma il titolo del libro di Calleja indica dove l'autore vuole arrivare. Nel suo modello ogni dimensione si può osservare su due scale di tempo: il coinvolgimento **micro**, momento per momento, mentre si gioca, e quello **macro**, che riguarda il rapporto con il gioco nel tempo, anche quando non si sta giocando (la voglia di tornarci, il pensarci, il parlarne). La parola *incorporation*, che Calleja propone al posto di *immersion*, indica il momento in cui le diverse dimensioni del coinvolgimento si fondono e il giocatore "assimila" il mondo di gioco nella propria coscienza come uno spazio abitabile, con il proprio avatar come punto di presenza. Il passaggio è lo stesso che il docente chiede agli studenti: sostituire una metafora vaga (l'immersione, come in una piscina) con una descrizione delle componenti, su cui si può lavorare una per una.

2.3.10 Il Mixing Desk of Larp

"Rieccoci qua". Dopo la pausa (Lorenzo conferma di esserci) si parte con il secondo framework grosso, "non che quello di prima fosse piccolo".

2.3.10.1 Da dove viene

Il framework **nasce all'interno dei movimenti scandinavi** di gioco di ruolo, nella scena del **Nordic LARP**[23]. Ma per sua natura è uno strumento **estremamente flessibile**: è stato applicato anche al gioco

[22] La trascrizione dice "sono le 21, facciamo pausa fino alle 30" e subito dopo "ci si ritrova qui alle 18.30": la pausa comincia quindi alle 18:21 circa. Durante la pausa la trascrizione registra per decine di righe frasi senza senso ripetute in loop.

[23] Il *Mixing Desk of Larp* è stato sviluppato nei primi anni Dieci da designer norvegesi legati alla *Larpwriter Summer School*, e poi più volte rielaborato; la storia è raccontata nell'articolo di *Analog Game Studies* che il docente consiglia alla fine della

da tavolo e si può applicare ad altre tipologie di giochi. È **modulare**: lo si può modificare, aggiungere o togliere pezzi in base a quello che serve per il proprio gioco, "perché non tutti i moduli, non tutte le parti di questo framework vi servono sempre".

Molti esempi saranno giochi di ruolo dal vivo, "solo perché è più facile spiegare" alcuni cursori così. Il docente chiede di fare ogni volta il **parallelismo**: questo pezzo del framework è nato sul gioco di ruolo, ma come si applica al resto? Per alcuni cursori il salto è facile, per altri più complesso; ma "se tutto va bene, e mi è sembrato bene in passato", alla fine gli studenti dovrebbero avere un'ottima idea di come funziona e, soprattutto, di come **applicarlo in modo pragmatico**.

2.3.10.2 Un doppio uso

Ancora più del Player Involvement Model, il Mixing Desk ha un **doppio utilizzo**:

Analisi si prende un gioco già pubblicato, lo si "fa passare" nel framework e si guarda che cosa ne esce.

Progettazione è "un framework ottimo per crearvi delle **linee guida**": voglio un gioco che faccia A, B, C, D, e il framework aiuta a capire che cosa sono A, B, C, D e come si vuole che arrivino.

2.3.10.3 Il mixer

Il nome è "abbastanza autoesplicativo": la slide mostra quello che sembra un **banco di mixaggio** luci e audio. L'idea è che ogni gioco possa essere identificato da una serie di **cursori** (*slider*, o *fader*), le levette che si spingono su e giù in un mixer, e che l'insieme dei cursori usati per progettare o analizzare il gioco ne dia una **visione d'insieme**: che cosa fa questo gioco, "non tanto a livello micro, cioè nelle singole regole, ma a livello esperienziale, cioè dall'alto" (figura 2.3.7).

L'ultimo cursore della slide è **"inserisci qui il tuo fader"** (*your fader here*): "modificate questo slider come volete". Nella sua storia il Mixing Desk è stato elaborato, modificato e rimaneggiato più volte, anche da settori paralleli come il gioco di ruolo formativo e il gioco da tavolo. Il docente mostra "il primo, quello da cui è nato"; online se ne trovano "mille versioni". Il consiglio non è andarle a cercare, ma **capire che cosa serve al proprio gioco e creare i propri cursori**.

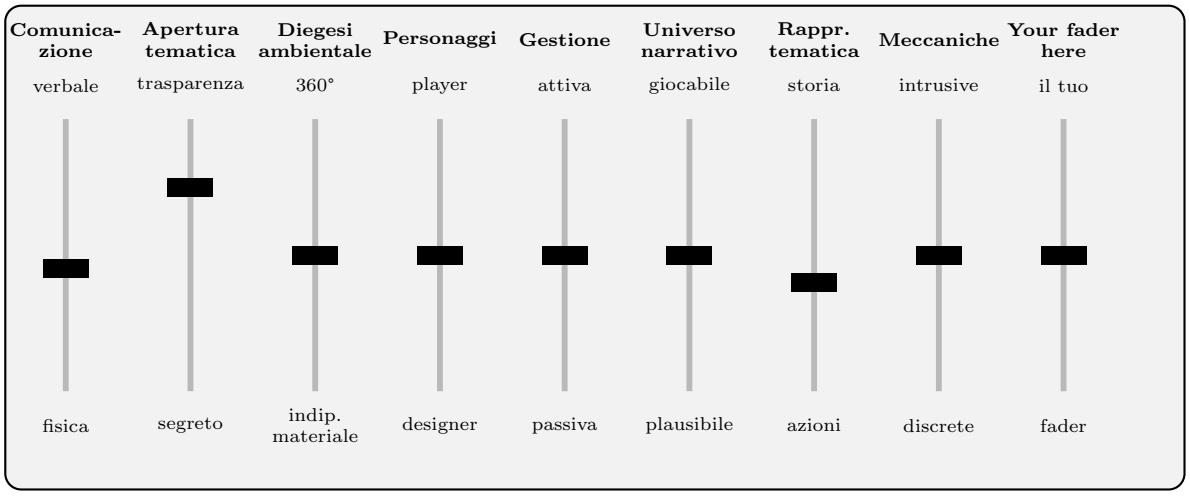


Figure 2.3.7: Il Mixing Desk come banco di mixaggio: otto cursori della versione mostrata a lezione più il cursore libero. Le posizioni sono qui tutte centrali o quasi, solo per mostrare che ogni cursore può stare a qualsiasi altezza: nessun estremo è "meglio" dell'altro.

lezione (sezione 2.3.14).

2.3.11 I cursori del Mixing Desk

2.3.11.1 Comunicazione: verbale o fisica

Si prende un LARP, un gioco in cui si interpreta fisicamente il proprio personaggio. **La comunicazione tra chi gioca è verbale o fisica?** E "lo ricordo: questo è uno slider, non è un pulsante on, off, uno, zero". Ci sono vie di mezzo.

- Un **boffer LARP** è molto fisico: il combattimento è una parte importante del gioco, e impugnare un'arma in plastazote, colpire, schivare, prendere lo scudo, riattaccare dal fianco è un'interazione molto fisica.
- Il gioco della coppia perfetta, il Jeepform citato prima, è "estremamente verbale".
- Nei giochi da tavolo, un **party game** va di solito più sul verbale. Negli **scacchi** si parla pochissimo: l'interazione ludica non è di dialogo, è "io muovo cose", e muovere cose è un'azione fisica.

La domanda da farsi: **il mio gioco è un gioco in cui si parla o un gioco in cui si spostano cose?** E le "cose" possono essere anche le persone stesse, come negli sport o nel gioco dal vivo. "Spostare cose" riassume il fatto di interagire fisicamente con qualcosa. Ci si può stare in mezzo: un gioco in cui si parla e si spostano cose, ma si parla più di quanto si spostino cose, o viceversa.

I giochi verbali e i loro limiti. "Qualcuno ha un esempio di un gioco da tavolo estremamente verbale?" Uno studente propone **Dixit**. È verbale, "ma non è così tanto verbale", perché ha moltissime **limitazioni sulla comunicazione**: se in *Dixit* si potesse parlare liberamente di tutto, il gioco crollerebbe. Lo stesso vale per *Secret Hitler*, per *Taboo*, per *Codenames* ("nomi in codice"), dove si può dire una parola e un numero, non di più. Molti giochi deduttivi o induttivi hanno fortissime limitazioni su che cosa si può e non si può dire.

L'obiezione di Gianmarco. Gianmarco vede una difficoltà: qui verbale e fisico sono messi in contrapposizione, ma l'esempio dei giochi verbali con limitazioni sembra non stare sullo stesso asse. Il docente spiega che sono messi come **estremi**, come **direzioni di game design diverse**: "immagina che hai cento chili di roba nel tuo game design, e di decidere quanti chili voglio che siano fisici e quanti verbali" ("una metafora che non funziona, ma spero che ci siamo capiti lo stesso"). Una studentessa aggiunge che *Taboo* non è solo verbale: c'è gestualità, comunicazione non verbale, "non giochiamo bendati". E le limitazioni verbali, chiede Gianmarco, si trasformano in dinamiche fisiche? "Non necessariamente", ma spesso si trasformano in **altre cose da fare al tavolo**: le carte illustrate di *Dixit* sono un elemento fisico usato per comunicare in modo non verbale; la griglia di parole di *Codenames* è un aspetto fisico di interazione, anche se ha parole scritte sopra. Il Mixing Desk non va guardato come "lo stato oggettivo del gioco dall'alto", ma come una **bilancia**: se si spinge da un lato, l'altro tende a scendere.

Gli estremi. Ci possono essere anche giochi all'estremo totale. *Stranger*, il LARP della designer danese visto nei casi studio, quello della nazione blu e della nazione verde, è un gioco in cui **è vietato parlare** e fare suoni: è tutto fisico. All'opposto, ci sono giochi con *roleplay* in cui la componentistica è praticamente inesistente; e anche *Dungeons & Dragons*, se si giocano i combattimenti senza griglia, diventa molto verbale e molto poco fisico. Ma "non siamo esperimenti in astratto": è quasi impossibile eliminare del tutto uno dei due estremi.

2.3.11.2 Apertura tematica: trasparenza o segreto

"Cercherò di spiegarmi al meglio, perché è importante ma apre dei vasi di Pandora". Supponiamo che il gioco abbia un **tema**. Quel tema arriva a chi gioca **in modo trasparente**, o è un **tema segreto** che si scopre durante o alla fine del gioco?

Trasparenza. *Twilight Struggle*, della GMT: Stati Uniti contro URSS, la **Guerra Fredda**. Due persone, una gioca gli Stati Uniti, l'altra l'URSS; non ci si combatte direttamente con i carri armati, ma si cerca

di vincere ottenendo influenza e supporto di varie nazioni. Il tema è trasparente: entrambi i giocatori sanno che cosa stanno giocando.

Segreto. Un gioco in cui si trasportano cubetti e risorse in treno da un posto all'altro, e vince chi ne trasporta di più. Alla fine si scopre che si stavano trasportando **ebrei nei campi di concentramento**. "Questo è un caso studio esistente": è *Train* di **Brenda Romero**[24], "storicamente riconosciuto come gioco dal tema offuscato" che viene rivelato durante il gioco.

Anche qui ci sono vie di mezzo: molti LARP "vecchia scuola" hanno **trame segrete**, plot nascosti noti solo ad alcuni personaggi, che si rivelano nel *debriefing* ("sapevate che in realtà Tizio faceva anche questo?").

La preferenza del docente. È una cosa soggettiva, e il docente lo dice come tale: preferisce i giochi **trasparenti**. Trova "molto pigro" affidarsi ai segreti, specialmente quando si tratta di temi delicati. Ma giochi così esistono, e il cursore esiste.

Si gioca una volta sola? Tiziana chiede se un gioco così si giochi una volta e poi rischi lo *spoiler*. "Non è un problema": non sono esperienze meno significative. Come i **giochi investigativi**, che si basano proprio su un'apertura tematica segreta. Fare un investigativo in cui si dice subito chi è l'assassino e come ha ucciso è "molto difficile", ma ci sono le eccezioni. Il docente cita **il tenente Colombo**, "che amo particolarmente" ("vi prego, ditemi che non sono l'unico"): la serie si apriva mostrando l'assassino e l'omicidio, e solo dopo arrivava Colombo a indagare. Lo spettatore sa già tutto; il protagonista no. L'interesse sta nel conflitto tra chi sa e chi non sa, nel capire come Colombo arriverà a ciò che lo spettatore ha visto. Allo stesso modo si può avere un investigativo da tavolo in cui si sa già com'è morta una persona: "è una bella sfida".

Un segnalibro. Quando si parla di **giochi formativi o didattici**, questo cursore è per il docente "particolarmente sensibile e importante". E, di nuovo, non tutti i cursori interessano ogni gioco: in un astratto alla scacchi o dama, "che ve ne importa dell'apertura tematica?".

Approfondimento: *Train* e il tema sotto le meccaniche. *Train* compare anche in *A Theory of Fun for Game Design* di Raph Koster, che lo cita come esempio della serie *The Mechanic is the Message*: un gioco sulla **complicità** dei giocatori nel compiere azioni terribili, o nel trovare il modo di sovvertire il sistema. Lo stesso libro propone un esperimento mentale che illumina il rapporto tra tema e meccaniche: si immagini un gioco in cui si lasciano cadere vittime innocenti di ogni forma e misura in una camera a gas a forma di pozzo, dove cercano di arrampicarsi le une sulle altre; se ci riescono si perde, se le si incastra abbastanza bene quelle in fondo muoiono. "Non voglio giocare questo gioco. E voi?", chiede Koster. Eppure è *Tetris*. La conclusione di Koster è che **le meccaniche nude non determinano il significato** di un gioco: come il cinema non è solo fotografia o sceneggiatura, il gioco è l'insieme di meccaniche, regia, grafica. *Train* usa esattamente questa distanza tra meccanica e tema: le stesse azioni, caricare cubetti su un vagone, cambiano completamente significato nel momento in cui il tema viene aperto. Il cursore dell'apertura tematica, in questo caso, è la leva principale dell'esperienza.

2.3.11.3 Diegesi ambientale: 360° o indipendenza materiale

Quello che c'è attorno a chi gioca, l'ambiente in cui si gioca: il coinvolgimento spaziale è **a 360 gradi**, o il gioco ha un'**indipendenza materiale**? Nel gioco di ruolo dal vivo:

- si gioca un fantasy dentro un castello, a cavallo, "e abbiamo un vero cavallo" sotto di noi;
- oppure si gioca il gioco della coppia perfetta nel bagno di casa, ma lo si potrebbe giocare in salotto, o in giardino.

[24] La trascrizione dice "Trains di Prenderò Mero". *Train* (2009) fa parte della serie di giochi da tavolo *The Mechanic is the Message* di Brenda Romero, all'epoca Brathwaite, coautrice di *Challenges for Game Designers*.

Il gioco richiede una **diegesi il più alta possibile** per essere giocato, oppure no? E ci sono vie di mezzo: si può giocare nello studio di casa **vestiti da cavalieri**, "why not?". Molti **chamber LARP** italiani degli anni '90 e 2000 funzionavano così, e il docente mostra al volo una slide: un LARP in costume giocato **nello scantinato** della casa di qualcuno. Non è completamente indipendente dal materiale, perché i costumi erano richiesti, ma "non siamo in un castello, è uno scantinato".

Vale anche per gli altri giochi: un **dungeon crawler** da tavolo mostra l'ambiente nel dettaglio, o ne dà una rappresentazione metaforica, astratta?

2.3.11.4 Creazione dei personaggi: chi gioca o chi progetta

Ancora pensando al LARP ("sui personaggi abbiamo una lezione apposta", e arriverà presto): i personaggi possono essere scritti **da chi partecipa** o **da chi progetta il gioco**, e sono due esperienze chiaramente diverse.

Lato player. Chi gioca crea il proprio personaggio e decide chi interpretare. È spesso la formula dei **giochi molto grossi**, i LARP *blockbuster* da 300, 400, 500 persone, o i boffer tedeschi da mille persone: i designer non si mettono a scrivere mille personaggi, ma chiedono di scriverlo e di mandarlo, a volte, per approvazione. Se non ci sono cose particolarmente sbagliate, si gioca chi si vuole.

Lato designer. Il personaggio è creato da zero da chi progetta il gioco. Il vantaggio: si può mettere **tematicamente** nel personaggio quello che si vuole.

Non è così diverso dai videogiochi di ruolo: ci sono i *Final Fantasy*, soprattutto i più recenti, in cui i designer hanno creato i personaggi e se ne segue la storia; e ci sono i giochi di ruolo occidentali, i CRPG, con una fase di *character creation* in mano al giocatore, magari con dei paletti messi dal designer.

I paletti di *Non cedere al sonno*. Un esempio di via di mezzo. In *Non cedere al sonno* si giocano persone **insonni** che vedono una città che non esiste, piena di orrore e di incubi. Si può fare il personaggio che si vuole, ma con due paletti forti, decisi dal designer: il personaggio deve essere **insonne**, e deve avere **un problema che lo tiene sveglio**, cioè una motivazione specifica e consapevole della propria insonnia. Non si può giocare un insonne che non sa perché non dorme. Uno studente chiede di ripetere il titolo: *Non cedere al sonno*, ripubblicato di recente in italiano da un editore che la trascrizione chiama "Grampy Bear"[25].

2.3.11.5 Gestione del gioco: attiva o passiva

C'è una figura, come un **master** o una persona di **facilitazione**, che fa andare avanti il gioco, o un gruppo può giocare e finire il gioco **senza nessuno che facilita**? Riguarda giochi pervasivi, giochi di ruolo dal vivo e da tavolo.

Gestione attiva. Nei giochi di ruolo grossi, a campagna, o nei *blockbuster* da centinaia di persone, ci sono i cosiddetti **PNG** (*NPC*), spesso membri dell'organizzazione, che sono lì per **innescare eventi specifici** e far andare avanti il gioco. C'è una persona del team che spinge attivamente avanti il gioco.

Gestione passiva. Nessuno del team o della facilitazione spinge il gioco: le persone che giocano lo portano avanti in autonomia.

Uccidere il re. L'esempio: in un LARP fantasy, a un certo punto qualcuno deve uccidere il re, per ragioni di trama, e non si sa se i giocatori lo faranno. Allora uno dell'organizzazione si veste da assassino e uccide il re: **gestione attiva**. All'opposto, il gioco chiede ai giocatori di uccidere il re, magari con una meccanica apposita che qualcuno deve attivare, e nessuno dell'organizzazione interviene: le persone lo fanno in autonomia, **gestione passiva**.

[25] Con ogni probabilità Grumpy Bear, editore italiano di giochi di ruolo.

Il parallelo con il gioco da tavolo. Gianmarco prova: gli **eventi** di *Nemesis* come gestione attiva, e *Root*[26], "in cui sono i giocatori a menarsi", come gestione passiva? Non del tutto, risponde il docente: gli eventi di *Nemesis* si mescolano, non se ne decide l'ordine. Un esempio migliore di gestione attiva nel gioco da tavolo sono i **giochi a campagna**, come i dungeon crawler con scenari numerati da giocare in ordine: il designer ha deciso che si gioca prima questo scenario, poi questo, poi quest'altro, e non si gioca il terzo prima "perché ci va". Lì la gestione è, in parte, di chi ha progettato la campagna.

E nei giochi pervasivi? Una studentessa chiede se il cursore riguardi solo i giochi di ruolo. No:

- *Lettere anonime per un camminatore*, il caso di Beatrice con l'MP3 in cuffia, è a metà: c'è una forte gestione attiva nella traccia preregistrata da chi organizza, ma le persone camminano con la propria velocità e i propri tempi, "non puoi obbligare la persona a correre";
- *Journey to the End of the Night* è a gestione **passiva**: runner e chaser possono metterci un'ora o cinque ore, e nessuno li disturberà;
- *A cena col nonno* è a metà: la traccia audio dà dei tempi precisi (gestione attiva), ma la conversazione al tavolo non è controllata da nessun PNG, e tutte le persone sono giocatori;
- nel gioco di ruolo da tavolo, il **master** è il lato attivo della gestione, se il gioco ne ha uno.

La plancia a due facce. Giordano chiede: e i giochi con una **plancia a due facce**, in cui il regolamento dice di usare per la prima partita il lato base e non quello avanzato[27]? È una forma di gestione attiva? Il docente non la considera tale: rientra nelle questioni di **onboarding**, accessibilità e approccio al gioco, "come dire: gioco il tutorial e poi, quando ho imparato, gioco la parte completa". Il gioco non *impone* l'ordine: lo consiglia. Se si è già giocato e lo si fa provare a un amico, si sceglie la plancia in base all'amico. Avere quella libertà vuol dire che la gestione non è attiva; diverso è un Legacy o un gioco a campagna in cui non si può giocare l'ottavo scenario prima del primo. Lo stesso vale per alcuni giochi di carte collezionabili: in *Magic* il mazzo inteso è da sessanta carte, ma in vari eventi si sono usati **mazzi dimostrativi** preconfezionati più piccoli, da quaranta carte, per spiegare il gioco a chi non l'ha mai giocato.

2.3.11.6 Universo narrativo: giocabile o plausibile

"Questo, vi giuro, è forse il mio slider preferito". Se c'è un cursore da portarsi a casa dal Mixing Desk, "secondo me questo forse è quello che vi serve di più nella vita. Come paradigma". L'universo narrativo del gioco è **più giocabile o più plausibile**? E uno dei punti del framework è che le due cose sono **in opposizione**: **più un gioco è giocabile, meno è plausibile**. Per esperienza, è difficile da far combaciare nella testa se non ci si è mai pensato a fondo.

La provocazione di Gianmarco. "Chi non è d'accordo?" Tutti d'accordo. Gianmarco fa una provocazione: forse l'unico gioco veramente giocabile e plausibile insieme è "il gioco della nostra vita: io gioco a essere Gianmarco". "Però quello non è un gioco, purtroppo". È una provocazione, ma porta alla domanda giusta: plausibile da che punto di vista?

Che cosa vuol dire plausibile. Su questo cursore da solo esiste un talk di Johanna Koljonen, la designer della citazione iniziale, che "dura un'ora e passa"; il docente lo riassume. Un universo narrativo **plausibile** è un universo che **simula la propria esistenza**: le cose hanno azioni e conseguenze simulate **come se fosse la realtà**. Ma spesso l'approccio simulativo va in **diretta opposizione alla giocabilità**.

[26] La trascrizione dice "in route": con ogni probabilità *Root*, il gioco asimmetrico di fazioni in cui sono i giocatori a scontrarsi.
[27] La trascrizione cita come esempio un gioco chiamato "Ivan", non identificabile.

La vedetta. L'esempio "banalissimo". Un LARP fantasy medievale, magari un 360°, in un castello bellissimo, per tre giorni di fila: si dorme, si mangia, si cammina in personaggio, vestiti di tutto punto in armatura. È notte, qualcuno deve fare la guardia, e tocca a voi: state sulle mura di vedetta. Se il gioco fosse **pienamente plausibile**, non succederebbe niente: statisticamente i castelli non vengono attaccati ogni giorno, e la normale notte di vedetta è una notte in cui si sta svegli **otto ore** a guardare il cielo. "Se io metto una persona a giocare questo ruolo per otto ore, questa persona si butta giù dalle mura". La **giocabilità** è l'approccio più ludico: chi vuole giocare la vedetta vuole che la vedetta noti cose, dia l'allarme. Lo stesso vale per il cavaliere: "siete Sir Cavaliere, ecco la vostra spada, ecco il vostro scudo", e in due giorni non c'è un combattimento perché è un periodo di pace. Plausibile; ingiocabile.

GURPS. Si può voler creare un gioco simulativo e plausibile, "e ce ne sono tantissimi". Nel gioco di ruolo il caso tipico è **GURPS**[28]: nasce con l'intenzione di essere plausibile, e infatti ha "microregolette per tutto"; chi l'ha giocato sa che al tavolo risulta "estremamente ostile all'essere giocato". All'opposto, ci sono giochi molto giocabili ma flessibili sulla plausibilità del mondo, perché sono più interessati ad avere scene e momenti di gioco interessanti che plausibili.

Gli open world. Uno studente chiede dei giochi digitali: *GTA* non riesce ad avere entrambe le cose? Gli *open world* sono un genere che si associa alla plausibilità, come la serie *Yakuza / Like a Dragon*. Ma hanno "grossissimi compromessi" di giocabilità: in *GTA* si può essere investiti, uccisi, colpiti, e si riparte dall'ospedale come se niente fosse, perdendo solo un po' di soldi. "La morte non è letale": se fosse plausibile, al primo incidente mortale il gioco cancellerebbe il salvataggio. Ci si può buttare da un palazzo e, a seconda di come si atterra, non farsi niente, o risvegliarsi in ospedale con la spina dorsale intatta. Molto plausibile, ma con aspetti di giocabilità che non toglie.

***Desert Bus* e *The Longing*.** Uno studente cita un simulatore di guida di un autobus in cui si percorre un'intera strada nel deserto in tempo reale, con il bus che tende leggermente a destra e costringe a restare ai comandi: è *Desert Bus*[29]. "E infatti non è giocabile". Tutto plausibilissimo, "un gioco che puoi giocare", ma non **giocabile**: perché "giocabile" non vuol dire che si può giocare, vuol dire **progettato per far spuntare momenti di gioco**. Un altro studente cita *The Longing*, il videogioco in cui si devono aspettare **400 giorni** reali: un "bel mix", dice il docente, perché il passaggio del tempo è esattamente quello reale ("io gioco domani, e sono al giorno successivo"), e se si scava un passaggio con il piccone ci vogliono trenta giorni. Interessantissimo, ma "molto poco giocabile": è un gioco che gioca sulla **frizione**.

Le vie di mezzo. "Ricordatelo: non è un pulsante". La maggior parte dei **giochi strategici e di guerra da tavolo** sono plausibilissimi, puntano sulla plausibilità del conflitto, ma la rendono anche giocabile.

Che cosa vuol dire giocabile. Una studentessa torna sulla definizione: giocabile non vuol dire "si può giocare". "È molto nebulosa questa cosa", ammette il docente, ma se la si intende come "si può giocare o no", allora **tutto è giocabile**: anche andare in banca si può gamificare, e non per questo diventa un'esperienza giocabile. Giocabile vuol dire che **dà spunto a momenti di gioco**, momenti in cui si ha un approccio ludico. Un gioco completamente plausibile è come la realtà, e quindi, da un certo punto di vista, non è più un gioco. "Un vaso di Pandora gigantesco".

Aragorn nella locanda. Un altro esempio, dal *Signore degli Anelli*: all'inizio Aragorn, alias Grampasso, aspetta gli hobbit nella locanda, incappucciato in un angolo. Lo si immagini in un LARP: l'obiettivo è aspettare gli hobbit senza farsi riconoscere, senza parlare con nessuno, perché parlare rischia di farlo scoprire. È un ruolo "estremamente non bello da giocare": una persona che per tre, quattro ore sta incappucciata in silenzio in un angolo senza interagire con niente. Plausibile narrativamente, pochissimo giocabile.

[28] La trascrizione lo chiama "Gurps", "Goobs" e "GuardStore". *GURPS* (*Generic Universal RolePlaying System*) è il sistema di Steve Jackson Games.

[29] La trascrizione lo chiama prima "simulatore di Road 66" e "il bus di raglia", poi "desert bus"; lo studente dice otto ore, il docente sei. *Desert Bus* era un minigioco della raccolta mai pubblicata *Penn & Teller's Smoke and Mirrors* (1995): il viaggio da Tucson a Las Vegas dura circa otto ore in tempo reale.

Plausibile non vuol dire peggiore. "Questi slider non sono qualitativi". Un gioco molto giocabile e poco plausibile non è per questo migliore, e viceversa: bisogna chiedersi che cosa serve al proprio gioco. Il docente rimette la slide con *Waiting for Flight*, il LARP dell'aereo, delle valigie e dei cuori di vetro: tre ore in una sala d'attesa, con le informazioni centellinate da chi facilita, che arriva due o tre volte a portare notizie (un po' di gestione attiva, quindi). Per il resto, si è abbandonati a sé stessi, senza informazioni. **Molto plausibile**, e il gioco è "incredibile": proprio il lasciare nell'ansia del non sapere è parte della "botta emotiva". Anche un gioco molto plausibile può far arrivare emozioni fortissime.

Approfondimento: plausibilità, coerenza e mimesi. *21st Century Game Design* affronta lo stesso nodo dal lato del videogioco, parlando di **astrazioni** del mondo di gioco. Bateman e Boon notano che la scelta delle astrazioni dipende da che cosa il gioco vuole essere: in un gioco di sfida (*agon*) le astrazioni devono garantire equità e il giusto grado di difficoltà; in un gioco di **mimesi** (*mimicry*) devono invece servire la sospensione dell'incredulità, perché lì "l'immersione è più importante del bilanciamento". È la stessa tensione tra giocabile e plausibile. Il libro aggiunge un punto utile anche a chi sceglie la giocabilità: il giocatore presuppone una **logica coerente** del mondo, e le eccezioni capricciose (una sola porta che si può distruggere, in un gioco in cui tutte le altre sono indistruttibili, che fa chiedere "perché non posso farlo altrove?") minano il coinvolgimento. Le convenzioni di *GTA* (si muore e si riparte dall'ospedale) funzionano proprio perché sono regole coerenti e applicate sempre, non eccezioni: si può sacrificare la plausibilità, ma non la coerenza.

2.3.11.7 Rappresentazione tematica: storia o azioni

"Siamo quasi alla fine, vi giuro". Mettiamo che il gioco non sia astratto, e che ci sia un **tema** da trattare. Il tema arriva a chi gioca **con la storia** o **con le azioni**? L'esempio: si vuole dare un'esperienza **horror**.

Con la storia. Una narrazione cupa, *grim*, le illustrazioni, il world building, i personaggi per come sono scritti e rappresentati anche visivamente.

Con le azioni. Una meccanica di **follia**, eventi completamente oppressivi e cupi che si accaniscono contro i giocatori.

È un cursore, si possono usare entrambe le cose. Ma al docente capita spesso di ricevere giochi da valutare presentati come **tematici**, in cui il tema è solo la narrazione "agganciata sopra": **le meccaniche non dicono niente di quella roba lì**. Più raramente, "per fortuna", arrivano giochi tematicamente fortissimi nelle azioni, ma in cui, come diceva Tiziana a inizio lezione, il world building e l'ambientazione si potrebbero spostare di duecento anni o in un'altra regione e il gioco resterebbe lo stesso. Non è giusto o sbagliato in assoluto: dipende dal tipo di gioco e dalle persone che lo giocano. Ma bisogna chiedersi **se le cose che il gioco trasmette le trasmette con il narrative design o con le meccaniche**.

L'unico cursore "qualitativo" del corso. Qui il docente fa un'eccezione dichiarata. Il corso è di **game design**, e gli studenti sono lì per diventare game designer, "non necessariamente narrative designer". Quindi questo è uno dei pochi cursori che **nel corso** diventa qualitativo, "non perché è meglio", ma per lo scopo didattico: nei task e soprattutto nel progetto finale, **la priorità andrà alle azioni, non alla storia**. "Io voglio vedere che voi mi fate giochi che trasmettono le emozioni con le meccaniche". Il perché, "da prendere con le pinze": è facile colpire emotivamente qualcuno con il mezzo della scrittura, "che è in giro da mille anni"; è molto più difficile far arrivare un'emozione con una meccanica progettata con l'intenzione di innescare proprio quell'emozione, e vedere che quell'emozione arriva. È la parte difficile del lavoro. E infatti nel settore ci sono parecchi narrative designer e world builder; di persone che fanno game design "ce ne sono tante, ma brave poche". "Se il tema non arriva con le meccaniche, che cosa fate un gioco a fare? Scrivete un racconto, fate una canzone, fate un quadro".

Non vuol dire astratto. Dare priorità alle meccaniche non vuol dire fare giochi "asciuttissimi" e astratti, né scacchi, né dama, né "briscola cyberpunk". Che, giura il docente, esiste davvero: un gruppo di designer spagnoli ha fatto un gioco ambientato nelle **città italiane in versione cyberpunk**. "Torniamo sul punto senza andare sui meme". E se non è chiaro, "fermatemi: mi faccio prendere, poi mi scaldo, vado a sessanta gradi corporei".

Approfondimento: il "vestito" del gioco. La posizione del docente ha un precedente illustre e una sfumatura importante. In *A Theory of Fun for Game Design* Raph Koster sostiene che il tema è spesso un **vestito** (*dressing*): il fatto di poter descrivere *Death Race*, il famigerato gioco in cui si investono pedoni, come "un gioco in cui si raccolgono oggetti su un piano bidimensionale" dimostra che il suo vestito è in gran parte irrilevante rispetto a ciò che il gioco insegna davvero. Investire pedoni, uccidere terroristi o mangiare puntini scappando dai fantasmi sono "metafore comode" per il sistema formale che sta sotto, e i giocatori esperti imparano a vedere oltre. È esattamente il problema del gioco "tematico" in cui le meccaniche non dicono niente del tema: il tema scivola via. Ma Koster stesso, come si è visto con *Tetris* e la camera a gas, riconosce che il significato di un gioco nasce dall'insieme. La richiesta del docente va letta così: non togliere il tema, ma **farlo passare attraverso le azioni**, in modo che il sistema formale e il vestito dicano la stessa cosa.

2.3.11.8 Meccaniche: intrusive o discrete

"Anche il prossimo è tosto: ho lasciato per ultimi gli slider un po' più tosti". Le meccaniche sono **intrusive** o **discrete**? Per spiegarlo il docente deve prima spiegare le **metatecniche**, e per spiegare le metatecniche deve spiegare il **metagioco**.

Il metagioco. "Qualcuno ha mai sentito parlare di metagioco?" Rispondono Lorenzo, Tiziana e Giordano. La prima definizione, dal punto di vista di *D&D* e dell'esperienza di un gruppo di gioco: il *metagame* è quando un personaggio dice o fa qualcosa che **non dovrebbe sapere**, perché lo sa il giocatore. Il docente la riformula: una **dissonanza tra quello che il personaggio sa e quello che dovrebbe sapere**, spesso perché chi lo gioca sa più cose del personaggio, e quella conoscenza passa al personaggio "dall'alto", senza che il personaggio la impari. Tiziana e Giordano confermano la stessa definizione.

Ha senso usare "metagioco" in altri ambiti? Uno studente ipotizza che, in un gioco da tavolo, possa indicare le **strategie**: ci sono le regole, e poi le strategie che si sviluppano attraverso quelle regole. "Esiste anche questa, esatto". È un **livello che non è presente nel gioco** ma nella sua **comunità**: strategie, azioni, sequenze di azioni vincenti. Chi le conosce ottiene un vantaggio rispetto a chi non le conosce. A pensarci, non è così diverso dal caso di *D&D*: anche se *D&D* non ha una vera condizione di vittoria, il meccanismo è lo stesso, si usa una **conoscenza esterna** per avere in gioco più possibilità di superare gli ostacoli.

Le metatecniche. Nel gioco di ruolo tradizionale (*D&D*, *Vampire: The Masquerade*, GURPS, *Pathfinder*) il metagioco è visto come un **problema**, una rottura del coinvolgimento narrativo: se si gioca per il massimo coinvolgimento, non si dovrebbero dare al personaggio informazioni che non ha. Ma la **scuola scandinava** si è chiesta: perché? Perché non fare giochi fortemente basati sul metagioco, e anzi **giocare con il metagioco** come punto di interesse ludico? Nasce così una serie di meccaniche chiamate **metatecniche**, che nei LARP di stampo scandinavo funzionano tendenzialmente così: **chi gioca fa qualcosa, meccanicamente, che il proprio personaggio non fa**, e questo ha un impatto significativo in gioco.

Ping the glass. L'esempio è "forse la più comune metatecnica di tutte". Un LARP su un party dell'alta società della Silicon Valley: si gira con il proprio cocktail, si parla con le persone. Ognuno ha una matita nel taschino, o qualcosa del genere, con cui può fare *ping* sul bicchiere della persona con cui sta parlando; quella persona ha allora **20–30 secondi per dire ad alta voce che cosa sta pensando il suo personaggio**. Il personaggio non sta toccando un bicchiere con una matita, il giocatore sì; il personaggio non sta dicendo ad alta voce quello che pensa, il giocatore sì. Ne nascono momenti molto interessanti: un investitore dice "interessantissimo, vedrò di investire nel tuo progetto"; poi qualcuno gli tocca il bicchiere e lui dice: "vedrò di fregarlo. Questo è uno schema Ponzi, ma ancora non lo sa". E poi si continua come se niente fosse, per i personaggi; ma non per i giocatori. Sono giochi che danno molto **non detto**, un po' come al cinema in *Dogville*, con i muri che non sono muri e attraverso cui si vedono i personaggi.

Uno studente chiede conferma: il personaggio continua a non sapere quella cosa? Sì, il personaggio no, ma il giocatore sì. **Perché** questo sia interessante in un gioco lo si vedrà nella lezione sulla progettazione delle emozioni; per ora basta sapere che le metatecniche esistono, "ce ne sono a tonnellate", e se ne vedranno altre.

Intrusivo non vuol dire negativo. Tornando al cursore: le metatecniche sono meccaniche **intrusive** perché **fermano il gioco**. E intrusivo "non è negativo", non è che in alto è sbagliato e in basso è giusto. Anche **tirare i dadi** durante un combattimento di *D&D* è una meccanica intrusiva: si ferma per un momento il gioco per applicare una meccanica e vederne la risoluzione. Le meccaniche **discrete**, invece, non fermano il gioco. L'esempio: in un LARP, un **adesivo colorato** sul petto[30] dice quanto contatto fisico si accetta: con il rosso non si deve toccare; con il giallo si può toccare, ma bisogna fermarsi appena l'altro lo chiede; con il verde si possono avere anche interazioni aggressive dirette, di combattimento. Quella meccanica è discreta: non si interrompe il gioco per dire "voglio fare un combattimento con te".

Gioco da tavolo e gioco pervasivo. Nel **gioco da tavolo** le meccaniche discrete sono molto difficili da fare, perché per sua natura il gioco da tavolo ha meccaniche molto intrusive. Nel **gioco pervasivo** è quasi il contrario: spesso si basa sull'idea che la realtà abbia un **livello alternativo** (*alternate reality game*) e che tutto sia in un gioco più o meno plausibile. Un enigma in città può essere osservare una statua e notare che ha tre fiori in mano, e quei tre fiori sono la chiave dell'enigma: niente ferma il gioco.

Il collegamento con il metagioco. Uno studente non ha capito il nesso con il metagioco inteso come strategie efficaci. Il docente chiarisce: l'esempio delle metatecniche serviva a far capire **l'intrusione**. Per meccanica intrusiva si intende qualcosa che **chi gioca fa** e che si mette **in mezzo alle cose che il suo personaggio sta facendo**. Se si interpreta il proprio personaggio senza interruzioni, non c'è intrusione; se il gioco chiede in certi momenti "tu, persona che giochi, devi fare questa cosa", in quel momento si interrompe l'essere il personaggio per una meccanica. E se la conoscenza esterna si mette in mezzo tra sé e il personaggio, anche quello, meccanicamente, può essere intrusivo. Di nuovo: "nessuno slider è meglio o peggio". Anzi, il docente consiglia i giochi con le metatecniche, "estremamente interessanti", che fanno cose che si vedono raramente: "vi auguro di poterne progettare uno domani, o all'interno del corso, chissà". E la maggior parte dei board game ha meccaniche intrusive: "proprio non ci sfuggite".

Daggerheart. **Nicola** propone un esempio dal gioco di ruolo: le meccaniche di **Paura e Speranza** di *Daggerheart*. Il master ha sempre accesso a una riserva di segnalini, visibile a tutti i giocatori, che può usare per peggiorare la situazione; i giocatori (non i personaggi) hanno a loro volta una riserva di Speranza che possono usare per attivare poteri speciali. È intrusiva, perché non si sta semplicemente facendo roleplay: c'è una meccanica in mezzo. Il docente non l'ha ancora giocato e non entra nel dettaglio, ma conferma: come il tiro di dado per sfondare una porta a calci.

Fiasco* e *Waiting for Flight. Due esempi dal lato discreto. *Fiasco*, citato da Daniele nella lezione sui casi studio, è interessante perché **nelle scene** è quasi tutto discreto: le scene sono un flusso di impostazione e risoluzione senza tirare un singolo dado, e il gioco dice solo chi ha l'autorità di decidere che cosa. Tra una scena e l'altra ci sono meccaniche intrusive, anche tante, ma dentro la scena tutto è "molto liscio e fluido". *Waiting for Flight* ha meccaniche intrusive solo alla fine, quando arrivano le valigie; nelle tre ore prima è tutto discreto: niente regole di combattimento, niente metatecniche, solo l'attesa. C'è la gestione attiva di chi facilita, che porta informazioni, ma quelle non sono meccaniche: non chiedono ai giocatori di fare qualcosa, danno informazioni **attraverso il filtro di un personaggio**, con cui si parla.

Approfondimento: il metagioco come dinamica. *Challenges for Game Designers* offre una terza definizione di metagioco, utile per collegare le due date a lezione. Nel capitolo sugli "atomi" del game design, Brathwaite e Schreiber notano che non tutte le dinamiche sono definite o imposte dalle meccaniche: alcune avvengono tra i giocatori **fuori dallo stato del gioco**, e sono "comunemente chiamate metagioco": trattative, discussioni, alleanze, chat, provocazioni. È un'accezione a metà strada tra le due della lezione: come il metagioco del gioco di ruolo, è qualcosa che passa *sopra* la finzione e le regole; come il metagioco delle strategie, è qualcosa che nasce dalla comunità dei giocatori e non dal regolamento. Le metatecniche nordiche fanno il passo successivo: prendono questo livello "esterno" e lo trasformano in una regola esplicita, progettata.

[30] La trascrizione attribuisce due volte il colore rosso a significati opposti; dal contesto, il terzo colore, quello che permette le interazioni più aggressive, è con ogni probabilità il verde.

2.3.12 La modularità e gli altri cursori

"Con gli slider mi fermo qui". Ma il docente ricorda la **modularità**: quali altri cursori si possono avere? Oltre al *your fader here* del Mixing Desk originale, la slide ne mostra altri che vengono da un **framework didattico-pedagogico**:

- la materia trattata è affrontata in modo **performativo**, comportamentale, o come **elaborazione interna** di chi gioca?
- il gioco è **competitivo** o **cooperativo**, o una via di mezzo?
- il gioco punta sull'**interazione sociale** con le altre persone (*social play*) o sul **gioco individuale** (*individual play*)?

Valgono anche per il gioco da tavolo. Nei giochi "alla tedesca" gira l'espressione **solitario multiplayer**: si gioca in quattro, competitivo, ma ognuno fa le proprie cose, nessuno può distruggere niente all'altro, e l'interazione è magari indiretta, su un mercato comune ("io compro una carta che volevi tu"), "molto più fredda rispetto a *ti attacco direttamente con i carri armati*". Sono giochi che puntano più sull'*individual play* che sul *social play*.

E se ne possono aggiungere o togliere in base al gioco. Il docente ne suggerisce altri, "anche banalmente":

- gioco **lungo** o **breve**;
- per **poche** o per **tante** persone;
- interazione **diretta** o **indiretta**;
- informazioni **segrete** o **pubbliche**: che è un'altra cosa rispetto all'apertura tematica. In *Hanabi*[31] il tema è trasparente, ma durante il gioco si hanno informazioni a cui non si ha accesso (le proprie carte).

La tabella 2.3.2 riassume tutti i cursori visti.

Lo si userà nella ricerca e sviluppo. "Questi slider torneranno": si parlerà di nuovo del Mixing Desk nella lezione su **ricerca e sviluppo**, perché applicarlo ai giochi pubblicati aiuta. Che cosa esce se ci si butta dentro *Dungeons & Dragons*? E *Non cedere al sonno*? E *Monopoly*, *Risiko*, *I coloni di Catan*, *Magic: The Gathering*? E facendolo si vede anche **quali cursori sono importanti** per il tipo di gioco che si sta facendo, e quali no.

2.3.13 Un'analisi dal vivo: Exit, Hidden Games e Profiler

"Questa era l'ultima slide di oggi". Uno studente chiede se si possano fare degli esempi insieme, provando a vedere questi aspetti su un gioco. Il docente ha sempre una mezz'ora per le domande alla fine ("adesso un po' meno, perché siamo stati lunghi"): ci sono una venticinquina di minuti. "Che gioco volete che vediamo passare in questo filtro?" Dopo un'esitazione ("non fatevi remore, raga, è utile"), uno studente propone una tipologia invece di un gioco: le **escape room**. Siccome dipende molto dal formato (giocare un'escape room da tavolo a casa è un'altra cosa), si sceglie ***Exit***, la serie di escape room **da tavolo**: ibridi tra escape room e board game, **non rigiocabili**, che durano tra i 50 minuti e l'ora e mezza a seconda dell'esperienza, e che chiedono di **modificare fisicamente i componenti**: strappare, bagnare, piegare. Strada facendo si aggiungono due termini di paragone: gli investigativi **Hidden Games**[32] e ***Profiler***, nel formato tascabile, che il docente mostra in video (tabella 2.3.3).

[31] La trascrizione dice "Anabi".

[32] La trascrizione li chiama "Hidden Games", "In The Games", "Indie Games", "ED Games" e "Den Gains". Il titolo del caso giocato, "Veleno verde", è riportato come nella trascrizione.

Cursore	**Estremi**	**La domanda e gli esempi della lezione**
Comunicazione	verbale – fisica	si parla o si spostano cose? Coppia perfetta, party game, *Dixit*, *Taboo*, *Codenames* contro boffer, scacchi, *Stranger*
Apertura tematica	trasparenza – segreto	il tema è noto da subito o viene rivelato? *Twilight Struggle* contro *Train*; trame segrete, investigativi, il tenente Colombo
Diegesi ambientale	360° – indipendenza materiale	serve l'ambiente vero? Castello e cavallo veri contro bagno di casa; chamber LARP in costume nello scantinato
Creazione dei personaggi	chi gioca – chi progetta	chi scrive il personaggio? *Blockbuster* e CRPG contro *Final Fantasy*; i paletti di *Non cedere al sonno*
Gestione del gioco	attiva – passiva	qualcuno spinge avanti il gioco? PNG che uccidono il re, campagne numerate, master, audio guida contro *Journey to the End of the Night*
Universo narrativo	giocabile – plausibile	momenti di gioco o simulazione del reale? La vedetta, GURPS, *GTA*, *Desert Bus*, *The Longing*, Aragorn, *Waiting for Flight*
Rappresentazione tematica	storia – azioni	il tema passa dalla narrazione o dalle meccaniche? Horror narrato contro meccanica di follia (nel corso: priorità alle azioni)
Meccaniche	intrusive – discrete	la meccanica interrompe il gioco? Metatecniche, *ping the glass*, dadi, *Daggerheart* contro adesivi colorati, scene di *Fiasco*, enigmi urbani
Your fader here	liberi	performativo – elaborazione interna, competitivo – cooperativo, social – individual play, durata, numero di giocatori, interazione, informazioni, single – multiplayer, *table presence*

Table 2.3.2: I cursori del Mixing Desk come presentati a lezione. Nessun estremo è migliore dell'altro; l'unica eccezione dichiarata, valida solo per i lavori del corso, è la priorità alle azioni nella rappresentazione tematica.

Comunicazione. *Exit* è più verbale o più fisica? "Dipende", dice uno studente, perché lo si gioca con altri. Si può giocare anche da soli, ma per come è inteso è un'esperienza **abbastanza verbale**: il gruppo condivide idee e cerca soluzioni. Non del tutto, però, perché i **componenti fisici** sono essenziali: senza non si risolve nulla. Anzi, quello che rende *Exit* più fisico di altre escape room da tavolo è che usa **la scatola stessa** come elemento di gioco: chiede di inserire cose nel coperchio, di guardare da una prospettiva, nasconde scritte dietro il codice a barre, "in posti che non ti aspetteresti".

Apertura tematica. Abbastanza **trasparente**, ma non del tutto: alcune scatole hanno un finale con qualche twist. La narrazione è leggera ("non stiamo parlando del *Signore degli Anelli*"): c'è un tesoro nascosto, bisogna trovarlo, chissà chi l'ha nascosto e perché. Trasparente nella maggior parte, con qualche aspetto segreto.

Diegesi ambientale. Più verso l'**indipendenza materiale**, ma non del tutto. È vero che non si è davvero sull'isola a cercare il tesoro; ma gli *Exit* fanno della **materialità** un aspetto chiave: lo spessore della scatola, quanto è alta e larga, che cosa c'è dentro, quanto è spessa una tessera, possono essere chiesti da un enigma. Se lo si stampa con la stampante di casa, ci sono *Exit* che non si possono finire, perché serve la scatola in mano, o serve vedere che cosa succede bagnando i componenti. Gli Hidden Games vanno oltre: in quello giocato dal docente (e da uno studente) c'era un **sacchetto di terra**, e bisognava fare il **test del suolo**: prendere un bicchiere in cucina, riempirlo d'acqua, metterci la terra, inserire la cartina tornasole e misurare il pH, perché un enigma lo chiede. Non si può giocare con un *print and play*. È "molto verso il 360°", anche se si è nel proprio salotto e non in un laboratorio della scientifica. Uno studente chiede conferma: direbbe davvero "molto verso il 360°"? Sì, o comunque a metà: "quella è terra, non è un'immagine del terreno con scritto pH 5". Sporca le mani, fa un macello sul tavolo, fa fango quando la si bagna, e rende l'esperienza molto intensa. Della materialità, del resto, si parlerà in una lezione apposta sul **paragame**.

Personaggi. In *Exit* o negli Hidden Games i personaggi "forse non esistono", ma esiste un **ruolo**: si è degli **investigatori**. Come ci si chiama, quanto si è alti, come si hanno i capelli, che cosa si indossa, "non interessa a nessuno"; il gioco non chiede di indossare il cappello di Sherlock Holmes. Ma si è investigatori, e il gioco lo dice molto bene. Una studentessa chiede se rientrino qui i **ruoli sociali** che si assumono in un'escape room (l'analitico, l'organizzatore): no, se ne parlerà nella lezione sui personaggi ("il classico esempio in cui vi rimando ad altre lezioni, ma ve lo prometto"). Quindi il designer ha deciso il personaggio? Sì, ma "decide quasi niente": in questi giochi il cursore pesa poco e **non è detto che serva**. Diverso è ***Alibi***[33]: un investigativo in cui si interpretano i personaggi della vicenda e il colpevole è uno di loro; lì il cursore conta, perché dice chi si è, che cosa sa il proprio personaggio, che cosa non vuole che si sappia. "Come nel LARP *A cena col nonno*", aggiunge uno studente: esatto. Che un cursore esista non vuol dire che pesi uguale in tutti i giochi.

Gestione del gioco. *Exit* e Hidden Games sono simili: gestione **abbastanza attiva**, "ma con un twist". Gli enigmi vanno risolti in ordine, e l'ordine è gestione attiva; ma il tempo speso su ciascuno lo decidono i giocatori, che possono correre come matti o metterci otto ore. All'estremo opposto c'è ***Profiler***, che il docente mostra: un pacchettino con un mazzo di carte **non numerate**, senza ordine, e senza nemmeno un regolamento, perché è chiarissimo come si gioca. Una carta presenta il caso; ogni altra carta è un'informazione: una fotografia, una mappa, un documento, una brochure, un foglio Excel, una lista di cose da fare, il contenuto di una valigetta. Il gioco dice solo di spargere tutte le carte sul tavolo e di rispondere ad alcune domande. Tutto è trasparente, si ha accesso a tutte le informazioni contemporaneamente, senza punizioni e senza tempo. "Sembra facile, non è così facile". È l'equivalente di un *open world*: si va dove si vuole tra i documenti, e sta a chi gioca unire i fili. Gestione **passiva**, a parte la prima carta.

Una studentessa chiede di *Sherlock Holmes Consulente Investigativo*. "Decidi tu": è molto **attivo**, perché è più vicino al libro-gioco: per risolvere il caso bisogna toccare i punti che i designer vogliono. Ma è passivo nel fatto che si può andare dove si vuole, il che può voler dire perdersi andando a caso, o farsi ingannare da una falsa pista (*red herring*) e buttare tempo.

[33] La trascrizione dice "Alibi di Genos da Vinci"; l'editore non è ricostruibile con certezza.

Universo narrativo. Qui *Exit* e Hidden Games vanno in **direzioni opposte**. Gli *Exit* sono **molto giocabili** e poco plausibili: enigmi "giocattolosi", come trovarsi in una baita abbandonata e dover risolvere un cubo di Rubik per aprire una porta. "Chi è il pazzo che mette un cubo di Rubik come serratura?" Gli Hidden Games puntano sulla **plausibilità** del caso, come mostra la terra: nell'investigazione, se ci si vuole sentire veri investigatori, risolvere un enigma plausibile, coeso e coerente dà più soddisfazione. Per questo non sono ambientati nel fantasy ma nel **mondo reale**, con cellulari, computer e documenti, e un world building semplice: "basta guardare il mondo fuori dalla finestra".

Rappresentazione tematica. Uno studente metterebbe gli Hidden Games **a metà**, ed è giusto: c'è la storia (l'omicidio, il caso, che arrivano tramite testimonianze, documenti, personaggi che parlano), ma per far avanzare il gioco servono le azioni. Gli Hidden Games chiedono di **chiamare un numero di telefono**, di mandare una mail e riceverne la risposta, di andare su **siti fittizi** creati per le aziende del caso per capire che cosa vendevano. Ma soprattutto: non si è investigatori seduti in poltrona a chiedersi come sia morto qualcuno, si è investigatori che "si fanno un mazzo così" per unire le prove, cercare chi ha un'informazione, riascoltare un audio trenta volte per capire se una frase nasconde qualcosa, rileggere tre volte un foglio Excel per trovare il numero sfuggito. Quelle sono **azioni**: "siamo investigatori perché facciamo gli investigatori, non perché il gioco ci dice che facciamo gli investigatori".

Meccaniche. *Exit* è "totalmente intrusivo", "più intrusivo di così difficile". *Profiler* è "ultra discreto": l'unica cosa intrusiva è leggere la prima carta. Anzi, dopo un *Exit* si resta straniati, perché ci si aspetta di essere guidati passo passo, e invece si ha tutto subito: si spargono i documenti sul tavolo, si collegano cose "come nei meme dei pazzi che uniscono i fili al muro". Il gioco non ferma mai, se non quando il gruppo crede di avere la soluzione e va a verificare.

I cursori che mancano. "È chiaro che ci mancano degli slider, qua". Per esempio:

- **single player – multiplayer**: *Profiler* si può giocare da soli, ma è palese che non è inteso così ("cinquanta carte sparse sul tavolo da solo, ti ammazzi"), mentre gli *Exit* potenzialmente sì;

- la **materialità**, e in particolare uno dei cursori che il docente usa per i board game: la ***table presence***, quanto spazio prende il gioco sul tavolo, da *Twilight Imperium* a *UNO*, con tutte le vie di mezzo. Un *Exit* si gioca su un tavolino da caffè; *Profiler*, con le sue cinquanta carte, va sul tavolo della cucina, per avere la visione d'insieme. E *MicroMacro* ha una mappa della città gigantesca, praticamente un A2: la si appende al muro e si gioca in piedi davanti al poster, "perché sul tavolo non ci sta". È parte dell'esperienza: si indaga su un foglio appeso al muro, "ti dà anche delle vibes di un certo tipo".

2.3.14 Perché proprio questi due framework

Il docente chiude la parte teorica con una dichiarazione di metodo, "con trasparenza": **non è un grande amante dei framework teorico-accademici fini a sé stessi**. Li legge, legge i paper, lo intrigano, e il lato accademico del gioco e dei *game studies* è "interessantissimo" per lui. Ma un corso per persone che vogliono fare game design, e non necessariamente game studies, deve insegnare a **saper fare i giochi**; e saper fare i giochi vuol dire anche usare strumenti che rendono la vita più facile. Il **Player Involvement Model** di Calleja e il **Mixing Desk** sono, per lui, "due strumenti utilissimi".

Per chi è interessato, sul sito **Analog Game Studies** c'è un articolo sulla **storia del Mixing Desk**, su come è nato e come è stato sviluppato: molto dettagliato, "il mio articolo preferito di tutto il sito". Il docente lo carica sul canale Discord del corso (i link in chat, come si è visto, non funzionano). È "lunghetto" e ripercorre in gran parte cose già viste, quindi non è obbligatorio: "quando qualcosa vi verrà richiesto, lo saprete. E di solito sono i task".

Cursore	***Exit***	**Hidden Games**	***Profiler***
Comunicazione	abbastanza verbale, ma la scatola stessa è un enigma	verbale e manipolativa	molto verbale (gruppo)
Apertura tematica	quasi trasparente, qualche twist	segreto investigativo	caso dichiarato dalla prima carta
Diegesi ambientale	verso l'indipendenza, ma serve la scatola vera	verso il 360°: terra vera, pH, telefonate, siti	carte sul tavolo
Personaggi	un ruolo generico (si è "investigatori")	idem; il cursore pesa poco	idem
Gestione	attiva nell'ordine degli enigmi, libera nei tempi	come *Exit*	passiva, tranne la prima carta
Universo narrativo	molto giocabile, poco plausibile (il cubo di Rubik)	molto plausibile, mondo reale	plausibile
Rappresentazione tematica	tema leggero	a metà, ma "siamo investigatori perché lo facciamo"	azioni di collegamento
Meccaniche	totalmente intrusive	intrusive	ultra discrete
Table presence	tavolino da caffè	variabile	tavolo della cucina

Table 2.3.3: L'analisi dal vivo fatta in classe, ricostruita. Le celle di Hidden Games e *Profiler* per i cursori non discussi esplicitamente sono deduzioni da quanto detto a lezione.

2.3.15 Le domande finali

Restano sette minuti per le domande, "pochi, molto pochi, ma meglio che niente". "Non censuratevi in autonomia".

Il venerdì e i tutoraggi. Una studentessa torna su una cosa che non aveva capito: gli orari "del venerdì". In realtà intendeva i **tutoraggi** del sabato di cui si era parlato all'inizio. Il docente chiarisce: la lezione di **venerdì 18** è dalle 17 alle 20, come le altre, e sarà quella sui **personaggi**; poi arriverà quella sull'*emotional design*, "finalmente"[34]. I **tutoraggi** invece arriveranno più avanti, quando si lavorerà già ai giochi, e **non sono lezioni**: niente slide, sono momenti di **confronto diretto** con il docente sui progetti, per chiedere aiuto, una mano, "un attimo magari rimettervi sulla strada per non perdervi col progetto di gruppo". La studentessa aveva capito tutt'altro, e aveva aspettato la fine per chiedere. "Regà, ma io cosa vi dico? Dovete farmi queste domande".

Il silenzio e il *cringe*. Mancano cinque minuti. "Se state in silenzio, io divento super *cringe*, mi metto a straparlare, e nessuno vuole questa scena". Quindi: "fate le domande e preservate il momento dalla contaminazione del *cringe*".

I cursori cambiano con chi gioca? Uno studente fa "un piccolo salto logico", ripartendo da quanto aveva scritto Lorenzo: il designer pensa a un'esperienza, ma il giocatore la vive in modo diverso. Allora anche i cursori possono cambiare a seconda di come si gioca. L'esempio: per lui *Exit* è meno verbale di come lo si è descritto. Il docente precisa che "molto verbale" era relativo: più verbale di un *Agricola* o di un *Catan*, meno di un Hidden Games o di *Profiler*. Lo studente spiega: quando gioca con la sua compagna, sanno perfettamente chi dei due sa fare bene certe cose, e parlano meno, perché la loro esperienza di gioco è cambiata. La domanda: possiamo dire che un gioco è tanto più ben fatto quanto più **la media dei cursori dei giocatori corrisponde ai cursori del designer**?

[34] Il passaggio della trascrizione è confuso ("Il tutoraggio nella lezione, venerdì 18, è dalle 17 alle 20. Fra l'altro, se lo diano con me, farete la lezione sui personaggi. Faremo quella sull'emotion design"). La lettura proposta è coerente con il resto della lezione, in cui il docente rimanda più volte alla lezione sui personaggi e dice che quella sull'emotional design arriverà "non nella prossima, ma in quella dopo".

Un buon design fa arrivare le intenzioni. "Se per ben progettato intendiamo un gioco che fa arrivare l'esperienza intesa nella maggior parte dei casi, sì". Il che non vuol dire che sia una *bella* esperienza per chi la riceve: lo si vedrà nella lezione sull'emotional design, con molti esempi. Ma in generale **fare un buon design è assicurarsi che le proprie intenzioni arrivino**. L'esempio: si progetta un gioco per una certa esperienza, e chi lo gioca ha un'esperienza incredibile, fantastica, ma **ha capito male le regole** e le ha sbagliate tutte. "Come è andato questo gioco? Che cosa vi fa provare questo risultato?" E non basta che l'esperienza sia fantastica: deve essere in linea con i propri principi. Magari l'esperienza è fantastica e le persone dicono: "perfetto, ho proprio capito che adesso devo andare per strada e picchiare la gente che incontro, così starò meglio". Esperienza fantastica, messaggio che non si voleva far arrivare.

2.3.15.1 La responsabilità di chi progetta

Di queste cose, dal punto di vista del docente, **il designer ha la responsabilità**. E cita una frase che gli disse il suo professore di *foundations of playing games* al master, "molto forte", che non farà piacere sentire:

> **i game designer fanno *brainwashing*.**

Quando si fa game design si sta plasmando la mente delle persone: le si convince a fare cose che normalmente non farebbero. "Nessuno, di propria volontà, si mette a muovere pedine su un tavolo senza che ci sia una ragione dietro". Il designer dà alle persone **ragioni** per fare cose che non farebbero: degli **alibi**, delle motivazioni, delle spinte emotive. E questo porta con sé una responsabilità: "io non voglio che qualcuno si faccia male giocando il mio gioco".

- È tanto più importante nel **gioco pervasivo**, che ha a che fare con città, strade e persone che non sanno che si sta giocando: richiede "una delicatezza e una responsabilità forte e alta". Se si organizza un *Journey to the End of the Night*, non si vuole che un giocatore venga investito da un autobus.

- In un board game il rischio fisico è più basso, "ma non è necessariamente più basso per davvero": c'è il **rischio mentale, psicologico**, che si vedrà nell'emotional design. "Potete fare dei danni alle persone che non avete idea".

"Non ve lo dico per spaventarvi o per disincentivarvi da fare questo lavoro", ma per essere persone più consapevoli della responsabilità che si ha quando si fanno delle cose. Se ne parlerà più nel dettaglio. "Spero di non avervi spaventato".

Approfondimento: la lente della responsabilità. Schell chiude *The Art of Game Design* con un capitolo intitolato "i designer hanno certe responsabilità". L'immagine è quella dell'anello di ferro che, dal 1922, gli ingegneri canadesi ricevono alla laurea, al mignolo della mano dominante, in un rituale ideato da Rudyard Kipling per ricordare il loro obbligo verso la società. Schell ne offre uno, invisibile, ai lettori: "se credete davvero che i giochi possano aiutare le persone", mettetelo, e lasciate che quelle responsabilità guidino la vostra mano; "pensateci bene, però, perché non si toglie". La lente corrispondente (la *Lens of Responsibility*) si riduce a una domanda: **il mio gioco aiuta le persone? Come?** È il rovescio positivo dell'avvertimento del docente: se i giochi possono plasmare le menti abbastanza da fare danni, possono farlo anche abbastanza da fare del bene, ed è il designer a scegliere in che direzione.

2.3.15.2 La chiusura

Il docente ringrazia e congeda la classe: "potete andare, giocate". E ricorda di giocare anche su **Tabletop Simulator**, perché il corso avrà una licenza, e vorrebbe che gli studenti sapessero già muoversi nell'ambiente: così sarà più facile spiegare come si creano giochi lì dentro. Per le domande, le email dei docenti le hanno. Lui cercherà di capire come caricare le slide nelle cartelle degli studenti, "che di solito lo fa Daniele". "Potete andare, siete persone libere e autonome". "Ciao a tutti, buona serata".

Punti chiave del capitolo

- I **framework** raccolgono soluzioni a problemi che altri hanno già incontrato: servono a perdere meno tempo, a progettare in modo più focalizzato e a far arrivare l'esperienza voluta. Sono anche strumenti di **analisi** dei giochi esistenti.
- **Il design è l'opposto della tradizione** (Koljonen): chiedersi che cosa si mette nel gioco e perché, distinguendo ciò che ha senso da ciò che c'è solo perché "il genere l'ha sempre avuto" (master, dadi, schede nel GDR).
- **Il gioco è un'esperienza creata da regole** (Anna Anthropy): sulle regole si ha controllo, sull'esperienza no. Il lavoro del designer è creare regole che creino esperienze. Il gioco per tutti non esiste; l'accessibilità riguarda tutte le persone *per cui* il gioco è.
- Le **lenti** di Schell sono "cappelli" per guardare il design; utili da conoscere, non tutte utili. I giochi sono un **medium**: comunicazione bidirezionale, differita, **mediata** (regolamento, procedure, workshop) ed **emergente**.
- La pipeline **design, playtest, iterazione** non si salta, nemmeno per giochi non commerciali. Il playtest è una spia; l'iterazione può voler dire riscrivere o tagliare; la perfezione ha un costo infinito. Dal **feedback** si prende il sentimento, non la soluzione.
- **MDA**: si controllano le meccaniche, non si impongono le dinamiche. C'è un gap tra estetica promessa ed estetica arrivata, e il costruttivismo personale rende ogni esperienza diversa. Imparare a riconoscere giochi progettati bene che non piacciono e progettati male che piacciono. Il **DPE** (Winn) ridistribuisce i termini in design, play, experience su più livelli, compreso l'apprendimento.
- **"Immersione" non dà strumenti**: scacchi, sport, tornei e LARP in castello sono "immersivi" in sensi diversi (flow, sospensione dell'incredulità...). Il **Player Involvement Model** di Calleja la sostituisce con sei forme di coinvolgimento, non esclusive e non booleane: **cinestetico, spaziale, condiviso, narrativo, affettivo, ludico**. Non è qualitativo: dice su che cosa il gioco spinge.
- Affettivo e narrativo non coincidono (gli scacchi coinvolgono affettivamente; i Jeepform quasi senza storia). Il condiviso include entità non umane. Il multiplayer ha sempre coinvolgimento condiviso: se non lo si progetta, lo progettano i giocatori.
- Il **Mixing Desk of Larp**, nato nel Nordic LARP, descrive un gioco con cursori: comunicazione (verbale/fisica), apertura tematica (trasparenza/segreto), diegesi ambientale (360°/indipendenza materiale), creazione dei personaggi (chi gioca/chi progetta), gestione (attiva/ passiva), universo narrativo (giocabile/plausibile), rappresentazione tematica (storia/azioni), meccaniche (intrusive/discrete), più *your fader here*. È modulare e serve sia ad analizzare sia a scrivere linee guida.
- **Giocabile e plausibile** sono in opposizione: plausibile è un mondo che simula la realtà, giocabile un mondo progettato per far nascere momenti di gioco (la vedetta, GURPS, *Desert Bus*, *Waiting for Flight*). Nessuno dei due è migliore.
- Nel corso, la **rappresentazione tematica** è l'unico cursore trattato come qualitativo: il tema deve arrivare **con le azioni**, con le meccaniche, non solo con la storia.
- Le **metatecniche** (come *ping the glass*) fanno fare al giocatore ciò che il personaggio non fa; sono meccaniche **intrusive**, come i dadi. Intrusivo non vuol dire negativo.
- Analisi dal vivo: *Exit* (giocabile, intrusivo, la scatola come enigma), Hidden Games (plausibili, materiali, azioni da investigatore), *Profiler* (passivo, ultra discreto). Cursori aggiuntivi: single/ multiplayer, *table presence*.
- Un buon design fa arrivare **le intenzioni**; e chi progetta giochi "fa *brainwashing*": dà alle persone ragioni per fare cose che non farebbero, e ne ha la **responsabilità**, fisica e psicologica.

Chapter 2.4

Personaggi e ruoli: progettare personaggi giocabili

Questa è la **quarta lezione** del corso di **Analog Game Design**, ed è dedicata a **personaggi e ruoli**: "chiaramente, nell'ottica non di un romanzo narrativo, ma nell'ottica di game design, cioè di creare giochi". Dopo i framework della lezione precedente, il docente avverte che anche questa è "un po' intensa", ma meno, e soprattutto **più interattiva**: buona parte della lezione è fatta di domande rivolte alla classe ("e voglio le vostre risposte"), di schede personaggio proiettate a schermo da commentare, di immagini di ambienti da "leggere". Ed è la lezione in cui arriva il **primo task** del corso: progettare una scheda personaggio per un LARP fittizio, intrecciandola con quelle di tutti i compagni.

La lezione si svolge **online**, su Zoom, ed è tenuta ancora una volta da **Ruggero**[1]: "adesso avete un po' di lezioni con me di fila, quindi vedrete solo la mia faccia". La classe è di **nove persone**; quelle che intervengono, e che il capitolo nomina, sono **Gianmarco**, **Tancredi**, **Tiziana**, **Rita**, **Angelo**, **Lorenzo** e **Nicola**, che arriva a lezione iniziata. Rita deve andarsene prima della fine; "nessun problema, la lezione è registrata".

Quanto alla data, la lezione si colloca con ogni probabilità **venerdì 18 settembre 2026**[2], dalle 17 alle 20 come sempre, con una pausa poco dopo le 18 e un'altra poco dopo le 19. La lezione successiva, quella sull'*emotional design*, è annunciata per lunedì 21.

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi, soprattutto quelli dei giochi e quelli stranieri. Il personaggio d'esempio della lezione, la detective **Joey Doe**, diventa di volta in volta "Joy Do", "Gioido", "Zoidò" e "Joey Doh"; *Root* diventa "Route", *Dungeons & Dragons* "Dungeon Dragon", *world building* "war building", LARP spesso "l'ARP", le metatecniche "metotecniche". Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante le due pause la trascrizione registra per decine di righe frasi senza senso ripetute in loop ("qual è la risposta che ci può essere?", "la capacità di impostare la loro dottrina sul mercato del lavoro"), che il capitolo ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro (il docente li chiama per nome) e resta generico altrimenti.

Come leggere questo capitolo. Come la precedente, questa è una lezione di **concetti**, ma costruita quasi tutta su **esempi**: la stessa detective presentata con sette schede diverse, le sue tre cerchie di relazioni, le sue spinte, gli ambienti in cui la si può mettere. Il capitolo segue gli esempi uno per uno, perché è lì che i concetti prendono forma, e riporta le risposte degli studenti insieme ai commenti del docente. Il filo

[1] Anche questa volta il nome non viene mai detto dal docente stesso, ma lo pronunciano gli studenti (la trascrizione dice "Ruggia" e "Rugger") e lo usa lui, parlando di sé in terza persona, durante le domande sul task: "non dovete pensare a cosa piace a Ruggero". È anche il docente che lavora per l'editore di giochi da tavolo di cui si parla nelle domande finali (sezione 2.4.12).

[2] La data non è mai detta per intero, ma è ricostruibile con buona sicurezza: alla fine della lezione precedente il docente aveva annunciato che "venerdì 18" sarebbe stata la lezione sui personaggi; qui il task viene dato "per il 21 settembre, tra tre giorni"; il docente scherza sull'"effetto venerdì pomeriggio" e congeda la classe con "buon weekend" e "ci vediamo lunedì".

conduttore è un'idea che il docente ripete in molte forme: **un personaggio non è giocabile solo perché qualcuno può interpretarlo**; è giocabile se chi lo riceve sa *come* giocarlo, *con chi* giocarlo, *per che cosa* e *facendo che cosa*. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell, che dedica ai personaggi un capitolo intero); sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa prima dell'inizio: i componenti di *Nemesis* e di *The Gallerist*, la coesione tra tema e meccaniche, i giochi astratti (sezione 2.4.1);
2. i tre concetti di base: **player**, **character** e **role** (sezione 2.4.2);
3. che cosa rende **giocabile** un personaggio, e i **quattro pilastri** (sezione 2.4.3);
4. la **chiarezza**, con le sette schede di Joey Doe (sezione 2.4.4);
5. la **connettività** e le tre cerchie (sezione 2.4.5);
6. le **spinte** e le **attività** (sezione 2.4.6);
7. chi decide chi siamo: i tre slider di **estetica**, **fiction** e **ruolo** (sezione 2.4.7);
8. le altre figure di un gioco: **spettatori**, **componenti**, **NPC** e **arbitri** (sezione 2.4.8);
9. lo **spazio** dei personaggi come area progettabile (sezione 2.4.9);
10. il **primo task**: la scheda personaggio per il gala urban fantasy, e le molte domande che ha sollevato (sezioni 2.4.10 e 2.4.11);
11. le **domande finali**: framework e playtest, il catalogo di un editore, le ristampe (sezione 2.4.12).

2.4.1 L'attesa: componenti, tema e giochi astratti

Come al solito, nei primi dieci minuti si aspettano le persone, e il docente apre la parola: "se qualcuno ha domande, dubbi, o vuol dire qualcosa, questo è un buon momento". Si comincia a segnare le presenze. Uno studente è collegato dalla macchina e non ha la telecamera ("tanto sono in macchina, c'era poco da dire").

Il metagioco, rinviato. Tancredi vorrebbe riprendere il discorso sul **metagioco** della lezione precedente. Il docente gli chiede di aspettare: sulla questione delle **metatecniche**[3] in particolare si tornerà nella lezione sul **progettare emozioni**, dove se ne vedranno altre e sarà chiaro "che cosa fanno nello specifico da un punto di vista di game design, non solo teorico". "Non perché non voglio risponderti, ma perché lì vedremo altre cose sulla questione".

Il caldo di settembre. Arriva Tiziana, e con lei qualche chiacchiera sul caldo ("ma come fa a essere ancora settembre? Sembra luglio"): c'è chi si collega da Torino, dove qualcuno ha vissuto sette anni e ricorda un primo inverno gelido, e si scherza sul ciclone delle Azzorre di questi giorni[4].

2.4.1.1 Componenti e aspettative: *Nemesis* e *The Gallerist*

La parola passa a **Gianmarco**, che sta facendo un ragionamento su tre giochi con una componentistica "abbastanza significativa e importante": *Nemesis*, *Praga Caput Regni* e *Lisboa*[5], e poi *The Gallerist*. L'altro giorno ha giocato a *Nemesis: Retaliation* con un amico che aveva preso il *pledge* completo del crowdfunding,

[3] La trascrizione dice "metotecniche" e "metratecniche".

[4] Il passaggio della trascrizione è molto confuso e non permette di attribuire con certezza le battute.

[5] *Praga Caput Regni* è di Vladimír Suchý; *Lisboa* e *The Gallerist* sono di Vital Lacerda ("la cerda" nella trascrizione, che storpia il secondo titolo in "Galleris", "De Galles", "The Goddess" e "The Galadies"). *Nemesis* e *Nemesis: Retaliation* sono pubblicati da Awaken Realms.

"con tutti i materiali più belli del mondo", ed era davvero coinvolgente: quel materiale gli dava la sensazione di *vederla*, la nave, la stazione spaziale. Quando invece gioca a *The Gallerist*, "che è un German purissimo"... e qui la connessione di Gianmarco salta, lasciando la frase in sospeso ("mi fa il cliffhanger"), finché non passa all'hotspot del telefono.

Nell'attesa il docente si entusiasma: *The Gallerist* è "uno dei miei giochi preferiti in assoluto", "forse il mio preferito di quel designer". È un gioco "sul come il capitalismo ha distrutto il mondo dell'arte": si gestiscono delle gallerie d'arte, bilanciando la scoperta di nuovi talenti con lo sfruttamento del loro valore commerciale, per arricchirsi vendendo i loro quadri e acquisire nuove opere. "Gioco devastante a livello tematico, però bellissimo da giocare".

Gianmarco riprende: in *The Gallerist* trova che la componentistica non sia dissonante rispetto al gioco. Il docente spiega perché i due esempi funzionano entrambi: *Nemesis* e *The Gallerist* appartengono a due **macrocategorie**, i giochi **American** e gli **Eurogame**, che hanno alle spalle "un carico storico di produzioni diverse". Al punto che *The Gallerist* è coeso con se stesso pur essendo fatto "solo di cartone, carta e legno", e *Nemesis* è coeso con se stesso pur essendo "plastica a manetta". La ragione sono le **aspettative** di chi gioca:

- *Nemesis* promette un gioco di miniature con combattimenti contro gli alieni, un clima oppressivo e adrenalinico su un'astronave, in cui la strategia e la tattica esistono, ma sono secondarie all'**entropia** della situazione;

- *The Gallerist* è un gioco molto più gestionale, "con la visione dall'alto", e in quel genere di giochi il **caso** è trattato, e vissuto, in modo completamente diverso.

Un gioco *overdesigned*. Gianmarco ha però un controesempio: *Praga Caput Regni*, un gioco che trova "molto carino, molto interessante", ma, per usare un termine forte, **overdesigned**: "tutto quel materiale io non sento che mi serva". Il docente risponde prima con una battuta sul "vaso di Pandora" ("vuoi aprire i vasi di Pandora?"), e sul fatto che "c'è un sacco di gente che vuole fare il Lacerda, ma non sa fare il Lacerda". Poi spiega la differenza: *The Gallerist* è un gioco intenso, pesante, pieno di componenti, ma **i componenti che ha vengono tutti usati tematicamente**: "è difficile trovare una meccanica o un componente in *The Gallerist* che sia fuori tema". Molti Eurogame, invece, mettono dei **minigiochi nei minigiochi** per dare una sfida intellettuale, magari interessante, che però a livello tematico ha "molto poco aggancio". Gianmarco riformula: il componente non è messo lì in sé, ma ha una funzione all'interno del gioco.

Il tema intercambiabile. Il docente aggiunge un'osservazione storica, che ricorda di aver già fatto con Tiziana la volta precedente: molti Eurogame hanno un tema che potrebbe essere Praga, "ma potrebbe essere Dublino per un altro publisher", e il gioco sarebbe identico. Molti German "ragionavano le pubblicazioni in questo modo": si prende un gioco con le sue meccaniche e "ci sbattiamo sopra una città del Mediterraneo, che sia Ostia o che sia Genova, chi se ne frega". Con gli anni questa abitudine si sta un po' perdendo; resta frequente soprattutto nei giochi che vengono dal centro Europa, "però storicamente dobbiamo fare i conti con questo passato".

Approfondimento: la lente dell'unificazione. È la stessa intuizione che Jesse Schell formalizza in *The Art of Game Design* con la *Lens of Unification*: qual è il mio tema, e sto usando ogni mezzo possibile per rafforzarlo? Per Schell il tema può passare da una riga di testo, da una scelta di colore, da un effetto sonoro, e "una volta presa l'abitudine di far entrare nel tema più cose possibile, è difficile smettere". La lente si usa insieme alla tetrade elementare (meccaniche, storia, estetica, tecnologia), che separa gli elementi del gioco per guardarli ciascuno alla luce del tema. Il componente di *The Gallerist* che "non è mai fuori tema" e la città intercambiabile dell'Eurogame sono i due estremi di questa domanda.

2.4.1.2 Tabletop Simulator e un canale per la classe

Una domanda pratica, "approfittando della presenza dei compagni di corso": a che cosa state giocando su **Tabletop Simulator**, così mi scarico lo stesso gioco e magari giochiamo insieme? Uno studente ha appena

fatto una partita a *Le cronache di Avel*[6], già provato il giorno prima in associazione, perché dicevano che non si riusciva a vincere: "ho capito forse un po' di cose".

Il docente racconta come si sono organizzate le classi precedenti: alcune aprivano un loro gruppo **Telegram**, altre usavano il **Discord** dell'ente per interagire tra loro. Il consiglio è di trovare un modo per parlarsi, in modo che sia facile organizzarsi per giocare una sera su Tabletop, in vista di una lezione o anche solo per giocare. Il Discord, nota, qualcuno lo sta già usando: Tancredi ci ha scritto per chiedere le **slide**, che il docente ha caricato proprio oggi. C'era stato un equivoco tra i docenti ("io pensavo che le mettessi tu, lui pensava che le mettessi io"): "non succederà più. Ve le metto io le slide delle prossime lezioni".

2.4.1.3 Meccaniche perfette, tema assente

L'ultima domanda prima di cominciare è ancora di Gianmarco, "più nel ciarlare": recuperando la registrazione della lezione precedente, lo aveva colpito che al docente capiti di **revisionare giochi** che funzionano molto bene meccanicamente, ma che, quando l'autore spiega l'intento, risultano completamente slegati dall'emozione o dal tema che aveva in mente.

"Non è solo questo". Per il docente rientra nell'idea che il game design "può essere tante cose per tante persone". Per lui uno degli aspetti più importanti è la **coesione tematico-meccanica**: le meccaniche devono lavorare in accordo con il tema che si vuole passare, con le emozioni che si vogliono far provare. Ma non tutte le persone che fanno design si approcciano al game design in questo modo, e ci sono generi in cui la coesione è molto difficile da sostenere:

- la maggior parte dei **giochi astratti**, e dei designer moderni di giochi astratti: sono giochi in cui è difficile partire da un lato tematico;
- all'estremo opposto, nel **gioco di ruolo** gli aspetti di *world building* sono spesso preponderanti, più ancora che in un board game; lo stesso vale per gli **American Freeform**, "un po' board game, un po' role playing game", o per i **LARP pervasivi**, che hanno un layer di personaggi e di world building che un gioco pervasivo come *Pac-Manhattan*[7] non ha.

È una valutazione da fare in base al tipo di gioco che si vuole creare.

Nikoli e lo Slitherlink. Gianmarco precisa: gli capita di vedere un gioco meccanicamente perfetto che però "non funziona"? Il docente risponde con un esempio personale. È un grande amante dei giochi di **Nikoli**, lo studio giapponese che da decenni crea puzzle logico-matematici e il cui gioco più famoso è il **Sudoku**. Alcuni sono tra i suoi preferiti, come lo **Slitherlink**[8], che gioca e adora da anni. Ma è un puzzle logico-matematico: "non è che abbia chissà quale tema che vuole comunicare". E si può apprezzare comunque: **non bisogna apprezzare solo giochi con un fortissimo tema**, un fortissimo setting.

Bisogna vedere qual è l'**intento** del gioco:

- se un gioco si presenta come **narrativo** e poi la parte di world building, personaggi e storia è scadente, quello è per il docente un esempio **fallimentare**;
- se un gioco si presenta come **astratto**, ma nonostante questo riesce a comunicare un tema anche forte, è "un grosso vantaggio".

"Alla fine della lezione ci sarà sempre la mia mezz'ora di domande", e lì si potrà approfondire. Il docente lascia aperto il registro delle presenze e chiede, come sempre, di avvisarlo se qualcuno entra mentre lui guarda le slide. Condivide lo schermo, e si comincia.

[6] La trascrizione dice "le cronache di Havel": è con ogni probabilità *Le Cronache di Avel* (*The Chronicles of Avel*), cooperativo per famiglie.

[7] "Pac-Man Hatton" nella trascrizione.

[8] "Slider Link" nella trascrizione.

2.4.2 Player, character e role

La lezione parte da **tre concetti** che "ci serve sapere, ci serve capire soprattutto": **player**, **character** e **role**. Il docente li sente spesso discutere "quasi come se fossero la stessa cosa", ma non lo sono, ed è importantissimo capirne la differenza in termini di game design, "proprio di che cosa stiamo progettando".

Player. È la **persona giocante**. Tranne in alcuni generi (giochi di realtà alternata, giochi pervasivi), la persona giocante **non esiste nel world building del gioco**, non fa parte della narrazione: è una persona esterna che si interfaccia con il gioco come **artefatto**.

Character. È il **personaggio**: l'identità specifica di chi è questa persona dentro il gioco, con il suo background, il suo passato, la sua storia, un nome, all'interno di un **universo narrativo**. Il personaggio ci dice *chi è*, e lo lega alla narrazione, al world building, al plot, alla storia.

Role. È il **ruolo**: "un insieme di doveri, autorità e possibilità meccaniche". Il ruolo ci dice *che cosa può fare* questa persona, e che cosa è **legittimata** a fare.

L'esempio è **Carlo Magno**. Carlo Magno era un re, ma aveva un suo background, un passato, una storia, un nome. *Carlo Magno personaggio* e *Carlo Magno re* sono due cose diverse, perché la parte di re è una parte di ruolo. Ruolo e personaggio vengono spesso sovrapposti, "ma sono due cose che a noi game designer serve ed è utile separare".

Il ruolo c'è quasi sempre. I giochi di ruolo storici mostrano bene la distinzione: in *Dungeons & Dragons*[9] abbiamo un personaggio, ma quel personaggio ha un ruolo specifico, che nel caso di D&D è la **classe**, con tutta una serie di responsabilità e possibilità meccaniche. Ma vale anche per gli altri giochi, e gli esempi del docente scendono via via verso l'astratto:

- in *The Gallerist* non abbiamo un personaggio, ma abbiamo un ruolo: siamo una persona che gestisce una galleria;
- negli **scacchi** non abbiamo un personaggio, ma il ruolo ce l'abbiamo: siamo una fazione in battaglia con un'altra;
- in *Pong* "siamo una racchetta": qualcuno che deve impedire che la pallina arrivi dal suo lato del campo.

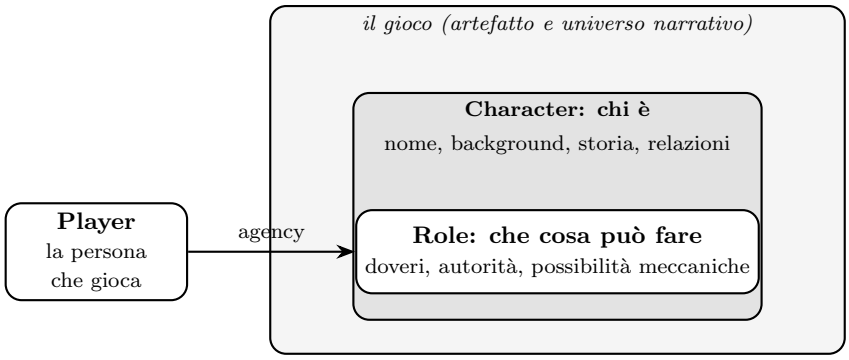
"In praticamente tutti i giochi abbiamo un ruolo". Poi il ruolo può essere accompagnato da un personaggio, e spesso le due cose vanno molto bene insieme. Il peso relativo dipende dal genere: nei giochi pesantemente **narrativi** il personaggio è di solito più curato, si spende di più, sia in risorse sia in peso all'interno del gioco; nei giochi più **astratti** abbiamo praticamente solo un ruolo, "sottilissimo".

All'estremo opposto ci sono giochi molto legati meccanicamente al ruolo. L'esempio è *Root*[10], un gioco da tavolo a **fazioni asimmetriche**: ogni fazione ha le sue meccaniche, e come ruoli sono molto diverse tra loro, al punto che "giocare a *Root* con quattro fazioni è come se quattro persone giocassero quattro giochi diversi".

L'agency passa dal ruolo. Nei giochi, in generale, esprimiamo la nostra **agency** tramite il ruolo. Il ruolo deve esserci, in qualche modo: magari non è specificato, magari non lo si dichiara apertamente, ma sarà implicito che abbiamo un ruolo, e in particolare un ruolo con possibilità di agency all'interno del gioco: possiamo impattare il mondo di gioco, l'universo narrativo, possiamo decidere cose. E il ruolo porta con sé **responsabilità** e **possibilità meccaniche**. Non a caso, "di solito per spiegare le meccaniche spieghiamo che ruolo abbiamo": il ruolo giustifica chi siamo e che cosa facciamo.

In *Nemesis*, per esempio, siamo dei **sopravvissuti** su un'astronave dove ci sono degli alieni che vogliono ucciderci, e dobbiamo andarcene salvando la pelle. Ci sono anche personaggi asimmetrici con le loro caratteristiche, "ma il nostro ruolo ci dice già l'obiettivo del gioco: dobbiamo sopravvivere a questa situazione".

[9] "Dungeon Dragon" nella trascrizione, qui e altrove.
[10] "Route" nella trascrizione.



Carlo Magno: "re" è il ruolo; il nome, il passato, la storia sono il personaggio.
Scacchi, *Pong*, *The Gallerist*: ruolo senza personaggio.
D&D: personaggio con un ruolo (la classe).
Root: ruoli così diversi da sembrare giochi diversi.

Figure 2.4.1: Player, character e role nella lettura del docente. Chi gioca sta fuori dal gioco e agisce attraverso il ruolo; il personaggio, quando c'è, avvolge il ruolo e lo lega all'universo narrativo. Il ruolo c'è in quasi tutti i giochi, il personaggio no.

2.4.2.1 Il ruolo non basta

Detto questo, "potrebbe passare l'idea sbagliata": che i personaggi non servano, che siano quasi un appiglio, una **decorazione** del ruolo. "Non è così". Il ruolo, di per sé, è **molto freddo**. Se si vuole fare un gioco che susciti emozioni molto calde, molto specifiche, non basta un ruolo che dica che cosa si può fare meccanicamente: servono elementi che ci aggancino a una **narrazione**, a un **plot**, a un **world building**. È estremamente utile in un gioco di ruolo, in un gioco di ruolo dal vivo, ma serve anche in un gioco da tavolo e in altre tipologie di giochi. Semplicemente, storicamente, i giochi di ruolo sono quelli che hanno prodotto la maggior parte dei framework sulla questione dei personaggi; per questo, avverte il docente, gli esempi della lezione verranno soprattutto da lì, "non perché non servano per gli altri giochi", ma perché lì le sfumature sono più variegate e danno più strumenti per capirle.

2.4.3 Che cosa rende giocabile un personaggio

"Partiamo da una prima domanda che vi faccio, e voglio le vostre risposte": **che cos'è che rende un personaggio giocabile?** "Non abbiate paura".

Le risposte della classe. Le risposte arrivano una dopo l'altra:

- deve avere **possibilità di interazione** con il contesto: una pietra non è un personaggio, perché non può interagire in alcun modo con il gioco;
- tutto quello che lo caratterizza deve potersi **tradurre in azioni** nel gioco, in qualcosa che sia chiaro e realizzabile ("che è un po' quello che ha detto lui"); qualsiasi caratteristica, anche quella che gli dà solo spessore, e le sue abilità devono essere trasferibili in qualcosa che ha a che fare col gioco stesso. Il docente riassume: "un personaggio che può fare azioni che hanno impatto nel gioco? Sì, fondamentalmente sì";
- è un personaggio che **posso controllare** direttamente;
- deve essere **in linea con le regole** del gioco: chi crea il proprio personaggio basandosi sul manuale del giocatore sa che cosa può fare e che cosa no, ha una direzione.

Sull'ultima risposta il docente si ferma: è "un esempio molto interessante", perché presuppone un gioco in cui il personaggio lo si crea, un gioco di ruolo tradizionale come *D&D*. Ma allora, in un gioco da tavolo, si

può avere un personaggio giocabile? Una studentessa cita *I viaggi di Marco Polo*, con i suoi personaggi dalle caratteristiche e proprietà precise, "incastrabili" con il gioco: è chiaro che cosa possono fare e che cosa no. "Anche questa è una cosa fondamentale".

Il cavaliere, il mercante, l'orco. Il docente rilancia con un esperimento mentale. Un gioco fantasy, non importa se di ruolo o da tavolo. Il nostro personaggio è un **cavaliere**: è giocabile? "Di base sì". E un **mercante**? "Potrebbe essere giocabile anche questo, dipende". Vi vengono in mente esempi di personaggi *non* giocabili? Qualcuno propone un marine, che con il fantasy non c'entra niente: "però quello è un personaggio che non esiste nel gioco". La domanda riguarda personaggi che *esistono* nel gioco, ma non sono giocabili. Un **orco**? "Perché no? Potrebbe esserlo". Ma allora, il dubbio: **tutti i personaggi sono giocabili**? Sì, risponde qualcuno, se si ha un'idea di che cosa fa il personaggio.

I PNG non sono "non giocabili". Lorenzo porta l'esempio di *D&D*: dipende dal master e dai giocatori, perché i giocatori hanno il loro personaggio, mentre il master gioca anche i PNG. Il docente coglie l'occasione per il termine tecnico: gli **NPC**, *non-playable characters* (i PNG, personaggi non giocanti), sono, nel gioco di ruolo da tavolo ma anche nel LARP, i personaggi non giocati da chi siede al tavolo, ma controllati da chi facilita o dal master. Sono identificati come "non giocabili", "ma è una falsa definizione, in realtà: se ci pensate, qualcuno li sta giocando, quei personaggi". Per mille motivi il termine si è solidificato e si usa così.

Ma la questione qui è un'altra. Se devo progettare un personaggio *per essere giocabile*, "non mi basta fare un personaggio che una persona può giocare". Altrimenti si torna all'esempio della lezione precedente: **Aragorn che aspetta in taverna**. Posso interpretare Aragorn per due ore, nella taverna, ad aspettare: ma è giocabile, quel personaggio? Non posso fare praticamente niente di quello che quel personaggio sa fare; anzi, il gioco mi dice che *non devo* farlo.

2.4.3.1 I quattro pilastri

Si arriva così a **quattro pilastri** che il corso associa all'idea di personaggio giocabile, e che il docente chiede di tenere a mente ogni volta che si progettano personaggi "con l'intento di darli in mano a una persona che deve giocare il vostro gioco" (figura 2.4.2):

1. **chiarezza**: il personaggio fornisce informazioni chiare su *come* deve essere giocato;
2. **connettività**: il personaggio fornisce informazioni sulle sue *connessioni* in rapporto ad altre entità;
3. **spinte**: il personaggio fornisce informazioni sulle sue *motivazioni* e sui suoi *obiettivi* a breve e lungo termine;
4. **attività**: il personaggio fornisce informazioni su che cosa *può* e *deve* fare.

2.4.4 La chiarezza: sette schede per Joey Doe

Si parte dalla **chiarezza**: **il personaggio fornisce informazioni chiare sul come deve essere giocato**. Vedendo il personaggio, leggendo la sua scheda, guardando la sua carta, a seconda del tipo di gioco, devo avere chiaro *come* giocarlo. Se leggo "cavaliere" avrò delle idee su che cosa posso fare e su che cosa finirò per fare, e lo stesso se leggo "mercante". Se invece leggo un'etichetta, una definizione poco chiara, che non mi dice che cosa devo fare, "il gioco non lo passerò giocando quel personaggio, lo passerò capendo che cosa diavolo devo fare".

L'esperimento. Per mostrarlo, il docente proietta una serie di **proto-schede** ("alcune sono molto minimali, vi avviso"). Il contesto è sempre lo stesso: un **gioco di ruolo dal vivo** in cui, all'inizio, si riceve un foglio con il proprio personaggio. Dopo ogni scheda chiede: come giochereste questo personaggio? Che cosa avete capito? Che cosa vi aspettate di fare? Il **setting** è tolto "dall'equazione", perché "già col personaggio abbiamo diversi problemi" (del world building si parlerà con Daniele): si suppone che tutti abbiano presente



Figure 2.4.2: I quattro pilastri del personaggio giocabile. Nessuno basta da solo: "il personaggio giocabile è l'insieme di questi quattro pilastri". Spinte e attività sono strettamente connesse: le attività sono ciò che si può fare per inseguire le spinte.

il classico **film investigativo noir**, americano, anni Trenta e Quaranta, in bianco e nero[11]. Il personaggio è sempre lo stesso: **Joey Doe**.

2.4.4.1 Scheda 1: solo il ruolo

Si riceve un foglietto con scritto soltanto:

> *Joey Doe, detective.*

Chi siete? Che cosa potete fare? "Molto poco". Ma il gioco sta per iniziare, e questo è il vostro personaggio. Che cosa vi aspettate di giocare? "Un detective". E che cosa vuol dire giocare un detective? Le risposte: scoprire qualcosa, trovare informazioni ("però non sappiamo dove, con che strumenti, che cosa"); esplorare ambienti, interrogare persone; se c'è un omicidio, trovare l'assassino (Lorenzo); fare i monologhi interiori dei film noir, raccontando quello che si pensa e si vede (Angelo). "Avrai i tuoi pensieri, certo".

2.4.4.2 Scheda 2: un elenco di caratteristiche

Stesso setting, ma la scheda è:

> *Nome: Joey Doe. Professione: detective. Caratteristiche: ha un istinto acuto; ha un'alta determinazione; ha il rispetto dei colleghi.*

Che cosa cambia? "A me niente, sinceramente", dice Tiziana: o molto poco. Perché? Perché la scheda le dice eventualmente *come* fa le cose, ma non *che cosa* fa: sono le stesse identiche cose di prima. Il docente le dà ragione, ma indica la differenza. "Detective, nel mondo, ce ne sono mille": può essere il tenente Colombo, Sherlock Holmes, l'interrogatore della polizia che va avanti a pugni e revolver, e tutte e tre queste identità rientrano nella definizione. Qui la scheda esplicita qualcosa in più del personaggio, che a chi aveva già in mente un certo tipo di detective sembra non servire; "non è ancora sufficiente per molte persone", ma è meglio di ricevere solo il nome.

Che cos'altro fa questa scheda? Fornisce **caratteristiche personali** sul carattere, dice Rita: la personalità del detective, "l'atteggiamento che bisogna avere durante il gioco". "Bravissima". Aiuta a calarsi nel ruolo, aggiunge, pur non avendo mai giocato un gioco di ruolo ("non importa"). Il docente fissa il punto: la scheda precedente dava **solo il ruolo**; qui compaiono degli aspetti del **personaggio**. Istinto acuto, determinazione, rispetto dei colleghi "non sono meccaniche": non dicono che cosa si può fare meccanicamente, dicono che il personaggio ha certi aspetti caratteriali o di background.

[11] La trascrizione aggiunge "quelli da Ibs Lee", un riferimento non ricostruibile.

2.4.4.3 Scheda 3: una voce esterna

Stesso gioco, stesso setting, e questa scheda:

> *Sei Joey Doe, una detective esperta, nota per i tuoi istinti acuti e la tua determinazione incrollabile. Hai una mente acuta, sempre in grado di mettere insieme anche i più piccoli indizi per risolvere i casi complessi. I tuoi colleghi rispettano le tue capacità e hai la reputazione di arrivare sempre alla radice dei problemi.*

Che cosa cambia? "Conosciamo il genere" (la detective è una donna); le caratteristiche di prima sono più dettagliate; e forse c'è un indizio su *come* applicare l'essere detective: stare attenti ai piccoli indizi. "Ti inizia a dare delle frasi non su come sei, ma su che cosa fai come personaggio": non ancora come ruolo, non ancora come meccaniche.

E poi, nota Gianmarco, la scheda "sembra più raccontata": quasi la storia del personaggio, invece che un elenco schematico. "Bravissimo". Ma *chi* ce lo racconta, questo personaggio? Un narratore onnisciente, propone qualcuno. Il docente sposta l'attenzione sul confronto: la scheda precedente non la racconta nessuno, è un insieme di caratteristiche prese come **fatti**; qui invece ci arriva l'**opinione esterna** di qualcuno che conosce Joey Doe e ci dice, a grandi linee, chi siamo in questo gioco.

2.4.4.4 Scheda 4: la sua voce

> *Sono Joey Doe, una detective che ha un po' di esperienza alle spalle. I miei forti istinti e la mia determinazione implacabile mi hanno aiutata a risolvere molti casi difficili. La mia mente acuta mi consente di collegare gli indizi anche quando non sembrano correlati. I miei colleghi rispettano le mie capacità e ho la reputazione di arrivare sempre alla radice dei problemi.*

Che cosa cambia qui? È **il suo parere su se stessa**: è lei che si descrive. A che cosa serve? A calarsi meglio nei panni del personaggio: "sentiamo la sua voce", sentiamo come parla. Ma, dal punto di vista del game designer, "dall'alto", serve anche a un'altra cosa molto importante: con schede scritte così si può **giocare su incomprensioni e punti di vista diversi**. Posso dare a un collega di Joey Doe una scheda che dice "Joey Doe non capisce niente, crede di essere la migliore del dipartimento, ma fa un sacco di disastri", mentre Joey Doe ha questa scheda. Con una serie di schede di questo tipo si possono creare **punti di vista contrastanti**.

Rita collega la scheda al **coinvolgimento narrativo** della lezione precedente: è lì che entra in gioco? "Esattamente": nella prima scheda il coinvolgimento narrativo "non esiste. Zero". Di scheda in scheda, anche se sono esempi da tre righe, lo si sta **centellinando**, mettendolo "sotto pelle", ma sempre al servizio del pilastro della chiarezza: leggere la scheda e sapere chi è Joey Doe e che cosa ci si può fare.

La prova: il gioco inizia subito. "Il gioco inizia subito, questa è la sua scheda, prego. Che cosa farete?" La risposta di una studentessa è ricca: risolvere casi difficilissimi in maniera impeccabile, stando attenta ai piccoli dettagli e alle indicazioni degli altri, contando sull'appoggio dei colleghi. E interagire con molti personaggi; come? Con convinzione, in modo spigliato, brillante, "sempre sul pezzo", senza insicurezze, perché dalla personalità raccontata "esce fuori un'identità molto sicura e molto forte". Ecco perché **questa scheda è più chiara delle altre**: "so che sembra una banalità", ma non è più chiara perché spiega di più con più parole: è più chiara perché dà **più informazioni su come giocare** Joey Doe.

2.4.4.5 Scheda 5: una fotografia

Ora, sempre lo stesso gioco, il docente consegna un'altra scheda: una **fotografia**, con scritto soltanto il nome, Joey Doe. Che cosa giocate? Chi siete? "Un maschio, sicuramente".

È il momento in cui entra **Nicola**, che ha avuto difficoltà a collegarsi ("fatti miei": "non è un problema"). Il docente lo rassicura (la lezione sarà registrata e le slide condivise) e lo porta in pari: si stanno guardando delle schede fittizie di un gioco dal vivo investigativo noir anni Quaranta, e ci si chiede, d'istinto, chi si è e che cosa si può fare.

Che cosa dice la foto? "Potrebbe essere benissimo l'investigatore come il *serial killer*". Esatto: "qui non ho scritto detective da nessuna parte". Un'identità misteriosa, certo, ma la foto porta con sé anche altro.

Come mi comporterò, in un gioco dal vivo dove altri personaggi girano per l'ambiente? "Sembra un tipo sicuro di sé"; forse sprezzante del pericolo, anche se non necessariamente. Che cosa fa questa scheda? Dà **suggestioni** più che indicazioni: un tipo che sta nell'ombra, che non si capisce chi sia. E dà l'immagine visiva dell'ambiente, del *mood* del gioco, quell'atmosfera nebbiosa.

Che cosa fa meglio. Il docente aggiunge una cosa che "sembra una banalità": con la foto si ha un'idea di **come si veste** Joey Doe. In un gioco di ruolo dal vivo, molti in costume, questo vuol dire sapere come vestirsi: un cappello, un cappotto. La foto dà indizi su come il personaggio si pone a livello visivo, cosa che le schede precedenti non facevano.

Che cosa fa peggio. Non **guida** il giocatore a capire che cosa deve fare. Per alcuni è un investigatore, per altri un serial killer, per altri un criminale: non è sbagliato, perché la parola detective non c'è.

2.4.4.6 Scheda 6: gli oggetti

Stesso gioco, stesso setting. "Siete Joey Doe", e il docente consegna fisicamente degli **oggetti**: sono vostri, potete usarli durante il gioco. Che cosa posso fare nei panni di Joey Doe? "Figo". Fare fotografie, ammanettare, sparare: perché vedo una macchina fotografica, delle manette, una pistola.

Nei LARP questi oggetti si chiamano **prop**: strumenti con una funzione **più o meno meccanica**. Se ammanetto qualcuno, il gioco tendenzialmente mi dirà che cosa succede quando si ammanetta; se il gioco mi mette in mano una pistola, di solito mi dice che cosa succede quando si punta o si spara. Ma gli oggetti non sono solo e puramente meccaniche: sono **possibilità di agency**, modi che il personaggio ha per interagire con il mondo. Posso documentare, ammanettare, sparare.

E che ruolo avrò? Sono un investigatore o un serial killer? C'è di nuovo ambiguità, "ma non è un'ambiguità totale": "sicuramente non sono un panettiere". Gli oggetti comunicano già qualcosa con chiarezza; ma sono, nello specifico, cose che Joey Doe **può fare**, più che cose che Joey Doe **è**. E un personaggio è l'insieme delle due: di quello che è e di quello che può fare, di personaggio e di ruolo, che spesso si intersecano.

2.4.4.7 Scheda 7: le istruzioni di comportamento

Ultima scheda, stesso setting:

> *Nome: Joey Doe. Devi guardare gli altri personaggi in modo severo, devi farli sentire sempre sotto interrogatorio, devi farli sentire come se leggessi la loro mente.*

Uno studente chiede se debba tener conto delle slide precedenti: no, è una nuova scheda, "le precedenti non esistono", è l'unica cosa che si riceve per giocare Joey Doe. "Allora potrebbe essere mia madre", scherza qualcuno. "Potrebbe essere tua madre. Ma come giocherai Joey Doe?" Come verrà percepito il personaggio, in un gioco fatto spesso di interazioni sociali? Un rompiscatole, rispondono; qualcuno che invade lo spazio privato degli altri.

Il setting è sempre il noir investigativo del 1940. Il docente non sta dicendo che si è un detective, e il personaggio potrebbe davvero essere "la madre di qualcuno". Ma il **comportamento** è quello di una persona che vuole tirare fuori qualcosa, mettere sotto pressione gli altri, estrarre informazioni: quindi **funge da** quello che ci serve che faccia il detective. E qui sta la domanda di design:

> **che cosa serve nel nostro gioco? Un detective come personaggio, o un detective come ruolo?** Qualcuno che sia un detective come background, o qualcuno che agisca da detective e faccia cose da detective?

E quanto di questo è chiaro? Magari al game designer serve un personaggio che sia *meccanicamente* un detective, ma la scheda non lo rende chiaro; e allora chi lo gioca fa "un poliziotto generico" che cerca di capire chi ha commesso il crimine, ma non va a fare pressione, non mette sotto torchio gli altri personaggi.

Scheda	Che cosa dà	Che cosa manca o rischia
1. "Joey Doe, detective"	solo il ruolo, un'etichetta	che detective? Dove, come, con quali strumenti?
2. Elenco di caratteristiche	aspetti del personaggio (istinto, determinazione, rispetto dei colleghi) presentati come fatti	cambia poco su *che cosa* si fa
3. Seconda persona	un'opinione esterna; il genere; i primi accenni a che cosa si fa (gli indizi)	la voce del personaggio
4. Prima persona	la voce del personaggio, il suo modo di vedersi; permette punti di vista contrastanti tra schede	— (la più chiara della serie per questo gioco)
5. Fotografia	mood, atmosfera, come vestirsi e porsi	nessuna guida: detective, serial killer o criminale?
6. Oggetti (prop)	possibilità di agency e meccaniche: documentare, ammanettare, sparare	che cosa il personaggio *è*; resta ambiguità
7. Istruzioni di comportamento	il comportamento: funge da detective	non dice chi è; detective come ruolo, non come personaggio

Table 2.4.1: Le sette schede di Joey Doe, a parità di gioco e di setting (LARP investigativo noir, anni Quaranta). Nessuna è "giusta" in assoluto: la chiarezza si misura su ciò che serve al gioco che si sta progettando.

2.4.4.8 Chiarezza non vuol dire lunghezza

Il docente ci tiene a precisare che **questi esempi non sono qualitativi** rispetto al gioco che si vuole progettare: "non è detto che una scheda più ricca di parole sia più chiara per quello che serve a noi". I giochi del movimento **Dogma 99**, ispirato al Dogma 95 del cinema e citato nelle lezioni precedenti, hanno schede personaggio fatte così: "sei Tizio Caio. Fine".

E se la chiarezza non ha un risvolto meccanico? Una domanda "velocissima": la chiarezza deve avere necessariamente un risolto meccanico? Lo studente pensa a *Le case della follia*[12], dove la scheda dice chi è il personaggio, ma quell'informazione, a livello pratico e meccanico, non servirà mai. "Non necessariamente", risponde il docente, ma *Le case della follia* fa un'altra cosa: dice qual è il **ruolo** dei personaggi. Chi si gioca? Un gruppo di persone che in teoria dovrebbero essere investigatori; che cosa fanno di sicuro? **Indagano**: sono persone che entrano in una villa misteriosa per indagare. Quello è il loro ruolo, e quella parte deve essere chiara, e lì lo è. Poi il gioco aggiunge dei **layer di flavor**: i giochi da tavolo spesso relegano questa parte al *flavor*, e se non si legge il background di un personaggio "basta giocare per il ruolo".

In un gioco di ruolo, invece, è difficile che queste cose escano in bel gioco senza una scheda curata: sono, come già detto, tipi di giochi diversi, e il livello narrativo richiesto da un personaggio in un board game "è difficile che sia equiparabile" a quello richiesto in un gioco di ruolo. E comunque, *Le case della follia* dà degli aspetti di giocabilità: dei personaggi stanno indagando in una magione misteriosa e qualcosa va storto, ed è quello che serve meccanicamente per giocare. In più è un gioco molto flessibile: i personaggi si adattano in corso d'opera, anche in base al **branching narrativo**, e può darsi che il mio personaggio non fosse semplicemente un sopravvissuto nella casa, ma "in realtà il colpevole nascosto"[13].

2.4.5 La connettività: le tre cerchie

Secondo pilastro: la **connettività**. **Il personaggio fornisce informazioni sulle sue connessioni in rapporto ad altre entità**. Perché "le persone non esistono in un vacuum": sono connesse tra loro. Un

[12] *Mansions of Madness*, nell'edizione italiana.

[13] La trascrizione dice "era in realtà il giallo nascosto"; il senso dell'"esempio stupido" è comunque quello di un ruolo rivelato a partita in corso.

personaggio **lupo solitario**, che non conosce nessuno, senza background, senza genitori, figli, fratelli, sorelle, amici, compagni di classe, colleghi, "non è giocabile". "So che questa cosa ci metterà un po' a interiorizzarla", ma la giocabilità di un personaggio è data anche dalle **relazioni sociali** in cui è espresso e messo in gioco.

Tre gruppi. Di solito, quando si presentano personaggi giocabili dal punto di vista della connettività, si fa riferimento a **tre gruppi**: tre gruppi *separati* di persone che conoscono il personaggio e a cui il personaggio è legato. Quelli della slide sono di esempio: non devono per forza essere questi, anche perché dipendono dal gioco (in un mondo post-apocalittico, "il circolo del lavoro non è proprio la prima cosa che viene in mente"). Per Joey Doe (figura 2.4.3):

Lavoro. Joey Doe è un'ex poliziotta che ora fa la detective privata. Ha quindi tutto il gruppo dei poliziotti: ex colleghi, altri detective, il capo della polizia, il sovrintendente di zona, e così via.

Amicizia. La sera, per decomprimere, va al pub *Il Barile*. Magari è amica del barista, chiacchiera con due o tre clienti abituali. Non è lavoro; magari si interseca con il lavoro (al pub c'è anche un ex poliziotto suo amico), ma è una cerchia separata: non è detto che il barista conosca il responsabile di zona della prefettura.

Famiglia. Magari Joey Doe è l'ultima rimasta della famiglia Doe. Che cosa è successo alla famiglia? Suo fratello è scomparso. Dov'è finito? È ancora vivo? Anche questa è una cerchia a sé: non è detto che il fratello conosca il barista, né i colleghi.

Perché serve. Si torni al LARP investigativo anni Quaranta. Joey Doe entra nella stanza e vede uno dei suoi amici del pub, un ex collega poliziotto e suo fratello, vivo. Ha già **tre direzioni di connessione che la tirano**, e queste direzioni "sono molto giocabili": danno possibilità di gioco. È diverso da un Joey Doe che si trova in un LARP da venti persone, c'è stato un omicidio, lei è l'investigatrice e non conosce nessuno. "C'è un crimine e devo investigare" è meno giocabile di: c'è un crimine, e nel frattempo mio fratello è ancora vivo, il barista ha un problema di soldi con la malavita, e il mio ex collega ha scoperto una verità sul sovrintendente che potrebbe ribaltare la città.

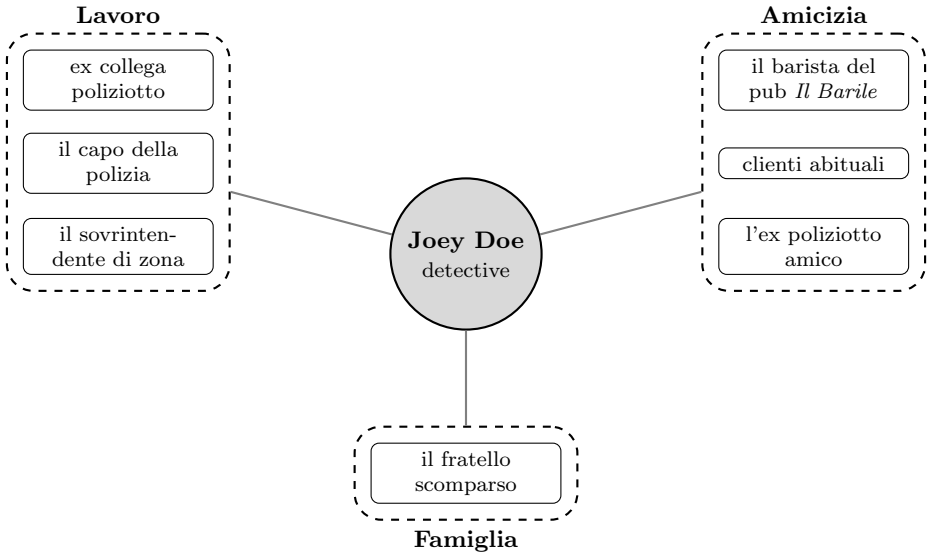


Figure 2.4.3: Le tre cerchie di Joey Doe. Ogni cerchia è un gruppo separato: i suoi membri conoscono Joey Doe, ma non necessariamente gli altri gruppi. Le intersezioni sono possibili (l'ex poliziotto che frequenta il pub), ma le cerchie restano distinte. Ogni legame è una direzione di gioco, a patto che l'altro personaggio sia davvero in gioco.

2.4.5.1 Le relazioni esistono solo se le giochi

Una slide fa da *reminder*:

> **le relazioni del tuo personaggio non esistono finché non ti comporti in modo da farle esistere.**

Si può scrivere sulla scheda di Joey Doe: sei amica del barista del pub *Il Barile*, cerchi da dieci anni tuo fratello scomparso, hai una faida con l'ex capo del dipartimento. Ma se questi personaggi sono in gioco e non ci si parla mai, non ci si interagisce, quelle relazioni **non esistono**, non sono giocabili. Allo stesso modo, se la scheda dice "cerchi tuo fratello" e il fratello in gioco non c'è, nessuno lo gioca e non ci sono informazioni su di lui, "è un dato in più che non ti serve a niente". Ha un impatto narrativo, "ma a livello di giocabilità non esiste".

Per progettare personaggi fatti bene, cioè giocabili, si deve poter **interagire e agire sulle proprie relazioni**. Questo rende i personaggi più vivi e dà coinvolgimento narrativo, ma anche affettivo. Di nuovo, generi diversi hanno necessità diverse: la connettività in un gioco di ruolo richiede un certo livello di profondità e di cura; in *Le case della follia* basta sapere che si è in cinque a esplorare una casa abbandonata.

Approfondimento: la rete dei personaggi. Schell propone uno strumento molto vicino alle cerchie, la *Lens of the Character Web*: si fa la lista di tutti i personaggi e ci si chiede, per ciascuno, che cosa prova esattamente verso ciascuno degli altri; se ci sono connessioni non considerate, e come usarle; se ci sono troppe connessioni simili, e come differenziarle. Schell nota che il successo di serie televisive come *Lost* viene da un'esplorazione profonda di questa rete, e che nei giochi è raro vederla esplorata bene, "il che la rende un'opportunità da cogliere". La regola del docente aggiunge la condizione propria del gioco: una connessione conta solo se tutti e due i capi del filo sono in gioco, e se qualcuno la tira.

2.4.6 Spinte e attività

Gli ultimi due pilastri il docente li affronta insieme, "perché sono connessi tra di loro".

Spinte. Il personaggio fornisce informazioni sulle sue **motivazioni** e sui suoi **obiettivi** a breve e a lungo termine. Sapere che cosa il personaggio vuole lo rende giocabile: "se non sappiamo quale obiettivo il nostro personaggio vuole, che cosa facciamo?"

Attività. Sono connesse alle spinte: sono le informazioni su che cosa il personaggio **può** e **deve** fare. Di solito sono un insieme di **possibilità meccaniche** e di **responsabilità meccaniche**, e le due cose vanno un po' insieme.

Gli esempi della slide, sempre su Joey Doe:

Ottenere la promozione. È una spinta: voglio essere promossa nel dipartimento. Come? Che attività posso fare? Magari risolvere un caso importante; magari smascherare la corruzione nel dipartimento, in modo che il mio diretto superiore finisca in prigione e si liberi un posto.

Ritrovare tuo fratello. Un'altra spinta. Attività possibili: indagare nell'ultimo posto in cui è stato visto; raccogliere informazioni su di lui, magari parlando con qualcuno che l'ha visto per l'ultima volta.

Il consiglio: ragionate sempre su **che cosa vuole** il vostro personaggio giocabile, e su **che cosa, in gioco, gli si può far fare** per andare avanti con i suoi obiettivi.

Spinte raggiungibili. "Le spinte e le attività rendetele giocabili". Se il personaggio è un rivoluzionario che vuole assassinare il dittatore, e il dittatore non è in gioco, quella spinta **non esiste come spinta**: non ci sono attività da fare. Le spinte devono essere qualcosa di **raggiungibile**, cose che in gioco si sa di poter fare; altrimenti "diventano background, diventano pura narrazione", e non sono meccanicamente giocabili. "So che può sembrare molto astratta, questa roba".



Figure 2.4.4: Spinte (in maiuscolo, come nella slide del docente) e attività (i punti elenco). Una spinta senza attività possibili in gioco non è una spinta: è narrazione.

Il pilastro della chiarezza, violato. Gianmarco fa una domanda "veramente banalissima": nella slide, ciò che è in maiuscolo sono le spinte e i punti elenco le attività? "Esattamente". "Scusami, non ho rispettato il pilastro della chiarezza", scherza il docente: "fortunatamente questa non è una scheda personaggio". Quelle in grassetto maiuscolo sono le spinte, quelle sotto le attività: le spinte sono motivazioni e obiettivi a breve e lungo termine, le attività sono che cosa posso e devo fare.

Una spinta ingiocabile è almeno chiara? Gianmarco rilancia: una spinta o un'attività non giocabile diventa quasi un elemento di chiarezza; se devo uccidere un personaggio che non c'è, comunque mi dice qualcosa su ciò che devo fare. "Certo, però ricordiamoci che il personaggio giocabile è **l'insieme di questi quattro pilastri**". La chiarezza da sola non basta: posso avere chiarissimo come il mio personaggio deve essere giocato, ma se il gioco non mi dà possibilità meccaniche, cioè le attività, il personaggio non è giocabile.

Il resto è permesso. Nulla impedisce di mettere nella scheda informazioni di background che non sono spinte né attività: anzi, possono arricchire il personaggio, dare informazioni sulle *vibes*, aiutare a creare **coinvolgimento affettivo**, "per mille ragioni". Magari il personaggio ha la camicia hawaiana, e chi lo legge pensa: "mi stai simpatico". È un collegamento affettivo, ma non dice niente delle sue meccaniche e non lo rende più giocabile.

2.4.6.1 Pilastri e coinvolgimento

Il docente precisa che il framework dei quattro pilastri **non è mutualmente esclusivo** con i ragionamenti sulle sei aree di coinvolgimento del Player Involvement Model visto nella lezione precedente. Una studentessa chiede di chiarire il parallelismo. La risposta: le necessità del creare un personaggio giocabile e quelle dell'avere un gioco che coinvolge nelle sei aree "non è detto che siano le stesse". Si può avere un gioco coinvolgente a livello narrativo e affettivo che non ha personaggi giocabili fatti bene. Sono due framework che a volte "si possono parlare": non ci si deve limitare a dire "lascio questa informazione come background, senza meccaniche, senza spinte né attività, tanto mi dà coinvolgimento narrativo". È vero, "ma magari possiamo fare qualcosa di meglio".

La prima pausa. Prima di passare al prossimo argomento il docente insiste: "se non mi sto spiegando, vi prego, ditemelo". Poi manda la classe in pausa: sette minuti, "così facciamo il numero tondo alle 18:10"; ma su richiesta diventano dieci. Al ritorno, priorità a chi ha domande o ragionamenti maturati nel frattempo.

Approfondimento: tratti e funzioni. Il pilastro della chiarezza ha un parente stretto in Schell, la *Lens of Character Traits*: quali tratti definiscono il personaggio, e come si manifestano nelle sue parole, nelle

sue azioni e nel suo aspetto? Schell consiglia di non fermarsi a una "bibbia" esaustiva del personaggio, ma di distillarne pochi tratti che lo accompagnino in molte situazioni, anche un po' contraddittori, "perché le persone vere hanno tratti contraddittori". È esattamente il passaggio dalla scheda 2 di Joey Doe (i tratti elencati come fatti) alla scheda 4 (i tratti che si sentono nella voce). Accanto, la *Lens of Character Function* chiede di elencare le *funzioni* che i personaggi devono svolgere nel gioco, separandole dall'immagine che si ha dei personaggi, e di vedere come combaciano: è un altro modo di dire che ruolo e personaggio vanno pensati separatamente, e poi fatti incontrare.

2.4.7 Chi decide chi siamo: estetica, fiction e ruolo

Si riparte dopo la pausa ("l'effetto venerdì pomeriggio": qualcuno è rimasto indietro, e una studentessa non riusciva ad accendere il microfono per via della banda). Nessuna domanda nuova su quanto visto finora: chiarezza, connettività, spinte, attività.

Quando si progettano personaggi, cioè quando si fa game design di un gioco che ha dei personaggi, c'è una domanda da porsi: **chi decide chi siamo in un gioco?** Ci sono tre aree da considerare:

Estetica. L'estetica del personaggio "a tutto tondo": non solo visiva, ma anche in termini tematici; il *look and feel* del personaggio.

Fiction. Come il personaggio è integrato nella narrazione e nel world building: chi è all'interno dell'universo narrativo.

Ruolo. Quello di cui si è già parlato: le possibilità e gli obblighi meccanici del personaggio.

Su ciascuna si ragiona con uno **slider**, come quelli del Mixing Desk della lezione precedente: sono "slider ad hoc" per i personaggi, con a un estremo il **designer** e all'altro il **player** (figura 2.4.5). "Sono slider come nel Mixing Desk, non sono pulsanti": non è o l'uno o l'altro, e anzi la posizione è quasi sempre una via di mezzo.

2.4.7.1 Lo slider dell'estetica

Sul lato **player**, una qualunque scheda di *D&D* con il riquadro per disegnare il proprio personaggio: è chi gioca a decidere, ha l'autorità di disegnare che faccia ha il personaggio. Sul lato **designer**, giochi di ruolo con **personaggi prescritti e predisegnati**, come *Witch: The Road to Lindisfarne*[14]: lo si può giocare solo interpretando i personaggi che chi lo ha progettato ha deciso. Quello mostrato a schermo, con dei volatili in mano, è il personaggio che si occupa delle cene della carovana. Il gioco parla di accompagnare una donna che forse è una strega, forse no, a essere giudicata, in un'Inghilterra devastata dalla peste. I personaggi hanno un disegno preciso, e "noi giochiamo questa faccia qua": al massimo si decide come interpretarne il carattere, che cosa fargli dire, ma non la faccia, non gli abiti. L'estetica è in mano a chi ha fatto il gioco.

Le vie di mezzo: in *Non cedere al sonno*[15] l'estetica la si decide in parte, ma è certo che i personaggi sono **umani**, non mostri: persone che soffrono d'insonnia. Sono aspetti decisi dal designer. E anche le **classi** di *D&D* hanno, per certi versi, una loro estetica: "è difficile vedere un ranger in armatura a piastre".

2.4.7.2 Lo slider della fiction

Chi decide chi è il personaggio nell'universo narrativo? Lato **player**: di nuovo *Non cedere al sonno*, dove chi gioca scrive che cosa lo tiene sveglio, che cosa gli è appena successo, come gli altri lo vedono da fuori e che cosa c'è sotto la sua pelle. Lato **designer**: *Lady Blackbird*[16], "gioco bellissimo, tra l'altro anche gratuito", i cui personaggi sono prescritti: background, nome e relazioni sono già decisi da chi ha fatto il design. Se ne giocano le azioni, ma il loro posto nel mondo narrativo è già stabilito.

[14] "Witcher of the Lindisfarne" nella trascrizione. È un gioco di ruolo narrativo di Kevin Weir, con personaggi predefiniti e illustrati. Il docente lo ambienta "nella Londra sotto la peste" e parla di portare la presunta strega "all'inquisizione"; nel gioco il viaggio attraversa l'Inghilterra medievale colpita dalla peste, fino all'isola di Lindisfarne, dove la donna dovrà essere giudicata.

[15] L'edizione italiana di *Don't Rest Your Head* di Fred Hicks, già citato nelle lezioni precedenti.

[16] Di John Harper, distribuito gratuitamente.

E la generazione casuale? Gianmarco, che ha giocato di ruolo in passato ma poi ha smesso, chiede se esistano giochi in cui la fiction è generata casualmente, da carte o simili. "Sì, ce ne sono parecchi" (il docente promette esempi più precisi a fine lezione). Dove sta, allora, la decisione? È un mix, e la domanda è: in fase di creazione abbiamo *solo* elementi generati a caso, o un **pool casuale da cui scegliere**? L'esempio: un gioco di ruolo con dieci classi possibili; a inizio partita se ne pescano tre e se ne sceglie una. Chi ha fatto design ha deciso l'insieme delle classi e il pool; chi gioca decide chi è dentro quella scelta. È una via di mezzo "molto verso il designer". E anche *Non cedere al sonno* non è puro: si giocano personaggi insonni, e l'essere insonni è un aspetto di fiction deciso dal designer.

2.4.7.3 Lo slider del ruolo

Il ruolo meccanico: le possibilità di azione, le cose che si possono fare. Chi le decide? In *D&D*, una qualunque edizione, non si ha accesso a tutte le abilità, a tutte le potenzialità meccaniche di una classe: da quel pool si selezionano quelle che il personaggio possiede. Altri giochi fanno diversamente.

Apocalypse World. È un gioco di ruolo da tavolo con master in cui i personaggi sono **meccanicamente diversi**. Si gioca un **libretto** (*playbook*), che dice che classe di personaggio si ha, come il paladino o il guerriero, "applicata al post-apocalittico". Quello mostrato è l'**autista** (*the Driver*): la sua classe è guidare un veicolo, e tutte le sue **mosse** (*moves*), cioè i punti di agency, le cose meccaniche che può fare per impattare il mondo, sono legate ai veicoli. Il killer prezzolato avrà altre mosse, il recuperatore di rottami altre ancora. L'autista, per esempio, ha una mossa che gli permette di usare il proprio veicolo come un'arma; gli altri, quando guidano, non possono farlo, "perché non è nelle loro meccaniche".

Uno studente collega l'esempio a *Nemesis: Retaliation*: anche lì ogni scheda personaggio dice che cosa il personaggio può fare. "Esattamente". Nei giochi da tavolo questo succede tantissimo: il ruolo è spesso deciso dal designer. Magari si pescano tre personaggi e se ne sceglie uno, e allora un po' di agency c'è, "ma è normalissimo il caso in cui i ruoli meccanici sono prescritti".

Un vaso di Pandora: l'agency emergente. Il perché apre una questione che il docente rimanda ("un vaso di Pandora che non apriamo adesso"): quella dell'**agency emergente** e **discreta**. L'esempio lampante: in un gioco da tavolo le meccaniche sono tutte prescritte, il regolamento indica tutte le azioni meccaniche possibili. In un gioco di ruolo, invece, si può descrivere come il personaggio interagisce con qualcosa in un modo mai codificato dalle regole; per questo i giochi di ruolo hanno spesso un aspetto emergente dalla narrazione, nella gestione degli imprevisti e dei conflitti. "Ne parliamo più avanti, quasi sicuramente, perché tornerà".

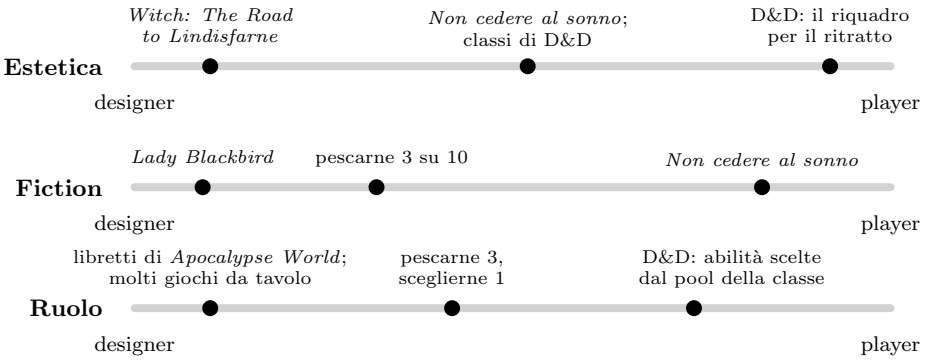


Figure 2.4.5: I tre slider "chi decide chi siamo", con gli esempi della lezione. Le posizioni sono qualitative e servono solo a ricordare che si tratta di cursori, non di interruttori: quasi ogni gioco sta in una via di mezzo.

2.4.7.4 Pro e contro

"Non ci sono risposte giuste o sbagliate": dipende dal gioco che si vuole fare e, soprattutto, da che cosa servono i personaggi. Pro e contro del lasciare la decisione al designer?

- **Contro**: lascia meno libertà a chi gioca, "ovviamente".
- **Pro**: tutto è più **prevedibile**, e quindi più gestibile. Se il gioco dice "giocate un cavaliere, un ranger e un ladro", si sa già che cosa aspettarsi da quel gruppo. Se invece c'è libertà, e uno fa "un alchimista tecnomante", un altro "un empate telecinetico" e un terzo "un mutaforma che in realtà è una pianta", è molto difficile controllare il tema del gioco e l'esperienza che si vuole far arrivare al tavolo. Prescrivendo i personaggi, chi fa design ha più controllo sugli aspetti di world building e di narrazione, su chi è in scena e in gioco.
- Anche la **coerenza**, aggiunge uno studente, è più facile da garantire. "Certamente".

Ma non è sempre meglio: un gioco può avere punti tematici che non dipendono da quali personaggi si giocano, perché passano attraverso altre cose (anche qui, il docente promette un esempio concreto a fine lezione).

Approfondimento: l'avatar ideale e la lavagna bianca. Schell, parlando dell'avatar, descrive due tipi di personaggio in cui chi gioca si proietta bene: la *forma ideale*, ciò che si è sempre voluto essere (il guerriero possente, la spia irresistibile), e la *lavagna bianca*, il personaggio "iconico" alla Scott McCloud: meno dettagli ha, più spazio lascia alla proiezione di chi gioca. Le due cose spesso si mescolano (Spider-Man è una forma ideale, ma la maschera lo rende una lavagna bianca). È un altro modo di guardare lo slider dell'estetica: il riquadro vuoto della scheda di *D&D* è una lavagna bianca che chi gioca riempie, il ritratto di *Witch* è un dettaglio che chi gioca deve accogliere. Schell nota anche che i sistemi che mettono la foto del giocatore sull'avatar non attecchiscono mai, perché "le persone non giocano per essere se stesse, ma per essere le persone che vorrebbero essere".

2.4.8 Spettatori, componenti, NPC e arbitri

"Che cos'altro possiamo essere?" Un gioco non ha solo personaggi, ruoli e player. La slide ne elenca altre quattro figure (tabella 2.4.2).

Spettatori. Spesso non li si considera, perché è difficile che ci siano spettatori in un gioco di ruolo da tavolo o in un gioco da tavolo. Ma in un **gioco pervasivo urbano** ci sono tante persone in città che ci guardano, che non stanno giocando, che non sono personaggi, né giocanti né non giocanti: sono **spettatori esterni**. Se il gioco interagisce con queste persone, bisogna considerare *come* lo fa. L'esempio: si prenda il LARP investigativo noir anni Quaranta, ma come *pervasive urban LARP*, in città. C'è un omicidio, e quattro persone, nei panni degli investigatori, transennano un'area in piazza. "Che cosa succede in quel momento, in quella piazza?" Attenzione a che cosa si mette nel proprio gioco, "perché può avere delle conseguenze, come tutto il resto".

Componenti. "A livello etico non è mai bene pensare alle persone come componenti, però è possibile", e tutti ne hanno almeno un esempio: gli **scacchi viventi**. Non due persone che giocano a scacchi al parco, ma le partite in cui ogni persona è un pezzo. Quelle persone non hanno agency, non possono muoversi dove vogliono quando vogliono: in quel momento sono componenti di gioco.

NPC. In un LARP grosso, da quattrocento persone, servono figure che siano personaggi gestiti da chi facilita, per esempio per la **gestione attiva** dell'avventura: qualcuno deve fare un nobile particolare, un mercante. Quei personaggi non sono giocanti, non hanno agency: hanno uno **scopo di facilitazione**, "l'unico scopo di far avanzare la gestione del gioco". Per questo non richiedono lo stesso tipo di progettazione: un personaggio giocabile e un NPC rispondono a domande completamente diverse. L'NPC risponde alla domanda **"come faccio, diegeticamente, a far sì che alcune cose di trama succedano in gioco?"**. Può servire anche nei giochi pervasivi.

Uno studente porta un esempio vissuto: una **caccia al tesoro a Venezia**, in cui a un certo punto si arrivava su una piattaforma dove c'era un pittore che fingeva di dipingere al cavalletto, con l'unico scopo di consegnare a chi arrivava un foglio necessario per proseguire. "Esattamente, è un NPC". Chi interpreta quel pittore non lo gioca come si gioca un personaggio: è "estremamente poco giocabile". L'NPC ha obiettivi molto precisi, ma sono obiettivi **meta**: non pensa a che cosa vuole il suo personaggio, quali spinte e attività ha, ma "devo assicurarmi che chi parla con me riceva questa busta", "devo assicurarmi che, alle X, qualcuno provi ad assassinare il re".

Arbitri. In un gioco pervasivo o in uno sport ci possono essere figure che non sono NPC né giocanti, ma sono lì per assicurarsi che tutto rientri nelle **regole**.

E altri ancora. Viste le mille ibridazioni, possono esistere ruoli non ancora codificati. "Il game design è in evoluzione, non abbiamo la verità in tasca". Non esiste una stele di Rosetta in cui sia scritto con precisione che cos'è il game design dalla A alla Z.

Figura	Agency	A che cosa serve	Esempi citati a lezione
Personaggio giocabile	piena	giocare: spinte, attività, relazioni	Joey Doe
Spettatore	nessuna nel gioco	non serve al gioco, ma il gioco lo coinvolge	i passanti in piazza durante un LARP urbano
Componente	nessuna	è un pezzo del sistema	gli scacchi viventi
NPC	minima, meta	far succedere diegeticamente cose di trama; facilitare	il nobile o il mercante del LARP da 400 persone; il pittore della caccia al tesoro a Venezia
Arbitro	sulle regole	far rispettare le regole	giochi pervasivi, sport

Table 2.4.2: Le figure che possono abitare un gioco oltre ai personaggi giocabili. La lista non è chiusa: "il game design è in evoluzione".

Approfondimento: la funzione prima del personaggio. La domanda che il docente attribuisce all'NPC ("come faccio a far succedere questa cosa in gioco?") è la stessa da cui parte la *Lens of Character Function* di Schell: prima si elencano le funzioni di cui il gioco ha bisogno (chi spiega, chi consegna, chi fa da ostacolo, chi va salvato), poi si vede quali personaggi le possono coprire, eventualmente accorpandone più d'una in uno stesso personaggio. Un NPC ben progettato è, in questo senso, una funzione con un costume.

2.4.9 Lo spazio dei personaggi

Riprendendo la connettività: i personaggi non hanno solo relazioni. "Non esistono in un vacuum, non è che esistono solo in una scatola nera con niente attorno": di solito **abitano uno spazio**. E lo spazio, "vi sorprenderà sapere", è un'**area progettabile**: in fase di game design lo si può progettare e pensare, e anche lo spazio informa su **che cosa può fare il personaggio**. Il docente lo mostra con una serie di immagini, senza dire di quali giochi siano, "perché se no vi do delle informazioni meccaniche".

Il dungeon. Che cos'è? Una mappa di gioco, "un dungeon". Che cosa facciamo nei dungeon? Esploriamo; combattiamo, se ci sono mostri; facciamo scattare le trappole; cerchiamo soldi, cerchiamo *loot*; interagiamo con tutto quello che c'è; ci nascondiamo. L'ambiente ci ha appena dato **chiarezza** sulle cose che il personaggio può fare: "l'ambiente ci informa su che cosa facciamo".

Il deserto. E qui? Camminare, certo. E cercare l'**acqua**. Perché l'acqua qui, e non nel dungeon, dove pure si potrebbe avere sete? Perché il deserto comunica la necessità di **sopravvivere**: si è esposti al calore.

Anche nel dungeon si è in pericolo, ma i pericoli sono diversi: là trappole e combattimento, qui bere e mangiare.

Il villaggio. Qui viene da rilassarsi, dice una studentessa: è un ambiente sereno, bucolico. Che cosa facciamo? "Viviamo". Che cosa vuol dire? Con chi parliamo? Magari abbiamo una famiglia; perché nelle case vivono le persone. Questo tipo di clima e di ambiente dà informazioni su che cosa fa il personaggio. Joey Doe in una via buia, di notte, con un cadavere per terra, farà certe cose; Joey Doe al pub *Il Barile* a fine giornata ne farà altre. "È sempre Joey Doe": sappiamo già che cosa *vogliamo* fare con lei, ma ciò che *possiamo* fare cambia. Nessuno, qui, dirà "evito trappole e combatto orchi sottoterra", così come nel villaggio nessuno dirà "cerco acqua, cerco cibo". L'ambiente è una fonte di **interazione**, di **conflitto**, spesso di **ostacoli**.

Il Titanic. Cerchiamo una via di fuga; "siamo sul Titanic". Che priorità avrà il personaggio? Sopravvivere. Ma anche nel deserto si sopravvive: che cosa cambia? Qui c'è un **limite di tempo**: in un'immagine l'acqua sta salendo, nell'altra la nave affonda. "Io però non vi ho detto niente meccanicamente": siete voi a vedere l'acqua. Perché? Perché, come nella lettura dei personaggi, anche nella lettura degli ambienti agisce il nostro **costruttivismo personale**: valutiamo ciò che vediamo attraverso le nostre esperienze passate. Non perché si sia vissuto il Titanic, ovviamente, ma perché si sono già esperiti tonnellate di media, racconti, storie, mondi, giochi e universi su certi concetti. Se vediamo un deserto sappiamo più o meno che cosa ci si può fare; se vediamo una nave così, sappiamo che c'è il tempo, l'acqua che sale, la pressione, la necessità di fuggire. E questo dà **informazioni meccaniche**.

E se Joey Doe fosse su quella nave, e la nave stesse affondando, ma ci fosse anche stato un omicidio? "Stiamo già avendo tanta roba per rendere giocabile il nostro personaggio". A che cosa dare priorità? È anche ciò che succede spesso nei giochi di ruolo: lo stesso personaggio, in due partite diverse, può essere giocato in modo molto diverso. In una partita Joey Doe dà priorità all'omicidio, a rischio di restare sul Titanic finché non affonda; in un'altra dice "dell'omicidio non mi importa niente, io torno a casa: scialuppa e via".

La metropoli. A sinistra, "farei shopping". E poi, dice Lorenzo, interagiamo con le persone: potrebbero darci informazioni, o potremmo comprare qualcosa che serve al personaggio. Che cosa cambia rispetto al villaggio? Nel villaggio si è in un ambiente familiare, dove si conoscono almeno alcune persone, "ci dà l'idea di essere a casa"; qui l'ambiente è conosciuto lo stesso, ma si interagisce con tantissime persone che non si conoscono. Nessuno dei due fa pensare a pericoli, ad acqua e cibo o a dover picchiare qualcuno per il loot, ma sono **ambienti di interazione sociale molto diversi**: le metropoli sono "esplosive" in termini di quantità di persone, il piccolo villaggio dà un senso bucolico, di relax, "vibes completamente diverse". Anche l'**approccio** cambia: un omicidio in un paese di dieci persone o nel centro di Tokyo vuol dire una **scala** dell'indagine completamente diversa.

La guerra. Ultimo esempio: che cosa farà il personaggio? "Combatte". O magari, guardando la prima immagine, salva dei civili e li protegge. E qui nasce un'altra domanda: **chi è il nostro personaggio?** Lo stesso ambiente può avere **affordance** estremamente diverse in base a chi siamo. Essere lì come un soldato americano, o come una persona che vive in quel paese da quando è nata: che cosa ci dice l'ambiente, e che tipo di agency ci fa esprimere?

2.4.9.1 Personaggi non umani

***Vast*: si può giocare la caverna.** Pensate alle **fonti di asimmetria meccanica** che un ambiente può dare. Il docente cita *Vast*, gioco da tavolo asimmetrico[17]: una persona gioca una specie di ladro, un cavaliere, che deve entrare nella caverna per rubare; un'altra gioca il gruppo di **goblin** che ci vive e che vuole eliminarlo; un'altra ancora un grosso **drago** che vive nella caverna, che mangia anche i goblin e che non vuole che gli si rubi niente. E una quarta persona **gioca la caverna**, e la caverna può vincere.

[17] *Vast: The Crystal Caverns* (2016), di Leder Games, l'editore che in seguito pubblicherà *Root*.

Ambiente	Che cosa suggerisce di fare	Che cosa comunica
Dungeon	esplorare, combattere, evitare trappole, cercare loot, nascondersi	pericolo: trappole e mostri
Deserto	camminare, cercare acqua e cibo	pericolo: la sopravvivenza
Villaggio	vivere, rilassarsi, parlare con la famiglia e i vicini	casa, relazioni note, vibes bucoliche
Titanic	cercare una via di fuga, sopravvivere; e, se c'è un omicidio, scegliere le priorità	urgenza: il tempo, l'acqua che sale
Metropoli	fare shopping, interagire con molte persone, raccogliere informazioni	interazione sociale con sconosciuti; scala enorme
Guerra	combattere, oppure salvare e proteggere civili	dipende da chi siamo: soldato straniero o abitante

Table 2.4.3: Gli ambienti mostrati a lezione e le letture della classe. Lo spazio dà chiarezza su che cosa il personaggio può fare, prima ancora di qualsiasi regola.

Salvage Crew. "Il vostro personaggio non deve necessariamente essere un essere umano". Uno dei romanzi più interessanti letti di recente dal docente è *Salvage Crew*, di uno scrittore dello Sri Lanka[18]. Fantascienza, futuro prossimo: mega-corporazioni mandano squadre sui pianeti a recuperare e riciclare materiali per guadagnarci. L'azienda è gestita da macchine, e gli esseri umani non possono fare i manager a meno di rinunciare al proprio corpo: il protagonista è un ex essere umano che vive ormai solo come coscienza, mandato sul pianeta con una squadra di umani da coordinare. E il protagonista *è* la capsula sparata sul pianeta con gli umani dentro: il romanzo si vive dal punto di vista dell'edificio che li contiene. Tutta la narrazione è ciò che lui vede e sente attraverso i sensori: "se le telecamere si rompono, lui non vede più". È comunque un personaggio estremamente interessante, "e vorrei un gioco su questa roba, adesso, se potessi".

Animali. Non limitatevi a pensare i personaggi solo come esseri umani, anche perché, storicamente, i giochi con protagonisti animali sono tantissimi: giochi di ruolo in cui si giocano cani, giochi da tavolo in cui si giocano animali, "ce ne sono a tonnellate".

2.4.10 Il primo task: una scheda personaggio

È il momento del task per la prossima lezione, e il docente lo introduce mettendo le mani avanti: lo guarderete e vi spaventerete; poi io vi rassicurerò, vi dirò che non è così complesso; ne parleremo, perché "sarà scritto in modo barocco e poco chiaro per colpa mia"; e alla fine sarà chiaro, e "mi consegnerete delle figate atomiche" che si vedranno ed elaboreranno insieme la volta dopo. "Il task è scritto comunque più chiaro di *Joey Doe, detective*. Questo ve lo posso assicurare".

La consegna. Il task è per il **21 settembre**, tra tre giorni. Le slide, "questa volta davvero", saranno caricate nella cartella subito dopo la lezione. La richiesta:

- creare una **scheda personaggio** in un **documento a testa**, con il proprio nome, nelle cartelle su Drive;
- **massimo una pagina**: "non voglio cose più lunghe di una pagina"; più corte, quanto si vuole;
- la scheda è pensata per un **LARP fittizio**, ambientato in una **serata di gala nella Milano contemporanea**, con elementi **urban fantasy**. "Non vado oltre nel taglio, volutamente";
- l'obiettivo: una scheda **chiara**, **connessa**, con **spinta** e con **attività**;

[18] Yudhanjaya Wijeratne (2020). Il riassunto è quello che il docente fa a lezione, "senza spoiler".

- *come* scriverla lo decide ciascuno, da solo: "non voglio che vi parliate a vicenda per dire qual è il modo migliore di scriverla". Si sono visti gli esempi di Joey Doe, se ne possono pensare mille altri, mischiarli, ibridarli.

La connettività, invece, è strutturata. La classe è di **nove persone**. Ciascuno deve avere **tre cerchie** (**amici**, **famiglia**, **lavoro**), ognuna formata dal proprio personaggio e da **altri due** personaggi dei compagni: gruppi da tre. Nella scheda, al personaggio a cui si è dato un nome e "magari due informazioni su chi è", vanno elencate queste connessioni, con i **nomi dei personaggi** dei compagni. Quindi il personaggio conosce **sei** altri personaggi; e, su nove, ce ne sono **due** che non ha mai visto né sentito (figura 2.4.6).

Un esempio passo per passo. Uno studente ammette di essersi perso, e il docente fa un esempio. Si scrivono due righe su chi si è (nome, cognome eccetera) e ci si organizza. Mettiamo che Gianmarco voglia fare il barista del pub *Il Barile*: scrive nel gruppo della classe chi vuole essere connesso a lui per lavoro. Magari un personaggio è chi porta la fornitura di birre con il camion, un altro chi pulisce il locale dopo la nottata: il gruppo del lavoro lo fa, per esempio, con Lorenzo e Tancredi. Poi gli serve un gruppo di amici: chi sono gli amici del barista? Magari quello con cui gioca a bocce e l'arbitro delle partite di bocce: il personaggio di Angelo è l'amico, quello di Tiziana l'arbitro (che magari arbitra nel tempo libero, e di lavoro fa altro). Infine un terzo gruppo, la famiglia, con altre due persone ancora, magari Rita e Nicola: il personaggio di Rita è sua zia, quello di Nicola sua figlia. Alla fine, nella scheda ci sono sei personaggi dei compagni.

"Però non li conosciamo", obietta qualcuno. "Ci dovete parlare, anzi: come fate a sapere chi sono se non ci parlate?" Basta sapere il nome o bisogna conoscerne le caratteristiche? Sono gruppi di connettività: se si hanno due colleghi si saprà chi sono, "magari non sai tutto", non si conoscono i loro gruppi di amicizia o di famiglia, le loro spinte e attività, "ma sai sicuramente che è tuo figlio o tuo collega".

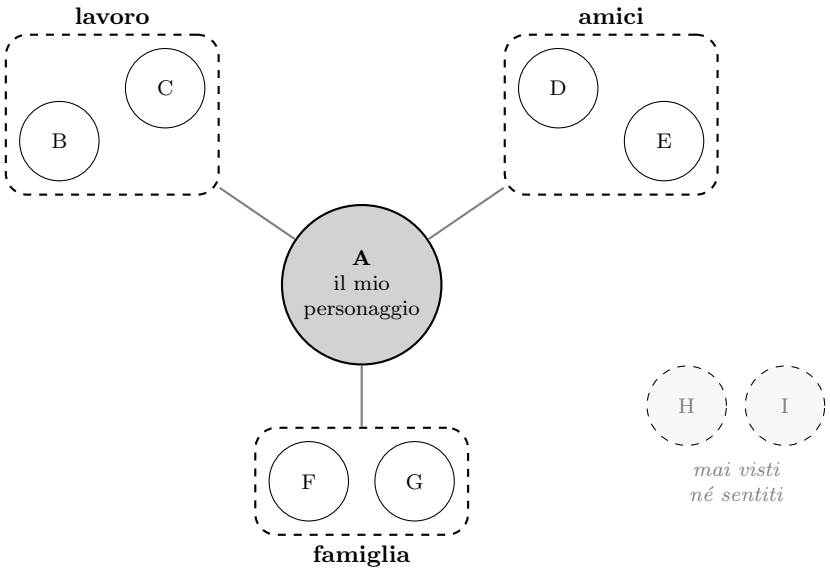


Figure 2.4.6: La struttura di connettività chiesta dal task, vista da uno dei nove studenti (A). Tre cerchie mutuamente esclusive da tre persone ciascuna: il proprio personaggio più due compagni. Sei personaggi noti, due sconosciuti. In un LARP da duecento persone, avverte il docente, ci sarebbero più gruppi, con sottogruppi.

Le schede dell'anno scorso. Il docente mostra alcune schede delle edizioni precedenti, "con il trucco" che, se le si guardano, non si può fare niente di identico. Una è di una pagina, con alcune parole in grassetto per renderle più chiare e immediate e per lasciar intuire che cosa fossero meccanicamente; elenca spinte e attività, e nelle cerchie di famiglia, amici e lavoro nomina altri personaggi (Jessica Crepuscolo, Giacomo d'Avanzo, Chris Betty...). Un'altra, quella di Chris Colombo, uno degli amici di Innocenzo d'Avanzo, è pensata

come una **brochure fronte e retro**, con famiglia, amicizia, lavoro e le proprie spinte. Un'altra ancora, Cristiano Colombo, ha caratteristiche, *tag*, una sezione "tu chi sei", i gruppi e una serie di obiettivi[19]. "Io non voglio che voi copiate questi personaggi": i vostri personaggi sono frutto vostro, ma anche **della vostra interazione con le altre persone della classe**. Spinte, attività e chi siete lo decidete completamente voi, ma verrà naturale costruire i gruppi anche in base alle persone che ne fanno parte.

La consegna, letteralmente. La scadenza è **la mattina della lezione**, anzi il giorno stesso, "anche cinque minuti prima": le schede si guarderanno insieme all'inizio della lezione del 21. Il docente ha appena creato, dentro la cartella dei task del Drive di Analog, una cartella **"Personaggi giocabili"**: lì ciascuno carica **un singolo documento**, non una cartella.

Gruppi mutuamente esclusivi. Angelo chiede se qualcuno possa appartenere a più gruppi. No: se Gianmarco ha il personaggio di Angelo tra gli amici, il personaggio di Angelo non può essere anche tra i colleghi o in famiglia. "I gruppi sono, in questo caso, mutuamente esclusivi", perché la classe è di nove persone. In un LARP da duecento persone ci sarebbero gruppi con sottogruppi: in quelli più grossi può esserci un dipartimento di trenta persone, con un ufficio A e un ufficio B da dieci persone ciascuno.

A che cosa servono i gruppi? Qualcuno non ne capisce il senso. Servono a sapere chi sono gli altri personaggi della propria scheda: non si vogliono nominare persone a caso, ma i personaggi giocabili che altri hanno creato. Ma, insiste lo studente, i gruppi sono "relativi": è tutto un intreccio, non si fanno fisicamente tre gruppi; ciascuno sa di doversi rapportare con due personaggi di un tipo, due di un altro e due di un altro. Il docente spiega perché bisogna comunque parlarsi: per avere **relazioni coerenti**. Se Tiziana dice "il personaggio di Gianmarco è mio fratello" e i due non si parlano, Gianmarco potrebbe scrivere "il personaggio di Tiziana è mia figlia". Su come organizzarsi a livello logistico, i docenti non dicono niente, "per mille motivi, e ne parleremo il 21": dicono che cosa vogliono ottenere. Call su Discord in nove? "Top". Ci si sente separatamente, in modo asincrono? "Benissimo". Una sola regola: **non si organizzano i gruppi durante la lezione, né prima delle 20 di stasera**.

Strumenti e forma. Si può usare qualsiasi strumento, Word, Canva, "purché ci sia un documento nella cartella Drive". La struttura della scheda è libera: font, colori, grassetto, corsivo, tabelle o no, elenchi. "Libertà totale". Ma **queste cose ci devono essere**, e saranno quelle valutate: il personaggio sarà letto attraverso il filtro dei quattro pilastri. Leggendolo è chiaro come deve essere giocato; fornisce informazioni chiare sulle sue connessioni; ha motivazioni e obiettivi a breve e lungo termine; ha cose che può e deve fare. E attenzione allo **spazio**: non si gioca in astratto, ma a una serata di gala a Milano con elementi urban fantasy.

Il bonus: un potere sovrannaturale. Chi vuole può dare al personaggio un **potere**, "anzi, bonus". Non bonus di voto: "io sono anti voto proprio"; su questo task non ci sarà un voto. Chi "si sente particolarmente frizzante" può ragionare su una **meccanica** legata a un potere sovrannaturale; ma deve dire *meccanicamente* che cosa fa, ricordando che è un LARP: volare, per esempio, è difficile fisicamente. L'esempio, che ovviamente non si potrà usare: un personaggio **telepate** può avere una metatecnica come il *ping the glass* della lezione precedente, o una variante: **la mano sulla spalla**. Se il mio personaggio ti mette una mano sulla spalla, tu mi devi dire che cosa stai pensando.

È il momento in cui Rita deve lasciare la lezione: "nessun problema, la lezione è registrata". Per le domande, ricorda il docente, si può scrivere anche via email o Telegram.

Fin dove spingere un potere. Uno studente, che gioca di ruolo da tavolo ma non ha mai giocato un LARP, chiede quali siano i limiti effettivi: nei giochi da tavolo ci sono regole a cui chiunque può attingere, mentre qui non c'è un arbitro definito dal manuale. Se definisse un potere come "se ottengo il tuo nome e cognome, posso chiederti un favore non dannoso per te", quanto andrebbe a intaccare l'agency di un altro giocatore? Per questo task si può fare, risponde il docente, perché qui **il setting è un MacGuffin**: due righe per dare contesto, "il LARP non esiste nella pratica". Va bene tutto ciò che non crea problemi

[19] I nomi dei personaggi sono riportati come nella trascrizione.

diegetici (come il volo). "Però ricordiamolo: **sono meccaniche, non sono *wishful thinking***". Si deve poter comunicare all'altra persona che si sta facendo quella cosa: il *ping the glass* funziona perché crea un'**interazione specifica e regolata**. Se ne parlerà più nel dettaglio nella lezione sul progettare emozioni.

La seconda pausa. Il docente carica subito le slide nella cartella, "così le avete, non come la volta scorsa", e propone una pausa: sono le 19:08, si torna alle 19:15.

2.4.11 Le domande sul task

Al ritorno, si riparte dal task. Nessuna domanda? Allora il docente apre la fine della lezione alle domande aggiuntive, compresi i casi studio dell'inizio; Gianmarco non ha altro da aggiungere. "Siamo ammutoliti dal task? Vi ho fatto paura col task?" Una studentessa ammette di essere un po' preoccupata per l'organizzazione: bisogna trovare un canale. Il docente la rassicura: è il primo task che si dà ogni anno, proprio perché la classe si conosca un po', e l'organizzazione "è l'ostacolo più grosso". Scrivere la scheda, in sé, è un lavoro da un quarto d'ora.

Niente AI. L'estetica della scheda non è troppo importante: "vi lascio agency totale, apposta", e negli anni si sono viste cose molto interessanti. Ma una cosa, già detta da Daniele nella lezione introduttiva: **non consegnate cose fatte con l'AI**. "Se vedo anche solo un personaggio disegnato con l'AI, vomito sulla vostra scheda". Chi vuole disegnarlo lo faccia a matita, anche male ("io disegno malissimo"): non è un task che valuta le capacità di disegno. Il task è esattamente quello che c'è scritto; il resto è un di più, che può anche diventare parte del progetto. Ma la **cura**, nel game design, non è solo fare esattamente quello che viene chiesto: è anche metterci il proprio tempo, capire che cosa si vuole fare.

Un personaggio che vorreste giocare. "Una cosa che vi chiedo, però, vi giuro": fate **un personaggio che vorreste giocare**. Non un personaggio "solo per fare il task". Anche senza aver mai giocato un LARP, la scheda deve far dire, a chi la legge: questa cosa da giocare dev'essere estremamente interessante. "Perché se voi create cose che nemmeno voi volete giocare, la possibilità che vogliano giocarle altre persone è molto bassa". E le domande nascoste vanno tirate fuori, "perché poi diventano mostri giganti che vi rodono dall'interno", e il task sembra una cosa titanica; invece è "un piccolo micetto spaventato".

I poteri degli altri. Se ognuno crea un'abilità per il proprio personaggio, come si fa a saperlo? Ci si può organizzare anche su quello, oltre che sulla connettività. La vera domanda è: **il vostro personaggio sa che cosa può fare l'altro personaggio?** Magari il gruppo di lavoro è "siamo cacciatori di vampiri", e allora ci si conosce le abilità; magari è "siamo colleghi in ufficio", e uno dei colleghi è un vampiro e gli altri non lo sanno. La scheda **non è onnisciente**: la legge e la gioca chi interpreterà quel personaggio, e la domanda è se abbia senso che sappia certe cose. Si è visto con Joey Doe: alcune schede nascondono informazioni, altre danno fatti, altre hanno un punto di vista che può anche **mentire**. Va benissimo che Tancredi scriva "il personaggio di Gianmarco è il mio migliore amico per sempre" e che sulla scheda di Gianmarco ci sia "Tancredi è uno scocciatore che mi segue sempre": la **collisione** e l'**asimmetria di informazioni** fanno parte di come si costruiscono i personaggi. Basta mettere ciò che ha senso: se il personaggio di Tancredi non ha motivo di sapere che quello di Gianmarco legge nel pensiero, lo scriverà solo la scheda di Gianmarco.

Non tutti hanno la magia. Il docente potrebbe fare altri esempi di poteri, "però poi vi impedisco di usarli, quindi a vostro rischio e pericolo". "No, è meglio non saperlo". Del resto il potere è davvero un di più: il setting è urban fantasy, ma "non è che in un setting urban fantasy tutti hanno la magia". Ci sarà chi serve il caffè, chi fa le fotocopie in ufficio, chi fa il tassista.

Fate cose che vi interessano. Vale per questo task, "ma è ancora più vero per quelli dopo": "questi non sono compiti a casa" per mettere una spunta su un Excel. Il punto è imparare pipeline e framework. "Non dovete pensare a cosa piace a Ruggero, a che cosa fare per prendere un voto alto: buttate questo pensiero dalla finestra". Le persone possono voler giocare cose diverse, come si è visto con *Fat Man Down* e altri

giochi intensi: il personaggio non deve per forza essere felice da giocare, e se c'è varietà tra i personaggi, tanto meglio.

Animaleschi, orchi, normali. Una domanda "velocissima": i personaggi possono essere di categorie diverse, uno più animalesco, uno più da orco, uno più normale? Sì, ma è un LARP: bisogna dire a chi gioca **che cosa fa** con questa cosa. Si può indicare come vestirsi (abiti A, B, C), o "metti due orecchie da gatto". I LARP hanno le loro particolarità: alcune schede indicano un'età, e non è detto che chi gioca abbia l'età del personaggio ("hai 50 anni", e magari se ne hanno 36); lo stesso vale per il genere del personaggio, e per l'estetica. Ma una cosa che di solito non si fa nelle schede di LARP è descrivere l'**aspetto morfologico del corpo**. Sui vestiti si ha molta agency: si possono cambiare, si possono mettere occhiali da sole, un cappello, un accappatoio, una camicia, andare a petto nudo; non si può cambiare il numero di braccia o la faccia. Con un asterisco, perché ci sono generi in cui si può fare: ma in un LARP ci sono requisiti fisici dati dal fatto che si fa un **embodiment** del personaggio, e il corpo del personaggio e quello di chi gioca condividono lo stesso spazio fisico. E di nuovo: non dimenticate il **luogo**, perché spinte e attività sono legate anche a quello.

Niente spettatori. Un'altra domanda nascosta: il personaggio esiste in una situazione pervasiva, con persone non giocanti? "No": per semplicità, è un **LARP ad alto budget** che ha noleggiato un ambiente di alta classe, e così si toglie l'anomalia. Sarebbe interessantissimo un gioco così, "ma nel primo task non voglio darvi anche il come interfacciarvi con le persone non giocanti", che è un layer di complessità in più. E restano i limiti fisici: niente personaggio che vola, niente mezzo pipistrello. "Voi qui siete le persone che fanno design": il personaggio non lo giocherete voi, andrà a un'altra persona che deve capire come giocarlo.

Fuori contesto, ma con un alibi. Si può costruire un personaggio volutamente fuori contesto? Lo studente ne ha in mente uno "che è una rovina per chiunque lo voglia giocare": un personaggio in accappatoio a un gala. Si può fare **se ha senso che ci sia**: l'importante è che sia giocabile. La questione è che cosa ha senso che rimanga in gioco per la durata della serata. Se il personaggio è il nemico numero uno dell'Interpol, che tutti arresterebbero a vista, non ha senso in questo scenario: verrebbe preso e ingabbiato al minuto due. Servono **motivazioni per cui si è lì e non si viene arrestati subito**. Non bisogna per forza essere un nobile aristocratico: si può essere l'addetto alle pulizie, chi è stato chiamato a servire al bar, o un ladro che sta svaligiando una cassaforte nella stanza accanto, travestito da ospite a un ballo in maschera. È un'abilità implicita nel ruolo di game designer: **dare giustificazioni, alibi, motivazioni**. "Ricordati che il tuo personaggio è comunque nelle sue cerchie".

Il personaggio nudo, e il pazzo. L'estremo che lo studente vuole esplorare è un personaggio nudo che, con un potere, convince tutti con la parola di non esserlo. "Se vuoi, sì, però cerchiamo di fare personaggi giocabili". Il docente lo collega a un vecchio vizio dei giochi di ruolo anni Novanta, tipo *Vampire*: il personaggio che è "il pazzo". A parte che è un tropo "perniciosissimo", in termini di giocabilità dopo quindici minuti ci si è già stancati di giocarlo, perché un personaggio fuori di testa non ha spinte che gli permettano di pianificare e agire a breve e lungo termine: non ha un rigore logico. Lo stesso vale per il **personaggio misterioso** che sta in un angolo del gala e non deve parlare con nessuno: "terribile da progettare per questo gioco". "La cosa forse più importante di questo personaggio sono le sue **tre cerchie**": sono ciò che lo rende giocabile in termini di relazioni, "perché i LARP non sono giochi single player", sono giochi in cui la propria agency è condivisa con tutte le altre persone che giocano ("un pantano gigantesco, che non apriamo adesso"). Lo studente precisa che il suo personaggio non sarebbe fuori di testa: voleva solo capire fin dove spingersi. Si possono fare anche personaggi un po' *cringe*, un po' buffi: "non deve avere la serietà di un film di Nolan". Ci sono persone e mood diversi: Joey Doe, super seria e investigativa, e il barista completamente scanzonato che fa battute ogni tre secondi. E il setting non va progettato: "è uno spunto per i vostri personaggi".

La spinta può riguardare un altro personaggio? Angelo chiede se l'obiettivo del personaggio possa essere un altro personaggio: un sicario, per esempio, che deve uccidere qualcuno. "Sì". L'esempio del docente: il mio personaggio deve trovare un modo per restituire dei soldi al personaggio di Gianmarco, che magari è un criminale e gli ha detto "entro stasera voglio i soldi". È una bella spinta, che spinge a fare molte

cose che magari non si volevano fare: derubare qualcuno, ricattare qualcun altro, riscuotere un favore. Anzi, parte della giocabilità sta in come si leggono queste cose insieme: se devo dei soldi a Gianmarco, ma grazie alle mie cerchie un altro personaggio mi deve un favore, "di giocabilità ne ho già tantissima, stasera".

Il test del "che cosa faccio adesso?". Il criterio pratico: mettetevi nei panni del personaggio nella scena in cui è appena entrato al gala, e chiedetevi **che cosa faccio adesso?**. Se a questa domanda si hanno almeno **due o tre risposte possibili, in direzioni diverse**, il personaggio "per me è già bombissimo". Nell'esempio dei soldi:

- vado dal personaggio di Gianmarco a chiedere una proroga: stasera non ce la faccio, ma la settimana prossima sì, e provo a convincerlo;
- vado dall'amico che mi deve un favore: devo dei soldi a qualcuno di pericoloso entro stasera, dammi una mano, voglio quel favore adesso;
- vado dal sergente di polizia e gli chiedo di arrestare il personaggio di Gianmarco, che è un criminale: così il mio problema sparisce.

Una spinta vaga. Angelo prova a generalizzare: allora la spinta può essere molto vaga, tipo "voglio evitare che mi spezzino le rotule", e poi sta a me trovare il modo. Sì, ma in un gioco ben progettato, perché quella spinta funzioni, deve esistere un altro personaggio con la spinta opposta: *devo spezzare le rotule al personaggio di Angelo*. Se la spinta è "devo evitare l'arresto" e nessuno deve arrestarti, quella spinta non esiste. È di nuovo la slide: **le relazioni del vostro personaggio non esistono finché non vi comportate in modo da farle esistere**.

Quindici minuti di lavoro. Tiziana non ha altre domande: ascoltando le altre, è tutto abbastanza chiaro. Il docente sa che la lunghezza della slide del task ha intimorito, e che la parte centrale è logisticamente complessa. Ma il task in sé, anche per l'esperienza dei gruppi precedenti, prende **15-20 minuti al massimo** di lavoro sul documento. La logistica sta agli studenti, e serve anche ai docenti per capire con chi formare i gruppi. Le domande sul task si chiudono "per ora, non per sempre": se ne vengono in mente altre tra due ore, email o Telegram, "io vi rispondo lo stesso".

Approfondimento: un compito che si gioca solo insieme. Il task applica alla lettera un'idea che la lezione aveva messo in una slide: la connettività non si scrive, si negozia. Nessuna scheda è completa da sola, perché metà delle sue informazioni (chi sono i sei personaggi nominati, che cosa vogliono, che cosa sanno) sta nelle schede degli altri. È lo stesso principio della *Lens of the Character Web* di Schell, ma con una differenza importante: la rete non la disegna un solo autore, la tessono nove persone che si devono parlare. Ed è anche, dichiaratamente, uno strumento di osservazione: come gli studenti si organizzano dice ai docenti qualcosa su come lavoreranno in gruppo.

2.4.12 Le domande finali

Restano una ventina di minuti per le domande sul resto: la lezione precedente, i casi studio, il metagioco di Tancredi. "Abituatevi, perché le lezioni con me hanno sempre la parte di domande in fondo, e io divento super *cringe* se non ci sono domande". "Adesso sono curioso di vedere quanto puoi diventare cringe", ribatte uno studente.

2.4.12.1 Framework, dinamiche e playtest

Gianmarco condivide una considerazione sui framework visti finora, MDA, Mixing Desk e Player Involvement Model. L'MDA gli sembra "fondamentale, come fondamenta". La parte di **esperienza** la si coprirà con l'emotional design, che diventa "l'innesco di tutto quello che c'è prima"; la parte di **meccaniche**, in qualche modo, con gli elementi di gioco. Gli sembra un po' scoperta la parte di **dinamiche**, che però è importante.

Per il docente la parte delle dinamiche è "la parte principale della questione dei **playtest**". Le dinamiche sono l'impatto in tempo reale tra chi gioca e le meccaniche progettate, e quello è l'ambito del playtest. Un gioco può esistere anche se nessuno lo gioca o lo testa: si può scrivere un bellissimo regolamento, fare la scatola, i componenti, e il gioco esiste lo stesso; ma **le dinamiche non esistono**, perché nessuno lo sta giocando. Il playtest avrà una lezione dedicata, "ve lo assicuro".

Gianmarco aggiunge che gli è utile ragionare per **ipotesi**: quando testa, cerca di validare o no un'ipotesi fatta *ex ante*; gli mancava l'elemento di *che cosa* valutare nel playtest. E gli sembra che diversi **slider** del Mixing Desk siano elementi di dinamiche. Alcuni lo sono, concorda il docente: la gestione attiva o passiva, le meccaniche intrusive. Ma ci sono anche aspetti produttivi: ci sarà una lezione con **Beatrice** sulla progettualità e sulla gestione del progetto, che può aprire altri slider, legati per esempio a quanto tempo dedicare a certe parti della progettazione. Il docente si scusa di rimandare ("la risposta c'è, ma abbiamo una lezione apposta").

Il Mixing Desk per leggere un catalogo. Gianmarco vede il Mixing Desk come un ottimo strumento per modellare le dinamiche (pur avendo, confessa, "un problema di memoria con i nomi"). Il docente ricorda che è uno strumento molto flessibile: di progettazione, ma anche di **ricerca e sviluppo**. Si possono passare al filtro giochi già pubblicati, per esempio l'**intero catalogo di un editore**, per capire che cosa hanno in comune i suoi giochi secondo il Mixing Desk: se il proprio gioco ci rientra, o per avere le basi per dire "questo editore fa queste cose; se progetto un gioco di questo tipo ho più possibilità che lo pubblichi".

L'esempio è di prima mano: "noi", cioè **MS Edizioni**[20], non pubblichiamo giochi per due persone: tutti i giochi in catalogo arrivano ad almeno tre. Chi propone un gioco per due va già contro la linea editoriale. "Non è che i giochi da due siano più brutti": semplicemente il dipartimento commerciale gestisce altre cose, e in catalogo non c'è una nicchia per i giochi da due. Il settore, poi, è fatto di persone tendenzialmente amichevoli: quando arriva un prototipo valido, ma non adatto, il docente di solito consiglia a quali editori proporlo, e più volte i prototipi sono stati passati ad altri.

Gianmarco trova la cosa interessante: al di là dei casi noti (Leder Games fa giochi asimmetrici, "e lo sai") e delle macrocategorie, queste linee editoriali non sono palesi[21]. Arriverà una lezione "scivolosa", una delle preferite del docente: quella sul **paragame**, con informazioni importantissime proprio su questo ragionamento lato editori. Il docente controlla il calendario: la prossima lezione è **emotional design**, il 21; il paragame è previsto per il **9 ottobre**. "Per ora": alcune lezioni potrebbero spostarsi in base ai task.

Il corso non è un filtro. I task informano su ciò che gli studenti sanno fare in questo momento, "ma questo non è un filtro": il docente non vuole un corso da cui si esce bocciati o promossi, "non è il mio stile". Vuole che si esca con gli strumenti per fare i giochi che si vogliono fare, anche se nel corso, per ragioni di tempo e di logistica, non si potrà fare "un trading card game con 5.000 carte", anche se fosse il proprio sogno. Va interiorizzata l'idea che lo stesso strumento può non servire a due persone diverse, o servire in modo diverso. I primi task sono abbastanza uniformi come struttura, ma nel progetto lungo di gruppo i docenti si aspettano, come negli anni scorsi, progetti molto diversi: l'anno scorso, per esempio, sono arrivati un gioco di ruolo da tavolo e un card game. "Non vogliamo una cosa fatta con lo stampino": si lavori su qualcosa che piace, usandolo come campo di prova per le pipeline, i framework, il Mixing Desk, le aree progettabili, i personaggi giocabili.

2.4.12.2 *Vast*, Leder Games e le ristampe

Uno studente torna sul gioco in cui si gioca la caverna: gli interessa molto. È *Vast: The Crystal Caverns*, un gioco del 2016, ormai fuori produzione, "e vorrei che lo ripubblicassero": è dell'editore originale di *Root*, "il gioco asimmetrico prima di *Root*". Esiste anche una reimplementazione in cui, al posto della caverna, c'è una magione infestata (*Vast: The Mysterious Manor*, del 2019), e una persona gioca la magione. Il docente spera vivamente che lo ristampino, e "se decidono di pubblicarlo, io sono già lì per chiedergli di localizzarlo".

[20] La casa editrice per cui il docente lavora, come dice lui stesso rispondendo alla domanda "chi sono?".

[21] Qui la trascrizione riporta anche un esempio incomprensibile ("non mi aspetto da Minecraft che faccia Tommy Carter"), che non è stato possibile ricostruire.

Le ristampe centralizzate. Qualcuno chiede se all'epoca lo avesse localizzato MS Edizioni: forse sì, ma non loro. Il problema è che **Leder Games** ha una **gestione centralizzata delle ristampe**: chi vuole localizzare i suoi giochi in altre lingue non può farlo liberamente, ma solo quando Leder ristampa. È Leder ad avvisare: stiamo per ristampare *Root*, volete ristamparlo, quante copie? "Se perdi il treno, ciao". MS Edizioni ha localizzato con loro un gioco ("Aoi", nella trascrizione[22]) che è andato molto bene, esaurito subito; ma Leder non ha riaperto le ristampe per **otto mesi**, e per otto mesi le persone chiedevano dove trovarlo, e l'editore italiano, che voleva ristamparlo, non poteva. Lo fanno per creare scarsità? No: per **assicurarsi che i componenti siano identici in tutte le copie**. Altri editori se ne disinteressano, con altre conseguenze: a volte le edizioni locali sono migliori delle originali, come certe edizioni coreane *deluxe* dei giochi di Reiner Knizia, "incredibili". "È un mondo anche quello", e se ne parlerà in parte nella lezione sul paragame.

Come funziona la caverna. Un'ultima curiosità su *Vast*: come si gioca la caverna? Tendenzialmente la caverna gestisce le **tessere**: può creare e chiudere passaggi in cui si muovono i goblin, il cavaliere, il drago. Il suo obiettivo è fare in modo che nessuno degli altri vinca: se qualcuno sta vincendo, gioca per mettergli i bastoni tra le ruote. "Però la caverna può vincere". "Bellissimo". Su Vinted si trova qualche copia, ma essendo fuori produzione da anni le copie usate hanno "un costo che non ha senso". "Pazienza, ragazzi".

2.4.12.3 La chiusura

Mancano otto minuti, che il docente mette "nello schermo centrale". Si possono usare per cominciare a organizzarsi per il task? "Assolutamente no": "io sono molto permissivo, anche se non sembra, ma questo no". Rispetto alle regole, è rigoroso: "in questa casa non si giocano *home rules*, mai". Per lavorarci anche nel tempo libero, poi: se c'è da sistemare un gioco, "pagatemi e lo sistemiamo".

Nessun'altra domanda. "Non è possibile, sono un essere umano, anch'io avrò sbagliato qualcosa". Uno studente osserva che la lezione è molto pratica, e che certe cose diventano davvero visualizzabili solo quando si mettono le mani in pasta. Il docente è d'accordo: è lì che si capisce, e a quel punto ci sono le email sue, di Daniele e di Beatrice. Si scusa per il weekend di mezzo ("peggio per noi, ma peggio anche per voi"): sarà impegnato a una manifestazione[23], ma risponderà entro 24 ore. "Fate domande, vi prego, prima che sia troppo tardi". "Io ho il cervello in pappa", confessa qualcuno.

L'anticipazione: la prossima lezione è l'**emotional design**, "per me una lezione importantissima, veramente", con concetti molto forti che chiariranno molte cose dette finora, e con parti interattive che apriranno conversazioni. Ma prima, all'inizio, **20-30 minuti** per guardare insieme i task, con commenti e feedback. "Siete persone libere con agency", e il docente lascia andare la classe "in pace o in guerra, a seconda di come vi sentite". "Buon lavoro, ci vediamo lunedì". "Buon weekend e buon task".

Punti chiave del capitolo

- La **coesione tematico-meccanica** è un valore per il docente, ma non l'unico: i giochi astratti (Nikoli, lo Slitherlink) si apprezzano anche senza tema. Fallisce il gioco che si presenta come narrativo e non lo è; vince l'astratto che riesce comunque a comunicare un tema. I componenti vanno giudicati rispetto alle aspettative del genere (*Nemesis* contro *The Gallerist*) e al loro uso tematico.

- **Player** è la persona che gioca, fuori dal gioco; **character** è chi si è nell'universo narrativo (nome, background, storia); **role** è l'insieme di doveri, autorità e possibilità meccaniche. Il ruolo c'è quasi sempre (scacchi, *Pong*, *The Gallerist*), il personaggio no. L'agency passa dal ruolo; ma il ruolo da solo è freddo.

- Un personaggio non è giocabile solo perché qualcuno può interpretarlo (Aragorn che aspetta in taverna; i PNG "non giocabili" che qualcuno gioca). I **quattro pilastri** del personaggio giocabile sono **chiarezza**, **connettività**, **spinte** e **attività**, e valgono solo insieme.

[22] Il titolo non è ricostruibile con certezza dalla trascrizione; si tratta comunque di un gioco Leder Games.

[23] La trascrizione dice "c'è il to play questo weekend": probabilmente un evento o una fiera di giochi, non identificabile con certezza.

- **Chiarezza**: il personaggio dice come va giocato. Le sette schede di Joey Doe (etichetta, caratteristiche, seconda persona, prima persona, foto, oggetti, istruzioni di comportamento) danno informazioni diverse: la voce in prima persona permette punti di vista contrastanti; la foto dà mood e costume; i prop danno agency. Detective come personaggio o come ruolo? Più parole non vuol dire più chiarezza (Dogma 99).

- **Connettività**: niente lupi solitari. Di solito tre cerchie separate (lavoro, amicizia, famiglia), ognuna una direzione di gioco. Le relazioni **esistono solo se le giochi**, e solo se l'altro capo del filo è in gioco.

- **Spinte**: motivazioni e obiettivi a breve e lungo termine. **Attività**: ciò che si può e si deve fare per inseguirle. Una spinta irraggiungibile in gioco (il dittatore assente) è solo background. Il background extra è lecito (la camicia hawaiana), ma non rende più giocabili. I pilastri dialogano con il Player Involvement Model senza coincidere.

- **Chi decide chi siamo**: tre slider designer–player per **estetica** (*Witch* contro il riquadro di D&D), **fiction** (*Lady Blackbird* contro *Non cedere al sonno*) e **ruolo** (i libretti di *Apocalypse World* contro le abilità scelte in D&D). Prescrivere toglie libertà ma dà prevedibilità, controllo sul tema e coerenza.

- Oltre ai personaggi giocabili ci sono **spettatori** (i passanti di un LARP urbano), **componenti** (gli scacchi viventi), **NPC** (funzioni di facilitazione con obiettivi meta) e **arbitri**; e altri ruoli non ancora codificati.

- Lo **spazio** è un'area progettabile: dungeon, deserto, villaggio, Titanic, metropoli e guerra suggeriscono attività, pericoli, priorità e scale diverse, attraverso il costruttivismo personale. Le affordance dipendono da chi si è. I personaggi possono non essere umani (la caverna di *Vast*, *Salvage Crew*, gli animali).

- Il **task** (per il 21 settembre): una scheda di massimo una pagina per un LARP a una serata di gala nella Milano contemporanea urban fantasy; chiara, connessa, con spinta e attività; tre cerchie mutuamente esclusive con due compagni ciascuna (sei noti, due ignoti su nove); forma libera, niente AI; potere sovrannaturale facoltativo, ma come meccanica, non come *wishful thinking*.

- Il test pratico: appena entrati al gala, **"che cosa faccio adesso?"**. Due o tre risposte in direzioni diverse vogliono dire un personaggio giocabile. Il pazzo senza obiettivi e il misterioso nell'angolo non lo sono; serve sempre un alibi per essere lì.

- Il Mixing Desk serve anche alla **ricerca**: per esempio per leggere il catalogo di un editore. Le dinamiche esistono solo quando qualcuno gioca: sono il territorio del **playtest**. Il corso non è un filtro: fate personaggi, e giochi, che vorreste giocare.

Chapter 2.5

Progettare emozioni: il divertimento, il bleed e le meccaniche di sicurezza

Questa è la **quinta lezione** del corso di **Analog Game Design**, ed è quella che il docente aveva annunciato alla fine della precedente come "per me una lezione importantissima, veramente": la lezione sull'**emotional design**, o, come la chiamano le slide, sul **progettare emozioni**. È anche la lezione in cui il docente scioglie finalmente uno dei nodi lasciati aperti fin dal primo giorno: perché *divertimento* sia una delle parole che, in questo corso, non si possono usare. Dopo la spiegazione arrivano un piccolo gioco giocato dal vivo da uno studente (un "nanogioco" di paracadutismo che finisce male), il concetto di **bleed**, una lunga rassegna delle **meccaniche di sicurezza** nate nella scena del gioco di ruolo dal vivo nordico e, infine, il **secondo task**: scrivere un gioco di ruolo completo in **200 parole** che faccia provare un'emozione precisa *con le meccaniche*.

La lezione si svolge **online**, su Zoom, ed è tenuta ancora una volta da **Ruggero**[1]. Si colloca con ogni probabilità **lunedì 21 settembre 2026**[2], dalle 17 alle 20 come sempre, con **una sola pausa**, intorno alle 18:15. Gli studenti che intervengono, e che il capitolo nomina, sono **Tiziana**, **Rita**, **Giordano**, **Nicola**, **Angelo**, **Lorenzo**, **Tancredi** e **Gianmarco**, che arriva a lezione iniziata ("di solito il lunedì ha impegni fino alle cinque e mezza", ricorda qualcuno).

In coda alla stessa trascrizione c'è un secondo documento, di natura diversa: una **dispensa video** registrata a parte, qualche settimana dopo, da un altro docente del corso, dedicata alla **ricerca e sviluppo**: design thinking, Double Diamond, strumenti di fattibilità e di analisi del mercato, *brainstorming* strutturato, strumenti di convergenza, pitch deck. Il capitolo la riporta per intero nelle ultime sezioni (dalla 2.5.15 in poi), tenendola separata dalla lezione.

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi, soprattutto quelli dei giochi e quelli stranieri. *John Company* è per fortuna quasi sempre riconoscibile, ma *The Cost* diventa "The Coast", *Ars Amandi* "Lars Amandi", il *bleed* "grid", le *metatecniche* "beta tecniche" e "mega insicurezza", il *de-rolling* "deroga". Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante la pausa la trascrizione registra per decine di righe frasi senza senso ripetute in loop ("un mercato interno", "qual è la risposta che ci può essere?"), che il capitolo ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro e resta generico altrimenti.

Come leggere questo capitolo. La lezione ha un andamento particolare, che il docente stesso riconosce alla fine ("mi rendo conto, un po' strana"): parte da una domanda semplice (*che cosa prova chi gioca?*), la usa per smontare la parola *divertimento*, poi la mette alla prova facendo *provare* qualcosa a uno studente

[1] Come nelle lezioni precedenti, il docente non si nomina mai da solo; lo chiama per nome Tiziana durante le domande ("scusa, Ruggero"; la trascrizione dice anche "Rugger"). È lo stesso docente delle lezioni 3 e 4: parla dei propri LARP, cita *Bunker 101*, a cui ha partecipato, e alla fine rimanda alla lezione successiva "con me e con Beatrice".

[2] La data non è detta, ma è ricostruibile con sicurezza: alla fine della lezione precedente (venerdì 18) il docente aveva fissato la consegna del primo task "per il 21 settembre" e aveva annunciato per lunedì la lezione sull'*emotional design*; qui si comincia proprio guardando le schede consegnate, e il nuovo task è "per il 25".

davanti a tutti, e da lì passa a ciò che un designer deve fare quando un gioco fa provare emozioni forti: prendersi cura delle persone. Il filo conduttore è una frase che chiude la lezione: **le persone sono più importanti dei giochi**. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell e *A Theory of Fun for Game Design* di Raph Koster); sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione e della dispensa:

1. l'attesa: una domanda sulle **game jam** e su che cosa fare dell'idea nata in una jam (sezione 2.5.1);
2. la revisione delle **schede personaggio** del primo task, una per una (sezione 2.5.2);
3. **che cosa prova chi gioca**: emozioni diverse, contesti diversi (sezione 2.5.3);
4. il problema della parola **divertimento** (sezione 2.5.4);
5. i **giochi su temi difficili**, e la responsabilità di chi li fa (sezione 2.5.5);
6. il **nanogioco del paracadute** (sezione 2.5.6);
7. il **bleed** (sezione 2.5.7);
8. la **sicurezza** nei giochi: prima, durante e dopo (sezioni da 2.5.8 a 2.5.11);
9. il **200 Word RPG Challenge** e il **secondo task** (sezioni 2.5.12 e 2.5.13);
10. le **domande finali**: workshop e debriefing in concreto, il debriefing nei giochi da tavolo, le persone prima dei giochi (sezione 2.5.14);
11. la **dispensa** sulla ricerca e sviluppo: design thinking e Double Diamond, dati e insight, fattibilità, stakeholder e mercato, *brainstorming* strutturato, convergenza e pitch deck (sezioni da 2.5.15 a 2.5.20).

2.5.1 L'attesa: che cosa resta di una game jam

La registrazione si apre nel mezzo di una chiacchierata informale: qualcuno sta commentando a voce alta una partita, sperando di trovare la *Blue Map*, che "mi permetterebbe di vedere subito dove sono le stanze segrete senza sprecare bombe"[3]. Poi si comincia. Il docente chiede com'è andato il task: "un gran casino", risponde qualcuno. "A volte servono anche i gran casini per imparare qualcosa". Ha già visto la cartella delle consegne; c'è ancora un file chiamato "Presentazione senza titolo"[4] di cui vuole capire l'autore ("forse è di Tancredi, per esclusione"). Come sempre, nei primi minuti si aspettano le persone e si apre la parola.

L'idea della jam dopo la jam. Uno studente ha appena concluso una **game jam**: ci era entrato come game designer e ne è uscito, "per qualche motivo", come programmatore del gruppo. La sua domanda è generale: quanto bisogna **perseverare** con un'idea che tutto il gruppo ritiene valida, ma che nella jam non si è riusciti a realizzare ("al 100%, anzi neanche al 5%")? Vale la pena riprenderla e svilupparla più avanti, o lasciarla morire lì?

Le questioni correlate, risponde il docente, sono tantissime. La prima: il progetto nasce *per* la jam, come pretesto per mettersi al lavoro, o nasce da un'analisi del tipo "vogliamo creare questa cosa insieme, e siamo disposti a farlo anche dopo"? La seconda riguarda il lavoro con il gruppo. Una jam ha un limite di tempo molto forte, e quasi sempre si finisce per fare una versione molto ridotta dell'idea iniziale. Se però il gruppo ha lavorato bene, "nulla vi vieta di andare oltre la jam" e dire: il gioco sembra promettente, perché no? Andiamo avanti. "Specialmente in questa fase, in cui state ancora sperimentando molto". Bisogna però riconoscere quando quel "5%" è figlio di un **overscoping**: "vogliamo fare il nuovo grande [titolo enorme] e siamo in cinque" è un problema di ambizione, non di tempo.

[3] È con ogni probabilità *The Binding of Isaac*, in cui la *Blue Map* è un oggetto che rivela le stanze segrete; la trascrizione parla di "secret room" e di "cacche bomba". Il gioco non viene mai nominato.

[4] La trascrizione dice "Presenza senza titolo".

Lo studente precisa che il caso era l'opposto: un *platform* con meccaniche un po' diverse, sul tema dell'oro, con un personaggio alla Re Mida che trasforma i nemici in oro e li usa come piattaforme, con un potenziale anche per dei puzzle. Ma il gruppo è cambiato, e la domanda diventa un'altra: da neofita, conviene sperimentare con tanti progetti e tante persone, invece di fissarsi su un'idea?

Le jam, dice il docente, servono proprio a questo. Spesso permettono di **testare gruppi diversi**: ci si rende conto che con certe persone si lavora meglio, con altre peggio, con altre ancora benissimo ma solo su cose molto compatte e focalizzate. "Alla fine, le jam sono più un alibi per capire con quali persone lavori bene che una vera fonte di progetti commerciali completi". Anche i giochi nati da una jam che arrivano su Steam e fanno "un bel giro" sono quasi sempre frutto del **lavoro successivo**, che con la struttura della jam ha molto poco a che fare.

Che cos'è una game jam. Un altro studente chiede, "perdonate l'ignoranza", che cosa sia esattamente una jam[5]. Sono eventi che danno un tempo limitato (le più comuni durano **48 ore**, ma ce ne sono da 72 ore, o addirittura di settimane o mesi) per creare un progetto, di solito su un **tema** molto ampio e interpretabile. Non sono concorsi ("non vince nessuno, di solito"): sono un alibi per avere una spinta a creare qualcosa, magari sperimentando il lavoro di gruppo con persone che sono lì anche loro per imparare. E non sono eventi solo per game designer: si lavora in gruppi misti, con chi fa sound design, art, modelli 3D, programmazione. La più famosa è forse la **Global Game Jam**, che è sia digitale sia analogica ("si può partecipare anche facendo giochi da tavolo, giochi di ruolo"); c'è poi la **GMTK Game Jam**, per il digitale, e la piattaforma **itch.io**[6]. Di dove trovarle si parlerà nella lezione su ricerca e sviluppo.

Mancano ancora Gianmarco e un altro studente; si aspetta un paio di minuti, poi si parte dai task.

2.5.2 Le schede personaggio del primo task

Prima delle slide ("oggi ce ne sono un bel po'"), il docente vuole dare una rapida *overview* dei lavori consegnati. Il metodo è sempre lo stesso: chi presenta condivide lo schermo con il proprio documento e lo si guarda insieme; "passiamo tutti, quindi chi vuole andare prima è solo una questione di tempo, non è che qualcuno scappa da questo processo". Chi ha appena presentato sceglie chi viene dopo, così la persona successiva ha il tempo di prepararsi.

Una studentessa, prima di cominciare, racconta com'è andata dietro le quinte: è stato interessante lavorare insieme, soprattutto nell'**intrecciare i personaggi** e costruire le relazioni, al punto che ne è uscita "una specie di trama comune".

Ricordiamo il task (capitolo precedente): una scheda di **massimo una pagina** per un LARP ambientato a una **serata di gala** nella Milano contemporanea *urban fantasy*, con un personaggio chiaro, connesso, dotato di spinte e di attività, inserito in **tre cerchie** (famiglia, amici, lavoro) di due compagni ciascuna. Le schede, lette ad alta voce dal docente, compongono un cast fortemente intrecciato attorno alla stilista **Maria Bianca Giannini** e alla sua sfilata. Il capitolo le riassume nell'ordine in cui vengono presentate.

2.5.2.1 Tiziana: Lin Mezzacorona, la giornalista mezzelfa

Il primo turno è di Tiziana, dopo qualche minuto passato a capire come si condivide lo schermo su Zoom ("dovresti avere una serie di rettangoli con una freccia, con scritto *Share*"; schermo intero o singola finestra "è indifferente"). Il personaggio è **Lin Mezzacorona**[7]: una **mezzelfa** di 40 anni. Al gala va per farsi notare: la sua vanità la porta a desiderare di essere osservata con invidia, "e magari con un pizzico di desiderio". La scheda dice anche **come vestirsi**: un abito rosso fiammante, lungo e scollato (niente paillettes, che non le sono mai piaciute), tacco 12 per svettare, perché "ti piace guardare il mondo dall'alto"; i capelli lunghi nascondono le orecchie appena appuntite, perché in questo mondo le arti magiche non sono ben viste. Quell'aria vanitosa e superficiale è un vantaggio: **nasconde le sue intenzioni**.

Lin è una **giornalista freelance**: scriverà un articolo sull'evento su commissione di Maria Bianca Giannini, che le ha anche regalato il vestito per la serata. Il lavoro le dà accesso a molte informazioni e molti

[5] La trascrizione scrive quasi sempre "gem" o "gemma".
[6] La trascrizione dice "ICHAIO".
[7] La trascrizione dice "lì in mezzo corona" e, più avanti, "Lynn"; la grafia esatta del nome non è verificabile.

luoghi, utili al suo **scopo segreto**: ritrovare il fratellastro scomparso, **Ezio Nava**[8], con cui non ha legami di sangue (è figlio del secondo marito della madre) ma con cui è cresciuta ed era molto legata. A un certo punto ha perso ogni contatto con lui; teme che abbia pestato i piedi a qualche malvivente locale, perché Ezio, a differenza sua, ha sempre avuto una propensione per gli affari loschi. Per sicurezza legale, e per piacere personale, Lin ha preso anche il **patentino da investigatrice privata**. Seguono hobby e dettagli magici (ricostruibili solo in parte: la trascrizione parla di un ippogrifo, che Lin possiede, e di partite a bocce "senza magia"), un passaggio su come giocarla, e le relazioni divise per cerchie: al lavoro Maria Bianca Giannini ("già menzionata prima: molto bene la reference multipla") e Giovanni Palmieri; tra gli amici Ambra e Lucio Paoli; in famiglia Ezio ed Ernesto. Poi gli **obiettivi**: a breve termine ritrovare Ezio; a lungo termine, la giustizia, con articoli scritti anche in collaborazione con le forze dell'ordine. E che cosa sa fare: **sparire** senza che le persone se ne accorgano, coprendosi con un mantello (un potere magico), investigare, fare domande, intervistare, e ha accesso ad alcuni documenti della polizia.

Il commento. "Direi perfetta": ci sono le spinte, le attività che le spinte fanno fare, le relazioni. "Hai integrato tutto quello che era richiesto. Dal mio punto di vista non ho nessun commento negativo su questa scheda". Al docente piace in particolare la parte su come giocarla, perché "dà anche proprio un *drive* su come giocare".

2.5.2.2 Rita: Ambra, la eco fashion designer

Poi tocca a Rita (dopo che Tiziana ha imparato a interrompere la condivisione: "basta fermare la condivisione"). La scheda è quella di **Ambra**, 35 anni, single, dallo spirito rivoluzionario e indipendente, con uno stile fuori dal comune: è una **eco fashion designer** in cerca di fama e visibilità, coraggiosa e strategica. **Connettività**: in famiglia è figlia della stilista Maria Bianca Giannini e dell'imprenditore Frank; al lavoro ha ingaggiato Ernesto, il ladro specializzato, ed Ezio Nava, per orchestrare una truffa ("qui ritorna il personaggio che ho visto nella scheda precedente"); tra gli amici ci sono Lucio Paoli, giornalista a caccia di *scoop*, Lin Mezzacorona, amica d'infanzia, e Giovanni Palmieri, l'eco-chef, con cui discute spesso perché sospetta che sia un impostore. **Spinte**: ottenere visibilità nella moda e uscire dall'ombra dei genitori, che non hanno mai creduto in lei né investito su di lei ("fanno solo finta"); anzi, la madre le ruba gli schizzi per le proprie collezioni senza darle alcun merito. E prendere in mano l'azienda di famiglia a qualunque costo. **Attività**: boicottare il gala sfruttando le relazioni con i personaggi già ingaggiati, farsi notare, e usare una campanella con poteri speciali, capace di ipnotizzare e far addormentare profondamente chi è intorno; e poi i social.

Il commento. "Direi perfetto anche qui". Al docente piace moltissimo il **disegno** dell'abito ("avendo studiato comunicazione visiva, non riesco senza un'immagine"); tra gli esempi di Joey Doe della lezione precedente c'erano proprio la foto e gli oggetti, perché l'elemento visivo può dare una mano. E gli viene un'idea: quel disegno è Ambra con il suo abito, o è un abito che può diventare centrale? Nella scheda di Tiziana il vestito di Lin è un regalo di Maria Bianca Giannini; ma se l'avesse disegnato Ambra, le cui idee la madre ruba, "si creerebbe un ulteriore livello di spinte e attività molto interessante": le due amiche potrebbero litigare durante il gala, quando Ambra la vede con quell'abito addosso. Una cosa che si poteva aggiungere è **come si veste** il personaggio, perché in un LARP si può lasciare libertà a chi gioca ma anche dare delle dritte, come aveva fatto la scheda precedente. Ma "c'è davvero tutto": le connessioni sono un po' distillate, "ma di nuovo, in una pagina vanno più che bene".

Quanto sono lunghe le schede? Rita non sapeva se la parte più narrata avesse senso. Il docente lascia libertà, e coglie l'occasione per un tema che non si era ancora toccato: la **lunghezza** delle schede nei giochi dal vivo varia moltissimo. "L'ultima cosa che vi chiedo sono quelle schede in stile anni Novanta da sei-sette pagine, che vi assicuro non rimpiange nessuno. Io di sicuro".

2.5.2.3 Giordano: Frank, il ricettatore

Giordano chiede prima dove sia finito il file di bozza creato sul Drive per il task ("è nella cartella di supporto"). La sua scheda non riporta il nome del personaggio, che è solo nel nome del file: "se fosse una scheda stampata,

[8] La trascrizione dice "Ezionava", "Ezion" e "azionava".

il nome va messo da qualche parte". Il personaggio è **Frank**: arrivato a Milano guidato da un forte spirito imprenditoriale, ci ha fatto fortuna e ha costruito una rete di contatti, finché non ha conosciuto e sposato **Maria Bianca Giannini**. Dalla relazione è nata Ambra, per la quale dopo il divorzio è diventato quasi un estraneo. Ha nascosto il dispiacere nel **gioco d'azzardo**, nelle bische clandestine, e non ha fatto fatica a mescolarsi agli ambienti malavitosi: "in fondo c'è un interesse comune fondamentale, il denaro". La vendita di merce rubata e di artefatti magici si è rivelata molto remunerativa, e le sue relazioni in società sono la copertura ideale. Per arrivare fin qui ha imparato a mentire e manipolare.

Le cerchie sono ben separate. **Famiglia**: Maria Bianca Giannini, l'ex moglie (una relazione intensa ma destinata a spegnersi, "per due persone così ambiziose lo scontro era inevitabile"), e Ambra, la figlia, per la quale ha un po' sofferto, prima che la cupidigia soffocasse il suo interesse per i rapporti umani. **Lavoro**: Ezio Nava, che gli porta merce di qualità mentre lui gli procura oggetti magici, senza domande sulla provenienza; e Giovanni Palmieri, "il più chiacchierato degli ambienti malavitosi di Milano": se devi far sparire qualcuno, sai chi chiamare. **Amici**: Lucio Paoli, giornalista, ed Ernesto, un ladro abilissimo ma meno scaltro di quanto voglia far credere, che almeno con lui ha il vizio di vantarsi troppo dei suoi colpi ("molto bella questa cosa qua"). Le **spinte** sono due, in parallelo: i rapporti con l'ex moglie (dalla buona riuscita del gala dipendono ottime occasioni di investimento) e altre forme di guadagno. **Attività**: la lista degli invitati l'ha colpito, "certi nomi non passano inosservati": indagare sulle loro intenzioni, ostacolarne i piani o allearsi. **Meccanica**: ha sempre con sé un mazzo di carte e un paio di dadi, e sa **alterare magicamente i risultati**, ma non deve farlo troppo spesso, perché qualcuno potrebbe sospettare: al massimo due utilizzi.

Il commento. Gruppi "perfetti, anche ben dettagliati"; due spinte in parallelo. L'unico appunto riguarda le **attività**: "hai diversi nomi che non passano inosservati" è chiaro a chi ha scritto il personaggio, "ma chi gioca si chiede: chi sono questi nomi? A chi devo prestare attenzione?". Meglio ricordarglielo esplicitamente: "presta attenzione a X, Y, Z". Per il resto c'è tutto, ed è chiaro il carattere, il tipo di *drive*, l'approccio. Giordano spiega che l'aveva pensato proprio così: un giocatore può decidere di aiutare Maria Bianca, un altro di aiutare i ladri. Va benissimo: "questo tipo di flessibilità è parte di quegli aspetti di agency e di spinta che dicevamo", non avere una sola direzione ma più cose da fare, dando priorità a una o all'altra.

2.5.2.4 Nicola: Lucio Paoli, il giornalista senza scrupoli

Poi Nicola[9]. La scheda è di **Lucio Paoli**, giornalista generalista che "afferma di essere specializzato in ogni cosa gli capiti sotto il naso". Stasera è l'alta moda, "anche se non saprebbe distinguere un abito da una tenda" ("molto bello"). Il gala non è una normale sfilata: sarà il suo **trampolino**, e se per avere la notizia dovrà fabbricarla o sabotare l'evento, lo farà senza esitare. Seguono le **direttive di redazione**, scritte come un manifesto: "la verità è che [la notizia tranquilla] non vende"[10]; il pubblico vuole vedere gli dèi crollare; se la serata scorre impeccabile, è dovere del reporter accendere la miccia; "i fatti non si aspettano, si provocano"; "più l'uomo è grande, più farà rumore quando cade"; Maria Bianca Giannini ha chiamato la stampa per farsi abbellire, "ma la stampa raccoglie macerie". E il pezzo porterà una sola firma: la concorrenza, colleghi compresi, va neutralizzata o depistata. Poi l'**indole da pavone**: ogni volta che si apre uno spiraglio di conversazione su temi artistici, tecnici o giornalistici, "hai l'obbligo morale di monopolizzare il discorso", sfoggiando una superiorità indiscutibile; se qualcuno dimostra una competenza reale e smonta le sue tesi, la priorità è salvare l'apparenza e non ammettere mai l'errore: accusare l'esperto di pedanteria e abbandonare la scena con sdegno ("bellissima questa parte").

Infine il **dossier** sui contatti, ognuno con una battuta tagliente: Giovanni Palmieri; Ezio Nava, un parente "di cui ignoro il ramo e dimentico il volto", che conviene tenere accanto perché si può sempre rivelare che la sua identità è fasulla, "una mina vagante"; Lin Mezzacorona, "una dilettante con il brutto vizio di mettersi in mezzo ai miei scoop", una minaccia concreta da depistare; **Carletto Magnamani**, "la prima volta che lo sentiamo nominare, chissà chi sarà", conosciuto al pub *Il Grillo Parlante*[11], tenuto di riserva per le emergenze: se la serata non decolla, basta lanciarlo contro il buffet dei VIP e raccontare il disastro; Ambra,

[9] L'attribuzione è probabile ma non certa: dopo Giordano il docente dice "Nicola, perfetto" e chi presenta interviene subito con "vado io". Alla fine del commento la trascrizione fa dire al docente "Angui", incomprensibile.

[10] La trascrizione dice "Nude Crodan non vende": la parola esatta non è ricostruibile.

[11] La trascrizione dice "al brillo parlante" qui e "Il Birillo, il Brillo Parlante" più avanti: il nome esatto del pub non è ricostruibile.

che sfoga le frustrazioni familiari giocando alla contestatrice, "un'alleata preziosa" per farsi consegnare i segreti di casa; e Maria Bianca Giannini, "il bersaglio grosso": pedinarla e registrarne ogni passo falso; "di sicuro c'è del marcio, e se non c'è basterà crearlo".

Il commento. "Fantastico. Spinte e attività sono chiarissime". E si capisce al volo che tipo di personaggio è (il docente lo riassume, tra le risate, con un epiteto poco ripetibile). La parte delle connessioni è "molto lista" e non aiuta a capire al volo chi sta in quale gruppo, come invece accadeva nelle schede precedenti; ma con un personaggio così aperto non è necessariamente un problema: "nel momento in cui il tuo scopo è fare casino, e sei già dentro una polveriera con la dinamite, probabilmente ti basterà aspettare per sapere che cosa fare".

2.5.2.5 Angelo: Ezio Nava, il truffatore

Angelo, che si dichiara "un po' incompetente a livello di tecnologia", riesce a condividere lo schermo con l'aiuto del docente. "Oh, Ezio Nava, finalmente". La scheda è scritta in seconda persona, con la voce di qualcuno che parla a Ezio prima del colpo: "che ti prende? Stai tremando, non è da te. Calmati, ripassiamo il piano". Stasera Ezio non sarà Ezio Nava, truffatore, ma **Ivan**, capo redattore della rivista *Glam Amor*[12] e grande appassionato di gioielleria. L'obiettivo è semplice: impadronirsi del **Topazio di Luna**[13], il gioiello di punta della sfilata. Il suo compito è raccogliere più informazioni possibili su dove si trova e su chi e che cosa lo protegge, passarle al complice, lasciare che sia lui a fare il lavoro sporco e intanto preparare una via d'uscita per quando il complice si farà beccare. Al bisogno ha **due fiammiferi di loto**: accendendone uno, può far dimenticare a qualcuno la conversazione appena avuta con lui ("bello").

Tra gli invitati ci sono facce note: **Ernesto**, il complice, "se fosse sveglio quanto è bravo a rubare, oggi avrebbe un'isola tutta sua"; **Ambra**, che lo ha assunto ma non ha preteso una parte del bottino: "o è pazza o trama qualcosa, meglio non fidarsi"; **Maria Bianca**, di cui ha conquistato con fatica la fiducia e che ora lo adora, ma che stasera sarà al centro dell'attenzione, mentre lui non deve farsi notare; **Carletto**, se c'è qualcuno capace di far saltare la serata è questa specie di moderno Che Guevara che ha cominciato a combattere i ricchi rovinando loro le feste, "sarà da divertirsi"; **Lucio**, "che cosa c'è di peggio di un cugino giornalista? E persino più falso di te"; e **Lin**, la sorella, che non vede da anni e non sospetta nulla: "una brava persona, che potrebbe far saltare la tua copertura". Chiusura: "fai un bel respiro, indossa il più sincero dei tuoi falsi sorrisi, si va in scena".

Il commento. Le spinte e le attività ci sono tutte. Anche qui i gruppi sono una lunga lista che non aiuta a distinguerli al volo, "però c'è tutto". Al docente piace molto lo stile: "parlo a te, ma è come se fosse la tua voce che sta parlando a te". Angelo temeva che non si capisse: "No, no, anzi, mi piace molto".

2.5.2.6 Giovanni Palmieri, l'alchimista

Lo studente successivo non riesce a condividere dal Mac ("mi dice di aprire le impostazioni di sistema... mi dice di uscire da Zoom"), e allora il docente apre il file e lo condivide lui[14]. È **Giovanni Palmieri**, l'**alchimista**: eco-chef e sicario ("interessante"), 37 anni, creativo e calcolatore. La scheda è in prima persona. Viene dall'inquinata **Valle del Sacco**, in Ciociaria, dove ha sviluppato un'insolita **resistenza ai veleni**. È fuggito da giovane in cerca di un luogo dove esprimersi come eco-chef e si è formato nelle cucine italiane, ma non aveva i soldi per aprire un ristorante e ha dovuto affidarsi a persone discutibili: grazie al fratellastro ritrovato, **Ernesto Lesto**, ladro specializzato in furti d'arte, è entrato nella malavita milanese come **sicario e avvelenatore**, e in cambio i boss gli hanno aperto il suo *fast food* ecosostenibile, dove lavora anche Lin. Quando non prepara panini vegani o veleni, passa il tempo al pub, dove ha conosciuto il suo ragazzo, **Lucio Paoli**, e la rivoluzionaria Ambra, con cui condivide birre e chiacchiere, a volte un po' troppo radicali; e poi c'è Carletto Magnamani, il critico gastronomico, che "giudica troppo e mangia altrettanto".

[12] La trascrizione dice "Ivan Klenionovna" e, nella scheda di Maria Bianca Giannini, "Ivan temino"; il cognome non è ricostruibile. Anche il nome della rivista è come lo riporta la trascrizione.

[13] La trascrizione dice "la luna topazio" qui e "l'armonito pazzo" nella scheda di Ernesto; il nome del gioiello è incerto.

[14] La trascrizione non permette di capire chi sia l'autore di questa scheda.

Come giocarlo: è un introverso, una personalità che al primo impatto può sembrare antipatica, ma diventa più spigliato quando è a suo agio con le persone giuste (Lucio, la dipendente Lin, l'amica Ambra); anche allora resta attento, "non sai mai chi hai davvero davanti". Il suo pregio migliore è la creatività, che lo fa spiccare in cucina come da sicario; mantiene la calma anche nel caos. **Obiettivi**: aiutare Lucio con lo scoop della sua vita; fare in modo che Lin non scopra che il fratello che sta cercando, Ezio Nava, il cugino di Lucio, è un truffatore che ha pestato i piedi alle persone sbagliate e che sarà al gala. Giovanni curerà il **catering**, grazie alle sue conoscenze, e Lucio, che conosce la sua doppia vita e la sua abilità di alchimista, gli ha chiesto di "animare" la serata; così lo ha avvisato di non mangiare nulla al buffet, perché il cibo conterrà un **siero della verità** che impedisce di mentire per un'ora. In un ambiente così delicato ciò creerà sicuramente un po' di caos, e lui sa di lavorare meglio nella confusione. È consapevole che una relazione con il suo ragazzo potrebbe incrinarsi se lui scoprisse tutto, ma si consola sapendo che Lucio avrà lo scoop della sua vita. Infine, uno schema riassuntivo delle **relazioni**.

Il commento. È organizzata in modo molto particolare: il materiale della categoria "relazioni" è in realtà sparso nelle altre parti, e lo schema finale è un riassunto al volo. "Mi sembra molto funzionale". Piacciono molto gli **obiettivi**, *front and center*: "se hai un dubbio su che cosa fare adesso, guardi quelli e una direzione ti viene sicuramente". E c'è la voce del personaggio, che dà un'idea di come parla e di com'è. "Un *thumbs up* anche questa".

A questo punto entra **Gianmarco**: "fantastico". L'ordine dei turni rimanenti diventa Lorenzo, Tancredi e Gianmarco ("scusatemi, ho perso il conto come al solito").

2.5.2.7 Lorenzo: Carletto Magnamani, il critico che mangia tutto

Lorenzo ha solo la bozza, "si sta caricando". Prima osservazione, che il docente dice "anche per il futuro": il **nome di chi ha scritto il personaggio** non va sulla scheda, perché la scheda la legge chi la gioca. Il personaggio è **Carletto Magnamani**, che vuole approfittare dell'occasione per far fallire l'evento **finendo il cibo a disposizione**: ha un apparato digerente estremamente efficiente, può mangiare a oltranza. Odia la ricchezza concentrata nelle mani di pochi: farà fallire i ricchi e manometterà velatamente l'ordine degli eventi. La sua altissima tolleranza ai veleni e alla sporcizia, unita alla capacità di analizzare la struttura molecolare di ciò che ingerisce, gli permette perfino di fare **analisi chimiche** di quello che mangia (con un inconveniente: "purtroppo devo comunque andare ai servizi, e capita spesso"). I cibi li **recensisce**, e deve far sapere a tutti che cosa ne pensa. È legato alla malavita, anche se non sembra: odia il mondo in cui sta, che però lo fa guadagnare. Lo aiuta il suo essere espansivo: tutti pensano di essere suoi amici. **Famiglia**: Ambra, l'ex fidanzata, per cui prova ancora tenerezza ("se non fosse sempre così dura con me"), e Maria Bianca Giannini, la madre di Ambra, che non gli sta simpatica, a differenza del padre. **Amici**: un amico di pub che può essergli utile ed Ernesto Lesto, ideologicamente affine. **Lavoro**: Giovanni Palmieri, che stima perché "ha fatto strada venendo dal basso", e per cui copre molti controlli di qualità che non dovrebbero passare, "ma si sa, una mano lava l'altra"; e Lin, che sospetta possa essere una potenziale minaccia investigativa.

Il commento. Il docente nota che nella scheda di Ambra Carletto non compariva come ex fidanzato; Rita conferma di non averlo proprio menzionato. Ed è proprio questo che entusiasma il docente: che una delle due schede taccia la relazione è una scelta, e "la scelta di non menzionarla mi piace tantissimo" ("capolavoro"). Detto questo, la scheda è molto *wall of text*: mettere in grassetto, dividere in blocchi famiglia, lavoro e amici aiuterebbe la lettura; ma, ricorda, non era una cosa richiesta, perché la lezione sul **paragame** deve ancora arrivare. In termini di contenuto c'è tutto. L'appunto vero riguarda la **spinta** e l'**attività**, un po' più deboli di quelle degli altri personaggi: se la domanda è "che cosa faccio adesso?", il *drive* che la scheda dà è "vado a mangiare qualcosa", "che non è proprio un drive fortissimo, perlomeno sul lungo termine". A breve termine le cose da fare sono molte, a lungo termine meno. Ma i gruppi ci sono, e caratterialmente è chiaro che tipo di personaggio sia.

2.5.2.8 Tancredi: Ernesto Lesto, il ladro gentiluomo

Tancredi ("vai pure alla seconda pagina") presenta **Ernesto Lesto**; anche qui c'è il nome di chi ha scritto la scheda, e anche qui "i riferimenti a chi ha scritto il personaggio non vanno sulla scheda". Ernesto è un ladro

specializzato in furti d'arte: il soprannome "Lesto" se l'è guadagnato rubando gioielli dovunque andasse, senza mai farsi beccare. Viene dai bassifondi di Milano, dove è cresciuto con il fratellastro, Giovanni Palmieri. Stasera parteciperà al Gran Gala di Milano, la più prestigiosa sfilata di moda: entrare per lui è facile, perché nonostante la reputazione gode delle simpatie di Maria Bianca Giannini, che lo invita sempre. Ma stasera non è lì per raccontare qualche storia all'alta borghesia: "stasera ti puoi sistemare a vita". Ambra lo ha ingaggiato per rubare il gioiello più importante della collezione della madre, senza interesse ad avere la sua fetta. Le informazioni in suo possesso sono poche: dovrà scoprire dove sia il gioiello senza destare sospetti, restare vago, raccontare qualche storiella e, "quando la prova avrà fatto effetto", iniziare la caccia. "Fai attenzione a che cosa bevi: è un'occasione troppo importante per sciuparla". Non sarà solo: c'è il socio, Ezio Nava, il ricettatore con cui spartirà il bottino ("forse"), che ultimamente chiede sempre di lui. Alla festa ci sarà anche la cugina Lin, che scrive cose noiose sul suo giornaletto, "non come Lucio Paoli, lui sì che scrive articoli come si deve": la passione per il giornalismo li accomuna poco, ma per Lucio ha sempre avuto una certa simpatia.

Il commento. "Molto bene". Non ci sono categorie separate per famiglia, amici e lavoro, ma tutti i **nomi in grassetto** e le parti che sono spinte e attività **sottolineate**: "per me va benissimo, c'è tutto". Il personaggio non ha poteri particolari; Tancredi spiega che l'idea è il suo modo di parlare, il fatto che la borghesia ami sentire le sue storie. Va benissimo, ed è coerente con quello che la scheda stessa comunica.

2.5.2.9 Gianmarco: Maria Bianca Giannini, la stilista

Per ultimo Gianmarco, dopo un piccolo incidente tecnico (il docente, cercando di dargli i permessi di condivisione, gli blocca per sbaglio la telecamera: "scusa, prova ora"). "E finalmente vediamo Maria Bianca Giannini". La scheda ha una piccola copertina ed è in **formato intervista**: è Lucio Paoli che la intervista alla vigilia della sfilata. La trascrizione di questa parte è molto rovinata, e il capitolo ne riporta solo ciò che è ricostruibile: si parla del marito, che si dice non stia bene; delle voci su una storia poco limpida, fatta anche di ricatti, che lei respinge; del fatto che dalla riuscita della sfilata dipende la sua carica davanti al consiglio di amministrazione ("sono sicura che è tutto perfetto, fino all'ultima cucitura"); delle origini (una famiglia, negli anni Settanta, un treno per Milano) e di un passato legato a pozioni di erbe; del rapporto con la figlia Ambra ("se c'è qualche attrito, troviamo sempre un modo per andare d'accordo"); e della sua passione fuori dalle passerelle, le aste e la collezione di quadri del Novecento (la motivazione, un confronto tra gli abiti, che passano, e i quadri, è leggibile solo a metà).

Segue la parte destinata a chi gioca. Maria Bianca deve fare in modo che stasera sia tutto perfetto, perché dal gala dipende la sua carica (segue una frase su amici e nemici che la trascrizione rende incomprensibile). È molto **risoluta**: dà del lei a tutti tranne che ad Ambra e a Frank, scandisce le parole e, quando è sotto pressione, dice ogni cosa come un ordine, e la tensione diventa aggressività. Non sopporta di perdere uno scontro verbale: se succede, si infuria e resta nervosa tutta la serata, prendendosela con il primo che capita a tiro. Durante il gala deve controllare che ogni cosa sia al suo posto, capire, partendo dalla sua rete di conoscenze, chi vuole tradirla e di chi può fidarsi, tenersi buoni i giornalisti e tenere d'occhio Ambra, perché il suo talento deve mettersi al servizio dell'azienda. E segretamente è una **fattucchiera**: può improvvisare **pozioni** che addormentano, confondono e seducono. Quando la giocatrice o il giocatore di Maria Bianca offre una bevanda a un altro personaggio, la bevanda può essere infusa di erbe eccitanti, seducenti, confondenti o soporifere, e **gli effetti vengono comunicati solo dopo che si è bevuto**. Connessioni: in famiglia Ambra, la figlia (che lei apprezza per il talento, mentre Ambra pensa il contrario), e Frank, l'ex marito, con cui i rapporti sono torbidi; tra gli amici Ivan ("qui è il nome che il personaggio deve pensare che abbia": "fantastico") ed Ernesto Lesto, che le racconta le sue storie improbabili; al lavoro Lin Mezzacorona e Lucio Paoli.

Gianmarco ha anche fatto una **versione alternativa**, in cui ha provato a mettere tutto *dentro al gioco*: l'articolo è più o meno lo stesso, mentre spinte e carattere sono annotati come appunti su un foglio, insieme alle pozioni. "Fatto molto bene": tutto questo si vedrà nella lezione sul **paragame**.

Il commento. Due attenzioni. La prima: in una scheda per un LARP non bisogna mettere la **faccia** del personaggio, perché non sappiamo chi giocherà Maria Bianca, e non essendo un gioco di ruolo da tavolo quella faccia non le corrisponderà; la parte visiva si può compensare come aveva fatto Rita, descrivendo come

si veste, che abiti indossa, che stile ha. La seconda: nelle istruzioni c'è scritto "il giocatore", al maschile. "Noi non sappiamo chi giocherà questo personaggio": ci sono vari modi per scrivere in maniera **neutra**, e lo si vedrà quando si parlerà di come si scrivono i regolamenti. Per il resto, "c'è tutto": spinte chiare, connessioni chiarissime e richiamate anche altrove, evidenziate le parti chiave a cui prestare attenzione. "Per me, super *thumbs up*".

Personaggio	**Che cosa funziona**	**Che cosa si può migliorare**
Lin Mezzacorona	tutto: spinte, attività, relazioni, come vestirsi e come giocarla	nulla di negativo
Ambra	spinte, attività, disegno dell'abito	dire come si veste; l'abito potrebbe diventare un nodo di trama
Frank	cerchie ben dettagliate; due spinte parallele; flessibilità di gioco	nominare esplicitamente le persone a cui badare
Lucio Paoli	spinte e attività chiarissime; carattere immediato	connessioni "molto lista"
Ezio Nava	seconda persona con la voce del personaggio; spinte e attività	gruppi in lista poco distinguibili
Giovanni Palmieri	obiettivi *front and center*; voce; schema riassuntivo delle relazioni	—
Carletto Magnamani	relazione taciuta con Ambra ("capolavoro"); carattere chiaro	spinta e attività deboli a lungo termine; *wall of text*; nome dell'autore sulla scheda
Ernesto Lesto	nomi in grassetto; spinte e attività sottolineate	nome dell'autore sulla scheda
Maria Bianca Giannini	formato intervista; pozioni come meccanica; versione tutta in gioco	niente foto del viso in un LARP; scrivere in modo neutro

Table 2.5.1: La revisione delle schede del primo task, in sintesi. Il giudizio complessivo del docente è che "avete fatto tutti un ottimo lavoro".

2.5.2.10 L'età dei personaggi

Chiusa la revisione ("dal mio punto di vista avete fatto tutti un ottimo lavoro"), Lorenzo pone un dubbio: se non si deve mettere l'aspetto fisico, perché non sappiamo chi giocherà il personaggio, lo stesso vale per l'**età**? L'età, risponde il docente, è più flessibile, e qui si apre "un altro vaso di Pandora", perché i giochi di ruolo dal vivo hanno approcci diversi. Nell'approccio **chamber** di solito non è un problema: si può avere un personaggio di 90 anni e giocarlo avendone 15 o 30. Nei **blockbuster**, i giochi più grossi e corposi, l'età è legata alle dinamiche di **casting**: ci sono vere e proprie fasi di casting in cui, in base a chi si è, alcuni personaggi possono non essere accessibili (per esempio, se serve un capo villaggio anziano, magari non lo fanno scegliere a chi ha meno di quarant'anni). Anche lì c'è flessibilità: un personaggio di 80 anni lo si può giocare avendone 70 o 60. Ma è comunque meno problematico delle **foto**, perché viso, capelli e corporatura sono meno generalizzabili: due persone diverse di trent'anni possono avere un'età percepita molto diversa a seconda di chi le osserva.

Si passa alla lezione, "abbastanza intensa", alla fine della quale arriverà un task "contemporaneamente più facile e più difficile" di quello appena visto.

2.5.3 Che cosa prova chi gioca

"Iniziamo sul leggero". Nella primissima lezione erano state indicate tre parole da non usare. Una, *immersione*, è già stata smontata con il Player Involvement Model, che offre aree progettabili "molto più chiare, molto più precise" e più utili a chi fa game design. Del *divertimento*, invece, non si è ancora detto quasi niente: "in teoria, se tutto va bene, alla fine della lezione di oggi, e anche della prima parte, dovrebbe essere abbastanza chiaro".

La domanda di partenza è: **che cosa prova chi gioca?** Le risposte arrivano una dopo l'altra: **ansia**, **sfida**, **appagamento**, **curiosità**. Sono emozioni lontanissime tra loro (ansia e curiosità, per esempio), "ma sono vere comunque". È il punto di base: come altre forme mediatiche, il gioco può dare moltissime emozioni diverse, e **lo stesso gioco** può darne di diverse **a seconda di chi lo gioca**.

L'ansia di Angelo. Il docente chiede ad Angelo da che cosa venga la sua ansia. Non da un gioco in particolare: "la prima immagine che mi è venuta è il momento in cui sto per fare la mia mossa migliore, guardo in faccia gli altri e so che uno di loro me la può fermare". Perfetto, perché in quella risposta c'è già uno scenario molto preciso: un gioco **competitivo**, non cooperativo (altrimenti l'ansia non sarebbe diretta agli altri giocatori), in cui vince chi massimizza le proprie azioni. "Non è una sensazione che useresti per la tombola".

La curiosità di Tiziana. Tiziana l'ha provata spesso ultimamente, perché ha provato tantissimi giochi che non conosceva, e li affronta con la curiosità di capire come funzionano. Questo, osserva il docente, è un altro livello, addirittura un **meta-livello**: non è un'emozione data da un gioco specifico, ma da *come ci si avvicina al giocare*. E non è eliminabile: quando giochiamo restiamo persone, con la nostra vita intorno. Magari è uno dei nostri giochi preferiti, ma se quel giorno siamo stati male, siamo inciampati per strada, abbiamo mangiato qualcosa che non dovevamo e passiamo mezza giornata in bagno, "forse non siamo nell'umore per giocare quel gioco in questo momento. E non è colpa del gioco in sé".

Giordano aggiunge un'immagine: allo stesso tavolo c'è Tiziana, curiosa di sperimentare le meccaniche; c'è qualcuno di estremamente competitivo, che pensa solo a fare tutto perfetto; e c'è Angelo, che vive l'ansia della mossa. "Eppure è lo stesso gioco". "Esattamente". Tiziana conferma per esperienza: le è capitato di giocare in condizioni pessime (caldo, umido, gran baccano), e quei giochi non le sono piaciuti; li ha messi in *stand by*, per dar loro una seconda possibilità in altre condizioni. "Perché non siamo delle macchine, siamo delle persone".

La rabbia di *John Company*. Gianmarco cita un'emozione che non era ancora uscita: nei giochi ad altissima interazione possono nascere al tavolo dinamiche di **aggressività** e **rabbia**, che di solito finiscono lì. Come in una recente partita a *John Company*, "un gioco tagliagole". Il docente conferma: "ho avuto un amico che mi ha tassato al punto che la mia anziana zia, in gioco, è finita in strada per la tassa sulle finestre. Dico solo questo". La violenza "c'è nel gioco", e la si cavalca un po'.

2.5.4 Il problema della parola "divertimento"

Per il docente è una battaglia quotidiana: *immersione* si è già vista, ma *divertimento* sembra inscindibile dal gioco, e combatterla sembra "una battaglia destinata a essere persa, come Don Chisciotte contro i mulini a vento". "Ma sono una persona troppo testarda per abbandonarla".

2.5.4.1 Che cosa vuol dire "mi sto divertendo"?

La domanda è pragmatica, non etimologica ("non voglio qualcuno che mi dica divertimento da divergere"): quando qualcuno dice "mi sono divertito a questo gioco", che cosa sta dicendo? Le risposte: che sta andando bene, per il gruppo, per la serata; che si stanno provando emozioni positive; che si stanno ricevendo stimoli, forse nuovi; che ci si sta **rigenerando**, ricaricando, facendo qualcosa che fa stare bene.

Il docente rilancia con l'esempio di Gianmarco. A *John Company* non ci si rigenera, la situazione non va bene quasi mai, non ci si ricarica né si sta in pace col mondo; eppure è un gioco eccezionale. "C'è qualcosa che non quadra". *Mi sto divertendo* non copre tutti i tipi di giochi. Si può dire anche "ho visto un film divertente", ma non tutti i film lo sono: le emozioni umane sono tante, e un gioco può volerci far provare emozioni molto particolari. Con un film horror vogliamo tensione, vogliamo spaventarci, sentire il cuore battere fortissimo, "stare sull'orlo della sanità mentale"; non è un'esperienza che vorremmo nella vita. Allo stesso modo, il gioco può farci provare qualcosa di molto forte **in un contesto più sicuro**. *John Company* non lo si gioca per passatempo, né per rilassarsi: "usciamo dalla partita che vogliamo accoltellare i nostri amici con i coltelli da cucina".

2.5.4.2 Il divertimento non è nel gioco, è nel giocare

Perché allora pretendiamo che un gioco *debba* essere divertente? Alla domanda "che cosa ci sta divertendo?" la risposta automatica è "il gioco": questo è un gioco divertente. Ma, e qui il docente riprende un punto della prima lezione che "diventa particolarmente importante", **quando parliamo di divertimento non stiamo mai parlando del gioco**. "Mai". Stiamo parlando del **giocare**: è l'attività, il *play*, che possiamo al massimo definire divertente, o far rientrare nell'insieme di emozioni che potremmo chiamare divertenti. Il gioco in sé non ha nessun aspetto formale, nelle meccaniche o nelle regole, che sia "divertimento".

> Non possiamo meccanizzare il divertimento. Non è un componente che metto in gioco, tipo: il gioco contiene un regolamento, dieci carte, un dado e divertimento. Non esiste questa roba qua. Il divertimento spunta dalle dinamiche, spunta da noi che stiamo giocando. Se nessuno gioca un gioco, quel gioco non può far provare divertimento a nessuno.



Figure 2.5.1: Il divertimento secondo il docente. Il designer ha potere sulle regole e sui componenti; ciò che chi gioca prova emerge dal giocare, cioè dalle dinamiche, dalle persone e dal contesto. Dire "questo gioco è divertente" attribuisce al gioco qualcosa che appartiene al giocare.

Una fonte di conflitti. Che cosa succede invece? Che si gioca un gioco, lo si trova divertente o no, e poi se ne parla come se fosse *implicitamente* un gioco divertente o non divertente. Succede "in maniera pervasiva": nei canali di recensioni, nei gruppi di settore. E il termine è spesso **fonte di conflitto**: se il gioco è divertente e io mi sono divertito, non è possibile che tu non ti sia divertito; quindi "non capisci niente tu". "E invece no": è che le persone provano cose diverse, e il gioco in sé non ha niente di implicitamente legato al divertimento. Ci sono, certo, meccaniche e dinamiche che è normale associare al divertimento: i **party game** sono percepiti come attività rilassate, comiche, ilari, perché hanno meccaniche che spingono su certe dinamiche sociali; mentre nessuno pensa come prima cosa ai grandi strategici di **GMT** da quattro-sei ore come "divertenti", perché visti da fuori "sembra che tu stia facendo la dichiarazione dei redditi".

Divertente uguale bello. C'è poi un altro livello del problema: nelle conversazioni sui giochi, *divertente* è un **descrittore unicamente positivo**. Mi sto divertendo, quindi è un bel gioco; qualcuno mi ha detto che è divertente, quindi è un bel gioco. "Se non è un gioco divertente, non è un bel gioco: questa è la norma". Ed è "un gigantesco problema" per chi fa game design, perché **limita l'area progettabile** di ciò che si vuole e si può comunicare con un gioco. "È come pretendere che tutti i film siano divertenti. Che cosa facciamo dei documentari? Di *Schindler's List*? Dei film horror?".

2.5.5 Giochi su temi difficili

Il gioco può trattare questi temi, ma quando lo fa spuntano commenti come "non puoi fare un gioco su questa cosa". Il docente porta alcuni esempi.

Il black box su Cuba. Già visto nei casi studio[15]: un LARP black box in cui si gioca nella Cuba che opprime le minoranze. "Nessuno esce da lì felice di aver passato quella serata: su dieci persone, si piange tutti quanti".

The Cost. Un gioco da tavolo sulla produzione dell'**amianto**[16], in cui la meccanica è far morire i lavoratori producendo senza sicurezza, per produrre di più e guadagnare di più. È chiaramente un gioco **critico** del tema, ma il tema "non è proprio una passeggiata": "stasera giochiamo a *Ticket to Ride* o stasera produciamo amianto e facciamo ammazzare i lavoratori per vendere di più?".

Holding On. "Un grandissimo gioco"[17]: un cooperativo in cui si è personale infermieristico in un reparto di **cure palliative**. Arriva d'urgenza un anziano signore, Billy Kerr, colto da un malore in aeroporto; bisogna ricostruire la sua storia mentre sta morendo, e capire come accompagnarlo alla fine della vita in modo che muoia in pace, senza soffrire. "Anche questo lo chiamereste divertente?".

Charnel Houses of Europe. Un gioco di ruolo sulla **Shoah**[18], pubblicato dalla Black Dog, una branca della White Wolf ("quelli di *Vampire: The Masquerade*"). Richard Dansky[19], racconta il docente, propose di fare un gioco sull'Olocausto usando l'ambientazione dei fantasmi; "che ci sta, solo che noi non ne sappiamo niente". Chiamarono allora una saggista che scriveva sull'Olocausto (non giochi: saggi), chiedendole di scrivere la parte di *world building*, di fiction e di collegamenti con la realtà. Lei rifiutò seccamente; ma un paio di settimane dopo li richiamò: "ci ho ripensato e lo faccio. Se scrivo un altro saggio, lo legge chi già legge i saggi. Se voglio parlare a qualcuno che non comprerebbe mai un mio saggio, devo parlargli con un altro medium, magari un medium che si *esperisce*". E il manuale si fece.

Se si mantiene l'equazione *gioco divertente = gioco bello*, allora nessuno di questi è un bel gioco, "sono tutti giochi terribili". "E non so voi, ma io non lo voglio un medium in cui mi viene imposto di non poter fare giochi su determinate tematiche, perché si è deciso che i giochi devono essere solo esperienze felici, passatempi ed escapismo". Nessuno dice di eliminare i party game, i giochi leggeri, i giochi su temi allegri. Ma se si vuole trattare il gioco come **medium legittimo** ("e in quanto persone che vogliono fare game design, io mi auguro che vogliate farlo"), bisogna accettarlo: altrimenti salta tutta l'area del **gioco formativo**, perché se il gioco deve essere solo divertimento ed escapismo non può essere didattico. "Credo che ne beneficiamo tutti, in quanto umanità, se iniziamo a trattare il gioco come medium legittimo".

È già successo con altri media. Questo tentativo di sminuire un medium si è visto fin dall'inizio della storia. Prendiamo l'Olocausto: nessuno direbbe a *Schindler's List* "non puoi fare un film su questo tema". A *Maus*, come fumetto, all'epoca lo hanno detto: non puoi fare un fumetto sull'Olocausto. "Adesso chi si sente di dire che *Maus* non aveva il diritto di essere un fumetto?".

Esperienza e regole. Tiziana collega gli esempi a qualcosa che pensa anche lei: far giocare le persone vuol dire far fare loro un'**esperienza**, come fanno i bambini quando giocano imitando gli adulti; e in questo senso nel gioco può rientrare qualsiasi cosa. Il docente richiama la definizione già vista nelle prime lezioni, da *Rise of the Videogame Zinesters* di Anna Anthropy ("a pagina 43"): il gioco come **esperienza creata da regole**. Non in senso "memetico" ("allora anche andare in banca è un gioco"), ma come indicazione per chi progetta: le due cose da tenere fisse in mente sono **l'esperienza e le regole**. Sulle regole avete potere, le decidete voi; l'esperienza è quella che quelle regole producono. Tiziana trova interessante "chiudere il cerchio" così.

[15] La trascrizione lo chiama qui "socialismo di muerte" (nella lezione sui casi studio era "Socialism and Word"): il titolo esatto non è verificabile.
[16] La trascrizione dice "The Coast".
[17] Il titolo completo è *Holding On: The Troubled Life of Billy Kerr*.
[18] La trascrizione dice "Charnel as a House of Europe". Il manuale è *Charnel Houses of Europe: The Shoah*, uscito per *Wraith: The Oblivion*.
[19] La trascrizione dice "Richard Dunstan" e, più avanti, "Richard Dansky". Il docente lo presenta come direttore creativo.

2.5.5.1 Si può fare, ma non da soli

"Occhio, però": il docente non vuole che passi un altro messaggio "ugualmente problematico e pernicioso". Si possono fare giochi su qualunque tema; **non vuol dire che si debba**, né che si debba farlo al posto di qualcun altro, né che lo si debba fare **da soli**. "Se mi mettessi a fare un gioco per sensibilizzare sull'operare a cuore aperto, io non sono un chirurgo, non so niente del tema, e ne uscirebbe un gioco sicuramente terribile". Quando si fanno giochi su temi del genere è buona cosa appoggiarsi a persone che **sanno di che cosa stanno parlando**.

Un esempio dal suo lavoro: un gioco formativo per ragazze e ragazzi in età scolare, tra gli 11 e i 13 anni, sul **diabete**[20]. Ad affiancare i designer c'era un team dell'**OMS**, la World Health Organization, con esperti del tema, per assicurarsi che il gioco non diventasse "una serie di sciocchezze": un gioco che dà informazioni sbagliate sul diabete, quando il suo scopo è informare, sarebbe stato devastante. Lì è palese che serva un **consulente esterno** (un *cultural consultant*); ma è molto utile anche nei giochi più divulgativi, o di puro intrattenimento.

Per approfondire. Il docente lascia nelle slide tre riferimenti:

- un **talk di Richard Dansky** sul caso del gioco di ruolo sulla Shoah;
- ***The Positive Negative Experience in Extreme Role-Playing*** di **Markus Montola**[21], "uno dei miei paper preferiti": perché alcune persone giocano giochi di ruolo palesemente su esperienze traumatiche e terribili, e come giocarle possa essere, in termini personali, un'esperienza positiva. Vi sono citati anche alcuni giochi Jeepform che il docente non aveva portato nei casi studio, "molto, molto più intensi di *Fat Man Down*";
- un articolo di **Jaakko Stenros**[22] ("una persona squisita") su un LARP sull'**Alzheimer**, pensato per sensibilizzare sul tema e trattato in modo delicato e rispettoso, che fu **escluso dal programma** di una conferenza: il pubblico vide solo "hanno fatto un gioco sull'Alzheimer; l'Alzheimer non è un gioco", e l'organizzazione, sotto pressione, lo cancellò. "Il che fa arrabbiare".

Approfondimento: il gioco non è l'esperienza. La distinzione del docente tra *gioco* e *giocare* è la stessa da cui parte Jesse Schell in *The Art of Game Design*: il designer crea un gioco, ma ciò che gli interessa davvero è l'esperienza, che il gioco rende possibile senza esserlo; e l'esperienza avviene nella mente di chi gioca, per questo il designer non può controllarla direttamente, solo influenzarla attraverso le regole. È anche la ragione per cui Schell dedica lenti apposite all'esperienza e all'emozione, chiedendo al designer quali emozioni vuole che il giocatore provi e perché, non soltanto se "si diverte". Raph Koster, in *A Theory of Fun for Game Design*, usa invece la parola *fun*, ma la definisce in modo preciso (il piacere di imparare e padroneggiare degli schemi) e riconosce che i giochi possono e devono saper dare anche altre emozioni: una posizione che, pur con un lessico diverso, va nella stessa direzione della lezione.

La pausa. "È tempo di saltare". Ma prima del salto il docente fa fare una pausa, "perché voi non sapete che cosa state per fare, ma io sì": dieci minuti, fino alle 18:25[23].

2.5.6 Il nanogioco del paracadute

Alla ripresa serve una persona **volontaria** ("non la sceglierò io"). Si propone Angelo, che accende la telecamera. Agli altri il docente chiede di **osservare bene** che cosa sta per succedere; ad Angelo, di seguire il più possibile le istruzioni a schermo. È "un nanogioco": alla fine si analizzerà insieme che cosa il gioco fa e non fa.

[20] La trascrizione dice "diabete di tipo B"; e dice che il docente ci aveva lavorato "con Gustavo Copenaghen", un riferimento non ricostruibile (forse una persona o una realtà di Copenaghen).
[21] La trascrizione dice "The Positive Negative Experience of Steam Role Playing".
[22] La trascrizione dice "Jacopo Stenros".
[23] La trascrizione dice "sono le 17.00, facciamo a 18.25": con ogni probabilità erano le 18:15.

Le istruzioni. Prima un respiro profondo. Poi **due minuti** per pensare a un personaggio, che sarà quello da giocare in questo nano gioco di ruolo: nome, cognome, soprannome, dove vive, dove è cresciuto, qual è il suo film preferito, dove gli piacerebbe vivere, che lavoro fa, che lavoro vorrebbe fare, eccetera. Il mondo è quello reale, ai giorni nostri, e il personaggio sta per andare a fare **paracadutismo**. Due regole: non scrivere niente da nessuna parte, e non dire niente ad alta voce. "Mi rendo conto che saranno due minuti di silenzio, non esattamente la cosa più intrattenente del mondo, ma servono". Parte il timer; il docente va a prendere un bicchiere d'acqua.

Il salto. Allo scadere, la fase successiva. Il personaggio salta dall'aereo, e il docente ha **40 domande** scritte su un foglio. Il salto dura **cinque minuti**, e il personaggio si accorge che **il paracadute non si apre**: sa che è giunta la sua ora. Il gioco ha **due esiti**. Se si risponde a tutte le 40 domande, il personaggio ha chiuso i conti, ha fatto il resoconto della sua vita e **muore in pace**. Se si schianta al suolo dopo cinque minuti senza aver risposto a tutte, se ne va con **questioni in sospeso**. Le domande vengono fatte in ordine, e **ogni volta che si esita** troppo a lungo per pensare alla risposta, **si riparte dalla prima**. Il docente non dirà quanto tempo resta: dirà solo quando il timer arriva a zero. "Quando sei pronto, dimmi che salti".

"Salto".

La partita. Il personaggio di Angelo si chiama **Andrea Ricciardi**. La partita, ricostruita dalla trascrizione, procede così (le domande sono quelle che il docente arriva a porre; le ripartenze dalla prima domanda sono almeno sei):

1. Come ti chiami? *Andrea Ricciardi.*
2. Dove sei cresciuto? *A Pavia.*
3. Hai avuto un'infanzia felice? *Molto.*
4. Chi era il tuo idolo da bambino? *Mio papà.*
5. Qual era il tuo sogno da bambino? *Fare il pilota di Formula 1.*
6. Dove andavi quando volevi stare da solo? (Qui la prima esitazione, e si ricomincia.) *In soffitta.*
7. Quando ci sei andato per la prima volta? *A sei anni.*
8. Ci vai ancora? *No.*
9. Chi è stato il tuo primo amore? *Mai avuto.*
10. Come mai? *Sono timido.*
11. Quando te ne sei andato di casa? *Non me ne sono mai andato.*
12. In che rapporti sei con i tuoi genitori? *Li adoro.*
13. Fratelli o sorelle? *Nessuno.*
14. I rapporti con la tua famiglia sono cambiati col passare degli anni? *No.*
15. Qual è il tuo colore preferito? *Blu.*
16. Che cosa ti viene in mente quando ci pensi? *Il mare.*
17. Qual è stato il momento più felice della tua vita? *Quando ho iniziato a lavorare con mio padre.*
18. Quale momento della tua vita vorresti cancellare? (la risposta, nella trascrizione, è incomprensibile: "e fallire").
19. Dove hai sempre voluto andare? *A Londra.*
20. Perché, che cosa ti attira di quel posto? *È una città molto più grande di Pavia.*
21. Qual è il tuo hobby? *Colleziono francobolli.*

E poi: "*Splat*". Il personaggio si è appena spiaccicato al suolo, a circa metà delle domande.

L'analisi. Andrea, osserva il docente, è un personaggio creato da zero, separato da Angelo. Ci sono state domande a cui Angelo ha risposto *come Angelo* e non come Andrea? "No, nessuna", dice Angelo d'istinto. "Il colore preferito?". "Il colore preferito, sì". E fratelli o sorelle? "Io ho un fratello, quindi no". Che cosa è successo? Angelo, pur giocando Andrea, ha dato risposte, anche solo in alcuni dettagli, *in quanto Angelo*. "E non è colpa tua".

2.5.7 Il bleed

Quella cosa ha un nome specifico: nel settore la si chiama **bleed**[24]. Non "bleed" come sanguinare, ma come il *color bleed*: come i panni lavati insieme in lavatrice, che si macchiano a vicenda. La definizione proiettata a schermo:

> Una persona esperisce bleed quando i suoi pensieri, le sue azioni e le sue emozioni sono influenzati dai pensieri e dalle emozioni del suo personaggio, o viceversa. Con l'aumentare del bleed, la linea tra chi gioca e il personaggio diventa gradualmente più trasparente.

Il bleed ha **due direzioni**:

Bleed in: pensieri, emozioni e azioni di chi gioca influenzano il personaggio. Il blu era il colore preferito di Angelo, e in quel momento è diventato il colore preferito di Andrea.

Bleed out: pensieri, emozioni e azioni del personaggio influenzano chi gioca. Magari Angelo, anche solo per un momento, ha provato un po' d'ansia; "ma tu, Angelo, non ti stai buttando col paracadute. È il tuo personaggio che si sta buttando. L'emozione che lui dovrebbe provare, però, la stai provando tu".

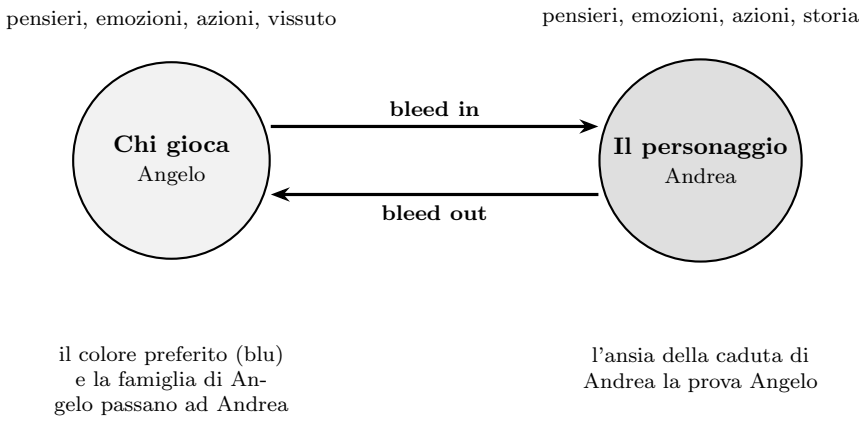


Figure 2.5.2: Le due direzioni del bleed, con gli esempi del nanogioco. Più il bleed cresce, più la linea tra chi gioca e il personaggio diventa trasparente. Entrambe le direzioni sono aree progettabili.

Un'area progettabile. Il punto essenziale è che queste due aree sono **progettabili**: si possono progettare giochi che **facilitano** il bleed e giochi che lo **allontanano**. I giochi molto centrati sul bleed, di solito, lasciano dei **vuoti** nei personaggi, che chi gioca riempie con cose personali; poi il gioco fa leva proprio su quelle cose personali per colpire emozioni molto più legate alla persona che al personaggio. Il nanogioco lo faceva in modo evidente; e lo faceva già *Fat Man Down*, il Jeepform visto nei casi studio, in cui ci si pesa prima di giocare e la persona che pesa di più interpreta il protagonista: "è già un forte elemento di bleed", perché il gioco lega alla persona reale gli aspetti del suo personaggio.

[24] La trascrizione scrive spesso "grid".

Il gioco non è separato dalla vita. Che cosa comporta? Che un gioco può fare **molto male emotivamente**, può toccare cose molto personali, anche dei *trigger*. Siamo abituati culturalmente a pensare al gioco come a qualcosa di separato dalla vita: sto giocando, non sto facendo cose reali, cose che hanno un impatto reale. Le definizioni di gioco che escludono l'impatto reale "sono definizioni abbastanza vecchie, e anche abbastanza ignoranti". Se il gioco può essere uno strumento formativo, didattico, allora può impattare la vita reale: posso imparare cose giocando, e allo stesso modo posso provare emozioni giocando. E le emozioni che provo possono causarmi delle cose: non sono neutrali, e **non si fermano automaticamente con la fine del gioco**. "Non è che, a posto, gioco giocato, rimesso nella scatola, e io non ho più niente di quello che ho vissuto". Ciò che si è esperito in un gioco resta, che lo si voglia o no.

Alcuni generi lo calcano di più e sono più abituati a lavorarci: i giochi dal vivo, i LARP della scena scandinava. È difficile che un gioco da tavolo lasci dei traumi dopo una partita, "a meno che non sia *John Company*, ma quello è un altro discorso".

Non è un segno di debolezza. Il docente ci tiene a una precisazione: chi prova bleed non è impazzito e non sta ragionando male. **Chiunque** può provarlo, anche le persone più centrate ed equilibrate. Alcune frange conservatrici del gioco di ruolo hanno l'approccio per cui "se provi le emozioni del tuo personaggio sei pazzo". "Questo non è il caso".

Approfondimento: personaggio e giocatore. Il bleed è un concetto nato negli studi sul gioco di ruolo nordico, e ha dei parenti nei libri di game design più generali. Schell, parlando dei personaggi e dell'identificazione, osserva che chi gioca tende a proiettarsi nel personaggio e che questa proiezione è una delle leve emotive più potenti del medium; la differenza introdotta dalla lezione è che la proiezione funziona in entrambi i sensi, e che la direzione "dal personaggio alla persona" (il *bleed out*) è quella di cui il designer deve preoccuparsi di più, perché è quella che può restare addosso a chi gioca dopo la fine della partita.

2.5.8 La sicurezza nei giochi

Proprio per questo, negli anni, molti giochi hanno cominciato a introdurre le **meccaniche di sicurezza**. A che cosa serve la sicurezza in un gioco? "A non farci troppo male, anche emotivamente", risponde qualcuno. Certo: vale per mille altri ambienti. Sulle **montagne russe** siamo (si spera) saldamente ancorati al seggiolino; le montagne russe di per sé non implicano essere legati, ma ci agganciano con le sbarre di sicurezza. Senza, potremmo morire, o comunque subire danni reali, non solo a breve ma anche a lungo termine.

2.5.8.1 Le origini: dalla sicurezza fisica a quella emotiva

Nella lezione sui casi studio, a proposito di *Fat Man Down*, il docente aveva accennato a giochi Jeepform molto più pesanti ("qualcuno è andato a cercare? Non era un task, quindi va benissimo se non l'avete fatto"). Non li approfondisce: "è un gioco estremamente pesante", un tema complesso da affrontare in classe, e se interessa se ne può parlare via mail separatamente. Ma ce n'è uno, che la trascrizione chiama ***GR***[25], le cui premesse sono, nel racconto del docente, la causa scatenante delle meccaniche di sicurezza nei giochi.

Nella scena del gioco di ruolo dal vivo di quegli anni (il docente parla di "fine anni Ottanta, inizio anni Novanta") c'erano approcci diversi. Quello **americano** aveva un forte focus sulla sicurezza, ma sulla **sicurezza fisica**: se in un LARP, con un'arma di gommapiuma, ti colpisco al punto da spaccarti un ginocchio e devi andare all'ospedale, nel contesto americano "a te costa uno sfacelo", e le associazioni avevano tutto l'interesse a creare regole perché nessuno potesse denunciarle. Da lì regolamenti molto rigidi, poi diventati standard: quanto deve essere spesso il nucleo delle armi in lattice e gommapiuma, quanta gommapiuma minima ci deve essere attorno, quanto deve piegarsi, quanto è rigido; non colpire in testa, e così via.

La scena **scandinava** rispose: "tutto bellissimo, però io posso fare un sacco di male alle persone *senza neanche toccarle*". *GR* nasce proprio per dimostrarlo: è un gioco in cui, da regolamento, non si possono toccare le altre persone, e che nel regolamento dichiara di non essere pensato per essere divertente, e nemmeno per essere giocato: "fa quello che deve fare anche solo leggendolo, e si capisce al volo quanto male puoi fare a

[25] Il docente non ne spiega il contenuto, e il capitolo rispetta questa scelta riportando il titolo solo come lo dà la trascrizione.

qualcuno". Dopo *GR*, "panico generale della scena", e molti giochi cominciarono a introdurre **metatecniche**, o meccaniche di sicurezza. Gli esempi che seguono vengono quasi tutti dalla scena scandinava, "perdonatemi se non ne ho di più generali": da lì sono poi passati al gioco di ruolo da tavolo e altrove, e ciò che conta adesso è capire *che cosa fanno*.

Prima, durante, dopo. La sicurezza in un gioco **si progetta prima, durante e dopo**. "So che vi verrà da pensare: se mi interessa fare giochi da tavolo, di queste cose non me ne frega niente". Ma, ricorda il docente, da un approccio *cross-mediale* escono cose utili anche per altri giochi; molte di queste tecniche sono già sfociate in altri ambiti, e non è detto che non arrivino al gioco da tavolo. E soprattutto: "il gioco è vostro", siete voi a decidere di che cosa ha bisogno, e alcuni giochi potrebbero aver bisogno di alcune di queste cose.

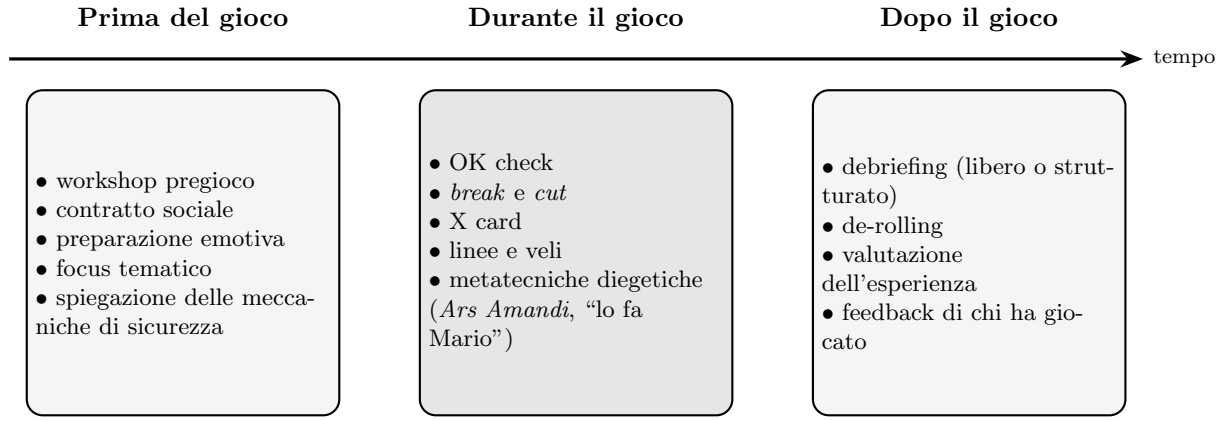


Figure 2.5.3: La sicurezza si progetta prima, durante e dopo il gioco: gli strumenti presentati a lezione, quasi tutti nati nella scena del LARP nordico. Nessun gioco li richiede tutti; il designer sceglie quelli di cui il suo gioco ha bisogno.

2.5.9 Prima del gioco

Prima di giocare si parla di **workshop**, **contratto sociale**, **preparazione emotiva**, **focus tematico** e **spiegazione delle tecniche di sicurezza**. Alcune di queste cose sono specifiche della scena del gioco di ruolo, e vanno spiegate.

Il workshop pregioco. Se ne era parlato a proposito dei black box scandinavi. *Workshop* non è una parola usata solo lì; qui si intendono le attività **pregioco** progettate all'interno dei giochi di ruolo dal vivo, che in Scandinavia **non sono facoltative**: spesso sono obbligatorie per accedere al gioco ("se non fate i workshop, non partecipate"). Servono ad **allineare** le persone che giocano. L'esempio del docente è un gioco che la trascrizione chiama *Sigris Dotter*[26], di alcune designer scandinave che volevano sensibilizzare sul **patriarcato**, e in particolare su certe dinamiche di abuso implicito in comportamenti maschili normalizzati. Per farlo costruirono un'**ucronia** che ribaltava i comportamenti sociali: un gioco dal vivo di **72 ore** in cui il potere sociale è tutto dalla parte delle donne (i personaggi femminili possono andare in giro a petto scoperto, quelli maschili devono coprirsi sempre, e così via). C'era un lungo workshop per assicurarsi che tutte le persone fossero allineate sul tema e sullo scopo del gioco, "anche per evitare che qualcuno andasse lì con lo scopo di rompere le scatole. Lo dico chiaramente". I workshop servono anche a questo.

Il contratto sociale. Chi viene dal gioco di ruolo da tavolo l'avrà sentito, perché è un termine che lì "si è normalizzato moltissimo". È un **accordo su che cosa vogliamo vedere al tavolo**. Se si gioca a *Dungeons*

[26] Titolo non verificabile; è riportato come nella trascrizione.

& Dragons in quattro o cinque, e io voglio solo picchiare gli orchi e prendere il *loot*, mentre un'altra persona è interessata alla parte sociale, alle taverne, alle *quest*, avremo entrambi un'esperienza pessima: per me ci sarà poco combattimento, per lei poco sviluppo sociale, narrativo, di *world building*. Diverso è un gruppo di cinque persone che vogliono solo picchiare orchi, o di cinque persone che vogliono giocarsi tre ore in taverna. Il contratto sociale è semplicemente essere **allineati su che cosa stiamo giocando**. "Dichiarazione di intenti", suggerisce Lorenzo: "esatto, l'hai detta", è un altro dei nomi con cui lo si chiama. Vale anche fuori dal gioco di ruolo: se propongo agli amici "una serata board game tranquilla" e poi io e Gianmarco tiriamo fuori *John Company* mentre gli altri pensavano a *Ticket to Ride*, "finisce che ci accoltelliamo". Meglio dire prima: vogliamo giocare a *John Company*, chi gioca con noi? Così chi si siede sa esattamente che cosa sta per succedere.

La preparazione emotiva. Alcuni giochi la richiedono. Succede spesso in ambito **formativo**, nel *team building* o in giochi su temi delicati: le persone potrebbero non sapere, o non essere preparate al fatto, che stanno per fare un gioco intenso. Si fa allora una preparazione pregioco: "ragazzi, stiamo per giocare un gioco triste, si piangerà parecchio. Ve lo diciamo prima, così arrivate già mentalmente allineati".

Il focus tematico. L'esempio è un gioco di ruolo realizzato anni fa per **Amnesty International**, sulla **pena di morte**[27], che Amnesty ha usato per divulgare il tema attraverso il gioco. Nel manuale c'è una buona parte di **saggi** curati da Amnesty sulla pena di morte, perché si vuole che il focus del gioco sia quello: si arricchisce il tema con contesto culturale, saggi, informazioni, in modo che nessuno possa dire "l'ho giocato, ma non sapevo che parlasse di questo". "Non puoi non saperlo": è chiaro che cosa il gioco vuole fare.

Spiegare la sicurezza. Infine, se un gioco ha tecniche di sicurezza, **vanno spiegate prima** di giocare. "È come se ti dicessi: in questa montagna russa c'è un sistema di allacciamento che la rende sicura, ma non te lo spiego. Anzi, ti do un paracadute e non ti dico come aprirlo". Tanto vale non darlo: un paracadute di cui non si sa come funziona non è utile, non rassicura, non fa stare tranquilli. "E finisce come prima con Angelo. Non bellissimo".

2.5.10 Durante il gioco: le meccaniche di sicurezza

Che cos'è una meccanica di sicurezza? Qualcuno pensa alla **parola di sicurezza**, e uno studente, chiedendo se ne può parlare, nomina il **BDSM**. Sembra che non c'entri niente, dice il docente, e invece c'entra: chi progettava i giochi scandinavi aveva il problema di come introdurre meccaniche di sicurezza, e lo risolse **prendendo da altre comunità** che avevano lo stesso problema. Parlarono con chi si occupava di sicurezza negli ambienti di lavoro pericolosi, con chi stava nella scena BDSM: tutti ambienti con un'attività consensuale ma pericolosa, in cui le persone devono avere strumenti per uscire dall'attività, abbassare i toni, allontanarsi dal problema. E le integrarono in modo cross-mediale.

L'OK check. Una delle primissime, nel LARP scandinavo. Mettiamo che il tuo personaggio sia molto aggressivo verso il mio. Io, *player* e non personaggio, faccio un gesto (il segno dell'OK con la mano, sul petto). Se mi rispondi con un OK, posso continuare. Viene da ambienti di lavoro molto rumorosi, in cui non si può parlare per chiedere "è tutto a posto, siamo pronti?", e quindi si va sul **non verbale**. È uno strumento non verbale per chiedere: posso continuare? Il tema è troppo forte, o posso spingere oltre?

***Break* e *cut*.** Spesso usati insieme. Sono due parole che di solito **non appartengono alla lingua** in cui si gioca: se si gioca in italiano si usano *break* e *cut*; se si gioca in inglese si potrebbe usare "rallenta" e "taglia". Proprio perché non devono essere parole che si possono dire per sbaglio durante una conversazione. *Break* vuol dire rallenta, frena: la persona che sta interagendo con me abbassa i toni. *Cut* vuol dire "è già troppo tardi": mi hai già toccato qualcosa, sto avendo una crisi, devo fermare il gioco. Fermare il gioco può voler dire molte cose a seconda del gioco; spesso, con un *cut*, chi facilita o organizza si prende un momento

[27] La trascrizione riporta il titolo come "Precious" e l'autore come "Jacopo Frigerio"; nessuno dei due è verificabile con certezza.

con chi l'ha chiamato, per assicurarsi che stia bene, e poi si capisce se si può proseguire o se quella persona deve uscire dal gioco.

La X card. È usata soprattutto nel gioco di ruolo **da tavolo**, e in particolare in **fiere e convention**: non tanto in casa con gli amici, quanto dove può capitare di giocare con persone che non si conoscono, e quindi di non sapere quale tema possa creare problemi all'altro. Magari io non voglio vedere violenza sui bambini, tu non lo sai e dici: "andiamo nel villaggio e catturiamo tutti i bambini". La X card è una carta con una X **al centro del tavolo**: chiunque può toccarla, e quando la si tocca la scena va in **dissolvenza al nero** (*fade to black*): si interrompe la narrazione attuale e si salta un po' avanti nel tempo, oltre la scena che stava venendo descritta.

Linee e veli. Tiziana interviene con un'esperienza: in un gioco senza master giocato a Bracciano[28], l'animatore che lo conosceva, quando si toccavano certi temi, chiedeva ai partecipanti: possiamo parlarne? Quanto profondamente? "È la prossima slide". Sono le **linee e i veli** (*lines and veils*), usatissimi in convention e in ambienti pubblici, ma anche tra amici. Di solito sono due fogli su cui si scrive (possono anche essere orali, ma devono essere ben definiti): un gruppo di tematiche *linee* e un gruppo di tematiche *veli*.

Le linee sono tematiche che **non vogliamo entrino in gioco**. Se una linea è la violenza sui bambini, nessuna persona al tavolo, per contratto sociale, la metterà in scena, la descriverà o la porterà in gioco.

I veli sono tematiche che **possono entrare in gioco, ma non si approfondiscono**. Se la violenza sui bambini fosse un velo, potrei dire "vado a rapirgli i figli", ma senza descrivere come, senza entrare nel dettaglio dell'atto: si mettono dei gradi di separazione tra le persone e le tematiche problematiche.

Trigger, fobie, altri *content warning*: persone diverse ne hanno di diversi. Di solito si va al **minimo comune denominatore**: se anche una sola persona mette qualcosa come linea, quella linea vale per tutto il tavolo. E linee, veli e X card di norma **non si sovrappongono al tema del gioco**: se si gioca a un gioco di ruolo sulla malattia terminale, la malattia terminale non può essere né una linea né un velo. Si usano per tematiche contingenti o parallele.

Strumento	**Dove nasce / si usa**	**Come funziona**	**A che cosa serve**
OK check	LARP scandinavo; da ambienti di lavoro rumorosi	gesto non verbale fuori dal personaggio; si continua solo se l'altro risponde OK	chiedere se si può proseguire o spingere oltre
Break	LARP	parola fuori lingua: l'interazione si attenua	rallentare, abbassare i toni
Cut	LARP	parola fuori lingua: il gioco si ferma, chi facilita verifica che la persona stia bene	fermare tutto quando è già troppo tardi
X card	gioco di ruolo da tavolo, fiere e convention	carta al centro del tavolo; toccarla manda la scena in dissolvenza	saltare un contenuto non voluto senza spiegazioni
Linee	convention, gruppi di amici	elenco di temi esclusi; basta una persona per escluderli per tutti	tenere fuori ciò che non si vuole
Veli	convention, gruppi di amici	elenco di temi presenti ma non dettagliati	mettere distanza da ciò che è delicato

Table 2.5.2: Le meccaniche di sicurezza "generiche" presentate a lezione. Sono adattabili a qualunque gioco, e per questo, secondo il docente, a volte più deboli di quelle legate al tema del gioco.

[28] La trascrizione dice "l'Overcraft Esk a Bracciano": il titolo del gioco non è ricostruibile.

Un via libera, non un freno. Si pensa spesso, sbagliando, che questi strumenti servano solo a proteggere le persone dai temi problematici. In realtà **sono un via libera**: una volta tolto ciò che è problematico per qualcuno al tavolo, ci si sente più al sicuro nell'affrontare *il resto*, anche in profondità, specie su temi delicati. Se in un gioco sulla malattia terminale nessuno ha messo il *body horror* come linea o velo, posso andare nel dettaglio, perché magari è un aspetto che voglio esplorare per vedere come si relaziona con il resto. "Mi rendo conto che in Scandinavia è un argomento comunissimo, ma in Italia ancora poco: fatemi domande".

2.5.10.1 Le domande su linee e veli

Si possono cambiare? Gianmarco chiede che cosa succede ai temi "abilitati". Il docente aggiunge una cosa che non aveva detto: linee e veli **non sono scritti sulla pietra** prima del gioco, **si possono aggiornare** in corso d'opera: "attenzione, non avevo pensato a questo tema, lo vorrei come velo, lo aggiungiamo adesso".

Non limitano la libertà? Gianmarco insiste con un esempio "un po' antipatico": chi gioca non potrebbe vedere queste cose come una limitazione della propria libertà, fino a non voler più giocare? È possibile, risponde il docente, ma questi non sono elementi che decide il game designer: **emergono dal gruppo**. Nessun gioco mette nel manuale le linee e i veli; al massimo mette lo *strumento*, dicendo: "gruppo di gioco, decidete voi che cosa ci va dentro, se vi serve". E poi, aggiunge: "stiamo parlando di giochi, gente. **Tutti i giochi limitano la nostra libertà**". Nel gioco dell'oca potrei portare la mia pedina subito all'ultima casella; non lo faccio perché c'è una regola che me lo impedisce. Accettiamo, nel contratto sociale, una limitazione per avere un'esperienza specifica. È il **cerchio magico**: sul ring di boxe posso tirarti un pugno sui denti, e va bene; se lo faccio per strada mi arrestano.

***Obscura* e la scelta del tono.** A Giordano viene in mente un gioco di ruolo horror ispirato agli horror anni Novanta e al *found footage*[29], in cui il designer scrive esplicitamente di decidere col gruppo se fare un horror cupo, di terrore, o una commedia horror. "Assolutamente". E il docente aggiunge un altro livello: questi elementi sono **informativi** prima ancora che limitativi. Conosce molte persone grandissime fan degli horror che però non sopportano i *jump scare*, perché li trovano una soluzione un po' *cheap*: sapere in anticipo se un film ne ha non impedisce loro di vedere niente che non vedrebbero comunque, anzi, dà loro la tranquillità di vedere tutto il resto. E fa un esempio personale: per un **disturbo alimentare** di cui soffre (l'ARFID), fa molta fatica a giocare scene di roleplay incentrate sul cibo; per lui il cibo è spesso un **velo**, "non perché sia un tema pesante: il cibo non è un tema pesante". Se vuole giocare con tranquillità, anche un gioco sulla malattia terminale, può non voler pensare a dinamiche legate al cibo in quel momento. Queste meccaniche "non castrano chi gioca": **eliminano le preoccupazioni non necessarie** e permettono di spingere di più sui temi che il gioco vuole affrontare. "È come giocare un LARP di due ore sapendo che c'è un bagno disponibile, invece di non saperlo e doversi tenere tutto: se so che c'è, vivrò più rilassato anche tutto il resto".

2.5.10.2 Le metatecniche diegetiche

Le meccaniche di sicurezza, nel tempo, sono diventate anche **diegetiche**: si è provato a legarle ai temi dei giochi in cui vengono usate. Linee, veli e X card sono molto generiche, adattabili a qualunque gioco, "e proprio per questo spesso più deboli di altre".

Ars Amandi. In Scandinavia, con giochi già incentrati su *slice of life*, vita quotidiana e interazione sociale, ci si chiese: possiamo fare un gioco di ruolo che metta in scena la dinamica dell'**intimità** e del sesso? All'epoca, quando due personaggi avevano un momento romantico o intimo, nel gioco dal vivo la cosa era molto imbarazzante, "un ce la raccontiamo", strano per un LARP. Alcune designer svedesi crearono allora ***Ars Amandi***[30], una metatecnica: quando i personaggi vogliono un momento di intimità, non si vuole una meccanica che descriva che cosa fanno, ma una che faccia **provare quel tipo di emozione**, "senza chiaramente che vi mettiate a fare sesso sui tavoli, ve lo dico molto chiaro". Consiste nel toccare l'altra

[29] La trascrizione dice "Obscure"; il titolo esatto non è verificabile.
[30] La trascrizione dice "Lars Amandi". La tecnica è attribuita alla designer svedese Emma Wieslander.

persona **dai gomiti alle mani, e sul collo**, guardandosi negli occhi. Vista da fuori è molto imbarazzante, ma da dentro restituisce un po' di quell'imbarazzo e di quella sensazione di intimità con un'altra persona. La Scandinavia ha prodotto "dozzine e dozzine" di meccaniche per simulare l'intimità tra personaggi.

"Lo fa Mario". Un'altra, "ultra-diegetica, forse una delle mie preferite", viene dal designer norvegese **Eirik Fatland**[31]: **Mario**. Il gioco è ambientato in una famiglia criminale, in stile *Il Padrino*; i personaggi sono persone poco raccomandabili, destinate a compiere azioni "non proprio eticissime", ma non sono l'ultimo gradino della famiglia. L'ultimo gradino è Mario, lo scagnozzo più in basso di tutti. **Mario non lo gioca nessuno**: esiste come personaggio, ma nessuna persona lo interpreta. Ed è una meccanica di sicurezza: se al tuo personaggio viene ordinato di fare qualcosa di terribile ("porta questo in bagno e picchialo finché non confessa"), si può dire **"lo fa Mario"**. Di fatto se ne occupano gli organizzatori, e la cosa che deve succedere, nella fiction, succede; nessuno si chiede come. "Le cose che vogliamo fare le facciamo; le cose che non vogliamo fare le fa Mario". Si delega a Mario ciò che non si vuole trattare.

2.5.11 Dopo il gioco

C'è anche un **post-gioco**: **debriefing**, **de-rolling**, **valutazione dell'esperienza**, **feedback** di chi ha giocato.

Il debriefing. Chi ha giocato LARP ne ha visto qualche forma. Tiziana, che ne ha giocati, lo riassume come "dire com'è andato il gioco". Semplificando molto, sì: dopo il gioco, **fuori dal personaggio**, le persone che hanno giocato si trovano per parlare del gioco. Può essere **destrutturato** (com'è andata, una chiacchierata con la persona con cui si è giocato, o andare a dire "scusa" alla persona il cui personaggio il nostro ha trattato malissimo), oppure **strutturato**, con procedure che aiutano a organizzare la conversazione, specie nei giochi per molte persone: un unico cerchio di 300 persone che si parlano "non è proprio liscissimo", per cui sono comuni i debriefing di gruppo, con sottogruppi.

Il de-rolling. È spesso incluso nel debriefing: sono le attività con cui si **esce dal personaggio**. Se si gioca un LARP di due o tre giorni di fila, si incorporano senza accorgersene aspetti *cinestetici* del personaggio, "di nuovo il bleed". Se per tre giorni si gioca un personaggio che zoppica, "buona fortuna a tornare subito alla vostra camminata normale"; e quello è un esempio stupido: si pensi a un personaggio paranoico, giocato per tre giorni.

L'esempio personale del docente: a ***Bunker 101***, un gioco di ruolo dal vivo italiano di tre giorni, in un vero bunker, in stile *Fallout*, era uno dei responsabili della sicurezza del bunker. Ore e ore dopo la fine del gioco, tornato in città, di notte, vede un camion di operatori ecologici con le tute anti-biohazard; si avvicina e chiede: "tutto a posto?". "Non avrei dovuto farlo. L'ho fatto perché per tre giorni era quello che facevo regolarmente, e non c'era stato un de-rolling". Un esempio che fa ridere; ma si pensi a un gioco in cui si interpreta un personaggio odioso, detestato da tutti, e nessuno fa de-rolling. Il docente conosce persone che sono state malissimo, e che non giocano più LARP, perché hanno avuto la sensazione di essere ancora quella persona orribile, "perché il contesto sociale non le ha separate dal loro personaggio".

Come si fa, concretamente? Tiziana chiede quanto tempo ci si dedica e come lo si fa. "Questo è un vaso di Pandora". Ci sono articoli scritti apposta, di **Mo Holkar** e di **Nina Runa Essendrop**[32]. Il tempo dipende molto dal gioco: un gioco di un'ora potrebbe avere un debriefing di dieci minuti, uno di tre giorni fasi di de-rolling più corpose. Non c'è universalità, anche perché non tutti i giochi sono problematici: è facile immaginare queste cose su giochi dai temi delicati, ma un gioco potrebbe non aver bisogno di un debriefing strutturato. "Nessuno gioca a *Catan* e ha bisogno di un debriefing". "A *John Company* servirebbe, secondo me", ribatte qualcuno. Per gli esempi concreti il docente rimanda a fine lezione, per non saltare le slide (la risposta è nella sezione 2.5.14).

[31] La trascrizione dice "Eric Fatland". Il gioco in cui nasce è riportato come "Michel Scheller", titolo non ricostruibile.
[32] La trascrizione dice "Moholkar", "Mo Holker" e "Ninero con Essendrop", "Niruma S. Dop".

La valutazione dell'esperienza. È un altro strumento post-gioco: chiedere alle persone **come è stata l'esperienza**. Non "è stato bello o brutto?", e soprattutto non "è stato divertente?": "questo non fatelo mai, se no sbrocco". Si usa molto in ambito **formativo**, quando serve capire che impatto ha avuto il gioco: dopo un LARP o un gioco per le scuole su un certo tema, si vuole sapere che cosa le persone hanno capito e che cosa no. Non è un debriefing e non è un de-rolling: è capire *che cosa è arrivato*. I **feedback** di chi ha giocato, infine, sono utilissimi soprattutto in fase di **playtest**, per capire che cosa calibrare: se ne parlerà nelle lezioni dedicate.

2.5.12 Il 200 Word RPG Challenge

Il tempo è poco; il docente chiede se va bene proseguire senza un'altra pausa ("non sono un poliziotto: se qualcuno vuole una pausa, alzate la mano"). Prima di dire che cosa si dovrà fare per il 25, un salto verso un tema che "vi sembrerà scollegato dalla progettazione delle emozioni; ma se tutto va bene, con il task vi sarà chiaro esattamente perché". È il **200 Word RPG Challenge**, già accennato nei casi studio: giochi di ruolo di ogni tipo (da tavolo, LARP, giochi pervasivi urbani) accomunati da un solo vincolo, un **regolamento di 200 parole**. Le slide ne riportano diversi per intero, "dal più regolare ai più *deranged*, fino ad alcuni che secondo me sono illegali da giocare"; tutto quello che c'è scritto nella slide è il regolamento completo, e non serve altro.

Nature Planet. Un gioco di ruolo da tavolo per quattro persone che interpretano una troupe di **documentaristi**, con ruoli asimmetrici (la guida, il producer, il cameraman...). Le scene sono ostacoli alla produzione del documentario: quelle superate con successo finiscono nel documentario, quelle fallite nei *bloopers*. "Fine".

Heist. Si fa un colpo, appunto. Può andare bene o male, e se va male non è detto che vada male per tutti: si può riuscire a scappare e cavarsela lo stesso. Usa un mazzo di carte francesi.

Snitch. Ci hanno beccati: siamo una banda che ha fatto qualcosa di illegale, la polizia ha delle prove ma non sappiamo quante. Ci interrogano **separatamente**, in scene uno a uno, mentre gli altri aspettano in un'altra stanza; durante l'interrogatorio si può vuotare il sacco incolpando gli altri, sapendo che potrebbero fare lo stesso. "Un po' dilemma del prigioniero, un po' gioco di ruolo".

New Gods of Babel. "Per me incredibile". Si è divinità che plasmano un mondo, con una **torre di Jenga**. Al proprio turno si può spostare un blocco in cima per aggiungere un dettaglio al mondo; spostarne due per cambiare un dettaglio creato da un altro dio; rimuovere un blocco senza rimetterlo, togliendolo per sempre, per eliminare un dettaglio altrui; oppure ritirarsi a osservare, senza creare più nulla. Se la torre cade, tutte le divinità ancora impegnate nella creazione vengono cancellate dall'Olimpo, per la loro *hybris*.

One Night Stand. Di un autore italiano[33]: un gioco di ruolo per due persone su un incontro intimo di una notte, giocandone il prima, il dopo, la scena.

Paperwork. Simula la **burocrazia**, per due persone: una gioca un cittadino che vuole un'informazione, l'altra un impiegato comunale che non gliela vuole dare. C'è un **timer**: se finisce, l'ufficio chiude e il cittadino perde; se il cittadino sbaglia uno dei compiti assegnati dall'impiegato, deve tornare il giorno dopo, e perde. L'impiegato perde se il cittadino ottiene l'informazione. "Un gioco di ruolo in cui si può vincere o perdere. Una distopia burocratica, forse per molti neanche troppo distopia".

Sidewalker. Un gioco pervasivo urbano: si esce su un **marciapiede**, se ne dichiara una porzione territorio proprio, una **micronazione**, e se ne è ministri. Ai passanti si può chiedere di unirsi: se accettano diventano nuovi cittadini e ricevono un ruolo (ministro dei lavori pubblici, per esempio); se ci ignorano del tutto, sono **disastri naturali**, e bisogna tirare per il terremoto, lo tsunami e così via.

[33] La trascrizione dice "Stefano Burke": il cognome non è verificabile.

8525 kHz. Un gioco sulle **number station**, le stazioni radio che trasmettevano solo numeri (chiaramente codici per agenti sul campo; su YouTube ce n'è un grosso archivio). Si interpretano agenti segreti: si apre una di queste registrazioni, si segnano i numeri su un foglio e, "in stile CIA", si cancellano con un pennarello i numeri usati per le prove. Un 9 è molto buono, un 1 molto cattivo; ma chi usa solo i 9 resta con i numeri peggiori, e bisogna capire quando conviene fallire una prova per tenere i numeri alti per dopo.

The Phone Call of Death. "Secondo me è illegale, ma ve lo dico lo stesso"[34]. Un gioco di ruolo senza master, a tema *Saw*: si è persone intrappolate in trappole mortali. Si scrive il numero di telefono di una persona cara che si chiamerebbe in difficoltà, **lasciando vuota l'ultima cifra**; al proprio turno si tira un d10 per completarlo e si chiama. Se non risponde nessuno, si è ancora in trappola ma si sopravvive. Se qualcuno risponde, bisogna descrivere, in personaggio, la propria trappola e chiedere aiuto per uscirne; e si conta dopo quanto la persona riattacca. Prima di 10 secondi: si muore nella trappola. Da 11 a 20: una ferita grave, da descrivere. Da 21 a 30: una ferita più lieve. Oltre 30: si è fuggiti. E poiché lo scopo del "genio del male" è insegnare un'importante lezione di vita, chi scappa descrive quale lezione ha imparato. Il gioco finisce quando tutti sono morti o sopravvissuti: "non ci sono vincitori, solo sopravvissuti". "Non fatelo".

2.5.13 Il secondo task: un gioco di ruolo in 200 parole

"Come forse avete intuito", il task è scrivere un **gioco di ruolo in 200 parole**. Le regole, riunite:

- il task è **individuale**, "in solitaria" (ci si può confrontare tra compagni, ma ognuno fa il proprio documento: "vi assicuro che possiamo vedere chi crea e chi modifica i file");
- **200 parole al massimo**, "e sarò rigidissimo": è un *hard limit*, "anche 201 per me è problematico, e vi dirò la prossima volta perché"; si possono usare meno parole. Il **titolo**, se è dentro il testo, conta; il nome dell'autore conta se è nel testo, non se è nel nome del file;
- un **Google Doc** nella cartella del task, **solo testo**: niente illustrazioni ("scusa, Rita, lo so che ti aiutano: non hai sbagliato a metterle nella scheda, è che nel 200 Word RPG il gioco deve essere solo testo"), nemmeno un cambio di font; si tiene quello di default (Arial 11); punteggiatura e a capo vanno benissimo, "spero che li usiate". Il conteggio sarà quello dello strumento *word count* di Google Docs;
- nome del file uniforme: **nome del gioco, trattino, nome della persona** ("sul primo task sono stato permissivo, ma non potete strutturarmi i nomi dei file in dieci modi diversi");
- un **regolamento completo**, non un *pitch*: "se lo seguo dall'inizio alla fine, posso giocarlo";
- scegliere **un'emozione** dalla **ruota delle emozioni** proiettata a lezione ("ce ne sono mille; questa è quella che uso di più nei playtest, è la più ricca"), in inglese ma liberamente traducibile; precisa, specifica e focalizzata, quindi presa dal **cerchio esterno**, non dai due interni;
- il gioco deve far **spuntare quell'emozione giocandolo**, **con le meccaniche e non con il setting**, e l'emozione **non va mai nominata** nel regolamento;
- consegna **per il 25** (venerdì 25 settembre); all'inizio della lezione successiva si guarderanno insieme i giochi, come oggi le schede.

"Mi rendo conto che può suonare difficilissimo, ma ho fiducia in voi, e avendo visto le vostre schede personaggio non ho dubbi che ci riusciate".

[34] La trascrizione riporta il titolo come "Axo the Phone Call of Death" e lo presenta come "il gioco di ruolo dell'Enigmista", cioè il villain della saga di *Saw*, noto in Italia con quel nome: la prima parte del titolo non è ricostruibile.

I consigli. "Siete tutti ammutoliti". Qualche suggerimento per partire:

- guardare con calma i **casi studio** in fondo alle slide e chiedersi, per ciascuno, *che emozione vuole dare questo gioco?* (con *The Phone Call of Death* è facile, con altri meno);
- esplorare il **sito** del challenge, linkato nella slide: ci sono circa **5000** giochi, non tutti belli, ma era un concorso con premi, e i **vincitori** sono tendenzialmente giochi interessanti e molto chiari in ciò che vogliono fare;
- **non ricopiarli**: gli esempi sono volutamente diversissimi tra loro. "Fare design non è seguire la tradizione. Non mettetemi il master e i dadi solo perché sì". 200 parole sono poche, volutamente, "eppure più di 5000 persone ci sono riuscite";
- lasciarsi **ispirare dal personale**: scegliere un'emozione mai provata è molto difficile; una provata più volte, di cui si sa come la si vive, semplifica il lavoro di molto.

Ancora sul nanogioco. Tiziana torna sul paracadute: con due minuti per pensare a un personaggio, la cosa più semplice per lei sarebbe stata parlare di sé, "perché mi conosco", anche se non avrebbe saputo rispondere a tutte le domande. "Ma il gioco è fatto apposta": rispondere a tutto è molto difficile, e la maggior parte delle persone si schianta prima. Il gioco **facilita il bleed**: in un momento di crisi e di adrenalina si tende a riempire i buchi con cose personali, che non bisogna inventare perché le si sa già. "Quando ti chiedo il colore preferito, non ne crei uno da zero: è più facile dire il tuo". Alcune persone scivolano nel bleed più facilmente di altre; in certe partite, senza saperlo, qualcuno ha messo al centro del proprio personaggio pezzi molto vicini al proprio vissuto.

Come si fa il documento. Uno studente ha un Mac e non ha Word. Nessun problema: il docente condivide lo schermo e mostra la procedura nella cartella "200 Word RPG" (vuota): tasto destro, *Google Docs*, *Blank document*; il font di default è Arial 11, e va lasciato così. Nel nome del file, nome del gioco e nome della persona; il titolo può anche non essere ripetuto in alto. Nel menu *Strumenti* c'è il conteggio parole: "guarderò quel valore lì". "Pulito, liscio, semplice. Sono 200 parole, ci mettete poco a scriverle. Chiaramente, la partita del task non è scrivere 200 parole".

Dove comincia il gioco di ruolo? Gianmarco ("una domanda estremamente stupida, ma io mi conosco") sa che tenderà a fare qualcosa di molto meccanico, più vicino al gioco da tavolo: dove inizia l'RPG? Il punto chiave è che il regolamento deve essere **completo**. Si possono chiedere **materiali di uso comune** ("tira un dado", "vi servono un foglio e una matita": va bene, ma va detto); "prendete un mazzo di carte da poker" va bene; "c'è un mazzo di 100 carte" o "pescate una carta imprevisto" no, se quelle carte non ci sono. (Nel frattempo rientra Rita, che si era disconnessa: "nessun problema, la lezione è registrata".) Gianmarco teme di fare "solo regole", senza la parte di narrazione. Ma, attenzione: un gioco senza setting, senza *world building*, con sole meccaniche "schiette e fredde", non è un gioco che non dà emozioni. **Nascondino e scacchi** non hanno chissà che ambientazione, eppure danno due esperienze emotive diversissime; lo stesso vale per *Forza 4* e gli scacchi. Il focus è l'emozione.

> Io non guarderò il setting. Mi rendo conto che è un *hot take*, ma il mio take è che se stiamo facendo un gioco e la nostra emozione viene data solo dal setting, è un gioco debole. Se la risposta è "potevo scrivere un romanzo, un racconto, invece di un gioco", il gioco è debole. Qui stiamo facendo game design: vogliamo che impariate a dare emozioni con le meccaniche di gioco. Se no andiamo a scrivere romanzi, a creare canzoni, a fare quadri. Non è sminuente: che figata che c'è gente che fa emozioni con quelle cose. Ma qui ci occupiamo di game design.

C'è un limite? Un'ultima domanda, "per porre dei paletti, perché la testa mi parte per la tangente": sembra che si possa fare tutto ("sei giocatori, ci dividiamo in coppie, prendono oggetti a caso e organizzano una scena..."). È *limitless*, risponde il docente, "con un grosso asterisco": deve essere un gioco che si spiega **completamente in 200 parole**, e ci si accorgerà presto che non tutti i giochi, né tutti i generi, ci stanno. Uno strategico in stile GMT, o un *John Company*, è praticamente impossibile: "non prendetela

come una sfida personale". Il consiglio è restare **focalizzati**: scegliere un'emozione e provare a darla con le meccaniche. I giochi più grossi difficilmente danno una sola emozione, ma qui si parte dal piccolo, e già una sarà complessa. "Devo essere in grado di leggere il regolamento e capire che emozione mi state dando, con le meccaniche".

Approfondimento: emozioni dalle meccaniche. Il task mette alla prova l'idea che attraversa tutta la lezione, e che ritorna nei libri di riferimento. Koster insiste sul fatto che le regole di un gioco sono il suo vero contenuto, e che ciò che un gioco "insegna" e fa provare sta nei suoi sistemi più che nel suo rivestimento: la grafica e l'ambientazione colorano l'esperienza, ma da sole non la cambiano. Schell, con la lente dell'emozione, chiede al designer di partire dall'emozione desiderata e di chiedersi quali elementi del gioco la producono davvero. In 200 parole non c'è spazio per il rivestimento: o l'emozione è nelle meccaniche, o non c'è.

2.5.14 Le domande finali

2.5.14.1 Workshop e debriefing in concreto

Il docente torna alla domanda di Tiziana. C'è un sito, **Nordic Larp**, con moltissimi articoli di design, e in particolare un articolo di **Mo Holkar** ("per me un capolavoro") su come strutturare il **workshop pregioco**, con una comodissima tabella: per ogni possibile parte del workshop, lo scopo, se includerla o no (con condizioni del tipo "sì, se il gioco ha questa caratteristica"), esempi da altri giochi, e il focus, cioè se quella parte riguarda il personaggio o la persona che gioca, e se lo scopo è pratico (le aspettative sul gioco) o di sicurezza (la spiegazione delle meccaniche di sicurezza). "Molto pragmatico". Ma, di nuovo, è nato nella scena del gioco di ruolo: "prendete quello che vi serve, adattate il resto". Sul sito ci sono "tonnellate" di articoli anche su debriefing e de-rolling.

Un esempio dal suo lavoro. Il docente ha "molte remore" a parlare dei propri giochi, "perché non voglio sembrare il professore che parla dei propri libri"; ma sul workshop sente di poter fare un esempio concreto, perché sa che cosa ha fatto e perché. Mostra il documento di un suo **black box**[35], nato al **Black Box Copenhagen** nel **2016** e poi organizzato in vari eventi in Europa e in Italia (a metà spiegazione la connessione salta per qualche secondo: "la mia connessione è instabile"; "va bene, anch'io sono instabile"). Usa una **metafora** per parlare di **relazioni abusive**, con lo scopo di far capire che esistono relazioni abusive **non violente**: quelle violente sono, tra virgolette, facili da vedere; quelle non violente sono più difficili da riconoscere, ma comunque problematiche. Essendo un tema delicato, ha un **workshop obbligatorio** di un'ora e un quarto, un'ora e mezza: le prime sette pagine del documento sono il workshop strutturato, l'ultima è il **debriefing**, circa mezz'ora dopo il gioco.

Il setting: una delle **esposizioni universali** dell'Ottocento, in chiave un po' *steampunk*; fino a **20 persone**, che giocano **in coppia**. Metà interpretano i **creatori**, inventori e scienziati, l'altra metà gli **automi** che i creatori hanno costruito: automi con una coscienza propria, creati però con uno scopo preciso e **posseduti**, letteralmente, dai loro creatori. È un gioco molto introspettivo, con molti monologhi interiori e molte scene di silenzio (*inner play*, che si vedrà più avanti nel corso). Il debriefing è strutturato in tre momenti:

1. un **debriefing solitario**, che include il **de-rolling**: poiché il gioco impone molte limitazioni fisiche, soprattutto agli automi, si fanno **scuotere i corpi**, chiudere gli occhi, immaginare il proprio personaggio **da fuori** e farlo allontanare; e gesti pragmatici come **togliersi il nome del personaggio e buttarlo**. "Lette qui sembrano cose super imbarazzanti; ma sul momento, quando hai appena vissuto una cosa del genere e stai piangendo a dirotto, questi appigli per tornare fuori dal personaggio danno una mano gigantesca";

2. un **debriefing strutturato di coppia**: il gioco ha meccaniche fortemente asimmetriche anche in termini di potere sociale (dei creatori sugli automi), e quindi, prima del cerchio di gruppo, creatore e automa hanno alcuni minuti **in privato** per parlare e analizzare le interazioni avute tra loro. Il facilitatore li guida da fuori, dando tempi e indicazioni, "ma non vado a farmi gli affari loro";

[35] La trascrizione riporta il titolo come "Critosphere": la grafia esatta non è verificabile.

3. infine il debriefing si allarga al **gruppo**, e lì si trattano le tematiche ombrello, le interazioni fuori dalla coppia, le dinamiche sociali viste dall'alto, gli scopi e il focus del gioco.

Il documento, nota il docente, è "brutto", ed è giusto così: serve a lui per ricordarsi che cosa far fare; non è un manuale venduto al pubblico, deve essere funzionale e pratico per chi facilita. E in questo caso, dato il tema, è lui stesso a facilitare.

Da dove vengono queste tecniche? Tiziana chiede se vengano dal mondo del game design o se siano importate da altri mondi. Il docente riapre i materiali dei casi studio: il LARP nordico prende da moltissime altre cose. Il debriefing e il de-rolling vengono dallo **psicodramma**, dal **sociodramma**, dall'uso psicologico del gioco di ruolo; le meccaniche di sicurezza dagli ambienti di lavoro pericolosi e dalla scena BDSM. È una scena che, quando ha un problema di design, va a cercare le soluzioni in altre scene con problemi simili: "abbiamo bisogno di una meccanica che crei sicurezza tra persone che stanno facendo qualcosa di consensuale ma pericoloso: dove la troviamo? Andiamo a parlare con chi fa altro, altrove". E non è diverso dal resto del mondo ludico: il gioco da tavolo prende dal videogioco, il videogioco dal gioco da tavolo. "Il game design non è chiuso in un barattolo".

2.5.14.2 Il debriefing nei giochi da tavolo

Restano dieci minuti, "meno di quelli che di solito voglio per le domande". "Se nessuno chiede niente divento super imbarazzante e comincio a parlare troppo". "Se no finiamo schiantati al suolo, o ricominci con le 40 domande".

Uno studente commenta che un momento di debriefing servirebbe anche in alcuni giochi da tavolo: "è una tematica che non si tocca mai". "Sfondi una porta aperta". Molte persone reagiscono male a questa parte perché non ne vedono lo scopo; e non tutti i giochi ne hanno bisogno; ma non è detto che anche giochi da tavolo non possano beneficiarne. "Voglio che facciate le cose non per come sono sempre state fatte, ma per quello che serve al vostro gioco".

Tiziana si chiede chi, concretamente, lo farebbe: le viene in mente l'ambito formativo ed educativo, dove è normalizzato. La scena scandinava, risponde il docente, l'ha risolto rendendolo **obbligatorio**: se partecipi, lo fai. Nel gioco da tavolo è più complesso. Alcuni giochi però fanno qualcosa di simile:

- i **giochi investigativi da tavolo** (non solo *Sherlock Holmes Consulting Detective*) hanno una fase di fine gioco con una procedura precisa, che serve a capire com'era la storia, a spiegare i retroscena e a dare al gruppo una valutazione della prestazione: un debriefing in forma più tematica;
- ***Fog of Love***[36], un board game (non un gioco di ruolo, ed è pubblicato anche in italiano) per due persone, in cui si gioca una coppia che comincia a frequentarsi. Ha un sistema di fine partita "incredibile": non si vince né si perde, non è né cooperativo né competitivo. Durante la partita si possono **scartare tipologie di finale**, e si arriva alla fine con il finale che si vorrebbe, o che si può avere; ma **ognuno ha il suo**. Può finire che io abbia soddisfatto il mio finale e l'altra persona no, ma la coppia stia ancora insieme; o che io sia soddisfatto ma la coppia si lasci. "È molto più granulare e sfumato". È danese: "torniamo alla Scandinavia, siamo sempre lì: per fortuna con la mente, purtroppo perché il corpo è in Italia".

Un altro studente racconta che, da master, dopo le sessioni fa sempre una valutazione e raccoglie i feedback dei giocatori, quando c'è tempo. Il gioco di ruolo lo fa più del gioco da tavolo, anche per abitudine, conferma il docente; ma un conto è farlo perché si vuole, un conto è seguire un **workflow strutturato** pensato per essere utile al gioco, come quelli degli articoli citati.

2.5.14.3 Le persone prima dei giochi

La chiusura è un principio di metodo. Nelle sue lezioni, il docente vorrebbe che gli studenti progettassero pensando all'**ideale** del proprio gioco, al miglior gioco possibile: "vorrei che nel vostro gioco non ci fosse bisogno, da parte di chi gioca, di inventarsi procedure esterne per risolvere un problema che voi non avete

[36] La trascrizione dice "Fogo Flow" e "Focus Flop"; il docente poi ripete il titolo corretto.

risolto". Se ci si occupa di queste cose prima di chi gioca, gli si dà una mano. E di nuovo: **non sono ostacoli al giocare, sono strumenti per giocare più a fondo**. Uno dei suoi giochi di ruolo preferiti degli ultimi anni ("credo che fossero sei o sette anni che non ne giocavo uno così bello, anche solo da leggere") è un gioco sui ***death game***[37], sul modello di *Zero Escape*, *Danganronpa*[38], *Squid Game*, *Battle Royale*: adolescenti costretti a uccidersi in un gioco mortale in cui sono finiti. È molto intenso, e ha **una pagina di content warning**, meccaniche di sicurezza, linee e veli; e tutto questo "ti aiuta a spingere di più sul tema, non ti frena".

> Sembra scontato, ma purtroppo alcune persone se lo dimenticano: le persone sono più importanti dei giochi. Le persone che giocheranno ai vostri giochi sono più importanti dei vostri giochi; le persone che giocano ai miei sono più importanti dei miei giochi. Chi fa game design ha la responsabilità di prendersi cura delle persone che stanno giocando. Non vuol dire non far giocare cose tragiche, terribili, intime, personali. Anzi: fuori dall'orario di lavoro faccio giochi devastanti, in cui la gente piange sempre. Ma l'obiettivo dovrebbe essere prendersi cura delle persone che giocano.

Sono le otto. Il task c'è, le slide sono già nella cartella ("grazie a un sistema di specchi, leve e trucchetti"); per le domande ci sono la mail e Telegram ("su Telegram di solito rispondo prima"). "Spero che sia stata una lezione utile; mi rendo conto, un po' strana, ma le lezioni con me sono così, abituatevi". La prossima volta, "se tutto va bene" con lui e con Beatrice, si parlerà di **ricerca e sviluppo**, dopo aver guardato i task. "Un saluto anche a chi è a casa e sta recuperando la lezione".

2.5.15 La dispensa: dalla ricerca al design

Alla trascrizione della lezione segue, nello stesso file, la trascrizione di una **dispensa video**: una lezione registrata da sola, senza studenti, e caricata sul Drive del corso. Il docente non si nomina, ma è con ogni probabilità **Daniele**[39]. È stata registrata una **domenica mattina** ("ci sono i pranzi da preparare"), e il docente dice che "domani è il 5 ottobre": quindi **domenica 4 ottobre 2026**.

Si scusa per il ritardo ("sono stati giorni molto, molto frenetici"), ma la dispensa dovrebbe arrivare "più o meno nel periodo ideale" per essere vista. È dedicata alla **ricerca e sviluppo**: il **design thinking**, il **brainstorming**, come presentare le idee, e una parte sui **pitch** di varie tipologie. Il consiglio è di guardarla **subito dopo una sessione di brainstorming**: nella sua esperienza la parte sul brainstorming si capisce meglio se si è appena visto un brainstorming *non strutturato*, poi si vedono questi strumenti, e poi se ne fa uno *strutturato*, toccando con mano la differenza. Avverte che farà qualche digressione ("rimanete con me"); le domande vanno scritte su **Discord** o **Telegram**, ricordandosi di **taggarlo**, "altrimenti faccio fatica con tutte le notifiche". Sarà a lezione il **12 ottobre** per i **pitch**, la settimana dopo. La registrazione è segnata da un forte mal di schiena ("ho proprio il colpo della strega", "ho preso una pastiglia"), da una breve pausa e da un'espulsione improvvisa dalla piattaforma di registrazione.

2.5.16 Design thinking, Agile e design sprint

"Iniziamo dalla ricerca al design". La prima parte spiega perché la ricerca serve al design, e qual è la base teorica di un approccio particolare, il design thinking; "non è una lezione di storia", ma serve a introdurre uno strumento molto utile per capire come funzionano le fasi di ricerca e di design: il **Double Diamond**.

Il design thinking. È una disciplina nata tra gli **anni Ottanta e Novanta** negli Stati Uniti, per il **design di prodotto**: voleva combattere un design molto industriale, poco legato all'utente e al perché

[37] La trascrizione riporta il titolo come "The Spirit Dilemma": non è verificabile.
[38] La trascrizione dice "Dungeon Romp".
[39] Gli indizi: parla di Ruggero in terza persona ("e Ruggero ve lo potrà dire", "poi Ruggero vi farà la parte dei playtest"); dice di avere "una lezione completa nel corso di narrative"; lavora in consulenza di *branding* su prodotto (Daniele si era presentato come *brand advisor*); annuncia "una lezione dedicata al role playing e alla fiction, più o meno verso novembre", che è la sua parte del corso; e chiede di scrivergli su Discord o Telegram, i suoi canali.

l'utente dovrebbe usare un prodotto. Già nella prima versione ha le sue **cinque fasi**, in cui chi sa che cosa vuol dire fare un gioco riconoscerà delle sovrapposizioni:

Empathize: si empatizza con l'utente, si cerca di capire a che cosa gli serve il prodotto;

Define: si definiscono i problemi che il prodotto deve risolvere, i vantaggi che offre, i *pain point* dell'utente;

Ideate: dopo la ricerca e la definizione, si idea una soluzione;

Prototype e **Test:** si fa il prototipo, lo si testa, e si torna indietro, in un ciclo continuo.

Lo hanno reso popolare realtà come **IDEO**[40] e la **d.school**. IDEO è ancora attiva e fa **human-centered design**, che per il docente è "il nome corretto della parola *gamification*"; sul suo sito (*Design Thinking by IDEO*) ci sono molte risorse, tra cui lo *starter kit* della d.school. Il design thinking è, a suo parere, "la punta di diamante" del design di prodotti ed esperienze da un punto di vista funzionale: **orientato al business, ma non guidato dal business** o dal marketing.

Strumenti del marketing, scopi diversi. Molti strumenti che si vedranno (nel design thinking, nel brainstorming, nel design di giochi) vengono dal **marketing** o dal *business development*. Ma vengono usati per un obiettivo diverso: "non bisogna pensare che applicando uno strumento di marketing andiamo a fare marketing". Li usiamo per **validare le idee** e fare un design dell'esperienza migliore, più vicino all'essere umano. Il docente si dichiara un sostenitore del **marketing tradizionale**, più che di quello digitale, "che ormai si è mangiato tutto": nel marketing tradizionale si comunicano i valori di un prodotto, i *gain* che può dare all'utente, i *pain* che può risolvergli, in modo etico, portando un valore reale.

L'Agile. Popolarizzato tra gli anni Novanta e Duemila dal **Manifesto Agile** ("il sito non cambia da anni, è una fotografia"; esiste in tutte le lingue, anche in italiano), è un manifesto per lo sviluppo agile di software scritto da un gruppo di persone variegato, fatto di **dodici principi**, che il docente legge:

1. la massima priorità è soddisfare il cliente rilasciando software di valore fin da subito e in maniera continua;
2. si accolgono i cambiamenti nei requisiti anche a stadi avanzati dello sviluppo: i processi agili sfruttano il cambiamento a favore del vantaggio competitivo del cliente;
3. si consegna frequentemente software funzionante, con cadenza variabile da un paio di settimane a un paio di mesi, preferendo i periodi brevi;
4. committenti e sviluppatori lavorano insieme quotidianamente per tutta la durata del progetto;
5. i progetti si fondano su individui motivati, a cui si danno l'ambiente e il supporto di cui hanno bisogno, confidando nella loro capacità di portare a termine il lavoro;
6. la conversazione faccia a faccia è il modo più efficiente ed efficace per comunicare con il team e nel team;
7. il software funzionante è la principale misura di progresso;
8. i processi agili promuovono uno sviluppo sostenibile: sponsor, sviluppatori e utenti dovrebbero poter mantenere indefinitamente un ritmo costante;
9. la continua attenzione all'eccellenza tecnica e alla buona progettazione esalta l'agilità;
10. (la semplicità, l'arte di massimizzare il lavoro non svolto, è essenziale: il principio è nel manifesto, ma la lettura del docente lo salta);
11. le architetture, i requisiti e le progettazioni migliori emergono da team che si auto-organizzano;

[40] La trascrizione dice prima "IDEA", poi si corregge.

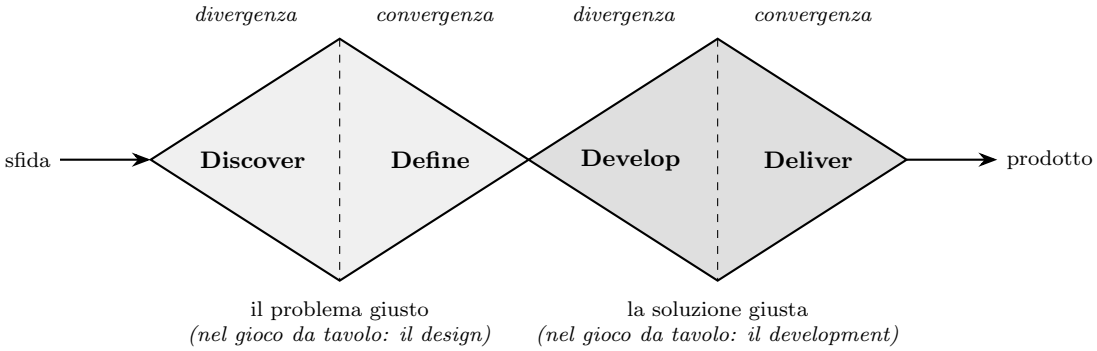
12. a intervalli regolari il team riflette su come diventare più efficace, poi regola e adatta il proprio comportamento di conseguenza.

Un manifesto ormai "dimenticato dalle più grandi aziende, soprattutto dopo l'avvento dell'IA". Lo **Scrum**, nato come parte dell'Agile e poi diventato più famoso da solo, è un *framework* per gestire i cicli di sviluppo e le persone durante la giornata, con una gerarchia di ruoli: *product owner*, team di sviluppo, *Scrum Master*, *stakeholder*. L'esperienza di Agile e Scrum è stata utile, negli anni Duemila, per impostare l'informatica contemporanea.

Il design sprint. Intorno al **2010**, dopo molte esperienze in **Google Ventures** e **Google X** (realtà dedicate alle startup, "ormai scomparse"), l'approccio è stato evoluto nel **design sprint**, basato sul design thinking e sull'Agile. È l'Agile ad aver introdotto il concetto di **sprint**: nel manifesto si parla di consegnare software in cicli "da un paio di settimane a un paio di mesi", e quello è uno sprint; di solito dura circa **due settimane** (il docente lavora con alcuni clienti proprio con sprint di due settimane). Funziona molto bene, come dice il manifesto, per prodotti a **produzione continua**: non un prodotto con un inizio, una fine e un'uscita dopo la quale non viene più seguito, ma videogiochi, software, o giochi come *Magic: The Gathering*, con una produzione e una consegna continue. Per usare il design thinking anche su prodotti *one shot*, e per potenziare lo sprint, si è diffuso il design sprint: in due settimane, o anche in una ("recentemente me ne hanno fatto fare uno addirittura in un pomeriggio, che per me è un po' una scemenza"), Google Ventures e Google X aiutavano aziende, o se stesse, a sviluppare un prodotto secondo cinque fasi derivate dal design thinking. Nella slide c'è il link a un articolo su Medium che spiega le differenze tra design thinking, Agile, Scrum e design sprint.

2.5.16.1 Il Double Diamond: divergenza e convergenza

Ciò che interessa davvero, da design thinking e design sprint, è il concetto di **divergenza** e **convergenza**. La struttura che lo rappresenta è il **Double Diamond**, un processo di design pubblicato nel **2005** dal **British Design Council**; cercandolo online se ne trovano tantissime varianti, "e a noi i dettagli non interessano". Rispetto alle cinque fasi del design thinking ne ha **quattro**: **Discover**, **Define**, **Develop**, **Deliver**. La differenza sta nel *deliver*: l'Agile era per l'*ongoing*, si continua a iterare e a rilasciare; design thinking e design sprint sono per la *delivery*. Ma dentro la *delivery* c'è anche il prototipo, e quindi le quattro fasi si possono dividere in cinque.



Divergenza: ricerca, *discovery*, brainstorming, pensiero laterale, mentalità aperta, tutte le interpretazioni possibili.
Convergenza: pensiero focalizzato, *problem solving* con le ricerche fatte, *kill your darlings*: a un certo punto bisogna tagliare.

Figure 2.5.4: Il Double Diamond del British Design Council (2005), con la lettura del docente: due fasi di divergenza e due di convergenza, che si sovrappongono, "più o meno, per casualità", alla distinzione tra design e development nel gioco da tavolo. In un progetto lungo, i diamanti possono essere decine, e sfasati tra reparti.

Le due fasi. La **divergenza** è il nome delle fasi in cui si fa *brainstorming*, si esplorano le possibilità, si cerca una soluzione usando molto il **pensiero laterale** e la mentalità aperta: è la fase di creatività spinta, in cui si cercano tutte le interpretazioni possibili di qualcosa. Nella divergenza c'è anche la fase di *discovery*, di **ricerca**. La **convergenza** è invece una pratica che **focalizza** e costruisce un processo, con il **pensiero focalizzato**, "che uccide i tuoi gattini": *kill your darlings*. A un certo punto bisogna convergere, bisogna tagliare. Nella convergenza c'è il *problem solving*, cioè l'uso delle ricerche fatte. Molte persone, soprattutto alle prime armi, tendono ad avere moltissime fasi di divergenza, di brainstorming e di design creativo infinito, **senza mai arrivare a una chiusura**.

Design e development. Il Double Diamond, nota il docente, si sovrappone più o meno allo sviluppo del gioco da tavolo: *Discover* e *Define* sono più o meno il **design**, *Develop* e *Deliver* più o meno il **game development**, secondo la distinzione già vista nel corso: il development costruisce il prodotto per l'editore e la pubblicazione, il design è la parte di gameplay e di design dell'esperienza.

Uno strumento, non un mantra. Non va imparato a memoria: va usato come strumento. "Dovete essere voi" (ed è così che lo usa al meglio, in consulenza ma anche negli sviluppi interni) a capire quando è il momento di divergere e quando di convergere: non tutte le fasi sono adatte a entrambi i tipi di pensiero. "Se siete sotto *deadline*, nell'ultima settimana è meglio che convergiate". E, paradossalmente, anche nel brainstorming ci sono strumenti di convergenza: la gente pensa che serva solo al pensiero laterale e alla creatività, ma la creatività, a suo parere, ha **due facce**. Una è l'**espressività**, che si usa molto nella divergenza: buttare fuori idee al cento per cento, anche quelle che sembrano inutili o stupide, perché si provano tutte. L'altra è la creatività in senso stretto, utile anche nella divergenza ma soprattutto nella convergenza: **trovare soluzioni ai problemi**. "Essere creativi, secondo me, è molto un elemento di *problem solving*".

Molti diamanti. Nella figura i diamanti sono due perché il Double Diamond è nato per il design sprint, che deve essere compatto, in due settimane. Ma in un progetto di anni gestito in Agile, come un videogioco, le fasi di divergenza e convergenza possono essere **decine**, e possono essere **diverse tra reparto e reparto**. Al docente è capitato di avere reparti in divergenza mentre altri erano in convergenza: quando lo sviluppo sta chiudendo la *vertical slice* di un videogioco, ed è in convergenza, di solito sei, otto, dieci o dodici mesi prima dell'uscita, entra in divergenza il **marketing**. Il marketing non parte tre o otto anni prima: parte in un momento calibrato, né troppo vicino né troppo lontano dall'uscita. Se parte troppo presto, le persone si dimenticano del prodotto ("esce a dicembre, tu parti a gennaio, e a giugno me lo sono già dimenticato"); se parte troppo tardi, rischia di non informarle.

2.5.17 Dati e insight

Si potrebbero fare tante distinzioni: **dati**, **informazioni**, **conoscenze**, **insight**, **saggezza** (*wisdom*), costruirsi una teoria o no. La più importante, per il docente, è quella tra **dato** e **insight**. Si dichiara "una persona molto *data driven*", ma per dati non intende solo numeri: i **dati qualitativi** di un playtest, "e Ruggero ve lo potrà dire", sono fondamentali. Ogni playtest dà dei dati.

I **dati** sono informazioni **grezze**, quantitative o qualitative, e non bisogna cadere nella tentazione di trarne subito conclusioni. Esempio: da un playtest risulta, dai moduli compilati ("poi Ruggero vi farà la parte dei playtest: non sempre si usano i moduli, ve lo dico"), che il gioco è **troppo lungo**; quindi va accorciato. "No: quello è un dato, non è un insight, non è ancora una conoscenza". Il dato è: X persone che hanno giocato lo ritengono troppo lungo. L'**insight** è chiedersi **perché**: mettere il dato in un contesto, collegare due dati tra loro, identificare modelli, relazioni, tendenze, e risalire alla vera fonte del problema. Magari il gioco *sembra* troppo lungo perché la spiegazione è troppo lunga, perché il regolamento è confuso, o perché alcune fasi sono troppo arzigogolate. Allora il problema non è accorciare il gioco, ma modificare le parti che lo fanno sembrare lungo. La distinzione vale anche per la ricerca.

Il paradosso della novità. Un altro errore, tipico di chi non usa il Double Diamond, è quello che il docente chiama **paradosso della novità**. Un designer, soprattutto alle prime armi, vede qualcosa di nuovo (al cinema, in televisione, facendo ricerca) mentre sta già convergendo, o comunque quando molte cose sono

già definite. Mettiamo un gioco fantasy, con cavalieri e draghi, in un mondo ispirato al ciclo arturiano. A un certo punto vede *Star Wars* e dice: "perché non ci mettiamo le spade laser?". È un esempio estremizzato, "ma non avete idea di quante volte mi è capitato, anche in aziende strutturate": manager o leader che vedono crescere una tendenza, tornano in ufficio il lunedì e dicono "adesso dobbiamo cambiare tutto e andare in quella direzione". No: **la tendenza è il dato**; l'insight è ragionarci sopra. Quando esce il gioco? Tra tre anni. Siamo ancora in una fase in cui si può cambiare? Sì. La tendenza ci sarà ancora tra tre anni? Non è detto: allora riflettiamoci. In fase di ricerca, la differenza tra dato e insight insegna a non saltare alle conclusioni: raccogliere i dati e definire quali dati servono.

2.5.18 Fattibilità, stakeholder e mercato

Si passa agli **strumenti di fattibilità** e ad altri strumenti presi dal mondo del business: "sembrano molto spesso banali, ma vi posso assicurare che, se riuscite a usarli, possono essere molto rivelatori".

2.5.18.1 L'analisi di fattibilità

Ce ne sono molte versioni ("cercate online *canvas fattibilità prodotto* e ne trovate tante"): la slide ne mostra una a tre criteri, **fattibile**, **desiderabile**, **sostenibile**, e una a quattro, quella che il docente usa di solito: **desirable**, **viable**, **usable**, **feasible**.

Desirable (desiderabile): le persone lo vogliono? Ogni prodotto ha bisogno di una domanda e di un'offerta: le persone desiderano davvero questo prodotto, al punto di pagare qualcosa?

Viable (praticabile): possiamo costruire e sostenere questa cosa? Abbiamo le capacità, le forze necessarie? Possiamo raggiungere le persone e rendere il prodotto presente sul mercato?

Usable (usabile): ci sono persone che vogliono questa cosa, la capiscono e la usano abbastanza? Non basta che sia desiderata: quando la si mette fuori, verrà usata?

Feasible (realizzabile): possiamo costruirla **al livello della promessa**, di ciò che abbiamo promesso?

Lo schema si legge in due direzioni. In verticale, una riga riguarda il **prodotto** (usabile e realizzabile) e l'altra la **distribuzione** (desiderabile, cioè c'è abbastanza mercato, e praticabile, cioè si possono raggiungere quelle persone)[41]. In orizzontale, a sinistra c'è la **domanda** (desiderabile e usabile: il concetto di **valore**, che nel caso dei giochi è un'esperienza ricercata o interessante), a destra l'**offerta**, "le cosette business" ("io non le chiamo *money*, le chiamo *business*").

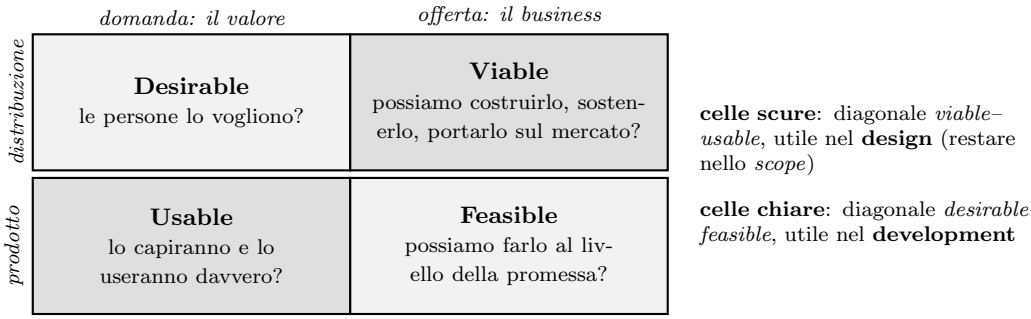


Figure 2.5.5: L'analisi di fattibilità a quattro criteri che il docente usa di solito, con le due letture (domanda/offerta, distribuzione/prodotto) e le due diagonali, una per fase.

[41] Nella trascrizione il docente inverte a un certo punto "alto" e "basso"; la figura segue la lettura per righe e colonne, che è quella coerente con il resto della spiegazione.

Il marketing non è un censore. È uno strumento importante perché, soprattutto agli inizi, si rischia di creare prodotti **non desiderabili**. "Questo non significa che voi non possiate produrre un gioco, ed è quello che dovete capire da questa lezione". Il marketing e il business **non sono censori** della volontà del designer, né devono diventare i leader del design: "non facciamo giochi solo perché il marketing dice di farli; in alcune aziende è così, ma io non sono di questa idea". Si fa un prodotto perché si ha voglia di farlo e si vuole che le persone lo giochino; design thinking, marketing e business aiutano a renderlo **fruibile** e **visibile**, anche se non lo si fa per soldi, anche se lo si pubblica gratis. "E non c'è niente di male nell'avere profitto da un prodotto, anzi, ci mancherebbe". Quando, nelle sue consulenze di *branding*, un'azienda scopre con questo strumento che il prodotto non è desiderabile, tutti vanno nel panico: "dobbiamo ripensare tutto". "E io di solito dico: no, dobbiamo chiederci *come renderlo desiderabile*". Non bisogna cambiare l'idea d'origine, ma capire come si incastra con il pubblico, e per chi è: magari si è sbagliato pubblico, si è sbagliato a chi parlare.

Le due diagonali. È uno strumento soprattutto da *development*, ma già in fase di design aiuta, e lì è interessante la **diagonale viable–usable**; in development, la diagonale opposta, **desirable–feasible**. Perché nel design una delle cose più difficili e importanti, all'inizio, è **stare nello scope**: chiedersi se si hanno le forze per fare quel prodotto. Se l'idea geniale è "un mega gioco, una mega *challenge* fatta in sette giorni a San Francisco da un team di 100 persone", e si è in tre, non è praticabile: è **overscope**. Lo strumento aiuta a renderla praticabile: magari non facciamo la mega *challenge* a San Francisco; ma qual è il **cuore** del prodotto, *in nuce*? Possiamo usare solo quello, focalizzarci solo su quello?

2.5.18.2 L'analisi degli stakeholder

Altro strumento importante, di business e di design, soprattutto per i prodotti analogici di tipo esperienziale: i **giochi pervasivi**, ma anche i LARP; utile anche nel gioco da tavolo e di ruolo, "ma più utile nel LARP e nelle esperienze condivise". Lo **stakeholder** è qualsiasi individuo o gruppo che ha un **interesse** verso il progetto.

L'esempio è un'esperienza di gioco pervasivo fatta dal docente in **Sicilia** con alcune **scuole superiori**: una raccolta delle **memorie storiche** del loro paese, rielaborate in un "LARP digitale", cioè una storia su **Twine** (uno strumento per la narrazione ramificata) che faceva da prototipo per l'esperienza vera. Più che un LARP era un gioco pervasivo: gli studenti si disponevano per tutta la città e agivano come personaggi non giocanti, come **maschere**, raccontando la storia, e i giocatori dovevano fare il giro della città seguendo degli indizi. Chi sono gli stakeholder? Le scuole, gli studenti, i loro genitori, i **nonni** (a cui gli studenti chiedevano i ricordi per il recupero storico), la città stessa, il comune, gli abitanti: "passano delle persone dalla tua via, magari tu non lo sai: devi andarle ad avvisare".

Lo strumento proiettato è una **matrice potere contro interesse**. Il comune ha molto più potere di un libero cittadino, ma ha anche più interesse? "Dipende, non si sa". Le scuole avevano un buon potere e un ottimo interesse, ed erano quindi nel quadrante *high power, high interest*: per loro il lavoro aiutava gli studenti a ragionare sulla storia locale e, poiché Twine crea siti web ramificati, a sviluppare o mettere in pratica le competenze di base sui siti web (erano tutte classi di istituti informatici). Perché è importante? Perché fa capire, già in progettazione, **con chi si dovrà parlare** e **quali sono i limiti** dello sviluppo. È un modo di rendere concreta l'analisi della praticabilità e dell'usabilità, ma anche della desiderabilità; e magari, nell'analisi, si trova anche qualcuno che aiuta a rendere il prodotto realizzabile, della qualità che si vuole raggiungere.

2.5.18.3 La value curve

Poi ci sono gli strumenti di **ricerca di mercato** veri e propri, come la **value curve**: un grafico di **comparazione tra prodotti**. Si usa molto nel videogioco, ma il docente l'ha vista usare anche nel gioco da tavolo. Si prendono due, tre, quattro titoli (la slide confronta *Deus Ex* e *BioShock*) e si scompongono in **caratteristiche**, che sono le colonne: per esempio *cyberpunk*, *sci-fi*, *stealth gameplay*, *reactive story*, *dark atmosphere*, *first-person combat*, "come se fosse il Mixing Desk del design". A ogni prodotto si assegna un colore e una forma (di solito quattro prodotti, il proprio più tre: **cerchio, triangolo, quadrato e rombo**) e per ogni colonna si valuta dove si colloca quel gioco, ottenendo la sua curva.

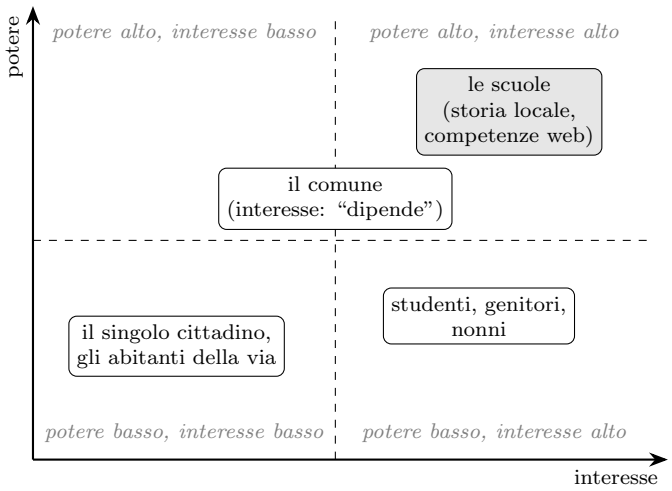


Figure 2.5.6: La matrice potere–interesse applicata, in modo qualitativo, al gioco pervasivo siciliano raccontato nella dispensa. Solo le scuole sono collocate esplicitamente dal docente (potere e interesse alti); le altre posizioni sono indicative.

È uno strumento più da *development* che da design ("oddio, può essere anche di design"), fatto soprattutto per motivi di mercato, per capire **contro che cosa ci si scontra**, e a quale gioco si somiglia di più: "a volte creiamo giochi talmente folli che è difficile capire a che cosa somigliano". È semplice da fare, immediata da leggere, molto comoda nei **pitch**, quando si vuole spiegare perché il proprio prodotto possa essere desiderabile per contrasto ("le persone hanno amato tanto *BioShock*: perché dovrebbero amare *Deus Ex*? Perché adorano i giochi alla *BioShock*, ma manca loro un'esperienza di tipo Y").

2.5.18.4 Le mappe di posizionamento

Un altro strumento di ricerca di mercato, "fortissimo anche per il design", che il docente usa moltissimo ("a me piace molto, sono un esperto di questo"): le **mappe di posizionamento**. Sono due **assi cartesiani**; su ogni asse si mettono due poli, e gli assi devono essere **quantitativi**, con un **gradiente**: "molto spesso vedo mappe qualitative". *Hardcore–casual* è già borderline; vanno bene prezzo alto–medio–basso, oppure *casual game*, *medium game*, *hardcore game*. Ci si chiede dove si posizionano il gioco che si sta creando e i prodotti di riferimento, quelli simili.

Gli oceani. Lo strumento serve a cercare il cosiddetto **blue ocean**. È più da development, ma il docente l'ha usato anche nel design, per variare leggermente un progetto e renderlo più appetibile, per non avere troppa competizione, e anche per **evitare i plagi**. L'idea è distinguersi dalla massa, ma non troppo: il prodotto deve essere simile ad altri, e se nella mappa non si trovano giochi simili "c'è qualcosa che non va"; ma si vuole essere l'unico di quel tipo. Per questo il proprio prodotto, di solito, deve stare nell'**angolo in alto a destra**, ed è il motivo per cui a volte le mappe vengono fatte male. Come si fa: si analizzano i giochi, si segnano tutti i modi in cui li si può analizzare, si prendono vari assi quantitativi e si comincia. È uno strumento *trial and error*: si mette un asse verticale e uno orizzontale, si analizza, si vede dove cade il proprio gioco. Se cade dove ci sono molti competitor, molto simili, è un **red ocean**, un mercato in cui si compete duramente. Meglio un **blue ocean**, con pochi competitor, che però non sia un **black ocean**: il quadrante in cui non ci sono acquirenti ("*hardcore game* a basso prezzo? Magari qualcuno. *Casual game* ad alto prezzo? Magari nessuno"). Si cambiano e si ribaltano gli assi finché non si trova una coppia in cui il proprio prodotto è in alto a destra, da solo: "non accontentatevi della prima mappa, continuate a farla". Aiuta molto anche a capire che tipo di gioco si sta facendo e come lo vedranno le persone.

Empatia, anche qui. "Non pensate a questi strumenti come a un marketing asettico, sterile". Si è in un'ottica di design thinking, e il primo punto è **empatizzare**: il mercato è fatto di persone. Quando si fa la

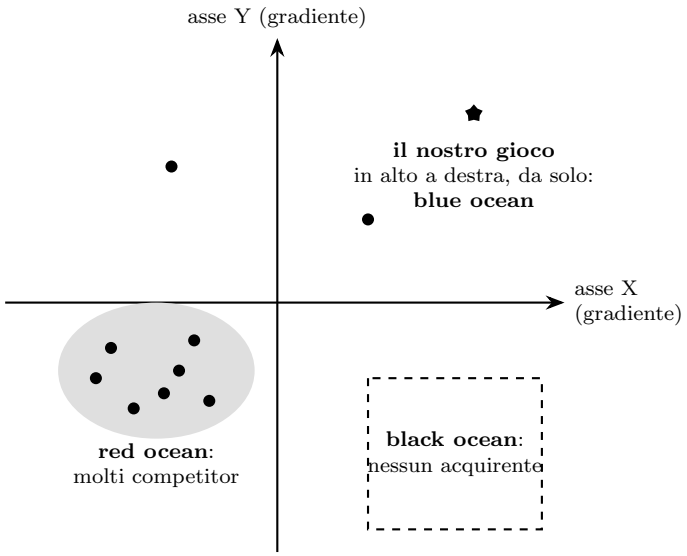


Figure 2.5.7: Una mappa di posizionamento schematica. Se il gioco cade dove ci sono molti prodotti simili è in un *red ocean*; l'obiettivo è un *blue ocean*, pochi competitor ma un pubblico reale, evitando i *black ocean*, quadranti in cui non c'è nessuno che compri. Gli assi si cambiano e si ribaltano finché non si trova la coppia che mette il proprio gioco in alto a destra, da solo.

value curve, si pensa alle persone che vedranno valore nel prodotto; lo stesso con la mappa di posizionamento.

2.5.18.5 Lavorare sulle proprie idee: la domanda latente

Si può lavorare sulle proprie idee? "Assolutamente sì": il marketing deve essere di supporto. E il modo migliore per lavorare sulle proprie idee ed essere vincenti è lavorare sulla **domanda latente** ("bisogna essere un po' fortunati"). La slide distingue quattro tipi di domanda, che il capitolo riassume nella tabella 2.5.3.

Domanda	**Che cos'è, secondo la dispensa**
effettiva (conscia)	una domanda reale: le persone hanno bisogno di questo prodotto, e lo dicono
latente	conscia, ma non ancora espressa: se intervisti le persone non ti dicono "lo vorrei", ma una ricerca approfondita mostra dei fili rossi che indicano che il mercato ne ha bisogno; difficile da cogliere, ed è il terreno migliore per le proprie idee
potenziale	qualcosa che in futuro, magari, le persone vorranno
improbabile	inconscia e vincolata (*constrained*): per farla emergere serve moltissimo tempo

Table 2.5.3: I tipi di domanda presentati nella dispensa. Il docente si corregge a voce: la domanda latente "non è subconscia, è conscia, ma non è ancora uscita". "Farei vedere questo schema a chi oggi lavora su tecnologie avanzate come IA e robotica, perché fallirebbero il test del quadrante delle domande".

2.5.18.6 *Games User Research* e il design sprint

Il libro di *Games User Research*. Il docente consiglia il libro ***Games User Research*** di **Anders Drachen**, **Pejman Mirza-Babaei** e **Lennart Nacke**[42], uno strumento di *user research* per giochi, con

[42] La trascrizione dice "il Gurmethod, che è fatto da Drake, Mirza Babei e Nacche". Il docente ricorda che un tempo il PDF era scaricabile gratuitamente; durante la registrazione prova a ritrovarlo online ("il sito è down"), lo trova su Google Books e promette di caricare sul Drive una dispensa di sintesi se ritrova la sua copia.

una serie di strumenti reali non solo per UX e UI, ma anche per la ricerca di mercato. La slide riporta la loro tabella, che colloca gli strumenti nelle fasi di sviluppo (*envisioning*, *design*, *build*, *release*, *post-release*), divisi per **attitudine** e **comportamento** (*attitude* e *behavior*) e per **quantitativi** e **qualitativi**: i quantitativi in *tondo* (*roman*, "il nome che si dà alle lettere tonde normali"), i qualitativi in *corsivo* (*italic*). La forma indica la domanda fondamentale a cui lo strumento risponde: il **rombo** "per chi è il prodotto?", il **cerchio** "dovremmo costruirlo?", il **triangolo** "è adatto al nostro modello di business?", il **quadrato** "come dovremmo farlo?". "Se siete un po' furbi, avete riconosciuto i quattro quadranti di desirable, viable, usable e feasible: non è fatto a caso". Il colore indica quanto è veloce da implementare: **verde** un giorno, **blu** una settimana, **arancione** due settimane. "Ci sono tutti": lo studio etnografico, la segmentazione di mercato, il *diary study*[43] ("molto, molto carino"), le *personas*, il *focus group*, il *benchmark playtest*, la *telemetry analysis*, l'*A/B testing*, la *narrative usability*. Sono pensati per il videogioco, ma molti sono applicabilissimi al design analogico.

Il libro sul design sprint. Infine il design sprint, nel libro di **Banfield**, **Lombardo** e **Wax**[44], che il docente possiede e da cui promette di ricavare un'altra dispensa. La struttura va dalle due settimane a un minimo di cinque giorni ("io l'ho fatto anche in tre, perché le aziende non hanno tempo") e serve a costruire un **MVP**, *Minimum Viable Product*: il prodotto più piccolo, ma praticabile, cioè producibile da noi, che passi dal pensiero all'azione in modo immediato. Le fasi sono cinque: **understand**, **diverge**, **converge**, **prototype**, **test**. In questo libro divergenza e convergenza sono in due punti diversi, ma non in un doppio diamante. "Vi leggete il libro, vi leggete la dispensa, vi fate una cultura".

2.5.19 Il brainstorming strutturato

Gli strumenti di *brainstorming* sono "importantissimi", perché il brainstorming è uno dei momenti in cui le persone, di solito, fanno "il caos più totale". Il **brainstorming non strutturato** ha diversi difetti:

- è poco efficace a livello di **espressività**, perché tendono a esprimersi solo le persone socialmente più brave: la personalità istrionica è avvantaggiata rispetto alle altre;
- non permette alle buone idee di emergere, e di solito non ha un punto di partenza;
- non ha **ruoli**: dovrebbe esserci sempre una persona che prende appunti, sempre un **recap** alla fine, e delle **decisioni**;
- senza struttura, può portare a fasi di divergenza infinite che non arrivano mai al punto.

Per questo è fondamentale **strutturare le sessioni**.

2.5.19.1 Le Liberating Structures

Uno dei primi strumenti, gratuito online, sono le **Liberating Structures**[45]: non nascono per il brainstorming, ma come semplici **schemi di interazione** per "*local action to spark global ripples*", con scopi di base politici o organizzativi. Sono esercizi di team "della serie: se ci credi funzionano, se non ci credi non funzionano", che aiutano un gruppo a strutturare meglio la conversazione e a liberarne il potenziale. Sono tutte descritte con la stessa struttura; c'è anche una convention annuale, in giro per il mondo, in cui le persone vengono invitate a praticarle.

15% Solutions. L'esempio aperto a schermo. "*Discover and focus on what each person has the freedom and resources to do now*": utile quando, in un'azienda o in un gruppo, si vuole capire su che cosa le persone possono operare; richiede solo 25–30 minuti. Lo scopo: i partecipanti identificano **piccole azioni che possono fare subito** in risposta a una sfida, concentrandosi su ciò che è sotto il loro controllo, per passare

[43] La trascrizione dice "camera di arrestar" e "Camera Dire Study".
[44] La trascrizione dice "Benfield Lombardo Wix". Si tratta con ogni probabilità di *Design Sprint* di Richard Banfield, C. Todd Lombardo e Trace Wax.
[45] La trascrizione dice "liberating structure" e "libera instructor".

dal sentirsi senza potere a prendere iniziative e creare subito un'inerzia per il cambiamento. I passi: si condivide l'invito; ognuno scrive da solo che cosa può fare *adesso*, realmente, per affrontare la sfida; ci si divide in gruppi di due o tre per condividere le idee, facendosi da *coach* a vicenda con domande di chiarimento e consigli, assicurandosi che chi parla abbia davvero il potere di rendere pratico quel 15% delle sue soluzioni; poi si torna insieme e si condividono le ispirazioni. "Non tutto quello che hai pensato è fattibile: cerchiamo il 15% che lo è". Sembrano cose banali, "ma usate correttamente sono molto, molto potenti"; meglio provarle in piccoli gruppi di tre: non tutte funzionano, alcune funzionano molto.

Le preferite e le meno operative. Il docente ne passa in rassegna altre:

- **W³** (*What, So What, Now What*), "molto interessante": parte dai dati e dall'esperienza e arriva fino all'azione da adottare; per gli studenti può essere utile anche per decidere chi fa che cosa nel team e come si producono le cose;
- **Design Elements**, più complessa, da fare a livello strutturale (si vede già da quanto è articolata la sua descrizione), ma fatta apposta per stimolare il design, soprattutto nella differenza tra micro e macro;
- **1-2-4-All**, che Design Elements sfrutta ("alcune di queste strutture ne usano altre"), ottima per generare domande, idee e suggestioni: si condivide l'invito e si identifica una sfida comune; un minuto in silenzio per riflettere e scrivere; in coppia si condividono le idee, rimandando il giudizio e tenendo ogni idea distinta; poi quartetti o terzetti per rifinirle; infine tutti insieme. "Molto carina, ma per gruppi più numerosi dei vostri, che siete solo in tre";
- **25/10 Crowd Sourcing**, "molto interessante";
- **Nine Whys**, "veramente, veramente interessante", basata sull'abitudine dei bambini di chiedere sempre "perché?", ma strutturata come vero strumento di brainstorming;
- **Wicked Questions**, "molto bella";
- meno operative, a suo parere: **Impromptu Networking** ("l'ho provato e non mi era piaciuto"), **Panarchy** (idem) ed **Ecocycle Planning** ("interessante come teoria, ma non ci ho mai visto un'operatività importante"). Di alcune strutture nuove non ha esperienza. "Comunque andate a esplorarle".

2.5.19.2 I canvas

Un altro strumento di divergenza sono i **canvas**, di business e di design thinking. La slide mostra un *game canvas*, "ma non è solo questo": ne esistono a centinaia, a migliaia, con il Double Diamond o senza: il *Business Model Canvas*, il *Project Canvas*, il *Value Proposition Canvas*. Sono pagine A3 o A4, anche su Miro o su una lavagna digitale, e servono per fare **ice breaking**. L'errore più grande, che fanno "tantissimi direttori di azienda", è pensare che basti una riunione di due ore in cui si compila il canvas per aver fatto tutto. "No, il canvas è un *icebreaker*": è un ottimo modo per divergere (si può usare anche per convergere, ma è molto difficile), dà idee, avvia il lavoro e la conversazione nel team; ma **non sostituisce il design**. "Ho fatto il *game canvas*, allora non devo usare i framework, non devo fare l'*emotional design*: no".

2.5.19.3 SCAMPER

Lo **SCAMPER** è uno strumento di divergenza "molto, molto potente", nato per trovare **soluzioni provocatorie**: "quando faccio lo SCAMPER, le spariamo proprio". Funziona molto bene su un **problema locale** del prodotto ("deve essere un problema locale"): per esempio, dai test emerge che il movimento della pedina non funziona. È un acronimo che elenca sette possibilità minime ("non sono tutte, ma sono il minimo") per immaginare altre soluzioni.

	Operazione	Note del docente
S	sostituire (*substitute*)	"fondamentale"
C	combinare (*combine*)	"fenomenale"
A	adattare (*adapt*)	a volte poco utile
M	modificare (*modify*)	tra le più potenti
P	mettere ad altro uso (*put to another use*)	a volte poco utile
E	eliminare (*eliminate*)	tra le più potenti
R	invertire (*reverse*)	"mi piace tantissimo"

Table 2.5.4: Lo SCAMPER. Sono processi mentali di base; lo strumento serve a *ricordarseli*, perché davanti a un problema ci si concentra talmente sul problema da non vedere le soluzioni.

2.5.19.4 Il Crazy Eight

"Un altro mio cavallo di battaglia", che il docente ha adattato negli anni partendo da questa base. È uno strumento di divergenza, di brainstorming, "ma ha altri vantaggi".

1. Si prepara un **foglio A3** per ogni partecipante (oppure una board su Miro), con una penna e degli **adesivi**. Il foglio si piega a metà tre volte e si riapre: **otto riquadri**.
2. Si definisce insieme il **problema** da risolvere.
3. Si dà un tempo: 5, 10 o 15 minuti, a seconda delle capacità artistiche delle persone ("non dovete essere Leonardo da Vinci"). Si deve **disegnare, non scrivere**: l'unica scrittura ammessa è il **titolo** di ciascun riquadro. Ogni riquadro è una **soluzione possibile e implementabile**.
4. Ognuno sceglie la propria **top 3** tra gli otto disegni, la presenta, e si discutono solo quelle: in cinque persone, 15 soluzioni.
5. Si votano le soluzioni di tutti e si fa una **top 5**, da quella con più verdi a quella con più rossi; e si discutono i voti: "non è che quella che ha preso troppi rossi è da buttare e non ne parliamo: la discutiamo".

Per votare il docente usa il **metodo a semaforo**: ogni persona ha **tre adesivi verdi, tre gialli e tre rossi** e vota le soluzioni degli altri. Si può anche dare a ogni persona un proprio colore, "ma lo sconsiglio", perché poi le persone litigano sul fatto che qualcuno non le ha votate.

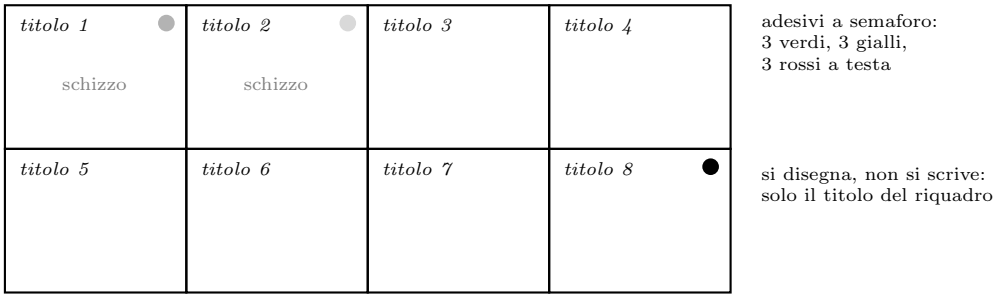


Figure 2.5.8: Il foglio del Crazy Eight: un A3 piegato in otto, un riquadro per ogni soluzione possibile, disegnata (anche con gli *stickman*) e accompagnata solo da un titolo. Chi non sa o non vuole disegnare può riempire i riquadri con un collage di foto.

Perché funziona. È interessantissimo quando bisogna creare **soluzioni visibili**: nel design dei giochi, nell'arte, negli sketch, nell'animazione, nel cinema, ogni volta che bisogna far capire agli altri una soluzione visiva. E con il Crazy Eight diventa automatico capire quando una soluzione **non è fattibile**: "se non puoi

disegnarla, anche brutalmente, se non puoi farne uno sketch, non è fattibile". Chi non sa disegnare può farlo con un **collage di foto**: gli otto riquadri si riempiono con immagini prese online che rappresentano la soluzione pensata.

2.5.19.5 Il parking lot

Un altro strumento "interessantissimo", che il docente usa in tutte le sessioni di brainstorming e anche nelle riunioni (ultimamente un po' meno, perché i suoi team sono molto strutturati). Serve quando si ha un **gruppo caotico**: persone che parlano senza alzare la mano, si interrompono, dicono sciocchezze, non sono brave a parlare. Funziona così: c'è una **cartella condivisa** (su Drive, o in cloud) con un **file per ogni partecipante**; se si è in quattro, Blu, Rosso, Verde e Giallo, ci sono quattro file, e tutti vedono i file di tutti. Quello è il *parking lot*. La regola: **non si scrive sul proprio**, si scrive su quello degli altri: commenti, idee o feedback che si vuole che l'altro ricordi (o che si vuole ricordare), ma che **non c'entrano con la conversazione in corso**, per non interromperne il flusso. Invece di interrompere ("ah, ma io volevo dire questa cosa", e non c'entra niente), lo si scrive nel parking lot di Tizio, Caio o Sempronio, e a fine sessione lo si legge insieme e si decide che cosa fare. È comodo anche per le **azioni da compiere**: se Lucia deve ricordarsi di fare una cosa, non la si interrompe, glielo si scrive nel suo parking lot.

2.5.19.6 Gli strumenti ibridi: le votazioni

Utili sia in divergenza sia in convergenza sono gli **strumenti di votazione**. La dispensa ne presenta tre.

Plus/Delta. Ottimo per i gruppi che "si prendono molto male" se ricevono feedback troppo diretti. Si fa **alla fine di ogni sessione**, con una board divisa in **più** (+) e **delta** (Δ): il più è che cosa è andato bene nella sessione e si vuole ripetere; il delta è che cosa si vuole cambiare, implementare in modo diverso la prossima volta, anche nel lavoro. Delta non vuol dire per forza negativo: nell'esempio in slide qualcuno ha scritto in azzurro "*do it more*" (mi è piaciuta questa sessione, facciamone di più), qualcun altro in verde "*focus narrower*" (la prossima volta un focus più stretto).

Fibonacci vote. Si può fare con carte o numeri, in qualsiasi momento: c'è uno stallo, mettiamola ai voti; "ma attenzione, bisogna farlo in un modo molto particolare". Serve a **rompere l'omertà** dei gruppi che non vogliono criticarsi e a stimolare il dialogo tra posizioni opposte. Dopo un Crazy Eight, per esempio, si votano le proposte di tutti, ma si può votare **solo con certi numeri: 0, 3, 5, 8, 13**. Rispetto al classico voto da 1 a 5, le differenze tra un 3 e un 5, un 5 e un 8, un 8 e un 13 sono molto più marcate. Poi, "non è finita": **chi ha votato in modo opposto** sulla stessa cosa spiega perché e dialoga con l'altro, mentre tutti gli altri stanno zitti. Se io ho votato 13 e qualcuno 0 o 3, ci spieghiamo: non dobbiamo trovare un compromesso né cambiare voto, ma **esplicitare il motivo**. Si può fare anche con chi ha votato uguale (perché abbiamo votato tutti 13, o tutti 5?). È molto comodo per stimare l'**impegno** (*effort*): è difficile che con il Fibonacci le persone non votino davvero, perché costringe a dire "questa attività è molto impegnativa", "no, è un 3", e tra un 3 e un 5 c'è una bella differenza.

Silent critique. Per il caso opposto: persone troppo vocali, "tantissimo vocali", che si interrompono e continuano a offendersi. Si danno adesivi e post-it, e un timer di 20 o 30 minuti durante il quale si può votare e dare feedback agli altri **solo scrivendo**: i dialoghi sono vietati, "vietato, vietato". A volte il docente vieta anche la scrittura: feedback solo visivi, "una sorta di Crazy Eight con silent critique". "In realtà la silent critique mi serve per insegnare ai gruppi a stare zitti". Molti di questi strumenti, ammette, più che brainstorming sono **gestione del team**; "però sono molto comodi".

2.5.20 Gli strumenti di convergenza e il pitch deck

"Veniamo verso la fine agli strumenti di convergenza": strumenti che servono a **convergere**, ad arrivare a un unico punto.

Strumento	Per quale gruppo	Come funziona
Plus/Delta	chi soffre i feedback diretti	a fine sessione: + che cosa ripetere, Δ che cosa cambiare (non per forza negativo)
Fibonacci vote	gruppi omertosi, che non si criticano	voti solo 0, 3, 5, 8, 13; chi ha votato agli estremi spiega il perché, gli altri tacciono; utile per stimare l'impegno
Silent critique	gruppi troppo vocali	20–30 minuti di voti e feedback su post-it, senza parlare (a volte anche senza scrivere)

Table 2.5.5: Gli strumenti di votazione "ibridi", utilizzabili sia in divergenza sia in convergenza.

2.5.20.1 Il one-liner

Lo si può chiamare *one-liner*, *tagline*, *pitch* in una riga, *high concept*: "hanno tanti nomi, ragazzi", ma *one-liner* è il più descrittivo. È il **gioco spiegato in una riga di testo**, al massimo in un paragrafo. È uno strumento di convergenza proprio perché **obbliga** a metterlo in una sola riga. (Il docente ha una lezione intera, nel corso di narrativa, sui *concept* e su come costruirli in modo strutturato.) Nel libro *The Game Narrative Toolbox* i one-liner sono presentati come uno strumento noto nell'industria videoludica; "non è davvero così": il docente ha trovato aziende in cui non ci si era mai pensato, o in cui *one-liner* vuol dire una battuta di dialogo registrata da un doppiatore. In generale, comunque, è una frase che descrive il **mood** del gioco, il suo **stile**, com'è strutturato, oppure lo descrive per **comparazione**. Gli esempi della slide, dai retro delle scatole:

Terraforming Mars. Sotto il testo di ambientazione ("*The taming of the Red Planet has begun...*") c'è il one-liner di tipo **descrittivo**: "*In Terraforming Mars, you control a corporation with a certain profile. Play project cards, build up production, place your cities and green areas on the map, and race for milestones and awards. Will your corporation lead the way into humanity's new era?*". Descrive il modo di giocare, lo stile, il gameplay, la struttura, che cosa si fa.

Carcassonne. Un one-liner **scomposto in tre punti**, classico dei retro di copertina. Il one-liner non è la frase sui milioni di copie vendute nel mondo, "che non vi dice niente", ma "*place a tile, place a meeple, score points and win*".

Arcs. Qui il one-liner è più lungo, e il docente si concentra sulla prima parte, una **domanda**: "*Long ago, the Reach tried, but our ancestors failed us... As we wrestle over the galaxy's edge, can we break the cycle? Can we build something new?*"[46]. Sotto c'è un one-liner per **comparazione**: "*Arcs is a sharp sci-fi strategy game for two to four players, set in a dark yet silly universe*".

Comparazione non vuol dire confrontare titoli, ma usare termini che fanno paragonare il gioco a una cosa invece che a un'altra: *strategy game*, non *deduction game*; *sci-fi*, non fantasy. "È questo, non quello".

2.5.20.2 I riflettori

"Un altro strumento di convergenza fenomenale", che il docente usa anche in consulenza: "ho visto letteralmente direttori creativi piangere su questo strumento". Sono i **riflettori**, uno strumento che il docente ha costruito negli anni "ma senza inventare niente", perché è un'idea **teatrale**, diffusa negli anni Sessanta e Settanta, soprattutto nell'improvvisazione, come idea "maschile" e idea "femminile": in ogni scena, in ogni piccola idea teatrale, c'è un'idea "maschile", il **contenuto**, e un'idea "femminile", il **contenitore**. È una terminologia che, dice, "nel 2026 può essere aggiornata": lui la divide in **contenuto A** e **contenuto B**, cioè contenuto e contenitore (di solito personaggio e storia, oppure storia e *world building*, contesto e così via).

Luce e ombra. Ma il vero strumento, e il motivo del nome, è la divisione **orizzontale**: le celle **in luce** (tre per colonna, gialle nella slide) e le celle **in ombra** (le restanti)[47]. La metafora viene dalla regia. Immaginiamo

[46] Citazione ricostruita dalla lettura del docente; la trascrizione è qui imprecisa.

[47] La trascrizione parla prima di "tre celle gialle e sette celle", poi di "sette idee in ombra e sette idee in luce"; il numero certo, ripetuto più avanti, è quello delle celle in luce: tre per il contenuto A e tre per il contenuto B. La figura mostra quindi

una scena di festa in cui i due protagonisti, ai lati opposti del palcoscenico, capiscono di amarsi: la festa deve continuare, ma il pubblico deve capire che è il momento importante, in cui i due si girano, si guardano e, da lontano, capiscono. Come si fa? Si puntano **due riflettori** sui protagonisti, che mettono in luce il punto fondamentale della scena; attorno restano le idee in ombra. Le **idee in ombra** non sono idee da scartare: sono idee che verranno dopo, o minoritarie, meno importanti. Le **idee in luce** sono le informazioni **vitali ed esplicite**, non implicite.

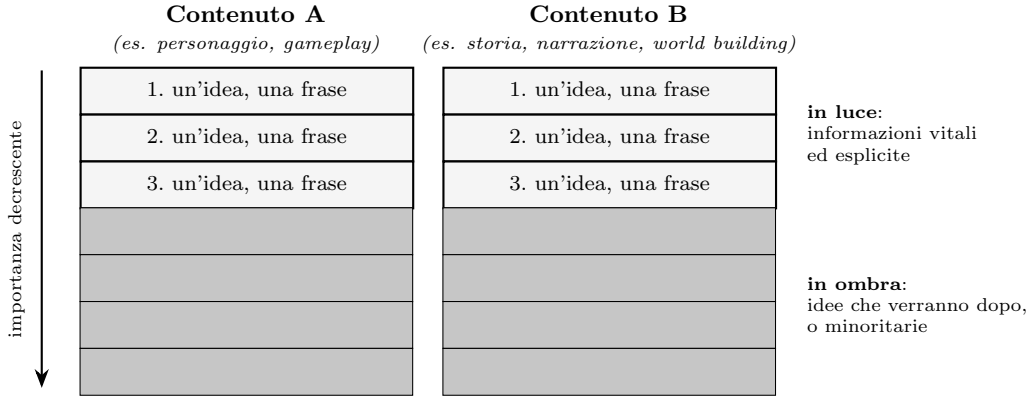


Figure 2.5.9: I riflettori. Due colonne (contenuto e contenitore, chiamati dal docente contenuto A e contenuto B), una frase per cella, in ordine di importanza dall'alto. Solo tre celle per colonna sono "in luce": il limite numerico è ciò che rende lo strumento efficace.

Il limite numerico. Che cosa funziona così bene? "Il limite numerico, ragazzi". Quando si hanno solo **tre spazi** per il contenuto A e tre per il B (personaggio e trama, personaggio e gameplay, gameplay e narrazione, narrazione e *world building*: "fatelo come volete"), ci si rende conto che chi gioca ha un **limite alla quantità di informazioni** che gli si possono dare, "non perché sia stupido". In ogni cella va **un concetto, una frase**. Le celle sono in ordine di priorità: più un'informazione è in alto, più è importante; e la discussione nasce proprio da qui, perché qualcuno vorrà mettere al primo posto una cosa e qualcun altro un'altra. Funziona molto bene per i **pitch da fiera** (le cose da dire e da non dire quando si presenta il gioco in fiera) e ogni volta che bisogna convergere su una soluzione con poco spazio. Il docente lo usa soprattutto in consulenza, all'inizio di un lavoro con gli autori, l'editore o il cliente, e "non c'è cliente, anche tra quelli che ci hanno fatto fatica, che non l'abbia apprezzato": serve a focalizzare, a dire "il mio prodotto non può essere mille cose: è questa cosa qua".

2.5.20.3 Lo high vision document

Un altro strumento, più corposo, che il docente consiglia sempre al termine della fase di brainstorming, appena prima di un pitch a un editore, di un *game design document* o del design stesso: lo **high vision document**[48]. È un documento **di una pagina**, due al massimo, che riassume i **temi**, i **pilastri** del gioco, le **meccaniche**, le **estetiche**, la **narrativa**. Ci si possono mettere anche solo gli strumenti appena visti: già il one-liner e i riflettori insieme costituiscono "una sorta di contratto che firmate tra di voi": è ciò che vogliamo ottenere dal gioco. Si possono mettere in alto le **emozioni** o i *key element* ("io di solito metto direttamente le emozioni"); il modello in slide è pensato per i videogiochi, "per i giochi da tavolo è addirittura più semplice". Vanno messi gli **obiettivi** del gioco, le **metafore**, i **significati**. Il docente di solito lo divide in **must**, **must not** e **nice to have** (necessario, da evitare, buono da avere se c'è tempo e spazio), e aggiunge l'**unique selling point**: il motivo per cui il prodotto dovrebbe vendere, che lo distingue dagli altri; ma "vendere" non vuol dire incassare: vuol dire che, quando lo spieghi in fiera, le persone sono catturate dall'idea.

È anche uno strumento di **controllo costante della convergenza**: se chi sviluppa il gioco cambia idea in continuazione, gli si può dire "guarda che ci stiamo allontanando dalla high vision". Non descrive

tre celle in luce per colonna e lascia indicativo il numero di quelle in ombra.

[48] La trascrizione scrive "high vision", "IVision" e "Eye Vision".

in dettaglio il gioco, ma la **visione**, l'esperienza che si vuole raggiungere: è uno strumento duttile, ma che permette di "tenere la barra dritta".

2.5.20.4 Il pitch deck come validazione

"*Last but not least*": il **pitch deck** (a volte *pitch document*), usato come strumento di **validazione**. È una presentazione (PowerPoint o Google Slides) con tutte le informazioni necessarie perché un publisher, un editore o il team diano il *green light* al progetto. Il modello di base, "interessante", è quello di **Jason Della Rocca**, che lo spiega lui stesso su YouTube in molte versioni (in slide c'è il video *Jason Della Rocca's Guide to Pitching Your Video Game*, di circa sei anni prima: "ha fatto questo talk tantissime volte, in tante salse"). È utile per validare l'idea anche quando non si deve presentarla a nessuno: "quando faccio un gioco per me, a chi lo presento? A me stesso; lo pubblico su itch.io, che mi frega. Ma lo faccio per vedere se l'idea è valida".

	Slide	**Che cosa contiene**
1	artwork	un'immagine (anche presa online) che catturi l'attenzione
2	goal / overview	l'obiettivo del prodotto, il one-liner (come quelli di *Arcs* o *Terraforming Mars*)
3	proof of concept	un'idea vaga, anche disegnata a mano, di come dovrebbe essere il gioco, con quali contenuti e componenti; è la slide in cui il team si chiede "siamo sicuri di mettere questa componente?"
4	unique selling point	che cosa rende il gioco diverso dagli altri, l'unica cosa che permette di dire che è fenomenale
5	"provami che è cool"	feedback dei playtester o, se non ci sono, prodotti simili e commenti online che validano l'idea
6	business model / analisi competitiva	non un *forecast*: target, analisi di mercato, genere e segmento
7	timeline	quante settimane, con quale processo (le colonne *ready*, *in dev*, *to do* si possono omettere)
8	team e pedigree	chi c'è e perché può portare a termine il progetto: non solo il ruolo, ma che cosa porta ciascuno (per esempio, esperto del campo)
9	domande e contatti	con qualcosa di memorabile, una GIF, un meme

Table 2.5.6: Il pitch deck secondo il modello di Jason Della Rocca, come lo presenta la dispensa. Non è l'unico modello; il docente suggerisce di aggiungere sempre il *core loop* (il gioco in tre passi, come sul retro di *Carcassonne*) e l'obiettivo del gioco.

Uno strumento di convergenza, non di brainstorming. "Ricordatevi: è uno strumento di convergenza. Non usatelo per il brainstorming, perché non avrete le informazioni". Lo si usa dopo il brainstorming e dopo la ricerca, quando un'idea c'è già. La **timeline** è importante proprio come validazione: "quando mi portate questa slide il 12 ottobre, la timeline non la mettete per me, la mettete per voi". Siete voi a dover dire: siamo sicuri di farcela in sei settimane? Quando i team mettono una timeline, di solito ci si chiede: ce la facciamo davvero, o dobbiamo ridurre lo *scope*? Anche il **team** conta, e non solo come elenco di nomi: in un videogioco ci sono ruoli come *lead level designer*, *lead game designer* ("per voi non ha senso", dato che lavorano in tre), ma il docente ha visto di recente, negli indie e nei team analogici, mettere accanto a ogni persona **perché è utile al team**, che competenze porta. Il pitch deck, infine, fa capire l'**ordine delle informazioni** e funziona come una **to-do list**: ci siamo ricordati di mettere il *goal*? L'*overview*? Il *core loop*? L'obiettivo del gioco?

La chiusura della dispensa. "Questo è tutto". La dispensa sarà caricata sul Drive; il docente spera che la registrazione sia venuta bene, dopo un paio di interruzioni, ma non aveva altro modo di registrarla: "domani è il 5 ottobre e mi partono tutte le lezioni". Chiede di vederla il prima possibile; lo scriverà su Discord appena il video sarà pronto, e risponderà alle domande lì, a chi lo tagga.

Punti chiave del capitolo

- Le **game jam** (48 o 72 ore, settimane, mesi; tema ampio, gruppi misti, non concorsi) servono soprattutto a capire **con chi si lavora bene**. Si può portare avanti un'idea oltre la jam, se il gruppo funziona e se il "5%" non è figlio di un *overscoping*; i giochi da jam che arrivano su Steam sono frutto del lavoro successivo.

- Nella revisione delle schede: niente **nome dell'autore** sulla scheda; niente **foto del viso** in un LARP (meglio descrivere come vestirsi); **scrittura neutra** rispetto al genere di chi gioca; nominare esplicitamente le persone a cui badare; le relazioni divise per cerchie si leggono meglio delle liste; una relazione taciuta in una delle due schede può essere un colpo di genio. L'età è più flessibile dell'aspetto (chamber contro casting dei blockbuster).

- **Lo stesso gioco fa provare cose diverse** a persone diverse e nello stesso momento (ansia, sfida, appagamento, curiosità, rabbia), e anche alla stessa persona in contesti diversi: chi gioca resta una persona, con il suo umore e la sua giornata.

- **Il divertimento non è nel gioco, è nel giocare**: non lo si può meccanizzare, non è un componente; emerge dalle dinamiche. L'equazione *divertente = bello* è fonte di conflitti e **restringe l'area progettabile**, come se tutti i film dovessero essere divertenti.

- Il gioco è un **medium legittimo** e può trattare qualunque tema (il black box su Cuba, *The Cost*, *Holding On*, *Charnel Houses of Europe*), come hanno fatto il cinema e il fumetto (*Schindler's List*, *Maus*). Ma potere non vuol dire dovere, né farlo da soli: sui temi delicati servono **consulenti** che sappiano di che cosa si parla (il gioco sul diabete con l'OMS). Un gioco è un'**esperienza creata da regole**.

- Il **bleed** è l'influenza reciproca tra chi gioca e il personaggio: **bleed in** (dalla persona al personaggio: il colore preferito di Angelo che diventa quello di Andrea) e **bleed out** (dal personaggio alla persona: l'ansia della caduta). È un'**area progettabile**: i giochi che lo cercano lasciano vuoti da riempire con il personale. Può far male; le emozioni non si fermano a fine partita; provarlo non è segno di debolezza.

- La **sicurezza** si progetta **prima** (workshop, contratto sociale o dichiarazione di intenti, preparazione emotiva, focus tematico, spiegazione delle tecniche), **durante** (OK check, *break* e *cut*, X card, linee e veli, metatecniche diegetiche come *Ars Amandi* e "lo fa Mario") e **dopo** (debriefing, de-rolling, valutazione dell'esperienza, feedback). Nel LARP nordico nasce dal passaggio dalla sicurezza fisica a quella emotiva, e prende strumenti da altri mondi (lavoro, BDSM, psicodramma).

- Linee e veli **emergono dal gruppo**, si aggiornano in corso d'opera, vanno al minimo comune denominatore e non toccano il tema del gioco. Non sono un freno ma un **via libera**: come sapere che c'è un bagno disponibile. Tutte le regole limitano la libertà: è il cerchio magico.

- Il **secondo task** (per il 25 settembre): un gioco di ruolo completo in **200 parole** al massimo (*hard limit*, titolo compreso), individuale, Google Doc in Arial 11 senza immagini, file "nome del gioco – nome"; un'emozione precisa dal cerchio esterno della ruota, **mai nominata**, data **con le meccaniche e non con il setting**; materiali solo di uso comune e dichiarati.

- "Se l'emozione viene solo dal setting, il gioco è debole": nascondino e scacchi non hanno ambientazione, eppure fanno provare cose diversissime. Progettare il gioco ideale vuol dire non lasciare a chi gioca problemi da risolvere da sé. E soprattutto: **le persone sono più importanti dei giochi**.

- **Dispensa**: il **design thinking** (empathize, define, ideate, prototype, test), l'**Agile** e i suoi principi, lo **Scrum** e il **design sprint**; il **Double Diamond** (discover, define, develop, deliver) con le sue fasi di **divergenza** e **convergenza**, da usare come strumento e non come mantra.

- Un **dato** non è un **insight**: "il gioco è troppo lungo" va interpretato prima di agire. Il **paradosso della novità** (le spade laser nel ciclo arturiano) è inseguire una tendenza senza chiedersi se esisterà ancora all'uscita.

- Fattibilità: **desirable, viable, usable, feasible**; il marketing aiuta a rendere un gioco fruibile e visibile, non è un censore; restare nello *scope*. Poi l'analisi degli **stakeholder** (matrice potere–interesse), la **value curve**, le **mappe di posizionamento** (red, blue e black ocean; in alto a destra, da soli), la **domanda latente**, *Games User Research* e il libro sul design sprint (MVP).
- **Brainstorming strutturato**: ruoli, recap, decisioni; Liberating Structures (15% Solutions, W^3, 1-2-4-All, Nine Whys...); i canvas come *icebreaker*; SCAMPER sui problemi locali; Crazy Eight; parking lot; votazioni con Plus/Delta, Fibonacci (0, 3, 5, 8, 13) e silent critique.
- **Convergenza**: il **one-liner** (descrittivo, in tre passi, per comparazione), i **riflettori** (contenuto e contenitore, tre idee in luce per colonna, in ordine di importanza), lo **high vision document** (must, must not, nice to have, USP) e il **pitch deck** di Jason Della Rocca come validazione, con la timeline per sé stessi e la verifica dello *scope*.

Chapter 2.6

Ricerca e sviluppo: i giochi in 200 parole, la creatività e le persone per cui si progetta

Questa è la **sesta lezione** del corso di **Analog Game Design**, e ha due anime. Nella prima parte si guardano insieme, uno per uno, i **giochi di ruolo in 200 parole** consegnati per il secondo task: il docente li legge ad alta voce come se li avesse appena scaricati, dice tutti i dubbi che gli vengono e poi chiede alla classe *quale emozione* il gioco voglia far provare, prima di chiederlo a chi l'ha scritto. Nella seconda parte, dopo una breve pausa, arriva un argomento nuovo, annunciato fin dalle prime battute ("oggi è ricerca e sviluppo"): **perché** e **per che cosa** si fa ricerca quando si progetta un gioco, che cos'è la **creatività** ("copiare, trasformare, combinare"), come si passa da un'idea a un gioco validato, e soprattutto **per chi** si progetta: il target, le persone vere che ci stanno dietro, la **mappa dell'empatia**. La lezione si chiude con una rassegna di **strumenti e siti** per la ricerca e con una lunga domanda che riprende il filo della prima parte: esiste un posto in cui si possa leggere *quale meccanica produce quale emozione*?

La lezione si svolge **online**, su Zoom, ed è tenuta da **due docenti**. La prima parte è di **Ruggero**[1], che conduce la revisione dei giochi; la seconda è a due voci con **Beatrice** ("Bea"), progettista di giochi, che entra durante la revisione, ascolta "in background" e prende la parola dopo la pausa; i due si passano la parola più volte ("passiamo la regia a Ruggero", "ripasso la parola a Bea"). La lezione si colloca con ogni probabilità **venerdì 25 settembre 2026**[2], dalle 17 alle 20 come sempre: alle cinque si aspettano gli ultimi arrivati, alle "cinque e dieci" si comincia, la pausa è di cinque minuti con ripresa alle **18:25**. Gli studenti che intervengono, e che il capitolo nomina, sono **Tiziana**, **Tancredi**, **Gianmarco**, **Giordano**, **Nicola**, **Angelo**, che arriva a lezione iniziata, e i **due Lorenzo** del corso[3]. **Rita** non c'è: seguirà la registrazione, e il suo gioco viene letto lo stesso, per ultimo.

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi, soprattutto quelli dei giochi, dei siti e dei luoghi stranieri. *itch.io* diventa "HIV/IO", "CIO" e "H.I.O.", *BoardGameGeek* "BoardGain Geek", lo *Spiel des Jahres* "Speed the Yard" e perfino "Hospital Yard", gli *haiku* "Aiko" e "IQ", il *kami* del gioco di Giordano "camion". Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante la pausa la trascrizione registra per decine di righe frasi senza senso ripetute in loop ("qual è la risposta che ci può essere?"), e qualche frase ripetuta compare

[1] Come nelle lezioni precedenti, il docente non si presenta; lo salutano per nome gli studenti all'inizio ("ciao Ruggero") e lo chiama così Beatrice nella seconda parte (la trascrizione scrive anche "Ruger" e "Rugger"). È lo stesso docente delle lezioni dalla terza alla quinta: si riferisce al nanogioco del paracadute "dell'altra volta" e alle meccaniche di sicurezza viste "la scorsa volta".

[2] La data non è detta, ma è ricostruibile: il secondo task era da consegnare "per il 25" (venerdì 25 settembre) e all'inizio della lezione successiva si sarebbero guardati insieme i giochi, come qui si fa; Beatrice augura "buon venerdì" e "buon weekend", e tutti si salutano con un "ci vediamo lunedì".

[3] In classe ci sono due Lorenzo, come si era visto nella prima lezione: Lorenzo Vergata e "l'altro Lorenzo". La trascrizione li distingue solo in un punto (il docente chiama per cognome Lorenzo Vergata durante la revisione del suo gioco); altrove il capitolo scrive semplicemente "Lorenzo" e, quando dal contesto si capisce che a parlare è l'altro, lo dice.

anche altrove ("devo andare in giro" all'inizio, "una specie di routine" a metà della parte di Beatrice); il capitolo le ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro e resta generico altrimenti.

Come leggere questo capitolo. La prima parte è un laboratorio: nove regolamenti brevissimi, nove modi diversi di provare a far nascere un'emozione con le regole, e una serie di osservazioni ricorrenti (le regole omesse, il *player count*, l'*agency*, i temi delicati, la coerenza della terminologia, la differenza tra un regolamento e un *pitch*) che valgono per qualunque gioco. La seconda parte è più teorica, ma il filo è lo stesso: un gioco si fa *per qualcuno*, e la ricerca serve a sapere che cosa esiste già, che cosa si vuole ottenere e chi lo giocherà. Molti degli strumenti appena accennati qui (il Double Diamond, il *brainstorming*, la fattibilità, gli stakeholder) sono sviluppati per esteso nella dispensa che Daniele prepara per il corso, riportata nel libro in coda al capitolo precedente, e nelle lezioni successive con Beatrice. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell e *A Theory of Fun for Game Design* di Raph Koster); sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa: le difficoltà del secondo task, un'emozione o un'area emotiva, troppe o troppo poche idee, come cercare un gioco che forse esiste già (sezione 2.6.1);
2. il metodo di lettura dei regolamenti (sezione 2.6.2);
3. i **nove giochi in 200 parole**, uno per uno (sezioni da 2.6.3 a 2.6.3.9), con una tabella riassuntiva (sezione 2.6.4);
4. **perché** e **per che cosa** si fa ricerca (sezioni 2.6.5 e 2.6.6);
5. **divergenza e convergenza** (sezione 2.6.7);
6. **che cos'è la creatività**: copiare, trasformare, combinare, e il problema del plagio (sezioni 2.6.8 e 2.6.9);
7. gli **step per validare un'idea**, a partire dall'obiettivo personale (sezione 2.6.10);
8. il **target**, la persona e la **mappa dell'empatia**, come raccogliere le informazioni, il target troppo largo (sezioni da 2.6.11 a 2.6.14);
9. le **risorse** e gli **stakeholder** (sezione 2.6.15);
10. gli **strumenti e i siti** per la ricerca (sezione 2.6.16);
11. le **domande finali**: meccaniche ed emozioni, il divario tra chi fa e chi gioca, le escape room (sezione 2.6.17).

2.6.1 L'attesa: un'emozione, troppe idee, lo stato dell'arte

Sono le cinque, e come sempre si aspetta che arrivino tutti. Il docente spiega "logisticamente" come funzionerà la lezione: come la volta precedente, una prima parte in cui si guardano i task consegnati, e una seconda parte con Beatrice, su **ricerca e sviluppo**, con una panoramica di strumenti che verranno ripresi anche nella lezione successiva, sempre con Beatrice, dedicata a *come preparare il proprio progetto*. All'inizio di quella lezione, inoltre, verranno comunicati i **gruppi**. Nel frattempo, "se ci sono domande, dubbi", lo spazio è aperto; "se no, aspettiamo qui come al solito nel silenziosissimo *cringe*".

Un'emozione o un'area? Le domande ci sono, ed è Tiziana ad aprire, raccontando il "dietro le quinte" del task: la sensazione che aveva era che **non fosse possibile individuare una singola emozione** da far vivere con il gioco. Si può individuare un *range* di emozioni, ma la cosa fondamentale è il tipo di esperienza che si vuole far fare; poi l'emozione la vive chi gioca, e su quella non si ha controllo. "Esattamente", risponde il docente: era uno dei pilastri della lezione sull'*emotional design*. **Non abbiamo controllo diretto sull'esperienza di chi gioca**; quello che possiamo fare è cercare di definire un'**area emotiva** in cui vogliamo che le persone vadano, sapendo che le persone sono diverse. La vera domanda è allora: *come si affina quell'area, come la si riduce al punto da essere più o meno sicuri che lì si andrà a parare?* Lo si vedrà proprio oggi, nella revisione dei giochi, e poi nella parte di ricerca e sviluppo. Tiziana immagina che si faccia provare il gioco alle persone e si raccolgano i feedback: sì, "c'è un po' di *foreshadowing*", ci sarà una lezione sul *playtest* e se ne parlerà lì, "ma non solo lì".

Da dove si parte? Gianmarco condivide "appieno" quello che ha detto Tiziana e aggiunge la sua difficoltà: **partire**. Gli sarebbe utile conoscere delle tecniche per farsi venire l'ispirazione, o di *brainstorming*. Se ne accennerà oggi, risponde il docente, ma la lezione successiva con Beatrice darà strumenti per il *brainstorming* e per le fasi che lo seguono: "non vi abbandoniamo". Per Gianmarco "è stato molto difficile partire"; una volta partito, il lavoro è andato, ma l'inizio è stato "abbastanza *painful*". L'altro nodo: faceva molta fatica a capire se partire **dall'esperienza** che voleva dare o **da una dinamica, da una meccanica** che trovava intrigante, e costruirci intorno il resto. È "un po' la parte difficile", conferma il docente, e anche di questo si parlerà, perché il rischio è di **affezionarsi alle meccaniche** che vogliamo vedere nel gioco e di aggiustare il gioco intorno a quelle; "ma magari quelle meccaniche non devono stare in quel gioco, ci stanno solo ostacolando". Da qui la questione della **convergenza** e della **divergenza** e il *kill your darlings*, che si vedranno più avanti. Gianmarco precisa: il lavoro che ha fatto non è innovativo, ma una volta quadrate le cose è andato dritto, con degli schemi per capire che cosa doveva coprire, recuperare, affinare.

Niente mazzi di prompt, niente colpi di scena. Tiziana torna con una limitazione che ha incontrato: dover **descrivere tutto nel regolamento**, perché non si poteva avere, per esempio, un mazzo di carte scritte. "Come tanti hanno chiesto: si può citare un mazzo di carte?" (anche su Telegram, ricorda il docente). La risposta: un **mazzo di carte standard**, da poker, va bene, perché è un **componente**, come si è visto in alcuni casi studio; un mazzo di carte *imprevisti* o di *prompt*, invece, no: "non è il tipo di formato in cui puoi starci", perché **nelle 200 parole deve starci l'intero gioco**. L'ideale è che una persona legga il 200 Word RPG e si chieda: *con le cose che ho in casa adesso posso partire a giocare? Sì o no?* Questo per Tiziana era chiaro; il problema era che così doveva **spoilerare** tutto quello che sarebbe successo. Certo, risponde il docente: il 200 Word RPG **non è il formato in cui giocare sul *twist* di trama**, sul colpo di scena, "perché avete 200 parole in tutto"; in quel tipo di gioco ci si deve basare su altro. L'ultima cosa di Tiziana è una domanda che si è posta per tutto il lavoro: "mi è venuta questa idea, la seguo, poi andrà come andrà: sarò riuscita a ottenere l'effetto che volevo?" "Quello adesso lo vediamo tra un attimo".

Troppe idee. Qualcuno è uscito dalla stanza (forse Rita); la classe è di nove persone, e manca ancora Angelo. Poi la parola passa a Lorenzo, che aveva dimenticato le domande "e pure di avercele", ma se ne ricorda almeno due. La prima è il problema opposto a quello di Gianmarco: avere in testa **troppe idee** "che si prendono a pugni", completamente diverse, al punto da scrivere tre giochi diversi, tutti da 200 parole, tutti proponibili, senza riuscire a dire quale sia palesemente il migliore. Il docente risponde in due tempi. Primo: niente impedisce di fare più di un 200 Word RPG per conto proprio, "non sono il poliziotto dei giochi che create". Secondo, e soprattutto: **nel task non interessa vedere il gioco migliore**, quello selezionato tra le proprie idee, ma vedere il **processo** con cui lo si fa e parlarne. "Le idee ce ne sono tantissime, ma è l'**esecuzione** di queste idee la parte importante". Anche questo tornerà oggi, con la convergenza e la divergenza, e nella lezione successiva con Beatrice.

Come si fa lo stato dell'arte di un gioco inventato? Sono le cinque e dieci, e Lorenzo ha "un'altra piccolissima domanda". Gli è capitato di improvvisare su due piedi, a una serata, un gioco di **mimo** di oggetti estraniati dal loro uso: prendo un oggetto, per esempio una trombetta, e lo mimo come se fosse un motorino; gli altri devono indovinare ("ah, quello è un motorino"), e chi indovina per primo diventa il

nuovo mimo. Non conosce giochi simili: come può fare la ricerca dello **stato dell'arte**, scoprire se esiste già e come si chiama? In parte lo si vedrà oggi, risponde il docente, perché la seconda parte è anche sulla *ricerca*, non solo sullo sviluppo. Intanto, esistono **terminologie** e **framework** che aiutano, anche in modo crossmediale. Nell'ambito dei *party game* certe cose non sono ben definite, ma gli stessi termini esistono nel **LARP**: se parlo di **prop**, cioè di un oggetto che uso per simboleggiare un altro oggetto, posso codificare quelle regole in modo non ambiguo. È un altro "vaso di Pandora": quello di **come si scrivono le regole**, che probabilmente ha frenato molti nel task, perché scrivere regole chiare e non ambigue in 200 parole è molto difficile. Parte della risposta arriverà oggi, parte più avanti, e "una grossa parte" nella lezione sul **paragame**.

2.6.2 Come si legge un regolamento

Rita è assente e seguirà in differita, e Angelo non è ancora arrivato: i loro giochi, dice il docente, li aprirà alla fine e li leggerà lo stesso; ovviamente non ci sarà chi li ha scritti a fare da controaltare, ma "sentitevi liberi di mandarmi un'email, un Telegram, e ne parliamo anche in differita". Non è che non se ne parla solo perché qualcuno non c'è alla lezione.

Poi spiega il metodo, che vale per tutti i giochi:

- il docente apre il gioco a schermo e **legge il regolamento ad alta voce**, esprimendo **tutti i dubbi** che gli vengono mentre legge;
- chi l'ha scritto **non risponde** finché la lettura non è finita: il docente si mette "nei panni di una persona che ha appena preso questo gioco, scaricato da itch.io[4], preso a una fiera, non importa", e che lo legge in autonomia, a casa propria; chi l'ha scritto **non è al tavolo** di chi lo gioca;
- il consiglio a chi ha scritto il gioco: **segnarsi le cose ambigue**, quelle che potrebbero essere spiegate meglio, i dubbi che spuntano;
- finita la lettura, il docente chiede alla **classe** che emozione il gioco voglia dare ("voglio almeno tre persone che mi diano tre opinioni, probabilmente diverse"), e solo dopo chiede all'**autore** o all'autrice che cosa volesse ottenere;
- infine il controllo del conteggio delle parole (il *word count*) e il verdetto: *il gioco c'è*, è giocabile, "*thumbs up*".

"Non abbiate paura: non vi do un voto per indovinare o meno l'emozione. Anzi, se siete in dubbio, ditela". E, come era stato detto, è difficilissimo che un gioco dia una sola emozione: "non per colpa di chi l'ha scritto, ma perché è ovvio".

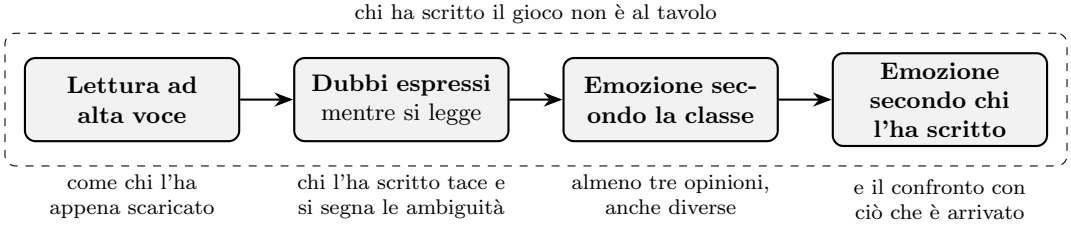


Figure 2.6.1: Il metodo con cui il docente legge i giochi in 200 parole: il regolamento deve reggersi da solo, senza che chi l'ha scritto possa spiegarlo. Il confronto tra l'emozione percepita e quella voluta mostra quanto l'area emotiva sia stata ristretta.

Chi vuole iniziare? "Tanto dobbiamo andare a tutte e tutti". Rompe il ghiaccio Tiziana ("dai, tanto è uguale"), e subito dopo alza la mano Giordano. Il docente apre lui stesso il documento e lo condivide.

[4]La trascrizione dice "HIV/IO"; più avanti "CIO" e "H.I.O.". È *itch.io*, la piattaforma di pubblicazione indipendente di cui si parla nella seconda parte della lezione.

2.6.3 I giochi di ruolo in 200 parole

I giochi sono nove, uno per ogni studente; li si vede nell'ordine in cui il docente li apre. Di ciascuno il capitolo riassume il regolamento come lo legge il docente (senza riprodurlo parola per parola: la trascrizione è troppo rovinata per farlo, e alcuni passaggi sono ricostruiti dal senso), i dubbi emersi durante la lettura, le emozioni proposte dalla classe, quella voluta da chi l'ha scritto e il commento finale.

2.6.3.1 Tiziana: *Parlatene*

Il regolamento. *Parlatene* è un gioco per **2–4 giocatori**: uno impersona il **medico** di un ospedale, gli altri i potenziali **pazienti**. Subito dopo, una riga: *puoi non giocare se nella tua vita soffri di malattie e ciò ti turba*. Servono un **mazzo di carte francesi**, fogli e penne. Il primo che pesca una carta di **picche** è il medico. A turno, ciascun paziente si presenta al medico, descrive i suoi sintomi e brevemente la propria vita; tutti ascoltano. Prima di parlare, ognuno pesca una carta dal mazzo e non la mostra: il **seme** decide i sintomi (cuori: lievi e occasionali; fiori: persistenti; quadri: non permettono di dormire; picche: interferiscono con le normali attività). Il medico, prima di comunicare la diagnosi, **che è sempre nefasta**, pesca a sua volta una carta senza mostrarla, e il seme decide *come* la comunica: con cuori mostra i lati positivi della situazione; con fiori cerca di indorare la pillola, con molti giri di parole; con quadri spiega tutto nel minimo dettaglio, in modo rigoroso e scientifico; con picche comunica direttamente la diagnosi. Alla replica dei pazienti, il medico risponde che non c'è nulla da fare e che possono decidere di ricoverarsi per le cure palliative. Alla fine ognuno scrive al medico, su un foglio, come si sente, e lo legge ad alta voce. Chiude il titolo: "Quindi: parlatene".

I dubbi. È un **freeform**, a metà tra il gioco di ruolo da tavolo e il LARP, perché lo si gioca da seduti, "come di fatto siamo quando parliamo col medico". Ci sono **regole omesse ma implicite**: "il primo che pesca una carta di picche è il medico" implica che si sta pescando per decidere chi sia il medico, ma il regolamento dà direttamente il risultato dell'azione senza descriverla. Il **valore** delle carte non viene usato, conta solo il seme: lo si deduce dal fatto che il numero non è mai citato. Altre cose sono molto in stile freeform: non si dice quanto dura la chiacchierata, né quando si passa al paziente successivo. "Tutte cose omettibili": il docente potrebbe prendere il gioco adesso e giocarlo con altre tre persone, "non spuntano particolari dubbi".

Una meccanica di sicurezza. La riga *puoi non giocare se...* è "molto interessante": è una forma di **meccanica di sicurezza**, rientra nella parte di *safety*. Nello specifico ricorda un approccio "svedese": se penso che non siate in grado di giocare il gioco perché tocca cose vostre, lo esplicito direttamente. La stessa cosa si può mettere in mille modi, per esempio come **content warning**. Messa così, però, è **molto forte**: sembra che sia il gioco a decidere se possiamo giocare o no, mentre il content warning è più morbido ("si parlerà di malattie": poi decido io se giocarlo, senza che mi si dia il permesso). Tiziana precisa che il senso era proprio "decidi tu". "No, ci sta. È solo scritto in una formula che ho visto poche volte, e mi ha fatto un attimo ragionare". Non c'è niente di sbagliato: è un **freeform da camera** (*chamber*), che rientra anche tra i **parlor game** fatti di un'unica conversazione.

Le emozioni. Mentre la classe ragiona, entra Beatrice ("ciao, Beatrice, ti ho vista adesso"). Lorenzo vede **ansia**; Giordano **impotenza**, a seconda di come escono le carte; Gianmarco, senza sapere se sia sulla ruota delle emozioni ("la ruota era uno spunto, non vi interrogherò sulla specifica ruota"), un tema di **condivisione del problema**. Tiziana voleva dare **frustrazione**, un **senso di sconforto**. Ci sta, commenta il docente, con l'impotenza di Giordano ma anche con l'ansia di Lorenzo: c'è un problema, sul piano narrativo, che non solo non si può risolvere, ma che probabilmente l'interazione con il medico esaspera. E c'è, "un po' subdolo e un po' omesso", un **debriefing** nascosto nelle ultime due parole: *parlatene*, cioè fate poi un debriefing.

Il verdetto. 199 parole: "siamo a postissimo". Tiziana ha dovuto tagliare. "Mi sorprenderebbe se qualcuno tra voi non abbia dovuto tagliare: sono 200 parole". Il suo caso è curioso: verso la fine, senza la conclusione, era a 189 parole, e le 11 rimaste non bastavano; ha dovuto limare e cercare che cosa togliere

senza perdere il senso. "Molto meglio di quello che succede di solito", cioè finire, guardare il conteggio, scoprire di essere a 450 e doverne tagliare più della metà: "lì si suda freddissimo".

2.6.3.2 Giordano: *Yamanobe no Michi*

Il regolamento. 200 parole tonde ("attenzione!"). *Yamanobe no Michi*[5] è un gioco di ruolo per **tre giocatori**; servono carta e penna. Un giocatore interpreta un **kami**[6] che veglia sugli altri due, che impersonano dei **pellegrini**. Il loro cammino è diviso in **cinque tappe**: il santuario di Isonokami, il villaggio rurale di Takenouchi, i *kofun* (i tumuli funerari) di Kurozuka, i frutteti e il sacro monte Miwa. Il kami descrive la prima tappa in forma di **haiku**, una breve poesia di tre versi; poi i due giocatori scrivono la propria azione nello stesso stile. Il kami assegna un punteggio a ogni componimento in base al contenuto: un punto se descrive le stagioni, uno se cita un animale, uno se cita una figura storica, uno per un elemento del folklore, uno se la metrica è **5-7-5**. Il kami può assegnare un **punto extra** a un pellegrino; l'altro può contestarlo **con un haiku**, e il kami può riassegnare il bonus. Si ripete tutto per ogni tappa. Il pellegrino che termina il viaggio con **meno punti** ripete il suo ultimo componimento **come un mantra**. In fondo, il testo che il kami recita (nella trascrizione: "prendi il pennello e mostrerà un lieto cammino; la poesia, come Yatagarasu, è guida fidata; ogni lento passo non cantato è inesistente; ... un umile monaco recita sutra", ricostruibile solo in parte).

I dubbi. A metà lettura il docente si ferma: "riassegnare il bonus" vuol dire che i punti sono **mutuamente esclusivi**? Se io prendo il punto per le stagioni, l'altro non può prenderlo? "Non lo so ancora; vediamo come va avanti". Alla fine: è più un gioco di ruolo da tavolo; tre persone, chiarissimo; il tema chiarissimo, e le meccaniche **molto coese con il tema**. Qualche dubbio sul lato meccanico: il punto extra lo decide il kami, immagino, ma non è specificato *come*. L'altro pellegrino può contestarlo con un haiku: ma per contestarlo deve sapere *perché* è stato assegnato, o lo contesta per il solo fatto che l'altro l'ha preso? E la contestazione può essere ricontestata? Chiaro invece il finale: l'ultimo componimento è quello della quinta tappa. Il testo del kami in fondo è l'**inizio del gioco** (Giordano conferma: il kami legge il regolamento e poi lo recita all'inizio), ma è messo dopo la fine; e non è chiaro se il kami descriva in haiku solo la prima tappa o l'inizio di ogni tappa. "Comunque sono davvero piccolezze".

Un parente: *Haiku Kaiju*. Il gioco ricorda al docente un altro gioco che usa gli haiku come meccanica, dello stesso designer del nanogioco del paracadute "in cui ci siamo schiantati al suolo l'altra volta"[7]: *Haiku Kaiju*, in cui si descrive un *kaiju* alla Godzilla che arriva in città, spacca tutto e se ne va, tramite haiku su quello che fa. "Questo mi sembra ancora più coeso tematicamente": molto liscio, *smooth*.

Le emozioni. Lorenzo dice **creatività e giudizio**; qualcuno, "sarà per il Giappone", **serenità**. Tiziana non è riuscita a entrarci: gli haiku le piacciono tanto, ma non le è arrivato niente. Giordano voleva proprio **serenità e creatività**. Nel primo *draft* c'era la **competizione**: il primo che arriva con più punti vince. Poi ha **rovesciato** la cosa per indirizzare il gioco verso la tranquillità: nessuno vince, semplicemente chi arriva secondo deve riflettere un po' su quello che ha scritto. Qualcuno osserva che la creatività in sé non è un'emozione; semmai, scrivendo, può venire un po' di ansia da prestazione ("oddio, mi deve venire in mente come scrivere"), che però, in una situazione tranquilla e non competitiva, si stempera.

***Tokaido* e i giochi che non sono per noi.** Esistono già giochi da tavolo che vanno in una direzione simile: *Tokaido*, un gioco a punti ambientato in un pellegrinaggio, "sempre a sfondo giapponese", in cui a ogni locanda ci si ferma e si aspettano gli altri prima di ripartire; non bisogna stare primi, e vince chi, tra virgolette, **si gode di più il viaggio**. Poi (mentre entra Angelo: "ti segno presente, così posso chiudere

[5] La trascrizione scrive "Yamanobe Nomici" e "Yamanobano Amici". Lo *Yamanobe no michi* è l'antica strada che costeggia le colline a est del bacino di Nara, in Giappone, considerata una delle più antiche del paese; le tappe del gioco sono luoghi reali lungo il percorso. I nomi che seguono sono ricostruiti dalla trascrizione ("Isokami, Isonokami", "Takenuchi", "Kofon di Kuroza", "Yattagarasu"); in particolare "Kuroza" è con ogni probabilità il tumulo *Kurozuka*.

[6] La trascrizione scrive spesso "camion".

[7] Il nome del designer nella trascrizione è irriconoscibile ("Switch Designer di Cernodate Grammi Trollari"); il capitolo non prova a ricostruirlo.

l'appello, che mi dà sempre moltissima ansia") il docente sottolinea una cosa già detta: **i giochi non sono per tutti**. Un gioco fatto bene come design non è necessariamente un gioco che ci fa dire "che bello, voglio giocarlo"; molti giochi fatti bene **non sono per noi**, o non lo sono con un certo gruppo ma lo sono con un altro. Il verdetto: "c'è tutto", e in 200 parole tonde.

2.6.3.3 Gianmarco: *Beniamino va all'università*

Il regolamento. Il conteggio stupisce di nuovo il docente: "ma siete precisi? Ma chi siete? Avete un range di più o meno una parola, incredibile". Gianmarco è partito da 300. *Beniamino va all'università* è un gioco di ruolo per **5 giocatori**: si costruisce la vita di Beniamino, i cui genitori stanno divorziando mentre lui deve scegliere l'università. Un giocatore è **Beniamino** e si descrive; poi assegna a **due giocatori** un **valore** ciascuno, scelto tra *libertà, avventura, edonismo, successo, prestigio, sicurezza, dovere, radici, affetti* e *ideali*. Gli altri due sono **voci interiori**, dormienti. Si giocano **tre round**. Beniamino racconta un **dilemma** di vita (il primo è l'università, i successivi riguardano la sua vita). Ogni valore propone una **strada** coerente con sé, attraente, ma **con un prezzo**; le voci interiori commentano le strade. Tutti, Beniamino compreso, **votano in segreto** su carta ciò che ritengono meglio per Beniamino, poi si rivela: la strada più votata accade. Se Beniamino l'ha votata, è **in armonia** con sé stesso. Nei pareggi decide lui, ma mai per la strada che ha votato. Il valore che ha proposto la strada ne racconta le conseguenze: positive se c'è armonia, altrimenti un evento drammatico. Poi Beniamino sceglie quale voce interiore si risveglia e diventa un valore: se c'è stata armonia nel voto, lo sceglie Beniamino dalla lista; altrimenti lo sceglie la voce interiore che sta diventando un valore. Alla fine del terzo round, se ci sono state due armonie, Beniamino sceglie quale valore lo rappresenta; altrimenti scelgono i valori.

I dubbi, risolti leggendo. Il docente si chiede subito come tornino i conti: cinque giocatori, uno è Beniamino, due ricevono un valore da una lista che ne ha più di due, quindi alcuni valori non verranno usati; "per ora andiamo avanti e lo scopriamo". Le altre due persone sono le voci interiori. Alla fine: "bellissimo. È tutto coeso e coerente". È anch'esso un gioco di ruolo da tavolo. Per i limiti di parole alcune cose sono scritte in una forma che in un regolamento diverso avrebbe più **qualità della vita**: il "listone con le virgole", per esempio, di solito diventa un **elenco puntato**, anche solo per facilità di lettura. Ma c'è tutto, e il *twist* piace molto: "poveretto, questo Beniamino, mi fa già pena; ma sono sicuro che avrà almeno una partita su tre in cui gli andrà bene".

Le emozioni. Qualcuno dice **speranza**; Angelo, a cui il docente spiega in due parole che cosa si sta facendo, **incertezza** ("mi viene confusione, ma non è confusione"); qualcun altro, per controbilanciare, **disperazione**. Lorenzo elenca i valori, "sono tanti". Qui il docente fa una precisazione importante: **l'emozione non la si fa provare scrivendola**. Se nel regolamento c'è scritto *successo*, il gioco non diventa automaticamente un gioco sul successo. Ciò che dà le emozioni è la **parte meccanica**: le voci interiori e i valori che propongono alternative. Beniamino è sballottato a destra e a sinistra da proposte che non può decidere in termini di *framing*, ma solo di reazione: "io butto la mia, ma poi dentro di me è un casino tale che vai a sapere". Il docente lo chiama **sballottamento emotivo**: non è sulla ruota, "ma il senso di essere una pallina del *pachinko* c'è"[8]. Bello anche il *flavor*, che non è meccanica ma fa già dire "poverino": i genitori che divorziano, l'università. Gianmarco voleva dare proprio quel senso di ***out of control***, di non riuscire a guidare le situazioni: "tradotto in sballottamento della pallina del pachinko, perfetto". **199 parole**: il gioco c'è.

2.6.3.4 Nicola: *Finché resta cenere*

Il regolamento. **189 parole**, "abbiamo 11 parole in meno". E c'è il **grassetto**: "attenzione, questa è un'occhiataccia". Non era vietato, ma è qualcosa che nel 200 Word RPG non si sarebbe potuto fare. *Finché resta cenere*: siete l'ultimo gruppo di missionari in un mondo mangiato dal gelo, incaricati di portare la **fiamma** alla **caldera** per riscaldare il mondo. Per due o più giocatori[9]. Il viaggio attraversa la notte bianca in **tre tappe**. Ognuno annota **tre ricordi umani**: un amore, una colpa, un sogno. La fiamma è alla fine

[8] La trascrizione dice "pacinco" e "placino".
[9] La trascrizione dice "tu e più giocatori"; con ogni probabilità il testo indicava "2+ giocatori".

della sua vita e ha **cinque tizzoni**. A ogni tappa si interpreta il gelo che avanza; poi **pugno chiuso al centro** e, dopo un conto alla rovescia, ognuno **apre la mano vuota** o **tiene il pugno stretto**. Gli esiti:

Tutti donano: proteggete il fuoco con la vostra anima; la fiamma perde **un** tizzone e ognuno cancella un ricordo. Il gelo cancella la vostra memoria, ma restate umani insieme.

Presa contro dono: chi fa la presa **ruba il calore** e preserva i propri ricordi; chi ha donato viene tradito e perde un ricordo; la fiamma perde **due tizzoni per ogni ladro**.

Tutti presa: nessun calore; ognuno perde **due ricordi**, la fiamma due tizzoni.

La conclusione: se la fiamma arriva a zero, o non resta nessun umano con dei ricordi, cala l'**oblio eterno**; se giunge alla caldera, narrate chi o che cosa siete diventati all'ascensione del nuovo mondo.

Il dilemma del prigioniero. È il **dilemma del prigioniero** come tipo di dinamica, "una cosa frizzantina nel giocarla". Le emozioni proposte: un senso di **insicurezza** ("magari io gioco la mano aperta e gli altri mi fregano": il dubbio di essere pugnalati alle spalle) e uno di **sollievo**, se dopo l'esito c'è stata condivisione; il **rimorso**, se per un'azione sbagliata fallisce l'obiettivo ("io rubo alla fine, sono gli ultimi tizzoni, e moriamo tutti"), o anche solo dover rinunciare a un ricordo in particolare. Tiziana lo trova molto articolato: il dono e la presa mettono in luce cose contrapposte, il calore contro il freddo, l'altruismo contro l'egoismo. È il classico, commenta il docente: sulla carta dovremmo tutti collaborare, tutti doniamo e arriviamo tranquillissimi; ma il gioco funziona in modo tale che chi non collabora ne trae un vantaggio diretto, e quindi c'è un conflitto **comunità contro individuo** "sempre molto frizzantino".

Che cosa voleva Nicola. La **sfiducia**, soprattutto. Nicola adora il dilemma del prigioniero nei giochi di ruolo, "lo metto sempre", e ha deciso di farci un gioco: ci siamo accordati, ma se gli altri non lo fanno? Questo porta a **esitare**, e a giocare anche la mano che non si vorrebbe: si ruba per paura degli altri. "Funziona", ed è coeso anche nelle 200 parole: "bravo, Nicola".

Una domanda tra studenti. Qualcuno chiede a Nicola un chiarimento: se tutti donano per tre round, i ricordi finiscono e si perde? No: si arriva alla caldera, anche se le persone sono "finite". Lo studente propone, "da un punto di vista personalissimo", che per vincere qualcuno debba tradire almeno una volta: una questione più filosofica. Per Nicola, però, il bello è **non sapere** se un traditore ci sia o meno, non sapere che deve esserci per forza; e questo toglierebbe l'esitazione. Qualcun altro confessa che come emozione gli ha dato **rabbia**: "penso sempre bene degli altri", e nei dilemmi del prigioniero può capitare di essere traditi all'ultimo.

2.6.3.5 L'altro Lorenzo: *Out There*

Il regolamento. 141 parole, "ma lusso, siamo lisciissimi". *Out There*[10] è un gioco di ruolo **post-apocalittico** in cui un giocatore è il **leader** di una comunità e gli altri sono **sopravvissuti** che devono scontare una pena. A inizio gioco il leader stabilisce, in base al numero di giocatori, quante risorse del mondo esterno i sopravvissuti devono raccogliere entro **tre giorni** per non essere esiliati. Ogni round i sopravvissuti tirano **1d20** per determinare quante risorse trovano. A fine round il leader controlla le risorse: il sopravvissuto che ne ha raccolte **di meno** deve convincere il leader a dargli un'altra possibilità, **giocando la scena**. Se il leader è convinto, il sopravvissuto ha un'altra chance; altrimenti viene **esiliato** e smette di giocare. Allo scadere dei tre giorni, i sopravvissuti che hanno trovato il numero di risorse richiesto restano nella comunità; altrimenti vengono esiliati tutti.

I dubbi meccanici. Il primo: **i pareggi**. Se due sopravvissuti hanno lo stesso numero minimo di risorse? Lorenzo l'aveva negli appunti e si è dimenticato di scriverlo: entrambi fanno appello e il leader sceglie. Ma chi legge come lo risolverebbe? "Con il buon senso"? "Sei tu il designer: sei tu che devi decidere che cosa vuol dire buon senso nel tuo gioco". Il secondo: il **numero di giocatori**. Se si gioca in due, uno è il leader

[10] Il gioco è dell'altro Lorenzo: più avanti il docente chiama per cognome Lorenzo Vergata a proposito di un altro gioco, e durante questa revisione "anche Lorenzo" interviene con un suggerimento.

e l'altro è per forza quello che raccoglie meno: viene esiliato e il leader resta da solo. Serve un minimo, e serve dire "da quanti a quanti". Il terzo: tre round, quindi **tre esili**. Così com'è scritto, **non c'è modo di non essere esiliati**, perché non c'è una **soglia** da superare: qualcuno avrà comunque raccolto meno degli altri, anche se ha fatto 20, 20, 20 e 19 e ha portato "una tonnellata di risorse".

Le emozioni. Prima di arrivare all'ultimo punto, la classe. Per uno studente il gioco dà, forse escluso chi viene esiliato, un senso di **esclusione**; per Gianmarco di **risentimento** ("sono andato a lavorare e vengo pure esiliato"). Questi però sono dal punto di vista della comunità: *il leader che cosa prova?* Per Giordano è un po' **isolato**, sta lì a guardare cosa fanno gli altri, è quasi **impotente**; anche gli altri lo sono, visto che è il dado a decidere. Tiziana invece vede nel leader **un gran potere**: controlla le risorse e può accogliere o no la richiesta.

Dov'è l'*agency*? Ed ecco il punto: l'**agency** di chi gioca. Dal punto di vista dei sopravvissuti, l'ambito decisionale potrebbe non esistere. Supponiamo che io non sia mai quello che raccoglie meno, né quello che raccoglie di più: sto in mezzo, e ogni round mi salvo. Allora il mio gioco non ha contributi narrativi di nessun tipo: non giocherò mai una scena, tirerò solo dadi senza avere neanche uno *spotlight*. "Qui, paradossalmente, **per giocare devo perdere**": per avere una scena di *roleplay* devo essere a rischio di esilio. Il docente precisa che l'approccio *play to lose* della scena nordica, già menzionato ("muoiono tutti", il *focus* sul momento critico), è un approccio giusto; ma qui la situazione è diversa: non è che verranno tutti esiliati uno dopo l'altro, è che **ci possono essere persone in partita senza agency**. Dipende da che emozione si voleva dare. Lorenzo spiega che c'era un'**ambivalenza**: per i sopravvissuti l'**inadeguatezza** e l'**impotenza**; per il leader il **giudizio** e il **controllo**. Il gioco è giocabile, conclude il docente; l'unico consiglio è: **attenzione a tutte le meccaniche che tolgono agency a chi gioca**. Come nel **gioco dell'oca**: puoi anche vincerlo, "ma sei tu che hai vinto, o è il dado che ha deciso che tu hai vinto?"

Due proposte dalla classe. Uno studente suggerisce che chi ha raccolto abbastanza risorse possa **intervenire a favore** di chi ne ha raccolte meno: "io ti do delle mie risorse così andiamo in pari e nessuno viene eliminato", oppure "ho deciso che vieni eliminato, non ti do niente"; si creano dinamiche di gruppo interessanti. Anche Lorenzo (l'altro rispetto all'autore) stava per dire qualcosa di simile: una **componente democratica**, anche minima, in cui il leader ha un potere decisionale più alto, ma se tutto il popolo decide "no, stai dicendo una gran boiata", è il leader ad andare in esilio. Il docente avverte che così cambia molto il tema del gioco e l'emozione, e servono più parole: la domanda è se ci stia nelle 200. "Forse una cosa del genere potrebbe starci". L'autore vuole sperimentare: "non mi convinceva del tutto neanche a me".

2.6.3.6 Tancredi: *Qualcosa di più grande*

Il regolamento. 200 parole tonde: "ma chi siete?". *Qualcosa di più grande* è un gioco di ruolo per **tre persone**. **M** è una persona normale finita in una situazione più grande di lei: senza saperlo, ha salvato il **Protettore**, la divinità più potente che esista, dal tentativo del **Vendicatore** di spodestarlo. Infuriato per il fallimento, il Vendicatore vuole uccidere M, ma il Protettore, riconoscente, non lo permetterà. I **ruoli**: si prendono da un mazzo il **2 di quadri** (M), il **jack di picche** (il Vendicatore) e il **re di cuori** (il Protettore), si mescolano e se ne consegna una a ciascuno: quello è il ruolo da interpretare[11]. Ognuno caratterizza il proprio personaggio e poi, insieme, si definisce meglio la trama: che cosa ha fatto ciascuno nella propria vita, quale trappola aveva preparato il Vendicatore, che cosa ha fatto M per salvare il Protettore. Il gioco procede per **scene**, ognuna in tre fasi: **azione** (chi è M imposta una scena che lo ritrae mentre fa qualcosa), **vendetta** (il Vendicatore arriva e cerca di uccidere M con tutti i suoi poteri), **salvataggio** (il Protettore interviene e salva M). Quando ogni giocatore è soddisfatto, si passa alla scena successiva **o si cambiano i ruoli**.

Ruoli o personaggi? È un gioco di ruolo da tavolo. Il dubbio riguarda l'ultima frase: "si cambiano i ruoli" vuol dire che la scena dopo io gioco M e tu il Protettore, e poi si ruota ancora? E il personaggio di M

[11] L'associazione tra carte e ruoli è ricostruita: la trascrizione dice "il 2 di quadri, il jack di picche e il re di cuori, protettore"; l'ordine fa pensare a M, Vendicatore, Protettore.

resta M, o può diventare il Vendicatore il personaggio che prima era Vendicatore? Probabilmente, ipotizza il docente, si intendeva che **ci si scambiano i personaggi**, e i ruoli restano associati ai personaggi. Tancredi spiega da dove viene la regola: non aveva capito bene se **l'emozione dovesse essere provata da tutti** i giocatori. "Ma è impossibile, esatto". Visto che nel gioco l'emozione è di uno dei personaggi, l'idea era di fare tutte le scene con gli stessi personaggi e poi cambiare, ricominciando il gioco.

Le emozioni. Per Tiziana risalta la **vendetta**, dal lato del Vendicatore. Ma che cosa provano M e il Protettore? "È un asimmetrico, veramente". M potrebbe provare **paura** ("vogliono ucciderlo, ci sta"); il Protettore l'**ansia** di riuscire a proteggerlo. Dal regolamento, però, sembra che **M sia il fulcro**: Protettore e Vendicatore si interfacciano al gioco in base a M, e quindi l'emozione di M è il cuore di come il gioco va guardato meccanicamente. È poi molto **freeform**: non dice quanto dura una scena né come deve essere, lascia tantissima libertà, tenendo però l'**architettura dall'alto** (M è M, il Vendicatore il Vendicatore, il Protettore il Protettore). Tancredi voleva dare la **gratitudine** di M verso il Protettore, che continua a proteggerlo; riconosce che la paura ha senso, perché alcune cose non sono chiarissime. Ci sta, risponde il docente: la gratitudine nasce proprio dal fatto che qualcuno tenta regolarmente di ucciderti, quindi la paura di sopravvivere c'è comunque, e le due cose si intrecciano.

Non delegare il design al tavolo. L'ultima osservazione riguarda la frase finale ("quando ogni giocatore è soddisfatto, si passa alla prossima scena o si cambiano i ruoli"): "qui tu non stai facendo design, stai dicendo al gruppo *fate un po' come volete*. Però il gioco è il tuo: sei tu che devi capire qual è la versione migliore del gioco". È come preparare un gioco e dire alla fine: "ho tre finali in mente, scegliete quello che volete". Ci sta, se il gioco è proprio su quello, se si vuole fare un punto tematico sul fatto che i finali li decidono i giocatori; altrimenti sembra solo che non si sia sicuri di che cosa far giocare. Vale per entrambe le cose, il passaggio di scena e il cambio di ruoli, ma pesa di più il secondo, perché è un **cambiamento meccanico grosso**: è ciò che trasforma il gioco da **asimmetrico** a gioco **a ruoli condivisi**, in cui M non lo gioco solo io, lo giochiamo tutti. Sull'idea che tutti debbano provare la stessa emozione: "questo è il tuo lavoro", ma l'intento deve essere comunque un controllo, capire che cosa si vuole che il gioco trasmetta. Il gioco c'è: *thumbs up*.

2.6.3.7 Angelo: *Una brutta serata*

Il regolamento. 147 parole. *Una brutta serata*, per **due giocatori**: un uomo cammina lungo un ponte quando nota sul bordo qualcuno pronto a lanciarsi. Chi interpreta la persona che vuole togliersi la vita vince se si butta; chi interpreta il **passante** deve convincerla a rinunciare. La prima sta **in piedi su una sedia**, il passante **alle sue spalle**. A turno, la persona sulla sedia espone un motivo per buttarsi, e il passante risponde con una soluzione al problema o un motivo per vivere; poi arriva un nuovo motivo da disinnescare, e così via, finché uno dei due non riesce più a rispondere o a dare un motivo credibile. Se vince chi è sulla sedia, scende **saltando**; altrimenti si gira verso il passante e scende **tenendolo per mano**. Superati i **5 minuti** di gioco arrivano i soccorsi, e il passante vince.

Un tema delicatissimo, senza protezioni. Il tema è delicatissimo, e qui, contrariamente al gioco di Tiziana, **non ci sono content warning**, che forse ci sarebbero stati bene; e forse anche uno o due **meccanismi di sicurezza**. È vero che il limite di parole è strettissimo, ma il tema lo è di più: così com'è, il gioco **delega tutta la sicurezza** di chi gioca a chi gioca. È proprio il tipo di gioco in cui un content warning o una meccanica di sicurezza il docente l'avrebbe apprezzato. Angelo ammette di non averci pensato. "Non è un problema: devi anche imparare, va benissimo". E il docente chiarisce il perché: **non per non fare giochi su questo tema**, che esistono già, né per scoraggiare chi vuole farli; ma "se volete farli, **prendetevi cura** di chi li sta giocando". E se fosse un gioco più grosso, pubblicato, con un altro *scope*, sarebbe il tipo di gioco per cui pagare un **consulente** specifico sul tema[12].

[12] La trascrizione dice "un cultural consultant, un sesto consultant": con ogni probabilità un *sensitivity consultant* o un consulente esperto del tema, in linea con quanto detto nella lezione precedente sui giochi che trattano temi difficili.

Le emozioni. Con questo tema, tante emozioni possibili, e negative: qualcuno ne vede di positive? Uno studente vede **disperazione** da entrambe le parti: di chi vuole buttarsi e di chi tenta fino all'ultimo di salvarlo. Un altro lo apprezza per lo **spunto di riflessione**: ci si sforza di empatizzare con i motivi di chi vuole lanciarsi. Il docente ne loda la **coesione** meccanica: la sedia, gli elementi "di *dexterity*, tra virgolette", lo stare in piedi su una sedia danno un **coinvolgimento cinestetico** tale che questo **non è un gioco di ruolo da tavolo**: è praticamente un **chamber LARP**, un freeform, in cui si è fisicamente il proprio personaggio. Anche il dare le spalle, e altre scelte, sono molto belle come **metaforizzazione** della situazione in una scena da camera. Angelo non è sicuro neanche lui dell'emozione; tra tutte, una **sensazione di impotenza**, sia di chi si butta sia di chi cerca di aiutare. "Direi che ci sta". Il gioco c'è; per il futuro, se il tema è delicato: **content warning e meccanismi di sicurezza**, come visto la volta precedente.

2.6.3.8 Lorenzo Vergata: *What Remains of the Stories Behind Us*

Il regolamento. Lorenzo l'ha scritto **in inglese**, perché aveva visto gli esempi in inglese; nessun problema, il docente lo traduce al volo. **190 parole**. *What Remains of the Stories Behind Us* ("cosa resta delle storie dietro di noi") è un **LARP cooperativo per tre persone**. Due persone, selezionate a caso, sono una **persona non vedente** e una **guida**. Si intraprende un viaggio fisico, *a physical round trip*, attraverso la vita della persona non vedente, in **tre capitoli**: la perdita della vista (dalla nascita ai 35 anni), la mezza età (dai 35 ai 70) e la vecchiaia (dai 70 in poi)[13]. La guida improvvisa una storia incentrata sulla persona non vedente; le altre persone fanno il **framing** degli eventi usando oggetti, spazi e interazioni; possono parlare alla persona non vedente, mentre la guida può temporaneamente scomparire. Il viaggio finisce dove è iniziato. Nella vecchiaia la persona non vedente **riottiene la vista** oppure **muore**, in base agli eventi giocati e alle scelte, inclusi dei *supernatural bargains*, dei patti soprannaturali. Se la vista torna, la persona racconta la propria vita dalla propria prospettiva, ricostruendo eventi, movimenti, oggetti, relazioni, come Papà Castoro che racconta la storia ai nipoti. Se muore, le altre persone (i *performers*) ricordano la sua vita dalla loro prospettiva, ricordando come ha reagito agli eventi, come i nipoti che si raccontano la storia di famiglia, dividendola in parti uguali, ciascuno dando corpo (*embodiment*) alla persona. Infine: se la storia **stalla**, la vita si riavvolge e altre persone diventano la persona non vedente e la guida.

I dubbi. Mentre legge, il docente si chiede se la guida sia solo un **ruolo** o anche un **personaggio**: all'inizio sembrava un personaggio che viaggia con la persona non vedente, poi sembra più un **custode tematico**. Alla fine, le osservazioni:

- "**cooperativo**" è un termine non necessario per un LARP, perché praticamente tutti i LARP sono implicitamente una co-creazione cooperativa della narrazione;
- i *supernatural bargains* non sono una meccanica: si legge come un implicito "potrebbe accadere una cosa del genere" sul piano narrativo, ma arriva *out of the blue*. Fin lì il gioco era di **realismo contemporaneo**, "stile Jeepform"[14]; con i patti soprannaturali **si cambia genere in corso d'opera** senza che il gioco l'abbia detto;
- il riferimento a **Papà Castoro** è **culturalmente specifico**: a seconda del paese, si può non sapere chi sia. In un regolamento è rischioso, perché lo capisce solo chi l'ha visto, una nicchia ancora più piccola di chi parla la lingua del regolamento. Certo, con cinque parole si spiegano le quaranta che servirebbero, e in un 200 Word RPG ci sta: in classe, chi l'ha visto ha capito; ma chi non l'ha visto non ha la minima idea di che cosa si parli;
- il riavvolgimento è "un po' bizzarro": *chi* decide che la storia stalla, e *quando*? Che cosa vuol dire che la vita si riavvolge: si rigiocano scene già giocate, si ricomincia la partita? Chi diventa persona non vedente e guida, e come ci si scambia?
- la **consistenza terminologica**: prima si parla dei *remaining players*, poi dei *performers*, e probabilmente sono le stesse persone, chiamate in due modi diversi. "Qui è piccola, ma in un regolamento più

[13] Le età sono ricostruite: la trascrizione legge "0–35", poi "37–50", poi "37–70", poi "70 anni in poi".
[14] La trascrizione dice "G-Phorm".

grande occhio a questa cosa": bisogna identificare le **parole meccaniche** che servono e **mantenerle per tutto il regolamento**, perché sono un riferimento. Altrimenti chi legge si chiede: *chi sono questi? Sono quelli che all'inizio facevano A, B, C?*

Le emozioni. Per Giordano, da un lato **intimità**, per l'idea di condividere una storia, ma anche molta **agency** alla guida, che improvvisando la storia può impostarla verso l'impotenza o verso il suo contrario. Qualcuno dice **nostalgia**: ripercorrere le tappe della vita, i ricordi, la famiglia. Il docente aggiunge una cosa: il gioco rientra nella famiglia dei **freeform**, e con un freeform **creare emozioni focalizzate è molto difficile**, perché i freeform lasciano tanto spazio a chi gioca, nel bene e nel male. La vita di questa persona può cambiare drasticamente a seconda del gruppo: un gruppo la gioca come una vita idilliaca, in cui la cecità non è un problema; un altro come una vita tragica, piena di stenti e sofferenza. Lo stesso gioco dà emozioni molto diverse. *Che cosa accomuna le due giocate?* È in quell'aspetto meccanico che si vede che cosa il gioco fa. "Se dovessi identificare il tuo cuore meccanico, Lorenzo, dove lo metteresti?" Lorenzo voleva lavorare su tutte le emozioni legate all'**improvvisazione** (che però, osserva il docente, è implicita nel LARP, come la creatività), e sull'**introspezione**: la lettura del comportamento dell'altra persona da parte di chi sta mettendo in scena gli eventi, che cerca di capire perché la persona non vedente reagisce così alle cose che le mette davanti.

Il rischio del tema aperto. Il punto del docente è di stare molto attenti al **piano tematico**, perché può sfuggire di mano, "e questo è tanto più grave nell'ambito formativo". Supponiamo di voler fare un gioco formativo per mostrare che una persona cieca può avere una vita assolutamente dignitosa, legittima, piena di cose belle. (Qualcuno ha fatto a Milano il percorso al buio[15]: "hai già capito cosa intendo".) Se quello è il messaggio, e un gruppo gioca una vita di stenti e sofferenza, il messaggio che arriva diventa "era meglio se questa persona non fosse nemmeno nata": un rischio che non si vuole mettere in un gioco. Quando si apre tematicamente, è molto difficile avere il controllo, **anche in termini di sicurezza** di chi gioca. Detto questo, Lorenzo aveva ben chiaro che cosa voleva fare: a parte i dubbi, il gioco c'è.

2.6.3.9 Rita: il tabellone dei futuri possibili

Il testo. Rita segue in differita, quindi non si interagisce; il docente legge comunque e chiede alla classe. **200 parole** tonde ("non ho parole"). Il **tabellone dei futuri possibili** viene realizzato da giovani attivisti di una sezione di quartiere che si radunano nel cortile della scuola. Ha **cento sportelli** colorati e sigillati; di ognuno, oltre al colore, si legge solo il titolo ("l'invenzione che pulisce i fiumi", "la città senza auto", "il segreto dei centenari"). I ragazzi devono collaborare e, per sbloccare i sigilli, risolvere insieme piccoli **enigmi** legati a storie contemporanee di successo: progetti ecologici innovativi, scoperte scientifiche, start-up guidate da giovani. Dietro ogni sportello ci sono veri e propri indizi: campioni di nuovi materiali biodegradabili, QR code o video su imprese, messaggi di coetanei dall'altra parte del pianeta. Seguono frasi che descrivono l'atmosfera (risate, schiamazzi, serenità, sorpresa, scambi entusiasti), i giocatori che si muovono in modo frenetico, attratti dai colori e dalla voglia di scoprire, "nell'aria il desiderio di sperare, di farsi stupire da ciò che l'umanità sta costruendo per il domani", e una domanda finale: che cosa può nascere dalla condivisione?

Un *pitch*, non un regolamento. "Rita, qui mi sembra più un **pitch** di un gioco". Il primo paragrafo può starci, quasi come un *setup*: descrive delle procedure, serve un tabellone con degli sportelli, dice che cosa scriverci sopra, lasciando la libertà di riempirne centinaia. Dopo il primo paragrafo, però, il testo **smette di essere un regolamento**: descrive *come ci si immagina che il gioco sia*, non *come deve essere giocato*. Data a una persona che non ha in testa quello che ha in testa l'autrice, questa pagina non permette di giocare: al massimo di provare a ricostruire le sue aspettative. A livello meccanico, qual è l'**azione di chi gioca**? Aprire gli sportelli o prepararli? Chi mette le cose dentro il tabellone: i giovani attivisti? E allora il regolamento è **per chi**: per gli attivisti o per chi gioca? I regolamenti di solito sono per chi gioca, non per chi organizza; esistono regolamenti **per chi facilita**, ma in quel caso va scritto all'inizio, come in quello mostrato la volta precedente[16]. Il primo paragrafo, al limite, potrebbe essere parte della preparazione di un

[15] La trascrizione è confusa ("l'avevo fatto a Pacecchia, vivaio"); con ogni probabilità si tratta di *Dialogo nel buio*, il percorso esperienziale dell'Istituto dei Ciechi di Milano.

[16] La trascrizione dice "di Creatorsphere"; il riferimento non è ricostruibile con sicurezza.

gioco pervasivo, visto che c'è un ambiente fisico (un cortile di scuola) e un artefatto (il tabellone); poi il regolamento non c'è più.

Non si scrive come si sentiranno le persone. L'ultima parte è "forse la più perniciosa": descrive **come le persone si sentono a giocarlo**. "Questo è impossibile": nessun regolamento dirà come ci si deve sentire, o come si sentiranno le persone. È ***wishful thinking***. Non è una buona pratica scrivere nei regolamenti che cosa si vuole che le persone provino, perché non si ha il controllo sulle loro emozioni: lo si ha sulle regole, non su un "dovete essere felici, sorridenti, pieni di sorprese". Quel blocco può andare bene come pitch, come documento di *brainstorming* su dove si vuole arrivare ("io voglio arrivare a un gioco che fa questa cosa"), ma **non può essere un regolamento**.

Il docente è visibilmente dispiaciuto che Rita non ci sia: "questo era forse il caso più importante da affrontare in classe", e non vuole affrontarlo senza di lei. La invita a scrivergli per email o su Telegram, "quando hai tempo, se ne hai voglia", per parlarne nel dettaglio; alla classe chiede soltanto se il punto è passato. Una studentessa chiede conferma: oltre al fatto di non poter dire nel regolamento che cosa si deve provare, ci sono anche cose citate che non si possono usare? Sì, è quello che diceva: è il pitch di qualcosa a cui si vorrebbe arrivare, "ma non è un regolamento".

Finiti i regolamenti, cinque minuti di pausa: si riprende alle 18:25, con le slide di Beatrice e del docente.

2.6.4 Che cosa insegnano i nove giochi

La tabella 2.6.1 riassume la revisione. Prima, le osservazioni che tornano in più giochi e che valgono ben oltre il formato delle 200 parole:

- **Le regole implicite** vanno bene finché chi legge le ricostruisce senza dubbi (chi pesca picche è il medico; il valore delle carte non conta); diventano un problema quando lasciano aperta una scelta (i pareggi di *Out There*, il bonus contestato di *Yamanobe no Michi*).
- **Il *player count*** va dichiarato, e va verificato che il gioco regga al minimo e al massimo.
- **L'*agency***: attenzione alle meccaniche che la tolgono, a chi resta in partita senza mai decidere nulla, al dado che vince al posto di chi gioca.
- **Il buon senso** non è una regola: "sei tu il designer". Lasciare al tavolo le scelte strutturali ("fate un po' come volete") va bene solo se è una scelta tematica dichiarata.
- **L'emozione non si scrive**: né nominando un valore (*successo*) né descrivendo come si sentiranno le persone. La danno le meccaniche.
- **I temi delicati** chiedono content warning e meccaniche di sicurezza; nei giochi più grandi, consulenti. Nei freeform aperti il tema può sfuggire di mano, e con lui il messaggio e la sicurezza.
- **La terminologia** va fissata e mantenuta; i riferimenti culturali locali (Papà Castoro) escludono chi non li conosce; un cambio di genere va annunciato.
- **Un regolamento non è un pitch**: dice come si gioca, a chi gioca (o, se è per chi facilita, lo dichiara).

Approfondimento: il regolamento come unica voce del designer. Il metodo del docente, leggere "come chi l'ha appena scaricato", corrisponde a un'idea che torna in molti libri di game design: chi progetta **non è al tavolo**, e ciò che non è scritto non esiste. Schell, in *The Art of Game Design*, insiste che il designer non crea direttamente l'esperienza, ma solo il gioco, cioè le regole e gli oggetti da cui l'esperienza nasce nella mente di chi gioca; la revisione dei nove giochi ne è una dimostrazione pratica. Le stesse regole hanno prodotto nella classe emozioni diverse da quelle volute, e il modo per avvicinarle non è descriverle nel testo, ma restringere l'area con le meccaniche e poi verificarla con il playtest.

Gioco (autore)	**Parole**	**Emozioni viste dalla classe**	**Emozione voluta**	**Osservazioni principali**
Parlatene (Tiziana)	199	ansia, impotenza, condivisione del problema	frustrazione, sconforto	regole implicite chiare; "puoi non giocare" come sicurezza forte; debriefing nel titolo
Yamanobe no Michi (Giordano)	200	creatività e giudizio, serenità	serenità, creatività	bonus contestabile poco definito; testo iniziale messo in fondo; competizione rovesciata
Beniamino va all'università (Gianmarco)	199	speranza, incertezza, disperazione	essere *out of control*	coeso; l'emozione dalle meccaniche, non dai nomi dei valori; "pachinko"
Finché resta cenere (Nicola)	189	insicurezza, sollievo, rimorso, rabbia	sfiducia, esitazione	dilemma del prigioniero; comunità contro individuo; grassetto da evitare
Out There (l'altro Lorenzo)	141	esclusione, risentimento; leader isolato o potente	inadeguatezza, impotenza; giudizio, controllo	pareggi, player count, nessuna soglia; agency assente per chi sta nel mezzo
Qualcosa di più grande (Tancredi)	200	vendetta, paura, ansia	gratitudine	asimmetrico; "fate come volete" non è design; ruoli o personaggi?
Una brutta serata (Angelo)	147	disperazione, riflessione	impotenza	tema delicatissimo senza content warning né sicurezza; chamber LARP molto coeso
What Remains of the Stories Behind Us (Lorenzo Vergata)	190	intimità, agency della guida, nostalgia	improvvisazione, introspezione	cambio di genere improvviso; Papà Castoro; terminologia incoerente; tema aperto
Il tabellone dei futuri possibili (Rita)	200	—	—	un pitch, non un regolamento; nessuna azione di gioco; emozioni descritte

Table 2.6.1: I nove giochi in 200 parole del secondo task, nell'ordine della revisione. Rita era assente e la classe non si è espressa sull'emozione del suo testo.

2.6.5 Perché si fa ricerca

Alla ripresa il docente condivide le slide e lascia la parola a Beatrice, che ha seguito la revisione "in background" e si complimenta con la classe. Dopo un controllo tecnico (da tutta la lezione Zoom le mostra un banner di errore, ma la si vede e la si sente), l'argomento è annunciato: **ricerca e sviluppo**, in varie forme, con delle anticipazioni di ciò che verrà. Due premesse.

La prima: tutto quello che si vedrà "si può esplorare molto di più": sono **inizi di riflessione**. Ciò che Beatrice vorrebbe che la classe si portasse a casa è che il lavoro di design è **una costante ricerca e sviluppo**, una costante **iterazione**. I punti di vista e gli strumenti presentati sono usati in varie forme, ma **non tutti insieme** e **non per tutti i progetti**: la ricerca e sviluppo dipende molto dal tipo di progetto. Per questo era interessante la domanda dell'inizio ("ho più di un'idea interessante, come scelgo?"): dipende dal **contesto** in cui si sta producendo quell'idea, e se ne parlerà nelle prossime lezioni.

La seconda: la ricerca ha **due livelli**. Da un lato è legata al progetto specifico; dall'altro è un lavoro costante, "come se faceste non solo game design ma botanica": ci sono momenti in cui si approfondisce lo studio specifico che si sta facendo, ma in generale ci si tiene aggiornati sulle pubblicazioni del settore e sui dialoghi intorno alla disciplina.

Le risposte della classe. "Perché, parlando di progettare un gioco, parliamo anche di fare ricerca?" La palla passa alla classe.

- Lorenzo: per vedere che cosa hanno fatto gli altri, come **seme** per poter poi fiorire e proseguire; per lo scambio di idee; per esplorare cose a cui non si stava pensando ("ah, posso prendere questa cosa da quell'ambito e metterla nella mia meccanica, farla variare un po'"); o, al contrario, per capire che cosa cambiare di una meccanica esistente per essere più innovativi, o per farla propria e diventare esperti di quel gioco specifico.

- Tiziana: per **rispondere a delle domande**. "Io ho una domanda e ho necessità di trovare una risposta". Bravissima: magari si ha in mente un'esperienza da far vivere ma manca qualche pezzo, e cercare risponde alle domande che la progettazione genera.

- Gianmarco, facendo "un inviluppo" delle due risposte: per lui la ricerca è collegata all'**innovazione** ma non al prodotto: si fa per poter implementare qualcosa di mai visto, slegata dall'ambito finanziario, dal ritorno economico. Anche questo è un tema, risponde Beatrice: si fanno giochi per tanti motivi, c'è una parte economica se la si vuole, ma ci sono i giochi formativi e tantissimi altri motivi.

- Giordano: al contrario, si può fare ricerca proprio per **posizionarsi sul mercato**. Se si ha l'idea di un gioco di *draft* medievale e si scopre che negli ultimi anni ne sono usciti cinquanta, forse è meglio cambiare le meccaniche o la tematica. Ottimo tema: conoscere lo **stato dell'arte del mercato** può dirigere la progettazione.

I motivi. Beatrice riprende le risposte. Conoscere lo **stato dell'arte** aiuta a **prendere decisioni**: anche la domanda dell'inizio ("come faccio a sapere se questo gioco esiste già, o se esiste qualcosa di simile?") va in questa direzione, come l'osservazione di Giordano. "Mercato" è detto per estensione: i **giochi formativi** spesso non si basano sulla compravendita ma su altre fonti di sostentamento (bandi, sponsor, varie possibilità) e seguono altre dinamiche; ma anche lì è importante sapere in quale stato dell'arte ci si inserisce con la propria proposta. Poi la ricerca aiuta a **validare** idee e a **trovare soluzioni**: risponde a domande di ogni tipo, su che cosa ha offerto il mercato ma anche su che cosa si vuole raggiungere e come. Se manca una meccanica, o se si vuole capire come esperienze simili sono state comunicate lato *art*, immagini, *copy*, testi, la ricerca dà idee già in parte testate o validate. E soprattutto, per Beatrice, la ricerca e lo sviluppo **mantengono la mente elastica** sul contesto: che si progetti con uno scopo economico o semplicemente per esprimersi, parlare di qualcosa che ci interessa, sensibilizzare su un tema, la ricerca permette di capire **come le altre persone sono abituate** a ricevere certe tematiche. Infine l'**innovazione**, "una parola importante", su cui ci si dovrebbe soffermare di più: **l'innovazione ha bisogno di essere comunicata**. Fare ricerca serve anche a capire come temi complessi siano stati comunicati e fatti esperire in altri momenti e situazioni: è insieme una forma di **paragone** e un modo per capire **come discostarsi** da ciò che esiste già.

2.6.6 Per che cosa si fa ricerca

Dopo il *perché*, il *per che cosa*. La slide mostra una freccia con le fasi della creazione di un gioco, e per ciascuna un "motore" diverso della ricerca[17] (figura 2.6.2).

Concept: l'ispirazione. Il primo motore è l'**ispirazione**, che è anche un po' il nostro modo di stare al mondo: vedere che cosa fanno le altre persone, che cosa il mondo ci propone. E, "badate bene", l'ispirazione **non viene per forza da altri giochi**: un quadro che ci ha colpito, una conversazione, una persona conosciuta, un libro. È una delle basi della creatività: una cosa che si connette all'altra.

Pre-produzione: cristallizzare la visione. Nella pre-produzione, cioè la creazione di un primo prototipo, la ricerca aiuta a **cristallizzare la visione**. L'esempio: vogliamo fare un'esperienza di gioco che racconti che cosa significa **passare una notte da soli in campeggio nel bosco**. Abbiamo trovato delle dinamiche che ci danno la sensazione di poterla raccontare; andiamo a vedere quali esperienze simili possono darci tasselli in più. Nel mondo del gioco: esistono giochi di ruolo, videogiochi, giochi da tavolo che raccontano una notte da soli nel bosco? Li si prova, si capisce che esperienza si vive, si guardano altre persone giocarli. Ma anche fuori: film, canzoni che danno quella sensazione, o perfino passare una parte di notte nel bosco.

Mix e remix. Una fase su cui si tornerà subito dopo, con Ruggero.

Sviluppo, validazione, mercato. Lo sviluppo, la **validazione**, il *decision making* e il mercato e la vendita: qui si torna a parlare di come **testare** l'idea e il prototipo. Sul **playtest** ci sarà una lezione apposita, e lì sarà chiaro che cosa significhi fare ricerca dalla validazione in poi.

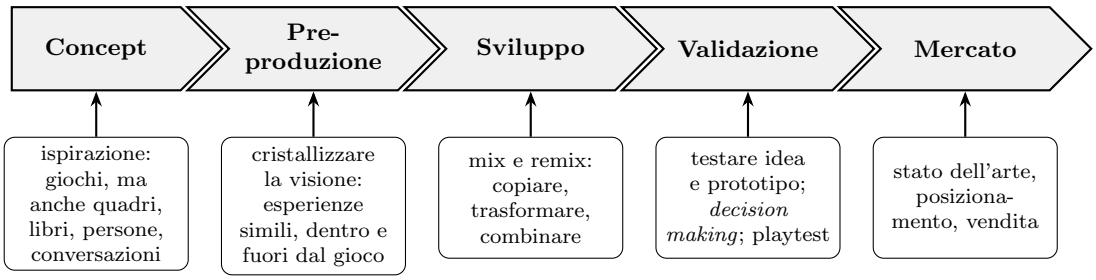


Figure 2.6.2: Per che cosa si fa ricerca: per ogni fase della creazione di un gioco, un motore diverso. Ricostruzione schematica della slide di Beatrice; la collocazione del "mix e remix" accanto allo sviluppo è un'ipotesi.

2.6.7 Divergenza e convergenza

Poi due concetti "spolverati", da approfondire nella lezione successiva: le fasi **divergente** e **convergente** della creazione. Qualcuno le ha già sentite, anche fuori dai giochi? "Forse nel *design thinking*", dice Tancredi. "Bravissimo". Per chi le sente per la prima volta, il messaggio è questo: non è una regola, ci sono tanti modi di creare e progettare, ma in linea generale l'atto di creazione, **soprattutto quando avviene in gruppo** (come succederà nei prossimi incontri), ha due fasi, o almeno gli si dà un ordine in due fasi:

La fase divergente: si prendono **tutte le idee** che vengono in mente e si mettono in una sorta di **calderone**.

La fase convergente: le idee raccolte vengono **pulite e valutate**, scartate o fatte proseguire: una **selezione in base agli obiettivi** di progetto.

[17] Questo passaggio della trascrizione è in parte rovinato: per alcune righe si ripete in loop "gli altri sono invece una specie di routine". La ricostruzione della figura 2.6.2 segue le parti leggibili: concept, pre-produzione, "mix e remix", sviluppo, validazione e *decision making*, mercato e vendita.

Le due fasi si ripetono nello schema a **doppio diamante** (*Double Diamond*), che si vedrà nel dettaglio la prossima volta.

Ciò che interessa oggi è un'altra cosa, e risponde alla domanda di Lorenzo dell'inizio ("ho tre idee che mi sembrano tutte valide, come scelgo?"): **è normale**. Ci sono momenti in cui si hanno tantissime idee, su un gioco intero o su una singola meccanica, ed è normale che sia così. Più ci si allena nella progettazione, più si capisce **quali idee scartare e quali tenere in base all'obiettivo**. "Non spaventiamoci se abbiamo tante idee e siamo un po' in confusione": è la fase divergente. Poi ci si allena a stare nella fase convergente, a scegliere le idee che funzionano meglio, a testarle, a iterarle, e a continuare "questo balletto" finché non si ha un **prototipo funzionante**, che avrà poi le sue fasi successive (figura 2.6.3).

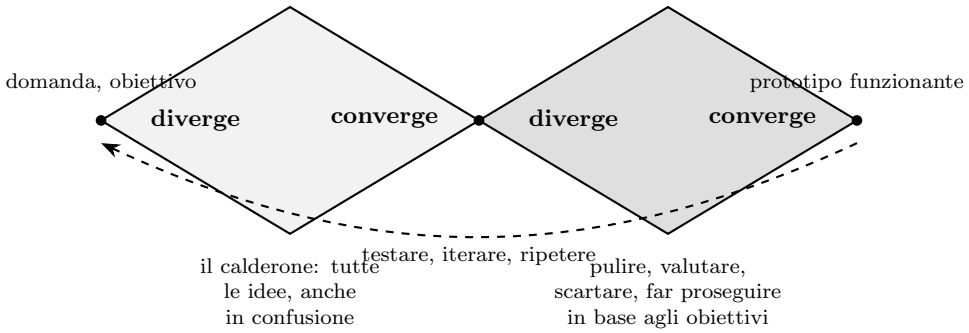


Figure 2.6.3: Divergenza e convergenza nello schema a doppio diamante, come lo introduce Beatrice. L'alternanza si ripete finché non si arriva a un prototipo funzionante; il dettaglio delle fasi è rimandato alla lezione successiva e alla dispensa.

La dispensa di Daniele. Ruggero aggiunge un annuncio: molte cose dette da Beatrice si vedranno la volta successiva, ma intanto Daniele metterà nella cartella del Drive una **dispensa** con uno *zoom* sugli strumenti, perché per motivi di tempo non era possibile elencare a lezione tutti gli strumenti del *brainstorming* e del *design thinking*. Con un avvertimento, lo stesso di Beatrice: **non sono strumenti da imparare tutti**. Aziende e realtà diverse hanno strumentazioni interne diverse, al punto che uno strumento utilissimo in un'azienda in un'altra non lo conoscono nemmeno. Il consiglio è di dare un'occhiata a tutti e capire quali servono, o quali **risuonano**. Sul *brainstorming* in particolare "c'è una valanga" di materiale: ci si potrebbe fare una lezione, anzi un corso intero, ma non è il focus di questo. Non ci saranno interrogazioni né task sul *brainstorming*; gli strumenti saranno però utili nel progetto più grosso.

2.6.8 Che cos'è la creatività: *Everything is a Remix*

"E qui arriviamo a una domanda chiave, cuore del corso, che abbiamo agilmente schivato fino ad ora": **che cos'è la creatività?** "So che sembra banale". Le risposte:

- qualcuno: **trovare soluzioni alternative a un problema noto**;
- Tiziana parte dalla parola: l'attività di essere creativi, di **creare**. Nella sua definizione, nota il docente, non c'è un problema da risolvere;
- Lorenzo: **dare forma a un'idea**, a un pensiero, a qualcosa che prima è astratto, renderlo "tra virgolette" **tangibile**. Molto interessante, perché mette già implicitamente una fase di **esecuzione**, mentre trovare una soluzione a un problema non è necessariamente pragmatico.

Di definizioni ce ne sono tante, e il docente non vuole darne una; vuole però **evitare** che la classe finisca in un'area precisa di definizioni: quella che vede la creatività come una **cosa mistica** che scende dall'alto e ci illumina quando siamo sotto la doccia, o mentre sogniamo di notte. La creatività va vista come qualcosa di **pragmatico**, come dinamiche che accadono e che si possono identificare. Non è il primo corso a farlo:

molto materiale, in video, video saggi, documentari e framework specifici, l'ha già affrontata così. La slide si intitola ***Everything is a Remix***[18], e la creatività è fatta di **tre elementi**: **copiare, trasformare, combinare**. Qualunque atto creativo ne ha almeno uno, "sempre"; più è complesso, più ne ha. "Sarà molto difficile trovare un atto creativo che non ne includa nessuno".

Copiare. Prendo qualcosa che mi piace, o che voglio ricreare, e lo ricreo. L'esempio: *Age of Steam* e *Steam*, un altro gioco che prende *Age of Steam* e lo rifà. Si faceva ben prima dei giochi: nella pittura, nella scultura, nella musica, nel cinema, nella letteratura.

Trasformare. Prendo una cosa e ne cambio un pezzo. Prendo *Magic: The Gathering* e la creazione del mazzo, invece di metterla prima del gioco, la metto **in mezzo al gioco**: arrivo a *Dominion*, ai giochi di **deck building**. Non è solo copiare: si cambia qualcosa per arrivare a qualcos'altro. Anche qui gli esempi sono mille, in tutti i media.

Combinare. Prendo due cose, "le schiaccio insieme e vedo che cosa esce". *Scrabble* (Scarabeo), gioco di parole, più *Dominion*, deck building: *Paperback*, un deck building in cui si formano parole. Si comincia con un mazzo di lettere che durante il gioco cresce e si modifica in base alle scelte.

Le tre dinamiche **non sono mutuamente esclusive**: nei giochi complessi avvengono più volte, su più aspetti, in molteplici forme ("magari copio tre cose, ne trasformo altre quattro e ne combino altre due"). Qualunque gioco che si abbia in mente è figlio di queste operazioni, anche in modo **crossmediale**: vedo un film e voglio copiarne le *vibes* nel gioco; prendo due generi di videogiochi, li combino e ne faccio un gioco da tavolo. "Questa cosa avverrà sempre. Il mio consiglio è: **siatene consapevoli**". Cercare di capire quando sta succedendo, che cosa si sta copiando, trasformando, combinando.

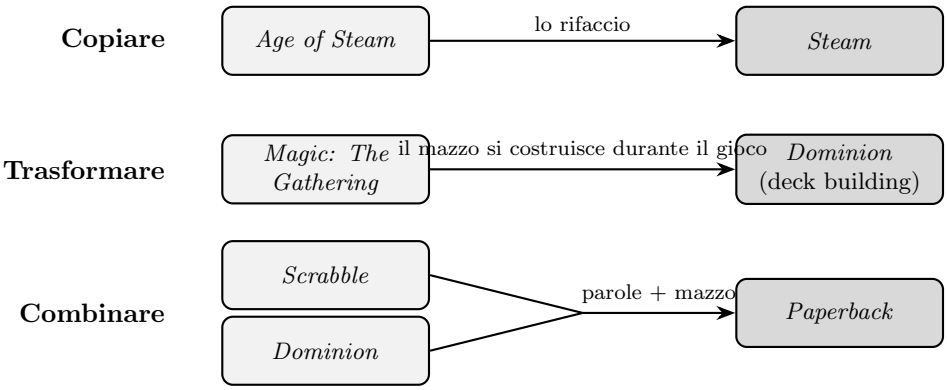


Figure 2.6.4: Le tre operazioni della creatività secondo *Everything is a Remix*, con gli esempi della lezione. Non sono mutuamente esclusive: nei giochi complessi si sovrappongono più volte.

2.6.9 Il plagio e le ispirazioni dichiarate

Essere consapevoli serve anche a **non commettere plagio**, "che può essere un problema, specialmente nel mondo moderno in cui il copyright è una materia molto complessa". Nel gioco, "per mille questioni vestigiali", **le meccaniche non sono tutelabili**; lo sono invece molte altre cose, a volte bizzarre e apparentemente poco legate al gioco: elementi grafici e di presentazione[19], **loghi**, **nomi**. "Le meccaniche no. Altrimenti il deck building sarebbe morto con *Dominion*, se non prima". La ricerca e sviluppo è anche capire come navigare questo mare di copiare, trasformare e combinare, che è inevitabile, "senza finire in gattabuia". Nel gioco si ha la vita più facile che in altri settori: in un corso di musica o di letteratura questa sezione sarebbe

[18] È il titolo della serie di video saggi di Kirby Ferguson, che riassume la creatività nelle tre operazioni di *copy*, *transform* e *combine*. A lezione non si nomina l'autore.

[19] La trascrizione dice "elementi di Wix"; il senso è quello dell'aspetto visivo e del confezionamento (il cosiddetto *trade dress*), insieme a loghi e nomi.

molto diversa. Il che non vuol dire "andate su BoardGameGeek, copiate tutto a manetta e state a posto": ci sono svariate cause legali in corso anche per queste questioni.

Il caso *Bang!*. La più famosa, forse, in Italia. Un editore cinese aveva preso *Bang!*, l'aveva **ritematizzato** in ambito di Asia feudale e l'aveva venduto senza pagare un centesimo di *royalties* né all'editore originale, dV Giochi[20], né al designer, Emiliano Sciarra[21]. La difesa era stata: sì, ma *Bang!* è Far West e questo è a tema Asia feudale, quindi non è lo stesso gioco, "perché le meccaniche non erano tutelabili". Come si sono tutelati? Hanno fatto ***Samurai Sword***[22], cioè *Bang!* ritematizzato a tema samurai, così da poter dire: "vedete, questo ha un elemento in più, e adesso è copiare". Il tema di brevetti, marchi e copyright è un altro vaso di Pandora; il docente è sicuro che la classe ci farà attenzione, e che abbia già la consapevolezza di non fare un gioco su Topolino senza aver parlato con la Disney, o su Super Mario senza Nintendo: "e se non lo sapete, ve lo farà sapere Disney o Nintendo, che sono molto veloci a farvelo sapere".

La regola generale (Beatrice). Beatrice aggiunge che la normativa italiana sul plagio di un gioco da tavolo, o di un gioco in generale, è **molto sfumata**. C'è però una regola generale: **fate qualcosa che sentite vostro**, attingendo a quello che vedete, e **non usate proprietà intellettuali altrui** (niente Harry Potter, niente Topolino). Non vuol dire non ricombinare: ci si accorge da soli quando si rischia il plagio. Non c'è un discrimine chiaro nella norma, ma "è sfumato quando lo è, e molto chiaro quando non lo è: lo sapete nel vostro cuoricino".

Dichiarate le ispirazioni (Ruggero). Ruggero aggiunge una cosa "magari scontata", da riprendere nella lezione sul regolamento e sul paragame: **rendete palesi le vostre ispirazioni**. Nel gioco di ruolo è molto comune trovare in fondo al manuale le ispirazioni (serie TV, musiche, libri, altri giochi), e lo si fa anche in giochi grossi come *Blades in the Dark*[23]. "Fatelo anche voi", e non solo per i docenti, ma per chi compra: "io conosco persone che guardano se ci sono le ispirazioni, e se non ci sono non lo comprano". Dichiararle non toglie niente all'atto creativo: "anzi, è una buona prassi".

Approfondimento: niente nasce dal nulla. L'idea che la creatività sia ricombinazione è condivisa da entrambi i libri di riferimento. Koster, in *A Theory of Fun for Game Design*, osserva che molti giochi nuovi sono variazioni di giochi precedenti e che i generi si formano proprio per accumulo di piccole variazioni; Schell dedica ampio spazio alle tecniche per generare idee e insiste sull'abitudine di attingere a tutto ciò che si conosce, dentro e fuori dal gioco. Nella lezione questa idea diventa un'indicazione pratica a tre facce: essere consapevoli di che cosa si copia, trasforma e combina; non usare proprietà intellettuali altrui; dichiarare apertamente le proprie fonti.

2.6.10 Dall'idea al gioco validato

Si passa ai **concetti e strumenti di ricerca preliminare**. La prima slide mostra gli **step per validare un'idea**. Per ora basta sapere che esistono: nei prossimi incontri si vedranno nel dettaglio, anche per immaginare che cosa fare in ciascuno, così che nei lavori di gruppo ci sia già un minimo di coordinamento (figura 2.6.5).

Prima del goal: l'obiettivo personale. Il primo step è la **definizione del goal**, l'obiettivo del gioco. Ma a monte, secondo Beatrice ("i colleghi in caso mi correggeranno"), c'è una riflessione ancora precedente. L'obiettivo di un gioco può essere anche leggerissimo: "io sono grande fan anche dei giochi senza un obiettivo così profondo. Voglio fare un gioco con dei **pinguini blu**, ho proprio voglia di avere i pinguini blu". Prima di questo, però, bisogna chiedersi **personalmente perché si fa design in quel momento**. Beatrice lavora come **freelance**: non sempre con la stessa casa editrice o la stessa entità, ma con vari clienti, e per

[20] La trascrizione dice "DV".
[21] La trascrizione dice "Emiliano Sharra". Il docente non ricorda il nome del gioco cinese; con ogni probabilità è *Legends of the Three Kingdoms* (*Sanguosha*), ambientato nella Cina dei Tre Regni.
[22] La trascrizione dice "Samurai's World".
[23] La trascrizione dice "Blaze in the Dark".

ogni cliente ha obiettivi diversi (un gioco per formare, e tanto altro); poi fa giochi "per me", che vende direttamente al pubblico occupandosi di marketing e pubblicità; e giochi "per me" nel senso di espressione personale. Gli esempi:

- voglio fare un gioco **per esprimermi**, per raccontare la mia storia familiare con una breve avventura interattiva online;
- voglio fare un gioco per un **committente** che vuole vendere più caffè: l'obiettivo del gioco sarà vendere più caffè, ma l'obiettivo di Beatrice sarà farsi pagare rispondendo ai bisogni del committente;
- voglio creare un gioco che venda **almeno mille copie**: allora serviranno indagini di mercato; l'obiettivo di Beatrice sarà un gioco efficace ed efficiente in termini economici e di vendita, quello della designer trovare il target giusto e la modalità giusta;
- voglio essere **pubblicata da un editore**: l'obiettivo personale è la pubblicazione, quello del gioco rispondere a qualcosa che l'editore troverà interessante.

Se non ci si fa prima la domanda sull'obiettivo personale, rischia di non essere chiaro nemmeno l'obiettivo del gioco; e in progettazione, "siccome basta un filo di vento per perdere la direzione", sarà facilissimo perderla. È un consiglio che Beatrice dice di non aver sentito molto nella formazione sul game design, ma che ritiene molto importante: **partite chiedendovi che cosa cercate voi**. La risposta cambierà con i contesti; e non vuol dire che sia poi quello che si raggiunge, "perché il processo di creazione porta in tante strade", ma serve per un **dialogo condiviso** sugli obiettivi e per un'**autoconsapevolezza** sempre maggiore.

L'esempio del corso. Nel corso si dovrà creare un gioco **in gruppi** (Beatrice chiede conferma, e vede annuire). L'obiettivo del gioco lo sceglierà il gruppo; ma prima ci sarà l'obiettivo personale di ognuno, che Beatrice ha inserito negli **esercizi della prossima volta**, per esplicitarlo e condividerlo. Per esempio, in un gruppo con Beatrice, Tiziana e Tancredi, una persona potrebbe essere lì per **esplorare**, senza interessi ulteriori; un'altra vorrebbe **sperimentare sui giochi formativi**; un'altra vorrebbe "sporcarsi le mani" con l'**art**, le illustrazioni, uno strumento nuovo. "Parlarne vi darà già una marcia in più".

L'obiettivo del gioco. Definito questo, si arriva all'obiettivo del progetto. Ci sono varie modalità per definirlo; per oggi conta che **un obiettivo condiviso dal team deve esserci**. Spesso sono più obiettivi, e spesso, "ahimè", non sono chiari: soprattutto con aziende ed editori ci sono tanti dialoghi "un po' fumosi". Capiterà che in alcuni progetti la direzione non sia chiara, "ve lo dico fuori dai denti"; nel corso si cercherà di dare gli strumenti per averla chiara come gruppo. Gli esempi di obiettivo: un gioco con i pinguini blu; un gioco che sensibilizzi sullo **smaltimento della plastica**; un gioco che replichi il mondo fantasy "ma in chiave **calzini spaiati**".

Ipotesi, prototipo, validazione. Definito l'obiettivo, si sviluppa un'**ipotesi**: *come* voglio arrivarci? Quali riflessioni servono, e quali criteri ci diranno che l'obiettivo è raggiunto? In pratica, con i tempi del corso, già in questa fase si buttano sul piatto **idee sparse**: è *brainstorming*, con un piede già nello sviluppo; tante idee, poi scremate con un processo di design. Dalle ipotesi si passa al **prototipo**, il lato di sperimentazione: si mischiano le idee e si vede che cosa salta fuori (anche qui ci sono modalità, e la prossima volta la classe sarà guidata). Quando c'è un primo prototipo che più o meno sta in piedi, con un **percorso dell'utente** chiaro (che cosa fanno le persone giocanti dall'inizio alla fine), lo si **valida**, cioè lo si testa: inizia "una delle fasi più importanti e interessanti del game design", il **playtest**. Si vede che cosa funziona e che cosa no, si aggiusta il tiro, "e facciamo questa cosa tutte le volte che possiamo". Domande? Pollici in su ("potete sbizzarrirvi anche con le emoji"; Gianmarco manda un pinguino, "adoro").

Una linea guida, non un binario. Beatrice mette subito le mani avanti: nel corso verrà data una struttura ("fase 1 fate questo, fase 2 fate quest'altro"), "e se fosse come ve la racconteremo, sarebbe una figata". Nella realtà, che si lavori in gruppo, per qualcuno o da soli come pura ricerca esplorativa, **"ci sarà un gran casino"**: di idee, di scelte, momenti di panico, obiettivi che cambiano a progetto quasi finito ("ma perché non cambiamo obiettivo?"), e magari il gruppo decide di farlo. "Ci sta". Quella che si dà è una **linea guida** a cui tornare quando ci si perde, e in cui stare quando ancora non si sa quale linea seguire.

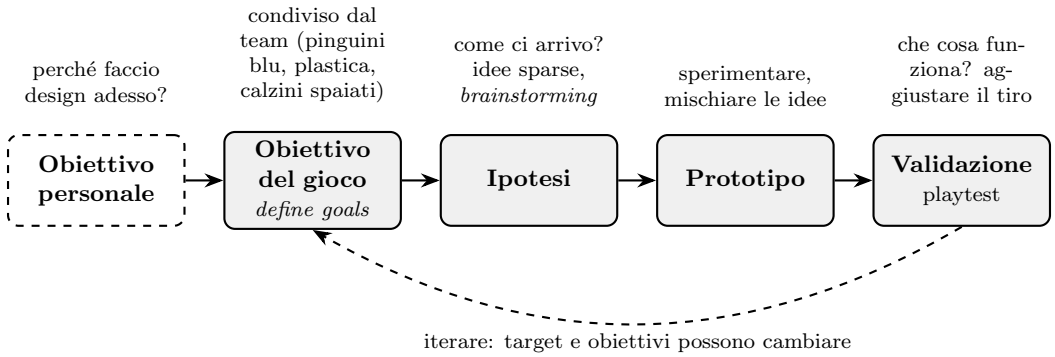


Figure 2.6.5: Gli step per validare un'idea, con il passo "zero" che Beatrice antepone all'obiettivo del gioco: l'obiettivo personale di chi progetta. Il ciclo si ripete "tutte le volte che possiamo".

2.6.11 A chi sto parlando: il target

Come si definisce l'obiettivo? Beatrice è "grande fan delle domande, perché ci aiutano a dare una direzione", e la slide ne mette alcune che, all'inizio della creazione di un gioco, sono **più o meno tutte prioritarie allo stesso modo**: ce le si butta davanti tutte per iniziare a ragionare.

- **A chi sto parlando?**
- È una mia **ricerca artistica** personale? Un mio modo di **esprimermi**?
- Voglio **venderlo**? Voglio **pubblicarlo**? Voglio trovare un **editore**?
- Che **esperienza** do alle persone che lo giocano?
- A che cosa sono **abituate** queste persone?

Sono domande da farsi a monte, prima ancora di dire se l'obiettivo sono i pinguini blu o no: "non ci arriviamo neanche lì". Nel momento in cui si definisce un obiettivo, ci si è già fatti qualcuna di queste domande, o ce la si sta per fare.

Il target viene prima. Beatrice userà la parola **target**, pur non trovandola adeguata: è solo una parola per darsi una direzione. Qualunque strumento di dialogo (un libro, un film, un gioco) si può pensare perché qualcun altro lo esperisca oppure no; ma quest'ultimo è un tipo di sperimentazione in cui non si entra. "Il 99,9% dei giochi che produciamo sono pensati per essere giocati da qualcun altro", e di quel qualcun altro bisogna tenere molto in mente la presenza, già in fase di immaginazione e di definizione dell'obiettivo: a chi metteremo in mano il gioco, a chi diremo "ehi, ciao, vuoi provare questa esperienza? Tieni, provala". All'inizio il target si definisce come **macro categoria**. Un gioco per **bambini di 7 anni**, da giocare a casa la sera, è diversissimo da un gioco per **banchieri dai 40 anni in su in pausa pranzo**: diverse le competenze, l'alfabetizzazione, le energie, la voglia di giocare. Tutto questo va tenuto in conto **prima ancora di immaginare l'obiettivo**; e poi tutto il resto, dinamiche, meccaniche, grafiche: "prima di tutto deve arrivare il target".

Non solo marketing. La slide mette il target sotto la parola *marketing*, ma non è solo marketing. Anche se non si vuole vendere niente ("voglio creare una caccia al tesoro per esplorare la città e farla esplorare"), sarà diverso crearla per un bambino di 7 anni o per una persona di 30, con competenze, voglie e interessi diversi. A maggior ragione con un **editore**, a cui bisognerà argomentare le proprie scelte: "ci siamo permessi di usare più testo perché sappiamo che il target ama leggere, viene dalla lettura"; oppure "abbiamo fatto scelte di design diverse perché abbiamo un pubblico più digitalizzato". Queste informazioni aiutano a parlare con l'editore e, sul piano del marketing, a capire quanto potrà vendere qualcosa, perché si sa già che scelte faranno le persone in base a ciò che già comprano ed esperiscono. Sul piano del **design**, poi, bisogna sapere

più o meno come si muove il target per dargli l'esperienza che vorremmo avesse. Si pensa in parallelo a **com'è il gioco** e a **come sono le persone** che lo giocheranno, in un circolo: si definisce un target, poi si verifica se è davvero quello.

2.6.12 Dall'etichetta alle persone: la mappa dell'empatia

Per capire se il gioco funziona come lo si è immaginato ci sono "N modi, N strumenti", e Ruggero ne parlerà a fondo nella lezione sul playtest. Beatrice vuole però accennare a uno strumento che trova molto potente per capire che, quando si racconta qualcosa a qualcun altro, quella cosa cambia forma, "perché non siamo nella testa di un'altra persona".

La persona target, e il suo limite. Nel marketing, e anche nei giochi, si usa spesso l'idea della **persona target**: per esempio, la **mamma lavoratrice di 45 anni che abita in una grande città**. Pensarla solo così è un problema: non basta, e non si dovrebbero nemmeno categorizzare così le persone. Questa categoria "a volo d'aquila" serve per iniziare a creare; ma quando si scende nella progettazione vera e propria, e ancor più nel playtest, bisogna andare a capire **chi sono** le persone individuate. Se il target sono le mamme lavoratrici over 40 di una grande città, bisogna **andarle a cercare**: una lista di 10, 20, 30 persone (a seconda dell'obiettivo, della grandezza del progetto, delle risorse), con dei nomi, perché **devono esistere realmente**. L'errore che Beatrice vede fare spesso, nel marketing e nella progettazione dei giochi, è scegliere il target mettendoci sopra un'**etichetta fittizia**. Già in fase di progettazione bisogna dire "questa è l'idea che mi sono fatta del target", e poi andare a cercare quelle persone: "sto producendo un gioco, sto facendo una sperimentazione: ti va di dialogare con me, di provare il mio gioco, di rispondere a delle domande?". Lo scopo dell'individuare un target **non è avere una persona immaginaria**, ma una **lista di persone con nome, cognome, email e contatti**, che rientrano in ciò che pensiamo sia il target, per capire che cosa pensano, vivono, sentono e fanno.

La mappa. Lo strumento è la **mappa dell'empatia** (*empathy map*): nata nel marketing, ma per Beatrice molto utile nel design, perché ricorda che **le persone per cui progettiamo esistono**, hanno vite, menti e bisogni diversi dai nostri, cose che le accomunano e cose che le differenziano. La mappa è un modo per allenarsi a questo tipo di pensiero, che è indispensabile (la mappa in sé no). Ha **sei sezioni**, con al centro la persona: "proprio al centro", perché si fa **una mappa per ogni persona** che diventerà tester del gioco. Questa è per Lucia; poi se ne stampa una per Francesca, una per Ruggero, una per Tiziana. Per ogni sezione si fanno domande o si osserva la persona nel suo ambiente (figura 2.6.6):

Che cosa sente: i contesti in cui la persona è inserita, che cosa sente dire dagli amici. "Lucia dice: la mia amica Francesca ha detto che Fede è un grande". Ce lo si appunta.

Che cosa pensa e prova: forse la sezione più importante, e la più difficile da raggiungere, "fuori dai denti". Riguarda le aspirazioni, le preoccupazioni, le cose che la persona ritiene importanti. Lo si capisce chiedendo direttamente, ma anche **osservando le scelte** che fa: come usa il suo tempo, le scelte di consumo, di vita, di relazioni, le situazioni in cui si ritrova, le cose a cui dà la priorità.

Che cosa vede: l'ambiente: dove abita, che cosa vede sui social, che cosa vede fare, dire, indossare, scegliere dagli altri, che cosa legge online e sui giornali. Non cose che sceglie, ma cose che "le arrivano addosso".

Che cosa dice e fa: le scelte con cui si presenta al mondo; le si appunta e le si osserva.

Le fatiche: dal pacchetto di informazioni raccolte, chiedendo e osservando, emergono le fatiche che la persona vive: le **paure**, le **frustrazioni**, gli **ostacoli** che incontra;

Le aspirazioni: e i **desideri**, le cose che le piacciono, i **bisogni più profondi**.

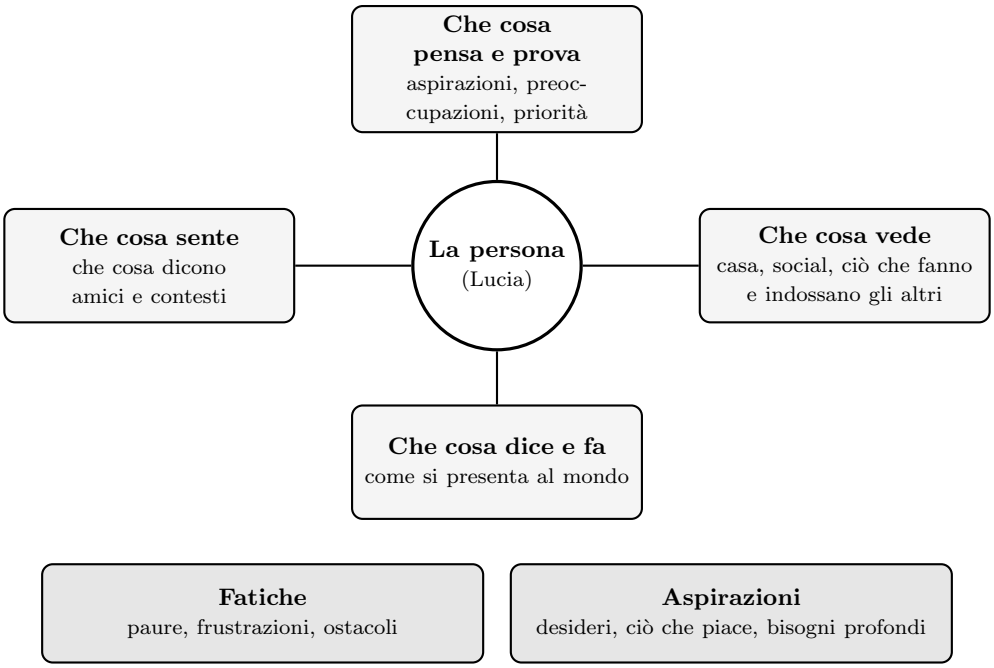


Figure 2.6.6: La mappa dell'empatia come la descrive Beatrice: sei sezioni, con la persona al centro, e una mappa per ogni tester. Le quattro sezioni superiori si riempiono chiedendo e osservando; fatiche e aspirazioni sono ciò che se ne ricava.

Non approfondire è una scelta; non sapere è un errore. Con i tempi che di solito si hanno per fare design, non si andrà così nel dettaglio su tutte queste cose ("oppure sì, dipende"). Ma bisogna sapere che queste informazioni danno tantissimi strumenti per progettare meglio. **Decidere di non approfondirle** può essere una scelta ("non abbiamo il tempo di andare a vedere in che quartiere abita Francesca, che cosa guarda sui social: dedichiamo più tempo ad altro"); **non sapere che sono utili è un errore**.

La mappa come verifica del target. La mappa serve anche a **verificare se il target è quello giusto**. Si parte magari con l'idea di un gioco che insegni l'inglese, rivolto alle mamme lavoratrici over 40 di una grande città; e poi si scopre che a loro di un gioco che insegna l'inglese non importa niente, o che importerebbe se cambiasse la forma del gioco. Questo può far interrogare sull'obiettivo stesso: forse non è il target giusto, oppure lo è ma bisogna cambiare qualcos'altro. È un circolo che si raffina pian piano: "più facciamo design, più ci è chiaro che direzione dare".

2.6.13 Come si raccolgono le informazioni

Non sempre c'è il tempo di approfondire così tanto, e non è detto che si ottengano le risposte che servono: le persone possono non avere tempo, o non avere interesse a essere sincere, oneste, dirette. La mappa dà le **direzioni** sulle macro tematiche; per raccogliere le informazioni si possono fare "N cose":

- **domande dirette**, telefonate (Beatrice salta la parte sul **GDPR**, dando per scontato che all'inizio, in questo contesto, si lavori con persone che si conoscono; in ambienti più grandi ci sono norme da valutare). Dove c'è il tempo, il modo e la situazione per **parlare a voce**, "top": è sempre positivo, anche per comunicare il progetto, perché è uno scambio in cui si imparano cose sul target e allo stesso tempo ci si fa conoscere;
- **internet**, "un grandissimo dono": i social network (Reddit, Pinterest, Instagram) e gli strumenti per vere e proprie indagini di mercato (Google Trends, NotJustAnalytics[24] e tanti altri);

[24] La trascrizione dice "Google Trend, Not Just Analytics". NotJustAnalytics è uno strumento di analisi dei profili Instagram.

- le domande **senza comunicazione diretta**: sondaggi, email a cui rispondere, e così via[25].

Intervistare le persone resta essenziale. Se lo si riesce a fare già in pre-progettazione, "top"; se no, e si fa ricerca in altro modo, sarà comunque essenziale nel **playtest**, nell'iterazione e nell'analisi dei feedback, da osservare direttamente nei primi playtest o da chiedere.

I colleghi. "Un'altra cosa che sottovalutiamo molto è chiedere ai colleghi e alle colleghe". Qui Gianmarco interviene: nei playtest si consiglia di **registrare** come vanno, per vedere che cosa succede? Beatrice vede Ruggero "che sta vibrando" e gli passa la domanda. "La risposta breve è sì, è possibile, ma con un asterisco su che cosa comporta": se ne parlerà in modo molto approfondito nella lezione sul playtest, "tre ore solo sul playtest". Ricerca e sviluppo e playtest hanno parti che si sovrappongono, e qui Beatrice copre "la parte più a ombrello" del processo.

Con le persone bisogna parlarci. Beatrice riconosce di aver mischiato le fasi, e spiega perché: le vede spesso sottovalutate, "e provo proprio rabbia", perché ci si appoggia all'idea del **target come etichetta** (donna di 40 anni, uomo, giocatore di D&D), ed è "sbagliatissimo": poi non si ha la base per raccogliere feedback reali, non si capisce più che cosa si sta facendo, e intanto si lavora su cose che non sono quelle corrette. Del resto la divisione in fasi (progettazione, playtest, iterazione, uscita) è un'astrazione: durante il playtest, a seconda del tipo di gioco, possono cambiare cose anche molto grosse; è come se ci fossero più fasi di *brainstorming*, di playtest, di iterazione, e ogni volta che cambia anche solo un dettaglio del target o dell'obiettivo bisogna tenerne conto, e la cosa si ripete. La regola minima, "insindacabile per un progetto sperimentale in un corso di sperimentazione": in tutto il percorso di un gioco bisogna aver fatto domande e **ascoltato la voce delle persone**; non per forza filmarle, non per forza sempre, ma **almeno due persone dovete sentirle con la voce**. "Se non lo fate, c'è un problema". Poi ci sono anche altri modi, ma una parte delle persone va sentita dal vivo.

2.6.14 Il target troppo largo: partire da chi si esclude

Beatrice dà per scontato che a un certo punto ogni gruppo sceglierà un target, e che in corso d'opera si accorgerà che non è proprio quello corretto; in particolare, che è **troppo ampio**. Lo chiama *errore*, "ma è talmente normale nel passaggio creativo che non so neanche se chiamarlo errore": lo chiama così per ricordare che bisogna starci attenti. Si deciderà di fare una cosa per "le mamme quarantenni lavoratrici", e poi si dirà: no, è troppo grande. **Non esistono giochi per tutte le persone**. Anche quando sembra di aver scelto un target con precisione e con tante caratteristiche, si deve dare per scontato che sia **una nicchia troppo grande**: ci si accorgerà che si sta creando un'esperienza per uno specifico tipo di persone.

La scuola di pensiero di Beatrice: **se il vostro gioco, e anche la vostra comunicazione, non esclude qualcuno, state sbagliando qualcosa**. E quindi, se non si sa da chi partire (perché "che target scelgo?" è una domanda gigante), **partite almeno da chi non volete**. Se il gioco ha una piccola app, non si può partire da chi non ha una base di alfabetizzazione digitale; se è un certo tipo di gioco da tavolo[26], non si può partire da chi non ha mai giocato da tavolo. Poi si definisce il **segmento**, e infine la **nicchia** (figura 2.6.7).

2.6.15 Le risorse e gli stakeholder

Le risorse. Nella lezione successiva si vedrà in dettaglio come decidere che scelte fare in base alle **risorse**, che sono di vario tipo: le **persone** coinvolte, quindi le competenze; le **tempistiche**; i **soldi**, quando ci sono; i **materiali**. Se voglio produrre un gioco per conto mio e investirci i miei soldi, devo valutare quanti e di che tipo: se voglio le pedine **in avorio**, il costo sarà diverso che **in legno**. Queste analisi vanno fatte sempre **in parallelo** alla progettazione; spesso sono fatte a monte, ma per ora l'invito è a farle almeno in parallelo. "Forse sto andando un po' lunga".

[25] Beatrice accenna a una slide successiva con un libro, "adesso ci arriviamo"; nella trascrizione il libro non viene poi nominato.
[26] La trascrizione dice "se fate un gioco, un'America"; il tipo di gioco non è ricostruibile.

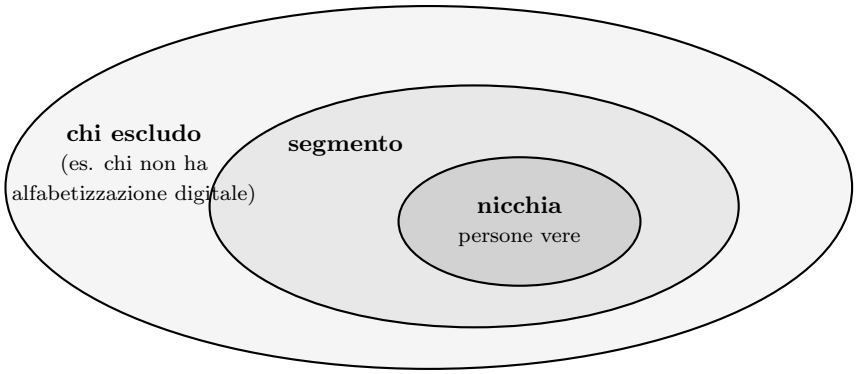


Figure 2.6.7: Restringere il target: se non si sa da chi partire, si parte da chi si esclude, poi si definiscono il segmento e la nicchia. "Non esistono giochi per tutte le persone".

Gli stakeholder. "Ecco, questa slide ci fa abbastanza schifo"; non la slide in sé, "frutto del lavoro di persone di qualità", ma ciò di cui bisogna tenere conto: "ci fanno schifo gli **stakeholder**", esattamente. Si torna al punto di prima: se si vuole progettare per i fatti propri, "top", e questa slide non interessa. Se invece si vuole progettare **tenendo conto dei soldi** ("ve la dico proprio schietta"), cioè di da dove arrivano i soldi per pagare il lavoro di design, la **promozione** ("ahimè, **un gioco senza promozione non esiste**, ragazzi: questa cosa dobbiamo mettercela via") e i materiali, le voci sono queste: la **creazione**, la **produzione** con i suoi materiali, la **promozione**. Per finanziarle ci sono vari modi già visti: andare da un editore con un'idea propria, rispondere alla richiesta di un committente. Tutti richiedono di avere in mente i **portatori di interesse**, che non sono solo il committente o l'editore, ma anche i **potenziali giocatori e giocatrici**, che hanno interesse a giocare se il gioco è reso di loro interesse. Da un lato ci sono le **entità produttive**, dall'altro quelle che **giocheranno**; tenerne conto in progettazione è essenziale. In un corso come questo, però, Beatrice non si soffermerebbe troppo sull'analisi degli stakeholder: basta sapere che, decidendo *perché* si fa un gioco, si stanno già includendo o escludendo certi stakeholder. Andare da un editore implica dinamiche diverse dal fare un **crowdfunding su Kickstarter**; e voler raccogliere 10.000 euro e non 170.000, facendosi promozione e "cosine" per i fatti propri, è un'altra dinamica ancora.

Prepararsi a reagire. Ruggero chiarisce lo scherzo: non è che gli stakeholder facciano schifo; è che spesso non è utile tenere tutto in mente. Anche perché quando ci si interfaccerà con un editore, spesso non si avrà la minima idea di chi siano gli stakeholder, "perché neanche l'editore spesso ce l'ha". Dentro un publisher ci sono il **project manager**, lo **sviluppatore** che lavora sul gioco, la persona della **tipografia** che deve tenere bassi i costi: tutte persone con un interesse nel gioco, che non è detto si conoscano. Il quadro è tanto più compatto quanto meno il gioco è commerciale (un gioco per sé, che non deve essere venduto né fatto con nessuno), ma anche lì ci sono persone che sfuggono al controllo. Quindi: **non cercate di controllare tutto degli stakeholder**, perché è impossibile. È una slide "paradigmatica", per **prepararsi a reagire** a ciò che succede più che per calcolare tutto il possibile, "perché se no non ne uscite vivi, e si vede da me in primis".

Approfondimento: il giocatore come persona, non come categoria. Schell dedica un intero capitolo al fatto che il gioco è fatto *per un giocatore*, discutendo demografia (età, genere) e psicografia (che cosa le persone desiderano dal gioco) e invitando il designer a mettersi nei panni di chi gioca. La lezione spinge questo invito un passo oltre: le categorie servono a orientarsi "a volo d'aquila", ma il lavoro vero comincia quando la categoria diventa un elenco di persone reali da ascoltare. È la stessa ragione per cui Schell considera il playtest una fonte di informazioni insostituibile: ciò che le persone fanno e dicono davanti al gioco corregge ciò che il designer immaginava di loro.

2.6.16 Gli strumenti e i siti per la ricerca

"Bellissimo: abbiamo un po' di strumenti per fare ricerca e sviluppo, ma *dove* inizio? Che cosa guardo, dove vado, che cosa leggo?" Ruggero passa agli **strumenti specifici**, o meglio consigliati (tabella 2.6.2).

BoardGameGeek. "Forse il più grosso": **BoardGameGeek**[27] è un sito **aggregatore** di giochi da tavolo (c'è anche una branca, **RPGGeek**, sui giochi di ruolo, meno curata e meno seguita). È gigantesco: se un gioco da tavolo esiste ed è uscito sul mercato, è in quel database, ed è molto informativo. Dice chi è l'editore, in quante versioni è stato fatto, chi ha lavorato alle traduzioni, quali editori l'hanno pubblicato in quali lingue, anni ed edizioni. Rende facile cercare giochi specifici, divisi per **tema**, per **categorie**, per **meccaniche**. In homepage c'è la lista della cosiddetta **Hotness**: una cinquantina di giochi, i più cercati e con più interazioni sul sito in quel periodo, aggiornata (se il docente ricorda bene) ogni settimana, con l'indicazione di quali salgono, quali scendono e quali restano stabili. È una **fotografia dell'adesso**: di quali giochi si sta parlando in questo momento nell'ambiente del gioco da tavolo.

itch.io. Già menzionato da Beatrice e nei casi studio, **itch.io** è il luogo per la **pubblicazione indipendente**: per chi vuole scavalcare un publisher e pubblicare per conto proprio. È pieno di giochi in **print and play** e gratuiti: non si deve andare in stampa, si può vendere anche solo un PDF, spesso con la formula ***pay what you want*** (gratis, o un'offerta libera). Si può cercare per **tag** (LARP, giochi da tavolo, a pagamento o gratuiti) e scendere ancora più nel dettaglio sul gameplay. Ed è **crossmediale**: giochi di ruolo, giochi da tavolo in print and play, videogiochi, giochi per VR, "tanta, tanta roba"; un repository pensato per la pubblicazione indipendente.

Analog Game Studies. Una ricerca meno sui singoli casi e più su un approccio **teorico e accademico**: è il sito citato a proposito del **Mixing Desk of Larp**, il cui framework era stato documentato anche a livello di archiviazione storica proprio lì. Ci sono articoli di *game studies*, di framework, analisi di casi specifici. È molto variegato e non tutti gli articoli hanno la stessa qualità, ma molti sono "davvero molto, molto di qualità", come quello sul Mixing Desk. Per il docente è più utile per i **framework** che per i casi studio, che a volte sono un po' *mainstream*, pur potendo interessare molte persone.

I content creator. Database come itch.io e BGG sono bellissimi, ma "di giochi all'anno ne escono 10.000": come si capisce che cosa è interessante? "Purtroppo la soluzione è capire e **filtrare i content creator** nel tempo". La slide ne mette alcuni, misti:

- **Hollandspiele**[28] di **Amabel Holland**, una designer che fa articoli e video sui giochi che fa, ma anche molti **video essay** sulle dinamiche del settore e sul game design. Sta anche facendo un lavoro gigantesco di **archiviazione storica**: recuperare chi ha lavorato ai giochi prima che fosse normale mettere il nome del designer sulla scatola, cercandolo nei brevetti, nelle documentazioni storiche, negli articoli di giornale;
- **Shut Up & Sit Down** e **No Pun Included**[29], che recensiscono i giochi da tavolo in uscita, spesso in anteprima, aggiungendo al "è bello o brutto da giocare" un layer di **contesto culturale**: quanto il tema è coeso, di che cosa parla il gioco, quanto le meccaniche portano l'emozione o le remano contro;
- **Ludology**[30], un podcast di game design gigantesco, "qualitativamente altalenante, con puntate bellissime e puntate tremende", molto sull'approccio al game design dei giochi analogici, da tavolo e di ruolo;
- **Cardboard East**, un canale YouTube in inglese che si occupa solo di giochi fatti in **Asia** e nel Sud-est asiatico (Cina, Giappone, Taiwan, Corea, Indonesia e così via). Il consiglio è di guardare anche **altre nazioni e culture**: vedere come lo stesso tema è affrontato da un gioco occidentale e da uno asiatico è molto interessante;

[27] La trascrizione dice "BoardGain Geek" e "Boarding Geek".
[28] La trascrizione dice "Holland Spill".
[29] La trascrizione dice "Shadow Pants e Down, Nopane Included".
[30] La trascrizione dice "L'Udology".

- **Daniel Solis**, il suo blog e il suo YouTube: non di game design in senso stretto, ma di **pratiche parallele** (graphic design, layout, UX) trattate con casi studio di giochi da tavolo.

Il tempo è limitato, e la scelta dipende dal **progetto**: per un gioco di carte magari Cardboard East non serve, e Daniel Solis sì. È una lista "microscopica" e "molto soggettiva": creator che piacciono al docente, "e neanche a tutti".

Premi e jam. Le *game jam* erano già state toccate (se ne era parlato all'inizio della lezione precedente); anche i **premi** servono alla ricerca: capire chi vince che cosa indica la direzione del mercato.

- Lo **Spiel des Jahres**[31] è il premio più famoso del gioco da tavolo. Si può non essere d'accordo con i giochi che premia ("in molti non lo siete, io incluso"), e ci sono anche questioni sociali "perniciose" che il docente non apre; ma i giochi premiati vendono moltissimo, e capire quali vengono premiati dice che cosa il **mercato europeo** vuole vedere sul tavolo, il che informa i negozianti, gli editori, e così a catena.
- Eventi più piccoli, che non hanno premi ma danno un **tempo limitato** per creare un gioco: la "Pub Jam" per i giochi da tavolo, la **Golden Cobra Challenge** per i LARP, e un contest per giochi di ruolo da tavolo e LARP che la trascrizione chiama "Freak Form" e "Flickform"[32].
- Premi più indipendenti, "non meno importanti": lo **Zenobia Award**, per giochi da tavolo di persone che vengono da minoranze o da comunità che non hanno avuto grande spazio nel gioco da tavolo, e il **Diana Jones Award**, che premia l'eccellenza soprattutto nel gioco di ruolo da tavolo. Seguire chi vince, che cosa pubblica, quali editori lo seguono, è utile se si vuole andare in quella direzione.

I motori di ricerca. Infine, "so che sembra banale, ma non è scontato": **cercate**. "Vi sorprenderà sapere quante persone stanno facendo un gioco su un tema e non hanno cercato neanche su Google quali giochi su quel tema sono usciti". Non per bloccarsi ("l'hanno già fatto, a posto, non lo faccio più"), ma per sapere che cosa hanno fatto gli altri: in che cosa è simile a ciò che si vuole offrire, in che cosa è diverso. Sullo stesso tema si possono avere *take* completamente diversi. L'esempio, "stupido, ma per capirci": i **giochi di guida**. C'è il simulativo (*Gran Turismo*), l'arcade (*OutRun*), il party "molto caciarone" (*Super Mario Kart*): tutti giochi di guida, con take diversissimi. "Non vorrei che pensaste: voglio fare *Gran Turismo*, però esiste già *Super Mario Kart*, allora non lo faccio". **Non fatevi limitare dai giochi che esistono**: copiate, trasformate, combinate; fate diverso ciò che non vi piace, simile ciò che vi piace, e **ditelo**.

Progettate il vostro percorso. "Questo è tutto per quanto riguarda le slide". È stata tanta roba, e altra arriverà in asincrono con la dispensa di Daniele. Non c'è l'obbligo di usare tutti questi strumenti: i lavori di gruppo saranno in autogestione. "Però vogliamo vedere che **qualcosa usate**", che non si vada alla cieca, ma che idealmente, come ha detto Beatrice (e come si vedrà in due lezioni con lei), **ci si progetti il percorso**. Anche perché così si butta via meno tempo: se si sa che per una cosa ci vuole tanto tempo e per un'altra tanto, non si finisce col dire "mi aspettavo un gioco da fare in tre ore e sono due mesi che scrivo il regolamento". (Il docente chiude con una battuta un po' nera che richiama il gioco di Angelo e la raccomandazione sui content warning.)

2.6.17 Le domande finali

"Che silenzi. Qualcuno ha domande?"

2.6.17.1 Esiste un catalogo di meccaniche ed emozioni?

Tiziana chiede conferma di una cosa che forse le è già stata detta: dopo l'esercizio del task, esiste qualcosa da leggere che, a partire da ciò che è già stato fatto, spieghi la **relazione tra meccaniche ed emozioni**, o esperienze?

[31] La trascrizione dice "Speed the Yard" e "Hospital Yard".

[32] Né la "Pub Jam" né "Freak Form"/"Flickform" sono ricostruibili con sicurezza dalla trascrizione; il capitolo riporta le forme registrate.

Strumento	**Tipo**	**A che cosa serve**
BoardGameGeek (e RPGGeek)	database aggregatore	ogni gioco da tavolo uscito: editori, edizioni, lingue, temi, categorie, meccaniche; la Hotness come fotografia del momento
itch.io	piattaforma indipendente	print and play, PDF, *pay what you want*; ricerca per tag; crossmediale (anche videogiochi e VR)
Analog Game Studies	sito accademico	articoli teorici, framework (il Mixing Desk), casi studio di qualità variabile
Hollandspiele (Amabel Holland)	designer, video essay	dinamiche del settore, archiviazione storica dei designer non accreditati
Shut Up & Sit Down, No Pun Included	recensioni	giochi in uscita con contesto culturale: coesione del tema, meccaniche ed emozione
Ludology	podcast	game design dei giochi analogici; qualità altalenante
Cardboard East	canale YouTube	giochi dell'Asia e del Sud-est asiatico; confronto tra culture
Daniel Solis	blog e YouTube	graphic design, layout e UX applicati ai giochi da tavolo
Spiel des Jahres	premio	che cosa vuole il mercato europeo; i premiati vendono moltissimo
Golden Cobra e altre jam	contest a tempo	creare in tempo limitato (LARP, giochi da tavolo, giochi di ruolo)
Zenobia Award, Diana Jones Award	premi indipendenti	minoranze nel gioco da tavolo; eccellenza nel gioco di ruolo
Motori di ricerca	—	sapere che cosa esiste sul tema, per differenziarsi e non per rinunciare

Table 2.6.2: Gli strumenti per la ricerca consigliati da Ruggero. "Una lista microscopica e molto soggettiva": da filtrare in base al progetto.

Non esiste una fonte singola. "Vorrei dirti di sì, ma non esiste". Esistono tanti frammenti, di persone con opinioni molto diverse, a volte contrastanti, "a volte anche che litigano di brutto tra di loro". Per capire quanto è profonda "la tana del Bianconiglio": nell'ambito accademico c'è stata una diatriba durata una dozzina d'anni, e in alcune nicchie ancora viva, tra **ludologi** e **narratologi**. I narratologi sostenevano che il gioco comunica attraverso i suoi aspetti narrativi: se un gioco ha emozioni e un tema, sono gli aspetti narrativi a darli. I ludologi rispondevano che il gioco comunica con le **meccaniche**, non con la narrazione, e che un gioco che si basa sulla narrazione e non solo sulle meccaniche è un gioco fallimentare. Entrambi i gruppi, per il docente, sono "completamente sregolati nel loro estremismo"; ma la faida fa capire quanto siano diversi i punti di vista su che cosa un gioco debba avere. Il docente non dirà di non mettere narrazione, o di mettere solo narrazione: vuole dare più strumenti possibili per portare chi gioca in un'**area emotiva**. E quella cosa, "purtroppo o per fortuna", **si fa di solito con le meccaniche**; ci sono tante cose che danno una mano (si vedranno nel paragame), "ma stiamo facendo giochi e non libri", e la priorità va alle meccaniche.

I cataloghi esistenti, e il loro limite. Che cosa portano le meccaniche "è un bel casino". Alcuni ambiti del gioco analogico hanno dedicato più spazio a questa domanda: *Building Blocks of Tabletop Game Design* di Geoffrey Engelstein (e Isaac Shalev)[33], "la versione inglese, se vogliamo", del libro di **Walter Nuccio**, un database di meccaniche che prova a spiegare per ciascuna, più o meno, dove porta emotivamente. Ma è molto **atomizzato**: non dice che cosa succede quando si mettono insieme tre o quattro meccaniche diverse, e bisogna intuirlo. Solo nel gioco da tavolo le meccaniche sono tantissime (*drafting*, *worker placement*, aste aperte e chiuse, *take that*, controllo d'area, *set collection*...): che cosa succede quando le si mischia? E due giochi che sulla carta hanno le stesse meccaniche possono dare al tavolo esperienze completamente diverse, "e questo spero che lo sappiate già".

La frustrazione e i layer meta. Tiziana capisce: si individua un'area, e ci si concentra più sull'esperienza che sull'emozione specifica. La sua era una curiosità: per esperienza, la cosa macro che può dire di un gioco da tavolo è se prova **frustrazione** o no, in base a quanto riesce a gestire il gioco. Certo, risponde il docente, ma ci sono dei **layer meta**: si può provare frustrazione non perché il gioco voglia farla provare, ma perché "il regolamento è scritto male", e allora non sono le meccaniche, è il regolamento a portarla. E c'è un **layer di interpretazione** di che cosa il gioco voglia dire. Il gioco di Angelo: un gruppo lo legge come un gioco che comunica **disperazione** e lo gioca su quell'emozione; un altro lo legge come un gioco di **speranza** ("posso salvare anche la persona che è sull'orlo"), e quella partita sarà emotivamente diversa. È sempre un lavoro di **compromesso**: la difficoltà di chi fa game design è capire come **ridurre quell'area il più possibile**, per renderla meno ambigua. Non è una cosa che si ragiona in astratto: è esperienza "con le mani in pasta".

Nessuno lo sa leggendo. "Ve lo dico e ve lo giuro: di giochi ne avrò fatti più di cinquanta, e tuttora non conosco una singola persona, me incluso, che dalla lettura di un regolamento sappia dire esattamente che cosa quel gioco porta emotivamente, senza vederlo giocato o senza giocarlo". Se fosse così facile, **non esisterebbero i playtest**. "Purtroppo, o per fortuna": farlo è **fare i giochi**, testarli, vederli giocare, giocarli. La parola chiave è **euristiche**: l'esperienza pregressa che informa come si vivono le esperienze successive. Dopo tre o quattro giochi di un certo tipo, "un po' di cose le ho imparate: questo genere di giochi so farlo un po' meglio".

Il contesto cambia (Beatrice). Beatrice aggiunge: la cosa importante è fare **tanta ricerca e tanta sperimentazione**, perché è così che si capiscono e si vedono le cose; per questo il corso dedica tanto spazio alla ricerca e sviluppo, e il playtest sarà fondamentale per ascoltare i feedback. Ma la **certezza** non ci sarà mai, perché non si conosce il contesto in cui tutte le persone sono inserite: per questo contano la mappa dell'empatia e l'idea di target. Se un gioco fatto oggi lo si gioca tra vent'anni, il contesto è già cambiato: *Casablanca* visto adesso e visto quando è uscito dà effetti emotivi molto diversi, come qualunque film o gioco vecchio. Non si può avere piena consapevolezza di tutte le persone che giocheranno, in che contesto saranno, che cosa saranno; si può lavorare sul **qui e ora**, "che è anche un grande esercizio di vita", e informarsi il più possibile. Non esistono giochi per tutti, e nessuna persona risponde esattamente a un'etichetta: "è un po' un balletto". Il consiglio: puntare meno sul *come ottengo esattamente quello che voglio* e più sul **come ascolto**

[33] La trascrizione dice "Building Blocks of Board Game Design, quello di Geofengelstein" e "il libro di Valternuccio".

bene i feedback che ricevo, e ragionare su quelli. "È un cambio di paradigma importante": sembra di rinunciare a un po' di controllo, "ma non è vero".

Il divario tra chi fa e chi gioca. Tiziana precisa che non cercava il controllo: già facendo l'esercizio aveva capito che non è possibile. Basta mettere ruoli diversi, come il medico e il paziente del suo gioco, perché le emozioni siano completamente diverse; per i suoi compagni di gioco erano in alcuni casi diametralmente opposte. Qual è allora l'emozione dominante? Dipende dal personaggio. Il docente (con ogni probabilità Ruggero) le dice che il tema gli interessa particolarmente: la sua **tesi di master in design** era proprio sul **gap di interpretazione** tra l'emozione intesa da chi fa il gioco e quella di chi lo gioca, intervistando separatamente le due parti ("tu, designer, che cosa volevi fare? Tu che hai giocato, che cosa secondo te il gioco comunicava?"). **È impossibile colmare quel gap del tutto**: lo si può solo **ridurre**, "perché nessuno è voi, solo voi siete voi". E molte cose sfuggono al controllo anche sul piano produttivo: si vuole stampare il gioco in un modo e non si può, o costa troppo, e i compromessi sono tagliati a monte. In più, come si vedrà nella parte su prototipazione e regolamento, **il regolamento è un medium lineare, ma i giochi sono ecosistemi**: una regola ne impatta un'altra, che impatta la precedente. Come le si mette in formato lineare? "Lo vedremo più avanti".

Beatrice chiude il punto sottolineando che nessuno aveva colto in Tiziana una voglia di controllo: il discorso voleva solo dire che la classe ha fatto un grande lavoro sulle emozioni, e che poteva passare il sottotesto "avete pensato che arrivate a B, e arrivate sicuramente a B". "Invece non è proprio così: è più sfumato".

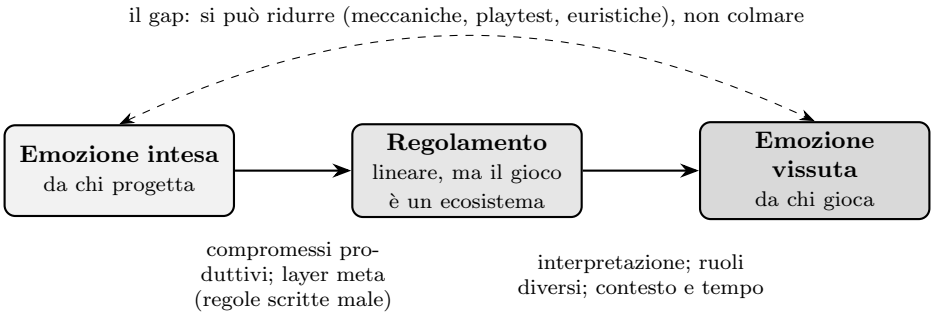


Figure 2.6.8: Il divario tra l'emozione intesa e l'emozione vissuta, come emerge dalle domande finali. Lo si restringe con le meccaniche, il playtest e l'esperienza accumulata; non lo si elimina.

2.6.17.2 Le escape room

Beatrice usa gli ultimi minuti per una domanda "che può sembrare un po' strana": a quante persone del gruppo interessano le **escape room** come progettazione? Rispondono una studentessa[34] e Lorenzo ("sì, sono interessato"). "Mi è già sufficiente". Il motivo: per le lezioni successive i docenti hanno preparato una sorta di **percorso guidato**, uno schema di come si può progettare un gioco con i tempi e la struttura del corso, per allenarsi ad avere una direzione. Beatrice vorrebbe inserirci anche un **modulo sulle escape room**, già previsto, ma a cui si è dato meno spazio per scelte fatte in corso d'opera, "perché adattiamo molto il corso alle persone che partecipano". Ci dormirà su, si confronterà con i colleghi, e il tema sarà comunque toccato; si capirà insieme quanto approfondirlo.

2.6.17.3 I saluti

C'è tempo per un'ultima domanda. Uno studente ringrazia: tra dieci minuti deve uscire di casa, "ci vediamo lunedì". Un'altra chiede se i docenti manderanno delle domande a cui rispondere. No: Daniele manderà una **dispensa**, "non credo oggi, nei prossimi giorni", e comunque **non è per lunedì**. Non sono domande, è **materiale aggiuntivo**: uno *zoom* più dettagliato sugli strumenti di *brainstorming* che per motivi di tempo

[34] La trascrizione dice "Cristiana", nome che non corrisponde a nessuno degli iscritti; potrebbe trattarsi di Tiziana.

non sono entrati nella lezione, "che era già bella piena". Non è obbligatorio e non ci saranno interrogazioni sugli strumenti; come sempre, sono strumenti in più per lavorare. Li si trova nel Drive, nella cartella delle dispense. Beatrice saluta e augura buon weekend; Ruggero ricorda che lunedì la lezione è con lei. "Ciao ciao, buon weekend".

Punti chiave del capitolo

- Non si controlla l'esperienza di chi gioca: si definisce un'**area emotiva** e la si restringe con le meccaniche e il playtest. Nel 200 Word RPG ci sono componenti comuni (un mazzo standard sì, un mazzo di prompt no) e niente colpi di scena: "con le cose che ho in casa adesso posso partire a giocare?". Nel task conta il **processo**, non l'idea migliore: le idee sono tante, conta l'**esecuzione**.

- Il **metodo di lettura**: il regolamento letto ad alta voce come da chi l'ha appena scaricato, l'autore che tace e si segna le ambiguità, poi le emozioni secondo la classe e secondo chi l'ha scritto.

- Dai nove giochi: regole implicite sì, se non lasciano scelte aperte; dichiarare il **player count**; attenzione alle meccaniche che tolgono **agency** (il gioco dell'oca); il **buon senso** non è una regola, e "fate come volete" non è design; l'emozione **non si scrive** (né nominando un valore né descrivendo come ci si sentirà); i temi delicati chiedono **content warning**, meccaniche di sicurezza e consulenti; i freeform aperti rischiano di far sfuggire tema e messaggio; **terminologia** costante, niente riferimenti culturali locali, nessun cambio di genere non annunciato; **un regolamento non è un pitch**.

- Si fa ricerca per conoscere lo **stato dell'arte**, prendere decisioni, posizionarsi, validare idee, rispondere a domande, tenere la mente elastica e **comunicare l'innovazione**. È costante ("come la botanica") e legata al progetto. Per ogni fase un motore: ispirazione (anche fuori dai giochi), cristallizzare la visione (la notte da soli nel bosco), mix e remix, validazione e mercato.

- **Divergenza** (il calderone delle idee) e **convergenza** (selezione in base agli obiettivi), ripetute nel doppio diamante: avere tante idee è normale. Gli strumenti della dispensa di Daniele non vanno imparati tutti: si prendono quelli che risuonano.

- La creatività non è mistica: è **copiare, trasformare, combinare** (*Age of Steam* e *Steam*; *Magic* e *Dominion*; *Scrabble* più *Dominion* uguale *Paperback*), anche crossmediale. Siatene consapevoli. Le meccaniche non sono tutelabili, nomi, loghi e aspetto sì (il caso *Bang!* e *Samurai Sword*); niente proprietà intellettuali altrui; **dichiarate le ispirazioni**.

- Gli step per validare un'idea: **obiettivo personale** (perché faccio design adesso?), **obiettivo del gioco** condiviso dal team (i pinguini blu), **ipotesi**, **prototipo**, **validazione** con il playtest; e poi di nuovo. È una linea guida: "ci sarà un gran casino".

- Il **target** viene prima di meccaniche e grafiche, e non è solo marketing: bambini di 7 anni e banchieri in pausa pranzo sono giochi diversi. Ma il target non è un'etichetta: è una **lista di persone vere**, con nome e contatti. La **mappa dell'empatia** (sente, pensa e prova, vede, dice e fa; fatiche e aspirazioni) va fatta per ogni tester, e serve anche a verificare il target. Non approfondire è una scelta, non sapere è un errore.

- Le informazioni si raccolgono a voce (il meglio), online (social, Google Trends), con sondaggi; si chiede ai colleghi. Regola minima: **almeno due persone sentite con la voce**. Il target sarà sempre troppo largo: non esistono giochi per tutti; **se il gioco non esclude qualcuno, c'è qualcosa che non va**; si parte da chi si esclude, poi segmento e nicchia.

- Le **risorse** (persone, tempi, soldi, materiali: avorio o legno) si valutano in parallelo al design. Gli **stakeholder** vanno dall'editore ai giocatori; "un gioco senza promozione non esiste"; non si controllano tutti: ci si prepara a reagire.

- Gli strumenti: **BoardGameGeek** (e la Hotness), **itch.io**, **Analog Game Studies**, i content creator (Amabel Holland, Shut Up & Sit Down, No Pun Included, Ludology, Cardboard East, Daniel Solis), i premi (Spiel des Jahres, Zenobia, Diana Jones) e le jam (Golden Cobra), e i motori di ricerca: per differenziarsi, non per rinunciare (*Gran Turismo*, *OutRun*, *Mario Kart*).
- Non esiste un catalogo unico di **meccaniche ed emozioni**: c'è la faida tra ludologi e narratologi, ci sono cataloghi atomizzati (Engelstein, Nuccio), ci sono layer meta e di interpretazione. Nessuno sa che cosa farà provare un gioco leggendone il regolamento: per questo esiste il playtest, e contano le **euristiche**. Il gap tra emozione intesa e vissuta si riduce, non si colma; il contesto cambia nel tempo (*Casablanca*); meglio puntare sull'**ascolto dei feedback** che sul controllo. Il regolamento è lineare, il gioco è un ecosistema.

Chapter 2.7

Obiettivi di design e design sprint: i gruppi, il perché di un gioco e un primo gioco inventato insieme

Questa è la **settima lezione** del corso di **Analog Game Design**, ed è la lezione in cui il corso cambia passo. Fino a qui gli studenti hanno lavorato da soli, sui task "didattici" (il personaggio, il gioco di ruolo in 200 parole); da oggi lavoreranno **in gruppo** sul **project work**, il gioco analogico che accompagnerà tutto il resto del corso. La lezione si apre quindi con Daniele, che annuncia i **gruppi**, il *mandate* (che cosa si deve fare e che cosa no), il **calendario** e le regole pratiche per lavorare. Poi la parola passa a Beatrice, che fa due cose. La prima è un "cappellino introduttivo" su **perché si fa game design**: un gioco ha un obiettivo suo, ma anche chi lo fa ha un **obiettivo personale** (pubblicare, esprimersi, formare, fare marketing), e ognuno di questi obiettivi chiede competenze diverse. La seconda, che occupa la maggior parte del tempo, è un **esercizio di design sprint**: tutta la classe finge di essere un unico gruppo di nove persone e, su una lavagna condivisa, percorre le fasi che porterebbero da zero al *pitch* di un gioco. L'esercizio produce un vincitore inatteso ("i funghi hanno un sacco di successo, non me lo immaginavo") e, soprattutto, un metodo da riusare quando un gruppo si blocca: timer, post-it, votazione a pallini, ricerca, **mappa impatto-sforzo**, **percorso dell'utente**, e la regola d'oro "**state sul semplice**".

La trascrizione contiene anche, in coda, la registrazione del **primo tutoraggio**, che il capitolo riporta per intero perché ne è la naturale prosecuzione: un'ora con **Ruggero**, tre giorni dopo, "guidata dalle vostre domande". Lì tornano i temi della lezione, visti dall'interno dei gruppi appena nati: perché partire dalle **emozioni** e non dalle meccaniche, che cosa ci si aspetta davvero dal **pitch** del 12 ottobre, come usare il **Mixing Desk** senza farsene schiacciare, e come prepararsi alla lezione su **Tabletop Simulator** del giorno dopo.

La lezione è **online** e ha tre voci. Apre **Daniele**[1], che resta solo "per la presentazione dei gruppi" e poi si scollega per un'altra riunione; prosegue **Beatrice** ("Bea"), che conduce tutta la lezione e l'esercizio; **Ruggero** non c'è alla lezione (è nominato più volte, anche come "Ruger" e "Rugger") ma conduce il tutoraggio. La lezione si colloca con ogni probabilità **lunedì 28 settembre 2026**[2], dalle 17 alle 20 circa: la pausa è dalle **19:10** alle **19:15**, e la sessione di ricerca finisce alle **19:32**. Il tutoraggio è **giovedì 1 ottobre**, alle 17, su Meet, e dura un'ora. In classe ci sono **Tiziana**, **Rita**, **Tancredi**, **Giordano**, **Angelo**, **Gianmarco** (che arriva a esercizio iniziato), **Nicola** (che arriva ancora più tardi) e i **due Lorenzo**: Lorenzo Vergata e l'altro Lorenzo, che durante la lezione propone di farsi chiamare con il secondo nome, "John", per evitare confusioni[3].

[1] Come nelle lezioni precedenti, il cognome non è ricostruibile con certezza: qui la trascrizione lo fa pronunciare a Daniele stesso come "Daniele Fusetto" (nella prima lezione era "Daniel Cusetto").

[2] La data non è detta, ma è ricostruibile: Daniele parla della lezione di "venerdì" (quella sulla ricerca e sviluppo, del 25 settembre) come appena passata, ricorda che Gianmarco "come sempre, lunedì" arriva verso le sei, e annuncia il tutoraggio per "giovedì 1 ottobre". Nel tutoraggio uno studente parla della "lezione di lunedì" sui funghi e Ruggero osserva "siamo a ottobre".

[3] "Non ce la posso fare, i doppi nomi mi devastano", dice Beatrice. L'altro Lorenzo spiega che il suo secondo nome è John, "non sto scherzando", e Beatrice se lo appunta come "Lorenzo L.". Nel capitolo lo si chiama "l'altro Lorenzo"; Lorenzo Vergata

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Angelo diventa spesso "Angela" (e una volta "Angevo"), Tiziana "Tiziano" e "Cristiana", Bea "Bianca", Tancredi "Tankerat" e "Tank", Canva "Kamba" e "gamba", Tabletop Simulator "Temuto Simulator", "Table of Simulator" e "Debox Monitor"; i giochi citati non vanno meglio (*Hanabi* è "Anabis", *Mage Knight* "Image Night", *Harmonies* "Armonies"). Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: nei momenti in cui la classe lavora in silenzio sotto timer (mentre si scrivono i post-it, durante la votazione, nella pausa e nei dieci minuti di ricerca) la trascrizione registra per decine di righe frasi senza alcun rapporto con la lezione, ripetute in loop ("la situazione è questa, la migliore dal 2005", "il numero di persone è molto più basso", "qual è la risposta che ci può essere?", discorsi su governi, parlamenti e "dottrine sul mercato del lavoro"); anche l'apertura contiene alcune frasi di questo tipo, e a un certo punto un "scusi" ripetuto trenta volte. Sono artefatti del riconoscimento vocale e il capitolo le ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro e resta generico altrimenti.

Come leggere questo capitolo. La lezione è in gran parte un **laboratorio**: conta meno *che cosa* è uscito dall'esercizio (un gioco sui funghi che nessuno svilupperà) e più *come* ci si è arrivati, e che cosa è andato storto lungo la strada (i tempi sforati, i commenti che si infilano tra le domande, le idee troppo personali). Beatrice lo dice più volte: "lo strumento ci interessa il giusto", "ci interessa il procedimento". Il capitolo segue quindi la lezione passo per passo, riportando le idee di tutti, e segnala ogni volta la regola di metodo che il passaggio illustra. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell) e alla letteratura sul design sprint; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa (sezione 2.7.1);
2. il **project work**: calendario, *mandate*, gruppi, regole di lavoro (sezioni da 2.7.2 a 2.7.6);
3. **perché facciamo game design**: obiettivo del gioco e obiettivo personale, e le competenze che ogni obiettivo richiede (sezioni 2.7.7 e 2.7.8);
4. il **design sprint**: che cos'è, il file con le fasi, la chiacchierata iniziale, la lavagna di Canva (sezioni da 2.7.9 a 2.7.10);
5. l'esercizio: l'**obiettivo del gioco**, il racconto delle idee di tutti, la **votazione a pallini** (sezioni da 2.7.11 a 2.7.13);
6. dopo la pausa: i tempi, la **ricerca e sviluppo**, le fasi successive, la **mappa impatto-sforzo**, il **percorso dell'utente** e "state sul semplice" (sezioni da 2.7.14 a 2.7.18);
7. la ricerca condivisa e la chiusura (sezioni 2.7.19 e 2.7.20);
8. il **primo tutoraggio** con Ruggero (sezione 2.7.21 e seguenti).

2.7.1 L'attesa

"Ciao ciao. Oggi è...": la lezione comincia, come sempre, aspettando chi si collega. Daniele arriva per primo, e la docente gli chiede come va: "Eh, insomma". È un periodo pieno, "un periodo un po' particolare"; niente di grave, "è la vita", "siamo qua per il momento". Daniele prende le presenze (Angelo è già collegato; poi arrivano Lorenzo, Giordano, Tancredi) e avverte che dopo la presentazione dei gruppi lascerà la lezione per un'altra riunione. Prima, però, nomina Beatrice **coorganizzatrice** della chiamata, così che possa condividere lo schermo e gestire la registrazione: "se vedi in alto Rec, hai i poteri". C'è anche un piccolo mistero amministrativo: Giordano era stato presentato alla classe? Nella lezione in cui avrebbe dovuto esserlo Daniele non c'era, e "Ruggero si stava per dimenticare ancora". "Gravissimo".

è chiamato per nome e cognome quando serve distinguerli.

Daniele è contento di vedere la classe "belli attivi su Discord": parlarsi e dialogare spesso è "uno dei modi migliori per fare al meglio questo corso". Ora cominciano anche gli altri corsi; Gianmarco, come ogni lunedì, arriverà "un pelo più in ritardo", verso le sei. Mancano Tiziana e Nicola; Tiziana arriva poco dopo. Daniele ha aggiornato le slide e ha "un paio di comunicazioni"; poi rivedrà la classe nella lezione su "fiction e fiction, gioco di ruolo e quant'altro".

Nell'attesa si chiacchiera. Beatrice mostra in camera un antistress, un piccolo *fidget* ("guardate che bello", "in questo periodo proprio necessario"), e il discorso scivola sui videogiochi: Daniele è contento perché è appena uscito un titolo di cui è fan[4] ("anche se fosse stato brutto mi sarebbe piaciuto comunque, però non lo è stato"); qualcuno è fan di Wolverine, "però non dei videogame". Sullo *Spider-Man* più recente il giudizio è che "non è un brutto gioco, ma non è un gioco da 300 milioni di investimenti": non è il gioco che ci si aspetta con quel tipo di budget. "Succede". Nicola non si collega; Daniele decide di cominciare.

2.7.2 Il project work: il calendario

"Annunciamo i gruppi. Da oggi potete iniziare a fare i giochi". Daniele precisa subito, scherzando: in realtà "potevate fare i giochi comunque, non è che dobbiamo darvi noi il permesso", ma da oggi si può fare *il gioco per il corso*.

Prima dei gruppi, però, una notizia sul calendario: il **tutoraggio del 3 ottobre** (un sabato) è stato **spostato**. Ruggero propone il **1 ottobre**, alle 5 del pomeriggio. Daniele dice prima "mercoledì", poi si corregge: il 1 ottobre è **giovedì** ("nella mia testa era giovedì", "se ho scritto mercoledì, non so perché"). I tutoraggi sono su **Meet**, vengono registrati, e **non serve che ci siano tutti**: basta **una persona per gruppo**. Sono "a supporto proprio dello sviluppo dei giochi". Per Beatrice la nuova data è "fantastica"; ne è contenta anche Tiziana, che aveva problemi il sabato.

Gli altri tutoraggi restano, come gli altri anni, di **sabato alle 11**; ma se qualcuno ha problemi "basta dirlo e accordarsi con il docente": si possono spostare. Il quadro complessivo è quello della tabella 2.7.1.

Data	Che cosa succede
lun 28 settembre	questa lezione: annuncio dei gruppi e del *mandate*
gio 1 ottobre, ore 17	primo tutoraggio con Ruggero, su Meet (spostato dal sabato 3 ottobre); basta una persona per gruppo
lun 12 ottobre	**pitch** del gioco, durante la lezione
sab 7 novembre, ore 11	tutoraggio per lo sviluppo vero e proprio
sab 14 novembre, ore 11	tutoraggio per lo sviluppo vero e proprio
ven 4 dicembre, entro mezzanotte	**consegna** del gioco (non c'è lezione)
venerdì successivo	lezione in cui si vedono i giochi

Table 2.7.1: Il calendario del project work come lo ripete Daniele all'inizio della lezione. I tutoraggi del sabato si possono spostare accordandosi con il docente.

Perché la consegna è il 4 dicembre. Il 4 dicembre "non c'è lezione": va solo consegnato il gioco, entro mezzanotte. La lezione in cui se ne parla è il venerdì successivo, perché i docenti hanno bisogno di **una settimana** per rivedere i lavori.

2.7.3 Che cosa si porta al pitch del 12 ottobre

"Ora, cosa ci aspettiamo il 12?". Non il gioco completo, ma **un'idea del gioco strutturata**: che **tipo di gioco** è, che tipo di **emotion design** ha, che cosa si sta cercando di creare come **esperienza di gioco**. I gruppi devono presentarla e "cercare di vendercela" con delle slide, delle immagini, Canva, Google Slides,

[4] La trascrizione scrive "è uscito contro il resonant"; il titolo non è ricostruibile con certezza. Che Daniele abbia in mente la serie *Control* lo fa pensare il fatto che, poco dopo, la citi tra le proprietà intellettuali da non usare.

"come volete". Anche qui verrà in aiuto la dispensa sul *brainstorming*, con i suoi strumenti. Quello che Daniele vuole è "una sorta di: questo è quello su cui noi vorremmo lavorare".

Il motivo è pratico. Tra il 12 ottobre e la settimana successiva, o anche il giorno stesso, se i tre docenti (Daniele, Beatrice e Ruggero) vedono che un gruppo ha tra le mani un gioco **troppo complesso** da sviluppare nel poco tempo disponibile, lo dicono subito: "attenzione, no, non va bene, è troppo complesso, vi state entrando in una gabola". È un modo per far **risparmiare tempo** ai gruppi e per i docenti un modo di dare **feedback**. (Nel tutoraggio Ruggero tornerà sul punto, precisando che cosa il pitch *non* è: si veda la sezione 2.7.21.7.)

2.7.4 Il mandate

Il project work **non ha un tema**: a differenza di una *game jam*, dove il tema c'è, nel corso di Analog Game Design "non c'è un tema, ma deve essere un gioco analogico". La slide successiva elenca il *mandate*, cioè i vincoli del progetto (figura 2.7.1).

Un gioco analogico. *Board game*, *tabletop RPG*, LARP, *pervasive game*, ibridi.

Obbligatorio l'emotion driven design. "Io voglio vedervi, anche Ruggero, anche Bea, vogliamo vedervi lavorare sulle emozioni". Daniele ha appena finito una chiamata con un ragazzo del terzo anno di game design "che non ha molto capito queste robe qua": è la cosa più difficile da capire, e per questo la lezione sull'*emotion driven design* è molto importante.

Niente proprietà intellettuali esistenti. Non si possono usare IP esistenti, "tranne quelle per cui l'autore è morto da 70 anni". Anche qui con una riserva personale: "anche se io aborro", perché Lovecraft "l'abbiamo visto in tutte le salse", "veramente tante volte". Va bene Lovecraft, non va bene usare *Destiny*, *Control*, le IP della Disney o della Marvel.

Niente overscope. Non fare progetti sovracomplicati: "avete due mesi e qualcosa", non è il tempo per progetti "magniloquenti". Soprattutto per i giochi di ruolo cartacei: si possono fare, ma "non va bene un *Dungeons & Dragons*". Meglio giochi più piccoli: one page, o giochi di una decina di pagine, focalizzati, come *Lady Blackbird* o come un altro titolo che la trascrizione rende con "Acqua e Tier" e che non è ricostruibile. Oppure, si può "spaziare" e fare un *pervasive game*.

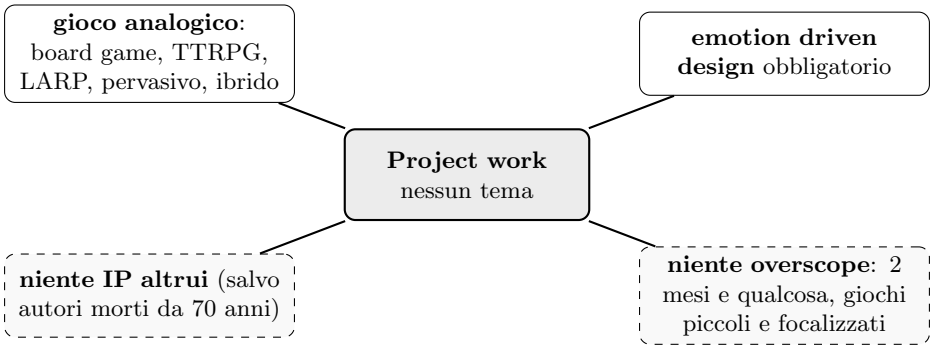


Figure 2.7.1: Il *mandate* del project work: due cose da fare (in alto) e due da non fare (in basso, tratteggiate). Ricostruzione schematica della slide di Daniele.

2.7.5 I gruppi

Poi i gruppi, tre gruppi da tre (tabella 2.7.2). La scelta è dei docenti ed è "insindacabile", ma è basata sulle attività e sui task fatti finora, "sulla base di quello che vi abbiamo fatto fare negli ultimi giorni, come lezioni,

Gruppo	Componenti
Gruppo 1	Giordano, Nicola, Rita
Gruppo 2	Gianmarco, Tancredi, Lorenzo Vergata
Gruppo 3	Tiziana, Angelo, l'altro Lorenzo

Table 2.7.2: I gruppi del project work come li annuncia Daniele[5].

sulla vostra task e su quello che è stato un po' il nostro modo di vedere". Si è cercato di bilanciare anche gli **impegni di lavoro** di ciascuno.

Il progetto va sviluppato **durante il corso**: "non vanno bene idee pregresse, non portate nei vostri gruppi idee precedenti". Le idee si sviluppano insieme agli altri.

2.7.6 Le regole per lavorare in gruppo

Prima di lasciare la lezione a Beatrice, Daniele dà "altri consigli per gli acquisti".

Rispettarsi. "La maggior parte del lavoro di un gruppo di sviluppo è di **soft skill** più che di skill". Bisogna capire come collaborare: se qualcuno ha impegni di lavoro e fa fatica a garantire del tempo, "partite da quello". Non si parte con "un progetto grande come un castello disneyano" per poi trovarsi senza tempo; ci si dice invece "io posso lavorare solo in questi momenti, lui o lei solo in questi momenti", e si cerca di fare "qualcosa che abbia senso fare per il tempo che abbiamo a disposizione".

Taggare i docenti. In caso di problemi si scrive ai docenti su **Discord**, ma **taggandoli**: "non lasciate messaggi senza taggarci, se no li vedo una settimana dopo". Oppure su **Telegram**.

Usare subito questi giorni. Questi giorni vanno usati per il *brainstorming*: il tutoraggio con Ruggero è a breve (Daniele parla di "48 ore" per impostare il lavoro e capire che cosa fare).

Lavorare nel Drive. Questa è una regola **obbligatoria**. Nel Drive condiviso del corso ci sono le cartelle "attività in classe", "dispense", "materiali didattici" e "project work"; dentro "project work" Daniele crea tre cartelle, **gruppo 1**, **gruppo 2** e **gruppo 3**. Lavorare nel Drive vuol dire evitare il più possibile di creare file fuori e caricarli dopo: si aprono i Google Document lì e si lavora **in cloud**. Il motivo: se un gruppo ha problemi e Daniele vuole aiutarlo, ha bisogno di vedere il **processo mentale**, cioè che cosa è stato fatto nel documento, "lo **storico** del documento". Un PDF o un `.docx` caricati così come sono non hanno storico. Le cartelle vanno organizzate al meglio possibile; per qualsiasi problema, "basta chiedere".

La dispensa sul brainstorming. A seguito di questa lezione, "tra stasera e domani mattina", Daniele registrerà una **dispensa**: il pezzo che venerdì non c'è stato modo di fare, sul *brainstorming* e sulla ricerca e sviluppo "lato marketing" e sugli strumenti di marketing nella ricerca e sviluppo. Metterà a disposizione le slide e un video, con le slide del **brainstorming strutturato**.

Nessuna domanda ("molti pollicioni, va bene"). Daniele ringrazia Bea "per il supporto e la condivisione", saluta ("ci vediamo sicuramente il 12 di ottobre") e lascia la lezione.

2.7.7 Perché facciamo game design: due obiettivi

"Ciao gruppo". Beatrice comincia dando un po' di contesto: il corso è alla **seconda edizione**, ci sono cambiamenti rispetto alla prima, e i docenti stanno cercando di capire "come rendervi il lavoro più agile e fattibile possibile". Nelle slide di oggi c'è molto, "sarò onesta, credo troppo per la lezione di oggi", e quello

[5]La trascrizione dice "Giordano, Nicole, Rita, gruppo 2 Gianmarco, Tancredi e Lorenzo V, gruppo 3 Tiziana, Angelo e Lorenzo". "Lorenzo V" è Lorenzo Vergata, come confermano poi i messaggi tra lui e Tancredi durante la pausa; "Nicole" è Nicola, a cui più tardi Beatrice dice che è nel gruppo "con Giordano e Rita".

che non si farà si riprenderà nella lezione successiva. Il programma: "mettere le mani in pasta" per prepararsi ad affrontare un lavoro di design; un po' di *brainstorming* (da approfondire con i materiali di Daniele); e una sorta di percorso di creazione di un gioco con un piccolo **esercizio di design sprint**, a cui sarà dedicata la maggior parte della lezione. Alla fine del corso i docenti chiederanno anche dei **feedback** su come è andato tutto.

Prima, però, un "cappellino introduttivo" sul **perché facciamo design**, già anticipato nelle lezioni precedenti. Ci sono **due entità** che in questo contesto si chiamano **obiettivi di design**:

1. l'**obiettivo del gioco**, della produzione che si fa;

2. l'**obiettivo personale** di chi progetta.

L'obiettivo del gioco. Un obiettivo di design del gioco, o del momento in cui si sta facendo design, ha molte possibilità: raccontare un'esperienza di vita, esplorare meccaniche o dinamiche, e così via; gli esempi della slide sono "buttati lì, sempre in brainstorming". Si può decidere di fare un gioco per N motivi, alcuni personali e altri esterni al proprio bisogno personale.

Il punto è che **il motivo per cui affrontiamo la creazione di un gioco cambia anche come progettiamo**. Beatrice vuole invitare la classe a ragionare su queste cose, che spesso restano **ragionamenti interni** e invece è importante che siano anche **condivisi**.

L'obiettivo personale. L'obiettivo personale è semplice: rispondere alla domanda **"perché sto facendo questo gioco?"**. Beatrice propone **quattro categorie**, "ma ce ne sono molte altre", e probabilmente ne emergeranno di nuove col crescere della disciplina; per la classe, i macro-obiettivi saranno verosimilmente questi (figura 2.7.2).

1. **Voglio pubblicare un gioco.** Come gruppo di lavoro o come persona, voglio parlare a degli editori e pubblicare. Sotto questo cappello stanno sia la **pubblicazione con un editore** sia l'**autopubblicazione**.

2. **Voglio esprimere me stesso o me stessa.** È "molto sottovalutato, troppo sottovalutato": il gioco è uno strumento di espressione molto grande, che coinvolge anche altre persone. Magari non si vuole pubblicare né vendere, ma trasmettere davvero qualcosa che si è vissuto, qualcosa che si vuole raccontare; o si fa attivismo politico e si vuole un'attività che sensibilizzi sull'ecologia o su altri temi.

3. **Voglio formare, informare o sensibilizzare.** Può sovrapporsi al punto precedente. Un gioco per insegnare meglio la matematica, per far conoscere un quartiere della propria città o un evento storico, per far crescere una competenza (aiutare le persone a parlare meglio in gruppo): "N situazioni di formazione".

4. **Voglio usare i giochi per il marketing e la comunicazione.** Portare l'attenzione su qualcosa: può essere un'espressione personale o una sensibilizzazione, ma anche un **prodotto** o un **servizio**. Lo si è visto con gli *alternate reality game* del marketing, e in tante altre situazioni.

2.7.8 Le competenze che ogni obiettivo richiede

Ognuno di questi obiettivi personali richiede delle **competenze**, che ci si fa "in corso d'opera". I docenti non si aspettano che gli studenti approfondiscano ogni slide durante il corso, ma vogliono dire una cosa: se si crea un gioco, tutta l'analisi di cui si è parlato (del mercato, delle persone, del target) va tenuta in considerazione **in funzione dell'obiettivo**. Ogni obiettivo ha un modo diverso di guardare il gioco e di produrlo. La slide mostra una "carrellata" di competenze e conoscenze (tabella 2.7.3).

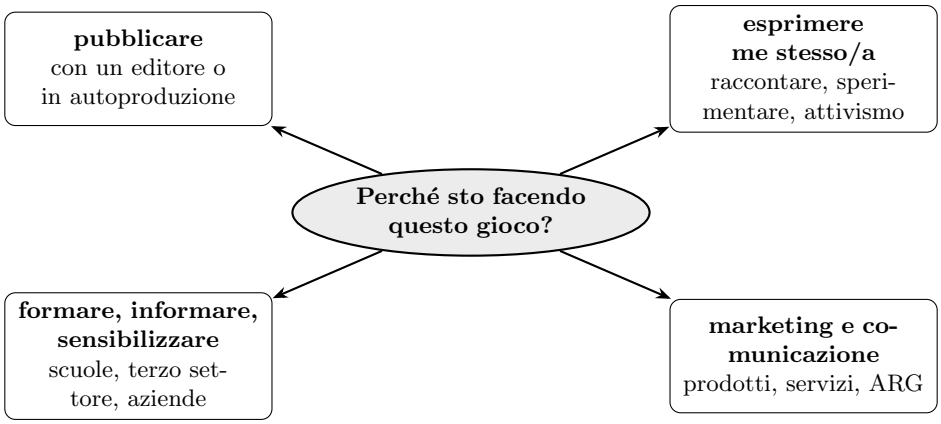


Figure 2.7.2: Le quattro categorie di obiettivo personale proposte da Beatrice. "Ce ne sono molte altre"; esprimersi e sensibilizzare spesso si sovrappongono, e uno stesso gioco può rispondere a più obiettivi.

Pubblicare con un editore. Sono emerse domande anche dalla classe su come capire che cosa sia meglio fare. Bisogna **conoscere il mondo della produzione** di giochi da tavolo, di giochi di ruolo e di eventi ludici: chi esiste, che cosa fa, a che prezzi mette fuori le cose, che tipo di prodotti vende già; se ci si riesce, quante copie vengono vendute, quante persone comprano un biglietto. E **conoscere le persone**: non solo "c'è la casa editrice con il nome X", ma "all'interno ci lavorano X, Y, Z". Partecipare agli eventi, mettersi in connessione: sono tutte competenze, orientate a capire **come inserirsi** nelle realtà che possono avere interesse a pubblicare.

Autoprodursi. Le logiche sono diverse. Il mondo editoriale va conosciuto comunque, perché in questo caso sono **competitor**; ma bisogna ragionare di più verso il **pubblico**: capire come arrivare al momento in cui le persone acquistano. Quindi parlare con gli utenti, affinare il marketing, analizzare i competitor. E i competitor non sono solo quelli che fanno la stessa attività: per un **evento dal vivo** un competitor può essere una fiera o un concerto, non per forza un'attività di gioco; per un **gioco da tavolo formativo** un competitor può essere un **corso**, non per forza un altro gioco. "Immaginate i competitor un po' più ad ampio spettro". Infine, tutta una serie di informazioni "non scontate" e "spesso prese sotto gamba": **fiscalità e normative**.

Il GDPR: un'anticipazione. Su questo, nella prossima lezione, Beatrice farà "un piccolo cappello" sulle basi del **GDPR**, per poter raccogliere i dati degli utenti e ricontattarli. È utile qualunque sia l'obiettivo: è sempre utile avere, in una forma che permetta di risentirle, i dati delle persone con cui si è venuti in contatto, che magari hanno testato il gioco, per tornare a scrivergli e capire una serie di cose.

Esprimere se stessi. Per sperimentare, per esplorare: qui ci sono competenze "spesso non raccontate, perché meno orientate al mercato", che riguardano **come trasmettiamo i contenuti** che vogliamo portare. È il lavoro di **emotional design**, che "sembra complicatissimo", o comunque difficile da definire; lo è, perché cambia per ogni gioco, ma col tempo si raffina sempre di più. Chi vuole usare il gioco come sperimentazione personale, come linguaggio di dialogo personale, deve capire bene quali sono i propri obiettivi: può essere "voglio comunicare la mia storia personale", ma anche "voglio trovare persone a me affini", o, "banalmente", "voglio essere la persona che ha fatto il gioco più conosciuto sulle **lumache da prato**". Se è quello che si vuole raggiungere, va bene: capito bene l'obiettivo, si capisce meglio tutto il resto, cioè **che cosa voglio dire, a chi dirlo e perché** voglio dire determinate cose.

Formare, informare, sensibilizzare. "C'è un mondo" su come si fa questo design; qualcosa si vedrà nella prossima lezione, con le **escape room** e le escape room formative. Per capire il mercato che può pagare queste attività, secondo l'esperienza di Beatrice ("che non è onnisciente, ma è abbastanza puntuale

al giorno d'oggi"), le macro-voci in cui ci sono dei soldi, escludendo l'**autofinanziamento** (che lei mette in una voce a sé), sono tre:

- il **terzo settore** e i **bandi**;
- il mondo delle **scuole** e della **formazione**;
- la **formazione aziendale**: *learning and development*, risorse umane e simili.

Sono tre mondi con logiche "completamente diverse" da quelle viste prima, e Beatrice non ci si sofferma, perché non è il fulcro del corso. Ma già queste voci bastano per sapere dove approfondire, se si vuole fare un gioco formativo: che cos'è il terzo settore e che cosa significa proporgli dei giochi; che cosa sono i bandi pubblici e quali enti li finanziano; e lo stesso per le scuole e per le aziende. E poi c'è "il magico mondo della **formazione finanziata**", che sta sempre sotto il cappello delle risorse umane: già googlando queste due parole si trovano molte situazioni in cui si inserisce la proposta di giochi.

Il marketing. Qui la parte principale da sapere è quella già vista sull'**analisi del target** e sull'individuazione delle persone da coinvolgere. Beatrice non si sofferma, perché secondo lei il marketing "arriva dopo": quando si è progettato un po' di più (vale anche per la formazione) e si è sperimentato quello che si è in grado di fare, allora si possono immaginare giochi per il marketing, che sono giochi che portano l'utente a fare determinate cose. Per chi vuole approfondire, la cosa principale è questa: si parla con delle **aziende**, e le aziende, "anche le più virtuose", hanno l'obiettivo di "fare dei soldini". Bisogna saper rispondere alla domanda **"come aiuta l'azienda a fare più soldi?"**. I casi sono due: **aumenta le vendite**, oppure fa aumentare **quanto l'azienda è conosciuta**, o, se è conosciuta male, ne modifica in positivo la **reputazione**. "Non è un mondo bellissimo, ma è un mondo che esiste": giochi di questo tipo esistono tantissimo, e spesso non sono neanche raccontati come tali. Beatrice lo dice perché magari si ha "il gioco migliore del mondo" e non si sa come pagarsene lo sviluppo: questo può essere un modo.

Obiettivo	**Che cosa bisogna conoscere**
Pubblicare con un editore	il mondo della produzione (chi c'è, che cosa fa, a che prezzi, quanto vende); le persone dentro le case editrici; eventi e contatti
Autoprodursi	il mondo editoriale come competitor; il pubblico; marketing; competitor "ad ampio spettro" (una fiera, un concerto, un corso); fiscalità e normative; GDPR
Esprimere se stessi	come trasmettere i contenuti; *emotional design*; che cosa voglio dire, a chi e perché
Formare e sensibilizzare	terzo settore e bandi; scuole e formazione; formazione aziendale (*learning and development*, risorse umane); formazione finanziata
Marketing	analisi del target; "come aiuta l'azienda a fare più soldi?": vendite, notorietà, reputazione

Table 2.7.3: Le competenze per obiettivo personale, ricostruite dalla carrellata di Beatrice.

Lo stesso gioco, più obiettivi. Uno stesso gioco può rispondere a più obiettivi. Si fa un gioco formativo, o un gioco per esprimersi, e si scopre che potrebbe essere venduto a un'azienda, rispondere a un bando, entrare perfettamente nel catalogo di un editore estero "che non conoscevate fino a ieri", o di un editore italiano conosciuto in fiera. Quello che interessa a Beatrice è che gli studenti abbiano chiari **gli interessi del mondo a cui parlano**, quando hanno un gioco in progettazione o già fatto. Se si è consapevoli degli interessi degli altri e del proprio obiettivo personale, "diventa molto più facile il dialogo". Riuscire poi a vendere e "imbroccare le cose giuste" è un altro tema; ma questi due elementi vanno avuti **prima**.

Approfondimento: il perché del designer. Il tema dell'obiettivo personale ha un parallelo in *The Art of Game Design* di Jesse Schell, che apre il libro non dal gioco ma dal designer stesso ("in the beginning, there is the designer"), e più avanti dedica capitoli interi al rapporto con il *cliente* e al *pitch*, cioè proprio al "dialogo" con gli interessi altrui di cui parla Beatrice. Nella lezione, però, la distinzione tra *obiettivo del gioco* e *obiettivo personale* ha soprattutto una funzione pratica: è il primo argomento della chiacchierata iniziale del design sprint.

2.7.9 Il design sprint

"Ora sono andata un po' veloce, e sono comunque lenta rispetto agli orari": Beatrice passa all'esercizio. Condivide un altro file ("che voi vedete bruttarello qui e che poi vi caricherò"); chi non lo legge può mandarle "un'emoji triste o buffa". Ha preparato **due moduli**; quello di adesso è su **come fare una sessione di design sprint**.

Che cos'è. Il design sprint si può usare per N cose: "dal decidere come arredare casa a come risolvere il problema di non avere abbastanza soldi per fare la spesa", per qualunque problema. **Non è uno strumento nato per il game design**: è nato per ragionare su problemi di progettazione e cercare soluzioni di design. Beatrice lo ha **adattato all'uso sui giochi**, e lo ha diviso in **fasi**, perché sono le fasi che invita i gruppi a provare a seguire. Il file verrà caricato nella cartella condivisa degli strumenti. Oggi racconterà la prima fase, ma la salterà, e si partirà dalla seconda.

Un invito, non un obbligo. È "uno strumento in più che vi dà una direzione in caso la direzione non riusciate a trovarla in breve tempo". Beatrice lo dice apertamente: l'anno scorso alcuni gruppi hanno investito molto tempo in alcune fasi della progettazione e non sono riusciti a concludere il gioco. Quest'anno si vuole evitarlo, e per questo ci sono **due strumenti**: questo, e quello che si vedrà alla fine di questa lezione e all'inizio della prossima, cioè tutte le voci che compongono la progettazione di un gioco, per avere una **visione d'insieme** e poter dire: "abbiamo queste settimane prima del pitch e questi mesi prima della consegna". Ognuno ha un lavoro, una famiglia, dorme, mangia, fa cose che occupano tempo: bisogna farsi "dei conti pratici", un tot di ore per una parte e un tot per un'altra.

Il project management che non c'è. Tutto questo fa parte del **project management** di un progetto, e Beatrice avverte che nel mondo del lavoro, "nel gaming perlomeno", **non esiste**: nella slide in cui sono state presentate tutte le professioni non c'è un *project manager* dei giochi, perché le aziende sono "ancora un po' in costruzione" su questo lato; quelle italiane almeno, mentre all'estero ci sono casi molto diversi, e nel videogioco funziona in un altro modo. Questi strumenti vanno conosciuti per essere più consapevoli di **come si funziona come persone**, preparandosi al fatto che il mondo esterno non sempre segue logiche di organizzazione e "più spesso bisogna un po' navigare le onde". Per il corso, però, sono molto utili: quando il gruppo "per un attimo si perde", si può tornare a fare il punto con questi strumenti.

2.7.9.1 Le fasi del file

Beatrice scorre i "titoloni" del documento (figura 2.7.3).

1. **La chiacchierata iniziale**, che oggi si salta (si veda sotto).
2. **Definire l'obiettivo di gruppo** del gioco: proposte e votazioni.
3. Una piccola sessione di **ricerca e sviluppo** basata sull'obiettivo.
4. Una prima sessione di **brainstorming** su quello che emerge, per rispondere all'obiettivo.
5. La **valutazione** di tutto il brainstorming: che cosa si tiene e che cosa si butta, di tutte le idee uscite.
6. Da lì in poi, l'**assegnazione dei ruoli**: chi fa cosa, con che tempi.

Oggi si faranno le fasi dalla seconda in poi, fingendo che la classe sia un unico grande gruppo di **nove persone** che inventa un gioco: "faremo tutto quello che serve per fare un brainstorming per avere il pitch del gioco". Sarà un gioco "del gruppo tutto", quindi non lo si potrà portare a conclusione nel corso: serve ad allenarsi con lo strumento.

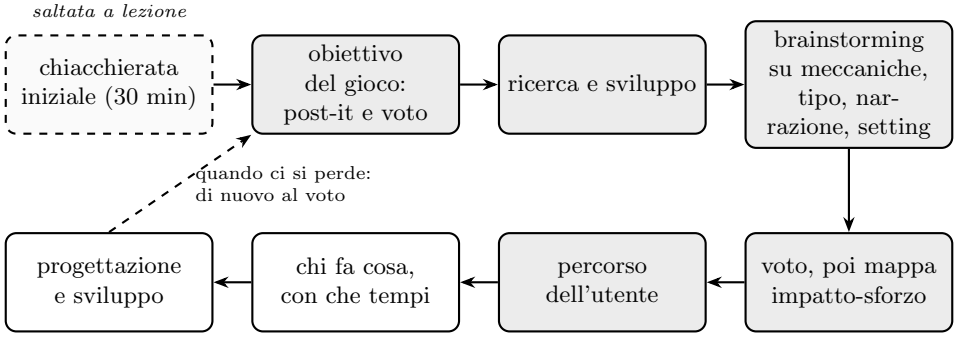


Figure 2.7.3: Le fasi del design sprint adattato da Beatrice per il corso. In grigio le fasi toccate a lezione (alcune solo raccontate per mancanza di tempo); la chiacchierata iniziale è saltata. La freccia tratteggiata riassume il consiglio "quando vi perdete, andate a votazione". Ricostruzione schematica del file.

Una scaletta per il corso, non il design sprint. Beatrice lo ripeterà più avanti, a Nicola: la scaletta **non è una fase standard**, "non voglio che vi portiate a casa che il design sprint è tutto così, sempre, a queste fasi". Prende delle metodologie e le porta in una scaletta pensata per il corso.

2.7.9.2 La chiacchierata iniziale

La prima fase è un **allineamento di gruppo**, e oggi si salta. Le domande:

- quali sono i nostri **obiettivi personali** (quelli visti prima)? "Magari il mio obiettivo in questo corso è provare a sperimentare. Oppure Lorenzo vuole fare un gioco per provare a venderlo da qualche parte, oppure io voglio fare un gioco per raccontare la storia della mia famiglia". Allinearsi non vuol dire che sarà quello che si porterà a casa, ma che si conoscono i desiderata degli altri e c'è uno spazio per raccontare i propri;

- **che competenze ho**? A quali progetti ho già lavorato, e che cosa facevo in quei progetti?

- la "domandona": in questo corso voglio lavorare sulle **stesse competenze** su cui ho lavorato in passato, o ci sono attività che non ho ancora esplorato, che non ho mai fatto e vorrei esplorare qui? "Non ho mai scritto testi, ma qua mi andrebbe di provare", "non ho mai fatto illustrazioni e qua mi andrebbe di vedere": qui si può fare, e conviene capire in che ruolo mettere le proprie competenze. (Beatrice corregge anche un refuso nel file, in diretta.)

Per gruppi di tre persone, **30 minuti** bastano e avanzano: uno scambio in cui "ognuno sa la situazione delle altre persone". Beatrice invita a farla quando inizierà il lavoro di gruppo.

Approfondimento: lo Sprint di Google Ventures. Il *design sprint* nella forma più nota è quello descritto da Jake Knapp, con John Zeratsky e Braden Kowitz, nel libro *Sprint* (2016), nato a Google Ventures: cinque giorni per passare da un problema a un prototipo testato con utenti reali, con passaggi molto simili a quelli della lezione (mappare il problema, raccogliere ispirazioni, proporre soluzioni individualmente e in silenzio, votare con adesivi a pallino, decidere). L'adattamento di Beatrice ne conserva il cuore (lavoro individuale silenzioso, presentazione, voto a pallini, tempi rigidi) ma lo piega ai tempi e agli scopi di un corso: niente giornate intere, e la prima fase dedicata alle persone del gruppo, non al problema.

2.7.10 La lavagna di Canva

"Ora vi condivido questa fantastica lavagna". Per il *brainstorming* ci sono "tantissimi" strumenti e modalità; oggi si usa il design sprint come modalità e una **lavagna di Canva** come strumento.

Entrare. Il link nella chat non è cliccabile, né si riesce a copiarlo ("nemmeno copia e incolla"). Qualcuno lo copia a mano nella barra del browser e riesce a entrare; un altro lo mette su Discord, dove "funziona", anche se a qualcuno risulta che dal link di Discord non ha l'autorizzazione ("essendo che il link l'ho copiato io, magari non ci sono i permessi"). Alla fine Beatrice crea un link accorciato, "più dettabile" (`urli.it/31GRXQ`, secondo la trascrizione).

Gli strumenti in generale. Canva è uno strumento per sessioni di *brainstorming* simile a molti altri; un altro è **Miro**, "ma ce ne sono davvero tanti". Beatrice usa Canva perché è il più comodo per lei. L'impostazione della lavagna **simula un incontro dal vivo**: le attività di oggi si possono fare in digitale, a distanza, con uno strumento di questo tipo, oppure dal vivo con **carta e penna**, un tavolo dove appoggiare i bigliettini, un muro dove attaccare i post-it. "Ve lo dico perché può essere utile anche per il futuro".

Come si usa. Beatrice ha "loccato" (bloccato) gli elementi della lavagna, così che nessuno sposti per errore le cornici; si può selezionare e zoomare liberamente (avvicinandosi le lavagne "diventano più grandi"), ma bisogna stare attenti a non cancellare il lavoro degli altri. Per cancellare: si seleziona l'elemento e si preme il tasto di cancellazione, oppure il piccolo cestino nel menu dell'elemento. Gli strumenti utili sono nel **menu a sinistra** di Canva, che ha sette voci (Canva AI, Modelli, Elementi, Testo, Brand, Caricamenti, **Strumenti**); sotto **Strumenti** (l'icona con una "pennina") ci sono, tra gli altri, il **testo** e i **foglietti adesivi**: "veri e propri post-it, come quelli che usereste dal vivo". Si sceglie un colore, il post-it compare, lo si ingrandisce o rimpicciolisce e ci si scrive sopra.

Le prove. La prova serve anche ad allinearsi sullo strumento, ed è movimentata. I post-it compaiono uno dopo l'altro ("Giordano, anche Lorenzo", "vedo una C", "Rita sta scrivendo"); Gianmarco entra senza il suo account e compare come "Leone Marino", uno dei nomi di animale che lo strumento assegna agli utenti anonimi[6]. Si scopre che se la lavagna di base è bloccata non si riesce a scrivere nei post-it ("io l'ho imparato adesso, perché credevo non funzionasse così"), e Beatrice sblocca. Qualcuno sposta per sbaglio l'intera cornice ("oh mio Dio, sono uscito, e qualcuno ha spostato tutta la lavagna"), qualcuno ha post-it finiti fuori dalla lavagna o giganteschi, una studentessa ("una grafica fissata con l'impaginazione") prova a riallineare tutto. Angelo, che è collegato **dal cellulare**, non riesce a trascinare il post-it ("non me lo fa trascinare"); Beatrice lo guida passo per passo e alla fine decide di creargli lei dei post-it con il proprio nome sopra, che Angelo userà ("ci ricorderemo che i tuoi post-it sono quelli con il nome di Beatrice"), anche se poco dopo Angelo riesce a crearne uno da solo ("ce l'ho fatta"). "Dal telefono è anche più complicato, è normalissimo". Beatrice avverte che **Miro è un po' più intuitivo** di Canva, e che servirà molto spazio: il gioco di fare le cose piccole e zoomare sarà utile soprattutto nella ricerca e sviluppo.

Lo strumento conta il giusto. "Di tutto questo, lo strumento ci interessa il giusto": si può usare Miro, si possono scrivere le cose a mano. **Ci interessa il procedimento**.

2.7.11 Prima fase: che obiettivo vorrei dare a questo gioco?

La domanda della prima fase è **"che obiettivo vorrei dare a questo gioco?"**. Le regole:

- ognuno scrive **al massimo tre risposte**, e almeno una;
- **una risposta per post-it**;

[6] La trascrizione dice "è come l'anonimo di Giuseppe": con ogni probabilità il riferimento è agli "animali anonimi" con cui Google e altri servizi mostrano chi partecipa senza account.

- non è il genere del gioco e non sono le meccaniche: è "più a volo d'aquila", quello che si vorrebbe che il gioco ottenesse; può riguardare anche le **emozioni**, cioè che cosa far provare alla persona che gioca.

Gli esempi del file mescolano apposta livelli diversi: "un gioco formativo per insegnare l'inglese", "un gioco con dei pinguini blu", "un GDR per sensibilizzare sulla lotta di classe", "un gioco fantasy sulla vendetta", "un gioco con delle pedine gigantesche", "un gioco che faccia esplorare un quartiere".

I timer. E qui la cosa da ricordare di tutto il procedimento: **mettere i timer**. Nei processi creativi di gruppo, e probabilmente lo si è già visto succedere, ci si fa prendere da quello che sta avvenendo, dai dialoghi, dalle idee che emergono. Beatrice richiama la lezione precedente su **divergenza e convergenza**: c'è un momento in cui si apre, si esplora, si buttano idee; ma "a un certo punto bisogna **quagliare**". Darsi un tempo di orologio dice fino a quando si esplora; quando suona il timer, bisogna riportare in ordine la situazione. "È la cosa importante di tutto il processo: dare dei tempi e provare a rispettarli". Il primo timer è di **5 minuti** ("dai 5 ai 10").

Gianmarco arriva. Gianmarco, "arrivato un po' in corsa", chiede il contesto: si può buttare qualsiasi idea? È un esercizio, risponde Beatrice, per imparare a fare *brainstorming* e poi mettere in ordine le idee: "vai pure libero".

La parola proibita. Mentre scrivono, Rita avverte i compagni: qualcuno ha scritto *divertire* su un post-it, e "dovremmo evitare di usare questo verbo"[7], "conviene modificarlo prima che lo veda qualcuno" (Ruggero, si capisce). Beatrice: "Io non dirò niente, però è un ottimo consiglio". Del resto, aggiunge, sono idee che non verranno condivise con gli altri docenti: è un esercizio per imparare la procedura, che non è neanche detto si userà esattamente così. Le idee di oggi possono avere vita propria e crescere, "ma non in questo corso": i gruppi del progetto partiranno da zero.

2.7.12 Il racconto delle idee

Prima della votazione, che Beatrice anticipa (si veda la sezione 2.7.13), **ognuno racconta che cosa ha scritto** e dice **dove sono** i suoi post-it, così che tutti possano trovarli sulla lavagna. In questa fase valgono due regole:

- le altre persone possono fare **solo domande**: chiarimenti su che cosa si intende, su perché è emersa un'idea; "tutte le domande che vi vengono in mente vanno benissimo", anzi Beatrice invita a farle, perché "c'è ancora tanto che può uscire";
- **niente commenti**, "né nient'altro". Se non si hanno domande, non si dice nulla.

E Beatrice si fa da parte: "fate finta che io non esista". Ricorderà solo di dire dove sono i post-it; il gruppo deve autogestirsi, come in un vero lavoro di gruppo in cui nessuno dà la direzione. Poi si accorge di aver dimenticato il timer anche qui: tutte le idee vanno raccontate in circa **15 minuti**, "tenetene conto quando fate le domande, quando rispondete, quando raccontate".

2.7.12.1 Rita

Rita (post-it arancione, rosso e giallo, sovrapposti nell'angolo in alto a sinistra) ha scritto tre idee.

Un gioco educativo. Si è avvicinata a questo mondo con l'idea di dare qualcosa in più alle nuove generazioni: è quello che l'ha spinta a diventare game designer.

Un gioco che costringe le persone a collaborare. Quando la collaborazione è necessaria per raggiungere un obiettivo, bisogna affrontare problemi con se stessi e con gli altri, e questo è costruttivo e stimolante per tutti. È anche il suo approccio alla vita: "condividere, collaborare per imparare".

[7] Il riferimento è alla regola, ricorrente nel corso, per cui "far divertire" non è un obiettivo di design: è troppo generico per orientare le scelte.

Un gioco sull'arte. È la sua più grande passione: avvicinare le persone a un'arte "non proiettata verso l'IA".

Le domande. Il gioco sull'arte è sulla storia dell'arte, i grandi pittori, o un gioco in cui **le persone fanno gli artisti**? La seconda: avvicinarle alla disciplina in maniera attiva; e in effetti, osserva Rita, potrebbe rientrare nel gioco educativo, con le scuole. Il gioco educativo, per che età e in che contesto? Non ci ha ancora pensato nello specifico, ma pensava a bambini che stanno crescendo, elementari e medie: per gli adolescenti ci vuole qualcosa di più attivo e stimolante a livello emotivo, forse un LARP. Poi chiede lei un parere a una compagna con esperienza di questi giochi: ci sono fasce d'età più interessate a certi tipi di gioco? "Dipende", risponde la compagna: si sente il gruppo, e se il gioco è educativo lo si orienta anche, per esempio decidendo se farlo competitivo o no. A lei è sempre piaciuto **mescolare competizione e collaborazione**: le **cacce al tesoro** sono notoriamente competitive, ma le sue richiedevano sempre una collaborazione tra i gruppi, e alla fine **il tesoro era per tutti**: c'era la soddisfazione di chi era arrivato primo, ma il premio era di tutti. "Bello, e stimolante, perché si deve collaborare per raggiungere un obiettivo".

2.7.12.2 Nicola arriva

A questo punto entra Nicola. Beatrice gli spiega in breve che cosa sta succedendo: un esercizio di design sprint, "una metodologia, anzi varie metodologie che riguardano tante forme di design, non solo quello dei giochi", per avere una direzione quando un gruppo si perde o non sa come allinearsi; la prima fase è rispondere a "che obiettivo vogliamo dare al gioco"; i compagni hanno dato tre idee ciascuno, che ora si leggono e poi si votano. E gli dice il suo gruppo: "sei nel gruppo 1, con Giordano e Rita", "non per motivi", lo rivedrà nelle slide che Daniele caricherà. Nicola entra nella lavagna come "Nick".

Poco dopo, Nicola chiede "un quick check-up": si sente "in alto mare", "una zavorra". I post-it sono già divisi in gruppi? Sono concept di personaggio o di gioco, di ogni singola persona? Si stanno comparando? Beatrice riconosce di non essersi spiegata abbastanza e rispiega: si sta **testando la metodologia**, fingendo di essere un unico gruppo di nove da cui deve uscire "una bozza di gioco"; la scaletta non è standard ma costruita per il corso; per decidere l'obiettivo del gioco ognuno butta giù idee in silenzio sui post-it, poi le si leggono ("una fase di brainstorming aperto") e infine le si vota.

Nicola chiede com'è il voto: qualitativo o quantitativo? Si vota sulla quantità di contenuto, sulla tipologia, "un voto vale più di un altro"? Nessun voto vale più di un altro, risponde Beatrice: ognuno avrà **sei voti**, "immagina di avere sei pallini adesivi", e può metterli anche tutti sulla stessa idea. A Nicola lo schema ricorda quello studiato in **organizzazione aziendale**, quando si avvia un progetto; Beatrice conferma che il design sprint si usa in moltissimi contesti, "molto sulle soluzioni da proporre", anche in sociologia. Nicola chiede ancora se c'è stata una spiegazione prima del lavoro: c'è stata una piccola introduzione sui motivi per cui si fa design (recuperabile dalle slide o dalla registrazione), e poi ci si è buttati sulla domanda. Infine: ci sono linee guida, "o siamo randomici"? "Per ora è tutto libero, rimaniamo leggerissimi". Nicola si scusa con la classe per il tempo rubato.

2.7.12.3 Lorenzo Vergata

Lorenzo Vergata (post-it sopra quelli di Rita) presenta tre idee, mentre "qualcuno ha mosso la lavagna" e per un momento "è scomparso tutto".

I funghi. Esplorazione del bosco per sensibilizzare sulle specie fungine e sul rispetto della natura, "semplicemente perché siamo in periodo di funghi": la prima cosa che gli è venuta in mente.

Frenetismo. Partite rapide, "quasi caotiche dall'esterno, ma pseudo-organizzate dall'interno fra i giocatori". Si è chiesto che cosa cerca, a 26 anni, in un gioco: non partite lunghe, ma frenesia. Frenesia, quindi, "per gente neolaureata e simile".

Paura. Nascondino; fuggire da qualcuno all'aperto, senza luci; o in casa, staccando la corrente all'inizio del gioco. "Il gioco deve far paura": senza aver fatto ricerca, pensa che non ci siano molti giochi che sfruttano la paura dei giocatori come dinamica.

Le domande, e qualche commento che sfugge alla regola. L'idea della paura è pensata come un **LARP**, all'esterno, o come un gioco da tavolo? Lorenzo pensa che il modo più semplice di creare paura sia un LARP, ma "nulla toglie" di farlo da tavolo, studiando come un gioco da tavolo possa incutere timore. Una compagna osserva che la paura si definisce in tanti modi: **pressione**, **ansia** (più facili da tavolo) e **jumpscare** (più da LARP); bisognerebbe dare una dimensione al concetto, ma è "una bella sfida". Le ricorda quando da ragazzina giocava a **guardie e ladri** al mare, di notte, nelle case degli amici: si creava un clima di paura e tensione, anche perché ci si facevano gli scherzi.

Un altro studente, che gioca "per la maggior parte del tempo" a giochi di ruolo da tavolo[8], conferma che la paura è "un fantastico strumento", ma pone un problema: se diventa un **gioco pervasivo**, c'è un **danno collaterale**, cioè **spaventare chi non gioca**. "Immagino una persona che corre a mezzanotte in giro per Roma urlando di paura": terrificante per chi la vede, e si coinvolgono persone che non vorrebbero provare quella paura. Lorenzo concorda: bisognerebbe tarare, andando più verso quello che diceva la compagna, cioè il momento in cui sei nascosto e senti che chi ti cerca è "letteralmente a fianco a te", e ti pervadono paura e ansia. Ma "basta poco" per tirare in mezzo estranei: chi vede una persona che ne insegue un'altra che cerca di nascondersi "chiama la polizia al minuto zero". Se sono bambini di sei anni, "stanno giocando"; se vedi un ventenne, "che cosa sta accadendo?".

Infine l'esplorazione del bosco: come sarebbe, come gioco? Una guida che ti porta nel bosco? "Fatta sul posto è tosta": sarebbe semplicemente una gita con un **micologo**. Ma **da tavolo** si potrebbe fare. Sul fronte della natura si possono inserire meccaniche che **simulano il comportamento dei funghi**: sotto di loro c'è il **micelio**, che fa da rete di comunicazione tra le piante, e ne potrebbe uscire un gioco "pseudostrategico". Sulla **commestibilità** si può immaginare uno scontro carta contro carta, una specie di sasso-carta-forbice: "il fungo buono te lo annullo con questo cattivo", mentre certi funghi che "sembrano buoni" rovinano tutto il raccolto, cioè una sola carta può rovinare molte carte di funghi buoni. "Potrebbe anche essere frenetico, chissà".

2.7.12.4 Tancredi

Tancredi ha scritto in modo "molto più riassunto".

Un gioco cooperativo. È "un grande fan dei cooperativi", anche piccoli, come *Hanabi*[9], *Sky Team*, *Quicksand*: gli piace molto la meccanica della cooperazione.

Un gioco per il terzo settore. Riprende la categoria vista prima, ma senza grandi idee: per questo tipo di gioco sarebbe importante capire **per chi** lo si fa, e non sa quanto sia realizzabile nel loro caso.

Un GDR. Non conosce praticamente per niente i giochi di ruolo, li ha scoperti in queste lezioni: "perché no?", provare a sperimentare quel settore.

Che cosa aveva immaginato per il terzo settore? Secondo Tancredi un modo sarebbe cercare delle realtà, anche solo su internet, sui loro siti, e capire che cosa potrebbe essere interessante per loro, "anche se poi non glielo si propone veramente, perché questa è un'esercitazione"; con l'obiettivo di dare una mano, diffondere quello che fanno. Un compagno racconta (più un commento che una domanda) di aver giocato *Nemesis: Retaliation* dal vivo nella modalità semicooperativa, con i traditori, e ora online su **Board Game Arena**[10] con più o meno gli stessi amici, in modalità cooperativa, usando la chat: "stratosferico, così mi diverto". Se invece "a un certo punto uno ti dà una ginocchiata sui denti"... Tancredi: in associazione "quasi mai nessuno vuole fare cooperativi", e lui è sempre lì; gli piacciono anche i competitivi, ma certi giochi "vale la pena giocarli così".

2.7.12.5 Tiziana

Tiziana (post-it sotto quelli di Tancredi) ha scritto tre idee.

[8] Dal contesto, con ogni probabilità Nicola, che poco dopo si presenta come "un grande giocatore di TTRPG".
[9] La trascrizione dice "Anabis".
[10] La trascrizione scrive "BGA", la sigla abituale del sito.

Un board game, di costruzione, gestionale ambientato. Le piacerebbe fare esperienza di costruzione di un gioco da tavolo e non saprebbe da che parte cominciare (come Tancredi con i giochi di ruolo: "non ho idea, quindi mi piacerebbe provare"). Un **gestionale** perché le piace l'aspetto gestionale, ma **ambientato**, "non so se è una chimera".

Un'escape room per far divertire. "Lo so che non devo usare divertire, ma non sono riuscita a trovare nessun'altra cosa", e soprattutto far divertire lei mentre la costruisce. E far nascere la **collaborazione**. Dal vivo o da tavolo, entrambi vanno bene; il tema non lo sa. Le piacciono gli **indovinelli**, trovare soluzioni; ma vorrebbe enigmi **misti**, non tutti dello stesso tipo, perché quando sono tutti uguali diventano "o troppo cervellotici o ripetitivi".

Un gioco interattivo per far conoscere la città. Una parte di essa, perché Roma "è abbastanza gigantesca"; l'aveva immaginato coinvolgendo perfino i **passanti**, "ma non so se sia fattibile".

Tancredi chiede: quando parli di gestionale, hai un riferimento? Un gioco economico, con competizione e interazione molto alte, o qualcosa di meno interattivo, "più German"? Tiziana non lo sa: da giocatrice è più "mi coltivo il mio orticello" (gli economici interattivi "non sono i miei preferiti"), ma potrebbe essere anche interattivo: intendeva la **gestione delle risorse** in generale. La cosa fondamentale è che sia ambientato, "che senti di essere in quel posto". Un compagno le suggerisce *Akropolis*, per l'ambientazione e per l'idea di costruire la propria città con tessere di tre esagoni sovrapposte, in cui i colori che restano in cima diventano i quartieri: un approccio per non renderlo "troppo gestionale German", magari con una città costruita tutti insieme. Tiziana non lo conosce. Beatrice interviene: si possono fare domande, "ma non altre cose", altrimenti si usa troppo tempo; e manca un minuto. Tancredi: "domanda sbagliata, scusate". "Va tutto bene, siamo qui apposta per capire le dinamiche".

Il timer che scade. Manca un minuto e mancano ancora cinque persone. Beatrice concede **altri dieci minuti**, "ma dentro questi dieci minuti dovete starci": alla fine si passa allo step successivo, "quello che è stato detto è stato detto". Qualcuno propone due minuti a testa; "se volete, sì, però non posso dirvelo io": serve ad allenare il gruppo.

2.7.12.6 Angelo

Angelo, colto un po' di sorpresa ("ci ho messo un'eternità a scriverlo, abbiate pazienza"), ha i post-it in basso a sinistra.

Un gioco di ruolo basato su un'app. Così che la **mappa** possa essere continuamente aggiornata.

Un board game con possibili espansioni. Qualcosa "tipo *Magic*, ma non per forza così evoluto": ogni tanto si aggiungono carte che modificano la strategia.

Il tema è variabile, ma più fantasy o fantascienza che realismo; non aveva pensato a un'emozione particolare. Qualcuno trova interessante il board game "aggiornabile". E che cosa intende per mondo condiviso? Qualcosa come i giochi online divisi per zone (Europa dell'Est, Europa dell'Ovest), ma molto più ristretto: si parte con un'avventura in un *medieval fantasy*, e qualcuno nella tua zona pensa a un'isola con ambientazione *steampunk* e la aggiunge al mondo, così che altri possano andarci e "ruolare" in un mondo steampunk. "Una sorta di *sandbox*": ognuno parte dal suo stile e ci si trova in posti sempre diversi, senza un paesaggio ripetitivo.

2.7.12.7 Gianmarco

Gianmarco (in basso a destra) presenta tre idee.

Un gioco da tavolo per bambini dai 6 agli 8 anni. Che li spinga a **raccontare**, incentrato su **esplorazione e scoperta**: è la fase in cui i bambini iniziano a staccarsi un po' dai genitori, e nell'esplorare provano "quel misto di curiosità e paura". Con elementi grafici, un po' di casualità; una parte di sistema per sviluppare la **logica** e una parte di immagini per l'**immaginazione**.

Un board game narrativo sulla guerra. Con le meccaniche di *Mage Knight*[11]: "l'ho veramente buttato a caso, è una cosa senza senso".

Un'escape room basata sul tempo. Si deve uscire dall'ufficio, ma il tempo non scorre mai: bisogna risolvere enigmi per far scattare **le 17** e uscire finalmente.

Un compagno ricorda una scena dei *Simpson* in cui Bart guarda continuamente l'orologio della classe, che a un certo punto torna indietro; e suggerisce una meccanica del tempo non in tempo reale ma a eventi, "cose che si innescano dopo tot minuti" ("so che tra tre minuti mi bolle l'acqua e succede qualcosa"). Un altro nota che è bella "la cosa al rovescio": di solito il timer si accorcia e ti esaurisce, qui devi farlo andare avanti. Qualcuno trova curiosa l'idea centrale e chiede perché le meccaniche di *Mage Knight* dovrebbero rappresentare l'orrore della guerra. Il motivo è semplice: è un periodo di guerra, a Gianmarco piace *Mage Knight* e gli piacciono i board game narrativi, "ho messo tutto insieme, ho fatto un po' un pastiche".

2.7.12.8 Nicola

Mancano tre minuti e mezzo per tre persone: "un minuto a testa". Nicola va "rapidissimo".

Un gioco di ruolo sui tempi morti: la cucina. È un grande giocatore di TTRPG, e ha notato che i giochi di ruolo da tavolo hanno regole ben stabilite ma quasi nessuno ha meccaniche sui **tempi morti**: in un fantasy alla *D&D*, quando ci si accampa "è sempre molto randomico". Gli interesserebbe un GDR basato sulla **cucina**: restare sazi per andare avanti nelle missioni, arrangiarsi con quello che si trova; ispirandosi a *Dungeon Food*[12], dove si mangiano i mostri per andare avanti, perché non sempre si hanno le provviste.

Un gioco strategico con bluff e ruoli nascosti. Da giocatore di ruolo, perché non provare uno strategico da tavolo? Lo baserebbe sulla deduzione, sul **bluff** e sui **ruoli nascosti**. Su una gigantesca plancia ognuno parte senza nazione e, ispirandosi al videogioco strategico *Humankind*, ne sblocca: Roma, che vince **conquistando**, e allora la tua strategia diventa conquistare; più avanti gli Egizi, che vincono **costruendo opere**. L'obiettivo è non far capire **in che modo vuoi vincere**: quale nazione, o quale somma di nazioni, sei, e quali sono i tuoi metodi di vittoria.

Un LARP sul cambiamento visibile dei personaggi. Non ne sa niente, ma gli interessa: persone vestite di bianco, "come dei camici bianchi", senza emozioni. Chi si sporca di **rosso** diventa più nervoso, rabbioso, isterico; di **blu**, più triste; di rosso e blu, arrabbiato e triste; di **verde**, prova anche disgusto. Una **somma di emozioni**, con obiettivi che costringono a sporcarsi: "devi prendere la chiave segreta dentro il secchio rosso; ora hai la mano e mezza manica sporche di rosso, sei anche rabbioso".

2.7.12.9 L'altro Lorenzo

L'altro Lorenzo non è sicuro di aver seguito bene l'obiettivo: forse è stato "troppo personale, troppo generico". Ha scritto: **esprimere me stesso**; **raccontare il mio mondo fantasy attraverso un GDR**; **esplorare tipologie di gioco nuove**, almeno per lui, come l'escape room o il LARP. Non ha in mente cose specifiche. Qualcuno gli chiede se ne aveva in mente parlando del suo gioco: sì, ma era un pensiero personale, e in un lavoro di gruppo "non è che si può imporre per forza il proprio mondo scritto".

2.7.12.10 Giordano

Il timer è scaduto, ma Beatrice concede a Giordano, l'ultimo, una breve presentazione (all'inizio non si sente). Giordano non ha puntato su un gioco specifico ma su **aspetti da inserire** in un gioco. Il primo è l'**accessibilità**: "non vorrei che qualcuno potesse mai dire *non posso giocare a questo gioco*; al massimo, *non mi interessa*". Il secondo è la **creatività**: di chi gioca, precisa a una domanda, non di chi progetta.

[11] La trascrizione dice "Image Night" e "Mage Night".

[12] Così nella trascrizione. Si tratta con ogni probabilità di *Dungeon Meshi* (in italiano *Dungeon Food*), manga e anime in cui gli avventurieri cucinano e mangiano i mostri del dungeon.

Chi	Le idee sui post-it
Rita	gioco educativo (elementari e medie); gioco che costringe a collaborare; gioco sull'arte, in cui si fa gli artisti
Lorenzo Vergata	esplorazione del bosco per sensibilizzare sulle specie fungine e sul rispetto della natura; frenesia (partite rapide, caotiche fuori e organizzate dentro); paura (nascondino, fuga al buio)
Tancredi	gioco cooperativo; gioco per il terzo settore; GDR
Tiziana	board game gestionale ambientato; escape room con enigmi misti e collaborazione; gioco interattivo per far conoscere una parte di Roma
Angelo	GDR con app e mappa condivisa aggiornabile; board game con espansioni che cambiano la strategia
Gianmarco	gioco per bambini di 6–8 anni su esplorazione e scoperta; board game narrativo sulla guerra con le meccaniche di *Mage Knight*; escape room in cui bisogna far arrivare le 17
Nicola	GDR sulla cucina e i tempi morti; strategico con bluff e nazioni nascoste; LARP delle emozioni a colori
l'altro Lorenzo	esprimere se stesso; il proprio mondo fantasy in un GDR; esplorare escape room e LARP
Giordano	accessibilità; creatività di chi gioca

Table 2.7.4: Le idee della prima fase dell'esercizio, persona per persona.

2.7.13 La votazione a pallini

Beatrice blocca la lavagna (i post-it non si possono più spostare, ma ci si può scrivere sopra), allarga un po' lo spazio dei post-it di Rita, e spiega prima l'attività in sé, poi come farla su Canva.

Le regole. Ognuno ha **sei pallini**: "immaginatevi proprio di avere dei pallini adesivi".

- Si mettono sulle idee che piacciono di più, cioè quelle **che si vorrebbero produrre**, su cui si vorrebbe lavorare.
- Si possono mettere **più pallini sulla stessa idea**, anche tutti e sei.
- Si può **votare la propria idea**.
- **Non si deve spiegare** perché si vota.
- Si devono **usare tutti e sei**: non si può non votare o metterne meno.

Alla fine restano solo i post-it con più pallini. Se uno ne ha più degli altri, è l'idea su cui si lavora; se ce ne sono due o tre in testa, **si rivota** tra quelli.

Su Canva. Beatrice propone, "brainstormando sempre, quindi non so se è l'idea migliore", di usare i **pennarelli** di Canva (ancora in Strumenti), ognuno con un colore. Seguono le prove fuori dalla lavagna (Beatrice disegna "un cuore gigantesco viola") e un lungo appello dei colori, complicato dal fatto che il colore del cursore di qualcuno non coincide con quello del suo pennarello e che compaiono più "soli" disegnati per prova[13]. Nessuno è "morto" finora ("vivi, vive"), e dopo la votazione ci sarà una pausa. Timer di **5 minuti**.

Il timer di Canva. Durante il voto Rita scopre che Canva ha un **timer integrato**, in basso, e lo attiva ("timer gigante"): da qui in poi lo si usa. "Questa è una cosa che era molto utile", dice Beatrice: aveva provato a metterlo in altre forme. Chi ha finito i pallini mette il mouse "nel parcheggio per i mouse". Già durante il voto Beatrice nota: "i funghi hanno un sacco di successo, non me lo immaginavo".

[13] L'assegnazione dei colori, per quanto ricostruibile: Giordano arancione, Nicola verde chiaro, Lorenzo Vergata "blu pastello", Tancredi blu, Gianmarco viola scuro, l'altro Lorenzo rosso, Tiziana nero, Angelo azzurro (assegnato da Beatrice), e una studentessa, probabilmente Rita, un giallino "che forse non si nota". Per i colori poco visibili Beatrice suggerisce pallini grossi accanto al post-it.

La conta, fatta dal gruppo. "Proprio perché ci stiamo allenando", Beatrice non conta: si muta e lascia che il gruppo si organizzi per capire quali post-it hanno più voti. Lo scopo è **selezionare i due post-it più votati**, cerchiandoli; non si controlla che tutti abbiano messo davvero sei pallini ("ci fidiamo"): con più tempo, e fuori da un esperimento, si conterebbero anche quelli. La conta è confusa, come ogni conta vera: ci sono "schiribizzi" di cui non si capisce il colore, un pallino finito per sbaglio che non è un voto, due arancioni che sembrano un arancione e un rosso. Nicola spiega di aver messo due voti verdi sullo stesso post-it, "per votare più forte": è consentito. Alla fine ci sono un post-it con più voti e due **a pari merito** per il secondo e terzo posto. Vince **l'esplorazione del bosco per sensibilizzare sulle specie fungine**, che ha almeno un voto di quasi ogni colore. Manca solo il blu: è Tancredi, che si scusa ("non dirmi che ti devi scusare, ognuno vota quel che vuole"): è già arrivato ultimo in due giochi sui funghi, e comincia ad avere "un certo odio verso i miceti".

Il tema scelto. "Pare che sia stato scelto il tema del gioco: esplorazione del bosco per la sensibilizzazione sulle specie fungine e il rispetto della natura". Ci si lavorerà, "facciamo finta di andare a lavorare". Pausa di cinque minuti, dalle **19:10** alle **19:15**. Prima di sparire, Beatrice avverte Lorenzo Vergata che Tancredi gli ha scritto su Discord: i due, compagni di gruppo, si scambiano i contatti per fare un gruppo su Telegram ("preferisco tutta la vita Telegram").

Approfondimento: il voto a pallini. La tecnica che Beatrice usa è nota come *dot voting* (o *dotmocracy*) ed è parte integrante dello Sprint di Google Ventures, dove compare in due momenti: la *heat map*, in cui i pallini segnalano le parti interessanti delle proposte, e lo *straw poll*, in cui ognuno indica la soluzione preferita prima della decisione finale. Le sue virtù sono quelle che la lezione mette in scena: è rapida, non richiede di argomentare (e quindi non premia chi parla meglio o più forte), rende visibile il consenso. I suoi limiti si vedono anche loro: i voti multipli sulla stessa idea pesano più di quanto sembri, e la conta, se i pallini non sono chiari, diventa a sua volta una discussione.

2.7.14 Dopo la pausa: il problema dei tempi

Al rientro Beatrice chiede un segnale a tutti (pollicioni, un cuoricino che le era sfuggito) e, con la confusione dei due Lorenzo ("quale Lorenzo?", "Lorenzo, non tu"), nasce la proposta di chiamare l'altro Lorenzo "John". Poi fa una piccola autocritica, che è anche una lezione.

"Io ho il dubbio che forse non ci stiamo". Le fasi successive sono diverse, e il tempo è poco. "Non sto facendo bene il mio lavoro": ha lasciato che la classe si godesse il momento creativo ("che è carino, è briccicolino"), ma avrebbe dovuto "starvi più sotto" e ricordare che **ci sono dei tempi da rispettare**. Il tema dei tempi "viene molto, molto sottovalutato". Non lo dice per fare la severa, ma perché nei lavori di gruppo **serve che qualcuno lo faccia**: se non lo fa nessuno, si finisce per "*overworkare*", cioè fare tanto lavoro su cose non necessarie, e progettare "diventerà non bello". Da qui in poi sarà più ligia, e l'esperienza sarà "forse meno goduriosa", ma serve a vedere tutti i passaggi.

Fare pulizia. Primo gesto, "una buona pratica proprio da fare": Beatrice **copia il post-it vincente** e lo trascina in un'area della lavagna chiamata **tema selezionato**. Serve solo per fare pulizia, per avere chiaro che cosa è stato selezionato e che cosa no.

2.7.15 La ricerca e sviluppo sul tema

Sul tema bisogna costruire un gioco. Le fasi successive sono una prima sessione di **ricerca e sviluppo** e un primo *brainstorming* su come costruire il gioco. Restano **40 minuti**, "pochissimo": la ricerca avrà **10 minuti**, mentre nella versione che Beatrice aveva preparato erano 40.

Come detto nella lezione precedente, la ricerca e sviluppo **riguarda i giochi, ma non solo**: un film, una canzone, un quadro, che ricordano il tema e il **mood** che si vuole dare al gioco. In dieci minuti si cerca in internet, o si pensa a quali **esperienze, suoni, gusti, colori** portano a quel tema. "Più giochi troviamo, meglio è", ma partendo anche da cose diverse dai giochi si può arrivare a giochi utili per l'obiettivo.

Le cose trovate ci si appuntano dove si vuole: su un file, in una parte della lavagna, sulle note del telefono, su un foglietto. Tiziana chiede se nei dieci minuti successivi le si trasferisce sulla lavagna: no, nei dieci minuti successivi si **elencano e se ne parla** insieme; popolare la lavagna non è obbligatorio, perché "ho paura che vi rallenti dover copiare, incollare e scrivere". Timer fino alle **19:32**.

Il timer suona ("dovreste averlo sentito nelle vostre orecchie"), ma Zoom non condivide l'audio dello schermo, e Beatrice lo annuncia a voce. Visto il tempo, decide di tornare al file di lavoro e mostrare **tutto il procedimento** da lì in poi; poi, se resta tempo, si faranno le fasi in piccolo, "un minuto a testa". Chiede a tutti di tornare su Zoom.

Quando fare la ricerca. Alcuni hanno già messo sulla lavagna titoli e cose legate ai funghi e alle esperienze boschive. Questa è una fase **solo di ricerca**, per trovare ispirazioni. Non è detto che vada fatta subito dopo la scelta del tema: Beatrice l'ha messa qui perché nel percorso dell'esercizio **può sbloccare la creatività**. Se un gruppo "parte come una mina", con un sacco di idee su meccaniche, art, azioni, narrazioni e setting, la ricerca si fa in un secondo momento; **ma si fa**, perché è "molto, molto importante".

2.7.16 Le fasi successive del file

Il brainstorming su meccaniche, tipo, narrazione e setting. Subito dopo (o subito prima) della ricerca c'è il *brainstorming* su **meccaniche**, **tipo di gioco**, **narrazione** e **setting**. Il procedimento è identico a quello per il tema: ognuno butta giù post-it, ma **senza più il limite di tre**: tutti quelli che vuole, purché riguardino meccaniche, tipo di gioco, narrazione e setting che rispondano all'obiettivo scelto (oggi i funghi, il bosco, l'esplorazione; nei progetti del corso, l'obiettivo di ogni gruppo). Il file indica **10 minuti**, "ma valutate voi": i tempi sono calibrati per gruppi di tre. Poi di nuovo la **votazione con sei pallini**.

Il balletto dei cambiamenti. A questo punto "inizia la vera e propria progettazione". Si hanno idee che convincono più o meno; ma succederà di cambiare idea: sull'obiettivo, che non convince più come prima, o su qualcosa che, a progettazione iniziata, sembra meglio sostituire. Da qui in poi inizia la progettazione e lo sviluppo del gioco, ed è normale che sia "un po' un **balletto** di cambiamenti". Nel file questa parte non c'è, perché non ci sono vere strategie per gestirla, o meglio ce ne sono tante in vari contesti; ma per un progetto stretto nel tempo come questo il consiglio è uno: **quando vi perdete, andate a votazione**, con il sistema dei post-it, e per il resto esplorate dove vi porta la creatività.

Si impara facendo. "Il grosso tema della progettazione di giochi" (e non solo di giochi) è che **si impara a farli meglio più li si fa**. Attraversare questo percorso creativo è la parte importante del corso, e Beatrice invita a farlo "il più possibile, in tutti i contesti che potete": partecipare alle *jam*, inventare giochi propri, buttarsi in progetti. Al di là di tutti gli strumenti creativi, descrittivi e informativi, "l'unico modo è farli. Farne tanti", provare tanto e **testare** tanto: "la lezione sul playtest vi cambierà la vita, secondo me".

Dormirci su. Un buon modo di progettare è fare queste fasi e, **dopo ogni fase, dormirci su**: prendere decisioni, ma non considerarle definitive finché non ci si è dormito su e non le si è testate, elaborate, sperimentate. Il ciclo si ripete più volte: si cambia idea su una meccanica, forse sull'obiettivo, sull'emozione, sul target; "e a un certo punto avrete la quadra", cioè un gioco con un **percorso dell'utente**.

2.7.17 La mappa impatto-sforzo

"Scusate, stavo anticipando troppo. Ho saltato un testo importantissimo". Per darsi una direzione su quali meccaniche e dinamiche scegliere tra i post-it votati, dopo la sessione di *brainstorming* c'è una mappa "molto interessante": la **mappa impatto-sforzo** (figura 2.7.4). È un diagramma in cui si posizionano i post-it vincitori; Beatrice l'ha messo anche sulla lavagna di Canva.

L'esempio. Supponiamo che il voto abbia deciso che il gioco sui funghi prende la forma di un'**escape room**, che gli **enigmi sono solo su carta** e che i **disegni sono fatti a mano con il pennino stilografico**: tre post-it pieni di pallini. Ognuno va messo nel diagramma, che chiede due cose.

Impatto. Quanto quella scelta **ci porta vicino all'obiettivo**, cioè a fare un gioco sui funghi.

Sforzo. Quanto ci costa. Un'escape room richiede enigmi: un tot di ore di ricerca degli enigmi, un tot di costruzione, un tot di *testing*, un tot di grafica. Un **gioco di ruolo** richiederebbe altre ore: la narrativa, i personaggi, renderli giocanti, uno sviluppo emotivo di un certo tipo. Il **gioco dell'oca a tema funghi** ha uno sforzo diverso: ci porta forse meno vicino all'obiettivo, ma costa molto meno.

Lo sforzo si misura in **risorse**: tempo, competenze, energie, e anche **cose da produrre**. Se il gioco ha **100 carte**, quelle carte vanno create e poi **stampate**, con un costo economico di materiale; se si parte con l'idea di cento carte bisogna rendersi conto che servono ore, e "se non le abbiamo, non porteremo a casa le 100 carte".

A che cosa serve. "Questa mappa ci interessa non tanto per fare il gioco più figo del mondo, ma **per finirlo**, il gioco che vogliamo fare": arrivare a un gioco che si è in grado di fare, "che non ha mille carte", perché nei tempi non le si produrrebbe. Si devono fare le scelte più sensate in termini di come raggiungere l'obiettivo e di quanto tempo e risorse si possono investire. ("Mi rendo conto che non mi stanno uscendo le cose chiare come vorrei"; Tiziana scrive in chat che il diagramma è "utilissimo".)

Dove guardare. Si scelgono i post-it che finiscono **in alto a sinistra**: **alto impatto** (raggiungono meglio l'obiettivo) e **basso sforzo** (meno tempo, meno materiali, meno cose da produrre), la zona che Beatrice ha colorato di grigio. Sono quelli che bisogna far entrare in un percorso di gioco. (A un certo punto Beatrice dice "in basso a destra", uno studente chiede conferma, e lei corregge: "in alto a sinistra", "non mi stupirei di aver detto il contrario"; nel file, preparato "con più calma", è tutto chiaro.)

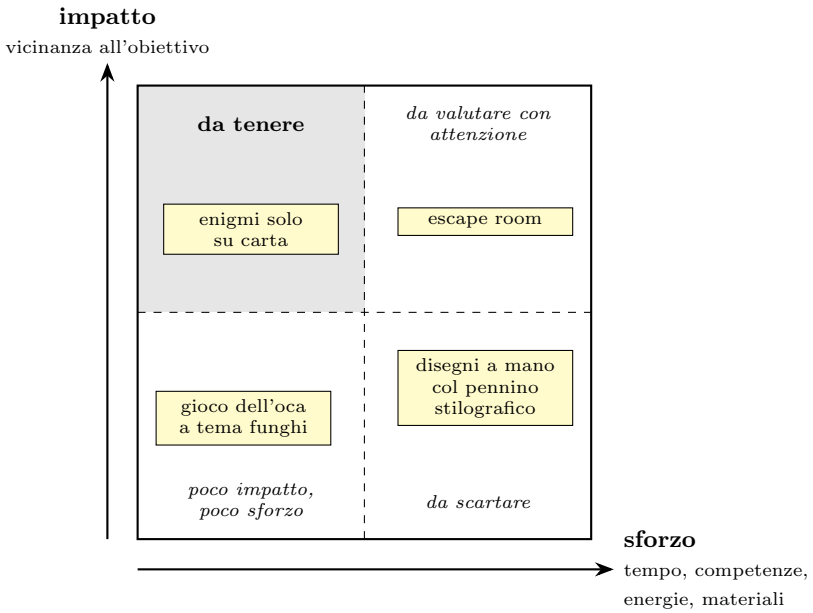


Figure 2.7.4: La mappa impatto-sforzo con gli esempi di Beatrice. Si tengono i post-it che cadono nella zona grigia, in alto a sinistra: alto impatto, basso sforzo. La posizione dei singoli post-it è un'illustrazione, non una valutazione fatta a lezione (Beatrice dice solo che l'escape room e il gioco di ruolo costano molte ore, e che il gioco dell'oca costa poco e porta meno vicino all'obiettivo).

Approfondimento: una matrice da project management. La mappa impatto-sforzo è una delle matrici più diffuse nella gestione dei progetti e nel *product design* (a volte chiamata *impact/effort matrix* o matrice delle priorità), con i quattro quadranti spesso battezzati *quick wins* (alto impatto, basso sforzo), *progetti maggiori*, *riempitivi* e *da evitare*. Nel contesto della lezione ha un uso preciso: non serve a decidere che cosa è più bello, ma che cosa **si riesce a finire**. È la stessa preoccupazione per lo *scope* che Daniele aveva espresso con il *mandate* ("niente overscope") e che Ruggero riprenderà nel tutoraggio.

2.7.18 Il percorso dell'utente e "state sul semplice"

"Vi do due consigli".

Primo consiglio: ripercorrete il percorso dell'utente. È un consiglio "sottovalutato tantissimo", perché non è sempre applicabile come lo si dice: "prendetelo come un esercizio di stile, non come una regola". Se si hanno un sacco di belle idee e non ci si ferma a chiedersi **che cosa fa la mia persona giocante dall'inizio alla fine**, è come progettare un parco divertimenti, o un **percorso nella città**, dicendo: vogliamo passare per la piazza principale, per la fontanella, per le scuole pubbliche. Bene: ma **in quanto tempo** passo di lì? Che cosa vedo **prima** e che cosa **dopo**? Bisogna sapere che cosa vede la persona mentre cammina. Fuor di metafora: nel percorso di gioco bisogna avere chiaro che cosa succede, "come se facessimo un **diagramma delle varie fasi del gioco** e di che cosa fa la persona giocante nelle varie fasi".

È complicato, e per questo si salta: in un gioco di ruolo, come si vede tutto quello che può succedere? In un gioco a scelte multiple? Si segna tutto quello che si riesce, per immaginare il percorso. E serve **prima di iniziare a fare i materiali**: se mancano dei pezzi, si rischia di iniziare un gioco che non si riesce a concludere. "Non ha senso iniziare a progettare le grafiche, il copy e tutte queste cose se il gioco non sta in piedi".

La sequenza. La prima indicazione, che Beatrice invita "fortemente" a seguire, mette in fila i passaggi:

1. valutare con i **pallini** meccaniche, dinamiche, setting e tipo di gioco da tenere;
2. mettere i post-it vincitori nella **mappa impatto-sforzo** e scegliere quelli in alto a sinistra;
3. farli entrare in un **percorso di gioco**.

Il percorso di gioco va definito **prima di aver prodotto qualunque cosa**: "non avete fatto una carta, non avete scritto mezza parola sul gioco". Non bisogna iniziare a scrivere un testo senza avere "una mezza idea, non deve essere precisa, ma almeno una mezza idea" di che cosa succede all'utente. Non perché seguire quello che chiama di più non vada bene, "anzi, a livello creativo va benissimo", ma perché si rischia di perdere un sacco di tempo, e non conviene in certe situazioni: con un **committente**, con un capo che dice "il giorno X, all'ora Y dovete consegnarmi la cucina", e in un corso il cui scopo è consegnare un gioco "fatto e finito, fatto e testato", per allenarsi a creare giochi. Dopo il percorso, si definiscono gli step, **chi fa cosa**: ci si arriverà "con più tranquillità" superate queste parti. Il file, "letto, riletto, preparato con più calma", resta il riferimento: "seguendolo non c'è rischio che non concludiate il gioco".

Secondo consiglio: state sul semplice. C'è poi "un'altra regola importantissima che non vi ho messo abbastanza in grande" (Beatrice la cerca nel file per ingrandirla): **state sul semplice**. Lo si scopre e lo si sa fare meglio con l'esperienza di progettazione: "inizierete quasi sempre con progetti complicati, farete per anni progetti inutilmente complicati, e capirete che sapete fare dei giochi quando riuscirete a farli semplici, quando non ci saranno tanti ghirigori". Semplice **non significa banale**, non significa "non complesso": significa **pulito**, cioè che si è tolto tutto quello che non serve. Questa mentalità si acquisisce facendolo "come esercizio di stile tutte le volte che possiamo".

Ritorno alla mappa. Quindi: che cosa ci porta all'obiettivo? Se un'idea non ci porta all'obiettivo, **per ora non la teniamo**: resta un'idea, la si può sviluppare in un altro gioco o prendersi del tempo per ragionarci, ma non entra in questo gioco.

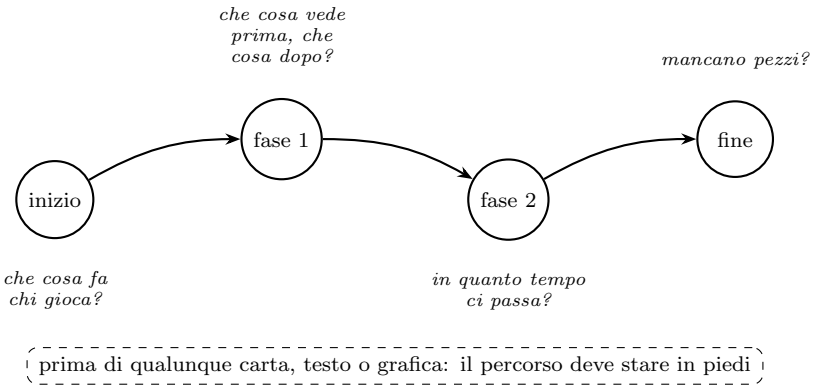


Figure 2.7.5: Il percorso dell'utente come lo descrive Beatrice: un diagramma delle fasi del gioco e di che cosa fa la persona giocante in ognuna, come un itinerario in una città. Ricostruzione schematica.

Approfondimento: la semplicità in Schell. L'invito a "stare sul semplice" ha un'eco precisa in *The Art of Game Design*, dove Schell propone la *lente della semplicità e complessità* (*Lens of Simplicity/Complexity*) e distingue la complessità *innata*, aggiunta a forza di regole, dalla complessità *emergente*, che nasce da poche regole semplici; ed è la stessa distinzione che Beatrice fa tra "semplice" e "banale". Il percorso dell'utente, a sua volta, corrisponde a quello che nel design dei videogiochi si chiama *player journey* o *core loop* visto lungo l'intera partita: un modo di verificare l'esperienza prima di produrre i materiali.

2.7.19 La ricerca condivisa

Restano 13 minuti. Beatrice chiede alla classe se preferisce provare a immaginare qualche meccanica (o narrazione: il setting, in questo caso, c'è già), vedere che cosa si è trovato nella ricerca, o fare domande. Una studentessa vorrebbe chiudere la ricerca, "per finire quella cosa lì"; "hai ragionissimo"; pollicioni da Lorenzo, da Tancredi, nessuno si oppone. In questa fase ognuno, sempre con un timer, racconta **che cosa ha trovato** e, "ancor di più", **come lo connette al gioco**. Ci sono dieci minuti; il resto si riprenderà nella lezione successiva.

2.7.19.1 La ricerca di Lorenzo Vergata

Comincia Lorenzo Vergata, "che avevo proposto l'idea", condividendo lo schermo.

- Su **BoardGameGeek**[14], il sito consigliato nella lezione precedente, c'è una lista di giochi da tavolo dedicati al tema. Tra questi *Mycelia* ("Micelia", un *deck-building* che Lorenzo ha già giocato) e *Undergrove*[15], "della stessa autrice di *Wingspan*".
- *Wingspan* fa un po' di "didattica, tra virgolette" sugli uccelli: ogni carta giocata riporta una **curiosità** sull'uccello che rappresenta. Si vince facendo la riserva naturale più bella, con uccelli e uova.
- **Film**: ne ha trovati pochi, e poco interessanti per il tema.
- Curiosità: c'è gente che si **traveste da fungo** nei boschi, scegliendo la propria specie.
- **Siti** che descrivono in dettaglio il **comportamento dei funghi**, in natura e nel nostro corpo: utili per estrarne meccaniche.
- In ambito **musicale**: si crea musica dagli **impulsi elettrici dei funghi**, "più una *gimmick*".

[14] La trascrizione dice "Burgame Geek" e "Board Game Geek".
[15] La trascrizione dice "Underground" e "Undergroove".

- Un **manuale di micologia** di quasi 700 pagine, "un bel malloppazzo", sulla maggior parte delle specie italiane e su come vivono; e un **sito di foto di funghi**, da cui prendere specie e peculiarità per estrarre meccaniche.

- Che cosa si potrebbe cercare: la situazione emotiva desiderata è quella della **sorpresa** e della **scoperta** quando si va nel bosco a trovare i funghi, o il piacere dei **trivia** sulle peculiarità delle specie trovate. E, "evidentemente", ricette e cibo.

- Le idee di gioco che ne nascono: uno **strategico in un mondo popolato da piante**, ispirato a *Wingspan*, in cui si costruisce "la parte di foresta migliore"; oppure qualcosa come *Spirit Island*, dove bisogna tenere lontani i conquistatori da un'isola: qui si terrebbero lontani dalla foresta i **visitatori**, sfruttando flora e fauna, i **poteri dei funghi** e degli animali. In generale, studiare *Mycelia* e *Undergrove* per capire come metterci mano. (Aveva proposto anche un altro titolo, che la trascrizione rende con "Bridge".)

Lorenzo si chiede se fosse questo il tipo di ricerca richiesto: "forse ho frainteso". È corretto, risponde Beatrice.

2.7.19.2 Ascoltare senza giudicare

Beatrice coglie l'occasione per un'osservazione di tecnica. Mentre Lorenzo parlava le sono venute in mente molte cose da dirgli, "però in realtà me le dovrei annotare, e queste cose vengono dette in un altro momento". In questa fase si possono fare domande, ma ragionando **sul perché** una persona propone quello che propone, non chiedendo, per esempio, "ok, ma di questo gioco che meccaniche metteresti nel nostro?": "perché ti porta fuori direzione in questa fase. In questa fase stai **esplorando**". È una fase di **ascolto senza giudizio**: si accoglie quello che arriva, "ci dormi su", e si cerca di capire **perché** gli altri sono stati colpiti da quello che portano. In fase di progettazione, invece, si va sul pratico: "abbiamo raccolto queste cose, però ci manca il gioco: come lo creiamo?", e lì si possono fare molte più domande e selezioni. È una direzione tipica, non una risposta rigida: i gruppi sono fatti di persone con metodi e tempi diversi, ed è "tutto molto più caotico" di così. Beatrice accenna anche al fatto che ci sono molti modi di fare ricerca e che nella prossima lezione si vedrà quanto tempo dedicare a ogni cosa (il file ha già dei tempi indicativi).

Tiziana e la curiosità sulla carta. Tiziana porta una riflessione nata ascoltando Lorenzo: se l'obiettivo è **far conoscere** i funghi, forse va **dosato l'aspetto competitivo**. In *Wingspan*, la "cosa carinissima" della descrizione sulla carta lei non l'ha mai letta, "perché sono concentrata su vincere il gioco". Secondo lei dovrebbero essere **le meccaniche a portarti a conoscere la specie**: usare l'abilità dell'uccello per vincere, e così entrare dentro l'abilità dell'uccello. "Perfetto", risponde Beatrice: la ricerca e sviluppo porta proprio a questo, a dire "aspetta, ci sono cose che possiamo usare, altre no". Beatrice non vuole dire "fate in un modo specifico", perché l'atto creativo è "talmente personale" che va costruito sulla propria persona e sul proprio gruppo. Ma come linea guida, in un caso come questo, il consiglio è **segnarsi le cose che funzionano o non funzionano**: per esempio "vorrei che siano le meccaniche del gioco a trasmettere i contenuti", scritto su un post-it al primo *brainstorming* su che cosa si vuole e non si vuole nel gioco. Non è una regola: è quello che farebbe lei nei tempi del corso, valutando anche come reagiscono gli altri, perché magari un gruppo "si trova meglio in un'ora parlando solo di *Wingspan*" e trova la quadra senza scrivere mezzo post-it.

Approfondimento: la coesione tra tema e meccaniche. L'osservazione di Tiziana tocca un problema classico: un contenuto "appoggiato" sul gioco (il testo sulla carta) contro un contenuto *agito* attraverso le regole. È ciò che nel dibattito critico si chiama, nei casi peggiori, *dissonanza ludonarrativa*, e che Ruggero, nel tutoraggio, chiamerà con un'espressione positiva: **coesione tematico-meccanica** (sezione 2.7.21.8). Raph Koster, in *A Theory of Fun for Game Design*, ne dà una versione radicale: quello che un gioco insegna davvero è ciò che le sue meccaniche chiedono di imparare per vincere, non ciò che la sua superficie racconta.

2.7.20 La chiusura della lezione

Restano due minuti. Beatrice anticipa la prossima lezione, "molto frontale" e divisa in due, con tempi già "troppo stretti":

- da un lato le **escape room** e le **escape room formative**, e un **mazzo di carte** da provare sulla creazione di un **gioco urbano**, cioè che usa lo spazio pubblico;
- dall'altro una parte dedicata ai **tempi** (quanto tempo dare a ogni fase della progettazione).

Se la classe le concede di essere "un po' più serrata" con i tempi, si chiuderà la ricerca e sviluppo nel primo quarto d'ora, con ognuno pronto a raccontare brevemente la sua parte.

Le domande sul calendario. Uno studente, con la mano alzata da un po' ("ah, non ho visto, dimmi"), chiede se le informazioni sui gruppi saranno sul Drive e se la prossima lezione sia quella con Ruggero "questo giovedì". Beatrice sta per caricare le slide; quanto al giovedì, non c'era quando Daniele ne ha parlato. Ne nasce un piccolo groviglio. Qualcuno dice che al **tutoraggio** di giovedì, "dalle 17 alle 19", non potrà esserci; viene ricordato che basta una persona per gruppo ("a me quindi basta: o Giordano o Rita"; "io ci sono"; "anch'io, a posto"). Gianmarco chiede se ci sono sia il giovedì sia il venerdì: secondo Beatrice sì, c'è la lezione di venerdì e il tutoraggio di giovedì, che però "dura un'ora sola, se non ricordo male". Un altro fa notare che il tutoraggio originale del sabato 3 era dalle 11 alle 13, quindi due ore. "Allora non mi ascoltate": Beatrice ammette che la visione completa del calendario le sfugge sempre in qualche pezzo, e promette di chiedere conferma e di far scrivere la risposta su Discord o via mail[16].

"Allora io direi che, se volete, siamo liberi di andare a cena". Ci si rivede alla prossima lezione ("lunedì, se non sbaglio"); Beatrice ringrazia "della pazienza anche nel caos tempistico": "siete stati e siete state bravissimi e bravissime". "Grazie di tutto, buona serata".

2.7.21 Il primo tutoraggio con Ruggero

La registrazione prosegue con il **primo tutoraggio**, giovedì 1 ottobre, su Meet. All'inizio Daniele e Ruggero si accordano sulla registrazione ("la vedo io in automatico sul Drive e poi la carico io nella cartella dei ragazzi"); Daniele augura "buon tutoraggio" e se ne va.

"Allora, buondì, intanto. Questa non è una lezione normale, come vedete". Ruggero si scusa per l'aspetto ("sono ai livelli dell'impresentabilità"): è un'ora **libera**, "sono qui per voi". Il tutoraggio non ha slide ed è **guidato dalle domande** degli studenti: "se vi serve un aiuto, una mano per qualcosa, sono qui per darvela". Come va con i gruppi? Uno ha già fatto "un gruppo WhatsApp, questo è il nostro livello di organizzazione". A metà ora riesce a entrare anche uno dei due Lorenzo, che non riusciva ad aprire il link ("adesso c'è uno stato di polizia su Meet", scherza Ruggero: gli altri anni non aveva mai avuto questi problemi).

2.7.21.1 Ripassare i passi del design sprint

Una studentessa, che non è riuscita a guardare il materiale, chiede di ripassare i passi da compiere: all'inizio c'era una cosa da fare insieme, di 30 minuti. Chiede agli altri gruppi se l'hanno già fatta: no, non ancora. Ruggero ricorda: è il **design sprint** di Beatrice, con la fase in cui ognuno prima pensa per conto suo e poi condivide. Un altro gruppo non ha ancora avuto modo di incontrarsi "digitalmente" per farla; c'è un gruppo su Discord.

2.7.21.2 Le emozioni prima delle meccaniche

La domanda di Tancredi. Tancredi chiede dell'*emotion driven design*. Il suo gruppo ha cominciato a buttare giù idee su meccaniche che potrebbero piacere, giochi da cui prendere spunto; ma "il discorso del partire dalle emozioni mi manda un po' in confusione": viene più spontaneo pensare a come un gioco possa interessare, quindi a qualche meccanica.

[16] Il tutoraggio, registrato nella stessa trascrizione, durerà in effetti **un'ora**.

Il giocatore e il designer. Il ragionamento non è sbagliato, risponde Ruggero: quando ci si avvicina ai giochi **da giocatori**, l'approccio è proprio quello. Ma quando li si **progetta**, l'approccio non è lo stesso di quando se ne cerca uno da giocare. Richiama i framework **MDA** e **DPE** ("prendendo il DPE come esempio"): chi vuole giocare si avvicina **dalla parte in fondo**, dall'esperienza che il gioco può dare, e poi **risale** al design giocando. Il designer deve fare il percorso inverso: progettare le meccaniche **per** l'esperienza (figura 2.7.6). "L'esperienza che arriva alla persona deve essere la prima cosa che hai in mente. Se no, rischi di fare delle meccaniche e di non avere controllo sull'esperienza che sta arrivando".

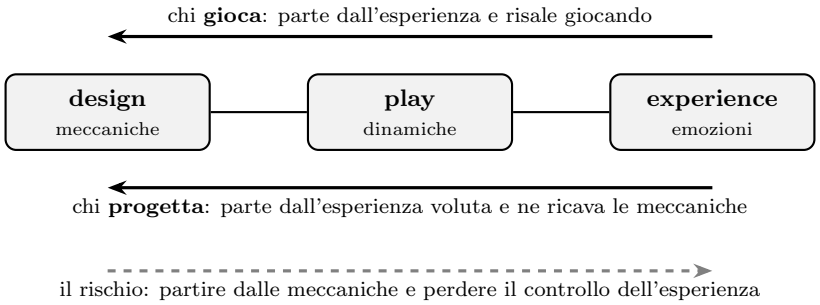


Figure 2.7.6: Le due direzioni secondo Ruggero, sul DPE (*design, play, experience*). Chi gioca e chi progetta partono entrambi dall'esperienza; il designer che parte dalle meccaniche (freccia tratteggiata) rischia di non controllare quello che arriva a chi gioca. Ricostruzione schematica.

Un po' di storia. Partire dalle meccaniche è "un modo un po' vecchio stile": nell'ambito analogico, parecchi anni fa, si facevano giochi *mechanic driven*. Il settore si è evoluto ("non siamo ancora rimasti a dei sassolini su una scacchiera"); l'*emotion driven* è la parte "più contemporanea" del design. Quando si progetta, ci si chiede **che esperienza voglio che arrivi a chi gioca**, e da lì si risale alle meccaniche.

Il database personale. Col tempo, dice Ruggero, non solo il singolo ma il gruppo impara a raccogliere meccaniche in un proprio **database personale**, a prendere padronanza di specifiche meccaniche e a sapere: "se voglio rendere il gioco più adrenalinico, queste due, tre, quattro cose mi aiutano, queste altre no; se voglio renderlo più leggero, queste mi aiutano, quest'altre no".

Il Mixing Desk nella ricerca. Può aiutare anche il **Mixing Desk**, che Ruggero ha usato in aula non solo come framework per creare ma anche come strumento di **ricerca e sviluppo**: quando si ha un'emozione, si pensa a quali altri giochi la danno e si ricostruisce "come fa quest'altro gioco a darmi questa emozione?".

Emozione o esperienza? Tancredi insiste: focalizzarsi sulle emozioni lo mette in difficoltà; gli viene più facile pensare all'**esperienza**, a come vuole che si sentano le persone che giocano. "È quello", risponde Ruggero: usa *emozioni* come scorciatoia, ma il punto è far arrivare a chi gioca un'esperienza, un'emozione, ragionando su quale esperienza si vuole dare. Quindi non "voglio fare un *worker placement*" o "voglio fare un fantasy" (l'approccio meccanico, l'approccio di setting, l'approccio di ambientazione), ma "che emozione voglio dare", e da lì meccaniche e setting. Altrimenti si rischia di andare sul **vestigiale**: "voglio fare un gioco fantasy", e si rifanno le cose fatte da mille altri giochi a tema Tolkien senza sapere che emozione si vuole dare. "E non è che tutti i giochi sulle navi hanno la stessa emozione".

"La risposta non ti piacerà". Quindi le due cose vanno di pari passo, meccaniche ed emozione? "La risposta non ti piacerà: **l'emozione viene prima**. Le meccaniche, se non vanno nella direzione dell'emozione, non usarle". Ruggero capisce che "il cuore" dice "voglio le meccaniche che mi piacciono": lui stesso ama giocare in tempo reale e prova sempre a mettere meccaniche in tempo reale quando prototipa qualcosa. Ma se vanno contro l'emozione, "devono volare nel suolo, quelle meccaniche". E questa fase non si può fare **senza ricerca e sviluppo**: la progettazione deve andare insieme al cercare altri giochi e altre opere.

2.7.21.3 Il gioco del traditore

"Qualche gruppo ha già almeno una prima fase di brainstorming, un'idea?". Il gruppo di Tancredi sì, e racconta a voce ("è un tutoraggio: avremo delle lezioni ad hoc in cui guarderemo più nel dettaglio"; Ruggero non vuole vedere lo stadio del progetto, ma capire la direzione, anche come tipologia di gioco). Hanno fatto prima una fase sugli **obiettivi**, come suggerito da Beatrice, per capire che cosa interessasse al gruppo. Sono orientati verso il **gioco da tavolo**, perché è il mondo che conoscono meglio: almeno uno di loro dei giochi di ruolo "non sa proprio niente" e li trova molto distanti. Hanno fatto una fase di *brainstorming* "totale" (giochi che sono piaciuti, emozioni, "io emozioni non tantissimo", meccaniche interessanti) e hanno sfoltito a **quattro idee principali**. Ora sono orientati verso un **gioco con il traditore**: vari personaggi, e il senso di **non sapere ancora se sarai tu il traditore**. Il traditore non è noto all'inizio; andando avanti qualcuno lo diventa; e, come sottocategoria, il traditore potrebbe **passare il ruolo** a qualcun altro: non solo espandersi "come il vampiro che morde altri vampiri", ma proprio "passare la patata bollente". Stanno capendo se è fattibile.

Non abbiate timore di sperimentare. Ruggero dice due cose. La prima: il fatto di conoscere solo i giochi da tavolo **non dovrebbe limitare**. L'anno scorso persone che non avevano mai giocato di ruolo hanno fatto "un gioco di ruolo incredibile", e un altro gruppo, anch'esso senza esperienza, "un gioco pervasivo incredibile". Con la formazione del corso si è in grado di portare a termine, in teoria, qualunque tipologia di gioco, che la si sia giocata o no. Anzi, "per mille ragioni che non ampliamo adesso", è possibile fare cose più interessanti proprio nei generi che non si conoscono bene[17].

Witch: The Road to Lindisfarne. La seconda: se il tema del traditore intriga, c'è un gioco di ruolo che va molto in quella direzione e ha "idee interessanti": *Witch: The Road to Lindisfarne*[18], pubblicato anche in italiano (*Witch: la strada per Lindisfarne*) da un editore che però "ha chiuso"[19], "e chissà dove si può reperire"; in inglese si trova in PDF. La meccanica: **il ruolo di uno dei personaggi è deciso all'inizio della partita, ma si svela solo alla fine**, dopo che tutte le persone hanno già deciso varie cose. È un **ibrido**: ha aspetti da board game, da LARP e da gioco di ruolo da tavolo, "un mix strano".

2.7.21.4 Il Go con i funghi

Un altro studente del gruppo di Tancredi racconta un'idea avuta per conto suo e poi scartata dal gruppo. Voleva prendere una meccanica del **Go** e, visto che lunedì a lezione si era parlato di funghi e del far conoscere specie e generi, unire i due mondi: un Go in cui le **pedine sono specie di funghi** con abilità particolari; un dado in cui 1–4 indicano la **stagione**, che determina quali pedine si possono collocare, e il 5 magari un **evento imprevisto**, come il passaggio dei **cinghiali**, che costringe a rimuovere certe pedine. L'hanno scartata perché il Go è complesso: "nell'arco di poche mosse si raggiunge" un numero enorme di combinazioni.

La complessità non è un limite in sé. "Questo non è necessariamente un problema": si possono creare giochi con un campo strategico più ampio di quello che si riesce a elaborare, "se no il Go non l'avrebbe creato nessuno", se per farlo bisognava già sapere come funzionava. È sicuramente tosto in termini di **bilanciamento matematico**: più un gioco si basa sull'**informazione perfetta** e sul "passo" (chi è davanti all'altro nel modello matematico), più il lavoro che arriverà dopo sarà duro, e si farà su **Excel**, "il programma di riferimento per fare queste robe". Non andare in quella direzione può essere una buona idea, "se non volete stare su Excel per tutta la seconda metà di questo corso"; ma non è impossibile, "non fatevi limitare". Lo studente pensa di farne un progetto personale: "nulla vi vieta" di fare progetti in parallelo al corso, come freelance, "anzi, sono contentissimo" se si usano le cose imparate per fare cose per conto proprio; ma i feedback dei docenti riguardano il progetto del corso, che è la base per valutare come si sta lavorando. Ruggero ricorda gruppi del passato che hanno fatto qualcosa sul Go: "ok, è divertente, vogliamo vedere come si evolve".

[17] Qui la trascrizione è confusa ("in questo caso di Pandora", "questo è un verso di Pandora"); il senso generale è chiaro, il riferimento preciso no.

[18] La trascrizione dice "Witch Road to Lindisfarne", "la strada per l'Indesfarn" e "Rod to Liendish Farming".

[19] La trascrizione scrive "Dreamlord"; il nome dell'editore non è verificabile.

Un libro sulla matematica dei giochi. Lo stesso studente chiede se ci sono paper sulla modellazione di **automi**, cioè su come simulare le reazioni di giocatori che non ci sono. Non sugli automi in particolare, risponde Ruggero, ma consiglia "assolutamente" un libro di **Maria Chiara Baghi**[20], che ha lavorato in matematica e ha scritto un libro proprio sul **gioco e la matematica**: per i modelli matematici è "il libro che consiglio", nonostante "un sottotitolo discutibile" (che "non piaceva neanche a lei, non l'ha deciso lei"). È semi-introduttivo e piuttosto divulgativo: non è comprensibile solo a chi fa game design. ("Siamo a ottobre. Fa ancora caldo come a luglio, è incredibile".)

2.7.21.5 Tabletop Simulator: prepararsi alla lezione di domani

Ruggero dà qualche informazione utile. Il giorno dopo c'è lezione con lui, su **Tabletop Simulator**, e il consiglio "fortissimo" è di avere **il software già installato**: Ruggero condividerà lo schermo e mostrerà le cose, e l'ideale è che gli studenti le facciano in parallelo, "mani in pasta nel software". Darà per scontato che lo si sia già usato per giocare: muovere la telecamera, muoversi per il tavolo, zoomare. Chi non l'ha fatto ha tempo fino a domani: "ci mettete 15 minuti, davvero", perché all'apertura c'è un **tutorial** in gioco che insegna proprio questo. (Uno studente seguirà la lezione mentre cammina verso un altro posto: domande sul software in diretta non potrà farne, ma chi segue in differita può scrivergli.)

Che cosa giocare per prendere confidenza. Tiziana chiede se debba giocare un gioco che conosce già dal vivo. Sì, e Ruggero spiega il perché. Lo usa da anni ("quasi 3.000 ore", anzi "3.012"). Tabletop Simulator **non è un software per fare videogiochi**, né per giocare la versione videoludica dei giochi da tavolo: è un **simulatore fisico di tavolo**. Non si programmano le regole del gioco: i componenti stanno su un tavolo come nella vita reale e li si sposta in completa autonomia. Prendere confidenza è più facile con un gioco già giocato di persona, perché si sa come "manovrarlo". L'impatto iniziale può essere *clunky*: nella vita si hanno due mani, in Tabletop una sola, il mouse, "una mano con due dita". Ma ci sono anche dei pro: una visione a volo d'aquila, la possibilità di ruotare il punto di vista, mentre al tavolo bisognerebbe alzarsi.

Perché è il software di settore. Servirà soprattutto a chi fa giochi da tavolo, perché è il **software di settore**. Quando un publisher vuole vedere **prototipi in digitale**, Tabletop Simulator è la via, sia per i prototipi sia per il **playtest**. Inoltre i publisher lo usano come *host* per le **demo**: su Kickstarter, per le anteprime, o dopo l'uscita, mettono moduli ufficiali per far provare il gioco e far decidere se comprarlo. È una buona mossa, perché vuol dire essere così sicuri del gioco da scommettere che chi lo prova in digitale poi lo compri; "e statisticamente è così": Ruggero e i suoi amici hanno speso in giochi fisici più di prima, "una maledizione che vi beccherete". Per la lezione, però, interessa la parte di **creazione dei moduli e dei prototipi**, non il gioco; ma chi lo usa già per giocare conosce metà delle cose (navigazione, menu) e non serve spiegargliele da zero.

Il Workshop e i moduli *scripted*. Tiziana guarda i giochi *free to play* senza trovarne uno che conosce. "Ma c'è il **Workshop**". Ruggero condivide lo schermo (dopo aver combattuto con gli aggiornamenti di Steam) e cerca un titolo che lei conosce: *Carcassonne*, di cui ci sono molte versioni (Junior, base più espansioni, versioni di fan). Avverte che non tutti i publisher hanno la stessa politica: alcuni dei più grossi sono restrittivi sulla proprietà intellettuale e i loro giochi non si trovano. Poi *Harmonies*, che ha tre versioni: una per il gioco in solitario, una **scripted**, una **non scripted**. Il consiglio "fortissimo": per domani **non giocare moduli scripted**, cioè moduli con un'aggiunta di programmazione che automatizza delle operazioni. Per la lezione serve conoscere il gioco **senza automatismi**.

Le istruzioni pratiche. Seguono alcuni minuti di assistenza tecnica. Da Mac, Tiziana non trova il Workshop: è nei tre puntini, e c'è una barra di ricerca. Ci si iscrive a un modulo con "Sottoscrivi" dalla pagina del Workshop e il modulo viene aggiunto da solo, senza scaricare niente. Ma conviene farlo **a software chiuso**, perché Tabletop Simulator carica i moduli sottoscritti quando lo si riapre (Tiziana non trovava il gioco proprio perché il software era già aperto). Condividendo lo schermo (dopo qualche tentativo con le

[20] La trascrizione scrive "Maria Chiara Bagli" e "Chiarabaghi"; la grafia esatta del cognome e il titolo del libro, che Ruggero non dice, non sono verificabili dalla trascrizione.

finestre di Meet: meglio lo schermo intero, perché Steam apre finestre e pop-up separati), Ruggero la guida: Steam, libreria, Gioca, *Create*, una stanza *single player*, poi la sezione **Workshop**, accanto a Classic e DLC (e a una quarta voce che la trascrizione rende con "7 log"). Due consigli "per la sanità mentale": togliere la musica del menu principale dall'ingranaggio delle impostazioni (*main menu volume*), "perché fa impazzire le persone"; e ricordare che dal menu (o con Esc) si vedono le scorciatoie da tastiera, e che la lampadina al centro del menu iniziale è il tutorial ("in 5–10 minuti è abbastanza veloce"). Il modulo che Tiziana ha aperto, però, è scripted: "attenzione, così non va bene".

Giocate in multiplayer e organizzatevi. Uno studente racconta di aver fatto il tutorial e di aver provato un gioco che gioca anche da solo, per vedere se si ricordava abbastanza le regole: "è andata abbastanza bene". Il consiglio di Ruggero è giocare in **multiplayer**, perché aprire una stanza in multiplayer è diverso dall'aprirla in single player; e parlare con i compagni di corso, "il modo più veloce per mettere le mani su questi software". Tiziana ci ha provato, ha scritto su Discord, "ma nessuno mi ha risposto". Ruggero: "imparate alla svelta a organizzarvi in gruppo, perché altrimenti, quando si arriva al playtest, lì piangerete sangue". Se non ci si organizza per giocare, organizzarsi per strutturare un playtest è "dieci volte più difficile". Ma è fattibile: tutti i task sono fattibili, e negli anni scorsi tutti i gruppi hanno portato a termine il gioco.

2.7.21.6 Le idee "a caso" e le emozioni

Tancredi torna sull'*emotion driven*. Per trovare idee gli viene da partire da cose che succedono nella vita, o da strumenti come *Dixit* o i dadi di *Rory's Story Cubes*, che ha letto essere usati per farsi venire idee; ma così gli vengono **immagini**, meno **emozioni**.

Il metodo di Ruggero. Ruggero racconta il suo approccio, lo stesso che usa in workshop e *jam* in cui si crea un nanogioco in 3–4 ore "dal nulla". Parte dalla **ruota delle emozioni**, quella vista nella lezione sull'*emotion driven design*, e focalizza le emozioni che interessano: non è più un gioco di ruolo in 200 parole, quindi ci può stare una focalizzazione più ampia, **due o tre emozioni**, non una sola. Poi parte il **ping pong con la ricerca e sviluppo** (figura 2.7.7): quali giochi danno queste emozioni, e che esperienza voglio far arrivare a chi gioca? Poi: come faccio arrivare questa esperienza? E di nuovo la ricerca: quali giochi fanno questa esperienza, o qualcosa di simile; che cosa mi dà una mano meccanicamente, che cosa mi allontana meccanicamente.



Figure 2.7.7: Il "ping pong" con la ricerca e sviluppo descritto da Ruggero: a ogni passo, dalle emozioni all'esperienza alle meccaniche, si torna a cercare che cosa fanno altri giochi. Ricostruzione schematica.

Esempi banalissimi, apposta. "Voglio dare dell'**adrenalina**: metto una **clessidra**, e quando la clessidra è a zero tutti perdono", con meccaniche per far girare la clessidra. Oppure "voglio dare il senso della **corsa**": una **pista**, un percorso dal punto A al punto B in cui delle pedine avanzano fisicamente. Il metodo si estende a sfumature più complesse: far sentire una persona **innamorata**, "un po' di infatuazione", come fa molto bene *Fog of Love*; far sentire una persona disperata, triste, atterrita: ci sono giochi che lo fanno. "Ogni emozione, potenzialmente, la potete riportare **metaforizzandola in meccaniche**".

Non inseguite i trend. Non bisogna preoccuparsi di usare "le meccaniche che vendono in questo momento": è un ragionamento **fallace**. Le meccaniche che vendono adesso sono quelle su cui altri lavorano già da due o tre anni; quando si sarà pronti con il proprio gioco, quel trend sarà nato "due o tre anni fa". Giochi da tavolo, giochi di ruolo, LARP e giochi pervasivi richiedono tempo per essere fatti, e quindi inseguire i trend è "proprio fallimentare". Lo si è visto: i publisher che hanno chiuso con questo approccio sono "innumerevoli". "Se volete fallire immediatamente, è quella la via perfetta: guardate cosa vende adesso e fate quello".

Il corso non è un publisher. Nel corso non si prendono soldi con il proprio gioco: "non siamo un publisher, non dovete convincerci a pubblicare il vostro gioco". Si possono sperimentare anche giochi che normalmente non si avrebbe un incentivo economico a sviluppare. Al corso interessa che si impari a **usare gli strumenti del game design**, più che fare un gioco di successo; nulla vieta poi di applicarli nel lavoro, "domani o in parallelo". E Tancredi, conclude Ruggero, queste cose già le sa: i giochi che ci piacciono non ci piacciono in astratto, ma perché ci danno qualcosa di emotivo; capire **come** ce lo danno è la parte di ricerca e sviluppo che interessa.

2.7.21.7 Che cos'è il pitch del 12 ottobre

Angelo[21] chiede dove indirizzare il lavoro: le prossime task saranno sul progetto? Non ci sono altre task, risponde Ruggero: quelle fatte con lui all'inizio (il personaggio, il gioco di ruolo in 200 parole) erano **task didattiche**; da adesso alla fine del corso c'è **il project**, il gioco di gruppo, con le sue **milestone**. Ognuno dovrebbe aver già fatto una piccola conversazione nel gruppo; magari non tutto quello che c'è da fare, ma i gruppi ci sono.

Il cuore del gioco. Il 12 ottobre, "tra 10 e 11 giorni", c'è il pitch, e lì bisogna avere "chiara e **solidificata nella pietra** l'idea al cuore del vostro gioco". Il minimo necessario: **che tipo di gioco è** e, lato esperienziale, **che cosa prova chi gioca**. Il lavoro da qui al 12 è il cuore del gioco: fare *brainstorming*, trovare un'idea e un tipo di gioco che interessano, e **focalizzarsi** su quello. **Niente overscoping**, "non andate a sparare grosso": meglio un gioco piccolo e compatto che si riesce a fare di un gioco che vuole essere il migliore del suo genere e non si fa in tempo a realizzare. E farlo **emotion driven**: cercare l'emozione, focalizzarsi su quella, anche con qualcosa di compatto; "c'è sempre tempo poi di aggiungere cose dopo", ma il cuore deve esserci.

Non è un pitch commerciale. Ci sarà una lezione sulla struttura dei pitch in termini di produzione, nella parte produttiva del corso (anche con Flavio), ma **volutamente non prima del 12**: il 12 **non** è un pitch commerciale ai docenti. L'hanno chiamato pitch "perché pitch può voler dire mille cose": il 12 i docenti vogliono sapere **su che cosa si sta lavorando**. Ed è una milestone importante: **una volta solidificata, non si cambia**. "Se decidete di fare, che so, un card game per due persone adrenalinico, quello è il gioco su cui lavorerete. Punto". Non esiste che la settimana dopo si cambi idea e si porti "un gioco pervasivo con gli ombrelli".

Ruggero lo ripete, "perché sono sempre direttissimo con voi": non bisogna vendergli il gioco. Non darà un voto in base a quanto il gioco piaccia a lui: valuta il **percorso di game design**, cioè che gli strumenti si sappiano usare e che ci si sia ragionato sopra. E i dieci giorni che mancano "sembrano tanti, sembrano pochissimi, ma sono giusti": "e quella è la parte facile".

2.7.21.8 "Io vi chiederò perché"

"Questo è uno spoiler, ve lo dico prima perché lo faccio sempre". Durante le presentazioni dei giochi, Ruggero chiederà **perché**, "un sacco di volte", e bisogna avere una risposta. "L'ultima cosa che voglio è che abbiate deciso qualcosa senza sapere perché. Non esiste". Si può non essere d'accordo sul fatto che una scelta sia l'opzione migliore, "ma il perché dovete sapermelo dare". Vale per tutto: le meccaniche, l'emozione, la

[21] La trascrizione non segnala chi fa la domanda, ma Ruggero, rispondendo, si rivolge ad "Angela", cioè, con ogni probabilità, ad Angelo.

durata, il numero di giocatori. Perché per 3–4 persone? Perché in solitario? Perché per 5? Perché mezz'ora, perché un'ora? Perché questa meccanica e non un'altra?

La coesione tematico-meccanica. Lo scopo ultimo è la **coesione tematico-meccanica**: se le meccaniche e il tema lavorano insieme per l'esperienza, "io sono in scioglimento e sono contento"[22].

"Siete una classe sveglia". Ed ecco lo sfogo, che Ruggero stesso chiama così ("scusate, mi inalbero", "mi sono scaldato"): "siete una classe sveglia, cazzo". Gli sembra una classe che **dubita molto di sé**, e non capisce perché: i task sul **gioco di ruolo in 200 parole** e sul **personaggio** "sono forse l'annata migliore" che abbia visto, e ha avuto classi che hanno fatto task peggiori ed erano più sicure di sé. "Voi queste cose le sapete già fare, dovete solo strutturare un attimo gli strumenti": le slide e le lezioni ci sono, c'è tutto quello che serve. E non si è soli: ci sono Ruggero, Beatrice e Daniele, con Telegram e email, e più avanti le lezioni con Marta e Flavio. Mancano ancora degli strumenti, che arriveranno a tempo debito; "per ora vi accompagniamo in questo percorso". Ad alcuni che gli hanno scritto su Telegram ha risposto "praticamente subito, a prescindere dall'orario": "non fatevi problemi". "Meno dubbi su di voi"; i dubbi sul giudizio e sul lungo termine "ci stanno, state ancora imparando".

La lezione di domani. Una studentessa ammette di non avere ancora domande: non ha avuto tempo di guardare le slide, e deve organizzarsi con il gruppo. Va bene: questo è **il primo di tre tutoraggi**, gli altri due saranno dopo il pitch, e Ruggero confida che al prossimo arriveranno "attrezzati e attrezzate con 112 domande". La lezione di domani, avverte, sarà "secca, desertica quasi": un software, due o tre cose da vedere per dare strumenti pratici, "mani nel motore", "non una lezione di alta filosofia"; "forse un po' noiosa per alcune persone, ma *that's it*". La si compenserà con la seconda lezione di Beatrice, lunedì.

2.7.21.9 Il Mixing Desk: paletti, non validazione

Tancredi ha provato a usare il **Mixing Desk** con gli slider sul gioco del traditore, e ha il dubbio di averlo usato **troppo presto** nel flusso appena descritto (emozione, esperienza, ricerca). "Ho già capito dove stai andando".

Il Mixing Desk ha **due scopi separati**: applicarlo a un gioco che esiste già, per vedere che cosa fa; oppure applicarlo per **mettere dei paletti** su che cosa si vuole che il proprio gioco sia. Il rischio, che Ruggero ha visto spesso in chi comincia a usarlo, è prendere il Mixing Desk e applicarlo alla propria idea come se si dovesse **valutare un gioco**, usandolo come strumento di **validazione**. "Non è uno strumento di validazione": in fase di game design è uno strumento di **paletti paradigmatici**, per piazzare **fondamenti e confini**, non per giudicare se l'idea è buona o no. "Se usi lo scopo di valutare un gioco sulla creazione, non ne esci vivo". Tancredi riformula: serve a mettere dei paletti, "cosa voglio che il mio gioco sia, e da qui non devo scappare". Esatto: se si decide che il gioco è **non verbale**, quello è un paletto, da mantenere per tutto il design (tabella 2.7.5).

Inoltre **non tutti gli slider servono** in questa fase. Per un gioco da tavolo, per esempio, lo slider sulla *runtime direction*[23] "non ti serve". Gli slider non pertinenti si possono lasciare da parte, e se ne servono altri, "se li crea". Per chi vuole approfondire, Ruggero rimanda all'articolo su **Analog Game Studies** già citato, che racconta la storia dello strumento e le sfumature degli slider.

2.7.21.10 Il gioco di gruppo è un compromesso

Mancano pochi minuti. "Non c'è nulla di cui disperarsi: è un corso breve ma intenso, però è fattibile": anche gli anni scorsi persone che lavoravano hanno finito il gioco. Ora sembra tanta roba perché i gruppi sono ancora nella fase di **apertura**, in cui tutto è possibile, "mille idee, mille possibili progetti". Bisogna divergere, "divergenza e convergenza", come avrà detto anche Beatrice: le idee aperte vanno chiuse, con un *brainstorming* strutturato con i compagni di gruppo.

[22] Così nella trascrizione; il senso, "sono contentissimo", è chiaro.

[23] Il cursore sulla gestione del gioco durante la partita (attiva o passiva), cioè su quanto l'organizzazione o un master intervengano mentre si gioca: pensato per il LARP, ha poco senso per un gioco da tavolo senza master.

	Uso analitico	**Uso progettuale**
Su che cosa	un gioco che esiste già	il proprio gioco, ancora da fare
A che cosa serve	capire che cosa fa quel gioco, anche in ricerca e sviluppo ("come fa a darmi questa emozione?")	mettere **paletti**: fondamenti e confini da non superare ("un gioco non verbale")
Che cosa non è	—	**non** uno strumento di validazione dell'idea
Gli slider	tutti, per descrivere	solo quelli pertinenti; se ne possono aggiungere

Table 2.7.5: I due usi del Mixing Desk secondo Ruggero.

E, di nuovo, è un **progetto di gruppo**: "nessuno uscirà da qui con il gioco che voleva fare". Anzi, Ruggero "dubiterebbe e sospetterebbe fortissimo" delle dinamiche sociali di un gruppo in cui qualcuno dicesse "è il mio gioco preferito della vita, quello che voglio fare da sempre": vorrebbe dire che ha sovrastato gli altri. Uscirà un gioco che è **il compromesso inevitabile** tra le cose che si vogliono fare; l'importante è che sia **interessante per tutte le persone** del gruppo, non solo come gioco che piace giocare ma anche come **aree progettuali**: un gruppo che non ha mai fatto un gioco di ruolo potrebbe voler fare un gioco di ruolo, "va benissimo, anzi, da esperienze così escono spesso i giochi migliori".

Le ultime battute. Uno studente confessa che in quel momento sta programmando per una *jam* in corso, in cui è "l'unico programmatore", "e anche la UI": "mille cappelli, va bene". L'ultima domanda: il multiplayer di Tabletop Simulator si fa solo con persone che si conoscono? No: esistono un gruppo Discord e gruppi Telegram di Tabletop Simulator dove la gente si trova a giocare, anche in modo casual. Ma bisogna accordarsi per giocare **nello stesso momento**, perché si gioca **in tempo reale**: "non è una versione digitale dei vostri giochi da tavolo, non è asincrono: è come se invitaste delle persone nella vostra cucina", e devono essere lì per giocare. "Ci vediamo domani".

Punti chiave del capitolo

- Il **project work**: tre gruppi da tre, scelti dai docenti; gioco analogico di qualunque tipo, senza tema; **emotion driven design** obbligatorio; niente IP altrui (salvo autori morti da 70 anni); niente overscope ("non un *D&D*"). Calendario: tutoraggio il 1 ottobre, **pitch** il 12 ottobre, tutoraggi il 7 e 14 novembre, consegna il 4 dicembre. Si lavora **nel Drive**, perché lo storico mostra il processo; si taggano i docenti su Discord; il lavoro di gruppo è soprattutto **soft skill**.
- Ci sono due **obiettivi di design**: quello del gioco e quello **personale** ("perché sto facendo questo gioco?"): pubblicare (editore o autoproduzione), esprimersi, formare e sensibilizzare, fare marketing. Ogni obiettivo chiede competenze diverse (il mondo editoriale, i competitor "ad ampio spettro", fiscalità e GDPR, terzo settore, bandi, scuole, formazione aziendale e finanziata, "come aiuta l'azienda a fare più soldi"). Lo stesso gioco può servire più obiettivi; conoscere gli interessi altrui rende facile il dialogo.
- Il **design sprint** non nasce per i giochi: Beatrice lo adatta in fasi (chiacchierata iniziale, obiettivo, ricerca, brainstorming, valutazione, ruoli) come strumento per ritrovare la direzione. Il project management nel settore quasi non esiste, ma questi strumenti aiutano a conoscersi.
- Le regole dell'esercizio: post-it individuali e in silenzio (al massimo tre per l'obiettivo, senza limite per le meccaniche); presentazione con **solo domande**, niente commenti; **sei pallini** a testa, anche tutti sulla stessa idea, anche sulla propria, senza spiegare; se c'è parità si rivota. Soprattutto: **i timer**. Divergere e poi "quagliare"; qualcuno nel gruppo deve tenere i tempi, o si "overworka".
- Ha vinto **l'esplorazione del bosco per sensibilizzare sulle specie fungine**. Nella ricerca contano anche film, musica, quadri, sensazioni; va fatta comunque, prima o dopo. Quando si condivide la

ricerca si **ascolta senza giudizio** e si chiede *perché* qualcosa ha colpito; si annota che cosa funziona ("che siano le meccaniche a trasmettere i contenuti", come osserva Tiziana su *Wingspan*).

- La **mappa impatto-sforzo**: si tengono le idee ad **alto impatto e basso sforzo** (in alto a sinistra). Lo sforzo sono ore, competenze, energie e materiali (cento carte vanno create e stampate). Serve a **finire** il gioco, non a farlo "più figo".
- Prima di produrre qualunque materiale, il **percorso dell'utente**: che cosa fa chi gioca, prima e dopo, in quanto tempo. Poi chi fa cosa. **State sul semplice**: semplice non è banale, è pulito. Quando ci si perde, si torna a votare; dopo ogni fase si dorme su; si impara facendo molti giochi.
- Dal tutoraggio: chi gioca parte dall'esperienza e risale; chi progetta parte dall'esperienza *voluta* e ne ricava le meccaniche. **L'emozione viene prima**: le meccaniche che non vanno in quella direzione si tolgono, anche se piacciono. Ruota delle emozioni, due o tre emozioni, ping pong con la ricerca (la clessidra, la pista, *Fog of Love*); non inseguire i trend.
- Il **pitch** del 12 non è commerciale: fissa "nella pietra" il tipo di gioco e l'esperienza, e poi non si cambia. Ruggero chiederà **perché** di ogni scelta; l'obiettivo è la **coesione tematico-meccanica**.
- Il **Mixing Desk** in progettazione mette **paletti**, non valida l'idea; non tutti gli slider servono. La complessità alla Go non è vietata, ma costa bilanciamento (su Excel). Il gioco di gruppo è un **compromesso** che deve interessare tutti.
- **Tabletop Simulator** è un simulatore fisico di tavolo, non un motore: si impara con un gioco già noto, con moduli **non scripted**, in multiplayer; è il software di settore per prototipi, playtest e demo dei publisher.

Chapter 2.8

Tabletop Simulator: costruire il modulo di un prototipo, dal tavolo vuoto al Workshop

Questa è l'**ottava lezione** del corso di **Analog Game Design**, ed è la lezione più tecnica del corso. Non si parla di emozioni, di framework o di ricerca, ma di uno **strumento**: **Tabletop Simulator** (spesso abbreviato in **TTS**), il programma con cui, nel settore dei giochi da tavolo, si costruiscono i **prototipi digitali**, si fanno i *playtest* a distanza e si mostrano i giochi ai *publisher*. La lezione era stata annunciata nel primo tutoraggio, il giorno prima, dove Ruggero aveva chiesto alla classe di installare il software e di fare il tutorial di base; qui si passa dal saper *giocare* su Tabletop Simulator al saperci *costruire* un gioco: un **modulo**, nel lessico del software.

Il docente è **Ruggero**, che conduce da solo tutta la lezione condividendo lo schermo e costruendo dal vivo, passo per passo, un "prototipo brutto" (il nome che gli darà caricandolo sul Workshop). Il suo metodo è dichiarato all'inizio: la lezione sarà "piena di banalità", frasi che suonano ovvie ("un gioco da tavolo è un gioco su un tavolo") ma che, nell'ottica di Tabletop Simulator, "non sono così banali". E il principio che ripete "fino allo sfinimento" è uno solo: **Tabletop Simulator non è un software per fare videogiochi**, è un **simulatore fisico** di un tavolo, in cui ogni oggetto ha un peso, una massa, uno spazio e delle collisioni. Quasi ogni consiglio pratico della lezione discende da qui. Il secondo filo conduttore è meno teorico e molto sentito: il software è il riferimento del settore, ma è pieno di "scelte vestigiali" e di stranezze che vanno conosciute per non perderci giorni, e Ruggero le segnala tutte, con un affetto esasperato ("sono pazzi", "è un miracolo che esista").

La lezione è **online** e si colloca con ogni probabilità **venerdì 2 ottobre 2026**[1], dalle 17 alle 20 circa. Ci sono due pause: la prima di sette minuti, fino alle **18:40**; la seconda fino alle **19:35**. Dopo la seconda pausa la lezione diventa una sessione di **domande libere**, che negli ultimi minuti si trasforma in una chiacchierata sul mestiere tra Ruggero e gli ultimi due studenti rimasti ("siamo rimasti solo noi tre"). All'inizio sono collegati **Tancredi**, **Gianmarco**, **Nicola**, **Angelo** e **Tiziana**; Ruggero aspetta ancora Lorenzo e Giordano ("hanno ancora 4–5 minuti per entrare"), e la trascrizione non dice con certezza se arrivino. Più avanti intervengono **Gianmarco** con domande sulle carte e **Rita**, che a fine lezione dice che è stata "la lezione più bella finora". Tiziana, a quanto si capisce, lascia la lezione dopo la prima pausa.

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi, a cominciare da quello del software, che diventa di volta in volta "Tabletop Shield", "DevOps Simulator", "Tempodos Simulator", "Temuto Simulator", "Teledo Simulator", "Bluetooth Simulator", "Moto Simulator" e, per sigla, "DTS", "TBTS", "PPS" e "TDS". Il capitolo usa sempre *Tabletop Simulator* o *TTS*. Altre storpiature ricorrenti: "Stem" per Steam, "gubetto" e "tubetti" per cubetto, "Asol"

[1] La data non è detta, ma è ricostruibile: all'inizio Ruggero dice che la domanda sul pitch "è stata già chiesta ieri nel tutorial" e che certe cose le aveva dette "nel tutoraggio di ieri" (il primo tutoraggio è stato giovedì 1 ottobre); dopo la seconda pausa si descrive "stanco, venerdì pomeriggio, dopo tre ore di Tabletop"; e nel ripassare il calendario parla di "lunedì" come della prossima lezione, seguita dal "venerdì prossimo" e poi dal 12.

e “terraform in Mark” per *Azul* e *Terraforming Mars*, “Becky Zidden” e “Becky’s Hidden” per l’opzione *Backs is hidden*, “l’ifo” per LIFO, “Miple” per *meeple*. Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante le due pause la trascrizione registra per decine di righe frasi senza alcun rapporto con la lezione, ripetute in loop (“qual è la risposta che ci può essere?”, “grazie a tutti per l’attenzione che ci ha spiegato”, “la situazione è questa, la migliore dal 2005”, discorsi sulla “dottrina sul mercato del lavoro” e perfino sul “governo di Dalai Lama”); lo stesso tipo di frasi apre il file, prima che la lezione cominci. Sono artefatti del riconoscimento vocale e il capitolo le ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro e resta generico altrimenti.

Come leggere questo capitolo. La lezione è una **dimostrazione dal vivo**: Ruggero clicca, carica immagini, sposta oggetti, e la trascrizione registra solo la sua voce. Il capitolo ricostruisce quindi le **procedure** (dove si clicca, che cosa chiede il software, che cosa conviene rispondere) e le **ragioni** che Ruggero dà per ogni scelta, nell’ordine in cui le mostra. Le tabelle riassumono le opzioni e le differenze tra componenti; le figure ricostruiscono i flussi di lavoro (il mazzo custom, il caricamento sul Workshop). Quando l’interfaccia di Tabletop Simulator è citata, i nomi dei comandi sono quelli inglesi che Ruggero pronuncia leggendo lo schermo; la sua copia del software è in parte in italiano (il menu “Oggetti”, il “tasto destro”), e il capitolo segue la lingua in cui il comando è detto. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell) e alla pratica della prototipazione; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l’attesa e le domande sul **pitch** del 12 ottobre (sezioni 2.8.1 e 2.8.2);
2. l’avvio del software e il principio della lezione: TTS è un **simulatore fisico** (sezioni da 2.8.3 a 2.8.5);
3. il **tavolo**, il caricamento delle immagini, lo **sfondo** (sezioni da 2.8.6 a 2.8.8);
4. i **componenti standard**: blocchi e cubetti, misure, plance, carte, dadi, figurine, miscellanea, **sacchetti** (sezioni da 2.8.9 a 2.8.14);
5. i **componenti custom**: PDF e luce, **token** e **tile**, dado, **mazzo**, carte bucate e tabelloni (sezioni da 2.8.15 a 2.8.20);
6. dopo la prima pausa: penna e testo, il **salvataggio** e il **Workshop**, il riuso dei componenti altrui (sezioni da 2.8.21 a 2.8.23);
7. i **moduli professionali** di Ruggero e le funzioni avanzate, fino al divieto degli **script** (sezioni da 2.8.24 a 2.8.29);
8. dopo la seconda pausa: fisica e *dexterity*, *The Box*, il calendario, le risorse, il mercato del gioco (sezioni da 2.8.30 a 2.8.37).

2.8.1 L’attesa

La lezione comincia, come sempre, con qualche minuto di attesa. Ruggero saluta una studentessa (la trascrizione la chiama “Lucia”, ma il nome non è affidabile) con cui non c’è stato tempo di parlare di un *task*: “ci sarà modo sicuramente”, e in generale gli si può scrivere “via mail o via Telegram senza problemi”. Lei ha recuperato la lezione e il punto è “assolutamente chiaro”. Arrivano Tancredi, Gianmarco, Nicola. “Come al solito, se avete delle domande, dubbi o altro, siamo qua”.

La registrazione del tutoraggio. La prima domanda è pratica: verrà caricata la registrazione del tutoraggio di ieri? Sì, ma con un po’ di ritardo: le lezioni sono su **Zoom**, i tutoraggi su **Meet**, e la registrazione di Meet non arriva in automatico; all’inizio la si gestisce a mano, e se ne occuperà Daniele (“credo che vi abbia già risposto nel gruppo Discord”). Ruggero aggiunge che non sa bene che cosa vedano gli studenti: dal lato dei docenti, per esempio per le slide, “abbiamo un’altra interfaccia, non è la stessa vostra”.

2.8.2 Il 12 ottobre: un pitch che non è un pitch

La seconda domanda riprende un tema del tutoraggio: il 12 ottobre bisogna *consegnare* un pitch o *fare* un pitch?

Una lezione che non è una lezione. Il 12 è "una lezione particolare": dalle 5 alle 8, ma "non è una lezione, è un incontro per vedere da vicino i vostri progetti". Tolti "i tempi burocratici", i docenti dedicheranno a ogni gruppo **40–50 minuti**. Da lì vogliono il pitch, ma **non il pitch commerciale** ("l'ho detto anche al tutoraggio"): il gruppo deve dire con chiarezza **su che cosa vuole lavorare**. Il dettaglio (le singole meccaniche, il bilanciamento, il come quell'idea sarà realizzata) verrà dopo: "ci sarà tempo". Nessuno si aspetta un prototipo già il 12. Ma i docenti vogliono, almeno a livello di espressione orale del gruppo, "un *agreement*": questa è la cosa su cui lavoreremo, "messa proprio per iscritto".

La forma è libera. Il modo di comunicarlo è libero. Gli anni scorsi alcuni gruppi hanno portato presentazioni con le slide, "proprio classiche"; altri una presentazione "più dinamica, non preparata in maniera pregressa, ma che rispondesse comunque ai nostri dubbi". "Fossi in voi non mi preoccuperei della forma". Di nuovo: "non dovete venderci il vostro gioco, non dovete convincerci che il vostro gioco sia il più bello del mondo. Dovete farci capire su cosa lavorerete per il resto del corso".

Perché non c'è una scaletta. Uno studente chiede se esiste un formato, una scaletta dei punti da toccare, visto che si tratta di un intervento lungo. I docenti ce l'hanno, ma **non la danno volutamente** (la domanda era già stata fatta il giorno prima nel tutoraggio). Ci sono molti modi di fare un pitch; per i pitch commerciali esistono schemi che aiutano, "ma non è questo il punto". Il gruppo non deve ottenere un finanziamento, e i docenti non sono un *publisher*: "siamo noi professori che parliamo con voi gruppi". La forma rigida di un pitch di publishing "si applica molto poco in questa fase", perché i gruppi sono ancora nel passaggio tra il **brainstorming** e il **playstorming**, e verso la realizzazione concreta dell'idea.

Che cosa bisogna sapere. Non serve niente di fisicamente preparato. Ma il gruppo deve sapere bene, "per esempio": se vuole fare un gioco di ruolo, un gioco di carte, un ibrido; **per quante persone**; **qual è l'esperienza** che vuole dare e **come** intende darla; la **durata**; "cose del genere". Per qualsiasi dubbio ci sono le email di tutti i docenti, "non solo la mia, ma anche quella di Daniele, Beatrice, eccetera".

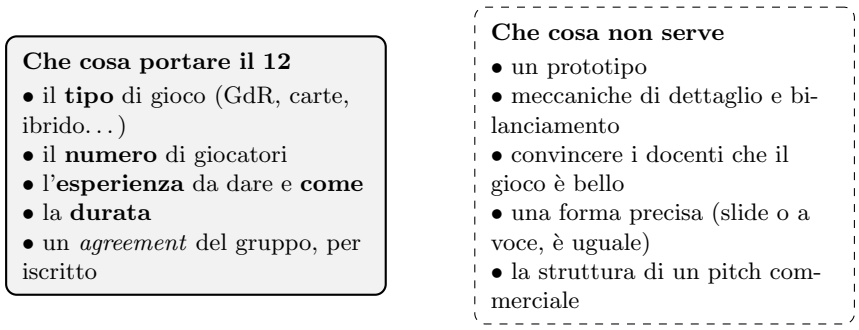


Figure 2.8.1: Il "pitch" del 12 ottobre come lo descrive Ruggero all'inizio della lezione: un accordo sul *che cosa*, non una vendita. Ricostruzione schematica.

2.8.3 Il programma: le mani nel motore

"Oggi, a livello logistico, vi dico cosa andiamo a fare". Oggi è la lezione su **Tabletop Simulator**: "lavorare mani nel motore". Il software va tenuto aperto per tutta la lezione; chi non l'ha ancora installato lo installi subito ("non è pesante, dovreste averlo pronto per tra 4 o 5 minuti"). Ci saranno domande e momenti in cui chiedere di "fermarmi a rispiegare una cosa": "fatelo, perché sono qui apposta".

Che cosa si dà per scontato. Il software "nasce per prototipare e giocare" giochi da tavolo. Ruggero dà per scontato che gli studenti l'abbiano già usato **da giocatori** (*user*, "proprio di *player*"), magari giocando tra loro qualcosa che piace: serve "il minimo necessario del sapersi spostare all'interno del software", cioè muovere la telecamera, cliccare sui menu, sapere che cosa fanno i menu. Sono tutte cose che, come detto nel tutoraggio, "il tutorial di base vi spiega in 10 minuti". Più complesso è **creare i moduli**, ed è quello che si fa oggi.

"Sì, ma con un asterisco". Tabletop Simulator è "un software gigantesco" ed è impossibile coprirlo tutto in una lezione. Ruggero coprirà le **basi** e lascerà l'ultima parte ai bisogni degli studenti: domande come "si può fare questa cosa qua su Tabletop Simulator?". La risposta, "come sempre, è quasi sempre sì": "si può fare, con un asterisco: ma con un compromesso". Il resto della lezione spiegherà che cosa vuol dire.

Ruggero tiene il registro delle presenze in vista ("se per caso entra anche Giordano, avvisatemi") e condivide **tutto lo schermo**, non solo la finestra, perché altrimenti i *pop-up* del software non si vedono. Avverte che il programma, "se non l'avete mai aperto prima, può essere abbastanza ostile all'inizio"; ma alla fine sarà chiaro perché è "usatissimo nel settore", e che "non dovete stare 3000 ore per saperlo usare bene".

2.8.4 Avviare il software e creare una stanza

Si lancia Tabletop Simulator da **Steam**. All'avvio il programma può proporre un'opzione di lancio: giocare normalmente o avviare in modalità VR. Si sceglie il gioco normale: "è possibile usarlo con i set di realtà virtuale", ma "non siamo qua per usare Tabletop in modalità VR".

Il menu iniziale. Nel menu si sente "una musichetta fastidiosa": si toglie dall'ingranaggio, alla voce dei suoni, abbassando il volume del menu, "così magari riuscite anche a sentirmi senza impazzire". L'icona della **lampadina** apre il **tutorial** di base: "io spero che voi l'abbiate già fatto. Se non l'avete fatto, non ditemelo. Fatelo dopo la lezione senza dirmi niente, e io farò finta che voi l'abbiate già fatto". Poi ci sono le due voci grandi:

Join serve a unirsi a una partita che un'altra persona ha aperto; gli studenti dovrebbero già conoscerla, se hanno giocato con persone amiche.

Create è quella che serve oggi: crea una partita **single player**, una **multiplayer** o una **hot seat**. L'*hot seat* si ignora ("non la usa credo nessuno su questo pianeta, mai"). Il *multiplayer* serve a creare partite con altri. Il *single player* crea stanze in cui può entrare solo chi le crea, ed è quello che si usa per **costruire i moduli**.

I moduli si costruiscono in single player. Ruggero è esplicito: "non create i moduli in multiplayer". In multiplayer, del resto, solo l'*host* può crearli; ma soprattutto in single player è più comodo, perché "la connessione internet non vi chiederà costantemente il ping dei vostri compagni di stanza".

All'apertura della stanza compare un *pop-up* centrale, perché Tabletop Simulator "dà per scontato che noi vogliamo giocare qualcosa, non che vogliamo creare un modulo". Si chiude. Si resta con **un tavolo vuoto**, un panorama di sfondo scelto a caso da un insieme predefinito e svariati menu. "E a questo punto ci dobbiamo chiedere: ok, benissimo. Cosa facciamo adesso?".

2.8.5 "Un gioco da tavolo è un gioco su un tavolo"

Si parte "dalle basi, proprio basi, basi". Ruggero avverte che la lezione sarà "piena di banalità": frasi che suonano banalissime, ma che nell'ottica di Tabletop Simulator "non sono così banali". E ripete un'informazione che era già stata messa lì, "nascosta, per fare *foreshadowing*", e che ripeterà "fino allo sfinimento":

> **Tabletop Simulator non è un software per fare videogiochi.** Non uscirete da qua con il videogioco del vostro gioco da tavolo. Non vi serve a quello. Tabletop Simulator, come il nome fa intuire, è un simulatore di giochi *tabletop*. Vuol dire che tutto quello che vedrete qua è **simulato fisicamente**: con un peso, con una massa, con uno spazio, con delle collisioni.

È il principio che regge tutta la lezione, e Ruggero lo riprenderà ogni volta che un componente si comporta in modo inatteso: un mazzo che sul bordo del tavolo resta inclinato, un cubetto che rotola giù dalla cornice di una plancia, una carta bucata che però collide come se fosse intera. "Questo è un simulatore di spazio fisico. Spero che, se c'è una cosa che vi entra da oggi, è questa". La conseguenza pratica: i componenti di un modulo **vanno trattati come componenti fisici**, con le stesse attenzioni che si avrebbero su un tavolo vero.

Approfondimento: il prototipo come simulazione. La distinzione di Ruggero tra *simulatore* e *motore* è la stessa che separa un prototipo di un gioco da tavolo dalla sua versione digitale. Un adattamento digitale *applica* le regole al posto dei giocatori; un simulatore fisico, invece, lascia le regole ai giocatori, come il tavolo vero, e si limita a riprodurre gli oggetti. È proprio questo che lo rende adatto al *playtest*: Schell, in *The Art of Game Design*, insiste che ogni prototipo deve rispondere a una domanda precisa, e che la sua fedeltà va scelta in funzione di quella domanda. Un prototipo su TTS risponde bene alle domande su regole, spazio e flusso del turno, perché i giocatori devono ancora eseguire tutto a mano, come farebbero con la scatola; risponde male, per costruzione, alle domande che riguardano la manualità e il tatto. Il divieto degli *script* che Ruggero darà a fine lezione (sezione 2.8.29) nasce dalla stessa logica.

2.8.6 Il tavolo e la table presence

Nel menu in alto c'è la voce **Oggetti**, con quattro sottomenu: **componenti**, **tavoli**, **background** e **oggetti salvati**. Si parte dal tavolo.

Non tutti i giochi stanno su tutti i tavoli. Nella vita reale, "nella vita di tutti i giorni, fisicamente", alcuni giochi da tavolo su alcuni tavoli non si possono giocare. Su un tavolino da caffè "non ci sta probabilmente tutto *Twilight Imperium*"; sui tavolini del treno non ci si gioca un *eurogame*, "perché il tavolo è più piccolo". Allo stesso modo, quando si pensa alla **table presence** del proprio gioco, cioè allo **spazio che occupa sul tavolo**, la scelta non è casuale: non lo è nel mondo reale e non lo è su Tabletop Simulator, perché lo spazio fisico è limitato: "le cose che non sono sul tavolo cadono dal tavolo".

Il software offre **dieci tavoli**. Ruggero li mostra uno per uno (tabella 2.8.1), guardando ogni volta due cose: quanti **posti** ha e com'è fatto il **bordo**.

Le mani sono i posti a sedere. Per contare i posti si usa, nel menu a sinistra, la voce **Zone**, sottovoce **Hands**: mostra le **mani** di carte, cioè le persone al tavolo. "La *hand* è in tutto e per tutto quanti posti a sedere ci sono". Se il tavolo ha quattro mani, "questo gioco non può tenere fisicamente più di quattro persone"; se si progetta un gioco da 6–8 persone, quel tavolo "già non va bene".

Perché guardare i bordi. "So che vi sembra apparentemente inutile questa informazione", ma sopra il tavolo ci saranno dei componenti, e i componenti sono oggetti fisici. Un bordo tridimensionale impedisce di tenere a bordo tavolo, "belle piatte, tranquille", le plance dei giocatori; un rilievo al centro impedisce di appoggiarvi piatta una plancia o un tabellone, perché "questa salita qua è una salita fisica che andrà a impattare i vostri componenti".

Perché il software è fatto così. Davanti al tavolo rotondo con il rilievo al centro, la domanda viene da sé: perché qualcuno l'ha fatto? La risposta di Ruggero, che tornerà molte volte nella lezione, è un misto di esasperazione e di affetto: "perché sono pazzi", "è un miracolo che esista". Chi ha creato il software "non sapeva cosa stava facendo"; oggi ci si ritrovano "delle scelte vestigiali di gente fulminata di anni fa che, contro la loro volontà, hanno purtroppo creato il software di riferimento del settore, che è quello che tutti usano adesso". E con queste scelte "dobbiamo venire a patti".

[3] Per il tavolo da poker e per quello di vetro la trascrizione non riporta il numero di posti (per il secondo dice solo "per otto persone"). Il nome "Round Glass" per il tavolo di vetro è quello dell'interfaccia, che Ruggero non pronuncia: lo descrive soltanto.

Tavolo	Posti	Che cosa ne dice Ruggero
Square	4	piccolo, "piccolino"; quattro persone "strettine"
Hexagon	6	un posto per lato, forma "bizzarra"; **bordi tridimensionali**: niente plance piatte a bordo tavolo
Octagon	8	come l'esagonale, con otto posti
Round	8	bordi molto meno marcati, ma al centro un **cerchio di vetro in rilievo**: niente tabellone piatto al centro
Poker	–	"non vi servirà, a meno che non dobbiate fare poker"
Rectangle	8	"belli larghi"; bordi scanalati, ma superficie ampia: va bene per "la maggior parte dei giochi pubblicati oggi"
Round Glass	8	rotondo, grande, di vetro semitrasparente, con le gambe visibili sotto, "in stile Escher": "il tavolo peggiore di tutti. Non usatelo mai"
Custom Rectangle	8	accetta un'immagine come superficie; il più grande; non se ne sceglie la dimensione; l'immagine viene **stirata**
Custom Square	8	accetta un'immagine; gambe metalliche riflettenti; "sembra piccolo, ma è gigante"; nessun bordo strano
None	–	nessun tavolo: i componenti "cadono all'infinito in questo pozzo senza fondo". È l'opzione preferita di Ruggero, per un motivo che spiegherà dopo (sezione 2.8.24)

Table 2.8.1: I dieci tavoli di Tabletop Simulator come li presenta Ruggero. I nomi sono quelli dell'interfaccia; i posti sono le *hands* visibili nella voce *Zone*[3].

La scelta della lezione. Per il prototipo dimostrativo Ruggero sceglie il **tavolo quadrato custom** con una texture di legno: "abbiamo un gioco piccolo, magari un *fillerino* per quattro persone, perfetto". Sul tavolo senza tavolo (*None*) promette di tornare: "ve lo spiego dopo, perché potrebbe essere interessante".

2.8.7 Caricare un'immagine: file locale e cloud

Il tavolo *Custom Rectangle*, appena selezionato, apre un *pop-up* che chiede un'**immagine**. È il primo incontro con una procedura che si ripeterà per quasi ogni componente custom, e Ruggero ne fissa subito le due regole.

Regola 1: mai un URL, sempre un file. Su Tabletop Simulator un oggetto custom si può creare in due modi: dandogli l'**URL** di un'immagine online o caricando un'immagine dal proprio computer (**Browse Local Files**). "Il mio consiglio personale è: non dategli mai un URL immagine", perché se qualcuno, per qualunque motivo, toglie quell'immagine dal suo indirizzo, "il vostro modulo crolla". Se online c'è un'immagine che piace, la si **scarica** e la si ricarica dalla propria cartella.

Regola 2: mai in locale, sempre in cloud. Quando si carica un file, il software chiede se caricarlo **in cloud** o **in locale**. "Non caricate mai niente del vostro modulo in locale": un modulo con immagini locali non si può usare in multiplayer, funziona solo sul proprio computer, e questo "sconfigge un po' l'obiettivo" di farlo vedere, per esempio, a un *publisher* esterno. Si carica quindi in cloud, gli si può dare un nome, si aspetta che "finisca di circolare questo cerchiettino" e si fa **Import**.

Le proporzioni. Ruggero carica prima una texture di legno, poi un tabellone più elaborato[4]: sul tavolo si può avere un tabellone, uno schema, delle zone, "già qualcosa di meccanico". Ma la dimensione del tavolo non si può definire: è il tavolo più grande che c'è, e l'immagine vi si adatta. Con una texture verticale (una "A" arancione), il software "la streccia come se non ci fosse un domani", e in più crea una luce arancione in faccia "che sembra uscita da un film sci-fi". Quindi: **occhio alle proporzioni** delle immagini. Di solito

[4]La trascrizione descrive l'immagine come "le influenze moderne nella scena del Nord di Clarp": con ogni probabilità è la copertina o una pagina di un lavoro sul LARP nordico, usata a caso come esempio; il contenuto dell'immagine non ha importanza.

il tavolo si usa **con una texture**, e i tabelloni si mettono **a parte**, senza caricarli direttamente sul tavolo, perché il tavolo li distorce.

2.8.8 Lo sfondo cambia la luce

Scelto il tavolo, Ruggero fa notare che la scena è buia: il suo sfondo è "sera al crepuscolo", in mezzo a un prato, "con tutto il problema di illuminazione che ne deriva". Il motivo è che Tabletop Simulator simula la *skybox*, l'ambiente attorno al tavolo. Dalla voce **Background** si sceglie lo sfondo: "in questo momento abbiamo il *sunset*", poi c'è il *tunnel*, e così via.

Cambiando sfondo, Ruggero chiede di guardare il tavolo: che cosa succede? Uno studente risponde: cambia il riflesso. Sì, e cambiano anche il livello di illuminazione e l'angolazione. Non solo sulle gambe metalliche, che riflettono, ma anche sulla texture di legno, che non è metallica: ingrandita, mostra colori "più freddi, più caldi in base a dove sono". La luce, in Tabletop Simulator, arriva **da tre direzioni**: **dall'alto**, **dall'orizzonte** e come **riflesso dal basso**. "Stiamo nel tecnico, mi rendo conto"; per ora il prato va benissimo. La questione tornerà con il PDF, quando si vedrà che lo sfondo può rendere illeggibile un componente (sezione 2.8.16).

2.8.9 I componenti: importare o creare?

Abbiamo il tavolo, abbiamo lo sfondo, "fantastico". Ma mancano i componenti per giocare. "Vi sorprenderà sapere" che dalla voce **Oggetti**, sottovoce **Componenti**, si possono mettere dei componenti sul tavolo. I componenti sono divisi in **16 categorie**, alcune più personalizzabili di altre. E l'approccio da avere con questo software, per ogni pezzo del proprio gioco, è una domanda in due tempi:

> Devo importare un componente nel mio gioco? Ho qualcosa che mi può servire **che già esiste**?
> O devo **crearlo da zero**?

Ruggero passa quindi prima in rassegna i componenti "più o meno standardizzati" e il modo di adattarli, e solo alla fine i componenti *ad hoc*, i **custom**.

2.8.9.1 Blocchi e cubetti

Nella categoria **Blocchi** ci sono un quadrato, un rettangolo e un triangolo; si trascinano sul tavolo e si nota che sono **3D**. Rettangoli e triangoli per ora non servono: si tengono i **cubetti**, una risorsa "usatissima" nei giochi da tavolo. Ruggero chiede un esempio di gioco con i cubetti: gli studenti citano *Terraforming Mars* e *Azul*[5].

Ricolorare. Il problema è che il cubetto è rosso, e nei giochi citati i cubetti hanno altri colori. "Ci può andare bene come forma, ma non come colore". Per ricolorarlo: **tasto destro**, **Color Tint**, e si sceglie il colore con la classica barra di tonalità e saturazione, "inclusa la semitrasparenza": la barra in fondo regola la trasparenza, e un cubetto blu con la barra abbassata diventa semitrasparente. Un cubetto "di grano" si fa giallo, e così via.

Copiare. Mentre si creano i moduli si possono usare `Ctrl+C` e `Ctrl+V`: "io posso copiarmi questi cubetti quanti ne voglio".

Ridimensionare. Un componente si può ingrandire o rimpicciolire dalla tastiera, con i tasti `+` e `-`: per esempio un cubo più grande degli altri, che valga tre o cinque risorse.

[5] "Asol" e "terraform in Mars" (poi "terraform in Mark") nella trascrizione. Ruggero aggiunge che in *Azul* ci sono "i cubetti neri per il contafonti": il dettaglio non è ricostruibile con certezza dalla trascrizione.

2.8.9.2 Le dimensioni reali

Il ridimensionamento serve per tutti i componenti, e qui Ruggero chiede di "prepararsi psicologicamente": quando si fanno moduli su Tabletop Simulator, **è buona pratica mantenere le dimensioni e le proporzioni reali dei componenti**. Ma come si sa quanto è grande un cubo? "Può essere 10 cm, come 150, che ne sappiamo".

Lo strumento Linea. Nel menu a sinistra c'è lo strumento **Linea** (il righello). Nelle sue impostazioni si sceglie l'**unità**: Ruggero consiglia i **centimetri** e la modalità 2D[6]. Con lo strumento si tira una linea e il software scrive la misura sul bordo: il lato del cubo è "più o meno 8 cm".

Misurare dall'alto. Per misurare con precisione si cambia prospettiva con il tasto `P`, passando alla vista **dall'alto** (*top-down*): senza il problema del 3D si misura "da un angolo all'altro". Il lato risulta di 7 cm. Se il cubetto dev'essere di 20 mm, lo si ridimensiona con `+` e `-` finché la misura non è "più o meno 2 cm".

Una domanda di chiarimento. Uno studente non ha capito: il cubo è stato ridimensionato con la linea? No: il cubo si ridimensiona **con i tasti più e meno**; la linea **misura** soltanto. Il tasto `Tab`, premuto una volta, mette le freccette per indicare le cose; **tenuto premuto**, mostra la linea di misura. Un'altra studentessa non ha capito come cambiare prospettiva: tasto `P`, "di prospettiva", che passa tra prima persona, terza persona e vista dall'alto. Sono cose che il tutorial dovrebbe aver già insegnato; in ogni caso, con il tasto `Esc` si apre l'**Help**, con tutte le scorciatoie del software. Anche spostare più componenti insieme e interagire con i componenti sono cose del tutorial.

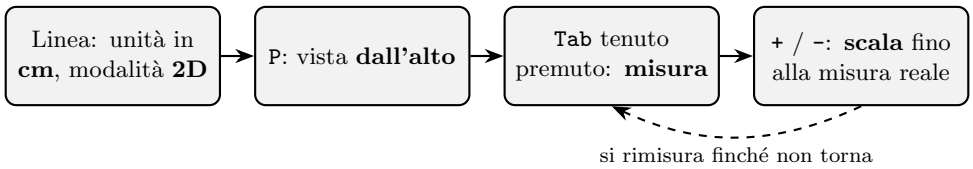


Figure 2.8.2: Come portare un componente alle sue dimensioni reali, secondo la procedura mostrata da Ruggero: nell'esempio, un cubetto da 7 cm ridotto a 2 cm.

2.8.10 Le plance

La categoria **Board** contiene le **plance**: backgammon, scacchi o dama, un'altra versione della scacchiera, dama cinese, Go, Reversi. "La domanda però è: perché ci sono?". Il proprio gioco sarà sicuramente diverso da questi: quello che serve, di solito, è la **Custom Board**, che chiede, "indovinate un po"', un'immagine, come il tavolo custom.

Bloccare la plancia. La plancia si può spostare; con il tasto `L`, di *lock*, la si **blocca**, in modo che cliccandoci sopra, anche per sbaglio, non si sposti più. "Questa è una buona pratica, perché noi non vogliamo che qualcuno sposti il tabellone con tutte le cose sopra".

La plancia si adatta all'immagine. Uno studente nota che la plancia ha cambiato dimensione da sola. Esatto: **al contrario del tavolo**, che adatta l'immagine alla cornice (stirandola), **la board adatta la cornice all'immagine**. Un'immagine alta dà una plancia stretta e alta; cambiandola (tasto destro, **Custom**) con la texture del tavolo, la plancia si allunga dall'altro lato. Ne segue una regola per i tabelloni: importarli **nelle proporzioni che devono avere nella realtà**.

[6] La trascrizione rende le impostazioni consigliate come "2D free edge no log"; Ruggero dice di non volerle spiegare "per limiti di tempo" e che "centimetri, 2D" è quello che serve. La forma esatta delle altre voci non è ricostruibile dalla trascrizione.

Il problema della cornice. "Ma adesso ti faccio vedere un'altra cosa". La plancia custom ha un **bordo di legno** in rilievo, "nello stile delle vecchie scacchiere". E siccome i componenti sono 3D, un cubo appoggiato sul bordo **rotola giù**. "Quanti board game recenti avete visto con le plance che hanno la cornice in legno alta due centimetri?". Nessuno, risponde uno studente; "forse una versione super antica degli scacchi di mio nonno". "Esatto, perché è una roba da fuori di testa": nessuno mette una cornice di legno su un tabellone che costa 200 euro. La cornice non si può togliere. La soluzione arriverà più avanti (sezione 2.8.20); per ora: "tenetevi in background l'informazione".

2.8.11 Le carte standard

La categoria **Carte** contiene: uno *standard deck*, un *custom deck*, una *custom card*, una carta a caso ("randomicamente ci pesca una carta a caso"), un **Jolly** (perché il mazzo standard è "il classico deck da poker normale di 52 carte" e non ha i Jolly) e alcuni mazzi creati dagli autori di Tabletop Simulator per un loro gioco, "che non servono a niente di niente": "li lanciamo nel sole".

Come si interagisce con un mazzo (spostarlo, pescare, rivelare, pescare nella propria mano) lo insegna il tutorial. Quello che importa sapere è che il mazzo **ha una proprietà fisica**: la carta singola è più leggera del resto, il mazzo è tridimensionale, e un mazzo appoggiato sul bordo del tavolo resta **inclinato**, tanto che si può vedere che cosa c'è sotto. "Questo è un simulatore di spazio fisico. Spero che, se c'è una cosa che vi entra da oggi, è questa". I componenti "vanno trattati come se fossero componenti fisici".

E se il gioco usa un mazzo di carte evento personalizzate? "Ve lo spiego dopo": prima i componenti standard e come adattarli, "e poi passiamo per ultimo ai componenti *ad hoc* vostri" (sezione 2.8.19).

2.8.12 Dama, scacchi, dadi e figurine

Dama e scacchi. La quarta categoria sono i pezzi della **dama**, "e qui davvero abbiamo la follia di Tabletop Simulator": c'è di nuovo la plancia, identica a quella della categoria precedente, presente quindi in due categorie. Negli **scacchi** ci sono pezzi metallici o di legno; "se volete dei riflessi ambientali, quelli di metallo sono perfetti". Ma la domanda è sempre: servono? Trattandosi di prototipi, se serve un pedone, "nessuno si scandalizza se prendete un pedoncino così e magari lo colorate, che so, rosso".

I dadi. Nella categoria **Dadi** ci sono dadi metallici e di plastica, già colorati: d4, d6, d8, d10, d12 e d20, più altre versioni (per esempio d6 colorati con gli angoli arrotondati). Si possono ricolorare e ingrandire: "mi servono da 20, ma blu"; lo si ricolora, lo si fa più grosso degli altri, e si tira come gli altri. Ma attenzione: **ricolorare un componente non colora tutto**. Un dado bianco colorato di giallo mantiene i **pallini** neri; un dado metallizzato colorato di giallo ha i pallini bianchi, ed è **illeggibile**; un dado colorato di nero "fa schifo", uno colorato di bianco è "completamente illeggibile". Quindi: attenzione a quali componenti si scelgono come base.

Le figurine. Nella categoria **Figurine** ci sono gli *standee* (le sagome in piedi) e le figurine metalliche, "i classici omini di piombo", che non servono. Serve il **custom standee**, che chiede un'immagine di **fronte** e una di **retro**. Ruggero vi mette la sua "A" davanti e dietro, "ed ecco il nostro standee". Può essere un personaggio anche con la trasparenza: un PNG ritagliato mantiene lo sfondo trasparente. Per avere un'immagine più grande mantenendo la base piccola si usa il fattore di **scala**: con 2, l'immagine raddoppia e la base resta piccola. "Mi rendo conto che può essere estremamente noioso per ora questo software"; tra poco arrivano "le cose custom per davvero".

2.8.13 Go, biglie, miscellanea

Ruggero passa "velocissimamente" sul resto.

Le pedine del Go. Le pietre del Go "possono essere come risorse": si vogliono risorse che siano sassolini, anziché cubetti? Benissimo. Colorate di azzurro e rese semitrasparenti, diventano "delle gocce d'acqua".

Le biglie e i giochi di destrezza. Le **biglie** sono biglie, "letteralmente, con tutta la fisica che ne comporta": su una superficie piana continuano a rotolare. A che cosa servono? Ai **dexterity game**, i giochi di destrezza, che si possono fare anche qui. La dimostrazione: Ruggero prende un componente, lo mette sul tavolo e lo blocca; poi usa lo strumento **Flick** (il "colpetto" con il dito) sulla biglia, che va a sbatterci contro. "Voilà! È un simulatore fisico questo software".

La miscellanea. "Dentro la miscellanea abbiamo tante cose bizzarre".

- Una **ciotola**: come componente non serve, ma bloccata sul tavolo e riempita di cubetti dà "improvvisamente una *table presence* diversa".
- Una **moneta**, che lanciata dice testa o croce.
- Le pedine del **Reversi**, a due facce, una bianca e una nera: un *token* a due lati già pronto.
- Pezzi del **domino**, tessere del **Mahjong**[7], altri dischetti, tutti ricolorabili, da usare come risorse.
- Un insieme di componenti dei designer del software[8]: "se vi servono, bene; se no, il mio sospetto è che non vi serviranno mai".
- I **Player Pawn**, i classici pedoni di plastica, in **nove colori**. Perché nove versioni, se si possono ricolorare? "Perché sono pazzi, i designer: non ci hanno pensato". Ricolorateli come volete.

I colori dei giocatori. C'è però una ragione per ricolorare con criterio. Nel selettore dei colori, la colonna a destra contiene esattamente i **colori dei giocatori**, cioè i colori delle mani attorno al tavolo[9]. Se si vuole un codice colore preciso, legato al giocatore, si sceglie da lì: "fammelo di questo arancione" dà esattamente l'arancione del giocatore arancione.

Fiches e kit da GdR. Poi le **fiches** da poker, "classiche fiches da casinò"; e il **RPG Kit**: miniature su miniature da *dungeon crawler*, maghi, guerrieri, mostri, creature. "Si muovono da sole queste" (sono animate), "un po' anni Duemila, ma oh: per un prototipo su TTS, chi se ne frega?".

Le cose "fulminate". Infine la categoria preferita di Ruggero: "le cose così fulminate che mi fanno chiedere: ma perché, perché, perché hanno messo queste cose qua? Non avremo mai una risposta".

- Una **Notecard**: un foglietto su cui si scrive con il tasto destro, compilando **nome** (il titolo) e **descrizione**. Si chiamano **index card**, "usatissime per i giochi di ruolo in ambito americano, il *tabletop* RPG specialmente". Una studentessa si è persa: come si scrive? Tasto destro, e in fondo ci sono i due campi, nome e descrizione; si può andare a capo e mettere più righe; il testo "non è in tempo reale", si aggiorna solo quando si esce dal menu.
- Un **lettore MP3**: "che devo dirvi? C'è un lettore MP3. Se volete giocare con la musica, ce l'abbiamo". (Ruggero verifica che la classe senta l'audio del suo computer: "vi voglio far soffrire per davvero".)
- Un **contatore**, che si può far salire, scendere, azzerare, o in cui si scrivono direttamente i numeri.
- Un **tablet**, "letteralmente un tablet": ci si può aprire YouTube e guardare video. "È un tablet, ragà, è un tablet. Non so che dirvi".
- Una **sveglia** e una **calcolatrice**.

[7] "Testa del Maggiog" nella trascrizione.
[8] La trascrizione lo chiama "P-Spec"; il nome della categoria non è ricostruibile con certezza.
[9] Ruggero dice prima "questi 16 colori" e poi "esattamente i 12 colori player"; la trascrizione non permette di stabilire quale dei due numeri sia quello inteso.

2.8.14 I sacchetti

Nella miscellanea ci sono finalmente "le due cose che ci servono per davvero": il **sacchetto infinito** e il **sacchetto**.

Il sacchetto infinito. Parte vuoto e rende infinito qualunque cosa gli si metta dentro: con un dado blu diventa un sacchetto infinito di dadi blu. Si può ricolorare e si può azzerare (tasto destro, **Reset**) per farne, per esempio, un sacchetto di dadi rossi. Ma Ruggero fa una richiesta precisa: **nei moduli per questo corso non si usa il sacchetto infinito**, perché "a livello produttivo io vi chiederò quanti componenti avete, e infiniti non è un numero che è possibile stampare nella realtà". Per comodità è utilissimo, "ma non è questo il momento".

Il sacchetto normale. Parte vuoto (passandoci sopra con il mouse si legge zero) e ci si mette dentro qualunque cosa. Ruggero colora tre cubetti di giallo, rosso e blu e li mette nel sacchetto; poi, con il tasto destro, decide **in che ordine** il sacchetto restituisce gli oggetti (figura 2.8.3):

LIFO (*last in, first out*): l'ultimo oggetto messo è il primo a uscire. Messi blu, rosso, giallo, escono giallo, rosso, blu.

FIFO (*first in, first out*): il primo messo è il primo a uscire. Messi blu, rosso, giallo, escono blu, rosso, giallo.

Random : "è random".

"Chiaramente non siamo persone fulminate": un sacchetto, nella realtà, si comporta in modo casuale, e quindi si mette **Random**. Nella prova, blu, rosso, giallo escono giallo, rosso, blu.

Guardare dentro. Per vedere che cosa c'è nel sacchetto: tasto destro, **Search**. La stessa funzione si usa sui **mazzi**, per vedere le carte rimaste.

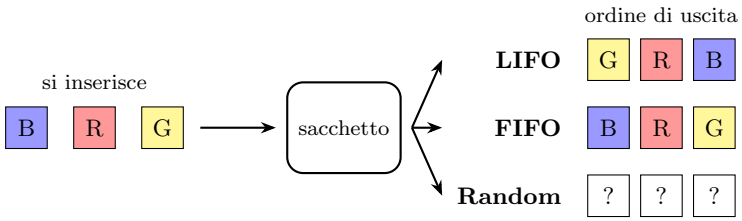


Figure 2.8.3: Le tre modalità del sacchetto, con l'esempio di Ruggero: tre cubetti inseriti nell'ordine blu, rosso, giallo (B, R, G). Per un gioco da tavolo si usa *Random*, che è il comportamento di un sacchetto reale.

Approfondimento: il prototipo e la produzione. La richiesta di non usare il sacchetto infinito anticipa un tema che il corso affronterà con la lezione sulla produzione: un gioco analogico è anche una **distinta dei componenti**, e ogni componente ha un costo. Un modulo che usa scorte infinite nasconde al designer la domanda "quanti me ne servono davvero?", che in un gioco stampato ha sempre una risposta. Lo stesso vale per i cubetti copiati a volontà con `Ctrl+V`: comodi per costruire, ma da contare prima di consegnare.

2.8.15 I componenti custom

"Allora, adesso arriviamo alla parte più incubo di tutte, che è la parte custom". La categoria **Custom** raduna tutte le opzioni personalizzabili, comprese quelle già viste (la plancia, le figurine). Sono dieci (tabella 2.8.2).

Componente	A che cosa serve e che cosa ne dice Ruggero
Custom Board	una plancia da un'immagine; la cornice si adatta all'immagine, ma ha il bordo di legno (sezione 2.8.10)
Custom Figurine	lo *standee* con fronte e retro (sezione 2.8.12)
Custom PDF	il **regolamento** del gioco, consultabile sul tavolo
Custom Token	un gettone con **forma libera** (gestisce la trasparenza e la collisione), colorabile, impilabile; il retro è specchiato
Custom Tile	una tessera di **forma regolare** (quadrata, esagonale, circolare, arrotondata) con fronte e retro corretti
Custom Deck	un **mazzo** di carte da una griglia di immagini: "la parte più difficile di tutte"
Custom Card	una carta singola, fronte e retro
Custom Dice	un dado con le facce personalizzate, da un *template*
Custom Model	un modello 3D (mesh, texture, collider): non si vede a lezione
Custom AssetBundle	pacchetti di modelli 3D, per esempio miniature: non si vede a lezione

Table 2.8.2: I dieci componenti della categoria *Custom*, come Ruggero li elenca ("board, tile, token, figurine, deck, card, dice, model, asset bundle, PDF"). Modelli 3D e asset bundle in fase di prototipo sono fortemente sconsigliati.

2.8.16 Il PDF del regolamento e la luce

Caricare un PDF. Il **Custom PDF** chiede di caricare un PDF. Ruggero ne carica uno a caso, "così a caso": un PDF sul *framework* DPE[10]. "Sempre aspettare che finisca questa rondella blu, se no TTS impazzisce". Il PDF appare sul tavolo: è **molto piccolo** ed è **molto bianco**, quasi illeggibile, perché l'illuminazione dello sfondo "fa schifo". Ed ecco il punto chiave sugli sfondi, "lasciato in sospeso" fino ad ora: cambiando lo sfondo cambia tutto. Ma magari non si vuole giocare "con le vibes di un pub di fumatori incalliti".

Uno sfondo da un pixel. Nel menu degli sfondi c'è un'opzione che Ruggero aveva taciuto apposta: **Custom**, che permette di scegliere uno sfondo proprio. Tabletop Simulator usa per gli sfondi panorami HDRI da 4000×2000 pixel, in un formato specifico; "ma non ce ne frega niente, perché in realtà noi possiamo fregare il sistema": come sfondo si può dare un'immagine di **un pixel per un pixel**, monocromatica, e si ottiene uno sfondo monocromatico attorno al tavolo. Ruggero ne ha già preparati alcuni: dal menu **Modding**, alla voce **Cloud Manager**, si vede tutto quello che si è caricato in cloud ("utile anche a voi per sapere cosa avete caricato"), e lui ha una cartella di immagini utili con, tra l'altro, uno sfondo nero da un pixel, "leggerissimo da caricare". Copiato e incollato nel *Custom Background*, il tavolo si ritrova "in un nulla cosmico". Ma la luce non cambia ancora: è ancora quella dello sfondo precedente.

Le impostazioni di luce. Dal menu **Options** si apre la voce **Lighting**. La prima cosa da fare è **slegare la luce dallo sfondo** (c'è una spunta apposta) e scegliere il **gradiente**: la visibilità migliora subito. Poi Ruggero va "voce per voce" (tabella 2.8.3): "so che vi sembra complessa questa roba qui, ma non vi servirà sapere tutto".

Come si vede la differenza. Per mostrare la differenza tra le due intensità, Ruggero porta prima al massimo e poi al minimo l'*ambient intensity*: "sembrano simili, ma non lo sono". Il trucco è guardare un cubetto giallo: cambiando la *light intensity*, i colori dei **bordi** restano uguali, perché è la luce verticale; cambiando l'*ambient intensity*, che è la luce laterale, la faccia **superiore** resta uguale. Per la *reflection*

[10] "DPA Framework" nella trascrizione; con ogni probabilità il materiale sul framework DPE (*Design, Play, Experience*) visto nelle lezioni precedenti.

[12] La trascrizione rende l'ultima voce come "loot loot contribution" e "Gaston, loot"; dovrebbe trattarsi delle impostazioni della LUT (*look-up table*), la tabella di correzione del colore. Ruggero aggiunge che chi è appassionato di grafica 3D può andare a vederle "in privato".

Voce	Che cosa fa e come impostarla
Luce legata allo sfondo	da **togliere**, scegliendo il gradiente
Light Intensity	la luce **dall'alto**, verticale: alta "brucia tutto", bassa "non si vede niente"; di solito verso il fondo della barra, ma non al minimo
Light Color	di default una luce calda, un bianco non puro: "lasciate questa, è una buona luce"
Ambient Intensity	la luce **laterale**, dell'ambiente: cambia i bordi degli oggetti, non la faccia superiore; va bilanciata con la precedente "in modo che non vi faccia sputare gli occhi"
Gradient Color (Sky, Equator, Ground)	"non toccateli"
Reflection Intensity	quanto riflettono i componenti **metallici** (tavolo, dadi, pezzi): abbassata, il tavolo di metallo "non riflette più niente" e un dado metallico diventa opaco
LUT e voci successive	"ignorate queste cose, per carità. Non apriamo questo vaso di Pandora"

Table 2.8.3: Le impostazioni di luce di Tabletop Simulator (menu *Options*, *Lighting*) con i consigli di Ruggero[12].

intensity Ruggero cambia tavolo ("lasciamo stare che sono volati via 38 componenti insieme, perché il tavolo l'ho rimpicciolito") e mostra che cambiano solo i componenti metallici, "dadi inclusi": un dado "nato metallico" sembra cambiare colore e diventa "bello matte".

Il regolamento sul tavolo. Tornando al PDF: lo si sfoglia con le freccette, lo si ingrandisce, lo si mette a schermo. "Questo è come importiamo i regolamenti dei nostri giochi": se il gioco ha un regolamento in PDF, si carica con il Custom PDF, "pulito, liscio, limpido, lampante".

Modelli e asset bundle: no. **Custom Model** e **Custom AssetBundle** servono a caricare modelli 3D personalizzati, per esempio miniature. Non si vedranno a lezione e non servono in fase di prototipo: "non solo non vi verrà richiesto, ma vi verrà fortemente incentivato a non usarli, a meno che non abbiate proprio un obbligo particolare".

2.8.17 Token e tile

Che cos'è un token. "Sapete cos'è un token?". "So che sembra una risposta banale, ma ci aiuta". Una studentessa risponde: sono "tipo quelle monetine che c'erano in alcuni giochi". Sì, per esempio; ma fisicamente che cos'è? Un gettone, sì, e di solito è qualcosa che viene **estratto da una fustella**, una *punchboard*: nei giochi da tavolo i token sono pezzi di cartone fustellato. "Se avete comprato un gioco da tavolo nuovo, probabilmente li avete visti: li spingete via con le manine ed estraete questi tokenini di cartone". Su Tabletop Simulator è la stessa cosa, solo che non li si estrae da una fustella.

2.8.17.1 Il Custom Token

Ruggero crea un token con un'immagine qualunque, il logo di Audacity: "voilà, abbiamo il nostro token". Le sue proprietà:

- ha uno **spessore**, che dalle impostazioni custom si può aumentare (al massimo diventa "un bel tokenone grosso");
- si ridimensiona con `+` e `-`, come gli altri componenti;
- può stare **in piedi** anziché coricato (*Stand Up*); "notate che è molto strano, vedremo dopo perché";
- può essere **impilabile** (*Stackable*).

Impilabile o no. La dimostrazione: due copie dello stesso token, una normale e una impilabile. I token non impilabili messi uno sopra l'altro formano una pila "fisicamente", ma ciascuno si sposta isolatamente. I token impilabili, invece, vengono **impilati e contati** da Tabletop Simulator: passando sopra la pila con il mouse, il software dice che è di 14 token.

Le proporzioni. Il token **mantiene le proporzioni** dell'immagine: un token da un'immagine rettangolare (una "B") è rettangolare, non viene stirato.

Il difetto: il retro è specchiato. Quando si gira il token, la "B" è **al contrario**. Con un token che porta delle scritte (Ruggero ne prende uno con una scritta) il problema è evidente: girato, la scritta è rovesciata. E nelle opzioni di importazione non c'è modo di dire "non flipparmi il retro al contrario": con il Custom Token **non si può importare un componente con fronte e retro corretti**.

2.8.17.2 Il Custom Tile

Per avere un retro corretto si usa il **Custom Tile**, la **tessera**. Chiede cose simili ma non identiche: un'immagine **top** e un'immagine **bottom**, cioè che cosa c'è davanti e che cosa c'è dietro. Ruggero crea un tile con la stessa immagine scritta del token e lo porta alla stessa dimensione, così da confrontarli. Le differenze:

- il token ha il bordo del colore del fronte, il tile ha il bordo **bianco**;
- girati, il tile **mantiene la scritta corretta**;
- il tile può avere un **retro diverso** dal fronte;
- il tile ha opzioni di **forma**: quadrato, **esagonale**, **circolare**, **rounded** (rettangolare con i bordini arrotondati); il token non le ha.

"E quindi la domanda è: perché mai dovrei usare un token, quando posso usare un tile?".

Il colore. La prima risposta è il colore. Colorati entrambi di giallo (poi di blu), il **tile** colora **solo il bordo**, mentre il **token** mette un **filtro** colorato su tutta l'immagine. Quindi, se si vogliono tre token con la stessa immagine, uno rosso, uno giallo e uno blu, il token permette di colorarli; e su un'immagine bianca il colore viene applicato preciso (un rettangolo bianco colorato di bianco resta il colore di base).

La trasparenza. La seconda risposta è la trasparenza. Il token permette di creare segnalini con la **trasparenza**, cioè **forme personalizzate**; il tile la trasparenza non la gestisce, e la stessa immagine darà sempre una tessera con lo sfondo bianco. Da cui la regola (tabella 2.8.4): **forma custom, token; forma regolare (circolare, quadrata, arrotondata), spesso è più comodo il tile**. "Se avete dubbi o domande, vi prego, chiedetele, perché da adesso in poi questo software fa solo che complicarsi la vita".

Le sagome di persone. Una domanda: le sagome di persone sono immagini trasparenti fatte a token? Sì: nel Custom Token c'è l'opzione **Stand Up**, e con un bordo un po' più ampio si ottiene una sagoma in piedi ("dev'essere molto grosso, questo però, capiamoci"). Se il bordo è impreciso, è un lavoro "di finezza" sulla **Merge Distance**: modificandola, il software ritaglia bordi più o meno accurati in base a che cosa vede nell'immagine.

2.8.18 Il dado custom

"E ora arriviamo al punto che è traumatico. Ma ci siamo". Mancano due componenti, il **dado custom** e il **mazzo custom**.

	Custom Token	**Custom Tile**
Forma	libera, dalla trasparenza del PNG	regolare: quadrata, esagonale, circolare, arrotondata
Trasparenza	sì	no (sfondo bianco)
Retro	lo stesso del fronte, **specchiato**	immagine *top* e *bottom*, corretta e anche diversa
Colore (Color Tint)	filtro su tutta l'immagine	solo il bordo
Bordo	del colore del fronte	bianco
Altre opzioni	spessore, *Stand Up*, *Stackable*	spessore, forma
Quando usarlo	segnalini sagomati, *meeple*, standee, tabelloni (con spessore minimo)	tessere, segnalini con scritte su entrambi i lati, plance

Table 2.8.4: Token e tile a confronto, secondo la dimostrazione di Ruggero.

Un'immagine per sei facce? Il **Custom Dice** chiede prima che dado si vuole (d4, d6, d8, d10, d12, d20), poi **un'immagine**. Una sola immagine per sei facce? Che immagine? Provando con un'immagine a caso, il software la **spalma** su tutto il dado. "Al che la risposta è: Tabletop Simulator, che cosa hai bevuto stasera? Come faccio a sapere che immagine devo caricare su questo dado?".

I template nascosti. La risposta sta nei file di installazione. Da **Steam**: tasto destro su Tabletop Simulator nella libreria, **Proprietà**, **File installati**, **Sfoglia**. Si apre la cartella del software, e qui l'avvertenza è perentoria: "modificare questi file fa esplodere Tabletop Simulator. Quindi non cancellate niente, non toccate niente". Dentro c'è la cartella **Modding**, con *Deck Builder*, *Deck Templates*, *Dice Templates* e altro. Nei **Dice Templates** ci sono le **UV map** dei dadi ("maledetti, infami"...).

Che cos'è una UV map. Per chi non ha mai fatto modellazione 3D, Ruggero usa un'immagine: i cioccolatini a forma di coniglio o di Babbo Natale avvolti nella carta metallizzata. Srotolata, la carta è un'immagine piatta, ma avvolge una forma tridimensionale. Allo stesso modo, il template è "la carta che va messa attorno a questo dado": dice dove sono, nell'immagine piatta, le facce che interessano.

La procedura. Si fa **una copia** del template del d6 ("non modificate direttamente quel file, vi prego"), la si carica così com'è, "e bum": il dado custom ha le facce di un d6 normale ("a questo punto usa il dado base, che era anche meno brutto da vedere"). Ma l'immagine si può modificare, con Photoshop, GIMP, "ma anche Paint". Ruggero la apre con Paint e fa "un lavoro approssimativo, fatto davvero male, giusto per capirci": i numeri pari rossi, i dispari verdi. Salva, ricarica, "e voilà: abbiamo il nostro bel dado. Bello è un eufemismo, chiaramente. Brutto, ma ce l'abbiamo". Le facce si possono quindi riempire di simboli o di qualunque altra cosa.

Dov'è la cartella? Uno studente si è perso: qual è la cartella? Ruggero ripete: libreria di Steam, tasto destro su Tabletop Simulator, Proprietà, File installati, Sfoglia; poi *Modding* e *Dice Templates*. "Lo so, è brutto. È così, va così".

2.8.19 Il mazzo custom

"Ora arriva la parte più difficile di tutte, che purtroppo è quella più usata": se si fa un gioco da tavolo, di solito si fa un mazzo di carte personalizzato. "Questa parte qua è un inferno, ve lo dico per prepararvi psicologicamente". Ruggero libera metà del tavolo ("buttiamo tutto qua in alto, anche la ciotola") e apre il **Custom Deck**.

2.8.19.1 Le opzioni di importazione

Il software chiede, nell'ordine:

La forma delle carte: rettangolari con i bordi arrotondati, rettangolari senza, esagonali, circolari. Si lascia *Rectangle (Rounded)*, "le classiche da poker".

La facciata (*Face*): l'immagine dei fronti.

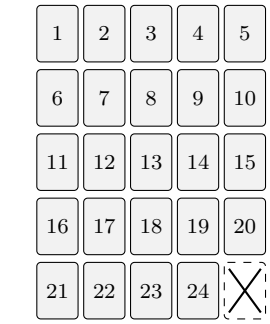
Unique Backs: se i retri sono **tutti uguali** (come nelle carte da poker) o se **ogni carta ha il suo retro**, come in certi giochi in cui "un evento che puoi girare quando succedono cose ha dietro un evento diverso". L'opzione più comune è nessun retro unico.

Il retro (*Back*).

Larghezza, altezza e numero: *Width*, *Height* e *Number*.

La griglia. Le ultime tre voci non sono centimetri: sono numeri che vanno da 2 a 10 (la larghezza), da 2 a 7 (l'altezza) e da 2 a 70 (il numero). Il motivo: su Tabletop Simulator **ogni mazzo è una griglia di carte**, un'unica immagine che le contiene tutte, e larghezza e altezza sono le dimensioni della griglia in carte. Una griglia 5×5 contiene 25 posizioni, e nel numero si scrive **24**.

Perché 24 e non 25. Per "motivi vestigiali", **l'ultima posizione della griglia non è usabile**: un tempo era usata per il retro, prima che venisse aggiunta l'opzione del retro separato, e cambiare tutto "farebbe saltare i vecchi moduli". Usarla dà problemi. Ne segue che il massimo di carte in un mazzo è $10 \times 7 - 1 = \mathbf{69}$. Serve un mazzo più grande? Lo si importa in **due mazzi separati** e poi li si mescola (figura 2.8.4).



Ogni mazzo è una griglia.
- larghezza da 2 a 10, altezza da 2 a 7
- l'**ultima cella** non si usa (era il vecchio retro)
- massimo $10 \times 7 - 1 = 69$ carte per mazzo
- più carte: due mazzi, poi si mescolano
- immagine della griglia: al massimo 10.000 pixel
- celle vuote in più: carte nere inutili

Figure 2.8.4: La griglia di un mazzo custom nell'esempio di Ruggero: 5×5, con 24 carte e l'ultima cella inutilizzabile. Ricostruzione schematica.

I template dei mazzi. Nella stessa cartella dei template dei dadi ci sono i **Deck Templates**, con esempi di griglie (anche qui l'ultima carta non è usata). Servono a poco: danno carte rettangolari e carte quadrate, ma "che dimensione sono queste carte? Io magari ho carte di dimensione diversa, strette e alte. Come faccio?". La griglia si può comporre a mano con qualunque programma, anche Photoshop, mettendo le carte "a manina una di fianco all'altra", "ma impazzite senza se e senza ma".

2.8.19.2 Il Deck Editor

Per questo, nella cartella *Modding*, alla voce *Deck Builder*, Tabletop Simulator ha un **software secondario**: il **TTS Deck Editor**. Prima pessima notizia: **non esiste per Mac**. "Spero che abbiate dei compagni di gruppo con Windows". (E Ruggero ricorda che fare un gioco da tavolo non è obbligatorio: "potete fare anche giochi pervasivi, LARP, giochi di ruolo".)

Il Deck Editor "è questa orripilezza qua, sembra fatto da un ingegnere". La procedura che Ruggero mostra (figura 2.8.5):

1. **Nuovo mazzo**: il programma chiede larghezza e altezza della griglia, "esattamente quelli che TTS ci chiede". Ruggero sceglie 5×5. La griglia è "grigia su grigia", quindi non si vede ("un altro aspetto della brillantezza geniale dei programmatori"): compare solo quando ci si trascinano dentro le carte.

2. Si **trascinano** le immagini delle carte nella griglia e le si copia quante volte serve. Nell'esempio: quattro copie dei numeri dal 2 al 6 e quattro Jack, "a caso, non è un mazzo reale".

3. **File, Save Deck As**: si salva nella propria cartella di lavoro un file `.tsdb` (Ruggero lo chiama "mazzo"). Ma Tabletop Simulator **non apre i** `.tsdb`: quel file serve solo a riaprire il mazzo nel Deck Editor.

4. **File, Export**, con due accorgimenti: **togliere la spunta da Upload Image** (le immagini si caricano a mano, non online) e impostare **Max Deck Size a 10.000**. È "una *safety measure*": se le carte sono gigantesche, il programma le ridimensiona in modo che la griglia non superi i 10.000 pixel, perché "Tabletop Simulator crasha con le immagini più grosse di 10.000 pixel". Si esporta e si ottiene **l'immagine della griglia**.

5. In Tabletop Simulator, nel Custom Deck: l'immagine della griglia come fronte (in cloud), un retro generico ("questo blu mi va bene"), griglia 5×5, numero 24.

6. **Sideways**: è un'opzione vestigiale, "non spuntatela"; serviva ai vecchi mazzi, che non potevano avere carte orizzontali.

7. **Backs is hidden**: **va spuntata**.

8. **Import**, "ed eccolo qua: abbiamo il nostro mazzo".

Backs is hidden e le mani. Ruggero mostra a che cosa serve l'ultima spunta. Dà delle carte alla propria mano e ad altre mani: le carte esistono, ma lui non vede che cosa c'è nelle mani altrui. Cambiando colore e passando al giallo, vede il giallo e non vede più l'arancione. *Backs is hidden* significa questo: quando una persona ha una carta in mano, quella carta è nascosta, e nelle mani degli altri si vede **il retro** delle carte anziché il fronte.

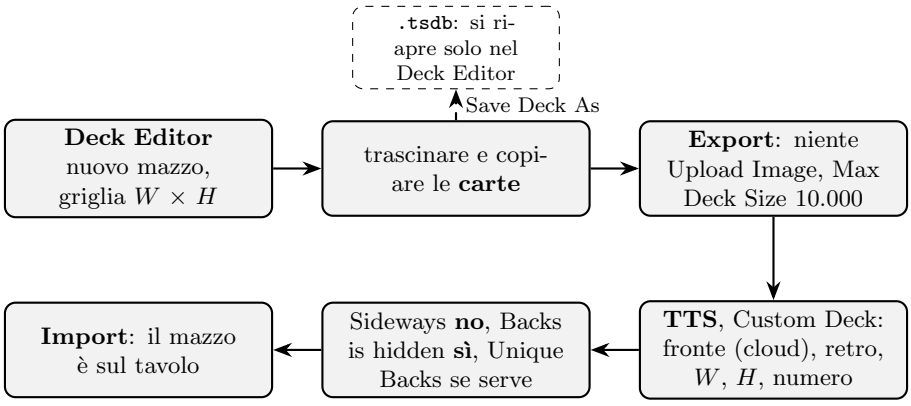


Figure 2.8.5: Il flusso di lavoro per creare un mazzo custom, come lo mostra Ruggero. Il file `.tsdb` conserva il mazzo per le modifiche successive; a Tabletop Simulator serve l'immagine esportata.

2.8.19.3 Le domande di Gianmarco

Gli angoli arrotondati. Gianmarco si è perso una cosa: l'arrotondamento degli angoli delle carte l'ha fatto Ruggero nelle immagini? Sì e no. Ruggero ha scaricato da internet un kit di carte singole, "ce ne sono mille online", già con gli angoli arrotondati; ma nelle opzioni custom le carte sono impostate come arrotondate. Impostandole rettangolari senza arrotondamento, gli angoli diventano neri, perché il software

riempie con il nero la trasparenza dell'immagine. Quando invece le si imposta arrotondate, è il software ad arrotondarle "a manina", e infatti l'arrotondamento del mazzo è un po' meno accentuato di quello delle immagini.

Più celle che carte. "È una domanda veramente stupida": che cosa succede se si mette una griglia 6×6 e poi un numero diverso dalle carte reali? Ruggero lo mostra sul 5×5, impostando 35 carte: "ti dà carte nere che non servono".

2.8.19.4 Carte con retri diversi

Finora le carte avevano tutte lo stesso retro. Per un mazzo in cui **ogni carta ha il suo retro**: Ruggero copia il file `.tsdb` ("lo chiamo mazzo retro, così lavoriamo su un mazzo diverso") e vi compone la versione "dark" delle stesse carte, nelle stesse posizioni (i numeri dal 2 al 6 neri, e il Jack). Nel Custom Deck, come fronte resta la griglia di prima; come retro si carica la **seconda griglia, che dev'essere identica** nella disposizione; si spunta **Unique Backs**. Importato, il mazzo ha "ognuna il suo retro". È anche la risposta alla domanda di uno studente sulle carte a doppia faccia, come quelle di *Taboo*[13].

2.8.19.5 La carta singola e le carte bucate

La carta singola. Il **Custom Card** chiede semplicemente fronte e retro, "senza tutto il resto". Ruggero crea una carta con il re bianco davanti e il re nero dietro: una carta singola, che si può mettere insieme alle altre.

Si possono bucare le carte? Una studentessa chiede se le carte possono essere forate, "tipo una sorta di stencil"[14]. Sì, perché Tabletop Simulator gestisce la trasparenza. Ruggero copia il re, lo apre in Photoshop ("per bucare le cose sono più comodo") e ci fa un buco e un taglio scanalato di fianco; basta un qualunque PNG con della trasparenza. Ricaricata, la carta **è bucata**: si vede attraverso. "Attenzione, però": è bucata **l'immagine**, ma non è bucata la **collisione**; un oggetto appoggiato sopra il buco non ci passa. E il retro può anche non essere bucato: Ruggero ha bucato solo uno dei due lati, "una carta che non può esistere fisicamente". La carta ha comunque uno spazio fisico che non si può cambiare. Se si vogliono carte con uno **spazio fisico bucato**, si usa il **token**, che gestisce la trasparenza *anche* nella collisione: un token ingrandito, appoggiato su un altro oggetto, non collide dove è trasparente.

2.8.20 Il tabellone senza cornice

Resta il problema della plancia con la cornice di legno (sezione 2.8.10). "Già lo sapete a questo punto come si fanno i tabelloni": **non li si importa come plance, li si importa come token**. Si crea un token con una **profondità molto bassa**, gli si dà l'immagine del tabellone, lo si ingrandisce in formato tabellone e lo si **blocca** sul tavolo. "Anche se tecnicamente, per TTS, è un token, questa per noi è una plancia".

La prima pausa. "Ora vi darei forse dei minuti di pausa prima di passare alla prossima fase, se vi va". "Se non è già troppo tardi, forse vi ho già consumato troppo". Pausa di sette minuti: si riprende alle **18:40**.

2.8.21 Penna, testo e fogli di carta

Al rientro Tiziana saluta ("nessun problema, Tiziana, va benissimo") e, a quanto si capisce, lascia la lezione. Ruggero riapre alle domande su quanto visto finora: caricare componenti, aggiungerli al tavolo.

[13] "Tabu" nella trascrizione.

[14] La trascrizione dice "una sorta di stendarda" e poi "una sorta di scherzo"; dal seguito si capisce che si parla di carte con dei fori, attraverso cui si vede ciò che sta sotto.

Esistono delle penne? Una studentessa chiede se esistono delle penne, per i giochi in cui bisogna appuntare dei punti o scrivere delle cose. Sì: c'è lo strumento **Draw**, la penna, in alto a sinistra, di cui si può cambiare il colore e con cui si disegna e si scrive sui componenti. (Ruggero si accorge di non aver ancora ricondiviso lo schermo dopo la pausa: "giustamente, se no non vedete".) Ma la penna non è un componente che si aggiunge: è uno degli **strumenti base** di TTS, come il *Grab* e lo strumento **Testo**. Sono strumenti usati solo nei giochi che li richiedono, per esempio i *party game* in cui si disegna.

Il foglio di carta. Quello che manca è il foglio. Ma il foglio può essere un **tile** o un **token bianco** su cui si disegna con la penna nera; oppure si può usare una texture di carta. O, ancora, si apre un **tavolo custom** con una texture bianca: allora "tutto il tavolo è il nostro tabellone".

2.8.22 Salvare e pubblicare il modulo

Il modulo ha luce, tavolo, sfondo e componenti. Come lo si salva?

Save & Load: solo in locale. La prima opzione è nel menu **Games**, alla voce **Save & Load**. Ruggero salva il modulo come "Prototipo"; il salvataggio compare nell'elenco e, cliccandoci, ricarica il modulo così com'era, anche dopo aver cambiato stanza: "si ricorda tutto quello che c'era". Ma un salvataggio è **usabile solo in locale**: non lo si può mandare a un *publisher* perché lo valuti. Per questo bisogna caricarlo come modulo sul **Workshop** di Steam, "e fare questa cosa è un po' più complesso".

2.8.22.1 Upload e Update

Dal menu **Modding**, la voce **Workshop Upload** apre una finestra con due schede:

Upload Workshop si usa **solo la prima volta** che si carica il modulo;

Update Workshop si usa per **tutti gli aggiornamenti** successivi, "ed è più semplice".

Le informazioni del modulo. Prima di caricare conviene compilare le **Info** del modulo (dal menu del gioco): nome, tipologia, complessità, tempo di gioco, numero di giocatori, *tag* del Workshop. "Queste sono informazioni meccaniche del Workshop". Ruggero imposta: tipo *game*, complessità bassa, 30–60 minuti, da 2 a 4 giocatori, nome "prototipo brutto". (Nel farlo, si accorge di aver aggiunto per sbaglio un modulo: "è un inferno, adesso".)

Il caricamento. Nell'Upload si scrivono il **titolo** ("prototipo brutto"), una **descrizione** ("quello che volete, lasciamola uguale al titolo") e un'**immagine di anteprima**: "mi sembra *fitting* metterci la texture del legno". Si fa Upload; dopo qualche secondo compare in alto una scritta verde: il modulo è caricato. Un *pop-up* mostra un numero, l'identificativo del Workshop, e raccomanda di conservarlo. "Non ce ne frega niente": si recupera quando si vuole, "non dobbiamo salvarlo o esplodere il mondo".

La pagina del Workshop. Nella libreria di Steam, su Tabletop Simulator, c'è la voce **Workshop**; da lì, **i tuoi oggetti**, **oggetti pubblicati**. C'è "prototipo brutto", con la sua pagina, "esattamente come le altre, solo che più brutta", con il tasto **Iscriviti** come gli altri moduli. In alto, però, c'è scritto che la visibilità attuale è **nascosta**: solo l'autore può vederlo.

2.8.22.2 La visibilità: unlisted

Dalla pagina si cambia la visibilità: **pubblico**, **solo amici**, **non in elenco** (*unlisted*), **nascosto**. Le opzioni che servono sono due; "le altre sono delle mezze vie che non servono a nessuno".

Pubblico: chiunque può entrare nel modulo, guardarlo, copiare le cose, prendere i componenti, salvarsi oggetti e immagini.

Unlisted: lo vede solo chi riceve il **link diretto**. L'URL del modulo è in alto nella pagina; oppure, tasto destro sullo sfondo, *Copia URL della pagina*.

Per questo corso, i moduli si pubblicano unlisted, "se dovete farlo". La pagina del Workshop si può modificare come quelle degli altri giochi (Ruggero ne apre una come esempio): descrizione, titolo, altre immagini, screenshot, "paro paro".

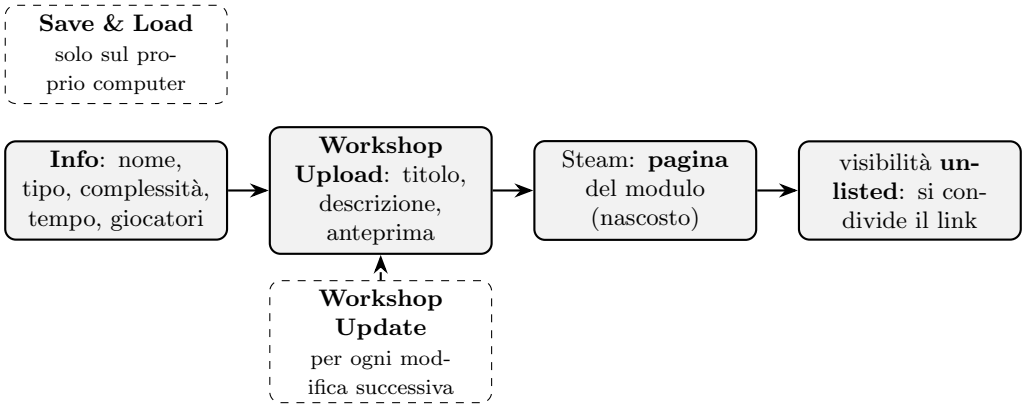


Figure 2.8.6: Dal salvataggio locale al modulo condivisibile, secondo la procedura di Ruggero. La prima pubblicazione si fa con l'Upload, tutte le successive con l'Update.

2.8.23 Prendere i componenti degli altri

Save Object. Una studentessa chiede: quindi chiunque potrebbe prendere i componenti di altri giochi, se li si fa pubblici? "Ah, sì, certo. Anzi, adesso arrivo a parlarvi di questo". Se serve un componente particolare trovato in un altro modulo, lo si può salvare. Ruggero apre un modulo pubblico[15] che ha dei token a forma di animale e prende una **gallina**: tasto destro, **Save Object**, e la si chiama "gallina". Tornati nel proprio modulo, dal menu Oggetti c'è l'ultima voce che non aveva ancora nominato, **Oggetti salvati**: la gallina è lì. "Così come io ho salvato un granchio col coltello, per esempio. Può sempre essere utile". "Ho anche un granchio senza coltello".

Dove finiscono gli oggetti salvati. Una studentessa ha salvato un mazzo di carte, con immagini caricate da lei, ma non lo trova più. Se ha fatto un *Save Object*, dovrebbe trovarlo negli oggetti salvati; caricare le immagini da qualche parte, da sola, non salva l'oggetto. In ogni caso, **tutto quello che si è caricato** è nel menu **Modding**, alla voce **Cloud Manager**. È anche buona pratica farci un po' di pulizia.

2.8.23.1 Quando la sorgente sparisce

Gli oggetti salvati restano utilizzabili **finché chi ha caricato l'immagine la tiene online**. Ruggero ha appena cancellato dal cloud un'immagine (una carta, "lanciata nel sole") e vuole mostrare che cosa succede: ricaricando il modulo dovrebbero comparire degli errori. Ma sul suo computer non succede, perché le immagini sono ancora nella *cache*, e per svuotarla dovrebbe chiudere e riaprire il software ("non ho voglia"). Allora condivide il link del Workshop perché qualcuno lo provi.

Discord, non la chat. Prova a scrivere il link nella chat della lezione; "No, su Discord, Ruggero, altrimenti non possiamo copiarlo e incollarlo". Ha ragione: gli anni scorsi non era così, ma la piattaforma ha cambiato le impostazioni di sicurezza ("con la questione delle IA hanno aumentato le difese, ma verso cosa? Comunque, questi sono i miei rant"). Uno studente vorrebbe provare a condividere lo schermo, ma è collegato dal computer aziendale, che non gli permette di installare Steam. Ruggero allora descrive l'effetto: chi carica

[15] "Farm Race" nella trascrizione: il titolo del modulo non è verificabile.

un modulo a cui manca la sorgente di un'immagine vede un **messaggio di errore rosso** in alto, ripetuto nella chat del gioco, e il componente può non caricarsi affatto.

La gallina era un modello 3D. Il caso della gallina è istruttivo. Aprendo il suo menu custom, si scopre che non è un token: è un **modello 3D custom**, a cui qualcuno ha dato una **mesh**, un'immagine **diffuse** e un **collider**. Per non perderla, bisognerebbe salvare sul proprio computer queste tre cose separatamente: "roba da uscirci impazziti". Quindi: **attenzione a che cosa si salva**, e da dove viene.

I moduli di meeple. Esistono però moduli che fanno "il lavoro sporco" per gli altri. Ruggero ne usava uno "che era una bomba atomica", in cui qualcuno aveva caricato più di cento modelli 3D di *meeple* di forme diverse; lo riapre, il software va in crash, e quando lo riapre davvero ne è sopravvissuta circa la metà: "RIP". Ma cercando "meeples" nel Workshop di Tabletop Simulator si trovano vari moduli che hanno l'unico scopo di fornire segnalini di varie forme[16]: un riccio, un omino col cappello, barche, camion, stivali, "tutti ricolorabili". "Avete l'imbarazzo della scelta".

2.8.24 I moduli di Ruggero

"Ora vi faccio vedere cosa si può fare su Tabletop Simulator quando si lavora per fare dei moduli": i moduli che Ruggero ha fatto in passato, "dai più *easy* ai più complessi".

2.8.24.1 Un gioco di carte

Il primo è un gioco "molto *card based*" fatto per un lavoro[17]. C'è il **regolamento** in alto, che si apre a doppia pagina; **nove mazzi** di carte; un **tabellone** con i segnalini che segnano le risorse e che **scattano in posizione** ("non stanno a metà via"); sacchetti con i dadi; plance con i tracciati e i segnalini delle risorse. "Liscio, regolare: tutte cose che in teoria dovreste già saper fare anche voi".

Ruggero ne approfitta per interrogare la classe:

- La **bandiera**, che componente è? Un **token**, perché ha una forma non regolare; e infatti, girata, è specchiata dietro. Lo stesso vale per un segnalino col numero 5, specchiato sul retro.
- Questi segnalini rotondi? Sono **tile** rotondi: il retro non è specchiato.
- La **plancia**? È una tessera.
- Le carte sono mazzi custom. Un mazzo è di **18 carte**: che griglia ha usato? Uno studente propone 7×3, "e poi ne butti via due"; un altro 10×2. È un 10×2. Con 7×3 si sprecherebbero tre celle (21 meno 18), e una comunque non si può usare; 9×2 non basta, perché l'ultima cella non è usabile; con 18 carte le celle sprecate sono almeno due. "Potevo fare anche un 5×4".

2.8.24.2 La demo di un gioco grande

Il secondo è il modulo completo della **demo** di un gioco da tavolo grande, legato a *Cyberpunk*[18], che ci mette un po' a caricare perché "ha tonnellate di roba": un tabellone gigantesco con segnalini ovunque, pile di token di forma custom, miniature su miniature, carte di diversi mazzi. Il *publisher* voleva anche la **scatola**, "perché volevano far vedere quanto bella è la scatola"; il regolamento è sospeso in aria, "perché la gente sta male. Richieste del publisher, ma va bene". Ci sono token, carte custom con fronte e retro separati: cose più complesse per forme e quantità, ma che gli studenti sanno ormai fare. Le **miniature** invece sono *asset bundle*: quelle no.

[16] La trascrizione cita un modulo che rende con "Wooden Talker" e un altro, "Mipolorama", probabilmente un gioco di parole su *meeple*; i titoli esatti non sono ricostruibili.

[17] La trascrizione dice "quando lavoravo per Camon"; il nome del committente non è ricostruibile.

[18] La trascrizione dice "il modulo completo della demo di Cyberpunk. Destiny 7"; più avanti Ruggero cita il suo lavoro su "Marvel United e Cyberpunk" e una cartella "Cyberpunk" nel suo Cloud Manager. Il titolo esatto del gioco non è ricostruibile dalla trascrizione.

Che cosa c'è di strano? "Chi la nota?". Uno studente: non c'è il tavolo. Esatto: Ruggero usa il **non-tavolo**, l'opzione *None*. È il motivo promesso all'inizio (sezione 2.8.6). Le mani dei giocatori si possono spostare dove si vuole; e siccome il software è un simulatore fisico, il tavolo lo crea lui: un **token gigantesco**, molto piatto e sottile, con una texture, reso **non interagibile** e **bloccato**. La texture sfuma verso il nero, e con lo sfondo monocromatico nero il bordo del tavolo si confonde con lo sfondo e non si nota. In streaming si vede poco; cambiando lo sfondo, si vede subito.

2.8.24.3 Gli ancoraggi

Una cosa che Ruggero non ha spiegato, "perché non vi servirà per questo prototipo", sono gli **snap point**, i **punti di ancoraggio**. In certe celle, un segnalino o una carta appoggiati si posizionano da soli nel punto giusto: "questo token, quando va avanti, si ferma lì". Attivando lo strumento degli ancoraggi (un puntino verde in fondo al menu), si vede che tutti i tracciati del modulo e delle plance sono ancorati.

Come si fanno. "A mano", spiega Ruggero: si copia l'oggetto più volte, lo si allinea con lo strumento **Gizmo**, che dà una precisione maggiore, e quando una copia è nel punto giusto ci si clicca sopra con lo strumento degli ancoraggi, che crea un ancoraggio lì: "questo componente scatterà esattamente qua". Poi si cancellano le copie in più, e gli ancoraggi restano. "Uno sbatti infinito. Però si può fare". Gli ancoraggi valgono per un solo tipo di componente? No: li si può dedicare a vari componenti, anche con dei **tag**. Ruggero mostra un gioco "devastante", in cui ogni casellina è un ancoraggio per delle piccole buste, e le teste di certi pezzi vanno ad agganciarsi al loro posto.

Che cosa si chiede al corso. "Noi non vi chiediamo questo livello di pulizia del prototipo". Ruggero mostra allora un prototipo su cui sta lavorando in quel periodo: è ancora grezzo, le icone e le grafiche non sono definitive, alcune carte non hanno ancora l'illustrazione ("dovrebbe avere una grafica, ok? È un prototipo"). "Non è neanche questo quello che ci aspettiamo da voi".

2.8.24.4 Un gioco di ruolo con lo scripting

Tabletop Simulator non è solo per i *board game*: si possono fare anche giochi di ruolo. Ruggero mostra un modulo fatto per un gioco di ruolo[19], in cui si è dovuto usare lo **scripting**: per esempio, bisognava poter scrivere la descrizione degli oggetti, quindi le carte sono **scrivibili**, e certi componenti permettono di cambiare i numeri.

Non è obbligatorio. Tornando al "prototipo brutto", Ruggero ripete che Tabletop Simulator probabilmente non servirà a tutti i gruppi: servirà a chi fa giochi da tavolo, magari un po' ibridi. **Non è obbligatorio** per il corso; ma per chi è interessato ai *board game*, è il software usato nel settore.

2.8.25 Perché proprio Tabletop Simulator

Una domanda "molto laterale": quanto è legato il settore a Tabletop Simulator? Se uscisse un simulatore molto migliore, lo si adotterebbe? Ne sono già usciti altri, risponde Ruggero, per esempio **Tabletopia**, ma "sono tutti peggiori in termini di usabilità". Tabletopia è gratuito, "ma fa schifo", e infatti lo usano in pochissimi, perché non ha la metà delle funzioni di Tabletop Simulator[20].

Le funzioni che non ha mostrato. E Ruggero, "per semplicità e per non farvi impazzire", ha taciuto cose molto più tecniche:

- si possono **creare nuove mani**, o ridimensionarle: una mano rossa molto più grande ("di boh, 30"), per un gioco in cui si tengono in mano tantissime carte;

[19] La trascrizione lo chiama "Magic of the Inventory"; il titolo non è verificabile.

[20] La trascrizione dice "non ha la metà delle funzioni che Tabletopia ha": dal contesto è evidente che il confronto è con Tabletop Simulator.

- si possono **incollare** i componenti tra loro con lo strumento della **catenella** (i *joint*): un dado ingrandito incollato sopra un cubetto diventa un oggetto solo;

- si possono fare **plance con alloggiamenti**, ancorati, in cui inserire altri componenti: Ruggero ne mostra un vecchio prototipo ("è brutto, ignoratelo");

- si possono giocare anche i **legacy**, grazie allo strumento degli **adesivi**: Ruggero crea un adesivo con la sua "A" e lo attacca a una carta (prima, per sbaglio, sul mazzo intero: "sono scemo"); la carta, mescolata con le altre, conserva l'adesivo.

"È che abbiamo una quantità di strumenti qua dentro che è delirante. Non ve ne ho fatti vedere neanche la metà, vi giuro": l'angolo di rotazione, l'altezza a cui si sollevano gli oggetti, "tutte cose tecniche che non vi servono adesso, ma in cui, se imparate questo software, c'è un *rabbit hole*". Non è possibile comprimerlo in tre ore. L'obiettivo di oggi è dare "il minimo necessario per riuscire a fare un gioco che abbia dei componenti, dei dadi, delle carte, un tabellone, una plancia"; per le idee strambe, si scrive a Ruggero.

Cancellare il modulo. Ultima procedura: come si cancella un modulo? "Il prototipo è troppo brutto per tenerlo". Dalla pagina del Workshop, in fondo, **Elimina**: il modulo viene cancellato dal Workshop, ma non del tutto, perché resta il **salvataggio** locale. Cancellato anche quello, "se ne va via tutto".

2.8.26 Prototipi in scala

Ruggero apre alle domande: "questa ultima parte della lezione è per rispondere a voi". Uno studente, "per curiosità": quanto sono verosimili i prototipi che Ruggero prepara per i *publisher*? Lo scopo è arrivare a una prova realistica?

Estremamente accurati. "Quelli che faccio io sono estremamente accurati": i componenti sono davvero **in scala**. Quelle carte sono in formato poker, 88 mm; quelle altre nel formato delle carte di *Dixit*[21]; le miniature hanno la base delle dimensioni delle miniature reali. Il motivo è il *playtest*: i componenti devono essere "il più simulati possibile" per avere un playtest vicino a quello reale, perché il playtest verifica anche **gli spazi**. "Mettiamo che io ho un distretto, e devo sapere se ci stanno otto miniature in questo distretto. Come faccio a saperlo, se io non le muovo?". Un playtest deve poter dire: "un po' strettine, ma ci stanno". E se gli spazi del tabellone devono contenere le carte, le carte devono essere in scala con il tabellone, e così i token e le miniature: "per forza di cose si va a creare tutto in dimensioni reali".

Si parte dal tabellone. Anche prima, nei primi prototipi, il passaggio non è diverso. Ruggero parte di solito **dal tabellone**, perché è la cosa più grande e quindi la più facile da mettere in scala. Tabletop Simulator misura in griglia, in pollici o in centimetri, **non in millimetri**: si lavora con i decimali dei centimetri, e su una carta che misura "8 virgola qualcosa" è difficile capire se sia 88, 89, 85 o 86 mm. Su un tabellone di 350 mm l'errore pesa molto meno. Quindi: prima il tabellone, poi tutto il resto in scala rispetto al tabellone.

Approfondimento: la fedeltà del prototipo. La risposta di Ruggero mostra che "prototipo grezzo" e "prototipo in scala" non sono in contraddizione. Il prototipo che ha mostrato poco prima ha grafiche provvisorie e carte senza illustrazione, ma le misure sono già quelle vere: la fedeltà *grafica* è bassa, quella *spaziale* è alta, perché è lo spazio che il playtest deve verificare. È lo stesso principio per cui Schell raccomanda di decidere, prima di costruire un prototipo, quali domande deve aiutare a risolvere: quello che non serve a rispondere può restare grezzo.

[21] "Le carte formato di exit" nella trascrizione: con ogni probabilità *Dixit*, le cui carte grandi sono un formato standard del settore.

2.8.27 Aggiornare un mazzo e tenere in ordine il cloud

Correggere una carta. L'ultima domanda dello stesso studente è "più tecnica": nei prototipi capita di dover correggere una carta. "Ottima domanda, ottima domanda". Ruggero riprende il mazzo fatto prima e finge un errore di bilanciamento: invece di due Jack servono, "che so, due Jack e due regine". Aggiorna il mazzo nel Deck Editor, lo salva, lo **riesporta**, e nel menu custom del mazzo ricarica **solo il fronte**, perché il retro non cambia. Con *Search* si vedono le due regine.

Correggere una sola immagine. Poi un caso più fine. Ruggero modifica in Paint una sola carta (la regina di quadri), salvando il file con lo stesso nome. Riaprendo il `.tsdb`, il Deck Editor riprende le carte dalla cartella e mostra già la carta aggiornata. Quindi: **se si mantengono uguali i nomi dei file**, si aggiorna la singola immagine, si riesporta nella stessa cartella da cui il `.tsdb` prende le carte, e il mazzo è aggiornato senza ricomporlo. "Anche perché questi moduli si aggiornano tante volte": il modulo del gioco che Ruggero sta per pubblicare[22] è stato aggiornato **79 volte**. È quello con la carta senza illustrazione: "appena l'illustratore me la finisce, io riaggiorno semplicemente quel file lì, e siamo a posto".

Le cartelle nel Cloud Manager. Una buona pratica: caricare le immagini in cloud **dentro delle cartelle**. Nel suo Cloud Manager Ruggero ha una cartella per ogni progetto: tutti i componenti del gioco di *Cyberpunk* in una, quelli dell'altro gioco in un'altra. "Dividervi in cartelle vi serve anche per sanità mentale", altrimenti ci si ritrova con "una sfilza di tutte le immagini che avete usato, caricate a caso".

Il limite dei 9.999 file. Ed è buona pratica **cancellare** ciò che non si userà. Non è un problema di oggi, ma Tabletop Simulator ha un **limite fisso di 9.999 file** caricati. "Sembrano tanti, e sono tanti se fate pochi giochi. Ma vi giuro che il limite lo si raggiunge. Io l'ho già raggiunto più volte". E i file si cancellano **uno alla volta**, a mano: non c'è un'opzione per cancellarne di più. "Lo so: sono pazzi".

2.8.28 Un software di settore suo malgrado

Lo stupore di Rita. Rita è colpita: per un software di questa portata, è incredibile che manchino cose così elementari. Lei è abituata ai software Adobe, "a un altro tipo di possibilità"; "però è molto interessante. La lezione di oggi mi ha stimolato un sacco. Credo che starò tutta la notte a provare". "È un software incredibile, Rita", risponde Ruggero, ma "fatto da tre appassionati che non hanno ragionato per diventare un software di settore. Sono diventati un software di settore contro la loro volontà".

Le cose stanno cambiando. Oggi ne guadagnano, e qualcosa si muove: dalla pagina della comunità su Steam, nelle notizie, Ruggero mostra la **roadmap** che gli sviluppatori hanno deciso di pubblicare dall'inizio dell'anno, "finalmente, dopo dieci anni e passa di software". Stanno aggiornando il motore, "dopo anni che glielo si chiedeva". E gli studenti hanno già visto una versione migliorata. Il Custom PDF, quando Ruggero ha cominciato, **non esisteva**: si metteva sul tavolo il **tablet** e ci si apriva un link al PDF. E se i file erano esportati in **CMYK** anziché in **RGB**, il software andava in crash "secco"; ora si limita a dire che non accetta quel tipo di file. "Per farti capire quanto fuori di testa era questa gente".

Studiare i moduli degli altri. Un consiglio: ora che si sa come sono fatti i componenti, si può aprire un modulo qualunque e studiarlo. Con tasto destro, **Custom**, su un componente, si vede **che cosa è**: "questi bellissimi tokenoni qua cosa sono? Sono dei modelli o sono dei token? Sono dei token, fantastico. Questi posso rifarli anch'io, so come farli". Su un modulo aperto come esempio[23] si vede che certi segnalini sagomati sono token, perché il retro è specchiato.

[22] La trascrizione lo chiama "Astronega" e poi "Astronica"; il titolo non è ricostruibile con certezza.
[23] "Sputacular" nella trascrizione: il titolo non è ricostruibile.

Il token fronte-retro. Esiste però un modo per fare token di forma particolare con il retro *dritto*: **incollarne due** con la catenella. "È un bello sbatti", e Ruggero non ricorda più in quale modulo l'aveva fatto. Uno studente chiede di rispiegarlo, e Ruggero lo rifà da zero: si crea un token con una forma particolare (un *meeple* giallo) e un secondo token con la stessa forma e il retro voluto (rosso); dal menu **Options** si attiva la **griglia**, che allinea i pezzi con precisione, così che siano nella stessa identica posizione, uno sopra l'altro; con la **catenella** li si aggancia; si toglie la griglia. Il risultato è un token custom con due facce diverse e il retro non specchiato.

2.8.29 Niente script

Si possono fare "cose fuori di testa": **perni mobili** (*joint*) per attaccare gli oggetti, e soprattutto **script** che automatizzino delle cose. Ed è qui che Ruggero mette l'unico divieto tecnico della lezione: **in questo corso gli script non si fanno**. Il motivo si capirà nella lezione sul playtest: "gli script vi rendono più complesso testare cose particolari, e quindi remano contro la vostra procedura di playtest". E siccome si vuole un playtest utile e strutturato bene, "quando arriveranno i playtest vi sarà chiaro perché gli script sono un problema".

Approfondimento: perché l'automazione disturba il playtest. La lezione sul playtest è ancora lontana, ma il senso del divieto si può già intuire dalla sezione 2.8.5. In un prototipo analogico i giocatori eseguono le regole a mano, e proprio eseguendole le capiscono, sbagliano, le contestano: sono i dati che un playtest raccoglie. Uno script che sposta i segnalini o calcola i punteggi toglie questi momenti dal tavolo e nasconde il carico di gestione che il gioco stampato avrà davvero. In più, ogni modifica alle regole tra un playtest e l'altro richiederebbe di riscrivere il codice: il prototipo diventerebbe più lento da cambiare proprio nella fase in cui deve cambiare più spesso.

La seconda pausa. "Abbiamo ancora mezz'ora di domande". Silenzio: "Tutto chiaro? Non è possibile, non posso essermi spiegato così bene". Uno studente prevede che la prima volta che darà un modulo a qualcuno "gli esploderà il computer". Ruggero propone un'altra pausa, fino alle **19:35**: "liberatevi un attimo il cervello, mi rendo conto che oggi è stata molto intensa". Al ritorno risponderà a domande e curiosità "anche generali, non solo sul software, a tutto tondo". Gianmarco dice che non gli vengono domande; "ma le domande verranno tra una settimana, tra un mese. È normale". Ogni anno qualcuno gli scrive giorni o settimane dopo: si può fare questa cosa? La risposta è "sì", perché Tabletop Simulator "è magia nera, è un patto col diavolo": sì, ma magari bisogna impazzire tre giorni su un modello 3D; oppure sì, ma bisogna programmare, "e piuttosto ti tagli le mani".

2.8.30 Dopo la pausa: scorciatoie, fisica e destrezza

"Rieccoci qua. Allora, come va? Come state? Io, stanco, venerdì pomeriggio, dopo tre ore di Tabletop". Se agli studenti "si spacca tutto in testa", è normale: "è traumatico". Quando Ruggero ha cominciato a usarlo, "odiarlo" era un eufemismo: non c'era la metà delle funzioni di oggi ed era molto più complesso. Ma ha "tante cose molto funzionali".

Ingrandire. Alcune cose da tutorial, che non ha mostrato: con il tasto `Alt` si **ingrandiscono a schermo** le carte e gli oggetti; con il tasto `M` si attiva la **lente** (*Magnify*). (Uno studente gli fa notare che non sta condividendo lo schermo: "giusto, hai ragione. In effetti sono pazzo".) Dal menu di `Esc` ci sono tutte le scorciatoie.

La fisica dei componenti. Ruggero non è entrato nel dettaglio del **peso** delle cose: si può cambiare la fisica dei componenti, la **massa**, l'**attrito** con l'aria, l'attrito angolare[24], e perfino la **gravità** del modulo.

[24] "La tritangolare" nella trascrizione; si intende con ogni probabilità l'attrito (o la resistenza) angolare, che regola quanto un oggetto continua a ruotare.

Lo mostra tirando dei dadi con la gravità bassissima e poi altissima. Serve, per esempio, a fare dei *dexterity game*.

Il minigolf. La gente usa il software "per fare le cose più folli". C'era un modulo di **minigolf**: si prendono le palline, si usa lo strumento *flick*, "e ok". "La gente sta male. In positivo, chiaramente, in questo caso". Su Tabletop Simulator si possono fare giochi da tavolo "nel senso più tradizionale del termine", *wargame*, giochi di miniature, giochi di destrezza: "ha un potenziale gigantesco", che però, con la sua interfaccia "un po' *cranky*, un po' *old school*" e i suoi aspetti vestigiali, spesso non arriva.

2.8.31 Un consiglio: giocate The Box

"Uno dei consigli che do sempre, se volete imparare a usare bene questo software e volete farlo giocando": giocare ***The Box***. È un gioco che **esiste solo su Tabletop Simulator**, che "non può esistere nella realtà per mille motivi che vedrete se lo giocherete". Ruggero lo apre, lo linka su Discord, e lo consiglia "a prescindere dal corso".

Perché giocarlo. "Come paradigma di game design, questo gioco vi dà una mano". È un gioco "che sulla carta non dovrebbe funzionare", con un game design che Ruggero trova "anche brutto", ma con "un'esperienza incredibile da giocare". E ragionare sul perché un gioco fatto male dia un'esperienza che può essere molto positiva "è molto utile, lato game design".

Come giocarlo. "Non leggetevi niente". Si organizza una serata (tiene fino a otto persone, "se non ricordo male"), si apre un tavolo su TTS e si legge il regolamento lì: è "una manciata di pagine", scritto "in font gigantesco", "non è bello da vedere". Il gioco è "bizzarro, strano": tre scatole, una delle quali ne contiene altre **80**, e un'altra altre **121**, con nomi bizzarri come "B126" o "diametro infinito". E fa interagire con moltissimi dei componenti visti a lezione: è l'opzione consigliata per "prendere confidenza con la varietà più disparata di componenti". ("Adesso che ve l'ho nominato, devo organizzare per giocarlo con qualcuno, perché è bellissimo".)

Approfondimento: imparare da un gioco "brutto". Il consiglio di Ruggero riprende un'idea che attraversa tutto il corso: l'esperienza del giocatore non coincide con la qualità "sulla carta" delle regole. Nel lessico dei framework visti nelle lezioni precedenti, un insieme di meccaniche discutibile può comunque produrre dinamiche e un'esperienza memorabili, e capire *perché* succede è un esercizio di analisi più istruttivo che studiare un gioco ben fatto. Schell lo formula come un'abitudine del designer: guardare all'esperienza prima che alle regole, e chiedersi quali elementi la producono davvero.

2.8.32 Saturi

"Oltre a TTS, come state?". Una studentessa è "saturatissima" di cose apprese: deve "fare un attimo ordine fra i pensieri" per capire che cosa provare per prima e che cosa chiedere. "Certamente"; e di nuovo, a Ruggero si può scrivere per mail o su Telegram. Un'altra è "satura di nozioni, non riesco più ad apprendere". Rita ringrazia: "è la lezione più bella finora per me".

Ruggero è contento: questa lezione, per molti, è "una mattonata". Le classi sono molto diverse, e capitano studenti che non hanno mai avuto confidenza con software tecnici, per cui questa lezione "è un po' una mazzata"; "sono contento, Rita, che ti sia piaciuta, anche perché abbiamo davvero sorvolato la punta dell'iceberg di Tabletop Simulator, questo ve lo assicuro".

2.8.33 Il playtest e il calendario del corso

Una studentessa ("sempre le mie domande..."): quanto è rilevante usare Tabletop Simulator nel *playtest*, e quanto invece è importante farlo **fisico**? "Ti sorprenderà sapere" che c'è una lezione sul playtest. "Immagi-

navo", risponde lei, "ma lo faccio apposta: tu mi anticipi ogni volta le cose"; e, come ha visto, nelle lezioni successive le cose vengono riprese.

Ruggero controlla quando, e già che c'è ripassa il resto del calendario (tabella 2.8.5).

Quando	Che cosa
lunedì (5 ottobre)	lezione con Beatrice, la seconda sulle **esperienze ludiche**
venerdì (9 ottobre)	lezione sul **paragame**[25], "una delle mie preferite"; si parlerà anche di TTS, "ma a tangente"
12 ottobre	il **pitch**: niente lezione, il *check* dei progetti
fino a novembre	periodo di lavoro senza lezioni
6 novembre	lezione con **Diego** sull'**RNG** (la casualità)
9 novembre	**playtest** con Ruggero: "playtest e metodi etnografici di raccolta feedback"
dopo	**gioco di ruolo da tavolo** con Daniele
dopo	"l'ondata lunga" di **sviluppo 1** e **sviluppo 2** con **Marta**: *development*, modelli matematici, bilanciamento e altri aspetti dello sviluppo
dopo	**produzione** con **Flavio**
dopo	altro periodo di lavoro
11 dicembre	**final review**
dopo la review	**tavola rotonda** con editori e persone del settore

Table 2.8.5: Il calendario del resto del corso come lo ripassa Ruggero. Le date tra parentesi sono ricostruite dal contesto; le altre sono dette a voce.

La tavola rotonda. Dopo la *final review*, nella lezione di **tavola rotonda** i docenti portano **editori** e altre persone del settore a dare un feedback sui progetti: un punto di vista che non sia solo quello degli insegnanti, ma di chi lavora nel settore. Gli ospiti si scelgono **in base ai giochi dei gruppi**. L'anno scorso, "per varie cose", non è uscito neanche un *board game*: giochi di ruolo, LARP, giochi pervasivi, ma nessun gioco da tavolo; e quindi non sono stati chiamati editori di giochi da tavolo, perché "non aveva senso dare feedback da qualcuno che lavora nel board game a un gioco di ruolo". Anche quest'anno la tavola rotonda sarà calibrata su quello che i gruppi faranno. La studentessa aggiunge che finora non ha mai strutturato i prototipi su TTS, li ha sempre fatti fisici; "non è un problema".

2.8.34 Domande e risposte, alla rovescia

"Avete altre domande, dubbi? Avete risposte? Mi date le risposte, io vi credo". Uno studente sta al gioco: "È blu, è blu, è blu". E poi: "come faccio un componente a goccia?". Ruggero ricorda un *trend* su Reddit: dare prima le risposte e poi inventare le domande, in modo che chi ha risposto sembri la persona peggiore possibile ("la mattina ne mangio due" — "quanti bambini mangi al mattino?"). "Va bene, non volevo neanche andarci. Dove stiamo andando?". La domanda sulla goccia resta senza risposta (ma si è visto prima come ottenere "gocce d'acqua" dalle pietre del Go, sezione 2.8.13, e come fare un token di forma libera, sezione 2.8.17.1).

Mancano 14 minuti. La battuta sui bambini ha inquietato una studentessa, che ha appena finito di ascoltare *Veleno*[26]: "è stato un attimo di infarto". Sono podcast italiani intensi, "tostarelli".

2.8.35 Le risorse da seguire

A proposito di podcast e risorse: Ruggero aveva suggerito delle risorse nelle slide di una lezione precedente, ma lo studente si è un po' confuso; quali sono i *must* da seguire su YouTube? Ruggero ne segue diverse, "ma per motivi molto diversi": alcune per i *game studies*, alcune per il game design, alcune per la grafica.

[25] La trascrizione dice "quella sul paragame"; il termine non è spiegato nella lezione.

[26] Il podcast d'inchiesta italiano di Pablo Trincia; la trascrizione lo rende "Veleno agli atti".

Daniel Solis è "molto bravo sulla parte di *graphic design* per i board game": il suo canale è molto tecnico su come progettare una grafica **funzionale** al proprio gioco.

Amabel Holland ha "un buon mix": fa molta "quasi archeologia ludica", ricerca storica sui giochi, ma anche video di game design "molto sul pezzo"[27].

Action Button è il suo canale preferito in assoluto. Ruggero ha una preferenza, "mi rendo conto mia", per i video-saggio molto lunghi ("se apro il mio YouTube, vedete video da quattro ore e passa"): Action Button analizza **singoli giochi** in video dalle tre alle dieci ore, andando molto a fondo non solo sul game design ma sulla **game culture** e sul **contesto**: in quale epoca il gioco è stato creato, che cosa nel mondo reale ha influenzato chi l'ha fatto, perché il gioco è uscito in quella forma.

Il video preferito. Il suo video preferito in assoluto è quello su *Tokimeki Memorial*[28]: "5 ore e 56 minuti, e lo so a memoria perché è il video che ho riguardato di più nella vita". "Mi rendo conto che è una nicchia di una nicchia di una nicchia", e potrebbe non essere per nessuno oltre a lui. È "un *roller coaster* emotivo": se ne esce tristi, arrabbiati, ma anche fiduciosi nell'umanità, soddisfatti; "molte cose provi". Lo studente vede anche un video su *Final Fantasy VII Remake*, che ha odiato: "allora amerai quel video!". Poi lo studente deve andare e saluta.

Il podcast che non si nomina. Ruggero aggiunge, imbarazzato, che ha anche lui un podcast, "ma fate come se non avessi detto niente"; non lo ha messo nelle slide perché "ho sempre imbarazzo sulle cose che faccio". Una studentessa ha già trovato il suo canale; lui insiste che non vuole fare "il prof che consiglia i propri libri" e che non parla quasi mai neanche dei suoi giochi. Sul suo sito c'è tutto. Ma il podcast non ha pubblicità né sponsor, volutamente: "non mi deve rompere nessuno. È una roba che faccio per pace mentale mia". La studentessa ("ho 30 anni, ho già studiato in tanti campi") dice di stimarlo proprio per questa scelta: "tutti invece vogliono fare i professori per vendere le loro cose", e questo modo di fare l'ha spinta ancora di più ad andare a vedere il suo lavoro.

2.8.36 Giochi invendibili e l'economia del gioco

"Io sono l'ultima persona che vi deve dire come comunicare questi giochi", dice Ruggero: comunicazione e vendita "mi fanno schifo" (lui, "come anche Daniele e Beatrice", non è lì per quello). I giochi che fa nel tempo libero "sono cose invendibili".

"Se volete perdere soldi". L'ultimo incontro commerciale in cui ha proposto a un *publisher* uno dei suoi giochi migliori, gli ha detto: "se volete perdere soldi, è l'ideale". "È terribile, non si fa"; ma è stato onesto. Ha fatto quell'incontro con un solo editore, l'unico di cui si fida al punto da proporglielo, sapendo che gli avrebbe fatto perdere soldi: "quindi devo essere molto onesto". Una studentessa chiede, senza voler sapere i dettagli, se il gioco è stato provato con delle persone: è "il mio *magnum opus*", ed è "talmente intenso che in un playtest una persona ha vomitato". Ma se alle persone piace, obietta lei, ci sarà un pubblico a cui venderlo.

I numeri del mercato italiano. La risposta è nella scala economica del settore, "molto bizzarra". In Italia, un gioco da tavolo che vende "regolarmente" vende **tra le mille e le duemila copie**; un editore è già contento se ne vende **più di mille**. Il gioco di ruolo è su una scala ancora più bassa: un manuale che vende **500 copie** in Italia è già un buon risultato. La fetta maggiore del mercato va a giochi come *Dungeons & Dragons*, che lasciano poco spazio al resto. Ruggero ha lì il suo gioco, perché deve portarlo a un evento il giorno dopo: una scatola bianca, anonima, senza niente sopra, "solo io so che gioco è". "Se ne vende 50 copie, sono 49 persone in più di quelle che pensavo potessero prenderlo".

[27] Ruggero cita anche altri "approcci" che la trascrizione rende in modo incomprensibile ("approcci cosmodiali"): un altro canale tra i suoi preferiti, il cui nome non è ricostruibile.

[28] "Took Me Me Me Me Me, Key Memorial" e "Toki Maki Memorial" nella trascrizione.

Sapere perché si fa un gioco. È il tema della prima lezione, ricorda la studentessa: bisogna avere chiaro **perché** si fa un gioco. Quando Ruggero ha lavorato a *Marvel United* e a *Cyberpunk*, giochi su IP gigantesche, il gioco doveva essere per il *mass market*, accessibile. Ma con un gioco suo, per scopi suoi, non commerciali, su cui non deve guadagnare, può fare "quello che voglio". "E non è un gioco peggiore o meno degno di esistere". Come nel cinema ci sono i film della Marvel, i documentari di nicchia e l'horror indipendente che va ai festival. La studentessa è d'accordo: è un'opera di creatività. Si scusa per averlo messo in imbarazzo; "no, sei tu, non sono io che sono inadeguato", scherza lui.

Approfondimento: obiettivi diversi, giochi diversi. Lo scambio riprende, dal punto di vista di un professionista, la distinzione tra l'obiettivo del gioco e l'obiettivo personale di chi lo fa, che il corso ha affrontato parlando del perché si fa game design. Un gioco su licenza per il grande pubblico e un gioco personale "invendibile" rispondono a obiettivi diversi, e si giudicano con metri diversi: il primo dall'accessibilità e dalle vendite, il secondo dall'esperienza che riesce a dare al suo pubblico, per quanto piccolo.

2.8.37 I giochi lunghi

"Siamo rimasti solo noi tre". Cinque minuti alla fine. Uno studente non ha una domanda ma una confessione: vedendolo nello scaffale alle spalle di Ruggero, ha comprato d'impulso un gioco lungo[29] che sa che non giocherà mai, "perché intavolare questi giochi è sempre un casino". Neanche Ruggero l'ha ancora giocato: dura molto, e alle persone non si può mentire: "guarda, secondo me almeno 4–5 ore siamo qui fissi".

Un evento per i giochi lunghi. Lo studente racconta che cosa ha fatto tornando in Puglia: ha contattato le associazioni del Sud Barese, dove vive, e insieme hanno creato un **evento a invito, a porte chiuse**, conosciuto solo dalle associazioni e riservato ai giocatori, per giocare proprio questi giochi molto lunghi. La maggior parte delle associazioni lavora con eventi aperti, in cui si fa da dimostratori dei giochi e non si può giocare davvero: un gioco così non lo si può portare quando arriva una persona che vede un gioco da tavolo per la prima volta. Il loro evento dura tutta la giornata, e hanno fatto "cose abbastanza grosse"; il prossimo piano era *Here I Stand*[30], poi saltato. Sullo scaffale ci sono diversi titoli della **GMT**: *Cuba Libre*, *A Distant Plain*, *Fire in the Lake*; e ancora da giocare un titolo strategico ambientato a Taiwan[31].

"Si gioca poco". "Il tempo per giocare è poco, c'è poco da fare". Lo studente immagina che, facendolo per lavoro, a un certo punto venga la nausea. "Ma non esiste". Ruggero gioca **400 giochi all'anno** e ne giocherebbe 800, se la gente venisse a giocare; "invece, purtroppo, le persone hanno una vita". La registrazione si interrompe qui.

Punti chiave del capitolo

- Il **pitch** del 12 ottobre non è commerciale: 40–50 minuti per gruppo per fissare un *agreement* su che cosa si farà (tipo di gioco, numero di giocatori, esperienza e come darla, durata). Forma libera; nessuna scaletta, volutamente.
- **Tabletop Simulator non è un motore per videogiochi**: è un **simulatore fisico** di un tavolo. Ogni oggetto ha massa, spazio, collisioni; i componenti vanno trattati come componenti fisici. È il software di riferimento del settore, pieno di scelte vestigiali.
- I moduli si costruiscono in **single player**. Le immagini si caricano **da file** (mai da URL) e **in cloud** (mai in locale).
- Il **tavolo** decide quanti giocatori ci stanno (le *hands*) e come si comportano i componenti sui bordi: è una questione di *table presence*. Il tavolo custom stira l'immagine; la plancia custom si adatta all'immagine, ma ha la cornice di legno.

[29] La trascrizione lo rende "Spichisi" e "Speak easy"; il titolo non è ricostruibile con certezza.
[30] "Il re stand" nella trascrizione: con ogni probabilità *Here I Stand*, gioco storico molto lungo della GMT.
[31] La trascrizione lo chiama "il 2045, quello strategico taiwanese"; il titolo non è ricostruibile con certezza.

- Lo **sfondo** cambia la luce: conviene uno sfondo monocromatico da un pixel e la luce slegata dallo sfondo, regolando *light*, *ambient* e *reflection intensity*.
- Per ogni componente: **esiste già o va creato?** Ricolorare (*Color Tint*) non ricolora tutto (i pallini dei dadi). Si lavora **in scala reale**: righello in cm, vista dall'alto, `+` e `-`. Niente **sacchetto infinito**: i componenti si devono poter contare; il sacchetto normale va in *Random*.
- **Token**: forma libera, trasparenza e collisione, colorabile, impilabile, ma retro specchiato. **Tile**: forme regolari, fronte e retro corretti, niente trasparenza. I **tabelloni** si fanno con un token sottile e bloccato.
- Il **dado custom** si fa dai template (UV map) nella cartella *Modding* dei file installati, lavorando su una copia.
- Il **mazzo custom** è una **griglia**: massimo 10×7, ultima cella inutilizzabile, quindi 69 carte. Si compone con il **Deck Editor** (solo Windows), si esporta senza upload e con *Max Deck Size* 10.000; in TTS, *Sideways* no, *Backs is hidden* sì, *Unique Backs* per i retri diversi. Mantenendo i nomi dei file si aggiorna una carta sola.
- Il salvataggio locale non si condivide: si pubblica sul **Workshop** (Upload la prima volta, poi Update), in **unlisted** per il corso. I componenti altrui si salvano con *Save Object*, ma dipendono da chi li ha caricati; il Cloud Manager va tenuto in cartelle (limite di 9.999 file).
- I moduli professionali usano il non-tavolo, ancoraggi, componenti in scala (si parte dal tabellone); al corso non si chiede quel livello. **Niente script**: remano contro il playtest.
- Dalle domande finali: giocare *The Box* per capire il software e per riflettere su un gioco "brutto" che dà un'esperienza bellissima; seguire Daniel Solis, Amabel Holland, Action Button; in Italia un gioco da tavolo che vende ne vende mille–duemila copie, un GdR 500: sapere **perché** si fa un gioco cambia il metro con cui lo si giudica.

Chapter 2.9

Organizzare la progettazione, giochi formativi ed escape room: tempi, risorse e un gioco che insegna

Questa è la **nona lezione** del corso di **Analog Game Design**, ed è una lezione di **strumenti pratici**. Dopo il design sprint della settima lezione e la lezione tecnica su Tabletop Simulator, i gruppi hanno ormai cominciato a lavorare sul **project work**, e a una settimana dal *pitch* del 12 ottobre Beatrice affronta la domanda che il design sprint aveva lasciato aperta: **come si arriva a un gioco concluso** nel tempo che si ha? La prima metà della lezione chiude le slide rimaste in sospeso sul **project management di un gioco**: le risorse da valutare (tempi, competenze, energie, soldi e materiali), il progetto pensato come una *game jam* con una scadenza che ci si dà da soli, e un documento da compilare voce per voce, dall'obiettivo fino al monitoraggio finale delle ore spese. Il principio che lo regge è semplice e ripetuto: **stimate in eccesso**, non allocate tutte le ore che avete, e tenete traccia di quanto ci mettete davvero, perché solo così, progetto dopo progetto, si impara a stimare.

La seconda metà, "Esperienze ludiche 2", è dichiaratamente una **miscellanea**: il **Game Design Document** (che cos'è, che cosa se ne dice nei manuali e che cosa succede davvero nell'industria analogica italiana), le **tre grandi domande** a cui un gioco concluso deve rispondere, la distinzione tra **gamification** e **game based learning**, con una lunga discussione sulla **volontarietà** del gioco e sulla **motivazione intrinseca ed estrinseca**, le **escape room formative** (tre strutture di enigmi, dove trovare gli enigmi, le indicazioni di un paper), la parte **burocratica** (GDPR, informativa sulla privacy, esonero di responsabilità, proprietà intellettuale) e, in chiusura, un mazzo di carte per inventare giochi nello spazio pubblico, l'**Urban Game Deck**. La lezione finisce con un giro di *feedback* sul corso: la classe è stanca, e Beatrice preferisce usare l'ultimo quarto d'ora per capire "lo *stress test* a cui vi stiamo sottoponendo".

La lezione è **online** e la conduce da sola **Beatrice**, che dice verso la fine di essere "all'ultima lezione, credo, di questo corso da sola". Si colloca con ogni probabilità **lunedì 5 ottobre 2026**[1], dalle 17 alle 20 circa (Beatrice chiude "5–10 minuti prima"). Ci sono due pause: la prima, di cinque minuti, a metà della parte sul project management, fino alle **17:36**; la seconda dalle **19:16** alle **19:21**. All'appello iniziale ci sono **Tiziana**, **Tancredi**, **Lorenzo Vergata** ("Lorenzo V"), **Nicola**, **l'altro Lorenzo** ("Lorenzo L") e **Giordano**; **Angelo** manca ("mi mancava Angelo; io però direi di no"), e più avanti interviene **Gianmarco**. Beatrice, a un certo punto, conta "noi nove persone qui in call".

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia molti nomi. Ruggero è "Ruger" e "RUG", la *game jam* è "gem" e "GEM", il *pitch* è "PIL" e "piccio", Huizinga è "Osinga", Sugata Mitra è "Sugatta Mitra", Genially è "Genialy", i *murder party* sono "Marvel Party"; dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante le due

[1] La data non è detta, ma è ricostruibile: Beatrice chiede "come arrivate a questo lunedì, a queste tarde ore di lunedì?", e alla fine dice che il *pitch* è "il 12 ottobre, tra una settimana". La lezione precedente su Tabletop Simulator era di venerdì 2 ottobre, e lì Ruggero aveva indicato "lunedì" come la lezione successiva.

pause la trascrizione registra per decine di righe frasi senza alcun rapporto con la lezione, ripetute in loop ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?"), e lo stesso accade in apertura (una frase sulla "struttura del giornalismo") e nei minuti finali ("la situazione è questa, la migliore dal 2005", "il numero di persone che hanno cercato di salvare vite"). Sono artefatti del riconoscimento vocale e il capitolo le ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro e resta generico altrimenti.

Come leggere questo capitolo. La lezione si appoggia a due **documenti** che Beatrice mostra a schermo e promette di caricare a fine lezione nella cartella condivisa delle dispense: il documento "**Project management di un gioco**" e un'**Experience Canvas** su Canva; a questi si aggiungono un **paper** sulle escape room formative, i **modelli legali** (informativa, esonero, consenso GDPR) e l'**Urban Game Deck**. Nessuno dei documenti è nella trascrizione: il capitolo ricostruisce ciò che Beatrice ne dice e ne legge, e segnala quando una voce del documento è solo nominata. Le tabelle riassumono le liste (le risorse, i vincoli, le fasi, le indicazioni del paper); le figure ricostruiscono gli schemi (il margine sulle ore, le tre strutture di enigmi, il ciclo crea-testa-itera). Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell) e alla letteratura su pianificazione, motivazione e giochi formativi; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa, lo stato dei gruppi e il programma della serata (sezioni 2.9.1 e 2.9.2);
2. il **project management**: le risorse, il progetto come una *jam*, il documento (sezioni da 2.9.3 a 2.9.5);
3. il documento voce per voce: obiettivo e target, **vincoli**, **tempo**, fasi e ore, emozioni e meccaniche, **percorso dell'utente**, **primo prototipo**, playtest e iterazione, documentazione e monitoraggio, la **stima** (sezioni da 2.9.6 a 2.9.17);
4. il commento della classe e "Esperienze ludiche 2": il **Game Design Document**, l'*Experience Canvas* e le **tre grandi domande** (sezioni da 2.9.18 a 2.9.22);
5. i **giochi formativi**: gamification, game based learning, la volontarietà, Duolingo, Sugata Mitra (sezioni da 2.9.23 a 2.9.25);
6. le **escape room formative**: strutture, enigmi, il paper (sezioni da 2.9.26 a 2.9.29);
7. dopo la seconda pausa: la parte **burocratica**, l'**Urban Game Deck** e il giro di *feedback* finale (sezioni da 2.9.30 a 2.9.33).

2.9.1 L'attesa: come state?

Come sempre, si aspettano "i primi 5 minuti" e poi si parte. Beatrice saluta Giordano, Lorenzo, Tiziana, e apre con una domanda che non è di rito: "come state intanto?". Lei stessa ammette di essere "un po' faticata nelle ultime settimane": "spero che voi ve la passiate meglio". E chiede come la classe arrivi "a queste tarde ore di lunedì".

Un caso di persona scambiata. Il giro è libero ("ruota libera"). Beatrice crede di aver intravisto Giordano quella mattina in un'altra lezione online e di aver scoperto che non fa game design ("fai 3D art?"). "No, nessuno dei due, in realtà": Giordano segue solo questo corso. "C'è stato uno scambio di persone": c'era un altro Giordano, forse con un cognome simile. Lui racconta di essere di Milano; Beatrice gli chiede se fosse il giorno prima a un festival che la trascrizione chiama "Combo"[2]. No, anche se a volte ci va: "magari c'è la possibilità di intravederci, visto che bazzico per le strade di Milano". Beatrice era lì, "presissima": "stanca ma felice". Uno studente spiega di avere la telecamera spenta perché, arrivato a casa, vuole "liberarsi" e si cambia: "va benissimo, fate bene, è uno dei vantaggi dell'online, che si può stare in pigiama".

[2] Il nome è riportato come nella trascrizione: dal contesto è un evento tenuto a Milano la domenica precedente, a cui Beatrice ha partecipato; non è possibile ricostruirlo con certezza.

L'appello. "Rileggo chi siete, che ho il terrore di perdermi qualcuno nell'appello": Tiziana, Tancredi, Lorenzo V, Nicola, Lorenzo L, Giordano "per ora". Le presenze le tiene d'occhio durante la lezione.

2.9.2 Il programma della serata

Beatrice riprende il filo. Dall'ultima lezione insieme la classe ha fatto un tutoraggio e altre lezioni, e "ci eravamo lasciate e lasciati un po' in corsa" con la grande lavagna su Canva del **design sprint**, nel mezzo della parte di **ricerca e sviluppo**. Il piano di oggi:

1. concludere le slide rimaste dalla lezione precedente, tutte sul **project management**;
2. passare alle slide nuove, "cose che vi servono dal lato pratico": un "piccolissimo passaggino" sul **Game Design Document**, poi le **escape room**;
3. fare un **esperimento con un mazzo di carte** per creare giochi;
4. tenere "come coda finale" la discussione sulla ricerca e sviluppo del design sprint.

Come si vedrà, l'ultima voce salterà per scelta della classe, e anche l'esperimento con il mazzo diventerà una semplice presentazione (sezione 2.9.32).

A che punto sono i gruppi. Prima di cominciare, Beatrice chiede se qualcuno ha già iniziato i lavori di gruppo, "non perché dobbiate averlo già fatto, non è questo lo scopo: è proprio mera curiosità". Un gruppo racconta di aver fatto un po' di **brainstorming** e di stare cercando di "scolpire" le idee: forse sono già focalizzati su un "giochino in particolare", e la sera stessa si sentiranno per avere un'idea finale su cui puntare. "Siete già in fase abbastanza avanzata".

Perché si parla di tempi. La curiosità ha uno scopo: oggi si vede "come prepararci a dividere i tempi". Il **12 ottobre** c'è il *pitch* del gioco, il **4 dicembre** la consegna: "dal pitch alla consegna ci sono, se non erro, un paio di mesi". Il primo passaggio della serata sarà quindi un'analisi di come usare il proprio tempo, "in modo da non ritrovarci dedicando tanto tempo a una o due voci e poi ci manca il tempo per fare tutto il resto". Beatrice ricorda che questo è un corso alla **seconda edizione**, che i docenti "iterano di volta in volta": questa parte è stata aggiunta proprio "per dare più ordine a come si arriva a portare il gioco concluso".

2.9.3 Organizzare la progettazione

Come per il design sprint, Beatrice ha preparato un **documento** da mettere nella cartella condivisa delle dispense (il documento del design sprint, ricorda, "dovrebbe chiamarsi *Light Sprint Design*"[3]). Crede che il nuovo si chiami "Organizzare la progettazione", va a controllare: "no". Si chiama **"Project management di un gioco"**, perché, come aveva detto la volta precedente, nel mondo del gaming analogico "la parte di project management non è spesso ben strutturata, per ora". Secondo lei, col tempo lo diventerà sempre di più, e "assomiglierà di più alle impostazioni di altre organizzazioni". Ma intanto, in questo progetto, è importante cominciare a farsi delle domande su **quanto tempo** e **quali risorse** si hanno.

Una forma mentis, non una tecnica di gruppo. Organizzare la progettazione serve "a prescindere da se lavorate in gruppo": è una *forma mentis*. Significa cominciare a valutare quanto tempo ci si mette a fare le cose e quanto tempo, quindi, allocare; e lo stesso vale per le risorse economiche e "proprio pratiche" (token, carte, tutti i materiali), e per le **competenze** del team, o le proprie se si lavora da soli. "Tutto questo pacchettone, imparare ad allocarlo e utilizzarlo, è essenziale".

[3] Così nella trascrizione ("LightSprint Design"); Beatrice aggiunge che "comunque lo capite perché c'è *sprint design* nel titolo".

Ci vogliono anni. Beatrice mette subito le mani avanti: non è una cosa "che si fa in una o due lezioni, o in uno o due anni: si fa col tempo". Più si fa attenzione a come si lavora, meglio si lavorerà, "e pian pianino metterete dei mattoncini". Partire senza "una mezza idea di come si allocano le risorse" non è un problema, ed è plausibile che quella sensazione duri a lungo. "Poi arriverà un momento magico in cui direte: caspita, adesso inizio a capire che su questa cosa ci vogliono tot ore, e ci metto davvero le tot ore che ho valutato di metterci".

Un invito, non un obbligo. L'esercizio che propone non si farà a lezione: si farà, "se vorrete", nei lavori di gruppo, ed "è un invito, non è un obbligo". Non serve ad avere tutto sotto controllo ("non avere neanche mezzo imprevisto è impossibile"), ma ad allenarsi, come persone e come gruppi, a conoscere meglio come si lavora: quanto tempo ci si mette, quante risorse e quali competenze servono.

2.9.3.1 Le risorse

Organizzare la progettazione vuol dire avere "una coscienza, nel senso di una consapevolezza", delle risorse che si hanno a disposizione e di quelle che servono al progetto specifico. Beatrice ne elenca quattro (tabella 2.9.1).

I tempi. È la voce su cui "faccio abbastanza la punta ai chiodi", perché è spesso sottovalutata, e invece "modifica veramente la qualità del lavoro e anche la serenità con cui si lavora". Parlarne e organizzarli è già metà del lavoro.

Le competenze. Se in un corso di videogiochi il team non ha nessuno che programma, e nessuno sa farlo, "dovremo o cambiare il gioco o trovare un modo per poter fare questa parte". Lo stesso vale per la grafica, per il *copy* dei testi, "per tutti i lavori che immaginate".

Le energie. Beatrice le nomina "in modo esplicito" perché sono le più sottovalutate: "come esseri umani abbiamo un tot di cucchiai durante la giornata"[4]. Anche avendo otto ore filate per inventare, scrivere, programmare, magari non si hanno le energie per farle di fila. Meglio lavorare un'ora, fare una pausa e valutare se si è ancora in grado, mentalmente e fisicamente, di continuare; e se no, avere un **piano B**: un altro lavoro "per staccare", o un modo per ricaricarsi.

Le risorse economiche e fisiche. "E infine, ma non infine": se si deve fare un gioco di carte e non ci sono i soldi per il prototipo, è una cosa su cui ragionare. E se il gioco ha **mille carte**, il problema va oltre il prototipo ("perché abbiamo il cugino che ci presta i soldi"): poi si dovrà andare da un editore, o fare autopromozione, o trovare un bando per produrlo, e sono soldi che andranno trovati; "forse mille carte è una spesa che non vale la candela".

Le quattro voci "sono tutte da valutare contemporaneamente": un lavoro corposo, ed è per questo che "pian pianino impareremo a farlo meglio. Non aspettiamoci di essere precisissimi".

Risorsa	Che cosa chiedersi	L'esempio di Beatrice
Tempi	Quante ore ho davvero? Quante ne serviranno?	Il *copy* del regolamento stimato in 3 ore che ne richiede 10
Competenze	Che cosa sa fare il gruppo? Che cosa manca?	Un videogioco senza nessuno che programma: cambiare gioco o trovare il modo
Energie	Quanto reggo di fila? Come mi ricarico?	Otto ore libere non sono otto ore di energia: un'ora, pausa, piano B
Economiche e fisiche	Quanto costa il prototipo? E la produzione?	Un gioco con mille carte: il prototipo si paga, la produzione forse no

Table 2.9.1: Le quattro risorse da valutare insieme quando si organizza la progettazione.

[4] Il riferimento, implicito, è alla *spoon theory* ("teoria dei cucchiai"), la metafora con cui si descrive l'energia quotidiana come una scorta limitata di cucchiai da spendere.

2.9.3.2 Stimare, monitorare, aggiustare il tiro

Le risorse vanno poi distribuite in un **arco temporale preciso**, e l'andamento va **monitorato**. L'esempio di Beatrice: stimo tre ore per il *copy*, "cioè la scrittura dei testi", del regolamento iniziale, e invece ce ne metto dieci. Lo annoto: avevo stimato tre ore, ne ho messe dieci. Me lo ricorderò per il futuro, e valuterò di volta in volta il perché: magari il gioco ha regole particolarmente complesse, o si basa su informazioni storiche e ha chiesto una ricerca diversa.

Un dato, non un giudizio. Il punto è che quel numero "non è un dato che ci dice che siamo lenti, punto". E viceversa: se si stimano tre ore e se ne usa una, non vuol dire che si è veloci. "È un dato semplicemente che ci dice che, in quel caso lì specifico, questo è successo". Si prende nota e basta.

Aggiustare il tiro. Lo scarto va però gestito *dentro* il progetto. Se si sono spese dieci ore invece di tre, le sette ore in più vanno recuperate da qualche altra parte: "sono proprio calcoli da fare", e Beatrice promette "un trucchetto per starci dentro" (la regola delle ore non allocate, sezione 2.9.9.1). Se invece si è messa un'ora invece di dieci, "figata": nove ore in più, "che posso andare al mare o distribuirmi sul progetto in diversi modi".

Il debriefing del progetto. Infine si **monitora come è andato tutto il progetto**, a conclusione, e si torna su quei dati di volta in volta: "è la cosa che ci fa progredire nel lungo periodo come professionisti e professioniste". Come in un gioco si fa un *debriefing* finale, così a fine percorso si guarda, "nella complessità", che cosa ha funzionato e che cosa no.

Tutte queste voci stanno sotto il cappello del **project management**, e "dovrebbero avere un *project manager*". Non ce l'hanno: "saremo noi ad essere il project manager di noi stessi e del gruppo".

Approfondimento: la fallacia della pianificazione. L'esempio delle tre ore che diventano dieci non è un caso: la tendenza a sottostimare il tempo dei propri compiti è uno dei fenomeni più studiati della psicologia delle decisioni, e Daniel Kahneman e Amos Tversky l'hanno chiamata *planning fallacy*. Si stima immaginando il caso in cui tutto va liscio, invece di guardare quanto sono durati davvero i compiti simili in passato. Il rimedio proposto in letteratura è proprio quello di Beatrice: raccogliere i propri dati storici (la "*reference class*") e stimare a partire da quelli, non dall'intuizione del momento. La frase ironica nota come legge di Hofstadter ("ci vuole sempre più tempo del previsto, anche tenendo conto della legge di Hofstadter") dice la stessa cosa in forma di battuta.

2.9.4 Il progetto come una game jam

Per questo progetto, e in generale per i primi progetti, a Beatrice piace immaginarlo "come una *jam*"[5], perché personalmente la fa uscire "un po' dalla dinamica della prestazione". Non sa se faccia lo stesso effetto alla classe, ma comunque è un'indicazione che aiuta.

Che cos'è una jam. "Do per scontato che tutti sappiamo che cosa sono, ma magari faccio una specifica": sono momenti in cui si partecipa con un tema o con delle caratteristiche specifiche e si crea un gioco. Chi indice la jam lancia un tema e una serie di caratteristiche, e chi partecipa inventa un gioco **in un tempo specifico**.

Darsi un tempo anche quando nessuno lo dà. È quello che Beatrice invita a fare sempre: capire che si ha un **tempo prestabilito**, "anche se nessuno ve l'ha dato". Nei progetti personali nessuno dirà quando consegnare. Se si vuole fare un gioco sulla propria esperienza di fare il bagno al mare, solo per sé, magari da autopubblicare col tempo o come ricerca personale "per raccontare delle cose di me", conviene comunque darsi un arco temporale: "voglio provare ad avere un prototipo entro quattro mesi, voglio provare ad avere il gioco finito entro un anno e mezzo", e allocare le risorse "in questo tempo che io mi auto do". Vale anche

[5] Nella trascrizione "gem" e "GEM"; è la *game jam*.

quando a dare i tempi è qualcun altro: fare lo stesso ragionamento "nel vostro cuoricino" aiuta a capire che cosa funziona e che cosa no, su di sé e nel gruppo.

2.9.5 Il documento "Project management di un gioco"

Una slide riassume "la lista delle cose che vanno fatte" in un percorso di progettazione di un gioco. Beatrice non la legge ("vi allunga il percorso"): serve solo "ad avere una lista a colpo d'occhio", perché le stesse voci si vedranno una per una nel documento. Controlla se nel frattempo è arrivato Angelo ("direi di no") e prosegue.

A che cosa serve. Il documento sarà caricato nella cartella condivisa a fine lezione, così che, "se nel momento in cui volete compilarlo vi siete un attimo persi o perse", si possa tornare alla registrazione. Ancora una volta: non c'è nessun obbligo di compilarlo, "né in linea generale nella vostra vita quando progettate, né per questo lavoro di gruppo". È "solo un documento che vi dà una direzione": quando ci si perde, è "una lista, un eserciziario" a cui attingere per ritrovarla. "Questo documento, se compilato, ci dà tutto quello che serve per progettare un gioco".

Un riassunto, non un punto di partenza. Il documento è pensato come **riassunto** di riflessioni già fatte altrove. L'ordine, nel metodo di Beatrice ("prendete sempre tutto come strumenti che vanno adattati alle vostre specificità"), è: prima la sessione di **design sprint**, poi, con ciò che ne è uscito, la compilazione di questo documento, "in modo da avere un po' di ordine nelle cose che stiamo facendo". La tabella 2.9.2 riassume le voci nell'ordine in cui Beatrice le presenta.

Voce	Che cosa contiene, secondo la lezione
Obiettivo e target	Che cosa voglio ottenere; per chi progetto; in che contesto si gioca; come capirò se ho raggiunto l'obiettivo
Vincoli già presenti	Scadenza, budget, spazio fisico, materiali, numero di partecipanti, strumenti, tecnologia, richieste del committente
Quanto tempo abbiamo	Data di inizio e di consegna; le ore davvero disponibili
Fasi e ore	Ore allocate per fase e sottofase, con l'*output* atteso; il totale deve restare sotto le ore disponibili
Emozioni (punto 4)	Uno spazio per esplicitare l'emozione scelta
Meccaniche e flusso (punto 5)	Meccaniche, che cosa fa il giocatore, che cosa voglio che provi; flusso e dinamiche
Percorso dell'utente	Che cosa fa la persona giocante dall'inizio alla fine
Primo prototipo	Che cosa serve, che forma ha; che cosa è fisso, che cosa provvisorio, che cosa non va ancora realizzato
Playtest	Le cose principali da tenere a mente (rimandato alla lezione di Ruggero)
Iterare	Modificare il gioco in base ai dati raccolti
Documentare	Foto, *feedback* scritti, diari, audio del processo
Monitorare	Ore stimate contro ore reali; che cosa ha funzionato, che cosa ha fatto perdere tempo, che cosa è stato sottostimato
Stima del tempo	Una formula (saltata): in sintesi, +20%

Table 2.9.2: Le voci del documento "Project management di un gioco", ricostruite da quanto Beatrice ne dice a lezione. Solo per le emozioni e le meccaniche la trascrizione riporta il numero del punto; l'ordine delle altre è quello della spiegazione.

2.9.6 Obiettivo e target: un recap

La prima parte del documento è "un recap di cose che dovremmo aver già fatto". Si parte dall'**obiettivo del gioco**, già visto nel design sprint: "cosa voglio, o vogliamo, ottenere attraverso questa esperienza".

L'obiettivo "può avere varie caratteristiche, come abbiamo visto". Poi il **target**, con tre domande (Beatrice controlla che si veda il mouse e le indica a schermo):

- **per chi stiamo progettando**, cioè chi sono le persone che giocheranno: è la riflessione sul target già fatta nel design sprint;
- **in quale contesto verrà giocato**: anche questo, "avremo già risposto prima";
- **come capiamo se abbiamo raggiunto l'obiettivo**.

La domanda senza documento. L'ultima domanda non fa parte della sessione di design sprint, ma "plausibilmente emergerà durante la sessione": definendo l'obiettivo e facendo brainstorming si è già parlato di cose che funzionano e di cose che no, di cose tenute e di cose scartate. Per questa domanda non c'è un documento dedicato, e i docenti non hanno esplicitato come ragionarci, "ma credo che avete tutti gli strumenti per poterlo fare in autonomia". Nessuna domanda dalla classe: "se avete domande, siete sempre liberi e libere di farle".

2.9.7 I vincoli già presenti

La voce successiva è una **lista non esaustiva** dei possibili vincoli. Beatrice l'ha inserita perché è importante averli in mente prima di cominciare; gli esempi sono "proprio esempi", e la lista "si può riempire di molto": il documento ha uno spazio apposta per aggiungere quelli che già si conoscono (tabella 2.9.3).

La scadenza. È "il primissimo vincolo che in genere si presenta quando lavoriamo a livello professionale": consegnare il gioco, o il prototipo, in un giorno, un mese, una settimana. Nel corso le scadenze sono due: il **12 ottobre**, con il *pitch*[6], e il **4 dicembre**, con "la consegna del gioco vero e proprio sulla cartella, con l'*upload* del gioco".

Il budget. "La maggior parte delle volte" c'è un vincolo di budget. Nei giochi da tavolo, per esempio, è raro vedere giochi "con tante, tante, tante carte", e se si vuole andare da un editore non gli si presenterà un gioco con tantissime carte, "per una questione meramente di budget". Il ragionamento conta anche quando si lavora **su commissione**, cioè quando un editore dà un gioco da sviluppare: lì bisogna chiedere, e loro devono dire, **quanto si aspettano di spendere per ogni copia**[7]. Non si andrà a vedere nel pratico quanto costa ogni cosa, "ma dovete avere una mezza idea", perché un certo costo per copia corrisponde a determinati prezzi di mercato.

Se ci si autoproduce. Per chi vuole autoprodurre un gioco, invece, la ricerca sui materiali va fatta "in modo molto certosino". "Produrre un libro stampato costa meno che fare un gioco da tavolo", e i costi delle carte non crescono in modo lineare con il numero[8]: "sono tutti ragionamenti che dobbiamo fare". Nel budget si può mettere una **voce complessiva** ("devo spendere 600 euro, e con 600 euro devo riuscire a tirar fuori un prototipo e 10 copie del gioco") oppure un **tetto per copia** ("2 euro massimo di stampa, 6 euro, 15 euro"), dove "di stampa" vuol dire di produzione di tutto: pedine, tabellone, "tutto quello che concerne". L'indicazione non è sempre esplicita né sempre necessaria, ma lo è nei contesti professionali appena descritti.

Lo spazio fisico. Un gioco pervasivo giocato in piscina è diverso da uno in città, o da uno pensato per le scuole; e anche un gioco da tavolo pensato per i *pub* è diverso da uno da giocare a casa con la famiglia.

I materiali di gioco. Sono "le cose che noi possiamo inserire nel gioco". Se si immagina una moneta fatta di due monete unite da una calamita interna, bisogna prendere atto che è una cosa da cercare e da costruire: "se non esiste sul mercato un fornitore che vi può dare questo materiale, è un vincolo importante". E di nuovo le mille carte: devono stare nel budget, e bisogna anche capire chi può produrle e fornirle.

[6] Qui la trascrizione dice prima "il PIL", poi si corregge.

[7] La trascrizione dice "per ogni coppia"; il senso è "per ogni copia" prodotta.

[8] La frase della trascrizione è "produrre mille carte costa meno che farne 30", da intendere probabilmente *per carta*: le economie di scala rendono la singola carta di una tiratura grande molto più economica di quella di una tiratura piccola.

Il numero di partecipanti. È un "vincolo importantissimo che dobbiamo darci a monte", l'unico che Beatrice definisce "sempre fisso". È un vincolo da **automettersi**: "è bene che sappiate, prima di iniziare a fare qualunque cosa, quante persone prevedete che giocheranno". (A metà della frase si scusa: deve recuperare "una caramellina" perché si sta "raschiando la gola".)

Strumenti, tecnologia, richieste del committente. Sono tre voci per monitorare che cosa si ha a disposizione e che cosa si deve usare. Magari il committente, o la situazione, impone l'uso dello smartphone o di un software specifico: "qualunque cosa può ricadere sotto la voce degli strumenti". Per le richieste esplicite del committente torna l'esempio usato "un po' più volte" nelle lezioni di Beatrice: "vogliono specificatamente un gioco con dei pinguini blu? Dovete mettere il gioco con dei pinguini blu", e va in questa voce.

Vincolo	**Esempio dalla lezione**
Scadenza	Nel corso: *pitch* il 12 ottobre, consegna (*upload*) il 4 dicembre
Budget	Complessivo (600 euro per prototipo e 10 copie) o per copia (2, 6, 15 euro massimo); su commissione, chiedere sempre il costo atteso per copia
Spazio fisico	Piscina, città, scuola; pub o casa con la famiglia
Materiali di gioco	La moneta doppia con la calamita: esiste un fornitore? Mille carte: chi le produce?
Numero di partecipanti	Da fissare prima di qualunque altra cosa
Strumenti e tecnologia	Smartphone obbligatorio, software specifico
Richieste del committente	"Un gioco con dei pinguini blu"

Table 2.9.3: I vincoli già presenti: la lista non esaustiva del documento, con gli esempi di Beatrice.

2.9.8 Quanto tempo abbiamo

"È semplice", almeno in apparenza: è la definizione di un arco temporale, e il documento ha due spazi, la **data di inizio** e la **data di consegna**. Ma tra le due date passano giorni, settimane, mesi, e nessuno sarà disponibile per tutto quell'arco: ognuno partecipa a modo suo. "Magari lavora, magari ha dei figli, magari lavora e studia". Bisogna quindi capire **quante ore** di quel tempo si possono davvero mettere nel progetto.

La prima pausa. Qui Beatrice si interrompe: "non so cosa mi è venuto". Si prende cinque minuti per andare a prendere la caramella per la gola: "ci vediamo alle 17:36". Al ritorno chiede "un cenno di emoji o telecamera aperta" e riceve "una pioggia di emoji".

Ragionare in ore, per eccesso. Si riparte dalle ore. Gli studenti hanno "tanto altro", e quindi non devono ragionare sui giorni o sulle settimane a disposizione, ma **in termini di ore**: "non è il ragionamento più facile da fare, soprattutto le prime volte, però è un invito che vi faccio". E le ore vanno stimate **per eccesso**: "mettete qualche ora in più rispetto a quella che immaginate".

2.9.9 Le fasi e le ore

Per fare tutto questo si divide il tempo **in fasi**, allocando delle ore per ciascuna. A destra della slide Beatrice ha scritto che si è "quasi sicuramente dimenticata qualcosa": le fasi possono essere di più di quelle previste nel documento. In linea generale, però, sono quelle già viste insieme:

1. il **brainstorming**, "la fase completamente divergente", con la ricerca dei titoli e delle cose che possono ispirare;
2. la **definizione del gioco**;

3. il **playtesting**;
4. l'**iterazione**, con il playtesting continuo, e la **conclusione del prototipo**.

Le fasi si possono dividere in **sottofasi**, "e vi invito a farlo il più possibile".

Quanto può durare una fase: il caso del design sprint. La scheda dà già delle indicazioni. Il documento del design sprint usato nella lezione precedente, racconta Beatrice, l'ha usato anche in sessioni di creazione di giochi e di attività che duravano **40 minuti in tutto**, per l'intero percorso. In classe, invece, ci sono volute quasi tre ore per la parte fatta, "che non lo abbiamo nemmeno finito". "Quindi il tema è allocare il tempo e provare a starci dentro".

Allocare e poi chiudere. Le indicazioni concrete:

- **definizione dell'obiettivo**: un'ora, o due, come riunione di gruppo. "Fine". Per il progetto del corso non si fanno più ore di queste: "l'obiettivo che è uscito è l'obiettivo che terremo per questo corso";
- **ricerca** di strumenti, titoli, esperienze simili, musiche che ispirano: un altro blocco, "da 40 minuti ad altre tre ore, anche di più, come si sente il vostro gruppo in base a quanto tempo ha". Ma "finito il tempo che abbiamo deciso di allocare, non si fa più ricerca. Fine".

Questo non significa che non si tornerà più sulla ricerca: significa che "lo slot di ricerca che il gruppo ha deciso di mettere qui non lo riapriremo più". Non ci sarà un'altra riunione dedicata; si faranno altre cose che possono integrare. (Beatrice tossisce, si scusa: "vi tossirò, mi sa, per un po' ".)

L'output atteso. Per il brainstorming e per le altre fasi il documento ha "una lista di voci" che Beatrice non legge tutte. Per ogni fase c'è anche l'**output atteso**: "quello che ci si aspetta che il gruppo abbia, o che noi come singole persone produciamo, alla fine di quella fase di lavoro". Ha davvero provato "a fare un recap di tutto quello che può essere sensato" in un progetto come questo; in base al tipo di gioco le fasi cambiano molto, e nel videogioco "ci sono fasi molto diverse". Il suo invito: fare **una prima riunione** in cui si dà un tempo a ogni passaggio e si capisce come arrivare all'output atteso, che è la seconda colonna della tabella del documento.

2.9.9.1 L'enorme regola: non allocate tutte le ore

"Ecco, enorme, enorme regola". Il gruppo ha valutato le proprie ore: c'è chi lavora e studia, chi ha altre cose nella vita, e insieme si ha un certo numero di ore complessive. **Non vanno usate tutte** nel documento. "Diciamo per assurdo che abbiamo 10 ore. Ne utilizziamo 8, ne utilizziamo 7".

Il documento ha in fondo un **totale**, la somma delle ore allocate alle fasi (Beatrice fatica a ritrovare la slide: "non riesco più a tornarci, porca miseria"). Quella somma deve dare **un valore minore** delle ore che si è valutato di avere. Il perché è semplice: "capiterà sicuramente" un imprevisto, una lungaggine; ci si appassiona alla ricerca, alle illustrazioni o alla scrittura, "e via se ne va un'ora, due ore, senza che neanche ve ne accorgete"; oppure si scopre che la musica scelta non è libera da diritti, "e allora altre due ore a cercare la musica". Non solo imprevisti veri e propri, dunque, "ma anche un po' di agio per poter vivere la propria vita e la propria progettazione". Tutto porta inevitabilmente a tempi più lunghi: "cercate sempre di stare all'eccesso" (figura 2.9.1).

Nel documento c'è anche una **formula** per calcolare il tempo da mettere in conto, ma Beatrice non la racconta, "perché è una cosa troppo approfondita per un percorso come il nostro"; l'ha messa "per dirvi forte e chiaro: calcolate i tempi in eccesso". La riprenderà in sintesi più avanti (sezione 2.9.16).

Approfondimento: il margine e il "buffer". Il consiglio di Beatrice ha un equivalente preciso nella gestione dei progetti: il *buffer*, una riserva di tempo non assegnata a nessuna attività, che assorbe gli scarti senza far saltare la scadenza. Nei metodi agili lo stesso principio si traduce nel non riempire mai lo *sprint* al 100% della capacità del team. Schell, in *The Art of Game Design*, osserva che i progetti di giochi sono particolarmente esposti agli sforamenti, perché il lavoro di design è iterativo per natura e non si sa in anticipo quanti cicli di test serviranno: proprio per questo il margine non è un lusso, ma la condizione per poter iterare.

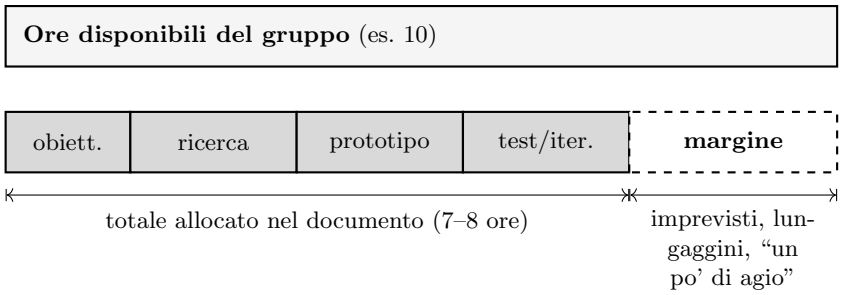


Figure 2.9.1: La "enorme regola" del documento: la somma delle ore allocate alle fasi deve restare sotto le ore disponibili. Le fasi e le proporzioni sono solo illustrative.

2.9.10 Emozioni, meccaniche e flusso: esplicitare

Le voci successive del documento sono cose che la classe "sa già tantissimo": uno spazio per ricordarsi di **esplicitarle**.

Le emozioni (punto 4). Il percorso sull'*emotional design* è già stato fatto. Il punto 4 serve solo a ricordare di scrivere quale emozione si è scelta, perché può succedere che durante il brainstorming ci si concentri molto sull'obiettivo ("magari decidete di fare un'escape room per divulgare i funghi e un viaggio nel bosco, come avevamo deciso nella lezione precedente"), si sia "molto presi dal tema" e ci si dimentichi di dare spazio alla parte emotiva. "Non è una cosa in più, è proprio solo per dare lo spazio".

Le meccaniche e il flusso (punto 5). Lo stesso vale per le meccaniche: si è già visto che serviranno meccaniche che portano a fare delle cose, e un **flusso di gioco** ("lo trovate anche a conclusione del documento sul design sprint"). Esplicitarle non vuol dire "sono queste, le manteniamo così per sempre": vuol dire ricordarsi di metterle nero su bianco, perché "magari le abbiamo molto chiare, ma non le abbiamo esplicitate", e averle scritte serve al gruppo e ai designer "per poter vedere cosa sta succedendo". Lo stesso vale per il **flusso del giocatore** e per le **dinamiche** che nascono dalle meccaniche. Nel documento ("in piccolino piccolino, che non so neanche se si legge") c'è una divisione in colonne: la **meccanica**, **che cosa fa il giocatore** in quella meccanica e **che cosa voglio che provi**.

2.9.11 Il percorso dell'utente

Il **percorso dell'utente** era rimasto in sospeso la volta precedente ("non abbiamo avuto il tempo di affrontarlo con calma"), e ora Beatrice gli dedica "un attimino". È un termine che viene "nuovamente" dal **marketing**: è ciò che fa l'utente per arrivare a un obiettivo desiderato. Nei giochi è molto più difficile da fare che nel marketing, "ma vi invito a farlo": soprattutto alle prime armi, ma anche dopo, perché "spesso non c'è il tempo di esplicitarlo, ma bisognerebbe".

Banalmente, dall'inizio. Si tratta di rendere esplicito che cosa fa la persona giocante dall'inizio alla fine, "ma banalmente, davvero".

- In un gioco da tavolo, la prima cosa che farà è **sedersi al tavolo**. "È così?". O il gioco è uno di quelli in cui si sta **in piedi** di fianco al tavolo[9].
- Oppure la prima cosa è **aprire un regolamento cartaceo** e sfogliarne le parti; la seconda è **guardare i componenti**, e probabilmente stenderli sul tavolo per vederli tutti.

[9]La trascrizione dice "come soccuadro", un titolo che non è possibile ricostruire con certezza; il senso è un gioco da tavolo che si gioca in piedi.

- In un **gioco pervasivo** in città, la prima cosa sarà uscire di casa e aprire il link ricevuto, oppure uscire con "la scatoletta di gioco" e trovare un posto dove appoggiarsi.

Questo è l'inizio. Poi c'è il **percorso del gioco**: la prima cosa è mettere la pedina sul punto 1 del tabellone, la seconda è pescare una carta, e così per cinque turni in cui succedono N cose; poi la **conclusione**: il giocatore ha fatto una serie di passaggi e il gioco finisce.

A che cosa serve: il passaggio mancante. Il percorso dell'utente serve a verificare che ci sia una **linearità**, e va fatto **prima** di arrivare alla conclusione della progettazione, "perché ci serve a capire cosa succede davvero alla persona giocante". È qualcosa che si spera venga "per forza di cose" progettando, ma esplicitarlo aiuta a ragionare: "spesso ci immaginiamo che succedano delle cose, e poi manca un passaggio per farle succedere". Banalmente: un'istruzione in più, una pedina in più, **la penna**. "Gli diciamo: scrivi, prendi appunti. Ma non ha la penna, e ormai è in strada a giocare, e nessuno gli ha detto di prendere la penna".

Prima del test. Quasi sicuramente il percorso andrà rivisto dopo il test. Ma prima di arrivarci, "alleniamoci a vedere tutti i passaggini che la persona mette in fila per arrivare a iniziare e finire il nostro gioco".

Approfondimento: la *customer journey* e la lente del giocatore. Il "percorso dell'utente" è la *user journey* (o *customer journey*) della progettazione di servizi e del marketing: la sequenza dei punti di contatto tra una persona e un prodotto, dall'avvicinamento all'uscita. Applicato a un gioco analogico, include ciò che i designer tendono a dimenticare perché lo danno per scontato: aprire la scatola, leggere il regolamento, preparare il tavolo, riporre i componenti. Schell suggerisce di guardare il gioco con gli occhi di chi lo incontra per la prima volta (la sua "lente del giocatore"), e osserva che molti problemi di un gioco non stanno nelle regole ma nei momenti che le precedono e le seguono. L'esempio della penna di Beatrice è esattamente uno di questi.

2.9.12 Il primo prototipo: fisso, provvisorio, non ancora

Poi si arriva alla **costruzione del primo prototipo**. Anche questo, immagina Beatrice, verrà "in modo abbastanza automatico": fare l'elenco di che cosa si vuole e che forma deve avere. I componenti e le meccaniche sono già stati decisi nel design sprint e nella prima parte del documento; qui si **esplicitano**. Se il componente sono delle carte, quante carte e come. Se è un **quadernino**, bisogna capire "di quante pagine", dove mettere le grafiche, che cosa deve contenere.

Le tre domande. Ma la parte importante ("e questo è importantissimo") è decidere, per ogni elemento, in quale di tre categorie sta (Beatrice legge le tre domande della slide una dopo l'altra, e se ne scusa):

1. che cosa **deve essere contenuto necessariamente** nel prototipo, e fissato;
2. che cosa si può **lasciare provvisorio**;
3. che cosa **non si deve ancora realizzare**.

Necessario. In un gioco in forma di libretto, o in un'escape room in libretto, le istruzioni di gioco e gli enigmi, con il loro funzionamento, vanno avuti già nella fase prototipale: "sono una cosa necessaria da avere". Le illustrazioni di contorno, o i *copy* specifici delle pagine, si possono sistemare in un secondo momento.

Non tutto testato, ma l'indispensabile sì. "Io credo ve l'abbiano già detto", e Beatrice lo risottolinea: lo scopo non è avere subito uno strumento playtestato e funzionante su tutto, su cui si è ragionato "in modo cristallino". È anzi capire quali cose **diventano indispensabili**, quelle che, una volta decise, se cambiano "vi cambia tutto il gioco". L'esempio: un'escape room con **tre enigmi principali**. Si testano prima quelli, poi si prosegue nella costruzione. Se invece si prepara tutto il resto (la storia che racconta gli enigmi, le grafiche "fatte a modino") e poi uno dei tre enigmi non funziona al test, bisogna rifare un lavoro importante.

Il criterio del provvisorio. La scelta si fa con una domanda: *se questa cosa cambia, dovrò cambiare tutte le cose che le stanno attorno?* Se la risposta è sì, le cose attorno vanno tenute **provvisorie** "per un attimo". È un lavoro che verrà più facile con l'esperienza, ma intanto conviene spendere un po' di tempo a valutare che cosa va fatto e testato subito e che cosa si può sistemare dopo.

Che cosa non realizzare ancora. Che cosa non serve per playtestare e raccogliere dati? Magari **la copertina**; "o magari sì, perché uno degli enigmi è nascosto nella copertina". Oppure un gioco di carte molto ricco di meccaniche, con una narrazione che è più un *setting* che un'esplorazione narrativa: forse **le illustrazioni** non servono ora, "posso farlo proprio senza illustrazioni". Di ciò che è necessario, poi, si distingue di nuovo che cosa deve essere già fisso da testare e che cosa può restare provvisorio, "perché se cambia qualcosa non dobbiamo rifare tutto il lavoro da capo" (figura 2.9.2).

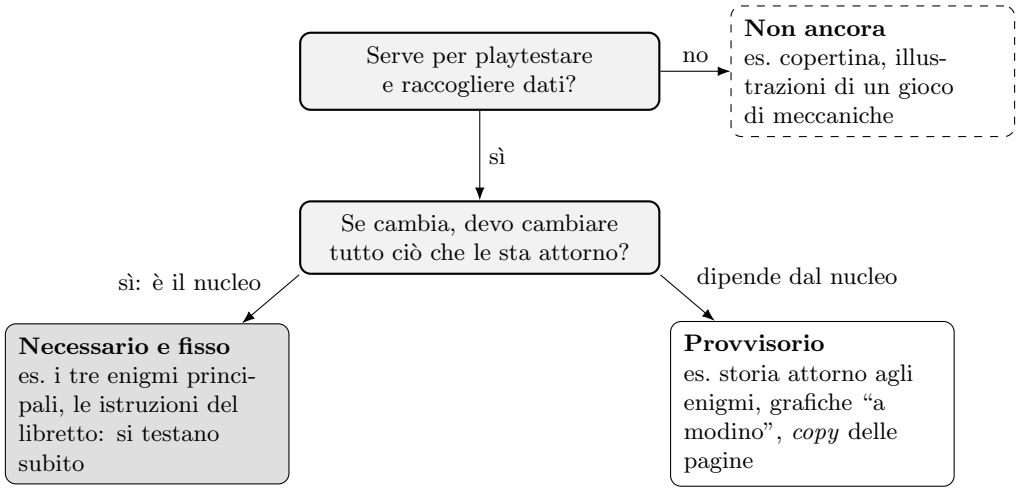


Figure 2.9.2: Le tre domande sul primo prototipo, come le pone Beatrice: che cosa non realizzare ancora, che cosa fissare e testare subito, che cosa tenere provvisorio. Ricostruzione schematica.

Approfondimento: il prototipo che risponde a una domanda. La regola di Beatrice è la stessa che Schell formula nel capitolo sui prototipi di *The Art of Game Design*: ogni prototipo deve rispondere a una domanda, e non deve contenere niente che non serva a rispondervi. Nel lessico dello sviluppo software, il nucleo da fissare per primo è ciò da cui "dipendono" le altre parti: costruire prima le dipendenze e poi ciò che dipende da loro evita di rifare il lavoro a cascata. Gli enigmi di un'escape room sono l'esempio perfetto, perché storia e grafica vengono scritte *attorno* a loro.

2.9.13 Playtest e iterazione

Sul **playtest** Beatrice chiede se la classe abbia già fatto la lezione di Ruggero sull'argomento. No. Allora, "per tempo", salta completamente quelle slide: la lezione di Ruggero farà "un bellissimo approfondimento" e renderà "cristallina" anche la slide appena raccontata. Dice solo che quella slide corrisponde a una parte del documento in cui mettere le cose principali da tenere a mente quando si playtesta: "di nuovo, uno spazio per ricordarci di tenerle a mente". Come tutto il documento, non è obbligatorio compilarlo.

Il ciclo. Lo stesso vale per l'**iterazione**, che Ruggero spiegherà nella lezione sul playtest e che la classe ha "già un po' intuito" da tutti i discorsi fatti. Di base, il percorso di creazione di un gioco è **creare** (inventare, progettare), **playtestare**, **raccogliere dati** e poi **iterare** sui dati raccolti (figura 2.9.3).

L'esempio della scritta gigante. Si è scritto "in caratteri cubitali" che il gioco si gioca **al buio**, e nessuna delle persone che lo testano legge quella frase: lo giocano tutti alla luce. "C'è qualcosa di enorme

che non va". Si deve iterare su quella voce: fare domande ("perché non hai letto questa scritta gigante? Cosa è successo?") e soprattutto **osservare** mentre le persone giocano. Magari la scritta è in un angolo dove l'occhio non cade, "e quindi lo vediamo non tanto parlando con la persona, ma osservando mentre legge i nostri materiali di gioco". Raccolto il dato, si modifica il gioco in base al dato: "e questa modifica è iterare".

Infinite volte. "Questo ciclo si fa infinite volte", tutte le volte che si può finché c'è tempo per testare: "questa è proprio una cosa cristallina che dovete portarvi a casa".

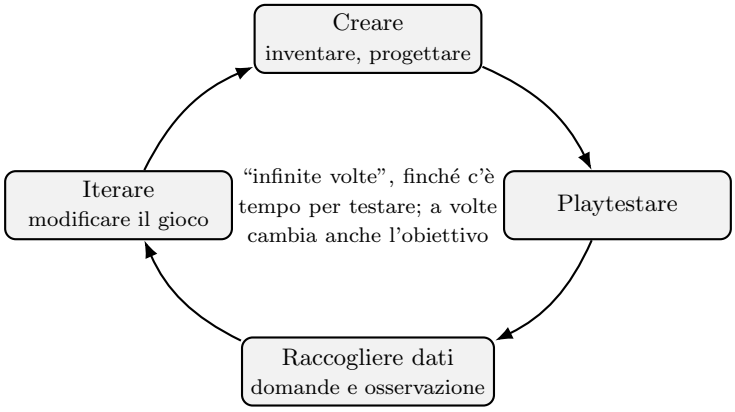


Figure 2.9.3: Il ciclo di progettazione come lo riassume Beatrice, in attesa della lezione di Ruggero sul playtest.

Quando cambia tutto. Nell'iterazione capiterà anche di accorgersi che c'è qualcosa "di molto grosso" che non va. Si è deciso di fare un'escape room per divulgare "tutta la scienza dei funghi" ("io dei funghi non so nulla, spero di non stare usando terminologie incorrette"), e dopo il quarto test ci si rende conto che non funziona come escape room: le persone, mentre giocano, "stanno ruolando quasi", ragionano e vivono molto di più la parte narrativa. Se così fosse, "non abbiamo paura di cambiare anche l'obiettivo del gioco, anche la forma del gioco", sempre in linea con i vincoli. Se il committente ha chiesto un'escape room sui funghi, si dovrà cambiare altro; e se non c'è il tempo di modificare, ci si tiene quel gioco "per ora".

Niente è buttato. Ma la *forma mentis* è: "non abbiamo paura di perdere del lavoro mentre progettiamo". Se in iterazione funziona meglio qualcosa che non si era valutato, lo si segue, perché "quello che scartate in un gioco vi servirà quasi sicuramente in un altro gioco che progetterete più avanti". Non nella stessa forma, ma ci si sarà già allenati a un tipo di pensiero, di meccanica, di dinamica, e lo si sarà perfino già testato: è già nel proprio "bagaglio formativo e professionale". "Quindi non abbiate paura di seguire il cambiamento che l'iterazione vi invita a fare, che i vostri playtester, che le vostre persone vi invitano a fare".

2.9.14 Documentare sempre

"E dovremmo essere verso la fine, così sto nei tempi". La voce successiva è **documentare sempre**. Durante il corso non ci sarà molto tempo per farlo, anche se sarà necessario per il playtest. Ma fuori dal corso Beatrice invita a documentare tutto:

- fare **foto** ai giocatori;
- chiedere **feedback scritti** "che siano quasi delle recensioni";
- tenere dei propri **giornali di game design**;
- registrare **audio** in cui appuntarsi il proprio processo creativo.

A che cosa serve. A due cose. A **vedere come si lavora**, acquisendo più esperienza su di sé; e a **poterlo raccontare fuori**. E raccontare come si lavora, anche senza autopubblicarsi, "è qualcosa che vi apre più porte, spesso e volentieri", più di tanti titoli sul curriculum o sul portfolio, più dei "contatti giusti": "può essere qualcosa che davvero è molto, molto potente". Quindi: raccogliere i dati e raccontarli fuori quando si può.

Il rimando al GDPR. Quel "quando potete" ha un limite normativo, e Beatrice lo anticipa: c'è "una grossa parte normativa", il **GDPR**, che si vedrà più avanti nelle slide di oggi, con "un piccolo strumento" per poter ripubblicare i dati che coinvolgono altre persone, cioè le proprie persone giocanti (sezione 2.9.30).

2.9.15 Monitorare: che tipo di stimatori siamo

"Documentare sempre lo abbiamo detto; poi monitoriamo". Si prende la lista di tutte le voci a cui si erano attribuite delle ore e si guarda se davvero ci si sono messe quelle ore. "Questa cosa sul subito non serve praticamente a niente": ci si ferma e si osserva che cosa è successo. Ma col tempo, raccolti dati su almeno **due o tre progetti**, con lo stesso gruppo o come singole persone, si potranno fare stime più realistiche.

Chi stima largo, chi stima stretto. Se su tre progetti si è sempre stimato più tempo del necessario, si sa di essere persone che **stimano alla larga**. Se invece, "come succede sempre a me", si stima **più stretto** ("dite ci metto tre ore e poi ce ne metto cinque"), si sa che in futuro conviene correggere: "a me verrebbe da mettere tre ore su questa voce, ma non gliene metto tre, perché so già da dati pregressi che mi conviene mettergliene cinque, sei, un po' di più".

Le domande del monitoraggio. Il documento dà "lo spazio fisico" per rispondere ad alcune domande, "non sono obblighi" (Beatrice qui accelera, "ora vado un po' più veloce"): che cosa ha funzionato; che cosa ha fatto **perdere tempo**; che cosa è stato **sottostimato**.

2.9.16 La stima: il 20% in più e la stima di pancia

La **formula per stimare il tempo** Beatrice la salta "a piè pari". "In estrema sintesi": **mettete sempre un 20% in più** di quello che immaginate. Se il gruppo è **nuovo** e non ha mai lavorato insieme, "anche un po' di più del 20%". E la formula va comunque verificata facendo diversi progetti: "non è una regola statistica, va valutata sulle vostre cose".

Quante ore mettere se non l'ho mai fatto. Resta la domanda più concreta: se non si è mai scritto un regolamento, quante ore mettere nella stima? L'invito è questo.

1. Se si hanno **dati pregressi**, cercare di ricordarli, anche se non sono stati annotati: "ci avete messo 4 ore a scrivere l'ultima tesina dell'università, oppure l'ultima ricerca online, oppure l'ultimo post su Instagram". Si prende il dato **più simile** all'attività da fare.
2. Se non c'è nessun dato simile, **non si lascia a zero**: si fa una **stima di pancia**, "ma comunque ipotizziamo".

La stima di pancia serve al ragionamento di prima: capire se si tende a stimare largo o stretto, come singole persone. "Tancredi, Beatrice, Tiziana": "Tancredi magari stima in eccesso, Tiziana stima più stretto, Beatrice stima più stretto". Ma è importante cominciare a notarlo, "e non possiamo notarlo se non facciamo una stima iniziale, quindi di pancia".

Perché vale la pena. Tornando a una slide saltata ("questo è importante in realtà, quindi ve lo dirò"), Beatrice spiega perché sapere quanto ci si mette è un investimento. Non sembrerà utile subito, perché non incide direttamente sulla progettazione, soprattutto nei primi progetti; ma col tempo è un'informazione essenziale:

- per **lavorare con altre persone**: si sa quanto ci si mette e si comincia a capire quanto ci mettono gli altri;
- per avere una **stima realistica**: "so che ci abbiamo messo tanto tempo a fare un gioco in questo team di tre persone che non aveva mai lavorato insieme". "È come chiedere dei preventivi, tutto sommato": senza nemmeno un appiglio numerico non si sa da dove partire;
- per **valutare le proposte**: se si viene invitati a collaborare a un gioco e viene chiesto di fare in due mesi un intero gioco di ruolo con diversi *branching* narrativi, o un librogame con molte illustrazioni, si sa che non è possibile, e si può scegliere consapevolmente se provarci lo stesso o dire "secondo me questi tempi non ci stanno";
- sul lato lavorativo, per capire se le proposte che arrivano sono sensate e, "ancora di più", **quanto farsi pagare** da *freelance*, "un tema di cui si parla ancora troppo poco". Spesso viene proposto un **forfait** "che non ha né capo né coda", e invece va valutato sulle ore che servono davvero. La stima diventa uno **strumento di contrattazione**: si sa già quante ore servono, e si può dire con certezza se una cosa si può fare o no.

La regola più importante. "L'ho già detto, lo ripeto in modo ancora più esplicito": tutto questo è un **allenamento**. I calcoli non saranno mai "matematici, precisi al millimetro": servono ad avere più consapevolezza "di come stare in questo mondo e di come funzioniamo".

L'esercizio per il corso. L'invito per il project work è stimare tutte le voci viste, in **due momenti**: prima ognuno da solo (quanto ci metterò io), poi il confronto nel gruppo. E poi monitorare entrambi i livelli. "E ci siamo! Sono perfettamente nei tempi che pensavo di utilizzare".

Perché un gioco concluso. Il documento resta a disposizione di tutti, da "sbirciare" senza alcun obbligo: è un'indicazione per riuscire a **concludere un gioco** in questi due mesi, "perché a noi interessa non un gioco perfetto, ma un gioco concluso", che serva per allenarsi su tutto: il project management, ma soprattutto la progettazione e la creazione. "E se il gioco non lo finite, il rischio è che perdete una grossa parte della formazione che può darvi questo corso". Tenere conto del tempo e delle risorse è "un vantaggio vostro".

Approfondimento: stimare per confronto. Il metodo di Beatrice (stimare prima da soli, poi confrontarsi nel gruppo) ricorda le tecniche di stima collettiva dello sviluppo agile, come il *planning poker*: ognuno stima in segreto, poi le stime si rivelano insieme, e le differenze più ampie diventano l'occasione per discutere ciò che uno ha visto e l'altro no. Stimare in privato prima del confronto evita l'effetto "àncora" della prima cifra detta ad alta voce. E il 20% in più è un modo semplice di includere il *buffer* di cui si è detto (sezione 2.9.9.1) nella stima stessa.

2.9.17 Semplice è meglio

L'ultima slide ("in ambito lavorativo poi lo leggete voi") contiene una regola che Beatrice vede spesso "non abbastanza valutata": **semplice è meglio**. Non vuol dire "fate giochi al ribasso": "seguite i vostri sogni, seguite quello che vi va di progettare", soprattutto in contesti senza committente e senza vincoli stringenti. Ma quando si fa il percorso dell'utente e si testa, la semplicità conviene a due "realtà": agli **utenti**, le persone che giocheranno, e ai **designer**, "per non complicarvi la vita".

La domanda da farsi ogni tanto. Nel pratico, Beatrice l'ha messa "proprio come domanda esplicita":

> Qual è la versione più semplice dell'idea che ho avuto, che riesce comunque a far vivere alla persona giocante l'esperienza che io voglio creare?

È un'indicazione per districarsi dai momenti complicati, quando si perde la rotta; ma "in linea generale è un'indicazione che diventa proprio un buon principio di design".

Approfondimento: la lente dell'eleganza. Schell dedica a questa idea la "lente dell'eleganza": un gioco elegante è quello in cui ogni elemento ha più di uno scopo e nulla è superfluo, e la domanda da farsi è quali elementi si potrebbero togliere senza perdere l'esperienza. È la stessa domanda di Beatrice, posta dal lato opposto: lei chiede qual è la versione più semplice che conserva l'esperienza, Schell chiede che cosa si può togliere senza perderla.

2.9.18 Il commento della classe

"Ok, abbiamo concluso la lezione della volta scorsa". Beatrice chiede cinque minuti di riflessione: com'è andata questa parte, "che era completamente nuova, e siete le prime persone a vederla in questa forma"?

Utile o superflua? Tiziana non ha domande, "ti dico la verità", ma trova molto comodo avere un documento riassuntivo con tutti i passaggi, e apprezza "tantissimo" le indicazioni "da chi ha già fatto", soprattutto sulla programmazione dei tempi e sulla documentazione: "ne farò tesoro". Beatrice spiega perché lo chiede: uno dei suoi dubbi, inserendo questa parte, era che fosse superflua. Al gruppo dell'anno precedente serviva, "però magari è superflua". Vuole sapere se è qualcosa a cui si sarebbe arrivati da soli o se "vale un'oretta di questo corso". Un altro studente è d'accordo con Tiziana: sarà utile, anche se deve ancora guardarsi il documento con calma per capire come applicarlo.

Il questionario di fine corso. Beatrice ricorda (e scopre che non era stato detto) che a fine corso ci sarà un **questionario**, uno spazio in cui raccontare che cosa ha funzionato e che cosa no, che cosa è piaciuto e che cosa no. Invita a parlarne anche per questa parte nuova: ora la si è vista "a volo d'aquila", ma si vedrà se servirà in fase di progettazione.

Un taglio sul lavoro. Aggiunge un'altra cosa su cui chiede un parere: nelle sue lezioni ha accentuato, più dei colleghi forse, "la parte un po' più sul lavoro", cioè il ragionare già in ottica di lavorare nel settore. "Anche qui vi invito a dirmi a voce se qualcosa non vi torna, che effetto vi fa". E la parte successiva va proprio in quella direzione.

2.9.19 Esperienze ludiche 2: due strumenti invece di una carrellata

Beatrice cambia presentazione: "Esperienze ludiche 2", "un po' una miscellanea di varie cose". E promette di tenere i tempi, "visto che vi ho fatto tutta questa cosa sui tempi, provo a fare la brava".

Perché non una carrellata. Voleva raccontare un po' di strumenti usati nel settore, ma si è accorta che rischiava di diventare "una carrellata di strumenti di diverso tipo", senza l'indicazione per andarli a vedere nel dettaglio. Era successo lo stesso con il design sprint: la lezione stava nascendo come un'altra carrellata, e ha preferito far **esperire** uno strumento solo, "ma con ordine", così che chi si trova bene possa riusarlo; per chi non si trova, ci sono le dispense con altre modalità di brainstorming e altri strumenti. Qui fa lo stesso ragionamento: invece di una carrellata, "un volo veloce su due strumenti", di cui uno è "sempre nominato nei manuali": il Game Design Document.

2.9.20 Il Game Design Document

Beatrice chiede una *reaction* a chi non ha mai sentito il termine **Game Design Document** (uno studente non ha capito la domanda: "forse parlo troppo in fretta"). Alcune manine si alzano, "sono manine di reaction, non sono domande"; e "c'è un po' di persone che l'hanno sentito molto bene".

Che cos'è. Di base, "lo dice un po' il titolo", è un documento che **racconta più o meno tutto il gioco**. Prende forme molto diverse ed è usato soprattutto nell'**industria videoludica**. "Immaginate una sorta di Bibbia del gioco": di qualunque gioco ("ti racconta sempre che esista, di Cluedo, un game design document") racconta, in fase di progettazione, gli obiettivi, come funziona, le varie fasi, perché sono state fatte certe scelte, la parte di *art* e tutto il resto. "Una grande Bibbia che, se tu leggi dall'inizio alla fine, hai una chiara visione del gioco".

Che cosa succede davvero nell'analogico. È un documento progettuale che, "nello stato attuale dell'industria analogica", non si usa quasi mai: la maggior parte delle volte **si lavora direttamente sul regolamento**. Uno studente chiede se si parla dell'industria analogica italiana, "perché immagino che all'estero funzioni diversamente". Beatrice precisa: dice quello che sa "per certo" sull'industria italiana, in cui è uno strumento "che non viene utilizzato granché".

Perché saperlo comunque. Ci tiene a parlarne perché viene raccontato come strumento necessario nei *game studies*, "cioè in tutta la parte accademica, tutta la parte di manuali su game design": va saputo che esiste. E nel mondo dei **bandi**[10] si usa ancora, "proprio per raccontare il progetto". Non ha un'unica forma: racconta tutto il gioco, o la parte che serve raccontare "in base a chi vi sta chiedendo questo documento".

Il desiderato e la realtà. Nel mondo ideale il GDD sarebbe anche un documento di lavoro **mutevole**, che chiunque può leggere e usare per lavorare sul gioco: il punto di riferimento per tutti i professionisti che collaborano allo stesso titolo. "Nel mondo pratico non funziona così", e si ragiona ancora molto "di riunioni, di documenti un po' sparpagliati". A Beatrice interessa dire "quello che viene raccontato, quello che è il desiderato e quella che è la realtà", e che si ricordi la terminologia. "Fine".

Che cosa chiede il corso. Per il corso, ricorda che cosa è stato fatto l'anno prima ("poi mi correggeranno in caso i colleghi"): non viene chiesto un documento di lavoro, "è proprio il gioco". Non c'è nessuna indicazione su come lavorare o su che cosa produrre per mettersi d'accordo: il design sprint e il documento sui tempi danno già una direzione, che può diventare "una lavagna Miro, un documento condiviso su Drive, una cartella condivisa con vari documenti". "Vi lascio piena interpretazione di come volete collaborare".

Approfondimento: il GDD nei manuali. Il Game Design Document è centrale nella manualistica sul videogioco, dove nasce come contratto tra i reparti di uno studio. Lo stesso Schell, però, mette in guardia dal documento-monolite: nel capitolo sulla documentazione osserva che non esiste un formato unico, che i documenti servono soprattutto come *memoria* e come *comunicazione*, e che un documento che nessuno legge e nessuno aggiorna non svolge nessuna delle due funzioni. È la distanza tra "desiderato" e "realtà" di cui parla Beatrice. Molti studi, anche nel digitale, sono passati a documenti più brevi e vivi (*one-page design*, *wiki* interne).

2.9.21 L'Experience Canvas

Il secondo strumento è per quando "il gruppo non sa bene che direzione prendere", se si perde la direzione comune. Beatrice lascerà nella cartella condivisa, tra gli allegati, un esempio: l'**Experience Canvas**, che deriva dai **Business Model Canvas**, "strumenti per mettere ordine a progetti complessi". L'ha ricreato apposta come posto dove riassumere **tutte le decisioni** che si prendono.

Come si usa. Secondo lei il documento sui tempi basta già per "fare un recapone" di ciò che succede nel gioco. Ma se serve più visualità, o un altro modo di lavorare, l'Experience Canvas è "un documento anche molto impostato" e "molto, molto comodo". Ha già dei piccoli esempi di compilazione, e una versione completamente vuota da riempire. È un documento di **Canva** ("quindi non il Canvas, ma proprio Canva, il software che abbiamo usato la volta scorsa"), da copiare nel proprio account e riempire in condivisione con

[10] La trascrizione dice "nel mondo della bandistica"; dal contesto si tratta dei bandi di finanziamento.

il gruppo. "È un documento operativo, nulla di più, non è un obbligo utilizzarlo. Se vi è utile usatelo, se non vi è utile, via nel sole".

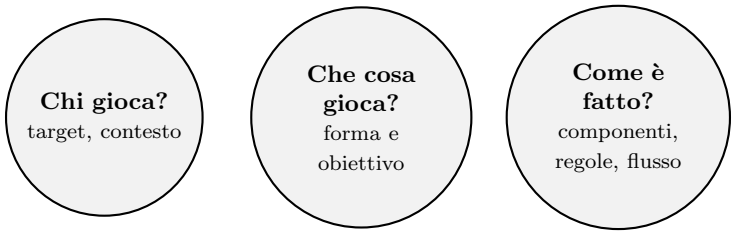
Approfondimento: dal Business Model Canvas ai canvas di design. Il *Business Model Canvas* di Alexander Osterwalder riassume un modello di business in nove riquadri su un unico foglio (clienti, proposta di valore, canali, risorse, costi...), e ha ispirato molti "canvas" derivati, anche per il game design: il pregio comune è costringere a una risposta breve per ogni voce e a vedere tutte le voci insieme, così che le incoerenze saltino all'occhio. Il contenuto preciso dell'Experience Canvas di Beatrice non è descritto a lezione.

2.9.22 Le tre grandi domande

E se non si usa nessuno di questi strumenti ("non il design sprint, non il documento sui tempi e non questo"), un'indicazione per arrivare a concludere il gioco è rispondere alle **tre grandi domande**:

1. **chi gioca?**
2. **che cosa gioca**, cioè che forma ha il gioco e che obiettivo ha?
3. **come è fatto il gioco?**

"A queste tre cose dovete rispondere. Non sapete rispondere? Non avete concluso il gioco". Può sembrare "una cosa scontata, una stupidata", ma se ci si perde per strada sono queste le domande a cui tornare: "non abbiamo risposto a una di queste domande? C'è un problema. Quindi dobbiamo tornarci su". Qualunque documento o strumento si usi, anche gli appunti su un quadernino, "a queste tre cose bisogna dare risposta. Fine di questa parte".



Se manca una risposta, il gioco non è concluso: "c'è un problema, dobbiamo tornarci su".

Figure 2.9.4: Le tre grandi domande di Beatrice, valide qualunque strumento usi il gruppo.

2.9.23 Giochi formativi: gamification e game based learning

"Andiamo finalmente ai **giochi formativi**". Si parte da due termini: **gamification**, che "ha preso molto piede negli ultimi anni", e **game based learning**. Beatrice chiede chi non li abbia mai sentiti: nessuno alza la mano per la gamification; per il game based learning, sì, Gianmarco[11].

Un termine improprio. Beatrice avverte che userà "giochi formativi" in modo un po' improprio, "perché la vera verità è che non abbiamo neanche ben chiaro, nel mondo dei Game Studies, come definire un gioco formativo". È un gioco che cambia il comportamento? Un gioco che dà informazioni nuove? Che cosa significa nel pratico? Non entra in tutta questa parte, "perché diventa una lezione troppo corposa" (se interessa, chiede di dirlo nei *feedback*). Le interessa invece dare almeno le due definizioni, che servono "sia a livello pratico che, di nuovo, professionale".

[11] La trascrizione dice prima "Marco", poi si corregge in "Gianmarco".

2.9.23.1 La gamification

La **gamification** è una **disciplina di progettazione**, "un'attività vera e propria", che usa gli **elementi** dei giochi all'interno di contesti che **non sono giochi**. La parola da tenere a mente è *elementi* ("ve l'ho messa in grassetto"): livelli, punti, missioni, classifiche, ricompense, "una sfilza di cosine".

Duolingo. L'esempio più classico è **Duolingo**, l'app per imparare le lingue, che usa tantissimo i punti, i livelli e l'interazione con il "piccolo gufino", il personaggio dell'app.

L'esempio della stellina. Per rendere la cosa tangibile, Beatrice prende uno "scopo formativo, passatemi il termine": **insegnare i segnali stradali** a chi sta imparando a guidare. L'attività gamificata è: *ti do una stellina per ogni segnale stradale che memorizzi, e a 10 stelline sali di livello, oppure raggiungi un premio.* Questo tocca la **motivazione estrinseca**.

2.9.23.2 Motivazione intrinseca ed estrinseca

Beatrice non sa se Ruggero ne abbia già parlato o ne parlerà, e riassume "tantissimo" ("è un po' spiegata molto male, ma giusto per capirci").

Motivazione intrinseca. È interna alla persona. Se sei appassionata di musica classica e ti propongo un'attività sulla musica classica, che ci siano un gioco, dei *badge*, delle stelline o no, sarai motivata a stare in quell'attività, "perché vado a toccare una tua motivazione intrinseca personale".

Motivazione estrinseca. Se invece la musica classica non ti interessa, posso metterti livelli, stelline, un premio, *feedback* immediati su che cosa funziona e che cosa no del comportamento che stai attuando. Provo a stimolare una motivazione *al di fuori di te*: legata ai "contratti sociali" in cui sei inserito, alle dinamiche di gruppo, alla voglia di scoprire che cosa succede nel livello successivo o qual è il premio segreto. E questa motivazione "ti porterà a esplorare un po' la musica classica".

La gamification punta molto sulla motivazione estrinseca: crea strutture basate su elementi di gioco "che ti portano a compiere determinate cose". Di qui l'esempio della stellina per ogni segnale memorizzato.

2.9.23.3 Il game based learning

Nel **game based learning** la parola chiave è **esperienza**, "fatta e finita". Non sono elementi di gioco sovrapposti a un ambiente che non è di gioco: è un'esperienza completa, che o crea un'esperienza a sé o, se si sovrappone a un ambiente non di gioco, "si intreccia proprio a questo ambiente".

L'esempio della macchinina. Stesso obiettivo formativo, i segnali stradali. L'esperienza completa è: *ti do una piccola macchinina su un tappetone con un percorso disegnato, e devi compiere il percorso più lungo del tabellone rispettando tutti i segnali stradali che vedi; a ogni segnale corretto ricevi una stellina, e la prima persona che arriva a 10 stelline vince un premio.* Ha elementi di gamification, "ma una costruzione dell'esperienza". È un esempio "molto banale", che forse non funzionerebbe nemmeno davvero per formare; serve a far capire la differenza tra *elementi di gioco piazzati da qualche parte che non è gioco* e *un'esperienza costruita dall'inizio alla fine* (tabella 2.9.4).

Perché la differenza conta. Le due modalità "si possono intrecciare tra di loro", ma conoscerne la differenza aiuta a capire quando si sta riuscendo a raggiungere la **motivazione intrinseca**, "che è rarissimo e difficilissimo", e quando si lavora sulla motivazione estrinseca, "punto".

2.9.24 La volontarietà: quando un gioco formativo non è un gioco

In ambito formativo la distinzione è "importantissima" per un motivo che, secondo Beatrice, il mondo della formazione ludica e interattiva dimentica spesso: le persone vengono invitate a formarsi con il gioco **in contesti in cui non stanno scegliendo appieno di giocare**. Usare un gioco per insegnare a programmare

	Gamification	**Game based learning**
Parola chiave	**Elementi** di gioco	**Esperienza** fatta e finita
Che cosa fa	Sovrappone punti, livelli, missioni, classifiche, ricompense a un contesto che non è un gioco	Costruisce un gioco completo, o lo intreccia al contesto
Motivazione	Soprattutto estrinseca	Può arrivare all'intrinseca ("rarissimo e difficilissimo")
Esempio sui segnali stradali	Una stellina per ogni segnale memorizzato; a 10 stelline sali di livello o vinci un premio	Una macchinina su un tappetone: il percorso più lungo rispettando i segnali, una stellina per segnale, vince chi arriva a 10
Esempio reale	Duolingo	(nessuno citato a lezione)

Table 2.9.4: Gamification e game based learning secondo la lezione: due modalità che si possono intrecciare, ma che conviene distinguere.

a scuola, e sceglierlo a casa propria comprandolo su Steam perché si ha voglia di provare quell'esperienza, "sono due attività completamente diverse". Spesso, quando si invita qualcuno a formarsi con un gioco formativo, "non sta davvero giocando".

Da Huizinga in poi. È da qui che nascono le riflessioni sulle due discipline: si cerca di usare modalità tipiche del gioco in contesti in cui non c'è **volontarietà**, mentre "ci viene detto da tutti, a destra e a sinistra, da Huizinga[12] ad adesso", che la volontarietà è ciò che rende il gioco un gioco: "altrimenti è qualcosa d'altro". ("Forse sto aprendo troppo una pippa su questo discorso, mi preme molto".)

Né demonizzare né idolatrare. Il messaggio è: esistono queste due grandi categorie, a volte si mischiano e a volte no, a volte funziona di più una e a volte l'altra. "Non c'è da demonizzare niente né da idolatrare niente". Ma a meno che una persona non si vada a comprare, o a scegliere in ludoteca, un gioco formativo, "in tutti gli altri casi non è vero e proprio gioco". Beatrice si dice disponibile a parlarne molto di più.

2.9.24.1 La domanda di Tiziana: che cosa vuol dire scegliere

Prima di passare oltre, Tiziana chiede di verificare di aver capito: se non si sceglie volontariamente il gioco, non si sta giocando. E se lo si sceglie *per formarsi*, se la motivazione è formarsi, nemmeno in quel caso si sta davvero giocando?

Se lo scegli tu, è gioco. Beatrice chiarisce: "se lo sceglie la persona, è sempre gioco". "Se tu, Tiziana, stai scegliendo adesso di giocare a qualcosa, tu stai giocando". Il problema sono i contesti in cui la scelta non è piena, e per spiegarlo prende un esempio dal suo lavoro ("ti invito anche a farmi esempi che vengono in mente a te, che magari è più utile"): il gioco formativo lo usa soprattutto a scuola, ma tra adulti l'esempio più chiaro è un'azienda.

L'evento aziendale. In un evento aziendale sono previste due ore di attività di gioco. Viene detto: "queste due ore le potete usare così, le potete anche usare senza fare niente, volendo, e potete scegliere tra sei titoli". Già qui non c'è una piena scelta: la proposta arriva "dall'alto, a livello proprio di gerarchia aziendale". Si può dire di no, ma bisogna esserne coscienti. Una percentuale di persone giocherà. E se non viene aperta **una strada reale per fare altro** ("tu puoi giocare e scegliere uno di questi sei giochi, oppure puoi sederti in quella stanzina lì, dove avrai il tuo computer, dove le altre persone possono sedersi con te, ci sono dei libri a disposizione", "sto inventando"), allora "se non c'è la scelta B, non è una vera libertà": si vivrà la pressione del gruppo che gioca, in una stanza dove tutti giocano e "tu non avrai nulla da fare". "Può sembrare che la

[12] Nella trascrizione "Osinga". Johan Huizinga, nell'*Homo ludens* (1938), pone la libertà, il carattere volontario, come prima caratteristica del gioco.

persona abbia la libertà di scegliere, ma non è una vera e propria scelta". E nel creare un gioco formativo "di questa cosa dobbiamo ricordarci".

Duolingo, visto da dentro. Tiziana precisa che cosa si era scritta: che non si sceglie volontariamente il gioco se lo si sceglie per formarsi; per esempio, se si usa Duolingo perché si vuole imparare una lingua. "No, no, quello lì è giusto, quello lì stai giocando, assolutamente"; e Beatrice la ringrazia per averle letto i suoi appunti. Se si sceglie, con piena possibilità di scegliere e senza pressioni attorno, si sta giocando: Duolingo è qualcosa che nessuno ti impone, non ci sono *malus* se non lo usi, "quindi sei in piena libertà e quindi è un gioco".

Ma Tiziana racconta la sua esperienza. All'inizio l'app le piaceva, anche perché sentiva che sosteneva la motivazione. "Però al tempo stesso mi sentivo ingabbiata": a spingerla a usarla non era più la voglia di imparare, che c'era all'inizio, ma **stare in classifica**, **ottenere i punti doppi**. "Ma io adesso non ho voglia di fare questa cosa, capito? Sono costretta, perché se non lo faccio adesso perdo questa opportunità". E così l'ha lasciata andare, un po' alla volta: "in realtà io non ho scelto di giocare. Io volevo imparare".

È proprio la gamification. "Assolutamente: quello che tu dici è proprio le dinamiche della gamification". Duolingo ("di cui peraltro io non sono grande fan, però ve l'ho portato come esempio perché mi immaginavo che tutti lo conoscessero") lavora sulle **motivazioni estrinseche**, "che non portano mai troppo lontano, che però sono le più semplici da progettare". (Nel frattempo Beatrice si accorge di un messaggio in chat di Tancredi sulla motivazione estrinseca: "sì, hai scritto giusto, comunque, assolutamente".)

Perché si progetta sull'estrinseca. Il motivo è pratico. "Come facciamo a sapere, anche solo di noi nove persone qui in call, quali sono le motivazioni intrinseche di ognuno di noi?", che cosa ci fa davvero venire voglia di imparare? Sono cose molto personali, che richiedono uno studio molto approfondito sul target, e spesso non ha nemmeno senso farlo, "perché è impossibile poi creare un'attività che vada bene per tutti". La gamification parte invece da ciò che gli esseri umani hanno in comune, a prescindere da che musica piace loro, dall'età, da dove vivono, da che computer sanno usare: caratteristiche legate a come funziona il cervello, a come ci si muove socialmente, a come si sta nel mondo, a come si usano le parole. Sono le stesse caratteristiche che si usano, "ancora prima di arrivare al termine gamification", nella *UX design*[13] delle piattaforme e perfino nelle scelte su come funzionano le città. "La gamification dice: utilizziamo le caratteristiche che sappiamo funzionare a livello emotivo per raggiungere degli obiettivi formativi o di comportamento". Ma è una motivazione **molto più debole** di quella intrinseca: "ed ecco che arriviamo a quello che dici tu".

Due filoni diversi. Beatrice separa le due cose: la volontarietà del gioco è una cosa; che un gioco iniziato volontariamente non funzioni su di te, a livello formativo o anche solo di intrattenimento, è un'altra. Tiziana precisa che a livello formativo funzionava ("se ci passi un sacco di tempo, funziona"), ma che si è accorta che l'app "mi aveva spostato dalla motivazione intrinseca mia a una motivazione estrinseca". "Ci sta, certo, assolutamente". Le definizioni servono a cominciare a ragionare; "poi, come quando parlavamo di definizioni dei giochi in generale, è molto più intrecciata la cosa": la motivazione intrinseca può cambiare nel tempo, e l'effetto che fa un gioco può cambiare anche se la motivazione resta uguale.

Approfondimento: l'effetto di sovragiustificazione. L'esperienza di Tiziana ha un nome nella psicologia della motivazione: *overjustification effect*. Nella teoria dell'autodeterminazione di Edward Deci e Richard Ryan, le ricompense esterne percepite come controllanti possono *ridurre* la motivazione intrinseca per un'attività che prima si faceva per piacere: l'attenzione si sposta dal fare la cosa all'ottenere il premio, e quando il premio perde interesse, si perde anche l'attività. È la critica più frequente alla gamification "a punti e classifiche" (spesso chiamata, con ironia, *pointsification*). Gli stessi autori indicano tre bisogni da sostenere per alimentare la motivazione intrinseca: **autonomia**, **competenza** e **relazione**. La definizione di gamification data da Beatrice (elementi di gioco in contesti non di gioco) coincide, del resto, con quella più citata in letteratura, proposta nel 2011 da Sebastian Deterding e colleghi.

[13] Nella trascrizione "Wix design"; dal contesto, il design dell'esperienza utente (UX) delle piattaforme.

2.9.25 Sugata Mitra e l'educazione minimamente invasiva

Su questo filone Beatrice lascia un **video** su YouTube, "accessibile, sottotitolato", "credo che siano massimo 40 minuti". È di uno studioso che ha approfondito quella che chiama **educazione minimamente invasiva**, che a suo dire è "molto connessa al fare game based learning": **Sugata Mitra**[14].

La libertà dell'utente. Mitra non parla di gioco, "ma ci dà il là per qualcosa che è estremamente importante": la **libertà dell'utente**. È utile proprio nei contesti in cui la persona non sceglie attivamente l'attività formativa: se si porta un gioco alle scuole medie, "o giocano o non possono fare altro in classe"; e lo stesso vale per le attività di gioco nelle aziende.

L'esperimento. Invece di formare facendo fare "ABCD", con una formazione frontale o con giochi che portano a formarsi in un certo modo, Mitra lancia delle **sfide** e lascia alle persone la possibilità di rispondere con gli strumenti che dà. L'esempio più noto è un esperimento in **India**: computer incassati in piloni di cemento "ad altezza bambino", con un software adatto ai bambini che permetteva di cercare in internet ciò che serviva per le ricerche proposte, lasciati accesi 24 ore su 24 in contesti "di grande povertà educativa, e anche fisica reale". Mitra lanciava sfide ("torniamo all'idea un po' della jam"): "dobbiamo capire come creare un esperimento scientifico: voi come lo fareste?". I risultati formativi sono stati altissimi: ragazzini tra gli 8 e i 12 anni gli hanno raccontato informazioni raccolte "con una competenza di un universitario o un'universitaria".

Il cambio di paradigma. "Io ve lo sto rovinando, raccontato così", ma il punto è che cambia il paradigma: da *ti guido a fare determinate cose* a *trovo il modo giusto di lanciarti una sfida, ti do gli strumenti corretti per raggiungere quell'obiettivo, ma poi tu trovi la tua strada.* "Ed è la formazione che io preferisco in assoluto, sia da esperire io come Beatrice, che da proporre e da fare". Beatrice l'ha inserita perché, pur senza parlare mai di gioco, descrive una delle cose principali che il gioco permette di fare: dà **agency** su ciò che si sta facendo, la possibilità di vedere che si ha potere decisionale, spazio per la propria creatività, gli strumenti per raggiungere gli obiettivi che si vogliono "senza essere incastrati in una scatola costruita da altri".

La regolina di design. È "un grosso strumento di design", soprattutto nei contesti formativi in cui il gioco non è volontario (dove la persona sceglie già un titolo, "l'esperienza emotiva è già diversa", e lo si capisce ricordando le volte in cui si è giocato per obbligo e quelle in cui si è giocato perché si voleva). Il "succo succone", anche per chi non guarderà il video:

> Quando vi trovate a fare un gioco formativo, non fate lunghe spiegazioni: create le condizioni perché le persone possano esplorare per arrivare all'obiettivo che vorreste raggiungere.

Approfondimento: *Hole in the Wall.* L'esperimento descritto da Beatrice è il celebre *Hole in the Wall*, iniziato da Mitra nel 1999 a Nuova Delhi con un computer incassato in un muro che dava su uno *slum*, e poi replicato in molte zone rurali dell'India. Da lì Mitra ha sviluppato l'idea dei *SOLE* (*Self-Organized Learning Environments*): gruppi di bambini, una domanda grande, un computer, e un adulto che incoraggia senza insegnare. L'approccio ha avuto anche critiche (sul ruolo degli adulti, sulla durata degli apprendimenti), ma l'idea che interessa a Beatrice è indipendente da quelle: progettare la *sfida* e gli *strumenti*, non il percorso. Nel game design è il principio dei sistemi che permettono soluzioni emergenti, invece di una sola soluzione prevista.

2.9.26 Le escape room formative

"Fino a qua, come sta andando?". Nessuna mano alzata, e Beatrice passa alle **escape room formative**. Ha diviso il modulo in più parti: qualche indicazione dalla sua esperienza di design, poi un documento.

[14] Nella trascrizione "Sugatta Mitra". Il video non è identificato con precisione a lezione.

Poca bibliografia, molti paper. Sulle escape room formative "non c'è tantissimo a livello di bibliografia": negli anni Beatrice si è formata sulle esperienze di altre persone e, fondamentalmente, sui **paper**. Ce ne sono "un'infinità"; lei ne avrebbe almeno dodici da condividere su come le escape room sono state usate per trasmettere informazioni, per modificare comportamenti, per fare *assessment*, per analizzare come funzionano i gruppi: tutto ciò che ricade sotto i giochi formativi. Ne ha scelto uno solo, "non perché sia il migliore", ma perché dà indicazioni **operative**. Invita però a cercare: più si legge e più si fa esperienza di progettare escape room, giochi di enigmi e di *puzzle*, più si acquisisce competenza. Chi vuole gli altri paper può scriverle; nella cartella non li mette, "perché diventa tanto materiale, e già così, secondo me, siete pieni come uova".

Un cappello sui giochi di enigmi. Sa che alcuni studenti hanno già lavorato su escape room e cacce al tesoro. Le strutture che presenta sono "solo un'indicazione" se non si sa che direzione prendere: "ce ne sono infinite", e quello che dice "è meno dell'inizio di tutto quello che vedremo nei prossimi anni" tra giochi "classici" e giochi pervasivi. Ma è un inizio, per chi volesse progettare un'escape room per il corso, formativa o no.

Su escape room, giochi di enigmi e giochi investigativi Beatrice ha lavorato molto, "soprattutto nella pandemia" (non ricorda se ha già parlato degli *Alternate Reality Game*, "ma direi di sì, nella lezione sui casi studio"). Cita in particolare quelli **in forma di PDF**, "molto comodi" perché:

- hanno un **bassissimo costo di produzione** per cominciare a progettare;
- richiedono **poche competenze**: non serve saper programmare;
- sono estremamente utili: li usa in molti corsi sulla costruzione di giochi formativi, "perché nel giro anche di poche ore si ottengono dei giochi straordinari".

Invita a esplorare **Genially**[15] e **Canva**: tra link interattivi e altre possibilità, con Genially si fanno "proprio dei micro videogiochi" punta e clicca, escape room interattive, "e con poche ore davvero potete tirare fuori dei piccoli gioielli". "Detto questo cappello, che non avevo previsto", si passa alle strutture.

2.9.27 Tre strutture di enigmi

Beatrice presenta tre strutture che un'escape room, o un gioco di enigmi, può avere (figura 2.9.5).

2.9.27.1 La struttura lineare

Un enigma che, risolto, porta a un altro enigma, che porta a un altro ancora, e così via fino all'**enigma finale**. Anche molte cacce al tesoro funzionano così.

Una bomba per uno, un problema per un gruppo. La struttura ha "una grandissima controindicazione". Se il gioco è pensato per **una persona sola**, "è una bomba". Se è per un **gruppo**, tutto il gruppo resterà concentrato su un singolo enigma alla volta, e "spesso e volentieri" una o due persone più sveglie[16], più energiche o più estroverse occuperanno molto più spazio delle altre. Tenendo tutto il gruppo su un solo enigma si crea una struttura "molto statica", in cui a un certo punto una parte del gruppo trascina il resto, "che invece si annoierà". È semplice da costruire e funziona molto bene con un solo solutore; per i gruppi, e soprattutto per le escape room formative per gruppi, "ha grosse lacune" per le dinamiche che genera. Va comunque conosciuta, perché si può progettare anche per una persona sola: "anzi, alcuni dei giochi migliori sono su questa modalità".

2.9.27.2 La struttura ad albero

Un **gruppo di enigmi** porta come soluzione a un altro enigma, che, accoppiato a un altro ancora, porta a un altro enigma. Per esempio: tre enigmi danno un numero ciascuno, e i tre numeri aprono un lucchetto.

[15] Nella trascrizione "Genialy".
[16] La trascrizione dice "più sgage"; dal contesto, più sveglie o più sicure.

Immaginata come un'unica escape room, la struttura è un **albero** in cui gli enigmi portano ad altri enigmi, fino all'enigma finale: aprire la porta, aprire il link segreto, trovare la password, "tutto quello che ci viene in mente". (La slide ha molti enigmi, osserva Beatrice: le escape room classiche hanno di solito la dinamica dei **60 minuti**, "che potete utilizzare come non utilizzare, non è quello che definisce il gioco".)

Più mani sugli enigmi. Il vantaggio rispetto alla struttura lineare è che, già in uno solo dei rami, una persona può mettere mano a più enigmi. E si può fare di più: dare **un enigma diverso a ogni persona**. "Tiziana, hai questa busta segreta col tuo nome. Stefano, hai questa busta segreta col tuo nome": ognuno ha il suo enigma di inizio. Il coinvolgimento è molto più forte; ognuno lavora sul suo enigma, "o ancora di più, se li possono scambiare". Beatrice ha ricevuto un enigma matematico "che proprio non è il suo", Stefano uno di pensiero laterale: mentre li guardano e non riescono a risolverli, se li scambiano, e si crea una dinamica di gruppo "che poi rimarrà fino alla fine del gioco". Per i gruppi è una struttura "molto molto utile".

Gruppi, non solo singoli. Uno studente chiede se "ogni ramo per una persona" fosse solo un esempio: potrebbe essere un ramo per una coppia, o per tre persone? "Certo, assolutamente".

2.9.27.3 La struttura a raccolta

È "una sorta di *mashup*" delle prime due. Ci sono molti enigmi, e ognuno può cominciare da enigmi diversi. Ogni enigma risolto dà **un'informazione**, e per arrivare alla soluzione finale bisogna **collezionarle tutte**. Due esempi:

- gli enigmi danno come soluzione parole, numeri a più cifre o codici alfanumerici; da qualche parte c'è un'indicazione che dice "dell'enigma numero 1 prendi solo la seconda cifra, dell'enigma numero 3 la quarta", e così via, fino alla password finale;
- la soluzione finale è un personaggio storico, una città, un'informazione, e ogni enigma dà un indizio. Per esempio, ogni enigma rivela qualcosa della vita di **Giulio Cesare**, e per aprire la porta bisogna dire a voce alta "Giulio Cesare", o digitarlo, o premere il pulsante con Giulio Cesare tra dieci pulsanti con dieci personaggi storici ("sto andando in brainstorming, è giusto per farvi capire").

Solo un inizio. Sono tre esempi "molto lineari, molto semplici", per essere già operativi e inventare un'escape room, durante il corso o dopo, e per avere una direzione quando c'è "un po' di caos nel creare". Ma ce ne sono tante altre, e ce ne saranno: "abbiamo ancora tanto margine di invenzione e creatività nel mondo dei giochi, per fortuna".

Approfondimento: lineare, aperta, a percorsi. Le tre strutture di Beatrice corrispondono quasi esattamente alla classificazione più diffusa negli studi sulle escape room, quella proposta da Scott Nicholson nel suo censimento delle escape room del 2015, *Peeking Behind the Locked Door*: struttura *lineare* (un enigma dopo l'altro), *aperta* (enigmi risolvibili in qualsiasi ordine, i cui risultati convergono nella soluzione finale, come nella terza struttura), *a percorsi* (*path-based*: più catene parallele che convergono) e le forme ibride, come quella detta *a piramide*, in cui gruppi di enigmi aperti alimentano meta-enigmi successivi, come nella seconda struttura. Nicholson nota lo stesso problema di Beatrice: le strutture lineari creano colli di bottiglia, e in un gruppo grande lasciano metà dei giocatori senza niente da fare.

2.9.28 Dove si trovano gli enigmi

"Ora, grossissima domanda: come troviamo gli enigmi?". Creare enigmi **originali**, mai visti, e **che funzionino** è "una delle parti più difficili" ("vabbè, non è vero, una delle") dello sviluppare un'escape room, e qui la lezione di Ruggero sul playtest "vi aprirà molte porte". Intanto Beatrice anticipa dove recupera gli enigmi e le strategie per crearli (tabella 2.9.5).

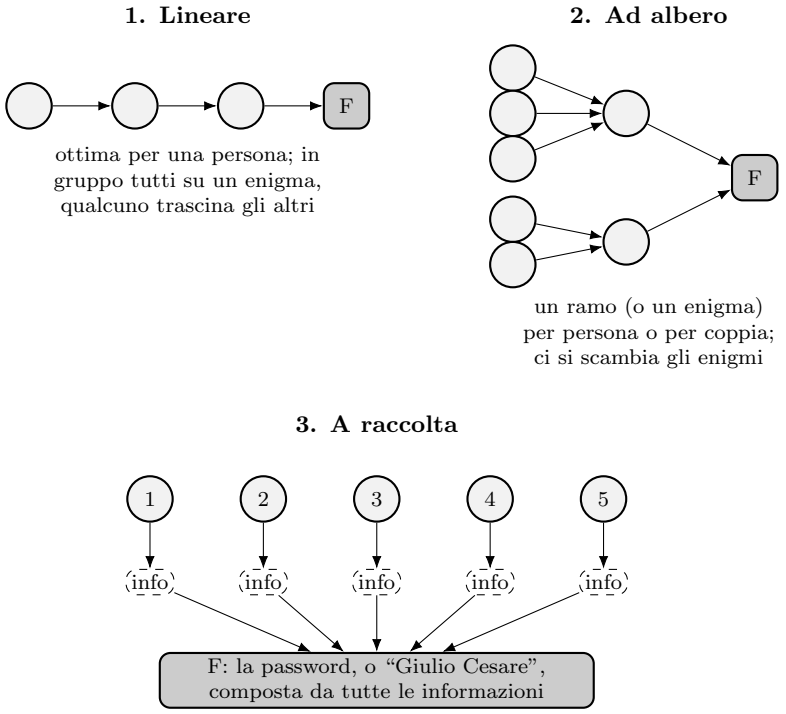


Figure 2.9.5: Le tre strutture di enigmi presentate da Beatrice. I cerchi sono enigmi, F è l'enigma o l'apertura finale. Ricostruzione schematica delle slide.

1. Giocare tanto, a prescindere. La primissima regola, già vista con la ricerca e sviluppo, è **giocare tanto**: negli enigmi fatti da altri vengono moltissime ispirazioni. A volte basta risolvere un enigma: a Beatrice è successo molte volte di non imbroccare la strada giusta, ma di restare con in mente la strada che aveva immaginato, "e quella strada può facilmente diventare un enigma", rielaborandola. Vale per i giochi da tavolo e per i videogiochi ("non ho avuto il tempo di aggiungere altri titoli", ma anche nei videogiochi e nelle app c'è "un'immensità" di giochi a cui ispirarsi). E soprattutto: "non giocate quando volete creare un'escape room: giocatene tante a prescindere", così che al momento di creare si abbiano già in mente esperienze vissute.

"Escape room" come cappello. Beatrice precisa che usa "escape room" impropriamente, come termine cappello per tutte le esperienze di enigmi: le escape room, ma anche le app, i giochi investigativi, i giochi in città, i giochi teatrali come i *murder party*[17], i giochi di enigmi in città e a teatro. E perfino film e serie TV: non ha messo la slide, "purtroppo no", ma c'è un mondo di storie che usano gli enigmi per portare avanti la trama. Il consiglio: essere curiosi di tutto ciò che riguarda questo mondo, e farsi **un proprio file** in cui segnare ciò che ha colpito: enigmi scoperti, esperienze vissute, il flusso di gioco e come è stato costruito, che cosa ha suscitato e perché, i colpi di scena. "Giochiamo tanto, esploriamo tanto".

2. Pinterest e i social. Poi si è in fase di progettazione e "non ci viene in mente manco mezzo enigma". Primo passaggio: **Pinterest**, che ha "un'infinità di enigmi di vario tipo", spesso anche escape room formative già pronte; "spesso anche non bellissime", ma con spunti per cominciare a ragionare. E i social in generale: cercando su Instagram e TikTok, "l'algoritmo vi spara dei nuovi suggerimenti". Strumenti sottovalutati, con un archivio "davvero infinito" non di enigmi "definiti", ma di ispirazioni.

3. I vecchi libri di enigmi. Altra cosa "assolutamente sottovalutata": i vecchi **libri di enigmi** delle bancarelle, "a costi molto contenuti" (anche i nuovi, "per carità"), e banalmente le **parole crociate** e i

[17] Nella trascrizione "Marvel Party".

cruciverba. Avere un piccolo archivio da sfogliare mentre si progetta dà ispirazioni e qualche direzione.

4. Osservare le cose. Ancora più sottovalutato: **osservare**. Si prende un oggetto qualunque, una forma, un testo, e nei tempi morti ("quando siete in bagno, quando si sta cuocendo la pasta, quando siete sull'autobus") ci si chiede: *come potrei trasformarlo in un enigma? Come potrei nascondere in quello che vedo una parola, un'informazione, un'immagine che non vedo?* È un allenamento nei tempi persi e uno strumento quando si progetta: per un gioco su Giulio Cesare ci si siede al tavolino, si guardano le sue immagini, le informazioni sulla sua vita, "le cose peculiari degli abiti che indossava", e di ogni cosa ci si chiede come trasformarla in un enigma. La domanda che aiuta Beatrice è: *come posso nascondere una parola o un codice alfanumerico in questa cosa che sto vedendo?* E se si vogliono usare i **cinque sensi** nelle escape room fisiche, "una cosa molto, molto figa": come nascondere un'informazione in un odore, in un sapore, in una cosa da toccare.

5. Chiedere ai tester come lo risolverebbero. L'ultima indicazione, "fuori dai denti", la fa un po' sorridere. Nella lezione sul playtest, Ruggero spiegherà che una cosa importante da raccogliere è la **verbalizzazione** delle persone che giocano: si chiede che cosa sta succedendo, perché fanno quella scelta, che cosa hanno capito del testo letto. In questo contesto, a Beatrice è successo molte volte di far testare un enigma (una cifra di lettere, un codice cifrato, un'immagine bizzarra, un oggetto da toccare) e di chiedere: *come lo risolveresti?* E la persona, mentre prova, descrive "una serie di cose che funzionano da Dio", che non sono la soluzione prevista "ma che sono una bomba". Lei se lo appunta e lo usa: in quel gioco, se l'enigma non funzionava, o in futuro. "Un enigma nuovo creato su un mio enigma vecchio": un'alternativa che "nasce proprio dall'utente".

6. Gli enigmi senza soluzione. E una cosa "testata ancora parzialmente, ma che funziona": far risolvere ai tester enigmi **di cui ancora non si conosce la soluzione**, "delle sorti di finti enigmi", per vedere come li risolverebbero. Dalle loro descrizioni nascono molte idee: molte non utilizzabili, "ma molte invece assolutamente, se rifinite, utilizzabili".

Fonte	Come usarla
Giocare tanto	Sempre, non solo quando si progetta; anche la strada sbagliata con cui si è tentato di risolvere un enigma può diventare un enigma. Tenere un file di appunti
Pinterest e social	Ispirazioni ed escape room già pronte, anche brutte; l'algoritmo di Instagram e TikTok propone altro
Libri di enigmi, cruciverba	Bancarelle, costi contenuti; un archivio da sfogliare mentre si progetta
Osservare le cose	Nei tempi morti: come nascondo una parola o un codice in questo oggetto? Anche nei cinque sensi
Chiedere ai tester	"Come lo risolveresti?": le soluzioni alternative diventano enigmi nuovi
Enigmi senza soluzione	Far "risolvere" finti enigmi e raccogliere le idee (testato solo in parte)

Table 2.9.5: Dove Beatrice trova gli enigmi e le strategie per crearli.

2.9.29 Le indicazioni del paper

"Fino a qua come sta andando?". Arrivano altre *reaction* (Beatrice nomina chi vede: "Cristiano", nella trascrizione, e Lorenzo). Si passa al **paper**: in esso l'autore[18] dà una serie di indicazioni elaborate dopo aver progettato diverse escape room in ambito **studentesco e formativo**. Beatrice le ha messe nelle slide (e si ferma ad "accendere un attimo la luce, che non mi si vede più la faccia"); il paper sarà caricato per approfondirle (tabella 2.9.6).

[18] Il titolo e l'autore del paper non sono detti a lezione; il paper sarà caricato nella cartella condivisa.

1. Il lavoro di gruppo non produce collaborazione da solo. "Non dare per scontato che il lavoro di gruppo produca in automatico della collaborazione di gruppo". Anzi: il gioco e i *puzzle* vanno progettati in modo da **creare la necessità** di collaborare. Lo si fa con un'**asimmetria informativa o meccanica**: "mi mancano dei pezzi". Beatrice ha dei pezzi dell'enigma, ma non tutti; gli altri hanno altri pezzi dello stesso enigma, e magari non sono esplicitati, così lo si scopre giocando: l'oggetto che lei ha in mano per l'enigma A contiene anche "delle letterine, delle cosine" che servono per l'enigma B; e provando a risolvere il B si accorge che quelle letterine richiedono altre letterine nascoste negli oggetti dei colleghi. "Questa asimmetria costringe la collaborazione del gruppo".

2. La linearità non porta collaborazione. Come si è visto, un enigma che porta a un altro enigma, "punto", tende a non far collaborare ed è più adatto al gioco in solitario. Per far lavorare il gruppo come un gruppo ha senso dividere in tanti **piccoli enigmi**, che l'autore chiama proprio **sottopuzzle**: ognuno dedica la sua attenzione a uno di essi, e pian piano si parla di come risolvere più enigmi in contemporanea. Una struttura in cui ogni enigma va risolto in un momento preciso e **blocca tutto il gioco** finché non è risolto porta invece a un lavoro di gruppo che non facilita la cooperazione.

3. Divergenza e comunicazione. Questa divisione in tanti piccoli enigmi (l'autore la chiama **divergenza**), insieme all'asimmetria ("sono due cose che si parlano"), spinge il gruppo a **comunicare**: nasce dalla struttura stessa che ognuno racconti istintivamente ciò che ha notato dell'enigma che aveva in mano. Con tanti enigmi per poche persone, ognuno guarda una parte del gioco, e viene automatico parlare e collaborare su ciò che si è visto.

4. Un solo obiettivo formativo, e ripetere. "Importantissima", e torna la regola d'oro: **rimaniamo sul semplice**. Non si punta a tanti obiettivi formativi: se ne sceglie **uno** e su quello si **ripete**. La ripetizione "è assolutamente sottovalutata": più si ripete, più è facile che la persona riceva il contenuto. "Non abbiamo paura di annoiare la persona con la ripetizione", perché il test dirà che cosa si ottiene davvero, ma "è meglio ripetere una volta in più che una in meno". Il paper lo dice con una frase che Beatrice adora: **ripetere è meglio che diversificare**, in questo contesto.

5. Prima insegnare, poi rendere necessario. Prima si **insegna la competenza**, poi si crea "da design" una situazione in cui quella competenza **serva**. Nascondere in un enigma qualcosa che serve più tardi è un modo; il paper parla proprio di competenze, e Beatrice inventa al momento un esempio: prima si dà un tabellone con le somme o con le tabelline, poi, in un altro enigma, si chiede il risultato di una tabellina specifica. È una dinamica sottovalutata, "ma in realtà vedrete, se la adotterete" (e invita a farlo anche nei test), "che è molto funzionale".

6. Il debriefing è parte del gioco. Una cosa che, crede, hanno già detto più volte Ruggero e Daniele, e che lei risottolinea: il **debriefing**. Nelle esperienze formative e in quelle emotivamente coinvolgenti è "proprio mandatorio" (e se possibile andrebbe fatto in tutte). Va previsto **già da progettazione**: il gioco non è concluso finché non c'è stato il debriefing, e le regole stesse dicono, a fine partita, "adesso si fa debriefing". Ed è un debriefing **guidato**: il designer dà indicazioni su come farlo, anche solo "il gruppo deve farsi tre domande e poi può concludere il gioco". Deve esserci un'indicazione, "perché il debriefing deve essere coerente con lo scopo di progetto". (Beatrice chiede se sia la prima volta che la classe ne sente parlare: no, "ne avevamo parlato con Ruggero"; "allora posso andare sparata".)

7. Non serve un mondo narrativo ricco. Infine, una cosa che Beatrice ha visto spesso: molto del tempo di progettazione delle escape room, formative e no, speso a creare un **mondo di gioco** e una **narrazione** molto ricchi. Non è negativo in sé; ma, tornando all'obiettivo e ai tempi a disposizione, in un'escape room "non è così necessario avere un mondo narrativo molto ricco". Un gioco funziona anche con una narrazione leggerissima, "banalissima", anche **una riga** che dà solo un obiettivo tematico; oppure con **tanti pezzi di narrazione**: se non si riesce a dare una coerenza totale tra gli enigmi e la storia scelta, si possono creare tante piccole narrazioni, "perché è un tipo di gioco che permette di farlo senza grossi problemi".

Indicazione	Come si traduce nel design
Il gruppo non collabora da solo	Asimmetria informativa o meccanica: i pezzi di un enigma sono distribuiti tra le persone, anche in modo non esplicito
Evitare la linearità	Tanti sottopuzzle risolvibili in parallelo; nessun enigma che blocca tutto il gioco
Divergenza e comunicazione	Tanti enigmi per poche persone: ognuno vede una parte e deve raccontarla agli altri
Un solo obiettivo, ripetuto	"Ripetere è meglio che diversificare"
Prima la competenza	Insegnarla (es. un tabellone di tabelline), poi renderla necessaria in un enigma successivo
Debriefing guidato	Previsto dalle regole, con indicazioni coerenti con lo scopo (es. tre domande prima di chiudere)
Narrazione leggera	Basta una riga; oppure tante piccole narrazioni

Table 2.9.6: Le indicazioni del paper sulle escape room formative, come Beatrice le riassume a lezione (le ultime due sono in parte sue).

Approfondimento: l'escape room in classe. Le indicazioni del paper sono coerenti con la letteratura sulle escape room educative, in cui la progettazione parte quasi sempre dall'obiettivo di apprendimento e non dalla storia, e in cui la fase di *riflessione* finale è considerata il momento in cui l'apprendimento si consolida: durante il gioco i partecipanti sono concentrati sulla soluzione, e solo dopo, guidati, riconoscono che cosa hanno imparato. Il debriefing di Beatrice svolge questa funzione. Il principio "prima insegnare, poi rendere necessario" è lo stesso che nel level design dei videogiochi viene chiamato *introduzione* e poi *test* di una meccanica: si mostra uno strumento in una situazione sicura, e solo dopo si costruisce una sfida che lo richiede.

La seconda pausa. "Fin qua ci siamo, e sono le 19:16". Beatrice si accorge di essersi "completamente dimenticata" di chiedere se serve una pausa; vede qualche reazione, e si prendono cinque minuti, fino alle **19:21**. "Preparatevi delle domande, se le avete, perché poi andrò un velocino. Quindi mi dovete proprio fermare per farle".

2.9.30 Dopo la pausa: la parte burocratica

"Allora, eccoci di nuovo". Beatrice chiede "un emoji da qualche parte" per vedere chi è tornato: Gianmarco ("bellissimo, molto in tema"), Giordano, "una faccia tutta triste". Si aspetta un minuto e si riparte con una parte che, ammette, "onestamente non so se vi è davvero utile, ma credo che sia importante saperlo"; e "dura proprio un minuto", quindi se non serve, "buttate nel sole".

Tre strumenti figli del GDPR. Sono strumenti **burocratici**, "due, ma tre, perché è un po' più ramificato", che è intelligente usare sia nei **playtest** sia negli **eventi dal vivo**. Probabilmente la classe ne ha già sentito parlare, perché "sono tutti figli del **GDPR**, il famosissimo trattamento sui dati europeo che, per fortuna, esiste". Beatrice caricherà a fine lezione due file come base (tabella 2.9.7).

Il consenso GDPR: per poter riscrivere alle persone. Se dopo un playtest o un evento si vuole **ricontattare** le persone in ottica pubblicitaria ("e purtroppo ricade sotto l'ottica pubblicitaria tutta una serie di cose che a volte possono sovrapporsi col playtest"), conviene, "per stare tranquille, tranquilli", far firmare il modulo. "Ve lo sintetizzo molto": che si scriva su WhatsApp, in una newsletter, o che si telefoni ("spero non sia necessario"), serve quel foglio firmato dalle persone che hanno giocato.

L'informativa sulla privacy. È il documento in cui l'organizzazione dice: "Carissimi amici, carissime amiche, stiamo organizzando questo evento dal vivo o questo playtest. Vi informiamo del fatto che utilizzeremo i vostri dati solo noi, non li vendiamo a terzi, non diamo il vostro numero di telefono a persone che non

siamo noi, e teniamo i vostri dati in modo tutelato. Tuttavia, vi chiediamo se possiamo farlo". La persona firma: "sì, puoi tenere i miei dati e non darli a nessun altro". Esistono varianti che autorizzano anche a cedere i dati a terzi: "a noi, per ora, non interessano"; e in generale "non ci interessano proprio".

La dichiarazione di esonero di responsabilità. Serve "nel caso remoto, e speriamo che non succeda", in cui un partecipante si sloga una gamba durante il gioco di ruolo, rompe una bicicletta, riga una vetrina con la bicicletta durante la caccia al tesoro: la responsabilità è della persona giocante e non dell'organizzazione. È "molto, molto importante" quando si organizza un evento dal vivo per cui non si è assicurati.

Documento	**A che cosa serve**	**Quando**
Consenso GDPR	Poter ricontattare i partecipanti (WhatsApp, newsletter, telefono)	Dopo playtest ed eventi, se si vuole riscrivere alle persone in ottica promozionale
Informativa sulla privacy	Dire come si useranno i dati (solo noi, non a terzi, conservati in modo tutelato) e chiedere il permesso	Playtest ed eventi in cui si raccolgono dati
Esonero di responsabilità	La responsabilità di un incidente resta del partecipante, non dell'organizzazione	Eventi dal vivo, soprattutto pervasivi, senza assicurazione

Table 2.9.7: I tre strumenti burocratici presentati da Beatrice. I modelli caricati nella cartella sono esempi: per l'uso reale vanno redatti da un legale.

Non usate questi file così come sono. Beatrice è netta. I file che carica servono a far vedere *come possono essere fatte* queste cose. Per essere sicuri che siano fatte bene serve il **consulto di un legale**, "che non è la mia professione e non è la professione di nessuno di questo corso di Analog". "Quindi quello che vi sto dicendo è: non prendete e usate questi file, punto". Nel corso non serve nemmeno farli firmare: si sta imparando, e intanto "iniziamo a masticare un po' la disciplina e lo strumento". Ma quando si comincerà a lavorare in modo più professionale, "non è che vi conviene, che è più sicuro, che è meglio: è necessario", e i documenti vanno redatti da un legale proprio o dell'entità per cui si crea il gioco. "Questo è imprescindibile".

Perché parlarne comunque. Perché "spesso e volentieri non se ne parla, si fanno le cose alla cieca". La maggior parte degli eventi a cui Beatrice partecipa "non fanno firmare neanche il modulo per poter fare le fotografie". Ai partecipanti può non importare niente, "ma magari l'organizzatore si mette nei casini, perché possono succedere tantissime cose spiacevoli". Il punto è sapere che queste cose esistono e che la normativa italiana chiede di farle in un certo modo. I modelli sono quelli che lei ha usato negli anni (lo scarico di responsabilità "andrebbe, immagino, aggiornato, perché ormai abbiamo l'assicurazione, ed è molto vecchio"): servono a vedere che cosa si può fare e ad avere "un minimo di paragone" quando ci si rivolgerà a un legale, o quando qualcuno dirà "questa è l'informativa che utilizziamo di solito". "Più o meno" è tutto chiaro; "ci sta che non sia chiarissimo".

La proprietà intellettuale. Un'ultima cosa, su cui Beatrice dice di sapere "molto meno", perché non usa mai **IP** già create, cioè proprietà intellettuali di altri. Se si vuole fare un gioco su Topolino o su Harry Potter, o usare una musica di Schubert, si ricade nella **proprietà intellettuale**, e bisogna esserne consapevoli. La sua indicazione, "in questa fase della vostra vita, finché non c'è qualcuno che ne sa a pacchi di questi argomenti", è: **non usate proprietà intellettuali di altri**. Esiste una normativa per cui dopo 70 anni un'opera diventa di pubblico dominio, "ma onesta, onesta, non so bene come funziona": se serve, si può chiedere ai docenti di approfondire. Chi vuole usarle deve informarsi "molto, molto, molto bene". Oppure, "che è la cosa più semplice, e secondo me anche la più utile in termini di allenarvi a progettare", usare le proprie: creare il proprio mondo narrativo, "il vostro personaggino". "È la cosa anche più bella da fare, da esperire".

Approfondimento: dati, immagini e diritti in pratica. Due precisazioni che la lezione lascia, giustamente, ai legali. La prima: nel GDPR il consenso è solo una delle basi giuridiche per trattare i dati, e il consenso al *marketing* va chiesto separatamente dagli altri trattamenti, in modo libero e specifico; le foto e i video in cui le persone sono riconoscibili sono dati personali, e in Italia la loro pubblicazione richiede di regola anche il consenso al ritratto, che è il "modulo per le fotografie" di cui parla Beatrice. La seconda: i "70 anni" sono la durata del diritto d'autore dopo la morte dell'autore, non dalla pubblicazione dell'opera. Una composizione di Schubert è di pubblico dominio, ma una sua *registrazione* recente ha diritti propri (dell'esecutore e del produttore): per un gioco, la partitura è libera, l'incisione no. Personaggi come Topolino, poi, sono protetti anche come marchi registrati, che non scadono finché vengono rinnovati.

2.9.31 Ricerca e sviluppo o mazzo di carte?

Beatrice arriva alla parte conclusiva: un esercizio con un **mazzo di carte** pensato per creare giochi nello spazio fisico. Ma prima chiede se la classe voglia tornare sulla **ricerca e sviluppo** del design sprint della volta precedente, il gioco dei funghi: la lezione si era chiusa dicendo che si sarebbe guardato il lavoro di ricerca fatto. Lei stessa non sa "quanto abbia senso", dato che ora i gruppi faranno un altro gioco; ma "se ci tenete, ci prendiamo volentieri una ventina di minuti". L'alternativa è il mazzo, "e mi piacerebbe farvi fare un po' di sperimentazione".

La classe è cotta. Tancredi preferisce il mazzo; un altro studente concorda, "è passato" il momento della ricerca sull'ipotesi dei funghi, anche se chi ne sente l'esigenza può esplorarla. Lo stesso per "entrambi i Lorenzo". "Ragazzi, avete ragionissimo, caspita". Uno studente dice di essere "abbastanza al limite della mia capacità di concentrazione oggi, per diversi motivi"; qualcuno aggiunge di avere "dubbi sinceri sulle lezioni da tre ore", ma precisa subito che non è la durata in generale: "ho giusto problemi miei", "un po' di baraonda in casa". Beatrice prende l'informazione. Giordano è per le carte; Nicola non ha preferenze, "anche perché tutto il mondo sperimentale" gli interessa ("hai aperto tutte le possibilità"). Vince il mazzo.

2.9.32 L'Urban Game Deck

Beatrice carica il file nelle dispense, così che ognuno possa aprire le carte e leggerle meglio: si chiama **Urban Game Deck**. Prova a condividere il link in chat, ma il problema dei link non cliccabili "non è stato risolto ancora"; tutti, comunque, sanno arrivare alle dispense nella cartella di didattica di Analog Game Design (lo confermano Lorenzo e Gianmarco). Intanto proietta lo schermo e ingrandisce le carte.

Che cos'è. È un **print and play**, un mazzo stampabile, fatto da un gruppo di designer "che purtroppo non è più attivo": sul sito indicato in basso a sinistra di ogni carta oggi c'è un'altra realtà che si occupa di giochi, "ma che non fa più la bellezza che facevano loro". Di mazzi per creare giochi ce ne sono "un'infinità"; questo è specifico per i **giochi nello spazio pubblico**, i giochi fisici: "immaginate Guardie e Ladri, immaginate Strega comanda color".

Un esercizio di elasticità. È un tipo di allenamento alla creazione di giochi che Beatrice non vede spesso usato nel gaming analogico, "e che però è un esercizio di elasticità mentale stupendo": una sessione di creazione di giochi che può durare **40 minuti**, "davvero molto poco". Ogni carta è una **caratteristica del gioco**; le carte si alzano una a una nel gruppo di creazione, e si decide quali tenere per progettare il gioco e quali no. Pur essendo pensato per i giochi all'aperto, è utile anche per il brainstorming visto nella lezione precedente, perché gli stessi concetti si applicano "sulle dinamiche specifiche" di un gioco da tavolo o di un gioco di ruolo.

Le carte. I colori dicono a quale categoria appartiene la carta. Beatrice ne legge diverse, prima e dopo il voto (tabella 2.9.8):

- **tipo di arma**: chi attacca deve *vedere* chi difende, e il contatto visivo è l'attacco, "come se fosse un colpo di pistola"; oppure al bersaglio viene *rubata una cosa che indossa*; oppure ci sono **carte vuote** da riempire come si preferisce (per esempio, l'attaccante usa una restrizione da definire);

- **effetto** dell'attacco, cioè il danno: si perde una capacità (non si può più camminare con una gamba, o si deve mettere una benda e non si vede più); si **cambia squadra** e si diventa attaccanti, "come poi succede in Guardie e Ladri"; non succede niente ("questa specifica classe di giocatori non subisce nessun danno"); si è **incapacitati** e ci si deve fermare finché un altro giocatore non salva;

- **condizioni di vittoria** e altre caratteristiche, come la composizione dei **team**;

- **velocità** del giocatore, una carta "costruttore" del gioco: si può *solo camminare*, non correre; oppure ci si muove "molto, molto, molto lentamente" ("la *reference* è uno zombie, immagino"); oppure ci si muove come si vuole, alla velocità che si vuole. Le condizioni sulla velocità "sono legate alle regole proprio del gioco che date inizialmente";

- **last man standing**: vince l'ultimo giocatore, o l'ultimo team, rimasto "vivo". A Beatrice ricorda *Rubabandiera*, ma si corregge ("non è l'esempio più calzante, non mi ascoltate, sono cotta anch'io"): lì vince il team che ha la bandiera, qui si parla di più team;

- **uccidere il VIP**: i giocatori vincono "uccidendo", ovviamente in senso metaforico, un personaggio con un ruolo speciale. Se si è deciso che colpire significa togliere qualcosa (come nell'illustrazione dell'alce a cui viene tolto un corno), uccidere il VIP vuol dire raggiungerlo e togliergli la coda attaccata alla cintura, o la valigetta che sta portando da un posto all'altro.

Categoria	**Carte lette a lezione**
Tipo di arma	Contatto visivo come colpo; rubare un oggetto indossato; carte vuote da riempire (es. una restrizione)
Effetto (danno)	Perdere una capacità (una gamba, la vista con una benda); cambiare squadra; nessun effetto per una classe di giocatori; incapacitato finché non viene salvato
Velocità	Solo camminare; lentissimo "come uno zombie"; libero
Vittoria	*Last man standing*; uccidere il VIP (coda alla cintura, valigetta)
Altro	Composizione dei team e altre caratteristiche non lette

Table 2.9.8: Le carte dell'Urban Game Deck lette da Beatrice. La suddivisione in categorie segue i colori delle carte come li descrive lei; i nomi delle categorie sono ricostruiti.

Tre modi di usare il mazzo. Per usarlo bisogna immaginare l'esercizio di design sprint della volta precedente:

1. ognuno sceglie **una o due carte** del mazzo e le mette sul tavolo, e si fa la **votazione con i pallini**, disegnati o adesivi;

2. si prendono le carte e si decide **una a una** quale tenere e quale no, con una discussione di due minuti "di orologio": "più lungo e dispendioso, a mio dire";

3. si disegna una **grande mappa**, sul tavolo o sulla lavagna online, con le zone *tenere* e *scartare*; con le carte tenute si fa un gioco, e mentre si progetta si decide che cosa tenere e che cosa togliere.

Ci sono tante altre modalità, "legate proprio a come si progettano le cose"; la preferita di Beatrice resta la votazione con i pallini, dopo una prima scrematura individuale. Le carte sono anche divise per **tematiche**, e si potrebbe dividere il tavolo per tematiche; "io non lo farei con questo mazzo specifico".

Niente prova. Restano quindici minuti, e "l'avervi sentiti cotti" fa pensare a Beatrice che chiedere di immaginare un gioco con il mazzo "forse è un po' troppo, è un po' *hard* come richiesta". Il mazzo è nella cartella, e non è difficile: i colori dicono la categoria, si sceglie che cosa tenere e che cosa no, e su quelle caratteristiche si crea il gioco. Il tempo rimasto lo usa per altro.

Approfondimento: le carte come vincoli creativi. I mazzi per inventare giochi funzionano perché trasformano la pagina bianca in una serie di **vincoli casuali**: invece di chiedersi "che gioco faccio?", il gruppo deve rispondere a "che gioco nasce da *questo* attacco, *questo* danno e *questa* condizione di vittoria?". È lo stesso principio delle carte di Schell, il mazzo di lenti che accompagna *The Art of Game Design*, e di molti strumenti di ideazione del design di servizi. Nel caso dei giochi di strada c'è anche un vantaggio storico: *Guardie e Ladri*, *Strega comanda color* e *Rubabandiera* sono già, di fatto, combinazioni di poche "carte" (un modo di colpire, un effetto, una condizione di vittoria), e il mazzo rende esplicita quella grammatica.

2.9.33 Come state? Il feedback sul corso

"Mi piacerebbe sentire come state, devo dire, e come state anche in linea generale all'interno del corso": usare il tempo per tirare il fiato e fissare "le nozioni che abbiamo sparato nel nostro cervello in tre ore".

Cinque ore e mezza. Uno studente sta bene, "un po' stanco anch'io": quel giorno c'erano anche lezioni dell'altro corso, quelle iniziali dei corsi lunghi, e "c'erano un po' più di cose oggi". Il mazzo l'ha incuriosito molto: se lo riguarderà, e lo stamperà ("fai benissimo"). Beatrice chiede quante ore di lezione abbiano fatto quel giorno sommando l'altro corso: "forse 5 e mezzo". "Grazie, così capisco quanto siete sotto stress".

Tre ore sono tante. Un'altra studentessa è "molto contenta del corso": le parti interattive aiutano a "smazzarsi" le tre ore. Ma tre ore "non sono poche", e si sommano alle lezioni del corso lungo: "mi capiterà di fare sei ore, tre ore alla mattina e tre ore al pomeriggio". "È bello impegnativo", anche se cerca di seguire tutte le lezioni dal vivo, "perché guardare la registrazione è peggio ancora".

Le pause. Beatrice prende nota. È, crede, all'ultima lezione del corso da sola, ma riconosce una cosa che dimentica sempre: **le pause**. "Se io, da corsista, mi trovassi a fare tre ore senza pause, farei molta fatica". Se anche la classe pensa che le pause aiutino, le farà presenti agli altri insegnanti; altrimenti la adotterà per sé. "Al netto del fatto che non sono sicura: magari fanno più pause di me e sono molto più organizzati su questo".

Che cosa si può ancora cambiare. Nell'attesa di altri interventi Beatrice spiega lo scopo della domanda: capire "lo *stress test* a cui vi stiamo sottoponendo". Alcune cose si possono modificare (le pause, i giorni); sulla quantità di ore "onestamente non lo so", deve sentire i colleghi, perché crede che sia l'unica possibilità che la scuola ha per questo tipo di corsi brevi[19]. Siccome si è ancora in tempo per le lezioni successive, sono cose implementabili, e poi da iterare per il corso dell'anno dopo. "Il come state non è un invito a raccontarmi i cavoli vostri, che è giusto che rimangano vostri": è un invito a raccontare quelli legati all'attività di lezione e al percorso. Ricorda che a fine corso ci sarà il **form**, **anonimo**, e che si può sempre scrivere ai docenti, anche da una mail anonima "se vi manda in crisi", o dalla propria.

Fine. Non arrivano altri interventi, e Beatrice chiude "5–10 minuti prima", "così potete andare a rilassarvi". Il mazzo è già nelle dispense; il resto (il documento sul project management e i materiali legali) lo carica subito dopo. Per qualsiasi domanda si può scriverle: "avete la mail". Ci si rivede al *pitch*: "devo riguardare il calendario, però credo di sì"; "non è tra tantissimo: è il 12 ottobre, tra una settimana". "Quindi buon lavoro e in bocca al lupo".

[19] La trascrizione nomina la scuola come "IV"; il nome non è ricostruibile con certezza.

Punti chiave del capitolo

- Organizzare la progettazione è una *forma mentis*: valutare insieme **tempi**, **competenze**, **energie** (i "cucchiai") e **risorse economiche e fisiche**. Si impara in anni, non in lezioni.
- Pensare ogni progetto come una **game jam**: darsi un tempo anche quando nessuno lo dà. Nel corso: *pitch* il 12 ottobre, consegna il 4 dicembre.
- Il documento "**Project management di un gioco**" riassume ciò che esce dal design sprint: obiettivo e target, **vincoli** (scadenza, budget, spazio, materiali, numero di partecipanti, strumenti, committente), ore, fasi con *output* atteso, emozioni, meccaniche, percorso dell'utente, prototipo, playtest, iterazione, documentazione, monitoraggio. Non è obbligatorio.
- Ragionare **in ore**, **per eccesso**; allocare ogni fase e poi chiuderla ("finito il tempo, non si fa più ricerca"). **Enorme regola**: la somma delle ore allocate deve stare sotto le ore disponibili. In sintesi, **+20%**, di più per un gruppo nuovo.
- Lo scarto tra stima e realtà è **un dato, non un giudizio**. Senza dati, stima **di pancia**, mai zero; con due o tre progetti si scopre se si stima largo o stretto. Serve anche a valutare proposte e a non accettare *forfait* senza senso.
- Il **percorso dell'utente** esplicita ogni passaggio, dall'inizio (sedersi, aprire il regolamento, uscire di casa) alla fine: serve a trovare il passaggio mancante ("non ha la penna").
- Nel **primo prototipo**: che cosa è necessario e va fissato e testato subito (gli enigmi principali), che cosa resta provvisorio, che cosa non va ancora fatto. Il ciclo crea-testa-raccogli-itera si fa "infinite volte", e ciò che si scarta non è mai buttato.
- **Documentare sempre**, per capire come si lavora e per raccontarlo fuori. **Semplice è meglio**: qual è la versione più semplice dell'idea che dà ancora l'esperienza voluta?
- Il **Game Design Document** è la "Bibbia" del gioco dei manuali e del videogioco; nell'analogico italiano quasi non si usa (si lavora sul regolamento), ma serve nei bandi. Il corso chiede il gioco, non un documento. Le **tre grandi domande**: chi gioca, che cosa gioca, come è fatto.
- **Gamification**: elementi di gioco in contesti non di gioco, che toccano soprattutto la motivazione **estrinseca** (Duolingo). **Game based learning**: un'esperienza completa. Il gioco è **volontario**: senza una vera "scelta B" non c'è libertà. Sugata Mitra: **create le condizioni** invece di spiegare.
- **Escape room**: la struttura **lineare** va bene per una persona, male per un gruppo; quella **ad albero** e quella **a raccolta** distribuiscono gli enigmi. Gli enigmi si trovano giocando tanto, su Pinterest, nei vecchi libri, osservando le cose e chiedendo ai tester "come lo risolveresti?".
- Il paper: **asimmetria** per costringere alla collaborazione, sottopuzzle in parallelo, **un solo obiettivo formativo ripetuto**, prima insegnare la competenza e poi renderla necessaria, **debriefing guidato** previsto dalle regole, narrazione leggera.
- Burocrazia: consenso **GDPR**, **informativa sulla privacy**, **esonero di responsabilità**; i modelli sono esempi, per l'uso reale serve un legale. Niente **proprietà intellettuali** altrui: create le vostre.
- L'**Urban Game Deck**: un mazzo *print and play* di caratteristiche (armi, effetti, velocità, vittoria) per inventare giochi di strada in 40 minuti, con la votazione a pallini.

Chapter 2.10

UX design e paragame: l'interfaccia del gioco e tutto ciò che non è né regole né fiction

Questa è la **decima lezione** del corso di **Analog Game Design**, e l'ultima prima del *pitch* del 12 ottobre. Il suo tema è annunciato da Ruggero già nei primi minuti, con un entusiasmo che non nasconde: "oggi, lezione sul paragioco. Quindi, sono carichissimo". La lezione ha due metà. La prima è dedicata alla **UX** (*user experience*) del gioco, "una roba gigantesca" che non riguarda solo il gioco analogico ma tutti i campi del design, e di cui il corso dà solo le basi: che cosa sta tra la persona che si siede al tavolo e l'ecosistema di regole, e quali sono i **quattro pilastri** di una buona UX (**immediatezza**, **efficienza**, **memorabilità**, **soddisfazione**), mostrati con decine di icone e componenti di giochi veri, più o meno riusciti. Chiude la prima metà una parte sull'**accessibilità**. Ruggero la presenta come la parte di "cose che magari già sapete".

La seconda metà, sul **paragame** (o *paragioco*), è invece "una delle mie parti preferite", e "sempre una parte delle lezioni abbastanza che spacca i cervelli". Il paragame è l'estensione al gioco del concetto letterario di *paratesto*: tutto ciò che di un gioco **non è regole e non è fiction**, ma incide sull'esperienza di chi gioca. Il materiale dei dadi, lo spessore delle carte, il numero di pagine del regolamento, la forma della scatola, il font, il tavolo su cui il gioco verrà apparecchiato, la musica, il contesto culturale, la lingua, la città. Dopo una carrellata delle aree progettabili, Ruggero mostra una lunga serie di **casi studio**, "da giochi più regolari a cose più *deranged*": un wargame con sei brochure, un gioco di ruolo fatto solo di un mazzo di carte, un videogioco che chiede di stampare un manuale, una gru da mettersi in testa, delle tessere di ghiaccio da tenere in freezer. Il messaggio di fondo è uno, e Ruggero lo ripete in molte forme: niente di tutto questo è "contorno". Il paragame **incide sulle regole** e sull'esperienza, e chi progetta "può lavorarci attorno" solo se ci ragiona. Per questo, ai *pitch* e alle revisioni, il docente chiederà "tantissimi perché".

La lezione è **online** e la conduce da solo **Ruggero**, condividendo le slide. Si colloca con ogni probabilità **venerdì 9 ottobre 2026**[1], dalle 17 alle 20 circa. Ci sono due pause: la prima, di dieci minuti, alla fine della parte sulla UX ("alle sei ci ritroviamo qua"); la seconda dalle **18:57** alle **19:05** ("ve ne do 8 per il numero tondo"), prima dei casi studio. All'inizio sono collegati, tra gli altri, **Tancredi** e **Gianmarco**; nel corso della lezione intervengono anche **Tiziana** e **Lorenzo**[2]. Tiziana lascia la lezione verso la fine ("Ciao, Tiziana").

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi, a cominciare da quelli dei due argomenti della lezione. La UX diventa "Wix", "l'usso" e "Wix design"; il paragame diventa "paragain", "paraghino", "paraggio", "parachain", "paragate", "paradigma", "paracadema" e perfino "guerra game". *Race for the Galaxy*, il gioco più citato della serata, è "Restore the Galaxy", "Rescord Galaxy", "RIS4D Galaxy", "Risordiality", "Redsward Galaxy", "Days

[1] La data non è detta, ma è ricostruibile. Nella lezione su Tabletop Simulator Ruggero aveva indicato venerdì 9 ottobre come la data della lezione sul paragame. In questa lezione dice che il *pitch* "del 12" è "lunedì", e in chiusura gli studenti si salutano con "buon weekend".

[2] La trascrizione non distingue tra i due Lorenzo del corso (Lorenzo Vergata e "l'altro Lorenzo"); in un passaggio Ruggero chiama "Tancredi, Lorenzo. Lorenzo", il che fa pensare che siano presenti entrambi. Il capitolo scrive semplicemente "Lorenzo".

for the Galaxy" e "Risk for the Gala". Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: durante le due pause la trascrizione registra per decine di righe frasi senza alcun rapporto con la lezione, ripetute in loop ("grazie a tutti per l'attenzione che ci ha spiegato", "qual è la risposta che ci può essere?"), e subito dopo la seconda pausa qualche frase spuria ("la situazione di oggi è la migliore dal 2000", "otto orecchie"). Sono artefatti del riconoscimento vocale e il capitolo le ignora. La terza: gli studenti intervengono spesso senza che la trascrizione segnali chi parla; il capitolo li nomina quando il contesto lo rende chiaro e resta generico altrimenti.

Come leggere questo capitolo. La lezione è costruita quasi interamente su **immagini**: icone, carte, scatole, componenti, foto di giochi apparecchiati. Nessuna di queste immagini è nella trascrizione. Il capitolo ricostruisce ciò che Ruggero ne dice e, dove serve, descrive l'immagine a parole; quando un dettaglio dell'immagine non è ricostruibile, lo segnala. Le tabelle riassumono le liste (i quattro pilastri, le aree del paragame, i casi studio); le figure ricostruiscono gli schemi (il posto della UX tra la persona e il gioco, le tre aree sovrapposte di regole, fiction e paragame, il ventaglio di carte in mano). Le slide della lezione, dice Ruggero, sono già nella cartella condivisa del corso. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell) e alla letteratura sul design dell'interazione e sul paratesto; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa e due domande su Tabletop Simulator (sezioni 2.10.1 e 2.10.2);
2. la **UX del gioco**: il suo posto nella *pipeline* e i **quattro pilastri**, ciascuno con i suoi esempi (sezioni da 2.10.3 a 2.10.7);
3. quante informazioni contiene una carta, il **wireframe** che cambia nel prototipo, l'**accessibilità** (sezioni da 2.10.8 a 2.10.9);
4. dopo la prima pausa, l'esercizio del **libro consigliato** e la definizione di **paragame** (sezioni 2.10.10 e 2.10.11);
5. le **aree progettabili** del paragame, una per una (sezioni da 2.10.12 a 2.10.23);
6. dopo la seconda pausa, i **casi studio** (sezione 2.10.24);
7. la **discussione aperta** e la chiusura (sezioni da 2.10.25 a 2.10.26).

2.10.1 L'attesa: il 12 "sta nella pietra"

Ruggero si collega con qualche minuto di ritardo ("pensavo di essere in ritardo": c'è stato un cambio di aula, "ma siamo in orario"), e il clima è quello di fine settimana. Si parla del caldo ("fa ancora troppo caldo per i miei gusti"), poi Ruggero chiede come siano andate le lezioni con Beatrice ("interessante", "bene") e come procedano i progetti di gruppo. Arrivano alla spicciolata gli studenti del **gruppo 2**, che hanno appena fatto "un piccolo meeting" e raccontano di aver **cambiato completamente** l'idea proposta. Ruggero non si scompone: "è normale". C'è tempo fino al 12, "però il 12 sta nella pietra". Uno studente, appena entrato, ammette di essere "un po' incasinato col gioco", e la risposta è la stessa: "ma è normale". Quando qualcuno, scherzando, teme che al *pitch* i docenti "sfasceranno" le idee dei gruppi, Ruggero rassicura: "no, non vi sfasciamo niente".

Ruggero aggiunge una promessa che riguarda direttamente il *pitch*: la lezione di oggi è "proprio fatta apposta" per chi ha ancora "qualche buco di dubbio". In particolare la seconda parte darà "delle idee intense", che "potrebbero portarvi a riparlare tra di voi di varie cose in gruppo"; ed è sicuro che "questo porterà lunedì ad avere dei *pitch* incredibili".

Come al solito, prima di cominciare apre lo spazio per "domande, dubbi, consigli, insulti", "altrimenti stiamo qui in silenzio a osservare lo scorrere del tempo, e lo sapete che non fa piacere a nessuno". E annuncia il tema: "UX design e paragame". Sulla prima parte, "molte persone tra voi già sono sul pezzo", perché la UX "non è una cosa esclusiva del gioco da tavolo" ma tocca tutti i campi del design; la seconda, invece, nelle edizioni passate è "sempre una parte delle lezioni abbastanza che spacca i cervelli. Quindi, non vedo l'ora".

2.10.2 Due domande su Tabletop Simulator

Una studentessa ne approfitta per due domande "molto semplici" su Tabletop Simulator, il software della lezione precedente[3].

Prendere un mazzo intero senza tenere premuto. La prima domanda riguarda un fastidio pratico: per prendere un mazzo di carte intero, e non una carta sola, bisogna tenere premuto il tasto del mouse per un paio di secondi. C'è una scorciatoia? Ruggero ricorda prima quella per **pescare**: basta passare col mouse sopra il mazzo e, senza cliccare, premere sulla tastiera il **numero** di carte da pescare ("premo 5 e mi dà 5 carte in mano"). Ma la studentessa vuole *spostare* il mazzo intero, e le capita spesso. "Che io sappia, bisogna tenere premuto", risponde Ruggero, che non ne ha mai avuto bisogno: se capita spesso, forse è "più un problema di logistica di come il modulo è *set up*", e si risolve "setappando il modulo bene". Forse esiste un "control-click", "non credo ci sia, ma chi può dirlo. Se lo scopri, dimmelo".

Simulare le luci. La seconda domanda riguarda il gioco a cui la studentessa lavora: un gioco con un **lumino** che proietta un'ombra, giocato "tutto al buio". Su Tabletop Simulator ha visto un modulo che simula le luci: si può fare? "Serve script? Assolutamente, per forza." E Ruggero sconsiglia "fortemente" di testare su TTS cose che lo richiedono: lì "si apre il vaso di Pandora degli script", una cosa impegnativa che il corso non copre perché usa **Lua** come linguaggio di programmazione, e fare "delle lezioni di Lua apposta per dei casi limite" non ha senso nel programma. La studentessa chiede se basti copiare l'asset di quel modulo. "Puoi copiare l'asset se lo salvi", ma dipende da come è stato costruito. Se chi l'ha creato è stato "furbo", lo script è dentro l'asset stesso; ma a Ruggero è capitato di trovare moduli con **script diffusi**, in parte in un componente e in parte in un altro: se se ne copia uno solo, "si rompe tutto e non funziona". Ruggero non conosce quel modulo: "poi fare un po' di prove, sicuramente".

Approfondimento: perché il gioco "al buio" è un tema di questa lezione. La domanda è più vicina all'argomento della serata di quanto sembri. Un lumino che proietta ombre su un tavolo buio è un elemento di *paragame* (luce, materiali, ambiente) che diventa una meccanica: proprio il tipo di intreccio che Ruggero mostrerà con *Nyctophobia*, *Scandal in the Deep* e *Meltdown* (sezione 2.10.24). Ed è anche un buon esempio del limite pratico del prototipo digitale: ciò che dipende dalla fisica e dalla percezione (la luce, il peso, la temperatura) si simula male in un software, e va provato sul tavolo vero.

2.10.3 La UX nel gioco: uno strato tra la persona e le regole

Ruggero condivide lo schermo e apre le slide: "UX design e paragame", una lezione "divisa più o meno in due", con "una pausa a metà" ("non sarà tutta una mitragliata secca di cose"). La seconda parte sarà "contemporaneamente un po' più intensa, ma anche un po' più *relaxed* della prima. Lo so, mi direte: com'è possibile? Lo vedrete".

Si parte dalla UX, con una premessa: è "una roba gigantesca" e "questo non è un corso di UX design"; la lezione si limita a dare "delle basi su cui ragionare". Gli esempi saranno quasi tutti di **giochi da tavolo**, e non per caso: è un genere che, più di altri, "ha richiesto in passato un uso della UX fatto meglio" rispetto, per esempio, a un gioco di ruolo, o a un LARP in cui "non è detto che venga neanche usato il materiale stampato".

2.10.3.1 Dove sta la UX

Il punto di partenza è una *pipeline* che la classe "dovrebbe già sapere": una **persona** che vuole giocare, il **gioco** (l'ecosistema di regole) e l'**esperienza** che ne nasce. Il lavoro della UX "sta qua in mezzo": è "un primo *layer* tra la persona che si approccia al gioco e l'ecosistema di regole del gioco" (figura 2.10.1). E "spesso non ci si pensa, ma può essere un incredibile aiuto o un gigantesco ostacolo". Si può avere un ecosistema di regole "lisciissimo, pulitissimo, fantastico", in cui "funziona tutto, bilanciato tutto"; "ma se

[3] La trascrizione non dice chi fa le domande; dal contesto è una persona che sta lavorando a un gioco "per conto mio", fuori dal progetto di gruppo.

a livello di interfaccia questa roba non arriva, non si riesce a giocare, non si riesce neanche a capire come giocare. Quel gioco finisce nel cestino".

Di qui la regola: quando si progetta un gioco "non ragioniamo solo alle meccaniche del gioco, ma anche a cosa c'è davanti al gioco". Che cosa c'è "prima ancora delle questioni di meccaniche, delle regole, ma anche prima del manuale": il regolamento in quanto oggetto, "come sono fatte le carte, come il gioco è apparecchiato sul tavolo".

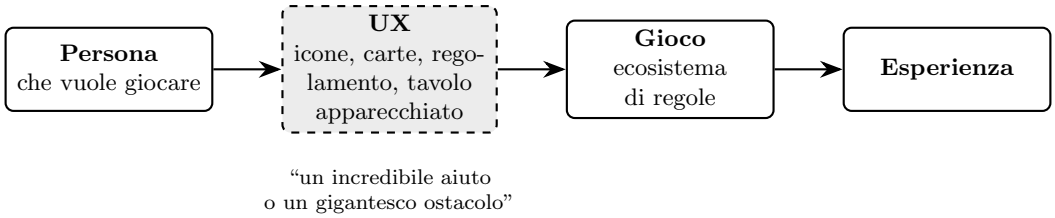


Figure 2.10.1: Il posto della UX nella *pipeline* persona–gioco–esperienza: un primo strato tra chi si avvicina al gioco e l'ecosistema di regole. Se questo strato non funziona, anche regole perfette non arrivano a chi gioca.

2.10.3.2 I quattro pilastri

Per progettare la UX di un gioco, Ruggero dà "quattro pilastri che è bene tenere a mente": una buona UX ha **immediatezza**, **efficienza**, **memorabilità** e **soddisfazione** (tabella 2.10.1). Le sezioni che seguono li vedono uno alla volta, "nel dettaglio", con esempi presi da giochi reali. Una precisazione vale per tutti gli esempi: "non vi serve sapere questi giochi da dove arrivano". Anzi, per alcune domande è meglio non conoscerli.

Pilastro	Che cosa vuol dire	Esempi della lezione
Immediatezza	si capisce subito il significato, con il minimo sforzo e senza confusione	spade e scudi sui dadi; il barile di petrolio; la corona d'alloro dei punti vittoria. Contro-esempi: *Race for the Galaxy*, *Argent*
Efficienza	racchiudere un'azione complessa nel minor spazio possibile	l'icona "pesca una carta"; l'icona del punto vittoria; le icone di *Race for the Galaxy* (poco immediate, ma molto efficienti)
Memorabilità	restare in mente, così che a distanza di anni si ricordi ancora che cosa fa	il +4 di *Uno*; il simbolo di "tappare" di *Magic*; le icone di *The Gallerist*
Soddisfazione	"bonus point": interagire con la grafica e i componenti è piacevole, gratificante	il tracciato a frecce di *Parks*; le carte trasparenti di *Canvas*; le armi in miniatura di *Tiny Epic Mechs*

Table 2.10.1: I quattro pilastri di una buona UX secondo Ruggero, con gli esempi mostrati a lezione.

2.10.4 Immediatezza

Un elemento grafico è **immediato** quando "comprendiamo subito il significato di quello che vuole fare, con il minimo sforzo e senza confusione". Il contrario è chiaro: se vediamo una cosa e non capiamo che cosa fa, dobbiamo sforzarci, ci confonde, "magari facendoci anche sbagliare le regole", e allora "quella roba lì è da buttare nel cestino".

2.10.4.1 Un tiro di dadi

Il primo esempio è un **tiro di tre dadi** di un gioco che nessuno sa quale sia. Le facce uscite mostrano **due spade**, **uno scudo** e **un fulmine**. "Che cosa ci dice questo lancio di dadi?" Le risposte arrivano subito: "che siamo stati fulminati", poi, più seriamente, "due attacchi, una difesa, e il fulmine non saprei". A livello di *vibes*, è "un lancio di combattimento, evidentemente", "perché ci sono le spade, gli scudi" ("diciamo, combattimento sotto la pioggia col fulmine", scherza Ruggero). E in che tipo di gioco potrebbe stare? Uno studente pensa a qualcosa come *Civilization*, la conquista di un territorio; un altro a un gioco fantasy, e cita un titolo che la trascrizione rende "Cronache di Havel"[4].

Il punto è che tutti hanno capito **immediatamente** che le spade sono attacco e lo scudo è difesa, e che il tiro è "più aggressivo che difensivo"; e nessuno sa che cosa sia il fulmine. "Ed è sempre così, quando io uso questo esempio." Spade e scudi sono usati "in talmente tanti giochi", a livello di **euristiche personali**, che si capiscono "al volo anche se non sapete i dettagli di questo gioco": non si sa come funzionano il combattimento, la difesa, la vita, ma si sa che due spade e uno scudo fanno un tiro aggressivo. Il fulmine, nel gioco, è **l'attivazione di un'abilità**, ma è "un'icona poco adatta a fare questo", perché nessuno l'ha riconosciuta come tale. Ed è normale: è molto più difficile trovare un'icona che rappresenti "in generale un'abilità, che può essere potenzialmente qualunque cosa", mentre spada e scudo sono di fatto delle **sineddoche**, "delle metafore, in questo caso, di *ti sto attaccando* o *mi sto difendendo*".

2.10.4.2 Quattro risorse

Il secondo esempio sono le icone di **quattro risorse**, in alto a destra nella slide. "Chi è che sa da quale gioco vengono?" Nessuno: "best result", perché la domanda "crolla" se si conosce già il gioco. Le icone riproducono i segnalini di legno delle risorse, con il loro colore e la loro forma. Che cosa rappresentano?

Le risposte della classe sono istruttive. Uno studente va "più sul colore che sulla forma": il giallo è il grano, il grigio è la pietra, il marrone "direi legno". **Tancredi** vede nella quarta, color ottanio, il **petrolio**, "perché mi sembra di qualche gioco che avevo provato". Tiziana nota che "la forma è contraddittoria", non corrisponde al colore; e una studentessa confessa di vederci tutt'altro: "una bara, una lapide, il lumino dove si mette il cero e una colonna che sorregge", insomma un cimitero.

Ruggero scioglie l'enigma partendo dalle due più semplici. Il **legno** "nessuno lo sbaglia": è a forma di tronco tagliato, nei limiti di un piccolo componente. Quella indicata come pietra è in realtà **metallo**, ma nel gioco non si confonde: il gioco è ambientato in un mondo *dieselpunk*[5], "hai questi giganteschi macchinari, per costruirli ti serve metallo", e quindi quello viene preso come metallo. Quello giallo confonde sempre chi non conosce il gioco, perché sembra "un sacchetto di qualcosa di giallo": le due risposte possibili sono oro o grano, e qui è **grano**. E quello in cui "non ci si sbaglia di solito" è proprio la risorsa color ottanio: "Tancredi l'ha detto giusto, è petrolio", perché usa "la colorazione dell'ottanio, che è il colore che culturalmente noi associamo alla benzina, al gasolio, al carburante". È forse l'icona più riconoscibile di tutte, perché è un **barile di petrolio**. Il contesto, poi, conta: chi pensasse a un gioco come *Concordia*, ambientato nel Mediterraneo antico, "non ha senso che sia petrolio".

Ne esce un'osservazione in più: **non è solo la forma**; anche il **colore** può essere fondamentale per riconoscere un'icona. "Se io vi facevo il tronco viola, non mi avreste detto legno. Se vi facevo il grano rosso, non mi avreste detto grano o soldi, mi avreste detto qualcos'altro."

2.10.4.3 La corona d'alloro

Terza domanda: l'icona in basso a destra, una corona con un **3**. "Tre punti vittoria." Perché? "Perché c'è l'alloro." E perché l'alloro significa punti vittoria? Perché l'alloro è un premio, un riconoscimento; e la **corona d'alloro** "si è normalizzata, specialmente nei giochi da tavolo *German*, negli Eurogame, come simbolo dei punti vittoria". In centinaia di giochi la coroncina d'alloro rappresenta i punti, e chi trova un gioco con una corona d'alloro e un 3 sa che sono tre punti.

[4] Con ogni probabilità *Le Cronache di Avel* (*Chronicles of Avel*), gioco fantasy cooperativo con combattimenti a dadi. Il gioco della slide, invece, non viene nominato.

[5] La trascrizione dice "in cui fai è site". Le quattro risorse descritte (legno, metallo, grano e petrolio) e l'ambientazione dieselpunk con giganteschi macchinari sono quelle di *Scythe* (Jamey Stegmaier, 2016), che è quasi certamente il gioco della slide.

Qui Ruggero fa una distinzione importante: **immediato non vuol dire immediato per chi non ha mai giocato**, "a tutte le persone". L'alloro è immediato per chi gioca abitualmente a giochi da tavolo, perché fa parte di un linguaggio condiviso. Poi, "chiaramente, la forza di un'icona forte è essere immediata anche per chi non ha giocato il gioco".

2.10.4.4 Le icone che fanno il contrario

Ci sono però icone che fanno "il contrario di quello che si dovrebbe fare di solito". Le prime due della slide, "che ci crediate o no, sono due icone diverse meccanicamente, che fanno due cose diverse". Ruggero non dice che cosa fanno ("ho una sanità mentale da risparmiarmi"): vengono da un gioco che la trascrizione chiama "Gemoni"[6], "bel gioco", ma con "molte icone molto simili", come queste due, che hanno i colori praticamente invertiti e fanno cose diverse.

Poi due icone di ***Race for the Galaxy***[7]. Gianmarco, che lo conosce, viene escluso dalla domanda ("non lo chiederò a te, perché *Race for the Galaxy* è una maledizione"). Il gioco ha questa proprietà: "per chi non lo conosce, guarda le icone e pensa: questi sono pazzi, questo gioco non ha senso. Per chi lo conosce, queste icone sono le più leggibili del mondo ed è immediato capire che cosa fanno". Ruggero prova a far indovinare la classe ("non do voti all'interrogazione sulle icone di *Race for the Galaxy*, anche perché sarebbe un disastro per chiunque, *me included*"), con un suggerimento: è un gioco fantascientifico.

Sulla prima icona, Lorenzo prova "annulla un comando" ("ti sorprenderà sapere che non è così"), un altro "cambio colore" ("ti piacerebbe; non è un gioco facile come *Uno*, questo"). Poi una studentessa tenta un'"archeologia ludica": se ci sono carte di vari colori, forse scartando una carta di quel colore succede qualcosa da qualche altra parte. "Un terzo l'hai beccata." Gianmarco completa: **scarti una carta e la usi come risorsa** da mettere su un **pianeta**, "che è quella cosa tonda".

"Adesso che abbiamo decifrato parte del linguaggio", la seconda icona riesce meglio. Uno studente: "se hai tre carte blu, ottieni un esagono che non so cosa sia, e peschi una carta?" "Vicinissimo, non sai quanto." L'icona parla di carte blu, ma non di carte qualsiasi: di carte **usate come risorsa blu**, quelle appena viste. E non ne servono tre: il numero dice che si può fare **fino a tre volte** la cosa che segue, cioè ottenere un **esagono** (i punti, in questo gioco) e pescare una carta. Tradotto: "posso usare da una a tre risorse per prendere da uno a tre punti e da una a tre carte".

Infine l'icona in basso al centro, che Ruggero non spiega: "è un'icona per cui dovrebbero arrestare i graphic designer immediatamente". Contiene, "se non vado errato", una specie di **talismano** di carta scritta, uno **scudo** e **due cappelli**, di cui uno in ombra. Viene da ***Argent***[8]: "gioco bellissimo, icona illegale". Che non sia immediata, "vi rendete conto anche voi".

Approfondimento: convenzioni, affordance e linguaggi appresi. Gli esempi di Ruggero corrispondono a una distinzione classica del design dell'interazione. Alcuni segni funzionano perché si appoggiano a **convenzioni** culturali molto diffuse (la spada per l'attacco, il colore ottanio per il carburante, l'alloro per la vittoria); altri richiedono di imparare un **linguaggio** specifico del singolo gioco, come le icone di *Race for the Galaxy*. Donald Norman, in *The Design of Everyday Things*, parla di *significanti* (*signifiers*) per indicare i segni che comunicano che cosa si può fare con un oggetto, e osserva che le convenzioni sono potenti proprio perché non vanno reimparate a ogni nuovo oggetto. Il limite che Ruggero indica (immediato "per chi gioca" non è immediato "per tutti") è lo stesso: una convenzione vale solo dentro la comunità che la condivide, e un gioco pensato per un pubblico nuovo non può darla per scontata.

2.10.5 Efficienza

Il secondo pilastro è l'**efficienza**: "comunicare più informazioni insieme, alleggerendo l'aggregazione di informazioni". Ruggero la spiega con "due esempi diretti".

[6] Il titolo non è ricostruibile con sicurezza dalla trascrizione.

[7] La trascrizione lo storpia in quasi ogni occorrenza: "Retribuzione", "Restore the Galaxy", "Rescord Galaxy", "RIS4D Galaxy", "Risordiality", "Redsward Galaxy", "Days for the Galaxy", "Risk for the Gala". *Race for the Galaxy* (Thomas Lehmann, 2007) è un gioco di carte di fantascienza celebre per la sua iconografia densa.

[8] *Argent: The Consortium* (Trey Chambers, 2015), gioco di piazzamento lavoratori ambientato in un'università di magia.

Pescare una carta. La prima icona mostra un mazzo di carte. Che cosa fa? "Peschi." Perché? "Perché c'è un mazzo di carte." Ma Ruggero insiste: che cosa si fa *fisicamente*, "a livello di propri atti fisici", seduti al tavolo, per attivare questa icona? "Rivelo la prima carta del mazzo, prendo la prima carta del mazzo, e poi... la metto in mano." Per chi gioca molto è una cosa semplicissima, "solo perché abbiamo giocato tonnellate di giochi". Ma l'atto in sé è complesso: c'è un componente che è un mazzo, che contiene tante carte; ne prendo la prima, la sollevo dal mazzo e la aggiungo alla mia mano, di cui ora fa parte. "È un'azione complessa, e un'icona così ci riassume tutto questo processo."

Il punto vittoria. Lo stesso vale per l'icona del punto vittoria vista prima. Che cosa si fa con un punto vittoria? "Muoviamo la pedina." Se il gioco ha un tracciato dei punti, quell'icona, "nella sua piccolezza", dice: prendi la tua pedina dei punti vittoria e muovila avanti sul tracciato di tante caselle quanti sono i punti.

"Sono esempi piccoli e banali. Non sottovalutate questa cosa qua." Ruggero ne ricava la definizione: un'icona efficiente **non** deve essere efficiente per chi non conosce il gioco; un'icona efficiente "è in grado di racchiudere al suo interno un'azione complessa nel minor spazio possibile". Ecco perché le due icone di *Race for the Galaxy* sono un caso interessante: sono **molto poco immediate**, ma **molto efficienti**. In uno spazio piccolissimo danno tante informazioni e dicono molto bene che cosa fare, "una volta che abbiamo decifrato l'alfabeto di *Race for the Galaxy*". I pilastri, insomma, sono indipendenti: un'icona può averne uno senza l'altro.

2.10.6 Memorabilità

Terzo pilastro: una buona icona deve "rimanere nella mente, in modo da facilitare il ricordo durante le partite successive". Il criterio di Ruggero è concreto: "se io gioco un gioco e poi lo rigioco cinque anni dopo, e so ancora che cosa fa quell'icona, quello è un buon gioco".

Il +4 di *Uno*. La prima carta della slide: che cosa fa? "Pescare quattro carte." Chi pesca? Il giocatore successivo. Il gioco è *Uno*. "E quand'è che hai giocato l'ultima volta?" "Boh. Almeno un paio d'anni fa." Non ci si ricorda nemmeno quando, ma ci si ricorda che cosa fa la carta. Perché è una carta **memorabile**: "quando ti viene giocata contro, bestemmie". Giocarla, o subirla, dà "un *boost* emotivo tale" che quello che fa diventa memorabile, "quasi memetico".

Il simbolo di "tappare". La seconda icona: dove nasce, che cosa chiede di fare? Una prima ipotesi ("invertire il turno") è sbagliata; poi uno studente fa l'"ipotesi molto scontata": ***Magic: The Gathering***. L'icona chiede di **tappare** la carta, cioè di **ruotarla di 90 gradi**: una carta tappata ha usato la sua abilità e non la si può riutilizzare nel turno. "Quand'è l'ultima volta che hai giocato a *Magic*?" "Cent'anni fa." Eppure ci si ricorda ancora che quel simbolo vuol dire "prendi la carta e ruotala di 90 gradi". Il motivo, qui, è diverso da *Uno*: è che il simbolo è **pervasivo**, "è ovunque"; chi ha giocato a *Magic* l'ha incontrato per forza.

Le icone di *The Gallerist*. Le ultime due icone sono riconosciute da uno studente, che ricorda il gioco ma non l'effetto: ***The Gallerist***[9]. La prima dà popolarità, la seconda soldi, in proporzione ai *meeple* presenti: un punto ogni due *meeple* di un colore (magenta, nel primo caso; marroni, nel secondo) oppure ogni *meeple* bianco. "Quindi, se ho quattro magenta e un bianco prendo tre; se ho otto magenta prendo quattro." Chi ha giocato a *The Gallerist* se le ricorda, perché sono "dappertutto in tutto il gioco". E sono icone che, quando te le spiegano la prima volta, "vi sparate"; "ma dopo il primo turno nessuno va a riguardare nel regolamento che cosa fanno".

Dai tre esempi escono tre strade diverse verso la memorabilità: l'**emozione** (*Uno*), la **pervasività** di un simbolo in tutto un sistema di gioco (*Magic*), la **ripetizione** dentro una singola partita (*The Gallerist*).

[9] *The Gallerist* (Vital Lacerda, 2015), gioco in cui si gestisce una galleria d'arte.

2.10.7 Soddisfazione

"Memorabili, immediate, efficienti: fantastico. Ma *bonus point* se è anche **soddisfacente**": se interagire con l'aspetto grafico, con la UX del gioco, è piacevole e gratificante. Tre esempi.

Il tracciato di *Parks*. In *Parks*[10] il tracciato su cui ci si sposta con i propri pedoncini è composto da **tessere**, e ci si muove in una sola direzione, da sinistra verso destra. Le tessere hanno una forma quasi a **freccia**, sono "un po' inclinate in modo che si incastrino", formando una specie di nastro che indica la direzione. "Potevano essere rettangoli? Sì, non cambia niente meccanicamente." Ma incastrarle in ordine, oltre a essere memorabile (perché ricorda la direzione), è anche "molto soddisfacente".

I quadri di *Canvas*. In *Canvas*[11] si compongono dei quadri con delle carte di **plastica semitrasparente**: ogni quadro è fatto di **tre strati** sovrapposti. Non solo gli elementi si combinano visivamente, ma anche il **nome del quadro** viene generato dalle carte giocate: alcune carte danno la prima parte del nome, altre la seconda.

Le armi di *Tiny Epic Mechs*. L'ultimo è un gioco di *mech* che combattono[12], in cui i *meeple* entrano dentro delle piccole **armature di plastica**; e quando si cambia equipaggiamento, si mettono loro in mano "le piccole armi e gli scudini". "Meccanicamente, non cambia niente": l'equipaggiamento è comunicato anche dalle carte. Ma è molto soddisfacente, e "ci ricordiamo al volo che cosa abbiamo in mano, perché fisicamente il nostro *meeple* ce l'ha in mano": un lanciagranate, uno scudo. E quando si cambia arma, si cambia non solo la carta, ma anche il pezzettino.

Approfondimento: la soddisfazione come "feedback" fisico. Schell, in *The Art of Game Design*, dedica all'interfaccia un intero capitolo e insiste su un punto che questi esempi rendono evidente: un'interfaccia non serve solo a trasmettere informazioni, ma a dare a chi gioca la sensazione di **controllo** e un **feedback** piacevole a ogni azione. Nel videogioco si parla di *juiciness*, la "succosità" delle reazioni alle azioni del giocatore; nel gioco analogico lo stesso ruolo è svolto dal gesto fisico: l'incastro di una tessera, la sovrapposizione di un foglio trasparente, l'arma infilata nella mano di una miniatura.

2.10.8 Quante informazioni porta una carta?

Dopo una breve verifica ("stiamo andando lisci"), Ruggero pone una "domanda da tenere conto sulla UX": **quante informazioni stiamo usando?**

Sulla slide ci sono due carte di due giochi di carte di fantascienza con un tema molto simile, la colonizzazione interplanetaria. Una è ancora di *Race for the Galaxy*: "pensavate che fosse brutta solo l'icona. E invece anche il *wireframe* della carta è molto brutto". È un peccato, perché il gioco "è bellissimo, incredibile"; Ruggero, che sembra fare *dissing*, lo possiede nell'edizione completa, "molto brutto da vedere, però glielo perdoniamo lo stesso, perché è molto bello da giocare". L'altra è di *Age of Galaxy*[13]. Quale delle due contiene più informazioni?

Tiziana dice la seconda. Altri, tra cui Lorenzo e Gianmarco, la prima. Lorenzo, che guarda anche le icone, pensa che abbiano le stesse informazioni; e un altro studente osserva che la prima sembra averne di più solo perché è *old school*: le schiaccia e le sposta sui bordi. È una domanda "molto divisiva", dice Ruggero, e lo è sempre: anche nelle lezioni passate, con classi più grandi, una parte risponde la prima e una parte la seconda. "Le grafiche parlano a persone diverse", e persone diverse percepiscono come più pesante l'una o l'altra. In termini di **funzionalità meccaniche**, però, le aree importanti sono le stesse.

[10] *Parks* (Henry Audubon, 2019), gioco di escursioni nei parchi nazionali statunitensi.
[11] *Canvas* (Jeff Chin e Andrew Nerger, 2021).
[12] La trascrizione dice "Tiny Pic Max": è *Tiny Epic Mechs* (Scott Almes, 2019).
[13] Esiste un gioco con questo titolo (*Age of Galaxy*), di tema e struttura compatibili con la descrizione.

2.10.8.1 L'icona capovolta

La carta di *Race for the Galaxy* ha poi delle scelte particolari. Di fianco al nome c'è un'icona con un numero (il punto 3 dell'analisi della slide), ed è **capovolta**. Perché? Le ipotesi della classe: è qualcosa che si disattiva; la carta va tenuta coperta; "devo girarmi la carta per sapere quanto do?" La risposta è un'altra: "è al contrario perché non è per noi, quell'icona. Serve agli avversari, serve ad altre persone dall'altra parte del tavolo. Loro devono sapere che cosa fa quell'icona, e quindi è girata verso di loro".

Ruggero ammette che si può "voler male" a *Race for the Galaxy* "in mille modi": per esempio perché riporta tutte le fasi del turno (1, 2, 3, $, 4, 5) di fianco alla carta, "inclusi quelli che non abbiamo e che non ci interessano in questa carta"; o perché nel riquadro di testo ripete esattamente la stessa informazione dell'icona, "ma via testo, rendendo quindi inutile l'iconografia". "Però in realtà, per l'utilizzo fisico della carta, *Race for the Galaxy* è molto intelligente." Le icone che sono per chi gioca la carta sono girate verso di lui, quelle per gli altri verso gli altri. E il **retro** della carta si può usare come materiale, come ha detto prima Gianmarco: le carte con il cerchio sono pianeti, e una carta a faccia in giù sotto un pianeta è una **risorsa** (arancione, in questo caso, perché il pianeta è arancione).

Uno studente obietta: se l'informazione del testo è ridondante, le informazioni della carta sono cinque, non sei. No, risponde Ruggero: il **4** non indica solo che cos'è il testo, ma anche la **fase** del turno in cui si attiva. È la stessa informazione divisa in due parti, e sulla slide lui l'ha numerata separatamente "per sanità mentale", perché due frecce sullo stesso punto erano illeggibili. Allo stesso modo, sulla carta di destra alcune voci sono accorpate (il punto 2), perché stanno nello stesso blocco, quello delle icone risorsa.

2.10.8.2 Una carta prima e dopo: il wireframe cambia

Il secondo esempio viene dal lavoro di Ruggero. È una carta di un gioco *Cyberpunk*[14] su cui ha lavorato, "nelle parti pre-andata in stampa": a destra la versione finale, a sinistra le versioni precedenti. "Ignorate il numero di pistole, che è stato ribilanciato": in termini di informazione, le carte sono rimaste praticamente uguali. Ma molte cose sono cambiate.

- Uno **0** è sparito: "tanto erano zero. Appesantiva senza motivo, e quindi è volato via. Non ci serve saperlo: basta che non mettiamo niente".
- La dicitura *starting* (carta iniziale) ha perso priorità, "giustamente", ed è stata spostata sotto il titolo. Non solo: le carte iniziali hanno il **nome in grigio**. Così, alla domanda "come faccio a sapere se è una carta iniziale, in mezzo al mazzo?", la risposta è: dal nome grigio invece che colorato.
- È stata introdotta un'**iconografia** per **ridurre il testo**: i testi delle prime versioni sono più lunghi di quelli delle ultime.
- Alcune cose hanno ricevuto un nome di *flavor* (Ruggero cita le *sticker condition*).

"La carta è la stessa." E questo è il tipo di ragionamento, e poi di test, che Ruggero si aspetta nella *pipeline* di prototipo dei gruppi: "le vostre icone, i vostri *wireframe* cambieranno in corso d'opera, le cose verranno spostate. È giusto che voi partiate con un'idea di come devono essere messe le cose". Ma con una raccomandazione netta: "**io non voglio vedere carte belle, voglio vedere carte funzionali** nei primi stadi". Il gruppo di lavoro di Ruggero ha testato con le prime carte "per mesi", e "erano letteralmente bianche, con due linee, due scritte e due iconcine". Non era un test di *look and feel*; di questo si riparlerà nella lezione sul *playtest*.

Approfondimento: ridurre il testo, non le informazioni. I tre interventi sulla carta *Cyberpunk* (togliere ciò che vale zero, spostare in secondo piano ciò che conta meno, sostituire testo ricorrente con icone) sono i tre strumenti classici della **gerarchia visiva**. Nessuno di essi toglie informazioni utili: cambiano l'ordine in cui l'occhio le incontra e lo sforzo necessario per leggerle. Un'icona che sostituisce una frase ricorrente funziona però solo se è *appresa*, e quindi va spiegata una volta e ripetuta in modo coerente; è lo stesso compromesso tra immediatezza ed efficienza visto con *Race for the Galaxy*.

[14] La trascrizione dice prima "Severpunk", poi "Cyberpunk", e "lo sapete già", come se il gioco fosse già stato mostrato nel corso. Il titolo esatto non è ricostruibile dalla trascrizione.

2.10.9 Accessibilità

"Quando pensiamo alla UX, non dobbiamo pensare solo e unicamente alla parte di funzionalità meccanica, ma anche all'**accessibilità**": quanto il gioco è, "come il nome fa intuire", accessibile.

Font per la dislessia. Il primo esempio della slide è un **font per persone con dislessia**. Ce ne sono molti, "anche gratuiti"; quello mostrato è gratuito per progetti non commerciali. Sono font che rendono più facile la lettura dei testi.

Il *curb cut effect*. Poi una domanda: qualcuno conosce il ***curb cut effect***[15]? Non è legato ai font né alla grafica, è "una cosa più a ombrello". Il *curb* è il **marciapiede**: quei marciapiedi che, in corrispondenza delle strisce pedonali, hanno "la piccola salitina". Perché ce l'hanno? Per permettere l'accesso alle persone in sedia a rotelle. Ma questo ha cambiato il marciapiede anche per tutti gli altri: per chi scarica i pacchi con il carrello, chi sale con la bicicletta, chi rientra in casa con un trolley, le persone più anziane, per cui un gradino potrebbe essere un pericolo. **Rendere una cosa più accessibile per qualcuno la rende più accessibile anche per tutte le altre persone.**

Il daltonismo. Il secondo strumento è un'app gratuita che Ruggero consiglia "caldamente" di avere sul cellulare (lui ce l'ha su tutti i telefoni di lavoro): **Chromatic Vision Simulator**. Si fotografa il proprio prototipo, e l'app mostra l'immagine con la visione regolare dei colori e con le tipologie più comuni di **daltonismo**, facendo vedere se due colori diventano simili. Nell'esempio della slide, con la tritanopia[16] il giallo e il rosa diventano molto simili, e in un'altra simulazione si confondono il verde e il rosso. "Scegliere i colori non è solo roba di *vibes*, *mood*": può rendere il gioco molto poco accessibile a chi è daltonico.

Ruggero porta la sua esperienza: nel suo gruppo di gioco abituale c'è un amico daltonico, "e ce ne accorgiamo subito se un gioco possiamo giocarlo insieme o non possiamo giocarlo". E chiude con un avvertimento: "non ha senso impedire a una persona daltonica di giocare un gioco da tavolo. Basta cambiare un colore, ragazzi, davvero. Non cadete in queste cose qua. Io li guarderò, i vostri giochi, con quest'app, quindi sappiatelo: usatela voi prima che la usi io su di voi".

Approfondimento: qualche numero e qualche regola pratica. Il daltonismo congenito riguarda circa l'8% degli uomini e lo 0,5% delle donne di origine europea, con forme più comuni nella confusione tra rosso e verde (deuteranopia e protanopia, con le loro forme attenuate) e più rare nella confusione tra blu e giallo (tritanopia). In un gruppo di quattro-cinque persone, la probabilità che ci sia almeno una persona daltonica non è trascurabile. La regola pratica più diffusa nel design dei giochi è non affidare mai un'informazione **solo** al colore: accompagnarlo con una forma, un simbolo o un motivo (le carte con semi diversi, non solo colori diversi). È, di nuovo, un *curb cut*: un simbolo in più aiuta anche chi gioca con una luce scarsa o chi guarda il tavolo dal lato opposto.

Con questo la prima parte si chiude. "Facciamo una pausetta": dieci minuti, e "alle sei ci ritroviamo qua" per il "blocco grosso" della lezione, con un invito a portare domande: "non nascondetele".

2.10.10 Dopo la pausa: il libro che avete consigliato

Al rientro nessuno ha domande, e Ruggero riassume il senso della prima parte: "vi è comunque utile ragionare sulle icone, su che cosa comunicano le icone in generale, sul *wireframe* delle carte", anche per presentare le proprie carte. Poi passa alla "parte succosa": il paragame.

Comincia con un esercizio, in due tempi. Nel primo, "non dovete rispondermi, dovete pensarci un attimo", trenta secondi. Bisogna pensare a un **libro**, con due vincoli: deve essere un libro di **narrativa**, "non un saggio", e nel formato più tradizionale ("non un fumetto, non una graphic novel, non un libro illustrato. Un classico romanzo"); e deve essere un libro che è **piaciuto**, idealmente uno che si è **consigliato a qualcuno**. "Non ditemi qual è."

[15] La trascrizione dice "curb effect". Il nome corrente è *curb cut effect*, "effetto del marciapiede ribassato".
[16] La trascrizione dice "tritanotapia".

Nel secondo tempo, invece, si risponde: **perché** l'avete consigliato? "Non vi sto interrogando, davvero: è utile per capire quello che verrà dopo." Le risposte:

- Uno studente lo consiglia di solito dentro una conversazione: "si sta parlando di qualcosa, e dico: ti consiglio questo, perché in questo punto c'è qualcosa di molto interessante", che dà "un valore aggiunto, una conoscenza in più". Ma pensava a *L'uomo che scambiò sua moglie per un cappello* di Oliver Sacks, che è un saggio, "è il confine"; Ruggero chiede di restare sulla fiction.

- Un altro (Lorenzo, a quanto si capisce) consiglia un libro per il **genere**: "il genere a me piace", e quello è un libro fatto bene per quel genere. Fatto bene vuol dire: la **trama** è scritta bene, "ha un buon nesso", i **personaggi** sono buoni, e si vede che c'è della **cura** nella scrittura, "la scelta delle parole, come sono formulate le frasi". Poi le **tematiche**, "emozioni particolarmente scritte bene, gestite bene".

- Un'altra lo consiglia "per ciò che mi ha suscitato": le **emozioni** provate leggendolo, "belle emozioni che può provare anche la persona a cui lo consiglio". Ruggero rilancia: "magari sai che a un tuo amico piacciono i libri che fanno piangere tantissimo, tu hai pianto tantissimo con un libro, e dici: sai che qua butti fuori un fiume".

Poi la domanda vera: "quel libro lo avreste apprezzato anche scritto in **Comic Sans** a grandezza 60?" Uno studente rivela il suo: *Il club dei delitti del giovedì*[17]. In Comic Sans a corpo 60 "vengono trenta miliardi di pagine", ma forse "ci starebbe meglio di *Dracula*, come *vibes*"; magari per un libro per bambini. E in **Times New Roman a corpo 6**? "Non ci leggi, a corpo 6"; "però ne sono andati in stampa, così", e almeno "si sprecano pochi soldi, perché sono pochissime pagine". E se fosse stampato su **pagine nere, o con fondo rosso gotico**? "Per *Dracula* magari va anche bene"; "terribile, ci lasci gli occhi"; "il rosso sul nero fa malissimo".

È a questo punto che una studentessa chiede chiarimenti su che cosa sia il paragame, e un'altra persona fa notare a Ruggero che le slide non si vedono: "non stai condividendo". "Ragazzi, sono pazzo." Ruggero ricondivide e ripercorre in un attimo le slide che aveva commentato a voce (il libro, il Comic Sans gigantesco, il font piccolino, il nero su rosso), scusandosi anche con chi vedrà la registrazione: per fortuna le sue slide "non sono muri di testo, sono letteralmente a supporto di quello che sto dicendo".

Il senso dell'esercizio, intanto, è chiaro. "Quando noi consigliamo un libro, è difficile che lo consigliamo per come il libro è stampato." Anzi, si dice proprio "non giudicare un libro dalla copertina". Eppure la copertina, il font, la dimensione del font, il numero di pagine, il **formato** (tascabile o no, un mattone, un A4 "ingestibile"), la copertina rigida o morbida *cambiano l'esperienza di lettura*. Ruggero aggiunge altri casi.

- Se le pagine fossero **talmente sottili da risultare quasi trasparenti**? Qualche decennio fa è uscita un'edizione "mammut" di tutti i racconti di **Lovecraft** in un unico volume di circa 1100 pagine, che per stare in un solo libro aveva le pagine "praticamente trasparenti": "impossibile leggerlo alla luce del sole".

- Se le pagine fossero di **plastica**? "Le posso leggere al mare, o nel bagno." Per un libro per bambini, ottimo; per *Dracula*, forse no.

- E l'**ambiente** in cui si legge: in spiaggia, in treno, di notte sotto le coperte. "Il *mood* dell'ambiente in cui stiamo, mentre stiamo leggendo."

2.10.11 Che cos'è il paragame

"Il **paragame** è un'intera area di design che **non è fiction**, **non è regole**, ma va a impattare l'esperienza", e a volte si incrocia con quelle altre due aree. Un font particolare, magari "che va sul fantasy medievale", può aiutare a trasmettere la **fiction**, il *world building*, il *setting*. Ruggero lo spiega con l'analogia da cui nasce il termine: "questo è il **paratesto**. Il paratesto nei libri, il paragioco nei giochi".

Quando la studentessa chiede di nuovo se il paragame sia "la modalità con cui il gioco è realizzato fisicamente", Ruggero precisa: è "un termine che è l'estensione di paratesto". Con paratesto, in letteratura,

[17] La trascrizione dice "Il Club dei Detti del Giovedì". È il romanzo giallo di Richard Osman (2020).

si intendono "tutte quelle parti di un libro che non sono il suo contenuto, la sua fiction, ma sono tutte quelle parti di forma, di formato": il font, la dimensione e il colore del testo, ma anche il **materiale**, lo spessore delle pagine, il formato del libro, la copertina rigida o morbida ("magari di quella morbida, con quell'effetto *matte* e vellutato che amo tantissimo"). "Sono tutti gli aspetti paratestuali, che sono di contorno." Il paragame è la stessa cosa per i giochi: "gli aspetti di paragame sono tutti quelli che ricoprono gli aspetti di un gioco che **non sono le regole e non sono la fiction**".

2.10.11.1 I bordi sono zone grigie

Il paragame "può sforare, chiaramente, perché i bordi non sono dei muri, sono delle zone grigie, non così definite" (figura 2.10.2). L'esempio "più diretto" sono i **librogame**. Qualcuno ne ha giocati? Sì: il più famoso, forse, *Lupo Solitario*[18]. Come paragame, i librogame sono libri: hanno una copertina, sono fatti "come dei manuali, tendenzialmente". Ma il loro **formato** va anche in base alle **regole**, perché i paragrafi numerati sono le opzioni tra cui si sceglie. "Non è un formato separato dalle regole, è un formato che va a intrecciarsi"; e il font scelto, le illustrazioni al loro interno, si intrecciano invece con la fiction.

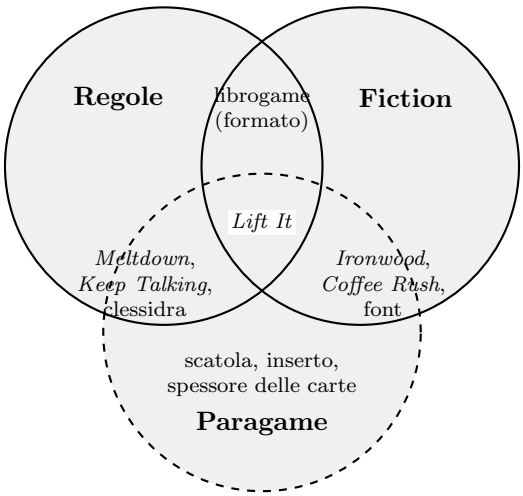


Figure 2.10.2: Regole, fiction e paragame come tre aree con i bordi sfumati. Il paragame è ciò che non è né regole né fiction, ma si sovrappone spesso a entrambe. La posizione degli esempi riprende i commenti di Ruggero durante la lezione: *Lift It* "è al centro", *Ironwood* aiuta soprattutto la fiction, *Meltdown* e *Keep Talking and Nobody Explodes* soprattutto le meccaniche.

Approfondimento: Genette e le "soglie" del testo. Il termine *paratesto* viene dal critico francese Gérard Genette, che lo ha sistematizzato in *Soglie* (*Seuils*, 1987). Per Genette il paratesto è ciò che fa di un testo un libro e lo presenta al lettore: il titolo, il nome dell'autore, la copertina, le dediche, le prefazioni, le note, ma anche il formato, la collana, la tipografia. È una "soglia" tra l'interno e l'esterno del testo, una zona di transizione che orienta la lettura. L'idea di applicare lo stesso concetto ai giochi (paragame, paragioco) è diffusa nei *game studies*, dove il termine *paratext* viene usato anche per tutto ciò che circonda un gioco (*trailer*, recensioni, guide, comunità). Nella lezione di Ruggero il senso è più vicino a quello originale di Genette: le caratteristiche materiali e formali dell'oggetto-gioco.

2.10.12 Le aree del paragame

"Su quali aspetti del paragame possiamo lavorare? E qui abbiamo l'ondata di robe. Andiamo con calma: sono tante zone, ma sono sicuro che le vediamo tutte." La tabella 2.10.2 le riassume; le sezioni seguenti le vedono una per una.

[18] La trascrizione dice "un po' solitario". *Lupo Solitario* (*Lone Wolf*, Joe Dever) è la serie di librogame più nota in Italia.

Area	**Domande da farsi**	**Esempi della lezione**
Materialità	di che cosa sono fatti i componenti? Serve un'app? Quanto sono spesse le carte? Clessidra o timer?	dadi di legno e di plastica; *Lands of Galzyr* su tablet e telefono; carte *blue core*; *standee*
Formato	come sono comunicati regole e testi? Che font, che dimensione, che misura della carta?	*Bureaucracy* di *Magic*; il *200 Word RPG*
Regolamento	quante pagine? Quanti libretti? Video, glossario, *playthrough*, *play aid*?	*Irish Gauge*; *Root*; riassunto delle icone
Scatola	dove andrà, nei negozi e a casa?	le vecchie scatole lunghe; la Kallax; *Dobro*
Grafica e illustrazioni	che cosa comunica l'aspetto visivo? Quante icone?	*Master of Rules*; *Rolling Stock Stars*; *A Feast for Odin*
Interfaccia analogica	com'è il *wireframe*? Dove stanno le informazioni quando il componente è in uso?	la carta di *Magic*; la plancia di *The Gallerist*; le carte a ventaglio e i mancini
Ambiente	dove si gioca? Quanto spazio serve? Che suoni, che vibrazioni?	il tavolo IKEA; il treno e l'aereo; *Jenga*
Audio	serve una colonna sonora?	*Escape: The Curse of the Temple*
Contesto	che cosa deve sapere chi gioca sul luogo e sulla cultura del gioco?	*Jakarta Traffic*; *This War of Mine*
Lingua	il gioco dipende dalla lingua?	i giochi di Lacerda; *Race for the Galaxy*; *Magic*
Spazio urbano e ibridi	la città, la stanza, il telefono come parte del gioco	*Pac-Manhattan*; *Vroom*; *World of Yoho*

Table 2.10.2: Le aree progettabili del paragame presentate a lezione, con le domande che suggeriscono e gli esempi usati da Ruggero. Inserti e *setup*, discussi alla fine (sezione 2.10.25.5), sono un'altra area della stessa famiglia.

2.10.13 Materialità

La prima area è la **materialità**: "le proprietà fisiche degli elementi di gioco". Legno, plastica, metallo. Prendiamo dei dadi classici di legno e dei dadi di plastica: che cosa cambia? "Il rimbalzo, il rumore, la sensazione al tatto." Sono aspetti **cinestetici**, "che assolutamente impattano". Ma cambia anche altro, "perché nel paragame spesso ci sono anche degli **aspetti culturali**". In un Eurogame, in un *German*, si troveranno dadi di legno; in un *American*, in un *dungeon crawler*, dadi di plastica. (Se n'era parlato, "vagamente", nella lezione sui casi studio, a proposito dei componenti.)

2.10.13.1 Il digitale come materiale

Anche il **digitale** fa parte della materialità: se il gioco ha o richiede un'app, un cellulare, un tablet, "quello è un aspetto di materialità da considerare". Ruggero porta un esempio di un gioco che ha giocato, che la trascrizione chiama "Lens of Galsy"[19]: "molto carino, ma chiaramente pensato per un tablet". L'app ha tanto testo e tante cose a schermo, e sul cellulare, molto più piccolo, "non si vede niente". "L'app è la stessa, ma in base a dove lo stiamo giocando, quel gioco può diventare un incubo di accessibilità, o qualcosa invece molto più comodo da giocare."

2.10.13.2 Le carte non sono tutte uguali

Quanto sono spesse le carte? Di che materiale sono? *Blue core*, *black core*, *white core*? "Le carte non sono tutte uguali. Vi sembrano uguali perché le usate da chi gioca, ma da produrre sono diverse." Quelle "che si spaccano al volo" si riconoscono subito, perché sono le più economiche: di solito quelle di giochi come *Uno* o di giochi di carte "più cheap da produrre", in cui basta una pieghetta per rovinare una carta "forever and ever". Altri giochi usano il *blue core* o il *black core*; le carte di *Magic*, per esempio, hanno il *blue core*.

Ruggero semplifica la struttura: di solito una carta da gioco ha **tre strati**, lo strato con l'immagine sul fronte, quello con l'immagine sul retro, e in mezzo il materiale che ne determina, tendenzialmente, la **rigidità**. "Poi è più complessa di così", perché ci possono essere filigrane, texture e altro; ma la parte centrale dice che cosa succede quando la carta si piega, o quando la si mescola male.

Le carte americane. Una studentessa chiede delle carte da gioco americane, "quelle di tela", come le *Bicycle*, che in Europa si vedono poco: c'è un motivo per cui sono "molto più fighe"? Sì: hanno una produzione che include **materiali plastici** nella loro texture, che le rende più resistenti e crea un **attrito** diverso. Sono carte che "scivolano molto bene" ("non ho più italiano, scusatemi"), e infatti sono usatissime negli spettacoli di prestigiazione, perché si possono manovrare velocemente, "per chi le sa maneggiare; io no, di sicuro". Le carte prodotte in Europa, invece, hanno di solito un attrito molto più alto, e sono "non fantastiche" da usare in modo "sgaggio".

Europa, America, Cina. Lo stesso vale per i dadi: l'America fa un uso abbondante di plastica nei giochi perché spesso produce in **Cina**, mentre tipografie, stamperie e aziende che producono carte in Europa hanno "molto poco accesso" a quei materiali. "Quindi spesso siamo più forti nel legno e nella carta che nella plastica." È "un vaso di Pandora che a voi interessa il giusto": per i progetti del corso, quel tipo di finiture "ve lo sognate, salvo pagare un bordello". Ci sono alcune fabbriche che possono fare cose bizzarre, nell'Est Europa e in Repubblica Ceca, ma la maggior parte di chi produce stampa in Cina, oppure non usa quei materiali.

Miniature e *standee*. Stesso discorso per le miniature o gli ***standee***, il cartoncino infilato in un piedistallo al posto della miniatura di plastica. In un *American* è facile trovare la plastica; in un gioco prodotto e stampato in Europa, "non trovate delle miniature di plastica", perché non c'è praticamente nessuna azienda europea di giochi da tavolo che possa stampare in Europa la plastica a iniezione usata per le miniature; anche chi produce in Europa giochi con parti di plastica le fa stampare in Cina. Per questo molti giochi europei degli anni '90 e 2000, *German* ed Eurogame, "la plastica proprio la skippano": se hanno delle figure, sono *standee*.

[19] Con ogni probabilità *Lands of Galzyr* (Snowbright Studio, 2021), gioco d'avventura narrativo con app.

Approfondimento: che cosa fa davvero il "core". Nella produzione di carte da gioco lo strato centrale (*core*) ha due funzioni: dà rigidità e soprattutto rende la carta **opaca**, impedendo di vedere il fronte in controluce, cosa essenziale in qualunque gioco con informazioni nascoste. *Blue core* e *black core* sono nomi commerciali di cartoncini a più strati con un'anima colorata scura. Sulle carte americane del tipo *Bicycle*, il cartoncino è in genere rivestito da una finitura goffrata e laccata (la cosiddetta *air-cushion finish*) che riduce l'attrito; le carte *interamente* di plastica (PVC o acetato) esistono, ma sono un prodotto diverso e più costoso.

2.10.13.3 Clessidra o timer?

"Occhio alla materialità, perché la materialità non va a impattare solo il paragame e basta." Il classico esempio di materialità che impatta anche le **regole** è la **clessidra**. Che differenza c'è tra un timer sul cellulare e una clessidra? Uno studente risponde: "la tangibilità". Sì, e soprattutto quella che si chiama di solito l'**informazione discreta**: con il timer si sa con precisione quando mancano un secondo, due, tre, quattro secondi. Con la clessidra no: si vede chiaramente quando è finita, ma ha "quel pizzichino di incertezza che crea una tensione molto precisa che il timer non ha". Entrambi creano tensione, adrenalina, frenesia, ma in modo diverso; e sapere quale dei due serve al proprio gioco "non è così scontato" (tabella 2.10.3).

	Timer (cellulare)	**Clessidra**
Informazione	discreta e precisa: si sa quanti secondi mancano	continua e incerta: si vede la sabbia scendere, ma non si sa con precisione quanto manca
Tensione	c'è, ed è "contata"	c'è, con "un pizzichino di incertezza" in più
Fiction	neutra, a meno di suoni scelti apposta	può essere diegetica (vedi il fiammifero di *Scandal in the Deep*, sezione 2.10.24)
Domanda da farsi	perché un timer e non una clessidra?	perché una clessidra e non un timer?

Table 2.10.3: Timer e clessidra come scelta di materialità che incide sulle regole e sulla tensione di gioco.

Qui Ruggero fa un annuncio che vale per il resto del corso: "io ve lo ricordo, ve l'avevo già accennato, ve lo ripeto: quando vedremo i *pitch*, ma anche i vostri progetti più avanti, nelle ultime lezioni, **io vi chiederò tantissimi perché**". Perché avete scelto una cosa, perché l'avete fatta, perché l'avete decisa. "Se avete una clessidra e non un timer, io vi chiederò perché avete una clessidra e non un timer. Perché avete la plastica, la carta, e così via. Voi dovete dirmi il perché delle cose che fate."

2.10.14 Formato

Il **formato** è il modo in cui sono comunicati, per esempio, regole e testi: frasi o icone? Che font? Quanto è alto, che dimensione ha? E la carta, com'è fatta, quanto è grande?

Bureaucracy. Il primo esempio è una carta di *Magic* "volutamente fatta male": *Bureaucracy*, da una di quelle serie *easter egg* (umoristiche) che *Magic* pubblica ogni tanto[20]. Per il testo è stato scelto un font piccolissimo ("mi sembra che fosse un corpo 5, o addirittura 6"), per renderlo "un muro di testo oppressivo": tematicamente coerente con il nome della carta.

La misura delle carte di *Magic*. Qualcuno sa quanto sono grandi le carte di *Magic*? "Una dimensione da poker." In millimetri sono "cifre bizzarre", perché in realtà la misura è di **3,5 per 2,5 pollici**[21], "in un

[20] La trascrizione aggiunge un dettaglio sul simbolo dell'espansione che non è ricostruibile ("ha l'uomo rotto suonate come simbolo"). *Bureaucracy* appartiene alle espansioni parodiche di *Magic*, le cosiddette *Un-set*.

[21] La trascrizione riporta cifre storpiate ("88,4 per 69,5"). 3,5 per 2,5 pollici sono circa 88,9 per 63,5 millimetri; la misura nominale indicata di solito per le carte di *Magic* è 63 per 88 millimetri.

sistema stupido americano di misurazione" ("ormai l'ho detto, è troppo tardi"). E in quella dimensione non c'è tutto lo spazio che si vuole, perché una parte è dedicata all'**illustrazione**: lo spazio per il testo, con il font e la dimensione scelti, è ancora più ridotto. Questo non impatta solo il formato, ma anche le **regole**: "una regola complessa sarà difficile da scrivere in quello spazio", e quindi il formato costringe a scegliere regole che ci stiano dentro.

Il *200 Word RPG*. Un caso estremo: il *200 Word RPG*, una sfida in cui si scrivono giochi di ruolo completi in **200 parole**. Il paragame è il formato stesso delle 200 parole: "ditemi voi se non vi impatta le regole, quel formato".

"Non prendete questa cosa qua come del contorno, come una cosa che è un di più. Questa roba qui vi va a impattare come voi create i giochi, e se lo sapete, potete lavorarci attorno."

2.10.15 Regolamento

Il **regolamento** è un'area a sé. Quante pagine ha? Una o quaranta? Che cosa cambia? "Se io vi dico: venite, c'è un party game, e tiro fuori un manuale da 60 pagine, che cosa succede?" Succede che nessuno lo gioca.

***Irish Gauge*: un caso positivo.** All'opposto, un caso "incredibile": *Irish Gauge*[22], un gioco economico ferroviario "bellissimo", di circa un'ora di partita, in cui si comprano azioni e si costruiscono ferrovie, con una complessità significativa sul tavolo, "un gioco comunque bello intenso". E il regolamento è **un singolo foglio di carta** "che lascia zero dubbi dopo che lo si è letto". "Bellissimo, irraggiungibile forse." È una sorpresa positiva; il caso contrario (il party game da 60 pagine) è una sorpresa negativa che uccide il gioco.

QR code e video. Chi gioca spesso avrà notato che sempre più regolamenti hanno un **QR code** o un link a un video in cui le regole vengono spiegate. Perché? "Non sappiamo più leggere" è "la versione più nichilista". La risposta "più banale e meno nichilista" è che **le persone imparano in modi diversi**: un regolamento scritto in un certo modo può andare benone per una persona ed essere un disastro per un'altra.

Più libretti per lo stesso gioco. Alcuni giochi hanno più di un libretto di regole. *Root*, per esempio, ha la **guida rapida** (il *learn to play*), scritta come un regolamento da leggere al tavolo, e la **Legge di Root**, scritta come un **manuale tecnico**: paragrafi 1.1.1, 1.1.2, 1.1.3, 1.2.1 e così via, con glossario. "Il gioco è lo stesso. Lo si impara a giocare in entrambi i modi, ma parlano a due persone completamente diverse."

I costi. Ogni scelta ha un costo. Un regolamento in più costa la produzione di un regolamento in più, **a ogni copia** stampata. Il video è un costo diverso: lo si paga una volta, all'inizio, lo si mette online "e quel video lì è lì". "Se tu stampi 100 copie o ne stampi 10.000, il video non ti costa di più. Il regolamento sì."

Glossario, *playthrough*, *play aid*. Altre domande: c'è un **glossario**? Un *playthrough*? Un *play aid*? E le icone, dove sono riassunte? Dove si trova di solito il riassunto delle icone? Due risposte, "entrambe vere". Tancredi: **nell'ultima pagina del regolamento**, usata spesso proprio per questo, perché si può tenere il regolamento girato sul tavolo e avere il riassunto davanti. Tiziana: in un ***play aid***, un cartoncino, magari uno a persona, con il riassunto delle icone. Quando l'uno e quando l'altro? Dipende da **quante icone** ha il gioco. In *Race for the Galaxy* un riassunto a testa non esiste, "perché il riassunto delle icone di *Race for the Galaxy* è così" (e Ruggero mostra quanto è lungo). In giochi con cinque, sei, una decina di icone è facile avere una carta personale che le riassume, magari con il **flusso del turno** sul retro.

[22] La trascrizione dice "Irish Gouge" e attribuisce il gioco a "Amable Holland", più avanti "Anne Blowland". È *Irish Gauge*, gioco ferroviario di Amabel Holland.

Il *playthrough* e l'*actual play*. Più avanti, su richiesta di uno studente ("Ruggero, non so se l'hai detto: il *playthrough*?"), Ruggero torna sul punto. Il ***playthrough*** è **una giocata descritta**. Molti giochi lo fanno spezzandolo in esempi: i classici esempi in cui "Tizio e Caio fanno A, B, C, poi Tizio fa B e Caio piange perché ha perso la partita". Questi esempi non sono solo un'applicazione delle regole: sono una descrizione di **che cosa fanno le persone al tavolo**. Alcuni giochi, soprattutto di ruolo, includono il cosiddetto ***actual play***: un'intera partita descritta, alternando le parti di *roleplay* e quelle meccaniche ("Tizio tira e fa 18, quindi apre questa cosa; poi Caio dice questa cosa con il suo personaggio"). Dove si trovano[23]? Sono "comunissimi nei giochi di ruolo": molti giochi di ruolo contemporanei hanno esempi di *actual play*, di solito in fondo al manuale. Gli esempi sparsi nei regolamenti sono "un'estrazione del *playthrough*", scritti con la stessa idea; il *playthrough* vero e proprio è l'esempio di partita completa. Nei giochi da tavolo più commerciali è raro, "perché le pagine dei regolamenti costano", e una partita completa prende molto spazio. Rientra quindi soprattutto nei giochi di ruolo, che hanno già manuali da cento pagine e passa e non hanno il problema di stare sotto le 16 pagine, "perché sennò la gente si spaventa" davanti a un regolamento da 20.

2.10.16 La scatola

Nel formato rientra anche la **scatola**. Ruggero è convinto di averne già parlato ("non abbiamo mai parlato delle scatole? Secondo me no") e la affronta con una domanda: come erano fatte le scatole del vecchio *Risiko!* o del vecchio *Monopoly*? "Basse e lunghe." Perché? "Ci stava il tabellone"; "si impilavano". La domanda, precisa Ruggero, non è che cosa c'era dentro, ma **dove doveva andare** la scatola: nei negozi di giocattoli e, a casa, dove venivano messi i giochi trenta, quarant'anni fa? Non nella libreria: **negli armadi**. "Non era una questione estetica": l'armadio è stretto e lungo, e la scatola doveva entrare lì.

E oggi, dove vanno messi? In libreria. Quale libreria? La **Kallax**[24] dell'IKEA, "la libreria più venduta del mondo". Com'è fatta? "**A cubi**." La scatola del vecchio *Monopoly* lì dentro non entra ("puoi metterla in cima, certo"). Ma se oggi un editore vuole che il suo gioco entri "nella libreria più venduta del mondo", il gioco deve entrare in un cubo. "È tutto qui. **Una scelta dell'IKEA ha deciso che le nostre scatole ora sono quadrate**." L'IKEA non ha convinto gli editori di niente: "è una roba che viene naturale, perché tutta l'umanità è intrecciata. Le decisioni che prende qualcuno vanno a impattare qualcun altro". Chi vuole che il proprio gioco sia in vista in libreria deve fare i conti con il cubo.

Ruggero si concede qui una raccomandazione generale: "se c'è una cosa che voglio che vi portiate via da questo corso, una cosa" ("poi, se fate anche giochi dopo, belli, benissimo"), "la cosa migliore per me, perché sono contento, ho fatto il mio lavoro, posso morire in pace", è questa: **ragionate quando fate le cose**. "Potete ragionare sulle cose quanto volete. Più ci ragionate, più avete aree operative su cui fare le cose."

Approfondimento: il "formato Kallax". Il vano interno di una Kallax misura circa 33 per 33 centimetri, per una profondità di poco inferiore ai 39. Non a caso un gran numero di giochi da tavolo contemporanei di formato "standard" ha una scatola quadrata di circa 30 centimetri di lato: è il formato che sta in un vano in piedi o coricato, e che si impila bene. Molti editori lo indicano ormai esplicitamente tra i vincoli di progettazione della scatola, insieme al peso e ai costi di spedizione.

2.10.17 Grafica e illustrazioni

"**Graphic design e illustrazioni sono due cose diverse**. Non confondete le cose." Le **illustrazioni** sono tendenzialmente l'*art*: le immagini, gli sfondi, i personaggi, i luoghi, la parte di *world building*. Il **graphic design** sono le icone, la UX. "Non fate fare queste due cose alla stessa persona, sennò di solito viene una porcheria."

[23] La trascrizione attribuisce la domanda a qualcuno che si rivolge a "Pietro"; con ogni probabilità è un errore di trascrizione per "Ruggero".

[24] La trascrizione dice "Callax" e, a tratti, "Galaxy".

2.10.17.1 Due edizioni, due destini

Sulla slide ci sono due scatole: una con un omino che tiene sopra di sé delle forme, e una con un'auto sotto un cielo stellato, una notte al neon. "Sono lo stesso gioco": *Master of Rules*[25], "un gioco bellissimo", che nella sua edizione americana è uscito con la prima scatola. "Vi sorprenderà sapere che in America non è andato per niente, nonostante sia un gioco incredibile. Qualcuno ha un'idea del perché?" Le risposte sono impietose: "fa schifo", "sembra quelli che trovi in edicola". E dentro, dice Ruggero, "è ancora più brutto".

Uno studente lo provoca: "io ho dei giochi che hanno copertine orrende da 100 euro": gli **Splotter**[26]. Ruggero non lo nega, ma ribalta la domanda: la grafica degli Splotter "è comprensibile perché la fanno così, ovvero perché non gliene frega niente"; ma "sarebbero giochi peggiori con una grafica migliore?" "La risposta, nel tuo cuore, la sai: no." Le immagini danno un'esperienza, un *look and feel*.

Tiziana porta un caso concreto: un amico ha comprato *Darwin's Journey*[27], "un gioco che secondo me è bellissimo", e l'ha rivenduto, perché l'impatto grafico per lui è stato terribile: "colori scuri sul marrone". "Non riusciva a vederlo."

Ruggero avrebbe potuto fare "mille altri esempi" di giochi usciti in due edizioni con grafiche diverse, "una che va a picco e non la vedete più, e l'altra che invece diventa roba di culto". Perché la grafica ha un impatto "non solamente in termini di leggibilità, ma anche di *vibes*". Lui, personalmente, non ci fa caso: "io mi gioco anche *Rolling Stock Stars*", che "sembra fatto da marito e moglie su Excel". Ma altre persone vedono quella grafica e dicono "non giochiamo a *Rolling Stock Stars*". E finisce che "il piccolo Ruggero" lo gioca "una volta ogni cinque anni, perché nessuno vuole giocarci". La conclusione: "un gioco può beneficiare dall'essere **non visivamente ostile** alle persone che lo vogliono giocare". Un altro studente aggiunge, a proposito di grafiche difficili, *Terraforming Mars* ("la carta con il cane"), e Ruggero si arrende: "volete farmi stare male oggi".

2.10.17.2 Il font, e quante icone

Anche il **font** rientra nella grafica, non solo nel formato: il font del titolo, il font dei contenuti. Ruggero cita un gioco che ha venduto "proprio per il font che aveva", e promette di mostrarlo dopo, perché "ci vuole del tempo per elaborare" (lo farà nella discussione finale: è *Al Rashid*, sezione 2.10.25.2).

Ultimo punto della grafica: **quante e quali icone** avete? Sulla slide c'è *A Feast for Odin*[28]: "quel gioco lì ha 50 azioni diverse. Non devo dirvi altro". Ragionate sulle icone che avete.

L'intervento di Tiziana. Tiziana porta un esempio di segno opposto: ha giocato di recente a un gioco che la trascrizione chiama "MNFR"[29], abbastanza complesso, con varie tipologie di azioni e una serie di cose che si succedono; ma le icone "sono sempre quelle", in numero limitato, e una volta capite "te le ritrovi in tutte le salse, nelle varie parti e sezioni del gioco". "E questo aiuta molto." Ruggero concorda e cita di nuovo *The Gallerist* ("bellissimo, un capolavoro"): sono giochi in cui, se si fa un buon lavoro sulle icone, il gioco "non ti chiederà mai di andare a leggere il regolamento durante la partita". Lui ha giocato di recente un Eurogame di designer giapponesi[30], con un *setting* un po' fantasy, in cui si costruiscono edifici su un'isola e ci si muove di porto in porto per vendere risorse: "ha tante icone, ma dopo i primi cinque minuti, per tutte le altre due ore della partita, non abbiamo guardato il regolamento una singola volta, perché è tutto chiaro, tutto immediato". Si torna ai pilastri: immediatezza, efficienza, memorabilità.

2.10.18 Interfaccia analogica

La voce successiva è l'**interfaccia analogica**: "si parla di *wireframing*, di solito". È la struttura nuda della carta, quelle linee che, in stampa, "verranno abbellite da frame, cornici". Ma il *wireframe* è quello. "Se vi chiedo di disegnarmi a occhi chiusi su un foglio una carta di *Magic*, voi sapete come disegnarmela": divisa

[25] Il titolo è quello della trascrizione e non è stato possibile verificarlo.
[26] Splotter Spellen, editore olandese noto per giochi molto pesanti e per una grafica spartana.
[27] La trascrizione dice "Darvis Journey".
[28] La trascrizione dice "Fist for Odin". *A Feast for Odin* (Uwe Rosenberg, 2016).
[29] Il titolo non è ricostruibile dalla trascrizione.
[30] La trascrizione dice "Elder Tide degli stessi di Gisoji"; titolo e autori non sono ricostruibili con sicurezza.

a metà, al centro un rettangolino molto sottile, un rettangolo più grosso in alto, uno più grosso in basso. "Quella parte lì è una parte su cui potete ragionare."

2.10.18.1 La scatola in piedi: *Dobro*

Anche la grafica della scatola è interfaccia. "Se io prendo la scatola e la metto in verticale, posso leggere il titolo?" C'è una "diatriba" che Ruggero non apre ("siete ancora persone sane di mente, non voglio rovinarvi la vita"): i giochi da tavolo si mettono in libreria in verticale o in orizzontale? Lui fa un mix, ma molte persone li mettono in verticale, e molte scatole non hanno il nome del gioco su tutti i lati: alcune lo hanno solo per il lungo, e in verticale il nome è storto. L'esempio positivo è *Dobro*, di cui il suo gruppo ha rifatto la scatola per l'edizione italiana: un lato lungo con il nome in orizzontale, il lato opposto con il nome in verticale, e lo stesso sui due lati corti. "Non importa come la mettete: un modo per trovarla con il nome che si vede da fuori lo trovate sicuramente." Mentre "ho delle scatole in cui non si vede il nome, perché purtroppo non l'hanno messo su tutti i lati". Quindi: **la grafica della scatola non va lasciata al caso**, "perché è funzionale". ("Mi sto arrabbiando, si vede, perché questa cosa mi fomenta. Purtroppo per voi.")

2.10.18.2 Le opere sulla plancia di *The Gallerist*

Una studentessa chiede di chiarire: l'interfaccia analogica è il discorso della scatola e della visibilità, o altro? Ruggero torna al *wireframe*, "cioè a come sono fatte le carte", e porta l'esempio di *The Gallerist*: si acquistano opere d'arte da esporre nella propria galleria, e la **plancia** personale raffigura la galleria. Le tre tessere delle opere esposte si sovrappongono perfettamente ai **muri disegnati sulla plancia**, "andando letteralmente a mettere le opere in vista nella nostra galleria". Non è casuale: hanno pensato la grafica in modo che aiutasse a capire che cos'è quel componente, e in modo che fosse anche soddisfacente. Di nuovo: soddisfazione, efficienza, memorabilità, immediatezza.

2.10.18.3 Dove stanno le informazioni quando le carte sono in mano

Un altro aspetto dell'interfaccia è **dove si piazzano le informazioni**. Con le carte da poker in mano, dove sono le icone? **In alto a sinistra**: quando le carte sono aperte a ventaglio, ciascuna copre buona parte di quella precedente, e l'angolo in alto a sinistra è "la cosa più visibile" (figura 2.10.3).

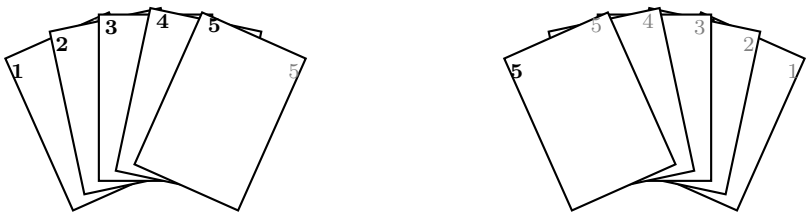
Tancredi apre una parentesi sui **mancini**: "io da mancino, tutte le volte che apro le carte da poker, non vedo assolutamente niente". Ruggero conferma ("ho degli amici mancini, e se avete un gioco che non pensa ai mancini lo sapete al volo, perché ve lo dicono"), e lo riconduce all'**accessibilità**: i mancini tengono in mano le carte come tutti, "è solo che per natura il ventaglio lo fanno nell'altro verso". Molti giochi per questo **duplicano** il seme della carta, mettendolo anche in alto a destra. In alcuni giochi è più complicato: quando l'angolo in alto a sinistra ospita icone meccaniche, come il costo, come in *Race for the Galaxy* ("è il problema minore di questa carta, mi sento di dire"), duplicare vuol dire affollare ancora di più una carta già piena. "Volete davvero duplicare le informazioni di questa carta dall'altro lato? Io non lo farei." Non sempre si può; ma "è assolutamente una considerazione da fare".

Ruggero annuncia che su grafica e formato c'è "un'altra cosa", da riprendere dopo (lo farà con gli inserti, sezione 2.10.25.5).

2.10.19 Ambiente

Dove giocheranno le persone al vostro gioco? Su un tavolo, a letto, per terra, in piedi? E **quanto spazio** prende il gioco? Qualcuno sa quanto spazio, al massimo, deve prendere un gioco da tavolo *family* per quattro persone? "La sapete nel vostro cuore, la risposta." Gianmarco: "**il tavolo IKEA**". "Bravissimo, Gianmarco, è sempre quello." Il tavolo IKEA più venduto: cercate quanti centimetri misura. "Se il vostro gioco non entra in quel tavolo, avete un grosso problema."

E se è un **party game**? Allora deve stare nel **tavolino da caffè** dell'IKEA, più piccolo, quello da divano, "per un altro tipo di ambiente". Se è un gioco *pocket*, portatile, deve stare nei tavolini da caffè, in quelli del **treno**, in quelli dell'**aereo**. "Quanto spazio prende il vostro gioco apparecchiato non è casuale. Non lasciate



ventaglio "destrorso"
restano visibili gli angoli in alto a **sinistra**

ventaglio "mancino"
restano visibili gli angoli in alto a **destra**: serve l'indice duplicato (in grigio)

Figure 2.10.3: Le carte a ventaglio. L'informazione più importante va nell'angolo che resta scoperto; chi apre il ventaglio nell'altro verso, come molti mancini, vede l'angolo opposto, e per questo molti giochi duplicano il seme o l'indice in alto a destra. Schema illustrativo: l'ordine con cui le carte si sovrappongono dipende da come le si apre.

al caso quella roba", altrimenti succede, come è capitato di vedere a Ruggero, di avere "giochi in solitaria che prendono un tavolo per sei persone".

Il mezzo di trasporto ha poi un altro problema: il treno e l'aereo **si muovono**. Il gioco può starci come spazio, ma i componenti? Uno studente ricorda le **versioni magnetiche** di scacchi e dama, e una versione mini di *Azul* con le tessere che si incastrano. E Ruggero aggiunge: *Jenga* ci sta, come spazio, nel tavolino di un aereo. Perché nessuno ci gioca in aereo? "Per le vibrazioni dovute al movimento." Appunto: "l'ambiente non è solo spazio. L'ambiente è anche suoni, è anche vibrazioni, è un sacco di altre cose".

Approfondimento: misurare il tavolo. Le misure dei tavoli più diffusi nelle case cambiano da modello a modello, ma un tavolo da cucina per quattro persone ha spesso un piano tra gli 80 per 80 e i 120 per 75 centimetri, e un tavolino da caffè è molto più piccolo (spesso intorno ai 90 per 55 centimetri). Il consiglio di Ruggero va preso alla lettera: misurare il piano reale di riferimento e fare un *mock-up* del gioco apparecchiato a grandezza naturale, lasciando spazio per le mani, i bicchieri, le plance personali e i mazzi degli scarti. È uno dei test più economici che si possano fare con un prototipo.

2.10.20 Audio

Altra area progettabile: l'**audio**. Serve la musica nel vostro gioco? Alcuni giochi usano una **colonna sonora** per dare il tempo. L'esempio è *Escape: The Curse of the Temple*[31]: un gioco in tempo reale in cui bisogna uscire dal tempio entro la durata della colonna sonora, "sennò siete fritti". E a intervalli la colonna sonora fa sentire dei **colpi di gong** che attivano eventi meccanici nel gioco. Quindi la colonna sonora non è "solamente un timer, una grossa clessidra": è un **elemento meccanico** che, con l'audio, fa altre cose. E fa anche **fiction**: dà elementi di *world building*, certe *vibes*. "È chiaro che *Escape* è molto Indiana Jones: non avete una colonna sonora techno IDM a palla."

2.10.21 Contesto

"Al **contesto** non ci pensa quasi mai nessuno, ma è molto importante", perché informa il gioco, "non solo in termini di *world building*, ma anche in termini di che cosa vuole che voi facciate".

Jakarta Traffic. *Jakarta Traffic*[32] è un gioco sul traffico di Giacarta, "che è molto ostile": si naviga il traffico per fare delle consegne, e molti elementi del gioco sono **culturali**. Spiegare le meccaniche a chi non sa come è fatta Giacarta, culturalmente, rende il gioco molto più complesso di quanto dovrebbe essere. Qui,

[31] La trascrizione inciampa più volte sul titolo, e Ruggero stesso si corregge ("scusatemi, perché in tedesco..."). Il gioco è *Escape: The Curse of the Temple* (Kristian Amundsen Østby, Queen Games, 2012).

[32] Titolo come nella trascrizione.

secondo Ruggero, hanno fatto un buon lavoro: il manuale aiuta a capire il contesto, e hanno perfino creato una colonna sonora usando il traffico di Giacarta come *sample*, per creare un certo *mood*.

This War of Mine. Ma il contesto non è solo *setting*: può anche **informare le scelte**. Qualcuno conosce il gioco da tavolo di *This War of Mine*[33]? Qual è il suo contesto? "È una guerra, ma non mi ricordo se è una guerra vera." "È a metà": non è scritto apertamente, ma è palesemente l'**assedio di Sarajevo**. E soprattutto, non si giocano militari: si giocano **civili**. Se ci si approccia al gioco "con tutto il bagaglio culturale dei giochi di guerra", ci si aspettano cose che il gioco non può dare, perché "non è un gioco di soldati in guerra: è un gioco in cui c'è la guerra, ma non è un gioco di guerra". Queste cose si risolvono con un mix: in parte con testo, *flavor* e *world building*, in parte con il paragame, con i materiali.

2.10.22 Lingua

Anche la **lingua** è un componente. Alcuni giochi sono **indipendenti dalla lingua**: in un gioco di Lacerda[34], o nella maggior parte dei *German*, sulle carte non c'è testo, "alla buon'icona". Lo stesso *Race for the Galaxy*, lasciando stare il nome e l'effetto ricopiato sotto, si può giocare anche senza leggere l'inglese: "mi basta sapere che cosa fanno quelle icone". Così *The Gallerist*, o *Master of Rules*. Altri giochi no: *Magic* è molto difficile da giocare se le carte non sono nella propria lingua.

2.10.23 La città, la stanza, il telefono

"In quali altri modi posso impattare l'esperienza di chi gioca senza passare per le regole e per la fiction?" Sulla slide c'è un'immagine di ***Pac-Manhattan***[35]: la città, gli aspetti **pervasivi urbani** sono anch'essi paragame.

La stanza come pista. Un altro esempio è un gioco di corsa cooperativo che la trascrizione chiama "Vroom"[36]: usa il **tavolo** come spazio di gioco e chiede di prendere degli **oggetti della propria stanza** e metterli sul tavolo come ostacoli, "fisicamente, proprio".

World of Yoho. Infine *World of Yoho*[37], un gioco di pirati che chiede di usare il **cellulare** su un tabellone gigantesco. Il telefono è la propria **nave**: quando lo si sposta, sullo schermo si muove la mappa, e le parti di quella casella di mare diventano interagibili in digitale. "Si poteva fare senza cellulare? Sì, con un libro degli eventi gigantesco, una seccatura infinita. Mentre così è una cosa immediata, che si spiega in tre minuti." Uno studente chiede se si giochi su un tabellone normale: sì, il gioco arriva in scatola con il suo tabellone, che si mette sul tavolo, e chiede semplicemente di usare il telefono come nave per tutta la partita. Ogni persona usa il proprio telefono; e nelle battaglie navali "vedi i proiettili che vengono sparati da una nave, che vanno sul bordo del telefono ed entrano nell'altro. Pazzi completi. Bellissimo".

Prima dei casi studio, una seconda pausa: "sono le 57", e Ruggero dà otto minuti "per il numero tondo": si riprende alle 19:05.

2.10.24 Casi studio

"Andiamo avanti nell'ultima parte": i **casi studio**, per vedere "come applichiamo le cose che abbiamo appena visto a giochi particolari". Ce ne sarebbero "a tonnellate"; Ruggero ne ha scelti un gruppo "che va da giochi più regolari a cose più *deranged*, come spesso succede", e per ciascuno dovrebbe essere chiaro perché è stato scelto. La tabella 2.10.4 li riassume.

[33] La trascrizione dice "Diceworld Mine".
[34] Vital Lacerda, autore portoghese di Eurogame pesanti, tra cui *The Gallerist*.
[35] La trascrizione dice "Pac-Man Hatton". *Pac-Manhattan* (2004) è un gioco urbano nato alla New York University, in cui persone vere interpretavano Pac-Man e i fantasmi nelle strade di Manhattan.
[36] Il titolo non è stato possibile verificarlo.
[37] La trascrizione dice "World of Yacht", "World of YOHO", "Word of Yoko".

Gioco	**Elemento di paragame**	**Che cosa tocca**
Cuba Libre	due regolamenti, due diagrammi di flusso per i bot, quattro brochure di fazione	regole, accessibilità di un gioco pesante
193 Days	solo un mazzo di carte, niente regolamento	si impara giocando
gioco "a metropolitana"	creazione del personaggio piegata come una mappa turistica	fiction, *world building*
Shenzhen I/O	un manuale fisico da stampare, il faldone della collector's	meccaniche (le schede tecniche), fiction, "feeling"
pinball in scatola	dadi da schicchera su una "scalinata" con i flipper	meccaniche
Lift It	la gru da mettersi in testa	"al centro" di regole, fiction e paragame
Coffee Rush	tazzine di plastica e ingredienti a forma di ingrediente	fiction, *vibes*
Ironwood	una fazione tutta di legno, una tutta di metallo	fiction, riconoscibilità, *vibes* delle fazioni
Nyctophobia	occhiali oscuranti per chi scappa	meccaniche, accessibilità, fiction
Keep Talking and Nobody Explodes	visore VR da una parte, manuale di carta dall'altra	meccaniche, comunicazione
City of Six Moons	nessun regolamento: un artefatto "alieno"	il gioco stesso è dedurre le regole
Scandal in the Deep	un fiammifero acceso come torcia	tempo reale, fiction diegetica
Metal	componenti di metalli diversi	meccaniche di destrezza (peso, massa)
Meltdown	tessere di ghiaccio che si sciolgono	tema e meccaniche, inseparabili

Table 2.10.4: I casi studio della lezione, in ordine di presentazione, con l'elemento di paragame che li rende interessanti e l'area con cui si intreccia.

2.10.24.1 *Cuba Libre*: sei brochure

Si parte da un gioco che Ruggero ama: ***Cuba Libre*** della GMT[38], uno strategico per quattro persone, da tre o quattro ore, sulla rivoluzione cubana. Ha tanto materiale cartaceo, inclusi dei **diagrammi di flusso**, "che è la cosa che tendenzialmente può intimorire le persone".

In alto, **due manuali**: *Playbook* e *Rules of Play*. Perché due? Nessuno ha giocato *Cuba Libre*, ma la classe sa già rispondere: "forse uno ha più le regole, l'altro lo sviluppo del gioco". Esatto. Il *Playbook* accompagna la partita dal *setup* al turno di gioco, "scritto come un manuale di gioco da tavolo"; le *Rules of Play* sono il manuale tecnico visto con *Root*: un manuale "con font di corpo 6-7, diviso in paragrafi, con il glossario tecnico alla fine", in cui, con un dubbio sul combattimento, si va alla sezione del combattimento e si è sicuri di trovare il sottoparagrafo che interessa.

Sotto ci sono **sei "foglioni"**: A4, ma fronte e retro e piegati a brochure, "come se avessimo quattro facciate". Che cosa sono? I due a destra sono **diagrammi di flusso**, uguali, con la scritta *Non-Player*: servono quando si gioca in **solitario**, o quando si vuole riempire una delle quattro fazioni. È un gioco a quattro fazioni asimmetriche che ha senso giocare con tutte e quattro: "ma se siamo in tre, o in due, che cosa facciamo? Non giochiamo a *Cuba Libre*?" I diagrammi simulano nel dettaglio il comportamento di questi **bot**, "un'intelligenza artificiale analogica", "nel senso vecchio del termine, non generativa: macchine a stati". E il testo scritto è notoriamente un mezzo **lineare**, che si applica poco alle macchine a stati; i diagrammi di flusso risolvono questo problema. Già dalla scelta dello strumento si capisce lo scopo: rendere "il più veloce e scorrevole possibile la gestione di automatismi" delle fazioni giocate dal gioco, "senza farle troppo stupide".

Gli altri quattro sono **brochure uguali**, fronte e retro: quattro facciate, una per **ogni fazione**. A che cosa servono, e perché tutti ne hanno una? Una studentessa: riassumono le regole specifiche e asimmetriche di ogni fazione, "e devo sapere che cosa possono fare gli altri perché possono influire sul mio gioco". Esatto: in uno strategico di *counterinsurgency* ("si fanno imboscate, ci si ammazza, ci si rapisce") bisogna sapere che cosa può fare l'avversario per impedirgli di fare "quelle cose brutte che vuole farmi". E per farlo bisogna avere accesso alle stesse informazioni che ha lui: nel "brochurone" c'è la propria fazione riassunta, e le altre tre. "È chiaro che questa cosa ha senso in un gioco strategico di questo peso, di questo publisher. Non metterei un brochurone così dentro *Risiko!*", perché si alienerebbe il *target*.

Approfondimento: il diagramma di flusso come interfaccia. L'osservazione sui diagrammi di flusso vale anche per i progetti del corso. Un automa (un *bot*, un avversario controllato dal gioco, un evento che reagisce allo stato del tavolo) è per natura una *macchina a stati*: in base alla situazione, sceglie un ramo. Descriverlo in prosa costringe chi legge a ricostruire mentalmente i rami; un diagramma li mostra tutti insieme e permette di seguirne uno solo. È lo stesso motivo per cui i diagrammi di flusso sono lo strumento standard per documentare la logica di un gioco, analogico o digitale.

2.10.24.2 *193 Days*: solo un mazzo di carte

All'estremo opposto del peso c'è ***193 Days***[39], un **gioco di ruolo per due persone**, senza *master*. I suoi componenti sono, tutti, un **mazzo di carte**: nessun libretto, nessun manuale, nessun QR code, nessun link. "Quando apro la scatola c'è letteralmente solo un mazzo di carte." Come si impara a giocare? Lorenzo e Tiziana ci arrivano: "si impara giocando"; "come negli *escape* in scatola: la prima carta ti dice come sistemare le altre". Ruggero lo mostra in camera ("ce l'ho qua dietro"): la prima carta spiega che cos'è il gioco e che cosa fare con il resto, cioè separare due mazzetti, blu e viola, e darne uno a ciascuna persona; poi ognuno rivela la carta *start*, che spiega come leggere le altre. "Il gioco ti dice passo passo che cosa devi fare." Non serve un regolamento, perché quello che chiede di fare è "talmente diretto, talmente semplice", pur essendo a livello di *world building* molto complesso.

La storia: due personaggi che vedono il futuro, e sanno che avranno una relazione di coppia che durerà **193 giorni**, dopodiché si lasceranno. Lo sanno entrambi. Ma uno vede **l'unico futuro possibile**, l'altro **molti futuri possibili**; e la meccanica è influenzare l'altra persona con il proprio modo di vedere: chi vede

[38] *Cuba Libre* (Jeff Grossman e Volko Ruhnke, GMT Games, 2013), secondo titolo della serie COIN (*COunter-INsurgency*).
[39] Titolo come nella trascrizione.

molti futuri può aprire alternative all'altro, chi ne vede uno solo può chiuderle. Così si crea la narrazione, senza regolamento.

2.10.24.3 Un gioco a forma di metropolitana

Segue un gioco di cui la trascrizione non restituisce il titolo[40]: "un gioco di ruolo con un asterisco", nel senso che il materiale si può usare anche al di là del gioco stesso, per creare personaggi per altri giochi. È strutturato come una **metropolitana**: seguire le linee fa creare il personaggio. Ed è piegato come una **mappa turistica** della città. "Poteva essere un PDF? Poteva essere un manuale normale? Sì." Ma creandolo come una mappa della città ci sono già il tema, il *world building*, "aspetti di paragame significativi".

2.10.24.4 *Shenzhen I/O*: un videogioco che chiede un manuale

Shenzhen I/O[41] è un videogioco, ma Ruggero lo include perché è **ibrido**. Si interpreta un ingegnere mandato a Shenzhen a lavorare per una piccola azienda di elettronica, e all'avvio il gioco dice: "stop, non iniziare ancora a giocare", e dà il link a un PDF da **stampare fisicamente**. Oppure, "se siete persone pazze come me", c'è la *collector's edition*, una delle pochissime che Ruggero ha in casa: un **faldone** con il manuale stampato, il visto per la Cina, il modulo di residenza da compilare, l'email della società che conferma l'assunzione, "e una busta che vi dice di non aprirla finché il videogioco non ve lo dice". "A questo punto non è più un videogioco, questo."

Il gioco chiede di programmare in un linguaggio simile all'**assembly**. Ruggero l'aveva studiato alle superiori e lo odiava ("perché devo scrivere 40 righe per fare questa cosa, quando posso usare il C?"); con *Shenzhen I/O* ha imparato ad amarlo, "anche a livello formativo". E soprattutto: il gioco **non dice** che cosa fanno i pezzi del puzzle, i componenti con cui si risolvono i problemi. Lo si scopre dal manuale, che contiene le **schede tecniche** dei vari componenti: come sono fatti, che cosa fanno, input e output. Non è un software per lavorare: ha una sua narrazione, colpi di scena, una trama che si evolve. "Ma ti chiede un manuale fisico per essere giocato, e non è casuale: contribuisce all'esperienza."

Ruggero lascia una domanda su cui ragionare, senza risposta: come sarebbe cambiato il gioco se quelle informazioni fossero state accessibili dentro il videogioco, "con un pop-up, un menu, un clic che mi dà la scheda tecnica"? Cambia "l'approccio fisico a come le persone giocano". Lo riprenderà più avanti, a proposito di *Keep Talking*: il gioco vuole ricreare il modo in cui si lavorava in *assembly* negli anni '80, e quel *feeling* specifico: "ho un'informazione, un contenuto limitato e discreto, in cui sono sicuro che ci sia. È qua dentro: devo solo sfogliarlo e capire dov'è".

2.10.24.5 Un flipper in scatola

Poi un gioco da tavolo che vuole ricreare il **flipper**[42]: in basso ci sono i due *flipper* ("ho dimenticato il nome tecnico dei due... bumper"), e si fanno scendere dei **dadi** con una schicchera dall'alto, lungo una "scalinata", per colpire zone speciali, abbattere delle sagome, raccogliere punti; si vince facendo l'*high score*, come nel flipper vero. "Abbastanza autoesplicativo."

2.10.24.6 *Lift It*: diventare la gru

Lift It[43] è un party game di costruzioni in cui "letteralmente diventiamo una gru": ci si mette in testa un supporto con un gancio, e si sollevano e impilano i pezzi con la testa. "Potevamo prendere questi pezzi e impilarli con la mano? Sì. Come cambiava il gioco?" È una scelta di paragame "che con pochissimo fa tantissimo".

Una studentessa osserva che, qui, "la distanza tra la meccanica e il paragame è proprio labile". Ruggero è d'accordo: *Lift It* è "in quell'intersezione"; "se vogliamo essere onesti, è al centro", perché è anche fiction ("noi siamo la gru, nel *world building* del gioco") (figura 2.10.2). "Ve l'ho detto, i confini qui sono molto

[40] La trascrizione dice soltanto "in un gioco complesso, nel frattempo, in metro".
[41] *Shenzhen I/O* (Zachtronics, 2016).
[42] La trascrizione dice "Box top pinball. Haunted house": dal contesto, un gioco della famiglia dei *flipper* da tavolo, nella sua versione "casa infestata". Il titolo esatto non è ricostruibile.
[43] Il titolo originale è *Lift it!*.

grigi." Paradossalmente, in altri casi il paragame è chiaramente più separato dalle regole, in altri ancora più intrecciato.

2.10.24.7 *Coffee Rush*: le tazzine

Coffee Rush[44] è un gioco di impostazione *German*, "anche se fatto da designer coreani", in cui si gestisce un **caffè**: si preparano bevande per i clienti che arrivano, il locale si affolla, a un certo punto c'è troppa gente e i clienti se ne vanno scontenti. "Il classico flusso di gioco da gestire un caffè." Ma le risorse non sono cubetti: hanno **la forma della risorsa**, e non si mettono su una carta, ma in **tazzine di plastica**. "Meccanicamente, sono letteralmente dei contenitori per risorse. Punto." Potevano essere carte. È "un esempio lampante" di paragame: le tazzine a forma di tazzina e gli ingredienti a forma di ingrediente danno quelle *vibes*, aiutano il *world building*, "oltre a essere molto carini al tavolo".

Facendo il punto, Ruggero distingue: *Lift It* e il flipper sono molto legati alla **meccanica**; *Coffee Rush* alla **fiction**.

2.10.24.8 *Ironwood*: legno contro metallo

Ironwood è "un ottimo esempio di paragame intrecciato alla fiction in maniera molto netta". È uno strategico per due, con due fazioni: dei "nani industriali" delle montagne, con un tema un po' fantasy, e un popolo dei boschi. Il popolo dei boschi può muoversi solo nei boschi, quello delle montagne solo nelle montagne. E il popolo dei boschi ha solo componenti di **legno**, quello delle montagne solo componenti di **metallo**. "Potevano essere di cartone? Meccanicamente sì, non cambia niente." Ma essere di metallo o di legno aiuta la **visibilità**, la **tattilità**, la **riconoscibilità** dei componenti al tavolo, "oltre che il *mood*, chiaramente", perché mantiene il tema del conflitto tra le due fazioni, già nel nome: *Iron* e *wood*.

2.10.24.9 *Nyctophobia*: giocare bendati

Nyctophobia[45] è un gioco fatto da una giovane designer e da suo zio, che è **cieco**. Lo zio aveva perso la vista non alla nascita, ma nel corso della vita, e i due giocavano molto da tavolo insieme; così lei ha deciso di fare un gioco *ad hoc* per poter continuare a giocare con lui. Il gioco ricrea il *feeling* di uno *slasher movie*: una creatura assassina nei boschi, di notte, al buio. Chi gioca le persone che scappano indossa degli **occhiali oscuranti** ("non vediamo niente") e deve orientarsi **toccando** le cose sul tabellone; chi gioca il *killer* vede tutto e può osservare liberamente. Anche qui il paragame non è solo materiale: è legato alle meccaniche, ed è coerente con la fiction.

2.10.24.10 *Keep Talking and Nobody Explodes*: la bomba e il manuale

Keep Talking and Nobody Explodes[46] è "un classico": un cooperativo nato per la **realtà virtuale**, in cui una persona, con il visore, vede una bomba in 3D che può manipolare con i *controller*, e deve disinnescarla. Ma non sa come. Le persone fuori dal visore hanno un **manuale tecnico** sparso in fogli su tutto il tavolo: ogni foglio riguarda un aspetto che la bomba può avere (i fili, i pulsanti, i numeri, i colori), con procedure del tipo "se la bomba ha otto fili fai questo, se ne ha sette fai quest'altro, se ha un filo blu...". Chi è dentro descrive ("vedo tre fili gialli"), chi è fuori chiede ("il secondo filo da sinistra è blu?") e risponde ("allora taglia il quarto filo"). La comunicazione tra i due gruppi è **necessaria**, ed è resa più intensa dal fatto che fuori ci sono quattro o cinque persone sullo stesso manuale, in tempo reale, con un timer sulla bomba. "Un giocone. Ve lo consiglio tantissimo."

Uno studente racconta di averlo giocato come videogioco, senza visore e, a quanto ricorda, senza manuale stampato. Ruggero precisa: il gioco si può giocare anche con lo schermo, ma le persone fuori devono stamparsi il manuale ("il gioco te lo diceva"); è nato come **cooperativo asimmetrico** e ibrido, e "non puoi giocarlo solo in videogioco". E ne trae una regola generale: "chi fa i giochi non può obbligare le persone a giocarli

[44] La trascrizione dice che è pubblicato in italiano da "Nasmodest": probabilmente un editore italiano, non ricostruibile con sicurezza.

[45] La trascrizione dice "Nictofobia" e "Vinto Fabio". *Nyctophobia* (Catherine Stippell, 2019).

[46] La trascrizione dice "Keep Talking in Omnid Explode" e "Keep Talking Nomad Explodes". Il gioco è di Steel Crate Games (2015).

come *intended*". Nessuno può andare a casa delle persone a fare "la polizia del gioco: non potete giocarlo così, vi arrestiamo". Si può però consigliare, dire chiaramente "guardate che il gioco inteso è così", come fa *Shenzhen I/O* all'avvio.

2.10.24.11 *City of Six Moons*: un gioco senza regole

Il caso successivo è di Amabel Holland, la stessa autrice di *Irish Gauge*: un gioco in **solitario**[47] che **non ha regolamento**. Il gioco è pensato per essere, **diegeticamente**, un **artefatto ludico alieno**: chi gioca è un essere umano che ne ha recuperato una copia, ma non sa leggere la lingua aliena, e quindi il gioco volutamente non dice come si gioca. È giocabile, ha delle sue regole; ma parte del gioco è **dedurre** e formare teorie su come si gioca. L'autrice ha detto che non risponderà mai a nessuna domanda e che non dirà mai quale sia la versione corretta, e sono nati gruppi di discussione che cercano di capirlo. È il caso estremo di ciò che si è detto su **contesto** e **lingua**: "questo gioco si regge sulla sua non leggibilità. Se avessimo il manuale, perderebbe buona parte del suo senso".

Uno studente lo paragona ai film di **David Lynch**. "Questo è un vaso di Pandora": Ruggero consiglia un video di analisi di *Twin Peaks* "della durata di credo 16 ore", alla fine del quale l'autore dice di credere di aver capito la maggior parte di ciò che Lynch voleva comunicare. "Questo è un gioco da tavolo, non una serie TV in tre stagioni, quindi in teoria è più facile da capire." E ricorda, "come al solito", che i giochi non sono per tutte le persone: i giochi deduttivi e induttivi a lui piacciono moltissimo, ma ha molti amici a cui non piacciono, e viceversa, "magari se a me proponi un gioco sportivo".

2.10.24.12 *Scandal in the Deep*: il fiammifero

Scandal in the Deep[48] è un gioco di ruolo per due persone, un *master* e un personaggio: la figlia di un ladruncolo che entra in un *dungeon*, ruba un artefatto e ora deve solo uscire. Ma ha solo **una torcia**. Il gioco usa un **fiammifero** vero: quando il fiammifero si consuma, la torcia si spegne. Il tempo letterale del fiammifero è quanto resta alla torcia, e questo crea "una coesione tematico-meccanica" tra l'oggetto reale e quello in gioco. "Poteva essere un timer, una clessidra. Lascio a voi la risposta." Con un timer sul cellulare sarebbe stato un gioco peggiore; il fiammifero è **diegetico** per il personaggio.

2.10.24.13 *Metal*: metalli diversi

Metal[49] è una raccolta, una scatola con dentro una dozzina di giochi, se Ruggero ricorda bene, che usano componenti in **metallo**, e di **metalli diversi**: ottone, ferro, alluminio, bronzo. Sono giochi di **destrezza**, e i metalli si comportano diversamente: a parità di dimensione, hanno proprietà fisiche diverse, "in termini anche di peso e di massa" ("se fossi stato un professore di fisica lo avrei saputo dire"). Essendo un *dexterity*, questo è uno degli aspetti meccanici che il gioco sfrutta.

2.10.24.14 *Meltdown*: tessere di ghiaccio

L'ultimo caso, "incredibile": ***Meltdown***[50], un gioco sul **riscaldamento globale**. Si gioca una famiglia di orsi, orso e piccolo orso, che deve salvarsi dallo scioglimento dei ghiacci. Il gioco chiede di mettere l'acqua negli stampi, metterli in freezer, toglierli dal freezer e poi giocare: le **tessere sono di ghiaccio**. Reggono finché reggono; poi si sciolgono, e a quel punto si affonda. "Potevano essere di cartone? Chiaramente no." Il gioco funziona solo perché sono di ghiaccio: "il tema qui è talmente connesso, e l'aspetto di paragame anche connesso alle meccaniche, che non potevano farle in un altro materiale".

Approfondimento: quando il materiale è il tema. Schell propone, tra le sue lenti, quella dell'*unificazione del tema*: ogni elemento del gioco (meccaniche, estetica, storia, tecnologia) dovrebbe rafforzare lo stesso tema. I casi studio di Ruggero mostrano che il materiale è uno degli strumenti più forti per farlo, perché agisce sul

[47] La trascrizione dice "City of Six Moon".
[48] Titolo come nella trascrizione.
[49] Titolo come nella trascrizione.
[50] Titolo come nella trascrizione.

corpo prima che sulla mente: il peso del metallo in *Ironwood*, il buio in *Nyctophobia*, la fiamma in *Scandal in the Deep*, il freddo e l'acqua in *Meltdown*. In tutti questi casi il materiale non illustra il tema, lo fa *provare*. La domanda che Ruggero chiede di farsi ("poteva essere di cartone?") è un buon test: se la risposta è "sì, e non cambierebbe niente", il materiale è solo decorazione; se è "no", è parte del design.

2.10.25 La discussione aperta

"Questa era l'ultima slide." Resta "la solita nostra mezz'ora di *open discussion*". Ruggero insiste: "il paragame dovrebbe essere chiaro a questo punto. Se non è chiaro, vi prego, fatemi delle domande". Non per riempire il buco: vuole che sia chiaro "non solo in termini astratti, ma anche per il vostro gruppo di lavoro", e che già da questa lezione i gruppi si chiedano "come possiamo integrare nella nostra idea di gioco, che *pitcheremo* lunedì, degli elementi che lo rendano un gioco meglio progettato, più sensato, migliore".

2.10.25.1 Il link a *World of Yoho*

La prima richiesta è pratica: una studentessa, che si era persa il nome durante la pausa, chiede il gioco "con i telefoni sulla mappa". Ruggero lo mostra: *World of Yoho* (lei l'aveva cercato scritto male). Consiglia "fortemente" di giocarlo con i telefoni carichi, perché una partita di 90 minuti con il telefono acceso "può risultare un po' intensa". Ed è interessante perché è "quasi un *open world*": ci sono delle *quest*, ci si muove liberamente, si può scegliere se dare la caccia ai mercanti o risolvere missioni in giro per il mondo. "Chiaramente è un gioco che esiste solo nel suo formato ibrido."

2.10.25.2 Il font di *Al Rashid*

Ruggero mantiene poi la promessa del gioco venduto per il font. Lo cerca nella sua collezione ("ci arrivo da *Caligula*, che è dello stesso designer...") e lo trova: ***Al Rashid***[51]. Per il nome del gioco è stato scelto un font con l'alfabeto latino ma "con queste *vibes* un po' mediorientali". Problema: non l'hanno usato solo per la copertina. Anche le carte usano lo stesso font, per parole come *Military*, *Culture*, *Espionage*. "Questo gioco è ingiocabile. Non si legge niente." Ed è "un gioco molto carino, a livello meccanico", ma con le carte scritte così, "buona fortuna". (Gianmarco fa una faccia eloquente: "quella è la mia espressione quando l'ho visto".)

2.10.25.3 Il paragame può agire sulle dinamiche?

Una studentessa dice che il tema le è chiaro ma le sembra "molto difficile, molto complesso", una di quelle cose che si imparano con l'esperienza, vedendone tanti. E chiede: si sono visti molti casi in cui il paragame influenza le meccaniche; ce ne sono in cui influenza **direttamente le dinamiche**, o le strategie?

Ruggero richiama il **framework MDA** e il **DPE**[52]: le dinamiche sono influenzate dalle meccaniche. "Impattando le meccaniche, noi impattiamo le dinamiche." Non si può impattare una dinamica senza passare per una meccanica: anche la colonna sonora al posto del timer agisce sulle meccaniche ("quando senti il gong, devi fare A, B, C sulla mappa"), e di solito anche sulla fiction, perché difficilmente si mette una colonna sonora neutra come un metronomo: la si usa per fare *world building*. E ricorda il principio della parte sui framework: "voi non potete agire sulle dinamiche direttamente. Voi, da persone che progettano, avete le meccaniche". Agire direttamente sulle dinamiche sarebbe come dire a qualcuno "spaventati, ora devi sentire l'adrenalina": "se ti dico sii triste, non diventi triste in automatico. Devi lavorare sulle meccaniche per fare in modo che poi sia quello che le persone vivono". La studentessa riformula: essendo un elemento di design, il paragame si controlla "al pari del regolamento". Esatto.

Ruggero aggiunge che alcuni elementi di paragame aiutano più di altri in una direzione o nell'altra. Il paragame di *Ironwood* aiuta più la fiction che le meccaniche, ma dà anche le *vibes* delle due fazioni asimmetriche: quella dei nani è aggressiva, militarista, "i pezzi di metallo pesano, ti senti uno che vuole

[51] Titolo come nella trascrizione; anche *Caligula* e il terzo titolo citato nel percorso ("Zizi") sono riportati come nella trascrizione.

[52] La trascrizione dice "DPA Framework". Sono i framework visti nelle prime lezioni del corso: MDA (*Mechanics, Dynamics, Aesthetics*) e DPE (*Design, Play, Experience*).

andare a spaccare la faccia"; i pezzi di legno "cadono come foglie, sei fragilissimo", e le dinamiche sono più da imboscata, "dal bosco vengo a rapirti le persone", più *counterinsurgency* che guerra aperta. In *Meltdown* e in *Keep Talking*, invece, il paragame lavora soprattutto sulla parte meccanica e sulla comunicazione. "Un paragame non meno importante", ma l'ideale è che le due cose **non siano mutualmente esclusive**: "nulla vi vieta di avere il paragame da una parte che dà una mano alla fiction, e su un altro aspetto del gioco il paragame che dà una mano alle meccaniche".

Lo scopo, alla fine, è lo stesso: ragionare sul paragame serve ad **assicurarsi**, o almeno a provare a fare in modo, **che le intenzioni di design arrivino a chi gioca**. "Voi non saprete chi gioca il vostro gioco, non potete essere nelle case delle persone che lo comprano." Se si possono comunicare il tema, le emozioni, "con altri elementi oltre alle regole, oltre alla fiction", il paragame è "un elemento in più che lavora al vostro servizio".

2.10.25.4 Graphic design o illustrazione?

Un'altra studentessa torna sulla distinzione tra grafica e illustrazioni: pensava che la grafica riguardasse l'estetica e le illustrazioni l'iconografia. "No, è proprio il contrario." Il **graphic design** è la parte di *design* delle grafiche: le icone, il *wireframing*, la UX, "l'aspetto più meccanico degli aspetti visuali": com'è fatta un'icona, come sono strutturate le carte, se il costo sta in alto a destra o in alto a sinistra. Le **illustrazioni** sono la parte di fiction, che aiuta il *world building*. E non vanno confuse, perché se si fa fare il graphic design a chi illustra, si ottiene l'icona di *Argent* vista all'inizio: "un'icona fatta da una persona che fa l'illustratrice, non da un graphic designer", e lo si vede dal tratto (tabella 2.10.5).

Dalla parte opposta c'è *Race for the Galaxy*, "fatta da un ingegnere, ovviamente". Ma tra le due Ruggero non ha dubbi: preferisce *Race for the Galaxy*. "Fa schifo finché non la imparo, ma dopo tre minuti che gioco so tutte le icone, e riesco a leggere anche quelle che non ho ancora visto, perché hanno una **grammatica** che sto imparando." Quella di *Argent*, invece, "è un disastro: non ne esci vivo".

	Graphic design	**Illustrazione**
Che cos'è	il design delle grafiche: icone, *wireframe*, UX	l'*art*: sfondi, personaggi, luoghi
Che cosa serve	la parte "più meccanica" del visivo: leggere, trovare, riconoscere	la fiction e il *world building*
Domande tipiche	dov'è il costo? quante icone? come si legge in mano?	che mondo è? che *mood* ha?
Se lo fa la persona sbagliata	icone "illustrate", illeggibili (*Argent*)	grafica corretta ma "ostile" o fredda (*Race for the Galaxy*, gli Splotter)

Table 2.10.5: La distinzione tra graphic design e illustrazione, come la chiarisce Ruggero rispondendo a una domanda. "Non fate fare queste due cose alla stessa persona."

2.10.25.5 *Mountaineers*, gli inserti e il *setup*

Ruggero aggiunge poi un'area che "per questioni di spazio" non era nelle slide, con un gioco che ha un problema proprio lì: ***Mountaineers***[53]. Dal lato grafico e delle illustrazioni è "incredibile": è un gioco sullo scalare montagne, con una montagna che si costruisce letteralmente in 3D, con dei pannelli al centro del tavolo, e su cui si scala piantando dei pioli. "Gioco incredibile, visualmente bellissimo." Ma **non ha un design dell'inserto**. Prepararlo è "una seccatura talmente grande" che Ruggero, che lo ama, non lo gioca quasi mai: "l'idea di prepararlo e di sistemarlo mi fa venire i brividi". E vorrebbe essere un gioco *family*, leggero. È un team piccolo e indipendente, che non viene dai giochi da tavolo: la scatola è perfino orizzontale, bassa, "tipo vecchio *Monopoly*, da armadio", non entra in nessuna Kallax, e montata la montagna va smontata ogni volta.

[53] *The Mountaineers*, di un piccolo editore indipendente svizzero.

Gli inserti fatti male. "Il design degli inserti non è casuale. La maggior parte degli inserti è fatta male." Spesso sono i cosiddetti *vacuum tray*, gli inserti di plastica termoformata sottovuoto. Ruggero racconta di aver lavorato in un'azienda in cui chi progettava quegli inserti "non giocava al gioco, non sapeva come fosse", e li dimensionava sulle misure del pacchetto di carte appena uscito dalla fabbrica: carte appena stampate, compresse, senza umidità. "Le carte le tieni cinque secondi all'aperto e già prendono mezzo centimetro in più di spazio." Risultato: tutte le carte uscivano dagli inserti. E se il gioco ha componenti sfusi, serve qualcosa che li blocchi (magari la plancia inserita in un certo modo), altrimenti, aprendo la scatola, tutto si sparpaglia.

Gli inserti stampati in casa. Ruggero è "talmente arrabbiato" con questa cosa che gli inserti se li ristampa in casa, in **stampa 3D**, e li mostra in camera. Un gioco[54] che nella versione base "non ha niente, è tutto sfuso", carte e *token*, e per cui lui ha ristampato l'inserto, con il tabellone che fa da chiusura. Poi, dopo una ricerca tra i giochi della **Oink**[55] ("al decimo gioco della Oink l'ho trovato"), un *filler* di carte con tanti *token* diversi: lui gli ha stampato un piccolo inserto "con una comodissima calamita", che si estrae e si tiene sul tavolo. Così si risparmia di dividere i *token* in mazzetti, "impilare i componenti da 2, da 10, da 5, da 20, da 1, in un gioco che dovrebbe durare 10 minuti": non ha senso spendere 10 minuti di preparazione per 10 minuti di gioco. "Poi io ho dei problemi, di nuovo, con questa cosa qua." Ma è solo il segnale che il settore spesso lavora male, "non del fatto che non si possano fare le cose bene": molti editori stanno iniziando a fare inserti sensati. *Bullet*[56], con la scatola azzurra, ha "un inserto incredibile", alla terza versione, "perché hanno imparato da quanto facevano male le scatole precedenti". E un gioco di treni con la scatola bianca[57] si prepara in cinque minuti, perché gli inserti si estraggono e hanno già dentro il mercato con le tessere. (Qui Ruggero starnutisce: "sono allergico agli inserti stupidi".)

Nessuno ama il *setup*. "Ricordatevi che preparare un gioco è una seccatura. La gente vuole giocarli, i giochi, non prepararli. Chi qui apprezza o ama fare il *setup*? Nessuno. E se qualcuno avesse alzato la mano, stareste mentendo." Quindi: non rendete più complessa del necessario la preparazione del gioco. E non vale solo per i giochi da tavolo, ma anche per i giochi di ruolo, i giochi di ruolo dal vivo, i giochi pervasivi: "la preparazione è una parte intensa: semplificatela il più possibile". Non solo nella **procedura**, ma nella **percezione**. Alcuni giochi "molto furbi" mettono il *setup* in una fase in cui si sta già giocando, con piccole scelte. A volte funziona, a volte no. In molti Eurogame non funziona, perché chiede di conoscere già il gioco per scegliere tra la carta A, B o C: "se non so ancora come funziona il gioco, come faccio a scegliere?". In giochi più leggeri, come *Magical Athlete* o giochi più *party*, è un modo per tenere coinvolte le persone facendole scegliere qualcosa, mentre chi facilita la partita sta già preparando i componenti. (A margine, un rumore di fondo dal microfono di Ruggero: "potevo mutarmi. Ma del resto, anche questo è paragame audio".)

2.10.25.6 Le domande "stupide"

Restano dieci minuti, "almeno due domande". Ruggero ricorda che le slide sono già nella cartella, insieme a quelle delle lezioni precedenti. E insiste: "tutto chiaro? Non è possibile. Sono un essere umano: qualcosa che ho detto non è chiaro, sicuramente". Gli studenti dicono di non avere dubbi "in questo momento" ("i dubbi vengono sempre dopo"). "Ma secondo voi vi giudico per i dubbi?" Un dubbio che sembra stupido non lo è, "e poi un'altra persona aveva lo stesso dubbio, e come voi non lo chiede". E torna al concetto della prima parte: "è il *curb cut effect*: siate la rampina del marciapiede anche per le altre persone". Promette, come sempre, che nelle sue lezioni la mezz'ora finale di domande ci sarà. Una studentessa è "cotta"; Ruggero spera di non aver bruciato il cervello a nessuno: "c'era tanta roba, ma potete guardare le slide con calma". E aggiunge che i casi studio "non sono mai solo casi studio, perché ci permettono di parlare poi di altre cose. Quindi, quando posso, li metto sempre".

[54] La trascrizione dice "Bubba Kiki"; il titolo non è ricostruibile con sicurezza.
[55] Oink Games, editore giapponese di giochi piccoli in scatole tascabili, tra cui *Deep Sea Adventure* (nella trascrizione "Dipsy Adventure").
[56] *Bullet* (Level 99 Games, 2021); il titolo originale ha un cuore dopo il nome. Nel testo il simbolo del titolo è omesso.
[57] La trascrizione dice "Spells and Steam, Empiria"; il titolo non è ricostruibile.

2.10.25.7 Chi ci pensa, al paragame? *Rhino Hero* e *The Cat and the Tower*

L'ultimo intervento lungo è di Gianmarco: il paragame gli sembra una di quelle cose che emergono con l'uso, quando un team "a un certo punto ha tirato fuori il gioco, ha visto l'impatto e poi ha corretto"; ha bisogno di un risvolto pratico. "Tu sei un visionario, Gianmarco", risponde Ruggero: nel mondo ideale è così, "ma purtroppo spesso la gente non ci pensa". E allora, chiede Gianmarco, nella sua esperienza chi ci pensa, e come, quando emergono questi problemi?

"Questo è un vaso di Pandora gigantesco." La risposta, "che so può sembrare paracula", è che **dipende da progetto a progetto**. Alcuni progetti arrivano con delle **direttive di produzione**, e lì c'è poco da fare: "se la produzione dice i costi sono questi, devi stare in questo costo per questa scatola, hai voglia di dire voglio una clessidra alta un metro d'oro puro: non te la daranno mai". Il vincolo produttivo "ti tarpa un po' le ali", giustamente, su materiali e formato. Altre cose sono legate agli aspetti meccanici del gioco, e lì la domanda è: che cosa ci aiuta e che cosa ci rema contro? E parte di queste cose emergono in fase di **ricerca e sviluppo**: guardare quali altri giochi sul tema sono stati fatti aiuta a capire che cosa funziona e che cosa no.

L'esempio finale è un confronto (figura 2.10.4). ***Rhino Hero***[58], "famosissimo", è un gioco di carte per bambini, quasi un *dexterity* semplificato, in cui si costruisce una torre di carte su cui sale un supereroe rinoceronte, con lo scopo di arrivare il più in alto possibile senza farla crollare. È un gioco che vuole dare **il senso di eroismo**, di salvare l'edificio per salvare la gente; esiste anche una versione gigante, *Rhino Hero: Super Battle*, con una torre di cartone più grossa e "delle scimmie infami che si appendono ai bordi". È un gioco che si basa sul ***power play*** come dinamica, sul supereroe.

The Cat and the Tower[59] è un gioco giapponese "bellissimo" in cui un gatto che ha perso la madre deve risalire una torre, metafora dell'aldilà, per rincontrarla: "un gioco tristissimo, straziante". Anche qui la torre è di cartone, esattamente come in *Rhino Hero*. Ma c'è una differenza "molto subdola": i muri sono diversi (con i mattoncini, con gli archetti, con la finestrella piccola), e in base alla loro forma alcuni sono **uno o due millimetri più bassi** degli altri. Nel complesso creano una torre **molto instabile**. "Qui il *power play* non lo vivete per niente. Qui avete il terrore costante che tutto crolli da un momento all'altro. Ed è parte tematica."

La stessa meccanica, dunque, reinterpretata: è ***copy, transform, combine***, "ci ricordiamo, no?". "Fate ricerca e sviluppo, perché vi serve anche per capire, nel paragame, che cosa succede se cambio qualche elemento. Che cosa posso prendere che mi fa la stessa cosa, ma magari reinterpreta il mio gioco?" E chiude con un invito: non solo giochi da tavolo, non solo giochi di ruolo, non solo giochi pervasivi e urbani: "mischiate le cose. **Cross-modalità**".

2.10.26 Chiusura: ci si vede il 12

Mancano tre minuti, "tempo per un'ultima domanda", magari "quella nell'angolino, un po' timida". Non arriva ("adesso non ce la faccio"). Gianmarco si smuta, "il segnale internazionale" di chi sta per salutare, ma non ha domande; Ruggero lo invita scherzosamente a dire anche solo "cose stupide, tipo: oggi ho ucciso una mosca in bagno".

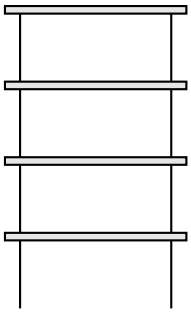
"Allora, gente, io vi saluto, e la lezione di oggi si conclude così." L'appuntamento è **il 12**, con i *pitch*: "non fate scherzi, so che li avete, so che li potete fare". Al *pitch* non ci sarà solo Ruggero: ci saranno anche **Beatrice** e **Daniele**. "Buona serata, buona cena, buon weekend."

Punti chiave del capitolo

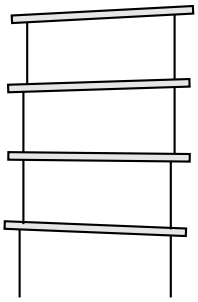
- La **UX** è un primo strato tra la persona che si avvicina al gioco e l'ecosistema di regole: può essere "un incredibile aiuto o un gigantesco ostacolo". Regole perfette che non arrivano a chi gioca finiscono "nel cestino".

[58] *Rhino Hero* (Scott Frisco e Steven Strumpf, HABA, 2011); la trascrizione dice "il gioco dell'Aba".

[59] La trascrizione dice "The Cat and Tower"; Ruggero aggiunge che è stato pubblicato in italiano come *Il gatto e la torre*, "l'anno scorso, mi sembra".



Rhino Hero
muri tutti uguali:
torre stabile
⇒ eroismo, *power play*



The Cat and the Tower
muri più bassi di 1–
2 mm: torre instabile
⇒ terrore che tutto crolli

Figure 2.10.4: Stessa meccanica, paragame diverso: due torri di cartone. In *Rhino Hero* i piani sono regolari e la dinamica è quella dell'eroe che sale; in *The Cat and the Tower* alcuni muri sono di uno o due millimetri più bassi, e la torre, sempre storta, trasmette paura e fragilità. Schema illustrativo, con le pendenze esagerate.

- I **quattro pilastri**: **immediatezza** (si capisce subito, senza sforzo: spade e scudi, il barile di petrolio, l'alloro dei punti vittoria), **efficienza** (un'azione complessa nel minor spazio: "pesca una carta"), **memorabilità** (il +4 di *Uno*, il simbolo di "tappare" di *Magic*), **soddisfazione** (*Parks*, *Canvas*, *Tiny Epic Mechs*). Sono indipendenti: le icone di *Race for the Galaxy* sono poco immediate ma molto efficienti.
- Immediato per chi gioca abitualmente non vuol dire immediato per tutti; anche il **colore** fa parte del significato di un'icona.
- Il **wireframe** delle carte cambia durante il prototipo: togliere ciò che vale zero, ridurre il testo con le icone, spostare ciò che conta meno. Nei primi stadi: **carte funzionali, non carte belle**.
- **Accessibilità**: font per la dislessia, *curb cut effect* (ciò che è più accessibile per qualcuno lo è per tutti), daltonismo da verificare con **Chromatic Vision Simulator**; i mancini e le carte a ventaglio.
- Il **paragame** estende al gioco il **paratesto**: tutto ciò che non è regole e non è fiction, ma incide sull'esperienza. I bordi sono zone grigie (i librogame, *Lift It*).
- Le aree: **materialità** (legno, plastica, metallo, app, spessore e *core* delle carte, *standee*; clessidra contro timer), **formato** (font, misura delle carte, *200 Word RPG*), **regolamento** (pagine, più libretti, video, glossario, *playthrough*, *play aid*), **scatola** (dall'armadio alla Kallax), **grafica e illustrazioni** (due mestieri diversi), **interfaccia analogica**, **ambiente** (il tavolo IKEA, il treno, *Jenga* in aereo), **audio**, **contesto**, **lingua**, **spazio urbano e ibridi digitali**, **inserti e setup**.
- Il paragame **incide sulle regole**: il formato limita la complessità delle regole, la clessidra cambia la tensione, la colonna sonora può essere una meccanica. Ai *pitch* e alle revisioni Ruggero chiederà **"tantissimi perché"**.
- I casi studio vanno da *Cuba Libre* (due regolamenti, diagrammi di flusso per i bot, brochure per fazione) a *193 Days* (solo un mazzo), da *Shenzhen I/O* (il manuale stampato) a *Meltdown* (tessere di ghiaccio). Il test: "**poteva essere di cartone?**"
- Il paragame non agisce direttamente sulle dinamiche: agisce sulle meccaniche (e sulla fiction), come tutto il design. Serve a far sì che **le intenzioni di design arrivino** a chi gioca, anche quando chi l'ha progettato non è al tavolo.
- Gli **inserti** non sono un dettaglio: nessuno ama il *setup*, e un buon inserto lo accorcia. Il *setup* si può anche rendere parte del gioco, nei giochi leggeri.

- **Ricerca e sviluppo** anche sul paragame: *Rhino Hero* e *The Cat and the Tower* hanno la stessa meccanica, ma muri di altezza diversa trasformano l'eroismo in terrore. *Copy, transform, combine*, e **cross-modalità**.

Chapter 2.11

Il pitch dei progetti di gruppo: tre giochi, tre minuti, e i feedback di chi li dovrà giocare

Questa è l'**undicesima lezione** del corso di **Analog Game Design**, ed è diversa da tutte le precedenti. "Non abbiamo slide, non abbiamo una lezione, nel senso di lezione frontale": oggi è il giorno del ***pitch***, quello che Ruggero, a fine della lezione sul paragame, aveva dato per certo come "il 12", che "sta nella pietra". I tre gruppi di lavoro del corso presentano in **tre minuti** l'idea del gioco che svilupperanno fino a dicembre; dopo ogni presentazione i docenti fanno qualche minuto di domande, poi il gruppo esce dalla stanza virtuale e i docenti discutono in privato. Quando tutti e tre i gruppi hanno presentato, la seconda metà della lezione è dedicata "solo ai feedback, esclusivamente ai feedback".

I tre progetti sono molto diversi tra loro. Il primo, ***Murder Memory***, è una *escape room* investigativa da tavolo in cui due squadre, la polizia e un indiziato che ha perso la memoria, risolvono enigmi in parallelo fino a un interrogatorio finale. Il secondo, ***La Peste***, è un *party game* a ruoli segreti, di deduzione e *bluff*, ambientato nel ballo della *Maschera della Morte Rossa* di Edgar Allan Poe: a ogni rintocco dell'orologio si balla, ci si scambiano le carte e la malattia si diffonde. Il terzo è un **gioco di ruolo in solitario**, *deck building*, *roguelike* e di *journaling*: si sopravvive il più a lungo possibile tenendo in equilibrio corpo, mente e status sociale di un personaggio, e alla fine della partita resta la sua storia, da scrivere.

I feedback sono franchi, e i docenti lo dicono subito: non saranno "consigli" ma, dove l'esperienza dice che un gruppo si sta infilando in un "ginepraio troppo grosso per il periodo di tempo che avete", **indicazioni** precise ("non fate questo, fate questo"). I temi che ritornano per tutti e tre i gruppi sono pochi e molto concreti: **convergere** su un'idea e uno *scope* realizzabili, **scrivere il regolamento** ("il vostro gioco lo devono poter giocare altre persone che non sono voi"), collegare l'**obiettivo** di chi gioca a ciò che fa **turno per turno**, partire dalle **emozioni** e non dai bilanciamenti, e usare i **casi studio** come strumenti di *problem solving*, non come modelli da copiare. Ogni gruppo riceve una lista di giochi da studiare, in tutto più di quindici titoli (tabella 2.11.3). L'ultima parte della lezione, aperta alle domande, diventa una lunga discussione su *roguelike*, generazione casuale e narrativa modulare, temi che due dei docenti praticano per mestiere.

La lezione è **online**, su Zoom, e si tiene **lunedì 12 ottobre 2026**[1], presumibilmente dalle 17 alle 20 come le altre: Daniele, che arriva "da sei ore di lezioni", ne fa "altre tre di fila" e a fine serata parla della sua "nona ora". La conducono insieme i tre docenti del corso: **Daniele**, che scandisce i tempi, fa partire il timer e gestisce la sala d'attesa; **Beatrice** ("Bea"); e **Ruggero**. Gli studenti sono divisi in tre gruppi: **Tiziana**, **Angelo** e **Lorenzo L.** (*Murder Memory*); **Tancredi**, **Gianmarco** e **Lorenzo** (*La Peste*)[2]; **Giordano**,

[1] La data è quella fissata nella lezione precedente ("il 12 sta nella pietra", e "lunedì" sono i *pitch*). In questa lezione Daniele ricorda che Gianmarco "di solito lunedì arriva un'ora dopo per motivi di lavoro".

[2] Nel corso ci sono due Lorenzo, e la trascrizione li distingue solo in un punto, quando Daniele presenta il primo gruppo come "Tiziana, Lorenzo L. e Angelo". L'altro Lorenzo è nel gruppo di Tancredi e Gianmarco ("io sono con Gianmarco e Lorenzo", dice Tancredi). Il capitolo scrive "Lorenzo L." per il primo e "Lorenzo" per il secondo.

Nicola e **Rita** (il *roguelike* in solitario)[3].

Tre avvertenze di lettura, le stesse dei capitoli precedenti. La prima: la trascrizione è automatica e storpia moltissimi nomi. Ruggero diventa "Ruger", "Rugger", "Brugger" e "Rugg"; *Murder Memory* diventa "Merd Memory", "Mordor Memory", "Marder Memory" e "Mordere Memory"; *Lupus in Tabula* è "Lupus in Fabula", "l'Opus in Fabula" e "l'upus in tabula"; *Slay the Spire* è "Slade of Spire", "Slade Spire", "Zade Spyder" e "As Lady Spire 2"; *Reigns* è "Rains", "Rain" e "Reins"; *The Quiet Year* è "The Quiet Ear", "White Tier", "quai tier" e "quietire"; *Inhuman Conditions* è "In Human Condition", "Unicondition" e "Non è un Condition"; Tabletop Simulator è "Temporal Simulator" e "Voltoltor Simulator". Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La seconda: con tre docenti e dieci studenti che parlano spesso uno sull'altro, la trascrizione quasi mai segnala chi sta parlando, e in più punti ripete due volte la stessa frase (un artefatto del riconoscimento vocale, che il capitolo ignora). Il capitolo attribuisce le frasi quando il contesto lo rende chiaro (chi viene chiamato per nome, chi dice "io sto creando un *roguelike*", chi ha scansionato un libro) e resta generico altrimenti. La terza: le slide dei gruppi non sono nella trascrizione; il capitolo ricostruisce i giochi da ciò che i gruppi dicono a voce, nelle presentazioni e nelle risposte.

Come leggere questo capitolo. I tre progetti sono descritti per come sono stati presentati il 12 ottobre: idee in fase embrionale, con parti "ancora da definire" che i gruppi stessi segnalano. Il capitolo non le completa e non le corregge: riporta le domande che i docenti fanno, le risposte, e poi i feedback. Le figure ricostruiscono gli schemi che emergono dal discorso (la procedura del *pitch*, il round della *Peste*, il legame tra obiettivo e *agency*, il calendario fino a dicembre); le tabelle riassumono i progetti e i casi studio. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento (soprattutto *The Art of Game Design* di Jesse Schell e *A Theory of Fun* di Raph Koster) e a qualche nozione di psicologia della comunicazione; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo segue quello della lezione:

1. l'attesa, la logistica e le **regole del *pitch*** (sezioni 2.11.1 e 2.11.2);
2. i tre *pitch*, ciascuno con le sue domande: *Murder Memory*, *La Peste*, il *roguelike* di *journaling* (sezioni da 2.11.3 a 2.11.5);
3. dopo la camera di consiglio, la **premessa ai feedback** e la domanda su **che cos'è un regolamento** (sezioni 2.11.6 e 2.11.7);
4. i **feedback** gruppo per gruppo, con i casi studio (sezioni da 2.11.8 a 2.11.11);
5. le **domande aperte**: il regolamento come strumento di test, la *curse of knowledge*, la vergogna delle idee (sezioni 2.11.12 e 2.11.13);
6. la discussione su ***roguelike*, RNG e narrativa modulare**, e la chiusura (sezioni da 2.11.14 a 2.11.15).

2.11.1 L'attesa: una "quotina di ansia"

I primi a collegarsi sono Angelo, Nicola, Lorenzo e Rita; Beatrice e Ruggero sono già in stanza. Daniele "arriva un attimino in ritardo perché sta chiudendo un'altra lezione". La prima domanda è quella di sempre, "come state? come vi sentite oggi?", e la risposta di uno studente riassume l'umore della classe: "leggermente preoccupato per il *pitch*, ma se no tranquillo". "Anch'io, è normale", gli risponde Beatrice: "ogni presentazione ha una quotina di ansia". Lei, dice, è "molto curiosa" e "abbastanza fiduciosa che usciranno bellissimi *pitch*": "non preoccupatevi, siamo proprio in ascolto anche per darvi una mano dove possiamo". Poi va a prendere "il mio quadernino di appunti, così mi segno le cose".

In attesa dei ritardatari, i docenti aprono come al solito lo spazio per le domande, magari di chi ha recuperato la lezione precedente. Uno studente la sta finendo di recuperare: "è una figata pazzesca". Ruggero conferma che "sulla questione del paragame c'è tantissimo da dire" e spera che abbia dato "qualche spunto".

[3] Il nome di Rita è storpiato in "Verità" ("Giordano, Nicola, Verità") e in "Piano" ("Rita, Piano e Nicola"); il contesto e un successivo "ha ragione anche Rita" lo rendono sicuro.

Poi spiega la logistica, che "già dovreste sapere": niente slide e niente lezione frontale; "oggi vediamo i vostri *pitch* e dedichiamo il tempo a ciascun gruppo". Si corregge subito dopo un lapsus ("dedichiamo il gruppo a ciascun tempo. Chi può dirlo? Qualcuno finirà nell'800, qualcuno finirà nel 2000"). L'idea è di dare ai gruppi dei feedback "in base anche al vostro *pitch*".

Arriva Daniele, accolto con un "grazie in fanfare": viene "da sei ore di lezioni" ("e invece adesso guarda, ne fai altre tre di fila"; "Madonna, Daniele, ci lasci le penne. Non svenire"). È lui a segnare le presenze. Arriva Tiziana, poi Tancredi. Mentre manda "due messaggi di lavoro", Daniele dà un consiglio organizzativo: decidete "dietro le quinte" l'ordine di presentazione dei gruppi, altrimenti lo sceglieranno i docenti. Un vincolo c'è già: il gruppo di **Gianmarco**, Tancredi e Lorenzo va lasciato per ultimo, perché Gianmarco il lunedì arriva circa un'ora dopo per lavoro ("tra una quarantina di minuti massimo"). Un altro vincolo arriva da una studentessa del primo gruppo, che deve staccarsi dieci minuti prima per prendere il treno delle otto: il suo gruppo, se i compagni sono d'accordo ("Sì, sì, va bene"), può andare per primo. E così si fa: "iniziamo voi".

Nel frattempo, il silenzio. "Avremo l'*horror vacui*", scherza Ruggero: "se nessuno fa domande, io non so gestire. Daniele, cosa devo fare? Come faccio?". Mancano soltanto Giordano e Gianmarco; Daniele chiede di avvisarlo se entrano mentre i gruppi presentano, così da non perdere le loro presenze.

C'è un ultimo intoppo tecnico, quello di ogni lezione online: la condivisione dello schermo da un Mac ("il problema sempre del Mac per le autorizzazioni"). Daniele sospetta un aggiornamento di Zoom ("e l'aula è diversa, tra l'altro: voi non ve ne rendete conto, ma è un'aula differente"), ma alla fine la presentazione del primo gruppo parte, in modalità "presenta", a schermo intero, con le frecce della tastiera per andare avanti.

2.11.2 Le regole del *pitch*

Prima di cominciare, Daniele ripete "bene le regole del *pitch* di oggi" (figura 2.11.1).

- **Tre minuti.** Daniele fa partire un timer, conta "3, 2, 1, via", e il gruppo spiega "tutto". Chi va oltre non viene interrotto ("vi faccio finire"), ma alla fine si sentirà dire se è andato "troppo lungo" o "troppo corto", "perché comunque sapevate che più o meno il tempo era quello". Il tempo è uguale per tutti: "non possiamo dare la possibilità a ogni team di avere più tempo rispetto agli altri. Altrimenti, è un casino".

- **Due o tre minuti di domande** da parte di Daniele, Ruggero e Beatrice ("comunque quanto tempo ci mettono").

- **La sala d'attesa.** Il gruppo esce da Zoom e rientra: Daniele ha attivato la sala d'attesa. I tre docenti si prendono "qualche appunto in privato di confronto, senza essere registrati", e poi riammettono tutti. Quando una studentessa chiede se la sala d'attesa sia "una stanza che c'è veramente", la risposta è che è "l'anticamera" di Zoom, dove però non si può parlare: chi vuole chiacchierare nel frattempo ha un canale vocale "Analog Game Design" sul Discord del corso.

- Si fa così per tutti i gruppi. La parte dei *pitch* "di solito dura un'ora, un'ora e mezza"; poi "vi diamo i feedback e vi diamo anche dei consigli su cosa fare, perché poi dovrete svilupparle, 'ste robe".

- **Il materiale.** Dopo ogni presentazione il gruppo condivide in chat il link alle slide ("Condividi", "chiunque abbia il link", copia, incolla) e carica tutto nella cartella *project work* del proprio gruppo.

Il primo gruppo avverte che improvviserà ("non ci siamo preparati sulla presentazione"). Nessun problema, ma conviene che parli una persona sola, se ne ha una che "sa tutto quanto"; gli altri intervengono se manca qualcosa. Le domande dei docenti, durante il primo *pitch*, si allungano: alla fine il primo gruppo occupa venti minuti, e Daniele stabilisce per gli altri "venti minuti più o meno a team". Nessun feedback a caldo, nemmeno per chi deve andarsene prima: i docenti vogliono prima "vedere il livello di tutti", "cosa hanno prodotto tutti".

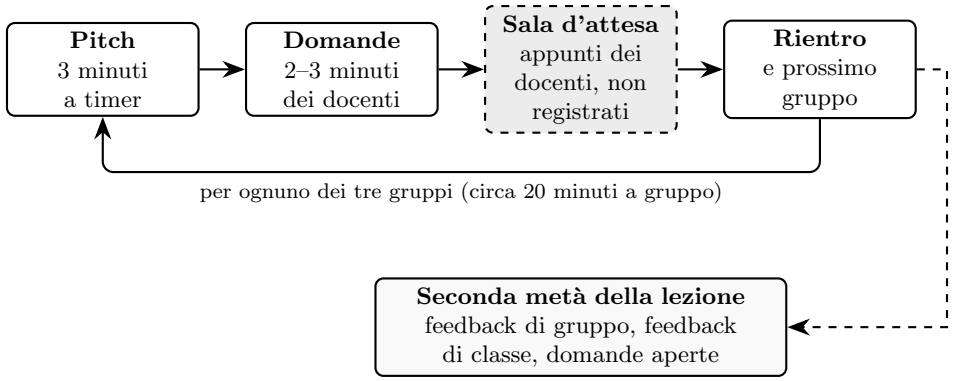


Figure 2.11.1: La procedura del *pitch* come la descrive Daniele. I feedback non arrivano a caldo: i docenti li danno solo dopo aver visto tutti e tre i gruppi.

Approfondimento: il *pitch* come prova di comunicazione. Il formato non è arbitrario, e Daniele lo spiegherà più avanti (sezione 2.11.9): tre minuti sono più o meno il tempo che un editore o un passante in fiera concede a un'idea. Ma il *pitch* è anche il primo momento in cui il gioco esce dalla testa del gruppo e arriva a qualcuno che non c'era quando è stato pensato. Schell, in *The Art of Game Design*, dedica a questo passaggio un capitolo intero ("The Designer Gives the Client a Pitch"), con l'idea di fondo che presentare un gioco serve anche a chi lo presenta: costringe a scegliere che cosa, del gioco, conta davvero. Le domande dei docenti che seguono ogni *pitch* di questa lezione sono, in gran parte, la verifica di questo: che cosa faccio, io che gioco, in questo momento?

2.11.3 Primo *pitch*: *Murder Memory*

Il primo gruppo è quello di **Tiziana**, **Lorenzo L.** e **Angelo**. Presenta Tiziana, con l'accordo dei compagni, che interverranno se lei dimentica qualcosa. Beatrice è "già curiosissima di questo *Murder Memory*", e quando lascia scappare un "ho visto che è un gioco di carte" viene fermata: "non ci si spoilera". "3, 2, 1, via."

2.11.3.1 Il *pitch*

Murder Memory è "una *escape room* investigativa da tavolo" ("ci siamo dimenticati di precisare questa cosa, ma sarà da tavolo"). Chi gioca si trova coinvolto in una **festa nella villa di un ricco proprietario**, durante la quale uno degli invitati viene ucciso. Il principale indiziato è un altro invitato, che ha perso i sensi e si sveglia **senza memoria**.

È un gioco **competitivo e cooperativo** insieme: due squadre si affrontano per arrivare alla verità entro un **tempo limitato**, cooperando al loro interno.

- Una squadra interpreta l'**indiziato** che ha perso la memoria. Non ricorda chi è né che cosa sia successo; potrebbe essere l'assassino e non saperlo. Si nasconde dalla polizia mentre cerca la verità, il che lo rende ancora di più il principale indiziato. "Enigma dopo enigma" deve scoprire chi è e svelare il mistero: "è davvero lui il colpevole, o è solo un'altra vittima?". Ogni giocatore della squadra potrebbe interpretare **un aspetto diverso della stessa persona**: è una cosa "ancora da definire", perché il gruppo non sa ancora se riuscirà a realizzarla in concreto.

- L'altra squadra interpreta i **poliziotti**, chiamati dal proprietario. Seguono le indagini, cercano indizi e interrogano gli invitati e il proprietario della villa, anche a proposito dell'indiziato che non riescono a trovare.

La struttura è una **successione lineare di enigmi**, che guida ciascuna squadra alla risoluzione finale; la risoluzione prevede un **confronto diretto** tra le due squadre, attraverso un **interrogatorio**. Il gruppo

vorrebbe diversificare il più possibile gli enigmi della successione, così da far partecipare più giocatori. L'obiettivo dell'indiziato è scoprire la verità su se stesso e su ciò che è successo; quello dei poliziotti, trovare il colpevole.

L'obiettivo del gruppo, come designer, è "esprimere noi stessi ed esplorare nuove tipologie di gioco", fondendo elementi investigativi e di gioco di ruolo con la struttura dell'*escape*; e "non ci dispiacerebbe pubblicarlo". Le **emozioni** che vogliono suscitare sono **incertezza**, **scoperta**, un'esperienza **adrenalinica**, **sorpresa** e **ansia**. Chiude una battuta ("prenotatevi per provarlo, nessuno vi picchierà") e la presentazione dei membri del gruppo, ognuno con un nome da personaggio[4].

Tempo: **tre minuti e quattordici secondi**. "Direi che siamo comunque in *time*", commenta Daniele: con un po' più di velocità ci sarebbero stati tranquillamente.

2.11.3.2 Le domande

Come si vince? Daniele parte con un apprezzamento e un dubbio. "Lato emotivo, molto bene": le emozioni sono chiare. "Lato meccanico, ho un po' di dubbi." Non è chiaro se il gioco abbia un ***endgame* predefinito**, cioè se a livello meccanico si vinca o si perda ("com'è che so se ho vinto o se ho perso?"), o se vada "nella direzione più RPG", in cui conta non chi vince ma "avere un epilogo strutturato".

Chi risolve gli enigmi? Un'altra domanda, più pragmatica, arriva dai docenti insieme: le squadre sono due, ma gli enigmi chi li risolve? Insieme? Una squadra sola mentre l'altra aspetta? ("Hanno due serie diverse.") Il punto, chiarisce chi chiede, è che non è chiaro "che cosa faccio, in base al ruolo in cui sono", e con quale obiettivo personale al tavolo. Gli studenti non sapevano se dovessero entrare in questo dettaglio nel *pitch*; la risposta dei docenti è: se la risposta c'è, ditela; se non c'è, "non generatela adesso".

La risposta c'è. Gli enigmi procedono **in contemporanea**, con lo stesso tempo per entrambe le squadre. I poliziotti hanno tutte le informazioni se riescono ad arrivare alla fine del loro percorso; l'indiziato, arrivando alla fine del suo o il più vicino possibile, ottiene informazioni su se stesso. **Allo scadere del tempo** le due squadre si confrontano: i poliziotti fanno all'indiziato delle domande "ovviamente previste", e l'indiziato saprà rispondere più o meno correttamente a seconda di quanto ha scoperto. Se non sa chi è, "rischia di rimanere un po' così" su certe domande: può inventarsi le risposte, ma rischia di non essere coerente. Ne nascono diversi esiti (tabella 2.11.1).

Situazione allo scadere del tempo	**Esito (come descritto nel *pitch*)**
Entrambe le squadre arrivano alla fine del percorso	"vincono entrambi": i poliziotti hanno gli elementi per capire chi è il vero colpevole, e l'indiziato sa rispondere
I poliziotti arrivano alla fine, l'indiziato no	vincono i poliziotti: l'indiziato non risponde correttamente alle domande e viene arrestato
L'indiziato ottiene le informazioni su di sé	l'indiziato vince: risponde correttamente e viene scagionato

Table 2.11.1: Gli esiti dell'interrogatorio finale di *Murder Memory*, come li descrive Tiziana rispondendo alle domande. Il gruppo li presenta come idea ancora aperta.

Quanti giocatori? Daniele ha "due domande molto brevi". La prima: per quanti giocatori è il gioco? Minimo due, uno per squadra; il gruppo pensava a un massimo di **quattro per squadra**. Si interpretano solo i membri delle squadre? Sì: gli invitati sono **NPC**. La seconda domanda, a Daniele, sfugge ("la sto perdendo? Arrivo"); Beatrice lo rassicura ("è normale, poi tornano le domande mentre si parla") e prende la parola.

[4]La trascrizione riporta "siamo Lorenzo, la Pomarda, Angelo, Tenente Colombo e io, Tiziana Carabayt": almeno "Tenente Colombo" è chiaramente un soprannome scherzoso a tema investigativo; le altre parole non sono ricostruibili con sicurezza.

L'indiziato vuole scagionarsi o sapere la verità? Beatrice ha "tantissime domande" e ne seleziona alcune. La prima: la squadra dell'indiziato ha più lo scopo di scagionarsi o di ricostruire la verità? Deve scoprire la verità su se stesso, risponde Tiziana: scoprendo chi è, può affrontare i poliziotti. Beatrice chiede allora conferma di un punto di design: si parte dall'idea che l'indiziato **non è colpevole**? Corretto: non è colpevole, "però non lo sa".

Le sfaccettature dell'indiziato. La seconda domanda di Beatrice riguarda le "varie modalità di essere" dell'indiziato, una per giocatore. Come funzionerà? Il dubbio di Beatrice è più ampio: "avete tanta carne al fuoco". Enigmi, *drive* narrativi, una storia raccontata in più versioni, una polizia che non è una persona sola. "Non vi sto invitando per forza a ridurre lo *scope*", ma "è bene che abbiate una direzione e una motivazione per tutte le scelte che prendete". L'idea dell'indiziato interpretato da una squadra intera le piace molto, ma non capisce che cosa farà lei, giocando l'indiziato, oltre al lavoro di ricerca con le altre parti di sé: e quando dovrà scagionarsi, come funzionerà? "Compileremo una scheda, oppure parleremo con la polizia?". La risposta: la **mente divisa** era un'ipotesi iniziale, "quasi necessaria" per giustificare più persone che fanno un'unica persona, ma non obbligatoria, perché "è nostra intenzione semplificare, non complicarci la vita".

"Dovete spoilerare". A questo punto il gruppo ammette che manca una parte, taciuta apposta "perché non volevamo spoilerare". La reazione dei docenti è immediata: "Dovete spoilerare. Non siamo giocatori". Al *pitch* i docenti sono come un *publisher* a cui si propone un gioco: se non gli si dicono le cose importanti, "noi come facciamo?". "Non lo sapevamo. Scusate." "Tutto bene. Dici tutto adesso."

Il retroscena è questo. Il **proprietario** della villa è un **appassionato di enigmi** ed è lui **l'assassino**: ha ucciso uno degli invitati per costruire tutta la messa in scena, "fondamentalmente per divertirsi". È lui a far perdere la memoria all'indiziato, facendogli bere qualcosa. L'indiziato si sveglia in un punto in cui non è visto, si accorge che è successo qualcosa (c'è la polizia) e va subito in agitazione: non ricorda chi è, e se gli fanno una domanda non saprà cosa rispondere. Mentre si nasconde trova **un oggetto che gli è familiare**, e da lì parte la serie degli enigmi. Ha la sensazione di essere **guidato**, pilotato, che ci sia qualcosa che non torna, ma "non è il momento" di chiederselo. Dall'altra parte, il proprietario chiama i poliziotti e si diverte anche alle loro spalle: li guida attraverso altri enigmi, con cui ricostruiscono le informazioni sugli invitati e, attraverso gli invitati, sull'indiziato che non trovano. "Scusa, no, è importantissimo", commentano i docenti.

Storia prefatta o emergente? Daniele fa un'ultima domanda, "molto semplice": il gioco ha una **storia prefatta**, definita, che i giocatori stanno solo scoprendo? Non è una storia emergente? Sì, è prefatta.

I giochi più vicini. Poi tocca a Ruggero, con una domanda "apparentemente dal nulla", le cui ragioni verranno dette dopo: nella fase di ricerca e sviluppo, quali sono i giochi più vicini al vostro? Ruggero ne ha già "due segnati qua" che "devono uscire fuori da questa risposta". Non è un indovinello, precisa: se i giochi citati dal gruppo coincidono con i suoi, bene; se sono molto diversi, "allora ci sono altri problemi", perché vuol dire che i docenti hanno capito un'altra cosa. Daniele allarga la domanda: non serve un gioco unico che spieghi tutto il progetto, che ibrida generi diversi ed è in gran parte sperimentale ("è molto chiaro quello che ci state proponendo"); bastano "un po' di ispirazioni", e la ricerca e sviluppo "può essere un film, può essere una canzone".

Tiziana racconta l'origine dell'idea: il gruppo ha scelto la proposta di Lorenzo L., un personaggio che perde la memoria e cerca di ricostruirla, e poi l'interesse comune per le scelte, per l'*escape* e per gli elementi investigativi. Lei ha cercato giochi simili "e non ho trovato niente"; ha giocato investigativi e qualche *escape*, ma non si è ispirata a nessuno in particolare ("se ci volete dare qualche indicazione, siamo contenti"). Aggiunge un'idea sua sul formato: dare i percorsi "in modalità busta", con buste da aprire che contengono il testo scritto; ma nel gruppo c'è chi ha un'idea diversa. "Quello lo svilupperete dopo". Lorenzo L., invece, un riferimento ce l'ha: quando aveva proposto la perdita di memoria, gli era venuto subito in mente un titolo della serie ***Hidden Games***, in cui tutti hanno perso la memoria, tutti indagano e uno di loro è l'assassino: *Freddo come il ghiaccio*[5]. "Bravissimo, Lorenzo." Angelo, a parte le *escape room* giocate ("non particolarmente tante"), non ha riferimenti: è proprio questo che lo ha spinto ad avvicinarsi al genere.

[5]Così nella trascrizione ("Freddo come il ghiaccio. Bravissimo, Lorenzo"). *Hidden Games* è una serie di giochi investigativi in scatola; il titolo esatto del caso citato non è verificabile dalla trascrizione.

Sono passati venti minuti, ed è arrivato Gianmarco. Daniele riassume per lui la procedura (presentazione, domande, sala d'attesa, feedback alla fine) e manda il primo gruppo in sala d'attesa.

2.11.4 Secondo *pitch*: *La Peste*

Visto che Gianmarco è arrivato, il secondo turno tocca al suo gruppo: **Tancredi**, **Gianmarco** e **Lorenzo**. Presenta Tancredi. Prima di partire, però, c'è un equivoco da chiarire: il gruppo "non si era orientato per i tre minuti". Aveva capito che il *pitch* fosse "una cosa un po' più lunga", e ha preparato una struttura "un po' più cicciotta", poi riadattata con alcune slide in appendice. "Sapevate quanto tempo ci davamo?" "Noi avevamo trenta minuti." Daniele si chiede se sia stata colpa sua, ma ricorda di essere stato chiaro: "i *pitch* erano da tre minuti, tre o quattro minuti di solito". Gli studenti stessi trovavano strano un *pitch* così lungo ("come idea avevamo tutti in mente un *pitch* che fosse una roba molto veloce"). Il materiale in più non va buttato: "tenetelo da parte, che servirà". E il gruppo, alla fine, sta nei tre minuti: "bravissimi".

2.11.4.1 Il *pitch*

Il gioco si chiama ***La Peste*** ed è ambientato nel racconto *La maschera della Morte Rossa* di Edgar Allan Poe ("ce la posso fare"). È un **party game a ruoli segreti**, di **deduzione e *bluff***. Uno degli invitati del ballo rappresenta la malattia, che il gruppo chiama **Pestilenza**, e il suo obiettivo è espandersi. A ogni **rintocco dell'orologio** (scandito da un piccolo timer, che definisce i round) ci si mette a **ballare**: durante il ballo i giocatori si scambiano le carte, e in base alle carte che si hanno in mano alla fine del turno si è sani o malati. Dopo il ballo c'è la **fase sociale**, "quella un po' più caciarona da *party game*", in cui ci si accusa e si cerca di capire chi si è ammalato e chi no.

I **pilastri** del gioco sono tre: la **dualità segreta** tra Pestilenza e Medico, che si combattono senza che nessuno sappia chi sono; la **malattia che si espande**; la **deduzione e il *bluff***. Le **emozioni** principali sono l'**incertezza** (non sapere bene i ruoli), l'**ansia** e la **preoccupazione**, e l'**inclusione**: i giocatori malati **non vengono eliminati**, come invece "ogni tanto succede nei giochi di questo tipo". Il formato è da **5 a 8 giocatori**, per circa **30 minuti** ("ancora un po' da decidere").

I ruoli sono tre (tabella 2.11.2). Il **setup** è a **buste**, come in *Secret Hitler*[6]: ogni busta è già preparata in base al numero di giocatori e contiene le carte di chi la riceve. Il gioco si sviluppa in **sette round**, il numero dei rintocchi ("va ancora sistemato"); ogni round ha tre fasi: il **ballo**, in cui ci si scambiano le carte; la **risoluzione**, in cui in base alle carte che si hanno "succedono cose" ("tendenzialmente ti ammali"); la **fase sociale**, la discussione. Alla fine si guardano le **condizioni di vittoria** (figura 2.11.2).

Ruolo	**Carte iniziali**	**Obiettivo e comportamento**
Pestilenza	una carta *peste*, che trasmette la malattia, e una carta vuota	l'unico che parte malato; vuole contagiare tutti
Medico	due *talismani*, che proteggono dalla malattia ("non ti fa ammalare")	combatte la Pestilenza senza farsi riconoscere
Cortigiani	solo carte vuote (*invito*)	finché sono sani giocano con il Medico; quando si infettano passano dalla parte della Pestilenza e vogliono far infettare tutti

Table 2.11.2: I ruoli della *Peste* come presentati al *pitch*. Le carte vuote, nella descrizione del *setup*, si chiamano "invito".

Il **target** è pensato per una serata in cui il gioco viene "intavolato" da una persona, magari "quella un po' più giovane", fan di questa categoria di giochi: l'appassionato di *Lupus in Tabula*, che non conosce *La Peste* ma sicuramente la apprezzerà, perché il genere gli piace. Accanto a lui, due target più *casual*: chi non ha mai giocato e si sta avvicinando ai giochi da tavolo per la prima volta, e chi "subisce il gioco", perché non sapeva che quella sera si sarebbe giocato.

[6] Nella trascrizione "Secret Teaser", poi "Secreti Hitler".

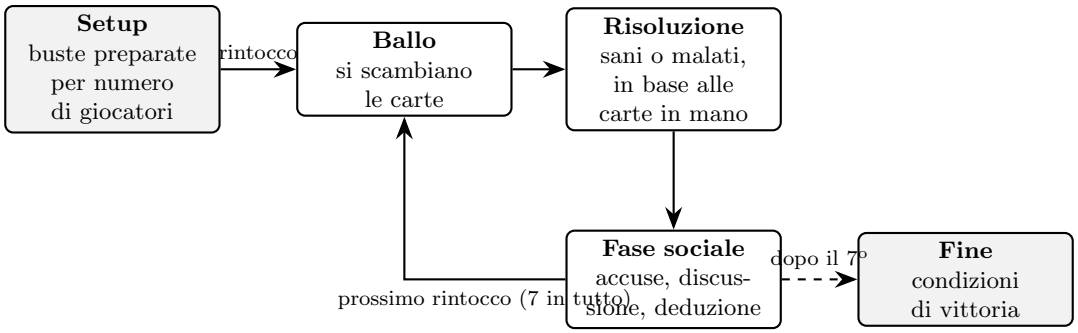


Figure 2.11.2: Il ciclo di gioco della *Peste* come descritto al *pitch*: sette round, uno per rintocco dell'orologio, ciascuno con ballo, risoluzione e fase sociale. Il numero di round è ancora da sistemare.

Tempo: **due minuti e cinquantotto secondi**. Tancredi aggiunge una cosa che stava nelle slide in appendice: una **variante interpretativa**, che al gruppo piacerebbe molto, perché potrebbe aiutare a sdrammatizzare, ma che rischia anche di rendere il gioco "un po' più caotico".

2.11.4.2 Le domande

Daniele chiede, come sempre, il link alle slide e il caricamento nella cartella *project work* ("c'è tutto, sia la versione *full* che la versione tagliata"). Poi la prima domanda, "la mia classica": quali titoli di riferimento avete osservato? "Secret Hitler e Lupus in Tabula: quelli sono già ovvi." Non sono nelle slide, ma ce ne sono vari.

I cortigiani possono difendersi? Beatrice fa una domanda "forse prematura": il Medico ha due carte per difendersi; i cortigiani, se si beccano la carta peste, hanno qualche arma? Per ora no. Il Medico, anzi, potrebbe essere immune: va testato quanto è forte. Si potrebbe dargli una sorta di protezione propria, "come se avesse già un talismano per sé". E c'è un'altra meccanica a cui il gruppo ha pensato: se un giocatore si ritrova con **due talismani**, **diventa anch'esso un Medico**. Ne nasce una tensione interessante: c'è chi non vuole cedere il proprio talismano per paura, "però meno lo fai girare, meno aiuti i compagni", e così, senza volerlo, aiuta la Pestilenza a espandersi. "Molto bene."

Si può giocare senza componenti? Ruggero, per ora, non ha domande. Daniele ne fa una "fuori dai denti": questo gioco si potrebbe intavolare senza nessuna componente materiale, come un *Lupus in Tabula* di base, o il gruppo pensa a un formato speciale? Le carte servono, risponde il gruppo, insieme a una scatola e a dei componenti. E una versione da LARP, in costume? Ci hanno ragionato, ma l'idea è giocarlo al tavolo, seduti, con le carte.

Le buste. Ultima domanda di Daniele: come si gestiscono le buste nel *setup*? Ogni busta conterrà la carta ruolo e le carte di gioco ("talismano, talismano, peste, invito, invito, invito"). Manca ancora la parte su come comporre le buste a seconda del numero di giocatori, che "cambieranno". Beatrice ha altre domande, ma le terrà per la discussione privata dei docenti. Il gruppo va in sala d'attesa ("ci vediamo qua fra pochissimo").

2.11.5 Terzo *pitch*: un *roguelike* di *journaling* in solitario

Mentre gli altri rientrano arriva **Giordano**, che è stato aggiornato sulla struttura della lezione dai compagni "mentre stavo tornando a casa". Il suo gruppo, con **Nicola** e **Rita**, è l'ultimo. Presenta Nicola, che condivide l'intero schermo e avverte: "a differenza degli altri" il gruppo ha raccolto tutto "all'interno di una singola slide". "Non c'era l'obbligo di più slide, non ti preoccupare."

2.11.5.1 Il *pitch*

Il gioco è un **gioco di ruolo in solitario** e un ***deck building game***. A differenza di molti giochi che vanno su approcci da librogame ("pagina X, pagina Y"), il gruppo ha voluto creare un ***loop* di gioco rigiocabile**, adatto a due tipi di persone: chi vuole avvicinarsi al gioco di ruolo da solo, "con la comodità di casa sua, senza avere paura delle altre persone"; e il giocatore di ruolo "molto avanzato", che vuole facilitare la **creazione del proprio personaggio** attraverso una storia dinamica.

Le ispirazioni dichiarate sono *Slay the Spire* e *Reigns*: ci si trova di fronte a delle **situazioni**, si risponde con delle **carte** e si devono tenere in equilibrio alcune **statistiche della vita** del personaggio, per esempio lo **stato fisico** del corpo, lo **stato della mente** e lo **status sociale**. Il giocatore è uno solo. Il gioco è pensato per essere **facilmente espandibile** con nuovi "settori": la prima versione ha un'**ambientazione moderna**, in cui si interpreta "X persona, senza genere, per essere inclusivo verso chiunque".

All'inizio si fa la prima pescata di carte, che assegna al personaggio dei **tratti casuali**: il personaggio può essere irascibile, con certi bonus e malus in certe situazioni, oppure può avere la malaria, con altri bonus e malus. Bisogna **sopravvivere** e andare "il più possibile lontano", fino a quando una delle barre si esaurisce. La morte può essere **fisica** (il corpo non ce la fa più, magari perché il personaggio è stato "un fumatore incallito per tutto il tempo"), **mentale** (impazzisce) o **sociale** ("non riesce più a sopportarsi all'interno del mondo"). L'ambientazione è moderna, ma il sistema è pensato come ***setting agnostic***: "l'obiettivo è stato il *loop* di gioco".

Chi gioca può anche non affrontare il lato narrativo, ma sarebbe "desiderato", perché l'obiettivo è incitare al ***journaling***: quando la partita finisce, con la morte del personaggio, si ha già la sua storia scritta, e si può fare *journaling* dal punto di vista narrativo, oppure usarla per imparare a fare una *run* migliore e arrivare più lontano.

Sui **componenti**: carte personalizzate, ma se stamparne "un numero gigantesco" dovesse essere un problema, il gruppo pensa di usare un **mazzo di carte francesi**, in cui seme, valore e carta determinano se si tratta di una situazione, di un evento o di una carta da giocare. I tratti sono casuali all'inizio, e altre carte si aggiungono durante la partita. "Fine."

Tempo: **tre minuti precisi**.

2.11.5.2 Le domande

Ruggero aveva già "elencato i giochi ispirati" e non ha domande, ma ha "un po' di cose su cui sto ragionando". Lascia la parola a Beatrice e a Daniele.

Quali emozioni? Beatrice cercava nella slide le emozioni e non le ha trovate. La risposta del gruppo: la **curiosità** di sapere che cosa c'è dopo, di andare avanti, e la **competizione** con se stessi, il portare "uno *score* sempre migliore": "alla fine è un *roguelike*, l'obiettivo è cercare di andare sempre più lontano". L'emozione di fondo è la curiosità, anche quella di provare altre emozioni, che dipendono da ciò che accade al personaggio. Beatrice precisa: quindi le emozioni che si troveranno nelle carte non sono ancora definite? No. "Avete una sorta di emozioni *overall*, però poi dovete ancora fare il design delle emozioni." Il gruppo spiega che voleva lasciare spazio all'esplorazione di qualsiasi situazione, grazie alla casualità: una *run* con un personaggio "depresso, malato, rovinato" durerà pochissimo, e l'emozione, immedesimandosi, sarà quella di una vita fallimentare; se invece "è andato tutto quanto *gold*", può essere soddisfacente. È la curiosità di sapere come andrà la prossima volta, e di "giocare la migliore mano possibile". "Un po' come la citazione che si dice nella vita: tu non decidi le tue carte, decidi come giocartele."

Quante carte, quante storie? Daniele chiede una proiezione: quante carte, di che tipo, con un esempio di carta evento e di carta personaggio. La stima è **tra 50 e 56 carte**, più o meno un mazzo di carte francesi. "Ma con queste 56 carte, quante storie volete narrare?" Con l'approccio "alla francese", risponde il gruppo, una carta può avere significati diversi a seconda della situazione in cui viene pescata. In alternativa ci sarebbero due mazzi: le **carte evento**, situazioni che accadono ("perdi tot, ottieni tot, pesca tot carte", oppure "le due prossime pescate sono forzate"), e le **carte da giocare**: una passiva negativa che non si può più togliere, una disponibilità economica che permette di superare una certa situazione al prezzo di un

sacrificio, "come in *Reigns*", in cui si sacrifica un lato per guadagnarne un altro. "Questo lo do nei feedback", annota Daniele.

***Journaling* e meccaniche.** L'ultima domanda di Daniele: c'è una connessione tra le risorse da tenere in equilibrio, le carte e il *journaling*? Oppure il *journaling* è lasciato al giocatore, che dopo ogni partita deve farlo da sé, senza che le meccaniche lo incitino direttamente? Al momento, la seconda. Ma il gruppo precisa poco dopo che la questione non è ancora decisa: "se l'obiettivo iniziale era incitare al *journaling*, dovremmo inserirlo anche nelle meccaniche"; in questa fase embrionale, però, voleva prima capire se l'idea andasse bene, senza entrare troppo nel merito. Chi vuole approfondire trova nel materiale il link alla lavagna Canva del gruppo, usata per far venire le idee e costruita con lo strumento suggerito da Beatrice, "modificandolo un po"'; il gruppo la aggiornerà con il numero di ore da dedicare a ogni cosa, una volta stabilito.

Chi ha giocato un *journaling RPG*? Ruggero, che sta zoomando sulle lavagne del gruppo ("se mi vedete *puzzling* non è perché non sto capendo"), fa una sola domanda: chi di voi ha giocato dei *journaling RPG*? Nicola ha giocato pochi librogame, ma nessun *journaling RPG*; quello che ne ha giocati di più è Giordano, che ha messo tra le ispirazioni *Apothecaria*[7], il gioco da cui il gruppo era partito.

Il gruppo esce. I docenti vanno "per l'ultima volta in camera caritatis" (cioè in camera di consiglio) e danno appuntamento a "una decina di minuti, un quarto d'ora".

2.11.6 Prima dei feedback: indicazioni, non consigli

Si rientra alla spicciolata. Lorenzo L. resta qualche minuto in "*joining*" ("è sempre Zoom"), e Daniele aspetta che ci sia, visto che i feedback andranno in ordine di gruppo. "Come vi sentite?" "Normale, sembrava quando esce la commissione della tesi di laurea."

Grazie, e su che cosa sarete valutati. Daniele apre con un ringraziamento: "non è normale arrivare a questo punto di un corso breve e impegnarsi a produrre qualcosa con altre persone che avete conosciuto al momento". Chi ha fatto molte slide e chi ne ha fatta una sola, "non importa": conta che si veda il lavoro di pensiero. I feedback riguarderanno **l'idea**, non il modo in cui è stata presentata. Il *pitching* non è materia di questo corso breve, e oggi non verrà valutato; ma Ruggero gli ha riferito che la classe vorrebbe una lezione sul *pitching*, e "se la volete veramente tanto, ve la posso aggiungere".

Indicazioni forti. Poi il punto più importante della premessa. I gruppi hanno poco tempo per sviluppare le idee, e dove l'esperienza dei docenti dice che "il ginepraio dove vi siete infilati è troppo grosso per il periodo di tempo che avete", i feedback saranno puntuali: "non fate questo, fate questo". "Ascoltate questi feedback, introiettateli. Questi non sono consigli. Questo è: fate questa cosa in questo modo." Altrimenti, dice Daniele con un'immagine, il rischio è di arrivare alla fine dei due mesi "che avete raccolto l'uva dalle vigne, che è buonissima, ma magari non era il motivo per cui eravate qui". Non è un trattamento speciale: succede "ogni anno", a ogni gruppo, perché l'obiettivo di tutti è che i gruppi **finiscano il loro gioco**.

Imparare dal fallimento, ma fino a un limite. Arrivare a fine corso con un gioco "che magari non è il massimo, un po' zoppo" va bene: da un prodotto finito in tre mesi, anche se non funziona, si impara moltissimo ("io quel prodotto lì magari non era funzionale, non era bellissimo, però ho imparato molto"). Ma c'è un limite: i docenti devono tutelare i gruppi dalla possibilità di ritrovarsi "tra le mani una patata bollente che va oltre il limite dell'imparo dal fallimento", una cosa così grande che non c'è proprio il tempo di dedicarle. Su quei punti arriveranno indicazioni puntuali; poi consigli, definizioni, e qualche cosa "che più o meno tutti i gruppi hanno bisogno di sentirsi dire": feedback di gruppo e feedback di classe.

[7] Nella trascrizione "apotecare". *Apothecaria* è un gioco di ruolo in solitario di *journaling*, in cui si interpreta un'erborista che cura gli abitanti di un villaggio.

Il regolamento di massima. Infine, il calendario (figura 2.11.3). I docenti rivedranno la classe **in aula il 6 novembre**. Da qui ad allora i gruppi devono sviluppare l'idea ("non la possibilità: si spera che lo facciate davvero") e arrivare con un **regolamento di massima**: "non impaginato, non imbellettito, senza disegni", ma un documento con le regole che permetta di giocare il gioco, e che sia la base da migliorare con le lezioni di novembre. I docenti vogliono poterlo leggere, immaginare di provarlo e valutare com'è strutturato il gioco. Il regolamento si consegna **una settimana prima della *review***, così i docenti avranno una settimana per leggerlo e preparare i feedback. "Il regolamento per il 6 deve esistere. Non bello": anche perché il **9 novembre** c'è la lezione sul regolamento e sul *playtest*, e Daniele non si aspetta che i gruppi sappiano già, prima di quella lezione, "cose che non abbiamo ancora detto". Una correzione e una novità: il tutoraggio in calendario non è il 6 ma **il 7 novembre**, e i docenti stanno valutando di **anticiparlo** di un paio di settimane, lasciando comunque ai gruppi il tempo di lavorare.



Figure 2.11.3: Il calendario dei progetti di gruppo come emerge dalla lezione. Le date sono quelle dette a voce; il tutoraggio del 7 novembre potrebbe essere anticipato, e Beatrice chiede che il flusso di gioco sia chiaro già entro fine ottobre. Il regolamento si consegna una settimana prima della *review*, perché i docenti possano leggerlo.

2.11.7 Che cos'è un regolamento (anche in una *escape room*)

Prima dei feedback, Tiziana fa una domanda che accende la discussione: "per esperienza, negli *escape* e negli investigativi non c'è un vero e proprio regolamento", c'è un'introduzione a come si gioca. "Io non ho mai trovato, in un investigativo, un regolamento."

I docenti cominciano a rispondere in due contemporaneamente, finché Daniele interviene ("una persona per volta, perché sennò è un casino nei feedback"; "hai ragione, Daniele"); e si premura di chiarire il tono: "ci stiamo scaldando, ma non siamo arrabbiati per la domanda. Hai fatto benissimo a farla".

Il regolamento dice che cosa puoi fare e che cosa no. Beatrice concede il primo punto: in una *escape room* il regolamento è molto più piccolo che in un gioco da tavolo. Ma delle regole ci sono sempre, anche solo un'introduzione al gioco, come negli *Hidden Games* citati dal gruppo. Il regolamento è "la cosa che dice al giocatore che cosa può fare e che cosa no". In una *escape room* fisica, per esempio, ne fanno parte: se hai bisogno di una mano puoi trovare aiuto schiacciando un pulsante o chiedendo a voce; non strappare i cavi elettrici; non rompere niente. Anche le indicazioni **logistiche** e peculiari del gioco sono regolamento: "non andare su internet"; per giocare servono carta, penna e il sito dedicato, oppure carta, penna e le dodici buste; se si è in sette ci si muove in un modo, se si è in otto in un altro; "non aprite tutte le buste subito"; prima la pagina 1, poi la pagina 2. Mettere in ordine "che cosa direte alla persona giocante per farla giocare": questo, per i docenti, è il regolamento. E scriverlo ha un effetto collaterale prezioso: "quando immaginate cosa dire alla persona giocante, iniziano a mettersi in ordine un po' di cose", a cominciare dal flusso di gioco.

Un gioco ibrido ha bisogno di un regolamento adeguato. Daniele aggiunge un punto rivolto a Tiziana: il gioco presentato dal suo gruppo "non è solo un *murder party*", ma una combinazione di più cose. Se un prodotto ibrido contiene parti che hanno bisogno di regole, allora "servirà un regolamento adeguato". Fa un esempio che non c'entra con il gruppo: un'esperienza LARP di *murder party* con degli **NPC**, persone che non sono giocatori e nemmeno attori, magari reclutate sul momento, ma che devono fare cose specifiche. Anche a loro vanno date regole e indicazioni; e probabilmente l'organizzatore avrà un regolamento per tutte

le persone che organizzano la serata. "Non soffermatevi solo all'esperienza che avete di quello che c'è là fuori": chi lo fa rischia di fare qualcosa di molto simile a ciò che esiste già, mentre il proprio gioco potrebbe avere bisogno di cose sue.

"Fare design è il contrario di fare tradizione." Daniele cita una frase che attribuisce a Ruggero ("non so se l'hai detta, Ruggero, la frase") e che sostiene "terribilmente": **fare design è il contrario di fare tradizione**. Se là fuori ci sono *murder party* senza regolamento, o con regolamenti brevi, non importa: può darsi che il vostro abbia bisogno di un regolamento più strutturato. "Fate quello che serve al gioco, e non quello che serve al mondo là fuori."

Il faro: lo devono poter giocare altre persone. Il principio che deve fare da "faro nel vostro cervello" è uno: il gioco "lo devono poter giocare altre persone che non sono voi, che non hanno parlato con voi, che non vi conoscono". Tutti i componenti della "scatola metaforica" consegnata a un gruppo di persone devono essere **comprensibili**, **organizzabili** o **facilitabili**. Vale per i giochi senza facilitatore (un *Unlock!*, un *Exit*, un *Hidden Games*, una scatola che si apre e si gioca in gruppo, ma con le indicazioni su che cosa fare) e vale per una *escape room* fisica: se la si consegna pronta a qualcuno, quel qualcuno dovrà facilitarla. Dovrà sapere come si risolvono gli enigmi, come si struttura il gioco, come si procede, quanto dura, se c'è un timer o se metterlo.

"Do not let the player see this". A conferma, Daniele racconta di aver visto qualche mese prima il **documento di *licensing*** di una *escape room* americana, solo fisica: i giocatori "vengono buttati dentro" e risolvono dei misteri. Chi compra la licenza eredita, insieme alla stanza da ricostruire, un regolamento scritto dagli autori, che i giocatori non devono vedere: in alto, "in grosso, in rosso", c'è scritto *Do not let the player see this*. Chi organizza l'*escape room* deve però leggerlo, anche perché contiene elementi di ***safety***: ci sono pulsanti, elettricità, cose da montare. "Ricordatevi il benessere del vostro gioco."

Approfondimento: il regolamento come interfaccia. La discussione riprende in forma diversa un tema della lezione sul paragame: il regolamento è parte dell'interfaccia tra chi gioca e l'ecosistema di regole, e per chi gioca le regole del gioco *coincidono* con ciò che riesce a ricavare dal regolamento e dai componenti. Nelle *escape room* commerciali questo strato è spesso "invisibile" ai giocatori perché lo porta il *game master*, ma esiste comunque, ed è scritto per qualcun altro. In termini di Schell, un gioco ha sempre un *pubblico* multiplo: chi gioca, chi spiega, chi organizza; e ognuno ha bisogno della sua porta d'ingresso.

2.11.8 Feedback a *Murder Memory*: convergere

Daniele propone di dividersi i gruppi tra docenti, ma Ruggero preferisce di no ("ho un po' di cose da dire su ogni team"), e si procede gruppo per gruppo, con tutti e tre che intervengono. Comincia Daniele.

Fare convergenza. Per come è stata presentata l'idea, il gruppo ha bisogno di "fare un po' di **convergenza**" (chi non l'ha vista può riguardare la dispensa che Daniele ha preparato su questo). Il gruppo ha fatto "tantissimo" lavoro di ricerca e sviluppo e ha tante idee, ma deve convergere "in un'idea coerente unica". Gli elementi sono molti: la persona che ha perso la memoria ("che ci avete detto che state ancora discutendo"), la polizia, il miliardario che ha invitato gli ospiti, "che è importante anche quello da sviluppare". Il suggerimento forte: nelle prossime due settimane rivedere gli appunti di ricerca e sviluppo e "cercare una quadra per far incastrare tutte queste cose".

Pochi enigmi, una direzione. Due consigli "fortissimi". Primo: piuttosto che fare tanti enigmi, concentrarsi su un paio. Secondo: piuttosto che unire tante tipologie di prodotto (una *escape room*, che però è anche un *murder party*, che però è anche un gioco da tavolo), sceglierne una sola, o ibridare solo piccole componenti. "Scegliete una direzione ben precisa", poi pulite l'idea, selezionate e **descrivete attentamente il flusso di gioco**. Soprattutto per **chi organizza** il gioco, più ancora che per chi partecipa: deve sapere come viene giocato, che cosa si gioca e quando, chi fa che cosa, se ci sono facilitatori o no. "State attenti allo *scope*, perché fare un prodotto del genere da qua a dicembre non è semplicissimo."

Semplificare non è la soluzione: scegliere sì. Daniele precisa: questo non significa che semplificare sia "meno sforzo", né che semplificare sia la soluzione. "Fare design a volte significa anche capire cosa tenere, cosa non tenere. Non è semplificare: a volte è scegliere." E *Murder Memory* "ha delle idee forti": a Daniele, per esempio, l'idea di protagonisti che sono parti di un'unica persona "non è dispiaciuta", e nemmeno quella dei poliziotti. Ma anche lì bisogna decidere "qual è il cuore del vostro prodotto".

Tre casi studio. Ruggero ha tre giochi, uno per ciascuna delle parti del progetto, e li scrive in chat "così non vi perdete i nomi".

- ***Voices in My Head***: un gioco da tavolo "semi-cooperativo, un po' come il vostro", in cui una persona interpreta il **pubblico ministero** in un processo e tutte le altre interpretano **aspetti della personalità dell'imputato**, con obiettivi diversi tra loro, a volte cooperativi e a volte competitivi. È la direzione da guardare se il gruppo vuole affinare l'indiziato con più personalità.
- ***Inhuman Conditions***[8]: "forse il gioco più bello che conosco su un **interrogatorio** poliziesco". Ha un'ambientazione molto alla *Blade Runner*, ma "non fatevi ingannare dal *setting*": è un poliziotto che cerca di capire la vera identità di chi ha davanti, con una **metafora meccanica** "molto, molto, molto bella" di che cosa una persona fa nel proprio turno, passo dopo passo.
- ***Fino a prova contraria***[9]: forse, "visto a ombrello", il più utile per capire come rendere fluido (*streamline*) tutto il processo. È un investigativo con enigmi, con un caso da risolvere passo per passo, e con un'interazione "quasi a ping pong" tra la difesa (chi gioca fa l'avvocato, e cerca di portare elementi dalla parte dell'imputato) e il pubblico ministero, che cerca di farlo condannare. "In questo ping pong si crea una frizione interessante."

I regolamenti di tutti e tre, aggiunge Ruggero, si trovano liberamente online; altrimenti ci sono dei *playthrough*. Visto che uno dei compiti è scrivere il regolamento, è utile anche guardare **come questi giochi comunicano il regolamento**.

A che cosa servono le *reference*. Ruggero chiarisce una cosa che vale per tutti i casi studio della serata.

1. "Non correte a modificare il gioco come le *reference*." Non è a questo che servono: servono "ad aprirvi la mente, o chiudervi la mente, e darvi soluzioni a problemi".
2. Quando i docenti danno una *reference*, non vogliono che il gioco del gruppo si avvicini a quella *reference*: vogliono che la si usi come spunto per fare ***problem solving*** quando ci si trova davanti a problemi importanti.
3. E servono a capire **qual è il cuore del proprio gioco**. Sapere se il proprio gioco si avvicina più a un riferimento o a un altro aiuta a chiarire "qual è la cosa che il vostro gioco deve fare meglio, tra tutte le varie sfaccettature". Per un gioco ibrido, "questo è molto importante".

Il flusso di gioco entro fine ottobre. Beatrice parla al gruppo ("ragazze, Tiziana, Angelo, Lorenzo") con franchezza: è "molto curiosa" di provare il gioco, e ciò che il gruppo ha presentato è "estremamente interessante"; ma la spaventa molto per i tempi. Per questo, senza averlo concordato con i colleghi ("mi sento di dirla qua"), chiede al gruppo di avere **entro fine ottobre** un **flusso di gioco chiaro**. Più che un regolamento vero e proprio, i docenti vogliono che il gruppo si impegni a descrivere il gioco dall'inizio alla fine a una persona che non ne sa niente e che deve capire che cosa succede in tutte le fasi. Il timore è che, avendo aperto "tante sperimentazioni, tante ibridazioni" ("che ci caricano tutti, perché è molto interessante"), il gruppo non riesca a concludere in tempo. Anche le *reference* di Ruggero sono "tanto lavoro": l'invito è a partire dal regolamento e guardare le *reference* in parallelo, "spulciandole". Ai docenti non interessa la perfezione: "sarebbe il top" un gioco finito e testabile, ma ora serve mettere le idee su carta, "perché ci serve

[8] Nella trascrizione "In Human Condition". *Inhuman Conditions* è un gioco per due persone, a tempo, in cui un investigatore interroga un sospettato per capire se è un umano o un robot.

[9] Nella trascrizione "fine prova contraria"; la ricostruzione del titolo è probabile ma non certa, e la parola che segue ("è praticamente il [...] da tavolo") non è leggibile.

un flusso di gioco che ora noi non vediamo, anche se siamo molto carichi". Le *reference* aprono tutte le porte che il gruppo ha aperto, "ma probabilmente non riuscirete davvero ad aprirle tutte entro dicembre". Meglio **scegliere una porta** per iniziare, e vedere che cosa succede.

Pensate al gioco come a qualcosa di testabile. Daniele riprende un punto di Beatrice e lo estende a tutti: i gruppi dovranno testare il gioco, e non da soli, ma **con persone che non lo conoscono**. "Più il gioco lo create semplice da capire per chi non lo conosce, più vi semplificate la vita." E chiede a tutti di non prendere i feedback "onesti e diretti" come distruttivi: a lui *Murder Memory* è piaciuto molto, il nome, il *concept*, la divisione in personalità, che gli ha fatto pensare a tantissimi giochi di ruolo. Ma senza onestà nei feedback di design si rischia di entrare in un ginepraio, e di non capire come replicare il processo fuori dal corso. E questo è ciò che gli preme di più: che si finisca o no il gioco entro dicembre ("ognuno di noi ha i suoi impegni di lavoro"), i gruppi devono poter **replicare lo stesso processo**, e meglio, anche in futuro.

Il libro di Beatrice. Manca ancora la *reference* di Beatrice, che il gruppo le ricorda. È la scansione, in PDF, di un libro intitolato ***Murder Party***. Beatrice non sa se sia la direzione giusta per il gioco, ma darà al gruppo un'impostazione per la parte investigativa, quella di gruppi di persone che indagano sulla stessa storia. Contiene una traccia di gioco completa, con una meccanica specifica che potrebbe aiutare: **ogni personaggio ha dei segreti che a un certo punto è costretto a rivelare**. "In una mezz'oretta vi leggete tutta la traccia." Lo caricherà nella cartella condivisa.

Migliorare, non stravolgere. Daniele chiude con una raccomandazione che vale per tutti: ci si rivede al tutoraggio, prima del 7 novembre; nelle prossime due settimane bisogna convergere verso un prodotto. Ma "non stravolgete i vostri prodotti per i feedback che vi diamo". I prodotti vanno migliorati: "*Murder Memory* deve rimanere *Murder Memory*, la vostra idea, la vostra visione, migliorata e più focalizzata su quello che volete davvero realizzare". Lorenzo L. ringrazia.

Approfondimento: divergere, convergere, scegliere. Il lessico di Daniele ("convergenza", "una quadra", "scegliere una porta") è quello del modello a *double diamond* del design: una fase di divergenza, in cui si aprono più strade possibile, seguita da una fase di convergenza, in cui le si chiude fino a una sola. I tre gruppi sono esattamente nel punto di passaggio; *Murder Memory* è quello che ha diverso di più. La precisazione "non è semplificare, è scegliere" è importante: un gioco convergente non è necessariamente più semplice, ma ogni sua parte serve lo stesso cuore. È ciò che Schell chiama la *lente dell'esperienza essenziale*: decidere quale esperienza si vuole creare, e chiedersi per ogni elemento se la serve.

2.11.9 Feedback a *La Peste*: obiettivo e *agency*

"Passiamo alla *Peste*." Daniele parte da ciò che è chiaro: il flusso, l'idea generale. È un gioco *sui generis* di un genere che esiste già, e "tutti noi almeno una volta abbiamo giocato a *Lupus in Tabula* o a *Werewolf*", o a una loro variante.

Le parti vaporose. "Attenzione alle parti vaporose anche per voi." Scrivere il regolamento, le procedure, che cosa succede, è importantissimo anche per questo gruppo. Daniele aggiunge due avvertenze. La prima: inserire nel gioco **le proprie emozioni**, ciò che il gruppo vuole dire, perché altrimenti si rischia di restare "veramente troppo attaccati alla *reference*"; serve la propria originalità. La seconda: attenzione al **bilanciamento** dei ruoli, a che cosa fanno nel passarsi le carte, a quante carte ci sono ("ma questo l'avete già detto"). La buona notizia: "teoricamente, con un po' di lavoro, questo è un gioco abbastanza *in scope* per la *deadline* di dicembre". Basta non perdersi e convergere bene verso le emozioni e le esperienze che si vogliono generare, "con la vostra inventiva" ("non invettiva", si corregge, "anche se la peste medievale l'invettiva un po' me la richiama").

Quattro casi studio. Ruggero ne ha quattro, e dice due righe su ciascuno.

- ***Human Punishment***: forse il più vicino a ciò che il gruppo vuole fare. L'ambientazione è diversa (è un gioco *cyberpunk*), ma meccanicamente fa qualcosa di molto simile. Ruggero precisa che non lo considera un buon gioco, "per varie ragioni che non vi dirò", ma contiene cose che possono aiutare.
- ***Trade on the Tigris***[10]: un gioco molto diverso, di negoziazione e scambi mercantili. Ha però una meccanica sulle carte che può ispirare: le carte hanno **aspetti pubblici** e **aspetti segreti** di informazione, e ogni negoziazione si porta dietro gli uni e gli altri. Può dare "un po' di spezie" al gioco.
- ***Inhuman Conditions***, di nuovo, ma per ragioni diverse da quelle del gruppo precedente: qui interessa come gestisce il ruolo degli **androidi**, quelli che non devono farsi scoprire, e come assegna loro **regole di gioco specifiche**. Secondo Ruggero quella parte arricchirebbe molto lo strato di *world building* e narrativo della *Peste*.
- ***Deception: Murder in Hong Kong***: per regolamento e flusso di gioco, "forse il *social deduction* mio preferito e più pulito", un "gioiello di pulizia" come struttura, più ancora di *Secret Hitler*, *The Resistance* e *Avalon*, gli altri derivati di *Werewolf*. Esiste anche in uno o più moduli su Tabletop Simulator, come *Inhuman Conditions* e molte delle altre *reference*.

Il motivo dell'ultimo è una domanda. L'organizzazione della *Peste* ha ancora delle aree "un po' vaporose, un po' nebulose": quanto sono nebulose perché *Lupus in Tabula* è nebuloso in molte cose, e quanto perché il gruppo non le ha ancora definite? "Il mio consiglio è: non lasciate le nebulose, andate a definirle."

Un *goal* chiaro, per ogni ruolo. Beatrice "risottolinea": in questa fase la cosa importante è cercare **un *goal* molto chiaro**. Daniele la ringrazia ("ne abbiamo discusso anche io e te") e precisa: il *goal* delle varie parti del gioco, perché ci sono la Pestilenza, i cortigiani, vari ruoli. Deve essere chiaro a chi gioca qual è l'obiettivo, perché gioca e che cosa vuole ottenere; "non solo tematico, ma anche meccanico".

"Chiaro in termini filosofici". A questo punto Gianmarco alza la mano, con un'obiezione franca: a lui l'obiettivo sembrava sufficientemente chiaro, "semplice e chiaro". I malati devono infettare tutti; i sani devono evitarlo. La risposta di Ruggero è una delle frasi chiave della serata: "È chiaro **in termini filosofici**. Ma noi non siamo te, non conosciamo il gioco turno per turno. Quindi la domanda è: **nel mio turno**, io, per perseguire il mio obiettivo, **che cosa devo fare**?". Gianmarco riformula: "quindi è collegare l'*agency* al *goal*". Esatto: bisogna connettere chiaramente la parte "a lungo termine, filosofica" con la parte "momento per momento": che cosa posso fare, che cosa faccio (figura 2.11.4). Per vedere come si fa, di nuovo, *Deception*, *Trade on the Tigris* e *Human Punishment*. E il discorso, aggiunge Ruggero, vale anche per gli altri gruppi.

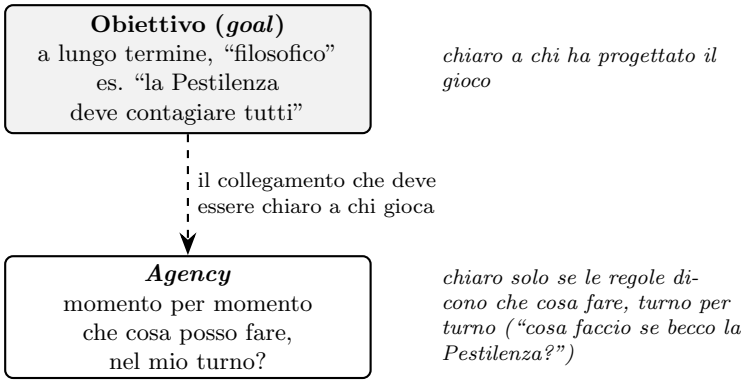


Figure 2.11.4: Obiettivo e *agency*: la distinzione nata dalla domanda di Gianmarco. Un obiettivo può essere chiarissimo "in termini filosofici" senza che chi gioca sappia che cosa fare nel proprio turno per perseguirlo.

[10] Nella trascrizione "Trident Tigris".

Daniele ringrazia Gianmarco per la domanda ("hai fatto benissimo"): è lo stesso tema del *goal*. Il gruppo ha in mente una cosa, i docenti ne hanno capito un'altra. Il *goal* finale è chiaro: la peste si deve diffondere, per chi la diffonde; il cortigiano non vorrebbe essere colpito. Ma il cortigiano, che cosa può fare e che cosa no? Qual è la sua direzione?

Registrarsi mentre si spiega il gioco. Beatrice propone, "inventandolo adesso", un esercizio che può servire anche agli altri gruppi: **registrare un file audio** in cui si spiega il gioco dall'inizio alla fine, come lo si spiegherebbe a un amico, che deve capire che cosa succede in ogni fase e che cosa fa ogni persona. "Nella vostra testa magari è chiarissimo", ma da fuori manca qualcosa, oppure si usa una parola e chi ascolta ne capisce un'altra. "A me non è arrivato chiaro che cosa faccio se becco la Pestilenza."

La parte interpretativa. Tancredi riapre il tema della variante interpretativa, su cui il gruppo ha fatto "molto ping pong" ("prima c'era, poi non c'era"). In un gioco così ha senso, come coinvolgimento? Ci sono giochi da guardare? Sì, risponde Ruggero: *Inhuman Conditions* è nella lista apposta, perché è un *social deduction* con una parte interpretativa "molto, molto, molto forte", "forse il picco" dell'interpretazione meccanizzata. Dura davvero dieci minuti, c'è un timer, e se ne possono giocare sei o sette partite di fila, anche su Tabletop Simulator. La parte interpretativa "ci può stare", ma se c'è **non deve essere nebulosa** come in *Lupus in Tabula*, dove è "un po' *free for all*", "lasciata nell'etere", mentre altri giochi la meccanizzano e la integrano molto bene. Per Ruggero è un valore aggiunto "solo se non è lì per caso": "dovete prendere delle decisioni da design: se la volete, la mettete; se non la volete, non la mettete". Se invece è una cosa che "può esserci o non esserci, in base a chi gioca e vuole", allora "non è più il vostro gioco. È il gioco di chi gioca. Io voglio il vostro gioco". (Più tardi, nella discussione aperta, Ruggero dirà al gruppo di non perderla: è una parte ibrida, "se è molto interessante, sarebbe un peccato perderla".)

Perché tre minuti. Prima di passare all'ultimo gruppo, Daniele spiega il formato del *pitch*, che a qualcuno era sembrato troppo stretto. "Non vi è arrivata una parte del *pitch*? Facevamo il *pitch* da venti minuti: vi sarebbe arrivato di più? No, saremmo stati più confusi": più tempo ci si mette, più si confondono le persone. E nell'industria raramente si ha tanto tempo. Chi ha fatto lo standista in fiera lo sa: bisogna "beccare le persone e farle sedere al tavolo in meno di tre minuti", in trenta secondi. Un *publisher*, se va bene, dedica a un gioco tre, quattro, cinque minuti. Un gioco deve poter essere presentato in poco tempo (online ci sono "tanti meme" su chi spiega le regole all'infinito), e **riassumere le regole in poco** è anche una **cartina di tornasole**: un modo per vedere se funzionano.

Approfondimento: obiettivi e azioni. La distinzione tra *goal* e *agency* ha un nome classico nella teoria del game design: Schell la tratta con le lenti degli *obiettivi* e delle *azioni*, e la sua domanda guida è proprio quella di Ruggero: le azioni che il gioco mette a disposizione permettono di perseguire l'obiettivo, e chi gioca lo capisce? Nei *social deduction* il problema è acuto, perché metà dell'informazione è nascosta: il giocatore deve sapere che cosa fare anche senza sapere chi ha davanti. È il motivo per cui Ruggero insiste su *Deception*, che dà a ogni ruolo un'azione concreta per ogni fase.

2.11.10 Feedback al *roguelike*: focalizzare

L'ultimo gruppo è quello di Rita, Giordano e Nicola. Daniele parte "dalle cose molto interessanti": un ***roguelike* di *journaling* con le carte** "è molto interessante", "apre delle cose fenomenali". Però, "e questo è proprio un consiglio: dovete fare questa cosa", **bisogna focalizzarlo**.

Cinquantasei carte non bastano per fare tutto. Con la possibilità di fare "qualsiasi cosa" con 56 carte, il gruppo sta **sottovalutando la quantità di RNG** (generazione casuale) necessaria. Daniele parla per esperienza diretta: sta creando "un *card game story-driven roguelike* da tavolo"; "è il mio lavoro nella vita", "vivo di combinazioni, di quantità di carte da mettere, di come incastrare la narrativa". E assicura che il lavoro richiesto da un gioco non focalizzato è molto più di quanto il gruppo pensi.

Che cosa vuol dire focalizzato. Vuol dire decidere **che cosa raccontare** con il gioco e **che cosa si gioca**, anche a livello di emozione. Il gruppo ha parlato di curiosità, "ma la curiosità non può essere vagamente la curiosità di giocare un'altra partita". Daniele gioca "avidamente" l'*early access* di *Slay the Spire 2*, e prova curiosità; ma è "una curiosità specifica, in un ambiente specifico, in un *setting* specifico, con un gioco specifico". Quindi: "non tenete generico questo gioco, focalizzatelo, fate un *emotion driven* molto più diretto".

***Emotion driven* non vuol dire un'emozione sola.** Vuol dire il **ventaglio di emozioni** che si vogliono creare con le meccaniche. Anche il gioco di Daniele è costruito in modo *emotion driven*, e "mi ha risolto un fracco di problemi", perché lavorare così costringe a chiedersi: che cosa voglio che il giocatore senta da queste meccaniche? E il *roguelike*, dice, è "proprio un caso studio fenomenale" sull'*emotion driven*.

Legare *journaling* e carte. Il secondo consiglio: mettere **un legame più forte tra *journaling* e *card game***. Esistono giochi in cui le due parti sono separate e funzionano; ma per questo gioco, secondo i docenti, sarebbe molto originale trovare un meccanismo per cui il gioco di carte **spinge direttamente** al *journaling*, con momenti in cui dice: adesso scrivi. È l'aspetto di ibridazione e di sperimentazione che rende interessante il progetto.

Carte personalizzate. Sull'RNG, se il gioco è focalizzato, 56 carte possono bastare, che siano carte personalizzate ("con Tabletop Simulator le potete fare") o un mazzo da poker. Il gruppo chiede se possa realizzare le carte *ad hoc*: "se focalizziamo di più, sì, assolutamente". E Daniele annuncia che dirà ancora qualcosa su *Reigns*, perché è un suo caso studio: "lo conosco come le mie tasche, so tutta la RNG di quel gioco. C'è da spaventarsi, ragazzi, su quell'RNG".

L'anti-*selling point*. Ruggero aggiunge una cosa a ciò che ha detto Daniele. Nella slide del gruppo, i *selling point* erano: l'approccio, lo strumento, la modularità, il sistema agnostico rispetto all'ambientazione. "Quello, per me, è l'**anti-*selling point***." Il vero *selling point* era il **sottotitolo**: "*deck builder, roguelike, journaling*". "Quello era il 'cavolo, 'sta roba può essere interessantissima'. Il *setting agnostic* è il ''sta cosa mi fa scappare'." I casi studio che il gruppo stesso aveva segnato (*Apothecaria*, *Psychonauts*[11], *Playlist for the End of the World*, *Reigns*, *Slay the Spire*) non sono agnostici, nessuno: per ognuno si sa che emozione vuole dare. "Voi state prendendo quei casi studio, che sono interessantissimi, togliendo le cose che per loro sono interessanti."

Il gioco che fa tutto non esiste. Focalizzando, "a livello di *vibes*" sembra di perdere qualcosa, delle possibilità; ma "è la normale convergenza": per forza di cose, convergendo, qualcosa si perde. "Il gioco che fa tutto non esiste. Se volete fare un gioco che fa tutto, non fate nessun gioco, e arrivate alla fine del corso senza niente."

Sette casi studio. Poi i casi studio, "preparatevi perché sono tanti": erano tre, poi quattro, e ora sono di più di quando ne ha parlato con i colleghi in privato. "Dobbiamo mettere una regola per limitare i casi studio", protesta Daniele; "non c'è la regola sui limiti di casi studio di Ruggero. Daniele, sei fregato" ("ho trovato un *bug*"; "è troppo tardi").

1. ***Reigns*, il gioco da tavolo**: esiste davvero[12]. Come l'hanno fatto? Che cosa fa il gioco da tavolo rispetto al videogioco? È analogico, quindi più vicino a ciò che fa il gruppo.

2. ***Cult Poker***[13]: un *roguelike* di *journaling* in solitario che usa fisicamente un mazzo di carte da poker e chiede di **scrivere sulle carte**, modificandole partita dopo partita. "Mi sembra molto, molto, molto utile al vostro gioco."

[11] Così nella trascrizione; non è chiaro se il gruppo intendesse il videogioco omonimo o un altro titolo dal nome simile.
[12] È *Reigns: The Council*, citato più avanti in chat.
[13] Così nella trascrizione; il titolo non è verificabile.

3. ***Dorfromantik*, il gioco da tavolo**[14], non il videogioco. Sembra non c'entrare niente, ma ha un componente che è una piccola **mappa** con cui il gruppo di gioco continua a interagire partita dopo partita: ci si scrive sopra, e si sbloccano cose future, con la dinamica tipica delle *run* dei *roguelike*. Lo fa in modo analogico, "molto *smooth*, molto semplice da spiegare"; ed è "un gioco da tavolo bellissimo".

4. ***Austerity***: un gioco da tavolo in solitario che esiste solo in *print and play*, gratuito, da stamparsi a casa. È un gestionale cittadino, "diciamo *roguelike*", super focalizzato sulla **gestione delle risorse**, che tematizza "molto forte".

5. ***Dead Cells*, il gioco da tavolo**[15]. Il gruppo aveva messo *Slay the Spire*, che ha anche una versione analogica; quella di *Dead Cells* ne è molto vicina: un *deck builder roguelike* con delle *run*, che fa cose molto interessanti. Ma è "al confine" di ciò che serve al gruppo: "guardatelo solo se avete la disperazione più totale nel cuore". A Ruggero è piaciuto moltissimo, ma potrebbe far deragliare il progetto.

6. ***The Quiet Year***, per i *prompt*, le tabelle, gli eventi, e per come rende "estremamente narrativo" un mazzo di carte da poker in termini di *journaling*. Esiste un'edizione italiana; Ruggero la nomina per onestà intellettuale ("mi sembra disonestà intellettuale non dire che esiste"), ma non la comprerebbe né la consiglierebbe mai, per ragioni di paragame che chi ha seguito la lezione precedente può intuire guardandola. "Ma non apriamo questo."

7. Un **documentario** di venti o trenta minuti, di cui Ruggero non dice niente: "se lo aprite, vedete i primi cinque secondi, già avete capito perché vi serve".

Daniele: provateci. Daniele riprende la parola. Primo punto: "avete visto quanto eravamo entusiasti tutti quanti" del *deck building roguelike* in solitario con *journaling*; lui vorrebbe giocarlo, perché adora i *roguelike*. Non è che l'idea non sia arrivata: è che "sappiamo per esperienza che farlo generico è problematico". Daniele ha studiato attentamente l'RNG di *Reigns*, il videogioco, "perché è fondamentale: è una pietra miliare della **narrativa combinatoria**. E la narrativa combinatoria, ragazzi, è complicatissima da fare". "Però provateci, provateci." Il consiglio è di fare carte personalizzate, di guardare bene *Cult Poker* e le altre *reference* in cui girare una carta genera eventi che **modificano le risorse**, come aveva detto Nicola nel *pitch*. E di non sottovalutare la cosa importante di *Reigns*, che "molti sottovalutano": l'***agency***. La carta non si limita a modificare le risorse: **dà una scelta**, andare da un lato o dall'altro. "Secondo me può essere fattibile"; ma bisogna stare molto attenti alla parte di universalità.

I giochi "generalisti" sono focalizzati. Daniele aggiunge un'osservazione che può sembrare strana: perché un prodotto che può fare tutto non sarebbe coerente con il mercato, con la vendita? Esistono, soprattutto tra i giochi di ruolo, i cosiddetti **generalisti**, come *GURPS* e *Fate*, che negli anni hanno avuto "il loro discreto successo". Ma attenzione a un pregiudizio dovuto al modo in cui vengono presentati: quei giochi **sono focalizzati**, solo che la loro focalizzazione non è sul genere o su ciò che si vuole narrare, ma su un'altra cosa.

- L'esempio "canonico" italiano è ***Not the End***. Si può interpretare qualsiasi personaggio, in qualsiasi scenario, ma il gioco dichiara il suo fuoco: **eroi che dicono "questa non è la fine, io non morirò qui"**, gesta eroiche di un certo tipo. Il sistema a *token* ha una sua texture narrativa e una sua struttura di regolamento, ed è proprio questo che ha venduto: qualunque cosa si giochi, ci saranno sempre dei punti cardine. Ma per il gruppo Daniele sconsiglia la "soluzione alla *Not the End*": nei mesi a disposizione c'è troppo poco tempo. Meglio focalizzare il gioco come *The Quiet Year* o *Austerity*.

- Gli autori di ***Fate*** volevano focalizzare il prodotto sul gameplay e sui tipi di narrazione possibili, tanto che i personaggi di *Fate* (terza edizione) sono **proattivi**: non sono personaggi che possono "tirarsi la zappa sui piedi". Non c'è il fallimento critico in cui "mi è caduta la spada sul piede e mi sono fatto male", e il manuale lo ripete più volte.

[14] Nella trascrizione "Dorf Romantic".
[15] Nella trascrizione "Dead Cell" e "Del Cell".

La lezione è generale: i sistemi che cercano di fare tutto rischiano di non fare niente, e i giocatori magari non capiscono nemmeno a che cosa serva il gioco. Anche i sistemi generali di successo "danno una loro direzione": "siamo generali, puoi farci quello che vuoi; ma se vuoi fare questo tipo di esperienza, ci siamo noi".

Tutto su Tabletop Simulator. Ruggero ha controllato nel frattempo: *The Quiet Year*, *Austerity* e il gioco da tavolo di *Reigns* sono tutti disponibili nel *workshop* di Tabletop Simulator.

La curiosità porta in posti confusi. Beatrice non ha niente di particolare da aggiungere, ma risottolinea, come per il gruppo precedente, la parte di ***emotional design***: "la curiosità come emozione primaria, secondo me, vi porta in posti confusi". Bisogna ragionare molto sulle emozioni da dare alla persona giocante, e su come dargliele.

Il rischio del librogame. Uno studente del gruppo ringrazia ("gli spunti che mi ha dato Ruggero sono una figata", compreso il documentario, utile "anche in futuro") e fa una domanda sulla focalizzazione: anche la narrativa deve essere più focalizzata? Il gruppo aveva paura di scivolare in una sorta di "libro troppo libro e poco *game*". La risposta: c'è "un grosso spettro", e focalizzare la narrativa non vuol dire diventare un librogame. Ruggero cita un titolo che alcuni giocano dicendo "è un librogame"[16], con cui non è d'accordo: ha componenti più guidate, ma non somiglia affatto a un librogame. Lo spettro è come quelli visti nelle lezioni precedenti; e ***The Quiet Year*** sta lì proprio per rispondere a questo dubbio: è focalizzato, narrativo, "e non è per niente un librogame". "Io credo che tutti voi dobbiate giocarlo almeno una volta", dice Ruggero: in un corso più lungo sarebbe stato obbligatorio. Daniele, che l'aveva giocato tempo prima, conferma: anche con le carte da poker "è fenomenale", "ha delle intuizioni fantastiche. Che tu dici: ma si può fare questa cosa con un gioco? Si può fare".

Approfondimento: la narrativa combinatoria. *Reigns* è un esempio classico di narrativa combinatoria: un mazzo di carte-evento, ciascuna con due scelte che spostano quattro indicatori, e una partita che finisce quando uno di essi va a zero o al massimo. La struttura è identica, nelle linee generali, a quella proposta dal gruppo (barre del corpo, della mente e dello status, morte quando una si esaurisce). La difficoltà di cui parla Daniele sta nel numero di combinazioni: con poche decine di carte, ogni carta deve avere senso dopo quasi qualsiasi altra, e ogni combinazione deve produrre una storia leggibile. È per questo che i docenti chiedono di fissare prima *che cosa* raccontare: un *setting* e un ventaglio di emozioni preciso restringono le combinazioni che contano, e rendono il lavoro gestibile.

2.11.11 I casi studio della serata

La tabella 2.11.3 riassume tutti i giochi e i materiali che i docenti hanno assegnato ai gruppi, con il motivo per cui sono stati scelti. Come ripete Ruggero, "i casi studio che vi ho dato non sono *task*, non sono obblighi": non vuole schede riassuntive né report ("sono per voi"), e vanno usati per fare *problem solving*, non come modelli.

2.11.12 Domande aperte: scrivere il gioco per chi lo gioca senza di voi

Finiti i feedback di gruppo, si passa alle domande generali. La prima novità è logistica: secondo i docenti è meglio **anticipare un tutoraggio a fine ottobre**. Ruggero non ci sarà, "però io e Bea probabilmente dovremmo farcela", dice Daniele.

[16] Nella trascrizione "Miriam Farah"; il titolo non è ricostruibile.

Gruppo	Caso studio	Perché
Murder Memory	*Voices in My Head*	un imputato interpretato da più persone, come aspetti della sua personalità; obiettivi a volte cooperativi, a volte competitivi
	Inhuman Conditions	l'interrogatorio: la metafora meccanica di che cosa si fa nel proprio turno
	Fino a prova contraria (titolo incerto)	un caso da risolvere passo per passo, con il ping pong tra difesa e accusa; come rendere fluido il processo
	Murder Party (libro, scansione di Beatrice)	una traccia completa; ogni personaggio ha segreti che a un certo punto deve rivelare
La Peste	*Human Punishment*	il più vicino meccanicamente, in un *setting cyberpunk* ("non un buon gioco", ma utile)
	Trade on the Tigris	carte con aspetti pubblici e segreti di informazione
	Inhuman Conditions	regole specifiche per i ruoli nascosti (gli androidi); la parte interpretativa meccanizzata
	Deception: Murder in Hong Kong	il *social deduction* più pulito, per regolamento e flusso di gioco
Roguelike di *journaling*	*Reigns: The Council*	come il gioco da tavolo traduce il videogioco; scelta binaria e risorse
	Cult Poker (titolo incerto)	mazzo da poker, carte su cui si scrive partita dopo partita
	Dorfromantik (gioco da tavolo)	una mappa su cui si scrive e che sblocca cose tra una partita e l'altra
	Austerity	*roguelike* in solitario, risorse fortemente tematizzate; *print and play* gratuito
	Dead Cells (gioco da tavolo)	*deck builder roguelike* a *run*, la progressione tra una *run* e l'altra; "al confine"
	The Quiet Year	*prompt*, tabelle, eventi; un mazzo da poker reso narrativo; non è un librogame
	un documentario (20–30 minuti)	non detto: "dai primi cinque secondi capite perché vi serve"

Table 2.11.3: I casi studio assegnati ai tre gruppi, con le ragioni date dai docenti. Più avanti, nella discussione aperta, si aggiungono *The Zone*, *Lovecraftesque* e altri titoli (sezione 2.11.14).

Che cosa si può già testare? Tancredi chiede del *testing*: ci sono cose a cui conviene già cominciare a pensare, dei test che si possono già fare, un modo di strutturarsi, prima della lezione dedicata? Nessuno pretenderà che i gruppi sappiano qualcosa sul *playtest* prima di quella lezione, risponde Daniele; ma "se vuoi un po' di *foreshadowing*", il consiglio più utile è quello già dato: **iniziate a ragionare sul regolamento**. "Non ve l'abbiamo detto a caso." Avere una procedura dall'inizio alla fine, messa per iscritto, aiuta moltissimo quando si va a testare il gioco.

Il test dell'amico. "Scrivete il gioco non per voi, ma per chi lo deve giocare senza di voi." Daniele propone un test mentale: immaginate di prestare il gioco a una persona amica, senza poterglielo spiegare, "per qualunque motivo". È in grado questa persona di giocarlo dall'inizio alla fine senza chiedervi niente? "Se la risposta è sì, siete già messi molto, molto bene."

Il panino con la marmellata. Per rendere l'idea, Daniele racconta "un meme bellissimo": un genitore ingegnere, che fa anche giochi da tavolo, chiede ai figli di scrivergli la procedura per preparare un **panino con la marmellata**, e poi la segue **alla lettera**[17]. A un certo punto i figli hanno scritto "mettere il coltello dentro il contenitore", e lui lo mette al contrario, dalla parte del manico, perché non c'era scritto di metterlo dalla parte della lama. "Non dico che dovete arrivare a quel livello", ma il principio è quello.

Leggere ad alta voce, pensare ad alta voce. Quando il regolamento sarà buono ("non subito, non all'inizio, ma quando vi sentite confidenti"), Daniele consiglia di farlo leggere a un amico, a qualcuno, **senza dire niente**: stare ad ascoltarlo e chiedergli di dire ad alta voce i ragionamenti che fa. "Cavolo, noterete tantissime cose che voi avete dato per scontato." È un *bias*, e ha un nome, che qualcuno scrive subito in chat ("bravissima"): la ***curse of knowledge***[18], la "maledizione della conoscenza". È il *bias* che si forma quando una persona ha addosso troppa conoscenza di qualcosa, e spiegandola dà per scontate cose che per lei lo sono, ma per chi ascolta no.

Prima ancora: mettere ordine alle proprie idee. Beatrice aggiunge che i motivi per cominciare subito a lavorare al regolamento sono due, e il primo viene "prima ancora di capire come andiamo a parlare alle persone giocanti": scrivere **mette ordine alle idee**. I gruppi sono in una fase molto embrionale, e stendere le idee dà chiarezza su quali sono i **buchi**, le **incongruenze**, le cose che mancano, prima di arrivare a capire come raccontarlo bene. "Iniziamo anche a raccontarlo male": già immaginare di raccontarlo a qualcun altro, per Beatrice, è lo *step* necessario in questa fase, e servirà poi anche per il regolamento vero. "In questa fase vi serve proprio avere il flusso di gioco, e iniziare a capire come raccontarlo. È quello che mette ordine al flusso di gioco."

Approfondimento: la maledizione della conoscenza. L'espressione *curse of knowledge* viene dall'economia comportamentale (Camerer, Loewenstein e Weber, 1989), ed è diventata famosa con un esperimento di Elizabeth Newton (1990): a una persona si chiede di tamburellare con le dita il ritmo di una canzone nota, e di stimare quante persone la riconosceranno. Chi tamburella, che "sente" la melodia nella testa, prevede che la riconoscerà circa una persona su due; in realtà la riconosce circa una persona su quaranta. Il regolamento di un gioco è lo stesso tamburellare: chi lo ha scritto "sente" il gioco che ha in testa, e non si accorge di quanto manca sulla pagina. Il test della lettura a voce alta, senza intervenire, è il modo più economico per misurare la distanza.

2.11.13 Il *pitch* visto da dentro: la vergogna delle idee

Che cosa "attizza" chi ascolta. Daniele torna sul *pitch*. Il corso non ha una lezione dedicata ("ma ripeto: se la volete, troviamo un buco"), ma c'è una cosa importantissima da imparare quando si presenta un'idea, anche se le prime volte non viene perfetta: **osservare che cosa accende le persone che ascoltano**.

[17] Nella trascrizione "il pallino alla marmellata". Il video a cui Daniele si riferisce è con ogni probabilità la *Exact Instructions Challenge*, diventata virale online, in cui un padre esegue alla lettera le istruzioni scritte dai figli per preparare un panino con burro d'arachidi e marmellata.

[18] Nella trascrizione "Course of Knowledge".

Spesso una cosa che per chi presenta è un'idea minore, data quasi per scontata, è quella per cui chi ascolta dice "wow, che fichissima questa cosa".

La vergogna delle idee. Daniele dà un nome al fenomeno opposto: la **vergogna delle idee**. La vede molto nella narrativa, e gli *editor* la conoscono bene quando ricevono *pitch* di libri: si hanno idee forti, ma non ci si crede fino in fondo, e allora le si chiude dentro un'"impalcatura" che cerca di convincere gli altri che il lavoro è stato fatto. I gruppi stessi, magari, hanno idee "fichissime" dentro il loro *pitch*, e faticano a vedere che sono le più forti; le avvolgono in un "involucro" che nasce solo dal voler fare bella figura, presentare bene, fare bene. "Invece dovete tirar fuori queste idee." Il primo esempio è già venuto dalla serata: il sottotitolo del terzo gruppo, "*deck builder roguelike journaling*", era il vero *selling point*, e stava sotto il titolo, quasi nascosto.

Non è andato tutto male. Se dopo il *pitch* si ha la sensazione che sia "andato tutto male", Daniele invita a non spaventarsi: i gruppi si sono impegnati, hanno dimostrato tutto, "siete arrivati con qualcosa". In altri corsi ci sono team che il giorno del *pitch* aprono il microfono e dicono: "ma noi in realtà non abbiamo niente".

Come vi sentite? Beatrice propone di usare il tempo rimasto, dopo le domande, per sentire come è andata per gli studenti: com'è stato il *pitch* "a livello emotivo", e come si sentono dopo i feedback. Daniele, che ha visto i *pitch* degli anni passati, è netto: "siete andati bene, e vi assicuro che se non andava bene anche solo un gruppo lo sapevate. Quindi non temete". Ammette che i docenti possono essere "un po' duri" ("questo è un problema mio, probabilmente"): ma se vedessero "come mi segno io i feedback sui miei prototipi", con "proprio gli insulti grossi verso di me", saprebbero di essere "sulla cresta del mondo".

Domande in chat. Intanto, qualcuno ha chiesto in chat i titoli esatti: il gioco da tavolo di *Reigns* è ***Reigns: The Council***; c'è *Austerity*; e per *The Quiet Year*, di cui esistono "mille moduli", basta in realtà il PDF del gioco e una **lavagna condivisa**[19]. Giordano saluta e lascia la lezione.

Lavorare con sconosciuti. Uno studente del terzo gruppo risponde alla domanda di Beatrice: è stato "molto utile e interessante" lavorare in gruppo, e confrontarsi con persone sconosciute, "anche di età mista", che hanno punti di vista molto diversi sulle stesse cose pur essendo interessate alle stesse cose. E la lezione di oggi è stata "superfiga": avere tutti e tre i docenti insieme e poter fare domande a tutti è "una cosa preziosissima". Il gruppo è felice che l'idea abbia in parte "agganciato" i docenti, e cercherà di "assorbire come delle spugne" le indicazioni. "Speriamo di farvelo giocare." "Io vorrei giocare", risponde Daniele.

La stima che sale. Daniele condivide con il gruppo "la vostra futura condizione di designer di *roguelike card game*": ci sono "delle cose toste" ("fatelo compatto, mi raccomando"), ma il genere dà "tante soddisfazioni", perché mettere insieme cose così complesse, quando ci si riesce, è incredibile. E vale anche per *Murder Memory* e per *La Peste*: quando si riesce a mettere insieme più cose e si vedono giocatori che magari non giocano esattamente come si aveva in mente, "ma il 70% delle cose sono lì", "la stima che ti sale".

Non perdete la parte interpretativa; scolpite. Ruggero riprende due punti. Alla *Peste*: non vorrebbe che il gruppo perdesse la parte interpretativa, e ha messo *Inhuman Conditions* tra i casi studio proprio per quella, "perché quella è una parte ibrida": "se è molto interessante, sarebbe un peccato perderla". A *Murder Memory*: ci sono "tante cose molto interessanti, seppure ancora un po' nebulose", e se il gruppo riesce ad affinare e a "tagliare questa scultura" può uscirne "tanta bella roba".

Parlate tra di voi. Poi una precisazione e un invito. I casi studio "non sono *task*, non sono obblighi": Ruggero non vuole schede riassuntive né report ("non sono per me quei *task*. Sono per voi"). E ora che ogni gruppo ha visto i giochi degli altri, i gruppi dovrebbero **parlare tra loro** e farsi domande a vicenda: "non siete entità separate".

[19] Segue nella trascrizione un riferimento non ricostruibile ("fai tutti l'ha fatto quello, Reins De Caos").

2.11.14 *Roguelike*, RNG e narrativa modulare

L'ultima mezz'ora diventa una discussione sul terzo progetto, che tocca temi su cui due dei docenti lavorano per mestiere: Daniele sta sviluppando un *roguelike* narrativo di carte, e fa "design di narrativa da tanti anni".

2.11.14.1 *Dead Cells* o *Slay the Spire*?

Uno studente del terzo gruppo chiede a Ruggero perché abbia consigliato il gioco da tavolo di *Dead Cells* invece di quello di *Slay the Spire*, che lui possiede e potrebbe spiegare in fretta ai compagni. Se non si è giocato nessuno dei due, risponde Ruggero, *Dead Cells* "è fatto meglio"; ma in realtà molte cose che fa *Dead Cells* le fa anche *Slay the Spire*. Ciò che interessa è la parte delle *run*, il **miglioramento *meta*** tra una *run* e l'altra ("Hai presente? Quello, esatto"), e vedere come lo fa. Ma è "un di più": *Dorfromantik* fa la stessa cosa "in uno *scope* che potete fare anche voi". Non per capacità, precisa, ma per i tempi del corso: giochi come *Dead Cells* e *Slay the Spire* da tavolo sono fuori dallo *scope* del progetto.

Due anni per un gioco da tavolo. Daniele porta un esempio di scala: il gioco da tavolo di *Cyberpunk*, che non ha aspetti *roguelike* ma ne ha di narrativi, coesi con le meccaniche, ha richiesto due anni di sviluppo; "e voi non avete due anni per questo gioco. Contenetevi, per non andare in *burnout*". Ci sarà una lezione sull'RNG, ma ogni volta che c'è casualità il lavoro è "mostruoso": la combinatoria e la modularità narrativa "sono veramente complesse". Per dubbi, soprattutto il terzo gruppo può scrivergli in privato, per mail o su Telegram ("scriveteci sempre in privato, questo vale sempre"): ha molta esperienza di questi giochi, e gli piacciono molto.

2.11.14.2 Da dove si comincia? Non dai *perk*

La domanda di Nicola. Nicola si definisce "un gigantesco fan dei *roguelike*" e della casualità, tra i primi giochi che ha amato, soprattutto nei videogiochi. Non sapendo programmare ("tuttora sono un po' una chiavica"), si costruiva le meccaniche nella testa, per capire come funzionavano, giocando per esempio a *Pixel Dungeon*, "il più basico *roguelike*", "il classicone". Per lui è uno dei generi più soddisfacenti anche per chi lo progetta: il designer stesso non sa che cosa gli uscirà durante la *run*, e si trova a "sbattere la testa contro le meccaniche" che ha costruito. Il suo blocco è sull'approccio: finora ha fatto design di *roguelike* e di RNG narrativa solo su basi già costruite. Da dove si parte? Prima la meccanica "basilarissima", "il succo, le ossa", e poi qualche carta, qualche *perk*? "Non so come approcciarmi alla cosa."

Ruggero: *The Quiet Year*. Ruggero risponde con il suo approccio ("prendilo con le pinze"): affrontare il problema da un altro punto di vista. Dei casi studio, il più utile per Nicola è ***The Quiet Year***. Daniele è d'accordo.

Daniele: si parte dalle emozioni. "Quando si fa un *roguelike*, anche videoludico, **non si parte dai *perk***." L'anno precedente un gruppo gli aveva portato un *roguelike* con i *perk*, tutta la tabella, tutto il bilanciamento: "io l'ho cassata e ho detto: voi adesso ripartite dalle emozioni che volete generare, perché i *perk* e i bilanciamenti sono l'ultima cosa". Nella modularità narrativa ci sono concetti molto complessi, e un caso studio come *The Quiet Year* li spiega meglio di quanto potrebbe fare lui. Paradossalmente, è più semplice fare questo tipo di ragionamento nei giochi analogici che nei videogiochi: i giochi procedurali come *Caves of Qud*[20] sono "molto, molto complessi".

La narrativa prima della modularità. Ciò che serve sempre, quando si vuole fare modularità narrativa, è **avere ben chiara la narrativa prima di renderla modulare**. Daniele fa un esempio "alla *Quiet Year*": un gioco combinatorio in cui, girando delle carte, si raccontano le avventure di un **sub** in un mare abitato da **creature cthulhoidi**. Le emozioni sono particolari: la scoperta, la curiosità, ma anche il terrore, il timore, e la voglia di proseguire; e l'*agency* può basarsi sul **rischio**: rischiare di più per scoprire di più, ma anche per

[20] Nella trascrizione "Caves of Could".

morire, o rischiare di meno e tornare indietro. Con una narrativa così, Daniele non parte dai pezzi e dalla modularità, "perché è difficile". Si chiede piuttosto: **se finisco una partita di esempio, che partita è?** Che tipo di esempi voglio? Sicuramente una partenza in cui il sub scende e comincia a vedere cose strane; dei momenti in cui il sovrannaturale aumenta; certe cose che vanno avanti. Solo dopo si costruiscono i pezzi (figura 2.11.5).

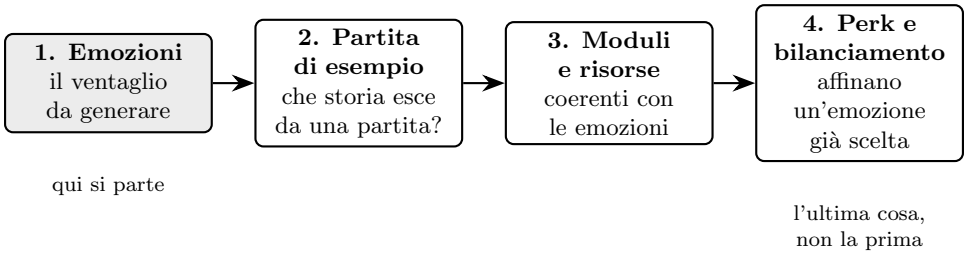


Figure 2.11.5: L'ordine di lavoro che Daniele propone per un *roguelike* narrativo. Chi parte dai *perk* rischia, dice, di progettare "la gestione delle offerte dell'Ipercoop".

The Zone. È esattamente l'approccio usato per ***The Zone***, un gioco di ruolo narrativo "fichissimo", giocabile gratuitamente online: è *Annihilation* fatto gioco di ruolo. Si gioca con i **tarocchi**, senza dadi, e ha "delle fichissime meccaniche di **frasi rituali**": si pronunciano delle frasi che attivano delle meccaniche, "favoloso". Il designer, racconta Daniele, lo spiega in alcune interviste: voleva avere un'ipotesi di come potesse essere la narrazione, basata sia sul romanzo sia sul film; aveva un'idea di come potesse essere una partita, e da lì ha costruito le modularità. In *The Zone* si sa fin dall'inizio che sopravviverà **una sola persona**: tutti i personaggi muoiono prima del finale tranne uno, e ci sono meccaniche con cui chi muore resta nella partita. Alcuni passi della narrazione sono sempre uguali, altri cambiano grazie ai tarocchi, da luogo a luogo, e tutto si basa sulla narrazione delle persone. "È bellissimo quel gioco, ragazzi!" È un gioco *sui generis*, come *The Quiet Year*. Spiegano la stessa cosa sulla modularità narrativa anche *Lovecraftesque* e *Bleak Spirit*[21], che secondo Daniele si giocano meglio senza carte.

Le risorse vengono dopo. Nel caso del terzo gruppo c'è anche una **modularità di risorse**. Il consiglio resta lo stesso: pensare a una struttura di partita di esempio, e da lì decidere **quali tipi di risorse** avere, coerenti con le emozioni da generare. Il bilanciamento si fa dopo, i *perk* si fanno dopo. "I *perk* servono ad affinarti un'emozione che hai già scelto. Se parli dei *perk* ma non dell'emozione al cuore, i *perk* rischiano di essere la gestione delle offerte dell'Ipercoop."

Le *build*. Lo stesso vale per le *build*, aggiunge Daniele, che gioca molto l'*early access* di *Slay the Spire 2* ("non è ancora bilanciato, ma è fighissimo"). Per creare le *build*, molto spesso i designer non partono dalle carte ma da che cosa vogliono che la *build* faccia, quali emozioni deve dare, che cosa deve generare; poi ne fanno una lista, e a ogni voce associano delle carte. *Perk*, elementi meccanici e il resto vengono **dopo le emozioni**. Anzi, spesso le *build meta* nascono dal basso, da chi gioca, non da chi progetta.

2.11.14.3 Le meccaniche sono metafore

Come si rompe la prima barriera? Nicola accetta che i *perk* vengano dopo, ma chiede come rompere "la prima barriera": partendo da un'emozione da trasmettere, come si arriva alla prima meccanica? Gli viene in mente di introdurre **personaggi giocabili**, come in *Slay the Spire*, ciascuno con i propri *perk*, le sue *build*, le sue carte e i suoi *set*: una scelta di tipologia, che non gli era venuta prima e che potrebbe aiutarlo a "sbattere contro la parete". La domanda è: che cosa faccio, come designer, cioè quale meccanica permette di trasmettere una narrativa, visto che "una narrativa dettata dalla casualità è un casino, a meno che non forzi con una linearità di sottofondo"?

[21] Così nella trascrizione; il secondo titolo non è verificabile.

Non un cacciavite: una metafora. La risposta di Ruggero è una delle frasi della serata: "Non pensare alle meccaniche come a un cacciavite o a un martello. **Le meccaniche sono metafore**: tutte sono metafore". Metafore di interazioni: tra persona e persona, tra persona e oggetto, tra persona e luogo. "Ragiona in termini di: come metaforizzo questa emozione?" Il caso studio girato al gruppo, "specialmente il documentario, che sembra quello messo lì a caso, ma non è a caso", fa capire al volo quante cose si possono fare con i componenti che si hanno, lavorando *emotion driven*. E poi: "non sei da solo, siete in un gruppo di tre".

Scegliere e incatenare. Nicola conviene: si voterà, si userà "il metodo di Beatrice", e ne usciranno. Basta scegliere uno, due, tre punti da cui partire; quale sia la narrativa principale è, secondo lui, abbastanza secondario: una volta scelta, il resto si incastrerà "a catena". "Essendo in tre, credo che ce la possiamo fare."

Non innamoratevi del prototipo. Un docente interviene "con un solo consiglio": "a voi non deve piacere il prototipo, deve piacere la versione finale". Spesso, lavorando a un gioco, si vorrebbe già lo stato finale; ma per il prototipo "brutto, rotto, non ancora perfetto" bisogna passarci. Il prototipo è una "piastrella" sulla strada per la versione finale; con il *playtest* i docenti daranno gli strumenti per affinarlo. "Pensate per ora alla parte grezza. Di pulirla c'è sempre tempo."

2.11.14.4 Giusto o sbagliato? Due livelli

La domanda di Nicola a Daniele. Nicola ha un'ultima domanda, rivolta a Daniele, "la persona che più si è dedicata all'RNG": "forse è stupido come pensiero", ma come si fa a sapere che una scelta è quella giusta? Iterare richiederebbe una quantità di tempo che il gruppo non ha.

La risposta di Rita. Risponde per prima Rita, compagna di gruppo: "giusto e sbagliato non esiste". Bisogna credere in una scelta fino in fondo, e allora "diventerà giusta", perché credendoci si prenderanno le scelte giuste. E poi ci sarà la lezione sul *playtest*.

Vere entrambe, su due livelli. I docenti danno ragione a tutti e due, su piani diversi. Quello che dice Nicola è molto vero **sull'RNG, e solo sull'RNG**: le tabelle, la matematica. Quello che dice Rita è vero per il resto, perché un gioco "è molto di più della sola RNG". Daniele, tornato in collegamento dopo una caduta della connessione ("ma alla nona ora va benissimo"), lo precisa: nell'RNG, cioè nella **tabella di bilanciamento delle meccaniche**, se non si decidono certe cose all'inizio dello sviluppo ci si perde in iterazioni senza senso. Ma questo non vale per lo sviluppo dell'intero gioco, in cui **l'iterazione è molto utile**. "Attenzione a non usare i criteri dell'RNG per tutto il gioco, e i criteri di tutto il gioco sull'RNG" (tabella 2.11.4).

	La tabella dell'RNG	**Il gioco nel suo insieme**
Che cos'è	tabelle, probabilità, bilanciamento delle meccaniche casuali	flusso, emozioni, regole, esperienza
Come si procede	decidere certe direttrici all'inizio; altrimenti "iterazioni che non hanno senso"	iterare: provare, testare, correggere
Chi ha ragione	Nicola ("come so che è la scelta giusta?")	Rita ("giusto e sbagliato non esiste: credeteci")
Quando	dopo, quando il resto è chiaro ("non fatelo adesso")	adesso

Table 2.11.4: I due livelli della risposta alla domanda di Nicola: i criteri dell'RNG non valgono per tutto il gioco, e viceversa.

La narrativa è l'assassino del re. Prima che la connessione cada, Daniele fa "un inciso" sulla narrativa nei giochi (mentre un Lorenzo saluta e lascia la lezione[22]). In un'esperienza ludica serve comunque il *gameplay*: la narrativa non va in secondo piano, ma "la narrativa senza un gameplay non è un gioco". La sua formula: "se il *gameplay* è il re, **la narrativa è l'assassino del re**". Oppure: è come il bassista in una band, di cui magari si pensa che non serva, "ma se lo togli lo senti che manca". Metterla "secondariamente, separatamente, successivamente" non la depotenzia. E la narrativa esiste anche dove non la si immagina: *Candy Crush* ha un team di *narrative designer*. Se ce l'ha un gioco così semplice, la narrativa è sicuramente importante; ma è secondaria rispetto al *gameplay*, e "secondario non vuol dire in ombra, non vuol dire minore".

Le risorse oltre i giochi. Mentre Daniele è disconnesso ("internet è crollato"), Ruggero ricorda a Nicola che ci sono anche risorse che non sono giochi, come il video consigliato, e che i casi studio non sono stati messi a caso: *Austerity* è molto meccanico, ma *The Quiet Year* è il più utile.

I giocatori come vantaggio. Al rientro, Daniele aggiunge una cosa che può risolvere molto del problema: **l'RNG di un gioco da tavolo è diversa da quella di un videogioco**, perché in un gioco da tavolo "avete un vantaggio in più: avete i giocatori". *The Quiet Year* lo mostra bene: quando la randomizzazione è troppo elevata, si può trasformarla in "una serie di domande e di risposte" affidate a chi gioca. Ruggero lo ferma scherzando ("non spoilerare *The Quiet Year*"), ma Daniele precisa che sta dicendo *che cosa* fa, non *come*.

L'ortogonalità. Sull'RNG, comunque, i gruppi non devono preoccuparsi: li aiuteranno i docenti al tutoraggio. Daniele promette un *workshop* che si chiama **ortogonalità**: "è complicato, ma se ve lo faccio fare io" in un quarto d'ora si trovano **le direttrici dell'RNG della tabella** e si risolvono le cose. Per ora, la priorità è capire che cosa si vuole creare a livello narrativo, anche come metafora di meccaniche; dalle meccaniche come metafore, e dalle emozioni, si può creare una narrativa. Altrimenti, "è come fare un rosone decoratissimo della finestra, ma non c'è ancora il muro".

Rimandare la paura. Un docente condivide con Nicola "la paura, il timore e il dramma" dell'RNG, "ma proprio perché lo condivido, non fatelo adesso. Fatelo dopo": magari alla fine lo si risolve con delle tabelle di probabilità trovate online. È una preoccupazione che si può rimandare. Ora bisogna concentrarsi sul **concetto di *roguelike***: guardare gli esempi, come ha detto Ruggero, e dire "un *roguelike* dovrebbe fare questo, questo e quest'altro"; e poi: noi come vogliamo farlo? Dove vogliamo andare? Che storie vogliamo raccontare? Che emozioni?

Approfondimento: ortogonalità. Il *workshop* non viene descritto, ma il termine ha un senso preciso nel design dei sistemi: due variabili sono *ortogonali* quando si possono modificare l'una indipendentemente dall'altra, e ciascuna produce un effetto distinguibile. In un mazzo di carte-evento come quello del terzo gruppo, le "direttrici" sono gli assi lungo cui le carte variano (per esempio quale barra toccano, in che verso, con quale intensità, con quale rischio). Se gli assi sono pochi e indipendenti, la tabella si può costruire in modo sistematico, coprendo le combinazioni che servono; se sono molti e intrecciati, ogni carta nuova cambia l'equilibrio di tutte le altre. Schell parla di questo con la lente dell'*eleganza*, Koster quando descrive i giochi come sistemi di pattern da imparare: in entrambi i casi, meno dimensioni ben scelte valgono più di molte dimensioni confuse.

2.11.15 Chiusura: la nona ora

"Ragazzi, abbiamo superato." Il tempo è finito, e Daniele deve staccare: "è la mia nona ora di lezione". "Vai a letto, vai a letto." Gli studenti ringraziano ("grazie, grazie davvero"); "figurati, mi siete trattenuti voi". Beatrice ha appena scansionato il libro *Murder Party* per il gruppo di *Murder Memory*, e lo metterà

[22] "Ciao Lorenzo, ok? Buona giornata a tutti, buona serata." È verosimilmente lo studente che aveva avvisato di dover andare via prima per prendere il treno.

nella cartella condivisa entro la sera stessa: “ce l’avete già da domani”. “Ciao ragazzi, buona serata. Ciao ciao.”

Punti chiave del capitolo

- Il ***pitch*** dura **tre minuti**, seguiti da qualche minuto di domande; i feedback arrivano solo dopo aver visto tutti i gruppi. Tre minuti non sono pochi: più tempo confonde, e in fiera o davanti a un *publisher* non se ne ha di più. Riassumere le regole in poco è una **cartina di tornasole**.
- Al *pitch* **bisogna “spoilerare”**: chi ascolta non è un giocatore ma, come un *publisher*, deve sapere le cose importanti.
- I tre progetti: ***Murder Memory*** (escape room investigativa da tavolo a due squadre, polizia e indiziato senza memoria, interrogatorio finale), ***La Peste*** (*social deduction* sul ballo della *Maschera della Morte Rossa*, sette rintocchi, malati che non vengono eliminati), un ***roguelike* di *journaling* in solitario** con carte e tre barre (corpo, mente, status).
- I feedback sono **indicazioni, non consigli**, dove c’è il rischio di una “patata bollente” troppo grande per il tempo del corso. Ma i giochi vanno **migliorati, non stravolti**.
- Entro il **6–7 novembre** serve un **regolamento di massima**: non impaginato, senza disegni, ma che permetta di giocare. Anche le *escape room* hanno un regolamento (che cosa si può fare e che cosa no, la logistica, la *safety*); un gioco ibrido ha bisogno del *suo*. “Fare design è il contrario di fare tradizione.”
- **Convergere**: pochi enigmi, una direzione, il flusso di gioco descritto per chi organizza. “Non è semplificare: a volte è scegliere.”
- **Obiettivo e *agency***: un *goal* chiaro “in termini filosofici” non basta; chi gioca deve sapere che cosa fare **nel proprio turno** per perseguirlo.
- Una meccanica opzionale “in base a chi gioca” rende il gioco “il gioco di chi gioca”. Le scelte di design vanno prese.
- **Focalizzare**: il gioco che fa tutto non esiste. Il *setting agnostic* è un anti-*selling point*; anche i giochi “generalisti” (*Fate*, *Not the End*) sono focalizzati su qualcosa.
- ***Emotion driven***: un ventaglio di emozioni specifiche, non una curiosità generica. Nei *roguelike* narrativi si parte dalle emozioni e da una **partita di esempio**; i *perk* e il bilanciamento vengono per ultimi. **Le meccaniche sono metafore.**
- I **casi studio** servono a fare *problem solving* e a capire il cuore del proprio gioco, non a copiare; non sono compiti.
- Scrivere il gioco **per chi lo gioca senza di voi**: il test dell’amico, il panino con la marmellata, la lettura ad alta voce contro la ***curse of knowledge***. Scrivere il regolamento serve prima di tutto a **mettere ordine alle idee**.
- La **vergogna delle idee**: le idee più forti spesso sono quelle che chi presenta nasconde. Osservare che cosa accende chi ascolta.
- **RNG e gioco intero** hanno criteri diversi: sulle tabelle bisogna decidere presto, sul resto si itera. L’RNG si rimanda; nel gioco da tavolo “avete i giocatori”. “La narrativa è l’assassino del re.”

Chapter 2.12

Prototipi, regolamenti e raccolta dei feedback: rendere il gioco giocabile da chi non siete voi

Questa è la **dodicesima lezione** del corso di **Analog Game Design**, la prima dopo il *pitch* e dopo le settimane di lavoro autonomo dei gruppi. Dopo una lezione "senza slide", si torna a una lezione "più tradizionale": ci sono delle slide, "non saranno lunghe, non sono tante", ma sono "molto poco teoriche, molto pratiche", pensate per dare ai gruppi degli strumenti "per raddrizzare un attimo il prototipo" e "portarlo efficacemente a essere testato". Il titolo della lezione è **prototipo, regolamento e raccolta feedback**. "C'è tantissimo da dire", ma il contenuto è stato "distillato il più possibile" in due o tre concetti chiave pragmatici, che i gruppi *possono* e, "idealmente", *devono* applicare ai propri prototipi.

La lezione è stata **anticipata**. In calendario, in questa data, doveva esserci la lezione di Diego sull'RNG; ma dopo il tutoraggio con Daniele e Beatrice, a cui Ruggero non ha potuto partecipare, i docenti hanno pensato che la lezione sul prototipaggio e sul *playtest* "avrebbe potuto avere senso farla prima", per dare ai gruppi gli strumenti per lavorare sul gioco non solo "come flusso di gioco" ma "già nel pratico": come si testa, come si va a vedere se quello che si è messo per iscritto funziona. È lo stesso principio che Ruggero spiegherà in chiusura: l'ordine delle lezioni si adatta allo stato dei progetti.

Le slide sono tre blocchi. Il primo è il **prototipo**: che cos'è, perché lo si fa, e il **triangolo dei prototipi** (ruolo, *look and feel*, implementazione), il modello che Ruggero usa per decidere *che cosa* testare. Il secondo è il **regolamento**: come si insegna un gioco a chi non lo conosce, perché scrivere un regolamento è difficile (la scrittura è un *medium* lineare, il gioco no), che cosa può e deve contenere. Il terzo sono i **metodi etnografici di raccolta dei feedback**: come si osserva una partita di test e come si raccolgono i pareri dopo, con i pro e i contro di ogni metodo. La regola che tiene insieme tutto è detta più volte: il gioco "deve sopravvivere all'allontanarsi da voi". Dopo la pausa, la seconda metà della lezione è dedicata ai progetti: ogni gruppo racconta che cosa è cambiato dal *pitch*, e Ruggero dà a ciascuno un'indicazione molto concreta su che cosa fare subito.

Data, formato, persone. La lezione è **online**, su Zoom ("le videolezioni", "uno Zoom ricorrente"), e dura presumibilmente circa tre ore, dalle 17 alle 20 come le altre, con una pausa di dieci minuti a metà. Si tiene con ogni probabilità **venerdì 6 novembre 2026**[1] Il docente è solo **Ruggero**; Daniele e Beatrice sono nominati più volte (il tutoraggio che hanno tenuto, le loro email, la decisione su un tutoraggio in più) ma non sono presenti. Gli studenti sono quelli dei tre gruppi del *pitch*: **Tiziana**, **Angelo** e **Lorenzo L.**

[1] La data non è mai detta. La si deduce da più indizi: nel calendario ripassato da Ruggero nella lezione su Tabletop Simulator il 6 novembre era la lezione di Diego sull'RNG e il 9 novembre quella di Ruggero su "*playtest* e metodi etnografici di raccolta feedback"; qui Ruggero dice che "oggi deve esserci in teoria Diego con l'RNG" e che la lezione sul *playtest* è stata messa prima. La lezione si chiude con "buon weekend", e il tutoraggio di cui si parla è "sabato 14 novembre", la settimana successiva. Nella lezione del *pitch* si era detto che il 6 novembre i docenti avrebbero rivisto la classe "in aula"; questa lezione, però, è su Zoom.

(*Murder Memory*); **Tancredi**, **Gianmarco** e **Lorenzo** (*La Peste*, che nel frattempo ha cambiato nome)[2]; **Rita**, **Giordano** e **Nicola** (il *roguelike* in solitario). All'inizio sono collegati Tiziana, Angelo, un Lorenzo e Tancredi; Rita, Gianmarco e un Lorenzo entrano a lezione iniziata; Giordano e Nicola non ci sono.

Avvertenze di lettura. Le stesse dei capitoli precedenti, con una novità. La trascrizione è automatica e storpia molti nomi: Ruggero è "Rugger" e "Rangri"; Tiziana è "Tatiana", "Diana", "Tiziano" e "Tessati"; il *core gameplay* è il "gore gameplay"; i test di *role* sono "test di RAW" e "test di Rolso"; i metodi etnografici diventano "metodi tecnologici", "iconografici" e "geografici"; *Magical Athlete* è "Magico Atleta" e "Magical Drake"; *Inhuman Conditions* è "in human condition". Dove la ricostruzione è sicura, o molto probabile, il capitolo usa la forma corretta e lo segnala in nota la prima volta; dove non lo è, riporta la forma della trascrizione e lo dice. La novità: in tre punti (in apertura, prima che la lezione cominci, e durante la pausa, per diversi minuti) la trascrizione contiene **frasi senza senso ripetute in ciclo** ("la situazione è questa, la migliore dal 2005", "qual è la risposta che ci può essere?"), tipiche di un riconoscimento vocale che lavora su un audio vuoto o di fondo. Non fanno parte della lezione e il capitolo le ignora. Infine, con molti studenti che intervengono, la trascrizione quasi mai segnala chi parla: il capitolo attribuisce le frasi quando il contesto lo rende chiaro (Ruggero chiama per nome chi ha risposto) e resta generico altrimenti.

Come leggere questo capitolo. Il capitolo segue l'ordine della lezione. Le slide non sono nella trascrizione: le figure ricostruiscono gli schemi che Ruggero descrive a voce (il triangolo dei prototipi, il regolamento come ecosistema, le combinazioni tra tipi di test e metodi di raccolta, le strutture degli enigmi, il calendario), e le tabelle riassumono i pro e i contro dei metodi così come emergono dal dialogo con la classe. Dove possibile, il capitolo ricostruisce anche la parte "socratica" della lezione: Ruggero fa quasi sempre prima la domanda alla classe ("pro e contro?") e poi completa le risposte. Qua e là, dei paragrafi di **approfondimento** collegano la lezione ai libri di game design di riferimento, soprattutto al capitolo sul *playtesting* di *The Art of Game Design* di Jesse Schell, e alla fonte del triangolo dei prototipi; sono segnalati come tali e non fanno parte della lezione.

Il percorso del capitolo:

1. l'attesa, le scadenze e la domanda su **quanto un gioco può cambiare dopo il *pitch*** (sezioni 2.12.1 e 2.12.2);
2. il **prototipo** e il **triangolo dei prototipi** (sezioni 2.12.3 e 2.12.4);
3. il **regolamento**: come si insegna un gioco, la scrittura lineare e il gioco come ecosistema, i blocchi, che cosa c'è dentro (sezioni da 2.12.5 a 2.12.6);
4. i **metodi etnografici di raccolta dei feedback**: durante la partita e dopo la partita, il sentimento dietro la soluzione, chi testa e dove (sezioni da 2.12.7 a 2.12.10);
5. le domande sulle slide (sezione 2.12.11);
6. dopo la pausa, il ruolo del *playtest* nel corso e lo **stato dei tre progetti** (sezioni da 2.12.12 a 2.12.15);
7. la chiusura: tutoraggi, calendario, contatti (sezione 2.12.16).

2.12.1 L'attesa: flusso di gioco, tutoraggi, scadenze

La lezione comincia, come spesso succede, con qualche minuto di attesa e di chiacchiere. Ruggero saluta chi c'è ("come va?", "bene, un po' così, ma bene dai"); qualcuno ha la connessione che va e viene, uno studente è in macchina con la videocamera spenta, un altro sta facendo merenda ("non so quanto sia bello vedermi"). Mentre aspetta gli altri, Ruggero chiede se va tutto bene e se ci sono domande prima di iniziare.

[2] Come nel capitolo precedente, "Lorenzo L." indica il Lorenzo del gruppo di *Murder Memory* e "Lorenzo" quello del gruppo di Tancredi e Gianmarco. Qui la trascrizione non li distingue mai: all'inizio Ruggero nota che c'è un Lorenzo ("credo anche Lorenzo. C'è l'altro Lorenzo"), più tardi saluta un Lorenzo appena entrato. Il capitolo dice "un Lorenzo" quando non è possibile capire quale.

Il flusso di gioco: lo vedremo oggi? La prima domanda è di uno studente del gruppo di *Murder Memory*: "noi abbiamo preparato il flusso di gioco, dobbiamo vederlo oggi?". Ruggero spiega il programma: oggi è una lezione con le slide, ma "come al solito" l'ultima parte è "la parte aperta di discussione e conversazione", e lì si potrà parlare anche dei singoli progetti. "Ma voi lo vedete il flusso di gioco? Ci date un feedback oppure no?", insiste lo studente. Ci saranno "sicuramente degli altri tutoraggi" in cui i docenti lo vedranno nel dettaglio, e "degli altri *check*" prima della *review* finale; oggi non nello specifico, ma alla fine delle slide se ne parlerà. Lo studente spiega il malinteso: si aspettava che per oggi ci fosse da consegnare il flusso di gioco, per avere un riscontro del tipo "ok, è chiaro, procedete pure"; la data era legata al tutoraggio con Daniele e Beatrice, e poi le date "si sono spostate". "Non vi preoccupate, oggi diamo un'occhiata anche a quello."

Il tutoraggio del 14 novembre. Il prossimo tutoraggio è **sabato 14 novembre**. Un Lorenzo chiede l'orario: Ruggero controlla e dice che "dovrebbe essere alle 13"; uno studente ricordava invece "10–13". "In generale, non vi preoccupate": Ruggero è flessibile, e se nessuno può nell'orario stabilito il tutoraggio si sposta, perché "l'ultima cosa che voglio è avere un tutoraggio in cui della classe c'è una sola persona o due persone". Anticipa anche una novità: con Daniele stanno discutendo di dare ai gruppi **un tutoraggio extra**; ne parlerà con Daniele e Beatrice dopo la lezione e comunicherà la data sul Discord. "È mia intenzione darvi anche un altro tutoraggio in più, comunque."

Il *playtest*: prima o dopo dicembre? Un'altra domanda, più ansiosa: "questa cosa del test è una cosa successiva a dicembre o bisogna farlo prima? Perché la vedo abbastanza dura". Il *playtest*, risponde Ruggero, "in teoria andrebbe fatto prima della *delivery* finale", quella dell'**11 dicembre**. Lo studente precisa che il gioco va consegnato **entro il 4 dicembre**: sì, "per la consegna del gioco il *playtest* noi lo chiediamo". Ma "non vi preoccupate": la lezione di oggi serve proprio a strutturare "un test che non vi faccia buttare del tempo" e che concentri "il poco tempo che avete, l'energia che avete" in test efficaci.

L'appello. Mancano molte persone. Ruggero vorrebbe "almeno una persona per gruppo" prima di iniziare e chiede se Nicola, Rita o Giordano arriveranno; nessuno lo sa ("io ho avuto contatti solo con i miei compagni di gruppo"; "ci sta: alla fine il grosso del lavoro lo fate con i vostri compagni di gruppo"). Gianmarco ha scritto che arriverà in ritardo. Ruggero tiene aperto il foglio delle presenze e chiede alla classe di avvisarlo se qualcuno entra mentre lui ha le slide a schermo intero, così da segnarlo. Intanto, dato che al tutoraggio non c'era, invita a fargli subito le domande che non hanno potuto fargli: "approfittiamone".

2.12.2 Quanto può cambiare un gioco dopo il *pitch*?

Una domanda arriva da uno studente che l'aveva già fatta durante il tutoraggio, e su cui dice di pensare "abbastanza spesso"[3]: **quanto può cambiare un gioco dopo il *pitch*?**

L'ideale è poco. "L'ideale è poco, perché il tempo che avete è poco." Nella vita professionale i progetti cambiano anche dopo il *pitch* iniziale, "anche drasticamente a volte", ma questo "implica spesso un allungare dei tempi di lavoro". E il corso non lavora con le logiche e i tempi della pubblicazione, ma con quelli dell'**imparare gli strumenti**. "In questo corso non vogliamo necessariamente giochi che siano i giochi più originali del mondo, né giochi *groundbreaking*, ma giochi che vi permettano di applicare quello che avete imparato." Anche un gioco che non vincerà lo *Spiel des Jahres*[4] va benissimo, se permette di applicare i *framework* e gli strumenti visti nelle lezioni: "è quello poi lo scopo di questo corso".

E se il *playtest* dice che qualcosa non funziona? Lo studente insiste: c'è tutta la parte in cui si testa "tantissimo", il processo di iterazione è lungo, ed è plausibile che le prove dicano "questo non funziona" e costringano a rivedere dei pezzi; quanto può influire sul prodotto finale? Ruggero rimanda in parte alle slide ("oggi un po' di risposte a questa domanda in teoria le avrai"), ma anticipa il punto: **le iterazioni dopo un**

[3] Ruggero si rivolge ad Angelo appena prima ("Angelo, visto che ti sei smutato"), ed è probabile che la domanda sia sua; la trascrizione non lo rende certo.

[4] Nella trascrizione "lo Spiel.dear": è lo *Spiel des Jahres*, il più noto premio tedesco per i giochi da tavolo.

playtest* di solito non sono drastiche sul *core gameplay. Sono interventi sul **modello matematico**, sull'aggiungere o togliere una regola, sul chiarire qualcosa nel regolamento. Non sono trasformazioni del tipo "da gioco di strategia a *card game* di dieci minuti". "Lo scopo del test è prendere il *core gameplay* che già c'è e limarlo, affinarlo, finché non diventi il più pulito possibile." E se un test spinge a dire "prendiamo questo gioco di tre ore e facciamolo diventare un *card game* da dieci minuti", allora "qualcosa è andato storto, molto, molto, molto, ben prima di quel test".

Non è il gioco della vostra vita. Ruggero chiude con un tema che tornerà più volte nella serata: "non creerete il gioco della vostra vita in questo corso", e anzi lo spera, "perché vuol dire che fuori da qua creerete i giochi migliori". Lo scopo è mettere "le mani in pasta" con gli strumenti, per poi usarli su giochi al di fuori del corso. Quindi anche se un gruppo lavora a un gioco che non è del suo genere preferito, non ha il suo *setting* preferito o usa meccaniche che non sceglierebbe nel tempo libero, "se questo vi permette di applicare gli strumenti del *playtest* e di iterazione, per noi va bene".

Poi, visto che nessun altro è ancora entrato, Ruggero condivide lo schermo ("vedete se si vede?") e comincia.

2.12.3 Che cos'è un prototipo, e perché lo facciamo

La prima slide parte da una domanda "che sembra banale": **che cos'è un prototipo, a che cosa serve, e perché quando facciamo giochi facciamo prototipi?**

Per testare. La prima risposta della classe: serve "proprio per testarlo, nel senso che devi vedere se funziona così come avevi pensato tu". Sì, incalza Ruggero: ma perché facciamo un prototipo e non siamo già al gioco completo? "Perché penso che sia più economico fare un prototipo che non il gioco vero."

Più economico, ma non solo in soldi. È più economico, certo: "stampare un prototipo con la stampante di casa costa meno che portarlo in tipografia". Ma c'è dell'altro. Il prototipo è anche **più facile da modificare**, e non solo per ragioni pratiche: c'è "un *trigger* psicologico, che è l'attaccarsi alle cose per cui spendiamo". Se abbiamo speso mille euro per un prototipo fatto benissimo, "sarà molto difficile che ci venga voglia di cambiarlo" e di spendere di nuovo quei soldi; se lo abbiamo fatto "sulla carta straccia", cambiare tutto costa poco, e non c'è "il muro psicologico del costo del cambiamento", che altrimenti "è gigantesco". L'ideale è quindi un prototipo **facile da iterare, da cambiare, da modificare**, "che possiamo prendere e buttare in pochissimo senza sentirci in colpa", senza il freno del "ci costa troppo farlo".

Più veloce. Il secondo vantaggio è il **tempo**. Un gioco completo richiede tantissimo tempo; un prototipo è "molto ridotto in scala". E visto che il tempo dei gruppi "già è pochissimo", va concentrato in un prototipo "asciutto, ma che contenga il necessario". Questo obbliga a una scelta: **che cosa metto a testare e che cosa no**. È il tema della slide successiva.

Approfondimento: il prototipo risponde a una domanda. Le due ragioni di Ruggero (il costo del cambiamento e la scala ridotta) corrispondono quasi alla lettera a due dei consigli di prototipazione che Jesse Schell elenca nel capitolo sull'iterazione di *The Art of Game Design*: "Forget Quality" (un prototipo brutto ma veloce vale più di uno bello e lento) e "Don't Get Too Attached" (il prototipo è fatto per essere buttato). Il primo consiglio della stessa lista, "Answer a Question", è il ponte verso il triangolo: **ogni prototipo dovrebbe rispondere a una domanda precisa**, e decidere quale domanda si sta facendo è il primo passo per non sprecare un test. Lo stesso Schell, nel capitolo sul *playtesting*, descrive il *playtest* come "un prototipo dell'esperienza di gioco", e ripete il principio: senza obiettivi specifici, un test rischia di essere tempo perso.

2.12.4 Il triangolo dei prototipi: ruolo, *look and feel*, implementazione

"Ci sono mille *framework* di prototipaggio e di *playtest*." Ruggero presenta quello che usa lui, "con cui mi trovo bene" e che secondo lui può essere utile anche ai gruppi: il **triangolo dei *playtest***. Viene da un *paper* che parlava di **elettronica**: era usato nella progettazione di cellulari, portatili, strumenti elettronici, ma "è applicabilissimo anche al gioco"[5]. Ai tre vertici del triangolo ci sono **implementazione**, **ruolo** e ***look and feel***; quando prototipiamo qualcosa, lo facciamo chiedendoci *che cosa* stiamo testando, cioè verso quale vertice ci stiamo muovendo (figura 2.12.1).

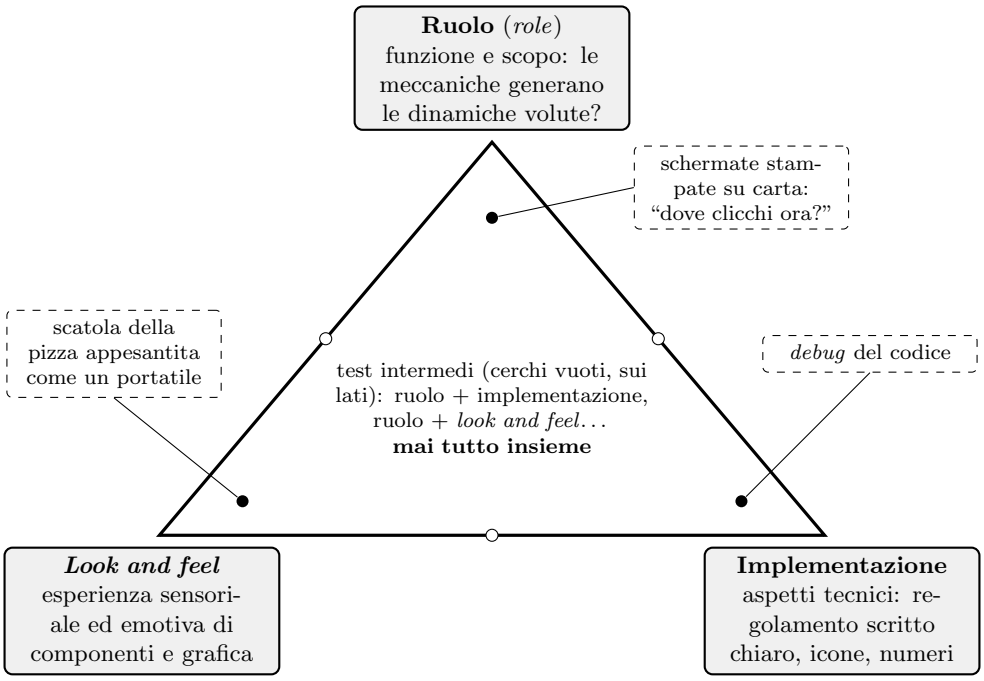


Figure 2.12.1: Il triangolo dei prototipi come lo presenta Ruggero, con i tre esempi "quasi puri" del *paper* da cui viene (pallini neri, vicini ai vertici). I test intermedi sono possibili; un test che cerca di stare al centro, testando tutto insieme, "fa tutto" e finisce "malissimo".

2.12.4.1 I test di ruolo

I **test di ruolo** (*role*) hanno lo scopo di analizzare **la funzione e lo scopo** di un gioco. La domanda è: **le meccaniche che abbiamo creato generano le dinamiche desiderate?** "Vi ricordate l'MDA, il DPE?" Abbiamo un'esperienza in mente: le meccaniche che abbiamo messo per iscritto creano quell'esperienza, o ne creano un'altra? Ruggero elenca le domande tipiche:

- **Le regole, per come sono scritte, ostacolano l'esperienza di chi gioca?** Nella nostra idea il gioco è semplicissimo, da dieci minuti; appena qualcuno lo prova ci mette quaranta minuti, cinquanta, un'ora. "Non perché sia complesso, ma perché magari le regole sono difficili da capire. Quello è un problema."
- **L'esperienza che arriva a chi partecipa è funzionale agli scopi del gioco?** È particolarmente vero per i **giochi didattici e formativi**, che hanno uno scopo al di là dell'intrattenimento: se voglio

[5] Ruggero non cita autori e titolo. Il modello è quasi certamente quello di Stephanie Houde e Charles Hill, *What Do Prototypes Prototype?* (1997), scritto quando gli autori lavoravano in Apple, nel quale ogni prototipo si colloca in un triangolo i cui vertici sono *role*, *look and feel* e *implementation*; gli esempi citati nella lezione (un portatile Apple simulato da una scatola appesantita, schermate di un'applicazione stampate su carta, codice da mettere a punto) sono compatibili con quel testo.

far passare una conoscenza, un concetto, quel concetto arriva o no? "Quella parte lì è importante testarla, e rientra nei test di ruolo." Lo stesso vale per le emozioni: vogliamo un gioco adrenalinico, frenetico, che crei ansia, un *horror*, e quello che arriva è "un gioco super rilassato, *chill*", che la gente gioca "col tè caldo in mano". "C'è qualcosa che non quadra." Magari è bellissimo, "ma non è quello che vogliamo fare".

- **Per quali persone il gioco risulta familiare, e per quali risulta alienante?** Se facciamo un gioco complesso, ma che non rientra in ciò che si aspetta chi gioca agli *eurogame* pesanti, "non stiamo parlando col *target* ideale". Il gioco magari è simile per peso, complicazione e complessità ad altri giochi di quel tipo, ma se chi lo gioca "non riesce a riconoscerlo, quel gioco lì non arriva".

I test di ruolo sono quelli che "culturalmente", se pensiamo a un *playtest*, vengono in mente per primi: il test che dice "il gioco funziona, il gioco fa quello che deve fare". Ma sono solo una delle tre direzioni possibili.

2.12.4.2 I test di *look and feel*

I test di ***look and feel*** riguardano **l'esperienza sensoriale ed emotiva** che il gioco trasmette non solo con le meccaniche, ma con i **componenti**, le **illustrazioni**, la **grafica**, il **paragame**. Che cosa si prova toccando i componenti, guardando il gioco, osservando il tabellone? Che *mood*, che *vibes* generano le illustrazioni, le luci, le forme, i materiali (legno, plastica, cartone)? "Che tipo di gioco si percepisce?"

Anche il regolamento ha un *look and feel*. I gruppi dovranno avere un regolamento, e anche il regolamento "come oggetto" comunica qualcosa: quante pagine ha? "Avete tenuto in mano dei regolamenti? Sapete quando un regolamento è pesante o no, anche solo guardandolo?" Se sto facendo un *card game* da dieci minuti, semplicissimo, e il manuale è di quaranta pagine, "questo è un problema". Viceversa, se sto facendo "un gioco da tre ore economico azionario a tema ferroviario" e il regolamento è "un piccolo *brochurino*", viene il dubbio che non sia stato scritto tutto, o che non sia stato scritto benissimo, che "salti qualcosa, che dia per scontato qualcosa". Il consiglio: **se compraste il vostro gioco, che regolamento vi aspettereste di trovarci dentro?** Lo stesso vale per i componenti: quanto sono grandi le carte, quante sono, i dadi, il tabellone, le plance. I test di *look and feel* analizzano "il *feeling* e le *vibes* date dall'aspetto estetico del gioco", dove estetico non vuol dire solo immagini, ma "proprio percezione sensoriale".

2.12.4.3 I test di implementazione

L'ultima direzione è l'**implementazione**: "questa è quella proprio noiosa, ma non meno importante". Riguarda gli **aspetti tecnici e meccanici** del gioco:

- il **regolamento**, non come numero di pagine ma come **struttura**: il flusso del regolamento, come si imparano le regole leggendolo, in che ordine sono spiegate le cose, come sono spiegate le meccaniche, se c'è un riassunto;
- come è spiegato il **modello matematico**, i numeri;
- le **icone** del gioco;
- gli **automatismi**, il ***bookkeeping*** (la contabilità che i giocatori devono tenere), il ***downtime*** di turno;
- in generale, tutto ciò che riguarda "l'implementazione tecnica delle nostre meccaniche".

Sono test che non si chiedono se una meccanica dà un sentimento X o Y, ma **se la meccanica è scritta giusta, se è scritta chiara**: "la gente capisce quello che sta leggendo?"

2.12.4.4 Un test alla volta, e nell'ordine giusto

Non testate tutto insieme. "Noi non possiamo fare test che testino tutte queste tre cose insieme, perché se no impazziamo." Un test che sia contemporaneamente di *look and feel*, di implementazione e di ruolo è "buttare via tempo": "siamo esseri umani", non riusciamo a tenere d'occhio tutto in contemporanea, e "cercando di fare tutto ci sfugge tutto". Il consiglio: **focalizzate i vostri test**.

La fase decide la priorità. In base alla fase in cui si è, alcuni test hanno la priorità. Se non si ha ancora chiaro il gioco e il suo flusso, "è inutile" testare il *look and feel* o l'implementazione, se il regolamento ancora non è scritto: si parte dai **test di ruolo**. Viceversa, se il lavoro sulle meccaniche è stato fatto e i test di ruolo dicono che il gioco fa quello che deve fare, a quel punto interessano magari solo i **test di implementazione**: il gioco funziona, ma il regolamento è scritto bene? Fa quello che deve fare? E i componenti?

Tre esempi quasi puri. Per rendere l'idea, Ruggero mostra tre esempi dal *paper*, "esempi elettronici che si capiscono al volo":

- **L'esempio 10, un test di *look and feel***: una **scatola della pizza** fatta come un vecchio portatile, "nel '92". Apple voleva testare i suoi portatili in termini di peso, dimensione, di come stanno negli zaini e di come si trasportano: la scatola era appesantita fino a pesare quanto il portatile, e si testava com'era portarsela in spalla, tenerla in mano, tenerla su un tavolo. "È chiaro che quello non è un computer": dentro non c'è elettronica, né software, né hardware. È un test su come l'oggetto viene percepito: peso, maneggevolezza, dimensione.

- **L'esempio 6, un test di ruolo**: un test per un software, un'**app a ipertesto**, con più schermate e menu[6]. L'app non esisteva, non era ancora programmata: avevano stampato un foglio per ogni schermata e chiedevano alle persone "ok, dove clicchi ora?"; a ogni "*tap*" cambiavano il foglio davanti alla persona. Non c'era codice, c'erano fogli stampati con dei *wireframe*. Ma già così si vedeva quanti *tap* servivano per passare da una schermata all'altra, se i menu erano intuitivi, se la pagina era leggibile. "Era un test di ruolo, non era un test di implementazione."

- **L'esempio 12, un test di implementazione**: il ***debug* del codice**. Il codice ha errori? Fa quello che deve fare? "Test di implementazione puro, praticamente."

Sono esempi facili da capire perché "quasi puri", cioè quasi sui vertici del triangolo. Ma dal triangolo si possono fare anche **test intermedi**: "non fate un test che fa tutto, perché finite malissimo", ma test di ruolo e implementazione, o di ruolo e *look and feel*, "secondo me potete già farli".

Approfondimento: il triangolo e le domande di Schell. Il triangolo è un modo per rispondere alla prima delle sei domande che Schell pone a ogni *playtest* (*why, who, when, where, what, how*): **perché** sto facendo questo test? Schell osserva che "il gioco è divertente?" non basta come domanda, e che le domande vanno rese il più specifiche possibile ("i giocatori capiscono come si gioca?", "ci sono strategie dominanti?", "ci sono momenti di noia, di confusione, di frustrazione?"). Il triangolo aiuta a raggruppare queste domande: quelle sull'esperienza e sulle dinamiche puntano al ruolo, quelle sulla percezione dei materiali al *look and feel*, quelle sulla chiarezza e sulla correttezza all'implementazione. E suggerisce, come Schell, che se le domande non possono trovare risposta in un solo test, conviene fare più "mini-test" separati, ciascuno con una domanda sola.

2.12.5 Il regolamento: come si insegna un gioco?

Su questa parte Ruggero dice di aver speso "un po' più di parole", perché i giochi dei gruppi sono **giochi analogici**, e i giochi analogici, rispetto ad altri tipi di giochi, "di solito usano i regolamenti per spiegare il gioco". Il punto è uno solo, e la lezione lo ripeterà in molte forme: **il vostro gioco deve essere capito, giocato, organizzato, facilitato da persone che non siete voi**. "Deve sopravvivere all'allontanarsi da voi, il gioco. Perché se il vostro gioco funziona solo con voi, quel gioco potenzialmente non fa una bella fine."

L'asterisco: LARP e giochi pervasivi. C'è "un grosso asterisco", con "grossissime eccezioni": tutto l'ambiente dei **LARP** e dei **giochi pervasivi**, che spesso sono **giochi a evento**, facilitati da chi li organizza. Ma dai *pitch* Ruggero ha capito che i gruppi sono orientati verso giochi che possono essere giocati da persone che non sono loro, e quindi, nel loro caso, il regolamento "è direi obbligatorio".

[6] Nella trascrizione l'esempio si chiama "Nord Navigator"; il nome non è ricostruibile con certezza.

La domanda alla classe. "Come si insegna un gioco? Come faccio io a spiegare a un'altra persona come si gioca un gioco?", esclusa la risposta ovvia, il regolamento. Rispondono in tre, in ordine ("Tiziana, poi Angelo"; "no, fai prima Angelo").

- **Angelo**: "lo si accompagna passo passo", "con una lunga spiegazione dei vari passaggi". "Lunga tra parentesi", ribatte Ruggero: lunga quanto? "C'è un lungo che è automaticamente corretto? Quando cerco di spiegarlo io, di solito lungo vuol dire troppo."

- **Un Lorenzo**: "innanzitutto deve averlo provato, il gioco". Ma qui c'è un problema, osserva Ruggero: **un videogioco contiene al suo interno le regole, automatizzate; il gioco da tavolo no**. "Io provare un gioco non posso farlo se non so come si gioca." "Tu puoi anche dirmi: prova a giocare a scacchi, ti spiego le regole dopo; ma mentre gioco io non so le regole." Ci viene chiesto qualcosa per cui ci manca la conoscenza che sta in mezzo.

- **Tiziana** racconta la sua esperienza: da quando è arrivata a Roma, le sono stati spiegati "tanti giochi" da varie persone, e "ci sono vari modi di spiegare il gioco", alcuni più efficaci, altri meno, anche in base al gioco. C'è chi è un giocatore e ti spiega l'essenziale, ti dice le regole e poi "gioca"; c'è chi cerca di farti entrare dentro il gioco e ti porta anche altri elementi; e c'è **chi ti facilita il gioco**, "ed è l'esperienza migliore in assoluto". Il suo ordine: prima **l'obiettivo** del gioco ("che cosa arrivo a fare con questo gioco"); poi una **panoramica generale**, mostrando che cosa c'è nel gioco; poi il dettaglio, un po' alla volta, le meccaniche, le azioni che si possono fare, come si ottengono punti. E la facilitazione "è come stendere un tappetino rosso davanti a chi inizia a giocare per la prima volta": che cosa farei io, che cosa non farei; "tu che giochi per la prima volta tieni conto di questo, questo e questo"; "come prima mossa potresti fare questa cosa". Perché spesso qualcuno spiega un gioco anche complesso e poi dice "boh, vai!", e chi gioca per la prima volta non sa che cosa fare: "finché io non lo sperimento, non ce l'ho effettivamente in mano". Essere accompagnati la prima volta è un'esperienza "molto più felice" che essere "buttati in pasto ai pesci", cioè agli avversari.

Regole ed euristiche. Ruggero raccoglie la distinzione implicita nella risposta di Tiziana: le **regole** si possono spiegare, ma la parte di **euristica**, "tutta quella parte di che cosa ti conviene fare quando", è difficile da capire se non si è già giocato quel gioco, o **giochi simili**: a volte le euristiche si imparano da altri giochi. "Se io gioco magari dieci *worker placement*, all'undicesimo *worker placement* alcune cose le avrò già capite dagli altri", con le differenze del caso. Ed è un'informazione utile per chi scrive il regolamento: chi ha già giocato giochi simili porta con sé una parte delle euristiche, chi non li ha mai giocati no.

Nessuno conosce il gioco. Poi un passo indietro. La facilitazione presuppone che qualcuno conosca il gioco; ma supponiamo che nessuno lo conosca: lo abbiamo comprato a una fiera o in un negozio, lo portiamo a casa, siamo in gruppo, apriamo la scatola. Come lo impariamo? "Leggendo il regolamento." Certo; ma il gioco "non è mai solo il gioco".

2.12.5.1 Pre-gioco, post-gioco, fiction, intenti

Il pre-gioco. Nel regolamento ci sono cose che vengono **prima del gioco**: il controllo dei componenti, la preparazione, il ***setup***. "Mentre facciamo un *setup*, non stiamo ancora giocando, stiamo preparando la nostra partita." E quella parte va spiegata. Ruggero chiede di tenere presenti le prossime slide quando i gruppi scriveranno i loro regolamenti, che **devono essere comprensibili per chi non è voi**. Questo vuol dire includere anche cose che per chi ha fatto il gioco sono ovvie: come si prepara il gioco, che componenti servono, che cosa bisogna procurarsi. Un esempio: in un gioco di ruolo spesso non c'è una scatola, c'è un manuale, e **dadi e matite non sono inclusi quasi mai**: il gioco chiede di procurarseli e di averli a portata di mano, e il manuale deve scriverlo, "non devo darlo per scontato". Oppure: se un gioco pervasivo richiede uno specifico ambiente, bisogna dirlo. L'anno scorso un progetto era un gioco pervasivo da giocare **in una stazione**, mentre si aspettano i treni, e "l'aspettare i treni è parte del gioco": questo è stato scritto.

Il post-gioco. Il gioco non finisce con l'ultima azione: "c'è tutta una fase dopo, che è comunque parte della procedura". Il **punteggio**, **chi vince**, lo **spareggio**. E se il gioco tocca temi delicati e si vuole un ***debriefing***, come si è visto nella lezione sull'*emotional design*, anche il *debriefing* è parte del post-gioco, e va scritto.

La fiction. Ma il gioco non è solo "pre, durante, post" in termini meccanici. C'è una parte di **fiction**, di *world building*, di *setting*: chi siete, che cosa fate in questo universo narrativo. Non vale solo per i giochi di ruolo, dove è molto presente: anche in *Monopoly* abbiamo un ruolo, in *Risiko* abbiamo un ruolo, in *Ticket to Ride* c'è un *setting*. E questa parte **può aiutare a spiegare il gioco**, specialmente se le meccaniche sono coese con il tema: se le azioni del mio turno sono tematicamente legate a chi sono, ricordarmele è più facile. "Stiamo giocando una squadra di ladri e devo scappare dalla polizia: le mie meccaniche mi fanno scappare dalla polizia, il tema mi ricorda che devo scappare dalla polizia, le due cose lavorano insieme."[7]

Gli intenti. "C'è un altro problema, però." Dove sono, nel regolamento, gli **intenti**? Che cosa volete da questo gioco, non solo in termini di obiettivo (come si vince e come si perde), ma in termini di **perché il vostro gioco esiste**, "perché qualcuno dovrebbe giocare il vostro gioco e non un altro". Uno studente chiede se si intendono gli intenti del *game designer* o del giocatore: gli intenti **di chi crea il gioco**, gli intenti del design. È una cosa "molto più comune nel gioco di ruolo che nel gioco da tavolo", ma anche alcuni giochi da tavolo hanno intenti molto chiari.

2.12.5.2 La scrittura è lineare, il gioco no

Messo tutto insieme (materiali, ruoli, paragame, pre-gioco, post-gioco, fiction, intenti), "notiamo subito che c'è un gran casino". Ed è un casino che deriva da una verità "a cui non potete scappare in nessun modo": **la scrittura è un *medium* lineare**. Quando leggiamo un regolamento, andiamo dall'inizio alla fine. Ma **i giochi non sono un *medium* lineare**: sono **un ecosistema di regole**, e in quanto ecosistema una meccanica ne influenza un'altra. "Ma io queste cose devo spiegarle a partire da un punto. Devo iniziare da qualche parte" (figura 2.12.2).

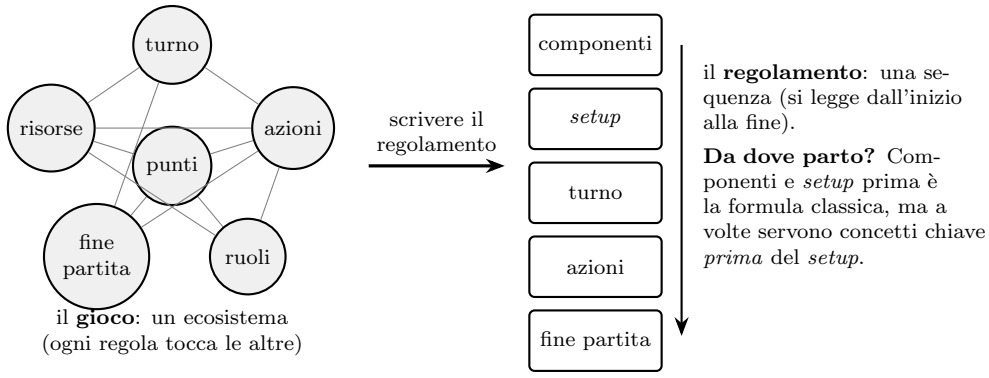


Figure 2.12.2: Il problema centrale della scrittura di un regolamento come lo pone Ruggero: il gioco è un ecosistema di regole che si influenzano a vicenda, ma la scrittura è lineare. Scegliere l'ordine in cui spiegare le cose (e verificarlo) è parte dei test di implementazione.

Che cosa spiego per primo. Parte dei **test di implementazione** è proprio capire **che cosa spiego per primo, che cosa metto davanti**. Una delle formule più classiche del gioco da tavolo è: prima i **componenti**, così le persone prendono familiarità con ciò che c'è nella scatola; poi il ***setup***, così vedono com'è il tavolo; poi il dettaglio (la plancia personale, il turno, il flusso di gioco). "Ma a volte non è così." A volte ci sono **concetti chiave prima del *setup***, perché ci sono cose che bisogna sapere prima ancora di preparare il gioco. "Non esiste una soluzione univoca, purtroppo, o per fortuna", a questo problema: i

[7] Nella trascrizione "una squadra di yard"; il senso (una banda in fuga dalla polizia) è chiaro, la parola esatta no.

gruppi devono ragionarci sopra. Nel gioco che stanno scrivendo, dovendo spiegare *tutto* ("non è che possiamo lasciare cose fuori"), da dove partono, che cosa spiegano e con quali metodi?

Panoramica e rimandi. Tiziana aggiunge che spesso nei regolamenti c'è una **panoramica generale** che poi rimanda allo specifico approfondimento, se lo si vuole, oppure lo si raggiunge leggendo linearmente. Ruggero conferma: molti regolamenti usano l'artificio del rimando ("vedi pagina 18 per il dettaglio del combattimento"), oppure un **indice**, magari all'inizio. Il problema che risolvono è un dubbio tipico: leggendo un regolamento per la prima volta, "questa cosa com'è? Come funziona?", senza sapere che verrà spiegata due pagine dopo. "È parte dei test di implementazione andare a scovare questi dubbi e spostare il regolamento in accordo."

2.12.5.3 Indici e blocchi: tre esempi

"Facciamo un po' di esempi." Ruggero mostra alcune pagine di regolamenti, colorate per blocchi (le immagini non sono nella trascrizione).

L'indice di un gioco di ruolo per due persone. Il primo è l'indice di un gioco di ruolo per due persone (il titolo non è detto), diviso in quattro tipi di **blocchi** (tabella 2.12.1): blocchi **generali** (introduzione, esempi di partita, obiettivi generali), blocchi **specifici** (le singole meccaniche), blocchi **personali** (le intenzioni dell'autore, fonti, riferimenti, ispirazioni) e **materiali di gioco** (per esempio schede stampabili, o il riassunto delle icone). Due precisazioni. Il **riassunto delle icone non è una regola**: è un materiale utile durante il gioco. La **premessa narrativa non è una regola**: è un aspetto generale che aiuta a contestualizzare ciò che si sta per fare. E "non c'è un ordine preciso" di questi blocchi.

Blocco	Che cosa contiene	Note
Generali	introduzione, premessa narrativa, glossario, tipo di gioco, esempi di partita, obiettivi generali	contestualizzano; la premessa non è una regola
Specifici	le singole meccaniche, il turno, il flusso di gioco	il cuore "tecnico" del regolamento
Personali	intenzioni dell'autore, note, fonti, riferimenti, ispirazioni, filmografie	"non nascondetele, rendetele trasparenti"
Materiali di gioco	schede stampabili, riassunto delle icone, schermi, risorse, tabelle di supporto	non sono regole: sono strumenti da usare durante il gioco
Indice	che cosa c'è e dove	comune nei GdR e nei giochi pesanti; assente nei *card game*

Table 2.12.1: I tipi di blocchi in cui Ruggero divide i regolamenti di esempio. L'ordine dei blocchi non è fisso, e cambiarlo è una delle cose che si possono testare.

Fiasco. Il secondo esempio ha una parte iniziale di introduzione, con un **glossario** e una piccola presentazione del tipo di gioco; poi il dettaglio meccanico; poi i **materiali**. ***Fiasco*** (il gioco di ruolo di Jason Morningstar) è un gioco che ha "molto peso sui materiali": la parte dei materiali non solo è molto grande, ma **non è alla fine del regolamento, è a metà**, perché *Fiasco* deve essere giocato con quei materiali specifici, e il gioco spende molte pagine per spiegarli. Poi c'è una fase di ***replay***, "se vogliamo un *actual play*", cioè una partita completa riassunta, cosa che nel gioco da tavolo "non vedremo mai". Poi le **note personali** dell'autore, le filmografie di riferimento, le ispirazioni. Poi di nuovo materiali di supporto, e infine un **indice**, "un aspetto generale di che cosa c'è nel regolamento". Non tutti i giochi hanno un indice: è più facile trovarlo nei giochi di ruolo, perché i manuali possono avere centinaia di pagine, e nei giochi complessi come *Root*, nei giochi della GMT, nei giochi strategici molto pesanti. "Non troverete mai un indice in un *card game*": "se comprate *Uno*, *Uno* non ha un indice in nessun regolamento"; "è già tanto se c'è un *brochurino* di una pagina".

Un gioco *cyberpunk*: spostare i blocchi. Più avanti Ruggero mostra un terzo esempio, da un gioco *cyberpunk*[8]: alcune parti del regolamento sono state **riallocate**, spiegate prima o dopo altre, in base a ciò che si era visto nei *pre-test*. I blocchi "li vedete anche a occhio nudo da lontano": strizzando gli occhi davanti alle pagine, si vede al volo dove sono il blocco rosso, il verde, il giallo, il viola. E sono in quell'ordine non perché sia "l'ordine naturale", ma per il problema dell'ecosistema: ci sono sei blocchi da spiegare "in parallelo", ma bisogna partire da uno. "Che cosa succede se sposto l'ordine di questi blocchi? Questa è un'altra cosa che si può testare."

Ragionate per blocchi. Il consiglio pratico: **ragionare per blocchi** quando si scrive un regolamento, specialmente se è prototipale. "Non preoccupatevi più di tanto di farlo bello, preoccupatevi di farlo funzionale a voi." Dividere in blocchi aiuta anche visivamente a capire dove si è messo che cosa, e a spostarlo per vedere che cosa succede.

2.12.6 Che cosa c'è in un regolamento

Poi "un po' più di esempi specifici", parte per parte. Per ognuna, la consegna implicita è la stessa: **chiedetevi come la scrivereste per il vostro gioco**.

Di che cosa parla il gioco. È "in generale buona pratica" averla. Può avere varie forme. Può essere una **premessa di genere**, in termini meccanici: questo è un "*up-and-down game*"[9], un *worker placement*, un gioco di *draft*, strategico, astratto. Ma può contenere anche un **aspetto narrativo**: qual è la fiction del gioco, chi siamo. Nei giochi da tavolo che si hanno in casa, "spesso e volentieri le due cose sono un po' mischiate": il regolamento parla del gioco contemporaneamente in termini di *setting* e di chi siamo al tavolo meccanicamente (un gioco economico, di strategia, tattico, cooperativo). "Ragionate anche voi, al vostro *project work*: che tipo di gioco è? Se doveste scrivere questa parte nel regolamento, che cosa scrivereste?"

Che cosa serve per giocare. Anche qui varia molto (gioco di ruolo, gioco da tavolo), ma in generale è il ***setup***, la **preparazione**, i **componenti**: quando mi siedo con un gruppo di persone per giocare, che cosa devo avere con me? Possono essere cose già nella scatola ("nella scatola ci sono A, B, C, D, E"), oppure cose da prendere da casa (penne, matite, dadi). Spesso questa parte dà anche informazioni su come preparare i componenti, o indicazioni come "assicuratevi di avere un tavolo abbastanza grande".

I principi di gioco. Questa parte "purtroppo è molto rara nel gioco da tavolo", e Ruggero vorrebbe davvero che fosse più presente. Compare a volte nei giochi da tavolo con **dinamiche sociali**: *social deduction*, cooperativi a ruoli segreti, giochi con limitazioni sulla comunicazione. È più comune nel gioco di ruolo. I **principi di gioco** dicono: **se giocate a questo gioco, che cosa dovete tenere a mente in termini di come approcciarvi al gioco?** Per esempio: non rivelate il vostro ruolo; non parlate liberamente delle carte che avete in mano; nel vostro turno non potete parlare, a meno che una carta non ve lo permetta. Sono principi che, se seguiti, aiutano a giocare il gioco **come è stato inteso**, come è stato progettato. "Questo è un punto un po' più opaco per chi viene dal *board game*"; se non è chiaro, Ruggero chiede di dirlo.

Principi o regole? Infatti uno studente chiede: che differenza c'è tra questo e le regole? In un gioco a ruoli segreti, "non rivelare il proprio ruolo" non è una regola? La differenza, risponde Ruggero, è che i principi **non sono regole e non sono meccaniche**: "stiamo chiedendo a chi gioca di approcciarsi al gioco in un certo modo. Non gli stiamo dicendo fai ABC o non puoi fare ABC; gli stiamo dicendo: se giochi questo gioco, l'approccio dovrebbe essere questo". Certo, "come spesso succede", ci sono zone grigie di incrocio, in cui la stessa cosa può essere una regola o non esserlo. Un esempio: ci sono **giochi di negoziazione in cui le promesse possono anche non essere mantenute**, ma il gioco ti chiede di porti come se dovessi mantenerle, "perché stai giocando dei mercanti, non dei truffatori". È una questione di approccio.

[8] Ruggero non dice quale, ma è con ogni probabilità il gioco da tavolo di *Cyberpunk* a cui ha lavorato, citato in altre lezioni e di nuovo più avanti in questa ("quando abbiamo progettato *Cyberpunk*").

[9] Così nella trascrizione; il genere a cui Ruggero si riferisce non è ricostruibile con certezza.

2.12.6.1 L'esperimento su *Diamant*

Per mostrare che cosa cambia, Ruggero la prende "un po' tangenziale". Si è già parlato in passato di ***Diamant***, pubblicato anche come ***Incan Gold*** a seconda dell'edizione, un gioco di *push your luck*[10]. È un gioco competitivo in cui si entra in rovine "in stile Indiana Jones" per prendere delle gemme. Più si va a fondo, più gemme si ottengono; in qualunque momento si può uscire, salvando tutte le gemme raccolte; ma se, andando a fondo, si incontra una trappola, si perde tutto. "Parte del gioco è capire quando torno indietro e quando sto dentro."

Tre temi, stesse regole. Il gioco è stato oggetto di **un esperimento sociale**[11]: lo hanno **ritematizzato** in tre versioni. Una è quella originale, **alla Indiana Jones**, con avventurieri in competizione per le gemme. Una è una versione in cui si è **pompieri** e, invece di prendere gemme, si **salvano persone da un edificio in fiamme**. E una è **puramente astratta**: più si va a fondo, più punti si prendono; uscendo si salvano i punti; si perde tutto se si becca la seconda X. "Il gioco è lo stesso meccanicamente, le regole sono identiche": stesso modello matematico, stesso numero di carte, stesso numero di carte con le gemme. Cambiano la parola usata, il *setting* e **i principi con cui ci si approccia al gioco**.

Il risultato. Nella versione dei pompieri che salvano vite, "quasi il 90% delle persone" va fino in fondo, mentre nella versione Indiana Jones la gente, quando può, se ne va con le gemme ottenute. In un caso ci si approccia con l'idea "stiamo salvando delle vite umane"; nell'altro con l'idea "prendiamo più che possiamo, e poi andiamocene". Queste cose rientrano nella questione del **paragame** di cui si è parlato in una lezione precedente, oltre che nella fiction: "non sottovalutatelo". Possono aiutare a spiegare il gioco (figura 2.12.3).

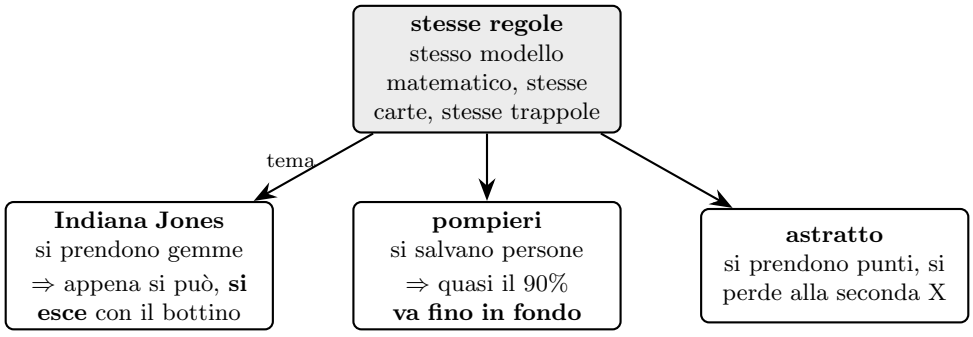


Figure 2.12.3: L'esperimento sul ritematizzare *Diamant* (o *Incan Gold*) come lo racconta Ruggero. Per la versione astratta Ruggero non dà un dato.

Voi non siete al tavolo. "Ricordatevi: quando fate giocare il vostro gioco, voi non siete nel gruppo di gioco. È raro che qualcuno compri il vostro gioco e che siate voi a spiegarglielo." Quindi scrivete il regolamento "in modo che sia il più comprensibile possibile", il che può voler dire usare anche dei "trucchi" (i principi di gioco, il *setting*, il tema) per far passare ciò che si vuole far passare.

Un regolamento può contenere tantissime cose. (Intanto entra Rita, che si scusa per il ritardo, e Ruggero consiglia a lei, a Lorenzo e a Gianmarco di recuperare l'inizio della lezione: "non sono tante slide, ma sono slide importanti".) Un regolamento può avere "tantissime cose dentro, spesso anche mischiate":

- il **pre-gioco**: *setup*, preparazione, che cosa avere al tavolo;

[10] Nella trascrizione "giochi pusher l'acqua", quasi certamente "*push your luck*", il genere a cui *Diamant* appartiene.
[11] Ruggero non cita la fonte dell'esperimento; il capitolo riporta i dati come li dice lui.

- un ***workshop***, specialmente in ambito formativo o didattico, dove spesso il gioco è preceduto da un *workshop* con chi gioca, come si è visto nei casi studio dei giochi *black box*, dei LARP scandinavi e dei giochi su tematiche delicate;
- la spiegazione dei **ruoli**;
- il **post-gioco** e il ***debriefing***;
- le **note personali** e le **ispirazioni** avute durante la progettazione: "non nascondetele, rendetele trasparenti, scrivetele nel regolamento";
- la **fiction**, il mondo di gioco, i **personaggi**;
- gli **esempi di gioco**, l'***actual play***.

"Tutte cose che vi possono essere utilissime per spiegare il vostro gioco." Non è detto che i gruppi siano nella posizione di spiegarlo di persona, e la domanda è sempre la stessa: **come mi assicuro che una persona che non sono io legga il regolamento, lo capisca e giochi il gioco come io intendo che debba essere giocato?**

Approfondimento: il regolamento come primo *tutorial*. Schell, nel capitolo sul *playtesting*, osserva un effetto collaterale dei test ripetuti: le istruzioni che il designer dà ai *tester* prima della partita vengono limate sessione dopo sessione, "una parola tolta qui, una frase aggiunta là", finché non diventano un discorso chiarissimo ed efficiente; e consiglia di **scriverlo**, perché può diventare la base del *tutorial* del gioco. Nel gioco analogico quel discorso *è* il regolamento, e la lezione di Ruggero dice la stessa cosa da un altro lato: la facilitazione di Tiziana (il "tappetino rosso", il "come prima mossa potresti") è ciò che un buon regolamento deve riuscire a fare da solo, quando il designer non c'è. Le euristiche ("che cosa ti conviene fare quando") sono la parte più difficile da mettere per iscritto, ed è per questo che molti regolamenti aggiungono esempi di partita, consigli per la prima partita e, nei giochi di ruolo, *actual play*.

2.12.7 Raccogliere i feedback: che cosa succede durante la partita

Nessuna domanda sulla parte del regolamento, e si passa all'ultimo blocco di slide: i **metodi etnografici per la raccolta dei feedback**. Il collegamento con il triangolo è esplicito. Testare in modo focalizzato (ruolo, implementazione, *look and feel*) "non è in automatico positivo in termini di risultati": posso fare un test in cui so benissimo che cosa sto testando e ho focalizzato il problema, "ma se non riesco a raccogliere i feedback in maniera funzionale, io non me ne faccio niente di quel test". Quindi: durante il test, e alla fine del test, **come faccio a estrapolare dei risultati**, a capire com'è andato?

La lezione distingue due momenti: **che cosa osservo durante la partita** (tre metodi) e **come raccolgo i feedback dopo la partita** (quattro metodi). Per ogni metodo, Ruggero chiede alla classe pro e contro, e poi completa. Le tabelle 2.12.2 e 2.12.3 riassumono il risultato.

2.12.7.1 L'osservazione in presenza

Il metodo "forse più banale e immediato": **sono fisicamente presente** e osservo il gruppo che sta giocando. Il gruppo di test gioca, noi che abbiamo creato il gioco siamo nella stessa stanza e lo osserviamo.

I pro. Risponde una studentessa: "possiamo renderci conto immediatamente di tante cose". Se la lettura del regolamento è chiara, per esempio, perché qualcuno farà dei commenti ("questa cosa non l'ho capita") o chiederà direttamente, "e sicuramente la chiedono a me, se sono lì e ho fatto io il gioco". E se i giocatori hanno dei dubbi, chi ha creato il gioco "in teoria" è in grado di rispondere, e può assicurarsi che il gioco venga giocato come è stato inteso. L'importante, aggiunge la studentessa, è **osservare le criticità**: i momenti in cui ci si inceppa, quelli in cui non sembra chiaro, che tipo di dubbi vengono. E, aggiunge un'altra voce, si può osservare il **linguaggio del corpo**.

I contro. "Però così detto sembra tutto rose e fiori." La classe ne trova diversi, e Ruggero li ordina.

- **Il feedback potrebbe non essere realistico**: c'è della *peer pressure*[12]. Le persone sono osservate da chi ha creato il gioco, e molte "si fanno dei problemi quando devono dire a qualcuno in faccia: la cosa che hai fatto fa schifo". Dipende dalle persone al tavolo, "con l'asterisco", ma alcune cercheranno di limare i feedback prima che arrivino, o avranno qualche remora a darli.
- **Chi gioca si aspetta che gli si dica come giocare**, invece di giocare attivamente come farebbe senza di noi: è condizionato nell'azione, e il *playtest* può essere falsato. In particolare, i giocatori hanno "poco incentivo a imparare il regolamento come se fossero a casa loro": c'è qui la persona che ha fatto il design, se ho dei dubbi non li cerco nel regolamento, li chiedo.
- **La scala**. Uno studente obietta che lui fa provare il gioco ai suoi amici, in quattro; ma il problema è un altro. Se dobbiamo essere presenti e dobbiamo fare dieci test, dobbiamo essere presenti a dieci partite. "Se io devo fare cento test, tutti e cento in presenza, e il gioco dura due ore, fate voi un po' di calcoli, e il tempo è poco."
- **La presenza crea dei *bias***. Il gruppo che testa con noi presenti non giocherà come se fosse a casa: "avrà un *feeling* di c'è qualcuno che mi osserva e mi giudica". E questo non lo si toglie nemmeno dicendo "non ti preoccupare, non ti giudichiamo, non è una gara, non è un'interrogazione, non è un torneo": qualcuno li sta osservando giocare. È per forza di cose un test lontano dal giocare in casa con gli amici, dove di solito nessuno ci osserva con giudizio.

"Nessun metodo è perfetto, ogni metodo ha i suoi pro e i suoi contro." La parte importante è conoscerli, "perché alcuni dei pro e alcuni dei contro possiamo combinarli ad altri per limitare i danni".

2.12.7.2 La registrazione audiovisiva

Seconda opzione: **una registrazione audiovisiva** del gruppo di gioco. Noi non siamo presenti; il gruppo gioca, e una telecamera registra la partita. "Ma in tempo reale?", chiede qualcuno. No: abbiamo una registrazione, tutta la partita registrata. E la domanda ha già toccato un pro.

I pro. Il primo: **può essere asincrono**. Posso organizzare dieci gruppi di test che giocano in contemporanea e avere dieci registrazioni nel tempo di una sola partita: "abbattiamo un po' il discorso della scala". Dovrò comunque guardarmi dieci partite, "ma sto abbattendo la durata" dell'organizzazione. Il secondo, "un grosso pro", lo trova Gianmarco: **la partita si può rivedere**. Si può tornare indietro, osservare turno per turno, tornare a uno stato precedente del gioco per capire perché le persone hanno fatto certe scelte, in modo dettagliato. "Nella partita che scorre in tempo reale possiamo perderci tantissime cose che, riguardandole, diventano più evidenti." Poter **archiviare** la partita, riguardarla, estrarne piccoli spezzoni specifici da analizzare, "è una parte importantissima, potenzialmente".

I contro. Li trova Rita: **non si può controllare tutta la stanza**, né tutti i movimenti dei giocatori, ma solo ciò che inquadra la telecamera, per esempio il tabellone, "come nella fotografia" della slide. "Questo è un punto importantissimo": in presenza siamo esseri umani che si possono spostare, guardare i volti, guardare il tabellone, sporgersi a guardare le carte in mano a una persona. Qui no. E, aggiunge una studentessa, nel *playtest* è importante osservare anche **se le emozioni che ci siamo prefissati vengono effettivamente provate**: senza vedere come si muovono le persone, è difficile. "Assolutamente."

2.12.7.3 Nessuna osservazione: il test a porte chiuse

Terzo metodo: **nessuna osservazione**. Il gruppo testa, noi non ci siamo, non viene registrato niente. Una studentessa non capisce ("com'è che funziona questa osservazione senza osservazione?"; "qui a me sembra un nonsenso"): vuol dire che non osserviamo la partita, non siamo presenti, non la registriamo. E come sappiamo che cosa è successo? Ce lo raccontano dopo, con uno dei metodi di raccolta successivi; "ma teniamoci per ora durante la partita".

[12] Nella trascrizione "pill pressure".

I contro. "Non sai come hanno giocato." Banale, "ma non sappiamo com'è andata, davvero". "Non ho dati", dice qualcuno; "è peggio di così", risponde Ruggero: **non sappiamo se hanno giocato il gioco correttamente**.

I pro. Ma anche questo metodo ha i suoi pro, "almeno altri due" oltre al primo.

- **Nessun *bias* dovuto alla presenza** di qualcuno che osserva. Uno studente porta un esempio personale: aveva provato due volte lo stesso esame all'università; la prima volta era andata male, perché era l'unico nell'aula e aveva il professore a fianco che guardava che cosa scriveva, e "non scrivevo le cose che volevo".
- **Interpretazioni impreviste**. Rita osserva che, non essendo condizionati da noi, i *tester* potrebbero dare un'altra interpretazione al regolamento e suggerire letture diverse del gioco, o scoprire una falla. È un effetto collaterale del "non sappiamo se giocano il gioco giusto", e magari emerge dopo. Ma è legato a un pro più generale: **è il metodo più vicino a come le persone giocano realmente i giochi in casa**. "A livello di *look and feel*, questo metodo è il più vicino in assoluto": quando giochiamo con gli amici non abbiamo una telecamera puntata, né il designer che ci osserva.
- **Il tempo**, che trova Tancredi: il tempo richiesto a chi fa il test "è esattamente quello di zero partite". Non devo essere presente, non devo guardare nulla, e posso fare anche mille partite in parallelo. Per questo il metodo è usatissimo per i *playtest* **quantitativi**, quelli sui **modelli matematici**, quelli su grande scala: "rilascio una beta e poi aspetto che la gente mi mandi feedback". Non so se lo giocano, come lo giocano, se lo giocano giusto, "ma a strascico qualcosa beccherò".

Metodo	Pro	Contro
In presenza	ci si accorge subito di dubbi e inciampi; si vedono volti, carte, linguaggio del corpo; si può garantire che il gioco sia giocato come inteso	feedback addolciti (*peer pressure*); i giocatori chiedono invece di leggere il regolamento; non scala (dieci test = dieci partite a cui assistere); *bias* dell'essere osservati e giudicati
Registrazione	asincrona (più gruppi in parallelo); la partita si può rivedere turno per turno, archiviare, spezzettare	si vede solo ciò che la telecamera inquadra; difficile cogliere volti ed emozioni; le partite vanno comunque guardate
Nessuna osservazione	nessun *bias* da osservatore; il più vicino al gioco reale in casa; interpretazioni impreviste; costo in tempo nullo, scala a migliaia di partite (test quantitativi, *beta*)	non si sa com'è andata, né se il gioco è stato giocato correttamente

Table 2.12.2: I tre modi di seguire una partita di test, con i pro e i contro emersi dalla discussione con la classe.

2.12.8 Raccogliere i feedback: che cosa succede dopo la partita

"Ok, la partita c'è stata": in presenza, con la telecamera, o a porte chiuse. Dopo la partita, che cosa facciamo?

2.12.8.1 Il *debriefing* orale di gruppo

L'opzione "forse più comune": alla fine del test, chi ha fatto il gioco, o chi facilita il test, fa un ***debriefing* di gruppo**.

I pro. Una studentessa: rispetto a un questionario, "le domande aperte ti permettono di raccogliere un feedback non previsto". Sì: siamo lì in tempo reale, e se emerge una domanda possiamo seguirla. Le persone "non sono delle macchine", e qualcuno dirà plausibilmente qualcosa che ci sorprende, un approccio al gioco a cui non avevamo pensato; e visto che siamo lì di persona, possiamo **costruire in tempo reale** su quel feedback qualcosa di più strutturato.

I contro. "Ce ne sono svariati." Tancredi trova il primo: **alcune persone prendono più spazio di altre**, se il *debriefing* non è gestito bene. "Questo è un punto importantissimo." E c'è una conseguenza "estremamente subdola", non necessariamente voluta ma implicita nel metodo: **la sovrascrittura delle opinioni**. Se una persona è indecisa se una meccanica le piace o no, e un'altra dice "'sta meccanica fa schifo, perché A, B, C", l'opinione forte sovrascrive parzialmente quella indecisa e la spinge verso il "fa schifo" ("o viceversa"). Non perché chi è indeciso sia "debole mentalmente", ma nel senso letterale: "sento opinioni che stanno influenzando la mia opinione". Ed è un peccato: "se facciamo un test e siamo in sei, a noi sarebbe utile avere sei opinioni sul gioco, non un'opinione molto forte e altre che sono impattate da quell'opinione". Un caso tipico è il **gruppo misto**: se su cinque *tester* due sono *game designer* e tre no, i primi tenderanno a dare feedback molto tecnici, e gli altri tre si sentiranno "la mia opinione non vale niente, non ha senso neanche che la dica"; e invece magari sono proprio le opinioni che servono di più.

2.12.8.2 Il *debriefing* orale uno a uno

L'alternativa: finita la partita, **immediatamente**, non si fa parlare il gruppo, lo si separa, e si fa un piccolo *debriefing* **con una persona alla volta**.

I pro. Il primo "è appena stato detto": **si evita la sovrascrittura**. Ma c'è di più: **si può andare molto più nel dettaglio**. In base a ciò che si è osservato, si possono avere domande specifiche per una persona specifica: "perché nel turno X hai fatto questo? Perché al turno Y hai attaccato quest'altra persona?". In un *debriefing* di gruppo questo tipo di indagine è scomodo; in un ambiente uno a uno è "l'ambiente perfetto". Un esempio: una persona ha perso la partita perché al quarto turno ha attaccato Tizio. Perché l'hai attaccato? La risposta può essere "mi sta antipatico e gli volevo dar fastidio", oppure una risposta tattica ("dalle cose che avevo, mi sembrava la scelta migliore"). "Sono due opzioni diverse", e il **modello matematico** ne risente in modo diverso. E poi si può chiedere all'altra persona: al turno X Caio ti ha attaccato; che cosa hai provato? Le **dinamiche interpersonali** si possono indagare molto bene in questo formato.

I contro. Il tempo scala con il numero di giocatori: un *party game* da dodici persone sono dodici interviste uno a uno, e se le si fa da soli diventa "un po' intensa". Magari ogni intervista dura cinque minuti, e quella di gruppo mezz'ora, ma il totale dura comunque il doppio. Se si è in più persone, ci si possono dividere le interviste ("io ne faccio tre, tu ne fai altre tre").

2.12.8.3 Il form chiuso

Terza opzione: **un modulo preparato**. Non c'è un *debriefing* orale: a fine partita si manda un *form*, e i *tester* lo compilano.

Non te lo compileranno. Il primo contro arriva subito dalla classe: "non te lo compileranno". "Diamo per scontato che le persone non siano tutte delle merde e che alcune te lo compilino"; ma sì, è uno dei contro: se non si obbligano le persone, non si avranno tutti i *form*. Una pratica che Ruggero ha visto fare anche di persona: **subito a fine partita** si dà ai *tester* un tablet con il *form*, o si manda subito una mail, e lo si fa compilare lì. "Più aspetti, meno è possibile che te lo compilino."

I grandi numeri. Un altro studente osserva che questo è il metodo che **richiede numeri su grande scala**, per assorbire gli *outlier* e le risposte date in fretta: lui stesso, ammette, ad alcune domande risponde bene e ad altre in modo frettoloso, e quella fretta si annulla se i *form* sono cento. È "contemporaneamente

un pro e un contro", intrinseco nello strumento, che infatti è usatissimo nei **test quantitativi**. "Se devo fare mille test, non mi sparo mille interviste uno a uno": non ce la si fa, in termini umani. Mandare mille *form* in contemporanea vuol dire che magari quattrocento non rispondono, e trecento rispondono "mentre sono ubriachi", ma per la legge dei grandi numeri qualcosa viene fuori.

Fuori dai margini delle domande. Il contro più importante lo trova una studentessa: con un *form* **non si esce dai margini delle domande**; serve a *validare*. Per renderlo efficace in termini quantitativi le domande devono essere molto restrittive: *slider*, voti da 1 a 5 o da 1 a 10, *checkbox* booleane (sì/no, vero/falso), scale numeriche. "È difficile, a quel punto, che salti fuori qualcosa che non ci aspettiamo." Nel feedback orale qualcuno può dire "sai che questa meccanica mi ha sorpreso tantissimo? Non me l'aspettavo"; qui quella parte non esiste. Per questo Ruggero **sconsiglia i *form* nelle primissime fasi di test**, quelle in cui non si sa ancora quali aspetti del sistema possono sorprendere. Sono invece ottimi per i **test di implementazione** e per i test su grandi numeri sui **modelli matematici**. Un esempio: "che fazione avevi? Hai vinto? Sì/no". Già solo questo, su grandi numeri, dice quante volte vince ciascuna fazione, in percentuale.

2.12.8.4 Il report libero

Ultima opzione: **il report libero**. Sempre digitale, si manda un link, e il *tester* scrive liberamente un documento sulla sua esperienza.

I contro, e un pro nascosto. Gianmarco: c'è il rischio che l'intervistato "vada per la tangente", scriva cose non inerenti, fuori dal *focus* che serve. Può scrivere quello che vuole; e in più "esacerbiamo ulteriormente" il problema della compilazione: se fare il *form* è già una seccatura, "farti il tema sul *playtest*, ti saluto proprio". Ma, supponendo persone ben intenzionate, ciò che ha detto Gianmarco è in realtà "una piccola parte dei pro": se nel *form* rigido alcune cose non spuntano, in quello aperto sì. Qui si può avere qualcosa di simile al "questa meccanica mi ha sorpreso" del feedback orale.

Il grande contro: leggerli tutti. "Non posso leggerli tutti, è molto difficile." Qui si perdono i vantaggi del quantitativo: senza una struttura standard, se arrivano mille report bisogna leggere mille documenti, "e tanto vale che rifaccio gli orali, che almeno ho spazio di manovra emergente". Un vantaggio, però, rispetto agli orali: **gli orali si perdono, questi sono scritti**. È di nuovo la questione "telecamera o non telecamera, testo scritto o testo orale".

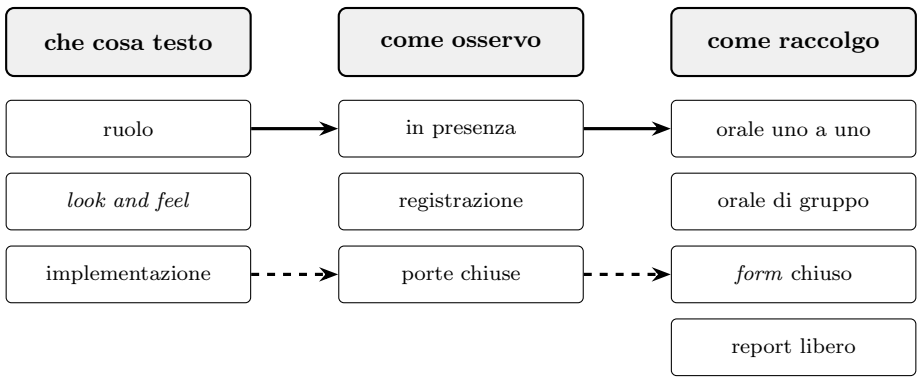
2.12.8.5 Combinare i metodi con il triangolo

"Io voglio che voi vi approcciate a questi metodi non come uno è meglio degli altri", ma con tre domande: **che cosa sto testando adesso? In che fase di test sono? Che cosa mi è utile per massimizzare i risultati di questo *playtest*?** Se si combinano questi metodi con il triangolo dell'inizio della lezione, "sono sicuro che avete già al volo alcune combinazioni molto, molto sensate". Due esempi: **un test di ruolo in presenza, con feedback orale uno a uno**; oppure **un test di implementazione da remoto, a porte chiuse, con *form* precompilato per il modello matematico** (figura 2.12.4).

Approfondimento: *debriefing* e interviste secondo Schell. Il capitolo sul *playtesting* di *The Art of Game Design* arriva a conclusioni molto vicine. Sulle interviste: **farle in privato quando possibile**, perché le persone parlano più onestamente uno a uno che quando ascoltano altri, soprattutto persone che conoscono; e, se i *tester* sono amici tra loro, fare l'intervista di gruppo *dopo* quelle private. Schell cita anche il *video closet* di Barbara Chamberlin: una stanzetta con una telecamera e una lavagna con una domanda, in cui i giocatori entrano uno alla volta e rispondono parlando alla telecamera, un rimedio contro il *groupthink* che aiuta a dare voce ai più silenziosi. Sui questionari: farli compilare **subito dopo la partita**, non metterci troppe domande, preferire scale a cinque punti con etichette chiare, e non prendere i dati come "vangelo". E sul designer presente al test, Schell riconosce il rischio che Ruggero descrive (il designer che "contamina" i *tester* con il suo punto di vista), ma conclude che, se si riesce a trattenersi, si impara molto di più osservando di persona: lui stesso dice di guardare soprattutto **i volti** dei giocatori, più che il gioco.

Metodo	Pro	Contro
Orale di gruppo	feedback imprevisti; si può costruire in tempo reale su ciò che emerge	chi parla di più occupa lo spazio; **sovrascrittura delle opinioni**; nei gruppi misti, chi non è del settore si autocensura
Orale uno a uno	niente sovrascrittura; domande mirate sulle singole scelte ("perché hai attaccato?"); dinamiche interpersonali indagate senza imbarazzo	il tempo cresce con il numero di giocatori (dodici giocatori, dodici interviste)
Form chiuso	scala a centinaia o migliaia di risposte; i grandi numeri assorbono le risposte frettolose; ottimo per implementazione e modello matematico	molti non lo compilano (va fatto compilare subito); domande restrittive, nessuna sorpresa: **valida, non scopre**; sconsigliato nelle prime fasi
Report libero	possono emergere cose che il *form* non prevede; resta per iscritto	ancora meno compilato; risposte fuori *focus*; impossibile da leggere su grande scala

Table 2.12.3: I quattro modi di raccogliere i feedback dopo la partita. Nessuno è migliore in assoluto: la scelta dipende da che cosa si sta testando e in che fase.



linea continua: test di ruolo, in presenza, *debriefing* uno a uno (le prime fasi, quando il gioco può ancora sorprendere).
linea tratteggiata: test di implementazione, a porte chiuse, *form* chiuso (molti test, modello matematico).

Figure 2.12.4: Le due combinazioni "molto sensate" citate da Ruggero tra tipo di test, modalità di osservazione e modalità di raccolta. Le altre combinazioni sono possibili; il criterio è sempre che cosa si sta testando e in che fase.

2.12.9 Il sentimento dietro la soluzione

"Ricordatevi poi un'altra cosa utilissima, questa è proprio a monte." Quando qualcuno dà un feedback, molte persone tenderanno anche a dare **la soluzione** ("devi fare così e così"). "Ma siete voi le persone che stanno facendo il *game design*." Bisogna prendere ciò che i *tester* dicono e **distillarlo**: capire il **sentimento all'origine**, la fonte. Le persone vedono spesso un aspetto delle cose, "ma non è detto che sappiano cosa c'è sotto il cofano".

La fazione troppo forte. L'esempio: qualcuno dice "guarda, questa fazione è rotta, è troppo forte, devi ridurle l'attacco di due". La **soluzione** proposta è "riduci l'attacco di due"; il **problema** percepito è "la fazione è troppo forte"; il **sentimento** è "questa fazione mi ha distrutto, mi sono sentito attaccato inutilmente da una fazione che sembra troppo aggressiva". Ma guardando la partita, magari si scopre che quella fazione era in mano a "Pierino", un suo amico che voleva fargliela pagare per motivi suoi. Il sentimento della fazione troppo forte, allora, non dice necessariamente quanto quella fazione sia forte nel modello matematico. Bisogna separare ciò che la persona dice in termini diretti da **che cosa ha fatto emergere il problema**: a volte sì, bisogna ridurre un numero di due, aumentare il numero di carte, dare un'unità in più a qualcuno; a volte il problema è frutto di un contesto, e non è colpa della fazione, del numero o della carta, ma di qualcos'altro che è successo nella partita (figura 2.12.5).

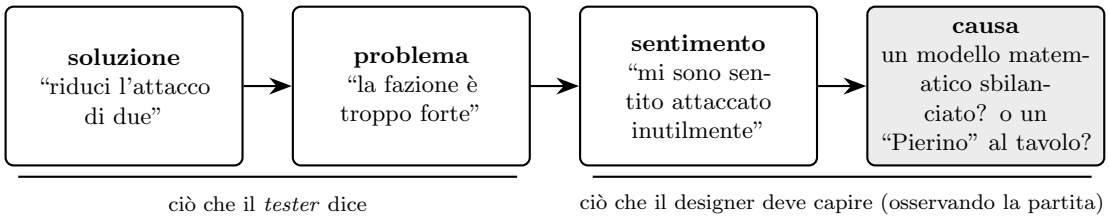


Figure 2.12.5: Distillare un feedback, sull'esempio di Ruggero: dalla soluzione proposta al sentimento che l'ha generata, e da lì alla causa. Le soluzioni sono di chi fa il design.

Approfondimento: i *tester* non sono designer. Anche questo consiglio ha un corrispettivo nel capitolo di Schell: *"Don't expect playtesters to be game designers"*. Domande come "il livello tre sarebbe stato migliore se fosse più difficile?" non danno buoni risultati, perché la maggior parte dei *tester* non è abituata a ragionare sulle meccaniche (e tende a volere giochi più facili); meglio chiedere "ci sono parti del livello tre noiose?", cioè chiedere il sentimento e non la soluzione. Le sei domande che Schell usa a fine partita (dette *FFWWDD*, da Shawn Patton) vanno nella stessa direzione: il momento più frustrante, il momento preferito, che cosa avresti voluto fare e non hai potuto, che cosa cambieresti con una bacchetta magica, che cosa stavi facendo nel gioco, come lo descriveresti a un amico. Solo la quarta chiede una soluzione, e anche quella serve soprattutto a capire che cosa il *tester* desidera.

2.12.10 Chi testa, dove, quanto, con chi

Le ultime "considerazioni extra" riguardano il contesto del test.

Chi è nel gruppo di *playtest*? Se testano gli amici o un gruppo di *game designer*, arriveranno feedback diversi. E non solo: **che cosa gioca il gruppo?** Se lo testano persone che giocano solo *filler*, e il gioco dura tre ore, il feedback "il gioco è troppo pesante" ha una certa importanza; se lo dicono persone che giocano solo giochi economici ferroviari da sei ore, è "un altro problema". Bisogna capire **da chi arriva il feedback e che esperienze ha**. È buona pratica, se non si conosce il gruppo di test, chiedere: **che cosa giocate di solito? Che tipo di giochi vi piacciono? Che meccaniche vi piacciono, e quali sopportate poco?** Ruggero lo fa quando arrivano *tester* che non conosce a provare i titoli della sua azienda: se qualcuno dice "il gioco non mi è piaciuto per niente", e a quella persona non piacciono i *worker placement*, e il gioco è un *worker placement*, "la sua opinione lascia il tempo che trova", se non come giudizio sui *worker placement*.

Dove si fa il test? In casa, a una *convention*, a una fiera, a un evento, in piazza, al bar, in una ludoteca. C'è casino, rumore, o silenzio e tranquillità?

Quanto dura? Chi facilita? Lo facilitate voi, o lo date a qualcun altro da facilitare? E in quel caso chi è: un'altra persona del gruppo, qualcuno che ha già testato in passato?

Nessun test nel vuoto. Tutte queste cose vanno sapute "perché vi danno degli strumenti per capire i feedback che vi arrivano". "Il *playtest* è un'attività umana, e come tale ha tante intersezioni mentre la stiamo facendo. Non esiste il *playtest* in gravità zero, nel vuoto, senza distrazioni."

Approfondimento: *who* e *where*. Sono, quasi parola per parola, la seconda e la quarta delle sei domande di Schell. Sul *chi*, Schell distingue sviluppatori (disponibili ma troppo vicini al gioco), amici e familiari (disponibili ma inclini a non ferire e a farsi piacere il gioco), giocatori esperti del genere (precisi ma più esigenti della media, con il rischio di tarare il gioco su una nicchia) e *tissue tester*, persone che non hanno mai visto il gioco e che si possono usare una volta sola, come un fazzoletto di carta; e conclude che la scelta dipende interamente da che cosa si vuole imparare. Sul *dove*, elenca lo studio, il laboratorio, un luogo pubblico (economico ma pieno di distrazioni), la casa dei *tester* (il "*habitat* naturale" del gioco) e l'online (quantità a scapito della qualità). È la stessa logica dei pro e contro di Ruggero: ogni scelta compra qualcosa e ne paga un'altra.

2.12.11 Domande sulle slide

Prima della pausa, Ruggero ripercorre le slide ("prototipo, regolamento, raccolta feedback"; il triangolo; il gioco come ecosistema e la scrittura come *medium* lineare; che cosa c'è nel regolamento; i metodi etnografici) e chiede se è tutto chiaro. Uno studente ha "un paio di dubbi".

Che cosa mi dice "perché hai attaccato"? Il primo riguarda il *debriefing* uno a uno: che informazioni ottengo chiedendo a qualcuno perché ha attaccato, o perché è stato attaccato? So come ha giocato lui, "ma è solo una persona". Ruggero precisa che era un esempio, valido per un gioco con meccaniche di **interazione diretta**. In un gioco strategico è utile capire se un'interazione diretta aveva uno scopo **tattico** o **sociale**: "posso attaccarti perché mi conviene meccanicamente, o perché voglio guadagnare punti sociali nel gruppo e dimostrare che non mi si deve dar fastidio: tu mi hai attaccato, adesso ti attacco anch'io". È un'informazione su come le persone giocano quel tipo di gioco. E se è una persona sola? "Certo, ma se quella cosa succede sempre quando si gioca una fazione specifica, quella cosa diventa importante. E se non l'hai chiesta, rischi di perdere questo dato."

Le psicometrie di *Cyberpunk*. Un esempio dalla sua esperienza: quando hanno progettato il gioco da tavolo di *Cyberpunk*[13], le diverse fazioni erano pensate per rispondere a diverse **psicometrie di gioco**: c'erano fazioni focalizzate sul voler attaccare direttamente gli altri, e fazioni focalizzate sul voler essere attaccate prima di attaccare. Serviva capire se era vero "o se era solo una nostra teoria", e nei test lo si osservava: hai la fazione di attacco diretto; la usi per attaccare direttamente o no? E quando lo fai, lo fai perché vedi il beneficio meccanico, o per motivi slegati dal beneficio meccanico? Il parallelo vale anche per altri giochi: in un cooperativo con due obiettivi paralleli, "ne hai scelto uno piuttosto che un altro: perché?". Nel *debriefing* uno a uno si possono indagare comportamenti personali **senza far sentire il soggetto in difficoltà**: chiedere in gruppo "ma perché hai fatto questa cosa?" sembra implicare "hai fatto una mossa stupida", e non mette chi ha fatto il test "in una posizione ideale"; in privato, la stessa domanda si fa "in modo un po' più morbido".

[13] Nella trascrizione "Cyberpunk a 2077": il gioco da tavolo legato a *Cyberpunk 2077* a cui Ruggero ha lavorato.

Come capisco se un gioco è ripetitivo? Il secondo dubbio: c'è un modo per capire, con il *playtest*, se un gioco è troppo ripetitivo da una partita all'altra? Prima di tutto, **la ripetitività non è oggettiva ma soggettiva**: bisogna definire che cosa vuol dire "ripetitivo" per il proprio gioco. E **rigiocabile non vuol dire non ripetitivo**: sono due cose diverse, e tutte le combinazioni sono possibili (molto rigiocabile ma ripetitivo, poco rigiocabile e ripetitivo, poco rigiocabile e non ripetitivo...). La ripetitività è sintomo di che cosa? Del "non voglio rigiocare il gioco"? Quello è un problema diverso, e lo si può indagare ("lo rigiocheresti, sì o no?"). Ma, collegandosi all'*emotional design* e alla ruota delle emozioni, è **molto più utile capire se il gioco riesce a suscitare specifiche emozioni**, che chiedersi se è rigiocabile o no. "C'è pieno di giochi che non sono rigiocabili e sono giochi ottimi": i ***legacy***, quelli in cui si strappano carte e si distruggono componenti, una volta finita la campagna non si giocano più. "Sul rendere rigiocabile un gioco ci sono mille approcci", ma per ora si sta parlando di *playtest* "a livello di ombrello".

La pausa. Non ci sono altre domande. Ruggero propone una pausa e poi di parlare dei progetti: "come siete messi" e i prossimi passi. Sono "le 34": dieci minuti bastano? "Bastano." Ci si ritrova "alle 45"[14].

2.12.12 Dopo la pausa: il *playtest* nel corso

Si riparte "con la classica domanda": ci sono domande? "Ve la chiedo in *debriefing* di gruppo orale, non uno a uno, e non vi mando un *form*", scherza Ruggero. Poi chiarisce che cosa si aspettano i docenti.

I *playtest* sono inevitabili, ma ne bastano uno o due. "I *playtest* sono una parte inevitabile del processo di creazione di un gioco. Noi ci aspettiamo che voi *playtestiate* i vostri giochi." Nel tempo che i gruppi hanno, non possono fare mille test, né cento, "neanche dieci". Ma "è più importante, in questo processo, che voi impariate questi strumenti", rispetto ad applicarli come se si trattasse di un gioco da pubblicare con un editore: in termini pragmatici, **anche uno o due *playtest* vanno bene**.

Il prima possibile: il rischio del *theory crafting*. Il consiglio è **testare il prima possibile**, perché spesso solo testando ci si rende conto che qualcosa non funziona. Finché il gioco resta nel regolamento e lo si immagina giocato, senza giocarlo davvero, "si crea un gap gigantesco". Ruggero l'ha visto succedere: gruppi che fanno il cosiddetto ***theory crafting***, ragionano sul regolamento, sul flusso di gioco, sugli intenti, ma non lo applicano nel pratico; poi arriva un test che "gli spacca tutto", e a quel punto c'è "una gigantesca crisi", perché il lavoro fatto fino a lì "è allucinante". "Spendete meno tempo per raffinare tutto quello che avete e più tempo per testare il prima possibile."

Un test per gruppo, e il prima e il dopo. I gruppi sono da tre, quindi testare "dovrebbe essere anche fattibile". Non serve che ogni persona del gruppo faccia un test separato: basta **anche un solo test**, organizzato da una persona del gruppo, che poi discute con gli altri i feedback e le iterazioni da fare. Ciò che i docenti vogliono vedere è **il gioco prima del test e il gioco dopo il test**, e come è cambiato. "E vi assicuro, è impossibile che non cambi, specialmente in queste prime fasi. Se fate un test e il gioco non cambia tra prima e dopo, qualcosa è andato storto in quel test."

Che cosa è cambiato dal *pitch*? Poi la domanda ai gruppi. Ruggero non ha potuto partecipare al tutoraggio, quindi ciò che sa dei progetti si limita al *pitch*: chi vuole iniziare a raccontare che cosa è cambiato da allora e lo stato attuale del progetto? La tabella 2.12.4 riassume ciò che emerge.

2.12.13 Il gruppo di Rita: fermare la ricerca, scrivere il regolamento

Parla per prima Rita, per il gruppo del *roguelike* in solitario. Di aggiornamenti di gruppo "non ne abbiamo". Uno dei compagni ha avuto un'influenza, l'altro probabilmente impegni suoi; Rita ha provato a contattarli

[14] L'ora non è detta; se la lezione è dalle 17 alle 20 come le altre, la pausa va dalle 18:35 alle 18:45 circa. Durante la pausa la trascrizione contiene diversi minuti di frasi ripetute senza senso, ignorate.

Gruppo	Stato a oggi	Indicazione di Ruggero
Roguelike di *journaling* (Rita, Giordano, Nicola)	difficoltà a trovarsi; ricerca e studio dei casi studio ancora in corso; nessun aggiornamento di gruppo	mettere in pausa la ricerca e sviluppo; scrivere subito un **draft di regolamento**, anche in asincrono (un Google Doc condiviso), con una *checklist* delle parti necessarie
Murder Memory (Tiziana, Angelo, Lorenzo L.)	direzione comune trovata: ispirazione a *Dieci piccoli indiani*, personaggi con schede, stanze come carte da sbloccare con enigmi, libretto e buste; bloccati sulla storia	regolamento **e** enigmi come due binari paralleli; rendere la storia più banale; pochi enigmi, un nodo per ogni informazione; un draft di tutti i materiali
Automa, ex *La Peste* (Tancredi, Gianmarco, Lorenzo)	nuovo *setting* (automi e umani nel castello di Geppetto); regole scritte; qualche test interno con *proxy* (chat e dadi, carte francesi)	le carte sono il gioco: preparare un ***print and play*** delle carte, uscire dalle tabelle; organizzare ora un *playtest* esterno, idealmente *blind*

Table 2.12.4: Lo stato dei tre progetti di gruppo come lo raccontano gli studenti, e l'indicazione principale che Ruggero dà a ciascuno. L'obiettivo comune è uno: arrivare il prima possibile a un gioco che una persona esterna possa giocare dall'inizio alla fine.

più volte durante la pausa didattica, ma non sono riusciti a trovare orari compatibili. I compagni le hanno detto che stavano continuando la ricerca e lo studio, provando i giochi consigliati dai docenti, ma non c'è stato modo di confrontarsi e procedere. E Rita lo dice apertamente: "emotivamente sono un po' in panico, perché mi rendo conto che il tempo stringe". Tutti, anche gli altri studenti, hanno altri impegni: "è una vita". Chiede consigli su come organizzarsi, o su come intervenire.

Mettete in pausa la ricerca e sviluppo. "È chiarissimo." Il consiglio di Ruggero: **fermate un attimo la fase di ricerca e sviluppo**. Non perché non sia importante, ma perché a questo punto va fatta **in parallelo**: se diventa totalizzante, c'è un problema. "Ricordatevi che state facendo questo gioco all'interno di un corso e non di un'azienda." In un'azienda, finché c'è il *budget*, si possono fare anche sei mesi di ricerca e sviluppo; qui, se si fanno sei mesi di ricerca e sviluppo, "poi il gioco non esiste alla fine". Quindi: passate il prima possibile a **buttare giù per iscritto il regolamento**.

Lavorare in asincrono. Non serve trovarsi tutti contemporaneamente. Non si può scrivere un regolamento "ognuno per i fatti suoi" (il documento deve essere fatto insieme), ma anche l'anno scorso ci sono stati gruppi che lavoravano in modo **asincrono**: se è difficile trovarsi, uno dei metodi è aprire **un Google Doc comune** e contribuire con pezzi di regolamento quando ognuno riesce. Questo richiederà poi un lavoro di *editing*, perché tre persone diverse scrivono in modo diverso e bisognerà uniformare, "ma quello è un problema per il futuro". E un primo draft, anche grezzo, permette già di fare un test: uno dei test di cui si è parlato è **il test del regolamento prima ancora del test del gioco**. Si dà il regolamento a una persona che non lo conosce e le si chiede: riusciresti a giocarlo? Dal regolamento capisci come si gioca, che cosa devi fare, che cosa devi preparare, che cosa ti serve? Riesci ad arrivare alla fine? Che meccaniche hai? Come si vince, come si perde? Un primo draft permette di iniziare dai **test di implementazione**, prima ancora degli altri.

Una scadenza: inizio della settimana prossima. Rita è d'accordo: nella fase iniziale di ideazione si erano visti di più e ne avevano parlato a voce, e mettere le cose per iscritto in modo asincrono può aiutare ora che non riescono a trovarsi; poi, quando si incontreranno, verrà da sé uniformare. Ruggero conferma: l'asincrono "è utile a passare questo primo scoglio", e quando il regolamento è finito ci si organizza per un *check* insieme in tempo reale, passo passo, "ma a quel punto non si parte dalla pagina bianca". L'obiettivo: avere un regolamento **idealmente per l'inizio della settimana prossima**, anche per arrivare al prossimo tutoraggio con qualcosa da guardare: se al tutoraggio si arriva "che ancora non avete niente e avete ancora la fase di ricerca e sviluppo attiva senza aver prodotto niente, i feedback non possono essere granché".

Il contributo di ciascuno si vede. Ruggero ricorda una cosa già detta in passato: nelle **cartelle del *project work*** "è chiaro chi ha lavorato a che cosa" in termini di file, e i docenti lo tengono d'occhio. Sanno che le persone hanno impegni diversi; ma "il progetto è di gruppo per rendervelo più leggero, non è di gruppo per cancellare il vostro contributo al progetto". Rita precisa che crede che anche i compagni stiano lavorando, ognuno per conto suo; ma senza confrontarsi non riescono a concretizzare un'idea che aveva entusiasmato lei e, in parte, i docenti. Li contatterà, cercherà un momento in cui ci sono tutti per fissare almeno i punti chiave del regolamento, e si daranno tempistiche più serrate, anche compilando "in maniera serrata" il modulo delle ore dedicate alle diverse fasi[15].

Una *checklist* dalle slide di oggi. Il modulo delle ore aiuta a gestire il tempo, ma sotto pressione serve un obiettivo concreto. Il consiglio ("prendetelo con le pinze, le persone sono diverse"): buttare giù un draft di regolamento usando le slide di oggi, che Ruggero ha già caricato nella cartella condivisa ("il file regolamento, raccolta feedback"). Da quelle, **fare una *checklist*** di tutto ciò che deve esserci (la preparazione, il *setup*, la premessa narrativa, i principi di gioco...) perché una persona che non è nel gruppo riesca a leggere il regolamento, capire come si gioca e giocarlo. "Mi rendo conto che è una parte complessa, ma è la parte chiave di questo passaggio." Se si arriva a uno stadio in cui il gioco è ancora da limare, alcune meccaniche non funzionano benissimo, ma una persona lo legge e dice "credo di capire più o meno come giocarlo", "a quel punto siete già a molto, molto buon punto".

2.12.14 *Murder Memory*: la storia non deve bloccare il gioco

Il secondo aggiornamento è di Tiziana (che si vede "sfocata" in videocamera: "si vede che mi considera sfondo"). Anche il suo gruppo ha avuto difficoltà simili a quelle del gruppo di Rita a trovare un momento comune, e il lavoro a più mani allunga i tempi ("butto giù la mia idea, poi interviene giustamente un altro e dice: no, secondo me va meglio quello"). Ma hanno trovato una direzione comune, e hanno cambiato diverse cose **con l'idea di semplificare**.

Che cosa è cambiato. Restano gli elementi iniziali: l'aspetto **investigativo**, l'***escape*** e gli elementi di **gioco di ruolo**, ma non come al *pitch*. Il gruppo si è ispirato a ***Dieci piccoli indiani***: tutti gli invitati hanno perso la memoria di ciò che è successo la sera prima, e quindi sono tutti coinvolti nel possibile omicidio, e tutti coinvolti nell'indagine. Delle due squadre del *pitch* (polizia e indiziato) non si parla più; la parte da gioco di ruolo coinvolge tutti: **ognuno interpreta un personaggio**, con una **scheda personaggio in parte compilata e in parte no**, perché ci sono ricordi da recuperare. C'è sempre il proprietario della villa, "un pazzo, un po' uno psicopatico"; e in realtà muore lui, e si tratta di un suicidio, ma i giocatori non lo sanno.

In termini meccanici. Ruggero interrompe la parte di trama: "io voglio sapere in termini meccanici di gioco". Gli elementi meccanici sono essenzialmente gli stessi: ci sono gli **enigmi** da risolvere per procedere. La novità sono le **stanze**: carte che rappresentano le stanze della villa, che si sbloccano risolvendo gli enigmi; i personaggi sono chiusi nella villa e devono arrivare in fondo per poter uscire (la parte *escape*). E durante la partita si raccolgono **indizi** per capire chi ha ucciso, e se c'è stato un omicidio.

Come arriva il gioco a chi lo gioca? Ruggero torna su un tema del *pitch*: la bilancia tra un gioco che si gioca con un regolamento e un gioco potenzialmente **a evento**. "Se io voglio giocare questo gioco, dove lo trovo? Devo prendere un biglietto, andare in un posto?" No, è **da tavolo**: idealmente si prende una scatola, la si apre. E che cosa c'è dentro? Tiziana descrive i componenti:

- **un libretto** che contiene il regolamento "ma non solo": la parte iniziale della storia, da leggere insieme; le indicazioni legate alle stanze visitate; le soluzioni degli enigmi; eventuali aiuti; e le istruzioni su quando aprire e non aprire, che cosa leggere e che cosa no;

[15] Nella trascrizione "quel forum per le ore" e "il forum dell'ora": con ogni probabilità un modulo in cui i gruppi registrano il tempo speso nelle varie fasi del progetto.

- **buste con le schede dei personaggi**: ognuno vede la propria e non la mostra agli altri; condivide le informazioni scritte, ma di sua iniziativa, non consegnando la scheda;

- nella busta di ogni personaggio, **un foglio** su cui scrivere le proprie ipotesi (chi è l'assassino, il movente, l'arma, "le tre cose" da indicare) e il **codice per uscire dalla villa**, di quattro o cinque cifre; alla fine le buste di tutti i giocatori vengono raccolte;

- **le carte** delle stanze.

Ruggero chiede se c'è un ruolo di facilitazione, visto che il libretto sembra "preparare" la partita: no, "sono tutti pari". E chiede chi decide la combinazione giusta del codice: il regolamento, i designer? I designer: c'è la possibilità di individuare le cifre durante il gioco, verso la fine, e poi di verificare se sono esatte. Se lo sono, i personaggi escono; se no, restano intrappolati, "e poi succedono determinate cose".

Due direzioni di lavoro. A livello pratico, Ruggero vede due binari. Il primo è **la creazione dei contenuti narrativi** che contengono gli elementi meccanici: se il gioco si basa su quello, quella parte va scritta il prima possibile, "anche magari scritta male". Il secondo è il **regolamento effettivo**, che dice a un gruppo che ha appena aperto la scatola che cosa fare. "Per esperienza: se vedete una scatola in cui ci sono delle buste chiuse e non dite alle persone *questa non devi aprirla finché non X, Y, Z*, le persone aprono tutto. Tutto quello che troviamo nella scatola, lo apriamo." Tiziana concorda: andrebbe scritto sulla scatola, o sulla prima cosa che si trova aprendola. Quindi: **il regolamento il prima possibile, anche in draft, ma deve esserci tutto**: "apro la scatola, leggo questo foglio, so che cosa devo fare a livello pratico con le cose che sono nella scatola".

Pochi enigmi, e tutti i materiali in draft. L'altro consiglio: **focalizzare gli enigmi** il più possibile. "Non state facendo un *escape room* da tavolo pubblicata, state facendo un corso accademico": meglio tre o quattro enigmi fatti bene che cinquanta che poi non si riescono a finire. Rispetto agli altri gruppi, *Murder Memory* ha una quantità di **paragame** e di materiali variegati che sono parte del genere: negli *escape* da tavolo, come gli *Exit* o gli *Hidden Games*, "dentro c'è tanta roba", e quella roba è parte fondamentale del gioco. Il gioco "si regge su quanto quei materiali fanno il lavoro che devono fare". Quindi serve il prima possibile un **draft di quei componenti**: la versione prototipale, "anche scritta a matita", "anche su un foglio scritto in Comic Sans". Non illustrazioni, mappe e giornali curati: il testo che deve esserci. "Finché non avete una versione pratica di quel materiale, non è possibile testare il vostro gioco." L'"ideale platonico" è che **entro la fine della prossima settimana** ci siano abbastanza cose in una scatola da darla a un'altra persona, che deve riuscire a giocare il gioco dall'inizio alla fine. "Magari è un'esperienza orribile, magari non sono belli da vedere, ma dovrebbe riuscire a giocarlo." E il lavoro si può dividere in asincrono: una bacheca su un Google Sheet con l'elenco dei materiali (cinque articoli di giornale, dieci fotografie, una lettera, delle schede) e chi si prende che cosa.

2.12.14.1 Il flusso di gioco non è il regolamento

Tiziana obietta: per dividere i materiali, prima bisogna avere chiara la storia; "noi abbiamo il regolamento prima della storia", e hanno già scritto il flusso di gioco. Ma la differenza, risponde Ruggero, è questa: **il flusso di gioco è una cosa che serve a voi; il regolamento serve a una persona che non è nella vostra testa**, non conosce il gioco, e deve sapere che cosa fare con i componenti della scatola. Tiziana precisa che il flusso l'aveva scritto pensando che lo leggessero i docenti, perché capissero come funziona il gioco, "non l'abbiamo scritto per noi". Ed è proprio questo il punto: "l'obiettivo non deve essere convincere i prof che il vostro gioco sia bello". I docenti, lavorando nel settore, riescono a immaginare un gioco anche da un *pitch*; "ma non è quello che dobbiamo valutare". Lo scopo è portare i gruppi a fare cose **non per i docenti, ma per una persona X** che non sono né i docenti né loro. La domanda chiave: **una persona X che passeggia nella fiera fittizia Y vede il vostro gioco in vendita o in dimostrazione, prende la scatola: riesce a giocarlo, sì o no?**

2.12.14.2 "Il vostro gioco è meglio come storia o come gioco?"

Tiziana insiste: finché non mettono giù la storia con tutti i suoi elementi, non riescono a dividerla nelle varie parti e nei vari indizi; parlandone, si aggiunge una cosa, se ne modifica un'altra ("ma potrebbe anche essere utile togliere cose, non aggiungere"), e "siamo bloccati in tutto". Ruggero risponde con un avvertimento che dice di aver visto avverarsi in passato.

Se il gioco diventa più storia che gioco. "Fate molta attenzione, perché se il vostro gioco diventa più storia che gioco, non arrivate a consegnare il gioco alla fine del corso." Scrivere una storia è molto complesso; sembra più facile, perché la storia è lineare, ma "le storie ne subiamo talmente tante che una storia brutta ti spacca tutto". In questo momento la qualità della storia non conta: "può essere la più banale del mondo, potete anche rifarmi *Dieci piccoli indiani*". Ciò che deve funzionare è la parte meccanica. Se questo fosse un corso di scrittura, la storia sarebbe la cosa importante; qui "ce ne frega il giusto": è **accessoria al gioco**. Non vuol dire che le storie non contino ("se fosse così, non avremmo fatto neanche la lezione su creare personaggi giocabili"), ma il corso non ha lo spazio dei moduli di *narrative design* di un corso di tre anni, e deve concentrarsi "molto, molto, molto, *laser*" sulle meccaniche, sulle parti che rendono l'esperienza un gioco.

Definire i punti essenziali. Tiziana precisa: non le interessa la storia "scritta bene", ma **definire i punti essenziali della storia**, perché alcune cose vengono continuamente rimesse in discussione, e cambiare certe variabili condiziona le meccaniche; vanno messe nero su bianco, con l'accordo di tutti e tre, "altrimenti continuiamo a rivangare". "Sembra un consiglio terribile", risponde Ruggero, "ma rendete più banale quella storia." **Se scrivere la storia è la cosa che vi blocca dallo scrivere il gioco, c'è un problema con la storia.** E apre "un vaso di Pandora gigantesco": **se il gioco si regge solo su una storia e sui suoi colpi di scena, è meglio come storia o come gioco?** Se è meglio come storia, "c'è un problema", almeno nell'ottica di un corso che deve dare gli strumenti del *game design*. Non che non si possano fare giochi *narrative driven*: negli anni scorsi ce ne sono stati, ma quelli riusciti (arrivati alla fine con il gruppo soddisfatto e con "un ottimo gioco") avevano una parte di narrazione, di *world building* e di trama, "ma le meccaniche erano il motore dell'esperienza". "Fatevi ispirare da quei giochi che semplificano la storia in favore del gioco."

Non tutto dipende dalla trama. Inoltre si può procedere in parallelo, perché non tutte le parti del gioco dipendono dalla trama principale. I personaggi hanno delle meccaniche asimmetriche, cose che solo loro possono fare, e quelle non sono necessariamente legate alla trama; alcuni enigmi, come quello del codice della porta, possono non esserlo. "Nel mondo ideale la coesione tematico-narrativa sarebbe totale, ma qui stiamo facendo un prototipo." Una storia migliorabile non obbliga il gioco a tenerla fino alla fine: meglio un draft "stilizzatissimo", da arricchire poi, che non fare niente sul lato meccanico finché tutta la storia non è sulla bacheca. Il rischio, visto in passato in alcuni gruppi, è **autobloccarsi**: "non abbiamo la storia, quindi non facciamo nient'altro", e intanto si usa un sacco di tempo, "vi sembra di fare tantissimi meeting, tantissimi incontri, e non avete niente in mano".

Che cosa serve per l'inizio della settimana. Almeno il regolamento, anche stilizzato e migliorabile: "apro la scatola, che cosa faccio con le cose nella scatola". Anche solo: "troverai queste buste, non aprirle finché non ti viene detto; troverai questo componente, fai questo con questo componente". L'obiettivo di gioco: qual è lo scopo, che cosa bisogna fare. E dalle parti del regolamento viste oggi, capire quali servono al loro gioco e che cosa scriverci. Angelo, che stava rileggendo il materiale del gruppo, dice che sta già scrivendo nel gruppo di chat le cose da fare; Ruggero raccomanda di averne uno ("per esperienza vi velocizza molti dei processi").

2.12.14.3 Gli enigmi: nodi, catene, parallelo

Uno studente del gruppo chiede che cosa si intenda per regolamento: l'idea è usare **un unico libretto**, come regolamento e incipit della storia, e poi, "un po' come in *Exit*", parti che si combinano con le carte per risolvere gli enigmi. Il regolamento è solo la parte iniziale con le regole? Nel regolamento, risponde Ruggero,

si possono usare dei ***placeholder***: se ha senso una parte di suggerimenti per gli enigmi, la si mette, anche se gli *hint* non ci sono ancora ("sapete che ci dovrà essere"). E questo fa nascere altre domande utili: **come mettiamo gli *hint*?** Molti giochi non li scrivono in chiaro, ma **a specchio** o capovolti, "perché l'ultima cosa che vuoi è sfogliare per sbaglio il regolamento e spoilerarti l'enigma finale". Bisogna avere un'idea di come è strutturato il regolamento: che cosa c'è, in che ordine, i componenti, la premessa, il *setup*.

Il *game design* degli enigmi. Subito dopo il regolamento, bisogna lavorare sugli enigmi: "alla fine il vostro è un gioco di enigmi". Un enigma dà un'informazione; quell'informazione si usa direttamente nell'enigma finale, o per risolvere un altro enigma, a catena, e la catena si può allungare. Ruggero fa un esempio astratto con le quattro cifre della porta (figura 2.12.6):

- **un enigma, quattro cifre**: arrivo all'enigma, lo risolvo, ho le cifre; "la versione più *smooth* e distillata";
- **quattro enigmi in parallelo**, affrontabili nell'ordine che voglio, ciascuno dei quali dà una cifra: l'esito è lo stesso, ma il processo **apre dell'*agency***;
- **quattro enigmi in catena**: il primo dà un elemento necessario al secondo, che ne dà uno per il terzo, e così via fino alle cifre.

"Se pensate agli *Exit*, agli *Unlock*, sono sempre una combinazione di queste cose": un enigma che dà X, più enigmi che insieme danno X, o una serie di enigmi la cui catena dà X. Poi sta ai designer decidere quante X e quanti nodi. Il consiglio, con poco tempo: **distillate, un nodo per ogni soluzione**. "Mi serve un'informazione? Ho un enigma che me la dà." Si è sempre in tempo, più avanti, a spezzare un nodo in due (una parte in un foglio di giornale, una in una fotografia), ma l'arco base va scritto il prima possibile.

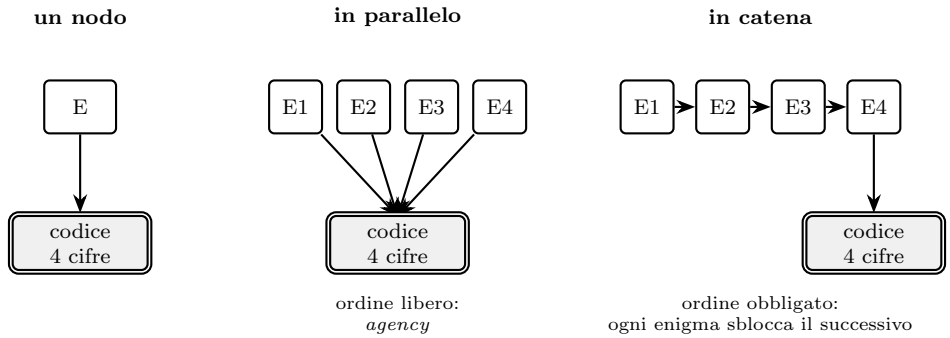


Figure 2.12.6: Tre strutture per arrivare alla stessa informazione (le quattro cifre del codice della porta), sull'esempio di Ruggero. Con poco tempo: un nodo per ogni informazione; si potrà spezzare dopo.

Le stanze: da sbloccare, poi da investigare. Lo studente spiega l'idea del gruppo, "un filo differente dalle *escape room* più classiche": risolvere gli enigmi serve a costruire la **mappa** della casa, stanza dopo stanza; una volta che si hanno tutte le stanze, il gioco diventa **investigativo**, e sul tavolo ci sono, nascosti, tutti gli indizi necessari. Ruggero chiede se le stanze sono stanze dell'universo narrativo o spazi fisici: dell'universo narrativo. Chi apre la scatola ha davanti "un mazzo di carte, diciamo fogli, perché devono essere belle grandi": si parte dalla prima, la stanza in cui si è chiusi, con l'immagine della stanza; dietro la carta c'è l'enigma, e magari una breve descrizione; risolvendolo si prende la seconda stanza e la si collega, e così via. Alla fine, guardando le immagini e leggendo le descrizioni, si deve arrivare al codice e all'assassino. Anche qui, osserva Ruggero, c'è una questione di **granularità**: per come se lo immaginano, sarà un gioco molto visivo, con la casa e le stanze disegnate; ma in fase di prototipo **una carta con scritta la descrizione di ciò che c'è va bene**, e c'è sempre tempo per dettagliare e abbellire. "I test di *look and feel* servono anche a quello. Ma servono se il gioco esiste." E aggiunge: "se il gioco è talmente bello nella vostra mente che farlo lo rende brutto, vi state creando da soli degli ostacoli".

Sherlock Holmes Consulente Investigativo. Lo studente precisa che non hanno mai pensato a grandi illustrazioni ("non abbiamo proprio modo"), ma a descrizioni minime. Ruggero chiede se qualcuno ha giocato ***Sherlock Holmes Consulente Investigativo***[16]: uno studente sì, una volta. "È tutto scritto, a parte la mappa di Londra": se si tolgono la mappa e il quotidiano, il gioco è praticamente tutto testuale, e dimostra che **si può fare un investigativo anche complesso solo con il testo**. "La vostra immaginazione su come sarà il gioco deve essere uno spunto, non un ostacolo."

2.12.15 *Automa*: le carte sono il gioco

L'ultimo gruppo è quello di Tancredi, Gianmarco e Lorenzo. Si sono concentrati sulla **parte interpretativa**, di cui si era parlato al *pitch*: hanno fatto un'analisi delle due varianti anche dal punto di vista estetico[17], delle emozioni e delle esperienze, e anche se le emozioni erano abbastanza simili e le esperienze cambiavano un po', l'approccio interpretativo "ci piaceva un po' di più". Hanno **scritto le regole** e fatto **un paio di test**.

Che cosa è cambiato. Il gioco ora si chiama ***Automa*** ("così ci aiuta a cambiarlo" anche nella testa), e il *setting* è diverso: non ci sono più i malati e la peste, ma gli **automi** ("i pupazzi") e gli **umani**, in una specie di castello di Geppetto. Gli automi devono capire chi sono gli umani, e gli umani devono capire chi sono gli automi. Non c'è più la dinamica per cui alcuni giocatori passano da un ruolo all'altro: l'obiettivo finale è **raccogliere informazioni senza farsi scoprire**.

Una persona esterna potrebbe giocarlo? Il regolamento c'è, anche se "scritto un po' così", perché stanno facendo molti test sulle meccaniche. "Quello non è ancora un problema." La domanda di Ruggero è un'altra: se dò ciò che avete a una persona che non è voi, riesce a giocare? E se no, che cosa manca? Tancredi è in dubbio: in teoria il regolamento è giocabile, ma non l'hanno fatto leggere a nessuno. Ruggero allarga: il regolamento è uno degli elementi, ma ci sono i **componenti**. "Se nel regolamento dico prendi le 60 carte del mazzo X e distribuiscine 6, è chiaro che cosa devo fare; ma se le carte non ce l'ho, non lo posso giocare." E le carte non le hanno fatte. Gianmarco spiega che per i test interni hanno usato dei ***proxy***: i contenuti delle carte erano scritti in una chat, e si tirava un dado per sapere quale carta usciva. Fare un prototipo giocabile da una persona esterna "non dovrebbe essere un lavoro enorme", ma il gioco è molto guidato da queste carte, "e queste carte sono totalmente in versione draft".

Un *print and play* delle carte. "Qui avete una parte critica." Se il regolamento c'è, ma le carte sono ciò che decide se una persona può giocare o no, il focus per la prossima volta dev'essere preparare le carte, anche in draft, **ma in un formato pensato per un gioco da tavolo**: "l'idea è che una persona armata di stampante può stamparsi il vostro gioco e giocarlo". Uno studente obietta che il gioco è giocabile anche solo con un **mazzo di carte francesi**: i colori per i ruoli e una sequenza numerica per le carte. "Se vai a dire a un editore che le persone possono giocare questo gioco con le carte francesi, l'editore ti dice: ma non te lo pubblichiamo mai." Per rendere il gioco pubblicabile a livello materiale, i componenti vanno pensati in modo funzionale al gioco. Si possono anche usare le carte dall'asso al re per assegnare tredici ruoli; ma se i ruoli hanno **poteri asimmetrici**, abilità specifiche, "ha senso che siano scritte sulla carta", soprattutto per la **qualità della vita** di chi gioca: "pesco una carta, è il 7 di cuori: che cosa fa il 7 di cuori? Vado a leggermi una tabella e lo scopro". Quindi: **preparate un mazzo**. In un foglio A4 stanno circa nove carte in formato *Magic: The Gathering*; un foglio *print and play* con quello che secondo il gruppo è il minimo necessario per giocare. Il gioco è giocabile anche da chi non è nel gruppo? (Gianmarco "fa la faccia" di chi non lo sa: ed è proprio il passo successivo.)

Un *playtest* strutturato, idealmente *blind*. "Preparatevi, perché ci arriverete, e ci arriveranno anche gli altri gruppi." I gruppi dovranno testare il gioco, e Ruggero intende **un *playtest* strutturato**: in presenza, registrato o a porte chiuse? I feedback come si raccolgono? E soprattutto **un *playtest* serio**, non

[16] Nella trascrizione "Sherlock con Suede Investigativo".

[17] Nella trascrizione "un'analisi della parte di Eye Statistica": probabilmente "estetica", nel senso delle estetiche dell'MDA, visto che subito dopo si parla di emozioni ed esperienze.

interno al gruppo: **con gente che non siete voi**. L'ideale "idealissimo" è il ***blind test***: i designer non spiegano il gioco, non rispondono a nessuna domanda, non chiariscono nessun dubbio; danno al gruppo la scatola, i componenti, il manuale, e osservano. "E solo a quel punto, se il gioco viene giocato come volevate voi, senza dirlo, il *playtest* è un successo." "Lo so, è una rottura, ma fare giochi è una rottura."

La parte preferita di Ruggero. "Se fosse per me, io farei solo *blind test* nella vita." È "la mia parte preferita in assoluto": vedere un gruppo che non conosce il gioco impararlo in tempo reale, e quando lo capisce e lo gioca come deve essere giocato, "è una delle cose più belle della vita" ("ma io sono pazzo, quindi prendetelo con le pinze"). Ruggero sa che, a chi non ha mai fatto *playtest*, i *playtest* "suonano come una rottura": è la logistica. Trovare persone che possano giocare nello stesso momento, tenendo del tempo per i feedback dopo la partita, fare un report per capire se è andata bene o male; e potenzialmente costringere quelle persone a giocare una versione che si sa imperfetta, "quindi è devastante". "Ma vi assicuro che va fatto."

Meglio i *playtest* di un bel gioco. "È più importante che facciate i *playtest* con coscienza, rispetto a che esca un gioco bello da giocare alla fine. Ve lo giuro." Il gioco finale è comunque il gioco di un corso accademico; lo scopo, come detto all'inizio, è dare gli strumenti per fare dopo i giochi che si vogliono fare. "Se il gioco più bello che fate è quello di questo corso, io ho fallito tutto." E queste cose si imparano qui, "a bassissimo rischio", e non con un editore. Ruggero racconta, senza fare nomi, di progetti con designer già pubblicati che la sua azienda non ha portato a termine perché i designer non sapevano fare i *playtest*, non sapevano fare *development*, non riuscivano a portare il gioco alla fine. "Se imparate davvero due o tre strumenti di *playtest* e di raccolta feedback, siete già anni luce davanti a molta gente pubblicata."

2.12.15.1 Dove si trovano i *tester*

Gianmarco interviene: il gruppo è "molto d'accordo", tanto che hanno provato fin da subito a portare il gioco al tavolo di altre persone. Ma trovarle non è stato facile: avevano organizzato una serata, lui e Lorenzo, ed è saltata all'ultimo; e in contesti fieristici sarebbe più facile, ma il gioco deve essere più maturo. "È vero, ed è lo scoglio più grande dei *playtest*: trovare persone disposte a fare questa cosa." Ruggero elenca alcune strade:

- gli **eventi dedicati ai *playtest***, come il *Playtest Con*[18] o, in alcuni momenti limitati, IDEAG[19], e le **convention con un angolo *playtest***, come Officine del Gioco: lì è più facile, perché le persone ci vanno con l'idea di testare e farsi testare, e di solito ci si scambia i test ("io testo il tuo gioco, tu testi il mio"); di contro, sono contesti limitati nel tempo;
- i **gruppi di *playtest* su Tabletop Simulator** su Telegram, gruppi in cui la gente si trova a giocare e che sono aperti ai *playtest*: si scrive "ho un *playtest*, mi serve qualcuno la prossima settimana", e spesso si trovano persone; gruppi degli anni scorsi l'hanno fatto;
- altri social: Ruggero usa poco Facebook, ma sa che ci sono gruppi di discussione dedicati ai *playtest*;
- le **associazioni ludiche locali**, che spesso hanno una serata settimanale di gioco e sono propense a testare qualcosa. A Torino ce ne sono diverse, ma anche nelle città piccole in cui Ruggero ha vissuto ce n'erano una o due; e chi è appassionato di giochi analogici probabilmente le conosce già.

I *tester* si costruiscono con gli anni. Specialmente all'inizio è difficile. Ruggero oggi ha il suo gruppo di *playtester*, "bello numeroso", ma i gruppi di *tester* "si costruiscono con il tempo, con gli anni". Chi proseguirà in questa professione imparerà anche a **selezionarli**: non tutti i *tester* danno feedback utili su tutti i giochi, e testando più volte si capisce con quale gruppo ha senso testare quale gioco (non il *filler* da dieci minuti con il gruppo A, non il gioco economico azionario a tema treni con il gruppo B). Ma è un passaggio successivo. **Adesso, paradossalmente, vanno bene anche amici e parenti**, perché in questo primo test **l'osservazione è più utile dei feedback diretti**: non si stanno facendo test quantitativi, né

[18] Nella trascrizione "la playtest con"; il nome esatto dell'evento non è ricostruibile con certezza.
[19] Nella trascrizione "idee AG": quasi certamente IDEAG, il circuito di incontri tra autori di giochi italiani.

test per affinare il modello matematico o per vedere se il regolamento è scritto bene; si sta testando "se il gioco funziona, prima ancora di tutto il resto". Poi, se arrivano feedback sul resto, sono utili. E proprio perché è la parte più faticosa, va resa pratica il prima possibile: "per esperienza, organizzare un *playtest* nella stessa settimana in cui lo si chiede è quasi impossibile". Se questa settimana si organizza il test per la settimana dopo, si arriva in tempo con il materiale.

2.12.15.2 Progettare le carte

Gianmarco solleva un altro tema. Il gioco, nel suo svolgimento, è semplice, "un peso leggero", e quindi gran parte della dinamica è affidata alle carte; ma non sanno bene come approcciare la progettazione delle carte in modo pratico: "non possiamo andare a casaccio". La sera prima ha provato ***Magical Athlete***[20] (Ruggero ce l'ha, e lo mostra): il cuore di quel gioco è come sono progettati i poteri degli atleti, e per *Automa* è lo stesso. Lui aveva creato dei filoni di carte, li ha mostrati a Tancredi e Lorenzo, e testandoli si sono accorti che uno non funzionava: "stiamo proprio avendo delle difficoltà a strutturare questo processo". Tancredi aggiunge il contesto: le carte sono le cose che si possono fare durante i turni, cose da dire, cose da fare, interazioni con gli altri giocatori.

***Magical Athlete* non è un metro.** Prima Ruggero ridimensiona il confronto. *Magical Athlete* "ha più o meno ventitré anni di sviluppo": la prima edizione è del 2003 (Ruggero controlla), e l'edizione attuale è iterata su ciò che si è imparato in ventitré anni; il designer è Richard Garfield, il designer di *Magic: The Gathering*, e c'è il lavoro di sviluppo di CMYK, "un *publisher* incredibile, di una pulizia allucinante". In trentasei carte, "se non ricordo male", riescono ad avere carte asimmetriche, "tutte non ambigue, tutte funzionali, tutte identitarie". "È difficile avere una roba del genere."

Due risposte. La prima è una risposta già data: **ricerca e sviluppo**, e in particolare ***Inhuman Conditions***[21], che il gruppo ha già visto. La seconda è "più astratta, ma forse paradossalmente più utile in questa fase": quando si sviluppa un'**asimmetria** di questo tipo, si può **ragionare a livello emotivo**, su che cosa dà, emotivamente, giocare uno specifico ruolo. "Prendete la ruota delle emozioni: avete dieci ruoli, sceglietevi dieci emozioni della ruota, e fate in modo che ogni ruolo abbia una meccanica che vada a giocare lì." Oppure si può lavorare su **psicometrie diverse**, cioè sulle dinamiche che si vogliono generare al tavolo: un ruolo di interazione diretta, uno di interazione indiretta, uno aggressivo, uno di negoziazione, uno di patteggiamento, uno cooperativo (figura 2.12.7). E comunque, "avere un draft migliorabile è più utile che non avere niente": meglio testare con le carte brutte che non testare.

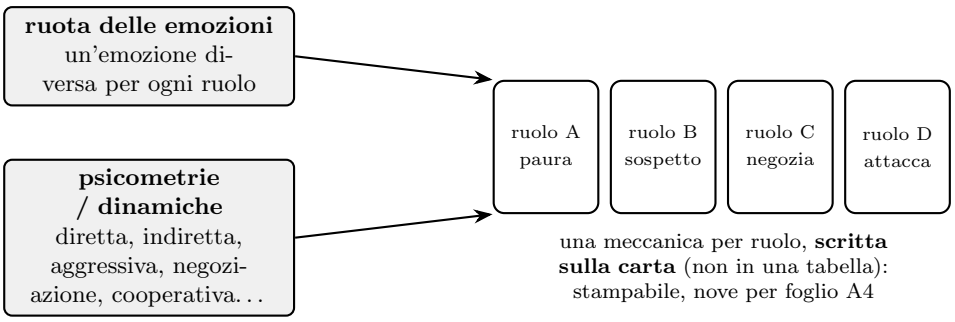


Figure 2.12.7: Due punti di partenza per progettare carte di ruolo asimmetriche, come li suggerisce Ruggero al gruppo di *Automa*. Le etichette sulle carte sono solo esempi illustrativi, non ruoli del gioco.

L'obiettivo immediato, per tutti. "Lo dico al vostro gruppo, ma vale anche per gli altri": l'obiettivo è arrivare il prima possibile a uno **stato materiale del gioco** per cui quel materiale si possa dare a una persona che non è nel gruppo, e quella persona possa giocare il gioco dall'inizio alla fine. Ogni gruppo lo

[20] Nella trascrizione "Magico Atleta" e "Magical Drake"; il *publisher* è "Chmik", cioè CMYK.

[21] Nella trascrizione "in human condition". È uno dei casi studio assegnati al gruppo nella lezione del *pitch*.

persegue in modo diverso: per il gruppo di Rita è prioritario il **regolamento**; per il gruppo di Tiziana, Angelo e Lorenzo L. il regolamento deve essere affiancato agli **enigmi**, "due binari paralleli"; per *Automa*, il regolamento in draft c'è già, "ottimo", ma **le carte sono il gioco**. "Non possono essere carte da poker, anche se sulla carta per voi lo sono": anche se meccanicamente il gioco è uguale, "pesca una carta e poi vai a vedere sulla tabella chi sei" e "pesca una carta e sulla carta c'è scritto che cosa devi fare" non sono la stessa cosa, e bisogna puntare alla seconda.

Non è solo *editing*: il *party game* delle tabelle. "Però, Ruggero, è veramente un lavoro di *editing*", obietta uno studente. "È lavoro di *editing*, ma non del tutto, perché è lavoro di paragame": cambia la dinamica di gioco. Ruggero racconta uno dei suoi primissimi giochi, "una ventina di anni fa": un *party game* sul tema dei ***talk show* televisivi**, in cui si giocavano degli ospiti di una trasmissione, ognuno con la propria personalità e il proprio modo di partecipare alla conversazione; in realtà era un gioco sul **rubarsi la parola** a vicenda per ottenere più *share* sotto i riflettori. Nella primissima versione, tutte le interruzioni, le personalità, le caratteristiche dei personaggi erano **tabelle**: "tiri il dado e vedi sulla tabella cos'è, tiri il dado e vedi sulla tabella cos'è". "Era una merda da giocare, ve lo dico." In un'iterazione successiva ha portato tutte quelle tabelle in **mazzi di carte separati**: tabelle sparite, dadi spariti. "Il gioco è andato da così a così." Tirare un dado e guardare una tabella "è quanto di più lontano da un *party game* ci possa essere": c'è un *downtime* in mezzo; la carta invece la pesco, ce l'ho in mano, la posso giocare, "e il gioco diventa già più leggero". Il consiglio: stampatelo, "anche con la stampante di casa, o sulla carta igienica piuttosto", "ma uscite da quelle tabelle". "Poi il gioco è vostro, fate quello che volete."

Il *proxy* va benissimo. Lo studente precisa che le tabelle erano solo un *proxy* per iniziare a testare e vedere se tutto funzionava. "Assolutamente: il *proxaggio* è una cosa che consiglio." Anzi, chi ha in casa delle bustine protettive vuote per le carte da *Magic* può infilarci dentro le carte stampate: "è una cosa comunissima", e per *Automa* forse il modo più immediato per testare.

Approfondimento: *blind test* e *tissue tester*. Il *blind test* di Ruggero corrisponde a due indicazioni di Schell. Sul *chi*: i *tissue tester*, persone che non hanno mai visto il gioco, sono i migliori per le domande di usabilità, di comunicazione e di primo impatto, perché vedono "con occhi nuovi" ciò a cui il designer si è abituato; e si possono usare una volta sola. Sul *come*: per alcuni test "non dirai niente ai giocatori, lascerai che il gioco parli da solo", soprattutto se vuoi vedere se riescono a capirlo da soli; e, quando si dice qualcosa, va scritto prima, perché anche poche parole fuori posto prima della partita possono rovinare il test. Schell, del resto, confessa nello stesso capitolo di *odiare* il *playtesting*, per paura che il gioco non piaccia, e di trovare ogni scusa per rimandarlo: la stessa resistenza che Ruggero riconosce negli studenti, e lo stesso rimedio ("va fatto").

2.12.16 Chiusura: la materialità del gioco, i tutoraggi, i contatti

Mancano cinque minuti. Ruggero chiede se ci sono altre domande, anche da altri gruppi o sulle slide, già caricate, e ripete a chi è arrivato tardi di recuperare la prima parte (prototipo e *focus* del *playtest*): "vi servirà".

La raccomandazione finale. "Molto, molto, molto sottolineato": **buttate giù un draft di regolamento il prima possibile** e **identificate la materialità del vostro gioco**, cioè che cosa c'è nella scatola, se arriva in scatola. Lo scopo è arrivare il prima possibile a dare quella scatola a un gruppo di persone che non sono i designer, e che riescano a giocarla. "Non stiamo ancora parlando di una bella esperienza, di un gioco bello. Siamo ancora a prima: devono perlomeno riuscire a giocarlo." Il caso peggiore è che leggano il regolamento e rispondano "non ho idea di come si giochi questa roba": Ruggero dice di averlo vissuto su giochi pubblicati, "ed è orribile come sentimento". In prototipo è la prima cosa da evitare, "perché questa è la morte del vostro gioco". E non solo in termini di prodotto: anche un gioco non commerciale, creato come mezzo personale per comunicare un'esperienza propria, anche intima, se la persona non riesce a capire come si gioca, "è tutto lavoro buttato". "E l'ultima cosa che voglio è farvi buttare via del tempo, che già è poco."

I contatti. Per i dubbi si può scrivere **via email o Telegram**, non solo a Ruggero ma anche a Daniele e Beatrice. Ruggero risponde in genere entro 24 ore; il Discord, "per colpa mia", lo apre meno di quanto vorrebbe: chi vuole essere sicuro di una risposta entro 24 ore usi Telegram o la mail.

Cambiano le lezioni? Tancredi chiede che cosa cambierà nelle prossime settimane. Gli **orari delle lezioni restano gli stessi**; forse cambia l'ordine di alcune lezioni, come oggi, quando la lezione di Diego sull'RNG ha lasciato il posto a quella su prototipi e *playtest*, ritenuta più prioritaria. I docenti adattano il contenuto allo stato dei progetti: negli anni passati, per esempio, i gruppi avevano già un draft prima, e la lezione sul migliorare il regolamento arrivava dopo; l'anno scorso c'era una lezione sul *world building*, che per questa classe non è così prioritaria, perché a livello di premessa narrativa i gruppi hanno già fatto il lavoro necessario, mentre è più urgente il *playtest*.

Il tutoraggio e il calendario unico. Sul tutoraggio Ruggero ne parlerà con Daniele e Beatrice, per trovare una data in cui la classe c'è: se sabato non va bene, come ha detto Tiziana, non ha senso tenerlo con due persone. Si cercherà, anche tramite Discord, una data che vada bene ai gruppi e si combini con gli impegni dei docenti, per massimizzare i gruppi presenti, "anche solo una persona per gruppo" (meglio tutto il gruppo, ma va bene anche un rappresentante). Tancredi chiede se si possono scrivere i tutoraggi nello stesso calendario delle lezioni, per non avere "due cose diverse in due posti diversi": ora i tutoraggi sono nelle slide, le lezioni nel calendario. "Ha sensissimo." La differenza è burocratica: le lezioni sono gestite dall'istituto con uno Zoom ricorrente nel calendario regolare, i tutoraggi i docenti devono segnarli a parte; ma cercheranno di comunicarli in un posto unico. "Che sia un link Zoom o un altro, sti cavoli, però almeno uno li vede subito tutti vicini", conclude Tancredi.

"Toppettissimo". Ruggero chiude: idealmente, per la prossima settimana, "*best result ever*": se per lunedì c'è già "un draftettino di regolamento", "toppettissimo"; ma il consiglio è farlo il prima possibile. "Buon weekend, ci vediamo la prossima. Ciao ciao." La figura 2.12.8 riassume le scadenze nominate nella lezione.

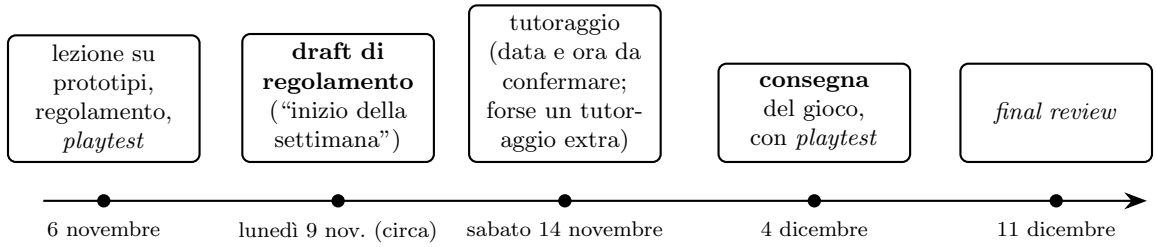


Figure 2.12.8: Le scadenze dei progetti di gruppo come emergono dalla lezione. Per *Murder Memory* Ruggero indica anche un obiettivo "platonico": materiali giocabili in una scatola entro la fine della settimana successiva. La data di lunedì è dedotta da "se per lunedì avete già un draftettino".

Punti chiave del capitolo

- Il **prototipo** costa meno, si fa prima e, soprattutto, si cambia senza il "muro psicologico del costo del cambiamento": deve poter essere buttato senza sensi di colpa. Obbliga a scegliere che cosa testare.
- Il **triangolo dei prototipi**: **ruolo** (le meccaniche generano le dinamiche e le emozioni volute? il gioco è familiare al suo *target*?), ***look and feel*** (componenti, grafica, materiali, anche il peso del regolamento), **implementazione** (struttura del regolamento, icone, numeri, *bookkeeping*, *downtime*). Mai tutto insieme; la fase decide la priorità; i test intermedi sono possibili.
- Il gioco deve **sopravvivere all'allontanarsi da voi**: chi lo compra non lo avrà spiegato da chi l'ha fatto. Il regolamento deve contenere anche ciò che per voi è ovvio: pre-gioco, *setup*, che cosa procurarsi, post-gioco, *debriefing*, fiction, intenti.

- **La scrittura è lineare, il gioco è un ecosistema**: decidere che cosa spiegare per primo è parte dei test di implementazione. Indici, rimandi, panoramiche e **blocchi** (generali, specifici, personali, materiali) aiutano; l'ordine dei blocchi si può testare.
- I **principi di gioco** non sono regole ma indicazioni di approccio; il tema cambia il comportamento (*Diamant* con i pompieri: quasi il 90% va fino in fondo).
- Durante la partita: **in presenza** (si vede tutto, ma *bias*, domande al designer, non scala), **registrazione** (asincrona e rivedibile, ma vede solo ciò che inquadra), **porte chiuse** (realistica e a costo zero, ma non si sa com'è andata).
- Dopo la partita: ***debriefing* di gruppo** (sorprese, ma **sovrascrittura delle opinioni**), **uno a uno** (dettaglio e niente imbarazzo, ma lento), ***form*** (scala, ma valida e non scopre: non nelle prime fasi), **report libero** (sorprese scritte, ma illeggibile su grande scala).
- Combinare: per esempio ruolo + presenza + uno a uno nelle prime fasi; implementazione + porte chiuse + *form* per il modello matematico.
- Dietro ogni **soluzione** proposta da un *tester* c'è un **sentimento**: distillarlo e cercare la causa; le soluzioni sono del designer. Sapere chi testa, che cosa gioca di solito, dove, quanto, chi facilita: "non esiste il *playtest* in gravità zero".
- **Ripetitivo non vuol dire non rigiocabile**; più utile chiedersi se il gioco suscita le emozioni volute.
- Nel corso bastano **uno o due *playtest***, ma il prima possibile, con persone esterne, e mostrando **il gioco prima e dopo**. Il rischio da evitare è il *theory crafting* che non arriva mai al tavolo. L'ideale è il ***blind test***.
- **Il flusso di gioco serve a voi, il regolamento a una persona X** in una fiera: riesce a giocarlo, sì o no? Se la storia blocca il gioco, **rendete più banale la storia**; pochi enigmi, un nodo per ogni informazione.
- Se le carte sono il gioco, vanno scritte **sulle carte**, non in tabelle: un *print and play*, anche con i *proxy*. Per i ruoli asimmetrici, partire dalle **emozioni** o dalle **psicometrie**.
- L'obiettivo immediato di tutti: **un draft di regolamento e la materialità del gioco**, in modo che qualcuno che non siete voi possa giocarlo dall'inizio alla fine.

Part IV

Alcune Professioni nel Mondo Videoludico

Part V

Game Design

Chapter 1

Il mestiere del game designer: introduzione al corso

Questo primo capitolo fa da soglia. Prima di affrontare metodi, framework e documenti, è necessario capire *in che modo* si impara il game design, *con chi* lo si fa, *con quali strumenti* e, soprattutto, *che cosa fa concretamente* un game designer nel suo lavoro quotidiano. Il capitolo segue l'impostazione della lezione inaugurale del corso: si parte dal contesto formativo (il corso come simulazione di lavoro), si passa al metodo di apprendimento (l'*inquiry-based learning*), si elencano letture e risorse di riferimento, e si arriva infine a una prima definizione operativa del ruolo del designer, dei limiti entro cui lavora e delle sue possibili specializzazioni.

Una premessa terminologica: il game design è una disciplina giovane e quasi tutta la sua letteratura è scritta in inglese. Per questo motivo molti termini tecnici verranno lasciati in lingua originale (*pipeline*, *pitching*, *tuning*, *feature team*, ...): sono i termini che si useranno nel lavoro, ed è importante imparare a riconoscerli e a usarli correttamente fin da subito.

1.1 Il contesto: un corso come simulazione di lavoro

1.1.1 Il principio fondamentale

Il primo concetto da fissare è che il corso **si presenta come una simulazione del lavoro**. Dal momento in cui si entra in aula si viene trattati come *potenziali professionisti dell'industria*: non soltanto studenti, ma lavoratori effettivi di uno studio (l'accademia), con una gerarchia, dei referenti, dei compiti e delle scadenze.

Ne derivano alcune conseguenze pratiche:

- il docente non assegna un "compitino" da risolvere per accumulare punti o presenze: ogni *task* è pensato per far dedicare tempo all'apprendimento di qualcosa di concreto;
- non consegnare un task non danneggia il docente (che "vive ancora domani e viene pagato ugualmente"), ma lo studente, che perde l'occasione di mettere le mani su un argomento e diventare più abile;
- docenti e studenti devono trattarsi **da colleghi**, perché è molto probabile che un domani lo diventino davvero.

1.1.2 Perché l'impegno conta più che mai

Il lavoro del game designer è descritto come *molto romantico, ma anche molto duro*, e sempre più duro col passare degli anni. Durante il periodo del Covid l'industria ha visto grandi investimenti; quei "rubinetti" sono stati chiusi da tempo, il rapporto con i publisher è diventato più complesso e le opportunità lavorative non sono più quelle di tre o quattro anni fa. Questo significa:

- più competizione per ogni posizione;
- aspettative più alte: ci si aspetta che i candidati siano ancora più preparati e sicuri delle proprie competenze;
- un unico modo per diventarlo: **lavorare**. Più tempo si dedica oggi ad apprendere il mestiere, più facilmente si sarà assumibili domani e più agevole sarà il lavoro di tutti i giorni.

1.1.3 Sbagliare qui per non sbagliare fuori

Il mantra del corso è: **quanto più provate qui, anche in modo goffo e sbagliando, tanto meno sbaglierete là fuori**. In accademia, per quanto la simulazione sia realistica, nessuno toglie lo stipendio e nessuno licenzia da un momento all'altro. Nell'industria sì, e un errore può costare molto denaro all'azienda, ancora di più se l'azienda è la propria.

1.1.4 Il valore del fallimento

Il fallimento, se affrontato con la mentalità corretta, è probabilmente *la cosa più vantaggiosa* che si possa fare durante la formazione:

- un successo ("avete superato questa deadline") conferma le proprie abilità, ma insegna relativamente poco;
- dopo n fallimenti si impara moltissimo, e i successi arrivano;
- è fondamentale non demoralizzarsi: il feedback negativo non è mai un giudizio sulla persona, ma su ciò che è stato consegnato.

Il docente riconosce apertamente che dedicare tempo ed energia a un progetto e vederlo rifiutato, o vederlo non uscire mai, è demotivante: è successo a lui e lo ha visto accadere anche su progetti molto grandi, con team numerosi. Il punto però non è ciò che si è ottenuto materialmente, ma **quanto si è cresciuti**. Da ogni esperienza ci si portano a casa due cose:

1. **l'esperienza**: quanto si è diventati bravi, veloci, praticamente abili nell'applicare un framework, quanto si è cresciuti come designer;
2. **il rapporto umano**: nel lavoro si "suda sangue" fianco a fianco con altre persone, e questo crea legami molto forti che portano avanti l'industria nel senso più positivo del termine: nascono progetti privati, ci si porta da un'azienda all'altra, si formano team con basi solide.

Va evitato anche l'eccesso opposto: la cultura del *grinding* si è appropriata di questi discorsi in modo distorto. Non bisogna diventare robot.

1.2 Organizzazione del corso e gerarchia

1.2.1 La struttura gerarchica

La struttura dei referenti del corso ricalca volutamente una **gerarchia tipica dell'industria**:

- al vertice c'è il **direttore del corso** (Diego Ricchiuti), che corrisponde al *creative director*: supervisiona la visione del progetto (il concetto di "visione" verrà approfondito più avanti);
- sotto di lui ci sono i **referenti di dipartimento**, uno per disciplina, con un corrispettivo per ciascuna sede (Milano e Roma);
- in fondo ci sono gli studenti, che occupano il primo gradino di un'azienda: l'*intern* (stagista) o il *junior*, il passo che precede il lavoratore "regolare".

Per il primo anno, il referente diretto degli studenti di game design è il docente di game design: è a lui che ci si rivolge per dubbi e domande. Altre figure utili sono il direttore didattico (severo, "giustamente severo", da contattare solo per questioni particolarmente importanti), l'amministrazione (contratti e documentazione) e la segreteria.

Disciplina	Milano	Roma
Direzione (*creative director*)	Diego Ricchiuti	
Game design	Mattia Podini	Giacomo "Jack" Danna
Level design	Stefano Zocchi	Andrea Dispoli
Narrative design	Daniele Fusetto (lezioni condivise), Giacomo Masi	
Programmazione (Unreal)	Alessandro Pregnolato	Matteo Velardi

Table 1.1: Referenti del corso per disciplina e sede (nomi come riportati in lezione).

1.2.2 I canali di comunicazione

Discord è il canale per le comunicazioni ufficiali del corso e per le domande che non richiedono urgenza immediata; può avere qualche ritardo.

E-mail è il canale per tutto ciò che è formale o "ancora più ufficiale" e per le comunicazioni rapide e immediate.

Telegram esistono residui di vecchi canali precedenti a Discord, da evitare.

Le domande che valgono per tutti vanno fatte in classe (anche da un portavoce a nome del gruppo); quelle singole e personali via Discord o, se formali, via e-mail. Se il docente non risponde a una domanda c'è un motivo, ad esempio ragioni legali, e lo dirà esplicitamente.

1.2.3 I tre moduli e la settimana tipo

Le tre discipline principali del primo anno (e non solo) sono:

- **Game Design** (tendenzialmente il lunedì);
- **Level Design** (tendenzialmente il mercoledì);
- **Programmazione** (tendenzialmente il venerdì).

Nel primo mese le lezioni sono tutte di game design (lunedì, mercoledì e venerdì). Sono previste anche lezioni di narrativa, ma il piano formativo si aggiorna ogni anno: le ore di narrativa sono state ridotte a favore di quelle di programmazione. Le poche lezioni di narrativa daranno quindi dei *cenni*, non la preparazione di un narrative designer. Ciò che serve davvero è imparare **come comunicano gli altri dipartimenti** e come lavorare con persone che fanno un lavoro diverso ma complementare al proprio.

1.2.4 Le due anime delle lezioni di game design

Design methods (metodologie) lezioni prevalentemente teoriche, ma con una buona componente pratica, in cui si studiano metodologie di design e i cosiddetti *framework* dell'industria.

Documentazione e pitching il cuore del lavoro quotidiano del designer.

La documentazione. Il docente si definisce "il più grande nerd della documentazione" che si possa incontrare. È una parte dura, perché a pochi piace documentare, ma il lavoro del designer *è principalmente questo*: scrivere documenti e, da quei documenti, passare alla pratica e all'implementazione. Chi non sa documentare bene parte male. Quando non si è presenti, il documento è il proprio **biglietto da visita**: qualcun altro lo leggerà, quindi va scritto nel modo più chiaro e comprensibile possibile.

Un esempio concreto del perché la documentazione conta: CD Projekt Red ha sviluppato un proprio engine proprietario, ma buona parte della sua documentazione è stata "buttata all'acqua". Chi deve usarlo o lo sa già usare, o ha competenze di un altro engine da trasferire, oppure è in difficoltà. Si documenta **perché altri possano leggere**.

Il pitching. Il *pitching* è l'esposizione, la presentazione orale. La parte creativa del team presenta spesso:

- le proprie idee ai colleghi;
- il gioco a convention ed eventi;
- il pitch del gioco agli investitori, per ottenere i fondi necessari a realizzarlo.

Saper parlare e comunicare chiaramente le idee è fondamentale. In sintesi: *il game designer parla*, e bisogna farsene una ragione.

1.2.5 Altre componenti del corso

Gioco critico e playtest la classe viene divisa in due gruppi: uno gioca, l'altro osserva. Chi osserva non applaude "come le scimmiette": annota, segna e analizza il *playtest*, esattamente come si fa nell'industria. Il playtest fa parte delle mansioni del designer, ma esiste anche come specializzazione autonoma.

Pipeline termine che indica tutto ciò che riguarda i processi formalizzati dell'industria; di solito si intende la *pipeline di produzione*: come si divide il team, qual è la sua gerarchia, come il *producer* gestisce le deadline. In breve: *come* e *quando* si fanno le cose. Al primo anno è meno rilevante (si dovrà "nuotare" un po' da soli); dal secondo anno si vedranno le metodologie effettivamente applicate ai progetti digitali. Si scoprirà quanto sia difficile lavorare a un progetto digitale, e ancor più fisico, senza un piano di sviluppo comprovato, come quelli usati per qualsiasi software in qualsiasi parte del mondo.

Engine il motore di riferimento è **Unreal Engine** (sezione 1.6).

Programmazione principalmente tramite *Blueprint* in Unreal, più eventuali sistemi di versionamento; il corso parte da zero.

Progetti pratici probabilmente la cosa più importante. Al primo anno i progetti di gruppo restano circoscritti al corso di design, perché mancano ancora alcune nozioni (come comunicare, affrontare problemi, dare e ricevere feedback). In seguito si allargheranno a progetti digitali, molto più difficili di quelli fisici, con membri di altri team: un rapporto non unilaterale, ma "covalente".

1.3 Il metodo: l'inquiry-based learning

1.3.1 Che cos'è

Il corso è costruito secondo l'**inquiry-based learning** (apprendimento basato sull'indagine), un approccio pedagogico nato intorno agli anni Sessanta, che si richiama a pensatori come Dewey e Piaget. In generale, le discipline da cui il design attinge sono quasi sempre la sociologia, la psicologia e, talvolta, l'economia.

Il funzionamento è un ciclo:

1. il docente fornisce metodologie e framework dell'industria durante la lezione;
2. lo studente, fuori dalle ore di lezione, lavora sui *task*: mette a terra le nozioni, le maneggia, le assorbe meglio;
3. da questo lavoro nascono nuovi dubbi;
4. i dubbi tornano in aula: lo studente chiede, il docente risponde (ammesso che dare la risposta sia utile, e ammesso che una risposta esista).

1.3.2 Non sempre esiste una risposta giusta

Non tutto ciò che si studia ha "la risposta migliore possibile in assoluto nel vuoto". Esistono metodologie, limiti e framework che aiutano il lavoro, ma non sono fissi: cambiano da azienda ad azienda, da lavoro a lavoro, da progetto a progetto, e a volte vengono stravolti. Sta alla **sensibilità del designer** capire quale strumento o framework usare in un dato momento, e questa sensibilità si impara solo con l'esperienza. Nel lavoro nessuno dirà "usa questa cosa" o "è meglio quest'altra": sarà il designer a scegliere.

Anche l'esperienza del docente ha confini: sei-sette anni di lavoro, un modo personale di approcciare il design e un certo insieme di progetti (ad esempio nessun MMO). Oltre quei confini si cercano risposte nella ricerca, studiando come aziende con esperienza specifica (ad esempio Riot) hanno affrontato problemi simili e hanno condiviso le loro soluzioni, sempre valutando quali metodi siano adatti alla dimensione del team, al tipo di progetto e al tempo disponibile.

1.3.3 Sensibilità, non memoria

Non interessa imparare a memoria un framework o la teoria di un autore. Certe terminologie vanno conosciute, ma per ragioni di comunicazione e chiarezza. Ciò che conta è sviluppare la **capacità di farsi domande**, di farle agli altri, di fare ricerca e di trovare una propria risposta. È esattamente ciò che si fa nel lavoro: davanti a qualcosa che non si sa fare, l'abilità del designer sta nel cercare come qualcun altro ha risolto un problema simile.

1.3.4 Le lezioni come corsi di aggiornamento

Le lezioni frontali simulano i *corsi di aggiornamento* aziendali: le sessioni che un lead di design tiene al suo team di design (o il lead di level design al suo team, e così via). Alla fine dell'aggiornamento c'è quasi sempre un lavoro da fare, un obiettivo da portare a terra: **l'obiettivo della settimana**. Si lavora di settimana in settimana, e il perché verrà chiarito in seguito.

Struttura tipica di una lezione:

1. breve sessione di domande e risposte sulla lezione precedente;
2. analisi di eventuali esercizi particolarmente critici;
3. lezione vera e propria;
4. discussione di gruppo, se c'è tempo;
5. assegnazione del task della settimana.

Il docente tende a "parlare e straparlare", ad aprire parentesi e a saltare da un argomento all'altro: l'unico modo per fermarlo è che siano gli studenti a interromperlo con domande.

1.3.5 La gerarchia della competenza

L'inquiry-based learning può essere rappresentato come una scala di competenza:

1. la sola lezione frontale porta a **conoscenze incomplete**, perché manca la parte pratica e di ricerca;
2. il lavoro a casa (task, ricerca) porta a una competenza **più completa**, non sbagliata in sé, ma ancora parziale e complementare ad altre;
3. la **condivisione** dei task e dei feedback con gli altri porta idealmente alla competenza più completa possibile;
4. il picco della competenza è la sua **applicazione nei progetti** (fisici o digitali, di gruppo): "con le cose che avete imparato e ricercato, riuscite a fare un gioco?". Quando la risposta diventa positiva, o comunque si inizia a navigare nei processi di design per realizzarne uno, quella è la vittoria.

Non si parte subito creando giochi: si parte dall'analisi, dal design e dalle regole generali di design, prima "nel vuoto", poi contestualizzando i framework, infine unendo i puntini nei progetti.

Rimarranno comunque delle lacune, per tutti, docente compreso, perché il game design **non è una scienza esatta**, anzi, a rigore non è affatto una scienza. Prende in prestito metodologie dalle discipline scientifiche, ma contiene una componente imprevedibile e incalcolabile: **la produzione**. Si possono applicare perfettamente le regole di un framework, e quel framework può comunque non essere adatto al gioco, al proprio modo di sviluppare o al tempo di produzione disponibile (alcuni framework richiedono una ricerca e un'attenzione che il tempo non concede).

C'è poi la componente creativa, la parte "romantica" del lavoro: esiste, ma è **metà del lavoro**. L'altra metà è ricerca, metodo scientifico al 100%. Dagli stessi dati n designer diversi, con sensibilità diverse, possono ottenere n risposte diverse. Non tutte saranno valide: vanno applicate, messe a terra, e questa scrematura fa parte del lavoro.

1.3.6 I livelli dell'apprendimento

Un secondo modello, a gradini, descrive i livelli di apprendimento (è la cosiddetta tassonomia SOLO):

Pre-strutturale il docente dà il compito e lo studente lo risolve, possibilmente il prima possibile. È esattamente ciò che *non* si vuole.

Uni-strutturale si affronta una sola disciplina: è il primo passo necessario, ed è il motivo per cui nel primo mese ci sono solo lezioni di game design.

Multi-strutturale entrano in gioco altri dipartimenti: level design, game design, programmazione e così via.

Relazionale si capisce che alcune cose valgono per più dipartimenti: alcuni framework di narrativa derivano dal design o possono essere usati per fare design; le regole di documentazione si applicano anche ad altri dipartimenti; le regole di level design si mescolano con quelle di game design. Dopotutto, per fare un gioco servono molti dipartimenti.

Astratto esteso il punto finale: muoversi autonomamente e capire cosa usare e quando, sulla base della propria esperienza. L'unico modo per arrivarci è fare pratica e fare progetti.

1.4 Task, feedback e regole del corso

1.4.1 I task come opportunità

I task non sono strettamente obbligatori (esiste un numero minimo di consegne), ma sono **opportunità**. Quanto e come ci si lavora dipende dallo studente. Un task fatto in cinque minuti, o in cui si è stati "furbi", non verrà letto né riceverà feedback.

Domande da porsi davanti a ogni task:

1. **Perché** mi è stato assegnato? Qual è l'obiettivo?
2. Come ci lavoro per ottenere un risultato **che abbia senso**? Se la ricerca e i dati indicano una direzione, è inutile andare in un'altra.

1.4.2 Quanto lavorare: un calcolo indicativo

Se il corso è una simulazione di lavoro, vale il carico di un lavoro normale:

- un lavoro a tempo pieno comprende da 30 a 45 ore settimanali; si considera una media di **40 ore**;
- si sottraggono circa 9 ore di lezione: restano ≈ 31, diciamo **30 ore**;
- con circa 3 task a settimana: $30/3 =$ **circa 10 ore per task**.

Il numero può variare, ma rende l'idea: dedicare un'ora a un task e poi fare altro non fa guadagnare nulla a nessuno. Lo studente "butta soldi", il docente risparmia tempo leggendo il task in un minuto (o non leggendolo). Dedicando invece attenzione, ricerca, cura nella formattazione e sperimentazione, e facendolo con cognizione di causa, si impara.

1.4.3 Niente voti, solo feedback

Ad eccezione del corso di programmazione, non ci sono voti (né "ottimo", né 10, né 100). Ci sono **feedback**:

- in game design, generalmente sulla documentazione;
- in level design, sui progetti pratici.

Il docente indica cosa va bene e cosa va modificato, concentrandosi in particolare su ciò che va cambiato. Il feedback **non è mai un giudizio sulla persona**, ma su ciò che è stato consegnato nel concreto.

1.4.4 La condivisione dei task

Tutti i task consegnati vengono **condivisi** con la classe, e possono essere letti solo dopo aver ricevuto i feedback del docente. Si cresce così non solo con i propri feedback, ma anche con quelli dati agli altri. Regola importante: ciò che il docente dice a uno studente **vale per tutti**. Se a una persona viene detto "questa cosa non va mai più fatta", da quel momento vale per l'intera classe.

1.4.5 Non copiare

Copiare i task degli altri è una delle cose peggiori che si possano fare, non per una questione morale, ma perché annulla il senso del percorso. Si può dedicare a ogni task il tempo e l'attenzione che si ritengono opportuni, ed è normale che col tempo ci si specializzi (preferendo, ad esempio, il level design o la programmazione tecnica) e si investa meno dove si sente meno necessità. In questi casi l'alternativa corretta è:

- non consegnare;
- studiare ciò che hanno fatto gli altri;
- eventualmente riconsegnare in seguito.

Ci saranno momenti in cui si verrà sommersi dalle cose da fare: bisogna imparare a scegliere.

1.4.6 Il "non leggo"

Il docente ha piena facoltà di non dare alcun feedback a un task (il cosiddetto **"non leggo"**) se:

- non si è lavorato con attenzione;
- non si rispettano le indicazioni date;
- è stata violata una regola estremamente importante.

Un task "non letto" equivale a un task non consegnato e non entra nel conteggio. Attenzione: un task *perfetto* e un task che *fallisce* nel comprendere o applicare un framework sono **ugualmente validi**, purché ci si sia impegnati e si sia dedicato tempo, pazienza e "sudore".

Riconsegna. Un task non letto, in ritardo o che si vuole migliorare può essere riconsegnato e verrà conteggiato, a patto che:

- i problemi segnalati siano stati risolti;
- sia stato fatto ulteriore lavoro, integrando anche tutto ciò che nel frattempo è stato detto agli altri colleghi nei loro feedback.

Il task va sviluppato con le competenze che si hanno nel momento della riconsegna, non con quelle del momento della prima consegna.

1.4.7 Requisiti per superare l'anno

- consegnare almeno l'**80% dei task** assegnati (soglia malleabile);
- partecipare almeno all'**80% delle lezioni**, in presenza o da remoto, preferibilmente in presenza; i ritardi vengono valutati caso per caso;
- comportarsi da persone corrette e rispettose (è stata firmata una carta dello studente): non urlare, non alzare le mani, non vandalizzare, non compiere atti illegali.

Impegnarsi di più porta inoltre a essere selezionati per varie opportunità.

1.4.8 Scadenze

- la scadenza di un task è **sempre prima dell'inizio della lezione** successiva indicata;
- se la classe, come team, mantiene comportamenti scorretti, il docente può **anticipare le deadline** per tutti.

1.4.9 Accessibilità e DSA

Gli studenti con disturbi specifici dell'apprendimento o neurodivergenze (dislessia, ADHD, autismo, ecc.) sono invitati a segnalarlo. Quando si affronteranno le regole di formattazione e documentazione (errori grammaticali, mancanze, incoerenze) se ne terrà conto e si potranno trovare dei compromessi, pur ricordando che nel mondo del lavoro nessuno farà "sconti". La documentazione può essere inviata all'amministrazione o alla segreteria, che la comunicherà a tutti i docenti, ma il consiglio è parlarne anche direttamente con ogni singolo docente.

1.5 Le competenze trasversali

1.5.1 I giochi si fanno in gruppo

Si possono fare giochi da soli, ed è un'impresa ammirevole; ma lavorando da soli **tutto ciò che si guadagna in libertà si perde in produttività**.

Il caso Stardew Valley. *Stardew Valley* è stato realizzato da una sola persona, Eric Barone, in anni di sviluppo e di sacrifici: faceva un altro lavoro per mantenersi (in un cinema) e di notte programmava, faceva l'arte e componeva la musica. Il risultato è stato straordinario e lo ha reso ricco, ma si tratta di un caso estremamente raro. Quanti giochi simili, nati nelle stesse condizioni, sono falliti? Molti di più.

Il rischio d'impresa. Lavorare da soli non è sbagliato di per sé e ha aspetti molto positivi: si ha il privilegio di creare esattamente ciò che si vuole. Ma ci si fa carico anche di tutto il **rischio d'impresa**: se si lavora a un gioco per cinque anni e poi il gioco va male, resta il vantaggio di averlo pubblicato, ma il dopo è sperare di vivere di quella release o di finanziare con essa il gioco successivo. Le grandi aziende hanno gli stessi problemi, ma li distribuiscono su più persone e più responsabilità.

L'obiettivo di un professionista è **saper lavorare in diverse condizioni**, e la più comune è quella del dipendente (o del freelance / libero professionista con partita IVA): si viene assunti per le proprie capacità da un'azienda, piccola o grande che sia, e si lavora con altre persone.

1.5.2 Essere persone con cui si vuole lavorare

Se le persone non vogliono lavorare con voi, quello è **il primo motivo di licenziamento**. E se non si viene licenziati, il clima nell'azienda diventa terribile: la produttività crolla, la motivazione crolla. Le *soft skill*, cioè l'insieme di abilità che rendono una persona piacevole con cui lavorare, vanno coltivate e diventano sempre più importanti col passare degli anni (relativamente meno ora, molto di più quando i team saranno più grandi). Anche chi è un "lupo solitario" deve imparare a:

- lavorare con altre persone;
- comunicare con altre persone;
- trovare un punto in comune da cui partire per fare le cose.

1.5.3 Indipendenza e proattività

Sono le altre due parole chiave del corso:

Indipendenza spesso si verrà lasciati da soli, non per cattiveria ma per sfida: quando si cerca una risposta da soli, ciò che conta non è tanto la risposta quanto **il processo**. Una risposta data dal docente non vale niente.

Proattività chiedere, domandare, agire da soli senza aspettare direttive, perché spesso quelle direttive non ci sono, a maggior ragione nei team piccoli.

Tutto questo si concretizza nel parlare, nel presentare e nel fare *post-mortem* (l'analisi a posteriori di un progetto concluso, che verrà approfondita più avanti), e in generale nell'attenzione alla crescita personale.

1.5.4 Fare domande

Il corso insiste sulle domande: alzare la mano, intervenire anche "a bomba", fermare il docente ogni volta che qualcosa non è chiaro. Se una persona non conosce qualcosa, molto probabilmente c'è almeno un'altra persona che non la conosce. Il docente può dare per scontata la conoscenza di termini che non lo sono, ed è del tutto normale, soprattutto all'inizio.

1.6 Strumenti: engine, Unreal e materiali

1.6.1 Unreal Engine e gli altri motori

Il motore di riferimento del corso, usato in level design, è **Unreal Engine**: "un gran mattone", ma che dà a un designer molte possibilità. Il vantaggio principale è che permette a un designer non pratico di programmazione di **prototipare tramite i Blueprint** (il sistema di scripting visuale di Unreal). Questo non impedisce di passare a Unity o ad altri engine, e viceversa: il docente stesso ha studiato e lavorato a lungo con Unreal, ma la sua ultima esperienza professionale (come level designer) è stata su Unity, un engine che non conosceva, a dimostrazione che il salto da un engine all'altro è possibile.

Panorama dei motori citati:

- **Unreal Engine** e **Unity**: i due più diffusi, con modelli di sorgente "più o meno" aperto;
- **Godot**: engine open source;
- **CryEngine**: proprietario di Crytek;
- **Snowdrop**: engine di Ubisoft;
- Rockstar usa un proprio engine proprietario;
- alcune aziende usano versioni **personalizzate** di engine esistenti: ad esempio Milestone, nei giochi di corse, usa un Unreal customizzato;
- CD Projekt Red ha un engine proprietario (con i problemi di documentazione già citati).

In sintesi: ogni azienda può avere un engine proprietario oppure una versione modificata di un engine commerciale, ma si parte sempre da una base.

1.6.2 Programmazione: cosa ci si aspetta da un designer

Non è richiesto alcun bagaglio pregresso di programmazione: il corso parte da zero. Gli studenti sono stati selezionati (il corso è a numero chiuso) per competenze analitiche e critiche sopra la media. Anche nel mercato del lavoro, per una posizione di game designer generalmente **non ci si aspetta competenza di scripting** (C++, Java, C#, ecc.), a meno di voler fare il *technical designer*, e anche lì varia da azienda ad azienda.

1.6.3 Versione e preparazione

Indicazioni pratiche emerse in aula:

- scaricare Unreal Engine; la versione consigliata per il corso è stata indicata nella **5.6** (con conferma successiva). La stabilità dipende da cosa si deve fare, da quali tool e plugin si usano: i problemi nascono spesso dai plugin. Nel passaggio da Unreal 4 a Unreal 5, ad esempio, il sistema *Landscape* era inizialmente inutilizzabile, mentre in Unreal 4 funzionava molto bene;

- è consigliato installare tramite **Epic Games Launcher**, più comodo: permette anche di riscattare giochi gratuiti periodici e di accedere ai pacchetti di **Fab** (il marketplace di asset acquisito da Epic), che ogni mese offre alcuni asset gratuiti utili per esercitarsi nel level design (ad esempio per fare *speed level design*);

- molti pacchetti di UI o di sistemi di interazione sono utili per prototipare al volo, ma spesso inutilmente complicati;

- un plugin di **materiali di prototipazione a griglia** (adattivi, con metriche, allineabili alla griglia, con materiali già pronti e colori modificabili, dal costo di circa 5 euro) è indicato come "salvavita" per il level designer; se necessario, i pacchetti indispensabili vengono forniti;

- portare **sempre il portatile** alle lezioni di level design (da novembre, a partire dall'interfaccia dell'engine); alle lezioni di game design basta che lo porti almeno metà della classe, per i lavori a coppie. Serve comunque qualcosa per prendere appunti ("tavolette d'argilla e cannucce, se volete fare i babilonesi", oppure carta e penna).

Chi ha un portatile poco potente dovrebbe comunque provare a installare l'engine e aprire un template per vedere come gira: all'inizio non si faranno cose pesanti. Nelle prime lezioni di level design è importante mettere subito le mani sull'interfaccia: ascoltare dove si trova uno strumento (ad esempio lo *snapping*) senza provarlo porta a dimenticarlo.

1.7 Letture e risorse di riferimento

1.7.1 I libri

***The Art of Game Design: A Book of Lenses* (Jesse Schell)** il testo **davvero importante**. Alcune lezioni finali dell'anno si baseranno sulle sue terminologie. È utile perché apre la mente sulla possibilità di applicare metodologie diverse a giochi diversi, ma soprattutto per la **terminologia** comune. Esiste anche in audiolibro (circa 25 ore).

***Challenges for Game Designers* (Brenda Romero e Ian Schreiber)** libro *entry level* con moltissimi esercizi, molto caro al direttore del corso.

***21st Century Game Design* (Chris Bateman)** utile come retrospettiva: è un libro visionario che ha previsto metodi e soprattutto *archetipizzazioni* dei giocatori tuttora valide nel design e nel marketing. Servirà per una delle lezioni più importanti dell'anno. È utile leggerlo per vedere quanto aveva previsto, ma è utile "fino a un certo punto".

***An Architectural Approach to Level Design* (Christopher Totten)** non strettamente necessario, ma di lettura semplice. Ha due anime: parte dall'architettura (babilonese, greca, ecc.) e la trasla su esempi di level design (come *Doom* e *Team Fortress*). Un tempo ne esisteva un riassunto su Gamasutra (oggi *Game Developer*).

***A Theory of Fun for Game Design* (Raph Koster)** utile fino a un certo punto: contiene diverse idee superate, che lo stesso Koster ha affrontato in vari talk reperibili nel GDC Vault. Verrà ripreso nelle prime lezioni, poi messo da parte.

1.7.2 Canali e conferenze

GDC (Game Developers Conference) il suo canale raccoglie talk e panel estremamente importanti: è "uno strumento misterioso che ci servirà più tardi". **Attenzione**: per molti talk mancano ancora le competenze per interpretarli, e alcuni potrebbero fuorviare o mettere in testa idee strane. La GDC si concentra spesso su produttività e mobile.

Il *Dragon Speech* di Chris Crawford il talk del fondatore della GDC: appartiene a un passato lontano, ma è molto motivante. Da vedere subito.

Game Maker's Toolkit canale particolarmente utile per nozioni pratiche, con video brevi e alcune interviste.

Nordic Game altra conferenza importante, con talk e panel di esperti del settore.

1.7.3 Associazioni e premi: come leggerli

Le associazioni di settore (ad esempio IIDEA in Italia, o la IGDA) non sono il male, anzi: sono un canale importante per affacciarsi all'estero o per accedere a fondi. Va però ricordato che **tutelano i propri associati**: non esiste solo il loro palinsesto, e bisogna valutare quanto spazio occupano nel panorama della visibilità.

Lo stesso vale per i **premi** (Game Awards, "Best Mobile Game", "Games for Change", ecc.): al designer *non interessano*. Che un gioco abbia vinto "più di 90 premi" non dice nulla di utile. Servono a dare visibilità e lustro ai giochi ed è giusto che esistano, perché ampliano il panorama e il numero di giocatori. Ma ciò che interessa al designer sono i **dati**:

- come sono state implementate certe cose;
- come è stato fatto il marketing e dove il gioco è stato presentato;
- non quante wishlist ha fatto, ma *come* le ha fatte;
- in che periodo storico è uscito e quali erano i competitor in quel momento.

1.8 Che cosa fa un game designer

Per arrivare a una prima definizione, la classe ha condotto un brainstorming: ognuno doveva indicare una cosa che, secondo lui, fa il game designer (senza ripetere quelle già dette). Ne è emersa la lista seguente, commentata punto per punto.

1.8.1 Progettare meccaniche e struttura del gioco

Il ruolo più generico è quello del **core mechanic designer**, detto anche *gameplay designer* e, un tempo, *black box designer*: si occupa delle **meccaniche di gioco**. Si va dal piccolo al grande (in funzione anche della dimensione dell'azienda):

- nel **piccolo** ci sono le singole meccaniche;
- nel **grande** ci sono i **sistemi**. Nei giochi *sistemici*, con molti sistemi interconnessi (ad esempio *GTA*), esistono specializzazioni dedicate a bilanciare e supervisionare i sistemi: è il *system design*.

Quando un gioco è "sistemico"? Non dipende solo da come le meccaniche interagiscono con il giocatore (le *dinamiche*, secondo i framework che passano dalle meccaniche alle dinamiche, che verranno studiati più avanti), ma anche da *quante* meccaniche ci sono e da quanto interagiscono tra loro. Non diremmo che *Super Mario* sia un gioco sistemico, eppure contiene dei sistemi: la linea è sottile. Quando i sistemi aumentano molto, sono complessi e si influenzano a vicenda, il gioco diventa sistemico. Esempio: in *The Legend of Zelda: Breath of the Wild* il vento modifica quanto il personaggio sale o scende; il fuoco può incendiare l'erba; l'erba prende fuoco se colpita da una freccia infuocata; e così via. Nel mercato moderno i giochi sistemici sono spesso, ma non sempre, legati a mappe *open world*.

1.8.2 Le 3C: camera, character, control

La base del gioco, da cui generalmente si inizia a lavorare (anche nei prototipi), è la "sacra trinità videoludica" delle **3C**:

Camera che tipo di inquadratura ha il gioco: prima persona, terza persona, isometrica, ecc.

Character quanto è grande il personaggio, che tipo di personaggio è e, di conseguenza, quali azioni può compiere.

Control i controlli: che cosa il personaggio può fare, quali sono i *verbi* di gioco.

Le 3C sono definite dal designer, ed esiste anche una specializzazione per questo lavoro. Esistono varianti: quella del direttore del corso è **4C + J**, cioè *camera, character, control, conflict* (conflitto) e *juice* (il "succo", il feedback che rende piacevole l'interazione). Verranno approfondite più avanti.

1.8.3 Il personaggio

Il designer definisce il personaggio dal punto di vista delle meccaniche (dimensioni, azioni, capacità), all'interno delle 3C. L'aspetto estetico è invece competenza di altri dipartimenti.

1.8.4 Documentare le informazioni per gli altri dipartimenti

Assolutamente vero: la documentazione è una delle prime attività del designer. Più l'azienda è grande, più questo lavoro diventa prominente; più è piccola, meno è rilevante. In **pre-produzione**, però, è sempre fondamentale.

1.8.5 Mediatore e leader "al servizio"

Il designer è un **mediatore** nei conflitti interni al team, una sorta di collante che si relaziona con dipartimenti diversi (non sempre gli stessi, a seconda della fase di produzione). A volte fa anche da leader, a seconda di come sono gestite la pipeline e la gerarchia.

Questa leadership è difficile da definire perché **non è verticale**: è una leadership *al servizio* (*servant leadership*). Il designer "è asservito al programmatore": il suo compito non è arrivare con un'idea e scaricarla addosso agli altri, né portare la soluzione e lasciare il problema a qualcun altro, ma **seguire il processo a basso livello**, sporcandosi le mani.

I feature team. Nei team medio-piccoli o medio-grandi possono esistere sotto-team (*feature team*) dedicati a un obiettivo specifico: ad esempio un designer incaricato del character o della camera, affiancato da uno o due programmatori e da un artista. In questi casi il designer supervisiona il lavoro e gestisce la produzione del sotto-team.

1.8.6 Gestire l'avanzamento del prodotto

Esiste una specializzazione che si occupa solo di questo, il **producer**, a un livello gerarchico completamente diverso (e che potrebbe essere una strada percorribile al termine del percorso di studi). All'interno di un feature team anche il designer può farlo, ma non è detto. Può certamente *dare la direzione*: nella sua

documentazione può indicare che per arrivare a uno stato giocabile del prototipo serve un certo lavoro di arte e di programmazione, per lo stato successivo un altro, e così via. In ambito produttivo esistono metodologie strutturate come **Agile**, che verranno viste in seguito.

1.8.7 Presentare le idee

Il pitching (sezione 1.2) è richiesto al designer sia in pre-produzione sia in produzione, in misura che dipende dall'azienda. Ad esempio Ubisoft lascia ai designer, e in realtà a qualsiasi membro, la possibilità di *pitchare* un'idea prima di una nuova produzione, perché possa iniziare a prendere vita.

1.8.8 Bozze e prototipi

Le bozze vanno bene, ma devono essere **chiare**. C'è un'eterna lotta tra documentazione e gioco giocabile: se si fa solo documentazione il gioco non avanza; se si fa solo il gioco manca la documentazione e si perde la visione. Sono due cose **complementari**.

Il modo per comunicare un'idea in forma grezza ma giocabile è il **prototipo**. Un prototipo può essere qualsiasi cosa: di carta, in Unreal con Blueprint "brutti". *Non deve essere bello, deve essere giocabile.*

Esempio: il prototipo di carta di Mario + Rabbids. Durante lo sviluppo di *Mario + Rabbids*, i programmatori di Ubisoft erano impegnati su altri progetti (come *Ghost Recon*). I designer realizzarono allora un **gioco da tavolo** che riproduceva le meccaniche del gioco, quelle di uno strategico a turni sul modello di *XCOM*. E funzionava. L'obiettivo era **validare un'idea**, validare un gameplay.

Altri modi per rendere chiara un'idea sono i *mock-up* nella documentazione e le immagini modificate o i *mash-up* di qualsiasi natura. Lato documentazione l'importante è che sia chiaro; lato prototipo, che sia giocabile.

1.8.9 Fare ricerca per l'idea, e poi smontarla

Il designer ha una componente creativa importante e fa ricerca per arrivare a un'idea. Ma ciò che deve poi fare necessariamente è **smontarla**. Il suo lavoro è scientifico, esattamente come il **metodo scientifico**, il cui obiettivo è *confutare un'ipotesi*: si arriva a un'ipotesi tramite sperimentazione e poi si cerca n volte di confutarla; se resiste, la si assume come valida *in un certo contesto* (le regole della fisica classica, ad esempio, non valgono nella meccanica quantistica).

Allo stesso modo il designer arriva a un'idea (tramite ricerca o chiedendo agli altri: le idee le hanno tutti, non sono un privilegio del designer), cerca di smontarla e poi cerca di **metterla a terra**. Se l'idea non è applicabile, non è implementabile, finché non si trova un modo per realizzarla va scartata. Può essere l'idea più bella dell'universo, ma **se non può essere messa a terra, non esiste**.

L'originalità. Serve un'idea originale? Dipende dall'obiettivo e dalla visione del progetto:

- se l'obiettivo è vendere molto, un'idea originale non serve necessariamente;
- se si vuole creare un nuovo filone, serve un'idea, se non originale, almeno interessante: può essere un discorso di marketing o una meccanica innovativa, con il rischio di non vendere.

Il ruolo del *director* e il concetto di *visione* verranno approfonditi in seguito.

1.8.10 Analizzare i competitor

Arrivati a un'idea ("vogliamo un certo tipo di combattimento"), si va a vedere se qualcuno l'ha già fatta e *come*. Le meccaniche non sono privilegio di nessuno, salvo quando vengono brevettate, come il **Nemesis System** di *La Terra di Mezzo: L'Ombra di Mordor*.

Esempio: il combattimento da ridimensionare. In un progetto (presso Stormind Games) il team si trovò con un sistema di combattimento troppo costoso: mancavano le risorse e gli animatori per realizzarlo. Poco output lato gameplay significava non poter fare qualcosa di complesso e profondo come in *Elden Ring*; al contrario, il team aveva molti artisti e poteva puntare su qualcosa di più artistico. Si iniziò così a orbitare intorno al corpo a corpo di *The Last of Us* (parte I e II). Sul canale della GDC esistono diversi talk che raccontano cosa è stato fatto e come, con successi, insuccessi e costi (ad esempio del *motion capture*): da quei talk si ricavarono moltissime informazioni utili.

I competitor possono essere:

- **diretti**: un gioco molto simile al proprio;
- **di mercato**: un gioco che domina il mercato in cui ci si vuole affacciare;
- **indiretti**: un gioco con un sistema simile al proprio, ma tutto il resto diverso.

Regola d'oro: **là fuori, nascosto nell'ombra, c'è sicuramente un gioco molto simile al vostro**. Va solo trovato. E se per qualche ragione non ha venduto, bisogna capire perché.

1.8.11 Lavorare in engine e fare tuning

Il level designer lavora in engine, così come il technical designer; ma qualsiasi game designer, a seconda di ciò che deve fare, può finire in engine, soprattutto per il **bilanciamento** e il **tuning**.

Il *tuning* è l'azione di bilanciare con precisione i dati. Esempio: in *Super Mario* si modifica quanto in alto salta il personaggio, con che velocità, in quanto tempo raggiunge l'apice, e così via. Questo lavoro rende il gioco:

- ad alto livello, dotato della **progressione** giusta;
- a basso livello, **piacevole da giocare**.

Il *fine tuning* è un lavoro di dettaglio, paziente e minuzioso.

1.8.12 Bilanciare

Il bilanciamento, emerso tardi nel brainstorming, è una delle attività centrali del designer e ha anche una specializzazione dedicata (l'*economy designer*, sezione 1.10).

1.8.13 Analizzare il mercato

In pre-produzione, di solito, sì: capire le tendenze, i generi che cambiano nel tempo, la compatibilità del gioco con il momento storico, in vista ad esempio di un possibile publisher. Va capito quanto, come e con quanto tempo farlo, perché si è sempre limitati da molte cose.

1.8.14 Sintesi

> Il designer parte dall'idea ma non si ferma lì: la confuta, lavora sui dettagli, bilancia, fa *fine tuning*; lavora con il team e fa da *manager* per diversi dipartimenti; e naviga costantemente tra i limiti.

1.9 I limiti del design

I limiti ci sono sempre, e nel mondo del design sono spesso numerosi. I più importanti entro cui navigare sono:

Tempo ad alto livello, l'idea deve essere realizzabile nel tempo di produzione disponibile; a basso livello, il proprio lavoro settimanale deve essere fattibile nel tempo che si ha. Non ci si può dividere tra mille feature team se realisticamente se ne può seguire uno solo.

Tecnologia si può o non si può fare qualcosa? Ci sono i server per un multiplayer con un certo numero di giocatori? Ci si può permettere, ad esempio, una fauna che reagisce alla fisica come quella di *GTA VI*, che richiede il meglio dei *technical artist*?

Background e competenze che cosa sa fare il team? Realisticamente, chi ha lavorato dieci anni su giochi horror viene assunto per un horror, perché ha le competenze per risolvere problemi già visti.

Persone e motivazione il team ha la motivazione e le competenze per fare un certo tipo di gioco, e per lavorare insieme?

Denaro tutto ruota intorno ai soldi. Che si lavori da soli o in mille, le persone vanno pagate e il gioco deve avere un ritorno economico. Bisogna decidere dove tracciare la linea: raggiungere il **break-even** (guadagnare esattamente quanto speso) o puntare a grandi guadagni. Con un **publisher**, inoltre, una parte consistente dei ricavi va a lui (ripartizioni dell'ordine di 80/20 o 70/30, che dipendono dal contratto firmato).

1.9.1 Non siete "gli uomini delle idee"

Per qualcuno può essere un problema scoprirlo, ma il designer non è "l'uomo (o la donna) delle idee": accanto alla parte creativa c'è una parte tecnica, e si è fortemente limitati da molti fattori. Questi limiti, però, sono un aiuto:

- riducono il ventaglio di possibilità potenzialmente distruttive per il team (cose che non si riescono a fare, che non piacciono o per cui mancano le risorse);
- aiutano a mirare bene, trovando soluzioni creative in uno spazio più piccolo e più attuabile.

Le idee sono nulla, l'implementazione è tutto.

1.10 Ruoli e specializzazioni

1.10.1 La scala gerarchica

Creative Director (CD) mantiene la **visione** e dà l'idea, in modo che tutto segua una certa direzione. In teoria può diventarlo chiunque, ma con un'esperienza notevole, nell'ordine dei decenni: si assume che in quel tempo abbia visto come lavorano molti dipartimenti. Naturalmente un CD con un background artistico ha meno competenze di design, e viceversa.

Director di disciplina uno per dipartimento: *game director*, *level director*, *art director*, ecc.

Lead il capo del dipartimento, che gestisce i gruppi di persone. Chi lavora in un dipartimento risponde al proprio lead (ad esempio un game designer risponde al *lead designer*, non al game director).

Senior / Principal i profili più esperti (a seconda dell'azienda).

Junior il profilo di inizio carriera.

Intern lo stagista.

Lo schema si replica uno a uno per ogni dipartimento: arte, suono, animazione (a volte arte e animazione insieme, o *rigging* e animazione insieme), programmazione, UX/UI e così via. A fare la differenza, a parità di ruolo, è il **livello di seniority**.

1.10.2 UX e UI

UX e UI stanno a metà tra game design e arte: a seconda dell'azienda il ruolo si chiama *UX/UI Artist* o *UX/UI Designer*.

UX (User Experience) come una persona interagisce con le cose. Esempio: su uno smartphone, un pulsante accende e spegne lo schermo; per digitare la password si scorre verso l'alto e compaiono i punti da toccare.

UI (User Interface) come sono "vestite" queste cose: ad esempio l'aspetto della barra dei punti vita (HP), realizzata con asset 2D o 3D.

1.10.3 Le specializzazioni del game design

Più l'azienda è grande, più la specializzazione è puntuale.

Gameplay / Core mechanics designer (sono la stessa cosa) si occupa delle meccaniche e delle 3C.

System designer si occupa dei sistemi e delle loro interazioni.

Economy designer si occupa del bilanciamento; lavora soprattutto con fogli di calcolo (Excel, Google Sheets).

Combat designer si occupa del combattimento; negli annunci di lavoro indica quasi sempre un combattimento *action*, con gameplay reattivo (non strategico), anche se non è una regola.

Multiplayer designer si occupa delle interazioni multigiocatore.

Technical designer fa da ponte tra game design e programmazione.

1.10.4 Le specializzazioni del level design

Encounter designer posiziona i nemici nella mappa in modo che gli scontri siano bilanciati e la progressione equilibrata come desiderato.

World / map designer popola il mondo di gioco.

Quest designer popola il mondo con le missioni e pensa alla progressione al loro interno, cioè a dove il giocatore deve andare (si vedano ad esempio gli annunci di lavoro di CD Projekt Red o di Larian).

Technical level designer fa da ponte tra level design e programmazione.

1.10.5 Discipline vicine: suono, UI, produzione

Il monte ore del corso non permette di uscire come *narrative designer* (o *sound designer*) completi. Ciò non toglie che si possano coltivare queste discipline:

- nei progetti di gruppo si può occuparsi anche del suono, della UI/UX, o provare ad applicare metodologie come Agile per fare *producing* e gestire il team;
- il tempo dedicato a queste attività va però sottratto al game design, e dipende da quante persone ci sono nel team, da quanto lavoro c'è e da quanto è grande il progetto;
- avere sviluppato e rilasciato un progetto, anche piccolo (anche una *game jam*), documentando i propri processi, è già un ottimo argomento da presentare in futuro;
- la via migliore resta lavorare nell'industria e, da lì, chiedere di imparare anche altre discipline.

Un gioco si può fare su qualsiasi cosa: sugli scarafaggi, sui piedini delle sedie, sui pennarelli. Si parte avvantaggiati da ciò che è vicino alla propria sensibilità: chi ama la musica può pensare a un gioco con meccaniche legate alla musica, con un gameplay innovativo ma contenuto e realizzabile. Il design è un cappello che comprende molte discipline.

1.11 Approfondimenti emersi in lezione

1.11.1 Roguelike e roguelite

Un esempio di come il marketing conia generi: Supergiant Games, con *Hades*, rese celebre il termine **roguelite** al posto di *roguelike* (pur non essendo il primo roguelite in assoluto). La differenza:

- il **roguelike** si fonda su una serie di punti fissati in una sorta di *manifesto* che definisce che cosa rende un gioco roguelike; uno su tutti: a ogni morte si ricomincia la *run* da capo;
- il **roguelite** rompe alcuni di questi punti, in particolare introducendo una **progressione esterna alla run**: il giocatore può potenziarsi fuori dalla singola partita per vincere più facilmente in quelle successive. *Hades* ne è il caso principe.

Analogamente, un mobile game sviluppato dal direttore del corso fu promosso come "il primo *donation game* al mondo": un gioco che convertiva il tempo di gioco in donazioni reali a enti benefici per animali. Anche qui un neologismo creato lato marketing per definire un genere.

1.11.2 Applicazioni fuori dall'industria videoludica

Le competenze sviluppate non servono solo ai videogiochi:

- i **render architettonici** oggi possono essere realizzati con Unreal, e diversi professionisti formati nel game design lavorano in questo campo;
- la **gamification**, cioè l'applicazione di regole videoludiche a contesti non ludici (mostre interattive, progetti culturali, consulenze come quella per applicare questi concetti alle macchine da allenamento di un'azienda di fitness), è in Italia uno dei modi più redditizi di spendere queste competenze, spesso con finanziamenti pubblici.

1.12 Il primo task

1. Creare un indirizzo e-mail **professionale** (consigliato: un indirizzo Gmail nella forma `nome.cognome@gmail.com`; l'anno di nascita è accettabile). Va bene anche un indirizzo professionale già esistente usato per lavoro. Da evitare indirizzi poco seri ("non `blackfire67`").
2. Una persona della classe raccoglie tutti gli indirizzi in un unico documento e lo invia via e-mail al docente, che aggiungerà tutti al Drive condiviso del corso (dove verranno caricate lezioni, slide e materiali, per avere i documenti tutti nello stesso posto).
3. Installare Discord e iscriversi al server del corso, dove passano le comunicazioni ufficiali. È consuetudine, e utile, creare anche un gruppo tra studenti.

Scadenza: prima della lezione del venerdì.

Punti chiave del capitolo

- Il corso è una **simulazione di lavoro**: ci si tratta da colleghi, con una gerarchia da studio di sviluppo.
- Si impara con l'**inquiry-based learning**: lezione, task, dubbi, domande, condivisione, progetti.
- Il fallimento affrontato con la mentalità giusta è la principale fonte di crescita; i feedback riguardano il lavoro, mai la persona.
- Il lavoro del designer è per metà creativo e per metà **ricerca e metodo scientifico**; il game design non è una scienza esatta.
- **Documentazione** e **pitching** sono il cuore del mestiere.

- Il designer progetta meccaniche e sistemi (partendo dalle **3C**), documenta, media, prototipa, confuta le idee, studia i competitor, bilancia e fa tuning.
- La leadership del designer è **al servizio** del team, non verticale.
- I limiti (tempo, tecnologia, competenze, persone, denaro) restringono e indirizzano la creatività.
- **Le idee sono nulla, l'implementazione è tutto.**

Chapter 2

Che cos'è un gioco: definizioni, componenti e lenti

Il primo capitolo ha descritto il contesto del corso e il mestiere del game designer. Questo secondo capitolo affronta la domanda da cui, in un modo o nell'altro, discende tutto il resto: **che cos'è un gioco?** È una domanda che sembra ovvia. Tutti sappiamo, a livello puramente euristico o olistico, che cos'è un gioco: ognuno ha una lista di Steam o di console piena di titoli, gioca a carte, a giochi di ruolo, a giochi da tavolo. Ma parte del percorso formativo consiste proprio nel **passare dalla visione del giocatore alla visione del creatore**. Un gioco visto dal giocatore e un gioco visto dal designer sono cose molto differenti, anzi, a voler essere precisi, completamente differenti; e questo cambio di *forma mentis*, questo modo diverso di guardare il mondo, si impara soltanto partendo dalle fondamenta.

Il percorso del capitolo ricalca quello della lezione, tenuta a tre voci dal direttore del corso, Diego Ricchiuti, e dai due docenti di game design delle sedi di Milano e Roma, Mattia Podini e Giacomo "Jack" Danna:

1. ogni studente scrive la propria definizione di gioco, e le definizioni vengono analizzate una per una (sezioni 2.2 e 2.3);
2. si confrontano con alcune definizioni "autorevoli" e si arriva a una conclusione spiazzante: il gioco *non è definibile* (sezioni 2.4 e 2.5);
3. se non si può definire il gioco, se ne possono però elencare le **componenti**: gli "ingredienti della zuppa" (sezione 2.6);
4. si studiano tre *framework* che elencano questi ingredienti in modi diversi, quelli di Jesse Schell, di Brenda Romero e Ian Schreiber, e di Stéphane Bura (sezioni 2.7, 2.8 e 2.9);
5. si introduce il concetto di **lente** e il fatto che non esiste un metodo unico di design (sezione 2.11);
6. si stabilisce il **vocabolario** del designer, a partire dalla prima parola che viene bandita dal corso (sezione 2.12);
7. si chiude con alcuni principi della *forma mentis* del designer secondo Chris Bateman e con il task della settimana (sezioni 2.13 e 2.14).

2.1 Una premessa: il direttore del corso

2.1.1 Chi è

La lezione è aperta dal direttore del corso, che si presenta formalmente agli studenti del primo anno (alcuni lo hanno già incontrato al colloquio di ammissione). Dirige il corso di game design da più di dieci anni ed è uno sviluppatore, precisamente un **game designer** e un **direttore creativo**; fa videogiochi da diciassette anni. Tra i titoli a cui ha lavorato, in vari ruoli e in varie parti dello sviluppo, cita:

- un titolo multiplayer legato alla serie *Poppy Playtime*, di cui ha curato la direzione, in uscita nel periodo di Halloween;
- *A Quiet Place: The Road Ahead*, di cui è stato direttore creativo;
- *Star Overdrive*, uscito di recente, e un altro titolo annunciato alla Gamescom;
- *Dying Light 2* e *Warhammer 40,000: Space Marine 2*.

È inoltre autore di due libri: *Game Design Tools*, dedicato alle tecniche di game design, e *Nobody Wants Your Game: The Business Guide to the Game Industry*, dedicato invece al lato commerciale del mestiere (vendere, creare e gestire la propria azienda, capire i costi): una parte "meno romantica" del lavoro, ma sempre molto utile.

Gli studenti lo incontreranno soprattutto al secondo e al terzo anno, ma da quest'anno interviene anche al primo con alcune lezioni "compassate", come questa. Il motivo dichiarato è la passione per l'insegnamento, la volontà di aiutare gli altri a crescere. Nel presentarsi, sottolinea la massima fiducia e il massimo rispetto per i due docenti di game design del primo anno, rispetto che, avverte, "scoprirete non essere così facile da ottenere".

Il suo invito è a dare il 100%: tra un anno, facendo un'introspezione, ci si renderà conto della differenza nelle conoscenze, nelle capacità e anche nella crescita personale. E a fare amicizia: se si è lì, si amano i videogiochi, e attorno c'è molta gente con la stessa passione.

2.1.2 Due consigli dal mestiere

Giocate ai giochi di ruolo. Il primo consiglio, che i docenti danno di solito al secondo anno, è di **giocare a D&D** o, più in generale, ai giochi di ruolo da tavolo: formare gruppi (dal vivo o su Discord, anche in digitale) e giocare. Non deve essere per forza *Dungeons & Dragons*: vanno benissimo *Pathfinder*, i giochi di *Warhammer*, i giochi narrativi moderni, *Vampire: The Masquerade*, *Tenebra*, *Dragonbane* (un ottimo manuale uscito di recente) e la famiglia dei cosiddetti *Borg-like*, giochi costruiti sullo stesso sistema di *Mörk Borg*: *CY_BORG*, *Black Powder & Brimstone*, *Pirate Borg* e altri, tra cui uno ambientato nelle rivoluzioni proletarie. Il gioco di ruolo è **un modo spontaneo e naturale di sperimentare il game design**: c'è tantissimo da imparare, e nel corso dell'anno ci saranno anche lezioni dedicate, orientate non tanto al giocare quanto alla creazione e alla comprensione di questi giochi.

È anche una cosa dell'industria: al DICE Summit, uno degli eventi più importanti del settore rivolto agli sviluppatori (non al pubblico), chi viene selezionato come *master* di D&D riceve un biglietto gratuito; secondo quanto riferito in lezione, il biglietto costa diverse migliaia di dollari.

Prendetevi cura del vostro corpo. Il secondo consiglio è di fare **esercizio fisico**. Il direttore racconta di soffrire di una protrusione alla colonna vertebrale, dovuta a uno sforzo eccessivo, che lo costringe a fare lezione in piedi, ad andare in palestra almeno tre volte alla settimana e a fare ginnastica posturale, perché altrimenti i dolori non lo fanno dormire. Chi fa videogiochi passa moltissime ore seduto davanti a uno schermo: per hobby, per passione, per lavoro e per l'accademia. Una giornata tipo può consistere nel fare lezione seduti, giocare seduti, andare la sera a giocare a carte seduti. Anche una sola volta alla settimana di attività fisica è, nelle sue parole, "il miglior investimento che potete fare sulla vostra carriera", perché la sedentarietà è un *silent killer*.

Una parentesi su Commander. Alla domanda su che mazzo giocherà quella sera a *Commander* (un formato di *Magic: The Gathering* su cui si tornerà nella sezione 2.7), il direttore descrive un mazzo costruito attorno a Mr. Negative, una carta che scambia i punti vita: chi attacca rischia di ricevere i punti vita dello scambio, e questo crea una sorta di *dilemma del prigioniero* in cui nessuno vuole essere l'ultimo a colpire. Il mazzo vince poco, ma genera una pressione psicologica molto interessante. È già un primo esempio di come un designer guarda i giochi: non solo "vinco o perdo", ma *che cosa provano le persone al tavolo*.

2.2 L'esercizio: scrivere la propria definizione di gioco

La lezione entra nel vivo con un esercizio molto semplice nella forma: **scrivere la propria definizione di gioco**. Le regole:

- si scrive su un blocco note digitale o su carta, indifferentemente;
- il tempo è di **cinque minuti** (Jack di solito ne dà due);
- **non** si consultano dizionari, enciclopedie o il *Cambridge Dictionary*: il punto è dare la *propria* definizione;
- una volta scaduto il tempo, la definizione **non va modificata** mentre si ascoltano quelle degli altri. Non c'è nessun premio: è una questione di condivisione e di comprensione.

Anche i tre docenti scrivono (o hanno già pronta) la loro definizione, che verrà letta per ultima.

Le definizioni vengono poi lette una per una e analizzate. Il metodo di analisi è sempre lo stesso, ed è importante capirlo prima di leggere la sezione seguente: per ogni definizione si cercano **gli elementi che include e quelli che esclude**. Ogni parola scelta lascia fuori qualcosa che pure chiameremmo "gioco". I docenti lo ripetono più volte: non si tratta di essere "infami" o di dimostrare che una definizione è sbagliata, ma di mostrare come ognuna contenga *un'idea* di gioco senza riuscire ad abbracciare l'intera definizione di gioco, per quanto ampia possa essere.

Un'avvertenza che emerge subito: **le parole hanno un significato molto preciso**. Se si scrive "spazio" intendendo anche uno spazio immaginario, ma non lo si specifica, si è scritto "spazio", non "spazio immaginario". Allo stesso modo, odiando tutti un po' la matematica, "numeri" e "numeri immaginari" sono cose estremamente differenti: definire i numeri non definisce i numeri immaginari.

2.3 Le definizioni della classe

Si riportano di seguito le definizioni lette in aula (in presenza e online), nell'ordine in cui sono state discusse, ciascuna con il commento dei docenti. Gli studenti sono indicati con il solo nome di battesimo.

2.3.1 Un prodotto con giocatori, regole, spazio, tempo e scopo

> *"Un prodotto a scopo ludico o di intrattenimento che presenta diverse caratteristiche comuni: dei giocatori, delle regole, uno spazio e un tempo definiti e uno scopo e/o degli obiettivi."* (Filippo)

Prodotto è un termine **commerciale**: un prodotto definisce una vendita. Scegliendolo si escludono giochi che chiamiamo tranquillamente tali: *mosca cieca*, ad esempio, non è un prodotto e non può essere venduto. È sbagliato? No: è una definizione.

Scopo ludico implica che un gioco abbia una funzione di intrattenimento. Ma esistono giochi il cui scopo non è affatto ludico. Il caso esemplare è *That Dragon, Cancer*: un gioco creato da uno sviluppatore che racconta la propria vera storia, quella di un padre che deve accettare che il figlio di quattro anni morirà di cancro. È straziante, "una delle cose più brutte che spero non mi accada mai", e giocarlo non ha nulla di ludico; eppure dura circa due ore, è altamente consigliato, è su Steam ed è anche un prodotto. È un gioco.

Spazio (osservazione di Jack) esistono giochi che si svolgono in uno spazio puramente immaginario: dov'è lo spazio di gioco degli indovinelli tra Bilbo e Gollum nello *Hobbit*? Lo studente intendeva lo spazio anche in senso non fisico, ma non lo ha scritto: "spazio" e "spazio immaginario" sono cose molto differenti.

2.3.2 Un'attività tra persone con regole concordate

> *"Attività tra persone che, tramite regole concordate, passano il tempo insieme con il fine di essere intrattenute."* (Damiano)

Attività indica una qualche forma di azione. Esistono però giochi quasi privi di attività: gli *autoplay* e gli *idle* (o AFK), in cui si prende qualche decisione all'inizio e poi il gioco va avanti da sé, o addirittura giochi che "si autogiocano". Su Twitch esistono streamer che fanno giocare animali: un pesce rosso ha finito *Pokémon Rosso* (impiegando l'equivalente di centinaia di giorni) e un pesce rosso ha sconfitto Malenia in *Elden Ring*. Il direttore, che ci ha messo ventiquattro tentativi, si chiede quanti ne abbia impiegati il pesce.

Regole concordate i bambini piccoli giocano molto spesso senza condividere le regole: "siamo pirati, però adesso arriva un mostro, però adesso divento un Super Saiyan, però dovevamo partire da dietro". Il gioco è un cambiamento costante, "melassoso", senza un inizio e senza una fine, e le regole non sono mai state concordate.

2.3.3 Un'attività o un oggetto che stimola emozioni e pensiero

"Un'attività o un oggetto capace di intrattenere il giocatore stimolando emozioni e pensiero." (Federico)

Oggetto è la prima volta che compaiono gli oggetti. Non è sbagliato, ma quando si parla di oggetti spesso si parla di **giocattoli**, e come si vedrà più avanti nel corso c'è una grossa differenza tra un giocattolo e un gioco; ciò non toglie che esistano giochi che sono oggetti.

Stimolare emozioni e pensiero esistono giochi molto passivi, che funzionano in modo estremamente automatico, "che si fanno con metà del cervello spenta, se non tutto". *Watch Paint Dry*, che ha venduto migliaia di copie su Steam, è una schermata monocolore che diventa leggermente più chiara col passare del tempo, perché simula il guardare una parete che si asciuga. *Placid Plastic Duck Simulator*, un successo italiano realizzato da un docente dell'accademia, ha venduto centinaia di migliaia di copie e fatto milioni di euro: è un gioco in cui si guardano delle paperelle di gomma muoversi in un ambiente.

2.3.4 Un prodotto che regala stimoli ed emozioni

"Un prodotto che regala stimoli ed emozioni a chi gioca e in parte anche a chi sviluppa." (Matteo)

Torna il termine *prodotto*, con il suo fine commerciale, che esclude i giochi di parole, i giochi infantili e uno spettro molto ampio di altre attività. Torna soprattutto il discorso delle **emozioni**, che inizia a essere una costante: il cosiddetto **responso emotivo** è un punto molto importante (è presente anche nella definizione di Mattia) e nel corso degli anni si vedranno diversi modi per definirlo. Restano però fuori elementi come lo spazio, il tempo, la condivisione e il fatto che il gioco sia o non sia giocato da un essere umano. Se a giocare a scacchi è una macchina (come i sistemi di DeepMind, *AlphaGo* per il go e *AlphaZero* per gli scacchi), e magari contro un'altra macchina, nessuno prova emozioni: eppure il gioco esiste. Esiste "nel vuoto"; ma una volta messo sul mercato il discorso cambia, e si pone una domanda che tornerà: **se un gioco esiste ma nessuno lo gioca, che valore ha?** È la versione ludica della classica domanda filosofica: se un albero cade nella foresta e nessuno lo sente, fa rumore?

2.3.5 Un'esperienza di immedesimazione e di fuga dalla realtà

"Un gioco è un'esperienza in grado di far immedesimare al meglio il giocatore, suscitare emozioni e sentimenti, far sviluppare senso critico e immaginazione: un modo per fuggire dalla realtà." (Sara)

È all'opposto della precedente: invece che molto ampia, è estremamente stretta.

- **Immedesimazione**: i giochi che non prevedono di immedesimarsi non sarebbero giochi.
- **Senso critico**: il *gioco dell'oca* non fa sviluppare alcun senso critico (nella migliore delle ipotesi è un passatempo); *Cookie Clicker* nemmeno. Secondo questa definizione non sarebbero giochi, come migliaia di altri titoli.

- **Fuggire dalla realtà**: è il punto più interessante. Una delle caratteristiche dei videogiochi (non di tutti, e non dei giochi in generale) è l'immaginazione, il poter creare cose che non esistono nel mondo reale: c'è *Modern Warfare 2* con le armi, ma c'è anche *Skyrim* con i draghi, la fantascienza, *Gothic* con gli orchi ("per fortuna gli orchi non esistono"). Non a caso si parla di *escape from reality*: non solo inventare o immaginare, ma *fuggire* dal mondo reale, che è complesso, difficile e spesso brutto (basta guardare le guerre in corso o millenni di storia umana). Il videogioco può presentare qualcosa di molto diverso, unico, magico e potenzialmente bellissimo.

2.3.6 Emozioni positive e negative

Una definizione successiva (Gaetano) torna sull'**attività**, sulle **emozioni**, sull'**interattività**, ma aggiunge un elemento nuovo: le emozioni possono essere **positive e negative**. In questo modo *That Dragon, Cancer* rientra nella definizione; restano invece fuori i prodotti privi di interattività. E qui nasce una domanda legittima: se un gioco può non essere interattivo, allora anche un film può essere un gioco? La risposta è: *potenzialmente sì*, e ci si arriverà (sezione 2.6). Restano esclusi anche i giochi che non danno emozioni, come *Watch Paint Dry*.

Il concetto di emozione negativa è molto interessante: certi giochi si giocano *per soffrire*, per spaventarsi o per arrabbiarsi. Contro Malenia il direttore ha perso ventiquattro volte: era frustrato e arrabbiato, il controller "ha sofferto", eppure ha continuato a giocare. Quando l'ha sconfitta, il grido gutturale di gioia avrà fatto pensare ai vicini di chiamare il 112; i tentativi precedenti avranno prodotto grida di odio, rabbia e sofferenza.

2.3.7 Un'esperienza individuale o di gruppo, con un obiettivo

> *"Un'attività o un'esperienza individuale o di gruppo con regole e meccaniche prestabilite e un obiettivo da raggiungere, a volte fine all'esperienza stessa, con il fine di intrattenere e far scaturire differenti emozioni al giocatore."* (Francesca)

Individuale o di gruppo elemento non ancora emerso: esistono il *single player* e il *multiplayer*, giochi digitali e non digitali, di gruppo e non di gruppo. Il videogioco può essere un'esperienza solitaria o un'esperienza condivisa.

Obiettivo esistono giochi senza obiettivo. In *Proteus* si cammina in un campo fiorito, e basta: nessun obiettivo, nessun inizio, nessuna fine; a un certo punto, si suppone, ci si annoia e lo si chiude. Un gioco "molto carino", ma senza obiettivo.

A volte fine all'esperienza stessa è la parte più interessante: l'obiettivo può essere *interno* al gioco (sconfiggi il cattivo, salva o conquista il mondo, distruggi l'impero) oppure *non legato al gioco stesso*. In *Magic: The Gathering* lo scopo delle carte sarebbe giocare a Magic, ma esiste un intero gruppo di persone il cui obiettivo è **collezionare** le carte, cosa che non ha nulla a che vedere con le partite, eppure fa parte dell'esperienza di Magic. In altri giochi questa componente è addirittura quella principale: il gioco di carte *Pokémon* soffre in questo periodo di un'eccessiva collezionabilità.

Anche qui, nonostante la definizione sia relativamente ampia, l'aver inserito gli obiettivi esclude i giochi privi di obiettivi.

2.3.8 Un'attività intrattenente basata sull'interattività

> *"Un'attività intrattenente che si basa sull'interattività di colui che ne usufruisce."* (Flavio)

Intrattenente esce dal campo delle emozioni: intrattenere significa anche, semplicemente, **non annoiare**. L'assenza di noia è già una forma di intrattenimento. È il motivo del successo, qualche anno fa, dei *fidget spinner*: non sono piacevoli in sé, ma rispetto alla noia sono meglio. Molte cose sono meglio della noia, che è una delle sensazioni peggiori che il cervello possa vivere.

L'interattività di chi ne usufruisce esclude chi non sta giocando. Ma esistono giochi interattivi *anche per chi non gioca*: i giochi pensati per gli streamer. *A Quiet Place* ha una modalità streaming in cui gli spettatori possono "incasinare" lo streamer: stanno interagendo, ma non sono i fruitori diretti. Sono un **secondo livello di fruitori**.

2.3.9 Vivere una storia in prima persona

> *"Un gioco è un'esperienza interattiva in cui il giocatore vive in prima persona una storia, interpreta un ruolo, compete o collabora con altri giocatori per raggiungere un obiettivo, immergendosi nel contesto creato."* (Francesco)

In prima persona non va inteso nel senso della camera, ma nel senso di impersonare un *avatar* (o *n* avatar). Esistono anche giochi che rendono il rapporto molto più contorto, mettendo il giocatore nei panni di una camera che osserva l'avatar.

Immergersi nel contesto creato esistono giochi **metanarrativi** che separano deliberatamente il giocatore dall'avatar, cercando una *dissonanza ludonarrativa* tra chi gioca e ciò che viene giocato: *Inscryption*, *The Stanley Parable*, *The Beginner's Guide* (dello stesso autore di *Stanley Parable*).

Storia la parola, per quanto ovvia, non era ancora stata usata. Ci sono giochi che sono praticamente storie, come *Gone Home* o in generale i cosiddetti *walking simulator*, e giochi che sono puro gameplay, come il flipper o *Tetris*. Una definizione centrata sulla storia esclude tutti i giochi puramente meccanici.

Contesto esistono giochi in cui il contesto non esiste e viene creato dai giocatori stessi. *Ten Candles* è un gioco di ruolo che si gioca con dieci candele (da cui il nome), e parte del gioco è proprio creare il gioco stesso: il contesto nasce da zero. Capita in moltissimi **giochi di ruolo di stampo narrativo** (in Italia ne pubblica molti, ad esempio, Narrattiva). Tra i preferiti del direttore c'è *Passione delle Passioni*, che simula una soap opera alla *Beautiful*: i personaggi non muoiono mai davvero, e quando il proprio personaggio muore torna due turni dopo con un colpo di scena ("era mio cugino", "in realtà sono naufragato su un'isola deserta e sono sopravvissuto per due anni"). Jack aggiunge che la genialità del gioco sta nel chiedere di interpretare *sia l'attore sia il personaggio* che l'attore interpreta.

2.3.10 Un'avventura da vivere come un alter ego

> *"Un'avventura in cui il giocatore vuole immergersi e viverla il più possibile, come se fosse una sorta di alter ego in cui vivi quel mondo nelle vesti del protagonista."* (Lorenzo)

Una definizione molto propositiva. La parola **avventura** propone già l'idea di qualcosa di più interessante del mondo reale, che è decisamente meno avventuroso. C'è poi un elemento di immersione e soprattutto di **volontà di partecipazione**, implicito ma percepibile, che nessuno aveva ancora citato: la **volontarietà**. Un gioco è un'esperienza volontaria? Dipende: ci sono giochi volontari e giochi non volontari. Con la sola parola "avventura", ovviamente, si potrebbe escludere di tutto.

2.3.11 L'interazione con un medium, con l'ambiente o con la propria immaginazione

> *"L'interazione attiva con un medium, con l'ambiente circostante o con la propria immaginazione, il cui scopo è principalmente quello di divertirsi e farsi coinvolgere."* (Federica)

La studentessa spiega di aver voluto essere molto generica, per includere molte cose: distinguere il gioco dal guardare un film, ma includere anche i giochi della sua infanzia, come "giocare a pensare", immaginare la vita di una persona, o saltare la campana. "Giocare a pensare" è senza dubbio un gioco, e qui si apre un intero mondo non ancora trattato: il **gioco pensato**. Non solo lo spazio immaginario, ma l'interazione immaginaria e le sue conseguenze: l'intera esperienza esiste in un ambito completamente fluido. È ciò che accade ai bambini che giocano insieme pur con regole completamente diverse, mai concordate.

Esiste un'intera branca della matematica dedicata a questi giochi: i **puzzle di logica**, interamente mentali, in cui si devono individuare o capire le regole. Oppure gli indovinelli del tipo "sei su un tetto, come fuggi?", in cui si possono fare solo domande a cui si risponde sì o no, o in cui si deve ricostruire come si è arrivati in una certa situazione: giochi deduttivi, ambientati in uno spazio puramente mentale, che si possono fare anche da soli. In ambito scientifico si parla di **esperimenti mentali**. Uno dei più famosi è il *paradosso dei gemelli* legato alla relatività di Einstein: se uno di due gemelli identici viaggia nello spazio a velocità prossime a quella della luce e l'altro resta sulla Terra, al ritorno saranno ancora identici? La risposta è no. Einstein non ha mai spedito due gemelli nello spazio: era un esperimento mentale.[1]

Jack fa notare che è stata anche la prima persona a usare, al posto di "intrattenimento", il termine *divertimento*: una parola su cui si tornerà prima della fine della lezione (sezione 2.12).

2.3.12 Un insieme di regole e sistemi

> *"Il gioco è un insieme di regole e sistemi con uno scopo ben preciso."* (Patrick)

Compare per la prima volta la parola **sistemi**. Finora si era parlato di regole e obiettivi; i sistemi aggiungono un livello ulteriore, perché le regole si incastrano in strutture più complesse delle regole stesse.

Esempio: il contropiede. Nel regolamento del calcio non esiste la "regola del contropiede". Il contropiede è una tattica che **deriva da come le regole si incastrano**: qualcuno ha capito che, se si lancia la palla dall'altra parte del campo quando gli avversari sono tutti nella propria metà e si corre "come pazzi", si ottiene un vantaggio strategico. Nessuna regola lo dice: è un sistema che nasce naturalmente dal gioco. Allo stesso modo, chiunque abbia scoperto una *build* "rotta" in un gioco ha trovato qualcosa che nessuna regola descriveva ("se fai A più B rompi il gioco"): lo ha scoperto analizzando i sistemi.

Si apre così un intero mondo di cui parlare: che cos'è un sistema? Quando nasce? Che cosa rende un gioco sistemico o non sistemico (si veda anche il capitolo 1)? Dopo una quindicina di definizioni, con altre nove ancora da leggere, nessuno aveva ancora toccato questo punto.

2.3.13 Un insieme di regole da rispettare

> *"Un gioco è un'attività definita da un insieme di regole da rispettare al fine di raggiungere un obiettivo."* (Davide)

Tornano obiettivo e regole, ma l'espressione "da rispettare" porta un elemento nuovo. Quanti hanno **barato** almeno una volta? A Monopoly, prendendo una banconota da cento quando nessuno guarda; a *UNO*, mettendo un +2 su un altro +2, che tecnicamente non è nelle regole ufficiali. Ci sono giochi che addirittura incorporano l'imbroglio: *Munchkin* ha una carta che si chiama proprio "Barare!" e dà il potere di farlo. E anche negli sport si bara: in Formula 1, quasi ogni anno, c'è uno scandalo per qualcuno che ha manomesso il motore o "sistemato" qualche componente. Si è barato, e si è comunque giocato.

Se con Lorenzo era emersa la volontarietà, con Davide emerge l'**onestà**, il tentativo di rispettare le regole: è vero che si bara, ma se ognuno facesse costantemente ciò che vuole il gioco sarebbe puro caos.

2.3.14 Un'attività interessante e non obbligatoria

> *"Un gioco è un'attività interessante, non obbligatoria, per passare il tempo."* (Federico)

- **Interessante** esclude la distinzione tra positivo e negativo: torna l'idea che "piuttosto che niente, è meglio piuttosto".
- **Non obbligatoria** ripropone la volontarietà.

[1] L'effetto di dilatazione temporale previsto dalla relatività è stato poi verificato sperimentalmente, ad esempio con orologi atomici trasportati in volo, ma l'esperimento dei gemelli in quanto tale resta un esperimento mentale.

- **Per passare il tempo** esclude tutti i giochi che hanno una qualche valenza di crescita: ad esempio i primi *Assassin's Creed*, diventati famosi perché in grado di insegnare la storia, a detta di alcuni, meglio dei libri di storia. Ma introduce anche un elemento che nessuno aveva considerato: i **giochi passatempo**. Sono più semplici, più banali se si vuole, ma esistono, e la loro funzione è proprio essere semplici. Lo stesso direttore ha sul telefono giochi che non richiedono grande impegno né grande **carico cognitivo**, adatti ai due o tre minuti di attesa dell'autobus o del treno.

2.3.15 Un regolamento creato da un designer

> *"Il gioco è un modo di svagarsi sottostando a un regolamento"*, a cui lo studente aggiunge: *"... di un designer"*. (Francesco)

Svagarsi richiama il passare del tempo, non necessariamente in modo positivo. La novità è l'aggiunta del **designer**: le regole sono state progettate da qualcuno. Questo, però, esclude il calcio, che è un **risultato evolutivo**: non esiste un inventore del calcio, che ha antenati antichissimi (si citano i giochi con la palla delle civiltà mesoamericane) e si è evoluto nei secoli fino alla forma moderna.

Anche ammettendo un designer, chi è? Il direttore è stato direttore creativo di *A Quiet Place*, "la punta di diamante del design", ma non l'unico designer: il design nasce da una visione di idee e di sistemi che coinvolge più persone. Si potrebbe allora dire "un gruppo di creatori"; ma che cos'è un gruppo, quanto è grande, come lo si definisce? All'estremo opposto ci sono i giochi di Hideo Kojima, che mette il proprio nome ovunque nei titoli di *Death Stranding* ("A Hideo Kojima Game", "Directed by Hideo Kojima"...): nei primi trenta minuti la parola "Kojima" compare un numero impressionante di volte.

2.3.16 Interazioni che influenzano il gioco, sfida e progressione narrativa

> *"Un qualunque medium di intrattenimento con interazioni che hanno un'influenza su di esso, con un livello di sfida e/o una progressione narrativa."* (Eric)

Progressione narrativa una storia che si evolve. Una storia può essere un semplice punto di partenza; qui invece la storia *progredisce*. Attenzione: "progredisce" non implica un finale.

Interazioni che hanno effetto sul gioco esclude le interazioni che non hanno effetto sul sistema, ad esempio quelle **sociali**: in un gioco multiplayer ridere e scherzare con chi sta giocando è un'interazione, ma non modifica il sistema di gioco.

2.3.17 Un'esperienza attraverso uno schermo, fatta di personaggi

> *"Un gioco è un'esperienza, qualcosa che riesce a darti una determinata soddisfazione attraverso uno schermo; è solitamente fatto di storia, personaggi, luoghi, ambientazioni e tanto altro che ti fanno provare certe sensazioni."* (Francesco)

Attraverso uno schermo esclude tutto ciò che non è videogioco o gioco mobile. Ma compaiono per la prima volta i **personaggi**: avatar del giocatore, personaggi, NPC sono elementi estremamente comuni nei giochi e fino a quel momento nessuno li aveva citati direttamente. Esistono giochi senza personaggi? Moltissimi. Esistono giochi basati *interamente* sui personaggi, fatti solo di interazioni con loro? Sì: uno verrà analizzato in una lezione specifica, e non viene nominato per non rovinare la sorpresa.

2.3.18 Un'opera d'arte

> *"Un gioco è un'opera d'arte, un modo evoluto di raccontare un mondo facendolo vivere in prima persona."* (Mattia)

Lo studente precisa che "raccontare un mondo" può avvenire anche attraverso *Pong*, attraverso la simulazione di una persona che gioca a ping pong, o può riguardare la storia stessa dello sviluppo del gioco. L'elemento chiave, però, è l'**opera d'arte**, concetto spesso in contrapposizione con quello di *prodotto* emerso nelle prime definizioni.

Esistono tante opere e poche opere d'arte: di violinisti, compositori e pianisti è pieno il mondo passato e presente, ma tra secoli ricorderemo ancora Mozart. Che cos'è allora un'opera d'arte, e chi la definisce? Una risposta classica è "ai posteri l'ardua sentenza". Un esempio caro al direttore è *Moby Dick*: alla sua uscita il romanzo fu accolto in modo pessimo, fu dimenticato per decenni e venne riscoperto solo molto tempo dopo, quando i critici lo rilanciarono come una delle più grandi opere mai scritte; oggi è considerato da tutti un capolavoro.[2] Non sappiamo quindi come si definisca un'opera d'arte; ma questa definizione aggiunge un **elemento romantico**: il mirare a creare qualcosa di grande, che ci si riesca o meno, una volontà di elevazione.

2.3.19 Divertire, insegnare, far riflettere, meravigliare

> *"Un gioco è un'attività di intrattenimento che ha lo scopo di divertire ma può anche insegnare, far riflettere o meravigliare, e fa tutto ciò in uno spazio di regole e intenzioni definiti."* (Claudio)

Tornano varie emozioni, tornano le regole e torna lo **spazio**, stavolta *definito*, cioè in qualche modo percepibile. Con i bambini non funzionerebbe, ma ci sono giochi in cui la definizione dello spazio è il punto di forza: *Pokémon GO* sovrappone uno spazio digitale allo spazio del mondo reale, e così faceva il suo "papà", *Ingress*.

2.3.20 Un'attività per la persona giocante

> *"Un'attività interattiva svolta per divertire o intrattenere la persona giocante."* (Antonino)

L'elemento nuovo è la **persona giocante**. Molte delle definizioni ascoltate *escludono il giocatore*: non lo nominano come parte della definizione. Si torna così alla domanda posta da Mattia: esiste un gioco senza giocatori? È di nuovo la domanda dell'albero che cade nella foresta, una questione filosofica antica.

2.3.21 Un'attività per una o più persone

> *"Un gioco è un'attività volta al divertimento, che può essere di diverse tipologie e comprendere più persone."* (Valerio)

Molto generica. *Volta al divertimento* esclude i giochi tristi; ritorna invece, rispetto alla precedente, il tema del **gruppo**: uno o più giocatori.

2.3.22 Una promessa e un punto di vista diverso dal proprio

> *"Un gioco promette [...]: permette al giocatore di prendere i panni del protagonista della storia per fargli vivere un'esperienza unica nel suo genere, facendogli vedere la realtà sotto il punto di vista dei personaggi e consentendogli di far valere le sue scelte morali."* (Andrea)

Lo studente premette di aver già capito, ascoltando le altre, che nella sua c'è qualcosa che non va. "Ma in tutte!", è la risposta: non ne è stata trovata nemmeno una senza buchi.

La promessa. *Promettere* è un termine interessantissimo. Quante volte ci si è arrabbiati con un gioco o con un film perché il trailer era diverso dall'esperienza? Un gioco, in un certo senso, deve definire se stesso. Non sempre lo fa: le regole del Monopoly non dicono come si *vive* il Monopoly, e il regolamento del calcio non descrive le emozioni del calcio. Altri giochi invece sì: un *God of War* cerca di spiegare come si giocherà e quale sarà l'esperienza; i libretti di istruzioni dei giochi di qualche generazione fa avevano proprio questa funzione. La promessa vale anche in altri ambiti: un ristorante che si chiama "Sushi King" e poi è una braceria senza sushi è strano, indipendentemente da quanto sia buona la brace.

[2] In lezione l'episodio è raccontato in forma aneddotica (copie bruciate, duecento anni di oblio). Storicamente il romanzo di Herman Melville (1851) ebbe un'accoglienza tiepida e vendite scarse, parte delle copie andò distrutta nell'incendio di un magazzino dell'editore nel 1853, e la sua riscoperta avvenne con la cosiddetta *Melville Revival* degli anni Venti del Novecento.

Il punto di vista dei personaggi e le scelte morali. Il gioco permette di vivere condizioni che non corrispondono necessariamente alla propria attitudine, ad esempio **interpretare il cattivo**. In *Fallout 3* c'è una scelta morale in cui un personaggio (Mr. Burke) offre al giocatore una suite di lusso nella Tenpenny Tower in cambio della detonazione della bomba atomica al centro della città di Megaton. Il direttore e uno studente l'hanno fatta esplodere; un altro studente, "sofferente", ha rinunciato. Nella vita reale nessuno farebbe esplodere una città per una casa; nel gioco sì, perché si sta vivendo l'esperienza di un personaggio cattivo.

In *Baldur's Gate 3* una delle *run* possibili è quella malvagia, con scelte moralmente discutibili. Jack racconta di trovarla straziante: già una delle prime scelte, saccheggiare il Boschetto di Smeraldo invece di proteggerlo e aprire il cancello ai goblin, porta a una scena in cui Zevlor guarda il giocatore non con rabbia, ma con l'incredulità di chi chiede "perché ci stai condannando tutti?". Mantenere una run malvagia è difficile, perché moralmente non si farebbe mai.

Il punto nuovo, quindi, non è la storia o la scelta (già emerse), ma l'**interpretazione di punti di vista potenzialmente non propri**, nel bene e nel male.

2.3.23 Creare una specifica esperienza

> *"Un gioco è un insieme di regole e sistemi con l'obiettivo di creare una specifica esperienza e far provare delle emozioni ai giocatori."* (Tancredi)

L'inizio è simile a quello di Patrick; il termine più importante è **specifica esperienza**. C'è una volontarietà simile a quella del designer citata da Francesco: voler creare qualcosa di specifico, di meno casuale. Eppure esistono moltissimi giochi casuali: la **roulette** è un gioco puramente casuale (se non si bara).

2.3.24 Scenari, meccaniche e personaggi ideati dal designer

> *"Un gioco è un'esperienza attraverso la quale il giocatore, ovvero chi lo gioca, riesce a immedesimarsi e/o a provare emozioni che il designer ha ideato con l'introduzione o la costruzione di scenari, meccaniche e personaggi."* (Angelo)

Compaiono un termine tecnico, **meccaniche**, che verrà studiato a fondo, e il concetto di **scenario**. Uno scenario indica che il contesto, sia fisico sia di spazio (digitale o meno), è stato creato: quando si immagina uno scenario si immaginano tutti i suoi elementi. Nessuno aveva ancora parlato di scenari, che pure sono onnipresenti nei giochi: anche un puzzle è uno scenario, un problema e la sua soluzione.

2.3.25 Un'esperienza sempre diversa

> *"È un medium attraverso il quale si intrattiene il giocatore, con scelte e obiettivi, e quindi ogni volta l'esperienza del giocatore è differente."* (Eric)

- **Ogni volta differente** esclude moltissimo: esistono giochi puramente lineari in cui l'esperienza è sempre identica.
- **Si intrattiene il giocatore** esclude chi non gioca: ci sono giochi che si giocano e giochi che si *guardano giocare*. Il direttore guarda giochi in *blind run* senza commento, proprio per godersi il gioco e non lo streamer. Un gioco può intrattenere anche chi non lo sta giocando.

2.3.26 Un viaggio o una sfida

> *"Un gioco può essere un viaggio piacevole o una sfida, che può essere in compagnia o in solitaria."* (Alberto Maria)

La parola nuova è **sfida**. Nessuno aveva ancora detto esplicitamente che un gioco può essere una *challenge* da superare, qualcosa di complesso: Malenia era stata citata più volte, ma il concetto di sfida era rimasto implicito.

2.4 Le definizioni "autorevoli"

Dopo circa un'ora di definizioni "tutte sbagliate", ci si potrebbe aspettare la definizione giusta. Ne vengono proposte alcune, con un avvertimento: *saranno comunque sbagliate*.

Oxford Dictionary un gioco è un'attività in cui ci si impegna (*engage*) per divertimento (*amusement* o *fun*), come forma di competizione o di sport, giocata seguendo delle regole.

Friedrich Schiller il *play* è "l'espressione priva di scopo di un'energia esuberante" (*the aimless expenditure of exuberant energy*). Qui serve una precisazione linguistica: in italiano "gioco" indica sia lo strumento sia l'attività, mentre l'inglese distingue **game** (il gioco come oggetto, struttura) e **play** (l'atto di giocare). È uno dei rari casi in cui l'inglese è più preciso dell'italiano. La definizione di Schiller è l'opposto di molte di quelle della classe: è un'attività *priva di obiettivo*, e non serve a passare il tempo, ma a spendere un'energia in eccesso, quando in un certo momento se ne ha più di quanta ne serva, senza alcuna ragione.

Tracy Fullerton, Chris Swain e Steven Hoffman (autori di *Game Design Workshop*) un gioco è un **sistema formale** (cioè definito, chiuso) in cui i giocatori interagiscono in una **struttura di conflitto** che si risolve in un **esito ineguale**. *Conflitto* non significa guerra, ma semplicemente un'interazione che porta a un risultato in cui qualcuno vince e qualcuno perde, qualcuno tiene i soldi e qualcuno no. Ovviamente esclude i giochi in cui vincono tutti, come i **cooperativi**.

Una citazione sul play riportata in lezione: "la verità riguardo al *play* è che non può essere definito, così come non può essere definita la vita stessa".

2.5 Il gioco non è definibile

2.5.1 Il gioco come stato mentale

L'ultima citazione è l'intero punto dell'esercizio:

Nessuna delle vostre definizioni è giusta. Nessuna di queste definizioni è giusta. Nessuna delle vostre definizioni è sbagliata.

Il gioco **non è definibile**, perché alla base il gioco è **uno stato mentale**: qualcosa che si interpreta. Si potrebbe decidere di "giocare" il proprio lavoro, di interpretare la propria vita come un gioco. Lo dice bene una citazione da *Mary Poppins*:

> *In every job that must be done there is an element of fun. You find the fun and — snap! — the job's a game.*

(In ogni lavoro che va fatto c'è un elemento di divertimento: trovalo e, schiocco di dita, il lavoro diventa un gioco.) C'è anche un detto italiano simile: "scegli un lavoro che ami e non lavorerai un solo giorno della tua vita". Spoiler: non è così, perché il lavoro resta lavoro; ma è meglio lavorare su qualcosa che si ama che su qualcosa che si odia.

Tuttavia, se il gioco non è definibile, è definibile *ciò che caratterizza* un gioco (sezione 2.6).

2.5.2 Le definizioni dei docenti

Prima di procedere, i tre docenti leggono le proprie definizioni, con lo stesso avvertimento: non vanno prese come "quella giusta". Ognuna esclude una marea di giochi.

Diego Ricchiuti un gioco è un'attività emozionale in cui gli utenti interagiscono con una struttura **ARBITRARIA**, in cui **PERCEPISCONO** un obiettivo e un conflitto. *Arbitrario* indica qualcosa di non equo, deciso da qualcuno che ha potere assoluto, che può essere il gioco stesso o il designer. *Percepiscono* significa che il conflitto può anche non essere presente, ma i giocatori sentono che c'è: sentono di farlo per una qualche ragione.

Mattia Podini un gioco è un sistema basato su regole che evoca **estetiche** di gioco tramite l'interazione (dove, per ora, "estetiche" va inteso come sinonimo di emozioni; il termine verrà formalizzato più avanti).

Jack Danna un gioco è un'attività gestita da regole che coinvolge l'utente tramite il raggiungimento di un obiettivo.

Vengono mostrate anche le definizioni di due ex docenti dell'accademia (che erano stati docenti di game design e level design dello stesso direttore, quando era studente):

Paolo, level designer un'attività coinvolgente in cui gli utenti interagiscono in modo arbitrario con una struttura e percepiscono un obiettivo che si risolve in modo ineguale. Come si vede, molto vicina a quella del direttore ("è stato convertito").

Valerio uno strumento di **esplorazione e apprendimento** (*a tool of exploration and learning*), dove "esplorazione" non è fisica ma significa sperimentare, che usa regole, obiettivi e simulazioni del mondo reale **abbassando il rischio**.

Quest'ultima è molto interessante perché coglie un elemento del *play*: **abbassare la posta in gioco facilita il gioco**. Chi può "giocare" in borsa? Chi è ricco: se perde i soldi, poco importa.

2.5.3 La definizione come bussola

Che cosa farsene, allora, delle definizioni scritte in aula, e perché dedicarvi un'ora? Per due ragioni. La prima è stata vista: capire che il gioco non è definibile. La seconda è più personale:

La vostra definizione di gioco definisce voi come designer.

Va riletta, studiata, analizzata:

- chi ha scritto *storia* ha un animo narrativo;
- chi ha scritto *sistemi* è un futuro *system designer*;
- chi ha scritto *multiplayer* è interessato a quell'ambito;
- chi ha scritto *avventura* è chiaramente interessato ai giochi d'avventura;
- chi ha scritto *emozioni* vede il lato più romantico ed emozionale del gioco;
- chi ha scritto *regole* vede la componente più classica del mestiere, quella del game designer che sceglie le regole del gioco;
- chi ha scritto *personaggi* è interessato alla simulazione delle interazioni umane tramite personaggi: non la storia del grande eroe che salva il mondo, ma l'umano, il piccolo, le persone che si confrontano, le loro vicissitudini;
- chi ha parlato di *risolvere problemi* tende, probabilmente, più al game design che ad altre discipline.

Sono tutte giuste. Sono utili perché diventano una **bussola**: un punto di riferimento per i progetti in accademia, per la carriera futura e per il tipo di designer che si vorrà essere. È inutile che chi ha scritto "narrativa" o "storia" si metta a fare giochi sistemici: non è ciò che gli interessa.

La raccomandazione pratica è di **riscrivere la propria definizione ogni tanto** (non ogni giorno: ogni anno, ogni biennio o più), *senza rileggere quella vecchia.* Le cose evolvono, i gusti cambiano, si giocano giochi diversi, si fanno esperienze diverse, si cresce come designer (che non significa necessariamente migliorare, ma cambiare).

La definizione del direttore mette in risalto, addirittura in maiuscolo, due parole estremamente teoriche: *arbitrario* e *percepito.* Non è un caso: è un designer che adora la **teoria del game design**, che studia libri di game design, psicologia, sociologia e antropologia, che fa ricerca per spingere la teoria un passo più in là. La sua definizione è legata alla percezione, al voler andare oltre ciò che si è già visto. Questo non la rende migliore: la rende una bussola per lui. Se tra qualche anno cambierà (e succederà sicuramente), rappresenterà il suo nuovo modo di affrontare il problema.

2.6 Gli ingredienti della zuppa

2.6.1 La metafora della zuppa

Non tutto è perduto. Se non si può definire il gioco, se ne possono definire le **caratteristiche**. Il gioco è come una zuppa: che cos'è una zuppa? Acqua con delle cose dentro? Una ricetta? Anche senza sapere definire con esattezza se qualcosa sia una zuppa, se ne possono **elencare gli ingredienti**, e da quelli capire se quella zuppa piace o no, o se è davvero una zuppa. Lo stesso vale per il gioco: anche se non lo si può definire, se ne possono definire le **componenti**, e attraverso la loro analisi capire se un gioco rientra nei propri gusti, oltre a molte altre caratteristiche che si scopriranno più avanti.

2.6.2 Raph Koster: artefatti ludici e sistemi ludici

Il primo esempio viene da Raph Koster, autore di *A Theory of Fun for Game Design* e designer noto soprattutto per *Ultima Online*, uno dei primi MMO della storia. Koster, invece di definire il gioco, ne distingue le componenti partendo da due categorie:

Artefatti ludici (*ludic artifacts*): sistemi **progettati per essere giocati** (*designed for play*). Esempi: il *go*, lo splendido gioco asiatico; *Magic: The Gathering*; il football e il calcio; *Werewolf* (in Italia noto come *Lupus in Tabula*); *Space Invaders*; *League of Legends*.

Sistemi ludici (sistemi *suscettibili di gioco*, cioè con cui si può interagire giocando): sistemi che *non* sono giochi, eppure possono essere giocati.

Gli esempi della seconda categoria sono illuminanti:

- **il mercato azionario** (*stock market*): può essere uno strumento per fare soldi, o un modo per passare il tempo, se fare soldi non è l'obiettivo;
- **uno strumento musicale**: si può suonare Beethoven al violino, oppure strimpellare una chitarra, ridere della propria incompetenza e far ridere gli altri, con sé e di sé;
- **la guerra**: per chi la vive è tutt'altro che un gioco, ma esistono i giochi di guerra, e spesso chi fa la guerra per lavoro (i generali) la deve "deumanizzare", valutarla come se fosse un gioco, perché pensando alle vite umane non si riuscirebbero a fare le scelte logicamente corrette;
- **la politica**: il direttore cita un episodio recente di una persona eletta consigliere comunale negli Stati Uniti dopo una campagna nata come scherzo ("votatemi, non sono competente, ma ne ho voglia"). È un caso estremo, ma si dice spesso che i politici "giocano" con la vita delle persone;
- **l'orecchio interno**: chi gira su se stesso (o lo ha fatto da bambino) sta usando la propria conoscenza, empirica o meno, di come funziona l'orecchio interno per crearsi un'emozione, la sensazione di essere storditi (*dizzy*). Sta "giocando" il proprio orecchio interno. Il direttore lo fa ancora con i nipotini: vince chi gira di più su se stesso e poi resta in piedi (e vincono loro);
- **la fisica**: la palla, l'oggetto più basilare del mondo, ha una reazione fisica ed è una forma di gioco con la fisica dell'universo. Basta cambiare quanto rimbalza la palla per cambiare completamente il gioco: si provi a giocare a calcio con il pallone da basket, a basket con il pallone da football americano, a tennis con il pallone da calcio. Non si è cambiata nessuna regola, solo l'interazione fisica dell'oggetto, eppure è cambiato tutto. Lo stesso vale per le palline di gomma dura grandi come una biglia che rimbalzano all'infinito (e si perdono ovunque), o per il salto della corda, dove la reazione fisica è affidata alle competenze del proprio corpo.

Anche i sistemi hanno regole. Mattia aggiunge che anche questi sono sistemi, e quindi hanno regole interne. Nella fisica c'è la gravità, ma c'è anche lo spazio: lo si assume infinito, e ci sono simulazioni che funzionano assumendolo tale, ma a livello computazionale c'è sempre un limite. Un esempio che gli studenti sperimenteranno presto: nelle prime lezioni di level design, quando si impara a modificare il *character*, quasi

tutti aumentano a dismisura velocità di movimento, altezza del salto e *air control*, ottenendo personaggi che saltano letteralmente *fuori dal mondo*. Lo spazio dell'engine è finito (una sorta di cupola), e con un salto abbastanza potente lo si vede rimpicciolirsi mentre si vola "verso l'infinito e oltre". "Lo farete, lo fanno tutti, è normale."

2.6.3 Game e play sono la stessa cosa

Perché questa distinzione? Anche Koster non ha una definizione giusta (nessuno ce l'ha e nessuno mai ce l'avrà), ma la sua divisione mostra che esistono **sistemi progettati per il play** (il gioco "classico", come lo hanno definito gli studenti) e una **mentalità di play** (il gioco inteso in senso largo, lo stato mentale: tutto può essere un gioco, a patto di essere nello stato mentale di giocarlo).

Qual è quello giusto? Sono **la stessa cosa**. Non due facce della stessa moneta: la stessa cosa. Comprenderlo è **il primo segreto per sviluppare giochi**: il gioco è un gioco, ma lo stato mentale è parte del gioco, anzi è il gioco stesso. C'è sempre uno stato mentale legato al gioco, e **la capacità di lavorare con lo stato mentale del giocatore** sarà parte del lavoro del designer. Più si andrà avanti, nel corso dell'anno e negli anni successivi, più si parlerà di psicologia, proprio perché lo stato mentale del giocatore è parte integrante dell'esperienza. Detto in inglese: uno è il *game*, l'altro è il *play*.

2.7 Il framework di Jesse Schell: la tetrade elementare

2.7.1 Che cosa sono i framework

Si entra ora "dentro la zuppa", con strumenti che da qui in poi si chiameranno **framework**: elenchi strutturati delle componenti di un gioco. I primi due sono quelli di Jesse Schell, autore di *The Art of Game Design: A Book of Lenses* (il testo di riferimento del corso, si veda la sezione 1.7), e di Brenda Romero e Ian Schreiber, autori di *Challenges for Game Designers*. Sono tre nomi tra i più importanti della storia del design: Schell, presentato in lezione come designer di grande esperienza anche nel gioco da tavolo;[3] Brenda Romero, una delle più importanti designer di videogiochi; Ian Schreiber, uno dei più importanti teorici del design. Le loro versioni del "contenuto della zuppa" sono completamente differenti.

Schell divide il gioco in quattro elementi, la cosiddetta **tetrade elementare**: meccaniche, narrativa, estetiche e tecnologia.

2.7.2 Meccaniche

Le **meccaniche** sono le procedure e le regole del gioco. Descrivono l'**obiettivo**, come i giocatori possono e non possono raggiungerlo e che cosa succede quando ci provano. Comprendono:

Le regole ciò che si può e non si può fare.

I verbi un altro modo di chiamare le meccaniche: un verbo è letteralmente un verbo (saltare, sparare, mirare). Le azioni del gioco si esprimono in verbi in *-are*, *-ere*, *-ire*.

I pezzi nessuna definizione della classe li aveva citati. I pezzi non sono solo il pedone degli scacchi: è un pezzo del gioco anche il controller, o il mouse. Gli FPS ne suggeriscono uno, i giochi di guida e d'azione un altro, al punto che certi controller vengono venduti a centinaia di euro perché danno un vantaggio (il direttore ne mostra uno con i tasti posteriori). Il termine corretto, come si vedrà, è *tecnologia*.

Schell dice quindi: la prima cosa da definire è l'obiettivo, poi come i giocatori lo raggiungono, che cosa possono e non possono fare, che cosa succede quando ci provano, le regole, i verbi e i pezzi.

[3] Jesse Schell è noto soprattutto per il lavoro ai Disney Imagineering VR Studio, per la fondazione di Schell Games e per l'insegnamento alla Carnegie Mellon University.

2.7.3 Narrativa

La **narrativa** è la sequenza di eventi che si sviluppa nel gioco, che può essere:

- **lineare e prescritta** (*scripted*);
- **ramificata**: non ha un unico finale e, in base a determinate scelte del giocatore, può prendere una strada o un'altra (*Baldur's Gate 3* è, da questo punto di vista, "completamente fuori di testa", con un numero enorme di combinazioni di finali);
- **emergente**: a grandi linee, dipende ed è attivamente influenzata dalle azioni del giocatore nel corso del gioco. Il concetto è molto più ampio e verrà trattato in una lezione specifica.

Due osservazioni importanti.

1. Schell parla della *sequenza di eventi che si sviluppa nel gioco*: non si riferisce solo ai giochi narrativi. Un gioco di auto non ha la narrativa percepibile di un *Life is Strange*, ma ha comunque una forma di narrativa interna.
2. Schell usa il termine **narrativa**, mai *storia*. La narrativa implica moltissime cose: la storia, ma anche i personaggi, il mondo, tutto ciò che riguarda una descrizione. Alcuni giochi hanno una storia, altri un mondo, altri una storia emergente.

2.7.4 Un principio fondamentale sui framework

Da qui discende una regola fondamentale:

> **Per essere valido, un framework deve potersi applicare a qualsiasi gioco.**

Non può esistere un framework che funzioni solo per un tipo di gioco. Tutto quello che Schell dice vale tanto per i giochi da tavolo quanto per i videogiochi. Questo perché Schell si allontana dalla *definizione*, facendo mentalmente lo stesso percorso fatto in aula:

- lo scopo di una **definizione** è *chiudere*: delimitare la parola, darle dei limiti;
- lo scopo di un **elenco di elementi** è l'opposto: *ingrandire*. Se domani Schell pubblicasse un secondo volume, potrebbe aggiungere un quinto elemento, se lo ritenesse necessario. **Non esiste un limite agli ingredienti della zuppa**: la zuppa cresce in base alla necessità.

Mattia aggiunge una precisazione: è vero che i framework sono fatti per raccogliere sotto il loro cappello quanti più giochi e metri di analisi possibili, ma resta vero che spetta alla **sensibilità del designer** scegliere, in base a ciò che vuole fare, a ciò in cui è bravo, a dove lavora, alle risorse e ai limiti (si veda la sezione 1.9). È anche il motivo per cui esistono due framework che dicono più o meno la stessa cosa, con liste diverse e in parte complementari. Quello di Schell, ad esempio, con la sua attenzione alla narrativa, si presta meglio a spiegare un gioco *story driven*; per confrontare due giochi con la stessa tecnologia (ad esempio giocati sulle stesse periferiche) è più adatto quello di Romero e Schreiber. Bisogna iniziare a pensare in termini di **malleabilità** dei framework: quando conviene usarne uno e quando un altro.

2.7.5 Estetiche

Attenzione: si incontreranno **due accezioni diverse** del termine *estetica*. Quella di Schell è la più ampia: le estetiche sono **come il gioco si vede, si sente, profuma, si assapora**. Profumo e sapore non sono un'iperbole: esistono gli scacchi alcolici, in cui ogni pezzo mangiato è un bicchierino da bere; il *chess boxing*, che alterna partite a scacchi e round di pugilato; i libri di Geronimo Stilton con le pagine profumate. In generale: **come il gioco viene percepito**.

Le estetiche sono un aspetto incredibilmente importante perché sono **la relazione più diretta del giocatore con l'esperienza**: il modo più diretto con cui il giocatore interagisce con il gioco, e la parte "finale" di ciò che percepisce. Da giocatori si vede come il gioco si presenta, quanto è piacevole giocarlo, quanto è difficile compiere un'azione, che cosa significa lanciare una palla di fuoco in *Super Mario*: vedere

la palla di fuoco. Quella però è la **vestizione** applicata sopra le meccaniche. La meccanica è "se A, allora succede B"; come B si presenta, di che colori è, che suono fa, se c'è o no la vibrazione (*rumble*, cioè feedback aptico sul controller), tutto questo ricade nel regno delle estetiche. Più avanti si vedrà una terminologia più formalizzata che separa ancora meglio i livelli di estetica e di meccanica.

Esempio: il coltello. Si immagini il proprio gioco preferito, ma ogni volta che si subisce un danno qualcuno compare dal nulla e accoltella il giocatore. Probabilmente non sarebbe più il gioco preferito. È un caso ridicolo, ma mostra che se la relazione estetica del gioco fosse il dolore, quasi tutti abbandonerebbero (non tutti, però, perché c'è chi apprezza il dolore). Gli elementi visivi, interattivi, perfino fisici sono parte dell'esperienza.

Esempio: le miniature. Chi gioca con le miniature si è punto innumerevoli volte con le lance (ad esempio di *Warhammer Fantasy*), soprattutto in fase di montaggio: un misto di amore e odio che fa parte del percorso. Una miniatura di metallo, di plastica, di plastica con un leggero strato metallico magnetizzabile, di ghisa: ognuna dà sensazioni diverse. Lo stesso vale per le carte *foil*, con la loro sensazione al tatto e la loro piegatura. Sono tutte **sensazioni tattili**, elementi fondamentali.

Esempio: Raggio Aurora. In *Pokémon* molte mosse sono, dal punto di vista meccanico, semplicemente una statistica di danno e un tipo: Braciere (fuoco) e Pistolacqua (acqua) sono meccanicamente quasi identiche, cambia solo il tipo. Eppure tutti hanno mosse preferite, non perché più efficaci, ma perché l'animazione è spettacolare. Il direttore, che ha giocato *Pokémon Rosso e Blu*, ricorda ancora **Raggio Aurora**, la cui animazione ("il raggio epilessia") è diventata sempre più vistosa di generazione in generazione. Raggio Aurora è una mossa mediocre: in competitivo nessuno la usa, perché esiste Geloraggio, che ne è la versione migliore. Eppure è la mossa che ricorda, per la sua **componente estetica**, che per i suoi gusti (non in assoluto) la rende più interessante di altre.

2.7.6 Tecnologia

La **tecnologia** è tutto il materiale, tecnologico o fisico, che riguarda il gioco: il computer su cui gira, l'engine che lo fa girare, il controller, il materiale delle miniature. È importante perché cambia l'esperienza:

- il *feeling* di una carta o di una miniatura cambia da materiale a materiale; giocare con un controller o con mouse e tastiera è diverso;
- se si fa girare *GTA V* su un vecchio portatile a 6 fps, l'esperienza è pessima: del gioco non è cambiato nulla, tranne il fatto che gira a 6 fps. È la ragione dei **requisiti minimi** e dei **requisiti consigliati** per un'esperienza decente;
- in *Pokémon GO*, che usa la posizione reale del giocatore, alcuni streamer sono stati derubati del telefono dopo essersi avventurati, per catturare un Pokémon raro, in zone poco sicure delle città: casi estremi, ma che mostrano quanto la tecnologia influenzi il gioco;
- in *Invizimals* (su PSP) si acquistavano in edicola delle carte o miniature dei mostri, che venivano poi "catturati" inquadrandoli con la fotocamera della console: anche questa è tecnologia.

Esempio: Commander e le cento carte. *Magic* nasce con mazzi da 60 carte. *Commander* è un formato creato dalla comunità, poi diventato famoso e consolidato, che si gioca con mazzi da **100 carte**. Qui emerge un limite della tecnologia di Magic: **mescolare cento carte è estremamente difficile e noioso**. In un gioco in cui mescolare è un'azione abbastanza frequente, questo diventa parte di un'esperienza negativa, tanto che esistono tavoli in cui i giocatori si accordano per non inserire carte che fanno mescolare il mazzo, così da farlo solo all'inizio. Il problema nasce dal fatto che Magic è stato pensato con carte di un certo peso, dimensione e costo, che funzionano in mazzi da 60, non da 100. Si potrebbe sostenere che non sia un errore di Wizards of the Coast, dato che il formato non è nato da loro; ma il punto è che la tecnologia cambia l'esperienza.

2.8 Il framework di Brenda Romero e Ian Schreiber

2.8.1 Perché più punti di vista

Si guarda ora il contenuto della zuppa dal punto di vista di Romero e Schreiber. Non perché siano più bravi di Schell, o viceversa, né perché la visione del direttore sia migliore delle loro: **non esiste un framework migliore**, così come la divisione di Koster non è migliore o peggiore delle altre. Mostrare la stessa cosa da più punti di vista serve a capirne la complessità; inoltre ognuno tenderà per natura a sentirsi più affine a uno o all'altro, perché ognuno ha una propria visione creativa, e lavorare con uno strumento allineato al proprio modo di pensare è più facile. Conoscere modi di pensare diversi serve proprio a questo.

2.8.2 Gli elementi

Game qualities (qualità di gioco) al posto delle meccaniche, Romero definisce i **sistemi di gioco**. Invece di concentrarsi su saltare, mirare, sparare, ricaricare, schivare (*dodge*), parare, attaccare (tutte meccaniche, tutti verbi), si ragiona per gruppi: ad esempio la qualità *shooting* include tutte le caratteristiche legate alle meccaniche di tiro, che non necessariamente sono tutte presenti in un dato gioco (in *Doom* non si ricarica, ma è comunque uno shooter). Altri esempi sono *PvP* e *PvE*: se un gioco è PvP ha il multiplayer, e non serve definirne la meccanica, è sottinteso; se ha il PvP avrà probabilmente un sistema di barre della vita o qualcosa di simile, e così via.

Giocatore chi gioca? Perché gioca? Che attività sta svolgendo? Che cosa desidera, che caratteristiche ha? È un elemento fondamentale, su cui si tornerà in altre lezioni: **il giocatore è parte integrante del gioco**. Se si mette il direttore a giocare a *Canasta*, non gli piace: non è il suo gioco. Questo non significa che Canasta sia un brutto gioco, ma che non è il gioco per lui, pur avendo la stessa tecnologia (le carte) di giochi che ama e molte caratteristiche simili (interazione con altri giocatori, combinazioni di carte, strategia). Allo stesso modo, nessuno gioca a *Tekken* "sul serio" contro un nipotino di tre o quattro anni: si sceglie piuttosto un gioco di macchinine, o *Mario Kart* rendendogli le cose più facili. La **difficoltà** dipende dal giocatore.

Tema tutti gli elementi legati alla storia e/o al **significato** del gioco. È simile alla narrativa di Schell, ma aggiunge il significato, che include ad esempio la comprensione di un sottotema filosofico o narrativo del prodotto.

Obiettivo lo scopo del gioco: che cosa si cerca di ottenere e di raggiungere. Molti studenti lo avevano inserito nella propria definizione.

Regole qui sono una struttura separata, che definisce soprattutto **ciò che il giocatore non può fare**. Nelle meccaniche di Schell le regole definivano anche ciò che si può fare, ciò che si vuole ottenere e che cosa succede; qui ciò che si può fare è già definito nelle *game qualities*.

Risorse (illustrate da Mattia) sono i *game pieces* che possono essere manipolati, usati, consumati, collezionati o modificati: elementi che possono cambiare, progredire o essere spesi all'interno dell'esperienza. In *The Witcher* sono risorse i soldi, ma anche armature e armi, e anche le carte di *Gwent*, con cui si costruisce, migliora e potenzia un mazzo.

Interazione la **sequenza di causa ed effetto**.

2.8.3 Confronto con Schell

Molto di ciò che Schell raccoglie in un unico punto, le meccaniche (con regole, verbi e pezzi), qui è sparso in più elementi: le *game qualities* lavorano a un livello più alto, le regole a un livello più basso (con particolare attenzione a ciò che non si può fare); regole e risorse insieme iniziano a dare un quadro molto simile alle meccaniche di Schell.

C'è però un rischio: nei giochi moderni, l'elenco delle risorse può essere **potenzialmente infinito**. In un *Final Fantasy* ogni pozione è una risorsa, ogni attacco, ogni personaggio, ogni punto vita, ogni punto

esperienza: è facile perdersi in una lista senza fine. Così, mentre sulle *game qualities* Romero sembra fare un lavoro di "asciugatura", sulle risorse il framework esplode e vuole definire ogni singolo elemento.

Anche l'**interazione** è interessante: in Schell causa ed effetto restano impliciti (se si definisce che cosa si può e non si può fare, come lo si fa e che cosa si vuole ottenere, si sta facendo come nel calcio, lasciando implicito il contropiede); qui invece si vogliono definire: "il giocatore potrebbe fare il contropiede".

Esempio: il camping. Nei giochi multiplayer il *camping* è l'atto di appostarsi in un punto particolarmente vantaggioso e strategico, allo scopo di eliminare gli altri giocatori che passeranno da lì, riducendo al minimo il proprio rischio. È una strategia validissima, solitamente considerata negativa perché ritenuta meno *skillful* (non è vero, ma è così che viene percepita). Nel framework di Romero e Schreiber il camping va inserito tra le interazioni; in Schell resta non definito. Il che non significa che non esista: semplicemente non è esplicitato.

Ancora una volta: nessuno dei due è migliore o peggiore. Sono due punti di vista.

2.9 Il framework di Stéphane Bura

2.9.1 Una tabella a incroci

Il terzo framework, introdotto da Jack, è quello di Stéphane Bura. A differenza dei precedenti, **si legge come una tabella a incroci**: non basta considerare i singoli termini, bisogna leggerli in combinazione. Sulle righe ci sono quattro **livelli**, sulle colonne tre **dimensioni**.

Action (azione) ciò che il giocatore fa, ciò che ha la possibilità di compiere. È il livello più basso, quello del *second to second*, del gameplay più rapido: la chiave che apre una porta, l'attacco base.

System (sistema) ciò che fa il gioco. È un livello superiore: finisco questo bioma e passo a un altro, vado di qua o di là.

Self (sé) l'interazione del giocatore come singolo individuo con il gioco: interazioni che non si limitano a sfruttare il sistema (quello è *action*), ma ne fanno un **uso creativo o strategico**.

Social (sociale) tutto ciò che è legato alla comunità: il giocatore immerso in una community, la sua capacità di relazionarsi con gli altri.

Ciascuno di questi livelli si articola in tre dimensioni: **Freedom** (libertà), **Mastery** (maestria, padronanza) e **Data** (dati, informazioni).

	Freedom	**Mastery**	**Data**
Action	opportunità, abilità	competenza nelle azioni a basso livello, spesso fisica	risorse del mondo di gioco, collezionabili, regole operative
System	esplorazione, sperimentazione, scopo	competenza sui sistemi complessi	regole e loro comprensione, sistemi che trasformano le risorse
Self	strategia, creatività	etica, sensibilità personale nell'uso del sistema	*mementos*, achievement, memoria
Social	community, supporto, esperienza condivisa	abilità sociali (mentire, leggere, collaborare)	meta, metagaming

Table 2.1: Il framework di Stéphane Bura come illustrato in lezione.

2.9.2 La colonna Freedom

Action × Freedom: opportunità e abilità una delle caratteristiche tipiche di un gioco è dare al giocatore delle abilità e lasciarlo libero di sceglierle. Il "mega pugno della morte" si attiva con L2 più quadrato quando si vuole: si è liberi, ed è un'azione.

System × Freedom: esplorazione, sperimentazione, scopo si è liberi di andare dove si vuole, ma *entro i limiti del gioco*. Se c'è un muro invalicabile, il muro è invalicabile; se il personaggio non sa nuotare, in acqua non si va. Il sistema limita la libertà.

Self × Freedom: strategia e creatività in *Minecraft* il *mine* (dare un pugno e rompere qualcosa) è un'azione. Capire che scavando un fossato abbastanza largo e profondo i creeper ci cadono dentro di notte, e quindi non si muore, non è un'abilità (non esiste un tasto che fa comparire un fossato): è un'**applicazione strategica o creativa del sistema**. Lo stesso vale per l'uso creativo di Minecraft: c'è chi ha ricostruito in scala 1:1 il continente di Westeros del *Trono di Spade*, cosa che nulla nel gioco chiede di fare. Jack cita anche *The Uncensored Library*, un progetto di Reporters sans frontières: una biblioteca costruita in Minecraft che raccoglie articoli e testi censurati in vari paesi, consultabili proprio perché si trovano dentro un gioco.

Social × Freedom: community, supporto, esperienza condivisa la libertà di entrare in una comunità, interna al gioco (una gilda) o esterna (un canale Discord), e di interagire attraverso di essa.

2.9.3 La colonna Mastery

Action × Mastery tutto ciò che riguarda lo sviluppo di competenza nelle azioni a basso livello, che a volte sono abilità molto fisiche: saper eseguire un attacco, combinare dei tasti. I *rhythm game*, come *osu!* o *Dance Dance Revolution*, richiedono proprio questo tipo di maestria fisica.

System × Mastery la competenza sui sistemi complessi. In *Pokémon* è la capacità di fare *team building*: costruire una squadra bilanciata con un attaccante fisico, un attaccante speciale, un "muro". In un gioco di carte come *Yu-Gi-Oh!* è sapere la ripartizione classica del mazzo (ad esempio 20 mostri, 10 magie, 10 trappole). Quanto bene si sono compresi i sistemi e come li si può sfruttare.

Self × Mastery un discorso legato a una certa **etica**, cioè alla sensibilità personale nell'uso del sistema (si veda l'esempio di Shudderwock qui sotto). Vi rientrano anche le creazioni complesse, come quelle di Minecraft.

Social × Mastery quanto si è bravi nelle relazioni sociali e nello sfruttare abilità sociali. In *Among Us* e in qualsiasi gioco di deduzione sociale: quanto si è bravi a mentire, a far credere di non avere un certo ruolo o di essere amichevoli; ma anche a leggere gli altri e a collaborare con loro.

Esempio: Shudderwock in Hearthstone. In *Hearthstone* le carte si giocano in un ambiente digitale, con animazioni, e ogni turno ha un limite di tempo. Quando fu introdotta la carta **Shudderwock**, il suo effetto era: quando viene giocata, riattiva tutti i *grido di battaglia* (*battlecry*, gli effetti che si attivano quando una carta viene messa in campo) giocati fino a quel momento nella partita. Nacque così un mazzo specifico, non previsto, riempito di carte con grido di battaglia; spesso lo Shudderwock veniva anche riattivato una seconda volta, facendo scattare tutti quegli effetti due volte. Le animazioni erano così lunghe da sforare nel turno dell'avversario, "mangiandoselo": chi giocava il mazzo poteva finire il proprio turno, mentre le animazioni continuavano e l'avversario di fatto non giocava. Una volta conosciuta questa cosa, **quanto è etico giocare quel mazzo**, sapendo che è estremamente facile vincere sfruttando una lacuna, un bug o qualcosa che gli sviluppatori non avevano previsto e che probabilmente verrà corretto? Questa è la dimensione *Mastery Self*. (Il problema fu poi risolto: Shudderwock fu indebolito e il sistema del tempo modificato.)

Jack aggiunge che una recente espansione di Hearthstone ha introdotto una meccanica, chiamata in italiano *Innesto*, che divide l'effetto dei servitori in due parti, la *condizione di attivazione* e la *conseguenza*, e permette di fonderne più di una. Ne nascono combinazioni estreme: il classico Doomsayer ("all'inizio del tuo prossimo turno distruggi tutti i servitori") fuso con un grido di battaglia diventa "grido di battaglia: distruggi tutti i servitori"; oppure "ogni volta che peschi una carta, pesca una carta", con effetti facili da immaginare (un "Vaso dell'Avidità" potenziato all'infinito, per citare *Yu-Gi-Oh!*).

La difficoltà nei framework. Uno o due studenti avevano citato la difficoltà, la sfida, nelle proprie definizioni. Non compare esplicitamente né in Schell né in Romero e Schreiber, mentre in Bura c'è, sotto forma di *mastery*, cioè la possibilità di diventare più bravi. Significa che per Schell e Romero la difficoltà non conta? No: la fanno rientrare in altri punti (nelle meccaniche, nel giocatore). **Gli ingredienti della zuppa sono sempre gli stessi**; i designer si concentrano su elementi diversi in base ai propri interessi, al proprio modo di pensare e di creare.

2.9.4 La colonna Data

Action × Data: risorse, collezionabili, regole operative le monete di *Super Mario*: si raccoglie una risorsa con una micro-azione che produce un cambiamento in quelle che Romero chiama *risorse*.

System × Data qualcosa di più complesso: le regole e la loro comprensione (tutorial, spiegazioni), ma anche i sistemi complessi che incidono in modo importante sulle risorse. Giochi come *Civilization* si basano interamente sull'acquisizione, sull'accumulo e sull'"esplosione" (aumento esponenziale) delle risorse; così molti strategici in tempo reale, come *Age of Empires*, basati sulla raccolta di risorse.

Self × Data: mementos, achievement, memoria ciò che il giocatore si porta via e che riguarda lui. Spesso sono *mementos*, ricordi: la *photo mode*, la propria *build*, il personaggio creato in un certo modo nell'editor iniziale. Il direttore racconta di aver creato, in un soulslike prima di una patch, un personaggio con un naso così lungo da deformare tutti gli elmi (che si adattano alle ossa del volto): nessuna funzione, nessun effetto sul gioco, ma "mi sentivo figo". Ancora più chiaramente rientrano qui gli **achievement**: per chi "platina" i giochi, ottenerli è parte integrante dell'esperienza.

Social × Data: il meta i dati sociali sono legati al **metagaming**: non gioco quella cosa perché non vince in classifica (*ladder*); gioco quel personaggio perché vince; uso quel mouse con la rotellina magnetica perché dà certi vantaggi.

Mastery o meta? Una domanda dall'aula: lo sfruttamento del meta (*Social Data*) non si confonde con la padronanza dei sistemi (*Mastery System*)? La risposta: la **mastery richiede una competenza**, il **meta richiede una conoscenza**. Se si mette qualcuno al volante di una Formula 1 Ferrari, la sua capacità di guidarla è la sua mastery: può crescere nel tempo, e dopo due anni sarà un pilota migliore. Il fatto che la scuderia gli dica "questa curva va presa a 197,5 km/h in quinta" è meta.

Un altro esempio: quanti costruiscono da soli la propria build o il proprio mazzo, invece di cercare online il più efficace? È normale divertirsi a costruire qualcosa di assurdo, ma quando si vuole qualcosa di efficace si cercano dati online, perché qualcuno ha già fatto i calcoli. Imparare poi a *usare* quel mazzo è un discorso separato.

Nei picchiaduro (*Mortal Kombat*, *Dragon Ball FighterZ*, *Dragon Ball: Sparking! Zero*, il seguito dei *Budokai Tenkaichi*) il meta può dire, per esempio, che Vegeta Super Saiyan 2 è il miglior *rusher*, cioè il personaggio migliore per andare addosso all'avversario, sulla base di dati calcolati (velocità delle animazioni e così via). Ma il direttore, che in questi giochi usa gli *zoner* (personaggi che combattono a distanza), perderebbe comunque tutte le partite con quel Vegeta. Conoscere l'informazione è un vantaggio, perché rende statisticamente più probabile vincere con qualcosa di matematicamente migliore; ma **sapere un'informazione non significa saperla pilotare**.

2.10 Il framework del direttore

Per mostrare ancora una volta che non esiste un framework giusto, il direttore mostra il proprio, che verrà spiegato più avanti nel corso. I suoi elementi sono: **player**, **meccaniche**, **dinamiche**, **tema** e **tecnologia**.

Ogni studente, col tempo, svilupperà il proprio modo di interpretare gli ingredienti della zuppa: un mix di questi framework e di altre cose, senza limitarsi a pensare "non lo aveva scritto Schell, non lo aveva scritto Romero, quindi non posso inserirlo". Prima, però, bisogna imparare perfettamente ad analizzare un gioco con i framework esistenti, perché a questo punto della carriera, statisticamente, nessuno è ancora Jesse Schell, Brenda Romero o Raph Koster. All'inizio ci si siede **sulle spalle dei giganti** (*standing on the shoulders of*

giants): loro hanno già fatto questo lavoro, e non serve ripartire da zero. Col tempo ognuno avrà il proprio, che potrebbe rivoluzionare l'industria o essere semplicemente uno strumento personale di lavoro, come quello del direttore.

2.11 Le lenti: non esiste un metodo unico

2.11.1 Il principio più importante

Qual è, dunque, il framework migliore? Nessuno: sono **lenti** (*lenses*), ed è il motivo per cui il libro di Schell ha come sottotitolo *A Book of Lenses*. È il primo e più importante principio da ricordare, da ora fino alla fine della propria carriera di designer, e non solo di videogiochi:

Non esiste una soluzione. Non esiste un santo Graal. Non esiste un unico metodo che risolva tutti i problemi.

Questo è al tempo stesso *orribile* e *meraviglioso*:

- orribile, perché a volte ci si scontra con problemi che sembrano insormontabili, e trovarsi davanti a qualcosa di apparentemente impossibile non è piacevole;
- meraviglioso, perché **tutto è superabile**: bisogna solo cambiare la lente con cui lo si guarda.

2.11.2 La migliore difesa contro l'intelligenza artificiale

C'è anche un secondo aspetto: questa è **la più grande protezione che il designer ha contro le intelligenze artificiali**. Perché un'AI è brava a programmare? Perché molti problemi di programmazione si risolvono in un modo noto. L'ordinamento (*sorting*), cioè prendere qualcosa di disordinato e ordinarlo, ha algoritmi già risolti: esistono e sono quelli. Non esiste invece un "sorting delle meccaniche": non si riesce nemmeno a definire che cos'è un gioco, figuriamoci a ordinarlo. Se non esiste un'unica soluzione, non esiste un algoritmo; e allora la creatività e la capacità di trovare metodi diversi restano il sistema migliore per fare design.

Il rovescio della medaglia è che bisogna imparare **più metodi possibili**. Si può avere un metodo preferito, ad esempio Romero, che risolve il 90% dei problemi; poi arriverà il problema che non risolve. Allora si prova Schell; se non funziona, un terzo, un quarto, un quinto, un sesto. I docenti ne forniranno moltissimi (non si chiede agli studenti di inventarli), e prima o poi uno funziona. Il prezzo è che, per sfortuna, si può doverli provare tutti e scoprire che era l'ultimo: ci si sarà messo più tempo del previsto. Per questo bisogna **sempre guardare tutto da più punti di vista**.

2.11.3 The right tool for the right job

Come dice Mattia, *the right tool for the right job*: alcuni framework funzionano meglio con certi giochi, altri con altri. E c'è un ulteriore discriminante: **quelli che funzionano meglio con voi**, con cui ci si trova più a proprio agio. Jack racconta di conoscere molti framework ma di usarne prevalentemente tre: se nessuno dei tre funziona passa ai successivi, ma non avrebbe senso non partire da quelli, perché sono diventati il suo *bread and butter* ("pane e burro", o "pane e marmellata" all'italiana): qualcosa di talmente rapido e naturale da non costare fatica.

2.11.4 Il Deck of Lenses

Il libro di Schell ha anche una versione in forma di mazzo di carte, il **Deck of Lenses** (ne esiste anche una versione online). Il funzionamento è semplice: davanti a un problema, si estrae una carta a caso e si prova ad analizzare il problema secondo quella lente.

Esempio: la lente della cooperazione. Si immagini di essere gli sviluppatori di *Hearthstone* alle prese con il problema di Shudderwock. Si mescola il mazzo e si estrae la *lente della cooperazione*. Ci si chiede, ad esempio, se nel problema ci sia abbastanza *comunicazione*. Ne nasce un'idea: un sistema per cui, se si gioca un mazzo con Shudderwock, gli altri lo sanno e possono decidere di non partecipare alla partita conoscendone il rischio. È una buona soluzione? Non si sa; ma è emersa ponendosi una domanda attraverso una lente.

Esempio: la lente della sfida. Si estrae un'altra carta, la *lente della sfida* (*challenge*): "la sfida è al centro di quasi tutto il gameplay; si potrebbe perfino dire che un gioco è definito dai suoi obiettivi e dalle sue sfide". La domanda da porsi è: *quali sono le sfide del mio gioco?* La sfida di Hearthstone è costruire un mazzo e pilotarlo. Il problema di Shudderwock, allora, è che è un mazzo *meta*: tutti possono copiarlo e applicarlo, togliendo parte della sfida. Una soluzione (inventata sul momento) potrebbe essere oscurare i siti che diffondono il meta. Probabilmente è un'idiozia, perché non si possono bloccare tutti i siti; ma, di nuovo, è emersa analizzando il problema da un punto di vista specifico.

Questo è lo scopo dei framework forniti nel corso: **analizzare i problemi da punti di vista diversi**, scoperti da designer non necessariamente più bravi degli studenti, ma con più esperienza. Magari un giorno qualcuno sarà più bravo di Jesse Schell (e ce lo si augura sempre: la qualità del design mondiale può solo beneficiarne); ma per natura ora si ha meno esperienza, e sedersi sulle spalle dei giganti fornisce un'intera serie di lenti.

2.12 Il vocabolario del designer

2.12.1 La metafora del medico

Mattia introduce l'ultima parte con una metafora. Quando si va dal medico e si dice "sto male, mi fa male la pancia, ho un fastidio generico", il medico non può aiutare. Il medico *indaga*: tocca, palpa, fa altre domande, raccoglie dati, cerca di isolare il problema. Nel game design funziona allo stesso modo. L'obiettivo finale riguarda il **responso emotivo** del giocatore: bisogna capire se è quello desiderato, se il giocatore sta provando le emozioni che si vuole provi in quel momento (e anche che tipo di giocatore sia). Se la gran parte dei giocatori prova le emozioni desiderate, è un grande successo; se no, bisogna indagare.

Ma già a monte bisogna stabilire, con il team, **che cosa il gioco deve far scaturire nel giocatore**. Dire "il nostro gioco deve essere divertente" non basta: non significa nulla, non dà alcuna informazione. Bisogna delimitare il problema, trovare **l'emozione specifica** o l'insieme di emozioni specifiche, di diversa natura e diversa intensità, che il gioco deve far provare.

2.12.2 La parola proibita

Da qui una regola del corso, tra le più importanti in assoluto: **non si usano più le parole "divertente", "intrattenente" e simili**, né le loro varianti o sinonimi (*fun*, *immersivo*...). Sono parole che non vogliono dire nulla. La regola vale in qualunque task e in qualunque conversazione, dentro e fuori dalla lezione. Chi la infrange viene "punito" (non fisicamente, ovviamente) con **task aggiuntivi**, con modalità diverse a seconda del docente:

- Jack applica una punizione *collettiva*: se una persona usa la parola proibita, **tutta la classe** riceve un task aggiuntivo. Il motivo è pratico: con gli studenti dell'anno precedente, che facevano molta fatica a smettere, ha funzionato proprio perché il peso non ricadeva solo sul singolo, ma su tutti;
- Mattia punisce il singolo, ma in modo *progressivo*: più volte si usa la parola, peggiore è il gioco da analizzare nel task aggiuntivo, partendo da un gioco "mediamente terribile" fino ad arrivare a titoli come *Hatoful Boyfriend* (un *dating sim* con i piccioni) o a simulatori di appuntamenti con gli insetti.

In aula uno studente racconta che uno studente del terzo anno gli aveva già consigliato, il primo giorno, di "dimenticare la parola divertente"; e nel raccontarlo la pronuncia, guadagnandosi il primo task aggiuntivo del corso.

Perché tanta severità? Perché la parola proibita è un concetto **completamente inutile**: non esprime *perché* un gioco funziona. Un gioco funziona perché suscita emozioni, e quelle emozioni vanno definite

attraverso un sistema di design. Anzi, come precisa Mattia, è inutile *nella migliore delle ipotesi*: quando la parola arriva da altre discipline o, peggio, da altri designer, diventa ancora più difficile capire qual è il fulcro del problema o qual è l'equivalente emotivo corretto. Mettere nella *visione* del progetto (concetto che verrà approfondito), tra i pilastri del gioco, "deve essere divertente" non significa nulla e crea problemi a tutto il team. In più, toglie **credibilità** a chi parla.

Non si tratta di una stranezza dei docenti: **parlare come un designer significa essere riconosciuti come designer**. Il vocabolario è parte integrante del mestiere, perché il designer è un **comunicatore**: comunica con il team, con il giocatore e con altri designer. Si vedranno tecniche di comunicazione e si imparerà a fare documentazione, che non è altro che una forma di comunicazione scritta (esistono la comunicazione orale, verbale, non verbale e scritta).

2.12.3 Non solo "divertente": anche "bello", "valido", "noioso"

Una studentessa chiede se la parola *bello* sia equivalente. Sì: dire "la narrativa è bella" è altrettanto inutile. *Schindler's List* è un libro meraviglioso, ma non è "bello" nello stesso modo in cui è bello *Harry Potter*: il primo racconta di Oskar Schindler e del salvataggio di centinaia di ebrei dallo sterminio nazista; il secondo è apprezzato per la fantasia, per le sensazioni che suscita. Leopardi scrive in un modo in cui scrive solo Leopardi: può piacere o non piacere, ma non lo si può definire "bello" come il *Decameron*, perché sono cose completamente diverse. "Bello" è semplicemente un **giudizio positivo**, e il problema è che non dice *perché* qualcosa è piaciuto:

- di Leopardi è piaciuta la scrittura, o i temi malinconici?
- del *Decameron* è piaciuto lo stile comico, o il fatto che rompesse le convenzioni sulla sessualità del suo tempo?
- di *Schindler's List* è piaciuto il coraggio storico di Schindler nell'affrontare i nazisti, o il modo in cui è scritto?

Neanche *valido* va bene, né *noioso* (vale esattamente lo stesso al contrario), né *schifoso*: nessuna di queste parole descrive una caratteristica. Bisogna esprimere **che cosa il gioco cerca di ottenere, se lo ottiene o no, e con quali caratteristiche**. Che una cosa sia buona o cattiva è irrilevante.

Una tecnica pratica. Quando viene da scrivere "questo gioco è [parola proibita]", si completi la frase: "questo gioco è [parola proibita] *perché* è comico". Poi si tagli la parte centrale: "questo gioco è comico". Bisogna imparare a parlare **dei dettagli**, del *perché* qualcosa funziona, non del *se* è buono o cattivo.

Esempio: Dan Brown. Al direttore piacciono le storie di Dan Brown, ma ne detesta profondamente lo stile di scrittura: per questo non ne legge i libri, ma ne guarda i film. Non può quindi dire che *Inferno* sia "bello" o "brutto": il libro ha una buona storia ma uno stile di scrittura che non apprezza; il film andrebbe giudicato su altre caratteristiche, come la recitazione. Le opere hanno una serie di caratteristiche e di obiettivi che dipendono dal medium: per i videogiochi la tecnologia, gli fps, la narrativa, le meccaniche e quindi il gameplay, le qualità di gioco, i giocatori. C'è chi potrebbe dire di *League of Legends* "adoro tutto tranne i giocatori" (cosa comune, visto che la community è spesso considerata tossica); lo stesso si sente dire di Commander, "formato splendido, giocatori orribili".

Mattia chiude il punto ricordando che il gioco non si colloca quasi mai "nel vuoto": va legato ai giocatori e alle loro necessità. Al direttore piacciono le storie di Dan Brown ma non il suo stile, per come è lui e per le sue necessità di lettore; è un discorso che si allargherà ad altri argomenti.

2.12.4 Il nuovo vocabolario: una prima lista di emozioni

Al posto della parola proibita si comincia a usare un vocabolario di emozioni specifiche. La lista seguente **non è esaustiva** (ne esistono molte di più, e verrà ampliata), ma è un punto di partenza.

Excitement (eccitazione) affrontare pericoli, andare veloci, effetti luminosi (il "raggio epilessia" di Raggio Aurora), suoni forti: tutto ciò che fa pompare il sangue nelle vene e crea uno stato di eccitazione.

Challenge (sfida) Malenia: è difficile, voglio batterla, devo riuscirci. Una sfida può essere anche mentale, non solo fisica.

Competition (competizione) io contro Mattia: chi fa di più, chi vince. Molti giochi, soprattutto non digitali (che di solito prevedono almeno due giocatori), si basano sulla competizione. **Attenzione alla differenza**, che negli anni ha confuso molti studenti: la *challenge* è contro una sfida, la *competition* è contro altre persone, altri giocatori.

Accomplishment (realizzazione, progressione) salire di livello, sbloccare la nuova armatura più potente, raggiungere il ramo dell'albero delle abilità che si desiderava: la sensazione di crescere dà senso a ciò che si fa.

Socialization (socializzazione) parlare con la gente: l'essere umano è un animale sociale, e interagire con le persone può essere una fonte di gioia (e non solo).

Narrative (narrativa) le storie: una storia appassionante, una storia triste, una storia comica, una battuta.

Informative (apprendimento) imparare. L'anime *Dr. Stone*, tra i preferiti del direttore, gli piace proprio perché è informativo: insegna qualcosa di vero.

Aesthetic (estetica) le cose *visivamente piacevoli*. Esistono regole, come quelle sui codici colore, che indicano combinazioni di colori e forme generalmente considerate piacevoli, cioè non fastidiose alla vista. Il "raggio epilessia" è l'opposto.

Auditory/visual stimulation (stimolazione uditiva e visiva) lo stesso discorso per i suoni. La sirena (creata dall'uomo) e il pianto di un bambino (creato dalla natura) sono fatti per essere fastidiosi, per spingere ad agire: la sirena per far spostare, il pianto perché il bambino, che non sa fare nulla, venga accudito. All'opposto, esistono suoni con caratteristiche piacevoli all'orecchio.

Humor (umorismo) è separato dalla narrativa, di cui potrebbe far parte, perché far ridere è difficilissimo: l'umorismo è una delle cose più personali in assoluto, e quello di una persona può essere completamente diverso da quello di un'altra. Riuscire a fare un prodotto umoristico è difficilissimo.

Physical manipulation (manipolazione fisica) il direttore ha giocato tutta la lezione con un fazzolettino, perché è piacevole: la texture, il poterlo piegare. I fidget spinner si basano su questo: usare le mani è piacevole.

Physicality (fisicità) lo stesso discorso ma con il corpo: correre con le scarpe giuste, fare l'attività fisica che piace.

Creation (creazione) la creatività. Gli studenti sono lì perché sono creativi, chi più chi meno, chi in un modo chi in un altro: hanno il desiderio di aggiungere al mondo qualcosa che prima non c'era (si pensi a chi ha ricostruito Westeros in Minecraft).

Vicarious experience (esperienza vicaria) fare cose che non sono comuni o che non esistono: il direttore non è un pilota di Formula 1, non vuole fare il soldato, non è uno stregone che combatte i draghi, ma può trovare piacevole vivere queste esperienze in forma simulata. Ci sono vari livelli: da ciò che non possiamo fare ma un essere umano potrebbe fare, a ciò che è del tutto al di fuori delle nostre possibilità, come essere un alieno in una galassia lontana lontana.

Logic/Puzzle (logica, problem solving) usare la propria mente per risolvere un problema.

Da subito bisogna usare questi termini; pian piano il vocabolario si espanderà.

2.13 La forma mentis del designer secondo Chris Bateman

L'ultima parte della lezione riprende alcuni consigli di Chris Bateman (autore, tra l'altro, di *21st Century Game Design*, si veda la sezione 1.7) per acquisire la *forma mentis* del designer.

Il game design è una metodologia iterativa ed evolutiva davanti a un problema bisogna procedere in modo **scientifico**. Non si può realizzare semplicemente la prima idea che viene in mente: bisogna generare un gran numero di idee, sia con la creatività sia con le metodologie (come i framework di analisi), e analizzare il problema da tutti i punti di vista, per trovare non *una* soluzione, ma **la soluzione migliore**. Una soluzione la trovano tutti: per risolvere il problema delle tasse in Italia si potrebbero, per paradosso, eliminare tutti i contribuenti; è una soluzione, ma non è certo la migliore. La soluzione migliore va analizzata e trovata.

Non esiste un unico metodo di design è quanto detto a proposito delle lenti. Più metodi si studiano e più tecnologie si imparano, più la mente si espande e si trovano modi di interagire con problemi, persone e design, e si diventa designer migliori. *The right tool for the right job*; ma uno strumento non basta, ne servono centomila.

Il design è *egoless* durante la lezione i docenti hanno smontato le definizioni degli studenti senza avercela minimamente con loro, e gli studenti avrebbero potuto smontare la loro: si stava discutendo di teoria del design. **Le idee non rappresentano chi le ha**: sono un elemento separato, da analizzare, distruggere, ricostruire, migliorare. Che un designer generi cento idee pessime non significa che sia incompetente: le idee vanno generate e analizzate. **Le idee possono venire da chiunque**: in uno dei giochi del direttore, una delle meccaniche migliori è stata suggerita da un programmatore. Il design consiste nell'analizzare l'idea, comprenderla, migliorarla e capire se funziona. Altre volte è stato Mattia, o Jack, lavorando con lui, a dirgli "sei sicuro che sia così?", e avevano ragione; questo non ha minimamente intaccato il suo valore di designer. Si crea, si analizza e si distrugge, sempre con lo scopo di trovare la soluzione migliore, qualunque essa sia.

Il design è guidato dagli utenti lo scopo finale, come molte definizioni avevano colto, è creare un gioco **che venga giocato**, che l'utente finale apprezzi. Non un gioco che il designer considera il migliore del mondo e che tutti gli altri odiano. Fare arte per se stessi è un approccio del tutto accettabile, e non c'è nulla di male nell'arte fine a se stessa: anche il direttore fa giochi "per Diego", esperimenti e tentativi, e ci saranno momenti per farli (ad esempio nelle *game jam*, che verranno spiegate più avanti). Ma il corso insegna a lavorare come designer, e il game design come professione è orientato all'utente finale: creare un prodotto che una, si spera decine o centinaia di milioni di persone possano giocare e apprezzare (il che, detto brutalmente, se succede rende anche molto ricchi). Il direttore, che fa consulenza a vari studi, ha visto spesso team fare un gioco, vederlo fallire e perdere tutti il lavoro: un peccato, perché con lo stesso studio si sarebbero potuti creare altri capolavori, o semplicemente dare lavoro alle persone.

2.14 Il task della settimana

1. Scegliere un **gioco arcade**.
2. Scegliere **uno** dei framework visti a lezione: Schell, Romero e Schreiber, oppure Bura (quest'ultimo è più complesso, ma accettato).
3. Analizzare il gioco usando il framework scelto.
4. Far emergere le **emozioni** verso cui il gioco è spiccatamente orientato, usando il vocabolario visto in lezione: individuarne principalmente **tre**.
5. Poiché in aula è stato chiesto che cosa si intenda per "gioco arcade", aggiungere al task anche **una propria definizione di gioco arcade**.

Vincoli e indicazioni:

- valgono tutti i **termini banditi** (sezione 2.12);
- le modalità di consegna (ad esempio documento su Drive) e le date verranno comunicate da ciascun docente per la propria sede; come regola generale, un task si considera da consegnare entro una settimana dall'assegnazione;
- lo studente che ha pronunciato la parola proibita deve svolgere l'analisi con **entrambi** i framework principali, in **due documenti distinti**;
- la lezione successiva si aprirà con una sessione di domande e risposte su questa lezione.

Punti chiave del capitolo

- Il primo passo del percorso è passare dalla visione del **giocatore** a quella del **creatore**.
- Ogni definizione di gioco include qualcosa ed esclude qualcos'altro: le parole hanno un significato preciso e vanno scelte con cura.
- Il gioco **non è definibile**, perché è anche uno **stato mentale**; *game* e *play* sono la stessa cosa, e lavorare con lo stato mentale del giocatore è parte del mestiere.
- La propria definizione di gioco è una **bussola**: dice che tipo di designer si è. Va riscritta periodicamente, senza rileggere la precedente.
- Non si può definire la zuppa, ma se ne possono elencare gli **ingredienti**: definire chiude, elencare apre.
- Un framework valido deve potersi applicare a **qualsiasi gioco**.
- **Schell**: meccaniche, narrativa, estetiche, tecnologia. **Romero e Schreiber**: game qualities, giocatore, tema, obiettivo, regole, risorse, interazione. **Bura**: action, system, self, social incrociati con freedom, mastery, data.
- Non esiste un metodo unico: i framework sono **lenti**, e servono a guardare un problema da più punti di vista. È anche la miglior difesa del designer contro l'automazione.
- La parola "divertente" (e con lei "bello", "noioso", ecc.) è **bandita**: bisogna nominare le **emozioni specifiche** che il gioco vuole suscitare.
- Il design è iterativo, plurale, **egoless** e orientato all'**utente**.

Chapter 3

Iterare e documentare: il ciclo di produzione e le regole della documentazione

I primi due capitoli hanno descritto il mestiere del designer e il modo in cui un designer guarda a un gioco. Questo terzo capitolo apre un modulo nuovo, quello dedicato alla **documentazione e al pitching**: come si documentano le cose, come le si spiega, come le si comunica. Si parla, ancora una volta, di *linguaggio e comunicazione*: il direttore aveva definito il documento come la forma di comunicazione "pratica", condensata, del designer, ed è di questo che ci si occupa.

La lezione è tenuta da Mattia Podini, docente di game design della sede di Milano, che si definisce un appassionato di documentazione ("è una mia specialità"). Il percorso del capitolo ne ricalca la struttura:

1. una sessione di domande e risposte sulla lezione precedente, che chiarisce e amplia alcuni punti dei framework di Schell, di Romero e Schreiber e di Bura (sezione 3.1);
2. un giro nel Drive del corso e le regole di consegna dei task (sezione 3.2);
3. la lezione "zero" di documentazione: il **ciclo di produzione** e il principio dell'**iterazione**, con i suoi corollari sul target audience, sui test e sui pilastri del progetto (sezioni 3.3, 3.4 e 3.5);
4. la prima vera lezione di documentazione: perché documentare, le **golden rules** della buona documentazione, come scegliere lo strumento giusto e le regole formali del corso (sezioni 3.6, 3.7, 3.8 e 3.9);
5. il task della settimana (sezione 3.10).

Già in apertura il docente avverte che la lezione sarà un "grande spasso" e precisa, a scanso di equivoci, che anche *spasso* è una parola proibita (si veda la sezione 2.12).

3.1 Domande e risposte sulla lezione precedente

Prima di entrare nel vivo, la lezione si apre con un ripasso guidato dalle domande degli studenti. Il docente avverte che le slide della lezione precedente sono state **aggiornate**: ha aggiunto un paio di cose e ne ha rimodernate altre, quindi conviene riguardarle.

3.1.1 Schell: game piece, verbi e regole

La prima domanda (Valerio) riguarda il blocco delle *meccaniche* nel framework di Jesse Schell (sezione 2.7): qual è la differenza tra *game piece* e *verbi di gioco*?

Game piece (pezzo di gioco) ha dentro di sé delle statistiche, delle informazioni pubbliche e private, una serie di caratteristiche. Va considerato come un **oggetto di gioco che cambia stato o fa cambiare stato ad altre cose**. In *Super Mario* un pezzo di gioco può essere un nemico (un Goomba), ma anche il fiore di fuoco, la moneta, o Mario stesso.

Verbo di gioco ha generalmente a che fare **solo con il giocatore**: serve al designer per definire che cosa il giocatore può fare (ed eventualmente che cosa non può fare). I verbi di Mario sono saltare, sparare (se ha il fiore di fuoco), camminare, corricchiare e correre: premendo leggermente la levetta si cammina, poi si inizia a corricchiare e infine si corre, una gradazione tipica dei giochi action (lo fa anche *Uncharted*). Il verbo deve essere qualcosa di **attuabile dal giocatore** che si traduce in un verbo, in un'azione.

Regole sono **tutto ciò che non ricade negli altri due blocchi**. Il limite dello spazio; il fatto che in *Mario*, se finisce il timer, è game over; la condizione per finire un livello (raggiungere la bandiera, il castello). Tutto ciò che fa da *struttura* del sistema.

Il verbo di gioco fuori dal framework. Al di là del framework, il termine "verbo di gioco" assume un **senso più generale** quando si parla in contesti meno strutturati: in un brainstorming con un altro designer, mentre si decide qualcosa, o quando si deve far capire il proprio gioco, la propria visione, a qualcun altro. In *Call of Duty*, come in qualsiasi sparatutto, il verbo principe è *sparare*. Ma poi, da gioco a gioco, bisogna capire che cosa significa: il verbo *sparare* funziona sia per *Call of Duty* sia per *Portal*, solo che in *Call of Duty* si spara per uccidere i nemici, in *Portal* si spara (con la Portal Gun) per risolvere puzzle. Nel mondo moderno il termine si usa quindi in maniera molto generica; quanto generica, lo si capirà nel tempo.

3.1.2 Romero e Schreiber: i player sono il target audience

La seconda domanda riguarda il framework di Brenda Romero e Ian Schreiber (sezione 2.8): è corretto intendere i *player* come il target? Sì: **i player sono esattamente il target audience**.

Qui il docente coglie l'occasione per un punto che non era emerso abbastanza: molti dei termini dei framework (meccaniche, estetiche, player...) **hanno molteplici definizioni a seconda del framework**. Alcuni sono linguaggi tecnici riutilizzati con accezioni diverse. Nel framework di Romero e Schreiber i player sono il target audience e **vanno definiti**: non è sufficiente scrivere "casual gamer". Servono informazioni specifiche:

- nazionalità ed eventualmente etnia;
- sesso o genere in cui si identificano;
- età;
- potenzialmente anche il background.

In generale, **tutti i dati che si possono ricavare**, purché si tratti di dati *che hanno un riscontro*. Come si fa questa ricerca verrà spiegato più avanti nel corso.

3.1.3 Perché due framework di core elements

Il motivo per cui sono stati presentati due framework di *core elements* non è solo dare più strumenti (che resta la ragione principale). Nel piccolo panorama dell'industria dei videogiochi, per quanto giovane, il framework di Schell è stato **più condiviso di recente**. C'è stato un periodo, quello delle console dei primi anni Duemila, in cui si preferiva Romero e Schreiber. Perché? Si immagini di giocare uno stesso titolo, ad esempio *Halo*, su due console della stessa generazione, come Xbox 360 e PlayStation 3: che differenza ci sarebbe? Probabilmente nessuna. Il pad è generalmente lo stesso e la tecnologia è simile, quindi era più utile definire un gioco attraverso le *game qualities*, cioè elementi atomici.[1]

Oggi invece giocare da mobile su uno schermo, o su un Nintendo DS con uno schermo interattivo, rispetto a qualcos'altro, apre possibilità di design diverse da quelle di altri progetti. Se la **tecnologia è un elemento**

[1] L'esempio è ipotetico: *Halo* è una serie esclusiva Microsoft, e solo di recente è stato annunciato il suo arrivo su altre piattaforme, cosa a cui si allude in lezione parlando di un *porting*.

dominante, può essere più rilevante usare il framework di Schell (che la include esplicitamente) piuttosto che quello di Romero e Schreiber. È un primo esempio concreto del principio delle lenti (sezione 2.11): la scelta del framework non dipende solo dalla propria sensibilità, ma anche da ciò che si ha davanti.

Quale framework per il task. Per il task in corso (sezione 2.14) il consiglio è di usare uno dei due framework di core elements, **Schell** oppure **Romero e Schreiber**. Si può usare anche quello di Stéphane Bura (il docente, scherzando, ammette di non ricordarne bene il nome e lo fa sillabare in aula: B-U-R-A, S-T-E-P-H-A-N-E), ma è più complesso. Con una nota di ironia, il docente aggiunge che il direttore "sbaglia parecchie nomenclature, ma se le ricorda": il consiglio è di seguire il filo del suo discorso anche quando i nomi non sono esatti.

3.1.4 Come si usa la matrice di Bura

La tabella vista nella lezione precedente (tabella 2.1) è una **versione semplificata** della matrice di Bura, che si può trovare facendo ricerca. Il docente ne completa la spiegazione.

La premessa. L'assunto da cui parte Bura è che gli elementi della matrice possano essere combinati e ricombinati fino ad avere una **formula** quasi matematica e **riproducibile**: per creare un certo tipo di gioco, o per far evocare un certo tipo di emozioni, in maniera riproducibile.

La conseguenza: dalla matrice al design. Finora i core elements sono serviti come **metodo di analisi** (ed è questo il task in corso); più avanti andranno usati anche come **metodo di design**. Con Bura si prende una meccanica, un pezzo di gioco, e si trova **la cella esatta** della matrice in cui ricade. Che cosa succede se quella cella viene *potenziata* o *bloccata*, poco o tanto? Qual è il responso emotivo? Esempio: in un gioco horror, se si inizia a togliere **freedom** all'**action**, ad esempio lato level design, e si stringono i corridoi fino a lasciare al giocatore un solo corridoio lineare, il giocatore ("perdonate il francese") inizierà ad avere molta paura. O almeno questo è il presupposto, ciò che si spera; e non è l'unico elemento in gioco. Questa è la parte finale del framework, su cui eventualmente si tornerà in futuro.

Il ciclo variabili–emozioni–interazioni. Un altro punto legato alla premessa: si può partire da dove si vuole, ma le **variabili di gameplay** (che si traducono, o almeno si identificano, nelle celle del framework) influenzano le emozioni del giocatore; il giocatore interagisce in qualche modo con l'ambiente di gioco, e l'ambiente di gioco, a sua volta, cambia le variabili di gameplay. Se si ha molta vita si fa "lo smargiasso", si va tronfi e senza paura nella mischia; combattendo si perde vita, e allora si diventa più cauti.

Esempio: Into the Breach. Il docente consiglia caldamente *Into the Breach* ("dovete giocarlo"; tra l'altro è stato distribuito gratuitamente su Epic Games ed è incluso nell'abbonamento Netflix): una sorta di *Pacific Rim* su griglia, in cui si combatte contro i Kaiju con tre unità (i *mech*), ciascuna con un pilota. Ne nasce in aula un'accesa discussione su quanta casualità ci sia nel gioco. La posizione del docente è che il gioco, pur essendo un roguelike (casuali sono i power-up, la creazione delle missioni, i biomi da affrontare), **in battaglia** sia quasi del tutto privo di casualità, "come gli scacchi": un PvE asimmetrico, con i comportamenti dei nemici dichiarati in anticipo, che dipendono dalla posizione dei mech, dalla distanza e dal massimo danno possibile. L'unica eccezione è la **griglia energetica**.[2]

Il punto dell'esempio, però, è il ciclo emozioni–interazioni. Se un mech finisce le tacche di vita si può perdere il pilota, e i piloti sono generalmente più forti dell'IA che comanda il mech in loro assenza (hanno delle abilità passive). La griglia dà dei bonus, ma se arriva a zero è game over: è, di fatto, la vita del giocatore. Allora: se si ha molta griglia si può rischiare di farla cadere ("500 civili in meno che saranno, un palazzo demolito che sarà, per il bene dell'umanità"); se si ha una sola tacca, si sarà molto più cauti. Oppure, e qui entrano in gioco i *tipi di giocatori*, di cui si parlerà più avanti, si può rischiare ugualmente,

[2] In *Into the Breach* le intenzioni dei nemici vengono mostrate prima che agiscano, rendendo ogni turno un problema a informazione quasi completa; la principale fonte di casualità durante il combattimento è la *grid defense*, la probabilità che un edificio resista a un attacco, che parte dal 15% e può essere aumentata. Alcuni studenti fanno notare che c'è comunque della casualità nelle scelte dei nemici; il docente rimanda ai talk degli sviluppatori.

perché c'è il 15% di possibilità che l'attacco a un edificio non vada a segno: "me la gioco", e magari va bene. Le variabili modificano il loop:

- **per il giocatore**, il ciclo va dalle variabili alle emozioni alle interazioni;
- **per il designer**, il ciclo si percorre *a ritroso*: si parte dalle emozioni che si vogliono far provare (la lista vista nella lezione precedente) e si pensa a quali variabili di gameplay e quali interazioni possono influire su di esse.

A volte questi elementi vengono scambiati di posto: dipende dal framework e dal design.

Le scelte come una forbice. Un ultimo punto: sia nel prendere una scelta sia nel mappare un gioco dall'inizio, le scelte diventano **progressivamente più ampie dall'inizio alla metà** del gioco e **progressivamente più strette dalla metà alla fine**. In *Pokémon* all'inizio si può scegliere tra tre starter; poi nel percorso se ne possono catturare altri cinque, poi altri centocinquantuno, e così via. Ma *Pokémon* ha un finale: si batte la Lega (i Superquattro), "baci e abbracci". Non vale per tutti i giochi: nei *sandbox* la forbice è molto ampia e potenzialmente non si chiude mai.

Altri esempi delle celle. Il docente mostra anche gli esempi che aveva preparato per le celle e che non c'era stato tempo di vedere:

Freedom × **Action** la chiave che apre la porta.

Freedom × **System** l'esplorazione: rischio di andare al prossimo falò, sapendo che in mezzo ci sono un certo numero di strade ed eventuali nemici pronti a "massacrarmi di botte".

Freedom × **Self** non è solo quanto detto nella lezione precedente: è anche avere la **capacità di attuare ciò che si vuole in quanto individuo**, avere gli strumenti per farlo. In *Celeste* (un platform con una colonna sonora notevole) si può attivare dalle opzioni un cronometro che mostra quanto tempo si sta impiegando a finire il gioco: uno strumento pensato per gli *speedrunner*, che trovano all'interno del gioco ciò che serve a fare quello che vogliono. Funziona anche a bassissimo livello: avere nel gioco gli strumenti per personalizzare i propri controlli, se lo si ritiene una necessità.

Freedom × **Social** poter parlare con altre persone.

3.1.5 Una critica personale al framework di Bura

Il docente premette di essere umano e quindi di avere delle preferenze: non è un grande fan del framework di Bura, e farà emergere le criticità dei materiali quando ci saranno.

- Termini come *freedom* non sono davvero adottati nel design, se non in maniera gergale; esistono sinonimi.
- *Mastery* è, nella sua visione, anche un **responso emotivo**: lo sviluppo di competenze è una possibile emozione, e comparirà nella lista. Esempio: il docente è "scarsissimo" a scacchi e ha amici molto forti che lo sfidano continuamente. Se volessero davvero dimostrare la propria maestria, che cosa farebbero? Andrebbero contro persone forti: nei tornei si cerca di avere un *bracket* bilanciato, in modo da affrontare giocatori con la stessa abilità o con abilità superiore, ed è lì che si vede chi è il migliore. Quindi il loro obiettivo, molto probabilmente, è un altro: farlo sentire inadeguato. Sono due risposte emotive molto diverse, e per questo il docente colloca la *mastery* altrove.
- Lo stesso vale per la *freedom*: ogni gioco ha dei limiti, **la vera libertà non esiste** (ci sarà una lezione apposita).

Lo stesso termine, significati diversi: le estetiche. Vale anche per le *estetiche*. Nel framework di Schell includono la *vestizione*, il *dressing*: l'arte, la direzione artistica, i suoni, tutto ciò che viene apposto sopra le meccaniche. In un altro framework, invece, si parla di **game aesthetics**, che sono un'altra cosa: sono quelle che compaiono nella definizione di gioco del docente (sezione 2.5), e che per ora vanno lette come sinonimo di *emozioni*.[3]

Parlare la stessa lingua. Esistono quindi molti sinonimi, alcuni più condivisi di altri. Non significa che uno sia giusto e l'altro sbagliato: dipende dal contesto, dall'azienda in cui si lavora, da chi si collabora. L'obiettivo è **comunicare nella stessa lingua**, e a volte, ahimè, bisognerà piegarsi agli altri, perché un termine è condiviso o imposto dall'alto.

Aneddoto: lo stagger. Il docente racconta (senza dire dove né con chi) un caso che "ha fatto impazzire" tutto il team di design. In una grossa azienda, in un gioco con un sistema di combattimento, il team presentava la documentazione al creative director in sessioni di *briefing* e *debriefing*. Generalmente, quando un nemico è *staggerato* (in *Sekiro* l'equivalente è lo sbilanciamento), non viene solo stordito: lo si interrompe, è un termine specifico. Il creative director voleva invece che *stagger* indicasse un'altra cosa, traslando termini come *stun* e *stagger* in accezioni non di uso comune, e non legate a come il giocatore le percepisce. Il team ha dovuto rispettare l'ordine, perché veniva dall'alto; lo ha fatto presente, ma la segnalazione non è stata accolta, e questo ha creato non pochi problemi. È un problema di direzione; ma dimostra che l'obiettivo è trovare il linguaggio più comune a tutti, entro dei limiti: un'azienda che assume non nasce con il nuovo assunto, ha già una struttura e delle regole, e bisognerà stare a quelle (limiti che, peraltro, possono anche essere utili).

3.1.6 Scrivere da designer, non da recensori

Una domanda di Federica sulla lezione precedente ("bello", "brutto", "noioso") porta a chiarire il senso dell'obbligo sul vocabolario (sezione 2.12): serve a far **parlare** gli studenti da designer, e in futuro a farli **ragionare** da designer. Il docente sa già che succederà: molti inizieranno a scrivere *come recensori*, non come designer, perché si è abituati a un certo tipo di editoria. Dire che un gioco è bello o brutto, che piace o non piace, è legato a una sensibilità personale che non deve esserci, perché **il design è *egoless*** (sezione 2.13).

Un modo per togliere questa sensibilità sono i **dati**. È il discorso dei player di Romero e Schreiber: colloquialmente si può dire che *Candy Crush* è un gioco "adatto alle casalinghe di mezza età", ma nei documenti questa frase non esiste; al massimo nel brainstorming, se va bene. Un documento tecnico richiede dati, e deve essere **il più inconfutabile possibile**, per due ragioni:

- il resto del team si aspetta che si sia fatto un certo tipo di ricerca e che si sia sicuri della propria idea;
- il designer stesso, per primo, deve smontare la propria idea ed essere sicuro dei dati: è il metodo scientifico.

Le cose poi cambiano, le meccaniche si iterano, ma l'assunto di base è essere il più sicuri possibile. Il resto del team viene a chiedere al designer quando ha un dubbio: se nemmeno il designer è sicuro di ciò che scrive e dice, come si fa?

3.2 Il Drive del corso e le regole dei task

3.2.1 La struttura del Drive

Tutti gli studenti sono stati aggiunti al Drive del corso, e il docente ne fa un giro esplorativo.

- C'è la cartella del **primo anno di game design**; ci saranno poi quelle del secondo e del terzo.
- Dentro, le cartelle sono divise in **lezioni**, **materiale studenti** e **task**.

[3] Il riferimento implicito è con ogni probabilità al framework MDA (*Mechanics, Dynamics, Aesthetics*) di Hunicke, LeBlanc e Zubek, in cui le *aesthetics* sono le risposte emotive desiderate nel giocatore.

- Le lezioni sono divise in *Game Design & Method* e *Documentation* (questa lezione appartiene alla seconda), più le lezioni condivise con i dipartimenti coinvolti (ad esempio quelle con narrativa).
- **Materiale studenti** è pensato per documenti aggiuntivi caricati dagli studenti: ad esempio, se qualcuno trovasse il PDF del framework di Bura, lo si potrebbe caricare lì. L'anno precedente non è servito a nulla, perché nessuno ha caricato niente, ma si cerca di mantenerlo. Materiali come i pacchetti di asset sono invece caricati di solito dal docente di level design nella cartella di level design.

I permessi. Gli studenti hanno i permessi per **modificare tutto**: possono rinominare, aggiungere cartelle e così via ("se volete distruggere casa vostra, fatelo, ma non ve lo consiglio"). Non possono però **cancellare**: se per errore si crea un documento che non dovrebbe esserci, bisogna avvisare il docente, che lo cancellerà. Senza abusarne: prima di creare documenti, assicurarsi che siano quelli corretti.

3.2.2 Come si consegna un task

1. Per ogni task **uno solo** degli studenti (il primo che arriva si prende "questo onere e onore") crea una cartella dentro la cartella dei task della disciplina (Task GD, Task ND o Task LD). Per il task degli arcade la cartella è già stata creata dal docente.
2. La cartella si rinomina nel formato **anno mese giorno titolo del task**. Chi la crea ha il merito di scegliere il titolo, purché abbia senso (niente "uova di Pasqua" se il task non riguarda quello).
3. Dentro quella cartella ciascuno crea **direttamente** il proprio documento.
4. Se si devono caricare più documenti, si crea una sottocartella **cognome e nome** e li si carica lì. Se il documento è uno solo, la sottocartella non serve.

Scadenza. Si ha **una settimana** dall'assegnazione del task. Il task in corso è stato assegnato mercoledì, quindi va consegnato **entro mercoledì alle ore 13**. Se lo si carica dopo, il docente assume che non sia stato fatto; se lo si modifica dopo la scadenza, il docente non lo legge. Solo dopo la scadenza il docente inizia a commentare.

Durante la revisione. Se arrivano le notifiche di Google e si vede che il docente è dentro il documento e sta commentando, non bisogna "andargli sopra", mandare emoji o fare domande: si aspetta che abbia finito, perché altrimenti lo si rallenta (e ci si mette ulteriormente in ansia). Il docente ammette di esserci passato anche lui: la curiosità è tanta e l'emozione molta. Come sapere quando ha finito? La cartella del task **sparisce da Task GD e finisce in "Revisionati"**. Da quel momento si ha piena libertà di leggere i propri documenti, rispondere ai commenti e fare domande. Per i dubbi si può sempre parlare prima o fuori dalla lezione, o scrivere su Discord. Ma prima di contestare un commento (il docente può fraintendere qualcosa) bisogna **leggere i commenti di tutti**, non solo i propri.

Solo la suite di Google. Per ora, e per il prossimo task, si possono usare **solo gli strumenti della suite di Google**. È un consiglio che per ora vale come obbligo, e molte di queste regole verranno meno nel tempo (il perché si vedrà in questa stessa lezione). Il documento va **creato direttamente nella cartella del task**: non lo si crea nel proprio Drive per poi copiarlo lì alla fine. Se si vuole una copia personale, la si crea a partire dall'originale, ma l'originale deve stare nella cartella del corso. Le ragioni:

- nessuno ruba i documenti (all'inizio ci sta un po' di gelosia), e bisogna abituarsi al fatto che **i documenti che si producono non sono propri, ma dell'azienda**;
- la cronologia delle modifiche permette al docente, in caso di dubbi, di verificare che il task non sia stato fatto la sera prima della scadenza;
- serve a tutela degli studenti in caso di copiature, che si assume non ci siano mai ma che ogni anno capita di scoprire; la punizione è estremamente dura.

Tutte queste regole sono raccolte in un documento apposito nella cartella dei task, che verrà aggiornato man mano che se ne aggiungeranno di nuove (si veda anche la sezione 3.9).

La punizione della parola proibita. Lo studente che deve consegnare due analisi (sezione 2.14) chiede se debba analizzare due giochi con un solo framework. Il consiglio è: **un gioco analizzato con Romero e Schreiber e un gioco diverso analizzato con Schell**, quindi due giochi e due framework diversi, sconsigliando Bura per semplicità (chi vuole impegnarsi di più può comunque farlo). In questo caso la punizione coincide con il task del giorno, ma generalmente le punizioni sono altro, qualcosa di diverso dall'argomento in corso.

3.3 Il ciclo di produzione

3.3.1 La lezione zero

La prima lezione del modulo di documentazione è la lezione **zero**: non la più importante, ma una di quelle cose che devono entrare nella *forma mentis*. Si parla di **ciclo di produzione**. (Come da tradizione, le lezioni di documentazione si aprono sempre con un *meme*, che il docente aggiorna ogni anno; quest'anno uno su *Metroid Prime 4*.)

Il primo ciclo mostrato è, per ammissione del docente, **bruttissimo**: fatto con Paint. E c'è una ragione, che si capirà più avanti (sezione 3.5.6).

3.3.2 Preproduzione, produzione, post-produzione

Dal secondo anno (non al primo, per le ragioni spiegate nella prima lezione) si vedranno diverse metodologie di produzione, tra cui quella principale. In generale, però, un progetto si divide in tre grandi blocchi.

Preproduzione il blocco iniziale, in cui si fa **tanta ricerca, tanta documentazione**, tante cose in vista del momento in cui "ci si sporcherà le mani". Le tre fasi non occupano un terzo del tempo ciascuna: questa è probabilmente la più breve, purtroppo. Purtroppo perché **quanto più è lunga e profonda la preproduzione, tanto meglio usciranno le altre due**. È una cosa che la gente non vuole mai capire, nemmeno le aziende; il docente capisce che dal punto di vista dei costi sia la fase meno conveniente, perché non sempre ha un *output* vendibile, ma assicura che è una delle più importanti, soprattutto per partire con il piede giusto.

Produzione la fase che prende gran parte del tempo: quando si fanno le cose, quando il gioco si crea.

Post-produzione a volte una fetta molto breve: tutto ciò che si fa dopo il rilascio o in vista del rilascio, cioè *polish* estremo, *bug fixing* ed eventualmente la creazione di DLC.

Il ciclo include anche il **testing**, la **beta** (un tipo di versione: ce ne sono diverse, se ne vedranno altre) e il **rilascio** (*release*).

3.3.3 DevOps e LiveOps

Questa suddivisione viene da altre industrie: preproduzione, produzione e soprattutto post-produzione esistono anche nell'industria cinematografica. Per i videogiochi, però, da qui in avanti ("nei secoli dei secoli, amen") il termine *post-produzione* si può dimenticare, perché esistono termini più corretti:

Esempio: Apex Legends. Interpellato, uno studente indica un *battle royale*, *Apex Legends*: nel suo ciclo di vita ha più LiveOps o DevOps? **LiveOps**. Non che la parte di sviluppo non sia stata grande, assolutamente; ma l'obiettivo dei *Games as a Service* è rilasciare il prima possibile e poi iterare, e la loro fase di LiveOps è estremamente grossa. Ed è difficilissima, probabilmente la cosa più difficile in assoluto: il prodotto è fuori, non deve crollare a picco, e intanto bisogna aggiungergli pezzi in corsa. I giochi mobile vivono generalmente di questo, a meno che l'obiettivo dell'azienda non sia buttare fuori n giochi mobile "al secondo", anche con l'uso dell'intelligenza artificiale, per vedere quale "si attacca al muro" e guadagnare un sacco di soldi: una strategia, purtroppo, dominante.

	DevOps	**LiveOps**
Quando	tutto ciò che avviene *prima* del rilascio	tutto ciò che avviene *dopo* il rilascio
Tipico di	giochi che escono "finiti"	*Games as a Service*: giochi mantenuti in vita dopo il rilascio
Esempi	—	*Candy Crush*, *Clash Royale*, *Fortnite*, *Apex Legends*, gran parte dei giochi mobile

Table 3.1: DevOps e LiveOps nel ciclo di vita di un gioco.

Il piccolo trucco. Molte delle cose che sembrano LiveOps in realtà rientrano nel DevOps. È impensabile, ad esempio, che un DLC venga fatto interamente dopo il rilascio del gioco: la sua produzione inizia uno o due mesi prima dell'uscita. Lo si completa dopo, ma gran parte del lavoro viene portata avanti prima.

3.3.4 Qual è la parte più importante?

Il docente chiede agli studenti quale sia, secondo loro, la parte più importante del processo, e perché.

- Una risposta (Francesca) è la **post-produzione**, perché è lì che ci si occupa di come si vede il gioco.
- Un'altra è lo **start**, l'inizio del progetto.
- Federica è indecisa tra **preproduzione**, perché è la base su cui si fonda tutto (e dalle frecce si vede che ci si torna più volte), e **testing**, perché bisogna vedere se ciò che si sta facendo funziona.
- Un'altra studentessa indica il **testing**, perché serve a mettersi nei panni del giocatore e a rendersi conto delle emozioni che il gioco regala.

Era una domanda "a trabocchetto", come molte delle domande del docente, ma Francesca e Federica ci sono andate vicino. La fase di rilascio è molto importante, così come quella che la precede, il *polish* estremo, che è ciò che rende davvero il gioco piacevole da giocare dal punto di vista del *feedback*, dei VFX, degli SFX, di tutto il *dressing* che potenzia le estetiche nel senso di Schell. Anche la preproduzione è estremamente importante, perché è il fondamento del gioco: se si è creata documentazione e si è fatta abbastanza ricerca, si hanno fondamenta solide su cui costruire.

Ma la risposta corretta era **la linea bianca**: la **freccia che torna indietro**. È il concetto a cui alludevano Federica e Francesca: il fatto che si torni spesso indietro. Il ciclo di produzione è **ricorsivo**. Non è importante *dove* si ritorni (dalla produzione alla produzione, o altrove), ma il fatto che si torni indietro, si testi, si veda come stanno andando le cose e si rivalutino scelte, modifiche e applicazioni. È questa la parte che rende i giochi (con licenza del termine) "belli". Non necessariamente quelli che vendono di più in assoluto, ma il prodotto mirato, con i target giusti e le idee che si vogliono trasmettere.

La ricerca nella preproduzione. Precisando la risposta di Federica, il docente sottolinea che la preproduzione è anche il momento in cui si identifica una soluzione a un problema, un buco di mercato, qualcosa che vende. Esempio: quando sono andati forte i *friendslop*, con *PEAK* "e compagnia cantante", tutti volevano farne, perché costano meno di un gioco indie "doppio" e hanno un ritorno molto elevato: un friendslop non lo si gioca da soli, lo si gioca con le persone. Cercare la vendibilità è giusto; ma è altrettanto importante che il prodotto abbia un'identità e un senso, per creare il miglior prodotto possibile, con un asterisco: *entro una serie di limiti*, quelli visti nel primo capitolo (sezione 1.9).

3.3.5 Tutto quello che facciamo è brutto

Perché esiste la freccia che torna indietro? Perché **tutto quello che si fa, al primo tentativo, è brutto**. Quello che fa il docente, quello che fanno gli studenti, quello che fa chiunque. Il documento del task, iniziato presumibilmente uno o due giorni prima, adesso è illeggibile e bruttissimo; lo sarebbe anche quello

del docente, che magari è solo più veloce. La questione è **quante volte ci si ritorna**: non "alla veloce", ma con senso e con criterio. Il criterio è vedere come risponde l'utenza, se il livello che si sta creando è comprensibile, se l'idea implementata evoca le emozioni che ci si aspetta.

Il lavoro del designer, infatti, è **far evocare emozioni specifiche nel giocatore**, e prevedere che cosa il giocatore percepirà giocando è difficilissimo: impossibile da fare "a scatola chiusa". Si può essere il designer più forte dell'universo, si può essere Miyamoto: ci si andrà vicini, ma non ci si prenderà mai al 100%. Ci saranno sempre cose sbagliate: le si testano, ci si ritorna, le si modificano e si va avanti. Se l'obiettivo è il gioco migliore possibile entro una serie di limiti, questo processo rende migliore il gioco, e di conseguenza il designer.

3.3.6 Tanti cicli, una sola forma

Il docente mostra diversi framework di cicli di produzione, da prendere come linee guida e "con le pinze", per far vedere una ricorrenza che torna in tutti.

Il ciclo in tre fasi *game idea*, *idea refinement* e ricerca nella prima parte; la produzione, con una parte a cavallo tra le due fasi che comunque continua; la documentazione e il mantenimento delle meccaniche, con il loro ciclo di frecce; la post-produzione, con il testing dell'intero gioco, il *deployment* (il rilascio) e il marketing. Diverse fasi della produzione richiedono **forze lavoro diverse**: i designer servono molto di più nelle prime due fasi; i programmatori servono molto in produzione e poi alla fine per correggere i bug; gli artisti servono da un certo punto della produzione in poi, perché finché le meccaniche vengono validate internamente si può "giocare con i cubi". Se si dà il prototipo a un altro designer dicendogli "questa cosa dovrebbe essere questo", va bene anche un cubo, un cono, una cosa bruttissima; se lo si deve dare in mano al giocatore, va vestito in un altro modo.

Il ciclo circolare preproduzione, produzione, testing, *pre-launch*, *launch*, *post-launch*: una vera circonferenza, sempre ricorsiva.

Il ciclo dello sviluppo ideazione, *prototyping*, *development*, *testing*, *deployment*, *maintenance*.

Il ciclo di vita di un prodotto (*life cycle assessment*) non legato ai videogiochi: *manufacturing*, *usage*, *device recovery*, *service*, *end of life*, pensato per il riciclo dei materiali di prodotti digitali (telefoni, computer, tablet). Anche qui il ciclo è continuo, e anzi a strati, dal dentro al fuori.

Design, develop, test, deploy, review vale ad alto e a basso livello. Gli **stakeholder** (altro termine importante), cioè chi detiene l'idea o i soldi, come il CEO, compiono questo processo a un livello più alto rispetto, ad esempio, al producer che pianifica. Prima si fa il design; lo si "mette a terra" e lo si sviluppa in piccolo; lo si testa al chiuso con una piccola parte di utenza, e poi si amplia piano piano (magari non al primo né al secondo, ma al decimo giro); una volta sicuri che funzioni, o che non funzioni, lo si implementa nel progetto; poi si fa *review*, si controlla che tutto vada bene, si aggiusta ciò che non va, si ritesta, e così via.

Exploring and formulating the problem esplorazione e formulazione del problema; *generating and searching ideas*; *concept*; *prototyping*; *testing and optimizing*; presentazione. La presentazione può valere in diverse fasi: presentare la propria idea, anche più volte, ad altri designer, spiegare al lead perché ha senso realizzarla rispetto a un'altra; se arriva l'approvazione, secondo un processo molto regolamentato, si passa alla fase successiva, che è sempre più un discorso di produzione: assicurarsi che i meccanismi della macchina girino tutti nello stesso modo.

Il ciclo in quattro spicchi analisi (che fa parte della preproduzione), sviluppo delle idee, creazione di una soluzione, valutazione della soluzione dopo il testing. Con il *design brief* come punto di partenza.

Mettere a terra un'idea. Il docente descrive il passaggio con più dettaglio. Si ha un'idea, la si è attaccata da una serie di punti di vista, si è fatta l'analisi e sembra funzionare: si prova a metterla a terra con un **prototipo**, qualcosa che sia giocabile in qualsiasi maniera, anche con le carte, con le scatole di cartone, con i disegni alla lavagna o con i blueprint di Unreal. Il prototipo **va giocato**: "non dimenticatevi mai quanto

è importante giocare le cose". Anche quando si è fatto solo un *fix*, bisogna testarlo e controllare che non abbia distrutto il resto del progetto. Avere un prototipo a sé permette di testare **solo quella feature**, in un ambiente chiuso. Ci si mette nei panni dell'utenza: piacerebbe a questo tipo di giocatore? Risolve il problema? Se la cosa è perfetta per l'utenza, o almeno si avvicina a ciò che si voleva, la si mette nel progetto e si passa a un'altra meccanica; altrimenti si migliora e si ritorna indietro.

Analisi, pianificazione, valutazione. Nell'ultimo ciclo mostrato si ritrovano tutte le fasi: l'analisi precedente, ad esempio del mercato e dei competitor (qualcuno ha già fatto una cosa simile, o proprio la cosa che si vuole fare?); lo sviluppo e la valutazione delle idee, smontandole o meno; la **pianificazione della produzione** dell'idea (quanto costa, quanto tempo ci vuole, che cosa si può testare prima e che cosa dopo). Si pensi a un sistema molto complesso come i portali di *Portal*: il primo prototipo non era perfetto, né testava tutte le meccaniche. Magari c'era un solo portale, poi due, poi i portali hanno iniziato a modificare la fisica, a mantenere la velocità del personaggio. Quale di queste cose sviluppare con priorità e quale dopo? Quale vestire e quale no? Poi si crea la soluzione (un prototipo, o un piano logico) e la si valuta. Vale per i prototipi e vale per la documentazione: si fa la prima versione, si controlla, si decide che cosa modificare, aggiungere o togliere, e si ricontrolla.

3.3.7 Il termine chiave: iterativo

Questi cicli esistono **per qualsiasi cosa**: per lo sviluppo, la creazione e l'assemblaggio di auto sportive; per computer, telefoni e dispositivi elettronici; per lo sviluppo di nuovi farmaci (con la ricerca, scherza il docente, non sui topi di laboratorio ma "su grossi blocchi di tofu"); persino per il design dei vasi da bonsai. Quello che hanno in comune tutti questi framework, non necessariamente legati al game design, è la **forma circolare**: sono ricorsivi, ripetono, ritornano, e c'è un punto in cui valutano, ordinano, filtrano, per poi tornare al punto precedente.

Sono iterativi. Fare design iterativo è l'unico modo per avere un buon prodotto.

Un prodotto migliore del precedente, più vicino a ciò che si vuole fare, potenzialmente più venduto, più apprezzato dall'utenza, più vicino all'intento di design di evocare emozioni specifiche nel giocatore: un prodotto migliore *in toto*, perché a ogni giro si tocca una parte diversa, o si migliora la stessa. Tutti i prodotti fatti dagli esseri umani, al primo giro, "fanno schifo", e a volte non si possono nemmeno usare.

3.4 Iterazione e realtà della produzione: le domande

3.4.1 Se una meccanica strutturale non funziona

Federica chiede: se alla millesima revisione ci si rende conto che una meccanica strutturale del gioco non funziona, bisogna cambiare tutto? Se, ad esempio, sta per uscire *Cyberpunk* e ci si accorge che qualcosa non va, si cancella tutto o lo si fa uscire comunque?

La risposta "da mondo del Mulino Bianco" è: si può anche **ripartire da zero**. Il motivo per cui si fa prototipazione è proprio questo: **abbandonare velocemente ciò che non funziona** e testare, il più velocemente possibile, se funzionano altre cose. Se qualcosa non funziona, va eliminato o aggiornato, e questo può valere anche per il gioco intero.

Ma i videogiochi sono prodotti, con regole specifiche. Una delle ragioni per cui ci sono fallimenti, o giochi che non ottengono il risultato sperato, è proprio questa: si è arrivati troppo tardi a modificare o a valutare qualcosa. Oppure è intervenuto qualcosa di incontrollabile: il mercato si è spostato. Il docente è sicuro che là fuori ci siano aziende (gliene vengono in mente già due) che stanno sviluppando un friendslop e potrebbero essere in ritardo: al rilascio il mercato potrebbe non assorbirlo più, perché saturo, o perché quel tipo di giocatore si è spostato su altri giochi. Ci sono casi controllabili e incontrollabili, e il designer deve agire **su ciò che può controllare**.

Alcuni punti fermi:

- **Comunicazione e soft skill** (sezione 1.5): non si può andare dal producer o dal capo a dirgli "sei uno scemo, dovevamo farla prima questa cosa", o "non possiamo farla perché non c'è più tempo". Bisogna

mediare. E spesso è lo stakeholder, o il CEO, a dettare a grandi linee che cosa va fatto: si agisce entro quei limiti.

- **Limitare i danni** fa parte, purtroppo, del lavoro.
- Conoscere il principio dell'iterazione permette di **ragionare già in vista del fatto che le cose dovranno cambiare**. Tanto più velocemente si controlla, si osserva, si "spacchetta", tanto meno si presenterà il problema. Una meccanica non la si "butta dentro" il progetto appena sembra funzionare: con i *branch*, con altri sistemi di versionamento e altre metodologie, ci si assicura prima che funzioni e che si sposi con il resto. Non si fa un test, né due, né tre: se ne fanno n, finché ci sono le risorse.
- Il lavoro del designer **non è buono al primo tentativo** e non lo sarà mai (ammesso che il gioco non debba uscire domani). Non si può fare *Cyberpunk* in un giorno, ed era chiaro anche agli operativi che non si poteva sviluppare *Cyberpunk* in quel tempo: lì il problema, senza voler puntare il dito, era principalmente produttivo, legato a un sistema specifico adottato. I giochi vanno tarati per gestire anche il tempo necessario a riguardare le cose più volte.

La risposta "legale", quindi: se qualcosa non funziona, si cambia, cercando il miglior compromesso possibile; idealmente, anche distruggendo il gioco. Ed è per questo che bisogna avere un prototipo il prima possibile: per testare subito se l'idea ha senso.

3.4.2 Se l'ordine arriva dall'alto

Federica chiede ancora: e se è il produttore a volere a tutti i costi che il gioco esca, anche se non funziona fin dall'inizio?

Il docente risponde "da persona operativa, moderatamente frustrata", perché gli è capitato non poche volte. Sviluppare progetti è un lavoro di team (primo capitolo): come il designer si fida del producer, e del fatto che non gli tiri via il tappeto da sotto i piedi, così il producer si fida del designer.

A chi si chiedono le stime. Il docente precisa di non essere, in quella fase, un lead, un creative director o un game director: è a un livello più basso. Nelle fasi iniziali la produzione "a grandi linee" la si chiede ai lead o a chi sta sopra. Non si va a chiedere a ogni singolo designer se si sta scegliendo di fare "*Cyberpunk 3099*"; non si va dal designer delle 3C a chiedere quanto ci vorrà per fare la camera del personaggio, perché bisognerebbe ripetere il processo per 700 persone, il che non è realistico. Lo si chiede al director, che darà una stima a spanne. Durante la produzione, di momento in momento, il producer può chiedere direttamente, e il lead si assicura che i task settimanali o bisettimanali siano nel reame della fattibilità, ora e in futuro. A quel punto sì, il producer chiede: "Mattia, secondo te quanto ci vuole per sviluppare questo sistema di combattimento?".

Nessuno lo sa davvero. E qui il docente ammette: **chi dice di saperlo è un bugiardo**. Dipende da quanto si è amici del producer, ma la risposta onesta è "non lo sappiamo, ti posso dare una stima a spanne". Anche quello del producer non è solo un lavoro preventivo: non si può prevedere esattamente quanto durerà la produzione. Il producer fa prima la pianificazione e poi il **damage control**, sistemando i problemi che sicuramente capiteranno.

Il margine. Come rendere veritiera una stima? Uno dei "buoni usi e costumi" più semplici è **aggiungere tempo**. Al designer è capitato di dare la propria stima aggiungendo tre giorni, e che il producer, benevolo, ne aggiungesse altri tre: un margine più ampio. Poi può succedere davvero un disastro: il designer viene investito da un'auto, il progetto "esplode", qualcuno elimina il repository. Lì si tratta di qualcosa di non prevedibile.

3.4.3 Publisher, licenze e stakeholder

Federica cita *Dragon Age: The Veilguard*, chiedendosi come un gioco del genere riesca alla fine a uscire. Il docente coglie l'occasione per allargare il discorso. (In una parentesi, cita anche un trailer di *Mass Effect*

realizzato con asset di preproduzione e diffuso da alcuni sviluppatori dopo la chiusura dello studio da parte di EA: un gesto comprensibile per la frustrazione, e bello da vedere per capire che cosa il gioco sarebbe voluto essere, ma **non professionale**, perché rompe qualsiasi tipo di NDA.)[4]

I limiti visti finora (soldi, tempo, numero di persone coinvolte) sono facilmente accettabili: chiunque deve farci i conti, e possono anche dare una spinta non indifferente alla creatività. Ma poiché il gioco è un prodotto che deve essere venduto, entrano in gioco altre questioni. L'azienda può appoggiarsi a un **publisher** che richiede una certa meccanica nel progetto, che vuole dire la sua. Qui prima il director e poi il designer devono saper **mediare**, capire che cosa vuole davvero lo stakeholder e tradurlo in qualcosa che abbia senso nel gioco, mantenendo una certa *linea editoriale*.

La control room. Il docente ricollega il discorso a un feedback ricevuto in mattinata da un'altra classe (Alessandro), da parte di una *control room*: un ente esterno, che funziona esattamente come un publisher. Uno studente obietta che, in quel caso, la control room sa bene di che cosa si parla; vero, ma non è necessariamente così nell'industria. I publisher potrebbero non sapere nulla di giochi e avere soltanto un enorme capitale da investire; essendo investitori, vogliono un ritorno altrettanto ampio. Poiché mettono i soldi, a seconda di ciò che si firma si può **cedere parte della propria libertà**. Bisogna saper muoversi in questi reami: capire che cosa cedere, soprattutto in base a chi chiede qualcosa e a dove si trova nella gerarchia, per fare il miglior lavoro possibile. Non per sé (oltre a dormire tranquilli la notte), ma **per il giocatore**: perché gli arrivi qualcosa che abbia senso giocare e che gli piaccia, cioè che evochi il responso emotivo corretto, desiderato. È la parte difficile, un amalgama di tante cose che si vedranno piano piano.

"Voglio le macchine rosse". E se il publisher chiede qualcosa che non funziona, si è costretti a farlo? Bisogna **capire che cosa vuole davvero**. Davanti a un diktat come "voglio le macchine rosse", la prima domanda da porsi è: *perché* le vuole rosse?

Le licenze se si firma un accordo con un marchio automobilistico, il marchio vuole che quella macchina finisca nel gioco così com'è: non le si possono mettere "gli alettoni, le ali volanti". C'è poco margine di manovra, e nei giochi simulativi conviene capire quanto margine abbiano i designer: spesso la Ferrari vuole che le sue auto siano *statisticamente* le più veloci e le più forti del gioco; oppure il marchio non vuole che l'auto possa essere distrutta. La Ferrari tiene alla propria immagine e non dà licenza per distruggere le proprie auto. *Mario Kart* ha collaborato con Mercedes (in aula si era ipotizzata la Opel), e lì il problema non si pone: in *Mario Kart* le auto non vengono distrutte né modificate; se arriva un guscio blu in testa, l'auto resta uguale.[5]

Il caso Wipeout in un gioco di corse (il docente cita *Wipeout*, pur non essendone certo), il publisher lamentava che le macchine fossero troppo lente. I designer erano sicuri che il sistema funzionasse correttamente, e il feedback del publisher non era davvero "le macchine vanno troppo lente", ma "fatemele più veloci". Il compito del designer è **continuare a fare domande** fino a trovare la **radice del problema**: le macchine non venivano *percepite* come veloci. Due effetti a schermo, due effetti di lente della camera, due VFX, due scie: problema risolto senza toccare nessuna statistica. Anche questo è lavoro di design.

Questioni che non riguardano il design. Ci sono poi questioni economiche: quanto costano i team, o perché certe acquisizioni vengono fatte (ad esempio per i ritorni e le agevolazioni fiscali legate alla quantità di persone e di team che si assorbono). Sono questioni complesse, di mantenimento dell'impresa e di marketing: riguardano il design solo se si è CEO, e non verranno trattate, probabilmente nemmeno nei tre anni, a meno di un workshop apposito.

3.5 I principi dell'iterazione

L'unico modo per rendere un prodotto sempre migliore, sempre meno peggiore, è il processo iterativo.

[4] L'episodio è riportato così come raccontato in lezione; non è stato verificato in questa sede.
[5] Si tratta del contenuto aggiuntivo gratuito Mercedes-Benz per *Mario Kart 8* (2014).

È **la prima regola** da "stamparsi in testa": assorbirla, masticarla, ingoiarla completamente. I progetti brutti, terribili (con un'arte terrificante, a cui manca un pezzo, pieni di bug), spesso lo sono perché sono passati poche volte attraverso questo processo. È il primo metro di misura.

Uno studente fa notare che vale anche per le **animazioni**. Certo: in un sistema di combattimento le animazioni iniziali sono provvisorie. Si può trovare in rete il prototipo del combattimento di *Bayonetta*: "fa schifo", ci sono figure placeholder e pose provvisorie che si alternano ogni due secondi, ma dà al designer l'idea delle tipologie di animazione legate al combattimento, come il *wind-up* e l'*anticipation*. Questo discorso si fa (o si dovrebbe fare) in tutti i dipartimenti.

Dal principio dell'iterazione discendono alcuni corollari.

3.5.1 Target audience

I prodotti sono creati per uno specifico **target audience**, abbreviato **TA**. L'obiettivo del professionista è creare *per altri*: il sudore, il tempo e il lavoro vanno fatti in direzione di quei giocatori.

E Balatro? Una domanda (che il docente attribuisce a Daniele, e che dice gli abbia "aperto il mondo") è: perché allora lo sviluppatore di *Balatro* ha avuto tanto successo facendo, di fatto, il gioco per sé? La risposta è semplicistica, e ci sono molte altre questioni da valutare, ma la prima, per il discorso sul target audience, è che **era lui il proprio target audience**, e "per puro caso" di persone come lui, con necessità e richieste simili, ce ne sono parecchie. Vale per molti altri sviluppatori indipendenti: Toby Fox con *Undertale*, per esempio. Poi si possono investigare altri motivi del successo: *Undertale* è pieno di citazioni e di riferimenti al mondo di internet e a un certo tipo di cultura moderna; *Stardew Valley* ha ampliato molti sistemi di *Harvest Moon* (il ciclo giorno–notte, le relazioni con gli NPC, la possibilità di personalizzare la propria fattoria, la progressione per cui si sblocca prima una zona, poi l'isola, poi il vulcano). Il docente invita quindi a "parafrasare" con cautela il caso dei giochi fatti per sé: **ogni gioco ha un target audience**, e va capito quanto è ampio o stretto.

Soddisfare l'utenza. Bisogna prestare attenzione all'utenza, cercare le cose che le piacciono, che cerca, o che la stimolano nella direzione corretta. La madre del docente non giocherebbe mai un gioco che punta a essere estremamente eccitante, o che fa paura, né un MMO: interpellata in aula (con i titoli che gioca), si capisce che non cerca il *grinding* estremo o la socialità estrema di un gruppo di cinquanta persone che fanno *raid*. Gli elementi implementati nel gioco sono per il target audience. Ci saranno compromessi con gli altri dipartimenti, tutto deve "cliccare" insieme, ma l'obiettivo finale è questo:

Ogni decisione di design è fatta per soddisfare il target audience.

Ne consegue che, quando si discute con un collega, non lo si fa per avere ragione o per dimostrare che il proprio design è migliore: si sceglie l'opzione **più sensata per un terzo**, l'audience. Non "ho ragione io", non "voglio questa parte, data da me, nel gioco". Il gioco è *di tutti e di nessuno* di quelli che ci lavorano: è di chi lo riceve.

3.5.2 Involvement

Il target audience dovrebbe essere **coinvolto** in parte della produzione; va capito quanto. Nei giochi indipendenti l'audience è chiamata spesso in causa, anche per necessità e per ragioni di marketing; nei giochi più grandi non la si coinvolge nei prototipi molto brutti, ma esistono gruppi di testing molto ridotti, composti da giocatori che potrebbero essere il target ideale, che testano il gioco e danno feedback.

- Va testato soprattutto il **core concept**, l'idea iniziale, per capire se ha senso. Un Metroidvania con dinosauri fatati, per il docente, è un'idea fantastica; ma magari solo per lui, e nessuno là fuori ha bisogno di quel gioco: allora forse è il caso di non farlo. Magari i dinosauri li vogliono demoniaci, e allora lo comprerebbero tutti. Forse: servono i dati, serve la ricerca.
- Vanno valutate **build testabili**, per assicurarsi che il gioco venga compreso. Capita continuamente di creare un livello in cui per il designer è tutto chiarissimo (la progressione, dove andare, che cosa fare) e di darlo in mano a qualcuno che non l'ha mai giocato e che non ha idea di che cosa fare.

- Quando si dà una build a un **publisher**, deve essere "a prova di scemo": molti publisher non sono giocatori, e lo stesso vale per alcuni giornalisti. Il docente cita il famoso video di un giornalista alle prese con la demo di *Cuphead*, incapace di superare il tutorial (bisognava premere un salto e poi uno scatto in aria, cioè due tasti in sequenza).[6] "Il giornalismo videoludico ha bisogno di un'evoluzione", commenta; ma se il gioco lo si deve vendere alla propria madre, non le si può mettere un ostacolo che richiede di premere più di un tasto alla volta.

- Si osserva anche, lato marketing, come risponde l'utenza: che cosa piace e che cosa no. I *reel* "brain rot" funzionano, ahimè; e tutti conoscono le pubblicità dei giochi mobile con gli omini che sparano e poi falliscono: sono fatte così per una ragione precisa, perché attirano la stragrande maggioranza dei giocatori.

3.5.3 Iteration

I giochi evolvono e migliorano grazie al **feedback** di utenti ed esperti. Prima si fa testare il proprio design, il proprio prototipo, a un altro designer, possibilmente uno che conosce poco il progetto (meno lo conosce, meglio è), e si verifica se recepisce le informazioni nella maniera corretta: se capisce dove deve andare, che cosa deve fare, se la storia che recepisce è quella che gli si vuole raccontare. Gli studenti di narrativa più tendenti al *worldbuilding* che al *narrative design*, ad esempio, tendono a scrivere storie estremamente complesse, con "27 milioni di nomi" e neologismi impossibili da ricordare ("il Bibus legato al re Bobus, del regno Namus").

Bisogna quindi testare, controllare e **chiedere**: hai capito questa cosa? Come l'hai recepita? Ti è piaciuta? Perché? Che emozioni ti ha dato? Una volta interiorizzata la lista delle emozioni (sezione 2.12), si può anche **ridurre il passo**: fare domande volutamente informali, che non si farebbero a un altro designer ("ti piace sparare? ti piace esplorare? ti piace parlare con altre persone? ti piace sbloccare achievement?"), e ricevere in cambio molte più informazioni. Si prendono le emozioni, le si accorpano in gruppi e si cerca la domanda più generica possibile che però indirizzi nel modo migliore verso uno di quei gruppi. HoYoverse (ex miHoYo, gli sviluppatori di *Genshin Impact*) è estremamente brava a fare questo tipo di sondaggi, e il docente ammette di "rubare" molto dalle loro domande. Forse, se ci sarà tempo, si farà un "Survey 101".

3.5.4 Holistic

Sempre per ragioni di produzione, l'idea è di **sviluppare in parallelo** quante più cose possibili, entro dei limiti.

Il block out. Il docente chiede quanti sappiano che cos'è un *block out*; è giusto non saperlo. I livelli si fanno così: il **level designer** lavora sugli **ingombri geometrici**, dà l'idea del livello e la documenta (se il blocco di "tofu" rappresenta una magione romana, bisogna spiegare come dovrebbe essere e che cosa contiene); poi l'**environment artist** lo *veste*, lo "dressa", mettendoci sopra i *prop*. È come dare a un bambino un disegno da colorare: il level designer traccia i contorni, gli artisti ci colorano dentro. Solo che, nel 50% dei casi (forse meno), **gli ingombri non vengono rispettati**: un albero si pianta davanti al giocatore e non lo lascia proseguire. Bisogna quindi testare il livello e modificarlo: è il discorso dell'iterazione, e succede sempre, tanto più quante più persone lavorano al progetto.

Se invece si sviluppa insieme (mentre si fa il prototipo della meccanica, il VFX artist fa l'effetto, l'SFX artist fa il suono, il programmatore inizia a montare le cose e il designer osserva che tutto vada in linea, che i tempi siano rispettati), si va più veloci: si raggiunge prima qualcosa di giocabile, lo si testa prima, lo si itera prima e più volte, e il gioco, per un'equazione semplice, diventa migliore.

Che cosa significa olistico. Uno studente propone una definizione: *olistico* è ciò che raccoglie più questioni; secondo una visione olistica, $x + x + x$ non è uguale a $3x$, ma a qualcosa di maggiore, perché la visione è l'insieme di tutto. Il docente precisa che qui il senso è più pratico: **includere tutti i dipartimenti**, più sfere possibili. Se si porta il proprio design (la meccanica, il sistema) il prima possibile davanti agli altri

[6] Il video è quello registrato da Dean Takahashi di VentureBeat alla Gamescom 2017, diventato virale proprio per la difficoltà mostrata nel tutorial.

dipartimenti, invece di rimbalzare tra team diversi con un'idea già terminata, il programmatore, l'artista o il sound designer possono vedere qualcosa che il designer non vede, o un limite del loro dipartimento. L'idea può venire da chiunque, ma ognuno è specializzato nel proprio campo: se l'artista dice "questa struttura modulare non riusciamo a farla, dobbiamo farla così", si può già pensare a un'altra soluzione.

C'è però un ordine: la pipeline ha dei passaggi precedenti e successivi. L'artista non si mette a vestire il livello se il livello non c'è, né mentre il level designer lo sta costruendo: prima lo si costruisce, lo si consegna all'artista, che lo veste e lo restituisce, e lo si testa. Allo stesso modo, prima si fa il design e poi il programmatore lo implementa, altrimenti si inventano cose che vanno rifatte. Ci si può portare avanti, ma l'obiettivo è **incastrare i lavori nel modo più efficiente possibile**: è compito del producer, e fa parte dell'ottimizzazione della macchina che permette iterazioni più veloci e più numerose.

3.5.5 Adaptive

È il discorso fatto con Federica e a proposito del producer: **il gioco cambia**, per forza. Bisogna essere bravi a prendere la palla al balzo e trovare il cambiamento più efficace.

Ci sono però cose che dovrebbero restare quelle: i **pilastri** del gioco, che si vedranno in un'altra lezione di documentazione, legata a un documento specifico. Quelli non si cambiano. Se vengono meno, **non è più lo stesso gioco**, non è la stessa esperienza, e a quel punto si fa un'altra cosa. È anche il motivo per cui spesso le produzioni sono infinitamente lunghe, o arrivano al traguardo con qualcosa di incoerente o molto diverso da ciò per cui si era fatta preproduzione: non si sono mantenuti, salvaguardati e condivisi i pilastri all'interno del team. Se uno di essi viene meno o non è condiviso, il progetto non c'è più, ed è molto più vantaggioso "tirare via tutto" e ricominciare da capo.

3.5.6 I segreti per iterare

Create un prototipo il prima possibile. Dopo la fase di ricerca e di documentazione, il prototipo va creato **il prima possibile**.

- Deve **costare poco**: non ci si deve spendere troppo tempo.
- Deve essere **efficiente** nel testare ciò che si vuole testare.
- Deve essere **abbastanza completo** rispetto a ciò che si deve testare: una feature (un sistema di camere, un sistema di locomozione) o l'intero progetto.
- Deve essere **giocabile**, non importa come: digitale, gioco di carte, scatoloni, o il docente e Valerio che prendono una spada di cartone e si "menano". Qualsiasi cosa può funzionare: un LARP (*live action role-playing*), cioè persone in un prato con le asce di polistirolo; D&D con le persone; persino il *modding* di altri giochi.

Il prototipo serve anche a comunicare. Uno studente chiede se anche per una presentazione del concept del progetto convenga avere già una build. Sì: è utile non solo per validare internamente se il gioco funziona, ma anche per **far capire a un altro che gioco si intende**. Non si ha idea di quanto sia difficile comunicare un gioco che non esiste. Anche i *core elements* della lezione precedente servono a questo: partire da dei pezzi, come le *game qualities* di Romero e Schreiber, dà già un'idea. "Metroidvania" può essere qualsiasi cosa; "Metroidvania gotico, ambientato in un mondo di insetti" (...e magari "se ne potrebbe pure fare un sequel") fa già capire a tutti che si parla di *Hollow Knight*. Più elementi chiari si aggiungono, più il gioco è chiaro. E se il gioco lo si può già giocare (se si vede il cavaliere e lo si può muovere, se si vede una concept art di come si presenta il mondo, o anche solo un quadrato con uno smile che muove un ago contro un blocco rosso, e si sente il *feeling* del salto, del colpo, della parata), il gioco **prende vita**, e lo si può comunicare, testare e valutare.

Fate giocare i prototipi. I prototipi non restano "lì nel nulla": vanno giocati da esperti e da parte del target audience, con dei limiti. Se la build è lacunosa in alcuni punti, lo si dice al giocatore e si creano dei *limiti* ("non date feedback su questa parte, concentratevi su quest'altra"), così come il docente aveva chiesto

alla classe di non badare all'aspetto delle proprie slide. Si prendono appunti e dati, non giudizi del tipo bello/brutto: si crea un form, si fanno domande, si cerca di ottenere quante più informazioni possibili. Poi si **analizzano** i dati ottenuti, per migliorare in piccolo quel design o quella feature, o in grande il progetto. E lo si fa non una, né due, né tre volte, ma **finché si ha il tempo di farlo**.

Il primo segreto. Il motivo per cui, soprattutto in passato, i tripla A erano così gradevoli da giocare è che questo processo iterativo veniva fatto per anni; potevano permetterselo perché avevano molti soldi. Ma se le risorse vengono sfruttate così, il guadagno, quello *artistico*, è assicurato.

Più è lungo il processo iterativo, migliore è il risultato. Prima ci si arriva, prima lo si fa, più lo si fa, e meglio il gioco diventa.

Il ciclo rivisto. Ecco perché il primo ciclo mostrato era bruttissimo: anche quello si può migliorare, si può iterare. Il docente lo ripresenta arricchito:

- la freccia dell'iterazione, come suggeriva Federica, può arrivare fino alla preproduzione, ma l'iterazione si fa sia sull'intero gioco sia, con frequenza ancora maggiore, sulle singole feature in produzione;
- non c'è solo la beta: ci sono diverse **milestone** e diverse **build**, le più note delle quali sono *alpha*, *beta* e *gold* (il loro significato verrà spiegato più avanti);
- dopo il rilascio si può proseguire con le LiveOps;
- tra l'inizio del progetto e la preproduzione c'è la **visione**: fa parte della produzione, ma è il momento in cui si condivide con il team la visione e i famosi pilastri. Se la preproduzione è lo scheletro della casa, la visione è la *fondamenta* del progetto, "la pavimentazione della pavimentazione"; i pilastri sono i massicci blocchi di cemento che tengono su la struttura;
- in **preproduzione** si fanno generalmente visione del progetto, ricerca, analisi, prototipazione e documentazione;
- in **produzione** si fanno implementazione, *dressing*, *management*, *tuning* (il bilanciamento, di cui si è parlato nel primo capitolo) e, ancora, documentazione, oltre alla prototipazione, che continua.

Il disegno è brutto perché l'idea è di iterare: prima lo si fa su Paint, in modo terribile, "con i cosi sbagliati" (anche per comodità, soprattutto per le modifiche), poi ci si pensa a migliorarlo. Non sarà mai la versione finale.

Il secondo segreto. "Vi ho mentito", ammette il docente: i segreti sono due.

Prima createlo, poi fatelo diventare bello.

Bisogna creare qualcosa di brutto, e non lasciare che il documento o il task resti una pagina bianca per paura di buttarci sopra delle cose. Si creano dati, si guardano video, si scrive, si butta giù qualsiasi cosa; poi si taglia, si "trincia". Queste due regole servono nel lavoro di tutti i giorni, adesso, e serviranno a breve, quando si parlerà di progetti.

3.6 Perché la documentazione è importante

3.6.1 Una pausa e un promemoria

Dopo una pausa di sette minuti, il docente ricorda di aver dimenticato di parlare di un concetto di **Raph Koster**, che non tratta subito per paura che prenda troppo tempo: chiede agli studenti di ricordarglielo nella lezione di lunedì. Si tratta di qualcosa di molto utile e introduttivo, legato alla lezione su che cos'è un gioco (capitolo 2): non proprio un framework (anche se un framework c'è), ma un concetto da cui sono stati derivati altri framework, legato alla mentalità di *dove sta il gioco nel play*, cioè che ruolo ha il giocare e soprattutto come viene **percepito**. Era emerso in alcune definizioni degli studenti, e Koster lo spiega in maniera regolata e limitata.

Il formato del task. Una studentessa chiede se il documento del task debba essere un testo libero o una serie di paragrafi. Il docente ne ha parlato a inizio lezione (sezione 3.2) e invita a riguardarla: per ora valgono soltanto le regole scritte nel documento delle regole dei task, più le precisazioni date a voce.

3.6.2 La prima vera lezione di documentazione

Dopo la lezione zero, inizia la prima vera lezione di documentazione. Anche questa si apre con un meme, "estremamente vero": se un professionista ti dà un documento terribile, **è una persona cattiva**.

Perché la documentazione è così importante, e perché il docente la sottolinea in modo particolare (oltre al fatto che è una sua specialità)?

- Perché **è il primo lavoro del designer**, la prima cosa che deve fare.
- Perché la capacità di documentare e di **adattarsi a diversi formati** di documentazione determina la differenza tra un lead e un junior o un intern, tra un game director e gli altri. Non in toto, ma la documentazione è già una buona *cartina tornasole* dell'esperienza di un designer.

Genji e Mercy. Nella slide compare Genji, il personaggio di *Overwatch*: il DPS che va in giro a dettare ordini e a "sparare documenti in testa alle persone". Il designer **non è** questo. Il designer è **Mercy**, il *support*: la persona che tende la mano agli altri.

3.6.3 A che cosa serve la documentazione

Mantenere la visione coerente tra tutti i membri del team, ai rispettivi livelli. È il primo punto: i pilastri di cemento devono essere visti allo stesso modo da tutti. Non deve succedere che ognuno veda una cosa diversa: che lo si guardi dal dipartimento di arte o da quello di design, il pilastro è lo stesso, la visione è la stessa. E l'unico modo per ottenerlo è **comunicare coerentemente**.

Aggiornare il lettore chi non sa nulla ed entra nel team, leggendo la documentazione, si mette in pari (*up to date*) senza dover parlare con tutti. Parlare è utile, ma non tutti avranno tempo per raccontare il gioco per una giornata intera, soprattutto nelle aziende molto grandi. I documenti raccontano la storia del team, delle persone e soprattutto di quale dovrebbe essere il progetto, di qual è la direzione.

Ricerca una buona documentazione rende i dati non "noiosi": le persone leggeranno i dati e la documentazione con piacere, senza esserne spaventate. La soglia di attenzione è stata "completamente annichilita": quanti, aprendo un libro o un documento, guardano per prima cosa quanto è grande la barra di scorrimento, per prepararsi a capire quanto è "gargantuesco"? Le cose che attirano sono quelle ben documentate. Per questo si cerca di dare un focus specifico a un certo dato, o di metterlo in grafico, perché sia comprensibile e facilmente digeribile. Il lato oscuro di questa capacità è la **manipolazione dei dati**: "da grandi poteri derivano grandi responsabilità". Va usata onestamente, perché l'obiettivo non è far valere la propria idea, ma trovare l'idea migliore per il target audience.

Supporto è il motivo per cui c'è Mercy: aiutare gli altri. Con la documentazione si aiutano gli altri dipartimenti a ragionare come il designer, a capire ciò che ha pensato e poi a metterlo a terra. Invece di passare tutto il tempo "sulla spalla" del programmatore a dirgli fai questo, fai quello, no così non va, gli si dà un documento con esattamente ciò che si vuole, nel modo esatto in cui lo si vuole, e il programmatore si preoccuperà di implementarlo. Vale anche per gli artisti: dove è ambientato il livello, come dovrebbe essere vestito. In un gioco ambientato a Babilonia: come ragionavano i babilonesi, com'erano strutturate le strade, com'era fatta una ziggurat. Così gli altri vedono un mondo che ancora non esiste nello stesso modo in cui lo vede il designer. Per questo il designer deve avere la visione il più chiara possibile: se ce l'ha chiara e la documenta nel modo migliore, **anche gli altri vedono ciò che non si vede**.

Controllo la documentazione permette di tenere sotto controllo la produzione: spacchettare le cose che servono, averle localizzate (sul Drive, su Confluence, su Miro, dove si vuole). Se si sa quali passi fare e quando, si sa anche se ci sono problemi: se ci si sta arenando su una parte di una feature, o non si riesce

a raggiungere una certa versione, lo si può affrontare con il team e con il producer. La documentazione serve a tracciare lo stato del progetto, nel grande e nelle sue piccole parti. Nessun designer documenta l'intero progetto: ognuno si occupa di una parte, e insieme fanno il tutto.

3.6.4 I documenti evolvono

I documenti evolvono con il team e con il progetto.

I documenti non restano scolpiti nella pietra, tranne alcuni. Il **documento di visione**, ad esempio, solidifica i pilastri: si fa e così resta. Ma gli altri, come il *game design document* di una feature, vengono aggiornati sulla base del processo iterativo: alla data del 23 ottobre la feature è così; si testa, si itera, non va bene, si aggiorna il documento; il 25 o 26 ottobre ci sarà un nuovo documento con le informazioni sullo stato della produzione in quel momento esatto.

3.6.5 La vostra firma e la vostra arte

Il biglietto da visita. La documentazione è la prima cosa che si fa ed è la propria **firma**, il proprio biglietto da visita, anche per il portfolio. I **test di selezione** delle aziende (Ubisoft, CD Projekt Red, Stormind e così via) sono generalmente **documenti**. Per un ruolo da level designer probabilmente verranno chiesti anche dei block out; per un ruolo da technical designer una bozza o un prototipo, ad esempio di un combattimento. Ma per il game designer (e a volte, è capitato, anche per il technical) il documento è sufficiente: chiedono proprio quello.

La vostra arte. Il docente sposta il discorso su un piano più romantico, che gli sta particolarmente a cuore: per lui, "Mattia Podini designer", fare documentazione è **piacevole**. Potrebbe non esserlo per tutti, ma se si trova il proprio *sweet spot*, un modo di ragionare e di documentare piacevole per sé in primo luogo, il lavoro diventa molto più semplice e anche più apprezzato. Non a tutti piace, ma fa parte del lavoro: trovare un modo buono e piacevole di mettere a terra e condividere le idee, di scrivere. "Questa roba la dovete fare: vi pagheranno per farla."

3.7 Le golden rules della documentazione

Esistono delle regole, le **golden rules**, per scrivere buona documentazione. Non sono dogmi scolpiti nella pietra: il docente stesso ne rompe alcune. Ma lo fa perché ha anni di esperienza alle spalle e conosce "un paio di segreti" della documentazione; le conosce tutte perché le ha *subite* sulla propria pelle quando studiava, perché è stato limitato tante volte, e ormai ne conosce il valore. Gli studenti no: quindi **sotto queste regole ci si sta e non le si rompe**, pena la non lettura. (Sono "i dieci comandamenti", anche se in realtà sono un po' di più.)

3.7.1 Mostra, non scrivere

Il modo migliore per comunicare qualcosa è **farlo vedere**, se si può: immagini, video, *flowchart* (si vedrà che cosa sono), link. Le informazioni vanno condensate in maniera visiva. Se si sta facendo il design di un menu, se si spiega una meccanica, si fa vedere l'immagine: il protagonista che attacca un nemico. Anche uno *stickman* con lo spadone e un nemico triste va benissimo ("punto uno, punto due: gli stacco la testa, esce un sacco di sangue, lui a terra"): un'immagine da sola non basta, ne servono di più, ma è più facile che una persona capisca con le immagini che con le parole. Una cosa non esclude l'altra: si può scrivere il testo e affiancargli l'immagine.

Mock-up e wireframe. Se si parla di UI si creano dei *mock-up*, i cosiddetti **wireframe**. Devono essere brutti (o meglio: più belli si riescono a fare, meglio è; ma averli brutti ha due vantaggi). Il docente chiede alla classe quali siano, insistendo perché gli studenti parlino e facciano domande ("c'è il solito discorso sulla comunicazione"):

1. **Ci vuole meno tempo**, e quindi, per usare la parola giusta (suggerita da uno studente), **si può iterare più velocemente**. Ed è comunque molto chiaro: si vedono gli spazi, come sono organizzati, dove devono andare le cose.

2. Il secondo vantaggio è "politico": si **evita di pestare i piedi** a un altro dipartimento.

Esempio: Hollow Knight e Zelda. Si supponga di avere un sistema simile a quello dei *charm* di *Hollow Knight*. Si confrontano due menu: quello di *Hollow Knight* e quello di uno *Zelda*. Entrambi sono giochi con combattimento corpo a corpo a breve distanza, con un'arma principale (l'ago in *Hollow Knight*, la Master Sword in *Zelda*); *Hollow Knight* ha un sistema di magie che *Zelda* non ha, mentre *Zelda* ha bombe, arco e così via; in *Hollow Knight* ci sono le maschere che aumentano la vita, in *Zelda* i cuori; ci sono gli oggetti (come le chiavi), e in *Hollow Knight* c'è una descrizione per ogni oggetto, mentre in *Zelda* no. Sono, fondamentalmente, molto simili: menu di inventario. Ma se il designer, invece di fare un wireframe neutro, colora il menu di arancione e al posto dell'ago mette una spada draconica trovata cercando "sword PNG", sta già comunicando qualcosa sullo **stile artistico**, che non è di sua competenza né del suo dipartimento: non è un direttore artistico né un UI designer. Se gli viene chiesto di fare la **struttura** del menu e le sue **interazioni**, il suo compito è fare in modo che premendo un tasto si vada sui charm, premendone un altro si vada sulla mappa, che si veda o meno la descrizione (come in *Zelda*): queste cose le decide lui. (Esiste una branca del design dedicata, quella dell'UX/UI design.) Se non si ha l'*ownership* artistica, vestire il wireframe può essere deleterio, perché comunica qualcosa che non dovrebbe.

Le alternative sono due: scrivere "a caratteri cubitali" sopra le immagini che si tratta di *render artistici non definitivi*; oppure, come fa il docente con la narrativa, dare comunque dei nomi (invece di "Pinco Pallino" o "nome TBD"), segnalarli in un certo modo e mettere prima un **disclaimer**: tutte le definizioni narrative sono da rivedere, sentitevi liberi di prenderle, smontarle e distruggerle. Ma va detto.

Un wireframe permette comunque di capire quanto dovrebbe essere grande un elemento nello spazio, se è una schermata intera, se dietro c'è la schermata di gioco o no, dove si va premendo un tasto, se bisogna tenerlo premuto per andare altrove, e così via. Se lo si fa vedere, in generale, si capisce di più.

3.7.2 Essenziale

I documenti devono essere **essenziali**. Non si scrivono documenti di 50 pagine, perché nessuno li legge: le persone che dovrebbero leggerli non vogliono farlo, e se non vogliono non lo fanno, anche se è il loro lavoro; oppure li leggono controvoglia e non capiscono, o non ricordano, nulla.

- Concentrarsi sull'**obiettivo** del documento, su ciò che si deve comunicare.
- Essere **chiari e concisi**.
- Non usare **parole deboli** come "forse sarà così", "potrebbe essere questo o quello". Se un dato è quello, si scrive il dato: se il personaggio ha 50 punti vita, si scrive 50, non "tra i 5 e i 100". Se non si ha un'idea precisa, si scrive comunque un valore, ad esempio "valore intero tra 0 e 100", e poi si vedrà.

Attenzione: essere essenziali non significa fare documenti di una pagina senza pensare attentamente a che cosa scriverci. È **più difficile** che scrivere 70 pagine, perché quelle 70 pagine magari si scrivono comunque, ma poi si procede **per sottrazione**: si parte da tante pagine e si taglia ciò che non serve. È anch'esso un processo iterativo: si parte dal grosso e si arriva al piccolo.

3.7.3 The right tool for the right job

Bisogna scegliere il **supporto ideale** per ciò che si deve comunicare (una domanda che aveva posto anche Antonino). Se si deve presentare un'idea, fare un *pitch*, si usano le **slide**: PowerPoint, Google Slides, Canva o qualsiasi strumento, purché permetta un largo uso di immagini. Se si deve bilanciare qualcosa, non lo si fa su Google Docs o su Word, ma con un foglio di calcolo (Excel o Google Sheets). Vale per il supporto fisico e vale per il contenuto, esattamente come per i framework: si sceglie il framework giusto per il gioco giusto, il tempo giusto, le risorse giuste. Raramente ce n'è uno migliore di tutti; ce ne sono alcuni migliori in determinati contesti (si veda la sezione 3.8).

3.7.4 Coerenza e sequenza

Bisogna comunicare in maniera **coerente** e con la **sequenza corretta**. Non si parla di una meccanica, poi si salta a un'altra, poi si va altrove e poi si rimanda indietro; né si scrive qualcosa a cui manca un'informazione che compare cinquanta paragrafi sotto.

Rientra in questo punto anche la **lingua**: niente errori grammaticali gravi. Eventuali difficoltà specifiche (dislessia, discalculia e simili, di cui si era parlato nel primo capitolo) vengono ovviamente considerate, ma bisogna cercare di essere compresi e coerenti: tempi verbali giusti, rapporto tra coordinate e subordinate corretto, e così via.

3.7.5 Mind the language

Se esiste una **nomenclatura tecnica**, la si sfrutta. Così come non si possono usare le parole bandite, si devono usare le parole e i framework visti a lezione. Come aveva detto il direttore: se il linguaggio condiviso è quello tecnico, e si parla da professionisti, si viene riconosciuti come tali.

3.7.6 Niente decorazioni inutili

Fondamentalmente: **evitare la "monnezza"**.

- Niente decorazioni per abbellire il documento: se un elemento non è utile a comunicare qualcosa, va tolto.
- Niente pagine (quasi) vuote: se in un documento ci sono intere pagine vuote, il docente non lo legge. I fogli A4 "sono oro": lo spazio disponibile va sfruttato tutto, "come l'acqua nel deserto". Bisogna pensare a ogni spazio e a ogni parola che si scrive.
- Non si sta scrivendo una tesi: il linguaggio non è quello accademico in senso lato. I documenti tecnici non sono testi fitti, e hanno regole specifiche, diverse da quelle accademiche.

3.7.7 Grafici con intento

Quando si può, i dati inseriti nei documenti vanno messi in **grafico**, in linea con il primo punto (mostra, non scrivere), ma con **cognizione di causa**: i grafici servono a semplificare i concetti e a far emergere immediatamente il dato che interessa, o il dato su cui si basa una parte del design. Se ne parlerà in lezioni future di documentazione e pitching, ma intanto un ripasso: esiste una miriade di grafici ("se dico grafico a corde, qualcuno sa che cos'è?"; il diagramma *sunburst*...), alcuni più esteticamente appaganti di altri. Si può dare colore alle categorie e modificare qualsiasi cosa, ma bisogna trovare **il grafico giusto che comunichi l'idea nel modo ideale**. Per dire la stessa cosa si può usare un grafico a torta o un grafico a barre; la torta la si può "bucare" (ciambella) o renderla 3D; si può usare un diagramma ad albero (*treemap*), che è "una torta con altre torte dentro". Se invece si deve mostrare una tendenza, ad esempio la progressione delle abilità di un personaggio, serve una curva. Dipende da che cosa si vuole comunicare: ancora una volta, *the right tool for the right job*.

3.7.8 Colori per separare, aggregare, far emergere

I **colori** servono a separare, aggregare e far "poppare fuori" le informazioni (nelle slide del docente, ad esempio, le cose importanti sono in grassetto e in arancione). Lo stesso vale per tutti gli strumenti di formattazione: grassetto, sottolineato, corsivo e così via.

- **Coerenza**: se gli attacchi, o un nome, sono scritti in rosso, si usa sempre quel rosso.
- **Colori morbidi**, che non distruggano la capacità di vedere le cose: un'immagine o una pagina in rosso RGB puro fa "non vederci più per una settimana".
- **Stampabilità**: un documento con testo bianco su fondo nero ha un'ottima visibilità, ma non si può stampare.

L'obiettivo è un risultato visivamente appagante, o il più possibile tale. Come regola del pollice: **guardare il documento da lontano**, a diverse percentuali di zoom. Al 50%, ad esempio, si riesce a identificare il paragrafo più importante? C'è un testo troppo fitto rispetto alle immagini, o un'immagine troppo grande rispetto al testo? Bisogna stabilire delle priorità.

3.7.9 Conosci il lettore

Un documento scritto per un *creative director* è diverso da uno scritto per un collega. Con il creative director non si sarà estremamente tecnici, perché non gli interessa, e troppe informazioni creerebbero più problemi che altro. Per un programmatore ci si preoccupa anche delle **variabili**, che non interesserebbero a un artista. Se si vuole fare il *tuning* dell'attacco o della velocità in un sistema di combattimento, all'artista non interessa; all'animatore forse sì, ma a lui non interessa quanta vita ha il personaggio o di che colore è la barra. Gli studenti, ad esempio, sanno che il task che stanno scrivendo va al docente: possono già ragionare secondo alcune regole, su come ragiona lui, su ciò che vorrebbe vedere un designer che è, potenzialmente, il loro capo.

3.7.10 Sempre aggiornati

I documenti devono essere **sempre aggiornati** (*up to date*), un po' per il processo iterativo, un po' perché la documentazione non resta mai ferma. Se è ferma, o è il "sacro graal" del progetto (la visione), oppure è *outdated* e qualcosa la sostituirà.

Un grosso asterisco. La regola è creare la migliore documentazione possibile affinché il progetto funzioni e venga implementato. Ma c'è un trucco: **la migliore documentazione possibile è il progetto funzionante**. Se si è uno sviluppatore solitario e si ha tutto in testa, scrivere pagine e pagine o documenti diversi per diverse meccaniche è probabilmente deleterio: si sa già che cosa fare (ammesso di non cambiare idea ogni due secondi e di ricordarsi i dati). La documentazione diventa importante quando bisogna parlare con tante persone, o quando bisogna **documentare i passi fatti in passato** per non ripercorrerli. Se si itera, l'iterazione precedente va documentata: se dovesse tornare un feedback simile, o qualcuno chiedesse "perché la meccanica è così e non così?", si può rispondere "perché così l'abbiamo già fatta un mese fa, e questi sono stati i risultati". È una tutela per il designer e per il progetto.

Sotto deadline può capitare di non avere il tempo di documentare, o di documentare bene. In quel caso può convenire non farlo, o farlo ragionando sui limiti e sulle risorse del progetto: si va direttamente *in game*, e quando ci sarà più respiro si aggiornerà la documentazione. Almeno il progetto va avanti. Non è così che si dovrebbe lavorare, ma è il discorso del target audience applicato ai documenti: chi li legge, e quanto tempo si ha. Se la deadline è domani, si può fare poco e bisogna tagliare da qualche parte. (Vale, ad esempio, per le *game jam*, di cui si parlerà.)

3.7.11 Iterate

L'ultimo punto è ormai un mantra: **iterate**, fin quando si hanno tempo e risorse.

3.8 Scegliere il documento giusto

Per ora, come detto, nel Drive si usano **solo gli strumenti di Google**: un documento Word inserito nel Drive non viene letto. Le regole per scegliere il documento sono semplici:

Esistono alternative, e dipendono da dove si lavora. Il docente, ad esempio, in Stormind Games ha usato molti metodi diversi a seconda dei progetti: uno richiedeva Confluence, un altro Miro, e ogni volta bisognava adattarsi a quel supporto. Nel corso ci si adatta agli strumenti indicati dal docente, perché **le aziende fanno così**: danno dei limiti, e si lavora entro quei limiti.

3.8.1 Less is more

Meno si scrive, con più cognizione di causa, più si taglia, e più si diventa comprensibili.

Regola	**In sintesi**
Mostra, non scrivere	immagini, video, flowchart, link, wireframe (brutti: si itera più in fretta e non si pestano i piedi ad altri dipartimenti)
Essenziale	obiettivo, chiarezza, concisione, niente parole deboli; si procede per sottrazione
Right tool for the right job	slide per presentare, fogli di calcolo per bilanciare, documenti per analisi e GDD
Coerenza e sequenza	ordine logico, nessuna informazione mancante, grammatica corretta
Mind the language	nomenclatura tecnica, niente parole bandite
Niente decorazioni	nessun elemento inutile, nessuna pagina vuota, non è una tesi
Grafici con intento	il grafico giusto per far emergere il dato giusto
Colori	separare, aggregare, far emergere; coerenti, morbidi, stampabili; guardare il documento da lontano
Conosci il lettore	livello di dettaglio diverso per creative director, programmatori, artisti
Sempre aggiornati	tranne la visione; la miglior documentazione è il progetto funzionante
Iterate	finché ci sono tempo e risorse

Table 3.2: Le golden rules della documentazione.

Strumento	**Quando usarlo**	**Esempi**
Google Docs	documentazione testuale (*text-telling*)	analisi, *game design document*
Google Sheets (Excel)	tutto ciò che è numericamente pesante	matrici da bilanciare, grafici da costruire e poi importare nei documenti; è "casa" degli *economy designer*
Google Slides (PowerPoint)	comunicazione estremamente visuale	qualsiasi presentazione, qualsiasi *pitch*

Table 3.3: Quale strumento della suite di Google usare.

Il documento di Diablo. Il docente mostra una pagina di un vecchio documento di design e chiede di indovinare il gioco senza leggere: un personaggio "demoniaco, maligno". È *Diablo*: uno dei documenti di design del primo *Diablo*, del 1994.[7] Da allora c'è stata un'evoluzione nel modo di fare documentazione, grazie ad alcuni "padri" che hanno cambiato il modo di documentare, tra cui **Stone Librande**: si è arrivati al punto in cui la documentazione può ridursi a **un singolo documento di una pagina** (non onnicomprensivo, ma il più possibile tale; su questo concetto si tornerà).[8]

Le design bible. In passato ("sfortunatamente passato") si facevano le cosiddette **bibbie di design**: enormi tomi da 50, 100, 150 pagine che condensavano tutto il gioco e che, teoricamente, tutti dovevano leggere, ma che nessuno leggeva davvero ("ma non te lo leggerà mai!"). Si facevano per ignoranza, che ci sta, e perché il mondo del design videoludico va molto veloce e non sempre tiene il passo con certe modifiche. Ancora oggi, ad esempio, alcune di queste idee vengono richieste per ottenere fondi: uno dei fondi europei richiede che il gioco abbia la *design bible*, un documento enorme di tutto il gioco. Ma nessuno parte da lì: la si fa perché si vogliono quei soldi, per "smarcare" l'obbligo; la compone il producer, incollando insieme i documenti che gli vengono dati. Si è andati molto oltre: così come oggi nessuno leggerebbe una bibbia di design (e in realtà nessuno la leggeva nemmeno allora; "non so quanti alberi siano stati tagliati per bibbie di design da mettere su ventimila scrivanie"), l'obiettivo è raggiungere qualcosa di **efficace e condensato** in molte meno pagine, e diviso anche per dipartimento (se ne parlerà più avanti).

La documentazione è ovunque. La comunicazione e la documentazione arrivano dappertutto. La documentazione di **Unreal** è documentazione tecnica a tutti gli effetti: documenti localizzati, una *wiki*. Quando serve qualcosa, ad esempio informazioni sulle *nav mesh*, la si cerca lì, e quella documentazione permette poi di lavorare sui blueprint: è un ponte che aiuta a fare cose, a implementare. Va capito quanto sia efficace e quanti problemi risolva, ma il concetto è che **più è efficace, meglio è**.

3.8.2 Strumenti aggiuntivi

Alcuni strumenti utili, soprattutto per rispondere alla regola *mostra, non scrivere* (non obbligatori, ma buoni per un lavoro preciso):

Photopea alternativa gratuita a Photoshop (che costa), utilizzabile online, per modificare foto e immagini e fare *mash-up*; ci sono le pubblicità, ma sono ignorabili.

Lucidchart (a pagamento) e **draw.io** (gratuito) per creare *chart*, flowchart, wireframe e qualsiasi altra cosa: i designer ci fanno anche dei template. "Uno strumentopolo misterioso che ci servirà più avanti".

Miro gratuito con dei limiti, per raccogliere idee ma anche, potenzialmente, per documentare.

Paint il "buon caro amico" Paint. Se si sta discutendo con un designer e bisogna spiegargli qualcosa in maniera immediata e chiara, uno schizzo su Paint "e passa la paura". Uno studente obietta che si può disegnare con qualsiasi strumento: certo, ma Paint è *percepito* come più grezzo, e quindi, in chiamata, si capisce che è una cosa destinata al cestino; si perde anche quell'attaccamento all'idea.

Meglio brutto e chiaro che perfetto, preciso, con tutte le spaziature, ma non chiaro e potenzialmente fraintendibile.

(Il docente precisa che lo dice anche per sé.)

3.9 Le regole formali del corso

In chiusura il docente aggiunge "un paio di trucchi, un paio di catene e catenacci": le regole formali che valgono per tutti i documenti del corso (verranno aggiunte anche al documento delle regole; "non fate i furbi").

[7] Si tratta del documento di *pitch* di *Diablo* redatto da Condor (poi Blizzard North) nel 1994, oggi liberamente consultabile online.

[8] Stone Librande ha presentato il proprio approccio dei *one-page designs* alla Game Developers Conference del 2010.

Margini tutti i documenti di Google Docs devono avere i margini **1,5 sopra, 1,5 sotto, 1,25 a sinistra, 1,25 a destra**.[9] Il documento deve essere **stampabile**: niente documenti con lo sfondo nero.

Niente formato senza pagine Google Docs permette di usare un formato "senza pagine", a pagina singola continua: è **bandito**, per ora. Non aiuta né nella stampabilità né nel ragionare entro i limiti della documentazione.

Font il font è categoricamente **Roboto 11**, per tutti i testi. Titoli e intestazioni sono esclusi dalla regola: anche lì il docente consiglia caldamente Roboto, ma si è più liberi. Se nel testo c'è anche una sola parola che non è in Roboto 11, il documento non viene letto ("state sicuri che io la becco"). Uno studente chiede se si possa impostare Roboto come predefinito: sì, si può fare.

Sfruttare la toolbar grassetto, corsivo, sottolineato, interlinea, interruzioni: gli strumenti sono moltissimi.

Colori banditi i colori rientrano negli strumenti, ma alcuni sono banditi. Il docente non ne fa la lista ("siete abbastanza adulti per riconoscerli"): è chiaro che i colori RGB puri non funzionano, che il blu acceso si legge a fatica, che il giallo su bianco è illeggibile, che il testo nero su rosso "fa perdere due o tre diottrie".

Link il sottolineato non è necessariamente un link, ma un link deve essere **ben differenziato** da tutto il resto: se non si sa che lì c'è un link, non ci si clicca mai; viceversa, se si pensa che ogni sottolineato sia un link, si clicca ovunque senza andare da nessuna parte.

Rinominare tutto il documento, il titolo e qualsiasi cosa contenga che sia navigabile, come le *tab*. Vale per Google Docs, per Google Sheets (ogni foglio va rinominato) e per qualsiasi altro strumento.

Margini e immagini un'immagine che sfora i margini, un'immagine sopra un'altra immagine, un testo che si sovrappone a un'immagine o un'immagine che copre parte del testo: il documento non viene letto.

Coerenza evitare incoerenze ed errori grammaticali. Se si chiude con il punto ogni frase o ogni voce di un elenco puntato, lo si fa per tutte. Chiudere le voci con il punto e virgola e l'ultima con il punto è accettato ("non sono un grande fan, ma è una questione di preferenze"), purché sia coerente; non lo è "punto, punto e virgola, punto e virgola, punto".

Si può fare tutto. Il mantra da portarsi a casa: **con i documenti di Google si può fare tutto**. Sembrano uno strumento limitante, ma "si può fare il mondo". Non lo si imparerà subito: lo si imparerà piano piano, soprattutto vedendo qualcuno che ha fatto qualcosa che non si pensava possibile, o vedendolo fare al docente. L'idea non è copiare, ma **capire**, mettere la tecnica nel proprio bagaglio e sapere quando usarla. Il docente, ad esempio, è un fan delle tabelle solo fino a un certo punto, mentre i colleghi di Roma ne sono grandissimi fan, "fin troppo". Non è che, se qualcuno è un maestro delle tabelle, allora si usano sempre quelle: si imparano i punti di forza degli altri, si prendono i propri e si migliora; gli altri vedono e migliorano a loro volta.

Gli strumenti da conoscere includono le **tabelle** (con il loro spazio interno: si possono allungare, colorare, se ne possono togliere le linee), gli **elenchi puntati** (difficile trovare una lista più leggibile) e la gestione degli **spazi**. Lo spazio è prezioso, e cederlo volutamente è rischioso, ma può portare grandi risultati: come sempre, a un rischio corrisponde un *reward*.

3.10 Il task della settimana

Non c'è un task nuovo: il task resta quello sui *core elements* dei giochi arcade (sezione 2.14), con la stessa scadenza (**mercoledì alle 13**). Ma ora va svolto applicando le regole viste in questa lezione.

1. Riguardare il documento già iniziato seguendo le **golden rules** (sezione 3.7).
2. Applicare le **regole formali** (sezione 3.9): margini, Roboto 11, colori, coerenza.
3. Il documento consegnato dovrà rispettare entrambe.

[9] L'unità di misura non viene specificata in lezione; nell'interfaccia italiana di Google Docs i margini sono espressi in centimetri.

Come procedere: la risposta a Federica. Federica chiede come collegare concretamente queste regole al task, rendendo il documento accattivante con le immagini giuste. La risposta non è semplice: obbliga a **pensare**, dopo la lezione, a ciò che si è detto, a rimuginarci, o a riguardare la lezione. Alcune indicazioni concrete:

- i **margini** vanno applicati a qualsiasi documento si stia facendo;
- uno studente di Roma ha impostato il task come una serie di slide: probabilmente non è il supporto adatto, perché si tratta di analisi e ricerca sui core elements, e il miglior candidato è un documento di Google. Un altro documento mostrato usa correttamente Roboto 11, ma non ha i margini corretti. Conviene quindi trasporre il lavoro già fatto in un documento di Google con i margini e il testo corretti, e lavorare lì;
- prendere le informazioni che si hanno e provare a modificarle: *come* sono scritte, *quante* sono, che cosa si può *mostrare*. Ad esempio, se si elencano i *game piece* di Schell, si può usare un elenco puntato; e se esistono gli sprite di quei pezzi di gioco, può essere utile mostrarli;
- usare lo **stesso colore** per parlare della stessa cosa, ad esempio dello stesso pezzo di gioco;
- usare il **linguaggio tecnico** ed evitare le parole bandite;
- imparare a non sprecare lo spazio ("lo vedrete sbagliando"); e per i colori, se si ha il dubbio che un colore non sia leggibile (come un verde chiaro su bianco), probabilmente non lo è.

È difficile assorbire tutto e subito: ci vogliono tempo, esperienza e tanta pratica. "Nessun documento andrà bene", rassicura il docente: è il primo documento che si consegna, e sarebbe strano il contrario. "Imparerò sbagliando", conclude Federica: "così si fa".

Lasciar sedimentare. Un ultimo consiglio, non scritto nelle slide: **non fare tutto subito**. Fatto il documento, lo si lascia riposare per qualche ora o un giorno, senza toccarlo, e lo si riguarda il giorno dopo: sicuramente alcune cose appariranno in modo diverso. Anche questo fa parte del processo iterativo. I designer non bilanciano a ruota tutto il giorno fino ad arrivare al punto corretto: se fanno lo stesso lavoro per tutto il giorno, si **abituano all'errore**, a qualcosa che magari è sbagliato ma che ormai sono abituati a vedere (un salto di quell'altezza, ad esempio). Di solito si lavora un po' oggi, ci si ferma, si riprende domani e si verifica se il *feeling* è lo stesso del giorno prima. La settimana a disposizione va sfruttata così: non lavorando ogni ora di ogni giorno (irrealistico), ma dedicando quotidianamente un po' di tempo al task.

Prossimi appuntamenti.

- La lezione di lunedì si terrà in presenza; ricordarsi di chiedere al docente il concetto di Raph Koster rimasto in sospeso.
- Dopo la consegna e i commenti, entro la settimana successiva (al massimo venerdì), si farà una **dimostrazione**: si prenderà un documento e lo si formatterà da capo, per vedere qualche trucco. Poiché venerdì è previsto uno sciopero dei mezzi, quella lezione potrebbe tenersi da remoto.
- Per dubbi e domande si possono usare le chat o parlarne a voce; ma a domande come "questo colore è corretto?" la risposta sarà "ci rivediamo mercoledì".

Punti chiave del capitolo

- Nei framework di core elements: i **game piece** sono oggetti che cambiano stato, i **verbi** sono ciò che il giocatore può fare, le **regole** sono tutto il resto. I **player** di Romero e Schreiber sono il target audience e vanno definiti con dati.
- Lo stesso termine può avere significati diversi in framework diversi (*estetiche*, *mastery*...): l'obiettivo è parlare la stessa lingua del team.

- Con Bura si individua la cella di una meccanica e la si potenzia o blocca per ottenere un responso emotivo; il designer percorre il ciclo variabili–emozioni–interazioni **a ritroso**.
- Il ciclo di produzione comprende preproduzione, produzione e rilascio; per i videogiochi si parla di **DevOps** (prima del rilascio) e **LiveOps** (dopo). Più è profonda la preproduzione, meglio vanno le altre fasi.
- La parte più importante del ciclo è **la freccia che torna indietro**: tutti i cicli di produzione sono **iterativi**, e fare design iterativo è l'unico modo per avere un buon prodotto.
- Ogni decisione di design è fatta per soddisfare il **target audience**; il gioco è di chi lo riceve, non di chi ci lavora.
- L'iterazione è **involvement**, **iteration**, **holistic** e **adaptive**; i **pilastri** invece non si cambiano.
- I due segreti: **più lungo è il processo iterativo, migliore è il risultato**; **prima createlo, poi fatelo diventare bello**.
- La documentazione è il primo lavoro del designer e la sua firma: mantiene coerente la visione, aggiorna, supporta, controlla, ed **evolve** con il progetto. Il designer è Mercy, non Genji.
- Golden rules: mostra non scrivere, essenziale, strumento giusto, coerenza, linguaggio tecnico, niente decorazioni, grafici con intento, colori con criterio, conosci il lettore, sempre aggiornati, itera.
- Regole del corso: solo suite di Google, margini 1,5/1,5/1,25/1,25, Roboto 11, documenti stampabili, tutto rinominato, coerenza formale.

Chapter 4

Problema e soluzione: l'atomo di Koster, il production dictionary e la ricerca in pre-produzione

Il capitolo precedente ha aperto il modulo dedicato alla documentazione e al pitching con il ciclo di produzione, l'iterazione e le regole della buona documentazione. Questo quarto capitolo lo prosegue e compie un passo ulteriore: dalla forma dei documenti si passa a *che cosa* si fa prima ancora di scriverli, cioè alla **ricerca**. Si torna, in altre parole, alla "preistoria" della produzione, la *pre-produzione*, e a tutto ciò che, secondo buon uso e costume, andrebbe fatto in quella fase.

La lezione è tenuta ancora da Mattia Podini, in aula a Milano con una parte della classe collegata da Roma. Il percorso del capitolo ne ricalca la struttura:

1. il concetto di Raph Koster rimasto in sospeso dalla lezione precedente: l'**atomo di gioco**, con il suo ciclo problema–azione–feedback–modello mentale (sezione 4.1);
2. un ripasso rapido della prima lezione di documentazione (sezione 4.2);
3. il primo tipo di documento specifico: il **production dictionary** (sezione 4.3), seguito da due principi generali sulla documentazione, tra cui il fatto che **non esistono template** (sezione 4.4);
4. la nuova lezione, **problema e soluzione**: il design come miglior compromesso possibile, il ruolo dei dati, il *design thinking* e la mappa delle attività di ricerca in pre-produzione (sezioni 4.5, 4.6 e 4.7);
5. gli strumenti di quella mappa, uno per uno: competitor diretti e indiretti, analisi di utenti, mercato e gameplay, *client pitch direction analysis*, stakeholder, problema contro necessità e analisi dei competitor (sezioni 4.8–4.13);
6. il task della settimana, il primo production dictionary (sezione 4.14).

La lezione sui problemi e sulle soluzioni non si conclude in questa giornata: il docente lo anticipa ("non credo riusciremo a finirlo") e infatti la chiusura è rimandata alla lezione successiva. Il capitolo riporta quindi la prima parte, fino all'analisi dei competitor.

4.1 L'atomo di gioco di Raph Koster

4.1.1 Il punto rimasto in sospeso

La lezione si apre, come d'abitudine, chiedendo se ci siano domande o dubbi sulla lezione precedente. Nessuno ne ha; ma, come gli studenti stessi ricordano al docente, alla fine della volta scorsa era rimasta in sospeso "una cosa di cui parlare" (sezione 3.10). Il docente non aveva detto quale: lo dice ora.

Si tratta di un'aggiunta alla lezione su che cos'è un gioco (capitolo 2), utile anche perché si collega a un paio di discorsi extra nati con alcuni studenti in fondo all'aula. In quella lezione il direttore aveva insistito in particolare sul **mindset giocoso**, sulla differenza tra *game* e *play*: l'approcciare un sistema con l'atteggiamento del *play*, su cui si fonda tutto ciò che si è detto finora e il motivo stesso per cui esiste la disciplina. Noi esseri umani prendiamo un sistema, vi applichiamo il concetto di *play* e ci giochiamo sopra. L'esempio era quello di puntare sul mercato azionario; ma il principio è lo stesso di chi, uscendo dall'aula, vede due piccioni che si azzuffano e ci scommette sopra ("esempio poco etico, insomma"). Si veda la distinzione di Koster tra artefatti ludici e sistemi ludici (sezione 2.6).

Da questa stessa riflessione di Raph Koster deriva un altro concetto: quello che in lezione viene chiamato **ludema** o **game atom**[1], l'*atomo videoludico*. Il nome importa poco; importa il concetto, che ritornerà più avanti nel corso "in diverse salse e in diverse forme". Per questo il docente lo presenta adesso: "ve lo portate a casa".

4.1.2 Gli elementi dell'atomo

L'atomo di Koster è un piccolo diagramma a ciclo. Lo si percorre punto per punto, soffermandosi su quelli più rilevanti.

Problema Si parte dal **problema**, inteso in senso lato (ed è, tra l'altro, anche l'argomento della lezione: il termine assumerà più avanti un altro significato, sezione 4.12). Il problema è la **situazione da affrontare di momento in momento** all'interno di un gioco, o di qualsiasi sistema ludico o *artefatto ludico*. Il problema sta dalla parte del **sistema**.

Preparazione Il problema porta alla **preparazione** da parte del giocatore, che sta dall'altra parte: c'è un boss da battere, come mi preparo? Come affronto questo combattimento? C'è una porta da aprire, che cosa faccio? "Gli esempi si sprecano."

Core mechanic Segue una serie di elementi già visti, con nomenclature diverse. La **core mechanic** (meccanica *core*) è simile al *verbo di gioco*; ma, come detto la lezione precedente, il verbo di gioco nel senso gergale moderno è *generico* (sezione 3.1), mentre la core mechanic è **specifica**. È ciò che costituisce il gameplay a basso livello, l'azione che si ripete tante volte, quella che il giocatore compie **molto più spesso di tutte le altre**. In *Bubble Bobble* si sparano le bolle; in *Mario* la core mechanic è saltare e muoversi, molto vicina al verbo perché molto semplice.

Spazio La core mechanic è sempre **calata in un contesto**, che Koster chiama *spazio* (si potrebbe espandere il discorso, ma per ora basta questo). A seconda di dove viene applicata, la stessa meccanica cambia effetto. In *Space Invaders* sparare è la core mechanic: se tra la propria navicella e l'alieno non c'è niente, l'alieno muore; se c'è una copertura, il colpo ne "mangia" un pezzo.

Feedback Qui si arriva alla parte che interessa di più. Quando si compie un'azione, il sistema deve rispondere al giocatore in un dato modo: deve dargli il **feedback**. Ricade qui **qualsiasi cosa comunichi l'effetto desiderato**, o comunque che l'azione è stata compiuta. Nel framework di Schell il feedback sta a metà tra meccaniche ed estetiche (sezione 2.7): lo sparo, il nemico che esplode, l'effetto sonoro (SFX) che fa *sentire* che il nemico è morto, un eventuale punteggio che sale. Sono tutti gli esiti, tutti gli effetti di quell'azione. Ciò che il giocatore riceve, poi, lo interpreta o meno.

Successo e fallimento A seconda della versione del diagramma, a questo punto c'è o non c'è una **biforcazione**. Per introdurre il feedback conviene non averla; ma è chiaro che se si compie l'azione desiderata, se si ha successo, succede qualcosa, e se si compie l'azione "sbagliata" succede qualcos'altro. Si apre così il **reame del successo** e il **reame del fallimento**.

Ricompensa Nel reame del successo entra la **ricompensa**: il premio per aver fatto ciò che il designer vorrebbe, quindi qualcosa di positivo. In una *boss battle* di *Zelda*: "hai battuto il boss, sei un grande, ti diamo l'arco". Se si sbaglia succede qualcos'altro: il game over.

[1] Nella trascrizione il termine compare storpiato ("l'Udema"). Koster parla di *ludemes* (le unità elementari del gioco) e di *game atoms*: il nome, come precisa il docente, è poco importante.

Modello mentale Infine, e qui sta il punto più importante, il feedback **aggiorna il modello mentale del giocatore**.

Il ciclo si può complicare a piacere. In *Dark Souls* ci sono i *mimic*, i forzieri che in realtà sono mostri: bisogna stare attenti alle casse. Se si è bravi e si riconosce che non è un mimic, la si apre e si prende la ricompensa; se si riconosce che è un mimic, la si colpisce, la si sconfigge e si prende comunque la ricompensa; se non la si riconosce, si viene mangiati. Reame del fallimento, reame del successo.

Table 4.1: L'atomo di gioco di Koster, come presentato in lezione.

	Elemento	**Dove sta**	**Esempio**
1	Problema	sistema	un boss da battere, una porta da aprire
2	Preparazione	giocatore	come affronto questo combattimento?
3	Core mechanic	giocatore	saltare (*Mario*), sparare (*Space Invaders*)
4	Spazio	sistema	la copertura che assorbe il colpo
5	Feedback	sistema	esplosione, SFX, punteggio che sale
6	Successo / fallimento	sistema	ricompensa (l'arco) o game over
7	Modello mentale	giocatore	"i barili rossi esplodono"

4.1.3 Il modello mentale del giocatore

Il modello mentale è la parte più importante del framework, anche perché il resto, osserva il docente, è in parte ripreso da un altro designer, di cui si vedrà brevemente un altro framework più avanti (e che si ritroverà, probabilmente, al terzo anno).[2] Questo elemento sta **specificamente nel reame del giocatore**: esiste solo nella sua testa.

Capire di aver fatto qualcosa di positivo o di negativo dipende, tra virgolette, solo da "quanto si è svegli"; o meglio, non necessariamente da quanto si è svegli, ma dal **bagaglio** che ci si porta dietro. A tutti è capitato: la prima volta che si prende in mano un controller si è neofiti del mondo ludico. Il docente ha cominciato, suo malgrado, con *Dino Crisis*, ed era ovviamente "massacrato ogni due per tre", tra l'altro da molto piccolo. Ma con il tempo quel bagaglio cresce e lo si porta con sé.

Le convenzioni codificate. Nel bagaglio rientrano le convenzioni: "il barile rosso esplode". Nella realtà non c'è alcun legame effettivo tra il colore e l'esplosione (non tutti i barili di benzina sono rossi, né tutti i barili esplosivi lo sono): è una scelta ormai **codificata nel mondo dei videogiochi**, come, banalmente, il forziere. Il giocatore ha imparato qualcosa del gioco che lo migliora come giocatore e gli permette di **approcciare di nuovo il sistema** con informazioni aggiuntive, si spera da giocatore migliore per quel tipo di gioco.

Il ciclo nella testa del giocatore. Se si compie un'azione positiva, il cervello non solo viene nutrito con i feedback giusti, ma *capisce* di aver fatto giusto: grande scarica di dopamina, e si va avanti. Se si sbaglia, stesso discorso: forse in quel burrone non ci si doveva buttare. È ciò che si fa spesso con il *danno da caduta*: esiste o non esiste? Scopriamolo: ci si tuffa da cinquecento metri e, se si muore, probabilmente non lo si rifà più. Lo stesso vale per il fuoco amico.

Ecco il legame con la lezione del direttore: la mentalità giocosa, il modo di approcciare un gioco, è legata al luogo in cui risiede lo stato di *play*, cioè **la mente del giocatore**. Lo si espanderà in futuro; per ora basta sapere che quel bagaglio viene aggiornato continuamente.

Che cosa si può assumere sul giocatore. A seconda del *target audience*, si può assumere che il giocatore arrivi nel gioco con delle competenze, oppure no; in ogni caso **qualche informazione bisogna dargliela**. Chi approccia *Dark Souls* da neofita avrà difficoltà a riconoscere una serie di cose, come i mimic, e potrebbe

[2] Il nome del designer non è ricostruibile con certezza dalla trascrizione. L'idea di un ciclo azione–feedback che aggiorna il modello del giocatore è al centro anche degli *skill atoms* di Daniel Cook, a cui l'atomo di Koster è spesso accostato.

non aspettarseli; ma anche un giocatore che non si definirebbe veterano, una volta mangiato, starà più attento.

4.1.4 A che cosa serve nel design

Una domanda possibile: come aiuta, nel design, tutto questo? In pratica, **poco**, se non nel ragionare in un certo reame: quello del problema e della preparazione, cioè *come presentare il problema* e che cosa il giocatore deve fare *prima* di affrontarlo. È un discorso complesso, che per ora resta lì ("un altro strumento misterioso che lasciamo da parte").

Il playtest. Ciò che di solito si fa, come accennato la volta precedente, è **far testare il proprio gioco ad altri**. Nella propria testa si ha un'idea di come si sono costruiti il livello, i sistemi e il resto; ma se si dà il livello in mano a una persona che non ne sa nulla, esperta o meno che sia, nel 90% dei casi si scontrerà con cose a cui non si era mai pensato. La messa in scena e la preparazione verranno trattate in futuro.

Il feedback non è mai abbastanza. Il feedback, invece, è **importantissimo** e ricade nel reame dell'iterazione visto la volta precedente (sezione 3.5). Non ci sono mai abbastanza feedback: si può sempre migliorare il modo in cui si premia o si punisce il giocatore quando ha fatto qualcosa di giusto o di sbagliato, anche con modalità che si appoggiano alla psicologia. Il feedback va **sempre** curato, spesso manca, e "non è mai abbastanza". Si fa il sistema, si osserva come reagisce il giocatore, si "pompa" il feedback o lo si rende più chiaro (un altro effetto visivo, un SFX migliore) e si ricomincia il giro: sempre il processo iterativo. Spesso, **se qualcosa non viene capito, il problema è lì**.

Approfondimento: la lente del feedback. Schell dedica al tema una lente specifica, la *Lens of Feedback* (lente n. 63 nella terza edizione di *The Art of Game Design*). Il feedback, scrive, è molte cose insieme: giudizio, ricompensa, istruzione, incoraggiamento e sfida. Per verificare che il ciclo di feedback crei l'esperienza desiderata, suggerisce di chiedersi *in ogni momento* del gioco: che cosa i giocatori *hanno bisogno* di sapere ora? Che cosa *vogliono* sapere? Che cosa si vuole che *provino*, e quale feedback può creare quella sensazione? Qual è il loro obiettivo in questo momento? Sono le stesse domande che, nel ciclo di Koster, collegano il feedback al modello mentale.

Che cosa portarsi a casa. Più che uno strumento operativo, l'atomo serve per la **nomenclatura** e per fissare il concetto di *play* come stato mentale. Ed è comunque utile avere una parola in più: la **core mechanic**, quasi sinonimo di verbo di gioco. L'atomo tornerà utile più avanti per spiegare perché alcuni design dominanti funzionano così bene (sezione 4.13).

4.2 Un ripasso della documentazione

Prima di passare oltre, il docente conclude ciò che era stato visto sulla documentazione la volta precedente, con un ripasso veloce "perché questa roba è importante".

- La documentazione è la **prima *duty*** del designer, il suo primo lavoro; a seconda dell'azienda può essere più o meno totalizzante (sezione 3.6). Serve a **trasmettere la visione**, anche in maniera specifica, con documenti adatti (lo si vedrà in futuro); a **fare ricerca** e a documentarla in maniera chiara; a **supportare il team**; a **controllare la produzione**. Sono i quattro reami principali.
- Ci sono le **golden rules** da applicare (sezione 3.7): "ricordatevele, stampatevele, tatuatevele, fate quello che volete. Ricordatevele, però."
- Ci sono i **tipi di documenti**. Dal prossimo task questo aspetto conterà; per il primo task il docente sarà più permissivo e leggerà tutti i documenti, a meno di errori davvero grossolani.
- Si usano **solo documenti della suite di Google** per qualsiasi task da consegnare, salvo indicazioni diverse, con le linee guida su quale strumento usare in quale momento: *the right tool for the right job* (sezione 3.8).

La Bibbia di design. Il discorso della *Bibbia di design* (detta anche *compendium*) era stato saltato velocemente; se ne riparlerà quando si tratteranno i tipi di documenti. "Purtroppo questa piaga è vera": esiste ancora oggi, per campi specifici. Di solito questi documenti sono **fuori dalla produzione operativa**: non servono a fare il gioco, ma a questioni, tra virgolette, *politiche*, come rispondere a stakeholder esterni. Non servono mai a fare effettivamente il gioco.

Documentare qualsiasi cosa. Con la documentazione si può comunicare qualsiasi cosa, anche concetti complessi: le *Blueprint* di Unreal hanno una documentazione che risponde a qualsiasi domanda su un dato nodo o su una data funzione. In teoria **tutti i dipartimenti dovrebbero documentare**; in pratica non sempre lo si fa, e a volte lo si fa quando la persona non c'è più. Per i programmatori, ad esempio, "la loro carne e le loro ossa sono la documentazione": a meno che un *engine programmer* non crei un tool che passerà a un altro programmatore che non incontrerà mai più in vita sua; in quel caso la documentazione la scrive, altrimenti "la vedo dura".

Gli strumenti non sono solo questi. Quelli indicati nella lezione precedente sono solo "briciole" (sezione 3.8): si può usare davvero qualsiasi strumento, anche per creare immagini (un generatore di meme, ad esempio, ammesso che esista ancora).

La dimostrazione pratica. Il docente assume che i task siano in corso applicando le regole e le golden rules viste. La dimostrazione promessa, in cui si prenderà un documento e lo si formatterà da capo, non si farà in questa settimana, perché la lezione di venerdì sarà da remoto e il docente preferisce farla "in carne e ossa": si farà **venerdì 6**, la prima lezione in presenza dopo il 30, compatibilmente con il tempo a disposizione. Nessuno ha altri dubbi sulla prima lezione di documentazione ("i dubbi sono sublimati"), e si può passare all'unica cosa rimasta da fare su quella lezione.

4.3 Il production dictionary

4.3.1 Un documento comune, con molti nomi

Nelle lezioni su documentazione e pitching si vedranno diversi tipi di documenti, ciascuno con il suo scopo. Il primo è il **production dictionary**, scelto anche perché è molto comune: probabilmente non è la prima volta che gli studenti vedono qualcosa di simile. È un **dizionario a tutti gli effetti** ed è **un documento di per sé**; a volte può essere ibridato con altri documenti, ma è davvero raro.

Ha soprattutto molti **corrispettivi**: documenti che sono la stessa cosa "in salsa diversa". I *narrative designer*, ad esempio (non è detto che lo si veda nelle lezioni di narrativa, viste le poche ore), lo chiamano **lexicon**: vi inseriscono le nomenclature, cioè le singole voci del dizionario, divise tra nomi comuni e termini che sono neologismi o idiomi. Il dettaglio interessa poco; il punto da sottolineare è che **alcuni di questi documenti hanno un corrispettivo in altri dipartimenti, o sono comuni a più dipartimenti**. Non è la prima volta che li si vede: probabilmente è la prima volta che li si trova applicati in una determinata maniera.

4.3.2 A che cosa serve

Come gran parte della documentazione, il production dictionary serve a **spiegare**: il suo obiettivo è **chiarire nomi specifici**, cioè la lista di informazioni e nomenclature che il lettore deve conoscere per comprendere il gioco.

Dove si mette. Il motivo per cui può essere ibridato è che, a volte, lo si trova *in testa* (o in coda) ad altri documenti: prima di leggere il documento, ecco la lista dei termini da sapere. Il docente precisa di riportarlo perché gli è stato detto, ma di non averlo mai visto in tutti i suoi anni di lavoro. Il problema è che **occupa spazio**. Oggi Google Docs ha le *schede* (*tab*), che permettono soluzioni molto più eleganti di una prima pagina che è "un mattone di testo" sulle informazioni da leggere. Non è piacevole, soprattutto

alla luce delle golden rules: il documento deve essere esteticamente appagante e deve far venire al lettore la voglia di leggerlo (sezione 3.7).

Il libro di cucina. Si immagini un libro di cucina che, invece di mostrare subito la grande torta della nonna con le more, con l'immagine e la ricetta, mostri prima un testo fitto che spiega che cos'è la spatola, che cos'è la teglia, che cos'è una forchetta, che cos'è un bicchiere. È chiaro che è meno accattivante per il lettore ("non sono un grande cuoco, questa è un'altra mia peculiarità"). Di solito, del resto, i termini davvero unici non sono nemmeno così tanti.

Così, nella stragrande maggioranza dei casi, il production dictionary è un **documento a sé**, che funziona come un dizionario e raccoglie tutti i termini necessari per comprendere il gioco.

4.3.3 Che cosa contiene

Generalmente contiene **termini unici**: le razze nuove, i tipi di qualcosa, e simili. Alcuni esempi dalle slide:

Risorse con nomi propri se nel gioco il mana non si chiama *mana* ma in un altro modo, va detto. Alla fine dei conti il mana funziona come una specifica *currency* che si spende per compiere azioni: "il mana è stamina blu". Ma serve una definizione, e se quella cosa non è mana, va detto che cos'è.

Termini ormai condivisi il *mulligan*, oggi condiviso, molti anni fa non lo era.[3]

Categorie del proprio sistema può essere utile definire che cos'è una *quest*, se ce ne sono di diversi tipi: se il sistema di quest distingue principali, secondarie, terziarie, o prevede delle *tier* specifiche, conviene definirle (che cosa significa "primaria", con un esempio, e così via).

Stati dell'IA nelle slide ci sono anche due definizioni di stati dell'intelligenza artificiale (IA videoludica, non generativa: se ne parlerà nella lezione apposita).

Gli stati dell'IA, un caso esemplare. A prescindere dal "cervello" usato per l'IA (quasi sempre si parlerà di *behavior tree* o di *state tree*, lo strumento più recente di Unreal Engine 5), cioè da quel framework applicativo che è lo standard dell'industria, **i nomi degli stati in cui si trova l'IA cambiano da azienda ad azienda**. Lo stato *idle*, o di calma, o qualsiasi nome gli si voglia dare, spesso viene rinominato. In IO Interactive, ad esempio, uno degli stati dei nemici si chiama *caccia*; altrove lo si chiama *chase*, altrove ancora in un altro modo. L'importante è **far capire a tutti di che cosa si sta parlando**: è, alla fine dei conti, l'obiettivo della documentazione.

4.3.4 Esercizio in aula: una carta di Hearthstone

Il docente mette alla prova la classe con una carta di *Hearthstone*, un *Golem cristallino* (che non ha mai giocato, perché è uscito "da questo turbinio" prima che venisse pubblicata): se si dovesse fare il production dictionary guardando solo questa carta, che cosa bisognerebbe definire?

- **I numeri sulla carta**: costo in mana, attacco, vita. Sono termini più o meno condivisi; spiegarli è a discrezione del designer o del *lead*, a seconda di **quanto sono condivisi**. Attacco, vita e mana sono generalmente comuni, il che non significa che non sia utile spiegarli. Ma conta anche quanto diventa "mastodontico" il dizionario e quanto tempo di produzione si ha a disposizione: come al solito ci sono delle **priorità**.
- **Le parole chiave**, che a volte non sono parole ma **simboli**.
- **Il tipo della carta**: *Elementale* è una tipologia (una sorta di tipo Elettro dei Pokémon).
- **Le keyword dei giochi di carte**, una applicazione specifica, vanno spiegate: *Provocazione* (*Taunt*: va attaccato per forza prima di poter colpire l'eroe avversario), *Rinascita* (*Reborn*: quando muore ritorna in vita), *Rantolo di morte* (*Deathrattle*: quando muore produce un certo effetto). Sono parole specifiche di *Hearthstone*, non usate altrove.

[3] Nei giochi di carte, il *mulligan* è la possibilità di rimescolare o sostituire, del tutto o in parte, la mano iniziale.

- **La rarità**: si può andare sempre più in profondità e definire anche la rarità. La carta è comune; ci sono i soliti tier (comune, rara, epica, leggendaria). *Leggendaria* è già più utile da spiegare in *Hearthstone*, perché porta con sé un'informazione che il giocatore deve avere: una leggendaria è limitata a **una copia per mazzo**, mentre delle altre se ne possono avere più copie.[4]

La regola: avere priorità. Bisogna sempre avere una **priorità sui termini da spiegare**: quelli più fraintendibili o necessari per la comprensione vanno spiegati per primi. In questo caso, **tutte le keyword**.

Con questo la lezione precedente è conclusa, e il docente assegna il task per la settimana (sezione 4.14).

4.4 Due principi sulla documentazione

Una breve parentesi: il docente chiede se gli studenti abbiano recuperato o rivisto delle lezioni sull'*hub* del corso, e se i paragrafi delle trascrizioni siano sensati ("non so come la macchina legga queste cose"). Poi, "prima che me lo dimentichi di nuovo, con la mia memoria da pesce rosso", aggiunge due cose importanti sulla documentazione.

4.4.1 La documentazione non sostituisce il gioco

La documentazione è importante e fondamentale ma, come detto nella lezione precedente, **non sostituisce il gioco**. Deve aiutare a creare e facilitare il lavoro: soprattutto nella pre-produzione, poi nell'implementazione, nella comunicazione e così via.

Non tutti i documenti sono fondamentali. Il production dictionary *potrebbe* essere fatto, ma non è necessario in ogni produzione: come sempre **dipende dalle risorse** e dal tempo a disposizione. In una game jam, con dieci giorni di sviluppo o lavorando in due, non serve; e vale per il primo documento visto come per molti altri.

I documenti come framework. Molti di questi documenti, tra l'altro, possono essere usati **come framework**: se ne prende un pezzo e lo si unisce a un'altra parte. La somma di queste cose è il "principe" della documentazione, il **Game Design Document** (GDD), che il docente spera di vedere il più avanti possibile ("ma per voi, non per me"). Basta mettere *DD* alla fine (GDD, LDD...) e quello è *il* documento. Dentro ci finiscono molte cose: il versionamento, potenzialmente il production dictionary, "un macello di cose", a seconda di chi lo scrive e di dove lo si scrive, cioè in quale azienda. Come al solito, le aziende abilitano o non abilitano certe cose. Come per i framework, bisogna capire **perché** questi documenti si fanno e perché hanno quel contenuto: a volte è un dogma, più spesso no.

4.4.2 Non esistono i template

La seconda cosa, importantissima, è anche il motivo per cui in questa prima lezione non c'è un esempio di production dictionary (mentre in futuro ci saranno esempi di documenti specifici): **in questo lavoro non esistono i template**.

Vale soprattutto per i GDD. Cercando corsi, approfondimenti o video, si trovano persone che condividono il proprio template di Game Design Document. Va bene, può essere un'informazione utile, e il docente stesso ha trovato utile vederne alcuni. Ma il consiglio è: **non applicateli mai uno a uno**. Anche quando si sarà trovato il proprio metodo di documentazione, e si faranno GDD in maniera estremamente efficace, non bisogna **rifugiarsi dentro template sicuri**, perché non si avrà quasi mai la certezza di poterli applicare al 100%: un po' perché i tempi non saranno sempre gli stessi, un po' perché l'azienda potrebbe non permetterlo (il GDD di alcune aziende ha quei punti specifici, e quelli sono; e vale anche per altri documenti).

I template aiutano a velocizzare il lavoro, o ad avere in testa che cosa è importante scrivere in un GDD o in un altro documento. Ma bisogna avere l'**elasticità mentale**, e poi la pratica, per cambiarli al volo quando serve. Se un template viene fornito, vale, ma ha sempre una **durata temporale** o, se non temporale,

[4] In lezione il docente esita tra due e tre copie. Nelle regole standard di *Hearthstone* il limite è di due copie per le carte non leggendarie e di una per le leggendarie.

è **circoscritto all'azienda** in cui si lavora. Ciò non significa che sia inutile: se ne può prendere il meglio e ibridarlo con altro, e questa è una grande evoluzione. Ma bisogna ricordarselo.

Approfondimento: il mito del Game Design Document. Schell dedica a questo punto un paragrafo di *The Art of Game Design* ("The Myth of the Game Design Document"). Molti aspiranti designer, scrive, immaginano che basti trovare "il formato giusto" di GDD per diventare professionisti; e lo smentisce scrivendolo a caratteri cubitali: *the magic template does not exist*, il template magico non esiste, non è mai esistito e non esisterà mai. Riporta anche le parole del designer Jason VandenBerghe: i GDD sono "letteralmente superati nel momento in cui li si scrive", perché esprimono *teorie* su che cosa renderà il gioco buono, non piani o progetti esecutivi. Ogni gioco è diverso e ha esigenze diverse, di memoria e di comunicazione, che vanno scoperte di volta in volta: un documento cresce organicamente intorno al design, non viene da un modello.

4.5 Problema e soluzione: il design come compromesso

4.5.1 La preistoria della produzione

Si apre la nuova lezione, **problema e soluzione**, che parte dalla "preistoria della produzione": la **preproduzione**. Si tratta tutto ciò che dovrebbe essere fatto in quella fase secondo buon uso e costume. Non sempre viene fatto, come si è detto parlando del ciclo iterativo (sezione 3.3); ma se venisse fatto, i risultati sarebbero "sicuramente molto migliori di quanto non siano nel 50% dei casi".

4.5.2 Il miglior compromesso possibile

Come detto nella primissima lezione (capitolo 1), **il design è il miglior compromesso possibile**: fare buon game design richiede di essere bravi nel compromesso, con le risorse, con la dimensione del team, con il tempo e così via. Le slide lo riassumono in quattro coppie di fattori (tabella 4.2).

Team e know-how Quante persone si hanno a disposizione, qual è il proprio "potere di fuoco", e che cosa si sa fare. Un designer esperto di giochi horror verrà tendenzialmente assunto per fare quello. Nel mondo *mobile*, guardando le offerte di lavoro, la prima cosa richiesta è aver già lavorato nel mobile o nelle *live ops* in generale: non significa che chi non lo ha fatto sia automaticamente escluso "da questo magico mondo", ma si dà priorità a chi ha già quel know-how. La *creazione* del team non è necessariamente compito del designer; la sua *gestione* sì, è possibile.

Visione e scienza È la parte **strettamente legata al design**: riguarda **che cosa il gioco deve essere**. È il "fantomatico" termine di *visione*, già incontrato rispondendo a Lorenzo sui pilastri (sezione 3.5), di cui si parlerà meglio con il **documento di visione**: la cristallizzazione dell'idea. Il docente di solito parla di "messa a terra", ma l'idea non è ancora del tutto messa a terra: "un piede nella fossa e uno no". Senza parlare ancora di documenti, la visione è una **bussola** che indirizza la produzione: dove si deve andare, che cosa si deve raggiungere con il prodotto e perché lo si deve fare. E deve essere già stata **confutata** il più possibile tramite il metodo scientifico.

Mercato e target audience Il prodotto è indirizzato, creato e ragionato sempre in funzione del target audience; il target audience è immerso nel mercato, e anche il prodotto, una volta rilasciato, lo sarà per forza di cose. Ci si aspetta che l'obiettivo sia vendere, o almeno fare **break even**, cioè guadagnare almeno quanto si è speso. Le aspettative possono essere (si spera) molto più idilliache: un'azienda non deve solo andare in pari, deve poter "campare" altri anni. Per farlo bisogna inserire il prodotto in un **mercato che possa assorbirlo**, che abbia abbastanza spazio perché il gioco venga visto, giocato e, generalmente, comprato.

Produzione e soldi I soldi sono "la benzina" dell'industria videoludica, almeno ad alto livello ("brutto vederla così, ma è così"). Mantenere un individuo costa una certa cifra; i soldi permettono di avere un team, di pagare strumenti e ricerca, sondaggi, **focus group** che giochino il gioco per verificare che in un certo stato della produzione abbia senso, e aggiustare il tiro (sempre l'iterazione). La **produzione**

determina invece quanto è efficace la macchina: lavorare 24 ore al giorno o 8, tre giorni su sette o cinque, gestire i task in modo che tutti i dipartimenti lavorino contemporaneamente o che qualcuno resti fermo. Se i soldi sono la benzina, la produzione dice quanto l'ingranaggio è oliato. È il lavoro del *producer*, una specializzazione "tosta" con i suoi documenti specifici da adottare, cambiare e riapplicare, di cui non si parla ora.

Table 4.2: Le quattro coppie del compromesso di design.

Coppia	**Domanda di fondo**
Team e know-how	Quante persone abbiamo e che cosa sappiamo fare?
Visione e scienza	Che cosa deve essere il gioco, e lo abbiamo verificato con i dati?
Mercato e target audience	A chi lo diamo, e c'è spazio per lui nel mercato?
Produzione e soldi	Quanta benzina abbiamo, e quanto è oliata la macchina?

4.5.3 Cinquanta per cento scienza, cinquanta per cento arte

Il docente riprende una citazione del direttore, "molto succinta, molto *on point*" (sezione 1.3):

Il game design è per il 50% scienza e per il 50% arte.

La parte **scientifica**, quella di questa lezione (e non solo), è il processo di *smontare l'idea*: fare ricerca, trovare i dati, anzi partire con l'obiettivo di **avere dei limiti** e di navigare entro quei limiti. I limiti potenziano la creatività e permettono di evitare strade problematiche fin dalla partenza. Si vuole fare un MMO, ma si è in tre persone? Forse non conviene. Si vuole fare un *hero shooter* con gli *stickman*? Forse sì, forse no: bisogna trovare i dati, non lo si può dire così. Con il processo scientifico si ottengono dati, li si lima, si cerca di arrivare a una serie di possibilità, di strade percorribili.

Una volta posto questo filtro scientifico, però, **dove andare lo decide il designer**, la sensibilità del singolo: è il 50% di arte. Può emergere in momenti diversi della produzione, anche prima, nella scelta dell'idea; ma l'idea in sé vale poco, perché le idee possono averle tutti. L'obiettivo è filtrarle, trovare la migliore possibile, applicarla, trovare l'implementazione giusta. Trovare soluzioni creative fa parte del lavoro, ma non ne è il tutto: ne è il 50%.

4.6 Senza dati si è solo un'altra persona con un'opinione

"Senza dati, sei solo un'altra persona con un'opinione."
(W. Edwards Deming)

È ciò che serve nel 50% di scienza del lavoro. Non si può confutare il design di qualcun altro, e nemmeno iniziare a discuterne, **senza dati**. I dati possono essere quelli che il designer ha cercato nella parte di ricerca del documento, ma vale anche il discorso fatto sul target audience (sezione 3.5): si può discutere di qualsiasi cosa, ma la scelta migliore tra la mia e la tua non dipende da quanto è forte l'uno o l'altro; dipende da **chi avrà in mano il gioco**, da chi dovrebbe giocarlo.

Gioco "bello", gioco "brutto". È il discorso fatto con Federica (sezione 3.1): non serve dire che un gioco è bello o brutto (parole bandite, sezione 2.12); serve dire che tipo di estetiche ha. Sono dati *qualitativi*, ma sono dati, e sono verificabili. Lo stesso vale per "all'utenza piacerebbe" o "non piacerebbe": si fa ricerca, si chiede all'utenza, al potenziale gruppo di persone. Se bisogna fare un gioco che venda su Reddit, si va su Reddit a chiedere; se servono i più grandi fanatici dei *soulslike*, nel peggiore dei casi si fermano le persone per strada: "sei un gran fan dei soulslike?". Prima o poi si trova qualcuno, gli si fa una serie di domande, si raccolgono i dati, li si filtra. Servono per iniziare a parlare di questioni concrete, o per andare a discuterne.

Deming, economista statunitense, ha fatto molto altro: ha avuto grande successo in Giappone, in particolare sui *processi produttivi*, temi più legati alla sfera della produzione che a quella del design; magari se ne parlerà più avanti.

4.6.1 Quanti dati, e quando

Come si ottengono i dati? "Guarda un po"': tramite la **ricerca**, la prima parte di questa lezione. Si possono raccogliere dati **quantitativi** e **qualitativi**, in qualsiasi maniera: l'importante è che **siano dati**.

Chiedere a una o due persone del target audience se ha senso fare una cosa o un'altra non basta: due persone sono troppo poche per un bacino statistico, ne servono **almeno cento**. Non sempre si avrà il tempo di farlo; ma nella pre-produzione la parte di ricerca va fatta **attentamente**, proprio per questo. Si costruiscono fondamenta solide, si raccoglie una "marea" di dati che magari non servono tutti subito ma a cui si può tornare in futuro, e soprattutto, facendolo ora, non lo si deve fare nel vivo della produzione, quando il tempo scarseggerà davvero.

La ricerca si fa in diverse maniere e forme: alcune molto aziendali e condivise, altre meno. La lezione serve proprio a dare **più strumenti possibili** per la ricerca e, potenzialmente, per l'ideazione e il *brainstorming*. Non tutti saranno sempre applicabili: è una lista "più che estensiva", e va valutato di volta in volta che cosa usare.

4.6.2 Il design thinking in cinque passi

Per rendere concreto il discorso, il docente mostra un breve video sul **design thinking**, che descrive il processo come un viaggio senza destinazione certa, "ma con la fiducia che si arriverà in un posto fantastico". L'esempio: un team riceve la sfida di far muovere di più i bambini americani, per contrastare l'obesità infantile. L'idea di partenza è dare ai bambini un lettore musicale digitale con un sensore di movimento, premiandoli in base all'attività. Ma la grande domanda è: i bambini lo userebbero davvero, e abbastanza a lungo da vederne i benefici?[5]

1. **Imparare dalle persone** (*learn from people*). Il team parla con i bambini, passa del tempo nelle loro case e nelle loro scuole, ascolta i loro limiti, le abitudini, ciò che li entusiasma e ciò che li frustra. La ricerca include bambini "nella media", ma anche bambini molto attivi e molto sedentari: chi sta agli **estremi** sa dare voce a problemi che chi sta nel mezzo percepisce, ma fatica a mettere a fuoco.

2. **Trovare degli schemi** (*find patterns*). Le osservazioni finiscono su centinaia di post-it, disposti per dare un senso a ciò che si è imparato; con un'intuizione informata si cercano gli schemi che indicano delle opportunità. Daniel dice: "mi annoio con i videogiochi in solitaria, sono quelli multigiocatore che mi fanno tornare". Meg dice: "non porto l'iPod quando corro perché voglio parlare con le persone".

3. **Definire i principi di design** (*define design principles*). Frasi come queste rivelano uno dei principi che porteranno al concept: *facilitare l'interazione sociale in ogni momento*. Ne emergono molti, e insieme formano i "pali della porta" di un'esperienza che il team è fiducioso possa funzionare con i bambini.

4. **Rendere tangibile** (*make tangible*). Domande del tipo "come potremmo...?" colmano la distanza tra principi e idee specifiche; le migliori vengono trasformate rapidamente in prototipi grezzi.

5. **Iterare senza sosta** (*iterate relentlessly*). Dispositivi fisici di cartone ed esperienze digitali simulate con carta e matita permettono di imparare in fretta; a ogni prototipo il concept si evolve, viene riportato ai bambini, si ascolta e si impara.

Il risultato non è l'idea iniziale del lettore musicale, ma un piccolo **monitor di attività** da agganciare ai vestiti o da mettere in tasca: un portale verso un mondo online in cui i ragazzi condividono e celebrano con gli altri e con le famiglie i risultati raggiunti nel mondo reale. In uno studio clinico, il risultato è stato un **aumento del 59% dell'attività fisica**.

"Grande successo: l'America è salvata dall'obesità", commenta il docente. L'obiettivo era far alzare i bambini dalla sedia e "fargli toccare l'erba". È un esempio molto contenuto di cinque punti che possono essere espansi, e che sono contenuti nella prima parte della lezione: sono **il cuore** di ciò che si vedrà, e non vanno dimenticati.

[5] Il video è quello con cui lo studio di design IDEO illustra il proprio approccio al design thinking; il progetto descritto ha portato a un dispositivo di monitoraggio dell'attività per ragazzi, collegato a una piattaforma online.

4.6.3 Partire da un problema o da una necessità

Il primo passo è partire da un **problema** o da una **necessità** (la distinzione verrà chiarita nella sezione 4.12): può essere semplicemente qual è l'obiettivo del progetto, del team. A volte su questo obiettivo si costruisce un'identità intera, cosa più riconoscibile tra i publisher, ma visibile anche tra gli studi.

Esempio: Milestone. Quale potrebbe essere la missione di Milestone? Per dirlo bisogna sapere che cosa fa: *MotoGP*, i giochi su Valentino Rossi e simili. L'obiettivo è fare **simulazioni di guida**, giochi a due e quattro ruote simulativi. Magari lo studio non era nato con quell'obiettivo, ma nel tempo si è solidificato: hanno già chiaro che cosa devono fare, quale problema risolvere e in quale mercato affacciarsi. Per un'altra azienda può non valere: si potrebbe volere "un sacco di soldi", e allora l'obiettivo è trovare un'applicazione, un progetto che dia quel risultato. Come? Facendo ricerca: chiedendo alle persone se c'è un problema o una necessità.

4.6.4 Esercizio in aula: che cosa non va nei soulslike?

Per mostrare come si raccolgono i dati, il docente intervista i giocatori di soulslike presenti in aula: che cosa non piace dei soulslike? Le risposte:

- una studentessa, gran fan del genere, fatica a trovare difetti; l'unica cosa che le viene in mente riguarda la **console** su cui gioca, non il gioco;
- Pedro cita le **risorse limitate**: in alcuni titoli, finiti gli oggetti curativi, bisognava tornare a **farmarli** prima di riprovare (il *grinding*);
- Antonino cita la **curva di difficoltà** tra l'inizio e il resto del gioco: l'inizio è difficilissimo;
- un altro studente porta l'esempio del DLC di *Elden Ring*: la difficoltà principale dei boss, secondo lui, non nasce dall'averli resi più dinamici, ma dall'aver alzato i numeri (vita, velocità, attacco, danno), mentre nei capitoli precedenti il combattimento era un'alternanza bilanciata tra schivata e attacco, sia del giocatore sia del boss.

Il docente avverte che il discorso è complesso e lo si sta semplificando. Se l'obiettivo è fare un soulslike, bisogna capire come ci si è arrivati: un buco di mercato, giocatori che chiedono qualcosa a gran voce, mille altre ragioni ("magari siamo FromSoftware e facciamo quello"). E bisogna chiedere al proprio target audience che cosa vuole.

La curva a campana. Va anche capito se gli intervistati sono **esattamente il tipo di giocatore che si vuole**, e dove si collocano sulla curva. La distribuzione normale vista nel video è la condizione naturale delle cose: se si chiede a tutta l'Italia "ti piace la pizza?", nel mezzo della curva a campana ci sono quelli che la mangiano una volta a settimana, e agli estremi chi non la mangia mai e chi la mangia ogni singolo giorno. Bisogna capire dove si trovano gli intervistati: grandissimi fan di *Dark Souls*, con "un santino di Miyazaki a casa", oppure no. Al docente, ad esempio, dei soulslike non piace per nulla la narrativa, "estremamente criptica, convoluta, complicata"; ma non sono quelli i problemi emersi in aula.

Registrare tutto. Dalle risposte emergono già due questioni, **complessità/difficoltà** e **grinding**. Queste interviste servono a prendere dati: bisogna segnare tutto e, potenzialmente, **filmare** (la lezione stessa è registrata). Bisogna osservare le reazioni delle persone: se fanno fatica a trovare le risposte, se si animano parlando di qualcosa. Non conta solo ciò che dicono, ma *come* lo mostrano e che cosa potrebbero pensare. Poi si setacciano i dati per vedere quali tornano utili.

Trovare archetipi e categorie. Il modo che conviene più di tutti è **trovare archetipi**, categorie in cui far ricadere le risposte. È anche per questo che è stata data la lista di emozioni (sezione 2.12), e il concetto verrà esplorato e rifinito. Tutto ciò di cui si è parlato in aula (la progressione, il tipo e la curva di progressione, la sfida, quanto alta o bassa, il grinding) ricade nello stesso reame: la **challenge**. Uno studente osserva che in *Elden Ring* il problema degli oggetti curativi è stato risolto (le fiasche si ricaricano

ogni volta che si muore), segno che probabilmente un processo del genere c'è stato; e lo stesso si può dire della telecamera e del *level design* dei primi *Dark Souls*. Se invece qualcuno avesse detto "in *Dark Souls* mi piace un sacco parlare con il Cavaliere Cipolla", o "vorrei un compagno, vorrei un cane", ci starebbe ("chi sono io per dire di no"), ma sarebbe rimasto fuori da questa potenziale "molla" della challenge, che include sfida, grinding e progressione.

4.6.5 Correlare, chiudere la visione, prototipare

Una volta ottenuta questa "enorme bacinona di dati", bisogna cercare la **correlazione**: trovare gruppi, cose che collidono, capire *perché* succedono certe cose. È più evidente, ad esempio, quando un mercato è molto saturo e le persone non vogliono più giocare a un certo genere: la questione del recente filone dei *friendslop*, per esempio.[6] Oppure il motivo per cui una parte dei giocatori dedica ai videogiochi meno tempo che, poniamo, nel 2022: il Covid ha avuto la sua influenza. Bisogna trovare questi legami per poter rispondere sensatamente alle domande che ci si pone dentro il team, e anche fuori.

Chiudere la visione. Verso la metà o la parte finale della ricerca bisogna, se non lo si è già fatto, **iniziare e poi chiudere la visione**. Ciò che si deve fare con il progetto viene **blindato**: questi sono i punti, queste le cose incluse, che per forza di cose ci devono essere. Altre, per forza di cose, non ci saranno. Se si fa un soulslike, i giocatori si aspettano un certo tipo di difficoltà e certi sistemi: se si muore si perdono le risorse e bisogna andarle a riprendere, è lo standard. A volte è un'aspettativa anche la narrativa, la cosiddetta narrativa *apofenica* dei soulslike (motivo per cui il docente "se la sorbisce" anche in *Hollow Knight*). La visione è ciò che deve condividere tutto il team: tutti devono essere *onboarded*, devono sapere dove si va e qual è la direzione del progetto.

Prototipare. Poi il **prototyping**: prima lo si fa, meglio è (lo si è già visto, sezione 3.5.6). Si raggiungono il prima possibile versioni concrete, fisiche o digitali, per testare ciò che si sta facendo; le si gioca, si raccolgono feedback, si modifica, si itera. In pre-produzione lo si fa in maniera ridotta rispetto alla produzione, ma già si mettono le mani nell'engine: è sicuramente ancora pre-produzione.

4.7 La mappa della ricerca in pre-produzione

Il docente presenta i punti che verranno toccati nella prima parte della lezione. Come sempre, la lista **non è esclusiva né limitata** a queste voci: sono modi di fare ricerca, prendere dati specifici e potenzialmente correlarli.

Client pitch direction analysis Capire se si ha **qualcuno sopra**, se c'è una situazione "clientelare": i limiti non sono solo quelli del target audience, ma anche quelli di chi fornisce i soldi. Se si fa un progetto per qualcun altro, bisogna capire che cosa vuole quella persona; e, come sempre, **ciò che le persone dicono non è sempre ciò che vogliono**: va tradotto, contestualizzato, analizzato. È legata alla definizione di stakeholder (sezioni 4.10 e 4.11).

Stakeholder e il loro peso Non tutti quelli che stanno sopra pesano allo stesso modo. Un investitore che ha messo un milione di dollari nel gioco ma ha il grosso dei suoi interessi altrove ("nel campo del petrolio") si interesserà relativamente del progetto: qualsiasi cosa si faccia, bene o male andrà in porto ("la semplifico molto, ma molto"). Uno stakeholder che si aggiorna spesso, impone deadline specifiche ed è sempre "sulla spalla" a controllare ha un peso diverso. C'è chi non dà soldi e non influisce economicamente, e chi dà soldi senza avere competenze definite in ambito videoludico. Bisogna capire chi c'è sopra, che cosa vuole e quale peso ha nella produzione.

Problema contro necessità Capire se c'è un problema o una necessità da risolvere nel mercato, e non solo (sezione 4.12).

[6] Termine gergale per l'ondata di giochi cooperativi economici, pensati per essere giocati con gli amici in streaming. Nella trascrizione compare come "friend slot".

Analisi dei competitor Fondamentale: "non si sfugge". Là fuori ci sono giochi che hanno qualcosa in comune con il proprio, anche se il proprio non è ancora uscito. Quali sono i propri competitor, **diretti** o **indiretti**? (Sezioni 4.8 e 4.13.)

Analisi degli utenti Chi dovrebbe giocare il gioco (sezione 4.9).

Analisi di mercato Dove ci si affaccia, e con quali regole.

Analisi di gameplay Correlare i dati e mettersi la maschera del target audience, fino alla **visione**.

4.8 Competitor diretti e indiretti

4.8.1 Il competitor diretto

Per chiarire la distinzione, il docente chiede un genere alla classe: i soulslike. Mostra *Enotria: The Last Song* e chiede quale potrà mai essere il competitor diretto di un soulslike ("perché avete paura di parlare, ragazzi?"). La risposta è ovvia: **qualsiasi cosa faccia FromSoftware**. Sono competitor diretti perché condividono la stragrande maggioranza degli aspetti del gioco: il sistema di progressione, la difficoltà, potenzialmente la telecamera.

Come osserva Federica, però, a seconda di quanto è innovativo un gioco **i competitor diretti non sono mai uno solo**: bisogna individuare quelli più vicini. Lo scopo del progetto e la sua grandezza sono molto minori rispetto a una produzione FromSoftware, quindi il competitor più diretto sarà qualcosa di più indipendente (un "doppia A", mettiamola così); ma il gioco è immerso nel mercato dei soulslike, e per forza di cose *Dark Souls* sarà uno dei suoi competitor diretti. Artisticamente, però, *Enotria* è diverso: gli sviluppatori lo definiscono un ***Summer Souls***, un soulslike "estivo". Anche questa è una scelta di vendita, come si era detto dei *roguelite* nella prima lezione.

4.8.2 Il competitor indiretto

Si può trovare anche un **competitor indiretto**: un gioco che ha in comune con il proprio *solo* un aspetto, artistico o meccanico, una specifica meccanica, un sistema o una direzione artistica. Qualcosa di **estremamente limitato e circoscritto** che ricorda un altro gioco, con cui però non si compete direttamente nello stesso mercato.

Pokémon e Palworld. *Palworld*, o qualsiasi titolo simile, è un competitor *diretto* di *Pokémon*, perché va a intaccarne il monopolio. Ha un'ampia distanza su una serie di cose ("lo schiavismo dei Pokémon, le armi, le mitragliatrici"), ma lo stile artistico è molto simile a quello dei *companion* e dei *monster collectible*, il genere su cui Pokémon ha una presenza estrema, un vero monopolio di mercato. *Clair Obscur: Expedition 33*, invece, non lo è.

Un caso italiano. Il docente cita anche un *monster collector* di un team italiano, che riprende da Pokémon e da altri titoli il sistema di combattimento con la *timeline* e ha una direzione artistica che sembra animata da uno studio d'animazione giapponese.[7] Non possono competere con Pokémon: hanno un costo in risorse diverso (e bisognerebbe capire quanto investono Nintendo e Game Freak) e uno scopo diverso, perché il loro è, teoricamente, un *roguelike*. Pokémon resta un competitor diretto, ma hanno **deviato il tiro** in un'altra direzione, per intercettare un certo tipo di persone ed evitare certi problemi, o rispondere a certe necessità, previste dalla ricerca. Anche questo è il punto della ricerca: identificare i competitor diretti o simili aiuta a capire che cosa gli altri hanno fatto bene o male, se si può fare meglio o peggio ("peggio si evita, chiaramente") e soprattutto **come ci si può differenziare**. E poi i competitor vanno **analizzati**: non restano lì nel nulla.

[7] I titoli e gli studi citati in questo passaggio non sono ricostruibili con certezza dalla trascrizione; si riporta il ragionamento, non i nomi.

4.8.3 Gli esempi della classe

La classe propone diversi esempi, e la discussione è vivace.

Pokémon TCG Pocket e Yu-Gi-Oh! Un gioco mobile, quindi bisogna capire il panorama mobile e prendere i TCG mobile con partite di durata ridotta. Indiretti potrebbero essere *Hearthstone* o *Gwent*; diretti, probabilmente, *Yu-Gi-Oh! Duel Links* (e, in misura minore, *Master Duel*), forse *Marvel Snap*.

Expedition 33 e la serie Mario & Luigi Uno studente li avvicina per il controllo che il giocatore ha durante il turno nemico (il *parry*, le azioni a tempo): non è un gioco di ruolo a turni "puro". Il docente concorda: è **indiretto**. Non lo fa solo lui, ma c'è quella specifica cosa in comune. Allo stesso modo il sistema di morte di *Dark Souls* viene ripreso in titoli che non sono soulslike. Pokémon ed *Expedition 33* hanno meccaniche simili, ma giocatori "completamente opposti".

Sekiro e Final Fantasy VII Remake Un altro studente si chiede se siano indiretti per il combattimento reattivo, non strategico. Non sono paragoni sbagliati: il punto è che **è uno spettro**, e quanto ci si avvicina o allontana dipende.

Resident Evil 2 e Silent Hill 2 Probabilmente diretti, forse più in passato che oggi: *Resident Evil* si è spostato molto più verso l'action che verso il survival, mentre *Silent Hill*, pur non avendo mantenuto del tutto la visione iniziale, resta molto meno action.

Indiretto non significa "stesso target, mezzi diversi". Uno studente chiede se indiretto significhi che il target è lo stesso ma con mezzi diversi. **Non necessariamente**. C'è un elemento, o più di uno, in comune con il proprio prodotto, ma è estremamente limitato; a volte è proprio un mercato diverso. Chi condivide lo stesso sistema ha molto più in comune di un *Expedition 33*; e due giochi con arte completamente diversa ma meccaniche simili, o con la stessa arte ma meccaniche diverse, sono casi ancora diversi. "Sono tutti esempi giusti", conclude il docente: sono su uno spettro, su scale diverse, ed era lui a sbagliare insistendo a cercarne uno ancora più vicino.

Un caso di confine: Nine Sols e Hollow Knight. Questi due giochi stanno davvero **sulla linea di demarcazione** tra diretto e indiretto. Possono essere percepiti come molto simili (e andrebbe verificato se i giocatori di *Hollow Knight* sono andati a comprare *Nine Sols* per la somiglianza); ma a livello di gameplay e di combattimento sono molto lontani: *Nine Sols* è molto più basato sul *parry* (il docente ci fa "una fatica tremenda"), *Hollow Knight* è più veloce. Sono lontani anche nella trama: in *Nine Sols* la parte narrativa, con il sistema di dialoghi e i personaggi che parlano, è molto più spinta ed esplicita. Senza dati, il docente li considera indiretti, "ma proprio di un pelo".

4.8.4 Come individuarli: i punti insindacabili

Come sempre è uno spettro, e lo si verifica **con i dati**: si fa ricerca, si prende una lista di più giochi possibile. Un modo giusto per arrivarci è partire da ciò che per il proprio gioco è **insindacabilmente necessario**: le cose ad alto livello che il gioco deve avere, ad esempio le *game qualities* di Romero e Schreiber (sezione 2.8). Magari sono già state messe sul tavolo dagli stakeholder: se si deve fare un monster collectible si è già molto dentro certi limiti. Se invece si deve fare un generico gioco da tavolo, bisogna capire l'obiettivo: se è vendere tanto, forse conviene guardare ai *party game*, ai giochi popolari che hanno venduto oltre una certa soglia, ai più conosciuti, o a quelli entro una certa fascia di prezzo.

Trovati i punti insindacabili, si limita su di essi la ricerca dei competitor:

- i giochi che ne hanno **di più** in comune, e sono più vicini, sono i **competitor diretti**;
- quelli che ne hanno magari **uno solo**, in maniera estremamente attinente, sono gli **indiretti**.

Non è un discorso facile di base, e serve tanta ricerca per far emergere queste cose; lo si approfondirà.

4.8.5 L'opposizione

Si possono identificare anche i giochi in **opposizione**: titoli molto lontani dal proprio, in netta contrapposizione. A che cosa servono? Non sempre a qualcosa: può non valere per prodotti che puntano a un mercato specifico. Ma possono servire a **fare innovazione**, ad esempio unire una direzione artistica unica a un gameplay mai fatto prima, o a qualcosa che nessuno ha fatto (o, molto più probabilmente, che nessuno ha fatto con successo). Unire titoli o meccaniche davvero lontani tra loro può **stimolare la creatività**, anche nel brainstorming.

Il docente prova a pensare a un esempio: un action con un'arte molto poco leggibile, come *Superhot* ("il tempo si muove solo se ti muovi"), che ha un'arte simile a quella di *The Unfinished Swan*. Non essendo un artista, nota però alcune differenze: *The Unfinished Swan* è più morbido nelle linee, più circolare, *Superhot* è molto più spigoloso; ma nella palette, bianco e nero, "siamo lì".

That's Not My Neighbor e Papers, Please. Uno studente chiede se *That's Not My Neighbor* e *Papers, Please* siano indiretti. Sì, per il docente è un buon esempio. Il primo è un gioco abbastanza semplice in cui si controllano le persone che bussano alla porta prima di farle entrare, con una deriva horror; *Papers, Please* è radicato in una realtà fittizia, con la sua dinamica burocratica. Lo studente osserva che sono giochi di una nicchia strana, in cui è difficile trovare un competitor diretto. È un'altra domanda da porsi: perché hanno meccaniche simili? Perché è una nicchia di mercato estremamente stretta, e chi gioca quel tipo di giochi può richiedere necessità specifiche. Probabilmente *That's Not My Neighbor* è andato in un'altra direzione proprio per non scontrarsi direttamente.

I competitor nel pitch. Soprattutto nel *pitching*, i competitor rendono molto più facile comunicare, anche la commistione di elementi che non sono meccaniche: "il nostro gioco è come *Limbo*, ma l'arte è completamente diversa". Si chiama in causa un gioco che magari non c'entra nulla, ma ha un punto artistico o sonoro vicino al proprio. Come analizzare i punti tra competitor diretti e indiretti lo si vedrà; e quasi certamente lo tratterà anche Daniele nelle lezioni di narrativa, perché il marketing è la sua grande specialità.

Analizzare i propri giochi. Valerio chiede se, potendo disporre di una vasta varietà di giochi propri (ad esempio, per Pokémon, i titoli principali e quelli secondari), li si possa usare nell'analisi. Certamente: si parte avvantaggiati, un po' perché la documentazione c'è già, un po' perché, nel caso di Game Freak, si ha il monopolio del mercato e si fa quella cosa da più di vent'anni. Proprio per questo diventa **ancora più importante chiedere al target audience che cosa vuole**; e quello di Pokémon non è uno solo, è molto frammentato. Ciò che il docente chiede a Pokémon non è ciò che chiedono i ragazzini che lo giocano per la prima volta, e il rapporto è probabilmente di uno a dieci. È il motivo per cui *Scarlatto e Violetto* e *Spada e Scudo* hanno una difficoltà e una struttura così lineari ed esplicite: la frangia di veterani è in numero molto minore rispetto a chi deve comprare il gioco. Si potrebbe anche discutere del fatto che i giochi non siano la principale fonte di guadagno di The Pokémon Company, che vuole vendere soprattutto le carte; ma per vendere peluche e carte serve una nuova generazione di Pokémon.

4.9 Analisi degli utenti, del mercato e del gameplay

4.9.1 Analisi degli utenti

Come si cercano i giochi simili al proprio, si cerca di capire **chi dovrebbe giocare** il proprio gioco. È la "cartina tornasole" di tutte le scelte di produzione: ciò che si fa è legato a questi dati. Se sono sbagliati, o estremamente vaghi, diventa difficile ragionare sulla soluzione migliore, perché non si è stretta abbastanza la "falce" sulle possibilità: si potrebbe fare qualsiasi cosa. Bisogna quindi capire:

- la **demografica** degli utenti;
- che cosa fanno, che cosa giocano;
- che cosa piace loro e che cosa non piace;

- che cosa dicono dei titoli già rilasciati e, in futuro, del proprio.

Ci si può già chiedere se un'idea funziona: si va dal proprio target audience ideale e si chiede "ti piacerebbe se facessimo *Dark Souls* con i pony di *My Little Pony*?". Se l'80% delle persone risponde "incredibile, non vedevo l'ora", si sta andando nella direzione giusta; se risponde "non so...", allora no.

Il vantaggio dell'opposizione. È anche il vantaggio di cercare i giochi in opposizione: unire generi lontani anni luce. Il docente giocherebbe volentieri *My Little Pony* in versione *Dark Souls*, e fa notare che sono giochi che esistono. *Dark Souls* è *grimdark*, è *Berserk*, è l'atmosfera cupa di un mondo completamente distrutto e alla deriva; se se ne prendono le meccaniche e lo si porta in fondo al mare, in un mondo da cartone animato, si ottiene *Another Crab's Treasure*, un soulslike con un granchio come protagonista. E, sempre per il target audience, ha una *easy mode* in cui il granchio ha una pistola e spara a tutti i boss: un pubblico chiaramente molto lontano da quello di FromSoftware, o meglio, non solo quello. *Palworld* funziona esattamente così: l'innocenza dei Pokémon unita allo schiavismo delle multinazionali e alle armi.

4.9.2 Analisi di mercato

Dove ci si affaccia? In quale mercato? È **saturo** o no?

- un **blue ocean** è uno spazio in cui c'è qualcosa che nessuno ha mai fatto (chiedendosi perché);
- un **red ocean** è il contrario: un mercato estremamente saturo di prodotti, in cui è estremamente difficile emergere. Lo sono, ad esempio, i *battle royale*, gli *hero shooter*, i *friendslop*, e ce ne sono altri.

Approfondimento: oceani e barriere. Schell riprende la stessa metafora in *The Art of Game Design*: i mercati inondati di offerta sono *red ocean*, un mondo "pesce mangia pesce" con molto sangue nell'acqua, in cui la sfida non è più arrivare nello store ma emergere tra migliaia di concorrenti. Per sopravvivere o si è il pesce più grande, o si trova un *blue ocean*, un posto con pochi concorrenti: a volte perché il mercato è nuovo, a volte perché ci sono **barriere** che tengono fuori gli altri (tecniche, hardware, di competenza, di vendita e distribuzione). Il suo esempio: in un mercato saturo come quello delle app iOS, all'epoca, circa il 90% dei giochi perdeva soldi; nel mercato delle slot machine, protetto da regolamenti, hardware e competenze specifiche, circa il 90% era in attivo.

Le regole del mercato. A seconda del mercato a cui ci si affaccia, alcune cose si possono fare e altre no, perché ci sono regole.

- **Cina**: se l'obiettivo è rilasciare in Cina, bisogna premunirsi. O si fa un gioco senza teschi, senza ossa e senza iconografia legata alla morte o al cristianesimo (come le tombe con la croce), oppure si dovrà rifare il lavoro dopo. È successo, ad esempio, a *Rainbow Six Siege*, che per il rilascio in Cina ha dovuto cambiare numerosi asset.
- **Germania**: non si può avere iconografia nazista; la svastica non può comparire in nessuna forma.
- **Belgio**: lo stesso vale per le *loot box*.

O ci si pensa prima, decidendo di non rilasciare in quel mercato, o si produce in modo che la produzione abbia quel margine. Se nel gioco ci sono ossa o tombe con la croce, e quell'asset si può rendere generico (versione X, versione Y, cambia solo una *decal* o un nome), adattarlo è semplice; se invece si è costruito un livello con mille asset raggruppati, diventa difficile. Poi ci sono casi come *Dead Space*: i nemici sono *necromorfi*, bestie mutate che si disintegrano con le ossa che escono, grondanti di sangue. *Dead Space* non si può fare senza necromorfi, e quindi non si vede in Cina: ma ci si preclude una grossa fetta di mercato. Questo ragionamento **va fatto prima**.

4.9.3 Analisi di gameplay e visione

L'analisi di gameplay, generica, si vedrà alla fine di tutto: si **correlano i dati** per raggiungere una certa consapevolezza. È come **mettersi la maschera del target audience**. Oltre ad avere dati su cui tornare e su cui discutere, l'obiettivo è mettersi nei panni di chi giocherà il gioco, capirne le necessità, i desideri, ciò che gli piacerebbe fare.

È più facile se si fa un gioco per cui si è già naturalmente predisposti come giocatori, ma va fatto comunque. Ed è più difficile, ed è ciò che fa diventare professionisti, mettersi nei panni di qualcuno molto lontano da ciò che si gioca. Anche il primo caso ha le sue insidie: il docente, grande amante dei TCG, quando ci ha lavorato ha sbagliato, mettendo **il proprio ego dentro il design**: voleva più complessità, perché è ciò che si aspetterebbe lui da giocatore, ma non necessariamente ciò che si aspetta il target audience di quel gioco.

La concretizzazione di tutto questo è la **visione di gioco** (*game vision*), molto più generale dei pilastri. Si cristallizza in un documento che, quando verrà fatto (e quando il docente lo insegnerà), "quello è e quello rimane": **la visione non può cambiare in corsa**. Ci sono dei limiti, e quelli sono.

4.9.4 Dopo la pausa: game qualities e generi

Dopo una pausa di sei minuti ("ci rivediamo alle 4"), e qualche battuta con Alessandro, studente del terzo anno presente in aula per studiare, il docente risponde a una domanda arrivata durante la pausa sulle *game qualities* come punti insindacabili. Se le game qualities sono "prima persona, PvP", i giochi sono un milione. Se si aggiunge il genere, *hero shooter*, la falce si chiude: dentro ci sono *Overwatch*, *Deadlock*, *Valorant* (in prima persona), *Team Fortress*. Identificare altri elementi, che non fanno necessariamente parte delle game qualities ma sono più specifici, rende più semplice spiegare il gioco e anche **pitcharlo**. Ci può stare anche la direzione artistica, se ad alto livello; ma bisogna considerare che le game qualities sono comunque **molto macroscopiche**.

4.10 Client pitch direction analysis

Si torna alla mappa e la si percorre punto per punto, partendo dalla *client pitch direction analysis*: i modi per cercare, fare domande, chiedere.

4.10.1 Fate domande

All'inizio si fanno **tante domande**. Ma vale sempre, sottolinea il docente: se si hanno dubbi, si fanno domande; se c'è qualcosa, si parla, si chiede. In aula e fuori, non bisogna avere paura di fare domande, perché altrimenti il rischio è **darsi da soli una risposta sbagliata**. Vale anche al lavoro: se non si capisce qualcosa, si chiede; se viene fatta una domanda senza tutte le informazioni necessarie, si chiede, "per favore, parlate" (si veda anche la sezione 1.5).

4.10.2 Le cinque W

I metodi sono estremamente semplici. Il primo sono le classiche domande inglesi, le **cinque W** (*where*, *when*, *who*, *why*, *what*), a cui si aggiunge *how*.[8] Danno informazioni specifiche su una serie di elementi molto diversi, come lo spazio e il tempo. Non sono un framework: sono domande, la base per spiegare qualsiasi tipo di storia. Se si deve spiegare il concept del proprio gioco, lo si può fare con queste domande in una frase, o al massimo in cinque righe, e diventa molto chiaro:

- *dove* è ambientato il gioco?
- *quando* è ambientato?
- *chi* è il protagonista?

[8] In aula la lista viene ricostruita a memoria con l'aiuto degli studenti; la formula classica è *5W + 1H*.

- *perché* fa ciò che sta facendo?
- *che cosa* deve fare?
- *come* lo fa?

Le risposte iniziano a separare le cose. *Metroid* e *Hollow Knight*, pur avendo lo stesso sistema di progressione, hanno stili di combattimento diversi: il *what* e lo *how* cambiano notevolmente (e, tra i due, anche il *who*).

4.10.3 I nove perché e la root cause analysis

L'altro metodo sono i **nove perché**: qui le domande sono solo *why*. C'è un "mantra" secondo cui si può raggiungere la radice di qualsiasi problema rispondendo nove volte alla domanda "perché?". C'è un problema: ci si chiede perché; l'interlocutore dà una risposta; si chiede di nuovo perché, e così via. Aiuta a **eliminare possibili fraintendimenti**. È il metodo dei bambini molto piccoli, che continuano a chiedere "perché?"; filosoficamente si potrebbe dire che serve a raggiungere l'essenza delle cose (il docente non ci crede, ma la possibilità c'è). Elencare le risposte a questi perché, sempre più in profondità, è la **root cause analysis**, l'analisi delle cause radice.[9]

4.10.4 Osservare, intervistare, filmare

- **Chiedere**: se c'è qualsiasi dubbio, se qualcosa è formulato in maniera incomprensibile, si chiede.
- **Osservare e intervistare** l'utente e il cliente, con **domande strutturate**. Quella fatta in aula ai fan dei soulslike ("che cosa ti piace, che cosa non ti piace?") è una domanda possibile, ma estremamente vaga. Con *poll* e *survey* (se ne parlerà) si possono limitare le risposte, se si ha già un'idea di dove ci si vuole muovere, o migliorare la domanda andando sempre più nello specifico.
- **Filmare** le persone, se possibile, per non perdere dati: mentre giocano, mentre le si intervista. Non bisogna perdersi niente: anche i responsi emotivi, i sorrisetti, i dubbi, i silenzi sono importanti. È un materiale enorme e non è detto che serva tutto; ma se serve, è lì.
- **Giocare e interagire**: tutto questo dovrebbe portare all'*insight*; ci si arriverà, e insieme alla correlazione porterà ad altro.

4.10.5 Priorità e visione del cliente

Ordinare le priorità. Il cliente avrà priorità maggiori e minori. A volte consegna la lista di ciò che vuole, ma **ordinarla spetta al designer**: alcune cose saranno più pertinenti, altre più negoziabili.

La visione e la linea editoriale del cliente. Qual è la visione del cliente? Come si diceva di Milestone, il cliente può avere una *missione*, una verticalità costruita negli anni: fa solo giochi, o videogiochi, o solo un certo tipo di esperienze. Se si cercano finanziamenti per un gioco con un mago gatto che spara vodka infuocata, forse a **Devolver Digital** può interessare: il suo catalogo è pieno di giochi di questo tipo, come *Enter the Gungeon*, dove tutto è proiettili, o *My Friend Pedro*, dove una banana parlante guida il protagonista a suon di sparatorie. Le **linee editoriali** sono importanti: il cliente potrebbe avere già una visione specifica da abbracciare, se si lavora sotto di lui.

4.10.6 Must have e nice to have

Questo è l'esempio che si riporterà nella visione: le cose da fare per forza.

Must have Ciò che è **indiscutibile**, senza compromessi: le meccaniche, i sistemi, le cose da implementare per forza.

[9] Nella letteratura di management la tecnica è più nota come *5 Whys* ("cinque perché"), nata nei processi produttivi di Toyota; la versione a nove domande, *Nine Whys*, è una delle *Liberating Structures* citate più avanti (sezione 4.12). Il principio è lo stesso: ripetere la domanda finché non emerge la causa di fondo.

Nice to have Ciò che è buono che ci sia, ma non è detto che ci sarà. Generalmente sono **aggiunte alla produzione** a cui, realisticamente, ci si può dedicare se si hanno più tempo e risorse, dopo aver risolto i must have. Se mancassero, l'anima del gioco, il suo *core*, resterebbe intatta: il gioco resta solido, comunica lo stesso messaggio, resta coerente con sé stesso. "Se ci fossero sarebbe meglio; se non ci sono, vanno in pace, ce ne faremo una ragione."

Dove finiscono i nice to have. Uno studente cita un gioco uscito senza un sistema di mappa, aggiunto due mesi dopo il lancio.[10] Esatto: **DLC e patch** sono gli esempi perfetti per inserire i nice to have o per rispondere a esigenze dell'utenza a cui prima non si era pensato o che non si riuscivano a risolvere. Nel DLC di *Elden Ring*, ad esempio, l'utenza faticava a battere Radahn, e il sistema è stato modificato in risposta (si può discutere se sia stato giusto, se il feedback sia stato recepito e se la reazione sia stata corretta, ma è successo). Uno studente aggiunge che, secondo lui, anche la seconda fase del boss finale del gioco base era pensata per essere affrontata a cavallo, cosa che non è successa; il docente non lo sapeva, ma è "un ottimo esempio". Sono sistemi molto specifici che restano nel reame del "possiamo farlo se": se si fa tutto il resto, se si ha tempo, se si hanno le risorse. Vanno identificati e messi nel documento di visione; se ne riparlerà.

4.10.7 Esempi concreti e situazioni estreme

Usare esempi concreti. Mentre si parla con l'utente o con il cliente, si usano esempi di giochi simili per capire se la linea desiderata è quella: giochi di successo e giochi che hanno fallito. Più avanti si cercherà anche di capire *perché* hanno fallito: se il proprio competitor diretto è un gioco che ha fallito, bisogna assolutamente rispondere a quella domanda. L'assunto di base, chiaramente, è che il proprio gioco non fallirà.

Pensare a situazioni estreme. Se nessuno giocasse al nostro gioco, falliremmo? Va capito. Non è necessariamente compito del designer, ma se si dovesse fare un piano di produzione e di rilascio, ci sono diverse fasce:

- quante copie servono per arrivare al **break even**, in pari tra spese e guadagni?
- in quante copie si traduce un **insuccesso**? E un risultato **disastroso** (probabilmente zero clienti)?
- quanto vale un **successo**? E un **successo folle**, quello per cui "andiamo tutti alle Maldive per tutta la vita" (o dovunque si voglia andare)?

E se si avesse un milione di utenti? Se il gioco è online, con server dedicati, e l'affluenza è esagerata, che cosa succede? È capitato più volte: con *Overwatch* all'inizio, e di recente con un altro gioco *live service*. Bisogna pensarci prima, per **limitare i danni**: prevedere tutto non è possibile, "è inumano". E se la Terra esplodesse? Provocatoriamente: forse si giocherebbe su Marte ("io sicuramente non ci giocherei, perché non ci arriveremo"). Anche questo è un esempio di root cause analysis.

4.11 Gli stakeholder

4.11.1 Chi sono

Uno **stakeholder** è qualsiasi persona, organizzazione, gruppo sociale o società in senso lato che ha uno *stake*, una posta, nel progetto: ha qualcosa da dire, "ha le mani in pasta". Ha dei **diritti** sul progetto, ma si porta dietro anche dei **doveri**. Gli stakeholder vanno **identificati**: bisogna capire, dentro e fuori dal team, chi c'è e quanta importanza ha.

Anche gli utenti rientrano in questa categoria; ma di solito, parlando di stakeholder, è più rilevante **chi sta sopra** che chi sta sotto. Gli utenti sono i "principi", ma qui interessa chi può modificare la traiettoria del progetto al di là di ciò che si potrebbe prevedere, chi ha (o avrebbe) qualcosa da dire durante una deadline, una milestone e così via. Si definisce, in sostanza, **tutto ciò che è fuori dal team operativo**.

Si torna alla piramide già vista: i *director* (creative director, game director, art director, sound director...), poi i *lead*, poi le seniority (senior, mid, junior, come si vuole). Accanto c'è la produzione, con il

[10] Il titolo non è ricostruibile con certezza dalla trascrizione.

producer, già un livello a sé, che potrebbe essere identificato come stakeholder; e da un'altra parte, completamente, c'è il **team di marketing**.

Il marketing e il CEO. "Una cosa molto brutta che succederà a voi e succederà anche a me in futuro": il team di marketing può bussare alla porta e dire "questa cosa non è abbastanza *screenshottabile*". Il designer può rispondere che a lui serve la meccanica, non che finisca su Instagram; ma il marketing può comunque venire a fare richieste. Oppure c'è il **CEO**, che con la produzione non ha nulla a che vedere: "ieri ho visto su History Channel che Giulio Cesare aveva una spada gigantesca: mettiamo una spada gigantesca nel gioco". Non dovrebbe essere il designer a rispondere di no: la richiesta dovrebbe essere filtrata prima, ma a volte non succede. Il CEO è uno stakeholder *definitivo*, come si vedrà.

4.11.2 Classificarli

Ci sono diversi modi di identificare gli stakeholder e di dar loro un volto **archetipale**, cioè farli ricadere in gruppi specifici. Aiuta a ragionare e a rispondere in maniera diversa, ma codificata, a seconda del gruppo: al marketing in un modo A, al CEO in un modo B, e così via.

La matrice potere–interesse. Un primo modo è collocarli su un piano cartesiano con due assi, **potere** e **interesse** (tabella 4.3).

Table 4.3: La matrice potere–interesse degli stakeholder.

	Interesse alto	**Interesse basso**
Potere alto	*Promotori* (es. il CEO): tenerli buoni, aggiornarli e rassicurarli costantemente	*Latenti* (es. un investitore interessato solo ai ricavi): tenerli soddisfatti
Potere basso	*Difensori* (es. la critica videoludica): tenerli informati	*Apatici*: monitorarli, perché possono cambiare posizione

Promotori molto potere e molto interesse: il CEO, ad esempio. Vanno tenuti buoni sempre, aggiornati costantemente ed eventualmente rassicurati, costantemente.

Latenti molto potere ma poco interesse: vanno **tenuti soddisfatti**. Se un'azienda ha investito nel gioco ma non è interessata a come esce, bensì a quanti soldi fa, le si danno informazioni sulla produzione: se si è in pari con le deadline, se i costi sono nel reame del previsto, se i prospetti di vendita sono allineati. Se si è cambiata una meccanica, se si è aggiunto o tolto un boss, se era un drago gigante o una viverna sottomarina, al latente interessa poco; e non conviene nemmeno dirglielo, perché potrebbe creare più problemi che altro. Se c'è un problema di costi, si parla di quello. Il design è meno rilevante.

Apatici poco potere e poco interesse: vanno **monitorati**. Possono cambiare il loro valore sulla scala: in questo momento della produzione restano lì, ma in futuro potrebbero avere più potere o più interesse.

Difensori poco potere e molto interesse: vanno **tenuti informati**. Possono essere, ad esempio, la critica videoludica (e bisognerebbe capire quanto potere abbia nel mondo moderno: relativamente poco, "o almeno mi piace pensarlo"). Non hanno potere sulla modifica del progetto, ma vanno tenuti informati.

Il modello della salienza. Un altro modo usa un diagramma a tre insiemi[11], le cui categorie sono **potere**, **legittimità** e **urgenza**. Uno stakeholder può avere molto potere sul progetto ma poca legittimità (ciò che dice è poco attinente), e molta urgenza (deve essere aggiornato di frequente). Un producer, ad esempio, ha generalmente sempre molta urgenza e, non necessariamente ma a seconda dei casi, meno legittimità nelle sue richieste. Lo stakeholder **definitivo** le ha tutte e tre; gli altri ne uniscono due o ne hanno una sola. Non si entra troppo nel dettaglio per questioni di tempo, ma lo strumento è nelle slide.

[11] Si tratta del *modello della salienza degli stakeholder* (*stakeholder salience model*) di Mitchell, Agle e Wood; nella trascrizione il nome è storpiato.

4.12 Problema contro necessità

4.12.1 Due definizioni

Nel gergo comune tutto è un problema: se c'è qualcosa da affrontare, un intoppo, è un problema. Ma si può dare una definizione più sottile, che distingue il problema dalla necessità.

Problema appare in una **struttura già esistente**. È qualcosa di facile da risolvere, ed è necessario risolverlo in maniera efficiente: prima lo si risolve, meglio è. Un buco nel muro; nascondere "la polvere sotto il tappeto"; qualcosa che si fa velocemente. Un **bug** è generalmente un problema, se non nasce da una struttura disintegrata del sistema. Di solito si assume (il docente lo assume) che il programmatore pensi l'implementazione in prospettiva futura: rende esplicite le variabili per i designer, o ne nasconde alcune perché il designer non distrugga tutto mettendo uno zero dove non deve. Se in un sistema del genere capita un bug, è un problema. Nel reame della vendita: se i fan dei soulslike chiedono qualcosa per sistemare la difficoltà o per avere una curva di progressione diversa, è un problema, perché sta in una struttura già esistente (il soulslike stesso): vanno toccati alcuni punti che non modificano l'identità del titolo.

Necessità è qualcosa di **nuovo**, che può portare a grossi danni a cascata. Se là fuori qualcuno chiede a gran voce un monster collectible in cui si schiavizzano i mostri, li si "massacra di botte" e li si arma con un AK-47, quella è una necessità: non appare in una struttura esistente, bisogna **costruirci un gioco attorno**.

Non sono mutualmente esclusivi. Uno studente riprende il boss finale di *Elden Ring* pensato per essere combattuto a cavallo, dove il cavallo non si poteva usare nelle prime patch: problema o necessità? Si può stare **a metà tra le due**. Probabilmente era più vicino alla necessità, ma è stato risolto come un problema, per i soliti limiti di tempo, soldi e produzione. Nulla vieta di farlo, se il tempo è poco.

Un caso vissuto: il quest log. Al docente è capitato di avere sistemi complessi che, tramite l'iterazione, mostravano delle mancanze: un sistema di quest, o il fatto che in un certo momento della progressione dovesse accadere un evento specifico. Serviva una **struttura nuova**, proprio per il discorso sulla programmazione fatto prima: qualcosa che accogliesse un sistema di quest complesso, un *quest log* fatto come si deve, sia nella presentazione all'utente sia "sotto il cofano", con un'implementazione sensata. Non c'era tempo, e si faceva un **"accrocchio" in Blueprint**, qualcosa che funzionasse e che, lato utente, desse lo stesso risultato: la quest veniva risolta, la porta si apriva, l'oggetto veniva dato. Ma non era la forma corretta: lì serviva un sistema complesso, era una necessità, e per necessità (appunto) la si è trattata come un problema. Non è l'ideale, ma capita. Succede anche quando si fa un titolo innovativo o si vuole testare un sistema nuovo: è una necessità perché richiede qualcosa che non è mai stato fatto, o un lavoro complesso che si porta dietro una forma complessa.

Forma e funzione. In modo semplicistico: nella necessità **la forma è maggiore della funzione** (quanto maggiore, va capito); nel problema la funzione "ha la meglio". È la differenza tra mettere un cerotto e fare i punti di sutura, o asportare un rene e richiudere la ferita. Sistemi complessi, potenzialmente innovativi, richiedono emozioni complesse da evocare e pensare, meccaniche e sistemi da soddisfare.

Approfondimento: la lente del problem statement. Schell parte dallo stesso punto: lo scopo del design è risolvere problemi, e prima di generare idee bisogna sapere *perché* lo si fa. Un buon *problem statement* dichiara insieme l'obiettivo e i vincoli ("come posso fare un gioco per browser che piaccia davvero agli adolescenti?"), e va calibrato: troppo largo porta a soluzioni che non centrano l'obiettivo, troppo stretto taglia fuori soluzioni brillanti. I vantaggi sono tre: uno spazio creativo più ampio, un criterio chiaro per misurare le idee (quanto bene risolvono il problema?) e una comunicazione migliore nel team. La *Lens of the Problem Statement* (lente n. 14) chiede: quale problema sto davvero cercando di risolvere? Ho fatto supposizioni che non c'entrano con il vero scopo? Un gioco è davvero la soluzione migliore? Come saprò che il problema è risolto? In apertura del paragrafo Schell riporta una massima anonima: "non innamorarti della tua soluzione, innamorati del tuo problema".

4.12.2 Strumenti per metterlo a terra

I modi per mettere a terra questa analisi sono parecchi.

Il diagramma a lisca di pesce. Il **diagramma di Ishikawa** (a lisca di pesce) aiuta a identificare le cause secondo dipartimenti, materiali, macchine e così via: per ogni area principale si individuano cause primarie e secondarie, disposte "a lisca". È un modo tra tanti, lasciato a disposizione.

Le Liberating Structures. Molti di questi strumenti si trovano nelle **Liberating Structures**, una raccolta di metodi che si usano sia nella fase di ricerca sia, più avanti, nel brainstorming. Esistono anche in forma di carte, con un valore simile a quello delle carte di design di Schell (il *Deck of Lenses*, sezione 2.11), per risolvere problemi. Tra queste ci sono i **nove perché** e, ad esempio, la *Troika Consulting*, una consulenza a turno in piccolo gruppo. Altri strumenti si vedranno in futuro, sempre per il brainstorming.

Il framework Cynefin. Un altro modo, se non si sbaglia derivato dallo stesso ambito[12], invece di distinguere problema e necessità, categorizza "l'impiccio" che si ha davanti (per non dire problema) in quattro categorie: **semplice**, **complicato**, **complesso**, **caotico** (tabella 4.4).

Semplice La difficoltà sta **nel compiere l'azione**, mai nella lista di azioni da fare. Non saper fare una UI in Unreal può essere un problema semplice: si cerca un tutorial, si ha la lista di cose da fare, le si fa, risolto. Una ricetta: fare la torta è un problema semplice, perché si assume che, seguendo gli step esattamente come indicato, il risultato sia **altamente riproducibile**.

Complicato Il grado di difficoltà si alza, perché la lista di protocolli non è legata a una sola disciplina, ma a **più discipline**, magari molto lontane tra loro e molto complesse. Far decollare un razzo richiede calcoli matematici complessi, simulazioni fisiche, materia prima (una lega che resista a quel calore), l'addestramento di chi ci sale, il carburante. La percentuale di successo è molto più bassa che nel semplice: **si può prevedere**, si può ridurre il rischio, come si fa con qualsiasi lancio, ma è più difficile.

Complesso La prevedibilità viene meno: il problema diventa **non categorizzabile**. L'esempio è educare un bambino, perché ogni individuo è a sé. Chi ha fratelli o sorelle è stato probabilmente educato in modo simile, perché il modello educativo dei genitori è quello; ma a seconda dei casi i genitori aggiustano il tiro sulla risposta dell'"utente", il figlio. Il docente ha una sorella ed è completamente diverso da lei, nonostante siano cresciuti in maniera molto simile.

Caotico L'unica discriminante la fanno la **collaborazione** tra i membri e l'**improvvisazione**. Gli esempi sono mosca cieca e *Taboo*: un certo numero di parole da non dire, un tempo limitato per far indovinare una parola al proprio compagno di squadra. Tra amici si usano *inside joke*, un gergo che solo il gruppo conosce: far capire qualcosa a qualcuno con cui si ha grande intimità è più semplice, perché si condividono esperienze e intuito. Per spiegare una parola si cita un meme che entrambi conoscono.

Table 4.4: Le quattro categorie dell'"impiccio" (framework Cynefin).

Categoria	Dove sta la difficoltà	Esempio
Semplice	nell'eseguire, non nel sapere che cosa fare	una ricetta, un tutorial di UI in Unreal
Complicato	più discipline intrecciate; prevedibile ma rischioso	il lancio di un razzo
Complesso	non prevedibile né categorizzabile; si aggiusta il tiro	educare un bambino
Caotico	solo collaborazione e improvvisazione	mosca cieca, *Taboo*

Sono modi per **categorizzare** il problema, la necessità, qualsiasi cosa ci sia da affrontare: dargli un nome o una categoria può aiutare, per semplicità.

[12] Si tratta del framework *Cynefin*, ideato da Dave Snowden, che le Liberating Structures includono tra i propri strumenti.

4.13 Analisi dei competitor

4.13.1 La SWOT e le domande

Ci sono diversi modi, più o meno precisi, per analizzare i competitor, e molti si vedranno nel corso: lo stesso framework dei *core elements* (sezione 2.7) serve anche all'analisi, ed è proprio ciò che si sta facendo nel task; non è l'unico.

La SWOT analysis. Tra i modi ormai conosciuti c'è la **SWOT analysis**: punti di forza (*strengths*), punti di debolezza (*weaknesses*), opportunità (*opportunities*) e minacce (*threats*), cioè ciò che minaccia il progetto. La si può disporre a matrice. Non si usa solo nell'industria videoludica, anzi: è presa dal management generico.

Le domande sui competitor. Per i singoli competitor conviene però porsi delle domande, soprattutto sui **diretti**, che vanno analizzati in profondità ("su questo non ci piove"):

- che cosa fanno, e come lo fanno?
- c'è un **design dominante**? Perché?
- c'è qualche design che si tende a **evitare**? Perché?
- quanto hanno venduto?
- quali sono i loro **unique selling point**?
- che cosa piace al loro target audience, e che cosa no? Che cosa comunica? Ci sono parole specifiche?
- c'è una categoria specifica di *influencer* che adotta o gioca quel tipo di giochi? (Ormai è necessario per il marketing.)
- ci sono feature di gameplay specifiche che vengono mostrate, o no, nel marketing? Come si fa marketing, con quale tono di voce? (Questo "grosso peso" il docente lo lascia a Daniele.)
- sono stati un successo o un fallimento? Perché?
- che cosa si può fare realisticamente, sempre entro soldi, team e così via?

Informarsi senza giocare. Bisogna guardare interviste, talk di settore, come hanno fatto. E se non si riesce a giocare un titolo (magari non è ancora uscito) si guarda il **gameplay senza commento**, come diceva il direttore. Nemmeno il docente gioca a tutto, e realisticamente non potrebbe: ci sono modi laterali per ottenere informazioni su un gioco guardandolo.

4.13.2 Il design dominante

Il **design dominante** è ciò che sta dietro al detto "non reinventate la ruota": quella ruota esiste già da qualche parte. È qualcosa che viene riapplicato N volte, nella stessa identica maniera: perché?

In *Dark Souls*, ad esempio, il sistema di morte e recupero delle anime è uno standard, e non solo lì: anche fuori dal genere. C'è un motivo: è **estremamente efficace**. E qui torna l'atomo di Koster visto all'inizio (sezione 4.1): *Dark Souls* funziona così bene proprio per quel discorso di **feedback** e di **educazione del modello mentale del giocatore**. Se ne riparlerà quando si parlerà di psicologia, a fine anno.

Si può brevettare una meccanica? Uno studente chiede se si possa brevettare una meccanica. Il grande vantaggio del game design è che **le meccaniche non hanno brevetti**, con un asterisco. Sono come **blocchi di Lego**: così specifiche nell'applicazione, ma così generali nelle regole. Non si può brevettare il salto, perché il verbo di gioco è lo stesso tra *Mario* e *Uncharted*, ma le applicazioni sono completamente diverse; anche tra *Celeste* e *Mario*, pur essendo entrambi platform. Si possono quindi prendere meccaniche e sistemi di gioco uno a uno da giochi anche lontani dal proprio, e va benissimo. A volte lo si fa per un gioco intero: il primo *Lords of the Fallen* è, di fatto, "la *skin* di *Dark Souls*", con gran parte dei sistemi riprodotti uno a uno. Funziona; non è particolarmente creativo e potrebbe non essere l'aspirazione di un designer, ma si fa, per tempistiche e per semplicità.

I casi limite. Ci sono casi limite di brevetti, ma sono estremamente rari.

- **Pokémon**: il brevetto usato nella causa contro *Palworld* riguarda, se il docente ricorda bene, la creatura che finisce dentro un oggetto portatile e la possibilità di evocare direttamente una creatura cavalcabile.[13] Lo si può aggirare in modo molto semplice: la creatura non entra nella sfera, entra nello zaino o nelle scarpe; e quando la si lancia non la si cavalca, ma ci si infila "come un canguro" nel suo marsupio. Non si può brevettare una meccanica estremamente specifica: "nessuna agenzia ti darà l'ok".

- **Il Nemesis System** di *Middle-earth: Shadow of Mordor*, un "caso triste" che il docente cita spesso, perché per chi l'ha giocato è probabilmente la cosa più memorabile del gioco, e non lo si usa più. L'orda di Mordor (orchi, sul modello del *Signore degli Anelli*) ha una **gerarchia interna**: capi, gruppi, sottogruppi. Quando si uccide, o non si uccide, un orco, questo può salire o scendere di grado. Se lo si uccide viene rimosso e qualcuno prende il suo posto; se sopravvive a uno scontro, si ricorderà del giocatore e vorrà vendicarsi. Se uccide il giocatore, questi rinasce normalmente: la morte è prevista nell'esperienza come prosecuzione del sistema, e l'orco viene promosso per aver ucciso una figura di alto rango. Questo sistema è **brevettato**. Il docente pensa che si possa fare qualcosa che faccia percepire lo stesso responso emotivo, ma è difficile.

Sono casi rari: in generale è molto liberatorio poter prendere qualsiasi meccanica e "rivestirla".

Perché il design dominante si riproduce. Il discorso di *Dark Souls* si ripete un po' perché funziona estremamente bene a livello di design (si capirà perché), un po' perché **il target audience se lo aspetta**. Un giocatore di soulslike che prova un altro soulslike si aspetta quel sistema: glielo si può dare tranquillamente, e anzi si fa meno lavoro, perché lo si riprende uno a uno. Sempre che non si voglia innovare: **innovare costa**, e non è nemmeno una necessità.

4.13.3 Gli unique selling point

Gli **unique selling point** (o *unique selling proposition*, USP) vanno valutati **prima della produzione**. Sono cose uniche: una **commistione di fattori** che solo quel gioco ha. Di solito sono **tre, al massimo tre**. Per *Palworld*, ad esempio, il fatto che ci sia un monster collectible con un'arte "pucciosa" e le armi: non è solo questo, ma è chiaramente l'elemento unico di punta.

Un nome famoso è un USP? Valerio chiede se possa essere un USP anche qualcosa che non è una feature del gioco: ad esempio, il nome di Kojima. La risposta è "sni". Volendo essere molto generosi, sì; ma è un *selling point*, non uno *unique* selling point, perché di giochi "di Kojima" ce n'è già più di uno. Certo, se si è Kojima si va da Sony o da Microsoft ("mi paga uno, mi paga l'altro, è uguale") e si ottengono i finanziamenti. Ma non bisogna limitarsi a questo: il docente è convinto che, nella direzione in cui sta andando l'industria, non sarà più così vero.

[13] Il docente afferma che Nintendo avrebbe perso; la vicenda legale tra Nintendo, The Pokémon Company e Pocketpair era ancora in corso di cronaca al momento della lezione, e se ne riporta l'affermazione così com'è.

Il terzo capitolo di una saga. Uno studente racconta che, al primo anno, si era discusso di quale fosse l'USP di un *Borderlands*: si citavano lo shooter e molte altre cose, finché qualcuno disse "è il terzo". Il punto è: che cosa dà in più rispetto agli altri? Il docente ricorda il caso del *Pre-Sequel*, che è insieme un prequel e un sequel: quello sì è un modo unico di vendere il prodotto. Allo stesso modo, in *Hollow Knight: Silksong* l'USP non è "siamo nel mondo di *Hollow Knight*", ma "non gioco con il Cavaliere, gioco con Hornet". Si può dire che il terzo titolo di una saga è unico, ma bisogna **formularlo in maniera più chiara**: il fatto che il titolo faccia parte della saga e *aggiunga un pezzo*. Lo si fa con gli spin-off: ambientato in quel mondo, ma con queste meccaniche in più. Il nuovo *A Plague Tale* citato in aula mantiene la mitologia e la proprietà intellettuale (IP, *intellectual property*) della serie, ma ha meccaniche diverse: meno stealth, più action.

Una parentesi: fate domande. A uno studente che si scusa per aver riportato la lezione indietro, il docente risponde che è assolutamente legittimo: "non ho interesse a concludere la lezione, fare le tre ore e andare a casa. Siamo qua per discutere". Se c'è qualcosa che non si capisce, ci si ferma e se ne parla; "e sono pure pagato per parlare di giochi". Se le domande deviano il discorso e rendono le informazioni più utili, tanto meglio.

Esempio: gli USP di Enotria. Essere un soulslike è un *selling point* (*Enotria* vende anche perché è un soulslike), ma non è *unique*. Gli USP di *Enotria* sono altri; il docente fatica a trovarne un terzo, ma due li trova:

1. sistemi da soulslike con una direzione artistica lontana dal grimdark: è il **Summer Souls**, e in più è **ambientato in Italia**, con il folklore italiano che si fonde con la mitologia tipica dei soulslike, boss enormi e mostruosi;

2. il **sistema di maschere**: in un contesto soulslike, si possono avere diverse *build* e cambiarle in tempo reale durante il combattimento, come se in *Bloodborne* si cambiasse arma mentre si combatte (cosa che, più o meno, in parte si può fare).

Si possono vendere tante cose, ma per pitchare il gioco come unico bisogna far emergere questo. Per giochi piccoli l'USP potrebbe essere anche uno solo (ma il docente invita a identificarne di più); per giochi molto grandi in genere si arriva a tre. Sono cose che nessun altro ha fatto: in quella forma, in quella dimensione, "in quella salsa", solo il proprio gioco.

Esempio: Metroid Other M e Metroid Dread. Il docente, grande fan della saga, porta l'esempio di *Metroid*. Gli USP si recuperano anche da trailer e interviste.

- *Metroid: Other M* è l'ennesimo capitolo della saga, e il suo USP era Samus come protagonista **umana**: si racconta il suo passato in un contesto in cui Samus parla e si relaziona con gli altri. Samus è riconoscibile, amata dai fan, ma più umana, e parla: "infatti è piaciuta a pochissimi". Ha delle intuizioni interessanti, ma la narrativa non è il suo punto forte. Samus, in genere, non parla in nessun gioco; qui "parla e straparla", mostrando lati deboli che non sono emersi nella maniera ideale per il target.

- *Metroid Dread* lo ha capito: Samus parla, ma non in una lingua comprensibile: ma nella lingua degli alieni (i Chozo, da cui è stata cresciuta). È "la cosa più figa dell'universo": non mostra paura, è dura, cruda, precisa, esattamente la cacciatrice di taglie che si è abituati a giocare.

Ma l'USP di punta di *Dread* è un altro: gli **E.M.M.I.**, nemici che inseguono e cacciano il giocatore e che non si possono uccidere fino a un certo punto. Ed è un collegamento con la storia della serie: in *Metroid Fusion* c'era già un nemico simile, l'SA-X, una copia della tuta di Samus presa vita a causa di un parassita, che dà la caccia al giocatore in alcune sezioni del livello. Lì era estremamente limitato (il GBA era un'altra epoca); gli E.M.M.I. di *Dread* sono quel concetto espanso, e qui è molto più predominante. L'USP è simile, ma diventa il centro dell'esperienza.

4.13.4 Che cosa resta da vedere

La prossima volta si concluderà questa parte e si vedrà che cos'altro si può fare con i competitor, perché non è limitato a questo. Questi sono modi di analizzarli ad alto livello, ponendosi un certo tipo di domande; ma molti dei framework di design visti servono anche a **spacchettare i giochi**, e conviene usarli nella fase di analisi.

4.14 Il task della settimana

Il nuovo task viene assegnato a metà lezione, subito dopo il production dictionary (sezione 4.3). La scadenza è **lunedì della settimana successiva** ("lunedì tra una settimana").

Che cosa Il **production dictionary** di un qualsiasi gioco **Pokémon della saga principale** (*mainline*) a scelta. Non valgono gli spin-off: niente *Mystery Dungeon*, niente *Pokémon Snap*, niente *Pokémon Rumble* ("ce ne sono un sacco, non li voglio"). A Daniele, uno studente, viene assegnato invece un *Final Fantasy* con un numero inferiore a 10.

Dove Questa volta spetta agli studenti "l'onere e l'onore" di creare la cartella e organizzare tutto secondo le regole del Drive (sezione 3.2).

Strumento Un documento di Google, con le regole formali del corso (sezione 3.9). L'impostazione della pagina resta quella standard, salvo indicazioni diverse: sempre A4, stampabile.

Vincolo Non si deve superare una pagina. Il limite serve a far ragionare sulla **priorità** delle parole e dei termini necessari: quali vanno spiegati per primi, perché più fraintendibili o più necessari alla comprensione.

Uno studente fa notare che i termini sono tanti e che, con limiti del genere, si rischia di dover correre: proprio per questo il limite c'è. Se ne riparlerà quando tutti avranno consegnato e il docente avrà letto i documenti.

Valgono, come sempre, le golden rules (sezione 3.7) e i termini banditi (sezione 2.12); e vale quanto detto sui template (sezione 4.4): nella lezione non viene mostrato un esempio di production dictionary proprio perché non ce n'è uno da copiare.

Prossimi appuntamenti.

- Le slide della lezione non vengono caricate subito: lo saranno quando la lezione sarà conclusa, cioè **mercoledì**.
- **Venerdì** la lezione si terrà **da remoto**, a causa dello sciopero dei mezzi (lo si scriverà anche su Discord per chi è arrivato dopo).
- La dimostrazione di formattazione di un documento si farà in presenza, **venerdì 6** (sezione 4.2).

Punti chiave del capitolo

- L'**atomo di gioco** di Koster: problema, preparazione, **core mechanic** calata in uno spazio, **feedback**, successo o fallimento (con ricompensa), e infine l'aggiornamento del **modello mentale** del giocatore, che vive solo nella sua testa.
- La core mechanic è l'azione specifica che il giocatore ripete più spesso; il verbo di gioco, in senso gergale, è più generico.
- Il feedback **non è mai abbastanza**: se qualcosa non viene capito, spesso il problema è lì. Si itera: si osserva, si rinforza, si ritesta.

- Il **production dictionary** chiarisce i termini necessari a comprendere il gioco; di solito è un documento a sé, ha corrispettivi in altri dipartimenti (il *lexicon*) e richiede **priorità**: prima i termini più fraintendibili o necessari.
- La documentazione non sostituisce il gioco, non tutti i documenti sono sempre necessari, e **non esistono template**: si prende il meglio, lo si ibrida, lo si adatta.
- Il design è il **miglior compromesso possibile** tra team e know-how, visione e scienza, mercato e target audience, produzione e soldi; è **50% scienza e 50% arte**.
- "Senza dati si è solo un'altra persona con un'opinione": servono dati, quantitativi e qualitativi, da almeno un centinaio di persone, raccolti in pre-produzione, osservati, filmati, ordinati in archetipi e correlati.
- Il design thinking in cinque passi: imparare dalle persone, trovare schemi, definire principi, rendere tangibile, iterare senza sosta.
- La mappa della ricerca: client pitch direction analysis, stakeholder, problema contro necessità, competitor, utenti, mercato, gameplay, fino a una **visione** che non cambia in corsa.
- I **competitor diretti** condividono la maggior parte dei punti insindacabili; gli **indiretti** uno solo, molto attinente; è uno spettro. I giochi in **opposizione** aiutano a innovare.
- Il mercato può essere un **red ocean** o un **blue ocean**, e ha regole (Cina, Germania, Belgio) a cui pensare prima.
- Strumenti di indagine: cinque W, nove perché e root cause analysis, interviste strutturate, must have contro nice to have, scenari estremi.
- Gli stakeholder si classificano per **potere e interesse** (promotori, latenti, difensori, apatici) o per **potere, legittimità e urgenza**.
- Il **problema** sta in una struttura esistente e si risolve in fretta (un cerotto); la **necessità** richiede una struttura nuova (i punti di sutura). Ishikawa, Liberating Structures e Cynefin aiutano a categorizzare.
- Nell'analisi dei competitor: SWOT, design dominante (le meccaniche, salvo rari brevetti, sono di tutti), **unique selling point** (al massimo tre, davvero unici).
- Task: production dictionary di un Pokémon mainline, **una sola pagina**, per lunedì della settimana successiva.

Chapter 5

Dalla ricerca alle idee: perceptual mapping, utenti, mercato, motivazioni e brainstorming

Il capitolo precedente si è fermato a metà strada: la lezione su *problema e soluzione* era stata annunciata come troppo lunga per una sola giornata, e si era chiusa con le domande per l'analisi dei competitor (sezione 4.13). Questo quinto capitolo la porta a termine. L'obiettivo dichiarato dal docente è "realisticamente" solo questo, finire la lezione; e in effetti la giornata copre tutto il resto della ricerca in pre-produzione, fino alla sua conclusione naturale (la *visione*), e apre la parte successiva, quella sulla **generazione delle idee**.

La lezione è tenuta ancora da Mattia Podini. Il percorso del capitolo ne ricalca la struttura:

1. le domande in apertura: il **production dictionary** ripreso e approfondito, con una digressione su come è organizzata la documentazione nelle produzioni moderne e sulla distinzione tra documenti di visione e documenti di produzione (sezioni 5.1 e 5.2);
2. una breve ripresa del punto in cui ci si era fermati, con una parentesi su Deming (sezione 5.3);
3. gli ultimi strumenti per l'analisi dei competitor: i framework usati anche per l'analisi, l'**elicitazione triadica** e la **perceptual mapping**, con due esercizi in aula (sezioni 5.4 e 5.5);
4. le altre fonti di dati, l'**analisi degli utenti** e l'**analisi di mercato** (sezioni 5.6, 5.7 e 5.8);
5. l'analisi di gameplay attraverso il modello delle **motivazioni** di Quantic Foundry (sezione 5.9);
6. la **correlazione** dei dati e la chiusura della pre-produzione con le domande che portano alla *game vision* (sezioni 5.10 e 5.11);
7. l'inizio della parte su **idee e brainstorming**: i principi, il *design elegante*, la creatività e una rassegna di tecniche (sezioni 5.12–5.15);
8. il task della settimana, a gruppi (sezione 5.16).

Come per il capitolo precedente, la lezione non arriva fino in fondo: la parte su *come* si conduce un brainstorming è rimandata alla lezione del venerdì successivo.

5.1 In apertura: il production dictionary, ripreso

5.1.1 Prima di cominciare

La registrazione si apre sulla coda di un momento informale: il docente sta commentando una partita, e osserva che una *Blue Map* "non sarebbe per niente male", perché permetterebbe di vedere subito dove sono

le stanze segrete senza sprecare bombe per cercarle; resta da trovare "al più presto un'altra monetina" che gliela consigli.[1]

Poi si entra nel vivo, con il solito rituale: prima di cominciare, domande o dubbi sulla lezione precedente, sui task in generale, su qualsiasi cosa. Il docente ricorda che il termine ultimo per la consegna dei **task sugli arcade** è scaduto: i documenti restano lì, "fermi, fissi", finché non li commenterà, probabilmente nel fine settimana (si veda il task in sezione 2.14).

Un altro brevetto aggirato. Prima della lezione alcuni studenti stavano discutendo "con grande fervore" di un altro esempio di brevetto aggirato, sulla scia del caso Pokémon visto la volta precedente (sezione 4.13). Il docente conferma che di esempi simili ce n'è più di uno, ma che non serve approfondirli; non sa che cosa sia stato detto prima del suo arrivo.

5.1.2 Nessun limite di pagine

Uno studente chiede di ripetere in cinque minuti il concetto di **production dictionary** (sezione 4.3). "Certamente."

Lo scopo di un production dictionary è lo stesso per cui esiste un dizionario: **spiegare terminologie specifiche**. Nelle produzioni vere non esiste una limitazione come quella data nel task: non esiste un production dictionary "di una pagina", "di due pagine" o comunque limitato in grandezza. Il limite di una pagina era un vincolo dell'esercizio (sezione 4.14).

5.1.3 File a sé o paragrafo d'apertura

La discriminante più grossa, come detto l'ultima volta, è **dove sta** il production dictionary nella struttura della documentazione. Il docente nota che è un argomento che non si tratta quasi mai, se lo segna e promette di aggiungere qualche slide.

Come documento a sé può restare un file autonomo all'interno del Drive ("assumiamo che la vostra azienda lavori con Google Drive, cosa possibilmente non vera").

Come elemento di un altro documento può diventare un paragrafo, o un'intestazione, "non so come volete chiamarla", posto **in apertura** di un singolo documento, per aiutare il lettore a capire in anticipo eventuali termini.

L'esempio di Wikipedia. Wikipedia funziona più o meno così. Gli argomenti sono larghissimi e diversi, ma ci sono sezioni che indirizzano a voci specifiche. Si prenda la pagina di *Liverpool*: tutte le informazioni al suo interno, o già la colonna a sinistra, rimandano a cose che si dovrebbero sapere per la buona comprensione di quella pagina, come "che cos'è una città" o "che cos'è una bandiera". Sono cose estremamente pratiche e semplici, ma il principio è lo stesso.

L'esempio del GDD dello sparo. Si supponga di scrivere un GDD (un *design document*: "GDD" qui vale per qualsiasi disciplina, e diventa il design document di quella disciplina) dedicato a una meccanica, per esempio il salto. In un gioco come *Mario* si può saltare e sparare: è possibile, non necessario, che nel documento si voglia spiegare anche la meccanica di sparare ("datemi un salto, lo sparo", si corregge il docente dopo essersi confuso tra le due). La si può spiegare nel production dictionary in testa al documento oppure, in alternativa, **linkare** e rimandare al documento "GDD Sparo". Il production dictionary aiuta proprio in questo: se ci sono terminologie necessarie alla buona comprensione di *quel* documento, le si chiarisce lì.

Attenzione al lettore. Resta valido quanto detto l'altra volta: un **blocco di terminologia** all'inizio di un documento probabilmente non è piacevole da vedere per il lettore (sezione 3.7). Le soluzioni sono quindi due: un documento a sé, che sta in un'altra parte rispetto agli altri, oppure un production dictionary **ibridato** con altre cose.

[1] Il riferimento è quasi certamente a *The Binding of Isaac*, in cui la *Blue Map* è un oggetto che rivela le stanze segrete della mappa. Il contesto (una partita in corso prima della lezione, forse con suggerimenti dalla classe) non è ricostruibile dalla trascrizione.

5.1.4 Ibridare i documenti

Il production dictionary non è l'unico caso: molti altri documenti, anzi in realtà tutti, si possono ibridare. Nessuno vieta di prendere i punti che un documento dovrebbe svolgere o contenere (le categorie che si vedranno per gli altri tipi di documento) e mescolarli con altro: "si fa". Il documento che più si presta all'ibridazione è proprio il **GDD**, perché non ha una forma specifica: è malleabile (si veda anche quanto detto sui template, sezione 4.4).

5.1.5 Che cosa ci va, e chi lo decide

Il production dictionary deve spiegare i **termini necessari per il prodotto**, termini che tutte le persone che ci lavorano devono condividere. Il *mana*, se il gioco lo ha, può sembrare un termine che non serve spiegare; ma il docente assicura che, parlando con un sound designer o con figure più lontane dal design, le cose cambiano. Molti collaboratori sono *outsourced*, presi al di fuori del team; lui stesso ha collaborato con tanti animatori che avevano poca conoscenza videoludica, perché venivano dal cinema o dall'animazione e non erano designer. Si assume che, bene o male, tutti giochino ai videogiochi (vale anche per gli artisti), ma spesso non è detto. Alcune terminologie che per un designer sono comuni devono esserlo **per tutti**, e quindi vanno spiegate.

Chi sceglie i termini. Che cosa mettere nel production dictionary lo sceglie il designer. In un documento specifico come quello del task non lo sceglie il junior: è il **lead** a decidere quali termini inserire. A riempirlo, però, sono poi molte mani diverse: chi ha fatto il documento del salto aggiunge le sue terminologie specifiche, chi ha fatto quello dello sparo avrà altre idee e le inserirà a sua volta, qualora servano.

Uno per progetto. Come scritto anche nelle slide, questi documenti possono avere un corrispettivo in altri dipartimenti (il *lexicon*, sezione 4.3). Ma c'è un'informazione che il docente non aveva scritto, e che dovrà aggiungere: di production dictionary ce n'è **uno per progetto**. È uno di quei documenti di cui ne esiste uno solo, e che viene sempre aggiornato.

5.2 Documenti di visione e documenti di produzione

5.2.1 Una categoria personale

Quanto appena detto vale, in generale, per tutti i documenti che il docente chiama **documenti di visione**. Avverte che non è una categoria largamente condivisa: alcuni documenti ci rientrano, altri no, ma nella sua testa "aiuta, mi dà ordine mentale", e potrebbe essere utile anche agli studenti.

Documenti di visione Rimangono "nei secoli dei secoli": vengono aggiornati, certo, ma aiutano a capire il progetto ad **altissimo livello**. Ne fanno parte il production dictionary (per la comprensione delle nomenclature), il **documento di visione** già accennato più volte, che si vedrà (uno per progetto: quando viene creato, "quello è e quello rimane"), ma anche i **pitch**, di cui esisterà magari qualche versione. Sono documenti che servono a vendere il progetto, o che ne incorporano l'idea, il concept, il cuore.

Documenti di produzione Sono i documenti che servono effettivamente a produrre il gioco: i **GDD** sono documenti di produzione. Ce ne sono anche altri che, per esempio, tracciano lo stato della produzione.

5.2.2 Dalla Bibbia alla wiki, e oltre

Il punto che chiarisce molte cose, e che l'ultima volta non era stato trattato, è che nelle produzioni moderne i documenti sono **separati**. Il grosso GDD, l'unico documento che spiega tutto il gioco *in toto*, a livello produttivo **non esiste più** (si veda l'approfondimento sul mito del Game Design Document, sezione 4.4). Sopravvive solo nel reame del marketing, o in approcci che non fanno parte della produzione e che qui non interessano.

Lo step intermedio sono state le **wiki di design**: è il caso di chi usa *Confluence*, che "è proprio questo". Dalla "Bibbia" si salva quindi la wiki, e poi c'è ciò che si usa oggi, che il docente chiama, in mancanza di un nome migliore, "una specie di wiki 2". Dipende dalle aziende; ma lavorando dentro Google Drive, gli studenti fanno già qualcosa di simile.

Documenti che smistano i lettori. In questa struttura c'è una documentazione di visione, di organizzazione del progetto (o di pre-produzione, "come vi pare") che guida chi lavora al progetto verso gli altri documenti. Aiuta la comprensione del progetto oppure reindirizza il lettore verso le parti specifiche che gli servono:

- un **artista** viene spedito nelle cartelle in cui sono archiviati gli asset;
- un **programmatore** viene indirizzato, per esempio, alla cartella in cui viene messa la build (se la si carica sul Drive);
- chi deve **implementare una meccanica**, programmatore o artista che sia, viene rimandato al documento specifico: deve implementare il salto, va al documento del salto; deve implementare lo sparo, va a quello dello sparo.

5.2.3 Chi tiene in ordine il "giardinetto"

La gerarchia di base è molto semplice; la sotto-gerarchia della sotto-gerarchia spetta a chi gestisce lo spazio di archiviazione: il docente dice "Drive", ma può essere qualsiasi storage di dati o file, "potrebbe essere pure su Git". Nel corso se ne occupano gli studenti, perché sono designer e il cerchio si restringe; ma in una grande produzione è responsabilità della **produzione**, cioè del team di produzione, generalmente composto da producer e production director (dipende da quanti sono). Sono loro a strutturare le cartelle e l'organizzazione del Drive, di Confluence o di qualsiasi altro strumento usato per raccogliere la documentazione. Poi ognuno lavora **nel proprio giardinetto**, che naturalmente va mantenuto ordinato.

È esattamente ciò che succede nel corso: ci sono delle regole, i documenti si creano nelle cartelle dei task (GDLD e così via), ed è il docente a spostarli quando serve (sezione 3.2). "Il rapporto è lo stesso, uno a uno."

Il docente nota il silenzio della classe ("non so se è un bene o un male; assumiamo che sia un bene"), invita a parlare in caso di perplessità e si torna alla lezione.

5.3 Dove eravamo rimasti

5.3.1 Il mantra del compromesso

La volta precedente si era iniziato il discorso su come individuare il problema o la necessità e come categorizzarli. Prima di proseguire, il docente torna sulla slide "più importante", il **mantra** entro il quale la produzione agisce: tutti gli ingranaggi si muovono e il **compromesso** determina che cosa si dovrebbe fare a livello di design, qual è la miglior soluzione possibile (sezione 4.5). Si erano già viste la citazione di Diego, il direttore del corso, e quella di Deming (sezione 4.6).

5.3.2 Una parentesi su Deming

Per chi volesse più informazioni, il docente lascia due "ganci". Non si può parlare di tutto ("mi piacerebbe, ma non si può"), quindi questi servono a fare ricerca per conto proprio.

Il ciclo di Deming. La metodologia di Deming è un modo di fare produzione basato su un ciclo: si pianifica, si fa qualcosa, si verifica, si agisce, e si ricomincia.[2] Il docente scherza anche sulla foto di Deming nelle slide ("bellissimo uomo").

[2] In aula l'ordine delle fasi viene detto in modo confuso ("check, act, plan, do"). La forma canonica del ciclo di Deming, o ciclo PDCA, è *Plan–Do–Check–Act*: pianificare, eseguire, verificare, agire (standardizzando ciò che funziona e correggendo ciò che non funziona), e poi ripartire.

Le sette malattie mortali. Un'altra cosa interessante di Deming sono le **sette malattie mortali della gestione occidentale**. Deming era americano, e poi si è spostato in Oriente (in Giappone); alcune di queste malattie risentono della sua cultura: al punto sei, per esempio, ci sono i *costi medici eccessivi*. Altre, invece, potrebbero essere rilevanti da approfondire.[3] Quella su cui si sofferma il docente, legata anche al terzo punto, è **dirigere un'azienda basandosi solo sui dati visibili e misurabili**: il fatto che la produzione possa non valutare il benessere o il morale del singolo individuo, degli operativi. Il docente lo ha visto spesso, ed è effettivamente un problema nelle produzioni. È uno dei motivi per cui molte produzioni *indie* sono più sostenibili: essendo in numero minore, ci si può preoccupare umanamente delle persone con cui si lavora. "Tuttavia, non è sempre vero."

5.3.3 La mappa della ricerca, a che punto siamo

Si torna alla **ricerca** e all'esempio guida della lezione precedente: il "magico" Poké Walker modificato, mettiamola così, per curare l'obesità infantile (sezione 4.6). Degli otto punti della mappa (sezione 4.7) si erano visti:

- i competitor diretti, indiretti e similari (sezione 4.8); altri strumenti si vedranno anche nella parte di brainstorming e di generazione delle idee;
- la definizione degli stakeholder (sezione 4.11);
- la definizione di problema contro necessità ("o comunque, se c'è un impiccio: non ho un sinonimo migliore"), con l'identificazione della categoria entro cui la questione si presenta, che dovrebbe aiutare a capire come affrontarla: nella maniera più efficiente e veloce possibile, o in una maniera più costosa ma potenzialmente con grandi ritorni a lungo termine (sezione 4.12);
- altre metodologie e categorizzazioni, e infine le domande sull'analisi dei competitor (sezione 4.13).

Come al solito, avverte il docente, in questa lezione soprattutto arriva una "valanga" di strumenti potenzialmente utili. Non vanno usati tutti: è irrealistico pensare di usarli sempre tutti.

5.4 Analisi dei competitor: framework ed elicitazione triadica

5.4.1 I framework servono anche all'analisi

Le domande viste l'ultima volta sono domande **ad alto livello** da cui partire per analizzare i competitor, o, più realisticamente, alcuni aspetti o sistemi di altri giochi. Per farlo si usano **framework adatti**: i framework visti in design, ma non solo (anche quelli di level design), si possono applicare sia all'analisi sia al design, e spesso ad entrambe contemporaneamente. Gli studenti hanno appena usato i *core elements* per analizzare gli arcade; li possono usare anche per progettare (sezione 2.8).

Anche per il pitch. Quando il docente fa una presentazione, per il proprio gioco o per quello di qualcun altro (gli è capitato di fare *pitching* anche per altri: esiste anche questa figura), una delle prime slide serve a far capire il gioco all'ascoltatore. Come? Per esempio con una lista di **game qualities** di Romero e Schreiber: che tipo di camera ha, che genere è, se è multiplayer o single player, eventuali altre cose. Lo stesso strumento serve quindi all'analisi, al pitching e al design. I *core elements* sono un framework ancora molto basilare, così come quello di Bura (sezione 2.9); il discorso su come fare design con quest'ultimo era già stato fatto, è complesso, e il docente invita prima a fare ricerca: sul sito di Bura si trova tutto.

[3] L'elenco, come formulato da Deming in *Out of the Crisis* (1982), è: (1) mancanza di costanza nello scopo; (2) enfasi sui profitti a breve termine; (3) valutazione delle prestazioni, *merit rating* o revisione annuale; (4) mobilità del management; (5) dirigere un'azienda basandosi solo sui dati visibili; (6) costi medici eccessivi; (7) costi eccessivi di garanzia e contenziosi legali.

5.4.2 L'elicitazione triadica

Un altro modo per analizzare i competitor ha un nome "estremamente complesso, per dire una cosa abbastanza semplice": l'**elicitazione triadica** (*triadic elicitation*).[4]

1. Si individuano i **tre competitor principali**: solo tre, sempre per una questione di compromesso, cioè del tempo da spendere in ricerca, che spesso non basta.
2. Si chiede al target audience di **raggruppare i due più simili**.
3. Si chiede **perché** sono simili e in che cosa sono diversi; che cosa piace di quei due, che cosa non piace.
4. Del gioco rimasto fuori, quello differente, si chiede perché è diverso dagli altri, in che cosa è simile, se è meglio o peggio (in alcuni aspetti sarà meglio, in altri peggio) e perché.

La "piaga" dei soulslike. C'è sempre l'esempio dei soulslike ("non che sia problematico, ma è sempre quello"): che cosa rende questi prodotti dei soulslike? Che cosa li fa rientrare in quel genere? Oppure: qual è quel *core element* che si sta cercando, quell'elemento che poi determinerà anche il proprio prodotto? L'elemento importante è la **precisione**: bisogna cercare di prendere l'elemento veramente **dominante**.

5.4.3 Parlare con l'utenza

È un primo punto: **parlare con l'utente**. Non ci si va senza nulla: ci si premunisce, si procurano prima i dati ed eventualmente già una lista di competitor. Poi si chiede al target audience quali sono i **pain point**, cioè le problematiche che individua, e anche che cosa gli piace, per arrivare potenzialmente all'**insight**: la consapevolezza su ciò che si è ottenuto.

Il rischio di allontanarsi dall'utente. Il metodo aiuta anche a capire come i giochi vengono **percepiti dall'utenza**. Più si fa design, più tempo si dedica a progettare giochi, più c'è il rischio di "sfilarsi" dal ruolo di utente e diventare soltanto creatori di contenuti; a meno di non fare proprio il gioco a cui si gioca ventiquattro ore su ventiquattro. Se il docente dovesse fare un gioco di guida, sarebbe molto lontano da quell'audience: analizzando i giochi da designer, potrebbe leggere una meccanica in un modo che non corrisponde affatto a come la percepisce l'audience. L'unico modo è, ancora una volta, mettersi nei panni dell'audience; e, come detto l'ultima volta, è difficile (sezione 4.9).

5.5 La perceptual mapping

5.5.1 Che cos'è

Un altro metodo, molto utile, è "il cavallo di battaglia" di tanti designer e di tante messe a terra di idee: funziona molto bene sia nel design sia nella spiegazione. È la **perceptual mapping** (il docente non ricorda la traduzione italiana esatta).[5]

Si prendono anche qui le **game qualities** principali dei competitor, o di un mercato in generale su cui ci si sta affacciando, e le si dispone su un grafico a due assi per individuare i **vuoti**: i punti in cui non c'è nessuno, cioè, fondamentalmente, i **vuoti di mercato**.

Gli assi sono spettri. Ogni asse deve essere uno **spettro**: va da un estremo, il più possibile verso 1, all'altro, il più possibile verso 0. Le slide mostrano due esempi lontani dai videogiochi.

Automobili Sugli assi ci sono "conservativo" contro "sportivo" (che, a rigore, non sono proprio uno il contrario dell'altro) e "vistoso, distintivo" contro "pratico, economico", che invece stanno bene o male ai due estremi.

[4] Nella trascrizione il termine compare storpiato ("licizzazione triadica"). L'elicitazione triadica è una tecnica di intervista nata in psicologia (nell'ambito della *repertory grid* di George Kelly) e usata anche nella ricerca di mercato e nella user research.

[5] In italiano si parla di *mappa percettiva* o *mappa di posizionamento*.

Farmaci Sugli assi ci sono l'efficacia e l'impatto sul corpo. Un antibiotico ha un effetto molto importante sul sistema immunitario; un fluidificante ("abbiamo fatto pubblicità occulta") è meno impattante; le caramelle alla menta, le Golia, ancora meno. Se l'obiettivo fosse un farmaco estremamente efficace che però "distrugga completamente il corpo del paziente", quel quadrante è coperto.

Un vuoto non è un invito. Nell'esempio dei farmaci c'è un intero quadrante vuoto: poco efficace e molto "gentile". Probabilmente c'è un motivo se è vuoto. Aver identificato un vuoto non significa doverci "fiondarsi a capofitto": è probabile che non ci sia grande richiesta per qualcosa di molto delicato e poco efficace. Qui alcuni studenti obiettano che in realtà quel mercato esiste, e il docente concede ("non in contesti farmaceutici specifici, però esiste, sì, avete ragione"): è un altro mercato. Si tornerà sul tema con la correlazione dei dati (sezione 5.10).

I due obiettivi. Lo stesso lavoro si fa sui giochi, e gli obiettivi principali per cui si fa una perceptual mapping sono due:

- trovare **combinazioni innovative**;
- trovare **buchi di mercato**.

5.5.2 La perceptual mapping nel design: i nemici

Il docente la usa soprattutto non per fare analisi, ma per fare **design**. Non crede che nel corso ci sarà modo di parlare di *enemy design* (forse nel *core combat*), ma cita *Doom* come caso di un roster di nemici costruito proprio così.[6] L'anno successivo, forse, ci sarà l'*encounter design* con Stefano; ma l'encounter design si basa sul **posizionamento** dei nemici, più che sulla loro tipologia, anche se il comportamento viene in parte toccato. "Apro e chiudo questa parentesi."

Il docente usa il metodo per trovare **combinazioni di nemici** che non sono ancora state fatte, o qualcosa che **spinga il giocatore** nella direzione voluta, lato gameplay. L'esempio:

- su un asse, **DPS** contro **supporto**;
- sull'altro, tanti HP (il **tank**) contro pochi HP: non necessariamente *glass cannon*, suggerisce uno studente, ma *squishy*.

Il glass cannon. Un nemico con tanti danni e pochi HP è un *glass cannon*, un "cannone di vetro": strategie, personaggi o nemici che fanno tantissimo danno ma che, se li si tocca, "saltano proprio per aria". Uno studente propone un esempio da una build con un *charm* (il riferimento sembra a *Hollow Knight*); il docente concede che non è proprio un glass cannon, perché "chiaramente non si può colpire" ("siete dei disonesti"). Esempi più netti sono i **cecchini**, anche in *XCOM*: tutti quei nemici che fanno tanti danni e la cui strategia, generalmente, è difendersi dietro ai tank, che hanno tanta vita ma fanno meno danni: assorbono, ma non colpiscono duro.

I punti ciechi. Mappati i nemici, si vedono i **punti ciechi**: magari manca un nemico di supporto con tanti punti vita. Magari non interessa, magari sì. Generalmente, soprattutto nell'enemy design, interessano gli **estremi**, perché sono quelli che **spingono il giocatore**.

Un grafico non basta. È l'altro punto "importantissimo", da imparare a memoria: **un grafico cartesiano con due assi non basta, ne servono più di uno**. Quello appena fatto non mappa tutti i nemici: il glass cannon potrebbe essere lento o veloce, e allora serve un asse *slow – fast*. In aula se ne aggiungono altri:

- la **grandezza**: interessa relativamente, può essere utile per comunicare al giocatore ma è più un discorso artistico; a meno che il gioco non abbia una meccanica di *shrinking* o non si basi proprio sulle dimensioni, e allora sì;

[6] Il passaggio nella trascrizione è disturbato ("se non è a Ro Doom, è stato fatto proprio così: i nemici di Doom sono stati fatti così"). Si riporta il riferimento a *Doom* senza poter precisare quale capitolo della serie intendesse il docente.

- **melee** contro **ranged** ("bravo").

Il cecchino, glass cannon, sarà ranged. A rigore di logica andrebbe messo su *slow*; ma se ci fosse un altro nemico con tanti DPS e tanti HP (che quindi non è più un glass cannon), probabilmente sarebbe quello ad andare su *slow*, e il cecchino su *fast*, o a metà: andrebbe capito insieme agli altri nemici.

Combinare le categorie. Già così si possono costruire categorie di nemici che, combinate, danno **sfide diverse** al giocatore:

- il cecchino che si mette dietro il tank: la sfida per il giocatore è **aggirare il cecchino** (il docente lo ripete perché Mattia, collegato da remoto, possa seguire meglio);
- il supporto, per cui la dinamica di gioco è bene o male sempre la stessa;
- il classico nemico che fa un sacco di danni e ti corre addosso per farsi saltare in aria, di solito anche lui *squishy*;
- il tank con tanta vita che fa poco danno; oppure il tank con tanta vita, poco danno e molto lento; e così via.

5.5.3 Esercizio in aula: la mappa dei metroidvania

Tornando all'analisi, si prova a farne una con la classe ("tanto il tempo lo abbiamo"). Si vuole fare un "nuovo, incredibile, innovativo gioco". Naturalmente non si parte dal nulla: si restringe il campo, a partire da un **genere**. Il genere scelto è il **metroidvania**.

Gli assi. Il docente chiede alla classe che cosa mettere sugli assi ("probabilmente qualsiasi cosa, veramente"). Non deve per forza esserci correlazione tra un asse e l'altro; ciò che conta è che ciò che si include sia **in comune** a tutti i giochi mappati. Emergono:

- **narrative first** contro **gameplay first** (proposto come "spiccatamente narrativo");
- **high challenge** contro **low challenge**, o accessibilità;
- **prima persona** contro **terza persona**, e **2D** contro **3D**: un'ottima scelta se si dovesse fare ricerca, perché dà già un'idea, oltre che del discorso di gameplay, anche di quello artistico;
- **level design focused** contro **combat focused** ("LD focused, combat focused").

Il docente suggerisce altri assi possibili, che però si escludono dal grafico per non complicarlo troppo: la durata del gameplay; la **progressione**, che può essere basata sul *grinding* o no, e che può essere una progressione di *efficienza* (il giocatore ha sempre le stesse meccaniche ma deve diventare più efficiente nell'usarle) o di *potere* (*power progression*, ci si tornerà con le motivazioni, sezione 5.9). Anche la camera e il controllo emergono, ma "inizia a diventare un po' complesso".

Una convenzione percettiva. Nel disegnare il grafico il docente nota di aver "sbagliato": c'è una convenzione percettiva per cui il valore *alto* si mette in su o a destra. "Ce ne faremo una ragione": nessuno vieta di **numerare** gli assi, di dare delle tacche. I valori vanno trovati in modo approssimativo, ma sempre con un discorso di precisione.

I giochi. Si comincia a riempire, "a spanne" ("chiaramente ci servirebbero dei dati").

- I primi tre *Castlevania* vengono proposti, ma il docente obietta che non sono metroidvania nel senso pieno.[7]

[7] La trascrizione è qui disturbata ("le prime tre Castlevania non sono mica in bagno"); il senso più probabile è che i primi tre *Castlevania* non siano ancora metroidvania, genere che nasce con *Castlevania: Symphony of the Night* e con la serie *Metroid*.

- *Hollow Knight*, posizionato da Federica: challenge abbastanza alta (terza tacca, tra la terza e la quarta); più *gameplay first*, ma con una componente narrativa importante, quindi un po' spostato verso la narrativa. Tra level design e combat, probabilmente un po' più verso il level design.

- *Hollow Knight: Silksong* sarebbe più difficile e più *gameplay first*: aumentano entrambe le dimensioni, ma il gameplay di più, forse anche per la quantità di contenuto rispetto a *Hollow Knight*, e la narrativa è meno comprensibile. Rispetto al primo sarebbe spostato verso il combat.

- *Castlevania: Aria of Sorrow*, secondo il docente, avrebbe una challenge forse un po' più bassa di *Hollow Knight*.

- *Metroid Dread*, più orientato allo *stealth*, resta 2D come la gran parte dei titoli.[8]

- La saga di *Metroid Prime* va in prima persona; tutti gli altri in terza.

- Un titolo "sistemico" citato la volta precedente, con mira in prima persona e movimento in terza, andrebbe più verso la terza persona.[9] Questi titoli avrebbero probabilmente una challenge più bassa degli altri: il primo *Metroid* sta in un punto, e *Metroid: Other M* in un altro, forse più in là.

- Gli studenti aggiungono altri titoli: *Iconoclasts* ("bello"), *Gato Roboto*, *Animal Well* e alcuni altri.[10]

Il docente lascia la mappa alla lavagna: chi vuole può aggiungere altri giochi dalla lista "non inclusiva" di metroidvania, e qualcuno è invitato a fotografarla e a mandarla anche a Mattia, collegato da remoto, che così la può vedere meglio.

Che cosa mostra la mappa. Qualcuno in aula nota che, guardando la mappa, si vede anche **il genere che si preferisce**: tutti i giochi più *combat focused*, dice, li ha abbandonati. Il docente risponde che sulla base di ciò che si inserisce e dei valori che gli si danno si riaggiusta tutto il resto: su quella categoria, un titolo molto spinto sul combattimento starebbe all'estremo *combat focused*, e un altro all'opposto.[11] Si vede anche come **gli elementi del mercato cambiano il valore**: se si inserisce *Silksong*, *Hollow Knight* va riposizionato rispetto a lui.

Assi polarizzanti. I giochi risultano molto polarizzati, anche perché alcuni assi scelti sono di per sé polarizzanti, come 2D e 3D: un gioco in 2.5D come *Dread* resterebbe comunque 2D. Si potrebbero definire meglio le categorie: se il livello è a scorrimento orizzontale ma con elementi in 3D, forse lo si potrebbe mettere a metà, ma diventa più complesso. E già si vede che le zone sono quasi tutte popolate: ne sono stati messi pochi; bisognerebbe inserirne un **grande numero** e cercare dove ci sono i buchi, o combinazioni mai pensate. "Più ne facciamo, più ne troviamo."

Non solo gameplay. Gli assi possono essere anche artistici o narrativi: lato artistico si potrebbe mettere il **realismo** da una parte e un'arte più **stilizzata** dall'altra. Al corso interessa più il lato gameplay, ma la perceptual mapping è un ottimo modo sia per fare **ricerca** sia, potenzialmente, per fare **brainstorming**: trovare le categorie che si vorrebbero avere e il modo di mescolarle per fare qualcosa che non è mai stato fatto, o che si pensa non sia mai stato fatto (si veda la tecnica dell'*opposite direction*, sezione 5.15).

[8] Nello stesso punto la trascrizione cita, come alternative 2D, titoli non riconoscibili ("Sintra, L'Enzion Mission, Tournus"); non si azzarda un'ipotesi.

[9] Il nome è illeggibile nella trascrizione ("Varen"). La descrizione (mira in prima persona, movimento in terza) corrisponde a *Metroid: Other M*, che però compare poco dopo con un nome a sé ("Oderamma"); il riferimento resta quindi incerto.

[10] Altri nomi proposti dagli studenti non sono riconoscibili nella trascrizione ("Rainbow Dash", "Pencil", "Mansions"); compare anche *Firewatch*, citato come esempio di gioco in prima persona e 3D più che come metroidvania.

[11] I due titoli sono storpiati ("la InSource" e "il regolamento"). Il primo potrebbe essere *Nine Sols*, metroidvania fortemente centrato sul combattimento, ma non è verificabile.

5.6 Altre fonti di dati

Un altro modo è andare a cercare dati effettivi su piattaforme specializzate. Il docente ne mostra una, limitata a Steam ma non l'unica, che permette di vedere che cosa viene giocato e che cosa è di tendenza. Va capito che cosa determina un trend: quali sono le categorie, quali i fattori. Ci sono i **tag di Steam**, che spesso non si riescono a vedere o si fa fatica a vedere, e i **dati di vendita**, ancora più difficili da ottenere.

Scorrendo la pagina, il docente commenta qualche titolo: un gioco che non ha ancora giocato e che i suoi amici vogliono giocare, e la classifica "best of 2026", in cui c'è un titolo in versione 1.0 che "non avrei mai detto che sarebbe finito qui dentro".[12] I giochi si possono dividere per generi, oppure per gratuità, "con dei grossi asterischi": *Life is Strange* compare tra i gratuiti, ma "è una trappola", perché gratuito è solo il primo episodio. Ci sono anche categorie "che forse è meglio non vedere".

5.7 L'analisi degli utenti

5.7.1 Uno dei punti più difficili

Si passa al punto quattro della mappa, l'**analisi degli utenti**, già introdotta la volta precedente (sezione 4.9). Per il docente è uno dei punti più difficili, certamente più dell'analisi dei competitor, che diventerà progressivamente più facile con l'aumentare degli strumenti a disposizione. L'analisi degli utenti richiede tanto lavoro di ricerca e di domanda: creare un *survey* apposito, con le domande giuste, o anche solo chiedere feedback, richiede lavoro **prima**, per decidere che cosa chiedere, e **dopo**, per prendere tutta la marea di cose che sono state dette, segnarle, spacchettarle e capirle.

5.7.2 Che cosa interessa sapere

I dati quantitativi. Degli utenti interessano anzitutto i dati **quantitativi**: l'età, il genere (o ciò in cui la persona si identifica), il lavoro, se ce l'hanno o no. Il lavoro aiuta a capire **quanto tempo hanno a disposizione** e quanta "potenza di fuoco" economica. Se si vuole fare un gioco con numerose microtransazioni, magari si deve fare un gioco *mobile* in cui la *stamina* si paga; e se ci si vuole rivolgere a un target che lavora, conviene prediligere **partite di durata più bassa**, ma la cui frequenza magari si paga, come in *Candy Crush*. Poi lo **status** e qualsiasi altro elemento utile a descrivere ad alto livello chi è questo individuo: "queste sarebbero le *game qualities* di una persona".

Gusti e influenze. Ci sono poi questioni più larghe. Età, sesso e lavoro si descrivono in una parola o due; qui diventa più complesso. Che cosa piace e che cosa non piace, non necessariamente in relazione al proprio gioco: il tipo di musica che ascolta, il tipo di film che guarda, il rapporto con anime e manga, che spesso sono una grande influenza sui videogiochi. Se si prende uno stile specifico e lo si trasporta nel mondo videoludico, può funzionare oppure no. L'esempio è quello fatto il lunedì precedente: un gioco, di cui il docente non ricorda già più il nome, che prende a piene mani lo stile dello **Studio Trigger** e lo riporta in ambito videoludico.[13] Chi ama quello stile ci va a nozze; altri un po' meno.

Che cosa fanno, e come: la routine. Insieme a questo, è importante capire la **routine**: come spendono la giornata. Sempre ad altissimo livello: si sta cercando di delineare il target audience ideale del proprio gioco. Quanto tempo hanno a disposizione; se giocano con altre persone o da soli; se giocano in famiglia (una madre che gioca con i figli, per esempio); quanto tempo dedicano al lavoro e quanto spazio resta per giocare.

[12] Il titolo è storpiato nella trascrizione ("Power 1.0"); potrebbe trattarsi di *Palworld*, ma non è verificabile. Anche il titolo citato subito dopo non è ricostruibile.

[13] Il titolo non è ricostruibile dalla trascrizione ("il gioco di Destiny Beat"), e la lezione del lunedì a cui il docente fa riferimento non è tra quelle trascritte nei capitoli precedenti.

5.7.3 Il caso Riot e League of Legends

Una partita di *League of Legends* dura in media, secondo il docente, circa un'ora.[14] Se generalmente se ne fa più di una ("non io: ho smesso con quella roba, con i tossici"), ci si aspetta che il target audience ideale di quel gioco abbia **ampio margine di tempo** da dedicargli.

Stessa IP, audience diversi. *League of Legends* esiste anche per telefono, ma è un altro gioco: non è un *porting* uno a uno della partita.[15] Sono due audience diversi. È un grande vantaggio, ed è il lavoro che ha fatto Riot stessa (oggi sotto Tencent): prendere lo **zoccolo duro** del target audience di *League of Legends*, che è molto variegato, e offrirgli esperienze diverse. La versione mobile ha una durata minore, per le persone fidelizzate che vogliono quell'esperienza da MOBA anche mentre aspettano l'autobus.

Gli altri giochi di Riot. Il docente passa in rassegna gli altri titoli di Riot, restando nel mondo di *LoL* (quindi non *Valorant*):

- un **picchiaduro**, con cui Riot ha cercato, secondo il docente senza successo, di convertire quell'utenza al genere dei picchiaduro.[16] Il margine era molto grande, perché le abilità di un giocatore di *League of Legends* sono molto lontane da quelle richieste normalmente in *Street Fighter*, *Tekken*, *Skullgirls* e negli altri picchiaduro tecnici; in più *LoL* si gioca in team, mentre i picchiaduro sono generalmente uno contro uno;
- l'**auto battler**, ancora più mobile, accessibile e *casual* (*Teamfight Tactics*);
- il **gioco di carte** di *LoL* (*Legends of Runeterra*; "adesso non so in che stato è");
- esperienze più **narrative**, come *Song of Nunu*, una *puzzle adventure* ("un'avventura platform 3D, mettiamola così") con i protagonisti di *LoL*. Prende un altro target, ma si assicura che una parte dell'utenza di base di *League of Legends* finisca lì dentro.

Riot può permettersi di spostare persone perché nel tempo quel mondo si è gonfiato moltissimo. Il docente stesso, quando giocava a *League of Legends*, era più interessato alla *lore* del mondo che al MOBA in sé: gli interessava leggere le storie e sentire i dialoghi tra i personaggi, su cui a un certo punto Riot ha investito molto.

I due centesimi su Arcane. Qui il docente si concede una parentesi su *Arcane*. Premette che sarà "blasfemia", ma lo stile artistico "alla *Dishonored*" non gli piace, e forse non è l'unico al mondo.[17] Gli è piaciuto, nella sua testa, fare tutti i collegamenti tra la lore della serie e quella del gioco: a volte sono uno a uno, a volte si allineano, a volte no (Warwick, per esempio, "non c'entra nulla"). Il suo problema è che nella seconda stagione sia stato "ficcato a forza" qualsiasi personaggio del mondo di *LoL*, quando non era necessario; e anche il fatto di rendere esplicita la relazione tra Jinx e Vi, che nel gioco ("forse adesso lo è") non era mai stata dichiarata: c'era qualche tipo di legame, ma non si era mai detto che fossero sorelle, come per Caitlyn. La prima parte ci può stare; la seconda è caricata di troppi personaggi, e a lui ha pesato, perché il valore artistico poteva essere più elevato. Non è *fan service*: l'obiettivo finale è la **trasmedialità** del prodotto, renderlo il più collegato possibile ad altri elementi, ed è proprio scrittura transmediale, che si insegna. Non è un problema di per sé: gli dà fastidio come utente, ma è "uno su tutte le persone" che gli hanno dato cinque stelle (o una). "Questi sono i miei due centesimi."

[14] La durata media di una partita classificata di *League of Legends* è in genere più vicina alla mezz'ora; si riporta la stima del docente, che non cambia il senso dell'esempio.

[15] Il titolo mobile è *League of Legends: Wild Rift*.

[16] Il titolo non è nominato in aula; si tratta quasi certamente di *2XKO*. Si riporta il giudizio del docente sul suo esito così come è stato espresso.

[17] Nella trascrizione i due nomi si sovrappongono ("Arcane" e "Arkane"): *Arcane* è la serie animata di Riot prodotta dallo studio Fortiche; Arkane Studios è lo sviluppatore di *Dishonored*, il cui stile pittorico viene spesso accostato a quello della serie.

Spostare il target. Il senso del discorso sul target è questo: se si ha già un target di base, come Riot, e lo si ha molto frammentato, si può pensare di **raggruppare e spostare** alcuni di quegli individui verso altri giochi, altri prodotti, "altri lidi". Il modo più conveniente per farlo, come si vedrà parlando di grafici, è il **clustering di dati**; ma non lo fa il singolo. È il lavoro del *data analyst*, e servono una mole enorme di dati e un algoritmo per separarli in modo sensato. Se ne riparlerà, sfiorando l'argomento, perché non è comune che lo facciano gli studenti, e non è comune nemmeno che le aziende medio-piccole abbiano un sistema del genere.

5.7.4 Chi gioca, chi giocherà

Tornando alle slide: serve capire **chi gioca** il gioco, ma non solo: chi lo giocherà, chi *potrebbe* giocarlo, e in che spettro. Ci sarà il **target audience dominante**, ma potrebbero esserci altre figure, altre demografiche che gli ruotano attorno, potenzialmente interessate e che magari si ha interesse a **convertire** in futuro: con i **DLC**, come detto l'ultima volta, che sono già un modo per inserire meccaniche che potrebbero piacere ad altri individui, o con gli **spin-off**, come appena visto.

5.7.5 Il feedback online e l'ombra digitale

Quali sono i loro feedback? Lo si è già visto per i competitor: se il proprio gioco non esiste ancora, si va a vedere come reagisce il pubblico ai giochi simili. Col passare del tempo diventa sempre più semplice: le reazioni sotto gli streamer, i commenti su YouTube, Reddit.

Un grosso asterisco. Ciò che si vede online, l'**ombra digitale** (*digital shadow*), non compone il 100% dell'utenza. **Chi è vocale online generalmente è polarizzato**: o ama alla follia un prodotto o lo odia. Il docente non commenta sotto i video di YouTube, ma se dovesse farlo andrebbe sotto il trailer di un nuovo *Metroid* a dire "figata, proprio quello che volevo"; altri andrebbero sotto lo stesso video a chiedere "fatemi una narrativa comprensibile".[18] Sono due poli:

- da una parte le persone che compreranno sicuramente il gioco, molto in target, e quelle che lo comprano e dicono agli altri di giocarlo: i cosiddetti **evangelisti**. Come un ex collega del docente che continua a dirgli "compra *Valheim*, giochiamo a *Valheim*": un evangelista, che cerca di convertirlo "con scarsi risultati";
- dall'altra le persone che il gioco non lo toccano, ed è giusto che non lo facciano, perché **nessun gioco è per tutti**, nessun prodotto è per tutti.

Minecraft è per tutti? Uno studente obietta: *Minecraft* non è per tutti? Ha una base enorme, ma no, risponde il docente: come *Fortnite*, non è per tutti. *Minecraft* è spintissimo verso l'esplorazione e la comunità. Lo studente cita le *Bed Wars*, che trasformano il gioco in un PvP; il docente concede che lì c'è tanto, non solo *live ops* ma molto materiale della comunità: in *Minecraft* si può, e si fa, moddare il gioco, mentre in *Fortnite* l'impatto che un giocatore può avere creando contenuto è molto più ridotto. Resta il fatto che **nel fulcro** *Minecraft* è mirato all'esplorazione e alla socialità, mentre *Fortnite* alla socialità e alla competitività (lo si rivedrà con le motivazioni, sezione 5.9). E restano la stragrande maggioranza delle meccaniche giocate e percepite dai giocatori, e quelle con cui il gioco si presenta nella prima o nelle prime due ore: quelle sono, e quelle rimangono.

5.7.6 Interviste, focus group e survey

Identificato il target audience, in determinati punti del processo di produzione ma anche prima, lo si **intervista**. Una volta fatta la ricerca, gli si chiede, come si diceva l'ultima volta: "stiamo facendo un soulslike con i pony; ti piacerebbe? Perché no?" (sezione 4.9). Serve come **cartina tornasole**: le risposte del target audience confermano o smentiscono ciò che si è pensato prima di metterlo a terra. Si chiedono opinioni: perché una cosa piace, perché non piace.

[18] Il passaggio è storpiato ("sotto il video di Ravenu [...] Samus che fa la merenda grande"); il senso, un commento entusiasta a un nuovo capitolo di *Metroid*, è chiaro, il dettaglio no.

I focus group. Se si hanno i soldi si fanno **focus group**: molto contenuti e omogenei, su diversi tipi di target audience, oppure misti, con diversi tipi di utenti o demografiche. Poi i dati vanno analizzati.

Survey pagati e survey gratuiti. Trovare le persone costa: chi riceve un survey di questo tipo di solito viene pagato, poco ma pagato. Ci sono piattaforme apposite, come *SurveyMonkey*.[19] Il vantaggio è doppio: si ha la certezza che il survey arrivi a certe persone e, utilissimo per un designer, che queste **lo finiscano**, perché sono pagate per farlo e hanno un incentivo molto elevato.

Se invece si è "poveri" (il docente si paragona all'immagine di un meme, forse Mr. Krabs, che non riesce a ritrovare), si fa il survey e lo si "spamma" il più possibile in giro. Da una parte si ottengono grandi entrate di dati, che poi comunque vanno smazzate nell'analisi; dall'altra il survey finisce davvero ovunque, e le persone che lo riempiono sono molto diverse tra loro.

Le domande di filtro. È un altro punto problematico, che il docente aggiungerà alla lezione sui feedback. Con un survey diffuso così, una parte del contenuto viene per forza "mangiata" dalle domande che servono a separare i rispondenti: l'età, che tipo di giochi piace giocare, come si spende il tempo, qual è il gioco preferito, e così via. Se si sta facendo un metroidvania e la risposta alla domanda sul gioco preferito è "non so, *The Sims*", forse la persona non è proprio in target (per semplicità, si assume di no). Per capirlo servono tante domande; ma **più domande ci sono, meno la persona sarà incentivata a concludere** il survey.

La regola del pollice. Il survey **non deve durare più di cinque minuti**, e bisogna "truffare" le persone dicendo: "questo survey dura meno di cinque minuti, tre minuti per concluderlo, ti prego, in ginocchio, completalo". Se lo si paga, la persona può anche completarlo con calma; altrimenti non lo farà.

Risposte a caso e outlier. Il docente stesso, quando compila un survey, alcune risposte le dà a caso; altre "come per dire"; a volte, quasi alla fine, lo interrompe e lo riprende dopo. Le singole risposte perdono significato e diventano *outlier*. Le soluzioni sono due:

- avere davvero una marea enorme di entrate, **molto superiore al centinaio** (sezione 4.6);
- saper fare i survey, perché "c'è un'arte": più le domande sono semplici, limitate nelle risposte, puntuali, meno probabilità c'è che si risponda a caso. Si legge la domanda, la si capisce subito, si leggono le risposte, A, B, C, D, e si va dritti.

La tendenza si può ridurre, ma non ignorare del tutto, a meno di stare dietro alla persona a guardare che cosa compila. Si può fare, per esempio alla Games Week o a un'altra fiera[20]: si fanno le domande del survey stando "sulla spalla" e ci si assicura che lo si compili. Ma anche così si modifica il modo in cui la persona risponde, che potrebbe sentirsi indispettita: "a ognuno il suo".

Approfondimento: i consigli di Schell sui survey. Nel capitolo sul playtesting di *The Art of Game Design*, Schell elenca alcuni consigli che si sposano con quelli del docente: usare immagini quando si chiede di elementi o scene del gioco; usare strumenti online come SurveyMonkey o Google Forms, gratuiti o economici; evitare le scale da 1 a 10 e preferire **scale a cinque punti** con ogni punto etichettato (da "pessimo" a "eccellente"), che danno risultati più coerenti; **non mettere troppe domande**, perché verso la fine le persone si distraggono e i risultati perdono valore; somministrare il survey subito dopo la sessione di gioco; avere qualcuno a disposizione per i chiarimenti; annotare età e genere di ciascun rispondente; e soprattutto **non prendere i dati del survey come vangelo**, perché difficilmente un survey è davvero scientifico e chi risponde tende a inventare quando non è sicuro. Per le domande troppo complesse per un modulo, Schell consiglia l'intervista, possibilmente privata e con una traccia di domande pronta.

[19] Il docente aggiunge "di Amazon, se non erro". *SurveyMonkey* è una piattaforma indipendente; il servizio di Amazon per reclutare partecipanti pagati a compiti e questionari è *Amazon Mechanical Turk*. È probabile che i due siano stati sovrapposti.

[20] Nella trascrizione il secondo nome è storpiato ("Fortale", "Force Playable"); non è ricostruibile.

5.7.7 Archetipi e osservazione

"Questo discorso di fare survey è tosto", e l'analisi dell'utenza è uno dei punti più difficili: non se ne scappa. L'unico modo è prendere più dati possibile e dedicare tanto tempo a smistarli. L'altro è creare il più possibile **gruppi** o **archetipi**: approssimazioni di una serie di utenti che però "salvano la vita" nella produzione, perché come sempre si hanno tempo, soldi ed energie limitati. Se ne vedranno alcuni: uno molto noto a breve, il più noto prima o dopo dicembre, a seconda del tempo.

Guardateli giocare. Come già detto: si guardano interagire i giocatori e, se possibile, li si **registra**, perché sono dati che poi non vengono persi (sezione 4.10).

Approfondimento: nessuno aveva mai fatto un vero survey. Nell'introduzione di *21st Century Game Design* (il libro di Bateman e Boon) si racconta che l'industria ha prodotto per trent'anni giochi fatti da giovani uomini occidentali per altri giovani uomini occidentali, saturando quel mercato e lasciando scoperte altre demografiche. E perfino quel mercato, si nota, non era davvero compreso: non tutti i giovani uomini sono uguali, preferiscono sfide diverse, hanno quantità diverse di denaro e di tempo. Ernest W. Adams, autore dell'introduzione, scrive che in otto anni in Electronic Arts non ha mai visto una ricerca di mercato davvero approfondita e condotta come si deve: la comprensione dei giocatori si basava su supposizioni e intuizioni. È il problema che l'analisi degli utenti cerca di risolvere, ed è il punto di partenza del loro modello demografico (sezione 2.13).

5.7.8 La mappa dell'empatia

Un altro modo di fare analisi degli utenti riguarda anche il **tono di voce**: gli studenti lo vedranno, probabilmente, anche con Daniele. L'obiettivo finale è sempre avere questo legame: **mettersi nei panni dell'utente**. Per organizzare i dati rispetto a un certo target audience o archetipo di utente, si inserisce tutto ciò che si è raccolto in **quattro componenti** (tabella 5.1).[21]

Che cosa dicono Le citazioni, le cose dette esplicitamente.

Che cosa fanno Le azioni. Si può dire "sono il più grande giocatore horror dell'universo", ma se davanti ad *Amnesia* si gioca con le mani davanti agli occhi e la luce accesa, quello che si fa dice una cosa diversa.

Che cosa pensano Le motivazioni, gli obiettivi. Si trovano anche con la ricerca, ma poi li si vede confermati: il ceto sociale di una persona, come si porta, lo status, il lavoro possono cozzare con ciò che pensa, con i suoi ideali, e così via.

Che cosa provano Le emozioni, *felt*: la lista di emozioni già vista (sezione 2.12). Qui si va nello specifico, nel tecnico: quanto più ci si indirizza già verso un modello o un framework, tanto più diventa utile, perché poi questi dati si riportano altrove.

Table 5.1: Le quattro componenti per organizzare i dati su un utente o un archetipo.

Componente	**Che cosa raccoglie**	**Esempio dalla lezione**
Dicono (*says*)	citazioni, affermazioni esplicite	"sono il più grande giocatore horror"
Fanno (*does*)	azioni, comportamenti osservati	gioca ad *Amnesia* con le mani sugli occhi
Pensano (*thinks*)	motivazioni, obiettivi, ideali	status e lavoro che cozzano con gli ideali
Provano (*feels*)	emozioni	la lista di emozioni del corso

[21] Lo strumento è comunemente noto come *empathy map* (mappa dell'empatia), con i quadranti *says*, *does*, *thinks* e *feels*; il nome non viene pronunciato in aula, ma le quattro componenti coincidono.

Senza empatia non c'è design. L'analisi degli utenti è incredibilmente importante perché dà i dati necessari per **empatizzare** con l'audience. Non significa doverne vestire sempre i panni; ma in certi momenti di discussione e di design è utile avere questa "maschera" da indossare (sezione 4.9). **Senza empatia non c'è design**: l'empatia è uno dei punti di arrivo.

Approfondimento: la lente del giocatore. Schell formalizza lo stesso atteggiamento nella *Lens of the Player* (lente n. 19): smettere di pensare al proprio gioco e cominciare a pensare al proprio giocatore. Le domande sono: in generale, che cosa gli piace? Che cosa non gli piace, e perché? Che cosa si aspetta di vedere in un gioco? Se fossi al suo posto, che cosa vorrei vedere? Che cosa gli piacerebbe, o non gli piacerebbe, del mio gioco in particolare? Un buon designer, scrive Schell, dovrebbe essere sempre un *avvocato del giocatore*.

5.8 L'analisi di mercato

5.8.1 C'è spazio per il mio gioco?

L'analisi di mercato estende ciò che si è visto con la perceptual mapping e con il discorso sull'audience (sezione 4.9). "Mercato" può voler dire un continente, un paese, un mercato saturo o non saturo. La domanda è: **esiste spazio per il mio gioco in questo momento storico?** Quanto è grande il mercato? È una **nicchia**? Come detto l'ultima volta, i giochi alla *Papers, Please* sono una nicchia; un altro genere, ormai, è un mercato molto ampio ma anche molto saturo.[22]

I giochi brevi. Uno studente fa notare che il mercato dei giochi simili a *Papers, Please* non è saturo: ogni volta che ne esce uno, sono giochi piccoli, la gente se li gioca, e poi non ce n'è più. Il docente apprezza: è un discorso molto interessante. Molti di questi giochi, come anche molti giochi mobile, hanno una vita o una durata molto ridotta, e permettono all'utente di provarne e finirne molti in poco tempo; cosa che si fa meno con i AAA e i AA, soprattutto con gli open world.

Monopolizzare l'utente. Quando si presenta un gioco AAA, la **longevità** del prodotto deve essere un punto di forza. Ci sono giochi che puntano il più possibile a **monopolizzare l'utente**: se un open world ha cento livelli, mille espansioni, "duecento milioni di biomi", il giocatore che gioca a quello non gioca all'open world B. Vale anche per gli *hero shooter*: chi è fidelizzato su *Valorant* o su *Counter-Strike* tendenzialmente non andrà a giocare a *Overwatch*. È anche uno dei motivi, oltre a una serie di altri, per cui *Concord* ha fallito.

5.8.2 Seguire i trend

È discutibile se seguire i trend sia un modo corretto di procedere, anche perché **i trend si seguono sempre in ritardo**: anticiparli è difficilissimo, e fanno fatica anche le aziende AAA.

I casi del pubblico dominio. Un caso in cui il trend era prevedibile è quello delle opere che diventano libere da diritti. È successo, in ambito cinematografico, con lo *Steamboat Willie* originale (non Topolino in generale, ma quella prima versione) e con *Winnie the Pooh*: quando sono scaduti i diritti d'autore, chiunque ha potuto usarli in qualsiasi prodotto.[23] È successo anche con Lovecraft e Cthulhu: c'è stata una valanga di giochi, titoli, scritti e film lovecraftiani. Molti prodotti hanno fatto la produzione prima e poi, alla scadenza, hanno rilasciato subito, tutti insieme o comunque nel tempo: c'è stata una bella finestra temporale in cui molti hanno usato temi cthulhiani.

[22] Il genere è storpiato nella trascrizione ("Solo Stripe"); dal seguito della lezione potrebbe trattarsi dei soulslike, ma non è certo.

[23] Il docente dice che queste opere sono "entrate in Creative Commons"; più precisamente sono entrate nel **pubblico dominio**, per scadenza del diritto d'autore (negli Stati Uniti nel 2022 per il primo libro di *Winnie the Pooh* e nel 2024 per *Steamboat Willie*). Le licenze Creative Commons sono invece concesse volontariamente dagli autori.

È difficile prevedere il mercato. Probabilmente Diego, il direttore del corso, direbbe che una certa grande azienda è brava a farlo; è vero, ma anche lei di recente ha fatto "dei grossi inciampi".[24] Il punto è che fare tanta ricerca, e farla bene, può assicurare una buona fetta di mercato e in generale la **sopravvivenza dell'azienda**; farla male può causare il contrario, il fallimento. E spesso il fallimento arriva anche con ricerche mediocri, o perfino positive, per una serie di altre questioni, prevedibili o meno, come gli andamenti del mercato, che sono poco prevedibili. Bisogna dedicarci tempo e attenzione.

5.8.3 Lo stato emotivo del mercato

Qual è lo stato "emotivo" del mercato, "mettiamola così"? Gli utenti sono contenti di ciò che c'è?

- Il mercato dei **soulslike** non solo è saturo, ma è saturo di titoli che la gente gioca con **moderato apprezzamento**. C'è sempre la "grande torta cicciotta" di FromSoftware, che non ne sbaglia quasi una, e poi una serie di "cupcake piccolini", tutti AA, "tripla I" o indie, con questo tipo di meccaniche.

- Il mercato dei **monster collectible** è il contrario: non è saturo e, in più, gli utenti sono scontenti.

5.8.4 Barriere, tasse e logistica

Ci sono barriere? Sono quelle viste l'ultima volta (sezione 4.9): si può o non si può fare qualcosa in quel mercato? Conviene pensarci prima, ed eventualmente decidere **consapevolmente** se affacciarsi o no.

- Se il gioco è completamente basato sulle **loot box**, si potrebbe non poterlo lanciare in **Belgio**: *Overwatch* ha tolto quel sistema in quel paese, e molti giochi mobile in Belgio non escono proprio per questa ragione.

- Ci sono **tasse**, regolamenti, regole. *Kickstarter*, per esempio, trattiene una percentuale sul titolo che si sta sviluppando, e quella percentuale va considerata.

- Se si fa un **gioco fisico**, bisogna considerare non solo i costi di produzione, ma il numero di copie, quante ne vanno vendute per avere un ritorno, e dove vanno stoccate: "non è che le mettete nel giardino di casa", serve un **magazzino**. E poi chi si occupa dei trasporti, chi dell'imballaggio, e così via.

Il docente valuta se fare la pausa subito o dopo, perché la parte successiva è "molto importante", e decide per subito: cinque minuti di pausa.[25]

5.9 L'analisi di gameplay: le motivazioni di Quantic Foundry

5.9.1 Un modo, non l'unico

Dopo la pausa si passa all'**analisi di gameplay**. Il docente precisa che quella che presenta non è l'unica: è uno dei modi possibili, ed è importante, perché ritornerà in futuro. Segnala anche una serie di talk che si possono cominciare a vedere, anche se non sa se consigliarli caldamente.

Il Dragon Speech. Prima, un'altra raccomandazione che gli è tornata in mente: il **Dragon Speech** di Chris Crawford. "Dovete andare a vederlo, non lo ripeto più. È importante."[26]

[24] Il nome dell'azienda non è ricostruibile dalla trascrizione ("Without Bordecost", "sotto Asgore").
[25] Durante la pausa la trascrizione automatica registra solo frasi ripetute in loop, prive di contenuto, che qui si omettono.
[26] Si tratta del celebre intervento di Chris Crawford alla Computer Game Developers Conference del 1992, in cui il designer annunciava il proprio abbandono dell'industria paragonando la propria ricerca a quella di un drago.

5.9.2 Quantic Foundry

Il docente apre il sito di **Quantic Foundry** e nota che è stato rifatto: ora è "vestito" in modo più carino, accessibile, giocoso e saturo di colori; a lui piaceva di più quello vecchio, in bianco e nero, "preciso, elegante", più stoico, "grezzo e duro, spartano".

Quantic Foundry ha raccolto dati da un numero enorme di giocatori (il docente parla di circa due milioni), e i suoi fondatori, uno dei quali è un *data analyst*, hanno cercato di correlarli per arrivare a una serie di risultati interessanti.[27] Alcuni li si vedrà più a fondo, come gli **archetipi di giocatori** di Quantic Foundry, che sono una cosa solo loro. Ciò che interessa ora è uno **spettro**: categorizzare le necessità e le motivazioni dell'utente secondo le emozioni della lista già vista (sezione 2.12).

I limiti dei dati. I dati sono **chiusi**: non possono essere visti se non da soci associati. E sono limitati ai **giocatori maggiorenni**: c'è sempre il limite del non fare ricerca statistica su minorenni, o comunque su giocatori molto piccoli. La "personalissima teoria del complotto" del docente (da raccontare per far ridere e dimenticare subito dopo) è che Yee abbia sempre avuto il pallino di estendere la ricerca anche ai più piccoli; ma servirebbe il consenso dei genitori, e le regolamentazioni cambiano da stato a stato: "è un casino". Il docente non ha idea, per esempio, di come abbiano gestito i dati raccolti in Russia rispetto a quelli in America, dove la soglia della maggiore età è diversa.

Il talk. Il GDC Talk su queste **dodici motivazioni** si trova sul sito della GDC e anche su YouTube (cercando "Quantic Foundry talk GDC"). Ce ne sono altri degli stessi autori; il docente consiglia solo il primo, se proprio lo si vuole vedere, ma se ne riparlerà comunque.

5.9.3 Sei gruppi, dodici motivazioni

Le motivazioni sono raggruppate a due a due in sei categorie, che per ora il docente chiama "emozioni". Ciascuna ha un corrispettivo nella lista di emozioni del corso (sezione 2.12):

Action corrisponde all'*excitement*.

Social corrisponde a *socialization*, *fellowship*, *community*, "chiamatela come volete".

Mastery è la padronanza, la *challenge*; ma è proprio lo sviluppo delle skill legate all'azione del giocatore, quello visto nel framework di Bura (sezione 2.9).

Achievement corrisponde all'*accomplishment*: "è la stessa roba".

Immersion corrisponde alla *fantasy*, ed è anche *vicarious experience* quando l'esperienza è molto lontana dalla nostra: non il pilota di corse, ma il cavalcatore di draghi.

Creativity corrisponde alla *creation*: poter giocare con le proprie idee, dare sfogo alla propria immaginazione tramite gli strumenti del gioco, come creare il Taj Mahal in *Minecraft*.

Ciascuna categoria contiene **due motivazioni**, e ciascuna motivazione è **uno spettro**: non è che un gioco ha o non ha la community; si colloca su uno spettro (tabella 5.2).

5.9.4 Community

Il primo spettro è la **community**: condividere esperienze. Da una parte c'è tutto ciò che è **collaborativo**: non solo incontrare altri esseri umani, ma interagirci insieme. Dall'altra il gioco completamente **single player**, in cui il giocatore è in controllo della propria esperienza: cooperazione da una parte, **indipendenza** dall'altra.

Una parentesi: la community fuori dal gioco. Un gioco completamente single player può avere zero community *dentro* il gioco. Ma esiste la community di *Hollow Knight* fuori dal gioco: prima si parlava del *meta* delle build di *Hollow Knight*, una cosa percepibile solo affacciandosi fuori. Se ne riparlerà.

[27] I fondatori di Quantic Foundry sono Nick Yee e Nicolas Ducheneaut; nella trascrizione il primo compare come "Nick Lie".

Table 5.2: Il modello delle motivazioni di Quantic Foundry, con i poli e gli esempi discussi in aula.

Gruppo	**Motivazione**	**Polo alto**	**Polo basso**
Social	Community	cooperazione: *WoW*, *LoL*, *Destiny*, *Rainbow Six*	indipendenza: giochi LEGO, *Hatoful Boyfriend*
	Competition	alto conflitto: *LoL*, *Dota 2*, *CoD*, *CS:GO*, *For Honor*	basso conflitto: *Firewatch*, *Myst*, *Gone Home*
Action	Excitement	*thrilling*: *CS:GO*, *Ready or Not*, *Bodycam*, *Smash Bros.*	rilassato: *Pokémon*, *Civilization*, *Animal Crossing*
	Destruction	caos: esplosioni, armi, *Doom*, muri che crollano	calma: *Professor Layton*, *Harvest Moon*
Achievement	Completion	*task oriented*: JRPG, *The Witcher*, *Zelda*	*self-driven*: *Minecraft*, *Kerbal Space Program*
	Power	progressione di potere: *Dark Souls*, *Diablo 3*, *Borderlands*	efficienza: *Mario Odyssey*, *Night in the Woods*, *Ico*
Mastery	Strategy	contemplativo: *Europa Universalis*, *Balatro*	spontaneo: *Candy Crush*, match-3, *Mario Kart*
	Challenge	*skill based*: *Dark Souls*, *Hollow Knight*, *Ori*	*easy fun*: *Oxenfree*, *Animal Crossing*
Immersion	Fantasy	lore profonda: *Mass Effect*, *Dark Souls*, *Fallout*	setting generico: *Street Fighter*, *Candy Crush*
	Story	dramma: Telltale, BioWare, *Persona*	*open ended*: *Factorio*, *SimCity*
Creativity	Discovery	esplorazione: *Skyrim*, *Minecraft*, *Subnautica*	pratico: *FIFA*, *Madden*, scacchi
	Design	*self-expression*: personalizzazione del personaggio	personaggio fisso: *Braid*, *Spelunky*

Gli esempi. Giochi con il *teamwork* alle stelle: qualsiasi *Warcraft*, potenzialmente anche *League of Legends*, *Destiny*, *Rainbow Six*. Giochi single player: i giochi LEGO, di solito, e *Hatoful Boyfriend*, il *dating sim* con i piccioni ("incredibile che ci sia"). A metà ci sta *Stardew Valley*: non è un'esperienza solo single player, ha la possibilità di cooperativa, ma la coop non è così spiccata e i suoi sistemi non sono così spinti in quella direzione.

Stardew Valley contro Animal Crossing. Uno studente propone *Animal Crossing* come gioco con un discorso di community. Il docente è d'accordo che sarebbe più spinto da quella parte rispetto a *Stardew Valley*. In *Stardew Valley* si condivide la propria fattoria con le persone che giocano con te: per ogni slot di salvataggio ci sono quattro posti; uno è il tuo, e una volta creati i personaggi per gli altri tre, per quanto ne sa il docente, non li si può più cancellare. In *Animal Crossing* si può andare a visitare l'isola di un altro e farlo venire sulla propria; le meccaniche sono semplici, e comunque qui si parla di motivazione, quindi di responso emotivo. Il docente è dell'idea che *Animal Crossing* sia più verso il teamwork, ma "andrebbero trovati i dati".

5.9.5 Competition

La **competition** è competere con un altro individuo, generalmente con una risoluzione uno a zero: uno vince, l'altro perde. Da una parte ci sono i giochi ad **alto conflitto**: competizioni dirette, arene, duelli, match uno contro l'altro, eventualmente *leaderboard* che mettono in rapporto ciò che si è fatto con ciò che hanno fatto gli altri, e *ranking*, cioè una scala gerarchica.

- *League of Legends* è ad alto conflitto: ha i ranghi, "da bronzo, legno, mattone fino a diamante incredibile, mega ultra top 1000 mondiale". *Dota 2* è lo stesso; *Call of Duty* e *CS:GO* hanno la stessa cosa.

- Anche i **giochi di carte** sono generalmente ad alto conflitto.

Reattivo o strategico, non importa. L'alto conflitto **non è legato al tipo di gameplay**, reattivo o strategico (se ne parlerà). In *Call of Duty* c'è la reazione *second to second*: scegliere dove andare, mirare, reagire. In *Hearthstone* si ha il proprio turno, circa un minuto, da giocare in tranquillità, anche fino agli ultimi dieci secondi: si ha modo di pensare alle proprie azioni. Ma il risultato, lato competizione, è lo stesso.

Dark Souls e For Honor. *Dark Souls* non è di per sé un gioco ad alto conflitto; ma se ci si concentrasse solo sul multiplayer, quella sfera lo sarebbe. Un esempio di alto conflitto "puro" è *For Honor* di Ubisoft, l'arena di combattimento medievale con classi come il vichingo e il cavaliere (il docente lo cerca su YouTube, tra pubblicità di ogni genere, e ne mostra un combattimento). Ci sono anche dei *minion*, come in *League of Legends*, NPC con poca vita, "carne da macello"; ma nei duelli uno contro uno c'è il *target lock* e un sistema di guardia molto interessante basato sulle **stance**: la spada in basso a destra, in basso a sinistra, in alto, e si difende dalla parte da cui arriva il colpo. Funziona come un "carta, forbice, sasso": se si entra dove l'avversario è scoperto si fanno più danni. L'obiettivo è parare quando arrivano i colpi e farli entrare dove l'altro non è difeso. Un sistema che, secondo il docente, si potrebbe "rubare" e mettere altrove per fare un gioco interessante.

Il basso conflitto. Dall'altra parte non c'è un avversario: niente duelli, niente sfida contro un altro giocatore. *Firewatch*, *Myst*, *Gone Home*, i *walking simulator* in generale: non si è in competizione né con sé stessi né con altri. Anche i giochi LEGO, a bassa competizione per definizione, e *Dragon Age 2*.

E le mod? Uno studente chiede che cosa succede se una mod non ufficiale aggiunge del competitivo a un gioco che non lo ha.[28] Bisognerebbe capire **quanto impatta**: magari la mod risponde a un'esigenza dell'utenza ("vorrei il gioco più competitivo"), ma vale lo stesso discorso di *Dark Souls*: la stragrande maggioranza del gioco è single player, è pensata come tale, e la sfera della competizione è molto ridotta. Il gioco si sposta sulla linea, ma non poi così tanto. Su ogni spettro **ci si sposta aggiungendo o togliendo contenuto** in una direzione specifica; va capito di quanto.

Le motivazioni dominanti. Ogni gioco ha **ciascuno** di questi elementi, più o meno spinto. Per semplicità, come si fa con le emozioni, per descrivere un gioco si prendono gli elementi **dominanti**: le motivazioni più spinte, le più identificative. Per *League of Legends*, tra *action*, *social*, *mastery*, *achievement*, *immersion* e *creativity*, probabilmente interessano *mastery*, *social* e *action*, in quest'ordine. Significa che non ha *achievement*, o *creativity*? No: li ha, ma in misura molto ridotta rispetto al resto. "Questo è il concetto."

5.9.6 Excitement

L'**excitement** è quanto il gioco è *fast paced*, quanto è *action*, quanti picchi di adrenalina ci sono in determinati secondi di gioco, nel *second to second*. Gli FPS in prima persona come *CS:GO* sono *thrilling*; ancora più thrilling potrebbe essere *Ready or Not*, e ancora più di questo *Bodycam*, che spinge il realismo più in alto. Ma vale anche per *Smash Bros.*, chiaramente *fast paced*, *action based*, *adrenaline rush*; e anche i giochi di corse sono generalmente thrilling, compreso il nuovo gioco di corse di *Star Wars*.

Dall'altra parte ci sono i giochi rilassati: qui si vedono già i due grandi tipi di gameplay che si studieranno, **action reactive** contro **strategico**, *planning* e *predictive*; o, come si dice di solito, *pace* elevato contro *pace* lento e calmo. Ci sono tutti i **turn based**, che possono essere messi in pausa senza conseguenze e tendono al rilassamento e alla prevedibilità: *Pokémon* ci ricade. Bassa stimolazione visiva (non all'estremo) come *Myst*; *Civilization*, che non è all'estremo perché a seconda dell'azione un po' di thrilling c'è, ma il fatto che sia a turni lo rende molto più calmo. *Animal Crossing*, chiaramente, è tutto spinto in questa direzione.

Smash è excitement o competition? Uno studente chiede se *Smash Bros.* non vada piuttosto sotto la competizione. Il docente è d'accordo: sicuramente c'è il thrilling, ma se si dovesse scegliere la motivazione principale, probabilmente avrebbe ragione lo studente. Sono i giochi **horror** a essere "questa roba": di

[28] Nella trascrizione il docente si rivolge a "Valeria"; nel resto della lezione compare uno studente di nome Valerio, ma non è possibile stabilire con certezza chi abbia fatto la domanda.

un *SOMA* o di un *Amnesia* è più necessario dire che sono thrilling. Per gli sport, la competizione è la motivazione principale.

Lo studente gioca a *Smash* in modo competitivo, e fa quindi parte del target audience; ma ci sono persone che giocano a *Smash Bros.* come *party game*, ed è probabilmente il suo punto di forza. Lo studente ribatte che *Ultimate* è stato pensato anche come party game, ma che alcune aggiunte di meccaniche lo hanno reso più difficile; il docente concede che c'è anche quella parte.

Mario Kart. Un altro esempio Nintendo: in *Mario Kart* la competizione è un punto forte, ma meno rilevante dell'excitement, perché è più un party game. L'obiettivo non è tanto vincere quanto partecipare: è più interessante vincere tutte le corse, oppure, anche arrivando ultimi, assicurarsi che l'amico con cui si gioca si becchi il guscio blu prima della linea del traguardo? Si dice che *Mario Kart* distrugga le amicizie, ed è pensato per quello. Il fatto che il risultato si possa ribaltare anche a un centimetro dalla fine è pensato proprio per rendere la competizione molto più leggera e il gioco molto più *party* (si ragionerà anche sul suo *loop* di gioco; e dipende dal *Mario Kart*).

5.9.7 Destruction

La **destruction** è l'entropia, la possibilità di modificare e distruggere l'ambiente. Ricade sempre nell'*action*, perché è la necessità del giocatore di "fare casino": distruggere cose, impattare con violenza il mondo di gioco o le persone che ci stanno dentro. Da una parte, nel reame del **caos**, ci sono le esplosioni, le pistole, il sangue, la violenza: strappare l'occhio al capo demone, far scoppiare qualcosa in un gioco di guerriglia e vedere il muro crollare fisicamente.

Dall'altra ci sono i giochi il cui mondo non si può distruggere. In *Professor Layton* non si può distruggere letteralmente niente: il mondo è fisso, si gira per la città (o per il treno, a seconda del capitolo), al massimo si parla con le persone. Non ci sono armi, non c'è *gore*, e generalmente nessuno muore (in qualche capitolo forse si vede "una mano", e basta). Ci ricadono anche *Animal Crossing* e *Harvest Moon*; *Stardew Valley* sta un gradino sopra, perché si possono fare piccoli danni: c'è la bomba, si possono distruggere i *tile* piazzati da sé, i mostri, anche i simpatici slime con il sorriso. Ma una bomba piazzata accanto a un NPC non gli fa nulla, né distrugge la sua casa.

5.9.8 Completion

La **completion** è avere o non avere **linee guida dirette**, vere e proprie *checklist* da riempire. Molti JRPG, molti *Dragon Age*, in un certo senso anche *The Witcher*, hanno obiettivi primari e secondari, *quest* principali e secondarie; anche *Zelda* è così. Diventa ancora più spinto tutto ciò che è legato agli *achievement*: "batti il boss dieci volte per sbloccare questa cosa" è **task oriented**. Dall'altra parte c'è il **self-driven**: è il giocatore a decidere le proprie priorità. *Minecraft* sta qui, come generalmente tutti i *sandbox*. *Kerbal Space Program* è questo: bisogna mandare gli astronauti alieni nello spazio, ma si decide come montare il razzo, nei limiti del gioco.

5.9.9 Power

Il **power** è un **falso amico**, e lo si capirà parlando di progressione. Indica la possibilità di **ottenere potere**: livellare, cioè modificare le statistiche, e migliorare le armi. Qui c'è sicuramente *Dark Souls*; e tutti i giochi come *Diablo 3* o *Borderlands*, con N difficoltà che scalano, in cui il giocatore "lutta" per ottenere la stessa arma, con la stessa forma e probabilmente la stessa rarità, ma con un numero N volte più alto. Tutto ciò che si può migliorare tramite statistiche o tramite lo sblocco di nuove meccaniche.

Il contrario è la **self-efficiency**, che il docente aveva anticipato nell'esercizio sui metroidvania: in *Super Mario Odyssey* non si diventa più forti. Ci sono nemici che, catturati, sbloccano nuovi verbi di gioco, ma sono limitati a dove si trova il nemico. Nella parabola di *Mario Odyssey* si sviluppa **competenza nel platforming**: si capisce meglio come lanciare il cappello, come fare il doppio salto sul cappello, e così via. Lo stesso in *Super Mario 64*.

All'estremo assoluto ci sarebbe un personaggio che ha già tutti i verbi di gioco e tutte le statistiche al massimo, o meglio, le cui statistiche **non cambieranno mai**. *Night in the Woods*, avventura narrativa,

è così. *Firewatch* no, perché si sbloccano cose, quindi sta un po' più in là, nonostante sia spiccatamente narrativo. *Her Story* è così: nessuna progressione sul personaggio.[29] *Ico* è così: c'è il protagonista e la ragazza da portare via tenendola per mano; "quello è, quello rimane", e il protagonista non sblocca abilità.

5.9.10 Strategy

La **strategy** è la complessità nelle decisioni: quanto il gameplay è strategico, o il contrario. Lo si vedrà definito in altri modi, ma in generale un gioco con molta excitement avrà poca strategia in questo senso; non è sempre detto, dipende dal framework. Da una parte c'è il **contemplativo**: poter pianificare, avere decisioni complesse e strategie a lungo termine, e il **tempo** per farle. *StarCraft* non sta su questa linea, perché c'è comunque un gameplay *second to second* a cui sottostare: il docente mostra un video dei migliori giocatori, che cliccano velocissimi ("ho cliccato troppe volte, scusate"). *Europa Universalis* è ancora più contemplativo; *Balatro* è completamente strategico: ci sono i turni, nessun limite di tempo, e bisogna scegliere che scala fare, se rischiare o no, se cambiare mano o no.

Dall'altra parte c'è lo **spontaneo**: gameplay reattivo e **basso peso cognitivo**. È diverso dall'excitement, che spesso ha un alto peso cognitivo: qui si fanno azioni in poco tempo con un basso costo cognitivo. *Disney Emoji Blitz*, *Candy Crush*, i *match-3*; e *Mario Kart*, con il suo gameplay reattivo nel *second to second*: si deve capire quando fare la curva, ma la si fa subito.

5.9.11 Challenge

La **challenge**, la sfida, va da **skill based** a **easy fun**. Il docente precisa subito che non è un errore: è probabilmente l'**unica eccezione** per cui si può usare il termine *fun*, bandito dal vocabolario del corso (sezione 2.12). Sono i **quattro tipi di fun di Nicole Lazzaro**: *easy fun*, *hard fun*, *serious fun* e *people fun*, "i quattro tipi di fun che potete citare". Il *people fun* è, per esempio, la community; l'*hard fun* è letteralmente il contrario dell'*easy fun*, cioè la sfida. Si rivedranno più avanti.

Skill based. *Dark Souls*, ma ancora di più, come si diceva prima, *Ori and the Blind Forest*: è skill based e anche molto reattivo. Si pensi alle sequenze di fuga, o al fatto che si salta "rimbalzando" sui proiettili dei nemici; c'è del *freestyle*. Ci sono *forgiveness mechanics* molto importanti ("ci mancherebbe"), ma è skill based. Lo è anche *Hollow Knight*: messi in ordine, ci sarebbe prima *Hollow Knight* e poi *Ori*, anche se secondo il docente non sono così lontani. Il docente ha giocato solo il primo *Ori*.

Una digressione su Hollow Knight. Qui la discussione con alcuni studenti devia su *Hollow Knight* e *Silksong*: il docente non ha completato una delle prove più dure (l'ha iniziata e ha detto basta), e non capisce il senso di un premio che è solo "la medaglia d'onore" di averla fatta; uno studente difende la parte narrativa, legata ai rapporti tra padre e figlio, che il docente trova sepolta dietro contenuti opzionali, legati peraltro al finale migliore e non a quello normale. È anche il motivo per cui ha abbandonato *Hollow Knight* un paio di volte prima di finirlo, oltre a una fase di *backtracking* poco chiara; lui che, assicura, non si è mai bloccato in un metroidvania in vita sua. "Piccolo odio", anzi, "odio, ci mancherebbe, avercene".[30]

Tutte le abilità dall'inizio. Lo skill based è legato al miglioramento delle skill del giocatore: è l'efficienza di cui si parlava. Si possono anche dare al giocatore tutte le abilità fin dall'inizio, come in *Street Fighter* o in *Smash Bros.*: l'obiettivo del giocatore sta nello sviluppare *mastery* nel sistema.

Easy fun. Dall'altra parte c'è l'*easy fun*, che nella definizione di Lazzaro è **curiosità, novità, immaginazione**; anche l'abilità intuitiva di risolvere cose, tendenzialmente legata ai puzzle, ma non solo. Ci sta *Oxenfree*, avventura narrativa horror, un *teen drama*: tutte le decisioni di gameplay, anche nei dialoghi, sono molto intuitive. Vuoi fare lo stronzo con tuo fratello o vuoi fare *bonding* con lui? Vuoi aiutare i tuoi amici a diventare una coppia o li vuoi separare? Ci stanno anche *Animal Crossing* e *Stardew Valley*, giochi

[29] Nella trascrizione il titolo compare come "Air Story"; la lettura *Her Story* è la più plausibile. Subito dopo il docente cita un altro titolo che dice di non aver giocato ("Easy Days"), non riconoscibile.

[30] Il passaggio è molto frammentato nella trascrizione, con più voci sovrapposte; se ne riporta solo il senso generale.

con una **bassa barriera di skill**; anche se *Stardew Valley* forse non sta proprio lì, perché c'è una quantità di informazioni così elevata, e spesso non spiegata, che un certo tipo di ricerca, non solo di intuizione, è richiesto. "Comunque, ripeto, è uno spettro."

Approfondimento: le quattro chiavi di Lazzaro. Bateman e Boon, in *21st Century Game Design*, usano lo studio di Nicole Lazzaro (*Why We Play Games: Four Keys to Emotion without Story*, 2004) per interpretare i loro tipi di giocatore. L'*hard fun* è il gameplay legato a sfide, strategie e puzzle significativi: Lazzaro vi associa rabbia, frustrazione e noia, ma soprattutto il *fiero*, l'euforia del trionfo personale su una difficoltà, "quella sensazione che fa alzare le braccia al cielo". L'*easy fun* è associato a meraviglia, stupore e mistero. Il *people factor* (il *people fun*) è il divertimento sociale, in cui la chiave è il piacere di stare con gli altri, schadenfreude compresa. La quarta chiave, che il libro chiama ancora *altered states* (gli stati emotivi generati dalla percezione e dal comportamento), è quella che in seguito Lazzaro ha rinominato *serious fun*. È lo stesso vocabolario che il modello di Quantic Foundry recupera per il polo basso della challenge.

5.9.12 Fantasy

La **fantasy** è la sospensione dell'incredulità. Da una parte la **lore profonda**: un mondo molto profondo, tanta narrativa di base. *Mass Effect*, *Dark Souls*, *Hollow Knight*, *Fallout*, *Dragon Age*: giochi con una componente narrativa e una lore non necessariamente alle spalle, ma anche nel mondo di gioco, quindi non solo legata a un passato ma anche a un presente, una narrativa presente e molto dominante.

Dall'altra parte il **setting generico o astratto**, o comunque giochi in cui la narrativa è un *flavor*, un *dressing*, non una componente importante del gameplay: non lo cambia. *Street Fighter*, *Gato Roboto* (citato prima), *Candy Crush*, *World of Tanks* (ci sono i carri armati che si sfidano, e basta); e anche un *Call of Duty*, per il tipo di motivazione che l'utente cerca. Al docente dà ragione il fatto che lì dentro il single player ormai quasi non esista più: è più spinto in questa direzione, soprattutto nel tempo, in una parabola discendente.

5.9.13 Story

Se la fantasy è la creazione del contorno del mondo di gioco, del contesto in cui il giocatore gioca, la **story** è il **dramma**: esiste una narrativa importante con un inizio e una fine nel mondo di gioco. Vale anche per la profondità dei personaggi: un cast molto ben scritto, personaggi a tutto tondo. Qui stanno tutti i giochi Telltale, i giochi BioWare ("riposi in pace"), i *Persona*, e in generale molti RPG con trame molto lunghe e personaggi molto convoluti.

Dall'altra parte l'**open ended**: a volte la storia nel mondo di gioco non è nemmeno spiegata, a volte è interpretata o ha un finale aperto. *Factorio* e *SimCity* sono chiaramente open ended. *Mario* starebbe più verso il dramma, perché uno script esiste: liberi la principessa da Bowser. Si può discutere sul fatto che non sia estremamente profondo né abbia un grande cast di personaggi dalle molte personalità, ma "è più qui che qui".

5.9.14 Discovery

La **discovery** è l'esplorazione, ciò che si diceva di *Minecraft*: la possibilità di esplorare qualcosa di sconosciuto, che a volte si traduce proprio nell'azione fisica di andare oltre la collina, per vedere che cosa c'è dall'altra parte. *The Elder Scrolls*, *Fallout*, *The Legend of Zelda*, ma soprattutto gli open world come *Minecraft*, *Valheim*, *Skyrim*; e *Subnautica*, quello sott'acqua, "bellissimo": è curiosità.[31]

Dall'altra parte i giochi estremamente **pratici**, tecnici, con regole definite, scriptate, e minime variabili nell'esplorazione. *FIFA*, *Madden* e i giochi sportivi hanno una discovery quasi a zero; così lo *Scarabeo*, gli *scacchi* e i giochi da tavolo riprodotti.

[31] Nella trascrizione il secondo titolo compare come "Venom"; la lettura *Valheim* è la più plausibile nel contesto, ma non certa.

5.9.15 Design

Design è un termine molto generico; come risposta emotiva e motivazione generale si traduce nella **self-expression**: la possibilità di modificare il modo di giocare, il personaggio principale, secondo le proprie esigenze. Il personaggio con i capelli corti o lunghi, rosa o verdi, le unghie nere, rosse o gialle; grasso, magro, alto, basso, e così via.

Dall'altra parte il **personaggio fisso**. *Mario* è *fixed*: nemmeno *Mario Odyssey* lo è del tutto, perché si possono cambiare i costumi, ma il personaggio resta quello. *Braid* è all'estremo finale, e anche *Spelunky*; *Spelunky 2* è "un tacchettino" più in là, perché si può scegliere tra una decina di personaggi. In *Super Mario Galaxy 2* si poteva usare anche Luigi (il docente non ne è sicuro): è a metà tra i due. Restano comunque personaggi fissi: l'asset è quello.

5.9.16 A che cosa serve

"Non dimenticatevi questo spettro": è molto importante per definire le **motivazioni principali** di un gioco, per **categorizzare l'utente** e per chiedere all'utente che cosa gli interessa. Naturalmente, le domande vanno fatte in un modo che abbia senso nel gergo del giocatore; ma intanto si ha già questo framework, che si vedrà ibridato più avanti.

Il sito e il survey. Sul sito di Quantic Foundry (il docente mostra anche la versione vecchia, "guarda che bello questo") ci sono **report sull'audience** e perfino una **perceptual mapping** che funziona su una serie di giochi: "e così si fa". Ci sono molti dati che in aula non si sono avuti. C'è anche un **survey** che si può compilare, e da cui si possono eventualmente prendere domande (il docente non sa se sia ancora aperto). Non solo categorizza negli archetipi di Quantic Foundry (che si rivedranno, insieme ad altri più largamente condivisi), ma propone anche, rispetto ai giochi preferiti, altri giochi vicini, che individui con motivazioni simili hanno indicato. "Strumento molto utile."

Dati chiusi. Purtroppo, come per altri framework, sondaggi, *audience report* e archetipizzazioni che si vedranno, i dati sono chiusi o sono stati chiusi. Ci sono eccezioni, ma non le tratterà il docente: arriveranno al terzo anno. "Questo è il mantra, e ci devo stare pure io."

5.10 Correlare: pattern, coincidenze e insight

5.10.1 Chiedersi perché

L'ultimo punto della ricerca è la **correlazione**. Una volta che si hanno tutti i dati, ci si chiede **perché** è successo qualcosa:

- perché c'è questo buco di mercato?
- perché questo gioco ha fallito, rispetto a ciò che si è ottenuto?
- perché a questa persona non piace? Perché *Concord* non è piaciuto all'utenza in generale? Com'è successo? Che cosa ha significato?
- c'è un **pattern**? Ogni tot anni esce questo tipo di gioco, emerge qualcosa, succede qualcosa? Come lo si può riprodurre? Come lo si può semplificare?

Per rispondere ad alcune di queste domande si sono già visti diversi metodi; il più semplice sono i **nove perché** (sezione 4.10).

5.10.2 Il cervello che vede pattern

Nella slide c'è "una grande immagine": il docente si dichiara grande fan delle teorie del complotto, di qualsiasi teoria del complotto aliena e non solo.[32] Non sono vere, non hanno una profondità reale; ma tutti abbiamo

[32] In questo punto la trascrizione automatica entra in loop su una parola ripetuta; il contenuto si recupera dalle frasi successive.

questo meccanismo nel cervello. È un **bias cognitivo**: riconosciamo pattern, molto bene o molto male (per esempio, nella statistica), perché in passato questo ci ha salvato la vita.

Se il nostro cervello fosse adattato a calcolare tutte le possibilità per cui, uscendo dalla stanza la mattina, si può morire (un'auto che ti investe, un asteroide in testa, un cane che ti azzanna alla gola), non usciremmo mai più di casa. Il cervello si è adattato a questo, ma anche a riconoscere pattern utili: le cose commestibili, le cose esteticamente appaganti. La **simmetria** è così importante per noi perché generalmente le cose simmetriche erano cose buone, per il nostro antenato nella vecchia caverna.

Lincoln e Kennedy. Molte correlazioni nascono da questa lettura sbagliata dei pattern. L'esempio della slide sono le famose coincidenze tra Lincoln e Kennedy: entrambi assassinati, con dettagli che sembrano ripetersi, come la moglie che avrebbe avvisato di non uscire quella mattina. Sono tutte coincidenze ("magari qualcuno vuole crederci, ma vi assicuro che sono coincidenze"): bisognerebbe investigare a fondo ciascun punto. Perché sono morti entrambi di venerdì? È una coincidenza relativa, perché molti eventi pubblici si tengono il venerdì, il sabato e la domenica, per massimizzare l'affluenza.

Smontare le proprie ipotesi. Quindi, una volta che si hanno i dati, o si è trovato un *blue ocean* (sezione 4.9), magari con la perceptual mapping (sezione 5.5), "non ci andate subito, per favore". Bisogna domandarsi, chiedere, approfondire, **cercare di smontare le proprie ipotesi**.

5.10.3 Dall'insight all'empatia

Correlando e sintetizzando la ricerca si capiscono i *perché*; i perché portano all'**insight** visto prima (sezione 5.4), rispondono ai dubbi e sviluppano **empatia** con il target audience. Se si ha insight sui dati, se si ha empatia con il target audience, se ci si è chiesti abbastanza volte perché, allora si possono **soddisfare le necessità e le emozioni del target audience**. È l'obiettivo finale, considerando tutti i limiti in cui si "nuota": gli stakeholder, il lato business, la visione, il gameplay. È ciò che si dovrebbe raggiungere alla fine, e lo si ritroverà quando si parlerà di visione, in una versione "rimaneggiata" della stessa slide ("una spolverata di olio di gomito").

5.11 La fine della pre-produzione

Una volta ottenuti i dati, la fine della pre-produzione si sancisce con una serie di **risposte** (tabella 5.3).

Che cosa vogliamo fare? Che cosa vogliamo fare noi, come team? Che cosa ci porta avanti, qual è la nostra motivazione, che giochi vogliamo fare?

Che cosa possiamo fare? Qui si va verso la visione più produttiva: che cosa si può fare realisticamente, con quali abilità, che si hanno o che si vogliono (e si possono) sviluppare. Per quanto al docente piaccia la pixel art, o fare UI, dire che da oggi a domani, o da oggi alla fine della produzione, diventerà un pixel artist o un 2D artist è improponibile. **Siate concreti; piuttosto, sparate basso.** Qui dentro ci sono le abilità del team e la **fattibilità** (*feasibility*), legata allo *scope* e alla scala del progetto: che cosa si può fare, dati questi tempi, questi limiti, queste risorse.

Che cosa sappiamo che funziona? Lo dicono il target audience e i competitor: sono i *must*.

Che cosa sappiamo che non funziona? Lo dicono il target audience e i *must not*. Li si vedrà.

Che cosa vogliono gli stakeholder? Lato manageriale e lato cliente (sezione 4.11).

Qual è il nostro business? Rispetto al target audience, a livello di vendite: un gioco con le pubblicità oppure no? Si aggiungono DLC? Si vuole un gioco transmediale, altri prodotti derivati dall'IP? "Dipende."

Table 5.3: Le domande che chiudono la pre-produzione e da cui nasce la game vision.

Domanda	**Da dove viene la risposta**
Che cosa vogliamo fare?	motivazione e desideri del team
Che cosa possiamo fare?	abilità del team, fattibilità, scope, tempi, risorse
Che cosa sappiamo che funziona?	target audience e competitor (*must*)
Che cosa sappiamo che non funziona?	target audience (*must not*)
Che cosa vogliono gli stakeholder?	management e cliente
Qual è il nostro business?	vendite: pubblicità, DLC, transmedialità

La game vision. L'idea che interconnette tutte queste risposte è la **game vision**: in concreto, il documento di visione finale, più i pilastri, più una serie di altre cose che si vedranno (sezioni 4.9 e 3.5). "Questo era per quanto riguarda la ricerca."

5.12 Idee e brainstorming: i principi

5.12.1 Le idee sono inutili

Si passa, "veloci", alle idee e al brainstorming. Anche qui arriva una serie di strumenti, da non usare sempre tutti ("è una lista, e ce ne sono molti altri"), ma che possono dare spunti su come discutere, con chi farlo e in che tempi.

Il punto di partenza è una provocazione: **le idee sono inutili**. Non esiste un'idea valida, che abbia senso nella produzione, se non è **contestualizzata** nei limiti, nella visione, in tutto ciò che non si può fare. Iterazione e implementazione sono tutto, in un progetto (sezione 3.5). Le slide sono provocazioni, avverte il docente: non dicono che cosa si deve fare esattamente, ma che cosa si potrebbe fare.

5.12.2 Generare tante idee, scartare le prime

L'obiettivo di un brainstorming è **generare un "macello" di idee**: la slide ne indica cento, ed è davvero tanto. Poi:

- si scartano quelle troppo vicine all'**affezione del team**, a cui ci si affeziona troppo e che quindi non si riesce a valutare in modo obiettivo; anche questa è una provocazione, ma tendenzialmente si possono **scartare le prime idee** che escono, le prime venti;
- si valutano le altre con i dati che si hanno, i pro e i contro, e così via.

La legge del martello. Perché scartare le prime venti? Andare contro i bias cognitivi è sempre difficile, ma esiste un principio che, semplificando molto, dice: **se hai un martello, vedrai tutti i problemi come chiodi** (il termine "chiodo" arriva dalla classe). Siamo abituati a risolvere i problemi con gli strumenti che usiamo di solito. "Squadra che vince non si cambia": se qualcosa funziona, se vediamo il mondo in un certo modo perché ci ha dato valore per anni o perché con quello risolviamo quasi tutti i problemi che ci arrivano addosso, potremmo avere i paraocchi e non vedere altro.

Non da soli. **Lavorare in team** aiuta a superare questo limite: io vedo il gioco, l'idea, il modo di risolvere i problemi in una certa maniera, e altri lo vedono in un'altra. Il docente ha sempre trovato quasi sufficiente lavorare con altri designer, con altre persone: fare brainstorming **non da soli**, perché da soli è difficile. Chi vuole farlo da solo, o fa fatica a mettersi nei panni degli altri o a "lasciar andare nel tritacarne" le proprie idee, può trovare utili altri metodi, come l'*acting brainstorming* o le altre tecniche che si vedranno.

Approfondimento: prima da soli, poi insieme. Schell, nei suoi sedici consigli pratici per il brainstorming (*The Art of Game Design*, capitolo 7), dedica l'ultimo al trovare un partner: in due si può arrivare a grandi soluzioni molto più in fretta, rimbalzandosi le idee; ma aggiungere persone non aiuta necessariamente, e di solito i gruppi migliori non superano le quattro persone. I gruppi funzionano meglio su un problema **circoscritto** che su uno ampio e aperto; e la ricerca, nota Schell, mostra che un gruppo che si riunisce "a freddo" per fare brainstorming spreca tempo: è molto meglio che ciascuno faccia prima brainstorming **da solo**, e che solo dopo ci si riunisca per condividere, mescolare le idee e risolvere problemi insieme. Vanno evitati come partner quelli che bucano ogni idea o che hanno gusti molto ristretti.

5.12.3 Le regole del brainstorming

Comprendere la mente umana. Per capire che cosa *non* fare, bisogna capire la mente umana: gli eventuali bias cognitivi, da mettere da parte se ci sono.

Tutti sono creativi. Bisogna trovare modi per far emergere il più possibile tante idee. Generare cento idee in poco tempo non è semplice: bisogna trovare il modo di "spararne fuori" il più possibile. Per chi non è particolarmente creativo, le slide indicano due modi molto utili, e se ne vedranno altri (sezione 5.15).

Lavorare dentro il framework, ma dopo. Vale in due sensi. Quando, un passo dopo il brainstorming, si comincia a setacciare le idee, vanno tolte tutte quelle non attuabili, non realistiche da applicare. Ma **non lo si fa prima**: durante il brainstorming non ci si frena, **anything goes**, si butta fuori qualsiasi cosa. Anche da qualcosa che poi verrà scartato, o che non ha senso nel contesto della produzione, può emergere qualcosa di interessante o unico. Dopo, quando si valuta, si ricordano la visione, le limitazioni, i documenti, i limiti.

Rispettare tempi e ruoli. Se il brainstorming ha regole di tempo o di ruoli, le si rispetta: se sta parlando il docente non parla Alessandro, e se sta parlando Lorenzo non parla il docente. L'obiettivo del brainstorming è **dare a tutti la propria voce**, fare in modo che tutti esprimano la propria idea. E, come sempre, se dopo (non in quel momento, molto dopo) la propria idea non viene accettata, non significa essere un cattivo designer o aver fatto un brutto lavoro: l'idea, quando viene messa a terra, passa in mano a tutti. Durante il brainstorming tutti devono poter parlare, sentirsi liberi di dire la propria e **non sentirsi giudicati**. Ci sarà il momento di valutare attentamente, ragionare, setacciare, tagliare, "bruciare" idee, ma non è quello.

Soluzioni *future proof*. Dopo, prima della produzione, durante la pre-produzione, è importante essere **severi** per primi con ciò che si idea e si documenta. Se non lo si fa lì, arriverà qualcuno, volutamente o no, con una domanda o un dubbio che distrugge quel design, o che mostra una situazione a cui non si era pensato, e quello è un problema. Bisogna avere il design **più solido possibile all'inizio della produzione**: tenere le soluzioni *future proof* e scartare le altre. Se qualcosa convince fino a un certo punto, ma ha dei limiti o delle pericolosità future, si valuta se il rischio vale la candela; se no, via.

5.13 Il design elegante

5.13.1 Che cosa vuol dire

"Puntate al design elegante. Questo non è vero sempre, ragazzi." Fare design elegante piace a tutti ("piacerà a voi quando saprete che cosa vuol dire"), ma è difficile. Un design elegante è non solo **easy to learn**, ma **difficult to master**; è **molto interconnesso**, con regole che hanno ricadute su altre regole. Un tiro di dado che gestisce sia la vita sia la stamina, una statistica che gestisce due cose diverse, un attacco che fa due cose: è elegante, perché è molto **condensato**.

Il rovescio della medaglia. Proprio per questo, se in futuro si deve toccare quella statistica, la vita o il sistema di dadi, l'intervento impatta su entrambe le cose. Se non lo si "blinda", se non ci si pensa molto

bene all'inizio, "vi ritorna indietro due volte sui denti": fa male il doppio. È il legame con le soluzioni *future proof*.

5.13.2 Elegante non vuol dire migliore

Non c'è sempre bisogno di fare design elegante. Ottimi giochi, soprattutto ottimi giochi da tavolo, lo fanno: *Root*, per esempio. Ma un ottimo gioco da tavolo che non è minimamente elegante è *Dune*, nella riedizione di Gale Force Nine: uno dei giochi da tavolo preferiti del docente, "non minimamente elegante di design, però funziona estremamente bene"; è anche vero che è figlio di un'altra epoca, di un altro grande gioco.[33]

Gli esempi della classe. Gli studenti propongono altri titoli:

- *Here to Slay*, che uno studente trova elegante per la sua condizione di vittoria legata ai mostri;
- *Unstable Unicorns*, che al docente era venuto in mente prima perché è ancora più elegante, anche se con meno meccaniche: "è *Yu-Gi-Oh!* semplificato, *Yu-Gi-Oh!* per *toddlers*";
- *7 Wonders*, con un sistema di *drafting* molto interessante, anche se il docente non lo considera elegantissimo (forse, considerando le espansioni, qualcosa si può pensare);
- il drafting di *Terraforming Mars* (confuso per un attimo con *Surviving Mars*: "uno è il gioco da tavolo e l'altro è il gioco digitale").

Anche i giochi molto contenuti, con scatole ridotte, possono essere estremamente eleganti; al docente sfugge il nome di un gioco "con l'animale", "perduto per sempre; tanto mi ritornerebbe prima o poi".

Dalle meccaniche ai sistemi. Uno studente collega il discorso al design di sistema. Il docente conferma: qui si parla di meccaniche, ma nella massima estensione i sistemi interconnessi diventano il **system design**, se il gioco è sistemico (e non è detto che lo sia). In generale, il design elegante è "una buona *forma mentis*".

Approfondimento: la lente dell'eleganza. Schell dà un modo pratico per misurare l'eleganza: **contare gli scopi** di ciascun elemento. I puntini di *Pac-Man* danno al giocatore un obiettivo a breve termine (mangiare quelli vicini) e uno a lungo termine (ripulire il livello); lo rallentano leggermente quando li mangia, rendendo più rischiosi i corridoi pieni; danno punti, misura del successo; e i punti fanno guadagnare vite extra. Cinque scopi per un elemento semplicissimo. La *Lens of Elegance* (lente n. 49) chiede: quali sono gli elementi del mio gioco? Quali scopi ha ciascuno? Quelli con uno o due scopi si possono combinare tra loro o eliminare? Quelli con molti scopi possono assumerne altri ancora? Subito dopo, però, Schell introduce la *Lens of Character* (lente n. 50): eleganza e carattere sono opposti da tenere in equilibrio. La torre di Pisa, resa "elegante" raddrizzandola, non la visiterebbe nessuno; le pedine di *Monopoly* non c'entrano nulla con il mercato immobiliare, e Mario non ha alcun motivo di essere un idraulico, ma è proprio questo che dà loro carattere. È il senso del "non sempre" del docente.

5.14 Essere creativi

Come essere creativi? Non c'è una vera soluzione, e non è importante se si è più o meno portati per la creatività. Conviene però identificare eventuali **momenti o rituali** in cui si pensa meglio.

I rituali. Woody Allen, racconta il docente, non riusciva a scrivere le sceneggiature dei suoi film se nel suo albergo non c'era una doccia: doveva per forza averla. Un altro esempio citato è quello di chi faceva un sonnellino e, al risveglio, aveva l'idea.[34] Ci possono essere momenti di calma, o al contrario di estrema agitazione, in cui si è più prolifici.

[33] La riedizione di Gale Force Nine (2019) riprende il *Dune* di Avalon Hill del 1979. La frase nella trascrizione è disturbata ("figlio di un'altra equa, grande gioco").

[34] Il secondo nome è storpiato nella trascrizione ("So, Cristiano"); l'aneddoto del sonnellino non è attribuibile con certezza.

La praticità. Il docente si trova molto più comodo a scrivere, come si vede, alla lavagna o sui fogli di carta, a scarabocchiare: la praticità lo aiuta, ad altri potrebbe dare fastidio. Racconta di un altro senior con cui lavorava, anche lui molto pratico ma con i fogli digitali: quando si parlava di combat, lui disegnava gli *stickman* che risolvevano il problema, mentre il docente era davanti alla camera "a prendere a bastonate il nulla". Non è obbligatorio trovare un metodo; ma se si ha bisogno di stimolare la creatività, conviene identificare i momenti in cui si è più prolifici.

5.15 Le tecniche per generare idee

5.15.1 Una rassegna, non un obbligo

Il docente procede "a ruota", fino a un certo punto, rimandando il resto al venerdì. Non si sofferma molto sui singoli punti, perché sono tutti modi di mettere giù un'idea, fare brainstorming o creare qualcosa, e non vanno applicati tutti: possono essere utili uno adesso, un altro tra vent'anni. Come sempre, sono strumenti, e ce n'è "davvero una miriade". In un brainstorming non codificato ognuno può fare quello che vuole; se è codificato, ci saranno limiti, tempistiche e un *handler* che decide chi parla e quando, "non perché è brutto e cattivo", ma per garantire che parlino tutti.

5.15.2 Le categorie sensoriali

Il primo modo divide la questione in **categorie**: teoricamente tutto si può scomporre in categoria **visiva**, **uditiva**, **tattile**, **cinestetica**, **spaziale**, **enattiva** (la messa in moto, l'azione effettiva), **iconica** (popolare, conosciuta) e **simbolica**. L'esempio della slide è la musica:

Visiva tutto ciò che è visualizzazione: il DJ che mostra le curve dei bassi e degli alti.

Simbolica uno spartito, la chiave di violino, le note: tutta la simbologia legata specificamente alla musica, che è anche visiva quando la si mette per iscritto.

Uditiva "manco lo dico".

Tattile sentire i bassi: le persone sorde sentono i bassi delle canzoni.

Cinestetica ballare, attuare quell'azione.

Spaziale lo spazio occupato da un'orchestra, o da una band rispetto al pubblico.

Enattiva suonare lo strumento.

Iconica un cantante pop, un'immagine musicale molto nota.

Si dovrebbe poterlo fare con qualsiasi cosa.

5.15.3 Opposite direction

È una delle tecniche preferite del docente.

1. Si prende un gioco o un'idea che piace. Se si è già oltre la ricerca, si ha già una lista di categorie, generi e competitor possibili.
2. Se ne scrivono le componenti o le *game qualities* principali.
3. Si trova il **gioco opposto**.
4. Si uniscono i due: non gli elementi, ma i **due giochi insieme**.

L'esempio: *Doom* è FPS, *fast paced*, *gory*; l'opposto è terza persona, *slow paced*, *cute*: *Animal Crossing*. Insieme: un gioco con un'arte simile ad *Animal Crossing*, ma con una palette meno satura, in cui di giorno si costruisce il proprio villaggio, con cagnolini "carini e pucciosi" come personaggi, e di notte il villaggio viene assalito da orde di bestie immonde a cui si spara con un fucile a canne mozze ("sono uscito un attimo dal seminato").

Si venderebbe. Probabilmente si potrebbe vendere, anche perché quando *Doom* uscì insieme ad *Animal Crossing* la community mise insieme Isabelle e il Doom Slayer.[35] Ci sono un sacco di fan art e animazioni con Isabelle che gira con la doppietta a uccidere mostri e il Doom Slayer che cura il suo giardinetto e annaffia le piante tranquillamente. La tecnica funziona soprattutto per creare cose **innovative** o potenzialmente mai viste: *Palworld*, secondo il docente, è stato creato sicuramente così (si veda anche l'opposizione, sezione 4.8).

5.15.4 Break the theme

Con la stessa idea di fondo, **rompere il tema**: si prende un'idea, se ne elencano le **tre componenti dominanti** e se ne fa il contrario. L'esempio: un ristorante serve il cibo, lo si paga, e lo chef cucina per te. Un ristorante che non serve cibo, in cui non si paga il cibo e lo chef non cucina per te, probabilmente non sarebbe un ristorante; ma togliendo alcune di queste componenti si arriva a luoghi in cui si portano gli alimenti e si cucina da sé, con eventualmente degli aiutanti. Il docente non mangerebbe mai così, ma il punto è un altro: non è necessario prendere tutti i punti, se ne possono prendere solo alcuni, e trovare così un potenziale nuovo gioco.

Approfondimento: distruggere le proprie assunzioni. Tra i consigli per il brainstorming di Schell ce n'è uno molto vicino, che lui attribuisce al designer Rob Daviau: fare una lista di tutto ciò che si *assume* vero sul proprio gioco ("assumo che si giochi al chiuso", "assumo che il giocatore guardi lo schermo", "assumo che usi un solo dito sul touchscreen"), e poi chiedersi, voce per voce, come funzionerebbe il gioco se quell'assunzione non fosse vera. Quasi sempre l'assunzione resta; ma ogni tanto farla saltare dà un'intuizione decisiva. Daviau è arrivato a *Risk Legacy* distruggendo l'assunzione standard dei giochi da tavolo: "una partita non influenza la successiva".

5.15.5 Le regole euristiche

Le **regole euristiche** sono una serie di regole che fanno leva su alcuni bias cognitivi; nel complesso sono "veramente complesse", e ce ne sono molte altre oltre a quelle della slide. Alcune delle principali:

- se hai difficoltà a identificare un problema, **visualizzalo**: scrivilo, scarabocchialo;
- se non hai una soluzione, prova ad **assumere una soluzione** e parti da lì (il cosiddetto *bottom-up design*);
- se hai un problema concreto, **astrai**; se hai un problema astratto, **concretizza**.

Sono questioni già largamente note.

5.15.6 What if

Si scrivono domande del tipo *what if*: "e se...?". Le slide ne riportano una lista. Il docente spiega perché la trova "una figata" con i fumetti: non è un assiduo lettore di Marvel o DC, ma divora qualsiasi cosa a fumetti. *Superman: Red Son* è il *what if* di Superman se atterrasse nell'Unione Sovietica di Stalin: diventa un compagno di Stalin, un superuomo stalinista, e l'America, senza il suo protettore, deve fare la corsa agli armamenti con Lex Luthor.[36] Molte delle storie più amate di entrambe le case sono *what if*: la Marvel li chiama proprio *What If*, la DC *Elseworlds*. C'è anche il *what if* Marvel con gli zombie, scritto da Robert Kirkman, da cui è stata tratta una serie.[37] "Si può creare molto così."

[35] Il docente esita tra *Doom* (2016) e il suo seguito: si tratta di *Doom Eternal*, uscito lo stesso giorno di *Animal Crossing: New Horizons* (20 marzo 2020).

[36] Nella trascrizione il titolo compare come "Superman 9 Son".

[37] *Marvel Zombies*; nella trascrizione l'autore compare come "Kierkegaard".

5.15.7 Wicked questions

Molto simili ai *what if*: ci si chiede **come può la soluzione al nostro problema essere X e Y contemporaneamente**, dove X e Y sono soluzioni proposte dal gruppo e scritte su foglietti, messi in un grande calderone da cui si pesca.[38]

5.15.8 SIT e TRIZ

Il **SIT** (*Systematic Inventive Thinking*) è simile all'euristica e ha le sue regole, elencate nelle slide:

- **sottrazione**, **moltiplicazione**, **divisione**: prendere un problema e scomporlo nelle sue componenti principali;
- **unificazione**: è così che funziona il design elegante (sezione 5.13);
- il **closed world**: il concetto per cui, quando si formula il problema, la soluzione risiede nel problema stesso; è solo una questione di formulazione, o di vederlo da un altro punto di vista.

Il concetto di *closed world* è alla base di un altro framework, estremamente complesso, che si vedrà: il **TRIZ**.[39]

5.15.9 Limitation

Si prende il proprio gioco, o un genere, e gli si impongono **tre forti limitazioni**. Un soulslike senza combattimento melee, senza schivata e senza atmosfera cupa: sarebbe ancora un soulslike? Probabilmente sì. *Enotria*, per esempio, ha fatto una di queste tre limitazioni: niente *dark mood*. Anche le build permanenti, nota il docente, non sono necessariamente uno dei punti identificativi di *Bloodborne* o di *Dark Souls*.[40]

5.15.10 Drawing e storyboard

Si disegna: *stickman*, nessuna parola. A volte si possono usare solo figure con un determinato simbolo, come il cerchio. Una variante è lo **storyboard**: si prende un foglio già diviso nei riquadri dello storyboard e ci si disegna dentro una storiella. Può servire a trovare qualcosa di non visto.

5.15.11 Sharing

Si parla con le persone, si condividono le cose. Nella slide c'è una mamma che ha fatto una maglia incredibile: ci sono persone che fanno cose che non si direbbe mai, o cose molto lontane dalla propria sensibilità. Più si conosce, meglio è, anche perché **si può fare un gioco su qualsiasi cosa**, veramente qualsiasi. Bisogna essere **curiosi** e fare domande.

Spesso è anche un obbligo: se si viene assunti per fare un gioco ambientato nell'antica Roma, bisogna conoscerla; verrà dato del materiale, ma bisognerà anche fare ricerca per conto proprio su usi e costumi. Non si sfugge. Ma si potrebbe anche voler partire da concetti lontani dalla sensibilità comune, o inserire cose curiose che possono catturare un certo tipo di utenza.

Approfondimento: lo *slow boil*. In *Challenges for Game Designers*, Brathwaite e Schreiber descrivono diversi approcci al game design. Uno è il *blue sky*, senza altri vincoli che l'immaginazione del designer (raro, perché di solito si lavora sotto vincoli esterni). Un altro è quello che chiamano *slow boil*: dato un tema e un'ambientazione (il loro esempio è "gli irlandesi nella Boston del 1880"), il designer avvia una massiccia missione di ricerca, con poca direzione purché copra il tema. Come dice uno di loro, "mi riempio la testa di tutto quello che trovo sull'argomento, e poi aspetto: un design emergerà". Reiner Knizia, ricordano, ha regolarmente una trentina di progetti in corso e preferisce rigirarli a lungo nella mente prima di passare al prototipo; e un errore comune dei designer alle prime armi è forzare un gioco troppo presto. È la stessa curiosità che il docente raccomanda con lo *sharing*.

[38] Le *wicked questions* sono una delle *Liberating Structures* (sezione 4.12).

[39] Nella trascrizione compare come "Trits". Il TRIZ è la teoria per la soluzione inventiva dei problemi sviluppata da Genrich Altshuller; il SIT ne è una derivazione semplificata.

[40] Nella trascrizione il primo titolo compare come "Blackboard"; la lettura *Bloodborne* è la più plausibile.

5.15.12 Ad absurdum

Si prendono il problema o la necessità, il contesto, gli stakeholder e così via, e li si **spinge al massimo** e **al minimo**; si risolve il caso estremo nei due sensi. Teoricamente, poi, si valuta la via di mezzo (si veda anche il ricorso alle situazioni estreme nella client pitch direction analysis, sezione 4.10).

5.15.13 Che cosa resta da vedere

Resta il *come* si fa brainstorming, ma il docente decide di lasciarlo al venerdì: "abbiamo quasi concluso". A quel punto dà il task, "perché altrimenti mi restate a bocca asciutta per troppo tempo".

5.16 Il task della settimana

Il brainstorming non è ancora stato visto per intero, ma la slide c'è, e se ne riparlerà. Il task si fa **a gruppi**: gruppi di due, tranne uno di tre.

I gruppi Francesca e Antonin[41]; Federica e Alberto; Francesco e Lorenzo; Valerio, Mattia e Daniele. Il docente si confonde un paio di volte nell'assegnarli ("quattro gruppi, oggi veramente fuori di testa") e chiede che qualcuno gli rimandi i gruppi per messaggio.

I giochi I primi due gruppi lavorano su **Marvel Snap**; gli ultimi due, compreso quello da tre ("vi beccate la roba più difficile"), su **Yu-Gi-Oh! Master Duel**. Sono due **digital card game**, non TCG, perché le carte non si possono scambiare; *Marvel Snap* è il più facile. Per *Master Duel* c'è una sola banlist, "credo". La ricerca va naturalmente limitata a questi giochi: è già un grosso lavoro.

La ricerca Tramite ricerca su internet (Reddit, Discord, dove si vuole) bisogna identificare **almeno tre problemi o necessità**: tre *pain point* che il target audience di riferimento ha individuato come problematici. Vanno trovati i dati (il feedback), analizzati, spacchettati: qual è il problema?

L'analisi Il problema va analizzato il più possibile con gli strumenti visti: identificato ed eventualmente categorizzato (con Cynefin, tabella 4.4, o con le Liberating Structures, sezione 4.12). Si può usare la perceptual mapping (sezione 5.5), anche per vedere come i competitor vicini hanno eventualmente risolto la stessa cosa. In ogni caso va documentata la parte di **dati**, la prima parte della lezione.

La soluzione Poi si genera una soluzione. Il consiglio del docente è usare una delle tecniche più semplici: **opposite direction**, **limitation** o **break the theme** (sezione 5.15). Sconsiglia SIT ed euristica, perché richiederebbero tanto lavoro di ricerca, tempo che è meglio dedicare a capire che cosa dicono i dati dell'utenza.

Il documento di brainstorming Ci deve essere un **documento di brainstorming** in cui si butta giù di tutto ("permettetemi il francese, cazzate"), ci si "tira la pizza addosso". Quel documento **non deve seguire le golden rules** (sezione 3.7) e resta agli studenti; ma va creato, per mostrare di aver lavorato. Il docente non lo leggerà prontamente.

Il documento consegnato Il docente leggerà il documento prodotto, composto da: i **tre pain point** dell'utente, **come** sono stati trovati, i **dati** spiegati e il **design** per risolvere ciascuno dei tre.

Scadenza Il docente concede un giorno in più del solito: **venerdì della settimana successiva**, due giorni in più, anche perché nella lezione sul brainstorming arriveranno altre informazioni da inserire. Intanto si può cominciare a fare ricerca, ragionare, scambiarsi i numeri di telefono e organizzarsi.

La lezione si chiude con le ultime domande e con l'appuntamento a venerdì, **in presenza**, dove si concluderà la parte su come fare brainstorming; sciopero permettendo, con la previsione che ci saranno comunque ritardi nei treni.

[41] Il nome compare così nella trascrizione; potrebbe essere troncato (Antonino, Antonio).

Punti chiave del capitolo

- Il **production dictionary** non ha limiti di pagine nelle produzioni vere; può essere un file a sé o un paragrafo d'apertura ibridato in un altro documento; i termini li sceglie il lead, li riempiono molte mani, e ce n'è **uno per progetto**.
- **Documenti di visione** (production dictionary, documento di visione, pitch) contro **documenti di produzione** (i GDD). Il GDD unico non esiste più: dalla "Bibbia" si è passati alle wiki e a strutture che smistano i lettori verso i documenti giusti, curate dalla produzione.
- Deming: il ciclo PDCA e le **sette malattie mortali**, tra cui dirigere un'azienda basandosi solo sui dati visibili.
- I framework di design (core elements, game qualities, Bura) servono anche ad **analisi** e **pitching**.
- L'**elicitazione triadica**: tre competitor, il target audience raggruppa i due più simili e spiega perché; si cerca l'elemento dominante, i pain point e l'insight.
- La **perceptual mapping**: assi come spettri, per trovare combinazioni innovative e buchi di mercato; un vuoto non è un invito; **un grafico non basta**. Utile anche nel design (nemici: DPS, HP, velocità, melee/ranged) e nel brainstorming.
- L'**analisi degli utenti** è tra i punti più difficili: dati quantitativi, gusti, routine; target dominante e demografiche attorno; il feedback online è **polarizzato**; nessun gioco è per tutti.
- Survey: brevi (**meno di cinque minuti**), domande semplici e puntuali, molto più di cento risposte; i paganti li finiscono. Poi archetipi, osservazione e la mappa *dicono, fanno, pensano, provano*. **Senza empatia non c'è design**.
- L'**analisi di mercato**: spazio, dimensione, saturazione, stato emotivo (soulslike saturi, monster collectible scontenti), trend (sempre in ritardo), barriere, tasse, logistica.
- Le **dodici motivazioni** di Quantic Foundry in sei gruppi (action, social, mastery, achievement, immersion, creativity), ognuna uno spettro; si descrive un gioco con le **dominanti**. L'unico *fun* ammesso è quello delle quattro chiavi di Lazzaro.
- **Correlare** i dati senza cadere nei falsi pattern; smontare le proprie ipotesi; arrivare a insight ed empatia.
- La pre-produzione si chiude con le domande su che cosa si vuole e si può fare, che cosa funziona e non funziona, che cosa vogliono gli stakeholder e il business: la risposta è la **game vision**.
- Le idee sono inutili senza contesto: generarne tante, scartare le prime, attenzione alla **legge del martello**, brainstorming in gruppo e senza giudizio, poi severità e soluzioni *future proof*. Il **design elegante** è desiderabile ma non obbligatorio.
- Tecniche: categorie sensoriali, **opposite direction**, **break the theme**, euristica, what if, wicked questions, SIT e TRIZ, **limitation**, drawing e storyboard, sharing, ad absurdum.
- Task a gruppi: tre pain point di *Marvel Snap* o *Master Duel*, con dati, analisi e soluzione (opposite direction, limitation o break the theme), più un documento di brainstorming; scadenza venerdì della settimana successiva.

Chapter 6

Dal brainstorming ai flowchart: condurre un brainstorming, flowchart, grafici e matematica di base

Con questo sesto capitolo si chiude, finalmente, la lunga lezione su *problema e soluzione* cominciata due capitoli fa (capitolo 4) e proseguita nel precedente (capitolo 5). Mancava l'ultima parte: *come* si conduce un brainstorming, con quali regole, quali tecniche e come si passa da cento idee a una manciata di idee valutate. Chiusa quella, la giornata cambia argomento e apre un blocco nuovo, dedicato agli strumenti **visivi** della documentazione: i **flowchart**, i **grafici** e quel poco di **matematica di base** che serve per costruirli e leggerli.

La lezione, un venerdì pomeriggio a ridosso di Halloween, si tiene da remoto ed è condotta ancora da Mattia Podini. Il percorso del capitolo ne ricalca la struttura:

1. le domande in apertura, ancora una volta sul **production dictionary** di Pokémon: che cosa si intende per *nomenclatura*, come ordinare le voci, che cosa lasciare ad altri documenti (sezione 6.1);
2. la conclusione della parte sul **brainstorming**: le condizioni per farlo bene, il **facilitatore**, le pause e il cibo (sezioni 6.2 e 6.3);
3. altre **tecniche** per generare idee, dai *sei cappelli* al **TRIZ**, le regole dell'innovazione di **Guy Kawasaki** e lo **SCAMPER** (sezioni 6.4 e 6.5);
4. come si passa **dalle cento alle dieci idee**: la *brainstorming matrix*, il confronto, il tentativo di distruggere le proprie idee, il prototipo e le routine da spezzare (sezioni 6.6 e 6.7);
5. i **flowchart**: che cosa sono, dove nascono, a che cosa servono a un designer, le regole di lettura, i blocchi, l'obbligo di chiudere il flusso, e una serie di esempi reali che le regole le piegano (sezioni 6.8–6.13);
6. il task della settimana: il **flowchart completo di Pac-Man** (sezione 6.14);
7. i **grafici**: un piccolo esercizio in aula (linea o gradini?), i grafici di base, quelli più ricchi e la mappa della campagna di Russia di Napoleone, "uno dei grafici più belli mai fatti" (sezioni 6.15–6.19);
8. la **matematica di base**: media, media pesata, proporzioni (sezione 6.20).

6.1 In apertura: ancora il production dictionary

6.1.1 Prima di cominciare

L'avvio è il solito avvio di una lezione online: il docente non sente la classe ("probabilmente perché non ho ancora condiviso lo schermo"), segna le presenze, saluta chi entra. Poi il rituale: domande o dubbi sulla

lezione precedente, che oggi si conclude, o sui task in generale, “anche perché il production dictionary scade lunedì” (il task del capitolo 4, sezione 4.14).

Problemi di gameplay, prima di tutto. La prima domanda riguarda invece il task a gruppi sui *pain point* (sezione 5.16): i tre problemi principali trovati devono riguardare il gameplay o possono toccare qualsiasi feature del gioco, per esempio il negozio? Qualsiasi tipo di feature, chiaramente. Il docente però **sconsiglia** di prendere un problema **artistico** o un problema di **performance**: se davvero non si trova altro, lo si può includere come punto finale, ma la priorità va ai **problemi di gameplay**.

6.1.2 Che cosa si intende per nomenclatura

Mattia, uno studente, confessa di non aver ancora capito la consegna del production dictionary. Ha provato a lavorarci in una cartella personale, ma si è accorto che stava scrivendo e *spiegando meccaniche*, e ha il sospetto che non sia quello che viene chiesto. “Che cosa si intende per nomenclatura?”

Il docente ricorda di averlo già ripreso all’inizio della lezione precedente (sezione 5.1) e lo rispiega. L’obiettivo è spiegare **nomi unici legati al gioco**, o alla produzione del gioco. Possono essere nomi di meccaniche, e nel caso di Pokémon lo possono essere tranquillamente; ma più spesso sono nomi che spiegano un oggetto del mondo, un *game object* del mondo di Pokémon. L’esempio è il **tipo** (o elemento): si spiega che cosa sono i vari tipi, che cos’è il tipo di un Pokémon.

Che cosa *non* ci va. Negli esempi mostrati nelle slide, come la voce *Chase* (lo stato di inseguimento di un’intelligenza artificiale, sezione 4.3), è chiaro che quello stato non verrà spiegato in un documento dedicato solo a lui: sarà spiegato nel documento delle IA. Lì si trovano le spiegazioni **profonde**: come funziona la meccanica, che parametri ha, che cosa richiede lato programmazione, lato arte e così via. Nel production dictionary finisce invece **a che cosa è legato quel nome**, a livello descrittivo: una o due righe, una descrizione succinta. Lo scopo è che il lettore, di qualunque natura sia (artista, grafico, programmatore, designer), sappia **a che cosa si riferisce quella parola**.

La webcam, per favore. A metà spiegazione il docente chiede a Mattia di accendere la webcam: “già faccio fatica a leggere naturalmente i volti delle persone, figuriamoci se non ti vedo”; vuole capire se la spiegazione arriva o no. Il problema dello studente era proprio questo: quando si metteva a scrivere, gli veniva spontaneo descrivere le meccaniche, non le nomenclature.

6.1.3 Priorità: un limite di una pagina

Per tutto ciò che è estremamente specifico si fa riferimento ai **singoli documenti**. E gli studenti hanno anche il limite di **una pagina**, quindi devono per forza scegliere le nomenclature **più importanti** del gioco. Con molto più spazio si potrebbe discutere se spiegare meccaniche uniche, specifiche, o qualcosa di estremamente laterale.

Un caso limite. Spiegare una singola mossa molto particolare è un caso estremamente unico,[1] ben diverso dallo spiegare, per esempio, che cos’è una **MT** o una **MN**. E anche per le MT conviene separare due piani:

- **come funziona il sistema**: se una MT è consumabile o no, quante mosse ci sono e così via; questo può essere relegato ad altra documentazione;
- **che cos’è**: questo sì, è materia da dizionario.

Ancora più a monte si potrebbe spiegare che cos’è un *oggetto* di gioco, un item della borsa; ma “oggetto” è abbastanza comprensibile. L’ultima parola spetta agli studenti, ma il criterio del docente è questo: si parla di **nature uniche del gioco**, oppure **fraintendibili**. È questa l’utilità del documento.

[1] Nella trascrizione la mossa compare come “fuoco-bomba”; non è chiaro a quale mossa o meccanica ci si riferisca. L’esempio vale comunque come caso estremamente specifico.

Non è un errore, è un'altra scelta. Quello che allo studente veniva istintivamente da fare non è sbagliato di per sé: rispetto alla consegna è un'altra cosa, ma dal punto di vista della produzione non è un errore. Come detto l'ultima volta, il production dictionary si può **ibridare** con altri documenti (sezione 5.1): lo si può mettere all'inizio di un documento, oppure spiegare le singole terminologie in un GDD o in un altro tipo di documentazione, e allora quello è il documento che spiega quella meccanica. Se il gioco **non ha un grande rischio di fraintendimento**, si può anche valutare di avere un documento a sé che funge da dizionario: non è obbligatorio.

6.1.4 Raggruppare o ordinare alfabeticamente?

Intanto Lorenzo ha la webcam spenta: "vedo che Lorenzo si è affondato nell'erba alta ed è diventato un Pokémon". La parola passa a Valerio, che ha stilato una serie di elementi e, siccome erano tanti, li ha **raggruppati per fili comuni**. È sbagliato?

Dipende dal numero, dalla mole di informazioni che si danno al lettore. Su una pagina sola può funzionare; quale sia la scelta ideale il docente non può dirlo ("sennò poi dopo me la modificate"). Ma il ragionamento è questo, esattamente come per un dizionario:

- la **maggior leggibilità**, nel vuoto, ce l'ha l'**ordine alfabetico**;
- l'ordine alfabetico però non risolve un problema: alcune nomenclature sono **legate ad altre**. L'esempio, forse un azzardo, è *Pokémon* e *Poké Ball*: al lettore potrebbe essere utile sapere prima l'informazione più rilevante e subito dopo quella associata.

Nel caso degli studenti può contare meno; in production dictionary esageratamente grossi conta di più. In ogni caso, se il production dictionary c'è (e non è una necessità averlo nella produzione), **deve essere ordinato**.

I problemi di stato. Valerio ha anche parlato delle lotte, elencando e descrivendo tutti i **problemi di stato**: dovrebbero stare in un documento a parte, dedicato alle lotte? "Posso fare un esempio?" "Certo che puoi farlo, però io poi sarò libero di non risponderti." È più un problema di **priorità sulle nomenclature**; sul fatto di avere un documento a parte o meno, il docente preferisce non rispondere adesso: se ne riparlerà dopo che avrà commentato il documento.

Una piccola nota di colore chiude il giro: Mattia non sa come abbassare la mano su Zoom. "Non sono lo Zoom guru, quindi figurati."

6.2 Dove eravamo rimasti

6.2.1 La forbice che si allarga

Se non c'è altro, si conclude la lezione su problema, idea, ricerca e brainstorming. L'ultima volta ci si era fermati quasi alla fine: manca la parte sulle **metodologie per fare brainstorming**.

Il docente ricorda una cosa che vale per tutta la lezione: più si va avanti, dall'inizio verso la fine, più **la forbice si amplia**.

Nella ricerca non ci sono limiti, ma c'è almeno una struttura: una forma dei dati, delle priorità (target audience, competitor e così via). Dare priorità è soprattutto una questione di **tempistiche**: in pre-produzione il tempo sarà poco e ci sarà tanto da fare, quindi quanto più si riesce a coprire di quella lista non omnicomprensiva (la mappa della ricerca, sezione 4.7), meglio è. Alcune cose andrebbero fatte subito: i **competitor**, fondamentali, e l'**utente**, più di tutti; anche il **mercato** è importante. Le priorità ci saranno per forza, perché il tempo non è infinito. E per la ricerca strutture e framework ce ne sono, e sono abbastanza codificati: la lista vista (le Liberating Structures), la SWOT analysis, la perceptual mapping (sezione 5.5).[2]

[2]Nella trascrizione compaiono come "la lista di pesciola" (verosimilmente un riferimento storpiato alla lista delle Liberating Structures, sezione 4.12) e "swap analysis", che è quasi certamente la SWOT analysis (*Strengths, Weaknesses, Opportunities, Threats*).

Nell'ideazione e nel brainstorming le tecniche ci sono, ma il campo è **probabilmente infinito**. Si può adottare più o meno quello che si vuole, soprattutto a seconda del gruppo che fa brainstorming: non è detto che ci sia l'intero team, magari solo il team di design, o un *feature team* all'interno del team di design. A seconda di chi si è nel team e di quanto è grande, si può davvero fare quello che si vuole: "potete giocare a freccette, far venire in mente le idee, o giocare a biliardo, davvero qualsiasi cosa funziona".

Va bene tutto, **fintanto che porta un risultato**: quella "fantomatica" ideazione di **cento idee** da *sortare* e scartare (sezione 5.12).

6.3 Come si conduce un brainstorming

6.3.1 Le condizioni

Ci sono però alcune condizioni importanti.

Nessuno viene fermato. Durante il brainstorming, cioè nel momento in cui si generano le idee, **nessuno deve essere fermato** dal generarle. Non si scartano subito potenziali idee che "non c'entrano nulla": **le si elenca tutte**. Poi, eventualmente, ci sarà una fase, insieme o da soli, in cui queste idee vengono **filtrate**.

Tutti devono poter parlare. Tutti devono sentirsi nella condizione di dire la propria e di far crescere quella grande lista di idee, che alla fine dei conti è l'obiettivo: buttare giù davvero qualsiasi cosa. Anche perché quello che verrà scartato può ritornare in un'altra forma, in un'altra maniera, o anche solo far scaturire qualche idea nuova per implementazioni future. Qualsiasi spunto è utile: lasciare la parola a tutti, e nella fase di brainstorming **qualsiasi cosa va bene**.

Non solo designer. Sempre per arrivare a una quantità maggiore, e potenzialmente più **diversificata**, di idee, conviene avere nel brainstorming **non solo designer**, o comunque non solo persone con lo stesso background. A seconda di quanto sono disponibili e di quanto sono dentro l'industria, si cercano **punti di vista diversi**: l'obiettivo è ampliare la propria visione il più possibile.

6.3.2 Il facilitatore

Questo vale sicuramente per i team grossi, ma va valutato anche per quelli piccoli, soprattutto le prime volte, se c'è difficoltà a parlare o se si è meno portati a rompere il ghiaccio o a essere espansivi: conviene avere un **facilitatore**. Non un leader, ma qualcuno che si preoccupa di **controllare il flusso** del brainstorming: quanto tempo si sta spendendo a discutere di un'idea, o "a tirare quella freccetta".

Il facilitatore ha fondamentalmente **due obiettivi**:

1. **assicurarsi che tutti stiano bene** durante il brainstorming. A livello pratico vuol dire che tutti parlino e abbiano lo stesso tempo per parlare; ma c'è un passo in più, che tutti si sentano **a proprio agio**. Garantire la parola a tutti non basta necessariamente: a seconda dei membri del team, e di quanto li si conosce, l'obiettivo è **farli sentire ascoltati**;

2. **assicurarsi che il brainstorming abbia uno scopo pratico**. Fare brainstorming "è una figata, piace a tutti", ma il rischio di "buttarla in caciara" è molto grosso. È bello avere un momento di comunione con il team, più o meno regolamentato; tuttavia **bisogna quagliare**, arrivare a una lista lunga e concreta di idee. Il facilitatore si chiede quanto tempo va dedicato al brainstorming (un'ora?), controlla che si stia andando nella direzione corretta ed eventualmente **tira le redini** se qualcuno sta divagando.

Approfondimento: chi deraglia, rimette sui binari. Schell, tra i suoi sedici consigli pratici per il brainstorming (*The Art of Game Design*, capitolo 7), incoraggia a **fare battute**: le battute sciolgono la mente e fanno vedere le cose da prospettive che si erano perse, ed è dalle prospettive nuove che nascono le grandi idee. Ma avverte che, soprattutto in gruppo, le battute possono portare fuori strada; uscire dai binari ogni tanto va bene ("le idee buone potrebbero non essere sui binari"), purché ci si prenda la responsabilità di rientrarci. Il suo comandamento è: *he who derails, rerails*, chi fa deragliare rimette sui binari. È, in piccolo, il secondo compito del facilitatore distribuito su tutti i partecipanti.

6.3.3 Pause e cibo

Fare delle pause. Conviene avere **pause tra una fase e l'altra**. Al secondo anno si parlerà meglio di come affrontare la produzione, di come lavorare con altri esseri umani, dell'importanza delle soft skill; cose importanti, che il docente non vuole sminuire, ma il primo anno è il campo di prova. Le pause servono a ridurre "il cortisolo, mettiamola così": evitare che le persone impazziscano o che comincino a percepirsi **screzi** forti tra gli individui.

È il discorso precedente visto dall'altro lato: se non si fa parlare tutti, e soprattutto se qualcuno comincia a **smontare le idee**, o fa già la fase di prendere un'idea e buttarla subito fuori dal cesto perché non ha senso con i limiti, con la visione o con quello che si vuole, qualcuno potrebbe "inalberarsi", prenderla male. E si ammetta pure che il team sia grosso e che ci siano già delle inimicizie al suo interno.

Portate il cibo. Per risolvere questo problema: pause, e poi **cibo**. Lo si vedrà in modo più strutturato al secondo anno, ma il mantra è: "portate il cibo, portate la pizza. Non lo so, quello che mangiano gli esseri umani generici. Io non mangio la pizza, però...". Generalmente avere delle pause in cui si mangia e si beve **aiuta notevolmente a ridurre la tensione**.

6.4 Altre tecniche per generare idee

6.4.1 Una lista, non un programma

I modi per fare brainstorming, per generare idee in generale, sono tanti. Il docente non li vede tutti punto per punto, ma li elenca; si aggiungono a quelli già visti nel capitolo precedente (sezione 5.15).

6.4.2 Circle

Ci si mette **in cerchio**, si scrive una parola, si va avanti. Si comincia con, per esempio, il tipo di gioco, una *game quality*, un tema, un'emozione; si passa il foglietto alla persona successiva, che aggiunge un'altra parola, e così via finché il giro non è finito. A volte si fanno più giri. È un modo per **ibridare la propria idea iniziale con quelle degli altri** e vedere che cosa esce.[3]

6.4.3 Acting brainstorming

Si **mette nei panni di un'altra persona**: nella slide, una persona famosa, un personaggio famoso. Ma vale anche, più in generale, per mettersi nei panni di qualcun altro: avere la capacità **empatica** di vedere come si è percepiti dall'altra parte, o che cosa si potrebbe provare dall'altra parte.

Fuori dal brainstorming. Il concetto è utile ben oltre la fase di brainstorming. Un esempio: un 3D artist o un concept artist consegna i suoi concept e gli vengono rimandati indietro, ogni volta, per una determinata ragione. È ovvio, o almeno molto probabile, che il suo **morale** si riduca. Mettersi nei suoi panni aiuta; poi bisognerà capire perché succede e come risolverlo. Mettersi nei panni dell'altro aiuta a spiegare alcune problematiche della produzione e del team: il morale basso, una persona che non è mai presente o che non parla mai durante i meeting. Come sempre, si cerca di arrivare alla **radice del problema** (sezione 4.12).

[3] Il docente lo associa, "se non sbaglio", a un gioco dei futuristi. Il meccanismo descritto (ciascuno aggiunge un pezzo senza controllare l'insieme) ricorda piuttosto il *cadavre exquis* dei surrealisti.

6.4.4 I sei cappelli

I **sei cappelli** vengono da un libro[4] e possono essere utili per diversi tipi di attività. Ogni cappello ha una **durata specifica** a seconda dell'attività ed è associato, un po' come nell'acting brainstorming, alla necessità di guardare o di agire da un determinato punto di vista:

Blu la *big picture*: si guarda il problema ad alto livello;

Bianco fatti e informazioni: si guardano i fatti, i dati;

Rosso più istintivo: *feelings and emotions*;

Nero il **giudizio critico**. Questo **rompe la fase di brainstorming**: è utile quando si comincia già a *sortare* le idee, "senza ciarle";

Giallo estremamente positivo: "yes", sorriso "a 34 denti", si va avanti;

Verde le **idee nuove**.

È chiaro quindi che la fase di brainstorming dovrebbe partire col **verde**, e comunque avere tanti momenti "col cappello verde". A volte si passa da una fase all'altra; la lista completa è nel libro, che non è necessario leggere, ma si trova anche in giro.

6.4.5 L'oggetto dal cappello

"Questo piace particolarmente a Diego", il direttore del corso, e serve a mostrare ancora una volta che nel generare idee si può infilare dentro qualsiasi cosa. Si sfruttano **oggetti particolari, gadget**, pescati da un "fantomatico" cappello, e si fa brainstorming su quell'oggetto: che cosa c'entra, e come può aiutare a risolvere il problema. "Utile? Non lo so, io non l'ho mai usato, però sta qua."

Approfondimento: i giocattoli sul tavolo. Schell propone qualcosa di molto simile tra i suoi consigli per il brainstorming (*The Art of Game Design*, capitolo 7, *Toys*): portare al tavolo dei giocattoli, alcuni legati al problema e altri che non c'entrano niente. Il suo paragone sono i ristoranti pieni di oggetti strani alle pareti: non sono decorazione, sono spunti di conversazione. I giocattoli coinvolgono la creatività anche in modo **tattile**; e se si porta un po' di plastilina per modellare le idee, "sembra stupido, ma la creatività è stupida".

6.4.6 Storyboard

Lo **storyboard** è un po' come il *drawing* (sezione 5.15): non importa saper disegnare bene o male, importa che sia **chiaro**, anche con gli stickman. Si disegna il processo, una potenziale storia che spieghi l'idea, che la "metta a terra" nel brainstorming.

6.4.7 Le domande

Domande: ci si butta dentro più domande possibili, su qualsiasi cosa. È anche un modo, più o meno serio, per iniziare quel tipo di "sortaggio" ("non sorteggio, sortaggio"), cioè di **valutazione** delle idee messe a terra.

6.4.8 Papà e mamma: peggiore è, meglio è

Un'altra tecnica importante è presentata nelle slide con una metafora familiare: da una parte il *papà*, la figura particolarmente **pragmatica**; dall'altra la parte in cui ci si butta dentro davvero qualsiasi cosa e va bene. Anzi: **peggiore è l'idea, meglio è**, almeno teoricamente. Sempre nell'idea che **tutto può aiutare**, e che la soluzione si può trovare partendo da qualsiasi cosa.[5]

[4] *Six Thinking Hats* di Edward de Bono (1985), noto in italiano come *Sei cappelli per pensare*.

[5] La trascrizione non permette di ricostruire con certezza il nome della tecnica nella slide. L'impostazione (cercare deliberatamente le idee peggiori) è quella nota come *worst possible idea* o *reverse brainstorming*.

Approfondimento: scrivere tutto, anche le idee stupide. Schell (*The Art of Game Design*, capitolo 7, *Write Everything*) cita Linus Pauling: "il modo migliore per avere una buona idea è avere molte idee". Il suo consiglio è di non aspettare l'idea brillante ma di scrivere **tutto** ciò che è anche solo lontanamente collegato al problema, ogni idea stupida che passa per la testa: bisogna far uscire quelle stupide prima che comincino ad arrivare quelle buone, e a volte un'idea stupida diventa l'ispirazione per un'idea geniale. Non ci si censura, e soprattutto in gruppo non si censurano gli altri: "le loro idee stupide valgono quanto le tue".

6.4.9 Associative e mind mapping

Con l'**associative** si parte da una parola e le si associano parole simili. Si può usare anche un *word association tool*, un'IA o qualsiasi software che associa parole.

È molto simile al **mind mapping**, che si vede di solito: "sicuramente l'avrete vista da qualche parte". Quando si parla di brainstorming, la visione pratica è generalmente questa: una **grande mappa sulla lavagna**, che tutti devono poter vedere. Si parte da un concetto, si dividono i rami per **categorie**, e i rami grossi portano a rami più piccoli, con parole o concetti sempre legati alla categoria principale (figura 6.1).

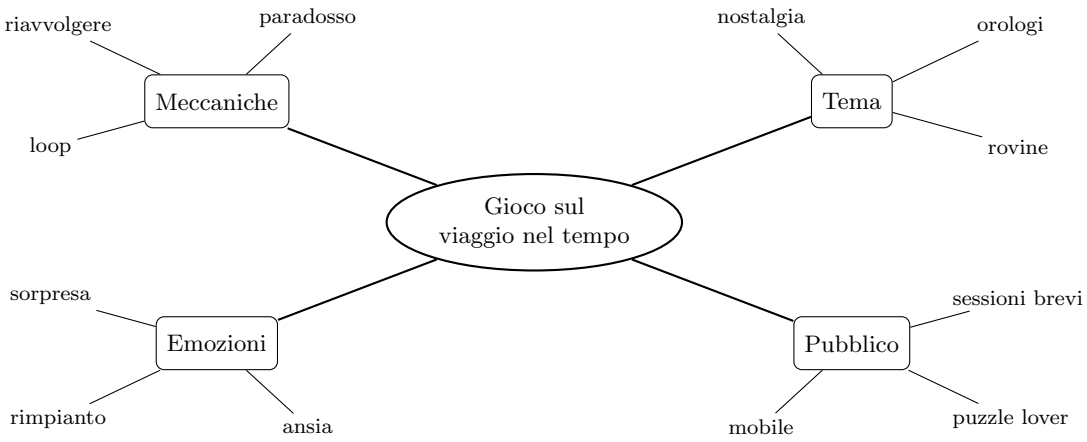


Figure 6.1: Struttura tipica di una mind map: un concetto centrale, rami principali per categoria e rami secondari sempre legati alla categoria da cui partono. Il contenuto è un esempio inventato.

Che cosa usare per il task. Per il task in corso (sezione 5.16) il mind mapping funziona molto bene, ma possono funzionare anche l'associative o il circle. Questi, i più **immediati**, sono quelli che il docente consiglia al momento.

Approfondimento: lo spazio ricorda. Due consigli di Schell (*The Art of Game Design*, capitolo 7) spiegano perché la mappa sulla lavagna funziona. Il primo (*The Writing on the Wall*): in gruppo serve una superficie che tutti vedano contemporaneamente; Schell preferisce grandi fogli adesivi o carta da pacchi alle pareti, che si possono riposizionare, staccare, arrotolare e conservare, per riprendere un anno dopo il brainstorming "come se non si fosse mai interrotto". Il secondo (*The Space Remembers*, espressione presa da Tom Kelley): la memoria per le liste è scarsa, ma quella per la **posizione** delle cose intorno a noi è ottima; appendere le idee nella stanza aiuta a ritrovarle e a vederne le connessioni. Un terzo consiglio è più sorprendente (*Number Your Lists*): **numerare** le liste le rende più facili da discutere ("mi piacciono dalla 3 alla 7") e, stranamente, dà alle voci una certa dignità, così che vengono prese più sul serio.

6.4.10 TRIZ

Il **TRIZ**, già accennato (sezione 5.15), viene solo "accennato" anche qui, perché è estremamente complesso. Nasce nell'**Unione Sovietica**. Il concetto è un po' quello dell'astrazione vista nel SIT, il *closed world*: il problema, per come viene formulato, contiene già la soluzione. Anzi, **non trovare la soluzione dipende**

dal fatto che il problema non è formulato correttamente. La soluzione è già lì; per arrivarci bisogna **astrarre il problema**, trovare **soluzioni astratte** e poi **concretizzarle**. "Questo è il salto."

Come funziona. Il TRIZ nasce dall'analisi di un numero enorme di **brevetti** sovietici, in cui il comune denominatore era astrarre i problemi e vedere come erano stati risolti. Problemi e soluzioni vengono messi in una **matrice** con righe e colonne: sulle righe e sulle colonne ci sono le stesse *feature*, da 1 a n ("sono 40, se non sbaglio, forse sono state anche aumentate nel tempo").[6] Le feature possono essere il peso di un oggetto in movimento, il peso di un oggetto stazionario, la lunghezza di un oggetto in movimento, la lunghezza di uno stazionario, l'area di un oggetto in movimento, e così via.

Se si riesce ad astrarre il proprio problema e a trovare la riga e la colonna su cui si fonda, si va a cercare la **cella** che le combina. Un esempio: il problema è mantenere caldo il contenuto di un bicchiere **senza far scottare le mani** di chi lo tiene. Il calore fa parte di quel tipo di feature: si cerca la cella che combina la colonna e la riga del problema.

Piegarsi alle regole. Il TRIZ si basa principalmente su **problemi meccanici**, e non è facile. La soluzione, se si usa il TRIZ, **deve derivare da quello che viene consigliato nella cella**: per usarlo non basta conoscerlo, bisogna anche piegarsi alle sue regole. Può essere un buon modo, se ci si leggono righe e colonne, soprattutto per **problemi pratici di produzione fisica**: per esempio nei giochi da tavolo, dove la stampa, la grandezza o il costo di una componente è eccessivo. Ma resta estremamente codificato: è tra le cose che si possono approfondire.

6.5 Le regole di Kawasaki e lo SCAMPER

6.5.1 Le regole dell'innovazione di Guy Kawasaki

Le **regole di Kawasaki** non vengono viste nel dettaglio: i punti sono segnati nella slide, per andare veloci, e chi vuole può guardare il video, "classico video di TEDx, informativo fino a un certo punto".[7] Alcune cose sono già state viste. Il docente le commenta così:

Make meaning (e non soldi). Se l'obiettivo è questo, l'obiettivo è portare **innovazione**; poi arriva il guadagno. Però non è detto: va valutato attentamente.

Make mantra Può essere utile durante il **pitch**: servono frasi che catturino l'audience o che facciano capire il prodotto in poche parole.

Jump to the next curve Far **evolvere** la propria idea: iterare, ma in positivo, sempre in modo migliorativo.

Roll the DICEE Creare qualcosa di elegante, completo, profondo, intelligente, non semplice.[8] Lo si può paragonare al **design elegante** (sezione 5.13) che, ripete il docente, non è sempre necessario, ma "è lì".

Don't worry, be crappy Creare **prototipi**: arrivarci il prima possibile e **non avere paura di fare schifo**.

Let a hundred flowers blossom Lasciare che **nuove dinamiche** emergano. Applicato ai videogiochi: che emergano nuove meccaniche, o che i giocatori vedano e usino in modo diverso le cose che si erano previste. Con il **playtest** si valuta se il giocatore fa qualcosa a cui non si era pensato, che non si

[6] Nella formulazione classica di Genrich Altshuller la *matrice delle contraddizioni* incrocia 39 parametri tecnici (righe: il parametro che si vuole migliorare; colonne: quello che peggiora di conseguenza), e ogni cella suggerisce alcuni dei **40 principi inventivi**. Versioni successive (come la *Matrix 2003*) hanno esteso i parametri. Nella trascrizione il nome compare come "Drizzt", "TRIPS" e "TRIZZE".

[7] Guy Kawasaki, *The Art of Innovation*, TEDxBerkeley, 2014. Nella trascrizione diversi titoli delle regole sono storpiati ("Con la nice people", "Charm baby charm", "Nishita e Sensor", "Dot worried e crappy"); qui sono riportati nella forma originale.

[8] L'acronimo di Kawasaki è DICEE: *deep, intelligent, complete, empowering, elegant.*

pensava possibile, e se ha senso all'interno della propria visione. Generalmente assecondare l'utente quando legge qualcosa a modo suo (ammesso che sia il proprio target audience) è positivo: non sempre, ma nella stragrande maggioranza dei casi. Il controllo del progetto resta però al team: è il team a decidere **se quel bug diventa una feature o no**.

Polarize people Se ne riparlerà nella lezione sul feedback, ma se ne è già cominciato a parlare. Le persone, statisticamente, su un argomento si distribuiscono su una **curva a campana**. Interessa capire chi sono gli **estremi**: chi ama alla follia il gioco e chi, invece, "vi vuole sgozzare". Va bene **non creare il prodotto per queste persone**; ma sapendo che cosa vogliono o non vogliono, si danno le priorità. Non bisogna convertire tutti: bisogna convertire la maggior parte delle persone, magari quelle **incerte** se comprare o no il prodotto. Chi lo odia non è necessario convertirlo.

Churn, baby, churn Ancora iterazione, ma **tagliando**, andando "per sottrazione": un po' quello che gli studenti devono fare nei loro task.

Niche thyself Lo stesso discorso: dedicare il gioco a una **specifica utenza**, cercare di conoscerla ed evitare il resto.[9]

Perfect your pitch Il solito discorso iterativo, lato pitch: come lo si presenta, come lo si espone (si veda il paragrafo seguente).

Ripetere il pitch, ma non a memoria. Se ne riparlerà quando si parlerà di pitch ("o forse lo ridirà Diego, perché dovrebbe farla lui la lezione quest'anno"), ma il docente lo anticipa. Chi è più timido nell'esposizione deve dedicare **attenzione, tempo e fatica** a **ripetere il pitch**, a perfezionarlo, a perfezionare il modo in cui lo espone: come magari si è già fatto per esami o interrogazioni. Ma c'è un ma: **il pitch non si può imparare a memoria**, e non andrà mai esattamente come previsto dall'inizio alla fine, perché alla fine ci saranno **domande** che si possono prevedere solo fino a un certo punto. Si possono lasciare volutamente aperte alcune cose, non dirle durante il pitch perché vengano chieste dopo, ma è **rischioso**. Il pitch deve avere una **componente di improvvisazione**: bisogna saper leggere le persone che ascoltano, capire se si stanno annoiando, ed eventualmente **deviare la rotta** anche a metà.

6.5.2 SCAMPER

Lo **SCAMPER** è un mantra, un acronimo: *Substitute, Combine, Adapt, Modify, Put to another use, Eliminate, Reverse* (sostituisci, combina, adatta, modifica, trova un altro uso, elimina, inverti). Alcune cose sono già state viste: combinare, sommare, togliere, sottrarre, dividere sono operazioni del SIT e delle regole euristiche (sezione 5.15).

6.6 Dalle cento alle dieci idee

6.6.1 La brainstorming matrix

Un'altra cosa molto utile per le idee è, fondamentalmente, una **perceptual mapping per le idee** (sezione 5.5). Vale più a livello **produttivo** che operativo, ad alto livello. Su due assi cartesiani si mettono **l'impatto** (alto o basso) e **l'effort** (basso o alto, cioè fondamentalmente il costo). Si cerca di prendere le idee ad **alto impatto e basso costo**; e si va a vedere anche una o più idee ad **alto impatto e alto costo**, che magari sono veramente innovative o estremamente stupefacenti (figura 6.2). È un altro modo di usare la perceptual mapping: la *brainstorming matrix*, "chiamatela come volete". È chiaro che qui si sta già facendo **sorting** delle idee.

[9] Nella trascrizione il docente aggiunge "stesso discorso del Dargate Open"; il riferimento non è ricostruibile.

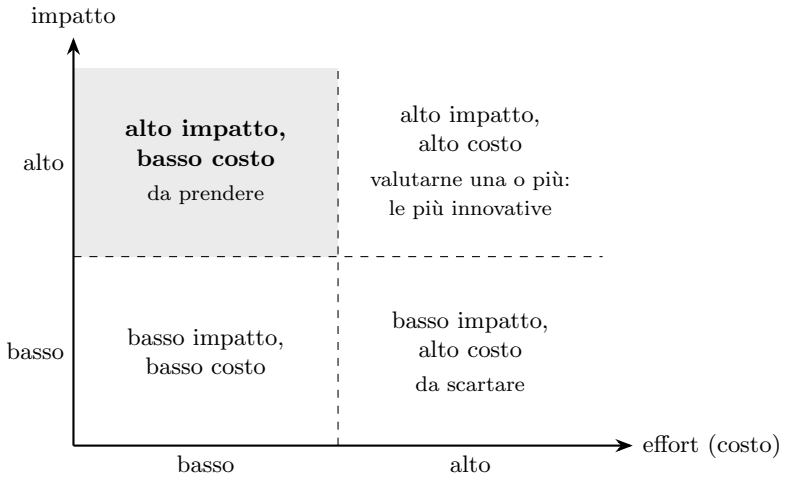


Figure 6.2: La *brainstorming matrix*: una perceptual mapping per le idee, con impatto ed effort (costo) sui due assi.

6.6.2 Almeno dieci idee diverse

Alla fine del brainstorming si dovrebbero avere **almeno dieci idee totalmente diverse**, già *sortate*, già valutate. Come sempre è una provocazione: non per forza dieci, ma si mira a quel numero. Da cento si deve **ridurre drasticamente**.

Si confronta ciascuna idea con le altre e si trova **il miglior compromesso possibile** (sezione 4.5): "è il nostro lavoro, è quello che si fa sempre". Le domande sono quelle di sempre (sezione 5.11):

- che cosa vorrebbe il **target audience**?
- che cosa il team **può** fare, e che cosa **vuole** fare?
- qual è la **visione**?

Il team che non amava le corse. Sul "vuole fare" il docente racconta un caso. Per il progetto annuale, un team aveva scelto un **gioco di corse**: era nel reame della fattibilità, aveva senso secondo il target audience e secondo una serie di altri criteri. Ma a nessuno nel team piacevano le corse, né le macchine in generale. La **motivazione** poi può diventare **determinante**: sta al team valutare se lasciarla a casa o includerla nel processo. E la visione, se si parla di *team vision*, può includere anche questo: che cosa **sprona** il team, qual è il suo *ultimate goal*.

6.6.3 Distruggere le proprie idee

"Questo non smetterò mai di dirvelo": bisogna sempre cercare di **distruggere le proprie idee**, smontarle, confutarle. Tutte le idee possono essere confutate, in misura più o meno grande. L'obiettivo non è trovare dei contro per poi scartare automaticamente l'idea: le idee si valutano, anche con la brainstorming matrix. Ogni design ha per forza dei **compromessi**, dei punti, "non dico ciechi", in cui è più debole; se quei contro sono accettabili, va benissimo.

Se non si trovano difetti. Il vero campanello d'allarme è il contrario: se non si riescono a trovare **problematiche**, difficoltà nella messa a terra dell'idea, allora c'è un problema da qualche parte. Come regola estrema, quell'idea va scartata, ignorata completamente. Oppure si dedicano altro tempo e altre risorse: non la si cerca di distruggere da soli, si chiede il parere di altri per trovare questi "fantomatici contro". Occhio, però: **non esiste l'idea perfetta**. Si trovano i punti problematici dell'idea, poi si valuta se sono compromessi accettabili o no, li si confronta e si trova il miglior compromesso possibile.

Approfondimento: distruggere i propri presupposti. Schell (*The Art of Game Design*, capitolo 7, *Destroy Your Assumptions*) riporta un consiglio del designer Rob Daviau: fare la lista di tutto ciò che si sta dando per scontato sul proprio gioco ("assumo che si giochi al chiuso", "assumo che il giocatore guardi lo schermo", "assumo che usi un solo dito"), e poi, voce per voce, chiedersi come funzionerebbe il gioco se quel presupposto non fosse vero. Quasi sempre il presupposto regge; ma ogni tanto farlo saltare dà un'intuizione enorme. Daviau è arrivato a *Risk Legacy* distruggendo il presupposto standard dei giochi da tavolo: "una partita non influenza la successiva". È lo stesso spirito con cui il docente chiede di smontare le idee, applicato però alle fondamenta invece che alle idee stesse.

6.7 Prototipi, movimento e routine da spezzare

6.7.1 KISS e il cuore del gioco

Come sempre, soprattutto all'inizio: **KISS**, *keep it simple, stupid*. L'obiettivo di un **prototipo** è **testare in piccolo** quello che si ha. Può essere brutto, può essere "terrificante" da giocare e da vedere: non è quello il problema, l'obiettivo è testarlo. Si ha un'idea di gioco: si comincia a sviluppare il **cuore**, l'anima, il **loop a basso livello** (del *game loop* si parlerà in futuro, sempre con il docente). Si trova il cuore del gioco e si testa subito quello; poi, piano piano, lo si espande, si aggiunge, si itera, si vede che cosa funziona, si migliora, si aggiunge un altro pezzo, e così via (sezione 3.5).

Approfondimento: un prototipo risponde a una domanda. Schell (*The Art of Game Design*, capitolo 8) raccoglie dieci consigli per la prototipazione, e i primi tre suonano come un commento a questa slide.

- **Rispondere a una domanda**: ogni prototipo deve essere pensato per rispondere a una o più domande, formulabili con chiarezza ("il gameplay di base è divertente? Lo resta a lungo?"); se non le si sa formulare, il prototipo rischia di diventare uno spreco di tempo invece dell'esperimento che fa risparmiare tempo.

- **Dimenticare la qualità**: conta solo che risponda alla domanda, e più in fretta lo fa meglio è. Un prototipo rifinito può addirittura peggiorare le cose, perché chi lo prova segnala più volentieri i problemi di qualcosa che sembra grezzo, e un prototipo lucidato nasconde i problemi veri.

- **Non affezionarsi**: Fred Brooks, in *The Mythical Man Month*, scriveva "pianificate di buttarne via uno: lo farete comunque". Ogni prototipo è un'occasione per imparare.

Il nono consiglio, **costruire prima il giocattolo**, è il "cuore del gioco" del docente: assicurarsi che il giocattolo sia divertente prima di costruirci sopra un gioco. Schell riporta due racconti di David Jones: *Lemmings* nacque da un piccolo mondo pieno di creaturine che sembrava divertente di per sé; e *Grand Theft Auto* nacque come una città viva, divertente da esplorare, a cui solo dopo si diedero le meccaniche di un labirinto prese da un gioco che si sapeva funzionare: "GTA viene da *Pac-Man*. I puntini sono le personcine, io sono nella mia macchinina gialla, e i fantasmi sono i poliziotti". (Di *Pac-Man* si tornerà a parlare molto presto, sezione 6.14.)

6.7.2 Muoversi, usare i sensi

Altra slide: **esagerare** (fino a un certo punto), **muoversi**, usare il corpo, usare tutti i sensi. Il concetto vale anche per quando si farà *tuning*: non si resta fermi a discutere di un'idea per tutto il giorno, chiusi in una stanza con le stesse persone, perché questo **falsa il giudizio**: ci si abitua a qualcosa che non è accettabile. Come quando si fa il **bilanciamento** di una meccanica: se lo si fa per tutto il giorno, costantemente, ci si può abituare a un dato o a un parametro che non è l'esperienza desiderata.

Quindi ci si ferma: si esce, si mangia, si beve, si gioca a biliardo, a ping pong, a quello che si vuole. Si fanno **altre attività** e si guarda il gioco **da altri punti di vista**.

Approfondimento: cambiare prospettiva. Anche questo è uno dei consigli di Schell (*The Art of Game Design*, capitolo 7, *Change Your Perspective*): non fare brainstorming solo seduti sulla propria sedia, "salite in piedi sulla sedia, da lassù le cose sembrano diverse"; andare in posti diversi, sull'autobus, in spiaggia, al centro commerciale, in un negozio di giocattoli. Tutto ciò che accende l'immaginazione e fa pensare cose nuove vale la pena di essere fatto.

6.7.3 Rituali che spezzano la routine

Lo si vedrà durante la produzione, ma conviene dedicare **momenti specifici**, magari **ritualistici**, per rompere le routine, perché i colleghi non facciano sempre la stessa cosa per lungo tempo. È una cosa che in Italia, purtroppo, fanno in pochi ("e non capisco perché non lo fanno tanto in Italia; c'è un motivo per cui alcuni prodotti non sono grandiosi"), mentre all'estero è molto adottata:

- momenti dedicati alla **merenda insieme**, durante la settimana: si porta la cassa di mele e tutti la mangiano;
- a livello più pratico, il **venerdì pomeriggio**, tardi, quando tutti sono stanchi, si fa **playtest**: si smette insieme e si gioca insieme il prodotto, cercando le problematiche.

Il playtest del venerdì. Un ex collega del docente lavorava in uno studio[10] in cui, durante la settimana, si implementavano magari meccaniche e sistemi nuovi; per il venerdì si preparava una sorta di **missione o puzzle ad hoc**, in modo che tutti poi la giocassero. Questo spezza molto il ritmo e dà modo al gioco di essere **testato**, che è poi l'obiettivo: far testare il gioco, prendere feedback, iterare, e così via.

Lo stand-up. Si possono quindi introdurre una serie di rituali che rompono la routine. Si vedranno le metodologie, ma per esempio con **Agile** c'è uno **stand-up mattutino** che tutti fanno ogni giorno, e che tendenzialmente è **odiato**, per quanto utile sia. Bisogna trovare il miglior compromesso possibile perché le persone siano invogliate anche a fare le cose obbligatorie.

6.7.4 Chiusura della parte su problema e idea

Non c'è altro su come generare idee. Il docente invita a fare mente locale sul task: se qualcosa non torna, va chiesto. E ripete il consiglio: nella fase di brainstorming conviene usare **le metodologie più semplici** viste, perché non va sottovalutata la mole di dati e di ricerca iniziale, e poi anche la documentazione del design; la prima volta che lo si fa è così. "Basta, quindi questo è per problema e idea."

Cinque minuti di pausa, poi un argomento nuovo, "magari noto ad alcuni, ad altri no": i **flowchart**, particolarmente utili per il discorso dello *show, don't tell*.

6.8 Flowchart: che cosa sono

6.8.1 Matematica? Poca, e di base

Al rientro dalla pausa (Lorenzo, "perso negli arbusti", riemerge "dall'erba alta" poco dopo) il docente presenta il blocco nuovo: flowchart e altre questioni "da smarcare lato matematico". Rassicura chi non è avvezzo alla matematica: per ora si parla di **matematica di base**, degli unici strumenti veramente basilari che servono (sezione 6.20). Lui stesso ammette di aver perso, negli anni di lavoro, molte delle conoscenze matematiche acquisite all'università: "peccato, è un compromesso, uno scambio".

Chi li conosce già. Quanti conoscono i flowchart? Lorenzo, e Antonino, che li ha già trattati nella vecchia scuola. Il docente se lo aspettava: per chi ha un passato o competenze più tecniche, i flowchart non sono una novità. Si riparte comunque da zero. E con un avviso: **non si useranno tutti i blocchi** dei flowchart, perché ce n'è un'infinità; ne sono stati aggiunti nel tempo e, a seconda dell'industria in cui si usano, possono assumere anche significati diversi. Servono i blocchi di base, "i mattoncini Lego basilari".

[10] Nella trascrizione il nome dello studio compare come "Daisy" e "Easy"; non è ricostruibile con certezza (una possibilità è DICE).

Per parlare con i programmatori. I flowchart sono particolarmente utili per **comunicare un'idea ai programmatori**. La slide mostra il solito meme sulla documentazione, con un flowchart e un "amico giapponese" che chiede a che cosa serva; la risposta: il flowchart serve come **integrazione alla documentazione**, oppure può essere un documento a sé (difficile, ma può funzionare).

6.8.2 Una definizione

Un **flowchart** è una **rappresentazione grafica formalizzata**: ha delle regole, sta sotto delle regole, e serve a spiegare processi o *workflow* in generale.

La slide ne mostra due esempi. Il secondo, la prima immagine che esce su Google cercando "flowchart" (forse la stessa di Wikipedia), è più corretto del primo, più strutturato nell'uso dei blocchi, ma ha una mancanza: "una minuzia, che minuzia non è". I blocchi hanno forme diverse e anche colori diversi; il colore non è una necessità, conta solo la **forma** ("ma essendo designer, poi lo estetizzeremo in qualche modo"). Ogni blocco ha un ruolo specifico: da qui si parte, qui si finisce, qui c'è una biforcazione, qui ci sono delle azioni. Le regole sono molto semplici, "state tranquilli", ma sono necessarie.[11]

6.8.3 Dove nascono

L'ambito in cui i flowchart vengono usati di più è lo **sviluppo software**, il coding in generale e le **produzioni tecniche**, perché si tratta di mettere a terra il **processo logico** di qualcosa. È come avere già uno sketch, una spina dorsale, o addirittura l'intero scheletro di come dovrebbe funzionare un processo: come funziona una pagina web, come è implementata un'intelligenza artificiale (che cosa deve fare, quali sono i suoi task), come dovrebbe funzionare un'interfaccia in un videogioco. E qui, "sublimando" piano piano verso i videogiochi: da dove si inizia, dove porta un tasto del menu, dove ne porta un altro, che cosa succede se si preme questo o quello. I flowchart nascono lì; i designer se ne appropriano.

6.9 A che cosa servono a un designer

Intelligenza artificiale I designer li usano per mettere a terra le **decisioni dell'IA**. Se ne parlerà brevemente l'anno prossimo; per ora il docente mostra un esempio (sezione 6.13), con un avvertimento scherzoso: "non abbiate paura, come gli angeli quando arrivano con le mille ali e gli occhi e vi dicono: non temete".

Produzione Si usano per esplicitare la **pipeline**: quello mostrato l'altra volta, e brevemente anche nella primissima lezione (sezione 1.2). Il producer che passa una cosa a qualcun altro, chi la passa a chi: la pipeline, la gerarchia, e come si dovrebbe sviluppare il lavoro nel team. Il flowchart è in realtà il modo più efficiente e più chiaro per farlo.

Design Servono come **supporto alla documentazione**. I modi più comuni: spiegare i **game loop** (che si vedranno), le **sequenze di gameplay** (dove si va, quale sequenza porta dove, che cosa porta a che cosa) e il **player journey** (dove deve andare il giocatore, e così via).

UX e UI Più comunemente ancora li usano gli **UX e UI designer** (la nomenclatura qui è "ballerina": UX designer, UI designer, UI artist...). Spiegano, e si fa sempre, la **relazione tra le singole interfacce**: dal main menu dove si va, che cosa fa un tasto, dove porta il journal, che cosa contiene, e così via.

L'obiettivo finale è **spiegare sistemi complessi, o relazioni tra sistemi, in maniera chiara**: il flowchart è la semplificazione di tutto il documento, quello in cui poi si spiegano punto per punto le singole cose. Un'immagine a supporto del documento è la cosa migliore che si possa fare: è la prima regola della documentazione, *show, don't tell* (sezione 3.7). L'obiettivo è **comunicare con gli altri dipartimenti in maniera chiara**.

[11] La trascrizione non dice quale sia la mancanza. Alla luce di quanto detto più avanti (sezione 6.12), è verosimile che si tratti di un ramo che non si ricollega alla fine o di una decisione con uscite non etichettate.

6.10 Le regole di lettura

6.10.1 Prima le si impara, poi le si piega

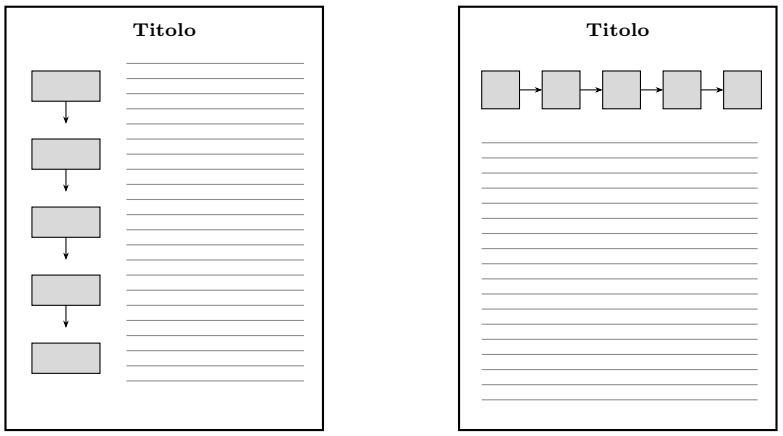
Una premessa: queste regole **non sono così vincolanti**. Si vedranno flowchart ed esempi che le rompono completamente. Se per il coding è meno vero, nel design va bene **fintanto che è comprensibile**, per chi lo fa e per chi lo riceve. Non bisogna preoccuparsi di rompere alcune regole in favore, per esempio, dello **spazio** nel documento o della **leggibilità**. Come sempre, però: prima le regole si conoscono, si imparano, *poi* si rompono.

6.10.2 Dall'alto in basso, da sinistra a destra

Il flowchart si legge **dall'alto verso il basso** e **da sinistra a destra**: in quest'ordine di priorità. Inizio, decisione, step 1; decisione, step 2, fine.

Il flowchart verticale e il wall of text. Il docente lo mostra alla lavagna (draw.io, con il titolo in testa alla pagina). Un flowchart scritto dall'alto verso il basso, che si estende verticalmente, inserito in un documento occupa una fascia stretta e lunga, "siamo abbondanti, un terzo di pagina"; nei due terzi restanti c'è tutto il testo, costretto a essere **estremamente schiacciato e bislungo**. Ne nasce il rischio del **wall of text** (sezione 3.7), "che ancora non ho commentato, ma sicuramente commenterò perché lo troverò nei vostri documenti". Gli studenti forse non lo temono ancora abbastanza: "fate male, lo dovete temere, e mi premurerò io di farvelo temere". Lo stesso numero di righe che, in un contenuto compatto, starebbe in poco spazio, si allunga nel documento e ne **riduce la leggibilità**.

Il caso è estremo, ma la soluzione è semplice: lo stesso contenuto può essere organizzato **da sinistra a destra**, in orizzontale, e funziona lo stesso (figura 6.3). Va valutato di volta in volta, sapendo però da dove arrivano le regole.



verticale: testo schiacciato, documento che si allunga

orizzontale: stesso contenuto, testo a piena larghezza

Figure 6.3: Lo stesso flowchart inserito in una pagina in verticale e in orizzontale. Lo schizzo ricostruisce la dimostrazione fatta alla lavagna.

6.10.3 Livelli e frecce

Lo stesso livello, la stessa gerarchia. Un'altra regola, non prioritaria (i designer che usano i flowchart di solito la rompono, per ragioni di formattazione): le cose che stanno **sullo stesso livello** hanno la **stessa gerarchia**. Step 1 e step 2, se allineati, sono allo stesso livello: aiuta a scandire le azioni, quando succedono e con che priorità. È poco rilevante per quello che devono fare gli studenti, ma è bene saperlo.

La direzione delle frecce. Sembra ovvio, ma va esplicitato: si legge **nella direzione in cui va la freccia**. Da *start* si va giù, perché la freccia punta in giù; poi a destra o a sinistra. In un flowchart che si sviluppa da sinistra a destra, le frecce andranno di conseguenza.

6.11 I blocchi

6.11.1 Inizio e fine

I blocchi di **inizio** e **fine** sono, nella forma più corretta, dei **rettangoli arrotondati** (*rounded rectangle*): *start* ed *end*. A volte si usa un cerchio (o un'ellisse), ma il cerchio potrebbe essere confuso con il blocco di **attesa** di n secondi, che qui non si vede: per questo il docente lo ha eliminato. Non è strettamente necessario, ma il rettangolo arrotondato è più corretto.

Non si sfugge. Un flowchart **deve per forza cominciare con un inizio** e **concludersi attaccandosi alla fine**. Dall'inizio parte la freccia principale; la fine deve essere collegata. *Start* ed *end* si possono anche rinominare (si vedrà come e in che condizioni).

6.11.2 Il rettangolo: un'azione

Tutto ciò che è un **rettangolo** è un'**azione da compiere** nel processo. Un blocco rettangolare molto semplice: inizio, "premi A", fine. "Premi A" è un'azione.

6.11.3 Il rombo: una decisione

Il **rombo** (o diamante) è una **domanda**: tutto ciò che è una decisione con potenzialmente più risultati. La condizione più semplice è **sì o no**. Un esempio (figura 6.4): inizio; attacca il giocatore 2; il giocatore 2 ha gli HP maggiori di zero? Sì, succede una cosa; no, ne succede un'altra.

Uscite esplicite Le uscite del rombo **non vanno numerate**, ma rese **esplicite**: la freccia in uscita deve riportare il risultato della risposta alla domanda. Sì/no, *true/false*, 1/0, quello che si vuole, ma **sempre etichettate**.

Il sì in basso Formalmente, con una decisione vero/falso, il **sì** esce dal vertice **basso** e il **no** dal vertice **destro**. Anche questo non è estremamente importante.

Più uscite Le uscite possono essere più di due: risposta 1, risposta 2, risposta 3, e una quarta, una quinta, e così via. Ma le risposte devono avere **priorità sui vertici**: con due risposte si usano due vertici, con tre i tre vertici, e non si fa uscire una risposta da un punto qualsiasi del lato, "per buon uso e costume".

Ci si fa quasi tutto. Queste sono le basi: con inizio e fine, il blocco azione e il rombo "ci fate quasi tutto. Veramente, qualsiasi cosa". Ci sono aggiunte utili, altri blocchi, ma non è detto che vadano usati in ogni flowchart: "teneteli, sappiatelo".

6.11.4 Il sotto-processo

Il primo blocco aggiuntivo è il **rettangolo con i bordi verticali doppi**: un'**azione con sotto-processo**. Il rettangolo semplice è un'azione semplice; questo è un'**azione complessa**. Che cosa sia semplice e che cosa complesso lo si sceglie; sta di fatto che questo blocco **fa risparmiare spazio**. Se si fa il flowchart di un grosso sistema, e al suo interno ci sono sotto-regole già spiegate altrove, in altri flowchart, si possono riassumere grossi pezzi con questo rettangolo. La regola però è ferrea: **il sotto-processo va spiegato in un altro flowchart**. Per forza.

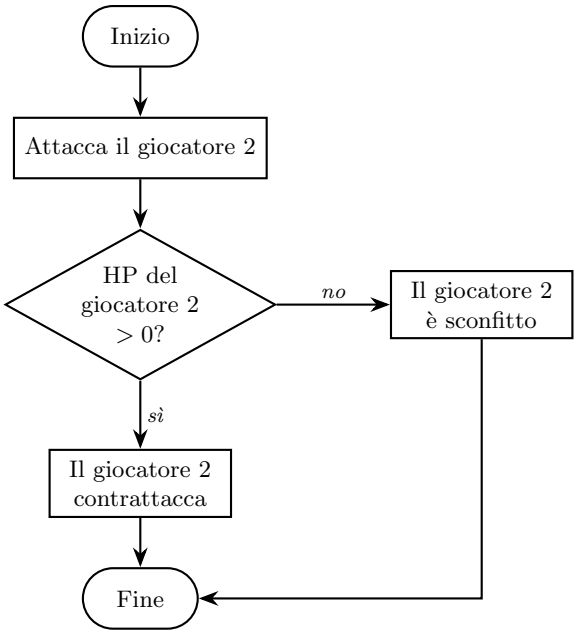


Figure 6.4: I tre blocchi fondamentali: inizio e fine (rettangoli arrotondati), azione (rettangolo), decisione (rombo) con uscite etichettate, il sì in basso e il no a destra. Gli esiti delle due uscite sono inventati.

La domanda di Valerio: i finali di Detroit. Valerio chiede se lo si potrebbe usare in un gioco a scelte multiple con finali diversi, come *Detroit: Become Human*, dove una scelta porta da una parte e un'altra da un'altra. Sì: se l'obiettivo è quello, si possono semplificare parti dell'albero, perché il motivo per cui si usa questo blocco è proprio **tenere compatto** il flowchart in alcune parti. Se si volesse fare il flowchart della narrativa di *Detroit* e interessasse solo uno dei finali, tutto il resto si potrebbe mettere in questi blocchi: si scrive "finale 1", non lo si spiega, e lo si collega alla fine.

Il docente lo mostra alla lavagna, da sinistra a destra (figura 6.5): start; succede qualcosa ("non lo so, cazzi"); una decisione ("anche senza il punto di domanda"), con tre esiti, A, B e C. Il finale A interessa relativamente: si mette il blocco *ending A* e lo si collega alla fine, "sarà molto più in là"; lo stesso per il finale C. Magari interessa spiegare solo il finale B, e allora si mettono i suoi blocchi, uno dopo l'altro. Ma il buon uso e costume dice che, se si fa una compressione del genere, i finali compressi vanno spiegati in **un altro flowchart**: uno che comincia con "ending A start", e che magari porta a un altro evento, a una sezione di gameplay, e così via.

"Ho risposto alla tua domanda, Valerio?" Lorenzo sembra perplesso, ma il problema è un altro: stava cercando di leggere la lavagna dalla miniatura invece che a schermo intero. D'ora in poi gli esempi si fanno su draw.io, così li si vede tutti e in modo più leggibile.[12]

Un sistema di salvataggio. Nella slide c'è un altro esempio: un sistema di salvataggio. Premi A; sovrascrivi il file?; file 1, file 2, file 3; fine. Tutta questa sequenza si potrebbe condensare in un unico blocco di sotto-processo, per esempio se si dovesse spiegare che cosa fa un tasto, o un'azione complessa dentro un menu, che si spiegherà altrove.

6.11.5 Il parallelogramma: input e output

Poi c'è il **romboide**, il parallelogramma: si usa quando il gioco deve **leggere o scrivere un input specifico**, cioè **lato macchina**. Nell'esempio della slide: leggi gli HP; HP maggiori di zero? Se vero, *outcome* A; se falso, *outcome* B (figura 6.6).

[12] Nella trascrizione compare come "Bro.io" e "Drop.io"; lo strumento è draw.io, già indicato tra gli strumenti di documentazione (sezione 3.8).

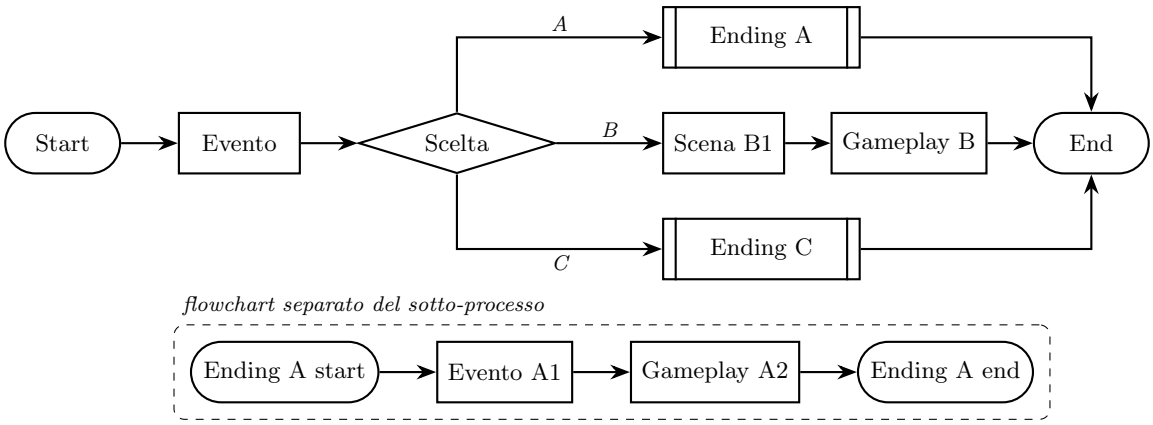


Figure 6.5: Compressione di rami narrativi con il blocco di sotto-processo (rettangolo con i bordi verticali doppi). Ogni sotto-processo va poi spiegato in un flowchart a sé, come quello tratteggiato in basso.

6.11.6 Il documento

L'ultimo blocco è il **documento**: qualsiasi documentazione associata. È un po' il concetto del sotto-processo, ma porta **altrove**: ci si clicca e si viene mandati al documento specifico. Magari una tabella su ciò che si può personalizzare, una tabella dei singoli punteggi, la leaderboard. Può esserci un documento solo o più documenti.

Come lo collega il docente. Sempre nella logica per cui alcune regole si possono piegare, il docente mostra il proprio modo, "non necessariamente scelta vostra": collega il documento al flowchart con una **linea senza freccia**. Per due ragioni.

1. Il documento non deve leggerlo la *macchina*. Il designer fa il flowchart perché lo leggano **altri esseri umani**: quel documento spiegherà delle regole che, eventualmente, il programmatore dovrà leggere. Non fa parte del flusso.

2. Se il flowchart descrive un loop di gioco, un sistema di gioco o un comportamento dell'IA, una freccia che entra in un documento sarebbe un ramo che porta fuori dal sistema: ci si aspetta invece che il sistema non sia "rotto" (si veda la sezione seguente).

(La lavagna di draw.io, intanto, è in modalità scura: "forse per la mia sanità mentale è meglio tenerla".)

Approfondimento: il catalogo dei blocchi. Riassumendo, i blocchi visti a lezione sono sei (tabella 6.1); i primi tre bastano per quasi tutto.

Forma	Significato	Note
Rettangolo arrotondato	inizio / fine	obbligatori; il cerchio si confonde con l'attesa
Rettangolo	azione	"premi A"
Rombo	decisione	uscite sempre etichettate; sì in basso, no a destra
Rettangolo con bordi doppi	sotto-processo	va spiegato in un altro flowchart
Parallelogramma	input/output lato macchina	"leggi gli HP"
Documento	documentazione associata	il docente lo collega senza freccia

Table 6.1: I blocchi del flowchart usati nel corso.

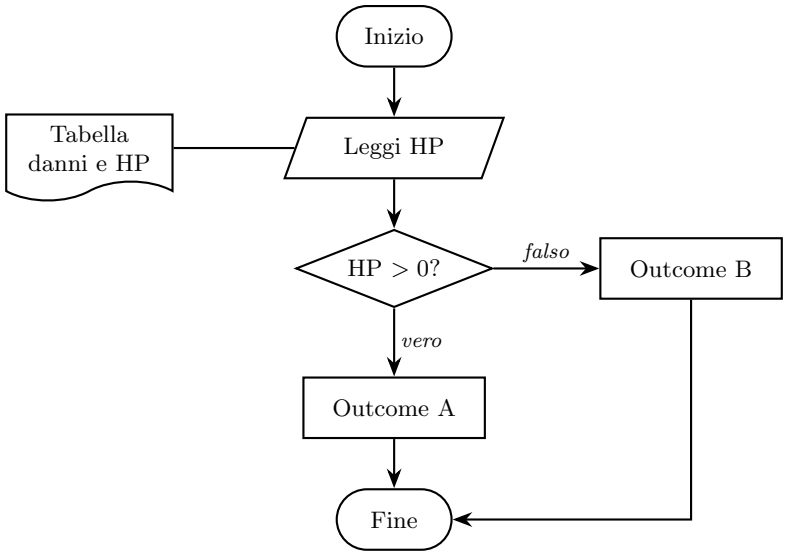


Figure 6.6: Il parallelogramma (lettura o scrittura di un dato lato macchina) e il blocco documento, collegato con una linea senza freccia come fa il docente: il documento è per i lettori umani, non fa parte del flusso. Il contenuto del documento è un esempio inventato.

6.12 Il flusso deve chiudersi

6.12.1 Che cosa c'è di sbagliato?

Il docente disegna un flowchart e chiede che cosa non va: inizio; "check qualsiasi roba"; se sì, A, poi la fine; se no, B. "Dipende da quello che sto documentando, ma assumiamo che io stia facendo un sistema di gioco o il comportamento di un'IA: questa cosa è sbagliata." Qualcuno si sente così *bold* da ipotizzare il perché?

Non solo **B non porta alla fine**: B è proprio lì, **nel nulla**. Una volta che si entra in B, non si esce più. Si immagini il comportamento di un'IA: se vede il giocatore fa una cosa, se non lo vede ne fa un'altra. Se lo vede fa la sua cosa, magari non finirà lì, ma farà qualcosa e poi **ritornerà** al controllo. Se invece fa B, l'IA **resta bloccata nello stato B** e non andrà da nessun'altra parte.

O si chiude, o ritorna indietro. Questo assumendo di usare i flowchart con le loro regole di base (con un *behavior tree*, si capirà, il discorso è diverso). Ma per un sistema di gioco o per un loop di gioco, **per sua stessa natura il flowchart deve loopare**: o **si chiude**, o **ritorna indietro**. Se vede il player, attacca; se non lo vede, torna a camminare. Le IA, per esempio, funzionano esattamente così (figura 6.7). Va bene anche che A finisca e B torni al controllo, "ammesso che in A qualche volta ci andrà". E alla lavagna mancava pure un'etichetta: "*yes*, *no*... ha scritto due volte *yes*. Porca miseria, terrificante."

6.12.2 Gli stessi blocchi, in tante salse

Come funziona l'IA di un gioco come *Minecraft*? Probabilmente con un *behavior tree* molto più semplice di altre IA, come gli NPC di *Mafia* o di *GTA*. Per descriverne il comportamento si userebbe un'altra applicazione del flowchart, ma con gli **stessi blocchi**. Il vantaggio è proprio questo: i blocchi sono **codificati**, e una volta capito come funziona un flowchart standard lo si può riapplicare n volte, in n salse diverse. L'importante è che sia **comprensibile** e che **affianchi e spieghi meglio** ciò che si documenta per filo e per segno nel GDD, nel documento specifico di design, in quello che si vuole.

Frecce che si sovrappongono. Alla lavagna ("ho distrutto tutto") il docente aggiunge una precisazione: due frecce possono **sovrapporsi** fintanto che vanno entrambe **nella stessa direzione**. Sarebbe un errore, invece, avere una punta che finisce su una freccia e un'altra punta che finisce dentro un blocco nello stesso punto. Con un po' di esperienza, mettendoci le mani, sarà più chiaro.

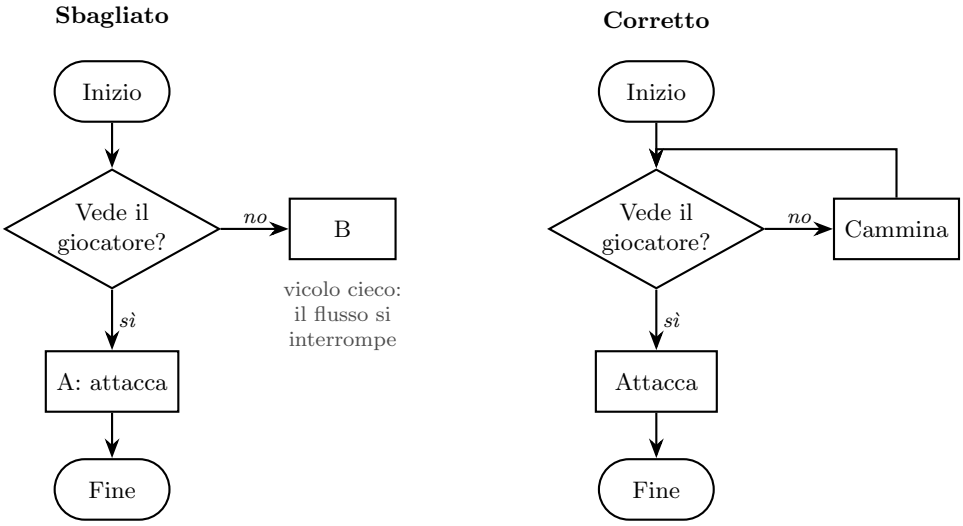


Figure 6.7: A sinistra il flowchart sbagliato: il ramo B resta nel nulla e un'IA che ci entra non ne esce più. A destra la versione corretta: il ramo "no" torna al controllo, quindi il flusso o si chiude o ritorna indietro.

6.12.3 Riepilogo e una domanda

I blocchi veramente importanti con cui si costruisce tutto sono quelli visti: inizio e fine, azione, decisione, sotto-processo, a cui si possono aggiungere eventualmente documento e parallelogramma. Ce ne sono altri, ma con questi quattro si può spiegare, più o meno, **qualsiasi sistema** in maniera chiara e coerente.

Il flowchart come documento? Uno studente chiede come funziona il documento: il flowchart è un documento? Di per sé no. Un flowchart si crea di solito **per affiancare qualcosa**, per spiegare qualcosa. Flowchart a sé stanti esistono, per esempio per spiegare la **produzione**: la pipeline, la gerarchia dell'azienda; o per spiegare come sono collegate tra loro tutte le **UI** di gioco. Ma il design di un nemico, gli stati di un nemico, di solito non si spiegano *solo* con un flowchart: il flowchart è affiancato a un altro documento, o incluso nel documento, di fianco o sotto il testo, **per potenziare quello che si dice**. L'obiettivo è sempre chiarire: invece di scrivere, la cosa migliore è **mostrare**. Scrivere di meno, e mostrare le relazioni dei sistemi o il loop di gioco con il flowchart.

Approfondimento: flowchart e diagrammi di stato. Schell (*The Art of Game Design*, capitolo 12) propone un parente stretto del flowchart: il **diagramma di stato**. Gli oggetti dei videogiochi, e soprattutto quelli che simulano personaggi intelligenti, hanno così tanti attributi e stati che è facile confondersi; conviene disegnare, per ogni attributo, quali stati sono collegati a quali e **che cosa fa scattare** ogni cambio di stato. Il suo esempio è proprio l'attributo "movimento" dei fantasmi di *Pac-Man*: lo stato iniziale "nella gabbia"; poi "inseguono Pac-Man" quando è ora di uscire; "blu, in fuga" quando Pac-Man mangia una power pill; "lampeggianti, in fuga" quando il timer sta per scadere, e di nuovo all'inseguimento quando scade; "occhi che tornano alla gabbia" quando vengono mangiati, e di nuovo nella gabbia quando gli occhi arrivano. Le frecce sono le **transizioni** possibili, ciascuna con l'evento che la provoca; e implementarle come macchina a stati vieta automaticamente le transizioni illegali (da "nella gabbia" a "blu"), riducendo i bug. Schell ne ricava una lente (#28, *The Lens of the State Machine*) con quattro domande: quali sono gli oggetti del gioco? Quali i loro attributi? Quali gli stati possibili di ogni attributo? Che cosa fa scattare i cambi di stato? Rispetto ai flowchart della lezione cambia la notazione (nel diagramma di stato i nodi sono *stati*, nel flowchart sono *azioni* e *decisioni*), ma lo scopo è identico: costringere a pensare a tutto ciò che può succedere e a che cosa lo fa succedere. Ed è la stessa regola del flusso chiuso: in un diagramma di stato ben fatto nessuno stato è un vicolo cieco.

Approfondimento: nodi e archi nel level design. Un uso simile dei diagrammi esiste anche nel level design. In *An Architectural Approach to Level Design* Christopher Totten riprende la *molecule design* di Luke McMillan e Nassib Azar: gli spazi di gioco (scontri, raccolta di oggetti, punti di spawn) sono **nodi**, e le relazioni tra loro sono **archi**, con un linguaggio visivo dichiarato: linee tratteggiate per gli spazi che si vedono l'uno dall'altro, linee piene per quelli tra cui ci si può muovere, frecce per i passaggi a senso unico. Come nei flowchart, il diagramma non è la pianta del livello: è la descrizione di *come* gli spazi interagiscono.

6.13 Flowchart nella pratica: esempi che piegano le regole

"Prima cosa, cercate di imparare le regole." Poi il docente mostra quattro flowchart reali, tre dei quali fatti da lui, che le regole le usano e le piegano.

6.13.1 Oxenfree: misurare il peso della narrativa

Il primo è un flowchart usato dal docente insieme a un collega per fare **analisi**: è *Oxenfree*, già citato la volta precedente (tra gli esempi di *easy fun*, tabella 5.2).[13] Per spiegarlo, il docente mostra un paio di minuti di gameplay, "ammesso ma non concesso che il video collabori" (lo schermo resta nero, poi carica, poi si abbassa la qualità a 720p).

Il gioco. A parte la *gimmick* della radio, il sistema più notevole di *Oxenfree* è il **sistema di dialogo**, unito al fatto che è un *teen horror drama*. L'obiettivo è sopravvivere all'isola e rompere il loop temporale che viene innescato; ma c'è anche la possibilità di fare *bonding* con il fratellastro o di essergli nemico, di far mettere insieme i due amici con cui si è sull'isola oppure di separare la coppia, e altro: tutte **interazioni narrative** del giocatore.

- Il dialogo è a **fumetti sopra la testa** dei personaggi: il giocatore ha una serie di opzioni tra cui scegliere.
- Le scelte sono **a tempo** (*time based*), senza che il tempo d'attesa venga mostrato in modo esplicito: quindi il giocatore **può anche non rispondere**. Uno dei rami è proprio il silenzio, mostrarsi "silente, misterioso, duro, forte e crudo".
- Le scelte (nella scena mostrata sono due, ma possono essere di più) aumentano un **conteggio di familiarità**: quanto si è amici o no di quelle persone, in questo caso quanto si fa bonding con il fratellastro.
- Come per *Detroit*, ci sono diversi **finali** possibili, in base non solo alle scelte narrative ma anche ad **azioni pratiche di gameplay**: se si è salvato o no un compagno, se si è riportato in vita o no qualcuno. I finali principali sono "tipo 4 o 5", il docente non ricorda.

L'analisi. Il flowchart serviva a vedere il **numero di interazioni narrative** rispetto ai segmenti di gioco: quanto è pesante la narrativa, quanto **carico cognitivo** si dà al giocatore, quante scelte ci sono, quanto è stato scritto e quanta scelta viene data. Nella slide c'è solo la prima parte, dal minuto 0 al minuto 5:40 del video, ed è **tutta una sola scelta**: non rispondere porta in una direzione, schernire il fratellastro in un'altra, essere amichevoli (quello che succede nel video, che è infatti linkato) in un'altra ancora; e ciascuna porta a situazioni di gameplay o a *flavor* narrativi. Le scelte "sono tante" e il documento completo è "infinito".

Questo flowchart **sfrutta ad hoc le regole** con una serie di abbellimenti: la divisione in segmenti, le informazioni esplicite sul numero di scelte, tutti i dialoghi, il totale delle *dialog choices*, e così via. E non è *color coded*: com'erano i flowchart alla nascita.

[13] *Oxenfree* (Night School Studio, 2016). Nella trascrizione compare come "Oxen Free".

6.13.2 Il gioco di carte: un flowchart per sé

Il secondo è un flowchart che il docente ha fatto per un **esperimento** con Diego: un gioco di carte, una sorta di **libro-game con un mazzo**. Si prende una carta, se ne applica l'effetto e si va a un'altra carta, A, B o C; le carte hanno fronte e retro. L'ambientazione è la classica **apocalisse zombie**.

- Si parte da **due carte** iniziali, e da lì si va ad altre carte.
- Ogni carta ha una serie di scelte: per esempio il parcheggio ha le scelte A, B, C e **S**, quella speciale, che generalmente richiede il **consumo** di qualcosa (la scelta S del parcheggio chiede di togliersi una granata, se la si ha).
- Oltre alle scelte narrative ci sono i **combattimenti** con gli zombie: si può combattere, perdere (e il gioco finisce) o scappare.

Perché è poco leggibile. Questo flowchart **serviva al docente**, e per questo potrebbe risultare poco leggibile agli studenti: doveva essere ordinato per lui, e nessun altro designer l'avrebbe letto se non Diego, con il suo aiuto (lavoravano sempre insieme sulle decisioni narrative). Può aiutare a mettere per iscritto una narrativa **ramificata** o un gameplay ramificato.

Regole rotte, regole garantite. Qui le regole sono rotte: gli *start*, per esempio, sono **due**. Ma le frecce sfruttano le stesse regole: una freccia, per quanto lunga, ha sempre **una sola punta**. Le regole di base sono garantite; ci sono poi alcune **libertà artistiche**, come una deviazione per evitare che una freccia finisca dentro un blocco e il flusso venga "annichilito", o meglio frainteso. Al netto delle regole di base (l'uso dei blocchi e delle frecce per come sono stati pensati) le libertà servono al designer per fare il design e il documento: "questo è piegare le regole". Prima le si impara, poi le si piega.

6.13.3 Machinations: un flowchart che si gioca

Il terzo esempio è nelle slide in PDF: cliccandoci si entra in un flowchart creato in **Machinations**, dove lo si può effettivamente **giocare**, testare, usare.[14] È il sistema **gacha** di *League of Legends*, come il docente lo ricorda da quando ha smesso di giocare:

- si **vince una partita**: c'è un tiro percentuale che può dare un **frammento di chiave**;
- se si è ottenuto un voto di S− o superiore sul personaggio, si ottiene uno **scrigno**;
- i frammenti si **accumulano**: con tre si forgia una **chiave**, e la chiave serve ad aprire lo scrigno;
- ci sono poi i **frammenti di skin**: ottenendone un tot si raggiunge la skin desiderata.

Premendo *play* si vedono i "pallini" percorrere il diagramma; dove il pallino non va avanti, si ferma, e i frammenti si accumulano.

draw.io o Machinations. Ci sono diversi modi per fare flowchart. Machinations è aperto al docente; nelle lezioni, compresa la prima di documentazione, si è visto **draw.io**, perché è **gratis**, è estremamente utile e si collega a **Google Drive**: di solito si usa quello, è molto più comune. Non lo si può "giocare", e proprio questo **obbliga a ragionare**, a verificare a mano se tutto torna. In Machinations, invece, ci sono elementi che nei flowchart standard **non esistono**.

Le regole sono un accordo. Bisogna capire che cosa è utile di volta in volta, e soprattutto se si lavora da soli o se il team condivide le stesse nomenclature e gli stessi usi dei blocchi. Ma è il designer a **condividere con il team** come usare il flowchart: se lo si fa, si può fare "tra virgolette" qualsiasi cosa, ammesso che le regole di base siano rispettate. La **leggibilità**, comunque, va oltre i flowchart.

[14] Nella trascrizione compare come "Macchinations". Machinations è uno strumento per modellare e simulare sistemi ed economie di gioco con diagrammi eseguibili.

6.13.4 Il behavior tree: il caso più estremo

Nella sua massima estensione, la cosa più "borderline" che si possa fare con i flowchart in ambito videoludico è il **behavior tree** di un'intelligenza artificiale, che sfrutta le stesse regole. Si legge sempre **dall'alto verso il basso e da sinistra verso destra**, sia quando lo si progetta sia quando lo si mette a terra (in Unity è così, e così era in Unreal quando aveva questo sistema in forma grafica).[15] Ma, come si vede, **non c'è una fine**: una volta che l'IA arriva a un blocco e fa quell'azione, o segue il ramo a destra, oppure si ferma e **ricomincia da capo**.

Il behavior tree è lo **standard** di programmazione dell'IA nei giochi, per un NPC o un nemico che sia. Nell'esempio della slide: si muove, attacca, sta in *idle*; se il personaggio è in range fa una cosa, altrimenti ne fa un'altra; e così via. Non si va oltre, perché lo si vedrà l'anno prossimo, con tutti i blocchi (cos'è un *sequencer*, e così via). Il punto, per ora, è che anche l'IA, nella massima estensione possibile, si può spiegare con un flowchart, **rompendo le regole standard** viste sull'inizio e sulla fine; ma prima bisogna aver capito quelle.

6.14 Il task della settimana: il flowchart di Pac-Man

6.14.1 La consegna

I task per **venerdì prossimo** sono due, perché c'è anche l'altro ancora aperto.[16] Quello nuovo è:

Che cosa Il **flowchart di Pac-Man**: **tutto** Pac-Man, l'intero gioco, documentato tramite flowchart.

Eccezione Daniele fa invece **Donkey Kong**, "l'arcade però".

Perché un arcade La documentazione di un intero gioco tramite flowchart oggi **non si fa più** ("non esistono più i cabinati"). *Pac-Man* viene dato perché è realistico riuscire a documentarlo tutto **a basso livello**, considerando potenzialmente tutte le interazioni, come ci si muove e così via; "e badate bene che non è semplice".

Come affrontarlo. Un consiglio basilare: **non partire con l'idea di fare il flowchart perfetto**. Si comincia a schizzare su un foglio, si prova a unire i blocchi, oppure si fanno **diversi flowchart** e poi si cerca di unirli. Per esempio, con l'aiuto della classe:

- un pezzo sul **movimento** di Pac-Man, come si muove, categoricamente;
- un pezzo su che cosa fa Pac-Man per **raccogliere punti** e oggetti: il sistema di **frutti** e oggetti, come si mangiano, che cosa succede quando li si mangia, e così le pillole e i pallini ("tanto qualcuno di voi lo starà già analizzando per i core elements", sezione 2.14);
- la **power pill**, che per esempio **cambia lo stato** di Pac-Man; e non solo il suo, ma anche quello dei **fantasmi**, che scappano e non lo inseguono più;
- i **fantasmi**, cioè l'intelligenza artificiale di Pac-Man, in cui ogni fantasma ha una leggera **variante nel comportamento**: il rosso "diventa un po' arrabbiato", il rosa dovrebbe fare una sorta di *flanking*, mirare dove Pac-Man sta andando. Che cosa fanno loro è un altro pezzo del flowchart.

Unirli dà l'intero gioco. Non c'è un modo giusto per farlo, e non è obbligatorio documentare le parti separatamente: si potrebbe dare più focus al movimento di Pac-Man come avatar, oppure agli input del giocatore. Ci sono n modi, ma devono essere **profondi**, dettagliati, perché devono descrivere l'intero gioco.

[15] Nella trascrizione l'engine compare come "ARIA" e "Ariane": si tratta di Unreal Engine (sezione 1.6). Il docente dice che Unreal "non ha più" questo sistema ma "un altro, con un altro nome, che è meglio"; il riferimento è verosimilmente agli *StateTree*, che Epic propone in Unreal Engine 5 accanto ai *Behavior Tree* tradizionali.

[16] Nella trascrizione: "l'altro avete il bonus di Tenny"; il riferimento non è ricostruibile.

Perché non si fa con GTA. Una cosa del genere si può fare con i giochi arcade del secolo scorso, con regole non esageratamente numerose: anche nei core elements, che il docente non ha ancora visto, gli studenti avranno segnato le regole, tutte, e sono tante ma gestibili. Al momento è improponibile pensare di fare il flowchart di *GTA*, e persino di un *Candy Crush* ("tosto, per il numero di livelli e di schermate parallele"). Di solito i flowchart si fanno **ad alto livello**: invece che dell'intero gioco in profondità, si fanno di una **feature**, di un **sistema** o dei **game loop**. Si guarda quali sistemi ci sono ad alto livello nel gioco, come sono interconnessi e come da uno si arriva a un altro: questo è l'uso comune oggi.

6.14.2 Le regole di consegna

Solo il flowchart Bisogna fare solo il flowchart di Pac-Man, nessuna documentazione oltre a quello.

Regole formali sospese Per questo task le regole della documentazione sono "tra virgolette" **sollevate** (sezione 3.9): non è obbligatorio usare il Roboto, si può scegliere qualsiasi altro font, **a patto che sia leggibile**. "Se mi mettete Comic Sans, io non lo leggo. Anzi: Papyrus. Comic Sans, *borderline*. Papyrus, non lo leggo."

Un'immagine commentabile Il risultato deve essere un'**immagine**: la si crea con draw.io o con qualsiasi altro strumento e la si esporta. Ma il docente deve poterla **commentare**, in modo moderatamente facile: la si può mettere in una presentazione Google o in un documento Google (che in questo caso non deve rispettare margini e formato).

Leggibile senza zoomare Il docente non deve dover zoomare ogni volta, aprire e tornare indietro. Esportata al 100%, "ragazzi, non leggo niente". Bisogna fare dei test: al **500%**, se il computer non esplode, dovrebbe essere ideale; dipende dalla grandezza del flowchart, e lui di solito esporta tra il **300% e il 500%**, mai sopra.

Attenzione allo sfondo Se si carica un **PNG senza sfondo**, il docente potrebbe vedere solo i blocchi e non le linee, e la leggibilità diventa un problema. Di nuovo: fare dei test.

Dove testare I test vanno fatti nel proprio Drive. Se li si fa per errore nel Drive del corso, bisogna avvisare il docente, che li cancellerà: "non testate tutto lì, perché poi devo tornarci".

Uno o più flowchart? Uno studente chiede se il flowchart debba essere unico per tutto il gioco, o se se ne possano fare di separati per meccanica. Va fatto **tutto il gioco**; ma c'è un modo furbo per fare quello che chiede, avendo un flowchart di tutto il gioco e insieme altri flowchart che descrivono altre cose, "e l'abbiamo visto". "Se ci pensi, hai già la risposta. O ti lascio pensare."[17]

Approfondimento: da dove partire. In *21st Century Game Design*, Chris Bateman e Richard Boon usano proprio *Pac-Man* come esempio di *game world abstraction*, cioè l'insieme delle regole del mondo di gioco: che cosa è il mondo, che interazioni ha il giocatore, come è rappresentato. La loro sintesi è quasi un indice dei sistemi da coprire. Il mondo è una serie lineare di labirinti 2D; Pac-Man si muove nei labirinti e guadagna punti mangiando i puntini, e quando sono finiti comincia un nuovo labirinto; i fantasmi colorati cercano di raggiungerlo e, se lo toccano, perde una vita; se mangia una power pill i fantasmi diventano blu, possono essere mangiati e cercano di scappare; i fantasmi mangiati diventano occhi che tornano al centro del labirinto e ridiventano fantasmi; mangiarne più di uno di fila dà punti in modo esponenziale; alla fine dell'effetto i fantasmi lampeggiano di bianco prima di tornare invulnerabili; ogni tanto compare la frutta, che dà punti extra; e sia Pac-Man sia i fantasmi possono uscire dai bordi del labirinto e ricomparire dal lato opposto. "Questa", scrivono, "è praticamente l'intera astrazione del mondo di gioco di *Pac-Man*". Ogni frase è un candidato a un pezzo di flowchart, o a un sotto-processo; il lavoro del task è scendere **più in profondità** di così, fino alle singole decisioni.

[17] Il docente non svela la risposta. Alla luce della lezione, la strada più naturale è il blocco di **sotto-processo** (sezione 6.11): un flowchart generale dell'intero gioco i cui blocchi complessi rimandano ad altri flowchart, uno per sistema.

6.14.3 Informazioni di servizio

Dopo un altro *coffee break* di cinque minuti, il docente dà un paio di informazioni di servizio.

- **Lunedì** non c'è lui, c'è Alessandro: la **prima lezione di programmazione**, in presenza. Chi non ha un computer portatile con **Unreal** installato (come Federica) lo installi; si può seguire da casa con un fisso, ma l'invito è ad avere il computer con Unreal già pronto.
- **Mercoledì** c'è Stefano con level design.
- **Venerdì** si ritrova il docente.

6.15 Grafici: linea o gradini?

6.15.1 La domanda

Si passa ai **grafici**, con una "domanda di rito". Sullo schermo ci sono due grafici: a sinistra un **grafico a linea**, a destra un **grafico a gradini** (*step chart*). Bisogna ignorare la nomenclatura dei dati e guardare solo la forma, e quello che può rappresentare. Il caso: descrivere gli **addominali** (o i push-up, "qualsiasi cosa") fatti in una settimana, con i giorni sull'asse delle ascisse e il numero di ripetizioni sulle ordinate: 5 il lunedì, 6 il martedì, 7 il venerdì, e così via. Quale dei due è il migliore per mettere a grafico questi dati?

Nessuno risponde, quindi il docente costringe a schierarsi: chi è della fazione *linea* alzi la mano, chi della fazione *gradini* pure. Tutti scelgono i gradini, e ora devono dire perché.

6.15.2 Le risposte della classe

Mattia Il dato è un numero intero e i giorni sono **discreti**: non avrebbe senso un valore intermedio tra un giorno e l'altro. "Spiegazione interessante."

Antonino Il grafico a gradini è più **leggibile** e chiaro: con le sue linee orizzontali e verticali si capisce meglio, mentre quello a linea è un'unica linea che sale ("non necessariamente verso l'alto, però sì"). Il docente, "così su due piedi, nel vuoto", direbbe invece che il grafico a linea è il più leggibile; il che non toglie che anche il grafico a gradini possa mostrare la tendenza con una **linea di tendenza**.

Lorenzo La linea potrebbe essere **fuorviante**, perché **interpola linearmente** da un punto all'altro: a mezzanotte tra lunedì e martedì il grafico suggerisce un valore intermedio, la "media" tra il giorno prima e il giorno dopo. Il grafico a gradini, invece, considera il valore **costante** nell'intervallo: giorno X questo, giorno Y quest'altro. Il grafico a linea è però ottimo per fare **previsioni**: "la settimana prossima potrò fare tanti addominali". Il grafico a gradini è più per chi si allena e dice "ho fatto questo"; quello a linea per il personal trainer, o in vista di una gara ("in dieci anni arriveremo a tanti addominali").

Valerio Il grafico a gradini aiuta a vedere se c'è una **progressione**: se si fa sempre di più, se ci sono dei cali.

6.15.3 La risposta del docente

La tendenza si legge in entrambi i grafici, e si sta parlando "davvero di una sottigliezza". Ma quello che dice Lorenzo è corretto: la **discriminante** è il **salto** tra una categoria e l'altra, tra un giorno e l'altro, in cui di per sé non si fanno addominali: cinque oggi, si scala a sei domani. Il grafico corretto è quello **a gradini** (figura 6.8). "Strano che nessuno abbia dato la risposta sbagliata. Sono contento, ma mi fate anche moderatamente paura."

Grafico a linea I due grafici comunicano quasi le stesse cose, ma la linea è il modo migliore per mostrare un'**evoluzione continua**, anche in previsione del futuro. Comunica la **direzione** dei dati e la tendenza; generalmente evolve **in tempo reale**: si aggiungono dati e la linea va in una determinata direzione.

Grafico a gradini Comunica l'**impatto** e la tendenza: serve per modifiche **brusche e immediate**. Da un giorno all'altro si scala, si salta. È l'ideale, non sempre ma spesso, quando si parla di **progressione**: si pensi ai giochi mobile, o ai *game as a service*, con livelli dal livello 1 al livello 2: all'inizio l'esperienza richiesta è poca, poi cominciano gradoni di esperienza richiesta sempre più **ripidi** (il termine arriva dalla classe). Il quadrante è scandito in gradini, ed è molto leggibile se salgono verso l'alto o scattano verso il basso.

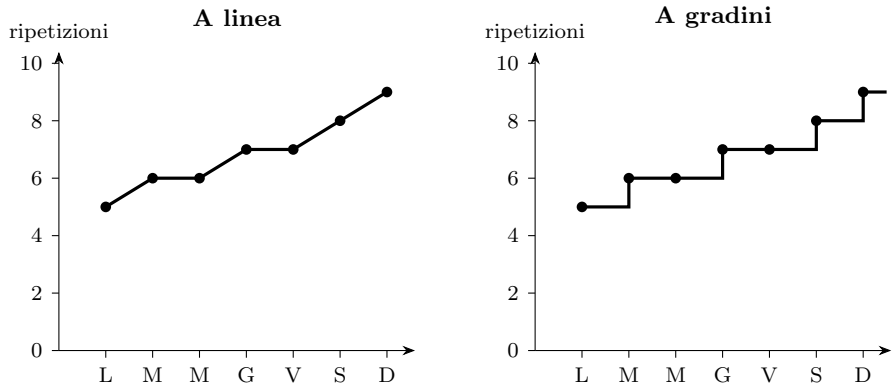


Figure 6.8: Gli stessi dati (ripetizioni giornaliere in una settimana) in un grafico a linea e in uno a gradini. La linea suggerisce valori intermedi tra un giorno e l'altro che non esistono; i gradini mostrano il salto discreto. Solo i valori di lunedì, martedì e venerdì vengono dalla lezione; gli altri sono inventati.

"Siete stati molto bravi, disgustosamente bravi." Ma il punto è l'abitudine: ogni volta che si hanno dei dati, chiedersi **qual è il grafico che comunica più correttamente** ciò che si mette sugli assi, o nel quadrante, o nell'immagine, perché molti grafici "subliminano" a un certo punto nelle **infografiche**.

6.16 I grafici di base

Si parte dai grafici di base, già incontrati quando si è parlato di *grafici con intento* (sezione 3.7).

6.16.1 Barre, colonne e istogrammi

Grafico a barre o a colonne. Un asse ha **dati quantitativi**, generalmente numeri; l'altro ha **classi qualitative**: maschi e femmine, basket, calcio o qualsiasi altro sport (con il numero di studenti che lo praticano), categorie che si possono dividere ma che non sono segmenti numerici, né sono associabili a numeri.

Barre impilate. Il grafico può essere **impilato** (*stacked bar chart* o *stacked column chart*): si aggiunge grafico su grafico, si suddivide ogni categoria. Il "periodo 1" potrebbe avere uomini e donne, asiatici ed europei, e così via; lo si può impilare n volte. Qui l'interesse è la **relazione delle parti con il tutto**: i sottogruppi 4, 3, 2, 1 rispetto alla totalità del gruppo 1, e così per il gruppo 2 e il gruppo 3. Le regole:

- le categorie si distinguono per **colore**: **stessa categoria, stesso colore**; i gruppi interni sono generalmente condivisi tra le singole barre;
- la **legenda** deve essere esplicita e comprensibile **prima** di leggere il grafico (o il documento): "non me la trovo a sorpresa dopo". Il lettore la deve vedere prima di leggere le informazioni.

Istogramma. La differenza rispetto al grafico a barre è che **entrambi gli assi** hanno **dati quantitativi**, e i segmenti sull'asse delle ascisse sono tendenzialmente **range uguali**: 10–14, 15–19, 20–24. È buon uso e costume che i range siano in **ordine crescente**, dal più basso al più alto.

6.16.2 La torta e la ciambella

Poi c'è il "rinomato, amato" **grafico a torta**: "in generale piace a tutti; magari ad alcuni no; a me piace". È estremamente comprensibile, estremamente grafico, ed è il grafico che di solito si usa nei **pitch**: per la divisione delle responsabilità, il numero di componenti del team, la divisione dei concept art ("anche questo fatto da me, una vecchia analisi"), la **divisione dei costi**. Se si chiedono dei fondi in un pitch, si mostra come vengono ripartiti sul totale, per dipartimenti o per categorie: tanti soldi per le macchine, i computer per far lavorare la gente; tanti per il dipartimento di arte con due persone; "un sacco di soldi" per il dipartimento di programmazione con sette persone.

- La **totalità** del grafico è il **100%** dei dati.
- Le singole fette sono la **percentuale sul totale**; ogni fetta è una **categoria** specifica, legata a un colore.
- Alcune torte sono in realtà **ciambelle sotto copertura**: il grafico ad anello (*donut*) è una torta con il buco, e non cambia niente. A volte viene classificato come un grafico diverso, ma a tutti gli effetti è la stessa cosa.

6.16.3 Linee e aree

Il **grafico a linea** e quello ad **area** riguardano dati quantitativi: uno dei due assi ha una **variabile indipendente**, come il tempo (generalmente è quella), e interessa la **tendenza** della linea, dove sta andando progressivamente nel tempo, nello spazio o in quello che si vuole. Le varianti sono diverse, ma il nucleo è sempre il grafico a linea:

Area È un grafico a linea proiettato in basso, che "si mangia" tutta la parte del piano sotto di sé.

Area impilata Le categorie si impilano l'una sull'altra; la loro somma dà il totale.

Area al 100% Le categorie sono mostrate come quote del totale, che è sempre il 100%.

6.16.4 Il clustering

L'ultimo, accennato già l'ultima volta e qui senza approfondire troppo, è il **clustering**, che di solito si può fare solo con grossi mezzi, **grossi dati** e grossi soldi. Si prende una serie di dati, tanti, "esageratamente tanti", e li si divide in **gruppi**: **massimizzando la coerenza interna** di ogni gruppo e **minimizzando la somiglianza** tra gruppi diversi. La slide mostra gli stessi dati prima non raggruppati e poi raggruppati (*clustered*). Serve però una **formula analitica** che differenzi i dati, e qualcuno che la calcoli: non è facile, di solito si fa con veri *big data*, quindi lo fanno aziende con un'enorme mole di dati sull'utenza. "Io stesso mai l'ho fatto in vita mia."

6.17 Oltre i grafici di base

"C'è un mondo di grafici là fuori, molto anche fichi." La slide ne mostra alcuni: sono grafici simili ad altri già visti, ma mostrano una mole di dati molto maggiore e comunicano tante cose che magari non interessa comunicare; "ma ora sapete che esistono".

6.17.1 Treemap e sunburst: torte travestite

Il grafico ad albero. Il **treemap** (grafico ad albero) è esattamente un grafico a torta: il rettangolo complessivo, con tutte le sezioni incluse, è il **100%**. Le fette sono ripartite tra loro: nell'esempio della slide, l'economia di uno stato, una fetta è la chimica (chimica medica, "prendiamone una più semplice") al 6,6%, e dentro ci sono tutte le **sottocategorie**. Fette ripartite in sotto-fette: una torta travestita che prende un altro nome.

Il sunburst. Il diagramma **sunburst** è ancora una volta una torta, solo con le fette ripartite in anelli concentrici di sottocategorie.[18]

6.17.2 Il grafico a radar

Il **grafico a radar** (o a ragnatela) è un altro grafico "molto fico", usato di solito banalmente **nel gameplay**: quando bisogna mostrare o modificare una statistica. Non deve per forza essere usato nel gameplay, ma è più comune averlo visto lì.

Dispatch. Il docente ne mostra l'applicazione "gamificata" in *Dispatch*, che usa questo sistema per il **check delle abilità** dei personaggi ("sperando di non beccare una scena esplicita"; e con una "piccola pausa pubblicitaria"). Al netto della narrativa, che è la componente fondamentale del gioco, la sua *backbone*, il gameplay è un gestionale: i supereroi, che si vedono in basso, hanno abilità spinte in una direzione o in un'altra (la forza contro i super cattivi, la difesa dei civili, l'agilità, la capacità di convincere le persone, l'intelligenza: queste sono le categorie).[19] Si manda il supereroe a fare il suo lavoro e, quando il lavoro è fatto, c'è un **tiro** rispetto alla previsione e a ciò che si è riempito con gli eroi: se la richiesta è completamente inclusa nelle abilità dell'eroe, il 100% viene assegnato automaticamente al risultato; nell'esempio è un 69%. È proprio un'applicazione del grafico a radar.

Mario & Sonic. Lo si usa anche, per esempio, in *Mario & Sonic ai Giochi Olimpici Invernali*, per mostrare le abilità dei personaggi: niente di esagerato, ma un modo estremamente grafico per mostrare **doti spiccate** o tendenze in **categorie qualitative** (esecuzione, atterraggio, stile, creatività, difficoltà).

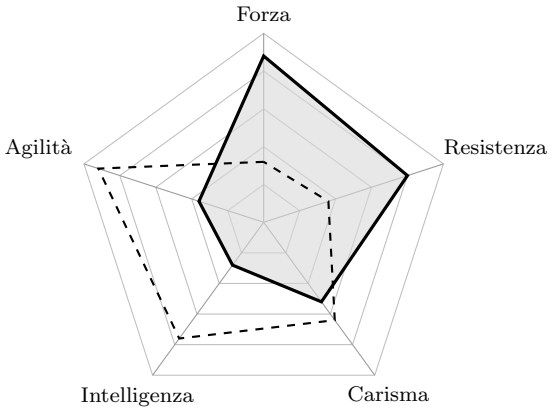


Figure 6.9: Un grafico a radar con due profili di personaggio (valori inventati): a colpo d'occhio si vede in quali categorie qualitative ciascuno è spiccato.

6.17.3 Il grafico a corde

Nel **grafico a corde** (*chord diagram*) ogni corda ha un'**entrata** e un'**uscita**, indicata dalla punta della freccia. L'esempio della slide mostra le **migrazioni globali** nel corso del tempo.

Le relazioni tra sistemi. Questo grafico è estremamente utile quando si parla di **relazioni tra sistemi**. Si mettono a grafico tutti i sistemi del gioco: il *weather system*, il *combat system*, gli NPC, la vita sociale, e così via. Si mostra quanto sono collegati e in che direzione (in un senso o in entrambi), e già a livello visivo si fa vedere quanto, per esempio, il combattimento è collegato a tutto il resto, o quanto un sistema invece

[18] Nella trascrizione compare come "Sandbox diagram"; dalla descrizione (una torta con le fette suddivise) e dal precedente accenno in sezione 3.7, si tratta del diagramma *sunburst*.

[19] In *Dispatch* (AdHoc Studio, 2025) le cinque statistiche degli eroi dovrebbero essere *Combat*, *Vigor*, *Mobility*, *Charisma* e *Intellect*; la trascrizione le parafrasa.

è poco impattante e interessa relativamente meno. È un altro modo per **mostrare** graficamente, invece di dover scrivere in maniera esagerata, le relazioni tra i sistemi.

"Di grafici ce ne sono davvero tantissimi." Sta al designer imparare a scegliere, come sempre, *the right tool for the right job*; "in questo caso, *the best tool for the right job*".

6.18 La campagna di Russia in un grafico

6.18.1 Due avvertenze

Per mostrare fin dove ci si può spingere, il docente ripropone un grafico già intravisto, "fichissimo, cioè per me è fichissimo", con due avvertenze.

1. Probabilmente lo si rivedrà al **terzo anno**, e verrà rifatta la stessa domanda: che cosa mostra? Ma il docente "non cede di un passo" e lo mostra lo stesso al primo anno, perché "è troppo bello". Al terzo anno, "non mentite: noi lo sappiamo".

2. Il grafico è legato al **design**: fa parte di quel processo di **semplificazione** e di resa più grafica dei documenti di cui si sta parlando nelle lezioni di documentazione, e tornerà quando si parlerà di uno specifico tipo di documento, la "summa", il fine ultimo di questo processo.

Il grafico viene attribuito a **Edward Tufte**, "probabilmente il padre delle infografiche": ha dato i principi di base, alcuni dei quali sono golden rules della documentazione (sezione 3.7), e sono presi come base per fare UX e UI.[20] Il docente darà poi il link a un talk che ne parla, "uno dei talk più belli che siano mai stati fatti sul design in generale, sul game design in generale. A me ha aperto la mente."

6.18.2 Che cosa mostra?

La regola del gioco: niente "furia", non si va a cercare che cos'è. Più ci si spreme le meningi, più ritorno si avrà dopo. Si sa solo che è una **variante del diagramma di Sankey**. E bisogna leggere il grafico, non i testi ("sennò è troppo semplice").

Valerio Potrebbe mostrare **scelte narrative** che proseguono con la storia. "In questo caso no. Però è interessante: c'è un certo tipo di **progressione** in questo grafico." E, ammette il docente, "ho mentito: sono due grafici", uno sopra e uno sotto, che si legano tra loro.

Lorenzo Sembra esserci una **correlazione** tra i due. La banda color ocra all'inizio è il 100%; poi una parte si stacca, e nel tempo, procedendo, la quantità cambia, forse in un tempo comune alle due curve rappresentato da quella sotto. Nella banda nera si discrimina un'altra popolazione che si prende un'altra fetta. E quindi non sono percentuali ma **quantità**, perché nel tempo entrambe **diminuiscono**. "Questo te lo confermo." Sembra uno strumento per analizzare più categorie e quantità correlate nel tempo. Il docente conferma la direzione: c'è una progressione, "tra l'altro fisica"; la banda si rompe in quella nera, le quantità di entrambe diminuiscono, e c'è la correlazione con il grafico sotto, che è un **grafico a linea delle temperature**. Dalla classe si nota anche che la banda nera sembra ingrossarsi a spese dell'altra, e qualcuno azzarda un'ipotesi: petrolio bruciato e anidride carbonica emessa.

Il docente indica allora alcune linee sottili, "con una forma che sicuramente in vita vostra avete già visto". Curve di livello? "No, sono troppo irregolari." Confini geografici? Una mappa delle temperature locali in 2D? Nessuno azzarda: sono **fiumi**.

6.18.3 La risposta

Il grafico mostra la quantità di uomini nell'**esercito di Napoleone durante la campagna di Russia**, verso Mosca.

[20] La mappa è in realtà opera dell'ingegnere francese **Charles Joseph Minard** (1869). È stato Tufte, in *The Visual Display of Quantitative Information* (1983), a renderla celebre, definendola probabilmente il miglior grafico statistico mai disegnato.

- La banda chiara è la **marcia di andata**, quella nera la **ritirata**.
- La **larghezza** della banda è la grandezza dell'esercito.
- Durante la marcia alcuni se ne vanno: l'esercito si divide, alcuni tornano indietro, altri vanno avanti.
- In corrispondenza di un fiume si vede la banda assottigliarsi drasticamente: lì sono morti moltissimi uomini ("adesso io ci rido, ma non fa ridere").
- Tutto in relazione alla **temperatura** in quei punti, che è il secondo grafico, sotto.

Per il docente "è una figata" perché, una volta letta la legenda, **comunica immediatamente**, con un impatto grafico fortissimo, dove è andato quell'esercito, che cosa ha vissuto, come si è piano piano non solo separato ma anche **decimato** nel tempo: una campagna completamente disastrosa. "Con i grafici potete fare questa roba qua."

6.18.4 Dalle infografiche ai documenti

In futuro si vedrà che lo stesso si può fare anche con la documentazione, benché con i documenti di design sia veramente difficile: nella sua massima espressione, **compressi in una singola pagina**. È il motivo per cui Tufte è stato preso ad esempio e riadattato all'arte del documentare da **Stone Librande**, il designer di *Spore*, dei *Sims* e di altri titoli.[21] Se ne riparlerà nella lezione di documentazione apposita. Non bisogna raggiungere subito quel livello, "perché è molto difficile, e non credo nemmeno di esserci arrivato io"; ma il margine è tanto: **il tetto è molto alto**. Si ragiona, si sperimenta, e si migliora nel tempo.

Approfondimento: Tufte nelle bibliografie di game design. Tufte compare nelle letture consigliate di entrambi i manuali del progetto. Schell, in *The Art of Game Design*, definisce *The Visual Display of Quantitative Information* "la Bibbia (o almeno l'Antico Testamento) del design delle interfacce grafiche", e apre la sua lente della trasparenza (#62) con una citazione di Tufte: "Per quanto bella sia la tua interfaccia, sarebbe meglio se ce ne fosse di meno". Brathwaite e Schreiber, in *Challenges for Game Designers*, consigliano *Visual Explanations*: per quanto si aderisca alle convenzioni di genere, sembra che esistano solo uno o due modi di mostrare una barra della vita o il conteggio delle munizioni; Tufte "distrugge questa illusione", mostrando molti modi diversi di visualizzare un'informazione importante, con pregi e difetti di ciascuno. È lo stesso messaggio della lezione: per ogni dato esiste un grafico migliore degli altri, e trovarlo è parte del lavoro.

6.19 Principi per grafici migliori

Ci sono comunque dei principi per migliorare, formattare e rendere più visibili i dati di un grafico: le classiche regole della **Gestalt** e i principi di design in generale. Cose che probabilmente si sanno già, ma è utile ricordarle, e che si sono già cominciate ad applicare nella documentazione; nei grafici le si può spingere "all'esponenziale", perché fare grafici è "una pratica grafica, appunto".

Spazio negativo Va sfruttato.

Scala Ciò che è più grande impatta di più il lettore e viene percepito come più importante; ciò che è più piccolo, meno.

Gerarchia L'ordine visivo delle informazioni.

Prossimità, forma, colore Elementi vicini, della stessa forma o dello stesso colore vengono letti come un gruppo.

Al netto della ricerca (scoprire nuovi tipi di grafico, capire qual è il più adatto) questa abilità si costruisce con l'**esperienza**: più grafici si fanno, più si diventa abili, "ancora di più rispetto alla documentazione base, anche perché questo è documento".

[21] Nella trascrizione compare come "Stanley Brandi". Il talk a cui il docente si riferisce è quasi certamente *One-Page Designs* di Stone Librande (GDC 2010), in cui la mappa di Minard e i principi di Tufte sono il punto di partenza per i documenti di design di una sola pagina.

6.20 La matematica di base

6.20.1 Quattro cose, da quinta elementare

Non c'è tempo per un'altra pausa ("al massimo finiamo cinque minuti prima, cosa che non è mai successa"), quindi si passa ai *basic math principles*. "Qua siamo probabilmente in quinta elementare": per chi non lo sapesse non è un problema; le cose sono veramente quattro.

6.20.2 La media

La **media** (aritmetica) è la somma di tutti i numeri divisa per il numero di elementi dell'insieme:

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

"Su questo penso che nessuno abbia dei dubbi."

6.20.3 La media pesata

Serve per alcuni framework o per alcuni calcoli in generale. Ci sono **due modi** per farla.

Primo modo: pesi che sommano a uno. Tutti i pesi sommati devono dare **uno**, cioè il totale, il 100%. Dati questi pesi, si **moltiplica** ciascun valore per il rispettivo peso, ottenendo il valore pesato, e poi si **somma**:

$$\bar{x}_w = \sum_{i=1}^{n} w_i\, x_i \qquad \text{con} \qquad \sum_{i=1}^{n} w_i = 1.$$

Secondo modo: pesi qualsiasi. I pesi possono **non** dare il 100%. Si moltiplica ciascun valore per il suo peso specifico, si sommano i risultati, e la somma si divide per la **somma dei pesi**:

$$\bar{x}_w = \frac{\sum_{i=1}^{n} w_i\, x_i}{\sum_{i=1}^{n} w_i}.$$

"Non è complessa, forse un passaggio in più." Il primo modo è un caso particolare del secondo: se i pesi sommano a uno, il denominatore vale 1.

Un esempio. Si vogliano confrontare due idee uscite dal brainstorming dando un voto da 1 a 10 a tre criteri, con pesi diversi: impatto (peso 3), costo contenuto (peso 2), coerenza con la visione (peso 1). Un'idea con voti 8, 4 e 9 ha media semplice $(8+4+9)/3 = 7$, ma media pesata

$$\frac{3\cdot 8 + 2\cdot 4 + 1\cdot 9}{3+2+1} = \frac{41}{6} \approx 6{,}83.$$

Con i pesi normalizzati ($3/6 = 0{,}5$; $2/6 \approx 0{,}33$; $1/6 \approx 0{,}17$, che sommano a 1) si ottiene lo stesso risultato con il primo modo. (L'esempio non è della lezione.)

Approfondimento: il valore atteso. Schell (*The Art of Game Design*, capitolo 12) mostra uno degli usi più frequenti della media pesata nel game design: il **valore atteso**, cioè la media di tutti i risultati possibili di un'azione, pesati con la loro probabilità (pesi che, come nel primo modo, sommano a uno). Se finendo su una casella verde si tira un dado a sei facce e si ottengono altrettanti punti, il valore atteso è $(1+2+3+4+5+6)/6 = 3{,}5$: è utile sapere che ogni volta, in media, si guadagnano 3,5 punti. Con esiti non equiprobabili si pesa: tirando due dadi, si vincono 5 dollari con un 7 o un 11 e se ne perde uno con qualsiasi altro risultato. Le probabilità sono 6/36 per il 7, 2/36 per l'11 e 28/36 per il resto, quindi

$$\tfrac{6}{36}\cdot 5 + \tfrac{2}{36}\cdot 5 + \tfrac{28}{36}\cdot(-1) \approx 0{,}83 + 0{,}28 - 0{,}78 = 0{,}33:$$

alla lunga si vincono in media 33 centesimi a partita. Se vince solo il 7, il valore atteso scende a zero (un gioco equo); se vince solo l'11, diventa negativo ($-0{,}86$ circa). Schell ne fa una lente (#35, *The Lens of Expected Value*): bilanciare un gioco passa spesso da conti di questo tipo.

6.20.4 Le proporzioni

L'ultima cosa è la **proporzione**: a sta a b come c sta a d,

$$a : b = c : d,$$

dove a e d sono gli **estremi** e b e c i **medi**. La proprietà fondamentale è che il prodotto dei medi è uguale al prodotto degli estremi ($b \cdot c = a \cdot d$). Quindi:

Incognita in un estremo Si **moltiplicano i medi** e si divide per l'**estremo noto**. Esempio della lezione: in $100 : 35 = 20 : x$, i medi sono 35 e 20, l'estremo noto è 100:

$$x = \frac{35 \cdot 20}{100} = \frac{700}{100} = 7.$$

Verifica: 100 sta a 35 come 20 sta a 7.

Incognita in un medio Si **moltiplicano gli estremi** e si divide per il **medio noto**. Esercizio della lezione: in $100 : 35 = x : 5$, gli estremi sono 100 e 5, il medio noto è 35. La classe risponde con il calcolo, "500", diviso 35:

$$x = \frac{100 \cdot 5}{35} = \frac{500}{35} \approx 14{,}3,$$

che il docente arrotonda a 14: 100 sta a 35 come (circa) 14 sta a 5.

"Avete tutte le informazioni basilari di matematica per sopravvivere alla quinta elementare e al primo anno."

6.20.5 Che cosa verrà dopo

Il discorso si complicherà al secondo anno, quando si parlerà di **regressione lineare**: ma sono strumenti che servono per framework specifici o per lavori specifici. Fare l'**economy designer** li richiede, e anche un **data analyst** ha determinate competenze matematiche; li si tratterà lì, punto per punto. Per ora interessa solo che si ricordino queste quattro cose: semplici, ma "una rinfrescata e averle lì non vi fa male".

6.21 Chiusura e prossimi appuntamenti

La lezione è finita ("yes, abbiamo finito"), con un leggero margine di anticipo. Il docente ricorda le scadenze:

- il **production dictionary** per **lunedì 2** (sezione 4.14);
- il **flowchart di Pac-Man** (o di *Donkey Kong*) per **venerdì 6** (sezione 6.14);
- l'altro task è già stato consegnato: il docente lo commenterà nel fine settimana o, al massimo, all'inizio della settimana successiva.

Leggere i commenti prima del 6. Dopo che il docente avrà commentato, bisogna **leggere i suoi commenti prima della lezione del 6**: in quella lezione si prenderà una pagina bianca e si comincerà a documentare e a formattare la pagina insieme (sezione 4.2), e prima di farlo è importante che il feedback ricevuto sia stato compreso. Come detto l'altra volta, bisogna aspettare che il docente abbia commentato tutto.

"Per lunedì portate il PC, perché Alessandro ne sarà sicuramente contento." I saluti sono di stagione: "Buon Halloween! Buona festa dei morti!"

Punti chiave del capitolo

- Nel **production dictionary** vanno i **nomi** unici o fraintendibili del gioco, spiegati in una o due righe (che cos'è una MT, che cos'è un tipo); il funzionamento profondo va nei documenti dedicati. Ordine alfabetico per leggibilità, ma le voci legate tra loro possono stare vicine; in ogni caso, **ordinato**.
- Nel brainstorming **nessuno viene fermato**, tutti parlano, si elenca tutto, si cercano background diversi. Un **facilitatore** (non un leader) fa sentire tutti ascoltati e tiene il brainstorming su uno scopo pratico. **Pause** e **cibo** riducono la tensione.
- Altre tecniche: circle, acting brainstorming, i **sei cappelli** di de Bono (partire dal verde; il nero è già giudizio), l'oggetto dal cappello, storyboard, domande, "peggiore è, meglio è", associative e **mind mapping**, il **TRIZ** (matrice di parametri, soluzione già nella cella, adatto a problemi fisici).
- Le regole di **Kawasaki** (make meaning, make mantra, next curve, DICEE, don't worry be crappy, hundred flowers, polarize, churn, niche, perfect your pitch) e lo **SCAMPER**. Il pitch si prova e riprova, ma **non si impara a memoria**.
- Dalle cento idee ad **almeno dieci**: **brainstorming matrix** impatto/effort, confronto, compromesso; contano anche la **motivazione** del team e la visione. Le idee vanno **distrutte**: se non si trova nessun difetto, c'è un problema.
- **KISS**: prototipare subito il cuore del gioco; muoversi, cambiare prospettiva, **rituali** che rompono la routine (il playtest del venerdì).
- Il **flowchart** è una rappresentazione grafica formalizzata di un processo; nasce nel software, e ai designer serve per IA, pipeline, game loop, sequenze di gameplay, player journey, UI. Serve a **mostrare invece di scrivere**.
- Regole: si legge **dall'alto in basso e da sinistra a destra**, nella direzione delle frecce; stesso livello, stessa gerarchia; il layout orizzontale evita il **wall of text**.
- Blocchi: **inizio/fine** (rettangolo arrotondato, obbligatori), **azione** (rettangolo), **decisione** (rombo, uscite sempre etichettate), **sotto-processo** (da spiegare in un altro flowchart), **input/output** (parallelogramma), **documento**.
- In un sistema o un loop di gioco il flusso **deve chiudersi o tornare indietro**: nessun ramo nel nulla. Le regole prima si imparano, poi si piegano (Oxenfree, il gioco di carte, Machinations, il behavior tree).
- Task: il **flowchart completo di Pac-Man** (Daniele: *Donkey Kong*), come immagine leggibile e commentabile, esportata al 300–500%, per venerdì 6.
- Grafici: **linea** per l'evoluzione continua e le previsioni, **gradini** per i salti discreti e le progressioni; barre e colonne (con legenda prima), istogrammi, torta e ciambella, aree, clustering; poi treemap, sunburst, **radar**, **corde** (relazioni tra sistemi) e la mappa di Minard resa celebre da Tufte. Il tetto è altissimo: fino ai documenti di una pagina di Librande.
- Matematica di base: **media**, **media pesata** (pesi che sommano a uno, oppure divisione per la somma dei pesi; il valore atteso ne è un caso), **proporzioni** (medi per medi, estremi per estremi).

Chapter 7

Eventi, nodi e variabili: la prima lezione di programmazione con i Blueprint

Fino a questo punto il corso di Game Design ha lavorato quasi solo su carta: che cos'è un gioco, come si produce, come si documenta, come si passa da un problema a un'idea e da un'idea a un flowchart (capitoli 1–6). Con questo settimo capitolo si cambia completamente registro: si apre **Unreal Engine** e si comincia a **programmare**, con i **Blueprint**. Il flowchart del capitolo precedente, del resto, era già una mezza introduzione: un Blueprint è, a guardarlo, un flowchart che il computer esegue.

Il docente lo dice subito, e lo ripeterà più volte durante la giornata: la lezione di oggi è "**super importante**", perché "quello che faremo qui oggi si baserà su tutto quanto il resto dei prossimi **tre anni** di programmazione". Per questo chiede alla classe di fermarlo a ogni dubbio, anche il più piccolo. Chi ha già seguito il corso breve su Unreal (capitoli 1.1 e 1.2) troverà qui molti concetti noti, ma visti da un'altra angolazione e con un altro obiettivo: non costruire un piccolo gioco guidato passo per passo, ma **prendere la mano** con i tre mattoni su cui si regge tutto il resto.

La lezione è **ibrida**: una parte della classe è in aula, una parte (Valerio, Federica, Mattia) è collegata da remoto su Zoom e condivide lo schermo su Discord. Il computer del docente, purtroppo, non regge: crasha più volte, va "lentissimo", e a un certo punto rende impossibile guardare gli schermi condivisi ("cambieremo PC a breve, non vi preoccupate"). La struttura della lezione, e quindi del capitolo, è questa:

1. le **presentazioni** e il quadro generale: C++ e Blueprint, e i **tre concetti** che bisogna imparare davvero (sezioni 7.1 e 7.2);
2. il progetto in terza persona e il **primo Blueprint**, un semplice cubo (sezioni 7.3 e 7.4);
3. gli **eventi** (Begin Play, Event Tick) e il primo nodo, il Print String (sezione 7.5);
4. le **variabili**: i tipi, i colori, il GET e il SET, il contatore che sale a ogni *frame* (sezioni 7.6 e 7.7);
5. il **personaggio**: gli eventi da tastiera, il lancio, la scala, il teletrasporto e il **Character Movement** (sezioni 7.8 e 7.9);
6. le **interazioni tra oggetti**: la trigger box, il *cast*, il *target*, lo *spawn*, la fisica (sezioni 7.10 e 7.11);
7. le **monete** e il contatore sul personaggio (sezione 7.12);
8. altri nodi utili e il **task in classe**: due livelli e almeno una zona che cambia il personaggio (sezioni 7.14 e 7.15);
9. l'Event Tick e i **delta seconds** (sezione 7.17);
10. il ripasso, il **sistema di vita**, Actor, Pawn e Character, il Line Trace (sezioni 7.18–7.20);

11. le **regole dei collegamenti** e le domande di chi parte da zero: che cos'è il filo bianco, che cosa vogliono dire i colori (sezioni 7.21 e 7.22).

7.1 Prima di cominciare: chi c'è, e chi ha già programmato

7.1.1 Le versioni dell'engine

Mentre si aspetta che arrivino tutti ("aspettiamo ancora cinque minuti"), la prima domanda è sulla versione di Unreal scaricata: la 5.6, la 5.7, la 5.8? Qualcuno ha già scaricato la 5.8. Per il docente non fa differenza: "tanto è uguale, non vi cambia molto". Lo ribadirà più tardi a uno studente che sta reinstallando l'engine: meglio avere la stessa versione degli altri (la 5.8), ma "se sei sulle ultime versioni non ti cambierà praticamente nulla"; anche una 5.4 "non è un problema". Per le cose che si fanno oggi le differenze tra le versioni recenti sono irrilevanti.

7.1.2 Un giro di presentazioni

Mancando ancora un paio di minuti, il docente chiede a ciascuno di raccontare qualcosa di sé: che cosa fa, che cosa gli piace, **se ha mai programmato** oppure no. Il quadro che ne esce è quello tipico di un primo anno, e vale la pena riportarlo perché spiega il ritmo della lezione:

- chi viene dal liceo scientifico ha visto un po' di **C**, **Python** e un po' di **SQL**: "comunque già un po' di basi";
- chi viene da altri indirizzi, come Francesco, che ha fatto il **classico**, parte "da zero zero zero zero";
- Valerio è un **grafico**: zero assoluto, se non un po' di codice per il web, "molto, molto poco";
- Mattia (Maestri) arriva da un'altra accademia di videogiochi, quella di Bologna, dove era al secondo anno: ha già usato Unreal "anche se per poco", con i Blueprint "a un livello basico", e si muove già un po' nell'editor, ma è "una materia in cui faccio abbastanza fatica";
- Federica dichiara una conoscenza "pari a zero, anche meno" ("tranquilla, non sei l'unica qui").

I giochi preferiti. Il giro comprende anche i giochi preferiti. Tra quelli che la trascrizione restituisce in modo riconoscibile: un gruppo di giochi "un pochettino cupi", tra cui *The Binding of Isaac*; la saga di *Monster Hunter* per Mattia; *Yakuza 0* e *Dave the Diver* ("bellissimo") per Valeria; e per Federica e Daniele due titoli "molto indie".[1] Il docente non li conosce tutti e lo ammette senza problemi ("non lo conosco, colpa mia"): il punto del giro non è il quiz, è sapere **da dove parte** ciascuno.

Chi parte da zero. Più avanti, a lezione inoltrata, Federica dirà di essersi persa all'inizio per colpa dei crash e di due computer in due stanze diverse. La risposta del docente fissa il tono di tutto il corso: "siamo alla prima lezione, non puoi essere indietro. Non è possibile stare indietro, proprio matematicamente." Se serve rifarla da capo, la si rifà da capo. L'importante è non restare indietro *dopo*: "se avete dubbi me li dovete fare qua, adesso, non dopo". Per tutto il resto c'è Discord, dove si può scrivere al docente (e, se è grave, chiedere una chiamata).

7.2 Due modi di programmare, tre concetti da imparare

7.2.1 C++ e Blueprint

Unreal è un **engine**, uno strumento per fare giochi. Lo si può programmare in due modi:

- in **C++**, il linguaggio scritto "classico". Nel corso non lo si vedrà, "se non proprio una cosina microscopica al secondo anno";

[1] Diversi titoli sono irriconoscibili nella trascrizione automatica ("It's All World", "il signor Sky", "Otherwise", "Discollision"); non li si ricostruisce per non attribuire agli studenti giochi che non hanno nominato.

- con i **Blueprint**, che saranno lo strumento di tutto il corso. I due sistemi, in realtà, "parlano tra loro", ma questo per ora non importa.

La comodità dei Blueprint è che non sono codice scritto come lo vedono i programmatori: sono composti da **blocchetti colorati**, e "componendo quelli fate tutto". Non serve ricordarsi come si scrive un comando né dove va una virgola: si fa tasto destro e si cerca.

7.2.2 Eventi, nodi, variabili

Il cuore della lezione sta in una frase: i Blueprint sono composti **soltanto da tre cose**, "le uniche tre che dovete imparare veramente".

I nodi sono **tutto ciò che si può fare**: stampare una scritta, lanciare il personaggio, spostare un oggetto, distruggerlo.

Le variabili sono **le cose che cambiano**: le informazioni che il gioco legge e modifica (quante monete ho, quanto salto in alto, dove mi trovo).

Gli eventi sono **l'inizio del codice**: il momento in cui qualcosa comincia a succedere (l'oggetto viene creato, passa un *frame*, il giocatore preme un tasto, qualcosa entra in un'area).

"Basta, non è che c'è altro." Se si capiscono bene questi tre concetti, "siamo a posto, perché tutto quello che arriverà dopo si basa su quello". Il docente invita a scriverseli.

Approfondimento: il percorso di esecuzione. Romero e Sewell, nel secondo capitolo del loro libro sui Blueprint, danno un nome preciso a quello che a lezione si vede con le mani: l'esecuzione di un Blueprint **comincia da un nodo evento** (rosso) e poi **segue il filo bianco da sinistra a destra** fino all'ultimo nodo. Questo filo si chiama *execution path*, il percorso di esecuzione; i pin bianchi dei nodi sono i *pin di esecuzione*, quelli colorati i *pin dei dati*. È la stessa distinzione che il docente spiegherà a fine lezione con l'immagine della corrente elettrica (sezione 7.22).

7.3 Il progetto in terza persona

7.3.1 Creare il progetto

Si apre Unreal, nella versione che si ha, e si crea un **progetto nuovo da zero** con il template **Third Person** (la categoria è quella dei giochi, *Games*, poi *Third Person*), lasciando tutto il resto "esattamente così com'è". Il nome proposto è **GG1** seguito dalla data. Il template in terza persona ha un vantaggio: c'è già un **personaggio di base** che si può controllare, quello standard.

Se il template è sbagliato. Un paio di studenti si ritrovano con una mappa diversa da quella del docente ("tipo un'arena, con le strutture verdi"): hanno scelto una variante sbagliata. Le soluzioni sono due: ricreare il progetto, oppure, se il progetto è vuoto, usare **Add Feature** (aggiungi funzionalità), scegliere il pacchetto in terza persona e fare *Add*: "così torni esattamente dove siamo".

7.3.2 L'editor, in due parole

Una volta caricato ("spero che il computer regga"), l'editor si presenta con due zone principali:

- la **viewport**, "dove c'è tutto quanto ciò che vediamo nella scena". Premendo **Play** parte il gioco ("incredibile"), e ci si muove come in un normale gioco in terza persona;
- il **Content Browser**, qua sotto, con gli **asset**: "praticamente tutto ciò che ci sarà nel nostro gioco", il codice, le texture, le mesh, i suoni, organizzati in cartelle. "Molto, molto facile."

7.3.3 Il personaggio: viewport ed Event Graph

Nel Content Browser, nella cartella *ThirdPerson* → *Blueprints*, si trova il **ThirdPersonCharacter**, il personaggio che compare quando si preme Play. Aprendolo si vedono molte schermate e molti tasti, ma il docente rassicura: "non fatevi intimorire". La cosa importante è che un Blueprint è suddiviso, fondamentalmente, in due parti:

- la **viewport**, la parte **visiva** dell'oggetto;
- l'**Event Graph**, la parte del **codice**.

Per semplificarsi ancora di più la vita, però, il docente lo chiude subito: si parte da zero, con un oggetto nuovo su cui mettere il codice.

7.4 Il primo Blueprint: un cubo

7.4.1 Blueprint Class, Actor

Nel Content Browser, in uno spazio vuoto della cartella *Content*: tasto destro, **Blueprint Class**. È il comando per creare un **oggetto nuovo qualsiasi**: "potrebbe essere un personaggio, una macchina, il gatto che passa, un muro, qualsiasi cosa che si mette nella scena". La finestra che si apre offre molte opzioni, che per ora si ignorano completamente: si sceglie sempre la prima, **Actor**, "che è la cosa base". Il docente lo chiama **test**, "per rimanere sul semplice". Doppio clic, e il Blueprint si apre. (La differenza tra Actor, Pawn e Character arriverà verso la fine della lezione, sezione 7.20.)

7.4.2 Un oggetto visibile, e più volte in scena

L'oggetto appena creato, visivamente, è il vuoto: non c'è niente. Per identificarlo gli si mette un **cubo**: **Add**, *Cube*. "Così ora questo oggetto è effettivamente un cubo." Non ha ancora codice, ma lo si può già **trascinare nella scena**, e lo si può fare **più volte**: ogni copia trascinata è un elemento in scena. Il docente ne piazza tre; premendo Play, i tre cubi sono lì. "Roba facilissima."

Approfondimento: classe e istanza. Quella che il docente descrive come "lo stesso oggetto trascinato più volte" è la distinzione tra **classe** e **istanza**. Il Blueprint *test* nel Content Browser è la classe, lo stampo; ogni cubo nella scena è un'istanza, una copia concreta costruita con quello stampo, con il suo stesso codice ma con una vita propria. Tra un attimo questa distinzione avrà una conseguenza visibile: un Print String nel cubo verrà eseguito **tre volte**, una per istanza.

7.5 Gli eventi e il primo nodo

7.5.1 Begin Play e Tick

Si va sull'**Event Graph** del cubo (chi si ritrova in un'altra schermata, per esempio nel *Construction Script*, deve cliccare sulla scheda *Event Graph* in alto). Ci sono già tre eventi preimpostati, e qui sta "la parte veramente importante". Gli eventi sono "tutti questi cosi qua che fondamentalmente partono e fanno cominciare il codice". Due sono quelli che interessano:

Event Begin Play viene chiamato **quando l'oggetto viene creato**. Per un oggetto già presente nella scena, questo significa: quando parte il gioco.

Event Tick viene chiamato **a ogni *frame***. "Se il gioco va a 60 fps, verrà chiamato 60 volte" al secondo.

Il terzo evento (*Event ActorBeginOverlap*) "non vi interessa, cancellatelo pure". Tornerà utile, in un'altra forma, con la trigger box.

7.5.2 Print String

Da un evento si trascina il filo e si scrive il nodo che si vuole: il primo è **Print String**, che stampa semplicemente qualcosa sullo schermo, in alto a sinistra. Di base stampa *Hello*. Si collega al Begin Play, si preme Play e compare *Hello*... tre volte. Perché? Il docente lascia la domanda alla classe: perché **ci sono tre cubi**, e ognuno, quando viene creato, esegue il proprio Begin Play.

Se si sposta lo stesso Print String sull'Event Tick, invece, "ne stamperebbe un sacchissimo": uno a ogni *frame*, per ogni cubo. Il docente lo fa vedere, e lo schermo si riempie della lista di tutti i messaggi.

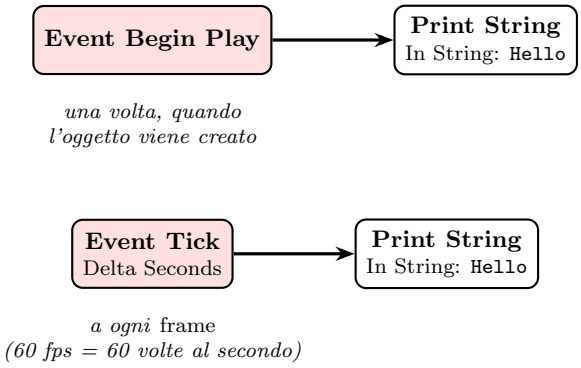


Figure 7.1: I due eventi che interessano: lo stesso Print String collegato al Begin Play stampa una volta per ogni istanza; collegato all'Event Tick stampa a ogni *frame*. Il filo spesso è il filo bianco dell'esecuzione.

Collegare e scollegare. I nodi si collegano trascinando il filo da un pin all'altro. Per interrompere un collegamento basta il tasto destro sul pin e **Break**: il flusso si interrompe. (In Unreal si può anche fare Alt + clic sul pin.)

7.6 Le variabili

7.6.1 Che cos'è una variabile

L'unico elemento che manca, "un attimino più complicato", sono le **variabili**. Sono semplicemente **informazioni** che si vanno a prendere e a modificare. E "tutto quanto si basa su questo: tutto quanto l'engine, tutto quanto la programmazione, in realtà".

Nel pannello a sinistra del Blueprint (*My Blueprint*) c'è la sezione **Variables**; con il + se ne crea una nuova. Il docente la chiama *var1* ("troppo semplice: variabile 1"). E una raccomandazione pratica: **fate un Compile** quando create variabili nuove, "se no non vi si ricarica" (e non compare il valore di default).

7.6.2 I tipi, e i loro colori

Le variabili possono essere di "migliaia di tipi, infiniti". Quelli che interessano adesso sono cinque o sei, e il docente chiede di **scriverseli**, con il loro **colore**: "le cose con gli stessi colori vanno sempre assieme", e questo semplifica moltissimo il lavoro.

Boolean Sì o no, vero o falso. "Il personaggio sta cadendo? Sì o no." "C'è questo oggetto nella scena? Sì o no." Non può esserci un numero.

Integer I numeri **interi**, senza virgola: 528, −82. Quante munizioni ho, il mio punteggio.

Float **Tutti** i numeri, compresi quelli con la virgola: 5,23; 7,2; −27. "Qualsiasi numero con la virgola o senza virgola può essere messo come float."

String Il **testo**. L'*Hello* stampato prima è rosa proprio perché è una stringa.

Vector **Tre float** messi insieme. Perché farne uno apposta? Perché spesso servono tre numeri, x, y, z, per esempio per sapere **dove si trova un oggetto nello spazio**.

"Per ora ci bastano questi, poi ne aggiungeremo degli altri, ma non troppi."

Tipo	Colore	Contiene	Esempi dalla lezione
Boolean	rosso scuro	vero / falso	sta cadendo? c'è l'oggetto in scena?
Integer	verde acqua	numeri interi	munizioni, punteggio, monete, 25
Float	verde chiaro	numeri con la virgola	$-1{,}45$, una velocità, la vita
String	rosa (magenta)	testo	*Hello*
Vector	giallo	tre float (x, y, z)	posizione, scala
Object reference	blu	un riferimento a un altro oggetto	il personaggio, una moneta
Il filo bianco non è un tipo di dato: è il flusso di esecuzione.			

Table 7.1: I tipi di variabile visti a lezione, con i colori dei rispettivi pin nell'editor.[3] Il riferimento a un oggetto (blu) compare solo a fine lezione, nelle domande di Federica.

Integer o Integer64? Mattia chiede che differenza c'è tra *integer* e *integer 64*, che compare nella lista. L'integer 64 è "per numeri **astronomicamente grandi**": "se devi contare il numero di pianeti nell'universo usi l'integer 64, ma nel 99 per cento dei casi l'integer e basta".

Approfondimento: quanto è grande "astronomicamente grande". Romero e Sewell danno i limiti esatti. L'**Integer** è un numero a 32 bit, da $-2\,147\,483\,648$ a $2\,147\,483\,647$: poco più di due miliardi in ciascuna direzione, più che abbastanza per monete, munizioni e punteggi. L'**Integer64** è a 64 bit e arriva a circa $9{,}2 \times 10^{18}$. Il libro elenca anche i tipi che a lezione si saltano: il **Byte** (interi da 0 a 255), il **Double** (un float con più precisione), il **Name** (un testo usato come identificatore), il **Text** (testo pensato per essere tradotto, cioè localizzato) e il **Rotator**, che come il Vector contiene tre numeri, ma rappresenta una rotazione (Roll, Pitch, Yaw). Il Rotator tornerà utile con le monete che ruotano (sezione 7.16).

7.6.3 Il valore di default

Selezionata una variabile e fatto il Compile, nel pannello dei dettagli compare il **valore di base**, cioè il valore da cui la variabile **parte**. Per un intero è 0; il docente scrive **25**. "Quindi, dentro questo oggetto che è il cubo, c'è una variabile chiamata var1 che è un intero con un valore di 25." A che cosa può servire? A moltissime cose: **quanto vale una moneta**, quante munizioni contiene una cassa di munizioni.

7.7 GET e SET

7.7.1 Leggere e scrivere

Trascinando la variabile nell'Event Graph, Unreal chiede che cosa farne. Ed ecco "l'ultima cosa importante": una variabile può essere usata in due modi.

GET significa **prendere** un'informazione: si **legge** il valore della variabile. "Se io voglio sapere quante munizioni hai, faccio GET."

SET significa **cambiare** il valore: si **imposta** il valore della variabile.

[3] A voce il docente chiama "verde" sia l'intero (che nell'editor è verde acqua, tendente all'azzurro) sia il float (verde chiaro), e in un altro momento chiama "blu" l'intero che viene convertito in stringa. Nella tabella si riportano i colori come appaiono nell'editor: il principio, comunque, è che pin dello stesso colore si collegano tra loro.

Un GET di var1 restituisce 25, ma da solo non serve a nulla: lo si collega al Print String. Il pin dell'intero è verde, quello del testo è rosa: Unreal **converte da solo** il numero in testo, inserendo un nodo di conversione, e il Print String può stamparlo. Play: 25, tre volte (tre cubi); cancellati gli altri due cubi, 25 una volta sola, "cioè quando parte".

Lo stesso con un float: una seconda variabile *var2*, di tipo float, con valore $-1{,}45$, letta con un GET e stampata. "Per ora so che sembrano numeri, abbastanza noiosetti, ma in poco tempo diventa codice effettivo di gioco, perché non è così distante."

7.7.2 Un numero che sale: il primo punteggio

Il primo esempio di codice "vero" è un contatore. Sull'Event Tick:

1. si prende la variabile (**GET** var1);
2. le si somma 1 (nodo **Add**, il $+$);
3. si prende la stessa variabile, questa volta con un **SET**, e le si collega il risultato della somma;
4. si stampa la variabile con un Print String.

Che cosa succede? "Quando passa il flusso qui in mezzo (perché il flusso passa sempre sul filo bianco) prende la variabile, le somma 1 e la imposta come variabile nuova. E poi prende la variabile e la stampa sullo schermo." Il numero parte da 25 e **sale di uno a ogni *frame***. "Potrebbe essere tranquillamente un punteggio: se so che il mio gioco va fisso a 30 fps, funzionerebbe già." (Il "se" non è casuale: il problema dei *frame* variabili tornerà nella sezione 7.17.)

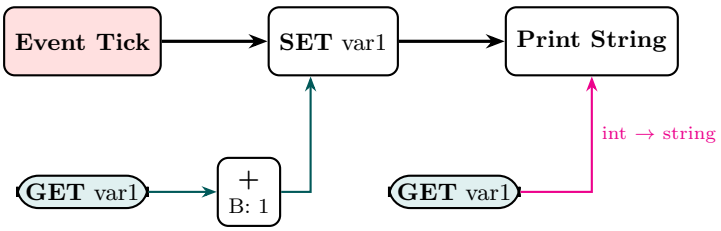


Figure 7.2: Il contatore: a ogni *frame* il valore di var1 (25 all'inizio) viene letto, aumentato di uno e reimpostato, poi stampato. Il filo nero spesso è il flusso (bianco nell'editor); i fili colorati sono dati.

Approfondimento: un SET restituisce il valore. Nell'editor il nodo SET ha anche un'uscita: il valore appena impostato. Si potrebbe quindi collegare direttamente quell'uscita al Print String, senza un secondo GET. Il risultato è identico; la versione del docente, con un GET separato, ha il pregio di rendere esplicito che la variabile, dopo il SET, *contiene* il nuovo valore.

7.8 Il personaggio: tasti, lanci, scala e teletrasporto

7.8.1 Un tasto come evento

Arrivati a questo concetto "abbastanza fondamentale", si passa a qualcosa di più interessante: il **personaggio**. Si chiude il cubo e si apre il ThirdPersonCharacter. Il codice che c'è già (il movimento, la camera) non interessa: lo si ignora completamente e si lavora in uno spazio libero dell'Event Graph.

Tasto destro e si cerca **Keyboard**: compaiono "tutti quanti i tasti possibili e immaginabili", e ognuno è un **evento**. Il docente sceglie la **E** e le collega un Print String. "In questo modo avrò creato un'interazione." L'evento di partenza è il tasto, il nodo è il Print String: premendo E, compare *Hello*. (Prima di provarlo, il docente cancella dalla scena l'ultimo cubo, che continuava a riempire lo schermo di messaggi.)

7.8.2 Launch Character

Qualcosa di più interessante: **Launch Character**, che "potrebbe sembrare molto più complesso, ma non lo è così tanto": prende il personaggio e lo **lancia**, con la velocità indicata. Il docente scrive sulla z "mille milioni", preme E, e il personaggio scompare verso il cielo. "Molto semplicemente, quindi, con questi eventi io posso far succedere qualsiasi cosa nel gioco."

7.8.3 La scala: la prima abilità

Il personaggio ha diverse **variabili interne**, per esempio la sua grandezza. Tutto ciò che si cambia si cambia con un SET, e per la grandezza il nodo è **Set Actor Scale 3D**. Con una scala di $3 \times 3 \times 3$ collegata alla E, premendo il tasto il personaggio "diventa gigantesco". "Quindi ho creato effettivamente un'**abilità** per il mio personaggio." Per ora resta così e non torna come prima; "poi lo farà".

Scala, posizione, rotazione. Il docente fissa le tre cose base della trasformazione di un oggetto, che hanno ognuna il proprio SET e il proprio GET:

- **scale** per la grandezza;
- **location** per la posizione;
- **rotation** per la rotazione.

7.8.4 Set Actor Location: il teletrasporto

Con **Set Actor Location** si può dire: "quando premo questo tasto mi teletrasporta" in una posizione precisa, qui 2500. Ovunque si trovi, il personaggio finisce lì, e se la posizione è nel vuoto "mi ritrovo all'infinito".

Teletrasportarsi nel terreno. A uno studente il teletrasporto in $(0, 0, 0)$ manda il personaggio dentro la struttura di partenza: nella mappa del template, l'origine è **sotto** quella struttura. Basta mettere una z più alta per partire sopra. A un altro, che aveva usato valori enormi, il personaggio è finito "fuori dal mondo, proprio oltre la Skybox".

Sommare alla posizione. Come prima si sommava 1 alla variabile, così si può sommare qualcosa alla posizione: **Get Actor Location**, più un vettore (per esempio +200 sulla z), poi Set Actor Location. A ogni pressione il personaggio si teletrasporta 200 unità più in alto. "E quindi già così si possono fare molte cose interessanti."

7.8.5 Cinque minuti per esplorare

"Già solo con questo" il docente lascia cinque minuti di prove libere, con SET e GET vari, per prendere la manualità. Visto che il PC non regge la condivisione su Discord, chi è da remoto condivide su Zoom, uno per volta. Il messaggio di fondo: "è chiaro il flusso, fondamentalmente: dagli eventi passa per tutti quanti i singoli nodi". Una studentessa, nel frattempo, ha già "completamente ucciso il personaggio".

7.9 Il Character Movement

7.9.1 Dove si trova il movimento

Durante le prove arriva la domanda giusta: dove si trova la forza del **salto**? Più in generale, dove sono tutte le cose del movimento di base del giocatore? Si trovano tutte nel componente **Character Movement**, nell'elenco dei componenti a sinistra. Lo si trascina nell'Event Graph e da lì si può fare il SET (o il GET) di "qualsiasi cosa, perché in realtà troviamo di tutto qui dentro". Due esempi:

Jump Z Velocity quanto salta in alto il personaggio. Collegato alla E, con un valore alto: prima del tasto "salta poco", dopo salta "super in alto".

Max Walk Speed la velocità massima di camminata, in **centimetri al secondo**. Prima del tasto si cammina lentamente, dopo "divento velocissimo".

"Se volete modificare qualsiasi cosa sul movimento del personaggio, la trovate qui." E naturalmente "con tutti quanti i SET c'è anche il GET". Se invece si vuole cambiare il valore **di base**, quello di partenza, si clicca sul Character Movement e lo si cambia nel pannello dei dettagli, "come le variabili: stesso principio".

La velocità massima non è la velocità. Uno studente prova un valore astronomico ("un trilione di trilioni") e si stupisce che il personaggio non parta come un razzo. Il motivo: *Max Walk Speed* è la velocità **massima**; il personaggio deve comunque **accelerare** "pian pianino" per arrivarci, e l'accelerazione è un altro parametro. "So che ce ne sono tantissimi, di numeri: è purtroppo così."

7.9.2 Premuto e rilasciato: lo sprint

Uno studente ha fatto un'idea più fine: quando si **preme** il tasto la velocità diventa alta, quando lo si **rilascia** torna bassa. Il nodo del tasto ha due uscite, **Pressed** e **Released**: a ciascuna si collega un Set Max Walk Speed con un valore diverso. È, di fatto, uno **sprint**.

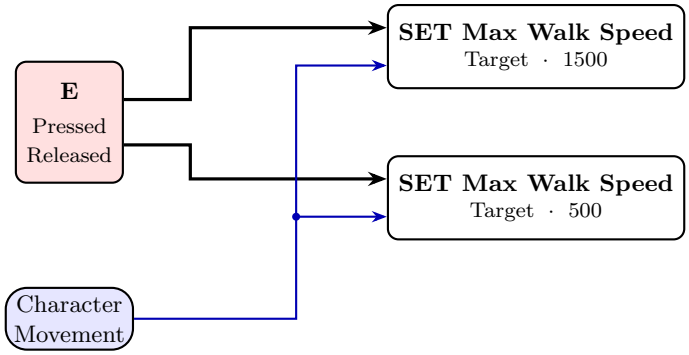


Figure 7.3: Lo sprint costruito da uno studente: alla pressione del tasto la velocità massima sale, al rilascio torna normale. Il Character Movement (blu, un riferimento a un componente) è il *target* di entrambi i SET. I valori sono indicativi.

Approfondimento: lo sprint del libro. Romero e Sewell costruiscono esattamente lo stesso meccanismo nel capitolo sul movimento del giocatore: un evento di input *Sprint* la cui uscita *Pressed* imposta un Max Walk Speed alto e la cui uscita *Released* lo riporta al valore normale. La differenza è solo che il libro usa un'azione di input con un nome (*Input Action*), che può essere associata a più tasti o al controller, invece del tasto fisico. È la via "pulita" per un gioco vero; per prototipare, il tasto diretto va benissimo.

7.9.3 Delay e Branch

Altri due nodi compaiono durante le prove, su richiesta degli studenti.

Delay "Io premo il tasto, aspetta tre secondi, e poi fa qualcos'altro." Il flusso entra, aspetta il tempo indicato, e solo dopo esce. Il docente lo riprenderà a fine lezione per spiegare come funziona il flusso (sezione 7.22).

Branch Il nodo della **decisione**, che uno studente trova da solo ("quello che cercavo, grazie"): riceve un booleano e manda il flusso sull'uscita *True* o *False*. È il rombo del flowchart (sezione 6.11) tradotto in Blueprint. Con un Branch lo studente alterna il personaggio grande e piccolo; il docente gli fa notare che aveva **invertito** le due uscite.

7.10 Le interazioni tra oggetti: la trigger box

7.10.1 Un'area che si accorge di chi entra

Il passo successivo sono le **interazioni tra oggetti diversi**. Si crea un nuovo Blueprint (tasto destro, Blueprint Class, Actor) chiamato **trigger**: sarà una **trigger box**, "cioè quando il personaggio ci entra succedono cose".

Aperto, è un Blueprint normalissimo, vuoto. Il menu **Add** elenca "tutte quante le cose che possiamo attaccare su qualsiasi oggetto": un audio, una mesh, una camera, un *behavior tree* per il movimento in giro, e così via. Qui si aggiunge una **Box Collision** (non un cubo): "una box che mi rileva non appena qualcosa ci entra dentro".

Selezionata la box, nel pannello dei dettagli ci sono tutte le sue proprietà (posizione e il resto). Scendendo "in fondo, in fondo, in fondo" si trovano gli **eventi**: con il + accanto a **On Component Begin Overlap** si crea un nuovo evento di inizio, chiamato **quando qualcosa entra nella box**.

La prova. Un Print String sull'evento, la trigger box trascinata nella scena e un po' ingrandita. Quando si preme Play non la si vede, perché **non ha una parte visiva**: ci si basa solo sul messaggio. Ci si cammina dentro e compare la scritta.

7.10.2 Controllare chi è entrato: il cast

Per ora, però, *qualsiasi cosa* entri nella box fa scattare l'evento: "ci va sopra un gatto, un pallone che passa lì: triggererebbe sempre". Se si vuole che reagisca **soltanto al giocatore**, e soprattutto se si vogliono modificare cose *sul* giocatore, serve un passo in più.

L'evento ha diverse uscite; quella che interessa è **Other Actor**, l'attore contro cui la box ha colliso. Da lì si tira fuori un **Cast To ThirdPersonCharacter**: "così controlliamo che sia soltanto il player che ci abbia colliso sopra". Il cast ha due uscite di flusso:

- se l'attore è davvero il ThirdPersonCharacter, il flusso prosegue dall'uscita principale;
- se è "una macchina, un pallone, qualsiasi cosa", finisce su **Cast Failed** e, se lì non è collegato niente, non succede niente. "Va sopra solamente se è il tipo giusto."

E la cosa comoda è che il cast restituisce anche il riferimento al personaggio (*As ThirdPersonCharacter*), con cui si può fare tutto ciò che si faceva dentro il personaggio.

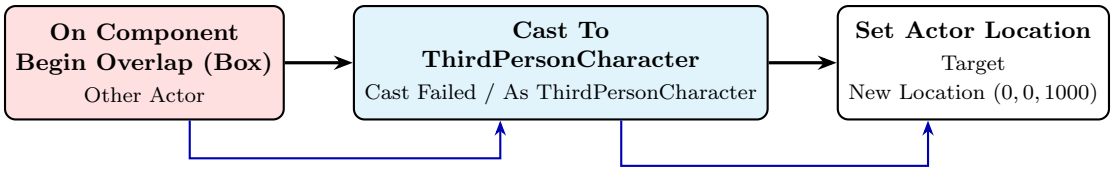
7.10.3 L'indovinello del *target*

Il docente collega al cast un **Set Actor Location** verso $(0, 0, 1000)$ e chiede: "se io facessi così, sarebbe sbagliato. Vediamo se qualcuno ci arriva del perché." Lo fa vedere: si entra nella box... e la box sparisce. "Non c'è più, ed è finita lì."

La spiegazione: ogni nodo che agisce su un oggetto ha un pin **Target**, e il target di base è **self**, sé stesso. "Se il codice è nella trigger box, teletrasporta la trigger box, non il player." Basta collegare il riferimento del personaggio uscito dal cast al pin Target, e a essere teletrasportato è il giocatore. Il docente ci rientra, e il personaggio cade dall'alto "tipo Portal, che cade all'infinito".

Sweep e Teleport. Sul Set Actor Location compaiono due caselle, *Sweep* e *Teleport*, e uno studente chiede a che cosa servano. Servono nei casi in cui lo spostamento è usato **come un movimento**: se, per esempio, ci si sposta da un punto all'altro e in mezzo c'è un'altra trigger box, con lo *sweep* attivo l'oggetto "la attraversa" davvero e la trigger box scatta. Per un teletrasporto puro non servono.

Approfondimento: che cos'è un cast. Il libro di Romero e Sewell dedica al *casting* un intero paragrafo, ed è utile per capire perché il nodo serve. Il pin Other Actor restituisce un riferimento di tipo **Actor**, generico: un Actor "sa" fare solo le cose che sanno fare tutti gli actor. Il Cast To **prova a convertire** quel riferimento in un tipo più specifico; se l'istanza è davvero di quel tipo, la conversione riesce e restituisce



senza questo filo il Target è self*: si teletrasporta la trigger box, non il giocatore*

Figure 7.4: La trigger box che teletrasporta il giocatore. Il cast verifica che l'attore entrato sia il personaggio e ne restituisce il riferimento, che va collegato al Target del Set Actor Location.

un riferimento che dà accesso alle variabili e alle funzioni della classe specifica (il Character Movement, le monete, la vita). Il libro lo usa nei due modi visti a lezione: per **verificare in sicurezza** che un riferimento sia del tipo desiderato (il suo esempio è proprio una collisione con il personaggio del template in terza persona) e per **accedere** alle variabili di una sottoclasse.

7.10.4 Tutto ciò che si poteva fare là, si può fare qua

Da qui il personaggio è a disposizione della trigger box: il Character Movement, i Delay, tutto ciò che si faceva dentro il personaggio "adesso possiamo farlo di là e triggerarlo quando ci vado sopra". Il docente lascia lavorare, girando tra i banchi.

7.11 Generare, distruggere, cadere

7.11.1 Spawn Actor e Destroy Actor

Per chi vuole spingersi oltre, il docente suggerisce un nodo per **generare** oggetti: **Spawn Actor** (nell'editor, *Spawn Actor from Class*). Gli si indica che cosa generare, per esempio un altro personaggio, un'altra trigger box, qualsiasi cosa; poi il pin **Spawn Transform** va diviso con il tasto destro, **Split Struct Pin**, per dirgli **dove**, con **che rotazione** e con **che scala**. "E così si genera lì." A uno studente che non vede comparire nulla, la correzione è proprio questa: "ti manca la transform: dove, con che rotazione e con che scala".

L'opposto è **Destroy Actor**, "per distruggere qualsiasi cosa".

Lo spawn che si spawna. Un caso istruttivo: uno studente fa sì che il Blueprint *test* generi un altro *test*… nello stesso *frame* in cui viene creato. Ogni copia, appena creata, esegue il proprio Begin Play e ne genera un'altra: il gioco continua a generare oggetti, tutti nello stesso punto, e si blocca. Generare qualcosa ha senso se lo si fa in un'unità di tempo, o su un evento, non in una catena infinita.

7.11.2 Simulate Physics

"Se vuoi mettere la gravità, facilissimo." Si prende un oggetto, per esempio una sfera, e nei dettagli della sua mesh si attiva **Simulate Physics**: l'oggetto "simula la fisica", cade, rotola, può essere spinto.

7.11.3 Ogni componente è una miniera di variabili

Un'osservazione di metodo: tutto ciò che si trova nell'elenco dei componenti (la mesh, la camera e così via) ha "un sacco di parametri", per esempio il **FOV** della camera. E **tutte queste informazioni possono essere prese e modificate**: "GET e SET, GET e SET, con tutto quanto quello che c'è sull'engine". Si può persino cambiare il personaggio e renderlo qualsiasi altra cosa.

7.12 Le monete

7.12.1 Costruire la moneta

Il terzo oggetto della giornata è la **moneta**. Tasto destro, Blueprint Class (sempre nel Content, "così creiamo roba nuova"), Actor, nome **coin**. Dentro:

- una **Sphere Collision**, "come la box collision, ma sferica, incredibilmente";
- un piccolo **cilindro**, ridimensionato, per farne visivamente una moneta.

Le monete vanno piazzate in giro per la mappa. Sono uguali, quindi si trascina più volte lo stesso Blueprint.

Spostare, ruotare, scalare. A chi chiede come si rimpiccioliscono le cose, il docente ricorda i due modi: selezionare l'elemento e cambiarne la **scala** nei dettagli, oppure usare i tasti in alto nella viewport (**muovi**, **ruota**, **scala**) e trascinare il gizmo.

7.12.2 Raccoglierla

Come con la trigger box: sulla Sphere Collision si crea l'evento **On Component Begin Overlap**, si fa il **Cast To ThirdPersonCharacter** e, se il cast riesce, **Destroy Actor**. Che cosa succede? Il docente lo chiede alla classe, poi lo verifica: si cammina sulla moneta e la moneta sparisce.

7.12.3 Contarle: la variabile sta sul personaggio

Le monete però "non ci stanno incrementando nessun valore". Serve un contatore, e il punto chiave è **dove** metterlo: **nel personaggio**, non nella moneta. Sul ThirdPersonCharacter si crea la variabile **coin**, di tipo **integer**, "perché di monete non ne abbiamo 0,3, tendenzialmente".

Poi, nella moneta, dopo il cast:

1. dal riferimento *As ThirdPersonCharacter* si fa **GET coin**;
2. si somma 1 (**Add**);
3. dallo stesso riferimento si fa **SET coin** con il risultato;
4. infine **Destroy Actor**, che distrugge la moneta (il target è *self*, ed è giusto così).

"Mi raccomando, da qua, dal ThirdPersonCharacter": se si cercasse *coin* nella moneta stessa, non la si troverebbe, "perché sta cercando il valore delle monete nella moneta stessa, invece l'abbiamo messo sul personaggio".

Per controllare che funzioni, sul personaggio un Event Tick con un Print String che stampa la variabile: si parte da zero, si prende una moneta, uno, due. "Perfetto, logicamente torna."

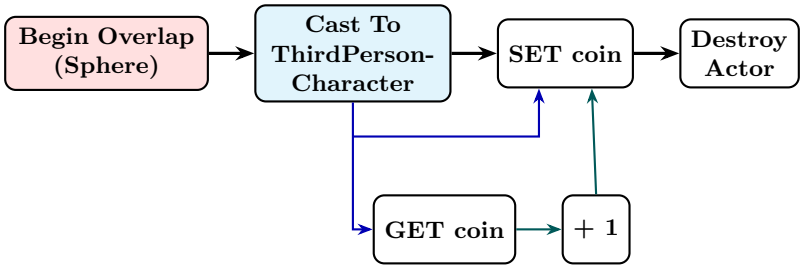


Figure 7.5: La moneta: verificato che a toccarla sia il personaggio, si legge la sua variabile *coin*, le si somma uno e la si reimposta *sul personaggio* (filo blu: il riferimento uscito dal cast); poi la moneta si distrugge.

7.12.4 Il debug in diretta

Le prove degli studenti producono una piccola collezione di errori tipici, ognuno con la sua lezione.

- **Un SET che non cambia niente.** Uno studente prende le monete e le reimposta "con valori identici": il GET va direttamente nel SET, senza la somma. "Non stai cambiando nulla, devi aumentare."
- **Fili non collegati.** A un altro manca il collegamento del riferimento al personaggio, oppure il risultato della somma non torna nel SET. "I bianchi sempre con i bianchi", e il risultato si ricollega al SET.
- **La collisione sbagliata.** Il caso più ostinato: lo studente ha fatto tutto bene, ma la moneta non si raccoglie. Prima verifica: "sei sicuro che quello lì sia un coin, effettivamente? Quello che hai messo nella scena, al 100%?" Seconda verifica: un Print String all'inizio dell'evento, "vedete se qua stampa qualsiasi cosa", per capire **dove** si ferma il flusso. Il problema, alla fine, era che l'evento controllava le collisioni **del cilindro**, non della sfera, e il cilindro non aveva le impostazioni di collisione giuste: bisognava selezionare la sfera e sistemare le *Collision* nei dettagli. "Effettivamente era complicato."
- **La stampa che si accumula.** Stampando a ogni *frame*, i messaggi si impilano sullo schermo. Nel Print String si può impostare la **durata** del messaggio (*Duration*): a zero, "così ogni *frame* si cancella" e resta visibile solo il valore corrente.

Approfondimento: il Print String come strumento di debug. La tecnica usata con lo studente, mettere un Print String in un punto del flusso per vedere se il flusso ci arriva, è il modo più elementare e più usato per il *debug* dei Blueprint. Si procede per bisezione: se il messaggio all'inizio dell'evento non compare, il problema è a monte (l'evento non scatta: collisioni, oggetto sbagliato in scena); se compare, si sposta il messaggio più avanti, dopo il cast, dopo il SET, fino a trovare il punto in cui il flusso si interrompe.

7.13 L'editor in inglese, e il salvataggio

A metà lezione il docente si accorge che alcuni hanno l'editor in italiano, e lo dice chiaramente: **va messo in inglese**, "perché altrimenti è un casino": i nomi dei nodi e dei menu non corrispondono a quelli usati a lezione e "non riuscite a seguire bene". Il percorso è **Edit** → **Editor Preferences** → **Region & Language** → *English*. In italiano: Modifica → Preferenze dell'editor → Regione e lingua. (Non *Project Settings*, che è un'altra cosa: a una studentessa, cercando lì, si apre invece il menu con Annulla, Cronologia, Taglia.)

Il layout perduto. Valerio, "armeggiando", ha chiuso il pannello degli asset e non trova più niente. Soluzione: **Window** (Finestra) → in fondo, **Load Layout** → il layout **UE4 classico**. Per farselo mostrare, il docente chiede di condividere **lo schermo intero**, non solo la finestra di Unreal, altrimenti non vede le finestre che si aprono.

Salvare, sempre. E una regola da trasformare in riflesso: **Ctrl + Shift + S**, salva tutto. "Deve essere un tic: Ctrl Shift S, sempre, costantemente, a prescindere."

7.14 Altri nodi per stimolarvi

Per dare spunti, il docente mostra "altri due o tre nodi" che possono tornare utili, soprattutto con le trigger box.

Open Level Permette di **cambiare mappa**, cioè cambiare scena: "raggiungo la fine del livello e mi teletrasporto al livello successivo". Esiste in due versioni, *by Name* e *by Object Reference*; con la seconda si seleziona la mappa da un menu, senza scriverne il nome.

Play Sound 2D Riproduce un suono qualsiasi tra quelli presenti nel progetto, oppure uno importato.

Quit Game Chiude il gioco.

7.14.1 Confronti e Branch

Le monete possono anche essere **confrontate**: si controlla se sono maggiori o minori di un certo valore, per esempio per far passare al livello successivo solo chi le ha prese tutte. "Molto facile": un nodo **maggiore** (>) restituisce un booleano, e il booleano va in un **Branch**. "Se ne ho più di 8 finirà qua, se ne ho meno di 8 finisce qua."

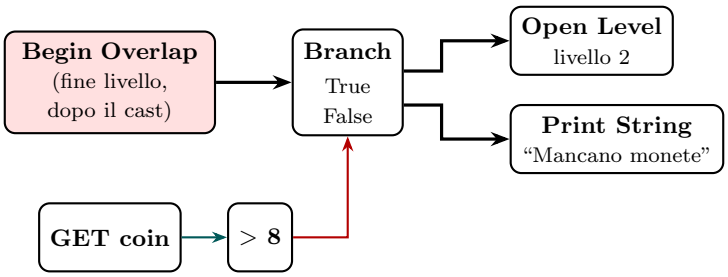


Figure 7.6: Passare al livello successivo solo con abbastanza monete: il confronto restituisce un booleano (rosso), il Branch sceglie il ramo. Il ramo *False* è un'aggiunta per mostrare che anche l'altra uscita può fare qualcosa.

7.14.2 Flip Flop e Sequence

Due nodi di controllo del flusso, mostrati più avanti nella lezione:

Flip Flop "Un nodo bellissimo": fa andare il flusso **una volta sopra e una volta sotto**, alternando le due uscite (A e B) a ogni esecuzione. Utile, per esempio, per un tasto che a ogni pressione attiva e disattiva qualcosa: schiaccio, fa una cosa; rischiaccio, fa l'altra. È la soluzione elegante al personaggio grande/piccolo.

Sequence Suggerito a uno studente che voleva eseguire più cose una dopo l'altra da un solo evento: il Sequence ha più uscite e le esegue **in ordine**.

Approfondimento: i nodi di controllo del flusso. Il libro di Romero e Sewell raccoglie questi nodi in un paragrafo dedicato (*flow control nodes*). Del **Flip Flop** precisa che ha anche un'uscita booleana, *Is A*, vera quando sta eseguendo il ramo A, e avverte che **non funziona dentro una funzione**, perché la variabile interna che ricorda lo stato viene cancellata a ogni chiamata (eseguirebbe sempre A). Del **Sequence** dice che esegue tutte le azioni collegate al primo pin, poi tutte quelle del secondo, e così via, ed è utile per organizzare gruppi di azioni. Nello stesso paragrafo compaiono altri nodi che torneranno nel corso: *Do Once*, *Do N*, *Gate*, *MultiGate*, *For Each Loop*, gli *Switch*.

7.15 Il task in classe: due livelli

7.15.1 La consegna

"Visto che in realtà siamo con larghissimo anticipo", il docente dà un piccolo **task da fare in classe**, "non a casa" ("Federica stava già andando in crisi: non vi preoccupate"). La consegna:

1. creare **due livelli**, separati;
2. uno deve **portare all'altro**: entrando in un'area, oppure con una condizione, per esempio dopo aver preso un certo numero di monete;
3. ci deve essere **almeno una zona** (una trigger box) che **cambia qualche parametro del personaggio**: lo rende più grande, lo lancia in aria, lo rende estremamente lento, lo fa saltare più in alto, gli dà poca frizione "come fosse ghiaccio". "Quello che vuoi tu, non è importante."

Begin ed End Overlap. Un suggerimento: sulla box, oltre al *Begin Overlap* c'è anche l'**End Overlap**, chiamato quando il personaggio **esce** dall'area. Così si può fare in modo che un effetto valga **soltanto dentro quell'area**: lo si attiva entrando e lo si annulla uscendo.

7.15.2 Requisiti minimi, e tutto il resto

Il docente spiega anche **come saranno fatti i task** del corso: ci sarà sempre un **task minimo**, "requisiti minimi, minimi, minimi: se non fate quello non passate". Ma l'invito è a fare sopra "tutto quanto quello che voi volete", perché "l'importante è che ci prendiate la mano". Il task non è "dovete fare questo", è "dovete fare *di base* questo, e poi fate altro". Portali che fanno entrare da una parte e uscire dall'altra, monete che rendono giganti: "fatelo, assolutamente". E chi lo fa bene ha un vantaggio: "ci sarà un task abbastanza simile", e si potrà riprendere il lavoro da qui.

Il codice in inglese. "Piccola cosa": per le prime due o tre lezioni non verrà richiesto, ma dai task successivi sì, **tutto il codice va scritto in inglese**: nomi delle variabili, dei Blueprint, delle funzioni.

7.15.3 Creare il secondo livello

Diversi studenti non sanno come creare una mappa nuova ("ma intendo un altro progetto?" "No, no, un'altra mappa, semplicemente"). La procedura:

1. **File** → **New Level**, e scegliere **Basic** come punto di partenza ("almeno hai il pavimento e il cielo");
2. mettere in scena **qualcosa**, qualsiasi cosa: una scena vuota non si lascia salvare;
3. salvare, con un nome, per esempio *Mappa2*.

A quel punto i livelli sono due: nel Content Browser c'è *Mappa2*, con doppio clic si apre; nella cartella del template c'è quella di prima. Per passare dall'una all'altra in gioco: **Open Level (by Object Reference)**, si seleziona *Mappa2* dal menu, e quando il flusso passa di lì il livello si apre.

Approfondimento: dove nasce il personaggio. Una mappa *Basic* non contiene il personaggio del template, eppure, aprendola dal gioco, il personaggio c'è: lo genera il *Game Mode* del progetto, che nel template in terza persona usa il ThirdPersonCharacter come personaggio di default. Lo fa comparire nel punto indicato da un *Player Start*; se la mappa non ne ha, lo genera nella posizione predefinita. Per avere il controllo su dove si comincia, conviene trascinare un Player Start nella nuova mappa.

7.16 Le monete che ruotano

Uno studente vuole far **ruotare** le monete su sé stesse. Il movimento nel tempo è "una cosa che succede tutti i *frame*", quindi va sull'**Event Tick**. Il primo tentativo è un **Set Relative Rotation** sul cilindro: ma così la rotazione viene *impostata* sempre allo stesso valore, "una volta, e non è che ti aggiungerà una rotazione: rimarrà sempre quella". Serve lo stesso schema del contatore: prendere la rotazione attuale (GET), sommarle qualcosa, reimpostarla (SET). Per sommare due rotazioni esiste un nodo apposito, **Combine Rotators**. (In alternativa, Unreal ha nodi che aggiungono direttamente una rotazione, come *Add Relative Rotation* o *Add Actor World Rotation*, usato nel capitolo 1.2.)

Il personaggio che ruota. Un altro studente prova a far ruotare il personaggio stesso, con un risultato "un filino buggato" ("tipo Mario Galaxy"). Il motivo è che la rotazione del personaggio passa da una cosa che si chiama **Controller**, "un pochettino più complessa da gestire del resto"; e in più il nodo non aveva il target collegato al giocatore.

7.17 L'Event Tick e i *delta seconds*

7.17.1 Un oggetto che si muove

"Un'altra informazione super importante, che vi servirà molto." Il docente crea un nuovo Blueprint, **Moving Object**, con dentro un cubo, e sul suo Event Tick costruisce lo schema ormai noto: **Get Actor Location**, più un vettore (10 sulla x, "in avanti"), **Set Actor Location**. Lo mette in scena, preme Play: l'oggetto si muove. "Funziona."

7.17.2 Il problema grave

"In realtà questa cosa non va bene. Concettualmente sì, ma c'è un problema **molto, molto grave**." Chi lo trova vince cento punti ("che non vogliono dire niente"). Il problema è che **non tutti i PC vanno alla stessa velocità**. L'oggetto avanza di 10 unità *a ogni frame*: se il computer fa molti fps, si muoverà molto più velocemente; se ne fa pochi, molto più lentamente. Lo stesso gioco, su due macchine diverse, si comporta in due modi diversi.

7.17.3 La soluzione: moltiplicare per i delta seconds

"Molto, molto facile." Se si deve fare qualcosa **nel tempo**, collegato all'Event Tick, bisogna **moltiplicare il valore per i delta seconds**. L'Event Tick ha un'uscita, **Delta Seconds**, che "sembra complessa ma non lo è": è **quanto tempo è passato tra l'ultimo *frame* e quello prima**.

- Se il gioco sta laggando, cioè va molto lento, questo numero è **alto**, e l'oggetto si muove di più a ogni *frame*.
- Se il gioco va fluido, il numero è **basso**, e l'oggetto si muove di meno a ogni *frame*.

Il risultato è che in un secondo l'oggetto percorre sempre la stessa distanza. Il valore è "molto legato al *frame rate*": facendo 1/delta seconds si ottiene proprio il frame rate. E quindi, con delta seconds × 100, l'oggetto si muoverà su tutti i computer di **100 centimetri al secondo**. "Punto. Non importa su che PC lo metto."

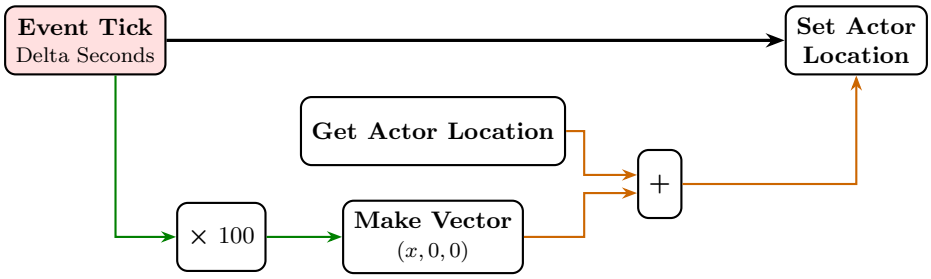


Figure 7.7: Un oggetto che avanza di 100 centimetri al secondo su qualsiasi computer: lo spostamento di ogni *frame* è velocità × delta seconds. Il verde è un float, il giallo (qui arancio) un vettore.

"Sul mio PC va." È un errore da cui guardarsi, perché è subdolo: "sul mio PC va, poi lo metto sul suo e non va più"; oppure va, "e poi ogni tanto non mi va, quando apro qualche browser". "Colpa di questa roba qua." La regola è semplice: nel 99% dei casi basta **moltiplicare per i delta seconds**. Lo stesso vale, a ritroso, per il contatore della sezione 7.7, che funzionava come punteggio solo "se il gioco va fisso a 30 fps".

Approfondimento: i conti. Con i numeri del docente: senza delta seconds, 10 unità per *frame* fanno 600 cm al secondo a 60 fps e 300 a 30 fps. Con i delta seconds e la velocità 100, a 60 fps ($\Delta t \approx 0{,}0167$ s)

ogni *frame* sposta l'oggetto di circa 1,67 cm; a 30 fps ($\Delta t \approx 0{,}033$ s) di circa 3,33 cm. In un secondo, in entrambi i casi,

$$\sum_{\text{frame in 1 s}} 100 \cdot \Delta t = 100 \cdot 1 \text{ s} = 100 \text{ cm}.$$

Il 100, insomma, non è più "centimetri per *frame*" ma **centimetri al secondo**. Lo stesso concetto, con una piattaforma che ruota, è nel capitolo 1.2 (sezione 1.2.20). Romero e Sewell, presentando i *timer*, ricordano anche l'altro lato della medaglia: tutto ciò che è collegato all'Event Tick viene eseguito a ogni *frame*, e ciò che non ha bisogno di aggiornarsi così spesso può essere eseguito a intervalli più lunghi, risparmiando lavoro al computer.

7.18 Ripasso: la trigger box, passo per passo

Verso la fine, visto che più di uno dovrà "riguardare la lezione", il docente crea una cartella nuova ("ormai c'è un po' di roba") e rifà la trigger box da capo, chiamandola *trigger 3*. Il procedimento completo:

1. un **Actor**, cioè un oggetto che si può mettere nella scena;
2. un componente **Box**: "questa sarà la nostra trigger box". Da sola non fa ancora nulla;
3. in fondo ai dettagli della box, il + su **On Component Begin Overlap**: l'evento che può scatenare qualsiasi cosa. Delle tante uscite interessa solo **Other Actor**, "l'attore che l'ha toccato";
4. da Other Actor (attenzione a prendere il pin giusto: una studentessa lo stava tirando fuori da *Overlapped Component*) un **Cast To ThirdPersonCharacter**, "soltanto per controllare che sia quell'oggetto lì e modificare quello lì";
5. una volta ottenuto il riferimento, gli si può fare qualsiasi cosa: teletrasportarlo, distruggerlo, ruotarlo. Per il movimento, dal riferimento si prende il **Character Movement** ("è in fondo") e da lì *Set Jump Z Velocity*, *Set Ground Friction*, *Set Max Walk Speed* e così via.

7.18.1 Perché non la Trigger Box già pronta?

Una studentessa ricorda che esiste una **Trigger Box** già pronta, da trascinare nella scena senza creare nessun Blueprint. "Sì, c'è anche la trigger box di base." Perché allora farla a mano? Perché "tutto quanto il resto poi funziona direttamente così". L'esempio è la porta: se si vuole mettere una trigger box *su una porta*, con la Trigger Box pronta non si può, bisogna creare due oggetti separati. Facendolo invece con un componente di collisione dentro un Blueprint, si può mettere la collisione su qualsiasi cosa. "Se lo sai fare così, sai fare tutto. Se lo sai fare soltanto nell'altro modo, poi non riesci a fare il resto." La Trigger Box pronta "non è altro che questa cosa qua", la stessa idea, solo che così ci si possono fare altre cose sopra.

Approfondimento: la Trigger Box nel Level Blueprint. Il libro di Romero e Sewell mostra anche l'altra strada: una Trigger Box piazzata nella mappa e gestita dal **Level Blueprint**, il Blueprint della mappa, dove si seleziona la box e si aggiunge l'evento *OnActorBeginOverlap (TriggerBox)*. Funziona, ma il codice resta legato a quella mappa; il Blueprint con il componente di collisione, invece, è un oggetto **riutilizzabile**, che si trascina in qualsiasi livello quante volte si vuole. È la stessa ragione per cui il docente insiste sul metodo "lungo".

7.19 Un sistema di vita

7.19.1 La proposta

Qualcuno ha fatto trappole che "fanno molto male" al personaggio. "E se muore?" "No, non muore": per ora non c'è niente che lo faccia morire. Il docente propone allora un esercizio per tutti: un **sistema di vita**.

1. una **variabile sul personaggio** che rappresenta la vita: un **float** o un **intero**, "dipende da che gioco è";
2. un Print String sull'Event Tick che la stampa a ogni *frame*, "così la vediamo in alto a sinistra";
3. ogni **trappola** od ostacolo, quando ci si va contro, la **abbassa**;
4. quando arriva a zero (o sotto), si **distrugge il personaggio** oppure si **fa ripartire il livello**.

Lo schema è lo stesso delle monete: le monete stesse "possono essere la vita del player", con il segno cambiato. "Cercate di creare qualcosa di un pochettino più complesso, di un po' più completo. Siate creativi." E alla fine delle lezioni il docente vorrebbe vedere i progetti di tutti.

7.19.2 Ripartire: Open Level sullo stesso livello

Un trucco "comodissimo": se con **Open Level** si riapre **lo stesso livello** già aperto, il livello **si resetta**. Per far ripartire un livello quando il personaggio muore, quindi, basta un Open Level verso la mappa corrente.

Il livello che riparte all'infinito. Uno studente ottiene un effetto curioso: il livello si ricarica di continuo. "Stai morendo costantemente, perché parti con **zero vita**." La variabile aveva valore di default 0: al primo controllo la condizione "vita ≤ 0" è vera, il livello si riapre, la vita riparte da 0, e così via. Serve un valore iniziale sensato, e il controllo va fatto con un **Branch** ("serve un Branch, qua"), collegando "i colori con i colori".

7.19.3 Le variabili non sopravvivono al cambio di livello

"Cosa super importante": **quando si cambia livello, tutte le variabili si resettano**. Se si hanno 70 punti vita e si passa al livello successivo, si riparte con la vita di default. Lo stesso per le monete. "Mi raccomando": con il restart o con l'apertura di un livello nuovo si azzera tutto quello che si è visto finora.

Approfondimento: la Game Instance. Il libro di Romero e Sewell spiega perché, e indica la soluzione. Ogni volta che un livello viene caricato, **tutti gli actor** del livello (personaggio compreso) vengono distrutti e ricreati, e con loro le loro variabili. Esiste però una classe, la **Game Instance**, la cui istanza viene creata all'avvio del gioco e distrutta solo quando il gioco si chiude: i dati che contiene **persistono tra un livello e l'altro**. Per conservare vita, monete o punteggio durante il cambio di livello, li si salva lì (la Game Instance da usare si imposta in *Project Settings* → *Maps & Modes*). A lezione non la si usa ancora; è la risposta alla domanda che il task dei due livelli fa nascere da solo.

7.20 Actor, Pawn, Character; e il Line Trace

7.20.1 Le tre classi

"Un'informazione molto generale, che ci servirà più avanti." Quando si crea un Blueprint Class, l'editor chiede sempre da che cosa partire: Actor, Pawn, Character e così via (le opzioni più in basso "adesso non ci interessano proprio"). La differenza:

Actor **tutti gli oggetti** che si possono mettere in una scena.

Pawn gli oggetti **possedibili**: i personaggi, le macchine, tutto ciò che può essere controllato da un giocatore o da un'intelligenza artificiale, "quindi anche i gatti che passano in giro". Non le sedie, "perché noi tendenzialmente non possediamo le sedie".

Character dei Pawn, quindi possedibili, che hanno **già il sistema di movimento di base**: camminare, saltare, quello che si sta provando oggi con il Character Movement.

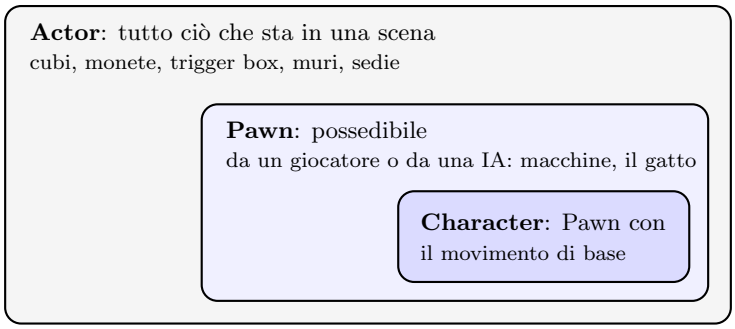


Figure 7.8: Actor, Pawn e Character: ogni insieme è contenuto nel precedente. Un Character è anche un Pawn, e un Pawn è anche un Actor.

Approfondimento: corpo e cervello. Il capitolo 1.2 (sezione 1.2.4) riporta la gerarchia completa dal libro di Romero e Sewell. L'immagine più utile è questa: il **Pawn** è il **corpo**, il **Controller** che lo possiede è il **cervello**. Un *Player Controller* è il cervello di un giocatore umano, un *AI Controller* quello di un'intelligenza artificiale. Spiega anche perché, poco prima, ruotare il personaggio era "un pochettino più complesso": la rotazione della visuale passa dal Controller. E il Character porta con sé, oltre alla mesh, una capsula per le collisioni e proprio il **Character Movement**, che gestisce camminata, salto, nuoto e volo.

7.20.2 Il Line Trace By Channel

"Per quelli che si sentono già bravi", un ultimo nodo, "potentissimo": **Line Trace By Channel**. "Si usa costantemente. Con questo potete fare un FPS solo con questa cosa." Il docente non lo spiega: lo lascia come sfida a chi vuole provarci, con l'invito a chiedere.

Approfondimento: che cosa fa un line trace. Il nodo traccia una **linea invisibile** tra due punti, *Start* ed *End*, e restituisce il **primo oggetto colpito** lungo quella linea (con il punto d'impatto e altre informazioni, raccolte in un *Hit Result*). Il *channel* indica quale tipo di collisione considerare, per esempio *Visibility* o *Camera*. Si capisce perché basti per un FPS: si parte dalla camera, si traccia una linea nella direzione in cui si guarda, e ciò che la linea colpisce è ciò che il proiettile ha colpito. Romero e Sewell lo trattano nel capitolo sui *trace*, insieme alla variante *Multi*, che restituisce tutti gli oggetti attraversati, e ai trace con forme (sfere, box, capsule) invece che con una linea.

7.21 Le regole dei collegamenti

Quando gli oggetti nell'Event Graph cominciano a essere tanti, il docente dà due o tre "regoline" con cui, dice, "fate tutto".

Destra con sinistra, mai destra con destra. Le uscite (a **destra** di un nodo) si collegano sempre con gli ingressi (a **sinistra** di un altro nodo). Mai destra con destra, mai sinistra con sinistra. Non è una convenzione: "non ha proprio senso logicamente". Collegare due uscite vorrebbe dire prendere un valore e, nello stesso punto, modificarlo; "è proprio sbagliato concettualmente".

Stesso colore, stessa famiglia. Tutte le cose dello **stesso colore** si abbinano: se hanno lo stesso colore, probabilmente vanno attaccate tra loro. Alcune si possono **convertire**, ma sono poche: per esempio da intero a stringa. In quel caso "ci pensa lui": se le si collega, l'editor inserisce il nodo di conversione.

I bianchi con i bianchi. I pin bianchi, il flusso, si collegano **solo con i bianchi**, e stanno **sempre in cima** al nodo: "è abbastanza facile".

7.22 Le domande di chi parte da zero

7.22.1 "Che valore ha la freccina bianca?"

Federica fa la domanda che probabilmente si stavano facendo in tanti: che differenza c'è tra il filo bianco e quelli colorati? "È una domanda molto base?" "Normalissima, è la lezione numero uno."

La risposta è un'immagine: bisogna pensare a delle **prese della corrente**. L'evento (il tasto E, per esempio) è il **generatore di corrente**, "da dove parte tutto quanto"; tutti i nodi collegati con il **cavo bianco** sono collegati alla corrente, e quando la corrente li attraversa **si eseguono**. Gli altri fili **non portano corrente**: portano soltanto **informazioni**, e da soli non eseguono nulla. I colori dicono di che informazione si tratta: il verdino è un numero con la virgola, il rosso un vero o falso, il rosa un testo, il blu un altro oggetto, il giallo tre numeri con la virgola, "che si usano solitamente per la posizione, quindi x, y, z, o per la scala, quando ingrandisci qualcosa" (tabella 7.1). Federica: "non avendo il programma sotto, non avevo collegato le variabili a quei colori".

7.22.2 La corrente che fa il giro: il Delay

Per far vedere il flusso, il docente costruisce una piccola catena sul Begin Play. In generale la corrente, in tutti i nodi, "entra ed esce contemporaneamente": entra, esce, entra, esce, **nello stesso *frame***. "Passa tutto uno dopo l'altro, ma assieme." Con un **Delay**, invece, la corrente entra, **aspetta** (per esempio tre secondi) e solo poi esce, facendo avvenire le cose dopo. Begin Play, Delay di tre secondi, un'azione, un altro Delay di tre secondi, l'azione inversa: "così vedi proprio la corrente che fa il giro", avanti e indietro.

E senza il Delay? "Parte subito." Il docente lo mostra sul personaggio: un teletrasporto altissimo (un milione di unità sulla z) sul Begin Play. Senza Delay, appena si preme Play il personaggio è già lassù. Con un Delay di 5 secondi in mezzo, il gioco parte normalmente e dopo cinque secondi "finisco lassù". Federica pensava che il Delay fosse "una meccanica di default che si doveva sempre fare": "No, no. Anzi, la maggior parte delle cose non hanno Delay dentro." Serviva solo a mostrare come si controlla il flusso del filo bianco, "perché alla fine è sempre quello".

7.22.3 L'interfaccia, ridotta all'osso

Le variabili, Federica le conosce già un po', per aver "smanettato" con **RPG Maker**: "non è la stessa cosa", ma le variabili sono comuni a tutti i linguaggi immaginabili, "quindi vai tranquilla, è la stessa roba". Il suo problema è l'interfaccia nuova.

La risposta del docente è uno screenshot dell'editor su cui cancella, pezzo per pezzo, tutto ciò che non serve: "tutta questa parte qua sopra, bruciala via", i pannelli laterali via, le barre via. "Questa è la quantità di interfaccia che ti serve per adesso. È già un pochettino più facile." Nella viewport, per ora, si tratta solo di premere Play e trascinare le cose dentro; le parti laterali si vedranno con il corso di **level design**.

Come Blender. Federica ha un po' di pratica con **Blender**, e il docente ne approfitta: "ti troverai super bene", prima di tutto con i **materiali**, che in Unreal sono schematici come in Blender, con i nodi, come i materiali procedurali. Ma anche il codice dei Blueprint "è tipo Blender con i nodi, se vuoi vederla così: lo stesso concetto". Con un vantaggio: sono blocchetti colorati, non bisogna ricordarsi come si scrivono le cose. "Mi sono dimenticato una virgola?" Non succede: si fa tasto destro e si cerca.

7.22.4 Interrompere è un favore a tutti

Federica si scusa di far perdere tempo: "io so che ci metto un bel po' a capire". La risposta merita di essere riportata per intero nel suo senso: "fidati, è molto più importante il fatto che tu lo impari, perché sennò poi diventa molto più grave dopo". Anche se fra sei mesi non ci si ricorda che cos'è il Begin Play visto oggi, "è normalissimo"; l'unica cosa è fare pratica. "Siamo tutti quanti qua da due ore e 43, non da mesi." E soprattutto: "quello che non hai capito tu è probabilissimo che non l'abbia capito qualcun altro. Quindi stai aiutando qualcun altro, sicuramente."

7.22.5 Il task, riassunto

Infine, "l'ultima cosa, poi torno nel silenzio": qual era il task? Il docente lo riassume (sezione 7.15): **due livelli separati** (File, New Level, Basic; qualcosa in scena; salva come *Mappa2*), uno che **porta all'altro** con un'area o una condizione ("prendo tot monete"), e **almeno una zona** che cambia qualcosa del personaggio: più grande, lanciato in aria, lentissimo, salto più alto, poca frizione come sul ghiaccio. Per aprire l'altro livello, Open Level (by Object Reference) con *Mappa2* selezionata; per far ripartire quello corrente, Open Level sullo stesso livello.

7.23 I progetti, e la chiusura

Negli ultimi dieci minuti il docente chiede chi vuole mostrare il proprio progettino su Discord. Uno studente condivide il suo: un'area che ruota il personaggio in automatico, monete da raccogliere che teletrasportano all'altro livello, e una zona in cui si va avanti diventando **sempre più veloci**, "contentissimo perché sei veloce", finché non si sbatte contro qualcosa e si muore. "Progetto magnifico." Gli altri non si fanno avanti ("che tristezza").

Un oggetto che non deve bloccare. Un'ultima domanda tecnica: come fare in modo che il personaggio passi *attraverso* un oggetto? Bisogna togliergli la collisione: selezionare la mesh, scorrere fino a **Collision** e, al posto di *BlockAll*, scegliere un preset che non blocca (per esempio *OverlapAll* o *NoCollision*).

I saluti. "Spero di non avervi terrorizzato con questa prima lezione." L'importante è non perdersi, "e soprattutto, se vi perdete, non adesso". La lezione registrata si può rivedere velocemente, "tanto potete skippare tre quarti di lezione", i tratti in cui il docente gira tra i banchi. Per dubbi e domande c'è Discord, e se è grave si può chiedere una chiamata. Ci si rivede alla lezione successiva.[4]

Punti chiave del capitolo

- Unreal si programma in **C++** (non nel corso) o con i **Blueprint**, blocchetti colorati da comporre. I Blueprint si reggono su tre concetti: i **nodi** (tutto ciò che si può fare), le **variabili** (le cose che cambiano), gli **eventi** (l'inizio del codice).
- Un Blueprint ha una **viewport** (la parte visiva) e un **Event Graph** (il codice). Un Blueprint Class di tipo **Actor** è un oggetto qualsiasi; trascinato più volte nella scena, ogni copia esegue il proprio codice.
- **Begin Play** scatta quando l'oggetto viene creato, **Event Tick** a ogni *frame*. Il **Print String** stampa a schermo ed è il primo strumento di debug.
- Tipi di variabile e colori: **Boolean** (rosso), **Integer** (verde acqua), **Float** (verde), **String** (rosa), **Vector** (giallo, tre float), riferimenti a oggetti (blu). Il **GET** legge, il **SET** imposta; GET + 1 + SET è il contatore. Compilare dopo aver creato una variabile.
- Sul personaggio: eventi da tastiera, **Launch Character**, **Set Actor Scale 3D**, **Set Actor Location**; scale, location e rotation. Il movimento sta nel **Character Movement** (Jump Z Velocity, Max Walk Speed in cm/s); Pressed e Released fanno uno sprint.
- La **trigger box** è un Actor con una **Box Collision** e l'evento **On Component Begin Overlap**; da **Other Actor** si fa **Cast To ThirdPersonCharacter** per reagire solo al giocatore. Il **Target** di base è *self*: va collegato il riferimento del personaggio. End Overlap per gli effetti limitati a un'area.
- **Spawn Actor** (con la transform divisa), **Destroy Actor**, **Simulate Physics**. Ogni componente ha parametri che si possono leggere e scrivere.

[4] La trascrizione riporta un'incertezza del docente sulla data ("mi sa venerdì prossimo, se non erro… no, lunedì"); non è possibile ricostruirla con sicurezza.

- Le **monete**: Sphere Collision + cilindro, cast, GET/+1/SET della variabile **sul personaggio**, Destroy Actor. Se non funziona, Print String per trovare dove si ferma il flusso e controllare le collisioni del componente giusto.
- Editor in **inglese** (Editor Preferences, Region & Language); **Ctrl + Shift + S** come riflesso; layout recuperabile da Window, Load Layout.
- Altri nodi: **Open Level** (riaprire lo stesso livello lo resetta), Play Sound 2D, Quit Game, confronti + **Branch**, **Flip Flop**, **Sequence**, **Delay**.
- Task in classe: **due livelli** collegati e **almeno una zona** che cambia un parametro del personaggio. I task avranno sempre requisiti minimi e tanto spazio per il resto; dal terzo-quarto task, codice in inglese.
- Tutto ciò che si muove nel tempo sull'Event Tick va **moltiplicato per i delta seconds**, altrimenti dipende dagli fps; $1/\Delta t$ è il frame rate, velocità $\times \Delta t$ dà unità al secondo.
- Le variabili **si azzerano** cambiando livello (la soluzione è la Game Instance). Un sistema di vita parte da un valore iniziale sensato e si controlla con un Branch.
- **Actor** (tutto), **Pawn** (possedibile), **Character** (Pawn con movimento). **Line Trace By Channel**: una linea invisibile che restituisce il primo oggetto colpito, la base di un FPS.
- Regole: uscite (destra) con ingressi (sinistra); stesso colore con stesso colore; il **bianco** è il flusso, la "corrente", e si collega solo con il bianco; gli altri fili sono informazioni. Interrompere con una domanda aiuta tutta la classe.

Chapter 8

Introduzione al level design: lo strumento sei tu, l'engine viene dopo

Il capitolo precedente ha aperto Unreal Engine dal lato del **codice**: eventi, nodi, variabili (capitolo 7). Questo capitolo lo riapre dal lato dello **spazio**. La lezione da cui nasce è infatti la **prima lezione del corso di Level Design** del primo anno, tenuta da un docente nuovo, **Stefano Zocchi**, game e level designer: "benvenuti alla prima lezione, per il primo anno, nel corso di Game Design, specificatamente alla lezione del corso di **Level Design**".

Il corso di level design, nel percorso di Game Design, è uno dei tre filoni che corrono in parallelo (gli studenti stessi parlano delle "tre materie che dobbiamo fare nel corso"), e ha un taglio preciso: **molta pratica in engine**, accompagnata da teoria e metodologia. Questa prima lezione, però, è soprattutto un'apertura: chi è il docente, chi sono gli studenti, che cos'è il level design, che cosa fa davvero un level designer, e qual è l'unico strumento di cui ha veramente bisogno. Solo nell'ultima ora si apre l'editor, per imparare a muoversi, piazzare forme e organizzare una scena; e il primo task è il più semplice e personale possibile: ricostruire con le forme di base **la propria stanza**.

Chi ha seguito il corso breve su Unreal troverà la parte pratica in gran parte nota (capitolo 1.1, in particolare le sezioni 1.1.16 e 1.1.17); qui la si ripercorre con l'occhio del level designer, che passa "quasi tutto il tempo in engine a piazzare cose, sistemare cose, testare cose". Il blockout vero e proprio, con le luci, verrà poi ripreso in modo approfondito nel corso su Unreal, nel capitolo dedicato alla *gym*, al blockout e alle luci.

La struttura della lezione, e quindi del capitolo, è questa:

1. gli **annunci pratici** e la presentazione del docente (sezioni 8.1 e 8.2);
2. il **giro di presentazioni**: sei domande a ciascuno, tra cui il livello peggiore che si sia mai giocato e come lo si cambierebbe (sezione 8.3);
3. perché presentarsi: la prima lezione seria è **saper parlare in pubblico** (sezione 8.4);
4. **che cos'è** il level design e **che cosa fa** un level designer (sezioni 8.5 e 8.6);
5. lo **strumento** del level designer, la parola magica "dipende" e l'imparare a imparare (sezione 8.7);
6. come si diventa level designer: teoria, pratica, expertise, esperienza, gruppo (sezione 8.8);
7. due curve da conoscere in anticipo: il **picco** della fiducia e l'**effetto Dunning-Kruger** (sezione 8.9);
8. i **tre corollari**: niente regole fisse, il design è olistico, il design è artigianato (sezione 8.10);
9. la **struttura del corso** e la pausa (sezione 8.11);
10. **Unreal Engine**: il progetto, l'editor, gli actor e gli asset, il transform, le scorciatoie, i livelli (sezioni 8.12–8.18);

11. il **task**: il blockout della propria stanza, e il materiale extra per chi vuole approfondire (sezioni 8.19 e 8.20).

8.1 Prima di cominciare: gli annunci pratici

8.1.1 Cinque minuti di margine

Il docente apre con una "piccola informazione di servizio": lascia **sempre cinque minuti** prima di iniziare, per dare a tutti il tempo di arrivare. Capita spesso, spiega, che ci sia "gente che viene da fuori, o che c'è gente che lavora"; e se ci sono autobus in ritardo o "casini vari", è giusto che chi arriva possa sedersi con calma. Solo dopo si fanno presentazioni, introduzioni e tutto il resto.

8.1.2 In aula o da remoto

Le lezioni, "come sapete perfettamente", si possono seguire **contemporaneamente** in aula e online. Il docente non nasconde la sua preferenza: in classe è "l'ambiente migliore", perché è più veloce parlare con le persone, far vedere le cose, **andare ai vostri computer** e verificare se c'è qualcosa da sistemare. Ma se serve seguire da remoto, "per me non c'è nessun problema". L'unica richiesta: se c'è un motivo particolare, avvisarlo prima se non si è in grado di venire. Per il resto, "la praticità conta più di tutto".

8.1.3 Pausa e appello

Il ritmo della lezione è sempre lo stesso:

- qualche minuto di margine all'inizio;
- a **metà lezione** una **pausa** di 15, 20 o 30 minuti, "a seconda di quanto è stato pesante l'argomento" trattato fino a quel momento;
- **dopo la pausa**, l'**appello**. Non lo fa subito proprio per segnare come presente chi arriva in ritardo. Chi arriva dopo l'appello deve farglielo sapere, così lo segna lo stesso; e se il docente si dimentica di farlo, va ricordato: "mi è capitato più di una volta. Siamo umani".

Un gruppo nuovo. Il primo anno di quest'anno è "più numeroso di quello dell'anno scorso, fantastico"; e, per la prima volta nella sede di Milano, ci sono anche delle ragazze a lezione, "un'ottima cosa".

8.2 Chi insegna

Regola numero uno. "Regola numero uno dell'apprendimento: **sapere sempre chi vi sta insegnando qualcosa**". Mai fidarsi della gente a caso su internet, "soprattutto su YouTube" (un tema che il docente promette di riprendere durante l'anno).

La carriera. Stefano Zocchi è un **game e level designer**. Il suo percorso, in breve:

- è partito dal mondo **indie**, partecipando a vari progetti, facendo "un po' di roba horror per i fatti miei" e lavorando con altre persone;
- è stato poi assunto da uno studio gallese, **Antimatter Games**, dove ha lavorato a **sparatutto stealth strategici**. Lo studio è stato chiuso durante l'ondata di chiusure che ha colpito l'industria dei videogiochi negli ultimi anni;
- ha lavorato per un periodo per **Behaviour Interactive**, nome noto soprattutto per *Dead by Daylight* (il gioco horror multiplayer in cui un giocatore interpreta il killer e gli altri quattro i sopravvissuti; il titolo glielo suggerisce la classe). Behaviour, però, è "un ombrello di un sacco di studi diversi", e lui non lavorava a quel gioco. A che cosa lavorasse, non può dirlo;
- oggi lavora come **freelance**: insegna, collabora con un paio di ex studenti e segue vari progetti.

L'NDA. Il motivo per cui non può dire a che cosa ha lavorato è un **NDA**, un *non-disclosure agreement*: una parte del contratto che si firma e che stabilisce che non si può parlare di ciò che lo studio non autorizza esplicitamente. C'è una "grossa parte" del suo curriculum per cui può dire "ho lavorato in questo studio", ma alla domanda "bello, cosa hai fatto?" la risposta è sempre la stessa: "vorrei continuare a lavorare nell'industria dei videogiochi". È una realtà del mestiere con cui chiunque, nel corso della carriera, si troverà a fare i conti.

Il progetto attuale. Il progetto più grosso a cui sta lavorando si chiama ***Break the Night***: un gioco d'azione corpo a corpo a tema un po' *Y2K*, un po' *pop punk*, "energico", con "un sacco di combo", attacchi pesanti, attacchi leggeri, mosse speciali, cancellazioni con gli scatti.[1] "Molto, molto divertente."

Come è arrivato a insegnare. Qualche anno fa un insegnante non poteva più seguire il corso di Milano; l'altro docente che si occupa di questi corsi gli ha chiesto se poteva coprirlo lui. Essendo freelance, si inseriva "abbastanza bene" nei lavori che stava facendo: bastava riorganizzare un po' gli orari. Da qualche anno, ormai, insegna lui il corso di level design.

Alessandro. In aula c'è anche **Alessandro**, che non fa parte del primo anno, ma che gli studenti vedranno "un po' dappertutto in questo edificio, come un fantasma".[2]

8.3 Il giro di presentazioni

"Basta con queste frasi noiose": è un anno nuovo, il docente non conosce ancora la classe, e vuole sentire tutti, in aula e online. Le domande vanno "dal semplice al più interessante", per capire che tipo di persona è ciascuno.

8.3.1 Le sei domande

1. **Chi siete?**
2. **Che tipo di giochi volete fare?**
3. **Qual è la vostra forza principale?**
4. **Qual è la vostra debolezza principale?**
5. **In che cosa vi vorreste specializzare**, se già avete un'idea?
6. E, visto il tema del corso: **un livello che vi ha proprio fatto schifo**, che vi ha fatto mollare il gioco, "che ancora ce l'avete qua" e vi chiedete come sia possibile che abbiano fatto "una roba così orrenda". **Perché** non vi è piaciuto, e **che cosa fareste per migliorarlo**?

L'ultima domanda è quella che conta per il corso: non basta dire che un livello è brutto, bisogna saper dire *perché* e proporre un *rimedio*. È, in piccolo, il lavoro di analisi che il level designer fa ogni giorno.

8.3.2 L'esempio del docente: la Cattedrale del Sangue Sacro

Siccome la prima persona a rispondere "si trova un attimo sulle spine", il primo a rispondere all'ultima domanda è il docente stesso. Ogni anno rifà le slide; questa, per la prima volta, non l'ha rifatta, perché il livello in questione ce l'ha in testa da quando ci ha giocato due anni fa e "continua a tormentarmi i sogni": gli appare ogni volta che sta facendo un livello, dicendogli "vuoi fare questa roba? Non ti libererai mai di me".

[1] La trascrizione definisce il genere con una parola storpiata ("una cashlash"); dalla descrizione (combo, attacchi leggeri e pesanti, mosse speciali, *cancel* con lo scatto) si tratta di un *action* corpo a corpo, verosimilmente un *hack and slash* o un *beat 'em up*, ma il termine esatto usato dal docente non è ricostruibile.

[2] La trascrizione aggiunge un dettaglio incomprensibile ("che va zika per via macchina"); non lo si ricostruisce.

Il livello è la **Cattedrale del Sangue Sacro** di ***Code Vein***, uno dei "tantissimi giochi" usciti sulla scia dei titoli di FromSoftware, un *souls-like* "tra virgolette".[3] Per il docente è uno degli esempi che toccano "il punto più fondo di tutto questo abisso di mediocrità".

La slide mostra la mappa del livello, realizzata da un utente di DeviantArt per chi non riusciva a capire dove andare.[4] Le linee grigie non sono strade: sono **passerelle**. La domanda alla classe è: guardando questa mappa, in un'ottica negativa, che cosa salta all'occhio? La risposta arriva subito: **è un labirinto**. Il docente elenca i problemi:

Un labirinto di corridoi. È "una lunghissima serie di corridoi", con bivi di cui non si capisce la direzione giusta; corridoi che "vanno in giro totalmente a caso".

Su più livelli, non collegati. I corridoi non sono tutti collegati fra loro: stanno su piani diversi. Se ci si trova in un punto e si vuole raggiungerne un altro, magari bisogna tornare indietro, scendere, prendere un altro corridoio e girare a destra; "e io non lo so".

Un "pastone" visivo. Tutti i corridoi sono circondati da enormi decorazioni gotiche, "un po' Duomo di Milano, un po' cattedrali inglesi": archi, pilastri, "rampart", decorazioni a caso. Rendono impossibile capire anche solo dove si trova il corridoio successivo.

Nessun punto di riferimento. Non si capisce dove sono i punti di riferimento, non si capisce dove andare anche quando si intuisce più o meno la direzione.

Ripetitivo e geometrico. È "ripetitivo da far paura" e, come mostra la struttura della mappa, totalmente geometrico: dall'inizio alla fine è sempre lo stesso schema, corridoio e torre, corridoio e torre. **Non c'è interesse nell'esplorazione**.

E i boss sono terribili. Risultato: "ho mollato il gioco" in quel punto.

È, in negativo, una lista di quasi tutti i temi del corso: navigazione, leggibilità, punti di riferimento (*landmark*), varietà, interesse dell'esplorazione.

8.3.3 Valerio: Subnautica

Il primo studente è **Valerio**. Non sa rispondere a due domande: che tipo di giochi vuole fare e in che cosa vorrebbe specializzarsi. "Ci sta. È abbastanza comune questa risposta al primo anno": si sta iniziando adesso, e probabilmente l'unica cosa che accomuna tutti i presenti è che gli piacciono i videogiochi ("c'è qualcuno a cui non piacciono i videogiochi?").

Più che un livello, Valerio cita un gioco intero che gli ha fatto smettere di giocare: ***Subnautica***, perché "non aveva troppa libertà" e non sapeva come muoversi; e, in generale, non ama i giochi di sopravvivenza e di esplorazione. Gliel'aveva consigliato un amico appassionato. Come debolezza, dice di fare fatica a impegnarsi in una cosa se non lo prende particolarmente; come forza, non sa. "Molto onesto, molto diretto. Apprezzo questo approccio." Alla domanda su che cosa cambierebbe di *Subnautica*, la risposta è la più breve della giornata: "cancellerei il gioco". Il docente apprezza il tempismo comico.

I nomi. Il docente avverte che ripeterà spesso i nomi di tutti, perché fa "una fatica enorme" a ricordarli, e si scusa in anticipo. E aggiunge che, come tutte le esperienze educative, anche questa serve "anche per scoprire chi siamo".

8.3.4 Alberto: Nightreign, e la differenza tra scelta ed errore

Alberto vorrebbe fare "un po' di tutto": non ha un genere preferito, dai giochi di calcio ai *gacha* giapponesi.[5] La sua forza è l'**attenzione ai dettagli**: nota subito se in una scena c'è qualcosa leggermente spostato, "un

[3] La trascrizione riporta prima "sulla scia di Vlaovic", poi si corregge in "sulla scia di From Software"; il primo nome non è ricostruibile.

[4] Il nome dell'utente ringraziato dal docente è storpiato nella trascrizione ("Cercheron Quartoro") e non è ricostruibile.

[5] La trascrizione è qui in loop ("un po' di tutto" ripetuto decine di volte) e contiene uno scambio scherzoso su un simulatore di autobus nella Berlino degli anni Quaranta, che non è chiaro a chi vada attribuito.

pallino". È un punto di forza, "ma anche no. Dipende dalla situazione". La sua debolezza è il **multitasking**: non riesce a concentrarsi su più cose nello stesso momento.

Il multitasking non esiste. Il docente lo rassicura: non è necessariamente una debolezza, perché, a livello di capacità biologica di elaborare le informazioni, **il multitasking non esiste**. Quello che esiste è spostare completamente l'attenzione da una cosa all'altra a velocità straordinaria. Se ne riparlerà più avanti nel corso (il docente accenna a "sistema 1 e sistema 2").

Level o game? Alberto è indeciso se specializzarsi in level design o in game design, "senza aver mai fatto una lezione di level design". "Ti farò cambiare idea", promette il docente.

Il livello: la mappa di Nightreign. Il livello che non gli è piaciuto viene da ***Elden Ring Nightreign***: una mappa (di un contenuto aggiuntivo) con un castello da una parte e un altro dall'altra, in cui bisogna risolvere delle cose su entrambi i lati per poi andare in un altro castello ancora, al centro, e battere vari boss in successione. Ma non c'è il tempo: come in un *battle royale*, **la zona si stringe**, e non si riescono a fare entrambi i castelli prima di arrivare a quello centrale.

Scelta creativa o errore? La domanda del docente è provocatoria: "secondo te, a FromSoftware sono così stupidi da non essersi resi conto che un giocatore non può fare entrambi i castelli?" No: se ne sono resi conto, ma la scelta è voluta, perché il gioco è pensato per essere giocato in gruppo, mandando qualcuno da una parte e qualcuno dall'altra. Qui, dice il docente, "iniziamo ad affrontare un discorso molto delicato e complesso": **quando una cosa che non ci piace è una scelta creativa, e quando è un errore?** In questo caso diventa sempre più chiaro che impedire al giocatore di fare entrambe le parti della mappa (o costringerlo a dividersi e riunirsi al centro) è fatto apposta.

La proposta, e le sue conseguenze. Alberto cambierebbe la cosa lasciando la libertà di andare in quella zona anche quando si chiude: rendere la parte fuori zona **non dannosa** per un certo tempo, così da poterla completare anche dopo. Il docente fa notare che la modifica "ha tutta una serie di conseguenze": prima di tutto è **antitetica all'idea di battle royale**, in cui il campo di gioco si restringe proprio per costringere i giocatori a scontrarsi. Alberto ribatte che lì lo scopo è esplorare e potenziarsi il più possibile per battere il boss finale.

L'intervento di Mattia. Interviene **Mattia**: più che un vero battle royale (in cui, come dice il nome, alla fine ne resta uno solo), *Nightreign* è una **boss rush a tempo**, in cui bisogna arrivare a un certo punto della mappa a seconda di come si restringe. Secondo lui il fattore di level design è proprio questo: a meno di dividersi, o se si gioca da soli, non si riesce a fare tutto, e questo serve a **impedire al giocatore singolo di diventare troppo forte**. Del resto, aggiunge, non è un gioco "troppo divertente" da soli. Il docente chiude la parentesi: la classe è numerosa e c'è meno tempo per "le mie teatralità".

8.3.5 Mattia: Monster Hunter, il sound design e la Città Infame

Mattia Maestri non sa ancora che tipo di gioco vorrebbe creare, ma ha un'idea dei giochi a cui gli piacerebbe lavorare: i **souls-like**, i suoi generi preferiti, e, sogno "nel cassetto", entrare in **Capcom** per lavorare a ***Monster Hunter***, la prima saga che abbia mai giocato e la sua preferita ("gli ho giocati praticamente quasi tutti"). Il docente nota la "linea di discendenza" interessante: i giochi di FromSoftware prendono molto da *Monster Hunter*.

Forza e debolezza. La sua forza è la **creatività**: le idee gli vengono facilmente ("che poi siano buone idee è un altro punto di vista"). La debolezza: farà fatica nel level design, soprattutto nella parte **tecnica**; si definisce "più una persona teorica che una persona pratica".

Il sound design. Oltre al game design, vorrebbe specializzarsi in **sound design**: ama la musica, "vivo di musica", e alcuni giochi li ha iniziati solo per la colonna sonora. *Kingdom Hearts*, per esempio, lo ha cominciato perché un amico gli aveva fatto sentire la musica del menu iniziale.[6] All'inizio dell'anno aveva già chiesto se il sound design sarebbe stato trattato, e gli era stato detto di approfondirlo fuori.

Il docente lo conferma: ci saranno "delle piccole cose" di sound design nel corso, "ma sono veramente poche". Il consiglio migliore è un altro: nei **progetti di gruppo** dei tre anni, quando il team si chiede "che cosa facciamo con quei suoni?", bisogna essere quelli che dicono "**ragazzi, ci penso io**". È così che ci si costruisce una specializzazione dentro un percorso generalista.

Il livello: la Città Infame di Dark Souls. Mattia ha platinato il primo *Dark Souls*, ma un livello gli è rimasto sullo stomaco: la **Città Infame** (*Blighttown*). È stata la parte che gli ha richiesto più tempo, per tre motivi:

- la **scarsa visibilità**, in una città tutta di legno e "tutta uguale";
- i **checkpoint**: solo due falò in tutta l'area, uno all'inizio e uno quando si è "già praticamente fuori", invece di tre o quattro intermedi;
- il **veleno**: quasi tutti i nemici lo infliggono, e si ha un tempo limitato per raggiungere il falò successivo, che però non si vede perché è lontanissimo.

Che cosa cambierebbe? Più visibilità, per capire meglio l'ambiente, oppure un **falò intermedio**.

Fare souls-like. Sui souls-like il docente è franco: da quando sono usciti i giochi di FromSoftware, "una quantità gigantesca" di studenti arriva dicendo "io vorrei fare i souls-like", e la risposta è sempre la stessa: **pochissima gente riesce a farli**. Mattia precisa che intende più in generale il genere **action RPG**. Non è impossibile: alcuni studenti hanno portato come progetto finale un piccolo souls-like "venuto sorprendentemente bene"[7]; ovviamente non si può fare un gioco da sessanta ore nel contesto di uno studio, ma il risultato era buono. E in Capcom qualche italiano c'è, anche se quelli che il docente vede lavorare all'estero sono più nel dipartimento artistico che in quello di design.

8.3.6 Francesco: i JRPG, Fabula Ultima e il salto finale

Francesco Muratore ama i **JRPG**: più *Persona* che i vecchi *Final Fantasy*, anche se ha giocato di recente al settimo capitolo; adesso sta giocando una saga "non famosissima", quella di *Trails*.[8] Il docente conosce la sensazione: a un certo punto ci si rende conto che una serie "ha quindici giochi", tutti collegati. "Giocare a certi JRPG è come giocare una telenovela lunga vent'anni."

Vorrebbe fare JRPG perché sono giochi con una storia che "sa riempire": ogni volta che ne finisce uno resta appagato per un mese, un mese e mezzo, prima di iniziarne un altro. Il docente traduce: "un livello di passione equivalente a quando ti rompi un ginocchio e non puoi andare a correre per un mese e mezzo".

Forza e debolezza. La forza: "mi rialzo facilmente dai miei problemi e li affronto", a testa alta. La debolezza: si distrae spesso, ma ci sta lavorando. Il docente osserva che non è solo una questione di disciplina: può essere una questione di **neurodivergenza**, di disturbi dell'attenzione ("forse anche io"), e non c'è "assolutamente niente di male": "siamo tutti a gestirci il corpo che abbiamo, e il cervello è un organo".

[6] Lo scambio sul brano è confuso: la classe cita un verso ("If you walk away…"), che appartiene a una canzone di *Kingdom Hearts II*, e il docente lo identifica con un titolo che la trascrizione riporta come "Everly Hearts", verosimilmente *Dearly Beloved*, il tema del menu della serie. Non è possibile stabilire con certezza a quale dei due brani si riferisse Mattia.

[7] Il titolo del progetto è storpiato nella trascrizione ("Steam qualcosa", "Steam Rush") e non è ricostruibile con certezza.

[8] La trascrizione riporta "trains in the sky"; dal seguito del discorso (una serie con moltissimi capitoli collegati tra loro) si tratta quasi certamente della serie *The Legend of Heroes: Trails*, di cui *Trails in the Sky* è il primo capitolo.

Fare JRPG in Italia. La reazione istintiva sarebbe "buona fortuna, fare JRPG in Italia". Eppure c'è ***Fabula Ultima***: un gioco di ruolo carta e penna italiano nato da chi era stanco del fantasy classico europeo e americano di *D&D* e ha fatto un gioco da tavolo basato sui JRPG, "molto Final Fantasy", come si capisce dal nome. Ha avuto un successo "stratosferico": secondo il docente è probabilmente il gioco di ruolo carta e penna italiano di maggior successo della storia.

Un paese di nippofili. Del resto, "viviamo in un paese di giapponofili": l'Italia è stata uno dei primi paesi al mondo a importare anime, già dagli anni Ottanta, ben prima degli Stati Uniti. Con la liberalizzazione dei canali, la televisione privata italiana aveva bisogno di riempire il palinsesto, e i giapponesi vendevano serie da doppiare a prezzi bassissimi: *Heidi*, *Sampei*, e poi *I Cavalieri dello Zodiaco*, *Dragon Ball*.[9] Non è impossibile, quindi; è difficile, perché il mercato occidentale è "un po' recalcitrante", ma "viviamo in un mondo globalizzato". Il consiglio: **studiare i sistemi**, perché i JRPG sono "tutti sistemi". Il loro level design, sinceramente, "tende a non essere granché": sono molto più concentrati su narrativa e sistemistica.

Il livello: un salto al 99,6%. Francesco cita un livello battuto da poco in cui, al **99,6%** del percorso, c'è un **salto finale precisissimo**: ci è morto almeno sette volte, dopo cinque minuti di livello tutti di fila.[10] È la classica **botta di difficoltà** finale che costringe a rifare tutto da capo. Taglierebbe quel pezzo? Il docente aggiunge una considerazione: forse proprio quella è la sfida vera, la parte divertente. Anche questo va considerato.

L'inferno voluto. Lo stesso vale per la Città Infame di Mattia: è **pensata per essere l'inferno**, il punto più basso di tutto il mondo di gioco. Una volta finita, la soddisfazione è "non ci devo più tornare, finalmente ce l'ho fatta". È "leggendaria", uno dei livelli più difficili e più "tossici" ("anche letteralmente") di tutti gli action RPG. E c'è un motivo se, scegliendo all'inizio la classe del ladro, si ha la **Chiave Maestra**: permette di saltare l'intero livello, ed è ciò che molti fanno alla seconda partita.

8.3.7 Federica: i giochi intimi e la mappa di Pentiment

Federica parte dalla debolezza, perché si collega al resto: una grandissima **sensibilità**, "in positivo, ma soprattutto in negativo", che la porta a rimuginare molto. Ed è anche ciò che vorrebbe fare: un **piccolo gioco intimo**, come *To the Moon*, *Undertale*, *Omori*. Il docente le consiglia ***OFF***: se le è piaciuto *Undertale*, è uno dei giochi a cui Toby Fox si è palesemente ispirato.

La sua forza è legata alle sue neurodivergenze, che a volte la aiutano a **guardare le cose fuori dagli schemi** e la rendono molto creativa. Non ha un'idea precisa di specializzazione: il suo desiderio è sempre stato lavorare per conto suo, fare il suo piccolo gioco, anche senza fare molti soldi, perché un lavoro ce l'ha già.

Il livello: la mappa di Pentiment. Cita ***Pentiment***, che le stava piacendo ma ha lasciato. Il problema è la **mappa**, e l'essere in **2D**: capisce la scelta di riprendere le miniature e il disegno dell'epoca, ma quando si apre la mappa e ci si vuole spostare da un punto all'altro si può andare "a destra o a sinistra, a grandi linee". Se si deve raggiungere un punto molto lontano, lo spostamento è dispendioso; e la mappa è poco chiara su quali NPC si trovino dove. La soluzione: un **viaggio rapido**, o **più indicatori** sulla mappa. Più che un livello specifico, nota il docente, è un problema di approccio alla comunicazione. Come livello vero e proprio, Federica cita anche il **Picco di Cristallo** di *Hollow Knight*, che però mette alla prova soprattutto la sua difficoltà con i *platform*.

"Pochi masochisti hardcore gamer quest'anno", commenta il docente (l'anno prima c'era un appassionato di *bullet hell*): Federica è una delle persone "con gli interessi più indie" che abbia avuto finora.

[9] La trascrizione cita anche un titolo storpiato ("l'Upen"), forse *Lupin III*; non è ricostruibile con certezza.

[10] Prima di questo esempio Francesco nomina un altro titolo, irriconoscibile nella trascrizione ("su Giove ti lascia"); anche il gioco del salto finale non è nominato in modo comprensibile.

8.3.8 Daniele: l'esplorazione e gli spazi vuoti

Si torna in aula, in senso antiorario, "per tenere la gente sulle spine". **Daniele** vorrebbe fare giochi con un po' di **esplorazione**, non necessariamente un *open world*: anche qualcosa di più piccolo, come esplorare una città.

Forza e debolezza. La forza: è **preciso**, attento ai dettagli "senza fare le maniacate". Il docente aggiunge una nota di produzione: **c'è sempre una quantità di tempo limitata**, e l'attenzione ai dettagli può diventare un problema quando si parla di tempistiche. "Non si può sempre fare tutto perfetto." La debolezza: non si sente tagliato per fare il **leader** di un gruppo, preferisce non assumersi responsabilità. Il docente: è importante saperlo. "È bello dire: ah, io sono il capo"; ma "vi assicuro di persona che essere il capo di un progetto non è divertente per niente", e non è sempre una buona idea essere quello che decide tutto e si occupa di tutto.

Il livello: una montagna vuota. Daniele descrive una zona di montagna, in un gioco che gli è piaciuto, che però è un **grande vuoto**: c'è poca roba, ci si arriva, ci si trova una grotta e poco altro.[11] Metterebbe **punti di interesse**: come in *Breath of the Wild*, dove su una montagna c'è magari un ricordo da recuperare e su un'altra un minigioco come la discesa sullo scudo. In breve, "riempirlo un po' di più". Il docente conferma: gli spazi vuoti sono "un grosso problema degli ultimi giochi di Pokémon". Daniele fa un confronto interno: un'altra parte della mappa, il **deserto**, è invece costruita **su più livelli**, ed è quindi uno spazio interessante; la montagna è "un grande vuoto piano".

8.3.9 Francesca: i giochi psicologici e un labirinto di piani

Francesca gioca "un po' a tutto", ma se deve scegliere preferisce i giochi con un **valore psicologico** importante: molto narrativi, con una psicologia di fondo da capire, non superficiale; horror indie, horror psicologici. La sua forza è essere **estremamente inclusiva**: non giudica gli altri e cerca di mettere tutti a proprio agio. La debolezza è l'**introversione**, su cui sta lavorando. Come Daniele, non si sente tagliata per fare la leader, pur sapendo comunicare e fare da mediatrice. Le interessano sia la parte visiva e creativa sia quella tecnica e di programmazione: per questo ha scelto il game design, "perché più o meno tocca tutti i punti".

Il livello: Slime Rancher. Il livello che non le è piaciuto per niente è un'area del primo ***Slime Rancher***.[12] Non capisce il livello, e le ha fatto rigiocare la zona "tantissime volte": la mappa non è affatto chiara e, come nell'esempio del docente, è tutto basato su **più piani intrecciati**, senza veri collegamenti. Per passare da un punto all'altro o si trova l'unica strada giusta "tra i mille" passaggi possibili, o ci si mette troppo tempo. Ed è paradossale per un gioco di *farming* ed esplorazione: va contro ciò che il gioco stesso dovrebbe essere.

8.3.10 Antonino: i roguelike e la tempesta di Frostpunk 2

L'ultimo è **Antonino Etiopia** ("c'è sempre una persona che deve essere ultima"). Gli andrebbe bene qualsiasi gioco, ma sceglierebbe i **roguelike**, per le sinergie e le scelte; e, data la loro popolarità nei giochi indie, avrebbe "molte occasioni di provarci". La sua forza è lavorare in gruppo: rende meglio perché, se lavorasse male, la cosa ricadrebbe sugli altri. "Il senso di colpa è un eccellente motivatore." La debolezza, che va un po' contro la forza, è l'introversione. Specializzazione: deve ancora ambientarsi nel mondo del game dev.

[11] Il passaggio è quasi del tutto incomprensibile nella trascrizione ("la parte della serena, papa, che è la montagna") e il gioco non è nominato in modo riconoscibile. Il docente collega l'esempio agli "ultimi giochi di Pokémon", e Daniele lo confronta con un'altra zona della stessa mappa, un deserto: è quindi probabile, ma non certo, che si tratti di un capitolo recente di *Pokémon*.

[12] La trascrizione riporta "Dreamland e The Grey Lightning"; nel primo *Slime Rancher* esiste un'area chiamata *The Grey Labyrinth*, che corrisponde alla descrizione, ma il nome esatto pronunciato da Francesca non è verificabile.

Il livello: la città nella tempesta. Il livello che gli ha fatto abbandonare il gioco viene da ***Frostpunk 2***. Il docente ricorda che *Frostpunk* è una serie di **gestionali strategici** in cui si governa una città in un'era glaciale, con risorse limitatissime e una capacità di riscaldare gli ambienti molto ridotta: bisogna fare scelte difficili su chi salvare, che cosa sacrificare, che cosa bruciare. Dopo che la città sopravvive alla prima tempesta, bisogna andare a esplorare un'altra città per recuperare risorse. Si hanno un certo numero di anni per prepararsi, ma **non si ha idea di che cosa ci sia**, perché sopra la città c'è una tempesta. Si arriva impreparati, e si perde tutto il lavoro.

L'analisi. Che non si sappia che cosa si andrà ad affrontare è "chiaramente intenzionale". Ma qual è l'effetto? Ci si adatta, si resiste, ci si prepara a perdere tutto e ad andare avanti comunque? O si ottiene l'effetto "perdo, ricomincio il gioco, e adesso che so cosa aspettarmi lo faccio meglio"? Antonino aveva pensato la stessa cosa, ma il suo salvataggio si era anche corrotto due volte, e ricominciare era diventato pesante ("e lì c'è poco da fare: quando qualcosa si rompe, è difficile dire se è fatto apposta").

La proposta: un indizio. Antonino darebbe, dopo la tempesta, almeno **una dritta** su che cosa preparare. Il docente suggerisce di inserirla nella **narrativa**: un sopravvissuto scappato da quella città che racconta che cosa c'è ("ci sono dei dinosauri allergici allo zafferano: manda la gente con i coltelli allo zafferano"; "sparo cavolate, però è possibile"). Qualcosa che permetta di **fare un piano** invece di buttarsi nel vuoto totale. *Frostpunk* è una serie hardcore, e mettere bastoni fra le ruote è lo scopo del gioco; ma qui sembra "un po' troppo hardcore", il tipo di cosa che "devi saperlo prima: ci muori una volta e riprovi". Arrivare senza le strutture giuste significa sacrificare un sacco di persone, e in un gioco così sbagliare una scelta tanto importante può condannare l'intera partita: per quanto ricorda il docente, non ci sono **meccaniche di recupero**. Se le cose vanno troppo male si entra in una **spirale** in cui si continuano a perdere risorse senza che la partita finisca.

8.4 Perché presentarsi: parlare in pubblico

"Congratulazioni, avete passato la prima esercitazione del corso." Il giro di presentazioni aveva due scopi. Il primo, dichiarato con ironia: il docente fa fatica a ricordarsi i nomi e si dà una scusa per ripeterli ("Valerio"). Il secondo è serio: **abituarsi a parlare in pubblico**.

Se si vuole fare qualunque cosa nei videogiochi, arriverà il momento in cui bisognerà **presentare quello che si sta facendo**, e presentarlo a più persone. Ci saranno molte occasioni per allenarsi, ma questa deve essere la prima cosa che resta impressa: bisogna imparare a presentare sé stessi e il proprio lavoro agli altri. Vale in quasi ogni settore lavorativo, "ma soprattutto nei settori creativi".

La prima lezione seria. "Prima lezione seria di questo corso":

> **un designer mediocre, ma in grado di fare bei discorsi e di convincere la gente, avrà sempre più successo nel proporre le proprie idee di un designer geniale che non riesce a parlare in pubblico.**

"Mi dispiace, la vita è così." Bisogna trovare un modo per superare le proprie difficoltà nel parlare in pubblico: è "assolutamente necessario". La buona notizia è che, a giudicare dal giro appena fatto, più o meno tutti hanno una base di partenza; si tratta di migliorarla.

8.5 Che cos'è il level design

La definizione. "Terra-terra": il **level design** è la disciplina che si occupa della **creazione di livelli, ambienti, locali**, di qualunque tipo di spazio all'interno di un videogioco.

Il level editor. Di solito lo si fa con un **level editor**. "Grazie a Dio non sono più gli anni Ottanta": oggi non si devono più fare i livelli su carta a quadretti, prendendo le coordinate, convertendole e inserendo tutto a mano. Per *Super Mario Bros.*, per esempio, non c'era un editor di livelli[13]: c'era il codice. Ci si segnava le cose, si scriveva il codice, e se un blocco finiva nel punto sbagliato bisognava tornare nel codice e modificare il punto in cui il blocco era scritto.

Gli editor nei giochi. Alcuni giochi includono un editor di livelli che permette di crearne di nuovi all'interno del gioco stesso: sono **ottimi per allenarsi** a fare level design con un set di meccaniche già pronto.

Un lato artistico e un lato tecnico. La cosa più importante da ricordare è che il level design **ha un lato artistico e un lato tecnico**. Lo si può trovare anche sotto altri nomi, come **environment design** o **game mapping**: nel settore "un sacco di roba ha nomi diversi".

Il blockout. Gli screenshot della slide mostrano l'aspetto di un videogioco **mentre viene sviluppato**: sono fasi di **blockout**, cioè fasi in cui non è ancora stata inserita l'arte finale e si usano forme geometriche di base per cominciare a strutturare il livello. Sarà uno degli argomenti del percorso.

Approfondimento: "applied game design". Christopher Totten, nell'introduzione di *An Architectural Approach to Level Design* (il libro che il docente consiglierà a fine lezione, sezione 8.20), raccoglie alcune definizioni classiche. Per Rudolf Kremers lo scopo del level design è "interpretare le regole del gioco e tradurle in un costrutto (un livello) che faciliti al meglio il gioco": il level design è *game design applicato*. Jay Wilbur e John Romero dicono che il level design è "dove la gomma tocca l'asfalto", e che il level designer è "in larga parte responsabile dell'implementazione del gameplay". Totten propone una definizione più spaziale: il level design è "l'esecuzione ragionata del gameplay nello spazio di gioco, perché i giocatori lo abitino". Ricorda anche che molti designer hanno cominciato proprio come suggerisce il docente, creando livelli personalizzati (*mod*) con gli strumenti di giochi esistenti.

8.6 Che cosa fa un level designer

8.6.1 La risposta breve

"Un level designer si occupa di **creare spazi**." Spazi virtuali: in 2D, in 3D, "in 1D, in 4D: esistono. Delirio totale, ma esistono".

8.6.2 La risposta lunga: un termine ombrello

La risposta lunga è che "level design" è un **termine ombrello**: una disciplina gigantesca che coinvolge diverse specializzazioni. A seconda del contesto, chi lavora al level design può occuparsi di cose molto diverse fra loro (figura 8.1).

Il blockout degli spazi. "Questo cubo è la casa, quest'altro cubo è il grattacielo, questo parallelepipedo più allungato è il ponte": la prima struttura del livello, fatta di forme di base.

Le luci. Illuminare non vuol dire "prendo il sole, metto il sole, problema risolto": le cose vanno illuminate **con cura**. Il level design si sovrappone molto alla **fotografia** e al **cinema**: nel cinema, le luci che illuminano una scena non sono mai quelle che si vedono. Ci sono sempre lampade fuori dall'inquadratura che illuminano con attenzione un personaggio o una zona, c'è la luce riflessa e soffusa che fa stagliare meglio i personaggi contro le ombre. Anche questo è un passaggio del level design: il **lighting pass**.

L'arredamento degli scenari. Occuparsi di come **arredare** gli spazi, il cosiddetto *set dressing*.[14]

[13] La trascrizione riporta l'anno 1984; *Super Mario Bros.* è uscito nel 1985.

[14] Il termine pronunciato dal docente è storpiato nella trascrizione ("la business plan"); dal contesto ("occuparsi di come arredare gli scenari") si tratta con ogni probabilità del *set dressing*, o di un *dressing pass*.

La documentazione. Una cosa che fanno tutti i designer, e i level designer sono designer, è **scrivere documentazione**: da passare ad altri o da usare per tenere d'occhio il proprio lavoro. Layout, mappe, appunti, **diagrammi di flusso** che permettono di visualizzare la comunicazione tra le zone, la progressione nel livello, le azioni del giocatore. Un lavoro "quasi burocratico" (si veda il capitolo 6 sui flowchart).

Il test. Soprattutto, si **testa**. Le cose non si fanno "su carta, poi in engine e via": si testa, si fa *playtest*, si prova un problema, si cambia, si riprova per vedere se un'altra opzione era migliore; si manda ai tester, e se c'è la possibilità si fa fare un giro al **QA**, persone esperte nel trovare *bug*. È un processo **iterativo**: "niente resta mai fisso nella prima forma che è stata creata". Vale per le meccaniche come per i livelli.

La narrazione ambientale. Vale anche per gli elementi più artistico-narrativi, come l'**environmental narrative**.

I criceti assassini. L'esempio del docente sulla narrazione ambientale merita di essere riportato. Nel livello c'è una registrazione: "Oh no, i criceti sono scappati dalla gabbia e hanno sete di sangue. Hanno già preso Frank, ma io non mi farò beccare. Se qualcuno sente questo messaggio, il codice della porta che porta alle gabbie è 723. Li sento: stanno graffiando le pareti. No...". Chi scrive la registrazione non è necessariamente il level designer; ma è il level designer che decide **dove piazzarla**. Davanti alla porta 723? Oppure dall'altro lato della mappa, così che il giocatore pensi: "ah, 723 è il codice della porta, devo tornare indietro, e attento che adesso ci sono i criceti assassini in giro"? La stessa informazione, collocata in due punti diversi, produce due esperienze diverse: una porta che si apre subito, oppure un ritorno carico di tensione.

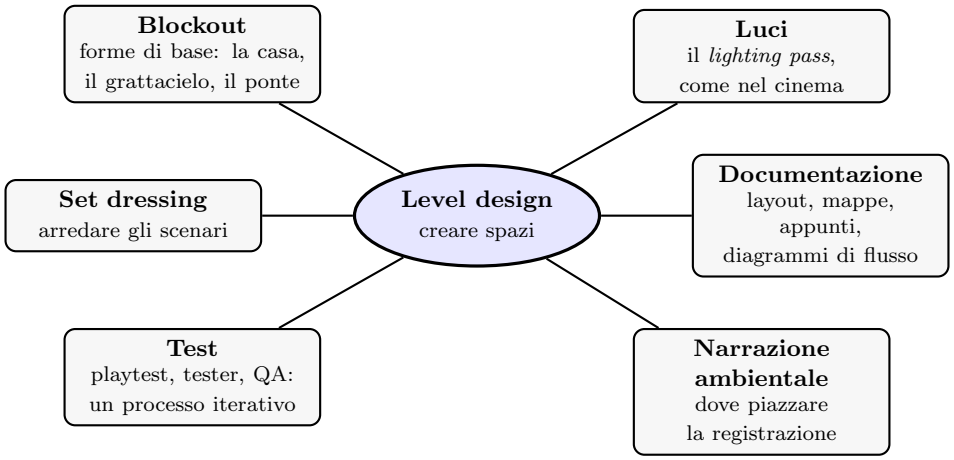


Figure 8.1: "Level design" è un termine ombrello: a seconda del team e del progetto, chi lo pratica può occuparsi di una o più di queste attività.

8.7 Lo strumento del level designer

8.7.1 Il braccio armato dello sviluppo

In un team di sviluppo, il level designer è quello che **mette di più le mani nei giri**: per motivi pratici passa quasi tutto il tempo in engine, a piazzare, sistemare, testare. "Noi siamo il **braccio armato** dello sviluppo, noi siamo quelli che zappano il terreno." La domanda, allora, è: **qual è la nostra zappa?** Qual è lo strumento che un level designer deve assolutamente conoscere per lavorare in un team?

8.7.2 La parola magica: "dipende"

La prima risposta della classe è ovvia: l'**engine** in cui si crea il gioco. Ma quale engine? **Dipende dall'azienda**, da quello che ha scelto per creare il gioco. E qui il docente introduce "la parola magica" del corso: nel corso dell'anno farà moltissime domande, e a moltissime di queste l'unica vera risposta sarà **"dipende"**. "Voglio che vi fissiate la parola magica del mio corso in testa."

L'engine potrebbe essere:

- **Unreal**;
- **Unity**;
- **Godot**, "che sta prendendo molto bene";
- un **framework personalizzato**[15];
- **GameMaker**: *Undertale* e *Deltarune* sono fatti con GameMaker;
- un **engine proprietario**. Chi vuole lavorare in Capcom, per esempio, si troverà a lavorare con l'engine interno con cui sono stati fatti i *Resident Evil* recenti e *Monster Hunter*.[16]

Lo strumento indispensabile, quindi, **non può essere un engine preciso**, perché non si sa mai quale si andrà a usare. Nel corso si userà **Unreal**, perché è considerato lo standard dell'industria "per adesso"; ma questo non significa che sia indispensabile conoscerlo. Ciò che si studia su Unreal ha applicabilità anche negli altri engine, magari con nomi diversi e procedimenti diversi.

8.7.3 La risposta vera: il level designer

"L'unico strumento di cui un level designer ha bisogno per fare il suo lavoro è **il level designer**."

Ecco lo scopo delle domande di inizio lezione: non servivano solo a conoscere la classe o a capire se fossero "tutti fan di rhythm game e di simulatori di gioco d'azzardo". Servivano a far capire che **lo strumento principale** che si userà in tutta la produzione di videogiochi **siete voi**. Conoscere le proprie forze e le proprie debolezze serve ad applicarle a contesti spesso molto diversi tra loro, perché **sono il contesto e le necessità del progetto a decidere quali strumenti si usano**.

Imparare a imparare. "La vostra capacità di imparare è il vostro asset migliore." Il docente può insegnare per tre anni come si usa Unreal; poi si va a lavorare in Capcom, o in FromSoftware, che usa un suo engine sviluppato dagli anni Duemila, totalmente diverso da Unreal. La capacità da sviluppare in questi tre anni non è saper usare Unreal: è **sapere come si impara**. La caratteristica principale dello sviluppatore di videogiochi è essere **uno studente continuo**: la flessibilità mentale di imparare in fretta strumenti diversi, saper fare ricerca, saper fare pratica, tenersi allenati, tenere d'occhio i cambiamenti.

Due esempi. Epic ha annunciato che molto cambierà con la **versione 6** di Unreal, "quando o se esce": quando succederà, se si deciderà di passarci, bisognerà attivarsi. E un gruppo di studenti del terzo anno ha chiesto di fare il proprio progetto in **Unity**: metà del gruppo ci si trovava meglio, l'altra metà non l'aveva mai toccato. Quest'altra metà "si è rimboccata le maniche" ed è andata a impararlo.

"Imparate a imparare: questa è la cosa più importante che dovete saper fare." (Seguito da un'autoironia: "vedo che alcuni di voi hanno la lacrimuccia... no, siamo tutti cinici, col cuore raggrinzito come una prugna.")

[15] La trascrizione riporta "un custom framework fatto in 3GS"; potrebbe trattarsi di una libreria web come *three.js*, ma non è verificabile.

[16] La trascrizione dice "è probabile che si trovino a lavorare con l'Unreal Engine", ma subito dopo parla dell'engine "con cui è stato fatto [Resident Evil] e con cui è stato fatto Monster Hunter". L'engine interno di Capcom è il RE Engine; è probabile che la prima frase sia un errore di trascrizione per "RE Engine".

8.8 Come si impara il level design

A livello pratico, che cosa bisogna fare per imparare il level design? Il docente indica alcune aree su cui concentrarsi.

8.8.1 La teoria

Il vantaggio della teoria è che **si applica a qualunque cosa**. I principi di level design valgono che si lavori in Unity, in Unreal o in Godot, che si stia facendo un FPS o un gioco di corse.

Il landmark. L'esempio è il **landmark**: "un landmark è un landmark è un landmark", ha sempre la stessa funzione in qualunque tipo di gioco. Un landmark è **un elemento dello scenario distinguibile e visibile dalla distanza, che aiuta il giocatore a orientarsi**. La **Madonnina** è il landmark di Milano. Lo si vedrà con attenzione nelle lezioni future; ma si noti che è esattamente ciò che mancava nella Cattedrale del Sangue Sacro.

Rifiutare una regola, sapendolo. Anche quando si lavora a un gioco che **rifiuta intenzionalmente** queste regole, bisogna sapere che le si sta rifiutando. Un horror alla *Silent Hill*, con la nebbia che impedisce di vedere a distanza, lo fa apposta: non è che i suoi autori non sappiano che cosa sia un landmark. C'è una conoscenza di questi elementi e un **rifiuto di usarli**, che fa parte della loro intenzione.

Approfondimento: Lynch e l'immagine della città. Totten dedica al tema una parte del suo libro, ispirata a Kevin Lynch e al suo studio sull'*immagine della città*: gli abitanti si orientano grazie a pochi elementi riconoscibili, tra cui i *percorsi* e i *landmark*, punti di riferimento visibili da lontano. Nel capitolo sul *pathfinding* con le "architectural weenies" (l'espressione con cui Walt Disney indicava gli elementi visibili da lontano che attirano il visitatore, come il castello al centro del parco), il landmark non serve solo a orientarsi ma anche a **attirare** il giocatore verso una meta.

8.8.2 La pratica

"La teoria non vale niente senza la pratica." Bisogna mettere le mani in engine. C'è chi è più interessato alla teoria; ma finché si seguono le lezioni di questo corso si fa level design, almeno parzialmente, almeno per imparare. "Più pratica fate, più imparate a fare quello che dovete fare, più diventate veloci, più cose potete fare." E nei videogiochi "la velocità è tutto".

8.8.3 L'expertise

L'**expertise** non è solo esperienza: è **l'insieme di tutto quello che si sa fare**. Chi sa fare gli *shader* può farsi un sacco di cose da solo, in fretta, senza aspettare altri. Chi sa fare i suoni non deve passare ore a cercare materiale, o rassegnarsi a risultati molto peggiori di quelli che otterrebbe un esperto.

8.8.4 L'esperienza sul campo

"Sembra stupido", ma vagare, esplorare, fare *trekking*, girare per le città, osservare l'ambiente rende un level designer migliore. *The Legend of Zelda* è nato perché **Shigeru Miyamoto** voleva ricreare l'esperienza che aveva avuto da bambino, esplorando la campagna giapponese intorno a casa sua.[17]

8.8.5 Il gruppo

Tutto questo (esperienza, ricerca, studio) non serve a niente se non lo si mette in pratica; e la pratica, in questo corso, avverrà spesso **in gruppo**. C'è chi vuole lavorare da solo, ma finché si segue il corso si lavora anche con gli altri, ed è fondamentale saperlo fare. "Vi assicuro che finirete questi tre anni che qualcuno

[17] La trascrizione riporta "Minamoto" e "Zelda di terraferma"; l'aneddoto è quello, noto, di Miyamoto e delle sue esplorazioni d'infanzia nei dintorni di Kyoto.

di voi odierà qualcun altro. Statisticamente è impossibile che non succeda." Ma anche lavorare con gli altri richiede pratica: è la pratica che rende veloci, che permette di ottenere risultati nei tempi previsti e di conoscere gli strumenti.

8.8.6 La parola che si ripete

"Siete persone intelligenti: starete notando che sto ripetendo molto spesso una certa parola." La parola è **pratica**. L'ultima cosa, ma non ultima per importanza: la cosa più importante quando si fa level design è **fare pratica**. Più si mettono le mani in pasta, meglio si lavora. "Tutto il resto dipende."

8.9 Due curve da conoscere in anticipo

Il docente mostra due grafici tratti dalla ricerca sociologica e psicologica, che descrivono ciò che succederà a quasi tutti nel corso di studi. Li presenta adesso perché "è inutile affrontarlo quando succede": prima ci si prepara, meglio è.

8.9.1 Il picco della fiducia

Si parte tutti più o meno da zero, o comunque da un livello molto basso. Nel corso del primo anno si impara una serie di cose: come mettere le mani in pasta, come analizzare i giochi. Più si impara, più cresce la **fiducia** di sapere che cosa si sta facendo, più ci si sente esperti: "più vi sentirete fighi, più andrete dai vostri amici su Discord a dire: eh, quel livello non è fatto troppo bene, io avrei messo un altro falò".

Poi, di solito tra la fine del primo anno e l'inizio del secondo, si arriva al **picco**, e ci si scontra con la realtà: non si è "così fighi" come si credeva. Ci si rende conto di quanto manca ancora da imparare, di quanto la propria conoscenza sia superficiale, di quanto i risultati siano scarsi rispetto alle speranze. "Alessandro lo sa." **Più si sa, più ci si rende conto di quanto poco si sa.** È **fisiologico**: il docente si aspetta che ognuno abbia "una botta di energia, figaggine, quanto sono bravo, quanto sono bello", e poi, tra primo e secondo anno, "la depressione". "Non c'è niente di male, succede." Se si riesce a superare quel picco, si possono fare grandi cose.[18]

8.9.2 L'effetto Dunning-Kruger

Il secondo grafico è l'**effetto Dunning-Kruger**, anch'esso un effetto osservato: la **fiducia** di una persona in quanto sa di un argomento segue una curva per cui, **meno si sa, più ci si sente sicuri** nel parlarne (figura 8.2).

L'allenatore di calcio. L'esempio del docente: parenti o amici che guardano il calcio e parlano con enorme sicurezza di quanto sia scemo l'allenatore, di quale giocatore avrebbe dovuto sostituire, "e non capisco perché". "Quello è l'effetto Dunning-Kruger in azione." Ne sanno di più di una persona che fa calcio da trent'anni? Probabilmente no.

Il giocatore che non ha mai fatto un gioco. Appena si impara poco di un argomento, la fiducia ("io so quasi tutto di questa roba, sono un esperto") schizza in alto. La gente che gioca ai videogiochi ma non li ha mai fatti si trova più o meno lì: "fiduciosissima" nel dire "questo gioco fa schifo, avrebbero dovuto cambiare questa cosa, avrebbero dovuto portare i giocatori di una mappa da 12 a 128". È chiaro che non sanno di che cosa parlano. Poi, più si impara, più ci si rende conto di quanto sia limitata la propria conoscenza: il punto più basso della curva è quando si sono imparate molte cose e si capisce quanto ne manchino ancora.

L'obiettivo. "Il vostro obiettivo all'interno di questo corso di studi è **sopravvivere ai due punti centrali di questo grafico, senza abbandonare la nave**." E, aggiunge, "non rendetemi la vita impossibile finché

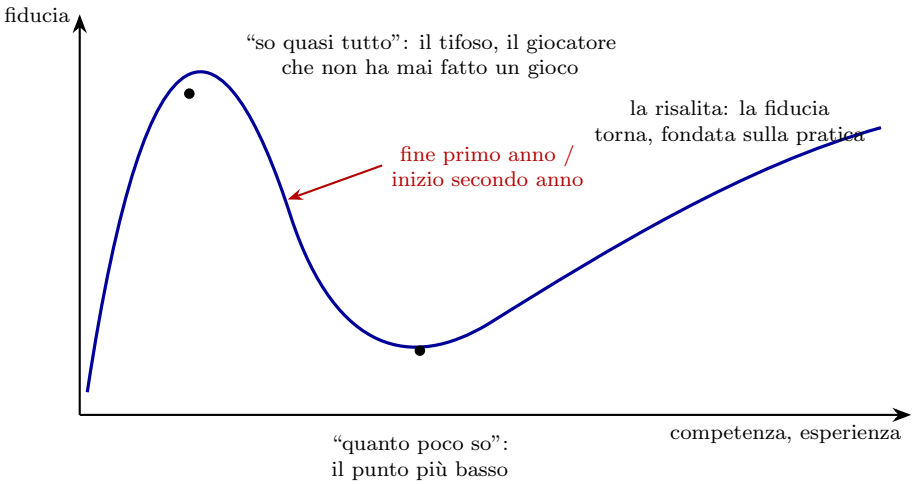


Figure 8.2: Schema dell'effetto Dunning-Kruger come lo descrive il docente: con poche conoscenze la fiducia schizza in alto; imparando si precipita nel punto in cui ci si rende conto di quanto poco si sa; solo dopo risale. L'obiettivo del corso è superare la parte centrale della curva "senza abbandonare la nave". Le proporzioni sono indicative.

siete ancora nella prima metà del grafico": durante l'anno ci si sentirà "fighissimi", e lui sarà lì a dire "bravi, bravi", aspettando che la prima fase passi. "L'annuncio di servizio ve l'ho fatto; il resto sta a voi."

Approfondimento: che cosa dice davvero lo studio. L'effetto prende il nome da David Dunning e Justin Kruger, che nel 1999 pubblicarono uno studio in cui le persone meno competenti in un compito tendevano a **sovrastimare** la propria prestazione, e le più competenti a sottostimarla leggermente. La curva "a montagna e valle" che circola sotto questo nome (e che si vede anche nella slide) è una divulgazione molto diffusa, più che il grafico originale dello studio; ma rende bene l'idea che il docente vuole trasmettere: la sicurezza non è una misura della competenza.

8.10 Tre corollari

8.10.1 Primo: non esistono regole fisse

Legato alla parola magica: **nel design non esistono regole fisse**, che si applichino invariabilmente a ogni cosa. Lo scopo non è mai trovare la risposta perfetta da applicare a ogni situazione: è **isolare la domanda e scoprire la ragione** per cui si sta facendo una scelta. Quella ragione la si può poi declinare a seconda della situazione. Progetti diversi, dimensioni diverse, disponibilità economiche o di risorse diverse, persone diverse nel team portano a risposte diverse.

L'artista del fantasy. Se l'artista che crea i modelli 3D non ha mai fatto fantascienza, fare un gioco di fantascienza "magari non è il massimo". Conviene considerare un'ambientazione fantasy, "perché è quello che il nostro disegnatore disegna da quando aveva cinque anni".

8.10.2 Secondo: il design è una disciplina olistica

"Chi conosce questa parola?" Silenzio. L'**olismo** è il considerare qualcosa non come la somma di più parti separate, ma come **un unico insieme**, diverso dalla somma delle parti. Un designer deve capire l'insieme, non solo le parti.

[18] Il grafico della slide non è disponibile e la trascrizione non ne riporta il titolo esatto ("il picco della realtà"); la descrizione è ricostruita dalle parole del docente.

L'esempio pratico, dal level design: bisogna sapere

- creare una **conformazione del terreno** interessante;
- costruire una **casa** che abbia, almeno visivamente, un senso architettonico, con gli strumenti a disposizione;
- posizionare gli **elementi sonori**: se c'è un fiume, il suono deve accompagnare il giocatore mentre si sposta lungo il fiume;
- posizionare l'**illuminazione**: la luce di una lampada non sta "dentro la lampada", va regolata con attenzione, e certi oggetti vanno esclusi dal proiettare ombre, "altrimenti diventa proprio brutto vedere tutte le ombre sparate sui muri";
- costruire **puzzle**, posizionare **nemici**.

Tutte queste cose formano un insieme che è più della somma delle parti. Un level designer deve avere almeno una **conoscenza di base di tutti gli strumenti e di tutte le discipline** con cui lavora; e più sa delle altre discipline, meglio riesce a **comunicare con i colleghi** che se ne occupano.

8.10.3 Terzo: il design non è arte, è artigianato

Qual è la differenza? L'**arte** è qualcosa che non può essere giudicato pienamente secondo canoni puramente tecnici. Non si può dire che i *Sex Pistols* facessero schifo perché non sapevano fare *shredding* velocissimo come i *Metallica*: sono due cose diverse, c'è un intento artistico.

L'**artigianato** è diverso. Non si può dire "questa sedia non è fatta per sedercisi, perché se ti siedi si spacca, però guarda com'è bella e artistica". No: è una sedia, il suo scopo è che ci si possa sedere sopra. Se non ci si può sedere, non si sta facendo una sedia: si sta facendo un'opera d'arte, e allora non la si mette sotto il tavolo, la si mette su un piedistallo. Nel design **si fa artigianato**: ciò che si costruisce ha uno **scopo**, e richiede capacità ed esperienza artigianali.

8.10.4 Sporcarsi le mani

Da qui i consigli pratici:

- **fare giochi**: più giochi si fanno, meglio si diventa come designer. Le *game jam*, se si vuole, ma anche progetti un po' più lunghi;
- **ricreare** ciò che si vede, con gli strumenti che si studieranno nel corso;
- **trovare altre persone**, "creare una scena": andare alle serate di *Milano Indies*, nei caffè in cui la gente si trova a programmare, costruirsi attorno un gruppo di persone che fanno la stessa cosa, "perché si migliora sempre insieme". I punk non stavano da soli in cameretta: si trovavano nei bar, andavano ai concerti insieme, si incontravano al parco, e hanno creato una delle scene musicali più importanti della storia;
- **farsi ispirare**: variare i giochi a cui si gioca, i film che si guardano, la musica che si ascolta.

8.11 La struttura del corso

Prima della parte pratica, un riassunto di come funzionerà il corso.

Pratica in engine. Si lavorerà molto in **Unreal Engine 5**. Ci saranno momenti di **pratica assistita**, in cui il docente mostra e guida, e momenti di **pratica a casa**.

I task. Il docente assegna dei compiti, da consegnare entro una o due settimane a seconda del task, e poi li si rivede insieme in classe per correggere eventuali problemi.

La teoria. Ci saranno momenti di studio teorico: **teoria** e **metodologia** del level design. Anche la teoria avrà momenti di pratica assistita, esercizi in classe ed esercizi a casa.

Le letture. Qualche libro per chi vuole approfondire, non necessariamente obbligatorio: il docente tende a lasciare suggerimenti più precisi a chi decide di specializzarsi nel level design, e lo si deciderà durante l'anno in base alle preferenze.

La pausa. Si è a metà lezione, ed è ora della pausa. Il docente ha una "malattia un po' pesante" e mal di gola, quindi la fa un po' più lunga, una ventina di minuti, "sennò arrivo stasera che sputo sangue". Al rientro si passa a Unreal: bisogna averlo installato, in una versione recente ("dopo la 5.5, 5.6 dovrebbe essere più o meno la stessa cosa"; al massimo cambia leggermente l'interfaccia).[19]

8.12 Il progetto in Unreal Engine

8.12.1 Lanciare l'engine in anticipo

Prima ancora di riprendere, il docente chiede di **lanciare Unreal** negli ultimi minuti della pausa: "ha la tendenza a metterci un pochino ad aprirsi". Va bene qualsiasi versione dalla 5.6 in su; sul computer dell'aula non è installata la 5.8, quindi il docente usa la 5.6. ("Non dirmi che si sta aggiornando. Ti prego.")

8.12.2 Creare il progetto

Qualunque progetto va bene per ciò che si farà oggi. I passaggi:

1. dal launcher si avvia Unreal Engine (5.6 o successivo): si apre il **Project Browser**;
2. si sceglie la categoria **Games** e il template **Third Person**;
3. si dà un nome al progetto (il docente usa "primo anno");
4. si fa **Create** e si aspetta che il progetto venga creato.

Lo starter content. La slide mostra una versione più vecchia del launcher, in cui lo **starter content** si includeva con una spunta al momento della creazione. Nelle versioni nuove non c'è più quell'opzione, ma lo si può aggiungere dopo (lo si vedrà tra poco, sezione 8.15). Per oggi non serve.

Il computer che crolla. Come nella lezione di programmazione (capitolo 7), il computer dell'aula non regge: l'editor si blocca, il disco "sta facendo un casino terrificante", e il docente deve riaprire tutto più volte, fino ad aprire un progetto qualsiasi già presente ("incrociamo le dita"). Nel frattempo fa l'appello.

Le slide. Le slide della lezione sono nel **drive condiviso** degli studenti: cartella di Milano, primo anno, *Lezioni*, *Level Design*. Il docente le scrive in modo molto esteso proprio per usarle anche come appunti, così che si possa rivedere tutto; ma consiglia comunque di **prendere appunti**, "perché è il modo migliore di fissarsi le cose in mente".

8.13 L'editor: muoversi e piazzare oggetti

L'editor è diviso in **finestre**. Si possono riarrangiare e spostare dove si preferisce, ma alcune vanno conosciute subito (si vedano la figura 1.1.4 e la sezione 1.1.16).

[19] La registrazione della pausa contiene quasi solo frammenti incomprensibili e conversazioni private degli studenti, che non sono riportate.

8.13.1 La viewport

La finestra più importante è quella **centrale**, che mostra l'aspetto del livello: la **viewport**, "il cuore del nostro editor". È probabilmente l'unica finestra che resterà sempre aperta, comunque si decida di organizzare l'editor. Cliccando sugli oggetti li si **seleziona**.

Che cos'è un actor. Un **actor** è **un oggetto all'interno del gioco**, nel livello. La definizione verrà precisata tra poco, distinguendolo dall'*asset* (sezione 8.15).

Muoversi. Per spostarsi nella viewport:

- si **tiene premuto il tasto destro** del mouse e si muove il mouse per **ruotare la telecamera**;
- tenendo premuto il tasto destro, con **W, A, S, D** ci si sposta "come in un first-person shooter";
- sempre con il tasto destro premuto, **Q** ed **E** spostano la telecamera in **verticale**.

"Prendeteci la mano": sono i controlli di base per esplorare la scena che Unreal crea automaticamente per ogni nuovo progetto.

8.13.2 Place Actors e le forme di base

A sinistra c'è la finestra **Place Actors**, che permette di selezionare degli actor da inserire nel livello. "Oggetti" non vuol dire necessariamente oggetti fisici: possono essere **luci**, **suoni**, **volumi** (che si vedranno più avanti). Si afferrano, si trascinano e si piazzano nel mondo di gioco.

Gli actor sono divisi in **categorie** (nelle versioni recenti l'elenco delle categorie è verticale; gli screenshot della slide vengono da una versione un po' precedente, ma le funzionalità sono le stesse). Se lo schermo è piccolo, **tre puntini** indicano che ci sono altre categorie nascoste. Quella che interessa oggi è **Shapes** (sotto *More*, nelle versioni recenti), che contiene le **forme di base**.

Si prende il **cubo**, lo si trascina in uno spazio libero della scena: "congratulazioni, avete fatto lo stesso passo che fa un bambino di due anni quando gioca con i cubi". Un passo alla volta.

8.13.3 Il gizmo

Quando si trascina un oggetto nella scena compaiono tre frecce: un piccolo **gizmo** che permette di spostarlo (figura 8.3).

- Afferrando una **freccia** si sposta l'oggetto lungo un solo asse: **X** (rossa), **Y** (verde) o **Z** (blu, la verticale).
- Afferrando il **centro**, lo si sposta in relazione alla telecamera, sul piano che la telecamera sta guardando.
- Afferrando uno dei **quadratini** tra due frecce, lo si sposta su **due assi contemporaneamente**: si evidenziano entrambe le frecce. Con il quadratino tra X e Y, per esempio, lo si sposta solo sul piano orizzontale, senza preoccuparsi di alzarlo o abbassarlo: se lo si era appoggiato a terra, resterà a terra.

"Prendeteci la mano, perché sarà la cosa che farete di più in questo engine." Ogni volta che si seleziona qualcosa, anche un elemento dello scenario, compare un gizmo, e lo si può spostare a piacere.

Il pivot non è sempre al centro. Cliccando in giro si nota subito che non tutti i gizmo stanno al centro dell'oggetto: certe pedane della scena, per esempio, lo hanno sulla parete frontale. Il punto in cui viene piazzato il gizmo è **arbitrario**: lo decide chi fa la *mesh*, e può essere in qualunque punto. Lo si può anche spostare, se serve cambiare l'origine di una mesh, ma se ne parlerà più avanti.

8.14 L'Outliner: tenere in ordine il livello

In alto a destra c'è l'**Outliner**: **la lista di tutti gli actor presenti nel livello**. (Il docente usa la terminologia inglese, "actor" piuttosto che "attori": "è più facile trovare risorse e abituarsi".)

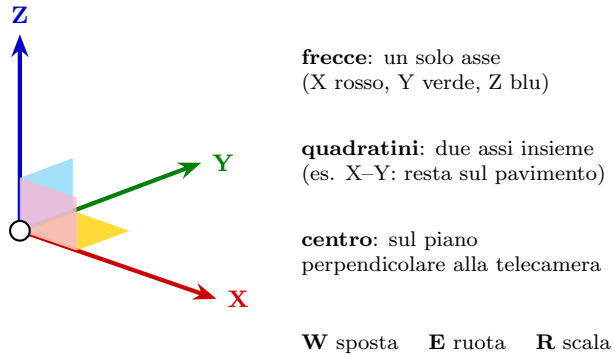


Figure 8.3: Il gizmo di traslazione, schematizzato. Il punto in cui compare (il *pivot*) non è necessariamente il centro geometrico dell'oggetto: lo decide chi ha creato la mesh.

Ogni oggetto ha un nome. Il cubo appena trascinato, nel caso del docente, si chiama *Cube2*, perché c'era già un altro cubo. Cliccandoci sopra nell'Outliner lo si seleziona, e lo si ritrova nella viewport.

Le cartelle. Tutto ciò che si trascina nel livello entra nell'Outliner secondo una **gerarchia**: gli actor si possono spostare, ordinare, raccogliere in **cartelle** (nella scena di esempio ce n'è una chiamata *Playground*). Una nuova cartella si crea con tasto destro, **Create Folder** (il docente la chiama "Bubba", "non so come chiamarla"). Organizzare le cose come file in cartelle, come sul proprio computer, aiuta a **tenere organizzato il livello**: ogni volta che si mette qualcosa nel gioco, fare attenzione al **nome** e alla **cartella**.

L'ordine alfabetico. Nota importante: l'Outliner è **ordinato alfabeticamente**. Non gli interessa che cosa è stato piazzato prima o dopo, né dove: l'ordine è sempre alfabetico. Le cartelle servono a raggruppare, e l'ordine alfabetico è quello in cui le cose verranno mostrate.

Selezionare più oggetti. L'Outliner rispetta le regole di Windows:

- cliccando sul primo elemento e poi, con **Shift**, sull'ultimo, si seleziona tutto ciò che sta in mezzo (il docente seleziona così tutti gli oggetti della cartella *Playground*);
- con tasto destro su una cartella, **Select** → **Immediate Children**, si selezionano rapidamente tutti gli elementi contenuti.

Rinominare. Come in Windows, si seleziona l'oggetto e si preme **F2** (il docente lo chiama "Cubo Super Ultra"). In alternativa, un secondo clic lento sul nome, oppure tasto destro, *Rename*. Ma il consiglio "spassionato" è imparare le **scorciatoie**: "vi velocizzano tanto il lavoro". I tasti *funzione*, quelli più in alto sulla tastiera, vale la pena impararli.

Ctrl+Z. Come in Windows, **Ctrl+Z** annulla l'ultima azione: "abituatevi a usarlo, è la scorciatoia più utile".

8.15 Il Content Browser: asset e actor

In basso c'è il **Content Browser**: **tutti gli asset che fanno parte del gioco**. Qui il docente introduce una distinzione importante:

Asset un oggetto **salvato tra i file del progetto**, su disco;

Actor un oggetto **nell'editor**, all'interno di un livello.

Gli asset stanno nel progetto e possono essere trascinati dal Content Browser dentro un livello; lì diventano actor.

Aggiungere lo starter content. Nelle versioni recenti lo starter content non viene inserito automaticamente. Per aggiungerlo (passaggio che non è nelle slide):

1. tasto destro nel Content Browser;
2. **Add Feature or Content Pack**;
3. nella sezione **Content**, **Starter Content**.

È un insieme di texture, oggetti e altro materiale, fornito gratuitamente da Epic con Unreal Engine, per cominciare a fare i propri progetti. Oggi non serve, perché si lavora solo con le forme di base; ce ne si preoccuperà dalla prossima lezione.[20]

Le finestre si ridimensionano. Come tutte le finestre, anche il Content Browser si può allargare o stringere a piacere.

8.15.1 Istanze e stampi

Il docente ci torna sopra, perché è una distinzione da avere chiara. Un **asset** è la versione salvata su disco di un oggetto del progetto. Un **actor**, in termini tecnici, è una **instance** (istanza): una **copia** di quella cosa salvata su disco, inserita nel livello. Il cubo "Cubo Super Ultra" è un'istanza del cubo; l'altro cubo nella scena è un'altra istanza dello stesso cubo.

Le modifiche restano sull'istanza. Quando si modifica un oggetto nei *Details* o con il gizmo, **si modifica l'istanza nel livello, non l'asset nel progetto**. Se si allarga una sedia, le si cambia il materiale e poi si trascina di nuovo la stessa sedia dal Content Browser, la nuova sarà identica alla sedia di partenza. Le modifiche fatte nell'editor restano solo su quell'istanza, e non vengono sovrascritte sull'originale.

L'immagine suggerita: pensare agli asset come a uno **stampo**, e alle istanze nel livello come alle **copie** create da quello stampo (figura 8.4).

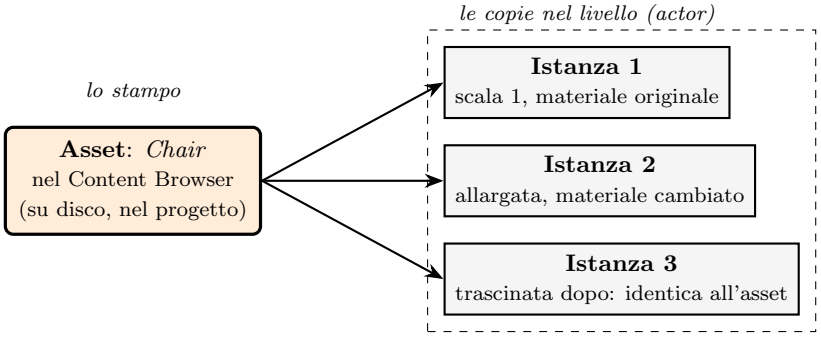


Figure 8.4: Asset e istanze: le modifiche fatte nel livello (nei Details o con il gizmo) restano sulla singola istanza e non toccano l'asset di partenza.

8.16 Il transform e la finestra Details

8.16.1 Copiare, incollare, annullare

Prima di parlare del transform, altre scorciatoie "ereditate" da Windows:

- **Ctrl+C**, **Ctrl+V**: copia e incolla. Si crea una copia del cubo, che poi si può spostare;

[20] Il docente aggiunge un commento sulle scelte recenti di Epic ("credo che tutti quelli che lavorano nel mondo dei videogiochi si stiano lamentando delle scelte che sta facendo Epic in questi ultimi mesi"), senza specificare a quali si riferisca.

- **Ctrl+clic** su più oggetti: selezione multipla (si vede un contorno giallo attorno a tutti gli oggetti selezionati). Con Ctrl+C, Ctrl+V si duplicano tutti insieme;
- **Ctrl+Z** ripetuto: si torna indietro fino all'inizio;
- per **rifare** un'azione annullata (*redo*): in alcuni programmi è **Ctrl+Y**, in altri **Ctrl+Shift+Z**, perché nel linguaggio di design di Windows **Shift inverte l'operazione** che si sta facendo. "È una gran rottura di scatole ricordarsi quale ha l'uno e quale l'altro": se non funziona uno, si prova l'altro (Blender, per esempio, usa Ctrl+Shift+Z).

8.16.2 Il transform

In termini tecnici, una **trasformazione** (*transform*) è l'insieme di tre valori: **posizione**, **rotazione** e **scala**; in inglese *location*, *rotation*, *scale*.

8.16.3 La finestra Details

Le caratteristiche dell'oggetto selezionato si vedono nella finestra in basso a destra, **Details**. Selezionando il cubo, in alto si legge il suo nome ("Cubo Super Ultra") e che si tratta di un'*instance*. Le prime tre voci sono i tre elementi del transform.

Location. Si può modificare cliccando e trascinando sul campo, oppure scrivendo il valore: **X** (rosso), **Y** (verde), **Z** (blu), "ricordatevelo". Il docente scrive 400 nella X, preme Invio, e il cubo si sposta: "l'ho spostato di 70 centimetri più indietro" (era a 330).

L'unità di misura. Nota importante: **l'unità di misura di base di Unreal è il centimetro** (si veda anche la sezione 1.1.17).

Rotation. Trascinando il primo valore, l'oggetto ruota **attorno all'asse X**: l'asse X "fa da perno". Il secondo valore lo fa ruotare attorno all'asse Y, il terzo attorno all'asse Z (quindi "così", sul piano orizzontale).

Scale. È il fattore per cui viene **moltiplicata** la dimensione dell'oggetto lungo ciascun asse. Aumentando la scala sulla X, l'oggetto si allunga lungo la X; lo stesso per Y e Z. Applicato a una sedia della scena, la si allunga in modo "psichedelico". Le tre trasformazioni si combinano per ottenere l'effetto cercato.

Tutto il resto. Sotto il transform ci sono tutte le altre informazioni sull'oggetto selezionato e sulle sue caratteristiche: il materiale, le caratteristiche fisiche, una casella per **simulare la fisica**, una per decidere se l'oggetto **proietta un'ombra** quando è colpito da una luce (*Cast Shadow*). Nel corso dell'anno se ne vedrà la maggior parte, non tutto: alcune voci sono estremamente tecniche e poco interessanti per il level design. Delle luci ci si occuperà in futuro.

8.16.4 W, E, R (e Q)

Anche il transform ha le sue scorciatoie da tastiera, che cambiano la modalità del gizmo:

- **W**: spostamento (la modalità di base);
- **E**: rotazione, con un gizmo pensato per ruotare;
- **R**: scala;
- **Q**: solo selezione ("non credo di averlo mai usato, tanto vale usare direttamente W").

L'asse appoggiata al muro. La dimostrazione del docente: per fare un'asse appoggiata al muro, **R** per scalare il cubo e renderlo sottile e lungo, **W** per avvicinarlo al muro, **E** per inclinarlo, di nuovo **W** per sistemarlo, "e tac, ho fatto la mia asse appoggiata al muro". Poi si tratterà di metterci una texture, e anche questo si vedrà. "Sembrano tante robe da memorizzare", ma più si fa pratica, più diventano automatiche: "è il bello del cervello umano, siamo in grado di abituarci molto velocemente".

8.16.5 I gruppi

Si possono **raggruppare** più oggetti per spostarli tutti insieme: selezionarli con **Ctrl+clic** e premere **Ctrl+G**. L'esempio: quando si prepara una libreria e ci si mettono dentro i libri, raggruppare i libri permette di spostarli, copiarli e incollarli velocemente.

8.16.6 Le operazioni nei campi numerici

Verso fine lezione, mentre la classe lavora, il docente mostra "un piccolo trucco magico" (a voce, perché ha chiuso Unreal per paura che il computer crolli di nuovo). Tutti i campi numerici dei Details **accettano operazioni matematiche** e le eseguono. Se un oggetto ha una scala di 2,75 e la si vuole triplicare, basta scrivere accanto al valore `*3` e premere Invio: il calcolo è automatico. Per dimezzare, `/2`; per aggiungere 3, `+3`. Vale per tutti i campi in cui si può inserire un numero, ed è così anche in Unity e negli altri engine. "È molto, molto comodo per modificare robe con precisione."

8.17 Leggere la documentazione

"Nota importantissima": **imparate a leggere la documentazione**, e a interpretare ciò che vi è scritto. "Quasi tutto quello che vi serve è scritto da qualche parte." Sul sito di Epic Games c'è un'enorme quantità di informazioni: come funzionano le basi, i contenuti, la creazione di mondi virtuali, la grafica. Nelle slide c'è un link alla documentazione di Unreal Engine 5.5, ma si può consultare direttamente quella della 5.8. Il docente nota, senza commentare oltre, che "Epic è impazzita per l'AI": nella documentazione c'è "un sacco di roba sull'AI".

È lo stesso principio della sezione 8.7: saper fare ricerca e sapere dove cercare è parte dell'imparare a imparare.

8.18 I livelli e il salvataggio

Un livello per ogni zona. Unreal è diviso in **livelli**: per creare una nuova zona, "tra virgolette", si crea un nuovo livello, a meno che non si stia facendo un *open world*. Quello è "un discorso particolare" che richiede un lavoro diverso; e anche lì, comunque, ci sono aree chiuse in cui si entra con caricamenti e simili.

Creare un livello. **File → New Level**, poi si sceglie un template. Il consiglio è partire da **Basic**, che ha già tutto il necessario. *Open World* per ora no; *Empty* nemmeno, perché per partire da un livello vuoto bisogna aggiungere un po' di cose, e se ne parlerà più avanti.

Salvare spesso. "Abituatevi a salvare spesso." In Unreal, un livello appena creato **non viene salvato finché non lo si salva manualmente**. Per salvare:

- l'icona del **floppy disk** ("per chi non sapesse esattamente cos'è": ormai è soprattutto il simbolo del salvataggio);
- **Ctrl+S**: salva il **contesto attuale** in cui si sta lavorando, dato che Unreal può avere più schede aperte in parallelo;
- **Ctrl+Shift+S**: salva **tutto** (a volte richiede un po' di calcolo).

In certi casi l'editor chiede dove salvare: si sceglie la cartella del progetto in cui si vuole mettere il file.

8.18.1 Le scorciatoie utili

Nelle slide c'è un elenco di **scorciatoie utili**, molte delle quali corrispondono a funzioni di Unreal che non hanno un pulsante nell'interfaccia e si possono usare **solo da tastiera**. Il consiglio è provarle almeno una alla volta. Quelle viste a lezione sono raccolte nella tabella 8.1.

Snap to floor. Una delle più utili è lo **snap to floor**: se un oggetto è sospeso in aria e non appoggia sul pavimento, premendo il tasto **Fine** (*End*) lo si attacca automaticamente alla superficie sottostante. (Sul portatile dell'aula il tasto *End* non c'è: "magnifico".)

Comando	Effetto
Tasto destro + mouse	ruota la telecamera nella viewport
Tasto destro + W/A/S/D	sposta la telecamera come in un FPS
Tasto destro + Q/E	sposta la telecamera in verticale
W / E / R	gizmo di spostamento / rotazione / scala
Q	solo selezione
F2	rinomina l'oggetto selezionato
Ctrl+clic	selezione multipla
Shift+clic (Outliner)	seleziona tutto tra due elementi
Ctrl+C / Ctrl+V	copia / incolla (duplica)
Ctrl+Z	annulla l'ultima azione
Ctrl+Shift+Z (o Ctrl+Y)	ripristina l'azione annullata
Ctrl+G	raggruppa gli oggetti selezionati
Fine (*End*)	*snap to floor*: appoggia l'oggetto alla superficie sotto
Ctrl+S	salva il contesto attuale
Ctrl+Shift+S	salva tutto
`*3`, `/2`, `+3` in un campo	esegue l'operazione sul valore

Table 8.1: Le scorciatoie e i trucchi dell'editor visti nella prima lezione di level design.

8.19 Il task: il blockout della propria stanza

Negli ultimi venticinque minuti circa si parla del lavoro per la volta successiva, e lo si comincia in classe.

> **Creare un blockout della propria stanza**: con le forme di base (cubi, ma anche cilindri, scale e tutto ciò che c'è nella sezione *Shapes*), modificate, spostate, allargate, ruotate, ricostruire una copia della stanza in cui si dorme. Niente colori, niente materiali: "solo blocchi". Consegna: **tre screenshot** nella cartella dei task, entro la settimana successiva.

Perché cominciare in classe. Il task si comincia subito, non tanto per finirlo quanto per verificare che tutto funzioni: finestre che non si vedono, problemi di installazione e simili si risolvono meglio insieme, in aula. "Non c'è bisogno che iniziate a fare subito la camera": si può cominciare spostando forme e prendendo la mano.

Le domande.

- *Bisogna fare anche pareti, porte, finestre, o bastano pavimento e mobili?* "Come preferite": è un esercizio molto libero, per prendere la mano. L'importante è allenarsi a lavorare **velocemente**: "metti la cera, togli la cera". Spostare, ridimensionare, ruotare.
- *Le forme sono molto geometriche: se volessi smussare i cuscini?* Le forme di base **non si possono modificare**, o meglio: si può, ma con uno strumento che si vedrà molto più avanti. Si possono però combinare forme diverse. Lo scopo è la pratica con le forme e le funzioni di base; chi vuole migliorare il risultato può farlo, ma l'importante è dimostrare di aver capito come funziona l'engine.

- *È il task per la settimana prossima o va consegnato oggi?* Per la settimana prossima: lo si è iniziato in classe solo per risolvere eventuali problemi tutti insieme.

La revisione. I lavori verranno guardati velocemente la volta successiva, proiettati sullo schermo dell'aula; ma sarà più una verifica che il task sia stato fatto. Le correzioni più precise arriveranno con i task successivi.

Approfondimento: perché la propria stanza. L'esercizio, oltre a far prendere la mano con gli strumenti, mette insieme due dei consigli della lezione: l'**esperienza sul campo** (osservare lo spazio reale, le sue misure e le sue proporzioni) e la **pratica**. Ricostruire uno spazio che si conosce perfettamente costringe a confrontarsi con la scala: un letto, una porta, una scrivania hanno misure precise in centimetri, e Unreal ragiona proprio in centimetri. È anche, in piccolo, il primo blockout: forme di base che rappresentano oggetti, come il cubo che "è la casa" della sezione 8.6.

8.20 Per chi vuole approfondire

Prima di chiudere, "cinque minuti e trenta secondi di attenzione". Il level design è un ambito molto specializzato. Tutti devono imparare almeno ciò che si fa al primo e al secondo anno: lavorando nei videogiochi si finisce per parlare con altri designer, e sapere di che cosa si occupano le varie discipline è importante. Ma è anche innegabile che c'è chi è più interessato al level design e chi ad altri aspetti, per cui il level design conta meno. Per i primi, prima della slide del task c'è una **slide extra** con materiale aggiuntivo, non obbligatorio.

I talk della GDC. I primi due link sono video su YouTube: registrazioni di talk della **GDC**, la *Game Developers Conference*[21]. È una convention annuale negli Stati Uniti, considerata ancora oggi **il punto di ritrovo dell'industria**. Non è la Milano Games Week, non è il Comic-Con di San Diego: non ci vanno i giocatori, ci vanno **gli sviluppatori**, solo chi i videogiochi li fa. Lì i designer tengono interventi, panel, conferenze e dibattiti sulla disciplina. Partecipare costa "veramente tantissimo" (è la critica principale che le viene fatta: l'idea è che il biglietto, intorno ai duemila euro, lo paghi lo studio), ma le registrazioni sono disponibili online. Si può vedere, per esempio, come gli sviluppatori di *Resident Evil 7* hanno rinnovato da zero il gioco, e l'intero franchise, raccontato da loro.

Il libro. C'è poi un link a ***An Architectural Approach to Level Design*** (di Christopher W. Totten), "uno dei migliori libri sul level design che abbia mai letto" e un ottimo modo per iniziare ad affrontare la disciplina. Lo si trova a molto meno dei 67 dollari indicati nel link; il docente lo "stra-consiglia".

Non sono obbligatori: chi è incuriosito li trova lì, soprattutto i talk; chi preferisce specializzarsi in altro non ha problemi. Alcuni degli argomenti trattati in quei talk e in quel libro verranno comunque affrontati nel corso dell'anno.

8.21 La chiusura

Come sempre, qualche minuto per domande "varie ed eventuali", anche se alla prima lezione di solito non c'è molto da chiedere; e per una volta, nota il docente, Unreal ha funzionato a tutti (tranne a lui). Uno studente ha già quasi finito la stanza, e chiede chiarimenti sulle forme geometriche (si veda la sezione 8.19).

Come contattare il docente. Lo si trova su **Discord**, cercando "Stefano Zocchi" nel server della scuola[22], oppure per email, che il docente comunica alla classe per le questioni più complesse.

Il riepilogo. "Per la prossima volta voglio che mi creiate una simulazione di camera vostra fatta solo con le forme di base, senza colori, niente, solo blocchi." Non devono essere per forza cubi: cilindri e tutte le altre forme della sezione *Shapes*. "Divertitevi, usate la fantasia." Tre screenshot nella cartella dei task. Ci si rivede la settimana successiva.

[21] Il docente la chiama "Game Developers Convention"; il nome ufficiale è *Game Developers Conference*.
[22] La trascrizione riporta il nome del server come "Hive"; non è verificabile.

Punti chiave del capitolo

- Il corso di **Level Design** unisce teoria, metodologia e soprattutto **pratica in Unreal Engine 5**, con pratica assistita, task a casa (da rivedere in classe) ed esercizi teorici.
- Un livello brutto va analizzato: **perché** non funziona e **come** lo si migliorerebbe. La Cattedrale del Sangue Sacro di *Code Vein* riassume gli errori tipici: labirinto, piani non collegati, rumore visivo, nessun landmark, ripetitività, nessun interesse nell'esplorare. Bisogna anche distinguere **scelte creative** (la Città Infame, *Nightreign*, *Frostpunk 2*) da **errori**, e valutare le conseguenze di ogni modifica.
- **Parlare in pubblico** è indispensabile: un designer mediocre che sa presentare le proprie idee avrà più successo di un designer geniale che non ci riesce.
- Il **level design** è la disciplina che crea livelli, ambienti, spazi di gioco, di solito con un **level editor**; ha un lato artistico e uno tecnico, e altri nomi (environment design, game mapping). Il **blockout** usa forme di base prima dell'arte finale.
- Il level designer **crea spazi**, ma "level design" è un **termine ombrello**: blockout, luci (il *lighting pass*, come nel cinema), set dressing, documentazione (layout, mappe, diagrammi di flusso), test iterativo (playtest, QA), narrazione ambientale (dove piazzare l'informazione cambia l'esperienza).
- La parola magica è **"dipende"**. L'engine dipende dall'azienda (Unreal, Unity, Godot, GameMaker, engine proprietari): lo strumento vero del level designer è **il level designer**, e la capacità più importante è **imparare a imparare**.
- Per imparare: **teoria** (vale ovunque; il **landmark** è un elemento distinguibile e visibile da lontano che aiuta a orientarsi; anche rifiutarlo dev'essere una scelta consapevole), **pratica**, **expertise**, **esperienza sul campo**, **lavoro di gruppo**. E soprattutto pratica.
- Il **picco della fiducia** (tra primo e secondo anno) e l'**effetto Dunning-Kruger** (meno si sa, più ci si sente sicuri): l'obiettivo è attraversare la valle senza abbandonare.
- Tre corollari: **non esistono regole fisse** (isolare la domanda e la ragione della scelta); il design è **olistico** (conoscere almeno le basi di tutte le discipline); il design è **artigianato**, non arte (ha uno scopo). Fare giochi, ricreare, trovare una comunità, farsi ispirare.
- Editor: **viewport** (tasto destro + WASD, Q/E), **Place Actors** (Shapes), **gizmo** (frecce, quadratini, centro; il pivot lo decide chi fa la mesh), **Outliner** (ordine alfabetico, cartelle, F2), **Content Browser**, **Details**.
- **Asset** = file nel progetto (lo stampo); **actor** = istanza nel livello (la copia). Le modifiche fatte nel livello restano sull'istanza.
- **Transform** = location, rotation, scale; l'unità è il **centimetro**. W/E/R per spostare, ruotare, scalare; Ctrl+G per raggruppare; *End* per lo snap to floor; i campi numerici accettano operazioni (`*3`, `/2`).
- **Leggere la documentazione**. Nuovo livello: File, New Level, Basic; **salvare spesso** (Ctrl+S, Ctrl+Shift+S).
- Task: il **blockout della propria stanza** con le sole forme di base, tre screenshot nella cartella dei task. Per approfondire: i talk della **GDC** e *An Architectural Approach to Level Design*.

Chapter 9

Agency, comportamento e visione: perché i giocatori scelgono, e la bussola del progetto

Dopo la parentesi del capitolo precedente, che era in realtà la prima lezione del corso di level design (capitolo 8), si torna al corso di game design vero e proprio, con il docente dei primi sei capitoli. Il filo riprende esattamente da dove era stato lasciato: il production dictionary (sezione 4.3), i *core elements* (capitolo 2), la fine della pre-produzione e la *game vision* (sezione 5.11).

Il docente stesso annuncia che quella di oggi è, in realtà, **due lezioni in una**: "una lezione di documentazione, *teaching*, e una lezione di design e *tools*". "Dovremmo riuscire a fare tutte e due, realisticamente. Scopriremo a breve." Ci riesce, e la giornata si chiude perfino con un esercizio dal vivo. La struttura della lezione, e quindi del capitolo, è questa:

1. le questioni pratiche in apertura: un problema di impostazione pagina nel task di Pokémon, che cosa succede a un task **non letto** e come lo si **riconsegna**, il limite dei cinque errori grammaticali (sezione 9.1);
2. la **libertà** che non esiste e l'**agency** che invece si progetta (sezione 9.2);
3. il **paradosso della scelta**: perché troppe scelte rendono infelici, e che cosa se ne ricava come designer (sezioni 9.3 e 9.4);
4. la **Theory of Planned Behavior**: come i giocatori prendono le loro decisioni, con le sue sei componenti, le barriere e una formula per stimare un comportamento (sezioni 9.5–9.7);
5. il **feedback** che rinforza i comportamenti e la **situational awareness** da cui la teoria proviene (sezione 9.8);
6. dopo la pausa, la documentazione: il **compendium** (o *design bible*), i **pilastri** di *God of War*, il documento di **vision**, la **team vision** e le domande che chiudono la pre-produzione (sezioni 9.9–9.14);
7. l'esercizio finale: i **core elements di Frogger** scritti dal vivo da uno studente e poi riformattati dal docente (sezioni 9.15 e 9.16).

9.1 In apertura: impaginazione, riconsegne, errori

9.1.1 Prima di cominciare

La registrazione parte sul solito rito delle lezioni ibride: il computer è nuovo, le postazioni sono nuove, la webcam ha "questo filtro molto triste, molto GTA 4", e per un paio di minuti il docente non sente chi è

collegato da casa (gli studenti remoti, tra loro, si sentono benissimo). Risolto il problema, la solita richiesta: "attivate le webcam, qualora voi non foste nel mezzo dell'apocalisse zombie".

9.1.2 Un problema di impostazione pagina

Prima che la lezione cominci, uno studente segnala un problema sul secondo task consegnato, quello su Pokémon, che il docente non ha letto proprio perché "i parametri erano sballati". Lo studente spiega di aver impostato i margini da *File* → *Impostazione pagina*, inserendo i valori richiesti; ma su quel documento il risultato è stato sbagliato, e non sapeva come correggerlo.

Il docente condivide lo schermo e individua il colpevole: la "**spazzatura** all'interno del documento", cioè un **rientro** residuo (un'indentazione rimasta impostata sul testo) che falsava l'impaginazione. La prova del nove è semplice: in un **documento nuovo**, con la stessa procedura, i margini risultano giusti. Il problema, quindi, non era nella procedura, ma in ciò che il vecchio documento si portava dietro.

Sperimentate, e chiedete. Il commento del docente vale in generale: "provate, sperimentatevi, non fermatevi al primo risultato possibile". E se ci sono dubbi, **chiedere** resta sempre una possibilità valida. Con una precisazione onesta: il docente non avrà sempre la risposta, e a volte, pur avendola, non potrà darla (perché trovarla da soli fa parte dell'esercizio).

9.1.3 Che cosa succede a un task non letto

La seconda domanda, che anche Francesco aveva posto in privato, è quella che interessa a tutti: **che cosa succede se un task non viene letto?** Le regole erano state date alla prima lezione (sezione 1.4); il docente le riprende e le precisa.

- Un task **non letto** è perduto nel conteggio finale dei task.
- Ma non è perduto **per sempre**: lo si può voler **recuperare**. Lo stesso vale per un task che non si è riusciti a consegnare entro la scadenza.
- Il docente **accetta le riconsegne**. Non saranno ideali come consegna finale, cioè non avranno lo stesso peso pieno di una consegna puntuale nel computo finale, ma vengono contate.

Quante volte? Una studentessa chiede quante volte si possa ricevere un "non leggo" e poi riconsegnare, perché nelle consegne "mi perdo veramente tante cose". La risposta è un numero preciso: fino al **20% dei task totali**. Resta poi una garanzia aggiuntiva: per il passaggio all'anno successivo, un task *consegnato e lavorato* ma non letto viene comunque considerato diversamente da un task *mai consegnato*. Il caso citato è proprio quello della studentessa con il production dictionary: "hai lavorato, però io non te l'ho letto", e questo viene tenuto in considerazione. E a maggior ragione se poi lo si rifà e lo si riconsegna.

9.1.4 Come si riconsegna

La procedura, mostrata aprendo la cartella dei task revisionati e prendendo "a caso" come esempio il task di Alberto, è la seguente.

1. **Non modificate il vecchio documento.** Non si risolvono i commenti, non si corregge la vecchia task: il documento commentato resta com'è.
2. Se ne crea una **copia** o, "ancora meglio", si crea un **documento da zero**.
3. Lo si riempie includendo **tutto ciò che il docente ha commentato** (o non ha commentato) nel proprio task precedente, **più tutto ciò che ha commentato nei task dei colleghi**.
4. Lo si inserisce nella **cartella di riferimento** del task (gli Arcade nella cartella degli Arcade, il production dictionary nella sua), o creando una sottocartella con *cognome e nome*, come aveva fatto Valerio con più documenti, o comunque con un **nome comprensibile** e in modo ordinato.
5. Si **scrive al docente**, perché deve sapere che c'è una riconsegna. Poi si attendono di nuovo i commenti, che probabilmente non saranno rapidi come quelli di un task normale.

Il task si riconsegna con ciò che si sa *oggi*. È il punto più importante, e il docente lo sottolinea: chi riconsegna, per esempio, il task sui core elements, non viene letto "come se fosse fermo a quel punto lì". La riconsegna deve contenere **tutte le nozioni apprese anche dopo**, cioè tutto ciò che si è accumulato fino al momento della nuova consegna: i commenti propri, i commenti ai colleghi, "tutte le altre cose che ci siamo detti nel mentre". Ed è questo che viene valutato.

Non riconsegnate il giorno dopo. Da qui un invito esplicito: se un task non viene letto, **non va riconsegnato il giorno dopo**, correggendo solo le cose commentate. Bisogna prendersi del tempo per capire, ma anche per **leggere i task degli altri** e integrare ciò che si è imparato, "da lì a una settimana, due settimane, un mese". Il docente assume che nel task riconsegnato ci sia gran parte delle conoscenze apprese fra la lettura e la riconsegna.

9.1.5 Il limite dei cinque errori e la dislessia

L'ultimo annuncio riguarda chi ha problemi di **dislessia** o difficoltà associate. Ora che la classe ha visto "con mano" che cosa significa contare gli errori nei task (il registro delle revisioni è nel capitolo 1), il docente invita chiunque si trovi in questa situazione (ne aveva già parlato con una studentessa e con Mattia) a **scrivergli in privato**: il **limite dei cinque errori grammaticali** (sezione 3.9) può essere sollevato. Con due precisazioni.

- Il motivo del limite è il "mondo brutale là fuori". Nei documenti operativi dei task non dovrebbe succedere ("almeno a me non è capitato; spero non succeda mai"), ma ci sono casi in cui la persona di riferimento si aggrappa a qualsiasi cosa per non leggere. Cinque errori grammaticali sono certamente rilevanti in un **curriculum** o in un **portfolio**: "sappiatelo".
- Chi ritiene che sollevare il limite gli faccia bene, viene accontentato; ma è una possibilità per chi ne ha bisogno, "non è che adesso avanzate tutti questa possibilità".

Il docente annuncia infine che nell'ultima ora si farà "questa cosa di cui magari qualcuno si è dimenticato; tuttavia io non dimentico, come gli elefanti": un volontario prenderà il posto del docente al computer e, "con il potere della forza e dell'amicizia", creerà insieme alla classe un nuovo documento di **core elements**, applicando tutto ciò che si è imparato finora sulla formattazione (era l'impegno preso alla fine del capitolo 6, sezione 6.21). È l'esercizio delle sezioni 9.15 e 9.16.

9.2 La libertà non esiste: esiste l'agency

9.2.1 Un concetto che ritorna da Bura

Il primo argomento di design è la **libertà**. Non è un concetto nuovo: compariva già nel framework di Stéphane Bura, nella colonna *Freedom* (sezione 2.9, tabella 2.1). Il docente ricorda di aver detto, presentandolo, che quel framework è complesso e che alcuni dei suoi aspetti sarebbero stati "rivisti in altre cose, nel corso del corso" ("e scusate il gioco di parole"). La libertà è uno di questi.

9.2.2 Ciò che le persone cercano davvero

"Ogni giocatore, chiunque, vuole libertà." Il docente scherza sul fatto che la battuta sulla libertà e sull'America "ormai si spreca" e, rimandando a una GIF mostrata prima, dichiara subito che l'affermazione è **falsa**. E non vale solo per i giocatori: vale per qualsiasi persona.

Ciò che le persone cercano non è la libertà in sé, "concetto molto esagerato, molto ampio e anche molto rattristante" (si capirà perché nella sezione 9.4). Ciò che le persone cercano è l'**agency**:

> la capacità, la condizione, lo stato di **esercitare potere entro alcuni limiti**.

I limiti sono parte della definizione. È anche il motivo per cui la definizione di gioco del docente ragiona sempre **entro i limiti del sistema**, entro le regole del sistema (sezione 2.10). Nel caso dei videogiochi, e dei giochi in generale, quei limiti li stabilisce il **designer**.

Una regola di vocabolario. Da qui in avanti, nel corso, "libertà" come termine **non esiste**. "Non è così grave come *divertente*" (la parola proibita, sezione 1.4.4), "però avete capito": il termine corretto, ciò che si va a cercare, è **agency**.

9.2.3 Esempi di agency

La scelta dello starter in Pokémon è "l'esempio ideale, proprio l'esempio perfetto". Ed è il motivo per cui gli starter sono **tre**.

La scelta della build iniziale in *Dark Souls*, all'inizio del gioco, dove le scelte sono estremamente limitate.

Le quattro mosse ancora in Pokémon, "che avete analizzato di recente": in battaglia si possono scegliere **quattro** mosse. Non 27, non 10: quattro.

I sistemi di combattimento molti si basano su pochi elementi: schivata; attacco (leggero, pesante, la combo da tre colpi, l'attacco caricato); parata o *parry*. Bene o male: offesa, schivata, difesa.

9.2.4 E il contrario: togliere agency

Vale anche il contrario: tutto ciò che limita la possibilità di agire e di scegliere del giocatore **toglie agency**.

- Lo **stun lock**:[1] sto combattendo e vengo trasformato in una pecorella che non può fare nulla, se non muoversi e saltellare in giro.
- In *League of Legends*, subire **CC** (*crowd control*): rimanere fermi senza poter usare nessuna abilità né fare nessun movimento, "sono lì a prendere pizze".

Sono esempi di **agency assente** o **limitata**, tolta al giocatore. Ciò che interessa al designer, però, è il discorso positivo: come si dà agency.

Approfondimento: la sensazione di libertà. Jesse Schell dedica alla libertà una lente intera, la *Lens of Freedom* (lente n. 79), e arriva a una conclusione molto vicina a quella del docente. Non bisogna sempre dare al giocatore *vera* libertà: basta dargli la **sensazione** di libertà. Se un designer abile riesce a far sentire libero un giocatore che in realtà ha pochissime scelte, si ottiene il meglio dei due mondi: il giocatore ha l'esperienza dell'agency, e il designer controlla l'esperienza in modo economico. Le domande della lente: quando i giocatori hanno libertà d'azione, e si sentono liberi in quei momenti? Quando sono vincolati, e si sentono vincolati? Dove li si potrebbe far sentire più liberi? E dove, al contrario, sono **sopraffatti da troppa libertà**?[2]

9.3 Troppe scelte: il carretto, i cereali, gli skill tree

9.3.1 Il carretto dei dolci

Per rendere il discorso concreto, il docente propone un esperimento mentale ("e peggio per voi che siete a casa"): qui fuori c'è un carrettino con **cioccolato gratis**, **gelato gratis** e **torta di mele gratis**. ("Non è vero, per chi se lo stesse chiedendo: non c'è il carrettino, purtroppo.") Uno a testa. Chi vuole il cioccolato? Chi il gelato? Chi la torta?

Le mani si alzano: il gelato parte basso e poi sale, "due persone, tre persone, quattro persone, credibile". Federica non vuole niente: non le piacciono molto i dolci. "Si può non scegliere?" Sì, e il docente nota che **non scegliere è una quarta possibilità** che le concede. ("Uno a testa, quindi la torta di Federica me la prendo io, e mangerò due torte di mele.") La scelta è stata facile: ognuno è "caduto sulla sua preferenza", o non preferenza.

[1] Nella trascrizione "di Stan Lock"; dal contesto (il giocatore bloccato in combattimento) si tratta quasi certamente dello *stun lock*, la condizione in cui un personaggio viene stordito ripetutamente senza poter reagire.

[2] Jesse Schell, *The Art of Game Design: A Book of Lenses*, terza edizione, cap. 18 (*Indirect Control*).

9.3.2 I cereali

Seconda domanda: "scegliete una marca di cereali a vostro piacimento". Il docente mostra un'immagine con molte scatole di cereali e mette fretta: "Francesco, veloce, fai una scelta". Francesco sceglie dei *Pops*; Antonino un altro tipo.[3] "E già vi ho dato un'altra limitazione, che è il **tempo**."

Poi la provocazione: "Francesco, sei sicuro della tua scelta? Anche se qui c'erano i cereali di **Gizmo**, dei *Gremlins*? E quelli di **Mr. T**? Non ti sarebbero piaciuti?" Nessuno aveva visto i cereali dei Gremlins, "nascosti" tra gli altri (il docente confessa un debole per Gizmo). Il punto è questo: anche questa scelta è più o meno limitata, ma **molto maggiore** rispetto alle tre del carretto. E con più opzioni aumenta il rischio di **non vedere** l'opzione che si sarebbe preferita, e di dubitare della propria scelta dopo averla fatta.

9.3.3 Gli skill tree

Il concetto "si può espandere molto". È la differenza tra uno skill tree contenuto e uno sterminato. Il docente mostra prima uno skill tree "abbastanza contenuto" (il riferimento è a *Star Wars Jedi: Fallen Order*)[4] e poi quello di ***Path of Exile***, "il Diablo-like", famoso per il suo albero passivo gigantesco; poi un terzo, "intermedio", sicuramente più piccolo.[5] In quasi tutti ci sono delle limitazioni: la classe scelta all'inizio indirizza già verso un ramo o un altro. Ma diventa chiaramente complesso **leggere** un albero del genere, o anche solo muoversi nella decisione.

Il rischio di pentirsi. Il rischio è di investire punti e risorse in un ramo e **pentirsene** dopo: perché la scelta ottimale era altrove, perché il proprio stile di gioco è cambiato, o perché nel frattempo si sono ottenute più informazioni sul gioco, sulle meccaniche, sui sistemi. (Uno studente ricorda che in Path of Exile esiste un meccanismo per tornare indietro; il docente non lo ricorda con certezza, e comunque il problema è a monte.)

Il problema è in partenza. Al netto di qualsiasi meccanica di rimborso, il problema è l'impatto: "immaginate che io entro in Path of Exile, apro lo skill tree e mi trovo davanti questa roba". Lo stesso valeva per ***Salt and Sanctuary***,[6] giocato dal docente in co-op locale: lui voleva fare il mago, l'amico il paladino, e all'inizio era complesso capire quale ramo ottimizzare, per di più ragionando su due giocatori. Nel peggiore dei casi si rimane **impietriti** davanti alle possibilità: bloccati.

9.3.4 Il cervello non è fatto per i grandi numeri

Il nostro cervello non è ottimizzato per ragionare su grandi numeri, ed è lo stesso motivo per cui non sappiamo leggere correttamente la probabilità statistica. Il docente cita un bias che si vedrà al secondo anno, che chiama "effetto Monte Carlo":[7] vado al casinò, alla roulette; oggi esce il 4 rosso, domani esce il 4 rosso, dopodomani esce il 4 rosso. Il quarto giorno "probabilmente non uscirà più", pensiamo: la probabilità che esca di nuovo ci sembra più bassa. "Ma non è che funzioni proprio così, la statistica": ogni giro è indipendente, e la probabilità resta la stessa. È un **bias cognitivo**, e mostra che il cervello non è così pronto a ragionare logicamente.

9.3.5 La paura di scegliere, e il rimorso

Mettere un giocatore davanti a un'enorme quantità di scelte, quindi, non solo può tenerlo **fermo**: genera

- la **paura di scegliere**, cioè di scegliere in maniera sbagliata (più o meno sentita a seconda dell'individuo, ma esiste);

[3] Nella trascrizione la scelta di Antonino compare come "Titanino", non riconoscibile.

[4] Nella trascrizione "Following Order": quasi certamente *Star Wars Jedi: Fallen Order* (2019), che ha uno skill tree relativamente piccolo. Il passaggio è confuso e in parte contraddittorio ("No, no, non Star Wars"), quindi l'attribuzione resta un'ipotesi.

[5] Il gioco dello skill tree intermedio compare nella trascrizione come "Paolo Zolaico", non riconoscibile.

[6] Nella trascrizione "Salt and Sentry".

[7] Il nome corretto è **fallacia di Monte Carlo**, nota anche come *fallacia del giocatore* (*gambler's fallacy*): la convinzione che eventi indipendenti passati influenzino la probabilità di quelli futuri. Prende il nome da un episodio del 1913 al casinò di Monte Carlo, in cui il nero uscì alla roulette ventisei volte di fila.

- il **rimorso** di aver scelto sbagliato, un domani.

9.3.6 Rimborsi e scelte permanenti

Una meccanica che può rimediare, più o meno, al problema è quella che suggeriva Francesco: **rimborsare**, permettere di tornare sui propri passi (il *respec*). Ma questo apre un altro problema, di cui si parlerà in una lezione successiva: **se una scelta non è permanente, il giocatore la percepisce come non così importante**, non così pesante. Le scelte narrative memorabili che ciascuno ricorda, in qualsiasi gioco, generalmente erano **irreversibili**: scegliere tra far vivere il personaggio X o il personaggio Y, o scegliere una classe che non si può più cambiare e che cambia tutto il gameplay dall'inizio alla fine.

Ci sono quindi una serie di rischi nel dare "così tanta libertà" al giocatore; anzi, di libertà non si parla: si ragiona sempre dentro alcuni limiti, che sono dati dal designer.

9.4 Il paradosso della scelta

9.4.1 Più scelte, più infelicità

Questo fenomeno ha un nome: il **paradosso della scelta**. Più scelte si danno a una persona (a un giocatore, ma in generale a chiunque), più c'è il rischio che quella persona si senta, "sto esagerando, sto astraendo", **infelice**. È un concetto studiato molte volte, diventato particolarmente visibile nel panorama americano.[8]

Il docente legge alcune citazioni dal libro ***The Paradox of Choice*** (*Il paradosso della scelta*) dello psicologo americano **Barry Schwartz**:[9]

> Autonomia e libertà di scelta sono critiche per il nostro benessere, e la scelta è critica per la libertà e l'autonomia. Eppure, sebbene gli americani moderni abbiano più scelta di qualsiasi altro gruppo di persone prima di loro, e quindi, presumibilmente, più libertà e autonomia, non sembriamo trarne beneficio psicologico.

Sulla prima parte "siamo tutti d'accordo": gli esseri umani tendono verso la libertà, e non a caso il sistema giudiziario punisce le persone con il carcere, cioè **togliendo libertà**. Il problema è la seconda parte: una **grande** libertà di scelta può provocare infelicità, perché ci si sente **meno soddisfatti** del risultato di una scelta fatta tra molte opzioni di quanto lo si sarebbe stati con meno opzioni. Con tre opzioni, A, B e C, è più facile ottimizzare il risultato, e c'è meno possibilità che esistesse un percorso migliore, anche solo leggermente migliore, che non si è preso.

Le basse aspettative. Un'altra citazione dal libro:

> Il segreto della felicità sono le basse aspettative.

Gli atleti olimpici. E un altro risultato citato: tra gli atleti sul podio olimpico, i più felici sono l'**oro** e il **bronzo**.[10] Il bronzo è "saltato" dal quarto posto, che generalmente nessuno ricorda, al terzo, ed è stato premiato; il secondo, si suol dire, è "il primo degli ultimi". Tutto dipende dal termine di paragone, e quindi dalle aspettative.

Il risultato di tutto questo è sempre lo stesso: **bassa felicità**, o comunque **insoddisfazione** dell'utente.

9.4.2 Che cosa ce ne facciamo come designer

"Non parleremo sempre di videogiochi", ma la domanda è: come aiuta, questo concetto? Il paradosso della scelta è utile da tenere in considerazione ogni volta che si parla di agency e di **possibilità limitate date**

[8] Nella trascrizione "studiato ... dai comunisti americani": quasi certamente una storpiatura (forse "economisti" o "psicologi"); non è ricostruibile con certezza.

[9] Barry Schwartz, *The Paradox of Choice: Why More Is Less*, 2004. Il nome dell'autore non compare nella trascrizione, ma il titolo e le citazioni corrispondono.

[10] Il riferimento è allo studio di Victoria Medvec, Scott Madey e Thomas Gilovich (1995) sulle espressioni dei medagliati alle Olimpiadi di Barcellona del 1992, che trovò i medagliati di bronzo mediamente più soddisfatti di quelli d'argento.

al giocatore. Se lo si tiene presente, si può aumentare l'agency del giocatore e, soprattutto, la **qualità** dell'agency che gli si dà: **meno scelte, ma più profonde**.

E come si raggiunge una scelta più profonda? Con un design migliore: impiegando le risorse su **tre sistemi invece che su cento**, si può iterare molto di più, "perché sono tre e non cento". I vantaggi, che il docente elenca da una slide, stanno su due fronti (tabella 9.1).

Table 9.1: *Less is more*: i vantaggi di dare meno scelte, ma più profonde.

Per il giocatore	**Per il progetto**
Aumenta la **soddisfazione percepita**: meno blocco, meno paura di agire davanti a una quantità esagerata di scelte.	**Riduzione a monte** della grandezza del progetto: meno cose di cui preoccuparsi in produzione, prima e dopo.
Si riduce la **curva d'apprendimento**: poche scelte all'inizio, di più quando il giocatore ha sviluppato *mastery*.	Più **risorse** di produzione dedicate a **meno cose**: costo ridotto, più iterazione.
Decisioni **meaningful**: scelte meno numerose, quindi più dettagliate e percepite come più importanti.	**Potenzia la creatività**: ragionare con pochi sistemi porta a un risultato migliore dal punto di vista del design.
Scelte migliori: scegliendo in una gamma più ridotta, il giocatore è più soddisfatto.	Design più **bilanciato**: si bilanciano 3, 5, 10 elementi invece di 10 000.

Ridurre la curva d'apprendimento non vuol dire dare sempre poco. Il docente precisa: non significa che si debba sempre dare un numero limitato di scelte. Lo si può **incrementare nel tempo**: poche scelte all'inizio, di più quando il giocatore ha sviluppato *mastery* sui sistemi. L'esempio, "una chiacchiera da bar, che bisognerebbe analizzare meglio", è ***Ori and the Blind Forest***:[11] all'inizio dà pochi sistemi di locomozione e poi spinge la curva, che il docente corregge a voce da "molto ripida" a "**molto morbida**". La progressione di quel gioco, soprattutto sul fronte della mastery, è per il docente un punto di riferimento ideale per i platformer. Tra l'altro ha una meccanica che si basa sul colpire i **proiettili dei nemici per saltare più in alto**,[12] cosa che in molti altri giochi "sarebbe stata disastrosa".

Decisioni meaningful. Dare meno scelte permette al giocatore di percepire **in maniera più importante** le scelte che adotta, perché sono meno e quindi possono avere più dettaglio, più iterazione. Non è l'unico ingrediente per rendere una decisione meaningful, "non è l'unica discriminante": è un discorso che verrà ripreso più avanti nel corso.

Tre è un ottimo punto di partenza. Dal lato del progetto, il grande vantaggio è il costo ridotto, le risorse risparmiate; ma, di contro, il limite **potenzia la creatività**. Ragionare in maniera limitata, non dover produrre mille sistemi ma tre, quattro, cinque, permette di raggiungere un risultato migliore e sicuramente più **bilanciato**. "Tre è un ottimo punto di partenza: non a caso gli starter sono tre, in ogni gioco." (Una mano si alza; era solo "una grattata".)

Approfondimento: quante scelte dare. Schell, nel capitolo sul bilanciamento, cita un'osservazione di Michael Mateas: il numero di scelte che un giocatore cerca dipende dal numero di cose che **desidera** fare.

- se le scelte sono **più** dei desideri, il giocatore è **sopraffatto**;
- se le scelte sono **meno** dei desideri, il giocatore è **frustrato**;
- se scelte e desideri coincidono, il giocatore prova una sensazione di **libertà** e di appagamento.

[11] Nella trascrizione "Orient e Blind Forest".

[12] Il docente la descrive come un *parry*; nel gioco l'abilità si chiama *Bash*, e permette di aggrapparsi per un istante a un nemico o a un proiettile e di lanciarsi nella direzione opposta, rilanciando il proiettile.

Scegliere tra la strada di sinistra e quella di destra è interessante; scegliere tra trenta stradine laterali è opprimente. Altre volte, invece, il giocatore desidera moltissime opzioni (l'esempio di Schell è il negozio di vestiti dei *Sims*). Schell aggiunge due trappole speculari: offrire scelte **prive di significato** (cinquanta auto che si guidano tutte allo stesso modo equivalgono a nessuna scelta) e offrire scelte che **nessuno vorrebbe** (dieci armi di cui una è chiaramente migliore: una *strategia dominante*). Ne nasce la *Lens of Meaningful Choices* (lente n. 39): quali scelte chiedo al giocatore? Sono significative, e perché? Gli sto dando il numero giusto di scelte, e ne servirebbero di più per farlo sentire potente, o di meno per rendere il gioco più chiaro?[13]

Approfondimento: il vincolo come controllo indiretto. Nello stesso spirito, Schell elenca il **vincolo** (*constraint*) come primo metodo di controllo indiretto. "Scegli un colore" e "scegli un colore: rosso, blu o verde" danno entrambi libertà di scelta, ma il secondo riduce le risposte possibili da milioni a tre, e chi avrebbe comunque scelto uno dei tre non nota nemmeno la differenza. Il suo aneddoto ricorda il carretto dei dolci: lavorando in un negozio di caramelle con sessanta gusti, recitarli tutti spaventava i clienti, e chiedere "dimmi tu il gusto che vuoi" li bloccava; funzionava invece dire "abbiamo quasi tutti i gusti, ma i più richiesti sono questi sei". I clienti si sentivano liberi, ma avevano davanti poche opzioni chiare, e sceglievano.[14]

9.5 Come decidono i giocatori: la Theory of Planned Behavior

9.5.1 Un framework per leggere i comportamenti

Fin qui si è parlato di *less is more* e di agency. Ora si aggiunge un pezzo: **come i giocatori prendono le loro decisioni**. Il framework "arriva direttamente dal secondo anno" ed è un inserimento recente nel programma: è "un'altra delle cose che finirà nella vostra cintura". Non lo si deve usare per forza, ma comincia già a dare informazioni su come i giocatori scelgono.

Ci si può fare design, come con quasi tutti i framework. Ma l'opinione del docente è che il suo vero uso sia l'**analisi**, e in particolare l'**analisi da playtest**: leggere come i giocatori hanno fatto, o "ancora meglio, **non** hanno fatto", alcune cose che il designer aveva previsto nel gioco, e perché. Serve soprattutto a **spiegare perché alcuni comportamenti vengono adottati e altri no**: brilla nella *data analysis*.

Il framework è la **Theory of Planned Behavior** (TPB),[15] che il docente presenta come una **sezione** di una teoria psicologica molto più complessa, la **situational awareness** (sezione 9.8): è la parte in cui il giocatore **adotta una scelta**. Ha anche elementi in comune con l'**atomo di gioco** di Raph Koster (sezione 4.1).

9.5.2 La struttura

La visualizzazione, semplificata, è quella della figura 9.1. Tre fattori, l'**attitudine**, le **norme soggettive** e il **perceived behavioral control** (PBC), portano all'**intenzione** di compiere un'azione. Tra l'intenzione e il **comportamento** effettivo, poi, si mettono di mezzo le **barriere**.

9.5.3 Intenzione e comportamento: l'orfanotrofio dei gattini

La distinzione tra intenzione e comportamento il docente la spiega con un esempio volutamente estremo: un **orfanotrofio di gattini** ("non so se esiste") **in fiamme**. Davanti alla porta, chiunque potrebbe avere l'**intenzione** di lanciarsi tra le fiamme e salvare più gattini possibili, "perché non sono un mostro". Però c'è un **ostacolo** enorme che ne impedisce il comportamento: le fiamme "fanno parecchio male" ("assumo: non sono mai stato dentro un incendio"). Il desiderio può essere elevatissimo, ma l'azione pratica non avviene. È esattamente la differenza tra intenzione e comportamento, e l'esempio tornerà per ciascun fattore.

[13] Schell, *The Art of Game Design*, cap. 13.

[14] Schell, *The Art of Game Design*, cap. 18.

[15] Nella trascrizione il nome è storpiato in molti modi ("Tier of Planned Behavior", "Team of Planned Behavior", "Tierra of Planned Behavior"). La teoria del comportamento pianificato è stata formulata dallo psicologo sociale Icek Ajzen (1985–1991), come estensione della teoria dell'azione ragionata. La suddivisione di ciascun fattore in due sotto-componenti, le barriere e la formula numerica delle sezioni 9.5.7 e 9.6 sono una versione semplificata e adattata ai giochi presentata a lezione, non la formulazione originale di Ajzen.

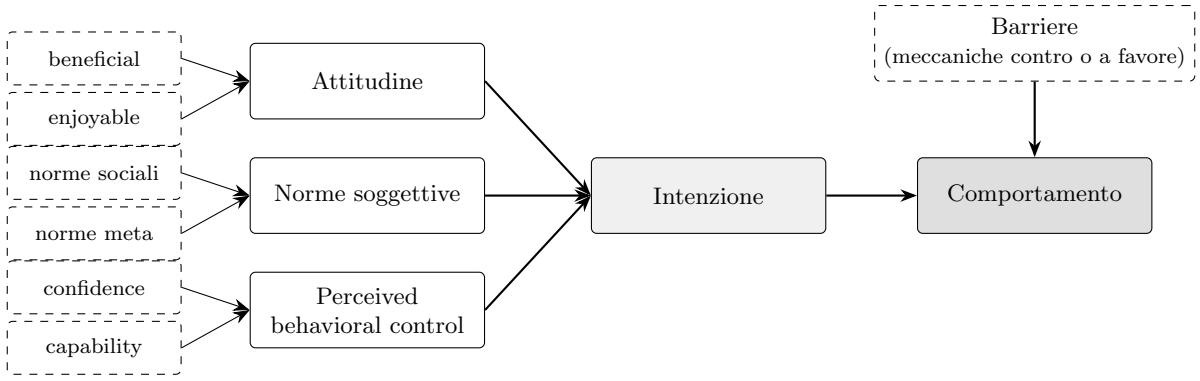


Figure 9.1: La Theory of Planned Behavior come presentata a lezione: tre fattori, ciascuno con due sotto-componenti, portano all'intenzione; le barriere si frappongono tra l'intenzione e il comportamento.

9.5.4 L'attitudine

L'**attitudine** è la tendenza di una persona a **voler fare** quella cosa: è legata all'agency personale rispetto alla decisione. Ha due componenti, che spesso avvengono contemporaneamente.

Beneficial quanto l'azione è **benefica** per la persona: se porta una *reward*, un *outcome* positivo, per semplicità esplicito. Il giocatore viene premiato, ottiene un guadagno (monetario, o qualsiasi altra cosa).

Enjoyable quanto l'azione è **piacevole** di per sé, cioè quanto è vicina alla **sensibilità personale**. Questo dipende da giocatore a giocatore: tipi diversi di giocatori hanno motivazioni diverse. Il docente rimanda alla lista delle emozioni e alle motivazioni di **Quantic Foundry** (sezione 5.9): per lui, per esempio, potrebbe essere più piacevole cercare la *discovery* che la *challenge*; per un altro la socializzazione o la personalizzazione più dell'*excitement*: "voglio giocare con i miei amici, parlare con altre persone, giocare a party game, piuttosto che far esplodere cose" o "piantare una pallottola in testa a un'altra persona". Dipende dal **target audience** e, ancora più nello specifico, da un **segmento** del target audience; se ne riparlerà più avanti.

9.5.5 Le norme soggettive

Se l'attitudine riguarda la tendenza del singolo, le **norme soggettive** riguardano il **singolo inserito in un contesto**. Anche qui due componenti.

Le norme sociali: che cosa ne pensa la mia cerchia. La prima domanda è: che cosa pensa la mia cerchia sociale di quello che farei? Il docente fa un primo esempio sul **fumo**, chiedendo a una studentessa fumatrice (con il dovuto tatto: "dimmelo, se fosse un tasto dolente") che cosa ne pensi la sua famiglia. Il caso reale non è l'esempio migliore ("mia madre ha fumato anche con me incinta"), ma il principio è chiaro: tutti sanno che il fumo fa male, e una persona può trovare il fumo estremamente piacevole (**attitudine elevata**) e avere una famiglia che lo vede come un tabù, o come qualcosa che non dovrebbe fare (**norma sociale bassa**).

I colori di Magic. Un esempio "forse migliore, e più legato" è *Magic: The Gathering*. Nessuno in classe ci gioca, quindi il docente riassume i **cinque colori** e i loro stili di gioco, con un parallelo con le classi di *Hearthstone*[16] (tabella 9.2).

[16]Nella trascrizione "Airstone". Le corrispondenze tra colori di Magic e classi di Hearthstone sono un'analogia del docente, riportata in modo confuso ("non è lo sciamano... è necromante... sì, anche per esempio lo sciamano è il verde"); nella tabella si riportano quelle comprensibili. Il docente premette di non essere un grande giocatore di Magic.

[17]La trascrizione accenna al "necromante". Nell'analogia più diffusa il nero corrisponde piuttosto allo stregone (*warlock*), che sacrifica punti vita per pescare carte.

Table 9.2: I colori di Magic come riassunti a lezione, con l'analogia del docente con Hearthstone.

Colore	**Stile di gioco**	**In Hearthstone**
Rosso	il colore "violento": fa *chip damage*, "va sempre faccia", cerca di attaccare il più possibile; dinamiche *aggro*	il cacciatore
Verde	grossi mostri, scalare il mana più velocemente degli altri (vantaggio di mana)	lo sciamano (secondo il docente)
Nero	enormi sacrifici di creature, carte o vita per ottenere vantaggio	non chiaro[17]
Bianco	tanti minion, orde di token, schiacciare l'avversario con i numeri	il paladino
Blu	giocare "agili", negare le giocate: è il colore del **counterspell**, che annulla la magia avversaria	il mago

Il punto è il **blu**. In Hearthstone c'è stato un periodo in cui il mago era particolarmente odiato; e in Magic, se si cerca su internet che cosa pensano le persone dei giocatori di blu, "penso che nemmeno i criminali di guerra vengano trattati così male". Le **norme sociali** rispetto al blu sono **molto basse**: se a me piace quello stile di gioco (**attitudine** alta), probabilmente non sarà apprezzato al tavolo in cui sto giocando un mono-blu. (Gli archetipi, poi, si possono mescolare.)

Le norme meta: che cosa dicono i dati. La seconda componente è il **meta**. Se le norme sociali si basano su tendenze, su ciò che le persone vicine (la famiglia, gli amici) percepiscono "a pelle", cioè su dati **qualitativi**, il meta è legato a dati **quantitativi**: c'è una **riprova**, un'informazione netta. Il blu non è necessariamente più forte degli altri colori: è semplicemente più **frustrante** giocarci contro, perché nega le giocate. Il meta determina invece se uno specifico tipo di mazzo, o una specifica carta ("parlando in termini di TCG"), è **statisticamente** più forte, cioè porta più probabilmente alla vittoria. "C'è informazione provata che prendere Charmander sia meglio che prendere gli altri? Esiste questo tipo di informazione, o no?" Questa, fondamentalmente, è la differenza tra le due norme.

9.5.6 Il perceived behavioral control

Il terzo fattore, il **perceived behavioral control** (PBC, controllo comportamentale percepito), si compone di **confidence** e **capability**.

Confidence la **fiducia** che il singolo ha di riuscire a compiere l'azione: quanto mi sento capace di farla, magari perché l'ho già fatta molte volte, o perché per me è semplice. Il docente, che apprezza particolarmente i giochi di carte e strategici, si sente più sicuro a costruire un mazzo che a costruire una build in uno sparatutto ("non dico che mi reputi un maestro"). Nell'esempio dei gattini: non si sente affatto sicuro di correre in un mare di fiamme.

Capability la **capacità** effettiva, cioè la prova pratica dell'azione: è legata alla **realtà**, alla capacità di fare qualcosa in modo **riproducibile**. Nell'esempio dei gattini è ancora più bassa della confidence: "non sono un atleta, non sono un pompiere, non ho un fisico particolarmente eccelso", e non ha nemmeno i vestiti ignifughi adatti.

Confidence e capability possono divergere. L'esempio più utile è il **parry**. Nei sistemi di combattimento che lo includono, il docente generalmente non lo usa: tra parry e schivata, schiva, perché non si sente **sicuro** di beccare il *frame* giusto (confidence bassa). Ma la sua **capacità** di fare parry è probabilmente molto più alta della media dei giocatori, e anche della sua stessa confidence, "perché sono un giocatore, ho dei riflessi bene o male sviluppati" (capability alta). La confidence è una **percezione** del singolo; la capability è la **capacità pratica** di fare qualcosa.

9.5.7 Le barriere

Tutto questo si porta dietro delle **barriere**. La barriera sta dal lato della **limitazione**, quindi dal lato dell'agency: **quante meccaniche remano a mio favore, e quante contro di me**.

- Se voglio catturare un certo Pokémon e quel Pokémon compare nell'erba con l'**1%** di probabilità, quella è una **barriera meccanica** maggiore nei miei confronti.

- Al contrario, una meccanica che lo rende più facile da trovare **riduce** la barriera: è come se il gioco "mi stesse facendo un favore", spingendomi a compiere quell'azione.[18]

La domanda è quindi doppia: quante meccaniche ci sono per **suggerire** e spingere un'azione, che poi il giocatore compirà o meno, e quante invece gli **mettono i bastoni fra le ruote**? Un'arma che costa tanto, un'arma bloccata dietro molti achievement, ma anche un'arma con **tante combo** e quindi tante dinamiche di gioco da padroneggiare.

È la differenza, in *Monster Hunter*, tra la **spada lunga** e lo **spadone**: con lo spadone "devi solo fare *bonk*", cioè picchiare, e ogni tanto caricare il colpo, caricarlo ancora un po' e scaricarlo in testa al mostro. La spada lunga chiede molto di più: la barriera, per chi si avvicina, è più alta.

9.6 Mettere a terra la TPB: due modi di usarla

"Ora forse vi potrete chiedere: ma a che serve questa roba? Come la mettiamo a terra?" I modi sono due.

9.6.1 Ad alto livello: riempire lo schema

Il primo, valido sia per il design sia per l'analisi, è semplicemente **prendere lo schema e riempirlo**, ragionando sull'azione che il giocatore va a fare. Lo si fa **per ogni azione**, data una premessa. Scegliere tra Charmander, Bulbasaur e Squirtle ("facciamone di più belli: Treecko, Mudkip e Torchic") presuppone **tre** schemi TPB diversi, uno per starter. Gli schemi si riempiono con considerazioni del designer o, meglio, con **citazioni dell'utenza** sulle singole parti.

9.6.2 Con i numeri: la formula semplificata

Il secondo modo è **calcolare**. Esiste una formula, legata alla situational awareness, per stimare matematicamente la probabilità di un comportamento. Il problema è che la formula completa può essere estremamente complessa: il docente ne cerca una online, ne trova una "già semplificata", poi una con le sommatorie ("e già inizia a diventare un po' più complessa"), e infine presenta la sua, "**estremamente semplificata**". La si può complicare a piacere, ma non è necessario.

I valori. A ciascuna sotto-componente si assegna un valore tra -10 e $+10$ (tabella 9.3). In realtà questi valori andrebbero ottenuti **dall'utenza**: non nel senso che l'utenza li dà già pronti, ma nel senso che si fanno domande all'utenza per poi arrivarci.

Confidence e capability si possono valutare **separatamente** (un valore da -10 a $+10$ ciascuna) oppure **accorpate** in un unico valore da -10 a $+10$: "è indifferente". Le barriere, per ora, si tengono da parte: entrano solo nell'ultimo passaggio.

I pesi. Dati i valori, si scelgono **tre pesi**: uno per l'attitudine (w_A), uno per le norme soggettive (w_N), uno per il PBC (w_P). Generalmente valgono **1**; si può "stressare" un peso fino a un massimo di **2** quando quella categoria è particolarmente impattante nel gioco. Se il gioco è multiplayer, o è un e-sport, le norme soggettive contano moltissimo: 1,5, oppure 2 per *League of Legends*, ma anche per Magic.

[18] In questo punto la trascrizione ripete in loop "È una barriera meccanica" per decine di volte, perdendo l'esempio di facilitatore che il docente stava facendo. Si riporta solo il principio, che è ricostruibile dal seguito.

Table 9.3: La scala dei valori della formula semplificata.

Fattore	**Componente**	-10	$+10$
Attitudine	beneficial (B)	danneggia il giocatore	altamente benefico
	enjoyable (E)	molto spiacevole	molto piacevole
Norme soggettive	sociali (S)	molto scoraggiato	molto incoraggiato
	meta (M)	nessuna prova empirica	la strategia migliore ("tier 0" nel meta attuale)
PBC	confidence (C)	nessuna fiducia	piena fiducia
	capability (K)	nessuna capacità	piena capacità
Barriere	(R)	molte barriere	molti facilitatori

L'intenzione. L'intenzione si calcola così:

$$I = w_A\,(B + E) + w_N\,(S + M) + w_P\,PBC,$$

dove PBC è il valore unico (se confidence e capability sono state accorpate) oppure la somma $C + K$ (se sono state valutate separatamente).

"Seguite il ragionamento?" Con tutti i pesi a 1, l'intenzione va da -50 **a** $+50$ se il PBC è accorpato (cinque termini da ± 10), e da -60 **a** $+60$ se è diviso in due (sei termini). L'intenzione esprime **l'idea di fare qualcosa**.

Il comportamento. Il comportamento si calcola aggiungendo all'intenzione le barriere, con un **quarto peso**:

$$Comp = I + w_R\,R, \qquad w_R = 2 \cdot \frac{w_A + w_N + w_P}{3}.$$

Il peso delle barriere è il **doppio della media** degli altri tre pesi. Perché il doppio? Perché **le barriere vengono sempre percepite come più pesanti di quanto non siano in realtà**.

Con i pesi a 1 e il PBC accorpato, $w_R = 2$ e il risultato è compreso tra -70, "una cosa che un giocatore non farà mai", e $+70$, "una cosa che il giocatore tendenzialmente farà, nella stragrande maggioranza dei casi".

Un esempio di calcolo. A lezione non si fa un calcolo insieme ("prenderemmo molto tempo"). A puro titolo illustrativo, con numeri **inventati** per questo capitolo e non tratti dalla lezione, si immagini un giocatore che valuta se usare il parry in un gioco d'azione, con tutti i pesi a 1 e il PBC diviso in due: $B = +6$ (il parry è premiato), $E = +4$, $S = +2$, $M = +7$ (il meta lo considera la scelta migliore), $C = -6$ (non si sente sicuro), $K = +5$ (in realtà è capace), $R = -3$ (la finestra di frame è stretta). Allora

$$I = (6 + 4) + (2 + 7) + (-6 + 5) = 18, \qquad Comp = 18 + 2 \cdot (-3) = 12,$$

su una scala da -80 a $+80$: un comportamento leggermente probabile, in cui pesano soprattutto la confidence bassa e la barriera, raddoppiata.

Table 9.4: Gli intervalli del risultato con tutti i pesi a 1.

	PBC accorpato	**PBC diviso**
Intenzione I	da -50 a $+50$	da -60 a $+60$
Barriere $w_R R$ (con $w_R = 2$)	da -20 a $+20$	da -20 a $+20$
Comportamento	da -70 a $+70$	da -80 a $+80$

9.7 Un esempio reale: il meta di Valorant

9.7.1 La scelta degli agenti

Il docente mostra invece un esempio già fatto, un'analisi sul **meta di *Valorant***, "del 2020, dovrebbe essere". Usa un'altra formula, in cui il PBC include insieme la confidence e le barriere. I pesi scelti erano:[19]

- circa **1,2** per l'attitudine;
- **1,9** per le norme soggettive, "perché è un e-sport";
- **1,5** per il PBC, perché non tutti gli agenti sono disponibili dall'inizio: alcuni sono suggeriti al giocatore e subito utilizzabili, altri vanno comprati o sbloccati.

Questi valori derivano da un **misto di scelte del designer e di dati sull'utenza**. Per ridurre lo scarto tra la stima della TPB e la realtà bisogna capire **chi sta giocando il gioco**: chi è, che cosa vuole. Servono le **demografiche**, "niente di più, niente di meno".

I risultati confermavano i dati: il pick più probabile, all'epoca, era **Omen**, il secondo **Cypher**, il terzo **Jett**.[20]

9.7.2 Il survey, e una lezione sulle risposte aperte

I dati demografici venivano da un **survey**. Il docente ne approfitta per anticipare l'argomento della lezione successiva ("lunedì": survey, poll, come ottenere e dare feedback, "dato che l'argomento ormai è bello caldo") e per ripetere un principio: anche nei sondaggi bisogna dare **agency oculata**, cioè **meno libertà possibile** a chi risponde.

Nel suo survey, la domanda sul genere aveva le opzioni *male*, *female*, *do not want to answer* e una **risposta aperta**. E nella risposta aperta sono arrivate, puntuali, le risposte-scherzo: il celebre meme dell'"elicottero d'attacco" (con tanto di modello e anno di produzione), e varianti più colorite. "Che cosa devo fare? Unirle insieme? L'anno di produzione dell'elicottero determina generi diversi?" È il problema della risposta aperta: aumenta il **lavoro dopo**. Nel caso del genere, chiarisce il docente, la risposta aperta è comunque **necessaria**; e siccome le risposte diverse da *male* e *female* erano poche rispetto al totale (il docente ricorda approssimativamente 600 risposte), gestirle non gli è costato molto. Ma l'obiettivo resta sempre **ridurre il carico di lavoro futuro** e **accorpare** quanti più dati possibile per fare i grafici.

Leggere i dati con giudizio. I grafici mostrati (in parte censurati) dicevano anche altro. I dati venivano da **Reddit**, quindi da una **nicchia** dell'utenza. Non è plausibile che la prima demografica dei giocatori di Valorant nel mondo sia l'America ("probabilmente è l'Asia"), ma il survey è finito su Reddit, e l'utenza di Reddit è principalmente americana; anche il 50% di rispondenti bianchi caucasici riflette questo. Se si includesse tutto il resto del target audience, probabilmente quei dati verrebbero "schiacciati". La fascia d'età di riferimento era, prevedibilmente, **18–24**. Chi raccoglie i dati deve sempre chiedersi **da dove vengono**.

9.8 Il feedback, e da dove viene la TPB

9.8.1 Senza feedback non c'è comportamento ripetuto

Per concludere il discorso: finora si è considerato il comportamento "non dico nel vuoto", ma come una parte di framework a sé. Per capire quanto è positivo o negativo, e soprattutto per **rinforzare** un comportamento e creare una certa **routine**, servono i **feedback**. Lo si era detto anche parlando dell'atomo di Koster (sezione 4.1): ripetere il comportamento e dare al giocatore un **segnale chiaro**, positivo o negativo, se ha fatto bene o se ha fatto male, è ciò che **rinforza o riduce** quel comportamento. È "una, se non la parte più importante" per un designer.

[19] La trascrizione è confusa ("Sono 1 e 2 per l'aptitude, 1,9 per la subject and norm, perché è un eSport, e 1,5 per i pesi"). La lettura più coerente con il resto della spiegazione è: 1,2 per l'attitudine, 1,9 per le norme soggettive, 1,5 per il PBC (che in questa formula include le barriere). I valori esatti non sono verificabili.

[20] Nella trascrizione "Jet". Il docente parla di "operatori"; in Valorant i personaggi giocabili si chiamano *agenti*.

Uno strumento potente, e rischioso. Il feedback può portare a tendenze "non dico scomode, ma rischiose", fino alla **ludopatia**: ci sono sistemi *gacha*[21] all'interno dei giochi che possono portare a questa tendenza. Se ne parlerà quando si tratteranno i **game loop** e il **condizionamento operante**.[22] Per ora basta ricordare che è uno strumento tanto importante quanto, in alcune applicazioni, rischioso.

Date feedback a qualsiasi cosa. "Più ne date, meglio è. Non potete sbagliare se date più feedback." L'unico problema è l'eccesso estremo: se si gettano a schermo mille VFX e mille SFX, il giocatore diventa "sordo e cieco". Ma lo si vede dal **playtest**, e nell'esperienza del docente non succede mai: "in tutti i casi che ho visto" il problema era che ne davamo **troppo pochi**.

- Se il giocatore non capisce di aver ottenuto una *reward*, non ha capito il vantaggio di quell'azione, e non la rifarà.
- Se non ha capito **perché** l'ha ottenuta, non può ripetere quel comportamento.

Il feedback va dato per ciò che il giocatore ha fatto **bene** e per ciò che ha fatto **male**.

Approfondimento: la lente del feedback. Schell dedica al feedback la *Lens of Feedback* (lente n. 63): il feedback che un giocatore riceve dal gioco è molte cose insieme, giudizio, ricompensa, istruzione, incoraggiamento e sfida. Le domande da porsi riguardano che cosa il giocatore ha bisogno di sapere in ogni momento, che cosa vuole sapere, che cosa il designer vuole che *provi*, e quale feedback può produrre quella sensazione. "Le esperienze senza feedback sono frustranti e confuse": l'esempio di Schell è il pulsante del semaforo pedonale che, senza una spia che si accenda, lascia chi lo preme nel dubbio di essere stato "ascoltato". E un'interfaccia ricca di feedback continuo e proporzionato è quella che chiama *juicy*.[23]

9.8.2 La situational awareness

La TPB, come anticipato, è presentata come una parte della **situational awareness** (SA, consapevolezza situazionale). È un metro di valutazione delle **decisioni umane** adottato di solito da piloti, militari e simili, in condizioni critiche: per valutare, ad esempio, se era corretto fare un atterraggio d'emergenza, o farlo in quel modo, o affrontare una situazione d'emergenza in un certo modo. Si fonda su tre livelli:

SA1 si ottengono e si percepiscono i **dati**;

SA2 si mettono in relazione i dati, li si **comprende** insieme;

SA3 si fa la **proiezione** degli stati futuri, "che è un po' l'intenzione che vedete anche qua".

Non è argomento del corso, né è necessaria alla comprensione della TPB.[24] Chi è interessato è invitato a fare ricerca, e a parlarne con il docente.

La pausa. "Siccome questo è un bel mattone": cinque minuti di pausa, anzi sette, anzi dieci ("esageriamo, tanto alla fine sono sempre di più"). Al rientro, la documentazione, "che so che è la vostra cosa preferita. Anche se non lo fosse, lo dico io."

9.9 Il compendium, o design bible

9.9.1 Due documenti di visione

Al rientro dalla pausa (con un controllo di chi è collegato: "Mattia c'è, e Valerio lo vedo vivo"), si entra nel dettaglio della documentazione specifica per la produzione. Dopo il production dictionary (sezione 4.3) si vedono due nuovi documenti: il **compendium** e la **vision**. Sono entrambi **documenti di visione**, secondo la distinzione del docente tra documenti di visione e documenti di produzione (sezione 5.2).

[21] Nella trascrizione "sistemi di caccia"; dal contesto (ludopatia) si tratta quasi certamente dei sistemi *gacha*.

[22] Nella trascrizione "overground conditioning": si tratta dell'*operant conditioning*, il condizionamento operante di B. F. Skinner.

[23] Schell, *The Art of Game Design*, cap. 15 (*Interface*).

[24] Il modello a tre livelli (percezione, comprensione, proiezione) è quello di Mica Endsley (1995).

Il meme di rito. Come ogni blocco di slide sulla documentazione, anche questo si apre con un meme: qualcuno davanti a **94 pagine** di documento sui pilastri di visione. "È vero, purtroppo: non dovrebbe accadere, ma capita." Nessuno lo capisce, nessuno vuole leggerlo, "è terrificante".

9.9.2 Dalla Bibbia alla wiki, al compendium

Il docente richiama ciò che era stato detto nelle prime lezioni sulla documentazione (sezioni 3.6 e 5.2): i documenti di design non si scrivono più come negli anni Novanta. Le **Bibbie di design**, come quella di *Doom* vista in classe, non esistono più come erano allora. Il salto è stato: Bibbia di design, poi **wiki** di design, poi il modo attuale, "un ibrido tra wiki e altre cose".

L'obiettivo attuale del **compendium**, o **design bible** come lo si chiama ancora, è proprio questo: essere un documento che **rimanda a tutti gli altri documenti**, che faccia da **gateway** per la lettura del Drive (se si usa Google Drive), di Confluence (se si usa Confluence), e così via.

- Prima come adesso, la Bibbia di design contiene **tutti** i documenti legati al gioco e al progetto.
- Ma oggi questa documentazione onnicomprensiva è **divisa tra i dipartimenti**: game, narrative, level, arte, concept e così via.
- Come per il production dictionary, il compendium è **uno e uno solo per progetto**.
- A differenza delle vecchie Bibbie, **non spiega tutto**: serve solo da indicazione. È "una cartina che rispedisce il lettore dove deve andare".

La Bibbia vecchio stile come formalità. Il docente ricorda quanto detto l'altra volta: può ancora capitare che si faccia un documento di design vecchio stile, perché qualcuno **fuori dal team operativo** lo richiede (un publisher, il marketing). Ma se un documento non serve per lavorare, è una formalità.

9.9.3 Che cosa contiene

Il docente mostra un esempio, con l'avvertenza che vale per tutti i documenti della giornata: sono **esempi**, "template che trovano il tempo che trovano". Ognuno deve sviluppare il proprio, con senso critico, sapendo che molte cose si possono migliorare. Anzi: gli studenti hanno "la mente più fresca" del docente, che ha già visto molti di questi documenti fatti in un certo modo, e possono innovare più facilmente di chi è "radicato" in due o tre soluzioni ideali.

I link a tutti gli altri documenti Per forza: è esattamente come una wiki. C'è il nome del documento, c'è il link, e il lettore viene rispedito lì. Per questo il compendium **ha poco testo**.

Lo status di ciascun documento Gli status più noti sono **To Do**, **Work in Progress**, **Done** e **Cut** (tagliato, "cattato"); a volte anche *Deleted*. Ma gli status codificati presuppongono una **legenda a monte**: se si inseriscono gli status nel compendium, serve una legenda che spieghi i singoli status scelti.

Essenzialità Non deve avere molto testo: l'obiettivo è **ridirezionare il lettore**. L'artista che non sa come vestire il personaggio principale, o il programmatore che non sa che cosa deve fare il personaggio quando salta, fa il doppio salto o si muove, va nel compendium, trova la riga giusta, clicca sul link e viene rispedito al documento che deve leggere.

Accessibilità Chiunque deve potervi accedere. Se questa è la **mappa** che ogni persona del team usa per navigare il Drive, deve poterla vedere chi non ha subito idea di dove andare: non tanto un designer, ma a maggior ragione artisti, programmatori, sound designer.

Divisione per dipartimenti Deve essere facilmente divisa, come si diceva, per dipartimento.

Mantenimento Vale per tutti i documenti di visione: il compendium deve **rimanere aggiornato**. Se si aggiunge un documento, va aggiornato; se cambia lo status di un documento, va aggiornato; se in un meeting, magari con il benestare del lead, si decide qualcosa, va aggiornato. Una persona all'interno del dipartimento (i **lead**, o chi sta sopra di loro) ha la responsabilità di aggiornarlo e di **notificare** gli aggiornamenti importanti ai membri del team interessati.

Gli status, prima o poi, altrove. Più avanti, parlando di produzione, si vedrà l'equivalente del compendium per i **task** e le cose da fare nel progetto: un documento non operativo ma **produttivo**. Gli status potrebbero non dover essere ripetuti in entrambi i documenti; e "se diventate particolarmente bravi", alla fine dell'anno si potrebbe perfino pensare di **ibridare** i due documenti. Per ora, essendo la prima volta, gli status vanno nel compendium.

Un documento che cresce. All'inizio del progetto il compendium è molto spoglio; con l'avanzare del progetto si **rimpolpa**. Ogni volta che si crea un documento nuovo, va inserito lì: "non dimenticatevi".

La tabella 9.5 mostra uno schema possibile, ricostruito a partire da questi requisiti (non è la slide del docente).

Table 9.5: Uno schema possibile di compendium: una riga per documento, divise per dipartimento, con link, status, responsabile e legenda in testa. Nomi e status sono di esempio.
Legenda status: TODO da iniziare; WIP in lavorazione; DONE completato e approvato; CUT tagliato dalla produzione.

Dipartimento	Documento (link)	Status	Responsabile
Generale	Vision	DONE	direzione
	Production dictionary	WIP	lead design
Game design	Movimento (salto, doppio salto)	WIP	game designer
	Sistema di combattimento	TODO	game designer
Level design	Livello 1 — blockout	WIP	level designer
Arte	Concept del personaggio principale	DONE	lead art
Narrativa	Sottotrama del mercante	CUT	narrative designer

9.10 I pilastri: l'esempio di God of War

9.10.1 La visione va documentata

"Ora, questo è un altro discorso molto importante, che abbiamo già iniziato a fare più volte; adesso lo vedete concretizzato." La **visione** di un progetto va **documentata**. È la **bussola** che aiuta il team quando serve o quando ha dubbi: dove andare, che scelte di design prendere, che cosa inserire e soprattutto **che cosa non inserire** nel progetto. Una parte di questa bussola sono i **pilastri**, i *design pillars*, o semplicemente *pillars*.

9.10.2 I tre pilastri di God of War

L'esempio viene da un talk alla **GDC** sul *God of War* "nuovo", quello del 2018,[25] in cui il team di Santa Monica spiegava i design pillar del gioco. Sono **tre**: "non più di tre, non meno di tre". I nomi sono anche generici: il docente invita ad avere nomi **evocativi**, ma se non serve, è giusto che siano **inequivocabili**; e infatti sotto a ciascuno ci sono delle sotto-categorie che lo precisano (tabella 9.6).

Combat: stesso pilastro, declinato diversamente. God of War è sempre stato un gioco *action*, e il pilastro del combattimento c'era anche nei capitoli precedenti. Ma qui è declinato in modo diverso, e la specificità delle sotto-voci è proprio ciò che separa la nuova trilogia ipotetica da quella precedente: diversa **tecnologia** (il mocap), diversa **mitologia**, più varietà di **armi**, e soprattutto Kratos **non combatte da solo**, ma con il figlio. Il docente collega lo shift a un paragone tra il registro dei film Marvel e quello dei fumetti Marvel.[26] Ciò che rimane uguale è il fatto di avere un gameplay action.

[25] Nella trascrizione "il talk della GTC sull'event design di God of War, quello nuovo, del 2019". Si tratta quasi certamente di un talk della Game Developers Conference del 2019 dedicato a *God of War* (2018), di Santa Monica Studio; il titolo esatto del talk non è ricostruibile dalla trascrizione.

[26] Nella trascrizione "la differenza tra Marvel Films e Marvel Comics"; non è chiaro in quale direzione il docente applichi il paragone, se non come differenza tra un combattimento più cinematografico e uno più "fumettistico".

Table 9.6: I tre pilastri di *God of War* (2018) come illustrati a lezione. Le voci in inglese seguono la slide; alcune sono ricostruite dalla trascrizione.

Pilastro	Sotto-voci e significato
Combat	combattimento più **cinematografico**, con la camera in un unico piano sequenza, sempre dietro la spalla; **nuove creature mitiche e armi** (l'ascia Leviatano, lo scudo, la mitologia norrena al posto di quella greca); **nuovo combattimento a media distanza e ravvicinato** (lanciare l'ascia o colpire direttamente); **high optical mocap fidelity** (animazioni in motion capture ad alta fedeltà); **integrated son companion** (Atreus che combatte al fianco del padre, con le frecce)
Father / Son	**umanizzare Kratos**; il padre insegna al figlio e il figlio insegna al padre; il figlio **aiuta**, non ostacola; il figlio è **credibile**, è un ragazzino; una relazione **simbiotica**
Exploration	non un open world, ma un forte **senso di scoperta**; zone "arena-like" da cui si ramificano possibilità di esplorazione; **raccolta di risorse**; **puzzle** (*cerebral engagement*); il legame con il figlio nei momenti di esplorazione; **hunting and making** (caccia e costruzione, a partire dalle risorse)

Father / Son: un pilastro nuovo. Il secondo pilastro è chiaramente diverso, e i capitoli precedenti non lo avevano: la **relazione tra padre e figlio**. L'obiettivo è **umanizzare Kratos**. Prima Kratos era "un pezzo di tufo", uno che andava in giro a spaccare crani e divinità per vendetta. Ora è diverso: Kratos, spartano "non solo di origine ma anche di anima", abbraccia il figlio e la famiglia, e diventa più morbido. All'inizio cerca, erroneamente, di difendere il figlio dai mali del mondo, e pensa che abbia ancora tutto da imparare ("non dico che sia un peso"); poi capisce che il figlio "non è così stupido", che lo aiuta e non lo ostacola. Il viaggio è fatto insieme, e il loro legame cerca di rafforzarsi, non di spezzarsi.

E il pilastro di un vecchio God of War? Il docente chiede alla classe quale potesse essere un pilastro dei vecchi capitoli, al posto di questo. La risposta, accettata, è un pilastro più spinto sul lato **narrativo** e sull'obiettivo del gioco: la **vendetta**, "tra l'altro senza fine", perché ucciso un dio se ne vuole un altro da massacrare, fino ad arrivare a Zeus. Un obiettivo di vendetta continuo e oppressivo.

Exploration: più spinta che in passato. Il terzo pilastro è l'**esplorazione**, molto più spinta rispetto ai capitoli precedenti. Non è un open world; c'è però un senso di scoperta, con zone che nel talk sono descritte come *arena-like*, da cui si ramificano possibilità di esplorazione, raccolta di risorse e anche **puzzle** (il *cerebral engagement*, che c'era già in passato ma qui è più espanso). Si aggiungono il legame con il figlio nei momenti di esplorazione e l'*hunting and making*: andare a caccia e costruire con le risorse che si hanno.

Ad alto livello, ma già con un passo in più. Questi pilastri sono "già anche uno step" oltre l'alto livello: nelle sotto-voci si parla già di meccaniche, di cose che i designer dovranno fare. I pilastri, comunque, restano sempre **tre** e restano **ad alto livello**; qui si inizia solo a entrare un po' più nel dettaglio.

Appesi nella hall. La concretizzazione ed espansione dei pilastri in modo chiaro e codificato è il documento di **vision**. Ma i pilastri, come la vision, il team deve **averli sempre a mente**: chiunque, nel team, deve ricordarli. A Santa Monica, racconta il docente, li avevano **appesi nella hall**, in grandi strisce, ben visibili all'ingresso. Vanno stampati e fatti vedere, devono essere **sempre visti**: "tipo i dieci comandamenti, se mi passate la metafora cristiana".

9.11 Il documento di vision

9.11.1 Che cos'è

La **vision** è il documento informativo sulla visione totale del progetto: la **formalizzazione e l'estensione dei pilastri**. È ciò che serve a **guidare il team**. Come sempre, l'esempio mostrato è solo un esempio: "non

rimanete incatenati a questa cosa". La tabella 9.7 riassume le sezioni descritte a lezione.

Table 9.7: Le sezioni di un documento di vision, come descritte a lezione.

Sezione	**Contenuto**	**Obbligatoria?**
Pilastri (o temi)	le tre "anime" del gioco: tutto il resto deve supportarle	sì
Prime emotions	le tre emozioni principali evocate, e perché (dalla lista delle emozioni)	di norma sì
Prime moods	i tre mood principali, e perché (più ampi: level design, referenze visive o sonore)	di norma sì
Obiettivo, meaning, morale	lo scopo meccanico; il significato e la morale narrativi	no, dipende
Must do	ciò che **deve** esserci (ad alto livello)	sì
Must not	ciò che **non deve** esserci	sì
Nice to have	ciò che si vuole, ma che si può tagliare	di norma sì
USP	i *unique selling point*, al massimo tre	di norma sì
In coda	narrativa, setting, target audience, estetiche (con must e must not), allineamento con i tipi di giocatore	no

9.11.2 Pilastri e prime emotions

I pilastri. Ci devono essere "per forza": **tre temi principali**, le tre anime che compongono il gioco. Tutto ciò che si scrive nella vision, e tutto ciò che in produzione viene incanalato nella creazione del gioco, deve **supportare** questi tre pilastri: narrativa, meccaniche, level design, qualsiasi cosa è costruita su questa base.

Prime emotions e prime moods. Di solito si distinguono le **prime emotions** e i **prime moods**: le **tre** principali emozioni evocate dal gioco, e perché; i **tre** principali mood, e perché. Per le emozioni c'è la **lista** vista nel corso ("che poi estenderemo o meno"): la si usi, o comunque si scelgano emozioni specifiche. Il mood è un discorso più ampio e più generico: dentro un mood possono ricadere molte cose. Può essere legato al level design, se il gioco è più spinto su quello; può essere una referenza visiva o sonora. Sui mood ci si può espandere. Sempre tre.

9.11.3 Obiettivo, meaning e morale

Questa parte **non è necessaria**: potrebbe non esserci all'inizio, o essere inclusa nei pilastri, a seconda dello stile di documentazione. Si tratta dell'**obiettivo** del gioco e del suo **meaning**, il significato narrativo, la **morale**.

L'esempio di Mario. Qual è l'obiettivo di Mario, meccanicamente? Arrivare alla fine degli N mondi, generalmente otto: "batto Bowser, salvo Peach". E la morale? La classe cerca "la più semplice possibile": **il bene vince sul male**. È un concetto molto semplicistico, e Mario "non è che abbia una morale spiccatissima": per questo alcuni di questi elementi sono **ignorabili**. Non i pilastri, certo; ma obiettivo e meaning sì.

Anche questa parte passa per un **processo iterativo**: va ragionata attentamente, e arriva dopo la pre-produzione, o comunque alla sua chiusura.

9.11.4 Must do, must not, nice to have, USP

La parte di **design** della vision è quella in cui i pilastri sono già espansi, e in cui c'è parte del design che potenzialmente andrebbe anche altrove. Le quattro voci, "di cui forse abbiamo già parlato", sono queste.

Must do: ciò che ci deve essere. Sono le cose che il gioco deve avere, **ad alto livello**: a questo punto le meccaniche non sono ancora fatte, i GDD non ci sono, manca ancora molto della produzione. Ma se ci sono meccaniche o sistemi che ci si porta dietro dall'**IP** o da titoli precedenti, vanno spiegati il più precisamente possibile.

- Il tipo di **3C**: che tipo di camera, controlli e personaggio ci si aspetta? A questo punto il docente si aspetta che ci sia già un **prototipo** e che si sia già pensato, per esempio, se il gioco ha una camera isometrica, una camera in terza persona a spalla, o è in prima persona.
- Se invece si parla di uno **skill tree**, nel must do si scrive che c'è; ma non interessa sapere quanto è ramificato o quanti rami abbia. "Non penso lo abbiate, a questo punto della produzione: è molto strano che lo abbiate qui."

Negli esempi di God of War, la *high optical mocap fidelity* è un must do; la *discovery* dell'esplorazione è un'emozione; l'*integrated son companion*, una volta stabilito il tipo di combattimento, è un must do, e lo è anche narrativamente. Il must do si può anche dividere per necessità di game design, di level design e così via: dipende dallo stile.

Must not: ciò che non ci deve essere. Per un gioco come *Animal Crossing*, nei must not potrebbe esserci il **gore**, o alcune meccaniche che identificano il gioco "come non quella roba lì". Nel caso di God of War, il combattimento potrebbe avere come must not il non essere simile in alcune parti a quello dei titoli precedenti; per contro, la **violenza**, che per Animal Crossing è un must not, per un Kratos è tutt'altro.

Nice to have. Le cose o i sistemi che interessa avere, ma che **non sono l'anima del gioco**. Se la produzione è travagliata o si finiscono i soldi e si deve tagliare da qualche parte, si taglia sui nice to have: è giusto che il gioco li abbia, ma se non ci fossero "ce ne faremo una ragione", e l'anima del gioco resta in piedi.

Unique selling point. Gli USP (o *unique selling proposition*). Come detto tempo addietro, l'ideale è averne **tre**, **al massimo tre**. Per giochi più di nicchia, indipendenti, esperienze ridotte, potrebbero essercene anche meno.

9.11.5 In coda al documento

Si può aggiungere, anche qui senza obbligo, il tipo di **narrativa**, il **setting**, il **target audience**. Nell'esempio mostrato ci sono le **estetiche** di gioco, sempre con must e must not, e il target audience, con quali elementi e quali emozioni sono allineati a specifici tipi di giocatori: "Serious Fun, Easy Fun", le chiavi del divertimento che si vedranno in futuro.[27] "L'unico caso in cui potete dire *fun* senza essere flagellati" (la parola proibita, sezione 1.4.4).

9.11.6 Le regole del documento

Una volta approvata, non cambia più Una volta creata e **approvata** da tutto il team (in un team indipendente di tre persone, tutti devono essere d'accordo; nei team grandi decidono i direttori, e poi la decisione va a cascata), la vision è **blindata**: "quello è". Proprio perché aiuta la produzione, rimane fissa: sono le **fondamenta** del gioco.

La funzione è più importante della forma Deve essere facile da leggere e soprattutto da **comprendere**.

Accessibile Tutti devono potervi accedere, e soprattutto ci si torna **quando si ha un dubbio**: se c'è un'incognita di design, si torna lì sopra. Va scritta, a maggior ragione, come una **linea guida**.

[27] Nella trascrizione "Sirius fan, Isley fan". Si tratta quasi certamente delle quattro chiavi del divertimento di Nicole Lazzaro (*Hard Fun*, *Easy Fun*, *Serious Fun*, *People Fun*).

Responsabilità La **direzione** si assicura che la vision venga prodotta; chiede pareri, opinioni e feedback su punti specifici ai **lead** di dipartimento (se un obiettivo è raggiungibile, se è desiderabile; lato arte, lato design, per le meccaniche, i sistemi e così via). Ma è responsabilità della direzione **crearla, blindarla, approvarla**. I lead non la modificano.

Approfondimento: la vision come vincolo utile. Anche il testo di Bateman e collaboratori sul game design del ventunesimo secolo parla di *vision statement*: un progetto parte spesso da una dichiarazione che ne fissa gli obiettivi generali, e deciderla presto aiuta a costruire l'identità del progetto, a **restringere le opzioni** di sviluppo a un sottoinsieme gestibile e a dare **focus**.[28] È lo stesso principio del paradosso della scelta (sezione 9.4), applicato al team invece che al giocatore: meno strade, ma scelte meglio.

9.12 La team vision

9.12.1 Un documento per il team, non per il gioco

Ci sono altri documenti che si possono pensare di fare. Questo è uno di quelli, "non così comune", soprattutto nelle aziende già strutturate, dove il suo equivalente è, di fatto, il **contratto**. Per i **team indipendenti**, però, può valere la pena creare una sorta di vision per il team: la **team vision**.

L'obiettivo è segnare chi fa parte del team e blindare l'**obiettivo** che ciascun membro ha, ma anche le sue **responsabilità**, i suoi **diritti** e i suoi **doveri**. Se uno sceglie di fare il team leader o il producer, lo si segna. Nei progetti dell'accademia non ci saranno molte figure diverse affiancate ai designer; ma magari qualcuno vorrà imparare o sperimentare di più il **level design**, e allora si segna che le sue responsabilità verteranno maggiormente su quello, mentre altri saranno più propensi al **game design** ("come i pazzi come Alessandro") o al **producing**.

La firma. È un documento **da firmare**. Non perché la firma valga qualcosa legalmente (nel contratto, invece, vale), ma perché in generale, se si appone una firma, si è più propensi a **rispettare** quello che si è scritto.

9.12.2 Siate onesti

La cosa più importante di tutto il discorso è una: **siate onesti**, con voi stessi in prima battuta e con i membri del team. Su che cosa si può e soprattutto **non si può** fare, su che cosa si vuole e soprattutto **non si vuole** fare, "cambiano le questioni". Se uno dichiara di avere cinque giorni su cinque liberi per lavorare a un progetto, gli altri si aspettano quel tipo di *commitment*. Quindi **non mentite**: se ciascuno è onesto con sé stesso e con gli altri, si raggiungono i risultati con meno problemi, e soprattutto si può **scalare** il progetto, in grandezza o in piccolezza. Trovare la dimensione giusta del progetto all'inizio della produzione è già difficile; se ci si aggiungono errori umani o menzogne, "nel bene o nel male", lo *scope* viene sicuramente sballato.

9.12.3 Che cosa contiene

La team vision è un documento informativo che contiene le **linee guida** su come il team viene gestito e i **principi** del team:

- la **missione** del team: che cosa vogliamo fare, qual è il nostro obiettivo (fare soldi, fare un progetto innovativo, imparare, sperimentare, mettere in pratica le nozioni di game, level o narrative design...);
- il **tipo di produzione**;
- i principi di **public relations**, per i team che poi si affacciano al mercato;
- la **gerarchia** e la **pipeline**;

[28] Chris Bateman, Richard Boon, *21st Century Game Design*, 2005, nel capitolo sulle astrazioni del mondo di gioco.

- i **diritti e i doveri** dei singoli membri.

Le regole sono le stesse della vision: una volta approvata **non cambia**, la si fa **firmare** ai membri del team, deve essere facile da leggere e da comprendere, chiunque nella produzione deve potervi accedere, e **direzione e management** sono incaricati di blindarla.

9.13 La fine della pre-produzione, ripresa

Il docente torna alla slide che chiudeva la parte sulla ricerca (sezione 5.11, tabella 5.3): le domande a cui bisogna saper rispondere alla fine della pre-produzione. La precisazione di oggi è che non si risponde **in un documento**, ma con **la somma delle cose**: la vision, la ricerca, tutti i documenti prodotti in pre-produzione devono dare le risposte. E le risposte non vanno tenute "lì, nel caso ce lo chiedano": bisogna avere ben chiaro che **questa è la linea che ci si è scelti**.

Visione che cosa vogliamo fare, noi, come team.

Obiettivo che cosa vogliamo raggiungere concretamente: soldi, fama, imparare, mettere in pratica una tecnologia (imparare Unity, "o qualsiasi cosa").

Risorse e scope che cosa possiamo fare realisticamente. Lo si vede con la ricerca e con i competitor: quanta esperienza si ha, quante cose si sanno fare nell'engine e fuori dall'engine, e soprattutto **quanto tempo** si ha a disposizione.

Target audience e competitor che cosa sappiamo che funziona, "che si attacca al muro": che cosa piace alla gente, che cosa viene comprato, che cosa viene giocato.

Target audience e must not che cosa sappiamo che invece non funziona.

Target audience e cliente e qui entrano gli **stakeholder** (sezione 4.11): che cosa vuole il cliente, se c'è; che cosa ha imposto; che limiti ha posto. Alcuni di questi limiti finiranno nei must do e nei must not della vision; a volte il cliente può imporre perfino un **pilastro**.

Marketing e piano qual è il business plan, e la pipeline in generale: più legato alle produzioni che si affacciano al mercato.

L'unione di tutto questo dà la **visione completa** del gioco.

9.14 Template e linee guida: sperimentate adesso

9.14.1 Linee guida, non template

Il docente ripete un discorso già fatto: tutto ciò che si è visto (il compendium, la vision, la team vision) **non sono template**, sono **linee guida**. Alcune cose non possono mancare: i **link** nel compendium; i **must do**, i **must not** e soprattutto i **pilastri** nella vision. Ma la forma può cambiare, le separazioni possono cambiare.

- Una parte sull'arte si può togliere del tutto se si sta facendo un gioco innovativo o sperimentale che non richiede un'arte specifica.
- Se si sta facendo *Tetris*, il concept brief e la narrativa sono poco rilevanti.
- Scrivere adesso alcune di queste cose (narrativa, setting, target) aiuta poi quando si parlerà di **pitch** e di documento di pitch; ma sono tutte cose che si possono estrarre dai pilastri, dai must do, dai must not e da come sta andando la produzione.

Il docente cerca di non dare template, per quanto possibile; ma è utile vedere come lo stesso tipo di documento è stato fatto da designer diversi, in produzioni diverse. Va guardato sempre con **occhio critico**, in vista di migliorarlo o di sperimentare.

9.14.2 Il campo di prova ideale

"Avete proprio adesso il campo di prova ideale per fare qualsiasi pazzia vogliate", lato formattazione e lato contenutistico, e vedere che cosa succede: che cosa è andato bene, che cosa è andato male. "Anche perché, sbagliare, alla fine, cosa può succedere? Che non vi leggo il documento. Chissà che roba." Certo, può mettere ansia e pressione, ma è "molte leghe sotto" rispetto a ciò che potrebbe capitare in un contesto lavorativo.

Il consiglio, quindi, è di **sperimentare e innovare**, ora che si ha una forma mentis "fresca e vergine" e non si hanno i limiti stretti di una produzione: le scadenze settimanali (che sono i task, più o meno, come gli *sprint* di una produzione agile), il programmatore che chiede un'altra cosa, il producer che chiede di fare la Bibbia di design unendo i documenti di design. "Queste cose voi ne dovete soffrire adesso; un domani, mi auguro di no, ma realisticamente penso succederà."

Gli strumenti cambiano. Ciò che farà la differenza è la pratica fatta nel corso e l'esperienza accumulata. Nei test che verranno chiesti per lavorare, lato game design ma non solo, la documentazione richiesta sarà diversa: a volte un GDD andrà fatto in un certo modo perché il tempo o il progetto lo richiedono. Non si potrà sempre fare la lista puntata o usare un certo template: magari l'azienda non lavora con Google Drive ma con **Miro**, e allora si usa Miro, "e sarà un concetto completamente diverso". "Sfruttate tutto quello che avete a disposizione adesso."

La seconda pausa. "Visto che so che è sempre mal assorbita, la documentazione": cinque minuti di pausa, fino alle 15. Nel frattempo, la consegna: pensare, tra chi è in aula, a un **gioco arcade**, per rifare i core elements partendo da zero.

9.15 L'esercizio: i core elements di Frogger, dal vivo

9.15.1 La scelta del gioco e del volontario

Al rientro, un nuovo controllo dell'audio per chi è da casa ("ho fatto un casino"), e la scelta del gioco. In aula non si era deciso niente; alla fine la scelta cade su ***Frogger***, "perché è in mezzo al documento". ("Frogger, non ne frega niente a nessuno di Frogger, maledetti": il docente lamenta che d'ora in poi gli comparirà Frogger tra i correlati ogni volta che cercherà qualcosa.) Il docente consiglia di navigare tra le **schede** del documento per orientarsi, e chiede un "**bold** essere umano" che venga al suo posto. Dopo qualche esitazione (e un dado tirato per decidere, per "evitare danni"), al computer va **Francesco**: lui è "il braccio", il resto della classe la mente, da aula e da casa.

9.15.2 La lista dei core elements, a memoria

Francesco comincia dal titolo del task, che posiziona in modo che compaia anche nell'elenco delle schede. Poi il docente pretende che la lista dei **core elements** sia ricostruita **a memoria**, senza guardare ("non vale guardare"): il framework da usare è quello di **Romero e Schreiber** (sezione 2.8). E qui il docente insiste su un punto di metodo: i framework vanno imparati, e il motivo per cui li si scrive non è solo per consegnare, ma **per memorizzarli**.

La classe, a fatica e con qualche suggerimento, ricostruisce i sette elementi:[29] **game qualities**, **tema**, **giocatori**, **interazione**, **obiettivi**, **regole**, **risorse**.

9.15.3 Le indicazioni del docente durante l'esercizio

Il docente lascia lavorare la classe, intervenendo con domande e consigli. Quelli comprensibili dalla registrazione sono i seguenti.

[29] La trascrizione di questa parte è in gran parte incomprensibile. Si riconoscono con certezza: *game qualities* come primo punto, *theme*, *players*, l'interazione, gli obiettivi, le risorse ("io volevo dire le risorse") e, implicitamente, le regole. L'ordine in cui sono stati detti non è ricostruibile.

Ricerca prima di scrivere. Per chi non conosce il gioco: "a pagina nuova, e si va a leggere, si va a fare ricerca". Ora lo si sta facendo in condizioni molto limitanti, ma un domani, in un test di lavoro, la situazione non sarà molto diversa: si dovrà fare molto di più, ma con un tempo indicativamente simile, e si dovrà fare ricerca anche sui titoli che non si conoscono.

Dividersi il lavoro. Il documento è condiviso: chi è da casa può entrare e mettere le mani dentro. Ci si possono **dividere le sezioni**, perché lavorare tutti sulla stessa porzione diventa problematico. E bisogna abituarsi a **parlare**, anche quando ci si sente di non conoscere abbastanza il gioco.

Titoli e sottotitoli. Mattia suggerisce, da subito, di dividere le sezioni (le game qualities, il tema...) con **titoli e sottotitoli** veri, in modo che il documento sia più leggibile per chi deve leggerlo. Il docente approva ("una cosa giustissima che Mattia ha detto, ma che nessuno ha messo in pratica", commenterà più tardi).

Che cosa sono, in concreto, le game qualities? È la domanda su cui la classe si incaglia. Il docente la pone in modo pratico: una volta scritte su un foglio bianco, **che cosa sono** le game qualities? Le meccaniche sono troppo specifiche: le game qualities "al massimo sono i sistemi", ma a un livello più alto. Una studentessa prova: sono "un insieme di qualità del gioco che lo rendono tale". Il docente accetta come punto di partenza: scritte in fila, già dicono qualcosa del **genere** del gioco.

Il 2D e la pixel art. Si discute se "2D" e lo stile grafico "pixel" siano game qualities. Lo scambio è poco comprensibile;[30] nella versione finale del docente, la *pixelated art* compare tra le game qualities.

Le celle vuote. Un consiglio di formattazione: se in una tabella una cella resta vuota perché l'informazione non è stata trovata, o non c'è, **non va lasciata vuota**. Ci si mette ciò che si vuole, purché lo si dica: **TBD** (*to be defined*) significa che poi bisognerà tornarci; una **barra** significa che non c'è.

L'obiettivo, e il punteggio. Per l'obiettivo la classe propone "attraversare la strada per arrivare alla fine, senza morire": raggiungere l'altro lato senza morire. Il docente chiede di ampliarlo, e poi pone la domanda: **in base a che cosa la rana riceve il punteggio**? E a che cosa serve il punteggio? La risposta è la **competizione**: il fatto che il giocatore debba raggiungere il punteggio più alto possibile è a sua volta un **obiettivo**, e comprende anche più di una qualità di gioco (il confronto con altri giocatori). Ne esce una frase in inglese, limata a più mani: *the player tries to achieve the best score possible while reaching...*

Il timing è una dinamica, non un obiettivo. Una proposta per gli obiettivi è "trovare il tempismo giusto per saltare sui tronchi" (*to catch the right timing to jump on the logs*). Il docente la sposta: è una **dinamica** del giocatore, una strategia che il designer spinge e che il giocatore deve adottare, ma è **il mezzo per arrivare al fine**, non il fine. Il fine è raggiungere la tana della rana.

Tronchi e tartarughe. Per le piattaforme del fiume, si discute se elencare tronchi e tartarughe o scrivere un termine generico; ma *obstacles* è già stato usato per le auto, quindi non lo si può riusare per indicare cose diverse. Si annota anche che si muovono **orizzontalmente** sullo schermo.

Tempo, vite, fine. Rimangono il **tempo** (c'è un limite di tempo per ogni rana), la meccanica delle **vite** e la domanda su come si arriva al **finale** del gioco: è un obiettivo, e va scritto lì, non nel tema.[31]

Il tema. Nel tema la classe discute se Frogger abbia una narrativa: la rana deve tornare a casa attraversando strada e fiume, ma una storia vera e propria non c'è.[32] Ciò che c'è, si scrive.

[30] La trascrizione riporta: "Allora, pixel si potrebbe descrivere come una game quality? No, essere un tema." Non è chiaro chi risponda, né se la risposta sia del docente. Nella riformattazione finale, comunque, il docente inserisce *Pixelated art* tra le game qualities.

[31] Questa parte della trascrizione è quasi del tutto incomprensibile ("il lavoro di SIGAPO" ripetuto in loop, una discussione sul tempo e sul "finale"). Si riportano solo i punti certi.

[32] Nella trascrizione compare anche un riferimento confuso a "il signor Hashimoto" e a una "macchina" che attraversa la strada, forse una storia sull'origine del gioco raccontata da uno studente; non è ricostruibile e non è qui riportata.

"Come formattereste questa roba?" Alla fine il docente chiede alla classe come formatterebbe ciò che è stato scritto, perché finora "è testo: testo con l'elenco puntato, però è testo". Ringrazia Francesco "per aver assorbito la responsabilità di fare questo documento" ("tra l'altro, non sufficientemente aiutato"), e prende il suo posto.

9.16 La riformattazione del docente

9.16.1 Un modo, non il modo

Il docente premette: quello che segue è un **esempio**, e nemmeno completo. "Non è l'unico modo per fare le cose, né il migliore, né qualsiasi cosa lo vogliate. È un modo; tra l'altro, un modo che uso io, e che uso io in questo momento: domani potrebbe cambiare." Ed è un lavoro fatto "all'arrembaggio", in pochi minuti: nei task si ha una settimana per pensare a tutte le implicazioni di strumenti diversi.

9.16.2 Il titolo

C'è un modo per rendere il titolo più comprensibile e più appagante, senza perdere informazioni? "Qualcuno l'ha già fatto": il **logo**. Il docente cerca "Frogger PNG", trova un logo utilizzabile (tra "tanto schifo"), e lo sistema velocemente con **Photopea**, "il nostro miglior amico". Accanto, il nome del task: *Arcade core elements*, **non tutto in maiuscolo**, in **bold**, in nero (un colore a caso viene provato e scartato: "teniamolo nero, forse è meglio"). Andrebbe testato: una soluzione che funziona qui potrebbe non funzionare lì.

Il logo **mangia spazio**, "per i miei gusti molto": il docente fa notare quanto ne prende sulla pagina. Ma come **prima iterazione** è accettabile.

9.16.3 Gli stili e le intestazioni

Poi mette in pratica il suggerimento di Mattia: usare gli **stili** di paragrafo, cioè dei veri titoli, per *Game qualities*, *Theme* e così via. Sul tema mostra che si può fare in diverse maniere: uno spazio prima e uno dopo, una linea, un colore diverso ("colore tossico, magari no"), l'interlinea ("forse ne parlava anche Francesca"). Tutte rendono più chiara la **divisione** tra le sezioni; alcune "mangiano tanto spazio". Per i *players* si può riprendere lo stesso stile, o un altro; "ci sono diversi modi".

9.16.4 Le game qualities: dalla lista alla tabella

Le game qualities sono una **lista**, e il docente la fa come tale. Ma è una lista di termini molto stringati. Si può ordinare per **importanza** o in ordine **alfabetico**; "per la mia sentimentale", in fretta, ne esce questa:

- *Arcade*;
- *PvP* e *indirect competition* (competizione indiretta, tramite il punteggio);
- *Pixelated art*;
- *Adaptive music*: il docente ricorda che, a quanto sa, *Frogger* è stato il primo gioco della storia con musica adattiva;
- *Fixed screen*: una schermata unica;
- *Leaderboard*, perché c'è il punteggio, "che non abbiamo mai citato".

Il problema della lista stringata. Una lista di parole singole, una sotto l'altra, spreca spazio e si legge male. La soluzione ("non l'unica") sono "le nostre migliori amiche", le **tabelle**: "hanno grande potere, ma grande responsabilità". Il docente nota che i colleghi della sede di Roma "generalmente ne abusano". E avverte che una tabella dopo l'altra resta comunque testo: occhio. Sistemata la lista in una tabella su più colonne, la si può colorare, separare di più o di meno, dare spessori diversi ai bordi: "ci sono mille modi per farlo".

Una tecnica per le tabelle. Il docente mostra anche come riorganizzare le righe di una tabella senza rifarla: aggiungere una riga sopra o sotto, copiare e incollare il contenuto, **unire le celle**, e cancellare ciò che non serve; oppure prendere la tabella intera e tagliarla, per poi reinserirla. Il punto è che *players* serve con la sua intestazione, come le altre sezioni.

9.16.5 Il tema in un box

Per il tema, il testo viene ripulito e riscritto in inglese: la rana deve tornare alla sua tana superando i pericoli della strada, e poi del fiume ("ci sono anche i coccodrilli, che non è che facciano proprio parte del traffico": *the dangers of the water*). Il docente nota che si sta lavorando con un testo "aberrantemente grosso", in corpo 11, e con troppo spazio dopo i paragrafi.

Il tema è diverso dal resto, e lo si può far vedere: una linea, un colore diverso, oppure **boxarlo**, usando una tabella di una sola cella con un bordo di 1,5 punti ("forse 2 è tanto"). "Guarda un po': è diventato un *disclaimer*." Di nuovo: non abusare di questo sistema. Il docente suggerisce anche di **sottolineare**, o comunque evidenziare, le informazioni importanti.

Rubare dalle scatole dei cereali. Un consiglio che vale per tutto il corso: quando si va a fare la spesa, guardare come sono formattate le **tabelle nutrizionali** sul retro delle confezioni, sulle scatole dei cereali. "Da lì si può rubare davvero tantissimo", perché c'è **poco spazio** e bisogna metterci **tanto contenuto**.

9.16.6 Gli obiettivi in due box

Per gli obiettivi non c'è tempo di fare tutto, ma l'idea c'è: **due box**, se si vuole restare coerenti con la scelta fatta per il tema.

1. **Meccanicamente**, raggiungere l'altro lato dello schermo (e il fatto che il gioco sia a schermata fissa va tra le game qualities);
2. avere il **punteggio più alto**.

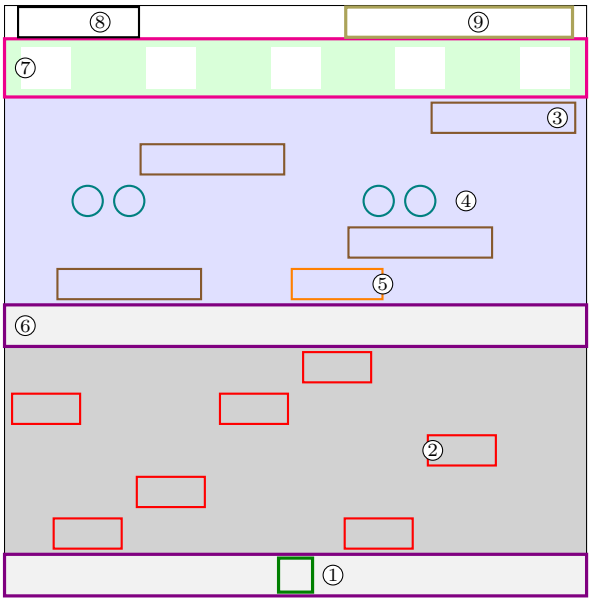
Il timing per saltare sulle piattaforme, come detto, resta fuori: è il mezzo, non il fine.

9.16.7 Le regole: mostrare invece di scrivere

Per le regole "si sprecano" le cose da dire. Ma *Frogger* ha un vantaggio raro: **tutti gli elementi del gioco sono in una sola schermata**. Il docente prende uno screenshot del gioco e ci disegna sopra, con colori diversi, i contorni degli elementi ("bear with me, stateci con i colori"). La figura 9.2 ne è una ricostruzione schematica.

Gli elementi cerchiati. In ordine:

- il **personaggio principale**, la rana;
- il **tempo**, che è una regola;
- gli **ostacoli** della strada: auto e camion, stessa categoria ma sprite diversi, ognuno cerchiato;
- i **coccodrilli**, "sempre da doppio tile", che andrebbero segnati con un altro colore. Lo screenshot scelto, però, non ne aveva uno ben visibile: "avremmo dovuto scegliere uno screen con un coccodrillo, mia colpa";
- le **tartarughe** e i **tronchi**, ciascuno con una meccanica specifica;
- le **tane**, cioè l'arrivo, e le caselle che servirebbero citate tutte;
- le **caselle viola**, che sono caselle neutre, sicure;
- le **vite** a schermo.



1 rana (personaggio principale)
2 auto e camion: ostacoli, stessa categoria, sprite diversi
3 tronchi: piattaforme che scorrono
4 tartarughe: piattaforme che possono immergersi
5 coccodrillo ("da doppio tile")
6 fasce sicure (partenza e mediana)
7 tane: l'arrivo
8 vite rimaste
9 tempo

Movimento: un tile alla volta, in alto, in basso, a destra, a sinistra.
Morte: investiti, caduti in acqua, tempo scaduto.

Figure 9.2: Ricostruzione schematica dell'esercizio del docente: lo schermo di *Frogger* con gli elementi cerchiati in colori diversi e numerati, da affiancare alla lista delle regole. Non è lo screenshot originale; posizioni e numero degli oggetti sono indicativi.

Sulle caselle il docente farebbe un discorso generale: la rana si muove **di tile in tile**, sopra, sotto, a sinistra, a destra, ma sempre di un tile alla volta; e ci sono tile **sicuri**. Uno studente ricorda le **condizioni di morte**: "assolutamente". Il docente le segnerebbe in maniera **visiva**.

L'immagine non è una decorazione. Lo screenshot va messo accanto al testo, ma non come decorazione: **interrompe il testo** perché **è** il contenuto. Il docente prova a inserirlo in una tabella ("una delle tabelle maledette"), che però si allarga troppo; alla fine lo mette in coda alla sezione. Mangia metà pagina, "e siamo d'accordo su questo"; ma permette di **fare riferimento in modo specifico**, nelle regole, agli elementi che si stanno già mostrando.

La legenda con i simboli colorati. Manca un titolo, e la lista andrebbe fatta come elenco puntato o tabella. Accanto a ogni voce, invece di ripetere il nome, il docente inserisce un **carattere speciale** (un quadratino) colorato come il contorno corrispondente nell'immagine, con una nota del tipo *element in the image*: una legenda più compatta e più chiara. Con un'avvertenza: i **caratteri speciali**, quando il documento viene esportato in PDF o stampato, potrebbero essere "completamente disintegrati", e non apparire come li si vede sullo schermo. Per i task di ora non è molto rilevante, ma va saputo.

Le interazioni e le risorse. Lo stesso approccio vale per le **interazioni**: si prende un'immagine del cabinato di *Frogger*, si ritagliano i controlli e si spiega a che cosa serve ciascuno ("e le monetine a cosa servono?"). E per le **risorse**: le si cerchia tutte con lo **stesso colore** e le si elenca nella sezione corrispondente.

9.16.8 Una o due pagine

"Si parte dalla lista; poi, quanto più riusciamo a renderla **grafica**, visibile e comprensibile, meglio è." Questo è un modo, reso possibile anche da un colpo di fortuna: Frogger è a schermata unica, quindi quasi tutti gli elementi si possono citare su un'unica immagine. Andrebbe spiegato meglio, ma come prima iterazione funziona: e si vede già come sia **molto più chiaro**, e come in **una o due pagine** "ce la caviamo". È lo stesso principio del *mostrare invece di scrivere* visto con i flowchart (sezione 6.9) e nelle revisioni del primo task (sezione 1.4.9).

9.17 La chiusura

"Non abbiamo più tempo; anzi, vi ho tenuto qua anche troppo." Per le domande, ci si scrive in privato o se ne parla. La lezione successiva è **lunedì**, subito dopo il weekend, e si aprirà con le eventuali domande. Se c'è qualcosa di "imperativo, vitale" da chiedere subito, il docente resta qualche minuto; altrimenti, "ci vediamo lunedì".

Punti chiave del capitolo

- Un task **non letto** è perso nel conteggio, ma si può **riconsegnare** (fino al 20% dei task): copia o documento nuovo, mai modificare il vecchio né risolverne i commenti; integrare i propri commenti e quelli ai colleghi; cartella giusta, nome chiaro, avvisare il docente. Si riconsegna con ciò che si sa *oggi*, e non il giorno dopo.
- Il limite dei **cinque errori grammaticali** può essere sollevato per chi ha dislessia o difficoltà simili, parlandone in privato; ma in un CV o in un portfolio quegli errori contano.
- La **libertà** non esiste come termine: si cerca l'**agency**, la capacità di esercitare potere **entro dei limiti** stabiliti dal designer (tre starter, quattro mosse, la build iniziale). Stun lock e crowd control la tolgono.
- Troppe scelte bloccano: **paura di scegliere** e **rimorso**; il cervello non regge i grandi numeri (fallacia di Monte Carlo). Il *respec* aiuta, ma una scelta reversibile pesa meno.
- Il **paradosso della scelta** (Schwartz): più scelte, meno soddisfazione. Meno scelte ma **più profonde**: più soddisfazione, curva d'apprendimento più morbida, decisioni meaningful; meno costi, più iterazione, più creatività, più bilanciamento. Tre è un ottimo punto di partenza.
- La **Theory of Planned Behavior** spiega perché un comportamento viene adottato o no: **attitudine** (beneficial, enjoyable), **norme soggettive** (sociali, meta), **perceived behavioral control** (confidence, capability) portano all'**intenzione**; le **barriere** si frappongono tra intenzione e **comportamento**. Brilla nell'analisi da playtest.
- Formula semplificata: valori da -10 a $+10$; pesi di norma 1, fino a 2; $I = w_A(B + E) + w_N(S + M) + w_P\, PBC$; comportamento $= I + w_R R$ con w_R pari al **doppio della media** dei pesi, perché le barriere si percepiscono più pesanti. Con pesi 1 e PBC accorpato: da -70 a $+70$.
- I valori vengono da designer e **utenza**: servono le demografiche, e bisogna chiedersi da dove vengono i dati (Reddit non è il mondo). Nei survey, **agency oculata**: risposte chiuse dove si può.
- Il **feedback** rinforza o riduce i comportamenti: datene il più possibile, per ciò che va bene e ciò che va male. È potente e, nei sistemi gacha, rischioso.
- Il **compendium** (design bible) è uno per progetto: una mappa di link a tutti i documenti, divisa per dipartimento, con **status** (e legenda), essenziale, accessibile a tutti, aggiornata dai lead.
- I **pilastri** sono **tre**, ad alto livello, inequivocabili (God of War: *Combat*, *Father/Son*, *Exploration*); il team li deve avere sempre sotto gli occhi.
- La **vision** formalizza i pilastri: prime emotions e moods (tre ciascuno), eventualmente obiettivo e morale, **must do**, **must not**, **nice to have**, al massimo tre **USP**. Una volta approvata dalla direzione è **blindata**; funzione prima della forma.
- La **team vision** fissa missione, tipo di produzione, PR, gerarchia, pipeline, diritti e doveri: si firma, e si scrive **onestamente**.
- Le risposte di fine pre-produzione stanno nella **somma** dei documenti. I documenti mostrati sono **linee guida**, non template: ora è il momento di sperimentare.

- Nei core elements di *Frogger*: game qualities come sistemi di alto livello; il timing è una dinamica, non un obiettivo; niente celle vuote (TBD o barra); titoli con stili veri; tabelle con giudizio; il tema in un box; uno screenshot annotato al posto di un muro di regole.

Chapter 10

Il feedback: come darlo, come raccoglierlo, e i documenti che lo tracciano

La lezione riprende il filo del capitolo precedente (capitolo 9): i task su compendium e vision sono appena stati assegnati, e i feedback sui task di design (i *pain point* dei giochi di carte collezionabili) sono appena arrivati agli studenti. "Dato che ormai li avete ricevuti", dice il docente, "sembra doveroso iniziare a parlarne in maniera più approfondita". Quella di oggi è quindi la **lezione sui feedback**: come si danno, come si ricevono, come si raccolgono dai giocatori e come si leggono. La lezione finisce in anticipo, e il tempo avanzato viene speso in una "lezione jolly" su due documenti legati al playtest. La struttura, e quindi quella del capitolo, è questa:

1. le questioni in apertura: il task su **compendium e vision di *Metroid Dread***, la **legenda della formattazione** e un disclaimer sui task di design consegnati, cioè l'obbligo di **scendere nel dettaglio** (sezioni 10.1–10.3);
2. come **dare** un feedback: la premessa sulla severità, gli assunti di base, le regole del feedback costruttivo, la tecnica del sandwich (che non funziona), l'essere chiari, precisi e analitici, e alcune pillole aggiuntive (sezioni 10.4–10.8);
3. il **task** per la lezione successiva, con le concept art di un ipotetico gioco di zombie, e l'emozione del **fiero** (sezione 10.9);
4. dopo la pausa, come **raccogliere** i feedback: i test, il target audience, i dati, i **survey** e la scala di Likert (sezioni 10.10 e 10.11);
5. come **spacchettare** i feedback, con l'esempio del primo *God of War*, e come leggerli nei canali dei competitor (sezioni 10.12 e 10.13);
6. la **curva normale** dell'utenza e i consigli finali: problemi e non soluzioni, difendere la visione, *data informed* e non *data driven* (sezione 10.14);
7. la lezione jolly: il **versioning tracker** e il **documento di playtest** (sezioni 10.15–10.17);
8. la chiusura, sulla documentazione pratica e sul lavorare entro dei limiti (sezione 10.18).

10.1 In apertura: il task su compendium e vision

10.1.1 Documentare, finché non avrete di meglio da fare

Fatte le presenze (la lezione è in presenza per tutti, nessuno da casa), il docente ricollega la giornata alla precedente: la documentazione è particolarmente importante, "finché non lo diventerà più, o meglio, finché

avrete qualcosa di più rilevante da fare", senza però mai dimenticarla, perché è **alla base del progetto**. Prima della lezione vera e propria annuncia **tre argomenti** su cui espandere: il task in scadenza, un eventuale ritorno sugli strumenti di comunicazione e i task appena commentati.

10.1.2 Ricordatemi i task

Il primo punto è il task da consegnare "per venerdì prossimo, cioè per questo venerdì", assegnato il venerdì precedente: il **compendium** e la **vision** di *Metroid Dread*. Il docente lo aveva ricordato "proprio in calcio d'angolo", alla fine della lezione, e ne approfitta per fissare una regola: **se durante la lezione dice "poi vi do il task", gli studenti glielo devono ricordare**. "Non fate i furbi": se il docente se ne ricorda un giorno dopo, quel giorno gli studenti lo hanno perso. Quindi non bisogna esitare a fermarlo.

10.1.3 Il compendium di Metroid Dread

Qualcuno aveva chiesto, dopo la lezione, come riempire un compendium senza avere i documenti del gioco. La risposta: il compendium (sezione 9.9) **resterà relativamente vuoto**, ed è normale. Ciò che interessa al docente è che:

- lo si **divida in categorie**, cioè in **dipartimenti**: non tutti quelli immaginabili, ma i **dipartimenti principali** coinvolti;
- si inizi a pensare a **che cosa potrebbe esservi presentato** su questo progetto;
- ci siano gli **status** (anche solo potenziali) della documentazione, **con annessa legenda**;
- valga sempre la regola: **funzione maggiore di forma**.

Questa prima parte è relativamente semplice, anche perché almeno un link lo si ha di sicuro: quello alla vision che si deve scrivere.

10.1.4 La vision di Metroid Dread

La vision (sezione 9.11) deve invece contenere **tutte le informazioni** viste a lezione: principalmente i **must do**, i **must not**, i **nice to have**, gli **unique selling point** e soprattutto i **temi**, cioè i pilastri;[1] e poi le **prime emotions** e i **prime moods**, che "ci devono essere". Narrativa, setting e target audience si possono invece **tralasciare** per questa volta, perché richiedono una parte di pre-produzione che gli studenti non hanno.

Ciò che interessa al docente è l'**attenzione** a che cosa inserire nella vision e a come inserirlo, in particolare per i **pilastri**.

- In una produzione reale i pilastri si fissano **a priori**.
- Qui li si fa **a posteriori**: li si va a **estrarre dal gameplay**, giocando il gioco o guardandolo giocare.

E la regola del tre vale per tutto: **tre prime moods, tre prime emotions, tre pilastri e (al massimo) tre USP**.[2]

Non fate macerare i dubbi. "Domande? Tutto chiaro?" Il docente insiste: i dubbi non vanno tenuti per sé, "non facciamoli macerare". Parlandone, di solito non solo si risolvono, ma emergono altre cose interessanti o altri agganci utili.

[1] Nella trascrizione "MassDu, MassNot, Nice2Ev, i Mixed Endpoint e sicuramente temi cluster". Le prime tre voci sono chiaramente *must do*, *must not* e *nice to have*; "Mixed Endpoint" è quasi certamente una storpiatura di *unique selling point*, che con le altre tre voci compone la parte di design della vision (tabella 9.7); "temi cluster" sembra riferirsi ai temi, cioè ai pilastri.

[2] Nella trascrizione "tre prime mood, tre premie morton, tre pilastri e tre universali importanti"; gli "universali" sono quasi certamente gli *unique selling point*, per i quali la regola, vista nel capitolo precedente, è "al massimo tre".

10.1.5 Gli altri due punti

Il secondo punto, se ci fosse stato tempo, era tornare sulla **comunicazione**: esempi, strumenti da usare, come sfruttare tutto ciò che si ha a disposizione. "Ci sono mille modi diversi"; ma si potrebbe espandere su quello che mancava, cioè le sezioni dei core elements su **obiettivo**, **regole**, **risorse** e **interazioni** (capitolo 2). "Non è proprio rilevante, non è necessario", ma se è utile lo si può prendere. Il terzo punto sono i **task consegnati e commentati** (sezione 10.3).

10.2 Serve una legenda della formattazione?

Prima di passare ai task, Francesco pone una domanda sulla documentazione: **quanto è giusto inserire una legenda** che spieghi gli strumenti di formattazione che si usano (colori, grassetti, evidenziature)? È utile, o "copre spazio e basta"?

Dipende da quanto siete chiari. La risposta è: dipende, prima di tutto, da **quanto è immediatamente comprensibile** per il lettore il codice che si usa. Se il rosso si usa solo per i nemici o per gli ostacoli di gioco ("dico un esempio a caso"), quanto è evidente?

Dipende da quanti elementi usate. Se ci sono **acronimi**, nomi specifici che si legano ad altri elementi del documento (come nel caso degli **status** del compendium), la legenda diventa **necessaria**. Se si usa un **enorme numero** di strumenti (corsivo, grassetto, sottolineato, evidenziatura, N colori diversi), allora potrebbe esserlo. Non c'è un numero preciso.

Una legenda della formattazione a monte? Se lo stile fosse davvero **codificato** ("usiamo sempre il rosso per una data cosa"), si potrebbe pensare a un **documento a parte**, una legenda a monte che spieghi tutto lo stile di documentazione: "un po' un production dictionary per la formattazione" (sezione 4.3). Ma il docente precisa due volte di **non averlo mai visto**, e che un documento del genere assume che **tutti usino lo stesso stile**, il che non è sempre vero. Ogni designer ha il proprio stile di documentazione: anche quando ci sono limiti stringenti (lo si vedrà con il **GDD**, in cui alcuni paragrafi devono esserci per forza), "ho sempre visto grande margine di differenze creative o stili di formattazione".

L'obiettivo è non renderla necessaria. Una legenda può essere utile; l'obiettivo, però, è **scrivere in modo che non serva**. L'unico caso in cui il docente la ritiene generalmente utile è quello dei documenti **veramente complessi**, grossi e con tante variabili: i **documenti di bilanciamento**, gli Excel enormi, in cui "il primo foglio è generalmente atto a spiegare questa roba qua", con una legenda sempre visibile immediatamente.

10.3 Il disclaimer sui task: il design va descritto nel dettaglio

10.3.1 Un design amputato

Sui task consegnati, il docente fa un disclaimer "di venti secondi" (che dura parecchio di più). Il task era un **primo tentativo di analisi, con conseguenti soluzioni di design** proposte dagli studenti. Il docente non si aspettava soluzioni *groundbreaking* o modi di fare design esagerati; e nella parte di **analisi** ci sono spunti più o meno corretti. Ma la parte di **design**, e questo vale per tutti, **mancava di pezzi specifici**: "come se fosse proprio amputata in una parte". Non è un problema, adesso: si vedrà come si fa.

Documentare qualcosa che non esiste. Quando si documenta una soluzione che non esiste ancora (perché la si è creata, o perché la si prende da altrove), bisogna essere **specifici, puntuali, precisi**: è la solita questione della documentazione. E il modo migliore per farlo è dare **esempi concreti**, visibili, magari in **tabella**.

10.3.2 L'esempio del battle pass

Alcuni studenti avevano proposto un nuovo sistema di progressione o di *reward* appoggiato a esempi noti: i *daily reward* legati al login, o un **battle pass**. Anche nella versione più macroscopica, va specificato:

- da **quanti elementi** è composto, cioè da quanti **livelli**, almeno indicativamente;
- **quanta esperienza** serve per un livello, magari non esattamente, ma almeno se è **la stessa** per ogni livello o se **cresce**;
- **che cosa succede quando il battle pass viene finito**.

I casi limite. Molte soluzioni descrivevano solo **il caso migliore**, quello in cui tutto va come previsto. "Ed è questo che manca a molti designer": bisogna pensare anche ai **casi limite**. Il giocatore che finisce tutto il battle pass, poi che fa? Può continuare a *grindare* per una ricompensa, oppure no, non succede nulla: va bene tutto, ma **c'è una scelta da fare, e va esplicitata**. Sono gli stessi casi limite visti nella ricerca, nel brainstorming e nella generazione delle idee (capitolo 6).

10.3.3 Nuove UI e nuove carte

Lo stesso vale per qualsiasi nuova **UI**: che cosa contiene, come ci si interagisce. E per una **nuova carta**: qual è l'effetto? Nel caso dei giochi di carte del task non è particolarmente interessante avere il numero esatto di attacco e difesa, o (per le carte di *Marvel Snap*) il costo in mana e l'attacco bilanciati. Ma se si inserisce un **effetto innovativo** o una **nuova parola chiave**, quella **va spiegata**.

10.3.4 Fare design significa trattare tutti i dettagli

Ci si arriverà piano piano, fino al **game design document**, che si vedrà l'anno successivo ("2026, sempre con me, non ora"). Ma va capito subito che **fare design significa trattare tutti i dettagli**. È il motivo per cui il docente è "così rompipalle" su alcuni elementi dei task: essere precisi, non usare a caso tutti gli strumenti, non lasciare spazio sprecato.

Non un wall of text. Elencare tutti i dettagli non significa scrivere un *wall of text*, ma **mostrarli**: per esempio con i **flowchart** appena visti (sezione 6.9), che si possono usare nella documentazione di design. Si tratta di trovare il modo migliore per mostrare un sistema nel modo più chiaro possibile **a qualcuno che non lo conosce** e che eventualmente lo dovrà **implementare**.

I programmatori fanno di testa loro. Il caso tipico è il programmatore: "se non glielo dite, fanno di testa loro". Ed è "la peggior causa possibile": inseriscono una feature che **non è la vostra**, e di cui poi **siete voi responsabili**.

Il tallone d'Achille. Spiegare il design nel dettaglio è "il tallone d'Achille"[3] di molti designer, ed è ciò che **differenzia una persona competente da una non competente**.

Il test dell'implementazione. L'invito finale è a ritornare sulle proprie soluzioni e chiedersi se sono chiare. Visto che gli studenti hanno da poco iniziato a programmare, il test è questo: *se dovessi implementare io questo sistema, avrei tutti gli estremi per scrivere il codice e inserirlo nel gioco? E un giocatore che lo giocasse avrebbe tutti gli estremi per farlo? Ho considerato tutte le possibili interazioni?* Non è esattamente ciò che il task chiedeva, "però fare design poi significa questo".

[3] Nella trascrizione "il pallone da killer", presentato dal docente come "una grande citazione, colta o non colta, a seconda dei casi". È quasi certamente il *tallone d'Achille*.

10.4 Una premessa sulla severità

10.4.1 Niente di personale

Si entra nella lezione sui feedback ("*feedback*, non traducibile in italiano, purtroppo: che peccato"). Il docente fa subito una premessa che con le classi precedenti faceva più tardi: va contestualizzata la **severità** dei feedback dati e ricevuti, perché non venga "assorbita" nel modo sbagliato.

- **Nulla** di ciò che il docente dice o commenta è rivolto **alla persona**.
- Tutto ciò che fornisce è **in funzione della crescita** dello studente.
- Si può discutere su un certo tipo di severità, ma è **necessaria**, "ed è per il vostro bene": l'obiettivo è migliorare il lavoro, far raggiungere uno step successivo.

10.4.2 Il mondo là fuori

L'obiettivo è **preparare a un domani** in cui, nel migliore dei casi, si è già abituati a ricevere un certo tipo di feedback e di commenti, perché alcuni di questi **non ricadono esattamente nel feedback costruttivo**: si potrebbe non avere il privilegio di riceverli in maniera corretta, "e ve lo dico per esperienza personale". In quel caso bisogna aver sviluppato la **pelle dura** nel proprio percorso, oppure trovare il modo di "smarcare" la cosa, riassestarsi e salvare il proprio posto, la propria sanità mentale e quella di tutti. Non è esattamente responsabilità del designer, ma è un problema in cui si potrebbe incorrere.

Non restare attivati. C'è poi una tendenza, più frequente in chi è più giovane, a **restare attivati**: ad assorbire ciò che si legge o che viene detto come un attacco personale. Cosa che, ripete il docente, non vuole assolutamente essere il caso.

10.4.3 Che cosa si vedrà

La lezione tratterà come **dare** ed eventualmente **raccogliere** feedback: prima in maniera generica, all'interno dei progetti, come professionisti; poi nelle **sessioni di playtest**, momenti più circoscritti. E c'è un altro beneficio: saper dare feedback aiuta a **collaborare**. Finché non si sa parlare correttamente con gli altri, soprattutto con chi fa un lavoro diverso dal proprio, è difficile, o rischioso, avere team grossi e sapere come aggiustare il tiro al loro interno.

Il meme. La slide di apertura mostra un gatto che porge una rosa ("questo non è un meme, sono io che vi porgo l'altra guancia"): il gatto è il docente, dall'altra parte ci sono gli studenti che ricevono i suoi commenti e le sue mail, "che sono 230.637". "Non è un feedback, comunque."

Un precedente. Gran parte di questi temi era già stata toccata, da un'altra angolatura, nel seminario sulla cultura del feedback (capitolo 0.1, in particolare le sezioni 0.1.13 e 0.1.14): qui il punto di vista è quello del designer, nel team e davanti ai giocatori.

10.5 Gli assunti di base

10.5.1 Dall'altra parte c'è un umano

L'assunzione di base nel comunicare, nel lavorare in gruppo e nel far capire agli altri se hanno fatto bene o male è che **dall'altra parte c'è un essere umano**. "Non so se avete mai provato a controllare se avete un esoscheletro": per ora si parla di umani che parlano ad altri umani, "poi magari in futuro cambierò le slide". Quindi serve un certo tipo di **empatia**.

Egoless, ma umani. È vero che il design è *egoless*; ma chi fa design è un essere umano, e ci saranno spesso momenti in cui si parlerà in maniera umana. Ci sono contesti lavorativi regolati, come quello dei feedback, e altri meno regolati, ed è bene così: un grande motivo per lavorare in questo campo è che le persone tendono ad avere **molte cose in comune**, ed è più facile fare *bonding*, stringere amicizie. Ma resta il fatto che dall'altra parte c'è qualcuno che assorbe ciò che gli si dice, e viceversa.

10.5.2 Le soft skill

Il secondo assunto, "la cosa più importante", poco rilevante al primo anno ma estremamente importante dal secondo in poi, sono le **soft skill**. La slide ne mostra una lista "non omnicomprensiva", con quelle illustrazioni dall'estetica aziendale di cui il docente ha appena scoperto che esiste un nome ("mi fa morire"):[4]

- comunicazione chiara;
- capacità di ascolto;
- autocontrollo ("la capacità di non uccidere altre persone");
- attitudine positiva;
- assertività;
- risoluzione dei conflitti;
- empatia;
- depersonalizzazione;
- prendersi le responsabilità;
- un pizzico di humor.

Nel team ci si compensa. La lista delle soft skill è enorme, e non tutti devono essere spiccatamente positivi o avere un grande senso dell'umorismo: all'interno del team **ci si compensa**.

La prima discriminante. Ma questa è, "purtroppo, per fortuna anche qui", **il primo elemento considerato per essere assunti o licenziati**. Le persone che vogliono lavorare con qualcuno non lo fanno principalmente perché è un bravo tecnico o ha estreme skill di design, ma per le sue **soft skill**. Le competenze (level design, narrative, programmazione, game design) ci devono essere; ma la prima discriminante sono queste, come lo sono per essere mandati a casa: se si alzano le mani su qualcuno o lo si insulta, la cosa viene punita, con una lettera di richiamo o con il licenziamento, a seconda dell'azienda.

Con gli altri dipartimenti. Le soft skill diventeranno notevolmente più importanti nei **progetti di gruppo**, soprattutto quando questi si proietteranno su altri dipartimenti: arte, programmazione, narrativa. Lì non si parla solo con altri esseri umani, ma con esseri umani che potenzialmente **non condividono il proprio linguaggio tecnico**.

10.5.3 I feedback sono costruttivi

Il terzo assunto è che **i feedback sono costruttivi**. "Non siamo qui per dire 'il tuo lavoro è una merda'": siamo qui per formulare un feedback che sia **recepito in modo chiaro** e che vada a **migliorare il lavoro dell'altro**, o il nostro, se il progetto è condiviso.

[4] Il docente dice "corporate qualcosa". Lo stile di illustrazione piatta, con figure dalle membra allungate, molto diffuso nella comunicazione aziendale, è comunemente detto *Corporate Memphis*.

10.5.4 Feedback 101

Le cose di base ("*basic things to say*") sono tre.

Precisi e rispettosi In ciò che si dice si è **precisi e netti**. Questo **non** significa nascondere le cose, o non parlare se c'è un problema: è tutto il contrario. Ma nel dirlo si è **rispettosi** del lavoro degli altri, del tempo dedicato, e così via. Si è **onesti**, e dall'altra parte si sa **ascoltare**, con l'umiltà di capire che cosa viene detto. Si è **professionisti** e **composti**: non lo si dice arrabbiandosi o ribaltando il tavolo, ma nemmeno abbracciando il collega e dandogli pacche sulla spalla.

Empatia Il discorso dell'essere umano. Come metro di misura, si cerca di capire **in che situazione è il collega**: ha delle difficoltà? Gli è stato rigettato il task ("nel vostro caso ventisette milioni di volte")? Ha dedicato tanto lavoro e tanto impegno, ma ritorna agli stessi problemi? Si cerca il punto critico che sta passando. E si **formula il feedback nella propria testa** prima di darlo: se lo dicessero a me, c'è il rischio che venga assorbito in modo problematico, o frainteso?

Coltivare Tutto questo **va coltivato**.[5] Chi ha già lavorato in team, o vive in un contesto con molte persone (una famiglia numerosa e stimolante, una grossa cerchia di amici che si vedono spesso), è avvantaggiato, per tendenza naturale o per esperienza; ma non c'è nulla che non si possa coltivare. Il picco, "perché poi alla fine questo dovrete andare a fare", è la relazione con gli altri in un contesto **formale**, quello lavorativo.

Iniziate a farci caso. Diventerà più importante al secondo anno, ma conviene iniziare subito a pensare a come si parla con i colleghi, a quanto tempo si passa nei propri gruppi ("gruppi illegali in cui siete con persone che non ci devono essere, non lo so, dipende da voi"), a come ci si relaziona agli altri, e a quanto ci si fa caso. È la discriminante principale per cui le persone vogliono o non vogliono lavorare con qualcuno: perché si trovano bene, perché possono fare affidamento su di lui se hanno un problema imprevisto, su ciò che sa fare e su come lo sa fare. Si diventa **un ponte** tra sé e gli altri: è l'obiettivo del lavoro di gruppo, ed è anche la difficoltà dei team grandi ed eterogenei.

10.6 Come si compone un feedback

Fissati gli assunti ("il primo passettino"), si passa a come si compone il feedback in sé. La tabella 10.1 riassume le cinque regole; le si vede una per una.

Table 10.1: Le regole del feedback come presentate a lezione.

Regola	In pratica
Costruttivo	preciso, chiaro, analitico; informazioni chiare, senza giri di parole; il termine specifico, non quello più oscuro
Neutrale	segnala gli aspetti negativi **e** quelli positivi, e perché; lo fa in uno stato emotivo neutro
Sincero	si assume che tutti siano adulti e professionisti: il feedback serve al miglioramento, e lo si riceve in modo neutrale
Risolutivo	dopo il problema, una **soluzione attuabile**, suggerita e non imposta, con margine di manovra
Senza giudizio	niente "bravissimo", "ottimo", "fa schifo"; attenzione soprattutto con i dipartimenti che non si conoscono

[5] Nella trascrizione il titolo del terzo punto è illeggibile ("Riso scritto"); il contenuto, che "incapsula" i due precedenti, è chiaramente la necessità di coltivare queste capacità relazionali.

10.6.1 Costruttivo

Si assume che il feedback sia costruttivo, e quindi valgono le solite regole della documentazione: deve essere **preciso, chiaro e analitico**. L'obiettivo è dare, in modo ancora più asciutto che in un documento, **informazioni chiare, senza giri di parole**. Se esiste un **termine specifico** per una cosa, si usa quello, e non il termine più oscuro o l'italiano più arcaico "per sembrare colti". "Dovete essere capiti: questo è il concetto."

Una confessione sui feedback del docente. Riallacciandosi alla premessa, il docente ammette che **alcuni** ("non molti, alcuni") dei suoi feedback sono **volutamente manchevoli di informazione**: gli interessa che lo studente faccia il lavoro extra di capire dove fosse il problema e quale, tra le mille soluzioni possibili, fosse quella adeguata. "Non nascondo che anche dal lato mio mi salva del tempo." Ma non è l'ideale: l'ideale è aiutarsi nel modo migliore possibile. Quindi, per tutto ciò che non è comprensibile o sembra manchevole, **scrivetegli**: o darà l'informazione, o dirà che non la può dare, e allora si è in quel caso.

10.6.2 Neutrale

Il feedback **non sottolinea solo gli aspetti negativi**: può e deve sottolineare anche quelli **positivi**, che magari si possono comunque migliorare. Ce ne si dimentica spesso, per due ragioni.

1. **Fattibilità**: se qualcosa funziona, generalmente lo si lascia stare e ci si concentra sul negativo, per portarlo a funzionare. "Squadra che vince non si cambia: si migliora, ma non si cambia." La priorità ce l'hanno le cose che non funzionano.

2. **Natura umana**: "siamo di tendenza bastarda come esseri umani", ed è più facile sottolineare il negativo che il positivo.

Ma, sempre per empatia, gli aspetti positivi **che non vanno cambiati** vanno detti, perché chi riceve il feedback potrebbe non dare per scontato che quella cosa funzioni, e cambiarla, magari in peggio. Quindi: **quello che funziona, ditelo; quello che non funziona, ditelo; e dite perché**.

Uno stato emotivo neutro. Queste cose vanno dette in uno **stato emotivo neutro**, nel modo più umano possibile. Non tutti sanno tenere al guinzaglio le proprie emozioni, "però ci sono dei libri". Il problema tipico è che il negativo viene detto in modo **rabbioso, violento, viscerale**; e, all'opposto, il positivo come se fosse il più grande capolavoro del mondo. Positivo e negativo vanno **percepiti nello stesso modo**: una cosa positiva funziona, va bene, o si migliora in un certo modo; una cosa negativa non è un problema, va sistemata. Il risultato è lo stesso: lavorare.

10.6.3 Sincero

La sincerità non è solo esprimere sinceramente che cosa funziona e che cosa no: è l'assunzione, "ed è il motivo per cui questa lezione la facciamo a tutti, e bisognerebbe farla in tutti gli altri corsi", che **tutti siano adulti e professionisti**. In un contesto lavorativo, in cui si deve migliorare il progetto e trovare i problemi, se si dà un feedback non lo si dà "per farti rifare il lavoro perché sono bastardo", ma perché l'obiettivo è il **miglioramento**: personale (nel rapporto docente-studenti) o del progetto (tra colleghi). Di conseguenza, chi riceve un feedback si mette nell'ottica di recepirlo **in modo neutrale**, senza esserne intaccato personalmente. E viceversa.

10.6.4 Risolutivo

Sottolineato il problema (o la cosa che funziona), **ci deve essere un dopo**. Manca spesso un pezzo, ed è il motivo per cui molti commenti **non sono proprio feedback**: la **parte risolutiva**. Emerso l'elemento in questione (la mancanza di un dato, la mancanza di una reference, nell'esempio del design), **qual è la soluzione attuabile?** Come si può migliorare? È "l'aggancio che spesso manca", e va dato nel modo più esplicito possibile.

Suggerire, non imporre. Con un feedback si deve **suggerire una strada**, non imporla: "attenzione". Più ampio è il ventaglio di possibilità che si mostra, meglio è. E dall'altra parte, tornando alla sincerità tra adulti, ci deve essere la maturità di capire che è **un consiglio e non un obbligo**: chi riceve il feedback valuta se quella strada è giusta, o se può "rubare un pezzo di idea" e implementare qualcos'altro che risolva comunque il problema. Nella soluzione ci deve essere **margine di manovra**.

10.6.5 Senza giudizio

Soprattutto quando si danno feedback, si cerca di **non dare giudizi**: niente "bravissimo", "ottimo", "fa schifo". Il docente cerca di farlo sempre nel modo più contenuto possibile, "anche se siamo umani". Il giudizio può funzionare come un pizzico di sale ("*as a sprinkle*"): sempre per empatia, per **potenziare il morale**, o per ridimensionare chi è estremamente egocentrico e "si monta la testa". Ma sono strategie **veramente al limite**.

Con i dipartimenti che non conoscete. È molto facile cadere nei giudizi soprattutto con i dipartimenti di cui si sa poco: non si sa quanto ci ha messo un animatore a fare quell'animazione, né quanto è facile cambiarla se gli si suggerisce di farlo in poco tempo. E diventa estremamente importante **più si sale nella piramide gerarchica**, "ve lo dico per esperienza": da lì possono scaturire estrema maleducazione o estrema sottovalutazione del tempo dedicato.

L'errore peggiore dei producer. "Visto che abbiamo qui il nostro invitato speciale, anzi, intruso speciale" (in aula c'è un ospite legato alla produzione), il docente nomina il peggior errore che un producer possa commettere: **non avere conoscenza del tempo pratico** che servono le cose, e **non fidarsi** degli altri all'interno del team.

10.6.6 La tecnica del sandwich non funziona

Spesso viene citata la **tecnica del sandwich**, o del panino: si comincia con un feedback positivo, ci si infila dentro "un kebab a tre euro di schifo" (tutte le cose negative, magari caricate nel modo sbagliato) e si chiude con un altro feedback positivo "per togliere il sapore di kebab marcio dalla lingua".

Non funziona. È esattamente il problema di *no judgment* e di *neutrality* visto prima: non è feedback costruttivo, non è una tecnica attuabile. Il feedback deve essere **chiaro**: non lo si deve nascondere, seppellire o rendere piacevole da digerire. Se riguarda un aspetto negativo, e l'aspetto è **critico**, lo si deve far emergere e mostrare **quanto** è critico. Sempre in modo neutrale: "è meglio una lista chiara che un pappone panino indigeribile" (figura 10.1).

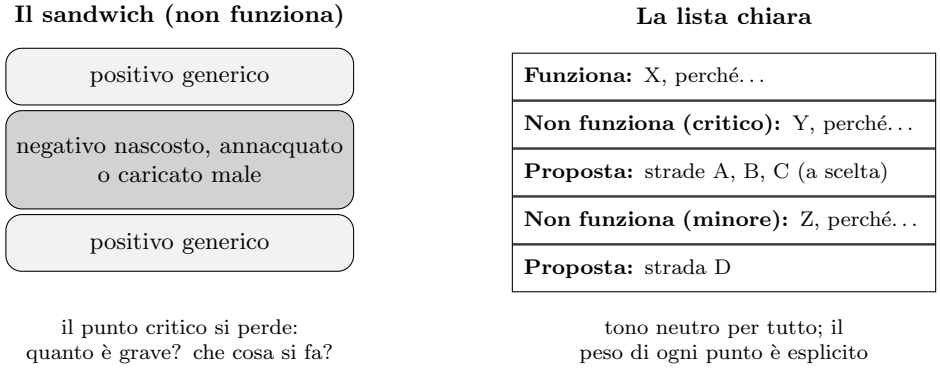


Figure 10.1: La tecnica del sandwich contro una lista chiara: ricostruzione schematica del confronto fatto a lezione, non la slide originale.

10.7 Chiari, precisi, analitici

"Fin qui tutto chiaro, intanto?" Il docente torna sulle tre qualità del feedback costruttivo e spiega come ottenerle.

10.7.1 Chiari

Evitare terminologie complesse e termini oscuri **per il solo motivo di sentirsi più intelligenti**. Il docente ammette di avere proprio questo problema: parla spesso in termini complessi e con metafore oscure. Se si ha questa tendenza, si **riformula**.

- Se qualcosa non viene capito, o si ha il dubbio che l'altro non stia capendo, lo si dice in un altro modo.
- Non solo lo si deve chiedere: **l'altro lo deve dire**, se non ha capito.
- Se non è chiaro, si trova un altro modo, finché **tutti e due non sono soddisfatti**.
- Riformulare non significa **ripetere uno a uno**: dire due volte la stessa frase non serve.

10.7.2 Precisi

Essere precisi significa **dare le reference**. Se si fa riferimento a un titolo o a una soluzione già adottata altrove, si dice **dove, esattamente quale**: si mostra il **link al minuto corretto** di un gameplay, si fa "la biblioteca", si prende un'immagine e la si fa vedere.

Fermarsi a guardare insieme. Se si discute di una meccanica con un altro designer e l'altro non ha chiaro di che meccanica o di che gioco si parli, **ci si ferma**, si va a vedere e si mostra: ci si prende il tempo per guardarlo e osservarlo insieme. Quel tempo è **necessario**.

Dare il tempo di assorbire. Per lo stesso motivo, se si dà una reference ("leggi questo libro", "gioca questo gioco"), non ci si aspetta la risposta pronta il giorno dopo: si dà all'altro il **tempo per assorbire** ciò che gli si è dato.

10.7.3 Analitici

Essere analitici significa analizzare le reference date e descrivere le cose **oggettivamente**, senza dimenticare l'empatia. Le domande da farsi sono queste.

- **Quale reazione viene evocata** da ciò che si analizza?
- **Qual era l'obiettivo** di ciò che si sta valutando? È un prototipo? E qual è l'obiettivo di quel prototipo? Un prototipo probabilmente non sarà esteticamente bello: avrà i cubi, avrà gli SFX placeholder. Ma il suo obiettivo era magari testare una meccanica di sparo, di salto, una meccanica specifica: "magari ci stiamo concentrando sulla cosa sbagliata". Questo, peraltro, dovrebbe dirlo chi consegna il prototipo.
- **Qual era l'intenzione** di chi l'ha creato?
- **Come questa cosa raggiunge o manca l'obiettivo?** È facile dire "non mi piace, non rientra qui dentro". È diverso dire: *questa meccanica non funziona all'interno del gioco perché non è allineata con la vision*, oppure *non rispetta un must do, o un must not.*
- Ci sono **dati** a supporto? Se ci sono, vanno sempre inseriti. Spesso non sarà il caso: si viene presi alla sprovvista, fermati in un momento in cui si deve dare il feedback sul posto, a maggior ragione davanti ai giocatori o in una fiera videoludica.
- **Come lo percepirà il target audience?** L'obiettivo finale è sempre ciò che si sta testando **in mano al giocatore ideale**: funziona o non funziona, lo si può già immaginare.

Il meme della performance review. La sezione si chiude con un meme: una *performance review* fatta di due pollici in su ("anche questa l'ho vissuta"; "ma no, è stata cambiata la questione, chissà se è ancora cambiata") e una slide di "*amazing tips and tricks*. Wow wow wow".

10.8 Alcune pillole aggiuntive

10.8.1 Siate veritieri: non state zitti

È il concetto delle soft skill e dell'**individualità del designer**: in un team di designer si è stati assunti **per quello che si è**, per ciò che si porta al tavolo, per l'aggiunta personale al team. Quindi bisogna essere **onesti con sé stessi**. Se c'è qualcosa che non funziona o non convince, non per istinto ma **per una ragione specifica**, **non si sta zitti**.

Anche se siete riservati. "Io capisco che alcuni di voi sono più riservati, meno espansivi, meno estroversi; mi ci metto anche io." Ma non si sta nell'angolino di un meeting se si vede qualcosa che non funziona. Per quanto scomodo sia, se non lo si dice **in quel momento**, il problema diventa **un mostro**: è l'**effetto valanga**. Un sasso che non viene mostrato e di cui non ci si preoccupa adesso diventa poi una cosa gigantesca, che può portare **interi progetti a essere cancellati**. "Se c'è un problema, alzate la mano: non abbiate timore di farlo." E, di nuovo: niente sandwich. Se qualcosa non funziona, lo si dice.

10.8.2 Teamwork: un team, non una famiglia

Si lavora in gruppo, "siamo un team, **non una grande famiglia**": si cerca di creare coesione **all'interno dell'orario lavorativo**.

- I **fallimenti** si spalmano su tutti: ci si divide il carico.
- I **successi** si festeggiano tutti insieme, ma si riconoscono anche i **meriti del singolo**: se qualcuno ha prodotto un lavoro eccezionale, lo si dice.
- Se c'è un problema nel team, **è di tutti**. Non è che c'è un bug, il programmatore lo fixa ed è responsabilità sua: qualcuno l'ha fatto emergere, qualcuno ha testato il gioco, i designer si sono confrontati. Se c'è un problema, ci si unisce e lo si risolve.

Questo si vede in come si affronta il lavoro tutti i giorni e in come si parla con gli altri: le responsabilità ce le si prende, ma **per il bene del team**. Si cerca di creare un senso di gregarietà. Non è facile, e più il team è grosso più diventa difficile; ma non va dimenticato.

10.8.3 Le nomenclature: una truffa che funziona

"Questa è una truffa, lo dico subito, mettiamo le mani avanti. Però spesso funziona." La fanno generalmente le **HR**, o i lead: se c'è qualcosa di poco piacevole, **lo si maschera** con un nome diverso. Non vale per i feedback in sé, ma per esempio per il **meeting** del lunedì, o per il meeting dei designer in cui tutti si allineano, "che era una roba estremamente pallosa".

Leggere tutta la documentazione. Il docente racconta che, quando è in un progetto, **legge tutta la documentazione di design** che esiste. "Non scherzo": in Stormind ci sono anni e anni di documentazione storica, e lui ha letto qualsiasi documento di design mai creato, anche su progetti mai usciti. Quindi è sempre allineato. Ma "è una follia", non è richiesta al singolo; e ci sono designer che, per necessità o per tendenza, hanno bisogno di momenti di allineamento. Allora, se l'allineamento è pesante, **lo si chiama in un altro modo**.[6] Lo stesso vale per le **review di fine anno** ("che non sono i due pollici in su"): di solito si trova un modo più carino perché siano digerite.

[6] Nella trascrizione "invece di design meeting, design thinker"; il nome alternativo esatto proposto sulla slide non è ricostruibile. Il senso è chiaro: un nome più leggero per lo stesso meeting.

Scegliere il termine meno pesante. Se si deve scegliere tra due terminologie, si sceglie quella **meno pesante**. *Design critique*, per esempio, potrebbe essere problematico: la parola "critica" non è l'ideale se si sta facendo un allineamento di design.

Il framing. È un concetto "al limite della truffa", ma **il framing è un tool potentissimo** ("non potente: potentissimo"), e lo si vedrà meglio al secondo anno: **vestire un termine**, trovargli un altro nome, un'altra veste. L'esempio migliore "senza spoilerare niente del secondo anno" sono le taglie di **Starbucks** (tabella 10.2). Non c'è la tazzina, c'è il *demi*; non c'è il piccolo, c'è lo *short*; il medio è il *tall*; il *grande* "non è il nostro grande, è il loro grande"; e poi il *venti* e il *trenta*. Nomi unici, per vendibilità.

Table 10.2: Il framing delle taglie di Starbucks, come citato a lezione. Le capacità sono indicative (quelle tipiche del mercato statunitense) e non vengono dalla lezione.

Nome Starbucks	Ciò che "in realtà" è	Capacità indicativa
Demi	la tazzina	3 oz, circa 90 ml
Short	il piccolo	8 oz, circa 240 ml
Tall	il medio ("non è il medium")	12 oz, circa 350 ml
Grande	il grande, "il loro"	16 oz, circa 470 ml
Venti	più grande ancora	20 oz, circa 590 ml
Trenta	il più grande (solo bevande fredde)	30 oz, circa 890 ml

La truffa nella truffa. Dentro le taglie ci sono altre truffe: alcune, rispetto al costo, sono **meno vantaggiose** per l'acquirente ("non so dire quali, perché non ci vado: bisognerebbe analizzare i dati"), così che, per ottimizzare la spesa, si è spinti a prendere la più piccola o la più grande. È lo stesso concetto che usano i **cinema** per i popcorn.[7]

Nei giochi, e fuori. Il framing nei giochi è, per esempio, **chiamare il mana in un altro modo**: trovare un'altra nomenclatura per far passare un concetto, o meglio, "far passare un altro concetto per vendere o per manipolare l'utenza". Ma il framing "controlla le nostre vite": il dentifricio approvato da **nove dentisti su dieci** (quali dentisti? su quale bacino?), lo spray che elimina il **99,9% dei batteri** ("e io sono un grande amante delle pulizie"). Si avrà supporto, in questo, dal **narrative design**, che dovrebbe essere la sua specialità; e il reame proprio è quello delle HR. Ma nei team piccoli può essere necessario adottare queste soluzioni da sé.

10.8.4 Il linguaggio del corpo

Si può essere maestri nel dare un feedback neutrale e intanto essere **paonazzi in volto** e battere il pugno sul tavolo; o, all'opposto, mostrare in un meeting visibile **noia**, disinteresse, sonno, sbuffi. Bisogna **trattenersi**: ciò che si proietta con il corpo conta quanto ciò che si dice.

Se non siete neutri, rimandate. Se in quel momento si è particolarmente arrabbiati (non si riesce a fixare un bug, non si capisce un problema, si è appena ricevuto a propria volta "un cazziatone"), e si deve dare un feedback, ci si **ferma**, magari **lo si rimanda**. Quando si dà un feedback si cerca di essere nello stato più neutro possibile, "siamo sempre umani": non sprizzare gioia da tutti i pori, non essere troppo tristi, non essere arrabbiati. "Siate neutrali: questo è chiaro?"

Approfondimento: le dieci chiavi della comunicazione nel team. Jesse Schell, nel capitolo sul lavoro in team, riduce la buona comunicazione a dieci chiavi, che ricalcano quasi una per una le regole viste a lezione.[8] La prima è l'**oggettività**: le discussioni devono vertere su quanto un'idea risolve il problema, non

[7] Il meccanismo è noto in economia comportamentale come *effetto esca* (*decoy effect*): un'opzione intermedia volutamente poco conveniente rende più attraente quella più grande.

[8] Jesse Schell, *The Art of Game Design: A Book of Lenses*, terza edizione, capitolo 26 (*The Designer Usually Works with a Team*), paragrafo *Team Communication* e lente #101, *The Lens of the Team*.

sulle preferenze personali; non si parla di "idea mia" o "idea di Sue", ma "dell'idea dell'astronave"; e, trucco che Schell attribuisce a Randy Pausch, si formulano le alternative come domande: non "A non va, preferisco B", ma "e se facessimo B invece di A?". Seguono la **chiarezza** (verificare che gli altri abbiano capito, illustrare le idee, non fingere mai di aver capito, e soprattutto essere concreti: "ti mando entro giovedì alle 17 tre o cinque pagine sull'interfaccia del combattimento a turni" invece di "faccio il combattimento entro giovedì"), la **persistenza** (scrivere le cose), il **comfort**, il **rispetto** (non interrompere, non alzare gli occhi al cielo; e, quando tutto il resto fallisce, chiedersi "e se mi sbagliassi?"), la **fiducia**, l'**onestà**, la **privacy** (le persone dicono la verità più facilmente a quattr'occhi), l'**unità** (il motto di Intel *disagree and commit*: si può non essere d'accordo sulla strada migliore, ma ci si accorda su che cosa fare adesso) e infine l'**amore** del team per il gioco. È la stessa catena del docente: oggettività e neutralità rendono possibile un feedback che nessuno vive come un giudizio.

10.9 Il task: siete il lead di un gioco di zombie

10.9.1 La consegna

Il task per il lunedì successivo è **dare feedback**, ciascuno i propri. L'assunto: **si è un lead**, e si sta creando "questo incredibile nuovo, mai visto prima, zombie game" chiamato ***Zoe*** ("potete dargli il nome che volete: è un gioco che non esiste, ve lo assicuro"). I punti della visione, cioè del *mandate*, sono riassunti nella tabella 10.3.

Table 10.3: Il mandate del gioco ipotetico *Zoe*, su cui basare i feedback del task.

Voce	Contenuto
Genere e setting	gioco di zombie in un mondo **post-apocalittico**
Budget	produzione di media scala[9]
Camera	**sulla spalla**, alla *Alan Wake 2* o alla *God of War*
Tema e mood	**realismo**; mood **dark e realistico**
Nemici	violenti, grondanti di sangue, "budella fuori"[10]
Gameplay	combattimento **reattivo**, basato sui riflessi, non strategico ("non è Pokémon, è Dark Souls")
Focus	combattimento **melee**, ravvicinato, corpo a corpo
Target audience	**uomo americano, 18–25 anni**, che cerca *excitement*, *challenge*, *violence* e l'essere parte di qualcosa
Prime emotions	**fiero**, **submission**, **community**

I due tipi di gameplay. Il gameplay **reattivo** è quello in cui il combattimento si basa sui **riflessi**; l'opposto è quello **strategico**, a turni o ragionato. "Non è Pokémon, è Dark Souls": sono i due tipi principali di gameplay.

Submission e community. Delle tre prime emotions, alcune gli studenti le hanno già viste: la **submission** è il *grinding*, ripetere la stessa azione, ottimizzare; la **community** "potete immaginarla". Il **fiero**, invece, non era ancora stato trattato.

10.9.2 Il fiero

Il fiero non è l'emozione principale, ma **uno dei picchi emotivi**. Ci sono diverse definizioni e diversi modi di vederlo, ma spesso viene messo **in una categoria a sé**: non sullo stesso livello della *challenge* o dell'*excitement*, ma "un gradino più elevato", e soprattutto non necessariamente legato a uno specifico target audience. Va "sempre trattato con i guanti", ma è **estremamente potente come gancio del gameplay**.

[9] Nella trascrizione "con budget doppia"; si tratta quasi certamente di una produzione *double-A* (AA), cioè di medio budget.
[10] Nella trascrizione "nemici di Sherad", ripetuto due volte; il termine non è ricostruibile con certezza. Il senso, dalla descrizione, è quello di nemici molto violenti e splatter.

Come nasce. Il fiero nasce dalla **challenge**, e richiede due condizioni:

- una **sfida estremamente elevata**, al di sopra delle skill attuali del giocatore in quel momento;
- che il giocatore abbia **speso tempo ed energia**: fa la sfida, fallisce, impara, ritenta; fallisce, impara, ritenta; finché non la supera, e **si sente estremamente vivo**.

Questo picco emotivo, derivato dall'aver superato una sfida estremamente elevata, è il *fiero*: un termine inglese "rubato dall'italiano". Alcuni lo considerano il culmine delle emozioni.[11]

Esempi. È una delle emozioni alla base di *Dark Souls*, ed è il motivo per cui funziona. Si basano sul fiero anche molti livelli finali, estremi e complessissimi, di *Mario*, e i livelli estremi di *Mario Maker*. Il docente chiede alla classe altri esempi, e accetta:

- *Hollow Knight*, "ottimo esempio", con il *Pantheon* (i Pantheon di Hallownest, le sequenze di boss in fila);[12]
- i roguelike, con distinzioni: alcuni sono più spinti in questo senso di altri, anche a seconda dell'obiettivo della *run*. In *The Binding of Isaac* si può voler fare un finale piuttosto che un altro, per esempio battere **Mega Satan**.

Non su tutto il gioco. Il fiero non deve necessariamente riguardare tutto il gioco: generalmente è legato a una **boss fight** o a una **sfida specifica**. Non tutti i boss di *Dark Souls* sono ugualmente difficili, e alcuni sono chiaramente "la sfida definitiva" di quel gioco.[13]

Approfondimento: il fiero secondo Lazzaro, Bateman e Boon. In *21st Century Game Design*, Bateman e Boon riprendono da Nicole Lazzaro (che a sua volta cita Paul Ekman) il *fiero* come "*personal triumph over adversity*": l'euforia che si prova quando si è superata una sfida difficile, "la sensazione che ti fa alzare le braccia al cielo e scuoterle in trionfo", come un tifoso quando la sua squadra segna in rimonta.[14] Due precisazioni del libro si sposano con la lezione. La prima: per questo stile di gioco "più grande è la sfida, più grande è la ricompensa", e il giocatore non deve necessariamente passare per la rabbia prima del fiero, ma **deve essersi impegnato**. La seconda: non tutti i giocatori *hardcore* cercano il fiero, e alcuni desiderano attivamente esperienze più facili. È il motivo per cui il docente dice che va trattato con i guanti: è un gancio potentissimo per chi lo cerca, e un muro per chi non lo cerca. Il fiero era già comparso tra le quattro chiavi del divertimento di Lazzaro (sezione 5.9 e seguenti, capitolo 5).

10.9.3 Che cosa consegnare

Sulla base di queste premesse, si danno **feedback a una serie di concept art** mostrate a lezione (una delle quali dall'ultimo *Resident Evil*); la lista la si ritrova nelle slide. Si crea un **documento**, in cui valgono:

- tutte le **regole della documentazione**, che valgono sempre;
- tutto ciò che si è visto oggi: essere **chiari** (far emergere esattamente il problema), **specifici nella soluzione**, dare **reference**.

"Ragionateci bene": gli studenti hanno molto più tempo di quanto ne abbia normalmente una persona per dare un feedback, quindi conviene sfruttare la settimana.

[11] Nella trascrizione "alcuni dicono che sia l'alte metri di emozioni"; il senso sembra essere quello di un apice, o di un'emozione "al di sopra" delle altre.

[12] Nella trascrizione "Night Shark" e "Pantheon Hedgehog"; il riferimento a *Hollow Knight* e ai suoi Pantheon è ricostruibile con buona sicurezza.

[13] Nella trascrizione "Calamit è chiaramente la sfida definitiva". Potrebbe trattarsi di Kalameet, il drago nero opzionale del DLC di *Dark Souls*, noto per la sua difficoltà; non è certo.

[14] Chris Bateman, Richard Boon, *21st Century Game Design*, 2005, capitolo 3 (*The DGD1 Demographic Model*), paragrafo sullo stile di gioco Type 1 Conqueror.

Le slide mancanti. Il docente si accorge di non aver caricato le slide della lezione precedente, quella su compendium e vision: se capita e non le si trovano, bisogna dirglielo.

La pausa. "Abbiamo visto come darli, i feedback; andiamo a vedere come riceverli. Prima, però, prendiamoci una pausa. Direi che oggi abbiamo grande margine di manovra."

10.10 Raccogliere i feedback

Al rientro, un *deep dive* su come **collezionare** i feedback. Anche questo si fonda su altra documentazione che si vedrà in futuro: i punti vanno presi come **mantra**, buone pratiche, linee guida su cui costruire, come per gran parte delle altre questioni affrontate. Sono cinque: **testare**, **chiedere**, **collezionare**, **sintetizzare** (i survey) e il **linguaggio**.

10.10.1 Testare: prima possibile

Come si diceva nelle prime lezioni sulla documentazione (sezione 3.6), bisogna **far testare**: il gioco, il prototipo, qualsiasi cosa. Anche i **documenti** possono essere testati, facendoli leggere a qualcuno che non li ha mai visti.

La ruota dell'iterazione. È la necessità dell'**iterazione veloce**: prima si fa vedere una cosa, prima si ricevono feedback, prima li si può affrontare, capire i problemi, migliorare, e far girare la ruota (figura 10.2). Si sfrutta documentazione specifica: il **documento di playtest**, "che per ora non è pervenuto" (arriverà nella stessa lezione, sezione 10.17).

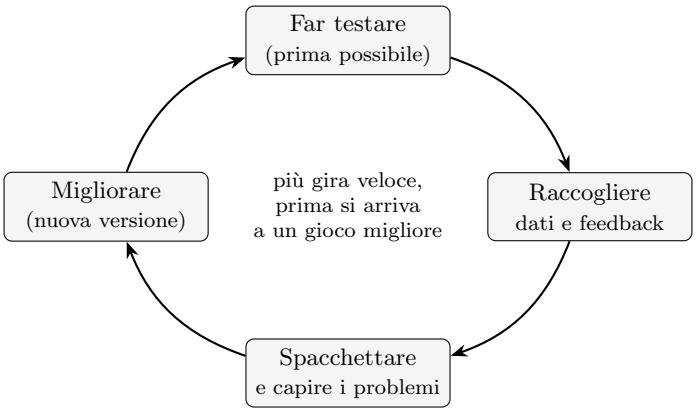


Figure 10.2: La ruota dell'iterazione a cui il docente fa riferimento più volte nella lezione (ricostruzione schematica).

Test diversi per fasi diverse. Ci sono test diversi a seconda della fase del progetto.

- **Test di validazione**, all'inizio del progetto.
- Durante le diverse *deadline* (**alpha**, **beta**, **gold**), e verso la fine del progetto, si possono fare test di **economia**, con il **QA** (*quality assurance*: il docente per un attimo non ricorda il termine e lo ricostruisce con la classe) e i QA tester, "che tra l'altro sono un'altra disciplina e un'altra specializzazione". Servono a vedere se il gioco è **bilanciato macroscopicamente**.

Vanno fatti, perché sono importanti; prima li si fa, meglio è. "Non fatela troppo tardi, ragazzi, perché questo poi vi taglia le gambe nel progetto."

Il caso Ubisoft. Un esempio, "nel bene o nel male", di test di validazione: in Ubisoft qualsiasi nuova IP o nuovo progetto che deve entrare in produzione passa dalle mani di **Yves Guillemot**.[15] Se l'esperienza non lo soddisfa, non se ne fa niente, "anche se il progetto è più incredibile dell'universo". Viceversa, ci sono stati salti di produzione nati dal fatto che Guillemot, passando in uno studio (il docente cita Ubisoft Lyon), prendeva in mano un prototipo e diceva: "Sai che questo è interessante? Vai, mettiamoci cento persone a lavorarci". È un caso limite, ma mostra quanto pesi la validazione iniziale.

10.10.2 Chiedere: chi sono i vostri tester?

I test hanno sempre, dall'altra parte, un **target audience**. Quindi si **chiede** alle persone: si identificano, si domanda che tipo di giocatori sono, quali emozioni ricercano, quali sono i loro giochi preferiti. Si trova cioè la **demografica** vista nella lezione di ricerca (sezione 5.7), per **dare il giusto peso** ai feedback. Persone diverse possono dire "questo boss mi è piaciuto molto" e "questo boss mi ha fatto schifo": bisogna capire quale dei due feedback è più importante, e poi come affrontarlo.

Ricerche e poll. Si può identificare il proprio target audience ("il target audience del vostro target audience") tramite ricerche e *poll*, ma molte di queste cose **costano**, e il docente non si aspetta che gli studenti le facciano. Di solito sono responsabilità di altri dipartimenti, come il **marketing**.

10.10.3 Collezionare: più dati possibili

"Non abbiate il braccino corto": si prende **tutto ciò che è possibile**. Alla peggio non lo si usa, o non si ha il tempo di setacciarlo e separarlo; ma lo si ha. Non lo si lascia nell'angolo: se ne possono usare dei pezzi, e farvi riferimento quando serve. L'obiettivo è avere dei **benchmark**, riferimenti a cui tornare.

Registrare. A maggior ragione se non si ha una buona memoria, si **registra chi gioca**, si registrano le **espressioni**, si **fanno domande**: è così che si ottengono più dati possibili.[16]

10.11 I survey

10.11.1 Gli strumenti

Per collezionare feedback, chiedere all'utenza e categorizzarla ci sono delle *best practice*, già accennate parlando di sondaggi nella lezione sulla ricerca (sezione 5.7). Il docente mostra di nuovo il survey che aveva fatto lui su *Valorant* (sezione 9.7). Gli strumenti possibili sono:

- **SurveyMonkey**, che richiede di pagare e permette di dare un certo tipo di *award* ai partecipanti;[17]
- i **Google Forms**, molto meno costosi: li si crea, si ha margine per renderli minimamente appetibili, e li si "spara in giro". È quello che fa anche l'accademia con i questionari di valutazione.

Parentesi: i questionari sul corso. Arriveranno altri questionari dell'accademia, generalmente a metà e alla fine dell'anno, su tutto il percorso. Il docente invita a pensarci **già adesso**: se si hanno feedback sul corso o sui docenti ("che sono sicuro avete"), o questioni che rischiano di essere dimenticate o di perdere rilevanza nel giro di qualche mese, conviene annotarle: **che cosa va bene, che cosa va male e come si può migliorare**, ora che si è vista la lezione sui feedback. E una "parentesi nella parentesi": siccome bisognerà valutare anche lui, "per favore, io vi chiedo di essere severi, **più severi** di quanto io non sia nei vostri confronti. Mi serve: se no non capisco cosa migliorare."

[15] Nella trascrizione "Yves Guémont" e "X Guemò"; il docente stesso si scusa del francese. Si tratta di Yves Guillemot, cofondatore e amministratore delegato di Ubisoft.

[16] La slide è accompagnata da un'immagine di un bambino al computer, che il docente commenta con una battuta; omessa.

[17] Il docente ripete, come nel capitolo 5, che SurveyMonkey sarebbe "di Amazon". In realtà è una società indipendente; probabilmente la confusione nasce con Amazon Mechanical Turk, la piattaforma di Amazon per reclutare partecipanti pagati.

10.11.2 Le regole del survey

Sintetici I survey devono essere **compatti**, a maggior ragione se non si offre alcuna ricompensa: la gente li apre, vede quanto sono lunghi e li chiude, "perché chi glielo fa fare? E li capisco". Come regola del pollice, **non più di cinque minuti**, e **lo si dichiara**, in positivo: "ci vogliono tre minuti per completarlo"; "va be', accettabile, ci sto". Il limite si può alzare se il survey è a pagamento, ma entro certi limiti.

Poca agency "Dato che abbiamo ragionato su questo termine" (sezione 9.2): si dà **meno agency** al tester. Si strutturano in modo preciso i limiti entro cui può dare la valutazione: **meno risposte aperte** ci sono, nelle domande con un obiettivo specifico, meglio è. La risposta aperta ci sarà, a volte nemmeno obbligatoria: uno spazio in cui il tester può "sbizzarrirsi". Ma per il resto si è concisi e si raccolgono dati **quantitativi**, numerici, facili da mettere in grafico.

Quantitativi, poi qualitativi Prima i numeri, poi il perché (si veda sotto).

Linguaggio semplice Il quinto punto, che vale anche per le **regole** di gioco e lo si ripeterà: si assume che dall'altra parte ci sia una persona con la **competenza di base** della lingua di riferimento (italiano base, inglese base), "proprio all'estremo". Le domande devono essere chiare, con gli articoli al loro posto, il meno convolute possibile.

Se possibile, ricompensate Con gli incontri ravvicinati è più facile: uno sticker, "una robina", e la gente si infervora. Diventa più problematico se il survey lo si manda in giro, a meno di usare altri supporti.

10.11.3 Sia lodata la scala di Likert

"L'ideale, sempre: sia lodata la **scala di Likert**." Da "sono estremamente d'accordo" a "sono estremamente in disaccordo", con **cinque valori**. Oppure: "elenca le meccaniche secondo il tuo tasso di gradimento", con la lista delle meccaniche e un valore **da 1 a 5**. È estremamente utile perché i dati arrivano già pronti per un grafico, e si vedono immediatamente (figura 10.3).

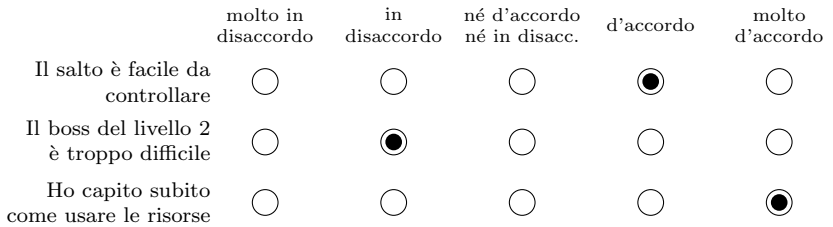


Figure 10.3: Un esempio di domande con scala di Likert a cinque valori (le affermazioni sono inventate a scopo illustrativo). Ogni risposta è un numero da 1 a 5: si aggrega e si mette in grafico immediatamente.

Il "ma" delle risposte chiuse. Ci sono dei "ma". Sul **genere**, per esempio, o comunque dove si vuole dare al tester ampio margine di espressione, il docente "per come è fatto" spingerebbe per la risposta libera. Si possono limitare le risposte, come si diceva l'altra volta ("maschio, femmina, altro, preferisco non rispondere") senza risposta libera: ma così **tutto lo spettro possibile viene mangiato via**. È un compromesso da fare consapevolmente.

La magnitudine, ma non il perché. I valori numerici servono a capire il **peso percepito** di una meccanica. Se c'è qualcosa di veramente problematico (i giocatori non hanno capito il level design, o la progressione è proprio rotta), lo si vede subito: si vede la **magnitudine**, se una cosa è piaciuta estremamente o è stata particolarmente non apprezzata. Ma **non si sa perché**, e non si sa se l'utenza ha suggerimenti.

Poi i dati qualitativi. Qui entrano in gioco i **dati qualitativi**: dopo i valori, si chiedono **maggiori informazioni**. Non tutti saranno disposti a darle, quindi la domanda può essere **facoltativa**; ma **va fatta**. È quello che il docente chiede anche agli studenti nei questionari: "a me non interessa che mi mettiate 1 o 5, mi interessa il perché. Se no non capisco."

10.11.4 Ricompense: meglio le gemme che gli euro

Il docente mostra esempi di aziende che fanno survey "estremamente bene", tra cui una in cui ha lavorato.[18] Alcune domande servono già a **mascherare una categorizzazione dell'utenza**: aiutano a capire non solo quale elemento è stato percepito come più importante, ma anche **che tipo di giocatore** si ha davanti. "Ma di questo ne parleremo in futuro, mi spiace."

La ricompensa in gioco. Il caso migliore di ricompensa, secondo il docente, è quella **inclusa nel gioco**: se ne possono generare N, pesa relativamente poco sulla progressione, ed è meglio dare dieci gemme, dieci *rainbow shards*, "dieci quello che è", piuttosto che pagare 100, 500 o 1000 euro in survey. Lo fa anche *Master Duel*, "visto che l'abbiamo visto recentemente".

Il rischio delle risposte a caso. C'è un rischio: soprattutto nei survey lunghi, se si dà una ricompensa il giocatore può **rispondere a caso** pur di ottenerla. Qui si può approfondire il discorso del **margine di errore**: se si vedono risposte particolarmente **incoerenti**, o non allineate con il tipo di giocatore che si ha in mente, si può prendere quella riga e **non considerarla**.

Approfondimento: survey e interviste secondo Schell. Nel capitolo sul playtesting, Schell dà una lista di consigli sui survey che coincide in buona parte con quella del docente, e la completa:[19] usare **immagini** quando si chiede di elementi o scene del gioco, perché il giocatore sappia di che cosa si parla; preferire agli online *SurveyMonkey* o *Google Forms*, gratuiti o economici; **non usare scale da 1 a 10**, ma scale a **cinque punti** con ogni punto etichettato (*terribile, abbastanza brutto, così così, buono, eccellente*), che danno risultati più coerenti; non mettere troppe domande, o la gente "si spegne" verso la fine; dare il survey **subito dopo** la sessione; avere qualcuno a disposizione per chiarimenti; annotare età e genere; e soprattutto **non prendere i dati come vangelo**, perché un survey raramente è davvero scientifico e i tester inventano quando non sono sicuri. Per le domande troppo complesse per un survey, Schell suggerisce l'**intervista**, con altri consigli che rafforzano la lezione: non aspettarsi che i tester siano game designer (non "il livello 3 sarebbe stato meglio più difficile?", a cui risponderanno di no perché i giocatori vogliono sempre il gioco più facile, ma "ci sono parti del livello 3 noiose?"); chiedere più di quanto serve ("quali sono le tre parti che ti sono piaciute meno?" dà più dati, già ordinati per priorità); e mettere da parte l'ego, senza difendere il gioco davanti al tester. Il set di domande che Schell usa nel suo studio, **FFWWDD**, è un buon modello per la parte qualitativa:

1. qual è stato il momento o l'aspetto più **frustrante**?
2. qual è stato il tuo momento o aspetto **preferito**?
3. c'era qualcosa che **volevi** fare e non hai potuto?
4. se avessi una **bacchetta magica** (*wand*), che cosa cambieresti, aggiungeresti o toglieresti?
5. che cosa stavi **facendo**, nell'esperienza (*doing*)?
6. come **descriveresti** il gioco ad amici e familiari?

L'ordine è voluto: prima si lascia sfogare il tester, poi si cerca il picco dell'esperienza; le domande 3 e 4 sembrano uguali ma non lo sono (la terza raccoglie piccoli desideri, la quarta idee fuori dagli schemi); la quinta rivela l'obiettivo che il giocatore si era dato, la sesta come ha percepito il senso complessivo dell'esperienza.

[18] Il nome dell'azienda nella trascrizione è illeggibile ("Olio Verso, il mio ex"); non lo si ricostruisce.

[19] Jesse Schell, *The Art of Game Design*, terza edizione, capitolo 28 (*Good Games Are Created Through Playtesting*), paragrafo *What Data Will I Collect after the Play Session?*.

10.12 Spacchettare i feedback: l'esempio di God of War

10.12.1 Due tester

"Ora facciamo un altro esempio, molto rapido e indolore, presuppongo." Si assume di raccogliere i feedback del **primo** *God of War*, quello con Kratos "incazzato nero", con le Lame del Caos. I tester sono due, "chiaramente con nomi fantastici": **Bella Isa**, conosciuta anche come Isabelle ("cuoricino"), e **Hugh Mikey Jacketman**. Ecco che cosa dicono (tabella 10.4).

Table 10.4: I feedback dei due tester sul primo *God of War*, come presentati a lezione (parafrasati dalla trascrizione).

Bella Isa	**Hugh Mikey Jacketman**
"Il gioco è estremamente violento: perché dovrei strappare la cassa toracica a questo tizio? Non basta ucciderlo?"	"Fratello, ho impalato Ares e stava agonizzando, piangeva come un bambino."
"Kratos è un protagonista monodimensionale, che non si evolve mai come personaggio."	"Kratos è una bestia, mi fa venire voglia di andare in palestra."
"È stato divertente battere quel tizio strano con la barba rossa: forse era Ade?"	"Quadrato, quadrato, triangolo: GG, easy mode."

Una volta ricevuti feedback del genere, bisogna farsi delle domande. Il docente le organizza in quattro passi.

10.12.2 Primo: identificare il target audience

La prima cosa, che la classe ha già inquadrato: **Isabelle non è il nostro target audience**, mentre Jacketman probabilmente lo è. Chi dice che *God of War* è troppo violento, che non vuole strappare la gabbia toracica a un nemico, va contro tutta la visione del gioco, contro la sua *action*: è condizione sufficiente per dire che non è il target.

Ma non si ignora. Questo **non** significa che abbia senso ignorare tutto ciò che dice: può aver sollevato punti interessanti. Quindi il primo punto è **identificare il target audience**, quanto più possibile.

10.12.3 Secondo: la rilevanza statistica

Il secondo punto è la **rilevanza statistica**: quanti tester dicono la stessa cosa?

- Se **sei Isabelle su otto** dicono che Kratos è monodimensionale, è **accettabilissimo**. Il docente non pensa ("qui qualcuno potrebbe smentirlo") che l'obiettivo narrativo del primo *God of War* fosse creare un personaggio con cui fare *bonding*, o che si evolvesse nel tempo. Lo sarebbe per il *God of War* del 2018,[20] e lì sarebbe un problema. Qui, il fatto che un pubblico non in target lo abbia notato è positivo: è **proprio l'obiettivo**.

- Se **due Jacketman** dicono "quadrato, quadrato, triangolo: GG easy mode", anche questo è esattamente ciò che si voleva. Quella era la **combo base** su cui è stato **bilanciato** il gioco, in modo che lo si potesse finire usando solo quella: la combo più semplice dell'universo rendeva il gioco completabile anche da persone con **bassa abilità reattiva**. Se lo dice un numero ridotto di persone, va bene: era pensata per essere così.

[20] Nella trascrizione "God of War 2019"; il capitolo con Atreus, di cui si sono visti i pilastri (sezione 9.10), è uscito nel 2018.

10.12.4 Terzo: tradurre il linguaggio

Bisogna anche **tradurre** ciò che dice l'utenza, perché si potrebbe non condividerne il linguaggio, il gergo, lo slang.

- "Il tizio strano con la barba rossa": chi era? Isabelle dice Ade, ma **non è Ade**: non c'è un Ade con la barba rossa nel primo *God of War*; è Ares. Bisogna capire che cosa intende. E una volta capito, allora sì che c'è un problema: **non ha compreso chi fosse** quel personaggio, né che rilevanza avesse nella storia.

- "Kratos è una bestia": è positivo? Il docente dice di sì; ma potrebbe intendere qualcos'altro. Lo stesso vale per "GG easy mode": non è scontato che un designer sappia che cosa significa *good game, easy*, né è fondamentale.

Chiedete. Più si va avanti con l'età, più probabilmente non si condividerà un certo slang con i giovani. Non serve aprire un dizionario, o andare per strada come nel meme del signore attempato che si veste da ragazzo e va in giro con lo skateboard.[21] Il tester ce l'avete davanti: **chiedeteglielo**.

10.12.5 Quarto: le risposte emotive

L'ultimo punto sono le **risposte emotive**, cioè *come* il tester dice le cose.

- Se Isabelle si lamenta della violenza **ed è triste**, va bene: non è il target. Ma se lo dice con un sorriso a trentadue denti, si potrebbe aver colto comunque il punto: potrebbe essere, a livello viscerale, proprio l'effetto che si cerca nel gameplay.

- "*It was fun*", come si sa (sezione 1.4.4), **non significa nulla**: bisogna capire quale emozione Isabelle associa a quel divertimento nello sconfiggere il "*red beard guy*". Potrebbe essere la **challenge**, o addirittura il **fiero** (era un ostacolo particolarmente difficile, e batterlo le ha dato piacere); potrebbe essere l'aver **masterato la combo** nel batterlo; o l'aver **punito il male**, più sul lato narrativo che meccanico. O si sa esattamente chi si ha davanti, e allora si sa da dove deriva il suo divertimento (anche qui con un margine di errore); oppure, più semplicemente, **glielo si chiede**.

- Jacketman che racconta in modo esagitato di aver impalato Ares e di aver goduto del suo pianto è **positivo**: con *God of War* il target vuole distruggere gli dèi, "strappargli le braccia, le palpebre e qualsiasi cosa sia legata al loro corpo". Non è solo ciò che si desidera, è la **reazione emotiva** che si cerca nell'utenza.

- E il fatto che, dopo aver giocato, corra in palestra a fare panca piana? "Probabilmente sì", è positivo: probabilmente è proprio ciò che si vorrebbe.

Un processo da cui non si sfugge. La figura 10.4 riassume il processo. "Questo è un processo che dovete fare: non si sfugge."

10.13 I feedback dei competitor

I feedback si possono prendere anche **sui competitor**, e non solo dai propri test: dai canali **Twitch**, **YouTube**, "quello che volete". Lì si vedono subito le cose particolarmente **memorabili**, o che l'utenza ricorda con **nostalgia** o piacere. Anche queste vanno spacchettate.

[21] Il docente lo chiama "il meme di Burns"; il meme descritto è quasi certamente quello di Steve Buscemi in *30 Rock* ("*How do you do, fellow kids?*").



Figure 10.4: I quattro passi per spacchettare un feedback dell'utenza, come emergono dall'esempio di *God of War* (ricostruzione schematica).

I commenti su YouTube. Il docente mostra alcuni commenti su *God of War*.

- Un utente scrive che l'**Idra** era il suo boss preferito, con cinque risposte e 97 like: magari è un'opinione condivisa. *God of War* va ridimensionato, "perché è enorme"; ma potrebbe valere la pena capire **perché** l'Idra era il boss preferito, ed eventualmente riprodurre quel sistema, o quella boss fight, in un'altra forma e in un'altra veste in un nuovo gioco. Una "mostra bestiale", e così via.
- Un altro scrive che un certo attacco "*will always be legendary*":[22] bisogna capire perché, e magari prevedere una meccanica o un sistema di combo che lo includa.
- Un terzo apprezza il Kratos nuovo, cresciuto, che ha imparato dai propri errori e sviluppa un senso di paternità verso il figlio. Funziona, e non solo perché è uno dei punti dominanti del gioco: anche perché **l'utenza è cresciuta**. Gli adolescenti che giocavano ai primi *God of War* ora sono adulti che giocano al nuovo, e si rivedono in quel Kratos con dei rimpianti: rileggono il personaggio anche attraverso la propria crescita.

Le recensioni su Steam. "Prendetelo ovunque", anche su **Steam** (si è visto anche *Steam 250*). Anche qui si spacchetta: quanti prodotti ha il recensore, che giocatore è, **quante ore** ha giocato, se ha avuto il gioco in **early access** o no, se l'ha ricevuto gratuitamente. I dati di Steam di solito mostrano queste cose. Il docente assume che si prendano feedback da giochi di cui non si ha il controllo, e quindi senza i profili che tracciano tutto ciò che l'utenza fa: si cerca di prendere ciò che dicono e di **estrapolarne le emozioni o le motivazioni principali**, anche con l'aiuto di Quantic Foundry (sezione 5.9).

Non sempre aiutano. Certe recensioni aiutano poco: "*Spooky scary game*" dice forse che il recensore cercava l'horror, qualche emozione trainante dell'horror, e che ama le battute. Ci sono recensori peculiari che recensiscono tutti i giochi nello stesso modo, "è l'internet". E un commento come "*next game: God of War 5*" fa presumere che il recensore voglia un quinto capitolo, ma non se ne ha la certezza.

10.14 La curva dell'utenza e i consigli finali

10.14.1 Evangelisti e odiatori

Come ultimi *tips and tricks*, si assume che tutta l'utenza (anzi, tutti i giocatori) si possa disporre su una **gaussiana**, una curva normale o "curva a campana". Come si diceva tempo addietro (sezione 5.7), ai due estremi ci sono le persone che amano alla follia il gioco, gli **evangelisti**, che lo consigliano agli altri, e i suoi profondi **odiatori**. Sono persone di cui **non è necessario modificare la traiettoria**.

[22] Nella trascrizione "The Light Attack" e "pseudo life attack"; l'attacco specifico citato nel commento non è ricostruibile con certezza.

- Gli **evangelisti** non sono del tutto ignorabili: a loro si possono chiedere feedback, si può chiedere se le cose piacciono, soprattutto molto prima che escano. Ma avranno la loro linea di pensiero e difficilmente la cambieranno: "dovete veramente distruggere l'anima del gioco per fargli cambiare idea".

- Gli **odiatori** difficilmente ameranno il gioco, perché **non sono il target**: sono molto lontani dalle motivazioni che il gioco può far scaturire.

Il formaggio. Al docente non piacciono la pizza con il formaggio sciolto né i latticini in generale: "questo non significa che Parmareggio debba preoccuparsi di cambiare il gusto per farmelo mangiare per forza". Non è parte di quel discorso. Allo stesso modo, un giocatore che non giocherà mai un PvP estremo, o un gioco di zombie, non va convertito.

Gli estremi sono i più vocali. Questi due estremi sono molto spesso **quelli che si fanno sentire di più**: c'è sempre chi si lamenta del bilanciamento, e di solito sono le persone che **giocano di più in assoluto**, e che hanno l'abitudine di mandare feedback e scrivere commenti. Quando si cercano feedback ci sono spesso questi *bias*. Anche se ci si riconosce in un giocatore che si lamenta, non è detto che sia da ascoltare: il target effettivo potrebbe non essere quello, "anche se vi piace tanto il gioco".

La rabbia attiva. C'è poi uno **sbilanciamento** tra i due estremi: generalmente si fanno sentire di più gli **odiatori**. Perché **la rabbia è un'emozione attivante**: porta le persone ad agire, a muoversi, a fare cose. La felicità, la pace dei sensi, portano invece a uno stato di calma: si vuole restare lì, "in ammollo", in quella beatitudine. Bisogna quindi badare bene a chi sta scrivendo: l'estremo negativo sarà sempre molto vocale. Se, fatti i conti, il feedback negativo è una percentuale minima del totale, la community è sana: "un 5% di community incazzata va bene", poi dipende dalla community.

Approfondimento: gli evangelisti secondo Bateman e Boon. Anche *21st Century Game Design* osserva che i giocatori più orientati alla sfida sono "l'*hardcore* vocale", che ama discutere di giochi online, e che molti giochi raggiungono il pubblico *casual* proprio passando per questi gruppi di **evangelisti**. Ma, aggiunge il libro, la facilità di gioco che il pubblico casual desidera è "anatema" per l'esperienza di sfida, dominata dal fiero, che gli evangelisti cercano.[23] È un esempio concreto del perché il feedback dell'estremo più vocale non coincide con quello del grosso dell'utenza.

10.14.2 Problemi, mai soluzioni

Secondo consiglio: **usate i feedback dei giocatori per identificare i problemi, mai le soluzioni**. Il giocatore può proporre una soluzione, magari anche attuabile, ma deve passare per una vostra "estrema valutazione cauta". Si prende il feedback per **mostrare i problemi**; non si sale mai a bordo delle soluzioni dell'utenza. Vale per i giocatori, e **al limite** anche per le persone esperte a cui si chiede un feedback.

10.14.3 Difendete la vostra visione

Terzo consiglio, che sembra andare contro alcune cose dette: **difendete la vostra visione**. Se si è fatto il design esattamente come si è detto, con la ricerca fatta "precisa, a modino", e si è sicuri delle decisioni di design e della loro implementazione, alla fine dei conti **i professionisti siete voi**. La scelta di modificare o meno, di togliere o meno qualcosa, passa per voi. Bisogna considerare il target audience e i suoi gusti; ma magari una cosa **la odia perché non l'ha compresa**, o perché c'è un altro problema, e non è detto che sia sbagliata di base, soprattutto se le meccaniche sono allineate con la visione. Assunto che si sia fatto tutto (ricerca dell'audience, lavoro per il suo bene), **l'ultima parola resta vostra**.

[23] Chris Bateman, Richard Boon, *21st Century Game Design*, 2005, capitolo 3, paragrafi sui cluster *Hardcore* e *Casual*.

Anche nel team. Vale forse meno con i giocatori e di più **all'interno del team**, se le responsabilità sono divise e le meccaniche sono date ai designer. Se gli artisti dicono "no, non ci piace", la cosa è relativa, con due premesse: che il gioco sia **design first**, e che ciò che si è identificato come meccanica di sistema sia da implementare. Gli artisti la **vestono**. Diverso è il caso in cui si è pensato a un livello che cambia l'arte ogni due secondi, e l'artista chiede "ce la facciamo?": quello è un altro discorso, di fattibilità.

10.14.4 Data informed, non data driven

Quarto consiglio, conseguenza del secondo: ***data informed design*, not *data driven***. Tutti i dati che si raccolgono, anche dai competitor, devono **aiutare a identificare un problema**, non dare già la soluzione. La slide dice "*unless specified*": alcuni giochi, generalmente i **game as a service**, i *live game* come *Candy Crush*, o i giochi di Supercell,[24] sono in gran parte *data driven*, per massimizzare l'utenza e per facilità. Ma è un caso legato ai game as a service, non ai molti casi di cui si parla nelle lezioni. "Ricordate sempre che il design è **50% scienza e 50% arte**."

10.14.5 Non eliminate subito ciò che l'utenza odia

Ultimo consiglio: se l'utenza odia qualcosa (una meccanica, un personaggio), **non eliminatelo subito**. Non si salta sulla soluzione appena arriva un feedback, non si corre immediatamente ai ripari: ci si ferma, si spacchetta il feedback, si fa un passo indietro.

La Control Room. È un problema di molte persone, e il docente lo lamentava anche con Alessandro. Il caso tipico, nei progetti annuali dell'accademia: arriva la **Control Room**, "la grande tavola rotonda delle persone che dovete temere", e dice "questa meccanica non funziona, me la cambiate?". Non si prende e si corre a sistemare tutto perché il capo, o chi sta sopra di lui, ha detto di farlo. Ci si ferma e si cerca di capire che cosa è stato detto: magari il feedback è legato a qualcosa che non avevano, o la meccanica non è stata capita, o il problema deriva da un'altra parte. Se poi, dopo averci riflettuto e averne parlato con lo **stakeholder** (sezione 4.11), il modus operandi resta quello indicato, va bene, è accettabile. Ma non si crea il panico: spesso queste reazioni portano a **problemi a cascata**. E magari ciò che la playerbase odia era già stato previsto: un altro designer ci aveva già pensato e lo aveva documentato. Ci si prende il tempo per **affrontare il feedback, tradurlo, e poi risolverlo**.

10.14.6 Prima e dopo il lancio

L'ultima slide è un'approssimazione di un caso ideale: il design perfetto, prima del lancio, con l'utenza disposta su una curva normale (figura 10.5).

- Agli estremi, circa il **3%** per parte: quelli che amano il gioco alla follia e quelli che lo odiano "alla morte", "quelli che vi mandano le minacce" e sono "il core della vostra tossicità". Non interessano.
- Poi circa il **13%** a cui il gioco piace molto, e il **13%** a cui non piace: non verranno sotto casa con i forconi di notte, ma il gioco non fa per loro.
- Al centro le due fasce che **interessano di più**: la fascia **neutra positiva** e quella **neutra negativa**. Sono le persone che **oscillano** tra il voler e il non voler giocare il gioco, tra il comprarlo e il non comprarlo.

Dopo il lancio: la conversione. Dopo il lancio, idealmente ("non è che funziona sempre così"), si deve aver **spostato la curva**: gli odiatori restano odiatori, ma una parte di chi era negativo diventa neutro, e la fascia neutra positiva diventa quella che **ha comprato il gioco e lo gioca**.[25] Cercare di attuare questa **conversione** è spesso il motivo per cui si fanno i **DLC**, il contenuto post-lancio, le *patch*: al di là del bug

[24] Nella trascrizione "i giochi super sell"; quasi certamente Supercell, lo studio di *Clash of Clans* e *Clash Royale*.

[25] La descrizione della slide post-lancio, nella trascrizione, è frammentaria ("questa faccia rimane rosso; questa scala diventa neutrale; e questa parte qui è ok, funziona"): il senso generale, uno spostamento delle fasce centrali verso il positivo, è chiaro; il dettaglio di ciascuna fascia no.

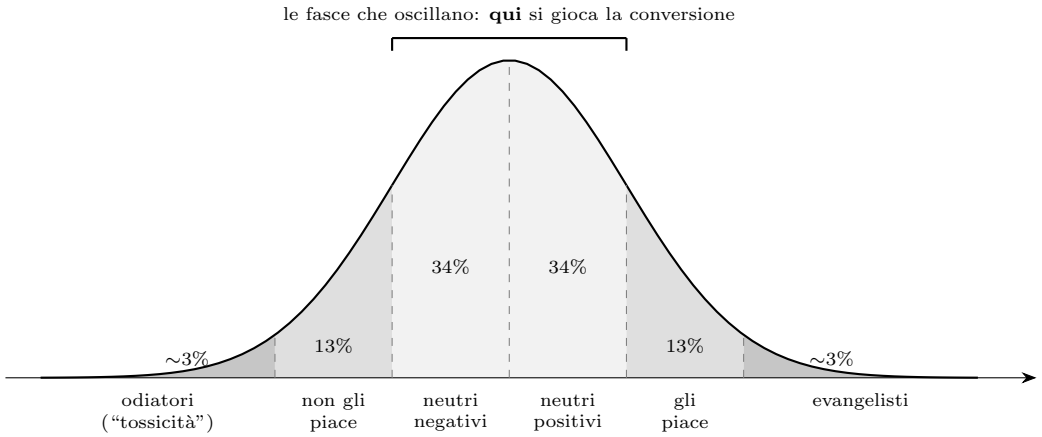


Figure 10.5: L'utenza disposta su una curva normale, prima del lancio, come approssimata a lezione. Le percentuali corrispondono a quelle di una distribuzione normale (circa 34% entro una deviazione standard dalla media per parte, 13,6% tra una e due, 2,1% oltre le due, arrotondato a lezione al 3%).

fixing, il contenuto post-lancio serve esattamente a convertire questa parte dell'utenza. È il caso migliore possibile, ma è un'approssimazione da tenere a mente quando si rilascia un gioco e quando si legge l'utenza.

10.15 Una lezione jolly

"Domande? Ok." La lezione sui feedback è finita "con ampio margine di anticipo". Il docente carica le slide ("non l'ho fatto prima, ahimè"; ma gli studenti hanno comunque la registrazione, e quando la rivedranno "sarete troppo *deep*" perché importi) e decide di usare il tempo rimasto. Invece di andare avanti con il programma, si fa un altro *deep dive* su documentazione e *pitching*: "lezione extra, chiamala così, una jolly: ce la giochiamo adesso".

Le due lezioni più importanti dell'anno. Prima, un avviso: nel mese ci sono ancora due lezioni, e gli studenti devono cercare di essere presenti a entrambe, perché tratteranno **gli argomenti più importanti dell'anno**. Sono due:

- la **profilazione dell'utenza** e gli **archetipi di giocatori**;
- il **"framework principe"**: uno dei più noti, "non necessariamente il più funzionale", ma estremamente **malleabile**, da cui ne sono nati molti altri; un **benchmark** per l'analisi e il design strutturato dei giochi.[26]

Due documenti senza forma netta. Visto che si è parlato di playtest, si vedono due documenti specifici del mondo del design: il **versioning tracker** (o *versioning document*, "come vogliate chiamarlo") e il **documento di playtest**, che accoglie i dati dei playtest. Il docente avverte subito che **nessuno dei due ha una forma netta**: dirà che esistono, a che cosa servono e come si fanno indicativamente, ma la forma "sta un po' a voi, e anche a me".

- Il documento di playtest dipende molto non solo dal progetto, ma da **che cosa si vuole far testare**: qual è l'obiettivo del playtest, che cosa si va a cercare.
- Il versioning è "un concetto un po' spinoso": serve non solo ai designer ma soprattutto ai **producer**, e può espandersi o contrarsi a seconda che riguardi l'intero progetto, un grosso sistema o una singola meccanica. Per questo spesso non è un documento a sé, ma **una parte del documento di design**, del

[26] Il docente non lo nomina. La descrizione (il più noto, malleabile, capostipite di molti altri framework di analisi) fa pensare all'MDA (*Mechanics, Dynamics, Aesthetics*) di Hunicke, LeBlanc e Zubek; ma è un'ipotesi.

processo di design che gli studenti hanno iniziato con l'ultimo task: il versioning dovrebbe **chiudere** quel documento.

10.16 Il versioning tracker

10.16.1 Il libretto di istruzioni dell'IKEA

La sezione si apre con un meme di *The Office* ("che io aborro profondamente, però sappiatelo"). L'obiettivo di un documento di tracciamento è **tracciare la produzione**: avere delle **linee guida**. "È come il libretto di istruzioni dell'IKEA": il designer crea una linea guida proiettata nel futuro che, se seguita, porta alla **versione corretta** della feature, del progetto, di qualsiasi cosa.

Spesso assorbito in altri documenti. Questa linea guida dipende così tanto dalla feature e dal progetto che, nella stragrande maggioranza dei casi, **viene assorbita** e diventa una parte di altri documenti: di design, di meccaniche, di sistemi, ma anche di livelli. È un tracker, "un *doc*, come volete chiamarlo", che generalmente è un **paragrafo specifico del GDD o dell'LDD**. Vale per molti documenti: più si andrà avanti, più si avranno conoscenza, esperienza e libertà nel combinare elementi, separarli, prendere un pezzo di un framework e unirlo a un altro. "Come sempre, valutate la migliore situazione possibile da momento a momento. Questo è importantissimo."

10.16.2 Le regole d'oro

Notificate il producer Se esiste un documento di tracking, lo si dice **sempre** al producer: dove sta e a che cosa serve.

Aggiornatelo Soprattutto, **se lo si inserisce, va aggiornato**, "e questo ce lo dimentichiamo sempre": a che punto della versione siamo, che cosa è stato fatto, che cosa no, che cosa manca.

Chiamate in causa gli altri dipartimenti Una feature include il lavoro di molti dipartimenti: il **sound** (il sound designer), i **VFX** (il VFX artist), le **animazioni** e il **rigging**, la **3D art** per un modello, e così via. Un tracker fatto bene chiama in causa i task di tutti. Se non si sa qualcosa, per esempio quanto tempo serve per un modello o un rig, e anche solo "vi solletica il cervello un leggero dubbio", **lo si chiede sempre**: "o mai lo saprete, ragazzi".

10.16.3 Che cosa contiene

Il docente mostra un esempio creato *ad hoc*, perché tutti quelli che ha sono sotto NDA; e, "che culo", è proprio un paragrafo specifico, fatto su Miro. L'**obiettivo** è il **controllo**: tenere traccia di tutte le versioni necessarie di una feature, dal punto in cui c'è un **prototipo giocabile e testabile** fino alla **versione completa nel core**, di solito la **1.0**.

La 1.0 è l'MVP. La 1.0 è il cosiddetto **MVP**, *Minimum Viable Product* (o *Project*): la versione minima, "vestita" con i feedback necessari per essere **implementabile** nel gioco, nella sua versione 1. Si può andare oltre (un bilanciamento di fino, feedback addizionali), ma questa timeline proiettata nel futuro va scritta, e serve a capire **dove si è** lungo di essa.

Gli elementi. Servono:

1. i diversi **stadi** (*stage*) della feature;
2. i **sottotask** inclusi in ciascuno stadio;
3. il **tempo** e le **risorse** allocati per ciascuno stadio (responsabilità soprattutto del producer);
4. i **check** che ciascuno stadio deve passare per saltare al successivo;

5. le **dipendenze** (*dependencies*): la relazione tra task e sottotask. Al documento di design o di analisi del designer può corrispondere, per l'artista, il *blockout* di una mesh o di un'animazione; va spiegato **quale dipartimento** lo fa e **in quale stadio** è necessario farlo, considerando il tempo a disposizione.

Funzione maggiore di forma. Deve essere **essenziale** da leggere: tutti lo leggono, quindi nessuno ha voglia di leggerlo, ed è spesso una lista immensa. Per ogni sottotask basta il **dipartimento** ("magari con un colorino associato, sarebbe carino"), lo **stadio** o lo **status**, e la **descrizione** del task. "Molto di più non serve."

Accessibile. Tutti devono potervi accedere. L'utilità di un documento di versioning a sé è proprio che i non designer (artisti, programmatori), interessati solo a sapere quali task devono fare, vanno lì senza doversi leggere il documento di design. D'altra parte, avere una serie di version document sparsi può essere **dispersivo**: ci si assicura che tutti, e soprattutto i producer, possano accedervi; e come indirizzare le persone nel posto giusto dipende dal **compendium** (sezione 9.9) o dal producer stesso.

Da qui a Jira. Un'altra cosa che si fa spesso è prendere le descrizioni dal tracker e riportarle su un **sistema di tracciamento** come **Jira** o **Asana**, quello che si usa, che il producer adopera per tracciare il progresso di tutto il gioco e assegnare i task di settimana in settimana. Ciò che è nel tracker è **uno a uno** con ciò che il producer comunica al team: è come un calendario delle cose da fare, per chi non è avvezzo a girare tra i documenti. "E ci sta, assolutamente."

Mantenimento. Come detto, è estremamente importante: i designer, o in generale gli **owner** della feature, lo modificano e lo tengono **aggiornato**; a ogni aggiornamento, i membri del team rilevanti per quella feature, sistema o progetto vanno **notificati**.

10.16.4 L'esempio: la camminata

L'esempio mostrato riguarda il **sistema di camminata e di locomozione base**, con un *disclaimer*: non è omnicomprensivo, ci sarebbero molte altre voci da includere, e andrebbe spiegata la legenda. La tabella 10.5 lo ricostruisce.

Table 10.5: Ricostruzione dell'esempio di versioning della camminata mostrato a lezione. Le versioni intermedie, i tempi e alcune voci sono indicativi; nell'esempio originale alcune celle erano lasciate a "TBD" per semplicità, cosa che in un documento vero non si fa.

Ver.	Dipartimento	Task	Tempo	Check per avanzare
0.1	Level design	creare la *gym* di test	1 h	bug check; test a livello di team; approvazione del lead di ogni dipartimento coinvolto
	Programmazione	premendo D il personaggio cammina verso destra a velocità 10		
	Game design	testare l'azione di camminata nella gym		
0.5	Programmazione Game design	sistema di accelerazione che modifica la velocità *tuning* della velocità	...	come sopra
1.0	Animazione, VFX, SFX	animazioni di camminata, effetti visivi e sonori; eventuali restrizioni artistiche	...	come sopra: la feature è pronta per essere inserita e *shippata* (MVP)
>1.0	...	bilanciamento fine, feedback aggiuntivi (1.5, 2.0...)	...	...

La 1.0 è accettabile. Si assume che, arrivati alla 1.0, la feature sia buona da inserire nel gioco: la si può **accettare**. È sempre migliorabile, quindi ci potrebbero essere una 1.5, una 2 e così via, da spiegare. E non si lasciano celle **TBD** ("io l'ho messo per semplicità, ma non è che lo lasciate"): alla 1 la feature è generalmente accettabile per essere messa nel progetto e *shippata*.

Un riassunto del documento di design. Il tracker **riassume** ciò che il documento di design spiega nel dettaglio. Per esempio, molte delle descrizioni scritte dagli studenti nei task avrebbero potuto benissimo stare qui, asciutte; ma avrebbero avuto bisogno di molta più spiegazione e profondità altrove, cioè nel documento di design.

I check di produzione. Per ciascuno stadio: **bug check**, **testing a livello di team** e **approvazione dei lead** di ciascun dipartimento coinvolto. Il **lead di programmazione** dà un'occhiata al codice, si assicura che la **nomenclatura** sia usata correttamente e che non siano state fatte "spaghettate di codice" all'interno del progetto. Il **lead di design** si preoccupa che il **GDD** sia allineato, e che la visione del design sia esattamente quella allineata con l'idea.

Più avanti nelle versioni. Alla 0.5 si **tuna la velocità**: il programmatore deve creare un sistema di accelerazione che modifica la velocità, con i suoi check. Per la 1.0 potrebbero servire **restrizioni artistiche** che prima non erano necessarie; nell'esempio mancano le **animazioni**, che vanno considerate, e i **VFX** e gli **SFX**. "Ripeto, non è omnicomprensivo: guardatelo sempre in maniera molto più intima."

Approfondimento: dire che cosa si consegna, e quando. Il versioning è la versione documentale di una delle chiavi di comunicazione di Schell viste sopra (sezione 10.8): la **chiarezza** come impegno concreto e specifico. "Faccio il combattimento entro giovedì" lascia la porta spalancata ai malintesi; "tre o cinque pagine sull'interfaccia del combattimento a turni entro giovedì alle 17" no. Un tracker con stadi, sottotask, dipartimenti, tempi e check è esattamente una lista di impegni del secondo tipo.

10.17 Il documento di playtest

10.17.1 Obiettivo: ottenere dati

"Ok, passiamo al documento di playtest, anche perché questo è l'ultimo." L'esempio mostrato tratta alcuni punti molto utili. L'obiettivo di un documento di playtest è **ottenere dati dal playtest**: "non molto di più, non molto di meno".

Perché è difficile codificarlo. Dipende da **che cosa si va a testare**.

- Se si dà un **prototipo** in mano a un **esperto** (e non a un utente), all'esperto si fanno alcune domande, ma non c'è tutto il gioco: si prendono **dati specifici** e si fanno **domande specifiche**.
- Se si ha **l'intero gioco**, il survey sarà fatto in un altro modo.
- Se è un **gioco fisico**, il documento viene messo in mano a qualcuno che lo riempie a ogni sessione di playtest, annotando che cosa è cambiato, che cosa è stato diverso, **chi ha vinto**, **quanto tempo** è durata la partita, **quanti giocatori** erano al tavolo, e così via (lo si è visto nel corso di analog game design, capitolo 2.12).
- Per un progetto **digitale** la questione è diversa: lo spettro è estremamente ampio.

10.17.2 I punti fermi

Alcune cose, però, si possono dire con precisione (tabella 10.6).

Un bacino di dati Il documento serve a **ottenere e controllare** feedback e dati: è un bacino di dati specifici dell'utenza.

Collegato al GDD Può essere unito a un **GDD**: se si fa il playtest di una specifica feature, il documento di playtest può essere collegato al GDD di quella feature, per vedere, versione per versione, quali feedback si sono ottenuti, che cosa è andato bene e che cosa male nella **0.3**, che cosa è cambiato nella **0.5**, e così via. È il punto d'incontro con il versioning.

Centrato sul test È centrato su **che cosa deve essere testato** e su che cosa deve ricevere particolare attenzione da chi testa.

Che cosa contiene Che cosa è stato testato; **come** i tester sono stati testati o osservati, cioè quali erano i riferimenti principali: ci interessava che fossero **spaventati** (un horror)? Che fossero in **tensione**? Che fossero **soddisfatti del bilanciamento** generale (un *economy test*)? E di quali **tool** sono stati dotati i tester.

Forma Finché è circoscritto al team di design, per la documentazione interna: **funzione maggiore di forma**. Per qualsiasi cosa destinata a tester e giocatori esterni al team ("generica persona che passa per strada, le dai il PC in mano e ti gioca il gioco"): **funzione maggiore o uguale a forma**. In quel caso, se ha la forma di un survey, va reso minimamente **esteticamente appagante**.

Accesso Solo i team legati alla feature e alle sessioni di playtest devono potervi accedere. "Gli altri non ci interessano."

Responsabilità I **designer** lo modificano. A volte il **lead** dà linee guida; ma generalmente, se si testa una feature, è il designer che l'ha fatta a creare il documento di playtest.

I tool dei QA. Il punto sui tool merita un esempio. Internamente, se c'è un dipartimento di **QA**, i tester sono provvisti di sistemi di **debug**, magari di *cheat* di **invincibilità** perché devono saltare delle parti, magari dell'intero gioco a disposizione. In un **open world** in cui interessa testare il bioma Y e non il bioma X, e si parte da molto lontano, si può fornire un sistema di **teletrasporto** e di invincibilità: tutte cose centrate a migliorare l'esperienza su **ciò che deve essere testato**, non su tutto il resto. Ma queste cose vanno **documentate**, e soprattutto sono i tester a dirlo quando danno il feedback: "ho testato il gioco con questi settaggi, ho fatto queste cose, è successo questo, questo e questo".

Table 10.6: Uno schema possibile di documento di playtest, ricostruito dai punti visti a lezione (non è il template del docente).

Voce	Esempio
Feature e versione	camminata, v0.5 (link al GDD della feature)
Obiettivo del test	la velocità e l'accelerazione risultano controllabili?
Che cosa osservare	frustrazione nei salti di precisione; tempo per attraversare la gym
Tester	numero, tipo di giocatore, target sì/no
Setup e tool	build, settaggi, cheat (invincibilità, teletrasporto)
Dati quantitativi	tempi, morti, valori Likert del survey
Dati qualitativi	risposte aperte, note di osservazione, citazioni
Rispetto alla versione precedente	che cosa è cambiato, che cosa è migliorato o peggiorato
Problemi individuati	elenco, con priorità (problemi, non soluzioni)

10.17.3 Il documento non basta: bisogna testare

"Lo ripetiamo adesso": è importante che il gioco venga testato. Il documento di playtest serve come **cartina tornasole**, come metro di misura di ciò che succede; ma bisogna **farlo giocare**, altrimenti è inutile fare questo tipo di documenti. Prima il gioco viene testato, meglio è, perché prima si arriva a una versione migliore, sapendo che cosa non funziona. "Facciamola girare più velocemente possibile, questa ruota, e mettiamola in moto il prima possibile" (figura 10.2).

Nessun task. Su questi due documenti non c'è un task. Ma bisogna ricordarsi che **esistono**, e il discorso su quanto possano essere legati o slegati da altri documenti, **in entrambi i casi principalmente il GDD**. "Questa è la cosa più importante."

Approfondimento: la lente del playtesting. Schell distingue quattro tipi di test che spesso si confondono:[27] i **focus group** (interviste a potenziali giocatori su ciò che piace o no, utili soprattutto per decidere la priorità relativa di feature ben definite, ma con una cattiva fama perché spesso condotti male); il **QA testing** (che non ha nulla a che fare con quanto il gioco sia piacevole: cerca bug); lo **usability testing** (se interfaccia e sistemi sono intuitivi); e il **playtesting** vero e proprio, cioè vedere se il gioco produce l'esperienza per cui è stato progettato. La sua lente riassume ciò che un documento di playtest deve saper dire prima ancora del test: *perché* facciamo questo playtest? *Chi* deve esserci? *Quando* testiamo? *Dove*? *Che cosa* cerchiamo? *Come* otterremo le informazioni che ci servono? E, aggiunge Schell, la domanda "il mio gioco è divertente?" non basta: le domande devono essere il più specifiche possibile ("i giocatori capiscono come si gioca?", "il livello 3 è troppo lungo?", "ci sono strategie dominanti?"). Schell confessa anche di **odiare** il playtest, per paura che il gioco non piaccia; ed è proprio il motivo per cui bisogna farlo presto e spesso.

10.18 La chiusura: lavorare entro dei limiti

10.18.1 Che cosa è arrivato della documentazione pratica?

Avanza ancora un po' di tempo, e il docente chiede se ci sono questioni da aprire. L'unica cosa che ritiene utile vedere, nel poco tempo rimasto, è il riscontro sul discorso della **documentazione pratica** fatto con l'esercizio sui core elements (sezione 9.16): "quanto è stato recepito, solo voi me lo potete dire".

Le risposte della classe. Una studentessa risponde che le ha mostrato dei **trucchi** utilizzabili di base, e le ha fatto capire in che cosa deve migliorare (nella trascrizione, non del tutto chiara, renderlo "più credibile"). "Questa è metà della partita", commenta il docente, "perché poi c'è tutta la parte di creazione del documento." Altri citano il **titolo** trattato come titolo della scheda (perché effettivamente è più grande), il **modo di sfruttare gli spazi** ("una cosa che sto imparando piano piano"), la distinzione tra gli spazi e le **linee**. "Molto bene."

Un modo, non l'unico. Ciò che si è visto non è l'unico modo, è **un** modo, e con solo alcuni strumenti. Ma il punto è **sfruttare in modo creativo le poche cose a disposizione**: i documenti Google non offrono chissà quante possibilità, ed è anche utile **lavorare entro certi limiti**.

10.18.2 La soluzione sta nel problema

"Filosoficamente" è ciò che tocca fare nel trovare soluzioni: prendere qualcosa che si ha sotto mano e che si usa nel modo previsto (prevedibile), e provare a vederlo da un altro punto di vista; cercare soluzioni con il poco che si ha, ma usato in modo **creativo e innovativo**. Spesso la soluzione di un problema **sta già nel modo in cui il problema è formulato**. Lo si vedrà più avanti con altri tipi di documentazione: c'è proprio una filosofia di design legata a **quanto è grande un documento**, a quanto farlo diventare enorme o quanto stringerlo. Per quanto possa piacere o no la documentazione, ragionare entro questi limiti costringe a formulare il problema in modo diverso, e formulandolo in modo diverso si trova qualcosa a cui non si era pensato.

Con due lire. "Sapete quante volte mi scervellavo su una soluzione con sistemi difficili da implementare, tecnologie complesse o design particolarmente stravaganti, quando invece la soluzione si poteva fare con veramente due lire, due spicci?" Non va dimenticato.

10.18.3 I blocchetti di Lego

"Ragionate per immagini. Quando possibile, **mostrate**." E soprattutto iniziate a pensare a come gli elementi visti finora possono **comporsi l'uno con l'altro**. Il **flowchart** (sezione 6.9): dove potrebbe essere inserito, in quale condizione, se si sta facendo design o se si deve prevedere che cosa aspettarsi dall'utenza? La **Theory of Planned Behavior** vista l'ultima volta (sezione 9.5):[28] un domani potrebbe servire a documentare un

[27] Jesse Schell, *The Art of Game Design*, terza edizione, capitolo 28, e lente #103, *The Lens of Playtesting*.
[28] Nella trascrizione "questo qui aterior plan".

sistema, o ancora meglio la **strategia** che il giocatore adotta e **perché** la adotta. Sono cose estremamente importanti, che diventano necessarie in futuro. "Iniziate a farlo ora, perché poi ve lo imporrò di più": unire questi blocchi, "questi blocchettini di Lego che vi do".

10.18.4 I prossimi appuntamenti

Si chiude con leggero anticipo. La lezione successiva non è il lunedì seguente: ci si rivede il **20**, poi il **27** ("da calendario, che avete tutti"). A dicembre ci sono due lezioni, una con il docente e una con Diego. E per chi ha recuperato la registrazione fin qui: "di solito si dice tipo scrivete 'zucca', e so che siete arrivati fin qui; ma io non posso leggere i vostri commenti, quindi...".

Punti chiave del capitolo

- Nel task su *Metroid Dread* il compendium resta quasi vuoto, ma va diviso per dipartimenti principali, con status e legenda; la vision contiene pilastri, prime emotions e moods, must do, must not, nice to have e USP, **sempre tre**. I pilastri qui si estraggono *a posteriori* dal gameplay. Se il docente dice "poi vi do il task", ricordateglielo.
- Una **legenda della formattazione** serve se ci sono acronimi, status o molti strumenti; l'obiettivo è non renderla necessaria. È utile soprattutto nei documenti di bilanciamento.
- Fare design significa **trattare tutti i dettagli**: numeri indicativi, casi limite (che succede quando il battle pass finisce?), nuove UI e nuove parole chiave spiegate. Test: con questo documento potrei implementare il sistema? Se non lo scrivete, i programmatori fanno di testa loro, e la responsabilità è vostra.
- La severità dei feedback non è mai personale ed è in funzione della crescita; il mondo là fuori non sempre dà feedback costruttivi.
- Assunti di base: dall'altra parte c'è un **umano**; le **soft skill** sono la prima discriminante per essere assunti o licenziati; i feedback sono **costruttivi**. Feedback 101: precisi e rispettosi, empatici, e coltivare queste capacità.
- Un feedback è **costruttivo** (preciso, chiaro, analitico), **neutrale** (dice anche ciò che funziona, e perché, in uno stato emotivo neutro), **sincero** (tra adulti e professionisti), **risolutivo** (una soluzione suggerita, non imposta) e **senza giudizio**. La **tecnica del sandwich non funziona**: meglio una lista chiara.
- **Chiari**: riformulare finché entrambi sono soddisfatti. **Precisi**: reference esatte, guardarle insieme, dare tempo. **Analitici**: obiettivo e intenzione di ciò che si valuta, allineamento con la vision, dati, percezione del target.
- Non state zitti nei meeting (effetto valanga); un team, non una famiglia: fallimenti condivisi, successi festeggiati, meriti riconosciuti. Il **framing** è potentissimo (Starbucks, i nove dentisti su dieci). Linguaggio del corpo neutro, o si rimanda.
- Il **fiero** nasce da una sfida molto al di sopra delle skill attuali, superata dopo tempo, fallimenti e tentativi; è un picco, va trattato con i guanti, ed è un gancio potentissimo (*Dark Souls*, *Hollow Knight*).
- Raccogliere: **testare presto** (anche i documenti), test di validazione all'inizio ed economy test con il QA; **chiedere** chi sono i tester per pesarne i feedback; **collezionare** più dati possibili, registrando.
- Survey: **sotto i cinque minuti** (dichiarati), poca agency, **scala di Likert** per la magnitudine, poi una domanda qualitativa (anche facoltativa) per il perché; linguaggio semplice; ricompense in gioco meglio che in denaro, scartando le risposte incoerenti.
- Spacchettare un feedback: **target** sì o no (ma non si ignora), **rilevanza statistica**, **tradurre** il gergo, leggere la **risposta emotiva**; nel dubbio, chiedere.

- L'utenza sta su una gaussiana: **evangelisti e odiatori** sono i più vocali (la rabbia attiva), ma le fasce che contano sono quelle **neutre**, da convertire anche con DLC e contenuti post-lancio.
- Feedback per i **problemi, mai per le soluzioni**; **difendete la visione**; *data informed*, non *data driven* (salvo i game as a service); non eliminate subito ciò che l'utenza odia: fermatevi e spacchettate.
- Il **versioning tracker** è il libretto IKEA della feature, spesso un paragrafo del GDD: stadi, sottotask, tempo e risorse, check, dipendenze, fino alla 1.0 (**MVP**); essenziale, accessibile, aggiornato, notificato al producer.
- Il **documento di playtest** serve a ottenere dati: che cosa si testa, come si osserva, con quali tool; collegabile al GDD versione per versione; funzione $>$ forma internamente, $\geq$ verso l'esterno. Ma senza far giocare il gioco non serve a nulla.
- Lavorare entro dei limiti costringe a riformulare i problemi: la soluzione spesso sta nel problema, e si fa con due lire. Iniziate a comporre i blocchetti (flowchart, TPB, documenti).

Chapter 11

Il vocabolario del level designer: blockout, coordinate, pivot, mesh e luci

Il capitolo 8 ha aperto il corso di **Level Design** del primo anno: chi è il level designer, perché "level design" è un termine ombrello, e i primissimi passi nell'editor di Unreal Engine, con il task di ricostruire **la propria stanza** usando solo forme di base. Questa lezione, tenuta dallo stesso docente (Stefano Zocchi), ne è la diretta prosecuzione: è la "**seconda parte del ciclo di lezioni introduttive**". Le lezioni introduttive, spiega il docente, servono a vedere "un po' di concetti, un po' di altra roba di base", con momenti di spiegazione e momenti di pratica.

Il docente lo dice subito con franchezza: questa è "una delle lezioni più frontali" dell'anno, "una lunga lista di termini che è necessario che sappiate se vogliamo iniziare a parlare con cognizione di causa degli argomenti del nostro corso". Sono concetti legati all'uso di Unreal, ma anche **concetti di base del mondo dello sviluppo di videogiochi e del level design**: molti studenti, almeno in parte, li conoscono già, ma il docente sceglie di vederli "in una sezione formale all'interno del corso" per assicurarsi che tutti sappiano esattamente che cosa sono. "La lezione di oggi sarà un po' noiosa, o magari queste cose vi appassionano, non lo so."

La lezione è anche segnata da un imprevisto tecnico: il computer dell'aula è **nuovo**, il docente non aveva ancora usato **Unreal Engine 5.8** (per lavoro usa Unity o altre versioni di Unreal), e nella 5.8 Epic ha cambiato diverse cose, tra cui il fatto che lo **Starter Content** non è più incluso. Una parte della lezione è quindi spesa a risolvere il problema, e il capitolo riporta anche la procedura trovata, perché è quella che gli studenti devono seguire.

La struttura della lezione, e quindi del capitolo, è questa:

1. la **struttura delle lezioni** del primo anno e la **revisione dei blockout** della propria stanza, con le digressioni sugli screenshot, sul "pizzico di caos" e sul **verosimile** (sezioni 11.1 e 11.2);
2. l'introduzione alla lezione e lo Starter Content mancante (sezione 11.3);
3. la distinzione fondamentale tra **lavoro artistico** e **lavoro di gameplay** nel level design (sezione 11.4);
4. il **blockout**: che cos'è, perché si fa, gli esempi di *Call of Duty* e *Uncharted*, e il gameplay che "regna sovrano" (sezioni 11.5 e 11.6);
5. i **cinque strumenti** del blockout: primitivi, kit modulari, brush e modellazione, scultura del terreno, spline (sezioni 11.7–11.7.5), e che cosa si farà nel primo anno (sezione 11.8);
6. la terminologia dell'editor: **asset e actor**, **coordinate globali e locali**, **genitori e figli**, **gruppi**, Ctrl+Z, **snapping**, assi di rotazione, Content Browser, **pivot** (sezioni 11.9–11.16);
7. **mesh e collisioni** (sezione 11.17);

8. le **luci**: point light, spot light, rect light e directional light (sezione 11.18);
9. l'importazione manuale dello **Starter Content** (sezione 11.19);
10. il **livello costruito da zero**: piano, sole, luce ambientale, cielo, la cartella *Lighting* (sezione 11.20);
11. **materiali e texture**, le **modalità di visualizzazione** e il **post process volume** (sezioni 11.21–11.23);
12. il **task** per la lezione successiva, le domande e la chiusura (sezioni 11.24 e 11.25).

11.1 La struttura delle lezioni del primo anno

11.1.1 Cinque minuti per chi arriva

Come fa sempre, il docente lascia qualche minuto "per dare il tempo a un pochino di gente di arrivare". La lezione è ibrida: la maggior parte della classe è in aula, mentre **Mattia** segue da casa (e combatterà a lungo con l'audio di Teams). Il docente sa che una persona gli ha scritto per dire che sarebbe stata assente; l'appello vero e proprio verrà fatto "da lezione", cioè alla fine, per essere sicuri che ci siano tutti quelli che ci devono essere (sezione 11.25).

11.1.2 Prima i task, tutti insieme

Il docente spiega come sono organizzate le sue lezioni, "soprattutto il primo anno": **la prima parte della lezione è dedicata a guardare in classe, tutti insieme, i task** che gli studenti hanno fatto.

- Questa abitudine **svanisce progressivamente** nel corso dell'anno, e dipende anche da **quante persone ci sono in classe**: con quattro o cinque persone c'è il tempo di vedere tutto insieme, con dieci o quindici molto meno.
- Il primo anno, però, il docente ci tiene a cominciare le lezioni **correggendo** o, come in questo caso, **dando un'occhiata**: le prime lezioni sono di pratica, e non è troppo rigido nel correggere.
- Lo scopo è doppio: tutti possono guardare i lavori degli altri e **rendersi conto del livello generale**; e, dato che ci saranno molti casi in cui tutti fanno **lo stesso errore**, il docente non deve ripeterlo più volte: "posso parlarne di uno e lo vedete tutti".

11.2 La revisione dei blockout della stanza

"Per oggi c'era un task. Il task era?" La classe risponde subito ("ottimo, dieci su dieci"): il **blockout della propria stanza**, il primo task di level design (sezione 8.19). Si procede **in ordine alfabetico**, proiettando gli screenshot caricati nella cartella dei task.

11.2.1 Francesca: cubi, piani e un CD sotto la sedia

La prima è Francesca (il docente si fa correggere la pronuncia del cognome). "Bella stanza." Ci sono delle **decorazioni**: la studentessa spiega che sono stilizzate, e il docente approva ("ci può stare"). Fa notare un elemento storto, ma il giudizio complessivo è "bel lavoro". Alla domanda se abbia usato solo cubi, Francesca risponde di aver usato **cubi e piani** (i piani per i poster), e c'è persino un dettaglio sotto la sedia ("dovrebbe essere un CD").

Lo scopo dell'esercizio. "Per questo task di solito non ho niente da dire": il docente lo guarda soprattutto per assicurarsi che tutti l'abbiano fatto. Il lavoro è "molto preciso", "decisamente molto dettagliato"; ma lo scopo dell'esercizio, ricorda, è **prendere la mano con gli strumenti**: passare dallo spostamento alla rotazione alla scala e così via, in modo che diventi **istintivo**, "non ci dovete più pensare una volta che l'avete imparato". Per questo va benissimo; e non si perde troppo tempo su ogni lavoro.

11.2.2 Antonino: una PlayStation 5, e la cartella condivisa

La stanza di Antonino è "molto più ristretta". Il docente si diverte a riconoscere gli oggetti: "fammi indovinare, questa è una PlayStation 5"; e poi il case del computer.

Un account condiviso. Nel frattempo, il docente spiega perché sullo schermo compare il nome di un altro docente (Mattia Podini): sta usando l'**account generale** che è stato lasciato sul computer dell'aula.

Non si cancella nella cartella dei task. Nella cartella compare anche una "bozza". Regola pratica: gli studenti **non possono cancellare** i file caricati nella cartella condivisa. Se serve eliminare qualcosa, bisogna **scrivere "da cancellare"** (nel nome del file): il docente lo cancellerà appena lo vede.

11.2.3 Federica: abbiamo appena traslocato

Federica mostra "solo la versione migliore", e ha già usato anche i **materiali**: "siamo già più avanti". La stanza è "un po' spoglia, forse"; ma c'è una ragione narrativa: "abbiamo appena traslocato, quindi la stanza è vuota". "Questo è un nuovo modo per imbrogliare un task", scherza il docente; ma il lavoro è buono, e soprattutto contiene un dettaglio che vale per tutti.

Un pizzico di caos. Le **scatole** del trasloco sono messe **sghembe**, di traverso, e questo crea una visuale più interessante. Stessa cosa per il **libro sul comodino**: "se l'avessi messo rigidamente regolare, sarebbe stato molto strano, tra virgolette, da vedere".

> Mettere un pizzico di caos in quello che facciamo rende gli ambienti più interessanti, rende gli ambienti più **vissuti**.

"Anche queste sono cose a cui arriveremo": è un primo assaggio del lavoro di *set dressing* (sezione 8.6).

11.2.4 Mattia: le foto allo schermo, e uno spadone

Mattia Maestri è il primo, nell'esperienza del docente, a consegnare **fotografie dello schermo** invece che screenshot. Segue da casa, ma all'inizio non lo si sente: il microfono è disattivato, poi l'audio della televisione dell'aula non funziona ("questo computer è nuovo, tra l'altro, e questo potrebbe crearci un paio di problemi dopo"), finché Mattia non si collega anche dal telefono, scoprendo nel frattempo di avere le cuffie rotte. La spiegazione delle foto: ha dovuto formattare da poco il PC fisso e non ha ancora reinstallato gli strumenti per fare e mandare gli screenshot.

Un trucco su Windows. Il docente ne approfitta per insegnare a tutti una cosa "abbastanza utile":

- **Win+Shift+S** è la scorciatoia di Windows per fare uno **screenshot**: si seleziona una parte dello schermo, e l'immagine viene salvata **negli appunti**; con **Ctrl+V** la si incolla dove serve. Funziona ovunque in Windows.
- Esiste anche una funzione **interna a Unreal** che permette di prendere uno screenshot direttamente dall'immagine renderizzata dall'engine: "una cosa alla volta, ci arriveremo anche a quello".[1]

Lo spadone. Il lavoro è "un po' meno dettagliato", ma Mattia ha avuto parecchi problemi con il computer. Nella stanza c'è uno spadone: "questa è la spada di Cloud?" No: è uno spadone che Mattia ha davvero in camera, uno spadone d'osso di *Monster Hunter*[2] ("ottimo, fantastico"; "ce ne sono tantissimi, sinceramente").

Daniele. Daniele oggi non c'è: è lui che aveva avvisato. Il docente dà comunque un'occhiata al suo lavoro: "ottimo lavoro".

[1] Il docente non la nomina; si tratta con ogni probabilità della funzione *High Resolution Screenshot* dell'editor, accessibile dal menu della viewport.

[2] Nella trascrizione "lo spadone d'osso dimostrante"; "dimostrante" è quasi certamente una storpiatura di "di *Monster Hunter*", serie di cui Mattia aveva parlato nel giro di presentazioni (sezione 8.3), e che ha effettivamente "tantissimi" spadoni.

11.2.5 Francesco: la stanza senza soffitto, e il verosimile

"Ci sono un sacco di Franceschi." La stanza di Francesco Muratore è "molto innovativa nell'avere una stanza senza soffitto", e "molto, molto orizzontale". Lo scopo dell'esercizio era fare pratica, quindi va bene; ma il docente dà un consiglio per il futuro: fare **ambienti un po' più credibili**. "Il tuo letto non è così stretto": così com'è, "sembra più una panchina che un letto". Per il resto ci sono le forme, le sedie, uno zaino.

Non realistico: verosimile. Serve "un po' più di attenzione per quanto riguarda gli spazi", e un approccio... "realistico non è il termine corretto". Il termine che il docente userà spesso durante l'anno è **verosimile**:

> I videogiochi puntano non al realismo, ma alla **verosimiglianza**.

Si torna così all'**organizzazione degli spazi**: un letto deve avere le proporzioni di un letto perché il giocatore lo legga come tale, anche se lo stile del gioco non è affatto realistico.

11.2.6 Alberto: lo showroom dell'IKEA

Anche Alberto ha lavorato con i **materiali**, per la maggior parte degli oggetti (il docente si scusa: "ci metterò un bel po' a ricordarmi i nomi di tutti"). C'è il condizionatore. Ma la stanza è "il vuoto totale": nessun poster, niente sugli scaffali vuoti. Va bene, ma la battuta è inevitabile: "Alberto vive in uno showroom dell'IKEA". È il rovescio del "pizzico di caos" di Federica: uno spazio senza segni di vita non sembra abitato.

11.2.7 Valerio: super preciso, e il problema della scala

"Valerio invece è andato super preciso." Valerio racconta di essersi accorto, provando a "entrare" nella stanza con il personaggio, che **era tutto enorme**: aveva capito che i valori erano in centimetri, ma inserendo, per esempio, 5 nel campo della scala, l'oggetto diventava di cinque metri. Ricordiamo infatti (sezione 8.16) che la *posizione* è in centimetri, ma la *scala* è un **moltiplicatore** della dimensione originale dell'oggetto: un cubo di base è largo un metro, e con scala 5 diventa largo cinque metri (si veda anche la sezione 11.14).

"Quello non è un problema": lo scopo era metterci mano, non preoccuparsi troppo delle dimensioni. Il lavoro è "un gran bel lavoro": super dettagliato, super preciso, con le sedie, il cestino, la *tower* del computer, e una lampada come "tocco di classe".[3]

Finché avete energia e tempo. "Ci sono state un paio di persone che hanno preso questo esercizio molto sul serio. Mi fa piacere." Il consiglio: finché si hanno l'energia e il tempo per farlo, **fatelo**. "Più andremo avanti nell'anno, meno tempo avrete. Ma è inevitabile." Il commento finale sulla revisione: "ottimo, ottimo, ottimo. Soddisfazione."

11.3 La lezione di oggi, e lo Starter Content che non c'è

Teoria formale, e un po' di pratica. Oggi si coprono concetti che sono legati all'uso di Unreal, ma che sono anche concetti di base dello sviluppo di videogiochi e del level design. Ci sarà anche un momento in cui si comincia a lavorare in Unreal con lo **Starter Content**.

Unreal 5.8 e lo Starter Content. Qui il docente "incrocia le dita": il computer dell'aula è appena stato cambiato, e lui non ha ancora usato la versione 5.8, perché i progetti a cui lavora usano Unity o versioni diverse di Unreal. E nella 5.8 sono cambiate un po' di cose, tra cui una che ha verificato proprio quella settimana: **lo Starter Content non è più incluso con Unreal**.[4] Con l'altro insegnante di level design è stato organizzato un **pacchetto da scaricare e importare**; la procedura si vedrà più avanti (sezione 11.19).

[3] Dopo il lavoro di Valerio la trascrizione riporta ancora un "congratulazioni, anche tu super dettagliata", senza nome: potrebbe riferirsi allo stesso lavoro o all'ultimo in ordine alfabetico. Le presenze a fine lezione confermano comunque che tutti i presenti avevano consegnato.

[4] Nel capitolo 8 (sezione 8.15) lo si poteva ancora aggiungere dal Content Browser con *Add Feature or Content Pack*. In questa lezione il docente constata che nella 5.8 quella via non c'è più ("una volta bastava fare tasto destro, importa starter content, hai finito. Hanno tolto sta roba, rottura gigantesca").

Le slide sono nella cartella condivisa. Chi preferisce seguire sulle slide invece che sullo stream può farlo: il docente cerca di **caricare sempre le slide in contemporanea**, quando le usa, nella cartella condivisa. Conviene, anche perché nelle slide ci sono dei **link** che serviranno.

11.4 Lavoro artistico e lavoro di gameplay

Prima della terminologia, una "piccola introduzione" a "uno dei concetti più importanti di questa disciplina": la distinzione fra **lavoro artistico** e **lavoro di gameplay**.

11.4.1 Il termine ombrello, di nuovo

"Cosa copre il termine level design? Quando dico level design, cosa intendo?" Qualcuno risponde con la parola magica, "dipende" ("è la risposta a praticamente ogni domanda che faccio, ma non sempre"); la risposta giusta, vista la lezione precedente, è che level design è un **termine ombrello** (figura 8.1): si usa per riferirsi a un sacco di cose diverse. Ma queste cose, "a livello di massima", si possono sintetizzare in **due campi**: il campo **artistico** e il campo **di design** (figura 11.1).

Il campo artistico. Un level designer può trovarsi a fare **illuminazione**; **enrichment**, che si vedrà durante l'anno, cioè **arricchire gli spazi**: mettere cose sugli scaffali, piante e così via; in certi casi il **post-processing**; in certi casi gli **effetti visivi**, a seconda di ciò che serve: mettere fumo, fuochi in giro per una stanza. Sono tutte cose legate al settore artistico.

Il campo del design. Dove metto i **nemici**? L'**encounter design**.[5] Dove metto i **blocchi** che serviranno a guidare gli artisti su come devono essere fatte le pareti: i **blockout**, eccetera. Sono cose di design perché si occupano, "non necessariamente puramente, ma principalmente", dell'**esperienza del giocare al gioco**.

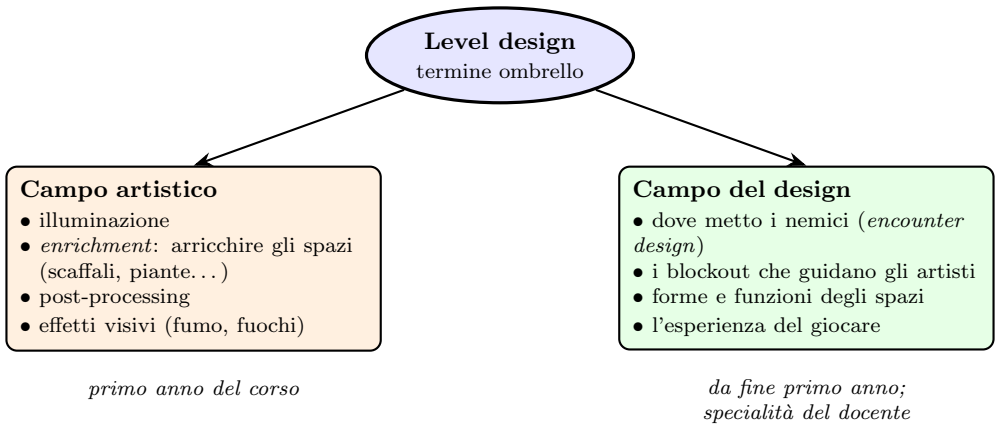


Figure 11.1: I due macro-campi del level design. Negli studi grandi tendono a essere coperti da persone diverse; negli studi piccoli lo stesso level designer fa entrambe le cose.

11.4.2 La specializzazione

Ecco perché il docente fa questa precisazione: nel mondo del lavoro **i level designer tendono a specializzarsi** in uno dei due campi.

- "Dipende, ovviamente": in uno **studio molto piccolo** bisogna fare entrambe le cose, "e a me capita di dover fare entrambi perché c'è poco da fare".

[5] Nella trascrizione "il counter design": dal contesto ("dove metto i nemici?") si tratta quasi certamente dell'*encounter design*, la progettazione degli scontri.

- In uno **studio più grosso** c'è una certa specializzazione, e certi ambiti del lato artistico (e del level design in generale) sono spesso coperti **direttamente dagli artisti**, soprattutto cose come l'**illuminazione**: se lo studio è abbastanza grande, c'è una persona esperta di illuminazione che si occupa solo di quello, perché ad alti livelli è una cosa piuttosto complessa.

Il test di assunzione. La conseguenza pratica: se un giorno si risponde a un annuncio per **level designer** e viene chiesto di fare un **test**, spesso si può **scegliere** se fare un **test artistico** o un **test di gameplay**, a seconda di ciò in cui si è specializzati.

Il docente, e il collega di Roma. Il docente è specializzato nel **design**, nel lato gameplay del level design: "che non vuol dire che non so niente del lato artistico, al contrario". Il suo collega di Roma, Andrea, proviene invece dal **lato artistico**, ed è esperto nel rendere gli spazi e le stanze.[6]

Che cosa vuol dire per il corso. Il **primo anno** è più incentrato sul **lato artistico**: comporre cose in engine, imparare come funziona l'engine, che cosa sono i materiali, l'illuminazione, eccetera. Verso la **fine del primo anno** si comincerà a toccare il **lato del design**. Del lato artistico il docente dice "non è il mio punto di forza, ma sono comunque cose che so fare"; del lato design, "questo è il mio punto di forza": "non vedo l'ora di arrivarci", perché è ciò di cui può parlare con più fiducia, confidenza e aneddoti. Ma intanto, "voglio che vi ricordiate questa distinzione principale".

11.5 Il blockout

"Si parte dai termini." Il primo è uno che gli studenti hanno già usato per il task: il **blockout**.

11.5.1 Un concetto, molti nomi

A livello tassonomico, si possono incontrare molti nomi per la stessa cosa: **block out**, **grey boxing**, **block mesh**, **white boxing**. "Questo termine ha un nome diverso praticamente in quasi ogni studio." Nell'ambiente in cui lavora il docente si tende a dire *blockout*; in studi in cui ha lavorato si diceva *white boxing*, in altri *grey boxing*.

Sono sinonimi. Il docente torna sul punto più avanti, a proposito di *Skyrim* (sezione 11.7.2): capiterà di incontrare qualcuno che dice "no, in realtà il grey boxing è una fase del blockout che implica questa cosa, il white boxing indica un'altra fase". "Io vi posso assicurare per esperienza personale che sono usati in maniera **completamente interscambiabile**": *white box*, *white blocking*, *grey blocking*, *grey box*, *blockout* sono "sempre la stessa cosa, a livello di massima". Saperlo evita di confondersi.

11.5.2 Perché non partire dall'arte finale?

Sulla slide c'è uno screenshot di un livello di *God of War* durante lo sviluppo:[7] tutto forme semplici e grigie. "Perché si fanno così i livelli? Perché non partire direttamente con la capanna e tutto quanto? Con il molo fatto bene, con la gru, con tutte le cose pronte degli artisti?" La risposta della classe: **prima si parte con una bozza**.

La matita prima dell'inchiostro. Il docente chiede chi sia appassionato di fumetti e disegno, e qual è il suo fumettista preferito. Il punto: quando si fa un disegno, di solito non si prendono subito gli inchiostri, perché se si sbaglia qualcosa "è tutto da buttare via". Si parte con la **matita**: uno sketch, si cancella, si riprova, si ricancella, si riprova, e solo quando le matite sono finite si passa all'inchiostro. "Il level design funziona in una maniera molto simile."

[6] Nella trascrizione "esperto nella redditività, stanze"; il senso è quello di un esperto del lato visivo degli ambienti (la resa, il *rendering* delle stanze), ma la parola esatta non è ricostruibile.

[7] Il docente lo indica come "quello del 2024". Non esiste un *God of War* uscito nel 2024: si tratta probabilmente di *God of War Ragnarök* (2022, arrivato su PC nel 2024) o del *God of War* del 2018; dalla trascrizione non è possibile stabilirlo.

Quando si comincia un livello, non si fa praticamente mai così: prendere gli asset (le case, le strade, tutto), metterli nel livello, testarlo e, se qualcosa non funziona, dover cambiare un sacco di roba. "No. Il livello di base è un blockout": **forme semplici pensate per buttare giù l'idea del livello il prima possibile**.

Arrivare presto a qualcosa di giocabile. Il motivo principale, come suggerisce uno studente (Alessandro), è che uno degli scopi dello sviluppo di un videogioco è **arrivare il prima possibile a qualcosa di giocabile**. Se si aspetta che gli artisti finiscano tutto (l'erba dappertutto, le case, il cielo, le rocce piazzate in giro) e solo a quel punto si fa il primo test, e ci si accorge che qualcosa non funziona, per esempio che gli spazi sono troppo stretti per il giocatore, bisogna **rifare tutto da capo**. Invece: prima si definiscono le forme generali più ampie, e poi si gioca. "Fondamentalmente si fa quasi tutto il gioco così."

Le realtà di produzione. Ovviamente le realtà di produzione sono più complesse: una **demo** o un **trailer** vanno fatti, e non si possono fare con un blockout. "Che cosa facciamo vedere alla gente del gioco che stiamo sviluppando, mentre lo sviluppiamo?" è un problema classico. Ma lo scopo principale resta arrivare a qualcosa di giocabile il più velocemente possibile. La questione dei blockout verrà ripresa più avanti; questa è un'introduzione a "una delle parti più importanti dello sviluppo di un livello".

11.5.3 Testare idee e rispondere a domande

Lo scopo del blockout è **testare idee e rispondere a domande**. L'esempio del docente:

> Da questo punto riesco a vedere la torre in cui devo andare? Ci metto un blocco che ha più o meno le dimensioni di una casa. No, non si vede bene, è un po' coperta da questa casa. Ok, la sposto più a sinistra. Perfetto, così si vede benissimo. Andiamo avanti, e poi la sostituiremo con i lavori degli artisti, dei modellatori 3D.

"Il blockout è il vostro sketch." Tutti gli studenti, con il task, hanno fatto lo sketch della propria cameretta. Lo scopo è rispondere alle possibili domande sui problemi che vengono fuori durante la creazione di un livello, e tirare giù idee. Ci sono anche altre fasi nella creazione di un livello, ma per ora è questa che interessa.

Approfondimento: il blockout secondo Totten. In *An Architectural Approach to Level Design*, Christopher Totten indica come errore comune proprio quello di **presumere che il gameplay non si possa testare finché non c'è l'arte** nell'engine: è molto più efficiente progettare gli spazi del livello (e gli script necessari) mentre l'arte viene sviluppata, per verificare se il progetto su carta produce davvero l'esperienza voluta. I prototipi si costruiscono con i **primitivi dell'engine**, e in assenza di primitivi (come in molti engine 2D) con *placeholder art*, la cosiddetta *programmer art*: quadrati, cerchi, triangoli. Nel libro, l'art director Greg Grimsby (*Warhammer Online*) descrive il proprio processo in sette passi: obiettivi di gameplay, tema visivo, punti di interesse, flusso, mappa, **prototipo in whitebox** ("l'obiettivo è avere il guscio di base per testare: il flusso, le visuali, i tempi di percorrenza, le linee di vista"), e solo alla fine, una volta validato il layout con i test, l'arte finale ("renderlo carino"). Ed è Grimsby stesso a ricordare il limite del whitebox: un livello può funzionare anche così, ma *Dead Space* "non fa altrettanta paura senza le luci e l'atmosfera" dell'arte: il blockout risponde alle domande di gameplay, non a quelle di atmosfera.

11.6 Due esempi: Call of Duty e Uncharted

"A me piace cercare di portare esempi pratici, quando riesco a scovarli fra archivi, dichiarazioni, o chiedendo ad amici o persone che hanno conoscenze."

11.6.1 Una mappa multiplayer di Call of Duty

Il primo esempio è una mappa multiplayer di *Call of Duty* ("quello del 2019", cioè *Call of Duty: Modern Warfare*), realizzata da un level designer di cui il docente mostra il lavoro.[8] In alto c'è la versione in blockout,

[8] Nella trascrizione la mappa è chiamata "DOCS" e il level designer "Brian Baker"; né l'una né l'altro sono verificabili dalla trascrizione, e li si riporta solo come indicazione. Il docente aggiunge: "non chiedetemi quale studio, perché a *Call of Duty* ci

in basso la versione finale (figura 11.2). “Che cosa notate?”

Materiali, griglia, colori. Anzitutto, nel blockout c’è una **scelta di materiali precisa**:

- una **griglia**, che aiuta a tenere presenti le distanze, la **metratura** di ciò che si sta facendo;
- dei **colori** pensati per **segnalare** qualcosa in particolare.

A che cosa servano esattamente quei colori “bisognerebbe chiederlo direttamente al level designer che li ha scelti”. Secondo il docente, lo scopo principale è aiutare a notare, in un ambiente in cui tutto ha la stessa texture, che lì c’è qualcosa: per esempio **una finestra**. E la cosa è stata rispettata: la finestra del blockout e quella della versione finale sono lo stesso edificio. Nel blockout interessano solo le forme; il lato estetico “assolutamente zero”.

Perché segnalare una finestra? L’orlo **rosso** intorno all’apertura serve a dire al giocatore: *occhio, lì c’è una finestra*. Ma perché un level designer vorrebbe farlo notare? La finestra è al primo piano di un edificio, e in *Call of Duty* del 2019 il giocatore non ci può arrivare.[9] Lo scopo, in realtà, è segnalare: **da lì potrebbe spararti qualcuno**. “Tieni presente che lì magari c’è qualcuno appostato che ti sta puntando col fucile.” E questa cosa è stata **mantenuta nella versione finale**, perché una delle cose che fa un blockout è **segnalare l’intento** delle varie parti.

La luce come segnale. Nella versione finale, guardando l’**illuminazione**, si nota un’attenzione particolare: il giocatore che arriva correndo e svolta l’angolo deve notare che lì c’è un colore diverso rispetto alla parete. L’hanno ottenuto inserendo una **luce** (probabilmente dentro c’è una lampadina) che fa notare: ehi, lì c’è un’apertura nel muro, potrebbe esserci qualcuno che ti aspetta dietro l’angolo per spararti. “Questo è il lavoro del level design: fare attenzione a questo tipo di dettagli.”

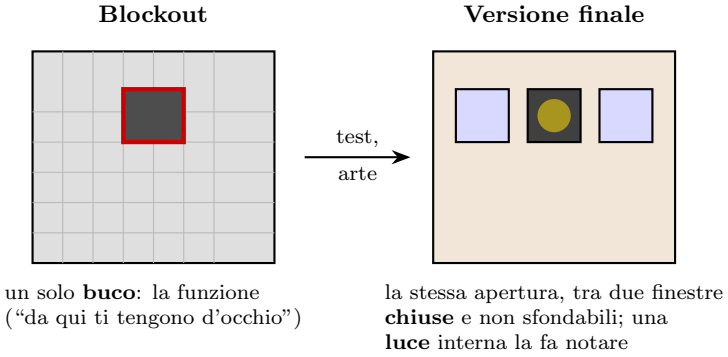


Figure 11.2: Ricostruzione schematica dell’esempio di *Call of Duty* (2019): nel blockout l’apertura è solo un buco segnalato in rosso, che dice “da lì potrebbero spararti”; nella versione finale l’intento è mantenuto, l’apertura diventa una finestra resa verosimile da altre finestre chiuse, e una luce la rende leggibile a chi svolta l’angolo.

Funzione prima, estetica poi. C’è anche una differenza tra blockout e versione finale: nel blockout c’è **una sola finestra aperta**, mentre nella versione finale ci sono più finestre. Il docente si spinge oltre: quando si faceva questo livello, nella fase delle forme, probabilmente non c’era neanche l’idea di una finestra. L’idea era: **questo è il buco da cui puoi tenere d’occhio il cortile**. “Quello è puro gameplay, quella è pura considerazione di gameplay.” Successivamente si sarà deciso che quel buco era una finestra, e per radicarlo in un mondo **verosimile** sono state aggiunte altre due finestre, chiuse, che però non si possono sfondare. A livello funzionale è un buco; a livello estetico, una serie di finestre.

lavorano tipo trenta studi diversi”.

[9] Uno studente obietta che in un altro capitolo della serie il personaggio ha il doppio salto (la trascrizione dice “Wolverine Warfare”, probabilmente uno dei capitoli con movimento avanzato, come *Advanced Warfare* o *Infinite Warfare*); il docente risponde che “è un discorso a parte”.

Perché il blockout costa meno. "Il blockout ha uno scopo puramente funzionale." Lo si fa, lo si testa, si gioca la mappa un paio d'ore con altre persone ("ovviamente tutto quello che sto dicendo è ipersemplificato: a *Call of Duty* avranno procedure molto più complesse"). L'apertura in quel punto non va bene? La si sposta un po' più a destra, si sistemano altre cose nel blockout, si va avanti. Se invece si fosse partiti dall'**edificio finito**, spostare quella finestra un po' più a destra "sarebbe stato un enorme casino", perché tutto il resto del modello andrebbe cambiato. Prima il blockout; poi, in base a ciò che viene fuori dai test e dai *playthrough*, si dice "ok, questo è il livello, non tocchiamo quasi più niente". Solo a quel punto si fanno i **modelli**: in base allo spazio da rispettare, alle forme stabilite, alle **funzioni legate al gameplay** date dal blockout iniziale del level designer.

11.6.2 Il level designer dà ordini agli artisti?

"Una domanda sorge spontanea, vi leggo nel cervello: ma Stefano, questo vuol dire che un level designer dà gli ordini agli artisti? Si metteranno a fare la casa in base a quello che decido io col livello?" La risposta è: **dipende**. Dalla struttura del team, dalle funzioni, dal fatto che la casa sia un **modello unico** o un insieme creato con **unità modulari** che si possono riarrangiare "tipo Lego" (sezione 11.7.2). Ci sono varie procedure. Ma il messaggio è un altro:

> Prima di tutto si decide il **gameplay**; prima di tutto si decide la **funzione** degli spazi; prima di tutto si decidono le **forme** degli spazi. Dopodiché questi spazi vengono **estetizzati**.

Il gameplay regna sovrano. Chi si è mai chiesto come sia possibile che ci siano videogiochi in cui **la trama viene scritta pochi mesi prima della fine** dello sviluppo, invece di essere decisa dall'inizio, ha qui la risposta: in questo tipo di processo **il gameplay regna sovrano**. Prima ci si assicura di fare un gioco funzionale, efficace, entusiasmante; poi si decide la veste da dargli.

Ma non a cascata. Con una precisazione importante: la **veste artistica** è ovviamente decisa **in precedenza**. "Non è che prima si fanno tutti i blockout e poi si decide: ah, questo è fantascientifico o medievale?" E molto spesso, per questioni di produzione, **non si lavora quasi mai a cascata** (si fa una cosa, poi l'altra, poi l'altra): quante più cose possibile procedono **in parallelo**, "perché il tempo è denaro".

11.6.3 Uncharted: dichiaro che è una cascata

Il secondo esempio è l'aspetto che aveva *Uncharted* durante lo sviluppo. Il level designer dice: "qui ci voglio mettere una piccola cascata. Questa è una cascata. **Dichiaro che è una cascata**." E poi sta agli artisti, al level designer che si occupa del lato artistico o allo stesso level designer andare a prendere gli asset, piazzarli lì e fare in modo che siano fatti bene.

"Voglio che vi fissiate in testa che **questo è l'aspetto che hanno tutti i videogiochi nel corso del loro sviluppo**." L'argomento tornerà, con lezioni dedicate.

11.7 I cinque strumenti del blockout

"Restando in argomento blockout, come si creano i blockout?" Esistono **cinque strumenti principali** (tabella 11.1). Prima di vederli uno per uno, il messaggio che il docente vuole far passare:

> Questi strumenti **non sono mutualmente esclusivi**. Possono essere usati tutti insieme, a seconda di quello che dobbiamo fare. Il trucco è prendere **lo strumento più veloce per lo scopo**.

E non servono solo al blockout: molti di questi strumenti vengono usati anche nelle **fasi finali**, per costruire i livelli veri e propri e posizionare gli oggetti.

Strumento	Che cos'è	Vantaggi	Svantaggi
Primitivi	forme di base dell'engine (cubo, cilindro, sfera, cono, piano, scale...)	semplicissimi da usare, veloci	limitati alle forme disponibili (una catena curva richiede una vita)
Kit modulari	set di pezzi a misura (pareti, porte, finestre, angoli, pilastri) che si incastrano	velocissimi; spazi già a misura del gioco; spesso già i pezzi finali	rigidi: difficile uscire dagli schemi (una stanza circolare è quasi impossibile)
Brush e modellazione	modifica diretta della geometria (vertici, facce, estrusioni), nell'engine o in Blender	qualunque forma; con la pratica, velocità fenomenale	più lenti, richiedono precisione; curva di apprendimento
Scultura (terreno)	editor di terreno: alzare, abbassare, allargare, lisciare una superficie	l'unico modo sensato per colline e terreni naturali	non lavora su facce e vertici singoli; adatto solo al terreno
Spline	curva definita da punti, lungo cui l'engine dispone e piega una o più mesh	strade, cavi, recinzioni curve senza modellarli a mano; modificabili in un attimo	più tecniche; se la mesh si allunga troppo si vedono stiramenti

Table 11.1: I cinque strumenti principali per costruire un blockout (e, in buona parte, anche i livelli finali).

11.7.1 I primitivi

I primi sono quelli già usati nel task: i **primitivi**. "Come dice il termine, un primitivo è una forma base." Le forme di base di Unreal, che "sono comuni in praticamente tutti gli engine", sono forme che **non possono essere create da altre forme**: sono gli **elementi di base**. Il cubo, il cilindro, la sfera, il cono, il piano ("semplicemente piatto, per pavimenti, pareti e cose del genere"); Unreal offre anche le scale e altre forme.[10]

È il modo più basilare di fare un blockout: "hai il tuo cubo, lo stretchi; vuoi fare la colonna, prendi sei cilindri, pam pam pam, colonne fatte, e in cinque minuti ho fatto la prima versione del tempio in cui voglio ambientare la mia boss fight".

Vantaggi e svantaggi. I primitivi sono **molto semplici da usare** e da sfruttare: si va relativamente veloci. Lo svantaggio è che **si è limitati alle forme che si hanno**. Se si sta facendo un livello "alla *Dark Souls*", "alla *Elden Ring*", dove c'è un punto con delle **catene gigantesche** su cui si deve correre, fare una catena curva con i primitivi "ci metto una vita". Per questo ci sono altri strumenti.

Approfondimento: primitivi e griglie secondo Totten. Totten osserva che i primitivi di un engine tipico comprendono cubi (o parallelepipedi), sfere, cilindri e piani, a volte anche scale e coni, e che con essi si può costruire "una varietà sorprendente" di geometria: in molti engine quasi tutta la geometria spaziale più grande di un livello può essere fatta solo con primitivi. Un vantaggio ulteriore è che i primitivi **seguono un sistema di misure standard** dettato dall'engine. Il caso estremo è l'editor **Hammer** di Valve (lo stesso citato dal docente nella sezione 11.7.3), che impone di lavorare su una griglia con lo snapping sempre attivo, per evitare buchi nel livello che ne impedirebbero la compilazione: gli oggetti scattano a misure in **potenze di due** (16, 32, 64, 128 unità...).

11.7.2 I kit modulari

Per strutture più complesse, più particolari, il secondo strumento sono i **kit modulari**. Sulla slide c'è una serie di pezzi: "che cosa sono?" Pareti, e i buchi al loro interno sono **porte** e **finestre**. Con **dodici elementi** si può costruire l'interno di un intero palazzo, cioè un insieme di stanze, perché quei pezzi coprono **tutte le**

[10] Il docente distingue di passaggio le *forme* di base dalle *geometrie* ("le geometrie sono una cosa diversa, anche quelle prima o poi le vedremo"): si riferisce ai *brush* di geometria, visti nella sezione 11.7.3.

possibili connessioni tra una stanza e l'altra, più il pavimento: angoli, porte, pilastri per gli angoli che si incontrano sul lato esterno, perfino spazi di dimensioni diverse a seconda di come si vuole organizzare. Si vogliono delle colonne al centro delle stanze? Si piazzano i pezzi giusti.

Lego e tasselli. "Se i primitivi sono Lego, i kit modulari sono le caselle con cui si costruisce uno spazio."

Vantaggi. Sono **molto più veloci**: si costruiscono spazi in pochissimo tempo. In più, essendo kit già sistemati (in teoria) con **dimensioni utili per il gioco**, non ci si deve neanche preoccupare degli spazi: automaticamente si sa che funzionano. Di kit modulari ce ne sono molti tipi diversi; e non contengono solo forme basiche, ma anche scale e forme più complesse, componibili velocemente. Danno una **visione del livello molto vicina a quella finale**, perché manca davvero solo il lato estetico. E in un gioco come *Skyrim*, il kit modulare **è già ciò che si usa nella versione finale**.

Svantaggi. "Fatemi una stanza circolare con questo kit." È sostanzialmente **impossibile**: non si hanno gli strumenti. I kit modulari permettono di fare un sacco di cose molto velocemente, ma rendono **molto più difficile uscire dagli schemi**: "tendono a peccare di creatività", perché con quegli strumenti non si può fare più di tanto. Ma, di nuovo, si combinano gli strumenti: se c'è uno spiazzo e poi si entra in un palazzo con delle stanze, le stanze le si fa velocemente con il kit, lo spiazzo con altri metodi. L'importante è fare tutto il prima possibile, per poter subito testare le meccaniche, vedere se il livello funziona, buttarci dentro i nemici e provare.

Sistemi e contenuti. Per capire dove i kit modulari brillano, il docente richiama una distinzione: i **sistemi** sono le meccaniche di gameplay; i **contenuti** sono "le cose a cui effettivamente giochi": i livelli, le armi, tutto ciò che "fa parte, lo dice il termine, del contenuto del gioco stesso". Giochi come *Skyrim* o *Elden Ring* richiedono una **quantità enorme di contenuti**, "veramente vastissima", e ci sono punti in cui bisogna essere intelligenti nello sfruttare ciò che si ha a disposizione.

I dungeon di Elden Ring. "Chi ha giocato a *Elden Ring*? Qual è la parte più ripetitiva?" Uno studente cita una zona della capitale; il docente punta su altro: i **dungeon segreti** (catacombe, grotte), "tutti quasi uguali". "Posso quasi garantire che sono stati fatti tutti con i kit modulari."

I roguelike. Un passaggio ulteriore: i **roguelike** che costruiscono le stanze in maniera casuale usano proprio questo approccio: **pezzi di stanza riorganizzati** da un **algoritmo** che li genera ogni volta.

Il kit di Skyrim. Sulla slide c'è il kit modulare dei dungeon di *Skyrim*: contiene sia parti **decorative** sia parti più **pratiche**, tutte le pareti usate per costruire i dungeon. Per chi vuole approfondire, nella slide c'è il link alla **trascrizione di un talk** tenuto nel 2013 da uno dei level designer di *Skyrim*, che spiega come sono stati realizzati i dungeon con queste metodologie (dimensioni, studi, eccetera) e come è stato creato il kit.[11] Loro lo chiamano *grey box*: "giusto per tornare all'argomento che ognuno chiama il blockout con il termine che preferisce".

Approfondimento: la modularità secondo Totten. Totten ricorda che la modularità viene dall'architettura: dalla Rivoluzione industriale gli edifici si costruiscono con **elementi prefabbricati**, assemblati in cantiere; Mies van der Rohe e Philip Johnson progettavano con componenti prefabbricati di acciaio e vetro, e Le Corbusier ha progettato l'Unité d'Habitation con appartamenti come unità modulari impilate. Totten usa proprio la metafora del Lego: con un set di pezzi modulari, i livelli si assemblano "come set di Lego invece che come opere d'arte su misura difficili da cambiare". La modularità aiuta anche le **metriche**: nel gioco *SWARM!* le piastrelle modulari larghe due unità erano la misura minima su cui il giocatore poteva restare in equilibrio senza che il gioco risultasse ingiusto. E, come gli architetti tengono traccia dei pezzi prefabbricati con i *building schedule* (tabelle che indicano tipo, misure e posizione di porte, finestre, impianti), il level designer può documentare i propri pezzi modulari allo stesso modo.

[11] Il docente non dice il nome. Si tratta quasi certamente del talk "*Skyrim's Modular Approach to Level Design*" tenuto alla GDC 2013 da Joel Burgess (Bethesda), la cui trascrizione è pubblicata sul blog dell'autore.

11.7.3 Brush e modellazione

"Ok, questa è dove le cose si fanno interessanti." Il terzo strumento sono i **brush**, o la **modellazione**. La domanda che potrebbe sorgere: se si sta facendo una cosa da zero, e l'unica cosa che interessa è la forma generale del livello per "trovare la quadra iniziale", **perché non usare direttamente uno strumento di modellazione 3D**, invece di giocare con le forme o con i kit modulari?

La risposta è che **si può fare**. Alcuni engine permettono di **modificare direttamente la geometria**, come se si lavorasse in "una versione ipersemplificata di Blender o Maya". Per fare un letto, per esempio, si ritaglia una sezione di un blocco, la si prende e la si tira su: "e ho fatto il mio letto". Si fa un **editing diretto** di ciò che si sta costruendo nel livello.

Blender e Unity. Una delle pipeline più comuni quando si lavora in **Unity** ("che non ci interessa, perché noi lavoreremo in Unreal, ma è importante saperlo nel caso vi capiti") è lavorare direttamente in **Blender**: molto spesso i level designer costruiscono il livello in Blender, che salva direttamente nel progetto Unity, e fanno così l'intero blockout del livello molto velocemente.

Vantaggi e svantaggi. Il vantaggio: **si può fare quello che si vuole**, letteralmente qualunque forma. Non c'è il problema "non ho il primitivo giusto": la forma la si crea, la si modifica, la si gira come si vuole. Lo svantaggio ("non è una questione di peso, in realtà"): è **un po' più lungo**, bisogna essere più **precisi**, bisogna **saper usare questi strumenti**. Ma c'è un "ma" importantissimo: se si è disposti a impararli, questi strumenti permettono di andare a **una velocità fenomenale**, soprattutto per i blockout. Il problema è la **curva di apprendimento**: Blender richiede un po' di tempo per essere imparato (nel corso lo si toccherà); ma si vedrà soprattutto come fare questo lavoro dentro Unreal, con i **brush di geometria**.

Valve e Hammer. Chi ha giocato a un gioco di **Valve** (*Half-Life*, *Half-Life 2*, *Half-Life: Alyx*, *Deadlock*: "ve li consiglio tutti")[12] ha visto il risultato di questo approccio. "C'è un motivo se Valve fa un gioco ogni dieci anni: se lo fa, vuol dire che sono riusciti a fare il proprio lavoro, dato che hanno soldi infiniti." Valve lavora con **Hammer**, un editor proprietario per il proprio engine (**Source**): in Hammer è **tutto editing diretto** delle mesh e delle forme nel livello, da parte del level designer.

Brush e modello 3D. Qual è la differenza tra un **brush** e un **modello 3D**? "Lo vedremo": sono due sistemi diversi. Per ora basta sapere che **è possibile creare forme direttamente nell'engine**, ed è un altro dei modi di fare il blockout.

11.7.4 La scultura del terreno

Così come si può modificare la geometria tirando punti, vertici, facce, estrudendo e suddividendo, **si può anche scolpire**. Il quarto strumento è l'**editor di terreno** di Unreal (il *Landscape*), che si vedrà durante l'anno.

Se si sta facendo un livello ambientato su una **collina**, la collina non la si può fare con dei cubi: non si riesce a fare un terreno scosceso con i primitivi. Per i paesini sulla collina di *Elden Ring* si mettono cubi dappertutto? "Perdo un sacco di tempo e il risultato è orrendo." No: si usa un editor che permette di **alzare, modificare, allargare, scolpire** una superficie.

Scolpire non è modellare. Scolpire significa che non si stanno tirando facce o vertici individuali, come con i brush o la modellazione: "passo col mouse, dove passo col mouse si alza un pochino, poi lo liscio un po' dall'altra parte". È così che si fanno gli **ambienti naturali**.

[12] Nella trascrizione "AlphaFlash, AlphaFlash, AlphaFlash, Alix, Deadlock": i primi tre sono evidentemente i titoli della serie *Half-Life*.

11.7.5 Le spline

"Ultimo ma non ultimo, e molto importante": le **spline**. È uno strumento un po' più tecnico, ma è il quinto dei cinque, e serve sia alla concezione iniziale del livello sia al **posizionamento degli oggetti** nelle fasi finali.

Il problema della strada. Sulla slide c'è una strada. Se la strada è dritta è facile: un pezzo di strada, poi un altro pezzo. Ma se fa una curva? Si chiede agli artisti di modellare a mano ogni possibile strada del gioco? "A livello pratico magari è fattibile." Ma se poi un'area viene cancellata, o la strada deve curvare a sinistra invece che a destra, bisogna rifare tutto da capo. "La risposta, ovviamente, è no." E la risposta "è nell'intrinseco della domanda": come si gestiscono le cose che devono essere **piegate** dentro un videogioco?

Che cos'è una spline. Attenzione: "la spline qui non è la strada, **la spline è la linea**". Una spline è **una curva definita da dei punti**, che curva in base ai dati contenuti in quei punti: la **rotazione**, le dimensioni dei **vettori di ingresso e di uscita** (le tangenti), e altro. A questa curva vengono poi assegnate **una o più mesh**, e l'engine fa in modo che la mesh **segua la curva** (figura 11.3). Che cosa sia esattamente una mesh si vedrà tra poco (sezione 11.17); per ora, "pensate a un modello 3D".

Un pattern? Uno studente chiede se si tratti di un **pattern**, cioè sempre lo stesso elemento copiato. "Molto spesso sì", e il docente ringrazia, perché stava per arrivarci: molto spesso la spline usa **più oggetti ripetuti**, attaccati ai bordi uno dell'altro e modificati in modo da piegarsi.

Il problema dello stretch. Il perché lo spiega alla lavagna. Si prenda una superficie divisa in poligoni. Se la si **allunga** (la si "stretcha"), i dati del modello 3D non cambiano: si sta solo dicendo "allungalo". Quindi gli spazi interni dei poligoni vengono stirati anch'essi: **non si aggiungono dettagli**. Se una spline avesse un solo modello da deformare, si noterebbe che è "superstretchato": le texture stirate, i poligoni sempre più grandi. Per questo la spline **attacca tra loro più copie** del modello, e in curva le **comprime leggermente**. Le facce restano le stesse, ma se i modelli sono fatti bene e con cognizione di causa (abbastanza densi di poligoni) e la modifica è minima, "il giocatore non lo nota". Poi, certo, guardando con attenzione in qualunque videogioco si notano le giunture da qualche parte.

I cavi. Lo stesso vale per i **cavi**: non esiste che, per ogni possibile modo di posare un cavo nel gioco, il modellatore 3D faccia un cavo diverso. Oltre al lavoro, questo approccio farebbe diventare le **dimensioni del gioco** enormi. Si fa **un cavo dritto**, e poi con metodi come le spline ("non per forza le spline, ma la spline è il metodo di base più utilizzato") se ne dispongono e piegano varie copie.

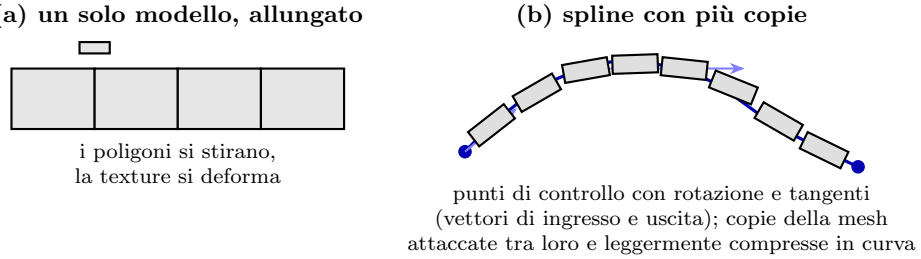


Figure 11.3: Perché le spline usano più copie di una mesh: allungare un solo modello (a) stira poligoni e texture senza aggiungere dettaglio; la spline (b) dispone lungo la curva più copie di un pezzo dritto, deformandole quel poco che basta perché il giocatore non lo noti.

Una lezione frontale. "Queste sono le cinque metodologie, per darvi un'infarinatura iniziale. Tutto chiaro fino ad adesso?" Il docente si rende conto che è una delle lezioni più frontali che deve fare, "ma vi devo insegnare tutta una serie di termini e c'è poco da fare".

11.8 Che cosa si farà nel primo anno

Kit modulari e primitivi. Nel primo anno si lavorerà **principalmente con kit modulari e primitivi**. Lo scopo è imparare il più possibile a usare gli strumenti. Si è deciso di partire dal **lato artistico** perché l'obiettivo è sviluppare una **conoscenza istintiva** di ciò che si usa: "in modo che se ti dici 'voglio fare un livello così', non devi perdere tempo a cercare come si fa: sai già come si fa, e vai".

L'allenamento dell'occhio. Quest'anno, quindi, si terrà d'occhio la **costruzione** e l'**estetica** dei lavori. Ci sarà una forte concentrazione su:

- attenzione a **ciò che si mette insieme**;
- attenzione alla **coerenza** del lavoro;
- attenzione agli **spazi**: "parleremo tanto di suddivisione degli spazi";
- attenzione all'**occhio**: "questo sarà l'anno dell'occhio".[13]

Prima si impara a usare gli strumenti, poi li si applicherà più precisamente ai videogiochi.

11.9 Asset e actor, di nuovo

Si passa alla terminologia dell'editor, partendo da un termine già visto: **asset** (sezione 8.15). Uno studente risponde correttamente, e il docente precisa:

Asset un **file salvato nel progetto**, su disco;

Actor un'**istanza** di quell'asset **creata all'interno del gioco**, nel livello.

Mentre Unreal si apre sul computer nuovo ("è totalmente vuoto", nessun progetto già pronto, e serve l'accesso a internet), il docente ricorda la disposizione dell'editor vista la volta precedente:

- in basso c'è il **Content Browser**: gli asset **del progetto**, interni al progetto;
- a sinistra (*Place Actors*) ci sono gli **asset di sistema di Unreal**: sono sempre asset, ma sono quelli **dell'engine**. Le luci, le forme di base e così via sono già presenti in Unreal, e non c'è bisogno di rifarle ogni volta che si crea un progetto nuovo.

Una conseguenza, emersa da una domanda durante la pausa: gli asset dell'engine, quelli presi dal pannello a sinistra, **non si possono modificare a livello di asset**, "perché sono di Unreal"; quelli modificabili sono quelli del Content Browser.

11.10 Coordinate globali e coordinate locali

Nell'attesa, il docente passa alla lavagna (Mattia conferma che da casa la si vede) per una distinzione che "tende a creare confusione all'inizio": quella tra **world coordinates** (coordinate globali) e **local coordinates** (coordinate locali). "So che probabilmente molti di voi già lo sanno, ma questa lezione mi serve principalmente per assicurarmi che sulla carta, in teoria, sappiate tutti di cosa sto parlando."

11.10.1 L'origine del mondo

Si disegna un mondo di gioco, con un giocatore. "Questo è il punto $(0, 0, 0)$. Che cos'è il punto zero?" Qualcuno propone il punto di spawn: no. Il **punto di spawn** è il punto in cui il giocatore viene messo in gioco, e non c'entra niente. Il punto $(0, 0, 0)$ è l'**origine delle coordinate** del mondo di gioco: il punto da cui partono tutte le coordinate.

[13] Nella trascrizione "questo sarà l'armo dell'occhio"; la lettura più plausibile è "l'anno dell'occhio", cioè dell'allenamento dello sguardo.

Quale asse è l'alto? Le coordinate sono x, y, z, ma l'ordine e il significato degli assi **cambiano a seconda dell'engine** (e a volte del progetto): in Unreal z **è l'altezza**; in altri engine z è la **profondità** e l'altezza è y. "Lungo dibattito su quale sarebbe la soluzione migliore; non esiste un modo condiviso." Bisogna farci attenzione quando si lavora con engine diversi: ciò che conta è sapere quale asse è altezza, quale larghezza e quale profondità.

Coordinate globali. Un giocatore che si trova, "a naso", a cinque unità dall'origine lungo un asse avrà coordinate $(5, 0, 0)$ ("pessima visualizzazione, perché è in 2D"). In 3D, un oggetto che svolazza nell'aria avrà coordinate come $(-3, -2, 5)$. Queste sono **coordinate globali** (*world coordinates*), perché sono basate sul **punto di origine del mondo** di gioco.

11.10.2 Coordinate locali: la spada e la cassa

Una **coordinata locale** è una coordinata **in base al genitore dell'oggetto**. L'esempio della lavagna: il giocatore ha in mano una **spada** (figura 11.4).

- La coordinata **globale** della spada è la distanza tra l'origine del mondo e la spada.
- La coordinata **locale** della spada è la distanza tra la spada e l'**origine del giocatore**, che di solito si trova **ai piedi**, a rappresentare lo zero.

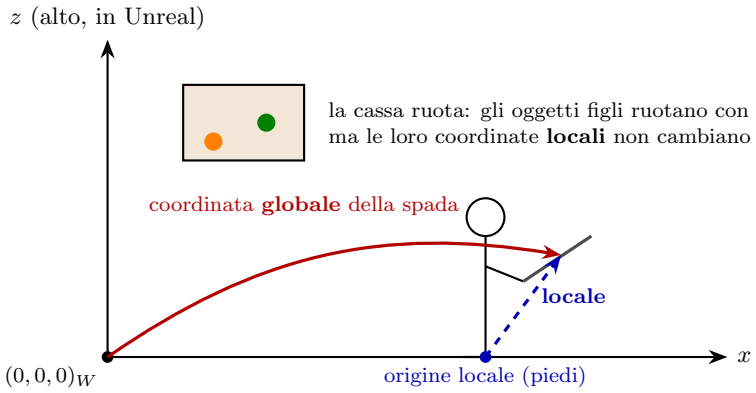


Figure 11.4: Coordinate globali e locali, come alla lavagna. La posizione globale si misura dall'origine del mondo; quella locale dall'origine del genitore (qui il giocatore, con l'origine ai piedi). Se il genitore si sposta o ruota, cambia la posizione globale dei figli, non quella locale.

Perché è importante. Quando si lavora in un livello, ma anche in qualsiasi cosa di programmazione, bisogna **rendersi conto se le coordinate che si stanno usando sono globali o locali**. Il motivo è che gli oggetti possono essere assegnati come **figli** di un altro oggetto. La spada è una cosa diversa dal giocatore, ma si sposta con il giocatore perché segue le **trasformazioni** applicate al giocatore. Una **trasformazione** (*transform*) è uno **spostamento**, un **cambio di dimensioni** o una **rotazione**. Se il giocatore si sposta o si gira, la spada si sposta o si gira; ma **le sue coordinate locali non cambiano**: all'interno del sistema di coordinate del giocatore, la spada è sempre nello stesso punto.

L'esempio della cassa. Quello della spada "non è il massimo come esempio", perché la spada si muove anche per conto suo, attaccata alla mano. Più pulito: una **cassa** con dentro degli oggetti. Se si gira la cassa, ci si aspetta che si girino anche gli oggetti. Se sono **figli** della cassa, si girano con lei; ma all'interno delle coordinate locali gli oggetti **non si sono spostati**, "perché il genitore, il mondo locale, si è spostato". Lo zero locale si muove con il genitore; rispetto a quello zero, i figli restano sempre uguali.

11.10.3 La pausa

Il progetto nuovo deve compilare gli shader da zero ("la speranza è l'ultima a morire"), quindi il docente anticipa leggermente la **pausa**: una ventina di minuti, "potete chiacchierare, ve lo concedo". Durante la pausa risponde ad alcune domande individuali:

- *Si possono unire più oggetti in un unico oggetto?* Risposta breve: non si può. Risposta lunga: ci saranno lezioni dedicate alla **modellazione 3D**, perché fondere più cose in uno stesso oggetto si può fare, ma è più complesso. Per ora: **raggrupparli** (Ctrl+G, sezione 11.12).
- Gli asset dell'engine non si modificano (sezione 11.9).
- Per far seguire un oggetto a un altro, nello spazio locale, lo si può **parentare**: lo si trascina sull'altro nell'Outliner, e "vedete che adesso sono uno sotto l'altro"; se lo si sposta, si sposta anche l'altro. Rifacendo il trascinamento, li si separa. "Adesso lo faccio vedere a tutti."

Prima di riprendere, l'invito a tenere Unreal aperto, con un progetto qualsiasi (anche quello del task), per vedere i concetti direttamente nell'interfaccia.

11.11 Lo spazio delle coordinate nell'editor

Ripresa la lezione (Mattia, da casa, sta finendo di installare la 5.8 sul portatile), il docente mostra in pratica i concetti spiegati alla lavagna.

Non serve un genitore. Prima una precisazione: la distinzione tra coordinate globali e locali vista alla lavagna serviva a dare un'idea di come si ragiona. In realtà, **non è necessario che ci sia un genitore** per parlare di coordinate locali: **ogni oggetto ha le proprie**.

Il pulsante con il mondo. Cliccando su un qualsiasi oggetto, in alto nella viewport c'è un pulsantino con l'icona di un **mondo**: indica **in quale spazio di coordinate** si sta lavorando. Nella traduzione italiana si chiama **Spazio del mondo** ("non chiedetemi come sono state fatte le scelte di traduzione di Epic: sospetto, vista la loro passione per l'intelligenza artificiale, che c'entri qualcosa"). Cliccandoci sopra si passa in rassegna:

Spazio del mondo (*World*): le frecce del gizmo indicano gli assi x, y, z **dell'intero mondo di gioco**, "il punto di riferimento più alto che si può avere". Ruotando un oggetto, le frecce restano uguali.

Spazio locale (*Local*): le frecce **seguono gli assi dell'oggetto stesso**, e ruotano con lui.

Spazio genitore (*Parent*): le frecce seguono gli assi del **genitore** dell'oggetto. Se l'oggetto non è figlio di niente, il suo genitore è il mondo, e lo spazio genitore coincide con quello globale.

"Ricordatevi: globali, locali, parentali."

La trave in diagonale. L'applicazione è chiara. Si prenda un oggetto allungato e ruotato in diagonale (una **trave**, un ponte). Se lo si deve spostare lungo la sua direzione lavorando in coordinate globali, è "un po' antipatico essere precisi": si può trascinare il centro del gizmo, ma ci si sposta leggermente a destra e a sinistra, senza garanzia di precisione. Passando alle **coordinate locali**, invece, la trave la si trascina **perfettamente nel suo "avanti"**, lungo il suo asse X locale (figura 11.5). "È parecchio importante."

11.12 Genitori, figli e gruppi

11.12.1 Parent e child

Lo spazio genitore introduce il concetto di **genitorialità**: il rapporto **parent** e **child**, genitore e figlio. Sullo schermo, due cubi: quello a sinistra è il genitore, quello a destra il figlio.

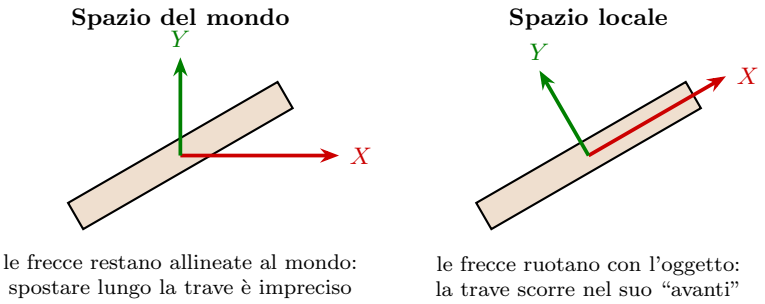


Figure 11.5: Lo stesso oggetto ruotato, con il gizmo in spazio del mondo e in spazio locale (vista dall'alto). La *scala*, invece, lavora sempre in spazio locale.

Come si crea. Nell'**Outliner**, a destra (sezione 8.14), si afferra un oggetto e lo si **trascina su un altro**: lo si sta assegnando come suo figlio. I figli compaiono sotto il genitore, e si possono **racchiudere e riaprire** con la freccina accanto al nome (il docente rinomina il figlio *Cube Child*).

L'asterisco. Una piccola parentesi: l'**asterisco** accanto al nome di un livello o di un asset significa che **c'è qualcosa che non è stato salvato**. Spostando un cubo, l'asterisco compare: il livello è stato modificato. "Se chiudete senza salvare, tutte le robe con gli asterischi sono perse." **Ctrl+S** salva; **Ctrl+Shift+S** salva tutto (sezione 8.18).

Che cosa eredita un figlio. "Che cos'è un child? Un child è qualcosa che **eredita tutte le trasformazioni del genitore**." Il docente chiede a una volontaria ("mi sembra una faccia da volontaria") che cosa succede se si sposta il genitore: "si sposta anche l'altro". Esatto: il figlio eredita, "da qui il termine", tutte le trasformazioni applicate al genitore, **incluse quelle di grandezza**. E qui c'è un dettaglio da notare: quando si **scala** il genitore, non si allarga solo l'oggetto, ma anche la **distanza del figlio dallo zero locale**, cioè dall'origine del genitore. Se si stretcha il genitore, si stretcha **l'intero insieme**, non i singoli oggetti ciascuno per conto proprio. Lo stesso per la rotazione e lo spostamento (e Ctrl+Z, Ctrl+Z, Ctrl+Z per tornare indietro).

Le gerarchie a più livelli. Con Ctrl+C e Ctrl+V (o Ctrl+D) si duplica il figlio, e si assegna il nuovo cubo (*Cube Child 2*) come figlio di *Cube Child*: ora c'è una **discendenza** (figura 11.6). Se le cose sono ereditate a livello di gerarchia, **quello più in alto influenza tutti quelli che gli stanno sotto**; ma modificando il secondo si influenza solo il suo figlio.

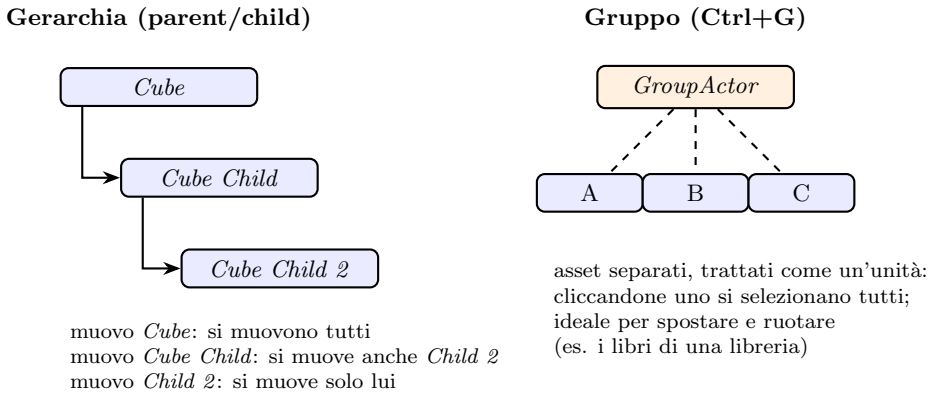


Figure 11.6: Gerarchia e gruppo. Nella gerarchia ogni livello eredita le trasformazioni di quello sopra; nel gruppo gli oggetti restano indipendenti ma vengono selezionati e trasformati insieme.

11.12.2 I gruppi

"La differenza, che potrebbe essere una domanda, tra questo e un **gruppo**." Selezionando più oggetti e premendo **Ctrl+G**, Unreal li raggruppa e crea un **Group Actor**, che segnala che tutti quegli oggetti formano un **gruppo unico**. Sono asset separati, ma l'editor dice: "queste cose qua sono un unico gruppo, considerale insieme".

- Spostando il gruppo, **si spostano tutti insieme**.
- Selezionando **uno** degli oggetti, il Group Actor li fa selezionare **tutti**, anche nell'Outliner.
- Si possono anche **scalare** insieme, come se fossero figli, anche se non lo sono; ma "se stretchate una roba raggruppata, possono succedere cose strane". Idealmente, i gruppi servono a **spostare e ruotare** cose insieme.

Il caso classico che capiterà di vedere nelle mappe fatte da altri è la **libreria**: i libri sono raggruppati, perché così è comodo spostarli e piazzarli in giro.

Stretchare: quasi solo nel blockout. Una nota a margine: lo **stretch** (la scala non uniforme) è qualcosa che capiterà quasi solo in fase di **blockout**. Una spada stretchata si vede che è stretchata, un mobile stretchato si vede che è stretchato: "è difficile che vi capiterà di stretchare le cose nella versione finale di un ambiente" (si ricordi il problema dello stretch nella sezione 11.7.5).

Una domanda dal fondo dell'aula. "Ci siete? Sto giocando a *Geometry Dash* là dietro: che state facendo?" Va bene, si riprende.

11.13 Ctrl+Z: anche cliccare è un'azione

"Piccola nota importante, che probabilmente avete già notato, ma è meglio dirlo in maniera esplicita." **Ctrl+Z** annulla l'ultima azione. Ma in Unreal, e in molti altri editor 3D, **cliccare su qualcosa nella viewport è considerato un'azione**. Se si clicca un oggetto, poi un altro, poi un altro, e si preme Ctrl+Z, si torna alla **selezione precedente**: non si annulla l'ultima modifica.

È utile, perché permette di tornare indietro anche nelle selezioni; ma all'inizio è **controintuitivo**, perché ci si aspetta che Ctrl+Z annulli l'ultima cosa *fatta*, non l'ultima cosa *selezionata*. Il rischio concreto: si fanno tre modifiche, si pensa "no, meglio di no", si preme Ctrl+Z tre volte; ma anche i clic tra una modifica e l'altra contano come azioni da annullare, e le modifiche non sono state annullate tutte. "Ricordate questa cosa: **cliccare su un oggetto è un'azione**."

11.14 Lo snapping

11.14.1 Che cos'è

"Che cos'è lo snapping? È molto semplice." Se si ha una **griglia** e si stanno posizionando degli oggetti (per esempio una serie di piattaforme), la griglia fa sì che ogni oggetto **si incastri automaticamente** in una posizione precisa. Dall'inglese *to snap*: qualcosa che **scatta** in una posizione. Afferrando un oggetto e trascinandolo, l'editor lo piazza automaticamente nel punto giusto della griglia. È molto utile con i **kit modulari** (sezione 11.7.2), ma "per un sacco di altra roba".

11.14.2 La barra degli strumenti della viewport

In alto a destra nella viewport ci sono i controlli già visti (sezione 8.16): la **selezione** (che "non usa mai nessuno da sola", tanto vale trascinare direttamente), lo spostamento, la **rotazione**, la **scala** (W, E, R), e lo **spazio delle coordinate** (sezione 11.11). Precisazione: la **scala è sempre locale**.

La griglia di posizione. Poi c'è il simbolino della **griglia**, "presente in tutti gli editor 3D". Spostando un cubo, si nota che non si muove di un pixel alla volta, ma a **incrementi di 10**: 10 unità di Unreal, cioè **10 centimetri** ("le unità di Unreal sono uno a uno con i centimetri"), perché la griglia è impostata a 10.

Il cubo da un metro. Il **cubo di base** di Unreal è largo **un metro**: un metro per un metro per un metro, con il pivot al centro (sezione 11.16). "Un metro, quanti centimetri sono?" ("Due tentativi, ma ce l'abbiamo fatta": cento.) Impostando la griglia a **100**, il cubo si sposta esattamente della propria larghezza. Con Ctrl+C e Ctrl+V si possono affiancare copie perfettamente allineate e costruire figure molto velocemente.

I valori disponibili. I valori della griglia sono **limitati** a quelli proposti nel menu ("un po' antipatico"): da qualche parte c'è un modo per impostarli a piacere, ma se si mantengono buone dimensioni quelli presenti bastano.

Quando spegnerla. Se invece si stanno mettendo, per esempio, delle **rocce** o degli **oggetti su un tavolo**, e gli scatti della griglia danno un risultato "molto falso", poco naturale, si clicca sull'icona per **disattivarla** e si sposta l'oggetto liberamente. Il consiglio:

> Usate la griglia quando serve. Abituatevi a pensare: questa cosa è più comoda se la faccio con una griglia che scatta, o no? Vi velocizza tanto il lavoro, rispetto a stare lì manualmente a spostare le cose un centimetro alla volta.

11.14.3 Lo snapping della rotazione

Lo stesso vale per la **rotazione**: di default procede a scatti di **10 gradi**. Disattivando lo snap si ruota liberamente. Per ruotare qualcosa esattamente di **90 gradi** si imposta 90, "tac" ("è un cubo, non si vede la differenza, però fidatevi"); per 45 gradi, 45.

I valori a mano, e le operazioni. Per un valore preciso che non è tra quelli proposti, lo si scrive nel campo della rotazione nei *Details*: ruotare di 12 gradi sull'asse z? Si scrive 12. Ruotarlo tre volte tanto? Si scrive `12*3`, Invio, e il risultato è 36 (il trucco dei campi numerici, sezione 8.16). "La matematica, purtroppo, sarà estremamente importante in questo tipo di mestiere."

11.14.4 Ruotare attorno a un asse

"Qualcuno si è chiesto: perché l'ha ruotato sull'asse z?" L'asse z è quello **verticale**. Quando si ruota **su un asse**, bisogna immaginare che si ruoti **attorno** a quell'asse: l'asse fa da **perno**. La rotazione sull'asse z (che infatti è blu, come la freccia di z) fa girare l'oggetto **orizzontalmente**, come una trottola.

Anche questo all'inizio può essere controintuitivo: si pensa che per farlo ruotare orizzontalmente si debba usare l'asse x, ma ruotando sull'asse x lo si fa girare **attorno all'asse orizzontale**, non sul piano orizzontale (tabella 11.2).

Asse (Unreal)	**Colore**	**Direzione**	**Ruotando attorno ad esso...**
X	rosso	avanti/indietro	l'oggetto si inclina di lato (*roll*)
Y	verde	destra/sinistra	l'oggetto si inclina avanti/indietro (*pitch*)
Z	blu	alto/basso	l'oggetto gira sul piano orizzontale (*yaw*)

Table 11.2: Gli assi di Unreal e l'effetto della rotazione attorno a ciascuno. I nomi *roll*, *pitch* e *yaw* sono quelli usati nei campi di rotazione dell'editor; a lezione il docente insiste soprattutto sull'asse z come perno verticale.

11.15 Il Content Browser e la lingua dell'editor

Il cassetto dei contenuti. "Piccola altra cosa": il **Content Browser**, che nella versione italiana si chiama **Cassetto dei contenuti**. Il docente usa Unreal in inglese e si scusa se non usa sempre la terminologia italiana. Il Content Browser si può **chiudere**: premendo **Ctrl+Spazio** lo si fa apparire, premendolo di nuovo lo si nasconde. Così si ha più spazio per vedere il mondo di gioco.

Cambiare la lingua. Uno studente chiede come cambiare la lingua dell'editor. Il docente non è sicuro che il percorso non sia cambiato nella 5.8, ma dovrebbe essere: **Modifica** → **Impostazioni dell'editor** (*Edit* → *Editor Preferences*), e cercare "lingua": lì la si sceglie. Da non confondere con l'**anteprima della lingua del gioco**, che serve, nei giochi localizzati in più lingue, a scegliere in quale lingua giocarli quando li si fa partire. Il docente, come si è visto, preferisce l'inglese: è più comodo, tra l'altro, per fare **ricerche** (sezione 11.22).

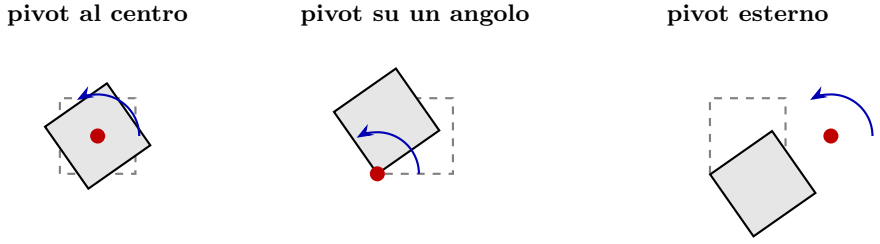
11.16 Il pivot

Si torna alle slide ("non chiudete Unreal, ci servirà"). **Pivot** ("a seconda che scegliate di pronunciarlo alla francese o all'inglese"): il pivot è **il punto che un actor considera l'origine delle sue coordinate**. Il cubo di base di Unreal ha il pivot **esattamente al centro**. Quindi, se lo si ruota, ruota **attorno al suo centro**. (Si era già visto, nella lezione precedente, che non tutti gli oggetti hanno il pivot al centro: lo decide chi crea la mesh, sezione 8.13 e figura 8.3.)

Il tasto G. Parentesi utile, anch'essa nella slide con le scorciatoie: premendo **G** si **nascondono i gizmo** e tutti gli strumenti interni dell'editor (frecce, righe, icone), per vedere la scena com'è davvero. Premendolo di nuovo, tornano.

Ruotare attorno a un angolo. "Molto figo, ma se questo cubo lo volessi ruotare attorno a un angolo?" Il caso classico: si sta posizionando un **ponte**, si sa dov'è il suo punto di partenza, ma il pivot del ponte è al centro, e ruotarlo per farlo combaciare è "una rottura": ruotare, spostare, non va bene, ruotare, spostare. Sarebbe comodo prendere il pivot e spostarlo all'inizio del ponte, così da ruotarlo attorno al punto di partenza.

Alt + tasto centrale. Si può fare: tenendo premuto **Alt** e cliccando con il **tasto centrale del mouse**, si sposta il pivot dell'oggetto **nel punto in cui si clicca**. Il docente clicca su un angolo del cubo, e ora il cubo ruota attorno a quell'angolo. Il punto può anche essere **esterno** all'oggetto: con Alt e clic centrale fuori dal cubo, il pivot finisce nel vuoto, e il cubo ruota (o si trascina) a partire da quel punto (figura 11.7). Può tornare utile per certi tipi di spostamenti.



Alt + clic centrale sposta il pivot **temporaneamente**
(anche fuori dall'oggetto); deselezionando torna quello del modellatore

Figure 11.7: Lo stesso cubo ruotato di 35 gradi attorno a tre pivot diversi (in rosso). Il tratteggio indica la posizione di partenza.

Uno spostamento temporaneo. Questo spostamento è **temporaneo**. Se non ci si ricorda la scorciatoia, lo si può fare anche dal **tasto destro** del mouse sull'oggetto, dove c'è una voce dedicata al pivot (per esempio *Imposta come offset del pivot*: in aula non funziona al primo colpo, ma le opzioni ci sono). Ci sono modi per **bloccare** il pivot e per impostare un pivot alternativo **che non viene resettato**; ma per adesso è meglio tenere a mente che **si resetta ogni volta**, "così non rischiate di fare casini": si sa sempre che il pivot è nel punto in cui l'ha impostato il **modellatore** che ha creato l'oggetto, e lo si cambia temporaneamente con Alt + tasto centrale solo quando serve.

11.17 Mesh e collisioni

11.17.1 Che cos'è una mesh

"Una nota importante: le mesh." Detto "terra terra, nella maniera più brutale possibile", una **mesh** è **un insieme di punti e di segmenti** che connettono questi punti. (Il docente la disegna alla lavagna: "i modellatori di Tadic che vedono che disegno una roba del genere mi sparano"[14], "ma è illustrativo".)

Tutto è fatto di triangoli. Chiunque abbia giocato a un videogioco avrà notato che, guardando le cose da vicino, sono composte di **poligoni**. "Non stiamo a parlare di quad e tutto il resto: stiamo semplici sulla teoria della modellazione 3D. Si parla di **triangoli**." Perché il triangolo? Perché è **la figura geometrica che si può costruire con il minor numero di punti**: un punto non è una figura, una linea non ha superficie. Tre punti collegati formano la **superficie più semplice** possibile: il triangolo è **l'unità di base** della grafica 3D. "In maniera molto semplificata, perché non è esattamente vero, ma per i nostri scopi": tutto ciò che si vede in un engine 3D è fatto di piccolissimi triangoli.

Una rete. *Mesh*, in inglese, vuol dire **rete**, "retina", come nei vestiti a rete. Una mesh è un **file** che specifica **l'insieme dei punti** (vertici) e **in che modo sono collegati tra loro** (le linee); le superfici delimitate da queste linee sono i **poligoni**, su cui vengono poi proiettati i **materiali** e le **texture** (sezione 11.21).

La testa in wireframe. Sulla slide c'è la mesh di un corpo: la testa è formata da poligoni. Quelli visualizzati non sono tutti triangoli (molti sono quadrilateri), ma "non stiamo entrando nel discorso di come funziona la modellazione 3D": edge, raggruppamenti e così via. Lo si potrà vedere in Unreal con la modalità *wireframe* (sezione 11.22).

11.17.2 Le collisioni sono un'altra cosa

"Nota importantissima, nota estremamente importante":

> **Mesh e collisioni sono due cose diverse.** Se vedete qualcosa in 3D, non è detto che sia fisico: qualcos'altro deve dire all'engine che quella cosa è solida.

Le mesh sono solo una **rappresentazione tridimensionale virtuale**. La **collisione** è una cosa completamente separata, e **non è automatica**. Unreal e molti altri engine possono usare la mesh stessa come modello di collisione, ma non è automatico e **non è consigliato**, perché la mesh è super dettagliata e il calcolo diventa molto pesante (di solito si usa una forma di collisione più semplice).

Lo studente che sprofonda. Quello che deve restare in testa: mettendo in gioco un modello 3D, **non è automatico che abbia una collisione**. Bisogna **controllarlo**, perché "è una cosa che succede spesso agli studenti": "che bello questo livello"; il personaggio cammina, salta sulla roccia e **sprofonda nel terreno**, perché la roccia non ha la collisione.

[14] Il nome è riportato così nella trascrizione; non è chiaro a chi o a quale studio si riferisca.

11.18 Le luci

"Altra cosa importante da vedere sono le luci. Per questo mi farebbe comodo un po' di buio." Ogni nuovo livello creato da un template di Unreal ha una cartella **Lighting**, che si vedrà tra poco (sezione 11.20); per ora il docente abbassa l'intensità della *directional light* per avere una scena più scura. Nel pannello a sinistra (*Place Actors*) c'è la categoria **Luci**: sono tutte le luci che si possono inserire (tabella 11.3). Il dettaglio di ciascuna verrà in lezioni dedicate: "questo è per farvi vedere cosa sono" e mettervi in mano la possibilità di fare cose velocemente.

Luce	**Come emette**	**Note dalla lezione**
Point light (punto luce)	in ogni direzione, a sfera	raggio di attenuazione; *source radius*: se maggiore di zero la luce non è più un punto e le ombre si sfumano; colore, intensità
Spot light (faretto)	a cono, in una direzione	come un faro, una torcia; intensità in candele (cd), il cursore si ferma a un massimo ma a mano si può andare oltre
Rect light (luce rettangolare)	da una superficie, a semisfera	poco usata; utile per una finestra illuminata che proietta luce verso l'esterno
Directional light (luce direzionale)	raggi paralleli e infiniti su tutto il mondo	è il sole: conta solo la rotazione, non la posizione; legata al cielo (tramonto); **una sola per livello**

Table 11.3: Le luci di Unreal viste a lezione.

11.18.1 La point light

Si comincia dal **punto luce** (*point light*). Trascinandolo nel livello (il docente sposta alcuni oggetti sul terreno, per avere qualcosa su cui proiettare), si vede che **all'interno di una sfera**, il gizmo della luce, viene proiettata luce.

La luce non è un'entità. Per selezionarla si clicca sull'**iconcina**: una luce non è un oggetto, "è un punto di diramazione di raggi di luce". Le iconcine servono a due cose: **segnalare** che lì c'è qualcosa, e permettere di **cliccarci sopra**. Premendo G (sezione precedente), infatti, lì non si vede niente.

Il modello della lampadina non fa luce. Ed ecco un'altra cosa importante: **la fonte di luce visibile** (il modello 3D di una lampadina, di un fuoco) **non emette luce** se non le si mette accanto una luce. "Poi in realtà un pochino ne emette, perché ci sono riflessi di colori ambientali, ma quella è un'altra cosa." Le luci dell'engine sono solo la *sorgente*; il modello è solo l'aspetto.

Il raggio. La luce si può selezionare e spostare, e spostandola si spostano anche le **ombre** che proietta. Ma le luci **non hanno influenza all'infinito** ("altrimenti buona fortuna a far funzionare tutto"): sono limitate al **raggio** visualizzato. Gli oggetti fuori dal raggio non vengono toccati dalla luce.

Le proprietà. Nei *Details* ci sono tutte le funzioni per modificarla:

- il **colore**, con il selettore;
- il **raggio di attenuazione** (*attenuation radius*): la distanza oltre la quale l'influenza della luce su un oggetto diventa zero;
- la **dimensione della sorgente** (*source radius*): se è zero, la sorgente è un **punto**; se è più larga di zero, è la dimensione della sfera da cui viene proiettata la luce. Allargandola, le **ombre diventano più sfumate**, perché la luce non è più un punto ma una sfera.

11.18.2 La spot light

La point light proietta luce **in ogni direzione**, a sfera. La **spot light**, invece, è una proiezione **a cono**: "lo dice il termine", è come un proiettore, "un faro". Serve quando invece di mandare la luce in tutte le direzioni la si vuole lanciare solo in una: "immaginatela come una **torcia**". Con G si vede il cono di luce proiettato sulla scena; anche qui la si seleziona cliccando sull'iconcina, e se ne regola tutto, compresa l'**intensità**: "volete spaccare gli occhi alla gente? Mettetela al massimo".

I campi non sono sempre bloccati. Una nota di usabilità. Trascinando il cursore dell'intensità, si nota che è **bloccato** a un massimo (nell'esempio, 160 candele: **cd**, perché Unreal usa unità di misura del mondo reale), e tornando indietro si ferma a zero. Ma questi campi **non sono sempre limitati brutalmente** ai valori del cursore: cliccandoci sopra e scrivendo il valore a mano, **in certi campi si può andare oltre il limite**. "Volete una luce da 2000 candele? Sì, però tenete presente che accecate chiunque."

11.18.3 La rect light

La **luce rettangolare** (*rect light*) è una **superficie** che proietta luce, non un punto: ruotandola si vede che la luce si interrompe, perché non proviene da un punto, ma da una superficie che illumina "a semisfera". Non è usata molto spesso, ma ha i suoi scopi: per esempio una **finestra illuminata** che proietta luce verso l'esterno di un palazzo, lungo tutta la sua larghezza.

11.18.4 La directional light: il sole

"Directional light: **non trascinatela** nel vostro progetto." Quando si apre un livello da un template di Unreal c'è già, nella cartella **Lighting** ("ignoriamo la fog per ora, che è la nebbia"). Il docente la riaccende e la rimette a **2 lux**.

Che cos'è. La **directional light** è **la luce del sole**: "è il vostro sole". Si chiama *direzionale* perché **tutti i raggi di luce sono paralleli**, nella stessa direzione. E se sono tutti paralleli e infiniti, perché coprono tutto il mondo di gioco, **il punto di origine non cambia assolutamente niente**: la si può mettere dove si vuole. Di solito, una volta trovato un posto in cui non dà fastidio, non la si tocca quasi più.

Il tasto F. Per ritrovarla, un'altra scorciatoia "importantissima" della slide: il tasto **F** porta la telecamera sull'**oggetto selezionato**. Si seleziona qualcosa nell'Outliner, F, e la viewport lo inquadra; si seleziona un'altra cosa, F, e così via.

Ruotare il sole. Per cambiare la posizione del sole **la si ruota**: ruotandola si spostano anche le ombre. (Il docente passa alle coordinate locali, "così ce l'ho più comodo"; e ripete: se un oggetto non è figlio di niente, il genitore è il mondo, e lo spazio genitore coincide con quello globale.)

Il sole e il cielo. Unreal **lega automaticamente** la directional light al **sole** del cielo (si veda la *sky atmosphere*, sezione 11.20): ruotando la luce si sposta il sole, con tutti i calcoli che ne derivano, incluso l'aspetto del gioco al **tramonto**, quando la luce diventa più rossa. "In questo modo potete giocare a essere Dio e spostare il sole, le stelle, eccetera." Le stelle sono un discorso diverso, perché quando la luce direzionale è sotto l'orizzonte non c'è niente che illumina il resto. E una curiosità: **nei videogiochi la luna, in realtà, è un sole con una texture diversa**.

Una sola. "Non mettetene mai due." **Ogni livello deve avere una sola directional light**: con due, si costringe l'engine a fare il doppio dei calcoli, senza senso.

11.19 Lo Starter Content, a mano

11.19.1 Un nuovo progetto, e un engine cambiato

"Ora costruiamo un livello da zero." Le slide erano state scritte basandosi sulla versione 5.6, quindi "ci sono un paio di cose che ignorate pure". Le istruzioni erano: creare un **nuovo progetto** con il **Third Person template**. Ma nella 5.8 la schermata di creazione del progetto è cambiata "totalmente": il docente crea un progetto vuoto e scopre che non ha più i preset di base che c'erano una volta ("fantastico"), mentre sui computer degli studenti i moduli necessari risultano già installati. È solo il computer nuovo a non averli.

Lo Starter Content non si importa più con un clic. Il problema principale è un altro, e il docente l'ha verificato il giorno prima: una volta bastava fare tasto destro, *importa Starter Content*, e si era finito. "Hanno tolto sta roba, rottura gigantesca." Per questo nella cartella condivisa c'è un **link a un pacchetto** con tutto lo Starter Content (texture, materiali, particelle, prop, eccetera) da importare manualmente.

L'accesso alla cartella. Gli studenti cominciano a scaricarlo. Il docente rende il link accessibile a chiunque lo abbia, chiedendo di **non richiedere l'accesso**: ogni richiesta di accesso a una cartella manda una mail a tutti quelli che fanno parte dell'organizzazione, e "ogni giorno ricevo un sacco di mail di gente che chiede accesso a roba che non ho mai neanche saputo che esistesse". (E il Wi-Fi dell'aula è un po' lento.)

La documentazione. Intanto, uno sguardo alla pagina da cui si è scaricato il pacchetto: il link sopra è alla **documentazione di Unreal** (nella versione 5.9: a sinistra si può scegliere la versione dell'engine, "perché un po' di robe cambiano, come avete visto"), che spiega come si importano le varie cose. Si ricordi l'invito della lezione precedente a leggere la documentazione (sezione 8.17).

11.19.2 Perché l'importazione normale non funziona

Il docente prova a importare il pacchetto con *Importa nella cartella corrente*, ma Unreal non lo prende. Chiede aiuto a uno studente ("Alessandro, vuoi fare da assistente? Ricordami come si importano gli asset da un progetto all'altro, che non lo faccio da una vita"), contatta il collega che gli ha passato il file, e si prende una "piccola seconda pausa" per risolvere.

La causa: i file del pacchetto sono `.uasset`, cioè **asset già importati** in Unreal. L'importazione normale serve a trasformare file esterni (immagini, modelli) in asset; questi asset lo sono già, e non vanno importati, ma **messi direttamente nella cartella del progetto**. (La slide con la procedura di importazione standard era sbagliata, perché il docente aveva frainteso che tipo di file gli venisse passato: "ignorate questa slide". La documentazione resta utile, ma per importare asset *non* pre-importati.)

11.19.3 La procedura

La procedura corretta, che gli studenti eseguono in aula:

1. **scaricare** il pacchetto e **decomprimerlo**;
2. in Unreal, nel Content Browser, **tasto destro** sulla cartella **Contenuto** (*Content*) e scegliere **Mostra in Esplora risorse** (*Show in Explorer*); in alternativa, aprire a mano la cartella `Content` del progetto;
3. **trascinare** la cartella decompressa **dentro** quella cartella, in Windows (non nel Content Browser: "non così, perché così vedete che non importa");
4. **chiudere e riaprire** Unreal.

"Che cosa sta succedendo? State mettendo dei nuovi file di Unreal dentro Unreal." Quando si avvia, Unreal **scansiona tutti i contenuti del progetto**, nota che ci sono dei file nuovi e li aggiunge. "Lo so, era molto più comodo quando lo si poteva selezionare con un clic."

Un cuscinetto di minuti. Il docente si scusa per il caos ("tra il computer nuovo e il fatto che io non uso la 5.8 per lavoro, incidenti di percorso") e spiega che, quando prepara una lezione, si lascia sempre **qualche minuto di cuscinetto** per imprevisti come questo, soprattutto con un engine aggiornato che ha cambiato un bel po' di cose, anche nell'interfaccia. Di conseguenza può capitare di finire la lezione un po' prima; in quei casi, se ci sono domande o argomenti di cui parlare, "chiedetelo voi". Mattia, da casa, avvisa che probabilmente riuscirà solo a seguire; "va benissimo".

11.19.4 Che cosa c'è dentro

Intanto si esplora il pacchetto. Lo Starter Content è suddiviso in **sottocartelle**:

- una cartella che ripete varie **forme di base** di Unreal ("ci interessa poco");
- **Architecture**: forme di base pensate per fare il blockout di architetture varie; "volete creare velocemente un edificio? Pam pam pam, e l'avete fatto";
- **Props**: oggetti di arredamento, sedie, lampade, lampade da parete, e altro;
- elementi **naturali**: una roccia, un cespuglio, cose che fanno sembrare tutto "un po' più naturale";
- una **statuetta di vetro**, trasparente: il vetro **deforma** ciò che c'è dietro, perché ha un materiale (uno *shader*) che deforma lo sfondo;
- e soprattutto i **materiali** (sezione 11.21).

Le anteprime mancanti. A volte, nel Content Browser, alcune anteprime non sono renderizzate (la roccia "non c'è"): Unreal ha avuto problemi a scattare l'istantanea. Trascinando l'oggetto nel livello, l'engine lo compila e poi mostra l'anteprima correttamente.

Troppe parentesi. Durante la dimostrazione si nota anche un difetto grafico (lo *z-fighting*: due superfici sovrapposte che "combattono" per essere visualizzate), ma il docente si trattiene: "purtroppo sono una persona che fa un sacco di parentesi". Spiegare una cosa gli fa pensare ad altre tre da spiegare, e cerca di tenere limitato il numero enorme di parentesi che potrebbe aprire.

11.20 Un livello da zero

11.20.1 Il livello vuoto

Ora si crea un livello "proprio partendo da zero, senza usare un template".

1. Nel Content Browser, tornare su **Contenuto** e lavorare **fuori** dalla cartella dello Starter Content: "cercate di tenere le cose ordinate".
2. **Tasto destro** → **Livello** (*Level*). Unreal chiede il nome (il docente lo chiama *NewLevel*).
3. Aprirlo con un doppio clic.

Il salvataggio. Aprendo il nuovo livello, Unreal chiede di salvare ciò che non è stato salvato finora: il livello con cui si è aperto l'engine risulta *Senza titolo*, cioè non era salvato da nessuna parte. Prima di cambiare livello, l'editor chiede conferma: "sicuro che non ti sei dimenticato qualcosa?". Va ricordato che **un nuovo livello non è salvato finché non lo si salva manualmente** (sezione 8.18).

Il nulla. "E abbiamo creato il nostro bellissimo nuovo livello. Siamo tutti d'accordo che questo è un livello? Siamo tutti appassionati di domande retoriche?" L'Outliner è vuoto: a parte le cose tecniche che Unreal genera per gestirsi, **nel livello non c'è assolutamente nulla**. Premendo Play, il vuoto.

11.20.2 Un piano, una luce, un cubo

Bisogna cominciare a metterci qualcosa:

1. un **piano** (dalle forme di base), messo in posizione $(0, 0, 0)$, e poi F per inquadrarlo; lo si allarga, perché il piano di partenza è sempre di un metro quadro;

2. "che cosa manca? Deselezionando il piano, qual è il problema?" **Manca la luce**. Uno studente propone la luce del cielo (*Sky Light*, "luce atmosferica" in italiano): inserita da sola, non succede niente. Si parte invece dalla **luce direzionale**: quando la si inserisce, Unreal la orienta automaticamente (a circa 46 gradi) "e ce l'abbiamo fatta";

3. un **cubo**, "giusto per avere un punto di riferimento chiaro".

"Tutto qua?" No: manca ancora qualcosa.

11.20.3 Una parentesi su Lumen

"Qui devo aprire una parentesi, legata a una roba che ha implementato Unreal." Sull'immagine si vede uno **sfarfallio**, un rumore. Quello è **Lumen**.[15] Chi ha giocato a giochi come *Clair Obscur: Expedition 33* lo avrà notato in certi angoli: la luce fa quell'effetto. Lumen è il sistema di **illuminazione globale dinamica** di Unreal, "una imitazione del *ray tracing*": una luce aggiornata in tempo reale che considera i **riflessi delle luci** sulle superfici circostanti. "È una finta luce, perché non è una vera luce: è una finzione dei riflessi della luce su una superficie." Infatti, quando la superficie esce dallo schermo, quegli effetti scompaiono, perché l'engine non sta più calcolando quei finti riflessi.

11.20.4 Perché il lato in ombra è nero

Guardando il **lato opposto** del cubo, è **completamente nero**. "Se io guardo sotto la mia mano, non è completamente nero. Che cosa sta succedendo, secondo voi, nel vuoto cosmico?" Uno studente: la luce si perde nel vuoto, non si riflette su nessuna superficie.

La luce ambientale. Esatto: in una situazione di vuoto totale, se non c'è una superficie che riflette la luce diretta del sole, il lato opposto a quello illuminato è totalmente nero ("è il motivo per cui la Terra ha la notte"). Nel mondo reale, invece, ci sono altre fonti di luce oltre al sole. La prima è l'**illuminazione ambientale**: la luce che **si riflette su qualunque cosa ci circonda**. È il motivo per cui, se si spegnessero le luci e ogni singolo computer dell'aula, ci sarebbe comunque un po' di luce: è la luce del sole che rimbalza su tutto ciò che sta fuori ed entra nella stanza.

Il cielo è una fonte di luce. Seconda cosa: il sole non è l'unica sorgente; **anche il cielo è una fonte di luce**. La luce del sole che si riflette sulla Terra e viene rispedita verso l'alto viene riflessa di nuovo verso il basso dall'**atmosfera**. È il motivo per cui il cielo è blu invece di essere nero ("se vi capiterà mai di andare nello spazio, non lo so cosa succederà nel 2050–2070": lo spazio è nero, il cielo della Terra è blu). L'atmosfera riflette la luce, ed essendo una luce che ci circonda totalmente, **contribuisce all'illuminazione ambientale**.

Il cielo, in Unreal. Ecco perché, trascinando solo una directional light, Unreal "dice: bene, vi ho fatto una luce direzionale; e voi guardate e dite: sì, ok, ma che cos'è questa roba?". Manca tutto il resto. Bisogna aggiungere anche la **Sky Light** (luce atmosferica) e una **simulazione del cielo**: la **Sky Atmosphere** (*atmosfera cielo*). Inserita quella, il cielo c'è, e c'è il sole.

[15] Nella trascrizione "Ruben".

11.20.5 La cartella Lighting: il modo pratico

Tutto questo si può fare **a mano**, un elemento alla volta, e nelle slide c'è il procedimento passo per passo. "Il vantaggio è che vi posso dare i consigli pratici da persona che con queste cose ci lavora": dopo averlo insegnato un po' di volte, il docente si è reso conto che **tutti i level designer**, invece di farlo a mano, **copiano la cartella**.

1. Aprire un livello che ha già la cartella **Lighting** (per esempio quello creato da un template).
2. Copiarne i **contenuti**. Attenzione: in Unreal, copiando la *cartella* dell'Outliner si copia solo la cartella, non ciò che contiene ("questa è la differenza tra lavorare con engine diversi"). Bisogna selezionare tutti gli elementi al suo interno.
3. Tornare al nuovo livello: per cambiare livello velocemente, **File** → **Livelli recenti**.
4. Assicurarsi che non ci siano già luci direzionali o atmosferiche (per non duplicarle) e **incollare**.

"La cartella è lì apposta per questo."

Che cosa c'è nella cartella. Ma bisogna **sapere che cosa c'è dentro**, perché regolando quegli elementi si cambia la luce atmosferica del gioco (tabella 11.4).

Elemento	A che cosa serve
Directional Light	il sole (sezione 11.18)
Exponential Height Fog	simula il **vapore acqueo** nell'atmosfera, che fa diventare le cose sempre più sbiadite con la distanza; "esponenziale" perché dipende dalla distanza dal terreno (l'altezza)
Volumetric Cloud	le **nuvole** volumetriche
Sky Atmosphere	la simulazione dell'atmosfera: il colore del cielo, il sole, il tramonto
Sky Light	l'illuminazione ambientale derivata dal cielo
Sky Sphere	lo **sfondo**: di base è bianco (deriva tutto dagli altri calcoli), ma ci si può mettere un materiale con un'immagine, come altri pianeti o le stelle; è ciò che nei videogiochi si chiama anche **skybox**

Table 11.4: Il contenuto della cartella *Lighting* di un livello creato da template, come descritto a lezione.

Lo skybox. Sulla **sky sphere** il docente si sofferma: è il vostro sfondo. Nascondendola (con l'icona dell'occhio nell'Outliner) non si nota niente, perché è totalmente bianca: lo sfondo deriva solo dagli altri calcoli. Ma se si vogliono altri pianeti sullo sfondo, o le stelle, le si applica un materiale con un'immagine. Tutto ciò che è **così lontano dal mondo di gioco** da poter essere tranquillamente una cosa in 2D, "tanto non si nota la differenza".

Il pavimento. Fatto tutto questo lavoro ("tutto obbligatorio"), si prende un **piano** e lo si allarga in modo che faccia da base del mondo di gioco. "Mi state seguendo? Vedo gente che guarda lo schermo senza far nulla."

11.21 Materiali e texture

11.21.1 Assegnare un materiale

Selezionando il piano, in basso a destra nei *Details* c'è una casella **Materiali**: da lì si sceglie il materiale da applicare. Il docente vuole un pavimento di pietre: sceglie **Cobblestone** (ciottolato), poi prova altro.

- Si può **scorrere** l'elenco, oppure **cercare** per nome: "voglio che la mia base sia erba": si scrive *grass*, e si trova l'erba.
- Oppure si apre lo Starter Content, cartella **Materials**, si prende il materiale e lo si **trascina** sull'oggetto: lo applica automaticamente (per esempio i mattoni).
- Selezionando più oggetti insieme, si può cambiare il materiale a tutti in contemporanea.

11.21.2 Il tiling e lo stretch

"Tenete presente che questo è tarato per essere un metro per un metro." È il discorso visto con le spline (sezione 11.7.5): allargando o riducendo un oggetto, **le dimensioni della texture rispetto all'oggetto non cambiano**, perché le dimensioni locali sono sempre le stesse. Quindi, se lo si allunga, **la texture si stira**.

Il pavimento a piastrelle. Per fare un pavimento senza che sia "tutto stretchato e brutto da vedere", per ora:

1. scegliere una **scala di base** per il piano: 1, oppure 2 se va bene che sia leggermente stirata;
2. impostare la **griglia a 100** (un metro per un metro, sezione 11.14);
3. fare un po' di **copia e incolla**, e affiancare i pezzi.

È un metodo da principianti, ma è voluto: "per il momento in cui siamo noi, lavoriamo in questo modo, invece di guardare quindici strumenti diversi in una volta sola".

Le giunture. Queste texture sono pensate per essere **ripetibili**, ma non tutte alla perfezione: tra un pezzo e l'altro si possono vedere i segni. Se dà fastidio, si cerca un materiale con un **tiling** migliore. Ma dalla distanza "non si nota tanto", e in questa fase va bene: "siamo agli inizi, non preoccupiamoci di UV e di tutte le altre cose".[16]

11.21.3 Texture e materiale non sono la stessa cosa

"Una nota importante: **materiali e texture sono due cose diverse**."

Texture è **un componente** di un materiale: il **file 2D**, l'immagine (per esempio un PNG).

Materiale è **un insieme di texture più un sacco di altri calcoli**, inclusi quelli che fanno sì che la texture **finga di essere in rilievo**.

Sui mattoni si vede che i mattoni "sporgono"; in realtà non sporgono affatto, la superficie è completamente piatta: è Unreal che, calcolandoli, finge che sporgano nel calcolare l'immagine da mostrare a schermo.[17] I materiali si trovano nella casella vista prima; e nelle slide c'è la spiegazione, un passaggio alla volta, di tutti gli elementi visti (*Exponential Height Fog* compresa).

11.22 Le modalità di visualizzazione

"Quanto tempo abbiamo? Ok, ce l'abbiamo." In alto nella viewport c'è una casella con scritto **Illuminato**, in inglese **Lit** ("io odio come è tradotto in italiano: è molto più comodo fare ricerche se l'avete in inglese, ma scegliete voi"). È la **modalità di visualizzazione**, o *View Mode*. Si sceglie a seconda di ciò che serve vedere (tabella 11.5).

[16] Durante la dimostrazione due piani che dovrebbero avere lo stesso materiale risultano avere materiali diversi ("che roba strana"), e il computer si blocca per qualche secondo; il docente non approfondisce.

[17] Il docente accenna, più avanti, alle *normali* ("quella roba di profondità che vi ho fatto vedere prima, che sono le normali"): l'effetto di rilievo si ottiene con una *normal map*, una texture che indica all'engine come la luce deve comportarsi su ogni punto della superficie. Se ne parlerà in un'altra lezione.

Modalità	Che cosa mostra, e quando usarla
Illuminato (*Lit*)	**tutti i calcoli** che danno l'immagine finale: ombre, luci, post-processing. È la modalità "totale".
Wireframe	i **poligoni** di cui è composto ogni oggetto: si vedono i triangoli (alcuni poligoni ne raggruppano più d'uno). Utile per capire com'è fatta una mesh.
Non illuminato (*Unlit*)	praticamente **solo le texture**, "come facevano i vecchi videogiochi, tipo *Quake*, *Doom*": nessun calcolo di illuminazione, nemmeno l'effetto di rilievo delle normali. Utile per lavorare quando si piazzano pareti e oggetti, o se Unreal è pesante sul proprio computer.

Table 11.5: Le tre modalità di visualizzazione su cui fermarsi per ora.

Wireframe. Su una scena con pochi oggetti, il wireframe non mostra molto; trascinando qualcosa di più complesso dallo Starter Content (una sedia tra i *Props*) si vedono i **triangoli**, come la testa della slide sulle mesh (sezione 11.17).

Lavorare in non illuminato. Se si stanno solo mettendo pareti e oggetti in una stanza, **non serve che il computer calcoli sempre tutte le ombre**: si può lavorare tranquillamente in *Unlit* e passare a *Lit* solo quando si lavora sulle luci o si vuole vedere il risultato. Se Unreal fa fatica sul proprio computer, è un modo per semplificarsi la vita. Ci sono poi anche le **impostazioni di scalabilità**, che si vedranno in un altro momento.

Molte altre. Ci sono tantissime altre modalità di visualizzazione, che servono per esempio a vedere se c'è qualche problema; per ora si resta su illuminato, non illuminato e wireframe, "se volete dare un'occhiata e vedere cosa succede dietro le quinte".

11.23 Il post process volume

"Un'ultima cosa che voglio dire, e questa è per voi, se volete divertirvi", "se volete proprio esagerare". Tra gli elementi da aggiungere al livello c'è il **Post Process Volume** (in italiano *volume di post processing*): un cubo che permette di aggiungere tutte le opzioni di **post-processing**.

Che cos'è il post-processing. Sono tutte le impostazioni dei **calcoli che vengono fatti sull'immagine una volta che è stata renderizzata**. "Detto così è molto tecnico." In pratica:

- il **motion blur**;
- il **bloom**: le cose particolarmente illuminate espandono la loro luce oltre i propri confini;
- la **profondità di campo** (*depth of field*): se una cosa in primo piano è a fuoco, lo sfondo non lo è;
- la **grana della pellicola** (*film grain*), la **correzione del colore**, il **lens flare**, l'**aberrazione cromatica**, e così via.

Tutto è calcolato dopo che la scena è stata renderizzata. Sono effetti abbastanza complessi da usare, perché ci sono tanti valori da valutare; non è "che clicchi e lo fai", richiede un po' di studio. Ma se si vuole "l'effetto videogioco super figo", il post process volume è lo strumento. Ci sarà una lezione dedicata; per ora il docente lo lascia esplorare da soli, "perché c'è sempre qualcuno che vuole strafare e si prende tantissimo bene".

Che cos'è un volume. Il post process volume è un **volume**, un altro elemento di Unreal: un oggetto nel gioco che **fa succedere qualcosa solo se altre cose sono al suo interno**. Il cubo delimita un'area: le impostazioni di post-processing si applicano solo se **la telecamera è dentro il volume**. Serve, per esempio, a cambiare il post-processing da una zona all'altra del livello.

Infinite Extent. L'unica cosa da ricordare (anche questa segnata sulla slide) è la casella **Infinite Extent (Unbound)**: se la si attiva, il volume si applica **a tutto il livello**, ovunque sia la telecamera. Il motivo per cui all'inizio della dimostrazione non si vedeva niente era proprio che non era attiva. Una volta attivata, alzando l'intensità del **bloom**, la luce diventa sempre più forte, soprattutto quella riflessa; scrivendo a mano un valore oltre il limite (50), "tutto si spara, flasha", perché il bloom è al massimo.

Una svista. Il docente prova anche la profondità di campo, che è più complessa da impostare perché vanno regolati tutti i parametri, e apre per sbaglio la sezione per il *mobile*: "scusate, ho detto una cavolata. La mobile non funziona in questo, perché non stiamo facendo un gioco per cellulare". Rimandato alla lezione dedicata.

11.24 Il task: una stanza, qualunque stanza

"Abbiamo coperto un sacco di roba oggi. E queste sono tutte lezioni di pratica iniziale." Quindi il task per la lezione successiva è, nella forma, lo stesso della volta precedente, ma con una differenza importante: non si ricopia più la propria stanza.

> **Creare una stanza**, "qualcosa, qualunque cosa, che sia una stanza". Possono essere anche più di una, ma **minimo una**. Usare **gli asset e i primitivi** a disposizione: le forme di base usate finora, i **materiali** visti oggi e le varie cose disponibili nello **Starter Content** (prop e così via). Consegna: **almeno tre screenshot**, nella cartella dei task, per la settimana successiva.

Un task creativo. "Questa è una task creativa, quindi divertitevi: potete scegliere voi che cosa fate." L'unico vincolo è che siano **stanze**: quest'anno si è deciso di concentrarsi sugli **interni**, per evitare troppa dispersione. Dentro le stanze, però, "potete farci quel che volete, non vi do nessuna limitazione". Lo Starter Content non ha molti prop: è più utile per i **materiali** (erba, legno, pietra, mattoni, "tutte quelle robe che possono coprire le vostre cose di base").

Tre screenshot: un minimo. Uno studente chiede se le tre foto siano un obbligo o un minimo. "Per me è un minimo": con una sola foto si vede solo una parete. Si può mandarne di più ("diciamo minimo tre"). Altri docenti sono più rigidi, ma lui non vuole limitare chi ha idee particolari, chi vuole mostrare angoli specifici della stanza o preferisce mostrare di più. Un'altra domanda riguarda se ogni oggetto debba avere necessariamente un materiale: si può anche lasciare un oggetto senza, se ha senso.

Guardate gli altri videogiochi. "Una nota importante. Io sono uno che ci tiene a dire: guardate gli altri videogiochi come **punto di riferimento**, cercate di capire che cosa fanno gli altri." Nel mondo dei videogiochi professionali non si guarda solo il proprio lavoro: ci si guarda attorno.

Ma siete all'inizio. Allo stesso tempo, "è chiaro che, se vi do questo task, la settimana prossima non mi aspetto che mi portiate una camera da letto al livello di *GTA 6*", anche perché si hanno a disposizione solo lo Starter Content e le forme di base. Non si può fare roba iperrealistica. "Metteteci un pochino di impegno, fate un po' quello che volete, ma non impazzite a fare cambiamenti super minuscoli": si è studenti appena all'inizio. Il principio:

> Usate gli strumenti che avete **nello scopo degli strumenti che avete**. Poi, piano piano, man mano che avremo sempre più strumenti a disposizione, porteremo sempre più in alto il livello del nostro lavoro.

Tutti questi primi esercizi servono a **prendere la mano con Unreal**: "mi interessa molto di più che vedere una roba artisticamente perfetta; voglio vedere che ci mettete l'impegno".

Approfondimento: dalla stanza copiata alla stanza inventata. Il passaggio dal primo al secondo task non è casuale. Ricopiare la propria stanza (sezione 8.19) allena la mano e il senso della scala; inventare una stanza obbliga a fare le scelte che finora erano già date: che funzione ha lo spazio, chi ci vive, come lo si *legge*. È lì che entrano in gioco le osservazioni della revisione: il **pizzico di caos** che rende uno spazio vissuto (le scatole sghembe di Federica), il rischio dello **showroom** (la stanza di Alberto), il criterio del **verosimile** (il letto di Francesco). Totten insiste su un'idea simile quando parla di *storytelling con asset modulari*: anche pochi pezzi standard, scelti e disposti con intenzione, possono raccontare chi abita uno spazio.

11.25 Le domande e la chiusura

Le ultime domande. Come sempre, gli ultimi minuti sono per domande e curiosità, e per chi vuole cominciare il task in classe. Uno studente non riesce a far comparire lo Starter Content: il docente gli rispiega che la cartella va trascinata **dentro la cartella del progetto aperta in Windows**, non nel Content Browser, perché "non state importando nuovi file: mi sono stati passati dei file che sono già stati importati". Poi si chiude e si riapre Unreal, che li trova da solo (sezione 11.19). "Lo so, era molto più comodo quando lo potevi selezionare con un clic." Il docente promette di studiare meglio la questione dopo la lezione: "veramente antipatico, ma lo dovrò studiare".

Le slide. "Come continuo a ripetere, le slide sono dentro la cartella."

Le presenze. Ci sono ancora due o tre minuti, ma "possiamo anche chiudere qui". Mattia, da casa, conferma che ha avuto problemi solo con il computer. E poi l'ultima cosa, che il docente stava per dimenticare "con tutto quello che è successo": le **presenze**. Attadia, Etiopia, Ferrara, Maestri (da casa), Muratore, Ronchi, Rossaro e Vergata ci sono; Maracino è assente. "Alla settimana prossima. Buon lavoro, arrivederci."

Punti chiave del capitolo

- Il primo anno, ogni lezione di level design comincia con la **revisione dei task** in classe: per vedere il livello generale e correggere una volta sola gli errori comuni. Nella cartella dei task non si cancella: si scrive "da cancellare". Screenshot con **Win+Shift+S**.
- Dalla revisione: **un pizzico di caos** (oggetti sghembi) rende gli ambienti vissuti; una stanza vuota sembra uno showroom; i videogiochi puntano al **verosimile**, non al realistico. La scala è un moltiplicatore: scala 5 su un cubo da un metro fa cinque metri.
- Il level design ha due macro-campi: **artistico** (luci, *enrichment*, post-processing, VFX) e **di design** (*encounter design*, blockout, funzione degli spazi). Negli studi grandi ci si specializza, e nei test di assunzione spesso si sceglie tra test artistico e test di gameplay. Il primo anno è sul lato artistico.
- Il **blockout** (*grey box*, *white box*, *block mesh*: sinonimi) è lo sketch a matita del livello: forme semplici per arrivare **il prima possibile a qualcosa di giocabile**, testare idee e rispondere a domande. Segnala l'**intento** (la finestra rossa di *Call of Duty*: "da lì possono spararti").
- Prima il gameplay, la funzione e la forma degli spazi, poi l'estetizzazione: **il gameplay regna sovrano**. Ma la veste artistica è decisa prima, e si lavora in parallelo, non a cascata.
- Cinque strumenti, non esclusivi: **primitivi** (semplici ma limitati), **kit modulari** (velocissimi ma rigidi; dungeon di *Skyrim* ed *Elden Ring*, roguelike), **brush e modellazione** (qualunque forma, curva di apprendimento; Hammer di Valve, Blender per Unity), **scultura del terreno** (colline), **spline** (strade e cavi: copie di una mesh lungo una curva, per evitare lo stretch). Si sceglie lo strumento più veloce per lo scopo.
- **Asset** = file nel progetto; **actor** = istanza nel livello. Gli asset dell'engine non si modificano.

- **Coordinate globali**: dall'origine $(0, 0, 0)$ del mondo (non è lo spawn); in Unreal z è l'alto. **Coordinate locali**: dall'origine dell'oggetto o del genitore. Nell'editor: spazio del mondo, locale, genitore; la scala è sempre locale.
- Un **figlio** eredita tutte le trasformazioni del genitore (anche la scala, distanze comprese); le gerarchie si propagano verso il basso. Un **gruppo** (Ctrl+G) seleziona e muove insieme oggetti indipendenti. L'asterisco segnala modifiche non salvate.
- **Ctrl+Z annulla anche le selezioni**: cliccare è un'azione.
- **Snapping**: griglia in centimetri (10 di default; 100 per il cubo da un metro), rotazione a 10 gradi; spegnerlo per oggetti naturali. Valori a mano e operazioni nei campi. Ruotare *sull'asse z* significa ruotare *attorno* alla verticale.
- Content Browser: Ctrl+Spazio. Lingua: Impostazioni dell'editor. Il **pivot** è l'origine delle coordinate di un actor: Alt + tasto centrale lo sposta temporaneamente (anche fuori dall'oggetto). G nasconde i gizmo, F inquadra l'oggetto selezionato.
- Una **mesh** è un insieme di punti e segmenti che formano triangoli; **mesh e collisioni sono cose diverse**: la collisione non è automatica, va controllata.
- Luci: **point** (sfera, raggio di attenuazione, source radius), **spot** (cono), **rect** (superficie), **directional** (il sole: conta la rotazione, una sola per livello). Il modello della lampadina non emette luce. Alcuni campi accettano valori oltre il limite del cursore.
- Lo **Starter Content** nella 5.8 va aggiunto a mano: decomprimere, trascinare la cartella nella cartella `Content` del progetto, riaprire Unreal.
- Un livello da zero ha bisogno di sole, **luce ambientale** e **cielo** (Sky Light, Sky Atmosphere, fog, nuvole, sky sphere): in pratica si copiano i contenuti della cartella *Lighting*. Lumen simula i riflessi della luce in tempo reale.
- **Texture** = immagine 2D; **materiale** = texture + calcoli (anche il finto rilievo). Scalare un oggetto stira la texture: per ora, piastrelle con griglia a 100.
- View mode: **Lit**, **Unlit** (più leggero), **Wireframe**. Il **Post Process Volume** agisce solo dentro il volume, a meno di **Infinite Extent**.
- Task: **una stanza inventata** (almeno una), con primitivi, Starter Content e materiali; almeno tre screenshot. Guardate gli altri giochi, ma usate gli strumenti nello scopo degli strumenti che avete.

Part VI

Revisioni dei Compiti

Chapter 1

Gli errori nei task: revisioni, correzioni e consigli del docente

Questo capitolo è diverso da tutti gli altri. Non nasce da una lezione, ma da ciò che succede *dopo* le lezioni: i task consegnati dalla classe e i **commenti di revisione** che il docente, Mattia Podini, lascia su ogni documento. È un capitolo **vivo**: a ogni task revisionato se ne aggiunge una sezione, e le parti comuni (il registro degli avvertimenti e la checklist di consegna) vengono aggiornate di conseguenza.

Il motivo per cui esiste è una regola che il docente ha fissato il primo giorno (sezione 1.4): i task vengono condivisi con tutta la classe dopo la revisione, e **ciò che il docente dice a uno studente vale per tutti**. Se a qualcuno viene detto "la prossima volta non leggo il documento", da quel momento la regola vale per l'intera classe. Leggere solo i propri commenti, quindi, non basta: un errore commentato sul documento di un collega è un errore che, ripetuto, costa la **non lettura** anche a chi non l'aveva commesso. Allo stesso modo, una riconsegna viene accettata solo se integra anche ciò che nel frattempo è stato detto agli altri.

1.1 Come usare questo capitolo

1.1.1 Struttura

Il capitolo ha due livelli.

1. Le parti **comuni**, valide per tutti i task, che si aggiornano a ogni revisione:
   - il **registro degli avvertimenti** (sezione 1.2): tutte le regole il cui mancato rispetto, da un certo momento in poi, comporta la non lettura del documento o una punizione;
   - la **checklist prima della consegna** (sezione 1.3): le domande da farsi prima di consegnare qualsiasi documento, ricavate dagli errori effettivamente commessi.
2. Una sezione per **ogni task revisionato**, sempre con la stessa struttura:
   - la consegna e il quadro delle consegne ricevute;
   - gli errori raggruppati per **tipo** (forma, linguaggio, contenuto, uso del framework…), ciascuno con i commenti del docente, il perché e la correzione, con esempi;
   - ciò che il docente ha **apprezzato**, perché anche quello è un'indicazione;
   - le **schede delle singole consegne**: l'elenco completo dei commenti ricevuti da ogni documento, nessuno escluso.

Table 1.1: Indice dei task revisionati.

N.	Task	Assegnato in	Consegne	Sezione
T1	Arcade core elements	capitolo 2, sezione 2.14	8 studenti, 9 documenti	1.4
T2	Production dictionary di un Pokémon	capitolo 4, sezione 4.14	9 studenti, 9 documenti (3 non letti)	1.5
T3	TCG pain points (a gruppi)	capitolo 5, sezione 5.16	4 gruppi, 4 documenti	1.6
T4	Flowchart di Pac-Man	capitolo 6, sezione 6.14	8 consegne (1 senza nome, 1 su *Donkey Kong*)	1.7

1.1.2 Legenda

Nelle schede delle singole consegne ogni commento è riportato con il punto del documento a cui il docente lo ha **ancorato** (il testo evidenziato), in corsivo tra virgolette, seguito dal commento. Quando un commento ha ricevuto risposte o aggiunte successive, queste seguono dopo "*E ancora:*". Quando nel thread interviene uno studente, il suo intervento è introdotto da "*Commento di Nome:*" e la replica del docente da "*Il docente:*". Quando il commento è ancorato a una zona di un'immagine (dal task T4, i flowchart), il blocco a cui si riferisce è ricostruito dal curatore e indicato con "[*presumibilmente* "*blocco*"]:". Due segni aiutano a orientarsi:

[!] il commento contiene un **avvertimento** ("la prossima volta non leggo", "errore grave", "non lo accetterò in futuro") o una punizione: è una regola che da ora vale per tutti;

[+] il commento è **positivo**: indica qualcosa da ripetere.

I commenti sono riportati **alla lettera**, con i loro refusi: sono appunti di revisione, non testo curato. Le parti in cui questo capitolo aggiunge una proposta propria (ad esempio le riscritture "prima e dopo") sono sempre segnalate come tali, per non confonderle con la voce del docente.

1.1.3 I tre livelli di gravità

Dai commenti emerge una scala abbastanza chiara, che conviene avere in mente leggendo il resto.

1. **Errore da correggere**: il docente lo segnala ("testo inutile", "quanti e quali?", "spazio extra") e si aspetta che non si ripeta. È la grande maggioranza dei commenti.
2. **Errore grave o ultimo avviso**: "prima e ultima volta che accetto una consegna simile", "la prossima volta non leggo il task". Il documento è stato letto, ma la stessa cosa, al prossimo task, costa la non lettura.
3. **Non lettura o punizione immediata**: il docente smette di leggere. Nel primo task è successo una volta sola, per l'uso della **parola proibita** (sezione 1.4.4); nel secondo a **tre documenti su nove**, tutti fermati al titolo per ragioni formali (sezione 1.5.3). Nel quarto un task consegnato **incompleto** non ha ricevuto commenti di merito (sezione 1.7.6).

1.2 Il registro degli avvertimenti

Le tabelle 1.2 e 1.3 raccolgono, task dopo task, tutte le regole su cui il docente ha dato un **ultimo avviso**. Si ricordi che valgono per **tutta la classe**, non solo per chi ha ricevuto il commento. Molte erano già state enunciate a lezione (le golden rules e le regole formali, sezioni 3.7 e 3.9); la revisione le ha rese, per così dire, esecutive.

Table 1.2: Registro degli avvertimenti (aggiornato al task T4). Nella colonna "Da" il task in cui la regola è stata richiamata; "T2: non letto" indica che la regola è già costata la non lettura di un documento.

Regola	**Da**	**Che cosa ha detto il docente**
Margini 1,5 / 1,5 / 1,25 / 1,25 e font **Roboto 11**	T1, T2: non letto	"La prossima volta non leggo il task" (più studenti); "prima e ultima volta che accetto una consegna simile"; nel T2: "Anche se li hai impostati correttamente, i margini non sono stati rispettati. Non leggo"
Niente **pagine o mezze pagine vuote**	T1	"Errore grave per il futuro", "errore grave da non ripetere, pena la non lettura"
Niente **wall of text** (paragrafi lunghi e fitti, documento non tecnico)	T1	"Dal prossimo task non leggerò una formattazione simile", "non lo accetterò in futuro"
Il documento è un **documento tecnico**, non una consegna accademica (niente testo con bibliografia in coda)	T1	"Dal prossimo task, una consegna simile non la leggerò e non avrà i miei feedback"
Si consegna con la **suite di Google** (Google Docs)	T1	"Lo leggo come prima e ultima eccezione, ma in futuro non lo accetterò"
Niente **colori banditi** (rosso RGB puro, giallo su bianco. . .)	T1	"La prossima volta che lo vedo non leggo il documento"
Il documento deve essere **stampabile**	T1	"Con una consegna simile in futuro rischi la non lettura"
Elenchi rientrati più volte che **sacrificano spazio** a sinistra	T1	"Ripetere questa cosa in futuro ti costa la non lettura"
Mai far leggere **due volte la stessa cosa** (stessa tab copiata in due documenti)	T1	"Errore grave"
Niente parole **al limite della parola proibita** ("accattivante")	T1	"Prossima volta rischi la non lettura"
Parola proibita ("intrattenimento", "divertente" e simili)	T1	Lettura interrotta subito e **task punitivo**: analisi dei core elements di Schell su *Hatoful Boyfriend*
Errori grammaticali (maiuscole incoerenti, spazi prima delle parentesi, accenti)	T1, T2	Il docente ricorda "la regola dei 5 errori grammaticali"; nel T2: "Sei al limite del non letto per numero di errori grammaticali accumulati", "Sei al limite del non letto"

Table 1.3: Registro degli avvertimenti (continua): le regole aggiunte dai task T2, T3 e T4.

Regola	**Da**	**Che cosa ha detto il docente**
Testo inutile in quantità (il lemma o il titolo ripetuti)	T2: non letto	"Non leggo data la quantità di testo inutile presente"
Nome del file **Cognome_Nome_TitoloTask**	T2: non letto	"Il documento non è nominato con Cognome_Nome_TitoloTask come da regole [...]. Non leggo"
Limiti della consegna (es. una pagina)	T2: non letto	"Il limite della consegna era di una pagina, ne hai prodotte quasi due. Non leggo"
Titolo (o immagine di titolo) che occupa troppo spazio	T2	"Ti 'mangia' almeno 1/4 del documento. Prima e ultima volta che leggo una consegna con un problema simile"
Elenco puntato che sacrifica spazio a destra	T2	"Nel miglior caso possibile, in una zona grigia delle regole. Lo accetto ora [...], non lo accetterò per task futuri"
Niente **reference o bibliografia in coda**: i link vanno dove servono	T1, T2	"Eliminabile"; nel T2: "qui diventa estremamente necessario, sono da eliminare"; nel T3 (in una scheda separata): "non risolve completamente il problema ma è la direzione corretta"
Formattazione curata anche (e soprattutto) nei **lavori di gruppo**	T3	"Avete svolto il task in team, e siete stati il team più numeroso, il risultato [...] è al limite dell'accettabile"
Completare il task: niente consegne a metà	T4: senza commenti	"Mi riservo di non dare commenti ad un task mancante di parti fondamentali"; "non aver terminato il task è un grosso problema"
Il **testo resta dentro il blocco** (flowchart, tabelle, immagini)	T4	"La prossima volta che vedo una svista simile non leggo"
Il **nome del protagonista** scritto come l'originale (*Pac-Man*, non "Pacman")	T4	"Assumo che hai utilizzato una licenza artistica [...]. Prima e ultima volta"
Anche i **documenti di supporto** seguono le regole formali (niente note in .txt)	T4	"Perché questo documento integrativo non si presenta secondo le regole stabilite?"; "anche questo deve adottare i margini e le regole viste a lezione"

La regola dei cinque errori grammaticali. Nel primo task il docente la richiama di passaggio ("ricordo la regola dei 5 errori grammaticali"), come regola già nota: a lezione era stato detto che non devono esserci errori grammaticali gravi e che la coerenza della lingua fa parte della sequenza corretta di un documento (sezione 3.7). La soglia esatta e la sua conseguenza non sono esplicitate nei commenti; la lettura più prudente è considerarla un limite oltre il quale il documento non viene letto.

1.3 La checklist prima della consegna

Questa checklist traduce in domande gli errori commessi davvero. Va ripercorsa **prima di ogni consegna**, su qualsiasi task. Accanto a ogni gruppo è indicata la sezione in cui gli errori corrispondenti sono spiegati nel dettaglio.

1.3.1 Forma e regole formali

- Il file è nominato **Cognome_Nome_TitoloTask**? Ho rispettato i **limiti della consegna** (numero di pagine)? (1.5.3)
- È un **Google Doc**, con margini 1,5 sopra e sotto e 1,25 a sinistra e a destra, e tutto il testo in **Roboto 11**? (1.4.3)
- C'è qualche **pagina o mezza pagina vuota**? Qualche spazio sprecato in fondo a una sezione o sotto un titolo?
- Ho usato **righe vuote** (Invio ripetuti) per distanziare i paragrafi, invece dell'interlinea e dello spazio dopo il paragrafo?
- Il testo è **giustificato**? (Sconsigliato "caldamente".)
- C'è qualche paragrafo che, visto al 50% di zoom, sembra un **muro di testo**?
- Qualche **tabella** o elenco è spezzato tra due pagine? Qualche immagine sfora i margini o è finita nella pagina dopo il testo che la richiama?
- Ci sono colori banditi, testo giallo su bianco, sfondi scuri? Il documento si può **stampare**?
- Il **titolo** dice che tipo di documento è (analisi, production dictionary…), su che gioco e, se serve, con quale framework? È senza ripetizioni, decorazioni e immagini che mangiano spazio? Le tab sono rinominate e ognuna contiene una sola cosa? (1.5.4)
- Elenchi puntati, sottoelenchi e colonne lasciano **spazio bianco a destra**? Ne vale il prezzo?
- Le **tabelle** arrivano ai margini, con lo spazio ripartito bene tra le colonne e un distacco dal testo prima e dopo? Ogni **immagine** porta un'informazione o è una decorazione? Il testo scorre attorno alle immagini invece di lasciare righe vuote? (1.6.3)
- C'è una sezione di **reference in coda**? (Va tolta: i link si mettono dove servono.)
- Grassetto, corsivo, sottolineato e colori seguono una **logica coerente** e dichiarabile?
- Maiuscole, punti, spazi dopo la punteggiatura e prima delle parentesi, parentesi chiuse: tutto in ordine?

1.3.2 Linguaggio

- Ogni frase **serve**? Ho ripetuto nel testo il nome del gioco, del framework o della sezione che è già nel titolo? (1.4.4)
- Ci sono **opinioni personali** ("sicuramente", "per me", "il migliore") al posto di dati?
- Ci sono **parole deboli** ("potrebbe", "forse", "una sorta di")?

- Ci sono aggettivi **non misurabili** (“lungo”, “breve”, “alto volume”, “semplice”, “molte piattaforme”) senza un metro di paragone o un numero?
- Ho scritto da **recensore** (battute, intercalari, domande retoriche, “ci viene mostrato”) invece che da designer?
- C’è la parola proibita, o qualcosa di simile?
- In un elenco di voci, ogni definizione **ripete il lemma** (“Pokédex: il Pokédex è…”)? (1.5.5)
- Ho scritto “pokémon” (o qualunque nome proprio) **sempre allo stesso modo**?

1.3.3 Contenuto e framework

- Ho **dichiarato** il framework con il suo nome preciso (“core elements di Romero e Schreiber”, “tetrade elementale di Schell”)? (1.4.6)
- Ogni informazione è nella **voce giusta** del framework? Ho nominato in una sezione qualcosa (una modalità, un sistema) mai spiegato nelle meccaniche?
- Per ogni meccanica: che cosa succede **premendo l’input**? E se lo si preme di nuovo? Con quali **numeri**? Che cosa succede al **game over** e al cambio di livello? (1.4.7)
- Per ogni oggetto di gioco: **quanti e quali**, quando compaiono, che cosa fanno, che cosa succede se il personaggio ci entra in contatto?
- Ho **mostrato** (immagini, UI, sprite, tabelle) ciò che si può mostrare? (1.4.9)
- Le affermazioni su pubblico, temi, prestazioni sono sostenute da **dati e reference**?
- Ho risposto a **tutto** ciò che chiedeva la consegna?
- In una definizione, ogni **termine unico** che uso ha una sua voce? Ho detto prima **che cos’è** (statistica, edificio, key item, consumabile…), poi chi, quanti, dove, a che costo, con che sistema? (1.5.6)
- Ci sono generalizzazioni false (“tutti”, “solo”, “sempre”)? Due voci dicono cose diverse sulla stessa cosa?
- Se c’è un limite di spazio: **ogni riga lavora**? Avanza spazio mentre alcune voci restano stringate? Posso accorpare voci simili? (1.5.7)

1.3.4 Dati, analisi e design

- Ogni problema è sostenuto da **dati** e dalla **voce degli utenti**, mostrata e linkata nel punto in cui serve? I criteri con cui ho scelto i problemi sono misurabili? (1.6.4)
- Il problema è **specifico**? Ho trovato il problema più grave e la sua **causa**, non solo il sintomo? I competitor sono nominati e confrontati per intero? (1.6.5)
- Ogni soluzione ha un **esempio concreto**, dei **numeri** (anche tentativi) e, se tocca l’interfaccia, un **mockup** o wireframe? (1.6.6)
- Ho provato a rompere la soluzione con i **casi limite**? Ne ho dichiarato costi, conseguenze, pro e contro rispetto alle alternative? È ancora coerente con l’identità del gioco?
- Nelle proposte c’è **linguaggio debole** (“si potrebbe”, “may”)?

1.3.5 Flowchart

- Da ogni **rettangolo** esce **una sola freccia**? Se le strade sono più d'una, ho usato un rombo? Se le azioni sono più d'una, le ho messe in sequenza? (1.7.3)
- Ogni uscita di ogni **rombo** è etichettata, con un esito diverso dalle altre (niente due "sì")? Il "sì" e il "no" sono dalla parte giusta?
- Ogni **sotto-processo** ha il suo flowchart? L'ordine dei sotto-processi segue l'ordine causale (prima ci si muove, poi si mangia)?
- Qualche freccia si **sovrappone** o si incrocia in modo fraintendibile? Qualche blocco è **scollegato** ("nothing happens")? Ho seguito a mano ogni loop?
- Se un programmatore implementasse il flowchart così com'è, il gioco funzionerebbe? (1.7.4)
- Vite rimaste, vite guadagnate, **game over**, **high score**, passaggio di livello: ci sono tutti?
- Ho dato i **numeri** (durate, punteggi, liste di oggetti) o rimandato a una tabella che li contiene?
- Il testo è di **una sola misura** e sta **dentro** i blocchi? Il nome del gioco e del protagonista è scritto giusto? (1.7.5)
- L'immagine si legge senza zoomare (esportazione al 300–500%)? I documenti di supporto rispettano le regole formali?
- Ho deciso **prima** la profondità che posso permettermi con il tempo che ho? Il task è **completo**, e coerente nella profondità delle sue parti? (1.7.6)

1.4 Task 1 — Arcade core elements

1.4.1 La consegna

Il task era stato assegnato alla fine della lezione su che cos'è un gioco (sezione 2.14) e confermato in quella successiva:

1. scegliere un **gioco arcade**;
2. scegliere **uno** dei framework visti a lezione (Schell, Romero e Schreiber, oppure Bura) e analizzare il gioco con quello;
3. far emergere le **tre emozioni** principali verso cui il gioco è orientato, con il vocabolario visto in aula;
4. aggiungere una **propria definizione di gioco arcade**;

con il vincolo dei termini banditi e, nella lezione sulla documentazione, le regole formali di consegna (sezione 3.9). Era il **primo task** di documentazione del corso, e il docente lo ricorda più volte nei commenti: diverse cose vengono "accettate per ora", ma con l'avvertimento che dal task successivo non lo saranno più.

1.4.2 Le consegne in sintesi

Il quadro che ne esce è netto. Quasi tutti gli errori ricadono in quattro famiglie, nell'ordine di frequenza:

1. **testo inutile** e scrittura poco puntuale (opinioni, parole deboli, aggettivi non misurati);
2. **impaginazione**: spazi vuoti, righe vuote al posto dell'interlinea, margini e font, muri di testo;
3. **meccaniche descritte ad alto livello**, quando l'analisi richiede un dettaglio "a basso livello";
4. **uso del framework**: informazioni nella voce sbagliata, voci mancanti, framework non dichiarato.

Le prossime sezioni li affrontano uno per uno.

Table 1.4: Le consegne del task T1.

Studente	**Gioco**	**Framework**	**Il nodo principale**
Francesca Attadia	*Bubble Bobble* (1986)	Schell	Buon primo tentativo; meccaniche poco dettagliate
Antonino Etiopia	*Top Gear*	Romero e Schreiber (core elements, non nominati)	Elenchi rientrati che sprecano spazio; dati sui giocatori assenti
Federica Ferrara	*Geometry Dash* (2013)	Schell	Margini errati; domanda sull'arcade lasciata senza risposta; tono da recensore
Mattia Maestri	*Donkey Kong* (1981)	non dichiarato	Margini, font e rosso RGB; framework incompleto; manca la definizione di arcade
Francesco Muratore	*Pac-Man*	Schell *e* Romero e Schreiber nella stessa tab	Margini; wall of text; linguaggio debole; tecnologia fraintesa
Alberto Maria Ronchi	*Tetris*	Schell (non nominato)	Margini; meccanica confusa con strategia; definizione che si contraddice
Valerio Rossaro	*Dino Rex* (1992) e *Punch-Out!!* (1984), due documenti	non dichiarato	Wall of text; stessa tab ripetuta in due documenti; **parola proibita**
Lorenzo Vergata	*Jet Set Radio*, *Crazy Taxi 2*, *The Legend of Valkyrie*	Schell, Romero e Schreiber, Bura	Documento accademico e non tecnico, in inglese, fuori dalla suite Google; definizione troppo larga

1.4.3 Errori di forma e di impaginazione

Margini e font. È l'errore più "economico" da evitare ed è anche quello su cui il docente è più netto. Quattro documenti su otto non avevano i margini stabiliti (1,5 sopra e sotto, 1,25 ai lati) e uno non era in Roboto:

> "Il documento non ha i margini corretti e il font non è Roboto. Prima e ultima volta che accetto una consegna simile." (Mattia)
> "I margini del documento non sono quelli stabiliti, la prossima volta non leggo il task." (Francesco, Alberto)
> "Hai sfruttato correttamente le tab del documento, ma i margini non sono quelli stabiliti. La prossima volta non leggerò il documento se ripeterai l'errore." (Federica)

Correzione: impostare margini e Roboto 11 come predefiniti **prima** di scrivere (Google Docs lo permette: File, Impostazione pagina, "Imposta come predefinito"), e ricontrollarli prima di consegnare.

Lo spazio sprecato. Il commento più ricorrente sulla forma, in tutte le sue varianti: "spazio extra", "doppio spazio", "mezza pagina bianca qui sotto, è spazio sprecato", "praticamente una pagina di spazio sprecato", "qui sotto c'è una pagina intera di spazio sprecato, per il futuro è considerato un errore grave". Lo spazio, a lezione, era stato definito "oro", "come l'acqua nel deserto" (sezione 3.7). I casi visti nel primo task:

- **mezza pagina o pagina vuota in coda** al documento o a una tab (Antonino, Federica, Mattia, Alberto, Lorenzo): errore grave;
- **una pagina usata per una battuta**: Lorenzo apre la seconda parte con un finto "esempio perfetto" (*Michael Jackson's Moonwalker*) seguito da "no, ok, just kidding": "hai buttato una pagina per un joke: non è un pitch";
- **una pagina usata per un passaggio di sezione** ("let's continue with the next game"): "per fare questo perdi una pagina di spazio completamente sprecato";
- **titoli sviluppati in verticale**, che lasciano vuoti i lati (Mattia: "il titolo è esteso verticalmente, troppo, considerando lo spazio che perdi sui lati");
- **elenchi rientrati tre volte**, che mangiano tutto il margine sinistro (Antonino: "questa cosa può funzionare per alcuni paragrafi, ma su tutto il documento no. Hai sacrificato molto spazio a sinistra, ripetere questa cosa in futuro ti costa la non lettura");
- **spazio a destra di un elenco** lasciato vuoto quando poteva ospitare un'immagine (Mattia: "hai ampio spazio qui sulla destra dell'elenco puntato").

Le righe vuote al posto dell'interlinea. Un commento ripetuto identico su cinque documenti: "Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato. Vale per ogni uso di spazio simile, per rompere il testo, nell'intero documento." Premere Invio due volte per staccare i paragrafi produce spazi irregolari, che si spostano a ogni modifica e finiscono in cima alle pagine. Lo strumento giusto è **Formato, Interlinea e spaziatura paragrafo**, con uno "spazio dopo" personalizzato, applicato in modo uniforme (meglio ancora se salvato negli stili dei titoli e del testo normale).

Il testo giustificato. "Sconsiglio caldamente l'uso del testo giustificato, in generale" (Federica, Valerio, Lorenzo). In un documento tecnico, in colonne strette e con molti termini lunghi o inglesi, la giustificazione crea spazi irregolari tra le parole e rende la lettura più faticosa. Si allinea a sinistra.

Il wall of text. È la forma che il docente ha bocciato con più decisione, e che dal secondo task non verrà più letta:

> "Questa pagina è tutto wall of text, dal prossimo task non leggerò una formattazione simile." (Francesco)
> "Tutto il documento è un pericoloso wall of text con leggibilità ridotta, riflettici attentamente perché in futuro non accetterò una formattazione simile." (Valerio, *Dino Rex*)
> "Questa formattazione è un grave wall of text. Non lo accetterò in futuro." (Valerio, *Punch-Out!!*)

Il caso di *Punch-Out!!* mostra bene il meccanismo: in un solo blocco finiscono la presentazione del gioco, l'obiettivo, lo scopo della singola partita e il pubblico di riferimento. "Questo è un altro punto, argomento, paragrafo. Unire tutto insieme riduce drasticamente la leggibilità." *Correzione*: un paragrafo, un argomento; elenchi puntati quando le voci sono parallele ("tutto questo può tranquillamente essere messo a elenco puntato migliorandone la leggibilità"); titoli o grassetti che permettano a chi legge di saltare direttamente al punto che gli interessa.

Documento tecnico, non accademico. Il documento di Lorenzo (*The Ingredients of Tasteful Arcade Games*) è il caso limite: una ricerca ampia e corretta, con fonti accademiche, ma impaginata come un saggio, in inglese, con la bibliografia in coda e, per di più, non in Google Docs.

> "Questo non è un documento tecnico: è un wall of text con reference in coda, più simile ad una consegna accademica, e non è l'obiettivo di un Game Designer. Dal prossimo task, una consegna simile non la leggerò e non avrà i miei feedback."

Il punto non è la quantità di lavoro (il docente si dice "personalmente contento di vedere lavoro extra"), ma il rapporto tra forma e fruibilità: "quello che guadagni in contenuto lo perdi in forma [...]: il rischio (e il risultato qui) è un documento non tecnico dalla ridotta leggibilità, che un domani nessuno leggerà veramente. O peggio, lo leggeranno dimenticandosi cosa implementare o cosa era importante." Le reference, poi, vanno **linkate nel punto in cui servono**: una bibliografia finale è "eliminabile, me li hai linkati dove era necessario".

Prima l'MVP, poi l'extra. Dallo stesso documento viene una regola generale: "prima di espandere la consegna la *rule of thumb* è iterare sull'MVP". Analizzare tre giochi con tre framework, quando la consegna ne chiede uno, ha senso solo se l'analisi richiesta è già impeccabile; altrimenti moltiplica i difetti.

Tabelle spezzate e scorrimento. "Mai, e dico mai, rompere una tabella in più pagine." Allo stesso modo, "stai rompendo il discorso in due pagine, cosa da evitare assolutamente". Sulla matrice di Bura di Lorenzo il docente aggiunge un'osservazione di metodo che vale per ogni documento:

> "Il lettore è qui, sta leggendo la riga e vuole vedere dove era nella matrice di Bura. Per come hai formattato, deve continuamente fare su e giù, potenzialmente dimenticandosi pezzi nel mentre. A questo non sempre c'è una soluzione ideale, o non è sempre necessario che ci sia, però sono dettagli sui quali devi pensare adesso. Perché dedicare tempo e attenzione ora ti renderà un GD poi."

Immagini: dove e come. Le immagini sono incoraggiate (si veda la sezione 1.4.9), ma devono stare **dentro i margini** ("l'immagine sotto non rispetta i margini, è un dettaglio assolutamente evitabile") e **accanto al testo che le richiama**: a Federica "le immagini sono cadute nella pagina seguente, intuizione corretta ma applicazione da rivedere". Un'immagine usata come titolo grafico non è sbagliata, ma va ragionata rispetto allo spazio e al titolo testuale già presente (Antonino).

Colori. Mattia ha scritto il titolo in **rosso RGB puro**: "citato come colore bandito nelle slide sulla documentazione. La prossima volta che lo vedo non leggo il documento". Francesco ha usato il **giallo su bianco**: "poco leggibile". Valerio ha consegnato un documento **non stampabile**: "con una consegna simile in futuro rischi la non lettura". Anche i link rossi di Lorenzo sono "non comuni, ma borderline accettabili": il link deve distinguersi, ma senza uscire dalla palette.

Formattazione senza logica. Ad Antonino: "inizio ad avere difficoltà a capire come hai scelto grassetto, italico e sottolineatura: perché mi sembra che il risultato sia il medesimo, far risaltare elementi importanti." Se tre strumenti fanno la stessa cosa, il lettore non impara a leggerli. Ogni stile deve avere **un significato**, sempre lo stesso.

Il titolo e l'apertura. Il titolo deve dire, in un colpo d'occhio, **che documento è**, **su che cosa** e **con quale framework**:

- "Che tipo di documento è? Design? Analisi? Secondo che framework? Mancano informazioni cruciali" (Mattia; identico a Valerio);
- "Il titolo appare più come una recensione piuttosto che un documento tecnico di analisi, inoltre è immotivatamente schiacciato a destra" (Lorenzo);
- positivo: "pur non osando con immagini, il titolo rimane essenziale e presenta tutti gli elementi necessari. Funziona" (Francesca); "titolo funzionale" (Federica, *Geometry Dash — Core Elements — Jesse Schell*).

Anche le **intestazioni interne** devono pesare per quello che sono: "Brenda Romero & Ian Schreiber's" scritto a 11 punti "apre il doc, non può essere a 11 di grandezza, deve essere rilevante per il lettore"; e il passaggio dalla definizione di arcade al framework "deve chiaramente presentarsi come divisore".

Le tab. Le tab di Google Docs servono a **separare parti diverse** dello stesso documento: "l'utilità di avere più tab è proprio quella di dividere parti differenti dello stesso doc, qui hai Schell e di là Romero+Schreiber" (Francesco, che li aveva messi insieme). Ogni tab ha un titolo ("manca il titolo della tab", Valerio) e va pensato **l'ordine**: la definizione di arcade di Alberto era nell'ultima tab, "questa tab avrei dovuto leggerla prima, non dopo". E soprattutto: "mai fare leggere al lettore due volte la stessa cosa" (Valerio aveva copiato la stessa tab sulla definizione di arcade in entrambi i documenti).

Punteggiatura e refusi. Maiuscole mancate ("Top gear", "n" a inizio frase, "Donkey kong"), punto mancante, spazio mancante prima di una parentesi o dopo un punto, parentesi chiusa ma mai aperta, "Un'altro". A Francesco, che apre e chiude le citazioni digitando due apostrofi di seguito, il docente ricorda: "usa la punteggiatura specifica quando possibile". Ciascuno di questi, preso da solo, è minimo; ma rientrano tutti nella regola dei cinque errori grammaticali.

1.4.4 Errori di scrittura e di linguaggio

"Testo inutile". È, di gran lunga, il commento più frequente del primo task: compare su **tutti** i documenti, decine di volte ("ma questo errore è presente più volte di quante io ne possa realisticamente commentare"). Le forme ricorrenti:

- **ripetere il titolo**: "Analisi del gioco Top Gear" sotto il titolo, "Pac-Man" come soggetto di ogni sezione, "Donkey Kong è un gioco arcade [...] nato nel 1981" quando titolo e anno sono già in testa ("lo hai scritto già sopra"); "testo inutile, il gioco è rimasto lo stesso";
- **annunciare invece di dire**: "nel gioco sono presenti diverse meccaniche:", "Ci sono diverse regole imposte dal sistema di gioco" ("testo inutile, cita le regole invece di scrivere questo"), "Top Gear presenta una serie di regole utili", "Il sistema di gioco mette a disposizione del giocatore una serie di comandi";
- **ripetere l'etichetta della sezione**: "La narrativa racconta di", "Dal punto di vista narrativo", "Il target di riferimento era", "L'obiettivo in Donkey Kong è", "Sta giocando al gioco un", "La tecnologia scelta";
- **rimandi a sé stessi**: "(come sopra citato)": "allora non dirlo"; "As said at the beginning of this work": "allora è inutile se lo hai già detto";

- **premesse e congedi**: “Ok, then. Excuse me for this long introduction” (“non è scusabile”), “That’s it for now. Bye! :)” (“ahimè, testo inutile”);
- **l’assunto**: frasi che ripetono ciò che la consegna già dà per scontato (“Using the vocabulary seen in class, Jet Set Radio, for me, is oriented towards four emotions:”: “questo è l’assunto, tutto evitabile”).

Il test: se cancello la frase, il lettore perde un’informazione? Se no, la frase va cancellata. Un esempio, dalla consegna di Mattia:

Nella consegna	**Riscrittura proposta**
Meccaniche. Nel gioco sono presenti diverse meccaniche: *Punteggio.* Il punteggio in Donkey Kong viene assegnato al giocatore in base alle azioni intraprese da quest’ultimo durante la partita.	*Punteggio.* Contatore sempre visibile nella UI [immagine con il contatore evidenziato], che aumenta in base alle azioni compiute nel quadro (tabella sotto).

La riscrittura è costruita sulla definizione che il docente stesso dà nel commento: “sistema che incrementa sulla base di azioni compiute nel quadro, aggiornando un contatore sempre presente nella UI”.

Opinioni personali. “Al limite dell’opinione personale” è il secondo commento più frequente. Esempi: “sicuramente” (“Ciò che attrae di più di Pac-Man sono sicuramente”), “una difficoltà moderata”, “in Top Gear ci sono poche risorse”, “molto semplice”, “the perfect example”, “first position in my heart”, “I consider myself a good Mario Kart gamer” (“opinione personale. Non può stare qui”), “assordante” (“opinione personale, non hai portato né dati né reference”). Un’opinione può diventare un’affermazione accettabile solo se è sostenuta da dati o reference; altrimenti si toglie.

Parole deboli. “Niente linguaggio debole. Stai scrivendo un documento di analisi, sii certo di quello che inserisci e di come lo dici” (Francesco, su “potrebbe avere”). Stessa sorte per “potrebbero essere”, “una persona potrebbe tranquillamente essere attratta” (“linguaggio debole, tutto evitabile”) e per “I would” (Lorenzo). È la regola delle golden rules: se il dato è quello, si scrive il dato (sezione 3.7).

Aggettivi senza metro. Il docente li segnala uno per uno, quasi sempre con la stessa domanda: *rispetto a che cosa?*

- “breve durata delle partite”: “relativo. Serve un metro di paragone”;
- “alto volume”: “da capire cosa significhi ‘alto’ ”;
- “lunghe corse”: “serve un metro di misura, al limite dell’opinione personale”;
- “semplicità del gameplay e immediatezza del gioco”: “da definire ‘semplice’ e ‘immediato’ ”;
- “prettamente semplici da memorizzare”: “senza un metro di misura questa è una opinione personale”;
- “vecchio gioco arcade”: “cosa significa questo?”;
- “per molto tempo”: “generico”;
- “molte altre piattaforme”, “numerosissime piattaforme”: “non ‘molte altre’, ‘le seguenti’, quelle che elenchi sotto. Sii puntuale”;
- “sacrificando un po’ di tempo”: “quantità specifica, anche con un margine di errore va bene”;
- “per un tot di secondi”: “ok, prossimo step: esplicitare questo parametro”;
- “rigidità”: “non significa nulla così”.

Correzione: un numero (anche approssimato, “circa 3 secondi”), un confronto esplicito (“partite di 2–5 minuti, contro le decine di minuti di una sessione console”), oppure un elenco completo al posto di “molti”.

Il tono da recensore. Un documento di analisi non è una recensione né un pezzo per un blog. Il docente lo segnala in molte forme:

- domande retoriche e "azzardi" dichiarati: "Innanzitutto, perché proprio Geometry Dash? perché questa scelta così azzardata?" (Federica): "questa è una frase da 'recensore'. Quello che scrivi nella descrizione già dovrebbe incorporare questo tuo 'azzardo', dissipando ogni potenziale dubbio. [...] Sono le spiegazioni concrete qua sotto e il loro legame che implicitamente ti definiscono come Designer";
- il "noi" e il "ci" rivolti al lettore: "Ci viene mostrato un mondo": "può esserti utile mettere della distanza tra te e il lettore: allena anche l'oggettività mettendo da parte l'ego";
- intercalari e battute: "I mean" ("nessun intercalare, non sei un recensore videoludico"), "(blasphemous)" ("inutile e fuori luogo"), "Do we have to put in our dish we're cooking drugs? [...]" ("commenti del genere non hanno spazio dentro documenti di analisi e/o design"), la playlist suggerita per la lettura ("soluzione interessante, ma discutibilmente fuori luogo per un documento di analisi");
- formule che promettono sintesi e non la mantengono: "Long story short" seguito da un paragrafo lungo: "questo è proprio falso";
- il "per me" e il "LA MIA definizione": "chiaro, il documento ha il tuo nome" (Alberto); "for me the most important of all": "tutto eliminabile, motivalo e basta" (Lorenzo).

Frasi impersonali e soggetti vaghi. "Una volta che si è riusciti a prenderle": "Chi? Il giocatore? Il main character? Perché la frase è impersonale?" In un documento tecnico il soggetto di ogni azione deve essere esplicito: **il giocatore** preme un input, **il main character** compie l'azione, **il sistema** reagisce.

Il tempo verbale. A Valerio, che descrive gli arcade solo al passato: "continui a parlare al passato, e nel presente?" Una definizione di genere deve reggere anche oggi.

Stereotipi. Sul passaggio in cui il mondo arcade viene definito "*stereotypically male-oriented*", il commento è secco: "cosa da evitare".

La parola proibita. È l'unico caso, nel primo task, di **lettura interrotta**. Nel documento su *Punch-Out!!* Valerio scrive che il pubblico di riferimento "cercava un *intrattenimento* più rapido":

> "Inaccettabile, smetto di leggere questo documento. Come punizione devi produrre una analisi dei core elements di Schell su *Hatoful Boyfriend*, per lunedì 9 novembre."

La regola era stata data alla seconda lezione (sezione 2.12): niente "divertente", "intrattenente" e varianti, perché non dicono *perché* un gioco funziona; e la punizione di Mattia Podini è proprio un'analisi su un gioco "terribile", fino a *Hatoful Boyfriend*. Attenzione anche ai **dintorni** della parola: a Francesca, per "accattivante", il docente scrive "al limite della parola proibita, prossima volta rischi la non lettura". La correzione è sempre la stessa: nominare l'**emozione** o la **qualità** precisa (sfida, tensione, *fiero*, ritmo, leggibilità...) con il vocabolario visto in aula.

Usare il vocabolario del corso. Il docente corregge anche le parole quasi giuste: "frustrazione e soddisfazione" → "*fiero* è la parola che cerchi" (Federica); "competizione amichevole" → "Cooperative, Fellowship, People Fun. Ne riparleremo" (Francesca); "stunts" → "ad alto livello, questo è un combo system" (Lorenzo). Su "dynamics", usato da Lorenzo per indicare le sezioni a ritmo serrato, il docente scrive "termine inesatto perché ha una sua controparte più largamente accettata", senza dire quale; "ci ritorneremo".[1]

[1] Nota del curatore: il termine a cui il docente allude non è esplicitato nel commento. Da tenere d'occhio nelle prossime lezioni.

1.4.5 La definizione di gioco arcade

Era una parte esplicita della consegna, e i commenti ne fanno emergere sei errori tipici.

1. **Mancare la definizione.** Mattia scrive soltanto che *Donkey Kong* "è un gioco arcade": "spiegare cosa significa era parte del task".

2. **Porre la domanda senza rispondere.** Federica chiude il paragrafo con "La domanda è: ad oggi, un gioco platform può essere considerato anche arcade?": "ok, ma mi devi rispondere tu, altrimenti il doc è amputato".

3. **Contraddirsi.** Alberto definisce gli arcade come i giochi "nati per essere giocati nei cabinati" e subito dopo aggiunge "anche se ora i cabinati sono tecnologia obsoleta": "allora stai già confutando la tua stessa tesi".

4. **Usare parole vuote.** "Gli arcade possono per esempio essere giochi con meccaniche semplici": "questo non significa nulla preso da solo". E sugli esempi moderni che seguono: "Hades ha 'meccaniche semplici'? UFO 50 ha letteralmente 50 titoli con meccaniche diverse, dovremmo prenderli singolarmente [...] o considerarli un unico titolo?"

5. **Definire troppo largo.** Lorenzo propone due definizioni di gioco ("A packaged state of mind that aims to give [...] chosen experiences"; "A juicy interaction infused with conflict") e il docente le smonta con controesempi: "secondo questa definizione, Journey è un arcade"; "secondo questa definizione, Call of Duty è un arcade"; e Flower, che Lorenzo stesso mette tra i "definitely not", rientrerebbe nella prima. "Comprendi che un risultato simile non è, purtroppo, accettabile date le premesse." Anche il criterio "reducing randomness to the lowest possible levels" cade su un controesempio: "diresti che Vampire Survivors non è arcade?"

6. **Fermarsi al passato.** Valerio descrive bene i cabinati degli anni '70–'90, ma "continui a parlare al passato, e nel presente?"; e "difficile ma facile da capire, con pochi comandi e tasti, e partite molto veloci" richiede "più informazioni specifiche".

Il controesempio come strumento. Il metodo del docente è lo stesso visto a lezione sulle definizioni di gioco (capitolo 2): ogni definizione include qualcosa ed esclude qualcos'altro, e la si mette alla prova cercando un gioco che la rispetta pur non essendo, evidentemente, ciò che si vuole definire. Prima di consegnare una definizione, conviene fare da sé questo esercizio con due o tre titoli noti.

Dati grezzi e sintesi. Il documento di Lorenzo parte da una ricerca ampia (fonti accademiche, storia dei cabinati, caratteristiche del genere), e il docente lo riconosce: "questa ricerca porta tutti dati utili nella definizione ultima". Il problema è che "restano dati grezzi": "la fase di selezione e spogliatura del non essenziale non è sufficiente. Il risultato sono due pagine di testo saltabile e una definizione di una pagina che è un wall of text". E più avanti: "quindi concretamente cosa è emerso da questo paragrafo di mezza pagina? Nessun punto critico che compone la tua ipotetica definizione finale." Chi legge vuole arrivare alla definizione "vuoi per praticità, vuoi per tempistiche": la ricerca va **distillata**, e in apertura serve sintesi ("io sono d'accordo con quello che scrivi, ma domandati come puoi essere più sintetico in apertura").

Che cosa ha funzionato. Francesca include il "sistema di memorizzazione dei punteggi migliori per stimolare la competizione tra i giocatori": "formulato correttamente". Federica sposta il criterio dal luogo all'esperienza ("non è più il luogo in sé a definire il genere", "almeno, non solo quello, ma sono d'accordo"), e il porting di *Geometry Dash* su cabinato "solidifica la tua tesi". Sui legami tra arcade, slot machine e coin-op, il docente corregge Lorenzo in modo istruttivo: l'associazione non deriva dai giochi d'azzardo, ma "a monte, dallo sfruttare bias cognitivi per la manipolazione del giocatore (senza entrare in discorsi etici)"; e la presenza maschile citata dagli studi è "legata principalmente alla posizione dei coin-op in passato". Infine, una domanda lasciata aperta: se un gioco qualsiasi diventa arcade quando è giocato in un cabinato, "la ritualità di una macchina con elementi fisici/meccanici è una discriminante? Non dico che sia sbagliato, ma apre possibilità discutibili".

1.4.6 Dichiarare e usare il framework

Dire quale framework. "Quale framework? Non ne esiste solo uno legato a loro. Parliamo dei Core Elements" (Antonino, su "framework di Romero e Schreiber"); "quale? Non ne esiste solo uno" (Alberto); "il framework resta quello dei core elements, da dire" (Lorenzo). Mattia e Valerio non lo dichiarano affatto. Il nome preciso va nel titolo o subito sotto: "Core elements (Romero e Schreiber)", "Tetrade elementale (Schell)", "Matrice di Bura". I framework sono descritti nel capitolo 2 (sezioni 2.7, 2.8 e 2.9).

Ogni cosa nella sua voce. Molti commenti spostano un'informazione nella voce corretta del framework:

- "Questa è una dinamica legata alle regole dello spazio di gioco, andava associata alle regole" (Francesca, sul passaggio dal fondo alla cima dello schermo);
- "Obiettivo: sarebbe stato utile inserirlo nel paragrafo delle meccaniche" (Mattia);
- "Va spiegato prima" (Valerio, sull'obiettivo arrivato a metà pagina); "mai parlato dello spazio di gioco prima di adesso, a metà della prima pagina";
- "Ti avrebbe potuto aiutare descrivere lo spazio di gioco all'inizio delle regole?" (Antonino);
- "Davvero è rilevante per il documento, in un elenco sulle risorse (e non, per esempio, nelle regole)?" (Antonino, sul consiglio di usare il nitro con parsimonia);
- "Non stai parlando di narrativa, stai parlando di estetiche qui" (Lorenzo, sulle influenze dell'art director);
- "Ha l'intera libertà di movimento", scritto nella narrativa: "meccanica, da riformulare in ogni caso" (Francesco); "a ogni reset il gioco è sempre più difficile": "come? Questa è una meccanica".

Niente che non sia stato introdotto. Un elemento citato in una voce deve essere stato spiegato prima, nella voce che gli compete. "Super Mode: mai parlato di questa modalità nelle meccaniche"; "punti: mai parlato di un sistema di punti, della coop locale né dell'high score"; "boss finale: di cui non so le meccaniche associate" (Francesca); "se prendi l'ultima bacca rimanente nel livello, questo si resetterà: purtroppo mai citato prima" (Francesco); "sistema di leaderboard da spiegare sopra" (Alberto); "Peccato non aver citato la bidimensionalità nelle meccaniche e nelle estetiche" (Federica, che la cita solo nella tecnologia).

La tecnologia, per Schell. Francesco, sotto "Tecnologia", descrive il movimento nel labirinto e i tunnel laterali: "quello che descrivi non è la tecnologia per Schell: dove gira Pac-Man? Su che piattaforme è disponibile? Quali sono i controlli fisici del gioco?" La stessa domanda torna ovunque: "cabinati con comandi fisici: quali? E ora, dove viene giocato o riprodotto?" (Francesca); "da cabinato di tipo upright. E ora? Su che macchine gira?" (Mattia). La tecnologia è **hardware, piattaforme (allora e oggi), controlli fisici**, e i limiti o le possibilità che ne derivano. Ogni affermazione tecnica va verificata: "con immediatezza e senza troppo input lag: da verificare e validare"; "molto semplice [il crossplatform]: opinione del gioco, non hai mai parlato del codice dell'applicazione". E ogni dettaglio va pesato rispetto all'analisi: i sequel su NES, Super Nintendo e Wii di *Punch-Out!!* sono "un dettaglio utile all'analisi tramite core elements? Domandatelo".

I sistemi di gioco. Mattia scrive che i sistemi di gioco "sono quelli di arrivare a fine livello collezionando più punti possibile": "No, i sistemi di gioco sono collezioni di meccaniche mirate allo stesso obiettivo: il sistema di punteggio, gli ostacoli di gioco, la locomozione del main character."

Meccanica o strategia? Alberto descrive la meccanica di *Tetris* come "incastrare pezzi senza lasciare spazi vuoti per completare le righe": "non è una meccanica questa, è una strategia". Il docente riscrive (si veda l'esempio nella sezione 1.4.7).

Risorse e modello mentale. Nel documento su *Crazy Taxi 2*, Lorenzo inserisce tra le risorse la "conoscenza della città": "non è una risorsa in gioco, fa parte del modello mentale". Al contrario, mancano risorse vere: "tempo e posizione?" (Antonino); "punti" e "punteggio", "altra risorsa" (Francesco). Nella matrice di Bura di *Valkyrie*, il docente nota che la voce più importante (la clessidra) è "in fondo all'elenco": l'ordine degli elenchi comunica priorità.

Game qualities. Le qualità sono "l'anima condensata del tuo gioco" (a chi aveva scritto "il PvE è l'essenza del gioco"). Gli studenti ne hanno indicate troppo poche. Ad Antonino: "puoi pensarne altri, come per esempio la camera di gioco"; "scalare le classifiche [...] può tranquillamente essere un'altra qualità"; il sistema di password "andrebbe citato nelle qualità del gioco". A Francesco, che indica solo il PvE: "ci sono altri elementi: Arcade, Top-Down Third Person. Ripensaci". A Lorenzo, che raggruppa i verbi in sistemi: "queste sono più qualità messe insieme, eppure non info sulle 3C del gioco" (camera, character, controls). "Elementi fisici", detto delle bacche di *Pac-Man*, è impreciso: "non seguono regole fisiche".

Players: servono dati. Nessuno ha portato dati sul pubblico, e il commento è sistematico: "servono i dati", "fascia di età" (Antonino); "qual è la fascia di età? La demografica dei giocatori? I dati?" (Francesco); "No. Non ci sono dati sulle demografiche: etnia, genere, segmenti di età, etc." (Lorenzo). Il docente riconosce che gli archetipi di giocatori non sono ancora stati spiegati ("non avendo ancora visto (giustamente) archetipi di giocatori non puoi semplificare chiaramente il concetto"), ma le affermazioni sul pubblico ("probabilmente perché ha poco tempo da spendere sui videogiochi") vanno sostenute da fonti.

Temi e obiettivi. Ad Antonino: "per quello che scrivi, c'è anche un tema riguardo alla competizione e alla scalata della vetta agonistica". Sull'obiettivo, a Federica: "l'obiettivo (meccanico e narrativo) resta arrivare alla fine [del livello] tutto d'un pezzo".

Le emozioni. Erano parte della consegna. A Lorenzo, che elenca "Auditory/visual stimulation" e "Challenge" senza commento: "nessuna spiegazione, nemmeno per challenge?" Ogni emozione va **motivata** con gli elementi del gioco che la producono, come il docente fa a Francesco: eccitazione e competizione "corretto, dato da elementi quali il chasing dei fantasmi, il time clock, la leaderboard".

1.4.7 Descrivere le meccaniche: il dettaglio richiesto

Il problema di contenuto più diffuso è uno: le meccaniche sono descritte **come le descriverebbe un giocatore** a un amico, non come le deve descrivere un designer a un team che dovrà ricostruirle. Il docente lo dice nel commento ad Alberto: "l'analisi presuppone un dettaglio a basso livello". E a Francesca suggerisce un metodo: "un modo per aiutare la comprensione è quello di partire dall'obiettivo e poi andare sempre più in dettaglio sulle meccaniche" (lo stesso approccio che, a Francesco, giudica funzionante: "iniziare con l'obiettivo può funzionare se stai descrivendo le meccaniche").

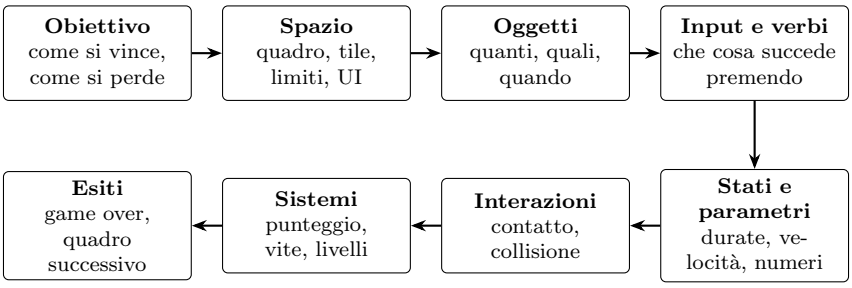


Figure 1.1: Dall'obiettivo al dettaglio: un ordine possibile per descrivere le meccaniche, ricostruito dalle domande che il docente pone nelle revisioni del task T1 (proposta del curatore, non uno schema mostrato a lezione).

Le domande del docente. Raccolte e raggruppate, le domande poste nei commenti formano una vera checklist di analisi. Conviene porsele per ogni meccanica, prima di consegnare.

Input "Cosa succede praticamente quando il giocatore preme il tasto? Quale effetto ha sul main character?" (Mattia, sul salto); "Se il main character sta compiendo un salto, e il giocatore preme di nuovo l'input associato, cosa succede?" (Francesca); "Con che input? Cosa succede durante l'azione 'scalare'?" (Mattia, sulle scale); "Come si compie questa azione di 'raccogliere'?" (Francesco); "Invece lato input di gioco?"; "Riguardo agli input coinvolti?" (Antonino); "E gli input lato giocatore/macchina?" (Lorenzo). Anche gli stati cambiano l'input: in *Geometry Dash* volare, cambiare forma e invertire la gravità sono "tutti verbi di gioco da spiegare nel dettaglio, perché con lo stesso input il risultato cambia a seconda dello stato attuale del main character".

Movimento e limiti "Muoversi liberamente": "Falso. [...] Se il giocatore si muove contro un muro? Può muoversi verso l'alto senza saltare?" (Francesca); "Allora non è 'libero'. E se finisce in un 'buco' ai limiti dello spazio fisico del gioco?" (Francesco); "Quanto posso uscire dal tracciato?" (Antonino); "Va descritto anche cosa significa 'partire' "; "va descritto il sistema di accelerazione e velocità generale".

Oggetti e nemici "Quanti e quali" (ripetuto più volte); "Quando vengono droppati? Che oggetti sono? Come si ottengono e come si usano?"; "Stanno fermi? Si muovono? Hanno comportamenti unici?" (Francesca, sui nemici); "Come si muove la polizia, che azioni ha a disposizione, che interazione può compiere o subire" (Lorenzo); "Mancano elementi chiave come ostacoli, NPC. Cosa popola il mondo di gioco?"; "ombrello, cappello, borsa: in quanto lettore, non so cosa siano questi vantaggi"; "spawn: non so quale sia né dove sia".

Collisioni "Per ciascuno [oggetto], cosa succede se il giocatore ci collide?" (Federica).

Parametri "Tot di secondi: esplicitare questo parametro"; "300, 500, 800: da spiegare"; "4 o più barili: contemporaneamente o in lasso di tempo definito?"; "determinato numero: quante?"; "determinato numero di punti: quanti? Link alla tabella della wiki"; "velocità della caduta: ok, ma puoi essere più specifico"; "colpi critici: cosa significa? Che regole hanno?".

Sistemi "Da spiegare il sistema di livelli, high score e game over"; "dunque esiste un sistema di tempo, da spiegare"; "punti: sistema da spiegare, come quello dei livelli"; "i punti collezionati: sono mostrati da qualche parte? Aumentano in tempo reale?"; "vite: come lo 'uccidono' e cosa succede a queste 'vite'?"; "checkpoints: che cosa sono? Come funzionano?".

Esiti "Ora so che il gioco si basa su livelli, ma non so cosa succede quando uno finisce e uno nuovo inizia"; "cosa succede dopo il game over?"; "manca l'outcome del game over: dove viene riportato il giocatore?"; "cosa comporta il game over"; "cosa significa 'game over'?"; "si resetterà: cosa significa a livello pratico?"; "round: come vengono segnate [le vittorie]? Dove?".

La core mechanic va trovata. Su *Tetris* il docente riscrive di persona il cuore dell'analisi di Alberto. È l'esempio migliore di che cosa si intende per "basso livello":

Nella consegna	**Riscrittura del docente**
La meccanica base del gioco è incastrare pezzi (tetramini) l'uno con l'altro, senza lasciare spazi vuoti per completare le righe, una volta che una riga viene completata sparisce dando dei punti.	"La core mechanic è spostare/ruotare il tetramino assegnato. Una volta posizionato, un nuovo tetramino viene generato e assegnato. Ci sono altre meccaniche: la UI che mostra il prossimo tetramino che verrà generato, il sistema di despawn delle righe (e quello di punteggio, che va spiegato), e così via."

Lo stesso Alberto, poco sotto, aveva colto il punto: le rotazioni e gli spostamenti "possono essere eseguiti tutte le volte desiderate, a una condizione, ovvero quella di farlo prima che avvenga un contatto con altri tetramini": "corretto, ecco le tue core mechanic". E ancora, per *Bubble Bobble*: "sparare bolle: questa meccanica (quella core) necessita di maggiori dettagli".

Un esempio da riscrivere: il cambio di stato. Su *Pac-Man* il docente abbozza la forma che la descrizione dovrebbe avere: "meccanica da spiegare: Pac-Man cambia stato per N secondi, così come i fantasmi che avranno come nuovo obiettivo fuggire da lui [...]. Va spiegato, nel dettaglio." Sviluppando lo schema (proposta del curatore):

Nella consegna	Riscrittura proposta
All'interno del mondo di gioco sono presenti anche delle bacche speciali, che rendono il giocatore "ostile" verso i fantasmi. Una volta raggiunto uno, questo non scomparirà, ma tornerà al proprio punto di spawn. (Per ogni fantasma catturato, vengono attribuiti 200/400/600/800 punti in più).	*Power pellet* (4 per quadro, agli angoli del labirinto [immagine]). Al contatto con Pac-Man: • Pac-Man entra nello stato *potenziato* per N secondi (N diminuisce con il livello); • i fantasmi passano allo stato *spaventato*: cambiano sprite, rallentano, invertono la direzione e fuggono; • al contatto con un fantasma spaventato, il fantasma torna alla casa al centro del labirinto e il giocatore guadagna 200, 400, 800, 1600 punti per fantasma consecutivo.

Si noti che la riscrittura risponde, una per una, alle domande del docente: "catturato: non ho idea di come farlo, come lettore" (al contatto), "spawn: non so quale sia né dove sia" (la casa al centro), "determinato: quante?" (4).[2]

Riassunti corretti, ma non abbastanza. Il docente riconosce più volte un buon riassunto, chiedendo però di scendere di livello: "tutta questa sezione è un corretto riassunto delle meccaniche di Geometry Dash, ma puoi essere più specifica sui singoli elementi [...]. Tutte domande che determinano la profondità dell'analisi" (Federica); "corretto. Avrei voluto leggere di più a riguardo" (Francesca, sul salto sulle bolle). Il riassunto è il punto di partenza, non il punto d'arrivo.

1.4.8 Narrativa, estetiche e tecnologia

La trama in 5W+H. A Francesca (*Bubble Bobble*): "prova a rispondere alle 5W+H" (*who, what, when, where, why* e *how*), e poi "un paio di righe in meno e ci siamo come riassunto". A Lorenzo, su una trama di *Jet Set Radio* lunga e piena di nomi propri: "tutto questo va riassunto con 5W+H". Il riferimento positivo è la riga che la precede: "JSR has a linear and scripted narrative, served in small portions between levels by DJ Professor K": "questo è corretto, preciso, succinto. Parti da questo". Anche il riassunto di Valerio su *Dino Rex* riceve "sintetico, funziona".

Il tipo di narrativa. Va detto: a Mattia, "manca il tipo di narrativa, che è lineare"; Francesca la dichiara ("ramificata, con tre finali") e i finali sono apprezzati come approfondimento sulla morale e sull'intento di design ("ottimo").

"Non c'è storia" non è mai vero. "There is no real story": "questo non è mai vero, capisco quello che intendi ma va riformulato" (Lorenzo). Allo stesso modo, "non racconta direttamente" "andrà formulato diversamente" (Federica, rimandato alle lezioni di narrativa). Anche quando il gioco non ha una trama esplicita, la narrativa passa da altri canali: "indubbiamente la cover art e la scocca del cabinato possono dare informazioni sulla narrativa di gioco, pensa a Space Invaders dove il tema è tutto 'spiegato' lì" (Alberto). Le letture personali (*Tetris* come "conflitto infinito tra caos e ordine") sono ammesse solo con dati: "una tua lettura, alla quale posso pure venire incontro, ma servono i dati"; e "molti designer": "chi? Dove sono i dati?".

[2] Nota del curatore: nella consegna i punti per fantasma erano indicati come 200/400/600/800; i valori del gioco originale sono 200, 400, 800 e 1600, raddoppiati a ogni fantasma mangiato con lo stesso power pellet. È esattamente il tipo di dato per cui il docente chiede il "link alla tabella della wiki".

Estetiche: concrete, non impressioni. Le estetiche sono state descritte quasi sempre per impressioni ("colorata e saturata", "giocosità", "colori accesi", "colorato e sgargiante"). Il docente chiede ogni volta l'elemento concreto:

- **palette** ("palette principale?"), **stile artistico** ("perché? Parla dello stile artistico, anche sinteticamente");
- **UI** ("mancano elementi come lo stile della UI, gli SFX"; "puoi parlare anche della UI qui");
- **audio**: stile della colonna sonora, BPM, SFX ("continui a non darmi informazioni estetiche sullo stile della colonna sonora, per esempio i BPM, e anche degli SFX");
- **reference**: "esempi, reference, link"; "esempi concreti, immagini"; "dati, reference"; "corretto, ma le reference?".

Le affermazioni devono corrispondere alle reference: "sanguinano copiosamente: in nessuna delle immagini che hai inserito c'è un rivolo di sangue". E vanno contestualizzate storicamente: "fotorealistica: questo dipende dal contesto storico, detto adesso è falso". Anche la lettura "soulful graphics" di Lorenzo è "inquantificabile rispetto anche alle tecnologie usate oggi, in contrapposizione al tubo catodico ormai desueto": una tecnica nata da limiti che oggi non esistono più. Mattia, infine, aveva omesso del tutto le estetiche: "mancano le estetiche".

Tecnologia. Si veda la sezione 1.4.6: per Schell è hardware, piattaforme e controlli fisici, allora e oggi. Il "feeling" dei comandi fisici è "da approfondire"; le piattaforme vanno elencate (o linkate, se sono molte: "quale delle N versioni portatili? Puoi anche linkare una pagina"); l'interpretazione dei fatti va evitata ("limitava il giocatore, perché non poteva giocarci dove voleva": "al limite della tua interpretazione dei fatti"). Bene, invece, la socialità del co-op "spalla a spalla" (Francesca: "questo invece è corretto") e il pubblico dietro al cabinato (Lorenzo: "interessante, mi trovi d'accordo").

1.4.9 Mostrare invece di scrivere

"Dovrei vederlo, non leggerlo." La prima golden rule (sezione 3.7) torna in decine di commenti:

- la UI e il punteggio: "puoi anche lavorare sulle immagini di gioco, mostrando chiaramente dove sia questo punteggio a schermo"; "barra della salute: come sono fatte? Ce ne sono più di una? Di che altri elementi si compone la UI?";
- gli sprite: "come lettore dovrei vedere questi sprite";
- le peculiarità visive: "quello inferiore dedicato all'incontro: perché non lo mostri?"; "come una figura trasparente di spalle: da vedere. Uguale per gli oggetti di gioco, lavora sulle immagini";
- i comandi: "assolutamente corretto, ma come puoi mostrarlo senza scriverlo così?" (un'immagine del pannello di controllo con le funzioni sui tasti dice tutto in meno spazio).

Tabelle e matrici. Quando i dati sono confrontabili, vanno **tabellati**. Ad Antonino, sulle quattro statistiche delle auto (velocità massima, accelerazione, grip, consumo): "prova a pensare a come sarebbero potute essere ancora più visibili come informazioni"; e su 8 nazioni $\times$ 4 gare: "ora immagina come avresti potuto condensare tutto, risparmiando spazio e scrivendo di meno, però con un risultato ancora più chiaro per il lettore". A Lorenzo, sul controllo finale della sua "ricetta" arcade: "come avresti potuto rendere questo una matrice/tabella visivamente chiara e compatta nel doc?". L'idea delle catene causa–effetto con le frecce ("Crazy Hop $\rightarrow$ shorter route $\rightarrow$ ...") è invece "interessante, iteraci sopra": è già un flowchart in forma testuale.

Immagini che lavorano. Quando l'immagine c'è ed è al posto giusto, il docente lo nota: l'elenco dei personaggi di Francesca "già così è molto chiaro considerando anche l'immagine a destra"; l'immagine dei tetramini di Alberto è "una aggiunta molto chiara e corretta". Sui tetramini manca però ancora "tipo e vestizione artistica", da mettere nelle estetiche.

1.4.10 Che cosa è stato apprezzato

Raccolti in un solo posto, i commenti positivi indicano che cosa **ripetere**:

- un **titolo essenziale** con tutti gli elementi necessari, anche senza immagini (Francesca); un titolo funzionale con gioco e framework (Federica);
- un uso oculato dello spazio, "considerando che questo è il primo task" (Francesca); intuizioni di formattazione nel paragrafo sulla tecnologia di Alberto;
- l'uso **corretto delle tab** (Federica);
- un **disclaimer** corretto: "ad eccezione dei livelli speciali creati dalla community" (Federica);
- le **statistiche fondamentali** tutte citate (le auto di *Top Gear*), la sezione delle **risorse** che "funziona per contenuto", le **interazioni** "essenziali, ok" (Antonino);
- le informazioni sui **finali**, che toccano "la morale del gioco e il possibile intento di design. Ottimo" (Francesca);
- la **socialità** del co-op e del pubblico come elemento di tecnologia ed esperienza (Francesca, Lorenzo);
- il **riassunto in apertura** del documento come strategia ("può essere una strategia che funziona, per dare al lettore info di partenza. Va ripensata la messa a terra, però", Valerio);
- il riassunto narrativo **sintetico** (Valerio, *Dino Rex*; Lorenzo, la riga sul DJ Professor K);
- le meccaniche di **timer e sudden death** spiegate bene ("la scrivi anche correttamente"), anche se in una parentesi da cui vanno tolte (Valerio);
- i **dati di ricerca** di Lorenzo, "tutti dati utili", e le osservazioni corrette sull'etica del continue ("did I really beat the game if I fed the machine with coins to continue?": "corretto") e sulla performatività del co-op;
- in generale, per Francesca: "un interessante primo tentativo di documentazione, sia per contenuto che per forma".

1.4.11 Le schede delle singole consegne

Seguono, documento per documento, **tutti** i commenti del docente, nell'ordine in cui compaiono nel documento, ciascuno con il punto a cui è ancorato. Le ripetizioni ("spazio extra", "testo inutile") sono lasciate: la loro frequenza è essa stessa un'informazione. Per la legenda si veda la sezione 1.1.

Francesca Attadia — *Bubble Bobble* (Schell)

Documento giudicato nel complesso "un interessante primo tentativo". Punti deboli: meccaniche e nemici descritti ad alto livello, elementi (punti, Super Mode, boss) citati senza essere stati introdotti, estetiche per impressioni, "accattivante" al limite della parola proibita.

- "*ARCADE CORE ELEMENTS*" : [+] Pur non osando con immagini, il titolo rimane essenziale e presenta tutti gli elementi necessari. Funziona *E ancora:* In generale, un interessante primo tentativo di documentazione, sia per contenuto che per forma
- "*videogioco arcade*" : Ripetizione evitabile, lo scrivi nella domanda
- "*l'immediatezza e l'intuitività dei comandi di base*" : Sarebbero necessari esempi concreti e quantificabili lato "immediatezza", ma per ora lo accetto
- "*breve durata delle partite*" : Anche qui, relativo. Serve un metro di paragone
- "*sistema di memorizzazione dei punteggi migliori per stimolare la competizione tra i giocatori*" : [+] Formulato correttamente
- "*sono spesso accattivanti*" : Al limite dell'opinione personale, non significa nulla

- "*alto volume*" : Anche questo, da capire cosa significhi "alto". L'inciso dopo non risolve completamente il problema
- "*spazio*" : Per ora va bene, ma nonostante sia uno spazio logico anche la UI dovrebbe essere inclusa in questo discorso. Inoltre, lo spazio in questo caso si fonda di elementi minori come i tile, che fanno parte delle regole
- "*muoversi liberamente*" : Falso. Ne riparleremo, ma se il giocatore si muove contro un muro? Può muoversi verso l'alto senza saltare? Queste sono solo alcune delle domande che puoi fare per smarcare questa dicitura inesatta
- "*livello*" : Ora so che il gioco si basa su livelli, ma non so cosa succede quando uno finisce e uno nuovo inizia *E ancora:* Un modo per aiutare la comprensione è quello di partire dall'obiettivo e poi andare sempre più in dettaglio sulle meccaniche
- "*bolle*" : Altra meccanica da inserire insieme al salto e al movimento
- "*oggetti droppati*" : Quando vengono droppati? Che oggetti sono? Come si ottengono e come si usano?
- "*boss finale*" : Di cui non so le meccaniche associate
- "*verso l'alto o il basso per spostarsi tra di esse*" : Se il main character sta compiendo un salto, e il giocatore preme di nuovo l'input associato, cosa succede?
- "*sparare bolle*" : Questa meccanica (quella core) necessita di maggiori dettagli
- "*non riuscirebbe ad arrivare con le sole piattaforme*" : [+] Corretto *E ancora:* Avrei voluto leggere di più a riguardo
- "*lo farà scendere immediatamente da un'apertura sopra lo schermo.*" : Questa è una dinamica legata alle regole dello spazio di gioco, andava associata alle regole
- "*nemici*" : Quanti e quali
- "*fargli perdere vita toccandoli o sparandogli da distanza*" : Stanno fermi? Si muovono? Hanno comportamenti unici? Serve maggiore dettaglio
- "*Altri pezzi del gioco fondamentali*" : Testo inutile
- "*punti*" : Mai parlato di un sistema di punti, della coop locale né dell'high score
- "*La narrativa*" : Testo quasi inutile
- "*Storia:*" : Prova a rispondere alle 5W+H *E ancora:* Un paio di righe in meno e ci siamo come riassunto
- "*Bub (Bubby) – il draghetto verde, il primo protagonista e player 1 Bob (Bobby) – il draghetto blu, il secondo protagonista e [. . .]*" : [+] Come per qualsiasi cosa, si può iterare, ma già così è molto chiaro considerando anche l'immagine a destra
- "*Super Mode*" : Mai parlato di questa modalità nelle meccaniche
- "*al giocatore rimane un senso di mistero:*" : [+] Benché queste informazioni aggiuntive sui finali non siano necessarie per la fruibilità base del documento, sono aggiunte interessanti che toccano in parte la morale del gioco e il possibile intento di design. Ottimo
- "*La grafica*" : Testo inutile
- "*colorata e saturata*" : Palette principale?
- "*giocosità*" : Perché? Parla dello stile artistico, anche sintenticamente
- "*sono acuti e trasmettono giocosità*" : Esempi, reference, link
- "*leggero*" : Più info a riguardo, al limite dell'opinione personale
- "*competizione amichevole*" : Cooperative, Fellowship, People Fun. Ne riparleremo
- "*Bubble Bobble*" : Testo inutile, non è cambiato l'oggetto dell'analisi
- "*cabinati con comandi fisici*" : Quali? *E ancora:* E ora, dove viene giocato o riprodotto?
- "*rende il feeling del gioco stesso più diretto e presente*" : Da approfondire
- "*accattivante*" : [!] Al limite della parola proibita, prossima volta rischi la non lettura
- "*grazie alla socialità che si crea tra i due sfidanti.*" : [+] Questo invece è corretto *E ancora:* Leggero spazio sprecato qui sotto, nel resto del documento invece è sfruttato in maniera oculata, considerando che questo è il primo task

Antonino Etiopia — *Top Gear* (core elements di Romero e Schreiber)

Contenuto solido su regole, risorse e interazioni; i problemi principali sono l'impaginazione (elenchi rientrati tre volte, mezza pagina vuota in coda), il framework non nominato, l'assenza di dati sui giocatori e le qualità del gioco troppo poche.

- "*Analisi del gioco Top Gear*" : Testo inutile, lo scrivi letteralmente sopra
- "*framework di Romero e Schreiber.*" : Quale framework? Non ne esiste solo uno legato a loro. Parliamo dei Core Elements *E ancora:* L'immagine qui a fianco per comunicare graficamente il titolo del gioco (e, per estensione, del documento) non è sbagliata di per sé. Però devi ragionare attentamente agli spazi, a dove inserirla (e come) soprattutto rispetto a quello che già scrivi come titolo "letterale"
- "*Game qualities:*" : [**!**] Puoi pensarne altri, come per esempio la camera di gioco *E ancora:* Inoltre aver scelto di inserire la lista di qualità in elenco puntato tre volte tabbato di mangia dello spazio. Non è di per sé una soluzione sbagliata, ma puoi iterare sulla messa a terra: pensaci *E ancora:* Sul mio commento qui sopra, questa cosa può funzionare per alcuni paragrafi, ma su tutto il documento no. Hai sacrificato molto spazio a sinistra, ripetere questa cosa in futuro ti costa la non lettura
- "*Top gear*" : Testo inutile, inoltre la "G" deve essere maiuscola
- "*lunghe corse*" : Serve un metro di misura, al limite dell'opinione personale
- "*scalare le classifiche*" : Questo è un elemento che può tranquillamente essere un'altra qualità
- "*essenza del gioco*" : Tutti le game qualities sono l'anima condensata del tuo gioco
- "*veicoli controllati dall'IA per poter proseguire nel gioco e conquistare la vittoria.*" : O lo scrivi sopra o lo scrivi qui, discutibilmente è un punto essenziale del PvE
- "*Gestione delle risorse:*" : Molto riassuntivo
- "*Top gear*" : Testo inutile
- "*semplicità del gameplay e dall'immediatezza del gioco*" : Da definire "semplice" e "immediato"
- "*I giocatori*" : Testo inutile
- "*Top gear*" : Errore grammaticale
- "*piacciono giochi immediati e con un gameplay concentrato sull'azione*" : Servono i dati
- "*probabilmente perché ha poco tempo da spendere sui videogiochi.*" : Anche qui, dati a supporto. Anche perché non avendo ancora visto (giustamente) archetipi di giocatori non puoi semplificare chiaramente il concetto
- "*bambini e i ragazzi*" : Fascia di età
- "*vecchio gioco arcade*" : Cosa significa questo?
- "*storia non è molto sviluppata*" : Inizio ad avere difficoltà sul capire come hai scelto grassetto, italico e sottolineatura: perché mi sembra che il risultato sia il medesimo, far risaltare elementi importanti
- "*sorpassando gli avversari uno per uno.*" : Per quello che scrivi, c'è anche un tema riguardo alla competizione e alla scalata della vetta agonistica
- "*(come sopra citato)*" : Allora non dirlo
- "*una serie di scenari*" : Quanti e quali
- "*richiede una password per accedervi*" : Altro sistema che andrebbe citato nelle qualità del gioco
- "*scenari (8 nazioni in totale) sono presenti 4 gare (32 in totale)*" : [+] Esattamente la risposta alla mia domanda sopra. Ora immagina come avresti potuto condensare tutto, risparmiando spazio e scrivendo di meno, però con un risultato ancora più chiaro per il lettore
- "*p*" : Errore grammaticale
- "*Top Gear*" : Testo inutile, il gioco è rimasto lo stesso
- "*regole utili*" : Testo inutile
- "*una difficoltà moderata*" : Opinione personale
- "*tutti partono allo stesso momento.*" : Va descritto anche cosa significa "partire"
- "*rendendole più veloci e rallentando se stessi*" : Va descritto il sistema di accelerazione e velocità generale
- "*ma soltanto tra una gara e l'altra.*" : Non stai dicendo come si usano e quando vengono consumate (sottinteso, ma non esplicito)

- "*velocità massima, accelerazione da 0 a 100, grip e consumo del carburante; il giocatore deve selezionare un'auto prima di [...]*" : [+] Tutte statistiche fondamentali da citare. Corretto *E ancora:* Prova a pensare a come sarebbero potute essere ancora più visibili come informazioni
- "*n*" : Maiuscola mancata
- "*ma uscire dal tracciato o urtare oggetti rallenterà il proprio veicolo.*" : [+] Corretto, quanto posso uscire dal tracciato?
- "*costretto a guidare nel tracciato*" : Nel punto sopra scrivi una cosa diversa
- "*mappa*" : Ti avrebbe potuto aiutare descrivere lo spazio di gioco all'inizio delle regole? Non è una necessità, ma riflettici
- "*Risorse:*" : Tempo e posizione? *E ancora:* In ogni caso, questa sezione funziona per contenuto (al netto dei commenti), per il futuro pensa a come potenziarne ancora di più la forma
- "*In Top Gear ci sono poche risorse*" : Opinione personale e testo inutile
- "*sacrificando un po' di tempo.*" : Quantità specifica, anche con un margine di errore va bene
- "*è da usare con parsimonia*" : Davvero è rilevante per il documento, in un elenco sulle risorse (e non, per esempio, nelle regole)?
- "*Interazioni:*" : [+] Essenziale, ok *E ancora:* Riguardo agli input coinvolti?
- *(riga vuota o spazio)* : Qui sotto c'è mezza pagina vuota, è tutto spazio sprecato

Federica Ferrara — *Geometry Dash* (Schell)

Tab usate correttamente e definizione di arcade interessante, ma lasciata con una domanda aperta; margini errati (ultimo avviso), spazi extra, tono da recensore, meccaniche corrette ma poco profonde.

- "——————" : Non estremamente esplicativa dell'obiettivo del documento, ma l'essenzialità del titolo c'è
- "*Innanzitutto, perché proprio Geometry Dash? perché questa scelta così azzardata? Cosa intendo per gioco arcade?*" : Questa è una frase da "recensore". Quello che scrivi nella descrizione già dovrebbe incorporare questa tua "azzardo", dissipando ogni potenziale dubbio. So bene che è la "tua" definizione in quanto il documento lo hai scritto tu, ma sono le spiegazioni concrete qua sotto e il loro legame che implicitamente di definisce come Designer
- "*p*" : Maiuscola
- "*un tempo*" : Sarebbe stato utile avere margini temporali, ma ok *E ancora:* Sconsiglio caldamente l'uso del testo giustapposto, in generale
- "*non è più il luogo in sé (o lo strumento) a definire il genere*" : Almeno, non solo quello, ma sono d'accordo
- "*In questo caso l'esperienza*" : Testo inutile, lo scrivi prima del punto
- "*intuitive*" : Da capire cosa significhi concretamente, provi a spiegarlo dopo e, per ora, lo accetto. Quando capiremo cosa significa definire il gameplay, avrai più strumenti. Detto questo, è una visione interessante
- "*ragionamento*" : Estremo questo
- *(riga vuota o spazio)* : Spaziatura extra
- "*La domanda è: ad oggi, un gioco platform può essere considerato anche arcade?*" : Ok, ma mi devi rispondere tu, altrimenti il doc è amputato
- *(riga vuota o spazio)* : Spaziatura extra 2
- "——————" : Mezza pagina bianca qui sotto, è spazio sprecato
- "*Geometry Dash Core Elements - Jesse Schell*" : [**!**] Titolo funzionale, puoi per il futuro pensare a come fargli occupare maggiore spazio a sinistra e destra, ma è una piccolezza *E ancora:* Hai sfruttato correttamente le tab del documento, ma i margini non sono quelli stabiliti. La prossima volta non leggerò il documento se ripeterai l'errore
- *(riga vuota o spazio)* : Scelta peculiare di spazio e punteggiatura, ma de gustibus
- "*)*" : Non hai aperto la parentesi
- "*Il tutto a ritmo di musica.*" : Tutta questa sezione è un corretto riassunto delle meccaniche di Geometry Dash, ma puoi essere più specifica sui singoli elmenti: come oggetti di gioco hai quelli elencati (monete, spine, seghe circolari, etc.), ma per ciascuno, cosa succede se il giocatore ci collide? Cosa succede se il giocatore muore? Cosa significa "game over"? Tutte domande che determinano la profondità dell'analisi

- *(riga vuota o spazio)* : Spazio extra 3, non li segno più ma rimangono un problema da addressare
- *(riga vuota o spazio)* : Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato
- *(riga vuota o spazio)* : Sia chiaro, questo vale ovunque nel documento
- "*volare, cambiare la propria forma e invertire la gravità.*" : Tutti verbi di gioco da spiegare nel dettaglio, perché con lo stesso input il risultato cambia a seconda dello stato attuale del main character
- "*checkpoints*" : Che cosa sono? Come funzionano? Anche queste sono meccaniche da spiegare
- "*Ad eccezione dei livelli speciali creati dalla community*" : [+] Disclaimer corretto
- "*non racconta direttamente*" : Andrà formulato diversamente, ma lascio la patata bollente alle lezioni da narrativa
- "*Ci*" : Sempre per quel discorso "recensore" che facevo, può esserti utile mettere della distanza tra te e il lettore: allena anche l'oggettività mettendo da parte l'ego
- "*spingendosi sempre più lontano.*" : L'obbiettivo (meccanico e narrativo) resta arrivare alla line di fine tutto d'un pezzo
- "*elettronici come dubstep e techno. Ogni ostacolo è messo a ritmo di musica*" : Ci sarebbe da fare un'eccezione sul quel "ogni", ma è corretto data l'aggiunta di reference
- "*colorato e sgargiante*" : Esempi concreti, immagini *E ancora:* Purtroppo le immagini sono cadute nella pagina seguente, intuizione corretta ma applicazione da rivedere
- "*frustrazione e soddisfazione*" : Fiero è la parola che cerchi
- "*successivo.*" : Spazio sprecato (minore) qui sotto
- "*Il gioco esce*" : Testo inutile
- "*con immediatezza e senza troppo input lag.*" : Da verificare e validare
- "*molto semplice*" : Opinione del gioco, non hai mai parlato del codice dell'applicazione
- "*su cabinati*" : [+] Assurdo. The more you know. Tra l'altro l'averlo messo su cabinato solidifica la tua tesi nella tab precedente
- "*che non sia bidimensionale.*" : [**!**] Peccato non aver citato la bidimensionalità nelle meccaniche e nelle estetiche *E ancora:* Mezza pagina di spazio sprecato sotto, errore grave per il futuro

Mattia Maestri — *Donkey Kong* (framework non dichiarato)

Il documento con più avvertimenti formali: margini e font errati ("prima e ultima volta"), titolo in rosso RGB (ultimo avviso), definizione di arcade assente, estetiche mancanti, framework non toccato in tutte le sue voci.

- "*DONKEY KONG*" : [**!**] Rosso RGB, citato come colore bandito nelle slide sulla documentazione. La prossima volta che lo vedo non leggo il documento
- "*1981)*" : Che tipo di documento è? Design? Analisi? Secondo che framework? Mancano informazioni cruciali *E ancora:* Inoltre il titolo è esteso verticalmente, troppo, considerando lo spazio che perdi sui lati. Questo spazio sprecato si ripete in tutto il resto del task, ed è un problema da risolvere per il futuro
- "*Cos'è?*" : [**!**] Il documento non ha i margini corretti e il font non è Roboto. Prima e ultima volta che accetto una consegna simile
- "*Donkey Kong*" : Testo inutile, è nel titolo
- "*arcade*" : Spiegare cosa significa era parte del task
- "*1981*" : Lo hai scritto già sopra
- *(riga vuota o spazio)* : Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato. Vale in tutto il documento ma lo scrivo solo qui per semplicità
- "*nel gioco sono presenti diverse meccaniche:*" : Testo inutile
- "*Punteggio*" : Sistema che incrementa sulla base di azioni compiute nel quadro, aggiornando un contatore sempre presente nella UI *E ancora:* Puoi anche lavorare sulle immagini di gioco, mostrando chiaramente dove sia questo punteggio a schermo. Dopotutto hai ampio spazio qui sulla destra dell'elenco puntato
- "*Il punteggio in Donkey Kong*" : Tutto testo inutile

- "*barile*" : Mi aspetto che poi vengano spiegati gli oggetti di gioco come gli ostacoli
- "*4*" : Contemporaneamente o in lasso di tempo definito?
- "*300-500-800*" : Da spiegare
- "*tempo impiegato per completare il livello*" : Dunque esiste un sistema di tempo, da spiegare
- "*il salto permette di superare ostacoli come i barili e le fiamme.*" : Ok, ma al netto del testo inutile, cosa succede praticamente quanto il giocatore preme il tasto? Quale effetto ha sul main character?
- *(riga vuota o spazio)* : Qui lo spazio sprecato è parecchio e ripetuto
- "*tot di secondi*" : Ok, prossimo step: esplicitare questo parametro
- "*Scale permettono di procede col livello.*" : Con che input? Cosa succede durante l'azione "scalare"?
- "*livelli*" : Da spiegare il sistema di livelli, high score e game over
- "*ombrello capello borsa*" : In quanto lettore, non so cosa siano questi vantaggi ai quali accenni
- "*La narrativa racconta di*" : Testo inutile
- *(riga vuota o spazio)* : Spazio extra, ma il riassunto della narrativa è corretto. Manca il tipo di narrativa, che è lineare
- "*Donkey kong*" : Errore grammaticale e testo inutile
- "*da cabinato di tipo upright.*" : E ora? Su che macchine gira?
- "*Obiettivo*" : Sarebbe stato utile inserirlo nel paragrafo delle meccaniche, mancano in ogni caso le estetiche
- "*L'obiettivo in Donkey Kong*" : Testo inutile
- "*I componenti del gioco sono quelli di arrivare a fine livello collezionando più punti possibile*" : No, i sistemi di gioco sono collezioni di meccaniche mirate allo stesso obiettivo: il sistema di punteggio, gli ostacoli di gioco, la locomozione del main character. Non comprendo l'utilità di questi paragrafi minori in coda al documento, con una pagina intera di spazio sprecato *E ancora:* Non tutti i punti del framework sono stati toccati nell'analisi, e mancano informazioni puntuali sulle meccaniche e la tecnologia. Questo è cruciale per lo svolgimento di un task

Francesco Muratore — *Pac-Man* (Schell e Romero e Schreiber)

Due framework nella stessa tab, margini errati (ultimo avviso), pagina di Romero e Schreiber in wall of text (ultimo avviso), linguaggio debole, tecnologia confusa con lo spazio di gioco, molte meccaniche citate e non spiegate.

- "*Pac-Man*" : Testo inutile, è scritto sopra
- "*secondo i framework di Brenda Romero e Jesse Schell*" : No, l'utilità di avere più tab è proprio quella di dividere parti differenti dello stesso doc, qui hai Schell e di là Romero+Schrieber
- "*Definizione di Gioco Arcade:*" : **[!]** I margini del documento non sono quelli stabiliti, la prossima volta non leggo il task
- "'" : Usa la punteggiatura specifica quando possibile
- "*lineari*" : Narrativamente o lato gameplay?
- "*prettamente simile*" : In che punti, nello specifico?
- "*potrebbe avere*" : Niente linguaggio debole. Stai scrivendo un documento di analisi, sii certo di quello che inserisci e di come lo dici
- "*prettamente semplici da memorizzare.*" : Senza un metro di misura questa è una opinione personale
- *(riga vuota o spazio)* : Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato. Vale per ogni uso di spazio simile, per rompere il testo, nell'intero documento
- "*Jesse Schell:*" : Qui passi da un punto, che è la definizione di arcade, a un framework. Questa "intestazione" deve chiaramente presentarsi come divisore, deve avere maggiore importanza per il lettore
- "*punti*" : Giallo su bianco è poco leggibile
- "*raccogliendo*" : Come si compie questa azione di "raccogliere"? *E ancora:* Iniziare con l'obiettivo può funzionare se stai descrivendo le meccaniche
- "*si*" : Chi? Il giocatore? Il main character? Perché la frase è impersonale?

- "*si resetterà*" : Cosa significa a livello pratico?
- "*punti collezionati*" : Sono mostrati da qualche parte? *E ancora:* Aumentano in tempo reale?
- "*ucciderlo togliendo lui tutte e 3 le vite*" : Come lo "uccidono" e cosa succede a queste "vite"
- *(riga vuota o spazio)* : Doppio spazio
- *(riga vuota o spazio)* : Spazio extra
- "*presentati diversi metodi*" : Quali?
- "*determinato*" : Quante?
- "*determinato numero di punti*" : Quanti? Link alla tabella della wiki
- "*che rendono il giocatore ''ostile" verso i fantasmi.*" : Meccanica da spiegare: Pac-Man cambia stato per N secondi, così come i fantasmi che avranno come nuovo obiettivo fuggire da lui [...] *E ancora:* Va spiegato, nel dettaglio
- "*spawn*" : Non so quale sia né dove sia
- "*catturato*" : Non ho idea di come farlo, come lettore
- "*Dal punto di vista narrativo,*" : Tutto testo inutile *E ancora:* Questa parte è al limite del wall of text, assolutamente da evitare
- "*ma non nell'azione.*" : Assolutamente non chiaro
- "*sempre lo stesso*" : Eccetto che per un elemento, quale?
- "*difficile*" : Come? Questa è una meccanica
- "*ha l'intera libertà di movimento all'interno del mondo di gioco, secondo i limiti di esso.*" : Meccanica, da riformulare in ogni caso
- *(riga vuota o spazio)* : Spazio extra
- "*L'estetica:*" : Mancano elementi come lo stile della UI, gli SFX, etc.
- "*Ciò che attrae di più di Pac-Man sono sicuramente*" : Al limite dell'opinione personale
- "*sensazione di eccitazione e di competizione*" : [+] Corretto, dato da elementi quali il chasing dei fantasmi, il time clock, la leaderboard (non citata nelle meccaniche)
- "*perenne*" : Eliminabile
- "*Un*"" : Grammatica
- "*colori accesi*" : Dati, reference
- "*una persona potrebbe tranquillamente essere attratta dai personaggi*" : Linguaggio debole, tutto evitabile
- "*prettamente semplice*" : Opinione personale
- "*Al centro, rispettivamente sia a destra che a sinistra, sono presenti due strade, che ti portano dall'altra parte (Se vai a [...]*" : Quello che descrivi non è la tecnologia per Schell: dove gira Pac-Man? Su che piattaforme è disponibile? Quali sono i controlli fisici del gioco?
- "*Brenda Romero & Ian Schreiber's:*" : [**!**] Questo apre il doc, non può essere a 11 di grandezza, deve essere rilevante per il lettore *E ancora:* Questa pagina è tutto wall of text, dal prossimo task non leggerò una formattazione simile
- "*Pac-Man*" : Al limite del testo inutile
- "*elementi fisici*" : Non seguono regole fisiche
- "*Infine le bacche speciali rendono il giocatore immune ai fantasmi, ed infatti questi risulteranno spaventati una volta che il [...]*" : Qui hai detto solo una qualità di Pac-Man nel concreto, il PvE. Ma ci sono altri elementi: - Arcade - Top-Down Third Person - Ripensaci
- "*Sta giocando al gioco un*" : Testo inutile
- "*E' un gioco da cabinato, nelle sale giochi Pac-Man era un gioco importantissimo, per tanto tutti riuscivano ad accedervi e a [...]*" : Non ci sono dati su chi gioca al di fuori della prima frase, qual è la fascia di età? La demografica dei giocatori? I dati?
- "*Gli elementi che danno un tema al gioco sono sicuramente*" : Tutto testo inutile
- "*potrebbero essere*" : Linguaggio debole

- "*L'obiettivo*" : Testo inutile
- "*Ci sono diverse regole imposte dal sistema di gioco.*" : Testo inutile, cita le regole invece di scrivere questo
- "*3 vite iniziali del giocatore, se vengono tolte, è game over.*" : Cosa non detta nell'altro documento. Manca in ogni caso l'outcome del game over: dove viene riportato il giocatore?
- "*liberamente nel mondo di gioco ma è principalmente fermato dai muri*" : Allora non è "libero". E se finisce in un "buco" ai limiti dello spazio fisico del gioco? Altra meccanica che manca. Come il comportamento diverso dei fantasmi
- "*Il giocatore può collezionare vari elementi, ma i principali, che rimangono, sono i frutti, e non solo.*" : Tutto da riformulare. Preciso. Puntuale. Essenziale
- "*punti*" : Altra risorsa
- "*punteggio*" : Altra risorsa
- "*Altri collezionabili sono le bacche speciali e le bacche normali, ma non sono permanenti.*" : Come lettore dovrei vedere questi sprite
- "*Se prendi l'ultima bacca rimanente nel livello, questo si resetterà.*" : Purtroppo mai citato prima *E ancora:* Invece lato input di gioco?

Alberto Maria Ronchi — *Tetris* (Schell)

Margini errati (ultimo avviso), meccanica confusa con strategia (con la riscrittura del docente), buona intuizione sulle core mechanic, definizione di arcade che si contraddice, pagina vuota finale (errore grave).

- "*ARCADE CORE ELEMENTS*" : [**!**] I margini del documento non sono quelli stabiliti, la prossima volta non leggo il task
- "*Tetris*" : Questo elemento è molto importante
- "*framework*" : Quale? Non ne esiste solo uno
- "*La meccanica base*" : Al limite del testo inutile
- "*(*" : Manca lo spazio
- "*senza lasciare spazi vuoti per completare le righe*" : Non è una meccanica questa, è una strategia
- "*una volta che una riga viene completata sparisce dando dei punti.*" : Questa è un'altra meccanica ancora *E ancora:* La core mechanic è spostare/ruotare il tetramino assegnato. Una volta posizionato, un nuovo tetramino viene generato e assegnato. Ci sono altre meccaniche: la UI che mostra il prossimo tetramino che verrà generato, il sistema di despawn delle righe (e quello di punteggio, che va spiegato), e così via. L'analisi presuppone un dettaglio a basso livello
- *(riga vuota o spazio)* : Spazio extra
- "*(*" : Manca lo spazio
- "*velocità della caduta dei tetramini*" : Ok, ma puoi essere più specifico per il futuro
- "*dall'alto*" : [+] Corretto, altra meccanica da spiegare
- *(riga vuota o spazio)* : Spazio extra
- "*ovvero quella di farlo prima che avvenga un contatto con altri tetramini.*" : [+] Corretto, ecco le tue core mechanic *E ancora:* Andavano detti tipo e vestizione artistica dei tetramini (magari questo punto non qui, ma nelle estetiche), però l'immagine è una aggiunta molto chiara e corretta
- "*punti*" : Sistema da spiegare, come quello dei livelli
- "*GAME OVER.*" : Cosa succede dopo il game over?
- *(riga vuota o spazio)* : Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato. Vale per ogni uso di spazio simile, per rompere il testo, nell'intero documento
- "*un conflitto infinito tra caos e ordine*" : Una tua lettura, alla quale posso pure venire incontro, ma servono i dati *E ancora:* Indubbiamente la cover art e la scocca del cabinato possono dare informazioni sulla narrativa di gioco, pensa a Space Invaders dove il tema è tutto "spiegato" lì
- "*Molti designer*" : Chi? Dove sono i dati?
- "*molto tempo*" : Generico

- "*comunque sulla semplicità.*" : Borderline da riformulare, puoi parlare anche della UI qui
- "*propri record o con altri giocatori.*" : Sistema di leaderboard da spiegare sopra, ma sono d'accordo
- "*la partita che può durare anche ore in base al livello di maestria del giocatore stesso.*" : Continui a non darmi informazioni estetiche sullo stile della colonna sonora, per esempio i BPM, e anche degli SFX
- "*La tecnologia*" : Testo inutile
- "*limitava il giocatore, perché non poteva giocarci dove voleva e quando voleva*" : Al limite della tua interpretazione dei fatti
- "*molte altre piattaforme.*" : Non "molte altre", "le seguenti", quelle che elenchi sotto. Sii puntuale
- "*versione portatile*" : Quale delle N versioni portatili? Puoi anche linkare una pagina se sono numerose le versioni
- "*numerosissime*" : Mancanza di precisione e puntualità, come commentato sopra *E ancora:* Spazio sprecato qui sotto, peccato perché questo paragrafo ha intuizioni interessanti lato formattazione
- "*LA MIA*" : Chiaro, il documento ha il tuo nome. Questa tab avrei dovuto leggerla prima, non dopo
- "*Per me*" : Testo inutile
- "*Anche se ora i cabinati sono tecnologia obsoleta.*" : Allora stai già confutando la tua stessa tesi
- "*con meccaniche semplici.*" : Questo non significa nulla preso da solo
- "*Alcuni esempi di arcade moderni sono:*" : [**!**] Continuo a non sapere il perché, in dettaglio, nonostante io possa essere d'accordo. Poi, Hades ha "meccaniche semplici"? UFO 50 ha letteralmente 50 titoli con meccaniche diverse, dovremmo prenderli singolarmente nel tuo esempio o considerali un unico titolo visto che un filo conduttore esiste, sia lato narrativo che meccanico *E ancora:* Qui sotto c'è una pagina intera di spazio sprecato, per il futuro è considerato un errore grave

Valerio Rossaro — *Dino Rex* (framework non dichiarato)

Primo dei due documenti consegnati. Tab iniziale sulla definizione di arcade senza titolo, documento non stampabile (ultimo avviso), wall of text su tutto il documento (ultimo avviso), testo giustificato, informazioni fondamentali chiuse tra parentesi.

- "*Che cos'è un gioco arcade?*" : Manca il titolo della tab
- "*facile da capire, con pochi comandi e tasti, e partite molto veloci.*" : Servono più informazioni specifiche a riguardo
- "." : Manca lo spazio
- "*spaziavano*" : Continui a parlare al passato, e nel presente?
- "*ecc*" : Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato. Vale per ogni uso di spazio simile, in tutte le tab dei documenti che hai caricato
- *(riga vuota o spazio)* : Spazio extra
- "*Space Invaders (1978, Taito) Ghost 'n Goblins (1985, Capcom) Ms. Pac-Man (1981, GCC Midway) Bomber Man World (1992, Irem) Street [...]*" : Non propriamente puntuale la relazione tra questi e le immagini sopra, ma è una scelta visiva sulla quale si può iterare lato formattazione
- "*DINO REX*" : [**!**] Che tipo di documento è? Analisi? Design? Che framework stai utilizzando? Mancano informazioni cruciali in apertura del documento *E ancora:* Inoltre il documento non è stampabile, con una consegna simile in futuro rischi la non lettura
- "*Dino Rex*" : Testo inutile, lo scrivi sopra
- "*Si contraddistingue per l'utilizzo della tecnica stop-motion per animare i dinosauri, digitalizzando in seguito i fotogrammi.*" : Avere un riassunto in capo al documento può essere una strategia che funziona, per dare al lettore info di partenza. Va ripensata la messa a terra, però *E ancora:* Sconsiglio caldamente il testo giustificato
- "*Come tutti i picchiaduro*" : Testo non propriamente utile
- "*round.*" : Come vengono segnate? Dove?
- "*Oltre alla possibilità di poter selezionare un dinosauro iniziale*" : Stai eclissando su questo punto, ma servono più informazioni: come funziona la selezione, la UI, il numero di dinosauri e cosa li distingue. Tutte meccaniche da spiegare

- "*(ogni incontro dura un massimo di 99 secondi, se il timer viene azzerato la vittoria viene assegnata al dinosauro con più salute)*" : Non può essere nella parentesi, questa spiegazione è fondamentale e la scribi anche correttamente *E ancora:* Stesso discorso per la Sudden Death
- "*colpi critici*" : Cosa significa? Che regole hanno?
- "*arena*" : Mai parlato dello spazio di gioco prima di adesso, a metà della prima pagina
- "*effetti diversi (guarigione, stordimento o fiamme).*" : Tutti da spiegare
- "*Il sistema di gioco mette a disposizione del giocatore una serie di comandi che può eseguire.*" : Puoi sintetizzare?
- "*Tramite il joystick può muoversi a destra e a sinistra, accovacciarsi, saltare o parare, mentre tramite i pulsanti può attaccare, [...]*" : [+] Assolutamente corretto, ma come puoi mostrarlo senza scriverlo così?
- "*rigidità*" : Non significa nulla così
- "*come interrompere un'animazione per parare o fuggire, oppure muoversi a mezz'aria. Anche pararsi o schivare durante l'animazione [...]*" : Tutto questo può tranquillamente essere messo a elenco puntato migliorandone la leggibilità
- "*La storia è ambientata in una preistoria alternativa dove esseri umani e dinosauri coesistono. Gli uomini lottano per ottenere [...]*" : [+] Sintentico, funziona
- "*tre parti che possiamo suddividere in:*" : Per ora lo accetto
- *(riga vuota o spazio)* : Spazio extra
- "*sanguinano copiosamente*" : In nessuna delle immagini che hai inserito c'è un rivolo di sangue, mancano le reference
- "*fotorealistica*" : Questo dipende dal contesto storico, detto adesso è falso
- "*assordante*" : Opinione personale, non hai portato né dati né reference
- "*Questa complessa combinazione tra modellazione fisica, stop-motion e compressione dei file permise di riprodurre l'effetto [...]*" : [!] Corretto, il problema risiede in come hai formattato il documento *E ancora:* Tutto il documento è un pericoloso wall of text con leggibilità ridotta, riflettici attentamente perché in futuro non accetterò una formattazione simile

Valerio Rossaro — *Punch-Out!!* (framework non dichiarato)

Secondo documento, con la stessa tab sulla definizione di arcade del primo ("mai fare leggere al lettore due volte la stessa cosa. Errore grave"). La lettura si interrompe sulla parola proibita, con il task punitivo su *Hatoful Boyfriend*.

- "*Che cos'è un gioco arcade?*" : [!] Commento la stessa tab nell'altro documento, dato che è la medesima. Mai fare leggere al lettore due volte la stessa cosa. Errore grave
- "*PUNCH-OUT!! (1984, Nintendo)*" : Stesso problema sul titolo di Dino Rex
- *(riga vuota o spazio)* : Problema sugli spazi commentato nell'altro documento che hai consegnato
- *(riga vuota o spazio)* : Same sul testo giustificato
- "*Punch-Out!!*" : Testo inutile
- "*videogioco arcade sportivo sulla boxe*" : [!] Questa formattazione è un grave wall of text. Non lo accetterò in futuro
- "*(dando poi vita al franchise con titoli sequel anche su console domestiche come NES, Super Nintendo e Wii)*" : Questo è un dettaglio utile all'analisi tramite core elements? Domandatelo
- "*mentre quello inferiore dedicato all'incontro che si sta giocando.*" : Perché non lo mostri?
- "*come una figura trasparente di spalle*" : Da vedere *E ancora:* Uguale per gli oggetti di gioco, lavora sulle immagini
- "*L'obiettivo del gioco*" : Da spiegare prima
- "*scalando la classifica*" : Come? Nel second-to-second gameplay, il giocatore, che fa?
- "*(azzerando così la sua barra della salute)*" : Ora so che ci sono, come sono fatte? Ce ne sono più di una? Di che altri elementi si compone la UI?
- "*Game Over immediato per il giocatore.*" : Cosa comporta il game over
- "*Il target di riferimento era un pubblico*" : Questo è un altro punto, argomento, paragrafo. Unire tutto insieme riduce drasticamente la leggibilità *E ancora:* L'immagine sotto non rispetta i margini, è un dettaglio assolutamente evitabile
- "*intrattenimento*" : [!] Inaccettabile, smetto di leggere questo documento. Come punizione devi produrre una analisi dei core elements di Schell su Hatoful Boyfriend, per lunedì 9 novembre

Lorenzo Vergata — *Jet Set Radio*, *Crazy Taxi 2*, *The Legend of Valkyrie* (Schell, Romero e Schreiber, Bura)

Il documento più lungo e con più commenti: ricerca ampia e corretta, ma in forma di saggio accademico in inglese (ultimo avviso), fuori dalla suite di Google (ultimo avviso), con definizioni troppo larghe, tono da recensore, pagine sprecate per battute e passaggi, extra framework prima di aver consolidato l'MVP.

- "*The Ingredients of Tasteful Arcade Games*" : [!] Il titolo appare più come una recensione piuttosto che un documento tecnico di analisi, inoltre è immotivatamente schiacciato a destra, in contrapposizione col testo *E ancora:* Non è un documento della suite di Google, lo leggo come prima e ultima eccezione, ma in futuro non lo accetterò *E ancora:* Posso dire già subito, ma ci ritornerò con altri commenti perché è estremamente importante e problematico, che questo non è un documento tecnico: è un wall of text con reference in coda, più simile ad una consegna accademica, e non è l'obiettivo di un Game Designer. Dal prossimo task, una consegna simile non la leggerò e non avrà i miei feedback
- *(riga vuota o spazio)* : Invece delle spaziature, sfrutta gli strumenti di Google come l'interlinea o lo spazio dopo il testo personalizzato. Vale per ogni uso di spazio simile, per rompere il testo, nell'intero documento
- "*Suggested playlist for the reading: Arcade Music! (Note: the word "arcade" is written too many times in this task).*" : [+] Soluzione interessante, ma discutibilmente fuori luogo per un documento di analisi. Al netto di tutto, la formattazione di questo consiglio va rivista *E ancora:* Sconsiglio caldamente il testo giustificato
- "*Defining terms in soft or inexact sciences is always a struggle. Given a certain description, it can be both correct and [...]*" : Io sono d'accordo con quello che scrivi, ma domandati come puoi essere più sintentico in apertura
- "*DiMarzio*" : Non comune avere i link rossi, ma borderline accettabile
- "*dynamics.*" : Inesatto, ci ritorneremo
- "*"soulful graphics"*" : Inquantificabile rispetto anche alle tecnologia usate oggi, in contrapposizione al tubo catodico ormai desueto. Questa è una lettura specifica di tecniche nate da limitazioni ormai non più vere
- "*as happens, for example, in the case of slot machines.*" : Altra questione che apre un abisso su interazione e loop di gioco, stai continuando a seminare elementi che però sono soffocati in pagine di testo skippabile per il lettore che vuole arrivare alla definizione (vuoi per praticità, vuoi per tempistiche)
- "*as happens with the same slot machines,*" : Legato principalmente alla posizione dei coin-op in passato
- "*If we talk through stereotypes, the arcade world is stereotypically male-oriented,*" : Cosa da evitare
- "*This isn't true anymore, because some years after those papers were published, we got cabinet games like Pang.*" : Quindi concretamente cosa è emerso da questo paragrafo di mezza pagina? Nessun punto critico che compone la tua ipotetica definizione finale. A livello pratico, come lettore, "ho perso tempo" (permettimi l'estremizzazione) esattamente come un artista leggendo le prime 20 pagine di una qualsiasi bibbia di design degli anni '90
- "*In the same paper, the authors outline three features in the genre:*" : Stai rompendo il discorso in due pagine, cosa da evitare assolutamente
- "*Just a digression: I consider myself a good Mario Kart gamer.*" : Opinione personale. Non può stare qui
- "*I mean*" : Nessun intercalare, non sei un recensore videoludico
- "*As said at the beginning of this work,*" : Allora è inutile se lo hai già detto
- "*Original form of gameplay. Arcade games contain the "prototype" of gameplay: short play sessions, simple intuitive controls, fast [...]*" : Per essere chiaro, questa ricerca porta tutti dati utili nella definizione ultima ma l'evidente problema è che restano dati "grezzi". Più precisamente, la fase di selezione e "spogliatura" del non essenziale non è sufficiente. Il risultato sono due pagine di testo saltabile e una definizione di una pagina che è un wall of text. In un documento tecnico non è accettabile
- "*(dynamics)*" : Termine inesatto perché ha una suo controparte più largamente accettata
- "*reducing randomness to the lowest possible levels.*" : Diresti che Vampire Survivors non è arcade?
- "*I would*" : Linguaggio debole
- "*derived from gambling games.*" : Questa associazione è inesatta, perché non deriva da gambling games ma, a monte, dallo sfruttare bias cognitivi per la manipolazione del giocatore (senza entrare in discorsi etici)
- "*Do we have to put in our dish we're cooking drugs? Certainly not, but doing so... Whatever, just stick with "know your audience tastes".*" : Commenti del genere non hanno spazio dentro documenti di analisi e/o design

- "*Ok, then FIFA is being played as an arcade game and may be considered, in that situation, an arcade game.*" : Quindi la ritualità di una macchina con elementi fisici/meccanici è una discriminante? Non dico che sia sbagliato, ma apre possibilità discutibili
- "*k*" : Manca il punto
- *(riga vuota o spazio)* : Spazio extra
- "*for me the most important of all*" : Tutto eliminabile, motivalo e basta
- "*j*" : Maiuscola
- "*A packaged state of mind that aims to give, granted a specific moment for the player(s), chosen experiences.*" : Secondo questa definizione, Journey è un arcade
- "*A juicy interaction infused with conflict.*" : Secondo questa definizione, Call of Duty è un arcade *E ancora:* Comprendi che un risultato simile non è, purtroppo, accettabile date le premesse
- "*Definitely not*" : Però secondo la "tua" definizione, Flower è "a packaged state of mind that aims to give, granted a specific moment for the player(s), chosen experiences."
- "*Ok, then. Excuse me for this long introduction to what may be an AG or, better, what they have. Now, we have to find the best [...]*" : Il punto è questo. Non è "scusabile", a maggior ragione considerando la mole di dati (corretta) dalla quale sei partito
- "*The perfect example*" : Opinione personale
- "*(no, ok, just kidding) For me, the first best example (let's say, third position) of an AG is...*" : Praticamente una pagina di spazio sprecato *E ancora:* Hai buttato una pagina per un joke: non è un pitch
- "*Long story short*" : Questo è proprio falso
- "*Graffiti points are marked by arrows and require paint*" : Dovrei vederlo, non leggerlo
- "*Police attacks drain your health, and red cans give it back.*" : Come si muove la polizia, che azioni ha a disposizione, che interazione può compiere o subire. Questo dovrei leggere ora, in maniera chiara. Puntuale. Formattata
- "*Skate, jump, grind, perform tricks, collect, spray, escape. The only weapon is a spray can.*" : Tutti questi verbi vanno spiegati nel dettaglio, qui, non altrove
- "*I would also say the analog stick of the Dreamcast controller: large tags require following on-screen stick commands, so the [...]*" : Mancano elementi chiave come ostacoli, NPC. Cosa popola il mondo di gioco?
- "*JSR has a linear and scripted narrative, served in small portions between levels by DJ Professor K, the voice of the pirate radio [...]*" : [+] Questo è corretto, preciso, succinto. Parti da questo
- "*The GG's fight three rival gangs (the Love Shockers, the Noise Tanks, and Poison Jam) for the streets of Tokyo-to, while Captain [...]*" : Tutto questo va riassunto con 5W+H
- "*Ryuta Ueda, cited Fight Club among his influences, together with the pop-culture approach of PaRappa the Rapper, and wanted "a [...]*" : Non stai parlando di narrativa, stai parlando di estetiche qui
- "*Look. Thick outlines on the objects that must stand out, flat bright colors through a cel-shading technique developed in-house. [...]*" : [+] Corretto, ma le reference?
- "*Jet Set Radio*" : Testo inutile, ma questo errore è presente più volte di quante io ne possa realisticamente commentare
- "*The arcade recipe check Going back to our ingredients:*" : Come avresti potuto rendere questo una matrice/tabella visivamente chiara e compatta nel doc?
- "*Using the vocabulary seen in class, Jet Set Radio, for me, is oriented towards four emotions:*" : Questo è l'assunto, tutto evitabile
- "*Auditory/visual stimulation.*" : Nessuna spiegazione, nemmeno per challenge?
- *(riga vuota o spazio)* : Spazio extra
- "*Let's continue with the next game (second position), from the same console.*" : Per fare questo perdi una pagina di spazio completamente sprecato
- "*Romero-Schreiber's Framework Applied to Crazy Taxi 2*" : Personalmente contento di vedere lavoro extra, ma prima di espandere la consegna la "rule of thumb" è iterare sull'MVP. Quello che guadagni in contenuto lo perdi in forma (e non solo), il rischio (e il risultato qui) è un documento non tecnico dalla ridotta leggibilità, che un domani nessuno leggerà veramente. O peggio, lo leggeranno dimenticandosi cosa implementare o cosa era importante *E ancora:* Il framework resta quello dei core elements, da dire

- "*The first Crazy Taxi*" : Hai usato una pagina intera di immagini per farlo capire al lettore, non è necessario riscriverlo
- "*Long story short*" : Ancora una volta, questo è falso
- "*The sequel adds the Crazy Hop, a dedicated button that fires the hydraulics of the cab and makes it jump over cars and onto [...]*" : Però tu stai parlando del primo, perché aggiungere informazioni non rilevanti?
- "*Stunts (tricks).*" : Ad alto livello, questo è un combo system
- "*Time attack/score attack, single-player only, with asynchronous competition through online scoreboards.*" : Queste sono più qualità messe insieme, eppure non info sulle 3C del gioco
- "*This is a game playable in short sessions that explores the daily routine of a taxi driver. The player is playing to beat their [...]*" : No. Non ci sono dati sulle demografiche: etnia, genere, segmenti di età, etc.
- "*There is no real story:*" : Questo non è mai vero, capisco quello che intendi ma va riformulato
- "*Earn as much money as possible before the main timer reaches zero. In the Crazy Pyramid, the goal changes: clear the technique [...]*" : [+] Corretto, poi eventualmente puoi approfondire nelle regole
- "*The player cannot:*" : E il "can"?
- *(riga vuota o spazio)* : Spazio extra. Ricordo la regola dei 5 errori grammaticali
- "*The cab and its driver: Slash is the all-rounder, Iceman focuses on top speed, Cinnamon on acceleration, and Hot-D on handling (Giant Bomb).*" : Qui urge fare una precisazione: può il giocatore modificare auto e guidatore? Dove?
- "*Knowledge of the city: shortcuts and rooftops are spent and earned like everything else.*" : Non è una risorsa in gioco, fa parte del modello mentale
- "*Interaction*" : E gli input lato giocatore/macchina?
- "→" : [+] Interessante, iteraci sopra
- "*(first position in my heart)*" : Per quanto mi duole dirtelo, è testo inutile qui
- "*Here we are finally in front of a cabinet, though I played it on the backward-compatible PS2. The Legend of Valkyrie was released [...]*" : Tutte queste sono introduzioni eccessivamente lunghe che possono essere spacchettate in paragrafi specifici, come per esempio meccaniche o obiettivo (prendendo come riferimento Schell)
- "*In a 1991 Gamest reader poll, it was voted the best arcade game of all time, and I agree with this statement if played with [...]*" : Testo inutile. Tutto
- "*Freedom Mastery Data*" : Mai, e dico mai, rompere una tabella in più pagine
- "*shops turn gold into spells and weapons, NPCs turn bubbles into spells, prices rise*" : Sni, configurazione di ciascun quadro e conoscenza (preventiva) degli ostacoli
- "*the audience behind the cabinet or in the room*" : [+] Interessante, mi trovi d'accordo
- "*hold Attack to cast the magic you bought.*" : Ci sono anche queste situazione legate al gameplay a basso livello
- "*but always within the limits of the game.*" : Questo è l'assunto
- "*A friend can drop a coin and join, and in the arcade there is always an audience behind you (performativity of Kum and Kim).*" : [+] Corretto
- "*The Mastery column*" : Sempre sul discorso formattazione e, manchevole, applicazione prova a pensare: il lettore è qui, sta leggendo la riga e vuole vedere dove era nella matrice di Bura. Per come hai formattato, deve continuamente fare su e giù, potenzialmente dimenticandosi pezzi nel mentre. A questo non sempre c'è una soluzione ideale, o non è sempre necessario che ci sia, però sono dettagli sui quali devi pensare adesso. Perché dedicare tempo e attenzione ora ti renderà un GD poi, discutibilmente in tempi brevi
- "*Ethics and personal sensitivity. That is, did I really beat the game if I fed the machine with coins to continue?*" : [+] Corretto
- "*(blasphemous)*" : Inutile e fuori luogo
- "*hourglass itself.*" : Questa è la cosa più importante, ed è in fondo all'elenco
- *(riga vuota o spazio)* : Spazio extra
- "*Mementos, achievements, memories.*" : [+] Ok, examples
- "*That's it for now. Bye! :)*" : Ahimè, testo inutile [emoticon]
- "*References Bellemare, Marc G., et al. 'The Arcade Learning Environment: An Evaluation Platform for General Agents'. Journal of [...]*" : [!] Eliminabile, me li hai linkati dove era necessario *E ancora:* Sotto rimane una pagina di spazio sprecato, errore grave da non ripetere, pena la non lettura

1.4.12 Punti chiave del task T1

- Margini, Roboto 11, Google Docs, stampabilità e colori non sono dettagli: dal task successivo il loro mancato rispetto costa la **non lettura**.
- Lo spazio è oro: niente pagine o mezze pagine vuote, niente righe vuote al posto dell'interlinea, niente testo giustificato, niente tabelle spezzate.
- Niente **wall of text**: un paragrafo, un argomento; elenchi e tabelle quando i dati sono paralleli o confrontabili.
- Un documento di game design è un **documento tecnico**: non una recensione, non un saggio accademico. Niente battute, intercalari, opinioni, linguaggio debole, bibliografia in coda.
- Ogni frase deve aggiungere informazione: il "testo inutile" è l'errore più frequente in assoluto.
- Ogni aggettivo va **misurato**: numeri, confronti, elenchi completi.
- La parola proibita interrompe la lettura e porta un task punitivo; anche le parole che le girano intorno sono a rischio.
- Si dichiara **il framework** con il suo nome preciso, e si mette ogni informazione nella voce giusta, senza citare nulla che non sia stato introdotto.
- Le meccaniche si descrivono **a basso livello**: input, stati, parametri numerici, collisioni, esiti (game over, cambio di livello), partendo dall'obiettivo.
- Narrativa in 5W+H, con il suo tipo (lineare, ramificata); estetiche con palette, stile, UI, audio e reference; tecnologia come hardware, piattaforme e controlli, allora e oggi.
- Pubblico ed emozioni vanno **motivati**: dati demografici, elementi di gioco che producono l'emozione.
- Una definizione si mette alla prova con i **controesempi** prima di consegnarla, e risponde alle domande che pone.
- Prima si itera sull'**MVP**, poi si aggiunge l'extra.

1.5 Task 2 — Production dictionary di un Pokémon

1.5.1 La consegna

Il task era stato assegnato a metà della lezione su problema e soluzione (capitolo 4, sezione 4.14), subito dopo la spiegazione del production dictionary (sezione 4.3):

1. scegliere un gioco **Pokémon della saga principale** (*mainline*), niente spin-off; a Daniele era stato assegnato invece un *Final Fantasy* con numero inferiore a 10;
2. scriverne il **production dictionary**: i termini unici o fraintendibili che chiunque lavori al gioco (artista, programmatore, sound designer, outsourcer) deve conoscere, ciascuno con una descrizione succinta di **che cosa è** (non di come funziona nel dettaglio, che spetta ai documenti specifici);
3. in un documento di Google, con le regole formali del corso (sezione 3.9), nominato e caricato secondo le regole del Drive (sezione 3.2);
4. con un vincolo preciso: **non superare una pagina**. Il limite serviva a far ragionare sulla **priorità** dei termini.

La consegna era stata ripresa due volte a lezione: per chiarire che il limite di una pagina è un vincolo dell'esercizio e non delle produzioni reali, dove stare il dizionario nella documentazione e chi ne sceglie i termini (sezione 5.1); e per chiarire la differenza tra *nomenclatura* e *spiegazione di meccanica*, e tra ordine alfabetico e raggruppamento (sezione 6.1).

Era il **secondo task** del corso, e il primo dopo la revisione degli arcade. Il docente lo aveva detto chiaramente nei commenti al primo task: molte cose erano state "accettate per ora". Questa volta non lo sono state. Le revisioni sono del 5 novembre 2026 (con due risposte del docente il giorno dopo, nel dialogo con Alberto).

1.5.2 Le consegne in sintesi

Table 1.5: Le consegne del task T2.

Studente	Gioco	Esito	Il nodo principale
Francesca Attadia	*Pokémon Perla* (2006)	letto	Giudizio positivo: titolo, distinzione tra termini generali e specifici, accorpamenti; alcune voci da completare
Antonino Etiopia	*Pokémon Spada e Scudo*	letto, al limite	Maiuscole di "pokémon" incoerenti ("al limite del non letto"); grassetto e sottolineato senza logica, di nuovo
Federica Ferrara	*Pokémon Nero e Bianco*	**non letto**	Quasi due pagine invece di una
Mattia Maestri	*Pokémon X e Y* (2013)	**non letto**	Nome del file con nome e cognome invertiti; margini non rispettati
Daniele Maracino	*Final Fantasy VII*	letto, ultimo avviso	Titolo-immagine che occupa un quarto della pagina; voci troppo stringate con spazio ancora libero
Francesco Muratore	*Pokémon Rosso Fuoco* (2004)	**non letto**	Testo inutile in ogni voce (il termine ripetuto come soggetto della definizione)
Alberto Ronchi	terza generazione (titolo non indicato)	letto, al limite	Titolo; elenco a tre livelli; spazi e refusi ("al limite del non letto"); definizioni incomplete
Valerio Rossaro	*Pokémon SoulSilver* (2009), in inglese	letto, ultimo avviso	Elenco puntato "in zona grigia" (non più accettato); termini usati e mai spiegati
Lorenzo Vergata	*Pokémon Ultra Sun & Ultra Moon*, in inglese	letto	Formattazione "ottimale"; definizioni sature di termini non spiegati; reference in coda, di nuovo

Il dato che salta all'occhio è l'esito: **tre documenti su nove non sono stati letti**, tutti per ragioni formali e tutti con un unico commento sul titolo. Nel primo task la non lettura era capitata una volta sola. Gli altri errori ricadono in cinque famiglie, nell'ordine di frequenza:

1. **termini complessi usati nelle definizioni e mai spiegati**: è l'errore di contenuto più diffuso, il corrispettivo del "mai parlato prima" del primo task;
2. **definizioni incomplete o vaghe**: chi, quanti, dove, a che costo, con che sistema;
3. **categoria sbagliata**: il termine viene definito come la cosa sbagliata (una "meccanica" che è una statistica, uno "scenario" che è un edificio);

4. **priorità e spazio**: righe libere lasciate vuote mentre le voci restano stringate, oppure spazio speso su voci secondarie;

5. **forma**: titolo, elenchi puntati che sprecano spazio a destra, righe vuote, maiuscole e refusi.

Il testo inutile, re assoluto del primo task, è sceso di frequenza: ma dove c'è, costa il documento intero.

1.5.3 I documenti non letti

I tre casi hanno una cosa in comune: il docente li ha fermati **al titolo**, con un solo commento, senza entrare nel merito. È l'applicazione pratica di ciò che nel primo task era stato annunciato come "ultimo avviso" (sezione 1.2).

Il limite di pagine. Federica ha consegnato un dizionario di *Pokémon Nero e Bianco* lungo quasi due pagine:

> "Il limite della consegna era di una pagina, ne hai prodotte quasi due. Non leggo."

Il limite non era un dettaglio: era **il cuore dell'esercizio**. Il docente lo aveva spiegato assegnando il task: una pagina sola costringe a scegliere quali termini spiegare per primi, perché più fraintendibili o più necessari (sezione 4.14). Sforare significa non aver fatto quella scelta. Si noti che il documento, al suo interno, conteneva voci molto specifiche (*Antigelo*, *Bulbo*, *Minorball*, *Scuroball*...), cioè proprio il tipo di voci che una selezione per priorità avrebbe tagliato per prime.

Il nome del file. Mattia ha nominato il documento con *Nome_Cognome* invece di *Cognome_Nome*:

> "Il documento non è nominato con Cognome_Nome_TitoloTask come da regole, hai invertito nome e cognome. Anche se li hai impostati correttamente, i margini non sono stati rispettati. Non leggo."

Due errori formali insieme: il nome del file (che serve a trovare e ordinare i documenti nella cartella del task, sezione 3.2) e i margini, verosimilmente impostati nelle opzioni ma non rispettati dal contenuto. Il secondo punto è sottile e vale per tutti: **impostare** i margini non basta, se poi un elemento (un'immagine, una tabella, un titolo) li sfora. Il nome del file, tra l'altro, era invertito già nel primo task (*Mattia_Maestri_ArcadeCoreElementals*), senza che il docente lo segnalasse: un errore non commentato non è un errore accettato.

Il testo inutile. Francesco ha costruito ogni voce del suo dizionario di *Pokémon Rosso Fuoco* ripetendo il termine come soggetto della definizione:

> *Pokédex*: Il Pokédex è uno strumento donato al protagonista [...]
> *Palestra*: Le palestre nel mondo di gioco non sono le stesse del mondo reale, ma sono dei luoghi [...]
> *Capopalestra*: Il Capopalestra è l'allenatore più forte [...]

Il docente segna "testo inutile" dodici volte (una sul sottotitolo) e una tredicesima "al limite del testo inutile"; poi, sul titolo, chiude: "Non leggo data la quantità di testo inutile presente". In un dizionario la ripetizione è ancora più evidente che in un'analisi: il termine è già in grassetto all'inizio della riga, e la definizione comincia dopo i due punti. Lo stesso documento si apriva con un sottotitolo che ripeteva il titolo ("Production Dictionary di: Pokémon Rosso Fuoco": "testo inutile, lo scrivi sopra"). Si confronti con il primo task (sezione 1.4.4), dove lo stesso errore era stato segnalato su tutti i documenti senza conseguenze: era il primo task.

La lezione. Una regola su cui si è ricevuto un avviso non dà diritto a un secondo avviso. E soprattutto: un documento non letto non riceve **nessun feedback** sul contenuto. Chi non è stato letto non sa se le sue definizioni funzionavano; deve ricavarlo dai commenti agli altri. È il motivo per cui le schede di questo capitolo vanno lette tutte.

1.5.4 Errori di forma e di impaginazione

Il titolo. In un documento di una pagina, il titolo è la prima voce di spesa dello spazio. I commenti:

- **troppo grande**: Daniele apre con un'immagine in stile *Final Fantasy VII* che contiene il titolo. "Nonostante il titolo sia esteticamente appagante e in tema con il gioco, ti 'mangia' almeno 1/4 del documento. Prima e ultima volta che leggo una consegna con un problema simile."
- **ripetuto**: un sottotitolo che ridice il titolo ("Production Dictionary" sotto "PokémonProductionDictionary", Alberto; "Production Dictionary di: Pokémon Rosso Fuoco", Francesco): "testo inutile, lo scrivi sopra";
- **incompleto**: Alberto indica "della terza generazione" invece del gioco: "ho chiesto un titolo, non una generazione". Una generazione comprende più giochi, e il dizionario è di un gioco;
- **senza spazi**: "PokémonProductionDictionary": "perché senza spaziature?" (Alberto);
- **decorato**: Valerio affianca al titolo uno sprite di un personaggio: "lo sprite sulla destra è una decorazione e va eliminata". E nel titolo c'è un errore: "POKÈMON", con l'accento grave invece dell'acuto ("errore grammaticale");
- **con la maiuscola sbagliata**: "Production dictionary" (Antonino): "avrei messo la maiuscola sulla 'D' ";
- positivo: "Titolo essenziale, nulla da dire" (Francesca, *PRODUCTION DICTIONARY — POKÉMON VERSIONE PERLA - (2006)*; Lorenzo, *Pokémon Ultra Sun & Ultra Moon: Production Dictionary*). Tipo di documento, gioco ed eventualmente anno: basta.

Elenchi puntati e spazio a destra. È il tema di forma nuovo di questo task. Tre documenti usano un elenco puntato per le voci, e il docente riconosce ogni volta il vantaggio ma ne indica il costo:

> "L'elenco puntato aumenta sicuramente la leggibilità, ma al costo di spazio perso sulla destra. Trade-off accettabile? Da discutere." (Daniele)
> "[...] ma al costo di tanto spazio perso (soprattutto per il sotto elenco del sotto elenco) sulla destra." (Alberto)
> "L'elenco puntato aumenta sicuramente la leggibilità e hai lavorato sul ridurre lo spazio perso a destra, cosa indubbiamente apprezzabile. Questo però è, nel miglior caso possibile, in una zona grigia delle regole. Lo accetto ora data la natura del documento, non lo accetterò per task futuri (al netto di modifiche alle regole)." (Valerio)

Il meccanismo è questo: ogni livello di rientro sposta il testo a destra, ma la riga finisce sempre allo stesso margine; quando le voci sono brevi, la riga si spezza presto e la parte destra della pagina resta bianca. Con tre livelli (Alberto: categoria, sottocategoria, voce) lo spreco si moltiplica. Si ricordi che nel primo task Antonino aveva ricevuto l'avvertimento sugli elenchi rientrati tre volte (sezione 1.4.3): è la stessa famiglia di errore. Per Valerio, che pure aveva già ridotto lo spreco, l'avviso è esplicito: dal prossimo task un elenco simile non verrà accettato.

Due colonne: lo stesso trade-off. Lorenzo ha impaginato il dizionario su **due colonne**, con le categorie in maiuscolo e a colori. Il docente ne apprezza il risultato complessivo ("formattato con scelte ottimali sia per forma che per chiarezza delle categorie, che risaltano subito all'occhio"), ma segnala il prezzo: "il trade-off della scelta di formattazione adottata ti obbliga ad avere un testo più compatto, che spesso cade verticalmente. Compromesso discutibilmente accettabile, chiaramente per definizioni più lunghe (ma corrette) come questa è molto visibile". In colonne strette una definizione di quattro righe ne occupa otto. E le colonne hanno lasciato spazio vuoto in fondo: "spazio sprecato, insieme a quello perso a destra non è poco".

Gli strumenti di Google, di nuovo. A Francesca e a Lorenzo, su una riga vuota usata per staccare il titolo dal testo: "Solita questione sullo sfruttare quanto più possibile tutti gli strumenti di Google Docs". È il commento del primo task sulle righe vuote al posto dell'interlinea e dello spazio dopo il paragrafo (sezione 1.4.3), ormai abbreviato in "solita questione". Ad Alberto, tre volte: "spazio extra".

Le reference in coda, di nuovo. Lorenzo chiude il documento con tre reference (Bulbapedia, Nintendo UK, Serebii):

> "Come commentato per il documento precedente, anzi qui diventa estremamente necessario, sono da eliminare. Con questo ti sei giocato molto spazio sprecato, insieme a quello perso a sinistra."

Nel primo task la bibliografia finale era già stata definita "eliminabile" (sezione 1.4.3). In un documento di una pagina diventa "estremamente necessario" toglierla: le righe usate per le fonti sono righe tolte alle definizioni. E sulla voce *Rotom Dex* il docente lo dice in modo diretto: "la definizione è satura di termini che avresti potuto inserire qui sotto. Invece ci sono le reference."

Grassetto, sottolineato, colore: con una logica. Ad Antonino il docente rivolge, per la seconda volta, la stessa domanda del primo task: "Non comprendo la differenza tra come hai usato il grassetto e come invece hai usato la sottolineatura. Ho bisogno di delucidazioni da parte tua." Francesca usa invece il sottolineato con una logica (i termini che hanno una voce propria nel dizionario), e il docente la mette alla prova: "L'underline è legato a termini trattati nel documento? Perché se è così, tu parli di 'Tipo elementale' che capisco sia la stessa cosa, ma potrebbe non essere comprensibile per un lettore disattento". Se un segno grafico rimanda a una voce, il termine sottolineato deve essere **identico** al lemma. Ad Alberto, che colora i termini per categoria, un suggerimento: "per rafforzare maggiormente il legame, avresti potuto richiamare il codice colore utilizzato negli stessi termini sopra (blu, in questo caso)". Il colore, come il sottolineato, funziona solo se è usato **ovunque** con lo stesso significato.

Maiuscole e refusi: la soglia. In questo task due documenti arrivano "al limite del non letto" per errori grammaticali:

- Antonino scrive "pokémon" a volte minuscolo e a volte maiuscolo: "Grammatica, o tutti minuscoli (strano, ma lo accetterei) o tutti maiuscoli. Mai un misto"; e subito dopo: "Sei al limite del non letto";
- Alberto accumula spazi mancanti prima delle parentesi (tre volte), maiuscole incoerenti ("Perché gli altri allenatori li citi con l'iniziale maiuscola e questo con la minuscola?"), uno spazio dentro "capi palestra": "Sei al limite del non letto per numero di errori grammaticali accumulati".

È la regola dei cinque errori grammaticali (sezione 1.2) che si avvicina. Il caso di Antonino insegna una cosa in più: la **coerenza** conta più della scelta. Anche una scelta strana (tutto minuscolo) è accettabile, purché sia sempre la stessa.

Ordine e categorie. Il tema era stato discusso a lezione (sezione 6.1): l'ordine alfabetico ha la massima leggibilità "nel vuoto", il raggruppamento tiene vicini i termini legati. I commenti confermano che entrambe le strade sono possibili, ma vanno usate con criterio:

- Valerio ordina alfabeticamente: "Ordine alfabetico? Ideale per Production Dictionary 'completi', qui è meno efficace". Su una pagina sola, con una ventina di voci, l'alfabeto allontana termini che si spiegano a vicenda (*Professor* e lo *starter* che consegna, *Pokémon League* e *Gym Leader*);
- Francesca divide in "Termini generali — saga Pokémon" e "Termini specifici — Pokémon versione Perla": "distinzione interessante, funziona";
- Antonino divide in "Termini comuni", "Oggetti", "Peculiarità del gioco": funziona, "però qui serve più dettaglio: 'comuni' a cosa?";
- Lorenzo usa categorie (*Battle*, *Progression*, *World & Ultra Space*, *Systems & Factions*), apprezzate; ma una voce (*Team Rainbow Rocket*) finisce lontana dalla sua intestazione: "perché questa sezione non è direttamente sotto a quella di 'SYSTEMS & FACTIONS'?".

1.5.5 Errori di scrittura e di linguaggio

Testo inutile: il lemma ripetuto. Oltre al caso di Francesco (sezione 1.5.3), la stessa forma compare altrove: "Livello: Livello del Pokémon" ("al limite del testo inutile"); "Evoluzione: Processo di evoluzione Pokémon" ("testo inutile"); "Rival: The rival of the player" ("quasi testo inutile"). E in un caso il testo da togliere è una frase narrativa: "Near a park, a big green area to capture Pokémon, we found the Pokéathlon Dome": "tutto eliminabile". *Il test*: la definizione inizia dopo i due punti, e la prima parola deve già dire **che cos'è** il termine ("consumabile", "edificio", "statistica", "NPC").

Parole vaghe. "Strumenti: sono oggetti utilizzati in vari campi": "sezioni di gioco/gameplay diversi". "Le Poké Ball [...] ne esistono differenti tipologie con caratteristiche diverse": "principalmente su cosa hanno effetto queste 'caratteristiche'?". "Poffin: dolcetti speciali preparati usando le Bacche": "come?". Ogni parola generica ("campi", "caratteristiche", "preparati", "species") è una domanda che il lettore non può fare.

I qualificatori precisi. Molti commenti del docente sono una sola parola, che corregge la definizione aggiungendo il qualificatore mancante:

- *radio* → "in-game" (Valerio: la radio è un oggetto del mondo di gioco, non una radio reale);
- *four islands* → "explorable" (Lorenzo);
- *(Ethan)* → "non-customizable sprites" (Valerio: il protagonista si sceglie tra due, ma non si personalizza);
- *starter Pokémon* → "one from three initial Pokémon chosen by the player" (Valerio);
- *villainous team* → "composto da allenatori" (Valerio);
- *currency* → "main/principale": "una descrizione così stringata funziona solo se citi il tipo di currency" (Valerio);
- *Pokémon Market* → "specific items" (Valerio);
- *Centro Pokémon: scenario* → "edificio" (Antonino).

Sono esempi preziosi perché mostrano quanto poco basta, a volte, per passare da una definizione ambigua a una corretta.

Nomi "user-generated". Due volte il docente nota che il termine scelto come lemma non è il nome ufficiale del gioco ma quello della community: *IV* (Francesca) e *Starter* (Antonino): "User-generated name, ma comprendo". Un production dictionary documenta i **nomi del progetto**; quando si usa un nome nato tra i giocatori, conviene saperlo (ed eventualmente indicare anche quello ufficiale).

Il vocabolario del design. Come nel primo task, il docente traduce alcuni termini nel lessico del corso: il *Chaining* del Poké Radar "è proprio un sistema basato su combo"; le *Virtù* delle Super Gare "anche queste sono statistiche, tra l'altro su un grafico a radar".

L'inglese è benvenuto. Valerio apre il suo documento con un commento: ha scritto in inglese dopo averne parlato al tutoring, e chiede se sia un problema. La risposta: "Certo, fa solo che bene anche perché nella stragrande maggioranza dei casi anche le aziende italiane fanno documentazione in inglese." Si noti la differenza con il primo task: lì il problema del documento di Lorenzo non era l'inglese, ma la forma accademica.

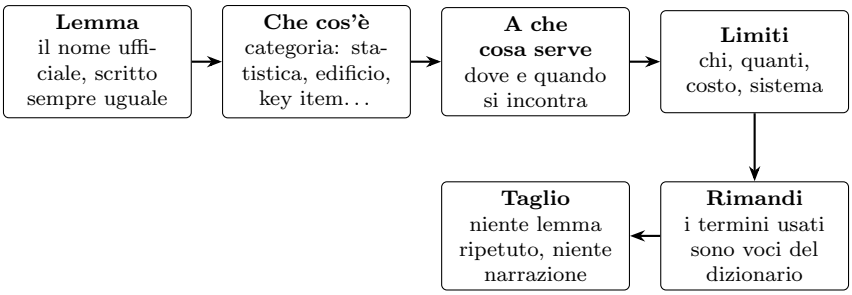


Figure 1.2: Anatomia di una voce di production dictionary, ricostruita dai commenti del docente al task T2 (proposta del curatore, non uno schema mostrato a lezione).

1.5.6 La voce del dizionario: che cosa deve contenere

Il production dictionary dice **a che cosa si riferisce una parola**, in una o due righe, per un lettore di qualunque disciplina (sezione 6.1). I commenti del task mostrano che cosa serve, concretamente, perché una voce raggiunga questo scopo.

Ogni termine complesso usato va spiegato. È il commento di contenuto più frequente del task, e il docente lo formula come regola a Valerio: "Se citi un termine complesso o unico in una descrizione per un Production Dictionary, poi va spiegato." I casi:

- *Gym*, *Gym Badge*: "non so cosa sia" (Valerio, nella voce *Gym Leader*); *Trainer*, usato ovunque e mai definito (Valerio);
- *statblock*: "se usi un termine simile nelle descrizioni, poi lo devi spiegare" (Antonino); *PP*: "va spiegato prima che questo è l'acronimo" (Antonino); *MP*: "hai spazio per spiegare l'acronimo" (Daniele);
- *Sinnoh* (Francesca), *leggendari*, *Pokémon selvatici*, *assegnabile* ("borderline da spiegare", Alberto e Antonino); *Speed Course, Power Course...*: "tutti termini complessi" (Valerio);
- nel documento di Lorenzo, la catena più lunga: *move*, *type*, *HM* ("da spiegare"), *IVs, Hidden Ability, Shiny, Adrenaline Orb* ("qui ti sei cacciato in un bel ginepraio. Hai chiamato in causa (joke not intended) una serie di termini unici che necessitano poi una loro descrizione"), *Dusk Mane* e *Dawn Wings Necrozma* e la Mega Evoluzione ("il lettore può non sapere chi siano"), *Beast Boost* ("termine complesso"), *Ultra Megalopolis* ("una catena di termini complessi"), *Rotom* e lo stesso *Pokédex* ("anche 'Pokédex' è un termine complesso").

È il gemello dell'errore del primo task "mai parlato di questo nelle meccaniche" (sezione 1.4.6): in un'analisi non si cita ciò che non è stato introdotto; in un dizionario non si usa, nella definizione, una parola che a sua volta richiederebbe il dizionario. *Correzione*: rileggere ogni definizione sottolineando i termini unici del gioco; ciascuno deve avere **una sua voce** (meglio se vicina) o essere sostituito da una parola comune. Se non c'è spazio per la voce, il termine va tolto dalla definizione.

Dire che cos'è: la categoria giusta. La prima informazione di una voce è la **natura** della cosa definita, e diversi commenti la correggono:

- *Natura* definita "meccanica di gioco": "a conti fatti è una statistica, un tag" (Francesca);
- *Centro Pokémon*, *Pokémon market*, *Palestra* definiti "scenario": "edificio" (Antonino);
- *Z-Power Ring* definito "an item": "questo è un key item" (Lorenzo);
- *HM* e *TM* definiti "item": "manca solo il discorso consumabile e non consumabile" (Valerio); allo stesso modo, ad Alberto: "esiste un'altra differenza principale tra MT e MN che non hai scritto";

- *Dynamax* definito “un’abilità”: “da attivare e valida solo in lotta” (Antonino);
- *WEAPON*, i mostri di *Final Fantasy VII*: “cita la questione ‘nemico’ o ‘ostacolo’ e ci siamo” (Daniele);
- *Capopalestra* “boss” e *Captain*: “Pokémon? Umano?” (Antonino, Lorenzo).

La categoria è ciò che permette all’artista, al programmatore o al sound designer di collocare il termine: un *key item* non si consuma e non si vende, un edificio si costruisce nel level design, una statistica sta nei dati di bilanciamento.

Le domande minime. Come nel primo task, le domande del docente formano una checklist. Per un dizionario sono più brevi, ma altrettanto sistematiche:

Chi? “Danni: a chi?”; “held item: da chi?”; “boss: Pokémon? Umano?”; “villainous team: composto da allenatori”.

Quanti? “Starter: quanti sono?”; “hai spazio per citare il numero di tipi presenti”; “probabilità: puoi citare gli estremi”.

Dove e quando? “Utilizzi massimi delle mosse: dove? In che momento di gioco?”; “Gil: usata dove?”; “Consumabili: dove stanno?”.

A che costo? “Curare: costa?” (Alberto e Valerio, sul Centro Pokémon).

Con che sistema? “Lotta in cui i pokémon [...]: non basta: è un combattimento reattivo in tempo reale? A turni?” (Alberto); “all against all: con che sistema? Combattimento a turni, va detto” (Lorenzo); “corretto, ma va spiegato il sistema di combattimento” (Valerio). In tre documenti su sei letti manca l’informazione più basilare del genere: la lotta è **a turni**.

Come? “Allenatori: NPC da sconfiggere: come?”; “Pokémon League [...] defeat the Elite Four and the Champion: come?”; “Poffin preparati: come?”.

E se...? “PS: unità di misura dell’energia vitale: se va a zero che succede?” (Francesca); “Pensione Pokémon [...] possono generare uova: da specificare meglio i limiti della meccanica” (Alberto).

A che cosa porta? “Forze/debolezze: a cosa possono portare”; “PokéKron: puoi citare alcuni strumenti come la calcolatrice, oppure fare riferimento alla loro utilità nel loop di gioco”; “Champion: hai sufficiente spazio per dire che è l’ultimo ostacolo per arrivare al finale”.

Attenzione: non si tratta di spiegare la meccanica nel dettaglio (quello spetta al documento specifico, sezione 6.1), ma di dare al lettore **il minimo per non fraintendere**. “A turni” è un’informazione da dizionario; la formula del danno no.

Esattezza: niente generalizzazioni false. Una definizione breve tende a generalizzare, e il docente trova i controesempi, come per le definizioni di arcade del primo task (sezione 1.4.5):

- “Pokédex: glossario contenente tutte le specie”: “inesatto, non in partenza e non con le stesse info” (Antonino); “a digital encyclopedia with all the information”: “in partenza?” (Valerio);
- “nei percorsi sarà possibile catturare”: “eppure c’è un Eevee in ‘regalo’ in una città in FR/LG, come c’è un Beldum a Verdeazzupoli” (Alberto);
- “Pokédollari: per acquistare strumenti”: “non solo per quello”; “ogni pokémon ha uno o più tipi”: “non solo”; “capi palestra”: “solo loro?”; “Pokémon Market: luogo in cui acquistare gli strumenti”: “tutti?” (Alberto);
- “Pietra evolutiva: causa l’immediata evoluzione”: “ne esistono di più tipi adatti a specifici Pokémon” (Antonino);

- "That Pokémon can be chosen from Grass, Fire and Water-type": "per come lo formuli sembra che possa sceglierlo tra tutti i Pokémon di quel tipo" (Valerio);
- "Mossa Fisica: dipende dalla statistica di Attacco": "sto pensando se nella quarta generazione ci sono eccezioni a riguardo" (Francesca). Il docente non risponde: segnala che una regola generale va verificata.[3]

Coerenza interna. Il dizionario è uno: due voci non possono dire cose diverse. "KO: raggiunto quando i PS arrivano a zero": "prima dici una cosa simile ma non uguale" (Antonino, che nella voce *Punti salute* aveva scritto "fuori combattimento"). "Gigamax: una variante [...] con potenza molto maggiore": "da riformulare, soprattutto rispetto a quello che hai scritto sotto 'Dynamax' ". E ancora ad Antonino: "Caramella rara: aumenta l'esperienza": "sotto la tua descrizione di livello non citi l'esperienza associata". Se un termine compare in due voci, deve significare la stessa cosa in entrambe.

Esempi che non tutti conoscono. Ad Antonino, che definisce l'Attacco "principalmente da danni fisici" (pensando a eccezioni come *Body Press*): "Può essere confusionaria questa spiegazione. Il lettore può non avere in mente un Body Press, per esempio." Un'eccezione che il lettore non conosce confonde invece di chiarire.

Numeri: meglio averli. Due studenti inseriscono dati numerici precisi: la probabilità di uno shiny ("1 ogni 8.192", Alberto) e i moltiplicatori della Beast Ball ("catch rate x5 on UBs, x0.1 on anything else", Lorenzo). Il docente aggiunge una prospettiva di produzione: "Potresti non avere questi valori come finali in un dato momento della produzione, soprattutto all'inizio quando è più probabile che si metta in piedi un Production Dictionary. Ma meglio averli che non averli." Il dizionario nasce presto e si aggiorna ("uno per progetto", sezione 5.1): i numeri vanno messi, sapendo che cambieranno.

Prima e dopo. La tabella 1.6 riporta quattro voci riscritte seguendo i commenti (proposte del curatore, non del docente).
Si noti che ogni riscrittura risponde a un commento: per Francesco toglie il lemma ripetuto e la frase sulla "piena libertà" (che non dice nulla), e precisa "in partenza" (il Pokédex non contiene tutto dall'inizio); per Antonino riequilibra lo spazio ("le Pokéball restano con una descrizione vaga"); per Valerio sposta il peso dall'evoluzione al Pokémon e definisce *Trainer* e il sistema di lotta; per Lorenzo spezza una voce satura in tre voci brevi, usando lo spazio occupato dalle reference.

1.5.7 Priorità: che cosa entra in una pagina

Il limite di una pagina chiedeva di scegliere. I commenti rivelano due modi opposti di non farlo.

Sforare. Federica ha superato il limite e non è stata letta (sezione 1.5.3).

Lasciare spazio libero con voci troppo corte. È l'errore opposto, ed è il commento più ripetuto del task: "hai spazio".

- "Hai una riga di spazio per ulteriori spiegazioni", "non solo qui", "si può dire di più sul sistema e hai spazio per farlo", "hai spazio per spiegare l'acronimo", "usata dove? Hai spazio" (Daniele, a cui il docente aggiunge: "voglio stressare particolarmente il fatto che tu stia scampando la non lettura per una piuma di Chocobo");
- "Hai spazio per citare il numero di tipi presenti", "puoi citare alcuni strumenti [...], hai spazio per farlo" (Francesca);

[3] Nota del curatore: la quarta generazione, a cui appartiene *Perla*, è quella che introduce la distinzione tra mosse fisiche e speciali per singola mossa (prima la categoria dipendeva dal tipo). Le eccezioni più note alla regola, come *Foul Play* o *Body Press*, che usano statistiche diverse dall'Attacco di chi le usa, arrivano in generazioni successive; già nella quarta esistono però mosse fisiche a danno fisso, come *Seismic Toss*, che dall'Attacco non dipendono. È lo stesso tipo di eccezione citato ad Antonino con *Body Press*.

Table 1.6: Prima e dopo: quattro voci di production dictionary riscritte seguendo i commenti del docente (proposta del curatore).

Nella consegna	**Riscrittura proposta**
Pokédex: Il Pokédex è uno strumento donato al protagonista per registrare tutti i Pokémon presenti nel mondo di gioco. All'interno di Pokémon: Rosso Fuoco sono presenti 151 Pokémon diversi e il giocatore ha la piena di libertà di registrarne quanti più possibile. (Francesco)	*Pokédex*: key item ricevuto a inizio avventura. Registra ogni specie *vista* (dati parziali) e *catturata* (scheda completa). Pokédex di Kanto: 151 specie.
Pokéball: Consumabile che permette di catturare un Pokémon selvatico. (Antonino)	*Pokéball*: consumabile lanciato in lotta contro un Pokémon selvatico per catturarlo; la probabilità cresce con PS bassi e problemi di stato. Più tipi con bonus diversi (es. Megaball, Ultraball). Inutilizzabile contro Pokémon di altri allenatori.
Pokémon: Mysterious creatures that live alongside humans, share adventures with them and challenge Trainers in Pokémon Battle. Pokémon can also evolve, a mysterious process that transforms [. . .] (Valerio)	*Pokémon*: creatures of the game world. The player catches them, keeps up to six in the party and uses them in turn-based battles against wild Pokémon and Trainers. Can evolve (see *Evolution*).
Rotom Dex: Pokédex inhabited by Rotom, with an affection value (0-1,000). Its Roto Loto hands out temporary Rotom Powers (e.g., Roto Catch); at 601 affection, Z-Rotom Power allows a second Z-Move in one battle. (Lorenzo)	*Rotom*: Electric/Ghost Pokémon that can inhabit electronic devices. *Pokédex*: key item that records seen and caught species. *Rotom Dex*: the player's Pokédex, inhabited by a Rotom; also shows the map. Grants *Rotom Powers* (see below).

- "Come? Hai spazio per espandere" (Alberto);
- "Hai sufficiente spazio per dire che è l'ultimo ostacolo", "furbo averli tutti qui, allora sfrutta lo spazio che hai guadagnato" (Valerio);
- "Capitoli precedenti, hai spazio per esplicitarlo" (Lorenzo, su *Sun/Moon*).

Un dizionario di una pagina in cui l'ultima riga di ogni voce è corta e la pagina non è piena non ha usato il limite: lo ha subito. Lo spazio, a lezione, era "oro" (sezione 3.7): vale anche lo spazio *dentro* le righe già occupate.

Spendere spazio sulle voci sbagliate. Il terzo modo è distribuire male lo spazio:

- Antonino elenca tutte le statistiche, una per voce: "Inserire tutte le statistiche ti ha obbligato ad avere descrizioni più succinte e meno dettagliate. Un trade-off discutibile"; e dedica "molta presenza a oggetti consumabili di ripristino e cura, ma le Pokéball restano con una descrizione vaga"; e poi "avrei voluto leggere più key item";
- Valerio dedica alla voce *Pokémon* "più del doppio dello testo" all'evoluzione: "avresti potuto espandere sul concetto di battaglia e 'Trainer', dato che quest'ultimo termine non viene spiegato nel dizionario"; nella voce *Move* accorpa "tante, discutibilmente troppe, informazioni in un'unica descrizione";
- Lorenzo usa le righe finali per le reference invece che per le voci che la sua stessa definizione di *Rotom Dex* richiedeva.

Gli accorpamenti furbi. Al contrario, il docente premia chi risparmia spazio raggruppando voci simili in una sola:

- “Rimedi/Strumenti Curativi”: “inserimento furbo per salvare spazio”, e subito dopo “uguale sotto”, cioè per la voce successiva (Francesca);
- “Statistiche Su: vari strumenti [...] (HP Su, Proteina, Ferro, Calcio, Zinco)”: “accorpamento furbo per salvare spazio” (Alberto);
- “Stats: [...] (Stamina, Attack, Special Attack, Defence, Special Defence and Speed)”: “furbo averli tutti qui” (Valerio), con il rilancio già citato: lo spazio guadagnato va usato.

Il confronto con Antonino è istruttivo: le stesse sei statistiche, in sei voci separate, hanno tolto spazio alle definizioni; in una voce sola, ne hanno liberato.

La regola, riformulata. Il criterio dato a lezione era: si spiegano le **nature uniche del gioco**, oppure i termini **fraintendibili** (sezione 6.1). Le revisioni aggiungono un corollario: una volta scelti i termini, **ogni riga della pagina deve lavorare**. Se avanza spazio, va alle definizioni più povere; se manca, si accorpa o si taglia la voce meno prioritaria, mai la chiarezza delle altre.

1.5.8 Che cosa è stato apprezzato

- il **titolo essenziale** (Francesca, Lorenzo): “nulla da dire”;
- la **distinzione** tra termini della saga e termini del singolo gioco (Francesca), e le **categorie** che “risaltano subito all’occhio” (Lorenzo);
- gli **accorpamenti** che salvano spazio (Francesca, Alberto, Valerio);
- le definizioni **chiare** o corrette: *Pokédex* (“chiaro”), *Evoluzione* e *MT* (“corretto”), la differenza tra IV ed EV (“aggiunta corretta”), il *Chaining* (“aggiunta corretta, questo è proprio un sistema basato su combo”) (Francesca); *Totem Pokémon* (“chiaro”), il *Version offset* (“aggiunta interessante”), *Ultra Warp Ride* (“ci siamo”), il *Poké Ride* che sostituisce le MN (“aggiunta corretta”) (Lorenzo); “compatible Pokémon” (“assolutamente corretto”), le regioni ispirate al Giappone reale (“aggiunta corretta”) (Valerio); *Rotomina* (“aggiunta corretta del termine”, Antonino);
- la scelta di scrivere **in inglese** (Valerio, Lorenzo);
- il **dialogo** nei commenti: Alberto risponde a due commenti spiegando le sue scelte, e il docente in un caso gli dà ragione (“Vero, allora aggiungi l’impossibilità di uscire dall’arena una volta ingaggiata e ci siamo”), nell’altro precisa (“però lo scopo generico va riassunto qui”). È esattamente l’uso dei commenti previsto dopo la revisione (sezione 3.2);
- il giudizio d’insieme su Lorenzo: “il documento è formattato con scelte ottimali sia per forma che per chiarezza delle categorie [...]. Un interessante primo tentativo di Production Dictionary”.

1.5.9 Dal task 1 al task 2: confronto e pattern ricorrenti

Due task sono già abbastanza per vedere come si muovono gli errori della classe. La tabella 1.7 confronta le famiglie di errori; la tabella 1.8 segue i singoli studenti.

Gli errori che persistono. Tre errori sono passati dal primo al secondo task **nelle stesse persone**, nonostante il commento ricevuto: la logica di grassetto e sottolineato (Antonino), la bibliografia in coda (Lorenzo), le righe vuote al posto degli strumenti di Google (“solita questione”, Francesca e Lorenzo). Sono errori di *abitudine*: non si correggono capendo il commento, ma cambiando il modo di lavorare (impostare gli stili di paragrafo prima di scrivere, decidere all’inizio che cosa significa il grassetto, non aprire una sezione “References”).

Table 1.7: Le famiglie di errori nei due task.

Famiglia	**Task T1**	**Task T2**
Conseguenze	1 lettura interrotta (parola proibita); molti ultimi avvisi	**3 documenti non letti** su 9, 4 letti con ultimo avviso o "al limite"
Margini e font	4 documenti su 8	1 documento, non letto
Testo inutile	su tutti i documenti, decine di volte	concentrato: un documento non letto, poche occorrenze altrove
Wall of text, tono da recensore, opinioni	frequenti	**scomparsi** (anche per la natura del documento)
Parola proibita	1 caso, task punitivo	nessun caso
Righe vuote e spazio sprecato	su quasi tutti i documenti	ancora presenti ("solita questione"), meno gravi
Elenchi e rientri	elenchi rientrati tre volte (Antonino)	elenchi puntati che sprecano spazio a destra (3 documenti), uno "in zona grigia"
Bibliografia in coda	Lorenzo	Lorenzo, di nuovo
Logica di grassetto e sottolineato	Antonino	Antonino, di nuovo
Errori grammaticali	richiamo alla regola dei 5 errori	2 documenti "al limite del non letto"
Citare ciò che non è introdotto	"mai parlato prima" nelle analisi	termini complessi usati nelle definizioni: **primo errore di contenuto**
Dettaglio	meccaniche "ad alto livello"	definizioni senza chi, quanti, dove, sistema
Categoria / voce sbagliata	informazione nella voce sbagliata del framework	termine definito come la cosa sbagliata (statistica, edificio, key item)
Nuovi	—	nome del file, limite di pagine, titolo che occupa spazio

Table 1.8: L'evoluzione dei singoli studenti dal task T1 al task T2.

Studente	**Task T1**	**Task T2**
Francesca Attadia	"Interessante primo tentativo"; meccaniche poco dettagliate	Giudizio positivo; resta la "solita questione" delle righe vuote; alcune voci da completare
Antonino Etiopia	Elenchi rientrati; grassetto, corsivo e sottolineato senza logica	**Ancora** grassetto e sottolineato senza logica; maiuscole incoerenti, "al limite del non letto"
Federica Ferrara	Margini errati (ultimo avviso); spazi extra	Limite di pagine superato: **non letto**
Mattia Maestri	Margini, font, rosso RGB; framework incompleto	Nome del file invertito (lo era già in T1) e margini: **non letto**
Daniele Maracino	(nessuna consegna nel T1)	Titolo-immagine: ultimo avviso; molte voci con spazio inutilizzato
Francesco Muratore	Margini, wall of text, linguaggio debole, testo inutile	Testo inutile in ogni voce: **non letto**
Alberto Ronchi	Margini; meccanica confusa con strategia	Titolo; elenco a tre livelli; refusi "al limite del non letto"; buon dialogo nei commenti
Valerio Rossaro	Wall of text; tab ripetuta; parola proibita	Nessun wall of text; inglese apprezzato; elenco "in zona grigia" (ultimo avviso); termini non spiegati
Lorenzo Vergata	Saggio accademico, fuori dalla suite Google, bibliografia in coda	Google Docs, titolo essenziale, formattazione "ottimale"; reference in coda **di nuovo**; definizioni sature di termini

Gli errori che cambiano forma. Altri errori non sono spariti: si sono adattati al nuovo tipo di documento.

- Lo spazio sprecato a sinistra degli elenchi rientrati (T1) diventa lo spazio sprecato *a destra* degli elenchi puntati e delle colonne (T2).
- Il "mai parlato prima" delle analisi (T1) diventa il termine complesso usato in una definizione e mai definito (T2).
- Le meccaniche descritte "ad alto livello" (T1) diventano le definizioni senza chi, quanti, dove e con che sistema (T2). Le domande del docente sono, nei due task, sorprendentemente simili: "quanti e quali", "come?", "cosa succede se va a zero?".
- L'informazione nella voce sbagliata del framework (T1) diventa il termine definito con la categoria sbagliata (T2).

Il pattern di fondo è unico: **il documento presuppone conoscenze che il lettore non ha**. Nel primo task il lettore non sapeva che cosa succede premendo il tasto; nel secondo non sa che cos'è un *Gym Badge*. In entrambi i casi chi scrive conosce il gioco e dimentica di non essere il destinatario.

Gli errori risolti. Wall of text, tono da recensore, opinioni personali, parole deboli e parola proibita non compaiono più. In parte è merito della forma del documento (un dizionario si presta poco al racconto), ma i casi più evidenti del primo task (Valerio, Lorenzo, Francesco sul wall of text) sono stati corretti davvero. Lo stesso vale per la suite di Google (Lorenzo) e per margini e font, con un solo caso segnalato su nove documenti.

La scala si è spostata. Nel primo task il livello più frequente era il primo (errore da correggere), e la non lettura un'eccezione. Nel secondo, tre consegne si sono fermate al titolo e quattro sono arrivate a un avviso esplicito. Gli avvisi del primo task erano promesse, e sono state mantenute; è ragionevole aspettarsi che quelli del secondo (titoli, elenchi, refusi) lo siano al terzo. Il registro degli avvertimenti (sezione 1.2) è aggiornato di conseguenza.

1.5.10 Le schede delle singole consegne

Seguono, documento per documento, **tutti** i commenti del docente (173 in tutto), nell'ordine in cui compaiono nel documento, ciascuno con il punto a cui è ancorato. Le risposte degli studenti sono riportate dove presenti (*Commento di Nome:*), seguite dalla replica del docente. Per la legenda si veda la sezione 1.1. Gli ancoraggi di una sola lettera ("p", "c", "(") indicano il carattere esatto evidenziato dal docente, quasi sempre per una maiuscola o uno spazio.

Francesca Attadia — *Pokémon Perla*

Il documento meglio accolto del task, con molte conferme ("corretto", "chiaro", "aggiunta corretta") e due accorpamenti premiati. Le richieste riguardano soprattutto il completamento di alcune voci (che cosa fanno le "caratteristiche", che cosa succede quando i PS arrivano a zero, come si preparano i Poffin) e un termine non spiegato (*Sinnoh*).

- "*PRODUCTION DICTIONARY POKÉMON VERSIONE PERLA - (2006)*" : [+] Titolo essenziale, nulla da dire
- *(riga vuota o spazio)* : Solita questione sullo sfruttare quanto più possibile tutti gli strumenti di Google Docs
- "*Termini generali – saga Pokémon*" : [+] Distinzione interessante, funziona
- "*livellare*" : Peculiare ma comprensibile
- "*Tipi*" : L'underline è legato a termini trattati nel documento? Perché se è così, tu parli di "Tipo elementale" che capisco sia la stessa cosa, ma potrebbe non essere comprensibile per un lettore disattento
- "*Evoluzione Pokémon*" : [+] Corretto

- "*debolezze e immunità uniche per tipo.*" : Hai spazio per citare il numero di tipi presenti o, ancora meglio, queste forze/debolezze a cosa possono portare
- "*Pokédex = Dispositivo che registra le informazioni dei Pokémon sfidati, catturati o incontrati dal giocatore.*" : [+] Chiaro
- "*caratteristiche diverse.*" : Principalmente su cosa hanno effetto queste "caratteristiche"?
- "*Meccanica di gioco*" : A conti fatti è una statistica, un tag
- "*IV*" : User-generated name, ma comprendo. Vale anche sotto
- "*A differenza degli IV, gli EV si accumulano attivamente.*" : [+] Aggiunta corretta
- "*Unità di misura dell'energia vitale di un Pokémon.*" : Se va a zero che succede?
- "*Rimedi/Strumenti Curativi*" : [+] Inserimento furbo per salvare spazio *E ancora:* Uguale sotto
- "*statistica di Attacco*" : Sto pensando se nella quarta generazione ci sono eccezioni a riguardo
- "*monouso*" : [+] Corretto
- "*del giocatore.*" : Puoi citare alcuni strumenti come la calcolatrice, oppure fare riferimento alla loro utilità nel loop di gioco, hai spazio per farlo
- "*caratteristiche*" : Anche queste sono statistiche, tra l'altro su un grafico a radar
- "*preparati*" : Come?
- "*Sinnoh*" : Da spiegare
- "*Chaining*" : [+] Aggiunta corretta, questo è proprio un sistema basato su combo

Antonino Etiopia — *Pokémon Spada e Scudo*

Struttura per categorie apprezzata, ma con molti punti da precisare (categorie, "chi", "quanti", coerenza tra voci) e una serie di errori sulle maiuscole di "pokémon" che porta il documento "al limite del non letto". Per la seconda volta il docente chiede la logica di grassetto e sottolineato.

- "*d*" : Avrei messo la maiuscola sulla "D"
- "*Termini comuni:*" : Dividere il documento a seconda delle categorie generali dei termini funziona, però qui serve più dettaglio: "comuni" a cosa?
- "*statblock*" : Se usi un termine simile nelle descrizioni, poi lo devi spiegare
- "*Starter:*" : User-generated name, ma comprendo *E ancora:* Quanti sono?
- "*Livello del Pokémon*" : Al limite del testo inutile
- "*Processo di evoluzione*" : Testo inutile
- "*consuma punti potenza*" : Non comprendo la differenza tra come hai usato il grassetto e come invece hai usato la sottolineatura. Ho bisogno di delucidazioni da parte tua
- "*danni*" : A chi?
- "*probabilità*" : Puoi citare gli estremi
- "*PP*" : Va spiegato prima che questo è l'acronimo
- "*principalmente da danni fisici.*" : Può essere confusionaria questa spiegazione. Il lettore può non avere in mente un Body Press, per esempio
- "*Velocità:*" : Inserire tutte le statistiche ti ha obbligato ad avere descrizioni più succinte e meno dettagliate. Un trade-off discutibile
- "*raggiunto quando i PS arrivano a zero.*" : Prima dici una cosa simile ma non uguale
- "*tutte le specie di Pokémon presenti nel gioco.*" : Inesatto, non in partenza e non con le stesse info
- "*Scenario*" : Edificio *E ancora:* Anche sotto
- "*boss*" : Pokémon? Umano?
- "*Consumabile che permette di catturare un Pokémon selvatico.*" : Hai dato molta presenza a oggetti consumabili di ripristino e cura, ma le Pokéball restano con una descrizione vaga
- "*l'immediata evoluzione*" : Ne esistono di più tipi adatti a specifici Pokémon
- "*esperienza*" : Sotto la tua descrizione di livello non citi l'esperienza associata

- "*assegnabile*" : Borderline da spiegare
- "*Pokémon selvatici*" : Borderline da spiegare
- "*Pokémon*" : Specifico
- "*p*" : Maiuscola
- "*Oggetto chiave*" : Avrei voluto leggere più key item
- "*Un'abilità*" : Da attivare e valida solo in lotta
- "*Una variante di pokémon con aspetto diverso e potenza molto maggiore alla norma.*" : Da riformulare, soprattutto rispetto a quello che hai scritto sotto "Dynamax". Inoltre c'è un errore grammaticale
- "*p*" : Grammatica, anche sulla "G" dopo
- "*p*" : Grammatica
- "*p*" : [**!**] Grammatica, o tutti minuscoli (strano, ma lo accetterei) o tutti maiuscoli. Mai un misto *E ancora:* Sei al limite del non letto
- "*funzionalità aggiuntive e opzionali.*" : Da spiegare, ma aggiunta corretta del termine "Rotomina"

Federica Ferrara — *Pokémon Nero e Bianco*

Non letto: il documento supera il limite di una pagina.

- "*Production Dictionary Pokémon Nero/Bianco*" : [**!**] Il limite della consegna era di una pagina, ne hai prodotte quasi due. Non leggo

Mattia Maestri — *Pokémon X e Y*

Non letto: nome del file non conforme alla regola *Cognome_Nome_TitoloTask* e margini non rispettati.

- "*PRODUCTION DICTIONARY*" : [**!**] Il documento non è nominato con Cognome_Nome_TitoloTask come da regole, hai invertito nome e cognome. Anche se li hai impostati correttamente, i margini non sono stati rispettati. Non leggo

Daniele Maracino — *Final Fantasy VII*

Unico documento non su Pokémon, per assegnazione del docente. Ultimo avviso sul titolo-immagine; le voci sono corrette ma troppo stringate, mentre lo spazio per completarle c'era ("stai scampando la non lettura per una piuma di Chocobo").

- *(riga vuota o spazio)*: [**!**] Nonostante il titolo sia esteticamente appagante e in tema con il gioco, ti "mangia" almeno 1/4 del documento. Prima e ultima volta che leggo una consegna con un problema simile
- "*Pianeta:*" : L'elenco puntato aumenta sicuramente la leggibilità, ma al costo di spazio perso sulla destra. Trade-off accettabile? Da discutere
- "*suoi abitanti.*" : Hai una riga di spazio per ulteriori spiegazioni *E ancora:* Non solo qui
- "*proprietà equipaggiabili dai personaggi.*" : Anche qui, si può dire di più sul sistema e hai spazio per farlo
- "*WEAPON: Giganteschi mostri bio-meccanici creati dal Pianeta per proteggersi.*" : Cita la questione "nemico" o "ostacolo" e ci siamo
- "*Misura di quante magie può lanciare un personaggio.*" : Hai spazio per spiegare l'acronimo
- "*Valuta del gioco.*" : [**!**] Usata dove? Hai spazio. Voglio stressare particolarmente il fatto che tu stia scampando la non lettura per una piuma di Chocobo

Francesco Muratore — *Pokémon Rosso Fuoco*

Non letto per la quantità di testo inutile: tutti i commenti successivi al primo sono segnalazioni di testo inutile, lasciate dal docente prima della decisione finale.

- "*Pokémon Rosso Fuoco (2004)*" : [**!**] Non leggo data la quantità di testo inutile presente
- "*Pokémon Rosso Fuoco*" : Testo inutile, lo scrivi sopra
- "*Un pokémon*" : Testo inutile

- "*I centri Pokémon*" : Testo inutile
- "*Il Pokédex*" : Testo inutile
- "*La parola ''Tipo''*" : Testo inutile
- "*Le palestre*" : Testo inutile
- "*Il Capopalestra*" : Testo inutile
- "*Il team rocket*" : Testo inutile
- "*Il rivale*" : Testo inutile
- "*Nello zaino*" : Al limite del testo inutile, ma la frase va riformulata
- "*Le ''MT''*" : Testo inutile
- "*I super 4*" : Testo inutile
- "*Il campione*" : Testo inutile

Alberto Ronchi — terza generazione

Il documento con più commenti, molti dei quali brevi domande sulle voci ("costa?", "tutti?", "come?"). Titolo da rivedere, elenco a tre livelli che spreca spazio, refusi "al limite del non letto"; ma anche un dialogo costruttivo con il docente.

- "*Production*" : Perché senza spaziature?
- "*Production Dictionary*" : Testo inutile, lo scrivi sopra
- "*terza generazione*" : Ho chiesto un titolo, non una generazione
- "*Pokémon:*" : L'elenco puntato aumenta sicuramente la leggibilità, ma al costo di tanto spazio perso (soprattutto per il sotto elenco del sotto elenco) sulla destra. Trade-off accettabile? Da discutere
- "*p*" : Discutibilmente con la maiuscola, ma al limite del de gustibus
- "*(1 ogni 8.192)*" : Questo dato potresti non averlo a inizio produzione, ma comprendo
- "*campi*" : Sezioni di gioco/gameplay diversi
- "*(*" : Manca lo spazio prima della parentesi
- "*(*" : Manca lo spazio prima della parentesi
- "*superare ostacoli ambientali.*" : Esiste un'altra differenza principale tra MT e MN che non hai scritto
- "*scopo preciso.*" : Dove stanno? *Commento di Alberto:* Sono elencati sotto alcuni consumabili con il proprio scopo *Il docente:* Però lo scopo generico va riassunto qui
- "*utilizzi massimi delle mosse*" : Dove? In che momento di gioco?
- "*Statistiche Su:*" : [+] Accorpamento furbo per salvare spazio
- "*(*" : Manca lo spazio prima della parentesi
- "*Pozioni: usate per curare i pokémon. Antidoti: usati per curare un'alterazione di stato dei pokémon.*" : Da spiegare più chiaramente le differenze
- *(riga vuota o spazio)* : Spazio extra
- "*per acquistare strumenti.*" : Non solo per quello
- "*c*" : Perché gli altri allenatori li citi con l'iniziale maiuscola e questo con la minuscola?
- *(riga vuota o spazio)* : Spazio extra
- "*sconfiggere*" : Come? Hai spazio per espandere
- "*leggendari*" : Borderline da spiegare
- "*lotta in cui i pokémon*" : Non basta: è un combattimento reattivo in tempo reale? A turni?
- *(riga vuota o spazio)* : Spazio extra
- "*ogni pokémon*" : Non solo
- "*catturare*" : Eppure c'è un Eevee in "regalo" in una città in FR/LG, come c'è un Beldum a Verdeazzupoli e così via
- "*curare*" : Costa?

- "*strumenti*" : Tutti?
- "*arena contenente i Superquattro e il campione*" : Come vanno affrontati? *Commento di Alberto:* Se intendi che vanno affrontati in successione lo avevo già scritto quando li ho spiegati sopra. / Intendi altro? *Il docente:* Vero, allora aggiungi l'impossibilità di uscire dall'arena una volta ingaggiata e ci siamo *E ancora:* In ogni caso, per rafforzare maggiormente il legame, avresti potuto richiamare il codice colore utilizzato negli stessi termini sopra (blu, in questo caso)
- "*capi palestra*" : [**!**] Solo loro? Inoltre c'è uno spazio in mezzo che non dovrebbe esserci *E ancora:* Sei al limite del non letto per numero di errori grammaticali accumulati
- "*dove possono generare uova*" : Da specificare meglio i limiti della meccanica

Valerio Rossaro — *Pokémon SoulSilver*

Primo documento in inglese, accolto con favore. Ultimo avviso sull'elenco puntato; molti termini usati nelle definizioni e mai spiegati (*Gym*, *Gym Badge*, *Trainer*); ordine alfabetico poco efficace su una pagina.

- "*Production Dictionary*" : *Commento di Valerio:* Facendo la lezione di Tutoring con Jacopo Silvestri, abbiamo parlato di che lingua è meglio usare per i progetti. Mi ha detto che non è richiesto per il primo anno fare progetti in inglese, ma se si comincia non fa sicuramente male. / Spero quindi non sia un problema se ci ho provato. *Il docente:* [+] Certo, fa solo che bene anche perché nella stragrande maggioranza dei casi anche le aziende italiane fanno documentazione in inglese
- "*POKÈMON*" : Errore grammaticale *E ancora:* Lo sprite sulla destra è una decorazione e va eliminata
- "*Champion:*" : Ordine alfabetico? Ideale per Production Dictionary "completi", qui è meno efficace
- "*most powerful Pokémon Trainer in the region.*" : Hai sufficiente spazio per dire che è l'ultimo ostacolo per arrivare al finale
- "*Elite Four:*" : [**!**] L'elenco puntato aumenta sicuramente la leggibilità e hai lavorato sul ridurre lo spazio perso a destra, cosa indubbiamente apprezzabile. Questo però è, nel miglior caso possibile, in una zona grigia delle regole. Lo accetto ora data la natura del documento, non lo accetterò per task futuri (al netto di modifiche alle regole)
- "*Gym*" : Non so cosa sia
- "*Gym Badge*" : Anche questo non so cosa sia. Se citi un termine complesso o unico in una descrizione per un Production Dictionary, poi va spiegato
- "*item*" : Manca solo il discorso consumabile e non consumabile
- "*compatible Pokémon*" : [+] Assolutamente corretto
- "*inside and outside of battle*" : Corretto, ma va spiegato il sistema di combattimento
- "*based on the real-world Kansai region of Japan*" : [+] Aggiunta corretta, come sotto
- "*Move:*" : Qui stai accorpando tante, discutibilmente troppe, informazioni in un'unica descrizione
- "*A mechanic that randomly assigns a higher and a lower statistic.*" : Puoi approfondire, ripensaci
- "*Near a park, a big green area to capture Pokémon, we found the Pokéathlon Dome*" : Tutto eliminabile
- "*Speed Course, Power Course, Skill Course, Stamina Course and Jump Course*" : Tutti termini complessi
- "*with all the information*" : In partenza?
- "*currency*" : Lo vedremo, ma una descrizione così stringata funziona solo se citi il tipo di currency, main/principale in questo caso
- "*Pokémon can also evolve, a mysterious process that transforms a certain type of Pokémon into a new and completely different creature when [. . .]*" : Più del doppio dello testo spiega l'evoluzione, non i Pokémon. Avresti potuto espandere sul concetto di battaglia e "Trainer", dato che quest'ultimo termine non viene spiegato nel dizionario
- "*fully restore*" : Costa?
- "*radio*" : In-game
- "*A specific location where the player can buy and sell items using Poké Dollars.*" : "Specific items" ma corretto
- "*defeat the Elite Four and the Champion.*" : Come?
- "*A physical, out-of-game step counter used to transfer a Pokémon outside the game and walk with it.*" : Qui ti sei scelto un termine tosto: va spiegato meglio il sistema e il supporto fisico associato

- "*starter Pokémon*" : One from three initial Pokémon chosen by the player
- "*Grass, Fire and Water-type.*" : Ok ma per come lo formuli sembra che possa sceglierlo tra tutti i Pokémon di quel tipo
- "*(Ethan)*" : Non-customizable sprites
- "*The rival of the player*" : Quasi testo inutile
- "*(Stamina, Attack, Special Attack, Defence, Special Defence and Speed)*" : Furbo averli tutti qui, allora sfrutta lo spazio che hai guadagnato e parla sinteticamente di queste statistiche
- "*villainous team*" : Composto da allenatori
- "*An item used to teach a compatible Pokémon a move.*" : Stesso discorso commentato sopra per le HM
- "*An elemental synergy of a Pokémon with specific elements*" : Ok la lista dei tag ma servono più informazioni sulla meccanica

Lorenzo Vergata — *Pokémon Ultra Sun & Ultra Moon*

Giudicato "un interessante primo tentativo di Production Dictionary", formattato "con scelte ottimali". I limiti: definizioni dense di termini unici non spiegati ("un bel ginepraio"), il trade-off delle due colonne, spazio sprecato in fondo e le reference in coda, già segnalate nel primo task.

- "*Pokémon Ultra Sun & Ultra Moon: Production Dictionary*" : [+] Titolo essenziale, nulla da dire
- *(riga vuota o spazio)* : Solita questione sullo sfruttare quanto più possibile tutti gli strumenti di Google Docs
- "*move*" : Borderline da spiegare
- "*Connected to the Z-Power Ring + Z-Crystal.*" : Qui il trade-off della scelta di formattazione adottata ti obbliga ad avere un testo più compatto, che spesso cade verticalmente. Compromesso discutibilmente accettabile, chiaramente per definizioni più lunghe (ma corrette) come questa è molto visibile
- "*held item*" : Da chi?
- "*type*" : Da spiegare
- "*item*" : Questo è un key item
- "*Sun/Moon*" : Capitoli precedenti, hai spazio per esplicitarlo
- "*IVs (hidden innate stats), Hidden Ability, and Shiny; the Adrenaline Orb makes calls likelier.*" : Qui ti sei cacciato in un bel ginepraio. Hai chiamato in causa (joke not intended) una serie di termini unici che necessitano poi una loro descrizione
- "*Totem Pokémon*" : [+] Chiaro
- "*Dusk Mane or Dawn Wings Necrozma*" : Il lettore può non sapere chi siano, né conoscere la meccanica di Mega Evoluzione
- "*all against all*" : Con che sistema? Combattimento a turni, va detto
- "*Alola*" : Citare la regione di riferimento, che comprende le isole che elenchi poi
- "*four islands*" : Explorable
- "*Captain*" : Umano? Pokémon?
- "*This is the Gym Leader equivalent.*" : Puoi tagliare, spiegando che questo Trial è in chiusura dell'isola esplorata e necessario per la progressione di gioco
- "*Version (clock) offset*" : [+] Aggiunta interessante
- "*species*" : Di cosa?
- "*Ability Beast Boost*" : Termine complesso, da spiegare
- "*catch rate x5 on UBs, x0.1 on anything else*" : Potresti non avere questi valori come finali in un dato momento della produzione, soprattutto all'inizio quando è più probabile che si metta in piedi un Production Dictionary. Ma meglio averli che non averli
- "*Ultra Warp Ride: minigame where the player flies Solgaleo (US) or Lunala (UM) through wormholes to reach Ultra Space.*" : [+] Al netto dei nomi unici dei Pokémon, ci siamo
- "*e.g., Tauros Charge breaks boulders.*" : Aggiunta corretta, HM da spiegare

- "*Rotom*" : Da spiegare chi sia, anche "Pokédex" è un termine complesso. La definizione è satura di termini che avresti potuto inserire qui sotto. Invece ci sono le reference
- "*Ultra Megalopolis*" : Anche qui c'è una catena di termini complessi da spiegare
- "*Team Rainbow Rocket:*" : Perché questa sezione non è direttamente sotto a quella di "SYSTEMS & FACTIONS"?
- *(riga vuota o spazio)* : Spazio sprecato, insieme a quello perso a destra non è poco *E ancora:* Ora, al netto di questo problema e delle definizioni commentate, il documento è formattato con scelte ottimali sia per forma che per chiarezza delle categorie, che risaltano subito all'occhio. Un interessante primo tentativo di Production Dictionary
- "*References Bulbapedia contributors. Bulbapedia, bulbapedia.bulbagarden.net. Nintendo UK. "10 New Things to Enjoy in Pokémon Ultra Sun and [...]*" : Come commentato per il documento precedente, anzi qui diventa estremamente necessario, sono da eliminare. Con questo ti sei giocato molto spazio sprecato, insieme a quello perso a sinistra

1.5.11 Punti chiave del task T2

- Gli ultimi avvisi del primo task sono diventati regole: tre documenti su nove **non sono stati letti**, fermati al titolo.
- Si rispettano i **vincoli della consegna** (una pagina) e le regole del Drive (*Cognome_Nome_TitoloTask*); i margini vanno rispettati dal contenuto, non solo impostati.
- Il titolo dice che cos'è il documento e su che gioco, senza ripetizioni, decorazioni o immagini che mangiano spazio.
- Elenchi puntati e colonne hanno un costo in spazio a destra: vanno ragionati, e dal prossimo task l'elenco "in zona grigia" non sarà accettato.
- Niente reference in coda, niente righe vuote: lo spazio va alle definizioni.
- Una voce dice prima **che cos'è** il termine (la categoria giusta), poi a che cosa serve e con quali limiti: chi, quanti, dove, a che costo, con che sistema.
- Ogni termine unico usato in una definizione deve avere **una sua voce**, o va sostituito.
- Niente lemma ripetuto nella definizione, niente generalizzazioni false ("tutti", "solo"), coerenza tra le voci.
- Su una pagina, **ogni riga deve lavorare**: se avanza spazio va alle definizioni povere; si accorpano le voci simili.
- I dati numerici sono benvenuti, sapendo che in produzione cambieranno.
- Grassetto, sottolineato e colore si usano con una logica dichiarabile e sempre uguale; maiuscole coerenti, sempre.
- L'inglese è benvenuto.

1.6 Task 3 — TCG pain points (Marvel Snap e Yu-Gi-Oh! Master Duel)

1.6.1 La consegna

Il task era stato assegnato alla fine della lezione dalla ricerca alle idee (capitolo 5, sezione 5.16). È il primo **task di gruppo** del corso e il primo in cui si chiede di **progettare**, non solo di analizzare o definire:

1. quattro gruppi: Francesca e Antonino, Federica e Alberto su *Marvel Snap*; Francesco e Lorenzo, Valerio, Mattia e Daniele su *Yu-Gi-Oh! Master Duel* (due *digital card game*);

2. trovare con una ricerca online (Reddit, Discord, Steam…) **almeno tre pain point** che il pubblico del gioco riconosce come problemi, con i **dati** che li sostengono;
3. analizzarli con gli strumenti visti (Cynefin, tabella 4.4; Liberating Structures, sezione 4.12; perceptual mapping, sezione 5.5);
4. proporre per ciascuno un **design** che lo risolva, preferibilmente con una tecnica semplice (*opposite direction*, *limitation*, *break the theme*, sezione 5.15);
5. consegnare un documento con i tre pain point, come sono stati trovati, i dati spiegati e le soluzioni, più un documento di brainstorming libero (che il docente non legge "prontamente").

Le revisioni sono del 7 novembre 2026. Nell'ordine di revisione è il terzo task del capitolo, e così è numerato qui.[4]

1.6.2 Le consegne in sintesi

Table 1.9: Le consegne del task T3.

Gruppo	Gioco	Pain point scelti	Il nodo principale
Francesca Attadia, Antonino Etiopia	*Marvel Snap*	Pay-to-win; bug e bilanciamento; "user experience"	Analisi ampia e ordinata; soluzioni senza esempi concreti, un pain point troppo largo
Federica Ferrara, Alberto Ronchi	*Marvel Snap*	Neofiti contro veterani; EOMM; carte tier 4/5; più quattro "necessità"	Commenti dell'utenza mostrati bene; soluzioni in una riga, senza mockup, formula o percentuali
Francesco Muratore, Lorenzo Vergata	*Master Duel*	Lancio della moneta; turni "in solitario"; tassa delle hand trap	Il documento più commentato: dati solidi; tabelle da impaginare meglio; soluzioni da mettere alla prova (asta di LP)
Mattia Maestri, Daniele Maracino, Valerio Rossaro	*Master Duel*	Difficoltà per i principianti; monotonia dei contenuti; cambio repentino del meta	Formattazione "al limite dell'accettabile"; manca il pain point principale del gioco (andare per primi)

Un task diverso. Il confronto con i due task precedenti salta all'occhio: **nessun documento non letto** e **nessun nuovo avviso di non lettura**. Il commento più duro riguarda la formattazione del gruppo da tre ("al limite dell'accettabile"). Le regole formali, ormai, sono in gran parte rispettate, e i commenti si spostano dove la consegna voleva portarli: sui **dati**, sull'**analisi** e soprattutto sul **design** delle soluzioni. Gli 80 commenti si distribuiscono in quattro famiglie:

1. **soluzioni non concrete**: manca l'esempio, il mockup, il numero, il caso limite;
2. **dati e fonti**: affermazioni senza dati, commenti dell'utenza citati ma non mostrati, link mancanti;
3. **analisi incompleta**: la causa vera del problema, il problema più importante, i termini del confronto;
4. **impaginazione**: tabelle, immagini decorative, spazi, gerarchia dei titoli.

[4]Nota del curatore: in classe i task possono essere contati in modo diverso (per esempio contando a parte il documento di brainstorming richiesto insieme a questo). In questo capitolo la numerazione segue l'ordine dei task revisionati, che coincide con l'ordine di assegnazione: arcade (capitolo 2), production dictionary (capitolo 4), pain point (capitolo 5).

1.6.3 Errori di forma e di impaginazione

Le tabelle. Il documento di Francesco e Lorenzo apre con una tabella riassuntiva dei tre pain point (che cosa dicono i giocatori, il numero, la categoria Cynefin, la tecnica e il design). Il contenuto funziona, ma il docente dà tre indicazioni di impaginazione che valgono per ogni tabella:

- **ripartire lo spazio tra le colonne**: "Qui avete del testo molto compresso mentre invece tutta la prima colonna ha molto (troppo) respiro. In questi casi, ridimensionate le celle in modo che sia propriamente ripartito tutto lo spazio nella tabella";
- **arrivare ai margini**: "la tabella deve toccare i margini: sfruttate tutto lo spazio che avete";
- **staccare la tabella dal testo**: "cercate sempre di avere un minimo di distacco tra testo e tabella, sia prima che dopo"; e centrare il testo nelle celle quando serve ("centrate il testo nella cella").

Le stesse osservazioni valgono per la seconda tabella del documento ("stesso discorso commentato sull'altra tabella"). Una tabella, inoltre, deve rendere evidenti le relazioni: il numero del pain point "2" accostato a "Solitaire turns" sembra parte del nome ("così sembra che sia in relazione al problema tipo 'due turni di solitario' ").

Le immagini: decorazione o informazione? Il tema del primo task (mostrare invece di scrivere, sezione 1.4.9) si raffina: un'immagine deve **dire qualcosa**.

- "Messa così è una decorazione eliminabile"; "l'immagine sotto è una decorazione" (Francesco e Lorenzo);
- "Per un lettore che non conosce MD quella moneta a destra non è estremamente chiara": un'icona del gioco non spiega nulla a chi non lo conosce;
- "L'immagine sotto vi divora molto spazio" (Francesca e Antonino); "l'immagine è estremamente vicina al testo";
- "Perché non avete usato le funzioni dell'immagine di Google per 'rompere' o mandare a capo il testo?": le opzioni di disposizione del testo intorno all'immagine evitano i blocchi di righe vuote accanto alle figure.

Al contrario, un'immagine che porta il dato funziona: "Il grafico sotto è molto chiaro come premessa alla soluzione".

La formattazione di un lavoro di gruppo. Al gruppo da tre:

> "Eccetto un paio di immagini, il documento è largamente carente lato formattazione. Avete svolto il task in team, e siete stati il team più numeroso, il risultato (sotto questo punto di vista) è al limite dell'accettabile."

Il ragionamento è implicito ma chiaro: più persone significa più ore a disposizione, e quindi un'aspettativa più alta. Lo stesso documento riceve anche un'indicazione di **gerarchia**: il titolo "Soluzione" è più vicino al paragrafo precedente che a quello che introduce, mentre "per gerarchia, questo testo deve essere più vicino al testo sotto". Uno spazio prima di un titolo deve essere maggiore di quello dopo.

Titoli e intestazioni. "How we found them" è "più una intestazione, un'apertura di paragrafo, che un titolo"; il sottotitolo "Three Pain Points and Three Solutions" è "indubbiamente più adatto di quello sotto, che invece fa parte della premessa". A Federica e Alberto, che mettono nel titolo "Ferrara Ronchi": "qui i vostri nomi per intero avrebbero reso meno fraintendibile questo ibrido". E ancora: "Problematica:" davanti a ogni problema è "al limite del testo inutile", anche se il colore richiama correttamente l'intestazione.

Reference e link. Per Francesco e Lorenzo le reference sono in una **scheda separata** del documento: "Averle in un'altra scheda non risolve completamente il problema ma è la direzione corretta". È il terzo passo di un percorso: bibliografia in coda e "eliminabile" nel primo task, "estremamente necessario" eliminarle nel secondo (sezione 1.5.4), in un'altra tab ora. La soluzione completa, già indicata nel primo task, è il **link nel punto in cui serve**: il docente chiede infatti "altri link" (sui *development posts*) e "link diretti o ai feedback dell'utenza o, ancora meglio, alla scheda che tratta il punto specifico". Un dettaglio: "vi sconsiglio di mettere il link sulla punteggiatura, ma non è un obbligo".

Spazi e refusi. "Spazio extra" (due volte, Francesco e Lorenzo), "spazio mancante" (due gruppi), "master duel" minuscolo ("grammatica"). Pochi, ma ancora presenti.

1.6.4 Dati e fonti: mostrare il feedback

Ogni affermazione ha bisogno del dato. È la richiesta del primo task ("servono i dati", sezione 1.4.6), che qui diventa il centro della consegna:

- "povertà nei contenuti": "servono i dati o i feedback linkati/mostrati" (Mattia, Daniele e Valerio);
- "troppe e troppo costose": "dati" (Federica e Alberto);
- "grandissima quantità di carte": "quante sono ad oggi? Un numero indicativo è sufficiente per rendere chiaro il problema";
- "criticato dalla fetta di utenza Free-to-play": "qui avrei voluto leggere le reference o i link più rilevanti" (Francesca e Antonino, che le inseriscono più avanti: "non la scelta che avrei adottato ma funziona").

Mostrare i commenti dell'utenza. Il pain point nasce dalla voce dei giocatori, e il docente vuole **vederla**. A Federica e Alberto, che inseriscono gli screenshot dei commenti accanto a ogni problema: "modo molto chiaro per chiamare in causa i commenti dell'utenza, soluzione corretta". Al gruppo da tre, che li aveva raccolti solo nel brainstorming: "perché non inserire o mostrare commenti che correttamente avete cercato, perché presenti con link del documento di brainstorming?". A Francesco e Lorenzo: "avreste anche potuto estrapolare immagini o citazioni dirette, sempre per il discorso 'show, don't tell' ".

Dare un significato preciso ai criteri. Francesco e Lorenzo dichiarano il metodo (problemi più citati, con dati misurabili, legati al gameplay): corretto, ma "cosa significa 'recurrent', nello specifico, per voi?". Un criterio di selezione vale solo se è misurabile (quante occorrenze, in che periodo, su quali fonti).

Spacchettare il feedback generico. "Gli utenti lamentano la necessità di migliorare l'esperienza di gioco": "feedback molto generico, classica questione da spacchettare". Un pain point è utile quando è **specifico**: che cosa, nell'esperienza, non funziona? Per la stessa ragione un pain point chiamato "User experience" è troppo largo: "se non parliamo di UI o di un sistema specifico, questo si porta dietro (potenzialmente) un'enormità di sistemi. Infatti avete incluso due problemi lontani: eventi e progressione di gioco. [...] Ciascuno necessita di una soluzione specifica, a meno che non abbiate creato la soluzione elegante definitiva."

I dati vanno visualizzati. Il gruppo da tre calcola in un paragrafo la progressione settimanale di gemme e pacchetti: "perché non usare un grafico per i dati sulla progressione settimanale?". Un calcolo in prosa è un dato che il lettore deve ricostruire da sé.

1.6.5 L'analisi: trovare il problema vero

Il pain point principale. Al gruppo da tre, che sceglie difficoltà per i principianti, monotonia e cambio del meta:

> "Il problema (o necessità) più rilevante del gioco è lo schiacciante impatto che ha l'andare per primi. Non averlo portato all'attenzione è un problema in sé."

La ricerca deve trovare **i** problemi del gioco, a partire dal più grave; scegliere problemi secondari, per quanto reali, significa che la fase di ricerca non ha funzionato. Lo stesso problema è il primo dell'altro gruppo su *Master Duel*, sostenuto da un dato ("69,7% of WCS 2025 duels won by the player going first"), che il docente commenta così: "dato tristemente vero".

La causa, non il sintomo. Su quel pain point, il docente aggiunge il passaggio che manca:

> "Il problema non è la moneta in sé, ma il sistema di risorse spendibili che il gioco possiede oppure non possiede. Manca questo 'piccolo' step che rende il tutto preciso."

Il lancio della moneta è il sintomo; la causa è che, senza una risorsa come il mana, chi gioca per primo può spendere tutto subito. Allo stesso modo il docente allarga l'analisi delle hand trap a una causa di design: "la dinamica di gioco che porta a riempire il mazzo di hand trap ha portato alla creazione di parecchie one-card-combo, il che ha portato all'aumento di hand trap nelle liste meta, e così via". È un **ciclo**, e un ciclo si descrive come tale.

Le eccezioni rafforzano il dato. Sull'affermazione che tutto il gioco è centrato sul primo turno: "chiaramente ci sono delle eccezioni, che non rendono il problema meno grave, anzi, rendono ancora più problematico quel 69.7% scritto sopra. Esistono deck going second". Citare le eccezioni non indebolisce l'analisi: la rende più precisa.

I confronti con i competitor. Il confronto deve essere completo e pertinente:

- su *Hearthstone*, che compensa il secondo giocatore con *The Coin* e una carta in più: "voi però non state citando le limitazioni del primo giocatore (la battle non presente) o i vantaggi del secondo (la pescata)", cioè le compensazioni che *Master Duel* ha già;
- su *Marvel Snap*: "infatti Marvel Snap è più lontano come TA rispetto a Hearthstone, Gwent o Runeterra": un competitor si sceglie anche in base al pubblico;
- "i segreti di HS sono letteralmente le trappole in Yugi": il paragone proposto esisteva già dentro il gioco;
- "competitor" senza nomi: "che sono?" (Francesca e Antonino);
- un sistema citato va spiegato: "EOMM: qui andrebbe spiegato il sistema e l'eventuale formula di matchmaking"; e una soluzione basata sull'MMR ha un riferimento noto: "di solito si fa riferimento al sistema di Elo degli scacchi, qui manca un pezzo" (Federica e Alberto; Francesca e Antonino lo avevano citato).

Termini e certezze. "Dinamica" usato in senso generico: "inesatto, ma lo vedremo" (come nel primo task, sezione 1.4.4). Sui formati con vecchi regolamenti: "non sono sicuro abbiano mai usato un formato tipo Edison, anche nella nomenclatura. Voi ne siete certi?". Sulla banlist come "quickest fix": "posso essere d'accordo ma va espanso o fatto un riferimento a banlist specifiche e come hanno cambiato il meta".

1.6.6 Il design delle soluzioni: dall'idea alla proposta

È la parte nuova del task, e quella con più commenti. Il messaggio è uno: una soluzione scritta in una riga è un'intenzione, non un design.

Un esempio concreto, sempre. È la richiesta più frequente:

- tutorial "più approfonditi e remunerativi": "un esempio concreto, rispetto all'implementazione attuale"; structure deck più al passo con il meta: "esempio di archetipo di cui si può fare uno structure e che core viene inserito nel mazzo" (Mattia, Daniele e Valerio);

- "a ogni tutorial completato il giocatore ottiene un premio": "portare un esempio concreto. [...] Avete margine di manovra per quanto riguarda la possibile presentazione della modalità: sbizzarritevi, questo è il momento per farlo" (Federica e Alberto);
- formati con limitazioni: "un esempio"; contenuti estetici al posto delle carte: "esempio di oggetto di gioco o nuovo sistema (es. di personalizzazione: banner del giocatore, nuova musica da acquistare, cosmetico X da definire)"; nuove modalità: "esempi di modalità nuove o ibridi di modalità vecchie?"; il sistema di arene di *Clash Royale*: "interessante, fate un esempio di 'arena' con potenziale gruppo di carte" (Francesca e Antonino);
- le hand trap assegnate a un trigger: "un esempio pratico di carte con trigger assegnato avrebbe potuto aiutare" (Francesco e Lorenzo).

Mostrare la soluzione: mockup e wireframe. Per le soluzioni che toccano l'interfaccia, il docente chiede di **vederle**: l'avviso quando si viene abbinati a un bot ("serve un mockup anche tentativo di questa soluzione"), il sistema di accesso giornaliero ("classico sistema di log-in, può funzionare anche per la retention. Però serve un mockup"), la lista amici ("cosa posso fare con gli amici. Che funzionalità ha questa lista e che UX ha? Wireframe"). È la regola "dovrei vederlo, non leggerlo" applicata al design.

Numeri e casi limite. Una proposta di design deve reggere alle domande del giocatore più furbo:

- il "tiro di dadi" per ottenere gratis la nuova carta: "vanno esplicitate queste percentuali, anche tentative. Inoltre, se il giocatore non ottiene la carta dopo 10 tiri di dadi consecutivi? Avete pensato ad una meccanica di pity?";
- l'asta di Life Points per decidere chi inizia (*Turn Bid*, Francesco e Lorenzo): "con una vittoria, dati alla mano, così schiacciante per i deck going first, cosa garantisce che i giocatori non puntino entrambi 7999 LP? E se entrambi puntano lo stesso numero?". Il docente apprezza la presentazione ("il grafico sotto è molto chiaro"), ma "è la chiarezza del contenuto che mi fa riflettere": se il primo turno vale quasi la partita, l'asta sposta il problema invece di risolverlo;
- il *Response Plan*: "non mi è chiaro se questo è esclusivamente collegato alla combo del giocatore di riferimento, oppure se funziona anche sulle risposte all'attivazione di carte dell'avversario", perché giocare attorno alle hand trap "è ormai parte delle strategie previste nel gioco".

Conseguenze, confronti, costi. Una soluzione non è mai gratuita. Il docente chiede di dichiararne:

- gli effetti sulla percezione: "come modifica la percezione del giocatore? Quali problematiche porta ed, eventualmente, perché sono accettabili (o non accettabili, a seconda del TA)";
- il confronto tra alternative: "queste soluzioni, che pro e che contro hanno confrontandosi l'una con l'altra?";
- il costo di produzione: "questa è una soluzione complessa e costosa"; "soluzione molto tecnica, ahimè dipende anche dall'infrastruttura del codice"; e, sulla ridistribuzione del team, "questo dipende molto dall'esperienza delle figure professionali e dalla pipeline interna" (pur riconoscendo che "qui la soluzione è già più concreta e chiara"), oltre al fatto che il team di Second Dinner "al momento ha un'altra linea di produzione in corsa";
- la coerenza con l'identità del gioco: "come si unisce però questo con lo USP dello 'snap'?" (su un matchmaking basato solo sull'abilità).

Francesco e Lorenzo, per la *Reserve*, si erano posti proprio questa domanda in una tabella ("Is it still MD?"), e il docente indica lì il punto più promettente: "qui discutibilmente risiede parte dell'innovazione proposta. Tentare (per ora ipoteticamente) un design elegante per risolvere problema 1 e 3, provate a pensare alle prize card nel TCG di Pokémon unite al side deck di Yugi". È il **design elegante** della lezione (sezione 5.13): una sola soluzione per più problemi.

Niente linguaggio debole, nemmeno in design. “Niente linguaggio debole, a maggior ragione nella fase di design, per quanto preliminare possa essere” (“may”, Francesco e Lorenzo); “quando fate design non utilizzate linguaggio debole” (“si potrebbe”, Mattia, Daniele e Valerio). Una proposta si scrive al presente o al futuro: “il giocatore sceglie”, non “si potrebbe far scegliere”.

Un esempio da riscrivere. Una soluzione di Federica e Alberto, sviluppata seguendo le domande del docente (proposta del curatore; i valori sono ipotetici, come il docente stesso concede: “anche tentative”).

Nella consegna	Riscrittura proposta
Ogni settimana, all’uscita della nuova carta, si ottiene un tiro di dadi che dà al giocatore la possibilità di riceverla gratuitamente, così anche i nuovi/vecchi giocatori hanno una possibilità in più per ottenerla.	*Tiro settimanale.* All’uscita di ogni nuova carta, ogni giocatore riceve 1 tiro a settimana (mockup del pannello). Probabilità di ottenere la carta: 10% al primo tiro, +10% per ogni tiro fallito consecutivo (*pity*: carta garantita al decimo tiro). Da stimare: quante carte nuove ottiene in media un giocatore free-to-play al mese, e l’impatto sugli acquisti.

1.6.7 Che cosa è stato apprezzato

- i **commenti dell’utenza mostrati** accanto al problema (Federica e Alberto: “soluzione corretta”);
- il **dato** forte e verificato (il 69,7% delle vittorie di chi inizia) e un **grafico** “molto chiaro come premessa alla soluzione” (Francesco e Lorenzo);
- un’**introduzione chiara** al gioco (Mattia, Daniele e Valerio);
- l’analisi del pay-to-win che il docente condivide (“posso essere d’accordo”) e la soluzione sul team di sviluppo, “già più concreta e chiara, sia nell’esposizione che nel risultato pratico”; l’idea delle arene di *Clash Royale* (“interessante”) (Francesca e Antonino);
- il riferimento al sistema **Elo** per il matchmaking (Francesca e Antonino), lo stesso che il docente suggerisce all’altro gruppo;
- le reference in una **scheda separata**: “la direzione corretta” (Francesco e Lorenzo);
- la domanda “è ancora *Master Duel*?” posta alla propria soluzione, dove “risiede parte dell’innovazione proposta” (Francesco e Lorenzo).

1.6.8 Dai task 1 e 2 al task 3: confronto e pattern ricorrenti

La stessa domanda, in tre forme. Il filo che lega i tre task è quello già indicato nel confronto precedente (sezione 1.5.9): il documento presuppone qualcosa che il lettore non ha. Nel primo task mancava il dettaglio delle meccaniche, nel secondo la definizione dei termini usati, nel terzo l’esempio concreto della soluzione. Le domande del docente si somigliano in modo quasi letterale: “quanti e quali?” (T1), “quanti sono?” (T2), “quante sono ad oggi?” (T3); “come?” in tutti e tre. È la domanda di chi deve **implementare**, e va anticipata.

Che cosa migliora. Le regole formali non sono più un problema; wall of text, opinioni e parola proibita non ricompaiono; il testo inutile è residuale. Nel percorso di Lorenzo la gestione delle fonti è passata dalla bibliografia accademica a una scheda separata, verso il link nel punto d’uso. Nel percorso di Valerio, dal wall of text del primo task si è arrivati a un documento di gruppo il cui problema non è più la densità ma l’assenza di formattazione.

Che cosa resta. Il linguaggio debole ricompare appena si passa dall’analisi alla proposta: scrivere una soluzione sembra autorizzare il condizionale. E l’immagine, che il primo task chiedeva di usare di più, ora va **scelta**: un’immagine che non porta informazione è una decorazione, e come tale è eliminabile (lo stesso giudizio dato allo sprite nel titolo di Valerio nel secondo task).

Table 1.10: Le famiglie di errori nei tre task.

Famiglia	**T1 (analisi)**	**T2 (dizionario)**	**T3 (pain point)**
Conseguenze	1 lettura interrotta	3 documenti non letti	nessun non letto, nessun nuovo avviso; un "al limite dell'accettabile"
Regole formali	margini e font in 4 documenti	nome file, limite di pagine, margini	nessuna violazione segnalata
Testo inutile	ovunque	un documento non letto	quasi scomparso ("al limite", "già scritto sopra")
Dati e reference	"servono i dati" su pubblico ed estetiche	—	"dati", "quante sono?", feedback da mostrare e linkare
Mostrare	immagini, UI, tabelle	—	mockup e wireframe delle soluzioni; immagini decorative da togliere
Dettaglio	meccaniche ad alto livello	definizioni senza chi, quanti, dove	soluzioni senza esempio, numeri, casi limite
Lettore che non sa	"mai parlato prima"	termini complessi non definiti	"competitor: che sono?", l'icona che chi non conosce MD non capisce
Linguaggio debole	frequente	raro	torna nelle proposte di design ("may", "si potrebbe")
Spazio e impaginazione	righe vuote, pagine vuote	elenchi e colonne	tabelle (colonne, margini, distacco), immagini e gerarchia
Reference in coda (Lorenzo)	"eliminabile"	"estremamente necessario" toglierle	in una scheda separata: "direzione corretta"

Che cosa cambia. Con il design arriva un tipo di commento nuovo: il docente non chiede più solo "che cosa?" ma "e se…?" (7999 LP, dieci tiri falliti, risposte all'avversario), "a che costo?" (pipeline, codice, team) e "è ancora il gioco?" (USP dello snap). Sono le domande di una revisione di design, e conviene porsele prima di consegnare.

1.6.9 Le schede delle singole consegne

Seguono, documento per documento, **tutti** gli 80 commenti del docente, nell'ordine in cui compaiono, ciascuno con il punto a cui è ancorato. Per la legenda si veda la sezione 1.1.

Francesca Attadia e Antonino Etiopia — *Marvel Snap*

Documento ordinato (per ogni pain point: analisi in tre punti con fonti, tre possibili risoluzioni). I commenti chiedono soprattutto esempi concreti per le soluzioni, il confronto tra alternative e la specificità di un pain point troppo largo ("User experience").

- "*fast-paced card game*" : Ci sono un paio di qualità che fanno di Snap un mobile unico, per esempio l'omonimo sistema. L'utilità della premessa rimane, con il contenuto portato avreste potuto tagliare una riga, rimanendo più sintetici *E ancora:* L'immagine sotto vi divora molto spazio
- "*criticato sia dalla fetta di utenza Free-to-play*" : Qui avrei voluto leggere le reference o i link più rilevanti *E ancora:* Ok, le inserite dopo, non la scelta che avrei adottato ma funziona
- "*User experience*" : Se non parliamo di UI o di un sistema specifico, questo si porta dietro (potenzialmente) un enormità di sistemi. Infatti avete incluso due problemi lontani: eventi e progressione di gioco *E ancora:* Non che questi non siano da addressare, anzi, sono pain points cruciali. Ma ciascuno necessita di una soluzione specifica, a meno che non abbiate creato la soluzione elegante definitiva. Continuo la lettura
- "*dinamica*" : Inesatto, ma lo vedremo
- "*competitor*" : Che sono?
- "*portando i giocatori alla quasi necessità di spendere denaro per permettersi di sbloccare le nuove carte in breve tempo.*" : [+] Posso essere d'accordo
- "*formati di gioco in cui vengono applicate limitazioni o condizioni*" : Un esempio *E ancora:* Inoltre, queste soluzioni, che pro e che contro hanno confrontandosi l'una con l'altra?
- "*differenti tra carte preesistenti e nuove uscite*" : Questa è una soluzione complessa e costosa, vi chiederei un esempio di design di una carta ma è uno spicchio di quello che serve
- "*più sul lato estetico che sull'ottenimento di carte nuove, permettendo a tutti un maggiore fair play.*" : Esempio di oggetto di gioco o nuovo sistema (es. di personalizzazione: banner del giocatore, nuova musica da acquistare, cosmetico X da definire)
- "*Focus sui test interni*" : Questo dipende molto dall'esperienza delle figure professionali e dalla pipeline interna, cose di cui nemmeno io ho chiara conoscenza. Detto questo, qui la soluzione è già più concreta e chiara, sia nell'esposizione che nel risultato pratico al lettore
- "*2. Ottimizzazione del gioco*" : Soluzione molto tecnica, ahimè dipende anche dall'infrastruttura del codice
- "*questi vengono quindi giudicati secondo la loro effettiva skill di gioco e non sulla casualità degli abbinamenti nel matchmaking.*" : Come si unisce però questo con lo USP dello "snap"? Possiamo espandere a lezione, perché è un tema sentito che si lega anche al lavoro di design, che avete iniziato a fare e sul quale espanderemo *E ancora:* L'immagine è estremamente vicina al testo
- "*necessità di migliorare l'esperienza di gioco*" : Feedback molto generico, classica questione da spacchettare
- "*aggiunte a rotazione è un metodo efficace, ma sfruttato male*" : Al netto della grandezza del team di Second Dinner, contando anche che al momento hanno un'altra linea di produzione in corsa, esempi di modalità nuove o ibridi di modalità vecchie?
- "*sistema di arene di Clash Royale che blocca tutte le carte sotto una determinata Arena o rank di gioco*" : [+] Interessante, fate un esempio di "arena" con potenziale gruppo di carte (raggruppate per tema, come in CR, o per impatto sul meta/complessità dell'effetto)

Federica Ferrara e Alberto Ronchi — *Marvel Snap*

Tre problematiche e quattro necessità, ciascuna con gli screenshot dei commenti dell'utenza (apprezzati). Le soluzioni sono di una riga: il docente chiede mockup, wireframe, percentuali, la formula del matchmaking e i casi limite.

- "*Ferrara Ronchi*" : Qui i vostri nomi per intero avrebbero reso meno fraintendibile questo ibrido
- "*Problematica:*" : Al limite del testo inutile, ok avere la relazione di colore con l'intestazione sopra
- *(riga vuota o spazio)* : [+] Modo molto chiare per chiamare in causa i commenti dell'utenza, soluzione corretta
- "*A ogni tutorial completato il giocatore ottiene un premio.*" : Sia per il punto sopra che, soprattutto, per questo: portare un esempio concreto. Per quando riguarda la soluzione "1", avete margine di manovra per quanto riguarda la possibile presentazione della modalità: sbizzarritevi, questo è il momento per farlo
- "*EOMM (Engagement Optimized MatchMaking):*" : Qui andrebbe spiegato il sistema e l'eventuale formula di matchmaking
- "*per forza un/o vantaggio/svantaggio per equilibrare il win rate del giocatore.*" : Ok, da chi la prendete? Di solito si fa rifermento al sistema di Elo degli scacchi, qui manca un pezzo
- "*Implementare un avviso a schermo quando un giocatore viene matchato con un bot.*" : Primo: serve un mockup anche tentativo di questa soluzione. Secondo: come modifica la percezione del giocatore? Quali problematiche porta ed, eventualmente, perché sono accettabili (o non accettabile, a seconda del TA)
- "*troppe e troppo costose.*" : Dati
- "*tiro di dadi che dà al giocatore la possibilità di riceverla gratuitamente*" : Vanno esplicitate queste percentuali, anche tentative. Inoltre, se il giocatore non ottiene la carta dopo 10 tiri di dadi consecutivi? Avete pensato ad una meccanica di pity?
- "*semplicemente facendo l'accesso giornaliero per 7 giorni.*" : Classico sistema di log-in, può funzionare anche per la retention. Però serve un mockup
- "*Implementare la lista amici.*" : Ok, cosa posso fare con gli amici. Che funzionalità ha questa lista e che UX ha? Wireframe
- "." : Spazio mancante

Francesco Muratore e Lorenzo Vergata — *Yu-Gi-Oh! Master Duel*

Il documento più commentato (34 commenti). Dati solidi e metodo dichiarato, tabelle da impaginare meglio, immagini decorative, soluzioni originali (*Turn Bid*, *Response Plan*, *Reserve*) da mettere alla prova contro i casi limite; reference in una scheda separata. Il file revisionato è arrivato come *TCGPainPoints*, senza i cognomi.

- "*Three Pain Points and Three Solutions*" : [+] Questo è indubbiamente un sottotitolo più adatto di quello sotto, che invece fa parte della premessa ancora più sotto
- "*How we found them*" : Più una intestazione, un'apertura di paragrafo, che un titolo in generale
- *(riga vuota o spazio)* : Spazio extra
- "*development posts*" : Qui altri link avrebbero aiutato
- "*We filtered the problems, choosing the ones that are: most cited, had measurable data, and were gameplay related.*" : Queste quattro premesse suonano come punti di una lista. Come sempre, l'elenco puntato è una soluzione ma non l'unica né necessariamente la migliore. Quello che scrivete è corretto, al netto di un paio di questioni da approfondire (cosa significa "recurrent", nello specifico, per voi?), però come può essere presentato in maniera ancora più chiara per il lettore? Pensateci
- "*issues:*" : Cercate sempre di avere un minimo di distacco tra testo e tabella, sia prima che dopo
- "*What players say*" : Anche qui, link diretti o ai feedback dell'utenza o, ancora meglio, alla scheda che tratta il punto specifico
- "*1 Coin toss*" : Centrate il testo nella cella
- "*69.7% of WCS 2025 duels won by the player going first*" : Qui avete del testo molto compresso mentre invece tutta la prima colonna ha molto (troppo) respiro. In questi casi, ridimensionate le celle in modo che sia propriamente ripartito tutto lo spazio nella tabella *E ancora:* Inoltre, la tabella deve toccare i margini: sfruttate tutto lo spazio che avete

- "*69.7%*" : [+] Dato tristemente vero
- "*2*" : Rendete evidenti le relazioni tra numeri e pain point, così sembra che sia in relazione al problema tipo "due turni di solitario"
- "*centered on the first turn.*" : Chiaramente ci sono delle eccezioni, che non rendono il problema meno grave, anzi, rendono ancora più problematico quel 69.7% scritto sopra. Esistono deck going second, come differenze tra deck spiccatamente basati su combo (parecchi) e "aggro" o "control" deck (discorso complesso in Yugi)
- *(riga vuota o spazio)* : Qui c'è anche un problema lato design (da parte di Konami). La dinamica di gioco che porta a riempire il mazzo di hand trap ha portato alla creazione di parecchie one-card-combo, il che ha portato all'aumento di hand trap nelle liste meta, e così via
- *(riga vuota o spazio)* : Messa così è una decorazione eliminabile
- "*Non-balanced Coin Toss*" : Ok, avreste anche potuto estrapolare immagini o citazioni dirette, sempre per il discorso "show, don't tell". Però corretto averlo inserito
- " " : Vi sconsiglio di mettere il link sulla punteggiatura, ma non è un obbligo
- "*This is not the case of Master Duel.*" : Qui il discorso andrebbe espanso. Voi probabilmente ci sarete arrivati o lo avrete in mente, ma non lo avete scritto: il problema non è la moneta in sé, ma il sistema di risorse spendibili che il gioco possiede oppure non possiede. Manca questo "piccolo" step che rende il tutto preciso
- *(riga vuota o spazio)* : Spazio extra
- *(riga vuota o spazio)* : Perché non avete usato le funzioni dell'immagine di Google per "rompere" o mandare a capo il testo? *E ancora:* Inoltre per un lettore che non conosce MD quella moneta a destra non è estremamente chiara
- "*and a bonus card*" : Voi però non state citato le limitazioni del primo giocatore (la battle non presente) o i vantaggi del secondo (la pescata)
- "*Marvel Snap*" : Infatti Marvel Snap è più lontano come TA rispetto a Hearthstone, Gwent o Runeterra
- "*may*" : Niente linguaggio debole, a maggior ragione nella fase di design, per quanto preliminare possa essere
- "*the player that pays the highest amount of LP starts as the first duelist.*" : Con una vittoria, dati alla mano, così schiacciante per i deck going first, cosa garantisce che i giocatori non puntino entrambi 7999 LP? E se entrambi puntano lo stesso numero? *E ancora:* Il grafico sotto è molto chiaro come premessa alla soluzione, è la chiarezza del contenuto che mi fa riflettere
- "*master duel*" : Grammatica
- "*Konami minimized the duel time limit from 480 to 300 seconds.*" : Già scritto sopra
- "*like Hearthstone's secrets*" : Non mi è chiaro se questo è esclusivamente collegato alla combo del giocatore di riferimento, oppure se funziona anche sulle risposte all'attivazione di carte dell'avversario. Cito il secondo punto, siccome questo è ormai parte delle strategie previste nel gioco (consumare la hand trap al momento giusto, oppure baitare/giocare attorno ad una possibile hand trap dell'avversario). In questo caso, un esempio pratico di carte con trigger assegnato avrebbe potuto aiutare, sempre considerando che l'immagine sotto è una decorazione *E ancora:* Inoltre i segreti di HS sono letteralmente le trappole in Yugi
- "*A banlist is the quickest fix.*" : Posso essere d'accordo ma va espanso o fatto un riferimento a banlist specifiche e come hanno cambiato il meta
- "*Limitation Is it still MD?*" : Stesso discorso commentato sull'altra tabella
- "*Yeah. In fact, it is as Hearthstone's Coin, a compensation mechanism adopted there*" : Qui discutibilmente risiede parte dell'innovazione proposta. Tentare (per ora ipoteticamente) un design elegante per risolvere problema 1 e 3, provate a pensare alle prize card nel TCG di Pokémon unite al side deck di Yugi
- "*References*" : [+] Averle in un'altra scheda non risolve completamente il problema ma è la direzione corretta

Mattia Maestri, Daniele Maracino e Valerio Rossaro — *Yu-Gi-Oh! Master Duel*

Introduzione chiara, ma formattazione "al limite dell'accettabile" per un gruppo di tre; dati e commenti dell'utenza non mostrati; il pain point principale del gioco (andare per primi) assente; soluzioni con linguaggio debole e senza esempi.

- "*Il duello si svolge tra due avversari con 8000 Life Points, dove con deck di 40-60 carte formati dai giocatori si dovranno sfidare con [...]*" : [+] Introduzione chiara

- "*grandissima quantità di carte*" : Quante sono ad oggi? Un numero indicativo è sufficiente per rendere chiaro il problema, che posso condivedere
- "*decidere come approcciarvisi o costruire un buon mazzo senza che diventi obsoleto nel giro di poco tempo.*" : Perché non inserire o mostrare commenti che correttamente avete cercato, perché presenti con link del documento di brainstorming?
- "*Soluzione*" : Per gerarchia, questo testo deve essere più vicino al testo sotto che a quello precedente, sopra
- "*si potrebbe*" : Quando fate design non utilizzate linguaggio debole
- "*approfonditi e remunerativi.*" : Un esempio concreto, rispetto all'implementazione attuale
- "*Dando anche qui la possibilità al nuovo giocatore di poter competere nella sezione competitiva.*" : Anche qui, esempio di archetipo di cui si può fare uno structure e che core viene inserito nel mazzo
- "*Problema: Monotonia dei contenuti di gioco*" : **[!]** Eccetto un paio di immagini, il documento è largamente carente lato formattazione. Avete svolto il task in team, e siete stati il team più numeroso, il risultato (sotto questo punto di vista) è al limite dell'accettabile
- "*povertà nei contenuti*" : Servono i dati o i feedback linkati/mostrati
- "*,*" : Spazio mancante
- "*vecchi regolamenti*" : In parte questo viene fatto, ma non sono sicuro abbiano mai usato un formato tipo Edison, anche nella nomenclatura. Voi ne siete certi?
- "*generato randomicamente*" : Tipo draft? Non chiaro
- "*Problema: Repentino cambio di metagame*" : Il problema (o necessità) più rilevante del gioco è lo schiacciante impatto che ha l'andare per primi. Non averlo portato all'attenzione è un problema in sé
- "*Infatti un giocatore può completare non più di 3 missioni al giorno per una ricompensa di 120 gemme a cui si possono aggiungere 20 gemme [...]*" : Perché non usare un grafico per i dati sulla progressione settimanale?

1.6.10 Punti chiave del task T3

- Un pain point si sostiene con i **dati** e con la **voce dei giocatori**, mostrata (screenshot, citazioni) e linkata nel punto in cui serve.
- Il pain point deve essere **specifico**: "user experience" o "migliorare l'esperienza" vanno spacchettati.
- La ricerca deve trovare il problema **più grave** del gioco, e l'analisi la sua **causa**, non il sintomo.
- I confronti con i competitor sono completi (anche ciò che il gioco analizzato già fa) e pertinenti al pubblico.
- Ogni soluzione ha un **esempio concreto**, i suoi **numeri** (anche tentativi), un **mockup** o wireframe se tocca l'interfaccia.
- Ogni soluzione regge ai **casi limite** ("e se...?"), dichiara costi e conseguenze e resta coerente con l'identità del gioco.
- Il design elegante risolve più problemi con una soluzione.
- Niente linguaggio debole, nemmeno nelle proposte.
- Tabelle che arrivano ai margini, con lo spazio ben ripartito tra le colonne e staccate dal testo; immagini che informano, non decorano.
- Un lavoro di gruppo è giudicato in proporzione al gruppo.

1.7 Task 4 — Il flowchart di Pac-Man

1.7.1 La consegna

Il task era stato assegnato alla fine della lezione su brainstorming, flowchart e grafici (capitolo 6, sezione 6.14):

1. documentare con i flowchart **tutto** *Pac-Man*, l'intero gioco, "a basso livello", considerando potenzialmente tutte le interazioni (Daniele: *Donkey Kong*, l'arcade);
2. usare i blocchi visti a lezione (tabella 6.1) con le loro regole (sezioni 6.10 e 6.11), con un flusso che si chiude o torna indietro (sezione 6.12);
3. consegnare **solo il flowchart**, come **immagine** commentabile (in una presentazione o in un documento Google), leggibile senza zoomare, esportata tra il 300 e il 500%, con uno sfondo che non faccia sparire le linee;
4. regole formali "sollevate": qualsiasi font, purché leggibile.

A lezione il docente aveva dato anche il metodo: non partire dal flowchart perfetto, schizzare, fare **diversi flowchart** (movimento, raccolta di punti e oggetti, power pill e cambi di stato, intelligenza dei fantasmi) e poi unirli; e aveva lasciato intendere che il modo "furbo" per avere un flowchart di tutto il gioco più altri flowchart di dettaglio è il blocco di **sotto-processo**. Le revisioni sono dell'11 novembre 2026.

1.7.2 Le consegne in sintesi

Un task diverso, gli stessi commenti. È il primo task che non è un documento di testo ma un'**immagine**, e il primo in cui le regole formali della documentazione sono sospese. I commenti, quindi, non parlano di margini e di Roboto, ma di due cose: la **grammatica del flowchart** (che cosa significano i blocchi e le frecce) e la **logica del gioco** (se, seguendo il flowchart, si ottiene davvero *Pac-Man*). Colpisce quanto i commenti si ripetano **alla lettera** tra un documento e l'altro: la stessa frase, "Quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme", compare in **cinque consegne su sette**; "Il flowchart ha numerose lacune nella logica presentata. Analizza i miei commenti e leggi i task dei tuoi colleghi, riflettici e se necessario ne riparliamo" in due; "Cosa succede al game over?" (o "dopo il game over") in quattro. Quando il docente ripete la stessa frase a cinque persone, l'errore non è di uno studente: è un punto della lezione che non è passato.

I 79 commenti (uno dei quali è un elenco di sei punti) si distribuiscono in quattro famiglie:

1. **regole del flowchart**: rettangoli con più uscite, uscite dei rombi non etichettate o duplicate, sotto-processi senza il loro flowchart, frecce sovrapposte, blocchi scollegati, loop involontari;
2. **logica del gioco**: vite, game over e high score, fantasmi (uscita dalla casa, spavento, comportamento), oggetti e numeri;
3. **leggibilità e forma**: misure del testo, testo fuori dal blocco, margini interni, titoli, nome del protagonista, documenti di supporto;
4. **completezza e scope**: un task consegnato a metà.

1.7.3 Le regole del flowchart

Un rettangolo ha una sola uscita. È l'errore più frequente del task, e il docente lo dice in due modi:

- "Da un rettangolo non può uscire più di una freccia, non è un rombo" (Valerio);
- "Quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme" (Valerio, Francesco, Mattia, Francesca due volte, e il flowchart senza nome);
- "Come lettore, io da questo blocco non ho idea di dove andare con la freccia in uscita" (Francesco).

Table 1.11: Le consegne del task T4.

Studente	**Che cosa ha consegnato**	**Commenti**	**Il nodo principale**
Valerio Rossaro	un flowchart (PNG/JPEG, sfondo scuro)	8	rettangoli con più uscite, sotto-processi non spiegati, vite e game over
Francesco Muratore	un flowchart	17	il più commentato: uscite, frecce sovrapposte, formattazione, regole del gioco mancanti o errate
Mattia Maestri	un flowchart (sfondo scuro)	10	due "yes" sullo stesso rombo, blocchi scollegati, fantasmi assenti, grammatica
Francesca Attadia	flowchart principale e flowchart dei fantasmi	12 + 4	uscite in sequenza, un loop involontario, un'uscita invertita; apprezzato il sotto-processo dei fantasmi
Antonino Etiopia	documento con quattro flowchart	5 (uno è un elenco di 6 punti)	blocchi scollegati, testo fuori dal blocco ([**!**]), nome del protagonista ([**!**]), high score
Lorenzo Vergata	9 flowchart su 15 (PDF, fatti con TikZ) e una nota in .txt	4	task **non concluso**: niente commenti di merito; la relazione tra frecce e blocchi è indicata come esempio
Autore non indicato (*Flowchat_pacman*)	tre flowchart in PDF (principale, Frightened, timer) e un documento di supporto	19	uscite, frecce che si intersecano, nomi e numeri da esplicitare, documento di supporto senza regole
Daniele Maracino	flowchart di *Donkey Kong*	—	nessun commento pervenuto al momento della stesura

Il motivo è nella lezione (sezione 6.11): il rettangolo è un'**azione**, e dopo un'azione il processo prosegue in un solo modo. Se da un'azione partono tre frecce, chi legge (o chi programma) non sa quale seguire: la scelta tra più strade è una **decisione**, e le decisioni sono rombi. A Francesca il docente esplicita l'assunto che sta sotto: "Quali uscite deve prendere il flowchart? L'assunto di base è che non possono essere vere entrambe contemporaneamente". E quando le cose da fare sono davvero più d'una (sommare 50 punti *e* rendere vulnerabili i fantasmi *e* controllare i puntini rimasti), la soluzione non è aprire più frecce ma metterle in fila: "Anche qui, una uscita per volta, in sequenza".

Le uscite multiple non sono solo un problema di forma. A Francesca: "Infatti se consideriamo solo l'uscita a destra, il flowchart continua a loopare finché non vieni colpito da un fantasma, il che è un bel bug": dal blocco "PACMAN MOVES" la freccia a destra va al controllo della collisione, e il "no" torna a "PACMAN MOVES"; chi segue quella strada non mangia mai un puntino.

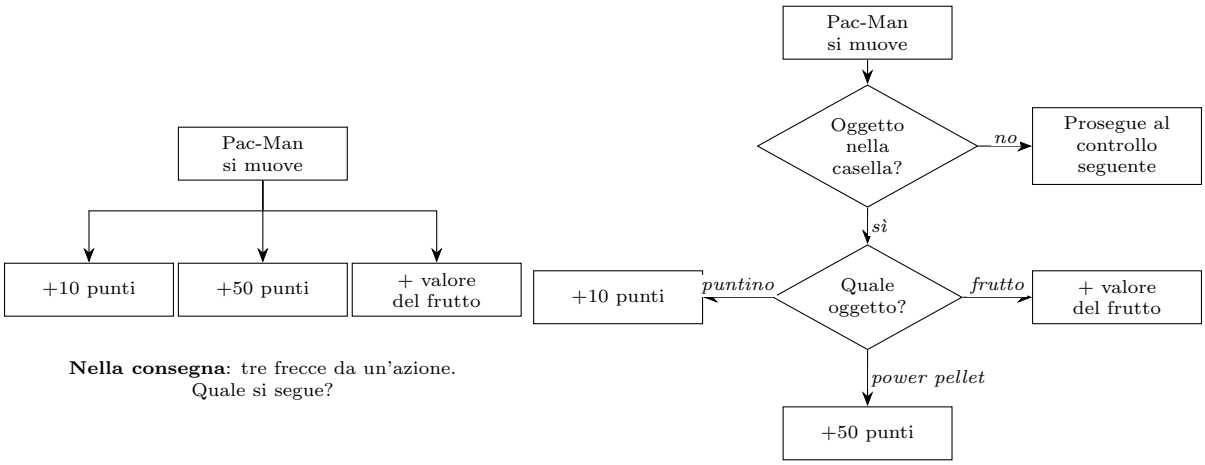


Figure 1.3: Il rettangolo con più uscite e la sua correzione (proposta del curatore). Le uscite del rombo sono le categorie, come suggerito dal docente ("invece di avere molteplici e fraintendibili 'sì', perché non sostituirli direttamente con le categorie"). Il caso "nessun oggetto" è separato da un primo rombo, così ogni esito ha il suo vertice.

Le uscite del rombo: una per esito, sempre etichettata. Il rombo è la decisione, ma anche il rombo ha le sue regole (sezione 6.11): uscite **esplicite**, e ogni uscita è **un** risultato.

- Due uscite uguali: "Sono due uscite con lo stesso valore 'yes', quale è vera?" (Mattia: dal rombo "move" escono due "yes"). È lo stesso errore che il docente aveva fatto per sbaglio alla lavagna: "ha scritto due volte *yes*. Porca miseria, terrificante" (sezione 6.12).

- Uscite senza etichetta: "Quale di queste uscite è legata al 'NO'?" (Francesco: dal rombo "Ghosts Hit Character?" partono due frecce a destra con una sola etichetta).

- Più "sì" per categorie diverse: "No, come lo hai scritto è un problema. Invece di avere molteplici e fraintendibili 'sì', perché non sostituirli direttamente con le categorie: power pellet, frutto bonus (la cui lista serve e manca), ecc." (flowchart senza nome, rombo "tocca elemento labirinto?").

- Più esiti, tutti nominati: per il movimento dei fantasmi di Francesca, dove un rettangolo "GHOSTS MOVE" si apre su quattro fantasmi: "Rombo con nomi dei fantasmi sulle quattro uscite". Quattro esiti, quattro vertici, quattro etichette.

- Uscite invertite: "Il contrario" (Francesca: "LIVES LEFT?" manda il "yes" al game over e il "no" al riposizionamento).

Ogni sotto-processo ha il suo flowchart. La regola era stata definita "ferrea" a lezione: il blocco con i bordi doppi **va spiegato in un altro flowchart**, "per forza" (sezione 6.11).

- "Ogni azione con sotto processo va spiegata con un flowchart associato. Ogni blocco simile che utilizzi, ne abbiamo parlato a lezione"; "Tutti sistemi da spiegare" (Valerio, che usa cinque sotto-processi senza spiegarne nessuno);
- "Va spiegato, dov'è il flowchart che mostra questa azione con sotto processo?" (Francesco);
- al contrario, a Francesca, che il sotto-processo dei fantasmi lo spiega in un secondo flowchart: "Considerando i comportamenti diversi dei singoli fantasmi, averlo come azione con sotto processo è corretto" ([+]).

Il sotto-processo è anche una questione di **coerenza della gerarchia**. Ad Antonino: "Il sistema di punti, in gerarchia, è prima del movimento. Ma nello spiegare l'azione con sotto processo citi il mangiare, che è legato al muoversi di Pac-Man". L'ordine dei blocchi nel flowchart principale deve rispettare l'ordine causale dei sistemi: si mangia perché ci si è mossi, quindi il punteggio non può venire prima del movimento. E un sotto-processo non va ripetuto altrove: "Meccaniche che citi anche nel flowchart principale" (Francesca), "Questo è presente anche nel flowchart principale" (flowchart senza nome).

Le frecce: niente sovrapposizioni fraintendibili. A lezione: due frecce possono sovrapporsi solo se vanno **nella stessa direzione** (sezione 6.12). Nelle consegne:

- "Questo è un problema, le frecce del flusso si sovrappongono creando potenziali fraintendimenti", "Non è presente solo qui" (Francesco);
- "Sovrapposizione fraintendibile di frecce nel flusso, è un problema" (Valerio);
- "La freccia a destra del Fine livello si interseca con un'altra, e crea una fraintendibilità che non può essere presente" (Antonino);
- "Qui sotto le frecce si ingarbugliano problematicamente. Ripensaci"; "Qui le frecce a sinistra si intersecano in una maniera fraintendibile, è un problema. Ripensa alla messa a terra"; "Problema di frecce sovrapposte a sinistra, riguarda tutti i flowchart che hai consegnato" (flowchart senza nome).

A Francesca, che agli incroci usa i "ponticelli" di draw.io, il docente riconosce una soluzione "eccezionale, comprensibile, ma resta una eccezione 'fatta in casa' " (il commento non specifica il punto; il riferimento più probabile sono proprio gli incroci). È la logica delle libertà artistiche della lezione (sezione 6.13): si può piegare una regola per la leggibilità, ma resta un'eccezione, e la soluzione migliore è un layout in cui le frecce non debbano incrociarsi.

Nessun blocco nel nulla. "O si chiude, o ritorna indietro" (sezione 6.12). Il caso più netto è quello di Mattia:

> "Questi blocchi non sono collegati con il resto del flusso, e vale per altri 'nothing happens' oppure 'game continues'. Questo è un problema, se dovessimo implementare questa logica il gioco sarebbe 'rotto' o con severi bug."

Ad Antonino: "Y = Y+1 interrompe il flusso, non è collegato al resto e deve esserlo siccome è un rettangolo. Stesso discorso vale per il blocco dopo 'Un fantasma?' ". Un blocco "non succede niente" non è mai necessario: se non succede niente, la freccia torna al punto da cui il flusso riprende.

Loop voluti e loop involontari. Il flusso deve tornare indietro, ma nel punto giusto. Oltre al caso di Francesca (il "bel bug"), nel flowchart senza nome: "Qui finché il timer non è scaduto il flowchart continua a loopare sul 'Frightened', è quello che volevi?". Se il "no" del timer torna al sotto-processo, per tutta la durata dello spavento non si torna al movimento di Pac-Man. Prima di consegnare, ogni ritorno va seguito a mano: che cosa **non** succede mentre il flusso gira lì?

Un esempio da seguire. Il flowchart di Lorenzo non è stato commentato nel merito (sezione 1.7.6), ma il docente lo indica alla classe: "per tutti gli altri tuoi colleghi, è necessario dire che la relazione tra le frecce e i blocchi è un esempio corretto e va seguito" ([+]). Ogni rettangolo ha una sola uscita, ogni rombo ha le uscite etichettate, ogni sotto-processo ha il suo FC numerato, le frecce non si incrociano e ogni flowchart ha inizio e fine.

1.7.4 La logica del gioco: implementabile o rotta?

Il criterio di fondo è quello di Mattia: "se dovessimo implementare questa logica il gioco sarebbe 'rotto' ". Un flowchart non descrive il gioco a parole; è una sequenza di istruzioni, e va letto **come lo leggerebbe una macchina**, facendo esattamente ciò che è scritto. Due commenti generali chiudono le consegne più lacunose: "Il flowchart ha numerose lacune nella logica presentata. Analizza i miei commenti e leggi i task dei tuoi colleghi, riflettici e se necessario ne riparliamo" (Valerio e Mattia); "Mancano molte regole del gioco, al di là di relazioni anche errate presenti nel flowchart" (Francesco).

Le vite. È il sistema con più errori, e i commenti mostrano tre modi di romperlo.

- *Una vita che vale tutte*: "Per la logica che hai scritto, appena un fantasma prende Pac-Man, gli toglie una vita e poi non ne ha più" (Valerio: "loses one life" porta dritto a "no lives left").
- *Il numero sbagliato, e controllato male*: "Più corretto il check su quante ne rimangono" e "Non sono 3 le vite di base" (Francesco, che chiede "Character loses 3 lives?"); "quando respawni non perdi automaticamente 3 vite né sono 3 le vite di base. Seguendo la logica di questo flowchart invece è così" (Mattia). Il controllo corretto è sulle vite **rimaste** ("vite > 0?"), non su quelle perse: così funziona con qualunque numero iniziale.
- *Una condizione falsa*: "Per come lo hai scritto, assumi come vero che se rimani fermo allora perdi una vita. Il che è falso" (Mattia: "stay still" porta a "lose one life").

E le vite si **guadagnano**: "Come ottiene nuove vite il giocatore?" (Francesco); "Non è legato a questo punto, ma come si ottengono altre vite?" (flowchart senza nome); "Quanti punti fanno una vita?" (Francesca).

Il game over non è la fine. Quattro consegne su sette ricevono la stessa domanda: "Cosa succede al game over?" (Valerio); "Cosa succede dopo il game over?" (Francesca); "Cosa succede al game over? I punti che il giocatore ha ottenuto, a che servono?" (flowchart senza nome); "Non viene considerata la leaderboard o il salvataggio dell'high score nel flowchart" (Antonino). A Francesco il docente dà la sequenza:

> "Game over, poi check high score, poi End."

In un arcade il punteggio serve a una cosa sola: battere il record. Un flowchart che dal game over va dritto alla fine ha dimenticato il motivo per cui si gioca. Lo stesso vale per la vittoria: "Che succede poi? Il giocatore affronta solo un livello e, se vince, è finita?" (Mattia). In *Pac-Man* un labirinto ripulito porta al livello successivo, non alla fine.

I fantasmi. Il loro comportamento è la metà del gioco, e nelle consegne è la metà che manca o non funziona.

- Assente: "Manca il comportamento dei fantasmi" (Mattia).
- Fermi nella casa: "Con la logica scritta i fantasmi non escono mai dallo spawn point" (Francesco); e una regola d'uscita sbagliata: "La meccanica legata al blocco Rilascia il prossimo fantasma è falsa. Pac-Man non segue quella regola" (Antonino).[5] Nel flowchart senza nome, il blocco "uscita fantasma, collisione": "Da spacchettare", *e ancora:* "Uscita fantasma dalla base?". Due sistemi diversi non stanno in un blocco solo.

[5]Nota del curatore. Nel flowchart di Antonino il prossimo fantasma esce se Pac-Man non mangia da 4 secondi, e "Rilascia il primo fantasma" si ripete a ogni giro del ciclo. Secondo *The Pac-Man Dossier* di Jamey Pittman, un timer di inattività di questo tipo esiste nell'originale, ma come meccanismo di riserva: la regola principale sono i **contatori di puntini** di ciascun fantasma (tabella 1.12). Una regola di riserva presentata come l'unica regola descrive un gioco diverso. Quale parte del blocco il docente considerasse falsa resta da chiarire con lui.

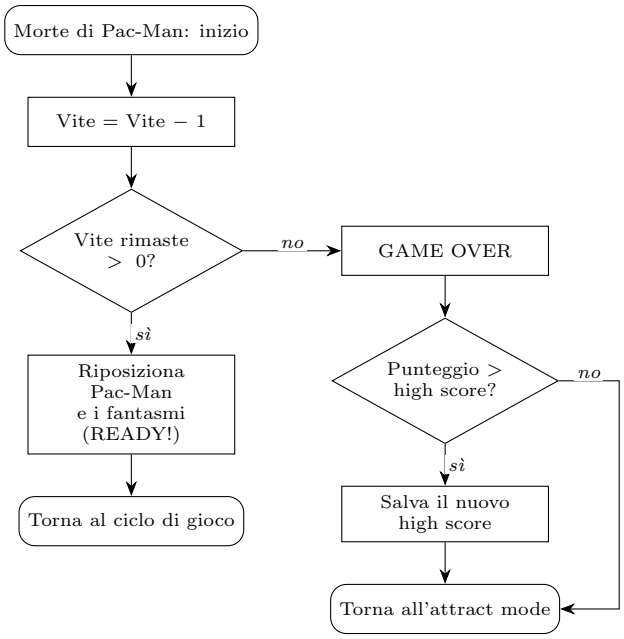


Figure 1.4: Perdita di una vita, game over e high score: il controllo è sulle vite rimaste, il game over passa dal confronto con il record. Proposta del curatore, nella sequenza indicata dal docente ("Game over, poi check high score, poi End"); in un cabinato la "fine" è il ritorno all'attract mode.

- Mangiati: "Cosa succede ai fantasmi che vengono mangiati?" (Antonino); e a Francesco, dove il fantasma mangiato torna alla base e da lì la freccia va a "Character collects a Berry": "Dopo questo risultato il giocatore ottiene una 'Berry' (che va spiegata)? Qualcosa mi sfugge, perché questa relazione non mi torna con le regole del gioco".

- Spaventati: "Dunque cosa fanno i fantasmi spaventati?" (Francesco, il cui blocco "Ghosts are scared" non porta a un comportamento); "Sempre in direzione opposta a Pac-Man" (Francesca, sul movimento casuale).[6]

- Diversi tra loro: "Che però è diversa a seconda del colore" (flowchart senza nome, su "fantasmi riprendono la loro direzione"); e i loro stati vanno controllati: "Da qui non fai un check sullo stato di Pac-Man/fantasma?" (sul blocco "fantasma normale", da cui il flusso va direttamente alla perdita di una vita).

- Spiegati: sulle meccaniche più tecniche del flowchart dei fantasmi di Francesca, "Qui un link alla documentazione o due parole in più le avrei spese per completezza".

Oggetti e numeri. Le domande del docente sono quelle dei task precedenti: **quanti e quali**.

- "Quanti?" (Mattia: "you can eat a ghost for a few seconds");

- "Tutti gli oggetti extra sono inclusi qui?" (Francesco, sulla "Berry"); "frutto bonus (la cui lista serve e manca)" (flowchart senza nome);

- "'x' e 'y' fanno riferimento a che numeri? Anche indicativi" (flowchart senza nome, sui timer di scatter e chase);

[6]Nota del curatore. Nel codice originale, all'inizio dello spavento i fantasmi invertono la direzione, poi scelgono le svolte agli incroci in modo pseudo-casuale (*The Pac-Man Dossier*); il comportamento percepito dal giocatore è la fuga. Un flowchart completo riporta entrambe le cose: l'inversione e la regola con cui si sceglie la svolta.

- "Anche questo per me lettore è molto oscuro" (Francesco); "Il '2' dopo lo Start non è chiaro" (Antonino).

Un flowchart può rimandare a una tabella per i valori (è l'uso del blocco documento, sezione 6.11), ma i valori devono esistere da qualche parte. A titolo di riferimento, la tabella 1.12 raccoglie quelli richiesti nei commenti.

Table 1.12: Approfondimento del curatore: i numeri di *Pac-Man* (arcade, 1980) chiesti nei commenti. Fonte: Jamey Pittman, *The Pac-Man Dossier*; dove il gioco permette di cambiarli tramite i DIP switch del cabinato, sono indicate le opzioni.

Che cosa	**Valore**
Vite iniziali	impostazione dell'operatore: 1, 2, 3 o 5. Per questo il flowchart le legge da una variabile e controlla quante ne **rimangono**
Vita extra	una sola, a 10.000 punti (opzioni: 10.000, 15.000, 20.000 o nessuna)
Puntini e power pellet	240 puntini da 10 punti, 4 power pellet da 50 punti
Fantasmi mangiati	200, 400, 800, 1600 punti, raddoppiando per ogni fantasma mangiato con la stessa power pellet
Frutta	compare due volte per livello, dopo 70 e 170 puntini mangiati, per circa 9–10 secondi; ciliegie 100 (livello 1), fragola 300 (2), pesca 500 (3–4), mela 700 (5–6), uva 1000 (7–8), Galaxian 2000 (9–10), campana 3000 (11–12), chiave 5000 (dal 13)
Spavento	6 secondi al livello 1, poi in generale meno, fino a zero (livello 17 e dal 19 in poi); i fantasmi lampeggiano prima della fine
Scatter e chase	livello 1: scatter 7 s, chase 20 s, scatter 7, chase 20, scatter 5, chase 20, scatter 5, poi chase per sempre; il timer si ferma durante lo spavento; a ogni cambio di modalità i fantasmi invertono la direzione
Uscita dalla casa	Blinky parte fuori; al livello 1 Pinky esce subito, Inky dopo 30 puntini, Clyde dopo 60 (contatori personali); se Pac-Man non mangia per 4 secondi (3 dal livello 5) esce il prossimo fantasma; dopo una vita persa vale un contatore globale (7, 17, 32 puntini)
Cruise Elroy	Blinky accelera quando restano pochi puntini (livello 1: 20, poi 10)
Livelli	dopo il labirinto ripulito si passa al successivo; il livello 256 è il *kill screen*, un bug che rende il gioco non completabile

1.7.5 Leggibilità e forma

Le regole formali sono sospese, la leggibilità no. A Francesco:

> "Lavora sulla formattazione: testi di misura diversa, uso esclusivo di bianco e nero, organizzazione generale dei blocchi. Puoi ripensarci."

Tre indicazioni: il testo dei blocchi va in **una sola misura** (nel suo flowchart convivono almeno tre corpi diversi); il bianco e nero esclusivo non è vietato, ma un uso coerente del colore (per esempio un colore per tipo di blocco, come nel flowchart di Lorenzo) aiuta la lettura; i blocchi vanno **organizzati**, allineati per livello e nel verso di lettura (sezione 6.10). Ad Antonino: "In generale, ci sono soluzioni che renderebbero il flowchart più leggibile. Pensaci".

Il testo resta dentro il blocco. Ad Antonino:

> "La domanda sul controllo dei pallini nel livello ha il testo che cade fuori dal blocco, la prossima volta che vedo una svista simile non leggo." **[!]**

È l'unico avviso di non lettura del task, e riguarda una svista che si vede al primo sguardo. Lo stesso tema, più lieve, a Francesca: "Attenzione ai margini", "E non vale solo qui": il testo che tocca i bordi del blocco ("GHOSTS RECOLOR AND BECOME INVULNERABLE AGAIN", "EATEN CONSECUTIVELY?") ha bisogno di respiro, come il testo di una pagina.

Il nome del protagonista. "Pacman" invece di *Pac-Man*: "Assumo che hai utilizzato una licenza artistica nello scrivere il nome del protagonista. Prima e ultima volta" (Antonino, **[!]**); "Pac-Man, assumo la borderline libertà artistica" (flowchart senza nome). È la regola del secondo task sui nomi propri ("pokémon" scritto sempre allo stesso modo, sezione 1.5.5): il nome di un gioco o di un personaggio si scrive come lo scrive chi lo ha fatto.

Grammatica. Anche in un'immagine la grammatica conta: Mattia riceve tre commenti "Grammatica", su "you muve", su "lose tre life" e su un terzo blocco non indicato (nel suo flowchart compare anche "respown"). Il flowchart è un documento tecnico, e un refuso in un blocco è un refuso nel documento.

Titoli e blocchi di setup. Al flowchart senza nome: "Un titolo più esplicito per il documento avrebbe aiutato, dato che è il primo in ordine di lettura nel Drive" (il file *Flowchat_pacman_Frightened* viene prima, in ordine alfabetico, del flowchart principale). E sul blocco iniziale che elenca tutto il setup ("creazione mappa, score 0, hp 3, 240 pacdots / 4 power pellet"): "Puoi anche pensare di separarli in modo da non avere il 'bloccone' con la lista, ma (al limite del) de gustibus". Infine: "Questo da riformulare, scritto così non è chiaro" ("salva tempo", nel flowchart dei timer).

I documenti di supporto seguono le regole. Il task chiedeva solo il flowchart, ma due consegne hanno aggiunto un documento, e in entrambi i casi il docente ricorda che un documento resta un documento:

- a Lorenzo, per una nota in formato `.txt`: "Perché questo documento integrativo non si presenta secondo le regole stabilite?";
- al flowchart senza nome, per il documento con le immagini: "In realtà sfrutterò le immagini caricate, perché mi permettono di commentare direttamente un blocco specifico o, in generale, più nel dettagli i singoli punti", *e ancora:* "Detto questo, se carichi un documento di supporto anche questo deve adottare i margini e le regole viste a lezione. Parliamo di formattazione base: titolo e singole descrizione per le immagini inserite, in questo caso".

Le regole formali erano sospese per il flowchart, non per tutto ciò che si consegna.

1.7.6 Completare il task: profondità, tempo e compromessi

Il caso. Lorenzo consegna 9 dei 15 flowchart previsti, fatti con il pacchetto TikZ di LaTeX, e una nota in `.txt` in cui dichiara quelli mancanti (FC5 movimento di Pac-Man, FC6 raccolta di puntini, power pellet e frutta, FC7 attivazione dello spavento, FC8 timer, FC11 uscita dalla casa dei fantasmi, FC12 collisioni) e ne attribuisce la causa allo strumento, "più complesso e lento" del previsto. Il docente non commenta i flowchart (**[!]**):

> "Per il discorso fatto nel documento integrativo, mi riservo di non dare commenti ad un task mancante di parti fondamentali."

Sullo strumento, apprezza la sperimentazione: "Non lo conoscevo, indubbiamente sperimentare con tool nuovi è sempre qualcosa di benvenuto. C'è però un 'ma', che commento sopra". E il "ma" è il commento più lungo del task:

> "Per quanto i flowchart presentati (al netto della quantità, separata) si propongano come analisi microscopica del loop di Pac-Man, il che è positivo, non aver terminato il task è un grosso problema. Qui mancano cose come il movimento, gli oggetti di gioco mangiabili, stati di Pac-Man e fantasmi (legati alla Power Pellet). Fare design significa adottare compromessi, idealmente il migliore possibile, ma sempre di compromessi parliamo. Prima di imbarcarti nella risoluzione di un task, devi valutare la profondità dell'analisi che ti puoi permettere di adottare (considerando il tempo a disposizione, quanto nel dettaglio vuoi andare e quanti dati inserire, e così via). Gli imprevisti capitano, ma trovare una soluzione in modo che il task sia quanto più completo possibile e quanto più coerente nella profondità delle sue singole parti (senza rimanere superficiale e, sia chiaro, non è questo il caso) fa parte del tuo lavoro. Quando un Lead ti assegnerà un compito o ti ritroverai a svolgere un task per una produzione o per un test, quel lavoro ci si aspetta che venga completato perché altrimenti ferma la produzione (miglior caso possibile in questo contesto ipotetico da evitare). Questo ti distinguerà come professionista."

Le lezioni del commento. Il commento vale per tutti, e si può scomporre in cinque regole.

1. **Completo prima che profondo.** Un flowchart parziale ma microscopico vale meno di uno completo e meno dettagliato: le parti mancanti (movimento, oggetti, stati legati alla power pellet) sono proprio il cuore del gioco. Il valore della profondità si misura solo su un lavoro finito.

2. **Il livello di dettaglio si decide prima.** "Prima di imbarcarti": tempo a disposizione, dettaglio desiderato, quantità di dati. È il compromesso di design della lezione su problema e soluzione (sezione 4.5, tabella 4.2) applicato al proprio lavoro: le risorse sono il tempo, il know-how e lo strumento.

3. **La profondità deve essere coerente.** Il task deve essere "quanto più coerente nella profondità delle sue singole parti": nove flowchart microscopici e sei assenti sono l'estremo opposto. Meglio quindici flowchart tutti allo stesso livello, anche se meno profondo.

4. **L'imprevisto si gestisce, non si dichiara.** Uno strumento più lento del previsto è un imprevisto normale; la risposta professionale è cambiare piano in tempo (ridurre il dettaglio, passare a uno strumento più rapido come draw.io per le parti restanti), non consegnare a metà spiegando perché.

5. **In produzione, un task incompleto ferma gli altri.** Un flowchart serve a qualcuno: al programmatore che lo implementa, al designer che lo integra. Se manca, il lavoro di chi aspetta si ferma. Ed ecco la conseguenza concreta: un task incompleto rischia di non ricevere nemmeno il feedback, cioè la cosa per cui lo si consegna.

Un metodo per non arrivarci (proposta del curatore). Il docente lo aveva suggerito a lezione (sezione 6.14): non partire dal flowchart perfetto. Tradotto in un piano di lavoro:

1. fare **prima il flowchart principale**, completo, con tutti i sistemi come sotto-processi: è il *must have* (sezione 4.10);

2. scrivere **tutti** i sotto-processi a un primo livello di dettaglio, anche grezzo;

3. solo allora scendere in profondità, sistema per sistema, sapendo che a ogni momento si ha in mano un task consegnabile;

4. se si usa uno strumento nuovo, provarlo su un flowchart piccolo e misurare il tempo, prima di scommetterci tutta la consegna.

1.7.7 Che cosa è stato apprezzato

- la **relazione tra frecce e blocchi** del flowchart di Lorenzo: "un esempio corretto e va seguito", detto esplicitamente per tutta la classe;

- l'ambizione dell'analisi "microscopica del loop di Pac-Man" ("il che è positivo") e la **sperimentazione con strumenti nuovi** ("sempre qualcosa di benvenuto"), al netto del task incompleto (Lorenzo);
- il **sotto-processo dei fantasmi** spiegato in un flowchart a sé: "averlo come azione con sotto processo è corretto"; una soluzione "chiara e funziona"; una soluzione "eccezionale, comprensibile" anche se "fatta in casa" (Francesca).

1.7.8 Dai task precedenti al task 4: confronto e pattern ricorrenti

La stessa domanda, in quattro forme. Il filo dei confronti precedenti (sezioni 1.5.9 e 1.6.8) continua: "quanti e quali?" (T1), "quanti sono?" (T2), "quante sono ad oggi?" (T3), "Quanti?", "Quanti punti fanno una vita?", "'x' e 'y' fanno riferimento a che numeri?" (T4). E "cosa succede al game over?" era già una domanda del primo task (sezione 1.4.7). Francesco, che nel primo task aveva analizzato proprio *Pac-Man*, aveva ricevuto sulle "3 vite iniziali" il commento "Manca in ogni caso l'outcome del game over: dove viene riportato il giocatore?", e sul cambio di stato della power pellet "Pac-Man cambia stato per N secondi [...] Va spiegato, nel dettaglio" (sezione 1.4.11). Nel quarto task ritornano le stesse lacune: vite, game over, fantasmi spaventati.

Che cosa cambia. Il flowchart rende visibile ciò che un testo nasconde. Nel primo task una meccanica descritta ad alto livello era "incompleta"; qui una regola mancante diventa un blocco nel nulla, un loop infinito, un fantasma che non esce mai. Il commento del docente cambia di conseguenza: non più "manca il dettaglio", ma "il gioco sarebbe rotto".

Che cosa resta. Il lettore che non sa ("come lettore, io da questo blocco non ho idea di dove andare"), i nomi propri scritti male, gli errori grammaticali, il documento che non segue le regole: sono gli stessi temi dei primi task, in un formato nuovo.

Che cosa è nuovo. La completezza come requisito. Nei task precedenti il problema era la profondità di ciò che c'era; qui, per la prima volta, un task viene lasciato senza commenti di merito perché **non finito**. La regola si aggiunge al registro degli avvertimenti (tabella 1.3).

1.7.9 Le schede delle singole consegne

Seguono, consegna per consegna, **tutti** i 79 commenti del docente. A differenza dei task precedenti, quasi tutti i commenti sono ancorati a una **zona dell'immagine** e non a un testo: quando il blocco commentato si ricava con sicurezza dal flowchart, è indicato con [*presumibilmente* "*blocco*"]: (ricostruzione del curatore); altrimenti il commento è riportato senza ancora. I commenti delle immagini sono nell'ordine in cui il docente li ha elencati; quelli dei PDF e dei documenti, con l'ancora esatta.

Valerio Rossaro

Un flowchart su sfondo scuro, con cinque sotto-processi non spiegati, rettangoli con più uscite e una perdita di vita che porta subito alla fine.

- Quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme
- Ogni azione con sotto processo va spiegata con un flowchart associato. Ogni blocco simile che utilizzi, ne abbiamo parlato a lezione
- Sovrapposizione fraintendibile di frecce nel flusso, è un problema
- Da un rettangolo non può uscire più di una freccia, non è un rombo
- Tutti sistemi da spiegare
- Il flowchart ha numerose lacune nella logica presentata. Analizza i miei commenti e leggi i task dei tuoi colleghi, riflettici e se necessario ne riparliamo
- [*presumibilmente* "*PAC-MAN LOSES ONE LIFE / NO LIVES LEFT*"]: Per la logica che hai scritto, appena un fantasma prende Pac-Man, gli toglie una vita e poi non ne ha più
- [*presumibilmente* "*END*"]: Cosa succede al game over?

Francesco Muratore

Il flowchart più commentato (17 commenti): rettangoli e rombi con uscite ambigue, sotto-processi e documenti non spiegati, frecce sovrapposte, formattazione disomogenea e molte regole del gioco mancanti o errate (vite, game over, fantasmi, "Berry").

- Come lettore, io da questo blocco non ho idea di dove andare con la freccia in uscita
- Quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme
- Va spiegato, dov'è il flowchart che mostra questa azione con sotto processo?
- Questo è un problema, le frecce del flusso si sovrappongono creando potenziali fraintendimenti *E ancora:* Non è presente solo qui
- [*presumibilmente* "*Ghosts Hit Character?*"]: Quale di queste uscite è legata al "NO"?
- Lavora sulla formattazione: testi di misura diversa, uso esclusivo di bianco e nero, organizzazione generale dei blocchi. Puoi ripensarci
- [*presumibilmente* "*Character collects a Berry*"]: Tutti gli oggetti extra sono inclusi qui?
- [*presumibilmente* "*Ghosts are scared*"]: Dunque cosa fanno i fantasmi spaventati?
- Come ottiene nuove vite il giocatore?
- Mancano molte regole del gioco, al di là di relazioni anche errate presenti nel flowchart
- [*presumibilmente* "*Character loses 3 lives?*"]: Più corretto il check su quante ne rimangono
- [*presumibilmente* "*Character loses 3 lives?*"]: Non sono 3 le vite di base
- Anche questo per me lettore è molto oscuro
- [*presumibilmente* "*End*"]: Game over, poi check high score, poi End
- [*presumibilmente* "*Ghost returns to spawnpoint*"]: Con la logica scritta i fantasmi non escono mai dallo spawn point
- [*presumibilmente* "*Ghost returns to spawnpoint* → *Character collects a Berry*"]: Dopo questo risultato il giocatore ottiene una "Berry" (che va spiegata)? Qualcosa mi sfugge, perché questa relazione non mi torna con le regole del gioco

Mattia Maestri

Un flowchart su sfondo scuro: un rombo con due "yes", rettangoli con più uscite, blocchi "nothing happens" scollegati, il comportamento dei fantasmi assente, vite e fine partita errate, refusi.

- [*presumibilmente* "*move*"]: Sono due uscite con lo stesso valore "yes", quale è vera?
- [*presumibilmente* "*you muve*"]: Grammatica, inoltre quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme
- [*presumibilmente* "*you can eat a ghost for a few seconds*"]: Quanti?
- Il flowchart ha numerose lacune nella logica presentata. Analizza i miei commenti e leggi i task dei tuoi colleghi, riflettici e se necessario ne riparliamo
- [*presumibilmente* "*ghost still move*"]: Manca il comportamento dei fantasmi
- [*presumibilmente* "*stay still*"]: Per come lo hai scritto, assumi come vero che se rimani fermo allora perdi una vita. Il che è falso
- [*presumibilmente* "*you win*"]: Che succede poi? Il giocatore affronta solo un livello e, se vince, è finita?
- Grammatica
- [*presumibilmente* "*nothing happens*"]: Questi blocchi non sono collegati con il resto del flusso, e vale per altri "nothing happens" oppure "game continues". Questo è un problema, se dovessimo implementare questa logica il gioco sarebbe "rotto" o con severi bug
- [*presumibilmente* "*lose tre life*"]: Grammatica, inoltre è falso: quando respawni non perdi automaticamente 3 vite né sono 3 le vite di base. Seguendo la logica di questo flowchart invece è così

Francesca Attadia — flowchart principale

Flowchart ordinato, con i punteggi di puntini, energizer e fantasmi e un sotto-processo per i fantasmi spiegato a parte. I commenti chiedono uscite in sequenza, correggono un loop involontario e un'uscita invertita, e riguardano il game over, la vita extra e i margini del testo.

- Quali uscite deve prendere il flowchart? L'assunto di base è che non possono essere vere entrambe contemporaneamente
- [+] Soluzione eccezionale, comprensibile, ma resta una eccezione "fatta in casa"
- [*presumibilmente* "*GHOSTS MOVE*"]: [+] Considerando i comportamenti diversi dei singoli fantasmi, averlo come azione con sotto processo è corretto
- [*presumibilmente* "*PACMAN MOVES*"]: Anche qui, una uscita per volta, in sequenza *E ancora:* Infatti se consideriamo solo l'uscita a destra, il flowchart continua a loopare finché non vieni colpito da un fantasma, il che è un bel bug
- [*presumibilmente* "*LIVES LEFT?*"]: Il contrario
- [*presumibilmente* "*GAME OVER*"]: Cosa succede dopo il game over?
- Anche qui, quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme
- [+] Questa soluzione è chiara e funziona
- Quanti punti fanno una vita?
- Attenzione ai margini *E ancora:* E non vale solo qui

Francesca Attadia — flowchart dei fantasmi

Il sotto-processo del flowchart principale: spavento, ritorno alla gabbia e comportamento dei quattro fantasmi (Blinky con le soglie di Elroy, Pinky, Inky, Clyde).

- [*presumibilmente* "*GHOSTS MOVE RANDOMLY*"]: Sempre in direzione opposta a Pac-Man
- Meccaniche che citi anche nel flowchart principale
- [*presumibilmente* "*GHOSTS MOVE*"]: Rombo con nomi dei fantasmi sulle quattro uscite
- Qui un link alla documentazione o due parole in più le avrei spese per completezza

Antonino Etiopia

Un documento con quattro flowchart (principale, sistema di punti, movimento di Pac-Man, movimento dei fantasmi), in italiano. Il docente lascia la lista dei problemi del flowchart principale in un unico commento; due avvisi, sul testo fuori dal blocco e sul nome del protagonista.

- "*Main flowchart*": Lascio qui la lista di commenti:
  - Il "2" dopo lo Start non è chiaro
  - La freccia a destra del Fine livello si interseca con un'altra, e crea una fraintendibilità che non può essere presente
  - Il sistema di punti, in gerarchia, è prima del movimento. Ma nello spiegare l'azione con sotto processo citi il mangiare, che è legato al muoversi di Pac-Man
  - [**!**] La domanda sul controllo dei pallini nel livello ha il testo che cade fuori dal blocco, la prossima volta che vedo una svista simile non leggo
  - La meccanica legata al blocco Rilascia il prossimo fantasma è falsa. Pac-Man non segue quella regola
  - Non viene considerata la leaderboard o il salvataggio dell'high score nel flowchart
- "*Sistema di punti*": Y = Y+1 interrompe il flusso, non è collegato al resto e deve esserlo siccome è un rettangolo. Stesso discorso vale per il blocco dopo Un fantasma?
- "*Movimento dei fantasmi*": Cosa succede ai fantasmi che vengono mangiati?
- "*Movimento dei fantasmi*": In generale, ci sono soluzioni che renderebbero il flowchart più leggibile. Pensaci
- "*Movimento dei fantasmi*": [**!**] Assumo che hai utilizzato una licenza artistica nello scrivere il nome del protagonista. Prima e ultima volta

Lorenzo Vergata

Nove flowchart su quindici (FC1–FC4, FC9, FC10, FC13–FC15), in inglese, fatti con TikZ e consegnati in PDF, più una nota in `.txt` con l'elenco dei sei mancanti. Il docente non commenta il merito dei flowchart; indica però la relazione tra frecce e blocchi come esempio per la classe. I commenti alla nota sono riportati con l'ancora più probabile.

- "*FC1: Complete game*" : [!] Per il discorso fatto nel documento integrativo, mi riservo di non dare commenti ad un task mancante di parti fondamentali. Detto questo, per tutti gli altri tuoi colleghi, è necessario dire che la relazione tra le frecce e i blocchi è un esempio corretto e va seguito
- [*presumibilmente* "*Small notes about the flowcharts*"] : Perché questo documento integrativo non si presenta secondo le regole stabilite?
- [*presumibilmente* "*I tried to use the Tikz package*"] : [+] Non lo conoscevo, indubbiamente sperimentare con tool nuovi è sempre qualcosa di benvenuto. C'è però un "ma", che commento sopra
- [*presumibilmente* "*Unfortunately, I didn't conclude some of them*"] : Per quanto i flowchart presentati (al netto della quantità, separata) si propongano come analisi microscopica del loop di Pac-Man, il che è positivo, non aver terminato il task è un grosso problema. Qui mancano cose come il movimento, gli oggetti di gioco mangiabili, stati di Pac-Man e fantasmi (legati alla Power Pellet). Fare design significa adottare compromessi, idealmente il migliore possibile, ma sempre di compromessi parliamo. Prima di imbarcarti nella risoluzione di un task, devi valutare la profondità dell'analisi che ti puoi permettere di adottare (considerando il tempo a disposizione, quanto nel dettaglio vuoi andare e quanti dati inserire, e così via). Gli imprevisti capitano, ma trovare una soluzione in modo che il task sia quanto più completo possibile e quanto più coerente nella profondità delle sue singole parti (senza rimanere superficiale e, sia chiaro, non è questo il caso) fa parte del tuo lavoro. Quando un Lead ti assegnerà un compito o ti ritroverai a svolgere un task per una produzione o per un test, quel lavoro ci si aspetta che venga completato perché altrimenti ferma la produzione (miglior caso possibile in questo contesto ipotetico da evitare). Questo ti distinguerà come professionista

Autore non indicato — *Flowchat_pacman*

Tre flowchart in PDF (principale, *Frightened*, timer di scatter e chase) e un documento di supporto con le stesse immagini, senza nome dell'autore nei file. Il docente commenta i PDF, con l'ancora esatta; i commenti al documento di supporto non hanno ancora.

- *Documento di supporto.*
- *(riga vuota o spazio)* : In realtà sfrutterò le immagini caricate, perché mi permettono di commentare direttamente un blocco specifico o, in generale, più nel dettagli i singoli punti
- *(riga vuota o spazio)* : Detto questo, se carichi un documento di supporto anche questo deve adottare i margini e le regole viste a lezione. Parliamo di formattazione base: titolo e singole descrizione per le immagini inserite, in questo caso
- *Flowchart principale.*
- "*uscita fantasma, collisione*" : Da spacchettare *E ancora:* Uscita fantasma dalla base?
- "*timer power pellet scaduto?*" : Qui finché il timer non è scaduto il flowchart continua a loopare sul "Frightened", è quello che volevi?
- "*fantasma normale*" : Da qui non fai un check sullo stato di Pac-Man/fantasma?
- "*hp -1*" : Non è legato a questo punto, ma come si ottengono altre vite?
- "*score 0, hp 3 / 240 pacdots / 4 power pellet*" : Puoi anche pensare di separarli in modo da non avere il "bloccone" con la lista, ma (al limite del) de gustibus
- "*pacman*" : Pac-Man, assumo la borderline libertà artistica
- "*Frightened*" : Qui sotto le frecce si ingarbugliano problematicamente. Ripensaci
- "*+50 score*" : Quali uscite deve prendere il flowchart? Non stai usando un rombo, ma un rettangolo. Riguarda la lezione oppure parliamone insieme
- "*sì*" : (uscita del rombo "hp = 0?" verso il game over) Cosa succede al game over? I punti che il giocatore ha ottenuto, a che servono?
- "*sì*" : (uscite del rombo "tocca elemento labirinto?") No, come lo hai scritto è un problema. Invece di avere molteplici e fraintendibili "sì", perché non sostituirli direttamente con le categorie: power pellet, frutto bonus (la cui lista serve e manca), ecc.

- *Flowchart Frightened.*
- "*START Frightened*" : Un titolo più esplicito per il documento avrebbe aiutato, dato che è il primo in ordine di lettura nel Drive
- "*fantasmi riprendono la loro direzione*" : Che però è diversa a seconda del colore
- "*pacman si muove*" : Questo è presente anche nel flowchart principale
- "*pacman mangia un fantasma?*" : Problema di frecce sovrapposte a sinistra, riguarda tutti i flowchart che hai consegnato
- *Flowchart dei timer.*
- "*timer > = y?*" : "x" e "y" fanno riferimento a che numeri? Anche indicativi
- "*pacman mangia un power pellet?*" : Qui le frecce a sinistra si intersecano in una maniera fraintendibile, è un problema. Ripensa alla messa a terra
- "*salva tempo*" : Questo da riformulare, scritto così non è chiaro

Daniele Maracino — *Donkey Kong*

Un flowchart di *Donkey Kong* (arcade) a colori, con blocchi rossi per le azioni e gialli per le decisioni. Al momento della stesura di questa sezione non sono pervenuti commenti del docente su questa consegna.

1.7.10 Punti chiave del task T4

- Un **rettangolo** ha **una sola uscita**. Se le strade sono più d'una, è un rombo; se le azioni sono più d'una, vanno in sequenza.
- Ogni uscita di un **rombo** è un esito diverso, etichettato: niente due "sì", niente uscite senza nome; con più esiti, le categorie (o i nomi dei fantasmi) sulle uscite.
- Ogni **sotto-processo** ha il suo flowchart, e l'ordine dei sotto-processi segue l'ordine causale dei sistemi.
- Le frecce **non si sovrappongono** né si incrociano in modo fraintendibile; nessun blocco resta **scollegato**; ogni loop si verifica a mano.
- Il flowchart si legge come una macchina: se, implementato, il gioco sarebbe **rotto**, il flowchart è sbagliato.
- Vite: si controllano quelle **rimaste**, si dice come se ne guadagnano. Game over, poi **high score**, poi fine. Vinto un livello, si passa al successivo.
- Fantasmi: uscita dalla casa, inseguimento, spavento, ritorno dopo essere stati mangiati, differenze tra i quattro.
- **Quanti e quali**: durate, punteggi, liste degli oggetti, valori di "x" e "y".
- Testo della stessa misura e **dentro il blocco**; il nome *Pac-Man* scritto giusto; anche i documenti di supporto seguono le regole.
- Un task si **completa**: la profondità si sceglie prima, in base al tempo, ed è coerente in tutte le parti. Un task a metà ferma la produzione e può restare senza feedback.